Graph Theory

Release 10.10

The Sage Development Team

Oct 28, 2026

CONTENTS

1 Graph objects and methods 1

2 Constructors and databases 963

3 Low-level implementation 1679

4 Hypergraphs 1821

5 Libraries of algorithms 1857

6 Indices and Tables 2397

Bibliography 2399

Python Module Index 2401

Index 2403

CHAPTER

ONE

GRAPH OBJECTS AND METHODS

1.1 Generic graphs (common to directed/undirected)

This module implements the base class for graphs and digraphs, and methods that can be applied on both. Here is what it can do:

Basic Graph operations:

networkx_graph()	Return a new `NetworkX` graph from the Sage graph
igraph_graph()	Return an `igraph` graph from the Sage graph
to_dictionary()	Create a dictionary encoding the graph.
copy()	Return a copy of the graph.
export_to_file()	Export the graph to a file.
adjacency_matrix()	Return the adjacency matrix of the (di)graph.
incidence_matrix()	Return an incidence matrix of the (di)graph
distance_matrix()	Return the distance matrix of the (strongly) connected (di)graph
weighted_adjacency_matrix()	Return the weighted adjacency matrix of the graph
kirchhoff_matrix()	Return the Kirchhoff matrix (a.k.a. the Laplacian) of the graph.
has_loops()	Return whether there are loops in the (di)graph
allows_loops()	Return whether loops are permitted in the (di)graph
allow_loops()	Change whether loops are permitted in the (di)graph
loops()	Return a list of all loops in the (di)graph
loop_edges()	Return a list of all loops in the (di)graph
number_of_loops()	Return the number of edges that are loops
loop_vertices()	Return a list of vertices with loops
remove_loops()	Remove loops on vertices in `vertices`.
has_multiple_edges()	Return whether there are multiple edges in the (di)graph.
allows_multiple_edges()	Return whether multiple edges are permitted in the (di)graph.
allow_multiple_edges()	Change whether multiple edges are permitted in the (di)graph.
multiple_edges()	Return any multiple edges in the (di)graph.
name()	Return or set the graph's name.
is_immutable()	Return whether the graph is immutable.
weighted()	Whether the (di)graph is to be considered as a weighted (di)graph.
antisymmetric()	Test whether the graph is antisymmetric
density()	Return the density
order()	Return the number of vertices.
size()	Return the number of edges.
add_vertex()	Create an isolated vertex.
add_vertices()	Add vertices to the (di)graph from an iterable container of vertices
delete_vertex()	Delete vertex, removing all incident edges.

continues on next page

Table 1 – continued from previous page

delete_vertices()	Delete vertices from the (di)graph taken from an iterable container of vertices.
has_vertex()	Check if `vertex` is one of the vertices of this graph.
random_vertex()	Return a random vertex of `self`.
random_vertex_iterator()	Return an iterator over random vertices of `self`.
random_edge()	Return a random edge of `self`.
random_edge_iterator()	Return an iterator over random edges of `self`.
vertex_boundary()	Return a list of all vertices in the external boundary of `vertices1`, intersected with `vertices2`.
set_vertices()	Associate arbitrary objects with each vertex
set_vertex()	Associate an arbitrary object with a vertex.
get_vertex()	Retrieve the object associated with a given vertex.
get_vertices()	Return a dictionary of the objects associated to each vertex.
vertex_iterator()	Return an iterator over the given vertices.
neighbor_iterator()	Return an iterator over neighbors of `vertex`.
vertices()	Return a list of the vertices.
neighbors()	Return a list of neighbors (in and out if directed) of `vertex`.
merge_vertices()	Merge vertices.
add_edge()	Add an edge from `u` to `v`.
add_edges()	Add edges from an iterable container.
subdivide_edge()	Subdivide an edge k times.
subdivide_edges()	Subdivide k times edges from an iterable container.
delete_edge()	Delete the edge from `u` to `v`
delete_edges()	Delete edges from an iterable container.
contract_edge()	Contract an edge from `u` to `v`.
contract_edges()	Contract edges from an iterable container.
delete_multiedge()	Delete all edges from `u` to `v`.
set_edge_label()	Set the edge label of a given edge.
has_edge()	Check whether `(u, v)` is an edge of the (di)graph.
edges()	Return a *EdgesView* of edges.
edge_boundary()	Return a list of edges `(u,v,l)` with `u` in `vertices1`
edge_iterator()	Return an iterator over edges.
edges_incident()	Return incident edges to some vertices.
edge_label()	Return the label of an edge.
edge_labels()	Return a list of the labels of all edges in `self`.
remove_multiple_edges()	Remove all multiple edges, retaining one edge for each.
clear()	Empty the graph of vertices and edges and removes name, associated objects, and position information.
degree()	Return the degree (in + out for digraphs) of a vertex or of vertices.
average_degree()	Return the average degree of the graph.
degree_histogram()	Return a list, whose i-th entry is the frequency of degree i.
degree_iterator()	Return an iterator over the degrees of the (di)graph.
degree_sequence()	Return the degree sequence of this (di)graph.
random_subgraph()	Return a random subgraph containing each vertex with probability `p`.
add_clique()	Add a clique to the graph with the given vertices.
add_cycle()	Add a cycle to the graph with the given vertices.
add_path()	Add a path to the graph with the given vertices.
complement()	Return the complement of the (di)graph.
line_graph()	Return the line graph of the (di)graph.
to_simple()	Return a simple version of itself (i.e., undirected and loops and multiple edges are removed).
disjoint_union()	Return the disjoint union of `self` and `other`.

continues on next page

Table 1 – continued from previous page

union()	Return the union of `self` and `other`.
relabel()	Relabel the vertices of `self`
degree_to_cell()	Return the number of edges from vertex to an edge in cell.
subgraph()	Return the subgraph containing the given vertices and edges.
is_subgraph()	Check whether `self` is a subgraph of `other`.

Graph products:

cartesian_product()	Return the Cartesian product of `self` and `other`.
tensor_product()	Return the tensor product, also called the categorical product, of `self` and `other`.
lexicographic_product()	Return the lexicographic product of `self` and `other`.
strong_product()	Return the strong product of `self` and `other`.
disjunctive_product()	Return the disjunctive product of `self` and `other`.

Paths and cycles:

eulerian_circuit()	Return a list of edges forming an Eulerian circuit if one exists.
minimum_cycle_basis()	Return a minimum weight cycle basis of the graph.
cycle_basis()	Return a list of cycles which form a basis of the cycle space of `self`.
all_paths()	Return a list of all paths (also lists) between a pair of vertices in the (di)graph.
all_paths_iterator()	Return an iterator over the paths of `self`.
all_simple_paths()	Return a list of all the simple paths of `self` starting with one of the given vertices.
triangles_count()	Return the number of triangles in the (di)graph.
shortest_simple_paths()	Return an iterator over the simple paths between a pair of vertices.
all_cycles_iterator()	Return an iterator over all cycles in the (di)graph.
all_simple_cycles()	Return a list of all simple cycles in the (di)graph.

Linear algebra:

spectrum()	Return a list of the eigenvalues of the adjacency matrix.
eigenvectors()	Return the *right* eigenvectors of the adjacency matrix of the graph.
eigenspaces()	Return the *right* eigenspaces of the adjacency matrix of the graph.

Some metrics:

cluster_triangles()	Return the number of triangles for the set nbunch of vertices as a dictionary keyed by vertex.
clustering_average()	Return the average clustering coefficient.
clustering_coeff()	Return the clustering coefficient for each vertex in nbunch
cluster_transitivity()	Return the transitivity (fraction of transitive triangles) of the graph.
szeged_index()	Return the Szeged index of the graph.
katz_centrality()	Return the katz centrality of the vertex u of the graph.
katz_matrix()	Return the katz matrix of the graph.
pagerank()	Return the PageRank of the vertices of `self`.

Automorphism group:

coarsest_equitable_refinement()	Return the coarsest partition which is finer than the input partition, and equitable with respect to `self`.
automorphism_group()	Return the largest subgroup of the automorphism group of the (di)graph whose orbit partition is finer than the partition given.
is_vertex_transitive()	Return whether the automorphism group of `self` is transitive within the partition provided
is_isomorphic()	Test for isomorphism between `self` and `other`.
canonical_label()	Return the canonical graph.
is_cayley()	Check whether the graph is a Cayley graph.
is_homeomorphic()	Check whether `G` and `H` are homeomorphic.
reduced_homeomorphic_graph()	Return the smallest graph homeomorphic to G.
has_homomorphism_to()	Check whether there is a homomorphism between two graphs.

Graph properties:

is_eulerian()	Return `True` if the graph has a (closed) tour that visits each edge exactly once.
is_planar()	Check whether the graph is planar.
is_circular_planar()	Check whether the graph is circular planar (outerplanar)
is_regular()	Return `True` if this graph is (k-)regular.
is_chordal()	Check whether the given graph is chordal.
is_bipartite()	Test whether the given graph is bipartite.
is_circulant()	Check whether the graph is a circulant graph.
is_interval()	Check whether the graph is an interval graph.
is_gallai_tree()	Return whether the current graph is a Gallai tree.
is_clique()	Check whether a set of vertices is a clique
is_cycle()	Check whether `self` is a (directed) cycle graph.
is_geodetic()	Check whether the input (di)graph is geodetic.
is_independent_set()	Check whether `vertices` is an independent set of `self`
is_transitively_reduced()	Test whether the digraph is transitively reduced.
is_equitable()	Check whether the given partition is equitable with respect to `self`.
is_self_complementary()	Check whether the graph is self-complementary.

Traversals:

breadth_first_search()	Return an iterator over the vertices in a breadth-first ordering.
depth_first_search()	Return an iterator over the vertices in a depth-first ordering.
lex_BFS()	Perform a lexicographic breadth first search (LexBFS) on the graph.
lex_UP()	Perform a lexicographic UP search (LexUP) on the graph.
lex_DFS()	Perform a lexicographic depth first search (LexDFS) on the graph.
lex_DOWN()	Perform a lexicographic DOWN search (LexDOWN) on the graph.

Distances:

centrality_betweenness()	Return the betweenness centrality
centrality_closeness()	Return the closeness centrality (1/average distance to all vertices)
distance()	Return the (directed) distance from u to v in the (di)graph
distance_all_pairs()	Return the distances between all pairs of vertices.
distances_distribution()	Return the distances distribution of the (di)graph in a dictionary.
girth()	Return the girth of the graph.
odd_girth()	Return the odd girth of the graph.
shortest_path()	Return a list of vertices representing some shortest path from u to v
shortest_path_length()	Return the minimal length of paths from u to v
shortest_paths()	Return a dictionary associating to each vertex v a shortest path from u to v, if it exists.
shortest_path_lengths()	Return a dictionary of shortest path lengths keyed by targets that are connected by a path from u.
shortest_path_all_pairs()	Compute a shortest path between each pair of vertices.
wiener_index()	Return the Wiener index of the graph.
average_distance()	Return the average distance between vertices of the graph.

Flows, connectivity, trees:

is_connected()	Check whether the (di)graph is connected.
connected_components()	Return the list of connected components
number_of_connected_components()	Return the number of connected components.
connected_components_subgraphs()	Return a list of connected components as graph objects.
connected_component_containing_vertex()	Return a list of the vertices connected to vertex.
connected_components_sizes()	Return the sizes of the connected components as a list.
blocks_and_cut_vertices()	Compute the blocks and cut vertices of the graph.
blocks_and_cuts_tree()	Compute the blocks-and-cuts tree of the graph.
is_biconnected()	Check whether the graph is biconnected.
biconnected_components()	Return the list of biconnected components.
biconnected_components_subgraphs()	Return a list of biconnected components as graph objects.
number_of_biconnected_components()	Return the number of biconnected components.
is_cut_edge()	Check whether the input edge is a cut-edge or a bridge.
is_edge_cut()	Check whether the input edges form an edge cut.
is_cut_vertex()	Check whether the input vertex is a cut-vertex.
is_vertex_cut()	Check whether the input vertices form a vertex cut.
edge_cut()	Return a minimum edge cut between vertices s and t
vertex_cut()	Return a minimum vertex cut between non-adjacent vertices s and t
flow()	Return a maximum flow in the graph from `x` to `y`
nowhere_zero_flow()	Return a k-nowhere zero flow of the (di)graph.
edge_disjoint_paths()	Return a list of edge-disjoint paths between two vertices
vertex_disjoint_paths()	Return a list of vertex-disjoint paths between two vertices
edge_connectivity()	Return the edge connectivity of the graph.
vertex_connectivity()	Return the vertex connectivity of the graph.
transitive_closure()	Compute the transitive closure of a graph and returns it.
transitive_reduction()	Return a transitive reduction of a graph.
min_spanning_tree()	Return the edges of a minimum spanning tree.
number_of_spanning_trees()	Return the number of spanning trees in a graph.
dominator_tree()	Return a dominator tree of the graph.
connected_subgraph_iterator()	Iterator over the induced connected subgraphs of order at most k

Plot/embedding-related methods:

set_embedding()	Set a combinatorial embedding dictionary to _embedding attribute.
get_embedding()	Return the attribute _embedding if it exists.
faces()	Return the faces of an embedded graph.
genus()	Return the number of faces of an embedded graph.
planar_dual()	Return the planar dual of an embedded graph.
get_pos()	Return the position dictionary
set_pos()	Set the position dictionary.
layout_planar()	Compute a planar layout of the graph using Schnyder's algorithm.
is_drawn_free_of_edge_crossings()	Check whether the position dictionary gives a planar embedding.
latex_options()	Return an instance of *GraphLatex* for the graph.
set_latex_options()	Set multiple options for rendering a graph with LaTeX.
layout()	Return a layout for the vertices of this graph.
layout_spring()	Return a spring layout for this graph
layout_ranked()	Return a ranked layout for this graph
layout_extend_randomly()	Extend randomly a partial layout
layout_circular()	Return a circular layout for this graph
layout_tree()	Return an ordered tree layout for this graph
layout_forest()	Return an ordered forest layout for this graph
layout_graphviz()	Call graphviz to compute a layout of the vertices of this graph.
GenericGraph._circle_embedding	Set some vertices on a circle in the embedding of this graph.
GenericGraph._line_embedding	Set some vertices on a line in the embedding of this graph.
graphplot()	Return a *GraphPlot* object.
plot()	Return a Graphics object representing the (di)graph.
show()	Show the (di)graph.
plot3d()	Plot the graph in three dimensions.
show3d()	Plot the graph using Tachyon, and shows the resulting plot.
graphviz_string()	Return a representation in the dot language.
graphviz_to_file_named()	Write a representation in the dot language in a file.
tikz()	Return a TikzPicture object representing the (di)graph.

Algorithmically hard stuff:

steiner_tree()	Return a tree of minimum weight connecting the given set of vertices.
edge_disjoint_spanning_trees()	Return the desired number of edge-disjoint spanning trees/arborescences.
feedback_vertex_set()	Compute the minimum feedback vertex set of a (di)graph.
multiway_cut()	Return a minimum edge multiway cut
max_cut()	Return a maximum edge cut of the graph.
longest_cycle()	Return the longest (induced) cycle of `self`.
longest_path()	Return a longest path of `self`.
traveling_salesman_problem()	Solve the traveling salesman problem (TSP)
is_hamiltonian()	Test whether the current graph is Hamiltonian.
hamiltonian_cycle()	Return a Hamiltonian cycle/circuit of the current graph/digraph
hamiltonian_path()	Return a Hamiltonian path of the current graph/digraph
multicommodity_flow()	Solve a multicommodity flow problem.
disjoint_routed_paths()	Return a set of disjoint routed paths.
dominating_set()	Return a minimum dominating set of the graph
greedy_dominating_set()	Return a greedy distance-k dominating set of the graph.
maximum_leaf_number()	Return the maximum leaf number of the graph.
subgraph_search()	Return a copy of `G` in `self`.
subgraph_search_count()	Return the number of labelled occurrences of `G` in `self`.
subgraph_search_iterator()	Return an iterator over the labelled copies of `G` in `self`.
subgraph_decompositions()	Return an iterator over the graph decompositions into isometric copies of another graph.
characteristic_polynomial()	Return the characteristic polynomial of the adjacency matrix of the (di)graph.
genus()	Return the minimal genus of the graph.
crossing_number()	Return the crossing number of the graph.

Miscellaneous

edge_polytope()	Return the edge polytope of `self`.
symmetric_edge_polytope()	Return the symmetric edge polytope of `self`.

1.1.1 Methods

class sage.graphs.generic_graph.**GenericGraph**

Bases: *GenericGraph_pyx*

Base class for graphs and digraphs.

__eq__(*other*)

Compare `self` and `other` for equality.

Do not call this method directly. That is, for `G.__eq__(H)` write `G == H`.

Two graphs are considered equal if the following hold:

- they are either both directed, or both undirected
- they have the same settings for loops, multiedges, and weightedness
- they have the same set of vertices

- they have the same (multi)set of arrows/edges, where labels of arrows/edges are taken into account if *and only if* the graphs are considered weighted. See `weighted()`.

Note that this is *not* an isomorphism test.

EXAMPLES:

```
sage: G = graphs.EmptyGraph()
sage: H = Graph()
sage: G == H
True
sage: G.to_directed() == H.to_directed()
True
sage: G = graphs.RandomGNP(8, .9999)
sage: H = graphs.CompleteGraph(8)
sage: G == H  # random - most often true
True
sage: G = Graph({0: [1, 2, 3, 4, 5, 6, 7]} )
sage: H = Graph({1: [0], 2: [0], 3: [0], 4: [0], 5: [0], 6: [0], 7: [0]} )
sage: G == H
True
sage: G.allow_loops(True)
sage: G == H
False
sage: G = graphs.RandomGNP(9, .3).to_directed()
sage: H = graphs.RandomGNP(9, .3).to_directed()
sage: G == H # most often false
False
sage: G = Graph(multiedges=True, sparse=True)
sage: G.add_edge(0, 1)
sage: H = copy(G)
sage: H.add_edge(0, 1)
sage: G == H
False
```

```
>>> from sage.all import *
>>> G = graphs.EmptyGraph()
>>> H = Graph()
>>> G == H
True
>>> G.to_directed() == H.to_directed()
True
>>> G = graphs.RandomGNP(Integer(8), RealNumber('.9999'))
>>> H = graphs.CompleteGraph(Integer(8))
>>> G == H  # random - most often true
True
>>> G = Graph({Integer(0): [Integer(1), Integer(2), Integer(3), Integer(4),␣
↪Integer(5), Integer(6), Integer(7)]} )
>>> H = Graph({Integer(1): [Integer(0)], Integer(2): [Integer(0)],␣
↪Integer(3): [Integer(0)], Integer(4): [Integer(0)], Integer(5):␣
↪[Integer(0)], Integer(6): [Integer(0)], Integer(7): [Integer(0)]} )
>>> G == H
True
>>> G.allow_loops(True)
```

(continues on next page)

(continued from previous page)

```
>>> G == H
False
>>> G = graphs.RandomGNP(Integer(9), RealNumber('.3')).to_directed()
>>> H = graphs.RandomGNP(Integer(9), RealNumber('.3')).to_directed()
>>> G == H # most often false
False
>>> G = Graph(multiedges=True, sparse=True)
>>> G.add_edge(Integer(0), Integer(1))
>>> H = copy(G)
>>> H.add_edge(Integer(0), Integer(1))
>>> G == H
False
```

Note that graphs must be considered weighted, or Sage will not pay attention to edge label data in equality testing:

```
sage: foo = Graph(sparse=True)
sage: foo.add_edges([(0, 1, 1), (0, 2, 2)])
sage: bar = Graph(sparse=True)
sage: bar.add_edges([(0, 1, 2), (0, 2, 1)])
sage: foo == bar
True
sage: foo.weighted(True)
sage: foo == bar
False
sage: bar.weighted(True)
sage: foo == bar
False
```

```
>>> from sage.all import *
>>> foo = Graph(sparse=True)
>>> foo.add_edges([(Integer(0), Integer(1), Integer(1)), (Integer(0),␣
↪Integer(2), Integer(2))])
>>> bar = Graph(sparse=True)
>>> bar.add_edges([(Integer(0), Integer(1), Integer(2)), (Integer(0),␣
↪Integer(2), Integer(1))])
>>> foo == bar
True
>>> foo.weighted(True)
>>> foo == bar
False
>>> bar.weighted(True)
>>> foo == bar
False
```

_circle_embedding(*vertices*, *center=(0, 0)*, *radius=1*, *shift=0*, *angle=0*, *return_dict=False*)

Set some vertices on a circle in the embedding of a this graph.

By default, this method modifies the graph's embedding so that the vertices listed in `vertices` appear in this ordering on a circle of given radius and center.

INPUT:

- `vertices` – an iterable container of vertices (list, set, dict, etc.). The order of the vertices in the circle

embedding is given by `list(vertices)`.

- `center` – tuple (default: $(0, 0)$); position of the center of the circle
- `radius` – (default: 1) the radius of the circle
- `shift` – (default: 0) rotation of the circle. A value of `shift=1` will replace in the drawing the i-th element of the list by the $(i-1)$-th. Non-integer values are admissible, and a value of α corresponds to a rotation of the circle by an angle of $\alpha 2\pi/n$ (where n is the number of vertices set on the circle).
- `angle` – (default: 0) rotate the embedding of all vertices. For instance, when `angle == 0`, the first vertex get position `(center[0] + radius, center[1])`. With a value of $\pi/2$, the first vertex get position `(center[0], center[1] + radius)`.
- `return_dict` – boolean (default: `False`); by default the computed positions of the specified vertices are stored in the position dictionary of the graph. When `return_dict == True`, a dictionary containing only the position of the specified vertices is returned.

EXAMPLES:

```
sage: g = graphs.CycleGraph(5)
sage: g._circle_embedding([0, 2, 4, 1, 3], radius=2, shift=.5)
sage: g.show()                                                              #␣
↪needs sage.plot

sage: g._circle_embedding(g.vertices(sort=True), angle=0)
sage: g._pos[0]
(1.0, 0.0)
sage: from math import pi
sage: g._circle_embedding(g.vertices(sort=True), angle=pi/2)
sage: g._pos[0]
(0.0, 1.0)

sage: g = graphs.CycleGraph(4)
sage: pos = g._circle_embedding(g.vertex_iterator(), return_dict=True)
sage: pos[0]
(1.0, 0.0)
```

```
>>> from sage.all import *
>>> g = graphs.CycleGraph(Integer(5))
>>> g._circle_embedding([Integer(0), Integer(2), Integer(4), Integer(1),␣
↪Integer(3)], radius=Integer(2), shift=RealNumber('.5'))
>>> g.show()                                                                #␣
↪needs sage.plot

>>> g._circle_embedding(g.vertices(sort=True), angle=Integer(0))
>>> g._pos[Integer(0)]
(1.0, 0.0)
>>> from math import pi
>>> g._circle_embedding(g.vertices(sort=True), angle=pi/Integer(2))
>>> g._pos[Integer(0)]
(0.0, 1.0)

>>> g = graphs.CycleGraph(Integer(4))
>>> pos = g._circle_embedding(g.vertex_iterator(), return_dict=True)
>>> pos[Integer(0)]
```

(continues on next page)

(continued from previous page)

```
(1.0, 0.0)
```

_get_weight_function(*by_weight=False*, *weight_function=None*, *check_weight=True*)

Return an edge weight function.

An edge weight function is a function that takes as input an edge and outputs its weight.

This method is a helper function for methods using the weight of edges. It either checks the validity of an input weight function, or returns a valid weight function on the edges.

INPUT:

- `by_weight` – boolean (default: `False`); if `True`, the edges in the graph are weighted, otherwise all edges have weight 1
- `weight_function` – function (default: `None`); a function that takes as input an edge `(u, v, l)` and outputs its weight. If not `None`, `by_weight` is automatically set to `True`. If `None` and `by_weight` is `True`, we use the edge label `l`, if `l` is not `None`, else `1` as a weight.
- `check_weight` – boolean (default: `True`); if `True`, we check that the weight_function outputs a number for each edge

EXAMPLES:

The default weight function outputs 1 for each edge:

```
sage: G = Graph([(0, 1, 1), (1, 2, 3), (2, 3, 2)])
sage: by_weight, weight_function = G._get_weight_function()
sage: by_weight
False
sage: [weight_function(e) for e in G.edges(sort=True)]
[1, 1, 1]
```

```
>>> from sage.all import *
>>> G = Graph([(Integer(0), Integer(1), Integer(1)), (Integer(1), Integer(2),␣
↪Integer(3)), (Integer(2), Integer(3), Integer(2))])
>>> by_weight, weight_function = G._get_weight_function()
>>> by_weight
False
>>> [weight_function(e) for e in G.edges(sort=True)]
[1, 1, 1]
```

The standard weight function outputs labels:

```
sage: G = Graph([(0, 1, 1), (1, 2, 3), (2, 3, 2)])
sage: by_weight, weight_function = G._get_weight_function(by_weight=True)
sage: by_weight
True
sage: [weight_function(e) for e in G.edges(sort=True)]
[1, 3, 2]
```

```
>>> from sage.all import *
>>> G = Graph([(Integer(0), Integer(1), Integer(1)), (Integer(1), Integer(2),␣
↪Integer(3)), (Integer(2), Integer(3), Integer(2))])
>>> by_weight, weight_function = G._get_weight_function(by_weight=True)
>>> by_weight
```

(continues on next page)

(continued from previous page)

```
True
>>> [weight_function(e) for e in G.edges(sort=True)]
[1, 3, 2]
```

However, it might be more complicated:

```
sage: G = Graph([(0, 1, {'name':'a', 'weight':1}), (1, 2, {'name': 'b',
↪'weight': 3}), (2, 3, {'name': 'c', 'weight': 2})])
sage: by_weight, weight_function = G._get_weight_function(weight_
↪function=lambda e:e[2]['weight'])
sage: by_weight
True
sage: [weight_function(e) for e in G.edges(sort=True)]
[1, 3, 2]
```

```
>>> from sage.all import *
>>> G = Graph([(Integer(0), Integer(1), {'name':'a', 'weight':Integer(1)}),␣
↪(Integer(1), Integer(2), {'name': 'b', 'weight': Integer(3)}), (Integer(2),␣
↪Integer(3), {'name': 'c', 'weight': Integer(2)})])
>>> by_weight, weight_function = G._get_weight_function(weight_
↪function=lambda e:e[Integer(2)]['weight'])
>>> by_weight
True
>>> [weight_function(e) for e in G.edges(sort=True)]
[1, 3, 2]
```

Numeric string as a weight in weight_function:

```
sage: G = Graph({0: {1: '123'}})
sage: by_weight, weight_function = G._get_weight_function(by_weight=True)
Traceback (most recent call last):
...
ValueError: the weight function cannot find the weight of (0, 1, '123')
```

```
>>> from sage.all import *
>>> G = Graph({Integer(0): {Integer(1): '123'}})
>>> by_weight, weight_function = G._get_weight_function(by_weight=True)
Traceback (most recent call last):
...
ValueError: the weight function cannot find the weight of (0, 1, '123')
```

_line_embedding(*vertices*, *first=(0, 0)*, *last=(0, 1)*, *return_dict=False*)

Set some vertices on a line in the embedding of this graph.

By default, this method modifies the graph's embedding so that the vertices of `vertices` appear on a line, where the position of `vertices[0]` is the pair `first` and the position of `vertices[-1]` is `last`. The vertices are evenly spaced.

INPUT:

- `vertices` – an iterable container of vertices (list, set, dict, etc.). The order of the vertices in the line embedding is given by `list(vertices)`.
- `first` – tuple (default: $(0, 0)$); first coordinate of the line

- `last` – tuple (default: $(0, 1)$); last coordinate of the line
- `return_dict` – boolean (default: `False`); by default the computed positions of the specified vertices are stored in the position dictionary of the graph. When `return_dict == True`, a dictionary containing only the position of the specified vertices is returned.

EXAMPLES:

```
sage: g = graphs.PathGraph(5)
sage: g._line_embedding([0, 2, 4, 1, 3], first=(-1, -1), last=(1, 1))
sage: g.show()                                                              #␣
↪needs sage.plot

sage: pos = g._line_embedding([4, 2, 0, 1, 3], first=(-1, -1), last=(1, 1),
....:                         return_dict=True)
sage: pos[0]
(0.0, 0.0)
sage: g.get_pos()[0]
(-1, -1)
```

```
>>> from sage.all import *
>>> g = graphs.PathGraph(Integer(5))
>>> g._line_embedding([Integer(0), Integer(2), Integer(4), Integer(1),␣
↪Integer(3)], first=(-Integer(1), -Integer(1)), last=(Integer(1),␣
↪Integer(1)))
>>> g.show()                                                                #␣
↪needs sage.plot

>>> pos = g._line_embedding([Integer(4), Integer(2), Integer(0), Integer(1),␣
↪Integer(3)], first=(-Integer(1), -Integer(1)), last=(Integer(1),␣
↪Integer(1)),
...                         return_dict=True)
>>> pos[Integer(0)]
(0.0, 0.0)
>>> g.get_pos()[Integer(0)]
(-1, -1)
```

add_clique(*vertices*, *loops=False*)

Add a clique to the graph with the given vertices.

If the vertices are already present, only the edges are added.

INPUT:

- `vertices` – an iterable container of vertices for the clique to be added, e.g. a list, set, graph, etc.
- `loops` – boolean (default: `False`); whether to add edges from every given vertex to itself. This is allowed only if the (di)graph allows loops.

EXAMPLES:

```
sage: G = Graph()
sage: G.add_clique(range(4))
sage: G.is_isomorphic(graphs.CompleteGraph(4))
True
sage: D = DiGraph()
sage: D.add_clique(range(4))
```

(continues on next page)

(continued from previous page)

```
sage: D.is_isomorphic(digraphs.Complete(4))
True
sage: D = DiGraph(loops=True)
sage: D.add_clique(range(4), loops=True)
sage: D.is_isomorphic(digraphs.Complete(4, loops=True))
True
sage: D = DiGraph(loops=False)
sage: D.add_clique(range(4), loops=True)
Traceback (most recent call last):
...
ValueError: cannot add edge from 0 to 0 in graph without loops
```

```
>>> from sage.all import *
>>> G = Graph()
>>> G.add_clique(range(Integer(4)))
>>> G.is_isomorphic(graphs.CompleteGraph(Integer(4)))
True
>>> D = DiGraph()
>>> D.add_clique(range(Integer(4)))
>>> D.is_isomorphic(digraphs.Complete(Integer(4)))
True
>>> D = DiGraph(loops=True)
>>> D.add_clique(range(Integer(4)), loops=True)
>>> D.is_isomorphic(digraphs.Complete(Integer(4), loops=True))
True
>>> D = DiGraph(loops=False)
>>> D.add_clique(range(Integer(4)), loops=True)
Traceback (most recent call last):
...
ValueError: cannot add edge from 0 to 0 in graph without loops
```

If the list of vertices contains repeated elements, a loop will be added at that vertex, even if `loops=False`:

```
sage: G = Graph(loops=True)
sage: G.add_clique([1, 1])
sage: G.edges(sort=True)
[(1, 1, None)]
```

```
>>> from sage.all import *
>>> G = Graph(loops=True)
>>> G.add_clique([Integer(1), Integer(1)])
>>> G.edges(sort=True)
[(1, 1, None)]
```

This is equivalent to:

```
sage: G = Graph(loops=True)
sage: G.add_clique([1], loops=True)
sage: G.edges(sort=True)
[(1, 1, None)]
```

```
>>> from sage.all import *
>>> G = Graph(loops=True)
>>> G.add_clique([Integer(1)], loops=True)
>>> G.edges(sort=True)
[(1, 1, None)]
```

add_cycle(*vertices*)

Add a cycle to the graph with the given vertices.

If the vertices are already present, only the edges are added.

For digraphs, adds the directed cycle, whose orientation is determined by the list. Adds edges `(vertices[u], vertices[u+1])` and `(vertices[-1], vertices[0])`.

INPUT:

- `vertices` – an ordered list of the vertices of the cycle to be added

EXAMPLES:

```
sage: G = Graph()
sage: G.add_vertices(range(10)); G
Graph on 10 vertices
sage: show(G)                                                                 #␣
↪needs sage.plot
sage: G.add_cycle(list(range(10, 20)))
sage: show(G)                                                                 #␣
↪needs sage.plot
sage: G.add_cycle(list(range(10)))
sage: show(G)                                                                 #␣
↪needs sage.plot
```

```
>>> from sage.all import *
>>> G = Graph()
>>> G.add_vertices(range(Integer(10))); G
Graph on 10 vertices
>>> show(G)                                                                   #␣
↪needs sage.plot
>>> G.add_cycle(list(range(Integer(10), Integer(20))))
>>> show(G)                                                                   #␣
↪needs sage.plot
>>> G.add_cycle(list(range(Integer(10))))
>>> show(G)                                                                   #␣
↪needs sage.plot
```

```
sage: D = DiGraph()
sage: D.add_cycle(list(range(4)))
sage: D.edges(sort=True)
[(0, 1, None), (1, 2, None), (2, 3, None), (3, 0, None)]
```

```
>>> from sage.all import *
>>> D = DiGraph()
>>> D.add_cycle(list(range(Integer(4))))
>>> D.edges(sort=True)
[(0, 1, None), (1, 2, None), (2, 3, None), (3, 0, None)]
```

add_edge(*u*, *v=None*, *label=None*)

Add an edge from `u` to `v`.

INPUT: The following forms are all accepted:

- G.add_edge(1, 2)
- G.add_edge((1, 2))
- G.add_edges([(1, 2)])
- G.add_edge(1, 2, 'label')
- G.add_edge((1, 2, 'label'))
- G.add_edges([(1, 2, 'label')])

WARNING: The following intuitive input results in nonintuitive output:

```
sage: G = Graph()
sage: G.add_edge((1, 2), 'label')
sage: G.edges(sort=False)
[('label', (1, 2), None)]
```

```
>>> from sage.all import *
>>> G = Graph()
>>> G.add_edge((Integer(1), Integer(2)), 'label')
>>> G.edges(sort=False)
[('label', (1, 2), None)]
```

You must either use the `label` keyword:

```
sage: G = Graph()
sage: G.add_edge((1, 2), label='label')
sage: G.edges(sort=False)
[(1, 2, 'label')]
```

```
>>> from sage.all import *
>>> G = Graph()
>>> G.add_edge((Integer(1), Integer(2)), label='label')
>>> G.edges(sort=False)
[(1, 2, 'label')]
```

Or use one of these:

```
sage: G = Graph()
sage: G.add_edge(1, 2, 'label')
sage: G.edges(sort=False)
[(1, 2, 'label')]
sage: G = Graph()
sage: G.add_edge((1, 2, 'label'))
sage: G.edges(sort=False)
[(1, 2, 'label')]
```

```
>>> from sage.all import *
>>> G = Graph()
>>> G.add_edge(Integer(1), Integer(2), 'label')
```

(continues on next page)

(continued from previous page)

```
>>> G.edges(sort=False)
[(1, 2, 'label')]
>>> G = Graph()
>>> G.add_edge((Integer(1), Integer(2), 'label'))
>>> G.edges(sort=False)
[(1, 2, 'label')]
```

Vertex name cannot be `None`, so:

```
sage: G = Graph()
sage: G.add_edge(None, 4)
sage: G.vertices(sort=True)
[0, 4]
```

```
>>> from sage.all import *
>>> G = Graph()
>>> G.add_edge(None, Integer(4))
>>> G.vertices(sort=True)
[0, 4]
```

add_edges(*edges*, *loops=True*)

Add edges from an iterable container.

INPUT:

- `edges` – an iterable of edges, given either as `(u, v)` or `(u, v, label)`
- `loops` – boolean (default: `True`); if `False`, remove all loops `(v, v)` from the input iterator. If `None`, remove loops unless the graph allows loops.

EXAMPLES:

```
sage: G = graphs.DodecahedralGraph()
sage: H = Graph()
sage: H.add_edges(G.edge_iterator()); H
Graph on 20 vertices
sage: G = graphs.DodecahedralGraph().to_directed()
sage: H = DiGraph()
sage: H.add_edges(G.edge_iterator()); H
Digraph on 20 vertices
sage: H.add_edges(iter([]))

sage: H = Graph()
sage: H.add_edges([(0, 1), (0, 2, "label")])
sage: H.edges(sort=True)
[(0, 1, None), (0, 2, 'label')]
```

```
>>> from sage.all import *
>>> G = graphs.DodecahedralGraph()
>>> H = Graph()
>>> H.add_edges(G.edge_iterator()); H
Graph on 20 vertices
>>> G = graphs.DodecahedralGraph().to_directed()
>>> H = DiGraph()
```

(continues on next page)

(continued from previous page)

```
>>> H.add_edges(G.edge_iterator()); H
Digraph on 20 vertices
>>> H.add_edges(iter([]))

>>> H = Graph()
>>> H.add_edges([(Integer(0), Integer(1)), (Integer(0), Integer(2), "label")])
>>> H.edges(sort=True)
[(0, 1, None), (0, 2, 'label')]
```

We demonstrate the `loops` argument:

```
sage: H = Graph()
sage: H.add_edges([(0, 0)], loops=False); H.edges(sort=True)
[]
sage: H.add_edges([(0, 0)], loops=None); H.edges(sort=True)
[]
sage: H.add_edges([(0, 0)]); H.edges(sort=True)
Traceback (most recent call last):
...
ValueError: cannot add edge from 0 to 0 in graph without loops
sage: H = Graph(loops=True)
sage: H.add_edges([(0, 0)], loops=False); H.edges(sort=True)
[]
sage: H.add_edges([(0, 0)], loops=None); H.edges(sort=True)
[(0, 0, None)]
sage: H.add_edges([(0, 0)]); H.edges(sort=True)
[(0, 0, None)]
```

```
>>> from sage.all import *
>>> H = Graph()
>>> H.add_edges([(Integer(0), Integer(0))], loops=False); H.edges(sort=True)
[]
>>> H.add_edges([(Integer(0), Integer(0))], loops=None); H.edges(sort=True)
[]
>>> H.add_edges([(Integer(0), Integer(0))]); H.edges(sort=True)
Traceback (most recent call last):
...
ValueError: cannot add edge from 0 to 0 in graph without loops
>>> H = Graph(loops=True)
>>> H.add_edges([(Integer(0), Integer(0))], loops=False); H.edges(sort=True)
[]
>>> H.add_edges([(Integer(0), Integer(0))], loops=None); H.edges(sort=True)
[(0, 0, None)]
>>> H.add_edges([(Integer(0), Integer(0))]); H.edges(sort=True)
[(0, 0, None)]
```

add_path(*vertices*)

Add a path to the graph with the given vertices.

If the vertices are already present, only the edges are added.

For digraphs, adds the directed path `vertices[0], ..., vertices[-1]`.

INPUT:

- `vertices` – an ordered list of the vertices of the path to be added

EXAMPLES:

```
sage: G = Graph()
sage: G.add_vertices(range(10)); G
Graph on 10 vertices
sage: show(G)                                                               #␣
↪needs sage.plot
sage: G.add_path(list(range(10, 20)))
sage: show(G)                                                               #␣
↪needs sage.plot
sage: G.add_path(list(range(10)))
sage: show(G)                                                               #␣
↪needs sage.plot
```

```
>>> from sage.all import *
>>> G = Graph()
>>> G.add_vertices(range(Integer(10))); G
Graph on 10 vertices
>>> show(G)                                                                 #␣
↪needs sage.plot
>>> G.add_path(list(range(Integer(10), Integer(20))))
>>> show(G)                                                                 #␣
↪needs sage.plot
>>> G.add_path(list(range(Integer(10))))
>>> show(G)                                                                 #␣
↪needs sage.plot
```

```
sage: D = DiGraph()
sage: D.add_path(list(range(4)))
sage: D.edges(sort=True)
[(0, 1, None), (1, 2, None), (2, 3, None)]
```

```
>>> from sage.all import *
>>> D = DiGraph()
>>> D.add_path(list(range(Integer(4))))
>>> D.edges(sort=True)
[(0, 1, None), (1, 2, None), (2, 3, None)]
```

add_vertex(*name=None*)

Create an isolated vertex.

If the vertex already exists, then nothing is done.

INPUT:

- `name` – an immutable object (default: `None`); when no name is specified (default), then the new vertex will be represented by the least integer not already representing a vertex. `name` must be an immutable object (e.g., an integer, a tuple, etc.).

As it is implemented now, if a graph G has a large number of vertices with numeric labels, then `G.add_vertex()` could potentially be slow, if `name=None`.

OUTPUT: if `name=None`, the new vertex name is returned. `None` otherwise

EXAMPLES:

```
sage: G = Graph(); G.add_vertex(); G
0
Graph on 1 vertex
```

```
>>> from sage.all import *
>>> G = Graph(); G.add_vertex(); G
0
Graph on 1 vertex
```

```
sage: D = DiGraph(); D.add_vertex(); D
0
Digraph on 1 vertex
```

```
>>> from sage.all import *
>>> D = DiGraph(); D.add_vertex(); D
0
Digraph on 1 vertex
```

add_vertices(*vertices*)

Add vertices to the (di)graph from an iterable container of vertices.

Vertices that already exist in the graph will not be added again.

INPUT:

- `vertices` – iterator container of vertex labels. A new label is created, used and returned in the output list for all `None` values in `vertices`.

OUTPUT:

Generated names of new vertices if there is at least one `None` value present in `vertices`. `None` otherwise.

EXAMPLES:

```
sage: d = {0: [1,4,5], 1: [2,6], 2: [3,7], 3: [4,8], 4: [9], 5: [7,8], 6: [8,
↪9], 7: [9]}
sage: G = Graph(d)
sage: G.add_vertices([10,11,12])
sage: G.vertices(sort=True)
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]
sage: G.add_vertices(graphs.CycleGraph(25).vertex_iterator())
sage: G.vertices(sort=True)
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21,
↪ 22, 23, 24]
```

```
>>> from sage.all import *
>>> d = {Integer(0): [Integer(1),Integer(4),Integer(5)], Integer(1):␣
↪[Integer(2),Integer(6)], Integer(2): [Integer(3),Integer(7)], Integer(3):␣
↪[Integer(4),Integer(8)], Integer(4): [Integer(9)], Integer(5): [Integer(7),
↪Integer(8)], Integer(6): [Integer(8),Integer(9)], Integer(7): [Integer(9)]}
>>> G = Graph(d)
>>> G.add_vertices([Integer(10),Integer(11),Integer(12)])
>>> G.vertices(sort=True)
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]
```

(continues on next page)

(continued from previous page)

```
>>> G.add_vertices(graphs.CycleGraph(Integer(25)).vertex_iterator())
>>> G.vertices(sort=True)
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21,
↪ 22, 23, 24]
```

```
sage: G = Graph()
sage: G.add_vertices([1, 2, 3])
sage: G.add_vertices([4, None, None, 5])
[0, 6]
```

```
>>> from sage.all import *
>>> G = Graph()
>>> G.add_vertices([Integer(1), Integer(2), Integer(3)])
>>> G.add_vertices([Integer(4), None, None, Integer(5)])
[0, 6]
```

adjacency_matrix(*sparse*, *vertices=None*, *base_ring=None*, ***kwds*)

Return the adjacency matrix of the (di)graph.

By default, the matrix returned is over the integers.

INPUT:

- `sparse` – boolean (default: `None`); whether to represent with a sparse matrix
- `vertices` – list; `None`, or `True` (default: `None`)
  - when a list, the i-th row and column of the matrix correspond to the i-th vertex in the ordering of `vertices`,
  - when `None`, the i-th row and column of the matrix correspond to the i-th vertex in the ordering given by *`GenericGraph.vertices()`* with `sort=True`.
  - when `True`, construct an endomorphism of a free module instead of a matrix, where the module's basis is indexed by the vertices.

  If the vertices are not comparable, the keyword `vertices` must be used to specify an ordering, or a `TypeError` exception will be raised.
- `base_ring` – a ring (default: `ZZ`); the base ring of the matrix space to use
- `**kwds` – other keywords to pass to `matrix()`

EXAMPLES:

```
sage: G = graphs.CubeGraph(4)
sage: G.adjacency_matrix()                                                  #␣
↪needs sage.modules
[0 1 1 0 1 0 0 0 1 0 0 0 0 0 0 0]
[1 0 0 1 0 1 0 0 0 1 0 0 0 0 0 0]
[1 0 0 1 0 0 1 0 0 0 1 0 0 0 0 0]
[0 1 1 0 0 0 0 1 0 0 0 1 0 0 0 0]
[1 0 0 0 0 1 1 0 0 0 0 0 1 0 0 0]
[0 1 0 0 1 0 0 1 0 0 0 0 0 1 0 0]
[0 0 1 0 1 0 0 1 0 0 0 0 0 0 1 0]
[0 0 0 1 0 1 1 0 0 0 0 0 0 0 0 1]
[1 0 0 0 0 0 0 0 0 1 1 0 1 0 0 0]
```

(continues on next page)

(continued from previous page)

```
[0 1 0 0 0 0 0 0 1 0 0 1 0 1 0 0]
[0 0 1 0 0 0 0 0 1 0 0 1 0 0 1 0]
[0 0 0 1 0 0 0 0 0 1 1 0 0 0 0 1]
[0 0 0 0 1 0 0 0 1 0 0 0 0 1 1 0]
[0 0 0 0 0 1 0 0 0 1 0 0 1 0 0 1]
[0 0 0 0 0 0 1 0 0 0 1 0 1 0 0 1]
[0 0 0 0 0 0 0 1 0 0 0 1 0 1 1 0]
```

```
>>> from sage.all import *
>>> G = graphs.CubeGraph(Integer(4))
>>> G.adjacency_matrix()                                                    #␣
↪needs sage.modules
[0 1 1 0 1 0 0 0 1 0 0 0 0 0 0 0]
[1 0 0 1 0 1 0 0 0 1 0 0 0 0 0 0]
[1 0 0 1 0 0 1 0 0 0 1 0 0 0 0 0]
[0 1 1 0 0 0 0 1 0 0 0 1 0 0 0 0]
[1 0 0 0 0 1 1 0 0 0 0 0 1 0 0 0]
[0 1 0 0 1 0 0 1 0 0 0 0 0 1 0 0]
[0 0 1 0 1 0 0 1 0 0 0 0 0 0 1 0]
[0 0 0 1 0 1 1 0 0 0 0 0 0 0 0 1]
[1 0 0 0 0 0 0 0 0 1 1 0 1 0 0 0]
[0 1 0 0 0 0 0 0 1 0 0 1 0 1 0 0]
[0 0 1 0 0 0 0 0 1 0 0 1 0 0 1 0]
[0 0 0 1 0 0 0 0 0 1 1 0 0 0 0 1]
[0 0 0 0 1 0 0 0 1 0 0 0 0 1 1 0]
[0 0 0 0 0 1 0 0 0 1 0 0 1 0 0 1]
[0 0 0 0 0 0 1 0 0 0 1 0 1 0 0 1]
[0 0 0 0 0 0 0 1 0 0 0 1 0 1 1 0]
```

```
sage: matrix(GF(2), G)  # matrix over GF(2)                                #␣
↪needs sage.modules sage.rings.finite_rings
[0 1 1 0 1 0 0 0 1 0 0 0 0 0 0 0]
[1 0 0 1 0 1 0 0 0 1 0 0 0 0 0 0]
[1 0 0 1 0 0 1 0 0 0 1 0 0 0 0 0]
[0 1 1 0 0 0 0 1 0 0 0 1 0 0 0 0]
[1 0 0 0 0 1 1 0 0 0 0 0 1 0 0 0]
[0 1 0 0 1 0 0 1 0 0 0 0 0 1 0 0]
[0 0 1 0 1 0 0 1 0 0 0 0 0 0 1 0]
[0 0 0 1 0 1 1 0 0 0 0 0 0 0 0 1]
[1 0 0 0 0 0 0 0 0 1 1 0 1 0 0 0]
[0 1 0 0 0 0 0 0 1 0 0 1 0 1 0 0]
[0 0 1 0 0 0 0 0 1 0 0 1 0 0 1 0]
[0 0 0 1 0 0 0 0 0 1 1 0 0 0 0 1]
[0 0 0 0 1 0 0 0 1 0 0 0 0 1 1 0]
[0 0 0 0 0 1 0 0 0 1 0 0 1 0 0 1]
[0 0 0 0 0 0 1 0 0 0 1 0 1 0 0 1]
[0 0 0 0 0 0 0 1 0 0 0 1 0 1 1 0]
```

```
>>> from sage.all import *
>>> matrix(GF(Integer(2)), G)  # matrix over GF(2)                          ␣
↪     # needs sage.modules sage.rings.finite_rings
```

(continues on next page)

(continued from previous page)

```
[0 1 1 0 1 0 0 0 1 0 0 0 0 0 0 0]
[1 0 0 1 0 1 0 0 0 1 0 0 0 0 0 0]
[1 0 0 1 0 0 1 0 0 0 1 0 0 0 0 0]
[0 1 1 0 0 0 0 1 0 0 0 1 0 0 0 0]
[1 0 0 0 0 1 1 0 0 0 0 0 1 0 0 0]
[0 1 0 0 1 0 0 1 0 0 0 0 0 1 0 0]
[0 0 1 0 1 0 0 1 0 0 0 0 0 0 1 0]
[0 0 0 1 0 1 1 0 0 0 0 0 0 0 0 1]
[1 0 0 0 0 0 0 0 0 1 1 0 1 0 0 0]
[0 1 0 0 0 0 0 0 1 0 0 1 0 1 0 0]
[0 0 1 0 0 0 0 0 1 0 0 1 0 0 1 0]
[0 0 0 1 0 0 0 0 0 1 1 0 0 0 0 1]
[0 0 0 0 1 0 0 0 1 0 0 0 0 1 1 0]
[0 0 0 0 0 1 0 0 0 1 0 0 1 0 0 1]
[0 0 0 0 0 0 1 0 0 0 1 0 1 0 0 1]
[0 0 0 0 0 0 0 1 0 0 0 1 0 1 1 0]
```

```
sage: D = DiGraph({0: [1, 2, 3], 1: [0, 2], 2: [3],
....:              3: [4], 4: [0, 5], 5: [1]})
sage: D.adjacency_matrix()                                                    #␣
↪needs sage.modules
[0 1 1 1 0 0]
[1 0 1 0 0 0]
[0 0 0 1 0 0]
[0 0 0 0 1 0]
[1 0 0 0 0 1]
[0 1 0 0 0 0]
```

```
>>> from sage.all import *
>>> D = DiGraph({Integer(0): [Integer(1), Integer(2), Integer(3)],␣
↪Integer(1): [Integer(0), Integer(2)], Integer(2): [Integer(3)],
...              Integer(3): [Integer(4)], Integer(4): [Integer(0),␣
↪Integer(5)], Integer(5): [Integer(1)]})
>>> D.adjacency_matrix()                                                      #␣
↪needs sage.modules
[0 1 1 1 0 0]
[1 0 1 0 0 0]
[0 0 0 1 0 0]
[0 0 0 0 1 0]
[1 0 0 0 0 1]
[0 1 0 0 0 0]
```

A different ordering of the vertices:

```
sage: graphs.PathGraph(5).adjacency_matrix(vertices=[2, 4, 1, 3, 0])          #␣
↪needs sage.modules
[0 0 1 1 0]
[0 0 0 1 0]
[1 0 0 0 1]
[1 1 0 0 0]
[0 0 1 0 0]
```

```
>>> from sage.all import *
>>> graphs.PathGraph(Integer(5)).adjacency_matrix(vertices=[Integer(2),␣
↪Integer(4), Integer(1), Integer(3), Integer(0)])        # needs sage.modules
[0 0 1 1 0]
[0 0 0 1 0]
[1 0 0 0 1]
[1 1 0 0 0]
[0 0 1 0 0]
```

A different base ring:

```
sage: graphs.PathGraph(5).adjacency_matrix(base_ring=RDF)                    #␣
↪needs sage.modules
[0.0 1.0 0.0 0.0 0.0]
[1.0 0.0 1.0 0.0 0.0]
[0.0 1.0 0.0 1.0 0.0]
[0.0 0.0 1.0 0.0 1.0]
[0.0 0.0 0.0 1.0 0.0]
sage: type(_)                                                                #␣
↪needs sage.modules
<class 'sage.matrix.matrix_real_double_dense.Matrix_real_double_dense'>
```

```
>>> from sage.all import *
>>> graphs.PathGraph(Integer(5)).adjacency_matrix(base_ring=RDF)             ␣
↪     # needs sage.modules
[0.0 1.0 0.0 0.0 0.0]
[1.0 0.0 1.0 0.0 0.0]
[0.0 1.0 0.0 1.0 0.0]
[0.0 0.0 1.0 0.0 1.0]
[0.0 0.0 0.0 1.0 0.0]
>>> type(_)                                                                  #␣
↪needs sage.modules
<class 'sage.matrix.matrix_real_double_dense.Matrix_real_double_dense'>
```

A different matrix implementation:

```
sage: graphs.PathGraph(5).adjacency_matrix(sparse=False,                     #␣
↪needs numpy sage.modules
....:                                      implementation='numpy')
[0 1 0 0 0]
[1 0 1 0 0]
[0 1 0 1 0]
[0 0 1 0 1]
[0 0 0 1 0]
sage: type(_)
<class 'sage.matrix.matrix_numpy_integer_dense.Matrix_numpy_integer_dense'>
```

```
>>> from sage.all import *
>>> graphs.PathGraph(Integer(5)).adjacency_matrix(sparse=False,              ␣
↪     # needs numpy sage.modules
...                                    implementation='numpy')
[0 1 0 0 0]
[1 0 1 0 0]
```

(continues on next page)

(continued from previous page)

```
[0 1 0 1 0]
[0 0 1 0 1]
[0 0 0 1 0]
>>> type(_)
<class 'sage.matrix.matrix_numpy_integer_dense.Matrix_numpy_integer_dense'>
```

As an immutable matrix:

```
sage: M = graphs.PathGraph(5).adjacency_matrix(sparse=False,                #␣
↪needs sage.modules
....:                                                 immutable=True); M
[0 1 0 0 0]
[1 0 1 0 0]
[0 1 0 1 0]
[0 0 1 0 1]
[0 0 0 1 0]
sage: M[2, 2] = 1                                                           #␣
↪needs sage.modules
Traceback (most recent call last):
...
ValueError: matrix is immutable; please change a copy instead
(i.e., use copy(M) to change a copy of M).
```

```
>>> from sage.all import *
>>> M = graphs.PathGraph(Integer(5)).adjacency_matrix(sparse=False,          ␣
↪     # needs sage.modules
...                                                 immutable=True); M
[0 1 0 0 0]
[1 0 1 0 0]
[0 1 0 1 0]
[0 0 1 0 1]
[0 0 0 1 0]
>>> M[Integer(2), Integer(2)] = Integer(1)                                  ␣
↪                         # needs sage.modules
Traceback (most recent call last):
...
ValueError: matrix is immutable; please change a copy instead
(i.e., use copy(M) to change a copy of M).
```

Creating a module endomorphism:

```
sage: D12 = posets.DivisorLattice(12).hasse_diagram()
sage: phi = D12.adjacency_matrix(vertices=True); phi
Generic endomorphism of
 Free module generated by {1, 2, 3, 4, 6, 12} over Integer Ring
sage: print(phi._unicode_art_matrix())
    1  2  3  4  6 12
 1⎛ 0  1  1  0  0  0⎞
 2⎜ 0  0  0  1  1  0⎟
 3⎜ 0  0  0  0  1  0⎟
 4⎜ 0  0  0  0  0  1⎟
 6⎜ 0  0  0  0  0  1⎟
```

(continues on next page)

(continued from previous page)

```
12⎛ 0  0  0  0  0  0⎞
```

```
>>> from sage.all import *
>>> D12 = posets.DivisorLattice(Integer(12)).hasse_diagram()
>>> phi = D12.adjacency_matrix(vertices=True); phi
Generic endomorphism of
 Free module generated by {1, 2, 3, 4, 6, 12} over Integer Ring
>>> print(phi._unicode_art_matrix())
    1  2  3  4  6 12
 1⎛ 0  1  1  0  0  0⎞
 2⎜ 0  0  0  1  1  0⎟
 3⎜ 0  0  0  0  1  0⎟
 4⎜ 0  0  0  0  0  1⎟
 6⎜ 0  0  0  0  0  1⎟
12⎝ 0  0  0  0  0  0⎠
```

all_cycles_iterator(*starting_vertices=None*, *simple=False*, *rooted=False*, *max_length=None*, *trivial=False*, *weight_function=None*, *by_weight=False*, *check_weight=True*, *report_weight=False*, *algorithm='A'*)

Return an iterator over all the cycles of `self` starting with one of the given vertices. Each edge must have a positive weight.

The cycles are enumerated in increasing length order. Here, a cycle means a closed walk $v_0e_0v_1e_1\dots v_{k-1}e_{k-1}v_0$ where v_i are vertices and e_i are edges. v_i and e_i $(0 \leq i < k)$ may not be distinct. If `simple=True`, then cycles with distinct v_i and e_i $(0 \leq i < k)$ are enumerated.

INPUT:

- `starting_vertices` – iterable (default: `None`); vertices from which the cycles must start. If `None`, then all vertices of the graph can be starting points. This argument is necessary if `rooted` is set to `True`.
- `simple` – boolean (default: `False`); if set to `True`, then only simple cycles are considered. A cycle is simple if the only vertex occurring twice in it is the starting and ending one and no edge occurs twice.
- `rooted` – boolean (default: `False`); if set to False, then cycles differing only by their starting vertex are considered the same (e.g. `['a', 'b', 'c', 'a']` and `['b', 'c', 'a', 'b']`). Otherwise, all cycles are enumerated.
- `max_length` – nonnegative integer (default: `None`); the maximum length of the enumerated paths. If set to `None`, then all lengths are allowed.
- `trivial` – boolean (default: `False`); if set to `True`, then the empty paths are also enumerated
- `weight_function` – function (default: `None`); a function that takes as input an edge `(u, v, l)` and outputs its weight. If not `None`, `by_weight` is automatically set to `True`. If `None` and `by_weight` is `True`, we use the edge label `l` as a weight.
- `by_weight` – boolean (default: `False`); if `True`, the edges in the graph are weighted, otherwise all edges have weight 1
- `check_weight` – boolean (default: `True`); whether to check that the `weight_function` outputs a number for each edge
- `report_weight` – boolean (default: `False`); if `False`, just a cycle is returned. Otherwise a tuple of cycle length and cycle is returned.
- `algorithm` – string (default: `'A'`); the algorithm used to enumerate the cycles.

- The algorithm `'A'` holds cycle iterators starting with each vertex, and output them in increasing length order.
- The algorithm `'B'` holds cycle iterators starting with each edge, and output them in increasing length order. It depends on the k-shortest simple paths algorithm. Thus, it is not available if `simple=False` or `rooted=True` or `starting_vertices` is not `None`.

OUTPUT: iterator

See also

- *all_simple_cycles()*
- *shortest_simple_paths()*

AUTHOR:

Alexandre Blondin Masse

EXAMPLES:

```
sage: g = DiGraph({'a': ['a', 'b'], 'b': ['c'], 'c': ['d'], 'd': ['c']},␣
↪loops=True)
sage: it = g.all_cycles_iterator()
sage: for _ in range(7): print(next(it))
['a', 'a']
['a', 'a', 'a']
['c', 'd', 'c']
['a', 'a', 'a', 'a']
['a', 'a', 'a', 'a', 'a']
['c', 'd', 'c', 'd', 'c']
['a', 'a', 'a', 'a', 'a', 'a']
```

```
>>> from sage.all import *
>>> g = DiGraph({'a': ['a', 'b'], 'b': ['c'], 'c': ['d'], 'd': ['c']},␣
↪loops=True)
>>> it = g.all_cycles_iterator()
>>> for _ in range(Integer(7)): print(next(it))
['a', 'a']
['a', 'a', 'a']
['c', 'd', 'c']
['a', 'a', 'a', 'a']
['a', 'a', 'a', 'a', 'a']
['c', 'd', 'c', 'd', 'c']
['a', 'a', 'a', 'a', 'a', 'a']
```

There are no cycles in the empty graph and in acyclic graphs:

```
sage: g = DiGraph()
sage: it = g.all_cycles_iterator()
sage: list(it)
[]
sage: g = DiGraph({0:[1]})
sage: it = g.all_cycles_iterator()
```

(continues on next page)

(continued from previous page)

```
sage: list(it)
[]
```

```
>>> from sage.all import *
>>> g = DiGraph()
>>> it = g.all_cycles_iterator()
>>> list(it)
[]
>>> g = DiGraph({Integer(0):[Integer(1)]})
>>> it = g.all_cycles_iterator()
>>> list(it)
[]
```

It is possible to restrict the starting vertices of the cycles:

```
sage: g = DiGraph({'a': ['a', 'b'], 'b': ['c'], 'c': ['d'], 'd': ['c']},␣
↪loops=True)
sage: it = g.all_cycles_iterator(starting_vertices=['b', 'c'])
sage: for _ in range(3): print(next(it))
['c', 'd', 'c']
['c', 'd', 'c', 'd', 'c']
['c', 'd', 'c', 'd', 'c', 'd', 'c']
```

```
>>> from sage.all import *
>>> g = DiGraph({'a': ['a', 'b'], 'b': ['c'], 'c': ['d'], 'd': ['c']},␣
↪loops=True)
>>> it = g.all_cycles_iterator(starting_vertices=['b', 'c'])
>>> for _ in range(Integer(3)): print(next(it))
['c', 'd', 'c']
['c', 'd', 'c', 'd', 'c']
['c', 'd', 'c', 'd', 'c', 'd', 'c']
```

Also, one can bound the length of the cycles:

```
sage: it = g.all_cycles_iterator(max_length=3)
sage: list(it)
[['a', 'a'], ['a', 'a', 'a'], ['c', 'd', 'c'],
 ['a', 'a', 'a', 'a']]
```

```
>>> from sage.all import *
>>> it = g.all_cycles_iterator(max_length=Integer(3))
>>> list(it)
[['a', 'a'], ['a', 'a', 'a'], ['c', 'd', 'c'],
 ['a', 'a', 'a', 'a']]
```

By default, cycles differing only by their starting point are not all enumerated, but this may be parametrized:

```
sage: it = g.all_cycles_iterator(max_length=3, rooted=False)
sage: list(it)
[['a', 'a'], ['a', 'a', 'a'], ['c', 'd', 'c'],
 ['a', 'a', 'a', 'a']]
sage: it = g.all_cycles_iterator(max_length=3, rooted=True)
```

(continues on next page)

(continued from previous page)

```
sage: list(it)
[['a', 'a'], ['a', 'a', 'a'], ['c', 'd', 'c'], ['d', 'c', 'd'],
 ['a', 'a', 'a', 'a']]
```

```
>>> from sage.all import *
>>> it = g.all_cycles_iterator(max_length=Integer(3), rooted=False)
>>> list(it)
[['a', 'a'], ['a', 'a', 'a'], ['c', 'd', 'c'],
 ['a', 'a', 'a', 'a']]
>>> it = g.all_cycles_iterator(max_length=Integer(3), rooted=True)
>>> list(it)
[['a', 'a'], ['a', 'a', 'a'], ['c', 'd', 'c'], ['d', 'c', 'd'],
 ['a', 'a', 'a', 'a']]
```

One may prefer to enumerate simple cycles, i.e. cycles such that the only vertex occurring twice in it is the starting and ending one (see also *all_simple_cycles()*):

```
sage: it = g.all_cycles_iterator(simple=True)
sage: list(it)
[['a', 'a'], ['c', 'd', 'c']]
sage: g = digraphs.Circuit(4)
sage: list(g.all_cycles_iterator(simple=True))
[[0, 1, 2, 3, 0]]
```

```
>>> from sage.all import *
>>> it = g.all_cycles_iterator(simple=True)
>>> list(it)
[['a', 'a'], ['c', 'd', 'c']]
>>> g = digraphs.Circuit(Integer(4))
>>> list(g.all_cycles_iterator(simple=True))
[[0, 1, 2, 3, 0]]
```

A cycle is enumerated in increasing length order for a weighted graph:

```
sage: g = DiGraph()
sage: g.add_edges([('a', 'b', 2), ('a', 'e', 2), ('b', 'a', 1), ('b', 'c', 1),
....:              ('c', 'd', 2), ('d', 'b', 1), ('e', 'a', 2)])
sage: it = g.all_cycles_iterator(simple=False, max_length=None,
....:                            by_weight=True, report_weight=True)
sage: for i in range(9): print(next(it))
(3, ['a', 'b', 'a'])
(4, ['a', 'e', 'a'])
(4, ['b', 'c', 'd', 'b'])
(6, ['a', 'b', 'a', 'b', 'a'])
(7, ['a', 'b', 'a', 'e', 'a'])
(7, ['a', 'b', 'c', 'd', 'b', 'a'])
(7, ['a', 'e', 'a', 'b', 'a'])
(8, ['a', 'e', 'a', 'e', 'a'])
(8, ['b', 'c', 'd', 'b', 'c', 'd', 'b'])
```

```
>>> from sage.all import *
>>> g = DiGraph()
```

(continues on next page)

(continued from previous page)

```
>>> g.add_edges([('a', 'b', Integer(2)), ('a', 'e', Integer(2)), ('b', 'a',␣
↪Integer(1)), ('b', 'c', Integer(1)),
...              ('c', 'd', Integer(2)), ('d', 'b', Integer(1)), ('e', 'a',␣
↪Integer(2))])
>>> it = g.all_cycles_iterator(simple=False, max_length=None,
...                            by_weight=True, report_weight=True)
>>> for i in range(Integer(9)): print(next(it))
(3, ['a', 'b', 'a'])
(4, ['a', 'e', 'a'])
(4, ['b', 'c', 'd', 'b'])
(6, ['a', 'b', 'a', 'b', 'a'])
(7, ['a', 'b', 'a', 'e', 'a'])
(7, ['a', 'b', 'c', 'd', 'b', 'a'])
(7, ['a', 'e', 'a', 'b', 'a'])
(8, ['a', 'e', 'a', 'e', 'a'])
(8, ['b', 'c', 'd', 'b', 'c', 'd', 'b'])
```

Each edge must have a positive weight:

```
sage: g = DiGraph()
sage: g.add_edges([('a', 'b', -2), ('b', 'a', 1)])
sage: next(g.all_cycles_iterator(simple=False, max_length=None,
....:                            by_weight=True, report_weight=True))
Traceback (most recent call last):
...
ValueError: negative weight is not allowed
```

```
>>> from sage.all import *
>>> g = DiGraph()
>>> g.add_edges([('a', 'b', -Integer(2)), ('b', 'a', Integer(1))])
>>> next(g.all_cycles_iterator(simple=False, max_length=None,
...                            by_weight=True, report_weight=True))
Traceback (most recent call last):
...
ValueError: negative weight is not allowed
```

The algorithm `'B'` works for undirected graphs as well:

```
sage: g = Graph({0: [1, 2], 1: [0, 2], 2: [0, 1, 3, 5], 3: [2, 4], 4: [3, 5],␣
↪5: [4, 2]})
sage: for cycle in g.all_cycles_iterator(algorithm='B', simple=True, by_
↪weight=True):
....:     print(cycle)
[0, 1, 2, 0]
[2, 3, 4, 5, 2]
```

```
>>> from sage.all import *
>>> g = Graph({Integer(0): [Integer(1), Integer(2)], Integer(1): [Integer(0),␣
↪Integer(2)], Integer(2): [Integer(0), Integer(1), Integer(3), Integer(5)],␣
↪Integer(3): [Integer(2), Integer(4)], Integer(4): [Integer(3), Integer(5)],␣
↪Integer(5): [Integer(4), Integer(2)]})
>>> for cycle in g.all_cycles_iterator(algorithm='B', simple=True, by_
```

(continues on next page)

(continued from previous page)

```
↪weight=True):
...     print(cycle)
[0, 1, 2, 0]
[2, 3, 4, 5, 2]
```

The algorithm `'B'` is unavailable when `simple=False` or `rooted=True` or `starting_vertices is not None`:

```
sage: g = DiGraph()
sage: g.add_edges([('a', 'b', 1), ('b', 'a', 1)])
sage: next(g.all_cycles_iterator(algorithm='B', simple=False))
....:
Traceback (most recent call last):
...
ValueError: The algorithm 'B' is unavailable when simple=False.
sage: next(g.all_cycles_iterator(algorithm='B', simple=True, rooted=True))
Traceback (most recent call last):
...
ValueError: The algorithm 'B' is unavailable when rooted=True.
sage: next(g.all_cycles_iterator(algorithm='B', simple=True, starting_
↪vertices=['a']))
Traceback (most recent call last):
...
ValueError: The algorithm 'B' is unavailable when starting_vertices is not␣
↪None.
```

```
>>> from sage.all import *
>>> g = DiGraph()
>>> g.add_edges([('a', 'b', Integer(1)), ('b', 'a', Integer(1))])
>>> next(g.all_cycles_iterator(algorithm='B', simple=False))
....:
Traceback (most recent call last):
...
ValueError: The algorithm 'B' is unavailable when simple=False.
>>> next(g.all_cycles_iterator(algorithm='B', simple=True, rooted=True))
Traceback (most recent call last):
...
ValueError: The algorithm 'B' is unavailable when rooted=True.
>>> next(g.all_cycles_iterator(algorithm='B', simple=True, starting_vertices=[
↪'a']))
Traceback (most recent call last):
...
ValueError: The algorithm 'B' is unavailable when starting_vertices is not␣
↪None.
```

The algorithm `'A'` works for undirected graphs as well. Specifically, each cycle is enumerated exactly once, meaning a cycle and its reverse are not listed separately:

```
sage: g = Graph({0: [1, 2], 1: [0, 2], 2: [0, 1]})
sage: for cycle in g.all_cycles_iterator(algorithm='A', simple=True):
....:     print(cycle)
[0, 1, 2, 0]
```

```
>>> from sage.all import *
>>> g = Graph({Integer(0): [Integer(1), Integer(2)], Integer(1): [Integer(0),␣
↪Integer(2)], Integer(2): [Integer(0), Integer(1)]})
>>> for cycle in g.all_cycles_iterator(algorithm='A', simple=True):
...     print(cycle)
[0, 1, 2, 0]
```

all_paths(*G*, *start*, *end*, *use_multiedges=False*, *report_edges=False*, *labels=False*)

Return the list of all paths between a pair of vertices.

If `start` is the same vertex as `end`, then `[[start]]` is returned – a list containing the 1-vertex, 0-edge path "`start`".

If `G` has multiple edges, a path will be returned as many times as the product of the multiplicity of the edges along that path depending on the value of the flag `use_multiedges`.

INPUT:

- `start` – a vertex of a graph, where to start
- `end` – a vertex of a graph, where to end
- `use_multiedges` – boolean (default: `False`); this parameter is used only if the graph has multiple edges
  - If `False`, the graph is considered as simple and an edge label is arbitrarily selected for each edge as in *sage.graphs.generic_graph.GenericGraph.to_simple()* if `report_edges` is `True`
  - If `True`, a path will be reported as many times as the edges multiplicities along that path (when `report_edges = False` or `labels = False`), or with all possible combinations of edge labels (when `report_edges = True` and `labels = True`)
- `report_edges` – boolean (default: `False`); whether to report paths as list of vertices (default) or list of edges, if `False` then `labels` parameter is ignored
- `labels` – boolean (default: `False`); if `False`, each edge is simply a pair `(u, v)` of vertices. Otherwise a list of edges along with its edge labels are used to represent the path.

EXAMPLES:

```
sage: eg1 = Graph({0:[1, 2], 1:[4], 2:[3, 4], 4:[5], 5:[6]})
sage: eg1.all_paths(0, 6)
[[0, 1, 4, 5, 6], [0, 2, 4, 5, 6]]
sage: eg2 = graphs.PetersenGraph()
sage: sorted(eg2.all_paths(1, 4))
[[1, 0, 4],
 [1, 0, 5, 7, 2, 3, 4],
 [1, 0, 5, 7, 2, 3, 8, 6, 9, 4],
 [1, 0, 5, 7, 9, 4],
 [1, 0, 5, 7, 9, 6, 8, 3, 4],
 [1, 0, 5, 8, 3, 2, 7, 9, 4],
 [1, 0, 5, 8, 3, 4],
 [1, 0, 5, 8, 6, 9, 4],
 [1, 0, 5, 8, 6, 9, 7, 2, 3, 4],
 [1, 2, 3, 4],
 [1, 2, 3, 8, 5, 0, 4],
 [1, 2, 3, 8, 5, 7, 9, 4],
 [1, 2, 3, 8, 6, 9, 4],
```

(continues on next page)

(continued from previous page)

```
 [1, 2, 3, 8, 6, 9, 7, 5, 0, 4],
 [1, 2, 7, 5, 0, 4],
 [1, 2, 7, 5, 8, 3, 4],
 [1, 2, 7, 5, 8, 6, 9, 4],
 [1, 2, 7, 9, 4],
 [1, 2, 7, 9, 6, 8, 3, 4],
 [1, 2, 7, 9, 6, 8, 5, 0, 4],
 [1, 6, 8, 3, 2, 7, 5, 0, 4],
 [1, 6, 8, 3, 2, 7, 9, 4],
 [1, 6, 8, 3, 4],
 [1, 6, 8, 5, 0, 4],
 [1, 6, 8, 5, 7, 2, 3, 4],
 [1, 6, 8, 5, 7, 9, 4],
 [1, 6, 9, 4],
 [1, 6, 9, 7, 2, 3, 4],
 [1, 6, 9, 7, 2, 3, 8, 5, 0, 4],
 [1, 6, 9, 7, 5, 0, 4],
 [1, 6, 9, 7, 5, 8, 3, 4]]
sage: dg = DiGraph({0:[1, 3], 1:[3], 2:[0, 3]})
sage: sorted(dg.all_paths(0, 3))
[[0, 1, 3], [0, 3]]
sage: ug = dg.to_undirected()
sage: sorted(ug.all_paths(0, 3))
[[0, 1, 3], [0, 2, 3], [0, 3]]

sage: g = Graph([(0, 1), (0, 1), (1, 2), (1, 2)], multiedges=True)
sage: g.all_paths(0, 2, use_multiedges=True)
[[0, 1, 2], [0, 1, 2], [0, 1, 2], [0, 1, 2]]

sage: dg = DiGraph({0:[1, 2, 1], 3:[0, 0]}, multiedges=True)
sage: dg.all_paths(3, 1, use_multiedges=True)
[[3, 0, 1], [3, 0, 1], [3, 0, 1], [3, 0, 1]]

sage: g = Graph([(0, 1, 'a'), (0, 1, 'b'), (1, 2, 'c'), (1, 2, 'd')],␣
↪multiedges=True)
sage: g.all_paths(0, 2, use_multiedges=False)
[[0, 1, 2]]
sage: g.all_paths(0, 2, use_multiedges=True)
[[0, 1, 2], [0, 1, 2], [0, 1, 2], [0, 1, 2]]
sage: g.all_paths(0, 2, use_multiedges=True, report_edges=True)
[[(0, 1), (1, 2)], [(0, 1), (1, 2)], [(0, 1), (1, 2)], [(0, 1), (1, 2)]]
sage: g.all_paths(0, 2, use_multiedges=True, report_edges=True, labels=True)
[((0, 1, 'b'), (1, 2, 'd')),
 ((0, 1, 'b'), (1, 2, 'c')),
 ((0, 1, 'a'), (1, 2, 'd')),
 ((0, 1, 'a'), (1, 2, 'c'))]
sage: g.all_paths(0, 2, use_multiedges=False, report_edges=True, labels=True)
[((0, 1, 'b'), (1, 2, 'd'))]
sage: g.all_paths(0, 2, use_multiedges=False, report_edges=False, labels=True)
[[0, 1, 2]]
sage: g.all_paths(0, 2, use_multiedges=True, report_edges=False, labels=True)
[[0, 1, 2], [0, 1, 2], [0, 1, 2], [0, 1, 2]]
```

```
>>> from sage.all import *
>>> eg1 = Graph({Integer(0):[Integer(1), Integer(2)], Integer(1):[Integer(4)],
↪ Integer(2):[Integer(3), Integer(4)], Integer(4):[Integer(5)],␣
↪Integer(5):[Integer(6)]})
>>> eg1.all_paths(Integer(0), Integer(6))
[[0, 1, 4, 5, 6], [0, 2, 4, 5, 6]]
>>> eg2 = graphs.PetersenGraph()
>>> sorted(eg2.all_paths(Integer(1), Integer(4)))
[[1, 0, 4],
 [1, 0, 5, 7, 2, 3, 4],
 [1, 0, 5, 7, 2, 3, 8, 6, 9, 4],
 [1, 0, 5, 7, 9, 4],
 [1, 0, 5, 7, 9, 6, 8, 3, 4],
 [1, 0, 5, 8, 3, 2, 7, 9, 4],
 [1, 0, 5, 8, 3, 4],
 [1, 0, 5, 8, 6, 9, 4],
 [1, 0, 5, 8, 6, 9, 7, 2, 3, 4],
 [1, 2, 3, 4],
 [1, 2, 3, 8, 5, 0, 4],
 [1, 2, 3, 8, 5, 7, 9, 4],
 [1, 2, 3, 8, 6, 9, 4],
 [1, 2, 3, 8, 6, 9, 7, 5, 0, 4],
 [1, 2, 7, 5, 0, 4],
 [1, 2, 7, 5, 8, 3, 4],
 [1, 2, 7, 5, 8, 6, 9, 4],
 [1, 2, 7, 9, 4],
 [1, 2, 7, 9, 6, 8, 3, 4],
 [1, 2, 7, 9, 6, 8, 5, 0, 4],
 [1, 6, 8, 3, 2, 7, 5, 0, 4],
 [1, 6, 8, 3, 2, 7, 9, 4],
 [1, 6, 8, 3, 4],
 [1, 6, 8, 5, 0, 4],
 [1, 6, 8, 5, 7, 2, 3, 4],
 [1, 6, 8, 5, 7, 9, 4],
 [1, 6, 9, 4],
 [1, 6, 9, 7, 2, 3, 4],
 [1, 6, 9, 7, 2, 3, 8, 5, 0, 4],
 [1, 6, 9, 7, 5, 0, 4],
 [1, 6, 9, 7, 5, 8, 3, 4]]
>>> dg = DiGraph({Integer(0):[Integer(1), Integer(3)],␣
↪Integer(1):[Integer(3)], Integer(2):[Integer(0), Integer(3)]})
>>> sorted(dg.all_paths(Integer(0), Integer(3)))
[[0, 1, 3], [0, 3]]
>>> ug = dg.to_undirected()
>>> sorted(ug.all_paths(Integer(0), Integer(3)))
[[0, 1, 3], [0, 2, 3], [0, 3]]

>>> g = Graph([(Integer(0), Integer(1)), (Integer(0), Integer(1)),␣
↪(Integer(1), Integer(2)), (Integer(1), Integer(2))], multiedges=True)
>>> g.all_paths(Integer(0), Integer(2), use_multiedges=True)
[[0, 1, 2], [0, 1, 2], [0, 1, 2], [0, 1, 2]]

>>> dg = DiGraph({Integer(0):[Integer(1), Integer(2), Integer(1)],␣
```

(continues on next page)

(continued from previous page)

```
↪Integer(3):[Integer(0), Integer(0)]}, multiedges=True)
>>> dg.all_paths(Integer(3), Integer(1), use_multiedges=True)
[[3, 0, 1], [3, 0, 1], [3, 0, 1], [3, 0, 1]]

>>> g = Graph([(Integer(0), Integer(1), 'a'), (Integer(0), Integer(1), 'b'),␣
↪(Integer(1), Integer(2), 'c'), (Integer(1), Integer(2), 'd')],␣
↪multiedges=True)
>>> g.all_paths(Integer(0), Integer(2), use_multiedges=False)
[[0, 1, 2]]
>>> g.all_paths(Integer(0), Integer(2), use_multiedges=True)
[[0, 1, 2], [0, 1, 2], [0, 1, 2], [0, 1, 2]]
>>> g.all_paths(Integer(0), Integer(2), use_multiedges=True, report_
↪edges=True)
[[(0, 1), (1, 2)], [(0, 1), (1, 2)], [(0, 1), (1, 2)], [(0, 1), (1, 2)]]
>>> g.all_paths(Integer(0), Integer(2), use_multiedges=True, report_
↪edges=True, labels=True)
[((0, 1, 'b'), (1, 2, 'd')),
 ((0, 1, 'b'), (1, 2, 'c')),
 ((0, 1, 'a'), (1, 2, 'd')),
 ((0, 1, 'a'), (1, 2, 'c'))]
>>> g.all_paths(Integer(0), Integer(2), use_multiedges=False, report_
↪edges=True, labels=True)
[((0, 1, 'b'), (1, 2, 'd'))]
>>> g.all_paths(Integer(0), Integer(2), use_multiedges=False, report_
↪edges=False, labels=True)
[[0, 1, 2]]
>>> g.all_paths(Integer(0), Integer(2), use_multiedges=True, report_
↪edges=False, labels=True)
[[0, 1, 2], [0, 1, 2], [0, 1, 2], [0, 1, 2]]
```

all_paths_iterator(*starting_vertices=None*, *ending_vertices=None*, *simple=False*, *max_length=None*, *trivial=False*, *use_multiedges=False*, *report_edges=False*, *labels=False*)

Return an iterator over the paths of `self`.

The paths are enumerated in increasing length order.

INPUT:

- `starting_vertices` – iterable (default: `None`); vertices from which the paths must start. If `None`, then all vertices of the graph can be starting points.
- `ending_vertices` – iterable (default: `None`); allowed ending vertices of the paths. If `None`, then all vertices are allowed.
- `simple` – boolean (default: `False`); if set to `True`, then only simple paths are considered. Simple paths are paths in which no two arcs share a head or share a tail, i.e. every vertex in the path is entered at most once and exited at most once.
- `max_length` – nonnegative integer (default: `None`); the maximum length of the enumerated paths. If set to `None`, then all lengths are allowed.
- `trivial` – boolean (default: `False`); if set to `True`, then the empty paths are also enumerated
- `use_multiedges` – boolean (default: `False`); this parameter is used only if the graph has multiple edges

  - If `False`, the graph is considered as simple and an edge label is arbitrarily selected for each edge as in *sage.graphs.generic_graph.GenericGraph.to_simple()* if `report_edges` is `True`
  - If `True`, a path will be reported as many times as the edges multiplicities along that path (when `report_edges = False` or `labels = False`), or with all possible combinations of edge labels (when `report_edges = True` and `labels = True`)

- `report_edges` – boolean (default: `False`); whether to report paths as list of vertices (default) or list of edges, if `False` then `labels` parameter is ignored
- `labels` – boolean (default: `False`); if `False`, each edge is simply a pair `(u, v)` of vertices. Otherwise a list of edges along with its edge labels are used to represent the path.

OUTPUT: iterator

AUTHOR:

Alexandre Blondin Masse

EXAMPLES:

```
sage: G = graphs.CompleteGraph(4)
sage: list(G.all_paths_iterator(starting_vertices=[1], ending_vertices=[3],␣
↪simple=True))
[[1, 3], [1, 0, 3], [1, 2, 3], [1, 0, 2, 3], [1, 2, 0, 3]]
sage: list(G.shortest_simple_paths(1, 3))
[[1, 3], [1, 2, 3], [1, 0, 3], [1, 0, 2, 3], [1, 2, 0, 3]]
sage: pi = G.all_paths_iterator(starting_vertices=[1], ending_vertices=[3])
sage: for _ in range(6):
....:     print(next(pi))
[1, 3]
[1, 0, 3]
[1, 2, 3]
[1, 0, 1, 3]
[1, 0, 2, 3]
[1, 2, 0, 3]

sage: g = DiGraph({'a': ['a', 'b'], 'b': ['c'], 'c': ['d'], 'd': ['c']},␣
↪loops=True)
sage: pi = g.all_paths_iterator(starting_vertices=['a'], ending_vertices=['d
↪'], report_edges=True, simple=True)
sage: list(pi)
[[('a', 'b'), ('b', 'c'), ('c', 'd')]]

sage: g = DiGraph([(0, 1, 'a'), (0, 1, 'b'), (1, 2,'c'), (1, 2,'d')],␣
↪multiedges=True)
sage: pi =  g.all_paths_iterator(starting_vertices=[0], use_multiedges=True)
sage: for _ in range(6):
....:     print(next(pi))
[0, 1]
[0, 1]
[0, 1, 2]
[0, 1, 2]
[0, 1, 2]
[0, 1, 2]
sage: pi =  g.all_paths_iterator(starting_vertices=[0], use_multiedges=True,␣
↪report_edges=True, labels=True)
```

(continues on next page)

(continued from previous page)

```
sage: for _ in range(6):
....:     print(next(pi))
[(0, 1, 'b')]
[(0, 1, 'a')]
[(0, 1, 'b'), (1, 2, 'd')]
[(0, 1, 'b'), (1, 2, 'c')]
[(0, 1, 'a'), (1, 2, 'd')]
[(0, 1, 'a'), (1, 2, 'c')]
sage: list(g.all_paths_iterator(starting_vertices=[0, 1], ending_vertices=[2],
↪ use_multiedges=False, report_edges=True, labels=True, simple=True))
[[(1, 2, 'd')], [(0, 1, 'b'), (1, 2, 'd')]]
sage: list(g.all_paths_iterator(starting_vertices=[0, 1], ending_vertices=[2],
↪ use_multiedges=False, report_edges=False, labels=True))
[[1, 2], [0, 1, 2]]
sage: list(g.all_paths_iterator(use_multiedges=True, report_edges=False,␣
↪labels=True, max_length=1))
[[1, 2], [1, 2], [0, 1], [0, 1]]
sage: list(g.all_paths_iterator(use_multiedges=True, report_edges=True,␣
↪labels=True, max_length=1))
[[(1, 2, 'd')], [(1, 2, 'c')], [(0, 1, 'b')], [(0, 1, 'a')]]

sage: g = DiGraph({'a': ['a', 'b'], 'b': ['c'], 'c': ['d'], 'd': ['c']},␣
↪loops=True)
sage: pi = g.all_paths_iterator()
sage: [len(next(pi)) - 1 for _ in range(7)]
[1, 1, 1, 1, 1, 2, 2]
```

```
>>> from sage.all import *
>>> G = graphs.CompleteGraph(Integer(4))
>>> list(G.all_paths_iterator(starting_vertices=[Integer(1)], ending_
↪vertices=[Integer(3)], simple=True))
[[1, 3], [1, 0, 3], [1, 2, 3], [1, 0, 2, 3], [1, 2, 0, 3]]
>>> list(G.shortest_simple_paths(Integer(1), Integer(3)))
[[1, 3], [1, 2, 3], [1, 0, 3], [1, 0, 2, 3], [1, 2, 0, 3]]
>>> pi = G.all_paths_iterator(starting_vertices=[Integer(1)], ending_
↪vertices=[Integer(3)])
>>> for _ in range(Integer(6)):
...     print(next(pi))
[1, 3]
[1, 0, 3]
[1, 2, 3]
[1, 0, 1, 3]
[1, 0, 2, 3]
[1, 2, 0, 3]

>>> g = DiGraph({'a': ['a', 'b'], 'b': ['c'], 'c': ['d'], 'd': ['c']},␣
↪loops=True)
>>> pi = g.all_paths_iterator(starting_vertices=['a'], ending_vertices=['d'],␣
↪report_edges=True, simple=True)
>>> list(pi)
[[('a', 'b'), ('b', 'c'), ('c', 'd')]]
```

(continues on next page)

(continued from previous page)

```
>>> g = DiGraph([(Integer(0), Integer(1), 'a'), (Integer(0), Integer(1), 'b'),
↪ (Integer(1), Integer(2),'c'), (Integer(1), Integer(2),'d')],␣
↪multiedges=True)
>>> pi =  g.all_paths_iterator(starting_vertices=[Integer(0)], use_
↪multiedges=True)
>>> for _ in range(Integer(6)):
...     print(next(pi))
[0, 1]
[0, 1]
[0, 1, 2]
[0, 1, 2]
[0, 1, 2]
[0, 1, 2]
>>> pi =  g.all_paths_iterator(starting_vertices=[Integer(0)], use_
↪multiedges=True, report_edges=True, labels=True)
>>> for _ in range(Integer(6)):
...     print(next(pi))
[(0, 1, 'b')]
[(0, 1, 'a')]
[(0, 1, 'b'), (1, 2, 'd')]
[(0, 1, 'b'), (1, 2, 'c')]
[(0, 1, 'a'), (1, 2, 'd')]
[(0, 1, 'a'), (1, 2, 'c')]
>>> list(g.all_paths_iterator(starting_vertices=[Integer(0), Integer(1)],␣
↪ending_vertices=[Integer(2)], use_multiedges=False, report_edges=True,␣
↪labels=True, simple=True))
[[(1, 2, 'd')], [(0, 1, 'b'), (1, 2, 'd')]]
>>> list(g.all_paths_iterator(starting_vertices=[Integer(0), Integer(1)],␣
↪ending_vertices=[Integer(2)], use_multiedges=False, report_edges=False,␣
↪labels=True))
[[1, 2], [0, 1, 2]]
>>> list(g.all_paths_iterator(use_multiedges=True, report_edges=False,␣
↪labels=True, max_length=Integer(1)))
[[1, 2], [1, 2], [0, 1], [0, 1]]
>>> list(g.all_paths_iterator(use_multiedges=True, report_edges=True,␣
↪labels=True, max_length=Integer(1)))
[[(1, 2, 'd')], [(1, 2, 'c')], [(0, 1, 'b')], [(0, 1, 'a')]]

>>> g = DiGraph({'a': ['a', 'b'], 'b': ['c'], 'c': ['d'], 'd': ['c']},␣
↪loops=True)
>>> pi = g.all_paths_iterator()
>>> [len(next(pi)) - Integer(1) for _ in range(Integer(7))]
[1, 1, 1, 1, 1, 2, 2]
```

It is possible to precise the allowed starting and/or ending vertices:

```
sage: pi = g.all_paths_iterator(starting_vertices=['a'])
sage: [len(next(pi)) - 1 for _ in range(5)]
[1, 1, 2, 2, 2]
sage: pi = g.all_paths_iterator(starting_vertices=['a'], ending_vertices=['b
↪'])
sage: for _ in range(5):
```

(continues on next page)

(continued from previous page)

```
....:     print(next(pi))
['a', 'b']
['a', 'a', 'b']
['a', 'a', 'a', 'b']
['a', 'a', 'a', 'a', 'b']
['a', 'a', 'a', 'a', 'a', 'b']
```

```
>>> from sage.all import *
>>> pi = g.all_paths_iterator(starting_vertices=['a'])
>>> [len(next(pi)) - Integer(1) for _ in range(Integer(5))]
[1, 1, 2, 2, 2]
>>> pi = g.all_paths_iterator(starting_vertices=['a'], ending_vertices=['b'])
>>> for _ in range(Integer(5)):
...     print(next(pi))
['a', 'b']
['a', 'a', 'b']
['a', 'a', 'a', 'b']
['a', 'a', 'a', 'a', 'b']
['a', 'a', 'a', 'a', 'a', 'b']
```

One may prefer to enumerate only simple paths (see *all_simple_paths()*):

```
sage: pi = g.all_paths_iterator(simple=True)
sage: sorted(list(pi), key=lambda x:(len(x), x))
[['a', 'a'], ['a', 'b'], ['b', 'c'], ['c', 'd'], ['d', 'c'],
    ['a', 'b', 'c'], ['b', 'c', 'd'], ['c', 'd', 'c'], ['d', 'c', 'd'],
    ['a', 'b', 'c', 'd']]
sage: pi = g.all_paths_iterator(simple=True)
sage: [len(p) - 1 for p in pi]
[1, 1, 1, 1, 1, 2, 2, 2, 2, 3]
```

```
>>> from sage.all import *
>>> pi = g.all_paths_iterator(simple=True)
>>> sorted(list(pi), key=lambda x:(len(x), x))
[['a', 'a'], ['a', 'b'], ['b', 'c'], ['c', 'd'], ['d', 'c'],
    ['a', 'b', 'c'], ['b', 'c', 'd'], ['c', 'd', 'c'], ['d', 'c', 'd'],
    ['a', 'b', 'c', 'd']]
>>> pi = g.all_paths_iterator(simple=True)
>>> [len(p) - Integer(1) for p in pi]
[1, 1, 1, 1, 1, 2, 2, 2, 2, 3]
```

Or simply bound the length of the enumerated paths:

```
sage: pi = g.all_paths_iterator(starting_vertices=['a'], ending_vertices=['b',
↪ 'c'], max_length=6)
sage: sorted(list(pi), key=lambda x:(len(x), x))
[['a', 'b'], ['a', 'a', 'b'], ['a', 'b', 'c'], ['a', 'a', 'a', 'b'],
    ['a', 'a', 'b', 'c'], ['a', 'a', 'a', 'a', 'b'],
    ['a', 'a', 'a', 'b', 'c'], ['a', 'b', 'c', 'd', 'c'],
    ['a', 'a', 'a', 'a', 'a', 'b'], ['a', 'a', 'a', 'a', 'b', 'c'],
    ['a', 'a', 'b', 'c', 'd', 'c'],
    ['a', 'a', 'a', 'a', 'a', 'a', 'b'],
```

(continues on next page)

(continued from previous page)

```
    ['a', 'a', 'a', 'a', 'a', 'b', 'c'],
    ['a', 'a', 'a', 'b', 'c', 'd', 'c'],
    ['a', 'b', 'c', 'd', 'c', 'd', 'c']]
sage: pi = g.all_paths_iterator(starting_vertices=['a'], ending_vertices=['b',
↪ 'c'], max_length=6)
sage: [len(p) - 1 for p in pi]
[1, 2, 2, 3, 3, 4, 4, 4, 5, 5, 5, 6, 6, 6, 6]
```

```
>>> from sage.all import *
>>> pi = g.all_paths_iterator(starting_vertices=['a'], ending_vertices=['b',
↪'c'], max_length=Integer(6))
>>> sorted(list(pi), key=lambda x:(len(x), x))
[['a', 'b'], ['a', 'a', 'b'], ['a', 'b', 'c'], ['a', 'a', 'a', 'b'],
    ['a', 'a', 'b', 'c'], ['a', 'a', 'a', 'a', 'b'],
    ['a', 'a', 'a', 'b', 'c'], ['a', 'b', 'c', 'd', 'c'],
    ['a', 'a', 'a', 'a', 'a', 'b'], ['a', 'a', 'a', 'a', 'b', 'c'],
    ['a', 'a', 'b', 'c', 'd', 'c'],
    ['a', 'a', 'a', 'a', 'a', 'a', 'b'],
    ['a', 'a', 'a', 'a', 'a', 'b', 'c'],
    ['a', 'a', 'a', 'b', 'c', 'd', 'c'],
    ['a', 'b', 'c', 'd', 'c', 'd', 'c']]
>>> pi = g.all_paths_iterator(starting_vertices=['a'], ending_vertices=['b',
↪'c'], max_length=Integer(6))
>>> [len(p) - Integer(1) for p in pi]
[1, 2, 2, 3, 3, 4, 4, 4, 5, 5, 5, 6, 6, 6, 6]
```

By default, empty paths are not enumerated, but it may be parametrized:

```
sage: pi = g.all_paths_iterator(simple=True, trivial=True)
sage: sorted(list(pi), key=lambda x:(len(x), x))
[['a'], ['b'], ['c'], ['d'], ['a', 'a'], ['a', 'b'], ['b', 'c'],
    ['c', 'd'], ['d', 'c'], ['a', 'b', 'c'], ['b', 'c', 'd'],
    ['c', 'd', 'c'], ['d', 'c', 'd'], ['a', 'b', 'c', 'd']]
sage: pi = g.all_paths_iterator(simple=True, trivial=True)
sage: [len(p) - 1 for p in pi]
[0, 0, 0, 0, 1, 1, 1, 1, 1, 2, 2, 2, 2, 3]
sage: pi = g.all_paths_iterator(simple=True, trivial=False)
sage: sorted(list(pi), key=lambda x:(len(x), x))
[['a', 'a'], ['a', 'b'], ['b', 'c'], ['c', 'd'], ['d', 'c'],
    ['a', 'b', 'c'], ['b', 'c', 'd'], ['c', 'd', 'c'], ['d', 'c', 'd'],
    ['a', 'b', 'c', 'd']]
sage: pi = g.all_paths_iterator(simple=True, trivial=False)
sage: [len(p) - 1 for p in pi]
[1, 1, 1, 1, 1, 2, 2, 2, 2, 3]
```

```
>>> from sage.all import *
>>> pi = g.all_paths_iterator(simple=True, trivial=True)
>>> sorted(list(pi), key=lambda x:(len(x), x))
[['a'], ['b'], ['c'], ['d'], ['a', 'a'], ['a', 'b'], ['b', 'c'],
    ['c', 'd'], ['d', 'c'], ['a', 'b', 'c'], ['b', 'c', 'd'],
    ['c', 'd', 'c'], ['d', 'c', 'd'], ['a', 'b', 'c', 'd']]
>>> pi = g.all_paths_iterator(simple=True, trivial=True)
```

(continues on next page)

(continued from previous page)

```
>>> [len(p) - Integer(1) for p in pi]
[0, 0, 0, 0, 1, 1, 1, 1, 1, 2, 2, 2, 2, 3]
>>> pi = g.all_paths_iterator(simple=True, trivial=False)
>>> sorted(list(pi), key=lambda x:(len(x), x))
[['a', 'a'], ['a', 'b'], ['b', 'c'], ['c', 'd'], ['d', 'c'],
    ['a', 'b', 'c'], ['b', 'c', 'd'], ['c', 'd', 'c'], ['d', 'c', 'd'],
    ['a', 'b', 'c', 'd']]
>>> pi = g.all_paths_iterator(simple=True, trivial=False)
>>> [len(p) - Integer(1) for p in pi]
[1, 1, 1, 1, 1, 2, 2, 2, 2, 3]
```

all_simple_cycles(*starting_vertices=None*, *rooted=False*, *max_length=None*, *trivial=False*, *weight_function=None*, *by_weight=False*, *check_weight=True*, *report_weight=False*, *algorithm='B'*)

Return a list of all simple cycles of `self`. The cycles are enumerated in increasing length order. Each edge must have a positive weight.

INPUT:

- `starting_vertices` – iterable (default: `None`); vertices from which the cycles must start. If `None`, then all vertices of the graph can be starting points. This argument is necessary if `rooted` is set to `True`.
- `rooted` – boolean (default: `False`); if set to False, then cycles differing only by their starting vertex are considered the same (e.g. `['a', 'b', 'c', 'a']` and `['b', 'c', 'a', 'b']`). Otherwise, all cycles are enumerated.
- `max_length` – nonnegative integer (default: `None`); the maximum length of the enumerated paths. If set to `None`, then all lengths are allowed.
- `trivial` – boolean (default: `False`); if set to `True`, then the empty paths are also enumerated
- `weight_function` – function (default: `None`); a function that takes as input an edge `(u, v, l)` and outputs its weight. If not `None`, `by_weight` is automatically set to `True`. If `None` and `by_weight` is `True`, we use the edge label `l` as a weight.
- `by_weight` – boolean (default: `False`); if `True`, the edges in the graph are weighted, otherwise all edges have weight 1
- `check_weight` – boolean (default: `True`); whether to check that the `weight_function` outputs a number for each edge
- `report_weight` – boolean (default: `False`); if `False`, just a cycle is returned. Otherwise a tuple of cycle length and cycle is returned.
- `algorithm` – string (default: `'B'`); the algorithm used to enumerate the cycles.
  - The algorithm `'A'` holds cycle iterators starting with each vertex, and output them in increasing length order.
  - The algorithm `'B'` holds cycle iterators starting with each edge, and output them in increasing length order.

OUTPUT: list

Note

Although the number of simple cycles of a finite graph is always finite, computing all its cycles may take a very long time.

EXAMPLES:

```
sage: g = DiGraph({'a': ['a', 'b'], 'b': ['c'], 'c': ['d'], 'd': ['c']},␣
↪loops=True)
sage: g.all_simple_cycles()
[['a', 'a'], ['c', 'd', 'c']]
```

```
>>> from sage.all import *
>>> g = DiGraph({'a': ['a', 'b'], 'b': ['c'], 'c': ['d'], 'd': ['c']},␣
↪loops=True)
>>> g.all_simple_cycles()
[['a', 'a'], ['c', 'd', 'c']]
```

The directed version of the Petersen graph:

```
sage: g = graphs.PetersenGraph().to_directed()
sage: g.all_simple_cycles(max_length=4)
[[0, 1, 0], [0, 4, 0], [0, 5, 0], [1, 2, 1], [1, 6, 1], [2, 3, 2],
 [2, 7, 2], [3, 4, 3], [3, 8, 3], [4, 9, 4], [5, 7, 5], [5, 8, 5],
 [6, 8, 6], [6, 9, 6], [7, 9, 7]]
sage: g.all_simple_cycles(max_length=6)
[[0, 1, 0], [0, 4, 0], [0, 5, 0], [1, 2, 1], [1, 6, 1], [2, 3, 2],
 [2, 7, 2], [3, 4, 3], [3, 8, 3], [4, 9, 4], [5, 7, 5], [5, 8, 5],
 [6, 8, 6], [6, 9, 6], [7, 9, 7], [0, 1, 6, 8, 5, 0],
 [0, 1, 6, 9, 4, 0], [0, 1, 2, 7, 5, 0], [0, 1, 2, 3, 4, 0],
 [0, 4, 9, 7, 5, 0], [0, 4, 9, 6, 1, 0], [0, 4, 3, 8, 5, 0],
 [0, 4, 3, 2, 1, 0], [0, 5, 8, 6, 1, 0], [0, 5, 8, 3, 4, 0],
 [0, 5, 7, 2, 1, 0], [0, 5, 7, 9, 4, 0], [1, 2, 7, 9, 6, 1],
 [1, 2, 3, 8, 6, 1], [1, 6, 9, 7, 2, 1], [1, 6, 8, 3, 2, 1],
 [2, 3, 8, 5, 7, 2], [2, 3, 4, 9, 7, 2], [2, 7, 9, 4, 3, 2],
 [2, 7, 5, 8, 3, 2], [3, 4, 9, 6, 8, 3], [3, 8, 6, 9, 4, 3],
 [5, 7, 9, 6, 8, 5], [5, 8, 6, 9, 7, 5], [0, 1, 2, 3, 8, 5, 0],
 [0, 1, 2, 7, 9, 4, 0], [0, 1, 6, 9, 7, 5, 0],
 [0, 1, 6, 8, 3, 4, 0], [0, 4, 3, 2, 7, 5, 0],
 [0, 4, 3, 8, 6, 1, 0], [0, 4, 9, 6, 8, 5, 0],
 [0, 4, 9, 7, 2, 1, 0], [0, 5, 7, 9, 6, 1, 0],
 [0, 5, 7, 2, 3, 4, 0], [0, 5, 8, 3, 2, 1, 0],
 [0, 5, 8, 6, 9, 4, 0], [1, 2, 3, 4, 9, 6, 1],
 [1, 2, 7, 5, 8, 6, 1], [1, 6, 8, 5, 7, 2, 1],
 [1, 6, 9, 4, 3, 2, 1], [2, 3, 8, 6, 9, 7, 2],
 [2, 7, 9, 6, 8, 3, 2], [3, 4, 9, 7, 5, 8, 3],
 [3, 8, 5, 7, 9, 4, 3]]
```

```
>>> from sage.all import *
>>> g = graphs.PetersenGraph().to_directed()
>>> g.all_simple_cycles(max_length=Integer(4))
[[0, 1, 0], [0, 4, 0], [0, 5, 0], [1, 2, 1], [1, 6, 1], [2, 3, 2],
```

(continues on next page)

(continued from previous page)

```
 [2, 7, 2], [3, 4, 3], [3, 8, 3], [4, 9, 4], [5, 7, 5], [5, 8, 5],
 [6, 8, 6], [6, 9, 6], [7, 9, 7]]
>>> g.all_simple_cycles(max_length=Integer(6))
[[0, 1, 0], [0, 4, 0], [0, 5, 0], [1, 2, 1], [1, 6, 1], [2, 3, 2],
 [2, 7, 2], [3, 4, 3], [3, 8, 3], [4, 9, 4], [5, 7, 5], [5, 8, 5],
 [6, 8, 6], [6, 9, 6], [7, 9, 7], [0, 1, 6, 8, 5, 0],
 [0, 1, 6, 9, 4, 0], [0, 1, 2, 7, 5, 0], [0, 1, 2, 3, 4, 0],
 [0, 4, 9, 7, 5, 0], [0, 4, 9, 6, 1, 0], [0, 4, 3, 8, 5, 0],
 [0, 4, 3, 2, 1, 0], [0, 5, 8, 6, 1, 0], [0, 5, 8, 3, 4, 0],
 [0, 5, 7, 2, 1, 0], [0, 5, 7, 9, 4, 0], [1, 2, 7, 9, 6, 1],
 [1, 2, 3, 8, 6, 1], [1, 6, 9, 7, 2, 1], [1, 6, 8, 3, 2, 1],
 [2, 3, 8, 5, 7, 2], [2, 3, 4, 9, 7, 2], [2, 7, 9, 4, 3, 2],
 [2, 7, 5, 8, 3, 2], [3, 4, 9, 6, 8, 3], [3, 8, 6, 9, 4, 3],
 [5, 7, 9, 6, 8, 5], [5, 8, 6, 9, 7, 5], [0, 1, 2, 3, 8, 5, 0],
 [0, 1, 2, 7, 9, 4, 0], [0, 1, 6, 9, 7, 5, 0],
 [0, 1, 6, 8, 3, 4, 0], [0, 4, 3, 2, 7, 5, 0],
 [0, 4, 3, 8, 6, 1, 0], [0, 4, 9, 6, 8, 5, 0],
 [0, 4, 9, 7, 2, 1, 0], [0, 5, 7, 9, 6, 1, 0],
 [0, 5, 7, 2, 3, 4, 0], [0, 5, 8, 3, 2, 1, 0],
 [0, 5, 8, 6, 9, 4, 0], [1, 2, 3, 4, 9, 6, 1],
 [1, 2, 7, 5, 8, 6, 1], [1, 6, 8, 5, 7, 2, 1],
 [1, 6, 9, 4, 3, 2, 1], [2, 3, 8, 6, 9, 7, 2],
 [2, 7, 9, 6, 8, 3, 2], [3, 4, 9, 7, 5, 8, 3],
 [3, 8, 5, 7, 9, 4, 3]]
```

The complete graph (without loops) on 4 vertices:

```
sage: g = graphs.CompleteGraph(4).to_directed()
sage: g.all_simple_cycles()
[[0, 1, 0], [0, 2, 0], [0, 3, 0], [1, 2, 1], [1, 3, 1], [2, 3, 2],
 [0, 1, 3, 0], [0, 1, 2, 0], [0, 2, 3, 0], [0, 2, 1, 0],
 [0, 3, 2, 0], [0, 3, 1, 0], [1, 2, 3, 1], [1, 3, 2, 1],
 [0, 1, 2, 3, 0], [0, 1, 3, 2, 0], [0, 2, 1, 3, 0],
 [0, 2, 3, 1, 0], [0, 3, 1, 2, 0], [0, 3, 2, 1, 0]]
```

```
>>> from sage.all import *
>>> g = graphs.CompleteGraph(Integer(4)).to_directed()
>>> g.all_simple_cycles()
[[0, 1, 0], [0, 2, 0], [0, 3, 0], [1, 2, 1], [1, 3, 1], [2, 3, 2],
 [0, 1, 3, 0], [0, 1, 2, 0], [0, 2, 3, 0], [0, 2, 1, 0],
 [0, 3, 2, 0], [0, 3, 1, 0], [1, 2, 3, 1], [1, 3, 2, 1],
 [0, 1, 2, 3, 0], [0, 1, 3, 2, 0], [0, 2, 1, 3, 0],
 [0, 2, 3, 1, 0], [0, 3, 1, 2, 0], [0, 3, 2, 1, 0]]
```

If the graph contains a large number of cycles, one can bound the length of the cycles, or simply restrict the possible starting vertices of the cycles:

```
sage: g = graphs.CompleteGraph(20).to_directed()
sage: g.all_simple_cycles(max_length=2)
[[0, 1, 0], [0, 2, 0], [0, 3, 0], [0, 4, 0], [0, 5, 0], [0, 6, 0], [0, 7, 0],
 [0, 8, 0], [0, 9, 0], [0, 10, 0], [0, 11, 0], [0, 12, 0], [0, 13, 0], [0, 14,
↪ 0],
```

(continues on next page)

(continued from previous page)

```
 [0, 15, 0], [0, 16, 0], [0, 17, 0], [0, 18, 0], [0, 19, 0], [1, 2, 1], [1, 3,
↪ 1],
 [1, 4, 1], [1, 5, 1], [1, 6, 1], [1, 7, 1], [1, 8, 1], [1, 9, 1], [2, 3, 2],
 [2, 4, 2], [2, 5, 2], [2, 6, 2], [2, 7, 2], [2, 8, 2], [2, 9, 2], [3, 4, 3],
 [3, 5, 3], [3, 6, 3], [3, 7, 3], [3, 8, 3], [3, 9, 3], [4, 5, 4], [4, 6, 4],
 [4, 7, 4], [4, 8, 4], [4, 9, 4], [5, 6, 5], [5, 7, 5], [5, 8, 5], [5, 9, 5],
 [6, 7, 6], [6, 8, 6], [6, 9, 6], [7, 8, 7], [7, 9, 7], [8, 9, 8], [10, 1,␣
↪10],
 [10, 2, 10], [10, 3, 10], [10, 4, 10], [10, 5, 10], [10, 6, 10], [10, 7, 10],
 [10, 8, 10], [10, 9, 10], [11, 1, 11], [11, 2, 11], [11, 3, 11], [11, 4, 11],
 [11, 5, 11], [11, 6, 11], [11, 7, 11], [11, 8, 11], [11, 9, 11], [11, 10,␣
↪11],
 [12, 1, 12], [12, 2, 12], [12, 3, 12], [12, 4, 12], [12, 5, 12], [12, 6, 12],
 [12, 7, 12], [12, 8, 12], [12, 9, 12], [12, 10, 12], [12, 11, 12], [13, 1,␣
↪13],
 [13, 2, 13], [13, 3, 13], [13, 4, 13], [13, 5, 13], [13, 6, 13], [13, 7, 13],
 [13, 8, 13], [13, 9, 13], [13, 10, 13], [13, 11, 13], [13, 12, 13], [14, 1,␣
↪14],
 [14, 2, 14], [14, 3, 14], [14, 4, 14], [14, 5, 14], [14, 6, 14], [14, 7, 14],
 [14, 8, 14], [14, 9, 14], [14, 10, 14], [14, 11, 14], [14, 12, 14], [14, 13,␣
↪14],
 [15, 1, 15], [15, 2, 15], [15, 3, 15], [15, 4, 15], [15, 5, 15], [15, 6, 15],
 [15, 7, 15], [15, 8, 15], [15, 9, 15], [15, 10, 15], [15, 11, 15], [15, 12,␣
↪15],
 [15, 13, 15], [15, 14, 15], [16, 1, 16], [16, 2, 16], [16, 3, 16], [16, 4,␣
↪16],
 [16, 5, 16], [16, 6, 16], [16, 7, 16], [16, 8, 16], [16, 9, 16], [16, 10,␣
↪16],
 [16, 11, 16], [16, 12, 16], [16, 13, 16], [16, 14, 16], [16, 15, 16], [17, 1,
↪ 17],
 [17, 2, 17], [17, 3, 17], [17, 4, 17], [17, 5, 17], [17, 6, 17], [17, 7, 17],
 [17, 8, 17], [17, 9, 17], [17, 10, 17], [17, 11, 17], [17, 12, 17], [17, 13,␣
↪17],
 [17, 14, 17], [17, 15, 17], [17, 16, 17], [18, 1, 18], [18, 2, 18], [18, 3,␣
↪18],
 [18, 4, 18], [18, 5, 18], [18, 6, 18], [18, 7, 18], [18, 8, 18], [18, 9, 18],
 [18, 10, 18], [18, 11, 18], [18, 12, 18], [18, 13, 18], [18, 14, 18], [18,␣
↪15, 18],
 [18, 16, 18], [18, 17, 18], [19, 1, 19], [19, 2, 19], [19, 3, 19], [19, 4,␣
↪19],
 [19, 5, 19], [19, 6, 19], [19, 7, 19], [19, 8, 19], [19, 9, 19], [19, 10,␣
↪19],
 [19, 11, 19], [19, 12, 19], [19, 13, 19], [19, 14, 19], [19, 15, 19], [19,␣
↪16, 19],
 [19, 17, 19], [19, 18, 19]]

sage: g = graphs.CompleteGraph(20).to_directed()
sage: g.all_simple_cycles(max_length=2, starting_vertices=[0], algorithm='A')
[[0, 1, 0], [0, 2, 0], [0, 3, 0], [0, 4, 0], [0, 5, 0],
 [0, 6, 0], [0, 7, 0], [0, 8, 0], [0, 9, 0], [0, 10, 0],
 [0, 11, 0], [0, 12, 0], [0, 13, 0], [0, 14, 0], [0, 15, 0],
 [0, 16, 0], [0, 17, 0], [0, 18, 0], [0, 19, 0]]
```

```
>>> from sage.all import *
>>> g = graphs.CompleteGraph(Integer(20)).to_directed()
>>> g.all_simple_cycles(max_length=Integer(2))
[[0, 1, 0], [0, 2, 0], [0, 3, 0], [0, 4, 0], [0, 5, 0], [0, 6, 0], [0, 7, 0],
 [0, 8, 0], [0, 9, 0], [0, 10, 0], [0, 11, 0], [0, 12, 0], [0, 13, 0], [0, 14,
↪ 0],
 [0, 15, 0], [0, 16, 0], [0, 17, 0], [0, 18, 0], [0, 19, 0], [1, 2, 1], [1, 3,
↪ 1],
 [1, 4, 1], [1, 5, 1], [1, 6, 1], [1, 7, 1], [1, 8, 1], [1, 9, 1], [2, 3, 2],
 [2, 4, 2], [2, 5, 2], [2, 6, 2], [2, 7, 2], [2, 8, 2], [2, 9, 2], [3, 4, 3],
 [3, 5, 3], [3, 6, 3], [3, 7, 3], [3, 8, 3], [3, 9, 3], [4, 5, 4], [4, 6, 4],
 [4, 7, 4], [4, 8, 4], [4, 9, 4], [5, 6, 5], [5, 7, 5], [5, 8, 5], [5, 9, 5],
 [6, 7, 6], [6, 8, 6], [6, 9, 6], [7, 8, 7], [7, 9, 7], [8, 9, 8], [10, 1,␣
↪10],
 [10, 2, 10], [10, 3, 10], [10, 4, 10], [10, 5, 10], [10, 6, 10], [10, 7, 10],
 [10, 8, 10], [10, 9, 10], [11, 1, 11], [11, 2, 11], [11, 3, 11], [11, 4, 11],
 [11, 5, 11], [11, 6, 11], [11, 7, 11], [11, 8, 11], [11, 9, 11], [11, 10,␣
↪11],
 [12, 1, 12], [12, 2, 12], [12, 3, 12], [12, 4, 12], [12, 5, 12], [12, 6, 12],
 [12, 7, 12], [12, 8, 12], [12, 9, 12], [12, 10, 12], [12, 11, 12], [13, 1,␣
↪13],
 [13, 2, 13], [13, 3, 13], [13, 4, 13], [13, 5, 13], [13, 6, 13], [13, 7, 13],
 [13, 8, 13], [13, 9, 13], [13, 10, 13], [13, 11, 13], [13, 12, 13], [14, 1,␣
↪14],
 [14, 2, 14], [14, 3, 14], [14, 4, 14], [14, 5, 14], [14, 6, 14], [14, 7, 14],
 [14, 8, 14], [14, 9, 14], [14, 10, 14], [14, 11, 14], [14, 12, 14], [14, 13,␣
↪14],
 [15, 1, 15], [15, 2, 15], [15, 3, 15], [15, 4, 15], [15, 5, 15], [15, 6, 15],
 [15, 7, 15], [15, 8, 15], [15, 9, 15], [15, 10, 15], [15, 11, 15], [15, 12,␣
↪15],
 [15, 13, 15], [15, 14, 15], [16, 1, 16], [16, 2, 16], [16, 3, 16], [16, 4,␣
↪16],
 [16, 5, 16], [16, 6, 16], [16, 7, 16], [16, 8, 16], [16, 9, 16], [16, 10,␣
↪16],
 [16, 11, 16], [16, 12, 16], [16, 13, 16], [16, 14, 16], [16, 15, 16], [17, 1,
↪ 17],
 [17, 2, 17], [17, 3, 17], [17, 4, 17], [17, 5, 17], [17, 6, 17], [17, 7, 17],
 [17, 8, 17], [17, 9, 17], [17, 10, 17], [17, 11, 17], [17, 12, 17], [17, 13,␣
↪17],
 [17, 14, 17], [17, 15, 17], [17, 16, 17], [18, 1, 18], [18, 2, 18], [18, 3,␣
↪18],
 [18, 4, 18], [18, 5, 18], [18, 6, 18], [18, 7, 18], [18, 8, 18], [18, 9, 18],
 [18, 10, 18], [18, 11, 18], [18, 12, 18], [18, 13, 18], [18, 14, 18], [18,␣
↪15, 18],
 [18, 16, 18], [18, 17, 18], [19, 1, 19], [19, 2, 19], [19, 3, 19], [19, 4,␣
↪19],
 [19, 5, 19], [19, 6, 19], [19, 7, 19], [19, 8, 19], [19, 9, 19], [19, 10,␣
↪19],
 [19, 11, 19], [19, 12, 19], [19, 13, 19], [19, 14, 19], [19, 15, 19], [19,␣
↪16, 19],
 [19, 17, 19], [19, 18, 19]]

>>> g = graphs.CompleteGraph(Integer(20)).to_directed()
```

(continues on next page)

(continued from previous page)

```
>>> g.all_simple_cycles(max_length=Integer(2), starting_vertices=[Integer(0)],
↪ algorithm='A')
[[0, 1, 0], [0, 2, 0], [0, 3, 0], [0, 4, 0], [0, 5, 0],
 [0, 6, 0], [0, 7, 0], [0, 8, 0], [0, 9, 0], [0, 10, 0],
 [0, 11, 0], [0, 12, 0], [0, 13, 0], [0, 14, 0], [0, 15, 0],
 [0, 16, 0], [0, 17, 0], [0, 18, 0], [0, 19, 0]]
```

One may prefer to distinguish equivalent cycles having distinct starting vertices (compare the following examples):

```
sage: g = graphs.CompleteGraph(4).to_directed()
sage: g.all_simple_cycles(max_length=2, rooted=False, algorithm='A')
[[0, 1, 0], [0, 2, 0], [0, 3, 0], [1, 2, 1], [1, 3, 1], [2, 3, 2]]
sage: g.all_simple_cycles(max_length=2, rooted=True, algorithm='A')
[[0, 1, 0], [0, 2, 0], [0, 3, 0], [1, 0, 1], [1, 2, 1], [1, 3, 1],
 [2, 0, 2], [2, 1, 2], [2, 3, 2], [3, 0, 3], [3, 1, 3], [3, 2, 3]]
```

```
>>> from sage.all import *
>>> g = graphs.CompleteGraph(Integer(4)).to_directed()
>>> g.all_simple_cycles(max_length=Integer(2), rooted=False, algorithm='A')
[[0, 1, 0], [0, 2, 0], [0, 3, 0], [1, 2, 1], [1, 3, 1], [2, 3, 2]]
>>> g.all_simple_cycles(max_length=Integer(2), rooted=True, algorithm='A')
[[0, 1, 0], [0, 2, 0], [0, 3, 0], [1, 0, 1], [1, 2, 1], [1, 3, 1],
 [2, 0, 2], [2, 1, 2], [2, 3, 2], [3, 0, 3], [3, 1, 3], [3, 2, 3]]
```

A cycle is enumerated in increasing length order for a weighted graph:

```
sage: cycles = g.all_simple_cycles(weight_function=lambda e:e[0]+e[1],
....:                              by_weight=True, report_weight=True)
sage: cycles
[(2.0, [0, 1, 0]), (4.0, [0, 2, 0]), (6.0, [0, 1, 2, 0]), (6.0, [0, 2, 1, 0]),
 (6.0, [0, 3, 0]), (6.0, [1, 2, 1]), (8.0, [0, 1, 3, 0]), (8.0, [0, 3, 1, 0]),
 (8.0, [1, 3, 1]), (10.0, [0, 2, 3, 0]), (10.0, [0, 3, 2, 0]), (10.0, [2, 3,␣
↪2]),
 (12.0, [0, 1, 3, 2, 0]), (12.0, [0, 1, 2, 3, 0]), (12.0, [0, 2, 3, 1, 0]),
 (12.0, [0, 2, 1, 3, 0]), (12.0, [0, 3, 2, 1, 0]), (12.0, [0, 3, 1, 2, 0]),
 (12.0, [1, 2, 3, 1]), (12.0, [1, 3, 2, 1])]
```

```
>>> from sage.all import *
>>> cycles = g.all_simple_cycles(weight_function=lambda␣
↪e:e[Integer(0)]+e[Integer(1)],
...                              by_weight=True, report_weight=True)
>>> cycles
[(2.0, [0, 1, 0]), (4.0, [0, 2, 0]), (6.0, [0, 1, 2, 0]), (6.0, [0, 2, 1, 0]),
 (6.0, [0, 3, 0]), (6.0, [1, 2, 1]), (8.0, [0, 1, 3, 0]), (8.0, [0, 3, 1, 0]),
 (8.0, [1, 3, 1]), (10.0, [0, 2, 3, 0]), (10.0, [0, 3, 2, 0]), (10.0, [2, 3,␣
↪2]),
 (12.0, [0, 1, 3, 2, 0]), (12.0, [0, 1, 2, 3, 0]), (12.0, [0, 2, 3, 1, 0]),
 (12.0, [0, 2, 1, 3, 0]), (12.0, [0, 3, 2, 1, 0]), (12.0, [0, 3, 1, 2, 0]),
 (12.0, [1, 2, 3, 1]), (12.0, [1, 3, 2, 1])]
```

The algorithm `'B'` can be used:

```
sage: cycles_B = g.all_simple_cycles(weight_function=lambda e:e[0]+e[1], by_
↪weight=True,
....:                                report_weight=True, algorithm='B')
sage: cycles_B
[(2.0, [0, 1, 0]), (4.0, [0, 2, 0]), (6.0, [0, 1, 2, 0]), (6.0, [0, 2, 1, 0]),
 (6.0, [0, 3, 0]), (6.0, [1, 2, 1]), (8.0, [0, 1, 3, 0]), (8.0, [0, 3, 1, 0]),
 (8.0, [1, 3, 1]), (10.0, [0, 2, 3, 0]), (10.0, [0, 3, 2, 0]), (10.0, [2, 3,␣
↪2]),
 (12.0, [0, 1, 3, 2, 0]), (12.0, [0, 1, 2, 3, 0]), (12.0, [0, 2, 3, 1, 0]),
 (12.0, [0, 2, 1, 3, 0]), (12.0, [0, 3, 2, 1, 0]), (12.0, [0, 3, 1, 2, 0]),
 (12.0, [1, 2, 3, 1]), (12.0, [1, 3, 2, 1])]
sage: cycles.sort() == cycles_B.sort()
True
```

```
>>> from sage.all import *
>>> cycles_B = g.all_simple_cycles(weight_function=lambda␣
↪e:e[Integer(0)]+e[Integer(1)], by_weight=True,
...                                report_weight=True, algorithm='B')
>>> cycles_B
[(2.0, [0, 1, 0]), (4.0, [0, 2, 0]), (6.0, [0, 1, 2, 0]), (6.0, [0, 2, 1, 0]),
 (6.0, [0, 3, 0]), (6.0, [1, 2, 1]), (8.0, [0, 1, 3, 0]), (8.0, [0, 3, 1, 0]),
 (8.0, [1, 3, 1]), (10.0, [0, 2, 3, 0]), (10.0, [0, 3, 2, 0]), (10.0, [2, 3,␣
↪2]),
 (12.0, [0, 1, 3, 2, 0]), (12.0, [0, 1, 2, 3, 0]), (12.0, [0, 2, 3, 1, 0]),
 (12.0, [0, 2, 1, 3, 0]), (12.0, [0, 3, 2, 1, 0]), (12.0, [0, 3, 1, 2, 0]),
 (12.0, [1, 2, 3, 1]), (12.0, [1, 3, 2, 1])]
>>> cycles.sort() == cycles_B.sort()
True
```

The algorithm `'A'` is available for undirected graphs. Specifically, each cycle is enumerated exactly once, meaning a cycle and its reverse are not listed separately:

```
sage: g = Graph({0: [1, 2], 1: [0, 2], 2: [0, 1]})
sage: g.all_simple_cycles(algorithm='A')
[[0, 1, 2, 0]]
```

```
>>> from sage.all import *
>>> g = Graph({Integer(0): [Integer(1), Integer(2)], Integer(1): [Integer(0),␣
↪Integer(2)], Integer(2): [Integer(0), Integer(1)]})
>>> g.all_simple_cycles(algorithm='A')
[[0, 1, 2, 0]]
```

all_simple_paths(*starting_vertices=None*, *ending_vertices=None*, *max_length=None*, *trivial=False*, *use_multiedges=False*, *report_edges=False*, *labels=False*)

Return a list of all the simple paths of `self` starting with one of the given vertices.

Simple paths are paths in which no two arcs share a head or share a tail, i.e. every vertex in the path is entered at most once and exited at most once.

INPUT:

- `starting_vertices` – list (default: `None`); vertices from which the paths must start. If `None`, then all vertices of the graph can be starting points.

- `ending_vertices` – iterable (default: `None`); allowed ending vertices of the paths. If `None`, then all vertices are allowed.
- `max_length` – nonnegative integer (default: `None`); the maximum length of the enumerated paths. If set to `None`, then all lengths are allowed.
- `trivial` – boolean (default: `False`); if set to `True`, then the empty paths are also enumerated
- `use_multiedges` – boolean (default: `False`); this parameter is used only if the graph has multiple edges
  - If `False`, the graph is considered as simple and an edge label is arbitrarily selected for each edge as in *sage.graphs.generic_graph.GenericGraph.to_simple()* if `report_edges` is `True`
  - If `True`, a path will be reported as many times as the edges multiplicities along that path (when `report_edges = False` or `labels = False`), or with all possible combinations of edge labels (when `report_edges = True` and `labels = True`)
- `report_edges` – boolean (default: `False`); whether to report paths as list of vertices (default) or list of edges, if `False` then `labels` parameter is ignored
- `labels` – boolean (default: `False`); if `False`, each edge is simply a pair `(u, v)` of vertices. Otherwise a list of edges along with its edge labels are used to represent the path.

OUTPUT: list

Note

Although the number of simple paths of a finite graph is always finite, computing all its paths may take a very long time.

EXAMPLES:

```
sage: G = graphs.CompleteGraph(4)
sage: G.all_simple_paths([1], [3])
[[1, 3], [1, 0, 3], [1, 2, 3], [1, 0, 2, 3], [1, 2, 0, 3]]
sage: list(G.shortest_simple_paths(1, 3))
[[1, 3], [1, 2, 3], [1, 0, 3], [1, 0, 2, 3], [1, 2, 0, 3]]
sage: G.all_simple_paths([0, 1], [2, 3])
[[1, 2],
 [1, 3],
 [0, 2],
 [0, 3],
 [0, 1, 2],
 [0, 1, 3],
 [0, 2, 3],
 [0, 3, 2],
 [1, 0, 2],
 [1, 0, 3],
 [1, 2, 3],
 [1, 3, 2],
 [1, 0, 2, 3],
 [1, 0, 3, 2],
 [1, 2, 0, 3],
 [1, 3, 0, 2],
 [0, 1, 2, 3],
```

(continues on next page)

(continued from previous page)

```
 [0, 1, 3, 2],
 [0, 2, 1, 3],
 [0, 3, 1, 2]]

sage: g = DiGraph({0: [0, 1], 1: [2], 2: [3], 3: [2]}, loops=True)
sage: g.all_simple_paths()
[[3, 2],
 [2, 3],
 [1, 2],
 [0, 0],
 [0, 1],
 [0, 1, 2],
 [1, 2, 3],
 [2, 3, 2],
 [3, 2, 3],
 [0, 1, 2, 3]]

sage: g = DiGraph([(0, 1, 'a'), (0, 1, 'b'), (1, 2,'c'), (1, 2,'d')],␣
↪multiedges=True)
sage: g.all_simple_paths(starting_vertices=[0], ending_vertices=[2], use_
↪multiedges=False)
[[0, 1, 2]]
sage: g.all_simple_paths(starting_vertices=[0], ending_vertices=[2], use_
↪multiedges=True)
[[0, 1, 2], [0, 1, 2], [0, 1, 2], [0, 1, 2]]
sage: g.all_simple_paths(starting_vertices=[0], ending_vertices=[2], use_
↪multiedges=True, report_edges=True)
[[(0, 1), (1, 2)], [(0, 1), (1, 2)], [(0, 1), (1, 2)], [(0, 1), (1, 2)]]
sage: g.all_simple_paths(starting_vertices=[0], ending_vertices=[2], use_
↪multiedges=True, report_edges=True, labels=True)
[[(0, 1, 'b'), (1, 2, 'd')],
 [(0, 1, 'b'), (1, 2, 'c')],
 [(0, 1, 'a'), (1, 2, 'd')],
 [(0, 1, 'a'), (1, 2, 'c')]]
sage: g.all_simple_paths(starting_vertices=[0, 1], ending_vertices=[2], use_
↪multiedges=False, report_edges=True, labels=True)
[[(1, 2, 'd')], [(0, 1, 'b'), (1, 2, 'd')]]
sage: g.all_simple_paths(starting_vertices=[0, 1], ending_vertices=[2], use_
↪multiedges=False, report_edges=False, labels=True)
[[1, 2], [0, 1, 2]]
sage: g.all_simple_paths(use_multiedges=True, report_edges=False, labels=True)
[[1, 2], [1, 2], [0, 1], [0, 1], [0, 1, 2], [0, 1, 2], [0, 1, 2], [0, 1, 2]]
sage: g.all_simple_paths(starting_vertices=[0, 1], ending_vertices=[2], use_
↪multiedges=False, report_edges=True, labels=True, trivial=True)
[[(1, 2, 'd')], [(0, 1, 'b'), (1, 2, 'd')]]
```

```
>>> from sage.all import *
>>> G = graphs.CompleteGraph(Integer(4))
>>> G.all_simple_paths([Integer(1)], [Integer(3)])
[[1, 3], [1, 0, 3], [1, 2, 3], [1, 0, 2, 3], [1, 2, 0, 3]]
>>> list(G.shortest_simple_paths(Integer(1), Integer(3)))
[[1, 3], [1, 2, 3], [1, 0, 3], [1, 0, 2, 3], [1, 2, 0, 3]]
```

(continues on next page)

(continued from previous page)

```
>>> G.all_simple_paths([Integer(0), Integer(1)], [Integer(2), Integer(3)])
[[1, 2],
 [1, 3],
 [0, 2],
 [0, 3],
 [0, 1, 2],
 [0, 1, 3],
 [0, 2, 3],
 [0, 3, 2],
 [1, 0, 2],
 [1, 0, 3],
 [1, 2, 3],
 [1, 3, 2],
 [1, 0, 2, 3],
 [1, 0, 3, 2],
 [1, 2, 0, 3],
 [1, 3, 0, 2],
 [0, 1, 2, 3],
 [0, 1, 3, 2],
 [0, 2, 1, 3],
 [0, 3, 1, 2]]

>>> g = DiGraph({Integer(0): [Integer(0), Integer(1)], Integer(1):␣
↪[Integer(2)], Integer(2): [Integer(3)], Integer(3): [Integer(2)]},␣
↪loops=True)
>>> g.all_simple_paths()
[[3, 2],
 [2, 3],
 [1, 2],
 [0, 0],
 [0, 1],
 [0, 1, 2],
 [1, 2, 3],
 [2, 3, 2],
 [3, 2, 3],
 [0, 1, 2, 3]]

>>> g = DiGraph([(Integer(0), Integer(1), 'a'), (Integer(0), Integer(1), 'b'),
↪ (Integer(1), Integer(2),'c'), (Integer(1), Integer(2),'d')],␣
↪multiedges=True)
>>> g.all_simple_paths(starting_vertices=[Integer(0)], ending_
↪vertices=[Integer(2)], use_multiedges=False)
[[0, 1, 2]]
>>> g.all_simple_paths(starting_vertices=[Integer(0)], ending_
↪vertices=[Integer(2)], use_multiedges=True)
[[0, 1, 2], [0, 1, 2], [0, 1, 2], [0, 1, 2]]
>>> g.all_simple_paths(starting_vertices=[Integer(0)], ending_
↪vertices=[Integer(2)], use_multiedges=True, report_edges=True)
[[(0, 1), (1, 2)], [(0, 1), (1, 2)], [(0, 1), (1, 2)], [(0, 1), (1, 2)]]
>>> g.all_simple_paths(starting_vertices=[Integer(0)], ending_
↪vertices=[Integer(2)], use_multiedges=True, report_edges=True, labels=True)
[[(0, 1, 'b'), (1, 2, 'd')],
```

(continues on next page)

(continued from previous page)

```
 [(0, 1, 'b'), (1, 2, 'c')],
 [(0, 1, 'a'), (1, 2, 'd')],
 [(0, 1, 'a'), (1, 2, 'c')]]
>>> g.all_simple_paths(starting_vertices=[Integer(0), Integer(1)], ending_
↪vertices=[Integer(2)], use_multiedges=False, report_edges=True, labels=True)
[[(1, 2, 'd')], [(0, 1, 'b'), (1, 2, 'd')]]
>>> g.all_simple_paths(starting_vertices=[Integer(0), Integer(1)], ending_
↪vertices=[Integer(2)], use_multiedges=False, report_edges=False,␣
↪labels=True)
[[1, 2], [0, 1, 2]]
>>> g.all_simple_paths(use_multiedges=True, report_edges=False, labels=True)
[[1, 2], [1, 2], [0, 1], [0, 1], [0, 1, 2], [0, 1, 2], [0, 1, 2], [0, 1, 2]]
>>> g.all_simple_paths(starting_vertices=[Integer(0), Integer(1)], ending_
↪vertices=[Integer(2)], use_multiedges=False, report_edges=True, labels=True,
↪ trivial=True)
[[(1, 2, 'd')], [(0, 1, 'b'), (1, 2, 'd')]]
```

One may compute all paths having specific starting and/or ending vertices:

```
sage: g = DiGraph({'a': ['a', 'b'], 'b': ['c'], 'c': ['d'], 'd': ['c']},␣
↪loops=True)
sage: g.all_simple_paths(starting_vertices=['a'])
[['a', 'a'], ['a', 'b'], ['a', 'b', 'c'], ['a', 'b', 'c', 'd']]
sage: g.all_simple_paths(starting_vertices=['a'], ending_vertices=['c'])
[['a', 'b', 'c']]
sage: g.all_simple_paths(starting_vertices=['a'], ending_vertices=['b', 'c'])
[['a', 'b'], ['a', 'b', 'c']]
```

```
>>> from sage.all import *
>>> g = DiGraph({'a': ['a', 'b'], 'b': ['c'], 'c': ['d'], 'd': ['c']},␣
↪loops=True)
>>> g.all_simple_paths(starting_vertices=['a'])
[['a', 'a'], ['a', 'b'], ['a', 'b', 'c'], ['a', 'b', 'c', 'd']]
>>> g.all_simple_paths(starting_vertices=['a'], ending_vertices=['c'])
[['a', 'b', 'c']]
>>> g.all_simple_paths(starting_vertices=['a'], ending_vertices=['b', 'c'])
[['a', 'b'], ['a', 'b', 'c']]
```

It is also possible to bound the length of the paths:

```
sage: g = DiGraph({0: [0, 1], 1: [2], 2: [3], 3: [2]}, loops=True)
sage: g.all_simple_paths(max_length=2)
[[3, 2],
 [2, 3],
 [1, 2],
 [0, 0],
 [0, 1],
 [0, 1, 2],
 [1, 2, 3],
 [2, 3, 2],
 [3, 2, 3]]
```

```
>>> from sage.all import *
>>> g = DiGraph({Integer(0): [Integer(0), Integer(1)], Integer(1):␣
↪[Integer(2)], Integer(2): [Integer(3)], Integer(3): [Integer(2)]},␣
↪loops=True)
>>> g.all_simple_paths(max_length=Integer(2))
[[3, 2],
 [2, 3],
 [1, 2],
 [0, 0],
 [0, 1],
 [0, 1, 2],
 [1, 2, 3],
 [2, 3, 2],
 [3, 2, 3]]
```

By default, empty paths are not enumerated, but this can be parametrized:

```
sage: g = DiGraph({'a': ['a', 'b'], 'b': ['c'], 'c': ['d'], 'd': ['c']},␣
↪loops=True)
sage: g.all_simple_paths(starting_vertices=['a'], trivial=True)
[['a'], ['a', 'a'], ['a', 'b'], ['a', 'b', 'c'],
 ['a', 'b', 'c', 'd']]
sage: g.all_simple_paths(starting_vertices=['a'], trivial=False)
[['a', 'a'], ['a', 'b'], ['a', 'b', 'c'], ['a', 'b', 'c', 'd']]
```

```
>>> from sage.all import *
>>> g = DiGraph({'a': ['a', 'b'], 'b': ['c'], 'c': ['d'], 'd': ['c']},␣
↪loops=True)
>>> g.all_simple_paths(starting_vertices=['a'], trivial=True)
[['a'], ['a', 'a'], ['a', 'b'], ['a', 'b', 'c'],
 ['a', 'b', 'c', 'd']]
>>> g.all_simple_paths(starting_vertices=['a'], trivial=False)
[['a', 'a'], ['a', 'b'], ['a', 'b', 'c'], ['a', 'b', 'c', 'd']]
```

allow_loops(*new*, *check=True*)

Change whether loops are permitted in the (di)graph.

INPUT:

- `new` – boolean
- `check` – boolean (default: `True`); whether to remove existing loops from the (di)graph when the new status is `False`

EXAMPLES:

```
sage: G = Graph(loops=True); G
Looped graph on 0 vertices
sage: G.has_loops()
False
sage: G.allows_loops()
True
sage: G.add_edge((0, 0))
sage: G.has_loops()
True
```

(continues on next page)

(continued from previous page)

```
sage: G.loops()
[(0, 0, None)]
sage: G.allow_loops(False); G
Graph on 1 vertex
sage: G.has_loops()
False
sage: G.edges(sort=True)
[]

sage: D = DiGraph(loops=True); D
Looped digraph on 0 vertices
sage: D.has_loops()
False
sage: D.allows_loops()
True
sage: D.add_edge((0, 0))
sage: D.has_loops()
True
sage: D.loops()
[(0, 0, None)]
sage: D.allow_loops(False); D
Digraph on 1 vertex
sage: D.has_loops()
False
sage: D.edges(sort=True)
[]
```

```
>>> from sage.all import *
>>> G = Graph(loops=True); G
Looped graph on 0 vertices
>>> G.has_loops()
False
>>> G.allows_loops()
True
>>> G.add_edge((Integer(0), Integer(0)))
>>> G.has_loops()
True
>>> G.loops()
[(0, 0, None)]
>>> G.allow_loops(False); G
Graph on 1 vertex
>>> G.has_loops()
False
>>> G.edges(sort=True)
[]

>>> D = DiGraph(loops=True); D
Looped digraph on 0 vertices
>>> D.has_loops()
False
>>> D.allows_loops()
True
```

(continues on next page)

(continued from previous page)

```
>>> D.add_edge((Integer(0), Integer(0)))
>>> D.has_loops()
True
>>> D.loops()
[(0, 0, None)]
>>> D.allow_loops(False); D
Digraph on 1 vertex
>>> D.has_loops()
False
>>> D.edges(sort=True)
[]
```

allow_multiple_edges(*new*, *check=True*, *keep_label='any'*)

Change whether multiple edges are permitted in the (di)graph.

INPUT:

- `new` – boolean; if `True`, the new graph will allow multiple edges
- `check` – boolean (default: `True`); if `True` and `new` is `False`, we remove all multiple edges from the graph
- `keep_label` – string (default: `'any'`); used only if `new` is `False` and `check` is `True`. If there are multiple edges with different labels, this variable defines which label should be kept:
  - `'any'` – any label
  - `'min'` – the smallest label
  - `'max'` – the largest label

Warning

`'min'` and `'max'` only works if the labels can be compared. A `TypeError` might be raised when working with non-comparable objects.

EXAMPLES:

The standard behavior with undirected graphs:

```
sage: G = Graph(multiedges=True, sparse=True); G
Multi-graph on 0 vertices
sage: G.has_multiple_edges()
False
sage: G.allows_multiple_edges()
True
sage: G.add_edges([(0, 1, 1), (0, 1, 2), (0, 1, 3)])
sage: G.has_multiple_edges()
True
sage: G.multiple_edges(sort=True)
[(0, 1, 1), (0, 1, 2), (0, 1, 3)]
sage: G.allow_multiple_edges(False); G
Graph on 2 vertices
sage: G.has_multiple_edges()
False
```

(continues on next page)

(continued from previous page)

```
sage: G.edges(sort=True)
[(0, 1, 3)]
```

```
>>> from sage.all import *
>>> G = Graph(multiedges=True, sparse=True); G
Multi-graph on 0 vertices
>>> G.has_multiple_edges()
False
>>> G.allows_multiple_edges()
True
>>> G.add_edges([(Integer(0), Integer(1), Integer(1)), (Integer(0),␣
↪Integer(1), Integer(2)), (Integer(0), Integer(1), Integer(3))])
>>> G.has_multiple_edges()
True
>>> G.multiple_edges(sort=True)
[(0, 1, 1), (0, 1, 2), (0, 1, 3)]
>>> G.allow_multiple_edges(False); G
Graph on 2 vertices
>>> G.has_multiple_edges()
False
>>> G.edges(sort=True)
[(0, 1, 3)]
```

If we ask for the minimum label:

```
sage: G = Graph([(0, 1, 1), (0, 1, 2), (0, 1, 3)], multiedges=True,␣
↪sparse=True)
sage: G.allow_multiple_edges(False, keep_label='min')
sage: G.edges(sort=True)
[(0, 1, 1)]
```

```
>>> from sage.all import *
>>> G = Graph([(Integer(0), Integer(1), Integer(1)), (Integer(0), Integer(1),␣
↪Integer(2)), (Integer(0), Integer(1), Integer(3))], multiedges=True,␣
↪sparse=True)
>>> G.allow_multiple_edges(False, keep_label='min')
>>> G.edges(sort=True)
[(0, 1, 1)]
```

If we ask for the maximum label:

```
sage: G = Graph([(0, 1, 1), (0, 1, 2), (0, 1, 3)], multiedges=True,␣
↪sparse=True)
sage: G.allow_multiple_edges(False, keep_label='max')
sage: G.edges(sort=True)
[(0, 1, 3)]
```

```
>>> from sage.all import *
>>> G = Graph([(Integer(0), Integer(1), Integer(1)), (Integer(0), Integer(1),␣
↪Integer(2)), (Integer(0), Integer(1), Integer(3))], multiedges=True,␣
↪sparse=True)
>>> G.allow_multiple_edges(False, keep_label='max')
```

(continues on next page)

(continued from previous page)

```
>>> G.edges(sort=True)
[(0, 1, 3)]
```

The standard behavior with digraphs:

```
sage: D = DiGraph(multiedges=True, sparse=True); D
Multi-digraph on 0 vertices
sage: D.has_multiple_edges()
False
sage: D.allows_multiple_edges()
True
sage: D.add_edges([(0, 1)] * 3)
sage: D.has_multiple_edges()
True
sage: D.multiple_edges()
[(0, 1, None), (0, 1, None), (0, 1, None)]
sage: D.allow_multiple_edges(False); D
Digraph on 2 vertices
sage: D.has_multiple_edges()
False
sage: D.edges(sort=True)
[(0, 1, None)]
```

```
>>> from sage.all import *
>>> D = DiGraph(multiedges=True, sparse=True); D
Multi-digraph on 0 vertices
>>> D.has_multiple_edges()
False
>>> D.allows_multiple_edges()
True
>>> D.add_edges([(Integer(0), Integer(1))] * Integer(3))
>>> D.has_multiple_edges()
True
>>> D.multiple_edges()
[(0, 1, None), (0, 1, None), (0, 1, None)]
>>> D.allow_multiple_edges(False); D
Digraph on 2 vertices
>>> D.has_multiple_edges()
False
>>> D.edges(sort=True)
[(0, 1, None)]
```

allows_loops()

Return whether loops are permitted in the (di)graph.

EXAMPLES:

```
sage: G = Graph(loops=True); G
Looped graph on 0 vertices
sage: G.has_loops()
False
sage: G.allows_loops()
```

(continues on next page)

(continued from previous page)

```
True
sage: G.add_edge((0, 0))
sage: G.has_loops()
True
sage: G.loops()
[(0, 0, None)]
sage: G.allow_loops(False); G
Graph on 1 vertex
sage: G.has_loops()
False
sage: G.edges(sort=True)
[]

sage: D = DiGraph(loops=True); D
Looped digraph on 0 vertices
sage: D.has_loops()
False
sage: D.allows_loops()
True
sage: D.add_edge((0, 0))
sage: D.has_loops()
True
sage: D.loops()
[(0, 0, None)]
sage: D.allow_loops(False); D
Digraph on 1 vertex
sage: D.has_loops()
False
sage: D.edges(sort=True)
[]
```

```
>>> from sage.all import *
>>> G = Graph(loops=True); G
Looped graph on 0 vertices
>>> G.has_loops()
False
>>> G.allows_loops()
True
>>> G.add_edge((Integer(0), Integer(0)))
>>> G.has_loops()
True
>>> G.loops()
[(0, 0, None)]
>>> G.allow_loops(False); G
Graph on 1 vertex
>>> G.has_loops()
False
>>> G.edges(sort=True)
[]

>>> D = DiGraph(loops=True); D
Looped digraph on 0 vertices
```

(continues on next page)

(continued from previous page)

```
>>> D.has_loops()
False
>>> D.allows_loops()
True
>>> D.add_edge((Integer(0), Integer(0)))
>>> D.has_loops()
True
>>> D.loops()
[(0, 0, None)]
>>> D.allow_loops(False); D
Digraph on 1 vertex
>>> D.has_loops()
False
>>> D.edges(sort=True)
[]
```

allows_multiple_edges()

Return whether multiple edges are permitted in the (di)graph.

EXAMPLES:

```
sage: G = Graph(multiedges=True, sparse=True); G
Multi-graph on 0 vertices
sage: G.has_multiple_edges()
False
sage: G.allows_multiple_edges()
True
sage: G.add_edges([(0, 1)] * 3)
sage: G.has_multiple_edges()
True
sage: G.multiple_edges()
[(0, 1, None), (0, 1, None), (0, 1, None)]
sage: G.allow_multiple_edges(False); G
Graph on 2 vertices
sage: G.has_multiple_edges()
False
sage: G.edges(sort=True)
[(0, 1, None)]

sage: D = DiGraph(multiedges=True, sparse=True); D
Multi-digraph on 0 vertices
sage: D.has_multiple_edges()
False
sage: D.allows_multiple_edges()
True
sage: D.add_edges([(0, 1)] * 3)
sage: D.has_multiple_edges()
True
sage: D.multiple_edges()
[(0, 1, None), (0, 1, None), (0, 1, None)]
sage: D.allow_multiple_edges(False); D
Digraph on 2 vertices
```

(continues on next page)

(continued from previous page)

```
sage: D.has_multiple_edges()
False
sage: D.edges(sort=True)
[(0, 1, None)]
```

```
>>> from sage.all import *
>>> G = Graph(multiedges=True, sparse=True); G
Multi-graph on 0 vertices
>>> G.has_multiple_edges()
False
>>> G.allows_multiple_edges()
True
>>> G.add_edges([(Integer(0), Integer(1))] * Integer(3))
>>> G.has_multiple_edges()
True
>>> G.multiple_edges()
[(0, 1, None), (0, 1, None), (0, 1, None)]
>>> G.allow_multiple_edges(False); G
Graph on 2 vertices
>>> G.has_multiple_edges()
False
>>> G.edges(sort=True)
[(0, 1, None)]

>>> D = DiGraph(multiedges=True, sparse=True); D
Multi-digraph on 0 vertices
>>> D.has_multiple_edges()
False
>>> D.allows_multiple_edges()
True
>>> D.add_edges([(Integer(0), Integer(1))] * Integer(3))
>>> D.has_multiple_edges()
True
>>> D.multiple_edges()
[(0, 1, None), (0, 1, None), (0, 1, None)]
>>> D.allow_multiple_edges(False); D
Digraph on 2 vertices
>>> D.has_multiple_edges()
False
>>> D.edges(sort=True)
[(0, 1, None)]
```

am(*sparse*, *vertices=None*, *base_ring=None*, ***kwds*)

alias of *adjacency_matrix()*.

antisymmetric()

Check whether the graph is antisymmetric.

A graph represents an antisymmetric relation if the existence of a path from a vertex x to a vertex y implies that there is not a path from y to x unless $x = y$.

EXAMPLES:

A directed acyclic graph is antisymmetric:

```
sage: G = digraphs.RandomDirectedGNR(20, 0.5)                                  #␣
↪needs networkx
sage: G.antisymmetric()                                                        #␣
↪needs networkx
True
```

```
>>> from sage.all import *
>>> G = digraphs.RandomDirectedGNR(Integer(20), RealNumber('0.5'))             ␣
↪                    # needs networkx
>>> G.antisymmetric()                                                         #␣
↪needs networkx
True
```

Loops are allowed:

```
sage: G.allow_loops(True)                                                      #␣
↪needs networkx
sage: G.add_edge(0, 0)                                                         #␣
↪needs networkx
sage: G.antisymmetric()                                                        #␣
↪needs networkx
True
```

```
>>> from sage.all import *
>>> G.allow_loops(True)                                                       #␣
↪needs networkx
>>> G.add_edge(Integer(0), Integer(0))                                          ␣
↪               # needs networkx
>>> G.antisymmetric()                                                         #␣
↪needs networkx
True
```

An undirected graph is never antisymmetric unless it is just a union of isolated vertices (with possible loops):

```
sage: graphs.RandomGNP(20, 0.5).antisymmetric()                                #␣
↪needs networkx
False
sage: Graph(3).antisymmetric()
True
sage: Graph([(i, i) for i in range(3)], loops=True).antisymmetric()
True
sage: DiGraph([(i, i) for i in range(3)], loops=True).antisymmetric()
True
```

```
>>> from sage.all import *
>>> graphs.RandomGNP(Integer(20), RealNumber('0.5')).antisymmetric()          ␣
↪                    # needs networkx
False
>>> Graph(Integer(3)).antisymmetric()
True
>>> Graph([(i, i) for i in range(Integer(3))], loops=True).antisymmetric()
True
```

(continues on next page)

(continued from previous page)

```
>>> DiGraph([(i, i) for i in range(Integer(3))], loops=True).antisymmetric()
True
```

automorphism_group(*partition=None*, *verbosity=0*, *edge_labels=False*, *order=False*, *return_group=True*, *orbits=False*, *algorithm=None*)

Return the automorphism group of the graph.

With `partition` this can also return the largest subgroup of the automorphism group of the (di)graph whose orbit partition is finer than the partition given.

INPUT:

- `partition` – default is the unit partition, otherwise computes the subgroup of the full automorphism group respecting the partition.
- `edge_labels` – (default: `False`) whether to allow only permutations respecting edge labels
- `order` – (default: `False`) if `True`, compute the order of the automorphism group
- `return_group` – (default: `True`)
- `orbits` – returns the orbits of the group acting on the vertices of the graph
- `algorithm` – if `algorithm='bliss'`, the automorphism group is computed using the optional package bliss (http://www.tcs.tkk.fi/Software/bliss/index.html). Setting it to `'sage'` uses Sage's implementation. If set to `None` (default), bliss is used when available.

OUTPUT: the order of the output is group, order, orbits. However, there are options to turn each of these on or off.

EXAMPLES:

Graphs:

```
sage: graphs_query = GraphQuery(display_cols=['graph6'],num_vertices=4)
sage: L = graphs_query.get_graphs_list()
sage: graphs_list.show_graphs(L)                                                #␣
↪needs sage.plot
sage: for g in L:
....:     G = g.automorphism_group()
....:     G.order(), G.gens()
(24, ((2,3), (1,2), (0,1)))
(4, ((2,3), (0,1)))
(2, ((1,2),))
(6, ((1,2), (0,1)))
(6, ((2,3), (1,2)))
(8, ((1,2), (0,1)(2,3)))
(2, ((0,1)(2,3),))
(2, ((1,2),))
(8, ((2,3), (0,1), (0,2)(1,3)))
(4, ((2,3), (0,1)))
(24, ((2,3), (1,2), (0,1)))
sage: C = graphs.CubeGraph(4)
sage: G = C.automorphism_group()
sage: M = G.character_table() # random order of rows, thus abs() below
sage: QQ(M.determinant()).abs()
712483534798848
```

(continues on next page)

(continued from previous page)

```
sage: G.order()
384
```

```
>>> from sage.all import *
>>> graphs_query = GraphQuery(display_cols=['graph6'],num_vertices=Integer(4))
>>> L = graphs_query.get_graphs_list()
>>> graphs_list.show_graphs(L)                                                #␣
↪needs sage.plot
>>> for g in L:
...     G = g.automorphism_group()
...     G.order(), G.gens()
(24, ((2,3), (1,2), (0,1)))
(4, ((2,3), (0,1)))
(2, ((1,2),))
(6, ((1,2), (0,1)))
(6, ((2,3), (1,2)))
(8, ((1,2), (0,1)(2,3)))
(2, ((0,1)(2,3),))
(2, ((1,2),))
(8, ((2,3), (0,1), (0,2)(1,3)))
(4, ((2,3), (0,1)))
(24, ((2,3), (1,2), (0,1)))
>>> C = graphs.CubeGraph(Integer(4))
>>> G = C.automorphism_group()
>>> M = G.character_table() # random order of rows, thus abs() below
>>> QQ(M.determinant()).abs()
712483534798848
>>> G.order()
384
```

```
sage: D = graphs.DodecahedralGraph()
sage: G = D.automorphism_group()
sage: A5 = AlternatingGroup(5)
sage: Z2 = CyclicPermutationGroup(2)
sage: H = A5.direct_product(Z2)[0] #see documentation for direct_product to␣
↪explain the [0]
sage: G.is_isomorphic(H)
True
```

```
>>> from sage.all import *
>>> D = graphs.DodecahedralGraph()
>>> G = D.automorphism_group()
>>> A5 = AlternatingGroup(Integer(5))
>>> Z2 = CyclicPermutationGroup(Integer(2))
>>> H = A5.direct_product(Z2)[Integer(0)] #see documentation for direct_
↪product to explain the [0]
>>> G.is_isomorphic(H)
True
```

Multigraphs:

```
sage: G = Graph(multiedges=True,sparse=True)
sage: G.add_edge(('a', 'b'))
sage: G.add_edge(('a', 'b'))
sage: G.add_edge(('a', 'b'))
sage: G.automorphism_group()                                                #␣
↪needs sage.groups
Permutation Group with generators [('a','b')]
```

```
>>> from sage.all import *
>>> G = Graph(multiedges=True,sparse=True)
>>> G.add_edge(('a', 'b'))
>>> G.add_edge(('a', 'b'))
>>> G.add_edge(('a', 'b'))
>>> G.automorphism_group()                                                  #␣
↪needs sage.groups
Permutation Group with generators [('a','b')]
```

Digraphs:

```
sage: D = DiGraph( { 0:[1], 1:[2], 2:[3], 3:[4], 4:[0] } )
sage: D.automorphism_group()                                                #␣
↪needs sage.groups
Permutation Group with generators [(0,1,2,3,4)]
```

```
>>> from sage.all import *
>>> D = DiGraph( { Integer(0):[Integer(1)], Integer(1):[Integer(2)],␣
↪Integer(2):[Integer(3)], Integer(3):[Integer(4)], Integer(4):[Integer(0)] }␣
↪)
>>> D.automorphism_group()                                                  #␣
↪needs sage.groups
Permutation Group with generators [(0,1,2,3,4)]
```

Edge labeled graphs:

```
sage: G = Graph(sparse=True)
sage: G.add_edges( [(0,1,'a'),(1,2,'b'),(2,3,'c'),(3,4,'b'),(4,0,'a')] )
sage: G.automorphism_group(edge_labels=True)                                #␣
↪needs sage.groups
Permutation Group with generators [(1,4)(2,3)]

sage: G.automorphism_group(edge_labels=True, algorithm='bliss')  # optional -␣
↪bliss
Permutation Group with generators [(1,4)(2,3)]

sage: G.automorphism_group(edge_labels=True, algorithm='sage')              #␣
↪needs sage.groups
Permutation Group with generators [(1,4)(2,3)]
```

```
>>> from sage.all import *
>>> G = Graph(sparse=True)
>>> G.add_edges( [(Integer(0),Integer(1),'a'),(Integer(1),Integer(2),'b'),
↪(Integer(2),Integer(3),'c'),(Integer(3),Integer(4),'b'),(Integer(4),
```

(continues on next page)

(continued from previous page)

```
↪Integer(0),'a')] )
>>> G.automorphism_group(edge_labels=True)                                    #␣
↪needs sage.groups
Permutation Group with generators [(1,4)(2,3)]

>>> G.automorphism_group(edge_labels=True, algorithm='bliss')  # optional -␣
↪bliss
Permutation Group with generators [(1,4)(2,3)]

>>> G.automorphism_group(edge_labels=True, algorithm='sage')                  #␣
↪needs sage.groups
Permutation Group with generators [(1,4)(2,3)]
```

```
sage: G = Graph({0 : {1 : 7}})
sage: G.automorphism_group(edge_labels=True)                                  #␣
↪needs sage.groups
Permutation Group with generators [(0,1)]

sage: foo = Graph(sparse=True)
sage: bar = Graph(sparse=True)
sage: foo.add_edges([(0,1,1),(1,2,2), (2,3,3)])
sage: bar.add_edges([(0,1,1),(1,2,2), (2,3,3)])
sage: foo.automorphism_group(edge_labels=True)
Permutation Group with generators [()]
sage: foo.automorphism_group()
Permutation Group with generators [(0,3)(1,2)]
sage: bar.automorphism_group(edge_labels=True)
Permutation Group with generators [()]
```

```
>>> from sage.all import *
>>> G = Graph({Integer(0) : {Integer(1) : Integer(7)}})
>>> G.automorphism_group(edge_labels=True)                                    #␣
↪needs sage.groups
Permutation Group with generators [(0,1)]

>>> foo = Graph(sparse=True)
>>> bar = Graph(sparse=True)
>>> foo.add_edges([(Integer(0),Integer(1),Integer(1)),(Integer(1),Integer(2),
↪Integer(2)), (Integer(2),Integer(3),Integer(3))])
>>> bar.add_edges([(Integer(0),Integer(1),Integer(1)),(Integer(1),Integer(2),
↪Integer(2)), (Integer(2),Integer(3),Integer(3))])
>>> foo.automorphism_group(edge_labels=True)
Permutation Group with generators [()]
>>> foo.automorphism_group()
Permutation Group with generators [(0,3)(1,2)]
>>> bar.automorphism_group(edge_labels=True)
Permutation Group with generators [()]
```

You can also ask for just the order of the group:

```
sage: G = graphs.PetersenGraph()
sage: G.automorphism_group(return_group=False, order=True)                    #␣
```

(continues on next page)

(continued from previous page)

```
↪needs sage.groups
120
```

```
>>> from sage.all import *
>>> G = graphs.PetersenGraph()
>>> G.automorphism_group(return_group=False, order=True)                    #␣
↪needs sage.groups
120
```

Or, just the orbits (note that each graph here is vertex transitive)

```
sage: G = graphs.PetersenGraph()
sage: G.automorphism_group(return_group=False, orbits=True, algorithm='sage')
[[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]]
sage: orb = G.automorphism_group(partition=[[0],list(range(1,10))],
....:                            return_group=False, orbits=True, algorithm=
↪'sage')
sage: sorted([sorted(o) for o in orb], key=len)
[[0], [1, 4, 5], [2, 3, 6, 7, 8, 9]]
sage: C = graphs.CubeGraph(3)
sage: orb = C.automorphism_group(orbits=True, return_group=False, algorithm=
↪'sage')
sage: [sorted(o) for o in orb]
[['000', '001', '010', '011', '100', '101', '110', '111']]
```

```
>>> from sage.all import *
>>> G = graphs.PetersenGraph()
>>> G.automorphism_group(return_group=False, orbits=True, algorithm='sage')
[[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]]
>>> orb = G.automorphism_group(partition=[[Integer(0)],list(range(Integer(1),
↪Integer(10)))],
...                            return_group=False, orbits=True, algorithm=
↪'sage')
>>> sorted([sorted(o) for o in orb], key=len)
[[0], [1, 4, 5], [2, 3, 6, 7, 8, 9]]
>>> C = graphs.CubeGraph(Integer(3))
>>> orb = C.automorphism_group(orbits=True, return_group=False, algorithm=
↪'sage')
>>> [sorted(o) for o in orb]
[['000', '001', '010', '011', '100', '101', '110', '111']]
```

One can also use the faster algorithm for computing the automorphism group of the graph - bliss:

```
sage: # optional - bliss
sage: G = graphs.HallJankoGraph()
sage: A1 = G.automorphism_group()                                            #␣
↪needs sage.groups
sage: A2 = G.automorphism_group(algorithm='bliss')
sage: A1.is_isomorphic(A2)                                                   #␣
↪needs sage.groups
True
```

```
>>> from sage.all import *
>>> # optional - bliss
>>> G = graphs.HallJankoGraph()
>>> A1 = G.automorphism_group()                                    #␣
↪needs sage.groups
>>> A2 = G.automorphism_group(algorithm='bliss')
>>> A1.is_isomorphic(A2)                                           #␣
↪needs sage.groups
True
```

average_degree()

Return the average degree of the graph.

The average degree of a graph $G = (V, E)$ is equal to $\frac{2|E|}{|V|}$.

EXAMPLES:

The average degree of a regular graph is equal to the degree of any vertex:

```
sage: g = graphs.CompleteGraph(5)
sage: g.average_degree() == 4
True
```

```
>>> from sage.all import *
>>> g = graphs.CompleteGraph(Integer(5))
>>> g.average_degree() == Integer(4)
True
```

The average degree of a tree is always strictly less than 2:

```
sage: tree = graphs.RandomTree(20)
sage: tree.average_degree() < 2
True
```

```
>>> from sage.all import *
>>> tree = graphs.RandomTree(Integer(20))
>>> tree.average_degree() < Integer(2)
True
```

For any graph, it is equal to $\frac{2|E|}{|V|}$:

```
sage: g = graphs.RandomGNP(20, .4)
sage: g.average_degree() == 2 * g.size() / g.order()
True
```

```
>>> from sage.all import *
>>> g = graphs.RandomGNP(Integer(20), RealNumber('.4'))
>>> g.average_degree() == Integer(2) * g.size() / g.order()
True
```

average_distance(*by_weight=False*, *algorithm=None*, *weight_function=None*, *check_weight=True*)

Return the average distance between vertices of the graph.

Formally, for a graph G this value is equal to $\frac{1}{n(n-1)} \sum_{u,v \in G} d(u, v)$ where $d(u, v)$ denotes the distance between vertices u and v and n is the number of vertices in G.

For more information on the input variables and more examples, we refer to *wiener_index()* and *shortest_path_all_pairs()*, which have very similar input variables.

INPUT:

- `by_weight` – boolean (default: `False`); if `True`, the edges in the graph are weighted, otherwise all edges have weight 1
- `algorithm` – string (default: `None`); one of the algorithms available for method *wiener_index()*
- `weight_function` – function (default: `None`); a function that takes as input an edge `(u, v, l)` and outputs its weight. If not `None`, `by_weight` is automatically set to `True`. If `None` and `by_weight` is `True`, we use the edge label `l`, if `l` is not `None`, else `1` as a weight.
- `check_weight` – boolean (default: `True`); if `True`, we check that the weight_function outputs a number for each edge

EXAMPLES:

From [GYLL1993]:

```
sage: g=graphs.PathGraph(10)
sage: w=lambda x: (x*(x*x -1)/6)/(x*(x-1)/2)
sage: g.average_distance()==w(10)
True
```

```
>>> from sage.all import *
>>> g=graphs.PathGraph(Integer(10))
>>> w=lambda x: (x*(x*x -Integer(1))/Integer(6))/(x*(x-Integer(1))/Integer(2))
>>> g.average_distance()==w(Integer(10))
True
```

Average distance of a circuit:

```
sage: g = digraphs.Circuit(6)
sage: g.average_distance()
3
```

```
>>> from sage.all import *
>>> g = digraphs.Circuit(Integer(6))
>>> g.average_distance()
3
```

biconnected_components(*G*, *forbidden_vertices=None*)

Return the list of biconnected components.

A biconnected component is a maximal subgraph on two or more vertices that is biconnected, i.e., removing any vertex does not disconnect it.

INPUT:

- `G` – the input graph
- `forbidden_vertices` – list (default: `None`); set of vertices to avoid during the search. This is equivalent to getting the biconnected components of the graph after the removal of the forbidden vertices.

EXAMPLES:

```
sage: from sage.graphs.connectivity import biconnected_components
sage: G = Graph({0: [1, 2], 1: [0, 2], 2: [0, 1, 3], 3: [2]})
sage: sorted(len(b) for b in biconnected_components(G))
[2, 3]
sage: sorted(len(b) for b in biconnected_components(2 * G))
[2, 2, 3, 3]
```

```
>>> from sage.all import *
>>> from sage.graphs.connectivity import biconnected_components
>>> G = Graph({Integer(0): [Integer(1), Integer(2)], Integer(1): [Integer(0),␣
↪Integer(2)], Integer(2): [Integer(0), Integer(1), Integer(3)], Integer(3):␣
↪[Integer(2)]})
>>> sorted(len(b) for b in biconnected_components(G))
[2, 3]
>>> sorted(len(b) for b in biconnected_components(Integer(2) * G))
[2, 2, 3, 3]
```

Check the behavior of parameter `forbidden_vertices`:

```
sage: G = graphs.CycleGraph(5)
sage: len(G.biconnected_components())
1
sage: len(G.biconnected_components(forbidden_vertices=[0]))
3
```

```
>>> from sage.all import *
>>> G = graphs.CycleGraph(Integer(5))
>>> len(G.biconnected_components())
1
>>> len(G.biconnected_components(forbidden_vertices=[Integer(0)]))
3
```

biconnected_components_subgraphs(*G*, *forbidden_vertices=None*, *immutable=None*)

Return a list of biconnected components as graph objects.

A biconnected component is a maximal subgraph on two or more vertices that is biconnected, i.e., removing any vertex does not disconnect it.

INPUT:

- `G` – the input graph
- `forbidden_vertices` – list (default: `None`); set of vertices to avoid during the search. This is equivalent to getting the biconnected components subgraphs of the graph after the removal of the forbidden vertices.
- `immutable` – boolean (default: `None`); whether to create a mutable/immutable graphs. `immutable=None` (default) means that the graph and the subgraphs of its connected components will behave the same way.

EXAMPLES:

```
sage: from sage.graphs.connectivity import biconnected_components_subgraphs
sage: G = Graph({0: [1, 2], 1: [0, 2], 2: [0, 1, 3], 3: [2]})
sage: L = biconnected_components_subgraphs(G)
```

(continues on next page)

(continued from previous page)

```
sage: L
[Subgraph of (): Graph on 2 vertices, Subgraph of (): Graph on 3 vertices]
sage: L[0].edges()
[(2, 3, None)]
sage: L[1].edges()
[(0, 1, None), (0, 2, None), (1, 2, None)]
```

```
>>> from sage.all import *
>>> from sage.graphs.connectivity import biconnected_components_subgraphs
>>> G = Graph({Integer(0): [Integer(1), Integer(2)], Integer(1): [Integer(0),␣
↪Integer(2)], Integer(2): [Integer(0), Integer(1), Integer(3)], Integer(3):␣
↪[Integer(2)]})
>>> L = biconnected_components_subgraphs(G)
>>> L
[Subgraph of (): Graph on 2 vertices, Subgraph of (): Graph on 3 vertices]
>>> L[Integer(0)].edges()
[(2, 3, None)]
>>> L[Integer(1)].edges()
[(0, 1, None), (0, 2, None), (1, 2, None)]
```

Check the behavior of parameter `forbidden_vertices`:

```
sage: G = graphs.CycleGraph(5)
sage: len(G.biconnected_components_subgraphs())
1
sage: len(G.biconnected_components_subgraphs(forbidden_vertices=[0]))
3
```

```
>>> from sage.all import *
>>> G = graphs.CycleGraph(Integer(5))
>>> len(G.biconnected_components_subgraphs())
1
>>> len(G.biconnected_components_subgraphs(forbidden_vertices=[Integer(0)]))
3
```

blocks_and_cut_vertices(*G*, *algorithm='Tarjan_Boost'*, *sort=False*, *key=None*, *forbidden_vertices=None*)

Return the blocks and cut vertices of the graph.

In the case of a digraph, this computation is done on the underlying graph.

A cut vertex is one whose deletion increases the number of connected components. A block is a maximal induced subgraph which itself has no cut vertices. Two distinct blocks cannot overlap in more than a single cut vertex.

INPUT:

- `algorithm` – string (default: `'Tarjan_Boost'`); the algorithm to use among:
  - `'Tarjan_Boost'` – default; Tarjan's algorithm (Boost implementation)
  - `'Tarjan_Sage'` – Tarjan's algorithm (Sage implementation)
- `sort` – boolean (default: `False`); whether to sort vertices inside the components and the list of cut vertices **currently only available for ``'Tarjan_Sage'``**

- `key` – a function (default: `None`); a function that takes a vertex as its one argument and returns a value that can be used for comparisons in the sorting algorithm (we must have `sort=True`)
- `forbidden_vertices` – list (default: `None`); set of vertices to avoid during the search

OUTPUT: `(B, C)`, where `B` is a list of blocks - each is a list of vertices and the blocks are the corresponding induced subgraphs - and `C` is a list of cut vertices.

ALGORITHM:

We implement the algorithm proposed by Tarjan in [Tarjan72]. The original version is recursive. We emulate the recursion using a stack.

See also

- *blocks_and_cuts_tree()*
- *sage.graphs.base.boost_graph.blocks_and_cut_vertices()*
- *is_biconnected()*
- *bridges()*

EXAMPLES:

We construct a trivial example of a graph with one cut vertex:

```
sage: from sage.graphs.connectivity import blocks_and_cut_vertices
sage: rings = graphs.CycleGraph(10)
sage: rings.merge_vertices([0, 5])
sage: blocks_and_cut_vertices(rings)
([[0, 1, 4, 2, 3], [0, 6, 9, 7, 8]], [0])
sage: rings.blocks_and_cut_vertices()
([[0, 1, 4, 2, 3], [0, 6, 9, 7, 8]], [0])
sage: B, C = blocks_and_cut_vertices(rings, algorithm='Tarjan_Sage',␣
↪sort=True)
sage: B, C
([[0, 1, 2, 3, 4], [0, 6, 7, 8, 9]], [0])
sage: B2, C2 = blocks_and_cut_vertices(rings, algorithm='Tarjan_Sage',␣
↪sort=False)
sage: Set(map(Set, B)) == Set(map(Set, B2)) and set(C) == set(C2)
True
```

```
>>> from sage.all import *
>>> from sage.graphs.connectivity import blocks_and_cut_vertices
>>> rings = graphs.CycleGraph(Integer(10))
>>> rings.merge_vertices([Integer(0), Integer(5)])
>>> blocks_and_cut_vertices(rings)
([[0, 1, 4, 2, 3], [0, 6, 9, 7, 8]], [0])
>>> rings.blocks_and_cut_vertices()
([[0, 1, 4, 2, 3], [0, 6, 9, 7, 8]], [0])
>>> B, C = blocks_and_cut_vertices(rings, algorithm='Tarjan_Sage', sort=True)
>>> B, C
([[0, 1, 2, 3, 4], [0, 6, 7, 8, 9]], [0])
>>> B2, C2 = blocks_and_cut_vertices(rings, algorithm='Tarjan_Sage',␣
↪sort=False)
```

(continues on next page)

(continued from previous page)

```
>>> Set(map(Set, B)) == Set(map(Set, B2)) and set(C) == set(C2)
True
```

The Petersen graph is biconnected, hence has no cut vertices:

```
sage: blocks_and_cut_vertices(graphs.PetersenGraph())
([[0, 1, 4, 5, 2, 6, 3, 7, 8, 9]], [])
```

```
>>> from sage.all import *
>>> blocks_and_cut_vertices(graphs.PetersenGraph())
([[0, 1, 4, 5, 2, 6, 3, 7, 8, 9]], [])
```

Decomposing paths to pairs:

```
sage: g = graphs.PathGraph(4) + graphs.PathGraph(5)
sage: blocks_and_cut_vertices(g)
([[2, 3], [1, 2], [0, 1], [7, 8], [6, 7], [5, 6], [4, 5]], [1, 2, 5, 6, 7])
```

```
>>> from sage.all import *
>>> g = graphs.PathGraph(Integer(4)) + graphs.PathGraph(Integer(5))
>>> blocks_and_cut_vertices(g)
([[2, 3], [1, 2], [0, 1], [7, 8], [6, 7], [5, 6], [4, 5]], [1, 2, 5, 6, 7])
```

A disconnected graph:

```
sage: g = Graph({1: {2: 28, 3: 10}, 2: {1: 10, 3: 16}, 4: {}, 5: {6: 3, 7: 10,
↪ 8: 4}})
sage: blocks_and_cut_vertices(g)
([[1, 2, 3], [5, 6], [5, 7], [5, 8], [4]], [5])
```

```
>>> from sage.all import *
>>> g = Graph({Integer(1): {Integer(2): Integer(28), Integer(3): Integer(10)},
↪ Integer(2): {Integer(1): Integer(10), Integer(3): Integer(16)},␣
↪Integer(4): {}, Integer(5): {Integer(6): Integer(3), Integer(7):␣
↪Integer(10), Integer(8): Integer(4)}})
>>> blocks_and_cut_vertices(g)
([[1, 2, 3], [5, 6], [5, 7], [5, 8], [4]], [5])
```

A directed graph with Boost's algorithm (Issue #25994):

```
sage: rings = graphs.CycleGraph(10)
sage: rings.merge_vertices([0, 5])
sage: rings = rings.to_directed()
sage: blocks_and_cut_vertices(rings, algorithm='Tarjan_Boost')
([[0, 1, 4, 2, 3], [0, 6, 9, 7, 8]], [0])
```

```
>>> from sage.all import *
>>> rings = graphs.CycleGraph(Integer(10))
>>> rings.merge_vertices([Integer(0), Integer(5)])
>>> rings = rings.to_directed()
>>> blocks_and_cut_vertices(rings, algorithm='Tarjan_Boost')
([[0, 1, 4, 2, 3], [0, 6, 9, 7, 8]], [0])
```

Check the behavior of parameter `forbidden_vertices`:

```
sage: G = graphs.WindmillGraph(4, 3)
sage: G.blocks_and_cut_vertices(sort=True)
([[0, 1, 2, 3], [0, 4, 5, 6], [0, 7, 8, 9]], [0])
sage: G.blocks_and_cut_vertices(sort=True, algorithm='Tarjan_Boost',␣
↪forbidden_vertices=[0])
([[1, 2, 3], [4, 5, 6], [7, 8, 9]], [])
sage: G.blocks_and_cut_vertices(sort=True, algorithm='Tarjan_Sage', forbidden_
↪vertices=[0])
([[1, 2, 3], [4, 5, 6], [7, 8, 9]], [])
sage: G.blocks_and_cut_vertices(sort=True, algorithm='Tarjan_Boost',␣
↪forbidden_vertices=[1])
([[0, 2, 3], [0, 4, 5, 6], [0, 7, 8, 9]], [0])
sage: G.blocks_and_cut_vertices(sort=True, algorithm='Tarjan_Sage', forbidden_
↪vertices=[1])
([[0, 2, 3], [0, 4, 5, 6], [0, 7, 8, 9]], [0])
sage: G = graphs.PathGraph(3)
sage: G.blocks_and_cut_vertices(sort=True, algorithm='Tarjan_Sage')
([[1, 2], [0, 1]], [1])
sage: G.blocks_and_cut_vertices(sort=True, algorithm='Tarjan_Sage', forbidden_
↪vertices=[0])
([[1, 2]], [])
sage: G.blocks_and_cut_vertices(sort=True, algorithm='Tarjan_Sage', forbidden_
↪vertices=[1])
([[0], [2]], [])
sage: G.blocks_and_cut_vertices(sort=True, algorithm='Tarjan_Boost',␣
↪forbidden_vertices=[1])
([[0], [2]], [])
```

```
>>> from sage.all import *
>>> G = graphs.WindmillGraph(Integer(4), Integer(3))
>>> G.blocks_and_cut_vertices(sort=True)
([[0, 1, 2, 3], [0, 4, 5, 6], [0, 7, 8, 9]], [0])
>>> G.blocks_and_cut_vertices(sort=True, algorithm='Tarjan_Boost', forbidden_
↪vertices=[Integer(0)])
([[1, 2, 3], [4, 5, 6], [7, 8, 9]], [])
>>> G.blocks_and_cut_vertices(sort=True, algorithm='Tarjan_Sage', forbidden_
↪vertices=[Integer(0)])
([[1, 2, 3], [4, 5, 6], [7, 8, 9]], [])
>>> G.blocks_and_cut_vertices(sort=True, algorithm='Tarjan_Boost', forbidden_
↪vertices=[Integer(1)])
([[0, 2, 3], [0, 4, 5, 6], [0, 7, 8, 9]], [0])
>>> G.blocks_and_cut_vertices(sort=True, algorithm='Tarjan_Sage', forbidden_
↪vertices=[Integer(1)])
([[0, 2, 3], [0, 4, 5, 6], [0, 7, 8, 9]], [0])
>>> G = graphs.PathGraph(Integer(3))
>>> G.blocks_and_cut_vertices(sort=True, algorithm='Tarjan_Sage')
([[1, 2], [0, 1]], [1])
>>> G.blocks_and_cut_vertices(sort=True, algorithm='Tarjan_Sage', forbidden_
↪vertices=[Integer(0)])
([[1, 2]], [])
>>> G.blocks_and_cut_vertices(sort=True, algorithm='Tarjan_Sage', forbidden_
```

(continues on next page)

(continued from previous page)

```
↪vertices=[Integer(1)])
([[0], [2]], [])
>>> G.blocks_and_cut_vertices(sort=True, algorithm='Tarjan_Boost', forbidden_
↪vertices=[Integer(1)])
([[0], [2]], [])
```

blocks_and_cuts_tree(*G*, *forbidden_vertices=None*)

Return the blocks-and-cuts tree of `self`.

This new graph has two different kinds of vertices, some representing the blocks (type B) and some other the cut vertices of the graph (type C).

There is an edge between a vertex u of type B and a vertex v of type C if the cut-vertex corresponding to v is in the block corresponding to u.

The resulting graph is a tree, with the additional characteristic property that the distance between two leaves is even. When `self` is not connected, the resulting graph is a forest.

When `self` is biconnected, the tree is reduced to a single node of type B.

We referred to [HarPri] and [Gallai] for blocks and cuts tree.

INPUT:

- `G` – a Sage graph
- `forbidden_vertices` – list (default: `None`); set of vertices to avoid during the search. This is equivalent to getting the blocks and cut tree of a graph in which the forbidden vertices have been removed.

See also

- *blocks_and_cut_vertices()*
- *is_biconnected()*

EXAMPLES:

```
sage: from sage.graphs.connectivity import blocks_and_cuts_tree
sage: T = blocks_and_cuts_tree(graphs.KrackhardtKiteGraph()); T
Graph on 5 vertices
sage: T.is_isomorphic(graphs.PathGraph(5))
True
sage: from sage.graphs.connectivity import blocks_and_cuts_tree
sage: T = graphs.KrackhardtKiteGraph().blocks_and_cuts_tree(); T
Graph on 5 vertices
```

```
>>> from sage.all import *
>>> from sage.graphs.connectivity import blocks_and_cuts_tree
>>> T = blocks_and_cuts_tree(graphs.KrackhardtKiteGraph()); T
Graph on 5 vertices
>>> T.is_isomorphic(graphs.PathGraph(Integer(5)))
True
>>> from sage.graphs.connectivity import blocks_and_cuts_tree
>>> T = graphs.KrackhardtKiteGraph().blocks_and_cuts_tree(); T
Graph on 5 vertices
```

The distance between two leaves is even:

```
sage: T = blocks_and_cuts_tree(graphs.RandomTree(40))
sage: T.is_tree()
True
sage: leaves = [v for v in T if T.degree(v) == 1]
sage: all(T.distance(u,v) % 2 == 0 for u in leaves for v in leaves)
True
```

```
>>> from sage.all import *
>>> T = blocks_and_cuts_tree(graphs.RandomTree(Integer(40)))
>>> T.is_tree()
True
>>> leaves = [v for v in T if T.degree(v) == Integer(1)]
>>> all(T.distance(u,v) % Integer(2) == Integer(0) for u in leaves for v in␣
↪leaves)
True
```

The tree of a biconnected graph has a single vertex, of type B:

```
sage: T = blocks_and_cuts_tree(graphs.PetersenGraph())
sage: T.vertices(sort=True)
[('B', (0, 1, 4, 5, 2, 6, 3, 7, 8, 9))]
```

```
>>> from sage.all import *
>>> T = blocks_and_cuts_tree(graphs.PetersenGraph())
>>> T.vertices(sort=True)
[('B', (0, 1, 4, 5, 2, 6, 3, 7, 8, 9))]
```

Check the behavior of parameter `forbidden_vertices`:

```
sage: G = graphs.CycleGraph(5)
sage: G.blocks_and_cut_vertices()
([[0, 1, 4, 2, 3]], [])
sage: G.blocks_and_cuts_tree()
Graph on 1 vertex
sage: G.blocks_and_cut_vertices(forbidden_vertices=[0])
([[3, 4], [2, 3], [1, 2]], [2, 3])
sage: G.blocks_and_cuts_tree(forbidden_vertices=[0])
Graph on 5 vertices
```

```
>>> from sage.all import *
>>> G = graphs.CycleGraph(Integer(5))
>>> G.blocks_and_cut_vertices()
([[0, 1, 4, 2, 3]], [])
>>> G.blocks_and_cuts_tree()
Graph on 1 vertex
>>> G.blocks_and_cut_vertices(forbidden_vertices=[Integer(0)])
([[3, 4], [2, 3], [1, 2]], [2, 3])
>>> G.blocks_and_cuts_tree(forbidden_vertices=[Integer(0)])
Graph on 5 vertices
```

breadth_first_search(*start*, *ignore_direction=False*, *distance=None*, *neighbors=None*, *report_distance=False*, *edges=False*, *forbidden_vertices=None*)

Return an iterator over the vertices in a breadth-first ordering.

INPUT:

- `start` – vertex or list of vertices from which to start the traversal
- `ignore_direction` – boolean (default: `False`); only applies to directed graphs. If `True`, searches across edges in either direction.
- `distance` – integer (default: `None`); the maximum distance from the `start` nodes to traverse. The `start` nodes are at distance zero from themselves.
- `neighbors` – function (default: `None`); a function that inputs a vertex and return a list of vertices. For an undirected graph, `neighbors` is by default the *`neighbors()`* function. For a digraph, the `neighbors` function defaults to the *`neighbor_out_iterator()`* function of the graph.
- `report_distance` – boolean (default: `False`); if `True`, reports pairs `(vertex, distance)` where `distance` is the distance from the `start` nodes. If `False` only the vertices are reported.
- `edges` – boolean (default: `False`); whether to return the edges of the BFS tree in the order of visit or the vertices (default). Edges are directed in root to leaf orientation of the tree.

  Note that parameters `edges` and `report_distance` cannot be `True` simultaneously.
- `forbidden_vertices` – list (default: `None`); set of vertices to avoid during the search. The start vertex `v` cannot be in this set.

See also

- *`breadth_first_search`* – breadth-first search for fast compiled graphs.
- *`depth_first_search`* – depth-first search for fast compiled graphs.
- *`depth_first_search()`* – depth-first search for generic graphs

EXAMPLES:

```
sage: G = Graph({0: [1], 1: [2], 2: [3], 3: [4], 4: [0]})
sage: list(G.breadth_first_search(0))
[0, 1, 4, 2, 3]
```

```
>>> from sage.all import *
>>> G = Graph({Integer(0): [Integer(1)], Integer(1): [Integer(2)],␣
↪Integer(2): [Integer(3)], Integer(3): [Integer(4)], Integer(4):␣
↪[Integer(0)]})
>>> list(G.breadth_first_search(Integer(0)))
[0, 1, 4, 2, 3]
```

By default, the edge direction of a digraph is respected, but this can be overridden by the `ignore_direction` parameter:

```
sage: D = DiGraph({0: [1, 2, 3], 1: [4, 5], 2: [5], 3: [6], 5: [7], 6: [7],␣
↪7: [0]})
sage: list(D.breadth_first_search(0))
[0, 1, 2, 3, 4, 5, 6, 7]
sage: list(D.breadth_first_search(0, ignore_direction=True))
[0, 1, 2, 3, 7, 4, 5, 6]
```

```
>>> from sage.all import *
>>> D = DiGraph({Integer(0): [Integer(1), Integer(2), Integer(3)],␣
↪Integer(1): [Integer(4), Integer(5)], Integer(2): [Integer(5)], Integer(3):␣
↪[Integer(6)], Integer(5): [Integer(7)], Integer(6): [Integer(7)],␣
↪Integer(7): [Integer(0)]})
>>> list(D.breadth_first_search(Integer(0)))
[0, 1, 2, 3, 4, 5, 6, 7]
>>> list(D.breadth_first_search(Integer(0), ignore_direction=True))
[0, 1, 2, 3, 7, 4, 5, 6]
```

You can specify a maximum distance in which to search. A distance of zero returns the `start` vertices:

```
sage: D = DiGraph({0: [1, 2, 3], 1: [4, 5], 2: [5], 3: [6], 5: [7], 6: [7],␣
↪7: [0]})
sage: list(D.breadth_first_search(0, distance=0))
[0]
sage: list(D.breadth_first_search(0, distance=1))
[0, 1, 2, 3]
```

```
>>> from sage.all import *
>>> D = DiGraph({Integer(0): [Integer(1), Integer(2), Integer(3)],␣
↪Integer(1): [Integer(4), Integer(5)], Integer(2): [Integer(5)], Integer(3):␣
↪[Integer(6)], Integer(5): [Integer(7)], Integer(6): [Integer(7)],␣
↪Integer(7): [Integer(0)]})
>>> list(D.breadth_first_search(Integer(0), distance=Integer(0)))
[0]
>>> list(D.breadth_first_search(Integer(0), distance=Integer(1)))
[0, 1, 2, 3]
```

Multiple starting vertices can be specified in a list:

```
sage: D = DiGraph({0: [1, 2, 3], 1: [4, 5], 2: [5], 3: [6], 5: [7], 6: [7],␣
↪7: [0]})
sage: list(D.breadth_first_search([0]))
[0, 1, 2, 3, 4, 5, 6, 7]
sage: list(D.breadth_first_search([0, 6]))
[0, 6, 1, 2, 3, 7, 4, 5]
sage: list(D.breadth_first_search([0, 6], distance=0))
[0, 6]
sage: list(D.breadth_first_search([0, 6], distance=1))
[0, 6, 1, 2, 3, 7]
sage: list(D.breadth_first_search(6, ignore_direction=True, distance=2))
[6, 3, 7, 0, 5]
```

```
>>> from sage.all import *
>>> D = DiGraph({Integer(0): [Integer(1), Integer(2), Integer(3)],␣
↪Integer(1): [Integer(4), Integer(5)], Integer(2): [Integer(5)], Integer(3):␣
↪[Integer(6)], Integer(5): [Integer(7)], Integer(6): [Integer(7)],␣
↪Integer(7): [Integer(0)]})
>>> list(D.breadth_first_search([Integer(0)]))
[0, 1, 2, 3, 4, 5, 6, 7]
>>> list(D.breadth_first_search([Integer(0), Integer(6)]))
[0, 6, 1, 2, 3, 7, 4, 5]
```

(continues on next page)

(continued from previous page)

```
>>> list(D.breadth_first_search([Integer(0), Integer(6)],␣
↪distance=Integer(0)))
[0, 6]
>>> list(D.breadth_first_search([Integer(0), Integer(6)],␣
↪distance=Integer(1)))
[0, 6, 1, 2, 3, 7]
>>> list(D.breadth_first_search(Integer(6), ignore_direction=True,␣
↪distance=Integer(2)))
[6, 3, 7, 0, 5]
```

More generally, you can specify a `neighbors` function. For example, you can traverse the graph backwards by setting `neighbors` to be the *neighbors_in()* function of the graph:

```
sage: D = DiGraph({0: [1, 2, 3], 1: [4, 5], 2: [5], 3: [6], 5: [7], 6: [7],␣
↪7: [0]})
sage: list(D.breadth_first_search(5, neighbors=D.neighbors_in, distance=2))
[5, 1, 2, 0]
sage: list(D.breadth_first_search(5, neighbors=D.neighbors_out, distance=2))
[5, 7, 0]
sage: list(D.breadth_first_search(5 ,neighbors=D.neighbors, distance=2))
[5, 1, 2, 7, 0, 4, 6]
```

```
>>> from sage.all import *
>>> D = DiGraph({Integer(0): [Integer(1), Integer(2), Integer(3)],␣
↪Integer(1): [Integer(4), Integer(5)], Integer(2): [Integer(5)], Integer(3):␣
↪[Integer(6)], Integer(5): [Integer(7)], Integer(6): [Integer(7)],␣
↪Integer(7): [Integer(0)]})
>>> list(D.breadth_first_search(Integer(5), neighbors=D.neighbors_in,␣
↪distance=Integer(2)))
[5, 1, 2, 0]
>>> list(D.breadth_first_search(Integer(5), neighbors=D.neighbors_out,␣
↪distance=Integer(2)))
[5, 7, 0]
>>> list(D.breadth_first_search(Integer(5) ,neighbors=D.neighbors,␣
↪distance=Integer(2)))
[5, 1, 2, 7, 0, 4, 6]
```

It is possible (Issue #16470) using the keyword `report_distance` to get pairs `(vertex, distance)` encoding the distance from the starting vertices:

```
sage: G = graphs.PetersenGraph()
sage: list(G.breadth_first_search(0, report_distance=True))
[(0, 0), (1, 1), (4, 1), (5, 1), (2, 2), (6, 2), (3, 2), (9, 2),
(7, 2), (8, 2)]
sage: list(G.breadth_first_search(0, report_distance=False))
[0, 1, 4, 5, 2, 6, 3, 9, 7, 8]

sage: D = DiGraph({0: [1, 3], 1: [0, 2], 2: [0, 3], 3: [4]})
sage: D.show()                                                          #␣
↪needs sage.plot
sage: list(D.breadth_first_search(4, neighbors=D.neighbor_in_iterator,
....:                             report_distance=True))
```

(continues on next page)

(continued from previous page)

```
[(4, 0), (3, 1), (0, 2), (2, 2), (1, 3)]

sage: C = graphs.CycleGraph(4)
sage: list(C.breadth_first_search([0, 1], report_distance=True))
[(0, 0), (1, 0), (3, 1), (2, 1)]
```

```
>>> from sage.all import *
>>> G = graphs.PetersenGraph()
>>> list(G.breadth_first_search(Integer(0), report_distance=True))
[(0, 0), (1, 1), (4, 1), (5, 1), (2, 2), (6, 2), (3, 2), (9, 2),
(7, 2), (8, 2)]
>>> list(G.breadth_first_search(Integer(0), report_distance=False))
[0, 1, 4, 5, 2, 6, 3, 9, 7, 8]

>>> D = DiGraph({Integer(0): [Integer(1), Integer(3)], Integer(1):␣
↪[Integer(0), Integer(2)], Integer(2): [Integer(0), Integer(3)], Integer(3):␣
↪[Integer(4)]})
>>> D.show()                                                              #␣
↪needs sage.plot
>>> list(D.breadth_first_search(Integer(4), neighbors=D.neighbor_in_iterator,
...                             report_distance=True))
[(4, 0), (3, 1), (0, 2), (2, 2), (1, 3)]

>>> C = graphs.CycleGraph(Integer(4))
>>> list(C.breadth_first_search([Integer(0), Integer(1)], report_
↪distance=True))
[(0, 0), (1, 0), (3, 1), (2, 1)]
```

You can get edges of the BFS tree instead of the vertices using the `edges` parameter:

```
sage: D = DiGraph({1:[2,3],2:[4],3:[4],4:[1],5:[2,6]})
sage: list(D.breadth_first_search(1, edges=True))
[(1, 2), (1, 3), (2, 4)]
```

```
>>> from sage.all import *
>>> D = DiGraph({Integer(1):[Integer(2),Integer(3)],Integer(2):[Integer(4)],
↪Integer(3):[Integer(4)],Integer(4):[Integer(1)],Integer(5):[Integer(2),
↪Integer(6)]})
>>> list(D.breadth_first_search(Integer(1), edges=True))
[(1, 2), (1, 3), (2, 4)]
```

BFS in a graph with forbidden vertices:

```
sage: G = graphs.PetersenGraph()
sage: list(G.breadth_first_search(0, forbidden_vertices=[1, 2]))
[0, 4, 5, 3, 9, 7, 8, 6]
```

```
>>> from sage.all import *
>>> G = graphs.PetersenGraph()
>>> list(G.breadth_first_search(Integer(0), forbidden_vertices=[Integer(1),␣
↪Integer(2)]))
[0, 4, 5, 3, 9, 7, 8, 6]
```

canonical_label(*partition=None*, *certificate=False*, *edge_labels=False*, *algorithm=None*, *return_graph=True*, *immutable=None*)

Return the canonical graph.

A canonical graph is the representative graph of an isomorphism class by some canonization function c. If G and H are graphs, then $G \cong c(G)$, and $c(G) == c(H)$ if and only if $G \cong H$.

See the Wikipedia article Graph_canonization for more information.

INPUT:

- `partition` – if given, the canonical label with respect to this set partition will be computed. The default is the unit set partition.
- `certificate` – boolean (default: `False`); when set to `True`, a dictionary mapping from the vertices of the (di)graph to its canonical label will also be returned.
- `edge_labels` – boolean (default: `False`); when set to `True`, allows only permutations respecting edge labels
- `algorithm` – string (default: `None`); the algorithm to use. Currently available:
  - `'bliss'`: use the optional package bliss (http://www.tcs.tkk.fi/Software/bliss/index.html);
  - `'sage'`: always use Sage's implementation.
  - `None` (default): use bliss when available and possible

  Note

  Make sure you always compare canonical forms obtained by the same algorithm.

- `return_graph` – boolean (default: `True`); when set to `False`, returns the list of edges of the canonical graph instead of the canonical graph; only available when `'bliss'` is explicitly set as algorithm.
- `immutable` – boolean (default: `None`); whether to create a mutable/immutable (di)graph. `immutable=None` (default) means that the (di)graph and its canonical (di)graph will behave the same way.

EXAMPLES:

Canonization changes isomorphism to equality:

```
sage: g1 = graphs.GridGraph([2,3])
sage: g2 = Graph({1: [2, 4], 3: [2, 6], 5: [4, 2, 6]})
sage: g1 == g2
False
sage: g1.is_isomorphic(g2)
True
sage: g1.canonical_label() == g2.canonical_label()
True
```

```
>>> from sage.all import *
>>> g1 = graphs.GridGraph([Integer(2),Integer(3)])
>>> g2 = Graph({Integer(1): [Integer(2), Integer(4)], Integer(3): [Integer(2),
↪ Integer(6)], Integer(5): [Integer(4), Integer(2), Integer(6)]})
>>> g1 == g2
False
```

(continues on next page)

(continued from previous page)

```
>>> g1.is_isomorphic(g2)
True
>>> g1.canonical_label() == g2.canonical_label()
True
```

We can get the relabeling used for canonization:

```
sage: g, c = g1.canonical_label(algorithm='sage', certificate=True)
sage: g
Grid Graph for [2, 3]: Graph on 6 vertices
sage: c
{(0, 0): 3, (0, 1): 4, (0, 2): 2, (1, 0): 0, (1, 1): 5, (1, 2): 1}
```

```
>>> from sage.all import *
>>> g, c = g1.canonical_label(algorithm='sage', certificate=True)
>>> g
Grid Graph for [2, 3]: Graph on 6 vertices
>>> c
{(0, 0): 3, (0, 1): 4, (0, 2): 2, (1, 0): 0, (1, 1): 5, (1, 2): 1}
```

Multigraphs and directed graphs work too:

```
sage: G = Graph(multiedges=True,sparse=True)
sage: G.add_edge((0,1))
sage: G.add_edge((0,1))
sage: G.add_edge((0,1))
sage: G.canonical_label()
Multi-graph on 2 vertices
sage: Graph('A?').canonical_label()
Graph on 2 vertices

sage: P = graphs.PetersenGraph()
sage: DP = P.to_directed()
sage: DP.canonical_label(algorithm='sage').adjacency_matrix()                #␣
↪needs sage.modules
[0 0 0 0 0 0 0 1 1 1]
[0 0 0 0 1 0 1 0 0 1]
[0 0 0 1 0 0 1 0 1 0]
[0 0 1 0 0 1 0 0 0 1]
[0 1 0 0 0 1 0 0 1 0]
[0 0 0 1 1 0 0 1 0 0]
[0 1 1 0 0 0 0 1 0 0]
[1 0 0 0 0 1 1 0 0 0]
[1 0 1 0 1 0 0 0 0 0]
[1 1 0 1 0 0 0 0 0 0]
```

```
>>> from sage.all import *
>>> G = Graph(multiedges=True,sparse=True)
>>> G.add_edge((Integer(0),Integer(1)))
>>> G.add_edge((Integer(0),Integer(1)))
>>> G.add_edge((Integer(0),Integer(1)))
>>> G.canonical_label()
```

(continues on next page)

(continued from previous page)

```
Multi-graph on 2 vertices
>>> Graph('A?').canonical_label()
Graph on 2 vertices

>>> P = graphs.PetersenGraph()
>>> DP = P.to_directed()
>>> DP.canonical_label(algorithm='sage').adjacency_matrix()                #␣
↪needs sage.modules
[0 0 0 0 0 0 0 1 1 1]
[0 0 0 0 1 0 1 0 0 1]
[0 0 0 1 0 0 1 0 1 0]
[0 0 1 0 0 1 0 0 0 1]
[0 1 0 0 0 1 0 0 1 0]
[0 0 0 1 1 0 0 1 0 0]
[0 1 1 0 0 0 0 1 0 0]
[1 0 0 0 0 1 1 0 0 0]
[1 0 1 0 1 0 0 0 0 0]
[1 1 0 1 0 0 0 0 0 0]
```

Edge labeled graphs:

```
sage: G = Graph(sparse=True)
sage: G.add_edges( [(0,1,'a'),(1,2,'b'),(2,3,'c'),(3,4,'b'),(4,0,'a')] )
sage: G.canonical_label(edge_labels=True)
Graph on 5 vertices
sage: G.canonical_label(edge_labels=True, algorithm='bliss',         #␣
↪optional - bliss
....:                   certificate=True)
(Graph on 5 vertices, {0: 4, 1: 3, 2: 1, 3: 0, 4: 2})

sage: G.canonical_label(edge_labels=True, algorithm='sage',
....:                   certificate=True)
(Graph on 5 vertices, {0: 4, 1: 3, 2: 0, 3: 1, 4: 2})
```

```
>>> from sage.all import *
>>> G = Graph(sparse=True)
>>> G.add_edges( [(Integer(0),Integer(1),'a'),(Integer(1),Integer(2),'b'),
↪(Integer(2),Integer(3),'c'),(Integer(3),Integer(4),'b'),(Integer(4),
↪Integer(0),'a')] )
>>> G.canonical_label(edge_labels=True)
Graph on 5 vertices
>>> G.canonical_label(edge_labels=True, algorithm='bliss',         # optional -
↪ bliss
...                   certificate=True)
(Graph on 5 vertices, {0: 4, 1: 3, 2: 1, 3: 0, 4: 2})

>>> G.canonical_label(edge_labels=True, algorithm='sage',
...                   certificate=True)
(Graph on 5 vertices, {0: 4, 1: 3, 2: 0, 3: 1, 4: 2})
```

Another example where different canonization algorithms give different graphs:

```
sage: g = Graph({'a': ['b'], 'c': ['d']})
sage: g_sage = g.canonical_label(algorithm='sage')
sage: g_bliss = g.canonical_label(algorithm='bliss')                    #␣
↪optional - bliss
sage: g_sage.edges(sort=True, labels=False)
[(0, 3), (1, 2)]
sage: g_bliss.edges(sort=True, labels=False)                            #␣
↪optional - bliss
[(0, 1), (2, 3)]
```

```
>>> from sage.all import *
>>> g = Graph({'a': ['b'], 'c': ['d']})
>>> g_sage = g.canonical_label(algorithm='sage')
>>> g_bliss = g.canonical_label(algorithm='bliss')                    # optional -
↪ bliss
>>> g_sage.edges(sort=True, labels=False)
[(0, 3), (1, 2)]
>>> g_bliss.edges(sort=True, labels=False)                            # optional -
↪ bliss
[(0, 1), (2, 3)]
```

cartesian_product(*other*, *immutable=None*)

Return the Cartesian product of `self` and `other`.

The Cartesian product of G and H is the graph L with vertex set $V(L)$ equal to the Cartesian product of the vertices $V(G)$ and $V(H)$, and $((u, v), (w, x))$ is an edge iff either - (u, w) is an edge of self and $v = x$, or - (v, x) is an edge of other and $u = w$.

INPUT:

- `other` – a graph or a digraph
- `immutable` – boolean (default: `None`); whether to create a mutable/immutable product. `immutable=None` (default) means that the graphs and their product will behave the same way. If only one of them is immutable, the product will be mutable.

See also

- *is_cartesian_product()* – factorization of graphs according to the Cartesian product
- *graph_products* – a module on graph products

categorical_product(*other*, *immutable=None*)

alias of *tensor_product()*.

centrality_betweenness(*k=None*, *normalized=True*, *weight=None*, *endpoints=False*, *seed=None*, *exact=False*, *algorithm=None*)

Return the betweenness centrality.

The betweenness centrality of a vertex is the fraction of number of shortest paths that go through each vertex. The betweenness is normalized by default to be in range (0,1).

Measures of the centrality of a vertex within a graph determine the relative importance of that vertex to its graph. Vertices that occur on more shortest paths between other vertices have higher betweenness than vertices that occur on less.

INPUT:

- `normalized` – boolean (default: `True`); if set to `False`, result is not normalized
- `k` – integer (default: `None`); if set to an integer, use `k` node samples to estimate betweenness. Higher values give better approximations. Not available when `algorithm="Sage"`.
- `weight` – string (default: `None`); if set to a string, use that attribute of the nodes as weight. `weight = True` is equivalent to `weight = "weight"`. Not available when `algorithm="Sage"`.
- `endpoints` – boolean (default: `False`); if set to `True` it includes the endpoints in the shortest paths count. Not available when `algorithm="Sage"`.
- `exact` – boolean (default: `False`); whether to compute over rationals or on `double` C variables. Not available when `algorithm="NetworkX"`.
- `algorithm` – string (default: `None`); can be either `'Sage'` (see *centrality*), `"NetworkX"` or `"None"`. In the latter case, Sage's algorithm will be used whenever possible.

See also

- *centrality_degree()*
- *centrality_closeness()*

EXAMPLES:

```
sage: g = graphs.ChvatalGraph()
sage: g.centrality_betweenness() # abs tol 1e-10
{0: 0.06969696969696969, 1: 0.06969696969696969,
 2: 0.0606060606060606, 3: 0.0606060606060606,
 4: 0.06969696969696969, 5: 0.06969696969696969,
 6: 0.0606060606060606, 7: 0.0606060606060606,
 8: 0.0606060606060606, 9: 0.0606060606060606,
 10: 0.0606060606060606, 11: 0.0606060606060606}
sage: g.centrality_betweenness(normalized=False) # abs tol 1e-10
{0: 3.833333333333333, 1: 3.833333333333333, 2: 3.333333333333333,
 3: 3.333333333333333, 4: 3.833333333333333, 5: 3.833333333333333,
 6: 3.333333333333333, 7: 3.333333333333333, 8: 3.333333333333333,
 9: 3.333333333333333, 10: 3.333333333333333,
 11: 3.333333333333333}
sage: D = DiGraph({0:[1,2,3], 1:[2], 3:[0,1]})
sage: D.show(figsize=[2,2])                                                    #␣
↪needs sage.plot
sage: D = D.to_undirected()
sage: D.show(figsize=[2,2])                                                    #␣
↪needs sage.plot
sage: D.centrality_betweenness() # abs tol abs 1e-10
{0: 0.16666666666666666, 1: 0.16666666666666666, 2: 0.0, 3: 0.0}
```

```
>>> from sage.all import *
>>> g = graphs.ChvatalGraph()
>>> g.centrality_betweenness() # abs tol 1e-10
{0: 0.06969696969696969, 1: 0.06969696969696969,
 2: 0.0606060606060606, 3: 0.0606060606060606,
```

(continues on next page)

(continued from previous page)

```
 4: 0.06969696969696969, 5: 0.06969696969696969,
 6: 0.0606060606060606, 7: 0.0606060606060606,
 8: 0.0606060606060606, 9: 0.0606060606060606,
 10: 0.0606060606060606, 11: 0.0606060606060606}
>>> g.centrality_betweenness(normalized=False) # abs tol 1e-10
{0: 3.833333333333333, 1: 3.833333333333333, 2: 3.333333333333333,
 3: 3.333333333333333, 4: 3.833333333333333, 5: 3.833333333333333,
 6: 3.333333333333333, 7: 3.333333333333333, 8: 3.333333333333333,
 9: 3.333333333333333, 10: 3.333333333333333,
 11: 3.333333333333333}
>>> D = DiGraph({Integer(0):[Integer(1),Integer(2),Integer(3)],␣
↪Integer(1):[Integer(2)], Integer(3):[Integer(0),Integer(1)]})
>>> D.show(figsize=[Integer(2),Integer(2)])                              ␣
↪              # needs sage.plot
>>> D = D.to_undirected()
>>> D.show(figsize=[Integer(2),Integer(2)])                              ␣
↪              # needs sage.plot
>>> D.centrality_betweenness() # abs tol abs 1e-10
{0: 0.16666666666666666, 1: 0.16666666666666666, 2: 0.0, 3: 0.0}
```

centrality_closeness(*vert=None*, *by_weight=False*, *algorithm=None*, *weight_function=None*, *check_weight=True*)

Return the closeness centrality of all vertices in `vert`.

In a (strongly) connected graph, the closeness centrality of a vertex v is equal to the inverse of the average distance between v and other vertices. If the graph is disconnected, the closeness centrality of v is multiplied by the fraction of reachable vertices in the graph: this way, central vertices should also reach several other vertices in the graph [OLJ2014]. In formulas,

$$c(v) = \frac{r(v) - 1}{\sum_{w \in R(v)} d(v, w)} \frac{r(v) - 1}{n - 1}$$

where $R(v)$ is the set of vertices reachable from v, and $r(v)$ is the cardinality of $R(v)$.

'Closeness centrality may be defined as the total graph-theoretic distance of a given vertex from all other vertices… Closeness is an inverse measure of centrality in that a larger value indicates a less central actor while a smaller value indicates a more central actor,' [Bor1995].

For more information, see the Wikipedia article Centrality.

INPUT:

- `vert` – the vertex or the list of vertices we want to analyze. If `None` (default), all vertices are considered
- `by_weight` – boolean (default: `False`); if `True`, the edges in the graph are weighted, and otherwise all edges have weight 1
- `algorithm` – string (default: `None`); one of the following algorithms:
  - `'BFS'`: performs a BFS from each vertex that has to be analyzed. Does not work with edge weights.
  - `'NetworkX'`: the NetworkX algorithm (works only with positive weights).
  - `'Dijkstra_Boost'`: the Dijkstra algorithm, implemented in Boost (works only with positive weights).
  - `'Floyd-Warshall-Cython'`: the Cython implementation of the Floyd-Warshall algorithm. Works only if `by_weight==False` and all centralities are needed.

  - `'Floyd-Warshall-Python'`: the Python implementation of the Floyd-Warshall algorithm. Works only if all centralities are needed, but it can deal with weighted graphs, even with negative weights (but no negative cycle is allowed).
  - `'Johnson_Boost'`: the Johnson algorithm, implemented in Boost (works also with negative weights, if there is no negative cycle).
  - `None` (default): Sage chooses the best algorithm: `'BFS'` if `by_weight` is `False`, `'Dijkstra_Boost'` if all weights are positive, `'Johnson_Boost'` otherwise.

- `weight_function` – function (default: `None`); a function that takes as input an edge `(u, v, l)` and outputs its weight. If not `None`, `by_weight` is automatically set to `True`. If `None` and `by_weight` is `True`, we use the edge label `l` as a weight, if `l` is not `None`, else `1` as a weight.
- `check_weight` – boolean (default: `True`); if `True`, we check that the `weight_function` outputs a number for each edge

OUTPUT:

If `vert` is a vertex, the closeness centrality of that vertex. Otherwise, a dictionary associating to each vertex in `vert` its closeness centrality. If a vertex has (out)degree 0, its closeness centrality is not defined, and the vertex is not included in the output.

See also

- *centrality_closeness_top_k()*
- *centrality_degree()*
- *centrality_betweenness()*

EXAMPLES:

Standard examples:

```
sage: (graphs.ChvatalGraph()).centrality_closeness()
{0: 0.61111111111111..., 1: 0.61111111111111...,
 2: 0.61111111111111..., 3: 0.61111111111111...,
 4: 0.61111111111111..., 5: 0.61111111111111...,
 6: 0.61111111111111..., 7: 0.61111111111111...,
 8: 0.61111111111111..., 9: 0.61111111111111...,
 10: 0.61111111111111..., 11: 0.61111111111111...}
sage: D = DiGraph({0:[1,2,3], 1:[2], 3:[0,1]})
sage: D.show(figsize=[2,2])                                                    #␣
↪needs sage.plot
sage: D.centrality_closeness(vert=[0,1])
{0: 1.0, 1: 0.3333333333333333}
sage: D = D.to_undirected()
sage: D.show(figsize=[2,2])                                                    #␣
↪needs sage.plot
sage: D.centrality_closeness()
{0: 1.0, 1: 1.0, 2: 0.75, 3: 0.75}
```

```
>>> from sage.all import *
>>> (graphs.ChvatalGraph()).centrality_closeness()
{0: 0.61111111111111..., 1: 0.61111111111111...,
```

(continues on next page)

(continued from previous page)

```
 2: 0.61111111111111..., 3: 0.61111111111111...,
 4: 0.61111111111111..., 5: 0.61111111111111...,
 6: 0.61111111111111..., 7: 0.61111111111111...,
 8: 0.61111111111111..., 9: 0.61111111111111...,
 10: 0.61111111111111..., 11: 0.61111111111111...}
>>> D = DiGraph({Integer(0):[Integer(1),Integer(2),Integer(3)],␣
↪Integer(1):[Integer(2)], Integer(3):[Integer(0),Integer(1)]})
>>> D.show(figsize=[Integer(2),Integer(2)])                              ␣
↪              # needs sage.plot
>>> D.centrality_closeness(vert=[Integer(0),Integer(1)])
{0: 1.0, 1: 0.3333333333333333}
>>> D = D.to_undirected()
>>> D.show(figsize=[Integer(2),Integer(2)])                              ␣
↪              # needs sage.plot
>>> D.centrality_closeness()
{0: 1.0, 1: 1.0, 2: 0.75, 3: 0.75}
```

In a (strongly) connected (di)graph, the closeness centrality of v is inverse of the average distance between v and all other vertices:

```
sage: g = graphs.PathGraph(5)
sage: g.centrality_closeness(0)
0.4
sage: dist = g.shortest_path_lengths(0).values()
sage: float(len(dist)-1) / sum(dist)
0.4
sage: d = g.to_directed()
sage: d.centrality_closeness(0)
0.4
sage: dist = d.shortest_path_lengths(0).values()
sage: float(len(dist)-1) / sum(dist)
0.4
```

```
>>> from sage.all import *
>>> g = graphs.PathGraph(Integer(5))
>>> g.centrality_closeness(Integer(0))
0.4
>>> dist = g.shortest_path_lengths(Integer(0)).values()
>>> float(len(dist)-Integer(1)) / sum(dist)
0.4
>>> d = g.to_directed()
>>> d.centrality_closeness(Integer(0))
0.4
>>> dist = d.shortest_path_lengths(Integer(0)).values()
>>> float(len(dist)-Integer(1)) / sum(dist)
0.4
```

If a vertex has (out)degree 0, its closeness centrality is not defined:

```
sage: g = Graph(5)
sage: g.centrality_closeness()
{}
```

(continues on next page)

(continued from previous page)

```
sage: print(g.centrality_closeness(0))
None
```

```
>>> from sage.all import *
>>> g = Graph(Integer(5))
>>> g.centrality_closeness()
{}
>>> print(g.centrality_closeness(Integer(0)))
None
```

Weighted graphs:

```
sage: D = graphs.GridGraph([2,2])
sage: weight_function = lambda e:10
sage: D.centrality_closeness([(0,0),(0,1)])                                    # tol␣
↪abs 1e-12
{(0, 0): 0.75, (0, 1): 0.75}
sage: D.centrality_closeness((0,0), weight_function=weight_function) # tol␣
↪abs 1e-12
0.075
```

```
>>> from sage.all import *
>>> D = graphs.GridGraph([Integer(2),Integer(2)])
>>> weight_function = lambda e:Integer(10)
>>> D.centrality_closeness([(Integer(0),Integer(0)),(Integer(0),Integer(1))])␣
↪                          # tol abs 1e-12
{(0, 0): 0.75, (0, 1): 0.75}
>>> D.centrality_closeness((Integer(0),Integer(0)), weight_function=weight_
↪function) # tol abs 1e-12
0.075
```

characteristic_polynomial(*var='x'*, *laplacian=False*)

Return the characteristic polynomial of the adjacency matrix of the (di)graph.

Let G be a (simple) graph with adjacency matrix A. Let I be the identity matrix of dimensions the same as A. The characteristic polynomial of G is defined as the determinant $\det(xI - A)$.

Note

`characteristic_polynomial` and `charpoly` are aliases and thus provide exactly the same method.

INPUT:

- `x` – (default: `'x'`) the variable of the characteristic polynomial
- `laplacian` – boolean (default: `False`); if `True`, use the Laplacian matrix

See also

- *kirchhoff_matrix()*
- *laplacian_matrix()*

EXAMPLES:

```
sage: P = graphs.PetersenGraph()
sage: P.characteristic_polynomial()                                          #␣
↪needs sage.modules
x^10 - 15*x^8 + 75*x^6 - 24*x^5 - 165*x^4 + 120*x^3 + 120*x^2 - 160*x + 48
sage: P.charpoly()                                                           #␣
↪needs sage.modules
x^10 - 15*x^8 + 75*x^6 - 24*x^5 - 165*x^4 + 120*x^3 + 120*x^2 - 160*x + 48
sage: P.characteristic_polynomial(laplacian=True)                            #␣
↪needs sage.modules
x^10 - 30*x^9 + 390*x^8 - 2880*x^7 + 13305*x^6 -
39882*x^5 + 77640*x^4 - 94800*x^3 + 66000*x^2 - 20000*x
```

```
>>> from sage.all import *
>>> P = graphs.PetersenGraph()
>>> P.characteristic_polynomial()                                            #␣
↪needs sage.modules
x^10 - 15*x^8 + 75*x^6 - 24*x^5 - 165*x^4 + 120*x^3 + 120*x^2 - 160*x + 48
>>> P.charpoly()                                                             #␣
↪needs sage.modules
x^10 - 15*x^8 + 75*x^6 - 24*x^5 - 165*x^4 + 120*x^3 + 120*x^2 - 160*x + 48
>>> P.characteristic_polynomial(laplacian=True)                              #␣
↪needs sage.modules
x^10 - 30*x^9 + 390*x^8 - 2880*x^7 + 13305*x^6 -
39882*x^5 + 77640*x^4 - 94800*x^3 + 66000*x^2 - 20000*x
```

charpoly(*var='x'*, *laplacian=False*)

alias of *characteristic_polynomial()*.

clear()

Empties the graph of vertices and edges and removes name, associated objects, and position information.

EXAMPLES:

```
sage: G = graphs.CycleGraph(4)
sage: G.set_vertices({0:'vertex0'})
sage: print(G.order(), G.size())
4 4
sage: G.name()
'Cycle graph'
sage: G.get_vertex(0)
'vertex0'
sage: H = G.copy(sparse=True)
sage: H.clear()
sage: print(H.order(), H.size())
0 0
sage: H.name()
''
sage: H.get_vertex(0)
sage: H = G.copy(sparse=False)
sage: H.clear()
sage: print(H.order(), H.size())
0 0
```

(continues on next page)

(continued from previous page)

```
sage: H.name()
''
sage: H.get_vertex(0)
```

```
>>> from sage.all import *
>>> G = graphs.CycleGraph(Integer(4))
>>> G.set_vertices({Integer(0):'vertex0'})
>>> print(G.order(), G.size())
4 4
>>> G.name()
'Cycle graph'
>>> G.get_vertex(Integer(0))
'vertex0'
>>> H = G.copy(sparse=True)
>>> H.clear()
>>> print(H.order(), H.size())
0 0
>>> H.name()
''
>>> H.get_vertex(Integer(0))
>>> H = G.copy(sparse=False)
>>> H.clear()
>>> print(H.order(), H.size())
0 0
>>> H.name()
''
>>> H.get_vertex(Integer(0))
```

cluster_transitivity()

Return the transitivity (fraction of transitive triangles) of the graph.

Transitivity is the fraction of all existing triangles over all connected triples (triads), $T = 3 \times \frac{\text{triangles}}{\text{triads}}$.

See also section "Clustering" in chapter "Algorithms" of [HSS].

EXAMPLES:

```
sage: graphs.FruchtGraph().cluster_transitivity()                    #␣
↪needs networkx
0.25
```

```
>>> from sage.all import *
>>> graphs.FruchtGraph().cluster_transitivity()                    #␣
↪needs networkx
0.25
```

cluster_triangles(*nbunch=None*, *implementation=None*)

Return the number of triangles for the set $nbunch$ of vertices as a dictionary keyed by vertex.

See also section "Clustering" in chapter "Algorithms" of [HSS].

INPUT:

- `nbunch` – list of vertices (default: `None`); the vertices to inspect. If `nbunch=None`, returns data for all vertices in the graph.

- `implementation` – string (default: `None`); one of `'sparse_copy'`, `'dense_copy'`, `'networkx'` or `None` (default). In the latter case, the best algorithm available is used. Note that `'networkx'` does not support directed graphs.

EXAMPLES:

```
sage: F = graphs.FruchtGraph()
sage: list(F.cluster_triangles().values())
[1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 0, 0]
sage: F.cluster_triangles()
{0: 1, 1: 1, 2: 0, 3: 1, 4: 1, 5: 1, 6: 1, 7: 1, 8: 0, 9: 1, 10: 1, 11: 0}
sage: F.cluster_triangles(nbunch=[0, 1, 2])
{0: 1, 1: 1, 2: 0}
```

```
>>> from sage.all import *
>>> F = graphs.FruchtGraph()
>>> list(F.cluster_triangles().values())
[1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 0, 0]
>>> F.cluster_triangles()
{0: 1, 1: 1, 2: 0, 3: 1, 4: 1, 5: 1, 6: 1, 7: 1, 8: 0, 9: 1, 10: 1, 11: 0}
>>> F.cluster_triangles(nbunch=[Integer(0), Integer(1), Integer(2)])
{0: 1, 1: 1, 2: 0}
```

```
sage: G = graphs.RandomGNP(20, .3)
sage: d1 = G.cluster_triangles(implementation='networkx')                    #␣
↪needs networkx
sage: d2 = G.cluster_triangles(implementation='dense_copy')
sage: d3 = G.cluster_triangles(implementation='sparse_copy')
sage: d1 == d2 and d1 == d3                                                  #␣
↪needs networkx
True
```

```
>>> from sage.all import *
>>> G = graphs.RandomGNP(Integer(20), RealNumber('.3'))
>>> d1 = G.cluster_triangles(implementation='networkx')                    #␣
↪needs networkx
>>> d2 = G.cluster_triangles(implementation='dense_copy')
>>> d3 = G.cluster_triangles(implementation='sparse_copy')
>>> d1 == d2 and d1 == d3                                                  #␣
↪needs networkx
True
```

clustering_average(*implementation=None*)

Return the average clustering coefficient.

The clustering coefficient of a node i is the fraction of existing triangles containing node i over all possible triangles containing i: $c_i = T(i)/\binom{k_i}{2}$ where $T(i)$ is the number of existing triangles through i, and k_i is the degree of vertex i.

A coefficient for the whole graph is the average of the c_i.

See also section "Clustering" in chapter "Algorithms" of [HSS].

INPUT:

- `implementation` – string (default: `None`); one of `'boost'`, `'sparse_copy'`, `'dense_copy'`,

'networkx' or None (default). In the latter case, the best algorithm available is used. Note that only 'networkx' supports directed graphs.

EXAMPLES:

```
sage: (graphs.FruchtGraph()).clustering_average()
1/4
sage: (graphs.FruchtGraph()).clustering_average(implementation='networkx')  #␣
↪needs networkx
0.25
```

```
>>> from sage.all import *
>>> (graphs.FruchtGraph()).clustering_average()
1/4
>>> (graphs.FruchtGraph()).clustering_average(implementation='networkx')  #␣
↪needs networkx
0.25
```

clustering_coeff(*nodes=None*, *weight=False*, *implementation=None*)

Return the clustering coefficient for each vertex in nodes as a dictionary keyed by vertex.

For an unweighted graph, the clustering coefficient of a node i is the fraction of existing triangles containing node i over all possible triangles containing i: $c_i = T(i)/\binom{k_i}{2}$ where $T(i)$ is the number of existing triangles through i, and k_i is the degree of vertex i.

For weighted graphs the clustering is defined as the geometric average of the subgraph edge weights, normalized by the maximum weight in the network.

The value of c_i is assigned 0 if $k_i < 2$.

See also section "Clustering" in chapter "Algorithms" of [HSS].

INPUT:

- nodes – an iterable container of vertices (default: None); the vertices to inspect. By default, returns data on all vertices in graph
- weight – string or boolean (default: False); if it is a string it uses the indicated edge property as weight. weight = True is equivalent to weight = 'weight'
- implementation – string (default: None); one of 'boost', 'sparse_copy', 'dense_copy', 'networkx' or None (default). In the latter case, the best algorithm available is used. Note that only 'networkx' supports directed or weighted graphs, and that 'sparse_copy' and 'dense_copy' do not support node different from None

EXAMPLES:

```
sage: graphs.FruchtGraph().clustering_coeff()
{0: 1/3, 1: 1/3, 2: 0, 3: 1/3, 4: 1/3, 5: 1/3,
 6: 1/3, 7: 1/3, 8: 0, 9: 1/3, 10: 1/3, 11: 0}

sage: # needs networkx
sage: import numpy
sage: if int(numpy.version.short_version[0]) > 1:
....:     numpy.set_printoptions(legacy="1.25")
sage: graphs.FruchtGraph().clustering_coeff(weight=True)
{0: 0.3333333333333333, 1: 0.3333333333333333, 2: 0,
 3: 0.3333333333333333, 4: 0.3333333333333333,
```

(continues on next page)

(continued from previous page)

```
 5: 0.3333333333333333, 6: 0.3333333333333333,
 7: 0.3333333333333333, 8: 0, 9: 0.3333333333333333,
 10: 0.3333333333333333, 11: 0}

sage: graphs.FruchtGraph().clustering_coeff(nodes=[0,1,2])
{0: 0.3333333333333333, 1: 0.3333333333333333, 2: 0.0}

sage: graphs.FruchtGraph().clustering_coeff(nodes=[0,1,2],                #␣
↪needs networkx
....:                                              weight=True)
{0: 0.3333333333333333, 1: 0.3333333333333333, 2: 0}

sage: graphs.GridGraph([5,5]).clustering_coeff(nodes=[(0,0),(0,1),(2,2)])
{(0, 0): 0.0, (0, 1): 0.0, (2, 2): 0.0}
```

```
>>> from sage.all import *
>>> graphs.FruchtGraph().clustering_coeff()
{0: 1/3, 1: 1/3, 2: 0, 3: 1/3, 4: 1/3, 5: 1/3,
 6: 1/3, 7: 1/3, 8: 0, 9: 1/3, 10: 1/3, 11: 0}

>>> # needs networkx
>>> import numpy
>>> if int(numpy.version.short_version[Integer(0)]) > Integer(1):
...     numpy.set_printoptions(legacy="1.25")
>>> graphs.FruchtGraph().clustering_coeff(weight=True)
{0: 0.3333333333333333, 1: 0.3333333333333333, 2: 0,
 3: 0.3333333333333333, 4: 0.3333333333333333,
 5: 0.3333333333333333, 6: 0.3333333333333333,
 7: 0.3333333333333333, 8: 0, 9: 0.3333333333333333,
 10: 0.3333333333333333, 11: 0}

>>> graphs.FruchtGraph().clustering_coeff(nodes=[Integer(0),Integer(1),
↪Integer(2)])
{0: 0.3333333333333333, 1: 0.3333333333333333, 2: 0.0}

>>> graphs.FruchtGraph().clustering_coeff(nodes=[Integer(0),Integer(1),
↪Integer(2)],                # needs networkx
...                                            weight=True)
{0: 0.3333333333333333, 1: 0.3333333333333333, 2: 0}

>>> graphs.GridGraph([Integer(5),Integer(5)]).clustering_
↪coeff(nodes=[(Integer(0),Integer(0)),(Integer(0),Integer(1)),(Integer(2),
↪Integer(2))])
{(0, 0): 0.0, (0, 1): 0.0, (2, 2): 0.0}
```

coarsest_equitable_refinement(*partition*, *sparse=True*)

Return the coarsest partition which is finer than the input partition, and equitable with respect to `self`.

A partition is equitable with respect to a graph if for every pair of cells C_1, C_2 of the partition, the number of edges from a vertex of C_1 to C_2 is the same, over all vertices in C_1.

A partition P_1 is finer than P_2 (P_2 is coarser than P_1) if every cell of P_1 is a subset of a cell of P_2.

INPUT:

- `partition` – list of lists
- `sparse` – boolean (default: `False`); whether to use sparse or dense representation - for small graphs, use dense for speed

EXAMPLES:

```
sage: G = graphs.PetersenGraph()
sage: G.coarsest_equitable_refinement([[0],list(range(1,10))])
[[0], [2, 3, 6, 7, 8, 9], [1, 4, 5]]
sage: G = graphs.CubeGraph(3)
sage: verts = G.vertices(sort=True)
sage: Pi = [verts[:1], verts[1:]]
sage: Pi
[['000'], ['001', '010', '011', '100', '101', '110', '111']]
sage: [sorted(cell) for cell in G.coarsest_equitable_refinement(Pi)]
[['000'], ['011', '101', '110'], ['111'], ['001', '010', '100']]
```

```
>>> from sage.all import *
>>> G = graphs.PetersenGraph()
>>> G.coarsest_equitable_refinement([[Integer(0)],list(range(Integer(1),
↪Integer(10)))])
[[0], [2, 3, 6, 7, 8, 9], [1, 4, 5]]
>>> G = graphs.CubeGraph(Integer(3))
>>> verts = G.vertices(sort=True)
>>> Pi = [verts[:Integer(1)], verts[Integer(1):]]
>>> Pi
[['000'], ['001', '010', '011', '100', '101', '110', '111']]
>>> [sorted(cell) for cell in G.coarsest_equitable_refinement(Pi)]
[['000'], ['011', '101', '110'], ['111'], ['001', '010', '100']]
```

Note that given an equitable partition, this function returns that partition:

```
sage: P = graphs.PetersenGraph()
sage: prt = [[0], [1, 4, 5], [2, 3, 6, 7, 8, 9]]
sage: P.coarsest_equitable_refinement(prt)
[[0], [1, 4, 5], [2, 3, 6, 7, 8, 9]]
```

```
>>> from sage.all import *
>>> P = graphs.PetersenGraph()
>>> prt = [[Integer(0)], [Integer(1), Integer(4), Integer(5)], [Integer(2),␣
↪Integer(3), Integer(6), Integer(7), Integer(8), Integer(9)]]
>>> P.coarsest_equitable_refinement(prt)
[[0], [1, 4, 5], [2, 3, 6, 7, 8, 9]]
```

```
sage: ss = (graphs.WheelGraph(6)).line_graph(labels=False)
sage: prt = [[(0, 1)], [(0, 2), (0, 3), (0, 4), (1, 2), (1, 4)], [(2, 3), (3,␣
↪4)]]
sage: ss.coarsest_equitable_refinement(prt)
Traceback (most recent call last):
...
TypeError: partition ([[(0, 1)], [(0, 2), (0, 3), (0, 4), (1, 2), (1, 4)],
[(2, 3), (3, 4)]]) is not valid for this graph: vertices are incorrect
```

```
>>> from sage.all import *
>>> ss = (graphs.WheelGraph(Integer(6))).line_graph(labels=False)
>>> prt = [[(Integer(0), Integer(1))], [(Integer(0), Integer(2)), (Integer(0),
↪ Integer(3)), (Integer(0), Integer(4)), (Integer(1), Integer(2)),␣
↪(Integer(1), Integer(4))], [(Integer(2), Integer(3)), (Integer(3),␣
↪Integer(4))]]
>>> ss.coarsest_equitable_refinement(prt)
Traceback (most recent call last):
...
TypeError: partition ([[(0, 1)], [(0, 2), (0, 3), (0, 4), (1, 2), (1, 4)],
[(2, 3), (3, 4)]]) is not valid for this graph: vertices are incorrect
```

```
sage: ss = (graphs.WheelGraph(5)).line_graph(labels=False)
sage: [sorted(cell) for cell in ss.coarsest_equitable_refinement(prt)]
[[(0, 1)], [(1, 2), (1, 4)], [(0, 3)], [(0, 2), (0, 4)], [(2, 3), (3, 4)]]
```

```
>>> from sage.all import *
>>> ss = (graphs.WheelGraph(Integer(5))).line_graph(labels=False)
>>> [sorted(cell) for cell in ss.coarsest_equitable_refinement(prt)]
[[(0, 1)], [(1, 2), (1, 4)], [(0, 3)], [(0, 2), (0, 4)], [(2, 3), (3, 4)]]
```

ALGORITHM: Brendan D. McKay's Master's Thesis, University of Melbourne, 1976.

complement(*immutable=None*)

Return the complement of the (di)graph.

The complement of a graph has the same vertices, but exactly those edges that are not in the original graph. This is not well defined for graphs with multiple edges.

INPUT:

- `immutable` – boolean (default: `None`); whether to return a mutable or an immutable version of `self`. By default (`None`), the graph and its complement behave the same.

EXAMPLES:

```
sage: P = graphs.PetersenGraph()
sage: P.plot()                      # long time                              #␣
↪needs sage.plot
Graphics object consisting of 26 graphics primitives
sage: PC = P.complement()
sage: PC.plot()                     # long time                              #␣
↪needs sage.plot
Graphics object consisting of 41 graphics primitives
```

```
>>> from sage.all import *
>>> P = graphs.PetersenGraph()
>>> P.plot()                        # long time                             #␣
↪needs sage.plot
Graphics object consisting of 26 graphics primitives
>>> PC = P.complement()
>>> PC.plot()                       # long time                             #␣
↪needs sage.plot
Graphics object consisting of 41 graphics primitives
```

```
sage: graphs.TetrahedralGraph().complement().size()
0
sage: graphs.CycleGraph(4).complement().edges(sort=True)
[(0, 2, None), (1, 3, None)]
sage: graphs.CycleGraph(4).complement()
complement(Cycle graph): Graph on 4 vertices
sage: G = Graph(multiedges=True, sparse=True)
sage: G.add_edges([(0, 1)] * 3)
sage: G.complement()
Traceback (most recent call last):
...
ValueError: This method is not known to work on graphs with
multiedges. Perhaps this method can be updated to handle them, but
in the meantime if you want to use it please disallow multiedges
using allow_multiple_edges().
```

```
>>> from sage.all import *
>>> graphs.TetrahedralGraph().complement().size()
0
>>> graphs.CycleGraph(Integer(4)).complement().edges(sort=True)
[(0, 2, None), (1, 3, None)]
>>> graphs.CycleGraph(Integer(4)).complement()
complement(Cycle graph): Graph on 4 vertices
>>> G = Graph(multiedges=True, sparse=True)
>>> G.add_edges([(Integer(0), Integer(1))] * Integer(3))
>>> G.complement()
Traceback (most recent call last):
...
ValueError: This method is not known to work on graphs with
multiedges. Perhaps this method can be updated to handle them, but
in the meantime if you want to use it please disallow multiedges
using allow_multiple_edges().
```

connected_component_containing_vertex(*G*, *vertex*, *sort=False*, *key=None*, *forbidden_vertices=None*)

Return a list of the vertices connected to vertex.

INPUT:

- `G` – the input graph
- `vertex` – the vertex to search for
- `sort` – boolean (default: `False`); if `True`, vertices inside the component are sorted according to the default ordering
- `key` – a function (default: `None`); a function that takes a vertex as its one argument and returns a value that can be used for comparisons in the sorting algorithm (we must have `sort=True`)
- `forbidden_vertices` – list (default: `None`); set of vertices to avoid during the search. The start `vertex` cannot be in this set.

EXAMPLES:

```
sage: from sage.graphs.connectivity import connected_component_containing_
↪vertex
sage: G = Graph({0: [1, 3], 1: [2], 2: [3], 4: [5, 6], 5: [6]})
```

(continues on next page)

(continued from previous page)

```
sage: connected_component_containing_vertex(G, 0, sort=True)
[0, 1, 2, 3]
sage: G.connected_component_containing_vertex(0, sort=True)
[0, 1, 2, 3]
sage: G.connected_component_containing_vertex(0, sort=True, forbidden_
↪vertices=[1, 3])
[0]
sage: D = DiGraph({0: [1, 3], 1: [2], 2: [3], 4: [5, 6], 5: [6]})
sage: connected_component_containing_vertex(D, 0, sort=True)
[0, 1, 2, 3]
sage: connected_component_containing_vertex(D, 0, sort=True, key=lambda x: -x)
[3, 2, 1, 0]
```

```
>>> from sage.all import *
>>> from sage.graphs.connectivity import connected_component_containing_vertex
>>> G = Graph({Integer(0): [Integer(1), Integer(3)], Integer(1): [Integer(2)],
↪ Integer(2): [Integer(3)], Integer(4): [Integer(5), Integer(6)],␣
↪Integer(5): [Integer(6)]})
>>> connected_component_containing_vertex(G, Integer(0), sort=True)
[0, 1, 2, 3]
>>> G.connected_component_containing_vertex(Integer(0), sort=True)
[0, 1, 2, 3]
>>> G.connected_component_containing_vertex(Integer(0), sort=True, forbidden_
↪vertices=[Integer(1), Integer(3)])
[0]
>>> D = DiGraph({Integer(0): [Integer(1), Integer(3)], Integer(1):␣
↪[Integer(2)], Integer(2): [Integer(3)], Integer(4): [Integer(5),␣
↪Integer(6)], Integer(5): [Integer(6)]})
>>> connected_component_containing_vertex(D, Integer(0), sort=True)
[0, 1, 2, 3]
>>> connected_component_containing_vertex(D, Integer(0), sort=True,␣
↪key=lambda x: -x)
[3, 2, 1, 0]
```

connected_components(*G*, *sort=False*, *key=None*, *forbidden_vertices=None*)

Return the list of connected components.

This returns a list of lists of vertices, each list representing a connected component. The list is ordered from largest to smallest component.

INPUT:

- `G` – the input graph
- `sort` – boolean (default: `False`); if `True`, vertices inside each component are sorted according to the default ordering
- `key` – a function (default: `None`); a function that takes a vertex as its one argument and returns a value that can be used for comparisons in the sorting algorithm (we must have `sort=True`)
- `forbidden_vertices` – list (default: `None`); set of vertices to avoid during the search

EXAMPLES:

```
sage: from sage.graphs.connectivity import connected_components
sage: G = Graph({0: [1, 3], 1: [2], 2: [3], 4: [5, 6], 5: [6]})
sage: connected_components(G, sort=True)
[[0, 1, 2, 3], [4, 5, 6]]
sage: G.connected_components(sort=True)
[[0, 1, 2, 3], [4, 5, 6]]
sage: D = DiGraph({0: [1, 3], 1: [2], 2: [3], 4: [5, 6], 5: [6]})
sage: connected_components(D, sort=True)
[[0, 1, 2, 3], [4, 5, 6]]
sage: connected_components(D, sort=True, key=lambda x: -x)
[[3, 2, 1, 0], [6, 5, 4]]
```

```
>>> from sage.all import *
>>> from sage.graphs.connectivity import connected_components
>>> G = Graph({Integer(0): [Integer(1), Integer(3)], Integer(1): [Integer(2)],
... Integer(2): [Integer(3)], Integer(4): [Integer(5), Integer(6)],␣
...Integer(5): [Integer(6)]})
>>> connected_components(G, sort=True)
[[0, 1, 2, 3], [4, 5, 6]]
>>> G.connected_components(sort=True)
[[0, 1, 2, 3], [4, 5, 6]]
>>> D = DiGraph({Integer(0): [Integer(1), Integer(3)], Integer(1):␣
...[Integer(2)], Integer(2): [Integer(3)], Integer(4): [Integer(5),␣
...Integer(6)], Integer(5): [Integer(6)]})
>>> connected_components(D, sort=True)
[[0, 1, 2, 3], [4, 5, 6]]
>>> connected_components(D, sort=True, key=lambda x: -x)
[[3, 2, 1, 0], [6, 5, 4]]
```

Connected components in a graph with forbidden vertices:

```
sage: G = graphs.PathGraph(5)
sage: connected_components(G, sort=True, forbidden_vertices=[2])
[[0, 1], [3, 4]]
sage: connected_components(G, sort=True,
....:     forbidden_vertices=G.neighbor_iterator(2, closed=True))
[[0], [4]]
```

```
>>> from sage.all import *
>>> G = graphs.PathGraph(Integer(5))
>>> connected_components(G, sort=True, forbidden_vertices=[Integer(2)])
[[0, 1], [3, 4]]
>>> connected_components(G, sort=True,
...     forbidden_vertices=G.neighbor_iterator(Integer(2), closed=True))
[[0], [4]]
```

connected_components_number(*G*, *forbidden_vertices=None*)

alias of *number_of_connected_components()*.

connected_components_sizes(*G*, *forbidden_vertices=None*)

Return the sizes of the connected components as a list.

The list is sorted from largest to lower values.

INPUT:

- `G` – the input graph
- `forbidden_vertices` – list (default: `None`); set of vertices to avoid during the search

EXAMPLES:

```
sage: from sage.graphs.connectivity import connected_components_sizes
sage: for x in graphs(3):
....:     print(connected_components_sizes(x))
[1, 1, 1]
[2, 1]
[3]
[3]
sage: for x in graphs(3):
....:     print(x.connected_components_sizes())
[1, 1, 1]
[2, 1]
[3]
[3]
sage: G = graphs.PathGraph(5)
sage: G.connected_components_sizes()
[5]
sage: G.connected_components_sizes(forbidden_vertices=[1])
[3, 1]
sage: G.connected_components_sizes(forbidden_vertices=[1, 3])
[1, 1, 1]
```

```
>>> from sage.all import *
>>> from sage.graphs.connectivity import connected_components_sizes
>>> for x in graphs(Integer(3)):
...     print(connected_components_sizes(x))
[1, 1, 1]
[2, 1]
[3]
[3]
>>> for x in graphs(Integer(3)):
...     print(x.connected_components_sizes())
[1, 1, 1]
[2, 1]
[3]
[3]
>>> G = graphs.PathGraph(Integer(5))
>>> G.connected_components_sizes()
[5]
>>> G.connected_components_sizes(forbidden_vertices=[Integer(1)])
[3, 1]
>>> G.connected_components_sizes(forbidden_vertices=[Integer(1), Integer(3)])
[1, 1, 1]
```

connected_components_subgraphs(*G*, *forbidden_vertices=None*, *immutable=None*)

Return a list of connected components as graph objects.

INPUT:

- `G` – the input graph
- `forbidden_vertices` – list (default: `None`); set of vertices to avoid during the search
- `immutable` – boolean (default: `None`); whether to create a mutable/immutable subgraphs. `immutable=None` (default) means that the graph and the subgraphs of its connected components will behave the same way.

EXAMPLES:

```
sage: from sage.graphs.connectivity import connected_components_subgraphs
sage: G = Graph({0: [1, 3], 1: [2], 2: [3], 4: [5, 6], 5: [6]})
sage: L = connected_components_subgraphs(G)
sage: graphs_list.show_graphs(L)                                              ␣
↪  # needs sage.plot
sage: L = connected_components_subgraphs(G, forbidden_vertices=[1, 3])
sage: graphs_list.show_graphs(L)                                              ␣
↪  # needs sage.plot
sage: D = DiGraph({0: [1, 3], 1: [2], 2: [3], 4: [5, 6], 5: [6]})
sage: L = connected_components_subgraphs(D)
sage: graphs_list.show_graphs(L)                                              ␣
↪  # needs sage.plot
sage: L = D.connected_components_subgraphs()
sage: graphs_list.show_graphs(L)                                              ␣
↪  # needs sage.plot
```

```
>>> from sage.all import *
>>> from sage.graphs.connectivity import connected_components_subgraphs
>>> G = Graph({Integer(0): [Integer(1), Integer(3)], Integer(1): [Integer(2)],
↪ Integer(2): [Integer(3)], Integer(4): [Integer(5), Integer(6)],␣
↪Integer(5): [Integer(6)]})
>>> L = connected_components_subgraphs(G)
>>> graphs_list.show_graphs(L)
↪# needs sage.plot
>>> L = connected_components_subgraphs(G, forbidden_vertices=[Integer(1),␣
↪Integer(3)])
>>> graphs_list.show_graphs(L)
↪# needs sage.plot
>>> D = DiGraph({Integer(0): [Integer(1), Integer(3)], Integer(1):␣
↪[Integer(2)], Integer(2): [Integer(3)], Integer(4): [Integer(5),␣
↪Integer(6)], Integer(5): [Integer(6)]})
>>> L = connected_components_subgraphs(D)
>>> graphs_list.show_graphs(L)
↪# needs sage.plot
>>> L = D.connected_components_subgraphs()
>>> graphs_list.show_graphs(L)
↪# needs sage.plot
```

connected_subgraph_iterator(*G*, *k=None*, *vertices_only=False*, *edges_only=False*, *labels=False*, *induced=True*, *exactly_k=False*)

Return an iterator over the induced connected subgraphs of order at most k.

This method implements a iterator over the induced connected subgraphs of the input (di)graph. An induced subgraph of a graph is another graph, formed from a subset of the vertices of the graph and all of the edges connecting pairs of vertices in that subset (Wikipedia article Induced_subgraph).

As for method *connected_components()*, edge orientation is ignored. Hence, the directed graph with a single arc $0 \to 1$ is considered connected.

INPUT:

- `G` – a *Graph* or a *DiGraph*; loops and multiple edges are allowed
- `k` – (optional) integer; maximum order of the connected subgraphs to report; by default, the method iterates over all connected subgraphs (equivalent to `k == n`)
- `vertices_only` – boolean (default: `False`); whether to return (Di)Graph or list of vertices. This parameter is ignored when `induced` is `True`.
- `edges_only` – boolean (default: `False`); whether to return (Di)Graph or list of edges. When `vertices_only` is `True`, this parameter is ignored.
- `labels` – boolean (default: `False`); whether to return labeled edges or not. This parameter is used only when `vertices_only` is `False` and `edges_only` is `True`.
- `induced` – boolean (default: `True`); whether to return induced connected sub(di)graph only or also non-induced sub(di)graphs. This parameter can be set to `False` for simple (di)graphs only.
- `exactly_k` – boolean (default: `False`); `True` if we only return graphs of order k, `False` if we return graphs of order at most k.

EXAMPLES:

```
sage: G = DiGraph([(1, 2), (2, 3), (3, 4), (4, 2)])
sage: list(G.connected_subgraph_iterator())
[Subgraph of (): Digraph on 1 vertex,
 Subgraph of (): Digraph on 2 vertices,
 Subgraph of (): Digraph on 3 vertices,
 Subgraph of (): Digraph on 4 vertices,
 Subgraph of (): Digraph on 3 vertices,
 Subgraph of (): Digraph on 1 vertex,
 Subgraph of (): Digraph on 2 vertices,
 Subgraph of (): Digraph on 3 vertices,
 Subgraph of (): Digraph on 2 vertices,
 Subgraph of (): Digraph on 1 vertex,
 Subgraph of (): Digraph on 2 vertices,
 Subgraph of (): Digraph on 1 vertex]
sage: list(G.connected_subgraph_iterator(vertices_only=True))
[[1], [1, 2], [1, 2, 3], [1, 2, 3, 4], [1, 2, 4],
 [2], [2, 3], [2, 3, 4], [2, 4], [3], [3, 4], [4]]
sage: list(G.connected_subgraph_iterator(k=2))
[Subgraph of (): Digraph on 1 vertex,
 Subgraph of (): Digraph on 2 vertices,
 Subgraph of (): Digraph on 1 vertex,
 Subgraph of (): Digraph on 2 vertices,
 Subgraph of (): Digraph on 2 vertices,
 Subgraph of (): Digraph on 1 vertex,
 Subgraph of (): Digraph on 2 vertices,
 Subgraph of (): Digraph on 1 vertex]
sage: list(G.connected_subgraph_iterator(k=3, vertices_only=True, exactly_
↪k=True))
[[1, 2, 3], [1, 2, 4], [2, 3, 4]]
sage: list(G.connected_subgraph_iterator(k=2, vertices_only=True))
[[1], [1, 2], [2], [2, 3], [2, 4], [3], [3, 4], [4]]
```

(continues on next page)

(continued from previous page)

```
sage: G = DiGraph([(1, 2), (2, 1)])
sage: list(G.connected_subgraph_iterator())
[Subgraph of (): Digraph on 1 vertex,
 Subgraph of (): Digraph on 2 vertices,
 Subgraph of (): Digraph on 1 vertex]
sage: list(G.connected_subgraph_iterator(vertices_only=True))
[[1], [1, 2], [2]]

sage: G = graphs.CompleteGraph(3)
sage: len(list(G.connected_subgraph_iterator()))
7
sage: len(list(G.connected_subgraph_iterator(vertices_only=True)))
7
sage: len(list(G.connected_subgraph_iterator(edges_only=True)))
7
sage: len(list(G.connected_subgraph_iterator(induced=False)))
10

sage: G = DiGraph([(0, 1), (1, 0), (1, 2), (2, 1)])
sage: len(list(G.connected_subgraph_iterator()))
6
sage: len(list(G.connected_subgraph_iterator(vertices_only=True)))
6
sage: len(list(G.connected_subgraph_iterator(edges_only=True)))
6
sage: len(list(G.connected_subgraph_iterator(induced=False)))
18
```

```
>>> from sage.all import *
>>> G = DiGraph([(Integer(1), Integer(2)), (Integer(2), Integer(3)),␣
↪(Integer(3), Integer(4)), (Integer(4), Integer(2))])
>>> list(G.connected_subgraph_iterator())
[Subgraph of (): Digraph on 1 vertex,
 Subgraph of (): Digraph on 2 vertices,
 Subgraph of (): Digraph on 3 vertices,
 Subgraph of (): Digraph on 4 vertices,
 Subgraph of (): Digraph on 3 vertices,
 Subgraph of (): Digraph on 1 vertex,
 Subgraph of (): Digraph on 2 vertices,
 Subgraph of (): Digraph on 3 vertices,
 Subgraph of (): Digraph on 2 vertices,
 Subgraph of (): Digraph on 1 vertex,
 Subgraph of (): Digraph on 2 vertices,
 Subgraph of (): Digraph on 1 vertex]
>>> list(G.connected_subgraph_iterator(vertices_only=True))
[[1], [1, 2], [1, 2, 3], [1, 2, 3, 4], [1, 2, 4],
 [2], [2, 3], [2, 3, 4], [2, 4], [3], [3, 4], [4]]
>>> list(G.connected_subgraph_iterator(k=Integer(2)))
[Subgraph of (): Digraph on 1 vertex,
 Subgraph of (): Digraph on 2 vertices,
 Subgraph of (): Digraph on 1 vertex,
```

(continues on next page)

(continued from previous page)

```
 Subgraph of (): Digraph on 2 vertices,
 Subgraph of (): Digraph on 2 vertices,
 Subgraph of (): Digraph on 1 vertex,
 Subgraph of (): Digraph on 2 vertices,
 Subgraph of (): Digraph on 1 vertex]
>>> list(G.connected_subgraph_iterator(k=Integer(3), vertices_only=True,␣
↪exactly_k=True))
[[1, 2, 3], [1, 2, 4], [2, 3, 4]]
>>> list(G.connected_subgraph_iterator(k=Integer(2), vertices_only=True))
[[1], [1, 2], [2], [2, 3], [2, 4], [3], [3, 4], [4]]

>>> G = DiGraph([(Integer(1), Integer(2)), (Integer(2), Integer(1))])
>>> list(G.connected_subgraph_iterator())
[Subgraph of (): Digraph on 1 vertex,
 Subgraph of (): Digraph on 2 vertices,
 Subgraph of (): Digraph on 1 vertex]
>>> list(G.connected_subgraph_iterator(vertices_only=True))
[[1], [1, 2], [2]]

>>> G = graphs.CompleteGraph(Integer(3))
>>> len(list(G.connected_subgraph_iterator()))
7
>>> len(list(G.connected_subgraph_iterator(vertices_only=True)))
7
>>> len(list(G.connected_subgraph_iterator(edges_only=True)))
7
>>> len(list(G.connected_subgraph_iterator(induced=False)))
10

>>> G = DiGraph([(Integer(0), Integer(1)), (Integer(1), Integer(0)),␣
↪(Integer(1), Integer(2)), (Integer(2), Integer(1))])
>>> len(list(G.connected_subgraph_iterator()))
6
>>> len(list(G.connected_subgraph_iterator(vertices_only=True)))
6
>>> len(list(G.connected_subgraph_iterator(edges_only=True)))
6
>>> len(list(G.connected_subgraph_iterator(induced=False)))
18
```

contract_edge(*u*, *v=None*, *label=None*)

Contract an edge from `u` to `v`.

This method returns silently if the edge does not exist.

INPUT: The following forms are all accepted:

- G.contract_edge(1, 2)
- G.contract_edge((1, 2))
- G.contract_edge([(1, 2)])
- G.contract_edge(1, 2, 'label')
- G.contract_edge((1, 2, 'label'))

- G.contract_edge([(1, 2, 'label')])

EXAMPLES:

```
sage: G = graphs.CompleteGraph(4)
sage: G.contract_edge((0, 1)); G.edges(sort=True)
[(0, 2, None), (0, 3, None), (2, 3, None)]
sage: G = graphs.CompleteGraph(4)
sage: G.allow_loops(True); G.allow_multiple_edges(True)
sage: G.contract_edge((0, 1)); G.edges(sort=True)
[(0, 2, None), (0, 2, None), (0, 3, None), (0, 3, None), (2, 3, None)]
sage: G.contract_edge((0, 2)); G.edges(sort=True)
[(0, 0, None), (0, 3, None), (0, 3, None), (0, 3, None)]
```

```
>>> from sage.all import *
>>> G = graphs.CompleteGraph(Integer(4))
>>> G.contract_edge((Integer(0), Integer(1))); G.edges(sort=True)
[(0, 2, None), (0, 3, None), (2, 3, None)]
>>> G = graphs.CompleteGraph(Integer(4))
>>> G.allow_loops(True); G.allow_multiple_edges(True)
>>> G.contract_edge((Integer(0), Integer(1))); G.edges(sort=True)
[(0, 2, None), (0, 2, None), (0, 3, None), (0, 3, None), (2, 3, None)]
>>> G.contract_edge((Integer(0), Integer(2))); G.edges(sort=True)
[(0, 0, None), (0, 3, None), (0, 3, None), (0, 3, None)]
```

```
sage: G = graphs.CompleteGraph(4).to_directed()
sage: G.allow_loops(True)
sage: G.contract_edge(0, 1); G.edges(sort=True)
[(0, 0, None),
 (0, 2, None),
 (0, 3, None),
 (2, 0, None),
 (2, 3, None),
 (3, 0, None),
 (3, 2, None)]
```

```
>>> from sage.all import *
>>> G = graphs.CompleteGraph(Integer(4)).to_directed()
>>> G.allow_loops(True)
>>> G.contract_edge(Integer(0), Integer(1)); G.edges(sort=True)
[(0, 0, None),
 (0, 2, None),
 (0, 3, None),
 (2, 0, None),
 (2, 3, None),
 (3, 0, None),
 (3, 2, None)]
```

contract_edges(*edges*)

Contract edges from an iterable container.

If e is an edge that is not contracted but the vertices of e are merged by contraction of other edges, then e will become a loop.

INPUT:

- `edges` – list containing 2-tuples or 3-tuples that represent edges

EXAMPLES:

```
sage: G = graphs.CompleteGraph(4)
sage: G.allow_loops(True); G.allow_multiple_edges(True)
sage: G.contract_edges([(0, 1), (1, 2), (0, 2)]); G.edges(sort=True)
[(0, 3, None), (0, 3, None), (0, 3, None)]
sage: G.contract_edges([(1, 3), (2, 3)]); G.edges(sort=True)
[(0, 3, None), (0, 3, None), (0, 3, None)]
sage: G = graphs.CompleteGraph(4)
sage: G.allow_loops(True); G.allow_multiple_edges(True)
sage: G.contract_edges([(0, 1), (1, 2), (0, 2), (1, 3), (2, 3)]); G.
↪edges(sort=True)
[(0, 0, None)]
```

```
>>> from sage.all import *
>>> G = graphs.CompleteGraph(Integer(4))
>>> G.allow_loops(True); G.allow_multiple_edges(True)
>>> G.contract_edges([(Integer(0), Integer(1)), (Integer(1), Integer(2)),␣
↪(Integer(0), Integer(2))]); G.edges(sort=True)
[(0, 3, None), (0, 3, None), (0, 3, None)]
>>> G.contract_edges([(Integer(1), Integer(3)), (Integer(2), Integer(3))]); G.
↪edges(sort=True)
[(0, 3, None), (0, 3, None), (0, 3, None)]
>>> G = graphs.CompleteGraph(Integer(4))
>>> G.allow_loops(True); G.allow_multiple_edges(True)
>>> G.contract_edges([(Integer(0), Integer(1)), (Integer(1), Integer(2)),␣
↪(Integer(0), Integer(2)), (Integer(1), Integer(3)), (Integer(2),␣
↪Integer(3))]); G.edges(sort=True)
[(0, 0, None)]
```

```
sage: D = digraphs.Complete(4)
sage: D.allow_loops(True); D.allow_multiple_edges(True)
sage: D.contract_edges([(0, 1), (1, 0), (0, 2)]); D.edges(sort=True)
[(0, 0, None),
 (0, 0, None),
 (0, 0, None),
 (0, 3, None),
 (0, 3, None),
 (0, 3, None),
 (3, 0, None),
 (3, 0, None),
 (3, 0, None)]
```

```
>>> from sage.all import *
>>> D = digraphs.Complete(Integer(4))
>>> D.allow_loops(True); D.allow_multiple_edges(True)
>>> D.contract_edges([(Integer(0), Integer(1)), (Integer(1), Integer(0)),␣
↪(Integer(0), Integer(2))]); D.edges(sort=True)
[(0, 0, None),
 (0, 0, None),
 (0, 0, None),
```

(continues on next page)

(continued from previous page)

```
(0, 3, None),
(0, 3, None),
(0, 3, None),
(3, 0, None),
(3, 0, None),
(3, 0, None)]
```

copy(*weighted=None*, *data_structure=None*, *sparse=None*, *immutable=None*, *hash_labels=None*)

Change the graph implementation.

INPUT:

- `weighted` – boolean (default: `None`); weightedness for the copy. Might change the equality class if not `None`.
- `sparse` – boolean (default: `None`); `sparse=True` is an alias for `data_structure="sparse"`, and `sparse=False` is an alias for `data_structure="dense"`. Only used when `data_structure=None`.
- `data_structure` – string (default: `None`); one of `'sparse'`, `'static_sparse'`, or `'dense'`. See the documentation of *Graph* or *DiGraph*.
- `immutable` – boolean (default: `None`); whether to create a mutable/immutable copy. Only used when `data_structure=None`.
  - `immutable=None` (default) means that the graph and its copy will behave the same way.
  - `immutable=True` is a shortcut for `data_structure='static_sparse'`
  - `immutable=False` means that the created graph is mutable. When used to copy an immutable graph, the data structure used is `'sparse'` unless anything else is specified.
- `hash_labels` – boolean (default: `None`); whether to include edge labels during hashing of the copy. This parameter defaults to `True` if the graph is weighted. This parameter is ignored when parameter `immutable` is not `True`. Beware that trying to hash unhashable labels will raise an error.

Note

If the graph uses *StaticSparseBackend* and the `_immutable` flag, then `self` is returned rather than a copy (unless one of the optional arguments is used).

OUTPUT: a Graph object

Warning

Please use this method only if you need to copy but change the underlying data structure or weightedness. Otherwise simply do `copy(g)` instead of `g.copy()`.

Warning

If `weighted` is passed and is not the weightedness of the original, then the copy will not equal the original.

EXAMPLES:

```
sage: g = Graph({0: [0, 1, 1, 2]}, loops=True, multiedges=True, sparse=True)
sage: g == copy(g)
True
sage: g = DiGraph({0: [0, 1, 1, 2], 1: [0, 1]}, loops=True, multiedges=True,␣
↪sparse=True)
sage: g == copy(g)
True
```

```
>>> from sage.all import *
>>> g = Graph({Integer(0): [Integer(0), Integer(1), Integer(1), Integer(2)]},␣
↪loops=True, multiedges=True, sparse=True)
>>> g == copy(g)
True
>>> g = DiGraph({Integer(0): [Integer(0), Integer(1), Integer(1), Integer(2)],
↪ Integer(1): [Integer(0), Integer(1)]}, loops=True, multiedges=True,␣
↪sparse=True)
>>> g == copy(g)
True
```

Note that vertex associations are also kept:

```
sage: d = {0: graphs.DodecahedralGraph(), 1: graphs.FlowerSnark(), 2: graphs.
↪MoebiusKantorGraph(), 3: graphs.PetersenGraph()}
sage: T = graphs.TetrahedralGraph()
sage: T.set_vertices(d)
sage: T2 = copy(T)
sage: T2.get_vertex(0)
Dodecahedron: Graph on 20 vertices
```

```
>>> from sage.all import *
>>> d = {Integer(0): graphs.DodecahedralGraph(), Integer(1): graphs.
↪FlowerSnark(), Integer(2): graphs.MoebiusKantorGraph(), Integer(3): graphs.
↪PetersenGraph()}
>>> T = graphs.TetrahedralGraph()
>>> T.set_vertices(d)
>>> T2 = copy(T)
>>> T2.get_vertex(Integer(0))
Dodecahedron: Graph on 20 vertices
```

Notice that the copy is at least as deep as the objects:

```
sage: T2.get_vertex(0) is T.get_vertex(0)
False
```

```
>>> from sage.all import *
>>> T2.get_vertex(Integer(0)) is T.get_vertex(Integer(0))
False
```

Examples of the keywords in use:

```
sage: G = graphs.CompleteGraph(9)
sage: H = G.copy()
```

(continues on next page)

(continued from previous page)

```
sage: H == G; H is G
True
False
sage: G1 = G.copy(sparse=True)
sage: G1 == G
True
sage: G1 is G
False
sage: G2 = copy(G)
sage: G2 is G
False
```

```
>>> from sage.all import *
>>> G = graphs.CompleteGraph(Integer(9))
>>> H = G.copy()
>>> H == G; H is G
True
False
>>> G1 = G.copy(sparse=True)
>>> G1 == G
True
>>> G1 is G
False
>>> G2 = copy(G)
>>> G2 is G
False
```

Argument `weighted` affects the equality class:

```
sage: G = graphs.CompleteGraph(5)
sage: H1 = G.copy(weighted=False)
sage: H2 = G.copy(weighted=True)
sage: [G.weighted(), H1.weighted(), H2.weighted()]
[False, False, True]
sage: [G == H1, G == H2, H1 == H2]
[True, False, False]
sage: G.weighted(True)
sage: [G == H1, G == H2, H1 == H2]
[False, True, False]
```

```
>>> from sage.all import *
>>> G = graphs.CompleteGraph(Integer(5))
>>> H1 = G.copy(weighted=False)
>>> H2 = G.copy(weighted=True)
>>> [G.weighted(), H1.weighted(), H2.weighted()]
[False, False, True]
>>> [G == H1, G == H2, H1 == H2]
[True, False, False]
>>> G.weighted(True)
>>> [G == H1, G == H2, H1 == H2]
[False, True, False]
```

crossing_number()

Return the crossing number of the graph.

The crossing number of a graph is the minimum number of edge crossings needed to draw the graph on a plane. It can be seen as a measure of non-planarity; a planar graph has crossing number zero.

See the Wikipedia article Crossing_number for more information.

EXAMPLES:

```
sage: P = graphs.PetersenGraph()
sage: P.crossing_number()
2
```

```
>>> from sage.all import *
>>> P = graphs.PetersenGraph()
>>> P.crossing_number()
2
```

ALGORITHM:

This is slow brute force implementation: for every k pairs of edges try adding a new vertex for a crossing point for them. If the result is not planar in any of those, try $k + 1$ pairs.

Computing the crossing number is NP-hard problem.

cycle_basis(*output='vertex'*)

Return a list of cycles which form a basis of the cycle space of `self`.

A basis of cycles of a graph is a minimal collection of cycles (considered as sets of edges) such that the edge set of any cycle in the graph can be written as a $Z/2Z$ sum of the cycles in the basis.

See the Wikipedia article Cycle_basis for more information.

INPUT:

- `output` – string (default: `'vertex'`); whether every cycle is given as a list of vertices (`output == 'vertex'`) or a list of edges (`output == 'edge'`)

OUTPUT:

A list of lists, each of them representing the vertices (or the edges) of a cycle in a basis.

ALGORITHM:

Uses the NetworkX library for graphs without multiple edges.

Otherwise, by the standard algorithm using a spanning tree.

EXAMPLES:

A cycle basis in Petersen's Graph

```
sage: g = graphs.PetersenGraph()
sage: g.cycle_basis()                                                    #␣
↪needs networkx, random (changes in networkx 3.2)
[[6, 8, 5, 7, 9],
 [2, 3, 8, 5, 7],
 [4, 3, 8, 5, 7, 9],
 [4, 0, 5, 7, 9],
 [2, 1, 0, 5, 7],
 [6, 1, 0, 5, 7, 9]]
```

```
>>> from sage.all import *
>>> g = graphs.PetersenGraph()
>>> g.cycle_basis()                                                    #␣
↪needs networkx, random (changes in networkx 3.2)
[[6, 8, 5, 7, 9],
 [2, 3, 8, 5, 7],
 [4, 3, 8, 5, 7, 9],
 [4, 0, 5, 7, 9],
 [2, 1, 0, 5, 7],
 [6, 1, 0, 5, 7, 9]]
```

One can also get the result as a list of lists of edges:

```
sage: g.cycle_basis(output='edge')                                     #␣
↪needs networkx, random (changes in networkx 3.2)
[[(6, 8, None), (8, 5, None), (5, 7, None), (7, 9, None), (9, 6, None)],
 [(2, 3, None), (3, 8, None), (8, 5, None), (5, 7, None), (7, 2, None)],
 [(4, 3, None),
  (3, 8, None),
  (8, 5, None),
  (5, 7, None),
  (7, 9, None),
  (9, 4, None)],
 [(4, 0, None), (0, 5, None), (5, 7, None), (7, 9, None), (9, 4, None)],
 [(2, 1, None), (1, 0, None), (0, 5, None), (5, 7, None), (7, 2, None)],
 [(6, 1, None),
  (1, 0, None),
  (0, 5, None),
  (5, 7, None),
  (7, 9, None),
  (9, 6, None)]]
```

```
>>> from sage.all import *
>>> g.cycle_basis(output='edge')                                       #␣
↪needs networkx, random (changes in networkx 3.2)
[[(6, 8, None), (8, 5, None), (5, 7, None), (7, 9, None), (9, 6, None)],
 [(2, 3, None), (3, 8, None), (8, 5, None), (5, 7, None), (7, 2, None)],
 [(4, 3, None),
  (3, 8, None),
  (8, 5, None),
  (5, 7, None),
  (7, 9, None),
  (9, 4, None)],
 [(4, 0, None), (0, 5, None), (5, 7, None), (7, 9, None), (9, 4, None)],
 [(2, 1, None), (1, 0, None), (0, 5, None), (5, 7, None), (7, 2, None)],
 [(6, 1, None),
  (1, 0, None),
  (0, 5, None),
  (5, 7, None),
  (7, 9, None),
  (9, 6, None)]]
```

Checking the given cycles are algebraically free:

```
sage: g = graphs.RandomGNP(30, .4)                                          #␣
↪needs networkx
sage: basis = g.cycle_basis()                                               #␣
↪needs networkx
```

```
>>> from sage.all import *
>>> g = graphs.RandomGNP(Integer(30), RealNumber('.4'))                      ␣
↪                    # needs networkx
>>> basis = g.cycle_basis()                                                #␣
↪needs networkx
```

Building the space of (directed) edges over $Z/2Z$. On the way, building a dictionary associating a unique vector to each undirected edge:

```
sage: m = g.size()
sage: edge_space = VectorSpace(FiniteField(2), m)                           #␣
↪needs sage.modules sage.rings.finite_rings
sage: edge_vector = dict(zip(g.edges(labels=False, sort=False),             #␣
↪needs sage.modules sage.rings.finite_rings
....:                           edge_space.basis()))
sage: for (u, v), vec in list(edge_vector.items()):                         #␣
↪needs sage.modules sage.rings.finite_rings
....:     edge_vector[(v, u)] = vec
```

```
>>> from sage.all import *
>>> m = g.size()
>>> edge_space = VectorSpace(FiniteField(Integer(2)), m)                    ␣
↪     # needs sage.modules sage.rings.finite_rings
>>> edge_vector = dict(zip(g.edges(labels=False, sort=False),               #␣
↪needs sage.modules sage.rings.finite_rings
...                           edge_space.basis()))
>>> for (u, v), vec in list(edge_vector.items()):                           #␣
↪needs sage.modules sage.rings.finite_rings
...     edge_vector[(v, u)] = vec
```

Defining a lambda function associating a vector to the vertices of a cycle:

```
sage: vertices_to_edges = lambda x: zip(x, x[1:] + [x[0]])
sage: cycle_to_vector = lambda x: sum(edge_vector[e] for e in vertices_to_
↪edges(x))
```

```
>>> from sage.all import *
>>> vertices_to_edges = lambda x: zip(x, x[Integer(1):] + [x[Integer(0)]])
>>> cycle_to_vector = lambda x: sum(edge_vector[e] for e in vertices_to_
↪edges(x))
```

Finally checking the cycles are a free set:

```
sage: basis_as_vectors = [cycle_to_vector(_) for _ in basis]                #␣
↪needs networkx sage.modules sage.rings.finite_rings
sage: edge_space.span(basis_as_vectors).rank() == len(basis)                #␣
↪needs networkx sage.modules sage.rings.finite_rings
True
```

```
>>> from sage.all import *
>>> basis_as_vectors = [cycle_to_vector(_) for _ in basis]                    #␣
↪needs networkx sage.modules sage.rings.finite_rings
>>> edge_space.span(basis_as_vectors).rank() == len(basis)                    #␣
↪needs networkx sage.modules sage.rings.finite_rings
True
```

For undirected graphs with multiple edges:

```
sage: G = Graph([(0, 2, 'a'), (0, 2, 'b'), (0, 1, 'c'), (1, 2, 'd')],
....:           multiedges=True)
sage: G.cycle_basis()
[[2, 0], [2, 0, 1]]
sage: G.cycle_basis(output='edge')
[[(0, 2, 'b'), (2, 0, 'a')], [(1, 2, 'd'), (2, 0, 'a'), (0, 1, 'c')]]
sage: H = Graph([(1, 2), (2, 3), (2, 3), (3, 4), (1, 4),
....:            (1, 4), (4, 5), (5, 6), (4, 6), (6, 7)], multiedges=True)
sage: H.cycle_basis()
[[4, 1], [3, 2], [4, 1, 2, 3], [6, 4, 5]]
```

```
>>> from sage.all import *
>>> G = Graph([(Integer(0), Integer(2), 'a'), (Integer(0), Integer(2), 'b'),␣
↪(Integer(0), Integer(1), 'c'), (Integer(1), Integer(2), 'd')],
...           multiedges=True)
>>> G.cycle_basis()
[[2, 0], [2, 0, 1]]
>>> G.cycle_basis(output='edge')
[[(0, 2, 'b'), (2, 0, 'a')], [(1, 2, 'd'), (2, 0, 'a'), (0, 1, 'c')]]
>>> H = Graph([(Integer(1), Integer(2)), (Integer(2), Integer(3)),␣
↪(Integer(2), Integer(3)), (Integer(3), Integer(4)), (Integer(1),␣
↪Integer(4)),
...            (Integer(1), Integer(4)), (Integer(4), Integer(5)),␣
↪(Integer(5), Integer(6)), (Integer(4), Integer(6)), (Integer(6),␣
↪Integer(7))], multiedges=True)
>>> H.cycle_basis()
[[4, 1], [3, 2], [4, 1, 2, 3], [6, 4, 5]]
```

Disconnected graph:

```
sage: G.add_cycle(["Hey", "Wuuhuu", "Really ?"])
sage: [sorted(c) for c in G.cycle_basis()]                                    #␣
↪needs networkx
[['Hey', 'Really ?', 'Wuuhuu'], [0, 2], [0, 1, 2]]
sage: [sorted(c) for c in G.cycle_basis(output='edge')]                       #␣
↪needs networkx
[[('Hey', 'Really ?', None),
  ('Really ?', 'Wuuhuu', None),
  ('Wuuhuu', 'Hey', None)],
 [(0, 2, 'a'), (2, 0, 'b')],
 [(0, 1, 'c'), (1, 2, 'd'), (2, 0, 'b')]]
```

```
>>> from sage.all import *
>>> G.add_cycle(["Hey", "Wuuhuu", "Really ?"])
```

(continues on next page)

(continued from previous page)

```
>>> [sorted(c) for c in G.cycle_basis()]                                    #␣
↪needs networkx
[['Hey', 'Really ?', 'Wuuhuu'], [0, 2], [0, 1, 2]]
>>> [sorted(c) for c in G.cycle_basis(output='edge')]                       #␣
↪needs networkx
[[('Hey', 'Really ?', None),
  ('Really ?', 'Wuuhuu', None),
  ('Wuuhuu', 'Hey', None)],
 [(0, 2, 'a'), (2, 0, 'b')],
 [(0, 1, 'c'), (1, 2, 'd'), (2, 0, 'b')]]
```

Graph that allows multiple edges but does not contain any:

```
sage: G = graphs.CycleGraph(3)
sage: G.allow_multiple_edges(True)
sage: G.cycle_basis()
[[2, 0, 1]]
```

```
>>> from sage.all import *
>>> G = graphs.CycleGraph(Integer(3))
>>> G.allow_multiple_edges(True)
>>> G.cycle_basis()
[[2, 0, 1]]
```

Not yet implemented for directed graphs:

```
sage: G = DiGraph([(0, 2, 'a'), (0, 1, 'c'), (1, 2, 'd')])
sage: G.cycle_basis()
Traceback (most recent call last):
...
NotImplementedError: not implemented for directed graphs
```

```
>>> from sage.all import *
>>> G = DiGraph([(Integer(0), Integer(2), 'a'), (Integer(0), Integer(1), 'c'),
↪ (Integer(1), Integer(2), 'd')])
>>> G.cycle_basis()
Traceback (most recent call last):
...
NotImplementedError: not implemented for directed graphs
```

degree(*vertices=None*, *labels=False*)

Return the degree (in + out for digraphs) of a vertex or of vertices.

INPUT:

- `vertices` – a vertex or an iterable container of vertices (default: `None`); if `vertices` is a single vertex, returns the number of neighbors of that vertex. If `vertices` is an iterable container of vertices, returns a list of degrees. If `vertices` is `None`, same as listing all vertices.
- `labels` – boolean (default: `False`); when `True`, return a dictionary mapping each vertex in `vertices` to its degree. Otherwise, return the degree of a single vertex or a list of the degrees of each vertex in `vertices`

OUTPUT:

- When `vertices` is a single vertex and `labels` is `False`, returns the degree of that vertex as an integer
- When `vertices` is an iterable container of vertices (or `None`) and `labels` is `False`, returns a list of integers. The i-th value is the degree of the i-th vertex in the list `vertices`. When `vertices` is `None`, the i-th value is the degree of i-th vertex in the ordering `list(self)`, which might be different from the ordering of the vertices given by `g.vertices(sort=True)`.
- When `labels` is `True`, returns a dictionary mapping each vertex in `vertices` to its degree

EXAMPLES:

```
sage: P = graphs.PetersenGraph()
sage: P.degree(5)
3
```

```
>>> from sage.all import *
>>> P = graphs.PetersenGraph()
>>> P.degree(Integer(5))
3
```

```
sage: K = graphs.CompleteGraph(9)
sage: K.degree()
[8, 8, 8, 8, 8, 8, 8, 8, 8]
```

```
>>> from sage.all import *
>>> K = graphs.CompleteGraph(Integer(9))
>>> K.degree()
[8, 8, 8, 8, 8, 8, 8, 8, 8]
```

```
sage: D = DiGraph({0: [1, 2, 3], 1: [0, 2], 2: [3], 3: [4], 4: [0,5], 5: [1]})
sage: D.degree(vertices=[0, 1, 2], labels=True)
{0: 5, 1: 4, 2: 3}
sage: D.degree()
[5, 4, 3, 3, 3, 2]
```

```
>>> from sage.all import *
>>> D = DiGraph({Integer(0): [Integer(1), Integer(2), Integer(3)],␣
↪Integer(1): [Integer(0), Integer(2)], Integer(2): [Integer(3)], Integer(3):␣
↪[Integer(4)], Integer(4): [Integer(0),Integer(5)], Integer(5): [Integer(1)]}
↪)
>>> D.degree(vertices=[Integer(0), Integer(1), Integer(2)], labels=True)
{0: 5, 1: 4, 2: 3}
>>> D.degree()
[5, 4, 3, 3, 3, 2]
```

When `vertices=None` and `labels=False`, the i-th value of the returned list is the degree of the i-th vertex in the list `list(self)`:

```
sage: D = digraphs.DeBruijn(4, 2)
sage: D.delete_vertex('20')
sage: print(D.degree())
[7, 7, 6, 7, 8, 8, 7, 8, 8, 7, 8, 8, 8, 7, 8]
sage: print(D.degree(vertices=list(D)))
[7, 7, 6, 7, 8, 8, 7, 8, 8, 7, 8, 8, 8, 7, 8]
```

(continues on next page)

(continued from previous page)

```
sage: print(D.degree(vertices=D.vertices(sort=False)))
[7, 7, 6, 7, 8, 8, 7, 8, 8, 7, 8, 8, 8, 7, 8]
```

```
>>> from sage.all import *
>>> D = digraphs.DeBruijn(Integer(4), Integer(2))
>>> D.delete_vertex('20')
>>> print(D.degree())
[7, 7, 6, 7, 8, 8, 7, 8, 8, 7, 8, 8, 8, 7, 8]
>>> print(D.degree(vertices=list(D)))
[7, 7, 6, 7, 8, 8, 7, 8, 8, 7, 8, 8, 8, 7, 8]
>>> print(D.degree(vertices=D.vertices(sort=False)))
[7, 7, 6, 7, 8, 8, 7, 8, 8, 7, 8, 8, 8, 7, 8]
```

degree_histogram()

Return a list, whose i-th entry is the frequency of degree i.

EXAMPLES:

```
sage: G = graphs.Grid2dGraph(9, 12)
sage: G.degree_histogram()
[0, 0, 4, 34, 70]
```

```
>>> from sage.all import *
>>> G = graphs.Grid2dGraph(Integer(9), Integer(12))
>>> G.degree_histogram()
[0, 0, 4, 34, 70]
```

```
sage: G = graphs.Grid2dGraph(9, 12).to_directed()
sage: G.degree_histogram()
[0, 0, 0, 0, 4, 0, 34, 0, 70]
```

```
>>> from sage.all import *
>>> G = graphs.Grid2dGraph(Integer(9), Integer(12)).to_directed()
>>> G.degree_histogram()
[0, 0, 0, 0, 4, 0, 34, 0, 70]
```

degree_iterator(*vertices=None*, *labels=False*)

Return an iterator over the degrees of the (di)graph.

In the case of a digraph, the degree is defined as the sum of the in-degree and the out-degree, i.e. the total number of edges incident to a given vertex.

INPUT:

- `vertices` – a vertex or an iterable container of vertices (default: `None`); if `vertices` is a single vertex, the iterator will yield the number of neighbors of that vertex. If `vertices` is an iterable container of vertices, return an iterator over the degrees of these vertices. If `vertices` is `None`, same as listing all vertices.
- `labels` – boolean (default: `False`); whether to return an iterator over degrees (`labels=False`), or over tuples `(vertex, degree)`

> **Note**
>
> The returned iterator yields values in order specified by `list(vertices)`. When `vertices` is `None`, it yields values in the same order as `list(self)`, which might be different from the ordering of the vertices given by `g.vertices(sort=True)`.

EXAMPLES:

```
sage: G = graphs.Grid2dGraph(3, 4)
sage: for i in G.degree_iterator():
....:     print(i)
2
3
3
...
3
2
sage: for i in G.degree_iterator(labels=True):
....:     print(i)
((0, 0), 2)
((0, 1), 3)
((0, 2), 3)
...
((2, 2), 3)
((2, 3), 2)
```

```
>>> from sage.all import *
>>> G = graphs.Grid2dGraph(Integer(3), Integer(4))
>>> for i in G.degree_iterator():
...     print(i)
2
3
3
...
3
2
>>> for i in G.degree_iterator(labels=True):
...     print(i)
((0, 0), 2)
((0, 1), 3)
((0, 2), 3)
...
((2, 2), 3)
((2, 3), 2)
```

```
sage: D = graphs.Grid2dGraph(2,4).to_directed()
sage: for i in D.degree_iterator():
....:     print(i)
4
6
...
6
```

(continues on next page)

(continued from previous page)

```
4
sage: for i in D.degree_iterator(labels=True):
....:     print(i)
((0, 0), 4)
((0, 1), 6)
...
((1, 2), 6)
((1, 3), 4)
```

```
>>> from sage.all import *
>>> D = graphs.Grid2dGraph(Integer(2),Integer(4)).to_directed()
>>> for i in D.degree_iterator():
...     print(i)
4
6
...
6
4
>>> for i in D.degree_iterator(labels=True):
...     print(i)
((0, 0), 4)
((0, 1), 6)
...
((1, 2), 6)
((1, 3), 4)
```

When `vertices=None` yields values in the order of `list(D)`:

```
sage: V = list(D)
sage: D = digraphs.DeBruijn(4, 2)                                    #␣
↪needs sage.combinat
sage: D.delete_vertex('20')                                          #␣
↪needs sage.combinat
sage: print(list(D.degree_iterator()))                               #␣
↪needs sage.combinat
[7, 7, 6, 7, 8, 8, 7, 8, 8, 7, 8, 8, 8, 7, 8]
sage: print([D.degree(v) for v in D])                                #␣
↪needs sage.combinat
[7, 7, 6, 7, 8, 8, 7, 8, 8, 7, 8, 8, 8, 7, 8]
```

```
>>> from sage.all import *
>>> V = list(D)
>>> D = digraphs.DeBruijn(Integer(4), Integer(2))                    ␣
↪              # needs sage.combinat
>>> D.delete_vertex('20')                                           #␣
↪needs sage.combinat
>>> print(list(D.degree_iterator()))                                #␣
↪needs sage.combinat
[7, 7, 6, 7, 8, 8, 7, 8, 8, 7, 8, 8, 8, 7, 8]
>>> print([D.degree(v) for v in D])                                 #␣
↪needs sage.combinat
[7, 7, 6, 7, 8, 8, 7, 8, 8, 7, 8, 8, 8, 7, 8]
```

degree_sequence()

Return the degree sequence of this (di)graph.

EXAMPLES:

The degree sequence of an undirected graph:

```
sage: g = Graph({1: [2, 5], 2: [1, 5, 3, 4], 3: [2, 5], 4: [3], 5: [2, 3]})
sage: g.degree_sequence()
[4, 3, 3, 2, 2]
```

```
>>> from sage.all import *
>>> g = Graph({Integer(1): [Integer(2), Integer(5)], Integer(2): [Integer(1),␣
↪Integer(5), Integer(3), Integer(4)], Integer(3): [Integer(2), Integer(5)],␣
↪Integer(4): [Integer(3)], Integer(5): [Integer(2), Integer(3)]})
>>> g.degree_sequence()
[4, 3, 3, 2, 2]
```

The degree sequence of a digraph:

```
sage: g = DiGraph({1: [2, 5, 6], 2: [3, 6], 3: [4, 6], 4: [6], 5: [4, 6]})
sage: g.degree_sequence()
[5, 3, 3, 3, 3, 3]
```

```
>>> from sage.all import *
>>> g = DiGraph({Integer(1): [Integer(2), Integer(5), Integer(6)],␣
↪Integer(2): [Integer(3), Integer(6)], Integer(3): [Integer(4), Integer(6)],␣
↪Integer(4): [Integer(6)], Integer(5): [Integer(4), Integer(6)]})
>>> g.degree_sequence()
[5, 3, 3, 3, 3, 3]
```

Degree sequences of some common graphs:

```
sage: graphs.PetersenGraph().degree_sequence()
[3, 3, 3, 3, 3, 3, 3, 3, 3, 3]
sage: graphs.HouseGraph().degree_sequence()
[3, 3, 2, 2, 2]
sage: graphs.FlowerSnark().degree_sequence()
[3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3]
```

```
>>> from sage.all import *
>>> graphs.PetersenGraph().degree_sequence()
[3, 3, 3, 3, 3, 3, 3, 3, 3, 3]
>>> graphs.HouseGraph().degree_sequence()
[3, 3, 2, 2, 2]
>>> graphs.FlowerSnark().degree_sequence()
[3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3]
```

degree_to_cell(*vertex*, *cell*)

Return the number of edges from vertex to an edge in cell.

In the case of a digraph, this returns a tuple (in_degree, out_degree).

EXAMPLES:

```
sage: G = graphs.CubeGraph(3)
sage: cell = G.vertices(sort=True)[:3]
sage: G.degree_to_cell('011', cell)
2
sage: G.degree_to_cell('111', cell)
0
```

```
>>> from sage.all import *
>>> G = graphs.CubeGraph(Integer(3))
>>> cell = G.vertices(sort=True)[:Integer(3)]
>>> G.degree_to_cell('011', cell)
2
>>> G.degree_to_cell('111', cell)
0
```

```
sage: D = DiGraph({ 0:[1,2,3], 1:[3,4], 3:[4,5]})
sage: cell = [0,1,2]
sage: D.degree_to_cell(5, cell)
(0, 0)
sage: D.degree_to_cell(3, cell)
(2, 0)
sage: D.degree_to_cell(0, cell)
(0, 2)
```

```
>>> from sage.all import *
>>> D = DiGraph({ Integer(0):[Integer(1),Integer(2),Integer(3)],␣
↪Integer(1):[Integer(3),Integer(4)], Integer(3):[Integer(4),Integer(5)]})
>>> cell = [Integer(0),Integer(1),Integer(2)]
>>> D.degree_to_cell(Integer(5), cell)
(0, 0)
>>> D.degree_to_cell(Integer(3), cell)
(2, 0)
>>> D.degree_to_cell(Integer(0), cell)
(0, 2)
```

delete_edge(*u*, *v=None*, *label=None*)

Delete the edge from `u` to `v`.

This method returns silently if vertices or edge does not exist.

INPUT: The following forms are all accepted:

- G.delete_edge(1, 2)
- G.delete_edge((1, 2))
- G.delete_edges([(1, 2)])
- G.delete_edge(1, 2, 'label')
- G.delete_edge((1, 2, 'label'))
- G.delete_edges([(1, 2, 'label')])

EXAMPLES:

```
sage: G = graphs.CompleteGraph(9)
sage: G.size()
36
sage: G.delete_edge( 1, 2 )
sage: G.delete_edge( (3, 4) )
sage: G.delete_edges( [ (5, 6), (7, 8) ] )
sage: G.size()
32
```

```
>>> from sage.all import *
>>> G = graphs.CompleteGraph(Integer(9))
>>> G.size()
36
>>> G.delete_edge( Integer(1), Integer(2) )
>>> G.delete_edge( (Integer(3), Integer(4)) )
>>> G.delete_edges( [ (Integer(5), Integer(6)), (Integer(7), Integer(8)) ] )
>>> G.size()
32
```

```
sage: G.delete_edge( 2, 3, 'label' )
sage: G.delete_edge( (4, 5, 'label') )
sage: G.delete_edges( [ (6, 7, 'label') ] )
sage: G.size()
32
sage: G.has_edge( (4, 5) ) # correct!
True
sage: G.has_edge( (4, 5, 'label') ) # correct!
False
```

```
>>> from sage.all import *
>>> G.delete_edge( Integer(2), Integer(3), 'label' )
>>> G.delete_edge( (Integer(4), Integer(5), 'label') )
>>> G.delete_edges( [ (Integer(6), Integer(7), 'label') ] )
>>> G.size()
32
>>> G.has_edge( (Integer(4), Integer(5)) ) # correct!
True
>>> G.has_edge( (Integer(4), Integer(5), 'label') ) # correct!
False
```

```
sage: C = digraphs.Complete(9)
sage: C.size()
72
sage: C.delete_edge( 1, 2 )
sage: C.delete_edge( (3, 4) )
sage: C.delete_edges( [ (5, 6), (7, 8) ] )
sage: C.size()
68
```

```
>>> from sage.all import *
>>> C = digraphs.Complete(Integer(9))
```

(continues on next page)

(continued from previous page)

```
>>> C.size()
72
>>> C.delete_edge( Integer(1), Integer(2) )
>>> C.delete_edge( (Integer(3), Integer(4)) )
>>> C.delete_edges( [ (Integer(5), Integer(6)), (Integer(7), Integer(8)) ] )
>>> C.size()
68
```

```
sage: C.delete_edge( 2, 3, 'label' )
sage: C.delete_edge( (4, 5, 'label') )
sage: C.delete_edges( [ (6, 7, 'label') ] )
sage: C.size() # correct!
68
sage: C.has_edge( (4, 5) ) # correct!
True
sage: C.has_edge( (4, 5, 'label') ) # correct!
False
```

```
>>> from sage.all import *
>>> C.delete_edge( Integer(2), Integer(3), 'label' )
>>> C.delete_edge( (Integer(4), Integer(5), 'label') )
>>> C.delete_edges( [ (Integer(6), Integer(7), 'label') ] )
>>> C.size() # correct!
68
>>> C.has_edge( (Integer(4), Integer(5)) ) # correct!
True
>>> C.has_edge( (Integer(4), Integer(5), 'label') ) # correct!
False
```

delete_edges(*edges*)

Delete edges from an iterable container.

EXAMPLES:

```
sage: K12 = graphs.CompleteGraph(12)
sage: K4 = graphs.CompleteGraph(4)
sage: K12.size()
66
sage: K12.delete_edges(K4.edge_iterator())
sage: K12.size()
60
```

```
>>> from sage.all import *
>>> K12 = graphs.CompleteGraph(Integer(12))
>>> K4 = graphs.CompleteGraph(Integer(4))
>>> K12.size()
66
>>> K12.delete_edges(K4.edge_iterator())
>>> K12.size()
60
```

```
sage: K12 = digraphs.Complete(12)
sage: K4 = digraphs.Complete(4)
sage: K12.size()
132
sage: K12.delete_edges(K4.edge_iterator())
sage: K12.size()
120
```

```
>>> from sage.all import *
>>> K12 = digraphs.Complete(Integer(12))
>>> K4 = digraphs.Complete(Integer(4))
>>> K12.size()
132
>>> K12.delete_edges(K4.edge_iterator())
>>> K12.size()
120
```

delete_multiedge(*u*, *v*)

Delete all edges from `u` to `v`.

EXAMPLES:

```
sage: G = Graph(multiedges=True, sparse=True)
sage: G.add_edges([(0, 1), (0, 1), (0, 1), (1, 2), (2, 3)])
sage: G.edges(sort=True)
[(0, 1, None), (0, 1, None), (0, 1, None), (1, 2, None), (2, 3, None)]
sage: G.delete_multiedge(0, 1)
sage: G.edges(sort=True)
[(1, 2, None), (2, 3, None)]
```

```
>>> from sage.all import *
>>> G = Graph(multiedges=True, sparse=True)
>>> G.add_edges([(Integer(0), Integer(1)), (Integer(0), Integer(1)),␣
↪(Integer(0), Integer(1)), (Integer(1), Integer(2)), (Integer(2),␣
↪Integer(3))])
>>> G.edges(sort=True)
[(0, 1, None), (0, 1, None), (0, 1, None), (1, 2, None), (2, 3, None)]
>>> G.delete_multiedge(Integer(0), Integer(1))
>>> G.edges(sort=True)
[(1, 2, None), (2, 3, None)]
```

```
sage: D = DiGraph(multiedges=True, sparse=True)
sage: D.add_edges([(0, 1, 1), (0, 1, 2), (0, 1, 3), (1, 0, None), (1, 2,␣
↪None), (2, 3, None)])
sage: D.edges(sort=True)
[(0, 1, 1), (0, 1, 2), (0, 1, 3), (1, 0, None), (1, 2, None), (2, 3, None)]
sage: D.delete_multiedge(0, 1)
sage: D.edges(sort=True)
[(1, 0, None), (1, 2, None), (2, 3, None)]
```

```
>>> from sage.all import *
>>> D = DiGraph(multiedges=True, sparse=True)
```

(continues on next page)

(continued from previous page)

```
>>> D.add_edges([(Integer(0), Integer(1), Integer(1)), (Integer(0),␣
↪Integer(1), Integer(2)), (Integer(0), Integer(1), Integer(3)), (Integer(1),␣
↪Integer(0), None), (Integer(1), Integer(2), None), (Integer(2), Integer(3),␣
↪None)])
>>> D.edges(sort=True)
[(0, 1, 1), (0, 1, 2), (0, 1, 3), (1, 0, None), (1, 2, None), (2, 3, None)]
>>> D.delete_multiedge(Integer(0), Integer(1))
>>> D.edges(sort=True)
[(1, 0, None), (1, 2, None), (2, 3, None)]
```

delete_vertex(*vertex*, *in_order=False*)

Delete vertex, removing all incident edges.

Deleting a non-existent vertex will raise an exception.

INPUT:

- `in_order` – boolean (default: `False`); if `True`, this deletes the i-th vertex in the sorted list of vertices, i.e. `G.vertices(sort=True)[i]`

EXAMPLES:

```
sage: G = Graph(graphs.WheelGraph(9))
sage: G.delete_vertex(0)
sage: G.show()                                                              #␣
↪needs sage.plot
```

```
>>> from sage.all import *
>>> G = Graph(graphs.WheelGraph(Integer(9)))
>>> G.delete_vertex(Integer(0))
>>> G.show()                                                                #␣
↪needs sage.plot
```

```
sage: D = DiGraph({0: [1, 2, 3, 4, 5], 1: [2], 2: [3], 3: [4], 4: [5], 5: [1]}
↪)
sage: D.delete_vertex(0); D
Digraph on 5 vertices
sage: D.vertices(sort=True)
[1, 2, 3, 4, 5]
sage: D.delete_vertex(0)
Traceback (most recent call last):
...
ValueError: vertex (0) not in the graph
```

```
>>> from sage.all import *
>>> D = DiGraph({Integer(0): [Integer(1), Integer(2), Integer(3), Integer(4),␣
↪Integer(5)], Integer(1): [Integer(2)], Integer(2): [Integer(3)],␣
↪Integer(3): [Integer(4)], Integer(4): [Integer(5)], Integer(5):␣
↪[Integer(1)]})
>>> D.delete_vertex(Integer(0)); D
Digraph on 5 vertices
>>> D.vertices(sort=True)
[1, 2, 3, 4, 5]
```

(continues on next page)

(continued from previous page)

```
>>> D.delete_vertex(Integer(0))
Traceback (most recent call last):
...
ValueError: vertex (0) not in the graph
```

```
sage: G = graphs.CompleteGraph(4).line_graph(labels=False)
sage: G.vertices(sort=True)
[(0, 1), (0, 2), (0, 3), (1, 2), (1, 3), (2, 3)]
sage: G.delete_vertex(0, in_order=True)
sage: G.vertices(sort=True)
[(0, 2), (0, 3), (1, 2), (1, 3), (2, 3)]
sage: G = graphs.PathGraph(5)
sage: G.set_vertices({0: 'no delete', 1: 'delete'})
sage: G.delete_vertex(1)
sage: G.get_vertices()
{0: 'no delete', 2: None, 3: None, 4: None}
sage: G.get_pos()
{0: (0, 0), 2: (2, 0), 3: (3, 0), 4: (4, 0)}
```

```
>>> from sage.all import *
>>> G = graphs.CompleteGraph(Integer(4)).line_graph(labels=False)
>>> G.vertices(sort=True)
[(0, 1), (0, 2), (0, 3), (1, 2), (1, 3), (2, 3)]
>>> G.delete_vertex(Integer(0), in_order=True)
>>> G.vertices(sort=True)
[(0, 2), (0, 3), (1, 2), (1, 3), (2, 3)]
>>> G = graphs.PathGraph(Integer(5))
>>> G.set_vertices({Integer(0): 'no delete', Integer(1): 'delete'})
>>> G.delete_vertex(Integer(1))
>>> G.get_vertices()
{0: 'no delete', 2: None, 3: None, 4: None}
>>> G.get_pos()
{0: (0, 0), 2: (2, 0), 3: (3, 0), 4: (4, 0)}
```

delete_vertices(*vertices*)

Delete vertices from the (di)graph taken from an iterable container of vertices.

Deleting a non-existent vertex will raise an exception, in which case none of the vertices in `vertices` is deleted.

EXAMPLES:

```
sage: D = DiGraph({0: [1, 2, 3, 4, 5], 1: [2], 2: [3], 3: [4], 4: [5], 5: [1]}
↪)
sage: D.delete_vertices([1, 2, 3, 4, 5]); D
Digraph on 1 vertex
sage: D.vertices(sort=False)
[0]
sage: D.delete_vertices([1])
Traceback (most recent call last):
...
ValueError: vertex (1) not in the graph
```

```
>>> from sage.all import *
>>> D = DiGraph({Integer(0): [Integer(1), Integer(2), Integer(3), Integer(4),␣
↪Integer(5)], Integer(1): [Integer(2)], Integer(2): [Integer(3)],␣
↪Integer(3): [Integer(4)], Integer(4): [Integer(5)], Integer(5):␣
↪[Integer(1)]})
>>> D.delete_vertices([Integer(1), Integer(2), Integer(3), Integer(4),␣
↪Integer(5)]); D
Digraph on 1 vertex
>>> D.vertices(sort=False)
[0]
>>> D.delete_vertices([Integer(1)])
Traceback (most recent call last):
...
ValueError: vertex (1) not in the graph
```

density()

Return the density of the (di)graph.

The density of a (di)graph is defined as the number of edges divided by number of possible edges.

In the case of a multigraph, raises an error, since there is an infinite number of possible edges.

EXAMPLES:

```
sage: d = {0: [1,4,5], 1: [2,6], 2: [3,7], 3: [4,8], 4: [9], 5: [7, 8], 6: [8,
↪9], 7: [9]}
sage: G = Graph(d); G.density()
1/3
sage: G = Graph({0: [1, 2], 1: [0]}); G.density()
2/3
sage: G = DiGraph({0: [1, 2], 1: [0]}); G.density()
1/2
```

```
>>> from sage.all import *
>>> d = {Integer(0): [Integer(1),Integer(4),Integer(5)], Integer(1):␣
↪[Integer(2),Integer(6)], Integer(2): [Integer(3),Integer(7)], Integer(3):␣
↪[Integer(4),Integer(8)], Integer(4): [Integer(9)], Integer(5): [Integer(7),␣
↪Integer(8)], Integer(6): [Integer(8),Integer(9)], Integer(7): [Integer(9)]}
>>> G = Graph(d); G.density()
1/3
>>> G = Graph({Integer(0): [Integer(1), Integer(2)], Integer(1): [Integer(0)]}
↪); G.density()
2/3
>>> G = DiGraph({Integer(0): [Integer(1), Integer(2)], Integer(1):␣
↪[Integer(0)]}); G.density()
1/2
```

Note that there are more possible edges on a looped graph:

```
sage: G.allow_loops(True)
sage: G.density()
1/3
```

```
>>> from sage.all import *
>>> G.allow_loops(True)
>>> G.density()
1/3
```

depth_first_search(*start*, *ignore_direction=False*, *neighbors=None*, *edges=False*, *forbidden_vertices=None*)

Return an iterator over the vertices in a depth-first ordering.

INPUT:

- `start` – vertex or list of vertices from which to start the traversal
- `ignore_direction` – boolean (default: `False`); only applies to directed graphs. If `True`, searches across edges in either direction.
- `neighbors` – function (default: `None`); a function that inputs a vertex and return a list of vertices. For an undirected graph, `neighbors` is by default the *`neighbors()`* function. For a digraph, the `neighbors` function defaults to the *`neighbor_out_iterator()`* function of the graph.
- `edges` – boolean (default: `False`); whether to return the edges of the DFS tree in the order of visit or the vertices (default). Edges are directed in root to leaf orientation of the tree.
- `forbidden_vertices` – list (default: `None`); set of vertices to avoid during the search. The start vertex `v` cannot be in this set.

See also

- *`breadth_first_search()`*
- *`breadth_first_search`* – breadth-first search for fast compiled graphs.
- *`depth_first_search`* – depth-first search for fast compiled graphs.

EXAMPLES:

```
sage: G = Graph({0: [1], 1: [2], 2: [3], 3: [4], 4: [0]})
sage: list(G.depth_first_search(0))
[0, 4, 3, 2, 1]
```

```
>>> from sage.all import *
>>> G = Graph({Integer(0): [Integer(1)], Integer(1): [Integer(2)],␣
↪Integer(2): [Integer(3)], Integer(3): [Integer(4)], Integer(4):␣
↪[Integer(0)]})
>>> list(G.depth_first_search(Integer(0)))
[0, 4, 3, 2, 1]
```

By default, the edge direction of a digraph is respected, but this can be overridden by the `ignore_direction` parameter:

```
sage: D = DiGraph({0: [1, 2, 3], 1: [4, 5], 2: [5], 3: [6], 5: [7], 6: [7],␣
↪7: [0]})
sage: list(D.depth_first_search(0))
[0, 3, 6, 7, 2, 5, 1, 4]
```

(continues on next page)

(continued from previous page)

```
sage: list(D.depth_first_search(0, ignore_direction=True))
[0, 7, 6, 3, 5, 2, 1, 4]
```

```
>>> from sage.all import *
>>> D = DiGraph({Integer(0): [Integer(1), Integer(2), Integer(3)],␣
↪Integer(1): [Integer(4), Integer(5)], Integer(2): [Integer(5)], Integer(3):␣
↪[Integer(6)], Integer(5): [Integer(7)], Integer(6): [Integer(7)],␣
↪Integer(7): [Integer(0)]})
>>> list(D.depth_first_search(Integer(0)))
[0, 3, 6, 7, 2, 5, 1, 4]
>>> list(D.depth_first_search(Integer(0), ignore_direction=True))
[0, 7, 6, 3, 5, 2, 1, 4]
```

Multiple starting vertices can be specified in a list:

```
sage: D = DiGraph({0: [1, 2, 3], 1: [4, 5], 2: [5], 3: [6], 5: [7], 6: [7],␣
↪7: [0]})
sage: list(D.depth_first_search([0]))
[0, 3, 6, 7, 2, 5, 1, 4]
sage: list(D.depth_first_search([0, 6]))
[0, 3, 6, 7, 2, 5, 1, 4]
```

```
>>> from sage.all import *
>>> D = DiGraph({Integer(0): [Integer(1), Integer(2), Integer(3)],␣
↪Integer(1): [Integer(4), Integer(5)], Integer(2): [Integer(5)], Integer(3):␣
↪[Integer(6)], Integer(5): [Integer(7)], Integer(6): [Integer(7)],␣
↪Integer(7): [Integer(0)]})
>>> list(D.depth_first_search([Integer(0)]))
[0, 3, 6, 7, 2, 5, 1, 4]
>>> list(D.depth_first_search([Integer(0), Integer(6)]))
[0, 3, 6, 7, 2, 5, 1, 4]
```

More generally, you can specify a `neighbors` function. For example, you can traverse the graph backwards by setting `neighbors` to be the *neighbors_in()* function of the graph:

```
sage: D = digraphs.Path(10)
sage: D.add_path([22, 23, 24, 5])
sage: D.add_path([5, 33, 34, 35])
sage: list(D.depth_first_search(5, neighbors=D.neighbors_in))
[5, 24, 23, 22, 4, 3, 2, 1, 0]
sage: list(D.breadth_first_search(5, neighbors=D.neighbors_in))
[5, 4, 24, 3, 23, 2, 22, 1, 0]
sage: list(D.depth_first_search(5, neighbors=D.neighbors_out))
[5, 33, 34, 35, 6, 7, 8, 9]
sage: list(D.breadth_first_search(5, neighbors=D.neighbors_out))
[5, 6, 33, 7, 34, 8, 35, 9]
```

```
>>> from sage.all import *
>>> D = digraphs.Path(Integer(10))
>>> D.add_path([Integer(22), Integer(23), Integer(24), Integer(5)])
>>> D.add_path([Integer(5), Integer(33), Integer(34), Integer(35)])
>>> list(D.depth_first_search(Integer(5), neighbors=D.neighbors_in))
```

(continues on next page)

(continued from previous page)

```
[5, 24, 23, 22, 4, 3, 2, 1, 0]
>>> list(D.breadth_first_search(Integer(5), neighbors=D.neighbors_in))
[5, 4, 24, 3, 23, 2, 22, 1, 0]
>>> list(D.depth_first_search(Integer(5), neighbors=D.neighbors_out))
[5, 33, 34, 35, 6, 7, 8, 9]
>>> list(D.breadth_first_search(Integer(5), neighbors=D.neighbors_out))
[5, 6, 33, 7, 34, 8, 35, 9]
```

You can get edges of the DFS tree instead of the vertices using the `edges` parameter:

```
sage: D = digraphs.Path(5)
sage: list(D.depth_first_search(2, edges=True))
[(2, 3), (3, 4)]
sage: list(D.depth_first_search(2, edges=True, ignore_direction=True))
[(2, 3), (3, 4), (2, 1), (1, 0)]
```

```
>>> from sage.all import *
>>> D = digraphs.Path(Integer(5))
>>> list(D.depth_first_search(Integer(2), edges=True))
[(2, 3), (3, 4)]
>>> list(D.depth_first_search(Integer(2), edges=True, ignore_direction=True))
[(2, 3), (3, 4), (2, 1), (1, 0)]
```

DFS in a graph with forbidden vertices:

```
sage: G = graphs.PetersenGraph()
sage: list(G.depth_first_search(0, forbidden_vertices=[1, 2]))
[0, 5, 8, 6, 9, 7, 4, 3]
```

```
>>> from sage.all import *
>>> G = graphs.PetersenGraph()
>>> list(G.depth_first_search(Integer(0), forbidden_vertices=[Integer(1),␣
↪Integer(2)]))
[0, 5, 8, 6, 9, 7, 4, 3]
```

disjoint_routed_paths(*pairs*, *solver*, *verbose=None*, *integrality_tolerance=0*)

Return a set of disjoint routed paths.

Given a set of pairs (s_i, t_i), a set of disjoint routed paths is a set of $s_i - t_i$ paths which can intersect at their endpoints and are vertex-disjoint otherwise.

INPUT:

- `pairs` – list of pairs of vertices
- `solver` – string (default: `None`); specifies a Mixed Integer Linear Programming (MILP) solver to be used. If set to `None`, the default one is used. For more information on MILP solvers and which default solver is used, see the method `solve` of the class `MixedIntegerLinearProgram`.
- `verbose` – integer (default: 0); sets the level of verbosity. Set to 0 by default, which means quiet.
- `integrality_tolerance` – float; parameter for use with MILP solvers over an inexact base ring; see `MixedIntegerLinearProgram.get_values()`.

EXAMPLES:

Given a grid, finding two vertex-disjoint paths, the first one from the top-left corner to the bottom-left corner, and the second from the top-right corner to the bottom-right corner is easy:

```
sage: g = graphs.Grid2dGraph(5, 5)
sage: p1,p2 = g.disjoint_routed_paths([((0, 0), (0, 4)), ((4, 4), (4, 0))])  ␣
↪          # needs sage.numerical.mip
```

```
>>> from sage.all import *
>>> g = graphs.Grid2dGraph(Integer(5), Integer(5))
>>> p1,p2 = g.disjoint_routed_paths([((Integer(0), Integer(0)), (Integer(0),␣
↪Integer(4))), ((Integer(4), Integer(4)), (Integer(4), Integer(0)))])       ␣
↪     # needs sage.numerical.mip
```

Though there is obviously no solution to the problem in which each corner is sending information to the opposite one:

```
sage: g = graphs.Grid2dGraph(5, 5)
sage: p1,p2 = g.disjoint_routed_paths([((0, 0), (4, 4)), ((0, 4), (4, 0))])  ␣
↪          # needs sage.numerical.mip
Traceback (most recent call last):
...
EmptySetError: the disjoint routed paths do not exist
```

```
>>> from sage.all import *
>>> g = graphs.Grid2dGraph(Integer(5), Integer(5))
>>> p1,p2 = g.disjoint_routed_paths([((Integer(0), Integer(0)), (Integer(4),␣
↪Integer(4))), ((Integer(0), Integer(4)), (Integer(4), Integer(0)))])       ␣
↪     # needs sage.numerical.mip
Traceback (most recent call last):
...
EmptySetError: the disjoint routed paths do not exist
```

disjoint_union(*other*, *labels='pairs'*, *immutable=None*)

Return the disjoint union of `self` and `other`.

INPUT:

- `labels` – string (default: `'pairs'`); if set to `'pairs'`, each element `v` in the first graph will be named `(0, v)` and each element `u` in `other` will be named `(1, u)` in the result. If set to `'integers'`, the elements of the result will be relabeled with consecutive integers.
- `immutable` – boolean (default: `None`); whether to create a mutable/immutable disjoint union. `immutable=None` (default) means that the graphs and their disjoint union will behave the same way.

See also

- *union()*
- *join()*

EXAMPLES:

```
sage: G = graphs.CycleGraph(3)
sage: H = graphs.CycleGraph(4)
```

(continues on next page)

(continued from previous page)

```
sage: J = G.disjoint_union(H); J
Cycle graph disjoint_union Cycle graph: Graph on 7 vertices
sage: J.vertices(sort=True)
[(0, 0), (0, 1), (0, 2), (1, 0), (1, 1), (1, 2), (1, 3)]
sage: J = G.disjoint_union(H, labels='integers'); J
Cycle graph disjoint_union Cycle graph: Graph on 7 vertices
sage: J.vertices(sort=True)
[0, 1, 2, 3, 4, 5, 6]
sage: (G + H).vertices(sort=True)  # '+'-operator is a shortcut
[0, 1, 2, 3, 4, 5, 6]
```

```
>>> from sage.all import *
>>> G = graphs.CycleGraph(Integer(3))
>>> H = graphs.CycleGraph(Integer(4))
>>> J = G.disjoint_union(H); J
Cycle graph disjoint_union Cycle graph: Graph on 7 vertices
>>> J.vertices(sort=True)
[(0, 0), (0, 1), (0, 2), (1, 0), (1, 1), (1, 2), (1, 3)]
>>> J = G.disjoint_union(H, labels='integers'); J
Cycle graph disjoint_union Cycle graph: Graph on 7 vertices
>>> J.vertices(sort=True)
[0, 1, 2, 3, 4, 5, 6]
>>> (G + H).vertices(sort=True)  # '+'-operator is a shortcut
[0, 1, 2, 3, 4, 5, 6]
```

```
sage: G = Graph({'a': ['b']})
sage: G.name("Custom path")
sage: G.name()
'Custom path'
sage: H = graphs.CycleGraph(3)
sage: J = G.disjoint_union(H); J
Custom path disjoint_union Cycle graph: Graph on 5 vertices
sage: J.vertices(sort=True)
[(0, 'a'), (0, 'b'), (1, 0), (1, 1), (1, 2)]
```

```
>>> from sage.all import *
>>> G = Graph({'a': ['b']})
>>> G.name("Custom path")
>>> G.name()
'Custom path'
>>> H = graphs.CycleGraph(Integer(3))
>>> J = G.disjoint_union(H); J
Custom path disjoint_union Cycle graph: Graph on 5 vertices
>>> J.vertices(sort=True)
[(0, 'a'), (0, 'b'), (1, 0), (1, 1), (1, 2)]
```

disjunctive_product(*other*, *immutable=None*)

Return the disjunctive product of `self` and `other`.

The disjunctive product of G and H is the graph L with vertex set $V(L) = V(G) \times V(H)$, and $((u, v), (w, x))$ is an edge iff either :

- (u, w) is an edge of G, or

- (v, x) is an edge of H.

INPUT:

- `other` – a graph or a digraph
- `immutable` – boolean (default: `None`); whether to create a mutable/immutable product. `immutable=None` (default) means that the graphs and their product will behave the same way. If only one of them is immutable, the product will be mutable.

EXAMPLES:

```
sage: Z = graphs.CompleteGraph(2)
sage: D = Z.disjunctive_product(Z); D
Graph on 4 vertices
sage: D.plot()                          # long time                          #␣
↪needs sage.plot
Graphics object consisting of 11 graphics primitives
```

```
>>> from sage.all import *
>>> Z = graphs.CompleteGraph(Integer(2))
>>> D = Z.disjunctive_product(Z); D
Graph on 4 vertices
>>> D.plot()                          # long time                          #␣
↪needs sage.plot
Graphics object consisting of 11 graphics primitives
```

```
sage: C = graphs.CycleGraph(5)
sage: D = C.disjunctive_product(Z); D
Graph on 10 vertices
sage: D.plot()                          # long time                          #␣
↪needs sage.plot
Graphics object consisting of 46 graphics primitives
```

```
>>> from sage.all import *
>>> C = graphs.CycleGraph(Integer(5))
>>> D = C.disjunctive_product(Z); D
Graph on 10 vertices
>>> D.plot()                          # long time                          #␣
↪needs sage.plot
Graphics object consisting of 46 graphics primitives
```

distance(*u*, *v*, *by_weight=False*, *weight_function=None*, *check_weight=True*)

Return the (directed) distance from `u` to `v` in the (di)graph.

The distance is the length of the shortest path from `u` to `v`.

This method simply calls *shortest_path_length()*, with default arguments. For more information, and for more option, we refer to that method.

INPUT:

- `by_weight` – boolean (default: `False`); if `False`, the graph is considered unweighted, and the distance is the number of edges in a shortest path. If `True`, the distance is the sum of edge labels (which are assumed to be numbers).

- `weight_function` – function (default: `None`); a function that takes as input an edge `(u, v, l)` and outputs its weight. If not `None`, `by_weight` is automatically set to `True`. If `None` and `by_weight` is `True`, we use the edge label `l`, if `l` is not `None`, else `1` as a weight.
- `check_weight` – boolean (default: `True`); whether to check that the `weight_function` outputs a number for each edge

EXAMPLES:

```
sage: G = graphs.CycleGraph(9)
sage: G.distance(0,1)
1
sage: G.distance(0,4)
4
sage: G.distance(0,5)
4
sage: G = Graph({0:[], 1:[]})
sage: G.distance(0,1)
+Infinity
sage: G = Graph({ 0: {1: 1}, 1: {2: 1}, 2: {3: 1}, 3: {4: 2}, 4: {0: 2}},␣
↪sparse = True)
sage: G.plot(edge_labels=True).show()        # long time                      #␣
↪needs sage.plot
sage: G.distance(0, 3)
2
sage: G.distance(0, 3, by_weight=True)
3
```

```
>>> from sage.all import *
>>> G = graphs.CycleGraph(Integer(9))
>>> G.distance(Integer(0),Integer(1))
1
>>> G.distance(Integer(0),Integer(4))
4
>>> G.distance(Integer(0),Integer(5))
4
>>> G = Graph({Integer(0):[], Integer(1):[]})
>>> G.distance(Integer(0),Integer(1))
+Infinity
>>> G = Graph({ Integer(0): {Integer(1): Integer(1)}, Integer(1):
↪{Integer(2): Integer(1)}, Integer(2): {Integer(3): Integer(1)}, Integer(3):
↪{Integer(4): Integer(2)}, Integer(4): {Integer(0): Integer(2)}}, sparse =␣
↪True)
>>> G.plot(edge_labels=True).show()        # long time                      #␣
↪needs sage.plot
>>> G.distance(Integer(0), Integer(3))
2
>>> G.distance(Integer(0), Integer(3), by_weight=True)
3
```

distance_all_pairs(*by_weight=False*, *algorithm=None*, *weight_function=None*, *check_weight=True*)

Return the distances between all pairs of vertices.

INPUT:

- `by_weight` – boolean (default: `False`); if `True`, the edges in the graph are weighted. If `False`, all

edges have weight 1.

- `algorithm` – string (default: `None`); one of the following algorithms:
  - `'BFS'`: the computation is done through a BFS centered on each vertex successively. Works only if `by_weight==False`.
  - `'Floyd-Warshall-Cython'`: the Cython implementation of the Floyd-Warshall algorithm. Works only if `by_weight==False`.
  - `'Floyd-Warshall-Python'`: the Python implementation of the Floyd-Warshall algorithm. Works also with weighted graphs, even with negative weights (but no negative cycle is allowed).
  - `'Dijkstra_NetworkX'`: the Dijkstra algorithm, implemented in NetworkX. It works with weighted graphs, but no negative weight is allowed.
  - `'Dijkstra_Boost'`: the Dijkstra algorithm, implemented in Boost (works only with positive weights).
  - `'Johnson_Boost'`: the Johnson algorithm, implemented in Boost (works also with negative weights, if there is no negative cycle).
  - `None` (default): Sage chooses the best algorithm: `'BFS'` if `by_weight` is `False`, `'Dijkstra_Boost'` if all weights are positive, `'Floyd-Warshall-Cython'` otherwise.
- `weight_function` – function (default: `None`); a function that takes as input an edge `(u, v, l)` and outputs its weight. If not `None`, `by_weight` is automatically set to `True`. If `None` and `by_weight` is `True`, we use the edge label `l`, if `l` is not `None`, else `1` as a weight.
- `check_weight` – boolean (default: `True`); whether to check that the `weight_function` outputs a number for each edge

OUTPUT: a doubly indexed dictionary

Note

There is a Cython version of this method that is usually much faster for large graphs, as most of the time is actually spent building the final double dictionary. Everything on the subject is to be found in the *distances_all_pairs* module.

Note

This algorithm simply calls *GenericGraph.shortest_path_all_pairs()*, and we suggest to look at that method for more information and examples.

EXAMPLES:

The Petersen Graph:

```
sage: g = graphs.PetersenGraph()
sage: print(g.distance_all_pairs())
{0: {0: 0, 1: 1, 2: 2, 3: 2, 4: 1, 5: 1, 6: 2, 7: 2, 8: 2, 9: 2},
 1: {0: 1, 1: 0, 2: 1, 3: 2, 4: 2, 5: 2, 6: 1, 7: 2, 8: 2, 9: 2},
 2: {0: 2, 1: 1, 2: 0, 3: 1, 4: 2, 5: 2, 6: 2, 7: 1, 8: 2, 9: 2},
 3: {0: 2, 1: 2, 2: 1, 3: 0, 4: 1, 5: 2, 6: 2, 7: 2, 8: 1, 9: 2},
 4: {0: 1, 1: 2, 2: 2, 3: 1, 4: 0, 5: 2, 6: 2, 7: 2, 8: 2, 9: 1},
 5: {0: 1, 1: 2, 2: 2, 3: 2, 4: 2, 5: 0, 6: 2, 7: 1, 8: 1, 9: 2},
```

(continues on next page)

(continued from previous page)

```
 6: {0: 2, 1: 1, 2: 2, 3: 2, 4: 2, 5: 2, 6: 0, 7: 2, 8: 1, 9: 1},
 7: {0: 2, 1: 2, 2: 1, 3: 2, 4: 2, 5: 1, 6: 2, 7: 0, 8: 2, 9: 1},
 8: {0: 2, 1: 2, 2: 2, 3: 1, 4: 2, 5: 1, 6: 1, 7: 2, 8: 0, 9: 2},
 9: {0: 2, 1: 2, 2: 2, 3: 2, 4: 1, 5: 2, 6: 1, 7: 1, 8: 2, 9: 0}}
```

```
>>> from sage.all import *
>>> g = graphs.PetersenGraph()
>>> print(g.distance_all_pairs())
{0: {0: 0, 1: 1, 2: 2, 3: 2, 4: 1, 5: 1, 6: 2, 7: 2, 8: 2, 9: 2},
 1: {0: 1, 1: 0, 2: 1, 3: 2, 4: 2, 5: 2, 6: 1, 7: 2, 8: 2, 9: 2},
 2: {0: 2, 1: 1, 2: 0, 3: 1, 4: 2, 5: 2, 6: 2, 7: 1, 8: 2, 9: 2},
 3: {0: 2, 1: 2, 2: 1, 3: 0, 4: 1, 5: 2, 6: 2, 7: 2, 8: 1, 9: 2},
 4: {0: 1, 1: 2, 2: 2, 3: 1, 4: 0, 5: 2, 6: 2, 7: 2, 8: 2, 9: 1},
 5: {0: 1, 1: 2, 2: 2, 3: 2, 4: 2, 5: 0, 6: 2, 7: 1, 8: 1, 9: 2},
 6: {0: 2, 1: 1, 2: 2, 3: 2, 4: 2, 5: 2, 6: 0, 7: 2, 8: 1, 9: 1},
 7: {0: 2, 1: 2, 2: 1, 3: 2, 4: 2, 5: 1, 6: 2, 7: 0, 8: 2, 9: 1},
 8: {0: 2, 1: 2, 2: 2, 3: 1, 4: 2, 5: 1, 6: 1, 7: 2, 8: 0, 9: 2},
 9: {0: 2, 1: 2, 2: 2, 3: 2, 4: 1, 5: 2, 6: 1, 7: 1, 8: 2, 9: 0}}
```

Testing on Random Graphs:

```
sage: g = graphs.RandomGNP(20,.3)
sage: distances = g.distance_all_pairs()
sage: all((g.distance(0,v) == Infinity and v not in distances[0]) or
....:     g.distance(0,v) == distances[0][v] for v in g)
True
```

```
>>> from sage.all import *
>>> g = graphs.RandomGNP(Integer(20),RealNumber('.3'))
>>> distances = g.distance_all_pairs()
>>> all((g.distance(Integer(0),v) == Infinity and v not in␣
↪distances[Integer(0)]) or
...     g.distance(Integer(0),v) == distances[Integer(0)][v] for v in g)
True
```

See also

- *distance_matrix()*
- *shortest_path_all_pairs()*

distance_matrix(*vertices*, *base_ring=None*, ***kwds*)

Return the distance matrix of (di)graph.

The (di)graph is expected to be (strongly) connected.

The distance matrix of a (strongly) connected (di)graph is a matrix whose rows and columns are by default (`vertices == None`) indexed with the positions of the vertices of the (di)graph in the ordering *vertices()*. When `vertices` is set, the position of the vertices in this ordering is used. The intersection of row i and column j contains the shortest path distance from the vertex at the i-th position to the vertex at the j-th position.

Note that even when the vertices are consecutive integers starting from one, usually the vertex is not equal to its index.

INPUT:

- `vertices` – list (default: `None`); the ordering of the vertices defining how they should appear in the matrix. By default, the ordering given by *`vertices()`* is used. Because *`vertices()`* only works if the vertices can be sorted, using `vertices` is useful when working with possibly non-sortable objects in Python 3.
- `base_ring` – a ring (default: determined from the weights); the base ring of the matrix space to use
- `**kwds` – other keywords to pass to the subfunction *`distance_all_pairs()`* or to `matrix()`

EXAMPLES:

```
sage: d = DiGraph({1: [2, 3], 2: [3], 3: [4], 4: [1]})
sage: d.distance_matrix()                                                   #␣
↪needs sage.modules
[0 1 1 2]
[3 0 1 2]
[2 3 0 1]
[1 2 2 0]
sage: d.distance_matrix(vertices=[4, 3, 2, 1])                              #␣
↪needs sage.modules
[0 2 2 1]
[1 0 3 2]
[2 1 0 3]
[2 1 1 0]

sage: G = graphs.CubeGraph(3)
sage: G.distance_matrix()                                                   #␣
↪needs sage.modules
[0 1 1 2 1 2 2 3]
[1 0 2 1 2 1 3 2]
[1 2 0 1 2 3 1 2]
[2 1 1 0 3 2 2 1]
[1 2 2 3 0 1 1 2]
[2 1 3 2 1 0 2 1]
[2 3 1 2 1 2 0 1]
[3 2 2 1 2 1 1 0]
```

```
>>> from sage.all import *
>>> d = DiGraph({Integer(1): [Integer(2), Integer(3)], Integer(2):␣
↪[Integer(3)], Integer(3): [Integer(4)], Integer(4): [Integer(1)]})
>>> d.distance_matrix()                                                     #␣
↪needs sage.modules
[0 1 1 2]
[3 0 1 2]
[2 3 0 1]
[1 2 2 0]
>>> d.distance_matrix(vertices=[Integer(4), Integer(3), Integer(2),␣
↪Integer(1)])                                  # needs sage.modules
[0 2 2 1]
[1 0 3 2]
[2 1 0 3]
```

(continues on next page)

(continued from previous page)

```
[2 1 1 0]

>>> G = graphs.CubeGraph(Integer(3))
>>> G.distance_matrix()                                                    #␣
↪needs sage.modules
[0 1 1 2 1 2 2 3]
[1 0 2 1 2 1 3 2]
[1 2 0 1 2 3 1 2]
[2 1 1 0 3 2 2 1]
[1 2 2 3 0 1 1 2]
[2 1 3 2 1 0 2 1]
[2 3 1 2 1 2 0 1]
[3 2 2 1 2 1 1 0]
```

The well known result of Graham and Pollak states that the determinant of the distance matrix of any tree of order n is $(-1)^{n-1}(n-1)2^{n-2}$:

```
sage: all(T.distance_matrix().det() == (-1)^9*(9)*2^8                     #␣
↪needs sage.modules
....:     for T in graphs.trees(10))
True
```

```
>>> from sage.all import *
>>> all(T.distance_matrix().det() == (-
↪Integer(1))**Integer(9)*(Integer(9))*Integer(2)**Integer(8)              ␣
↪       # needs sage.modules
...     for T in graphs.trees(Integer(10)))
True
```

See also

- *distance_all_pairs()* – computes the distance between any two vertices.

distances_distribution(G)

Return the distances distribution of the (di)graph in a dictionary.

This method *ignores all edge labels*, so that the distance considered is the topological distance.

OUTPUT:

A dictionary `d` such that the number of pairs of vertices at distance `k` (if any) is equal to $d[k] \cdot |V(G)| \cdot (|V(G)| - 1)$.

Note

We consider that two vertices that do not belong to the same connected component are at infinite distance, and we do not take the trivial pairs of vertices (v, v) at distance 0 into account. Empty (di)graphs and (di)graphs of order 1 have no paths and so we return the empty dictionary `{}`.

EXAMPLES:

An empty Graph:

```
sage: g = Graph()
sage: g.distances_distribution()
{}
```

```
>>> from sage.all import *
>>> g = Graph()
>>> g.distances_distribution()
{}
```

A Graph of order 1:

```
sage: g = Graph()
sage: g.add_vertex(1)
sage: g.distances_distribution()
{}
```

```
>>> from sage.all import *
>>> g = Graph()
>>> g.add_vertex(Integer(1))
>>> g.distances_distribution()
{}
```

A Graph of order 2 without edge:

```
sage: g = Graph()
sage: g.add_vertices([1,2])
sage: g.distances_distribution()
{+Infinity: 1}
```

```
>>> from sage.all import *
>>> g = Graph()
>>> g.add_vertices([Integer(1),Integer(2)])
>>> g.distances_distribution()
{+Infinity: 1}
```

The Petersen Graph:

```
sage: g = graphs.PetersenGraph()
sage: g.distances_distribution()
{1: 1/3, 2: 2/3}
```

```
>>> from sage.all import *
>>> g = graphs.PetersenGraph()
>>> g.distances_distribution()
{1: 1/3, 2: 2/3}
```

A graph with multiple disconnected components:

```
sage: g = graphs.PetersenGraph()
sage: g.add_edge('good','wine')
sage: g.distances_distribution()
{1: 8/33, 2: 5/11, +Infinity: 10/33}
```

```
>>> from sage.all import *
>>> g = graphs.PetersenGraph()
>>> g.add_edge('good','wine')
>>> g.distances_distribution()
{1: 8/33, 2: 5/11, +Infinity: 10/33}
```

The de Bruijn digraph dB(2,3):

```
sage: D = digraphs.DeBruijn(2,3)
↪  # needs sage.combinat
sage: D.distances_distribution()
↪  # needs sage.combinat
{1: 1/4, 2: 11/28, 3: 5/14}
```

```
>>> from sage.all import *
>>> D = digraphs.DeBruijn(Integer(2),Integer(3))
↪                  # needs sage.combinat
>>> D.distances_distribution()
↪# needs sage.combinat
{1: 1/4, 2: 11/28, 3: 5/14}
```

dominating_set(*g*, *k*, *independent=1*, *total=False*, *connected=False*, *value_only=False*, *solver=False*, *verbose=None*, *integrality_tolerance=0*)

Return a minimum distance-k dominating set of the graph.

A minimum dominating set S of a graph G is a set of its vertices of minimal cardinality such that any vertex of G is in S or has one of its neighbors in S. See the Wikipedia article Dominating_set.

A minimum distance-k dominating set is a set S of vertices of G of minimal cardinality such that any vertex of G is in S or at distance at most k from a vertex in S. A distance-0 dominating set is the set of vertices itself, and when k is the radius of the graph, any vertex dominates all the other vertices.

As an optimization problem, it can be expressed as follows, where $N^k(u)$ denotes the set of vertices at distance at most k from u (the set of neighbors when $k = 1$):

$$\text{Minimize : } \sum_{v \in G} b_v$$

$$\text{Such that : } \forall v \in G, b_v + \sum_{u \in N^k(v)} b_u \geq 1$$

$$\forall x \in G, b_x \text{ is a binary variable}$$

INPUT:

- `k` – nonnegative integer (default: 1); the domination distance
- `independent` – boolean (default: `False`); when `True`, computes a minimum independent dominating set, that is a minimum dominating set that is also an independent set (see also *`independent_set()`*)
- `total` – boolean (default: `False`); when `True`, computes a total dominating set (see the See the Wikipedia article Dominating_set)
- `connected` – boolean (default: `False`); when `True`, computes a connected dominating set (see Wikipedia article Connected_dominating_set)
- `value_only` – boolean (default: `False`); whether to only return the cardinality of the computed dominating set, or to return its list of vertices (default)

- `solver` – string (default: `None`); specifies a Mixed Integer Linear Programming (MILP) solver to be used. If set to `None`, the default one is used. For more information on MILP solvers and which default solver is used, see the method `solve` of the class `MixedIntegerLinearProgram`.
- `verbose` – integer (default: 0); sets the level of verbosity. Set to 0 by default, which means quiet.
- `integrality_tolerance` – float; parameter for use with MILP solvers over an inexact base ring; see `MixedIntegerLinearProgram.get_values()`.

EXAMPLES:

A basic illustration on a `PappusGraph`:

```
sage: g = graphs.PappusGraph()
sage: g.dominating_set(value_only=True)                                    ␣
↪  # needs sage.numerical.mip
5
```

```
>>> from sage.all import *
>>> g = graphs.PappusGraph()
>>> g.dominating_set(value_only=True)
↪# needs sage.numerical.mip
5
```

If we build a graph from two disjoint stars, then link their centers we will find a difference between the cardinality of an independent set and a stable independent set:

```
sage: g = 2 * graphs.StarGraph(5)
sage: g.add_edge(0, 6)
sage: len(g.dominating_set())                                              ␣
↪  # needs sage.numerical.mip
2
sage: len(g.dominating_set(independent=True))                              ␣
↪  # needs sage.numerical.mip
6
```

```
>>> from sage.all import *
>>> g = Integer(2) * graphs.StarGraph(Integer(5))
>>> g.add_edge(Integer(0), Integer(6))
>>> len(g.dominating_set())
↪# needs sage.numerical.mip
2
>>> len(g.dominating_set(independent=True))
↪# needs sage.numerical.mip
6
```

The total dominating set of the Petersen graph has cardinality 4:

```
sage: G = graphs.PetersenGraph()
sage: G.dominating_set(total=True, value_only=True)                        ␣
↪  # needs sage.numerical.mip
4
```

```
>>> from sage.all import *
>>> G = graphs.PetersenGraph()
```

(continues on next page)

(continued from previous page)

```
>>> G.dominating_set(total=True, value_only=True)
↪# needs sage.numerical.mip
4
```

The dominating set is calculated for both the directed and undirected graphs (modification introduced in Issue #17905):

```
sage: g = digraphs.Path(3)
sage: g.dominating_set(value_only=True)                                    ␣
↪  # needs sage.numerical.mip
2
sage: g = graphs.PathGraph(3)
sage: g.dominating_set(value_only=True)                                    ␣
↪  # needs sage.numerical.mip
1
```

```
>>> from sage.all import *
>>> g = digraphs.Path(Integer(3))
>>> g.dominating_set(value_only=True)
↪# needs sage.numerical.mip
2
>>> g = graphs.PathGraph(Integer(3))
>>> g.dominating_set(value_only=True)
↪# needs sage.numerical.mip
1
```

Cardinality of distance-k dominating sets:

```
sage: G = graphs.PetersenGraph()
sage: [G.dominating_set(k=k, value_only=True) for k in range(G.radius() + 1)]␣
↪  # needs sage.numerical.mip
[10, 3, 1]
sage: G = graphs.PathGraph(5)
sage: [G.dominating_set(k=k, value_only=True) for k in range(G.radius() + 1)]␣
↪  # needs sage.numerical.mip
[5, 2, 1]
```

```
>>> from sage.all import *
>>> G = graphs.PetersenGraph()
>>> [G.dominating_set(k=k, value_only=True) for k in range(G.radius() +␣
↪Integer(1))]   # needs sage.numerical.mip
[10, 3, 1]
>>> G = graphs.PathGraph(Integer(5))
>>> [G.dominating_set(k=k, value_only=True) for k in range(G.radius() +␣
↪Integer(1))]   # needs sage.numerical.mip
[5, 2, 1]
```

dominating_sets(*g*, *k*, *independent=1*, *total=False*, *connected=False*, *solver=False*, *verbose=None*, *integrality_tolerance=0*)

Return an iterator over the minimum distance-k dominating sets of the graph.

A minimum dominating set S of a graph G is a set of its vertices of minimal cardinality such that any vertex of G is in S or has one of its neighbors in S. See the Wikipedia article Dominating_set.

A minimum distance-k dominating set is a set S of vertices of G of minimal cardinality such that any vertex of G is in S or at distance at most k from a vertex in S. A distance-0 dominating set is the set of vertices itself, and when k is the radius of the graph, any vertex dominates all the other vertices.

As an optimization problem, it can be expressed as follows, where $N^k(u)$ denotes the set of vertices at distance at most k from u (the set of neighbors when $k = 1$):

$$\begin{aligned}\text{Minimize : } & \sum_{v \in G} b_v \\ \text{Such that : } \forall v \in G, b_v + & \sum_{u \in N^k(v)} b_u \geq 1 \\ \forall x \in G, & b_x \text{ is a binary variable}\end{aligned}$$

We use constraints generation to iterate over the minimum distance-k dominating sets. That is, after reporting a solution, we add a constraint to discard it and solve the problem again until no more solution can be found.

INPUT:

- `k` – nonnegative integer (default: 1); the domination distance
- `independent` – boolean (default: `False`); when `True`, computes minimum independent dominating sets, that is minimum dominating sets that are also independent sets (see also *`independent_set()`*)
- `total` – boolean (default: `False`); when `True`, computes total dominating sets (see the See the Wikipedia article Dominating_set)
- `connected` – boolean (default: `False`); when `True`, computes connected dominating sets (see Wikipedia article Connected_dominating_set)
- `solver` – string (default: `None`); specifies a Mixed Integer Linear Programming (MILP) solver to be used. If set to `None`, the default one is used. For more information on MILP solvers and which default solver is used, see the method `solve` of the class `MixedIntegerLinearProgram`.
- `verbose` – integer (default: 0); sets the level of verbosity. Set to 0 by default, which means quiet.
- `integrality_tolerance` – float; parameter for use with MILP solvers over an inexact base ring; see `MixedIntegerLinearProgram.get_values()`.

EXAMPLES:

Number of distance-k dominating sets of a Path graph of order 10:

```
sage: g = graphs.PathGraph(10)
sage: [sum(1 for _ in g.dominating_sets(k=k)) for k in range(11)]
↪  # needs sage.numerical.mip
[1, 13, 1, 13, 25, 2, 4, 6, 8, 10, 10]
```

```
>>> from sage.all import *
>>> g = graphs.PathGraph(Integer(10))
>>> [sum(Integer(1) for _ in g.dominating_sets(k=k)) for k in
↪range(Integer(11))]                # needs sage.numerical.mip
[1, 13, 1, 13, 25, 2, 4, 6, 8, 10, 10]
```

If we build a graph from two disjoint stars, then link their centers we will find a difference between the cardinality of an independent set and a stable independent set:

```
sage: g = 2 * graphs.StarGraph(5)
sage: g.add_edge(0, 6)
```

(continues on next page)

(continued from previous page)

```
sage: [sum(1 for _ in g.dominating_sets(k=k)) for k in range(11)]
↪  # needs sage.numerical.mip
[1, 1, 2, 12, 12, 12, 12, 12, 12, 12, 12]
```

```
>>> from sage.all import *
>>> g = Integer(2) * graphs.StarGraph(Integer(5))
>>> g.add_edge(Integer(0), Integer(6))
>>> [sum(Integer(1) for _ in g.dominating_sets(k=k)) for k in
↪range(Integer(11))]                # needs sage.numerical.mip
[1, 1, 2, 12, 12, 12, 12, 12, 12, 12, 12]
```

The total dominating set of the Petersen graph has cardinality 4:

```
sage: G = graphs.PetersenGraph()
sage: G.dominating_set(total=True, value_only=True)
↪  # needs sage.numerical.mip
4
sage: sorted(sorted(d) for d in G.dominating_sets(k=1))
↪  # needs sage.numerical.mip
[[0, 2, 6],
 [0, 3, 9],
 [0, 7, 8],
 [1, 3, 7],
 [1, 4, 5],
 [1, 8, 9],
 [2, 4, 8],
 [2, 5, 9],
 [3, 5, 6],
 [4, 6, 7]]
```

```
>>> from sage.all import *
>>> G = graphs.PetersenGraph()
>>> G.dominating_set(total=True, value_only=True)
↪# needs sage.numerical.mip
4
>>> sorted(sorted(d) for d in G.dominating_sets(k=Integer(1)))
↪          # needs sage.numerical.mip
[[0, 2, 6],
 [0, 3, 9],
 [0, 7, 8],
 [1, 3, 7],
 [1, 4, 5],
 [1, 8, 9],
 [2, 4, 8],
 [2, 5, 9],
 [3, 5, 6],
 [4, 6, 7]]
```

Independent distance-k dominating sets of a Path graph:

```
sage: G = graphs.PathGraph(6)
sage: sorted(sorted(d) for d in G.dominating_sets(k=1, independent=True))
```

(continues on next page)

(continued from previous page)

```
[[1, 4]]
sage: sorted(sorted(d) for d in G.dominating_sets(k=2, independent=True))
[[0, 3], [0, 4], [0, 5], [1, 3], [1, 4], [1, 5], [2, 4], [2, 5]]
sage: sorted(G.dominating_sets(k=3, independent=True))
[[2], [3]]
```

```
>>> from sage.all import *
>>> G = graphs.PathGraph(Integer(6))
>>> sorted(sorted(d) for d in G.dominating_sets(k=Integer(1),␣
↪independent=True))
[[1, 4]]
>>> sorted(sorted(d) for d in G.dominating_sets(k=Integer(2),␣
↪independent=True))
[[0, 3], [0, 4], [0, 5], [1, 3], [1, 4], [1, 5], [2, 4], [2, 5]]
>>> sorted(G.dominating_sets(k=Integer(3), independent=True))
[[2], [3]]
```

The dominating set is calculated for both the directed and undirected graphs (modification introduced in Issue #17905):

```
sage: g = digraphs.Path(3)
sage: g.dominating_set(value_only=True)
2
sage: sorted(sorted(d) for d in g.dominating_sets())
[[0, 1], [0, 2]]
sage: list(g.dominating_sets(k=2))
[[0]]
sage: g = graphs.PathGraph(3)
sage: g.dominating_set(value_only=True)
1
sage: next(g.dominating_sets())
[1]
```

```
>>> from sage.all import *
>>> g = digraphs.Path(Integer(3))
>>> g.dominating_set(value_only=True)
2
>>> sorted(sorted(d) for d in g.dominating_sets())
[[0, 1], [0, 2]]
>>> list(g.dominating_sets(k=Integer(2)))
[[0]]
>>> g = graphs.PathGraph(Integer(3))
>>> g.dominating_set(value_only=True)
1
>>> next(g.dominating_sets())
[1]
```

Minimum connected dominating sets of the Petersen graph:

```
sage: G = graphs.PetersenGraph()
sage: G.dominating_set(total=True, value_only=True)                    ␣
↪  # needs sage.numerical.mip
```

(continues on next page)

(continued from previous page)

```
4
sage: sorted(sorted(d) for d in G.dominating_sets(k=1, connected=True))
↪  # needs sage.numerical.mip
[[0, 1, 2, 6],
 [0, 1, 4, 5],
 [0, 3, 4, 9],
 [0, 5, 7, 8],
 [1, 2, 3, 7],
 [1, 6, 8, 9],
 [2, 3, 4, 8],
 [2, 5, 7, 9],
 [3, 5, 6, 8],
 [4, 6, 7, 9]]
```

```
>>> from sage.all import *
>>> G = graphs.PetersenGraph()
>>> G.dominating_set(total=True, value_only=True)
↪# needs sage.numerical.mip
4
>>> sorted(sorted(d) for d in G.dominating_sets(k=Integer(1),
↪connected=True))          # needs sage.numerical.mip
[[0, 1, 2, 6],
 [0, 1, 4, 5],
 [0, 3, 4, 9],
 [0, 5, 7, 8],
 [1, 2, 3, 7],
 [1, 6, 8, 9],
 [2, 3, 4, 8],
 [2, 5, 7, 9],
 [3, 5, 6, 8],
 [4, 6, 7, 9]]
```

Subgraph induced by the dominating set is connected:

```
sage: G = graphs.PetersenGraph()
sage: all(G.subgraph(vertices=dom).is_connected()
....:     for dom in G.dominating_set(k=1, connected=True))
True
```

```
>>> from sage.all import *
>>> G = graphs.PetersenGraph()
>>> all(G.subgraph(vertices=dom).is_connected()
...     for dom in G.dominating_set(k=Integer(1), connected=True))
True
```

Minimum distance-k connected dominating sets of the Tietze graph:

```
sage: G = graphs.TietzeGraph()
sage: sorted(sorted(d) for d in G.dominating_sets(k=2, connected=True))
[[0, 1], [0, 8], [0, 9], [2, 3], [3, 4], [3, 10], [5, 6], [6, 7], [6, 11]]
sage: sorted(G.dominating_sets(k=3, connected=True))
[[0], [1], [2], [3], [4], [5], [6], [7], [8], [9], [10], [11]]
```

```
>>> from sage.all import *
>>> G = graphs.TietzeGraph()
>>> sorted(sorted(d) for d in G.dominating_sets(k=Integer(2), connected=True))
[[0, 1], [0, 8], [0, 9], [2, 3], [3, 4], [3, 10], [5, 6], [6, 7], [6, 11]]
>>> sorted(G.dominating_sets(k=Integer(3), connected=True))
[[0], [1], [2], [3], [4], [5], [6], [7], [8], [9], [10], [11]]
```

dominator_tree(*g*, *root*, *return_dict=False*, *reverse=False*)

Use Boost to compute the dominator tree of `g`, rooted at `root`.

A node d dominates a node n if every path from the entry node `root` to n must go through d. The immediate dominator of a node n is the unique node that strictly dominates n but does not dominate any other node that dominates n. A dominator tree is a tree where each node's children are those nodes it immediately dominates. For more information, see the Wikipedia article Dominator_(graph_theory).

If the graph is connected and undirected, the parent of a vertex v is:

- the root if v is in the same biconnected component as the root;
- the first cut vertex in a path from v to the root, otherwise.

If the graph is not connected, the dominator tree of the whole graph is equal to the dominator tree of the connected component of the root.

If the graph is directed, computing a dominator tree is more complicated, and it needs time $O(m \log m)$, where m is the number of edges. The implementation provided by Boost is the most general one, so it needs time $O(m \log m)$ even for undirected graphs.

INPUT:

- `g` – the input Sage (Di)Graph
- `root` – the root of the dominator tree
- `return_dict` – boolean (default: `False`); if `True`, the function returns a dictionary associating to each vertex its parent in the dominator tree. If `False` (default), it returns the whole tree, as a `Graph` or a `DiGraph`.
- `reverse` – boolean (default: `False`); when set to `True`, computes the dominator tree in the reverse graph

OUTPUT:

The dominator tree, as a graph or as a dictionary, depending on the value of `return_dict`. If the output is a dictionary, it will contain `None` in correspondence of `root` and of vertices that are not reachable from `root`. If the output is a graph, it will not contain vertices that are not reachable from `root`.

EXAMPLES:

An undirected grid is biconnected, and its dominator tree is a star (everyone's parent is the root):

```
sage: g = graphs.GridGraph([2,2]).dominator_tree((0,0))
sage: g.to_dictionary()
{(0, 0): [(0, 1), (1, 0), (1, 1)], (0, 1): [(0, 0)], (1, 0): [(0, 0)], (1,␣
↪1): [(0, 0)]}
```

```
>>> from sage.all import *
>>> g = graphs.GridGraph([Integer(2),Integer(2)]).dominator_tree((Integer(0),
↪Integer(0)))
>>> g.to_dictionary()
```

(continues on next page)

(continued from previous page)

```
{(0, 0): [(0, 1), (1, 0), (1, 1)], (0, 1): [(0, 0)], (1, 0): [(0, 0)], (1,␣
↪1): [(0, 0)]}
```

If the graph is made by two 3-cycles C_1, C_2 connected by an edge (v, w), with $v \in C_1$, $w \in C_2$, the cut vertices are v and w, the biconnected components are C_1, C_2, and the edge (v, w). If the root is in C_1, the parent of each vertex in C_1 is the root, the parent of w is v, and the parent of each vertex in C_2 is w:

```
sage: G = 2 * graphs.CycleGraph(3)
sage: v = 0
sage: w = 3
sage: G.add_edge(v,w)
sage: G.dominator_tree(1, return_dict=True)
{0: 1, 1: None, 2: 1, 3: 0, 4: 3, 5: 3}
```

```
>>> from sage.all import *
>>> G = Integer(2) * graphs.CycleGraph(Integer(3))
>>> v = Integer(0)
>>> w = Integer(3)
>>> G.add_edge(v,w)
>>> G.dominator_tree(Integer(1), return_dict=True)
{0: 1, 1: None, 2: 1, 3: 0, 4: 3, 5: 3}
```

An example with a directed graph:

```
sage: g = digraphs.Circuit(10).dominator_tree(5)
sage: g.to_dictionary()
{0: [1], 1: [2], 2: [3], 3: [4], 4: [], 5: [6], 6: [7], 7: [8], 8: [9], 9:␣
↪[0]}
sage: g = digraphs.Circuit(10).dominator_tree(5, reverse=True)
sage: g.to_dictionary()
{0: [9], 1: [0], 2: [1], 3: [2], 4: [3], 5: [4], 6: [], 7: [6], 8: [7], 9:␣
↪[8]}
```

```
>>> from sage.all import *
>>> g = digraphs.Circuit(Integer(10)).dominator_tree(Integer(5))
>>> g.to_dictionary()
{0: [1], 1: [2], 2: [3], 3: [4], 4: [], 5: [6], 6: [7], 7: [8], 8: [9], 9:␣
↪[0]}
>>> g = digraphs.Circuit(Integer(10)).dominator_tree(Integer(5), reverse=True)
>>> g.to_dictionary()
{0: [9], 1: [0], 2: [1], 3: [2], 4: [3], 5: [4], 6: [], 7: [6], 8: [7], 9:␣
↪[8]}
```

If the output is a dictionary:

```
sage: graphs.GridGraph([2,2]).dominator_tree((0,0), return_dict=True)
{(0, 0): None, (0, 1): (0, 0), (1, 0): (0, 0), (1, 1): (0, 0)}
```

```
>>> from sage.all import *
>>> graphs.GridGraph([Integer(2),Integer(2)]).dominator_tree((Integer(0),
↪Integer(0)), return_dict=True)
{(0, 0): None, (0, 1): (0, 0), (1, 0): (0, 0), (1, 1): (0, 0)}
```

edge_boundary(*vertices1*, *vertices2=None*, *labels=True*, *sort=False*, *key=None*)

Return a list of edges `(u,v,l)` with `u` in `vertices1` and `v` in `vertices2`.

If `vertices2` is `None`, then it is set to the complement of `vertices1`.

In a digraph, the external boundary of a vertex v are those vertices u with an arc (v, u).

INPUT:

- `labels` – boolean (default: `True`); if `False`, each edge is a tuple (u, v) of vertices
- `sort` – boolean (default: `False`); whether to sort the result
- `key` – a function (default: `None`); a function that takes an edge as its one argument and returns a value that can be used for comparisons in the sorting algorithm (we must have `sort=True`)

EXAMPLES:

```
sage: K = graphs.CompleteBipartiteGraph(9, 3)
sage: len(K.edge_boundary([0, 1, 2, 3, 4, 5, 6, 7, 8], [9, 10, 11]))
27
sage: K.size()
27
```

```
>>> from sage.all import *
>>> K = graphs.CompleteBipartiteGraph(Integer(9), Integer(3))
>>> len(K.edge_boundary([Integer(0), Integer(1), Integer(2), Integer(3),␣
↪Integer(4), Integer(5), Integer(6), Integer(7), Integer(8)], [Integer(9),␣
↪Integer(10), Integer(11)]))
27
>>> K.size()
27
```

Note that the edge boundary preserves direction:

```
sage: K = graphs.CompleteBipartiteGraph(9, 3).to_directed()
sage: len(K.edge_boundary([0, 1, 2, 3, 4, 5, 6, 7, 8], [9, 10, 11]))
27
sage: K.size()
54
```

```
>>> from sage.all import *
>>> K = graphs.CompleteBipartiteGraph(Integer(9), Integer(3)).to_directed()
>>> len(K.edge_boundary([Integer(0), Integer(1), Integer(2), Integer(3),␣
↪Integer(4), Integer(5), Integer(6), Integer(7), Integer(8)], [Integer(9),␣
↪Integer(10), Integer(11)]))
27
>>> K.size()
54
```

```
sage: D = DiGraph({0: [1, 2], 3: [0]})
sage: D.edge_boundary([0], sort=True)
[(0, 1, None), (0, 2, None)]
sage: D.edge_boundary([0], labels=False, sort=True)
[(0, 1), (0, 2)]
```

```
>>> from sage.all import *
>>> D = DiGraph({Integer(0): [Integer(1), Integer(2)], Integer(3):␣
↪[Integer(0)]})
>>> D.edge_boundary([Integer(0)], sort=True)
[(0, 1, None), (0, 2, None)]
>>> D.edge_boundary([Integer(0)], labels=False, sort=True)
[(0, 1), (0, 2)]
```

Using the `key` argument to order multiple edges of incomparable types (see Issue #35903):

```
sage: G = Graph([(1, 'A', 4), (1, 2, 3)])
sage: G.edge_boundary([1], sort=True)
Traceback (most recent call last):
...
TypeError: unsupported operand parent(s) for <: 'Integer Ring' and '<class
↪'str'>'
sage: G.edge_boundary([1], sort=True, key=str)
[('A', 1, 4), (1, 2, 3)]
sage: G.edge_boundary([1], sort=True, key=lambda e:e[2])
[(1, 2, 3), ('A', 1, 4)]
sage: G.edge_boundary([1], labels=False, sort=True, key=lambda e:e[2])
Traceback (most recent call last):
...
IndexError: tuple index out of range
```

```
>>> from sage.all import *
>>> G = Graph([(Integer(1), 'A', Integer(4)), (Integer(1), Integer(2),␣
↪Integer(3))])
>>> G.edge_boundary([Integer(1)], sort=True)
Traceback (most recent call last):
...
TypeError: unsupported operand parent(s) for <: 'Integer Ring' and '<class
↪'str'>'
>>> G.edge_boundary([Integer(1)], sort=True, key=str)
[('A', 1, 4), (1, 2, 3)]
>>> G.edge_boundary([Integer(1)], sort=True, key=lambda e:e[Integer(2)])
[(1, 2, 3), ('A', 1, 4)]
>>> G.edge_boundary([Integer(1)], labels=False, sort=True, key=lambda␣
↪e:e[Integer(2)])
Traceback (most recent call last):
...
IndexError: tuple index out of range
```

edge_connectivity(*G*, *value_only=True*, *algorithm=None*, *by_weight=False*, *weight_function=None*, *check_weight=True*, *vertices=False*, *solver=None*, *verbose=0*, *integrality_tolerance=0.001*)

Return the edge connectivity of the graph.

For more information, see the Wikipedia article Connectivity_(graph_theory).

Note

When the graph is a directed graph, this method actually computes the *strong* connectivity, (i.e. a directed graph is strongly k-connected if there are k disjoint paths between any two vertices u, v). If you do not want to consider strong connectivity, the best is probably to convert your `DiGraph` object to a `Graph` object, and compute the connectivity of this other graph.

INPUT:

- `G` – the input Sage (Di)Graph
- `value_only` – boolean (default: `True`)
  - When set to `True` (default), only the value is returned.
  - When set to `False`, both the value and a minimum vertex cut are returned.
- `algorithm` – string (default: `None`); selects an algorithm:
  - `None` – default; selects the best algorithm available
  - `'boost'` – use the Boost graph library (which is much more efficient). It is not available when `edge_labels=True`, and it is rejected for directed graphs: the Boost implementation is for undirected graphs only and returns wrong values on digraphs (see Issue #18753).
  - `'Sage'` – use Sage's implementation based on integer linear programming
  - `'gabow'` – use the algorithm of Gabow [Gabow1995] implemented in *GabowEdgeConnectivity*. It is for directed graphs only, does not handle edge labels, and only computes the value of the edge connectivity, so it cannot be used with `value_only=False` or `vertices=True`. It is much faster than the integer linear program and is the default for digraphs when only the value is requested.
- `by_weight` – boolean (default: `False`); if `True`, computes a weighted minimum cut where each edge has the weight given by `weight_function`, or its label if `weight_function` is `None`. If `False`, each edge has weight 1. Implies `algorithm='sage'`.
- `weight_function` – function (default: `None`); a function that takes as input an edge `(u, v, l)` and outputs its weight. If not `None`, `by_weight` is automatically set to `True`. If `None` and `by_weight` is `True`, we use the edge label `l`, if `l` is not `None`, else 1 as a weight.
- `check_weight` – boolean (default: `True`); whether to check that the `weight_function` outputs a number for each edge
- `vertices` – boolean (default: `False`)
  - When set to `True`, also returns the two sets of vertices that are disconnected by the cut. Implies `value_only=False`.
- `solver` – string (default: `None`); specifies a Mixed Integer Linear Programming (MILP) solver to be used. If set to `None`, the default one is used. For more information on MILP solvers and which default solver is used, see the method `solve` of the class `MixedIntegerLinearProgram`.
- `verbose` – integer (default: 0); sets the level of verbosity. Set to 0 by default, which means quiet.
- `integrality_tolerance` – float; parameter for use with MILP solvers over an inexact base ring; see `MixedIntegerLinearProgram.get_values()`.

EXAMPLES:

A basic application on the PappusGraph:

```
sage: from sage.graphs.connectivity import edge_connectivity
sage: g = graphs.PappusGraph()
sage: edge_connectivity(g)
3
sage: g.edge_connectivity()
3
```

```
>>> from sage.all import *
>>> from sage.graphs.connectivity import edge_connectivity
>>> g = graphs.PappusGraph()
>>> edge_connectivity(g)
3
>>> g.edge_connectivity()
3
```

The edge connectivity of a complete graph is its minimum degree, and one of the two parts of the bipartition is reduced to only one vertex. The graph of the cut edges is isomorphic to a Star graph:

```
sage: g = graphs.CompleteGraph(5)
sage: [ value, edges, [ setA, setB ]] = edge_connectivity(g,vertices=True)
sage: value
4
sage: len(setA) == 1 or len(setB) == 1
True
sage: cut = Graph()
sage: cut.add_edges(edges)
sage: cut.is_isomorphic(graphs.StarGraph(4))
True
```

```
>>> from sage.all import *
>>> g = graphs.CompleteGraph(Integer(5))
>>> [ value, edges, [ setA, setB ]] = edge_connectivity(g,vertices=True)
>>> value
4
>>> len(setA) == Integer(1) or len(setB) == Integer(1)
True
>>> cut = Graph()
>>> cut.add_edges(edges)
>>> cut.is_isomorphic(graphs.StarGraph(Integer(4)))
True
```

Even if obviously in any graph we know that the edge connectivity is less than the minimum degree of the graph:

```
sage: g = graphs.RandomGNP(10,.3)
sage: min(g.degree()) >= edge_connectivity(g)
True
```

```
>>> from sage.all import *
>>> g = graphs.RandomGNP(Integer(10),RealNumber('.3'))
>>> min(g.degree()) >= edge_connectivity(g)
True
```

If we build a tree then assign to its edges a random value, the minimum cut will be the edge with minimum value:

```
sage: tree = graphs.RandomTree(10)
sage: for u,v in tree.edge_iterator(labels=None):
....:     tree.set_edge_label(u, v, random())
sage: minimum = min(tree.edge_labels())
sage: [_, [(_, _, l)]] = edge_connectivity(tree, value_only=False,
↪  # needs sage.numerical.mip
....:                                      by_weight=True)
sage: l == minimum
↪  # needs sage.numerical.mip
True
```

```
>>> from sage.all import *
>>> tree = graphs.RandomTree(Integer(10))
>>> for u,v in tree.edge_iterator(labels=None):
...     tree.set_edge_label(u, v, random())
>>> minimum = min(tree.edge_labels())
>>> [_, [(_, _, l)]] = edge_connectivity(tree, value_only=False,
↪# needs sage.numerical.mip
...                                      by_weight=True)
>>> l == minimum
↪# needs sage.numerical.mip
True
```

A `weight_function` can be used to read the weight from anywhere in the edge, and setting it implies `by_weight=True`:

```
sage: G = Graph([(0, 1, {'cost': 3}), (1, 2, {'cost': 1}),
....:            (2, 0, {'cost': 2})])
sage: edge_connectivity(G, weight_function=lambda e: e[2]['cost'])
↪  # needs sage.numerical.mip
3.0
```

```
>>> from sage.all import *
>>> G = Graph([(Integer(0), Integer(1), {'cost': Integer(3)}), (Integer(1),
↪Integer(2), {'cost': Integer(1)}),
...            (Integer(2), Integer(0), {'cost': Integer(2)})])
>>> edge_connectivity(G, weight_function=lambda e: e[Integer(2)]['cost'])
↪         # needs sage.numerical.mip
3.0
```

When `value_only=True` and `algorithm="sage"`, this function is optimized for small connectivity values and does not need to build a linear program.

It is the case for graphs which are not connected

```
sage: g = 2 * graphs.PetersenGraph()
sage: edge_connectivity(g, algorithm='sage')
0.0
```

```
>>> from sage.all import *
>>> g = Integer(2) * graphs.PetersenGraph()
```

(continues on next page)

(continued from previous page)

```
>>> edge_connectivity(g, algorithm='sage')
0.0
```

For directed graphs, the strong connectivity is tested through the dedicated function:

```
sage: g = digraphs.ButterflyGraph(3)
sage: edge_connectivity(g, algorithm='sage')
0.0
```

```
>>> from sage.all import *
>>> g = digraphs.ButterflyGraph(Integer(3))
>>> edge_connectivity(g, algorithm='sage')
0.0
```

We check that the result with Boost is the same as the result without Boost:

```
sage: g = graphs.RandomGNP(15, .3)
sage: (edge_connectivity(g, algorithm='boost')
↪   # needs sage.numerical.mip
....:     == edge_connectivity(g, algorithm='sage'))
True
```

```
>>> from sage.all import *
>>> g = graphs.RandomGNP(Integer(15), RealNumber('.3'))
>>> (edge_connectivity(g, algorithm='boost')
↪ # needs sage.numerical.mip
...     == edge_connectivity(g, algorithm='sage'))
True
```

The Boost implementation is for undirected graphs only. It considers a digraph as undirected and therefore returns wrong values, so it is rejected (see Issue #18753):

```
sage: g = digraphs.Path(3)
sage: edge_connectivity(g)
0
sage: edge_connectivity(g, algorithm='boost')
Traceback (most recent call last):
...
ValueError: the Boost implementation of the edge connectivity is for
undirected graphs only and returns wrong values on digraphs, see
https://github.com/sagemath/sage/issues/18753
```

```
>>> from sage.all import *
>>> g = digraphs.Path(Integer(3))
>>> edge_connectivity(g)
0
>>> edge_connectivity(g, algorithm='boost')
Traceback (most recent call last):
...
ValueError: the Boost implementation of the edge connectivity is for
undirected graphs only and returns wrong values on digraphs, see
https://github.com/sagemath/sage/issues/18753
```

The algorithm of Gabow is the default for digraphs when only the value is requested. It cannot return a cut, nor handle edge labels or undirected graphs:

```
sage: D = digraphs.Complete(5)
sage: edge_connectivity(D)
4
sage: edge_connectivity(D, algorithm='gabow')
4
sage: edge_connectivity(D, algorithm='gabow', value_only=False)
Traceback (most recent call last):
...
ValueError: the Gabow algorithm only computes the value of the edge
connectivity, it cannot return a cut
sage: edge_connectivity(graphs.PetersenGraph(), algorithm='gabow')
Traceback (most recent call last):
...
ValueError: the Gabow algorithm is for directed graphs only
```

```
>>> from sage.all import *
>>> D = digraphs.Complete(Integer(5))
>>> edge_connectivity(D)
4
>>> edge_connectivity(D, algorithm='gabow')
4
>>> edge_connectivity(D, algorithm='gabow', value_only=False)
Traceback (most recent call last):
...
ValueError: the Gabow algorithm only computes the value of the edge
connectivity, it cannot return a cut
>>> edge_connectivity(graphs.PetersenGraph(), algorithm='gabow')
Traceback (most recent call last):
...
ValueError: the Gabow algorithm is for directed graphs only
```

It also supports digraphs with multiple edges:

```
sage: D = DiGraph([(0, 1), (0, 1), (1, 0)], multiedges=True)
sage: edge_connectivity(D)
1
sage: D.add_edge(1, 0)
sage: edge_connectivity(D)
2
```

```
>>> from sage.all import *
>>> D = DiGraph([(Integer(0), Integer(1)), (Integer(0), Integer(1)),␣
↪(Integer(1), Integer(0))], multiedges=True)
>>> edge_connectivity(D)
1
>>> D.add_edge(Integer(1), Integer(0))
>>> edge_connectivity(D)
2
```

The `implementation` keyword is deprecated in favour of `algorithm`:

```
sage: edge_connectivity(graphs.PetersenGraph(), implementation='sage')
doctest:warning...
DeprecationWarning: use the option 'algorithm' instead of 'implementation'
See https://github.com/sagemath/sage/issues/42652 for details.
3
```

```
>>> from sage.all import *
>>> edge_connectivity(graphs.PetersenGraph(), implementation='sage')
doctest:warning...
DeprecationWarning: use the option 'algorithm' instead of 'implementation'
See https://github.com/sagemath/sage/issues/42652 for details.
3
```

The `use_edge_labels` keyword is deprecated in favour of `by_weight`:

```
sage: G = Graph([(0, 1, 3), (1, 2, 1), (2, 0, 2)])
sage: edge_connectivity(G, use_edge_labels=True)
↪  # needs sage.numerical.mip
doctest:warning...
DeprecationWarning: use the option 'by_weight' instead of 'use_edge_labels'
See https://github.com/sagemath/sage/issues/42740 for details.
3.0
```

```
>>> from sage.all import *
>>> G = Graph([(Integer(0), Integer(1), Integer(3)), (Integer(1), Integer(2),
↪Integer(1)), (Integer(2), Integer(0), Integer(2))])
>>> edge_connectivity(G, use_edge_labels=True)
↪# needs sage.numerical.mip
doctest:warning...
DeprecationWarning: use the option 'by_weight' instead of 'use_edge_labels'
See https://github.com/sagemath/sage/issues/42740 for details.
3.0
```

edge_cut(*s*, *t*, *value_only*, *use_edge_labels=True*, *vertices=False*, *algorithm=False*, *solver='FF'*, *verbose=None*, *integrality_tolerance=0*)

Return a minimum edge cut between vertices s and t.

A minimum edge cut between two vertices s and t of `self` is a set A of edges of minimum weight such that the graph obtained by removing A from the graph is disconnected. For more information, see the Wikipedia article Cut_(graph_theory).

INPUT:

- `s` – source vertex
- `t` – sink vertex
- `value_only` – boolean (default: `True`); whether to return only the weight of a minimum cut (`True`) or a list of edges of a minimum cut (`False`)
- `use_edge_labels` – boolean (default: `False`); whether to compute a weighted minimum edge cut where the weight of an edge is defined by its label (if an edge has no label, 1 is assumed), or to compute a cut of minimum cardinality (i.e., edge weights are set to 1)
- `vertices` – boolean (default: `False`); whether set to `True`, return a list of edges in the edge cut and the two sets of vertices that are disconnected by the cut

Note: `vertices=True` implies `value_only=False`.

- `algorithm` – string (default: `'FF'`); algorithm to use:
  - If `algorithm = "FF"`, a Python implementation of the Ford-Fulkerson algorithm is used
  - If `algorithm = "LP"`, the problem is solved using Linear Programming.
  - If `algorithm = "igraph"`, the igraph implementation of the Goldberg-Tarjan algorithm is used (only available when `igraph` is installed)
  - If `algorithm = None`, the problem is solved using the default maximum flow algorithm (see *flow()*)
- `solver` – string (default: `None`); specifies a Mixed Integer Linear Programming (MILP) solver to be used. If set to `None`, the default one is used. For more information on MILP solvers and which default solver is used, see the method `solve` of the class `MixedIntegerLinearProgram`.
- `verbose` – integer (default: 0); sets the level of verbosity. Set to 0 by default, which means quiet.
- `integrality_tolerance` – float; parameter for use with MILP solvers over an inexact base ring; see `MixedIntegerLinearProgram.get_values()`.

Note

The use of Linear Programming for non-integer problems may possibly mean the presence of a (slight) numerical noise.

OUTPUT:

Real number or tuple, depending on the given arguments (examples are given below).

EXAMPLES:

A basic application in the Pappus graph:

```
sage: g = graphs.PappusGraph()
sage: g.edge_cut(1, 2, value_only=True)
3
```

```
>>> from sage.all import *
>>> g = graphs.PappusGraph()
>>> g.edge_cut(Integer(1), Integer(2), value_only=True)
3
```

Or on Petersen's graph, with the corresponding bipartition of the vertex set:

```
sage: g = graphs.PetersenGraph()
sage: g.edge_cut(0, 3, vertices=True)
[3, [(0, 1, None), (0, 4, None), (0, 5, None)], [[0], [1, 2, 3, 4, 5, 6, 7, 8,
↪ 9]]]
```

```
>>> from sage.all import *
>>> g = graphs.PetersenGraph()
>>> g.edge_cut(Integer(0), Integer(3), vertices=True)
[3, [(0, 1, None), (0, 4, None), (0, 5, None)], [[0], [1, 2, 3, 4, 5, 6, 7, 8,
↪ 9]]]
```

If the graph is a path with randomly weighted edges:

```
sage: g = graphs.PathGraph(15)
sage: for u,v in g.edge_iterator(labels=None):
....:     g.set_edge_label(u, v, random())
```

```
>>> from sage.all import *
>>> g = graphs.PathGraph(Integer(15))
>>> for u,v in g.edge_iterator(labels=None):
...     g.set_edge_label(u, v, random())
```

The edge cut between the two ends is the edge of minimum weight:

```
sage: minimum = min(g.edge_labels())
sage: minimum == g.edge_cut(0, 14, use_edge_labels=True)
True
sage: [value, [e]] = g.edge_cut(0, 14, use_edge_labels=True, value_only=False)
sage: g.edge_label(e[0], e[1]) == minimum
True
```

```
>>> from sage.all import *
>>> minimum = min(g.edge_labels())
>>> minimum == g.edge_cut(Integer(0), Integer(14), use_edge_labels=True)
True
>>> [value, [e]] = g.edge_cut(Integer(0), Integer(14), use_edge_labels=True,␣
↪value_only=False)
>>> g.edge_label(e[Integer(0)], e[Integer(1)]) == minimum
True
```

The two sides of the edge cut are obviously shorter paths:

```
sage: value, edges, [set1, set2] = g.edge_cut(0, 14, use_edge_labels=True,
....:                                         vertices=True)
sage: g.subgraph(set1).is_isomorphic(graphs.PathGraph(len(set1)))
True
sage: g.subgraph(set2).is_isomorphic(graphs.PathGraph(len(set2)))
True
sage: len(set1) + len(set2) == g.order()
True
```

```
>>> from sage.all import *
>>> value, edges, [set1, set2] = g.edge_cut(Integer(0), Integer(14), use_edge_
↪labels=True,
...                                         vertices=True)
>>> g.subgraph(set1).is_isomorphic(graphs.PathGraph(len(set1)))
True
>>> g.subgraph(set2).is_isomorphic(graphs.PathGraph(len(set2)))
True
>>> len(set1) + len(set2) == g.order()
True
```

edge_disjoint_paths(*s*, *t*, *algorithm*, *solver='FF'*, *verbose=None*, *integrality_tolerance=False*)

Return a list of edge-disjoint paths between two vertices.

The edge version of Menger's theorem asserts that the size of the minimum edge cut between two vertices s and`t` (the minimum number of edges whose removal disconnects s and t) is equal to the maximum number of pairwise edge-independent paths from s to t.

This function returns a list of such paths.

INPUT:

- `algorithm` – string (default: `'FF'`); the algorithm to use among:
  - `'FF'`, a Python implementation of the Ford-Fulkerson algorithm
  - `'LP'`, the flow problem is solved using Linear Programming
- `solver` – string (default: `None`); specifies a Mixed Integer Linear Programming (MILP) solver to be used. If set to `None`, the default one is used. For more information on MILP solvers and which default solver is used, see the method `solve` of the class `MixedIntegerLinearProgram`.

  Only used when `algorithm` is `'LP'`.
- `verbose` – integer (default: 0); sets the level of verbosity. Set to 0 by default, which means quiet.

  Only used when `algorithm` is `'LP'`.
- `integrality_tolerance` – float; parameter for use with MILP solvers over an inexact base ring; see `MixedIntegerLinearProgram.get_values()`.

  Only used when `algorithm` is `'LP'`.

Note

This function is topological: it does not take the eventual weights of the edges into account.

EXAMPLES:

In a complete bipartite graph

```
sage: g = graphs.CompleteBipartiteGraph(2, 3)
sage: g.edge_disjoint_paths(0, 1)
[[0, 2, 1], [0, 3, 1], [0, 4, 1]]
```

```
>>> from sage.all import *
>>> g = graphs.CompleteBipartiteGraph(Integer(2), Integer(3))
>>> g.edge_disjoint_paths(Integer(0), Integer(1))
[[0, 2, 1], [0, 3, 1], [0, 4, 1]]
```

edge_disjoint_spanning_trees(*k*, *algorithm*, *root=None*, *solver=None*, *verbose=None*, *integrality_tolerance=None*, *labels=0*)

Return the desired number of edge-disjoint spanning trees/arborescences.

INPUT:

- `k` – integer or `None` (default: `None`); the required number of edge-disjoint spanning trees/arborescences. If `None`: for directed graphs, return as many arborescences as the edge connectivity of the digraph (such a packing always exists by Edmonds' theorem); for undirected graphs with `'Roskind-Tarjan'`, return a maximum packing of edge-disjoint spanning trees.
- `algorithm` – string (default: `None`); specify the algorithm to use among:
  - `'Roskind-Tarjan'` – use the algorithm proposed by Roskind and Tarjan [RT1985] for finding edge-disjoint spanning-trees in undirected simple graphs in time $O(m \log m + k^2 n^2)$.

  - `'MILP'` – use a mixed integer linear programming formulation. Graphs with loops and multiple edges are supported: loops are ignored and each parallel copy counts as a distinct edge.
  - `'Gabow'` – use the combinatorial algorithm of Gabow [Gabow1995] for packing edge-disjoint spanning arborescences. This is the default method for directed graphs. Only available for directed graphs; digraphs with loops and multiple edges are supported.
  - `None` – use `'Roskind-Tarjan'` for undirected graphs and `'Gabow'` for directed graphs.
- `root` – vertex (default: `None`); root of the disjoint arborescences when the graph is directed. If set to `None`, the first vertex in the graph is picked.
- `solver` – string (default: `None`); specifies a Mixed Integer Linear Programming (MILP) solver to be used. If set to `None`, the default one is used. For more information on MILP solvers and which default solver is used, see the method `solve` of the class `MixedIntegerLinearProgram`.
- `verbose` – integer (default: 0); sets the level of verbosity. Set to 0 by default, which means quiet.
- `integrality_tolerance` – float; parameter for use with MILP solvers over an inexact base ring; see `MixedIntegerLinearProgram.get_values()`.
- `labels` – boolean (default: `False`); whether the edges of the returned trees carry the labels of the corresponding input edges, so that parallel edges of a graph with multiple edges remain distinguishable in the output. Only supported by the `'Gabow'` and `'MILP'` algorithms.

ALGORITHM:

Mixed Integer Linear Program.

There are at least two possible rewritings of this method which do not use Linear Programming:

- The algorithm presented in the paper entitled "A short proof of the tree-packing theorem", by Thomas Kaiser [Kai2012].
- The implementation of a Matroid class and of the Matroid Union Theorem (see section 42.3 of [Sch2003]), applied to the cycle Matroid (see chapter 51 of [Sch2003]).

EXAMPLES:

The Petersen Graph does have a spanning tree (it is connected):

```
sage: g = graphs.PetersenGraph()
sage: [T] = g.edge_disjoint_spanning_trees(1)                                 #␣
↪needs sage.numerical.mip
sage: T.is_tree()                                                             #␣
↪needs sage.numerical.mip
True
```

```
>>> from sage.all import *
>>> g = graphs.PetersenGraph()
>>> [T] = g.edge_disjoint_spanning_trees(Integer(1))                          ␣
↪     # needs sage.numerical.mip
>>> T.is_tree()                                                             #␣
↪needs sage.numerical.mip
True
```

Though, it does not have 2 edge-disjoint trees (as it has less than $2(|V|-1)$ edges):

```
sage: g.edge_disjoint_spanning_trees(2)                                       #␣
↪needs sage.numerical.mip
```

(continues on next page)

(continued from previous page)

```
Traceback (most recent call last):
...
EmptySetError: this graph does not contain the required number of trees/
↪arborescences
```

```
>>> from sage.all import *
>>> g.edge_disjoint_spanning_trees(Integer(2))
↪     # needs sage.numerical.mip
Traceback (most recent call last):
...
EmptySetError: this graph does not contain the required number of trees/
↪arborescences
```

By Edmonds' theorem, a graph which is k-connected always has k edge-disjoint arborescences, regardless of the root we pick:

```
sage: g = digraphs.RandomDirectedGNP(11, .3)  # reduced from 30 to 11, cf.
↪#32169
sage: k = Integer(g.edge_connectivity())
sage: while not k:
....:     g = digraphs.RandomDirectedGNP(11, .3)
....:     k = Integer(g.edge_connectivity())
sage: arborescences = g.edge_disjoint_spanning_trees(k)     # long time (up
↪to 15s on sage.math, 2011)
sage: all(a.is_directed_acyclic() for a in arborescences)   # long time
True
sage: all(a.is_connected() for a in arborescences)  # long time
True
```

```
>>> from sage.all import *
>>> g = digraphs.RandomDirectedGNP(Integer(11), RealNumber('.3'))  # reduced
↪from 30 to 11, cf. #32169
>>> k = Integer(g.edge_connectivity())
>>> while not k:
...     g = digraphs.RandomDirectedGNP(Integer(11), RealNumber('.3'))
...     k = Integer(g.edge_connectivity())
>>> arborescences = g.edge_disjoint_spanning_trees(k)     # long time (up to
↪15s on sage.math, 2011)
>>> all(a.is_directed_acyclic() for a in arborescences)   # long time
True
>>> all(a.is_connected() for a in arborescences)  # long time
True
```

In the undirected case, we can only ensure half of it:

```
sage: g = graphs.RandomGNP(14, .3)  # reduced from 30 to 14, see #32169
sage: while not g.is_biconnected():
....:     g = graphs.RandomGNP(14, .3)
sage: k = Integer(g.edge_connectivity()) // 2
sage: trees = g.edge_disjoint_spanning_trees(k)
sage: all(t.is_tree() for t in trees)
True
```

```
>>> from sage.all import *
>>> g = graphs.RandomGNP(Integer(14), RealNumber('.3'))  # reduced from 30 to␣
↪14, see #32169
>>> while not g.is_biconnected():
...     g = graphs.RandomGNP(Integer(14), RealNumber('.3'))
>>> k = Integer(g.edge_connectivity()) // Integer(2)
>>> trees = g.edge_disjoint_spanning_trees(k)
>>> all(t.is_tree() for t in trees)
True
```

Check the validity of the algorithms for undirected graphs:

```
sage: g = graphs.RandomGNP(12, .7)
sage: k = Integer(g.edge_connectivity()) // 2
sage: trees = g.edge_disjoint_spanning_trees(k, algorithm='MILP')
sage: all(t.is_tree() for t in trees)
True
sage: all(g.order() == t.size() + 1 for t in trees)
True
sage: trees = g.edge_disjoint_spanning_trees(k, algorithm='Roskind-Tarjan')
sage: all(t.is_tree() for t in trees)
True
sage: all(g.order() == t.size() + 1 for t in trees)
True
```

```
>>> from sage.all import *
>>> g = graphs.RandomGNP(Integer(12), RealNumber('.7'))
>>> k = Integer(g.edge_connectivity()) // Integer(2)
>>> trees = g.edge_disjoint_spanning_trees(k, algorithm='MILP')
>>> all(t.is_tree() for t in trees)
True
>>> all(g.order() == t.size() + Integer(1) for t in trees)
True
>>> trees = g.edge_disjoint_spanning_trees(k, algorithm='Roskind-Tarjan')
>>> all(t.is_tree() for t in trees)
True
>>> all(g.order() == t.size() + Integer(1) for t in trees)
True
```

Example of Issue #32169:

```
sage: d6 = r'[E_S?_hKIH@eos[BSg???Q@FShGC?hTHUGM?IPug?JOEYCdOzdkQGo'
sage: d6 += r'@ADA@AAg?GAQW?[aIaSwHYcD@qQb@Dd?\hJTI@OHlJ_?C_OEIKoeC'
sage: d6 += r'R@_BC?Q??YBFosqITEA?IvCU_'
sage: G = DiGraph(d6, format='dig6')
sage: G.edge_connectivity()                                                 #␣
↪needs sage.numerical.mip
5
sage: G.edge_disjoint_spanning_trees(5)      # long time                    #␣
↪needs sage.numerical.mip
[Digraph on 28 vertices,
 Digraph on 28 vertices,
 Digraph on 28 vertices,
```

(continues on next page)

(continued from previous page)

```
 Digraph on 28 vertices,
 Digraph on 28 vertices]
```

```
>>> from sage.all import *
>>> d6 = r'[E_S?_hKIH@eos[BSg???Q@FShGC?hTHUGM?IPug?JOEYCdOzdkQGo'
>>> d6 += r'@ADA@AAg?GAQW?[aIaSwHYcD@qQb@Dd?\hJTI@OHlJ_?C_OEIKoeC'
>>> d6 += r'R@_BC?Q??YBFosqITEA?IvCU_'
>>> G = DiGraph(d6, format='dig6')
>>> G.edge_connectivity()                                                    #␣
↪needs sage.numerical.mip
5
>>> G.edge_disjoint_spanning_trees(Integer(5))      # long time              ␣
↪     # needs sage.numerical.mip
[Digraph on 28 vertices,
 Digraph on 28 vertices,
 Digraph on 28 vertices,
 Digraph on 28 vertices,
 Digraph on 28 vertices]
```

Small cases:

```
sage: Graph().edge_disjoint_spanning_trees(0)
[]
sage: Graph(1).edge_disjoint_spanning_trees(0)
[]
sage: Graph(2).edge_disjoint_spanning_trees(0)
[]
sage: Graph([(0, 1)]).edge_disjoint_spanning_trees(0)
[]
sage: Graph([(0, 1)]).edge_disjoint_spanning_trees(1)
[Graph on 2 vertices]
sage: Graph([(0, 1)]).edge_disjoint_spanning_trees(2)
Traceback (most recent call last):
...
EmptySetError: this graph does not contain the required number of trees/
↪arborescences
```

```
>>> from sage.all import *
>>> Graph().edge_disjoint_spanning_trees(Integer(0))
[]
>>> Graph(Integer(1)).edge_disjoint_spanning_trees(Integer(0))
[]
>>> Graph(Integer(2)).edge_disjoint_spanning_trees(Integer(0))
[]
>>> Graph([(Integer(0), Integer(1))]).edge_disjoint_spanning_trees(Integer(0))
[]
>>> Graph([(Integer(0), Integer(1))]).edge_disjoint_spanning_trees(Integer(1))
[Graph on 2 vertices]
>>> Graph([(Integer(0), Integer(1))]).edge_disjoint_spanning_trees(Integer(2))
Traceback (most recent call last):
...
EmptySetError: this graph does not contain the required number of trees/
```

(continues on next page)

(continued from previous page)

```
↪arborescences
```

edge_iterator(*vertices=None*, *labels=True*, *ignore_direction=False*, *sort_vertices=True*)

Return an iterator over edges.

The iterator returned is over the edges incident with any vertex given in the parameter `vertices`. If the graph is directed, iterates over edges going out only. If `vertices` is `None`, then returns an iterator over all edges. If `self` is directed, returns outgoing edges only.

INPUT:

- `vertices` – object (default: `None`); a vertex, a list of vertices or `None`
- `labels` – boolean (default: `True`); if `False`, each edge is a tuple (u, v) of vertices
- `ignore_direction` – boolean (default: `False`); only applies to directed graphs. If `True`, searches across edges in either direction.
- `sort_vertices` – boolean (default: `True`); only applies to undirected graphs. If `True`, sort the ends of the edges. Not sorting the ends is faster.

Note

It is somewhat safe to modify the graph during iterating.

`vertices` must be specified if modifying the vertices.

Without multiedges, you can safely use this graph to relabel edges or delete some edges. If you add edges, they might later appear in the iterator or not (depending on the internal order of vertices).

In case of multiedges, all arcs from one vertex to another are internally cached. So the iterator will yield them, even if you delete them all after seeing the first one.

EXAMPLES:

```
sage: for i in graphs.PetersenGraph().edge_iterator([0]):
....:  print(i)
(0, 1, None)
(0, 4, None)
(0, 5, None)
sage: D = DiGraph({0: [1, 2], 1: [0]})
sage: for i in D.edge_iterator([0]):
....:  print(i)
(0, 1, None)
(0, 2, None)
```

```
>>> from sage.all import *
>>> for i in graphs.PetersenGraph().edge_iterator([Integer(0)]):
...  print(i)
(0, 1, None)
(0, 4, None)
(0, 5, None)
>>> D = DiGraph({Integer(0): [Integer(1), Integer(2)], Integer(1):␣
↪[Integer(0)]})
>>> for i in D.edge_iterator([Integer(0)]):
```

(continues on next page)

(continued from previous page)

```
...  print(i)
(0, 1, None)
(0, 2, None)
```

```
sage: G = graphs.TetrahedralGraph()
sage: list(G.edge_iterator(labels=False))
[(0, 1), (0, 2), (0, 3), (1, 2), (1, 3), (2, 3)]
```

```
>>> from sage.all import *
>>> G = graphs.TetrahedralGraph()
>>> list(G.edge_iterator(labels=False))
[(0, 1), (0, 2), (0, 3), (1, 2), (1, 3), (2, 3)]
```

```
sage: G = graphs.TetrahedralGraph()
sage: list(G.edge_iterator(labels=False, sort_vertices=False))
[(1, 0), (2, 0), (3, 0), (2, 1), (3, 1), (3, 2)]
```

```
>>> from sage.all import *
>>> G = graphs.TetrahedralGraph()
>>> list(G.edge_iterator(labels=False, sort_vertices=False))
[(1, 0), (2, 0), (3, 0), (2, 1), (3, 1), (3, 2)]
```

```
sage: D = DiGraph({1: [0], 2: [0]})
sage: list(D.edge_iterator(0))
[]
sage: list(D.edge_iterator(0, ignore_direction=True))
[(1, 0, None), (2, 0, None)]
```

```
>>> from sage.all import *
>>> D = DiGraph({Integer(1): [Integer(0)], Integer(2): [Integer(0)]})
>>> list(D.edge_iterator(Integer(0)))
[]
>>> list(D.edge_iterator(Integer(0), ignore_direction=True))
[(1, 0, None), (2, 0, None)]
```

edge_label(*u*, *v*)

Return the label of an edge.

If the graph allows multiple edges, then the list of labels on the edges is returned.

See also

- *set_edge_label()*

EXAMPLES:

```
sage: G = Graph({0: {1: 'edgelabel'}})
sage: G.edge_label(0, 1)
'edgelabel'
```

(continues on next page)

(continued from previous page)

```
sage: D = DiGraph({1: {2: 'up'}, 2: {1: 'down'}})
sage: D.edge_label(2, 1)
'down'
```

```
>>> from sage.all import *
>>> G = Graph({Integer(0): {Integer(1): 'edgelabel'}})
>>> G.edge_label(Integer(0), Integer(1))
'edgelabel'
>>> D = DiGraph({Integer(1): {Integer(2): 'up'}, Integer(2): {Integer(1):
↪'down'}})
>>> D.edge_label(Integer(2), Integer(1))
'down'
```

```
sage: G = Graph(multiedges=True)
sage: [G.add_edge(0, 1, i) for i in range(1, 6)]
[None, None, None, None, None]
sage: sorted(G.edge_label(0, 1))
[1, 2, 3, 4, 5]
```

```
>>> from sage.all import *
>>> G = Graph(multiedges=True)
>>> [G.add_edge(Integer(0), Integer(1), i) for i in range(Integer(1),␣
↪Integer(6))]
[None, None, None, None, None]
>>> sorted(G.edge_label(Integer(0), Integer(1)))
[1, 2, 3, 4, 5]
```

edge_labels()

Return a list of the labels of all edges in `self`.

The output list is not sorted.

EXAMPLES:

```
sage: G = Graph({0: {1: 'x', 2: 'z', 3: 'a'}, 2: {5: 'out'}}, sparse=True)
sage: G.edge_labels()
['x', 'z', 'a', 'out']
sage: G = DiGraph({0: {1: 'x', 2: 'z', 3: 'a'}, 2: {5: 'out'}}, sparse=True)
sage: G.edge_labels()
['x', 'z', 'a', 'out']
```

```
>>> from sage.all import *
>>> G = Graph({Integer(0): {Integer(1): 'x', Integer(2): 'z', Integer(3): 'a'}
↪, Integer(2): {Integer(5): 'out'}}, sparse=True)
>>> G.edge_labels()
['x', 'z', 'a', 'out']
>>> G = DiGraph({Integer(0): {Integer(1): 'x', Integer(2): 'z', Integer(3): 'a
↪'}, Integer(2): {Integer(5): 'out'}}, sparse=True)
>>> G.edge_labels()
['x', 'z', 'a', 'out']
```

edge_polytope(*backend=None*)

Return the edge polytope of `self`.

The edge polytope (EP) of a Graph on n vertices is the polytope in $\mathbf{Z}^n$ defined as the convex hull of $e_i + e_j$ for each edge (i, j). Here $e_1, \ldots, e_n$ denotes the standard basis.

INPUT:

- `backend` – string or `None` (default); the backend to use; see the `Polyhedron constructor`

EXAMPLES:

The EP of a 4-cycle is a square:

```
sage: G = graphs.CycleGraph(4)
sage: P = G.edge_polytope(); P                                                  #␣
↪needs sage.geometry.polyhedron
A 2-dimensional polyhedron in ZZ^4 defined as the convex hull of 4 vertices
```

```
>>> from sage.all import *
>>> G = graphs.CycleGraph(Integer(4))
>>> P = G.edge_polytope(); P                                                  #␣
↪needs sage.geometry.polyhedron
A 2-dimensional polyhedron in ZZ^4 defined as the convex hull of 4 vertices
```

The EP of a complete graph on 4 vertices is cross polytope:

```
sage: G = graphs.CompleteGraph(4)
sage: P = G.edge_polytope(); P                                                  #␣
↪needs sage.geometry.polyhedron
A 3-dimensional polyhedron in ZZ^4 defined as the convex hull of 6 vertices
sage: P.is_combinatorially_isomorphic(polytopes.cross_polytope(3))              #␣
↪needs sage.geometry.polyhedron
True
```

```
>>> from sage.all import *
>>> G = graphs.CompleteGraph(Integer(4))
>>> P = G.edge_polytope(); P                                                  #␣
↪needs sage.geometry.polyhedron
A 3-dimensional polyhedron in ZZ^4 defined as the convex hull of 6 vertices
>>> P.is_combinatorially_isomorphic(polytopes.cross_polytope(Integer(3)))    ␣
↪     # needs sage.geometry.polyhedron
True
```

The EP of a graph is isomorphic to the subdirect sum of its connected components EPs:

```
sage: n = randint(3, 6)
sage: G1 = graphs.RandomGNP(n, 0.2)                                             #␣
↪needs networkx
sage: n = randint(3, 6)
sage: G2 = graphs.RandomGNP(n, 0.2)                                             #␣
↪needs networkx
sage: G = G1.disjoint_union(G2)                                                 #␣
↪needs networkx
sage: P = G.edge_polytope()                                                     #␣
↪needs networkx sage.geometry.polyhedron
sage: P1 = G1.edge_polytope()                                                   #␣
↪needs networkx sage.geometry.polyhedron
```

(continues on next page)

(continued from previous page)

```
sage: P2 = G2.edge_polytope()                                                  #␣
↪needs networkx sage.geometry.polyhedron
sage: P.is_combinatorially_isomorphic(P1.subdirect_sum(P2))                    #␣
↪needs networkx sage.geometry.polyhedron
True
```

```
>>> from sage.all import *
>>> n = randint(Integer(3), Integer(6))
>>> G1 = graphs.RandomGNP(n, RealNumber('0.2'))                                ␣
↪          # needs networkx
>>> n = randint(Integer(3), Integer(6))
>>> G2 = graphs.RandomGNP(n, RealNumber('0.2'))                                ␣
↪          # needs networkx
>>> G = G1.disjoint_union(G2)                                                  #␣
↪needs networkx
>>> P = G.edge_polytope()                                                      #␣
↪needs networkx sage.geometry.polyhedron
>>> P1 = G1.edge_polytope()                                                    #␣
↪needs networkx sage.geometry.polyhedron
>>> P2 = G2.edge_polytope()                                                    #␣
↪needs networkx sage.geometry.polyhedron
>>> P.is_combinatorially_isomorphic(P1.subdirect_sum(P2))                      #␣
↪needs networkx sage.geometry.polyhedron
True
```

All trees on n vertices have isomorphic EPs:

```
sage: n = randint(4, 10)
sage: G1 = graphs.RandomTree(n)
sage: G2 = graphs.RandomTree(n)
sage: P1 = G1.edge_polytope()                                                  #␣
↪needs sage.geometry.polyhedron
sage: P2 = G2.edge_polytope()                                                  #␣
↪needs sage.geometry.polyhedron
sage: P1.is_combinatorially_isomorphic(P2)                                     #␣
↪needs sage.geometry.polyhedron
True
```

```
>>> from sage.all import *
>>> n = randint(Integer(4), Integer(10))
>>> G1 = graphs.RandomTree(n)
>>> G2 = graphs.RandomTree(n)
>>> P1 = G1.edge_polytope()                                                    #␣
↪needs sage.geometry.polyhedron
>>> P2 = G2.edge_polytope()                                                    #␣
↪needs sage.geometry.polyhedron
>>> P1.is_combinatorially_isomorphic(P2)                                       #␣
↪needs sage.geometry.polyhedron
True
```

However, there are still many different EPs:

```
sage: len(list(graphs(5)))
34
sage: polys = []
sage: for G in graphs(5):                                                    #␣
↪needs sage.geometry.polyhedron
....:     P = G.edge_polytope()
....:     for P1 in polys:
....:         if P.is_combinatorially_isomorphic(P1):
....:             break
....:     else:
....:         polys.append(P)
sage: len(polys)                                                             #␣
↪needs sage.geometry.polyhedron
19
```

```
>>> from sage.all import *
>>> len(list(graphs(Integer(5))))
34
>>> polys = []
>>> for G in graphs(Integer(5)):                                             ␣
↪     # needs sage.geometry.polyhedron
...     P = G.edge_polytope()
...     for P1 in polys:
...         if P.is_combinatorially_isomorphic(P1):
...             break
...     else:
...         polys.append(P)
>>> len(polys)                                                              #␣
↪needs sage.geometry.polyhedron
19
```

edges(*vertices=None*, *labels=True*, *sort=False*, *key=None*, *ignore_direction=False*, *sort_vertices=True*)

Return a *EdgesView* of edges.

Each edge is a triple `(u, v, l)` where `u` and `v` are vertices and `l` is a label. If the parameter `labels` is `False` then a list of couple `(u, v)` is returned where `u` and `v` are vertices.

The returned *EdgesView* is over the edges incident with any vertex given in the parameter `vertices` (all edges if `None`). If `self` is directed, iterates over outgoing edges only, unless parameter `ignore_direction` is `True` in which case it searches across edges in either direction.

INPUT:

- `vertices` – object (default: `None`); a vertex, a list of vertices or `None`
- `labels` – boolean (default: `True`); if `False`, each edge is simply a pair `(u, v)` of vertices
- `sort` – boolean (default: `False`); whether to sort edges according the ordering specified with parameter `key`. If `False` (default), edges are not sorted. This is the fastest and less memory consuming method for iterating over edges.
- `key` – a function (default: `None`); a function that takes an edge (a pair or a triple, according to the `labels` keyword) as its one argument and returns a value that can be used for comparisons in the sorting algorithm
- `ignore_direction` – boolean (default: `False`); only applies to directed graphs. If `True`, searches across edges in either direction.

- `sort_vertices` – boolean (default: `True`); only applies to undirected graphs. If `True`, sort the ends of the edges. Not sorting the ends is faster.

OUTPUT: a *EdgesView*

Warning

Since any object may be a vertex, there is no guarantee that any two vertices will be comparable, and thus no guarantee how two edges may compare. With default objects for vertices (all integers), or when all the vertices are of the same simple type, then there should not be a problem with how the vertices will be sorted. However, if you need to guarantee a total order for the sorting of the edges, use the `key` argument, as illustrated in the examples below.

EXAMPLES:

```
sage: graphs.DodecahedralGraph().edges(sort=True)
[(0, 1, None), (0, 10, None), (0, 19, None), (1, 2, None),
 (1, 8, None), (2, 3, None), (2, 6, None), (3, 4, None),
 (3, 19, None), (4, 5, None), (4, 17, None), (5, 6, None),
 (5, 15, None), (6, 7, None), (7, 8, None), (7, 14, None),
 (8, 9, None), (9, 10, None), (9, 13, None), (10, 11, None),
 (11, 12, None), (11, 18, None), (12, 13, None), (12, 16, None),
 (13, 14, None), (14, 15, None), (15, 16, None), (16, 17, None),
 (17, 18, None), (18, 19, None)]
```

```
>>> from sage.all import *
>>> graphs.DodecahedralGraph().edges(sort=True)
[(0, 1, None), (0, 10, None), (0, 19, None), (1, 2, None),
 (1, 8, None), (2, 3, None), (2, 6, None), (3, 4, None),
 (3, 19, None), (4, 5, None), (4, 17, None), (5, 6, None),
 (5, 15, None), (6, 7, None), (7, 8, None), (7, 14, None),
 (8, 9, None), (9, 10, None), (9, 13, None), (10, 11, None),
 (11, 12, None), (11, 18, None), (12, 13, None), (12, 16, None),
 (13, 14, None), (14, 15, None), (15, 16, None), (16, 17, None),
 (17, 18, None), (18, 19, None)]
```

```
sage: graphs.DodecahedralGraph().edges(sort=True, labels=False)
[(0, 1), (0, 10), (0, 19), (1, 2), (1, 8), (2, 3), (2, 6), (3, 4),
 (3, 19), (4, 5), (4, 17), (5, 6), (5, 15), (6, 7), (7, 8), (7, 14),
 (8, 9), (9, 10), (9, 13), (10, 11), (11, 12), (11, 18), (12, 13),
 (12, 16), (13, 14), (14, 15), (15, 16), (16, 17), (17, 18),
 (18, 19)]
```

```
>>> from sage.all import *
>>> graphs.DodecahedralGraph().edges(sort=True, labels=False)
[(0, 1), (0, 10), (0, 19), (1, 2), (1, 8), (2, 3), (2, 6), (3, 4),
 (3, 19), (4, 5), (4, 17), (5, 6), (5, 15), (6, 7), (7, 8), (7, 14),
 (8, 9), (9, 10), (9, 13), (10, 11), (11, 12), (11, 18), (12, 13),
 (12, 16), (13, 14), (14, 15), (15, 16), (16, 17), (17, 18),
 (18, 19)]
```

```
sage: D = graphs.DodecahedralGraph().to_directed()
sage: D.edges(sort=True)
[(0, 1, None), (0, 10, None), (0, 19, None), (1, 0, None),
 (1, 2, None), (1, 8, None), (2, 1, None), (2, 3, None),
 (2, 6, None), (3, 2, None), (3, 4, None), (3, 19, None),
 (4, 3, None), (4, 5, None), (4, 17, None), (5, 4, None),
 (5, 6, None), (5, 15, None), (6, 2, None), (6, 5, None),
 (6, 7, None), (7, 6, None), (7, 8, None), (7, 14, None),
 (8, 1, None), (8, 7, None), (8, 9, None), (9, 8, None),
 (9, 10, None), (9, 13, None), (10, 0, None), (10, 9, None),
 (10, 11, None), (11, 10, None), (11, 12, None), (11, 18, None),
 (12, 11, None), (12, 13, None), (12, 16, None), (13, 9, None),
 (13, 12, None), (13, 14, None), (14, 7, None), (14, 13, None),
 (14, 15, None), (15, 5, None), (15, 14, None), (15, 16, None),
 (16, 12, None), (16, 15, None), (16, 17, None), (17, 4, None),
 (17, 16, None), (17, 18, None), (18, 11, None), (18, 17, None),
 (18, 19, None), (19, 0, None), (19, 3, None), (19, 18, None)]
sage: D.edges(sort=True, labels=False)
[(0, 1), (0, 10), (0, 19), (1, 0), (1, 2), (1, 8), (2, 1), (2, 3),
 (2, 6), (3, 2), (3, 4), (3, 19), (4, 3), (4, 5), (4, 17), (5, 4),
 (5, 6), (5, 15), (6, 2), (6, 5), (6, 7), (7, 6), (7, 8), (7, 14),
 (8, 1), (8, 7), (8, 9), (9, 8), (9, 10), (9, 13), (10, 0), (10, 9),
 (10, 11), (11, 10), (11, 12), (11, 18), (12, 11), (12, 13),
 (12, 16), (13, 9), (13, 12), (13, 14), (14, 7), (14, 13), (14, 15),
 (15, 5), (15, 14), (15, 16), (16, 12), (16, 15), (16, 17), (17, 4),
 (17, 16), (17, 18), (18, 11), (18, 17), (18, 19), (19, 0), (19, 3),
 (19, 18)]
```

```
>>> from sage.all import *
>>> D = graphs.DodecahedralGraph().to_directed()
>>> D.edges(sort=True)
[(0, 1, None), (0, 10, None), (0, 19, None), (1, 0, None),
 (1, 2, None), (1, 8, None), (2, 1, None), (2, 3, None),
 (2, 6, None), (3, 2, None), (3, 4, None), (3, 19, None),
 (4, 3, None), (4, 5, None), (4, 17, None), (5, 4, None),
 (5, 6, None), (5, 15, None), (6, 2, None), (6, 5, None),
 (6, 7, None), (7, 6, None), (7, 8, None), (7, 14, None),
 (8, 1, None), (8, 7, None), (8, 9, None), (9, 8, None),
 (9, 10, None), (9, 13, None), (10, 0, None), (10, 9, None),
 (10, 11, None), (11, 10, None), (11, 12, None), (11, 18, None),
 (12, 11, None), (12, 13, None), (12, 16, None), (13, 9, None),
 (13, 12, None), (13, 14, None), (14, 7, None), (14, 13, None),
 (14, 15, None), (15, 5, None), (15, 14, None), (15, 16, None),
 (16, 12, None), (16, 15, None), (16, 17, None), (17, 4, None),
 (17, 16, None), (17, 18, None), (18, 11, None), (18, 17, None),
 (18, 19, None), (19, 0, None), (19, 3, None), (19, 18, None)]
>>> D.edges(sort=True, labels=False)
[(0, 1), (0, 10), (0, 19), (1, 0), (1, 2), (1, 8), (2, 1), (2, 3),
 (2, 6), (3, 2), (3, 4), (3, 19), (4, 3), (4, 5), (4, 17), (5, 4),
 (5, 6), (5, 15), (6, 2), (6, 5), (6, 7), (7, 6), (7, 8), (7, 14),
 (8, 1), (8, 7), (8, 9), (9, 8), (9, 10), (9, 13), (10, 0), (10, 9),
 (10, 11), (11, 10), (11, 12), (11, 18), (12, 11), (12, 13),
```

(continues on next page)

(continued from previous page)

```
 (12, 16), (13, 9), (13, 12), (13, 14), (14, 7), (14, 13), (14, 15),
 (15, 5), (15, 14), (15, 16), (16, 12), (16, 15), (16, 17), (17, 4),
 (17, 16), (17, 18), (18, 11), (18, 17), (18, 19), (19, 0), (19, 3),
 (19, 18)]
```

The default is to sort the returned list in the default fashion, as in the above examples. This can be overridden by specifying a key function. This first example just ignores the labels in the third component of the triple:

```
sage: G = graphs.CycleGraph(5)
sage: G.edges(sort=True, key=lambda x: (x[1], -x[0]))
[(0, 1, None), (1, 2, None), (2, 3, None), (3, 4, None), (0, 4, None)]
```

```
>>> from sage.all import *
>>> G = graphs.CycleGraph(Integer(5))
>>> G.edges(sort=True, key=lambda x: (x[Integer(1)], -x[Integer(0)]))
[(0, 1, None), (1, 2, None), (2, 3, None), (3, 4, None), (0, 4, None)]
```

We set the labels to characters and then perform a default sort followed by a sort according to the labels:

```
sage: G = graphs.CycleGraph(5)
sage: for e in G.edges(sort=False):
....:     G.set_edge_label(e[0], e[1], chr(ord('A') + e[0] + 5 * e[1]))
sage: G.edges(sort=True)
[(0, 1, 'F'), (0, 4, 'U'), (1, 2, 'L'), (2, 3, 'R'), (3, 4, 'X')]
sage: G.edges(sort=True, key=lambda x: x[2])
[(0, 1, 'F'), (1, 2, 'L'), (2, 3, 'R'), (0, 4, 'U'), (3, 4, 'X')]
```

```
>>> from sage.all import *
>>> G = graphs.CycleGraph(Integer(5))
>>> for e in G.edges(sort=False):
...     G.set_edge_label(e[Integer(0)], e[Integer(1)], chr(ord('A') +␣
↪e[Integer(0)] + Integer(5) * e[Integer(1)]))
>>> G.edges(sort=True)
[(0, 1, 'F'), (0, 4, 'U'), (1, 2, 'L'), (2, 3, 'R'), (3, 4, 'X')]
>>> G.edges(sort=True, key=lambda x: x[Integer(2)])
[(0, 1, 'F'), (1, 2, 'L'), (2, 3, 'R'), (0, 4, 'U'), (3, 4, 'X')]
```

We can restrict considered edges to those incident to a given set:

```
sage: for i in graphs.PetersenGraph().edges(sort=True, vertices=[0]):
....:     print(i)
(0, 1, None)
(0, 4, None)
(0, 5, None)
sage: D = DiGraph({0: [1, 2], 1: [0]})
sage: for i in D.edges(sort=True, vertices=[0]):
....:     print(i)
(0, 1, None)
(0, 2, None)
```

```
>>> from sage.all import *
>>> for i in graphs.PetersenGraph().edges(sort=True, vertices=[Integer(0)]):
```

(continues on next page)

(continued from previous page)

```
...       print(i)
(0, 1, None)
(0, 4, None)
(0, 5, None)
>>> D = DiGraph({Integer(0): [Integer(1), Integer(2)], Integer(1):␣
↪[Integer(0)]})
>>> for i in D.edges(sort=True, vertices=[Integer(0)]):
...       print(i)
(0, 1, None)
(0, 2, None)
```

Ignoring the direction of edges:

```
sage: D = DiGraph({1: [0], 2: [0]})
sage: D.edges(sort=True, vertices=0)
[]
sage: D.edges(sort=True, vertices=0, ignore_direction=True)
[(1, 0, None), (2, 0, None)]
sage: D.edges(sort=True, vertices=[0], ignore_direction=True)
[(1, 0, None), (2, 0, None)]
```

```
>>> from sage.all import *
>>> D = DiGraph({Integer(1): [Integer(0)], Integer(2): [Integer(0)]})
>>> D.edges(sort=True, vertices=Integer(0))
[]
>>> D.edges(sort=True, vertices=Integer(0), ignore_direction=True)
[(1, 0, None), (2, 0, None)]
>>> D.edges(sort=True, vertices=[Integer(0)], ignore_direction=True)
[(1, 0, None), (2, 0, None)]
```

Not sorting the ends of the edges:

```
sage: G = Graph()
sage: G = Graph()
sage: G.add_edges([[1,2], [2,3], [0,3]])
sage: list(G.edge_iterator(sort_vertices=False))
[(3, 0, None), (2, 1, None), (3, 2, None)]
```

```
>>> from sage.all import *
>>> G = Graph()
>>> G = Graph()
>>> G.add_edges([[Integer(1),Integer(2)], [Integer(2),Integer(3)],␣
↪[Integer(0),Integer(3)]])
>>> list(G.edge_iterator(sort_vertices=False))
[(3, 0, None), (2, 1, None), (3, 2, None)]
```

edges_incident(*vertices=None*, *labels=True*, *sort=False*)

Return incident edges to some vertices.

If `vertices` is a vertex, then it returns the list of edges incident to that vertex. If `vertices` is a list of vertices then it returns the list of all edges adjacent to those vertices. If `vertices` is `None`, it returns a list of all edges in graph. For digraphs, only lists outward edges.

INPUT:

- `vertices` – object (default: `None`); a vertex, a list of vertices or `None`
- `labels` – boolean (default: `True`); if `False`, each edge is a tuple (u, v) of vertices
- `sort` – boolean (default: `False`); if `True` the returned list is sorted

EXAMPLES:

```
sage: graphs.PetersenGraph().edges_incident([0, 9], labels=False)
[(0, 1), (0, 4), (0, 5), (4, 9), (6, 9), (7, 9)]
sage: D = DiGraph({0: [1]})
sage: D.edges_incident([0])
[(0, 1, None)]
sage: D.edges_incident([1])
[]
```

```
>>> from sage.all import *
>>> graphs.PetersenGraph().edges_incident([Integer(0), Integer(9)],␣
↪labels=False)
[(0, 1), (0, 4), (0, 5), (4, 9), (6, 9), (7, 9)]
>>> D = DiGraph({Integer(0): [Integer(1)]})
>>> D.edges_incident([Integer(0)])
[(0, 1, None)]
>>> D.edges_incident([Integer(1)])
[]
```

eigenspaces(*laplacian=False*)

Return the *right* eigenspaces of the adjacency matrix of the graph.

INPUT:

- `laplacian` – boolean (default: `False`); if `True`, use the Laplacian matrix (see *kirchhoff_matrix()*)

OUTPUT:

A list of pairs. Each pair is an eigenvalue of the adjacency matrix of the graph, followed by the vector space that is the eigenspace for that eigenvalue, when the eigenvectors are placed on the right of the matrix.

For some graphs, some of the eigenspaces are described exactly by vector spaces over a `NumberField()`. For numerical eigenvectors use *eigenvectors()*.

EXAMPLES:

```
sage: P = graphs.PetersenGraph()
sage: P.eigenspaces()                                                    #␣
↪needs sage.modules sage.rings.number_field
[(3,
  Vector space of degree 10 and dimension 1 over Rational Field
  User basis matrix:
  [1 1 1 1 1 1 1 1 1 1]),
 (-2,
  Vector space of degree 10 and dimension 4 over Rational Field
  User basis matrix:
  [ 1  0  0  0 -1 -1 -1  0  1  1]
  [ 0  1  0  0 -1  0 -2 -1  1  2]
  [ 0  0  1  0 -1  1 -1 -2  0  2]
  [ 0  0  0  1 -1  1  0 -1 -1  1]),
```

(continues on next page)

(continued from previous page)

```
 (1,
  Vector space of degree 10 and dimension 5 over Rational Field
  User basis matrix:
  [ 1  0  0  0  0  1 -1  0  0 -1]
  [ 0  1  0  0  0 -1  1 -1  0  0]
  [ 0  0  1  0  0  0 -1  1 -1  0]
  [ 0  0  0  1  0  0  0 -1  1 -1]
  [ 0  0  0  0  1 -1  0  0 -1  1])]
```

```
>>> from sage.all import *
>>> P = graphs.PetersenGraph()
>>> P.eigenspaces()                                                    #␣
↪needs sage.modules sage.rings.number_field
[(3,
  Vector space of degree 10 and dimension 1 over Rational Field
  User basis matrix:
  [1 1 1 1 1 1 1 1 1 1]),
 (-2,
  Vector space of degree 10 and dimension 4 over Rational Field
  User basis matrix:
  [ 1  0  0  0 -1 -1 -1  0  1  1]
  [ 0  1  0  0 -1  0 -2 -1  1  2]
  [ 0  0  1  0 -1  1 -1 -2  0  2]
  [ 0  0  0  1 -1  1  0 -1 -1  1]),
 (1,
  Vector space of degree 10 and dimension 5 over Rational Field
  User basis matrix:
  [ 1  0  0  0  0  1 -1  0  0 -1]
  [ 0  1  0  0  0 -1  1 -1  0  0]
  [ 0  0  1  0  0  0 -1  1 -1  0]
  [ 0  0  0  1  0  0  0 -1  1 -1]
  [ 0  0  0  0  1 -1  0  0 -1  1])]
```

Eigenspaces for the Laplacian should be identical since the Petersen graph is regular. However, since the output also contains the eigenvalues, the two outputs are slightly different:

```
sage: P.eigenspaces(laplacian=True)                                    #␣
↪needs sage.modules sage.rings.number_field
[(0,
  Vector space of degree 10 and dimension 1 over Rational Field
  User basis matrix:
  [1 1 1 1 1 1 1 1 1 1]),
 (5,
  Vector space of degree 10 and dimension 4 over Rational Field
  User basis matrix:
  [ 1  0  0  0 -1 -1 -1  0  1  1]
  [ 0  1  0  0 -1  0 -2 -1  1  2]
  [ 0  0  1  0 -1  1 -1 -2  0  2]
  [ 0  0  0  1 -1  1  0 -1 -1  1]),
 (2,
  Vector space of degree 10 and dimension 5 over Rational Field
  User basis matrix:
```

(continues on next page)

(continued from previous page)

```
  [ 1  0  0  0  0  1 -1  0  0 -1]
  [ 0  1  0  0  0 -1  1 -1  0  0]
  [ 0  0  1  0  0  0 -1  1 -1  0]
  [ 0  0  0  1  0  0  0 -1  1 -1]
  [ 0  0  0  0  1 -1  0  0 -1  1])]
```

```
>>> from sage.all import *
>>> P.eigenspaces(laplacian=True)                                              #␣
↪needs sage.modules sage.rings.number_field
[(0,
  Vector space of degree 10 and dimension 1 over Rational Field
  User basis matrix:
  [1 1 1 1 1 1 1 1 1 1]),
 (5,
  Vector space of degree 10 and dimension 4 over Rational Field
  User basis matrix:
  [ 1  0  0  0 -1 -1 -1  0  1  1]
  [ 0  1  0  0 -1  0 -2 -1  1  2]
  [ 0  0  1  0 -1  1 -1 -2  0  2]
  [ 0  0  0  1 -1  1  0 -1 -1  1]),
 (2,
  Vector space of degree 10 and dimension 5 over Rational Field
  User basis matrix:
  [ 1  0  0  0  0  1 -1  0  0 -1]
  [ 0  1  0  0  0 -1  1 -1  0  0]
  [ 0  0  1  0  0  0 -1  1 -1  0]
  [ 0  0  0  1  0  0  0 -1  1 -1]
  [ 0  0  0  0  1 -1  0  0 -1  1])]
```

Notice how one eigenspace below is described with a square root of 2. For the two possible values (positive and negative) there is a corresponding eigenspace:

```
sage: C = graphs.CycleGraph(8)
sage: C.eigenspaces()                                                          #␣
↪needs sage.modules sage.rings.number_field
[(2,
  Vector space of degree 8 and dimension 1 over Rational Field
  User basis matrix:
  [1 1 1 1 1 1 1 1]),
 (-2,
  Vector space of degree 8 and dimension 1 over Rational Field
  User basis matrix:
  [ 1 -1  1 -1  1 -1  1 -1]),
 (0,
  Vector space of degree 8 and dimension 2 over Rational Field
  User basis matrix:
  [ 1  0 -1  0  1  0 -1  0]
  [ 0  1  0 -1  0  1  0 -1]),
 (a3,
  Vector space of degree 8 and dimension 2 over Number Field in a3 with␣
↪defining polynomial x^2 - 2
  User basis matrix:
```

(continues on next page)

(continued from previous page)

```
  [  1   0  -1 -a3  -1   0   1  a3]
  [  0   1  a3   1   0  -1 -a3  -1])]
```

```
>>> from sage.all import *
>>> C = graphs.CycleGraph(Integer(8))
>>> C.eigenspaces()                                                          #␣
↪needs sage.modules sage.rings.number_field
[(2,
  Vector space of degree 8 and dimension 1 over Rational Field
  User basis matrix:
  [1 1 1 1 1 1 1 1]),
 (-2,
  Vector space of degree 8 and dimension 1 over Rational Field
  User basis matrix:
  [ 1 -1  1 -1  1 -1  1 -1]),
 (0,
  Vector space of degree 8 and dimension 2 over Rational Field
  User basis matrix:
  [ 1  0 -1  0  1  0 -1  0]
  [ 0  1  0 -1  0  1  0 -1]),
 (a3,
  Vector space of degree 8 and dimension 2 over Number Field in a3 with␣
↪defining polynomial x^2 - 2
  User basis matrix:
  [  1   0  -1 -a3  -1   0   1  a3]
  [  0   1  a3   1   0  -1 -a3  -1])]
```

A digraph may have complex eigenvalues and eigenvectors. For a 3-cycle, we have:

```
sage: T = DiGraph({0: [1], 1: [2], 2: [0]})
sage: T.eigenspaces()                                                        #␣
↪needs sage.modules sage.rings.number_field
[(1,
  Vector space of degree 3 and dimension 1 over Rational Field
  User basis matrix:
  [1 1 1]),
 (a1,
  Vector space of degree 3 and dimension 1 over Number Field in a1 with␣
↪defining polynomial x^2 + x + 1
  User basis matrix:
  [      1      a1 -a1 - 1])]
```

```
>>> from sage.all import *
>>> T = DiGraph({Integer(0): [Integer(1)], Integer(1): [Integer(2)],␣
↪Integer(2): [Integer(0)]})
>>> T.eigenspaces()                                                          #␣
↪needs sage.modules sage.rings.number_field
[(1,
  Vector space of degree 3 and dimension 1 over Rational Field
  User basis matrix:
  [1 1 1]),
 (a1,
```

(continues on next page)

(continued from previous page)

```
 Vector space of degree 3 and dimension 1 over Number Field in a1 with␣
↪defining polynomial x^2 + x + 1
 User basis matrix:
 [      1      a1 -a1 - 1])]
```

eigenvectors(*laplacian=False*)

Return the *right* eigenvectors of the adjacency matrix of the graph.

INPUT:

- `laplacian` – boolean (default: `False`); if `True`, use the Laplacian matrix (see *kirchhoff_matrix()*)

OUTPUT:

A list of triples. Each triple begins with an eigenvalue of the adjacency matrix of the graph. This is followed by a list of eigenvectors for the eigenvalue, when the eigenvectors are placed on the right side of the matrix. Together, the eigenvectors form a basis for the eigenspace. The triple concludes with the algebraic multiplicity of the eigenvalue.

For some graphs, the exact eigenspaces provided by *eigenspaces()* provide additional insight into the structure of the eigenspaces.

EXAMPLES:

```
sage: P = graphs.PetersenGraph()
sage: P.eigenvectors()                                                      #␣
↪needs sage.modules sage.rings.number_field
[(3, [(1, 1, 1, 1, 1, 1, 1, 1, 1, 1)], 1),
 (-2,
  [(1, 0, 0, 0, -1, -1, -1, 0, 1, 1),
   (0, 1, 0, 0, -1, 0, -2, -1, 1, 2),
   (0, 0, 1, 0, -1, 1, -1, -2, 0, 2),
   (0, 0, 0, 1, -1, 1, 0, -1, -1, 1)],
  4),
 (1,
  [(1, 0, 0, 0, 0, 1, -1, 0, 0, -1),
   (0, 1, 0, 0, 0, -1, 1, -1, 0, 0),
   (0, 0, 1, 0, 0, 0, -1, 1, -1, 0),
   (0, 0, 0, 1, 0, 0, 0, -1, 1, -1),
   (0, 0, 0, 0, 1, -1, 0, 0, -1, 1)],
  5)]
```

```
>>> from sage.all import *
>>> P = graphs.PetersenGraph()
>>> P.eigenvectors()                                                        #␣
↪needs sage.modules sage.rings.number_field
[(3, [(1, 1, 1, 1, 1, 1, 1, 1, 1, 1)], 1),
 (-2,
  [(1, 0, 0, 0, -1, -1, -1, 0, 1, 1),
   (0, 1, 0, 0, -1, 0, -2, -1, 1, 2),
   (0, 0, 1, 0, -1, 1, -1, -2, 0, 2),
   (0, 0, 0, 1, -1, 1, 0, -1, -1, 1)],
  4),
 (1,
```

(continues on next page)

(continued from previous page)

```
  [(1, 0, 0, 0, 0, 1, -1, 0, 0, -1),
   (0, 1, 0, 0, 0, -1, 1, -1, 0, 0),
   (0, 0, 1, 0, 0, 0, -1, 1, -1, 0),
   (0, 0, 0, 1, 0, 0, 0, -1, 1, -1),
   (0, 0, 0, 0, 1, -1, 0, 0, -1, 1)],
  5)]
```

Eigenspaces for the Laplacian should be identical since the Petersen graph is regular. However, since the output also contains the eigenvalues, the two outputs are slightly different:

```
sage: P.eigenvectors(laplacian=True)                                    #␣
↪needs sage.modules sage.rings.number_field
[(0, [(1, 1, 1, 1, 1, 1, 1, 1, 1, 1)], 1),
 (5,
  [(1, 0, 0, 0, -1, -1, -1, 0, 1, 1),
   (0, 1, 0, 0, -1, 0, -2, -1, 1, 2),
   (0, 0, 1, 0, -1, 1, -1, -2, 0, 2),
   (0, 0, 0, 1, -1, 1, 0, -1, -1, 1)],
  4),
 (2,
  [(1, 0, 0, 0, 0, 1, -1, 0, 0, -1),
   (0, 1, 0, 0, 0, -1, 1, -1, 0, 0),
   (0, 0, 1, 0, 0, 0, -1, 1, -1, 0),
   (0, 0, 0, 1, 0, 0, 0, -1, 1, -1),
   (0, 0, 0, 0, 1, -1, 0, 0, -1, 1)],
  5)]
```

```
>>> from sage.all import *
>>> P.eigenvectors(laplacian=True)                                      #␣
↪needs sage.modules sage.rings.number_field
[(0, [(1, 1, 1, 1, 1, 1, 1, 1, 1, 1)], 1),
 (5,
  [(1, 0, 0, 0, -1, -1, -1, 0, 1, 1),
   (0, 1, 0, 0, -1, 0, -2, -1, 1, 2),
   (0, 0, 1, 0, -1, 1, -1, -2, 0, 2),
   (0, 0, 0, 1, -1, 1, 0, -1, -1, 1)],
  4),
 (2,
  [(1, 0, 0, 0, 0, 1, -1, 0, 0, -1),
   (0, 1, 0, 0, 0, -1, 1, -1, 0, 0),
   (0, 0, 1, 0, 0, 0, -1, 1, -1, 0),
   (0, 0, 0, 1, 0, 0, 0, -1, 1, -1),
   (0, 0, 0, 0, 1, -1, 0, 0, -1, 1)],
  5)]
```

```
sage: C = graphs.CycleGraph(8)
sage: C.eigenvectors()                                                  #␣
↪needs sage.modules sage.rings.number_field
[(2, [(1, 1, 1, 1, 1, 1, 1, 1)], 1),
 (-2, [(1, -1, 1, -1, 1, -1, 1, -1)], 1),
 (0, [(1, 0, -1, 0, 1, 0, -1, 0), (0, 1, 0, -1, 0, 1, 0, -1)], 2),
 (-1.414213562373095?,
```

(continues on next page)

(continued from previous page)

```
  [(1, 0, -1, 1.4142135623730950?, -1, 0, 1, -1.4142135623730950?),
   (0, 1, -1.4142135623730950?, 1, 0, -1, 1.4142135623730950?, -1)],
  2),
 (1.4142135623730950?,
  [(1, 0, -1, -1.4142135623730950?, -1, 0, 1, 1.4142135623730950?),
   (0, 1, 1.4142135623730950?, 1, 0, -1, -1.4142135623730950?, -1)],
  2)]
```

```
>>> from sage.all import *
>>> C = graphs.CycleGraph(Integer(8))
>>> C.eigenvectors()                                                    #␣
↪needs sage.modules sage.rings.number_field
[(2, [(1, 1, 1, 1, 1, 1, 1, 1)], 1),
 (-2, [(1, -1, 1, -1, 1, -1, 1, -1)], 1),
 (0, [(1, 0, -1, 0, 1, 0, -1, 0), (0, 1, 0, -1, 0, 1, 0, -1)], 2),
 (-1.4142135623730950?,
  [(1, 0, -1, 1.4142135623730950?, -1, 0, 1, -1.4142135623730950?),
   (0, 1, -1.4142135623730950?, 1, 0, -1, 1.4142135623730950?, -1)],
  2),
 (1.4142135623730950?,
  [(1, 0, -1, -1.4142135623730950?, -1, 0, 1, 1.4142135623730950?),
   (0, 1, 1.4142135623730950?, 1, 0, -1, -1.4142135623730950?, -1)],
  2)]
```

A digraph may have complex eigenvalues. Previously, the complex parts of graph eigenvalues were being dropped. For a 3-cycle, we have:

```
sage: T = DiGraph({0:[1], 1:[2], 2:[0]})
sage: T.eigenvectors()                                                  #␣
↪needs sage.modules sage.rings.number_field
[(1, [(1, 1, 1)], 1),
 (-0.50000000000000000? - 0.8660254037844439?*I,
  [(1, -0.50000000000000000? - 0.8660254037844439?*I, -0.50000000000000000? +␣
↪0.8660254037844439?*I)],
  1),
 (-0.50000000000000000? + 0.8660254037844439?*I,
  [(1, -0.50000000000000000? + 0.8660254037844439?*I, -0.50000000000000000? -␣
↪0.8660254037844439?*I)],
  1)]
```

```
>>> from sage.all import *
>>> T = DiGraph({Integer(0):[Integer(1)], Integer(1):[Integer(2)],␣
↪Integer(2):[Integer(0)]})
>>> T.eigenvectors()                                                    #␣
↪needs sage.modules sage.rings.number_field
[(1, [(1, 1, 1)], 1),
 (-0.50000000000000000? - 0.8660254037844439?*I,
  [(1, -0.50000000000000000? - 0.8660254037844439?*I, -0.50000000000000000? +␣
↪0.8660254037844439?*I)],
  1),
 (-0.50000000000000000? + 0.8660254037844439?*I,
  [(1, -0.50000000000000000? + 0.8660254037844439?*I, -0.50000000000000000? -␣
```

(continues on next page)

(continued from previous page)

```
↪0.866025403784439?*I)],
 1)]
```

eulerian_circuit(*return_vertices=False*, *labels=True*, *path=False*)

Return a list of edges forming an Eulerian circuit if one exists.

If no Eulerian circuit is found, the method returns `False`.

This is implemented using Hierholzer's algorithm.

INPUT:

- `return_vertices` – boolean (default: `False`); optionally provide a list of vertices for the path
- `labels` – boolean (default: `True`); whether to return edges with labels (3-tuples)
- `path` – boolean (default: `False`); find an Eulerian path instead

OUTPUT:

either ([edges], [vertices]) or [edges] of an Eulerian circuit (or path)

EXAMPLES:

```
sage: g = graphs.CycleGraph(5)
sage: g.eulerian_circuit()
[(0, 4, None), (4, 3, None), (3, 2, None), (2, 1, None), (1, 0, None)]
sage: g.eulerian_circuit(labels=False)
[(0, 4), (4, 3), (3, 2), (2, 1), (1, 0)]
```

```
>>> from sage.all import *
>>> g = graphs.CycleGraph(Integer(5))
>>> g.eulerian_circuit()
[(0, 4, None), (4, 3, None), (3, 2, None), (2, 1, None), (1, 0, None)]
>>> g.eulerian_circuit(labels=False)
[(0, 4), (4, 3), (3, 2), (2, 1), (1, 0)]
```

```
sage: g = graphs.CompleteGraph(7)
sage: edges, vertices = g.eulerian_circuit(return_vertices=True)
sage: vertices
[0, 6, 5, 4, 6, 3, 5, 2, 4, 3, 2, 6, 1, 5, 0, 4, 1, 3, 0, 2, 1, 0]
```

```
>>> from sage.all import *
>>> g = graphs.CompleteGraph(Integer(7))
>>> edges, vertices = g.eulerian_circuit(return_vertices=True)
>>> vertices
[0, 6, 5, 4, 6, 3, 5, 2, 4, 3, 2, 6, 1, 5, 0, 4, 1, 3, 0, 2, 1, 0]
```

```
sage: graphs.CompleteGraph(4).eulerian_circuit()
False
```

```
>>> from sage.all import *
>>> graphs.CompleteGraph(Integer(4)).eulerian_circuit()
False
```

A disconnected graph can be Eulerian:

```
sage: g = Graph({0: [], 1: [2], 2: [3], 3: [1], 4: []})
sage: g.eulerian_circuit(labels=False)
[(1, 3), (3, 2), (2, 1)]
```

```
>>> from sage.all import *
>>> g = Graph({Integer(0): [], Integer(1): [Integer(2)], Integer(2):␣
↪[Integer(3)], Integer(3): [Integer(1)], Integer(4): []})
>>> g.eulerian_circuit(labels=False)
[(1, 3), (3, 2), (2, 1)]
```

```
sage: g = DiGraph({0: [1], 1: [2, 4], 2:[3], 3:[1]})
sage: g.eulerian_circuit(labels=False, path=True)
[(0, 1), (1, 2), (2, 3), (3, 1), (1, 4)]
```

```
>>> from sage.all import *
>>> g = DiGraph({Integer(0): [Integer(1)], Integer(1): [Integer(2),␣
↪Integer(4)], Integer(2):[Integer(3)], Integer(3):[Integer(1)]})
>>> g.eulerian_circuit(labels=False, path=True)
[(0, 1), (1, 2), (2, 3), (3, 1), (1, 4)]
```

```
sage: g = Graph({0:[1,2,3], 1:[2,3], 2:[3,4], 3:[4]})
sage: g.is_eulerian(path=True)
(0, 1)
sage: g.eulerian_circuit(labels=False, path=True)
[(1, 3), (3, 4), (4, 2), (2, 3), (3, 0), (0, 2), (2, 1), (1, 0)]
```

```
>>> from sage.all import *
>>> g = Graph({Integer(0):[Integer(1),Integer(2),Integer(3)],␣
↪Integer(1):[Integer(2),Integer(3)], Integer(2):[Integer(3),Integer(4)],␣
↪Integer(3):[Integer(4)]})
>>> g.is_eulerian(path=True)
(0, 1)
>>> g.eulerian_circuit(labels=False, path=True)
[(1, 3), (3, 4), (4, 2), (2, 3), (3, 0), (0, 2), (2, 1), (1, 0)]
```

export_to_file(*filename*, *format=None*, ***kwds*)

Export the graph to a file.

INPUT:

- `filename` – string; a file name
- `format` – string (default: `None`); select the output format explicitly. If set to `None` (default), the format is set to be the file extension of `filename`. Admissible formats are: `'adjlist'`, `'dot'`, `'edgelist'`, `'gexf'`, `'gml'`, `'graphml'`, `'multiline_adjlist'`, `'pajek'`.
- All other arguments are forwarded to the subfunction. For more information, see their respective documentation:

'adjlist'	networkx.readwrite.adjlist.write_adjlist()
'dot'	networkx.drawing.nx_pydot.write_dot()
'edgelist'	networkx.readwrite.edgelist.write_edgelist()
'gexf'	networkx.readwrite.gexf.write_gexf()
'gml'	networkx.readwrite.gml.write_gml()
'graphml'	networkx.readwrite.graphml.write_graphml()
'multiline_adjlist'	networkx.readwrite.multiline_adjlist.write_multiline_adjlist()
'pajek'	networkx.readwrite.pajek.write_pajek()

See also

- save() – save a Sage object to a 'sobj' file (preserves all its attributes)

Note

This functions uses the write_* functions defined in NetworkX (see networkx.readwrite).

EXAMPLES:

```
sage: g = graphs.PetersenGraph()
sage: filename = tmp_filename(ext='.pajek')
sage: g.export_to_file(filename)                                            #␣
↪needs networkx
sage: import networkx                                                       #␣
↪needs networkx
sage: G_networkx = networkx.read_pajek(filename)                            #␣
↪needs networkx
sage: Graph(G_networkx).is_isomorphic(g)                                    #␣
↪needs networkx
True
sage: filename = tmp_filename(ext='.edgelist')
sage: g.export_to_file(filename, data=False)                                #␣
↪needs networkx
sage: h = Graph(networkx.read_edgelist(filename))                           #␣
↪needs networkx
sage: g.is_isomorphic(h)                                                    #␣
↪needs networkx
True
```

```
>>> from sage.all import *
>>> g = graphs.PetersenGraph()
>>> filename = tmp_filename(ext='.pajek')
>>> g.export_to_file(filename)                                              #␣
↪needs networkx
>>> import networkx                                                         #␣
↪needs networkx
>>> G_networkx = networkx.read_pajek(filename)                              #␣
```

(continues on next page)

(continued from previous page)

```
↪needs networkx
>>> Graph(G_networkx).is_isomorphic(g)                                    #␣
↪needs networkx
True
>>> filename = tmp_filename(ext='.edgelist')
>>> g.export_to_file(filename, data=False)                                #␣
↪needs networkx
>>> h = Graph(networkx.read_edgelist(filename))                           #␣
↪needs networkx
>>> g.is_isomorphic(h)                                                    #␣
↪needs networkx
True
```

faces(*embedding=None*)

Return the faces of an embedded graph.

A combinatorial embedding of a graph is a clockwise ordering of the neighbors of each vertex. From this information one can define the faces of the embedding, which is what this method returns.

If no embedding is provided or stored as `self._embedding`, this method will compute the set of faces from the embedding returned by *is_planar()* (if the graph is, of course, planar).

Warning

This method is not well defined when the graph is not connected. Indeed, the result may contain several faces corresponding to the external face.

INPUT:

- `embedding` – dictionary (default: `None`); a combinatorial embedding dictionary. Format: `{v1: [v2, v3], v2: [v1], v3: [v1]}` (clockwise ordering of neighbors at each vertex). If set to `None` (default) the method will use the embedding stored as `self._embedding`. If none is stored, the method will compute the set of faces from the embedding returned by *is_planar()* (if the graph is, of course, planar).

Note

`embedding` is an ordered list based on the hash order of the vertices of graph. To avoid confusion, it might be best to set the rot_sys based on a 'nice_copy' of the graph.

See also

- *set_embedding()*
- *get_embedding()*
- *is_planar()*
- *planar_dual()*

EXAMPLES:

Providing an embedding:

```
sage: T = graphs.TetrahedralGraph()
sage: T.faces({0: [1, 3, 2], 1: [0, 2, 3], 2: [0, 3, 1], 3: [0, 1, 2]})
[[(0, 1), (1, 2), (2, 0)],
 [(0, 2), (2, 3), (3, 0)],
 [(0, 3), (3, 1), (1, 0)],
 [(1, 3), (3, 2), (2, 1)]]
```

```
>>> from sage.all import *
>>> T = graphs.TetrahedralGraph()
>>> T.faces({Integer(0): [Integer(1), Integer(3), Integer(2)], Integer(1):␣
↪[Integer(0), Integer(2), Integer(3)], Integer(2): [Integer(0), Integer(3),␣
↪Integer(1)], Integer(3): [Integer(0), Integer(1), Integer(2)]})
[[(0, 1), (1, 2), (2, 0)],
 [(0, 2), (2, 3), (3, 0)],
 [(0, 3), (3, 1), (1, 0)],
 [(1, 3), (3, 2), (2, 1)]]
```

With no embedding provided:

```
sage: graphs.TetrahedralGraph().faces()
[[(0, 1), (1, 2), (2, 0)],
 [(0, 2), (2, 3), (3, 0)],
 [(0, 3), (3, 1), (1, 0)],
 [(1, 3), (3, 2), (2, 1)]]
```

```
>>> from sage.all import *
>>> graphs.TetrahedralGraph().faces()
[[(0, 1), (1, 2), (2, 0)],
 [(0, 2), (2, 3), (3, 0)],
 [(0, 3), (3, 1), (1, 0)],
 [(1, 3), (3, 2), (2, 1)]]
```

With no embedding provided (non-planar graph):

```
sage: graphs.PetersenGraph().faces()
Traceback (most recent call last):
...
ValueError: no embedding is provided and the graph is not planar
```

```
>>> from sage.all import *
>>> graphs.PetersenGraph().faces()
Traceback (most recent call last):
...
ValueError: no embedding is provided and the graph is not planar
```

feedback_vertex_set(*value_only*, *solver=False*, *verbose=None*, *constraint_generation=0*, *integrality_tolerance=True*)

Return the minimum feedback vertex set of a (di)graph.

The minimum feedback vertex set of a (di)graph is a set of vertices that intersect all of its cycles. Equivalently, a minimum feedback vertex set of a (di)graph is a set S of vertices such that the digraph $G - S$ is acyclic. For more information, see the Wikipedia article Feedback_vertex_set.

INPUT:

- `value_only` – boolean (default: `False`); whether to return only the minimum cardinal of a minimum vertex set, or the `Set` of vertices of a minimal feedback vertex set
- `solver` – string (default: `None`); specifies a Mixed Integer Linear Programming (MILP) solver to be used. If set to `None`, the default one is used. For more information on MILP solvers and which default solver is used, see the method `solve` of the class `MixedIntegerLinearProgram`.
- `verbose` – integer (default: 0); sets the level of verbosity. Set to 0 by default, which means quiet.
- `constraint_generation` – boolean (default: `True`); whether to use constraint generation when solving the Mixed Integer Linear Program
- `integrality_tolerance` – float; parameter for use with MILP solvers over an inexact base ring; see `MixedIntegerLinearProgram.get_values()`.

ALGORITHMS:

(Constraints generation)

When the parameter `constraint_generation` is enabled (default) the following MILP formulation is used to solve the problem:

$$\text{Minimize : } \sum_{v \in G} b_v$$

$$\text{Such that :}$$

$$\forall C \text{ circuits } \subseteq G, \sum_{v \in C} b_v \geq 1$$

As the number of circuits contained in a graph is exponential, this LP is solved through constraint generation. This means that the solver is sequentially asked to solve the problem, knowing only a portion of the circuits contained in G, each time adding to the list of its constraints the circuit which its last answer had left intact.

(Another formulation based on an ordering of the vertices)

When the graph is directed, a second (and very slow) formulation is available, which should only be used to check the result of the first implementation in case of doubt.

$$\text{Minimize : } \sum_{v \in G} b_v$$

$$\text{Such that :}$$

$$\forall (u,v) \in G, d_u - d_v + n b_u + n b_v \geq 0$$
$$\forall u \in G, 0 \leq d_u \leq |G|$$

A brief explanation:

An acyclic digraph can be seen as a poset, and every poset has a linear extension. This means that in any acyclic digraph the vertices can be ordered with a total order $<$ in such a way that if $(u, v) \in G$, then $u < v$. Thus, this linear program is built in order to assign to each vertex v a number $d_v \in [0, \ldots, n-1]$ such that if there exists an edge $(u, v) \in G$ then either $d_v < d_u$ or one of u or v is removed. The number of vertices removed is then minimized, which is the objective.

EXAMPLES:

The necessary example:

```
sage: g = graphs.PetersenGraph()
sage: fvs = g.feedback_vertex_set()
```

(continues on next page)

(continued from previous page)

```
sage: len(fvs)
3
sage: g.delete_vertices(fvs)
sage: g.is_forest()
True
```

```
>>> from sage.all import *
>>> g = graphs.PetersenGraph()
>>> fvs = g.feedback_vertex_set()
>>> len(fvs)
3
>>> g.delete_vertices(fvs)
>>> g.is_forest()
True
```

In a digraph built from a graph, any edge is replaced by arcs going in the two opposite directions, thus creating a cycle of length two. Hence, to remove all the cycles from the graph, each edge must see one of its neighbors removed: a feedback vertex set is in this situation a vertex cover:

```
sage: cycle = graphs.CycleGraph(5)
sage: dcycle = DiGraph(cycle)
sage: cycle.vertex_cover(value_only=True)
3
sage: feedback = dcycle.feedback_vertex_set()
sage: len(feedback)
3
sage: u,v = next(cycle.edge_iterator(labels=None))
sage: u in feedback or v in feedback
True
```

```
>>> from sage.all import *
>>> cycle = graphs.CycleGraph(Integer(5))
>>> dcycle = DiGraph(cycle)
>>> cycle.vertex_cover(value_only=True)
3
>>> feedback = dcycle.feedback_vertex_set()
>>> len(feedback)
3
>>> u,v = next(cycle.edge_iterator(labels=None))
>>> u in feedback or v in feedback
True
```

For a circuit, the minimum feedback arc set is clearly 1:

```
sage: circuit = digraphs.Circuit(5)
sage: circuit.feedback_vertex_set(value_only=True) == 1                    #␣
↪needs sage.numerical.mip
True
```

```
>>> from sage.all import *
>>> circuit = digraphs.Circuit(Integer(5))
>>> circuit.feedback_vertex_set(value_only=True) == Integer(1)              ␣
```

(continues on next page)

(continued from previous page)

```
↪      # needs sage.numerical.mip
True
```

flow(*x*, *y*, *value_only*, *integer=True*, *by_weight=False*, *weight_function=True*, *check_weight=None*, *vertex_bound=True*, *algorithm=False*, *solver=None*, *verbose=None*, *integrality_tolerance=0*)

Return a maximum flow in the graph from `x` to `y`.

The returned flow is represented by an optimal valuation of the edges. For more information, see the Wikipedia article Max_flow.

As an optimization problem, is can be expressed this way :

$$\text{Maximize : } \sum_{e \in G.edges()} w_e b_e$$

$$\text{Such that : } \forall v \in G, \sum_{(u,v) \in G.edges()} b_{(u,v)} \leq 1$$

$$\forall x \in G, b_x \text{ is a binary variable}$$

Observe that the integrality of the flow variables is automatic for all available solvers when all capacities are integers.

INPUT:

- `x` – source vertex
- `y` – sink vertex
- `value_only` – boolean (default: `True`); whether to return only the value of a maximal flow, or to also return a flow graph (a copy of the current graph, such that each edge has the flow using it as a label, the edges without flow being omitted)
- `integer` – boolean (default: `True`); whether to compute an optimal solution under the constraint that the flow going through an edge has to be an integer, or without this constraint
- `by_weight` – boolean (default: `True`); if `True`, computes a maximum flow where the capacity of an edge is given by `weight_function`, or by its label if `weight_function` is `None`. If `False`, every edge has capacity 1.
- `weight_function` – function (default: `None`); a function that takes as input an edge `(u, v, l)` and outputs its capacity. If not `None`, `by_weight` is automatically set to `True`. If `None` and `by_weight` is `True`, we use the edge label `l`, if `l` is not `None`, else 1 as a capacity.
- `check_weight` – boolean (default: `True`); whether to check that the `weight_function` outputs a number for each edge
- `vertex_bound` – boolean (default: `False`); when set to `True`, sets the maximum flow leaving a vertex different from x to 1 (useful for vertex connectivity parameters)
- `algorithm` – string (default: `None`); the algorithm to use among:
  - `'FF'`, a Python implementation of the Ford-Fulkerson algorithm (only available when `vertex_bound = False`)
  - `'LP'`, the flow problem is solved using Linear Programming
  - `'igraph'`, the `igraph` implementation of the Goldberg-Tarjan algorithm is used (only available when `igraph` is installed and `vertex_bound = False`)

  When `algorithm = None` (default), we use `LP` if `vertex_bound = True`, otherwise, we use `igraph` if it is available, `FF` if it is not available.

- `solver` – string (default: `None`); specifies a Mixed Integer Linear Programming (MILP) solver to be used. If set to `None`, the default one is used. For more information on MILP solvers and which default solver is used, see the method `solve` of the class `MixedIntegerLinearProgram`.

  Only useful when algorithm `'LP'` is used to solve the flow problem.

- `verbose` – integer (default: 0); sets the level of verbosity. Set to 0 by default, which means quiet.

  Only useful when algorithm `'LP'` is used to solve the flow problem.

- `integrality_tolerance` – float; parameter for use with MILP solvers over an inexact base ring; see `MixedIntegerLinearProgram.get_values()`.

  Only useful when `algorithm == "LP"` and `integer == True`.

Note

Even though the three different implementations are meant to return the same Flow values, they cannot be expected to return the same Flow graphs.

Besides, the use of Linear Programming may possibly mean a (slight) numerical noise.

EXAMPLES:

Two basic applications of the flow method for the `PappusGraph` and the `ButterflyGraph` with parameter 2

```
sage: g=graphs.PappusGraph()
sage: int(g.flow(1,2))
3
```

```
>>> from sage.all import *
>>> g=graphs.PappusGraph()
>>> int(g.flow(Integer(1),Integer(2)))
3
```

```
sage: b=digraphs.ButterflyGraph(2)
sage: int(b.flow(('00', 1), ('00', 2)))
1
```

```
>>> from sage.all import *
>>> b=digraphs.ButterflyGraph(Integer(2))
>>> int(b.flow(('00', Integer(1)), ('00', Integer(2))))
1
```

The flow method can be used to compute a matching in a bipartite graph by linking a source s to all the vertices of the first set and linking a sink t to all the vertices of the second set, then computing a maximum $s - t$ flow

```
sage: g = DiGraph()
sage: g.add_edges(('s', i) for i in range(4))
sage: g.add_edges((i, 4 + j) for i in range(4) for j in range(4))
sage: g.add_edges((4 + i, 't') for i in range(4))
sage: [cardinal, flow_graph] = g.flow('s', 't', integer=True, value_
↪only=False)
```

(continues on next page)

(continued from previous page)

```
sage: flow_graph.delete_vertices(['s', 't'])
sage: flow_graph.size()
4
```

```
>>> from sage.all import *
>>> g = DiGraph()
>>> g.add_edges(('s', i) for i in range(Integer(4)))
>>> g.add_edges((i, Integer(4) + j) for i in range(Integer(4)) for j in␣
↪range(Integer(4)))
>>> g.add_edges((Integer(4) + i, 't') for i in range(Integer(4)))
>>> [cardinal, flow_graph] = g.flow('s', 't', integer=True, value_only=False)
>>> flow_graph.delete_vertices(['s', 't'])
>>> flow_graph.size()
4
```

The undirected case:

```
sage: g = Graph()
sage: g.add_edges(('s', i) for i in range(4))
sage: g.add_edges((i, 4 + j) for i in range(4) for j in range(4))
sage: g.add_edges((4 + i, 't') for i in range(4))
sage: [cardinal, flow_graph] = g.flow('s', 't', integer=True, value_
↪only=False)
sage: flow_graph.delete_vertices(['s', 't'])
sage: flow_graph.size()
4
```

```
>>> from sage.all import *
>>> g = Graph()
>>> g.add_edges(('s', i) for i in range(Integer(4)))
>>> g.add_edges((i, Integer(4) + j) for i in range(Integer(4)) for j in␣
↪range(Integer(4)))
>>> g.add_edges((Integer(4) + i, 't') for i in range(Integer(4)))
>>> [cardinal, flow_graph] = g.flow('s', 't', integer=True, value_only=False)
>>> flow_graph.delete_vertices(['s', 't'])
>>> flow_graph.size()
4
```

A `weight_function` can read the capacity from anywhere in the edge, and setting it implies `by_weight=True`:

```
sage: D = DiGraph([(0, 1, {'cap': 3}), (1, 2, {'cap': 2})])
sage: D.flow(0, 2, weight_function=lambda e: e[2]['cap'])                    #␣
↪needs sage.numerical.mip
2
```

```
>>> from sage.all import *
>>> D = DiGraph([(Integer(0), Integer(1), {'cap': Integer(3)}), (Integer(1),␣
↪Integer(2), {'cap': Integer(2)})])
>>> D.flow(Integer(0), Integer(2), weight_function=lambda e: e[Integer(2)][
↪'cap'])                    # needs sage.numerical.mip
2
```

genus(*set_embedding=True*, *on_embedding=None*, *minimal=True*, *maximal=False*, *circular=None*, *ordered=True*, *algorithm=None*)

Return the minimal genus of the graph.

The genus of a compact surface is the number of handles it has. The genus of a graph is the minimal genus of the surface it can be embedded into. It can be seen as a measure of non-planarity; a planar graph has genus zero.

Note

This function uses Euler's formula and thus it is necessary to consider only connected graphs.

INPUT:

- `set_embedding` – boolean (default: `True`); whether or not to store an embedding attribute of the computed (minimal) genus of the graph
- **`on_embedding` – two kinds of input are allowed (default:** `None`):
  - a dictionary representing a combinatorial embedding on which the genus should be computed. Note that this must be a valid embedding for the graph. The dictionary structure is given by: `vertex1: [neighbor1, neighbor2, neighbor3], vertex2: [neighbor]` where there is a key for each vertex in the graph and a (clockwise) ordered list of each vertex's neighbors as values. The value of `on_embedding` takes precedence over a stored `_embedding` attribute if `minimal` is set to `False`.
  - The value `True`, in order to indicate that the embedding stored as `_embedding` should be used (see examples).
- `minimal` – boolean (default: `True`); whether or not to compute the minimal genus of the graph (i.e., testing all embeddings). If minimal is `False`, then either `maximal` must be `True` or `on_embedding` must not be `None`. If `on_embedding` is not `None`, it will take priority over `minimal`. Similarly, if `maximal` is `True`, it will take priority over `minimal`.
- `maximal` – boolean (default: `False`); whether or not to compute the maximal genus of the graph (i.e., testing all embeddings). If `maximal` is `False`, then either `minimal` must be `True` or `on_embedding` must not be `None`. If `on_embedding` is not `None`, it will take priority over `maximal`. However, `maximal` takes priority over the default `minimal`.
- `circular` – list (default: `None`); if `circular` is a list of vertices, the method computes the genus preserving a planar embedding of the this list. If `circular` is defined, `on_embedding` is not a valid option.
- `ordered` – boolean (default: `True`); if `circular` is `True`, then whether or not the boundary order may be permuted (default: `True`, which means the boundary order is preserved)
- `algorithm` – string or `None` (default: `None`); the algorithm to use when computing minimum genus. It must be one of `'page'`, `'multi_genus'`, or `'simple'`. If `None`, Sage uses `'page'` when the optional graph_genus package is installed and otherwise uses `'simple'`. The optional `'page'` algorithm is usually fastest on sparse low-degree graphs and graphs with high girth. The optional `'multi_genus'` algorithm is often very fast on small dense graphs and uses fewer CPU cores, but has fixed C integer-size limits. The built-in `'simple'` algorithm enumerates rotation systems and is useful for computing maximum genus.

EXAMPLES:

```
sage: g = graphs.PetersenGraph()
sage: g.genus() # tests for minimal genus by default
1
sage: g.genus(on_embedding=True, maximal=True) # on_embedding overrides␣
↪minimal and maximal arguments
1
sage: g.genus(maximal=True) # setting maximal to True overrides default␣
↪minimal=True
3
sage: g.genus(on_embedding=g.get_embedding()) # can also send a valid␣
↪combinatorial embedding dict
3
sage: (graphs.CubeGraph(3)).genus()
0
sage: K23 = graphs.CompleteBipartiteGraph(2,3)
sage: K23.genus()
0
sage: K33 = graphs.CompleteBipartiteGraph(3,3)
sage: K33.genus()
1
sage: K33.genus(algorithm='simple')
1
sage: K33.genus(algorithm='page')        # optional - graph_genus
1
sage: K33.genus(algorithm='multi_genus') # optional - graph_genus
1
```

```
>>> from sage.all import *
>>> g = graphs.PetersenGraph()
>>> g.genus() # tests for minimal genus by default
1
>>> g.genus(on_embedding=True, maximal=True) # on_embedding overrides minimal␣
↪and maximal arguments
1
>>> g.genus(maximal=True) # setting maximal to True overrides default␣
↪minimal=True
3
>>> g.genus(on_embedding=g.get_embedding()) # can also send a valid␣
↪combinatorial embedding dict
3
>>> (graphs.CubeGraph(Integer(3))).genus()
0
>>> K23 = graphs.CompleteBipartiteGraph(Integer(2),Integer(3))
>>> K23.genus()
0
>>> K33 = graphs.CompleteBipartiteGraph(Integer(3),Integer(3))
>>> K33.genus()
1
>>> K33.genus(algorithm='simple')
1
>>> K33.genus(algorithm='page')        # optional - graph_genus
1
```

(continues on next page)

(continued from previous page)

```
>>> K33.genus(algorithm='multi_genus') # optional - graph_genus
1
```

Using the circular argument, we can compute the minimal genus preserving a planar, ordered boundary:

```
sage: cube = graphs.CubeGraph(2)
sage: cube.genus(circular=['01','10'])
0
sage: cube.is_circular_planar()
True
sage: cube.genus(circular=['01','10'])
0
sage: cube.genus(circular=['01','10'], on_embedding=True)
Traceback (most recent call last):
...
ValueError: on_embedding is not a valid option when circular is defined
sage: cube.genus(circular=['01','10'], maximal=True)
Traceback (most recent call last):
...
NotImplementedError: cannot compute the maximal genus of a genus respecting a␣
↪boundary
```

```
>>> from sage.all import *
>>> cube = graphs.CubeGraph(Integer(2))
>>> cube.genus(circular=['01','10'])
0
>>> cube.is_circular_planar()
True
>>> cube.genus(circular=['01','10'])
0
>>> cube.genus(circular=['01','10'], on_embedding=True)
Traceback (most recent call last):
...
ValueError: on_embedding is not a valid option when circular is defined
>>> cube.genus(circular=['01','10'], maximal=True)
Traceback (most recent call last):
...
NotImplementedError: cannot compute the maximal genus of a genus respecting a␣
↪boundary
```

Note: not everything works for multigraphs, looped graphs or digraphs. But the minimal genus is ultimately computable for every connected graph – but the embedding we obtain for the simple graph can't be easily converted to an embedding of a non-simple graph. Also, the maximal genus of a multigraph does not trivially correspond to that of its simple graph:

```
sage: G = DiGraph({0: [0, 1, 1, 1], 1: [2, 2, 3, 3], 2: [1, 3, 3], 3: [0, 3]})
sage: G.genus()
Traceback (most recent call last):
...
NotImplementedError: cannot work with embeddings of non-simple graphs
sage: G.to_simple().genus()
0
```

(continues on next page)

(continued from previous page)

```
sage: G.genus(set_embedding=False)
0
sage: G.genus(maximal=True, set_embedding=False)
Traceback (most recent call last):
...
NotImplementedError: cannot compute the maximal genus of a graph with loops␣
↪or multiple edges
```

```
>>> from sage.all import *
>>> G = DiGraph({Integer(0): [Integer(0), Integer(1), Integer(1), Integer(1)],
↪ Integer(1): [Integer(2), Integer(2), Integer(3), Integer(3)], Integer(2):␣
↪[Integer(1), Integer(3), Integer(3)], Integer(3): [Integer(0), Integer(3)]})
>>> G.genus()
Traceback (most recent call last):
...
NotImplementedError: cannot work with embeddings of non-simple graphs
>>> G.to_simple().genus()
0
>>> G.genus(set_embedding=False)
0
>>> G.genus(maximal=True, set_embedding=False)
Traceback (most recent call last):
...
NotImplementedError: cannot compute the maximal genus of a graph with loops␣
↪or multiple edges
```

We break graphs with cut vertices into their blocks, which greatly speeds up computation of minimal genus. This is not implemented for maximal genus:

```
sage: G = graphs.RandomBlockGraph(10, 5)
sage: G.genus()
10
```

```
>>> from sage.all import *
>>> G = graphs.RandomBlockGraph(Integer(10), Integer(5))
>>> G.genus()
10
```

get_embedding()

Return the stored embedding or `None`.

If the stored embedding is no longer valid (because of vertex/edge additions) then the stored embedding is discarded and `None` is returned. In case some vertex/edge has been deleted, the stored embedding is updated accordingly.

EXAMPLES:

```
sage: G = graphs.PetersenGraph()
sage: G.genus(algorithm='simple')
1
sage: G.get_embedding()
{0: [1, 4, 5], 1: [0, 2, 6], 2: [1, 3, 7], 3: [2, 4, 8],
```

(continues on next page)

(continued from previous page)

```
 4: [0, 3, 9], 5: [0, 7, 8], 6: [1, 9, 8], 7: [2, 5, 9],
 8: [3, 6, 5], 9: [4, 6, 7]}
```

```
>>> from sage.all import *
>>> G = graphs.PetersenGraph()
>>> G.genus(algorithm='simple')
1
>>> G.get_embedding()
{0: [1, 4, 5], 1: [0, 2, 6], 2: [1, 3, 7], 3: [2, 4, 8],
 4: [0, 3, 9], 5: [0, 7, 8], 6: [1, 9, 8], 7: [2, 5, 9],
 8: [3, 6, 5], 9: [4, 6, 7]}
```

Note that the embeddings gets properly modified on vertex or edge deletion:

```
sage: G.delete_edge(0, 1)
sage: G.delete_vertex(3)
sage: G.get_embedding()
{0: [4, 5],
 1: [2, 6],
 2: [1, 7],
 4: [0, 9],
 5: [0, 7, 8],
 6: [1, 9, 8],
 7: [2, 5, 9],
 8: [6, 5],
 9: [4, 6, 7]}
```

```
>>> from sage.all import *
>>> G.delete_edge(Integer(0), Integer(1))
>>> G.delete_vertex(Integer(3))
>>> G.get_embedding()
{0: [4, 5],
 1: [2, 6],
 2: [1, 7],
 4: [0, 9],
 5: [0, 7, 8],
 6: [1, 9, 8],
 7: [2, 5, 9],
 8: [6, 5],
 9: [4, 6, 7]}
```

But not under edge addition:

```
sage: G.add_edge(0, 7)
sage: G.get_embedding() is None
True
```

```
>>> from sage.all import *
>>> G.add_edge(Integer(0), Integer(7))
>>> G.get_embedding() is None
True
```

get_pos(*dim=2*)

Return the position dictionary.

The position dictionary specifies the coordinates of each vertex.

INPUT:

- `dim` – integer (default: 2); whether to return the position dictionary in the plane (`dim == 2`) or in the 3-dimensional space

EXAMPLES:

By default, the position of a graph is None:

```
sage: G = Graph()
sage: G.get_pos()
sage: G.get_pos() is None
True
sage: P = G.plot(save_pos=True)                                                #␣
↪needs sage.plot
sage: G.get_pos()                                                              #␣
↪needs sage.plot
{}
```

```
>>> from sage.all import *
>>> G = Graph()
>>> G.get_pos()
>>> G.get_pos() is None
True
>>> P = G.plot(save_pos=True)                                                  #␣
↪needs sage.plot
>>> G.get_pos()                                                               #␣
↪needs sage.plot
{}
```

Some of the named graphs come with a pre-specified positioning:

```
sage: G = graphs.PetersenGraph()
sage: G.get_pos()
{0: (0.0, 1.0),
 ...
 9: (0.475..., 0.154...)}
```

```
>>> from sage.all import *
>>> G = graphs.PetersenGraph()
>>> G.get_pos()
{0: (0.0, 1.0),
 ...
 9: (0.475..., 0.154...)}
```

Note that the position dictionary is modified on vertex removal:

```
sage: G.delete_vertex(0)
sage: G.get_pos()
{1: (-0.951..., 0.309...),
```

(continues on next page)

(continued from previous page)

```
...
 9: (0.475..., 0.154...)}
```

```
>>> from sage.all import *
>>> G.delete_vertex(Integer(0))
>>> G.get_pos()
{1: (-0.951..., 0.309...),
...
 9: (0.475..., 0.154...)}
```

But is deleted on vertex addition:

```
sage: G.add_vertex(0)
sage: G.get_pos() is None
True
```

```
>>> from sage.all import *
>>> G.add_vertex(Integer(0))
>>> G.get_pos() is None
True
```

get_vertex(*vertex*)

Retrieve the object associated with a given vertex.

If no associated object is found, `None` is returned.

INPUT:

- `vertex` – the given vertex

EXAMPLES:

```
sage: d = {0: graphs.DodecahedralGraph(), 1: graphs.FlowerSnark(), 2: graphs.
↪MoebiusKantorGraph(), 3: graphs.PetersenGraph()}
sage: d[2]
Moebius-Kantor Graph: Graph on 16 vertices
sage: T = graphs.TetrahedralGraph()
sage: T.vertices(sort=True)
[0, 1, 2, 3]
sage: T.set_vertices(d)
sage: T.get_vertex(1)
Flower Snark: Graph on 20 vertices
```

```
>>> from sage.all import *
>>> d = {Integer(0): graphs.DodecahedralGraph(), Integer(1): graphs.
↪FlowerSnark(), Integer(2): graphs.MoebiusKantorGraph(), Integer(3): graphs.
↪PetersenGraph()}
>>> d[Integer(2)]
Moebius-Kantor Graph: Graph on 16 vertices
>>> T = graphs.TetrahedralGraph()
>>> T.vertices(sort=True)
[0, 1, 2, 3]
>>> T.set_vertices(d)
```

(continues on next page)

(continued from previous page)

```
>>> T.get_vertex(Integer(1))
Flower Snark: Graph on 20 vertices
```

get_vertices(*verts=None*)

Return a dictionary of the objects associated to each vertex.

INPUT:

- `verts` – iterable container of vertices

EXAMPLES:

```
sage: d = {0: graphs.DodecahedralGraph(), 1: graphs.FlowerSnark(), 2: graphs.
↪MoebiusKantorGraph(), 3: graphs.PetersenGraph()}
sage: T = graphs.TetrahedralGraph()
sage: T.set_vertices(d)
sage: T.get_vertices([1, 2])
{1: Flower Snark: Graph on 20 vertices,
 2: Moebius-Kantor Graph: Graph on 16 vertices}
```

```
>>> from sage.all import *
>>> d = {Integer(0): graphs.DodecahedralGraph(), Integer(1): graphs.
↪FlowerSnark(), Integer(2): graphs.MoebiusKantorGraph(), Integer(3): graphs.
↪PetersenGraph()}
>>> T = graphs.TetrahedralGraph()
>>> T.set_vertices(d)
>>> T.get_vertices([Integer(1), Integer(2)])
{1: Flower Snark: Graph on 20 vertices,
 2: Moebius-Kantor Graph: Graph on 16 vertices}
```

girth(*certificate=False*)

Return the girth of the graph.

The girth is the length of the shortest cycle in the graph (directed cycle if the graph is directed). Graphs without (directed) cycles have infinite girth.

INPUT:

- `certificate` – boolean (default: `False`); whether to return `(g, c)`, where `g` is the girth and `c` is a list of vertices of a (directed) cycle of length `g` in the graph, thus providing a certificate that the girth is at most `g`, or `None` if `g` infinite

EXAMPLES:

```
sage: graphs.TetrahedralGraph().girth()
3
sage: graphs.CubeGraph(3).girth()
4
sage: graphs.PetersenGraph().girth(certificate=True)  # random
(5, [4, 3, 2, 1, 0])
sage: graphs.HeawoodGraph().girth()
6
sage: next(graphs.trees(9)).girth()
+Infinity
```

```
>>> from sage.all import *
>>> graphs.TetrahedralGraph().girth()
3
>>> graphs.CubeGraph(Integer(3)).girth()
4
>>> graphs.PetersenGraph().girth(certificate=True)  # random
(5, [4, 3, 2, 1, 0])
>>> graphs.HeawoodGraph().girth()
6
>>> next(graphs.trees(Integer(9))).girth()
+Infinity
```

See also

- *odd_girth()* – return the odd girth of the graph.

graphics_array_defaults = {'graph_border': True, 'layout': 'circular', 'vertex_labels': False, 'vertex_size': 50}

graphplot(***options*)

Return a *GraphPlot* object.

See *GraphPlot* for more details.

INPUT:

- **options – parameters for the *GraphPlot* constructor

EXAMPLES:

Creating a *GraphPlot* object uses the same options as *plot()*:

```
sage: g = Graph({}, loops=True, multiedges=True, sparse=True)
sage: g.add_edges([(0,0,'a'),(0,0,'b'),(0,1,'c'),(0,1,'d'),
....:     (0,1,'e'),(0,1,'f'),(0,1,'f'),(2,1,'g'),(2,2,'h')])
sage: GP = g.graphplot(edge_labels=True, color_by_label=True,
....:                  edge_style='dashed')
sage: GP.plot()
Graphics object consisting of 22 graphics primitives
```

```
>>> from sage.all import *
>>> g = Graph({}, loops=True, multiedges=True, sparse=True)
>>> g.add_edges([(Integer(0),Integer(0),'a'),(Integer(0),Integer(0),'b'),
↪(Integer(0),Integer(1),'c'),(Integer(0),Integer(1),'d'),
...     (Integer(0),Integer(1),'e'),(Integer(0),Integer(1),'f'),(Integer(0),
↪Integer(1),'f'),(Integer(2),Integer(1),'g'),(Integer(2),Integer(2),'h')])
>>> GP = g.graphplot(edge_labels=True, color_by_label=True,
...                  edge_style='dashed')
>>> GP.plot()
Graphics object consisting of 22 graphics primitives
```

We can modify the *GraphPlot* object. Notice that the changes are cumulative:

```
sage: GP.set_edges(edge_style='solid')
sage: GP.plot()
Graphics object consisting of 22 graphics primitives
sage: GP.set_vertices(talk=True)
sage: GP.plot()
Graphics object consisting of 22 graphics primitives
```

```
>>> from sage.all import *
>>> GP.set_edges(edge_style='solid')
>>> GP.plot()
Graphics object consisting of 22 graphics primitives
>>> GP.set_vertices(talk=True)
>>> GP.plot()
Graphics object consisting of 22 graphics primitives
```

graphviz_string(*labels='string'*, *vertex_labels=True*, *edge_labels=False*, *edge_color=None*, *edge_colors=None*, *edge_options=()*, *color_by_label=False*, *rankdir='down'*, *subgraph_clusters=[]*, ***options*)

Return a representation in the `dot` language.

The `dot` language is a text based format for graphs. It is used by the software suite `graphviz`. The specifications of the language are available on the web (see the reference [dotspec]).

INPUT:

- `labels` – string (default: `'string'`); either `'string'` or `'latex'`. If labels is `'string'`, latex commands are not interpreted. This option stands for both vertex labels and edge labels.
- `vertex_labels` – boolean (default: `True`); whether to add the labels on vertices
- `edge_labels` – boolean (default: `False`); whether to add the labels on edges
- `edge_color` – (default: `None`) specify a default color for the edges. The color could be one of
  - a name given as a string such as `'blue'` or `'orchid'`
  - a HSV sequence in a string such as `'.52,.386,.22'`
  - a hexadecimal code such as `'#DA3305'`
  - a 3-tuple of floating point (to be interpreted as RGB tuple). In this case the 3-tuple is converted in hexadecimal code.
- `edge_colors` – dictionary (default: `None`); a dictionary whose keys are colors and values are list of edges. The list of edges need not to be complete in which case the default color is used. See the option `edge_color` for a description of valid color formats.
- `color_by_label` – boolean or dictionary or function (default: `False`); whether to color each edge with a different color according to its label; the colors are chosen along a rainbow, unless they are specified by a function or dictionary mapping labels to colors; this option is incompatible with `edge_color` and `edge_colors`. See the option `edge_color` for a description of valid color formats.
- `edge_options` – a function (or tuple thereof) mapping edges to a dictionary of options for this edge
- `rankdir` – `'left'`, `'right'`, `'up'`, or `'down'` (default: `'down'`, for consistency with `graphviz`); the preferred ranking direction for acyclic layouts; see the `rankdir` option of `graphviz`.
- `subgraph_clusters` – list of lists of vertices (default: `[]`); From [dotspec]: "If supported, the layout engine will do the layout so that the nodes belonging to the cluster are drawn together, with the entire drawing of the cluster contained within a bounding rectangle. Note that, for good and bad, cluster

subgraphs are not part of the `dot` language, but solely a syntactic convention adhered to by certain of the layout engines."

EXAMPLES:

```
sage: G = Graph({0: {1: None, 2: None}, 1: {0: None, 2: None},
....:            2: {0: None, 1: None, 3: 'foo'}, 3: {2: 'foo'}},
....:           sparse=True)
sage: print(G.graphviz_string(edge_labels=True))
graph {
  node_0  [label="0"];
  node_1  [label="1"];
  node_2  [label="2"];
  node_3  [label="3"];

  node_0 -- node_1;
  node_0 -- node_2;
  node_1 -- node_2;
  node_2 -- node_3 [label="foo"];
}
```

```
>>> from sage.all import *
>>> G = Graph({Integer(0): {Integer(1): None, Integer(2): None}, Integer(1):
↪{Integer(0): None, Integer(2): None},
...            Integer(2): {Integer(0): None, Integer(1): None, Integer(3):
↪'foo'}, Integer(3): {Integer(2): 'foo'}},
...           sparse=True)
>>> print(G.graphviz_string(edge_labels=True))
graph {
  node_0  [label="0"];
  node_1  [label="1"];
  node_2  [label="2"];
  node_3  [label="3"];
<BLANKLINE>
  node_0 -- node_1;
  node_0 -- node_2;
  node_1 -- node_2;
  node_2 -- node_3 [label="foo"];
}
```

A variant, with the labels in latex, for post-processing with `dot2tex`:

```
sage: print(G.graphviz_string(edge_labels=True, labels='latex'))
graph {
  node [shape="plaintext"];
  node_0  [label=" ", texlbl="$0$"];
  node_1  [label=" ", texlbl="$1$"];
  node_2  [label=" ", texlbl="$2$"];
  node_3  [label=" ", texlbl="$3$"];

  node_0 -- node_1;
  node_0 -- node_2;
  node_1 -- node_2;
  node_2 -- node_3 [label=" ", texlbl="$\text{\texttt{foo}}$"];
```

(continues on next page)

(continued from previous page)

```
}
```

```
>>> from sage.all import *
>>> print(G.graphviz_string(edge_labels=True, labels='latex'))
graph {
  node [shape="plaintext"];
  node_0  [label=" ", texlbl="$0$"];
  node_1  [label=" ", texlbl="$1$"];
  node_2  [label=" ", texlbl="$2$"];
  node_3  [label=" ", texlbl="$3$"];
<BLANKLINE>
  node_0 -- node_1;
  node_0 -- node_2;
  node_1 -- node_2;
  node_2 -- node_3 [label=" ", texlbl="$\text{\texttt{foo}}$"];
}
```

Same, with a digraph and a color for edges:

```
sage: G = DiGraph({0: {1: None, 2: None}, 1: {2: None}, 2: {3: 'foo'}, 3: {}},
....:             sparse=True)
sage: print(G.graphviz_string(edge_color='red'))
digraph {
  node_0  [label="0"];
  node_1  [label="1"];
  node_2  [label="2"];
  node_3  [label="3"];

edge [color="red"];
  node_0 -> node_1;
  node_0 -> node_2;
  node_1 -> node_2;
  node_2 -> node_3;
}
```

```
>>> from sage.all import *
>>> G = DiGraph({Integer(0): {Integer(1): None, Integer(2): None},␣
↪Integer(1): {Integer(2): None}, Integer(2): {Integer(3): 'foo'},␣
↪Integer(3): {}},
...              sparse=True)
>>> print(G.graphviz_string(edge_color='red'))
digraph {
  node_0  [label="0"];
  node_1  [label="1"];
  node_2  [label="2"];
  node_3  [label="3"];
<BLANKLINE>
edge [color="red"];
  node_0 -> node_1;
  node_0 -> node_2;
  node_1 -> node_2;
  node_2 -> node_3;
```

(continues on next page)

(continued from previous page)

```
}
```

A digraph using latex labels for vertices and edges:

```
sage: f(x) = -1 / x
sage: g(x) = 1 / (x + 1)
sage: G = DiGraph()
sage: G.add_edges((i, f(i), f) for i in (1, 2, 1/2, 1/4))
sage: G.add_edges((i, g(i), g) for i in (1, 2, 1/2, 1/4))
sage: print(G.graphviz_string(labels='latex',                # random
....:                         edge_labels=True))
digraph {
  node [shape="plaintext"];
  node_10  [label=" ", texlbl="$1$"];
  node_11  [label=" ", texlbl="$2$"];
  node_3  [label=" ", texlbl="$-\frac{1}{2}$"];
  node_6  [label=" ", texlbl="$\frac{1}{2}$"];
  node_7  [label=" ", texlbl="$\frac{1}{2}$"];
  node_5  [label=" ", texlbl="$\frac{1}{3}$"];
  node_8  [label=" ", texlbl="$\frac{2}{3}$"];
  node_4  [label=" ", texlbl="$\frac{1}{4}$"];
  node_1  [label=" ", texlbl="$-2$"];
  node_9  [label=" ", texlbl="$\frac{4}{5}$"];
  node_0  [label=" ", texlbl="$-4$"];
  node_2  [label=" ", texlbl="$-1$"];

  node_10 -> node_2 [label=" ", texlbl="$x \ {\mapsto}\ -\frac{1}{x}$"];
  node_10 -> node_6 [label=" ", texlbl="$x \ {\mapsto}\ \frac{1}{x + 1}$"];
  node_11 -> node_3 [label=" ", texlbl="$x \ {\mapsto}\ -\frac{1}{x}$"];
  node_11 -> node_5 [label=" ", texlbl="$x \ {\mapsto}\ \frac{1}{x + 1}$"];
  node_7 -> node_1 [label=" ", texlbl="$x \ {\mapsto}\ -\frac{1}{x}$"];
  node_7 -> node_8 [label=" ", texlbl="$x \ {\mapsto}\ \frac{1}{x + 1}$"];
  node_4 -> node_0 [label=" ", texlbl="$x \ {\mapsto}\ -\frac{1}{x}$"];
  node_4 -> node_9 [label=" ", texlbl="$x \ {\mapsto}\ \frac{1}{x + 1}$"];
}

sage: print(G.graphviz_string(labels='latex',                # random        #␣
↪needs sage.symbolic
....:                         color_by_label=True))
digraph {
  node [shape="plaintext"];
  node_10  [label=" ", texlbl="$1$"];
  node_11  [label=" ", texlbl="$2$"];
  node_3  [label=" ", texlbl="$-\frac{1}{2}$"];
  node_6  [label=" ", texlbl="$\frac{1}{2}$"];
  node_7  [label=" ", texlbl="$\frac{1}{2}$"];
  node_5  [label=" ", texlbl="$\frac{1}{3}$"];
  node_8  [label=" ", texlbl="$\frac{2}{3}$"];
  node_4  [label=" ", texlbl="$\frac{1}{4}$"];
  node_1  [label=" ", texlbl="$-2$"];
  node_9  [label=" ", texlbl="$\frac{4}{5}$"];
  node_0  [label=" ", texlbl="$-4$"];
```

(continues on next page)

(continued from previous page)

```
  node_2  [label=" ", texlbl="$-1$"];

  node_10 -> node_2 [color = "#ff0000"];
  node_10 -> node_6 [color = "#00ffff"];
  node_11 -> node_3 [color = "#ff0000"];
  node_11 -> node_5 [color = "#00ffff"];
  node_7 -> node_1 [color = "#ff0000"];
  node_7 -> node_8 [color = "#00ffff"];
  node_4 -> node_0 [color = "#ff0000"];
  node_4 -> node_9 [color = "#00ffff"];
}

sage: print(G.graphviz_string(labels='latex',                # random        #␣
↪needs sage.symbolic
....:                           color_by_label={f: "red", g: "blue"}))
digraph {
  node [shape="plaintext"];
  node_10  [label=" ", texlbl="$1$"];
  node_11  [label=" ", texlbl="$2$"];
  node_3  [label=" ", texlbl="$-\frac{1}{2}$"];
  node_6  [label=" ", texlbl="$\frac{1}{2}$"];
  node_7  [label=" ", texlbl="$\frac{1}{2}$"];
  node_5  [label=" ", texlbl="$\frac{1}{3}$"];
  node_8  [label=" ", texlbl="$\frac{2}{3}$"];
  node_4  [label=" ", texlbl="$\frac{1}{4}$"];
  node_1  [label=" ", texlbl="$-2$"];
  node_9  [label=" ", texlbl="$\frac{4}{5}$"];
  node_0  [label=" ", texlbl="$-4$"];
  node_2  [label=" ", texlbl="$-1$"];

  node_10 -> node_2 [color = "red"];
  node_10 -> node_6 [color = "blue"];
  node_11 -> node_3 [color = "red"];
  node_11 -> node_5 [color = "blue"];
  node_7 -> node_1 [color = "red"];
  node_7 -> node_8 [color = "blue"];
  node_4 -> node_0 [color = "red"];
  node_4 -> node_9 [color = "blue"];
}
```

```
>>> from sage.all import *
>>> __tmp__=var("x"); f = symbolic_expression(-Integer(1) / x).function(x)
>>> __tmp__=var("x"); g = symbolic_expression(Integer(1) / (x + Integer(1))).
↪function(x)
>>> G = DiGraph()
>>> G.add_edges((i, f(i), f) for i in (Integer(1), Integer(2), Integer(1)/
↪Integer(2), Integer(1)/Integer(4)))
>>> G.add_edges((i, g(i), g) for i in (Integer(1), Integer(2), Integer(1)/
↪Integer(2), Integer(1)/Integer(4)))
>>> print(G.graphviz_string(labels='latex',                # random
...                         edge_labels=True))
digraph {
```

(continues on next page)

(continued from previous page)

```
  node [shape="plaintext"];
  node_10  [label=" ", texlbl="$1$"];
  node_11  [label=" ", texlbl="$2$"];
  node_3  [label=" ", texlbl="$-\frac{1}{2}$"];
  node_6  [label=" ", texlbl="$\frac{1}{2}$"];
  node_7  [label=" ", texlbl="$\frac{1}{2}$"];
  node_5  [label=" ", texlbl="$\frac{1}{3}$"];
  node_8  [label=" ", texlbl="$\frac{2}{3}$"];
  node_4  [label=" ", texlbl="$\frac{1}{4}$"];
  node_1  [label=" ", texlbl="$-2$"];
  node_9  [label=" ", texlbl="$\frac{4}{5}$"];
  node_0  [label=" ", texlbl="$-4$"];
  node_2  [label=" ", texlbl="$-1$"];
<BLANKLINE>
  node_10 -> node_2 [label=" ", texlbl="$x \ {\mapsto}\ -\frac{1}{x}$"];
  node_10 -> node_6 [label=" ", texlbl="$x \ {\mapsto}\ \frac{1}{x + 1}$"];
  node_11 -> node_3 [label=" ", texlbl="$x \ {\mapsto}\ -\frac{1}{x}$"];
  node_11 -> node_5 [label=" ", texlbl="$x \ {\mapsto}\ \frac{1}{x + 1}$"];
  node_7 -> node_1 [label=" ", texlbl="$x \ {\mapsto}\ -\frac{1}{x}$"];
  node_7 -> node_8 [label=" ", texlbl="$x \ {\mapsto}\ \frac{1}{x + 1}$"];
  node_4 -> node_0 [label=" ", texlbl="$x \ {\mapsto}\ -\frac{1}{x}$"];
  node_4 -> node_9 [label=" ", texlbl="$x \ {\mapsto}\ \frac{1}{x + 1}$"];
}

>>> print(G.graphviz_string(labels='latex',               # random        #␣
↪needs sage.symbolic
...                          color_by_label=True))
digraph {
  node [shape="plaintext"];
  node_10  [label=" ", texlbl="$1$"];
  node_11  [label=" ", texlbl="$2$"];
  node_3  [label=" ", texlbl="$-\frac{1}{2}$"];
  node_6  [label=" ", texlbl="$\frac{1}{2}$"];
  node_7  [label=" ", texlbl="$\frac{1}{2}$"];
  node_5  [label=" ", texlbl="$\frac{1}{3}$"];
  node_8  [label=" ", texlbl="$\frac{2}{3}$"];
  node_4  [label=" ", texlbl="$\frac{1}{4}$"];
  node_1  [label=" ", texlbl="$-2$"];
  node_9  [label=" ", texlbl="$\frac{4}{5}$"];
  node_0  [label=" ", texlbl="$-4$"];
  node_2  [label=" ", texlbl="$-1$"];
<BLANKLINE>
  node_10 -> node_2 [color = "#ff0000"];
  node_10 -> node_6 [color = "#00ffff"];
  node_11 -> node_3 [color = "#ff0000"];
  node_11 -> node_5 [color = "#00ffff"];
  node_7 -> node_1 [color = "#ff0000"];
  node_7 -> node_8 [color = "#00ffff"];
  node_4 -> node_0 [color = "#ff0000"];
  node_4 -> node_9 [color = "#00ffff"];
}
```

(continues on next page)

(continued from previous page)

```
>>> print(G.graphviz_string(labels='latex',                # random        #␣
↪needs sage.symbolic
...                          color_by_label={f: "red", g: "blue"}))
digraph {
  node [shape="plaintext"];
  node_10  [label=" ", texlbl="$1$"];
  node_11  [label=" ", texlbl="$2$"];
  node_3  [label=" ", texlbl="$-\frac{1}{2}$"];
  node_6  [label=" ", texlbl="$\frac{1}{2}$"];
  node_7  [label=" ", texlbl="$\frac{1}{2}$"];
  node_5  [label=" ", texlbl="$\frac{1}{3}$"];
  node_8  [label=" ", texlbl="$\frac{2}{3}$"];
  node_4  [label=" ", texlbl="$\frac{1}{4}$"];
  node_1  [label=" ", texlbl="$-2$"];
  node_9  [label=" ", texlbl="$\frac{4}{5}$"];
  node_0  [label=" ", texlbl="$-4$"];
  node_2  [label=" ", texlbl="$-1$"];
<BLANKLINE>
  node_10 -> node_2 [color = "red"];
  node_10 -> node_6 [color = "blue"];
  node_11 -> node_3 [color = "red"];
  node_11 -> node_5 [color = "blue"];
  node_7 -> node_1 [color = "red"];
  node_7 -> node_8 [color = "blue"];
  node_4 -> node_0 [color = "red"];
  node_4 -> node_9 [color = "blue"];
}
```

By default `graphviz` renders digraphs using a hierarchical layout, ranking the vertices down from top to bottom. Here we specify alternative ranking directions for this layout:

```
sage: D = DiGraph([(1, 2)])
sage: print(D.graphviz_string(rankdir='up'))
digraph {
  rankdir=BT
  node_0  [label="1"];
  node_1  [label="2"];

  node_0 -> node_1;
}
sage: print(D.graphviz_string(rankdir='down'))
digraph {
  node_0  [label="1"];
  node_1  [label="2"];

  node_0 -> node_1;
}
sage: print(D.graphviz_string(rankdir='left'))
digraph {
  rankdir=RL
  node_0  [label="1"];
  node_1  [label="2"];
```

(continues on next page)

(continued from previous page)

```
  node_0 -> node_1;
}
sage: print(D.graphviz_string(rankdir='right'))
digraph {
  rankdir=LR
  node_0  [label="1"];
  node_1  [label="2"];

  node_0 -> node_1;
}
```

```
>>> from sage.all import *
>>> D = DiGraph([(Integer(1), Integer(2))])
>>> print(D.graphviz_string(rankdir='up'))
digraph {
  rankdir=BT
  node_0  [label="1"];
  node_1  [label="2"];
<BLANKLINE>
  node_0 -> node_1;
}
>>> print(D.graphviz_string(rankdir='down'))
digraph {
  node_0  [label="1"];
  node_1  [label="2"];
<BLANKLINE>
  node_0 -> node_1;
}
>>> print(D.graphviz_string(rankdir='left'))
digraph {
  rankdir=RL
  node_0  [label="1"];
  node_1  [label="2"];
<BLANKLINE>
  node_0 -> node_1;
}
>>> print(D.graphviz_string(rankdir='right'))
digraph {
  rankdir=LR
  node_0  [label="1"];
  node_1  [label="2"];
<BLANKLINE>
  node_0 -> node_1;
}
```

Edge-specific options can also be specified by providing a function (or tuple thereof) which maps each edge to a dictionary of options. Valid options are

- `'color'`
- `'dot'` (a string containing a sequence of options in `dot` format)
- `'label'` (a string)

- `'label_style'` (`'string'` or `'latex'`)
- `'edge_string'` (`'--'` or `'->'`)
- `'dir'` (`'forward'`, `'back'`, `'both'` or `'none'`)
- `'backward'` (boolean), instead of defining the edge in the graphviz string as `u -> v` it draws it as `v -> u [dir=back]` and instead of `u -> v [dir=back]` it draws it as `v -> u`, this changes the way it is drawn by Graphviz's dot program: vertex `v` will be *above* vertex `u` instead of below.

Here we state that the graph should be laid out so that edges starting from `1` are going backward (e.g. going up instead of down):

```
sage: def edge_options(data):
....:     u, v, label = data
....:     return {"dir":"back"} if u == 1 else {}
sage: print(G.graphviz_string(edge_options=edge_options))    # random        #␣
↪needs sage.symbolic
digraph {
  node_0  [label="-1"];
  node_1  [label="-1/2"];
  node_2  [label="1/2"];
  node_3  [label="-2"];
  node_4  [label="1/4"];
  node_5  [label="-4"];
  node_6  [label="1/3"];
  node_7  [label="2/3"];
  node_8  [label="4/5"];
  node_9  [label="1"];
  node_10  [label="2"];

  node_2 -> node_3;
  node_2 -> node_7;
  node_4 -> node_5;
  node_4 -> node_8;
  node_9 -> node_0 [dir=back];
  node_9 -> node_2 [dir=back];
  node_10 -> node_1;
  node_10 -> node_6;
}
```

```
>>> from sage.all import *
>>> def edge_options(data):
...     u, v, label = data
...     return {"dir":"back"} if u == Integer(1) else {}
>>> print(G.graphviz_string(edge_options=edge_options))    # random        #␣
↪needs sage.symbolic
digraph {
  node_0  [label="-1"];
  node_1  [label="-1/2"];
  node_2  [label="1/2"];
  node_3  [label="-2"];
  node_4  [label="1/4"];
  node_5  [label="-4"];
  node_6  [label="1/3"];
```

(continues on next page)

(continued from previous page)

```
  node_7  [label="2/3"];
  node_8  [label="4/5"];
  node_9  [label="1"];
  node_10  [label="2"];
<BLANKLINE>
  node_2 -> node_3;
  node_2 -> node_7;
  node_4 -> node_5;
  node_4 -> node_8;
  node_9 -> node_0 [dir=back];
  node_9 -> node_2 [dir=back];
  node_10 -> node_1;
  node_10 -> node_6;
}
```

We now test all options:

```
sage: def edge_options(data):
....:     u, v, label = data
....:     options = {"color": {f: "red", g: "blue"}[label]}
....:     if (u,v) == (1/2, -2): options["label"]       = "coucou"; options[
↪"label_style"] = "string"
....:     if (u,v) == (1/2,2/3): options["dot"]         = "x=1,y=2"
....:     if (u,v) == (1,   -1): options["label_style"] = "latex"
....:     if (u,v) == (1,  1/2): options["dir"]         = "back"
....:     return options
sage: print(G.graphviz_string(edge_options=edge_options))    # random        #␣
↪needs sage.symbolic
digraph {
  node_0  [label="-1"];
  node_1  [label="-1/2"];
  node_2  [label="1/2"];
  node_3  [label="-2"];
  node_4  [label="1/4"];
  node_5  [label="-4"];
  node_6  [label="1/3"];
  node_7  [label="2/3"];
  node_8  [label="4/5"];
  node_9  [label="1"];
  node_10  [label="2"];

  node_2 -> node_3 [label='coucou', color = "red"];
  node_2 -> node_7 [x=1,y=2, color = "blue"];
  node_4 -> node_5 [color = "red"];
  node_4 -> node_8 [color = "blue"];
  node_9 -> node_0 [label=" ", texlbl="$x \ {\mapsto}\ -\frac{1}{x}$", color␣
↪= "red"];
  node_9 -> node_2 [color = "blue", dir=back];
  node_10 -> node_1 [color = "red"];
  node_10 -> node_6 [color = "blue"];
}
```

```
>>> from sage.all import *
>>> def edge_options(data):
...     u, v, label = data
...     options = {"color": {f: "red", g: "blue"}[label]}
...     if (u,v) == (Integer(1)/Integer(2), -Integer(2)): options["label"]
↪   = "coucou"; options["label_style"] = "string"
...     if (u,v) == (Integer(1)/Integer(2),Integer(2)/Integer(3)): options[
↪"dot"]         = "x=1,y=2"
...     if (u,v) == (Integer(1),   -Integer(1)): options["label_style"] =
↪"latex"
...     if (u,v) == (Integer(1),  Integer(1)/Integer(2)): options["dir"]
↪   = "back"
...     return options
>>> print(G.graphviz_string(edge_options=edge_options))   # random        #
↪needs sage.symbolic
digraph {
  node_0  [label="-1"];
  node_1  [label="-1/2"];
  node_2  [label="1/2"];
  node_3  [label="-2"];
  node_4  [label="1/4"];
  node_5  [label="-4"];
  node_6  [label="1/3"];
  node_7  [label="2/3"];
  node_8  [label="4/5"];
  node_9  [label="1"];
  node_10  [label="2"];
<BLANKLINE>
  node_2 -> node_3 [label='coucou', color = "red"];
  node_2 -> node_7 [x=1,y=2, color = "blue"];
  node_4 -> node_5 [color = "red"];
  node_4 -> node_8 [color = "blue"];
  node_9 -> node_0 [label=" ", texlbl="$x \ {\mapsto}\ -\frac{1}{x}$", color
↪= "red"];
  node_9 -> node_2 [color = "blue", dir=back];
  node_10 -> node_1 [color = "red"];
  node_10 -> node_6 [color = "blue"];
}
```

We test the possible values of the `'dir'` edge option:

```
sage: edges = [(0,1,'a'), (1,2,'b'), (2,3,'c'), (3,4,'d')]
sage: G = DiGraph(edges)
sage: def edge_options(data):
....:     u,v,label = data
....:     if label == 'a': return {'dir':'forward'}
....:     if label == 'b': return {'dir':'back'}
....:     if label == 'c': return {'dir':'none'}
....:     if label == 'd': return {'dir':'both'}
sage: print(G.graphviz_string(edge_options=edge_options))
digraph {
  node_0  [label="0"];
  node_1  [label="1"];
```

(continues on next page)

(continued from previous page)

```
  node_2  [label="2"];
  node_3  [label="3"];
  node_4  [label="4"];

  node_0 -> node_1;
  node_1 -> node_2 [dir=back];
  node_2 -> node_3 [dir=none];
  node_3 -> node_4 [dir=both];
}
```

```
>>> from sage.all import *
>>> edges = [(Integer(0),Integer(1),'a'), (Integer(1),Integer(2),'b'),␣
↪(Integer(2),Integer(3),'c'), (Integer(3),Integer(4),'d')]
>>> G = DiGraph(edges)
>>> def edge_options(data):
...     u,v,label = data
...     if label == 'a': return {'dir':'forward'}
...     if label == 'b': return {'dir':'back'}
...     if label == 'c': return {'dir':'none'}
...     if label == 'd': return {'dir':'both'}
>>> print(G.graphviz_string(edge_options=edge_options))
digraph {
  node_0  [label="0"];
  node_1  [label="1"];
  node_2  [label="2"];
  node_3  [label="3"];
  node_4  [label="4"];
<BLANKLINE>
  node_0 -> node_1;
  node_1 -> node_2 [dir=back];
  node_2 -> node_3 [dir=none];
  node_3 -> node_4 [dir=both];
}
```

We test the same graph and `'dir'` edge options but with `backward=True`, which reverses the natural direction each edge wants to be pointing for the layout:

```
sage: def edge_options(data):
....:     u,v,label = data
....:     if label == 'a': return {'dir':'forward', 'backward':True}
....:     if label == 'b': return {'dir':'back', 'backward':True}
....:     if label == 'c': return {'dir':'none', 'backward':True}
....:     if label == 'd': return {'dir':'both', 'backward':True}
sage: print(G.graphviz_string(edge_options=edge_options))
digraph {
  node_0  [label="0"];
  node_1  [label="1"];
  node_2  [label="2"];
  node_3  [label="3"];
  node_4  [label="4"];

  node_1 -> node_0 [dir=back];
```

(continues on next page)

(continued from previous page)

```
  node_2 -> node_1;
  node_3 -> node_2 [dir=none];
  node_4 -> node_3 [dir=both];
}
```

```
>>> from sage.all import *
>>> def edge_options(data):
...     u,v,label = data
...     if label == 'a': return {'dir':'forward', 'backward':True}
...     if label == 'b': return {'dir':'back', 'backward':True}
...     if label == 'c': return {'dir':'none', 'backward':True}
...     if label == 'd': return {'dir':'both', 'backward':True}
>>> print(G.graphviz_string(edge_options=edge_options))
digraph {
  node_0  [label="0"];
  node_1  [label="1"];
  node_2  [label="2"];
  node_3  [label="3"];
  node_4  [label="4"];
<BLANKLINE>
  node_1 -> node_0 [dir=back];
  node_2 -> node_1;
  node_3 -> node_2 [dir=none];
  node_4 -> node_3 [dir=both];
}
```

graphviz_to_file_named(*filename*, ***options*)

Write a representation in the `dot` language in a file.

The `dot` language is a plaintext format for graph structures. See the documentation of *graphviz_string()* for available options.

INPUT:

- `filename` – the name of the file to write in
- `**options` – options for the graphviz string

EXAMPLES:

```
sage: G = Graph({0: {1: None, 2: None}, 1: {0: None, 2: None},
....:            2: {0: None, 1: None, 3: 'foo'}, 3: {2: 'foo'}},
....:           sparse=True)
sage: import tempfile
sage: with tempfile.NamedTemporaryFile(mode='a+t', delete_on_close=False) as␣
↪f:
....:     G.graphviz_to_file_named(f.name, edge_labels=True)
....:     print(f.read())
graph {
  node_0  [label="0"];
  node_1  [label="1"];
  node_2  [label="2"];
  node_3  [label="3"];
```

(continues on next page)

(continued from previous page)

```
  node_0 -- node_1;
  node_0 -- node_2;
  node_1 -- node_2;
  node_2 -- node_3 [label="foo"];
}
```

```
>>> from sage.all import *
>>> G = Graph({Integer(0): {Integer(1): None, Integer(2): None}, Integer(1):
↪{Integer(0): None, Integer(2): None},
...            Integer(2): {Integer(0): None, Integer(1): None, Integer(3):
↪'foo'}, Integer(3): {Integer(2): 'foo'}},
...           sparse=True)
>>> import tempfile
>>> with tempfile.NamedTemporaryFile(mode='a+t', delete_on_close=False) as f:
...     G.graphviz_to_file_named(f.name, edge_labels=True)
...     print(f.read())
graph {
  node_0  [label="0"];
  node_1  [label="1"];
  node_2  [label="2"];
  node_3  [label="3"];
<BLANKLINE>
  node_0 -- node_1;
  node_0 -- node_2;
  node_1 -- node_2;
  node_2 -- node_3 [label="foo"];
}
```

greedy_dominating_set(*G*, *k=1*, *vertices=None*, *ordering=None*, *return_sets=False*, *closest=False*)

Return a greedy distance-k dominating set of the graph.

A distance-k dominating set S of a graph G is a set of its vertices of minimal cardinality such that any vertex of G is in S or is at distance at most k from a vertex in S. See the Wikipedia article Dominating_set.

When G is directed, vertex u can be a dominator of vertex v if there is a directed path of length at most k from u to v.

This method implements a greedy heuristic to find a minimal dominatic set.

INPUT:

- `G` – a Graph
- `k` – integer (default: 1); the domination distance to consider
- `vertices` – iterable container of vertices (default: `None`); when specified, return a dominating set of the specified vertices only
- `ordering` – string (default: `None`); specify the order in which to consider the vertices
  - `None` – if `vertices` is `None`, then consider the vertices in the order given by `list(G)`. Otherwise, consider the vertices in the order of iteration of `vertices`.
  - `'degree_min'` – consider the vertices by increasing degree
  - `'degree_max'` – consider the vertices by decreasing degree

- `return_sets` – boolean (default: `False`); whether to return the vertices of the dominating set only (default), or a dictionary mapping each vertex of the dominating set to the set of vertices it dominates.
- `closest` – boolean (default: `False`); whether to attach a vertex to its closest dominator or not. This parameter is use only when `return_sets` is `True`.

EXAMPLES:

Dominating sets of a path:

```
sage: from sage.graphs.domination import greedy_dominating_set
sage: G = graphs.PathGraph(5)
sage: sorted(greedy_dominating_set(G, ordering=None))
[0, 2, 4]
sage: sorted(greedy_dominating_set(G, ordering='degree_min'))
[0, 2, 4]
sage: sorted(greedy_dominating_set(G, ordering='degree_max'))
[1, 3]
sage: sorted(greedy_dominating_set(G, k=2, ordering=None))
[0, 3]
sage: sorted(greedy_dominating_set(G, k=2, ordering='degree_min'))
[0, 4]
sage: sorted(greedy_dominating_set(G, k=2, ordering='degree_max'))
[1, 4]
sage: greedy_dominating_set(G, k=3, ordering='degree_min', return_sets=True,␣
↪closest=False)
{0: {0, 1, 2, 3}, 4: {4}}
sage: greedy_dominating_set(G, k=3, ordering='degree_min', return_sets=True,␣
↪closest=True)
{0: {0, 2, 3}, 4: {1, 4}}
```

```
>>> from sage.all import *
>>> from sage.graphs.domination import greedy_dominating_set
>>> G = graphs.PathGraph(Integer(5))
>>> sorted(greedy_dominating_set(G, ordering=None))
[0, 2, 4]
>>> sorted(greedy_dominating_set(G, ordering='degree_min'))
[0, 2, 4]
>>> sorted(greedy_dominating_set(G, ordering='degree_max'))
[1, 3]
>>> sorted(greedy_dominating_set(G, k=Integer(2), ordering=None))
[0, 3]
>>> sorted(greedy_dominating_set(G, k=Integer(2), ordering='degree_min'))
[0, 4]
>>> sorted(greedy_dominating_set(G, k=Integer(2), ordering='degree_max'))
[1, 4]
>>> greedy_dominating_set(G, k=Integer(3), ordering='degree_min', return_
↪sets=True, closest=False)
{0: {0, 1, 2, 3}, 4: {4}}
>>> greedy_dominating_set(G, k=Integer(3), ordering='degree_min', return_
↪sets=True, closest=True)
{0: {0, 2, 3}, 4: {1, 4}}
```

Asking for a dominating set of a subset of vertices:

```
sage: from sage.graphs.domination import greedy_dominating_set
sage: from sage.graphs.domination import is_dominating
sage: G = graphs.PetersenGraph()
sage: vertices = {0, 1, 2, 3, 4, 5}
sage: dom = greedy_dominating_set(G, vertices=vertices, return_sets=True)
sage: sorted(dom)
[0, 2]
sage: is_dominating(G, dom, focus=vertices)
True
sage: is_dominating(G, dom)
False
sage: dominated = [u for v in dom for u in dom[v]]
sage: sorted(dominated) == sorted(vertices)
True
```

```
>>> from sage.all import *
>>> from sage.graphs.domination import greedy_dominating_set
>>> from sage.graphs.domination import is_dominating
>>> G = graphs.PetersenGraph()
>>> vertices = {Integer(0), Integer(1), Integer(2), Integer(3), Integer(4),␣
↪Integer(5)}
>>> dom = greedy_dominating_set(G, vertices=vertices, return_sets=True)
>>> sorted(dom)
[0, 2]
>>> is_dominating(G, dom, focus=vertices)
True
>>> is_dominating(G, dom)
False
>>> dominated = [u for v in dom for u in dom[v]]
>>> sorted(dominated) == sorted(vertices)
True
```

Influence of the ordering of the vertices on the result:

```
sage: from sage.graphs.domination import greedy_dominating_set
sage: G = graphs.StarGraph(4)
sage: greedy_dominating_set(G, vertices=[0, 1, 2, 3, 4])
[0]
sage: sorted(greedy_dominating_set(G, vertices=[1, 2, 3, 4, 0]))
[1, 2, 3, 4]
```

```
>>> from sage.all import *
>>> from sage.graphs.domination import greedy_dominating_set
>>> G = graphs.StarGraph(Integer(4))
>>> greedy_dominating_set(G, vertices=[Integer(0), Integer(1), Integer(2),␣
↪Integer(3), Integer(4)])
[0]
>>> sorted(greedy_dominating_set(G, vertices=[Integer(1), Integer(2),␣
↪Integer(3), Integer(4), Integer(0)]))
[1, 2, 3, 4]
```

Dominating set of a directed graph:

```
sage: from sage.graphs.domination import greedy_dominating_set
sage: D = digraphs.Path(3)
sage: sorted(greedy_dominating_set(D, vertices=[0, 1, 2]))
[0, 2]
```

```
>>> from sage.all import *
>>> from sage.graphs.domination import greedy_dominating_set
>>> D = digraphs.Path(Integer(3))
>>> sorted(greedy_dominating_set(D, vertices=[Integer(0), Integer(1),␣
↪Integer(2)]))
[0, 2]
```

hamiltonian_cycle(*algorithm*, *solver='tsp'*, *constraint_generation=None*, *verbose=None*, *verbose_constraints=0*, *integrality_tolerance=False*)

Return a Hamiltonian cycle/circuit of the current graph/digraph.

A graph (resp. digraph) is said to be Hamiltonian if it contains as a subgraph a cycle (resp. a circuit) going through all the vertices.

Computing a Hamiltonian cycle/circuit being NP-Complete, this algorithm could run for some time depending on the instance.

ALGORITHM:

See *traveling_salesman_problem()* for 'tsp' algorithm and *find_hamiltonian()* from *sage.graphs.generic_graph_pyx* for 'backtrack' algorithm.

INPUT:

- `algorithm` – string (default: `'tsp'`); one of 'tsp' or 'backtrack'
- `solver` – string (default: `None`); specifies a Mixed Integer Linear Programming (MILP) solver to be used. If set to `None`, the default one is used. For more information on MILP solvers and which default solver is used, see the method `solve` of the class `MixedIntegerLinearProgram`.
- `constraint_generation` – boolean (default: `None`); whether to use constraint generation when solving the Mixed Integer Linear Program.

  When `constraint_generation = None`, constraint generation is used whenever the graph has a density larger than 70%.
- `verbose` – integer (default: 0); sets the level of verbosity. Set to 0 by default, which means quiet.
- `verbose_constraints` – boolean (default: `False`); whether to display which constraints are being generated
- `integrality_tolerance` – float; parameter for use with MILP solvers over an inexact base ring; see `MixedIntegerLinearProgram.get_values()`.

OUTPUT:

If using the `'tsp'` algorithm, returns a Hamiltonian cycle/circuit if it exists; otherwise, raises a `EmptySetError` exception. If using the `'backtrack'` algorithm, returns a pair `(B, P)`. If `B` is `True` then `P` is a Hamiltonian cycle and if `B` is `False`, `P` is a longest path found by the algorithm. Observe that if `B` is `False`, the graph may still be Hamiltonian. The `'backtrack'` algorithm is only implemented for undirected graphs.

Warning

The `'backtrack'` algorithm may loop endlessly on graphs with vertices of degree 1.

NOTE:

This function, as *is_hamiltonian()*, computes a Hamiltonian cycle if it exists: the user should *NOT* test for Hamiltonicity using *is_hamiltonian()* before calling this function, as it would result in computing it twice.

The backtrack algorithm is only implemented for undirected graphs.

EXAMPLES:

The Heawood Graph is known to be Hamiltonian

```
sage: g = graphs.HeawoodGraph()
sage: g.hamiltonian_cycle()                                                 #␣
↪needs sage.numerical.mip
TSP from Heawood graph: Graph on 14 vertices
```

```
>>> from sage.all import *
>>> g = graphs.HeawoodGraph()
>>> g.hamiltonian_cycle()                                                   #␣
↪needs sage.numerical.mip
TSP from Heawood graph: Graph on 14 vertices
```

The Petersen Graph, though, is not

```
sage: g = graphs.PetersenGraph()
sage: g.hamiltonian_cycle()                                                 #␣
↪needs sage.numerical.mip
Traceback (most recent call last):
...
EmptySetError: the given graph is not Hamiltonian
```

```
>>> from sage.all import *
>>> g = graphs.PetersenGraph()
>>> g.hamiltonian_cycle()                                                   #␣
↪needs sage.numerical.mip
Traceback (most recent call last):
...
EmptySetError: the given graph is not Hamiltonian
```

Now, using the backtrack algorithm in the Heawood graph

```
sage: G = graphs.HeawoodGraph()
sage: G.hamiltonian_cycle(algorithm='backtrack')
(True, [...])
```

```
>>> from sage.all import *
>>> G = graphs.HeawoodGraph()
>>> G.hamiltonian_cycle(algorithm='backtrack')
(True, [...])
```

And now in the Petersen graph

```
sage: G = graphs.PetersenGraph()
sage: B, P = G.hamiltonian_cycle(algorithm='backtrack')
sage: B
False
sage: len(P)
10
sage: G.has_edge(P[0], P[-1])
False
```

```
>>> from sage.all import *
>>> G = graphs.PetersenGraph()
>>> B, P = G.hamiltonian_cycle(algorithm='backtrack')
>>> B
False
>>> len(P)
10
>>> G.has_edge(P[Integer(0)], P[-Integer(1)])
False
```

Finally, we test the algorithm in a cube graph, which is Hamiltonian

```
sage: G = graphs.CubeGraph(3)
sage: G.hamiltonian_cycle(algorithm='backtrack')
(True, [...])
```

```
>>> from sage.all import *
>>> G = graphs.CubeGraph(Integer(3))
>>> G.hamiltonian_cycle(algorithm='backtrack')
(True, [...])
```

hamiltonian_path(*s*, *t=None*, *use_edge_labels=None*, *maximize=False*, *algorithm=False*, *immutable='MILP'*, *solver=None*, *verbose=None*, *integrality_tolerance=0*)

Return a Hamiltonian path of the current graph/digraph.

A path is Hamiltonian if it goes through all the vertices exactly once. Computing a Hamiltonian path being NP-Complete, this algorithm could run for some time depending on the instance.

When `use_edge_labels == True`, this method returns either a minimum weight hamiltonian path or a maximum weight Hamiltonian path (if `maximize == True`).

See also

- *longest_path()*
- *hamiltonian_cycle()*

INPUT:

- `s` – vertex (default: `None`); if specified, then forces the source of the path (the method then returns a Hamiltonian path starting at `s`)
- `t` – vertex (default: `None`); if specified, then forces the destination of the path (the method then returns a Hamiltonian path ending at `t`)

- `use_edge_labels` – boolean (default: `False`); whether to compute a weighted hamiltonian path where the weight of an edge is defined by its label (a label set to `None` or `{}` being considered as a weight of 1), or a non-weighted hamiltonian path
- `maximize` – boolean (default: `False`); whether to compute a minimum (default) or a maximum (when `maximize == True`) weight hamiltonian path. This parameter is considered only if `use_edge_labels == True`.
- `algorithm` – string (default: `'MILP'`); the algorithm the use among `'MILP'` and `'backtrack'`; two remarks on this respect:
  - While the MILP formulation returns an exact answer, the backtrack algorithm is a randomized heuristic.
  - The backtrack algorithm does not support edge weighting, so setting `use_edge_labels=True` will force the use of the MILP algorithm.
- `immutable` – boolean (default: `None`); whether to create a mutable/immutable (di)graph. `immutable=None` (default) means that the (di)graph and its hamiltonian path will behave the same way.
- `solver` – string (default: `None`); specifies a Mixed Integer Linear Programming (MILP) solver to be used. If set to `None`, the default one is used. For more information on MILP solvers and which default solver is used, see the method `solve` of the class `MixedIntegerLinearProgram`.
- `verbose` – integer (default: 0); sets the level of verbosity. Set to 0 by default, which means quiet.
- `integrality_tolerance` – float; parameter for use with MILP solvers over an inexact base ring; see `MixedIntegerLinearProgram.get_values()`.

OUTPUT:

A subgraph of `self` corresponding to a (directed if `self` is directed) hamiltonian path. If no hamiltonian path is found, return `None`. If `use_edge_labels == True`, a pair `weight, path` is returned.

EXAMPLES:

The 3×3-grid has a Hamiltonian path, a hamiltonian path starting from vertex $(0, 0)$ and ending at vertex $(2, 2)$, but no Hamiltonian path starting from $(0, 0)$ and ending at $(0, 1)$:

```
sage: g = graphs.Grid2dGraph(3, 3)
sage: g.hamiltonian_path()
Hamiltonian path from 2D Grid Graph for [3, 3]: Graph on 9 vertices
sage: g.hamiltonian_path(s=(0, 0), t=(2, 2))
Hamiltonian path from 2D Grid Graph for [3, 3]: Graph on 9 vertices
sage: g.hamiltonian_path(s=(0, 0), t=(2, 2), use_edge_labels=True)
(8, Hamiltonian path from 2D Grid Graph for [3, 3]: Graph on 9 vertices)
sage: g.hamiltonian_path(s=(0, 0), t=(0, 1)) is None
True
sage: g.hamiltonian_path(s=(0, 0), t=(0, 1), use_edge_labels=True)
(0, None)
```

```
>>> from sage.all import *
>>> g = graphs.Grid2dGraph(Integer(3), Integer(3))
>>> g.hamiltonian_path()
Hamiltonian path from 2D Grid Graph for [3, 3]: Graph on 9 vertices
>>> g.hamiltonian_path(s=(Integer(0), Integer(0)), t=(Integer(2), Integer(2)))
Hamiltonian path from 2D Grid Graph for [3, 3]: Graph on 9 vertices
>>> g.hamiltonian_path(s=(Integer(0), Integer(0)), t=(Integer(2), Integer(2)),
↪ use_edge_labels=True)
```

(continues on next page)

(continued from previous page)

```
(8, Hamiltonian path from 2D Grid Graph for [3, 3]: Graph on 9 vertices)
>>> g.hamiltonian_path(s=(Integer(0), Integer(0)), t=(Integer(0),␣
↪Integer(1))) is None
True
>>> g.hamiltonian_path(s=(Integer(0), Integer(0)), t=(Integer(0), Integer(1)),
↪ use_edge_labels=True)
(0, None)
```

has_edge(*u*, *v=None*, *label=None*)

Check whether `(u, v)` is an edge of the (di)graph.

INPUT: The following forms are accepted:

- G.has_edge(1, 2)
- G.has_edge((1, 2))
- G.has_edge(1, 2, 'label')
- G.has_edge((1, 2, 'label'))

EXAMPLES:

```
sage: graphs.EmptyGraph().has_edge(9, 2)
False
sage: DiGraph().has_edge(9, 2)
False
sage: G = Graph(sparse=True)
sage: G.add_edge(0, 1, "label")
sage: G.has_edge(0, 1, "different label")
False
sage: G.has_edge(0, 1, "label")
True
```

```
>>> from sage.all import *
>>> graphs.EmptyGraph().has_edge(Integer(9), Integer(2))
False
>>> DiGraph().has_edge(Integer(9), Integer(2))
False
>>> G = Graph(sparse=True)
>>> G.add_edge(Integer(0), Integer(1), "label")
>>> G.has_edge(Integer(0), Integer(1), "different label")
False
>>> G.has_edge(Integer(0), Integer(1), "label")
True
```

has_homomorphism_to(*G*, *H*, *core*, *solver=False*, *verbose=None*, *integrality_tolerance=0*)

Check whether there is a homomorphism between two graphs.

A homomorphism from a graph G to a graph H is a function $\phi : V(G) \mapsto V(H)$ such that for any edge $uv \in E(G)$ the pair $\phi(u)\phi(v)$ is an edge of H.

Saying that a graph can be k-colored is equivalent to saying that it has a homomorphism to K_k, the complete graph of order k.

For more information, see the Wikipedia article Graph_homomorphism.

INPUT:

- `G` – the graph to map
- `H` – the graph to which `G` should be sent
- `core` – boolean (default: `False`); whether to minimize the size of the mapping's image (see examples below). This is set to `False` by default.
- `solver` – string (default: `None`); specifies a Mixed Integer Linear Programming (MILP) solver to be used. If set to `None`, the default one is used. For more information on MILP solvers and which default solver is used, see the method `solve` of the class `MixedIntegerLinearProgram`.
- `verbose` – integer (default: 0); sets the level of verbosity. Set to 0 by default, which means quiet.
- `integrality_tolerance` – float; parameter for use with MILP solvers over an inexact base ring; see `MixedIntegerLinearProgram.get_values()`.

OUTPUT:

This method returns `False` when the homomorphism does not exist, and returns the homomorphism otherwise as a dictionary associating a vertex of H to a vertex of G.

EXAMPLES:

Is Petersen's graph 3-colorable:

```
sage: P = graphs.PetersenGraph()
sage: P.has_homomorphism_to(graphs.CompleteGraph(3)) is not False
↪  # needs sage.numerical.mip
True
```

```
>>> from sage.all import *
>>> P = graphs.PetersenGraph()
>>> P.has_homomorphism_to(graphs.CompleteGraph(Integer(3))) is not False
↪         # needs sage.numerical.mip
True
```

An odd cycle admits a homomorphism to a smaller odd cycle, but not to an even cycle:

```
sage: g = graphs.CycleGraph(9)
sage: g.has_homomorphism_to(graphs.CycleGraph(5)) is not False
↪  # needs sage.numerical.mip
True
sage: g.has_homomorphism_to(graphs.CycleGraph(7)) is not False
↪  # needs sage.numerical.mip
True
sage: g.has_homomorphism_to(graphs.CycleGraph(4)) is not False
↪  # needs sage.numerical.mip
False
```

```
>>> from sage.all import *
>>> g = graphs.CycleGraph(Integer(9))
>>> g.has_homomorphism_to(graphs.CycleGraph(Integer(5))) is not False
↪         # needs sage.numerical.mip
True
>>> g.has_homomorphism_to(graphs.CycleGraph(Integer(7))) is not False
↪         # needs sage.numerical.mip
```

(continues on next page)

(continued from previous page)

```
True
>>> g.has_homomorphism_to(graphs.CycleGraph(Integer(4))) is not False
↪          # needs sage.numerical.mip
False
```

One can compute the core of a graph (with respect to homomorphism) with this method:

```
sage: g = graphs.CycleGraph(8)
sage: mapping = g.has_homomorphism_to(g, core=True)
sage: print(f"The size of the core is {len(set(mapping.values()))}")
The size of the core is 2
sage: g = graphs.CycleGraph(9)
sage: mapping = g.has_homomorphism_to(g, core=True)
sage: print(f"The size of the core is {len(set(mapping.values()))}")
The size of the core is 9
```

```
>>> from sage.all import *
>>> g = graphs.CycleGraph(Integer(8))
>>> mapping = g.has_homomorphism_to(g, core=True)
>>> print(f"The size of the core is {len(set(mapping.values()))}")
The size of the core is 2
>>> g = graphs.CycleGraph(Integer(9))
>>> mapping = g.has_homomorphism_to(g, core=True)
>>> print(f"The size of the core is {len(set(mapping.values()))}")
The size of the core is 9
```

The chromatic number of a graph is the order of the smallest clique to which it has an homomorphism:

```
sage: g = graphs.CycleGraph(9)
sage: g.chromatic_number()
3
sage: g.has_homomorphism_to(graphs.CompleteGraph(3)) is not False
True
sage: g.has_homomorphism_to(graphs.CompleteGraph(2)) is not False
False
sage: K6 = graphs.CompleteGraph(6)
sage: g.has_homomorphism_to(K6) is not False
True
sage: mapping = g.has_homomorphism_to(K6, core=True)
sage: print(f"The size of the core is {len(set(mapping.values()))}")
The size of the core is 3
```

```
>>> from sage.all import *
>>> g = graphs.CycleGraph(Integer(9))
>>> g.chromatic_number()
3
>>> g.has_homomorphism_to(graphs.CompleteGraph(Integer(3))) is not False
True
>>> g.has_homomorphism_to(graphs.CompleteGraph(Integer(2))) is not False
False
>>> K6 = graphs.CompleteGraph(Integer(6))
>>> g.has_homomorphism_to(K6) is not False
```

(continues on next page)

(continued from previous page)

```
True
>>> mapping = g.has_homomorphism_to(K6, core=True)
>>> print(f"The size of the core is {len(set(mapping.values()))}")
The size of the core is 3
```

A circuit of order n admits a homomorphism to smaller circuit of order $p \leq n$ if p is a divisor of n:

```
sage: g = digraphs.Circuit(12)
sage: [i for i in range(2, g.order() + 1)
↪  # needs sage.numerical.mip
....:  if g.has_homomorphism_to(digraphs.Circuit(i)) is not False]
[2, 3, 4, 6, 12]
```

```
>>> from sage.all import *
>>> g = digraphs.Circuit(Integer(12))
>>> [i for i in range(Integer(2), g.order() + Integer(1))
↪                  # needs sage.numerical.mip
...  if g.has_homomorphism_to(digraphs.Circuit(i)) is not False]
[2, 3, 4, 6, 12]
```

has_loops()

Return whether there are loops in the (di)graph.

EXAMPLES:

```
sage: G = Graph(loops=True); G
Looped graph on 0 vertices
sage: G.has_loops()
False
sage: G.allows_loops()
True
sage: G.add_edge((0, 0))
sage: G.has_loops()
True
sage: G.loops()
[(0, 0, None)]
sage: G.allow_loops(False); G
Graph on 1 vertex
sage: G.has_loops()
False
sage: G.edges(sort=True)
[]

sage: D = DiGraph(loops=True); D
Looped digraph on 0 vertices
sage: D.has_loops()
False
sage: D.allows_loops()
True
sage: D.add_edge((0, 0))
sage: D.has_loops()
True
```

(continues on next page)

(continued from previous page)

```
sage: D.loops()
[(0, 0, None)]
sage: D.allow_loops(False); D
Digraph on 1 vertex
sage: D.has_loops()
False
sage: D.edges(sort=True)
[]
```

```
>>> from sage.all import *
>>> G = Graph(loops=True); G
Looped graph on 0 vertices
>>> G.has_loops()
False
>>> G.allows_loops()
True
>>> G.add_edge((Integer(0), Integer(0)))
>>> G.has_loops()
True
>>> G.loops()
[(0, 0, None)]
>>> G.allow_loops(False); G
Graph on 1 vertex
>>> G.has_loops()
False
>>> G.edges(sort=True)
[]

>>> D = DiGraph(loops=True); D
Looped digraph on 0 vertices
>>> D.has_loops()
False
>>> D.allows_loops()
True
>>> D.add_edge((Integer(0), Integer(0)))
>>> D.has_loops()
True
>>> D.loops()
[(0, 0, None)]
>>> D.allow_loops(False); D
Digraph on 1 vertex
>>> D.has_loops()
False
>>> D.edges(sort=True)
[]
```

has_multiple_edges(*to_undirected=False*)

Return whether there are multiple edges in the (di)graph.

INPUT:

- `to_undirected` – (default: `False`); if `True`, runs the test on the undirected version of a DiGraph. Otherwise, treats DiGraph edges `(u, v)` and `(v, u)` as unique individual edges.

EXAMPLES:

```
sage: G = Graph(multiedges=True, sparse=True); G
Multi-graph on 0 vertices
sage: G.has_multiple_edges()
False
sage: G.allows_multiple_edges()
True
sage: G.add_edges([(0, 1)] * 3)
sage: G.has_multiple_edges()
True
sage: G.multiple_edges()
[(0, 1, None), (0, 1, None), (0, 1, None)]
sage: G.allow_multiple_edges(False); G
Graph on 2 vertices
sage: G.has_multiple_edges()
False
sage: G.edges(sort=True)
[(0, 1, None)]

sage: D = DiGraph(multiedges=True, sparse=True); D
Multi-digraph on 0 vertices
sage: D.has_multiple_edges()
False
sage: D.allows_multiple_edges()
True
sage: D.add_edges([(0, 1)] * 3)
sage: D.has_multiple_edges()
True
sage: D.multiple_edges()
[(0, 1, None), (0, 1, None), (0, 1, None)]
sage: D.allow_multiple_edges(False); D
Digraph on 2 vertices
sage: D.has_multiple_edges()
False
sage: D.edges(sort=True)
[(0, 1, None)]

sage: G = DiGraph({1: {2: 'h'}, 2: {1: 'g'}}, sparse=True)
sage: G.has_multiple_edges()
False
sage: G.has_multiple_edges(to_undirected=True)
True
sage: G.multiple_edges()
[]
sage: G.multiple_edges(to_undirected=True)
[(1, 2, 'h'), (2, 1, 'g')]
```

```
>>> from sage.all import *
>>> G = Graph(multiedges=True, sparse=True); G
Multi-graph on 0 vertices
>>> G.has_multiple_edges()
False
```

(continues on next page)

(continued from previous page)

```
>>> G.allows_multiple_edges()
True
>>> G.add_edges([(Integer(0), Integer(1))] * Integer(3))
>>> G.has_multiple_edges()
True
>>> G.multiple_edges()
[(0, 1, None), (0, 1, None), (0, 1, None)]
>>> G.allow_multiple_edges(False); G
Graph on 2 vertices
>>> G.has_multiple_edges()
False
>>> G.edges(sort=True)
[(0, 1, None)]

>>> D = DiGraph(multiedges=True, sparse=True); D
Multi-digraph on 0 vertices
>>> D.has_multiple_edges()
False
>>> D.allows_multiple_edges()
True
>>> D.add_edges([(Integer(0), Integer(1))] * Integer(3))
>>> D.has_multiple_edges()
True
>>> D.multiple_edges()
[(0, 1, None), (0, 1, None), (0, 1, None)]
>>> D.allow_multiple_edges(False); D
Digraph on 2 vertices
>>> D.has_multiple_edges()
False
>>> D.edges(sort=True)
[(0, 1, None)]

>>> G = DiGraph({Integer(1): {Integer(2): 'h'}, Integer(2): {Integer(1): 'g'}}
↪, sparse=True)
>>> G.has_multiple_edges()
False
>>> G.has_multiple_edges(to_undirected=True)
True
>>> G.multiple_edges()
[]
>>> G.multiple_edges(to_undirected=True)
[(1, 2, 'h'), (2, 1, 'g')]
```

A loop is not a multiedge:

```
sage: g = Graph(loops=True, multiedges=True)
sage: g.add_edge(0, 0)
sage: g.has_multiple_edges()
False
```

```
>>> from sage.all import *
>>> g = Graph(loops=True, multiedges=True)
```

(continues on next page)

(continued from previous page)

```
>>> g.add_edge(Integer(0), Integer(0))
>>> g.has_multiple_edges()
False
```

has_vertex(*vertex*)

Check if `vertex` is one of the vertices of this graph.

INPUT:

- `vertex` – the name of a vertex (see *add_vertex()*)

EXAMPLES:

```
sage: g = Graph({0: [1, 2, 3], 2: [4]}); g
Graph on 5 vertices
sage: 2 in g
True
sage: 10 in g
False
sage: graphs.PetersenGraph().has_vertex(99)
False
```

```
>>> from sage.all import *
>>> g = Graph({Integer(0): [Integer(1), Integer(2), Integer(3)], Integer(2):␣
↪[Integer(4)]}); g
Graph on 5 vertices
>>> Integer(2) in g
True
>>> Integer(10) in g
False
>>> graphs.PetersenGraph().has_vertex(Integer(99))
False
```

igraph_graph(*vertex_list=None*, *vertex_attrs={}*, *edge_attrs={}*)

Return an `igraph` graph from the Sage graph.

Optionally, it is possible to add vertex attributes and edge attributes to the output graph.

Note

This routine needs the optional package igraph to be installed: to do so, it is enough to run `sage -i python_igraph`. For more information on the Python version of igraph, see https://python.igraph.org/.

INPUT:

- `vertex_list` – list (default: `None`); defines a mapping from the vertices of the graph to consecutive integers in $(0, \ldots, n-1)$. Otherwise, the result of *vertices()* will be used instead. Because *vertices()* only works if the vertices can be sorted, using `vertex_list` is useful when working with possibly non-sortable objects in Python 3.
- `vertex_attrs` – dictionary (default: `{}`); a dictionary where the key is a string (the attribute name), and the value is an iterable containing in position i the label of the i-th vertex in the list `vertex_list` if it is given or in *vertices()* when `vertex_list == None` (see https://python.igraph.org/en/stable/api/igraph.Graph.html#__init__ for more information)

- `edge_attrs` – dictionary (default: `{}`); a dictionary where the key is a string (the attribute name), and the value is an iterable containing in position i the label of the i-th edge in the list outputted by *`edge_iterator()`* (see https://python.igraph.org/en/stable/api/igraph.Graph.html#__init__ for more information)

Note

In `igraph`, a graph is weighted if the edge attribute `weight` is present. Hence, to create a weighted graph, it is enough to add this attribute.

Note

Often, Sage uses its own defined types for integers/floats. These types may not be igraph-compatible (see example below).

EXAMPLES:

Standard conversion:

```
sage: G = graphs.TetrahedralGraph()
sage: H = G.igraph_graph()           # optional - python_igraph
sage: H.summary()                    # optional - python_igraph
'IGRAPH U--- 4 6 -- '
sage: G = digraphs.Path(3)
sage: H = G.igraph_graph()           # optional - python_igraph
sage: H.summary()                    # optional - python_igraph
'IGRAPH D--- 3 2 -- '
```

```
>>> from sage.all import *
>>> G = graphs.TetrahedralGraph()
>>> H = G.igraph_graph()           # optional - python_igraph
>>> H.summary()                    # optional - python_igraph
'IGRAPH U--- 4 6 -- '
>>> G = digraphs.Path(Integer(3))
>>> H = G.igraph_graph()           # optional - python_igraph
>>> H.summary()                    # optional - python_igraph
'IGRAPH D--- 3 2 -- '
```

Adding edge attributes:

```
sage: G = Graph([(1, 2, 'a'), (2, 3, 'b')])
sage: E = list(G.edge_iterator())
sage: H = G.igraph_graph(edge_attrs={'label': [e[2] for e in E]})  # optional␣
↪- python_igraph
sage: H.es['label']                                                # optional␣
↪- python_igraph
['a', 'b']
```

```
>>> from sage.all import *
>>> G = Graph([(Integer(1), Integer(2), 'a'), (Integer(2), Integer(3), 'b')])
>>> E = list(G.edge_iterator())
```

(continues on next page)

(continued from previous page)

```
>>> H = G.igraph_graph(edge_attrs={'label': [e[Integer(2)] for e in E]})  #␣
↪optional - python_igraph
>>> H.es['label']                                                    # optional -␣
↪python_igraph
['a', 'b']
```

If edges have an attribute `weight`, the igraph graph is considered weighted:

```
sage: G = Graph([(1, 2, {'weight': 1}), (2, 3, {'weight': 2})])
sage: E = list(G.edge_iterator())
sage: H = G.igraph_graph(edge_attrs={'weight': [e[2]['weight'] for e in E]})
↪# optional - python_igraph
sage: H.is_weighted()
↪# optional - python_igraph
True
sage: H.es['weight']
↪# optional - python_igraph
[1, 2]
```

```
>>> from sage.all import *
>>> G = Graph([(Integer(1), Integer(2), {'weight': Integer(1)}), (Integer(2),␣
↪Integer(3), {'weight': Integer(2)})])
>>> E = list(G.edge_iterator())
>>> H = G.igraph_graph(edge_attrs={'weight': [e[Integer(2)]['weight'] for e␣
↪in E]})  # optional - python_igraph
>>> H.is_weighted()                                                        #␣
↪optional - python_igraph
True
>>> H.es['weight']                                                         #␣
↪optional - python_igraph
[1, 2]
```

Adding vertex attributes:

```
sage: G = graphs.GridGraph([2, 2])
sage: H = G.igraph_graph(vertex_attrs={'name': G.vertices(sort=True)})  #␣
↪optional - python_igraph
sage: H.vs()['name']                                                    #␣
↪optional - python_igraph
[(0, 0), (0, 1), (1, 0), (1, 1)]
```

```
>>> from sage.all import *
>>> G = graphs.GridGraph([Integer(2), Integer(2)])
>>> H = G.igraph_graph(vertex_attrs={'name': G.vertices(sort=True)})  #␣
↪optional - python_igraph
>>> H.vs()['name']                                                    #␣
↪optional - python_igraph
[(0, 0), (0, 1), (1, 0), (1, 1)]
```

Providing a mapping from vertices to consecutive integers:

```
sage: G = graphs.GridGraph([2, 2])
sage: V = list(G)
sage: H = G.igraph_graph(vertex_list=V, vertex_attrs={'name': V})  # optional␣
↪- python_igraph
sage: H.vs()['name'] == V                                          # optional␣
↪- python_igraph
True
```

```
>>> from sage.all import *
>>> G = graphs.GridGraph([Integer(2), Integer(2)])
>>> V = list(G)
>>> H = G.igraph_graph(vertex_list=V, vertex_attrs={'name': V})  # optional -␣
↪python_igraph
>>> H.vs()['name'] == V                                          # optional -␣
↪python_igraph
True
```

Sometimes, Sage integers/floats are not compatible with igraph:

```
sage: G = Graph([(0, 1, 2)])
sage: E = list(G.edge_iterator())
sage: H = G.igraph_graph(edge_attrs={'capacity': [e[2] for e in E]})  #␣
↪optional - python_igraph
sage: H.maxflow_value(0, 1, 'capacity')                               #␣
↪optional - python_igraph
1.0
sage: H = G.igraph_graph(edge_attrs={'capacity': [float(e[2]) for e in E]})
↪# optional - python_igraph
sage: H.maxflow_value(0, 1, 'capacity')
↪# optional - python_igraph
2.0
```

```
>>> from sage.all import *
>>> G = Graph([(Integer(0), Integer(1), Integer(2))])
>>> E = list(G.edge_iterator())
>>> H = G.igraph_graph(edge_attrs={'capacity': [e[Integer(2)] for e in E]})
↪# optional - python_igraph
>>> H.maxflow_value(Integer(0), Integer(1), 'capacity')                    ␣
↪        # optional - python_igraph
1.0
>>> H = G.igraph_graph(edge_attrs={'capacity': [float(e[Integer(2)]) for e in␣
↪E]})  # optional - python_igraph
>>> H.maxflow_value(Integer(0), Integer(1), 'capacity')                    ␣
↪               # optional - python_igraph
2.0
```

incidence_matrix(*oriented*, *sparse=None*, *vertices=True*, *edges=None*, *base_ring=None*, ***kwds*)

Return the incidence matrix of the (di)graph.

Each row is a vertex, and each column is an edge. The vertices are ordered as obtained by the method *vertices()*, except when parameter `vertices` is given (see below), and the edges as obtained by the method *edge_iterator()*.

If the graph is not directed, then return a matrix with entries in $\{0, 1, 2\}$. Each column will either contain

two 1 (at the position of the endpoint of the edge), or one 2 (if the corresponding edge is a loop).

If the graph is directed return a matrix in $\{-1, 0, 1\}$ where -1 and $+1$ correspond respectively to the source and the target of the edge. A loop will correspond to a zero column. In particular, it is not possible to recover the loops of an oriented graph from its incidence matrix.

See the Wikipedia article Incidence_matrix for more information.

INPUT:

- `oriented` – boolean (default: `None`); when set to `True`, the matrix will be oriented (i.e. with entries in -1, 0, 1) and if set to `False` the matrix will be not oriented (i.e. with entries in 0, 1, 2). By default, this argument is inferred from the graph type. Note that in the case the graph is not directed and with the option `directed=True`, a somewhat random direction is chosen for each edge.
- `sparse` – boolean (default: `True`); whether to use a sparse or a dense matrix
- `vertices` – list; `None`, or `True` (default: `None`)
  - when a list, the i-th row of the matrix corresponds to the i-th vertex in the ordering of `vertices`,
  - when `None`, the i-th row of the matrix corresponds to the i-th vertex in the ordering given by method *`vertices()`*,
  - when `True`, construct a morphism of free modules instead of a matrix, where the codomain's basis is indexed by the vertices.
- `edges` – list; `None`, or `True` (default: `None`)
  - when a list, the i-th column of the matrix corresponds to the i-th edge in the ordering of `edges`,
  - when `None`, the i-th column of the matrix corresponds to the i-th edge in the ordering given by method *`edge_iterator()`*,
  - when `True`, construct a morphism of free modules instead of a matrix, where the domain's basis is indexed by the edges.
- `base_ring` – a ring (default: `ZZ`); the base ring of the matrix space to use
- `**kwds` – other keywords to pass to `matrix()`

EXAMPLES:

```
sage: G = graphs.PetersenGraph()
sage: G.incidence_matrix()                                                  #␣
↪needs sage.modules
[1 1 1 0 0 0 0 0 0 0 0 0 0 0 0]
[1 0 0 1 1 0 0 0 0 0 0 0 0 0 0]
[0 0 0 1 0 1 1 0 0 0 0 0 0 0 0]
[0 0 0 0 0 1 0 1 1 0 0 0 0 0 0]
[0 1 0 0 0 0 0 1 0 1 0 0 0 0 0]
[0 0 1 0 0 0 0 0 0 0 1 1 0 0 0]
[0 0 0 0 1 0 0 0 0 0 0 0 1 1 0]
[0 0 0 0 0 0 1 0 0 0 1 0 0 0 1]
[0 0 0 0 0 0 0 0 1 0 0 1 1 0 0]
[0 0 0 0 0 0 0 0 0 1 0 0 0 1 1]
sage: G.incidence_matrix(oriented=True)                                     #␣
↪needs sage.modules
[-1 -1 -1  0  0  0  0  0  0  0  0  0  0  0  0]
[ 1  0  0 -1 -1  0  0  0  0  0  0  0  0  0  0]
[ 0  0  0  1  0 -1 -1  0  0  0  0  0  0  0  0]
```

(continues on next page)

(continued from previous page)

```
[ 0  0  0  0  0  1  0 -1 -1  0  0  0  0  0  0]
[ 0  1  0  0  0  0  0  1  0 -1  0  0  0  0  0]
[ 0  0  1  0  0  0  0  0  0  0 -1 -1  0  0  0]
[ 0  0  0  0  1  0  0  0  0  0  0  0 -1 -1  0]
[ 0  0  0  0  0  0  1  0  0  0  1  0  0  0 -1]
[ 0  0  0  0  0  0  0  0  1  0  0  1  1  0  0]
[ 0  0  0  0  0  0  0  0  0  1  0  0  0  1  1]

sage: G = digraphs.Circulant(4, [1, 3])
sage: G.incidence_matrix()                                                  #␣
↪needs sage.modules
[-1 -1  1  0  0  0  1  0]
[ 1  0 -1 -1  1  0  0  0]
[ 0  0  0  1 -1 -1  0  1]
[ 0  1  0  0  0  1 -1 -1]

sage: graphs.CompleteGraph(3).incidence_matrix()                            #␣
↪needs sage.modules
[1 1 0]
[1 0 1]
[0 1 1]
sage: G = Graph([(0, 0), (0, 1), (0, 1)], loops=True, multiedges=True)
sage: G.incidence_matrix(oriented=False)                                    #␣
↪needs sage.modules
[2 1 1]
[0 1 1]
```

```
>>> from sage.all import *
>>> G = graphs.PetersenGraph()
>>> G.incidence_matrix()                                                    #␣
↪needs sage.modules
[1 1 1 0 0 0 0 0 0 0 0 0 0 0 0]
[1 0 0 1 1 0 0 0 0 0 0 0 0 0 0]
[0 0 0 1 0 1 1 0 0 0 0 0 0 0 0]
[0 0 0 0 0 1 0 1 1 0 0 0 0 0 0]
[0 1 0 0 0 0 0 1 0 1 0 0 0 0 0]
[0 0 1 0 0 0 0 0 0 0 1 1 0 0 0]
[0 0 0 0 1 0 0 0 0 0 0 0 1 1 0]
[0 0 0 0 0 0 1 0 0 0 1 0 0 0 1]
[0 0 0 0 0 0 0 0 1 0 0 1 1 0 0]
[0 0 0 0 0 0 0 0 0 1 0 0 0 1 1]
>>> G.incidence_matrix(oriented=True)                                       #␣
↪needs sage.modules
[-1 -1 -1  0  0  0  0  0  0  0  0  0  0  0  0]
[ 1  0  0 -1 -1  0  0  0  0  0  0  0  0  0  0]
[ 0  0  0  1  0 -1 -1  0  0  0  0  0  0  0  0]
[ 0  0  0  0  0  1  0 -1 -1  0  0  0  0  0  0]
[ 0  1  0  0  0  0  0  1  0 -1  0  0  0  0  0]
[ 0  0  1  0  0  0  0  0  0  0 -1 -1  0  0  0]
[ 0  0  0  0  1  0  0  0  0  0  0  0 -1 -1  0]
[ 0  0  0  0  0  0  1  0  0  0  1  0  0  0 -1]
[ 0  0  0  0  0  0  0  0  1  0  0  1  1  0  0]
```

(continues on next page)

(continued from previous page)

```
[ 0  0  0  0  0  0  0  0  0  1  0  0  0  1  1]

>>> G = digraphs.Circulant(Integer(4), [Integer(1), Integer(3)])
>>> G.incidence_matrix()                                                    #␣
↪needs sage.modules
[-1 -1  1  0  0  0  1  0]
[ 1  0 -1 -1  1  0  0  0]
[ 0  0  0  1 -1 -1  0  1]
[ 0  1  0  0  0  1 -1 -1]

>>> graphs.CompleteGraph(Integer(3)).incidence_matrix()                     ␣
↪     # needs sage.modules
[1 1 0]
[1 0 1]
[0 1 1]
>>> G = Graph([(Integer(0), Integer(0)), (Integer(0), Integer(1)),␣
↪(Integer(0), Integer(1))], loops=True, multiedges=True)
>>> G.incidence_matrix(oriented=False)                                      #␣
↪needs sage.modules
[2 1 1]
[0 1 1]
```

A well known result states that the product of the (oriented) incidence matrix with its transpose of a (non-oriented graph) is in fact the Kirchhoff matrix:

```
sage: G = graphs.PetersenGraph()
sage: m = G.incidence_matrix(oriented=True)                                 #␣
↪needs sage.modules
sage: m * m.transpose() == G.kirchhoff_matrix()                             #␣
↪needs sage.modules
True

sage: K = graphs.CompleteGraph(3)
sage: m = K.incidence_matrix(oriented=True)                                 #␣
↪needs sage.modules
sage: m * m.transpose() == K.kirchhoff_matrix()                             #␣
↪needs sage.modules
True

sage: H = Graph([(0, 0), (0, 1), (0, 1)], loops=True, multiedges=True)
sage: m = H.incidence_matrix(oriented=True)                                 #␣
↪needs sage.modules
sage: m * m.transpose() == H.kirchhoff_matrix()                             #␣
↪needs sage.modules
True
```

```
>>> from sage.all import *
>>> G = graphs.PetersenGraph()
>>> m = G.incidence_matrix(oriented=True)                                   #␣
↪needs sage.modules
>>> m * m.transpose() == G.kirchhoff_matrix()                               #␣
↪needs sage.modules
```

(continues on next page)

(continued from previous page)

```
True

>>> K = graphs.CompleteGraph(Integer(3))
>>> m = K.incidence_matrix(oriented=True)                                    #␣
↪needs sage.modules
>>> m * m.transpose() == K.kirchhoff_matrix()                                #␣
↪needs sage.modules
True

>>> H = Graph([(Integer(0), Integer(0)), (Integer(0), Integer(1)),␣
↪(Integer(0), Integer(1))], loops=True, multiedges=True)
>>> m = H.incidence_matrix(oriented=True)                                    #␣
↪needs sage.modules
>>> m * m.transpose() == H.kirchhoff_matrix()                                #␣
↪needs sage.modules
True
```

A different ordering of the vertices:

```
sage: P5 = graphs.PathGraph(5)
sage: P5.incidence_matrix()                                                  #␣
↪needs sage.modules
[1 0 0 0]
[1 1 0 0]
[0 1 1 0]
[0 0 1 1]
[0 0 0 1]
sage: P5.incidence_matrix(vertices=[2, 4, 1, 3, 0])                          #␣
↪needs sage.modules
[0 1 1 0]
[0 0 0 1]
[1 1 0 0]
[0 0 1 1]
[1 0 0 0]
```

```
>>> from sage.all import *
>>> P5 = graphs.PathGraph(Integer(5))
>>> P5.incidence_matrix()                                                    #␣
↪needs sage.modules
[1 0 0 0]
[1 1 0 0]
[0 1 1 0]
[0 0 1 1]
[0 0 0 1]
>>> P5.incidence_matrix(vertices=[Integer(2), Integer(4), Integer(1),␣
↪Integer(3), Integer(0)])                          # needs sage.modules
[0 1 1 0]
[0 0 0 1]
[1 1 0 0]
[0 0 1 1]
[1 0 0 0]
```

A different ordering of the edges:

```
sage: E = list(P5.edge_iterator(labels=False))
sage: P5.incidence_matrix(edges=E[::-1])                                    #␣
↪needs sage.modules
[0 0 0 1]
[0 0 1 1]
[0 1 1 0]
[1 1 0 0]
[1 0 0 0]
sage: P5.incidence_matrix(vertices=[2, 4, 1, 3, 0], edges=E[::-1])          #␣
↪needs sage.modules
[0 1 1 0]
[1 0 0 0]
[0 0 1 1]
[1 1 0 0]
[0 0 0 1]
```

```
>>> from sage.all import *
>>> E = list(P5.edge_iterator(labels=False))
>>> P5.incidence_matrix(edges=E[::-Integer(1)])                              ␣
↪     # needs sage.modules
[0 0 0 1]
[0 0 1 1]
[0 1 1 0]
[1 1 0 0]
[1 0 0 0]
>>> P5.incidence_matrix(vertices=[Integer(2), Integer(4), Integer(1),␣
↪Integer(3), Integer(0)], edges=E[::-Integer(1)])          # needs sage.
↪modules
[0 1 1 0]
[1 0 0 0]
[0 0 1 1]
[1 1 0 0]
[0 0 0 1]
```

A different base ring:

```
sage: P5.incidence_matrix(base_ring=RDF)                                    #␣
↪needs sage.modules
[1.0 0.0 0.0 0.0]
[1.0 1.0 0.0 0.0]
[0.0 1.0 1.0 0.0]
[0.0 0.0 1.0 1.0]
[0.0 0.0 0.0 1.0]
```

```
>>> from sage.all import *
>>> P5.incidence_matrix(base_ring=RDF)                                      #␣
↪needs sage.modules
[1.0 0.0 0.0 0.0]
[1.0 1.0 0.0 0.0]
[0.0 1.0 1.0 0.0]
[0.0 0.0 1.0 1.0]
[0.0 0.0 0.0 1.0]
```

Creating an immutable matrix:

```
sage: m = P5.incidence_matrix(immutable=True); m                                #␣
↪needs sage.modules
[1 0 0 0]
[1 1 0 0]
[0 1 1 0]
[0 0 1 1]
[0 0 0 1]
sage: m[1,2] = 1                                                                #␣
↪needs sage.modules
Traceback (most recent call last):
...
ValueError: matrix is immutable; please change a copy instead
(i.e., use copy(M) to change a copy of M).
```

```
>>> from sage.all import *
>>> m = P5.incidence_matrix(immutable=True); m                                  #␣
↪needs sage.modules
[1 0 0 0]
[1 1 0 0]
[0 1 1 0]
[0 0 1 1]
[0 0 0 1]
>>> m[Integer(1),Integer(2)] = Integer(1)                                       ␣
↪                          # needs sage.modules
Traceback (most recent call last):
...
ValueError: matrix is immutable; please change a copy instead
(i.e., use copy(M) to change a copy of M).
```

Creating a module morphism:

```
sage: D12 = posets.DivisorLattice(12).hasse_diagram()
sage: phi_VE = D12.incidence_matrix(vertices=True, edges=True); phi_VE
Generic morphism:
  From: Free module generated by
          {(1, 2), (1, 3), (2, 4), (2, 6), (3, 6), (4, 12), (6, 12)}
        over Integer Ring
  To:   Free module generated by {1, 2, 3, 4, 6, 12} over Integer Ring
sage: print(phi_VE._unicode_art_matrix())
    (1, 2)  (1, 3)  (2, 4)  (2, 6)  (3, 6) (4, 12) (6, 12)
 1⎛     -1      -1       0       0       0       0       0⎞
 2⎜      1       0      -1      -1       0       0       0⎜
 3⎜      0       1       0       0      -1       0       0⎜
 4⎜      0       0       1       0       0      -1       0⎜
 6⎜      0       0       0       1       1       0      -1⎜
12⎝      0       0       0       0       0       1       1⎠
sage: E = phi_VE.domain()
sage: P1 = E.monomial((2, 4)) + E.monomial((4, 12)); P1
B[(2, 4)] + B[(4, 12)]
sage: P2 = E.monomial((2, 6)) + E.monomial((6, 12)); P2
B[(2, 6)] + B[(6, 12)]
```

(continues on next page)

(continued from previous page)

```
sage: phi_VE(P1 - P2)
0
```

```
>>> from sage.all import *
>>> D12 = posets.DivisorLattice(Integer(12)).hasse_diagram()
>>> phi_VE = D12.incidence_matrix(vertices=True, edges=True); phi_VE
Generic morphism:
  From: Free module generated by
          {(1, 2), (1, 3), (2, 4), (2, 6), (3, 6), (4, 12), (6, 12)}
        over Integer Ring
  To:   Free module generated by {1, 2, 3, 4, 6, 12} over Integer Ring
>>> print(phi_VE._unicode_art_matrix())
    (1, 2)  (1, 3)  (2, 4)  (2, 6)  (3, 6) (4, 12) (6, 12)
 1⎛     -1      -1       0       0       0       0       0⎞
 2⎜      1       0      -1      -1       0       0       0⎜
 3⎜      0       1       0       0      -1       0       0⎜
 4⎜      0       0       1       0       0      -1       0⎜
 6⎜      0       0       0       1       1       0      -1⎜
12⎝      0       0       0       0       0       1       1⎠
>>> E = phi_VE.domain()
>>> P1 = E.monomial((Integer(2), Integer(4))) + E.monomial((Integer(4),␣
↪Integer(12))); P1
B[(2, 4)] + B[(4, 12)]
>>> P2 = E.monomial((Integer(2), Integer(6))) + E.monomial((Integer(6),␣
↪Integer(12))); P2
B[(2, 6)] + B[(6, 12)]
>>> phi_VE(P1 - P2)
0
```

is_biconnected(*G*, *forbidden_vertices=None*)

Check whether the graph is biconnected.

A biconnected graph is a connected graph on two or more vertices that is not broken into disconnected pieces by deleting any single vertex.

INPUT:

- `G` – a Sage graph
- `forbidden_vertices` – list (default: `None`); set of vertices to avoid during the search. This is equivalent to checking whether the graph in which the forbidden vertices have been removed is biconnected.

See also

- *is_connected()*
- *blocks_and_cut_vertices()*
- *blocks_and_cuts_tree()*
- Wikipedia article Biconnected_graph

EXAMPLES:

```
sage: G = graphs.PetersenGraph()
sage: G.is_biconnected()
True
sage: G.add_path([0,'a','b'])
sage: G.is_biconnected()
False
sage: G.add_edge('b', 1)
sage: G.is_biconnected()
True
```

```
>>> from sage.all import *
>>> G = graphs.PetersenGraph()
>>> G.is_biconnected()
True
>>> G.add_path([Integer(0),'a','b'])
>>> G.is_biconnected()
False
>>> G.add_edge('b', Integer(1))
>>> G.is_biconnected()
True
```

Check the behavior of parameter `forbidden_vertices`:

```
sage: G = graphs.CycleGraph(5)
sage: G.is_biconnected()
True
sage: G.is_biconnected(forbidden_vertices=[0])
False
```

```
>>> from sage.all import *
>>> G = graphs.CycleGraph(Integer(5))
>>> G.is_biconnected()
True
>>> G.is_biconnected(forbidden_vertices=[Integer(0)])
False
```

is_bipartite(*certificate=False*)

Check whether the graph is bipartite.

Traverse the graph G with breadth-first-search and color nodes.

INPUT:

- `certificate` – boolean (default: `False`); whether to return a certificate. If set to `True`, the certificate returned is a proper 2-coloring when G is bipartite, and an odd cycle otherwise.

EXAMPLES:

```
sage: graphs.CycleGraph(4).is_bipartite()
True
sage: graphs.CycleGraph(5).is_bipartite()
False
sage: graphs.RandomBipartite(10, 10, 0.7).is_bipartite()                    #␣
↪needs numpy
True
```

```
>>> from sage.all import *
>>> graphs.CycleGraph(Integer(4)).is_bipartite()
True
>>> graphs.CycleGraph(Integer(5)).is_bipartite()
False
>>> graphs.RandomBipartite(Integer(10), Integer(10), RealNumber('0.7')).is_
↪bipartite()                    # needs numpy
True
```

A random graph is very rarely bipartite:

```
sage: g = graphs.PetersenGraph()
sage: g.is_bipartite()
False
sage: false, oddcycle = g.is_bipartite(certificate=True)
sage: len(oddcycle) % 2
1
```

```
>>> from sage.all import *
>>> g = graphs.PetersenGraph()
>>> g.is_bipartite()
False
>>> false, oddcycle = g.is_bipartite(certificate=True)
>>> len(oddcycle) % Integer(2)
1
```

The method works identically with oriented graphs:

```
sage: g = DiGraph({0: [1, 2, 3], 2: [1], 3: [4]})
sage: g.is_bipartite()
False
sage: false, oddcycle = g.is_bipartite(certificate=True)
sage: len(oddcycle) % 2
1

sage: graphs.CycleGraph(4).random_orientation().is_bipartite()
True
sage: graphs.CycleGraph(5).random_orientation().is_bipartite()
False
```

```
>>> from sage.all import *
>>> g = DiGraph({Integer(0): [Integer(1), Integer(2), Integer(3)],␣
↪Integer(2): [Integer(1)], Integer(3): [Integer(4)]})
>>> g.is_bipartite()
False
>>> false, oddcycle = g.is_bipartite(certificate=True)
>>> len(oddcycle) % Integer(2)
1

>>> graphs.CycleGraph(Integer(4)).random_orientation().is_bipartite()
True
>>> graphs.CycleGraph(Integer(5)).random_orientation().is_bipartite()
False
```

is_cayley(*return_group=False*, *mapping=False*, *generators=False*, *allow_disconnected=False*)

Check whether the graph is a Cayley graph.

If none of the parameters are `True`, return a boolean indicating whether the graph is a Cayley graph. Otherwise, return a tuple containing said boolean and the requested data. If the graph is not a Cayley graph, each of the data will be `None`.

The empty graph is defined to be not a Cayley graph.

> **Note**
>
> For this routine to work on all graphs, the optional package `gap_packages` needs to be installed: to do so, it is enough to run `sage -i gap_packages`.

INPUT:

- `return_group` – boolean (default: `False`); if `True`, return a group for which the graph is a Cayley graph
- `mapping` – boolean (default: `False`); if `True`, return a mapping from vertices to group elements
- `generators` – boolean (default: `False`); if `True`, return the generating set of the Cayley graph
- `allow_disconnected` – boolean (default: `False`); if `True`, disconnected graphs are considered Cayley if they can be obtained from the Cayley construction with a generating set that does not generate the group.

ALGORITHM:

For connected graphs, find a regular subgroup of the automorphism group. For disconnected graphs, check that the graph is vertex-transitive and perform the check on one of its connected components. If a simple graph has density over 1/2, perform the check on its complement as its disconnectedness may increase performance.

EXAMPLES:

A Petersen Graph is not a Cayley graph:

```
sage: g = graphs.PetersenGraph()
sage: g.is_cayley()                                                         #␣
↪needs sage.groups
False
```

```
>>> from sage.all import *
>>> g = graphs.PetersenGraph()
>>> g.is_cayley()                                                           #␣
↪needs sage.groups
False
```

A Cayley digraph is a Cayley graph:

```
sage: C7 = groups.permutation.Cyclic(7)                                     #␣
↪needs sage.groups
sage: S = [(1,2,3,4,5,6,7), (1,3,5,7,2,4,6), (1,5,2,6,3,7,4)]
sage: d = C7.cayley_graph(generators=S)                                     #␣
↪needs sage.groups
sage: d.is_cayley()                                                         #␣
```

(continues on next page)

(continued from previous page)

```
↪needs sage.groups
True
```

```
>>> from sage.all import *
>>> C7 = groups.permutation.Cyclic(Integer(7))                              ␣
↪     # needs sage.groups
>>> S = [(Integer(1),Integer(2),Integer(3),Integer(4),Integer(5),Integer(6),
↪Integer(7)), (Integer(1),Integer(3),Integer(5),Integer(7),Integer(2),
↪Integer(4),Integer(6)), (Integer(1),Integer(5),Integer(2),Integer(6),
↪Integer(3),Integer(7),Integer(4))]
>>> d = C7.cayley_graph(generators=S)                                       #␣
↪needs sage.groups
>>> d.is_cayley()                                                          #␣
↪needs sage.groups
True
```

Graphs with loops and multiedges will have identity and repeated elements, respectively, among the generators:

```
sage: g = Graph(graphs.PaleyGraph(9), loops=True, multiedges=True)
sage: g.add_edges([(u, u) for u in g])
sage: g.add_edges([(u, u+1) for u in g])
sage: _, S = g.is_cayley(generators=True)                                  #␣
↪needs sage.groups
sage: S  # random                                                          #␣
↪needs sage.groups
[(),
 (0,2,1)(a,a + 2,a + 1)(2*a,2*a + 2,2*a + 1),
 (0,2,1)(a,a + 2,a + 1)(2*a,2*a + 2,2*a + 1),
 (0,1,2)(a,a + 1,a + 2)(2*a,2*a + 1,2*a + 2),
 (0,1,2)(a,a + 1,a + 2)(2*a,2*a + 1,2*a + 2),
 (0,2*a + 2,a + 1)(1,2*a,a + 2)(2,2*a + 1,a),
 (0,a + 1,2*a + 2)(1,a + 2,2*a)(2,a,2*a + 1)]
```

```
>>> from sage.all import *
>>> g = Graph(graphs.PaleyGraph(Integer(9)), loops=True, multiedges=True)
>>> g.add_edges([(u, u) for u in g])
>>> g.add_edges([(u, u+Integer(1)) for u in g])
>>> _, S = g.is_cayley(generators=True)                                     #␣
↪needs sage.groups
>>> S  # random                                                            #␣
↪needs sage.groups
[(),
 (0,2,1)(a,a + 2,a + 1)(2*a,2*a + 2,2*a + 1),
 (0,2,1)(a,a + 2,a + 1)(2*a,2*a + 2,2*a + 1),
 (0,1,2)(a,a + 1,a + 2)(2*a,2*a + 1,2*a + 2),
 (0,1,2)(a,a + 1,a + 2)(2*a,2*a + 1,2*a + 2),
 (0,2*a + 2,a + 1)(1,2*a,a + 2)(2,2*a + 1,a),
 (0,a + 1,2*a + 2)(1,a + 2,2*a)(2,a,2*a + 1)]
```

is_chordal(*certificate=False*, *algorithm='B'*)

Check whether the given graph is chordal.

A Graph G is said to be chordal if it contains no induced hole (a cycle of length at least 4).

Alternatively, chordality can be defined using a Perfect Elimination Order:

A Perfect Elimination Order of a graph G is an ordering $v_1, ..., v_n$ of its vertex set such that for all i, the neighbors of v_i whose index is greater that i induce a complete subgraph in G. Hence, the graph G can be totally erased by successively removing vertices whose neighborhood is a clique (also called *simplicial* vertices) [FG1965].

(It can be seen that if G contains an induced hole, then it cannot have a perfect elimination order. Indeed, if we write $h_1, ..., h_k$ the k vertices of such a hole, then the first of those vertices to be removed would have two non-adjacent neighbors in the graph.)

A Graph is then chordal if and only if it has a Perfect Elimination Order.

INPUT:

- `certificate` – boolean (default: `False`); whether to return a certificate
  - If `certificate = False` (default), returns `True` or `False` accordingly.
  - If `certificate = True`, returns :
    * `(True, peo)` when the graph is chordal, where `peo` is a perfect elimination order of its vertices.
    * `(False, Hole)` when the graph is not chordal, where `Hole` (a `Graph` object) is an induced subgraph of `self` isomorphic to a hole.
- `algorithm` – string (default: `'B'`); the algorithm to choose among `'A'` or `'B'`. While they will agree on whether the given graph is chordal, they cannot be expected to return the same certificates.

ALGORITHM:

This method implements the algorithm proposed in [RT1975] for the recognition of chordal graphs. The time complexity of this algorithm is $O(n+m)$ for `SparseGraph` and $O(n^2)$ for `DenseGraph`. The algorithm works through computing a Lex BFS on the graph, then checking whether the order is a Perfect Elimination Order by computing for each vertex v the subgraph induced by its non-deleted neighbors, then testing whether this graph is complete.

EXAMPLES:

The lexicographic product of a Path and a Complete Graph is chordal

```
sage: g = graphs.PathGraph(5).lexicographic_product(graphs.CompleteGraph(3))
sage: g.is_chordal()
True
```

```
>>> from sage.all import *
>>> g = graphs.PathGraph(Integer(5)).lexicographic_product(graphs.
↪CompleteGraph(Integer(3)))
>>> g.is_chordal()
True
```

The same goes with the product of a random lobster (which is a tree) and a Complete Graph

```
sage: grl = graphs.RandomLobster(10, .5, .5)                                    #␣
↪needs networkx
sage: g = grl.lexicographic_product(graphs.CompleteGraph(3))                    #␣
↪needs networkx
sage: g.is_chordal()                                                            #␣
```

(continues on next page)

(continued from previous page)

```
↪needs networkx
True
```

```
>>> from sage.all import *
>>> grl = graphs.RandomLobster(Integer(10), RealNumber('.5'), RealNumber('.5
↪'))                                    # needs networkx
>>> g = grl.lexicographic_product(graphs.CompleteGraph(Integer(3)))          ␣
↪     # needs networkx
>>> g.is_chordal()                                                          #␣
↪needs networkx
True
```

The disjoint union of chordal graphs is still chordal:

```
sage: (2 * g).is_chordal()                                                  #␣
↪needs networkx
True
```

```
>>> from sage.all import *
>>> (Integer(2) * g).is_chordal()                                            ␣
↪     # needs networkx
True
```

Let us check the certificate given by Sage is indeed a perfect elimination order:

```
sage: _, peo = g.is_chordal(certificate=True)                               #␣
↪needs networkx
sage: for v in peo:                                                         #␣
↪needs networkx
....:     if not g.subgraph(g.neighbors(v)).is_clique():
....:         raise ValueError("this should never happen")
....:     g.delete_vertex(v)
```

```
>>> from sage.all import *
>>> _, peo = g.is_chordal(certificate=True)                                 #␣
↪needs networkx
>>> for v in peo:                                                           #␣
↪needs networkx
...     if not g.subgraph(g.neighbors(v)).is_clique():
...         raise ValueError("this should never happen")
...     g.delete_vertex(v)
```

Of course, the Petersen Graph is not chordal as it has girth 5:

```
sage: g = graphs.PetersenGraph()
sage: g.girth()
5
sage: g.is_chordal()
False
```

```
>>> from sage.all import *
>>> g = graphs.PetersenGraph()
```

(continues on next page)

(continued from previous page)

```
>>> g.girth()
5
>>> g.is_chordal()
False
```

We can even obtain such a cycle as a certificate:

```
sage: _, hole = g.is_chordal(certificate=True)
sage: hole
Subgraph of (Petersen graph): Graph on 5 vertices
sage: hole.is_isomorphic(graphs.CycleGraph(5))
True
```

```
>>> from sage.all import *
>>> _, hole = g.is_chordal(certificate=True)
>>> hole
Subgraph of (Petersen graph): Graph on 5 vertices
>>> hole.is_isomorphic(graphs.CycleGraph(Integer(5)))
True
```

is_circulant(*certificate=False*)

Check whether the graph is circulant.

For more information, see Wikipedia article Circulant_graph.

INPUT:

- `certificate` – boolean (default: `False`); whether to return a certificate for yes-answers (see OUTPUT section)

OUTPUT:

When `certificate` is set to `False` (default) this method only returns `True` or `False` answers. When `certificate` is set to `True`, the method either returns `(False, None)` or `(True, lists_of_parameters)` each element of `lists_of_parameters` can be used to define the graph as a circulant graph.

See the documentation of *`CirculantGraph()`* and *`Circulant()`* for more information, and the examples below.

See also

`CirculantGraph()` – a constructor for circulant graphs.

EXAMPLES:

The Petersen graph is not a circulant graph:

```
sage: g = graphs.PetersenGraph()
sage: g.is_circulant()                                                      #␣
↪needs sage.groups
False
```

```
>>> from sage.all import *
>>> g = graphs.PetersenGraph()
```

(continues on next page)

(continued from previous page)

```
>>> g.is_circulant()                                                    #␣
↪needs sage.groups
False
```

A cycle is obviously a circulant graph, but several sets of parameters can be used to define it:

```
sage: g = graphs.CycleGraph(5)
sage: g.is_circulant(certificate=True)                                  #␣
↪needs sage.groups
(True, [(5, [1, 4]), (5, [2, 3])])
```

```
>>> from sage.all import *
>>> g = graphs.CycleGraph(Integer(5))
>>> g.is_circulant(certificate=True)                                    #␣
↪needs sage.groups
(True, [(5, [1, 4]), (5, [2, 3])])
```

The same goes for directed graphs:

```
sage: g = digraphs.Circuit(5)
sage: g.is_circulant(certificate=True)                                  #␣
↪needs sage.groups
(True, [(5, [1]), (5, [3]), (5, [2]), (5, [4])])
```

```
>>> from sage.all import *
>>> g = digraphs.Circuit(Integer(5))
>>> g.is_circulant(certificate=True)                                    #␣
↪needs sage.groups
(True, [(5, [1]), (5, [3]), (5, [2]), (5, [4])])
```

With this information, it is very easy to create (and plot) all possible drawings of a circulant graph:

```
sage: g = graphs.CirculantGraph(13, [2, 3, 10, 11])
sage: for param in g.is_circulant(certificate=True)[1]:                 #␣
↪needs sage.groups
....:     graphs.CirculantGraph(*param)
Circulant graph ([2, 3, 10, 11]): Graph on 13 vertices
Circulant graph ([1, 5, 8, 12]): Graph on 13 vertices
Circulant graph ([4, 6, 7, 9]): Graph on 13 vertices
```

```
>>> from sage.all import *
>>> g = graphs.CirculantGraph(Integer(13), [Integer(2), Integer(3),␣
↪Integer(10), Integer(11)])
>>> for param in g.is_circulant(certificate=True)[Integer(1)]:          ␣
↪     # needs sage.groups
...     graphs.CirculantGraph(*param)
Circulant graph ([2, 3, 10, 11]): Graph on 13 vertices
Circulant graph ([1, 5, 8, 12]): Graph on 13 vertices
Circulant graph ([4, 6, 7, 9]): Graph on 13 vertices
```

is_circular_planar(*on_embedding=None*, *kuratowski=False*, *set_embedding=True*, *boundary=None*, *ordered=False*, *set_pos=False*)

Check whether the graph is circular planar (outerplanar).

A graph is circular planar if it has a planar embedding in which all vertices can be drawn in order on a circle. This method can also be used to check the existence of a planar embedding in which the vertices of a specific set (the *boundary*) can be drawn on a circle, all other vertices being drawn inside of the circle. An order can be defined on the vertices of the boundary in order to define how they are to appear on the circle.

INPUT:

- `on_embedding` – dictionary (default: `None`); the embedding dictionary to test planarity on (i.e.: will return `True` or `False` only for the given embedding)
- `kuratowski` – boolean (default: `False`); whether to return a tuple with boolean first entry and the Kuratowski subgraph (i.e. an edge subdivision of K_5 or $K_{3,3}$) as the second entry (see OUTPUT below)
- `set_embedding` – boolean (default: `True`); whether or not to set the instance field variable that contains a combinatorial embedding (clockwise ordering of neighbors at each vertex). This value will only be set if a circular planar embedding is found. It is stored as a Python dict: `v1: [n1,n2,n3]` where `v1` is a vertex and `n1,n2,n3` are its neighbors.
- `boundary` – list (default: `None`); an ordered list of vertices that are required to be drawn on the circle, all others being drawn inside of it. It is set to `None` by default, meaning that *all* vertices should be drawn on the boundary.
- `ordered` – boolean (default: `False`); whether or not to consider the order of the boundary. It required `boundary` to be defined.
- `set_pos` – boolean (default: `False`); whether or not to set the position dictionary (for plotting) to reflect the combinatorial embedding. Note that this value will default to `False` if `set_embedding` is set to `False`. Also, the position dictionary will only be updated if a circular planar embedding is found.

OUTPUT:

The method returns `True` if the graph is circular planar, and `False` if it is not.

If `kuratowski` is set to `True`, then this function will return a tuple, whose first entry is a boolean and whose second entry is the Kuratowski subgraph (i.e. an edge subdivision of K_5 or $K_{3,3}$) isolated by the Boyer-Myrvold algorithm. Note that this graph might contain a vertex or edges that were not in the initial graph. These would be elements referred to below as parts of the wheel and the star, which were added to the graph to require that the boundary can be drawn on the boundary of a disc, with all other vertices drawn inside (and no edge crossings).

ALGORITHM:

This is a linear time algorithm to test for circular planarity. It relies on the edge-addition planarity algorithm due to Boyer-Myrvold. We accomplish linear time for circular planarity by modifying the graph before running the general planarity algorithm.

REFERENCE:

[BM2004]

EXAMPLES:

```
sage: g439 = Graph({1: [5, 7], 2: [5, 6], 3: [6, 7], 4: [5, 6, 7]})
sage: g439.show()                                                          #␣
↪needs sage.plot
sage: g439.is_circular_planar(boundary=[1, 2, 3, 4])
False
sage: g439.is_circular_planar(kuratowski=True, boundary=[1, 2, 3, 4])
(False, Kuratowski subgraph of (): Graph on 8 vertices)
sage: g439.is_circular_planar(kuratowski=True, boundary=[1, 2, 3])
(True, None)
```

(continues on next page)

(continued from previous page)

```
sage: g439.get_embedding()
{1: [5, 7],
 2: [6, 5],
 3: [7, 6],
 4: [5, 6, 7],
 5: [2, 4, 1],
 6: [3, 4, 2],
 7: [1, 4, 3]}
```

```
>>> from sage.all import *
>>> g439 = Graph({Integer(1): [Integer(5), Integer(7)], Integer(2):␣
↪[Integer(5), Integer(6)], Integer(3): [Integer(6), Integer(7)], Integer(4):␣
↪[Integer(5), Integer(6), Integer(7)]})
>>> g439.show()                                                              #␣
↪needs sage.plot
>>> g439.is_circular_planar(boundary=[Integer(1), Integer(2), Integer(3),␣
↪Integer(4)])
False
>>> g439.is_circular_planar(kuratowski=True, boundary=[Integer(1), Integer(2),
↪ Integer(3), Integer(4)])
(False, Kuratowski subgraph of (): Graph on 8 vertices)
>>> g439.is_circular_planar(kuratowski=True, boundary=[Integer(1), Integer(2),
↪ Integer(3)])
(True, None)
>>> g439.get_embedding()
{1: [5, 7],
 2: [6, 5],
 3: [7, 6],
 4: [5, 6, 7],
 5: [2, 4, 1],
 6: [3, 4, 2],
 7: [1, 4, 3]}
```

Order matters:

```
sage: K23 = graphs.CompleteBipartiteGraph(2, 3)
sage: K23.is_circular_planar(boundary=[0, 1, 2, 3])
True
sage: K23.is_circular_planar(ordered=True, boundary=[0, 1, 2, 3])
False
```

```
>>> from sage.all import *
>>> K23 = graphs.CompleteBipartiteGraph(Integer(2), Integer(3))
>>> K23.is_circular_planar(boundary=[Integer(0), Integer(1), Integer(2),␣
↪Integer(3)])
True
>>> K23.is_circular_planar(ordered=True, boundary=[Integer(0), Integer(1),␣
↪Integer(2), Integer(3)])
False
```

With a different order:

```
sage: K23.is_circular_planar(set_embedding=True, boundary=[0, 2, 1, 3])
True
```

```
>>> from sage.all import *
>>> K23.is_circular_planar(set_embedding=True, boundary=[Integer(0),␣
↪Integer(2), Integer(1), Integer(3)])
True
```

is_clique(*vertices=None*, *directed_clique=False*, *induced=True*, *loops=False*)

Check whether a set of vertices is a clique.

A clique is a set of vertices such that there is exactly one edge between any two vertices.

INPUT:

- `vertices` – a single vertex or an iterable container of vertices (default: `None`); when set, check whether the set of vertices is a clique, otherwise check whether `self` is a clique
- `directed_clique` – boolean (default: `False`); if set to `False`, only consider the underlying undirected graph. If set to `True` and the graph is directed, only return `True` if all possible edges in _both_ directions exist.
- `induced` – boolean (default: `True`); if set to `True`, check that the graph has exactly one edge between any two vertices. If set to `False`, check that the graph has at least one edge between any two vertices.
- `loops` – boolean (default: `False`); if set to `True`, check that each vertex of the graph has a loop, and exactly one if furthermore `induced == True`. If set to `False`, check that the graph has no loop when `induced == True`, and ignore loops otherwise.

EXAMPLES:

```
sage: g = graphs.CompleteGraph(4)
sage: g.is_clique([1, 2, 3])
True
sage: g.is_clique()
True
sage: h = graphs.CycleGraph(4)
sage: h.is_clique([1, 2])
True
sage: h.is_clique([1, 2, 3])
False
sage: h.is_clique()
False
sage: i = digraphs.Complete(4)
sage: i.delete_edge([0, 1])
sage: i.is_clique(directed_clique=False, induced=True)
False
sage: i.is_clique(directed_clique=False, induced=False)
True
sage: i.is_clique(directed_clique=True)
False
```

```
>>> from sage.all import *
>>> g = graphs.CompleteGraph(Integer(4))
>>> g.is_clique([Integer(1), Integer(2), Integer(3)])
```

(continues on next page)

(continued from previous page)

```
True
>>> g.is_clique()
True
>>> h = graphs.CycleGraph(Integer(4))
>>> h.is_clique([Integer(1), Integer(2)])
True
>>> h.is_clique([Integer(1), Integer(2), Integer(3)])
False
>>> h.is_clique()
False
>>> i = digraphs.Complete(Integer(4))
>>> i.delete_edge([Integer(0), Integer(1)])
>>> i.is_clique(directed_clique=False, induced=True)
False
>>> i.is_clique(directed_clique=False, induced=False)
True
>>> i.is_clique(directed_clique=True)
False
```

is_connected(*G*, *forbidden_vertices=None*)

Check whether the (di)graph is connected.

Note that in a graph, path connected is equivalent to connected.

INPUT:

- `G` – the input graph
- `forbidden_vertices` – list (default: `None`); set of vertices to avoid during the search

See also

- *is_biconnected()*

EXAMPLES:

```
sage: from sage.graphs.connectivity import is_connected
sage: G = Graph({0: [1, 2], 1: [2], 3: [4, 5], 4: [5]})
sage: is_connected(G)
False
sage: G.is_connected()
False
sage: G.add_edge(0,3)
sage: is_connected(G)
True
sage: is_connected(G, forbidden_vertices=[3])
False
sage: is_connected(G, forbidden_vertices=[1])
True
sage: D = DiGraph({0: [1, 2], 1: [2], 3: [4, 5], 4: [5]})
sage: is_connected(D)
False
sage: D.add_edge(0, 3)
```

(continues on next page)

(continued from previous page)

```
sage: is_connected(D)
True
sage: D = DiGraph({1: [0], 2: [0]})
sage: is_connected(D)
True
```

```
>>> from sage.all import *
>>> from sage.graphs.connectivity import is_connected
>>> G = Graph({Integer(0): [Integer(1), Integer(2)], Integer(1): [Integer(2)],
↪ Integer(3): [Integer(4), Integer(5)], Integer(4): [Integer(5)]})
>>> is_connected(G)
False
>>> G.is_connected()
False
>>> G.add_edge(Integer(0),Integer(3))
>>> is_connected(G)
True
>>> is_connected(G, forbidden_vertices=[Integer(3)])
False
>>> is_connected(G, forbidden_vertices=[Integer(1)])
True
>>> D = DiGraph({Integer(0): [Integer(1), Integer(2)], Integer(1):␣
↪[Integer(2)], Integer(3): [Integer(4), Integer(5)], Integer(4):␣
↪[Integer(5)]})
>>> is_connected(D)
False
>>> D.add_edge(Integer(0), Integer(3))
>>> is_connected(D)
True
>>> D = DiGraph({Integer(1): [Integer(0)], Integer(2): [Integer(0)]})
>>> is_connected(D)
True
```

is_cut_edge(*G*, *u*, *v=None*, *label=None*)

Check whether the edge `(u, v)` is a cut-edge or a bridge of graph `G`.

A cut edge (or bridge) is an edge that when removed increases the number of connected components. This function works with simple graphs as well as graphs with loops and multiedges. In a digraph, a cut edge is an edge that when removed increases the number of (weakly) connected components.

INPUT: The following forms are accepted

- is_cut_edge(G, 1, 2)
- is_cut_edge(G, (1, 2))
- is_cut_edge(G, 1, 2, 'label')
- is_cut_edge(G, (1, 2, 'label'))

OUTPUT:

- Returns `True` if (u,v) is a cut edge, False otherwise

EXAMPLES:

```
sage: from sage.graphs.connectivity import is_cut_edge
sage: G = graphs.CompleteGraph(4)
sage: is_cut_edge(G,0,2)
False
sage: G.is_cut_edge(0,2)
False

sage: G = graphs.CompleteGraph(4)
sage: G.add_edge((0,5,'silly'))
sage: is_cut_edge(G,(0,5,'silly'))
True

sage: G = Graph([[0,1],[0,2],[3,4],[4,5],[3,5]])
sage: is_cut_edge(G,(0,1))
True

sage: G = Graph([[0,1],[0,2],[1,1]], loops = True)
sage: is_cut_edge(G,(1,1))
False

sage: G = digraphs.Circuit(5)
sage: is_cut_edge(G,(0,1))
False

sage: G = graphs.CompleteGraph(6)
sage: is_cut_edge(G,(0,7))
Traceback (most recent call last):
...
ValueError: edge not in graph
```

```
>>> from sage.all import *
>>> from sage.graphs.connectivity import is_cut_edge
>>> G = graphs.CompleteGraph(Integer(4))
>>> is_cut_edge(G,Integer(0),Integer(2))
False
>>> G.is_cut_edge(Integer(0),Integer(2))
False

>>> G = graphs.CompleteGraph(Integer(4))
>>> G.add_edge((Integer(0),Integer(5),'silly'))
>>> is_cut_edge(G,(Integer(0),Integer(5),'silly'))
True

>>> G = Graph([[Integer(0),Integer(1)],[Integer(0),Integer(2)],[Integer(3),
↪Integer(4)],[Integer(4),Integer(5)],[Integer(3),Integer(5)]])
>>> is_cut_edge(G,(Integer(0),Integer(1)))
True

>>> G = Graph([[Integer(0),Integer(1)],[Integer(0),Integer(2)],[Integer(1),
↪Integer(1)]], loops = True)
>>> is_cut_edge(G,(Integer(1),Integer(1)))
False
```

(continues on next page)

(continued from previous page)

```
>>> G = digraphs.Circuit(Integer(5))
>>> is_cut_edge(G,(Integer(0),Integer(1)))
False

>>> G = graphs.CompleteGraph(Integer(6))
>>> is_cut_edge(G,(Integer(0),Integer(7)))
Traceback (most recent call last):
...
ValueError: edge not in graph
```

is_cut_vertex(*G*, *u*, *weak=False*)

Check whether the input vertex is a cut-vertex.

A vertex is a cut-vertex if its removal from the (di)graph increases the number of (strongly) connected components. Isolated vertices or leaves are not cut-vertices. This function works with simple graphs as well as graphs with loops and multiple edges.

INPUT:

- `G` – a Sage (Di)Graph
- `u` – a vertex
- `weak` – boolean (default: `False`); whether the connectivity of directed graphs is to be taken in the weak sense, that is ignoring edges orientations

OUTPUT:

Return `True` if `u` is a cut-vertex, and `False` otherwise.

EXAMPLES:

Giving a LollipopGraph(4,2), that is a complete graph with 4 vertices with a pending edge:

```
sage: from sage.graphs.connectivity import is_cut_vertex
sage: G = graphs.LollipopGraph(4, 2)
sage: is_cut_vertex(G, 0)
False
sage: is_cut_vertex(G, 3)
True
sage: G.is_cut_vertex(3)
True
```

```
>>> from sage.all import *
>>> from sage.graphs.connectivity import is_cut_vertex
>>> G = graphs.LollipopGraph(Integer(4), Integer(2))
>>> is_cut_vertex(G, Integer(0))
False
>>> is_cut_vertex(G, Integer(3))
True
>>> G.is_cut_vertex(Integer(3))
True
```

Comparing the weak and strong connectivity of a digraph:

```
sage: D = digraphs.Circuit(6)
sage: D.is_strongly_connected()
True
sage: is_cut_vertex(D, 2)
True
sage: is_cut_vertex(D, 2, weak=True)
False
```

```
>>> from sage.all import *
>>> D = digraphs.Circuit(Integer(6))
>>> D.is_strongly_connected()
True
>>> is_cut_vertex(D, Integer(2))
True
>>> is_cut_vertex(D, Integer(2), weak=True)
False
```

Giving a vertex that is not in the graph:

```
sage: G = graphs.CompleteGraph(4)
sage: is_cut_vertex(G, 7)
Traceback (most recent call last):
...
ValueError: vertex (7) is not a vertex of the graph
```

```
>>> from sage.all import *
>>> G = graphs.CompleteGraph(Integer(4))
>>> is_cut_vertex(G, Integer(7))
Traceback (most recent call last):
...
ValueError: vertex (7) is not a vertex of the graph
```

is_cycle(*directed_cycle=True*)

Check whether `self` is a (directed) cycle graph.

We follow the definition provided in [BM2008] for undirected graphs. A cycle on three or more vertices is a simple graph whose vertices can be arranged in a cyclic order so that two vertices are adjacent if they are consecutive in the order, and not adjacent otherwise. A cycle on a vertex consists of a single vertex provided with a loop and a cycle with two vertices consists of two vertices connected by a pair of parallel edges. In other words, an undirected graph is a cycle if it is 2-regular and connected. The empty graph is not a cycle.

For directed graphs, a directed cycle, or circuit, on two or more vertices is a strongly connected directed graph without loops nor multiple edges with has many arcs as vertices. A circuit on a vertex consists of a single vertex provided with a loop.

INPUT:

- `directed_cycle` – boolean (default: `True`); if set to `True` and the graph is directed, only return `True` if `self` is a directed cycle graph (i.e., a circuit). If set to `False`, we ignore the direction of edges and so opposite arcs become multiple (parallel) edges. This parameter is ignored for undirected graphs.

EXAMPLES:

```
sage: G = graphs.PetersenGraph()
sage: G.is_cycle()
```

(continues on next page)

(continued from previous page)

```
False
sage: graphs.CycleGraph(5).is_cycle()
True
sage: Graph([(0,1 )]).is_cycle()
False
sage: Graph([(0, 1), (0, 1)], multiedges=True).is_cycle()
True
sage: Graph([(0, 1), (0, 1), (0, 1)], multiedges=True).is_cycle()
False
sage: Graph().is_cycle()
False
sage: G = Graph([(0, 0)], loops=True)
sage: G.is_cycle()
True
sage: digraphs.Circuit(3).is_cycle()
True
sage: digraphs.Circuit(2).is_cycle()
True
sage: digraphs.Circuit(2).is_cycle(directed_cycle=False)
True
sage: D = DiGraph(graphs.CycleGraph(3))
sage: D.is_cycle()
False
sage: D.is_cycle(directed_cycle=False)
False
sage: D.edges(sort=True, labels=False)
[(0, 1), (0, 2), (1, 0), (1, 2), (2, 0), (2, 1)]
```

```
>>> from sage.all import *
>>> G = graphs.PetersenGraph()
>>> G.is_cycle()
False
>>> graphs.CycleGraph(Integer(5)).is_cycle()
True
>>> Graph([(Integer(0),Integer(1) )]).is_cycle()
False
>>> Graph([(Integer(0), Integer(1)), (Integer(0), Integer(1))],␣
↪multiedges=True).is_cycle()
True
>>> Graph([(Integer(0), Integer(1)), (Integer(0), Integer(1)), (Integer(0),␣
↪Integer(1))], multiedges=True).is_cycle()
False
>>> Graph().is_cycle()
False
>>> G = Graph([(Integer(0), Integer(0))], loops=True)
>>> G.is_cycle()
True
>>> digraphs.Circuit(Integer(3)).is_cycle()
True
>>> digraphs.Circuit(Integer(2)).is_cycle()
True
>>> digraphs.Circuit(Integer(2)).is_cycle(directed_cycle=False)
```

(continues on next page)

(continued from previous page)

```
True
>>> D = DiGraph(graphs.CycleGraph(Integer(3)))
>>> D.is_cycle()
False
>>> D.is_cycle(directed_cycle=False)
False
>>> D.edges(sort=True, labels=False)
[(0, 1), (0, 2), (1, 0), (1, 2), (2, 0), (2, 1)]
```

is_drawn_free_of_edge_crossings()

Check whether the position dictionary for this graph is set and that position dictionary gives a planar embedding.

This simply checks all pairs of edges that don't share a vertex to make sure that they don't intersect.

Note

This function require that `_pos` attribute is set (Returns False otherwise)

EXAMPLES:

```
sage: D = graphs.DodecahedralGraph()
sage: pos = D.layout(layout='planar', save_pos=True)
sage: D.is_drawn_free_of_edge_crossings()
True
```

```
>>> from sage.all import *
>>> D = graphs.DodecahedralGraph()
>>> pos = D.layout(layout='planar', save_pos=True)
>>> D.is_drawn_free_of_edge_crossings()
True
```

is_edge_cut(*G*, *edges*)

Check whether `edges` form an edge cut.

A set of edges is an edge cut of a graph if its removal increases the number of connected components. In a digraph, we consider the number of (weakly) connected components.

This method is not working for (di)graphs with multiple edges. Furthermore, edge labels are ignored.

INPUT:

- `G` – a (di)graph
- `edges` – a set of edges

EXAMPLES:

A cycle graph of order 4:

```
sage: from sage.graphs.connectivity import is_edge_cut
sage: G = graphs.CycleGraph(4)
sage: is_edge_cut(G, [(1, 2)])
False
sage: is_edge_cut(G, [(1, 2), (2, 3)])
```

(continues on next page)

(continued from previous page)

```
True
sage: is_edge_cut(G, [(1, 2), (3, 0)])
True
```

```
>>> from sage.all import *
>>> from sage.graphs.connectivity import is_edge_cut
>>> G = graphs.CycleGraph(Integer(4))
>>> is_edge_cut(G, [(Integer(1), Integer(2))])
False
>>> is_edge_cut(G, [(Integer(1), Integer(2)), (Integer(2), Integer(3))])
True
>>> is_edge_cut(G, [(Integer(1), Integer(2)), (Integer(3), Integer(0))])
True
```

A pending edge is a cut-edge:

```
sage: G.add_edge((0, 5, 'silly'))
sage: is_edge_cut(G, [(0, 5, 'silly')])
True
```

```
>>> from sage.all import *
>>> G.add_edge((Integer(0), Integer(5), 'silly'))
>>> is_edge_cut(G, [(Integer(0), Integer(5), 'silly')])
True
```

Edge labels are ignored, even if specified:

```
sage: G.add_edge((2, 5, 'xyz'))
sage: is_edge_cut(G, [(0, 5), (2, 5)])
True
sage: is_edge_cut(G, [(0, 5), (2, 5, 'xyz')])
True
sage: is_edge_cut(G, [(0, 5, 'silly'), (2, 5)])
True
sage: is_edge_cut(G, [(0, 5, 'aa'), (2, 5, 'bb')])
True
```

```
>>> from sage.all import *
>>> G.add_edge((Integer(2), Integer(5), 'xyz'))
>>> is_edge_cut(G, [(Integer(0), Integer(5)), (Integer(2), Integer(5))])
True
>>> is_edge_cut(G, [(Integer(0), Integer(5)), (Integer(2), Integer(5), 'xyz
↪')])
True
>>> is_edge_cut(G, [(Integer(0), Integer(5), 'silly'), (Integer(2),␣
↪Integer(5))])
True
>>> is_edge_cut(G, [(Integer(0), Integer(5), 'aa'), (Integer(2), Integer(5),
↪'bb')])
True
```

The graph can have loops:

```
sage: G.allow_loops(True)
sage: G.add_edge(0, 0)
sage: is_edge_cut(G, [(0, 5), (2, 5)])
True
sage: is_edge_cut(G, [(0, 0), (0, 5), (2, 5)])
True
```

```
>>> from sage.all import *
>>> G.allow_loops(True)
>>> G.add_edge(Integer(0), Integer(0))
>>> is_edge_cut(G, [(Integer(0), Integer(5)), (Integer(2), Integer(5))])
True
>>> is_edge_cut(G, [(Integer(0), Integer(0)), (Integer(0), Integer(5)),␣
↪(Integer(2), Integer(5))])
True
```

Multiple edges are not allowed:

```
sage: G.allow_multiple_edges(True)
sage: is_edge_cut(G, [(0, 5), (2, 5)])
Traceback (most recent call last):
...
ValueError: This method is not known to work on graphs with
 multiedges. Perhaps this method can be updated to handle them, but in
 the meantime if you want to use it please disallow multiedges using
 allow_multiple_edges().
```

```
>>> from sage.all import *
>>> G.allow_multiple_edges(True)
>>> is_edge_cut(G, [(Integer(0), Integer(5)), (Integer(2), Integer(5))])
Traceback (most recent call last):
...
ValueError: This method is not known to work on graphs with
 multiedges. Perhaps this method can be updated to handle them, but in
 the meantime if you want to use it please disallow multiedges using
 allow_multiple_edges().
```

An error is raised if an element of `edges` is not an edge of G:

```
sage: G = graphs.CycleGraph(4)
sage: is_edge_cut(G, [(0, 2)])
Traceback (most recent call last):
...
ValueError: edge (0, 2) is not an edge of the graph
```

```
>>> from sage.all import *
>>> G = graphs.CycleGraph(Integer(4))
>>> is_edge_cut(G, [(Integer(0), Integer(2))])
Traceback (most recent call last):
...
ValueError: edge (0, 2) is not an edge of the graph
```

For digraphs, this method considers the number of (weakly) connected components:

```
sage: G = digraphs.Circuit(4)
sage: is_edge_cut(G, [(0, 1)])
False
sage: G = digraphs.Circuit(4)
sage: is_edge_cut(G, [(0, 1), (1, 2)])
True
```

```
>>> from sage.all import *
>>> G = digraphs.Circuit(Integer(4))
>>> is_edge_cut(G, [(Integer(0), Integer(1))])
False
>>> G = digraphs.Circuit(Integer(4))
>>> is_edge_cut(G, [(Integer(0), Integer(1)), (Integer(1), Integer(2))])
True
```

For disconnected (di)graphs, the method checks if the number of (weakly) connected components increases:

```
sage: G = graphs.CycleGraph(4) * 2
sage: is_edge_cut(G, [(1, 2), (2, 3)])
True
sage: G = digraphs.Circuit(4) * 2
sage: is_edge_cut(G, [(0, 1), (1, 2)])
True
```

```
>>> from sage.all import *
>>> G = graphs.CycleGraph(Integer(4)) * Integer(2)
>>> is_edge_cut(G, [(Integer(1), Integer(2)), (Integer(2), Integer(3))])
True
>>> G = digraphs.Circuit(Integer(4)) * Integer(2)
>>> is_edge_cut(G, [(Integer(0), Integer(1)), (Integer(1), Integer(2))])
True
```

is_equitable(*partition*, *quotient_matrix=False*)

Check whether the given partition is equitable with respect to `self`.

A partition is equitable with respect to a graph if for every pair of cells C1, C2 of the partition, the number of edges from a vertex of C1 to C2 is the same, over all vertices in C1.

INPUT:

- `partition` – list of lists
- `quotient_matrix` – boolean (default: `False`); if `True`, and the partition is equitable, returns a matrix over the integers whose rows and columns represent cells of the partition, and whose i,j entry is the number of vertices in cell j adjacent to each vertex in cell i (since the partition is equitable, this is well defined)

EXAMPLES:

```
sage: G = graphs.PetersenGraph()
sage: G.is_equitable([[0,4],[1,3,5,9],[2,6,8],[7]])
False
sage: G.is_equitable([[0,4],[1,3,5,9],[2,6,8,7]])
True
sage: G.is_equitable([[0,4],[1,3,5,9],[2,6,8,7]], quotient_matrix=True)      #␣
```

(continues on next page)

(continued from previous page)

```
↪needs sage.modules
[1 2 0]
[1 0 2]
[0 2 1]
```

```
>>> from sage.all import *
>>> G = graphs.PetersenGraph()
>>> G.is_equitable([[Integer(0),Integer(4)],[Integer(1),Integer(3),Integer(5),
↪Integer(9)],[Integer(2),Integer(6),Integer(8)],[Integer(7)]])
False
>>> G.is_equitable([[Integer(0),Integer(4)],[Integer(1),Integer(3),Integer(5),
↪Integer(9)],[Integer(2),Integer(6),Integer(8),Integer(7)]])
True
>>> G.is_equitable([[Integer(0),Integer(4)],[Integer(1),Integer(3),Integer(5),
↪Integer(9)],[Integer(2),Integer(6),Integer(8),Integer(7)]], quotient_
↪matrix=True)      # needs sage.modules
[1 2 0]
[1 0 2]
[0 2 1]
```

```
sage: ss = (graphs.WheelGraph(6)).line_graph(labels=False)
sage: prt = [[(0, 1)], [(0, 2), (0, 3), (0, 4), (1, 2), (1, 4)], [(2, 3), (3,␣
↪4)]]
```

```
>>> from sage.all import *
>>> ss = (graphs.WheelGraph(Integer(6))).line_graph(labels=False)
>>> prt = [[(Integer(0), Integer(1))], [(Integer(0), Integer(2)), (Integer(0),
↪ Integer(3)), (Integer(0), Integer(4)), (Integer(1), Integer(2)),␣
↪(Integer(1), Integer(4))], [(Integer(2), Integer(3)), (Integer(3),␣
↪Integer(4))]]
```

```
sage: ss.is_equitable(prt)
Traceback (most recent call last):
...
TypeError: Partition ([[(0, 1)], [(0, 2), (0, 3), (0, 4), (1, 2), (1, 4)],
[(2, 3), (3, 4)]]) is not valid for this graph: vertices are incorrect.
```

```
>>> from sage.all import *
>>> ss.is_equitable(prt)
Traceback (most recent call last):
...
TypeError: Partition ([[(0, 1)], [(0, 2), (0, 3), (0, 4), (1, 2), (1, 4)],
[(2, 3), (3, 4)]]) is not valid for this graph: vertices are incorrect.
```

```
sage: ss = (graphs.WheelGraph(5)).line_graph(labels=False)
sage: ss.is_equitable(prt)
False
```

```
>>> from sage.all import *
>>> ss = (graphs.WheelGraph(Integer(5))).line_graph(labels=False)
```

(continues on next page)

(continued from previous page)

```
>>> ss.is_equitable(prt)
False
```

is_eulerian(*path=False*)

Check whether the graph is Eulerian.

A graph is Eulerian if it has a (closed) tour that visits each edge exactly once.

INPUT:

- `path` – boolean (default: `False`); by default this function finds if the graph contains a closed tour visiting each edge once, i.e. an Eulerian cycle. If you want to test the existence of an Eulerian path, set this argument to `True`. Graphs with this property are sometimes called semi-Eulerian.

OUTPUT:

`True` or `False` for the closed tour case. For an open tour search (`path``=``True`) the function returns `False` if the graph is not semi-Eulerian, or a tuple (u, v) in the other case. This tuple defines the edge that would make the graph Eulerian, i.e. close an existing open tour. This edge may or may not be already present in the graph.

EXAMPLES:

```
sage: graphs.CompleteGraph(4).is_eulerian()
False
sage: graphs.CycleGraph(4).is_eulerian()
True
sage: g = DiGraph({0:[1,2], 1:[2]}); g.is_eulerian()
False
sage: g = DiGraph({0:[2], 1:[3], 2:[0,1], 3:[2]}); g.is_eulerian()
True
sage: g = DiGraph({0:[1], 1:[2], 2:[0], 3:[]}); g.is_eulerian()
True
sage: g = Graph([(1,2), (2,3), (3,1), (4,5), (5,6), (6,4)]); g.is_eulerian()
False
```

```
>>> from sage.all import *
>>> graphs.CompleteGraph(Integer(4)).is_eulerian()
False
>>> graphs.CycleGraph(Integer(4)).is_eulerian()
True
>>> g = DiGraph({Integer(0):[Integer(1),Integer(2)], Integer(1):[Integer(2)]}
↪); g.is_eulerian()
False
>>> g = DiGraph({Integer(0):[Integer(2)], Integer(1):[Integer(3)],␣
↪Integer(2):[Integer(0),Integer(1)], Integer(3):[Integer(2)]}); g.is_
↪eulerian()
True
>>> g = DiGraph({Integer(0):[Integer(1)], Integer(1):[Integer(2)],␣
↪Integer(2):[Integer(0)], Integer(3):[]}); g.is_eulerian()
True
>>> g = Graph([(Integer(1),Integer(2)), (Integer(2),Integer(3)), (Integer(3),
↪Integer(1)), (Integer(4),Integer(5)), (Integer(5),Integer(6)), (Integer(6),
↪Integer(4))]); g.is_eulerian()
False
```

```
sage: g = DiGraph({0: [1]}); g.is_eulerian(path=True)
(1, 0)
sage: graphs.CycleGraph(4).is_eulerian(path=True)
False
sage: g = DiGraph({0: [1], 1: [2,3], 2: [4]}); g.is_eulerian(path=True)
False
```

```
>>> from sage.all import *
>>> g = DiGraph({Integer(0): [Integer(1)]}); g.is_eulerian(path=True)
(1, 0)
>>> graphs.CycleGraph(Integer(4)).is_eulerian(path=True)
False
>>> g = DiGraph({Integer(0): [Integer(1)], Integer(1): [Integer(2),
↪Integer(3)], Integer(2): [Integer(4)]}); g.is_eulerian(path=True)
False
```

```
sage: g = Graph({0:[1,2,3], 1:[2,3], 2:[3,4], 3:[4]}, multiedges=True)
sage: g.is_eulerian()
False
sage: e = g.is_eulerian(path=True); e
(0, 1)
sage: g.add_edge(e)
sage: g.is_eulerian(path=False)
True
sage: g.is_eulerian(path=True)
False
```

```
>>> from sage.all import *
>>> g = Graph({Integer(0):[Integer(1),Integer(2),Integer(3)],␣
↪Integer(1):[Integer(2),Integer(3)], Integer(2):[Integer(3),Integer(4)],␣
↪Integer(3):[Integer(4)]}, multiedges=True)
>>> g.is_eulerian()
False
>>> e = g.is_eulerian(path=True); e
(0, 1)
>>> g.add_edge(e)
>>> g.is_eulerian(path=False)
True
>>> g.is_eulerian(path=True)
False
```

is_gallai_tree()

Return whether the current graph is a Gallai tree.

A graph is a Gallai tree if and only if it is connected and its 2-connected components are all isomorphic to complete graphs or odd cycles.

A connected graph is not degree-choosable if and only if it is a Gallai tree [ERT1979].

EXAMPLES:

A complete graph is, or course, a Gallai Tree:

```
sage: g = graphs.CompleteGraph(15)
sage: g.is_gallai_tree()
True
```

```
>>> from sage.all import *
>>> g = graphs.CompleteGraph(Integer(15))
>>> g.is_gallai_tree()
True
```

The Petersen Graph is not:

```
sage: g = graphs.PetersenGraph()
sage: g.is_gallai_tree()
False
```

```
>>> from sage.all import *
>>> g = graphs.PetersenGraph()
>>> g.is_gallai_tree()
False
```

A Graph built from vertex-disjoint complete graphs linked by one edge to a special vertex -1 is a "star-shaped" Gallai tree:

```
sage: g = 8 * graphs.CompleteGraph(6)
sage: g.add_edges([(-1, c[0]) for c in g.connected_components(sort=False)])
sage: g.is_gallai_tree()
True
```

```
>>> from sage.all import *
>>> g = Integer(8) * graphs.CompleteGraph(Integer(6))
>>> g.add_edges([(-Integer(1), c[Integer(0)]) for c in g.connected_
↪components(sort=False)])
>>> g.is_gallai_tree()
True
```

is_geodetic(G)

Check whether the input (di)graph is geodetic.

A graph G is *geodetic* if there exists at most one shortest path between every pair of its vertices. This can be checked in time $O(nm)$ for `SparseGraph` and $O(nm + n^2)$ for `DenseGraph` in unweighted (di)graphs with n nodes and m edges. Examples of geodetic graphs are trees, cliques and odd cycles. See the Wikipedia article Geodetic_graph for more details. Note that we do not require geodetic graphs to be connected.

(Di)graphs with multiple edges are not considered geodetic.

INPUT:

- `G` – a graph or a digraph

EXAMPLES:

Trees, cliques and odd cycles are geodetic:

```
sage: T = graphs.RandomTree(20)
sage: T.is_geodetic()
```

(continues on next page)

(continued from previous page)

```
True
sage: all(graphs.CompleteGraph(n).is_geodetic() for n in range(8))
True
sage: all(graphs.CycleGraph(n).is_geodetic() for n in range(3, 16, 2))
True
```

```
>>> from sage.all import *
>>> T = graphs.RandomTree(Integer(20))
>>> T.is_geodetic()
True
>>> all(graphs.CompleteGraph(n).is_geodetic() for n in range(Integer(8)))
True
>>> all(graphs.CycleGraph(n).is_geodetic() for n in range(Integer(3),␣
↪Integer(16), Integer(2)))
True
```

Even cycles of order at least 4 are not geodetic:

```
sage: all(graphs.CycleGraph(n).is_geodetic() for n in range(4, 17, 2))
False
```

```
>>> from sage.all import *
>>> all(graphs.CycleGraph(n).is_geodetic() for n in range(Integer(4),␣
↪Integer(17), Integer(2)))
False
```

The Petersen graph is geodetic:

```
sage: P = graphs.PetersenGraph()
sage: P.is_geodetic()
True
```

```
>>> from sage.all import *
>>> P = graphs.PetersenGraph()
>>> P.is_geodetic()
True
```

Grid graphs are not geodetic:

```
sage: G = graphs.Grid2dGraph(2, 3)
sage: G.is_geodetic()
False
```

```
>>> from sage.all import *
>>> G = graphs.Grid2dGraph(Integer(2), Integer(3))
>>> G.is_geodetic()
False
```

A graph is geodetic iff all its connected components are geodetic:

```
sage: all(G.is_geodetic() == all(H.is_geodetic() for H in G.connected_
↪components_subgraphs()) for G in graphs(5))
True
```

```
>>> from sage.all import *
>>> all(G.is_geodetic() == all(H.is_geodetic() for H in G.connected_
↪components_subgraphs()) for G in graphs(Integer(5)))
True
```

This method is also valid for digraphs:

```
sage: G = DiGraph(graphs.PetersenGraph())
sage: G.is_geodetic()
True
sage: G = digraphs.Path(5)
sage: G.add_path([0, 'a', 'b', 'c', 4])
sage: G.is_geodetic()
False
```

```
>>> from sage.all import *
>>> G = DiGraph(graphs.PetersenGraph())
>>> G.is_geodetic()
True
>>> G = digraphs.Path(Integer(5))
>>> G.add_path([Integer(0), 'a', 'b', 'c', Integer(4)])
>>> G.is_geodetic()
False
```

is_hamiltonian(*solver*, *constraint_generation=None*, *verbose=None*, *verbose_constraints=0*, *integrality_tolerance=False*)

Test whether the current graph is Hamiltonian.

A graph (resp. digraph) is said to be Hamiltonian if it contains as a subgraph a cycle (resp. a circuit) going through all the vertices.

Testing for Hamiltonicity being NP-Complete, this algorithm could run for some time depending on the instance.

ALGORITHM:

See *traveling_salesman_problem()*.

INPUT:

- `solver` – string (default: `None`); specifies a Mixed Integer Linear Programming (MILP) solver to be used. If set to `None`, the default one is used. For more information on MILP solvers and which default solver is used, see the method `solve` of the class `MixedIntegerLinearProgram`.
- `constraint_generation` – boolean; whether to use constraint generation when solving the Mixed Integer Linear Program. When `constraint_generation = None`, constraint generation is used whenever the graph has a density larger than 70%.
- `verbose` – integer (default: 0); sets the level of verbosity. Set to 0 by default, which means quiet.
- `verbose_constraints` – boolean (default: `False`); whether to display which constraints are being generated

- integrality_tolerance – float; parameter for use with MILP solvers over an inexact base ring; see MixedIntegerLinearProgram.get_values().

OUTPUT:

Returns True if a Hamiltonian cycle/circuit exists, and False otherwise.

NOTE:

This function, as hamiltonian_cycle and traveling_salesman_problem, computes a Hamiltonian cycle if it exists: the user should *NOT* test for Hamiltonicity using is_hamiltonian before calling hamiltonian_cycle or traveling_salesman_problem as it would result in computing it twice.

EXAMPLES:

The Heawood Graph is known to be Hamiltonian

```
sage: g = graphs.HeawoodGraph()
sage: g.is_hamiltonian()                                                    #␣
↪needs sage.numerical.mip
True
```

```
>>> from sage.all import *
>>> g = graphs.HeawoodGraph()
>>> g.is_hamiltonian()                                                      #␣
↪needs sage.numerical.mip
True
```

The Petergraph, though, is not

```
sage: g = graphs.PetersenGraph()
sage: g.is_hamiltonian()                                                    #␣
↪needs sage.numerical.mip
False
```

```
>>> from sage.all import *
>>> g = graphs.PetersenGraph()
>>> g.is_hamiltonian()                                                      #␣
↪needs sage.numerical.mip
False
```

is_homeomorphic(G, H)

Check whether G and H are homeomorphic.

Two graphs G and H are homeomorphic if there is an isomorphism from some subdivision of G to some subdivision of H. To check whether G and H are homeomorphic, it suffices to check whether their reduced homeomorphic (di)graphs are isomorphic. For more details, see the Wikipedia article Homeomorphism_(graph_theory).

INPUT:

- G, H – two (di)graphs

EXAMPLES:

```
sage: G = graphs.RandomGNP(10, .2)
sage: H = G.copy()
sage: for e in list(G.edges()):
```

(continues on next page)

(continued from previous page)

```
....:     G.subdivide_edge(e, randint(0, 5))
....:     H.subdivide_edge(e, randint(0, 5))
sage: G.is_homeomorphic(H)
True
sage: G = graphs.RandomGNP(10, .2)
sage: G.allow_multiple_edges(True)
sage: G.add_edges(G.edges())
sage: H = G.copy()
sage: for e in list(G.edges()):
....:     G.subdivide_edge(e, randint(0, 5))
....:     H.subdivide_edge(e, randint(0, 5))
sage: G.is_homeomorphic(H)
True

sage: G = digraphs.RandomDirectedGNP(10, .2)
sage: H = G.copy()
sage: for e in list(G.edges()):
....:     G.subdivide_edge(e, randint(0, 5))
....:     H.subdivide_edge(e, randint(0, 5))
sage: G.is_homeomorphic(H)
True
sage: G = digraphs.RandomDirectedGNP(10, .2)
sage: G.allow_multiple_edges(True)
sage: G.add_edges(G.edges())
sage: H = G.copy()
sage: for e in list(G.edges()):
....:     G.subdivide_edge(e, randint(0, 5))
....:     H.subdivide_edge(e, randint(0, 5))
sage: G.is_homeomorphic(H)
True

sage: G = digraphs.RandomDirectedGNP(10, .2)
sage: G.allow_loops(True)
sage: G.add_edges((u, u) for u in G if randint(0, 1))
sage: G.allow_multiple_edges(True)
sage: G.add_edges(G.edges())
sage: H = G.copy()
sage: for e in list(G.edges()):
....:     G.subdivide_edge(e, randint(0, 5))
....:     H.subdivide_edge(e, randint(0, 5))
sage: G.is_homeomorphic(H)
True
```

```
>>> from sage.all import *
>>> G = graphs.RandomGNP(Integer(10), RealNumber('.2'))
>>> H = G.copy()
>>> for e in list(G.edges()):
...     G.subdivide_edge(e, randint(Integer(0), Integer(5)))
...     H.subdivide_edge(e, randint(Integer(0), Integer(5)))
>>> G.is_homeomorphic(H)
True
>>> G = graphs.RandomGNP(Integer(10), RealNumber('.2'))
```

(continues on next page)

(continued from previous page)

```
>>> G.allow_multiple_edges(True)
>>> G.add_edges(G.edges())
>>> H = G.copy()
>>> for e in list(G.edges()):
...     G.subdivide_edge(e, randint(Integer(0), Integer(5)))
...     H.subdivide_edge(e, randint(Integer(0), Integer(5)))
>>> G.is_homeomorphic(H)
True

>>> G = digraphs.RandomDirectedGNP(Integer(10), RealNumber('.2'))
>>> H = G.copy()
>>> for e in list(G.edges()):
...     G.subdivide_edge(e, randint(Integer(0), Integer(5)))
...     H.subdivide_edge(e, randint(Integer(0), Integer(5)))
>>> G.is_homeomorphic(H)
True
>>> G = digraphs.RandomDirectedGNP(Integer(10), RealNumber('.2'))
>>> G.allow_multiple_edges(True)
>>> G.add_edges(G.edges())
>>> H = G.copy()
>>> for e in list(G.edges()):
...     G.subdivide_edge(e, randint(Integer(0), Integer(5)))
...     H.subdivide_edge(e, randint(Integer(0), Integer(5)))
>>> G.is_homeomorphic(H)
True

>>> G = digraphs.RandomDirectedGNP(Integer(10), RealNumber('.2'))
>>> G.allow_loops(True)
>>> G.add_edges((u, u) for u in G if randint(Integer(0), Integer(1)))
>>> G.allow_multiple_edges(True)
>>> G.add_edges(G.edges())
>>> H = G.copy()
>>> for e in list(G.edges()):
...     G.subdivide_edge(e, randint(Integer(0), Integer(5)))
...     H.subdivide_edge(e, randint(Integer(0), Integer(5)))
>>> G.is_homeomorphic(H)
True
```

is_immutable()

Check whether the graph is immutable.

EXAMPLES:

```
sage: G = graphs.PetersenGraph()
sage: G.is_immutable()
False
sage: Graph(G, immutable=True).is_immutable()
True
```

```
>>> from sage.all import *
>>> G = graphs.PetersenGraph()
>>> G.is_immutable()
```

(continues on next page)

(continued from previous page)

```
False
>>> Graph(G, immutable=True).is_immutable()
True
```

is_independent_set(*vertices=None*)

Check whether `vertices` is an independent set of `self`.

An independent set is a set of vertices such that there is no edge between any two vertices.

INPUT:

- `vertices` – a single vertex or an iterable container of vertices (default: `None`); when set, check whether the given set of vertices is an independent set, otherwise, check whether the set of vertices of `self` is an independent set

EXAMPLES:

```
sage: graphs.CycleGraph(4).is_independent_set([1,3])
True
sage: graphs.CycleGraph(4).is_independent_set([1,2,3])
False
```

```
>>> from sage.all import *
>>> graphs.CycleGraph(Integer(4)).is_independent_set([Integer(1),Integer(3)])
True
>>> graphs.CycleGraph(Integer(4)).is_independent_set([Integer(1),Integer(2),
↪Integer(3)])
False
```

is_interval(*certificate=False*)

Check whether the graph is an interval graph.

An *interval graph* is one where every vertex can be seen as an interval on the real line so that there is an edge in the graph iff the corresponding intervals intersects.

See the Wikipedia article Interval_graph for more information.

INPUT:

- `certificate` – boolean (default: `False`)
  - When `certificate=False`, returns `True` if the graph is an interval graph and `False` otherwise
  - When `certificate=True`, returns either `(False, None)` or `(True, d)` where `d` is a dictionary whose keys are the vertices and values are pairs of integers. They correspond to an embedding of the interval graph, each vertex being represented by an interval going from the first of the two values to the second.

ALGORITHM:

Through the use of PQ-Trees.

AUTHOR:

Nathann Cohen (implementation)

EXAMPLES:

```
sage: g = Graph({1: [2, 3, 4], 4: [2, 3]})
sage: g.is_interval()
True
sage: g.is_interval(certificate=True)
(True, {1: (0, 5), 2: (4, 6), 3: (1, 3), 4: (2, 7)})
```

```
>>> from sage.all import *
>>> g = Graph({Integer(1): [Integer(2), Integer(3), Integer(4)], Integer(4):␣
↪[Integer(2), Integer(3)]})
>>> g.is_interval()
True
>>> g.is_interval(certificate=True)
(True, {1: (0, 5), 2: (4, 6), 3: (1, 3), 4: (2, 7)})
```

The Petersen Graph is not chordal, so it cannot be an interval graph:

```
sage: g = graphs.PetersenGraph()
sage: g.is_interval()
False
```

```
>>> from sage.all import *
>>> g = graphs.PetersenGraph()
>>> g.is_interval()
False
```

A chordal but still not an interval graph:

```
sage: g = Graph({1: [4, 2, 3], 2: [3, 5], 3: [6]})
sage: g.is_interval()
False
```

```
>>> from sage.all import *
>>> g = Graph({Integer(1): [Integer(4), Integer(2), Integer(3)], Integer(2):␣
↪[Integer(3), Integer(5)], Integer(3): [Integer(6)]})
>>> g.is_interval()
False
```

See also

- *Interval Graph Recognition*.
- *PQ* – implementation of PQ-Trees
- *is_chordal()*
- *IntervalGraph()*
- *RandomIntervalGraph()*

is_isomorphic(*other*, *certificate=False*, *verbosity=0*, *edge_labels=False*)

Test for isomorphism between `self` and `other`.

INPUT:

- `certificate` – if `True`, then output is (a, b), where a is a boolean and b is either a map or `None`
- `edge_labels` – boolean (default: `False`); if `True` allows only permutations respecting edge labels

OUTPUT:

- either a boolean or, if `certificate` is `True`, a tuple consisting of a boolean and a map or `None`

EXAMPLES:

Graphs:

```
sage: from sage.groups.perm_gps.permgroup_named import SymmetricGroup        #␣
↪needs sage.groups
sage: D = graphs.DodecahedralGraph()
sage: E = copy(D)
sage: gamma = SymmetricGroup(20).random_element()                            #␣
↪needs sage.groups
sage: E.relabel(gamma)                                                       #␣
↪needs sage.groups
sage: D.is_isomorphic(E)
True
```

```
>>> from sage.all import *
>>> from sage.groups.perm_gps.permgroup_named import SymmetricGroup        #␣
↪needs sage.groups
>>> D = graphs.DodecahedralGraph()
>>> E = copy(D)
>>> gamma = SymmetricGroup(Integer(20)).random_element()                     ␣
↪     # needs sage.groups
>>> E.relabel(gamma)                                                       #␣
↪needs sage.groups
>>> D.is_isomorphic(E)
True
```

```
sage: D = graphs.DodecahedralGraph()
sage: S = SymmetricGroup(20)                                                 #␣
↪needs sage.groups
sage: gamma = S.random_element()                                             #␣
↪needs sage.groups
sage: E = copy(D)                                                            #␣
↪needs sage.groups
sage: E.relabel(gamma)                                                       #␣
↪needs sage.groups
sage: a,b = D.is_isomorphic(E, certificate=True); a
True
sage: from sage.graphs.generic_graph_pyx import spring_layout_fast
sage: position_D = spring_layout_fast(D)
sage: position_E = {}
sage: for vert in position_D:
....:  position_E[b[vert]] = position_D[vert]
sage: graphics_array([D.plot(pos=position_D), E.plot(pos=position_E)]).show()␣
↪ # long time, needs sage.plot
```

```
>>> from sage.all import *
>>> D = graphs.DodecahedralGraph()
>>> S = SymmetricGroup(Integer(20))
↪     # needs sage.groups
>>> gamma = S.random_element()
↪needs sage.groups
>>> E = copy(D)
↪needs sage.groups
>>> E.relabel(gamma)
↪needs sage.groups
>>> a,b = D.is_isomorphic(E, certificate=True); a
True
>>> from sage.graphs.generic_graph_pyx import spring_layout_fast
>>> position_D = spring_layout_fast(D)
>>> position_E = {}
>>> for vert in position_D:
...  position_E[b[vert]] = position_D[vert]
>>> graphics_array([D.plot(pos=position_D), E.plot(pos=position_E)]).show()
↪# long time, needs sage.plot
```

```
sage: g=graphs.HeawoodGraph()
sage: g.is_isomorphic(g)
True
```

```
>>> from sage.all import *
>>> g=graphs.HeawoodGraph()
>>> g.is_isomorphic(g)
True
```

Multigraphs:

```
sage: G = Graph(multiedges=True,sparse=True)
sage: G.add_edge((0,1,1))
sage: G.add_edge((0,1,2))
sage: G.add_edge((0,1,3))
sage: G.add_edge((0,1,4))
sage: H = Graph(multiedges=True,sparse=True)
sage: H.add_edge((3,4))
sage: H.add_edge((3,4))
sage: H.add_edge((3,4))
sage: H.add_edge((3,4))
sage: G.is_isomorphic(H)
True
```

```
>>> from sage.all import *
>>> G = Graph(multiedges=True,sparse=True)
>>> G.add_edge((Integer(0),Integer(1),Integer(1)))
>>> G.add_edge((Integer(0),Integer(1),Integer(2)))
>>> G.add_edge((Integer(0),Integer(1),Integer(3)))
>>> G.add_edge((Integer(0),Integer(1),Integer(4)))
>>> H = Graph(multiedges=True,sparse=True)
>>> H.add_edge((Integer(3),Integer(4)))
```

(continues on next page)

(continued from previous page)

```
>>> H.add_edge((Integer(3),Integer(4)))
>>> H.add_edge((Integer(3),Integer(4)))
>>> H.add_edge((Integer(3),Integer(4)))
>>> G.is_isomorphic(H)
True
```

Digraphs:

```
sage: A = DiGraph( { 0 : [1,2] } )
sage: B = DiGraph( { 1 : [0,2] } )
sage: A.is_isomorphic(B, certificate=True)
(True, {0: 1, 1: 0, 2: 2})
```

```
>>> from sage.all import *
>>> A = DiGraph( { Integer(0) : [Integer(1),Integer(2)] } )
>>> B = DiGraph( { Integer(1) : [Integer(0),Integer(2)] } )
>>> A.is_isomorphic(B, certificate=True)
(True, {0: 1, 1: 0, 2: 2})
```

Edge labeled graphs:

```
sage: G = Graph(sparse=True)
sage: G.add_edges( [(0,1,'a'),(1,2,'b'),(2,3,'c'),(3,4,'b'),(4,0,'a')] )
sage: H = G.relabel([1,2,3,4,0], inplace=False)
sage: G.is_isomorphic(H, edge_labels=True)
True
```

```
>>> from sage.all import *
>>> G = Graph(sparse=True)
>>> G.add_edges( [(Integer(0),Integer(1),'a'),(Integer(1),Integer(2),'b'),
...(Integer(2),Integer(3),'c'),(Integer(3),Integer(4),'b'),(Integer(4),
...Integer(0),'a')] )
>>> H = G.relabel([Integer(1),Integer(2),Integer(3),Integer(4),Integer(0)],␣
...inplace=False)
>>> G.is_isomorphic(H, edge_labels=True)
True
```

Edge labeled digraphs:

```
sage: G = DiGraph()
sage: G.add_edges( [(0,1,'a'),(1,2,'b'),(2,3,'c'),(3,4,'b'),(4,0,'a')] )
sage: H = G.relabel([1,2,3,4,0], inplace=False)
sage: G.is_isomorphic(H, edge_labels=True)
True
sage: G.is_isomorphic(H, edge_labels=True, certificate=True)
(True, {0: 1, 1: 2, 2: 3, 3: 4, 4: 0})
```

```
>>> from sage.all import *
>>> G = DiGraph()
>>> G.add_edges( [(Integer(0),Integer(1),'a'),(Integer(1),Integer(2),'b'),
...(Integer(2),Integer(3),'c'),(Integer(3),Integer(4),'b'),(Integer(4),
...Integer(0),'a')] )
```

(continues on next page)

(continued from previous page)

```
>>> H = G.relabel([Integer(1),Integer(2),Integer(3),Integer(4),Integer(0)],␣
↪inplace=False)
>>> G.is_isomorphic(H, edge_labels=True)
True
>>> G.is_isomorphic(H, edge_labels=True, certificate=True)
(True, {0: 1, 1: 2, 2: 3, 3: 4, 4: 0})
```

is_planar(*on_embedding=None*, *kuratowski=False*, *set_embedding=False*, *set_pos=False*, *immutable=None*)

Check whether the graph is planar.

This wraps the reference implementation provided by John Boyer of the linear time planarity algorithm by edge addition due to Boyer Myrvold. (See reference code in *planarity*).

Note

The argument on_embedding takes precedence over `set_embedding`. This means that only the `on_embedding` combinatorial embedding will be tested for planarity and no `_embedding` attribute will be set as a result of this function call, unless `on_embedding` is None.

REFERENCE:

[BM2004]

See also

- "Almost planar graph": *is_apex()*
- "Measuring non-planarity": *genus()*, *crossing_number()*
- *planar_dual()*
- *faces()*
- *is_polyhedral()*

INPUT:

- `on_embedding` – dictionary (default: `None`); the embedding dictionary to test planarity on (i.e.: will return `True` or `False` only for the given embedding)
- `kuratowski` – boolean (default: `False`); whether to return a tuple with boolean as first entry. If the graph is nonplanar, will return the Kuratowski subgraph (i.e. an edge subdivision of K_5 or $K_{3,3}$) as the second tuple entry. If the graph is planar, returns `None` as the second entry. When set to `False`, only a boolean answer is returned.
- `set_embedding` – boolean (default: `False`); whether to set the instance field variable that contains a combinatorial embedding (clockwise ordering of neighbors at each vertex). This value will only be set if a planar embedding is found. It is stored as a Python dict: `v1: [n1,n2,n3]` where `v1` is a vertex and `n1,n2,n3` are its neighbors.
- `set_pos` – boolean (default: `False`); whether to set the position dictionary (for plotting) to reflect the combinatorial embedding. Note that this value will default to False if set_emb is set to False. Also, the position dictionary will only be updated if a planar embedding is found.
- `immutable` – boolean (default: `None`); whether to create a mutable/immutable graph. `immutable=None` (default) means that the graph and the Kuratowski subgraph will behave the same way.

This parameter is ignored when `kuratowski=False`.

EXAMPLES:

```
sage: g = graphs.CubeGraph(4)
sage: g.is_planar()
False
```

```
>>> from sage.all import *
>>> g = graphs.CubeGraph(Integer(4))
>>> g.is_planar()
False
```

```
sage: g = graphs.CircularLadderGraph(4)
sage: g.is_planar(set_embedding=True)
True
sage: g.get_embedding()
{0: [1, 4, 3],
 1: [2, 5, 0],
 2: [3, 6, 1],
 3: [0, 7, 2],
 4: [0, 5, 7],
 5: [1, 6, 4],
 6: [2, 7, 5],
 7: [4, 6, 3]}
```

```
>>> from sage.all import *
>>> g = graphs.CircularLadderGraph(Integer(4))
>>> g.is_planar(set_embedding=True)
True
>>> g.get_embedding()
{0: [1, 4, 3],
 1: [2, 5, 0],
 2: [3, 6, 1],
 3: [0, 7, 2],
 4: [0, 5, 7],
 5: [1, 6, 4],
 6: [2, 7, 5],
 7: [4, 6, 3]}
```

```
sage: g = graphs.PetersenGraph()
sage: (g.is_planar(kuratowski=True))[1].adjacency_matrix()                    #␣
↪needs sage.modules
[0 1 0 0 0 1 0 0 0]
[1 0 1 0 0 0 1 0 0]
[0 1 0 1 0 0 0 1 0]
[0 0 1 0 0 0 0 0 1]
[0 0 0 0 0 0 1 1 0]
[1 0 0 0 0 0 0 1 1]
[0 1 0 0 1 0 0 0 1]
[0 0 1 0 1 1 0 0 0]
[0 0 0 1 0 1 1 0 0]
```

```
>>> from sage.all import *
>>> g = graphs.PetersenGraph()
>>> (g.is_planar(kuratowski=True))[Integer(1)].adjacency_matrix()
↪     # needs sage.modules
[0 1 0 0 0 1 0 0 0]
[1 0 1 0 0 0 1 0 0]
[0 1 0 1 0 0 0 1 0]
[0 0 1 0 0 0 0 0 1]
[0 0 0 0 0 0 1 1 0]
[1 0 0 0 0 0 0 1 1]
[0 1 0 0 1 0 0 0 1]
[0 0 1 0 1 1 0 0 0]
[0 0 0 1 0 1 1 0 0]
```

```
sage: k43 = graphs.CompleteBipartiteGraph(4, 3)
sage: result = k43.is_planar(kuratowski=True); result
(False,
 Kuratowski subgraph of (Complete bipartite graph of order 4+3): Graph on 6
↪vertices)
sage: result[1].is_isomorphic(graphs.CompleteBipartiteGraph(3, 3))
True
```

```
>>> from sage.all import *
>>> k43 = graphs.CompleteBipartiteGraph(Integer(4), Integer(3))
>>> result = k43.is_planar(kuratowski=True); result
(False,
 Kuratowski subgraph of (Complete bipartite graph of order 4+3): Graph on 6
↪vertices)
>>> result[Integer(1)].is_isomorphic(graphs.CompleteBipartiteGraph(Integer(3),
↪ Integer(3)))
True
```

Multi-edged and looped graphs are partially supported:

```
sage: G = Graph({0: [1, 1]}, multiedges=True)
sage: G.is_planar()
True
sage: G.is_planar(on_embedding={})
Traceback (most recent call last):
...
NotImplementedError: cannot compute with embeddings of
 multiple-edged or looped graphs
sage: G.is_planar(set_embedding=True)
Traceback (most recent call last):
...
NotImplementedError: cannot compute with embeddings of
 multiple-edged or looped graphs
sage: G.is_planar(kuratowski=True)
(True, None)
sage: G.is_planar(set_pos=True)
True
sage: sorted(G.get_pos().items())
[(0, [0, 0]), (1, [0, 1])]
```

```
>>> from sage.all import *
>>> G = Graph({Integer(0): [Integer(1), Integer(1)]}, multiedges=True)
>>> G.is_planar()
True
>>> G.is_planar(on_embedding={})
Traceback (most recent call last):
...
NotImplementedError: cannot compute with embeddings of
 multiple-edged or looped graphs
>>> G.is_planar(set_embedding=True)
Traceback (most recent call last):
...
NotImplementedError: cannot compute with embeddings of
 multiple-edged or looped graphs
>>> G.is_planar(kuratowski=True)
(True, None)
>>> G.is_planar(set_pos=True)
True
>>> sorted(G.get_pos().items())
[(0, [0, 0]), (1, [0, 1])]
```

Digraphs with multiple edges or loops or pairs of opposite arcs are partially supported (Issue #35152):

```
sage: D = digraphs.Complete(3)
sage: D.is_planar()
True
sage: D.is_planar(set_pos=True)
True
sage: sorted(D.get_pos().items())
[(0, [0, 1]), (1, [1, 1]), (2, [1, 0])]
sage: D.is_planar(on_embedding={})
Traceback (most recent call last):
...
NotImplementedError: cannot compute with embeddings of
 digraphs with pairs of opposite arcs
sage: D.is_planar(set_embedding=True)
Traceback (most recent call last):
...
NotImplementedError: cannot compute with embeddings of
 digraphs with pairs of opposite arcs
sage: D.is_planar(kuratowski=True)
(True, None)
sage: D.allow_multiple_edges(True)
sage: D.add_edges(D.edges(sort=False))
sage: D.allow_loops(True)
sage: D.add_edges((u, u) for u in D)
sage: D.is_planar()
True
sage: D.is_planar(kuratowski=True)
(True, None)
sage: D.is_planar(set_pos=True)
True
sage: D.is_planar(set_embedding=True)
```

(continues on next page)

(continued from previous page)

```
Traceback (most recent call last):
...
NotImplementedError: cannot compute with embeddings of
 multiple-edged or looped graphs
sage: D.is_planar(on_embedding={})
Traceback (most recent call last):
...
NotImplementedError: cannot compute with embeddings of
 multiple-edged or looped graphs
```

```
>>> from sage.all import *
>>> D = digraphs.Complete(Integer(3))
>>> D.is_planar()
True
>>> D.is_planar(set_pos=True)
True
>>> sorted(D.get_pos().items())
[(0, [0, 1]), (1, [1, 1]), (2, [1, 0])]
>>> D.is_planar(on_embedding={})
Traceback (most recent call last):
...
NotImplementedError: cannot compute with embeddings of
 digraphs with pairs of opposite arcs
>>> D.is_planar(set_embedding=True)
Traceback (most recent call last):
...
NotImplementedError: cannot compute with embeddings of
 digraphs with pairs of opposite arcs
>>> D.is_planar(kuratowski=True)
(True, None)
>>> D.allow_multiple_edges(True)
>>> D.add_edges(D.edges(sort=False))
>>> D.allow_loops(True)
>>> D.add_edges((u, u) for u in D)
>>> D.is_planar()
True
>>> D.is_planar(kuratowski=True)
(True, None)
>>> D.is_planar(set_pos=True)
True
>>> D.is_planar(set_embedding=True)
Traceback (most recent call last):
...
NotImplementedError: cannot compute with embeddings of
 multiple-edged or looped graphs
>>> D.is_planar(on_embedding={})
Traceback (most recent call last):
...
NotImplementedError: cannot compute with embeddings of
 multiple-edged or looped graphs
```

```
sage: G = graphs.CompleteGraph(5)
sage: G = Graph(G, multiedges=True)
sage: G.add_edge(0, 1)
sage: G.is_planar()
False
sage: b,k = G.is_planar(kuratowski=True)
sage: b
False
sage: k.vertices(sort=True)
[0, 1, 2, 3, 4]
```

```
>>> from sage.all import *
>>> G = graphs.CompleteGraph(Integer(5))
>>> G = Graph(G, multiedges=True)
>>> G.add_edge(Integer(0), Integer(1))
>>> G.is_planar()
False
>>> b,k = G.is_planar(kuratowski=True)
>>> b
False
>>> k.vertices(sort=True)
[0, 1, 2, 3, 4]
```

is_regular(*k=None*)

Check whether this graph is (*k*-)regular.

INPUT:

- `k` – integer (default: `None`); the degree of regularity to check for

EXAMPLES:

```
sage: G = graphs.HoffmanSingletonGraph()
sage: G.is_regular()
True
sage: G.is_regular(9)
False
```

```
>>> from sage.all import *
>>> G = graphs.HoffmanSingletonGraph()
>>> G.is_regular()
True
>>> G.is_regular(Integer(9))
False
```

So the Hoffman-Singleton graph is regular, but not 9-regular. In fact, we can now find the degree easily as follows:

```
sage: next(G.degree_iterator())
7
```

```
>>> from sage.all import *
>>> next(G.degree_iterator())
7
```

The house graph is not regular:

```
sage: graphs.HouseGraph().is_regular()
False
```

```
>>> from sage.all import *
>>> graphs.HouseGraph().is_regular()
False
```

A graph without vertices is k-regular for every k:

```
sage: Graph().is_regular()
True
```

```
>>> from sage.all import *
>>> Graph().is_regular()
True
```

is_self_complementary()

Check whether the graph is self-complementary.

A (di)graph is self-complementary if it is isomorphic to its (di)graph complement. For instance, the path graph P_4 and the cycle graph C_5 are self-complementary.

See also

- Wikipedia article Self-complementary_graph
- OEIS sequence A000171 for the numbers of self-complementary graphs of order n
- OEIS sequence A003086 for the numbers of self-complementary digraphs of order n.

EXAMPLES:

The only self-complementary path graph is P_4:

```
sage: graphs.PathGraph(4).is_self_complementary()
True
sage: graphs.PathGraph(5).is_self_complementary()
False
```

```
>>> from sage.all import *
>>> graphs.PathGraph(Integer(4)).is_self_complementary()
True
>>> graphs.PathGraph(Integer(5)).is_self_complementary()
False
```

The only self-complementary directed path is P_2:

```
sage: digraphs.Path(2).is_self_complementary()
True
sage: digraphs.Path(3).is_self_complementary()
False
```

```
>>> from sage.all import *
>>> digraphs.Path(Integer(2)).is_self_complementary()
True
>>> digraphs.Path(Integer(3)).is_self_complementary()
False
```

Every Paley graph is self-complementary:

```
sage: G = graphs.PaleyGraph(9)                                              #␣
↪needs sage.libs.pari
sage: G.is_self_complementary()                                             #␣
↪needs sage.libs.pari
True
```

```
>>> from sage.all import *
>>> G = graphs.PaleyGraph(Integer(9))                                        ␣
↪     # needs sage.libs.pari
>>> G.is_self_complementary()                                              #␣
↪needs sage.libs.pari
True
```

is_subgraph(*other*, *induced=True*, *up_to_isomorphism=False*)

Check whether `self` is a subgraph of `other`.

Warning

The arguments `induced` and `up_to_isomorphism` are set respectively to `True` and `False` by default.

INPUT:

- `other` – a Sage (Di)Graph
- `induced` – boolean (default: `True`); if set to `True` check whether the graph is an *induced* subgraph of `other` that is if the vertices of the graph are also vertices of `other`, and the edges of the graph are equal to the edges of `other` between the vertices contained in the graph.

  If set to `False` tests whether the graph is a subgraph of `other` that is if all vertices of the graph are also in `other` and all edges of the graph are also in `other`.
- `up_to_isomorphism` – boolean (default: `False`); if set to `True` check whether `other` is a subgraph ignoring the labeling of vertices and edges. Otherwise, vertex and edge labellings must coincide in the copy or induced copy.

OUTPUT:

boolean – `True` iff the graph is a (possibly induced) subgraph of `other`.

See also

For more advanced search of subgraphs isomorphic to a given graph, you could consider the following methods:

- *subgraph_search()* – find a subgraph isomorphic to `other` inside of the graph
- *subgraph_search_count()* – count the number of such copies

- *subgraph_search_iterator()* – iterator over all the copies of `other` contained in the graph

EXAMPLES:

```
sage: P = graphs.PetersenGraph()
sage: G = P.subgraph(range(6))
sage: G.is_subgraph(P)
True

sage: H = graphs.CycleGraph(5)
sage: G = graphs.PathGraph(5)
sage: G.is_subgraph(H)
False
sage: G.is_subgraph(H, induced=False)
True
sage: H.is_subgraph(G, induced=False)
False
```

```
>>> from sage.all import *
>>> P = graphs.PetersenGraph()
>>> G = P.subgraph(range(Integer(6)))
>>> G.is_subgraph(P)
True

>>> H = graphs.CycleGraph(Integer(5))
>>> G = graphs.PathGraph(Integer(5))
>>> G.is_subgraph(H)
False
>>> G.is_subgraph(H, induced=False)
True
>>> H.is_subgraph(G, induced=False)
False
```

The 4x4 grid contains a path of length 15 and an induced path of length 11:

```
sage: p11 = graphs.PathGraph(11)
sage: p15 = graphs.PathGraph(15)
sage: g = graphs.Grid2dGraph(4, 4)
sage: p15.is_subgraph(g, induced=False, up_to_isomorphism=True)             #␣
↪needs sage.modules
True
sage: p15.is_subgraph(g, induced=True, up_to_isomorphism=True)              #␣
↪needs sage.modules
False
sage: p11.is_subgraph(g, induced=True, up_to_isomorphism=True)              #␣
↪needs sage.modules
True
```

```
>>> from sage.all import *
>>> p11 = graphs.PathGraph(Integer(11))
>>> p15 = graphs.PathGraph(Integer(15))
>>> g = graphs.Grid2dGraph(Integer(4), Integer(4))
```

(continues on next page)

(continued from previous page)

```
>>> p15.is_subgraph(g, induced=False, up_to_isomorphism=True)              #␣
↪needs sage.modules
True
>>> p15.is_subgraph(g, induced=True, up_to_isomorphism=True)               #␣
↪needs sage.modules
False
>>> p11.is_subgraph(g, induced=True, up_to_isomorphism=True)               #␣
↪needs sage.modules
True
```

is_transitively_reduced()

Check whether the digraph is transitively reduced.

A digraph is transitively reduced if it is equal to its transitive reduction. A graph is transitively reduced if it is a forest.

EXAMPLES:

```
sage: d = DiGraph({0: [1], 1: [2], 2: [3]})
sage: d.is_transitively_reduced()
True

sage: d = DiGraph({0: [1, 2], 1: [2]})
sage: d.is_transitively_reduced()
False

sage: d = DiGraph({0: [1, 2], 1: [2], 2: []})
sage: d.is_transitively_reduced()
False
```

```
>>> from sage.all import *
>>> d = DiGraph({Integer(0): [Integer(1)], Integer(1): [Integer(2)],␣
↪Integer(2): [Integer(3)]})
>>> d.is_transitively_reduced()
True

>>> d = DiGraph({Integer(0): [Integer(1), Integer(2)], Integer(1):␣
↪[Integer(2)]})
>>> d.is_transitively_reduced()
False

>>> d = DiGraph({Integer(0): [Integer(1), Integer(2)], Integer(1):␣
↪[Integer(2)], Integer(2): []})
>>> d.is_transitively_reduced()
False
```

is_vertex_cut(*G*, *cut*, *weak=False*)

Check whether the input vertices form a vertex cut.

A set of vertices is a vertex cut if its removal from the (di)graph increases the number of (strongly) connected components. This function works with simple graphs as well as graphs with loops and multiple edges.

INPUT:

- `G` – a Sage (Di)Graph

- `cut` – a set of vertices
- `weak` – boolean (default: `False`); whether the connectivity of directed graphs is to be taken in the weak sense, that is ignoring edges orientations

EXAMPLES:

Giving a cycle graph of order 4:

```
sage: from sage.graphs.connectivity import is_vertex_cut
sage: G = graphs.CycleGraph(4)
sage: is_vertex_cut(G, [0, 1])
False
sage: is_vertex_cut(G, [0, 2])
True
```

```
>>> from sage.all import *
>>> from sage.graphs.connectivity import is_vertex_cut
>>> G = graphs.CycleGraph(Integer(4))
>>> is_vertex_cut(G, [Integer(0), Integer(1)])
False
>>> is_vertex_cut(G, [Integer(0), Integer(2)])
True
```

Giving a disconnected graph:

```
sage: from sage.graphs.connectivity import is_vertex_cut
sage: G = graphs.CycleGraph(4) * 2
sage: G.connected_components(sort=True)
[[0, 1, 2, 3], [4, 5, 6, 7]]
sage: is_vertex_cut(G, [0, 2])
True
sage: is_vertex_cut(G, [4, 6])
True
sage: is_vertex_cut(G, [0, 6])
False
sage: is_vertex_cut(G, [0, 4, 6])
True
```

```
>>> from sage.all import *
>>> from sage.graphs.connectivity import is_vertex_cut
>>> G = graphs.CycleGraph(Integer(4)) * Integer(2)
>>> G.connected_components(sort=True)
[[0, 1, 2, 3], [4, 5, 6, 7]]
>>> is_vertex_cut(G, [Integer(0), Integer(2)])
True
>>> is_vertex_cut(G, [Integer(4), Integer(6)])
True
>>> is_vertex_cut(G, [Integer(0), Integer(6)])
False
>>> is_vertex_cut(G, [Integer(0), Integer(4), Integer(6)])
True
```

Comparing the weak and strong connectivity of a digraph:

```
sage: D = digraphs.Circuit(6)
sage: D.is_strongly_connected()
True
sage: is_vertex_cut(D, [2])
True
sage: is_vertex_cut(D, [2], weak=True)
False
```

```
>>> from sage.all import *
>>> D = digraphs.Circuit(Integer(6))
>>> D.is_strongly_connected()
True
>>> is_vertex_cut(D, [Integer(2)])
True
>>> is_vertex_cut(D, [Integer(2)], weak=True)
False
```

Giving a vertex that is not in the graph:

```
sage: G = graphs.CompleteGraph(4)
sage: is_vertex_cut(G, [7])
Traceback (most recent call last):
...
ValueError: vertex (7) is not a vertex of the graph
```

```
>>> from sage.all import *
>>> G = graphs.CompleteGraph(Integer(4))
>>> is_vertex_cut(G, [Integer(7)])
Traceback (most recent call last):
...
ValueError: vertex (7) is not a vertex of the graph
```

is_vertex_transitive(*partition=None*, *verbosity=0*, *edge_labels=False*, *order=False*, *return_group=True*, *orbits=False*)

Return whether the automorphism group of `self` is transitive within the partition provided, by default the unit partition of the vertices of `self` (thus by default tests for vertex transitivity in the usual sense).

EXAMPLES:

```
sage: G = Graph({0:[1],1:[2]})
sage: G.is_vertex_transitive()
False
sage: P = graphs.PetersenGraph()
sage: P.is_vertex_transitive()                                              #␣
↪needs sage.groups
True
sage: D = graphs.DodecahedralGraph()
sage: D.is_vertex_transitive()                                              #␣
↪needs sage.groups
True
sage: R = graphs.RandomGNP(2000, .01)                                       #␣
↪needs networkx
sage: R.is_vertex_transitive()                                              #␣
```

(continues on next page)

(continued from previous page)

```
↪needs networkx
False
```

```
>>> from sage.all import *
>>> G = Graph({Integer(0):[Integer(1)],Integer(1):[Integer(2)]})
>>> G.is_vertex_transitive()
False
>>> P = graphs.PetersenGraph()
>>> P.is_vertex_transitive()                                                #␣
↪needs sage.groups
True
>>> D = graphs.DodecahedralGraph()
>>> D.is_vertex_transitive()                                                #␣
↪needs sage.groups
True
>>> R = graphs.RandomGNP(Integer(2000), RealNumber('.01'))                    ␣
↪                    # needs networkx
>>> R.is_vertex_transitive()                                                #␣
↪needs networkx
False
```

katz_centrality(*alpha*, *u=None*)

Return the Katz centrality of vertex u.

Katz centrality of a node is a measure of centrality in a graph network. Katz centrality computes the relative influence of a node within a network. Connections made with distant neighbors are, however penalized by an attenuation factor α.

See the Wikipedia article Katz_centrality for more information.

INPUT:

- `alpha` – a nonnegative real number, must be less than the reciprocal of the spectral radius of the graph (the maximum absolute eigenvalue of the adjacency matrix).
- `u` – the vertex whose Katz centrality needs to be measured (default: `None`)

OUTPUT: list containing the Katz centrality of each vertex if u=None otherwise Katz centrality of the vertex u.

EXAMPLES:

We compute the Katz centrality of a 4-cycle (note that by symmetry, all 4 vertices have the same centrality)

```
sage: G = graphs.CycleGraph(4)
sage: G.katz_centrality(1/20)                                              #␣
↪needs sage.modules sage.rings.number_field
{0: 1/9, 1: 1/9, 2: 1/9, 3: 1/9}
```

```
>>> from sage.all import *
>>> G = graphs.CycleGraph(Integer(4))
>>> G.katz_centrality(Integer(1)/Integer(20))                                 ␣
↪              # needs sage.modules sage.rings.number_field
{0: 1/9, 1: 1/9, 2: 1/9, 3: 1/9}
```

Note that in the below example the nodes having indegree 0 also have the Katz centrality value as 0, as these nodes are not influenced by other nodes.

```
sage: G = DiGraph({1: [10], 2:[10,11], 3:[10,11], 4:[], 5:[11, 4], 6:[11],
....:              7:[10,11], 8:[10,11], 9:[10], 10:[11, 5, 8], 11:[6]})
sage: G.katz_centrality(.85)  # rel tol 1e-14                                #␣
↪needs sage.modules sage.rings.number_field
{1: 0.000000000000000,
 2: 0.000000000000000,
 3: 0.000000000000000,
 4: 16.7319819819820,
 5: 18.6846846846847,
 6: 173.212076941807,
 7: 0.000000000000000,
 8: 18.6846846846847,
 9: 0.000000000000000,
 10: 20.9819819819820,
 11: 202.778914049184}
```

```
>>> from sage.all import *
>>> G = DiGraph({Integer(1): [Integer(10)], Integer(2):[Integer(10),
↪Integer(11)], Integer(3):[Integer(10),Integer(11)], Integer(4):[],␣
↪Integer(5):[Integer(11), Integer(4)], Integer(6):[Integer(11)],
...               Integer(7):[Integer(10),Integer(11)],␣
↪Integer(8):[Integer(10),Integer(11)], Integer(9):[Integer(10)],␣
↪Integer(10):[Integer(11), Integer(5), Integer(8)], Integer(11):[Integer(6)]}
↪)
>>> G.katz_centrality(RealNumber('.85'))  # rel tol 1e-14                    ␣
↪          # needs sage.modules sage.rings.number_field
{1: 0.000000000000000,
 2: 0.000000000000000,
 3: 0.000000000000000,
 4: 16.7319819819820,
 5: 18.6846846846847,
 6: 173.212076941807,
 7: 0.000000000000000,
 8: 18.6846846846847,
 9: 0.000000000000000,
 10: 20.9819819819820,
 11: 202.778914049184}
```

See also

- *katz_matrix()*
- Wikipedia article Katz_centrality

katz_matrix(*alpha*, *nonedgesonly=False*, *vertices=None*)

Return the Katz matrix of the graph.

Katz centrality of a node is a measure of centrality in a graph network. Katz centrality computes the relative influence of a node within a network. Connections made with distant neighbors are, however penalized by an attenuation factor α.

Adding the values in the Katz matrix of all columns in a particular row gives the Katz centrality measure of the vertex represented by that particular row. Katz centrality measures influence by taking into account the total number of walks between a pair of nodes.

See the Wikipedia article Katz_centrality for more information.

INPUT:

- `alpha` – a nonnegative real number, must be less than the reciprocal of the spectral radius of the graph (the maximum absolute eigenvalue of the adjacency matrix)
- `nonedgesonly` – boolean (default: `True`); if `True`, value for each edge present in the graph is set to zero
- `vertices` – list (default: `None`); the ordering of the vertices defining how they should appear in the matrix. By default, the ordering given by *`GenericGraph.vertices()`* is used.

OUTPUT: the Katz matrix of the graph with parameter alpha

EXAMPLES:

We find the Katz matrix of an undirected 4-cycle.

```
sage: G = graphs.CycleGraph(4)
sage: G.katz_matrix(1/20)                                                   #␣
↪needs sage.modules sage.rings.number_field
[1/198  5/99 1/198  5/99]
[ 5/99 1/198  5/99 1/198]
[1/198  5/99 1/198  5/99]
[ 5/99 1/198  5/99 1/198]
```

```
>>> from sage.all import *
>>> G = graphs.CycleGraph(Integer(4))
>>> G.katz_matrix(Integer(1)/Integer(20))                                   ␣
↪              # needs sage.modules sage.rings.number_field
[1/198  5/99 1/198  5/99]
[ 5/99 1/198  5/99 1/198]
[1/198  5/99 1/198  5/99]
[ 5/99 1/198  5/99 1/198]
```

We find the Katz matrix of an undirected 4-cycle with all entries other than those which correspond to non-edges zeroed out.

```
sage: G.katz_matrix(1/20, True)                                             #␣
↪needs sage.modules sage.rings.number_field
[    0     0 1/198     0]
[    0     0     0 1/198]
[1/198     0     0     0]
[    0     1/198 0     0]
```

```
>>> from sage.all import *
>>> G.katz_matrix(Integer(1)/Integer(20), True)                             ␣
↪              # needs sage.modules sage.rings.number_field
[    0     0 1/198     0]
[    0     0     0 1/198]
[1/198     0     0     0]
[    0     1/198 0     0]
```

This will give an error if alpha<=0 or alpha>=1/spectral_radius = 1/max (A.eigenvalues()).

We find the Katz matrix in a fan on 6 vertices.

```
sage: H = Graph([(0,1),(0,2),(0,3),(0,4),(0,5),(0,6),(1,2),(2,3),(3,4),(4,5)])
sage: H.katz_matrix(1/10)                                                   #␣
↪needs sage.modules sage.rings.number_field
[   169/2256    545/4512      25/188    605/4512      25/188    545/4512   ␣
↪485/4512]
[   545/4512 7081/297792  4355/37224    229/9024   595/37224 4073/297792   ␣
↪109/9024]
[     25/188  4355/37224    172/4653      45/376    125/4653   595/37224     ␣
↪ 5/376]
[   605/4512    229/9024      45/376    337/9024      45/376    229/9024   ␣
↪121/9024]
[     25/188   595/37224    125/4653      45/376    172/4653  4355/37224     ␣
↪ 5/376]
[   545/4512 4073/297792   595/37224    229/9024  4355/37224 7081/297792   ␣
↪109/9024]
[   485/4512    109/9024       5/376    121/9024       5/376    109/9024    ␣
↪97/9024]
```

```
>>> from sage.all import *
>>> H = Graph([(Integer(0),Integer(1)),(Integer(0),Integer(2)),(Integer(0),
↪Integer(3)),(Integer(0),Integer(4)),(Integer(0),Integer(5)),(Integer(0),
↪Integer(6)),(Integer(1),Integer(2)),(Integer(2),Integer(3)),(Integer(3),
↪Integer(4)),(Integer(4),Integer(5))])
>>> H.katz_matrix(Integer(1)/Integer(10))                                   ␣
↪              # needs sage.modules sage.rings.number_field
[   169/2256    545/4512      25/188    605/4512      25/188    545/4512   ␣
↪485/4512]
[   545/4512 7081/297792  4355/37224    229/9024   595/37224 4073/297792   ␣
↪109/9024]
[     25/188  4355/37224    172/4653      45/376    125/4653   595/37224     ␣
↪ 5/376]
[   605/4512    229/9024      45/376    337/9024      45/376    229/9024   ␣
↪121/9024]
[     25/188   595/37224    125/4653      45/376    172/4653  4355/37224     ␣
↪ 5/376]
[   545/4512 4073/297792   595/37224    229/9024  4355/37224 7081/297792   ␣
↪109/9024]
[   485/4512    109/9024       5/376    121/9024       5/376    109/9024    ␣
↪97/9024]
```

See also

- *katz_centrality()*
- Wikipedia article Katz_centrality

kirchhoff_matrix(*weighted=None*, *indegree=True*, *normalized=False*, *signless=False*, ***kwds*)

Return the Kirchhoff matrix (a.k.a. the Laplacian) of the graph.

The Kirchhoff matrix is defined to be $D + M$ if signless and $D - M$ otherwise, where D is the diagonal degree matrix (each diagonal entry is the degree of the corresponding vertex), and M is the adjacency matrix. If `normalized` is `True`, then the returned matrix is $D^{-1/2}(D + M)D^{-1/2}$ if signless and $D^{-1/2}(D - M)D^{-1/2}$ otherwise.

(In the special case of DiGraphs, D is defined as the diagonal in-degree matrix or diagonal out-degree matrix according to the value of `indegree`)

INPUT:

- `weighted` – boolean (default: `None`)
  - If `True`, the weighted adjacency matrix is used for M, and the diagonal matrix D takes into account the weight of edges (replace in the definition "degree" by "sum of the incident edges")
  - Else, each edge is assumed to have weight 1

  Default is to take weights into consideration if and only if the graph is weighted.
- `indegree` – boolean (default: `True`); this parameter is considered only for digraphs
  - If `True`, each diagonal entry of D is equal to the in-degree of the corresponding vertex
  - Else, each diagonal entry of D is equal to the out-degree of the corresponding vertex

  By default, `indegree` is set to `True`
- `normalized` – boolean (default: `False`)
  - If `True`, the returned matrix is $D^{-1/2}(D + M)D^{-1/2}$ for signless and $D^{-1/2}(D - M)D^{-1/2}$ otherwise, a normalized version of the Laplacian matrix. More accurately, the normalizing matrix used is equal to $D^{-1/2}$ only for non-isolated vertices. If vertex i is isolated, then diagonal entry i in the matrix is 1, rather than a division by zero
  - Else, the matrix $D + M$ for signless and $D - M$ otherwise is returned
- `signless` – boolean (default: `False`)
  - If `True`, $D + M$ is used in calculation of Kirchhoff matrix
  - Else, $D - M$ is used in calculation of Kirchhoff matrix
- `vertices` – list; `None`, or `True` (default: `None`)
  - when a list, the i-th row and column of the matrix correspond to the i-th vertex in the ordering of `vertices`,
  - when `None`, the i-th row and column of the matrix correspond to the i-th vertex in the ordering given by *GenericGraph.vertices()* with `sort=True`.
  - when `True`, construct an endomorphism of a free module instead of a matrix, where the module's basis is indexed by the vertices.

  If the vertices are not comparable, the keyword `vertices` must be used to specify an ordering, or a `TypeError` exception will be raised.

Note that any additional keywords will be passed on to either the *adjacency_matrix()* or *weighted_adjacency_matrix()* method.

AUTHORS:

- Tom Boothby
- Jason Grout

EXAMPLES:

```
sage: G = Graph(sparse=True)
sage: G.add_edges([(0, 1, 1), (1, 2, 2), (0, 2, 3), (0, 3, 4)])
sage: M = G.kirchhoff_matrix(weighted=True); M                                  #␣
↪needs sage.modules
[ 8 -1 -3 -4]
[-1  3 -2  0]
[-3 -2  5  0]
[-4  0  0  4]
sage: M = G.kirchhoff_matrix(); M                                               #␣
↪needs sage.modules
[ 3 -1 -1 -1]
[-1  2 -1  0]
[-1 -1  2  0]
[-1  0  0  1]
sage: M = G.laplacian_matrix(normalized=True); M                                #␣
↪needs sage.modules sage.symbolic
[                   1 -1/6*sqrt(3)*sqrt(2) -1/6*sqrt(3)*sqrt(2)         -1/
↪3*sqrt(3)]
[-1/6*sqrt(3)*sqrt(2)                    1                 -1/2              ␣
↪      0]
[-1/6*sqrt(3)*sqrt(2)                 -1/2                    1              ␣
↪      0]
[        -1/3*sqrt(3)                    0                    0              ␣
↪      1]
sage: M = G.kirchhoff_matrix(weighted=True, signless=True); M                   #␣
↪needs sage.modules
[8 1 3 4]
[1 3 2 0]
[3 2 5 0]
[4 0 0 4]

sage: G = Graph({0: [], 1: [2]})
sage: G.laplacian_matrix(normalized=True)                                       #␣
↪needs sage.modules
[ 0  0  0]
[ 0  1 -1]
[ 0 -1  1]
sage: G.laplacian_matrix(normalized=True, signless=True)                        #␣
↪needs sage.modules
[0 0 0]
[0 1 1]
[0 1 1]
```

```
>>> from sage.all import *
>>> G = Graph(sparse=True)
>>> G.add_edges([(Integer(0), Integer(1), Integer(1)), (Integer(1),␣
↪Integer(2), Integer(2)), (Integer(0), Integer(2), Integer(3)), (Integer(0),␣
↪Integer(3), Integer(4))])
>>> M = G.kirchhoff_matrix(weighted=True); M                                    #␣
↪needs sage.modules
[ 8 -1 -3 -4]
[-1  3 -2  0]
```

(continues on next page)

(continued from previous page)

```
[-3 -2  5  0]
[-4  0  0  4]
>>> M = G.kirchhoff_matrix(); M                                          #␣
↪needs sage.modules
[ 3 -1 -1 -1]
[-1  2 -1  0]
[-1 -1  2  0]
[-1  0  0  1]
>>> M = G.laplacian_matrix(normalized=True); M                           #␣
↪needs sage.modules sage.symbolic
[                   1 -1/6*sqrt(3)*sqrt(2) -1/6*sqrt(3)*sqrt(2)         -1/
↪3*sqrt(3)]
[-1/6*sqrt(3)*sqrt(2)                    1                 -1/2             ␣
↪      0]
[-1/6*sqrt(3)*sqrt(2)                -1/2                    1             ␣
↪      0]
[        -1/3*sqrt(3)                    0                    0             ␣
↪      1]
>>> M = G.kirchhoff_matrix(weighted=True, signless=True); M              #␣
↪needs sage.modules
[8 1 3 4]
[1 3 2 0]
[3 2 5 0]
[4 0 0 4]

>>> G = Graph({Integer(0): [], Integer(1): [Integer(2)]})
>>> G.laplacian_matrix(normalized=True)                                  #␣
↪needs sage.modules
[ 0  0  0]
[ 0  1 -1]
[ 0 -1  1]
>>> G.laplacian_matrix(normalized=True, signless=True)                   #␣
↪needs sage.modules
[0 0 0]
[0 1 1]
[0 1 1]
```

A weighted directed graph with loops, changing the variable `indegree`

```
sage: G = DiGraph({1: {1: 2, 2: 3}, 2: {1: 4}}, weighted=True, sparse=True)
sage: G.laplacian_matrix()                                               #␣
↪needs sage.modules
[ 4 -3]
[-4  3]
```

```
>>> from sage.all import *
>>> G = DiGraph({Integer(1): {Integer(1): Integer(2), Integer(2): Integer(3)},
↪ Integer(2): {Integer(1): Integer(4)}}, weighted=True, sparse=True)
>>> G.laplacian_matrix()                                                 #␣
↪needs sage.modules
[ 4 -3]
[-4  3]
```

```
sage: G = DiGraph({1: {1: 2, 2: 3}, 2: {1: 4}}, weighted=True, sparse=True)
sage: G.laplacian_matrix(indegree=False)                                    #␣
↪needs sage.modules
[ 3 -3]
[-4  4]
```

```
>>> from sage.all import *
>>> G = DiGraph({Integer(1): {Integer(1): Integer(2), Integer(2): Integer(3)},
↪ Integer(2): {Integer(1): Integer(4)}}, weighted=True, sparse=True)
>>> G.laplacian_matrix(indegree=False)                                      #␣
↪needs sage.modules
[ 3 -3]
[-4  4]
```

A different ordering of the vertices (see *adjacency_matrix()* and *weighted_adjacency_matrix()*):

```
sage: G = Graph(sparse=True)
sage: G.add_edges([(0, 1, 1), (1, 2, 2), (0, 2, 3), (0, 3, 4)])
sage: M = G.kirchhoff_matrix(vertices=[3, 2, 1, 0]); M                      #␣
↪needs sage.modules
[ 1  0  0 -1]
[ 0  2 -1 -1]
[ 0 -1  2 -1]
[-1 -1 -1  3]
sage: M = G.kirchhoff_matrix(weighted=True, vertices=[3, 2, 1, 0]); M       #␣
↪needs sage.modules
[ 4  0  0 -4]
[ 0  5 -2 -3]
[ 0 -2  3 -1]
[-4 -3 -1  8]
```

```
>>> from sage.all import *
>>> G = Graph(sparse=True)
>>> G.add_edges([(Integer(0), Integer(1), Integer(1)), (Integer(1),␣
↪Integer(2), Integer(2)), (Integer(0), Integer(2), Integer(3)), (Integer(0),␣
↪Integer(3), Integer(4))])
>>> M = G.kirchhoff_matrix(vertices=[Integer(3), Integer(2), Integer(1),␣
↪Integer(0)]); M                      # needs sage.modules
[ 1  0  0 -1]
[ 0  2 -1 -1]
[ 0 -1  2 -1]
[-1 -1 -1  3]
>>> M = G.kirchhoff_matrix(weighted=True, vertices=[Integer(3), Integer(2),␣
↪Integer(1), Integer(0)]); M       # needs sage.modules
[ 4  0  0 -4]
[ 0  5 -2 -3]
[ 0 -2  3 -1]
[-4 -3 -1  8]
```

When parameter `immutable=True` is passed, the output matrix is immutable:

```
sage: G = Graph([(0, 1)])
sage: M = G.kirchhoff_matrix(vertices=[0, 1], immutable=True)               #␣
```

(continues on next page)

(continued from previous page)

```
↪needs sage.modules
sage: M.is_immutable()                                                    #␣
↪needs sage.modules
True
```

```
>>> from sage.all import *
>>> G = Graph([(Integer(0), Integer(1))])
>>> M = G.kirchhoff_matrix(vertices=[Integer(0), Integer(1)], immutable=True)␣
↪              # needs sage.modules
>>> M.is_immutable()                                                      #␣
↪needs sage.modules
True
```

Creating a module morphism:

```
sage: G = Graph(sparse=True, weighted=True)
sage: G.add_edges([('A', 'B', 1), ('B', 'C', 2), ('A', 'C', 3), ('A', 'D',␣
↪4)])
sage: phi = G.laplacian_matrix(weighted=True, vertices=True); phi
Generic endomorphism of
 Free module generated by {'A', 'B', 'C', 'D'} over Integer Ring
sage: print(phi._unicode_art_matrix())
   A  B  C  D
A⎛ 8 -1 -3 -4⎞
B⎜-1  3 -2  0⎟
C⎜-3 -2  5  0⎟
D⎝-4  0  0  4⎠
```

```
>>> from sage.all import *
>>> G = Graph(sparse=True, weighted=True)
>>> G.add_edges([('A', 'B', Integer(1)), ('B', 'C', Integer(2)), ('A', 'C',␣
↪Integer(3)), ('A', 'D', Integer(4))])
>>> phi = G.laplacian_matrix(weighted=True, vertices=True); phi
Generic endomorphism of
 Free module generated by {'A', 'B', 'C', 'D'} over Integer Ring
>>> print(phi._unicode_art_matrix())
   A  B  C  D
A⎛ 8 -1 -3 -4⎞
B⎜-1  3 -2  0⎟
C⎜-3 -2  5  0⎟
D⎝-4  0  0  4⎠
```

kronecker_product(*other*, *immutable=None*)

alias of *tensor_product()*.

laplacian_matrix(*weighted=None*, *indegree=True*, *normalized=False*, *signless=False*, ***kwds*)

alias of *kirchhoff_matrix()*.

latex_options()

Return an instance of *GraphLatex* for the graph.

Changes to this object will affect the LaTeX version of the graph. For a full explanation of how to use LaTeX to render graphs, see the introduction to the *graph_latex* module.

EXAMPLES:

```
sage: g = graphs.PetersenGraph()
sage: opts = g.latex_options()
sage: opts
LaTeX options for Petersen graph: {}
sage: opts.set_option('tkz_style', 'Classic')                                #␣
↪needs sage.plot
sage: opts                                                                   #␣
↪needs sage.plot
LaTeX options for Petersen graph: {'tkz_style': 'Classic'}
```

```
>>> from sage.all import *
>>> g = graphs.PetersenGraph()
>>> opts = g.latex_options()
>>> opts
LaTeX options for Petersen graph: {}
>>> opts.set_option('tkz_style', 'Classic')                                  #␣
↪needs sage.plot
>>> opts                                                                     #␣
↪needs sage.plot
LaTeX options for Petersen graph: {'tkz_style': 'Classic'}
```

layout(*layout=None*, *pos=None*, *dim=2*, *save_pos=False*, ***options*)

Return a layout for the vertices of this graph.

INPUT:

- `layout` – string (default: `None`); specifies a layout algorithm among `'acyclic'`, `'acyclic_dummy'`, `'circular'`, `'ranked'`, `'graphviz'`, `'planar'`, `'tutte'`, `'spring'`, `'forest'` or `'tree'`
- `pos` – dictionary (default: `None`); a dictionary of positions
- `dim` – integer (default: 2); the number of dimensions of the layout, 2 or 3
- `save_pos` – boolean (default: `False`); whether to save the positions
- `**options` – layout options (see below)

If `layout` is set, the specified algorithm is used to compute the positions.

Otherwise, if `pos` is specified, use the given positions.

Otherwise, try to fetch previously computed and saved positions.

Otherwise use the default layout (usually the spring layout).

If `save_pos = True`, the layout is saved for later use.

EXAMPLES:

```
sage: g = digraphs.ButterflyGraph(1)
sage: D2 = g.layout(); D2  # random
{('0', 0): [2.69..., 0.43...],
 ('0', 1): [1.35..., 0.86...],
 ('1', 0): [0.89..., -0.42...],
 ('1', 1): [2.26..., -0.87...]}

sage: g.layout(layout='acyclic_dummy', save_pos=True)
```

(continues on next page)

(continued from previous page)

```
{('0', 0): [0.3..., 0],
 ('0', 1): [0.3..., 1],
 ('1', 0): [0.6..., 0],
 ('1', 1): [0.6..., 1]}
sage: g.get_pos()
{('0', 0): [0.3..., 0],
 ('0', 1): [0.3..., 1],
 ('1', 0): [0.6..., 0],
 ('1', 1): [0.6..., 1]}

sage: D3 = g.layout(dim=3); D3  # random
{('0', 0): [0.68..., 0.50..., -0.24...],
 ('0', 1): [1.02..., -0.02..., 0.93...],
 ('1', 0): [2.06..., -0.49..., 0.23...],
 ('1', 1): [1.74..., 0.01..., -0.92...]}
```

```
>>> from sage.all import *
>>> g = digraphs.ButterflyGraph(Integer(1))
>>> D2 = g.layout(); D2  # random
{('0', 0): [2.69..., 0.43...],
 ('0', 1): [1.35..., 0.86...],
 ('1', 0): [0.89..., -0.42...],
 ('1', 1): [2.26..., -0.87...]}

>>> g.layout(layout='acyclic_dummy', save_pos=True)
{('0', 0): [0.3..., 0],
 ('0', 1): [0.3..., 1],
 ('1', 0): [0.6..., 0],
 ('1', 1): [0.6..., 1]}
>>> g.get_pos()
{('0', 0): [0.3..., 0],
 ('0', 1): [0.3..., 1],
 ('1', 0): [0.6..., 0],
 ('1', 1): [0.6..., 1]}

>>> D3 = g.layout(dim=Integer(3)); D3  # random
{('0', 0): [0.68..., 0.50..., -0.24...],
 ('0', 1): [1.02..., -0.02..., 0.93...],
 ('1', 0): [2.06..., -0.49..., 0.23...],
 ('1', 1): [1.74..., 0.01..., -0.92...]}
```

Some safety tests:

```
sage: sorted(D2.keys()) == sorted(D3.keys()) == sorted(g)
True
sage: isinstance(D2, dict) and isinstance(D3, dict)
True
sage: [c in RDF for c in D2[('0', 0)]]
[True, True]
sage: [c in RDF for c in D3[('0', 0)]]
[True, True, True]
```

```
>>> from sage.all import *
>>> sorted(D2.keys()) == sorted(D3.keys()) == sorted(g)
True
>>> isinstance(D2, dict) and isinstance(D3, dict)
True
>>> [c in RDF for c in D2[('0', Integer(0))]]
[True, True]
>>> [c in RDF for c in D3[('0', Integer(0))]]
[True, True, True]
```

Here is the list of all the available layout options (`**options`):

```
sage: from sage.graphs.graph_plot import layout_options
sage: for key, value in sorted(layout_options.items()):
....:     print("option {} : {}".format(key, value))
option by_component : Whether to do the spring layout by connected component -
↪- boolean.
option dim : The dimension of the layout -- 2 or 3.
option external_face : A list of the vertices of the external face of the␣
↪graph if used for Tutte embedding layout; or an edge on the external face␣
↪if used for the 'planar' layout.
option external_face_pos : A dictionary specifying the positions of the␣
↪external face of the graph, used for Tutte embedding layout. If none␣
↪specified, the external face is a regular polygon.
option forest_roots : An iterable specifying which vertices to use as roots␣
↪for the ``layout='forest'`` option. If no root is specified for a tree,␣
↪then one is chosen close to the center of the tree. Ignored unless ``layout=
↪'forest'``.
option heights : A dictionary mapping heights to the list of vertices at this␣
↪height.
option iterations : The number of times to execute the spring layout␣
↪algorithm.
option layout : A layout algorithm -- one of: ...
option prog : Which graphviz layout program to use -- one of "circo", "dot",
↪"fdp", "neato", or "twopi".
option save_pos : Whether or not to save the computed position for the graph.
option spring : Use spring layout to finalize the current 'ranked' layout.
option tree_orientation : The direction of tree branches -- 'up', 'down',
↪'left' or 'right'.
option tree_root : A vertex designation for drawing trees. A vertex of the␣
↪tree to be used as the root for the ``layout='tree'`` option. If no root is␣
↪specified, then one is chosen close to the center of the tree. Ignored␣
↪unless ``layout='tree'``.
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_plot import layout_options
>>> for key, value in sorted(layout_options.items()):
...     print("option {} : {}".format(key, value))
option by_component : Whether to do the spring layout by connected component -
↪- boolean.
option dim : The dimension of the layout -- 2 or 3.
option external_face : A list of the vertices of the external face of the␣
↪graph if used for Tutte embedding layout; or an edge on the external face␣
```

(continues on next page)

(continued from previous page)

```
↪if used for the 'planar' layout.
option external_face_pos : A dictionary specifying the positions of the␣
↪external face of the graph, used for Tutte embedding layout. If none␣
↪specified, the external face is a regular polygon.
option forest_roots : An iterable specifying which vertices to use as roots␣
↪for the ``layout='forest'`` option. If no root is specified for a tree,␣
↪then one is chosen close to the center of the tree. Ignored unless ``layout=
↪'forest'``.
option heights : A dictionary mapping heights to the list of vertices at this␣
↪height.
option iterations : The number of times to execute the spring layout␣
↪algorithm.
option layout : A layout algorithm -- one of: ...
option prog : Which graphviz layout program to use -- one of "circo", "dot",
↪"fdp", "neato", or "twopi".
option save_pos : Whether or not to save the computed position for the graph.
option spring : Use spring layout to finalize the current 'ranked' layout.
option tree_orientation : The direction of tree branches -- 'up', 'down',
↪'left' or 'right'.
option tree_root : A vertex designation for drawing trees. A vertex of the␣
↪tree to be used as the root for the ``layout='tree'`` option. If no root is␣
↪specified, then one is chosen close to the center of the tree. Ignored␣
↪unless ``layout='tree'``.
```

Some of them only apply to certain layout algorithms. For details, see *layout_acyclic()*, *layout_planar()*, *layout_circular()*, *layout_spring()*, ...

Warning

unknown optional arguments are silently ignored

Warning

`graphviz` and `dot2tex` are currently required to obtain a nice `'acyclic'` layout. See *layout_graphviz()* for installation instructions.

A subclass may implement another layout algorithm `'blah'`, by implementing a method `.layout_blah`. It may override the default layout by overriding *layout_default()*, and similarly override the predefined layouts.

Todo

use this feature for all the predefined graphs classes (like for the Petersen graph, ...), rather than systematically building the layout at construction time.

layout_circular(*dim=2*, *center=(0, 0)*, *radius=1*, *shift=0*, *angle=0*, ***options*)

Return a circular layout for this graph.

INPUT:

- `dim` – integer (default: 2); the number of dimensions of the layout, 2 or 3
- `center` – tuple (default: `(0, 0)`); position of the center of the circle
- `radius` – (default: 1) the radius of the circle
- `shift` – (default: 0) rotation of the circle. A value of `shift=1` will replace in the drawing the i-th element of the list by the $(i-1)$-th. Non-integer values are admissible, and a value of α corresponds to a rotation of the circle by an angle of $\alpha 2\pi/n$ (where n is the number of vertices set on the circle).
- `angle` – (default: 0) rotate the embedding of all vertices. For instance, when `angle == 0`, the first vertex get position `(center[0] + radius, center[1])`. With a value of $\pi/2$, the first vertex get position `(center[0], center[1] + radius)`.
- `**options` – other parameters not used here

OUTPUT: a dictionary mapping vertices to positions

EXAMPLES:

```
sage: G = graphs.CirculantGraph(7, [1, 3])
sage: G.layout_circular()
{0: (0.0, 1.0),
 1: (-0.78...,  0.62...),
 2: (-0.97..., -0.22...),
 3: (-0.43..., -0.90...),
 4: (0.43...,  -0.90...),
 5: (0.97...,  -0.22...),
 6: (0.78...,   0.62...)}
sage: G.plot(layout='circular')                                    #␣
↪needs sage.plot
Graphics object consisting of 22 graphics primitives
```

```
>>> from sage.all import *
>>> G = graphs.CirculantGraph(Integer(7), [Integer(1), Integer(3)])
>>> G.layout_circular()
{0: (0.0, 1.0),
 1: (-0.78...,  0.62...),
 2: (-0.97..., -0.22...),
 3: (-0.43..., -0.90...),
 4: (0.43...,  -0.90...),
 5: (0.97...,  -0.22...),
 6: (0.78...,   0.62...)}
>>> G.plot(layout='circular')                                      #␣
↪needs sage.plot
Graphics object consisting of 22 graphics primitives
```

layout_default(*by_component=True*, ***options*)

alias of *layout_spring()*.

layout_extend_randomly(*pos*, *dim=2*)

Extend randomly a partial layout.

INPUT:

- `pos` – dictionary mapping vertices to positions
- `dim` – integer (default: 2); the number of dimensions of the layout, 2 or 3

OUTPUT: a dictionary mapping vertices to positions

The vertices not referenced in `pos` are assigned random positions within the box delimited by the other vertices.

EXAMPLES:

```
sage: H = digraphs.ButterflyGraph(1)
sage: pos = {('0', 0): (0, 0), ('1', 1): (1, 1)}
sage: H.layout_extend_randomly(pos)  # random
{('0', 0): (0, 0),
 ('0', 1): [0.0446..., 0.332...],
 ('1', 0): [0.1114..., 0.514...],
 ('1', 1): (1, 1)}
sage: xmin, xmax, ymin, ymax = H._layout_bounding_box(pos)
sage: (xmin, ymin) == (0, 0) and (xmax, ymax) == (1, 1)
True
```

```
>>> from sage.all import *
>>> H = digraphs.ButterflyGraph(Integer(1))
>>> pos = {('0', Integer(0)): (Integer(0), Integer(0)), ('1', Integer(1)):␣
↪(Integer(1), Integer(1))}
>>> H.layout_extend_randomly(pos)  # random
{('0', 0): (0, 0),
 ('0', 1): [0.0446..., 0.332...],
 ('1', 0): [0.1114..., 0.514...],
 ('1', 1): (1, 1)}
>>> xmin, xmax, ymin, ymax = H._layout_bounding_box(pos)
>>> (xmin, ymin) == (Integer(0), Integer(0)) and (xmax, ymax) == (Integer(1),␣
↪Integer(1))
True
```

layout_forest(*tree_orientation='down'*, *forest_roots=None*, ***options*)

Return an ordered forest layout for this graph.

The function relies on *layout_tree()* to deal with each connected component.

INPUT:

- `forest_roots` – an iterable of vertices (default: `None`); the root vertices of the trees in the forest; a vertex is chosen close to the center of each component for which no root is specified in `forest_roots` or if `forest_roots` is `None`
- `tree_orientation` – string (default: `'down'`); the direction in which the tree is growing, can be `'up'`, `'down'`, `'left'` or `'right'`
- `**options` – other parameters ignored here

EXAMPLES:

```
sage: G = graphs.RandomTree(4) + graphs.RandomTree(5) + graphs.RandomTree(6)
sage: p = G.layout_forest()
sage: G.plot(pos=p)  # random                                                #␣
↪needs sage.plot
Graphics object consisting of 28 graphics primitives

sage: P5 = graphs.PathGraph(5)
```

(continues on next page)

(continued from previous page)

```
sage: H = P5 + P5 + graphs.BalancedTree(2,2)                                #␣
↪needs networkx
sage: p = H.layout_forest(forest_roots=[14,3])                              #␣
↪needs networkx
sage: H.plot(pos=p)                                                         #␣
↪needs networkx sage.plot
Graphics object consisting of 32 graphics primitives
```

```
>>> from sage.all import *
>>> G = graphs.RandomTree(Integer(4)) + graphs.RandomTree(Integer(5)) +␣
↪graphs.RandomTree(Integer(6))
>>> p = G.layout_forest()
>>> G.plot(pos=p)  # random                                                 #␣
↪needs sage.plot
Graphics object consisting of 28 graphics primitives

>>> P5 = graphs.PathGraph(Integer(5))
>>> H = P5 + P5 + graphs.BalancedTree(Integer(2),Integer(2))                 ␣
↪              # needs networkx
>>> p = H.layout_forest(forest_roots=[Integer(14),Integer(3)])               ␣
↪              # needs networkx
>>> H.plot(pos=p)                                                           #␣
↪needs networkx sage.plot
Graphics object consisting of 32 graphics primitives
```

layout_graphviz(*dim=2*, *prog='dot'*, ***options*)

Call `graphviz` to compute a layout of the vertices of this graph.

INPUT:

- `dim` – integer (default: 2); the number of dimensions of the layout, 2 or 3
- `prog` – one of “dot”, “neato”, “twopi”, “circo”, or “fdp”
- `**options` – other parameters used by method *graphviz_string()*

EXAMPLES:

```
sage: g = digraphs.ButterflyGraph(2)
sage: g.layout_graphviz()  # optional - dot2tex graphviz
{('...', ...): [...,...],
 ('...', ...): [...,...],
 ('...', ...): [...,...],
 ('...', ...): [...,...],
 ('...', ...): [...,...],
 ('...', ...): [...,...],
 ('...', ...): [...,...],
 ('...', ...): [...,...],
 ('...', ...): [...,...],
 ('...', ...): [...,...],
 ('...', ...): [...,...],
 ('...', ...): [...,...]}
sage: g.plot(layout='graphviz')  # optional - dot2tex graphviz
Graphics object consisting of 29 graphics primitives
```

```
>>> from sage.all import *
>>> g = digraphs.ButterflyGraph(Integer(2))
>>> g.layout_graphviz()  # optional - dot2tex graphviz
{('...', ...): [...,...],
 ('...', ...): [...,...],
 ('...', ...): [...,...],
 ('...', ...): [...,...],
 ('...', ...): [...,...],
 ('...', ...): [...,...],
 ('...', ...): [...,...],
 ('...', ...): [...,...],
 ('...', ...): [...,...],
 ('...', ...): [...,...],
 ('...', ...): [...,...],
 ('...', ...): [...,...]}
>>> g.plot(layout='graphviz')  # optional - dot2tex graphviz
Graphics object consisting of 29 graphics primitives
```

Note: the actual coordinates are not deterministic

By default, an acyclic layout is computed using `graphviz`'s `dot` layout program. One may specify an alternative layout program:

```
sage: # optional - dot2tex graphviz
sage: g.plot(layout='graphviz', prog='dot')
Graphics object consisting of 29 graphics primitives
sage: g.plot(layout='graphviz', prog='neato')
Graphics object consisting of 29 graphics primitives
sage: g.plot(layout='graphviz', prog='twopi')
Graphics object consisting of 29 graphics primitives
sage: g.plot(layout='graphviz', prog='fdp')
Graphics object consisting of 29 graphics primitives
sage: g = graphs.BalancedTree(5,2)                                    #␣
↪needs networkx
sage: g.plot(layout='graphviz', prog='circo')                         #␣
↪needs networkx
Graphics object consisting of 62 graphics primitives
```

```
>>> from sage.all import *
>>> # optional - dot2tex graphviz
>>> g.plot(layout='graphviz', prog='dot')
Graphics object consisting of 29 graphics primitives
>>> g.plot(layout='graphviz', prog='neato')
Graphics object consisting of 29 graphics primitives
>>> g.plot(layout='graphviz', prog='twopi')
Graphics object consisting of 29 graphics primitives
>>> g.plot(layout='graphviz', prog='fdp')
Graphics object consisting of 29 graphics primitives
>>> g = graphs.BalancedTree(Integer(5),Integer(2))                    ␣
↪              # needs networkx
>>> g.plot(layout='graphviz', prog='circo')                          #␣
↪needs networkx
Graphics object consisting of 62 graphics primitives
```

Todo

Put here some cool examples showcasing graphviz features.

This requires `graphviz` and the `dot2tex` spkg. Here are some installation tips:

- Install `graphviz` >= 2.14 so that the programs `dot`, `neato`, etc. are in your path. The graphviz suite can be download from http://graphviz.org.
- Install `dot2tex` with `sage -i dot2tex`

Todo

Use the graphviz functionality of Networkx 1.0 once it will be merged into Sage.

layout_planar(*set_embedding=False*, *on_embedding=None*, *external_face=None*, *test=False*, *circular=False*, ***options*)

Compute a planar layout of the graph using Schnyder's algorithm.

If `set_embedding` is set, a new combinatorial embedding is computed for the layout. Otherwise: if `on_embedding` is provided, then that combinatorial embedding is used for the layout. Otherwise: if a combinatorial embedding is set to the instance field variable of the graph (e.g. using *`set_embedding()`*), then that one is used, and if no combinatorial embedding is set, then one is computed.

If the graph is not planar, an error is raised.

INPUT:

- `set_embedding` – boolean (default: `False`); whether to set the instance field variable that contains a combinatorial embedding to the combinatorial embedding used for the planar layout (see *`get_embedding()`*)
- `on_embedding` – dictionary (default: `None`); provide a combinatorial embedding
- `external_face` – a pair (u, v) of vertices (default: `None`); the external face of the drawing is chosen in such a way that u and v are consecutive vertices in the clockwise traversal of the external face, in particular uv has to be an edge of the graph. If `external_face == None`, an arbitrary external face is chosen.
- `test` – boolean (default: `False`); whether to perform sanity tests along the way
- `circular` – ignored

EXAMPLES:

```
sage: g = graphs.PathGraph(10)
sage: g.layout(layout='planar', save_pos=True, test=True)
{0: [0, 8],
 1: [8, 1],
 2: [1, 0],
 3: [7, 1],
 4: [1, 1],
 5: [5, 3],
 6: [2, 3],
 7: [2, 4],
 8: [1, 6],
```

(continues on next page)

(continued from previous page)

```
 9: [2, 5]}
sage: g = graphs.BalancedTree(3, 4)                                          #␣
↪needs networkx
sage: pos = g.layout(layout='planar', save_pos=True, test=True)              #␣
↪needs networkx
sage: pos[0]                                                                 #␣
↪needs networkx
[0, 119]
sage: pos[120]                                                               #␣
↪needs networkx
[21, 37]
sage: g = graphs.CycleGraph(7)
sage: g.layout(layout='planar', save_pos=True, test=True)
{0: [0, 5], 1: [5, 1], 2: [1, 0], 3: [4, 1], 4: [1, 1], 5: [2, 2], 6: [1, 2]}
sage: g = graphs.CompleteGraph(5)
sage: g.layout(layout='planar', save_pos=True, test=True, set_embedding=True)
Traceback (most recent call last):
...
ValueError: Complete graph is not a planar graph
```

```
>>> from sage.all import *
>>> g = graphs.PathGraph(Integer(10))
>>> g.layout(layout='planar', save_pos=True, test=True)
{0: [0, 8],
 1: [8, 1],
 2: [1, 0],
 3: [7, 1],
 4: [1, 1],
 5: [5, 3],
 6: [2, 3],
 7: [2, 4],
 8: [1, 6],
 9: [2, 5]}
>>> g = graphs.BalancedTree(Integer(3), Integer(4))                          ␣
↪              # needs networkx
>>> pos = g.layout(layout='planar', save_pos=True, test=True)              #␣
↪needs networkx
>>> pos[Integer(0)]                                                          ␣
↪     # needs networkx
[0, 119]
>>> pos[Integer(120)]                                                        ␣
↪     # needs networkx
[21, 37]
>>> g = graphs.CycleGraph(Integer(7))
>>> g.layout(layout='planar', save_pos=True, test=True)
{0: [0, 5], 1: [5, 1], 2: [1, 0], 3: [4, 1], 4: [1, 1], 5: [2, 2], 6: [1, 2]}
>>> g = graphs.CompleteGraph(Integer(5))
>>> g.layout(layout='planar', save_pos=True, test=True, set_embedding=True)
Traceback (most recent call last):
...
ValueError: Complete graph is not a planar graph
```

Choose the external face of the drawing:

```
sage: g = graphs.CompleteGraph(4)
sage: g.layout(layout='planar', external_face=(0,1))
{0: [0, 2], 1: [2, 1], 2: [1, 0], 3: [1, 1]}
sage: g.layout(layout='planar', external_face=(3,1))
{0: [2, 1], 1: [0, 2], 2: [1, 1], 3: [1, 0]}
```

```
>>> from sage.all import *
>>> g = graphs.CompleteGraph(Integer(4))
>>> g.layout(layout='planar', external_face=(Integer(0),Integer(1)))
{0: [0, 2], 1: [2, 1], 2: [1, 0], 3: [1, 1]}
>>> g.layout(layout='planar', external_face=(Integer(3),Integer(1)))
{0: [2, 1], 1: [0, 2], 2: [1, 1], 3: [1, 0]}
```

Choose the embedding:

```
sage: H = graphs.LadderGraph(4)
sage: em = {0:[1,4], 4:[0,5], 1:[5,2,0], 5:[4,6,1],
....:       2:[1,3,6], 6:[7,5,2], 3:[7,2], 7:[3,6]}
sage: p = H.layout_planar(on_embedding=em)
sage: p  # random
{2: [8.121320343559642, 1],
 3: [2.1213203435596424, 6],
 7: [3.1213203435596424, 0],
 0: [5.121320343559642, 3],
 1: [3.1213203435596424, 5],
 4: [4.121320343559642, 3],
 5: [4.121320343559642, 2],
 6: [3.1213203435596424, 1],
 9: [9.698670612749268, 1],
 8: [8.698670612749268, 1],
 10: [9.698670612749268, 0]}
```

```
>>> from sage.all import *
>>> H = graphs.LadderGraph(Integer(4))
>>> em = {Integer(0):[Integer(1),Integer(4)], Integer(4):[Integer(0),
↪Integer(5)], Integer(1):[Integer(5),Integer(2),Integer(0)],␣
↪Integer(5):[Integer(4),Integer(6),Integer(1)],
...       Integer(2):[Integer(1),Integer(3),Integer(6)],␣
↪Integer(6):[Integer(7),Integer(5),Integer(2)], Integer(3):[Integer(7),
↪Integer(2)], Integer(7):[Integer(3),Integer(6)]}
>>> p = H.layout_planar(on_embedding=em)
>>> p  # random
{2: [8.121320343559642, 1],
 3: [2.1213203435596424, 6],
 7: [3.1213203435596424, 0],
 0: [5.121320343559642, 3],
 1: [3.1213203435596424, 5],
 4: [4.121320343559642, 3],
 5: [4.121320343559642, 2],
 6: [3.1213203435596424, 1],
 9: [9.698670612749268, 1],
```

(continues on next page)

(continued from previous page)

```
 8: [8.698670612749268, 1],
 10: [9.698670612749268, 0]}
```

layout_ranked(*heights=None*, *dim=2*, *spring=False*, ***options*)

Return a ranked layout for this graph.

INPUT:

- `heights` – dictionary (default: `None`); a dictionary mapping heights to the list of vertices at this height
- `dim` – integer (default: 2); the number of dimensions of the layout, 2 or 3
- `spring` – boolean (default: `False`)
- `**options` – options for method *spring_layout_fast()*

OUTPUT: a dictionary mapping vertices to positions

Returns a layout computed by randomly arranging the vertices along the given heights

EXAMPLES:

```
sage: g = graphs.LadderGraph(3)
sage: g.layout_ranked(heights={i: (i, i+3) for i in range(3)})  # random
{0: [0.668..., 0],
 1: [0.667..., 1],
 2: [0.677..., 2],
 3: [1.34..., 0],
 4: [1.33..., 1],
 5: [1.33..., 2]}
sage: g = graphs.LadderGraph(7)
sage: g.plot(layout='ranked', heights={i: (i, i+7) for i in range(7)})      #␣
↪needs sage.plot
Graphics object consisting of 34 graphics primitives
```

```
>>> from sage.all import *
>>> g = graphs.LadderGraph(Integer(3))
>>> g.layout_ranked(heights={i: (i, i+Integer(3)) for i in range(Integer(3))}
↪)  # random
{0: [0.668..., 0],
 1: [0.667..., 1],
 2: [0.677..., 2],
 3: [1.34..., 0],
 4: [1.33..., 1],
 5: [1.33..., 2]}
>>> g = graphs.LadderGraph(Integer(7))
>>> g.plot(layout='ranked', heights={i: (i, i+Integer(7)) for i in␣
↪range(Integer(7))})      # needs sage.plot
Graphics object consisting of 34 graphics primitives
```

layout_spring(*by_component=True*, ***options*)

Return a spring layout for this graph.

INPUT:

- `by_components` – boolean (default: `True`)

- **`**options` – options for method**
  spring_layout_fast()

OUTPUT: a dictionary mapping vertices to positions

EXAMPLES:

```
sage: g = graphs.LadderGraph(3) #TODO!!!!
sage: g.layout_spring()  # random
{0: [1.0, -0.29...],
1: [1.64..., 0.30...],
2: [2.34..., 0.89...],
3: [1.49..., -0.83...],
4: [2.14..., -0.30...],
5: [2.80..., 0.22...]}
sage: g = graphs.LadderGraph(7)
sage: g.plot(layout='spring')                                                  #␣
↪needs sage.plot
Graphics object consisting of 34 graphics primitives
```

```
>>> from sage.all import *
>>> g = graphs.LadderGraph(Integer(3)) #TODO!!!!
>>> g.layout_spring()  # random
{0: [1.0, -0.29...],
1: [1.64..., 0.30...],
2: [2.34..., 0.89...],
3: [1.49..., -0.83...],
4: [2.14..., -0.30...],
5: [2.80..., 0.22...]}
>>> g = graphs.LadderGraph(Integer(7))
>>> g.plot(layout='spring')                                                    #␣
↪needs sage.plot
Graphics object consisting of 34 graphics primitives
```

`layout_tree`(*tree_orientation='down'*, *tree_root=None*, *dim=2*, ***options*)

Return an ordered tree layout for this graph.

The graph must be a tree (no non-oriented cycles). In case of doubt whether the graph is connected or not, prefer *layout_forest()*.

INPUT:

- `tree_root` – a vertex (default: `None`); the root vertex of the tree. By default (`None`) a vertex is chosen close to the center of the tree.
- `tree_orientation` – string (default: `'down'`); the direction in which the tree is growing, can be `'up'`, `'down'`, `'left'` or `'right'`
- `dim` – integer (default: 2); the number of dimensions of the layout, 2 or 3
- `**options` – other parameters not used here

If the tree has been given a planar embedding (fixed circular order on the set of neighbors of every vertex) using `set_embedding`, the algorithm will create a layout that respects this embedding.

OUTPUT: a dictionary mapping vertices to positions

EXAMPLES:

```
sage: G = graphs.RandomTree(80)
sage: G.plot(layout='tree', tree_orientation='right')                          #␣
↪needs sage.plot
Graphics object consisting of 160 graphics primitives

sage: T = graphs.RandomLobster(25, 0.3, 0.3)                                   #␣
↪needs networkx
sage: T.show(layout='tree', tree_orientation='up')                             #␣
↪needs networkx sage.plot

sage: G = graphs.HoffmanSingletonGraph()
sage: T = Graph()
sage: T.add_edges(G.min_spanning_tree(starting_vertex=0))
sage: T.show(layout='tree', tree_root=0)                                       #␣
↪needs sage.plot

sage: G = graphs.BalancedTree(2, 2)                                            #␣
↪needs networkx
sage: G.layout_tree(tree_root=0)                                               #␣
↪needs networkx
{0: [1.5, 0],
 1: [2.5, -1],
 2: [0.5, -1],
 3: [3.0, -2],
 4: [2.0, -2],
 5: [1.0, -2],
 6: [0.0, -2]}

sage: G = graphs.BalancedTree(2, 4)                                            #␣
↪needs networkx
sage: G.plot(layout='tree', tree_root=0, tree_orientation='up')                #␣
↪needs networkx sage.plot
Graphics object consisting of 62 graphics primitives
```

```
>>> from sage.all import *
>>> G = graphs.RandomTree(Integer(80))
>>> G.plot(layout='tree', tree_orientation='right')                            #␣
↪needs sage.plot
Graphics object consisting of 160 graphics primitives

>>> T = graphs.RandomLobster(Integer(25), RealNumber('0.3'), RealNumber('0.3
↪'))                                    # needs networkx
>>> T.show(layout='tree', tree_orientation='up')                               #␣
↪needs networkx sage.plot

>>> G = graphs.HoffmanSingletonGraph()
>>> T = Graph()
>>> T.add_edges(G.min_spanning_tree(starting_vertex=Integer(0)))
>>> T.show(layout='tree', tree_root=Integer(0))                                ␣
↪     # needs sage.plot

>>> G = graphs.BalancedTree(Integer(2), Integer(2))                            ␣
```

(continues on next page)

(continued from previous page)

```
↪              # needs networkx
>>> G.layout_tree(tree_root=Integer(0))                                    ␣
↪     # needs networkx
{0: [1.5, 0],
 1: [2.5, -1],
 2: [0.5, -1],
 3: [3.0, -2],
 4: [2.0, -2],
 5: [1.0, -2],
 6: [0.0, -2]}

>>> G = graphs.BalancedTree(Integer(2), Integer(4))                        ␣
↪              # needs networkx
>>> G.plot(layout='tree', tree_root=Integer(0), tree_orientation='up')     ␣
↪     # needs networkx sage.plot
Graphics object consisting of 62 graphics primitives
```

Using the embedding when it exists:

```
sage: T = Graph([(0, 1), (0, 6), (0, 3), (1, 2), (1, 5), (3, 4), (3, 7), (3,␣
↪8)])
sage: T.set_embedding({0: [1, 6, 3], 1: [2, 5, 0], 2: [1], 3: [4, 7, 8, 0],
....:     4: [3], 5: [1], 6: [0], 7: [3], 8: [3]})
sage: T.layout_tree()
{0: [2.166..., 0],
 1: [3.5, -1],
 2: [4.0, -2],
 3: [1.0, -1],
 4: [2.0, -2],
 5: [3.0, -2],
 6: [2.0, -1],
 7: [1.0, -2],
 8: [0.0, -2]}
sage: T.plot(layout='tree', tree_root=3)                                  #␣
↪needs sage.plot
Graphics object consisting of 18 graphics primitives
```

```
>>> from sage.all import *
>>> T = Graph([(Integer(0), Integer(1)), (Integer(0), Integer(6)),␣
↪(Integer(0), Integer(3)), (Integer(1), Integer(2)), (Integer(1),␣
↪Integer(5)), (Integer(3), Integer(4)), (Integer(3), Integer(7)),␣
↪(Integer(3), Integer(8))])
>>> T.set_embedding({Integer(0): [Integer(1), Integer(6), Integer(3)],␣
↪Integer(1): [Integer(2), Integer(5), Integer(0)], Integer(2): [Integer(1)],␣
↪Integer(3): [Integer(4), Integer(7), Integer(8), Integer(0)],
...     Integer(4): [Integer(3)], Integer(5): [Integer(1)], Integer(6):␣
↪[Integer(0)], Integer(7): [Integer(3)], Integer(8): [Integer(3)]})
>>> T.layout_tree()
{0: [2.166..., 0],
 1: [3.5, -1],
 2: [4.0, -2],
 3: [1.0, -1],
```

(continues on next page)

(continued from previous page)

```
 4: [2.0, -2],
 5: [3.0, -2],
 6: [2.0, -1],
 7: [1.0, -2],
 8: [0.0, -2]}
>>> T.plot(layout='tree', tree_root=Integer(3))                              ␣
↪     # needs sage.plot
Graphics object consisting of 18 graphics primitives
```

layout_tutte(*external_face=None*, *external_face_pos=None*, ***options*)

Compute graph layout based on a Tutte embedding.

The graph must be 3-connected and planar.

INPUT:

- `external_face` – list (default: `None`); the external face to be made a polygon
- `external_face_pos` – dictionary (default: `None`); the positions of the vertices of the external face. If `None`, will automatically generate a unit sided regular polygon.
- `**options` – other parameters not used here

OUTPUT: a dictionary mapping vertices to positions

EXAMPLES:

```
sage: g = graphs.WheelGraph(n=7)
sage: g.plot(layout='tutte', external_face=[0,1,2])                          #␣
↪needs sage.plot
Graphics object consisting of 20 graphics primitives
sage: g = graphs.CubeGraph(n=3, embedding=2)
sage: g.plot(layout='tutte', external_face=['101','111','001',
....:       '011'], external_face_pos={'101':(1,0), '111':(0,0),
....:       '001':(2,1), '011':(-1,1)})                                      #␣
↪needs sage.plot
Graphics object consisting of 21 graphics primitives
sage: g = graphs.CompleteGraph(n=5)
sage: g.plot(layout='tutte', external_face=[0,1,2])
Traceback (most recent call last):
...
ValueError: graph must be planar
sage: g = graphs.CycleGraph(n=10)
sage: g.layout(layout='tutte', external_face=[0,1,2,3,4,5,6,7,8,9])
Traceback (most recent call last):
...
ValueError: graph must be 3-connected
```

```
>>> from sage.all import *
>>> g = graphs.WheelGraph(n=Integer(7))
>>> g.plot(layout='tutte', external_face=[Integer(0),Integer(1),Integer(2)]) ␣
↪                      # needs sage.plot
Graphics object consisting of 20 graphics primitives
>>> g = graphs.CubeGraph(n=Integer(3), embedding=Integer(2))
>>> g.plot(layout='tutte', external_face=['101','111','001',
```

(continues on next page)

(continued from previous page)

```
...       '011'], external_face_pos={'101':(Integer(1),Integer(0)), '111
↪':(Integer(0),Integer(0)),
...       '001':(Integer(2),Integer(1)), '011':(-Integer(1),Integer(1))})
↪                                # needs sage.plot
Graphics object consisting of 21 graphics primitives
>>> g = graphs.CompleteGraph(n=Integer(5))
>>> g.plot(layout='tutte', external_face=[Integer(0),Integer(1),Integer(2)])
Traceback (most recent call last):
...
ValueError: graph must be planar
>>> g = graphs.CycleGraph(n=Integer(10))
>>> g.layout(layout='tutte', external_face=[Integer(0),Integer(1),Integer(2),
↪Integer(3),Integer(4),Integer(5),Integer(6),Integer(7),Integer(8),
↪Integer(9)])
Traceback (most recent call last):
...
ValueError: graph must be 3-connected
```

lex_BFS(*G*, *reverse=False*, *tree=False*, *initial_vertex=None*, *algorithm='fast'*)

Perform a lexicographic breadth first search (LexBFS) on the graph.

INPUT:

- `G` – a sage graph
- `reverse` – boolean (default: `False`); whether to return the vertices in discovery order, or the reverse
- `tree` – boolean (default: `False`); whether to return the discovery directed tree (each vertex being linked to the one that saw it last)
- `initial_vertex` – (default: `None`) the first vertex to consider
- `algorithm` – string (default: `'fast'`); algorithm to use among:
  - `'slow'` – it use the generic algorithm for all the lexicographic searchs. See the documentation of the *traversals* module for more details.
  - `'fast'` – this algorithm uses the notion of *partition refinement* to determine the position of the vertices in the ordering. The time complexity of this algorithm is in $O(n + m)$, and our implementation follows that complexity for `SparseGraph`. For `DenseGraph`, the complexity is $O(n^2)$. See [HMPV2000] and [TCHP2008] for more details. This algorithm is also used to compute slice decompositions of undirected graphs, a more thorough description can be found in the documentation of the *slice_decomposition* module.

Note

Loops and multiple edges are ignored during the computation of `lex_BFS` and directed graphs are converted to undirected graphs.

See also

- Wikipedia article Lexicographic_breadth-first_search

- *slice_decomposition* module and *slice_decomposition()* – compute a slice decomposition of the graph using an extended lex BFS algorithm
- *lex_DFS()* – perform a lexicographic depth first search (LexDFS) on the graph
- *lex_UP()* – perform a lexicographic UP search (LexUP) on the graph
- *lex_DOWN()* – perform a lexicographic DOWN search (LexDOWN) on the graph

EXAMPLES:

A Lex BFS is obviously an ordering of the vertices:

```
sage: g = graphs.CompleteGraph(6)
sage: set(g.lex_BFS()) == set(g)
True
```

```
>>> from sage.all import *
>>> g = graphs.CompleteGraph(Integer(6))
>>> set(g.lex_BFS()) == set(g)
True
```

Lex BFS ordering of the 3-sun graph:

```
sage: g = Graph([(1, 2), (1, 3), (2, 3), (2, 4), (2, 5), (3, 5), (3, 6), (4,␣
↪5), (5, 6)])
sage: g.lex_BFS()
[1, 2, 3, 5, 4, 6]
```

```
>>> from sage.all import *
>>> g = Graph([(Integer(1), Integer(2)), (Integer(1), Integer(3)),␣
↪(Integer(2), Integer(3)), (Integer(2), Integer(4)), (Integer(2),␣
↪Integer(5)), (Integer(3), Integer(5)), (Integer(3), Integer(6)),␣
↪(Integer(4), Integer(5)), (Integer(5), Integer(6))])
>>> g.lex_BFS()
[1, 2, 3, 5, 4, 6]
```

The method also works for directed graphs:

```
sage: G = DiGraph([(1, 2), (2, 3), (1, 3)])
sage: correct_anwsers = [[2, 1, 3], [2, 3, 1]]
sage: G.lex_BFS(initial_vertex=2, algorithm='slow') in correct_anwsers
True
sage: G.lex_BFS(initial_vertex=2, algorithm='fast') in correct_anwsers
True
```

```
>>> from sage.all import *
>>> G = DiGraph([(Integer(1), Integer(2)), (Integer(2), Integer(3)),␣
↪(Integer(1), Integer(3))])
>>> correct_anwsers = [[Integer(2), Integer(1), Integer(3)], [Integer(2),␣
↪Integer(3), Integer(1)]]
>>> G.lex_BFS(initial_vertex=Integer(2), algorithm='slow') in correct_anwsers
True
```

(continues on next page)

(continued from previous page)

```
>>> G.lex_BFS(initial_vertex=Integer(2), algorithm='fast') in correct_anwsers
True
```

For a Chordal Graph, a reversed Lex BFS is a Perfect Elimination Order:

```
sage: g = graphs.PathGraph(3).lexicographic_product(graphs.CompleteGraph(2))
sage: g.lex_BFS(reverse=True)
[(2, 1), (2, 0), (1, 1), (1, 0), (0, 1), (0, 0)]
```

```
>>> from sage.all import *
>>> g = graphs.PathGraph(Integer(3)).lexicographic_product(graphs.
↪CompleteGraph(Integer(2)))
>>> g.lex_BFS(reverse=True)
[(2, 1), (2, 0), (1, 1), (1, 0), (0, 1), (0, 0)]
```

And the vertices at the end of the tree of discovery are, for chordal graphs, simplicial vertices (their neighborhood is a complete graph):

```
sage: g = graphs.ClawGraph().lexicographic_product(graphs.CompleteGraph(2))
sage: v = g.lex_BFS()[-1]
sage: peo, tree = g.lex_BFS(initial_vertex = v,  tree=True)
sage: leaves = [v for v in tree if tree.in_degree(v) ==0]
sage: all(g.subgraph(g.neighbors(v)).is_clique() for v in leaves)
True
```

```
>>> from sage.all import *
>>> g = graphs.ClawGraph().lexicographic_product(graphs.
↪CompleteGraph(Integer(2)))
>>> v = g.lex_BFS()[-Integer(1)]
>>> peo, tree = g.lex_BFS(initial_vertex = v,  tree=True)
>>> leaves = [v for v in tree if tree.in_degree(v) ==Integer(0)]
>>> all(g.subgraph(g.neighbors(v)).is_clique() for v in leaves)
True
```

Different orderings for different traversals:

```
sage: G = digraphs.DeBruijn(2,3)
sage: G.lex_BFS(initial_vertex='000', algorithm='fast')
['000', '001', '100', '010', '011', '110', '101', '111']
sage: G.lex_BFS(initial_vertex='000', algorithm='slow')
['000', '001', '100', '010', '011', '110', '101', '111']
sage: G.lex_DFS(initial_vertex='000')
['000', '001', '100', '010', '101', '110', '011', '111']
sage: G.lex_UP(initial_vertex='000')
['000', '001', '010', '101', '110', '111', '011', '100']
sage: G.lex_DOWN(initial_vertex='000')
['000', '001', '100', '011', '010', '110', '111', '101']
```

```
>>> from sage.all import *
>>> G = digraphs.DeBruijn(Integer(2),Integer(3))
>>> G.lex_BFS(initial_vertex='000', algorithm='fast')
['000', '001', '100', '010', '011', '110', '101', '111']
```

(continues on next page)

(continued from previous page)

```
>>> G.lex_BFS(initial_vertex='000', algorithm='slow')
['000', '001', '100', '010', '011', '110', '101', '111']
>>> G.lex_DFS(initial_vertex='000')
['000', '001', '100', '010', '101', '110', '011', '111']
>>> G.lex_UP(initial_vertex='000')
['000', '001', '010', '101', '110', '111', '011', '100']
>>> G.lex_DOWN(initial_vertex='000')
['000', '001', '100', '011', '010', '110', '111', '101']
```

lex_DFS(*G*, *reverse=False*, *tree=False*, *initial_vertex=None*)

Perform a lexicographic depth first search (LexDFS) on the graph.

INPUT:

- `G` – a sage graph
- `reverse` – boolean (default: `False`); whether to return the vertices in discovery order, or the reverse
- `tree` – boolean (default: `False`); whether to return the discovery directed tree (each vertex being linked to the one that saw it last)
- `initial_vertex` – (default: `None`) the first vertex to consider

Note

Loops and multiple edges are ignored during the computation of `lex_DFS` and directed graphs are converted to undirected graphs.

See also

- *lex_BFS()* – perform a lexicographic breadth first search (LexBFS) on the graph
- *lex_UP()* – perform a lexicographic UP search (LexUP) on the graph
- *lex_DOWN()* – perform a lexicographic DOWN search (LexDOWN) on the graph

ALGORITHM:

See the documentation of the *traversals* module.

EXAMPLES:

A Lex DFS is obviously an ordering of the vertices:

```
sage: g = graphs.CompleteGraph(6)
sage: set(g.lex_DFS()) == set(g)
True
```

```
>>> from sage.all import *
>>> g = graphs.CompleteGraph(Integer(6))
>>> set(g.lex_DFS()) == set(g)
True
```

Lex DFS ordering of the 3-sun graph:

```
sage: g = Graph([(1, 2), (1, 3), (2, 3), (2, 4), (2, 5), (3, 5), (3, 6), (4,␣
↪5), (5, 6)])
sage: g.lex_DFS()
[1, 2, 3, 5, 6, 4]
```

```
>>> from sage.all import *
>>> g = Graph([(Integer(1), Integer(2)), (Integer(1), Integer(3)),␣
↪(Integer(2), Integer(3)), (Integer(2), Integer(4)), (Integer(2),␣
↪Integer(5)), (Integer(3), Integer(5)), (Integer(3), Integer(6)),␣
↪(Integer(4), Integer(5)), (Integer(5), Integer(6))])
>>> g.lex_DFS()
[1, 2, 3, 5, 6, 4]
```

The method also works for directed graphs:

```
sage: G = DiGraph([(1, 2), (2, 3), (1, 3)])
sage: correct_anwsers = [[2, 1, 3], [2, 3, 1]]
sage: G.lex_DFS(initial_vertex=2) in correct_anwsers
True
```

```
>>> from sage.all import *
>>> G = DiGraph([(Integer(1), Integer(2)), (Integer(2), Integer(3)),␣
↪(Integer(1), Integer(3))])
>>> correct_anwsers = [[Integer(2), Integer(1), Integer(3)], [Integer(2),␣
↪Integer(3), Integer(1)]]
>>> G.lex_DFS(initial_vertex=Integer(2)) in correct_anwsers
True
```

Different orderings for different traversals:

```
sage: G = digraphs.DeBruijn(2,3)
sage: G.lex_BFS(initial_vertex='000')
['000', '001', '100', '010', '011', '110', '101', '111']
sage: G.lex_DFS(initial_vertex='000')
['000', '001', '100', '010', '101', '110', '011', '111']
sage: G.lex_UP(initial_vertex='000')
['000', '001', '010', '101', '110', '111', '011', '100']
sage: G.lex_DOWN(initial_vertex='000')
['000', '001', '100', '011', '010', '110', '111', '101']
```

```
>>> from sage.all import *
>>> G = digraphs.DeBruijn(Integer(2),Integer(3))
>>> G.lex_BFS(initial_vertex='000')
['000', '001', '100', '010', '011', '110', '101', '111']
>>> G.lex_DFS(initial_vertex='000')
['000', '001', '100', '010', '101', '110', '011', '111']
>>> G.lex_UP(initial_vertex='000')
['000', '001', '010', '101', '110', '111', '011', '100']
>>> G.lex_DOWN(initial_vertex='000')
['000', '001', '100', '011', '010', '110', '111', '101']
```

lex_DOWN(*G*, *reverse=False*, *tree=False*, *initial_vertex=None*)

Perform a lexicographic DOWN search (LexDOWN) on the graph.

INPUT:

- `G` – a sage graph
- `reverse` – boolean (default: `False`); whether to return the vertices in discovery order, or the reverse
- `tree` – boolean (default: `False`); whether to return the discovery directed tree (each vertex being linked to the one that saw it)
- `initial_vertex` – (default: `None`) the first vertex to consider

Note

Loops and multiple edges are ignored during the computation of `lex_DOWN` and directed graphs are converted to undirected graphs.

See also

- *lex_BFS()* – perform a lexicographic breadth first search (LexBFS) on the graph
- *lex_DFS()* – perform a lexicographic depth first search (LexDFS) on the graph
- *lex_UP()* – perform a lexicographic UP search (LexUP) on the graph

ALGORITHM:

See the documentation of the *traversals* module.

EXAMPLES:

A Lex DOWN is obviously an ordering of the vertices:

```
sage: g = graphs.CompleteGraph(6)
sage: set(g.lex_DOWN()) == set(g)
True
```

```
>>> from sage.all import *
>>> g = graphs.CompleteGraph(Integer(6))
>>> set(g.lex_DOWN()) == set(g)
True
```

Lex DOWN ordering of the 3-sun graph:

```
sage: g = Graph([(1, 2), (1, 3), (2, 3), (2, 4), (2, 5), (3, 5), (3, 6), (4,␣
↪5), (5, 6)])
sage: g.lex_DOWN()
[1, 2, 3, 4, 6, 5]
```

```
>>> from sage.all import *
>>> g = Graph([(Integer(1), Integer(2)), (Integer(1), Integer(3)),␣
↪(Integer(2), Integer(3)), (Integer(2), Integer(4)), (Integer(2),␣
↪Integer(5)), (Integer(3), Integer(5)), (Integer(3), Integer(6)),␣
↪(Integer(4), Integer(5)), (Integer(5), Integer(6))])
>>> g.lex_DOWN()
[1, 2, 3, 4, 6, 5]
```

The method also works for directed graphs:

```
sage: G = DiGraph([(1, 2), (2, 3), (1, 3)])
sage: correct_anwsers = [[2, 1, 3], [2, 3, 1]]
sage: G.lex_DOWN(initial_vertex=2) in correct_anwsers
True
```

```
>>> from sage.all import *
>>> G = DiGraph([(Integer(1), Integer(2)), (Integer(2), Integer(3)),␣
↪(Integer(1), Integer(3))])
>>> correct_anwsers = [[Integer(2), Integer(1), Integer(3)], [Integer(2),␣
↪Integer(3), Integer(1)]]
>>> G.lex_DOWN(initial_vertex=Integer(2)) in correct_anwsers
True
```

Different orderings for different traversals:

```
sage: G = digraphs.DeBruijn(2,3)
sage: G.lex_BFS(initial_vertex='000')
['000', '001', '100', '010', '011', '110', '101', '111']
sage: G.lex_DFS(initial_vertex='000')
['000', '001', '100', '010', '101', '110', '011', '111']
sage: G.lex_UP(initial_vertex='000')
['000', '001', '010', '101', '110', '111', '011', '100']
sage: G.lex_DOWN(initial_vertex='000')
['000', '001', '100', '011', '010', '110', '111', '101']
```

```
>>> from sage.all import *
>>> G = digraphs.DeBruijn(Integer(2),Integer(3))
>>> G.lex_BFS(initial_vertex='000')
['000', '001', '100', '010', '011', '110', '101', '111']
>>> G.lex_DFS(initial_vertex='000')
['000', '001', '100', '010', '101', '110', '011', '111']
>>> G.lex_UP(initial_vertex='000')
['000', '001', '010', '101', '110', '111', '011', '100']
>>> G.lex_DOWN(initial_vertex='000')
['000', '001', '100', '011', '010', '110', '111', '101']
```

lex_UP(*G*, *reverse=False*, *tree=False*, *initial_vertex=None*)

Perform a lexicographic UP search (LexUP) on the graph.

INPUT:

- `G` – a sage graph
- `reverse` – boolean (default: `False`); whether to return the vertices in discovery order, or the reverse
- `tree` – boolean (default: `False`); whether to return the discovery directed tree (each vertex being linked to the one that saw it last)
- `initial_vertex` – (default: `None`) the first vertex to consider

Note

Loops and multiple edges are ignored during the computation of `lex_UP` and directed graphs are converted to undirected graphs.

See also

- *lex_BFS()* – perform a lexicographic breadth first search (LexBFS) on the graph
- *lex_DFS()* – perform a lexicographic depth first search (LexDFS) on the graph
- *lex_DOWN()* – perform a lexicographic DOWN search (LexDOWN) on the graph

ALGORITHM:

See the documentation of the *traversals* module.

EXAMPLES:

A Lex UP is obviously an ordering of the vertices:

```
sage: g = graphs.CompleteGraph(6)
sage: set(g.lex_UP()) == set(g)
True
```

```
>>> from sage.all import *
>>> g = graphs.CompleteGraph(Integer(6))
>>> set(g.lex_UP()) == set(g)
True
```

Lex UP ordering of the 3-sun graph:

```
sage: g = Graph([(1, 2), (1, 3), (2, 3), (2, 4), (2, 5), (3, 5), (3, 6), (4,␣
↪5), (5, 6)])
sage: g.lex_UP()
[1, 2, 4, 5, 6, 3]
```

```
>>> from sage.all import *
>>> g = Graph([(Integer(1), Integer(2)), (Integer(1), Integer(3)),␣
↪(Integer(2), Integer(3)), (Integer(2), Integer(4)), (Integer(2),␣
↪Integer(5)), (Integer(3), Integer(5)), (Integer(3), Integer(6)),␣
↪(Integer(4), Integer(5)), (Integer(5), Integer(6))])
>>> g.lex_UP()
[1, 2, 4, 5, 6, 3]
```

The method also works for directed graphs:

```
sage: G = DiGraph([(1, 2), (2, 3), (1, 3)])
sage: correct_anwsers = [[2, 1, 3], [2, 3, 1]]
sage: G.lex_UP(initial_vertex=2) in correct_anwsers
True
```

```
>>> from sage.all import *
>>> G = DiGraph([(Integer(1), Integer(2)), (Integer(2), Integer(3)),␣
```

(continues on next page)

(continued from previous page)

```
↪(Integer(1), Integer(3))])
>>> correct_anwsers = [[Integer(2), Integer(1), Integer(3)], [Integer(2),␣
↪Integer(3), Integer(1)]]
>>> G.lex_UP(initial_vertex=Integer(2)) in correct_anwsers
True
```

Different orderings for different traversals:

```
sage: G = digraphs.DeBruijn(2,3)
sage: G.lex_BFS(initial_vertex='000')
['000', '001', '100', '010', '011', '110', '101', '111']
sage: G.lex_DFS(initial_vertex='000')
['000', '001', '100', '010', '101', '110', '011', '111']
sage: G.lex_UP(initial_vertex='000')
['000', '001', '010', '101', '110', '111', '011', '100']
sage: G.lex_DOWN(initial_vertex='000')
['000', '001', '100', '011', '010', '110', '111', '101']
```

```
>>> from sage.all import *
>>> G = digraphs.DeBruijn(Integer(2),Integer(3))
>>> G.lex_BFS(initial_vertex='000')
['000', '001', '100', '010', '011', '110', '101', '111']
>>> G.lex_DFS(initial_vertex='000')
['000', '001', '100', '010', '101', '110', '011', '111']
>>> G.lex_UP(initial_vertex='000')
['000', '001', '010', '101', '110', '111', '011', '100']
>>> G.lex_DOWN(initial_vertex='000')
['000', '001', '100', '011', '010', '110', '111', '101']
```

lexicographic_product(*other*, *immutable=None*)

Return the lexicographic product of `self` and `other`.

The lexicographic product of G and H is the graph L with vertex set $V(L) = V(G) \times V(H)$, and $((u, v), (w, x))$ is an edge iff :

- (u, w) is an edge of G, or
- $u = w$ and (v, x) is an edge of H.

INPUT:

- `other` – a graph or a digraph
- `immutable` – boolean (default: `None`); whether to create a mutable/immutable product. `immutable=None` (default) means that the graphs and their product will behave the same way. If only one of them is immutable, the product will be mutable.

EXAMPLES:

```
sage: Z = graphs.CompleteGraph(2)
sage: C = graphs.CycleGraph(5)
sage: L = C.lexicographic_product(Z); L
Graph on 10 vertices
sage: L.plot()                          # long time                          #␣
↪needs sage.plot
Graphics object consisting of 36 graphics primitives
```

```
>>> from sage.all import *
>>> Z = graphs.CompleteGraph(Integer(2))
>>> C = graphs.CycleGraph(Integer(5))
>>> L = C.lexicographic_product(Z); L
Graph on 10 vertices
>>> L.plot()                          # long time                          #␣
↪needs sage.plot
Graphics object consisting of 36 graphics primitives
```

```
sage: D = graphs.DodecahedralGraph()
sage: P = graphs.PetersenGraph()
sage: L = D.lexicographic_product(P); L
Graph on 200 vertices
sage: L.plot()                           # long time                           #␣
↪needs sage.plot
Graphics object consisting of 3501 graphics primitives
```

```
>>> from sage.all import *
>>> D = graphs.DodecahedralGraph()
>>> P = graphs.PetersenGraph()
>>> L = D.lexicographic_product(P); L
Graph on 200 vertices
>>> L.plot()                          # long time                          #␣
↪needs sage.plot
Graphics object consisting of 3501 graphics primitives
```

line_graph(*g*, *labels=True*, *return_labels=False*, *immutable=None*)

Return the line graph of the (di)graph `g` (multiedges and loops allowed).

INPUT:

- `labels` – boolean (default: `True`); whether edge labels should be taken in consideration. If `labels=True`, the vertices of the line graph will be triples `(u,v,label)`, and pairs of vertices otherwise. In case of multiple edges, the vertices of the line graph will be triples `(u,v,an integer)`.
- `return_labels` – boolean (default: `False`); whether edge labels should be stored or not. If g has multiple edges and if `return_labels=True`, the method returns a list the first element of which is the line-graph of g and the second element is a dictionary {vertex of the line-graph: former corresponding edge with its original label}. If `return_labels=False`, the method returns only the line-graph.
- `immutable` – boolean (default: `None`); whether to create a mutable/immutable (di)graph. `immutable=None` (default) means that the (di)graph and its line (di)graph will behave the same way.

The line graph of an undirected graph G is an undirected simple graph H such that the vertices of H are the edges of G and two vertices e and f of H are adjacent if e and f share a common vertex in G. In other words, an edge in H represents a path of length 2 in G.

Loops are not adjacent to themselves.

The line graph of a directed graph G is a directed graph H such that the vertices of H are the edges of G and two vertices e and f of H are adjacent if e and f share a common vertex in G and the terminal vertex of e is the initial vertex of f. In other words, an edge in H represents a (directed) path of length 2 in G.

Note

As a *Graph* object only accepts hashable objects as vertices (and as the vertices of the line graph are the edges of the graph), this code will fail if edge labels are not hashable. You can also set the argument `labels=False` to ignore labels.

See also

- The *line_graph* module
- *line_graph_forbidden_subgraphs()* – the forbidden subgraphs of a line graph
- *is_line_graph()* – tests whether a graph is a line graph

EXAMPLES:

```
sage: g = graphs.CompleteGraph(4)
sage: h = g.line_graph()
sage: h.vertices(sort=True)
[(0, 1, None),
 (0, 2, None),
 (0, 3, None),
 (1, 2, None),
 (1, 3, None),
 (2, 3, None)]
sage: h.am()                                                                #␣
↪needs sage.modules
[0 1 1 1 1 0]
[1 0 1 1 0 1]
[1 1 0 0 1 1]
[1 1 0 0 1 1]
[1 0 1 1 0 1]
[0 1 1 1 1 0]
sage: h2 = g.line_graph(labels=False)
sage: h2.vertices(sort=True)
[(0, 1), (0, 2), (0, 3), (1, 2), (1, 3), (2, 3)]
sage: h2.am() == h.am()                                                     #␣
↪needs sage.modules
True
sage: g = DiGraph([[1..4], lambda i,j: i < j])
sage: h = g.line_graph()
sage: h.vertices(sort=True)
[(1, 2, None),
 (1, 3, None),
 (1, 4, None),
 (2, 3, None),
 (2, 4, None),
 (3, 4, None)]
sage: h.edges(sort=True)
[((1, 2, None), (2, 3, None), None),
 ((1, 2, None), (2, 4, None), None),
```

(continues on next page)

(continued from previous page)

```
 ((1, 3, None), (3, 4, None), None),
 ((2, 3, None), (3, 4, None), None)]
```

```
>>> from sage.all import *
>>> g = graphs.CompleteGraph(Integer(4))
>>> h = g.line_graph()
>>> h.vertices(sort=True)
[(0, 1, None),
 (0, 2, None),
 (0, 3, None),
 (1, 2, None),
 (1, 3, None),
 (2, 3, None)]
>>> h.am()                                                                  #␣
↪needs sage.modules
[0 1 1 1 1 0]
[1 0 1 1 0 1]
[1 1 0 0 1 1]
[1 1 0 0 1 1]
[1 0 1 1 0 1]
[0 1 1 1 1 0]
>>> h2 = g.line_graph(labels=False)
>>> h2.vertices(sort=True)
[(0, 1), (0, 2), (0, 3), (1, 2), (1, 3), (2, 3)]
>>> h2.am() == h.am()                                                       #␣
↪needs sage.modules
True
>>> g = DiGraph([(ellipsis_range(Integer(1),Ellipsis,Integer(4))), lambda i,
↪j: i < j])
>>> h = g.line_graph()
>>> h.vertices(sort=True)
[(1, 2, None),
 (1, 3, None),
 (1, 4, None),
 (2, 3, None),
 (2, 4, None),
 (3, 4, None)]
>>> h.edges(sort=True)
[((1, 2, None), (2, 3, None), None),
 ((1, 2, None), (2, 4, None), None),
 ((1, 3, None), (3, 4, None), None),
 ((2, 3, None), (3, 4, None), None)]
```

Examples with multiple edges:

```
sage: L = Graph([(0,1),(0,1),(1,2)],multiedges=True).line_graph()
sage: L.edges()
[((0, 1, 0), (0, 1, 1), None), ((0, 1, 1), (1, 2, 2), None),
((0, 1, 0), (1, 2, 2), None)]
sage: G = Graph([(0,1),(0,1,'a'),(0,1,'b'),(0,2),(1,2,'c')],
....: multiedges=True)
sage: L = G.line_graph(False,True)
```

(continues on next page)

(continued from previous page)

```
sage: L[0].edges()
[((0, 1, 1), (0, 1, 2), None), ((0, 1, 0), (0, 1, 2), None),
((0, 1, 2), (0, 2, 3), None), ((0, 1, 2), (1, 2, 4), None), ((0, 1, 0),
(0, 1, 1), None), ((0, 1, 1), (0, 2, 3), None), ((0, 1, 1),
(1, 2, 4), None), ((0, 1, 0), (0, 2, 3), None), ((0, 1, 0),
(1, 2, 4), None), ((0, 2, 3), (1, 2, 4), None)]
sage: L[1]
{(0, 1, 0): (0, 1, None),
(0, 1, 1): (0, 1, 'b'),
(0, 1, 2): (0, 1, 'a'),
(0, 2, 3): (0, 2, None),
(1, 2, 4): (1, 2, 'c')}
sage: g = DiGraph([(0,1),(0,1),(1,2)],multiedges=True)
sage: g.line_graph().edges()
[((0, 1, 1), (1, 2, 2), None), ((0, 1, 0), (1, 2, 2), None)]
```

```
>>> from sage.all import *
>>> L = Graph([(Integer(0),Integer(1)),(Integer(0),Integer(1)),(Integer(1),
↪Integer(2))],multiedges=True).line_graph()
>>> L.edges()
[((0, 1, 0), (0, 1, 1), None), ((0, 1, 1), (1, 2, 2), None),
((0, 1, 0), (1, 2, 2), None)]
>>> G = Graph([(Integer(0),Integer(1)),(Integer(0),Integer(1),'a'),
↪(Integer(0),Integer(1),'b'),(Integer(0),Integer(2)),(Integer(1),Integer(2),
↪'c')],
... multiedges=True)
>>> L = G.line_graph(False,True)
>>> L[Integer(0)].edges()
[((0, 1, 1), (0, 1, 2), None), ((0, 1, 0), (0, 1, 2), None),
((0, 1, 2), (0, 2, 3), None), ((0, 1, 2), (1, 2, 4), None), ((0, 1, 0),
(0, 1, 1), None), ((0, 1, 1), (0, 2, 3), None), ((0, 1, 1),
(1, 2, 4), None), ((0, 1, 0), (0, 2, 3), None), ((0, 1, 0),
(1, 2, 4), None), ((0, 2, 3), (1, 2, 4), None)]
>>> L[Integer(1)]
{(0, 1, 0): (0, 1, None),
(0, 1, 1): (0, 1, 'b'),
(0, 1, 2): (0, 1, 'a'),
(0, 2, 3): (0, 2, None),
(1, 2, 4): (1, 2, 'c')}
>>> g = DiGraph([(Integer(0),Integer(1)),(Integer(0),Integer(1)),(Integer(1),
↪Integer(2))],multiedges=True)
>>> g.line_graph().edges()
[((0, 1, 1), (1, 2, 2), None), ((0, 1, 0), (1, 2, 2), None)]
```

An example with a loop:

```
sage: g = Graph([(0,0),(0,1),(0,2),(1,2)],multiedges=True,loops=True)
sage: L = g.line_graph()
sage: L.edges()
[((0, 0, None), (0, 1, None), None), ((0, 0, None), (0, 2, None), None),
((0, 1, None), (0, 2, None), None), ((0, 1, None), (1, 2, None), None),
((0, 2, None), (1, 2, None), None)]
```

```
>>> from sage.all import *
>>> g = Graph([(Integer(0),Integer(0)),(Integer(0),Integer(1)),(Integer(0),
↪Integer(2)),(Integer(1),Integer(2))],multiedges=True,loops=True)
>>> L = g.line_graph()
>>> L.edges()
[((0, 0, None), (0, 1, None), None), ((0, 0, None), (0, 2, None), None),
((0, 1, None), (0, 2, None), None), ((0, 1, None), (1, 2, None), None),
((0, 2, None), (1, 2, None), None)]
```

longest_cycle(*induced*, *use_edge_labels=False*, *immutable=False*, *solver=None*, *verbose=None*, *integrality_tolerance=0*)

Return the longest (induced) cycle of `self`.

This method uses an integer linear programming formulation based on subtour elimination constraints to find the longest cycle. This cycle is *elementary* (or *simple*), and so without repeated vertices. When searching for an *induced* cycle (i.e., a cycle without chord), it uses in addition cycle elimination constraints as proposed in [MMRS2022].

We assume that the longest cycle in graph has at least 3 vertices and at least 2 in a digraph. The longest induced cycle as at least 4 vertices in both a graph and a digraph.

Note

Graphs and digraphs with loops or multiple edges are currently not accepted. It is certainly possible to extend the method to accept them.

INPUT:

- `induced` – boolean (default: `False`); whether to return the longest induced cycle or the longest cycle
- `use_edge_labels` – boolean (default: `False`); whether to compute a cycle with maximum weight where the weight of an edge is defined by its label (a label set to `None` or `{}` being considered as a weight of 1), or to compute a cycle with the largest possible number of edges (i.e., edge weights are set to 1)
- `immutable` – boolean (default: `None`); whether to create a mutable/immutable (di)graph. `immutable=None` (default) means that the (di)graph and its longest cycle will behave the same way.
- `solver` – string (default: `None`); specifies a Mixed Integer Linear Programming (MILP) solver to be used. If set to `None`, the default one is used. For more information on MILP solvers and which default solver is used, see the method `solve` of the class `MixedIntegerLinearProgram`.
- `verbose` – integer (default: 0); sets the level of verbosity. Set to 0 by default, which means quiet.
- `integrality_tolerance` – float; parameter for use with MILP solvers over an inexact base ring; see `MixedIntegerLinearProgram.get_values()`

OUTPUT:

A subgraph of `self` corresponding to a (directed if `self` is directed) longest (induced) cycle. If `use_edge_labels == True`, a pair `weight, cycle` is returned.

EXAMPLES:

Longest (induced) cycle of a graph:

```
sage: G = graphs.Grid2dGraph(3, 4)
sage: G.longest_cycle(induced=False)
```

(continues on next page)

(continued from previous page)

```
longest cycle from 2D Grid Graph for [3, 4]: Graph on 12 vertices
sage: G.longest_cycle(induced=True)
longest induced cycle from 2D Grid Graph for [3, 4]: Graph on 10 vertices
```

```
>>> from sage.all import *
>>> G = graphs.Grid2dGraph(Integer(3), Integer(4))
>>> G.longest_cycle(induced=False)
longest cycle from 2D Grid Graph for [3, 4]: Graph on 12 vertices
>>> G.longest_cycle(induced=True)
longest induced cycle from 2D Grid Graph for [3, 4]: Graph on 10 vertices
```

Longest (induced) cycle in a digraph:

```
sage: D = digraphs.Circuit(8)
sage: D.add_edge(0, 2)
sage: D.longest_cycle(induced=False)
longest cycle from Circuit: Digraph on 8 vertices
sage: D.longest_cycle(induced=True)
longest induced cycle from Circuit: Digraph on 7 vertices
sage: D.add_edge(1, 0)
sage: D.longest_cycle(induced=False)
longest cycle from Circuit: Digraph on 8 vertices
sage: D.longest_cycle(induced=True)
longest induced cycle from Circuit: Digraph on 7 vertices
sage: D.add_edge(2, 0)
sage: D.longest_cycle(induced=False)
longest cycle from Circuit: Digraph on 8 vertices
sage: D.longest_cycle(induced=True)
longest induced cycle from Circuit: Digraph on 0 vertices
```

```
>>> from sage.all import *
>>> D = digraphs.Circuit(Integer(8))
>>> D.add_edge(Integer(0), Integer(2))
>>> D.longest_cycle(induced=False)
longest cycle from Circuit: Digraph on 8 vertices
>>> D.longest_cycle(induced=True)
longest induced cycle from Circuit: Digraph on 7 vertices
>>> D.add_edge(Integer(1), Integer(0))
>>> D.longest_cycle(induced=False)
longest cycle from Circuit: Digraph on 8 vertices
>>> D.longest_cycle(induced=True)
longest induced cycle from Circuit: Digraph on 7 vertices
>>> D.add_edge(Integer(2), Integer(0))
>>> D.longest_cycle(induced=False)
longest cycle from Circuit: Digraph on 8 vertices
>>> D.longest_cycle(induced=True)
longest induced cycle from Circuit: Digraph on 0 vertices
```

Longest (induced) cycle when considering edge weights:

```
sage: D = digraphs.Circuit(15)
sage: for u, v in D.edges(labels=False):
```

(continues on next page)

(continued from previous page)

```
....:     D.set_edge_label(u, v, 1)
sage: D.add_edge(0, 10, 50)
sage: D.add_edge(11, 1, 1)
sage: D.add_edge(13, 0, 1)
sage: D.longest_cycle(induced=False, use_edge_labels=False)
longest cycle from Circuit: Digraph on 15 vertices
sage: D.longest_cycle(induced=False, use_edge_labels=True)
(55, longest cycle from Circuit: Digraph on 6 vertices)
sage: D.longest_cycle(induced=True, use_edge_labels=False)
longest induced cycle from Circuit: Digraph on 11 vertices
sage: D.longest_cycle(induced=True, use_edge_labels=True)
(54, longest induced cycle from Circuit: Digraph on 5 vertices)
```

```
>>> from sage.all import *
>>> D = digraphs.Circuit(Integer(15))
>>> for u, v in D.edges(labels=False):
...     D.set_edge_label(u, v, Integer(1))
>>> D.add_edge(Integer(0), Integer(10), Integer(50))
>>> D.add_edge(Integer(11), Integer(1), Integer(1))
>>> D.add_edge(Integer(13), Integer(0), Integer(1))
>>> D.longest_cycle(induced=False, use_edge_labels=False)
longest cycle from Circuit: Digraph on 15 vertices
>>> D.longest_cycle(induced=False, use_edge_labels=True)
(55, longest cycle from Circuit: Digraph on 6 vertices)
>>> D.longest_cycle(induced=True, use_edge_labels=False)
longest induced cycle from Circuit: Digraph on 11 vertices
>>> D.longest_cycle(induced=True, use_edge_labels=True)
(54, longest induced cycle from Circuit: Digraph on 5 vertices)
```

longest_path(*s*, *t=None*, *use_edge_labels=None*, *algorithm=False*, *immutable='MILP'*, *solver=None*, *verbose=None*, *integrality_tolerance=0*)

Return a longest path of `self`.

INPUT:

- `s` – a vertex (default: `None`); forces the source of the path (the method then returns the longest path starting at `s`). The argument is set to `None` by default, which means that no constraint is set upon the first vertex in the path.

  This parameter can only be used when `algorithm` is `'MILP'`.

- `t` – a vertex (default: `None`); forces the destination of the path (the method then returns the longest path ending at `t`). The argument is set to `None` by default, which means that no constraint is set upon the last vertex in the path.

  This parameter can only be used when `algorithm` is `'MILP'`.

- `use_edge_labels` – boolean (default: `False`); whether to compute a path with maximum weight where the weight of an edge is defined by its label (a label set to `None` or `{}` being considered as a weight of 1), or to compute a path with the longest possible number of edges (i.e., edge weights are set to 1)

  This parameter can only be used when `algorithm` is `'MILP'`.

- `algorithm` – string (default: `'MILP'`); the algorithm to use among `'MILP'` and `'heuristic'`:

  - `'MILP'` returns an exact answer.

  - `'heuristic'` is a randomized heuristic for finding a long path in an unweighted (di)graph. This heuristic does not take into account parameters `s`, `t` and `use_edge_labels`. An error is raised if these parameters are set.

- `immutable` – boolean (default: `None`); whether to create a mutable/immutable (di)graph. `immutable=None` (default) means that the (di)graph and its longest path will behave the same way.
- `solver` – string (default: `None`); specifies a Mixed Integer Linear Programming (MILP) solver to be used. If set to `None`, the default one is used. For more information on MILP solvers and which default solver is used, see the method `solve` of the class `MixedIntegerLinearProgram`.
- `verbose` – integer (default: 0); sets the level of verbosity. Set to 0 by default, which means quiet.
- `integrality_tolerance` – float; parameter for use with MILP solvers over an inexact base ring; see `MixedIntegerLinearProgram.get_values()`.

Note

The length of a path is assumed to be the number of its edges, or the sum of their labels (when `use_edge_labels == True`).

OUTPUT:

A subgraph of `self` corresponding to a (directed if `self` is directed) longest path. If `use_edge_labels == True`, a pair `weight, path` is returned.

ALGORITHM:

Mixed Integer Linear Programming (this problem is known to be NP-Hard).

EXAMPLES:

Petersen's graph being hypohamiltonian, it has a longest path of length $n - 2$:

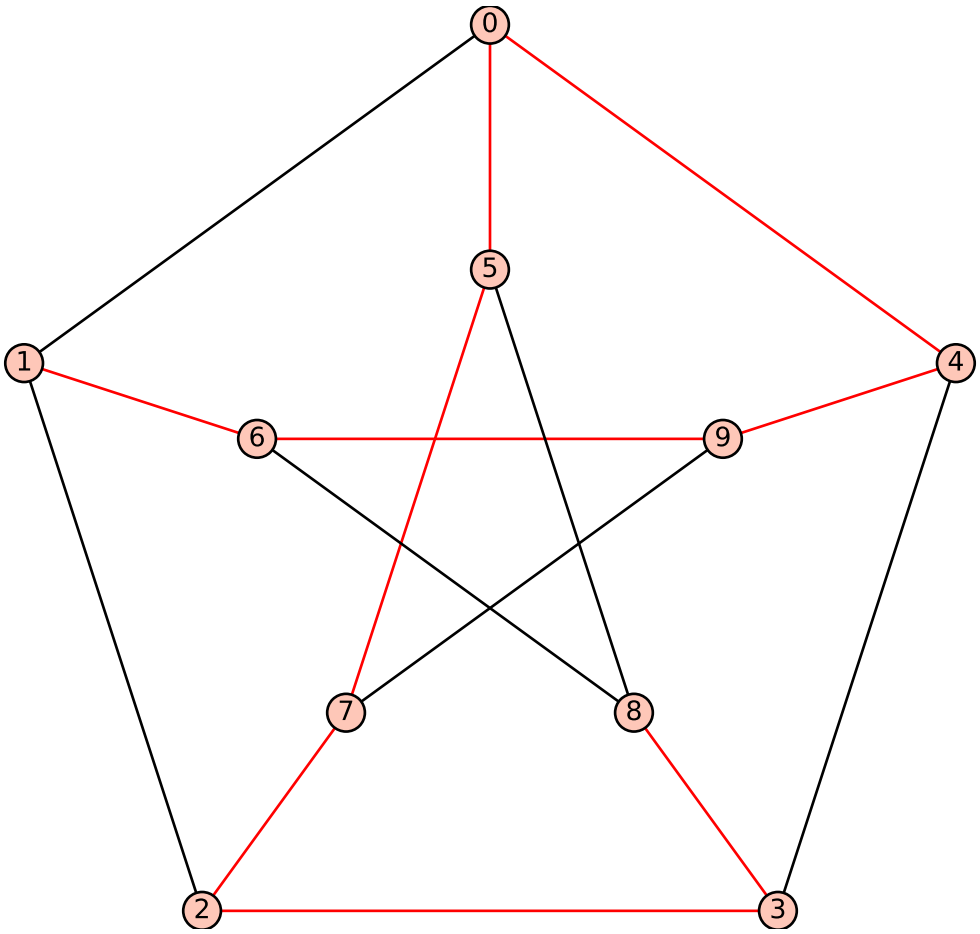
```
sage: g = graphs.PetersenGraph()
sage: lp = g.longest_path()                                                 #␣
↪needs sage.numerical.mip
sage: lp.order() >= g.order() - 2                                           #␣
↪needs sage.numerical.mip
True
```

```
>>> from sage.all import *
>>> g = graphs.PetersenGraph()
>>> lp = g.longest_path()                                                   #␣
↪needs sage.numerical.mip
>>> lp.order() >= g.order() - Integer(2)                                    ␣
↪     # needs sage.numerical.mip
True
```

The heuristic totally agrees:

```
sage: g = graphs.PetersenGraph()
sage: p = g.longest_path(algorithm='heuristic').edges(sort=True, labels=False)
sage: len(p)
9
```

```
>>> from sage.all import *
>>> g = graphs.PetersenGraph()
>>> p = g.longest_path(algorithm='heuristic').edges(sort=True, labels=False)
>>> len(p)
9
```



Let us compute the longest path on a random graph with random weights, and ensure the resulting graph is indeed a path:

```
sage: g = graphs.RandomGNP(15, 0.3)
sage: for u, v in g.edge_iterator(labels=False):
....:     g.set_edge_label(u, v, random())
sage: lp = g.longest_path()                                                 #␣
↪needs sage.numerical.mip
sage: (not lp.is_forest() or not max(lp.degree()) <= 2                      #␣
↪needs sage.numerical.mip
....:  or not lp.is_connected())
False
```

```
>>> from sage.all import *
>>> g = graphs.RandomGNP(Integer(15), RealNumber('0.3'))
>>> for u, v in g.edge_iterator(labels=False):
```

(continues on next page)

(continued from previous page)

```
...       g.set_edge_label(u, v, random())
>>> lp = g.longest_path()                                                    #␣
↪needs sage.numerical.mip
>>> (not lp.is_forest() or not max(lp.degree()) <= Integer(2)                ␣
↪      # needs sage.numerical.mip
...  or not lp.is_connected())
False
```

loop_edges(*labels=True*)

Return a list of all loops in the (di)graph.

INPUT:

- `labels` – boolean (default: `True`); whether returned edges have labels (`(u,v,l)`) or not (`(u,v)`)

EXAMPLES:

```
sage: G = Graph(loops=True); G
Looped graph on 0 vertices
sage: G.has_loops()
False
sage: G.allows_loops()
True
sage: G.add_edges([(0, 0), (1, 1), (2, 2), (3, 3), (2, 3)])
sage: G.loop_edges()
[(0, 0, None), (1, 1, None), (2, 2, None), (3, 3, None)]
sage: G.loop_edges(labels=False)
[(0, 0), (1, 1), (2, 2), (3, 3)]
sage: G.allows_loops()
True
sage: G.has_loops()
True
sage: G.allow_loops(False)
sage: G.has_loops()
False
sage: G.loop_edges()
[]
sage: G.edges(sort=True)
[(2, 3, None)]

sage: D = DiGraph(loops=True); D
Looped digraph on 0 vertices
sage: D.has_loops()
False
sage: D.allows_loops()
True
sage: D.add_edge((0, 0))
sage: D.has_loops()
True
sage: D.loops()
[(0, 0, None)]
sage: D.allow_loops(False); D
Digraph on 1 vertex
```

(continues on next page)

(continued from previous page)

```
sage: D.has_loops()
False
sage: D.edges(sort=True)
[]

sage: G = graphs.PetersenGraph()
sage: G.loops()
[]
```

```
>>> from sage.all import *
>>> G = Graph(loops=True); G
Looped graph on 0 vertices
>>> G.has_loops()
False
>>> G.allows_loops()
True
>>> G.add_edges([(Integer(0), Integer(0)), (Integer(1), Integer(1)),␣
↪(Integer(2), Integer(2)), (Integer(3), Integer(3)), (Integer(2),␣
↪Integer(3))])
>>> G.loop_edges()
[(0, 0, None), (1, 1, None), (2, 2, None), (3, 3, None)]
>>> G.loop_edges(labels=False)
[(0, 0), (1, 1), (2, 2), (3, 3)]
>>> G.allows_loops()
True
>>> G.has_loops()
True
>>> G.allow_loops(False)
>>> G.has_loops()
False
>>> G.loop_edges()
[]
>>> G.edges(sort=True)
[(2, 3, None)]

>>> D = DiGraph(loops=True); D
Looped digraph on 0 vertices
>>> D.has_loops()
False
>>> D.allows_loops()
True
>>> D.add_edge((Integer(0), Integer(0)))
>>> D.has_loops()
True
>>> D.loops()
[(0, 0, None)]
>>> D.allow_loops(False); D
Digraph on 1 vertex
>>> D.has_loops()
False
>>> D.edges(sort=True)
[]
```

(continues on next page)

(continued from previous page)

```
>>> G = graphs.PetersenGraph()
>>> G.loops()
[]
```

```
sage: D = DiGraph(4, loops=True)
sage: D.add_edges([(0, 0), (1, 1), (2, 2), (3, 3), (2, 3)])
sage: D.loop_edges()
[(0, 0, None), (1, 1, None), (2, 2, None), (3, 3, None)]
```

```
>>> from sage.all import *
>>> D = DiGraph(Integer(4), loops=True)
>>> D.add_edges([(Integer(0), Integer(0)), (Integer(1), Integer(1)),␣
↪(Integer(2), Integer(2)), (Integer(3), Integer(3)), (Integer(2),␣
↪Integer(3))])
>>> D.loop_edges()
[(0, 0, None), (1, 1, None), (2, 2, None), (3, 3, None)]
```

```
sage: G = Graph(4, loops=True, multiedges=True, sparse=True)
sage: G.add_edges((i, i) for i in range(4))
sage: G.loop_edges()
[(0, 0, None), (1, 1, None), (2, 2, None), (3, 3, None)]
sage: G.add_edges([(0, 0), (1, 1)])
sage: G.loop_edges(labels=False)
[(0, 0), (0, 0), (1, 1), (1, 1), (2, 2), (3, 3)]
```

```
>>> from sage.all import *
>>> G = Graph(Integer(4), loops=True, multiedges=True, sparse=True)
>>> G.add_edges((i, i) for i in range(Integer(4)))
>>> G.loop_edges()
[(0, 0, None), (1, 1, None), (2, 2, None), (3, 3, None)]
>>> G.add_edges([(Integer(0), Integer(0)), (Integer(1), Integer(1))])
>>> G.loop_edges(labels=False)
[(0, 0), (0, 0), (1, 1), (1, 1), (2, 2), (3, 3)]
```

loop_vertices()

Return a list of vertices with loops.

EXAMPLES:

```
sage: G = Graph({0: [0], 1: [1, 2, 3], 2: [3]}, loops=True)
sage: G.loop_vertices()
[0, 1]
```

```
>>> from sage.all import *
>>> G = Graph({Integer(0): [Integer(0)], Integer(1): [Integer(1), Integer(2),␣
↪Integer(3)], Integer(2): [Integer(3)]}, loops=True)
>>> G.loop_vertices()
[0, 1]
```

loops(*labels=True*)

alias of *loop_edges()*.

max_cut(*value_only*, *use_edge_labels=True*, *vertices=False*, *solver=False*, *verbose=None*, *integrality_tolerance=0*)

Return a maximum edge cut of the graph.

For more information, see the Wikipedia article Maximum_cut.

INPUT:

- `value_only` – boolean (default: `False`); whether to return only the size of the maximum edge cut, or to also return the list of edges of the maximum edge cut
- `use_edge_labels` – boolean (default: `False`); whether to compute a weighted maximum cut where the weight of an edge is defined by its label (if an edge has no label, 1 is assumed), or to compute a cut of maximum cardinality (i.e., edge weights are set to 1)
- `vertices` – boolean (default: `False`); whether to return the two sets of vertices that are disconnected by the cut. This implies `value_only=False`.
- `solver` – string (default: `None`); specifies a Mixed Integer Linear Programming (MILP) solver to be used. If set to `None`, the default one is used. For more information on MILP solvers and which default solver is used, see the method `solve` of the class `MixedIntegerLinearProgram`.
- `verbose` – integer (default: 0); sets the level of verbosity. Set to 0 by default, which means quiet.
- `integrality_tolerance` – float; parameter for use with MILP solvers over an inexact base ring; see `MixedIntegerLinearProgram.get_values()`.

EXAMPLES:

Quite obviously, the max cut of a bipartite graph is the number of edges, and the two sets of vertices are the two sides:

```
sage: g = graphs.CompleteBipartiteGraph(5,6)
sage: [ value, edges, [ setA, setB ]] = g.max_cut(vertices=True)
sage: value == 5*6
True
sage: bsetA, bsetB = map(list, g.bipartite_sets())
sage: ((bsetA == setA and bsetB == setB)
....:  or (bsetA == setB and bsetB == setA))
True
```

```
>>> from sage.all import *
>>> g = graphs.CompleteBipartiteGraph(Integer(5),Integer(6))
>>> [ value, edges, [ setA, setB ]] = g.max_cut(vertices=True)
>>> value == Integer(5)*Integer(6)
True
>>> bsetA, bsetB = map(list, g.bipartite_sets())
>>> ((bsetA == setA and bsetB == setB)
...  or (bsetA == setB and bsetB == setA))
True
```

The max cut of a Petersen graph:

```
sage: g = graphs.PetersenGraph()
sage: g.max_cut()
↪# needs sage.numerical.mip
12
```

```
>>> from sage.all import *
>>> g = graphs.PetersenGraph()
>>> g.max_cut()                                                                #␣
↪needs sage.numerical.mip
12
```

maximum_leaf_number(*G*, *solver=None*, *verbose=0*, *integrality_tolerance=0.001*)

Return the maximum leaf number of the graph.

The maximum leaf number is the maximum possible number of leaves of a spanning tree of G. This is also the cardinality of the complement of a minimum connected dominating set. See the Wikipedia article Connected_dominating_set.

The MLN of a graph with less than 2 vertices is 0, while the MLN of a connected graph with 2 or 3 vertices is 1 or 2 respectively.

INPUT:

- `G` – a Graph
- `solver` – string (default: `None`); specifies a Mixed Integer Linear Programming (MILP) solver to be used. If set to `None`, the default one is used. For more information on MILP solvers and which default solver is used, see the method `solve` of the class `MixedIntegerLinearProgram`.
- `verbose` – integer (default: 0); sets the level of verbosity. Set to 0 by default, which means quiet.
- `integrality_tolerance` – float; parameter for use with MILP solvers over an inexact base ring; see `MixedIntegerLinearProgram.get_values()`.

EXAMPLES:

Empty graph:

```
sage: G = Graph()
sage: G.maximum_leaf_number()
0
```

```
>>> from sage.all import *
>>> G = Graph()
>>> G.maximum_leaf_number()
0
```

Petersen graph:

```
sage: G = graphs.PetersenGraph()
sage: G.maximum_leaf_number()
6
```

```
>>> from sage.all import *
>>> G = graphs.PetersenGraph()
>>> G.maximum_leaf_number()
6
```

merge_vertices(*vertices*)

Merge vertices.

This function replaces a set S of vertices by a single vertex v_{new}, such that the edge uv_{new} exists if and only if $\exists v' \in S : (u, v') \in G$.

The new vertex is named after the first vertex in the list given in argument. If this first name is `None`, a new vertex is created.

In the case of multigraphs, the multiplicity is preserved.

INPUT:

- `vertices` – the list of vertices to be merged

Note

If `u` and `v` are distinct vertices in `vertices`, any edges between `u` and `v` will be lost.

EXAMPLES:

```
sage: g = graphs.CycleGraph(3)
sage: g.merge_vertices([0, 1])
sage: g.edges(sort=True)
[(0, 2, None)]

sage: P = graphs.PetersenGraph()
sage: P.merge_vertices([5, 7])
sage: P.vertices(sort=True)
[0, 1, 2, 3, 4, 5, 6, 8, 9]
```

```
>>> from sage.all import *
>>> g = graphs.CycleGraph(Integer(3))
>>> g.merge_vertices([Integer(0), Integer(1)])
>>> g.edges(sort=True)
[(0, 2, None)]

>>> P = graphs.PetersenGraph()
>>> P.merge_vertices([Integer(5), Integer(7)])
>>> P.vertices(sort=True)
[0, 1, 2, 3, 4, 5, 6, 8, 9]
```

When the first vertex in `vertices` is `None`, a new vertex is created:

```
sage: g = graphs.CycleGraph(5)
sage: g.vertices(sort=True)
[0, 1, 2, 3, 4]
sage: g.merge_vertices([None, 1, 3])
sage: g.edges(sort=True, labels=False)
[(0, 4), (0, 5), (2, 5), (4, 5)]
```

```
>>> from sage.all import *
>>> g = graphs.CycleGraph(Integer(5))
>>> g.vertices(sort=True)
[0, 1, 2, 3, 4]
>>> g.merge_vertices([None, Integer(1), Integer(3)])
>>> g.edges(sort=True, labels=False)
[(0, 4), (0, 5), (2, 5), (4, 5)]
```

With a Multigraph

```
sage: g = graphs.CycleGraph(3)
sage: g.allow_multiple_edges(True)
sage: g.merge_vertices([0, 1])
sage: g.edges(sort=True, labels=False)
[(0, 2), (0, 2)]
```

```
>>> from sage.all import *
>>> g = graphs.CycleGraph(Integer(3))
>>> g.allow_multiple_edges(True)
>>> g.merge_vertices([Integer(0), Integer(1)])
>>> g.edges(sort=True, labels=False)
[(0, 2), (0, 2)]
```

min_spanning_tree(*weight_function=None*, *algorithm='Prim_Boost'*, *starting_vertex=None*, *check=False*, *by_weight=False*, *check_weight=True*)

Return the edges of a minimum spanning tree.

At the moment, no algorithm for directed graph is implemented: if the graph is directed, a minimum spanning tree of the corresponding undirected graph is returned.

We expect all weights of the graph to be convertible to float. Otherwise, an exception is raised.

INPUT:

- `algorithm` – string (default: `'Prim_Boost'`); the algorithm to use in computing a minimum spanning tree of `G`. The following algorithms are supported:
  - `'Prim_Boost'` – Prim's algorithm (Boost implementation)
  - `'Prim_fringe'` – a variant of Prim's algorithm that ignores the labels on the edges
  - `'Prim_edge'` – a variant of Prim's algorithm
  - `'Kruskal'` – Kruskal's algorithm
  - `'Filter_Kruskal'` – a variant of Kruskal's algorithm [OSS2009]
  - `'Kruskal_Boost'` – Kruskal's algorithm (Boost implementation)
  - `'Boruvka'` – Boruvka's algorithm
  - `NetworkX` – uses NetworkX's minimum spanning tree implementation
- `starting_vertex` – a vertex (default: `None`); the vertex from which to begin the search for a minimum spanning tree (available only for `Prim_fringe` and `Prim_edge`).
- `check` – boolean (default: `False`); whether to first perform sanity checks on the input graph `G`. If appropriate, `check` is passed on to any minimum spanning tree functions that are invoked from the current method. See the documentation of the corresponding functions for details on what sort of sanity checks will be performed.
- `weight_function` – function (default: `None`); a function that takes as input an edge `(u, v, l)` and outputs its weight. If not `None`, `by_weight` is automatically set to `True`. If `None` and `by_weight` is `True`, we use the edge label `l` , if `l` is not `None`, else `1` as a weight. The `weight_function` can be used to transform the label into a weight (note that, if the weight returned is not convertible to a float, an error is raised)
- `by_weight` – boolean (default: `False`); if `True`, the edges in the graph are weighted, otherwise all edges have weight 1

- `check_weight` – boolean (default: `True`); whether to check that the `weight_function` outputs a number for each edge

OUTPUT:

The edges of a minimum spanning tree of `G`, if one exists, otherwise returns the empty list.

See also

- *sage.graphs.spanning_tree.kruskal()*
- *sage.graphs.spanning_tree.filter_kruskal()*
- *sage.graphs.spanning_tree.boruvka()*
- *sage.graphs.base.boost_graph.min_spanning_tree()*

EXAMPLES:

Kruskal's algorithm:

```
sage: g = graphs.CompleteGraph(5)
sage: len(g.min_spanning_tree())
4
sage: weight = lambda e: 1 / ((e[0] + 1) * (e[1] + 1))
sage: E = g.min_spanning_tree(weight_function=weight)
sage: T = Graph(E)
sage: set(g) == set(T) and T.order() == T.size() + 1 and T.is_tree()
True
sage: sum(map(weight, E))
5/12
sage: E = g.min_spanning_tree(weight_function=weight,
....:                         algorithm='Kruskal_Boost')
sage: Graph(E).is_tree(); sum(map(weight, E))
True
5/12
sage: g = graphs.PetersenGraph()
sage: g.allow_multiple_edges(True)
sage: g.add_edges(g.edge_iterator())
sage: T = Graph(g.min_spanning_tree())
sage: set(g) == set(T) and T.order() == T.size() + 1 and T.is_tree()
True
```

```
>>> from sage.all import *
>>> g = graphs.CompleteGraph(Integer(5))
>>> len(g.min_spanning_tree())
4
>>> weight = lambda e: Integer(1) / ((e[Integer(0)] + Integer(1)) *␣
↪(e[Integer(1)] + Integer(1)))
>>> E = g.min_spanning_tree(weight_function=weight)
>>> T = Graph(E)
>>> set(g) == set(T) and T.order() == T.size() + Integer(1) and T.is_tree()
True
>>> sum(map(weight, E))
5/12
```

(continues on next page)

(continued from previous page)

```
>>> E = g.min_spanning_tree(weight_function=weight,
...                         algorithm='Kruskal_Boost')
>>> Graph(E).is_tree(); sum(map(weight, E))
True
5/12
>>> g = graphs.PetersenGraph()
>>> g.allow_multiple_edges(True)
>>> g.add_edges(g.edge_iterator())
>>> T = Graph(g.min_spanning_tree())
>>> set(g) == set(T) and T.order() == T.size() + Integer(1) and T.is_tree()
True
```

Boruvka's algorithm:

```
sage: sorted(g.min_spanning_tree(algorithm='Boruvka'))
[(0, 1, None), (0, 4, None), (0, 5, None), (1, 2, None), (1, 6, None),
 (2, 3, None), (2, 7, None),  (3, 8, None), (4, 9, None)]
```

```
>>> from sage.all import *
>>> sorted(g.min_spanning_tree(algorithm='Boruvka'))
[(0, 1, None), (0, 4, None), (0, 5, None), (1, 2, None), (1, 6, None),
 (2, 3, None), (2, 7, None),  (3, 8, None), (4, 9, None)]
```

Prim's algorithm:

```
sage: g = graphs.CompleteGraph(5)
sage: for algo in ['Prim_edge', 'Prim_fringe', 'Prim_Boost']:
....:     E = g.min_spanning_tree(algorithm=algo, weight_function=weight)
....:     T = Graph(E)
....:     print(set(g) == set(T) and T.order() == T.size() + 1 and T.is_
↪tree())
True
True
True
```

```
>>> from sage.all import *
>>> g = graphs.CompleteGraph(Integer(5))
>>> for algo in ['Prim_edge', 'Prim_fringe', 'Prim_Boost']:
...     E = g.min_spanning_tree(algorithm=algo, weight_function=weight)
...     T = Graph(E)
...     print(set(g) == set(T) and T.order() == T.size() + Integer(1) and T.
↪is_tree())
True
True
True
```

NetworkX algorithm:

```
sage: sorted(g.min_spanning_tree(algorithm='NetworkX'))                    #␣
↪needs networkx
[(0, 1, None), (0, 2, None), (0, 3, None), (0, 4, None)]
```

```
>>> from sage.all import *
>>> sorted(g.min_spanning_tree(algorithm='NetworkX'))                    #␣
↪needs networkx
[(0, 1, None), (0, 2, None), (0, 3, None), (0, 4, None)]
```

More complicated weights:

```
sage: G = Graph([(0, 1, {'name': 'a', 'weight': 1}),
....:            (0, 2, {'name': 'b', 'weight': 3}),
....:            (1, 2, {'name': 'b', 'weight': 1})])
sage: sorted(G.min_spanning_tree(algorithm='Boruvka',
....:                            weight_function=lambda e: e[2]['weight']))
[(0, 1, {'name': 'a', 'weight': 1}), (1, 2, {'name': 'b', 'weight': 1})]
```

```
>>> from sage.all import *
>>> G = Graph([(Integer(0), Integer(1), {'name': 'a', 'weight': Integer(1)}),
...            (Integer(0), Integer(2), {'name': 'b', 'weight': Integer(3)}),
...            (Integer(1), Integer(2), {'name': 'b', 'weight': Integer(1)})])
>>> sorted(G.min_spanning_tree(algorithm='Boruvka',
...                            weight_function=lambda e: e[Integer(2)]['weight
↪'']))
[(0, 1, {'name': 'a', 'weight': 1}), (1, 2, {'name': 'b', 'weight': 1})]
```

If the graph is not weighted, edge labels are not considered, even if they are numbers:

```
sage: g = Graph([(1, 2, 1), (1, 3, 2), (2, 3, 1)])
sage: sorted(g.min_spanning_tree(algorithm='Boruvka'))
[(1, 2, 1), (1, 3, 2)]
```

```
>>> from sage.all import *
>>> g = Graph([(Integer(1), Integer(2), Integer(1)), (Integer(1), Integer(3),␣
↪Integer(2)), (Integer(2), Integer(3), Integer(1))])
>>> sorted(g.min_spanning_tree(algorithm='Boruvka'))
[(1, 2, 1), (1, 3, 2)]
```

In order to use weights, we need either to set variable `weighted` to `True`, or to specify a weight function or set by_weight to `True`:

```
sage: g.weighted(True)
sage: Graph(g.min_spanning_tree()).edges(sort=True)
[(1, 2, 1), (2, 3, 1)]
sage: g.weighted(False)
sage: Graph(g.min_spanning_tree()).edges(sort=True)
[(1, 2, 1), (1, 3, 2)]
sage: Graph(g.min_spanning_tree(by_weight=True)).edges(sort=True)
[(1, 2, 1), (2, 3, 1)]
sage: Graph(g.min_spanning_tree(weight_function=lambda e: e[2])).
↪edges(sort=True)
[(1, 2, 1), (2, 3, 1)]
```

```
>>> from sage.all import *
>>> g.weighted(True)
```

(continues on next page)

(continued from previous page)

```
>>> Graph(g.min_spanning_tree()).edges(sort=True)
[(1, 2, 1), (2, 3, 1)]
>>> g.weighted(False)
>>> Graph(g.min_spanning_tree()).edges(sort=True)
[(1, 2, 1), (1, 3, 2)]
>>> Graph(g.min_spanning_tree(by_weight=True)).edges(sort=True)
[(1, 2, 1), (2, 3, 1)]
>>> Graph(g.min_spanning_tree(weight_function=lambda e: e[Integer(2)])).
↪edges(sort=True)
[(1, 2, 1), (2, 3, 1)]
```

Note that the order of the vertices on each edge is not guaranteed and may differ from an algorithm to the other:

```
sage: g.weighted(True)
sage: sorted(g.min_spanning_tree())
[(2, 1, 1), (3, 2, 1)]
sage: sorted(g.min_spanning_tree(algorithm='Boruvka'))
[(1, 2, 1), (2, 3, 1)]
sage: Graph(g.min_spanning_tree()).edges(sort=True)
[(1, 2, 1), (2, 3, 1)]
```

```
>>> from sage.all import *
>>> g.weighted(True)
>>> sorted(g.min_spanning_tree())
[(2, 1, 1), (3, 2, 1)]
>>> sorted(g.min_spanning_tree(algorithm='Boruvka'))
[(1, 2, 1), (2, 3, 1)]
>>> Graph(g.min_spanning_tree()).edges(sort=True)
[(1, 2, 1), (2, 3, 1)]
```

minimum_cycle_basis(*algorithm=None*, *weight_function=None*, *by_weight=False*, *check_weight=True*)

Return a minimum weight cycle basis of the graph.

A cycle basis is a list of cycles (list of vertices forming a cycle) of `self`. Note that the vertices are not necessarily returned in the order in which they appear in the cycle.

A minimum weight cycle basis is a cycle basis that minimizes the sum of the weights (length for unweighted graphs) of its cycles.

Not implemented for directed graphs and multigraphs.

INPUT:

- `algorithm` – string (default: `None`); algorithm to use:
  - If `algorithm = "NetworkX"`, use networkx implementation
  - If `algorithm = None`, use Sage Cython implementation
- `weight_function` – function (default: `None`); a function that takes as input an edge `(u, v, l)` and outputs its weight. If not `None`, `by_weight` is automatically set to `True`. If `None` and `by_weight` is `True`, we use the edge label `l`, if `l` is not `None`, else `1` as a weight.
- `by_weight` – boolean (default: `False`); if `True`, the edges in the graph are weighted, otherwise all edges have weight 1

- `check_weight` – boolean (default: `True`); whether to check that the `weight_function` outputs a number for each edge

EXAMPLES:

```
sage: g = Graph([(1, 2, 3), (2, 3, 5), (3, 4, 8), (4, 1, 13),
....:            (1, 3, 250), (5, 6, 9), (6, 7, 17), (7, 5, 20)])
sage: sorted(g.minimum_cycle_basis(by_weight=True))
[[1, 2, 3], [1, 2, 3, 4], [5, 6, 7]]
sage: sorted(g.minimum_cycle_basis(by_weight=False))
[[1, 2, 3], [1, 3, 4], [5, 6, 7]]
sage: sorted(g.minimum_cycle_basis(by_weight=True, algorithm='NetworkX'))    #␣
↪needs networkx, random (changes in networkx 3.2)
[[2, 3, 1], [2, 3, 4, 1], [6, 7, 5]]
sage: g.minimum_cycle_basis(by_weight=False, algorithm='NetworkX')           #␣
↪needs networkx, random (changes in networkx 3.2)
[[3, 4, 1], [2, 3, 1], [6, 7, 5]]
```

```
>>> from sage.all import *
>>> g = Graph([(Integer(1), Integer(2), Integer(3)), (Integer(2), Integer(3),␣
↪Integer(5)), (Integer(3), Integer(4), Integer(8)), (Integer(4), Integer(1),␣
↪Integer(13)),
...            (Integer(1), Integer(3), Integer(250)), (Integer(5),␣
↪Integer(6), Integer(9)), (Integer(6), Integer(7), Integer(17)), (Integer(7),
↪ Integer(5), Integer(20))])
>>> sorted(g.minimum_cycle_basis(by_weight=True))
[[1, 2, 3], [1, 2, 3, 4], [5, 6, 7]]
>>> sorted(g.minimum_cycle_basis(by_weight=False))
[[1, 2, 3], [1, 3, 4], [5, 6, 7]]
>>> sorted(g.minimum_cycle_basis(by_weight=True, algorithm='NetworkX'))    #␣
↪needs networkx, random (changes in networkx 3.2)
[[2, 3, 1], [2, 3, 4, 1], [6, 7, 5]]
>>> g.minimum_cycle_basis(by_weight=False, algorithm='NetworkX')           #␣
↪needs networkx, random (changes in networkx 3.2)
[[3, 4, 1], [2, 3, 1], [6, 7, 5]]
```

```
sage: g = Graph([(1, 2), (2, 3), (3, 4), (4, 5), (5, 1), (5, 3)])
sage: sorted(g.minimum_cycle_basis(by_weight=False))
[[1, 2, 3, 5], [3, 4, 5]]
sage: sorted(g.minimum_cycle_basis(by_weight=False, algorithm='NetworkX'))  #␣
↪needs networkx, random (changes in networkx 3.2)
[[3, 4, 5], [5, 3, 2, 1]]
```

```
>>> from sage.all import *
>>> g = Graph([(Integer(1), Integer(2)), (Integer(2), Integer(3)),␣
↪(Integer(3), Integer(4)), (Integer(4), Integer(5)), (Integer(5),␣
↪Integer(1)), (Integer(5), Integer(3))])
>>> sorted(g.minimum_cycle_basis(by_weight=False))
[[1, 2, 3, 5], [3, 4, 5]]
>>> sorted(g.minimum_cycle_basis(by_weight=False, algorithm='NetworkX'))  #␣
↪needs networkx, random (changes in networkx 3.2)
[[3, 4, 5], [5, 3, 2, 1]]
```

See also

- *cycle_basis()*
- Wikipedia article Cycle_basis

multicommodity_flow(*terminals*, *integer*, *by_weight=True*, *weight_function=False*, *check_weight=None*, *vertex_bound=True*, *solver=False*, *verbose=None*, *integrality_tolerance=0*)

Solve a multicommodity flow problem.

In the multicommodity flow problem, we are given a set of pairs (s_i, t_i), called terminals meaning that s_i is willing some flow to t_i.

Even though it is a natural generalisation of the flow problem this version of it is NP-Complete to solve when the flows are required to be integer.

For more information, see the Wikipedia article Multi-commodity_flow_problem.

INPUT:

- `terminals` – list of pairs (s_i, t_i) or triples (s_i, t_i, w_i) representing a flow from s_i to t_i of intensity w_i. When the pairs are of size 2, an intensity of 1 is assumed.
- `integer` – boolean (default: `True`); whether to require an integer multicommodity flow
- `by_weight` – boolean (default: `False`); if `True`, the capacity of an edge is given by `weight_function`, or by its label if `weight_function` is `None`. If `False`, every edge has capacity 1.
- `weight_function` – function (default: `None`); a function that takes as input an edge `(u, v, l)` and outputs its capacity. If not `None`, `by_weight` is automatically set to `True`. If `None` and `by_weight` is `True`, we use the edge label `l`, if `l` is not `None`, else 1 as a capacity.
- `check_weight` – boolean (default: `True`); whether to check that the `weight_function` outputs a number for each edge
- `vertex_bound` – boolean (default: `False`); whether to require that a vertex can stand at most 1 commodity of flow through it of intensity 1. Terminals can obviously still send or receive several units of flow even though `vertex_bound` is set to `True`, as this parameter is meant to represent topological properties.
- `solver` – string (default: `None`); specifies a Mixed Integer Linear Programming (MILP) solver to be used. If set to `None`, the default one is used. For more information on MILP solvers and which default solver is used, see the method `solve` of the class `MixedIntegerLinearProgram`.
- `verbose` – integer (default: 0); sets the level of verbosity. Set to 0 by default, which means quiet.
- `integrality_tolerance` – float; parameter for use with MILP solvers over an inexact base ring; see `MixedIntegerLinearProgram.get_values()`.

  Only useful when parameter `integer` is `True`.

ALGORITHM:

(Mixed Integer) Linear Program, depending on the value of `integer`.

EXAMPLES:

An easy way to obtain a satisfiable multicommodity flow is to compute a matching in a graph, and to consider the paired vertices as terminals

```
sage: g = graphs.PetersenGraph()
sage: matching = [(u,v) for u,v,_ in g.matching()]                    #␣
↪needs networkx
sage: h = g.multicommodity_flow(matching)                             #␣
↪needs networkx
sage: len(h)                                                          #␣
↪needs networkx
5
```

```
>>> from sage.all import *
>>> g = graphs.PetersenGraph()
>>> matching = [(u,v) for u,v,_ in g.matching()]                      #␣
↪needs networkx
>>> h = g.multicommodity_flow(matching)                               #␣
↪needs networkx
>>> len(h)                                                            #␣
↪needs networkx
5
```

We could also have considered `g` as symmetric and computed the multicommodity flow in this version instead. In this case, however edges can be used in both directions at the same time:

```
sage: h = DiGraph(g).multicommodity_flow(matching)                    #␣
↪needs networkx
sage: len(h)                                                          #␣
↪needs networkx
5
```

```
>>> from sage.all import *
>>> h = DiGraph(g).multicommodity_flow(matching)                      #␣
↪needs networkx
>>> len(h)                                                            #␣
↪needs networkx
5
```

An exception is raised when the problem has no solution

```
sage: h = g.multicommodity_flow([(u,v,3) for u,v in matching])        #␣
↪needs networkx
Traceback (most recent call last):
...
EmptySetError: the multicommodity flow problem has no solution
```

```
>>> from sage.all import *
>>> h = g.multicommodity_flow([(u,v,Integer(3)) for u,v in matching])  ␣
↪     # needs networkx
Traceback (most recent call last):
...
EmptySetError: the multicommodity flow problem has no solution
```

multiple_edges(*to_undirected=False*, *labels=True*, *sort=False*, *key=None*)

Return any multiple edges in the (di)graph.

INPUT:

- `to_undirected` – boolean (default: `False`)
- `labels` – boolean (default: `True`); whether to include labels
- `sort` – boolean (default: `False`); whether to sort the result
- `key` – a function (default: `None`); a function that takes an edge as its one argument and returns a value that can be used for comparisons in the sorting algorithm (we must have `sort=True`)

EXAMPLES:

```
sage: G = Graph(multiedges=True, sparse=True); G
Multi-graph on 0 vertices
sage: G.has_multiple_edges()
False
sage: G.allows_multiple_edges()
True
sage: G.add_edges([(0, 1)] * 3)
sage: G.has_multiple_edges()
True
sage: G.multiple_edges(sort=True)
[(0, 1, None), (0, 1, None), (0, 1, None)]
sage: G.allow_multiple_edges(False); G
Graph on 2 vertices
sage: G.has_multiple_edges()
False
sage: G.edges(sort=True)
[(0, 1, None)]

sage: D = DiGraph(multiedges=True, sparse=True); D
Multi-digraph on 0 vertices
sage: D.has_multiple_edges()
False
sage: D.allows_multiple_edges()
True
sage: D.add_edges([(0, 1)] * 3)
sage: D.has_multiple_edges()
True
sage: D.multiple_edges(sort=True)
[(0, 1, None), (0, 1, None), (0, 1, None)]
sage: D.allow_multiple_edges(False); D
Digraph on 2 vertices
sage: D.has_multiple_edges()
False
sage: D.edges(sort=True)
[(0, 1, None)]

sage: G = DiGraph({1: {2: 'h'}, 2: {1: 'g'}}, sparse=True)
sage: G.has_multiple_edges()
False
sage: G.has_multiple_edges(to_undirected=True)
True
sage: G.multiple_edges()
[]
```

(continues on next page)

(continued from previous page)

```
sage: G.multiple_edges(to_undirected=True, sort=True)
[(1, 2, 'h'), (2, 1, 'g')]
```

```
>>> from sage.all import *
>>> G = Graph(multiedges=True, sparse=True); G
Multi-graph on 0 vertices
>>> G.has_multiple_edges()
False
>>> G.allows_multiple_edges()
True
>>> G.add_edges([(Integer(0), Integer(1))] * Integer(3))
>>> G.has_multiple_edges()
True
>>> G.multiple_edges(sort=True)
[(0, 1, None), (0, 1, None), (0, 1, None)]
>>> G.allow_multiple_edges(False); G
Graph on 2 vertices
>>> G.has_multiple_edges()
False
>>> G.edges(sort=True)
[(0, 1, None)]

>>> D = DiGraph(multiedges=True, sparse=True); D
Multi-digraph on 0 vertices
>>> D.has_multiple_edges()
False
>>> D.allows_multiple_edges()
True
>>> D.add_edges([(Integer(0), Integer(1))] * Integer(3))
>>> D.has_multiple_edges()
True
>>> D.multiple_edges(sort=True)
[(0, 1, None), (0, 1, None), (0, 1, None)]
>>> D.allow_multiple_edges(False); D
Digraph on 2 vertices
>>> D.has_multiple_edges()
False
>>> D.edges(sort=True)
[(0, 1, None)]

>>> G = DiGraph({Integer(1): {Integer(2): 'h'}, Integer(2): {Integer(1): 'g'}}
↪, sparse=True)
>>> G.has_multiple_edges()
False
>>> G.has_multiple_edges(to_undirected=True)
True
>>> G.multiple_edges()
[]
>>> G.multiple_edges(to_undirected=True, sort=True)
[(1, 2, 'h'), (2, 1, 'g')]
```

Using the `key` argument to order multiple edges of incomparable types (see Issue #35903):

```
sage: G = Graph([('A', 'B', 3), (1, 2, 1), ('A', 'B', 4), (1, 2, 2)],␣
↪multiedges=True)
sage: G.multiple_edges(sort=True)
Traceback (most recent call last):
...
TypeError: unsupported operand parent(s) for <: 'Integer Ring' and '<class
↪'str'>'
sage: G.multiple_edges(labels=False, sort=True, key=str)
[('A', 'B'), ('A', 'B'), (1, 2), (1, 2)]
sage: G.multiple_edges(sort=True, key=str)
[('A', 'B', 3), ('A', 'B', 4), (1, 2, 1), (1, 2, 2)]
sage: G.multiple_edges(labels=True, sort=True, key=lambda e:e[2])
[(1, 2, 1), (1, 2, 2), ('A', 'B', 3), ('A', 'B', 4)]
sage: G.multiple_edges(labels=False, sort=True, key=lambda e:e[2])
Traceback (most recent call last):
...
IndexError: tuple index out of range
```

```
>>> from sage.all import *
>>> G = Graph([('A', 'B', Integer(3)), (Integer(1), Integer(2), Integer(1)), (
↪'A', 'B', Integer(4)), (Integer(1), Integer(2), Integer(2))],␣
↪multiedges=True)
>>> G.multiple_edges(sort=True)
Traceback (most recent call last):
...
TypeError: unsupported operand parent(s) for <: 'Integer Ring' and '<class
↪'str'>'
>>> G.multiple_edges(labels=False, sort=True, key=str)
[('A', 'B'), ('A', 'B'), (1, 2), (1, 2)]
>>> G.multiple_edges(sort=True, key=str)
[('A', 'B', 3), ('A', 'B', 4), (1, 2, 1), (1, 2, 2)]
>>> G.multiple_edges(labels=True, sort=True, key=lambda e:e[Integer(2)])
[(1, 2, 1), (1, 2, 2), ('A', 'B', 3), ('A', 'B', 4)]
>>> G.multiple_edges(labels=False, sort=True, key=lambda e:e[Integer(2)])
Traceback (most recent call last):
...
IndexError: tuple index out of range
```

multiway_cut(*vertices*, *value_only*, *use_edge_labels=False*, *solver=False*, *verbose=None*, *integrality_tolerance=0*)

Return a minimum edge multiway cut.

A multiway cut for a vertex set S in a graph or a digraph G is a set C of edges such that any two vertices u, v in S are disconnected when removing the edges of C from G. (cf. http://www.d.kth.se/~viggo/wwwcompendium/node92.html)

Such a cut is said to be minimum when its cardinality (or weight) is minimum.

INPUT:

- `vertices` – iterable; the set of vertices
- `value_only` – boolean (default: `False`); whether to return only the size of the minimum multiway cut, or to return the list of edges of the multiway cut
- `use_edge_labels` – boolean (default: `False`); whether to compute a weighted minimum multiway

cut where the weight of an edge is defined by its label (if an edge has no label, 1 is assumed), or to compute a cut of minimum cardinality (i.e., edge weights are set to 1)

- `solver` – string (default: `None`); specifies a Mixed Integer Linear Programming (MILP) solver to be used. If set to `None`, the default one is used. For more information on MILP solvers and which default solver is used, see the method `solve` of the class `MixedIntegerLinearProgram`.
- `verbose` – integer (default: 0); sets the level of verbosity. Set to 0 by default, which means quiet.
- `integrality_tolerance` – float; parameter for use with MILP solvers over an inexact base ring; see `MixedIntegerLinearProgram.get_values()`.

EXAMPLES:

Of course, a multiway cut between two vertices correspond to a minimum edge cut:

```
sage: g = graphs.PetersenGraph()
sage: g.edge_cut(0,3) == g.multiway_cut([0,3], value_only=True)                 #␣
↪needs sage.numerical.mip
True
```

```
>>> from sage.all import *
>>> g = graphs.PetersenGraph()
>>> g.edge_cut(Integer(0),Integer(3)) == g.multiway_cut([Integer(0),
↪Integer(3)], value_only=True)              # needs sage.numerical.mip
True
```

As Petersen's graph is 3-regular, a minimum multiway cut between three vertices contains at most 2×3 edges (which could correspond to the neighborhood of 2 vertices):

```
sage: g.multiway_cut([0,3,9], value_only=True) == 2*3                           #␣
↪needs sage.numerical.mip
True
```

```
>>> from sage.all import *
>>> g.multiway_cut([Integer(0),Integer(3),Integer(9)], value_only=True) ==␣
↪Integer(2)*Integer(3)                        # needs sage.numerical.mip
True
```

In this case, though, the vertices are an independent set. If we pick instead vertices $0, 9$, and 7, we can save 4 edges in the multiway cut:

```
sage: g.multiway_cut([0,7,9], value_only=True) == 2*3 - 1                       #␣
↪needs sage.numerical.mip
True
```

```
>>> from sage.all import *
>>> g.multiway_cut([Integer(0),Integer(7),Integer(9)], value_only=True) ==␣
↪Integer(2)*Integer(3) - Integer(1)                    # needs sage.numerical.
↪mip
True
```

This example, though, does not work in the directed case anymore, as it is not possible in Petersen's graph to mutualise edges:

```
sage: g = DiGraph(g)
sage: g.multiway_cut([0,7,9], value_only=True) == 3*3                    #␣
↪needs sage.numerical.mip
True
```

```
>>> from sage.all import *
>>> g = DiGraph(g)
>>> g.multiway_cut([Integer(0),Integer(7),Integer(9)], value_only=True) ==␣
↪Integer(3)*Integer(3)                          # needs sage.numerical.mip
True
```

Of course, a multiway cut between the whole vertex set contains all the edges of the graph:

```
sage: C = g.multiway_cut(g.vertices(sort=False))                         #␣
↪needs sage.numerical.mip
sage: set(C) == set(g.edges(sort=False))                                 #␣
↪needs sage.numerical.mip
True
```

```
>>> from sage.all import *
>>> C = g.multiway_cut(g.vertices(sort=False))                           #␣
↪needs sage.numerical.mip
>>> set(C) == set(g.edges(sort=False))                                   #␣
↪needs sage.numerical.mip
True
```

n_edges()

alias of *size()*.

n_faces(*embedding=None*)

Return the number of faces of an embedded graph.

If no embedding is provided or stored as `self._embedding`, this method uses Euler's formula (see the Wikipedia article Euler_characteristic) to determine the number of faces if the graph is planar. If the graph is not planar, an error is raised.

If an embedding is provided or stored as `self._embedding`, this method calls method *faces()* to get the list of faces induced by the embedding in each connected component of the graph. Then it returns the sum of size of these lists minus the number of connected components plus one to ensure that the external face is counted only once.

INPUT:

- `embedding` – dictionary (default: `None`); a combinatorial embedding dictionary. Format: `{v1: [v2, v3], v2: [v1], v3: [v1]}` (clockwise ordering of neighbors at each vertex). If set to `None` (default) the method will use the embedding stored as `self._embedding`. If none is stored, the method will compute the set of faces from the embedding returned by *is_planar()* (if the graph is, of course, planar).

EXAMPLES:

```
sage: T = graphs.TetrahedralGraph()
sage: T.n_faces()
4
```

```
>>> from sage.all import *
>>> T = graphs.TetrahedralGraph()
>>> T.n_faces()
4
```

The external face of a disconnected graph is counted only once:

```
sage: (T + T).n_faces()
7
sage: (T + T + T).n_faces()
10
```

```
>>> from sage.all import *
>>> (T + T).n_faces()
7
>>> (T + T + T).n_faces()
10
```

Trees and forests have a single face:

```
sage: T = graphs.RandomTree(10)
sage: T.n_faces()
1
sage: (T + T).n_faces()
1
```

```
>>> from sage.all import *
>>> T = graphs.RandomTree(Integer(10))
>>> T.n_faces()
1
>>> (T + T).n_faces()
1
```

n_vertices()

alias of *order()*.

name(*new=None*)

Return or set the graph's name.

INPUT:

- `new` – string (default: `None`); by default (`new == None`), the method returns the name of the graph. When `name` is set, the string representation of that object becomes the new name of the (di)graph (`new == ''` removes any name).

EXAMPLES:

```
sage: d = {0: [1,4,5], 1: [2,6], 2: [3,7], 3: [4,8], 4: [9], 5: [7, 8], 6: [8,
↪9], 7: [9]}
sage: G = Graph(d); G
Graph on 10 vertices
sage: G.name("Petersen Graph"); G
Petersen Graph: Graph on 10 vertices
sage: G.name(new=""); G
```

(continues on next page)

(continued from previous page)

```
Graph on 10 vertices
sage: G.name()
''
sage: G.name(42); G
42: Graph on 10 vertices
sage: G.name()
'42'
```

```
>>> from sage.all import *
>>> d = {Integer(0): [Integer(1),Integer(4),Integer(5)], Integer(1):␣
↪[Integer(2),Integer(6)], Integer(2): [Integer(3),Integer(7)], Integer(3):␣
↪[Integer(4),Integer(8)], Integer(4): [Integer(9)], Integer(5): [Integer(7),␣
↪Integer(8)], Integer(6): [Integer(8),Integer(9)], Integer(7): [Integer(9)]}
>>> G = Graph(d); G
Graph on 10 vertices
>>> G.name("Petersen Graph"); G
Petersen Graph: Graph on 10 vertices
>>> G.name(new=""); G
Graph on 10 vertices
>>> G.name()
''
>>> G.name(Integer(42)); G
42: Graph on 10 vertices
>>> G.name()
'42'
```

neighbor_iterator(*vertex*, *closed=False*)

Return an iterator over neighbors of `vertex`.

When `closed` is set to `True`, the returned iterator also contains `vertex`.

INPUT:

- `vertex` – a vertex of `self`
- `closed` – boolean (default: `False`); whether to return the closed neighborhood of `vertex`, i.e., including `vertex`, or the open neighborhood in which `vertex` is included only if there is a loop on that vertex.

EXAMPLES:

```
sage: G = graphs.PetersenGraph()
sage: for i in G.neighbor_iterator(0):
....:     print(i)
1
4
5
sage: D = G.to_directed()
sage: for i in D.neighbor_iterator(0):
....:     print(i)
1
4
5
```

```
>>> from sage.all import *
>>> G = graphs.PetersenGraph()
>>> for i in G.neighbor_iterator(Integer(0)):
...     print(i)
1
4
5
>>> D = G.to_directed()
>>> for i in D.neighbor_iterator(Integer(0)):
...     print(i)
1
4
5
```

```
sage: D = DiGraph({0: [1, 2], 3: [0]})
sage: sorted(D.neighbor_iterator(0))
[1, 2, 3]
```

```
>>> from sage.all import *
>>> D = DiGraph({Integer(0): [Integer(1), Integer(2)], Integer(3):␣
↪[Integer(0)]})
>>> sorted(D.neighbor_iterator(Integer(0)))
[1, 2, 3]
```

```
sage: g = graphs.CubeGraph(3)
sage: sorted(g.neighbor_iterator('010', closed=True))
['000', '010', '011', '110']
```

```
>>> from sage.all import *
>>> g = graphs.CubeGraph(Integer(3))
>>> sorted(g.neighbor_iterator('010', closed=True))
['000', '010', '011', '110']
```

```
sage: g = Graph(3, loops = True)
sage: g.add_edge(0,1)
sage: g.add_edge(0,0)
sage: list(g.neighbor_iterator(0, closed=True))
[0, 1]
sage: list(g.neighbor_iterator(2, closed=True))
[2]
```

```
>>> from sage.all import *
>>> g = Graph(Integer(3), loops = True)
>>> g.add_edge(Integer(0),Integer(1))
>>> g.add_edge(Integer(0),Integer(0))
>>> list(g.neighbor_iterator(Integer(0), closed=True))
[0, 1]
>>> list(g.neighbor_iterator(Integer(2), closed=True))
[2]
```

neighbors(*vertex*, *closed=False*)

Return a list of neighbors (in and out if directed) of `vertex`.

G[vertex] also works. When closed is set to True, the returned iterator also contains vertex.

INPUT:

- vertex – a vertex of self
- closed – boolean (default: False); whether to return the closed neighborhood of vertex, i.e., including vertex, or the open neighborhood in which vertex is included only if there is a loop on that vertex.

EXAMPLES:

```
sage: P = graphs.PetersenGraph()
sage: sorted(P.neighbors(3))
[2, 4, 8]
sage: sorted(P[4])
[0, 3, 9]
sage: sorted(P.neighbors(3, closed=True))
[2, 3, 4, 8]
```

```
>>> from sage.all import *
>>> P = graphs.PetersenGraph()
>>> sorted(P.neighbors(Integer(3)))
[2, 4, 8]
>>> sorted(P[Integer(4)])
[0, 3, 9]
>>> sorted(P.neighbors(Integer(3), closed=True))
[2, 3, 4, 8]
```

networkx_graph(*weight_function=None*)

Return a new NetworkX graph from the Sage graph.

INPUT:

- weight_function – function (default: None); a function that takes as input an edge (u, v, l) and outputs its weight

EXAMPLES:

```
sage: G = graphs.TetrahedralGraph()
sage: N = G.networkx_graph()                                                #␣
↪needs networkx
sage: type(N)                                                               #␣
↪needs networkx
<class 'networkx.classes.graph.Graph'>

sage: def weight_fn(e):
....:     return e[2]
sage: G1 = Graph([(1,2,1), (1,3,4), (2,3,3), (3,4,4)])
sage: H = G1.networkx_graph(weight_function=weight_fn)                      #␣
↪needs networkx
sage: H.edges(data=True)                                                    #␣
↪needs networkx
EdgeDataView([(1, 2, {'weight': 1}), (1, 3, {'weight': 4}),
              (2, 3, {'weight': 3}), (3, 4, {'weight': 4})])
sage: G2 = DiGraph([(1,2,1), (1,3,4), (2,3,3), (3,4,4), (3,4,5)],
....:              multiedges=True)
```

(continues on next page)

(continued from previous page)

```
sage: H = G2.networkx_graph(weight_function=weight_fn)                    #␣
↪needs networkx
sage: H.edges(data=True)                                                  #␣
↪needs networkx
OutMultiEdgeDataView([(1, 2, {'weight': 1}), (1, 3, {'weight': 4}),
                      (2, 3, {'weight': 3}), (3, 4, {'weight': 5}),
                      (3, 4, {'weight': 4})])
```

```
>>> from sage.all import *
>>> G = graphs.TetrahedralGraph()
>>> N = G.networkx_graph()                                                #␣
↪needs networkx
>>> type(N)                                                               #␣
↪needs networkx
<class 'networkx.classes.graph.Graph'>

>>> def weight_fn(e):
...     return e[Integer(2)]
>>> G1 = Graph([(Integer(1),Integer(2),Integer(1)), (Integer(1),Integer(3),
↪Integer(4)), (Integer(2),Integer(3),Integer(3)), (Integer(3),Integer(4),
↪Integer(4))])
>>> H = G1.networkx_graph(weight_function=weight_fn)                      #␣
↪needs networkx
>>> H.edges(data=True)                                                    #␣
↪needs networkx
EdgeDataView([(1, 2, {'weight': 1}), (1, 3, {'weight': 4}),
              (2, 3, {'weight': 3}), (3, 4, {'weight': 4})])
>>> G2 = DiGraph([(Integer(1),Integer(2),Integer(1)), (Integer(1),Integer(3),
↪Integer(4)), (Integer(2),Integer(3),Integer(3)), (Integer(3),Integer(4),
↪Integer(4)), (Integer(3),Integer(4),Integer(5))],
...               multiedges=True)
>>> H = G2.networkx_graph(weight_function=weight_fn)                      #␣
↪needs networkx
>>> H.edges(data=True)                                                    #␣
↪needs networkx
OutMultiEdgeDataView([(1, 2, {'weight': 1}), (1, 3, {'weight': 4}),
                      (2, 3, {'weight': 3}), (3, 4, {'weight': 5}),
                      (3, 4, {'weight': 4})])
```

nowhere_zero_flow(*k*, *solver=None*, *verbose=None*, *integrality_tolerance=0*)

Return a `k`-nowhere zero flow of the (di)graph.

A flow on a graph $G = (V, E)$ is a pair (D, f) such that D is an orientation of G and f is a function on E satisfying

$$\sum_{u \in N_D^-(v)} f(uv) = \sum_{w \in N_D^+(v)} f(vw),\ \forall v \in V.$$

A `nowhere zero flow` on a graph $G = (V, E)$ is a flow (D, f) such that $f(e) \neq 0$ for every $e \in E$. For a positive integer k, a k-flow on a graph $G = (V, E)$ is a flow (D, f) such that $f : E \to Z$ and $-(k-1) \leq f(e) \leq k-1$ for every $e \in E$. A k-flow is positive if $f(e) > 0$ for every $e \in E$. A k-flow which is nowhere zero is called a *k-nowhere zero flow* (or k-NZF).

The following are equivalent.

- G admits a positive k-flow.
- G admits a k-NZF.
- Every orientation of G admits a k-NZF.

Furthermore, a (di)graph admits a k-NZF if and only if it is bridgeless and every bridgeless graph admits a 6-NZF [Sey1981]. See the Wikipedia article Nowhere-zero_flow for more details.

ALGORITHM:

If `self` is not directed, we search for a k-NZF on any orientation of `self` and then build a positive k-NZF by reverting edges with negative flow.

INPUT:

- `k` – integer (default: 6); when set to a positive integer ≥ 2, search for a k-nowhere zero flow
- `solver` – string (default: `None`); specifies a Mixed Integer Linear Programming (MILP) solver to be used. If set to `None`, the default one is used. For more information on MILP solvers and which default solver is used, see the method `solve` of the class `MixedIntegerLinearProgram`.
- `verbose` – integer (default: 0); sets the level of verbosity. Set to 0 by default, which means quiet.
- `integrality_tolerance` – float; parameter for use with MILP solvers over an inexact base ring; see `MixedIntegerLinearProgram.get_values()`.

OUTPUT:

A digraph with flow values stored as edge labels if a k-nowhere zero flow is found. If `self` is undirected, the edges of this digraph indicate the selected orientation. If no feasible solution is found, an error is raised.

EXAMPLES:

The Petersen graph admits a (positive) 5-nowhere zero flow, but no 4-nowhere zero flow:

```
sage: g = graphs.PetersenGraph()
sage: h = g.nowhere_zero_flow(k=5)                                          #␣
↪needs sage.numerical.mip
sage: sorted(set(h.edge_labels()))                                          #␣
↪needs sage.numerical.mip
[1, 2, 3, 4]
sage: h = g.nowhere_zero_flow(k=3)                                          #␣
↪needs sage.numerical.mip
Traceback (most recent call last):
...
EmptySetError: the problem has no feasible solution
```

```
>>> from sage.all import *
>>> g = graphs.PetersenGraph()
>>> h = g.nowhere_zero_flow(k=Integer(5))                                    ␣
↪     # needs sage.numerical.mip
>>> sorted(set(h.edge_labels()))                                           #␣
↪needs sage.numerical.mip
[1, 2, 3, 4]
>>> h = g.nowhere_zero_flow(k=Integer(3))                                    ␣
↪     # needs sage.numerical.mip
Traceback (most recent call last):
...
EmptySetError: the problem has no feasible solution
```

The de Bruijn digraph admits a 2-nowhere zero flow:

```
sage: g = digraphs.DeBruijn(2, 3)                                           #␣
↪needs sage.combinat
sage: h = g.nowhere_zero_flow(k=2)                                          #␣
↪needs sage.combinat sage.numerical.mip
sage: sorted(set(h.edge_labels()))                                          #␣
↪needs sage.combinat sage.numerical.mip
[-1, 1]
```

```
>>> from sage.all import *
>>> g = digraphs.DeBruijn(Integer(2), Integer(3))                           ␣
↪              # needs sage.combinat
>>> h = g.nowhere_zero_flow(k=Integer(2))                                   ␣
↪     # needs sage.combinat sage.numerical.mip
>>> sorted(set(h.edge_labels()))                                           #␣
↪needs sage.combinat sage.numerical.mip
[-1, 1]
```

num_edges()

alias of *size()*.

num_faces(*embedding=None*)

alias of *n_faces()*.

num_verts()

alias of *order()*.

number_of_biconnected_components(*G*, *forbidden_vertices=None*)

Return the number of biconnected components.

A biconnected component is a maximal subgraph on two or more vertices that is biconnected, i.e., removing any vertex does not disconnect it.

INPUT:

- `G` – the input graph
- `forbidden_vertices` – list (default: `None`); set of vertices to avoid during the search. This is equivalent to getting the number of biconnected components of the graph after the removal of the forbidden vertices.

See also

- *blocks_and_cut_vertices()*
- *is_biconnected()*

EXAMPLES:

The disjoint union of cycles has as many biconnected components as the number of cycles:

```
sage: G = graphs.CycleGraph(5)
sage: G.number_of_biconnected_components()
1
```

(continues on next page)

(continued from previous page)

```
sage: (3 * G).number_of_biconnected_components()
3
```

```
>>> from sage.all import *
>>> G = graphs.CycleGraph(Integer(5))
>>> G.number_of_biconnected_components()
1
>>> (Integer(3) * G).number_of_biconnected_components()
3
```

A block graph is a connected graph in which every biconnected component (block) is a clique. Hence its number of biconnected components is its number of blocks:

```
sage: number_of_blocks = randint(4, 10)
sage: G = graphs.RandomBlockGraph(number_of_blocks, 5)
sage: G.number_of_biconnected_components() == number_of_blocks
True
```

```
>>> from sage.all import *
>>> number_of_blocks = randint(Integer(4), Integer(10))
>>> G = graphs.RandomBlockGraph(number_of_blocks, Integer(5))
>>> G.number_of_biconnected_components() == number_of_blocks
True
```

By definition, an edge is a biconnected component. Hence, the number of biconnected components of a tree is its number of edges:

```
sage: T = graphs.RandomTree(randint(0, 10))
sage: T.number_of_biconnected_components() == T.size()
True
```

```
>>> from sage.all import *
>>> T = graphs.RandomTree(randint(Integer(0), Integer(10)))
>>> T.number_of_biconnected_components() == T.size()
True
```

An isolated vertex is a block but not a biconnected component:

```
sage: G = Graph(3)
sage: len(G.blocks_and_cut_vertices()[0])
3
sage: G.number_of_biconnected_components()
0
```

```
>>> from sage.all import *
>>> G = Graph(Integer(3))
>>> len(G.blocks_and_cut_vertices()[Integer(0)])
3
>>> G.number_of_biconnected_components()
0
```

Check the behavior of parameter `forbidden_vertices`:

```
sage: G = graphs.CycleGraph(5)
sage: G.number_of_biconnected_components()
1
sage: G.number_of_biconnected_components(forbidden_vertices=[0])
3
```

```
>>> from sage.all import *
>>> G = graphs.CycleGraph(Integer(5))
>>> G.number_of_biconnected_components()
1
>>> G.number_of_biconnected_components(forbidden_vertices=[Integer(0)])
3
```

number_of_connected_components(*G*, *forbidden_vertices=None*)

Return the number of connected components.

INPUT:

- `G` – the input graph
- `forbidden_vertices` – list (default: `None`); set of vertices to avoid during the search

EXAMPLES:

```
sage: from sage.graphs.connectivity import number_of_connected_components
sage: G = Graph({0: [1, 3], 1: [2], 2: [3], 4: [5, 6], 5: [6]})
sage: number_of_connected_components(G)
2
sage: G.number_of_connected_components()
2
sage: D = DiGraph({0: [1, 3], 1: [2], 2: [3], 4: [5, 6], 5: [6]})
sage: number_of_connected_components(D)
2
sage: number_of_connected_components(D, forbidden_vertices=[1, 3])
3
```

```
>>> from sage.all import *
>>> from sage.graphs.connectivity import number_of_connected_components
>>> G = Graph({Integer(0): [Integer(1), Integer(3)], Integer(1): [Integer(2)],
↪ Integer(2): [Integer(3)], Integer(4): [Integer(5), Integer(6)],␣
↪Integer(5): [Integer(6)]})
>>> number_of_connected_components(G)
2
>>> G.number_of_connected_components()
2
>>> D = DiGraph({Integer(0): [Integer(1), Integer(3)], Integer(1):␣
↪[Integer(2)], Integer(2): [Integer(3)], Integer(4): [Integer(5),␣
↪Integer(6)], Integer(5): [Integer(6)]})
>>> number_of_connected_components(D)
2
>>> number_of_connected_components(D, forbidden_vertices=[Integer(1),␣
↪Integer(3)])
3
```

number_of_loops()

Return the number of edges that are loops.

EXAMPLES:

```
sage: G = Graph(4, loops=True)
sage: G.add_edges([(0, 0), (1, 1), (2, 2), (3, 3), (2, 3)])
sage: G.edges(sort=True, labels=False)
[(0, 0), (1, 1), (2, 2), (2, 3), (3, 3)]
sage: G.number_of_loops()
4
```

```
>>> from sage.all import *
>>> G = Graph(Integer(4), loops=True)
>>> G.add_edges([(Integer(0), Integer(0)), (Integer(1), Integer(1)),␣
↪(Integer(2), Integer(2)), (Integer(3), Integer(3)), (Integer(2),␣
↪Integer(3))])
>>> G.edges(sort=True, labels=False)
[(0, 0), (1, 1), (2, 2), (2, 3), (3, 3)]
>>> G.number_of_loops()
4
```

```
sage: D = DiGraph(4, loops=True)
sage: D.add_edges([(0, 0), (1, 1), (2, 2), (3, 3), (2, 3)])
sage: D.edges(sort=True, labels=False)
[(0, 0), (1, 1), (2, 2), (2, 3), (3, 3)]
sage: D.number_of_loops()
4
```

```
>>> from sage.all import *
>>> D = DiGraph(Integer(4), loops=True)
>>> D.add_edges([(Integer(0), Integer(0)), (Integer(1), Integer(1)),␣
↪(Integer(2), Integer(2)), (Integer(3), Integer(3)), (Integer(2),␣
↪Integer(3))])
>>> D.edges(sort=True, labels=False)
[(0, 0), (1, 1), (2, 2), (2, 3), (3, 3)]
>>> D.number_of_loops()
4
```

number_of_spanning_trees(*root_vertex=None*)

Return the number of spanning trees in a graph.

In the case of a digraph, counts the number of spanning out-trees rooted in `root_vertex`. Default is to set first vertex as root.

This computation uses Kirchhoff's Matrix Tree Theorem [1] to calculate the number of spanning trees. For complete graphs on n vertices the result can also be reached using Cayley's formula: the number of spanning trees are $n^(n-2)$.

For digraphs, the augmented Kirchhoff Matrix as defined in [2] is used for calculations. Here the result is the number of out-trees rooted at a specific vertex.

INPUT:

- `root_vertex` – a vertex (default: `None`); the vertex that will be used as root for all spanning out-trees if the graph is a directed graph. Otherwise, the first vertex returned by *`vertex_iterator()`* is used.

This argument is ignored if the graph is not a digraph.

See also

spanning_trees() – enumerates all spanning trees of a graph

REFERENCES:

- [1] http://mathworld.wolfram.com/MatrixTreeTheorem.html
- [2] Lih-Hsing Hsu, Cheng-Kuan Lin, "Graph Theory and Interconnection Networks"

AUTHORS:

- Anders Jonsson (2009-10-10)

EXAMPLES:

```
sage: G = graphs.PetersenGraph()
sage: G.number_of_spanning_trees()                                          #␣
↪needs sage.modules
2000
```

```
>>> from sage.all import *
>>> G = graphs.PetersenGraph()
>>> G.number_of_spanning_trees()                                          #␣
↪needs sage.modules
2000
```

```
sage: n = 11
sage: G = graphs.CompleteGraph(n)
sage: ST = G.number_of_spanning_trees()                                     #␣
↪needs sage.modules
sage: ST == n ^ (n - 2)                                                     #␣
↪needs sage.modules
True
```

```
>>> from sage.all import *
>>> n = Integer(11)
>>> G = graphs.CompleteGraph(n)
>>> ST = G.number_of_spanning_trees()                                     #␣
↪needs sage.modules
>>> ST == n ** (n - Integer(2))                                             ␣
↪       # needs sage.modules
True
```

```
sage: M = matrix(3, 3, [0, 1, 0, 0, 0, 1, 1, 1, 0])
sage: D = DiGraph(M)
sage: D.number_of_spanning_trees()
1
sage: D.number_of_spanning_trees(0)
1
sage: D.number_of_spanning_trees(2)
2
```

```
>>> from sage.all import *
>>> M = matrix(Integer(3), Integer(3), [Integer(0), Integer(1), Integer(0),␣
↪Integer(0), Integer(0), Integer(1), Integer(1), Integer(1), Integer(0)])
>>> D = DiGraph(M)
>>> D.number_of_spanning_trees()
1
>>> D.number_of_spanning_trees(Integer(0))
1
>>> D.number_of_spanning_trees(Integer(2))
2
```

odd_girth(*algorithm='bfs'*, *certificate=False*)

Return the odd girth of the graph.

The odd girth is the length of the shortest cycle of odd length in the graph (directed cycle if the graph is directed). Bipartite graphs have infinite odd girth.

INPUT:

- `algorithm` – string (default: `'bfs'`); the algorithm to use:
  - `'bfs'` – BFS-based algorithm
  - any algorithm accepted by `charpoly()` for computation from the characteristic polynomial (see [Har1962] and [Big1993], p. 45)
- `certificate` – boolean (default: `False`); whether to return `(g, c)`, where `g` is the odd girth and `c` is a list of vertices of a (directed) cycle of length `g` in the graph, thus providing a certificate that the odd girth is at most `g`, or `None` if `g` is infinite. So far, this parameter is accepted only when `algorithm = "bfs"`.

EXAMPLES:

The McGee graph has girth 7 and therefore its odd girth is 7 as well:

```
sage: G = graphs.McGeeGraph()
sage: G.girth()
7
sage: G.odd_girth()
7
```

```
>>> from sage.all import *
>>> G = graphs.McGeeGraph()
>>> G.girth()
7
>>> G.odd_girth()
7
```

Any complete (directed) graph on more than 2 vertices contains a (directed) triangle and has thus odd girth 3:

```
sage: G = graphs.CompleteGraph(5)
sage: G.odd_girth(certificate=True)  # random
(3, [2, 1, 0])
sage: G = digraphs.Complete(5)
sage: G.odd_girth(certificate=True)  # random
(3, [1, 2, 0])
```

```
>>> from sage.all import *
>>> G = graphs.CompleteGraph(Integer(5))
>>> G.odd_girth(certificate=True)  # random
(3, [2, 1, 0])
>>> G = digraphs.Complete(Integer(5))
>>> G.odd_girth(certificate=True)  # random
(3, [1, 2, 0])
```

Bipartite graphs have no odd cycle and consequently have infinite odd girth:

```
sage: G = graphs.RandomBipartite(6, 6, .5)                                    #␣
↪needs numpy
sage: G.odd_girth()                                                           #␣
↪needs numpy
+Infinity
sage: G = graphs.Grid2dGraph(3, 4)
sage: G.odd_girth()
+Infinity
```

```
>>> from sage.all import *
>>> G = graphs.RandomBipartite(Integer(6), Integer(6), RealNumber('.5'))      ␣
↪                              # needs numpy
>>> G.odd_girth()                                                            #␣
↪needs numpy
+Infinity
>>> G = graphs.Grid2dGraph(Integer(3), Integer(4))
>>> G.odd_girth()
+Infinity
```

The odd girth of a (directed) graph with loops is 1:

```
sage: # needs networkx
sage: G = graphs.RandomGNP(10, .5)
sage: G.allow_loops(True)
sage: G.add_edge(0, 0)
sage: G.odd_girth()
1
sage: G = digraphs.RandomDirectedGNP(10, .5)
sage: G.allow_loops(True)
sage: G.add_edge(0, 0)
sage: G.odd_girth()
1
```

```
>>> from sage.all import *
>>> # needs networkx
>>> G = graphs.RandomGNP(Integer(10), RealNumber('.5'))
>>> G.allow_loops(True)
>>> G.add_edge(Integer(0), Integer(0))
>>> G.odd_girth()
1
>>> G = digraphs.RandomDirectedGNP(Integer(10), RealNumber('.5'))
>>> G.allow_loops(True)
>>> G.add_edge(Integer(0), Integer(0))
```

(continues on next page)

(continued from previous page)

```
>>> G.odd_girth()
1
```

See also

- *girth()* – return the girth of the graph.

order()

Return the number of vertices.

Note that `len(G)` and *n_vertices()* also return the number of vertices in G.

EXAMPLES:

```
sage: G = graphs.PetersenGraph()
sage: G.order()
10
```

```
>>> from sage.all import *
>>> G = graphs.PetersenGraph()
>>> G.order()
10
```

```
sage: G = graphs.TetrahedralGraph()
sage: len(G)
4
```

```
>>> from sage.all import *
>>> G = graphs.TetrahedralGraph()
>>> len(G)
4
```

pagerank(*alpha=0.85*, *personalization=None*, *by_weight=False*, *weight_function=None*, *check_weight=True*, *dangling=None*, *algorithm='scipy'*)

Return the PageRank of the vertices of `self`.

PageRank is a centrality measure earlier used to rank web pages. The PageRank algorithm outputs the probability distribution that a random walker in the graph visits a vertex.

See the Wikipedia article PageRank for more information.

INPUT:

- `alpha` – float (default: 0.85); damping parameter for PageRank. `alpha` is the click-through probability useful for preventing sinks. The probability at any step, that an imaginary surfer who is randomly clicking on links will continue is a damping factor d.
- `personalization` – dictionary (default: `None`); a dictionary keyed by vertices associating to each vertex a value. The personalization can be specified for a subset of the vertices, if not specified a nodes personalization value will be taken as zero. The sum of the values must be nonzero. By default (`None`), a uniform distribution is used.
- `by_weight` – boolean (default: `False`); if `True`, the edges in the graph are weighted, otherwise all edges have weight 1

- `weight_function` – function (default: `None`); a function that takes as input an edge `(u, v, l)` and outputs its weight. If not `None`, `by_weight` is automatically set to `True`. If `None` and `by_weight` is `True`, we use the edge label `l`, if `l` is not `None`, else `1` as a weight.
- `check_weight` – boolean (default: `True`); whether to check that the `weight_function` outputs a number for each edge
- `dangling` – dictionary (default: `None`); a dictionary keyed by a vertex the outedge of "dangling" vertices, (i.e., vertices without any outedges) points to and the dict value is the weight of that outedge. By default, dangling vertices are given outedges according to the personalization vector (uniform if not specified). It may be common to have the dangling dict to be the same as the personalization dict.
- `algorithm` – string (default: `None`); the algorithm to use in computing PageRank of `G`. The following algorithms are supported:
  - `'NetworkX'` – uses NetworkX's default implementation (Scipy as of 2.6)
  - `'Scipy'` – uses Scipy's PageRank algorithm implementation
  - `'igraph'` – uses igraph's PageRank algorithm implementation
  - `None` – uses best implementation available

OUTPUT: a dictionary containing the PageRank value of each node

Note

Parameters `alpha`, `by_weight` and `weight_function` are common to all algorithms. Parameters `personalization` and `dangling` are used only by algorithms `NetworkX`, `Numpy` and `Scipy`.

EXAMPLES:

```
sage: G = graphs.CycleGraph(4)
sage: G.pagerank(algorithm="Networkx")                                      #␣
↪needs networkx
{0: 0.25, 1: 0.25, 2: 0.25, 3: 0.25}
sage: G.pagerank(alpha=0.50, algorithm='igraph')        # abs tol 1e-9,␣
↪optional - python_igraph
{0: 0.25, 1: 0.25, 2: 0.25, 3: 0.25}
sage: G = Graph([(1, 2, 40), (2, 3, 50), (3, 4, 60),
....:            (1, 4, 70), (4, 5, 80), (5, 6, 20)])
sage: G.pagerank(algorithm="NetworkX")                  # abs tol 1e-9      #␣
↪needs networkx
{1: 0.16112205885619563,
 2: 0.1619531043247219,
 3: 0.16112205885619563,
 4: 0.2374999999999999,
 5: 0.17775588228760858,
 6: 0.100546895675278}
sage: G.pagerank(algorithm='NetworkX', by_weight=True)  # abs tol 1e-9      #␣
↪needs networkx
{1: 0.16459583718588994,
 2: 0.13977928595154515,
 3: 0.16539840184339605,
 4: 0.3063198690713853,
 5: 0.1700057609707141,
```

(continues on next page)

(continued from previous page)

```
 6: 0.05390084497706962}
sage: G.pagerank(algorithm='Scipy')                          # abs tol 1e-9      #␣
↪needs networkx scipy
{1: 0.16112205885619563,
 2: 0.1619531043247219,
 3: 0.16112205885619563,
 4: 0.2374999999999999,
 5: 0.17775588228760858,
 6: 0.100546895675278}
sage: G.pagerank(algorithm='Scipy', by_weight=True)          # abs tol 1e-9      #␣
↪needs networkx scipy
{1: 0.16459583718588994,
 2: 0.13977928595154515,
 3: 0.16539840184339605,
 4: 0.3063198690713853,
 5: 0.1700057609707141,
 6: 0.05390084497706962}
sage: G.pagerank(algorithm='igraph')                         # abs tol 1e-9,␣
↪optional - python_igraph
{1: 0.16112198303979128,
 2: 0.16195368558382262,
 3: 0.16112198303979125,
 4: 0.23749999999999993,
 5: 0.17775603392041744,
 6: 0.10054631441617742}
sage: G.pagerank()                                           # abs tol 1e-9      #␣
↪needs networkx
{1: 0.16112205885619563,
 2: 0.1619531043247219,
 3: 0.16112205885619563,
 4: 0.2374999999999999,
 5: 0.17775588228760858,
 6: 0.100546895675278}
sage: G.pagerank(by_weight=True)                             # abs tol 1e-9      #␣
↪needs networkx
{1: 0.16459583718588994,
 2: 0.13977928595154515,
 3: 0.16539840184339605,
 4: 0.3063198690713853,
 5: 0.1700057609707141,
 6: 0.05390084497706962}
```

```
>>> from sage.all import *
>>> G = graphs.CycleGraph(Integer(4))
>>> G.pagerank(algorithm="Networkx")                                              #␣
↪needs networkx
{0: 0.25, 1: 0.25, 2: 0.25, 3: 0.25}
>>> G.pagerank(alpha=RealNumber('0.50'), algorithm='igraph')          # abs tol␣
↪1e-9, optional - python_igraph
{0: 0.25, 1: 0.25, 2: 0.25, 3: 0.25}
>>> G = Graph([(Integer(1), Integer(2), Integer(40)), (Integer(2), Integer(3),
↪ Integer(50)), (Integer(3), Integer(4), Integer(60)),
```

(continues on next page)

(continued from previous page)

```
...             (Integer(1), Integer(4), Integer(70)), (Integer(4), Integer(5),
↪ Integer(80)), (Integer(5), Integer(6), Integer(20))])
>>> G.pagerank(algorithm="NetworkX")                    # abs tol 1e-9      #␣
↪needs networkx
{1: 0.16112205885619563,
 2: 0.1619531043247219,
 3: 0.16112205885619563,
 4: 0.2374999999999999,
 5: 0.17775588228760858,
 6: 0.100546895675278}
>>> G.pagerank(algorithm='NetworkX', by_weight=True)  # abs tol 1e-9      #␣
↪needs networkx
{1: 0.16459583718588994,
 2: 0.13977928595154515,
 3: 0.16539840184339605,
 4: 0.3063198690713853,
 5: 0.1700057609707141,
 6: 0.05390084497706962}
>>> G.pagerank(algorithm='Scipy')                       # abs tol 1e-9      #␣
↪needs networkx scipy
{1: 0.16112205885619563,
 2: 0.1619531043247219,
 3: 0.16112205885619563,
 4: 0.2374999999999999,
 5: 0.17775588228760858,
 6: 0.100546895675278}
>>> G.pagerank(algorithm='Scipy', by_weight=True)     # abs tol 1e-9      #␣
↪needs networkx scipy
{1: 0.16459583718588994,
 2: 0.13977928595154515,
 3: 0.16539840184339605,
 4: 0.3063198690713853,
 5: 0.1700057609707141,
 6: 0.05390084497706962}
>>> G.pagerank(algorithm='igraph')                      # abs tol 1e-9,␣
↪optional - python_igraph
{1: 0.16112198303979128,
 2: 0.16195368558382262,
 3: 0.16112198303979125,
 4: 0.23749999999999993,
 5: 0.17775603392041744,
 6: 0.10054631441617742}
>>> G.pagerank()                                        # abs tol 1e-9      #␣
↪needs networkx
{1: 0.16112205885619563,
 2: 0.1619531043247219,
 3: 0.16112205885619563,
 4: 0.2374999999999999,
 5: 0.17775588228760858,
 6: 0.100546895675278}
>>> G.pagerank(by_weight=True)                          # abs tol 1e-9      #␣
↪needs networkx
```

(continues on next page)

(continued from previous page)

```
{1: 0.16459583718588994,
 2: 0.13977928595154515,
 3: 0.16539840184339605,
 4: 0.3063198690713853,
 5: 0.1700057609707141,
 6: 0.05390084497706962}
```

See also

- Wikipedia article PageRank

planar_dual(*embedding=None*)

Return the planar dual of an embedded graph.

A combinatorial embedding of a graph is a clockwise ordering of the neighbors of each vertex. From this information one can obtain the dual of a plane graph, which is what the method returns. The vertices of the dual graph correspond to faces of the primal graph.

INPUT:

- `embedding` – dictionary (default: `None`); a combinatorial embedding dictionary. Format: `{v1: [v2, v3], v2: [v1], v3: [v1]}` (clockwise ordering of neighbors at each vertex). If set to `None` (default) the method will use the embedding stored as `self._embedding`. If none is stored, the method will compute the set of faces from the embedding returned by *`is_planar()`* (if the graph is, of course, planar).

EXAMPLES:

```
sage: C = graphs.CubeGraph(3)
sage: C.planar_dual()
Graph on 6 vertices
sage: graphs.IcosahedralGraph().planar_dual().is_isomorphic(graphs.
↪DodecahedralGraph())
True
```

```
>>> from sage.all import *
>>> C = graphs.CubeGraph(Integer(3))
>>> C.planar_dual()
Graph on 6 vertices
>>> graphs.IcosahedralGraph().planar_dual().is_isomorphic(graphs.
↪DodecahedralGraph())
True
```

The planar dual of the planar dual is isomorphic to the graph itself:

```
sage: g = graphs.BuckyBall()
sage: g.planar_dual().planar_dual().is_isomorphic(g)
True
```

```
>>> from sage.all import *
>>> g = graphs.BuckyBall()
```

(continues on next page)

(continued from previous page)

```
>>> g.planar_dual().planar_dual().is_isomorphic(g)
True
```

See also

- *faces()*
- *set_embedding()*
- *get_embedding()*
- *is_planar()*

Todo

Implement the method for graphs that are not 3-vertex-connected, or at least have a faster 3-vertex-connectivity test (Issue #24635).

plot(***options*)

Return a Graphics object representing the (di)graph.

INPUT:

See the documentation of the *sage.graphs.graph_plot* module for supported parameters. In addition, this method supports all parameters of sage.plot.graphics.Graphics.show().

Note

- Default parameters for this method *and a specific graph* can also be set through the options mechanism. For more information on this different way to set default parameters, see the help of the options decorator.
- See also the *sage.graphs.graph_latex* module for ways to use LaTeX to produce an image of a graph.

EXAMPLES:

```
sage: from sage.graphs.graph_plot import graphplot_options
sage: sorted(graphplot_options.items())
[...]

sage: from math import sin, cos, pi
sage: P = graphs.PetersenGraph()
sage: d = {'#FF0000': [0, 5], '#FF9900': [1, 6], '#FFFF00': [2, 7],
....:      '#00FF00': [3, 8], '#0000FF': [4, 9]}
sage: pos_dict = {}
sage: for i in range(5):
....:  x = float(cos(pi/2 + ((2*pi)/5)*i))
....:  y = float(sin(pi/2 + ((2*pi)/5)*i))
....:  pos_dict[i] = [x,y]
```

(continues on next page)

(continued from previous page)

```
sage: for i in range(5, 10):
....:  x = float(0.5*cos(pi/2 + ((2*pi)/5)*i))
....:  y = float(0.5*sin(pi/2 + ((2*pi)/5)*i))
....:  pos_dict[i] = [x,y]
sage: pl = P.plot(pos=pos_dict, vertex_colors=d)                              #␣
↪needs sage.plot
sage: pl.show()                                                              #␣
↪needs sage.plot
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_plot import graphplot_options
>>> sorted(graphplot_options.items())
[...]

>>> from math import sin, cos, pi
>>> P = graphs.PetersenGraph()
>>> d = {'#FF0000': [Integer(0), Integer(5)], '#FF9900': [Integer(1),␣
↪Integer(6)], '#FFFF00': [Integer(2), Integer(7)],
...       '#00FF00': [Integer(3), Integer(8)], '#0000FF': [Integer(4),␣
↪Integer(9)]}
>>> pos_dict = {}
>>> for i in range(Integer(5)):
...  x = float(cos(pi/Integer(2) + ((Integer(2)*pi)/Integer(5))*i))
...  y = float(sin(pi/Integer(2) + ((Integer(2)*pi)/Integer(5))*i))
...  pos_dict[i] = [x,y]
>>> for i in range(Integer(5), Integer(10)):
...  x = float(RealNumber('0.5')*cos(pi/Integer(2) + ((Integer(2)*pi)/
↪Integer(5))*i))
...  y = float(RealNumber('0.5')*sin(pi/Integer(2) + ((Integer(2)*pi)/
↪Integer(5))*i))
...  pos_dict[i] = [x,y]
>>> pl = P.plot(pos=pos_dict, vertex_colors=d)                               #␣
↪needs sage.plot
>>> pl.show()                                                               #␣
↪needs sage.plot
```

```
sage: C = graphs.CubeGraph(8)
sage: P = C.plot(vertex_labels=False, vertex_size=0, graph_border=True)      #␣
↪needs sage.plot
sage: P.show()                                                               #␣
↪needs sage.plot
```

```
>>> from sage.all import *
>>> C = graphs.CubeGraph(Integer(8))
>>> P = C.plot(vertex_labels=False, vertex_size=Integer(0), graph_
↪border=True)      # needs sage.plot
>>> P.show()                                                                #␣
↪needs sage.plot
```

```
sage: G = graphs.HeawoodGraph()
sage: for u, v, l in G.edges(sort=False):
```

(continues on next page)

(continued from previous page)

```
....:     G.set_edge_label(u, v, '(' + str(u) + ',' + str(v) + ')')
sage: G.plot(edge_labels=True).show()                                          #␣
↪needs sage.plot
```

```
>>> from sage.all import *
>>> G = graphs.HeawoodGraph()
>>> for u, v, l in G.edges(sort=False):
...     G.set_edge_label(u, v, '(' + str(u) + ',' + str(v) + ')')
>>> G.plot(edge_labels=True).show()                                            #␣
↪needs sage.plot
```

```
sage: D = DiGraph({0: [1, 10, 19], 1: [8, 2], 2: [3, 6], 3: [19, 4],
....:              4: [17, 5], 5: [6, 15], 6: [7], 7: [8, 14],
....:              8: [9], 9: [10, 13], 10: [11], 11: [12, 18],
....:              12: [16, 13], 13: [14], 14: [15], 15: [16],
....:              16: [17], 17: [18], 18: [19]}, sparse=True)
sage: for u,v,l in D.edges(sort=False):
....:     D.set_edge_label(u, v, '(' + str(u) + ',' + str(v) + ')')
sage: D.plot(edge_labels=True, layout='circular').show()                        #␣
↪needs sage.plot
```

```
>>> from sage.all import *
>>> D = DiGraph({Integer(0): [Integer(1), Integer(10), Integer(19)],␣
↪Integer(1): [Integer(8), Integer(2)], Integer(2): [Integer(3), Integer(6)],␣
↪Integer(3): [Integer(19), Integer(4)],
...               Integer(4): [Integer(17), Integer(5)], Integer(5):␣
↪[Integer(6), Integer(15)], Integer(6): [Integer(7)], Integer(7):␣
↪[Integer(8), Integer(14)],
...               Integer(8): [Integer(9)], Integer(9): [Integer(10),␣
↪Integer(13)], Integer(10): [Integer(11)], Integer(11): [Integer(12),␣
↪Integer(18)],
...               Integer(12): [Integer(16), Integer(13)], Integer(13):␣
↪[Integer(14)], Integer(14): [Integer(15)], Integer(15): [Integer(16)],
...               Integer(16): [Integer(17)], Integer(17): [Integer(18)],␣
↪Integer(18): [Integer(19)]}, sparse=True)
>>> for u,v,l in D.edges(sort=False):
...     D.set_edge_label(u, v, '(' + str(u) + ',' + str(v) + ')')
>>> D.plot(edge_labels=True, layout='circular').show()                          #␣
↪needs sage.plot
```

```
sage: from sage.plot.colors import rainbow
sage: C = graphs.CubeGraph(5)
sage: R = rainbow(5)
sage: edge_colors = {R[i]: [] for i in range(5)}
sage: for u, v, l in C.edges(sort=False):
....:  for i in range(5):
....:      if u[i] != v[i]:
....:          edge_colors[R[i]].append((u, v, l))
sage: C.plot(vertex_labels=False, vertex_size=0,
....:        edge_colors=edge_colors).show()
```

```
>>> from sage.all import *
>>> from sage.plot.colors import rainbow
>>> C = graphs.CubeGraph(Integer(5))
>>> R = rainbow(Integer(5))
>>> edge_colors = {R[i]: [] for i in range(Integer(5))}
>>> for u, v, l in C.edges(sort=False):
...  for i in range(Integer(5)):
...      if u[i] != v[i]:
...          edge_colors[R[i]].append((u, v, l))
>>> C.plot(vertex_labels=False, vertex_size=Integer(0),
...        edge_colors=edge_colors).show()
```

```
sage: D = graphs.DodecahedralGraph()
sage: Pi = [[6,5,15,14,7], [16,13,8,2,4], [12,17,9,3,1], [0,19,18,10,11]]
sage: D.show(partition=Pi)                                                  #␣
↪needs sage.plot
```

```
>>> from sage.all import *
>>> D = graphs.DodecahedralGraph()
>>> Pi = [[Integer(6),Integer(5),Integer(15),Integer(14),Integer(7)],␣
↪[Integer(16),Integer(13),Integer(8),Integer(2),Integer(4)], [Integer(12),
↪Integer(17),Integer(9),Integer(3),Integer(1)], [Integer(0),Integer(19),
↪Integer(18),Integer(10),Integer(11)]]
>>> D.show(partition=Pi)                                                  #␣
↪needs sage.plot
```

```
sage: G = graphs.PetersenGraph()
sage: G.allow_loops(True)
sage: G.add_edge(0, 0)
sage: G.show()                                                              #␣
↪needs sage.plot
```

```
>>> from sage.all import *
>>> G = graphs.PetersenGraph()
>>> G.allow_loops(True)
>>> G.add_edge(Integer(0), Integer(0))
>>> G.show()                                                              #␣
↪needs sage.plot
```

```
sage: D = DiGraph({0: [0, 1], 1: [2], 2: [3]}, loops=True)
sage: D.show()                                                              #␣
↪needs sage.plot
sage: D.show(edge_colors={(0, 1, 0): [(0, 1, None), (1, 2, None)],          #␣
↪needs sage.plot
....:                     (0, 0, 0): [(2, 3, None)]})
```

```
>>> from sage.all import *
>>> D = DiGraph({Integer(0): [Integer(0), Integer(1)], Integer(1):␣
↪[Integer(2)], Integer(2): [Integer(3)]}, loops=True)
>>> D.show()                                                              #␣
↪needs sage.plot
```

(continues on next page)

(continued from previous page)

```
>>> D.show(edge_colors={(Integer(0), Integer(1), Integer(0)): [(Integer(0),␣
↪Integer(1), None), (Integer(1), Integer(2), None)],          # needs sage.
↪plot
...                     (Integer(0), Integer(0), Integer(0)): [(Integer(2),␣
↪Integer(3), None)]})
```

```
sage: pos = {0: [0.0, 1.5], 1: [-0.8, 0.3], 2: [-0.6, -0.8], 3: [0.6, -0.8],␣
↪4: [0.8, 0.3]}
sage: g = Graph({0: [1], 1: [2], 2: [3], 3: [4], 4: [0]})
sage: g.plot(pos=pos, layout='spring', iterations=0)                          #␣
↪needs sage.plot
Graphics object consisting of 11 graphics primitives
```

```
>>> from sage.all import *
>>> pos = {Integer(0): [RealNumber('0.0'), RealNumber('1.5')], Integer(1): [-
↪RealNumber('0.8'), RealNumber('0.3')], Integer(2): [-RealNumber('0.6'), -
↪RealNumber('0.8')], Integer(3): [RealNumber('0.6'), -RealNumber('0.8')],␣
↪Integer(4): [RealNumber('0.8'), RealNumber('0.3')]}
>>> g = Graph({Integer(0): [Integer(1)], Integer(1): [Integer(2)],␣
↪Integer(2): [Integer(3)], Integer(3): [Integer(4)], Integer(4):␣
↪[Integer(0)]})
>>> g.plot(pos=pos, layout='spring', iterations=Integer(0))                  ␣
↪     # needs sage.plot
Graphics object consisting of 11 graphics primitives
```

```
sage: G = Graph()
sage: P = G.plot()                                                           #␣
↪needs sage.plot
sage: P.axes()                                                               #␣
↪needs sage.plot
False
sage: G = DiGraph()
sage: P = G.plot()                                                           #␣
↪needs sage.plot
sage: P.axes()                                                               #␣
↪needs sage.plot
False
```

```
>>> from sage.all import *
>>> G = Graph()
>>> P = G.plot()                                                           #␣
↪needs sage.plot
>>> P.axes()                                                               #␣
↪needs sage.plot
False
>>> G = DiGraph()
>>> P = G.plot()                                                           #␣
↪needs sage.plot
>>> P.axes()                                                               #␣
↪needs sage.plot
False
```

```
sage: G = graphs.PetersenGraph()
sage: G.get_pos()
{0: (0.0..., 1.0...),
 1: (-0.95..., 0.30...),
 2: (-0.58..., -0.80...),
 3: (0.58..., -0.80...),
 4: (0.95..., 0.30...),
 5: (0.0..., 0.5...),
 6: (-0.47..., 0.15...),
 7: (-0.29..., -0.40...),
 8: (0.29..., -0.40...),
 9: (0.47..., 0.15...)}
sage: P = G.plot(save_pos=True, layout='spring')                    #␣
↪needs sage.plot
```

```
>>> from sage.all import *
>>> G = graphs.PetersenGraph()
>>> G.get_pos()
{0: (0.0..., 1.0...),
 1: (-0.95..., 0.30...),
 2: (-0.58..., -0.80...),
 3: (0.58..., -0.80...),
 4: (0.95..., 0.30...),
 5: (0.0..., 0.5...),
 6: (-0.47..., 0.15...),
 7: (-0.29..., -0.40...),
 8: (0.29..., -0.40...),
 9: (0.47..., 0.15...)}
>>> P = G.plot(save_pos=True, layout='spring')                    #␣
↪needs sage.plot
```

The following illustrates the format of a position dictionary:

```
sage: G.get_pos()  # currently random across platforms, see #9593          #␣
↪needs sage.plot
{0: [1.17..., -0.855...],
 1: [1.81..., -0.0990...],
 2: [1.35..., 0.184...],
 3: [1.51..., 0.644...],
 4: [2.00..., -0.507...],
 5: [0.597..., -0.236...],
 6: [2.04..., 0.687...],
 7: [1.46..., -0.473...],
 8: [0.902..., 0.773...],
 9: [2.48..., -0.119...]}
```

```
>>> from sage.all import *
>>> G.get_pos()  # currently random across platforms, see #9593          #␣
↪needs sage.plot
{0: [1.17..., -0.855...],
 1: [1.81..., -0.0990...],
 2: [1.35..., 0.184...],
```

(continues on next page)

(continued from previous page)

```
 3: [1.51..., 0.644...],
 4: [2.00..., -0.507...],
 5: [0.597..., -0.236...],
 6: [2.04..., 0.687...],
 7: [1.46..., -0.473...],
 8: [0.902..., 0.773...],
 9: [2.48..., -0.119...]}
```

```
sage: T = list(graphs.trees(7))
sage: t = T[3]
sage: t.plot(heights={0: [0], 1: [4, 5, 1], 2: [2], 3: [3, 6]})             #␣
↪needs sage.plot
Graphics object consisting of 14 graphics primitives
```

```
>>> from sage.all import *
>>> T = list(graphs.trees(Integer(7)))
>>> t = T[Integer(3)]
>>> t.plot(heights={Integer(0): [Integer(0)], Integer(1): [Integer(4),␣
↪Integer(5), Integer(1)], Integer(2): [Integer(2)], Integer(3): [Integer(3),␣
↪Integer(6)]})             # needs sage.plot
Graphics object consisting of 14 graphics primitives
```

```
sage: T = list(graphs.trees(7))
sage: t = T[3]
sage: t.plot(heights={0: [0], 1: [4, 5, 1], 2: [2], 3: [3, 6]})             #␣
↪needs sage.plot
Graphics object consisting of 14 graphics primitives
sage: t.set_edge_label(0, 1, -7)
sage: t.set_edge_label(0, 5, 3)
sage: t.set_edge_label(0, 5, 99)
sage: t.set_edge_label(1, 2, 1000)
sage: t.set_edge_label(3, 2, 'spam')
sage: t.set_edge_label(2, 6, 3/2)
sage: t.set_edge_label(0, 4, 66)
sage: t.plot(heights={0: [0], 1: [4, 5, 1], 2: [2], 3: [3, 6]},             #␣
↪needs sage.plot
....:        edge_labels=True)
Graphics object consisting of 20 graphics primitives
```

```
>>> from sage.all import *
>>> T = list(graphs.trees(Integer(7)))
>>> t = T[Integer(3)]
>>> t.plot(heights={Integer(0): [Integer(0)], Integer(1): [Integer(4),␣
↪Integer(5), Integer(1)], Integer(2): [Integer(2)], Integer(3): [Integer(3),␣
↪Integer(6)]})             # needs sage.plot
Graphics object consisting of 14 graphics primitives
>>> t.set_edge_label(Integer(0), Integer(1), -Integer(7))
>>> t.set_edge_label(Integer(0), Integer(5), Integer(3))
>>> t.set_edge_label(Integer(0), Integer(5), Integer(99))
>>> t.set_edge_label(Integer(1), Integer(2), Integer(1000))
>>> t.set_edge_label(Integer(3), Integer(2), 'spam')
```

(continues on next page)

(continued from previous page)

```
>>> t.set_edge_label(Integer(2), Integer(6), Integer(3)/Integer(2))
>>> t.set_edge_label(Integer(0), Integer(4), Integer(66))
>>> t.plot(heights={Integer(0): [Integer(0)], Integer(1): [Integer(4),␣
↪Integer(5), Integer(1)], Integer(2): [Integer(2)], Integer(3): [Integer(3),␣
↪Integer(6)]},             # needs sage.plot
...         edge_labels=True)
Graphics object consisting of 20 graphics primitives
```

```
sage: T = list(graphs.trees(7))
sage: t = T[3]
sage: t.plot(layout='tree')                                                 #␣
↪needs sage.plot
Graphics object consisting of 14 graphics primitives
```

```
>>> from sage.all import *
>>> T = list(graphs.trees(Integer(7)))
>>> t = T[Integer(3)]
>>> t.plot(layout='tree')                                                 #␣
↪needs sage.plot
Graphics object consisting of 14 graphics primitives
```

```
sage: t = DiGraph('JCC???@A??GO??CO??GO??')
sage: t.plot(layout='tree', tree_root=0, tree_orientation='up')             #␣
↪needs sage.plot
Graphics object consisting of 22 graphics primitives
sage: D = DiGraph({0: [1, 2, 3], 2: [1, 4], 3: [0]})
sage: D.plot()                                                              #␣
↪needs sage.plot
Graphics object consisting of 16 graphics primitives

sage: D = DiGraph(multiedges=True,sparse=True)
sage: for i in range(5):
....:   D.add_edge((i, i + 1, 'a'))
....:   D.add_edge((i, i - 1, 'b'))
sage: D.plot(edge_labels=True, edge_colors=D._color_by_label())             #␣
↪needs sage.plot
Graphics object consisting of 34 graphics primitives
sage: D.plot(edge_labels=True, color_by_label={'a': 'blue', 'b': 'red'},    #␣
↪needs sage.plot
....:        edge_style='dashed')
Graphics object consisting of 34 graphics primitives

sage: g = Graph({}, loops=True, multiedges=True, sparse=True)
sage: g.add_edges([(0, 0, 'a'), (0, 0, 'b'), (0, 1, 'c'), (0, 1, 'd'),
....:   (0, 1, 'e'), (0, 1, 'f'), (0, 1, 'f'), (2, 1, 'g'), (2, 2, 'h')])
sage: g.plot(edge_labels=True, color_by_label=True, edge_style='dashed')    #␣
↪needs sage.plot
Graphics object consisting of 22 graphics primitives
```

```
>>> from sage.all import *
>>> t = DiGraph('JCC???@A??GO??CO??GO??')
```

(continues on next page)

(continued from previous page)

```
>>> t.plot(layout='tree', tree_root=Integer(0), tree_orientation='up')       ␣
↪     # needs sage.plot
Graphics object consisting of 22 graphics primitives
>>> D = DiGraph({Integer(0): [Integer(1), Integer(2), Integer(3)],␣
↪Integer(2): [Integer(1), Integer(4)], Integer(3): [Integer(0)]})
>>> D.plot()                                                                 #␣
↪needs sage.plot
Graphics object consisting of 16 graphics primitives

>>> D = DiGraph(multiedges=True,sparse=True)
>>> for i in range(Integer(5)):
...   D.add_edge((i, i + Integer(1), 'a'))
...   D.add_edge((i, i - Integer(1), 'b'))
>>> D.plot(edge_labels=True, edge_colors=D._color_by_label())               #␣
↪needs sage.plot
Graphics object consisting of 34 graphics primitives
>>> D.plot(edge_labels=True, color_by_label={'a': 'blue', 'b': 'red'},     #␣
↪needs sage.plot
...         edge_style='dashed')
Graphics object consisting of 34 graphics primitives

>>> g = Graph({}, loops=True, multiedges=True, sparse=True)
>>> g.add_edges([(Integer(0), Integer(0), 'a'), (Integer(0), Integer(0), 'b'),
↪ (Integer(0), Integer(1), 'c'), (Integer(0), Integer(1), 'd'),
...   (Integer(0), Integer(1), 'e'), (Integer(0), Integer(1), 'f'),␣
↪(Integer(0), Integer(1), 'f'), (Integer(2), Integer(1), 'g'), (Integer(2),␣
↪Integer(2), 'h')])
>>> g.plot(edge_labels=True, color_by_label=True, edge_style='dashed')     #␣
↪needs sage.plot
Graphics object consisting of 22 graphics primitives
```

```
sage: S = SupersingularModule(389)
sage: H = S.hecke_matrix(2)
sage: D = DiGraph(H, sparse=True)
sage: P = D.plot()                                                          #␣
↪needs sage.plot
```

```
>>> from sage.all import *
>>> S = SupersingularModule(Integer(389))
>>> H = S.hecke_matrix(Integer(2))
>>> D = DiGraph(H, sparse=True)
>>> P = D.plot()                                                            #␣
↪needs sage.plot
```

```
sage: G = Graph({'a': ['a','b','b','b','e'], 'b': ['c','d','e'],
....:            'c':['c','d','d','d'],'d':['e']}, sparse=True)
sage: G.show(pos={'a':[0,1],'b':[1,1],'c':[2,0],'d':[1,0],'e':[0,0]})       #␣
↪needs sage.plot
```

```
>>> from sage.all import *
>>> G = Graph({'a': ['a','b','b','b','e'], 'b': ['c','d','e'],
```

(continues on next page)

(continued from previous page)

```
...             'c':['c','d','d','d'],'d':['e']}, sparse=True)
>>> G.show(pos={'a':[Integer(0),Integer(1)],'b':[Integer(1),Integer(1)],'c
↪':[Integer(2),Integer(0)],'d':[Integer(1),Integer(0)],'e':[Integer(0),
↪Integer(0)]})       # needs sage.plot
```

plot3d(*bgcolor=(1, 1, 1)*, *vertex_colors=None*, *vertex_size=0.06*, *vertex_labels=False*, *edge_colors=None*, *edge_size=0.02*, *edge_size2=0.0325*, *pos3d=None*, *color_by_label=False*, *engine='threejs'*, ***kwds*)

Plot a graph in three dimensions.

See also the *sage.graphs.graph_latex* module for ways to use LaTeX to produce an image of a graph.

INPUT:

- `bgcolor` – rgb tuple (default: `(1,1,1)`)
- `vertex_size` – float (default: 0.06)
- `vertex_labels` – boolean (default: `False`); whether to display vertices using text labels instead of spheres
- `vertex_colors` – dictionary (default: `None`); optional dictionary to specify vertex colors: each key is a color recognizable by `tachyon` (rgb tuple (default: `(1,0,0)`)), and each corresponding entry is a list of vertices. If a vertex is not listed, it looks invisible on the resulting plot (it does not get drawn).
- `edge_colors` – dictionary (default: `None`); a dictionary specifying edge colors: each key is a color recognized by `tachyon` (default: `(0,0,0)`), and each entry is a list of edges.
- `color_by_label` – boolean or dictionary or function (default: `False`) whether to color each edge with a different color according to its label; the colors are chosen along a rainbow, unless they are specified by a function or dictionary mapping labels to colors; this option is incompatible with `edge_color` and `edge_colors`.
- `edge_size` – float (default: 0.02)
- `edge_size2` – float (default: 0.0325); used for `Tachyon` sleeves
- `pos3d` – a position dictionary for the vertices
- `layout`, `iterations`, ... – layout options; see *layout()*
- `engine` – string (default: `'threejs'`); the renderer to use among:
  - `'threejs'`: interactive web-based 3D viewer using JavaScript and a WebGL renderer
  - `'jmol'`: interactive 3D viewer using Java
  - `'tachyon'`: ray tracer generating a static PNG image
- `xres` – resolution
- `yres` – resolution
- `**kwds` – passed on to the rendering engine

EXAMPLES:

```
sage: G = graphs.CubeGraph(5)
sage: G.plot3d(iterations=500, edge_size=None, vertex_size=0.04)     # long␣
↪time, needs sage.plot
Graphics3d Object
```

```
>>> from sage.all import *
>>> G = graphs.CubeGraph(Integer(5))
>>> G.plot3d(iterations=Integer(500), edge_size=None, vertex_size=RealNumber(
↪'0.04'))     # long time, needs sage.plot
Graphics3d Object
```

We plot a fairly complicated Cayley graph:

```
sage: A5 = AlternatingGroup(5); A5                                          #␣
↪needs sage.groups
Alternating group of order 5!/2 as a permutation group
sage: G = A5.cayley_graph()                                                 #␣
↪needs sage.groups
sage: G.plot3d(vertex_size=0.03, edge_size=0.01,     # long time            #␣
↪needs sage.groups sage.plot
....:          vertex_colors={(1,1,1): list(G)}, bgcolor=(0,0,0),
....:          color_by_label=True, iterations=200)
Graphics3d Object
```

```
>>> from sage.all import *
>>> A5 = AlternatingGroup(Integer(5)); A5                                    ␣
↪     # needs sage.groups
Alternating group of order 5!/2 as a permutation group
>>> G = A5.cayley_graph()                                                  #␣
↪needs sage.groups
>>> G.plot3d(vertex_size=RealNumber('0.03'), edge_size=RealNumber('0.01'),
↪# long time             # needs sage.groups sage.plot
...          vertex_colors={(Integer(1),Integer(1),Integer(1)): list(G)},␣
↪bgcolor=(Integer(0),Integer(0),Integer(0)),
...          color_by_label=True, iterations=Integer(200))
Graphics3d Object
```

Some `Tachyon` examples:

```
sage: D = graphs.DodecahedralGraph()
sage: P3D = D.plot3d(engine='tachyon')                                      #␣
↪needs sage.plot
sage: P3D.show()                    # long time                             #␣
↪needs sage.plot
```

```
>>> from sage.all import *
>>> D = graphs.DodecahedralGraph()
>>> P3D = D.plot3d(engine='tachyon')                                        #␣
↪needs sage.plot
>>> P3D.show()                    # long time                               #␣
↪needs sage.plot
```

```
sage: G = graphs.PetersenGraph()
sage: G.plot3d(engine='tachyon',     # long time                            #␣
↪needs sage.plot
....:          vertex_colors={(0,0,1): list(G)}).show()
```

```
>>> from sage.all import *
>>> G = graphs.PetersenGraph()
>>> G.plot3d(engine='tachyon',     # long time                                #␣
↪needs sage.plot
...          vertex_colors={(Integer(0),Integer(0),Integer(1)): list(G)}).
↪show()
```

```
sage: C = graphs.CubeGraph(4)
sage: C.plot3d(engine='tachyon',     # long time                              #␣
↪needs sage.plot
....:          edge_colors={(0,1,0): C.edges(sort=False)},
....:          vertex_colors={(1,1,1): list(C)}, bgcolor=(0,0,0)).show()
```

```
>>> from sage.all import *
>>> C = graphs.CubeGraph(Integer(4))
>>> C.plot3d(engine='tachyon',     # long time                              #␣
↪needs sage.plot
...          edge_colors={(Integer(0),Integer(1),Integer(0)): C.
↪edges(sort=False)},
...          vertex_colors={(Integer(1),Integer(1),Integer(1)): list(C)},␣
↪bgcolor=(Integer(0),Integer(0),Integer(0))).show()
```

```
sage: K = graphs.CompleteGraph(3)
sage: K.plot3d(engine='tachyon',     # long time                              #␣
↪needs sage.plot
....:          edge_colors={(1,0,0): [(0,1,None)],
....:                       (0,1,0): [(0,2,None)],
....:                       (0,0,1): [(1,2,None)]}).show()
```

```
>>> from sage.all import *
>>> K = graphs.CompleteGraph(Integer(3))
>>> K.plot3d(engine='tachyon',     # long time                              #␣
↪needs sage.plot
...          edge_colors={(Integer(1),Integer(0),Integer(0)): [(Integer(0),
↪Integer(1),None)],
...                       (Integer(0),Integer(1),Integer(0)): [(Integer(0),
↪Integer(2),None)],
...                       (Integer(0),Integer(0),Integer(1)): [(Integer(1),
↪Integer(2),None)]}).show()
```

A directed version of the dodecahedron

```
sage: D = DiGraph({0: [1, 10, 19], 1: [8, 2], 2: [3, 6], 3: [19, 4],
....:              4: [17, 5], 5: [6, 15], 6: [7], 7: [8, 14],
....:              8: [9], 9: [10, 13], 10: [11], 11: [12, 18],
....:              12: [16, 13], 13: [14], 14: [15], 15: [16], 16: [17],
....:              17: [18], 18: [19], 19: []})
sage: D.plot3d().show()             # long time                              #␣
↪needs sage.plot
```

```
>>> from sage.all import *
>>> D = DiGraph({Integer(0): [Integer(1), Integer(10), Integer(19)],␣
```

(continues on next page)

(continued from previous page)

```
↪Integer(1): [Integer(8), Integer(2)], Integer(2): [Integer(3), Integer(6)],␣
↪Integer(3): [Integer(19), Integer(4)],
...               Integer(4): [Integer(17), Integer(5)], Integer(5):␣
↪[Integer(6), Integer(15)], Integer(6): [Integer(7)], Integer(7):␣
↪[Integer(8), Integer(14)],
...               Integer(8): [Integer(9)], Integer(9): [Integer(10),␣
↪Integer(13)], Integer(10): [Integer(11)], Integer(11): [Integer(12),␣
↪Integer(18)],
...               Integer(12): [Integer(16), Integer(13)], Integer(13):␣
↪[Integer(14)], Integer(14): [Integer(15)], Integer(15): [Integer(16)],␣
↪Integer(16): [Integer(17)],
...               Integer(17): [Integer(18)], Integer(18): [Integer(19)],␣
↪Integer(19): []})
>>> D.plot3d().show()             # long time                                #␣
↪needs sage.plot
```

```
sage: P = graphs.PetersenGraph().to_directed()
sage: from sage.plot.colors import rainbow                                    #␣
↪needs sage.plot
sage: R = rainbow(P.size(), 'rgbtuple')                                       #␣
↪needs sage.plot
sage: edge_colors = {R[i]: [e] for i, e in enumerate(P.edge_iterator())}      #␣
↪needs sage.plot
sage: P.plot3d(engine='tachyon', edge_colors=edge_colors).show()     # long␣
↪time, needs sage.plot
```

```
>>> from sage.all import *
>>> P = graphs.PetersenGraph().to_directed()
>>> from sage.plot.colors import rainbow                                      #␣
↪needs sage.plot
>>> R = rainbow(P.size(), 'rgbtuple')                                         #␣
↪needs sage.plot
>>> edge_colors = {R[i]: [e] for i, e in enumerate(P.edge_iterator())}        #␣
↪needs sage.plot
>>> P.plot3d(engine='tachyon', edge_colors=edge_colors).show()     # long time,
↪ needs sage.plot
```

```
sage: G = Graph({'a': ['a','b','b','b','e'], 'b': ['c','d','e'],
....:            'c': ['c','d','d','d'], 'd': ['e']}, sparse=True)
sage: G.show3d()
Traceback (most recent call last):
...
NotImplementedError: 3D plotting of multiple edges or loops not implemented
```

```
>>> from sage.all import *
>>> G = Graph({'a': ['a','b','b','b','e'], 'b': ['c','d','e'],
...            'c': ['c','d','d','d'], 'd': ['e']}, sparse=True)
>>> G.show3d()
Traceback (most recent call last):
...
NotImplementedError: 3D plotting of multiple edges or loops not implemented
```

Using the `partition` keyword:

```
sage: G = graphs.WheelGraph(7)
sage: G.plot3d(partition=[[0], [1, 2, 3, 4, 5, 6]])                    #␣
↪needs sage.plot
Graphics3d Object
```

```
>>> from sage.all import *
>>> G = graphs.WheelGraph(Integer(7))
>>> G.plot3d(partition=[[Integer(0)], [Integer(1), Integer(2), Integer(3),␣
↪Integer(4), Integer(5), Integer(6)]])                    # needs sage.
↪plot
Graphics3d Object
```

See also

- *plot()*
- *graphviz_string()*

power(k)

Return the k-th power graph of `self`.

In the k-th power graph of a graph G, there is an edge between any pair of vertices at distance at most k in G, where the distance is considered in the unweighted graph. In a directed graph, there is an arc from a vertex u to a vertex v if there is a path of length at most k in G from u to v.

INPUT:

- `k` – integer; the maximum path length for considering edges in the power graph

OUTPUT: the k-th power graph based on shortest distances between nodes

EXAMPLES:

Testing on undirected graphs:

```
sage: G = Graph([(0, 1), (1, 2), (2, 3), (3, 0), (2, 4), (4, 5)])
sage: PG = G.power(2)
sage: PG.edges(sort=True, labels=False)
[(0, 1), (0, 2), (0, 3), (1, 2), (1, 3), (1, 4), (2, 3), (2, 4), (2, 5), (3,␣
↪4), (4, 5)]
```

```
>>> from sage.all import *
>>> G = Graph([(Integer(0), Integer(1)), (Integer(1), Integer(2)),␣
↪(Integer(2), Integer(3)), (Integer(3), Integer(0)), (Integer(2),␣
↪Integer(4)), (Integer(4), Integer(5))])
>>> PG = G.power(Integer(2))
>>> PG.edges(sort=True, labels=False)
[(0, 1), (0, 2), (0, 3), (1, 2), (1, 3), (1, 4), (2, 3), (2, 4), (2, 5), (3,␣
↪4), (4, 5)]
```

Testing on directed graphs:

```
sage: G = DiGraph([(0, 1), (1, 2), (2, 3), (3, 0), (2, 4), (4, 5)])
sage: PG = G.power(3)
sage: PG.edges(sort=True, labels=False)
[(0, 1), (0, 2), (0, 3), (0, 4), (1, 0), (1, 2), (1, 3), (1, 4), (1, 5), (2,␣
↪0), (2, 1), (2, 3), (2, 4), (2, 5), (3, 0), (3, 1), (3, 2), (4, 5)]
```

```
>>> from sage.all import *
>>> G = DiGraph([(Integer(0), Integer(1)), (Integer(1), Integer(2)),␣
↪(Integer(2), Integer(3)), (Integer(3), Integer(0)), (Integer(2),␣
↪Integer(4)), (Integer(4), Integer(5))])
>>> PG = G.power(Integer(3))
>>> PG.edges(sort=True, labels=False)
[(0, 1), (0, 2), (0, 3), (0, 4), (1, 0), (1, 2), (1, 3), (1, 4), (1, 5), (2,␣
↪0), (2, 1), (2, 3), (2, 4), (2, 5), (3, 0), (3, 1), (3, 2), (4, 5)]
```

random_edge(***kwds*)

Return a random edge of `self`.

INPUT:

- `**kwds` – arguments to be passed down to the *edge_iterator()* method

EXAMPLES:

The returned value is an edge of `self`:

```
sage: g = graphs.PetersenGraph()
sage: u,v = g.random_edge(labels=False)
sage: g.has_edge(u,v)
True
```

```
>>> from sage.all import *
>>> g = graphs.PetersenGraph()
>>> u,v = g.random_edge(labels=False)
>>> g.has_edge(u,v)
True
```

As the *edges()* method would, this function returns by default a triple `(u, v, l)` of values, in which `l` is the label of edge `(u, v)`:

```
sage: g.random_edge()  # random
(3, 4, None)
```

```
>>> from sage.all import *
>>> g.random_edge()  # random
(3, 4, None)
```

random_edge_iterator(**args*, ***kwds*)

Return an iterator over random edges of `self`.

The returned iterator enables to amortize the cost of accessing random edges, as can be done with multiple calls to method *random_edge()*.

INPUT:

- `*args` and `**kwds` – arguments to be passed down to the *edge_iterator()* method

EXAMPLES:

The returned value is an iterator over the edges of `self`:

```
sage: g = graphs.PetersenGraph()
sage: it = g.random_edge_iterator()
sage: [g.has_edge(next(it)) for _ in range(5)]
[True, True, True, True, True]
```

```
>>> from sage.all import *
>>> g = graphs.PetersenGraph()
>>> it = g.random_edge_iterator()
>>> [g.has_edge(next(it)) for _ in range(Integer(5))]
[True, True, True, True, True]
```

As the *edges()* method would, this function returns by default a triple `(u, v, l)` of values, in which `l` is the label of edge `(u,v)`:

```
sage: print(next(g.random_edge_iterator())) # random
(0, 5, None)
sage: print(next(g.random_edge_iterator(labels=False))) # random
(5, 7)
```

```
>>> from sage.all import *
>>> print(next(g.random_edge_iterator())) # random
(0, 5, None)
>>> print(next(g.random_edge_iterator(labels=False))) # random
(5, 7)
```

random_subgraph(*p*, *inplace=False*)

Return a random subgraph containing each vertex with probability `p`.

INPUT:

- `p` – the probability of choosing a vertex
- `inplace` – boolean (default: `False`); using `inplace=True` will simply delete the extra vertices and edges from the current graph. This will modify the graph.

EXAMPLES:

```
sage: P = graphs.PetersenGraph()
sage: P.random_subgraph(.25)
Subgraph of (Petersen graph): Graph on ... vert...
```

```
>>> from sage.all import *
>>> P = graphs.PetersenGraph()
>>> P.random_subgraph(RealNumber('.25'))
Subgraph of (Petersen graph): Graph on ... vert...
```

random_vertex(***kwds*)

Return a random vertex of `self`.

INPUT:

- `**kwds` – arguments to be passed down to the *vertex_iterator()* method

EXAMPLES:

The returned value is a vertex of self:

```
sage: g = graphs.PetersenGraph()
sage: v = g.random_vertex()
sage: v in g
True
```

```
>>> from sage.all import *
>>> g = graphs.PetersenGraph()
>>> v = g.random_vertex()
>>> v in g
True
```

random_vertex_iterator(**args*, ***kwds*)

Return an iterator over random vertices of `self`.

The returned iterator enables to amortize the cost of accessing random vertices, as can be done with multiple calls to method *random_vertex()*.

INPUT:

- `*args` and `**kwds` – arguments to be passed down to the *vertex_iterator()* method

EXAMPLES:

The returned value is an iterator over the vertices of `self`:

```
sage: g = graphs.PetersenGraph()
sage: it = g.random_vertex_iterator()
sage: [next(it) in g for _ in range(5)]
[True, True, True, True, True]
```

```
>>> from sage.all import *
>>> g = graphs.PetersenGraph()
>>> it = g.random_vertex_iterator()
>>> [next(it) in g for _ in range(Integer(5))]
[True, True, True, True, True]
```

reduced_homeomorphic_graph(*G*, *allow_multiple_edges=False*, *allow_loops=False*, *return_steps=False*, *immutable=None*)

Return the smallest graph homeomorphic to G.

Two graphs G and H are homeomorphic if there is an isomorphism from some subdivision of G to some subdivision of H. For more details, see the Wikipedia article Homeomorphism_(graph_theory).

By default (i.e., when `allow_multiple_edges == False` and `allow_loops == False`), given a graph G, a vertex u of degree two and its neighbors x and y, with $x \neq y$, this methods replaces the path (x, u, y) with the edge (x, y) unless the graph already has edge (x, y). This process is repeated for each vertex of degree two. The resulting graph H is the smallest graph that is homeomorphic to G.

When `allow_multiple_edges == True` and `allow_loops == False`, this method always replaces the path (x, u, y) with a new edge (x, y). Hence, the resulting graph may have several edges between x and y. This operation is performed only if $x \neq y$.

When `allow_loops == True`, this method also assumes that `allow_multiple_edges == True`. If a vertex u of degree two is connected by two edges to a vertex x, this method replaces the two edges by a loop edge on x.

For digraphs, the method considers the vertices with in and out degree one.

INPUT:

- `G` – a graph or a digraph
- `allow_multiple_edges` – boolean (default: `False`); whether to allow the creation of new multiple edges. This parameter is considered `True` when `allow_loops` is `True`.
- `allow_loops` – boolean (default: `False`); whether to allow the creation of new loops
- `return_steps` – boolean (default: `False`); whether to return the steps of the reduction as a list of triples (x, u, y) indicating that path (x, u, y) has been replaced by edge (x, y). The original graph can be reconstructed by using this list in reverse order.
- `immutable` – boolean (default: `None`); whether to create a mutable/immutable (di)graph. `immutable=None` (default) means that the (di)graph and its reduced (di)graph will behave the same way.

OUTPUT: When `return_steps` is `False`, this method returns the reduced graph. When `return_steps` is `True`, this method returns both the reduced graph and the ordered list of reduction operations. Each reduction operation is a triple (x, u, y) indicating that the path (x, u, y), with u of degree two (or with in and out degree one for digraphs), has been replaced by edge (x, y).

EXAMPLES:

Reduction of a Cycle Graph:

```
sage: G = graphs.CycleGraph(4)
sage: G.reduced_homeomorphic_graph()
Graph on 3 vertices
sage: G.reduced_homeomorphic_graph(allow_multiple_edges=True)
Multi-graph on 2 vertices
sage: G.reduced_homeomorphic_graph(allow_loops=True)
Looped multi-graph on 1 vertex
```

```
>>> from sage.all import *
>>> G = graphs.CycleGraph(Integer(4))
>>> G.reduced_homeomorphic_graph()
Graph on 3 vertices
>>> G.reduced_homeomorphic_graph(allow_multiple_edges=True)
Multi-graph on 2 vertices
>>> G.reduced_homeomorphic_graph(allow_loops=True)
Looped multi-graph on 1 vertex
```

Reduction of a Circuit:

```
sage: G = digraphs.Circuit(4)
sage: G.reduced_homeomorphic_graph()
Digraph on 2 vertices
sage: G.reduced_homeomorphic_graph(allow_multiple_edges=True)
Multi-digraph on 2 vertices
sage: G.reduced_homeomorphic_graph(allow_loops=True)
Looped multi-digraph on 1 vertex
```

```
>>> from sage.all import *
>>> G = digraphs.Circuit(Integer(4))
>>> G.reduced_homeomorphic_graph()
```

(continues on next page)

(continued from previous page)

```
Digraph on 2 vertices
>>> G.reduced_homeomorphic_graph(allow_multiple_edges=True)
Multi-digraph on 2 vertices
>>> G.reduced_homeomorphic_graph(allow_loops=True)
Looped multi-digraph on 1 vertex
```

Check that the construction is reversible:

```
sage: def revert_steps(g, steps):
....:     h = g.copy(immutable=False)
....:     for P in reversed(steps):
....:         h.add_path(P)
....:         h.delete_edge(P[0], P[2])
....:     return h

sage: G = graphs.WindmillGraph(3, 5)
sage: G.order(), G.size()
(11, 15)
sage: H, steps = G.reduced_homeomorphic_graph(return_steps=True)
sage: H.order(), H.size()
(11, 15)
sage: G.is_isomorphic(revert_steps(H, steps))
True
sage: H, steps = G.reduced_homeomorphic_graph(allow_multiple_edges=True,␣
↪return_steps=True)
sage: H.order(), H.size()
(6, 10)
sage: G.is_isomorphic(revert_steps(H, steps))
True
sage: H, steps = G.reduced_homeomorphic_graph(allow_loops=True, return_
↪steps=True)
sage: H.order(), H.size()
(1, 5)
sage: len(H.loop_edges())
5
sage: G.is_isomorphic(revert_steps(H, steps))
True
```

```
>>> from sage.all import *
>>> def revert_steps(g, steps):
...     h = g.copy(immutable=False)
...     for P in reversed(steps):
...         h.add_path(P)
...         h.delete_edge(P[Integer(0)], P[Integer(2)])
...     return h

>>> G = graphs.WindmillGraph(Integer(3), Integer(5))
>>> G.order(), G.size()
(11, 15)
>>> H, steps = G.reduced_homeomorphic_graph(return_steps=True)
>>> H.order(), H.size()
(11, 15)
```

(continues on next page)

(continued from previous page)

```
>>> G.is_isomorphic(revert_steps(H, steps))
True
>>> H, steps = G.reduced_homeomorphic_graph(allow_multiple_edges=True, return_
↪steps=True)
>>> H.order(), H.size()
(6, 10)
>>> G.is_isomorphic(revert_steps(H, steps))
True
>>> H, steps = G.reduced_homeomorphic_graph(allow_loops=True, return_
↪steps=True)
>>> H.order(), H.size()
(1, 5)
>>> len(H.loop_edges())
5
>>> G.is_isomorphic(revert_steps(H, steps))
True
```

Random digraph:

```
sage: G = digraphs.RandomDirectedGNP(20, 0.05, loops=True)
sage: H, steps = G.reduced_homeomorphic_graph(return_steps=True)
sage: G.is_isomorphic(revert_steps(H, steps))
True
sage: H, steps = G.reduced_homeomorphic_graph(allow_multiple_edges=True,␣
↪return_steps=True)
sage: G.is_isomorphic(revert_steps(H, steps))
True
sage: H, steps = G.reduced_homeomorphic_graph(allow_loops=True, return_
↪steps=True)
sage: G.is_isomorphic(revert_steps(H, steps))
True
```

```
>>> from sage.all import *
>>> G = digraphs.RandomDirectedGNP(Integer(20), RealNumber('0.05'),␣
↪loops=True)
>>> H, steps = G.reduced_homeomorphic_graph(return_steps=True)
>>> G.is_isomorphic(revert_steps(H, steps))
True
>>> H, steps = G.reduced_homeomorphic_graph(allow_multiple_edges=True, return_
↪steps=True)
>>> G.is_isomorphic(revert_steps(H, steps))
True
>>> H, steps = G.reduced_homeomorphic_graph(allow_loops=True, return_
↪steps=True)
>>> G.is_isomorphic(revert_steps(H, steps))
True
```

relabel(*perm=None*, *inplace=True*, *return_map=False*, *check_input=True*, *complete_partial_function=True*, *immutable=None*)

Relabel the vertices of `self`.

INPUT:

- `perm` – a function, dictionary, iterable, permutation, or `None` (default: `None`)
- `inplace` – boolean (default: `True`)
- `return_map` – boolean (default: `False`)
- `check_input` – boolean; whether to test input for correctness. *This can potentially be very time-consuming !*
- `complete_partial_function` – boolean; whether to automatically complete the permutation if some elements of the graph are not associated with any new name. In this case, those elements are not relabeled *This can potentially be very time-consuming !*.
- `immutable` – boolean; with `inplace=False`, whether to create a mutable/immutable relabelled copy. `immutable=None` (default) means that the graph and its copy will behave the same way.

If `perm` is a function `f`, then each vertex `v` is relabeled to `f(v)`.

If `perm` is a dictionary `d`, then each vertex `v` (which should be a key of `d`) is relabeled to `d[v]`.

If `perm` is a list (or more generally, any iterable) of length `n`, then the first vertex returned by `G.vertices(sort=True)` is relabeled to `l[0]`, the second to `l[1]`, ...

If `perm` is a permutation, then each vertex `v` is relabeled to `perm(v)`.

If `perm` is `None`, the graph is relabeled to be on the vertices $\{0, 1, ..., n-1\}$. This is *not* any kind of canonical labeling, but it is consistent (relabeling twice will give the same result).

If `inplace` is `True`, the graph is modified in place and `None` is returned. Otherwise a relabeled copy of the graph is returned.

If `return_map` is `True` a dictionary representing the relabelling map is returned (incompatible with `inplace==False`).

EXAMPLES:

```
sage: G = graphs.PathGraph(3)
sage: G.am()                                                                #␣
↪needs sage.modules
[0 1 0]
[1 0 1]
[0 1 0]
```

```
>>> from sage.all import *
>>> G = graphs.PathGraph(Integer(3))
>>> G.am()                                                                  #␣
↪needs sage.modules
[0 1 0]
[1 0 1]
[0 1 0]
```

Relabeling using a dictionary. Note that the dictionary does not define the new label of vertex 0:

```
sage: G.relabel({1:2,2:1}, inplace=False).am()                              #␣
↪needs sage.modules
[0 0 1]
[0 0 1]
[1 1 0]
```

```
>>> from sage.all import *
>>> G.relabel({Integer(1):Integer(2),Integer(2):Integer(1)}, inplace=False).
↪am()                                  # needs sage.modules
[0 0 1]
[0 0 1]
[1 1 0]
```

This is because the method automatically "extends" the relabeling to the missing vertices (whose label will not change). Checking that all vertices have an image can require some time, and this feature can be disabled (at your own risk):

```
sage: G.relabel({1:2,2:1}, inplace=False,                                    #␣
↪needs sage.modules
....:            complete_partial_function=False).am()
Traceback (most recent call last):
...
KeyError: 0
```

```
>>> from sage.all import *
>>> G.relabel({Integer(1):Integer(2),Integer(2):Integer(1)}, inplace=False,  ␣
↪                                  # needs sage.modules
...            complete_partial_function=False).am()
Traceback (most recent call last):
...
KeyError: 0
```

Relabeling using a list:

```
sage: G.relabel([0,2,1], inplace=False).am()                                 #␣
↪needs sage.modules
[0 0 1]
[0 0 1]
[1 1 0]
```

```
>>> from sage.all import *
>>> G.relabel([Integer(0),Integer(2),Integer(1)], inplace=False).am()       ␣
↪                          # needs sage.modules
[0 0 1]
[0 0 1]
[1 1 0]
```

Relabeling using an iterable:

```
sage: G.relabel(iter((0,2,1)), inplace=False).am()                           #␣
↪needs sage.modules
[0 0 1]
[0 0 1]
[1 1 0]
```

```
>>> from sage.all import *
>>> G.relabel(iter((Integer(0),Integer(2),Integer(1))), inplace=False).am()  ␣
↪                          # needs sage.modules
```

(continues on next page)

(continued from previous page)

```
[0 0 1]
[0 0 1]
[1 1 0]
```

Relabeling using a Sage permutation:

```
sage: G = graphs.PathGraph(3)
sage: from sage.groups.perm_gps.permgroup_named import SymmetricGroup       #␣
↪needs sage.groups
sage: S = SymmetricGroup(3)                                                 #␣
↪needs sage.groups
sage: gamma = S('(1,2)')                                                    #␣
↪needs sage.groups
sage: G.relabel(gamma, inplace=False).am()                                  #␣
↪needs sage.groups sage.modules
[0 0 1]
[0 0 1]
[1 1 0]

sage: C5 = graphs.CycleGraph(5)
sage: C5.relabel({u: str(u) for u in C5}, inplace=True)
sage: ar = C5.automorphism_group().random_element()
sage: R = C5.relabel(ar, inplace=False)
sage: perm = C5.is_isomorphic(R, certificate=True)[1]
sage: perm == {u: ar(u) for u in C5}
True
```

```
>>> from sage.all import *
>>> G = graphs.PathGraph(Integer(3))
>>> from sage.groups.perm_gps.permgroup_named import SymmetricGroup       #␣
↪needs sage.groups
>>> S = SymmetricGroup(Integer(3))                                            ␣
↪     # needs sage.groups
>>> gamma = S('(1,2)')                                                    #␣
↪needs sage.groups
>>> G.relabel(gamma, inplace=False).am()                                  #␣
↪needs sage.groups sage.modules
[0 0 1]
[0 0 1]
[1 1 0]

>>> C5 = graphs.CycleGraph(Integer(5))
>>> C5.relabel({u: str(u) for u in C5}, inplace=True)
>>> ar = C5.automorphism_group().random_element()
>>> R = C5.relabel(ar, inplace=False)
>>> perm = C5.is_isomorphic(R, certificate=True)[Integer(1)]
>>> perm == {u: ar(u) for u in C5}
True
```

A way to get a random relabeling:

```
sage: set_random_seed(0)  # Results are reproducible
```

(continues on next page)

(continued from previous page)

```
sage: D = DiGraph({1: [2], 3: [4]})
sage: D.relabel(Permutations(D.vertices(sort=True)).random_element())
sage: D.sources()
[1, 4]
```

```
>>> from sage.all import *
>>> set_random_seed(Integer(0))  # Results are reproducible
>>> D = DiGraph({Integer(1): [Integer(2)], Integer(3): [Integer(4)]})
>>> D.relabel(Permutations(D.vertices(sort=True)).random_element())
>>> D.sources()
[1, 4]
```

Relabeling using an injective function:

```
sage: G.edges(sort=True)
[(0, 1, None), (1, 2, None)]
sage: H = G.relabel(lambda i: i+10, inplace=False)
sage: H.vertices(sort=True)
[10, 11, 12]
sage: H.edges(sort=True)
[(10, 11, None), (11, 12, None)]
```

```
>>> from sage.all import *
>>> G.edges(sort=True)
[(0, 1, None), (1, 2, None)]
>>> H = G.relabel(lambda i: i+Integer(10), inplace=False)
>>> H.vertices(sort=True)
[10, 11, 12]
>>> H.edges(sort=True)
[(10, 11, None), (11, 12, None)]
```

Relabeling using a non injective function has no meaning:

```
sage: G.edges(sort=True)
[(0, 1, None), (1, 2, None)]
sage: G.relabel(lambda i: 0, inplace=False)
Traceback (most recent call last):
...
NotImplementedError: Non injective relabeling
```

```
>>> from sage.all import *
>>> G.edges(sort=True)
[(0, 1, None), (1, 2, None)]
>>> G.relabel(lambda i: Integer(0), inplace=False)
Traceback (most recent call last):
...
NotImplementedError: Non injective relabeling
```

But this test can be disabled, which can lead to … problems:

```
sage: G.edges(sort=True)
[(0, 1, None), (1, 2, None)]
```

(continues on next page)

(continued from previous page)

```
sage: G.relabel(lambda i: 0, check_input = False)
sage: G.edges(sort=True)
[]
```

```
>>> from sage.all import *
>>> G.edges(sort=True)
[(0, 1, None), (1, 2, None)]
>>> G.relabel(lambda i: Integer(0), check_input = False)
>>> G.edges(sort=True)
[]
```

Recovering the relabeling with `return_map`:

```
sage: G = graphs.CubeGraph(3)
sage: G.relabel(range(8), return_map=True)
{'000': 0,
 '001': 1,
 '010': 2,
 '011': 3,
 '100': 4,
 '101': 5,
 '110': 6,
 '111': 7}
```

```
>>> from sage.all import *
>>> G = graphs.CubeGraph(Integer(3))
>>> G.relabel(range(Integer(8)), return_map=True)
{'000': 0,
 '001': 1,
 '010': 2,
 '011': 3,
 '100': 4,
 '101': 5,
 '110': 6,
 '111': 7}
```

When no permutation is given, the relabeling is done to integers from 0 to N-1 but in an arbitrary order:

```
sage: G = graphs.CubeGraph(3)
sage: G.vertices(sort=True)
['000', '001', '010', '011', '100', '101', '110', '111']
sage: G.relabel()
sage: G.vertices(sort=True)
[0, 1, 2, 3, 4, 5, 6, 7]
```

```
>>> from sage.all import *
>>> G = graphs.CubeGraph(Integer(3))
>>> G.vertices(sort=True)
['000', '001', '010', '011', '100', '101', '110', '111']
>>> G.relabel()
>>> G.vertices(sort=True)
[0, 1, 2, 3, 4, 5, 6, 7]
```

In the above case, the mapping is arbitrary but consistent:

```
sage: map1 = G.relabel(inplace=False, return_map=True)
sage: map2 = G.relabel(inplace=False, return_map=True)
sage: map1 == map2
True
```

```
>>> from sage.all import *
>>> map1 = G.relabel(inplace=False, return_map=True)
>>> map2 = G.relabel(inplace=False, return_map=True)
>>> map1 == map2
True
```

```
sage: G = graphs.PathGraph(3)
sage: G.relabel(lambda i: i+10, return_map=True)
{0: 10, 1: 11, 2: 12}
```

```
>>> from sage.all import *
>>> G = graphs.PathGraph(Integer(3))
>>> G.relabel(lambda i: i+Integer(10), return_map=True)
{0: 10, 1: 11, 2: 12}
```

remove_loops(*vertices=None*)

Remove loops on vertices in `vertices`.

If `vertices` is `None`, removes all loops.

EXAMPLES:

```
sage: G = Graph(4, loops=True)
sage: G.add_edges([(0, 0), (1, 1), (2, 2), (3, 3), (2, 3)])
sage: G.edges(sort=True, labels=False)
[(0, 0), (1, 1), (2, 2), (2, 3), (3, 3)]
sage: G.remove_loops()
sage: G.edges(sort=True, labels=False)
[(2, 3)]
sage: G.allows_loops()
True
sage: G.has_loops()
False

sage: D = DiGraph(4, loops=True)
sage: D.add_edges([(0, 0), (1, 1), (2, 2), (3, 3), (2, 3)])
sage: D.edges(sort=True, labels=False)
[(0, 0), (1, 1), (2, 2), (2, 3), (3, 3)]
sage: D.remove_loops()
sage: D.edges(sort=True, labels=False)
[(2, 3)]
sage: D.allows_loops()
True
sage: D.has_loops()
False
```

```
>>> from sage.all import *
>>> G = Graph(Integer(4), loops=True)
>>> G.add_edges([(Integer(0), Integer(0)), (Integer(1), Integer(1)),␣
↪(Integer(2), Integer(2)), (Integer(3), Integer(3)), (Integer(2),␣
↪Integer(3))])
>>> G.edges(sort=True, labels=False)
[(0, 0), (1, 1), (2, 2), (2, 3), (3, 3)]
>>> G.remove_loops()
>>> G.edges(sort=True, labels=False)
[(2, 3)]
>>> G.allows_loops()
True
>>> G.has_loops()
False

>>> D = DiGraph(Integer(4), loops=True)
>>> D.add_edges([(Integer(0), Integer(0)), (Integer(1), Integer(1)),␣
↪(Integer(2), Integer(2)), (Integer(3), Integer(3)), (Integer(2),␣
↪Integer(3))])
>>> D.edges(sort=True, labels=False)
[(0, 0), (1, 1), (2, 2), (2, 3), (3, 3)]
>>> D.remove_loops()
>>> D.edges(sort=True, labels=False)
[(2, 3)]
>>> D.allows_loops()
True
>>> D.has_loops()
False
```

remove_multiple_edges()

Remove all multiple edges, retaining one edge for each.

See also

See also *allow_multiple_edges()*

EXAMPLES:

```
sage: G = Graph(multiedges=True, sparse=True)
sage: G.add_edges([(0,1), (0,1), (0,1), (0,1), (1,2)])
sage: G.edges(sort=True, labels=False)
[(0, 1), (0, 1), (0, 1), (0, 1), (1, 2)]
```

```
>>> from sage.all import *
>>> G = Graph(multiedges=True, sparse=True)
>>> G.add_edges([(Integer(0),Integer(1)), (Integer(0),Integer(1)),␣
↪(Integer(0),Integer(1)), (Integer(0),Integer(1)), (Integer(1),Integer(2))])
>>> G.edges(sort=True, labels=False)
[(0, 1), (0, 1), (0, 1), (0, 1), (1, 2)]
```

```
sage: G.remove_multiple_edges()
```

(continues on next page)

(continued from previous page)

```
sage: G.edges(sort=True, labels=False)
[(0, 1), (1, 2)]
```

```
>>> from sage.all import *
>>> G.remove_multiple_edges()
>>> G.edges(sort=True, labels=False)
[(0, 1), (1, 2)]
```

```
sage: D = DiGraph(multiedges=True, sparse=True)
sage: D.add_edges([(0, 1, 1), (0, 1, 2), (0, 1, 3), (0, 1, 4), (1, 2, None)])
sage: D.edges(sort=True, labels=False)
[(0, 1), (0, 1), (0, 1), (0, 1), (1, 2)]
sage: D.remove_multiple_edges()
sage: D.edges(sort=True, labels=False)
[(0, 1), (1, 2)]
```

```
>>> from sage.all import *
>>> D = DiGraph(multiedges=True, sparse=True)
>>> D.add_edges([(Integer(0), Integer(1), Integer(1)), (Integer(0),␣
↪Integer(1), Integer(2)), (Integer(0), Integer(1), Integer(3)), (Integer(0),␣
↪Integer(1), Integer(4)), (Integer(1), Integer(2), None)])
>>> D.edges(sort=True, labels=False)
[(0, 1), (0, 1), (0, 1), (0, 1), (1, 2)]
>>> D.remove_multiple_edges()
>>> D.edges(sort=True, labels=False)
[(0, 1), (1, 2)]
```

rooted_product(*G*, *H*, *root=None*, *immutable=None*)

Return the rooted product of G and H.

The rooted product of two graphs G and H is the graph R defined as follows: take a copy of G and $|V(G)|$ copies of H, and for every vertex g_i of G, identify g_i with the root of the i-th copy of H. Mode formally, let $V(G) = \{g_1, g_2, \ldots, g_n\}$, $V(H) = \{h_1, h_2, \ldots, h_m\}$, and let h_1 be the root vertex of H. The vertex set $V(R)$ is equal to the cartesian product of the sets of vertices $V(G)$ and $V(H)$, that is $V(R) = \{(g_i, h_j) : g_i \in V(G), h_j \in V(H)\}$. The edge set $E(R)$ is the union of the edges of a copy of G, that is $\{((g_i, h_1), (g_j, h_1)) : (g_i, g_j) \in E(G)\}$, and the edges of the copies of H for every $g_i \in V(G)$, that is $\{((g_i, h_j), (g_i, h_k)) : (h_j, h_k) \in V(H)\}$.

See Wikipedia article Rooted_product_of_graphs for more details.

See also

- *cartesian_product()* – return the cartesian product of two graphs
- *graph_products* – a module on graph products

INPUT:

- `G, H` – two (di)graphs
- `immutable` – boolean (default: `None`); whether to create a mutable/immutable (di)graph. When `immutable=None` (default) the rooted product will be mutable if one of `G` or `H` is mutable and immutable otherwise.

EXAMPLES:

The rooted product of two trees is a tree:

```
sage: T1 = graphs.RandomTree(7)
sage: T2 = graphs.RandomTree(8)
sage: T = T1.rooted_product(T2)
sage: T.is_tree()
True
```

```
>>> from sage.all import *
>>> T1 = graphs.RandomTree(Integer(7))
>>> T2 = graphs.RandomTree(Integer(8))
>>> T = T1.rooted_product(T2)
>>> T.is_tree()
True
```

The rooted product of G and H depends on the selected root in H:

```
sage: G = graphs.CycleGraph(4)
sage: H = graphs.PathGraph(3)
sage: R1 = G.rooted_product(H, root=0)
sage: R2 = G.rooted_product(H, root=1)
sage: R1.is_isomorphic(R2)
False
sage: sorted(R1.degree())
[1, 1, 1, 1, 2, 2, 2, 2, 3, 3, 3, 3]
sage: sorted(R2.degree())
[1, 1, 1, 1, 1, 1, 1, 1, 4, 4, 4, 4]
```

```
>>> from sage.all import *
>>> G = graphs.CycleGraph(Integer(4))
>>> H = graphs.PathGraph(Integer(3))
>>> R1 = G.rooted_product(H, root=Integer(0))
>>> R2 = G.rooted_product(H, root=Integer(1))
>>> R1.is_isomorphic(R2)
False
>>> sorted(R1.degree())
[1, 1, 1, 1, 2, 2, 2, 2, 3, 3, 3, 3]
>>> sorted(R2.degree())
[1, 1, 1, 1, 1, 1, 1, 1, 4, 4, 4, 4]
```

The domination number of the rooted product of any graph G and a path of order 2 is the order of G:

```
sage: G = graphs.RandomGNP(20, .3)
sage: P = graphs.PathGraph(2)
sage: R = G.rooted_product(P)
sage: len(R.dominating_set()) == G.order()
↪  # needs sage.numerical.mip
True
sage: G = digraphs.RandomDirectedGNP(20, .3)
sage: P = digraphs.Path(2)
sage: R = G.rooted_product(P)
sage: len(R.dominating_set()) == G.order()
```

(continues on next page)

(continued from previous page)

```
↪  # needs sage.numerical.mip
True
```

```
>>> from sage.all import *
>>> G = graphs.RandomGNP(Integer(20), RealNumber('.3'))
>>> P = graphs.PathGraph(Integer(2))
>>> R = G.rooted_product(P)
>>> len(R.dominating_set()) == G.order()
↪# needs sage.numerical.mip
True
>>> G = digraphs.RandomDirectedGNP(Integer(20), RealNumber('.3'))
>>> P = digraphs.Path(Integer(2))
>>> R = G.rooted_product(P)
>>> len(R.dominating_set()) == G.order()
↪# needs sage.numerical.mip
True
```

The rooted product of two graphs is a subgraph of the cartesian product of the same two graphs:

```
sage: G = graphs.RandomGNP(6, .4)
sage: H = graphs.RandomGNP(7, .4)
sage: R = G.rooted_product(H)
sage: C = G.cartesian_product(H)
sage: R.is_subgraph(C, induced=False)
True
```

```
>>> from sage.all import *
>>> G = graphs.RandomGNP(Integer(6), RealNumber('.4'))
>>> H = graphs.RandomGNP(Integer(7), RealNumber('.4'))
>>> R = G.rooted_product(H)
>>> C = G.cartesian_product(H)
>>> R.is_subgraph(C, induced=False)
True
```

Corner cases:

```
sage: Graph().rooted_product(Graph())
Rooted product of Graph on 0 vertices and Graph on 0 vertices: Graph on 0␣
↪vertices
sage: Graph(1).rooted_product(Graph())
Rooted product of Graph on 1 vertex and Graph on 0 vertices: Graph on 0␣
↪vertices
sage: Graph().rooted_product(Graph(1))
Rooted product of Graph on 0 vertices and Graph on 1 vertex: Graph on 0␣
↪vertices
sage: Graph(1).rooted_product(Graph(1))
Rooted product of Graph on 1 vertex and Graph on 1 vertex: Graph on 1 vertex
```

```
>>> from sage.all import *
>>> Graph().rooted_product(Graph())
Rooted product of Graph on 0 vertices and Graph on 0 vertices: Graph on 0␣
↪vertices
```

(continues on next page)

(continued from previous page)

```
>>> Graph(Integer(1)).rooted_product(Graph())
Rooted product of Graph on 1 vertex and Graph on 0 vertices: Graph on 0␣
↪vertices
>>> Graph().rooted_product(Graph(Integer(1)))
Rooted product of Graph on 0 vertices and Graph on 1 vertex: Graph on 0␣
↪vertices
>>> Graph(Integer(1)).rooted_product(Graph(Integer(1)))
Rooted product of Graph on 1 vertex and Graph on 1 vertex: Graph on 1 vertex
```

set_edge_label(*u*, *v*, *l*)

Set the edge label of a given edge.

> **Note**
>
> There can be only one edge from u to v for this to make sense. Otherwise, an error is raised.

INPUT:

- `u`, `v` – the vertices (and direction if digraph) of the edge
- `l` – the new label

EXAMPLES:

```
sage: d = {1: [18, 2], 2: [5, 3], 3: [4, 6], 4: [7, 2], 5: [4],
....:      6: [13, 12], 7: [18, 8, 10], 8: [6, 9, 10], 9: [6],
....:      10: [11, 13], 11: [12], 12: [13], 13: [17, 14],
....:      14: [16, 15], 15: [2], 16: [13], 17: [15, 13], 18: [13]}
sage: SD = DiGraph(d, sparse=True)
sage: SD.set_edge_label(1, 18, 'discrete')
sage: SD.set_edge_label(4, 7, 'discrete')
sage: SD.set_edge_label(2, 5, 'h = 0')
sage: SD.set_edge_label(7, 18, 'h = 0')
sage: SD.set_edge_label(7, 10, 'aut')
sage: SD.set_edge_label(8, 10, 'aut')
sage: SD.set_edge_label(8, 9, 'label')
sage: SD.set_edge_label(8, 6, 'no label')
sage: SD.set_edge_label(13, 17, 'k > h')
sage: SD.set_edge_label(13, 14, 'k = h')
sage: SD.set_edge_label(17, 15, 'v_k finite')
sage: SD.set_edge_label(14, 15, 'v_k m.c.r.')
sage: posn = {1: [3, -3], 2: [0, 2], 3: [0, 13], 4: [3, 9],
....:         5: [3, 3], 6: [16, 13], 7: [6, 1], 8: [6, 6],
....:         9: [6, 11], 10: [9, 1], 11: [10, 6], 12: [13, 6],
....:         13: [16, 2], 14: [10, -6], 15: [0, -10], 16: [14, -6],
....:         17: [16, -10], 18: [6, -4]}
sage: SD.plot(pos=posn, vertex_size=400, vertex_colors={'#FFFFFF
↪':list(range(1,19))}, edge_labels=True).show()  # long time, needs sage.plot
```

```
>>> from sage.all import *
>>> d = {Integer(1): [Integer(18), Integer(2)], Integer(2): [Integer(5),␣
↪Integer(3)], Integer(3): [Integer(4), Integer(6)], Integer(4): [Integer(7),␣
```

(continues on next page)

(continued from previous page)

```
↪Integer(2)], Integer(5): [Integer(4)],
...       Integer(6): [Integer(13), Integer(12)], Integer(7): [Integer(18),␣
↪Integer(8), Integer(10)], Integer(8): [Integer(6), Integer(9), Integer(10)],
↪ Integer(9): [Integer(6)],
...       Integer(10): [Integer(11), Integer(13)], Integer(11): [Integer(12)],␣
↪Integer(12): [Integer(13)], Integer(13): [Integer(17), Integer(14)],
...       Integer(14): [Integer(16), Integer(15)], Integer(15): [Integer(2)],␣
↪Integer(16): [Integer(13)], Integer(17): [Integer(15), Integer(13)],␣
↪Integer(18): [Integer(13)]}
>>> SD = DiGraph(d, sparse=True)
>>> SD.set_edge_label(Integer(1), Integer(18), 'discrete')
>>> SD.set_edge_label(Integer(4), Integer(7), 'discrete')
>>> SD.set_edge_label(Integer(2), Integer(5), 'h = 0')
>>> SD.set_edge_label(Integer(7), Integer(18), 'h = 0')
>>> SD.set_edge_label(Integer(7), Integer(10), 'aut')
>>> SD.set_edge_label(Integer(8), Integer(10), 'aut')
>>> SD.set_edge_label(Integer(8), Integer(9), 'label')
>>> SD.set_edge_label(Integer(8), Integer(6), 'no label')
>>> SD.set_edge_label(Integer(13), Integer(17), 'k > h')
>>> SD.set_edge_label(Integer(13), Integer(14), 'k = h')
>>> SD.set_edge_label(Integer(17), Integer(15), 'v_k finite')
>>> SD.set_edge_label(Integer(14), Integer(15), 'v_k m.c.r.')
>>> posn = {Integer(1): [Integer(3), -Integer(3)], Integer(2): [Integer(0),␣
↪Integer(2)], Integer(3): [Integer(0), Integer(13)], Integer(4): [Integer(3),
↪ Integer(9)],
...         Integer(5): [Integer(3), Integer(3)], Integer(6): [Integer(16),␣
↪Integer(13)], Integer(7): [Integer(6), Integer(1)], Integer(8): [Integer(6),
↪ Integer(6)],
...         Integer(9): [Integer(6), Integer(11)], Integer(10): [Integer(9),␣
↪Integer(1)], Integer(11): [Integer(10), Integer(6)], Integer(12):␣
↪[Integer(13), Integer(6)],
...         Integer(13): [Integer(16), Integer(2)], Integer(14): [Integer(10),
↪ -Integer(6)], Integer(15): [Integer(0), -Integer(10)], Integer(16):␣
↪[Integer(14), -Integer(6)],
...         Integer(17): [Integer(16), -Integer(10)], Integer(18):␣
↪[Integer(6), -Integer(4)]}
>>> SD.plot(pos=posn, vertex_size=Integer(400), vertex_colors={'#FFFFFF
↪':list(range(Integer(1),Integer(19)))}, edge_labels=True).show()  # long␣
↪time, needs sage.plot
```

```
sage: G = graphs.HeawoodGraph()
sage: for u,v,l in G.edges(sort=False):
....:  G.set_edge_label(u, v, '(' + str(u) + ',' + str(v) + ')')
sage: G.edges(sort=True)
    [(0, 1, '(0,1)'),
     (0, 5, '(0,5)'),
     (0, 13, '(0,13)'),
     ...
     (11, 12, '(11,12)'),
     (12, 13, '(12,13)')]
```

```
>>> from sage.all import *
>>> G = graphs.HeawoodGraph()
>>> for u,v,l in G.edges(sort=False):
...  G.set_edge_label(u, v, '(' + str(u) + ',' + str(v) + ')')
>>> G.edges(sort=True)
    [(0, 1, '(0,1)'),
     (0, 5, '(0,5)'),
     (0, 13, '(0,13)'),
     ...
     (11, 12, '(11,12)'),
     (12, 13, '(12,13)')]
```

```
sage: g = Graph({0: [0, 1, 1, 2]}, loops=True, multiedges=True, sparse=True)
sage: g.set_edge_label(0, 0, 'test')
sage: g.edges(sort=True)
[(0, 0, 'test'), (0, 1, None), (0, 1, None), (0, 2, None)]
sage: g.add_edge(0, 0, 'test2')
sage: g.set_edge_label(0,0,'test3')
Traceback (most recent call last):
...
RuntimeError: cannot set edge label, since there are multiple edges from 0 to␣
↪0
```

```
>>> from sage.all import *
>>> g = Graph({Integer(0): [Integer(0), Integer(1), Integer(1), Integer(2)]},␣
↪loops=True, multiedges=True, sparse=True)
>>> g.set_edge_label(Integer(0), Integer(0), 'test')
>>> g.edges(sort=True)
[(0, 0, 'test'), (0, 1, None), (0, 1, None), (0, 2, None)]
>>> g.add_edge(Integer(0), Integer(0), 'test2')
>>> g.set_edge_label(Integer(0),Integer(0),'test3')
Traceback (most recent call last):
...
RuntimeError: cannot set edge label, since there are multiple edges from 0 to␣
↪0
```

```
sage: dg = DiGraph({0: [1], 1: [0]}, sparse=True)
sage: dg.set_edge_label(0, 1, 5)
sage: dg.set_edge_label(1, 0, 9)
sage: dg.outgoing_edges(1)
[(1, 0, 9)]
sage: dg.incoming_edges(1)
[(0, 1, 5)]
sage: dg.outgoing_edges(0)
[(0, 1, 5)]
sage: dg.incoming_edges(0)
[(1, 0, 9)]
```

```
>>> from sage.all import *
>>> dg = DiGraph({Integer(0): [Integer(1)], Integer(1): [Integer(0)]},␣
↪sparse=True)
>>> dg.set_edge_label(Integer(0), Integer(1), Integer(5))
```

(continues on next page)

(continued from previous page)

```
>>> dg.set_edge_label(Integer(1), Integer(0), Integer(9))
>>> dg.outgoing_edges(Integer(1))
[(1, 0, 9)]
>>> dg.incoming_edges(Integer(1))
[(0, 1, 5)]
>>> dg.outgoing_edges(Integer(0))
[(0, 1, 5)]
>>> dg.incoming_edges(Integer(0))
[(1, 0, 9)]
```

```
sage: G = Graph({0: {1: 1}}, sparse=True)
sage: G.n_edges()
1
sage: G.set_edge_label(0, 1, 1)
sage: G.n_edges()
1
```

```
>>> from sage.all import *
>>> G = Graph({Integer(0): {Integer(1): Integer(1)}}, sparse=True)
>>> G.n_edges()
1
>>> G.set_edge_label(Integer(0), Integer(1), Integer(1))
>>> G.n_edges()
1
```

set_embedding(*embedding*)

Set a combinatorial embedding dictionary to `_embedding` attribute.

The dictionary `embedding` represents a combinatorial embedding of `self` and is organized as a mapping from vertex labels to list of vertex neighbors in clockwise order.

Parameter `embedding` is error-checked for validity.

Warning

Combinatorial embeddings are defined for simple graphs only (i.e., without loops or multiple edges). Therefore, an error is raised when this method is used for a graph with loops or multiple edges.

INPUT:

- `embedding` – dictionary representing a combinatorial embedding of `self`. Format: "{v1: [v2,v3], v2: [v1], v3: [v1]}" (clockwise ordering of neighbors at each vertex).

EXAMPLES:

```
sage: G = graphs.PetersenGraph()
sage: G.set_embedding({0: [1, 5, 4], 1: [0, 2, 6], 2: [1, 3, 7],
....:                  3: [8, 2, 4], 4: [0, 9, 3], 5: [0, 8, 7],
....:                  6: [8, 1, 9], 7: [9, 2, 5], 8: [3, 5, 6],
....:                  9: [4, 6, 7]})
sage: G.set_embedding({'s': [1, 5, 4], 1: [0, 2, 6], 2: [1, 3, 7],
....:                  3: [8, 2, 4], 4: [0, 9, 3], 5: [0, 8, 7],
```

(continues on next page)

(continued from previous page)

```
....:                  6: [8, 1, 9], 7: [9, 2, 5], 8: [3, 5, 6],
....:                  9: [4, 6, 7]})
Traceback (most recent call last):
...
ValueError: vertices in ['s'] from the embedding do not belong to the graph
```

```
>>> from sage.all import *
>>> G = graphs.PetersenGraph()
>>> G.set_embedding({Integer(0): [Integer(1), Integer(5), Integer(4)],␣
↪Integer(1): [Integer(0), Integer(2), Integer(6)], Integer(2): [Integer(1),␣
↪Integer(3), Integer(7)],
...                   Integer(3): [Integer(8), Integer(2), Integer(4)],␣
↪Integer(4): [Integer(0), Integer(9), Integer(3)], Integer(5): [Integer(0),␣
↪Integer(8), Integer(7)],
...                   Integer(6): [Integer(8), Integer(1), Integer(9)],␣
↪Integer(7): [Integer(9), Integer(2), Integer(5)], Integer(8): [Integer(3),␣
↪Integer(5), Integer(6)],
...                   Integer(9): [Integer(4), Integer(6), Integer(7)]})
>>> G.set_embedding({'s': [Integer(1), Integer(5), Integer(4)], Integer(1):␣
↪[Integer(0), Integer(2), Integer(6)], Integer(2): [Integer(1), Integer(3),␣
↪Integer(7)],
...                   Integer(3): [Integer(8), Integer(2), Integer(4)],␣
↪Integer(4): [Integer(0), Integer(9), Integer(3)], Integer(5): [Integer(0),␣
↪Integer(8), Integer(7)],
...                   Integer(6): [Integer(8), Integer(1), Integer(9)],␣
↪Integer(7): [Integer(9), Integer(2), Integer(5)], Integer(8): [Integer(3),␣
↪Integer(5), Integer(6)],
...                   Integer(9): [Integer(4), Integer(6), Integer(7)]})
Traceback (most recent call last):
...
ValueError: vertices in ['s'] from the embedding do not belong to the graph
```

set_latex_options(***kwds*)

Set multiple options for rendering a graph with LaTeX.

INPUT:

- `kwds` – any number of option/value pairs to set many graph latex options at once (a variable number, in any order). Existing values are overwritten, new values are added. Existing values can be cleared by setting the value to `None`. Possible options are documented at *sage.graphs.graph_latex.GraphLatex.set_option()*.

This method is a convenience for setting the options of a graph directly on an instance of the graph. For a full explanation of how to use LaTeX to render graphs, see the introduction to the *graph_latex* module.

EXAMPLES:

```
sage: g = graphs.PetersenGraph()
sage: g.set_latex_options(tkz_style='Welsh')                                #␣
↪needs sage.plot
sage: opts = g.latex_options()                                              #␣
↪needs sage.plot
sage: opts.get_option('tkz_style')                                          #␣
```

(continues on next page)

(continued from previous page)

```
↪needs sage.plot
'Welsh'
```

```
>>> from sage.all import *
>>> g = graphs.PetersenGraph()
>>> g.set_latex_options(tkz_style='Welsh')                                    #␣
↪needs sage.plot
>>> opts = g.latex_options()                                                  #␣
↪needs sage.plot
>>> opts.get_option('tkz_style')                                              #␣
↪needs sage.plot
'Welsh'
```

set_pos(*pos*, *dim=2*)

Set the position dictionary.

The position dictionary specifies the coordinates of each vertex.

INPUT:

- `pos` – a position dictionary for the vertices of the (di)graph
- `dim` – integer (default: 2); the number of coordinates per vertex

EXAMPLES:

```
sage: G = graphs.PetersenGraph()
sage: G.get_pos()
{0: (..., ...),
 ...
 9: (..., ...)}
```

```
>>> from sage.all import *
>>> G = graphs.PetersenGraph()
>>> G.get_pos()
{0: (..., ...),
 ...
 9: (..., ...)}
```

The method *get_pos()* check the position dictionary so that invalid positioning are ignored:

```
sage: G.set_pos(dict(enumerate('abcdefghi')))
sage: P = G.plot()  # positions are ignored                                   #␣
↪needs sage.plot
sage: G.get_pos() is None                                                     #␣
↪needs sage.plot
True
```

```
>>> from sage.all import *
>>> G.set_pos(dict(enumerate('abcdefghi')))
>>> P = G.plot()  # positions are ignored                                     #␣
↪needs sage.plot
>>> G.get_pos() is None                                                       #␣
```

(continues on next page)

(continued from previous page)

```
↪needs sage.plot
True
```

set_vertex(*vertex*, *object*)

Associate an arbitrary object with a vertex.

INPUT:

- `vertex` – which vertex
- `object` – object to associate to vertex

EXAMPLES:

```
sage: T = graphs.TetrahedralGraph()
sage: T.vertices(sort=True)
[0, 1, 2, 3]
sage: T.set_vertex(1, graphs.FlowerSnark())
sage: T.get_vertex(1)
Flower Snark: Graph on 20 vertices
sage: T.set_vertex(4, 'foo')
Traceback (most recent call last):
...
ValueError: vertex (4) not in the graph
```

```
>>> from sage.all import *
>>> T = graphs.TetrahedralGraph()
>>> T.vertices(sort=True)
[0, 1, 2, 3]
>>> T.set_vertex(Integer(1), graphs.FlowerSnark())
>>> T.get_vertex(Integer(1))
Flower Snark: Graph on 20 vertices
>>> T.set_vertex(Integer(4), 'foo')
Traceback (most recent call last):
...
ValueError: vertex (4) not in the graph
```

set_vertices(*vertex_dict*)

Associate arbitrary objects with each vertex, via an association dictionary.

INPUT:

- `vertex_dict` – the association dictionary

EXAMPLES:

```
sage: d = {0: graphs.DodecahedralGraph(), 1: graphs.FlowerSnark(), 2: graphs.
↪MoebiusKantorGraph(), 3: graphs.PetersenGraph()}
sage: d[2]
Moebius-Kantor Graph: Graph on 16 vertices
sage: T = graphs.TetrahedralGraph()
sage: T.vertices(sort=True)
[0, 1, 2, 3]
sage: T.set_vertices(d)
sage: T.get_vertex(1)
Flower Snark: Graph on 20 vertices
```

```
>>> from sage.all import *
>>> d = {Integer(0): graphs.DodecahedralGraph(), Integer(1): graphs.
↪FlowerSnark(), Integer(2): graphs.MoebiusKantorGraph(), Integer(3): graphs.
↪PetersenGraph()}
>>> d[Integer(2)]
Moebius-Kantor Graph: Graph on 16 vertices
>>> T = graphs.TetrahedralGraph()
>>> T.vertices(sort=True)
[0, 1, 2, 3]
>>> T.set_vertices(d)
>>> T.get_vertex(Integer(1))
Flower Snark: Graph on 20 vertices
```

shortest_path(*u*, *v*, *by_weight=False*, *algorithm=None*, *weight_function=None*, *check_weight=True*)

Return a list of vertices representing some shortest path from `u` to `v`.

If there is no path from u to v, the returned list is empty.

For more information and more examples, see *shortest_paths()* (the inputs are very similar).

INPUT:

- `u`, `v` – the start and the end vertices of the paths
- `by_weight` – boolean (default: `False`); if `True`, the edges in the graph are weighted, otherwise all edges have weight 1
- `algorithm` – string (default: `None`); one of the following algorithms:
  - `'BFS'`: performs a BFS from `u`. Does not work with edge weights.
  - `'BFS_Bid'`: performs a BFS from `u` and from `v`. Does not work with edge weights.
  - `'Dijkstra_NetworkX'`: the Dijkstra algorithm, implemented in NetworkX. Works only with positive weights.
  - `'Dijkstra_Bid_NetworkX'`: performs a Dijkstra visit from `u` and from `v` (NetworkX implementation). Works only with positive weights.
  - `'Dijkstra_Bid'`: a Cython implementation that performs a Dijkstra visit from `u` and from `v`. Works only with positive weights.
  - `'Bellman-Ford_Boost'`: the Bellman-Ford algorithm, implemented in Boost. Works also with negative weights, if there is no negative cycle.
  - `None` (default): Sage chooses the best algorithm: `'BFS_Bid'` if `by_weight` is `False`, `'Dijkstra_Bid'` otherwise.

  Note

  If there are negative weights and algorithm is `None`, the result is not reliable. This occurs because, for performance reasons, we cannot check whether there are edges with negative weights before running the algorithm. If there are, the user should explicitly input `algorithm='Bellman-Ford_Boost'`.

- `weight_function` – function (default: `None`); a function that takes as input an edge `(u, v, l)` and outputs its weight. If not `None`, `by_weight` is automatically set to `True`. If `None` and `by_weight` is `True`, we use the edge label `l`, if `l` is not `None`, else `1` as a weight.

- `check_weight` – boolean (default: `True`); if `True`, we check that the weight_function outputs a number for each edge

EXAMPLES:

```
sage: D = graphs.DodecahedralGraph()
sage: D.shortest_path(4, 9)
[4, 17, 16, 12, 13, 9]
sage: D.shortest_path(4, 9, algorithm='BFS')
[4, 3, 2, 1, 8, 9]
sage: D.shortest_path(4, 8, algorithm='Dijkstra_NetworkX')                    #␣
↪needs networkx
[4, 3, 2, 1, 8]
sage: D.shortest_path(4, 8, algorithm='Dijkstra_Bid_NetworkX')                #␣
↪needs networkx
[4, 3, 2, 1, 8]
sage: D.shortest_path(4, 9, algorithm='Dijkstra_Bid')
[4, 3, 2, 1, 8, 9]
sage: D.shortest_path(5, 5)
[5]
sage: D.delete_edges(D.edges_incident(13))
sage: D.shortest_path(13, 4)
[]
sage: G = Graph({0: {1: 1}, 1: {2: 1}, 2: {3: 1}, 3: {4: 2}, 4: {0: 2}},
....:           sparse=True)
sage: G.plot(edge_labels=True).show()       # long time                      #␣
↪needs sage.plot
sage: G.shortest_path(0, 3)
[0, 4, 3]
sage: G.shortest_path(0, 3, by_weight=True)
[0, 1, 2, 3]
sage: G.shortest_path(0, 3, by_weight=True,                                  #␣
↪needs networkx
....:                 algorithm='Dijkstra_NetworkX')
[0, 1, 2, 3]
sage: G.shortest_path(0, 3, by_weight=True,                                  #␣
↪needs networkx
....:                 algorithm='Dijkstra_Bid_NetworkX')
[0, 1, 2, 3]
```

```
>>> from sage.all import *
>>> D = graphs.DodecahedralGraph()
>>> D.shortest_path(Integer(4), Integer(9))
[4, 17, 16, 12, 13, 9]
>>> D.shortest_path(Integer(4), Integer(9), algorithm='BFS')
[4, 3, 2, 1, 8, 9]
>>> D.shortest_path(Integer(4), Integer(8), algorithm='Dijkstra_NetworkX')    ␣
↪              # needs networkx
[4, 3, 2, 1, 8]
>>> D.shortest_path(Integer(4), Integer(8), algorithm='Dijkstra_Bid_NetworkX
↪')              # needs networkx
[4, 3, 2, 1, 8]
>>> D.shortest_path(Integer(4), Integer(9), algorithm='Dijkstra_Bid')
[4, 3, 2, 1, 8, 9]
```

(continues on next page)

(continued from previous page)

```
>>> D.shortest_path(Integer(5), Integer(5))
[5]
>>> D.delete_edges(D.edges_incident(Integer(13)))
>>> D.shortest_path(Integer(13), Integer(4))
[]
>>> G = Graph({Integer(0): {Integer(1): Integer(1)}, Integer(1): {Integer(2):␣
↪Integer(1)}, Integer(2): {Integer(3): Integer(1)}, Integer(3): {Integer(4):␣
↪Integer(2)}, Integer(4): {Integer(0): Integer(2)}},
...           sparse=True)
>>> G.plot(edge_labels=True).show()        # long time                      #␣
↪needs sage.plot
>>> G.shortest_path(Integer(0), Integer(3))
[0, 4, 3]
>>> G.shortest_path(Integer(0), Integer(3), by_weight=True)
[0, 1, 2, 3]
>>> G.shortest_path(Integer(0), Integer(3), by_weight=True,                  ␣
↪              # needs networkx
...                 algorithm='Dijkstra_NetworkX')
[0, 1, 2, 3]
>>> G.shortest_path(Integer(0), Integer(3), by_weight=True,                  ␣
↪              # needs networkx
...                 algorithm='Dijkstra_Bid_NetworkX')
[0, 1, 2, 3]
```

Todo

Add options to return a path as a list of edges with or without edge labels. This can be useful in (di)graphs with multiple edges.

shortest_path_all_pairs(*by_weight=False*, *algorithm=None*, *weight_function=None*, *check_weight=True*)

Return a shortest path between each pair of vertices.

INPUT:

- `by_weight` – boolean (default: `False`); if `True`, the edges in the graph are weighted, otherwise all edges have weight 1
- `algorithm` – string (default: `None`); one of the following algorithms:
  - `'BFS'`: the computation is done through a BFS centered on each vertex successively. Works only if `by_weight==False`.
  - `'Floyd-Warshall-Cython'`: the Cython implementation of the Floyd-Warshall algorithm. Works only if `by_weight==False`.
  - `'Floyd-Warshall-Python'`: the Python implementation of the Floyd-Warshall algorithm. Works also with weighted graphs, even with negative weights (but no negative cycle is allowed).
  - `'Floyd-Warshall_Boost'`: the Boost implementation of the Floyd-Warshall algorithm. Works also with weighted graphs, even with negative weights (but no negative cycle is allowed).
  - `'Floyd-Warshall_SciPy'`: the SciPy implementation of the Floyd-Warshall algorithm. Works also with weighted graphs, even with negative weights (but no negative cycle is allowed).

  - `'Dijkstra_NetworkX'`: the Dijkstra algorithm, implemented in NetworkX. It works with weighted graphs, but no negative weight is allowed.
  - `'Dijkstra_Boost'`: the Dijkstra algorithm, implemented in Boost (works only with positive weights).
  - `'Johnson_Boost'`: the Johnson algorithm, implemented in Boost (works also with negative weights, if there is no negative cycle).
  - `None` (default): Sage chooses the best algorithm: `'BFS'` if `by_weight` is `False`, `'Dijkstra_Boost'` if all weights are positive, `'Floyd-Warshall_Boost'` otherwise.
- `weight_function` – function (default: `None`); a function that takes as input an edge `(u, v, l)` and outputs its weight. If not `None`, `by_weight` is automatically set to `True`. If `None` and `by_weight` is `True`, we use the edge label `l`, if `l` is not `None`, else `1` as a weight.
- `check_weight` – boolean (default: `True`); if `True`, we check that the weight_function outputs a number for each edge

OUTPUT:

A tuple `(dist, pred)`. They are both dicts of dicts. The first indicates the length `dist[u][v]` of the shortest weighted path from u to v. The second is a compact representation of all the paths - it indicates the predecessor `pred[u][v]` of v in the shortest path from u to v.

Note

Only reachable vertices are present in the dictionaries.

Note

There is a Cython version of this method that is usually much faster for large graphs, as most of the time is actually spent building the final double dictionary. Everything on the subject is to be found in the *distances_all_pairs* module.

EXAMPLES:

Some standard examples (see *shortest_paths()* for more examples on how to use the input variables):

```
sage: G = Graph({0: {1: 1}, 1: {2: 1}, 2: {3: 1}, 3: {4: 2}, 4: {0: 2}},␣
↪sparse=True)
sage: G.plot(edge_labels=True).show()        # long time                    #␣
↪needs sage.plot
sage: dist, pred = G.shortest_path_all_pairs(by_weight = True)
sage: dist
{0: {0: 0, 1: 1, 2: 2, 3: 3, 4: 2},
 1: {0: 1, 1: 0, 2: 1, 3: 2, 4: 3},
 2: {0: 2, 1: 1, 2: 0, 3: 1, 4: 3},
 3: {0: 3, 1: 2, 2: 1, 3: 0, 4: 2},
 4: {0: 2, 1: 3, 2: 3, 3: 2, 4: 0}}
sage: pred
{0: {0: None, 1: 0, 2: 1, 3: 2, 4: 0},
 1: {0: 1, 1: None, 2: 1, 3: 2, 4: 0},
 2: {0: 1, 1: 2, 2: None, 3: 2, 4: 3},
 3: {0: 1, 1: 2, 2: 3, 3: None, 4: 3},
```

(continues on next page)

(continued from previous page)

```
 4: {0: 4, 1: 0, 2: 3, 3: 4, 4: None}}
sage: pred[0]
{0: None, 1: 0, 2: 1, 3: 2, 4: 0}
sage: G = Graph({0: {1: {'weight':1}}, 1: {2: {'weight':1}}, 2: {3: {'weight
↪':1}},
....:            3: {4: {'weight':2}}, 4: {0: {'weight':2}}}, sparse=True)
sage: dist, pred = G.shortest_path_all_pairs(weight_function=lambda e: e[2][
↪'weight'])
sage: dist
{0: {0: 0, 1: 1, 2: 2, 3: 3, 4: 2},
 1: {0: 1, 1: 0, 2: 1, 3: 2, 4: 3},
 2: {0: 2, 1: 1, 2: 0, 3: 1, 4: 3},
 3: {0: 3, 1: 2, 2: 1, 3: 0, 4: 2},
 4: {0: 2, 1: 3, 2: 3, 3: 2, 4: 0}}
sage: pred
{0: {0: None, 1: 0, 2: 1, 3: 2, 4: 0},
 1: {0: 1, 1: None, 2: 1, 3: 2, 4: 0},
 2: {0: 1, 1: 2, 2: None, 3: 2, 4: 3},
 3: {0: 1, 1: 2, 2: 3, 3: None, 4: 3},
 4: {0: 4, 1: 0, 2: 3, 3: 4, 4: None}}
```

```
>>> from sage.all import *
>>> G = Graph({Integer(0): {Integer(1): Integer(1)}, Integer(1): {Integer(2):␣
↪Integer(1)}, Integer(2): {Integer(3): Integer(1)}, Integer(3): {Integer(4):␣
↪Integer(2)}, Integer(4): {Integer(0): Integer(2)}}, sparse=True)
>>> G.plot(edge_labels=True).show()        # long time                       #␣
↪needs sage.plot
>>> dist, pred = G.shortest_path_all_pairs(by_weight = True)
>>> dist
{0: {0: 0, 1: 1, 2: 2, 3: 3, 4: 2},
 1: {0: 1, 1: 0, 2: 1, 3: 2, 4: 3},
 2: {0: 2, 1: 1, 2: 0, 3: 1, 4: 3},
 3: {0: 3, 1: 2, 2: 1, 3: 0, 4: 2},
 4: {0: 2, 1: 3, 2: 3, 3: 2, 4: 0}}
>>> pred
{0: {0: None, 1: 0, 2: 1, 3: 2, 4: 0},
 1: {0: 1, 1: None, 2: 1, 3: 2, 4: 0},
 2: {0: 1, 1: 2, 2: None, 3: 2, 4: 3},
 3: {0: 1, 1: 2, 2: 3, 3: None, 4: 3},
 4: {0: 4, 1: 0, 2: 3, 3: 4, 4: None}}
>>> pred[Integer(0)]
{0: None, 1: 0, 2: 1, 3: 2, 4: 0}
>>> G = Graph({Integer(0): {Integer(1): {'weight':Integer(1)}}, Integer(1):
↪{Integer(2): {'weight':Integer(1)}}, Integer(2): {Integer(3): {'weight
↪':Integer(1)}},
...            Integer(3): {Integer(4): {'weight':Integer(2)}}, Integer(4):
↪{Integer(0): {'weight':Integer(2)}}}, sparse=True)
>>> dist, pred = G.shortest_path_all_pairs(weight_function=lambda e:␣
↪e[Integer(2)]['weight'])
>>> dist
{0: {0: 0, 1: 1, 2: 2, 3: 3, 4: 2},
 1: {0: 1, 1: 0, 2: 1, 3: 2, 4: 3},
```

(continues on next page)

(continued from previous page)

```
 2: {0: 2, 1: 1, 2: 0, 3: 1, 4: 3},
 3: {0: 3, 1: 2, 2: 1, 3: 0, 4: 2},
 4: {0: 2, 1: 3, 2: 3, 3: 2, 4: 0}}
>>> pred
{0: {0: None, 1: 0, 2: 1, 3: 2, 4: 0},
 1: {0: 1, 1: None, 2: 1, 3: 2, 4: 0},
 2: {0: 1, 1: 2, 2: None, 3: 2, 4: 3},
 3: {0: 1, 1: 2, 2: 3, 3: None, 4: 3},
 4: {0: 4, 1: 0, 2: 3, 3: 4, 4: None}}
```

So for example the shortest weighted path from 0 to 3 is obtained as follows. The predecessor of 3 is `pred[0][3] == 2`, the predecessor of 2 is `pred[0][2] == 1`, and the predecessor of 1 is `pred[0][1] == 0`.

```
sage: G = Graph({0: {1:None}, 1: {2:None}, 2: {3: 1}, 3: {4: 2}, 4: {0: 2}},␣
↪sparse=True)
sage: G.shortest_path_all_pairs()
({0: {0: 0, 1: 1, 2: 2, 3: 2, 4: 1},
1: {0: 1, 1: 0, 2: 1, 3: 2, 4: 2},
2: {0: 2, 1: 1, 2: 0, 3: 1, 4: 2},
3: {0: 2, 1: 2, 2: 1, 3: 0, 4: 1},
4: {0: 1, 1: 2, 2: 2, 3: 1, 4: 0}},
{0: {0: None, 1: 0, 2: 1, 3: 4, 4: 0},
1: {0: 1, 1: None, 2: 1, 3: 2, 4: 0},
2: {0: 1, 1: 2, 2: None, 3: 2, 4: 3},
3: {0: 4, 1: 2, 2: 3, 3: None, 4: 3},
4: {0: 4, 1: 0, 2: 3, 3: 4, 4: None}})
sage: G.shortest_path_all_pairs(weight_function=lambda e:(e[2] if e[2] is not␣
↪None else 1))
({0: {0: 0, 1: 1, 2: 2, 3: 3, 4: 2},
1: {0: 1, 1: 0, 2: 1, 3: 2, 4: 3},
2: {0: 2, 1: 1, 2: 0, 3: 1, 4: 3},
3: {0: 3, 1: 2, 2: 1, 3: 0, 4: 2},
4: {0: 2, 1: 3, 2: 3, 3: 2, 4: 0}},
{0: {0: None, 1: 0, 2: 1, 3: 2, 4: 0},
1: {0: 1, 1: None, 2: 1, 3: 2, 4: 0},
2: {0: 1, 1: 2, 2: None, 3: 2, 4: 3},
3: {0: 1, 1: 2, 2: 3, 3: None, 4: 3},
4: {0: 4, 1: 0, 2: 3, 3: 4, 4: None}})
```

```
>>> from sage.all import *
>>> G = Graph({Integer(0): {Integer(1):None}, Integer(1): {Integer(2):None},␣
↪Integer(2): {Integer(3): Integer(1)}, Integer(3): {Integer(4): Integer(2)},␣
↪Integer(4): {Integer(0): Integer(2)}}, sparse=True)
>>> G.shortest_path_all_pairs()
({0: {0: 0, 1: 1, 2: 2, 3: 2, 4: 1},
1: {0: 1, 1: 0, 2: 1, 3: 2, 4: 2},
2: {0: 2, 1: 1, 2: 0, 3: 1, 4: 2},
3: {0: 2, 1: 2, 2: 1, 3: 0, 4: 1},
4: {0: 1, 1: 2, 2: 2, 3: 1, 4: 0}},
{0: {0: None, 1: 0, 2: 1, 3: 4, 4: 0},
1: {0: 1, 1: None, 2: 1, 3: 2, 4: 0},
```

(continues on next page)

(continued from previous page)

```
2: {0: 1, 1: 2, 2: None, 3: 2, 4: 3},
3: {0: 4, 1: 2, 2: 3, 3: None, 4: 3},
4: {0: 4, 1: 0, 2: 3, 3: 4, 4: None}})
>>> G.shortest_path_all_pairs(weight_function=lambda e:(e[Integer(2)] if␣
↪e[Integer(2)] is not None else Integer(1)))
({0: {0: 0, 1: 1, 2: 2, 3: 3, 4: 2},
1: {0: 1, 1: 0, 2: 1, 3: 2, 4: 3},
2: {0: 2, 1: 1, 2: 0, 3: 1, 4: 3},
3: {0: 3, 1: 2, 2: 1, 3: 0, 4: 2},
4: {0: 2, 1: 3, 2: 3, 3: 2, 4: 0}},
{0: {0: None, 1: 0, 2: 1, 3: 2, 4: 0},
1: {0: 1, 1: None, 2: 1, 3: 2, 4: 0},
2: {0: 1, 1: 2, 2: None, 3: 2, 4: 3},
3: {0: 1, 1: 2, 2: 3, 3: None, 4: 3},
4: {0: 4, 1: 0, 2: 3, 3: 4, 4: None}})
```

Checking that distances are equal regardless of the algorithm used:

```
sage: g = graphs.Grid2dGraph(5,5)
sage: d1, _ = g.shortest_path_all_pairs(algorithm='BFS')
sage: d2, _ = g.shortest_path_all_pairs(algorithm='Floyd-Warshall-Cython')
sage: d3, _ = g.shortest_path_all_pairs(algorithm='Floyd-Warshall-Python')
sage: d4, _ = g.shortest_path_all_pairs(algorithm="Dijkstra_NetworkX")      #␣
↪needs networkx
sage: d5, _ = g.shortest_path_all_pairs(algorithm='Dijkstra_Boost')
sage: d6, _ = g.shortest_path_all_pairs(algorithm='Johnson_Boost')
sage: d7, _ = g.shortest_path_all_pairs(algorithm='Floyd-Warshall_Boost')
sage: d8, _ = g.shortest_path_all_pairs(algorithm='Floyd-Warshall_SciPy')   #␣
↪needs scipy
sage: d1 == d2 == d3 == d4 == d5 == d6 == d7 == d8                          #␣
↪needs networkx scipy
True
```

```
>>> from sage.all import *
>>> g = graphs.Grid2dGraph(Integer(5),Integer(5))
>>> d1, _ = g.shortest_path_all_pairs(algorithm='BFS')
>>> d2, _ = g.shortest_path_all_pairs(algorithm='Floyd-Warshall-Cython')
>>> d3, _ = g.shortest_path_all_pairs(algorithm='Floyd-Warshall-Python')
>>> d4, _ = g.shortest_path_all_pairs(algorithm="Dijkstra_NetworkX")      #␣
↪needs networkx
>>> d5, _ = g.shortest_path_all_pairs(algorithm='Dijkstra_Boost')
>>> d6, _ = g.shortest_path_all_pairs(algorithm='Johnson_Boost')
>>> d7, _ = g.shortest_path_all_pairs(algorithm='Floyd-Warshall_Boost')
>>> d8, _ = g.shortest_path_all_pairs(algorithm='Floyd-Warshall_SciPy')   #␣
↪needs scipy
>>> d1 == d2 == d3 == d4 == d5 == d6 == d7 == d8                          #␣
↪needs networkx scipy
True
```

Checking that distances are equal regardless of the algorithm used:

```
sage: g = digraphs.RandomDirectedGNM(6,12)
```

(continues on next page)

(continued from previous page)

```
sage: d1, _ = g.shortest_path_all_pairs(algorithm='BFS')
sage: d2, _ = g.shortest_path_all_pairs(algorithm='Floyd-Warshall-Cython')
sage: d3, _ = g.shortest_path_all_pairs(algorithm='Floyd-Warshall-Python')
sage: d4, _ = g.shortest_path_all_pairs(algorithm="Dijkstra_NetworkX")       #␣
↪needs networkx
sage: d5, _ = g.shortest_path_all_pairs(algorithm='Dijkstra_Boost')
sage: d6, _ = g.shortest_path_all_pairs(algorithm='Johnson_Boost')
sage: d7, _ = g.shortest_path_all_pairs(algorithm='Floyd-Warshall_Boost')
sage: d8, _ = g.shortest_path_all_pairs(algorithm='Floyd-Warshall_SciPy')    #␣
↪needs scipy
sage: d1 == d2 == d3 == d4 == d5 == d6 == d7 == d8                          #␣
↪needs networkx scipy
True
```

```
>>> from sage.all import *
>>> g = digraphs.RandomDirectedGNM(Integer(6),Integer(12))
>>> d1, _ = g.shortest_path_all_pairs(algorithm='BFS')
>>> d2, _ = g.shortest_path_all_pairs(algorithm='Floyd-Warshall-Cython')
>>> d3, _ = g.shortest_path_all_pairs(algorithm='Floyd-Warshall-Python')
>>> d4, _ = g.shortest_path_all_pairs(algorithm="Dijkstra_NetworkX")       #␣
↪needs networkx
>>> d5, _ = g.shortest_path_all_pairs(algorithm='Dijkstra_Boost')
>>> d6, _ = g.shortest_path_all_pairs(algorithm='Johnson_Boost')
>>> d7, _ = g.shortest_path_all_pairs(algorithm='Floyd-Warshall_Boost')
>>> d8, _ = g.shortest_path_all_pairs(algorithm='Floyd-Warshall_SciPy')    #␣
↪needs scipy
>>> d1 == d2 == d3 == d4 == d5 == d6 == d7 == d8                          #␣
↪needs networkx scipy
True
```

Checking that weighted distances are equal regardless of the algorithm used:

```
sage: g = graphs.CompleteGraph(5)
sage: import random
sage: for v, w in g.edges(labels=False, sort=False):
....:     g.add_edge(v, w, random.uniform(1, 10))
sage: d1, _ = g.shortest_path_all_pairs(algorithm='Floyd-Warshall-Python')
sage: d2, _ = g.shortest_path_all_pairs(algorithm="Dijkstra_NetworkX")       #␣
↪needs networkx
sage: d3, _ = g.shortest_path_all_pairs(algorithm='Dijkstra_Boost')
sage: d4, _ = g.shortest_path_all_pairs(algorithm='Johnson_Boost')
sage: d5, _ = g.shortest_path_all_pairs(algorithm='Floyd-Warshall_Boost')
sage: d6, _ = g.shortest_path_all_pairs(algorithm='Floyd-Warshall_SciPy')    #␣
↪needs scipy
sage: d1 == d2 == d3 == d4 == d5 == d6                                       #␣
↪needs networkx scipy
True
```

```
>>> from sage.all import *
>>> g = graphs.CompleteGraph(Integer(5))
>>> import random
>>> for v, w in g.edges(labels=False, sort=False):
```

(continues on next page)

(continued from previous page)

```
...     g.add_edge(v, w, random.uniform(Integer(1), Integer(10)))
>>> d1, _ = g.shortest_path_all_pairs(algorithm='Floyd-Warshall-Python')
>>> d2, _ = g.shortest_path_all_pairs(algorithm="Dijkstra_NetworkX")      #␣
↪needs networkx
>>> d3, _ = g.shortest_path_all_pairs(algorithm='Dijkstra_Boost')
>>> d4, _ = g.shortest_path_all_pairs(algorithm='Johnson_Boost')
>>> d5, _ = g.shortest_path_all_pairs(algorithm='Floyd-Warshall_Boost')
>>> d6, _ = g.shortest_path_all_pairs(algorithm='Floyd-Warshall_SciPy')   #␣
↪needs scipy
>>> d1 == d2 == d3 == d4 == d5 == d6                                      #␣
↪needs networkx scipy
True
```

Checking a random path is valid:

```
sage: dist, path = g.shortest_path_all_pairs(algorithm='BFS')
sage: u,v = g.random_vertex(), g.random_vertex()
sage: p = [v]
sage: while p[0] is not None:
....:   p.insert(0,path[u][p[0]])
sage: len(p) == dist[u][v] + 2
True
```

```
>>> from sage.all import *
>>> dist, path = g.shortest_path_all_pairs(algorithm='BFS')
>>> u,v = g.random_vertex(), g.random_vertex()
>>> p = [v]
>>> while p[Integer(0)] is not None:
...   p.insert(Integer(0),path[u][p[Integer(0)]])
>>> len(p) == dist[u][v] + Integer(2)
True
```

Negative weights:

```
sage: g = DiGraph([(0,1,-2),(1,0,1)], weighted=True)
sage: g.shortest_path_all_pairs(by_weight=True)
Traceback (most recent call last):
...
ValueError: the graph contains a negative cycle
```

```
>>> from sage.all import *
>>> g = DiGraph([(Integer(0),Integer(1),-Integer(2)),(Integer(1),Integer(0),
↪Integer(1))], weighted=True)
>>> g.shortest_path_all_pairs(by_weight=True)
Traceback (most recent call last):
...
ValueError: the graph contains a negative cycle
```

Unreachable vertices are not present in the dictionaries:

```
sage: g = DiGraph([(0,1,1),(1,2,2)])
sage: g.shortest_path_all_pairs(algorithm='BFS')
```

(continues on next page)

(continued from previous page)

```
({0: {0: 0, 1: 1, 2: 2}, 1: {1: 0, 2: 1}, 2: {2: 0}},
 {0: {0: None, 1: 0, 2: 1}, 1: {1: None, 2: 1}, 2: {2: None}})
sage: g.shortest_path_all_pairs(algorithm='Dijkstra_NetworkX')                #␣
↪needs networkx
({0: {0: 0, 1: 1, 2: 2}, 1: {1: 0, 2: 1}, 2: {2: 0}},
 {0: {0: None, 1: 1, 2: 1}, 1: {1: None, 2: 2}, 2: {2: None}})
sage: g.shortest_path_all_pairs(algorithm='Dijkstra_Boost')
({0: {0: 0, 1: 1, 2: 2}, 1: {1: 0, 2: 1}, 2: {2: 0}},
 {0: {0: None, 1: 0, 2: 1}, 1: {1: None, 2: 1}, 2: {2: None}})
sage: g.shortest_path_all_pairs(algorithm='Floyd-Warshall-Python')
({0: {0: 0, 1: 1, 2: 2}, 1: {1: 0, 2: 1}, 2: {2: 0}},
 {0: {0: None, 1: 0, 2: 1}, 1: {1: None, 2: 1}, 2: {2: None}})
sage: g.shortest_path_all_pairs(algorithm='Floyd-Warshall-Cython')
({0: {0: 0, 1: 1, 2: 2}, 1: {1: 0, 2: 1}, 2: {2: 0}},
 {0: {0: None, 1: 0, 2: 1}, 1: {1: None, 2: 1}, 2: {2: None}})
sage: g.shortest_path_all_pairs(algorithm='Floyd-Warshall_SciPy')             #␣
↪needs scipy
({0: {0: 0.0, 1: 1.0, 2: 2.0}, 1: {1: 0.0, 2: 1.0}, 2: {2: 0.0}},
 {0: {0: None, 1: 0, 2: 1}, 1: {1: None, 2: 1}, 2: {2: None}})
```

```
>>> from sage.all import *
>>> g = DiGraph([(Integer(0),Integer(1),Integer(1)),(Integer(1),Integer(2),
↪Integer(2))])
>>> g.shortest_path_all_pairs(algorithm='BFS')
({0: {0: 0, 1: 1, 2: 2}, 1: {1: 0, 2: 1}, 2: {2: 0}},
 {0: {0: None, 1: 0, 2: 1}, 1: {1: None, 2: 1}, 2: {2: None}})
>>> g.shortest_path_all_pairs(algorithm='Dijkstra_NetworkX')                  #␣
↪needs networkx
({0: {0: 0, 1: 1, 2: 2}, 1: {1: 0, 2: 1}, 2: {2: 0}},
 {0: {0: None, 1: 1, 2: 1}, 1: {1: None, 2: 2}, 2: {2: None}})
>>> g.shortest_path_all_pairs(algorithm='Dijkstra_Boost')
({0: {0: 0, 1: 1, 2: 2}, 1: {1: 0, 2: 1}, 2: {2: 0}},
 {0: {0: None, 1: 0, 2: 1}, 1: {1: None, 2: 1}, 2: {2: None}})
>>> g.shortest_path_all_pairs(algorithm='Floyd-Warshall-Python')
({0: {0: 0, 1: 1, 2: 2}, 1: {1: 0, 2: 1}, 2: {2: 0}},
 {0: {0: None, 1: 0, 2: 1}, 1: {1: None, 2: 1}, 2: {2: None}})
>>> g.shortest_path_all_pairs(algorithm='Floyd-Warshall-Cython')
({0: {0: 0, 1: 1, 2: 2}, 1: {1: 0, 2: 1}, 2: {2: 0}},
 {0: {0: None, 1: 0, 2: 1}, 1: {1: None, 2: 1}, 2: {2: None}})
>>> g.shortest_path_all_pairs(algorithm='Floyd-Warshall_SciPy')               #␣
↪needs scipy
({0: {0: 0.0, 1: 1.0, 2: 2.0}, 1: {1: 0.0, 2: 1.0}, 2: {2: 0.0}},
 {0: {0: None, 1: 0, 2: 1}, 1: {1: None, 2: 1}, 2: {2: None}})
```

In order to change the default behavior if the graph is disconnected, we can use default values with dictionaries:

```
sage: G = 2*graphs.PathGraph(2)
sage: d,_ = G.shortest_path_all_pairs()
sage: import itertools
sage: from sage.rings.infinity import Infinity
sage: for u,v in itertools.combinations(G.vertex_iterator(), 2):
....:     print("dist({}, {}) = {}".format(u,v, d[u].get(v,+Infinity)))
```

(continues on next page)

(continued from previous page)

```
dist(0, 1) = 1
dist(0, 2) = +Infinity
dist(0, 3) = +Infinity
dist(1, 2) = +Infinity
dist(1, 3) = +Infinity
dist(2, 3) = 1
```

```
>>> from sage.all import *
>>> G = Integer(2)*graphs.PathGraph(Integer(2))
>>> d,_ = G.shortest_path_all_pairs()
>>> import itertools
>>> from sage.rings.infinity import Infinity
>>> for u,v in itertools.combinations(G.vertex_iterator(), Integer(2)):
...     print("dist({}, {}) = {}".format(u,v, d[u].get(v,+Infinity)))
dist(0, 1) = 1
dist(0, 2) = +Infinity
dist(0, 3) = +Infinity
dist(1, 2) = +Infinity
dist(1, 3) = +Infinity
dist(2, 3) = 1
```

shortest_path_length(*u*, *v*, *by_weight=False*, *algorithm=None*, *weight_function=None*, *check_weight=True*)

Return the minimal length of a path from `u` to `v`.

If there is no path from u to v, returns `Infinity`.

For more information and more examples, we refer to *shortest_path()* and *shortest_paths()*, which have very similar inputs.

INPUT:

- `u`, `v` – the start and the end vertices of the paths
- `by_weight` – boolean (default: `False`); if `True`, the edges in the graph are weighted, otherwise all edges have weight 1
- `algorithm` – string (default: `None`); one of the following algorithms:
  - `'BFS'`: performs a BFS from `u`. Does not work with edge weights.
  - `'BFS_Bid'`: performs a BFS from `u` and from `v`. Does not work with edge weights.
  - `'Dijkstra_NetworkX'`: the Dijkstra algorithm, implemented in NetworkX. Works only with positive weights.
  - `'Dijkstra_Bid_NetworkX'`: performs a Dijkstra visit from `u` and from `v` (NetworkX implementation). Works only with positive weights.
  - `'Dijkstra_Bid'`: a Cython implementation that performs a Dijkstra visit from `u` and from `v`. Works only with positive weights.
  - `'Bellman-Ford_Boost'`: the Bellman-Ford algorithm, implemented in Boost. Works also with negative weights, if there is no negative cycle.
  - `None` (default): Sage chooses the best algorithm: `'BFS_Bid'` if `by_weight` is `False`, `'Dijkstra_Bid'` otherwise.

Note

If there are negative weights and algorithm is `None`, the result is not reliable. This occurs because, for performance reasons, we cannot check whether there are edges with negative weights before running the algorithm. If there are, the user should explicitly input `algorithm='Bellman-Ford_Boost'`.

- `weight_function` – function (default: `None`); a function that takes as input an edge `(u, v, l)` and outputs its weight. If not `None`, `by_weight` is automatically set to `True`. If `None` and `by_weight` is `True`, we use the edge label `l`, if `l` is not `None`, else `1` as a weight.
- `check_weight` – boolean (default: `True`); if `True`, we check that the weight_function outputs a number for each edge

EXAMPLES:

Standard examples:

```
sage: D = graphs.DodecahedralGraph()
sage: D.shortest_path_length(4, 9)
5
sage: D.shortest_path_length(4, 9, algorithm='BFS')
5
sage: D.shortest_path_length(4, 9, algorithm='Dijkstra_NetworkX')           #␣
↪needs networkx
5
sage: D.shortest_path_length(4, 9, algorithm='Dijkstra_Bid_NetworkX')       #␣
↪needs networkx
5
sage: D.shortest_path_length(4, 9, algorithm='Dijkstra_Bid')
5
sage: D.shortest_path_length(4, 9, algorithm='Bellman-Ford_Boost')
5
sage: D.shortest_path_length(5, 5)
0
sage: D.delete_edges(D.edges_incident(13))
sage: D.shortest_path_length(13, 4)
+Infinity
sage: G = Graph({0: {1: 1}, 1: {2: 1}, 2: {3: 1}, 3: {4: 2}, 4: {0: 2}},
....:           sparse=True)
sage: G.plot(edge_labels=True).show()       # long time                     #␣
↪needs sage.plot
sage: G.shortest_path_length(0, 3)
2
sage: G.shortest_path_length(0, 3, by_weight=True)
3
sage: G.shortest_path_length(0, 3, by_weight=True,                          #␣
↪needs networkx
....:                        algorithm='Dijkstra_NetworkX')
3
sage: G.shortest_path_length(0, 3, by_weight=True,                          #␣
↪needs networkx
....:                        algorithm='Dijkstra_Bid_NetworkX')
3
```

```
>>> from sage.all import *
>>> D = graphs.DodecahedralGraph()
>>> D.shortest_path_length(Integer(4), Integer(9))
5
>>> D.shortest_path_length(Integer(4), Integer(9), algorithm='BFS')
5
>>> D.shortest_path_length(Integer(4), Integer(9), algorithm='Dijkstra_
↪NetworkX')           # needs networkx
5
>>> D.shortest_path_length(Integer(4), Integer(9), algorithm='Dijkstra_Bid_
↪NetworkX')       # needs networkx
5
>>> D.shortest_path_length(Integer(4), Integer(9), algorithm='Dijkstra_Bid')
5
>>> D.shortest_path_length(Integer(4), Integer(9), algorithm='Bellman-Ford_
↪Boost')
5
>>> D.shortest_path_length(Integer(5), Integer(5))
0
>>> D.delete_edges(D.edges_incident(Integer(13)))
>>> D.shortest_path_length(Integer(13), Integer(4))
+Infinity
>>> G = Graph({Integer(0): {Integer(1): Integer(1)}, Integer(1): {Integer(2):␣
↪Integer(1)}, Integer(2): {Integer(3): Integer(1)}, Integer(3): {Integer(4):␣
↪Integer(2)}, Integer(4): {Integer(0): Integer(2)}},
...            sparse=True)
>>> G.plot(edge_labels=True).show()       # long time                      #␣
↪needs sage.plot
>>> G.shortest_path_length(Integer(0), Integer(3))
2
>>> G.shortest_path_length(Integer(0), Integer(3), by_weight=True)
3
>>> G.shortest_path_length(Integer(0), Integer(3), by_weight=True,          ␣
↪              # needs networkx
...                        algorithm='Dijkstra_NetworkX')
3
>>> G.shortest_path_length(Integer(0), Integer(3), by_weight=True,          ␣
↪              # needs networkx
...                        algorithm='Dijkstra_Bid_NetworkX')
3
```

If Dijkstra is used with negative weights, usually it raises an error:

```
sage: G = DiGraph({0: {1: 1}, 1: {2: 1}, 2: {3: 1}, 3: {4: 2}, 4: {0: -2}},
....:             sparse=True)
sage: G.shortest_path_length(4, 1, by_weight=True, algorithm=None)
Traceback (most recent call last):
...
ValueError: the graph contains an edge with negative weight
sage: G.shortest_path_length(4, 1, by_weight=True,
....:                        algorithm='Bellman-Ford_Boost')
-1
```

```
>>> from sage.all import *
>>> G = DiGraph({Integer(0): {Integer(1): Integer(1)}, Integer(1):
↪{Integer(2): Integer(1)}, Integer(2): {Integer(3): Integer(1)}, Integer(3):
↪{Integer(4): Integer(2)}, Integer(4): {Integer(0): -Integer(2)}},
...              sparse=True)
>>> G.shortest_path_length(Integer(4), Integer(1), by_weight=True,␣
↪algorithm=None)
Traceback (most recent call last):
...
ValueError: the graph contains an edge with negative weight
>>> G.shortest_path_length(Integer(4), Integer(1), by_weight=True,
...                        algorithm='Bellman-Ford_Boost')
-1
```

However, sometimes the result may be wrong, and no error is raised:

```
sage: G = DiGraph([(0,1,1), (1,2,1), (0,3,1000), (3,4,-3000), (4,2,1000)])
sage: G.shortest_path_length(0, 2, by_weight=True,
....:                        algorithm='Bellman-Ford_Boost')
-1000
sage: G.shortest_path_length(0, 2, by_weight=True)
2
```

```
>>> from sage.all import *
>>> G = DiGraph([(Integer(0),Integer(1),Integer(1)), (Integer(1),Integer(2),
↪Integer(1)), (Integer(0),Integer(3),Integer(1000)), (Integer(3),Integer(4),-
↪Integer(3000)), (Integer(4),Integer(2),Integer(1000))])
>>> G.shortest_path_length(Integer(0), Integer(2), by_weight=True,
...                        algorithm='Bellman-Ford_Boost')
-1000
>>> G.shortest_path_length(Integer(0), Integer(2), by_weight=True)
2
```

shortest_path_lengths(*u*, *by_weight=False*, *algorithm=None*, *weight_function=None*, *check_weight=True*)

Return the length of a shortest path from `u` to any other vertex.

Returns a dictionary of shortest path lengths keyed by targets, excluding all vertices that are not reachable from u.

For more information on the input variables and more examples, we refer to *shortest_paths()* which has the same input variables.

INPUT:

- `u` – the starting vertex
- `by_weight` – boolean (default: `False`); if `True`, the edges in the graph are weighted, otherwise all edges have weight 1
- `algorithm` – string (default: `None`); one of the following algorithms:
  - `'BFS'`: performs a BFS from `u`. Does not work with edge weights.
  - `'Dijkstra_NetworkX'`: the Dijkstra algorithm, implemented in NetworkX (works only with positive weights).

  - `'Dijkstra_Boost'`: the Dijkstra algorithm, implemented in Boost (works only with positive weights).
  - `'Bellman-Ford_Boost'`: the Bellman-Ford algorithm, implemented in Boost (works also with negative weights, if there is no negative cycle).
  - `None` (default): Sage chooses the best algorithm: `'BFS'` if `by_weight` is `False`, `'Dijkstra_Boost'` if all weights are positive, `'Bellman-Ford_Boost'` otherwise.

- `weight_function` – function (default: `None`); a function that takes as input an edge `(u, v, l)` and outputs its weight. If not `None`, `by_weight` is automatically set to `True`. If `None` and `by_weight` is `True`, we use the edge label `l`, if `l` is not `None`, else `1` as a weight.
- `check_weight` – boolean (default: `True`); if `True`, we check that the weight_function outputs a number for each edge

EXAMPLES:

Unweighted case:

```
sage: D = graphs.DodecahedralGraph()
sage: D.shortest_path_lengths(0)
{0: 0, 1: 1, 2: 2, 3: 2, 4: 3, 5: 4, 6: 3, 7: 3, 8: 2, 9: 2, 10: 1,
 11: 2, 12: 3, 13: 3, 14: 4, 15: 5, 16: 4, 17: 3, 18: 2, 19: 1}
```

```
>>> from sage.all import *
>>> D = graphs.DodecahedralGraph()
>>> D.shortest_path_lengths(Integer(0))
{0: 0, 1: 1, 2: 2, 3: 2, 4: 3, 5: 4, 6: 3, 7: 3, 8: 2, 9: 2, 10: 1,
 11: 2, 12: 3, 13: 3, 14: 4, 15: 5, 16: 4, 17: 3, 18: 2, 19: 1}
```

Weighted case:

```
sage: G = Graph({0: {1: 1}, 1: {2: 1}, 2: {3: 1}, 3: {4: 2}, 4: {0: 2}},
....:           sparse=True)
sage: G.plot(edge_labels=True).show()       # long time                    #␣
↪needs sage.plot
sage: G.shortest_path_lengths(0, by_weight=True)
{0: 0, 1: 1, 2: 2, 3: 3, 4: 2}
```

```
>>> from sage.all import *
>>> G = Graph({Integer(0): {Integer(1): Integer(1)}, Integer(1): {Integer(2):␣
↪Integer(1)}, Integer(2): {Integer(3): Integer(1)}, Integer(3): {Integer(4):␣
↪Integer(2)}, Integer(4): {Integer(0): Integer(2)}},
...           sparse=True)
>>> G.plot(edge_labels=True).show()       # long time                    #␣
↪needs sage.plot
>>> G.shortest_path_lengths(Integer(0), by_weight=True)
{0: 0, 1: 1, 2: 2, 3: 3, 4: 2}
```

Using a weight function:

```
sage: D = DiGraph([(0,1,{'weight':1}), (1,2,{'weight':3}), (0,2,{'weight':5}
↪)])
sage: weight_function = lambda e: e[2]['weight']
sage: D.shortest_path_lengths(1, algorithm='Dijkstra_NetworkX',             #␣
```

(continues on next page)

(continued from previous page)

```
↪needs networkx
....:                           by_weight=False)
{1: 0, 2: 1}
sage: D.shortest_path_lengths(0, weight_function=weight_function)
{0: 0, 1: 1, 2: 4}
sage: D.shortest_path_lengths(1, weight_function=weight_function)
{1: 0, 2: 3}
```

```
>>> from sage.all import *
>>> D = DiGraph([(Integer(0),Integer(1),{'weight':Integer(1)}), (Integer(1),
↪Integer(2),{'weight':Integer(3)}), (Integer(0),Integer(2),{'weight
↪':Integer(5)})])
>>> weight_function = lambda e: e[Integer(2)]['weight']
>>> D.shortest_path_lengths(Integer(1), algorithm='Dijkstra_NetworkX',␣
↪     # needs networkx
...                           by_weight=False)
{1: 0, 2: 1}
>>> D.shortest_path_lengths(Integer(0), weight_function=weight_function)
{0: 0, 1: 1, 2: 4}
>>> D.shortest_path_lengths(Integer(1), weight_function=weight_function)
{1: 0, 2: 3}
```

Negative weights:

```
sage: D = DiGraph([(0,1,{'weight':-1}), (1,2,{'weight':3}), (0,2,{'weight':5}
↪)])
sage: D.shortest_path_lengths(0, weight_function=weight_function)
{0: 0, 1: -1, 2: 2}
```

```
>>> from sage.all import *
>>> D = DiGraph([(Integer(0),Integer(1),{'weight':-Integer(1)}), (Integer(1),
↪Integer(2),{'weight':Integer(3)}), (Integer(0),Integer(2),{'weight
↪':Integer(5)})])
>>> D.shortest_path_lengths(Integer(0), weight_function=weight_function)
{0: 0, 1: -1, 2: 2}
```

Negative cycles:

```
sage: D = DiGraph([(0,1,{'weight':-5}), (1,2,{'weight':3}), (2,0,{'weight':1}
↪)])
sage: D.shortest_path_lengths(0, weight_function=weight_function)
Traceback (most recent call last):
...
ValueError: the graph contains a negative cycle
```

```
>>> from sage.all import *
>>> D = DiGraph([(Integer(0),Integer(1),{'weight':-Integer(5)}), (Integer(1),
↪Integer(2),{'weight':Integer(3)}), (Integer(2),Integer(0),{'weight
↪':Integer(1)})])
>>> D.shortest_path_lengths(Integer(0), weight_function=weight_function)
Traceback (most recent call last):
```

(continues on next page)

(continued from previous page)

```
...
ValueError: the graph contains a negative cycle
```

Checking that distances are equal regardless of the algorithm used:

```
sage: g = graphs.Grid2dGraph(5,5)
sage: d1 = g.shortest_path_lengths((0,0), algorithm='BFS')
sage: d2 = g.shortest_path_lengths((0,0), algorithm="Dijkstra_NetworkX")     #␣
↪needs networkx
sage: d3 = g.shortest_path_lengths((0,0), algorithm='Dijkstra_Boost')
sage: d4 = g.shortest_path_lengths((0,0), algorithm='Bellman-Ford_Boost')
sage: d1 == d2 == d3 == d4                                                   #␣
↪needs networkx
True
```

```
>>> from sage.all import *
>>> g = graphs.Grid2dGraph(Integer(5),Integer(5))
>>> d1 = g.shortest_path_lengths((Integer(0),Integer(0)), algorithm='BFS')
>>> d2 = g.shortest_path_lengths((Integer(0),Integer(0)), algorithm="Dijkstra_
↪NetworkX")     # needs networkx
>>> d3 = g.shortest_path_lengths((Integer(0),Integer(0)), algorithm='Dijkstra_
↪Boost')
>>> d4 = g.shortest_path_lengths((Integer(0),Integer(0)), algorithm='Bellman-
↪Ford_Boost')
>>> d1 == d2 == d3 == d4                                                     #␣
↪needs networkx
True
```

shortest_paths(*u*, *by_weight=False*, *algorithm=None*, *weight_function=None*, *check_weight=True*, *cutoff=None*)

Return a dictionary associating to each vertex `v` a shortest path from `u` to `v`, if it exists.

If u and v are not connected, vertex v is not present in the dictionary.

INPUT:

- `u` – the starting vertex
- `by_weight` – boolean (default: `False`); if `True`, the edges in the graph are weighted, otherwise all edges have weight 1
- `algorithm` – string (default: `None`); one of the following algorithms:
  - `'BFS'`: performs a BFS from `u`. Does not work with edge weights.
  - `'Dijkstra_NetworkX'`: the Dijkstra algorithm, implemented in NetworkX (works only with positive weights).
  - `'Dijkstra_Boost'`: the Dijkstra algorithm, implemented in Boost (works only with positive weights).
  - `'Bellman-Ford_Boost'`: the Bellman-Ford algorithm, implemented in Boost (works also with negative weights, if there is no negative cycle).
  - `None` (default): Sage chooses the best algorithm: `'BFS'` if `by_weight` is `False`, `'Dijkstra_Boost'` if all weights are positive, `'Bellman-Ford_Boost'` otherwise.

- `weight_function` – function (default: `None`); a function that takes as input an edge `(u, v, l)` and outputs its weight. If not `None`, `by_weight` is automatically set to `True`. If `None` and `by_weight` is `True`, we use the edge label `l`, if `l` is not `None`, else `1` as a weight.
- `check_weight` – boolean (default: `True`); if `True`, we check that the weight_function outputs a number for each edge
- `cutoff` – integer (default: `None`); depth to stop search (used only if `algorithm=='BFS'`)

EXAMPLES:

Standard example:

```
sage: D = graphs.DodecahedralGraph()
sage: D.shortest_paths(0)
{0: [0], 1: [0, 1], 2: [0, 1, 2], 3: [0, 19, 3], 4: [0, 19, 3, 4],
 5: [0, 1, 2, 6, 5], 6: [0, 1, 2, 6], 7: [0, 1, 8, 7], 8: [0, 1, 8],
 9: [0, 10, 9], 10: [0, 10], 11: [0, 10, 11], 12: [0, 10, 11, 12],
 13: [0, 10, 9, 13], 14: [0, 1, 8, 7, 14],
 15: [0, 19, 18, 17, 16, 15], 16: [0, 19, 18, 17, 16],
 17: [0, 19, 18, 17], 18: [0, 19, 18], 19: [0, 19]}
```

```
>>> from sage.all import *
>>> D = graphs.DodecahedralGraph()
>>> D.shortest_paths(Integer(0))
{0: [0], 1: [0, 1], 2: [0, 1, 2], 3: [0, 19, 3], 4: [0, 19, 3, 4],
 5: [0, 1, 2, 6, 5], 6: [0, 1, 2, 6], 7: [0, 1, 8, 7], 8: [0, 1, 8],
 9: [0, 10, 9], 10: [0, 10], 11: [0, 10, 11], 12: [0, 10, 11, 12],
 13: [0, 10, 9, 13], 14: [0, 1, 8, 7, 14],
 15: [0, 19, 18, 17, 16, 15], 16: [0, 19, 18, 17, 16],
 17: [0, 19, 18, 17], 18: [0, 19, 18], 19: [0, 19]}
```

All these paths are obviously induced graphs:

```
sage: all(D.subgraph(p).is_isomorphic(graphs.PathGraph(len(p))) for p in D.
↪shortest_paths(0).values())
True
```

```
>>> from sage.all import *
>>> all(D.subgraph(p).is_isomorphic(graphs.PathGraph(len(p))) for p in D.
↪shortest_paths(Integer(0)).values())
True
```

```
sage: D.shortest_paths(0, cutoff=2)
{0: [0], 1: [0, 1], 2: [0, 1, 2], 3: [0, 19, 3], 8: [0, 1, 8],
 9: [0, 10, 9], 10: [0, 10], 11: [0, 10, 11], 18: [0, 19, 18],
 19: [0, 19]}
sage: G = Graph( { 0: {1: 1}, 1: {2: 1}, 2: {3: 1}, 3: {4: 2}, 4: {0: 2} },␣
↪sparse=True)
sage: G.plot(edge_labels=True).show()        # long time                    #␣
↪needs sage.plot
sage: G.shortest_paths(0, by_weight=True)
{0: [0], 1: [0, 1], 2: [0, 1, 2], 3: [0, 1, 2, 3], 4: [0, 4]}
```

```
>>> from sage.all import *
>>> D.shortest_paths(Integer(0), cutoff=Integer(2))
{0: [0], 1: [0, 1], 2: [0, 1, 2], 3: [0, 19, 3], 8: [0, 1, 8],
 9: [0, 10, 9], 10: [0, 10], 11: [0, 10, 11], 18: [0, 19, 18],
 19: [0, 19]}
>>> G = Graph( { Integer(0): {Integer(1): Integer(1)}, Integer(1):
↪{Integer(2): Integer(1)}, Integer(2): {Integer(3): Integer(1)}, Integer(3):
↪{Integer(4): Integer(2)}, Integer(4): {Integer(0): Integer(2)} },␣
↪sparse=True)
>>> G.plot(edge_labels=True).show()        # long time                        #␣
↪needs sage.plot
>>> G.shortest_paths(Integer(0), by_weight=True)
{0: [0], 1: [0, 1], 2: [0, 1, 2], 3: [0, 1, 2, 3], 4: [0, 4]}
```

Weighted shortest paths:

```
sage: D = DiGraph([(0,1,1),(1,2,3),(0,2,5)])
sage: D.shortest_paths(0)
{0: [0], 1: [0, 1], 2: [0, 2]}
sage: D.shortest_paths(0, by_weight=True)
{0: [0], 1: [0, 1], 2: [0, 1, 2]}
```

```
>>> from sage.all import *
>>> D = DiGraph([(Integer(0),Integer(1),Integer(1)),(Integer(1),Integer(2),
↪Integer(3)),(Integer(0),Integer(2),Integer(5))])
>>> D.shortest_paths(Integer(0))
{0: [0], 1: [0, 1], 2: [0, 2]}
>>> D.shortest_paths(Integer(0), by_weight=True)
{0: [0], 1: [0, 1], 2: [0, 1, 2]}
```

Using a weight function (this way, `by_weight` is set to `True`):

```
sage: D = DiGraph([(0,1,{'weight':1}),(1,2,{'weight':3}),(0,2,{'weight':5})])
sage: weight_function = lambda e:e[2]['weight']
sage: D.shortest_paths(0, weight_function=weight_function)
{0: [0], 1: [0, 1], 2: [0, 1, 2]}
```

```
>>> from sage.all import *
>>> D = DiGraph([(Integer(0),Integer(1),{'weight':Integer(1)}),(Integer(1),
↪Integer(2),{'weight':Integer(3)}),(Integer(0),Integer(2),{'weight
↪':Integer(5)})])
>>> weight_function = lambda e:e[Integer(2)]['weight']
>>> D.shortest_paths(Integer(0), weight_function=weight_function)
{0: [0], 1: [0, 1], 2: [0, 1, 2]}
```

If the weight function does not match the label:

```
sage: D.shortest_paths(0, weight_function=lambda e:e[2])
Traceback (most recent call last):
...
ValueError: the weight function cannot find the weight of (0, 1, {'weight': 1}
↪)
```

```
>>> from sage.all import *
>>> D.shortest_paths(Integer(0), weight_function=lambda e:e[Integer(2)])
Traceback (most recent call last):
...
ValueError: the weight function cannot find the weight of (0, 1, {'weight': 1}
↪)
```

However, if `check_weight` is set to `False`, unexpected behavior may occur:

```
sage: D.shortest_paths(0, algorithm='Dijkstra_NetworkX',                    #␣
↪needs networkx
....:                  weight_function=lambda e:e[2], check_weight=False)
Traceback (most recent call last):
...
TypeError: unsupported operand type(s) for +: 'int' and 'dict'
```

```
>>> from sage.all import *
>>> D.shortest_paths(Integer(0), algorithm='Dijkstra_NetworkX',              ␣
↪     # needs networkx
...                   weight_function=lambda e:e[Integer(2)], check_
↪weight=False)
Traceback (most recent call last):
...
TypeError: unsupported operand type(s) for +: 'int' and 'dict'
```

Negative weights:

```
sage: D = DiGraph([(0,1,1),(1,2,-2),(0,2,4)])
sage: D.shortest_paths(0, by_weight=True)
{0: [0], 1: [0, 1], 2: [0, 1, 2]}
```

```
>>> from sage.all import *
>>> D = DiGraph([(Integer(0),Integer(1),Integer(1)),(Integer(1),Integer(2),-
↪Integer(2)),(Integer(0),Integer(2),Integer(4))])
>>> D.shortest_paths(Integer(0), by_weight=True)
{0: [0], 1: [0, 1], 2: [0, 1, 2]}
```

Negative cycles:

```
sage: D.add_edge(2,0,0)
sage: D.shortest_paths(0, by_weight=True)
Traceback (most recent call last):
...
ValueError: the graph contains a negative cycle
```

```
>>> from sage.all import *
>>> D.add_edge(Integer(2),Integer(0),Integer(0))
>>> D.shortest_paths(Integer(0), by_weight=True)
Traceback (most recent call last):
...
ValueError: the graph contains a negative cycle
```

shortest_simple_paths(*source*, *target*, *weight_function=None*, *by_weight=False*, *check_weight=True*, *algorithm=None*, *report_edges=False*, *labels=False*, *report_weight=False*)

Return an iterator over the simple paths between a pair of vertices.

This method returns an iterator over the simple paths (i.e., without repetition) from `source` to `target`. By default (`by_weight` is `False`), the paths are reported by increasing number of edges. When `by_weight` is `True`, the paths are reported by increasing weights.

In case of weighted graphs negative weights are not allowed.

If `source` is the same vertex as `target`, then `[[source]]` is returned – a list containing the 1-vertex, 0-edge path `source`.

By default Postponed Node Classification (`algorithm=PNC`) algorithm [ACN2023] is used.

The loops and the multiedges if present in the given graph are ignored and only minimum of the edge labels is kept in case of multiedges.

INPUT:

- `source` – a vertex of the graph, where to start
- `target` – a vertex of the graph, where to end
- `weight_function` – function (default: `None`); a function that takes as input an edge `(u, v, l)` and outputs its weight. If not `None`, `by_weight` is automatically set to `True`. If `None` and `by_weight` is `True`, we use the edge label `l` as a weight.
- `by_weight` – boolean (default: `False`); if `True`, the edges in the graph are weighted, otherwise all edges have weight 1
- `check_weight` – boolean (default: `True`); whether to check that the `weight_function` outputs a number for each edge
- `algorithm` – string (default: `None`); the algorithm to use in computing `k` shortest paths of `self`. The following algorithms are supported:
  - `'Yen'` – Yen's algorithm [Yen1970] (*yen_k_shortest_simple_paths()*)
  - `'Feng'` – an improved version of Yen's algorithm [Feng2014] (*feng_k_shortest_simple_paths()*)
  - `'PNC'` – an improved version of Feng's algorithm [ACN2023] (*pnc_k_shortest_simple_paths()*)
- `report_edges` – boolean (default: `False`); whether to report paths as list of vertices (default) or list of edges. When set to `False`, the `labels` parameter is ignored.
- `labels` – boolean (default: `False`); if `False`, each edge is simply a pair `(u, v)` of vertices. Otherwise a list of edges along with its edge labels are used to represent the path.
- `report_weight` – boolean (default: `False`); if `False`, just the path between `source` and `target` is returned. Otherwise a tuple of path length and path is returned.

EXAMPLES:

```
sage: g = DiGraph([(1, 2, 20), (1, 3, 10), (1, 4, 30),
....:              (2, 5, 20), (3, 5, 10), (4, 5, 30)])
sage: list(g.shortest_simple_paths(1, 5, by_weight=True, algorithm='Yen'))
[[1, 3, 5], [1, 2, 5], [1, 4, 5]]
sage: list(g.shortest_simple_paths(1, 5, algorithm='Yen'))
[[1, 2, 5], [1, 3, 5], [1, 4, 5]]
sage: list(g.shortest_simple_paths(1, 1))
[[1]]
```

(continues on next page)

(continued from previous page)

```
sage: list(g.shortest_simple_paths(1, 5, by_weight=True,
....:                              report_edges=True, report_weight=True,␣
↪labels=True))
[(20.0, [(1, 3, 10), (3, 5, 10)]),
 (40.0, [(1, 2, 20), (2, 5, 20)]),
 (60.0, [(1, 4, 30), (4, 5, 30)])]
sage: list(g.shortest_simple_paths(1, 5, by_weight=True, algorithm='Feng',
....:                              report_edges=True, report_weight=True))
[(20.0, [(1, 3), (3, 5)]), (40.0, [(1, 2), (2, 5)]), (60.0, [(1, 4), (4, 5)])]
sage: list(g.shortest_simple_paths(1, 5, report_edges=True, report_
↪weight=True))
[(2.0, [(1, 2), (2, 5)]), (2.0, [(1, 4), (4, 5)]), (2.0, [(1, 3), (3, 5)])]
sage: list(g.shortest_simple_paths(1, 5, by_weight=True, report_edges=True))
[[(1, 3), (3, 5)], [(1, 2), (2, 5)], [(1, 4), (4, 5)]]
sage: list(g.shortest_simple_paths(1, 5, by_weight=True, algorithm='Feng',
....:                              report_edges=True, labels=True))
[[(1, 3, 10), (3, 5, 10)], [(1, 2, 20), (2, 5, 20)], [(1, 4, 30), (4, 5, 30)]]
sage: g = Graph([(1, 2, 20), (1, 3, 10), (1, 4, 30), (2, 5, 20),
....:            (3, 5, 10), (4, 5, 30), (1, 6, 100), (5, 6, 5)])
sage: list(g.shortest_simple_paths(1, 6, by_weight = True))
[[1, 3, 5, 6], [1, 2, 5, 6], [1, 4, 5, 6], [1, 6]]
sage: list(g.shortest_simple_paths(1, 6, algorithm='Yen'))
[[1, 6], [1, 2, 5, 6], [1, 3, 5, 6], [1, 4, 5, 6]]
sage: list(g.shortest_simple_paths(1, 6,
....:                              report_edges=True, report_weight=True,␣
↪labels=True))
[(1.0, [(1, 6, 100)]),
 (3.0, [(1, 4, 30), (4, 5, 30), (5, 6, 5)]),
 (3.0, [(1, 3, 10), (3, 5, 10), (5, 6, 5)]),
 (3.0, [(1, 2, 20), (2, 5, 20), (5, 6, 5)])]
sage: list(g.shortest_simple_paths(1, 6, by_weight=True,
....:                              report_edges=True, report_weight=True,␣
↪labels=True))
[(25.0, [(1, 3, 10), (3, 5, 10), (5, 6, 5)]),
 (45.0, [(1, 2, 20), (2, 5, 20), (5, 6, 5)]),
 (65.0, [(1, 4, 30), (4, 5, 30), (5, 6, 5)]),
 (100.0, [(1, 6, 100)])]
sage: list(g.shortest_simple_paths(1, 6, by_weight=True,
....:                              report_edges=True, labels=True))
[[(1, 3, 10), (3, 5, 10), (5, 6, 5)],
 [(1, 2, 20), (2, 5, 20), (5, 6, 5)],
 [(1, 4, 30), (4, 5, 30), (5, 6, 5)],
 [(1, 6, 100)]]
sage: list(g.shortest_simple_paths(1, 6, report_edges=True, labels=True))
[[(1, 6, 100)],
 [(1, 4, 30), (4, 5, 30), (5, 6, 5)],
 [(1, 3, 10), (3, 5, 10), (5, 6, 5)],
 [(1, 2, 20), (2, 5, 20), (5, 6, 5)]]
```

```
>>> from sage.all import *
>>> g = DiGraph([(Integer(1), Integer(2), Integer(20)), (Integer(1),␣
↪Integer(3), Integer(10)), (Integer(1), Integer(4), Integer(30)),
```

(continues on next page)

(continued from previous page)

```
...              (Integer(2), Integer(5), Integer(20)), (Integer(3),␣
↪Integer(5), Integer(10)), (Integer(4), Integer(5), Integer(30))])
>>> list(g.shortest_simple_paths(Integer(1), Integer(5), by_weight=True,␣
↪algorithm='Yen'))
[[1, 3, 5], [1, 2, 5], [1, 4, 5]]
>>> list(g.shortest_simple_paths(Integer(1), Integer(5), algorithm='Yen'))
[[1, 2, 5], [1, 3, 5], [1, 4, 5]]
>>> list(g.shortest_simple_paths(Integer(1), Integer(1)))
[[1]]
>>> list(g.shortest_simple_paths(Integer(1), Integer(5), by_weight=True,
...                              report_edges=True, report_weight=True,␣
↪labels=True))
[(20.0, [(1, 3, 10), (3, 5, 10)]),
 (40.0, [(1, 2, 20), (2, 5, 20)]),
 (60.0, [(1, 4, 30), (4, 5, 30)])]
>>> list(g.shortest_simple_paths(Integer(1), Integer(5), by_weight=True,␣
↪algorithm='Feng',
...                              report_edges=True, report_weight=True))
[(20.0, [(1, 3), (3, 5)]), (40.0, [(1, 2), (2, 5)]), (60.0, [(1, 4), (4, 5)])]
>>> list(g.shortest_simple_paths(Integer(1), Integer(5), report_edges=True,␣
↪report_weight=True))
[(2.0, [(1, 2), (2, 5)]), (2.0, [(1, 4), (4, 5)]), (2.0, [(1, 3), (3, 5)])]
>>> list(g.shortest_simple_paths(Integer(1), Integer(5), by_weight=True,␣
↪report_edges=True))
[[(1, 3), (3, 5)], [(1, 2), (2, 5)], [(1, 4), (4, 5)]]
>>> list(g.shortest_simple_paths(Integer(1), Integer(5), by_weight=True,␣
↪algorithm='Feng',
...                              report_edges=True, labels=True))
[[(1, 3, 10), (3, 5, 10)], [(1, 2, 20), (2, 5, 20)], [(1, 4, 30), (4, 5, 30)]]
>>> g = Graph([(Integer(1), Integer(2), Integer(20)), (Integer(1), Integer(3),
↪ Integer(10)), (Integer(1), Integer(4), Integer(30)), (Integer(2),␣
↪Integer(5), Integer(20)),
...            (Integer(3), Integer(5), Integer(10)), (Integer(4), Integer(5),
↪ Integer(30)), (Integer(1), Integer(6), Integer(100)), (Integer(5),␣
↪Integer(6), Integer(5))])
>>> list(g.shortest_simple_paths(Integer(1), Integer(6), by_weight = True))
[[1, 3, 5, 6], [1, 2, 5, 6], [1, 4, 5, 6], [1, 6]]
>>> list(g.shortest_simple_paths(Integer(1), Integer(6), algorithm='Yen'))
[[1, 6], [1, 2, 5, 6], [1, 3, 5, 6], [1, 4, 5, 6]]
>>> list(g.shortest_simple_paths(Integer(1), Integer(6),
...                              report_edges=True, report_weight=True,␣
↪labels=True))
[(1.0, [(1, 6, 100)]),
 (3.0, [(1, 4, 30), (4, 5, 30), (5, 6, 5)]),
 (3.0, [(1, 3, 10), (3, 5, 10), (5, 6, 5)]),
 (3.0, [(1, 2, 20), (2, 5, 20), (5, 6, 5)])]
>>> list(g.shortest_simple_paths(Integer(1), Integer(6), by_weight=True,
...                              report_edges=True, report_weight=True,␣
↪labels=True))
[(25.0, [(1, 3, 10), (3, 5, 10), (5, 6, 5)]),
 (45.0, [(1, 2, 20), (2, 5, 20), (5, 6, 5)]),
 (65.0, [(1, 4, 30), (4, 5, 30), (5, 6, 5)]),
```

(continues on next page)

(continued from previous page)

```
 (100.0, [(1, 6, 100)])]
>>> list(g.shortest_simple_paths(Integer(1), Integer(6), by_weight=True,
...                              report_edges=True, labels=True))
[[(1, 3, 10), (3, 5, 10), (5, 6, 5)],
 [(1, 2, 20), (2, 5, 20), (5, 6, 5)],
 [(1, 4, 30), (4, 5, 30), (5, 6, 5)],
 [(1, 6, 100)]]
>>> list(g.shortest_simple_paths(Integer(1), Integer(6), report_edges=True,␣
↪labels=True))
[[(1, 6, 100)],
 [(1, 4, 30), (4, 5, 30), (5, 6, 5)],
 [(1, 3, 10), (3, 5, 10), (5, 6, 5)],
 [(1, 2, 20), (2, 5, 20), (5, 6, 5)]]
```

show(*method='matplotlib'*, ***kwds*)

Show the (di)graph.

INPUT:

- `method` – string (default: `'matplotlib'`); method to use to display the graph, either `'matplotlib'`, or `'js'` to visualize it in a browser using d3.js.
- Any other argument supported by the drawing functions:
  - `'matplotlib'` – see *GenericGraph.plot* and `sage.plot.graphics.Graphics.show()`
  - `'js'` – see *gen_html_code()*

EXAMPLES:

```
sage: C = graphs.CubeGraph(8)
sage: P = C.plot(vertex_labels=False, vertex_size=0, graph_border=True)     #␣
↪needs sage.plot
sage: P.show()                       # long time (3s on sage.math, 2011),␣
↪needs sage.plot
```

```
>>> from sage.all import *
>>> C = graphs.CubeGraph(Integer(8))
>>> P = C.plot(vertex_labels=False, vertex_size=Integer(0), graph_
↪border=True)     # needs sage.plot
>>> P.show()                         # long time (3s on sage.math, 2011), needs␣
↪sage.plot
```

show3d(*bgcolor=(1, 1, 1)*, *vertex_colors=None*, *vertex_size=0.06*, *edge_colors=None*, *edge_size=0.02*, *edge_size2=0.0325*, *pos3d=None*, *color_by_label=False*, *engine='threejs'*, ***kwds*)

Plot the graph and show the resulting plot.

INPUT:

- `bgcolor` – rgb tuple (default: `(1,1,1)`)
- `vertex_size` – float (default: 0.06)
- `vertex_labels` – boolean (default: `False`); whether to display vertices using text labels instead of spheres

- `vertex_colors` – dictionary (default: `None`); optional dictionary to specify vertex colors: each key is a color recognizable by `tachyon` (rgb tuple (default: `(1,0,0)`)), and each corresponding entry is a list of vertices. If a vertex is not listed, it looks invisible on the resulting plot (it doesn't get drawn).
- `edge_colors` – dictionary (default: `None`); a dictionary specifying edge colors: each key is a color recognized by `tachyon` (default: `(0,0,0)`), and each entry is a list of edges.
- `color_by_label` – boolean or dictionary or function (default: `False`) whether to color each edge with a different color according to its label; the colors are chosen along a rainbow, unless they are specified by a function or dictionary mapping labels to colors; this option is incompatible with `edge_color` and `edge_colors`.
- `edge_size` – float (default: 0.02)
- `edge_size2` – float (default: 0.0325); used for `Tachyon` sleeves
- `pos3d` – a position dictionary for the vertices
- `layout`, `iterations`, ... – layout options; see *layout()*
- `engine` – string (default: `'threejs'`); the renderer to use among:
  - `'threejs'`: interactive web-based 3D viewer using JavaScript and a WebGL renderer
  - `'jmol'`: interactive 3D viewer using Java
  - `'tachyon'`: ray tracer generating a static PNG image
- `xres` – resolution
- `yres` – resolution
- `**kwds` – passed on to the rendering engine

EXAMPLES:

```
sage: G = graphs.CubeGraph(5)
sage: G.show3d(iterations=500, edge_size=None, vertex_size=0.04)     # long␣
↪time, needs sage.plot
```

```
>>> from sage.all import *
>>> G = graphs.CubeGraph(Integer(5))
>>> G.show3d(iterations=Integer(500), edge_size=None, vertex_size=RealNumber(
↪'0.04'))     # long time, needs sage.plot
```

We plot a fairly complicated Cayley graph:

```
sage: A5 = AlternatingGroup(5); A5                                          #␣
↪needs sage.groups
Alternating group of order 5!/2 as a permutation group
sage: G = A5.cayley_graph()                                                 #␣
↪needs sage.groups
sage: G.show3d(vertex_size=0.03,     # long time                            #␣
↪needs sage.groups sage.plot
....:          edge_size=0.01, edge_size2=0.02,
....:          vertex_colors={(1,1,1): list(G)}, bgcolor=(0,0,0),
....:          color_by_label=True, iterations=200)
```

```
>>> from sage.all import *
>>> A5 = AlternatingGroup(Integer(5)); A5                                   ␣
```

(continues on next page)

(continued from previous page)

```
↪     # needs sage.groups
Alternating group of order 5!/2 as a permutation group
>>> G = A5.cayley_graph()                                                    #␣
↪needs sage.groups
>>> G.show3d(vertex_size=RealNumber('0.03'),     # long time                 ␣
↪          # needs sage.groups sage.plot
...           edge_size=RealNumber('0.01'), edge_size2=RealNumber('0.02'),
...           vertex_colors={(Integer(1),Integer(1),Integer(1)): list(G)},␣
↪bgcolor=(Integer(0),Integer(0),Integer(0)),
...           color_by_label=True, iterations=Integer(200))
```

Some `Tachyon` examples:

```
sage: D = graphs.DodecahedralGraph()
sage: D.show3d(engine='tachyon')     # long time                            #␣
↪needs sage.plot
```

```
>>> from sage.all import *
>>> D = graphs.DodecahedralGraph()
>>> D.show3d(engine='tachyon')     # long time                              #␣
↪needs sage.plot
```

```
sage: G = graphs.PetersenGraph()
sage: G.show3d(engine='tachyon',     # long time                            #␣
↪needs sage.plot
....:           vertex_colors={(0,0,1): list(G)})
```

```
>>> from sage.all import *
>>> G = graphs.PetersenGraph()
>>> G.show3d(engine='tachyon',     # long time                              #␣
↪needs sage.plot
...           vertex_colors={(Integer(0),Integer(0),Integer(1)): list(G)})
```

```
sage: C = graphs.CubeGraph(4)
sage: C.show3d(engine='tachyon',     # long time                            #␣
↪needs sage.plot
....:           edge_colors={(0,1,0): C.edges(sort=False)},
....:           vertex_colors={(1,1,1): list(C)}, bgcolor=(0,0,0))
```

```
>>> from sage.all import *
>>> C = graphs.CubeGraph(Integer(4))
>>> C.show3d(engine='tachyon',     # long time                              #␣
↪needs sage.plot
...           edge_colors={(Integer(0),Integer(1),Integer(0)): C.
↪edges(sort=False)},
...           vertex_colors={(Integer(1),Integer(1),Integer(1)): list(C)},␣
↪bgcolor=(Integer(0),Integer(0),Integer(0)))
```

```
sage: K = graphs.CompleteGraph(3)
sage: K.show3d(engine='tachyon',     # long time                            #␣
↪needs sage.plot
```

(continues on next page)

(continued from previous page)

```
....:           edge_colors={(1,0,0): [(0, 1, None)],
....:                        (0, 1, 0): [(0, 2, None)],
....:                        (0, 0, 1): [(1, 2, None)]})
```

```
>>> from sage.all import *
>>> K = graphs.CompleteGraph(Integer(3))
>>> K.show3d(engine='tachyon',     # long time                                #␣
↪needs sage.plot
...          edge_colors={(Integer(1),Integer(0),Integer(0)): [(Integer(0),␣
↪Integer(1), None)],
...                       (Integer(0), Integer(1), Integer(0)): [(Integer(0),␣
↪Integer(2), None)],
...                       (Integer(0), Integer(0), Integer(1)): [(Integer(1),␣
↪Integer(2), None)]})
```

size()

Return the number of edges.

Note that *n_edges()* also returns the number of edges in G.

EXAMPLES:

```
sage: G = graphs.PetersenGraph()
sage: G.size()
15
```

```
>>> from sage.all import *
>>> G = graphs.PetersenGraph()
>>> G.size()
15
```

spanning_trees_count(*root_vertex=None*)

alias of *number_of_spanning_trees()*.

spectral_radius(*G*, *prec=1e-10*)

Return an interval of floating point number that encloses the spectral radius of this graph

The input graph `G` must be *strongly connected*.

INPUT:

- `prec` – (default: `1e-10`) an upper bound for the relative precision of the interval

The algorithm is iterative and uses an inequality valid for nonnegative matrices. Namely, if A is a nonnegative square matrix with Perron-Frobenius eigenvalue λ then the following inequality is valid for any vector x

$$\min_i \frac{(Ax)_i}{x_i} \leq \lambda \leq \max_i \frac{(Ax)_i}{x_i}$$

Note

The speed of convergence of the algorithm is governed by the spectral gap (the distance to the second largest modulus of other eigenvalues). If this gap is small, then this function might not be appropriate.

The algorithm is not smart and not parallel! It uses basic interval arithmetic and native floating point arithmetic.

EXAMPLES:

```
sage: from sage.graphs.base.static_sparse_graph import spectral_radius

sage: G = DiGraph([(0,0),(0,1),(1,0)], loops=True)
sage: phi = (RR(1) + RR(5).sqrt() ) / 2
sage: phi  # abs tol 1e-14
1.618033988749895
sage: e_min, e_max = spectral_radius(G, 1e-14)
sage: e_min, e_max     # abs tol 1e-14
(1.618033988749894, 1.618033988749896)
sage: (e_max - e_min)  # abs tol 1e-14
1e-14
sage: e_min < phi < e_max
True
```

```
>>> from sage.all import *
>>> from sage.graphs.base.static_sparse_graph import spectral_radius

>>> G = DiGraph([(Integer(0),Integer(0)),(Integer(0),Integer(1)),(Integer(1),
↪Integer(0))], loops=True)
>>> phi = (RR(Integer(1)) + RR(Integer(5)).sqrt() ) / Integer(2)
>>> phi  # abs tol 1e-14
1.618033988749895
>>> e_min, e_max = spectral_radius(G, RealNumber('1e-14'))
>>> e_min, e_max     # abs tol 1e-14
(1.618033988749894, 1.618033988749896)
>>> (e_max - e_min)  # abs tol 1e-14
1e-14
>>> e_min < phi < e_max
True
```

This function also works for graphs:

```
sage: G = Graph([(0,1),(0,2),(1,2),(1,3),(2,4),(3,4)])
sage: e_min, e_max = spectral_radius(G, 1e-14)
sage: e = max(G.adjacency_matrix().charpoly().roots(AA,␣
↪multiplicities=False))  # needs sage.modules sage.rings.number_field
sage: e_min < e < e_max                                                    ␣
↪  # needs sage.modules sage.rings.number_field sage.symbolic
True

sage: G.spectral_radius()  # abs tol 1e-9
(2.48119430408, 2.4811943041)
```

```
>>> from sage.all import *
>>> G = Graph([(Integer(0),Integer(1)),(Integer(0),Integer(2)),(Integer(1),
↪Integer(2)),(Integer(1),Integer(3)),(Integer(2),Integer(4)),(Integer(3),
↪Integer(4))])
```

(continues on next page)

(continued from previous page)

```
>>> e_min, e_max = spectral_radius(G, RealNumber('1e-14'))
>>> e = max(G.adjacency_matrix().charpoly().roots(AA, multiplicities=False))
↪# needs sage.modules sage.rings.number_field
>>> e_min < e < e_max
↪# needs sage.modules sage.rings.number_field sage.symbolic
True

>>> G.spectral_radius()  # abs tol 1e-9
(2.48119430408, 2.4811943041)
```

A larger example:

```
sage: G = DiGraph()
sage: G.add_edges((i,i+1) for i in range(200))
sage: G.add_edge(200,0)
sage: G.add_edge(1,0)
sage: e_min, e_max = spectral_radius(G, 0.00001)
sage: p = G.adjacency_matrix(sparse=True).charpoly()
sage: p
x^201 - x^199 - 1
sage: r = p.roots(AA, multiplicities=False)[0]
↪  # needs sage.rings.number_field
sage: e_min < r < e_max
↪  # needs sage.rings.number_field
True
```

```
>>> from sage.all import *
>>> G = DiGraph()
>>> G.add_edges((i,i+Integer(1)) for i in range(Integer(200)))
>>> G.add_edge(Integer(200),Integer(0))
>>> G.add_edge(Integer(1),Integer(0))
>>> e_min, e_max = spectral_radius(G, RealNumber('0.00001'))
>>> p = G.adjacency_matrix(sparse=True).charpoly()
>>> p
x^201 - x^199 - 1
>>> r = p.roots(AA, multiplicities=False)[Integer(0)]
↪         # needs sage.rings.number_field
>>> e_min < r < e_max
↪# needs sage.rings.number_field
True
```

A much larger example:

```
sage: G = DiGraph(100000)
sage: r = list(range(100000))
sage: while not G.is_strongly_connected():
....:     shuffle(r)
....:     G.add_edges(enumerate(r), loops=False)
sage: spectral_radius(G, 1e-10)  # random  # long time
(1.9997956006500042, 1.9998043797692782)
```

```
>>> from sage.all import *
>>> G = DiGraph(Integer(100000))
>>> r = list(range(Integer(100000)))
>>> while not G.is_strongly_connected():
...     shuffle(r)
...     G.add_edges(enumerate(r), loops=False)
>>> spectral_radius(G, RealNumber('1e-10'))  # random  # long time
(1.9997956006500042, 1.9998043797692782)
```

The algorithm takes care of multiple edges:

```
sage: G = DiGraph(2,loops=True,multiedges=True)
sage: G.add_edges([(0,0),(0,0),(0,1),(1,0)])
sage: spectral_radius(G, 1e-14)  # abs tol 1e-14
(2.414213562373094, 2.414213562373095)
sage: max(G.adjacency_matrix().eigenvalues(AA))
↪  # needs sage.modules sage.rings.number_field
2.414213562373095?
```

```
>>> from sage.all import *
>>> G = DiGraph(Integer(2),loops=True,multiedges=True)
>>> G.add_edges([(Integer(0),Integer(0)),(Integer(0),Integer(0)),(Integer(0),
↪Integer(1)),(Integer(1),Integer(0))])
>>> spectral_radius(G, RealNumber('1e-14'))  # abs tol 1e-14
(2.414213562373094, 2.414213562373095)
>>> max(G.adjacency_matrix().eigenvalues(AA))
↪# needs sage.modules sage.rings.number_field
2.414213562373095?
```

Some bipartite graphs:

```
sage: G = Graph([(0,1),(0,3),(2,3)])
sage: G.spectral_radius()  # abs tol 1e-10
(1.6180339887253428, 1.6180339887592732)

sage: G = DiGraph([(0,1),(0,3),(2,3),(3,0),(1,0),(1,2)])
sage: G.spectral_radius() # abs tol 1e-10
(1.5537739740270458, 1.553773974033029)

sage: G = graphs.CompleteBipartiteGraph(1,3)
sage: G.spectral_radius()  # abs tol 1e-10
(1.7320508075688772, 1.7320508075688774)
```

```
>>> from sage.all import *
>>> G = Graph([(Integer(0),Integer(1)),(Integer(0),Integer(3)),(Integer(2),
↪Integer(3))])
>>> G.spectral_radius()  # abs tol 1e-10
(1.6180339887253428, 1.6180339887592732)

>>> G = DiGraph([(Integer(0),Integer(1)),(Integer(0),Integer(3)),(Integer(2),
↪Integer(3)),(Integer(3),Integer(0)),(Integer(1),Integer(0)),(Integer(1),
↪Integer(2))])
>>> G.spectral_radius() # abs tol 1e-10
```

(continues on next page)

(continued from previous page)

```
(1.5537739740270458, 1.553773974033029)

>>> G = graphs.CompleteBipartiteGraph(Integer(1),Integer(3))
>>> G.spectral_radius()  # abs tol 1e-10
(1.7320508075688772, 1.7320508075688774)
```

spectrum(*laplacian=False*)

Return a list of the eigenvalues of the adjacency matrix.

INPUT:

- `laplacian` – boolean (default: `False`); if `True`, use the Laplacian matrix (see *kirchhoff_matrix()*)

OUTPUT:

A list of the eigenvalues, including multiplicities, sorted with the largest eigenvalue first.

See also

The method *spectral_radius()* returns floating point approximation of the maximum eigenvalue.

EXAMPLES:

```
sage: P = graphs.PetersenGraph()
sage: P.spectrum()                                                          #␣
↪needs sage.modules sage.rings.number_field
[3, 1, 1, 1, 1, 1, -2, -2, -2, -2]
sage: P.spectrum(laplacian=True)                                            #␣
↪needs sage.modules sage.rings.number_field
[5, 5, 5, 5, 2, 2, 2, 2, 2, 0]
sage: D = P.to_directed()
sage: D.delete_edge(7, 9)
sage: D.spectrum()                                                          #␣
↪needs sage.modules sage.rings.number_field
[2.9032119259..., 1, 1, 1, 1, 0.8060634335..., -1.7092753594..., -2, -2, -2]
```

```
>>> from sage.all import *
>>> P = graphs.PetersenGraph()
>>> P.spectrum()                                                            #␣
↪needs sage.modules sage.rings.number_field
[3, 1, 1, 1, 1, 1, -2, -2, -2, -2]
>>> P.spectrum(laplacian=True)                                              #␣
↪needs sage.modules sage.rings.number_field
[5, 5, 5, 5, 2, 2, 2, 2, 2, 0]
>>> D = P.to_directed()
>>> D.delete_edge(Integer(7), Integer(9))
>>> D.spectrum()                                                            #␣
↪needs sage.modules sage.rings.number_field
[2.9032119259..., 1, 1, 1, 1, 0.8060634335..., -1.7092753594..., -2, -2, -2]
```

```
sage: C = graphs.CycleGraph(8)
sage: C.spectrum()                                                          #␣
↪needs sage.modules sage.rings.number_field
[2, 1.4142135623..., 1.4142135623..., 0, 0, -1.4142135623..., -1.4142135623...
↪, -2]
```

```
>>> from sage.all import *
>>> C = graphs.CycleGraph(Integer(8))
>>> C.spectrum()                                                            #␣
↪needs sage.modules sage.rings.number_field
[2, 1.4142135623..., 1.4142135623..., 0, 0, -1.4142135623..., -1.4142135623...
↪, -2]
```

A digraph may have complex eigenvalues. Previously, the complex parts of graph eigenvalues were being dropped. For a 3-cycle, we have:

```
sage: T = DiGraph({0: [1], 1: [2], 2: [0]})
sage: T.spectrum()                                                          #␣
↪needs sage.modules sage.rings.number_field
[1, -0.5000000000... + 0.8660254037...*I, -0.5000000000... - 0.8660254037...
↪*I]
```

```
>>> from sage.all import *
>>> T = DiGraph({Integer(0): [Integer(1)], Integer(1): [Integer(2)],␣
↪Integer(2): [Integer(0)]})
>>> T.spectrum()                                                            #␣
↪needs sage.modules sage.rings.number_field
[1, -0.5000000000... + 0.8660254037...*I, -0.5000000000... - 0.8660254037...
↪*I]
```

steiner_tree(*vertices*, *weighted*, *solver=False*, *verbose=None*, *integrality_tolerance=0*)

Return a tree of minimum weight connecting the given set of vertices.

Definition :

Computing a minimum spanning tree in a graph can be done in $n \log(n)$ time (and in linear time if all weights are equal) where $n = V + E$. On the other hand, if one is given a large (possibly weighted) graph and a subset of its vertices, it is NP-Hard to find a tree of minimum weight connecting the given set of vertices, which is then called a Steiner Tree.

See the Wikipedia article Steiner_tree_problem for more information.

INPUT:

- `vertices` – the vertices to be connected by the Steiner Tree
- `weighted` – boolean (default: `False`); whether to consider the graph as weighted, and use each edge's label as a weight, considering `None` as a weight of 1. If `weighted=False` (default) all edges are considered to have a weight of 1.
- `solver` – string (default: `None`); specifies a Mixed Integer Linear Programming (MILP) solver to be used. If set to `None`, the default one is used. For more information on MILP solvers and which default solver is used, see the method `solve` of the class `MixedIntegerLinearProgram`.
- `verbose` – integer (default: 0); sets the level of verbosity. Set to 0 by default, which means quiet.

- `integrality_tolerance` – float; parameter for use with MILP solvers over an inexact base ring; see `MixedIntegerLinearProgram.get_values()`.

> **Note**
>
> - This problem being defined on undirected graphs, the orientation is not considered if the current graph is actually a digraph.
> - The graph is assumed not to have multiple edges.

ALGORITHM:

Solved through Linear Programming.

COMPLEXITY:

NP-Hard.

Note that this algorithm first checks whether the given set of vertices induces a connected graph, returning one of its spanning trees if `weighted` is set to `False`, and thus answering very quickly in some cases

EXAMPLES:

The Steiner Tree of the first 5 vertices in a random graph is, of course, always a tree:

```
sage: g = graphs.RandomGNP(30, .5)
sage: first5 = g.vertices(sort=True)[:5]
sage: st = g.steiner_tree(first5)                                    #␣
↪needs sage.numerical.mip
sage: st.is_tree()                                                   #␣
↪needs sage.numerical.mip
True
```

```
>>> from sage.all import *
>>> g = graphs.RandomGNP(Integer(30), RealNumber('.5'))
>>> first5 = g.vertices(sort=True)[:Integer(5)]
>>> st = g.steiner_tree(first5)                                      #␣
↪needs sage.numerical.mip
>>> st.is_tree()                                                     #␣
↪needs sage.numerical.mip
True
```

And all the 5 vertices are contained in this tree

```
sage: all(v in st for v in first5)                                   #␣
↪needs sage.numerical.mip
True
```

```
>>> from sage.all import *
>>> all(v in st for v in first5)                                     #␣
↪needs sage.numerical.mip
True
```

An exception is raised when the problem is impossible, i.e. if the given vertices are not all included in the same connected component:

```
sage: g = 2 * graphs.PetersenGraph()
sage: st = g.steiner_tree([5, 15])
Traceback (most recent call last):
...
EmptySetError: the given vertices do not all belong to the same connected␣
↪component. This problem has no solution !
```

```
>>> from sage.all import *
>>> g = Integer(2) * graphs.PetersenGraph()
>>> st = g.steiner_tree([Integer(5), Integer(15)])
Traceback (most recent call last):
...
EmptySetError: the given vertices do not all belong to the same connected␣
↪component. This problem has no solution !
```

strong_product(*other*, *immutable=None*)

Return the strong product of `self` and `other`.

The strong product of G and H is the graph L with vertex set $V(L) = V(G) \times V(H)$, and $((u, v), (w, x))$ is an edge of L iff either :

- (u, w) is an edge of G and $v = x$, or
- (v, x) is an edge of H and $u = w$, or
- (u, w) is an edge of G and (v, x) is an edge of H.

In other words, the edges of the strong product is the union of the edges of the tensor and Cartesian products.

INPUT:

- `other` – a graph or a digraph
- `immutable` – boolean (default: `None`); whether to create a mutable/immutable product. `immutable=None` (default) means that the graphs and their product will behave the same way. If only one of them is immutable, the product will be mutable.

EXAMPLES:

```
sage: Z = graphs.CompleteGraph(2)
sage: C = graphs.CycleGraph(5)
sage: S = C.strong_product(Z); S
Graph on 10 vertices
sage: S.plot()                          # long time                          #␣
↪needs sage.plot
Graphics object consisting of 36 graphics primitives
```

```
>>> from sage.all import *
>>> Z = graphs.CompleteGraph(Integer(2))
>>> C = graphs.CycleGraph(Integer(5))
>>> S = C.strong_product(Z); S
Graph on 10 vertices
>>> S.plot()                          # long time                          #␣
↪needs sage.plot
Graphics object consisting of 36 graphics primitives
```

```
sage: D = graphs.DodecahedralGraph()
sage: P = graphs.PetersenGraph()
sage: S = D.strong_product(P); S
Graph on 200 vertices
sage: S.plot()                          # long time                          #␣
↪needs sage.plot
Graphics object consisting of 1701 graphics primitives
```

```
>>> from sage.all import *
>>> D = graphs.DodecahedralGraph()
>>> P = graphs.PetersenGraph()
>>> S = D.strong_product(P); S
Graph on 200 vertices
>>> S.plot()                          # long time                          #␣
↪needs sage.plot
Graphics object consisting of 1701 graphics primitives
```

subdivide_edge(**args*)

Subdivide an edge k times.

INPUT:

The following forms are all accepted to subdivide 8 times the edge between vertices 1 and 2 labeled with `'my_label'`.

- `G.subdivide_edge( 1, 2, 8 )`
- `G.subdivide_edge( (1, 2), 8 )`
- `G.subdivide_edge( (1, 2, "my_label"), 8 )`

Note

- If the given edge is labelled with l, all the edges created by the subdivision will have the same label
- If no label is given, the label used will be the one returned by the method *edge_label()* on the pair `u,v`

EXAMPLES:

Subdividing 5 times an edge in a path of length 3 makes it a path of length 8:

```
sage: g = graphs.PathGraph(3)
sage: edge = next(g.edge_iterator())
sage: g.subdivide_edge(edge, 5)
sage: g.is_isomorphic(graphs.PathGraph(8))
True
```

```
>>> from sage.all import *
>>> g = graphs.PathGraph(Integer(3))
>>> edge = next(g.edge_iterator())
>>> g.subdivide_edge(edge, Integer(5))
>>> g.is_isomorphic(graphs.PathGraph(Integer(8)))
True
```

Subdividing a labelled edge in two ways:

```
sage: g = Graph()
sage: g.add_edge(0, 1, "label1")
sage: g.add_edge(1, 2, "label2")
sage: print(g.edges(sort=True))
[(0, 1, 'label1'), (1, 2, 'label2')]
```

```
>>> from sage.all import *
>>> g = Graph()
>>> g.add_edge(Integer(0), Integer(1), "label1")
>>> g.add_edge(Integer(1), Integer(2), "label2")
>>> print(g.edges(sort=True))
[(0, 1, 'label1'), (1, 2, 'label2')]
```

Specifying the label:

```
sage: g.subdivide_edge(0, 1, "label1", 3)
sage: print(g.edges(sort=True))
[(0, 3, 'label1'), (1, 2, 'label2'), (1, 5, 'label1'), (3, 4, 'label1'), (4,␣
↪5, 'label1')]
```

```
>>> from sage.all import *
>>> g.subdivide_edge(Integer(0), Integer(1), "label1", Integer(3))
>>> print(g.edges(sort=True))
[(0, 3, 'label1'), (1, 2, 'label2'), (1, 5, 'label1'), (3, 4, 'label1'), (4,␣
↪5, 'label1')]
```

The lazy way:

```
sage: g.subdivide_edge(1, 2, "label2", 5)
sage: print(g.edges(sort=True))
[(0, 3, 'label1'), (1, 5, 'label1'), (1, 6, 'label2'),
 (2, 10, 'label2'), (3, 4, 'label1'), (4, 5, 'label1'),
 (6, 7, 'label2'), (7, 8, 'label2'), (8, 9, 'label2'),
 (9, 10, 'label2')]
```

```
>>> from sage.all import *
>>> g.subdivide_edge(Integer(1), Integer(2), "label2", Integer(5))
>>> print(g.edges(sort=True))
[(0, 3, 'label1'), (1, 5, 'label1'), (1, 6, 'label2'),
 (2, 10, 'label2'), (3, 4, 'label1'), (4, 5, 'label1'),
 (6, 7, 'label2'), (7, 8, 'label2'), (8, 9, 'label2'),
 (9, 10, 'label2')]
```

If too many arguments are given, an exception is raised

```
sage: g.subdivide_edge(0,1,1,1,1,1,1,1,1,1,1)
Traceback (most recent call last):
...
ValueError: this method takes at most 4 arguments
```

```
>>> from sage.all import *
>>> g.subdivide_edge(Integer(0),Integer(1),Integer(1),Integer(1),Integer(1),
↪Integer(1),Integer(1),Integer(1),Integer(1),Integer(1),Integer(1))
Traceback (most recent call last):
...
ValueError: this method takes at most 4 arguments
```

The same goes when the given edge does not exist:

```
sage: g.subdivide_edge(0, 1, "fake_label", 5)
Traceback (most recent call last):
...
ValueError: the given edge does not exist
```

```
>>> from sage.all import *
>>> g.subdivide_edge(Integer(0), Integer(1), "fake_label", Integer(5))
Traceback (most recent call last):
...
ValueError: the given edge does not exist
```

See also

- *subdivide_edges()* – subdivides multiples edges at a time

subdivide_edges(*edges*, *k*)

Subdivide k times edges from an iterable container.

For more information on the behaviour of this method, please refer to the documentation of *subdivide_edge()*.

INPUT:

- `edges` – list of edges
- `k` – integer; common length of the subdivisions

Note

If a given edge is labelled with l, all the edges created by its subdivision will have the same label.

EXAMPLES:

If we are given the disjoint union of several paths:

```
sage: paths = [2, 5, 9]
sage: paths = map(graphs.PathGraph, paths)
sage: g = Graph()
sage: for P in paths:
....:     g = g + P
```

```
>>> from sage.all import *
>>> paths = [Integer(2), Integer(5), Integer(9)]
```

(continues on next page)

(continued from previous page)

```
>>> paths = map(graphs.PathGraph, paths)
>>> g = Graph()
>>> for P in paths:
...   g = g + P
```

Subdividing edges in each of them will only change their lengths:

```
sage: edges = [next(P.edge_iterator()) for P in g.connected_components_
↪subgraphs()]
sage: k = 6
sage: g.subdivide_edges(edges, k)
```

```
>>> from sage.all import *
>>> edges = [next(P.edge_iterator()) for P in g.connected_components_
↪subgraphs()]
>>> k = Integer(6)
>>> g.subdivide_edges(edges, k)
```

Let us check this by creating the graph we expect to have built through subdivision:

```
sage: paths2 = [2 + k, 5 + k, 9 + k]
sage: paths2 = map(graphs.PathGraph, paths2)
sage: g2 = Graph()
sage: for P in paths2:
....:   g2 = g2 + P
sage: g.is_isomorphic(g2)
True
```

```
>>> from sage.all import *
>>> paths2 = [Integer(2) + k, Integer(5) + k, Integer(9) + k]
>>> paths2 = map(graphs.PathGraph, paths2)
>>> g2 = Graph()
>>> for P in paths2:
...   g2 = g2 + P
>>> g.is_isomorphic(g2)
True
```

See also

- *subdivide_edge()* – subdivides one edge

subgraph(*vertices=None*, *edges=None*, *inplace=False*, *vertex_property=None*, *edge_property=None*, *algorithm=None*, *immutable=None*)

Return the subgraph containing the given vertices and edges.

If either vertices or edges are not specified, they are assumed to be all vertices or edges. If edges are not specified, returns the subgraph induced by the vertices.

INPUT:

- `inplace` – boolean (default: `False`); using `inplace=True` will simply delete the extra vertices and edges from the current graph. This will modify the graph.

- `vertices` – a single vertex or an iterable container of vertices, e.g. a list, set, graph, file or numeric array. If not passed (i.e., `None`), defaults to the entire graph.
- `edges` – as with `vertices`, edges can be a single edge or an iterable container of edges (e.g., a list, set, file, numeric array, etc.). By default (`edges=None`), all edges are assumed and the returned graph is an induced subgraph. In the case of multiple edges, specifying an edge as (u, v) means to keep all edges (u, v), regardless of the label.
- `vertex_property` – function (default: `None`); a function that inputs a vertex and outputs a boolean value, i.e., a vertex `v` in `vertices` is kept if `vertex_property(v) == True`
- `edge_property` – function (default: `None`); a function that inputs an edge and outputs a boolean value, i.e., a edge `e` in `edges` is kept if `edge_property(e) == True`
- `algorithm` – string (default: `None`); one of the following:
  - If `algorithm="delete"` or `inplace=True`, then the graph is constructed by deleting edges and vertices
  - If `algorithm="add"`, then the graph is constructed by building a new graph from the appropriate vertices and edges. Implies `inplace=False`.
  - If `algorithm=None`, then the algorithm is chosen based on the number of vertices in the subgraph.
- `immutable` – boolean (default: `None`); whether to create a mutable/immutable subgraph. `immutable=None` (default) means that the graph and its subgraph will behave the same way.

EXAMPLES:

```
sage: G = graphs.CompleteGraph(9)
sage: H = G.subgraph([0, 1, 2]); H
Subgraph of (Complete graph): Graph on 3 vertices
sage: G
Complete graph: Graph on 9 vertices
sage: J = G.subgraph(edges=[(0, 1)])
sage: J.edges(sort=True, labels=False)
[(0, 1)]
sage: set(J) == set(G)
True
sage: G.subgraph([0, 1, 2], inplace=True); G
Subgraph of (Complete graph): Graph on 3 vertices
sage: G.subgraph() == G
True
```

```
>>> from sage.all import *
>>> G = graphs.CompleteGraph(Integer(9))
>>> H = G.subgraph([Integer(0), Integer(1), Integer(2)]); H
Subgraph of (Complete graph): Graph on 3 vertices
>>> G
Complete graph: Graph on 9 vertices
>>> J = G.subgraph(edges=[(Integer(0), Integer(1))])
>>> J.edges(sort=True, labels=False)
[(0, 1)]
>>> set(J) == set(G)
True
>>> G.subgraph([Integer(0), Integer(1), Integer(2)], inplace=True); G
Subgraph of (Complete graph): Graph on 3 vertices
```

(continues on next page)

(continued from previous page)

```
>>> G.subgraph() == G
True
```

```
sage: D = digraphs.Complete(9)
sage: H = D.subgraph([0, 1, 2]); H
Subgraph of (Complete digraph): Digraph on 3 vertices
sage: H = D.subgraph(edges=[(0, 1), (0, 2)])
sage: H.edges(sort=True, labels=False)
[(0, 1), (0, 2)]
sage: set(H) == set(D)
True
sage: D
Complete digraph: Digraph on 9 vertices
sage: D.subgraph([0, 1, 2], inplace=True); D
Subgraph of (Complete digraph): Digraph on 3 vertices
sage: D.subgraph() == D
True
```

```
>>> from sage.all import *
>>> D = digraphs.Complete(Integer(9))
>>> H = D.subgraph([Integer(0), Integer(1), Integer(2)]); H
Subgraph of (Complete digraph): Digraph on 3 vertices
>>> H = D.subgraph(edges=[(Integer(0), Integer(1)), (Integer(0), Integer(2))])
>>> H.edges(sort=True, labels=False)
[(0, 1), (0, 2)]
>>> set(H) == set(D)
True
>>> D
Complete digraph: Digraph on 9 vertices
>>> D.subgraph([Integer(0), Integer(1), Integer(2)], inplace=True); D
Subgraph of (Complete digraph): Digraph on 3 vertices
>>> D.subgraph() == D
True
```

A more complicated example involving multiple edges and labels:

```
sage: G = Graph(multiedges=True, sparse=True)
sage: G.add_edges([(0, 1, 'a'), (0, 1, 'b'), (1, 0, 'c'), (0, 2, 'd'), (0, 2,
↪'e'), (2, 0, 'f'), (1, 2, 'g')])
sage: G.subgraph(edges=[(0, 1), (0, 2,'d'), (0, 2, 'not in graph')]).
↪edges(sort=True)
[(0, 1, 'a'), (0, 1, 'b'), (0, 1, 'c'), (0, 2, 'd')]
sage: J = G.subgraph(vertices=[0, 1], edges=[(0, 1, 'a'), (0, 2, 'c')])
sage: J.edges(sort=True)
[(0, 1, 'a')]
sage: J.vertices(sort=True)
[0, 1]
sage: G.subgraph(vertices=G) == G
True
```

```
>>> from sage.all import *
```

(continues on next page)

(continued from previous page)

```
>>> G = Graph(multiedges=True, sparse=True)
>>> G.add_edges([(Integer(0), Integer(1), 'a'), (Integer(0), Integer(1), 'b'),
↪ (Integer(1), Integer(0), 'c'), (Integer(0), Integer(2), 'd'), (Integer(0),␣
↪Integer(2), 'e'), (Integer(2), Integer(0), 'f'), (Integer(1), Integer(2), 'g
↪')])
>>> G.subgraph(edges=[(Integer(0), Integer(1)), (Integer(0), Integer(2),'d'),␣
↪(Integer(0), Integer(2), 'not in graph')]).edges(sort=True)
[(0, 1, 'a'), (0, 1, 'b'), (0, 1, 'c'), (0, 2, 'd')]
>>> J = G.subgraph(vertices=[Integer(0), Integer(1)], edges=[(Integer(0),␣
↪Integer(1), 'a'), (Integer(0), Integer(2), 'c')])
>>> J.edges(sort=True)
[(0, 1, 'a')]
>>> J.vertices(sort=True)
[0, 1]
>>> G.subgraph(vertices=G) == G
True
```

```
sage: D = DiGraph(multiedges=True, sparse=True)
sage: D.add_edges([(0, 1, 'a'), (0, 1, 'b'), (1, 0, 'c'), (0, 2, 'd'), (0, 2,
↪'e'), (2, 0, 'f'), (1, 2, 'g')])
sage: D.subgraph(edges=[(0, 1), (0, 2, 'd'), (0, 2, 'not in graph')]).
↪edges(sort=True)
[(0, 1, 'a'), (0, 1, 'b'), (0, 2, 'd')]
sage: H = D.subgraph(vertices=[0, 1], edges=[(0, 1, 'a'), (0, 2, 'c')])
sage: H.edges(sort=True)
[(0, 1, 'a')]
sage: H.vertices(sort=True)
[0, 1]
```

```
>>> from sage.all import *
>>> D = DiGraph(multiedges=True, sparse=True)
>>> D.add_edges([(Integer(0), Integer(1), 'a'), (Integer(0), Integer(1), 'b'),
↪ (Integer(1), Integer(0), 'c'), (Integer(0), Integer(2), 'd'), (Integer(0),␣
↪Integer(2), 'e'), (Integer(2), Integer(0), 'f'), (Integer(1), Integer(2), 'g
↪')])
>>> D.subgraph(edges=[(Integer(0), Integer(1)), (Integer(0), Integer(2), 'd'),
↪ (Integer(0), Integer(2), 'not in graph')]).edges(sort=True)
[(0, 1, 'a'), (0, 1, 'b'), (0, 2, 'd')]
>>> H = D.subgraph(vertices=[Integer(0), Integer(1)], edges=[(Integer(0),␣
↪Integer(1), 'a'), (Integer(0), Integer(2), 'c')])
>>> H.edges(sort=True)
[(0, 1, 'a')]
>>> H.vertices(sort=True)
[0, 1]
```

Using the property arguments:

```
sage: P = graphs.PetersenGraph()
sage: S = P.subgraph(vertex_property=lambda v: not (v % 2))
sage: S.vertices(sort=True)
[0, 2, 4, 6, 8]
```

```
>>> from sage.all import *
>>> P = graphs.PetersenGraph()
>>> S = P.subgraph(vertex_property=lambda v: not (v % Integer(2)))
>>> S.vertices(sort=True)
[0, 2, 4, 6, 8]
```

```
sage: C = graphs.CubeGraph(2)
sage: S = C.subgraph(edge_property=(lambda e: e[0][0] == e[1][0]))
sage: C.edges(sort=True)
[('00', '01', None), ('00', '10', None), ('01', '11', None), ('10', '11',␣
↪None)]
sage: S.edges(sort=True)
[('00', '01', None), ('10', '11', None)]
```

```
>>> from sage.all import *
>>> C = graphs.CubeGraph(Integer(2))
>>> S = C.subgraph(edge_property=(lambda e: e[Integer(0)][Integer(0)] ==␣
↪e[Integer(1)][Integer(0)]))
>>> C.edges(sort=True)
[('00', '01', None), ('00', '10', None), ('01', '11', None), ('10', '11',␣
↪None)]
>>> S.edges(sort=True)
[('00', '01', None), ('10', '11', None)]
```

The algorithm is not specified, then a reasonable choice is made for speed:

```
sage: g = graphs.PathGraph(1000)
sage: g.subgraph(list(range(10)))  # uses the 'add' algorithm
Subgraph of (Path graph): Graph on 10 vertices
```

```
>>> from sage.all import *
>>> g = graphs.PathGraph(Integer(1000))
>>> g.subgraph(list(range(Integer(10))))  # uses the 'add' algorithm
Subgraph of (Path graph): Graph on 10 vertices
```

The vertices and edges can be specified using generator expressions (see Issue #41130):

```
sage: g = graphs.CompleteGraph(5)
sage: h = g.subgraph(vertices=(v for v in range(4)), edges=((0, v) for v in␣
↪range(5)))
sage: h.edges(labels=False)
[(0, 1), (0, 2), (0, 3)]
```

```
>>> from sage.all import *
>>> g = graphs.CompleteGraph(Integer(5))
>>> h = g.subgraph(vertices=(v for v in range(Integer(4))),␣
↪edges=((Integer(0), v) for v in range(Integer(5))))
>>> h.edges(labels=False)
[(0, 1), (0, 2), (0, 3)]
```

subgraph_decompositions(*H*, *induced=False*)

Return an iterator over the H-decompositions of a graph.

For a graph G, we say a collection of graphs $H_1, \ldots, H_m$ is a *decomposition* of G, if G is an edge-disjoint union of $H_1, \ldots, H_m$. See arXiv 2308.11613.

For graphs G and H, an H*-decomposition* of G is a partition of the edges of G into subgraphs isomorphic to H. See arXiv 1401.3665.

INPUT:

- `H` – the graph whose copies we are looking for in `self`
- `induced` – boolean (default: `False`); whether or not to consider only the isometric copies of `H` which are induced subgraphs of the graph `self`

OUTPUT:

An iterator of lists of lists of edges

EXAMPLES:

```
sage: G1 = Graph( [(0, 1), (0, 2), (0, 3), (0, 4), (1, 2), (1, 3),
....: (1, 5), (2, 3), (2, 4), (3, 5), (4, 6), (4, 7), (5, 6), (5, 7), (6, 8),
....: (6, 10), (7, 9), (7, 11), (8, 9), (8, 10), (9, 11), (10, 11)])
sage: claw = graphs.ClawGraph()
sage: list(G1.subgraph_decompositions(claw))
[]
```

```
>>> from sage.all import *
>>> G1 = Graph( [(Integer(0), Integer(1)), (Integer(0), Integer(2)),␣
↪(Integer(0), Integer(3)), (Integer(0), Integer(4)), (Integer(1),␣
↪Integer(2)), (Integer(1), Integer(3)),
... (Integer(1), Integer(5)), (Integer(2), Integer(3)), (Integer(2),␣
↪Integer(4)), (Integer(3), Integer(5)), (Integer(4), Integer(6)),␣
↪(Integer(4), Integer(7)), (Integer(5), Integer(6)), (Integer(5),␣
↪Integer(7)), (Integer(6), Integer(8)),
... (Integer(6), Integer(10)), (Integer(7), Integer(9)), (Integer(7),␣
↪Integer(11)), (Integer(8), Integer(9)), (Integer(8), Integer(10)),␣
↪(Integer(9), Integer(11)), (Integer(10), Integer(11))])
>>> claw = graphs.ClawGraph()
>>> list(G1.subgraph_decompositions(claw))
[]
```

```
sage: G2 = Graph([(0, 1), (0, 2), (0, 3), (0, 4), (1, 2), (1, 3), (1, 4),
....:     (1, 5), (2, 4), (2, 5), (3, 5), (4, 5)])
sage: it = G2.subgraph_decompositions(claw)
sage: next(it)                              # random
[[(0, 1), (0, 2), (0, 3)],
 [(0, 4), (1, 4), (2, 4)],
 [(1, 2), (1, 3), (1, 5)],
 [(2, 5), (3, 5), (4, 5)]]
```

```
>>> from sage.all import *
>>> G2 = Graph([(Integer(0), Integer(1)), (Integer(0), Integer(2)),␣
↪(Integer(0), Integer(3)), (Integer(0), Integer(4)), (Integer(1),␣
↪Integer(2)), (Integer(1), Integer(3)), (Integer(1), Integer(4)),
...     (Integer(1), Integer(5)), (Integer(2), Integer(4)), (Integer(2),␣
↪Integer(5)), (Integer(3), Integer(5)), (Integer(4), Integer(5))])
```

(continues on next page)

(continued from previous page)

```
>>> it = G2.subgraph_decompositions(claw)
>>> next(it)                                # random
[[(0, 1), (0, 2), (0, 3)],
 [(0, 4), (1, 4), (2, 4)],
 [(1, 2), (1, 3), (1, 5)],
 [(2, 5), (3, 5), (4, 5)]]
```

It has no claw-decomposition if we restrict to claws that are induced subraphs:

```
sage: list(G2.subgraph_decompositions(claw, induced=True))
[]
```

```
>>> from sage.all import *
>>> list(G2.subgraph_decompositions(claw, induced=True))
[]
```

It has no claw-decomposition if the number of edges of the graph is not a multiple of the number of edges of the claw (3):

```
sage: G = Graph([(0, 1), (0, 2), (0, 3), (0, 4)])
sage: list(G.subgraph_decompositions(claw))
[]
```

```
>>> from sage.all import *
>>> G = Graph([(Integer(0), Integer(1)), (Integer(0), Integer(2)),␣
↪(Integer(0), Integer(3)), (Integer(0), Integer(4))])
>>> list(G.subgraph_decompositions(claw))
[]
```

Works for digraphs:

```
sage: G = DiGraph([(0,1), (1,2), (2,3), (3,4)], format='list_of_edges')
sage: H = DiGraph([(0,1), (1,2)], format='list_of_edges')
sage: sorted(sorted(edges) for edges in G.subgraph_decompositions(H))
[[[(0, 1), (1, 2)], [(2, 3), (3, 4)]]]
```

```
>>> from sage.all import *
>>> G = DiGraph([(Integer(0),Integer(1)), (Integer(1),Integer(2)),␣
↪(Integer(2),Integer(3)), (Integer(3),Integer(4))], format='list_of_edges')
>>> H = DiGraph([(Integer(0),Integer(1)), (Integer(1),Integer(2))], format=
↪'list_of_edges')
>>> sorted(sorted(edges) for edges in G.subgraph_decompositions(H))
[[[(0, 1), (1, 2)], [(2, 3), (3, 4)]]]
```

```
sage: G = DiGraph([(0,1), (1,2), (2,3), (4,3)], format='list_of_edges')
sage: H = DiGraph([(0,1), (1,2)], format='list_of_edges')
sage: sorted(sorted(edges) for edges in G.subgraph_decompositions(H))
[]
```

```
>>> from sage.all import *
>>> G = DiGraph([(Integer(0),Integer(1)), (Integer(1),Integer(2)),␣
```

(continues on next page)

(continued from previous page)

```
↪(Integer(2),Integer(3)), (Integer(4),Integer(3))], format='list_of_edges')
>>> H = DiGraph([(Integer(0),Integer(1)), (Integer(1),Integer(2))], format=
↪'list_of_edges')
>>> sorted(sorted(edges) for edges in G.subgraph_decompositions(H))
[]
```

```
sage: G = DiGraph([(0,1), (1,0), (1,2), (2,1)], format='list_of_edges')
sage: H = DiGraph([(0,1), (1,2)], format='list_of_edges')
sage: sorted(sorted(edges) for edges in G.subgraph_decompositions(H))
[[[(0, 1), (1, 2)], [(1, 0), (2, 1)]]]
```

```
>>> from sage.all import *
>>> G = DiGraph([(Integer(0),Integer(1)), (Integer(1),Integer(0)),␣
↪(Integer(1),Integer(2)), (Integer(2),Integer(1))], format='list_of_edges')
>>> H = DiGraph([(Integer(0),Integer(1)), (Integer(1),Integer(2))], format=
↪'list_of_edges')
>>> sorted(sorted(edges) for edges in G.subgraph_decompositions(H))
[[[(0, 1), (1, 2)], [(1, 0), (2, 1)]]]
```

subgraph_search(*G*, *induced=False*)

Return a copy of `G` in `self`.

INPUT:

- `G` – the (di)graph whose copy we are looking for in `self`
- `induced` – boolean (default: `False`); whether or not to search for an induced copy of `G` in `self`

OUTPUT:

If `induced=False`, return a copy of `G` in this graph. Otherwise, return an induced copy of `G` in `self`. If `G` is the empty graph, return the empty graph since it is a subgraph of every graph. Now suppose `G` is not the empty graph. If there is no copy (induced or otherwise) of `G` in `self`, we return `None`.

Note

The vertex labels and the edge labels in the graph are ignored.

See also

- *subgraph_search_count()* – counts the number of copies of H inside of G
- *subgraph_search_iterator()* – iterator over the copies of H inside of G

ALGORITHM:

See the documentation of *SubgraphSearch*.

EXAMPLES:

The Petersen graph contains the path graph P_5:

```
sage: g = graphs.PetersenGraph()
sage: h1 = g.subgraph_search(graphs.PathGraph(5)); h1                          #␣
↪needs sage.modules
Subgraph of (Petersen graph): Graph on 5 vertices
sage: h1.vertices(sort=True); h1.edges(sort=True, labels=False)                #␣
↪needs sage.modules
[0, 1, 2, 3, 4]
[(0, 1), (1, 2), (2, 3), (3, 4)]
sage: I1 = g.subgraph_search(graphs.PathGraph(5), induced=True); I1            #␣
↪needs sage.modules
Subgraph of (Petersen graph): Graph on 5 vertices
sage: I1.vertices(sort=True); I1.edges(sort=True, labels=False)                #␣
↪needs sage.modules
[0, 1, 2, 3, 8]
[(0, 1), (1, 2), (2, 3), (3, 8)]
```

```
>>> from sage.all import *
>>> g = graphs.PetersenGraph()
>>> h1 = g.subgraph_search(graphs.PathGraph(Integer(5))); h1                   ␣
↪    # needs sage.modules
Subgraph of (Petersen graph): Graph on 5 vertices
>>> h1.vertices(sort=True); h1.edges(sort=True, labels=False)                  #␣
↪needs sage.modules
[0, 1, 2, 3, 4]
[(0, 1), (1, 2), (2, 3), (3, 4)]
>>> I1 = g.subgraph_search(graphs.PathGraph(Integer(5)), induced=True); I1    ␣
↪    # needs sage.modules
Subgraph of (Petersen graph): Graph on 5 vertices
>>> I1.vertices(sort=True); I1.edges(sort=True, labels=False)                  #␣
↪needs sage.modules
[0, 1, 2, 3, 8]
[(0, 1), (1, 2), (2, 3), (3, 8)]
```

It also contains the claw $K_{1,3}$:

```
sage: h2 = g.subgraph_search(graphs.ClawGraph()); h2
Subgraph of (Petersen graph): Graph on 4 vertices
sage: h2.vertices(sort=True); h2.edges(sort=True, labels=False)
[0, 1, 4, 5]
[(0, 1), (0, 4), (0, 5)]
sage: I2 = g.subgraph_search(graphs.ClawGraph(), induced=True); I2
Subgraph of (Petersen graph): Graph on 4 vertices
sage: I2.vertices(sort=True); I2.edges(sort=True, labels=False)
[0, 1, 4, 5]
[(0, 1), (0, 4), (0, 5)]
```

```
>>> from sage.all import *
>>> h2 = g.subgraph_search(graphs.ClawGraph()); h2
Subgraph of (Petersen graph): Graph on 4 vertices
>>> h2.vertices(sort=True); h2.edges(sort=True, labels=False)
[0, 1, 4, 5]
[(0, 1), (0, 4), (0, 5)]
```

(continues on next page)

(continued from previous page)

```
>>> I2 = g.subgraph_search(graphs.ClawGraph(), induced=True); I2
Subgraph of (Petersen graph): Graph on 4 vertices
>>> I2.vertices(sort=True); I2.edges(sort=True, labels=False)
[0, 1, 4, 5]
[(0, 1), (0, 4), (0, 5)]
```

Of course the induced copies are isomorphic to the graphs we were looking for:

```
sage: I1.is_isomorphic(graphs.PathGraph(5))                                    #␣
↪needs sage.modules
True
sage: I2.is_isomorphic(graphs.ClawGraph())                                     #␣
↪needs sage.modules
True
```

```
>>> from sage.all import *
>>> I1.is_isomorphic(graphs.PathGraph(Integer(5)))                             ␣
↪    # needs sage.modules
True
>>> I2.is_isomorphic(graphs.ClawGraph())                                      #␣
↪needs sage.modules
True
```

However, the Petersen graph does not contain a subgraph isomorphic to K_3:

```
sage: g.subgraph_search(graphs.CompleteGraph(3)) is None                       #␣
↪needs sage.modules
True
```

```
>>> from sage.all import *
>>> g.subgraph_search(graphs.CompleteGraph(Integer(3))) is None               ␣
↪    # needs sage.modules
True
```

Nor does it contain a nonempty induced subgraph isomorphic to P_6:

```
sage: g.subgraph_search(graphs.PathGraph(6), induced=True) is None             #␣
↪needs sage.modules
True
```

```
>>> from sage.all import *
>>> g.subgraph_search(graphs.PathGraph(Integer(6)), induced=True) is None     ␣
↪    # needs sage.modules
True
```

The empty graph is a subgraph of every graph:

```
sage: g.subgraph_search(graphs.EmptyGraph())                                   #␣
↪needs sage.modules
Graph on 0 vertices
sage: g.subgraph_search(graphs.EmptyGraph(), induced=True)                     #␣
```

(continues on next page)

(continued from previous page)

```
↪needs sage.modules
Graph on 0 vertices
```

```
>>> from sage.all import *
>>> g.subgraph_search(graphs.EmptyGraph())                                    #␣
↪needs sage.modules
Graph on 0 vertices
>>> g.subgraph_search(graphs.EmptyGraph(), induced=True)                      #␣
↪needs sage.modules
Graph on 0 vertices
```

The subgraph may just have edges missing:

```
sage: k3 = graphs.CompleteGraph(3); p3 = graphs.PathGraph(3)
sage: k3.relabel(list('abc'))
sage: s = k3.subgraph_search(p3)                                              #␣
↪needs sage.modules
sage: s.edges(sort=True, labels=False)                                        #␣
↪needs sage.modules
[('a', 'b'), ('b', 'c')]
```

```
>>> from sage.all import *
>>> k3 = graphs.CompleteGraph(Integer(3)); p3 = graphs.PathGraph(Integer(3))
>>> k3.relabel(list('abc'))
>>> s = k3.subgraph_search(p3)                                                #␣
↪needs sage.modules
>>> s.edges(sort=True, labels=False)                                          #␣
↪needs sage.modules
[('a', 'b'), ('b', 'c')]
```

Of course, P_3 is not an induced subgraph of K_3, though:

```
sage: k3 = graphs.CompleteGraph(3); p3 = graphs.PathGraph(3)
sage: k3.relabel(list('abc'))
sage: k3.subgraph_search(p3, induced=True) is None                            #␣
↪needs sage.modules
True
```

```
>>> from sage.all import *
>>> k3 = graphs.CompleteGraph(Integer(3)); p3 = graphs.PathGraph(Integer(3))
>>> k3.relabel(list('abc'))
>>> k3.subgraph_search(p3, induced=True) is None                              #␣
↪needs sage.modules
True
```

If the graph has labels, the labels are just ignored:

```
sage: g.set_vertex(0, 'foo')
sage: c = g.subgraph_search(graphs.PathGraph(5))                              #␣
↪needs sage.modules
sage: c.get_vertices()                                                        #␣
```

(continues on next page)

(continued from previous page)

```
↪needs sage.modules
{0: 'foo', 1: None, 2: None, 3: None, 4: None}
```

```
>>> from sage.all import *
>>> g.set_vertex(Integer(0), 'foo')
>>> c = g.subgraph_search(graphs.PathGraph(Integer(5)))
↪     # needs sage.modules
>>> c.get_vertices()
↪needs sage.modules
{0: 'foo', 1: None, 2: None, 3: None, 4: None}
```

subgraph_search_count(*G*, *induced=False*)

Return the number of labelled occurrences of `G` in `self`.

INPUT:

- `G` – the (di)graph whose copies we are looking for in `self`
- `induced` – boolean (default: `False`); whether or not to count induced copies of `G` in `self`

Note

The vertex labels and the edge labels in the graph are ignored.

ALGORITHM:

See the documentation of *SubgraphSearch*.

See also

- *subgraph_search()* – finds a subgraph isomorphic to H inside of a graph G
- *subgraph_search_iterator()* – iterator over the copies of a graph H inside of a graph G

EXAMPLES:

Counting the number of paths P_5 in a PetersenGraph:

```
sage: g = graphs.PetersenGraph()
sage: g.subgraph_search_count(graphs.PathGraph(5))
↪needs sage.modules
240
```

```
>>> from sage.all import *
>>> g = graphs.PetersenGraph()
>>> g.subgraph_search_count(graphs.PathGraph(Integer(5)))
↪     # needs sage.modules
240
```

Requiring these subgraphs be induced:

```
sage: g.subgraph_search_count(graphs.PathGraph(5), induced=True)                #␣
↪needs sage.modules
120
```

```
>>> from sage.all import *
>>> g.subgraph_search_count(graphs.PathGraph(Integer(5)), induced=True)       ␣
↪     # needs sage.modules
120
```

If we define the graph T_k (the transitive tournament on k vertices) as the graph on $\{0, ..., k-1\}$ such that $ij \in T_k$ if $i < j$, how many directed triangles can be found in T_5 ? The answer is of course 0:

```
sage: T5 = digraphs.TransitiveTournament(5)
sage: T5.subgraph_search_count(digraphs.Circuit(3))                             #␣
↪needs sage.modules
0
```

```
>>> from sage.all import *
>>> T5 = digraphs.TransitiveTournament(Integer(5))
>>> T5.subgraph_search_count(digraphs.Circuit(Integer(3)))                    ␣
↪    # needs sage.modules
0
```

If we count instead the number of T_3 in T_5, we expect the answer to be $\binom{5}{3}$:

```
sage: T3 = digraphs.TransitiveTournament(3)
sage: T5.subgraph_search_count(T3)                                              #␣
↪needs sage.modules
10
sage: binomial(5,3)                                                             #␣
↪needs sage.symbolic
10
sage: T3.is_isomorphic(T5.subgraph(vertices=[0, 1, 2]))                        #␣
↪needs sage.modules
True
```

```
>>> from sage.all import *
>>> T3 = digraphs.TransitiveTournament(Integer(3))
>>> T5.subgraph_search_count(T3)                                              #␣
↪needs sage.modules
10
>>> binomial(Integer(5),Integer(3))                                             ␣
↪             # needs sage.symbolic
10
>>> T3.is_isomorphic(T5.subgraph(vertices=[Integer(0), Integer(1),␣
↪Integer(2)]))                    # needs sage.modules
True
```

The empty graph is a subgraph of every graph:

```
sage: g.subgraph_search_count(graphs.EmptyGraph())                              #␣
↪needs sage.modules
1
```

```
>>> from sage.all import *
>>> g.subgraph_search_count(graphs.EmptyGraph())                    #␣
↪needs sage.modules
1
```

If the graph has vertex labels or edge labels, the label is just ignored:

```
sage: g.set_vertex(0, 'foo')
sage: g.subgraph_search_count(graphs.PathGraph(5))                  #␣
↪needs sage.modules
240
```

```
>>> from sage.all import *
>>> g.set_vertex(Integer(0), 'foo')
>>> g.subgraph_search_count(graphs.PathGraph(Integer(5)))                ␣
↪     # needs sage.modules
240
```

subgraph_search_iterator(*G*, *induced=False*, *return_graphs=True*)

Return an iterator over the labelled copies of `G` in `self`.

INPUT:

- `G` – the graph whose copies we are looking for in `self`
- `induced` – boolean (default: `False`); whether or not to iterate over the induced copies of `G` in `self`
- `return_graphs` – boolean (default: `True`); whether to return (di)graphs or only the list of vertices of the found (di)graphs

Note

The vertex labels and the edge labels in the graph are ignored.

ALGORITHM:

See the documentation of *SubgraphSearch*.

OUTPUT:

Iterator over the labelled copies of `G` in `self`, as *lists* or *(di)graphs*. For each value $(v_1, v_2, ..., v_k)$ returned, the first vertex of G is associated with v_1, the second with v_2, etc.

Note

This method works on *Graph*, *DiGraph* and *BipartiteGraph*.

See also

- *subgraph_search()* – finds a subgraph isomorphic to H inside of G
- *subgraph_search_count()* – counts the number of copies of H inside of G

EXAMPLES:

Iterating through all the labelled P_3 of P_5:

```
sage: g = graphs.PathGraph(5)
sage: P3 = graphs.PathGraph(3)
sage: for p in g.subgraph_search_iterator(P3, return_graphs=False):          #␣
↪needs sage.modules
....:     print(p)
[0, 1, 2]
[1, 2, 3]
[2, 1, 0]
[2, 3, 4]
[3, 2, 1]
[4, 3, 2]
sage: for p in g.subgraph_search_iterator(P3, return_graphs=True):           #␣
↪needs sage.modules
....:     print(p)
Subgraph of (Path graph)
Subgraph of (Path graph)
Subgraph of (Path graph)
Subgraph of (Path graph)
Subgraph of (Path graph)
Subgraph of (Path graph)
sage: all(h.is_isomorphic(P3) for h in g.subgraph_search_iterator(P3))      #␣
↪needs sage.modules
True
```

```
>>> from sage.all import *
>>> g = graphs.PathGraph(Integer(5))
>>> P3 = graphs.PathGraph(Integer(3))
>>> for p in g.subgraph_search_iterator(P3, return_graphs=False):          #␣
↪needs sage.modules
...     print(p)
[0, 1, 2]
[1, 2, 3]
[2, 1, 0]
[2, 3, 4]
[3, 2, 1]
[4, 3, 2]
>>> for p in g.subgraph_search_iterator(P3, return_graphs=True):           #␣
↪needs sage.modules
...     print(p)
Subgraph of (Path graph)
Subgraph of (Path graph)
Subgraph of (Path graph)
Subgraph of (Path graph)
Subgraph of (Path graph)
Subgraph of (Path graph)
>>> all(h.is_isomorphic(P3) for h in g.subgraph_search_iterator(P3))      #␣
↪needs sage.modules
True
```

If the graph has vertex labels or edge labels, the label is just ignored:

```
sage: g.set_vertex(0, 'foo')
sage: for p in g.subgraph_search_iterator(P3, return_graphs=False):          #␣
↪needs sage.modules
....:     print(p)
[0, 1, 2]
[1, 2, 3]
[2, 1, 0]
[2, 3, 4]
[3, 2, 1]
[4, 3, 2]
```

```
>>> from sage.all import *
>>> g.set_vertex(Integer(0), 'foo')
>>> for p in g.subgraph_search_iterator(P3, return_graphs=False):          #␣
↪needs sage.modules
...     print(p)
[0, 1, 2]
[1, 2, 3]
[2, 1, 0]
[2, 3, 4]
[3, 2, 1]
[4, 3, 2]
```

Search for induced subgraphs:

```
sage: H = graphs.HouseGraph()
sage: P4 = graphs.PathGraph(4)
sage: all(h.is_isomorphic(P4)                                               #␣
↪needs sage.modules
....:     for h in H.subgraph_search_iterator(P4, induced=True))
True
sage: sum(1 for h in H.subgraph_search_iterator(P4, induced=True))          #␣
↪needs sage.modules
4
sage: sum(1 for h in H.subgraph_search_iterator(P4, induced=False))         #␣
↪needs sage.modules
20
```

```
>>> from sage.all import *
>>> H = graphs.HouseGraph()
>>> P4 = graphs.PathGraph(Integer(4))
>>> all(h.is_isomorphic(P4)                                                 #␣
↪needs sage.modules
...     for h in H.subgraph_search_iterator(P4, induced=True))
True
>>> sum(Integer(1) for h in H.subgraph_search_iterator(P4, induced=True))    ␣
↪     # needs sage.modules
4
>>> sum(Integer(1) for h in H.subgraph_search_iterator(P4, induced=False))   ␣
↪     # needs sage.modules
20
```

Search for subdigraphs:

```
sage: H = digraphs.Complete(5)
sage: P4 = digraphs.Path(4)
sage: sum(1 for _ in H.subgraph_search_iterator(P4, induced=True))          #␣
↪needs sage.modules
0
sage: sum(1 for _ in H.subgraph_search_iterator(P4, induced=False))         #␣
↪needs sage.modules
120
```

```
>>> from sage.all import *
>>> H = digraphs.Complete(Integer(5))
>>> P4 = digraphs.Path(Integer(4))
>>> sum(Integer(1) for _ in H.subgraph_search_iterator(P4, induced=True))    ␣
↪     # needs sage.modules
0
>>> sum(Integer(1) for _ in H.subgraph_search_iterator(P4, induced=False))   ␣
↪     # needs sage.modules
120
```

This method also works for bipartite graphs:

```
sage: K33 = BipartiteGraph(graphs.CompleteBipartiteGraph(3, 3))
sage: K22 = BipartiteGraph(graphs.CompleteBipartiteGraph(2, 2))
sage: sum(1 for _ in K33.subgraph_search_iterator(K22))                     #␣
↪needs sage.modules
72
```

```
>>> from sage.all import *
>>> K33 = BipartiteGraph(graphs.CompleteBipartiteGraph(Integer(3),␣
↪Integer(3)))
>>> K22 = BipartiteGraph(graphs.CompleteBipartiteGraph(Integer(2),␣
↪Integer(2)))
>>> sum(Integer(1) for _ in K33.subgraph_search_iterator(K22))               ␣
↪     # needs sage.modules
72
```

symmetric_edge_polytope(*backend=None*)

Return the symmetric edge polytope of `self`.

The symmetric edge polytope (SEP) of a Graph on n vertices is the polytope in $\mathbf{Z}^n$ defined as the convex hull of $e_i - e_j$ and $e_j - e_i$ for each edge (i, j). Here $e_1, \ldots, e_n$ denotes the standard basis.

INPUT:

- `backend` – string or `None` (default); the backend to use; see the `Polyhedron constructor`

EXAMPLES:

The SEP of a 4-cycle is a cube:

```
sage: G = graphs.CycleGraph(4)
sage: P = G.symmetric_edge_polytope(); P                                    #␣
↪needs sage.geometry.polyhedron
A 3-dimensional polyhedron in ZZ^4 defined as the convex hull of 8 vertices
sage: P.is_combinatorially_isomorphic(polytopes.cube())                     #␣
```

(continues on next page)

(continued from previous page)

```
↪needs sage.geometry.polyhedron
True
```

```
>>> from sage.all import *
>>> G = graphs.CycleGraph(Integer(4))
>>> P = G.symmetric_edge_polytope(); P                                    #␣
↪needs sage.geometry.polyhedron
A 3-dimensional polyhedron in ZZ^4 defined as the convex hull of 8 vertices
>>> P.is_combinatorially_isomorphic(polytopes.cube())                     #␣
↪needs sage.geometry.polyhedron
True
```

The SEP of a complete graph on 4 vertices is a cuboctahedron:

```
sage: G = graphs.CompleteGraph(4)
sage: P = G.symmetric_edge_polytope(); P                                  #␣
↪needs sage.geometry.polyhedron
A 3-dimensional polyhedron in ZZ^4 defined as the convex hull of 12 vertices
sage: P.is_combinatorially_isomorphic(polytopes.cuboctahedron())          #␣
↪needs sage.geometry.polyhedron
True
```

```
>>> from sage.all import *
>>> G = graphs.CompleteGraph(Integer(4))
>>> P = G.symmetric_edge_polytope(); P                                    #␣
↪needs sage.geometry.polyhedron
A 3-dimensional polyhedron in ZZ^4 defined as the convex hull of 12 vertices
>>> P.is_combinatorially_isomorphic(polytopes.cuboctahedron())            #␣
↪needs sage.geometry.polyhedron
True
```

The SEP of a graph with edges on n vertices has dimension n minus the number of connected components:

```
sage: n = randint(5, 12)
sage: G = Graph()
sage: while not G.n_edges():                                              #␣
↪needs networkx
....:     G = graphs.RandomGNP(n, 0.2)
sage: P = G.symmetric_edge_polytope()                                     #␣
↪needs networkx sage.geometry.polyhedron
sage: P.ambient_dim() == n                                                #␣
↪needs networkx sage.geometry.polyhedron
True
sage: P.dim() == n - G.number_of_connected_components()                   #␣
↪needs networkx sage.geometry.polyhedron
True
```

```
>>> from sage.all import *
>>> n = randint(Integer(5), Integer(12))
>>> G = Graph()
>>> while not G.n_edges():                                                #␣
↪needs networkx
```

(continues on next page)

(continued from previous page)

```
...       G = graphs.RandomGNP(n, RealNumber('0.2'))
>>> P = G.symmetric_edge_polytope()                                    #␣
↪needs networkx sage.geometry.polyhedron
>>> P.ambient_dim() == n                                               #␣
↪needs networkx sage.geometry.polyhedron
True
>>> P.dim() == n - G.number_of_connected_components()                  #␣
↪needs networkx sage.geometry.polyhedron
True
```

The SEP of a graph is isomorphic to the subdirect sum of its connected components SEP's:

```
sage: n = randint(3, 6)
sage: G1 = graphs.RandomGNP(n, 0.2)                                    #␣
↪needs networkx
sage: n = randint(3, 6)
sage: G2 = graphs.RandomGNP(n, 0.2)                                    #␣
↪needs networkx
sage: G = G1.disjoint_union(G2)                                        #␣
↪needs networkx
sage: P = G.symmetric_edge_polytope()                                  #␣
↪needs networkx sage.geometry.polyhedron
sage: P1 = G1.symmetric_edge_polytope()                                #␣
↪needs networkx sage.geometry.polyhedron
sage: P2 = G2.symmetric_edge_polytope()                                #␣
↪needs networkx sage.geometry.polyhedron
sage: P.is_combinatorially_isomorphic(P1.subdirect_sum(P2))            #␣
↪needs networkx sage.geometry.polyhedron
True
```

```
>>> from sage.all import *
>>> n = randint(Integer(3), Integer(6))
>>> G1 = graphs.RandomGNP(n, RealNumber('0.2'))                                ␣
↪          # needs networkx
>>> n = randint(Integer(3), Integer(6))
>>> G2 = graphs.RandomGNP(n, RealNumber('0.2'))                                ␣
↪          # needs networkx
>>> G = G1.disjoint_union(G2)                                          #␣
↪needs networkx
>>> P = G.symmetric_edge_polytope()                                    #␣
↪needs networkx sage.geometry.polyhedron
>>> P1 = G1.symmetric_edge_polytope()                                  #␣
↪needs networkx sage.geometry.polyhedron
>>> P2 = G2.symmetric_edge_polytope()                                  #␣
↪needs networkx sage.geometry.polyhedron
>>> P.is_combinatorially_isomorphic(P1.subdirect_sum(P2))              #␣
↪needs networkx sage.geometry.polyhedron
True
```

All trees on n vertices have isomorphic SEPs:

```
sage: n = randint(4, 10)
```

(continues on next page)

(continued from previous page)

```
sage: G1 = graphs.RandomTree(n)
sage: G2 = graphs.RandomTree(n)
sage: P1 = G1.symmetric_edge_polytope()                                    #␣
↪needs sage.geometry.polyhedron
sage: P2 = G2.symmetric_edge_polytope()                                    #␣
↪needs sage.geometry.polyhedron
sage: P1.is_combinatorially_isomorphic(P2)                                 #␣
↪needs sage.geometry.polyhedron
True
```

```
>>> from sage.all import *
>>> n = randint(Integer(4), Integer(10))
>>> G1 = graphs.RandomTree(n)
>>> G2 = graphs.RandomTree(n)
>>> P1 = G1.symmetric_edge_polytope()                                      #␣
↪needs sage.geometry.polyhedron
>>> P2 = G2.symmetric_edge_polytope()                                      #␣
↪needs sage.geometry.polyhedron
>>> P1.is_combinatorially_isomorphic(P2)                                   #␣
↪needs sage.geometry.polyhedron
True
```

However, there are still many different SEPs:

```
sage: len(list(graphs(5)))
34
sage: polys = []
sage: for G in graphs(5):                                                  #␣
↪needs sage.geometry.polyhedron
....:     P = G.symmetric_edge_polytope()
....:     for P1 in polys:
....:         if P.is_combinatorially_isomorphic(P1):
....:             break
....:     else:
....:         polys.append(P)
sage: len(polys)                                                           #␣
↪needs sage.geometry.polyhedron
25
```

```
>>> from sage.all import *
>>> len(list(graphs(Integer(5))))
34
>>> polys = []
>>> for G in graphs(Integer(5)):                                                ␣
↪     # needs sage.geometry.polyhedron
...     P = G.symmetric_edge_polytope()
...     for P1 in polys:
...         if P.is_combinatorially_isomorphic(P1):
...             break
...     else:
...         polys.append(P)
>>> len(polys)                                                             #␣
```

(continues on next page)

(continued from previous page)

```
↪needs sage.geometry.polyhedron
25
```

A non-trivial example of two graphs with isomorphic SEPs:

```
sage: G1 = graphs.CycleGraph(4)
sage: G1.add_edges([[0, 5], [5, 2], [1, 6], [6, 2]])
sage: G2 = copy(G1)
sage: G1.add_edges([[2, 7], [7, 3]])
sage: G2.add_edges([[0, 7], [7, 3]])
sage: G1.is_isomorphic(G2)
False
sage: P1 = G1.symmetric_edge_polytope()                                    #␣
↪needs sage.geometry.polyhedron
sage: P2 = G2.symmetric_edge_polytope()                                    #␣
↪needs sage.geometry.polyhedron
sage: P1.is_combinatorially_isomorphic(P2)                                 #␣
↪needs sage.geometry.polyhedron
True
```

```
>>> from sage.all import *
>>> G1 = graphs.CycleGraph(Integer(4))
>>> G1.add_edges([[Integer(0), Integer(5)], [Integer(5), Integer(2)],␣
↪[Integer(1), Integer(6)], [Integer(6), Integer(2)]])
>>> G2 = copy(G1)
>>> G1.add_edges([[Integer(2), Integer(7)], [Integer(7), Integer(3)]])
>>> G2.add_edges([[Integer(0), Integer(7)], [Integer(7), Integer(3)]])
>>> G1.is_isomorphic(G2)
False
>>> P1 = G1.symmetric_edge_polytope()                                      #␣
↪needs sage.geometry.polyhedron
>>> P2 = G2.symmetric_edge_polytope()                                      #␣
↪needs sage.geometry.polyhedron
>>> P1.is_combinatorially_isomorphic(P2)                                   #␣
↪needs sage.geometry.polyhedron
True
```

Apparently, glueing two graphs together on a vertex gives isomorphic SEPs:

```
sage: n = randint(3, 7)
sage: g1 = graphs.RandomGNP(n, 0.2)                                        #␣
↪needs networkx
sage: g2 = graphs.RandomGNP(n, 0.2)                                        #␣
↪needs networkx
sage: G = g1.disjoint_union(g2)                                            #␣
↪needs networkx
sage: H = copy(G)                                                          #␣
↪needs networkx
sage: G.merge_vertices((0, randrange(n)), (1, randrange(n)))               #␣
↪needs networkx
sage: H.merge_vertices((0, randrange(n)), (1, randrange(n)))               #␣
↪needs networkx
```

(continues on next page)

(continued from previous page)

```
sage: PG = G.symmetric_edge_polytope()                                    #␣
↪needs networkx sage.geometry.polyhedron
sage: PH = H.symmetric_edge_polytope()                                    #␣
↪needs networkx sage.geometry.polyhedron
sage: PG.is_combinatorially_isomorphic(PH)                                #␣
↪needs networkx sage.geometry.polyhedron
True
```

```
>>> from sage.all import *
>>> n = randint(Integer(3), Integer(7))
>>> g1 = graphs.RandomGNP(n, RealNumber('0.2'))                           ␣
↪          # needs networkx
>>> g2 = graphs.RandomGNP(n, RealNumber('0.2'))                           ␣
↪          # needs networkx
>>> G = g1.disjoint_union(g2)                                            #␣
↪needs networkx
>>> H = copy(G)                                                          #␣
↪needs networkx
>>> G.merge_vertices(((Integer(0), randrange(n)), (Integer(1),␣
↪randrange(n))))              # needs networkx
>>> H.merge_vertices(((Integer(0), randrange(n)), (Integer(1),␣
↪randrange(n))))              # needs networkx
>>> PG = G.symmetric_edge_polytope()                                     #␣
↪needs networkx sage.geometry.polyhedron
>>> PH = H.symmetric_edge_polytope()                                     #␣
↪needs networkx sage.geometry.polyhedron
>>> PG.is_combinatorially_isomorphic(PH)                                 #␣
↪needs networkx sage.geometry.polyhedron
True
```

szeged_index(*G*, *algorithm=None*)

Return the Szeged index of the graph G.

Let $G = (V, E)$ be a connected graph, and for any $uv \in E$, let $N_u(uv) = \{w \in V : d(u, w) < d(v, w)\}$ and $n_u(uv) = |N_u(uv)|$. The Szeged index of G is then defined as [KRG1996]

$$\sum_{uv \in E(G)} n_u(uv) \times n_v(uv)$$

See the Wikipedia article Szeged_index for more details.

INPUT:

- `G` – a Sage graph
- `algorithm` – string (default: `None`); algorithm to use among:
  - `'low'` – algorithm with time complexity in $O(nm)$ and space complexity in $O(m)$. This implementation is currently valid only for simple (without loops or multiple edges) connected graphs.
  - `'high'` – algorithm with time complexity in $O(nm)$ and space complexity in $O(n^2)$. It cannot be used on graphs with more than $65536 = 2^{16}$ vertices.

  By default (`None`), the `'low'` algorithm is used for graphs and the `'high'` algorithm for digraphs.

EXAMPLES:

True for any connected graph [KRG1996]:

```
sage: from sage.graphs.distances_all_pairs import szeged_index
sage: g = graphs.PetersenGraph()
sage: g.wiener_index() <= szeged_index(g)
True
```

```
>>> from sage.all import *
>>> from sage.graphs.distances_all_pairs import szeged_index
>>> g = graphs.PetersenGraph()
>>> g.wiener_index() <= szeged_index(g)
True
```

True for all trees [KRG1996]:

```
sage: g = Graph()
sage: g.add_edges(graphs.CubeGraph(5).min_spanning_tree())
sage: g.wiener_index() == szeged_index(g)
True
```

```
>>> from sage.all import *
>>> g = Graph()
>>> g.add_edges(graphs.CubeGraph(Integer(5)).min_spanning_tree())
>>> g.wiener_index() == szeged_index(g)
True
```

Check that both algorithms return same value:

```
sage: # long time, needs networkx
sage: G = graphs.RandomBarabasiAlbert(100, 2)
sage: a = szeged_index(G, algorithm='low')
sage: b = szeged_index(G, algorithm='high')
sage: a == b
True
```

```
>>> from sage.all import *
>>> # long time, needs networkx
>>> G = graphs.RandomBarabasiAlbert(Integer(100), Integer(2))
>>> a = szeged_index(G, algorithm='low')
>>> b = szeged_index(G, algorithm='high')
>>> a == b
True
```

The Szeged index of a directed circuit of order n is $(n-1)^2$:

```
sage: [digraphs.Circuit(n).szeged_index() for n in range(1, 8)]
[0, 1, 4, 9, 16, 25, 36]
```

```
>>> from sage.all import *
>>> [digraphs.Circuit(n).szeged_index() for n in range(Integer(1),␣
↪Integer(8))]
[0, 1, 4, 9, 16, 25, 36]
```

tensor_product (*other*, *immutable=None*)

Return the tensor product of `self` and `other`.

The tensor product of G and H is the graph L with vertex set $V(L)$ equal to the Cartesian product of the vertices $V(G)$ and $V(H)$, and $((u, v), (w, x))$ is an edge iff - (u, w) is an edge of self, and - (v, x) is an edge of other.

The tensor product is also known as the categorical product and the Kronecker product (referring to the Kronecker matrix product). See the Wikipedia article Kronecker_product.

INPUT:

- `other` – a graph or a digraph
- `immutable` – boolean (default: `None`); whether to create a mutable/immutable product. `immutable=None` (default) means that the graphs and their product will behave the same way. If only one of them is immutable, the product will be mutable.

EXAMPLES:

```
sage: Z = graphs.CompleteGraph(2)
sage: C = graphs.CycleGraph(5)
sage: T = C.tensor_product(Z); T
Graph on 10 vertices
sage: T.size()
10
sage: T.plot()                          # long time                              #␣
↪needs sage.plot
Graphics object consisting of 21 graphics primitives
```

```
>>> from sage.all import *
>>> Z = graphs.CompleteGraph(Integer(2))
>>> C = graphs.CycleGraph(Integer(5))
>>> T = C.tensor_product(Z); T
Graph on 10 vertices
>>> T.size()
10
>>> T.plot()                          # long time                              #␣
↪needs sage.plot
Graphics object consisting of 21 graphics primitives
```

```
sage: D = graphs.DodecahedralGraph()
sage: P = graphs.PetersenGraph()
sage: T = D.tensor_product(P); T
Graph on 200 vertices
sage: T.size()
900
sage: T.plot()                          # long time                              #␣
↪needs sage.plot
Graphics object consisting of 1101 graphics primitives
```

```
>>> from sage.all import *
>>> D = graphs.DodecahedralGraph()
>>> P = graphs.PetersenGraph()
>>> T = D.tensor_product(P); T
Graph on 200 vertices
>>> T.size()
900
```

(continues on next page)

(continued from previous page)

```
>>> T.plot()                          # long time                                #␣
↪needs sage.plot
Graphics object consisting of 1101 graphics primitives
```

tikz(*format=None*, *edge_labels=None*, *color_by_label=False*, *prog='dot'*, *rankdir='down'*, *standalone_config=None*, *usepackage=None*, *usetikzlibrary=None*, *macros=None*, *use_sage_preamble=None*, ***kwds*)

Return a `TikzPicture` of the graph.

If graphviz and dot2tex are available, it uses these packages for placements of vertices and edges.

INPUT:

- `format` – string (default: `None`), `'dot2tex'` or `'tkz_graph'`. If `None`, it is set to `'dot2tex'` if dot2tex is present, otherwise it is set to `'tkz_graph'`.
- `edge_labels` – bool (default: `None`), if `None` it is set to `True` if and only if format is `'dot2tex'`
- `color_by_label` – boolean or dictionary or function (default: `False`); whether to color each edge with a different color according to its label; the colors are chosen along a rainbow, unless they are specified by a function or dictionary mapping labels to colors;

When using format `'dot2tex'`, the following inputs are considered:

- `prog` – string (default: `'dot'`) the program used for the layout corresponding to one of the software of the graphviz suite: 'dot', 'neato', 'twopi', 'circo' or 'fdp'.
- `rankdir` – string (default: `'down'`), direction of graph layout when prog is `'dot'`, possible values are `'down'`, `'up'`, `'right'` and `'left'`.
- `subgraph_clusters` – (default: `[]`) a list of lists of vertices, if supported by the layout engine, nodes belonging to the same cluster subgraph are drawn together, with the entire drawing of the cluster contained within a bounding rectangle.

Additional keywords arguments are forwarded to *sage.graphs.graph_latex.GraphLatex.set_option()*.

The following inputs define the preamble of the latex standalone document class file containing the tikzpicture:

- `standalone_config` – list of strings (default: `["border=4mm"]`); latex document class standalone configuration options
- `usepackage` – list of strings (default: `[]`); latex packages
- `usetikzlibrary` – list of strings (default: `[]`); tikz libraries to use
- `macros` – list of strings (default: `[]`); list of newcommands needed for the picture
- `use_sage_preamble` – bool (default: `None`), if `None` it is set to `True` if and only if format is `'tkz_graph'`

OUTPUT:

An instance of `TikzPicture`.

Note

Prerequisite: dot2tex optional Sage package and graphviz must be installed when using format `'dot2tex'`.

EXAMPLES:

```
sage: g = graphs.PetersenGraph()
sage: tikz = g.tikz()                    # optional - dot2tex graphviz
↪# long time
sage: _ = tikz.pdf(view=False)           # optional - dot2tex graphviz latex
↪# long time
```

```
>>> from sage.all import *
>>> g = graphs.PetersenGraph()
>>> tikz = g.tikz()                    # optional - dot2tex graphviz        #␣
↪long time
>>> _ = tikz.pdf(view=False)           # optional - dot2tex graphviz latex  #␣
↪long time
```

```
sage: tikz = g.tikz(format='tkz_graph')
sage: _ = tikz.pdf(view=False)           # optional - latex
```

```
>>> from sage.all import *
>>> tikz = g.tikz(format='tkz_graph')
>>> _ = tikz.pdf(view=False)           # optional - latex
```

Using another value for `prog`:

```
sage: tikz = g.tikz(prog='neato')        # optional - dot2tex graphviz
↪# long time
sage: _ = tikz.pdf()                     # optional - dot2tex graphviz latex
↪# long time
```

```
>>> from sage.all import *
>>> tikz = g.tikz(prog='neato')        # optional - dot2tex graphviz        #␣
↪long time
>>> _ = tikz.pdf()                     # optional - dot2tex graphviz latex  #␣
↪long time
```

Using `color_by_label` with default rainbow colors:

```
sage: G = DiGraph({0: {1: 333, 2: 444}, 1: {0: 444}, 2: {0: 555}})
sage: t = G.tikz(color_by_label=True)    # optional - dot2tex graphviz
↪# long time
sage: _ = t.pdf(view=False)              # optional - dot2tex graphviz latex
↪# long time
```

```
>>> from sage.all import *
>>> G = DiGraph({Integer(0): {Integer(1): Integer(333), Integer(2):␣
↪Integer(444)}, Integer(1): {Integer(0): Integer(444)}, Integer(2):
↪{Integer(0): Integer(555)}})
>>> t = G.tikz(color_by_label=True)    # optional - dot2tex graphviz        #␣
↪long time
>>> _ = t.pdf(view=False)              # optional - dot2tex graphviz latex  #␣
↪long time
```

Using `color_by_label` with colors given as a dictionary:

```
sage: G = DiGraph({0: {1: 333, 2: 444}, 1: {0: 444}, 2: {0: 555}})
sage: cbl = {333:'orange', 444: 'yellow', 555: 'purple'}
sage: t = G.tikz(color_by_label=cbl)     # optional - dot2tex graphviz
↪# long time
sage: _ = t.pdf(view=False)              # optional - dot2tex graphviz latex
↪# long time
```

```
>>> from sage.all import *
>>> G = DiGraph({Integer(0): {Integer(1): Integer(333), Integer(2):␣
↪Integer(444)}, Integer(1): {Integer(0): Integer(444)}, Integer(2):
↪{Integer(0): Integer(555)}})
>>> cbl = {Integer(333):'orange', Integer(444): 'yellow', Integer(555):
↪'purple'}
>>> t = G.tikz(color_by_label=cbl)     # optional - dot2tex graphviz        #␣
↪long time
>>> _ = t.pdf(view=False)              # optional - dot2tex graphviz latex  #␣
↪long time
```

Using `color_by_label` with colors given as a function:

```
sage: G = DiGraph({0: {1: -333, 2: -444}, 1: {0: 444}, 2: {0: 555}})
sage: cbl = lambda label:'green' if label >= 0 else 'orange'
sage: t = G.tikz(color_by_label=cbl)     # optional - dot2tex graphviz
↪# long time
sage: _ = t.pdf(view=False)              # optional - dot2tex graphviz latex
↪# long time
```

```
>>> from sage.all import *
>>> G = DiGraph({Integer(0): {Integer(1): -Integer(333), Integer(2): -
↪Integer(444)}, Integer(1): {Integer(0): Integer(444)}, Integer(2):
↪{Integer(0): Integer(555)}})
>>> cbl = lambda label:'green' if label >= Integer(0) else 'orange'
>>> t = G.tikz(color_by_label=cbl)     # optional - dot2tex graphviz        #␣
↪long time
>>> _ = t.pdf(view=False)              # optional - dot2tex graphviz latex  #␣
↪long time
```

Using another value for `rankdir`:

```
sage: tikz = g.tikz(rankdir='right')     # optional - dot2tex graphviz       #␣
↪long time
sage: _ = tikz.pdf(view=False)           # optional - dot2tex graphviz latex #␣
↪long time
```

```
>>> from sage.all import *
>>> tikz = g.tikz(rankdir='right')     # optional - dot2tex graphviz       #␣
↪long time
>>> _ = tikz.pdf(view=False)           # optional - dot2tex graphviz latex #␣
↪long time
```

Using subgraphs clusters (broken when using labels, see Issue #22070):

```
sage: S = FiniteSetMaps(5)
sage: I = S((0,1,2,3,4))
sage: a = S((0,1,3,0,0))
sage: b = S((0,2,4,1,0))
sage: roots = [I]
sage: succ = lambda v: [v*a,v*b,a*v,b*v]
sage: R = RecursivelyEnumeratedSet(roots, succ)
sage: G = R.to_digraph()
sage: G
Looped multi-digraph on 27 vertices
sage: C = G.strongly_connected_components()
sage: tikz = G.tikz(subgraph_clusters=C)# optional - dot2tex graphviz       #␣
↪long time
sage: tikz.add_usepackage('amstext')    # optional - dot2tex graphviz       #␣
↪long time
sage: _ = tikz.pdf(view=False)          # optional - dot2tex graphviz latex #␣
↪long time
```

```
>>> from sage.all import *
>>> S = FiniteSetMaps(Integer(5))
>>> I = S((Integer(0),Integer(1),Integer(2),Integer(3),Integer(4)))
>>> a = S((Integer(0),Integer(1),Integer(3),Integer(0),Integer(0)))
>>> b = S((Integer(0),Integer(2),Integer(4),Integer(1),Integer(0)))
>>> roots = [I]
>>> succ = lambda v: [v*a,v*b,a*v,b*v]
>>> R = RecursivelyEnumeratedSet(roots, succ)
>>> G = R.to_digraph()
>>> G
Looped multi-digraph on 27 vertices
>>> C = G.strongly_connected_components()
>>> tikz = G.tikz(subgraph_clusters=C)# optional - dot2tex graphviz       #␣
↪long time
>>> tikz.add_usepackage('amstext')    # optional - dot2tex graphviz       #␣
↪long time
>>> _ = tikz.pdf(view=False)          # optional - dot2tex graphviz latex #␣
↪long time
```

An example coming from `graphviz_string` documentation in SageMath:

```
sage: f(x) = -1 / x
sage: g(x) = 1 / (x + 1)
sage: G = DiGraph()
sage: G.add_edges((i, f(i), f) for i in (1, 2, 1/2, 1/4))
sage: G.add_edges((i, g(i), g) for i in (1, 2, 1/2, 1/4))
sage: tikz = G.tikz(format='dot2tex')   # optional - dot2tex graphviz       #␣
↪long time
sage: _ = tikz.pdf(view=False)          # optional - dot2tex graphviz latex #␣
↪long time
sage: def edge_options(data):
....:     u, v, label = data
....:     options = {"color": {f: "red", g: "blue"}[label]}
....:     if (u,v) == (1/2, -2): options["label"]       = "coucou"; options[
↪"label_style"] = "string"
```

(continues on next page)

(continued from previous page)

```
....:     if (u,v) == (1/2,2/3): options["dot"]         = "x=1,y=2"
....:     if (u,v) == (1,   -1): options["label_style"] = "latex"
....:     if (u,v) == (1,  1/2): options["dir"]         = "back"
....:     return options
sage: tikz = G.tikz(format='dot2tex',   # optional - dot2tex graphviz       #␣
↪long time
....:               edge_options=edge_options)
sage: _ = tikz.pdf(view=False)          # optional - dot2tex graphviz latex #␣
↪long time
```

```
>>> from sage.all import *
>>> __tmp__=var("x"); f = symbolic_expression(-Integer(1) / x).function(x)
>>> __tmp__=var("x"); g = symbolic_expression(Integer(1) / (x + Integer(1))).
↪function(x)
>>> G = DiGraph()
>>> G.add_edges((i, f(i), f) for i in (Integer(1), Integer(2), Integer(1)/
↪Integer(2), Integer(1)/Integer(4)))
>>> G.add_edges((i, g(i), g) for i in (Integer(1), Integer(2), Integer(1)/
↪Integer(2), Integer(1)/Integer(4)))
>>> tikz = G.tikz(format='dot2tex')   # optional - dot2tex graphviz       #␣
↪long time
>>> _ = tikz.pdf(view=False)          # optional - dot2tex graphviz latex #␣
↪long time
>>> def edge_options(data):
...     u, v, label = data
...     options = {"color": {f: "red", g: "blue"}[label]}
...     if (u,v) == (Integer(1)/Integer(2), -Integer(2)): options["label"]   ␣
↪   = "coucou"; options["label_style"] = "string"
...     if (u,v) == (Integer(1)/Integer(2),Integer(2)/Integer(3)): options[
↪"dot"]         = "x=1,y=2"
...     if (u,v) == (Integer(1),   -Integer(1)): options["label_style"] =
↪"latex"
...     if (u,v) == (Integer(1),  Integer(1)/Integer(2)): options["dir"]     ␣
↪   = "back"
...     return options
>>> tikz = G.tikz(format='dot2tex',   # optional - dot2tex graphviz       #␣
↪long time
...               edge_options=edge_options)
>>> _ = tikz.pdf(view=False)          # optional - dot2tex graphviz latex #␣
↪long time
```

to_dictionary(*edge_labels=False*, *multiple_edges=False*)

Return the graph as a dictionary.

INPUT:

- `edge_labels` – boolean (default: `False`); whether to include edge labels in the output
- `multiple_edges` – boolean (default: `False`); whether to include multiple edges in the output

OUTPUT: the output depends on the input:

- If `edge_labels == False` and `multiple_edges == False`, the output is a dictionary associating to each vertex the list of its neighbors.

- If `edge_labels == False` and `multiple_edges == True`, the output is a dictionary the same as previously with one difference: the neighbors are listed with multiplicity.
- If `edge_labels == True` and `multiple_edges == False`, the output is a dictionary associating to each vertex u [a dictionary associating to each vertex v incident to u the label of edge (u, v)].
- If `edge_labels == True` and `multiple_edges == True`, the output is a dictionary associating to each vertex u [a dictionary associating to each vertex v incident to u [the list of labels of all edges between u and v]].

Note

When used on directed graphs, the explanations above can be understood by replacing the word "neighbors" by "out-neighbors"

EXAMPLES:

```
sage: g = graphs.PetersenGraph().to_dictionary()
sage: [(key, sorted(g[key])) for key in g]
[(0, [1, 4, 5]),
 (1, [0, 2, 6]),
 (2, [1, 3, 7]),
 (3, [2, 4, 8]),
 (4, [0, 3, 9]),
 (5, [0, 7, 8]),
 (6, [1, 8, 9]),
 (7, [2, 5, 9]),
 (8, [3, 5, 6]),
 (9, [4, 6, 7])]
sage: graphs.PetersenGraph().to_dictionary(multiple_edges=True)
{0: [1, 4, 5], 1: [0, 2, 6],
 2: [1, 3, 7], 3: [2, 4, 8],
 4: [0, 3, 9], 5: [0, 7, 8],
 6: [1, 8, 9], 7: [2, 5, 9],
 8: [3, 5, 6], 9: [4, 6, 7]}
sage: graphs.PetersenGraph().to_dictionary(edge_labels=True)
{0: {1: None, 4: None, 5: None},
 1: {0: None, 2: None, 6: None},
 2: {1: None, 3: None, 7: None},
 3: {2: None, 4: None, 8: None},
 4: {0: None, 3: None, 9: None},
 5: {0: None, 7: None, 8: None},
 6: {1: None, 8: None, 9: None},
 7: {2: None, 5: None, 9: None},
 8: {3: None, 5: None, 6: None},
 9: {4: None, 6: None, 7: None}}
sage: graphs.PetersenGraph().to_dictionary(edge_labels=True,multiple_
↪edges=True)
{0: {1: [None], 4: [None], 5: [None]},
 1: {0: [None], 2: [None], 6: [None]},
 2: {1: [None], 3: [None], 7: [None]},
 3: {2: [None], 4: [None], 8: [None]},
 4: {0: [None], 3: [None], 9: [None]},
```

(continues on next page)

(continued from previous page)

```
 5: {0: [None], 7: [None], 8: [None]},
 6: {1: [None], 8: [None], 9: [None]},
 7: {2: [None], 5: [None], 9: [None]},
 8: {3: [None], 5: [None], 6: [None]},
 9: {4: [None], 6: [None], 7: [None]}}
```

```
>>> from sage.all import *
>>> g = graphs.PetersenGraph().to_dictionary()
>>> [(key, sorted(g[key])) for key in g]
[(0, [1, 4, 5]),
 (1, [0, 2, 6]),
 (2, [1, 3, 7]),
 (3, [2, 4, 8]),
 (4, [0, 3, 9]),
 (5, [0, 7, 8]),
 (6, [1, 8, 9]),
 (7, [2, 5, 9]),
 (8, [3, 5, 6]),
 (9, [4, 6, 7])]
>>> graphs.PetersenGraph().to_dictionary(multiple_edges=True)
{0: [1, 4, 5], 1: [0, 2, 6],
 2: [1, 3, 7], 3: [2, 4, 8],
 4: [0, 3, 9], 5: [0, 7, 8],
 6: [1, 8, 9], 7: [2, 5, 9],
 8: [3, 5, 6], 9: [4, 6, 7]}
>>> graphs.PetersenGraph().to_dictionary(edge_labels=True)
{0: {1: None, 4: None, 5: None},
 1: {0: None, 2: None, 6: None},
 2: {1: None, 3: None, 7: None},
 3: {2: None, 4: None, 8: None},
 4: {0: None, 3: None, 9: None},
 5: {0: None, 7: None, 8: None},
 6: {1: None, 8: None, 9: None},
 7: {2: None, 5: None, 9: None},
 8: {3: None, 5: None, 6: None},
 9: {4: None, 6: None, 7: None}}
>>> graphs.PetersenGraph().to_dictionary(edge_labels=True,multiple_edges=True)
{0: {1: [None], 4: [None], 5: [None]},
 1: {0: [None], 2: [None], 6: [None]},
 2: {1: [None], 3: [None], 7: [None]},
 3: {2: [None], 4: [None], 8: [None]},
 4: {0: [None], 3: [None], 9: [None]},
 5: {0: [None], 7: [None], 8: [None]},
 6: {1: [None], 8: [None], 9: [None]},
 7: {2: [None], 5: [None], 9: [None]},
 8: {3: [None], 5: [None], 6: [None]},
 9: {4: [None], 6: [None], 7: [None]}}
```

to_simple(*to_undirected=True*, *keep_label='any'*, *immutable=None*)

Return a simple version of the `self`.

In particular, loops and multiple edges are removed, and the graph might optionally be converted to an undirected graph.

INPUT:

- `to_undirected` – boolean (default: `True`); if `True`, the graph is also converted to an undirected graph
- `keep_label` – string (default: `'any'`); if there are multiple edges with different labels, this variable defines which label should be kept:
  - `'any'` – any label
  - `'min'` – the smallest label
  - `'max'` – the largest label

> **Warning**
>
> `'min'` and `'max'` only works if the labels can be compared. A `TypeError` might be raised when working with non-comparable objects.

- `immutable` – boolean (default: `Non`); whether to create a mutable/immutable copy. `immutable=None` (default) means that the graph and its copy will behave the same way.

EXAMPLES:

```
sage: G = DiGraph(loops=True, multiedges=True, sparse=True)
sage: G.add_edges([(0, 0, None), (1, 1, None), (2, 2, None),
....:              (2, 3, 1), (2, 3, 2), (3, 2, None)])
sage: G.edges(sort=True, labels=False)
[(0, 0), (1, 1), (2, 2), (2, 3), (2, 3), (3, 2)]
sage: H = G.to_simple()
sage: H.edges(sort=True, labels=False)
[(2, 3)]
sage: H.is_directed()
False
sage: H.allows_loops()
False
sage: H.allows_multiple_edges()
False
sage: G.to_simple(to_undirected=False, keep_label='min').edges(sort=True)
[(2, 3, 1), (3, 2, None)]
sage: G.to_simple(to_undirected=False, keep_label='max').edges(sort=True)
[(2, 3, 2), (3, 2, None)]
```

```
>>> from sage.all import *
>>> G = DiGraph(loops=True, multiedges=True, sparse=True)
>>> G.add_edges([(Integer(0), Integer(0), None), (Integer(1), Integer(1),␣
↪None), (Integer(2), Integer(2), None),
...              (Integer(2), Integer(3), Integer(1)), (Integer(2),␣
↪Integer(3), Integer(2)), (Integer(3), Integer(2), None)])
>>> G.edges(sort=True, labels=False)
[(0, 0), (1, 1), (2, 2), (2, 3), (2, 3), (3, 2)]
>>> H = G.to_simple()
>>> H.edges(sort=True, labels=False)
[(2, 3)]
>>> H.is_directed()
```

(continues on next page)

(continued from previous page)

```
False
>>> H.allows_loops()
False
>>> H.allows_multiple_edges()
False
>>> G.to_simple(to_undirected=False, keep_label='min').edges(sort=True)
[(2, 3, 1), (3, 2, None)]
>>> G.to_simple(to_undirected=False, keep_label='max').edges(sort=True)
[(2, 3, 2), (3, 2, None)]
```

transitive_closure(*loops=None*, *immutable=None*)

Return the transitive closure of the (di)graph.

The transitive closure of a graph G has an edge (x, y) if and only if there is a path between x and y in G.

The transitive closure of any (strongly) connected component of a (di)graph is a complete graph. The transitive closure of a directed acyclic graph is a directed acyclic graph representing the full partial order.

INPUT:

- `loops` – boolean (default: `None`); whether to allow loops in the returned (di)graph. By default (`None`), if the (di)graph allows loops, its transitive closure will have one loop edge per vertex. This can be prevented by disallowing loops in the (di)graph (`self.allow_loops(False)`).
- `immutable` – boolean (default: `None`); whether to create a mutable/immutable transitive closure. `immutable=None` (default) means that the (di)graph and its transitive closure will behave the same way.

EXAMPLES:

```
sage: g = graphs.PathGraph(4)
sage: g.transitive_closure()
Transitive closure of Path graph: Graph on 4 vertices
sage: g.transitive_closure().is_isomorphic(graphs.CompleteGraph(4))
True
sage: g = DiGraph({0: [1, 2], 1: [3], 2: [4, 5]})
sage: g.transitive_closure().edges(sort=True, labels=False)
[(0, 1), (0, 2), (0, 3), (0, 4), (0, 5), (1, 3), (2, 4), (2, 5)]
```

```
>>> from sage.all import *
>>> g = graphs.PathGraph(Integer(4))
>>> g.transitive_closure()
Transitive closure of Path graph: Graph on 4 vertices
>>> g.transitive_closure().is_isomorphic(graphs.CompleteGraph(Integer(4)))
True
>>> g = DiGraph({Integer(0): [Integer(1), Integer(2)], Integer(1):␣
↪[Integer(3)], Integer(2): [Integer(4), Integer(5)]})
>>> g.transitive_closure().edges(sort=True, labels=False)
[(0, 1), (0, 2), (0, 3), (0, 4), (0, 5), (1, 3), (2, 4), (2, 5)]
```

On an immutable digraph:

```
sage: digraphs.Path(5).copy(immutable=True).transitive_closure()
Transitive closure of Path: Digraph on 5 vertices
```

```
>>> from sage.all import *
>>> digraphs.Path(Integer(5)).copy(immutable=True).transitive_closure()
Transitive closure of Path: Digraph on 5 vertices
```

The transitive closure of a (di)graph allowing loops has by default a loop edge per vertex. Parameter `loops` allows to prevent that:

```
sage: G = digraphs.Circuit(3)
sage: G.transitive_closure().loop_edges(labels=False)
[]
sage: G.allow_loops(True)
sage: G.transitive_closure().loop_edges(labels=False)
[(0, 0), (1, 1), (2, 2)]
```

```
>>> from sage.all import *
>>> G = digraphs.Circuit(Integer(3))
>>> G.transitive_closure().loop_edges(labels=False)
[]
>>> G.allow_loops(True)
>>> G.transitive_closure().loop_edges(labels=False)
[(0, 0), (1, 1), (2, 2)]
```

Check the behavior of parameter `loops`:

```
sage: G = graphs.CycleGraph(3)
sage: G.transitive_closure().loop_edges(labels=False)
[]
sage: G.transitive_closure(loops=True).loop_edges(labels=False)
[(0, 0), (1, 1), (2, 2)]
sage: G.allow_loops(True)
sage: G.transitive_closure().loop_edges(labels=False)
[(0, 0), (1, 1), (2, 2)]
sage: G.transitive_closure(loops=False).loop_edges(labels=False)
[]
```

```
>>> from sage.all import *
>>> G = graphs.CycleGraph(Integer(3))
>>> G.transitive_closure().loop_edges(labels=False)
[]
>>> G.transitive_closure(loops=True).loop_edges(labels=False)
[(0, 0), (1, 1), (2, 2)]
>>> G.allow_loops(True)
>>> G.transitive_closure().loop_edges(labels=False)
[(0, 0), (1, 1), (2, 2)]
>>> G.transitive_closure(loops=False).loop_edges(labels=False)
[]
```

Check the behavior of parameter `immutable`:

```
sage: G = Graph([(0, 1)])
sage: G.transitive_closure().is_immutable()
False
sage: G.transitive_closure(immutable=True).is_immutable()
```

(continues on next page)

(continued from previous page)

```
True
sage: G = Graph([(0, 1)], immutable=True)
sage: G.transitive_closure().is_immutable()
True
sage: G.transitive_closure(immutable=False).is_immutable()
False
```

```
>>> from sage.all import *
>>> G = Graph([(Integer(0), Integer(1))])
>>> G.transitive_closure().is_immutable()
False
>>> G.transitive_closure(immutable=True).is_immutable()
True
>>> G = Graph([(Integer(0), Integer(1))], immutable=True)
>>> G.transitive_closure().is_immutable()
True
>>> G.transitive_closure(immutable=False).is_immutable()
False
```

transitive_reduction(*immutable=None*)

Return a transitive reduction of a graph.

A transitive reduction H of G has a path from x to y if and only if there was a path from x to y in G. Deleting any edge of H destroys this property. A transitive reduction is not unique in general. A transitive reduction has the same transitive closure as the original graph.

A transitive reduction of a complete graph is a tree. A transitive reduction of a tree is itself.

INPUT:

- `immutable` – boolean (default: `None`); whether to create a mutable/immutable transitive closure. `immutable=None` (default) means that the (di)graph and its transitive closure will behave the same way.

EXAMPLES:

```
sage: g = graphs.PathGraph(4)
sage: g.transitive_reduction() == g
True
sage: g = graphs.CompleteGraph(5)
sage: h = g.transitive_reduction(); h.size()
4
sage: (2*g).transitive_reduction().size()
8
sage: g = DiGraph({0: [1, 2], 1: [2, 3, 4, 5], 2: [4, 5]})
sage: g.transitive_reduction().size()
5
sage: (2*g).transitive_reduction().size()
10
```

```
>>> from sage.all import *
>>> g = graphs.PathGraph(Integer(4))
>>> g.transitive_reduction() == g
True
>>> g = graphs.CompleteGraph(Integer(5))
```

(continues on next page)

(continued from previous page)

```
>>> h = g.transitive_reduction(); h.size()
4
>>> (Integer(2)*g).transitive_reduction().size()
8
>>> g = DiGraph({Integer(0): [Integer(1), Integer(2)], Integer(1):␣
↪[Integer(2), Integer(3), Integer(4), Integer(5)], Integer(2): [Integer(4),␣
↪Integer(5)]})
>>> g.transitive_reduction().size()
5
>>> (Integer(2)*g).transitive_reduction().size()
10
```

traveling_salesman_problem(*use_edge_labels*, *maximize=False*, *solver=False*, *constraint_generation=None*, *verbose=None*, *verbose_constraints=0*, *integrality_tolerance=False*)

Solve the traveling salesman problem (TSP).

Given a graph (resp. a digraph) G with weighted edges, the traveling salesman problem consists in finding a Hamiltonian cycle (resp. circuit) of the graph of minimum cost.

This TSP is one of the most famous NP-Complete problems, this function can thus be expected to take some time before returning its result.

INPUT:

- `use_edge_labels` – boolean (default: `False`); whether to solve the weighted traveling salesman problem where the weight of an edge is defined by its label (a label set to `None` or `{}` being considered as a weight of 1), or the non-weighted version (i.e., the Hamiltonian cycle problem)
- `maximize` – boolean (default: `False`); whether to compute a minimum (default) or a maximum (when `maximize == True`) weight tour (or Hamiltonian cycle). This parameter is considered only if `use_edge_labels == True`.
- `solver` – string (default: `None`); specifies a Mixed Integer Linear Programming (MILP) solver to be used. If set to `None`, the default one is used. For more information on MILP solvers and which default solver is used, see the method `solve` of the class `MixedIntegerLinearProgram`.
- `constraint_generation` – boolean (default: `None`); whether to use constraint generation when solving the Mixed Integer Linear Program.

  When `constraint_generation = None`, constraint generation is used whenever the graph has a density larger than 70%.
- `verbose` – integer (default: 0); sets the level of verbosity. Set to 0 by default, which means quiet.
- `verbose_constraints` – boolean (default: `False`); whether to display which constraints are being generated
- `integrality_tolerance` – float; parameter for use with MILP solvers over an inexact base ring; see `MixedIntegerLinearProgram.get_values()`.

OUTPUT:

A solution to the TSP, as a `Graph` object whose vertex set is $V(G)$, and whose edges are only those of the solution.

ALGORITHM:

This optimization problem is solved through the use of Linear Programming.

Note

This function is correctly defined for both graph and digraphs. In the second case, the returned cycle is a circuit of optimal cost.

EXAMPLES:

The Heawood graph is known to be Hamiltonian:

```
sage: g = graphs.HeawoodGraph()
sage: tsp = g.traveling_salesman_problem(); tsp                                 #␣
↪needs sage.numerical.mip
TSP from Heawood graph: Graph on 14 vertices
```

```
>>> from sage.all import *
>>> g = graphs.HeawoodGraph()
>>> tsp = g.traveling_salesman_problem(); tsp                                   #␣
↪needs sage.numerical.mip
TSP from Heawood graph: Graph on 14 vertices
```

The solution to the TSP has to be connected:

```
sage: tsp.is_connected()                                                        #␣
↪needs sage.numerical.mip
True
```

```
>>> from sage.all import *
>>> tsp.is_connected()                                                          #␣
↪needs sage.numerical.mip
True
```

It must also be a 2-regular graph:

```
sage: tsp.is_regular(k=2)                                                       #␣
↪needs sage.numerical.mip
True
```

```
>>> from sage.all import *
>>> tsp.is_regular(k=Integer(2))                                                ␣
↪     # needs sage.numerical.mip
True
```

And obviously it is a subgraph of the Heawood graph:

```
sage: tsp.is_subgraph(g, induced=False)                                         #␣
↪needs sage.numerical.mip
True
```

```
>>> from sage.all import *
>>> tsp.is_subgraph(g, induced=False)                                           #␣
↪needs sage.numerical.mip
True
```

On the other hand, the Petersen Graph is known not to be Hamiltonian:

```
sage: g = graphs.PetersenGraph()
sage: tsp = g.traveling_salesman_problem()                                  #␣
↪needs sage.numerical.mip
Traceback (most recent call last):
...
EmptySetError: the given graph is not Hamiltonian
```

```
>>> from sage.all import *
>>> g = graphs.PetersenGraph()
>>> tsp = g.traveling_salesman_problem()                                    #␣
↪needs sage.numerical.mip
Traceback (most recent call last):
...
EmptySetError: the given graph is not Hamiltonian
```

One easy way to change it is obviously to add to this graph the edges corresponding to a Hamiltonian cycle. If we do this by setting the cost of these new edges to 2, while the others are set to 1, we notice that not all the edges we added are used in the optimal solution

```
sage: for u, v in g.edge_iterator(labels=None):
....:     g.set_edge_label(u, v, 1)

sage: cycle = graphs.CycleGraph(10)
sage: for u,v in cycle.edges(labels=None, sort=False):
....:     if not g.has_edge(u, v):
....:         g.add_edge(u, v)
....:     g.set_edge_label(u, v, 2)

sage: tsp = g.traveling_salesman_problem(use_edge_labels=True)              #␣
↪needs sage.numerical.mip
sage: sum( tsp.edge_labels() ) < 2 * 10                                     #␣
↪needs sage.numerical.mip
True
```

```
>>> from sage.all import *
>>> for u, v in g.edge_iterator(labels=None):
...     g.set_edge_label(u, v, Integer(1))

>>> cycle = graphs.CycleGraph(Integer(10))
>>> for u,v in cycle.edges(labels=None, sort=False):
...     if not g.has_edge(u, v):
...         g.add_edge(u, v)
...     g.set_edge_label(u, v, Integer(2))

>>> tsp = g.traveling_salesman_problem(use_edge_labels=True)                #␣
↪needs sage.numerical.mip
>>> sum( tsp.edge_labels() ) < Integer(2) * Integer(10)                     ␣
↪              # needs sage.numerical.mip
True
```

If we pick $1/2$ instead of 2 as a cost for these new edges, they clearly become the optimal solution:

```
sage: for u, v in cycle.edges(labels=None, sort=False):
....:     g.set_edge_label(u,v,1/2)

sage: tsp = g.traveling_salesman_problem(use_edge_labels=True)              #␣
↪needs sage.numerical.mip
sage: sum(tsp.edge_labels()) == (1/2) * 10                                  #␣
↪needs sage.numerical.mip
True
```

```
>>> from sage.all import *
>>> for u, v in cycle.edges(labels=None, sort=False):
...     g.set_edge_label(u,v,Integer(1)/Integer(2))

>>> tsp = g.traveling_salesman_problem(use_edge_labels=True)              #␣
↪needs sage.numerical.mip
>>> sum(tsp.edge_labels()) == (Integer(1)/Integer(2)) * Integer(10)          ␣
↪                        # needs sage.numerical.mip
True
```

Search for a minimum and a maximum weight Hamiltonian cycle:

```
sage: G = Graph([(0, 1, 1), (0, 2, 2), (0, 3, 1), (1, 2, 1), (1, 3, 2), (2, 3,
↪ 1)])
sage: tsp = G.traveling_salesman_problem(use_edge_labels=True,
....:                                    maximize=False)
sage: print(sum(tsp.edge_labels()))
4
sage: tsp = G.traveling_salesman_problem(use_edge_labels=True,
....:                                    maximize=True)
sage: print(sum(tsp.edge_labels()))
6
```

```
>>> from sage.all import *
>>> G = Graph([(Integer(0), Integer(1), Integer(1)), (Integer(0), Integer(2),␣
↪Integer(2)), (Integer(0), Integer(3), Integer(1)), (Integer(1), Integer(2),␣
↪Integer(1)), (Integer(1), Integer(3), Integer(2)), (Integer(2), Integer(3),␣
↪Integer(1))])
>>> tsp = G.traveling_salesman_problem(use_edge_labels=True,
...                                    maximize=False)
>>> print(sum(tsp.edge_labels()))
4
>>> tsp = G.traveling_salesman_problem(use_edge_labels=True,
...                                    maximize=True)
>>> print(sum(tsp.edge_labels()))
6
```

triangles_count(*algorithm=None*)

Return the number of triangles in the (di)graph.

For digraphs, we count the number of directed circuit of length 3.

INPUT:

- `algorithm` – string (default: `None`); specifies the algorithm to use (note that only `'iter'` is available

for directed graphs):

- `'sparse_copy'` – counts the triangles in a sparse copy of the graph (see *sage.graphs.base.static_sparse_graph*). Calls *static_sparse_graph.triangles_count*
- `'dense_copy'` – counts the triangles in a dense copy of the graph (see *sage.graphs.base.static_dense_graph*). Calls *static_dense_graph.triangles_count*
- `'matrix'` uses the trace of the cube of the adjacency matrix
- `'iter'` iterates over the pairs of neighbors of each vertex. No copy of the graph is performed
- `None` – for undirected graphs, uses `'sparse_copy'` or `'dense_copy'` depending on whether the graph is stored as dense or sparse. For directed graphs, uses `'iter'`.

EXAMPLES:

The Petersen graph is triangle free and thus:

```
sage: G = graphs.PetersenGraph()
sage: G.triangles_count()
0
```

```
>>> from sage.all import *
>>> G = graphs.PetersenGraph()
>>> G.triangles_count()
0
```

Any triple of vertices in the complete graph induces a triangle so we have:

```
sage: G = graphs.CompleteGraph(15)
sage: G.triangles_count() == binomial(15, 3)                                #␣
↪needs sage.symbolic
True
```

```
>>> from sage.all import *
>>> G = graphs.CompleteGraph(Integer(15))
>>> G.triangles_count() == binomial(Integer(15), Integer(3))                 ␣
↪              # needs sage.symbolic
True
```

The 2-dimensional DeBruijn graph of 2 symbols has 2 directed C_3:

```
sage: G = digraphs.DeBruijn(2,2)                                            #␣
↪needs sage.combinat
sage: G.triangles_count()                                                   #␣
↪needs sage.combinat
2
```

```
>>> from sage.all import *
>>> G = digraphs.DeBruijn(Integer(2),Integer(2))                             ␣
↪              # needs sage.combinat
>>> G.triangles_count()                                                    #␣
↪needs sage.combinat
2
```

The directed n-cycle is trivially triangle free for $n > 3$:

```
sage: G = digraphs.Circuit(10)
sage: G.triangles_count()                                                    #␣
↪needs sage.modules
0
```

```
>>> from sage.all import *
>>> G = digraphs.Circuit(Integer(10))
>>> G.triangles_count()                                                      #␣
↪needs sage.modules
0
```

union(*other*, *immutable=None*)

Return the union of `self` and `other`.

If the graphs have common vertices, the common vertices will be identified.

If one of the two graphs allows loops (or multiple edges), the resulting graph will allow loops (or multiple edges).

If both graphs are weighted the resulting graphs is weighted.

If both graphs are immutable, the resulting graph is immutable, unless requested otherwise.

INPUT:

- `immutable` – boolean (default: `None`); whether to create a mutable/immutable union. `immutable=None` (default) means that the graphs and their union will behave the same way.

See also

- *disjoint_union()*
- *join()*

EXAMPLES:

```
sage: G = graphs.CycleGraph(3)
sage: H = graphs.CycleGraph(4)
sage: J = G.union(H); J
Graph on 4 vertices
sage: J.vertices(sort=True)
[0, 1, 2, 3]
sage: J.edges(sort=True, labels=False)
[(0, 1), (0, 2), (0, 3), (1, 2), (2, 3)]
```

```
>>> from sage.all import *
>>> G = graphs.CycleGraph(Integer(3))
>>> H = graphs.CycleGraph(Integer(4))
>>> J = G.union(H); J
Graph on 4 vertices
>>> J.vertices(sort=True)
[0, 1, 2, 3]
>>> J.edges(sort=True, labels=False)
[(0, 1), (0, 2), (0, 3), (1, 2), (2, 3)]
```

vertex_boundary(*vertices1*, *vertices2=None*)

Return a list of all vertices in the external boundary of `vertices1`, intersected with `vertices2`.

If `vertices2` is `None`, then `vertices2` is the complement of `vertices1`. This is much faster if `vertices1` is smaller than `vertices2`.

The external boundary of a set of vertices is the union of the neighborhoods of each vertex in the set. Note that in this implementation, since `vertices2` defaults to the complement of `vertices1`, if a vertex v has a loop, then `vertex_boundary(v)` will not contain v.

In a digraph, the external boundary of a vertex v are those vertices u with an arc (v, u).

EXAMPLES:

```
sage: G = graphs.CubeGraph(4)
sage: l = ['0111', '0000', '0001', '0011', '0010', '0101', '0100', '1111',
↪'1101', '1011', '1001']
sage: sorted(G.vertex_boundary(['0000', '1111'], l))
['0001', '0010', '0100', '0111', '1011', '1101']
```

```
>>> from sage.all import *
>>> G = graphs.CubeGraph(Integer(4))
>>> l = ['0111', '0000', '0001', '0011', '0010', '0101', '0100', '1111', '1101
↪', '1011', '1001']
>>> sorted(G.vertex_boundary(['0000', '1111'], l))
['0001', '0010', '0100', '0111', '1011', '1101']
```

```
sage: D = DiGraph({0: [1, 2], 3: [0]})
sage: D.vertex_boundary([0])
[1, 2]
```

```
>>> from sage.all import *
>>> D = DiGraph({Integer(0): [Integer(1), Integer(2)], Integer(3):␣
↪[Integer(0)]})
>>> D.vertex_boundary([Integer(0)])
[1, 2]
```

vertex_connectivity(*G*, *value_only=True*, *sets=False*, *k=None*, *solver=None*, *verbose=0*, *integrality_tolerance=0.001*)

Return the vertex connectivity of the graph.

For more information, see the Wikipedia article Connectivity_(graph_theory) and the Wikipedia article K-vertex-connected_graph.

Note

- When the graph is directed, this method actually computes the *strong* connectivity, (i.e. a directed graph is strongly k-connected if there are k vertex disjoint paths between any two vertices u, v). If you do not want to consider strong connectivity, the best is probably to convert your `DiGraph` object to a `Graph` object, and compute the connectivity of this other graph.
- By convention, a complete graph on n vertices is $n - 1$ connected. In this case, no certificate can be given as there is no pair of vertices split by a cut of order $k - 1$. For this reason, the certificates returned in this situation are empty.

INPUT:

- `G` – the input Sage (Di)Graph
- `value_only` – boolean (default: `True`)
  - When set to `True` (default), only the value is returned.
  - When set to `False`, both the value and a minimum vertex cut are returned.
- **`sets` – boolean (default: `False`); whether to also return the two**
  sets of vertices that are disconnected by the cut (implies `value_only=False`)
- `k` – integer (default: `None`); when specified, check if the vertex connectivity of the (di)graph is larger or equal to k. The method thus outputs a boolean only.
- `solver` – string (default: `None`); specifies a Mixed Integer Linear Programming (MILP) solver to be used. If set to `None`, the default one is used. For more information on MILP solvers and which default solver is used, see the method `solve` of the class `MixedIntegerLinearProgram`.
- `verbose` – integer (default: 0); sets the level of verbosity. Set to 0 by default, which means quiet.
- `integrality_tolerance` – float; parameter for use with MILP solvers over an inexact base ring; see `MixedIntegerLinearProgram.get_values()`.

EXAMPLES:

A basic application on a `PappusGraph`:

```
sage: from sage.graphs.connectivity import vertex_connectivity
sage: g = graphs.PappusGraph()
sage: vertex_connectivity(g)                                                ␣
↪    # needs sage.numerical.mip
3
sage: g.vertex_connectivity()                                               ␣
↪    # needs sage.numerical.mip
3
```

```
>>> from sage.all import *
>>> from sage.graphs.connectivity import vertex_connectivity
>>> g = graphs.PappusGraph()
>>> vertex_connectivity(g)                                                  ␣
↪ # needs sage.numerical.mip
3
>>> g.vertex_connectivity()                                                 ␣
↪ # needs sage.numerical.mip
3
```

In a grid, the vertex connectivity is equal to the minimum degree, in which case one of the two sets is of cardinality 1:

```
sage: g = graphs.GridGraph([ 3,3 ])
sage: [value, cut, [ setA, setB ]] = vertex_connectivity(g, sets=True)      ␣
↪    # needs sage.numerical.mip
sage: len(setA) == 1 or len(setB) == 1                                      ␣
↪    # needs sage.numerical.mip
True
```

```
>>> from sage.all import *
>>> g = graphs.GridGraph([ Integer(3),Integer(3) ])
>>> [value, cut, [ setA, setB ]] = vertex_connectivity(g, sets=True)
↪ # needs sage.numerical.mip
>>> len(setA) == Integer(1) or len(setB) == Integer(1)
↪                   # needs sage.numerical.mip
True
```

A vertex cut in a tree is any internal vertex:

```
sage: tree = graphs.RandomTree(15)
sage: val, [cut_vertex] = vertex_connectivity(tree, value_only=False)
↪    # needs sage.numerical.mip
sage: tree.degree(cut_vertex) > 1
↪    # needs sage.numerical.mip
True
```

```
>>> from sage.all import *
>>> tree = graphs.RandomTree(Integer(15))
>>> val, [cut_vertex] = vertex_connectivity(tree, value_only=False)
↪ # needs sage.numerical.mip
>>> tree.degree(cut_vertex) > Integer(1)
↪          # needs sage.numerical.mip
True
```

When `value_only = True`, this function is optimized for small connectivity values and does not need to build a linear program.

It is the case for connected graphs which are not connected:

```
sage: g = 2 * graphs.PetersenGraph()
sage: vertex_connectivity(g)
↪    # needs sage.numerical.mip
0
```

```
>>> from sage.all import *
>>> g = Integer(2) * graphs.PetersenGraph()
>>> vertex_connectivity(g)
↪ # needs sage.numerical.mip
0
```

Or if they are just 1-connected:

```
sage: g = graphs.PathGraph(10)
sage: vertex_connectivity(g)
↪    # needs sage.numerical.mip
1
```

```
>>> from sage.all import *
>>> g = graphs.PathGraph(Integer(10))
>>> vertex_connectivity(g)
↪ # needs sage.numerical.mip
1
```

For directed graphs, the strong connectivity is tested through the dedicated function:

```
sage: g = digraphs.ButterflyGraph(3)
sage: vertex_connectivity(g)                                                ␣
↪    # needs sage.numerical.mip
0
```

```
>>> from sage.all import *
>>> g = digraphs.ButterflyGraph(Integer(3))
>>> vertex_connectivity(g)                                                  ␣
↪ # needs sage.numerical.mip
0
```

A complete graph on 10 vertices is 9-connected:

```
sage: g = graphs.CompleteGraph(10)
sage: vertex_connectivity(g)                                                ␣
↪    # needs sage.numerical.mip
9
```

```
>>> from sage.all import *
>>> g = graphs.CompleteGraph(Integer(10))
>>> vertex_connectivity(g)                                                  ␣
↪ # needs sage.numerical.mip
9
```

A complete digraph on 10 vertices is 9-connected:

```
sage: g = DiGraph(graphs.CompleteGraph(10))
sage: vertex_connectivity(g)                                                ␣
↪    # needs sage.numerical.mip
9
```

```
>>> from sage.all import *
>>> g = DiGraph(graphs.CompleteGraph(Integer(10)))
>>> vertex_connectivity(g)                                                  ␣
↪ # needs sage.numerical.mip
9
```

When parameter `k` is set, we only check for the existence of a vertex cut of order at least `k`:

```
sage: g = graphs.PappusGraph()
sage: vertex_connectivity(g, k=3)                                           ␣
↪    # needs sage.numerical.mip
True
sage: vertex_connectivity(g, k=4)                                           ␣
↪    # needs sage.numerical.mip
False
```

```
>>> from sage.all import *
>>> g = graphs.PappusGraph()
>>> vertex_connectivity(g, k=Integer(3))                                    ␣
↪          # needs sage.numerical.mip
```

(continues on next page)

(continued from previous page)

```
True
>>> vertex_connectivity(g, k=Integer(4))
↪          # needs sage.numerical.mip
False
```

vertex_cut(*s*, *t*, *value_only*, *vertices=True*, *solver=False*, *verbose=None*, *integrality_tolerance=0*)

Return a minimum vertex cut between non-adjacent vertices s and t represented by a list of vertices.

A vertex cut between two non-adjacent vertices is a set U of vertices of `self` such that the graph obtained by removing U from `self` is disconnected. For more information, see the Wikipedia article Cut_(graph_theory).

INPUT:

- `value_only` – boolean (default: `True`); whether to return only the size of the minimum cut, or to also return the set U of vertices of the cut
- `vertices` – boolean (default: `False`); whether to also return the two sets of vertices that are disconnected by the cut. Implies `value_only` set to `False`.
- `solver` – string (default: `None`); specifies a Mixed Integer Linear Programming (MILP) solver to be used. If set to `None`, the default one is used. For more information on MILP solvers and which default solver is used, see the method `solve` of the class `MixedIntegerLinearProgram`.
- `verbose` – integer (default: 0); sets the level of verbosity. Set to 0 by default, which means quiet.
- `integrality_tolerance` – float; parameter for use with MILP solvers over an inexact base ring; see `MixedIntegerLinearProgram.get_values()`.

OUTPUT:

Real number or tuple, depending on the given arguments (examples are given below).

EXAMPLES:

A basic application in the Pappus graph:

```
sage: g = graphs.PappusGraph()
sage: g.vertex_cut(1, 16, value_only=True)
↪# needs sage.numerical.mip
3
```

```
>>> from sage.all import *
>>> g = graphs.PappusGraph()
>>> g.vertex_cut(Integer(1), Integer(16), value_only=True)
↪               # needs sage.numerical.mip
3
```

In the bipartite complete graph $K_{2,8}$, a cut between the two vertices in the size 2 part consists of the other 8 vertices:

```
sage: g = graphs.CompleteBipartiteGraph(2, 8)
sage: [value, vertices] = g.vertex_cut(0, 1, value_only=False)
↪# needs sage.numerical.mip
sage: print(value)
↪# needs sage.numerical.mip
8
sage: vertices == list(range(2, 10))
```

(continues on next page)

(continued from previous page)

```
↪# needs sage.numerical.mip
True
```

```
>>> from sage.all import *
>>> g = graphs.CompleteBipartiteGraph(Integer(2), Integer(8))
>>> [value, vertices] = g.vertex_cut(Integer(0), Integer(1), value_
↪only=False)               # needs sage.numerical.mip
>>> print(value)                                                          #␣
↪needs sage.numerical.mip
8
>>> vertices == list(range(Integer(2), Integer(10)))                      ␣
↪               # needs sage.numerical.mip
True
```

Clearly, in this case the two sides of the cut are singletons:

```
sage: [value, vertices, [set1, set2]] = g.vertex_cut(0, 1, vertices=True)
↪# needs sage.numerical.mip
sage: len(set1) == 1
↪# needs sage.numerical.mip
True
sage: len(set2) == 1
↪# needs sage.numerical.mip
True
```

```
>>> from sage.all import *
>>> [value, vertices, [set1, set2]] = g.vertex_cut(Integer(0), Integer(1),␣
↪vertices=True)    # needs sage.numerical.mip
>>> len(set1) == Integer(1)                                               ␣
↪      # needs sage.numerical.mip
True
>>> len(set2) == Integer(1)                                               ␣
↪      # needs sage.numerical.mip
True
```

vertex_disjoint_paths(*s*, *t*, *solver*, *verbose=None*, *integrality_tolerance=0*)

Return a list of vertex-disjoint paths between two vertices.

The vertex version of Menger's theorem asserts that the size of the minimum vertex cut between two vertices s and t (the minimum number of vertices whose removal disconnects s and t) is equal to the maximum number of pairwise vertex-independent paths from s to t.

This function returns a list of such paths.

INPUT:

- `s`, `t` – two vertices of the graph
- `solver` – string (default: `None`); specifies a Mixed Integer Linear Programming (MILP) solver to be used. If set to `None`, the default one is used. For more information on MILP solvers and which default solver is used, see the method `solve` of the class `MixedIntegerLinearProgram`.
- `verbose` – integer (default: 0); sets the level of verbosity. Set to 0 by default, which means quiet.
- `integrality_tolerance` – float; parameter for use with MILP solvers over an inexact base ring; see `MixedIntegerLinearProgram.get_values()`.

EXAMPLES:

In a complete bipartite graph

```
sage: g = graphs.CompleteBipartiteGraph(2, 3)
sage: g.vertex_disjoint_paths(0, 1)                                             #␣
↪needs sage.numerical.mip
[[0, 2, 1], [0, 3, 1], [0, 4, 1]]
```

```
>>> from sage.all import *
>>> g = graphs.CompleteBipartiteGraph(Integer(2), Integer(3))
>>> g.vertex_disjoint_paths(Integer(0), Integer(1))                             ␣
↪              # needs sage.numerical.mip
[[0, 2, 1], [0, 3, 1], [0, 4, 1]]
```

vertex_iterator(*vertices=None*, *degree=None*, *vertex_property=None*)

Return an iterator over the given vertices.

Returns `False` if not given a vertex, sequence, iterator or `None`. `None` is equivalent to a list of every vertex. Note that `for v in G` syntax is allowed.

INPUT:

- `vertices` – iterated vertices are these intersected with the vertices of the (di)graph
- `degree` – nonnegative integer (default: `None`); a vertex `v` is kept if `degree(v) == degree`
- `vertex_property` – function (default: `None`); a function that inputs a vertex and outputs a boolean value, i.e., a vertex `v` is kept if `vertex_property(v) == True`

EXAMPLES:

```
sage: P = graphs.PetersenGraph()
sage: for v in P.vertex_iterator():
....:     print(v)
0
1
2
...
8
9
```

```
>>> from sage.all import *
>>> P = graphs.PetersenGraph()
>>> for v in P.vertex_iterator():
...     print(v)
0
1
2
...
8
9
```

```
sage: G = graphs.TetrahedralGraph()
sage: for i in G:
....:     print(i)
```

(continues on next page)

(continued from previous page)

```
0
1
2
3
```

```
>>> from sage.all import *
>>> G = graphs.TetrahedralGraph()
>>> for i in G:
...     print(i)
0
1
2
3
```

```
sage: C = graphs.ClawGraph().complement()
sage: for v in C.vertex_iterator(degree=0):
....:     print(v)
0
```

```
>>> from sage.all import *
>>> C = graphs.ClawGraph().complement()
>>> for v in C.vertex_iterator(degree=Integer(0)):
...     print(v)
0
```

```
sage: H = graphs.PathGraph(5)
sage: prop = lambda l: l % 3 == 1
sage: for v in H.vertex_iterator(degree=1, vertex_property=prop):
....:     print(v)
4
```

```
>>> from sage.all import *
>>> H = graphs.PathGraph(Integer(5))
>>> prop = lambda l: l % Integer(3) == Integer(1)
>>> for v in H.vertex_iterator(degree=Integer(1), vertex_property=prop):
...     print(v)
4
```

vertices(*sort=False*, *key=None*, *degree=None*, *vertex_property=None*)

Return a list of the vertices.

INPUT:

- `sort` – boolean (default: `False`); whether to sort vertices according the ordering specified with parameter `key`. If `False` (default), vertices are not sorted.
- `key` – a function (default: `None`); a function that takes a vertex as its one argument and returns a value that can be used for comparisons in the sorting algorithm (we must have `sort=True`)
- `degree` – nonnegative integer (default: `None`); a vertex `v` is kept if `degree(v) == degree`
- `vertex_property` – function (default: `None`); a function that inputs a vertex and outputs a boolean value, i.e., a vertex `v` is kept if `vertex_property(v) == True`

OUTPUT:

The list of vertices of the (di)graph.

Warning

Since any object may be a vertex, there is no guarantee that any two vertices will be comparable. With default objects for vertices (all integers), or when all the vertices are of the same simple type, then there should not be a problem with how the vertices will be sorted. However, if you need to guarantee a total order for the sorting of the edges, use the `key` argument, as illustrated in the examples below.

EXAMPLES:

```
sage: P = graphs.PetersenGraph()
sage: P.vertices(sort=True)
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

```
>>> from sage.all import *
>>> P = graphs.PetersenGraph()
>>> P.vertices(sort=True)
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

If you do not care about sorted output and you are concerned about the time taken to sort, consider the following alternative:

```
sage: # not tested
sage: timeit V = P.vertices(sort=True)
625 loops, best of 3: 3.86 [micro]s per loop
sage: timeit V = P.vertices(sort=False)
625 loops, best of 3: 2.06 [micro]s per loop
sage: timeit V = list(P.vertex_iterator())
625 loops, best of 3: 2.05 [micro]s per loop
sage: timeit('V = list(P)')
625 loops, best of 3: 1.98 [micro]s per loop
```

```
>>> from sage.all import *
>>> # not tested
>>> timeit V = P.vertices(sort=True)
625 loops, best of 3: 3.86 [micro]s per loop
>>> timeit V = P.vertices(sort=False)
625 loops, best of 3: 2.06 [micro]s per loop
>>> timeit V = list(P.vertex_iterator())
625 loops, best of 3: 2.05 [micro]s per loop
>>> timeit('V = list(P)')
625 loops, best of 3: 1.98 [micro]s per loop
```

We illustrate various ways to use a `key` to sort the list:

```
sage: H = graphs.HanoiTowerGraph(3, 3, labels=False)
sage: H.vertices(sort=True)
[0, 1, 2, 3, 4, ... 22, 23, 24, 25, 26]
sage: H.vertices(sort=True, key=lambda x: -x)
[26, 25, 24, 23, 22, ... 4, 3, 2, 1, 0]
```

```
>>> from sage.all import *
>>> H = graphs.HanoiTowerGraph(Integer(3), Integer(3), labels=False)
>>> H.vertices(sort=True)
[0, 1, 2, 3, 4, ... 22, 23, 24, 25, 26]
>>> H.vertices(sort=True, key=lambda x: -x)
[26, 25, 24, 23, 22, ... 4, 3, 2, 1, 0]
```

```
sage: G = graphs.HanoiTowerGraph(3, 3)
sage: G.vertices(sort=True)
[(0, 0, 0), (0, 0, 1), (0, 0, 2), (0, 1, 0), ... (2, 2, 1), (2, 2, 2)]
sage: G.vertices(sort=True, key = lambda x: (x[1], x[2], x[0]))
[(0, 0, 0), (1, 0, 0), (2, 0, 0), (0, 0, 1), ... (1, 2, 2), (2, 2, 2)]
```

```
>>> from sage.all import *
>>> G = graphs.HanoiTowerGraph(Integer(3), Integer(3))
>>> G.vertices(sort=True)
[(0, 0, 0), (0, 0, 1), (0, 0, 2), (0, 1, 0), ... (2, 2, 1), (2, 2, 2)]
>>> G.vertices(sort=True, key = lambda x: (x[Integer(1)], x[Integer(2)],␣
↪x[Integer(0)]))
[(0, 0, 0), (1, 0, 0), (2, 0, 0), (0, 0, 1), ... (1, 2, 2), (2, 2, 2)]
```

The discriminant of a polynomial is a function that returns an integer. We build a graph whose vertices are polynomials, and use the discriminant function to provide an ordering. Note that since functions are first-class objects in Python, we can specify precisely the function from the Sage library that we wish to use as the key:

```
sage: t = polygen(QQ, 't')
sage: K = Graph({5*t: [t^2], t^2: [t^2+2], t^2+2: [4*t^2-6], 4*t^2-6: [5*t]})
sage: from sage.rings.polynomial.polynomial_rational_flint import Polynomial_
↪rational_flint
sage: dsc = Polynomial_rational_flint.discriminant
sage: verts = K.vertices(sort=True, key=dsc); verts
[t^2 + 2, t^2, 5*t, 4*t^2 - 6]
sage: [x.discriminant() for x in verts]
[-8, 0, 1, 96]
```

```
>>> from sage.all import *
>>> t = polygen(QQ, 't')
>>> K = Graph({Integer(5)*t: [t**Integer(2)], t**Integer(2):␣
↪[t**Integer(2)+Integer(2)], t**Integer(2)+Integer(2):␣
↪[Integer(4)*t**Integer(2)-Integer(6)], Integer(4)*t**Integer(2)-Integer(6):␣
↪[Integer(5)*t]})
>>> from sage.rings.polynomial.polynomial_rational_flint import Polynomial_
↪rational_flint
>>> dsc = Polynomial_rational_flint.discriminant
>>> verts = K.vertices(sort=True, key=dsc); verts
[t^2 + 2, t^2, 5*t, 4*t^2 - 6]
>>> [x.discriminant() for x in verts]
[-8, 0, 1, 96]
```

weighted(*new=None*)

Whether the (di)graph is to be considered as a weighted (di)graph.

INPUT:

- `new` – boolean (default: `None`); if it is provided, then the weightedness flag is set accordingly. This is not allowed for immutable graphs.

> **Note**
>
> Changing the weightedness flag changes the ==-class of a graph and is thus not allowed for immutable graphs.
>
> Edge weightings can still exist for (di)graphs `G` where `G.weighted()` is `False`.

EXAMPLES:

Here we have two graphs with different labels, but `weighted()` is `False` for both, so we just check for the presence of edges:

```
sage: G = Graph({0: {1: 'a'}}, sparse=True)
sage: H = Graph({0: {1: 'b'}}, sparse=True)
sage: G == H
True
```

```
>>> from sage.all import *
>>> G = Graph({Integer(0): {Integer(1): 'a'}}, sparse=True)
>>> H = Graph({Integer(0): {Integer(1): 'b'}}, sparse=True)
>>> G == H
True
```

Now one is weighted and the other is not, and thus the graphs are not equal:

```
sage: G.weighted(True)
sage: H.weighted()
False
sage: G == H
False
```

```
>>> from sage.all import *
>>> G.weighted(True)
>>> H.weighted()
False
>>> G == H
False
```

However, if both are weighted, then we finally compare 'a' to 'b':

```
sage: H.weighted(True)
sage: G == H
False
```

```
>>> from sage.all import *
>>> H.weighted(True)
>>> G == H
False
```

weighted_adjacency_matrix(*sparse*, *vertices=True*, *default_weight=None*, *base_ring=None*, ***kwds*)

Return the weighted adjacency matrix of the graph.

By default, each vertex is represented by its position in the list returned by method *vertices()*.

INPUT:

- sparse – boolean (default: True); whether to use a sparse or a dense matrix
- vertices – list; None, or True (default: None)
  - when a list, the i-th row and column of the matrix correspond to the i-th vertex in the ordering of vertices,
  - when None, the i-th row and column of the matrix correspond to the i-th vertex in the ordering given by *GenericGraph.vertices()* with sort=True.
  - when True, construct an endomorphism of a free module instead of a matrix, where the module's basis is indexed by the vertices.

  If the vertices are not comparable, the keyword vertices must be used to specify an ordering, or a TypeError exception will be raised.
- default_weight – (default: None) specifies the weight to replace any None edge label. When not specified an error is raised if the label of an edge is None.
- base_ring – a ring (default: determined from the weights); the base ring of the matrix space to use
- **kwds – other keywords to pass to *matrix()*

EXAMPLES:

```
sage: G = Graph(sparse=True, weighted=True)
sage: G.add_edges([(0, 1, 1), (1, 2, 2), (0, 2, 3), (0, 3, 4)])
sage: M = G.weighted_adjacency_matrix(); M                                  #␣
↪needs sage.modules
[0 1 3 4]
[1 0 2 0]
[3 2 0 0]
[4 0 0 0]
sage: H = Graph(data=M, format='weighted_adjacency_matrix', sparse=True)    #␣
↪needs sage.modules
sage: H == G                                                                #␣
↪needs sage.modules
True
sage: G.weighted_adjacency_matrix(vertices=[3, 2, 1, 0])                    #␣
↪needs sage.modules
[0 0 0 4]
[0 0 2 3]
[0 2 0 1]
[4 3 1 0]
```

```
>>> from sage.all import *
>>> G = Graph(sparse=True, weighted=True)
>>> G.add_edges([(Integer(0), Integer(1), Integer(1)), (Integer(1),␣
↪Integer(2), Integer(2)), (Integer(0), Integer(2), Integer(3)), (Integer(0),␣
↪Integer(3), Integer(4))])
>>> M = G.weighted_adjacency_matrix(); M                                    #␣
↪needs sage.modules
[0 1 3 4]
[1 0 2 0]
[3 2 0 0]
```

(continues on next page)

(continued from previous page)

```
[4 0 0 0]
>>> H = Graph(data=M, format='weighted_adjacency_matrix', sparse=True)     #␣
↪needs sage.modules
>>> H == G                                                                 #␣
↪needs sage.modules
True
>>> G.weighted_adjacency_matrix(vertices=[Integer(3), Integer(2), Integer(1),␣
↪Integer(0)])                      # needs sage.modules
[0 0 0 4]
[0 0 2 3]
[0 2 0 1]
[4 3 1 0]
```

Using a different matrix implementation:

```
sage: M = G.weighted_adjacency_matrix(sparse=False, base_ring=ZZ,           #␣
↪needs numpy sage.modules
....:                                   implementation='numpy'); M
[0 1 3 4]
[1 0 2 0]
[3 2 0 0]
[4 0 0 0]
```

```
>>> from sage.all import *
>>> M = G.weighted_adjacency_matrix(sparse=False, base_ring=ZZ,           #␣
↪needs numpy sage.modules
...                                    implementation='numpy'); M
[0 1 3 4]
[1 0 2 0]
[3 2 0 0]
[4 0 0 0]
```

As an immutable matrix:

```
sage: M = G.weighted_adjacency_matrix(immutable=True); M                   #␣
↪needs sage.modules
[0 1 3 4]
[1 0 2 0]
[3 2 0 0]
[4 0 0 0]
sage: M[2, 2] = 1                                                          #␣
↪needs sage.modules
Traceback (most recent call last):
...
ValueError: matrix is immutable; please change a copy instead
(i.e., use copy(M) to change a copy of M).
```

```
>>> from sage.all import *
>>> M = G.weighted_adjacency_matrix(immutable=True); M                     #␣
↪needs sage.modules
[0 1 3 4]
[1 0 2 0]
```

(continues on next page)

(continued from previous page)

```
[3 2 0 0]
[4 0 0 0]
>>> M[Integer(2), Integer(2)] = Integer(1)                                    ␣
↪                        # needs sage.modules
Traceback (most recent call last):
...
ValueError: matrix is immutable; please change a copy instead
(i.e., use copy(M) to change a copy of M).
```

Creating a module morphism:

```
sage: G = Graph(sparse=True, weighted=True)
sage: G.add_edges([('A', 'B', 1), ('B', 'C', 2), ('A', 'C', 3), ('A', 'D',␣
↪4)])
sage: phi = G.weighted_adjacency_matrix(vertices=True); phi
Generic endomorphism of
 Free module generated by {'A', 'B', 'C', 'D'} over Integer Ring
sage: print(phi._unicode_art_matrix())
  A B C D
A⎛0 1 3 4⎞
B⎜1 0 2 0⎟
C⎜3 2 0 0⎟
D⎝4 0 0 0⎠
```

```
>>> from sage.all import *
>>> G = Graph(sparse=True, weighted=True)
>>> G.add_edges([('A', 'B', Integer(1)), ('B', 'C', Integer(2)), ('A', 'C',␣
↪Integer(3)), ('A', 'D', Integer(4))])
>>> phi = G.weighted_adjacency_matrix(vertices=True); phi
Generic endomorphism of
 Free module generated by {'A', 'B', 'C', 'D'} over Integer Ring
>>> print(phi._unicode_art_matrix())
  A B C D
A⎛0 1 3 4⎞
B⎜1 0 2 0⎟
C⎜3 2 0 0⎟
D⎝4 0 0 0⎠
```

wiener_index(*by_weight=False*, *algorithm=None*, *weight_function=None*, *check_weight=True*)

Return the Wiener index of `self`.

The graph is expected to have no cycles of negative weight.

The Wiener index of a undirected graph G is $W(G) = \frac{1}{2}\sum_{u,v\in G} d(u,v)$ where $d(u,v)$ denotes the distance between vertices u and v (see [KRG1996]).

The Wiener index of a directed graph G is defined as the sum of the distances between each pairs of vertices, i.e., $W(G) = \sum_{u,v\in G} d(u,v)$.

For more information on the input variables and more examples, we refer to *shortest_paths()* and *shortest_path_all_pairs()*, which have very similar input variables.

INPUT:

- `by_weight` – boolean (default: `False`); if `True`, the edges in the graph are weighted, otherwise all edges have weight 1

- `algorithm` – string (default: `None`); one of the following algorithms:
  - For `by_weight==False` only:
    - `'BFS'` – the computation is done through a BFS centered on each vertex successively
    - `'Floyd-Warshall-Cython'` – the Cython implementation of the Floyd-Warshall algorithm. Usually slower than `'BFS'`.
  - For graphs without negative weights:
    - `'Dijkstra_Boost'`: the Dijkstra algorithm, implemented in Boost.
    - `'Dijkstra_NetworkX'`: the Dijkstra algorithm, implemented in NetworkX. Usually slower than `'Dijkstra_Boost'`.
  - For graphs with negative weights:
    - `'Bellman-Ford_Boost'`: the Bellman-Ford algorithm, implemented in Boost.
    - `'Johnson_Boost'`: the Johnson algorithm, implemented in Boost.
    - `'Floyd-Warshall-Python'` – the Python implementation of the Floyd-Warshall algorithm. Usually slower than `'Johnson_Boost'`.
  - `None` (default): Sage chooses the best algorithm: `'BFS'` for unweighted graphs, `'Dijkstra_Boost'` if all weights are positive, `'Johnson_Boost'`, otherwise.
- `weight_function` – function (default: `None`); a function that takes as input an edge `(u, v, l)` and outputs its weight. If not `None`, `by_weight` is automatically set to `True`. If `None` and `by_weight` is `True`, we use the edge label `l` as a weight, if `l` is not `None`, else `1` as a weight.
- `check_weight` – boolean (default: `True`); if `True`, we check that the weight_function outputs a number for each edge

Note

Some algorithms (e.g., Boost algorithms) use floating point numbers for internal computations. Whenever the solution is integral, we try to convert the returned value to an integer.

EXAMPLES:

```
sage: G = Graph( { 0: {1: None}, 1: {2: None}, 2: {3: 1}, 3: {4: 2}, 4: {0: 2}
↪ }, sparse=True)
sage: G.wiener_index()
15
sage: G.wiener_index(weight_function=lambda e:(e[2] if e[2] is not None else␣
↪1))
20
sage: G.wiener_index(weight_function=lambda e:(e[2] if e[2] is not None else␣
↪200))
820
sage: G.wiener_index(algorithm='BFS')
15
sage: G.wiener_index(algorithm='Floyd-Warshall-Cython')
15
sage: G.wiener_index(algorithm='Floyd-Warshall-Python')
15
sage: G.wiener_index(algorithm='Dijkstra_Boost')
```

(continues on next page)

(continued from previous page)

```
15
sage: G.wiener_index(algorithm='Bellman-Ford_Boost')
15
sage: G.wiener_index(algorithm='Johnson_Boost')
15
sage: G.wiener_index(algorithm='Dijkstra_NetworkX')                          #␣
↪needs networkx
15
```

```
>>> from sage.all import *
>>> G = Graph( { Integer(0): {Integer(1): None}, Integer(1): {Integer(2):␣
↪None}, Integer(2): {Integer(3): Integer(1)}, Integer(3): {Integer(4):␣
↪Integer(2)}, Integer(4): {Integer(0): Integer(2)} }, sparse=True)
>>> G.wiener_index()
15
>>> G.wiener_index(weight_function=lambda e:(e[Integer(2)] if e[Integer(2)]␣
↪is not None else Integer(1)))
20
>>> G.wiener_index(weight_function=lambda e:(e[Integer(2)] if e[Integer(2)]␣
↪is not None else Integer(200)))
820
>>> G.wiener_index(algorithm='BFS')
15
>>> G.wiener_index(algorithm='Floyd-Warshall-Cython')
15
>>> G.wiener_index(algorithm='Floyd-Warshall-Python')
15
>>> G.wiener_index(algorithm='Dijkstra_Boost')
15
>>> G.wiener_index(algorithm='Bellman-Ford_Boost')
15
>>> G.wiener_index(algorithm='Johnson_Boost')
15
>>> G.wiener_index(algorithm='Dijkstra_NetworkX')                            #␣
↪needs networkx
15
```

Wiener index of complete (di)graphs:

```
sage: n = 5
sage: g = graphs.CompleteGraph(n)
sage: g.wiener_index() == (n * (n - 1)) / 2
True
sage: g = digraphs.Complete(n)
sage: g.wiener_index() == n * (n - 1)
True
```

```
>>> from sage.all import *
>>> n = Integer(5)
>>> g = graphs.CompleteGraph(n)
>>> g.wiener_index() == (n * (n - Integer(1))) / Integer(2)
True
```

(continues on next page)

(continued from previous page)

```
>>> g = digraphs.Complete(n)
>>> g.wiener_index() == n * (n - Integer(1))
True
```

Wiener index of circuit digraphs:

```
sage: n = 7
sage: g = digraphs.Circuit(n)
sage: w = lambda x: (x*x*(x-1))/2
sage: g.wiener_index(algorithm='Dijkstra_Boost') == w(n)
True
```

```
>>> from sage.all import *
>>> n = Integer(7)
>>> g = digraphs.Circuit(n)
>>> w = lambda x: (x*x*(x-Integer(1)))/Integer(2)
>>> g.wiener_index(algorithm='Dijkstra_Boost') == w(n)
True
```

Wiener index of a graph of order 1:

```
sage: Graph(1).wiener_index()
0
```

```
>>> from sage.all import *
>>> Graph(Integer(1)).wiener_index()
0
```

The Wiener index is not defined on the empty graph:

```
sage: Graph().wiener_index()
Traceback (most recent call last):
...
ValueError: Wiener index is not defined for the empty graph
```

```
>>> from sage.all import *
>>> Graph().wiener_index()
Traceback (most recent call last):
...
ValueError: Wiener index is not defined for the empty graph
```

sage.graphs.generic_graph.**graph_isom_equivalent_non_edge_labeled_graph**(*g*, *partition=None*, *standard_label=None*, *return_relabeling=False*, *return_edge_labels=False*, *inplace=False*, *ignore_edge_labels=False*, *immutable=None*)

Helper function for canonical labeling of edge labeled (di)graphs.

Translates to a bipartite incidence-structure type graph appropriate for computing canonical labels of edge labeled and/or multi-edge graphs. Note that this is actually computationally equivalent to implementing a change on an inner loop of the main algorithm – namely making the refinement procedure sort for each label.

If the graph is a multigraph, it is translated to a non-multigraph, where each instance of multiple edges is converted to a single edge labeled with a list `[[label1, multiplicity], [label2, multiplicity], ...]` describing how many edges of each label were originally there. Then in either case we are working on a graph without multiple edges. At this point, we create another (partially bipartite) graph, whose left vertices are the original vertices of the graph, and whose right vertices represent the labeled edges. Any unlabeled edges in the original graph are also present in the new graph, and – this is the bipartite aspect – for every labeled edge e from v to w in the original graph, there is an edge between the right vertex corresponding to e and each of the left vertices corresponding to v and w. We partition the left vertices as they were originally, and the right vertices by common labels: only automorphisms taking edges to like-labeled edges are allowed, and this additional partition information enforces this on the new graph.

INPUT:

- `g` – Graph or DiGraph
- `partition` – list (default: `None`); a partition of the vertices as a list of lists of vertices. If given, the partition of the vertices is as well relabeled
- `standard_label` – (default: `None`) edges in `g` with this label are preserved in the new graph
- `return_relabeling` – boolean (default: `False`); whether to return a dictionary containing the relabeling
- `return_edge_labels` – boolean (default: `False`); whether the different `edge_labels` are returned (useful if inplace is `True`)
- `inplace` – boolean (default: `False`); whether the input (di)graph `g` is modified or the return a new (di)graph. Note that attributes of `g` are *not* copied for speed issues, only edges and vertices.

  This parameter cannot be set to `True` if the input graph `g` is immutable.
- `ignore_edge_labels` – boolean (default: `False`); if `True`, ignore edge labels, so when constructing the new graph, only multiple edges are replaced with vertices. Labels on multiple edges are ignored – only the multiplicity is relevant, so multiple edges with the same multiplicity in the original graph correspond to right vertices in the same partition in the new graph.
- `immutable` – boolean (default: `None`); whether to create a mutable/immutable (di)graph. `immutable=None` (default) means that the (di)graph and the returned (di)graph will behave the same way.

  This parameter is ignored when `inplace` is `True`.

OUTPUT:

- if `inplace` is `False`: the unlabeled graph without multiple edges
- the partition of the vertices
- if `return_relabeling` is `True`: a dictionary containing the relabeling
- if `return_edge_labels` is `True`: the list of (former) edge labels is returned

EXAMPLES:

```
sage: from sage.graphs.generic_graph import graph_isom_equivalent_non_edge_
↪labeled_graph

sage: G = Graph(multiedges=True,sparse=True)
sage: G.add_edges((0, 1, i) for i in range(10))
sage: G.add_edge(1,2,'string')
```

(continues on next page)

(continued from previous page)

```
sage: G.add_edge(2,123)
sage: graph_isom_equivalent_non_edge_labeled_graph(G, partition=[[0,123],[1,2]])
[Graph on 6 vertices, [[1, 0], [2, 3], [5], [4]]]

sage: g, part = graph_isom_equivalent_non_edge_labeled_graph(G)
sage: g, sorted(part)
(Graph on 6 vertices, [[0, 1, 2, 3], [4], [5]])
sage: g.edges(sort=True)
[(0, 3, None), (1, 4, None), (2, 4, None), (2, 5, None), (3, 5, None)]

sage: g = graph_isom_equivalent_non_edge_labeled_graph(G, standard_label='string',
....:                                                  return_edge_labels=True)
sage: g[0]
Graph on 6 vertices
sage: g[0].edges(sort=True)
[(0, 5, None), (1, 4, None), (2, 3, None), (2, 4, None), (3, 5, None)]
sage: g[1]
[[0, 1, 2, 3], [4], [5]]
sage: g[2]
[[['string', 1]], [[0, 1], [1, 1], [2, 1], [3, 1], [4, 1], [5, 1], [6, 1], [7, 1],
 ↪ [8, 1], [9, 1]], [[None, 1]]]

sage: graph_isom_equivalent_non_edge_labeled_graph(G, inplace=True)
[[[0, 1, 2, 3], [5], [4]]]
sage: G.edges(sort=True)
[(0, 3, None), (1, 4, None), (2, 4, None), (2, 5, None), (3, 5, None)]

sage: G = Graph(multiedges=True,sparse=True)
sage: G.add_edges((0, 1) for i in range(10))
sage: G.add_edge(1, 2, 'a')
sage: G.add_edge(1, 3, 'b')
sage: G.add_edge(2, 3, 'b')
sage: graph_isom_equivalent_non_edge_labeled_graph(G)[0]
Graph on 8 vertices
sage: graph_isom_equivalent_non_edge_labeled_graph(G, ignore_edge_labels=True)[0]
Graph on 5 vertices

sage: G = Graph(multiedges=True,sparse=True)
sage: G.add_edges((0, 1, i) for i in range(5))
sage: G.add_edges((0, 2, i+10) for i in range(5))
sage: G.add_edges((0, 3) for i in range(4))
sage: g0 = graph_isom_equivalent_non_edge_labeled_graph(G)
sage: g1 = graph_isom_equivalent_non_edge_labeled_graph(G, ignore_edge_
 ↪labels=True)
sage: g0
[Graph on 7 vertices, [[0, 1, 2, 3], [4], [5], [6]]]
sage: g1
[Graph on 7 vertices, [[0, 1, 2, 3], [6], [4, 5]]]
```

```
>>> from sage.all import *
>>> from sage.graphs.generic_graph import graph_isom_equivalent_non_edge_labeled_
 ↪graph
```

(continues on next page)

(continued from previous page)

```
>>> G = Graph(multiedges=True,sparse=True)
>>> G.add_edges((Integer(0), Integer(1), i) for i in range(Integer(10)))
>>> G.add_edge(Integer(1),Integer(2),'string')
>>> G.add_edge(Integer(2),Integer(123))
>>> graph_isom_equivalent_non_edge_labeled_graph(G, partition=[[Integer(0),
↪Integer(123)],[Integer(1),Integer(2)]])
[Graph on 6 vertices, [[1, 0], [2, 3], [5], [4]]]

>>> g, part = graph_isom_equivalent_non_edge_labeled_graph(G)
>>> g, sorted(part)
(Graph on 6 vertices, [[0, 1, 2, 3], [4], [5]])
>>> g.edges(sort=True)
[(0, 3, None), (1, 4, None), (2, 4, None), (2, 5, None), (3, 5, None)]

>>> g = graph_isom_equivalent_non_edge_labeled_graph(G, standard_label='string',
...                                                  return_edge_labels=True)
>>> g[Integer(0)]
Graph on 6 vertices
>>> g[Integer(0)].edges(sort=True)
[(0, 5, None), (1, 4, None), (2, 3, None), (2, 4, None), (3, 5, None)]
>>> g[Integer(1)]
[[0, 1, 2, 3], [4], [5]]
>>> g[Integer(2)]
[[['string', 1]], [[0, 1], [1, 1], [2, 1], [3, 1], [4, 1], [5, 1], [6, 1], [7, 1],
↪ [8, 1], [9, 1]], [[None, 1]]]

>>> graph_isom_equivalent_non_edge_labeled_graph(G, inplace=True)
[[[0, 1, 2, 3], [5], [4]]]
>>> G.edges(sort=True)
[(0, 3, None), (1, 4, None), (2, 4, None), (2, 5, None), (3, 5, None)]

>>> G = Graph(multiedges=True,sparse=True)
>>> G.add_edges((Integer(0), Integer(1)) for i in range(Integer(10)))
>>> G.add_edge(Integer(1), Integer(2), 'a')
>>> G.add_edge(Integer(1), Integer(3), 'b')
>>> G.add_edge(Integer(2), Integer(3), 'b')
>>> graph_isom_equivalent_non_edge_labeled_graph(G)[Integer(0)]
Graph on 8 vertices
>>> graph_isom_equivalent_non_edge_labeled_graph(G, ignore_edge_
↪labels=True)[Integer(0)]
Graph on 5 vertices

>>> G = Graph(multiedges=True,sparse=True)
>>> G.add_edges((Integer(0), Integer(1), i) for i in range(Integer(5)))
>>> G.add_edges((Integer(0), Integer(2), i+Integer(10)) for i in␣
↪range(Integer(5)))
>>> G.add_edges((Integer(0), Integer(3)) for i in range(Integer(4)))
>>> g0 = graph_isom_equivalent_non_edge_labeled_graph(G)
>>> g1 = graph_isom_equivalent_non_edge_labeled_graph(G, ignore_edge_labels=True)
>>> g0
[Graph on 7 vertices, [[0, 1, 2, 3], [4], [5], [6]]]
```

(continues on next page)

(continued from previous page)

```
>>> g1
[Graph on 7 vertices, [[0, 1, 2, 3], [6], [4, 5]]]
```

sage.graphs.generic_graph.**tachyon_vertex_plot**(*g*, *bgcolor=(1, 1, 1)*, *vertex_colors=None*, *vertex_size=0.06*, *pos3d=None*, ***kwds*)

Helper function for plotting graphs in 3d with `Tachyon`.

Returns a plot containing only the vertices, as well as the 3d position dictionary used for the plot.

INPUT:

- `pos3d` – a 3D layout of the vertices
- various rendering options

EXAMPLES:

```
sage: G = graphs.TetrahedralGraph()
sage: from sage.graphs.generic_graph import tachyon_vertex_plot
sage: T,p = tachyon_vertex_plot(G, pos3d=G.layout(dim=3))                    #␣
↪needs sage.plot
sage: type(T)                                                                #␣
↪needs sage.plot
<class 'sage.plot.plot3d.tachyon.Tachyon'>
sage: type(p)                                                                #␣
↪needs sage.plot
<... 'dict'>
```

```
>>> from sage.all import *
>>> G = graphs.TetrahedralGraph()
>>> from sage.graphs.generic_graph import tachyon_vertex_plot
>>> T,p = tachyon_vertex_plot(G, pos3d=G.layout(dim=Integer(3)))              ␣
↪     # needs sage.plot
>>> type(T)                                                                 #␣
↪needs sage.plot
<class 'sage.plot.plot3d.tachyon.Tachyon'>
>>> type(p)                                                                 #␣
↪needs sage.plot
<... 'dict'>
```

1.2 Undirected graphs

This module implements functions and operations involving undirected graphs.

Algorithmically hard stuff

convexity_properties()	Return a `ConvexityProperties` object corresponding to `self`.
independent_set()	Return a maximum independent set.
independent_set_of_representatives()	Return an independent set of representatives.
is_perfect()	Test whether the graph is perfect.
matching_polynomial()	Compute the matching polynomial of the graph G.
minor()	Return the vertices of a minor isomorphic to H in the current graph.
pathwidth()	Compute the pathwidth of `G` (and provides a decomposition).
rank_decomposition()	Compute an optimal rank-decomposition of the given graph.
topological_minor()	Return a topological H-minor from `self` if one exists.
treelength()	Compute the treelength of G (and provide a decomposition).
treewidth()	Compute the treewidth of g (and provide a decomposition).
tutte_polynomial()	Return the Tutte polynomial of the graph G.
vertex_cover()	Return a minimum vertex cover of `self` represented by a set of vertices.

Basic methods

bipartite_color()	Return a dictionary with vertices as the keys and the color class as the values.
bipartite_double()	Return the (extended) bipartite double of this graph.
bipartite_sets()	Return (X, Y) where X and Y are the nodes in each bipartite set of graph G.
graph6_string()	Return the graph6 representation of the graph as an ASCII string.
is_directed()	Since graph is undirected, returns False.
join()	Return the join of `self` and `other`.
sparse6_string()	Return the sparse6 representation of the graph as an ASCII string.
to_directed()	Return a directed version of the graph.
to_undirected()	Since the graph is already undirected, simply returns a copy of itself.
write_to_eps()	Write a plot of the graph to `filename` in `eps` format.

Clique-related methods

all_cliques()	Iterator over the cliques in `graph`.
atoms_and_clique_separators()	Return the atoms of the decomposition of G by clique minimal separators.
clique_complex()	Return the clique complex of `self`.
clique_maximum()	Return the vertex set of a maximal order complete subgraph.
clique_number()	Return the order of the largest clique of the graph.
clique_polynomial()	Return the clique polynomial of `self`.
cliques_containing_vertex()	Return the cliques containing each vertex, represented as a dictionary of lists of lists, keyed by vertex.
cliques_get_clique_bipartite()	Return the vertex-clique bipartite graph of `self`.
cliques_get_max_clique_gr	Return the clique graph.
cliques_maximal()	Return the list of all maximal cliques.
cliques_maximum()	Return the vertex sets of *ALL* the maximum complete subgraphs.
cliques_number_of()	Return a dictionary of the number of maximal cliques containing each vertex, keyed by vertex.
cliques_vertex_clique_number()	Return a dictionary of sizes of the largest maximal cliques containing each vertex, keyed by vertex.
fractional_clique_number()	Return the fractional clique number of the graph.

Coloring

chromatic_index()	Return the chromatic index of the graph.
chromatic_number()	Return the minimal number of colors needed to color the vertices of the graph.
chromatic_polynomial()	Compute the chromatic polynomial of the graph G.
chromatic_quasisymmetric_function()	Return the chromatic quasisymmetric function of `self`.
chromatic_symmetric_function()	Return the chromatic symmetric function of `self`.
coloring()	Return the first (optimal) proper vertex-coloring found.
fractional_chromatic_index()	Return the fractional chromatic index of the graph.
fractional_chromatic_number()	Return the fractional chromatic number of the graph.
tutte_symmetric_function()	Return the Tutte symmetric function of `self`.

Connectivity, orientations, trees

acyclic_orientations()	Return an iterator over all acyclic orientations of an undirected graph G.
bounded_outdegree_orientation()	Return an orientation of G such that every vertex v has out-degree less than $b(v)$.
bridges()	Return an iterator over the bridges (or cut edges).
cleave()	Return the connected subgraphs separated by the input vertex cut.
degree_constrained_subgraph()	Return a degree-constrained subgraph.
ear_decomposition()	Return an Ear decomposition of the graph.
eulerian_orientation()	Return an Eulerian orientation of the graph G.
gomory_hu_tree()	Return a Gomory-Hu tree of `self`.
is_triconnected()	Check whether the graph is triconnected.
minimal_separators()	Return an iterator over the minimal separators of `G`.
minimum_outdegree_orientation()	Return an orientation of G with the smallest possible maximum outdegree.
orient()	Return an oriented version of G according the input function f.
orientations()	Return an iterator over orientations of G.
random_orientation()	Return a random orientation of a graph G.
random_spanning_tree()	Return a random spanning tree of the graph.
spanning_trees()	Return an iterator over all spanning trees of the graph g.
spqr_tree()	Return an SPQR-tree representing the triconnected components of the graph.
strong_orientation()	Return a strongly connected orientation of the graph G.
strong_orientations_iterator()	Return an iterator over all strong orientations of a graph G.

Distances

center()	Return the set of vertices in the center of the graph.
centrality_degree()	Return the degree centrality of a vertex.
diameter()	Return the diameter of the graph.
distance_graph()	Return the graph on the same vertex set as the original graph but vertices are adjacent in the returned graph if and only if they are at specified distances in the original graph.
eccentricity()	Return the eccentricity of vertex (or vertices) `v`.
hyperbolicity()	Return the hyperbolicity of the graph or an approximation of this value.
periphery()	Return the set of vertices in the periphery of the graph.
radius()	Return the radius of the graph.

Domination

is_dominating()	Check whether `dom` is a dominating set of `G`.
is_redundant()	Check whether `dom` has redundant vertices.
minimal_dominating_sets()	Return an iterator over the minimal dominating sets of a graph.
private_neighbors()	Return the private neighbors of a vertex with respect to other vertices.

Expansion properties

cheeger_constant()	Return the cheeger constant of the graph.
edge_isoperimetric_number()	Return the edge-isoperimetric number of the graph.
vertex_isoperimetric_number()	Return the vertex-isoperimetric number of the graph.

Graph properties

apex_vertices()	Return the list of apex vertices.
is_antipodal()	Check whether this graph is antipodal.
is_apex()	Test if the graph is apex.
is_arc_transitive()	Check if `self` is an arc-transitive graph.
is_asteroidal_triple_free()	Test if the input graph is asteroidal triple-free.
is_block_graph()	Return whether this graph is a block graph.
is_cactus()	Check whether the graph is cactus graph.
is_cartesian_product()	Test whether the graph is a Cartesian product.
is_chordal_bipartite()	Check whether the given graph is chordal bipartite.
is_circumscribable()	Test whether the graph is the graph of a circumscribed polyhedron.
is_cograph()	Check whether the graph is cograph.
is_comparability()	Check whether the graph is a comparability graph.
is_distance_regular()	Test if the graph is distance-regular.
is_edge_transitive()	Check if `self` is an edge transitive graph.
is_even_hole_free()	Test whether `self` contains an induced even hole.
is_forest()	Test if the graph is a forest, i.e. a disjoint union of trees.
is_half_transitive()	Check if `self` is a half-transitive graph.
is_inscribable()	Test whether the graph is the graph of an inscribed polyhedron.
is_line_graph()	Check whether the graph g is a line graph.

continues on next page

Table 2 – continued from previous page

is_long_antihole_free()	Test whether the given graph contains an induced subgraph that is isomorphic to the complement of a cycle of length at least 5.
is_long_hole_free()	Test whether `g` contains an induced cycle of length at least 5.
is_odd_hole_free()	Test whether `self` contains an induced odd hole.
is_overfull()	Test whether the current graph is overfull.
is_partial_cube()	Test whether the given graph is a partial cube.
is_path()	Check whether `self` is a path.
is_permutation()	Check whether the graph is a permutation graph.
is_polyhedral()	Check whether the graph is the graph of the polyhedron.
is_prime()	Test whether the current graph is prime.
is_semi_symmetric()	Check if `self` is semi-symmetric.
is_split()	Return `True` if the graph is a Split graph, `False` otherwise.
is_strongly_regular()	Check whether the graph is strongly regular.
is_tree()	Test if the graph is a tree.
is_triangle_free()	Check whether `self` is triangle-free.
is_weakly_chordal()	Test whether the given graph is weakly chordal, i.e., the graph and its complement have no induced cycle of length at least 5.

Leftovers

antipodal_graph()	Return the antipodal graph of G.
arboricity()	Return the arboricity of the graph and an optional certificate.
common_neighbors_matrix()	Return a matrix of numbers of common neighbors between each pairs.
cores()	Return the core number for each vertex in an ordered list.
effective_resistance()	Return the effective resistance between nodes i and j.
effective_resistance_matrix()	Return a matrix whose (i , j) entry gives the effective resistance between vertices i and j.
folded_graph()	Return the antipodal fold of this graph.
geodetic_closure()	Return the geodetic closure of the set of vertices S in G.
ihara_zeta_function_inverse()	Compute the inverse of the Ihara zeta function of the graph.
kirchhoff_symanzik_polynomial()	Return the Kirchhoff-Symanzik polynomial of a graph.
least_effective_resistance()	Return a list of pairs of nodes with the least effective resistance.
lovasz_theta()	Return the value of Lovász theta-function of graph.
magnitude_function()	Return the magnitude function of the graph as a rational function.
maximum_average_degree()	Return the Maximum Average Degree (MAD) of the current graph.
most_common_neighbors()	Return vertex pairs with maximal number of common neighbors.
seidel_adjacency_matrix()	Return the Seidel adjacency matrix of `self`.
seidel_switching()	Return the Seidel switching of `self` w.r.t. subset of vertices `s`.
two_factor_petersen()	Return a decomposition of the graph into 2-factors.
twograph()	Return the two-graph of `self`.

Matching

has_perfect_matching()	Return whether the graph has a perfect matching.
is_bicritical()	Check if the graph is bicritical.
is_factor_critical()	Check whether the graph is factor-critical.
is_matching_covered()	Check if the graph is matching covered.
matching()	Return a maximum weighted matching of the graph represented by the list of its edges.
perfect_matchings()	Return an iterator over all perfect matchings of the graph.

Modules

is_module()	Check whether `vertices` is a module of `self`.
modular_decomposition()	Return the modular decomposition of the current graph.

Traversals

lex_M()	Return an ordering of the vertices according the LexM graph traversal.
maximum_cardinality_search()	Return an ordering of the vertices according a maximum cardinality search.
maximum_cardinality_search_M()	Return the ordering and the edges of the triangulation produced by MCS-M.

Unsorted

bandwidth()	Compute the bandwidth of an undirected graph.
cutwidth()	Return the cutwidth of the graph and the corresponding vertex ordering.
slice_decomposition()	Compute a slice decomposition of the simple undirected graph

AUTHORS:

- Robert L. Miller (2006-10-22): initial version
- William Stein (2006-12-05): Editing
- **Robert L. Miller (2007-01-13): refactoring, adjusting for NetworkX-0.33, fixed** plotting bugs (2007-01-23): basic tutorial, edge labels, loops, multiple edges and arcs (2007-02-07): graph6 and sparse6 formats, matrix input
- Emily Kirkmann (2007-02-11): added graph_border option to plot and show
- **Robert L. Miller (2007-02-12): vertex color-maps, graph boundaries, graph6** helper functions in Cython
- Robert L. Miller Sage Days 3 (2007-02-17-21): 3d plotting in Tachyon
- Robert L. Miller (2007-02-25): display a partition
- **Robert L. Miller (2007-02-28): associate arbitrary objects to vertices, edge** and arc label display (in 2d), edge coloring
- **Robert L. Miller (2007-03-21): Automorphism group, isomorphism check,** canonical label
- Robert L. Miller (2007-06-07-09): NetworkX function wrapping
- Michael W. Hansen (2007-06-09): Topological sort generation

- Emily Kirkman, Robert L. Miller Sage Days 4: Finished wrapping NetworkX
- **Emily Kirkman (2007-07-21): Genus (including circular planar, all embeddings** and all planar embeddings), all paths, interior paths
- **Bobby Moretti (2007-08-12): fixed up plotting of graphs with edge colors** differentiated by label
- Jason Grout (2007-09-25): Added functions, bug fixes, and general enhancements
- Robert L. Miller (Sage Days 7): Edge labeled graph isomorphism
- Tom Boothby (Sage Days 7): Miscellaneous awesomeness
- Tom Boothby (2008-01-09): Added graphviz output
- David Joyner (2009-2): Fixed docstring bug related to GAP.
- **Stephen Hartke (2009-07-26): Fixed bug in blocks_and_cut_vertices() that** caused an incorrect result when the vertex 0 was a cut vertex.
- **Stephen Hartke (2009-08-22): Fixed bug in blocks_and_cut_vertices() where the** list of cut_vertices is not treated as a set.
- Anders Jonsson (2009-10-10): Counting of spanning trees and out-trees added.
- **Nathann Cohen (2009-09)** [Cliquer, Connectivity, Flows and everything that] uses Linear Programming and class numerical.MIP
- Nicolas M. Thiery (2010-02): graph layout code refactoring, dot2tex/graphviz interface
- David Coudert (2012-04) : Reduction rules in vertex_cover.
- **Birk Eisermann (2012-06): added recognition of weakly chordal graphs and** long-hole-free / long-antihole-free graphs
- Alexandre P. Zuge (2013-07): added join operation.
- Amritanshu Prasad (2014-08): added clique polynomial
- Julian Rüth (2018-06-21): upgrade to NetworkX 2
- David Coudert (2018-10-07): cleaning
- Amanda Francis, Caitlin Lienkaemper, Kate Collins, Rajat Mittal (2019-03-10): methods for computing effective resistance
- Amanda Francis, Caitlin Lienkaemper, Kate Collins, Rajat Mittal (2019-03-19): most_common_neighbors and common_neighbors_matrix added.
- **Jean-Florent Raymond (2019-04): is_redundant, is_dominating,** private_neighbors
- Cyril Bouvier (2024-11): is_module

1.2.1 Graph Format

Supported formats

Sage Graphs can be created from a wide range of inputs. A few examples are covered here.

- NetworkX dictionary format:

```
sage: d = {0: [1,4,5], 1: [2,6], 2: [3,7], 3: [4,8], 4: [9], \
....: 5: [7, 8], 6: [8,9], 7: [9]}
sage: G = Graph(d); G
Graph on 10 vertices
sage: G.plot().show()    # or G.show()
↪       # needs sage.plot
```

```
>>> from sage.all import *
>>> d = {Integer(0): [Integer(1),Integer(4),Integer(5)], Integer(1):
↪[Integer(2),Integer(6)], Integer(2): [Integer(3),Integer(7)],
↪Integer(3): [Integer(4),Integer(8)], Integer(4): [Integer(9)],
↪Integer(5): [Integer(7), Integer(8)], Integer(6): [Integer(8),
↪Integer(9)], Integer(7): [Integer(9)]}
>>> G = Graph(d); G
Graph on 10 vertices
>>> G.plot().show()    # or G.show()
↪     # needs sage.plot
```

- A NetworkX graph:

```
sage: # needs networkx
sage: import networkx
sage: K = networkx.complete_bipartite_graph(12,7)
sage: G = Graph(K)
sage: G.degree()
[7, 7, 7, 7, 7, 7, 7, 7, 7, 7, 7, 7, 12, 12, 12, 12, 12, 12, 12]
```

```
>>> from sage.all import *
>>> # needs networkx
>>> import networkx
>>> K = networkx.complete_bipartite_graph(Integer(12),Integer(7))
>>> G = Graph(K)
>>> G.degree()
[7, 7, 7, 7, 7, 7, 7, 7, 7, 7, 7, 7, 12, 12, 12, 12, 12, 12, 12]
```

- graph6 or sparse6 format:

```
sage: s = ':I`AKGsaOs`cI]Gb~'
sage: G = Graph(s, sparse=True); G
Looped multi-graph on 10 vertices
sage: G.plot().show()    # or G.show()
↪       # needs sage.plot
```

```
>>> from sage.all import *
>>> s = ':I`AKGsaOs`cI]Gb~'
>>> G = Graph(s, sparse=True); G
Looped multi-graph on 10 vertices
>>> G.plot().show()    # or G.show()
↪     # needs sage.plot
```

Note that the \ character is an escape character in Python, and also a character used by graph6 strings:

```
sage: G = Graph('Ihe\n@GUA')
Traceback (most recent call last):
...
RuntimeError: the string (Ihe) seems corrupt: for n = 10, the string is␣
↪too short
```

```
>>> from sage.all import *
>>> G = Graph('Ihe\n@GUA')
Traceback (most recent call last):
...
RuntimeError: the string (Ihe) seems corrupt: for n = 10, the string is␣
↪too short
```

In Python, the escaped character \ is represented by \\:

```
sage: G = Graph('Ihe\\n@GUA')
sage: G.plot().show()     # or G.show()                                    ␣
↪        # needs sage.plot
```

```
>>> from sage.all import *
>>> G = Graph('Ihe\\n@GUA')
>>> G.plot().show()     # or G.show()                                      ␣
↪     # needs sage.plot
```

- **adjacency matrix: In an adjacency matrix, each column and each row represent a** vertex. If a 1 shows up in row i, column j, there is an edge (i, j).

```
sage: M = Matrix([(0,1,0,0,1,1,0,0,0,0), (1,0,1,0,0,0,1,0,0,0),
....:             (0,1,0,1,0,0,0,1,0,0), (0,0,1,0,1,0,0,0,1,0),
....:             (1,0,0,1,0,0,0,0,0,1), (1,0,0,0,0,0,0,1,1,0), (0,1,0,0,0,0,
↪0,0,1,1),
....:             (0,0,1,0,0,1,0,0,0,1), (0,0,0,1,0,1,1,0,0,0), (0,0,0,0,1,0,
↪1,1,0,0)])
sage: M
[0 1 0 0 1 1 0 0 0 0]
[1 0 1 0 0 0 1 0 0 0]
[0 1 0 1 0 0 0 1 0 0]
[0 0 1 0 1 0 0 0 1 0]
[1 0 0 1 0 0 0 0 0 1]
[1 0 0 0 0 0 0 1 1 0]
[0 1 0 0 0 0 0 0 1 1]
[0 0 1 0 0 1 0 0 0 1]
[0 0 0 1 0 1 1 0 0 0]
[0 0 0 0 1 0 1 1 0 0]
sage: G = Graph(M); G
Graph on 10 vertices
sage: G.plot().show()     # or G.show()                                    ␣
↪    # needs sage.plot
```

```
>>> from sage.all import *
>>> M = Matrix([(Integer(0),Integer(1),Integer(0),Integer(0),Integer(1),
↪Integer(1),Integer(0),Integer(0),Integer(0),Integer(0)), (Integer(1),
```

(continues on next page)

(continued from previous page)

```
↪Integer(0),Integer(1),Integer(0),Integer(0),Integer(0),Integer(1),
↪Integer(0),Integer(0),Integer(0)),
...              (Integer(0),Integer(1),Integer(0),Integer(1),Integer(0),
↪Integer(0),Integer(0),Integer(1),Integer(0),Integer(0)), (Integer(0),
↪Integer(0),Integer(1),Integer(0),Integer(1),Integer(0),Integer(0),
↪Integer(0),Integer(1),Integer(0)),
...              (Integer(1),Integer(0),Integer(0),Integer(1),Integer(0),
↪Integer(0),Integer(0),Integer(0),Integer(0),Integer(1)), (Integer(1),
↪Integer(0),Integer(0),Integer(0),Integer(0),Integer(0),Integer(0),
↪Integer(1),Integer(1),Integer(0)), (Integer(0),Integer(1),Integer(0),
↪Integer(0),Integer(0),Integer(0),Integer(0),Integer(0),Integer(1),
↪Integer(1)),
...              (Integer(0),Integer(0),Integer(1),Integer(0),Integer(0),
↪Integer(1),Integer(0),Integer(0),Integer(0),Integer(1)), (Integer(0),
↪Integer(0),Integer(0),Integer(1),Integer(0),Integer(1),Integer(1),
↪Integer(0),Integer(0),Integer(0)), (Integer(0),Integer(0),Integer(0),
↪Integer(0),Integer(1),Integer(0),Integer(1),Integer(1),Integer(0),
↪Integer(0))])
>>> M
[0 1 0 0 1 1 0 0 0 0]
[1 0 1 0 0 0 1 0 0 0]
[0 1 0 1 0 0 0 1 0 0]
[0 0 1 0 1 0 0 0 1 0]
[1 0 0 1 0 0 0 0 0 1]
[1 0 0 0 0 0 0 1 1 0]
[0 1 0 0 0 0 0 0 1 1]
[0 0 1 0 0 1 0 0 0 1]
[0 0 0 1 0 1 1 0 0 0]
[0 0 0 0 1 0 1 1 0 0]
>>> G = Graph(M); G
Graph on 10 vertices
>>> G.plot().show()    # or G.show()
↪ # needs sage.plot
```

- **incidence matrix: In an incidence matrix, each row represents a vertex and** each column represents an edge.

```
sage: M = Matrix([(-1, 0, 0, 0, 1, 0, 0, 0, 0, 0,-1, 0, 0, 0, 0),
....:             ( 1,-1, 0, 0, 0, 0, 0, 0, 0, 0, 0,-1, 0, 0, 0),
....:             ( 0, 1,-1, 0, 0, 0, 0, 0, 0, 0, 0, 0,-1, 0, 0),
....:             ( 0, 0, 1,-1, 0, 0, 0, 0, 0, 0, 0, 0, 0,-1, 0),
....:             ( 0, 0, 0, 1,-1, 0, 0, 0, 0, 0, 0, 0, 0, 0,-1),
....:             ( 0, 0, 0, 0, 0,-1, 0, 0, 0, 1, 1, 0, 0, 0, 0),
....:             ( 0, 0, 0, 0, 0, 0, 0, 1,-1, 0, 0, 1, 0, 0, 0),
....:             ( 0, 0, 0, 0, 0, 1,-1, 0, 0, 0, 0, 0, 1, 0, 0),
....:             ( 0, 0, 0, 0, 0, 0, 0, 0, 1,-1, 0, 0, 0, 1, 0),
....:             ( 0, 0, 0, 0, 0, 0, 1,-1, 0, 0, 0, 0, 0, 0, 1)])
sage: M
[-1  0  0  0  1  0  0  0  0  0 -1  0  0  0  0]
[ 1 -1  0  0  0  0  0  0  0  0  0 -1  0  0  0]
[ 0  1 -1  0  0  0  0  0  0  0  0  0 -1  0  0]
[ 0  0  1 -1  0  0  0  0  0  0  0  0  0 -1  0]
```

(continues on next page)

(continued from previous page)

```
[ 0  0  0  1 -1  0  0  0  0  0  0  0  0 -1]
[ 0  0  0  0  0 -1  0  0  0  1  1  0  0  0  0]
[ 0  0  0  0  0  0  0  1 -1  0  0  1  0  0  0]
[ 0  0  0  0  0  1 -1  0  0  0  0  0  1  0  0]
[ 0  0  0  0  0  0  0  0  1 -1  0  0  0  1  0]
[ 0  0  0  0  0  0  1 -1  0  0  0  0  0  0  1]
sage: G = Graph(M); G
Graph on 10 vertices
sage: G.plot().show()     # or G.show()
↪    # needs sage.plot
sage: DiGraph(matrix(2, [0,0,-1,1]), format='incidence_matrix')
Traceback (most recent call last):
...
ValueError: there must be two nonzero entries (-1 & 1) per column
```

```
>>> from sage.all import *
>>> M = Matrix([(-Integer(1), Integer(0), Integer(0), Integer(0), Integer(1),
↪Integer(0), Integer(0), Integer(0), Integer(0), Integer(0),-Integer(1),
↪Integer(0), Integer(0), Integer(0), Integer(0)),
...              ( Integer(1),-Integer(1), Integer(0), Integer(0), Integer(0),
↪Integer(0), Integer(0), Integer(0), Integer(0), Integer(0), Integer(0),-
↪Integer(1), Integer(0), Integer(0), Integer(0)),
...              ( Integer(0), Integer(1),-Integer(1), Integer(0), Integer(0),
↪Integer(0), Integer(0), Integer(0), Integer(0), Integer(0), Integer(0),
↪Integer(0),-Integer(1), Integer(0), Integer(0)),
...              ( Integer(0), Integer(0), Integer(1),-Integer(1), Integer(0),
↪Integer(0), Integer(0), Integer(0), Integer(0), Integer(0), Integer(0),
↪Integer(0), Integer(0),-Integer(1), Integer(0)),
...              ( Integer(0), Integer(0), Integer(0), Integer(1),-Integer(1),
↪Integer(0), Integer(0), Integer(0), Integer(0), Integer(0), Integer(0),
↪Integer(0), Integer(0), Integer(0),-Integer(1)),
...              ( Integer(0), Integer(0), Integer(0), Integer(0), Integer(0),-
↪Integer(1), Integer(0), Integer(0), Integer(0), Integer(1), Integer(1),
↪Integer(0), Integer(0), Integer(0), Integer(0)),
...              ( Integer(0), Integer(0), Integer(0), Integer(0), Integer(0),
↪Integer(0), Integer(0), Integer(1),-Integer(1), Integer(0), Integer(0),
↪Integer(1), Integer(0), Integer(0), Integer(0)),
...              ( Integer(0), Integer(0), Integer(0), Integer(0), Integer(0),
↪Integer(1),-Integer(1), Integer(0), Integer(0), Integer(0), Integer(0),
↪Integer(0), Integer(1), Integer(0), Integer(0)),
...              ( Integer(0), Integer(0), Integer(0), Integer(0), Integer(0),
↪Integer(0), Integer(0), Integer(0), Integer(1),-Integer(1), Integer(0),
↪Integer(0), Integer(0), Integer(1), Integer(0)),
...              ( Integer(0), Integer(0), Integer(0), Integer(0), Integer(0),
↪Integer(0), Integer(1),-Integer(1), Integer(0), Integer(0), Integer(0),
↪Integer(0), Integer(0), Integer(0), Integer(1))])
>>> M
[-1  0  0  0  1  0  0  0  0  0 -1  0  0  0  0]
[ 1 -1  0  0  0  0  0  0  0  0  0 -1  0  0  0]
[ 0  1 -1  0  0  0  0  0  0  0  0  0 -1  0  0]
[ 0  0  1 -1  0  0  0  0  0  0  0  0  0 -1  0]
[ 0  0  0  1 -1  0  0  0  0  0  0  0  0  0 -1]
```

(continues on next page)

(continued from previous page)

```
[ 0  0  0  0  0 -1  0  0  0  1  1  0  0  0  0]
[ 0  0  0  0  0  0  0  1 -1  0  0  1  0  0  0]
[ 0  0  0  0  0  1 -1  0  0  0  0  0  1  0  0]
[ 0  0  0  0  0  0  0  0  1 -1  0  0  0  1  0]
[ 0  0  0  0  0  0  1 -1  0  0  0  0  0  0  1]
>>> G = Graph(M); G
Graph on 10 vertices
>>> G.plot().show()     # or G.show()
↪ # needs sage.plot
>>> DiGraph(matrix(Integer(2), [Integer(0),Integer(0),-Integer(1),
↪Integer(1)]), format='incidence_matrix')
Traceback (most recent call last):
...
ValueError: there must be two nonzero entries (-1 & 1) per column
```

- a list of edges:

```
sage: g = Graph([(1, 3), (3, 8), (5, 2)]); g
Graph on 5 vertices
```

```
>>> from sage.all import *
>>> g = Graph([(Integer(1), Integer(3)), (Integer(3), Integer(8)), (Integer(5),
↪Integer(2))]); g
Graph on 5 vertices
```

- an igraph Graph:

```
sage: import igraph                                    # optional - python_igraph
sage: g = Graph(igraph.Graph([(1,3),(3,2),(0,2)]))    # optional - python_igraph
sage: g                                                # optional - python_igraph
Graph on 4 vertices
```

```
>>> from sage.all import *
>>> import igraph                                      # optional - python_igraph
>>> g = Graph(igraph.Graph([(Integer(1),Integer(3)),(Integer(3),Integer(2)),
↪(Integer(0),Integer(2))]))   # optional - python_igraph
>>> g                                                  # optional - python_igraph
Graph on 4 vertices
```

1.2.2 Generators

Use `graphs(n)` to iterate through all non-isomorphic graphs of given size:

```
sage: for g in graphs(4):
....:     print(g.degree_sequence())
[0, 0, 0, 0]
[1, 1, 0, 0]
[2, 1, 1, 0]
[3, 1, 1, 1]
[1, 1, 1, 1]
[2, 2, 1, 1]
[2, 2, 2, 0]
```

(continues on next page)

(continued from previous page)

```
[3, 2, 2, 1]
[2, 2, 2, 2]
[3, 3, 2, 2]
[3, 3, 3, 3]
```

```
>>> from sage.all import *
>>> for g in graphs(Integer(4)):
...     print(g.degree_sequence())
[0, 0, 0, 0]
[1, 1, 0, 0]
[2, 1, 1, 0]
[3, 1, 1, 1]
[1, 1, 1, 1]
[2, 2, 1, 1]
[2, 2, 2, 0]
[3, 2, 2, 1]
[2, 2, 2, 2]
[3, 3, 2, 2]
[3, 3, 3, 3]
```

Similarly `graphs()` will iterate through all graphs. The complete graph of 4 vertices is of course the smallest graph with chromatic number bigger than three:

```
sage: for g in graphs():
....:     if g.chromatic_number() > 3:
....:         break
sage: g.is_isomorphic(graphs.CompleteGraph(4))
True
```

```
>>> from sage.all import *
>>> for g in graphs():
...     if g.chromatic_number() > Integer(3):
...         break
>>> g.is_isomorphic(graphs.CompleteGraph(Integer(4)))
True
```

For some commonly used graphs to play with, type:

```
sage: graphs.[tab]          # not tested
```

```
>>> from sage.all import *
>>> graphs.[tab]          # not tested
```

and hit {tab}. Most of these graphs come with their own custom plot, so you can see how people usually visualize these graphs.

```
sage: G = graphs.PetersenGraph()
sage: G.plot().show()     # or G.show()                                    #␣
↪needs sage.plot
sage: G.degree_histogram()
[0, 0, 0, 10]
sage: G.adjacency_matrix()                                                #␣
```

(continues on next page)

(continued from previous page)

```
↪needs sage.modules
[0 1 0 0 1 1 0 0 0 0]
[1 0 1 0 0 0 1 0 0 0]
[0 1 0 1 0 0 0 1 0 0]
[0 0 1 0 1 0 0 0 1 0]
[1 0 0 1 0 0 0 0 0 1]
[1 0 0 0 0 0 0 1 1 0]
[0 1 0 0 0 0 0 0 1 1]
[0 0 1 0 0 1 0 0 0 1]
[0 0 0 1 0 1 1 0 0 0]
[0 0 0 0 1 0 1 1 0 0]
```

```
>>> from sage.all import *
>>> G = graphs.PetersenGraph()
>>> G.plot().show()     # or G.show()                                    #␣
↪needs sage.plot
>>> G.degree_histogram()
[0, 0, 0, 10]
>>> G.adjacency_matrix()                                                #␣
↪needs sage.modules
[0 1 0 0 1 1 0 0 0 0]
[1 0 1 0 0 0 1 0 0 0]
[0 1 0 1 0 0 0 1 0 0]
[0 0 1 0 1 0 0 0 1 0]
[1 0 0 1 0 0 0 0 0 1]
[1 0 0 0 0 0 0 1 1 0]
[0 1 0 0 0 0 0 0 1 1]
[0 0 1 0 0 1 0 0 0 1]
[0 0 0 1 0 1 1 0 0 0]
[0 0 0 0 1 0 1 1 0 0]
```

```
sage: S = G.subgraph([0,1,2,3])
sage: S.plot().show()     # or S.show()                                  #␣
↪needs sage.plot
sage: S.density()
1/2
```

```
>>> from sage.all import *
>>> S = G.subgraph([Integer(0),Integer(1),Integer(2),Integer(3)])
>>> S.plot().show()     # or S.show()                                    #␣
↪needs sage.plot
>>> S.density()
1/2
```

```
sage: G = GraphQuery(display_cols=['graph6'], num_vertices=7, diameter=5)
sage: L = G.get_graphs_list()
sage: graphs_list.show_graphs(L)                                         #␣
↪needs sage.plot
```

```
>>> from sage.all import *
>>> G = GraphQuery(display_cols=['graph6'], num_vertices=Integer(7),␣
```

(continues on next page)

(continued from previous page)

```
↪diameter=Integer(5))
>>> L = G.get_graphs_list()
>>> graphs_list.show_graphs(L)                                                #␣
↪needs sage.plot
```

1.2.3 Labels

Each vertex can have any hashable object as a label. These are things like strings, numbers, and tuples. Each edge is given a default label of `None`, but if specified, edges can have any label at all. Edges between vertices u and v are represented typically as `(u, v, l)`, where `l` is the label for the edge.

Note that vertex labels themselves cannot be mutable items:

```
sage: M = Matrix([[0,0], [0,0]])                                              #␣
↪needs sage.modules
sage: G = Graph({ 0 : { M : None } })                                         #␣
↪needs sage.modules
Traceback (most recent call last):
...
TypeError: ...mutable matrices are unhashable...
```

```
>>> from sage.all import *
>>> M = Matrix([[Integer(0),Integer(0)], [Integer(0),Integer(0)]])            ␣
↪                                  # needs sage.modules
>>> G = Graph({ Integer(0) : { M : None } })                                  ␣
↪     # needs sage.modules
Traceback (most recent call last):
...
TypeError: ...mutable matrices are unhashable...
```

However, if one wants to define a dictionary, with the same keys and arbitrary objects for entries, one can make that association:

```
sage: d = {0 : graphs.DodecahedralGraph(), 1 : graphs.FlowerSnark(), \
....: 2 : graphs.MoebiusKantorGraph(), 3 : graphs.PetersenGraph() }
sage: d[2]
Moebius-Kantor Graph: Graph on 16 vertices
sage: T = graphs.TetrahedralGraph()
sage: T.vertices(sort=True)
[0, 1, 2, 3]
sage: T.set_vertices(d)
sage: T.get_vertex(1)
Flower Snark: Graph on 20 vertices
```

```
>>> from sage.all import *
>>> d = {Integer(0) : graphs.DodecahedralGraph(), Integer(1) : graphs.FlowerSnark(),␣
↪Integer(2) : graphs.MoebiusKantorGraph(), Integer(3) : graphs.PetersenGraph() }
>>> d[Integer(2)]
Moebius-Kantor Graph: Graph on 16 vertices
>>> T = graphs.TetrahedralGraph()
>>> T.vertices(sort=True)
[0, 1, 2, 3]
```

(continues on next page)

(continued from previous page)

```
>>> T.set_vertices(d)
>>> T.get_vertex(Integer(1))
Flower Snark: Graph on 20 vertices
```

1.2.4 Database

There is a database available for searching for graphs that satisfy a certain set of parameters, including number of vertices and edges, density, maximum and minimum degree, diameter, radius, and connectivity. To see a list of all search parameter keywords broken down by their designated table names, type

```
sage: graph_db_info()
{...}
```

```
>>> from sage.all import *
>>> graph_db_info()
{...}
```

For more details on data types or keyword input, enter

```
sage: GraphQuery?    # not tested
```

```
>>> from sage.all import *
>>> GraphQuery?    # not tested
```

The results of a query can be viewed with the show method, or can be viewed individually by iterating through the results

```
sage: Q = GraphQuery(display_cols=['graph6'],num_vertices=7, diameter=5)
sage: Q.show()
Graph6
----------------------
F?`po
F?gqg
F@?]O
F@OKg
F@R@o
FA_pW
FEOhW
FGC{o
FIAHo
```

```
>>> from sage.all import *
>>> Q = GraphQuery(display_cols=['graph6'],num_vertices=Integer(7),␣
↪diameter=Integer(5))
>>> Q.show()
Graph6
----------------------
F?`po
F?gqg
F@?]O
F@OKg
F@R@o
```

(continues on next page)

(continued from previous page)

```
FA_pW
FEOhW
FGC{o
FIAHo
```

Show each graph as you iterate through the results:

```
sage: for g in Q:                                                              #␣
↪needs sage.plot
....:     show(g)
```

```
>>> from sage.all import *
>>> for g in Q:                                                                #␣
↪needs sage.plot
...     show(g)
```

1.2.5 Visualization

To see a graph G you are working with, there are three main options. You can view the graph in two dimensions via matplotlib with `show()`.

```
sage: G = graphs.RandomGNP(15,.3)
sage: G.show()                                                                 #␣
↪needs sage.plot
```

```
>>> from sage.all import *
>>> G = graphs.RandomGNP(Integer(15),RealNumber('.3'))
>>> G.show()                                                                   #␣
↪needs sage.plot
```

And you can view it in three dimensions with `show3d()`.

```
sage: G.show3d()                                                               #␣
↪needs sage.plot
```

```
>>> from sage.all import *
>>> G.show3d()                                                                 #␣
↪needs sage.plot
```

Or it can be rendered with LaTeX. This requires the right additions to a standard TeX installation. Then standard Sage commands, such as `view(G)` will display the graph, or `latex(G)` will produce a string suitable for inclusion in a LaTeX document. More details on this are at the *sage.graphs.graph_latex* module.

```
sage: from sage.graphs.graph_latex import check_tkz_graph
sage: check_tkz_graph()  # random - depends on TeX installation
sage: latex(G)
\begin{tikzpicture}
...
\end{tikzpicture}
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_latex import check_tkz_graph
>>> check_tkz_graph()  # random - depends on TeX installation
>>> latex(G)
\begin{tikzpicture}
...
\end{tikzpicture}
```

1.2.6 Mutability

Graphs are mutable, and thus unusable as dictionary keys, unless `data_structure="static_sparse"` is used:

```
sage: G = graphs.PetersenGraph()
sage: {G:1}[G]
Traceback (most recent call last):
...
TypeError: ...This graph is mutable, and thus not hashable...
sage: G_immutable = Graph(G, immutable=True)
sage: G_immutable == G
True
sage: {G_immutable:1}[G_immutable]
1
```

```
>>> from sage.all import *
>>> G = graphs.PetersenGraph()
>>> {G:Integer(1)}[G]
Traceback (most recent call last):
...
TypeError: ...This graph is mutable, and thus not hashable...
>>> G_immutable = Graph(G, immutable=True)
>>> G_immutable == G
True
>>> {G_immutable:Integer(1)}[G_immutable]
1
```

1.2.7 Methods

class sage.graphs.graph.**Graph**(*data=None*, *pos=None*, *loops=None*, *format=None*, *weighted=None*, *data_structure='sparse'*, *vertex_labels=True*, *name=None*, *multiedges=None*, *convert_empty_dict_labels_to_None=None*, *sparse=True*, *immutable=False*, *hash_labels=None*)

Bases: *GenericGraph*

Undirected graph.

A graph is a set of vertices connected by edges. See the Wikipedia article Graph_(mathematics) for more information. For a collection of pre-defined graphs, see the *graph_generators* module.

A *Graph* object has many methods whose list can be obtained by typing `g.<tab>` (i.e. hit the `Tab` key) or by reading the documentation of *graph*, *generic_graph*, and *digraph*.

INPUT:

By default, a *Graph* object is simple (i.e. no *loops* nor *multiple edges*) and unweighted. This can be easily tuned with the appropriate flags (see below).

- `data` – can be any of the following (see the `format` argument):
  1. `Graph()` – build a graph on 0 vertices.
  2. `Graph(5)` – return an edgeless graph on the 5 vertices 0,…,4.
  3. `Graph([list_of_vertices, list_of_edges])` – returns a graph with given vertices/edges.

     To bypass auto-detection, prefer the more explicit `Graph([V, E], format='vertices_and_edges')`.
  4. `Graph(list_of_edges)` – return a graph with a given list of edges (see documentation of *`add_edges()`*).

     To bypass auto-detection, prefer the more explicit `Graph(L, format='list_of_edges')`.
  5. `Graph({1: [2, 3, 4], 3: [4]})` – return a graph by associating to each vertex the list of its neighbors.

     To bypass auto-detection, prefer the more explicit `Graph(D, format='dict_of_lists')`.
  6. `Graph({1: {2: 'a', 3:'b'} ,3:{2:'c'}})` – return a graph by associating a list of neighbors to each vertex and providing its edge label.

     To bypass auto-detection, prefer the more explicit `Graph(D, format='dict_of_dicts')`.

     For graphs with multiple edges, you can provide a list of labels instead, e.g.: `Graph({1: {2: ['a1', 'a2'], 3:['b']} ,3:{2:['c']}})`.
  7. `Graph(a_symmetric_matrix)` – return a graph with given (weighted) adjacency matrix (see documentation of *`adjacency_matrix()`*).

     To bypass auto-detection, prefer the more explicit `Graph(M, format='adjacency_matrix')`. To take weights into account, use `format='weighted_adjacency_matrix'` instead.
  8. `Graph(a_nonsymmetric_matrix)` – return a graph with given incidence matrix (see documentation of *`incidence_matrix()`*).

     To bypass auto-detection, prefer the more explicit `Graph(M, format='incidence_matrix')`.
  9. `Graph([V, f])` – return a graph from a vertex set `V` and a *symmetric* function `f`. The graph contains an edge u, v whenever `f(u,v)` is `True`.. Example: `Graph([ [1..10], lambda x,y: abs(x-y).is_square()])`
  10. `Graph(':I`ES@obGkqegW~')` – return a graph from a graph6 or sparse6 string (see documentation of *`graph6_string()`* or *`sparse6_string()`*).
  11. `Graph(a_seidel_matrix, format='seidel_adjacency_matrix')` – return a graph with a given Seidel adjacency matrix (see documentation of *`seidel_adjacency_matrix()`*).
  12. `Graph(another_graph)` – return a graph from a Sage (di)graph, pygraphviz graph, NetworkX graph, or igraph graph.
- `pos` – a positioning dictionary (cf. documentation of *`layout()`*). For example, to draw 4 vertices on a square:

```
{0: [-1,-1],
 1: [ 1,-1],
 2: [ 1, 1],
 3: [-1, 1]}
```

- **`name` – (must be an explicitly named parameter, i.e.,** `name='complete'`) gives the graph a name
- `loops` – boolean (default: `None`); whether to allow loops (ignored if data is an instance of the `Graph` class)

- `multiedges` – boolean (default: `None`); whether to allow multiple edges (ignored if data is an instance of the `Graph` class)
- `weighted` – boolean (default: `None`); whether graph thinks of itself as weighted or not. See *`weighted()`*.
- `format` – if set to `None` (default), *`Graph`* tries to guess input's format. To avoid this possibly time-consuming step, one of the following values can be specified (see description above): `'int'`, `'graph6'`, `'sparse6'`, `'rule'`, `'list_of_edges'`, `'dict_of_lists'`, `'dict_of_dicts'`, `'adjacency_matrix'`, `'weighted_adjacency_matrix'`, `'seidel_adjacency_matrix'`, `'incidence_matrix'`, `"NX"`, `'igraph'`.
- `sparse` – boolean (default: `True`); `sparse=True` is an alias for `data_structure="sparse"`, and `sparse=False` is an alias for `data_structure="dense"`.
- `data_structure` – one of the following (for more information, see *`overview`*)
  - `'dense'` – selects the *`dense_graph`* backend.
  - `'sparse'` – selects the *`sparse_graph`* backend.
  - `'static_sparse'` – selects the *`static_sparse_backend`* (this backend is faster than the sparse backend and smaller in memory, and it is immutable, so that the resulting graphs can be used as dictionary keys).
- `immutable` – boolean (default: `False`); whether to create a immutable graph. Note that `immutable=True` is actually a shortcut for `data_structure='static_sparse'`. Set to `False` by default.
- `hash_labels` – boolean (default: `None`); whether to include edge labels during hashing. This parameter defaults to `True` if the graph is weighted. This parameter is ignored if the graph is mutable. Beware that trying to hash unhashable labels will raise an error.
- `vertex_labels` – boolean (default: `True`); whether to allow any object as a vertex (slower), or only the integers $0, ..., n-1$, where n is the number of vertices.
- `convert_empty_dict_labels_to_None` – this arguments sets the default edge labels used by NetworkX (empty dictionaries) to be replaced by `None`, the default Sage edge label. It is set to `True` iff a NetworkX graph is on the input.

EXAMPLES:

We illustrate the first seven input formats (the other two involve packages that are currently not standard in Sage):

1. An integer giving the number of vertices:

```
sage: g = Graph(5); g
Graph on 5 vertices
sage: g.vertices(sort=True)
[0, 1, 2, 3, 4]
sage: g.edges(sort=False)
[]
```

```
>>> from sage.all import *
>>> g = Graph(Integer(5)); g
Graph on 5 vertices
>>> g.vertices(sort=True)
[0, 1, 2, 3, 4]
>>> g.edges(sort=False)
[]
```

2. A dictionary of dictionaries:

```
sage: g = Graph({0:{1:'x',2:'z',3:'a'}, 2:{5:'out'}}); g
Graph on 5 vertices
```

```
>>> from sage.all import *
>>> g = Graph({Integer(0):{Integer(1):'x',Integer(2):'z',Integer(3):'a'},␣
↪Integer(2):{Integer(5):'out'}}); g
Graph on 5 vertices
```

The labels ('x', 'z', 'a', 'out') are labels for edges. For example, 'out' is the label for the edge on 2 and 5. Labels can be used as weights, if all the labels share some common parent.:

```
sage: a, b, c, d, e, f = sorted(SymmetricGroup(3))                    ␣
↪  # needs sage.groups
sage: Graph({b: {d: 'c', e: 'p'}, c: {d: 'p', e: 'c'}})               ␣
↪  # needs sage.groups
Graph on 4 vertices
```

```
>>> from sage.all import *
>>> a, b, c, d, e, f = sorted(SymmetricGroup(Integer(3)))               ␣
↪          # needs sage.groups
>>> Graph({b: {d: 'c', e: 'p'}, c: {d: 'p', e: 'c'}})
↪# needs sage.groups
Graph on 4 vertices
```

3. A dictionary of lists:

```
sage: g = Graph({0:[1,2,3], 2:[4]}); g
Graph on 5 vertices
```

```
>>> from sage.all import *
>>> g = Graph({Integer(0):[Integer(1),Integer(2),Integer(3)],␣
↪Integer(2):[Integer(4)]}); g
Graph on 5 vertices
```

4. A list of vertices and a function describing adjacencies. Note that the list of vertices and the function must be enclosed in a list (i.e., [list of vertices, function]).

   Construct the Paley graph over GF(13).:

```
sage: g = Graph([GF(13), lambda i,j: i!=j and (i-j).is_square()])
↪# needs sage.rings.finite_rings
sage: g.vertices(sort=True)
↪# needs sage.rings.finite_rings
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]
sage: g.adjacency_matrix()
↪# needs sage.modules sage.rings.finite_rings
[0 1 0 1 1 0 0 0 0 1 1 0 1]
[1 0 1 0 1 1 0 0 0 0 1 1 0]
[0 1 0 1 0 1 1 0 0 0 0 1 1]
[1 0 1 0 1 0 1 1 0 0 0 0 1]
[1 1 0 1 0 1 0 1 1 0 0 0 0]
[0 1 1 0 1 0 1 0 1 1 0 0 0]
[0 0 1 1 0 1 0 1 0 1 1 0 0]
```

(continues on next page)

(continued from previous page)

```
[0 0 0 1 1 0 1 0 1 0 1 1 0]
[0 0 0 0 1 1 0 1 0 1 0 1 1]
[1 0 0 0 0 1 1 0 1 0 1 0 1]
[1 1 0 0 0 0 1 1 0 1 0 1 0]
[0 1 1 0 0 0 0 1 1 0 1 0 1]
[1 0 1 1 0 0 0 0 1 1 0 1 0]
```

```
>>> from sage.all import *
>>> g = Graph([GF(Integer(13)), lambda i,j: i!=j and (i-j).is_square()])      ␣
↪       # needs sage.rings.finite_rings
>>> g.vertices(sort=True)                                                     #␣
↪needs sage.rings.finite_rings
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]
>>> g.adjacency_matrix()                                                      #␣
↪needs sage.modules sage.rings.finite_rings
[0 1 0 1 1 0 0 0 0 1 1 0 1]
[1 0 1 0 1 1 0 0 0 0 1 1 0]
[0 1 0 1 0 1 1 0 0 0 0 1 1]
[1 0 1 0 1 0 1 1 0 0 0 0 1]
[1 1 0 1 0 1 0 1 1 0 0 0 0]
[0 1 1 0 1 0 1 0 1 1 0 0 0]
[0 0 1 1 0 1 0 1 0 1 1 0 0]
[0 0 0 1 1 0 1 0 1 0 1 1 0]
[0 0 0 0 1 1 0 1 0 1 0 1 1]
[1 0 0 0 0 1 1 0 1 0 1 0 1]
[1 1 0 0 0 0 1 1 0 1 0 1 0]
[0 1 1 0 0 0 0 1 1 0 1 0 1]
[1 0 1 1 0 0 0 0 1 1 0 1 0]
```

Construct the line graph of a complete graph.:

```
sage: g = graphs.CompleteGraph(4)
sage: line_graph = Graph([g.edges(sort=True, labels=false),
....:                     lambda i,j: len(set(i).intersection(set(j)))>0],
....:                    loops=False)
sage: line_graph.vertices(sort=True)
[(0, 1), (0, 2), (0, 3), (1, 2), (1, 3), (2, 3)]
sage: line_graph.adjacency_matrix()
↪# needs sage.modules
[0 1 1 1 1 0]
[1 0 1 1 0 1]
[1 1 0 0 1 1]
[1 1 0 0 1 1]
[1 0 1 1 0 1]
[0 1 1 1 1 0]
```

```
>>> from sage.all import *
>>> g = graphs.CompleteGraph(Integer(4))
>>> line_graph = Graph([g.edges(sort=True, labels=false),
...                     lambda i,j: len(set(i).intersection(set(j)))>
↪Integer(0)],
...                    loops=False)
```

(continues on next page)

(continued from previous page)

```
>>> line_graph.vertices(sort=True)
[(0, 1), (0, 2), (0, 3), (1, 2), (1, 3), (2, 3)]
>>> line_graph.adjacency_matrix()                                    #␣
↪needs sage.modules
[0 1 1 1 1 0]
[1 0 1 1 0 1]
[1 1 0 0 1 1]
[1 1 0 0 1 1]
[1 0 1 1 0 1]
[0 1 1 1 1 0]
```

5. A graph6 or sparse6 string: Sage automatically recognizes whether a string is in graph6 or sparse6 format:

```
sage: s = ':I`AKGsaOs`cI]Gb~'
sage: Graph(s, sparse=True)
Looped multi-graph on 10 vertices
```

```
>>> from sage.all import *
>>> s = ':I`AKGsaOs`cI]Gb~'
>>> Graph(s, sparse=True)
Looped multi-graph on 10 vertices
```

```
sage: G = Graph('G?????')
sage: G = Graph("G'?G?C")
Traceback (most recent call last):
...
RuntimeError: the string seems corrupt: valid characters are
?@ABCDEFGHIJKLMNOPQRSTUVWXYZ[\]^_`abcdefghijklmnopqrstuvwxyz{|}~
sage: G = Graph('G??????')
Traceback (most recent call last):
...
RuntimeError: the string (G??????) seems corrupt: for n = 8, the string is␣
↪too long
```

```
>>> from sage.all import *
>>> G = Graph('G?????')
>>> G = Graph("G'?G?C")
Traceback (most recent call last):
...
RuntimeError: the string seems corrupt: valid characters are
?@ABCDEFGHIJKLMNOPQRSTUVWXYZ[\]^_`abcdefghijklmnopqrstuvwxyz{|}~
>>> G = Graph('G??????')
Traceback (most recent call last):
...
RuntimeError: the string (G??????) seems corrupt: for n = 8, the string is␣
↪too long
```

```
sage: G = Graph(":I'AKGsaOs`cI]Gb~")
Traceback (most recent call last):
...
RuntimeError: the string seems corrupt: valid characters are
```

(continues on next page)

(continued from previous page)

```
?@ABCDEFGHIJKLMNOPQRSTUVWXYZ[\]^_`abcdefghijklmnopqrstuvwxyz{|}~
```

```
>>> from sage.all import *
>>> G = Graph(":I'AKGsaOs`cI]Gb~")
Traceback (most recent call last):
...
RuntimeError: the string seems corrupt: valid characters are
?@ABCDEFGHIJKLMNOPQRSTUVWXYZ[\]^_`abcdefghijklmnopqrstuvwxyz{|}~
```

There are also list functions to take care of lists of graphs:

```
sage: s = ':IgMoqoCUOqeb\n:I`AKGsaOs`cI]Gb~\n:I`EDOAEQ?PccSsge\\N\n'
sage: graphs_list.from_sparse6(s)
[Looped multi-graph on 10 vertices,
 Looped multi-graph on 10 vertices,
 Looped multi-graph on 10 vertices]
```

```
>>> from sage.all import *
>>> s = ':IgMoqoCUOqeb\n:I`AKGsaOs`cI]Gb~\n:I`EDOAEQ?PccSsge\\N\n'
>>> graphs_list.from_sparse6(s)
[Looped multi-graph on 10 vertices,
 Looped multi-graph on 10 vertices,
 Looped multi-graph on 10 vertices]
```

6. A Sage matrix: Note: If format is not specified, then Sage assumes a symmetric square matrix is an adjacency matrix, otherwise an incidence matrix.

   - an adjacency matrix:

```
sage: M = graphs.PetersenGraph().am(); M
↪  # needs sage.modules
[0 1 0 0 1 1 0 0 0 0]
[1 0 1 0 0 0 1 0 0 0]
[0 1 0 1 0 0 0 1 0 0]
[0 0 1 0 1 0 0 0 1 0]
[1 0 0 1 0 0 0 0 0 1]
[1 0 0 0 0 0 0 1 1 0]
[0 1 0 0 0 0 0 0 1 1]
[0 0 1 0 0 1 0 0 0 1]
[0 0 0 1 0 1 1 0 0 0]
[0 0 0 0 1 0 1 1 0 0]
sage: Graph(M)
↪  # needs sage.modules
Graph on 10 vertices
```

```
>>> from sage.all import *
>>> M = graphs.PetersenGraph().am(); M
↪# needs sage.modules
[0 1 0 0 1 1 0 0 0 0]
[1 0 1 0 0 0 1 0 0 0]
[0 1 0 1 0 0 0 1 0 0]
[0 0 1 0 1 0 0 0 1 0]
```

(continues on next page)

(continued from previous page)

```
[1 0 0 1 0 0 0 0 0 1]
[1 0 0 0 0 0 0 1 1 0]
[0 1 0 0 0 0 0 0 1 1]
[0 0 1 0 0 1 0 0 0 1]
[0 0 0 1 0 1 1 0 0 0]
[0 0 0 0 1 0 1 1 0 0]
>>> Graph(M)
↪# needs sage.modules
Graph on 10 vertices
```

```
sage: Graph(matrix([[1,2], [2,4]]), loops=True, sparse=True)
↪  # needs sage.modules
Looped multi-graph on 2 vertices

sage: M = Matrix([[0,1,-1], [1,0,-1/2], [-1,-1/2,0]]); M
↪  # needs sage.modules
[   0    1   -1]
[   1    0 -1/2]
[  -1 -1/2    0]
sage: G = Graph(M, sparse=True); G
↪  # needs sage.modules
Graph on 3 vertices
sage: G.weighted()
↪  # needs sage.modules
True
```

```
>>> from sage.all import *
>>> Graph(matrix([[Integer(1),Integer(2)], [Integer(2),Integer(4)]]),
↪loops=True, sparse=True)                # needs sage.modules
Looped multi-graph on 2 vertices

>>> M = Matrix([[Integer(0),Integer(1),-Integer(1)], [Integer(1),
↪Integer(0),-Integer(1)/Integer(2)], [-Integer(1),-Integer(1)/Integer(2),
↪Integer(0)]]); M                    # needs sage.modules
[   0    1   -1]
[   1    0 -1/2]
[  -1 -1/2    0]
>>> G = Graph(M, sparse=True); G
↪# needs sage.modules
Graph on 3 vertices
>>> G.weighted()
↪# needs sage.modules
True
```

- an incidence matrix:

```
sage: M = Matrix(6, [-1,0,0,0,1, 1,-1,0,0,0, 0,1,-1,0,0,
↪  # needs sage.modules
....:                0,0,1,-1,0, 0,0,0,1,-1, 0,0,0,0,0]); M
[-1  0  0  0  1]
[ 1 -1  0  0  0]
[ 0  1 -1  0  0]
```

(continues on next page)

(continued from previous page)

```
[ 0  0  1 -1  0]
[ 0  0  0  1 -1]
[ 0  0  0  0  0]
sage: Graph(M)                                                              ␣
↪  # needs sage.modules
Graph on 6 vertices

sage: Graph(Matrix([[1],[1],[1]]))                                          ␣
↪  # needs sage.modules
Traceback (most recent call last):
...
ValueError: there must be one or two nonzero entries per column
in an incidence matrix, got entries [1, 1, 1] in column 0
sage: Graph(Matrix([[1],[1],[0]]))                                          ␣
↪  # needs sage.modules
Graph on 3 vertices

sage: M = Matrix([[0,1,-1], [1,0,-1], [-1,-1,0]]); M                        ␣
↪  # needs sage.modules
[ 0  1 -1]
[ 1  0 -1]
[-1 -1  0]
sage: Graph(M, sparse=True)                                                 ␣
↪  # needs sage.modules
Graph on 3 vertices

sage: M = Matrix([[0,1,1], [1,0,1], [-1,-1,0]]); M                          ␣
↪  # needs sage.modules
[ 0  1  1]
[ 1  0  1]
[-1 -1  0]
sage: Graph(M)                                                              ␣
↪  # needs sage.modules
Traceback (most recent call last):
...
ValueError: there must be one or two nonzero entries per column
in an incidence matrix, got entries [1, 1] in column 2
```

```
>>> from sage.all import *
>>> M = Matrix(Integer(6), [-Integer(1),Integer(0),Integer(0),Integer(0),
↪Integer(1), Integer(1),-Integer(1),Integer(0),Integer(0),Integer(0),␣
↪Integer(0),Integer(1),-Integer(1),Integer(0),Integer(0),                   ␣
↪    # needs sage.modules
...                Integer(0),Integer(0),Integer(1),-Integer(1),
↪Integer(0), Integer(0),Integer(0),Integer(0),Integer(1),-Integer(1),␣
↪Integer(0),Integer(0),Integer(0),Integer(0),Integer(0)]); M
[-1  0  0  0  1]
[ 1 -1  0  0  0]
[ 0  1 -1  0  0]
[ 0  0  1 -1  0]
[ 0  0  0  1 -1]
[ 0  0  0  0  0]
```

(continues on next page)

(continued from previous page)

```
>>> Graph(M)
↪# needs sage.modules
Graph on 6 vertices

>>> Graph(Matrix([[Integer(1)],[Integer(1)],[Integer(1)]]))              ␣
↪                            # needs sage.modules
Traceback (most recent call last):
...
ValueError: there must be one or two nonzero entries per column
in an incidence matrix, got entries [1, 1, 1] in column 0
>>> Graph(Matrix([[Integer(1)],[Integer(1)],[Integer(0)]]))              ␣
↪                            # needs sage.modules
Graph on 3 vertices

>>> M = Matrix([[Integer(0),Integer(1),-Integer(1)], [Integer(1),
↪Integer(0),-Integer(1)], [-Integer(1),-Integer(1),Integer(0)]]); M     ␣
↪                   # needs sage.modules
[ 0  1 -1]
[ 1  0 -1]
[-1 -1  0]
>>> Graph(M, sparse=True)
↪# needs sage.modules
Graph on 3 vertices

>>> M = Matrix([[Integer(0),Integer(1),Integer(1)], [Integer(1),
↪Integer(0),Integer(1)], [-Integer(1),-Integer(1),Integer(0)]]); M      ␣
↪                    # needs sage.modules
[ 0  1  1]
[ 1  0  1]
[-1 -1  0]
>>> Graph(M)
↪# needs sage.modules
Traceback (most recent call last):
...
ValueError: there must be one or two nonzero entries per column
in an incidence matrix, got entries [1, 1] in column 2
```

Check that Issue #9714 is fixed:

```
sage: MA = Matrix([[1,2,0], [0,2,0], [0,0,1]])
sage: GA = Graph(MA, format='adjacency_matrix')
sage: MI = GA.incidence_matrix(oriented=False); MI
[2 1 1 0 0 0]
[0 1 1 2 2 0]
[0 0 0 0 0 2]
sage: Graph(MI).edges(sort=True, labels=None)
[(0, 0), (0, 1), (0, 1), (1, 1), (1, 1), (2, 2)]

sage: M = Matrix([[1], [-1]]); M                                        ␣
↪     # needs sage.modules
[ 1]
[-1]
```

(continues on next page)

(continued from previous page)

```
sage: Graph(M).edges(sort=True)
↪      # needs sage.modules
[(0, 1, None)]
```

```
>>> from sage.all import *
>>> MA = Matrix([[Integer(1),Integer(2),Integer(0)], [Integer(0),
↪Integer(2),Integer(0)], [Integer(0),Integer(0),Integer(1)]])
>>> GA = Graph(MA, format='adjacency_matrix')
>>> MI = GA.incidence_matrix(oriented=False); MI
[2 1 1 0 0 0]
[0 1 1 2 2 0]
[0 0 0 0 0 2]
>>> Graph(MI).edges(sort=True, labels=None)
[(0, 0), (0, 1), (0, 1), (1, 1), (1, 1), (2, 2)]

>>> M = Matrix([[Integer(1)], [-Integer(1)]]); M
↪                      # needs sage.modules
[ 1]
[-1]
>>> Graph(M).edges(sort=True)
↪    # needs sage.modules
[(0, 1, None)]
```

7. A Seidel adjacency matrix:

```
sage: from sage.combinat.matrices.hadamard_matrix import (
↪# needs sage.combinat sage.modules
....:  regular_symmetric_hadamard_matrix_with_constant_diagonal as rshcd)
sage: m = rshcd(16,1) - matrix.identity(16)
↪# needs sage.combinat sage.modules
sage: Graph(m,
↪# needs sage.combinat sage.modules
....:       format='seidel_adjacency_matrix').is_strongly_
↪regular(parameters=True)
(16, 6, 2, 2)
```

```
>>> from sage.all import *
>>> from sage.combinat.matrices.hadamard_matrix import (                    #␣
↪needs sage.combinat sage.modules
...  regular_symmetric_hadamard_matrix_with_constant_diagonal as rshcd)
>>> m = rshcd(Integer(16),Integer(1)) - matrix.identity(Integer(16))        ␣
↪                          # needs sage.combinat sage.modules
>>> Graph(m,                                                               #␣
↪needs sage.combinat sage.modules
...       format='seidel_adjacency_matrix').is_strongly_
↪regular(parameters=True)
(16, 6, 2, 2)
```

8. List of edges, or labelled edges:

```
sage: g = Graph([(1, 3), (3, 8), (5, 2)]); g
```

(continues on next page)

(continued from previous page)

```
Graph on 5 vertices

sage: g = Graph([(1, 2, "Peace"), (7, -9, "and"), (77, 2, "Love")]); g
Graph on 5 vertices
sage: g = Graph([(0, 2, '0'), (0, 2, '1'), (3, 3, '2')],
....:           loops=True, multiedges=True)
sage: g.loops()
[(3, 3, '2')]
```

```
>>> from sage.all import *
>>> g = Graph([(Integer(1), Integer(3)), (Integer(3), Integer(8)),␣
↪(Integer(5), Integer(2))]); g
Graph on 5 vertices

>>> g = Graph([(Integer(1), Integer(2), "Peace"), (Integer(7), -Integer(9),
↪"and"), (Integer(77), Integer(2), "Love")]); g
Graph on 5 vertices
>>> g = Graph([(Integer(0), Integer(2), '0'), (Integer(0), Integer(2), '1'),␣
↪(Integer(3), Integer(3), '2')],
...           loops=True, multiedges=True)
>>> g.loops()
[(3, 3, '2')]
```

9. A NetworkX MultiGraph:

```
sage: import networkx
↪# needs networkx
sage: g = networkx.MultiGraph({0:[1,2,3], 2:[4]})
↪# needs networkx
sage: Graph(g)
↪# needs networkx
Multi-graph on 5 vertices
```

```
>>> from sage.all import *
>>> import networkx                                                    #␣
↪needs networkx
>>> g = networkx.MultiGraph({Integer(0):[Integer(1),Integer(2),Integer(3)],␣
↪Integer(2):[Integer(4)]})                                  # needs networkx
>>> Graph(g)                                                           #␣
↪needs networkx
Multi-graph on 5 vertices
```

10. A NetworkX graph:

```
sage: import networkx
↪# needs networkx
sage: g = networkx.Graph({0:[1,2,3], 2:[4]})
↪# needs networkx
sage: DiGraph(g)
↪# needs networkx
Digraph on 5 vertices
```

```
>>> from sage.all import *
>>> import networkx                                                    #␣
↪needs networkx
>>> g = networkx.Graph({Integer(0):[Integer(1),Integer(2),Integer(3)],␣
↪Integer(2):[Integer(4)]})                                  # needs networkx
>>> DiGraph(g)                                                        #␣
↪needs networkx
Digraph on 5 vertices
```

11. An igraph Graph (see also *igraph_graph()*):

```
sage: import igraph                          # optional - python_igraph
sage: g = igraph.Graph([(0, 1), (0, 2)])  # optional - python_igraph
sage: Graph(g)                               # optional - python_igraph
Graph on 3 vertices
```

```
>>> from sage.all import *
>>> import igraph                           # optional - python_igraph
>>> g = igraph.Graph([(Integer(0), Integer(1)), (Integer(0), Integer(2))])  #␣
↪optional - python_igraph
>>> Graph(g)                                # optional - python_igraph
Graph on 3 vertices
```

If `vertex_labels` is `True`, the names of the vertices are given by the vertex attribute `'name'`, if available:

```
sage: # optional - python_igraph
sage: g = igraph.Graph([(0,1),(0,2)], vertex_attrs={'name':['a','b','c']})
sage: Graph(g).vertices(sort=True)
['a', 'b', 'c']
sage: g = igraph.Graph([(0,1),(0,2)], vertex_attrs={'label':['a','b','c']})
sage: Graph(g).vertices(sort=True)
[0, 1, 2]
```

```
>>> from sage.all import *
>>> # optional - python_igraph
>>> g = igraph.Graph([(Integer(0),Integer(1)),(Integer(0),Integer(2))],␣
↪vertex_attrs={'name':['a','b','c']})
>>> Graph(g).vertices(sort=True)
['a', 'b', 'c']
>>> g = igraph.Graph([(Integer(0),Integer(1)),(Integer(0),Integer(2))],␣
↪vertex_attrs={'label':['a','b','c']})
>>> Graph(g).vertices(sort=True)
[0, 1, 2]
```

If the igraph Graph has edge attributes, they are used as edge labels:

```
sage: g = igraph.Graph([(0, 1), (0, 2)],                                #␣
↪optional - python_igraph
....:                  edge_attrs={'name': ['a', 'b'], 'weight': [1, 3]})
sage: Graph(g).edges(sort=True)                                         #␣
↪optional - python_igraph
[(0, 1, {'name': 'a', 'weight': 1}), (0, 2, {'name': 'b', 'weight': 3})]
```

```
>>> from sage.all import *
>>> g = igraph.Graph([(Integer(0), Integer(1)), (Integer(0), Integer(2))],␣
↪                          # optional - python_igraph
...                   edge_attrs={'name': ['a', 'b'], 'weight': [Integer(1),␣
↪Integer(3)]})
>>> Graph(g).edges(sort=True)                                          # optional␣
↪- python_igraph
[(0, 1, {'name': 'a', 'weight': 1}), (0, 2, {'name': 'b', 'weight': 3})]
```

When defining an undirected graph from a function `f`, it is *very* important that `f` be symmetric. If it is not, anything can happen:

```
sage: f_sym = lambda x,y: abs(x-y) == 1
sage: f_nonsym = lambda x,y: (x-y) == 1
sage: G_sym = Graph([[4,6,1,5,3,7,2,0], f_sym])
sage: G_sym.is_isomorphic(graphs.PathGraph(8))
True
sage: G_nonsym = Graph([[4,6,1,5,3,7,2,0], f_nonsym])
sage: G_nonsym.size()
4
sage: G_nonsym.is_isomorphic(G_sym)
False
```

```
>>> from sage.all import *
>>> f_sym = lambda x,y: abs(x-y) == Integer(1)
>>> f_nonsym = lambda x,y: (x-y) == Integer(1)
>>> G_sym = Graph([[Integer(4),Integer(6),Integer(1),Integer(5),Integer(3),
↪Integer(7),Integer(2),Integer(0)], f_sym])
>>> G_sym.is_isomorphic(graphs.PathGraph(Integer(8)))
True
>>> G_nonsym = Graph([[Integer(4),Integer(6),Integer(1),Integer(5),Integer(3),
↪Integer(7),Integer(2),Integer(0)], f_nonsym])
>>> G_nonsym.size()
4
>>> G_nonsym.is_isomorphic(G_sym)
False
```

By default, graphs are mutable and can thus not be used as a dictionary key:

```
sage: G = graphs.PetersenGraph()
sage: {G:1}[G]
Traceback (most recent call last):
...
TypeError: ...This graph is mutable, and thus not hashable...
```

```
>>> from sage.all import *
>>> G = graphs.PetersenGraph()
>>> {G:Integer(1)}[G]
Traceback (most recent call last):
...
TypeError: ...This graph is mutable, and thus not hashable...
```

When providing the optional arguments `data_structure="static_sparse"` or `immutable=True` (both

mean the same), then an immutable graph results:

```
sage: G_imm = Graph(G, immutable=True)
sage: H_imm = Graph(G, data_structure='static_sparse')
sage: G_imm == H_imm == G
True
sage: {G_imm:1}[H_imm]
1
```

```
>>> from sage.all import *
>>> G_imm = Graph(G, immutable=True)
>>> H_imm = Graph(G, data_structure='static_sparse')
>>> G_imm == H_imm == G
True
>>> {G_imm:Integer(1)}[H_imm]
1
```

RankWidth

alias of `RankWidth`

acyclic_orientations(G)

Return an iterator over all acyclic orientations of an undirected graph G.

ALGORITHM:

The algorithm is based on [Sq1998]. It presents an efficient algorithm for listing the acyclic orientations of a graph. The algorithm is shown to require O(n) time per acyclic orientation generated, making it the most efficient known algorithm for generating acyclic orientations.

The function uses a recursive approach to generate acyclic orientations of the graph. It reorders the vertices and edges of the graph, creating a new graph with updated labels. Then, it iteratively generates acyclic orientations by considering subsets of edges and checking whether they form upsets in a corresponding poset.

INPUT:

- `G` – an undirected graph

OUTPUT: an iterator over all acyclic orientations of the input graph

Note

The function assumes that the input graph is undirected and the edges are unlabelled.

EXAMPLES:

To count the number of acyclic orientations for a graph:

```
sage: g = Graph([(0, 3), (0, 4), (3, 4), (1, 3), (1, 2), (2, 3), (2, 4)])
sage: it = g.acyclic_orientations()
sage: len(list(it))
54
```

```
>>> from sage.all import *
>>> g = Graph([(Integer(0), Integer(3)), (Integer(0), Integer(4)),␣
↪(Integer(3), Integer(4)), (Integer(1), Integer(3)), (Integer(1),␣
```

(continues on next page)

(continued from previous page)

```
↪Integer(2)), (Integer(2), Integer(3)), (Integer(2), Integer(4))])
>>> it = g.acyclic_orientations()
>>> len(list(it))
54
```

Test for arbitrary vertex labels:

```
sage: g_str = Graph([('abc', 'def'), ('ghi', 'def'), ('xyz', 'abc'),
....:                ('xyz', 'uvw'), ('uvw', 'abc'), ('uvw', 'ghi')])
sage: it = g_str.acyclic_orientations()
sage: len(list(it))
42
```

```
>>> from sage.all import *
>>> g_str = Graph([('abc', 'def'), ('ghi', 'def'), ('xyz', 'abc'),
...                ('xyz', 'uvw'), ('uvw', 'abc'), ('uvw', 'ghi')])
>>> it = g_str.acyclic_orientations()
>>> len(list(it))
42
```

Check that the method returns properly relabeled acyclic digraphs:

```
sage: g = Graph([(0, 1), (1, 2), (2, 3), (3, 0), (0, 2)])
sage: orientations = set([frozenset(d.edges(labels=false)) for d in g.acyclic_
↪orientations()])
sage: len(orientations)
18
sage: all(d.is_directed_acyclic() for d in g.acyclic_orientations())
True
```

```
>>> from sage.all import *
>>> g = Graph([(Integer(0), Integer(1)), (Integer(1), Integer(2)),␣
↪(Integer(2), Integer(3)), (Integer(3), Integer(0)), (Integer(0),␣
↪Integer(2))])
>>> orientations = set([frozenset(d.edges(labels=false)) for d in g.acyclic_
↪orientations()])
>>> len(orientations)
18
>>> all(d.is_directed_acyclic() for d in g.acyclic_orientations())
True
```

all_cliques (*graph*, *min_size=0*, *max_size=0*)

Iterator over the cliques in `graph`.

A clique is an induced complete subgraph. This method is an iterator over all the cliques with size in between `min_size` and `max_size`. By default, this method returns only maximum cliques. Each yielded clique is represented by a list of vertices.

Note

Currently only implemented for undirected graphs. Use *to_undirected()* to convert a digraph to an undirected graph.

INPUT:

- `min_size` – integer (default: 0); minimum size of reported cliques. When set to 0 (default), this method searches for maximum cliques. In such case, parameter `max_size` must also be set to 0.
- `max_size` – integer (default: 0); maximum size of reported cliques. When set to 0 (default), the maximum size of the cliques is unbounded. When `min_size` is set to 0, this parameter must be set to 0.

ALGORITHM:

This function is based on Cliquer [NO2003].

EXAMPLES:

```
sage: G = graphs.CompleteGraph(5)
sage: list(sage.graphs.cliquer.all_cliques(G))
[[0, 1, 2, 3, 4]]
sage: list(sage.graphs.cliquer.all_cliques(G, 2, 3))
[[3, 4],
 [2, 3],
 [2, 3, 4],
 [2, 4],
 [1, 2],
 [1, 2, 3],
 [1, 2, 4],
 [1, 3],
 [1, 3, 4],
 [1, 4],
 [0, 1],
 [0, 1, 2],
 [0, 1, 3],
 [0, 1, 4],
 [0, 2],
 [0, 2, 3],
 [0, 2, 4],
 [0, 3],
 [0, 3, 4],
 [0, 4]]
sage: G.delete_edge([1,3])
sage: list(sage.graphs.cliquer.all_cliques(G))
[[0, 2, 3, 4], [0, 1, 2, 4]]
```

```
>>> from sage.all import *
>>> G = graphs.CompleteGraph(Integer(5))
>>> list(sage.graphs.cliquer.all_cliques(G))
[[0, 1, 2, 3, 4]]
>>> list(sage.graphs.cliquer.all_cliques(G, Integer(2), Integer(3)))
[[3, 4],
 [2, 3],
 [2, 3, 4],
 [2, 4],
 [1, 2],
 [1, 2, 3],
 [1, 2, 4],
 [1, 3],
```

(continues on next page)

(continued from previous page)

```
 [1, 3, 4],
 [1, 4],
 [0, 1],
 [0, 1, 2],
 [0, 1, 3],
 [0, 1, 4],
 [0, 2],
 [0, 2, 3],
 [0, 2, 4],
 [0, 3],
 [0, 3, 4],
 [0, 4]]
>>> G.delete_edge([Integer(1),Integer(3)])
>>> list(sage.graphs.cliquer.all_cliques(G))
[[0, 2, 3, 4], [0, 1, 2, 4]]
```

Todo

Use the re-entrant functionality of Cliquer [NO2003] to avoid storing all cliques.

antipodal_graph(*G*, *immutable=None*)

Return the antipodal graph of G.

The antipodal graph of a graph G has the same vertex set of G and two vertices are adjacent if their distance in G is equal to the diameter of G.

This method first computes the eccentricity of all vertices and determines the diameter of the graph. Then, it for each vertex u with eccentricity the diameter, it computes BFS distances from u and add an edge in the antipodal graph for each vertex v at diameter distance from u (i.e., for each antipodal vertex).

The drawback of this method is that some BFS distances may be computed twice, one time to determine the eccentricities and another time is the vertex has eccentricity equal to the diameter. However, in practice, this is much more efficient. See *eccentricity()* with `algorithm='DHV'`.

INPUT:

- `G` – a *Graph*
- `immutable` – boolean (default: `None`); whether to create a mutable/immutable graph. `immutable=None` (default) means that the graph and its antipodal graph will behave the same way.

EXAMPLES:

The antipodal graph of a grid graph has only 2 edges:

```
sage: from sage.graphs.distances_all_pairs import antipodal_graph
sage: G = graphs.Grid2dGraph(5, 5)
sage: A = antipodal_graph(G)
sage: A
Antipodal graph of 2D Grid Graph for [5, 5]: Graph on 25 vertices
sage: A.order(), A.size()
(25, 2)
```

```
>>> from sage.all import *
>>> from sage.graphs.distances_all_pairs import antipodal_graph
>>> G = graphs.Grid2dGraph(Integer(5), Integer(5))
>>> A = antipodal_graph(G)
>>> A
Antipodal graph of 2D Grid Graph for [5, 5]: Graph on 25 vertices
>>> A.order(), A.size()
(25, 2)
```

The antipodal graph of a disconnected graph is its complement:

```
sage: G = Graph(5)
sage: H = G.antipodal_graph()
sage: H
Antipodal graph of Graph on 5 vertices: Graph on 5 vertices
sage: H.is_isomorphic(G.complement())
True
```

```
>>> from sage.all import *
>>> G = Graph(Integer(5))
>>> H = G.antipodal_graph()
>>> H
Antipodal graph of Graph on 5 vertices: Graph on 5 vertices
>>> H.is_isomorphic(G.complement())
True
```

The antipodal graph can also be constructed as the *distance_graph()* for diameter distance:

```
sage: G = graphs.RandomGNP(10, .2)
sage: A = G.antipodal_graph()
sage: B = G.distance_graph(G.diameter())
sage: A.is_isomorphic(B)
True
```

```
>>> from sage.all import *
>>> G = graphs.RandomGNP(Integer(10), RealNumber('.2'))
>>> A = G.antipodal_graph()
>>> B = G.distance_graph(G.diameter())
>>> A.is_isomorphic(B)
True
```

apex_vertices(*k=None*)

Return the list of apex vertices.

A graph is apex if it can be made planar by the removal of a single vertex. The deleted vertex is called an apex of the graph, and a graph may have more than one apex. For instance, in the minimal nonplanar graphs K_5 or $K_{3,3}$, every vertex is an apex. The apex graphs include graphs that are themselves planar, in which case again every vertex is an apex. The null graph is also counted as an apex graph even though it has no vertex to remove. If the graph is not connected, we say that it is apex if it has at most one non planar connected component and that this component is apex. See the Wikipedia article Apex_graph for more information.

See also

- *is_apex()*
- *is_planar()*

INPUT:

- `k` – integer (default: `None`); when set to `None`, the method returns the list of all apex of the graph, possibly empty if the graph is not apex. When set to a positive integer, the method ends as soon as k apex vertices are found.

OUTPUT:

By default, the method returns the list of all apex of the graph. When parameter `k` is set to a positive integer, the returned list is bounded to k apex vertices.

EXAMPLES:

K_5 and $K_{3,3}$ are apex graphs, and each of their vertices is an apex:

```
sage: G = graphs.CompleteGraph(5)
sage: G.apex_vertices()
[0, 1, 2, 3, 4]
sage: G = graphs.CompleteBipartiteGraph(3,3)
sage: G.is_apex()
True
sage: G.apex_vertices()
[0, 1, 2, 3, 4, 5]
sage: G.apex_vertices(k=3)
[0, 1, 2]
```

```
>>> from sage.all import *
>>> G = graphs.CompleteGraph(Integer(5))
>>> G.apex_vertices()
[0, 1, 2, 3, 4]
>>> G = graphs.CompleteBipartiteGraph(Integer(3),Integer(3))
>>> G.is_apex()
True
>>> G.apex_vertices()
[0, 1, 2, 3, 4, 5]
>>> G.apex_vertices(k=Integer(3))
[0, 1, 2]
```

A 4
$times4$-grid is apex and each of its vertices is an apex. When adding a universal vertex, the resulting graph is apex and the universal vertex is the unique apex vertex

```
sage: G = graphs.Grid2dGraph(4,4)
sage: set(G.apex_vertices()) == set(G.vertices(sort=False))
True
sage: G.add_edges([('universal',v) for v in G])
sage: G.apex_vertices()
['universal']
```

```
>>> from sage.all import *
>>> G = graphs.Grid2dGraph(Integer(4),Integer(4))
>>> set(G.apex_vertices()) == set(G.vertices(sort=False))
True
>>> G.add_edges([('universal',v) for v in G])
>>> G.apex_vertices()
['universal']
```

The Petersen graph is not apex:

```
sage: G = graphs.PetersenGraph()
sage: G.apex_vertices()
[]
```

```
>>> from sage.all import *
>>> G = graphs.PetersenGraph()
>>> G.apex_vertices()
[]
```

A graph is apex if all its connected components are apex, but at most one is not planar:

```
sage: M = graphs.Grid2dGraph(3,3)
sage: K5 = graphs.CompleteGraph(5)
sage: (M+K5).apex_vertices()
[9, 10, 11, 12, 13]
sage: (M+K5+K5).apex_vertices()
[]
```

```
>>> from sage.all import *
>>> M = graphs.Grid2dGraph(Integer(3),Integer(3))
>>> K5 = graphs.CompleteGraph(Integer(5))
>>> (M+K5).apex_vertices()
[9, 10, 11, 12, 13]
>>> (M+K5+K5).apex_vertices()
[]
```

Neighbors of an apex of degree 2 are apex:

```
sage: G = graphs.Grid2dGraph(5,5)
sage: v = (666, 666)
sage: G.add_path([(1, 1), v, (3, 3)])
sage: G.is_planar()
False
sage: G.degree(v)
2
sage: sorted(G.apex_vertices())
[(1, 1), (2, 2), (3, 3), (666, 666)]
```

```
>>> from sage.all import *
>>> G = graphs.Grid2dGraph(Integer(5),Integer(5))
>>> v = (Integer(666), Integer(666))
>>> G.add_path([(Integer(1), Integer(1)), v, (Integer(3), Integer(3))])
```

(continues on next page)

(continued from previous page)

```
>>> G.is_planar()
False
>>> G.degree(v)
2
>>> sorted(G.apex_vertices())
[(1, 1), (2, 2), (3, 3), (666, 666)]
```

arboricity(*certificate=False*, *immutable=None*)

Return the arboricity of the graph and an optional certificate.

The arboricity is the minimum number of forests that covers the graph.

See Wikipedia article Arboricity

INPUT:

- `certificate` – boolean (default: `False`); whether to return a certificate
- `immutable` – boolean (default: `None`); whether to create a mutable/immutable graph. `immutable=None` (default) means that the graph and the certificates will behave the same way.

OUTPUT:

When `certificate = True`, then the function returns (a, F) where a is the arboricity and F is a list of a disjoint forests that partitions the edge set of g. The forests are represented as subgraphs of the original graph.

If `certificate = False`, the function returns just a integer indicating the arboricity.

ALGORITHM:

Represent the graph as a graphical matroid, then apply matroid `partition` algorithm from the matroids module.

EXAMPLES:

```
sage: # needs sage.modules
sage: G = graphs.PetersenGraph()
sage: a, F = G.arboricity(True)
sage: a
2
sage: all(f.is_forest() for f in F)
True
sage: len(set.union(*[set(f.edges(sort=False)) for f in F])) == G.size()
True
```

```
>>> from sage.all import *
>>> # needs sage.modules
>>> G = graphs.PetersenGraph()
>>> a, F = G.arboricity(True)
>>> a
2
>>> all(f.is_forest() for f in F)
True
>>> len(set.union(*[set(f.edges(sort=False)) for f in F])) == G.size()
True
```

atoms_and_clique_separators(*G*, *tree=False*, *rooted_tree=False*, *separators=False*)

Return the atoms of the decomposition of G by clique minimal separators.

Let $G = (V, E)$ be a graph. A set $S \subset V$ is a clique separator if $G[S]$ is a clique and the graph $G \setminus S$ has at least 2 connected components. Let $C \subset V$ be the vertices of a connected component of $G \setminus S$. The graph $G[C + S]$ is an *atom* if it has no clique separator.

This method implements the algorithm proposed in [BPS2010], that improves upon the algorithm proposed in [TY1984], for computing the atoms and the clique minimal separators of a graph. This algorithm is based on the *maximum_cardinality_search_M()* graph traversal and has time complexity in $O(|V| \cdot |E|)$.

If the graph is not connected, we insert empty separators between the lists of separators of each connected components. See the examples below for more details.

INPUT:

- `G` – a Sage graph
- `tree` – boolean (default: `False`); whether to return the result as a directed tree in which internal nodes are clique separators and leaves are the atoms of the decomposition. Since a clique separator is repeated when its removal partition the graph into 3 or more connected components, vertices are labels by tuples (i, S), where S is the set of vertices of the atom or the clique separator, and $0 \leq i \leq |T|$.
- `rooted_tree` – boolean (default: `False`); whether to return the result as a `LabelledRootedTree`. When `tree` is `True`, this parameter is ignored.
- `separators` – boolean (default: `False`); whether to also return the complete list of separators considered during the execution of the algorithm. When `tree` or `rooted_tree` is `True`, this parameter is ignored.

OUTPUT:

- By default, return a tuple (A, S_c), where A is the list of atoms of the graph in the order of discovery, and S_c is the list of clique separators, with possible repetitions, in the order the separator has been considered. If furthermore `separators` is `True`, return a tuple (A, S_h, S_c), where S_c is the list of considered separators of the graph in the order they have been considered.
- When `tree` is `True`, format the result as a directed tree
- When `rooted_tree` is `True` and `tree` is `False`, format the output as a `LabelledRootedTree`

EXAMPLES:

Example of [BPS2010]:

```
sage: G = Graph({'a': ['b', 'k'], 'b': ['c'], 'c': ['d', 'j', 'k'],
....:            'd': ['e', 'f', 'j', 'k'], 'e': ['g'],
....:            'f': ['g', 'j', 'k'], 'g': ['j', 'k'], 'h': ['i', 'j'],
....:            'i': ['k'], 'j': ['k']})
sage: atoms, cliques = G.atoms_and_clique_separators()
sage: sorted(sorted(a) for a in atoms)
[['a', 'b', 'c', 'k'],
 ['c', 'd', 'j', 'k'],
 ['d', 'e', 'f', 'g', 'j', 'k'],
 ['h', 'i', 'j', 'k']]
sage: sorted(sorted(c) for c in cliques)
[['c', 'k'], ['d', 'j', 'k'], ['j', 'k']]
sage: T = G.atoms_and_clique_separators(tree=True)
sage: T.is_tree()
True
```

(continues on next page)

(continued from previous page)

```
sage: T.diameter() == len(atoms)
True
sage: all(u[1] in atoms for u in T if T.degree(u) == 1)
True
sage: all(u[1] in cliques for u in T if T.degree(u) != 1)
True
```

```
>>> from sage.all import *
>>> G = Graph({'a': ['b', 'k'], 'b': ['c'], 'c': ['d', 'j', 'k'],
...            'd': ['e', 'f', 'j', 'k'], 'e': ['g'],
...            'f': ['g', 'j', 'k'], 'g': ['j', 'k'], 'h': ['i', 'j'],
...            'i': ['k'], 'j': ['k']})
>>> atoms, cliques = G.atoms_and_clique_separators()
>>> sorted(sorted(a) for a in atoms)
[['a', 'b', 'c', 'k'],
 ['c', 'd', 'j', 'k'],
 ['d', 'e', 'f', 'g', 'j', 'k'],
 ['h', 'i', 'j', 'k']]
>>> sorted(sorted(c) for c in cliques)
[['c', 'k'], ['d', 'j', 'k'], ['j', 'k']]
>>> T = G.atoms_and_clique_separators(tree=True)
>>> T.is_tree()
True
>>> T.diameter() == len(atoms)
True
>>> all(u[Integer(1)] in atoms for u in T if T.degree(u) == Integer(1))
True
>>> all(u[Integer(1)] in cliques for u in T if T.degree(u) != Integer(1))
True
```

A graph without clique separator:

```
sage: G = graphs.CompleteGraph(5)
sage: G.atoms_and_clique_separators()
([{0, 1, 2, 3, 4}], [])
sage: ascii_art(G.atoms_and_clique_separators(rooted_tree=True))
{0, 1, 2, 3, 4}
```

```
>>> from sage.all import *
>>> G = graphs.CompleteGraph(Integer(5))
>>> G.atoms_and_clique_separators()
([{0, 1, 2, 3, 4}], [])
>>> ascii_art(G.atoms_and_clique_separators(rooted_tree=True))
{0, 1, 2, 3, 4}
```

Graphs with several biconnected components:

```
sage: G = graphs.PathGraph(4)
sage: ascii_art(G.atoms_and_clique_separators(rooted_tree=True))
  ____{2}____
 /           /
{2, 3}    __{1}__
```

(continues on next page)

(continued from previous page)

```
        /       /
       {1, 2} {0, 1}

sage: G = graphs.WindmillGraph(3, 4)
sage: G.atoms_and_clique_separators()
([{0, 1, 2}, {0, 3, 4}, {0, 5, 6}, {0, 8, 7}], [{0}, {0}, {0}])
sage: ascii_art(G.atoms_and_clique_separators(rooted_tree=True))
  _________{0}_________
 /                     /
{0, 1, 2}   ________{0}______
           /                /
          {0, 3, 4}   _____{0}___
                     /          /
                    {0, 8, 7} {0, 5, 6}
```

```
>>> from sage.all import *
>>> G = graphs.PathGraph(Integer(4))
>>> ascii_art(G.atoms_and_clique_separators(rooted_tree=True))
  ____{2}____
 /           /
{2, 3}   __{1}__
        /       /
       {1, 2} {0, 1}

>>> G = graphs.WindmillGraph(Integer(3), Integer(4))
>>> G.atoms_and_clique_separators()
([{0, 1, 2}, {0, 3, 4}, {0, 5, 6}, {0, 8, 7}], [{0}, {0}, {0}])
>>> ascii_art(G.atoms_and_clique_separators(rooted_tree=True))
  _________{0}_________
 /                     /
{0, 1, 2}   ________{0}______
           /                /
          {0, 3, 4}   _____{0}___
                     /          /
                    {0, 8, 7} {0, 5, 6}
```

When the removal of a clique separator results in $k > 2$ connected components, this separator is repeated $k - 1$ times, but the repetitions are not necessarily contiguous:

```
sage: G = Graph(2)
sage: for i in range(5):
....:     G.add_cycle([0, 1, G.add_vertex()])
sage: ascii_art(G.atoms_and_clique_separators(rooted_tree=True))
  __________{0, 1}_____
 /                     /
{0, 1, 4}   _________{0, 1}_____
           /                    /
          {0, 1, 2}   ________{0, 1}___
                     /                /
                    {0, 1, 3}   _____{0, 1}
                               /           /
                              {0, 1, 5} {0, 1, 6}
```

(continues on next page)

(continued from previous page)

```
sage: G = graphs.StarGraph(3)
sage: G.subdivide_edges(G.edges(sort=False), 2)
sage: ascii_art(G.atoms_and_clique_separators(rooted_tree=True))
  ______{5}______
 /               /
{1, 5}   ______{7}______
        /              /
       {2, 7}   ______{9}______
               /              /
              {9, 3}   ______{6}______
                      /              /
                     {6, 7}   ______{4}_____
                             /              /
                            {4, 5}   _____{0}_____
                                    /             /
                                   {0, 6}   ____{8}____
                                           /          /
                                          {8, 9}   __{0}__
                                                  /      /
                                                 {0, 8} {0, 4}
```

```
>>> from sage.all import *
>>> G = Graph(Integer(2))
>>> for i in range(Integer(5)):
...     G.add_cycle([Integer(0), Integer(1), G.add_vertex()])
>>> ascii_art(G.atoms_and_clique_separators(rooted_tree=True))
  _________{0, 1}_____
 /                   /
{0, 1, 4}   ________{0, 1}_____
           /                  /
          {0, 1, 2}   _______{0, 1}___
                     /                /
                    {0, 1, 3}   ____{0, 1}
                               /          /
                              {0, 1, 5} {0, 1, 6}

>>> G = graphs.StarGraph(Integer(3))
>>> G.subdivide_edges(G.edges(sort=False), Integer(2))
>>> ascii_art(G.atoms_and_clique_separators(rooted_tree=True))
  ______{5}______
 /               /
{1, 5}   ______{7}______
        /              /
       {2, 7}   ______{9}______
               /              /
              {9, 3}   ______{6}______
                      /              /
                     {6, 7}   ______{4}_____
                             /              /
                            {4, 5}   _____{0}_____
                                    /             /
```

(continues on next page)

(continued from previous page)

```
                                  {0, 6}   ____{8}____
                                          /           /
                                       {8, 9}    __{0}__
                                                /       /
                                             {0, 8} {0, 4}
```

If the graph is not connected, we insert empty separators between the lists of separators of each connected components. For instance, let G be a graph with 3 connected components. The method returns the list $S_c = [S_0, \cdots, S_i, \ldots, S_j, \ldots, S_{k-1}]$ of k clique separators, where i and j are the indexes of the inserted empty separators and $0 \leq i < j < k-1$. The method also returns the list $A = [A_0, \ldots, S_k]$ of the $k+1$ atoms, with $k+1 \geq 3$. The lists of atoms and clique separators of each of the connected components are respectively $[A_0, \ldots, A_i]$ and $[S_0, \ldots, S_{i-1}]$, $[A_{i+1}, \ldots, A_j]$ and $[S_{i+1}, \ldots, S_{j-1}]$, and $[A_{j+1}, \ldots, A_k]$ and $[S_{j+1}, \ldots, S_{k-1}]$. One can check that for each connected component, we get one atom more than clique separators:

```
sage: G = graphs.PathGraph(3) * 3
sage: A, Sc = G.atoms_and_clique_separators()
sage: A
[{1, 2}, {0, 1}, {4, 5}, {3, 4}, {8, 7}, {6, 7}]
sage: Sc
[{1}, {}, {4}, {}, {7}]
sage: i , j = [i for i, s in enumerate(Sc) if not s]
sage: i, j
(1, 3)
sage: A[:i+1], Sc[:i]
([{1, 2}, {0, 1}], [{1}])
sage: A[i+1:j+1], Sc[i+1:j]
([{4, 5}, {3, 4}], [{4}])
sage: A[j+1:], Sc[j+1:]
([{8, 7}, {6, 7}], [{7}])
sage: I = [-1, i, j, len(Sc)]
sage: for i, j in zip(I[:-1], I[1:]):
....:     print(A[i+1:j+1], Sc[i+1:j])
[{1, 2}, {0, 1}] [{1}]
[{4, 5}, {3, 4}] [{4}]
[{8, 7}, {6, 7}] [{7}]
sage: ascii_art(G.atoms_and_clique_separators(rooted_tree=True))
  ______{1}______
 /               /
{1, 2}   ______{}______
        /              /
       {0, 1}   _____{4}_____
               /             /
              {4, 5}   ____{}_____
                      /          /
                     {3, 4}   __{7}__
                             /       /
                            {6, 7} {8, 7}
```

```
>>> from sage.all import *
>>> G = graphs.PathGraph(Integer(3)) * Integer(3)
>>> A, Sc = G.atoms_and_clique_separators()
```

(continues on next page)

(continued from previous page)

```
>>> A
[{1, 2}, {0, 1}, {4, 5}, {3, 4}, {8, 7}, {6, 7}]
>>> Sc
[{1}, {}, {4}, {}, {7}]
>>> i , j = [i for i, s in enumerate(Sc) if not s]
>>> i, j
(1, 3)
>>> A[:i+Integer(1)], Sc[:i]
([{1, 2}, {0, 1}], [{1}])
>>> A[i+Integer(1):j+Integer(1)], Sc[i+Integer(1):j]
([{4, 5}, {3, 4}], [{4}])
>>> A[j+Integer(1):], Sc[j+Integer(1):]
([{8, 7}, {6, 7}], [{7}])
>>> I = [-Integer(1), i, j, len(Sc)]
>>> for i, j in zip(I[:-Integer(1)], I[Integer(1):]):
...     print(A[i+Integer(1):j+Integer(1)], Sc[i+Integer(1):j])
[{1, 2}, {0, 1}] [{1}]
[{4, 5}, {3, 4}] [{4}]
[{8, 7}, {6, 7}] [{7}]
>>> ascii_art(G.atoms_and_clique_separators(rooted_tree=True))
  ______{1}______
 /               /
{1, 2}   _______{}_______
        /               /
       {0, 1}    ______{4}______
                /              /
               {4, 5}    ____{}_____
                        /           /
                       {3, 4}    __{7}__
                                /       /
                               {6, 7} {8, 7}
```

Loops and multiple edges are ignored:

```
sage: G.allow_loops(True)
sage: G.add_edges([(u, u) for u in G])
sage: G.allow_multiple_edges(True)
sage: G.add_edges(G.edges(sort=False))
sage: ascii_art(G.atoms_and_clique_separators(rooted_tree=True))
  ______{1}______
 /               /
{1, 2}   _______{}_______
        /               /
       {0, 1}    ______{4}______
                /              /
               {4, 5}    ____{}_____
                        /           /
                       {3, 4}    __{7}__
                                /       /
                               {6, 7} {8, 7}
```

```
>>> from sage.all import *
```

(continues on next page)

(continued from previous page)

```
>>> G.allow_loops(True)
>>> G.add_edges([(u, u) for u in G])
>>> G.allow_multiple_edges(True)
>>> G.add_edges(G.edges(sort=False))
>>> ascii_art(G.atoms_and_clique_separators(rooted_tree=True))
  ______{1}______
 /               /
{1, 2}   ______{}______
        /              /
       {0, 1}   _____{4}_____
               /            /
              {4, 5}   ____{}_____
                      /          /
                     {3, 4}   __{7}__
                             /       /
                            {6, 7} {8, 7}
```

We can check that the returned list of separators is valid:

```
sage: G = graphs.RandomGNP(50, .1)
sage: while not G.is_connected():
....:     G = graphs.RandomGNP(50, .1)
sage: _, separators, _ = G.atoms_and_clique_separators(separators=True)
sage: for S in separators:
....:     H = G.copy()
....:     H.delete_vertices(S)
....:     if H.is_connected():
....:         raise ValueError("something goes wrong")
```

```
>>> from sage.all import *
>>> G = graphs.RandomGNP(Integer(50), RealNumber('.1'))
>>> while not G.is_connected():
...     G = graphs.RandomGNP(Integer(50), RealNumber('.1'))
>>> _, separators, _ = G.atoms_and_clique_separators(separators=True)
>>> for S in separators:
...     H = G.copy()
...     H.delete_vertices(S)
...     if H.is_connected():
...         raise ValueError("something goes wrong")
```

bandwidth(*G*, *k=None*)

Compute the bandwidth of an undirected graph.

For a definition of the bandwidth of a graph, see the documentation of the *bandwidth* module.

INPUT:

- `G` – a graph
- `k` – integer (default: `None`); set to an integer value to test whether $bw(G) \leq k$, or to `None` (default) to compute $bw(G)$

OUTPUT:

When k is an integer value, the function returns either `False` or an ordering of cost $\leq k$.

When k is equal to `None`, the function returns a pair `(bw, ordering)`.

See also

sage.graphs.generic_graph.GenericGraph.adjacency_matrix() – return the adjacency matrix from an ordering of the vertices.

EXAMPLES:

```
sage: from sage.graphs.graph_decompositions.bandwidth import bandwidth
sage: G = graphs.PetersenGraph()
sage: bandwidth(G,3)
False
sage: bandwidth(G)
(5, [0, 4, 5, 8, 1, 9, 3, 7, 6, 2])
sage: G.adjacency_matrix(vertices=[0, 4, 5, 8, 1, 9, 3, 7, 6, 2])
↪  # needs sage.modules
[0 1 1 0 1 0 0 0 0 0]
[1 0 0 0 0 1 1 0 0 0]
[1 0 0 1 0 0 0 1 0 0]
[0 0 1 0 0 0 1 0 1 0]
[1 0 0 0 0 0 0 0 1 1]
[0 1 0 0 0 0 0 1 1 0]
[0 1 0 1 0 0 0 0 0 1]
[0 0 1 0 0 1 0 0 0 1]
[0 0 0 1 1 1 0 0 0 0]
[0 0 0 0 1 0 1 1 0 0]
sage: G = graphs.ChvatalGraph()
sage: bandwidth(G)
(6, [0, 5, 9, 4, 10, 1, 6, 11, 3, 8, 7, 2])
sage: G.adjacency_matrix(vertices=[0, 5, 9, 4, 10, 1, 6, 11, 3, 8, 7, 2])
↪  # needs sage.modules
[0 0 1 1 0 1 1 0 0 0 0 0]
[0 0 0 1 1 1 0 1 0 0 0 0]
[1 0 0 0 1 0 0 1 1 0 0 0]
[1 1 0 0 0 0 0 0 1 1 0 0]
[0 1 1 0 0 0 1 0 0 1 0 0]
[1 1 0 0 0 0 0 0 0 0 1 1]
[1 0 0 0 1 0 0 1 0 0 0 1]
[0 1 1 0 0 0 1 0 0 0 1 0]
[0 0 1 1 0 0 0 0 0 0 1 1]
[0 0 0 1 1 0 0 0 0 0 1 1]
[0 0 0 0 0 1 0 1 1 1 0 0]
[0 0 0 0 0 1 1 0 1 1 0 0]
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_decompositions.bandwidth import bandwidth
>>> G = graphs.PetersenGraph()
>>> bandwidth(G,Integer(3))
False
>>> bandwidth(G)
(5, [0, 4, 5, 8, 1, 9, 3, 7, 6, 2])
```

(continues on next page)

(continued from previous page)

```
>>> G.adjacency_matrix(vertices=[Integer(0), Integer(4), Integer(5),␣
↪Integer(8), Integer(1), Integer(9), Integer(3), Integer(7), Integer(6),␣
↪Integer(2)])               # needs sage.modules
[0 1 1 0 1 0 0 0 0 0]
[1 0 0 0 0 1 1 0 0 0]
[1 0 0 1 0 0 0 1 0 0]
[0 0 1 0 0 0 1 0 1 0]
[1 0 0 0 0 0 0 0 1 1]
[0 1 0 0 0 0 0 1 1 0]
[0 1 0 1 0 0 0 0 0 1]
[0 0 1 0 0 1 0 0 0 1]
[0 0 0 1 1 1 0 0 0 0]
[0 0 0 0 1 0 1 1 0 0]
>>> G = graphs.ChvatalGraph()
>>> bandwidth(G)
(6, [0, 5, 9, 4, 10, 1, 6, 11, 3, 8, 7, 2])
>>> G.adjacency_matrix(vertices=[Integer(0), Integer(5), Integer(9),␣
↪Integer(4), Integer(10), Integer(1), Integer(6), Integer(11), Integer(3),␣
↪Integer(8), Integer(7), Integer(2)])       # needs sage.modules
[0 0 1 1 0 1 1 0 0 0 0 0]
[0 0 0 1 1 1 0 1 0 0 0 0]
[1 0 0 0 1 0 0 1 1 0 0 0]
[1 1 0 0 0 0 0 0 1 1 0 0]
[0 1 1 0 0 0 1 0 0 1 0 0]
[1 1 0 0 0 0 0 0 0 0 1 1]
[1 0 0 0 1 0 0 1 0 0 0 1]
[0 1 1 0 0 0 1 0 0 0 1 0]
[0 0 1 1 0 0 0 0 0 0 1 1]
[0 0 0 1 1 0 0 0 0 0 1 1]
[0 0 0 0 0 1 0 1 1 1 0 0]
[0 0 0 0 0 1 1 0 1 1 0 0]
```

bipartite_color()

Return a dictionary with vertices as the keys and the color class as the values.

Fails with an error if the graph is not bipartite.

EXAMPLES:

```
sage: graphs.CycleGraph(4).bipartite_color()
{0: 1, 1: 0, 2: 1, 3: 0}
sage: graphs.CycleGraph(5).bipartite_color()
Traceback (most recent call last):
...
RuntimeError: Graph is not bipartite.
```

```
>>> from sage.all import *
>>> graphs.CycleGraph(Integer(4)).bipartite_color()
{0: 1, 1: 0, 2: 1, 3: 0}
>>> graphs.CycleGraph(Integer(5)).bipartite_color()
Traceback (most recent call last):
...
RuntimeError: Graph is not bipartite.
```

bipartite_double(*extended=False*, *immutable=None*)

Return the (extended) bipartite double of this graph.

The bipartite double of a graph G has vertex set $\{(v, 0), (v, 1) : v \in G\}$ and for any edge (u, v) in G it has edges $((u, 0), (v, 1))$ and $((u, 1), (v, 0))$. Note that this is the tensor product of G with K_2.

The extended bipartite double of G is the bipartite double of G after added all edges $((v, 0), (v, 1))$ for all vertices v.

INPUT:

- `extended` – boolean (default: `False`); whether to return the extended bipartite double, or only the bipartite double (default)
- `immutable` – boolean (default: `None`); whether to create a mutable/immutable bipartite double. `immutable=None` (default) means that the graphs and its bipartite double will behave the same way.

OUTPUT: a graph; `self` is left untouched

EXAMPLES:

```
sage: G = graphs.PetersenGraph()
sage: H = G.bipartite_double()
sage: G == graphs.PetersenGraph()  # G is left invariant
True
sage: H.order() == 2 * G.order()
True
sage: H.size() == 2 * G.size()
True
sage: H.is_bipartite()
True
sage: H.bipartite_sets() == (set([(v, 0) for v in G]),
....: set([(v, 1) for v in G]))
True
sage: H.is_isomorphic(G.tensor_product(graphs.CompleteGraph(2)))
True
```

```
>>> from sage.all import *
>>> G = graphs.PetersenGraph()
>>> H = G.bipartite_double()
>>> G == graphs.PetersenGraph()  # G is left invariant
True
>>> H.order() == Integer(2) * G.order()
True
>>> H.size() == Integer(2) * G.size()
True
>>> H.is_bipartite()
True
>>> H.bipartite_sets() == (set([(v, Integer(0)) for v in G]),
... set([(v, Integer(1)) for v in G]))
True
>>> H.is_isomorphic(G.tensor_product(graphs.CompleteGraph(Integer(2))))
True
```

Behaviour with disconnected graphs:

```
sage: G1 = graphs.PetersenGraph()
sage: G2 = graphs.HoffmanGraph()
sage: G = G1.disjoint_union(G2)
sage: H = G.bipartite_double()
sage: H1 = G1.bipartite_double()
sage: H2 = G2.bipartite_double()
sage: H.is_isomorphic(H1.disjoint_union(H2))
True
```

```
>>> from sage.all import *
>>> G1 = graphs.PetersenGraph()
>>> G2 = graphs.HoffmanGraph()
>>> G = G1.disjoint_union(G2)
>>> H = G.bipartite_double()
>>> H1 = G1.bipartite_double()
>>> H2 = G2.bipartite_double()
>>> H.is_isomorphic(H1.disjoint_union(H2))
True
```

See also

Wikipedia article Bipartite_double_cover, WolframAlpha Bipartite Double, [VDKT2016] p. 20 for the extended bipartite double.

bipartite_sets()

Return (X, Y) where X and Y are the nodes in each bipartite set of graph G.

Fails with an error if graph is not bipartite.

EXAMPLES:

```
sage: graphs.CycleGraph(4).bipartite_sets()
({0, 2}, {1, 3})
sage: graphs.CycleGraph(5).bipartite_sets()
Traceback (most recent call last):
...
RuntimeError: Graph is not bipartite.
```

```
>>> from sage.all import *
>>> graphs.CycleGraph(Integer(4)).bipartite_sets()
({0, 2}, {1, 3})
>>> graphs.CycleGraph(Integer(5)).bipartite_sets()
Traceback (most recent call last):
...
RuntimeError: Graph is not bipartite.
```

bounded_outdegree_orientation(*G*, *bound*, *solver*, *verbose=None*, *integrality_tolerance=False*)

Return an orientation of G such that every vertex v has out-degree less than $b(v)$.

INPUT:

- `G` – an undirected graph
- `bound` – maximum bound on the out-degree. Can be of three different types :

  - An integer k. In this case, computes an orientation whose maximum out-degree is less than k.
  - A dictionary associating to each vertex its associated maximum out-degree.
  - A function associating to each vertex its associated maximum out-degree.

- `solver` – string (default: `None`); specifies a Mixed Integer Linear Programming (MILP) solver to be used. If set to `None`, the default one is used. For more information on MILP solvers and which default solver is used, see the method `solve` of the class `MixedIntegerLinearProgram`.
- `verbose` – integer (default: 0); sets the level of verbosity. Set to 0 by default, which means quiet.
- `integrality_tolerance` – float; parameter for use with MILP solvers over an inexact base ring; see `MixedIntegerLinearProgram.get_values()`.

OUTPUT:

A DiGraph representing the orientation if it exists. A `ValueError` exception is raised otherwise.

ALGORITHM:

The problem is solved through a maximum flow :

Given a graph G, we create a `DiGraph` D defined on $E(G) \cup V(G) \cup \{s, t\}$. We then link s to all of $V(G)$ (these edges having a capacity equal to the bound associated to each element of $V(G)$), and all the elements of $E(G)$ to t . We then link each $v \in V(G)$ to each of its incident edges in G. A maximum integer flow of value $|E(G)|$ corresponds to an admissible orientation of G. Otherwise, none exists.

EXAMPLES:

There is always an orientation of a graph G such that a vertex v has out-degree at most $\lceil \frac{d(v)}{2} \rceil$:

```
sage: g = graphs.RandomGNP(40, .4)
sage: b = lambda v: integer_ceil(g.degree(v)/2)
sage: D = g.bounded_outdegree_orientation(b)
sage: all(D.out_degree(v) <= b(v) for v in g)
True
```

```
>>> from sage.all import *
>>> g = graphs.RandomGNP(Integer(40), RealNumber('.4'))
>>> b = lambda v: integer_ceil(g.degree(v)/Integer(2))
>>> D = g.bounded_outdegree_orientation(b)
>>> all(D.out_degree(v) <= b(v) for v in g)
True
```

Chvatal's graph, being 4-regular, can be oriented in such a way that its maximum out-degree is 2:

```
sage: g = graphs.ChvatalGraph()
sage: D = g.bounded_outdegree_orientation(2)
sage: max(D.out_degree())
2
```

```
>>> from sage.all import *
>>> g = graphs.ChvatalGraph()
>>> D = g.bounded_outdegree_orientation(Integer(2))
>>> max(D.out_degree())
2
```

For any graph G, it is possible to compute an orientation such that the maximum out-degree is at most the maximum average degree of G divided by 2. Anything less, though, is impossible.

sage: g = graphs.RandomGNP(40, .4) sage: mad = g.maximum_average_degree() # needs sage.numerical.mip

Hence this is possible

```
sage: d = g.bounded_outdegree_orientation(integer_ceil(mad/2))                #␣
↪needs sage.numerical.mip
```

```
>>> from sage.all import *
>>> d = g.bounded_outdegree_orientation(integer_ceil(mad/Integer(2)))         ␣
↪     # needs sage.numerical.mip
```

While this is not:

```
sage: try:                                                                    #␣
↪needs sage.numerical.mip
....:     g.bounded_outdegree_orientation(integer_ceil(mad/2-1))
....:     print("Error")
....: except ValueError:
....:     pass
```

```
>>> from sage.all import *
>>> try:                                                                      #␣
↪needs sage.numerical.mip
...     g.bounded_outdegree_orientation(integer_ceil(mad/Integer(2)-
↪Integer(1)))
...     print("Error")
... except ValueError:
...     pass
```

The bounds can be specified in different ways:

```
sage: g = graphs.PetersenGraph()
sage: b = lambda v: integer_ceil(g.degree(v)/2)
sage: D = g.bounded_outdegree_orientation(b)
sage: b_dict = {u: b(u) for u in g}
sage: D = g.bounded_outdegree_orientation(b_dict)
sage: unique_bound = 2
sage: D = g.bounded_outdegree_orientation(unique_bound)
```

```
>>> from sage.all import *
>>> g = graphs.PetersenGraph()
>>> b = lambda v: integer_ceil(g.degree(v)/Integer(2))
>>> D = g.bounded_outdegree_orientation(b)
>>> b_dict = {u: b(u) for u in g}
>>> D = g.bounded_outdegree_orientation(b_dict)
>>> unique_bound = Integer(2)
>>> D = g.bounded_outdegree_orientation(unique_bound)
```

bridges(*G*, *labels=True*)

Return an iterator over the bridges (or cut edges).

A bridge is an edge whose deletion disconnects the undirected graph. A disconnected graph has no bridge.

INPUT:

- `labels` – boolean (default: `True`); if `False`, each bridge is a tuple (u, v) of vertices

EXAMPLES:

```
sage: from sage.graphs.connectivity import bridges
sage: from sage.graphs.connectivity import is_connected
sage: g = 2 * graphs.PetersenGraph()
sage: g.add_edge(1, 10)
sage: is_connected(g)
True
sage: list(bridges(g))
[(1, 10, None)]
sage: list(g.bridges())
[(1, 10, None)]
```

```
>>> from sage.all import *
>>> from sage.graphs.connectivity import bridges
>>> from sage.graphs.connectivity import is_connected
>>> g = Integer(2) * graphs.PetersenGraph()
>>> g.add_edge(Integer(1), Integer(10))
>>> is_connected(g)
True
>>> list(bridges(g))
[(1, 10, None)]
>>> list(g.bridges())
[(1, 10, None)]
```

Every edge of a tree is a bridge:

```
sage: g = graphs.RandomTree(100)
sage: sum(1 for _ in g.bridges()) == 99
True
```

```
>>> from sage.all import *
>>> g = graphs.RandomTree(Integer(100))
>>> sum(Integer(1) for _ in g.bridges()) == Integer(99)
True
```

center(*by_weight=False*, *algorithm=None*, *weight_function=None*, *check_weight=True*)

Return the set of vertices in the center of the graph.

The center is the set of vertices whose eccentricity is equal to the radius of the graph, i.e., achieving the minimum eccentricity.

For more information and examples on how to use input variables, see *shortest_paths()* and *eccentricity()*

INPUT:

- `by_weight` – boolean (default: `False`); if `True`, edge weights are taken into account; if `False`, all edges have weight 1
- `algorithm` – string (default: `None`); see method *eccentricity()* for the list of available algorithms
- `weight_function` – function (default: `None`); a function that takes as input an edge `(u, v, l)` and outputs its weight. If not `None`, `by_weight` is automatically set to `True`. If `None` and `by_weight` is `True`, we use the edge label `l` as a weight, if `l` is not `None`, else `1` as a weight.

- `check_weight` – boolean (default: `True`); if `True`, we check that the `weight_function` outputs a number for each edge

EXAMPLES:

Is Central African Republic in the center of Africa in graph theoretic sense? Yes:

```
sage: A = graphs.AfricaMap(continental=True)
sage: sorted(A.center())
['Cameroon', 'Central Africa']
```

```
>>> from sage.all import *
>>> A = graphs.AfricaMap(continental=True)
>>> sorted(A.center())
['Cameroon', 'Central Africa']
```

Some other graphs. Center can be the whole graph:

```
sage: G = graphs.DiamondGraph()
sage: G.center()
[1, 2]
sage: P = graphs.PetersenGraph()
sage: P.subgraph(P.center()) == P
True
sage: S = graphs.StarGraph(19)
sage: S.center()
[0]
```

```
>>> from sage.all import *
>>> G = graphs.DiamondGraph()
>>> G.center()
[1, 2]
>>> P = graphs.PetersenGraph()
>>> P.subgraph(P.center()) == P
True
>>> S = graphs.StarGraph(Integer(19))
>>> S.center()
[0]
```

centrality_degree(*v=None*)

Return the degree centrality of a vertex.

The degree centrality of a vertex v is its degree, divided by $|V(G)| - 1$. For more information, see the Wikipedia article Centrality.

INPUT:

- `v` – a vertex (default: `None`); set to `None` (default) to get a dictionary associating each vertex with its centrality degree

See also

- *centrality_closeness()*
- *centrality_betweenness()*

EXAMPLES:

```
sage: (graphs.ChvatalGraph()).centrality_degree()
{0: 4/11, 1: 4/11, 2: 4/11, 3: 4/11,  4: 4/11,  5: 4/11,
 6: 4/11, 7: 4/11, 8: 4/11, 9: 4/11, 10: 4/11, 11: 4/11}
sage: D = graphs.DiamondGraph()
sage: D.centrality_degree()
{0: 2/3, 1: 1, 2: 1, 3: 2/3}
sage: D.centrality_degree(v=1)
1
```

```
>>> from sage.all import *
>>> (graphs.ChvatalGraph()).centrality_degree()
{0: 4/11, 1: 4/11, 2: 4/11, 3: 4/11,  4: 4/11,  5: 4/11,
 6: 4/11, 7: 4/11, 8: 4/11, 9: 4/11, 10: 4/11, 11: 4/11}
>>> D = graphs.DiamondGraph()
>>> D.centrality_degree()
{0: 2/3, 1: 1, 2: 1, 3: 2/3}
>>> D.centrality_degree(v=Integer(1))
1
```

cheeger_constant(g)

Return the cheeger constant of the graph.

The Cheeger constant of a graph $G = (V, E)$ is the minimum of $|\partial S|/|Vol(S)|$ where $Vol(S)$ is the sum of degrees of element in S, ∂S is the edge boundary of S (number of edges with one end in S and one end in $V \setminus S$) and the minimum is taken over all non-empty subsets S of vertices so that $|Vol(S)| \leq |E|$.

See also

Alternative but similar quantities can be obtained via the methods *edge_isoperimetric_number()* and *vertex_isoperimetric_number()*.

EXAMPLES:

```
sage: graphs.PetersenGraph().cheeger_constant()
1/3
```

```
>>> from sage.all import *
>>> graphs.PetersenGraph().cheeger_constant()
1/3
```

The Cheeger constant of a cycle on n vertices is $1/\lfloor n/2 \rfloor$:

```
sage: [graphs.CycleGraph(k).cheeger_constant() for k in range(2,10)]
[1, 1, 1/2, 1/2, 1/3, 1/3, 1/4, 1/4]
```

```
>>> from sage.all import *
>>> [graphs.CycleGraph(k).cheeger_constant() for k in range(Integer(2),
↪Integer(10))]
[1, 1, 1/2, 1/2, 1/3, 1/3, 1/4, 1/4]
```

The Cheeger constant of a complete graph on n vertices is $\lceil n/2 \rceil/(n-1)$:

```
sage: [graphs.CompleteGraph(k).cheeger_constant() for k in range(2,10)]
[1, 1, 2/3, 3/4, 3/5, 2/3, 4/7, 5/8]
```

```
>>> from sage.all import *
>>> [graphs.CompleteGraph(k).cheeger_constant() for k in range(Integer(2),
↪Integer(10))]
[1, 1, 2/3, 3/4, 3/5, 2/3, 4/7, 5/8]
```

For complete bipartite:

```
sage: [graphs.CompleteBipartiteGraph(i,j).cheeger_constant() for i in range(2,
↪7) for j in range(2, i)]
[3/5, 1/2, 3/5, 5/9, 4/7, 5/9, 1/2, 5/9, 1/2, 5/9]
```

```
>>> from sage.all import *
>>> [graphs.CompleteBipartiteGraph(i,j).cheeger_constant() for i in␣
↪range(Integer(2),Integer(7)) for j in range(Integer(2), i)]
[3/5, 1/2, 3/5, 5/9, 4/7, 5/9, 1/2, 5/9, 1/2, 5/9]
```

More examples:

```
sage: G = Graph([(0, 1), (0, 3), (0, 8), (1, 4), (1, 6), (2, 4), (2, 7), (2,␣
↪9),
....:             (3, 6), (3, 8), (4, 9), (5, 6), (5, 7), (5, 8), (7, 9)])
sage: G.cheeger_constant()
1/6

sage: G = Graph([(0, 1), (0, 2), (1, 2), (1, 3), (1, 4), (1, 5), (2, 3), (3,␣
↪4), (3, 5)])
sage: G.cheeger_constant()
1/2

sage: Graph([[1,2,3,4],[(1,2),(3,4)]]).cheeger_constant()
0
```

```
>>> from sage.all import *
>>> G = Graph([(Integer(0), Integer(1)), (Integer(0), Integer(3)),␣
↪(Integer(0), Integer(8)), (Integer(1), Integer(4)), (Integer(1),␣
↪Integer(6)), (Integer(2), Integer(4)), (Integer(2), Integer(7)),␣
↪(Integer(2), Integer(9)),
...             (Integer(3), Integer(6)), (Integer(3), Integer(8)),␣
↪(Integer(4), Integer(9)), (Integer(5), Integer(6)), (Integer(5),␣
↪Integer(7)), (Integer(5), Integer(8)), (Integer(7), Integer(9))])
>>> G.cheeger_constant()
1/6

>>> G = Graph([(Integer(0), Integer(1)), (Integer(0), Integer(2)),␣
↪(Integer(1), Integer(2)), (Integer(1), Integer(3)), (Integer(1),␣
↪Integer(4)), (Integer(1), Integer(5)), (Integer(2), Integer(3)),␣
↪(Integer(3), Integer(4)), (Integer(3), Integer(5))])
>>> G.cheeger_constant()
1/2
```

(continues on next page)

(continued from previous page)

```
>>> Graph([[Integer(1),Integer(2),Integer(3),Integer(4)],[(Integer(1),
↪Integer(2)),(Integer(3),Integer(4))]]).cheeger_constant()
0
```

chromatic_index(*solver*, *verbose=None*, *integrality_tolerance=0*)

Return the chromatic index of the graph.

The chromatic index is the minimal number of colors needed to properly color the edges of the graph.

INPUT:

- `solver` – string (default: `None`); specifies a Mixed Integer Linear Programming (MILP) solver to be used. If set to `None`, the default one is used. For more information on MILP solvers and which default solver is used, see the method `solve` of the class `MixedIntegerLinearProgram`.
- `verbose` – integer (default: 0); sets the level of verbosity. Set to 0 by default, which means quiet.
- `integrality_tolerance` – float; parameter for use with MILP solvers over an inexact base ring; see `MixedIntegerLinearProgram.get_values()`.

This method is a frontend for method *sage.graphs.graph_coloring.edge_coloring()* that uses a mixed integer-linear programming formulation to compute the chromatic index.

See also

- Wikipedia article Edge_coloring for further details on edge coloring
- *sage.graphs.graph_coloring.edge_coloring()*
- *fractional_chromatic_index()*
- *chromatic_number()*

EXAMPLES:

The clique K_n has chromatic index n when n is odd and $n-1$ when n is even:

```
sage: graphs.CompleteGraph(4).chromatic_index()
3
sage: graphs.CompleteGraph(5).chromatic_index()
5
sage: graphs.CompleteGraph(6).chromatic_index()
5
```

```
>>> from sage.all import *
>>> graphs.CompleteGraph(Integer(4)).chromatic_index()
3
>>> graphs.CompleteGraph(Integer(5)).chromatic_index()
5
>>> graphs.CompleteGraph(Integer(6)).chromatic_index()
5
```

The path P_n with $n \geq 2$ has chromatic index 2:

```
sage: graphs.PathGraph(5).chromatic_index()                                 #␣
↪needs sage.numerical.mip
2
```

```
>>> from sage.all import *
>>> graphs.PathGraph(Integer(5)).chromatic_index()                          ␣
↪     # needs sage.numerical.mip
2
```

The windmill graph with parameters k, n has chromatic index $(k-1)n$:

```
sage: k,n = 3,4
sage: G = graphs.WindmillGraph(k,n)
sage: G.chromatic_index() == (k-1)*n                                        #␣
↪needs sage.numerical.mip
True
```

```
>>> from sage.all import *
>>> k,n = Integer(3),Integer(4)
>>> G = graphs.WindmillGraph(k,n)
>>> G.chromatic_index() == (k-Integer(1))*n                                 ␣
↪     # needs sage.numerical.mip
True
```

chromatic_number(*algorithm*, *solver='DLX'*, *verbose=None*, *integrality_tolerance=0*)

Return the minimal number of colors needed to color the vertices of the graph.

INPUT:

- `algorithm` – string (default: `'DLX'`); one of the following algorithms:
  - `'DLX'` (default): the chromatic number is computed using the dancing link algorithm. It is inefficient speedwise to compute the chromatic number through the dancing link algorithm because this algorithm computes *all* the possible colorings to check that one exists.
  - `'CP'`: the chromatic number is computed using the coefficients of the chromatic polynomial. Again, this method is inefficient in terms of speed and it only useful for small graphs.
  - `'MILP'`: the chromatic number is computed using a mixed integer linear program. The performance of this implementation is affected by whether optional MILP solvers have been installed (see the `MILP module`, or Sage's tutorial on Linear Programming).
  - `'parallel'`: all the above algorithms are executed in parallel and the result is returned as soon as one algorithm ends. Observe that the speed of the above algorithms depends on the size and structure of the graph.
- `solver` – string (default: `None`); specifies a Mixed Integer Linear Programming (MILP) solver to be used. If set to `None`, the default one is used. For more information on MILP solvers and which default solver is used, see the method `solve` of the class `MixedIntegerLinearProgram`.
- `verbose` – integer (default: 0); sets the level of verbosity. Set to 0 by default, which means quiet.
- `integrality_tolerance` – float; parameter for use with MILP solvers over an inexact base ring; see `MixedIntegerLinearProgram.get_values()`.

See also

For more functions related to graph coloring, see the module *sage.graphs.graph_coloring*.

EXAMPLES:

```
sage: G = Graph({0: [1, 2, 3], 1: [2]})
sage: G.chromatic_number(algorithm='DLX')
3
sage: G.chromatic_number(algorithm='MILP')
3
sage: G.chromatic_number(algorithm='CP')                                    #␣
↪needs sage.libs.flint
3
sage: G.chromatic_number(algorithm='parallel')
3
```

```
>>> from sage.all import *
>>> G = Graph({Integer(0): [Integer(1), Integer(2), Integer(3)], Integer(1):␣
↪[Integer(2)]})
>>> G.chromatic_number(algorithm='DLX')
3
>>> G.chromatic_number(algorithm='MILP')
3
>>> G.chromatic_number(algorithm='CP')                                      #␣
↪needs sage.libs.flint
3
>>> G.chromatic_number(algorithm='parallel')
3
```

A bipartite graph has (by definition) chromatic number 2:

```
sage: graphs.RandomBipartite(50,50,0.7).chromatic_number()                  #␣
↪needs numpy
2
```

```
>>> from sage.all import *
>>> graphs.RandomBipartite(Integer(50),Integer(50),RealNumber('0.7')).
↪chromatic_number()                  # needs numpy
2
```

A complete multipartite graph with k parts has chromatic number k:

```
sage: all(graphs.CompleteMultipartiteGraph([5]*i).chromatic_number() == i
....:     for i in range(2, 5))
True
```

```
>>> from sage.all import *
>>> all(graphs.CompleteMultipartiteGraph([Integer(5)]*i).chromatic_number()␣
↪== i
...     for i in range(Integer(2), Integer(5)))
True
```

The complete graph has the largest chromatic number from all the graphs of order n. Namely its chromatic number is n:

```
sage: all(graphs.CompleteGraph(i).chromatic_number() == i for i in range(10))
True
```

```
>>> from sage.all import *
>>> all(graphs.CompleteGraph(i).chromatic_number() == i for i in␣
↪range(Integer(10)))
True
```

The Kneser graph with parameters $(n, 2)$ for $n > 3$ has chromatic number $n - 2$:

```
sage: all(graphs.KneserGraph(i,2).chromatic_number() == i-2 for i in range(4,
↪6))
True
```

```
>>> from sage.all import *
>>> all(graphs.KneserGraph(i,Integer(2)).chromatic_number() == i-Integer(2)␣
↪for i in range(Integer(4),Integer(6)))
True
```

The Flower Snark graph has chromatic index 4 hence its line graph has chromatic number 4:

```
sage: graphs.FlowerSnark().line_graph().chromatic_number()
4
```

```
>>> from sage.all import *
>>> graphs.FlowerSnark().line_graph().chromatic_number()
4
```

chromatic_polynomial(*G*, *return_tree_basis=False*, *algorithm='C'*, *cache=None*)

Compute the chromatic polynomial of the graph G.

The algorithm used is a recursive one, based on the following observations of Read [Rea1968]:

- The chromatic polynomial of a tree on n vertices is x(x-1)^(n-1).
- If e is an edge of G, G' is the result of deleting the edge e, and G" is the result of contracting e, then the chromatic polynomial of G is equal to that of G' minus that of G".

INPUT:

- `G` – a Sage graph
- `return_tree_basis` – boolean (default: `False`); not used yet
- `algorithm` – string (default: `'C'`); the algorithm to use among
  - `'C'`, an implementation in C by Robert Miller and Gordon Royle.
  - `'Python'`, an implementation in Python using caching to avoid recomputing the chromatic polynomial of a graph that has already been seen. This seems faster on some dense graphs.
- `cache` – dictionary (default: `None`); this parameter is used only for algorithm `'Python'`. It is a dictionary keyed by canonical labelings of graphs and used to cache the chromatic polynomials of the graphs generated by the algorithm. In other words, it avoids computing twice the chromatic polynomial of isometric graphs. One will be created automatically if not provided.

EXAMPLES:

```
sage: graphs.CycleGraph(4).chromatic_polynomial()
x^4 - 4*x^3 + 6*x^2 - 3*x
sage: graphs.CycleGraph(3).chromatic_polynomial()
x^3 - 3*x^2 + 2*x
sage: graphs.CubeGraph(3).chromatic_polynomial()
x^8 - 12*x^7 + 66*x^6 - 214*x^5 + 441*x^4 - 572*x^3 + 423*x^2 - 133*x
sage: graphs.PetersenGraph().chromatic_polynomial()
x^10 - 15*x^9 + 105*x^8 - 455*x^7 + 1353*x^6 - 2861*x^5 + 4275*x^4 - 4305*x^3␣
↪+ 2606*x^2 - 704*x
sage: graphs.CompleteBipartiteGraph(3,3).chromatic_polynomial()
x^6 - 9*x^5 + 36*x^4 - 75*x^3 + 78*x^2 - 31*x
sage: for i in range(2,7):
....:     graphs.CompleteGraph(i).chromatic_polynomial().factor()
(x - 1) * x
(x - 2) * (x - 1) * x
(x - 3) * (x - 2) * (x - 1) * x
(x - 4) * (x - 3) * (x - 2) * (x - 1) * x
(x - 5) * (x - 4) * (x - 3) * (x - 2) * (x - 1) * x
sage: graphs.CycleGraph(5).chromatic_polynomial().factor()
(x - 2) * (x - 1) * x * (x^2 - 2*x + 2)
sage: graphs.OctahedralGraph().chromatic_polynomial().factor()
(x - 2) * (x - 1) * x * (x^3 - 9*x^2 + 29*x - 32)
sage: graphs.WheelGraph(5).chromatic_polynomial().factor()
(x - 2) * (x - 1) * x * (x^2 - 5*x + 7)
sage: graphs.WheelGraph(6).chromatic_polynomial().factor()
(x - 3) * (x - 2) * (x - 1) * x * (x^2 - 4*x + 5)
sage: C(x)=graphs.LCFGraph(24, [12,7,-7], 8).chromatic_polynomial()  # long␣
↪time (6s on sage.math, 2011)
sage: C(2)  # long time
0
```

```
>>> from sage.all import *
>>> graphs.CycleGraph(Integer(4)).chromatic_polynomial()
x^4 - 4*x^3 + 6*x^2 - 3*x
>>> graphs.CycleGraph(Integer(3)).chromatic_polynomial()
x^3 - 3*x^2 + 2*x
>>> graphs.CubeGraph(Integer(3)).chromatic_polynomial()
x^8 - 12*x^7 + 66*x^6 - 214*x^5 + 441*x^4 - 572*x^3 + 423*x^2 - 133*x
>>> graphs.PetersenGraph().chromatic_polynomial()
x^10 - 15*x^9 + 105*x^8 - 455*x^7 + 1353*x^6 - 2861*x^5 + 4275*x^4 - 4305*x^3␣
↪+ 2606*x^2 - 704*x
>>> graphs.CompleteBipartiteGraph(Integer(3),Integer(3)).chromatic_
↪polynomial()
x^6 - 9*x^5 + 36*x^4 - 75*x^3 + 78*x^2 - 31*x
>>> for i in range(Integer(2),Integer(7)):
...     graphs.CompleteGraph(i).chromatic_polynomial().factor()
(x - 1) * x
(x - 2) * (x - 1) * x
(x - 3) * (x - 2) * (x - 1) * x
(x - 4) * (x - 3) * (x - 2) * (x - 1) * x
(x - 5) * (x - 4) * (x - 3) * (x - 2) * (x - 1) * x
```

(continues on next page)

(continued from previous page)

```
>>> graphs.CycleGraph(Integer(5)).chromatic_polynomial().factor()
(x - 2) * (x - 1) * x * (x^2 - 2*x + 2)
>>> graphs.OctahedralGraph().chromatic_polynomial().factor()
(x - 2) * (x - 1) * x * (x^3 - 9*x^2 + 29*x - 32)
>>> graphs.WheelGraph(Integer(5)).chromatic_polynomial().factor()
(x - 2) * (x - 1) * x * (x^2 - 5*x + 7)
>>> graphs.WheelGraph(Integer(6)).chromatic_polynomial().factor()
(x - 3) * (x - 2) * (x - 1) * x * (x^2 - 4*x + 5)
>>> __tmp__=var("x"); C = symbolic_expression(graphs.LCFGraph(Integer(24),␣
↪[Integer(12),Integer(7),-Integer(7)], Integer(8)).chromatic_polynomial()  ).
↪function(x)# long time (6s on sage.math, 2011)
>>> C(Integer(2))  # long time
0
```

By definition, the chromatic number of a graph G is the least integer k such that the chromatic polynomial of G is strictly positive at k:

```
sage: G = graphs.PetersenGraph()
sage: P = G.chromatic_polynomial()
sage: min(i for i in range(11) if P(i) > 0) == G.chromatic_number()
True

sage: G = graphs.RandomGNP(10,0.7)
sage: P = G.chromatic_polynomial()
sage: min(i for i in range(11) if P(i) > 0) == G.chromatic_number()
True
```

```
>>> from sage.all import *
>>> G = graphs.PetersenGraph()
>>> P = G.chromatic_polynomial()
>>> min(i for i in range(Integer(11)) if P(i) > Integer(0)) == G.chromatic_
↪number()
True

>>> G = graphs.RandomGNP(Integer(10),RealNumber('0.7'))
>>> P = G.chromatic_polynomial()
>>> min(i for i in range(Integer(11)) if P(i) > Integer(0)) == G.chromatic_
↪number()
True
```

Check that algorithms `'C'` and `'Python'` return the same results:

```
sage: G = graphs.RandomGNP(8, randint(1, 9)*0.1)
sage: c = G.chromatic_polynomial(algorithm='C')
sage: p = G.chromatic_polynomial(algorithm='Python')
sage: c == p
True
```

```
>>> from sage.all import *
>>> G = graphs.RandomGNP(Integer(8), randint(Integer(1),␣
↪Integer(9))*RealNumber('0.1'))
>>> c = G.chromatic_polynomial(algorithm='C')
```

(continues on next page)

(continued from previous page)

```
>>> p = G.chromatic_polynomial(algorithm='Python')
>>> c == p
True
```

chromatic_quasisymmetric_function(*t=None*, *R=None*)

Return the chromatic quasisymmetric function of `self`.

Let G be a graph whose vertex set is totally ordered. The chromatic quasisymmetric function $X_G(t)$ was first described in [SW2012]. We use the equivalent definition given in [BC2018]:

$$X_G(t) = \sum_{\sigma=(\sigma_1,\ldots,\sigma_n)} t^{\mathrm{asc}(\sigma)} M_{|\sigma_1|,\ldots,|\sigma_n|},$$

where we sum over all ordered set partitions of the vertex set of G such that each block σ_i is an independent (i.e., stable) set of G, and where $\mathrm{asc}(\sigma)$ denotes the number of edges $\{u, v\}$ of G such that $u < v$ and v appears in a later part of σ than u.

INPUT:

- `t` – (optional) the parameter t; uses the variable t in $\mathbf{Z}[t]$ by default
- `R` – (optional) the base ring for the quasisymmetric functions; uses the parent of t by default

EXAMPLES:

```
sage: G = Graph([[1,2,3], [[1,3], [2,3]]])
sage: G.chromatic_quasisymmetric_function()
(2*t^2+2*t+2)*M[1, 1, 1] + M[1, 2] + t^2*M[2, 1]
sage: G = graphs.PathGraph(4)
sage: XG = G.chromatic_quasisymmetric_function(); XG
(t^3+11*t^2+11*t+1)*M[1, 1, 1, 1] + (3*t^2+3*t)*M[1, 1, 2]
 + (3*t^2+3*t)*M[1, 2, 1] + (3*t^2+3*t)*M[2, 1, 1]
 + (t^2+t)*M[2, 2]
sage: XG.to_symmetric_function()
(t^3+11*t^2+11*t+1)*m[1, 1, 1, 1] + (3*t^2+3*t)*m[2, 1, 1]
 + (t^2+t)*m[2, 2]
sage: G = graphs.CompleteGraph(4)
sage: G.chromatic_quasisymmetric_function()
(t^6+3*t^5+5*t^4+6*t^3+5*t^2+3*t+1)*M[1, 1, 1, 1]
```

```
>>> from sage.all import *
>>> G = Graph([[Integer(1),Integer(2),Integer(3)], [[Integer(1),Integer(3)],␣
↪[Integer(2),Integer(3)]]])
>>> G.chromatic_quasisymmetric_function()
(2*t^2+2*t+2)*M[1, 1, 1] + M[1, 2] + t^2*M[2, 1]
>>> G = graphs.PathGraph(Integer(4))
>>> XG = G.chromatic_quasisymmetric_function(); XG
(t^3+11*t^2+11*t+1)*M[1, 1, 1, 1] + (3*t^2+3*t)*M[1, 1, 2]
 + (3*t^2+3*t)*M[1, 2, 1] + (3*t^2+3*t)*M[2, 1, 1]
 + (t^2+t)*M[2, 2]
>>> XG.to_symmetric_function()
(t^3+11*t^2+11*t+1)*m[1, 1, 1, 1] + (3*t^2+3*t)*m[2, 1, 1]
 + (t^2+t)*m[2, 2]
>>> G = graphs.CompleteGraph(Integer(4))
```

(continues on next page)

(continued from previous page)

```
>>> G.chromatic_quasisymmetric_function()
(t^6+3*t^5+5*t^4+6*t^3+5*t^2+3*t+1)*M[1, 1, 1, 1]
```

Not all chromatic quasisymmetric functions are symmetric:

```
sage: G = Graph([[1,2], [1,5], [3,4], [3,5]])
sage: G.chromatic_quasisymmetric_function().is_symmetric()                    #␣
↪needs sage.combinat sage.modules
False
```

```
>>> from sage.all import *
>>> G = Graph([[Integer(1),Integer(2)], [Integer(1),Integer(5)], [Integer(3),
↪Integer(4)], [Integer(3),Integer(5)]])
>>> G.chromatic_quasisymmetric_function().is_symmetric()                     #␣
↪needs sage.combinat sage.modules
False
```

We check that at $t = 1$, we recover the usual chromatic symmetric function:

```
sage: p = SymmetricFunctions(QQ).p()                                         #␣
↪needs sage.combinat sage.modules
sage: G = graphs.CycleGraph(5)
sage: XG = G.chromatic_quasisymmetric_function(t=1); XG                      #␣
↪needs sage.combinat sage.modules
120*M[1, 1, 1, 1, 1] + 30*M[1, 1, 1, 2] + 30*M[1, 1, 2, 1]
 + 30*M[1, 2, 1, 1] + 10*M[1, 2, 2] + 30*M[2, 1, 1, 1]
 + 10*M[2, 1, 2] + 10*M[2, 2, 1]
sage: p(XG.to_symmetric_function())                                          #␣
↪needs sage.combinat sage.modules
p[1, 1, 1, 1, 1] - 5*p[2, 1, 1, 1] + 5*p[2, 2, 1]
 + 5*p[3, 1, 1] - 5*p[3, 2] - 5*p[4, 1] + 4*p[5]

sage: G = graphs.ClawGraph()
sage: XG = G.chromatic_quasisymmetric_function(t=1); XG                      #␣
↪needs sage.combinat sage.modules
24*M[1, 1, 1, 1] + 6*M[1, 1, 2] + 6*M[1, 2, 1] + M[1, 3]
 + 6*M[2, 1, 1] + M[3, 1]
sage: p(XG.to_symmetric_function())                                          #␣
↪needs sage.combinat sage.modules
p[1, 1, 1, 1] - 3*p[2, 1, 1] + 3*p[3, 1] - p[4]
```

```
>>> from sage.all import *
>>> p = SymmetricFunctions(QQ).p()                                           #␣
↪needs sage.combinat sage.modules
>>> G = graphs.CycleGraph(Integer(5))
>>> XG = G.chromatic_quasisymmetric_function(t=Integer(1)); XG                ␣
↪     # needs sage.combinat sage.modules
120*M[1, 1, 1, 1, 1] + 30*M[1, 1, 1, 2] + 30*M[1, 1, 2, 1]
 + 30*M[1, 2, 1, 1] + 10*M[1, 2, 2] + 30*M[2, 1, 1, 1]
 + 10*M[2, 1, 2] + 10*M[2, 2, 1]
>>> p(XG.to_symmetric_function())                                            #␣
↪needs sage.combinat sage.modules
```

(continues on next page)

(continued from previous page)

```
p[1, 1, 1, 1, 1] - 5*p[2, 1, 1, 1] + 5*p[2, 2, 1]
 + 5*p[3, 1, 1] - 5*p[3, 2] - 5*p[4, 1] + 4*p[5]

>>> G = graphs.ClawGraph()
>>> XG = G.chromatic_quasisymmetric_function(t=Integer(1)); XG
↪     # needs sage.combinat sage.modules
24*M[1, 1, 1, 1] + 6*M[1, 1, 2] + 6*M[1, 2, 1] + M[1, 3]
 + 6*M[2, 1, 1] + M[3, 1]
>>> p(XG.to_symmetric_function())
↪needs sage.combinat sage.modules
p[1, 1, 1, 1] - 3*p[2, 1, 1] + 3*p[3, 1] - p[4]
```

chromatic_symmetric_function(*R=None*, *weights=None*)

Return the chromatic symmetric function of `self`.

Let G be a graph. The chromatic symmetric function X_G was described in [Sta1995], specifically Theorem 2.5 states that

$$X_G = \sum_{F \subseteq E(G)} (-1)^{|F|} p_{\lambda(F)},$$

where $\lambda(F)$ is the partition of the (weighted) sizes of the connected components of the subgraph induced by the edges F and p_μ is the powersum symmetric function.

INPUT:

- `R` – (optional) the base ring for the symmetric functions; this uses **Z** by default
- `weights` – `dict` (optional); a mapping from the vertices of G to positive integers; this is $v \mapsto 1$ by default

ALGORITHM:

We traverse a binary tree whose leaves correspond to subsets of edges, and whose internal vertices at depth d correspond to a choice of whether to include the d-th edge in a given subset. The components of the induced subgraph are incrementally updated using a disjoint-set forest. If the next edge would introduce a cycle to the subset, we prune the branch as the terms produced by the two subtrees cancel in this case.

EXAMPLES:

```
sage: s = SymmetricFunctions(ZZ).s()
↪needs sage.combinat sage.modules
sage: G = graphs.CycleGraph(5)
sage: XG = G.chromatic_symmetric_function(); XG
↪needs sage.combinat sage.modules
p[1, 1, 1, 1, 1] - 5*p[2, 1, 1, 1] + 5*p[2, 2, 1]
 + 5*p[3, 1, 1] - 5*p[3, 2] - 5*p[4, 1] + 4*p[5]
sage: s(XG)
↪needs sage.combinat sage.modules
30*s[1, 1, 1, 1, 1] + 10*s[2, 1, 1, 1] + 10*s[2, 2, 1]
```

```
>>> from sage.all import *
>>> s = SymmetricFunctions(ZZ).s()
↪needs sage.combinat sage.modules
>>> G = graphs.CycleGraph(Integer(5))
>>> XG = G.chromatic_symmetric_function(); XG
```

(continues on next page)

(continued from previous page)

```
↪needs sage.combinat sage.modules
p[1, 1, 1, 1, 1] - 5*p[2, 1, 1, 1] + 5*p[2, 2, 1]
 + 5*p[3, 1, 1] - 5*p[3, 2] - 5*p[4, 1] + 4*p[5]
>>> s(XG)                                                                    #␣
↪needs sage.combinat sage.modules
30*s[1, 1, 1, 1, 1] + 10*s[2, 1, 1, 1] + 10*s[2, 2, 1]
```

Not all graphs have a positive Schur expansion:

```
sage: G = graphs.ClawGraph()
sage: XG = G.chromatic_symmetric_function(); XG                              #␣
↪needs sage.combinat sage.modules
p[1, 1, 1, 1] - 3*p[2, 1, 1] + 3*p[3, 1] - p[4]
sage: s(XG)                                                                  #␣
↪needs sage.combinat sage.modules
8*s[1, 1, 1, 1] + 5*s[2, 1, 1] - s[2, 2] + s[3, 1]
```

```
>>> from sage.all import *
>>> G = graphs.ClawGraph()
>>> XG = G.chromatic_symmetric_function(); XG                                #␣
↪needs sage.combinat sage.modules
p[1, 1, 1, 1] - 3*p[2, 1, 1] + 3*p[3, 1] - p[4]
>>> s(XG)                                                                    #␣
↪needs sage.combinat sage.modules
8*s[1, 1, 1, 1] + 5*s[2, 1, 1] - s[2, 2] + s[3, 1]
```

We show that given a triangle $\{e_1, e_2, e_3\}$, we have $X_G = X_{G-e_1} + X_{G-e_2} - X_{G-e_1-e_2}$:

```
sage: G = Graph([[1,2],[1,3],[2,3]])
sage: XG = G.chromatic_symmetric_function()
sage: G1 = copy(G)
sage: G1.delete_edge([1,2])
sage: XG1 = G1.chromatic_symmetric_function()
sage: G2 = copy(G)
sage: G2.delete_edge([1,3])
sage: XG2 = G2.chromatic_symmetric_function()
sage: G3 = copy(G1)
sage: G3.delete_edge([1,3])
sage: XG3 = G3.chromatic_symmetric_function()
sage: XG == XG1 + XG2 - XG3
True
```

```
>>> from sage.all import *
>>> G = Graph([[Integer(1),Integer(2)],[Integer(1),Integer(3)],[Integer(2),
↪Integer(3)]])
>>> XG = G.chromatic_symmetric_function()
>>> G1 = copy(G)
>>> G1.delete_edge([Integer(1),Integer(2)])
>>> XG1 = G1.chromatic_symmetric_function()
>>> G2 = copy(G)
>>> G2.delete_edge([Integer(1),Integer(3)])
>>> XG2 = G2.chromatic_symmetric_function()
```

(continues on next page)

(continued from previous page)

```
>>> G3 = copy(G1)
>>> G3.delete_edge([Integer(1),Integer(3)])
>>> XG3 = G3.chromatic_symmetric_function()
>>> XG == XG1 + XG2 - XG3
True
```

We give examples that a complete graph with weights λ produces/yields the monomial symmetric function m_λ (scaled by a constant):

```
sage: m = SymmetricFunctions(ZZ).m()
sage: K5 = graphs.CompleteGraph(5)
sage: m(K5.chromatic_symmetric_function())
120*m[1, 1, 1, 1, 1]
sage: m(K5.chromatic_symmetric_function(weights=enumerate([5,2,2,2,1])))
6*m[5, 2, 2, 2, 1]
```

```
>>> from sage.all import *
>>> m = SymmetricFunctions(ZZ).m()
>>> K5 = graphs.CompleteGraph(Integer(5))
>>> m(K5.chromatic_symmetric_function())
120*m[1, 1, 1, 1, 1]
>>> m(K5.chromatic_symmetric_function(weights=enumerate([Integer(5),
↪Integer(2),Integer(2),Integer(2),Integer(1)])))
6*m[5, 2, 2, 2, 1]
```

cleave(*G*, *cut_vertices=None*, *virtual_edges=True*, *solver=None*, *verbose=0*, *integrality_tolerance=0.001*)

Return the connected subgraphs separated by the input vertex cut.

Given a connected (multi)graph G and a vertex cut X, this method computes the list of subgraphs of G induced by each connected component c of $G \setminus X$ plus X, i.e., $G[c \cup X]$.

INPUT:

- `G` – a Graph
- `cut_vertices` – iterable container of vertices (default: `None`); a set of vertices representing a vertex cut of `G`. If no vertex cut is given, the method will compute one via a call to *vertex_connectivity()*.
- `virtual_edges` – boolean (default: `True`); whether to add virtual edges to the sides of the cut or not. A virtual edge is an edge between a pair of vertices of the cut that are not connected by an edge in `G`.
- `solver` – string (default: `None`); specifies a Mixed Integer Linear Programming (MILP) solver to be used. If set to `None`, the default one is used. For more information on MILP solvers and which default solver is used, see the method `solve` of the class `MixedIntegerLinearProgram`.
- `verbose` – integer (default: 0); sets the level of verbosity. Set to 0 by default, which means quiet.
- `integrality_tolerance` – float; parameter for use with MILP solvers over an inexact base ring; see `MixedIntegerLinearProgram.get_values()`.

OUTPUT: a triple (S, C, f), where

- S is a list of the graphs that are sides of the vertex cut.
- C is the graph of the cocycles. For each pair of vertices of the cut, if there exists an edge between them, C has one copy of each edge connecting them in `G` per sides of the cut plus one extra copy. Furthermore, when `virtual_edges == True`, if a pair of vertices of the cut is not connected by an edge in `G`, then it has one virtual edge between them per sides of the cut.

- f is the complement of the subgraph of `G` induced by the vertex cut. Hence, its vertex set is the vertex cut, and its edge set is the set of virtual edges (i.e., edges between pairs of vertices of the cut that are not connected by an edge in `G`). When `virtual_edges == False`, the edge set is empty.

EXAMPLES:

If there is an edge between cut vertices:

```
sage: from sage.graphs.connectivity import cleave
sage: G = Graph(2)
sage: for _ in range(3):
....:     G.add_clique([0, 1, G.add_vertex(), G.add_vertex()])
sage: S1,C1,f1 = cleave(G, cut_vertices=[0, 1])
sage: [g.order() for g in S1]
[4, 4, 4]
sage: C1.order(), C1.size()
(2, 4)
sage: f1.vertices(sort=True), f1.edges(sort=True)
([0, 1], [])
```

```
>>> from sage.all import *
>>> from sage.graphs.connectivity import cleave
>>> G = Graph(Integer(2))
>>> for _ in range(Integer(3)):
...     G.add_clique([Integer(0), Integer(1), G.add_vertex(), G.add_vertex()])
>>> S1,C1,f1 = cleave(G, cut_vertices=[Integer(0), Integer(1)])
>>> [g.order() for g in S1]
[4, 4, 4]
>>> C1.order(), C1.size()
(2, 4)
>>> f1.vertices(sort=True), f1.edges(sort=True)
([0, 1], [])
```

If `virtual_edges == False` and there is an edge between cut vertices:

```
sage: G.subgraph([0, 1]).complement() == Graph([[0, 1], []])
True
sage: S2,C2,f2 = cleave(G, cut_vertices=[0, 1], virtual_edges=False)
sage: (S1 == S2, C1 == C2, f1 == f2)
(True, True, True)
```

```
>>> from sage.all import *
>>> G.subgraph([Integer(0), Integer(1)]).complement() == Graph([[Integer(0),␣
↪Integer(1)], []])
True
>>> S2,C2,f2 = cleave(G, cut_vertices=[Integer(0), Integer(1)], virtual_
↪edges=False)
>>> (S1 == S2, C1 == C2, f1 == f2)
(True, True, True)
```

If cut vertices doesn't have edge between them:

```
sage: G.delete_edge(0, 1)
sage: S1,C1,f1 = cleave(G, cut_vertices=[0, 1])
```

(continues on next page)

(continued from previous page)

```
sage: [g.order() for g in S1]
[4, 4, 4]
sage: C1.order(), C1.size()
(2, 3)
sage: f1.vertices(sort=True), f1.edges(sort=True)
([0, 1], [(0, 1, None)])
```

```
>>> from sage.all import *
>>> G.delete_edge(Integer(0), Integer(1))
>>> S1,C1,f1 = cleave(G, cut_vertices=[Integer(0), Integer(1)])
>>> [g.order() for g in S1]
[4, 4, 4]
>>> C1.order(), C1.size()
(2, 3)
>>> f1.vertices(sort=True), f1.edges(sort=True)
([0, 1], [(0, 1, None)])
```

If `virtual_edges == False` and the cut vertices are not connected by an edge:

```
sage: G.subgraph([0, 1]).complement() == Graph([[0, 1], []])
False
sage: S2,C2,f2 = cleave(G, cut_vertices=[0, 1], virtual_edges=False)
sage: [g.order() for g in S2]
[4, 4, 4]
sage: C2.order(), C2.size()
(2, 0)
sage: f2.vertices(sort=True), f2.edges(sort=True)
([0, 1], [])
sage: (S1 == S2, C1 == C2, f1 == f2)
(False, False, False)
```

```
>>> from sage.all import *
>>> G.subgraph([Integer(0), Integer(1)]).complement() == Graph([[Integer(0),␣
↪Integer(1)], []])
False
>>> S2,C2,f2 = cleave(G, cut_vertices=[Integer(0), Integer(1)], virtual_
↪edges=False)
>>> [g.order() for g in S2]
[4, 4, 4]
>>> C2.order(), C2.size()
(2, 0)
>>> f2.vertices(sort=True), f2.edges(sort=True)
([0, 1], [])
>>> (S1 == S2, C1 == C2, f1 == f2)
(False, False, False)
```

If G is a biconnected multigraph:

```
sage: G = graphs.CompleteBipartiteGraph(2, 3)
sage: G.add_edge(2, 3)
sage: G.allow_multiple_edges(True)
sage: G.add_edges(G.edge_iterator())
```

(continues on next page)

(continued from previous page)

```
sage: G.add_edges([(0, 1), (0, 1), (0, 1)])
sage: S,C,f = cleave(G, cut_vertices=[0, 1])
sage: for g in S:
....:     print(g.edges(sort=True, labels=0))
[(0, 1), (0, 1), (0, 1), (0, 2), (0, 2), (0, 3), (0, 3), (1, 2), (1, 2), (1,␣
↪3), (1, 3), (2, 3), (2, 3)]
[(0, 1), (0, 1), (0, 1), (0, 4), (0, 4), (1, 4), (1, 4)]
```

```
>>> from sage.all import *
>>> G = graphs.CompleteBipartiteGraph(Integer(2), Integer(3))
>>> G.add_edge(Integer(2), Integer(3))
>>> G.allow_multiple_edges(True)
>>> G.add_edges(G.edge_iterator())
>>> G.add_edges([(Integer(0), Integer(1)), (Integer(0), Integer(1)),␣
↪(Integer(0), Integer(1))])
>>> S,C,f = cleave(G, cut_vertices=[Integer(0), Integer(1)])
>>> for g in S:
...     print(g.edges(sort=True, labels=Integer(0)))
[(0, 1), (0, 1), (0, 1), (0, 2), (0, 2), (0, 3), (0, 3), (1, 2), (1, 2), (1,␣
↪3), (1, 3), (2, 3), (2, 3)]
[(0, 1), (0, 1), (0, 1), (0, 4), (0, 4), (1, 4), (1, 4)]
```

clique_complex()

Return the clique complex of `self`.

This is the largest simplicial complex on the vertices of `self` whose 1-skeleton is `self`.

This is only makes sense for undirected simple graphs.

EXAMPLES:

```
sage: g = Graph({0:[1,2],1:[2],4:[]})
sage: g.clique_complex()
Simplicial complex with vertex set (0, 1, 2, 4) and facets {(4,), (0, 1, 2)}

sage: h = Graph({0:[1,2,3,4],1:[2,3,4],2:[3]})
sage: x = h.clique_complex()
sage: x
Simplicial complex with vertex set (0, 1, 2, 3, 4) and facets {(0, 1, 4), (0,␣
↪1, 2, 3)}
sage: i = x.graph()
sage: i==h
True
sage: x==i.clique_complex()
True
```

```
>>> from sage.all import *
>>> g = Graph({Integer(0):[Integer(1),Integer(2)],Integer(1):[Integer(2)],
↪Integer(4):[]})
>>> g.clique_complex()
Simplicial complex with vertex set (0, 1, 2, 4) and facets {(4,), (0, 1, 2)}

>>> h = Graph({Integer(0):[Integer(1),Integer(2),Integer(3),Integer(4)],
```

(continues on next page)

(continued from previous page)

```
↪Integer(1):[Integer(2),Integer(3),Integer(4)],Integer(2):[Integer(3)]})
>>> x = h.clique_complex()
>>> x
Simplicial complex with vertex set (0, 1, 2, 3, 4) and facets {(0, 1, 4), (0,␣
↪1, 2, 3)}
>>> i = x.graph()
>>> i==h
True
>>> x==i.clique_complex()
True
```

clique_maximum(*algorithm*, *solver='Cliquer'*, *verbose=None*, *integrality_tolerance=0*)

Return the vertex set of a maximal order complete subgraph.

INPUT:

- `algorithm` – the algorithm to be used :
  - If `algorithm = "Cliquer"` (default), wraps the C program Cliquer [NO2003].
  - If `algorithm = "MILP"`, the problem is solved through a Mixed Integer Linear Program.

    (see `MixedIntegerLinearProgram`)
  - If `algorithm = "mcqd"`, uses the MCQD solver (http://www.sicmm.org/~konc/maxclique/). Note that the MCQD package must be installed.
- `solver` – string (default: `None`); specifies a Mixed Integer Linear Programming (MILP) solver to be used. If set to `None`, the default one is used. For more information on MILP solvers and which default solver is used, see the method `solve` of the class `MixedIntegerLinearProgram`.
- `verbose` – integer (default: 0); sets the level of verbosity. Set to 0 by default, which means quiet.
- `integrality_tolerance` – float; parameter for use with MILP solvers over an inexact base ring; see `MixedIntegerLinearProgram.get_values()`.

Parameters `solver` and `verbose` are used only when `algorithm="MILP"`.

Note

Currently only implemented for undirected graphs. Use to_undirected to convert a digraph to an undirected graph.

ALGORITHM:

This function is based on Cliquer [NO2003].

EXAMPLES:

Using Cliquer (default):

```
sage: C = graphs.PetersenGraph()
sage: C.clique_maximum()
[7, 9]
sage: C = Graph('DJ{')
sage: C.clique_maximum()
[1, 2, 3, 4]
```

```
>>> from sage.all import *
>>> C = graphs.PetersenGraph()
>>> C.clique_maximum()
[7, 9]
>>> C = Graph('DJ{')
>>> C.clique_maximum()
[1, 2, 3, 4]
```

Through a Linear Program:

```
sage: len(C.clique_maximum(algorithm='MILP'))
4
```

```
>>> from sage.all import *
>>> len(C.clique_maximum(algorithm='MILP'))
4
```

clique_number(*algorithm*, *cliques='Cliquer'*, *solver=None*, *verbose=None*, *integrality_tolerance=0*)

Return the order of the largest clique of the graph.

This is also called as the clique number.

Note

Currently only implemented for undirected graphs. Use `to_undirected` to convert a digraph to an undirected graph.

INPUT:

- `algorithm` – the algorithm to be used :
  - If `algorithm = "Cliquer"`, wraps the C program Cliquer [NO2003].
  - If `algorithm = "networkx"`, uses the NetworkX's implementation of the Bron and Kerbosch Algorithm [BK1973].
  - If `algorithm = "MILP"`, the problem is solved through a Mixed Integer Linear Program.

    (see `MixedIntegerLinearProgram`)
  - If `algorithm = "mcqd"`, uses the MCQD solver (http://insilab.org/maxclique/). Note that the MCQD package must be installed.
- `cliques` – an optional list of cliques that can be input if already computed. Ignored unless `algorithm=="networkx"`.
- `solver` – string (default: `None`); specifies a Mixed Integer Linear Programming (MILP) solver to be used. If set to `None`, the default one is used. For more information on MILP solvers and which default solver is used, see the method `solve` of the class `MixedIntegerLinearProgram`.
- `verbose` – integer (default: 0); sets the level of verbosity. Set to 0 by default, which means quiet.
- `integrality_tolerance` – float; parameter for use with MILP solvers over an inexact base ring; see `MixedIntegerLinearProgram.get_values()`.

ALGORITHM:

This function is based on Cliquer [NO2003] and [BK1973].

EXAMPLES:

```
sage: C = Graph('DJ{')
sage: C.clique_number()
4
sage: G = Graph({0:[1,2,3], 1:[2], 3:[0,1]})
sage: G.show(figsize=[2,2])                                                    #␣
↪needs sage.plot
sage: G.clique_number()
3
```

```
>>> from sage.all import *
>>> C = Graph('DJ{')
>>> C.clique_number()
4
>>> G = Graph({Integer(0):[Integer(1),Integer(2),Integer(3)],␣
↪Integer(1):[Integer(2)], Integer(3):[Integer(0),Integer(1)]})
>>> G.show(figsize=[Integer(2),Integer(2)])                                    ␣
↪              # needs sage.plot
>>> G.clique_number()
3
```

By definition the clique number of a complete graph is its order:

```
sage: all(graphs.CompleteGraph(i).clique_number() == i for i in range(1,15))
True
```

```
>>> from sage.all import *
>>> all(graphs.CompleteGraph(i).clique_number() == i for i in␣
↪range(Integer(1),Integer(15)))
True
```

A non-empty graph without edges has a clique number of 1:

```
sage: all((i*graphs.CompleteGraph(1)).clique_number() == 1
....:     for i in range(1,15))
True
```

```
>>> from sage.all import *
>>> all((i*graphs.CompleteGraph(Integer(1))).clique_number() == Integer(1)
...     for i in range(Integer(1),Integer(15)))
True
```

A complete multipartite graph with k parts has clique number k:

```
sage: all((i*graphs.CompleteMultipartiteGraph(i*[5])).clique_number() == i
....:     for i in range(1,6))
True
```

```
>>> from sage.all import *
>>> all((i*graphs.CompleteMultipartiteGraph(i*[Integer(5)])).clique_number()␣
↪== i
```

(continues on next page)

(continued from previous page)

```
...       for i in range(Integer(1),Integer(6)))
True
```

clique_polynomial(*t=None*)

Return the clique polynomial of `self`.

This is the polynomial where the coefficient of t^n is the number of cliques in the graph with n vertices. The constant term of the clique polynomial is always taken to be one.

EXAMPLES:

```
sage: g = Graph()
sage: g.clique_polynomial()
1
sage: g = Graph({0:[1]})
sage: g.clique_polynomial()
t^2 + 2*t + 1
sage: g = graphs.CycleGraph(4)
sage: g.clique_polynomial()
4*t^2 + 4*t + 1
```

```
>>> from sage.all import *
>>> g = Graph()
>>> g.clique_polynomial()
1
>>> g = Graph({Integer(0):[Integer(1)]})
>>> g.clique_polynomial()
t^2 + 2*t + 1
>>> g = graphs.CycleGraph(Integer(4))
>>> g.clique_polynomial()
4*t^2 + 4*t + 1
```

cliques_containing_vertex(*vertices=None*, *cliques=None*)

Return the cliques containing each vertex, represented as a dictionary of lists of lists, keyed by vertex.

Returns a single list if only one input vertex.

Note

Currently only implemented for undirected graphs. Use to_undirected to convert a digraph to an undirected graph.

INPUT:

- `vertices` – the vertices to inspect (default: entire graph)
- `cliques` – list of cliques (if already computed)

EXAMPLES:

```
sage: # needs networkx
sage: C = Graph('DJ{')
sage: C.cliques_containing_vertex()
{0: [[0, 4]],
```

(continues on next page)

(continued from previous page)

```
 1: [[1, 2, 3, 4]],
 2: [[1, 2, 3, 4]],
 3: [[1, 2, 3, 4]],
 4: [[0, 4], [1, 2, 3, 4]]}
sage: C.cliques_containing_vertex(4)
[[0, 4], [1, 2, 3, 4]]
sage: C.cliques_containing_vertex([0, 1])
{0: [[0, 4]], 1: [[1, 2, 3, 4]]}
sage: E = C.cliques_maximal(); E
[[0, 4], [1, 2, 3, 4]]
sage: C.cliques_containing_vertex(cliques=E)
{0: [[0, 4]],
 1: [[1, 2, 3, 4]],
 2: [[1, 2, 3, 4]],
 3: [[1, 2, 3, 4]],
 4: [[0, 4], [1, 2, 3, 4]]}

sage: G = Graph({0:[1,2,3], 1:[2], 3:[0,1]})
sage: G.show(figsize=[2,2])                                                    #␣
↪needs sage.plot
sage: G.cliques_containing_vertex()                                            #␣
↪needs networkx
{0: [[0, 1, 2], [0, 1, 3]],
 1: [[0, 1, 2], [0, 1, 3]],
 2: [[0, 1, 2]],
 3: [[0, 1, 3]]}
```

```
>>> from sage.all import *
>>> # needs networkx
>>> C = Graph('DJ{')
>>> C.cliques_containing_vertex()
{0: [[0, 4]],
 1: [[1, 2, 3, 4]],
 2: [[1, 2, 3, 4]],
 3: [[1, 2, 3, 4]],
 4: [[0, 4], [1, 2, 3, 4]]}
>>> C.cliques_containing_vertex(Integer(4))
[[0, 4], [1, 2, 3, 4]]
>>> C.cliques_containing_vertex([Integer(0), Integer(1)])
{0: [[0, 4]], 1: [[1, 2, 3, 4]]}
>>> E = C.cliques_maximal(); E
[[0, 4], [1, 2, 3, 4]]
>>> C.cliques_containing_vertex(cliques=E)
{0: [[0, 4]],
 1: [[1, 2, 3, 4]],
 2: [[1, 2, 3, 4]],
 3: [[1, 2, 3, 4]],
 4: [[0, 4], [1, 2, 3, 4]]}

>>> G = Graph({Integer(0):[Integer(1),Integer(2),Integer(3)],␣
↪Integer(1):[Integer(2)], Integer(3):[Integer(0),Integer(1)]})
>>> G.show(figsize=[Integer(2),Integer(2)])                                    ␣
```

(continues on next page)

(continued from previous page)

```
↪              # needs sage.plot
>>> G.cliques_containing_vertex()                                    #␣
↪needs networkx
{0: [[0, 1, 2], [0, 1, 3]],
 1: [[0, 1, 2], [0, 1, 3]],
 2: [[0, 1, 2]],
 3: [[0, 1, 3]]}
```

Since each clique of a 2 dimensional grid corresponds to an edge, the number of cliques in which a vertex is involved equals its degree:

```
sage: # needs networkx
sage: F = graphs.Grid2dGraph(2,3)
sage: d = F.cliques_containing_vertex()
sage: all(F.degree(u) == len(cliques) for u,cliques in d.items())
True
sage: d = F.cliques_containing_vertex(vertices=[(0, 1)])
sage: list(d)
[(0, 1)]
sage: sorted(sorted(x for x in L) for L in d[(0, 1)])
[[(0, 0), (0, 1)], [(0, 1), (0, 2)], [(0, 1), (1, 1)]]
```

```
>>> from sage.all import *
>>> # needs networkx
>>> F = graphs.Grid2dGraph(Integer(2),Integer(3))
>>> d = F.cliques_containing_vertex()
>>> all(F.degree(u) == len(cliques) for u,cliques in d.items())
True
>>> d = F.cliques_containing_vertex(vertices=[(Integer(0), Integer(1))])
>>> list(d)
[(0, 1)]
>>> sorted(sorted(x for x in L) for L in d[(Integer(0), Integer(1))])
[[(0, 0), (0, 1)], [(0, 1), (0, 2)], [(0, 1), (1, 1)]]
```

cliques_get_clique_bipartite(***kwds*)

Return the vertex-clique bipartite graph of `self`.

In the returned bipartite graph, the `left` vertices are the vertices of `self` and the `right` vertices represent the maximal cliques of `self`. There is an edge from vertex v to clique C in the bipartite graph if and only if v belongs to C.

Note

Currently only implemented for undirected graphs. Use to_undirected to convert a digraph to an undirected graph.

EXAMPLES:

```
sage: CBG = graphs.ChvatalGraph().cliques_get_clique_bipartite(); CBG
Bipartite graph on 36 vertices
sage: CBG.show(figsize=[2,2], vertex_size=20, vertex_labels=False)          #␣
```

(continues on next page)

(continued from previous page)

```
↪needs sage.plot
sage: G = Graph({0:[1,2,3], 1:[2], 3:[0,1]})
sage: G.show(figsize=[2,2])                                                 #␣
↪needs sage.plot
sage: G.cliques_get_clique_bipartite()
Bipartite graph on 6 vertices
sage: G.cliques_get_clique_bipartite().show(figsize=[2,2])                  #␣
↪needs sage.plot
```

```
>>> from sage.all import *
>>> CBG = graphs.ChvatalGraph().cliques_get_clique_bipartite(); CBG
Bipartite graph on 36 vertices
>>> CBG.show(figsize=[Integer(2),Integer(2)], vertex_size=Integer(20), vertex_
↪labels=False)          # needs sage.plot
>>> G = Graph({Integer(0):[Integer(1),Integer(2),Integer(3)],␣
↪Integer(1):[Integer(2)], Integer(3):[Integer(0),Integer(1)]})
>>> G.show(figsize=[Integer(2),Integer(2)])                                 ␣
↪             # needs sage.plot
>>> G.cliques_get_clique_bipartite()
Bipartite graph on 6 vertices
>>> G.cliques_get_clique_bipartite().show(figsize=[Integer(2),Integer(2)])  ␣
↪             # needs sage.plot
```

cliques_get_max_clique_graph(*immutable=None*)

Return the clique graph.

Vertices of the result are the maximal cliques of the graph, and edges of the result are between maximal cliques with common members in the original graph.

For more information, see the Wikipedia article Clique_graph.

Note

Currently only implemented for undirected graphs. Use to_undirected to convert a digraph to an undirected graph.

INPUT:

- `immutable` – boolean (default: `None`); whether to create a mutable/immutable graph. `immutable=None` (default) means that the graph and its max clique graph behave the same way.

EXAMPLES:

```
sage: MCG = graphs.ChvatalGraph().cliques_get_max_clique_graph(); MCG
Graph on 24 vertices
sage: MCG.show(figsize=[2,2], vertex_size=20, vertex_labels=False)          #␣
↪needs sage.plot
sage: G = Graph({0:[1,2,3], 1:[2], 3:[0,1]})
sage: G.show(figsize=[2,2])                                                 #␣
↪needs sage.plot
sage: G.cliques_get_max_clique_graph()
Graph on 2 vertices
```

(continues on next page)

(continued from previous page)

```
sage: G.cliques_get_max_clique_graph().show(figsize=[2,2])                    #␣
↪needs sage.plot
```

```
>>> from sage.all import *
>>> MCG = graphs.ChvatalGraph().cliques_get_max_clique_graph(); MCG
Graph on 24 vertices
>>> MCG.show(figsize=[Integer(2),Integer(2)], vertex_size=Integer(20), vertex_
↪labels=False)          # needs sage.plot
>>> G = Graph({Integer(0):[Integer(1),Integer(2),Integer(3)],␣
↪Integer(1):[Integer(2)], Integer(3):[Integer(0),Integer(1)]})
>>> G.show(figsize=[Integer(2),Integer(2)])                                   ␣
↪              # needs sage.plot
>>> G.cliques_get_max_clique_graph()
Graph on 2 vertices
>>> G.cliques_get_max_clique_graph().show(figsize=[Integer(2),Integer(2)])    ␣
↪              # needs sage.plot
```

cliques_maximal(*algorithm='native'*)

Return the list of all maximal cliques.

Each clique is represented by a list of vertices. A clique is an induced complete subgraph, and a maximal clique is one not contained in a larger one.

INPUT:

- `algorithm` – can be set to `'native'` (default) to use Sage's own implementation, or to `"NetworkX"` to use NetworkX' implementation of the Bron and Kerbosch Algorithm [BK1973]

Note

Sage's implementation of the enumeration of *maximal* independent sets is not much faster than NetworkX' (expect a 2x speedup), which is surprising as it is written in Cython. This being said, the algorithm from NetworkX appears to be slightly different from this one, and that would be a good thing to explore if one wants to improve the implementation.

ALGORITHM:

This function is based on NetworkX's implementation of the Bron and Kerbosch Algorithm [BK1973].

EXAMPLES:

```
sage: graphs.ChvatalGraph().cliques_maximal()
[[0, 1], [0, 4], [0, 6], [0, 9], [1, 2], [1, 5], [1, 7], [2, 3],
 [2, 6], [2, 8], [3, 4], [3, 7], [3, 9], [4, 5], [4, 8], [5, 10],
 [5, 11], [6, 10], [6, 11], [7, 8], [7, 11], [8, 10], [9, 10], [9, 11]]
sage: G = Graph({0:[1,2,3], 1:[2], 3:[0,1]})
sage: G.show(figsize=[2, 2])                                                  #␣
↪needs sage.plot
sage: G.cliques_maximal()
[[0, 1, 2], [0, 1, 3]]
sage: C = graphs.PetersenGraph()
sage: C.cliques_maximal()
[[0, 1], [0, 4], [0, 5], [1, 2], [1, 6], [2, 3], [2, 7], [3, 4],
```

(continues on next page)

(continued from previous page)

```
 [3, 8], [4, 9], [5, 7], [5, 8], [6, 8], [6, 9], [7, 9]]
sage: C = Graph('DJ{')
sage: C.cliques_maximal()
[[0, 4], [1, 2, 3, 4]]
```

```
>>> from sage.all import *
>>> graphs.ChvatalGraph().cliques_maximal()
[[0, 1], [0, 4], [0, 6], [0, 9], [1, 2], [1, 5], [1, 7], [2, 3],
 [2, 6], [2, 8], [3, 4], [3, 7], [3, 9], [4, 5], [4, 8], [5, 10],
 [5, 11], [6, 10], [6, 11], [7, 8], [7, 11], [8, 10], [9, 10], [9, 11]]
>>> G = Graph({Integer(0):[Integer(1),Integer(2),Integer(3)],␣
↪Integer(1):[Integer(2)], Integer(3):[Integer(0),Integer(1)]})
>>> G.show(figsize=[Integer(2), Integer(2)])                                ␣
↪              # needs sage.plot
>>> G.cliques_maximal()
[[0, 1, 2], [0, 1, 3]]
>>> C = graphs.PetersenGraph()
>>> C.cliques_maximal()
[[0, 1], [0, 4], [0, 5], [1, 2], [1, 6], [2, 3], [2, 7], [3, 4],
 [3, 8], [4, 9], [5, 7], [5, 8], [6, 8], [6, 9], [7, 9]]
>>> C = Graph('DJ{')
>>> C.cliques_maximal()
[[0, 4], [1, 2, 3, 4]]
```

Comparing the two implementations:

```
sage: g = graphs.RandomGNP(20,.7)
sage: s1 = Set(map(Set, g.cliques_maximal(algorithm="NetworkX")))           #␣
↪needs networkx
sage: s2 = Set(map(Set, g.cliques_maximal(algorithm='native')))
sage: s1 == s2                                                              #␣
↪needs networkx
True
```

```
>>> from sage.all import *
>>> g = graphs.RandomGNP(Integer(20),RealNumber('.7'))
>>> s1 = Set(map(Set, g.cliques_maximal(algorithm="NetworkX")))             #␣
↪needs networkx
>>> s2 = Set(map(Set, g.cliques_maximal(algorithm='native')))
>>> s1 == s2                                                                #␣
↪needs networkx
True
```

cliques_maximum(*graph*)

alias of *all_max_clique()*.

cliques_number_of(*vertices=None*, *cliques=None*)

Return a dictionary of the number of maximal cliques containing each vertex, keyed by vertex.

This returns a single value if only one input vertex.

Note

Currently only implemented for undirected graphs. Use to_undirected to convert a digraph to an undirected graph.

INPUT:

- `vertices` – the vertices to inspect (default: entire graph)
- `cliques` – list of cliques (if already computed)

EXAMPLES:

```
sage: C = Graph('DJ{')
sage: C.cliques_number_of()
{0: 1, 1: 1, 2: 1, 3: 1, 4: 2}
sage: E = C.cliques_maximal(); E
[[0, 4], [1, 2, 3, 4]]
sage: C.cliques_number_of(cliques=E)
{0: 1, 1: 1, 2: 1, 3: 1, 4: 2}
sage: F = graphs.Grid2dGraph(2,3)
sage: F.cliques_number_of()
{(0, 0): 2, (0, 1): 3, (0, 2): 2, (1, 0): 2, (1, 1): 3, (1, 2): 2}
sage: F.cliques_number_of(vertices=[(0, 1), (1, 2)])
{(0, 1): 3, (1, 2): 2}
sage: F.cliques_number_of(vertices=(0, 1))
3
sage: G = Graph({0:[1,2,3], 1:[2], 3:[0,1]})
sage: G.show(figsize=[2,2])                                                    #␣
↪needs sage.plot
sage: G.cliques_number_of()
{0: 2, 1: 2, 2: 1, 3: 1}
```

```
>>> from sage.all import *
>>> C = Graph('DJ{')
>>> C.cliques_number_of()
{0: 1, 1: 1, 2: 1, 3: 1, 4: 2}
>>> E = C.cliques_maximal(); E
[[0, 4], [1, 2, 3, 4]]
>>> C.cliques_number_of(cliques=E)
{0: 1, 1: 1, 2: 1, 3: 1, 4: 2}
>>> F = graphs.Grid2dGraph(Integer(2),Integer(3))
>>> F.cliques_number_of()
{(0, 0): 2, (0, 1): 3, (0, 2): 2, (1, 0): 2, (1, 1): 3, (1, 2): 2}
>>> F.cliques_number_of(vertices=[(Integer(0), Integer(1)), (Integer(1),␣
↪Integer(2))])
{(0, 1): 3, (1, 2): 2}
>>> F.cliques_number_of(vertices=(Integer(0), Integer(1)))
3
>>> G = Graph({Integer(0):[Integer(1),Integer(2),Integer(3)],␣
↪Integer(1):[Integer(2)], Integer(3):[Integer(0),Integer(1)]})
>>> G.show(figsize=[Integer(2),Integer(2)])                                    ␣
↪              # needs sage.plot
>>> G.cliques_number_of()
```

(continues on next page)

(continued from previous page)

```
{0: 2, 1: 2, 2: 1, 3: 1}
```

cliques_vertex_clique_number(*algorithm='cliquer'*, *vertices=None*, *cliques=None*)

Return a dictionary of sizes of the largest maximal cliques containing each vertex, keyed by vertex.

Returns a single value if only one input vertex.

> **Note**
>
> Currently only implemented for undirected graphs. Use *to_undirected()* to convert a digraph to an undirected graph.

INPUT:

- `algorithm` – either `cliquer` or `networkx`
  - `cliquer` – this wraps the C program Cliquer [NO2003]
  - `networkx` – this function is based on NetworkX's implementation of the Bron and Kerbosch Algorithm [BK1973]
- `vertices` – the vertices to inspect (default: entire graph). Ignored unless `algorithm=='networkx'`.
- `cliques` – list of cliques (if already computed). Ignored unless `algorithm=='networkx'`.

EXAMPLES:

```
sage: C = Graph('DJ{')
sage: C.cliques_vertex_clique_number()                                    #␣
↪needs sage.plot
{0: 2, 1: 4, 2: 4, 3: 4, 4: 4}
sage: E = C.cliques_maximal(); E
[[0, 4], [1, 2, 3, 4]]
sage: C.cliques_vertex_clique_number(cliques=E, algorithm='networkx')      #␣
↪needs networkx
{0: 2, 1: 4, 2: 4, 3: 4, 4: 4}

sage: F = graphs.Grid2dGraph(2,3)
sage: F.cliques_vertex_clique_number(algorithm='networkx')                #␣
↪needs networkx
{(0, 0): 2, (0, 1): 2, (0, 2): 2, (1, 0): 2, (1, 1): 2, (1, 2): 2}
sage: F.cliques_vertex_clique_number(vertices=[(0, 1), (1, 2)])           #␣
↪needs sage.plot
{(0, 1): 2, (1, 2): 2}

sage: G = Graph({0:[1,2,3], 1:[2], 3:[0,1]})
sage: G.show(figsize=[2,2])                                               #␣
↪needs sage.plot
sage: G.cliques_vertex_clique_number()                                    #␣
↪needs sage.plot
{0: 3, 1: 3, 2: 3, 3: 3}
```

```
>>> from sage.all import *
>>> C = Graph('DJ{')
```

(continues on next page)

(continued from previous page)

```
>>> C.cliques_vertex_clique_number()                                    #␣
↪needs sage.plot
{0: 2, 1: 4, 2: 4, 3: 4, 4: 4}
>>> E = C.cliques_maximal(); E
[[0, 4], [1, 2, 3, 4]]
>>> C.cliques_vertex_clique_number(cliques=E, algorithm='networkx')      #␣
↪needs networkx
{0: 2, 1: 4, 2: 4, 3: 4, 4: 4}

>>> F = graphs.Grid2dGraph(Integer(2),Integer(3))
>>> F.cliques_vertex_clique_number(algorithm='networkx')                #␣
↪needs networkx
{(0, 0): 2, (0, 1): 2, (0, 2): 2, (1, 0): 2, (1, 1): 2, (1, 2): 2}
>>> F.cliques_vertex_clique_number(vertices=[(Integer(0), Integer(1)),␣
↪(Integer(1), Integer(2))])              # needs sage.plot
{(0, 1): 2, (1, 2): 2}

>>> G = Graph({Integer(0):[Integer(1),Integer(2),Integer(3)],␣
↪Integer(1):[Integer(2)], Integer(3):[Integer(0),Integer(1)]})
>>> G.show(figsize=[Integer(2),Integer(2)])                                ␣
↪              # needs sage.plot
>>> G.cliques_vertex_clique_number()                                    #␣
↪needs sage.plot
{0: 3, 1: 3, 2: 3, 3: 3}
```

coloring(*algorithm*, *hex_colors='DLX'*, *solver=False*, *verbose=None*, *integrality_tolerance=0*)

Return the first (optimal) proper vertex-coloring found.

INPUT:

- `algorithm` – select an algorithm from the following supported algorithms:
  - If `algorithm="DLX"` (default), the coloring is computed using the dancing link algorithm.
  - If `algorithm="MILP"`, the coloring is computed using a mixed integer linear program. The performance of this implementation is affected by whether optional MILP solvers have been installed (see the `MILP module`).
- `hex_colors` – boolean (default: `False`); if `True`, return a dictionary which can easily be used for plotting
- `solver` – string (default: `None`); specifies a Mixed Integer Linear Programming (MILP) solver to be used. If set to `None`, the default one is used. For more information on MILP solvers and which default solver is used, see the method `solve` of the class `MixedIntegerLinearProgram`.
- `verbose` – integer (default: 0); sets the level of verbosity. Set to 0 by default, which means quiet.
- `integrality_tolerance` – float; parameter for use with MILP solvers over an inexact base ring; see `MixedIntegerLinearProgram.get_values()`.

See also

For more functions related to graph coloring, see the module *sage.graphs.graph_coloring*.

EXAMPLES:

```
sage: G = Graph("Fooba")
sage: P = G.coloring(algorithm='MILP')
sage: Q = G.coloring(algorithm='DLX')
sage: def are_equal_colorings(A, B):
....:     return Set(map(Set, A)) == Set(map(Set, B))
sage: are_equal_colorings(P, [[1, 2, 3], [0, 5, 6], [4]])
True
sage: are_equal_colorings(P, Q)
True

sage: G.plot(partition=P)
Graphics object consisting of 16 graphics primitives
sage: G.coloring(hex_colors=True, algorithm='MILP')
{'#0000ff': [4], '#00ff00': [0, 6, 5], '#ff0000': [2, 1, 3]}
sage: H = G.coloring(hex_colors=True, algorithm='DLX'); H
{'#0000ff': [4], '#00ff00': [1, 2, 3], '#ff0000': [0, 5, 6]}
sage: G.plot(vertex_colors=H)
Graphics object consisting of 16 graphics primitives
```

```
>>> from sage.all import *
>>> G = Graph("Fooba")
>>> P = G.coloring(algorithm='MILP')
>>> Q = G.coloring(algorithm='DLX')
>>> def are_equal_colorings(A, B):
...     return Set(map(Set, A)) == Set(map(Set, B))
>>> are_equal_colorings(P, [[Integer(1), Integer(2), Integer(3)], [Integer(0),
↪ Integer(5), Integer(6)], [Integer(4)]])
True
>>> are_equal_colorings(P, Q)
True

>>> G.plot(partition=P)
Graphics object consisting of 16 graphics primitives
>>> G.coloring(hex_colors=True, algorithm='MILP')
{'#0000ff': [4], '#00ff00': [0, 6, 5], '#ff0000': [2, 1, 3]}
>>> H = G.coloring(hex_colors=True, algorithm='DLX'); H
{'#0000ff': [4], '#00ff00': [1, 2, 3], '#ff0000': [0, 5, 6]}
>>> G.plot(vertex_colors=H)
Graphics object consisting of 16 graphics primitives
```

common_neighbors_matrix(*vertices*, *nonedgesonly=None*, *base_ring=True*, ***kwds*)

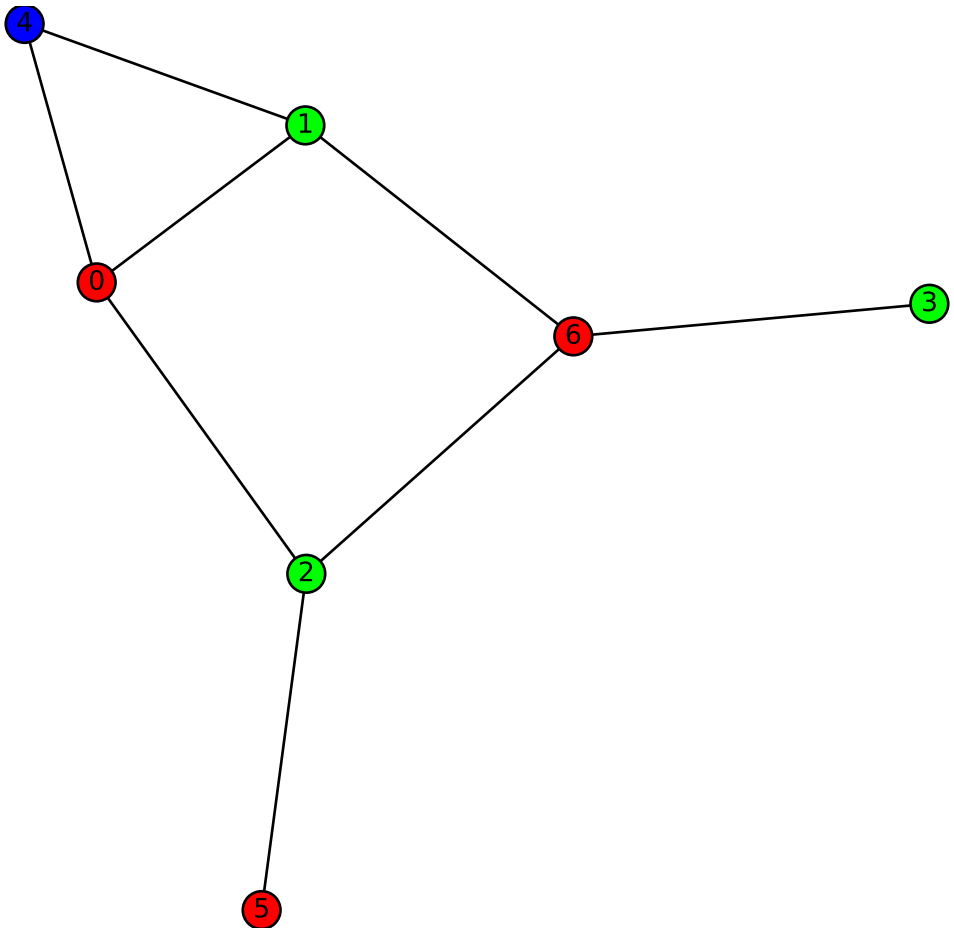
Return a matrix of numbers of common neighbors between each pairs.

The (i, j) entry of the matrix gives the number of common neighbors between vertices i and j.

This method is only valid for simple (no loops, no multiple edges) graphs.

INPUT:

- `nonedgesonly` – boolean (default: `True`); if `True`, assigns 0 value to adjacent vertices.
- `vertices` – list (default: `None`); the ordering of the vertices defining how they should appear in the matrix. By default, the ordering given by *`GenericGraph.vertices()`* is used.
- `base_ring` – a ring (default: `None`); the base ring of the matrix space to use


4
1
0
3
6
2
5

- `**kwds` – other keywords to pass to `matrix()`

OUTPUT: matrix

EXAMPLES:

The common neighbors matrix for a straight linear 2-tree counting only non-adjacent vertex pairs

```
sage: G1 = Graph()
sage: G1.add_edges([(0,1),(0,2),(1,2),(1,3),(3,5),(2,4),(2,3),(3,4),(4,5)])
sage: G1.common_neighbors_matrix(nonedgesonly=True)                          #␣
↪needs sage.modules
[0 0 0 2 1 0]
[0 0 0 0 2 1]
[0 0 0 0 0 2]
[2 0 0 0 0 0]
[1 2 0 0 0 0]
[0 1 2 0 0 0]
```

```
>>> from sage.all import *
>>> G1 = Graph()
>>> G1.add_edges([(Integer(0),Integer(1)),(Integer(0),Integer(2)),(Integer(1),
↪Integer(2)),(Integer(1),Integer(3)),(Integer(3),Integer(5)),(Integer(2),
↪Integer(4)),(Integer(2),Integer(3)),(Integer(3),Integer(4)),(Integer(4),
↪Integer(5))])
>>> G1.common_neighbors_matrix(nonedgesonly=True)                            #␣
↪needs sage.modules
[0 0 0 2 1 0]
[0 0 0 0 2 1]
[0 0 0 0 0 2]
[2 0 0 0 0 0]
[1 2 0 0 0 0]
[0 1 2 0 0 0]
```

We now show the common neighbors matrix which includes adjacent vertices

```
sage: G1.common_neighbors_matrix(nonedgesonly=False)                         #␣
↪needs sage.modules
[0 1 1 2 1 0]
[1 0 2 1 2 1]
[1 2 0 2 1 2]
[2 1 2 0 2 1]
[1 2 1 2 0 1]
[0 1 2 1 1 0]
```

```
>>> from sage.all import *
>>> G1.common_neighbors_matrix(nonedgesonly=False)                           #␣
↪needs sage.modules
[0 1 1 2 1 0]
[1 0 2 1 2 1]
[1 2 0 2 1 2]
[2 1 2 0 2 1]
[1 2 1 2 0 1]
[0 1 2 1 1 0]
```

The common neighbors matrix for a fan on 6 vertices counting only non-adjacent vertex pairs

```
sage: H = Graph([(0,1),(0,2),(0,3),(0,4),(0,5),(0,6),(1,2),(2,3),(3,4),(4,5)])
sage: H.common_neighbors_matrix()                                          #␣
↪needs sage.modules
[0 0 0 0 0 0 0]
[0 0 0 2 1 1 1]
[0 0 0 0 2 1 1]
[0 2 0 0 0 2 1]
[0 1 2 0 0 0 1]
[0 1 1 2 0 0 1]
[0 1 1 1 1 1 0]
```

```
>>> from sage.all import *
>>> H = Graph([(Integer(0),Integer(1)),(Integer(0),Integer(2)),(Integer(0),
↪Integer(3)),(Integer(0),Integer(4)),(Integer(0),Integer(5)),(Integer(0),
↪Integer(6)),(Integer(1),Integer(2)),(Integer(2),Integer(3)),(Integer(3),
↪Integer(4)),(Integer(4),Integer(5))])
>>> H.common_neighbors_matrix()                                            #␣
↪needs sage.modules
[0 0 0 0 0 0 0]
[0 0 0 2 1 1 1]
[0 0 0 0 2 1 1]
[0 2 0 0 0 2 1]
[0 1 2 0 0 0 1]
[0 1 1 2 0 0 1]
[0 1 1 1 1 1 0]
```

A different base ring:

```
sage: H.common_neighbors_matrix(base_ring=RDF)                             #␣
↪needs sage.modules
[0.0 0.0 0.0 0.0 0.0 0.0 0.0]
[0.0 0.0 0.0 2.0 1.0 1.0 1.0]
[0.0 0.0 0.0 0.0 2.0 1.0 1.0]
[0.0 2.0 0.0 0.0 0.0 2.0 1.0]
[0.0 1.0 2.0 0.0 0.0 0.0 1.0]
[0.0 1.0 1.0 2.0 0.0 0.0 1.0]
[0.0 1.0 1.0 1.0 1.0 1.0 0.0]
```

```
>>> from sage.all import *
>>> H.common_neighbors_matrix(base_ring=RDF)                               #␣
↪needs sage.modules
[0.0 0.0 0.0 0.0 0.0 0.0 0.0]
[0.0 0.0 0.0 2.0 1.0 1.0 1.0]
[0.0 0.0 0.0 0.0 2.0 1.0 1.0]
[0.0 2.0 0.0 0.0 0.0 2.0 1.0]
[0.0 1.0 2.0 0.0 0.0 0.0 1.0]
[0.0 1.0 1.0 2.0 0.0 0.0 1.0]
[0.0 1.0 1.0 1.0 1.0 1.0 0.0]
```

It is an error to input anything other than a simple graph:

```
sage: G = Graph([(0,0)], loops=True)
sage: G.common_neighbors_matrix()                                          #␣
↪needs sage.modules
Traceback (most recent call last):
...
ValueError: This method is not known to work on graphs with loops.
Perhaps this method can be updated to handle them, but in the
meantime if you want to use it please disallow loops using
allow_loops().
```

```
>>> from sage.all import *
>>> G = Graph([(Integer(0),Integer(0))], loops=True)
>>> G.common_neighbors_matrix()                                            #␣
↪needs sage.modules
Traceback (most recent call last):
...
ValueError: This method is not known to work on graphs with loops.
Perhaps this method can be updated to handle them, but in the
meantime if you want to use it please disallow loops using
allow_loops().
```

See also

- *most_common_neighbors()* – returns node pairs with most shared neighbors

convexity_properties()

Return a `ConvexityProperties` object corresponding to `self`.

This object contains the methods related to convexity in graphs (convex hull, hull number) and caches useful information so that it becomes comparatively cheaper to compute the convex hull of many different sets of the same graph.

See also

In order to know what can be done through this object, please refer to module *sage.graphs.convexity_properties*.

Note

If you want to compute many convex hulls, keep this object in memory! When it is created, it builds a table of useful information to compute convex hulls. As a result

```
sage: g = graphs.PetersenGraph()
sage: g.convexity_properties().hull([1, 3])
[1, 2, 3]
sage: g.convexity_properties().hull([3, 7])
[2, 3, 7]
```

```
>>> from sage.all import *
>>> g = graphs.PetersenGraph()
>>> g.convexity_properties().hull([Integer(1), Integer(3)])
[1, 2, 3]
>>> g.convexity_properties().hull([Integer(3), Integer(7)])
[2, 3, 7]
```

is a terrible waste of computations, while

```
sage: g = graphs.PetersenGraph()
sage: CP = g.convexity_properties()
sage: CP.hull([1, 3])
[1, 2, 3]
sage: CP.hull([3, 7])
[2, 3, 7]
```

```
>>> from sage.all import *
>>> g = graphs.PetersenGraph()
>>> CP = g.convexity_properties()
>>> CP.hull([Integer(1), Integer(3)])
[1, 2, 3]
>>> CP.hull([Integer(3), Integer(7)])
[2, 3, 7]
```

makes perfect sense.

cores(*k=None*, *with_labels=False*)

Return the core number for each vertex in an ordered list.

(for homomorphism cores, see *has_homomorphism_to()* method)

DEFINITIONS:

- *K-cores* in graph theory were introduced by Seidman in 1983 and by Bollobas in 1984 as a method of (destructively) simplifying graph topology to aid in analysis and visualization. They have been more recently defined as the following by Batagelj et al:

  Given a graph `G` with vertices set `V` and edges set `E`, the `k`-core of `G` is the graph obtained from `G` by recursively removing the vertices with degree less than `k`, for as long as there are any.

  This operation can be useful to filter or to study some properties of the graphs. For instance, when you compute the 2-core of graph G, you are cutting all the vertices which are in a tree part of graph. (A tree is a graph with no loops). See the Wikipedia article K-core.

  [PSW1996] defines a k-core of G as the largest subgraph (it is unique) of G with minimum degree at least k.

- Core number of a vertex

  The core number of a vertex v is the largest integer k such that v belongs to the k-core of G.

- Degeneracy

  The *degeneracy* of a graph G, usually denoted $\delta^*(G)$, is the smallest integer k such that the graph G can be reduced to the empty graph by iteratively removing vertices of degree $\leq k$. Equivalently, $\delta^*(G) = k-1$ if k is the smallest integer such that the k-core of G is empty.

IMPLEMENTATION:

This implementation is based on the NetworkX implementation of the algorithm described in [BZ2003].

INPUT:

- `k` – integer (default: `None`)
  - If `k = None` (default), returns the core number for each vertex.
  - If `k` is an integer, returns a pair `(ordering, core)`, where `core` is the list of vertices in the k-core of `self`, and `ordering` is an elimination order for the other vertices such that each vertex is of degree strictly less than k when it is to be eliminated from the graph.
- `with_labels` – boolean (default: `False`); when set to `False`, and `k = None`, the method returns a list whose i th element is the core number of the i th vertex. When set to `True`, the method returns a dictionary whose keys are vertices, and whose values are the corresponding core numbers.

See also

- Graph cores is also a notion related to graph homomorphisms. For this second meaning, see *has_homomorphism_to()*.
- Wikipedia article Degeneracy_(graph_theory)

EXAMPLES:

```
sage: (graphs.FruchtGraph()).cores()
[3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3]
sage: (graphs.FruchtGraph()).cores(with_labels=True)
{0: 3, 1: 3, 2: 3, 3: 3, 4: 3, 5: 3, 6: 3, 7: 3, 8: 3, 9: 3, 10: 3, 11: 3}

sage: set_random_seed(0)
sage: a = random_matrix(ZZ, 20, x=2, sparse=True, density=.1)
sage: b = Graph(20)
sage: b.add_edges(a.nonzero_positions(), loops=False)
sage: cores = b.cores(with_labels=True); cores
{0: 2, 1: 2, 2: 3, 3: 3, 4: 2, 5: 3, 6: 3, 7: 2, 8: 2,
 9: 2, 10: 3, 11: 1, 12: 3, 13: 3, 14: 3, 15: 3, 16: 3,
 17: 3, 18: 2, 19: 3}
sage: [v for v,c in cores.items() if c >= 2]  # the vertices in the 2-core
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 12, 13, 14, 15, 16, 17, 18, 19]
```

```
>>> from sage.all import *
>>> (graphs.FruchtGraph()).cores()
[3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3]
>>> (graphs.FruchtGraph()).cores(with_labels=True)
{0: 3, 1: 3, 2: 3, 3: 3, 4: 3, 5: 3, 6: 3, 7: 3, 8: 3, 9: 3, 10: 3, 11: 3}

>>> set_random_seed(Integer(0))
>>> a = random_matrix(ZZ, Integer(20), x=Integer(2), sparse=True,␣
↪density=RealNumber('.1'))
>>> b = Graph(Integer(20))
>>> b.add_edges(a.nonzero_positions(), loops=False)
>>> cores = b.cores(with_labels=True); cores
{0: 2, 1: 2, 2: 3, 3: 3, 4: 2, 5: 3, 6: 3, 7: 2, 8: 2,
 9: 2, 10: 3, 11: 1, 12: 3, 13: 3, 14: 3, 15: 3, 16: 3,
```

(continues on next page)

(continued from previous page)

```
 17: 3, 18: 2, 19: 3}
>>> [v for v,c in cores.items() if c >= Integer(2)]  # the vertices in the 2-
↪core
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 12, 13, 14, 15, 16, 17, 18, 19]
```

Checking the 2-core of a random lobster is indeed the empty set:

```
sage: g = graphs.RandomLobster(20, .5, .5)                                    #␣
↪needs networkx
sage: ordering, core = g.cores(2)                                             #␣
↪needs networkx
sage: len(core) == 0                                                          #␣
↪needs networkx
True
```

```
>>> from sage.all import *
>>> g = graphs.RandomLobster(Integer(20), RealNumber('.5'), RealNumber('.5'))␣
↪                                  # needs networkx
>>> ordering, core = g.cores(Integer(2))                                      ␣
↪     # needs networkx
>>> len(core) == Integer(0)                                                   ␣
↪     # needs networkx
True
```

Checking the cores of a bull graph:

```
sage: G = graphs.BullGraph()
sage: G.cores(with_labels=True)
{0: 2, 1: 2, 2: 2, 3: 1, 4: 1}
sage: G.cores(k=2)
([3, 4], [0, 1, 2])
```

```
>>> from sage.all import *
>>> G = graphs.BullGraph()
>>> G.cores(with_labels=True)
{0: 2, 1: 2, 2: 2, 3: 1, 4: 1}
>>> G.cores(k=Integer(2))
([3, 4], [0, 1, 2])
```

Graphs with multiple edges:

```
sage: G.allow_multiple_edges(True)
sage: G.add_edges(G.edges(sort=False))
sage: G.cores(with_labels=True)
{0: 4, 1: 4, 2: 4, 3: 2, 4: 2}
sage: G.cores(k=4)
([3, 4], [0, 1, 2])
```

```
>>> from sage.all import *
>>> G.allow_multiple_edges(True)
>>> G.add_edges(G.edges(sort=False))
>>> G.cores(with_labels=True)
```

(continues on next page)

(continued from previous page)

```
{0: 4, 1: 4, 2: 4, 3: 2, 4: 2}
>>> G.cores(k=Integer(4))
([3, 4], [0, 1, 2])
```

cutwidth(*G*, *algorithm='exponential'*, *cut_off=0*, *solver=None*, *verbose=False*, *integrality_tolerance=0.001*)

Return the cutwidth of the graph and the corresponding vertex ordering.

INPUT:

- `G` – a Graph or a DiGraph
- `algorithm` – string (default: `'exponential'`); algorithm to use among:
  - `exponential` – use an exponential time and space algorithm based on dynamic programming. This algorithm only works on graphs with strictly less than 32 vertices.
  - `MILP` – use a mixed integer linear programming formulation. This algorithm has no size restriction but could take a very long time
- `cut_off` – integer (default: 0); used to stop the search as soon as a solution with width at most `cut_off` is found, if any. If this bound cannot be reached, the best solution found is returned.
- `solver` – string (default: `None`); specifies a Mixed Integer Linear Programming (MILP) solver to be used. If set to `None`, the default one is used. For more information on MILP solvers and which default solver is used, see the method `solve` of the class `MixedIntegerLinearProgram`.
- `verbose` – boolean (default: `False`); whether to display information on the computations
- `integrality_tolerance` – float; parameter for use with MILP solvers over an inexact base ring; see `MixedIntegerLinearProgram.get_values()`.

OUTPUT:

A pair `(cost, ordering)` representing the optimal ordering of the vertices and its cost.

EXAMPLES:

Cutwidth of a Complete Graph:

```
sage: from sage.graphs.graph_decompositions.cutwidth import cutwidth
sage: G = graphs.CompleteGraph(5)
sage: cw,L = cutwidth(G); cw
6
sage: K = graphs.CompleteGraph(6)
sage: cw,L = cutwidth(K); cw
9
sage: cw,L = cutwidth(K+K); cw
9
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_decompositions.cutwidth import cutwidth
>>> G = graphs.CompleteGraph(Integer(5))
>>> cw,L = cutwidth(G); cw
6
>>> K = graphs.CompleteGraph(Integer(6))
>>> cw,L = cutwidth(K); cw
9
```

(continues on next page)

(continued from previous page)

```
>>> cw,L = cutwidth(K+K); cw
9
```

The cutwidth of a $p \times q$ Grid Graph with $p \leq q$ is $p + 1$:

```
sage: from sage.graphs.graph_decompositions.cutwidth import cutwidth
sage: G = graphs.Grid2dGraph(3,3)
sage: cw,L = cutwidth(G); cw
4
sage: G = graphs.Grid2dGraph(3,5)
sage: cw,L = cutwidth(G); cw
4
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_decompositions.cutwidth import cutwidth
>>> G = graphs.Grid2dGraph(Integer(3),Integer(3))
>>> cw,L = cutwidth(G); cw
4
>>> G = graphs.Grid2dGraph(Integer(3),Integer(5))
>>> cw,L = cutwidth(G); cw
4
```

degree_constrained_subgraph(*bounds*, *solver*, *verbose=None*, *immutable=0*, *integrality_tolerance=None*)

Return a degree-constrained subgraph.

Given a graph G and two functions $f, g : V(G) \to \mathbb{Z}$ such that $f \leq g$, a degree-constrained subgraph in G is a subgraph $G' \subseteq G$ such that for any vertex $v \in G$, $f(v) \leq d_{G'}(v) \leq g(v)$.

INPUT:

- `bounds` – (default: `None`) two possibilities:
  - A dictionary whose keys are the vertices, and values a pair of real values `(min,max)` corresponding to the values $(f(v), g(v))$.
  - A function associating to each vertex a pair of real values `(min,max)` corresponding to the values $(f(v), g(v))$.
- `solver` – string (default: `None`); specifies a Mixed Integer Linear Programming (MILP) solver to be used. If set to `None`, the default one is used. For more information on MILP solvers and which default solver is used, see the method `solve` of the class `MixedIntegerLinearProgram`.
- `verbose` – integer (default: 0); sets the level of verbosity. Set to 0 by default, which means quiet.
- `immutable` – boolean (default: `None`); whether to create a mutable/immutable graph. `immutable=None` (default) means that the graph and its degree constrained subgraph will behave the same way.
- `integrality_tolerance` – float; parameter for use with MILP solvers over an inexact base ring; see `MixedIntegerLinearProgram.get_values()`.

OUTPUT:

- When a solution exists, this method outputs the degree-constrained subgraph as a Graph object.
- When no solution exists, returns `False`.

Note

- This algorithm computes the degree-constrained subgraph of minimum weight.
- If the graph's edges are weighted, these are taken into account.
- This problem can be solved in polynomial time.

EXAMPLES:

Is there a perfect matching in an even cycle?

```
sage: g = graphs.CycleGraph(6)
sage: bounds = lambda x: [1,1]
sage: m = g.degree_constrained_subgraph(bounds=bounds)                    #␣
↪needs sage.numerical.mip
sage: m.size()                                                            #␣
↪needs sage.numerical.mip
3
```

```
>>> from sage.all import *
>>> g = graphs.CycleGraph(Integer(6))
>>> bounds = lambda x: [Integer(1),Integer(1)]
>>> m = g.degree_constrained_subgraph(bounds=bounds)                      #␣
↪needs sage.numerical.mip
>>> m.size()                                                              #␣
↪needs sage.numerical.mip
3
```

diameter(*by_weight=False*, *algorithm=None*, *weight_function=None*, *check_weight=True*)

Return the diameter of the graph.

The diameter is defined to be the maximum distance between two vertices. It is infinite if the graph is not connected.

The default algorithm is `'iFUB'` [CGILM2010] for unweighted graphs and `'DHV'` [Dragan2018] for weighted graphs.

For more information and examples on how to use input variables, see *shortest_paths()* and *eccentricity()*

INPUT:

- `by_weight` – boolean (default: `False`); if `True`, edge weights are taken into account; if `False`, all edges have weight 1
- `algorithm` – string (default: `None`); one of the following algorithms:
  - `'BFS'`: the computation is done through a BFS centered on each vertex successively. Works only if `by_weight==False`.
  - `'Floyd-Warshall-Cython'`: a Cython implementation of the Floyd-Warshall algorithm. Works only if `by_weight==False` and `v is None`.
  - `'Floyd-Warshall-Python'`: a Python implementation of the Floyd-Warshall algorithm. Works also with weighted graphs, even with negative weights (but no negative cycle is allowed). However, `v` must be `None`.

- `'Dijkstra_NetworkX'`: the Dijkstra algorithm, implemented in NetworkX. It works with weighted graphs, but no negative weight is allowed.
- `'DHV'` – diameter computation is done using the algorithm proposed in [Dragan2018]. Works only for nonnegative edge weights For more information, see *sage.graphs.distances_all_pairs.diameter()* with `algorithm='DHV'` and *sage.graphs.base.boost_graph.diameter_DHV()*.
- `'standard'`, `'2sweep'`, `'multi-sweep'`, `'iFUB'`: these algorithms are implemented in *sage.graphs.distances_all_pairs.diameter()* They work only if `by_weight==False`. See the function documentation for more information.
- `'Dijkstra_Boost'`: the Dijkstra algorithm, implemented in Boost (works only with positive weights).
- `'Johnson_Boost'`: the Johnson algorithm, implemented in Boost (works also with negative weights, if there is no negative cycle).
- `None` (default): Sage chooses the best algorithm: `'iFUB'` for unweighted graphs, `'Dijkstra_Boost'` if all weights are positive, `'Johnson_Boost'` otherwise.

- `weight_function` – function (default: `None`); a function that takes as input an edge `(u, v, l)` and outputs its weight. If not `None`, `by_weight` is automatically set to `True`. If `None` and `by_weight` is `True`, we use the edge label `l` as a weight, if `l` is not `None`, else `1` as a weight.
- `check_weight` – boolean (default: `True`); if `True`, we check that the `weight_function` outputs a number for each edge

EXAMPLES:

The more symmetric a graph is, the smaller (diameter - radius) is:

```
sage: G = graphs.BarbellGraph(9, 3)
sage: G.radius()
3
sage: G.diameter()
6
```

```
>>> from sage.all import *
>>> G = graphs.BarbellGraph(Integer(9), Integer(3))
>>> G.radius()
3
>>> G.diameter()
6
```

```
sage: G = graphs.OctahedralGraph()
sage: G.radius()
2
sage: G.diameter()
2
```

```
>>> from sage.all import *
>>> G = graphs.OctahedralGraph()
>>> G.radius()
2
>>> G.diameter()
2
```

distance_graph(*dist*, *immutable=None*)

Return the graph on the same vertex set as the original graph but vertices are adjacent in the returned graph if and only if they are at specified distances in the original graph.

INPUT:

- `dist` – nonnegative integer or a list of nonnegative integers; specified distance(s) for the connecting vertices. `Infinity` may be used here to describe vertex pairs in separate components.
- `immutable` – boolean (default: `None`); whether to create a mutable/immutable graph. `immutable=None` (default) means that the graph and its distance graph will behave the same way.

OUTPUT:

The returned value is an undirected graph. The vertex set is identical to the calling graph, but edges of the returned graph join vertices whose distance in the calling graph are present in the input `dist`. Loops will only be present if distance 0 is included. If the original graph has a position dictionary specifying locations of vertices for plotting, then this information is copied over to the distance graph. In some instances this layout may not be the best, and might even be confusing when edges run on top of each other due to symmetries chosen for the layout.

EXAMPLES:

```
sage: G = graphs.CompleteGraph(3)
sage: H = G.cartesian_product(graphs.CompleteGraph(2))
sage: K = H.distance_graph(2)
sage: K.am()                                                                #␣
↪needs sage.modules
[0 0 0 1 0 1]
[0 0 1 0 1 0]
[0 1 0 0 0 1]
[1 0 0 0 1 0]
[0 1 0 1 0 0]
[1 0 1 0 0 0]
```

```
>>> from sage.all import *
>>> G = graphs.CompleteGraph(Integer(3))
>>> H = G.cartesian_product(graphs.CompleteGraph(Integer(2)))
>>> K = H.distance_graph(Integer(2))
>>> K.am()                                                                  #␣
↪needs sage.modules
[0 0 0 1 0 1]
[0 0 1 0 1 0]
[0 1 0 0 0 1]
[1 0 0 0 1 0]
[0 1 0 1 0 0]
[1 0 1 0 0 0]
```

To obtain the graph where vertices are adjacent if their distance apart is `d` or less use a `range()` command to create the input, using `d + 1` as the input to `range`. Notice that this will include distance 0 and hence place a loop at each vertex. To avoid this, use `range(1, d + 1)`:

```
sage: G = graphs.OddGraph(4)
sage: d = G.diameter()
sage: n = G.n_vertices()
sage: H = G.distance_graph(list(range(d+1)))
```

(continues on next page)

(continued from previous page)

```
sage: H.is_isomorphic(graphs.CompleteGraph(n))
False
sage: H = G.distance_graph(list(range(1,d+1)))
sage: H.is_isomorphic(graphs.CompleteGraph(n))
True
```

```
>>> from sage.all import *
>>> G = graphs.OddGraph(Integer(4))
>>> d = G.diameter()
>>> n = G.n_vertices()
>>> H = G.distance_graph(list(range(d+Integer(1))))
>>> H.is_isomorphic(graphs.CompleteGraph(n))
False
>>> H = G.distance_graph(list(range(Integer(1),d+Integer(1))))
>>> H.is_isomorphic(graphs.CompleteGraph(n))
True
```

A complete collection of distance graphs will have adjacency matrices that sum to the matrix of all ones:

```
sage: P = graphs.PathGraph(20)
sage: all_ones = sum([P.distance_graph(i).am() for i in range(20)])          #␣
↪needs sage.modules
sage: all_ones == matrix(ZZ, 20, 20, [1]*400)                               #␣
↪needs sage.modules
True
```

```
>>> from sage.all import *
>>> P = graphs.PathGraph(Integer(20))
>>> all_ones = sum([P.distance_graph(i).am() for i in range(Integer(20))])    ␣
↪     # needs sage.modules
>>> all_ones == matrix(ZZ, Integer(20), Integer(20),␣
↪[Integer(1)]*Integer(400))                                  # needs sage.
↪modules
True
```

Four-bit strings differing in one bit is the same as four-bit strings differing in three bits:

```
sage: G = graphs.CubeGraph(4)
sage: H = G.distance_graph(3)
sage: G.is_isomorphic(H)
True
```

```
>>> from sage.all import *
>>> G = graphs.CubeGraph(Integer(4))
>>> H = G.distance_graph(Integer(3))
>>> G.is_isomorphic(H)
True
```

The graph of eight-bit strings, adjacent if different in an odd number of bits:

```
sage: # long time, needs sage.symbolic
sage: G = graphs.CubeGraph(8)
```

(continues on next page)

(continued from previous page)

```
sage: H = G.distance_graph([1,3,5,7])
sage: degrees = [0]*sum([binomial(8,j) for j in [1,3,5,7]])
sage: degrees.append(2^8)
sage: degrees == H.degree_histogram()
True
```

```
>>> from sage.all import *
>>> # long time, needs sage.symbolic
>>> G = graphs.CubeGraph(Integer(8))
>>> H = G.distance_graph([Integer(1),Integer(3),Integer(5),Integer(7)])
>>> degrees = [Integer(0)]*sum([binomial(Integer(8),j) for j in [Integer(1),
↪Integer(3),Integer(5),Integer(7)]])
>>> degrees.append(Integer(2)**Integer(8))
>>> degrees == H.degree_histogram()
True
```

An example of using `Infinity` as the distance in a graph that is not connected:

```
sage: G = graphs.CompleteGraph(3)
sage: H = G.disjoint_union(graphs.CompleteGraph(2))
sage: L = H.distance_graph(Infinity)
sage: L.am()                                                                #␣
↪needs sage.modules
[0 0 0 1 1]
[0 0 0 1 1]
[0 0 0 1 1]
[1 1 1 0 0]
[1 1 1 0 0]
sage: L.is_isomorphic(graphs.CompleteBipartiteGraph(3, 2))
True
```

```
>>> from sage.all import *
>>> G = graphs.CompleteGraph(Integer(3))
>>> H = G.disjoint_union(graphs.CompleteGraph(Integer(2)))
>>> L = H.distance_graph(Infinity)
>>> L.am()                                                                  #␣
↪needs sage.modules
[0 0 0 1 1]
[0 0 0 1 1]
[0 0 0 1 1]
[1 1 1 0 0]
[1 1 1 0 0]
>>> L.is_isomorphic(graphs.CompleteBipartiteGraph(Integer(3), Integer(2)))
True
```

AUTHOR:

Rob Beezer, 2009-11-25, Issue #7533

ear_decomposition()

Return an Ear decomposition of the graph.

An ear of an undirected graph G is a path P where the two endpoints of the path may coincide (i.e., form a cycle), but where otherwise no repetition of edges or vertices is allowed, so every internal vertex of P has

degree two in P.

An ear decomposition of an undirected graph G is a partition of its set of edges into a sequence of ears, such that the one or two endpoints of each ear belong to earlier ears in the sequence and such that the internal vertices of each ear do not belong to any earlier ear.

For more information, see the Wikipedia article Ear_decomposition.

This method implements the linear time algorithm presented in [Sch2013].

OUTPUT:

- A nested list representing the cycles and chains of the ear decomposition of the graph.

EXAMPLES:

Ear decomposition of an outer planar graph of order 13:

```
sage: g = Graph('LlCG{O@?GBOMW?')
sage: g.ear_decomposition()
[[0, 3, 2, 1, 0],
 [0, 7, 4, 3],
 [0, 11, 9, 8, 7],
 [1, 12, 2],
 [3, 6, 5, 4],
 [4, 6],
 [7, 10, 8],
 [7, 11],
 [8, 11]]
```

```
>>> from sage.all import *
>>> g = Graph('LlCG{O@?GBOMW?')
>>> g.ear_decomposition()
[[0, 3, 2, 1, 0],
 [0, 7, 4, 3],
 [0, 11, 9, 8, 7],
 [1, 12, 2],
 [3, 6, 5, 4],
 [4, 6],
 [7, 10, 8],
 [7, 11],
 [8, 11]]
```

Ear decomposition of a biconnected graph:

```
sage: g = graphs.CycleGraph(4)
sage: g.ear_decomposition()
[[0, 3, 2, 1, 0]]
```

```
>>> from sage.all import *
>>> g = graphs.CycleGraph(Integer(4))
>>> g.ear_decomposition()
[[0, 3, 2, 1, 0]]
```

Ear decomposition of a connected but not biconnected graph:

```
sage: G = Graph()
sage: G.add_cycle([0,1,2])
sage: G.add_edge(0,3)
sage: G.add_cycle([3,4,5,6])
sage: G.ear_decomposition()
[[0, 2, 1, 0], [3, 6, 5, 4, 3]]
```

```
>>> from sage.all import *
>>> G = Graph()
>>> G.add_cycle([Integer(0),Integer(1),Integer(2)])
>>> G.add_edge(Integer(0),Integer(3))
>>> G.add_cycle([Integer(3),Integer(4),Integer(5),Integer(6)])
>>> G.ear_decomposition()
[[0, 2, 1, 0], [3, 6, 5, 4, 3]]
```

The ear decomposition of a multigraph with loops is the same as the ear decomposition of the underlying simple graph:

```
sage: g = graphs.BullGraph()
sage: g.allow_multiple_edges(True)
sage: g.add_edges(g.edges(sort=False))
sage: g.allow_loops(True)
sage: u = g.random_vertex()
sage: g.add_edge(u, u)
sage: g
Bull graph: Looped multi-graph on 5 vertices
sage: h = g.to_simple()
sage: g.ear_decomposition() == h.ear_decomposition()
True
```

```
>>> from sage.all import *
>>> g = graphs.BullGraph()
>>> g.allow_multiple_edges(True)
>>> g.add_edges(g.edges(sort=False))
>>> g.allow_loops(True)
>>> u = g.random_vertex()
>>> g.add_edge(u, u)
>>> g
Bull graph: Looped multi-graph on 5 vertices
>>> h = g.to_simple()
>>> g.ear_decomposition() == h.ear_decomposition()
True
```

eccentricity(*v=None*, *by_weight=False*, *algorithm=None*, *weight_function=None*, *check_weight=True*, *dist_dict=None*, *with_labels=False*)

Return the eccentricity of vertex (or vertices) `v`.

The eccentricity of a vertex is the maximum distance to any other vertex.

For more information and examples on how to use input variables, see *shortest_path_all_pairs()*, *shortest_path_lengths()* and *shortest_paths()*

INPUT:

- `v` – either a single vertex or a list of vertices. If it is not specified, then it is taken to be all vertices

- by_weight – boolean (default: False); if True, edge weights are taken into account; if False, all edges have weight 1
- algorithm – string (default: None); one of the following algorithms:
  - 'BFS' – the computation is done through a BFS centered on each vertex successively. Works only if by_weight==False
  - 'DHV' – the computation is done using the algorithm proposed in [Dragan2018]. Works only if self has nonnegative edge weights and v is None or v should contain all vertices of self. For more information see method *sage.graphs.distances_all_pairs.eccentricity()* and *sage.graphs.base.boost_graph.eccentricity_DHV()*.
  - 'Floyd-Warshall-Cython' – a Cython implementation of the Floyd-Warshall algorithm. Works only if by_weight==False and v is None or v should contain all vertices of self.
  - 'Floyd-Warshall-Python' – a Python implementation of the Floyd-Warshall algorithm. Works also with weighted graphs, even with negative weights (but no negative cycle is allowed). However, v must be None or v should contain all vertices of self.
  - 'Dijkstra_NetworkX' – the Dijkstra algorithm, implemented in NetworkX. It works with weighted graphs, but no negative weight is allowed.
  - 'Dijkstra_Boost' – the Dijkstra algorithm, implemented in Boost (works only with positive weights)
  - 'Johnson_Boost' – the Johnson algorithm, implemented in Boost (works also with negative weights, if there is no negative cycle). Works only if v is None or v should contain all vertices of self.
  - 'From_Dictionary' – uses the (already computed) distances, that are provided by input variable dist_dict
  - None (default): Sage chooses the best algorithm: 'From_Dictionary' if dist_dict is not None, 'BFS' for unweighted graphs, 'Dijkstra_Boost' if all weights are positive, 'Johnson_Boost' otherwise.
- weight_function – function (default: None); a function that takes as input an edge (u, v, l) and outputs its weight. If not None, by_weight is automatically set to True. If None and by_weight is True, we use the edge label l as a weight, if l is not None, else 1 as a weight.
- check_weight – boolean (default: True); if True, we check that the weight_function outputs a number for each edge
- dist_dict – dictionary (default: None); a dict of dicts of distances (used only if algorithm=='From_Dictionary')
- with_labels – boolean (default: False); whether to return a list or a dictionary keyed by vertices

EXAMPLES:

```
sage: G = graphs.KrackhardtKiteGraph()
sage: G.eccentricity()
[4, 4, 4, 4, 4, 3, 3, 2, 3, 4]
sage: G.vertices(sort=True)
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
sage: G.eccentricity(7)
2
sage: G.eccentricity([7,8,9])
[2, 3, 4]
sage: G.eccentricity([7, 8, 9], with_labels=True) == {8: 3, 9: 4, 7: 2}
```

(continues on next page)

(continued from previous page)

```
True
sage: G = Graph({0: [], 1: [], 2: [1]})
sage: G.eccentricity()
[+Infinity, +Infinity, +Infinity]
sage: G = Graph({0:[]})
sage: G.eccentricity(with_labels=True)
{0: 0}
sage: G = Graph({0:[], 1:[]})
sage: G.eccentricity(with_labels=True)
{0: +Infinity, 1: +Infinity}
sage: G = Graph([(0, 1, 1), (1, 2, 1), (0, 2, 3)])
sage: G.eccentricity(algorithm='BFS')
[1, 1, 1]
sage: G.eccentricity(algorithm='Floyd-Warshall-Cython')
[1, 1, 1]
sage: G.eccentricity(by_weight=True, algorithm='Dijkstra_NetworkX')          #␣
↪needs networkx
[2, 1, 2]
sage: G.eccentricity(by_weight=True, algorithm='Dijkstra_Boost')
[2, 1, 2]
sage: G.eccentricity(by_weight=True, algorithm='Johnson_Boost')
[2, 1, 2]
sage: G.eccentricity(by_weight=True, algorithm='Floyd-Warshall-Python')
[2, 1, 2]
sage: G.eccentricity(dist_dict=G.shortest_path_all_pairs(by_weight=True)[0])
[2, 1, 2]
sage: G.eccentricity(by_weight=False, algorithm='DHV')
[1, 1, 1]
sage: G.eccentricity(by_weight=True, algorithm='DHV')
[2.0, 1.0, 2.0]
```

```
>>> from sage.all import *
>>> G = graphs.KrackhardtKiteGraph()
>>> G.eccentricity()
[4, 4, 4, 4, 4, 3, 3, 2, 3, 4]
>>> G.vertices(sort=True)
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
>>> G.eccentricity(Integer(7))
2
>>> G.eccentricity([Integer(7),Integer(8),Integer(9)])
[2, 3, 4]
>>> G.eccentricity([Integer(7), Integer(8), Integer(9)], with_labels=True) ==
↪{Integer(8): Integer(3), Integer(9): Integer(4), Integer(7): Integer(2)}
True
>>> G = Graph({Integer(0): [], Integer(1): [], Integer(2): [Integer(1)]})
>>> G.eccentricity()
[+Infinity, +Infinity, +Infinity]
>>> G = Graph({Integer(0):[]})
>>> G.eccentricity(with_labels=True)
{0: 0}
>>> G = Graph({Integer(0):[], Integer(1):[]})
>>> G.eccentricity(with_labels=True)
```

(continues on next page)

(continued from previous page)

```
{0: +Infinity, 1: +Infinity}
>>> G = Graph([(Integer(0), Integer(1), Integer(1)), (Integer(1), Integer(2),␣
↪Integer(1)), (Integer(0), Integer(2), Integer(3))])
>>> G.eccentricity(algorithm='BFS')
[1, 1, 1]
>>> G.eccentricity(algorithm='Floyd-Warshall-Cython')
[1, 1, 1]
>>> G.eccentricity(by_weight=True, algorithm='Dijkstra_NetworkX')          #␣
↪needs networkx
[2, 1, 2]
>>> G.eccentricity(by_weight=True, algorithm='Dijkstra_Boost')
[2, 1, 2]
>>> G.eccentricity(by_weight=True, algorithm='Johnson_Boost')
[2, 1, 2]
>>> G.eccentricity(by_weight=True, algorithm='Floyd-Warshall-Python')
[2, 1, 2]
>>> G.eccentricity(dist_dict=G.shortest_path_all_pairs(by_
↪weight=True)[Integer(0)])
[2, 1, 2]
>>> G.eccentricity(by_weight=False, algorithm='DHV')
[1, 1, 1]
>>> G.eccentricity(by_weight=True, algorithm='DHV')
[2.0, 1.0, 2.0]
```

edge_isoperimetric_number(*g*)

Return the edge-isoperimetric number of the graph.

The edge-isoperimetric number of a graph $G = (V, E)$ is also sometimes called the *isoperimetric number*. It is defined as the minimum of $|\partial S|/|S|$ where ∂S is the edge boundary of S (number of edges with one end in S and one end in $V \setminus S$) and the minimum is taken over all subsets of vertices whose cardinality does not exceed half the size $|V|$ of the graph.

See also

Alternative but similar quantities can be obtained via the methods *cheeger_constant()* and *vertex_isoperimetric_number()*.

EXAMPLES:

The edge-isoperimetric number of a complete graph on n vertices is $\lceil n/2 \rceil$:

```
sage: [graphs.CompleteGraph(n).edge_isoperimetric_number() for n in range(2,
↪10)]
[1, 2, 2, 3, 3, 4, 4, 5]
```

```
>>> from sage.all import *
>>> [graphs.CompleteGraph(n).edge_isoperimetric_number() for n in␣
↪range(Integer(2),Integer(10))]
[1, 2, 2, 3, 3, 4, 4, 5]
```

The edge-isoperimetric constant of a cycle on n vertices is $2/\lfloor n/2 \rfloor$:

```
sage: [graphs.CycleGraph(n).edge_isoperimetric_number() for n in range(2,15)]
[1, 2, 1, 1, 2/3, 2/3, 1/2, 1/2, 2/5, 2/5, 1/3, 1/3, 2/7]
```

```
>>> from sage.all import *
>>> [graphs.CycleGraph(n).edge_isoperimetric_number() for n in␣
↪range(Integer(2),Integer(15))]
[1, 2, 1, 1, 2/3, 2/3, 1/2, 1/2, 2/5, 2/5, 1/3, 1/3, 2/7]
```

In general, for d-regular graphs the edge-isoperimetric number is d times larger than the Cheeger constant of the graph:

```
sage: g = graphs.RandomRegular(3, 10)                                    ␣
↪  # needs networkx
sage: g.edge_isoperimetric_number() == g.cheeger_constant() * 3          ␣
↪  # needs networkx
True
```

```
>>> from sage.all import *
>>> g = graphs.RandomRegular(Integer(3), Integer(10))                    ␣
↪                  # needs networkx
>>> g.edge_isoperimetric_number() == g.cheeger_constant() * Integer(3)   ␣
↪          # needs networkx
True
```

And the edge-isoperimetric constant of a disconnected graph is 0:

```
sage: Graph([[1,2,3,4],[(1,2),(3,4)]]).edge_isoperimetric_number()
0
```

```
>>> from sage.all import *
>>> Graph([[Integer(1),Integer(2),Integer(3),Integer(4)],[(Integer(1),
↪Integer(2)),(Integer(3),Integer(4))]]).edge_isoperimetric_number()
0
```

effective_resistance(*i*, *j*, *base_ring*)

Return the effective resistance between nodes i and j.

The resistance distance between vertices i and j of a simple connected graph G is defined as the effective resistance between the two vertices on an electrical network constructed from G replacing each edge of the graph by a unit (1 ohm) resistor.

See the Wikipedia article Resistance_distance for more information.

INPUT:

- `i`, `j` – vertices of the graph
- `base_ring` – a ring (default: `None`); the base ring of the matrix space to use

OUTPUT: rational number denoting resistance between nodes i and j

EXAMPLES:

Effective resistances in a straight linear 2-tree on 6 vertices

```
sage: G = Graph([(0,1),(0,2),(1,2),(1,3),(3,5),(2,4),(2,3),(3,4),(4,5)])
sage: G.effective_resistance(0,1)
34/55
sage: G.effective_resistance(0,3)
49/55
sage: G.effective_resistance(1,4)
9/11
sage: G.effective_resistance(0,5)
15/11
```

```
>>> from sage.all import *
>>> G = Graph([(Integer(0),Integer(1)),(Integer(0),Integer(2)),(Integer(1),
↪Integer(2)),(Integer(1),Integer(3)),(Integer(3),Integer(5)),(Integer(2),
↪Integer(4)),(Integer(2),Integer(3)),(Integer(3),Integer(4)),(Integer(4),
↪Integer(5))])
>>> G.effective_resistance(Integer(0),Integer(1))
34/55
>>> G.effective_resistance(Integer(0),Integer(3))
49/55
>>> G.effective_resistance(Integer(1),Integer(4))
9/11
>>> G.effective_resistance(Integer(0),Integer(5))
15/11
```

Effective resistances in a fan on 6 vertices

```
sage: H = Graph([(0,1),(0,2),(0,3),(0,4),(0,5),(0,6),(1,2),(2,3),(3,4),(4,5)])
sage: H.effective_resistance(1,5)
6/5
sage: H.effective_resistance(1,3)
49/55
sage: H.effective_resistance(1,1)
0
```

```
>>> from sage.all import *
>>> H = Graph([(Integer(0),Integer(1)),(Integer(0),Integer(2)),(Integer(0),
↪Integer(3)),(Integer(0),Integer(4)),(Integer(0),Integer(5)),(Integer(0),
↪Integer(6)),(Integer(1),Integer(2)),(Integer(2),Integer(3)),(Integer(3),
↪Integer(4)),(Integer(4),Integer(5))])
>>> H.effective_resistance(Integer(1),Integer(5))
6/5
>>> H.effective_resistance(Integer(1),Integer(3))
49/55
>>> H.effective_resistance(Integer(1),Integer(1))
0
```

Using a different base ring:

```
sage: H.effective_resistance(1, 5, base_ring=RDF)    # abs tol 1e-14          #␣
↪needs numpy sage.modules
1.2000000000000000
sage: H.effective_resistance(1, 1, base_ring=RDF)                             #␣
```

(continues on next page)

(continued from previous page)

```
↪needs sage.modules
0.0
```

```
>>> from sage.all import *
>>> H.effective_resistance(Integer(1), Integer(5), base_ring=RDF)    # abs tol␣
↪1e-14          # needs numpy sage.modules
1.2000000000000000
>>> H.effective_resistance(Integer(1), Integer(1), base_ring=RDF)            ␣
↪               # needs sage.modules
0.0
```

See also

- *effective_resistance_matrix()* – a similar method giving a matrix full of all effective resistances between all nodes
- *least_effective_resistance()* – gives node pairs with least effective resistances
- See Wikipedia article Resistance_distance for more details.

effective_resistance_matrix(*vertices*, *nonedgesonly=None*, *base_ring=True*, ***kwds*)

Return a matrix whose (i, j) entry gives the effective resistance between vertices i and j.

The resistance distance between vertices i and j of a simple connected graph G is defined as the effective resistance between the two vertices on an electrical network constructed from G replacing each edge of the graph by a unit (1 ohm) resistor.

By default, the matrix returned is over the rationals.

INPUT:

- `nonedgesonly` – boolean (default: `True`); if `True` assign zero resistance to pairs of adjacent vertices.
- `vertices` – list (default: `None`); the ordering of the vertices defining how they should appear in the matrix. By default, the ordering given by *GenericGraph.vertices()* is used.
- `base_ring` – a ring (default: `None`); the base ring of the matrix space to use
- `**kwds` – other keywords to pass to `matrix()`

OUTPUT: matrix

EXAMPLES:

The effective resistance matrix for a straight linear 2-tree counting only non-adjacent vertex pairs

```
sage: G = Graph([(0,1),(0,2),(1,2),(1,3),(3,5),(2,4),(2,3),(3,4),(4,5)])
sage: G.effective_resistance_matrix()                                        #␣
↪needs sage.modules
[    0     0     0 49/55 59/55 15/11]
[    0     0     0     0  9/11 59/55]
[    0     0     0     0     0 49/55]
[49/55     0     0     0     0     0]
[59/55  9/11     0     0     0     0]
[15/11 59/55 49/55     0     0     0]
```

```
>>> from sage.all import *
>>> G = Graph([(Integer(0),Integer(1)),(Integer(0),Integer(2)),(Integer(1),
↪Integer(2)),(Integer(1),Integer(3)),(Integer(3),Integer(5)),(Integer(2),
↪Integer(4)),(Integer(2),Integer(3)),(Integer(3),Integer(4)),(Integer(4),
↪Integer(5))])
>>> G.effective_resistance_matrix()                                    #␣
↪needs sage.modules
[    0     0     0 49/55 59/55 15/11]
[    0     0     0     0  9/11 59/55]
[    0     0     0     0     0 49/55]
[49/55     0     0     0     0     0]
[59/55  9/11     0     0     0     0]
[15/11 59/55 49/55     0     0     0]
```

The same effective resistance matrix, this time including adjacent vertices

```
sage: G.effective_resistance_matrix(nonedgesonly=False)                 #␣
↪needs sage.modules
[    0 34/55 34/55 49/55 59/55 15/11]
[34/55     0 26/55 31/55  9/11 59/55]
[34/55 26/55     0  5/11 31/55 49/55]
[49/55 31/55  5/11     0 26/55 34/55]
[59/55  9/11 31/55 26/55     0 34/55]
[15/11 59/55 49/55 34/55 34/55     0]
```

```
>>> from sage.all import *
>>> G.effective_resistance_matrix(nonedgesonly=False)                   #␣
↪needs sage.modules
[    0 34/55 34/55 49/55 59/55 15/11]
[34/55     0 26/55 31/55  9/11 59/55]
[34/55 26/55     0  5/11 31/55 49/55]
[49/55 31/55  5/11     0 26/55 34/55]
[59/55  9/11 31/55 26/55     0 34/55]
[15/11 59/55 49/55 34/55 34/55     0]
```

This example illustrates the common neighbors matrix for a fan on 6 vertices counting only non-adjacent vertex pairs

```
sage: H = Graph([(0,1),(0,2),(0,3),(0,4),(0,5),(0,6),(1,2),(2,3),(3,4),(4,5)])
sage: H.effective_resistance_matrix()                                   #␣
↪needs sage.modules
[    0     0     0     0     0     0     0]
[    0     0     0 49/55 56/55   6/5 89/55]
[    0     0     0     0   4/5 56/55 81/55]
[    0 49/55     0     0     0 49/55 16/11]
[    0 56/55   4/5     0     0     0 81/55]
[    0   6/5 56/55 49/55     0     0 89/55]
[    0 89/55 81/55 16/11 81/55 89/55     0]
```

```
>>> from sage.all import *
>>> H = Graph([(Integer(0),Integer(1)),(Integer(0),Integer(2)),(Integer(0),
↪Integer(3)),(Integer(0),Integer(4)),(Integer(0),Integer(5)),(Integer(0),
↪Integer(6)),(Integer(1),Integer(2)),(Integer(2),Integer(3)),(Integer(3),
```

(continues on next page)

(continued from previous page)

```
↪Integer(4)),(Integer(4),Integer(5))])
>>> H.effective_resistance_matrix()                                    #␣
↪needs sage.modules
[     0      0      0      0      0      0      0]
[     0      0      0 49/55 56/55   6/5 89/55]
[     0      0      0      0   4/5 56/55 81/55]
[     0 49/55      0      0      0 49/55 16/11]
[     0 56/55   4/5      0      0      0 81/55]
[     0   6/5 56/55 49/55      0      0 89/55]
[     0 89/55 81/55 16/11 81/55 89/55      0]
```

A different base ring:

```
sage: H.effective_resistance_matrix(base_ring=RDF)[0, 0].parent()          #␣
↪needs numpy sage.modules
Real Double Field
```

```
>>> from sage.all import *
>>> H.effective_resistance_matrix(base_ring=RDF)[Integer(0), Integer(0)].
↪parent()           # needs numpy sage.modules
Real Double Field
```

See also

- *least_effective_resistance()* – gives node pairs with least effective resistances
- *effective_resistance()* – computes effective resistance for a single node pair
- See Wikipedia article Resistance_Distance for more details.

eulerian_orientation(G)

Return an Eulerian orientation of the graph G.

An Eulerian graph being a graph such that any vertex has an even degree, an Eulerian orientation of a graph is an orientation of its edges such that each vertex v verifies $d^+(v) = d^-(v) = d(v)/2$, where d^+ and d^- respectively represent the out-degree and the in-degree of a vertex.

If the graph is not Eulerian, the orientation verifies for any vertex v that $|d^+(v) - d^-(v)| \leq 1$.

INPUT:

- `G` – an undirected graph

ALGORITHM:

This algorithm is a random walk through the edges of the graph, which orients the edges according to the walk. When a vertex is reached which has no non-oriented edge (this vertex must have odd degree), the walk resumes at another vertex of odd degree, if any.

This algorithm has time complexity in $O(n + m)$ for `SparseGraph` and $O(n^2)$ for `DenseGraph`, where m is the number of edges in the graph and n is the number of vertices in the graph.

EXAMPLES:

The CubeGraph with parameter 4, which is regular of even degree, has an Eulerian orientation such that $d^+ = d^-$:

```
sage: g = graphs.CubeGraph(4)
sage: g.degree()
[4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4]
sage: o = g.eulerian_orientation()
sage: o.in_degree()
[2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2]
sage: o.out_degree()
[2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2]
```

```
>>> from sage.all import *
>>> g = graphs.CubeGraph(Integer(4))
>>> g.degree()
[4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4]
>>> o = g.eulerian_orientation()
>>> o.in_degree()
[2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2]
>>> o.out_degree()
[2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2]
```

Secondly, the Petersen Graph, which is 3 regular has an orientation such that the difference between d^+ and d^- is at most 1:

```
sage: g = graphs.PetersenGraph()
sage: o = g.eulerian_orientation()
sage: o.in_degree()
[2, 2, 2, 2, 2, 1, 1, 1, 1, 1]
sage: o.out_degree()
[1, 1, 1, 1, 1, 2, 2, 2, 2, 2]
```

```
>>> from sage.all import *
>>> g = graphs.PetersenGraph()
>>> o = g.eulerian_orientation()
>>> o.in_degree()
[2, 2, 2, 2, 2, 1, 1, 1, 1, 1]
>>> o.out_degree()
[1, 1, 1, 1, 1, 2, 2, 2, 2, 2]
```

folded_graph(*check=False*, *immutable=None*)

Return the antipodal fold of this graph.

Given an antipodal graph G let G_d be its distance-d graph. Then the folded graph of G has a vertex for each maximal clique of G_d and two cliques are adjacent if there is an edge in G connecting the two.

See also

is_antipodal()

INPUT:

- `check` – boolean (default: `False`); whether to check if the graph is antipodal. If `check` is `True` and the graph is not antipodal, then return `False`.
- `immutable` – boolean (default: `None`); whether to create a mutable/immutable graph. `immutable=None` (default) means that the graph and its folded graph will behave the same way.

OUTPUT: this function returns a new graph and `self` is not touched

> **Note**
>
> The input is expected to be an antipodal graph. You can check that a graph is antipodal using *is_antipodal()*.

EXAMPLES:

```
sage: G = graphs.JohnsonGraph(10, 5)
sage: H = G.folded_graph(); H
Folded Johnson graph with parameters 10,5: Graph on 126 vertices
sage: Gd = G.distance_graph(G.diameter())
sage: all(i == 1 for i in Gd.degree())
True
sage: H.is_distance_regular(True)
([25, 16, None], [None, 1, 4])
```

```
>>> from sage.all import *
>>> G = graphs.JohnsonGraph(Integer(10), Integer(5))
>>> H = G.folded_graph(); H
Folded Johnson graph with parameters 10,5: Graph on 126 vertices
>>> Gd = G.distance_graph(G.diameter())
>>> all(i == Integer(1) for i in Gd.degree())
True
>>> H.is_distance_regular(True)
([25, 16, None], [None, 1, 4])
```

This method doesn't check if the graph is antipodal:

```
sage: G = graphs.PetersenGraph()
sage: G.is_antipodal()
False
sage: G.folded_graph()  # some garbage
Folded Petersen graph: Graph on 2 vertices
sage: G.folded_graph(check=True)
False
```

```
>>> from sage.all import *
>>> G = graphs.PetersenGraph()
>>> G.is_antipodal()
False
>>> G.folded_graph()  # some garbage
Folded Petersen graph: Graph on 2 vertices
>>> G.folded_graph(check=True)
False
```

REFERENCES:

See [BCN1989] p. 438 or [Sam2012] for this definition of folded graph.

fractional_chromatic_index(*G*, *solver='PPL'*, *verbose_constraints=False*, *verbose=0*)

Return the fractional chromatic index of the graph.

The fractional chromatic index is a relaxed version of edge-coloring. An edge coloring of a graph being actually a covering of its edges into the smallest possible number of matchings, the fractional chromatic index of a graph G is the smallest real value $\chi_f(G)$ such that there exists a list of matchings $M_1, \ldots, M_k$ of G and coefficients $\alpha_1, \ldots, \alpha_k$ with the property that each edge is covered by the matchings in the following relaxed way

$$\forall e \in E(G), \sum_{e \in M_i} \alpha_i \geq 1.$$

For more information, see the Wikipedia article Fractional_coloring.

ALGORITHM:

The fractional chromatic index is computed through Linear Programming through its dual. The LP solved by sage is actually:

$$\text{Maximize : } \sum_{e \in E(G)} r_e$$

$$\text{Such that : }$$

$$\forall M \text{ matching } \subseteq G, \sum_{e \in M} r_v \leq 1$$

INPUT:

- `G` – a graph
- `solver` – (default: `'PPL'`) specify a Linear Program (LP) solver to be used. If set to `None`, the default one is used. For more information on LP solvers and which default solver is used, see the method `solve` of the class `MixedIntegerLinearProgram`.

  Note

  The default solver used here is `'PPL'` which provides exact results, i.e. a rational number, although this may be slower that using other solvers. Be aware that this method may loop endlessly when using some non exact solvers as reported in Issue #23658 and Issue #23798.

- `verbose_constraints` – boolean (default: `False`); whether to display which constraints are being generated
- `verbose` – integer (default: 0); sets the level of verbosity of the LP solver

EXAMPLES:

The fractional chromatic index of a C_5 is $5/2$:

```
sage: g = graphs.CycleGraph(5)
sage: g.fractional_chromatic_index()                                        ␣
↪  # needs sage.numerical.mip
5/2
```

```
>>> from sage.all import *
>>> g = graphs.CycleGraph(Integer(5))
>>> g.fractional_chromatic_index()
↪# needs sage.numerical.mip
5/2
```

fractional_chromatic_number(*G*, *solver='PPL'*, *verbose=0*, *check_components=True*, *check_bipartite=True*)

Return the fractional chromatic number of the graph.

Fractional coloring is a relaxed version of vertex coloring with several equivalent definitions, such as the optimum value in a linear relaxation of the integer program that gives the usual chromatic number. It is also equal to the fractional clique number by LP-duality.

ALGORITHM:

The fractional chromatic number is computed via the usual Linear Program. The LP solved by sage is essentially,

$$\text{Minimize : } \sum_{I \in \mathcal{I}(G)} x_I$$

$$\text{Such that : }$$

$$\forall v \in V(G), \sum_{I \in \mathcal{I}(G),\, v \in I} x_v \geq 1$$

$$\forall I \in \mathcal{I}(G), x_I \geq 0$$

where $\mathcal{I}(G)$ is the set of maximal independent sets of G (see Section 2.1 of [CFKPR2010] to know why it is sufficient to consider maximal independent sets). As optional optimisations, we construct the LP on each biconnected component of G (and output the maximum value), and avoid using the LP if G is bipartite (as then the output must be 1 or 2).

Note

Computing the fractional chromatic number can be very slow. Since the variables of the LP are independent sets, in general the LP has size exponential in the order of the graph. In the current implementation a list of all maximal independent sets is created and stored, which can be both slow and memory-hungry.

INPUT:

- `G` – a graph
- `solver` – (default: `'PPL'`) specify a Linear Program (LP) solver to be used. If set to `None`, the default one is used. For more information on LP solvers and which default solver is used, see the method `solve` of the class `MixedIntegerLinearProgram`.

  Note

  The default solver used here is `'PPL'` which provides exact results, i.e. a rational number, although this may be slower that using other solvers.

- `verbose` – integer (default: 0); sets the level of verbosity of the LP solver
- `check_components` – boolean (default: `True`); whether the method is called on each biconnected component of G
- `check_bipartite` – boolean (default: `True`); whether the graph is checked for bipartiteness. If the graph is bipartite then we can avoid creating and solving the LP.

EXAMPLES:

The fractional chromatic number of a C_5 is $5/2$:

```
sage: g = graphs.CycleGraph(5)
sage: g.fractional_chromatic_number()
↪  # needs sage.numerical.mip
5/2
```

```
>>> from sage.all import *
>>> g = graphs.CycleGraph(Integer(5))
>>> g.fractional_chromatic_number()
↪# needs sage.numerical.mip
5/2
```

fractional_clique_number(*solver='PPL'*, *verbose=0*, *check_components=True*, *check_bipartite=True*)

Return the fractional clique number of the graph.

A fractional clique is a nonnegative weight function on the vertices of a graph such that the sum of the weights over any independent set is at most 1. The fractional clique number is the largest total weight of a fractional clique, which is equal to the fractional chromatic number by LP-duality.

ALGORITHM:

The fractional clique number is computed via the Linear Program for fractional chromatic number, see *fractional_chromatic_number*

INPUT:

- `solver` – (default: `'PPL'`) specify a Linear Program (LP) solver to be used. If set to `None`, the default one is used. For more information on LP solvers and which default solver is used, see the method `solve` of the class `MixedIntegerLinearProgram`.

  Note

  The default solver used here is `'PPL'` which provides exact results, i.e. a rational number, although this may be slower that using other solvers.

- `verbose` – integer (default: 0); sets the level of verbosity of the LP solver
- `check_components` – boolean (default: `True`); whether the method is called on each biconnected component of G
- `check_bipartite` – boolean (default: `True`); whether the graph is checked for bipartiteness. If the graph is bipartite then we can avoid creating and solving the LP.

EXAMPLES:

The fractional clique number of a C_7 is $7/3$:

```
sage: g = graphs.CycleGraph(7)
sage: g.fractional_clique_number()                                  #␣
↪needs sage.numerical.mip
7/3
```

```
>>> from sage.all import *
>>> g = graphs.CycleGraph(Integer(7))
>>> g.fractional_clique_number()                                    #␣
↪needs sage.numerical.mip
7/3
```

geodetic_closure(G, S)

Return the geodetic closure of the set of vertices S in G.

The geodetic closure $g(S)$ of a subset of vertices S of a graph G is in [HLT1993] as the set of all vertices that lie on a shortest $u-v$ path for any pair of vertices $u, v \in S$. We assume that $g(\emptyset) = \emptyset$ and that $g(\{u\}) = \{u\}$ for any u in G.

Warning

This operation is **not** a closure function. Indeed, a closure function must satisfy the property that $f(f(X))$ should be equal to $f(X)$, which is not always the case here. The term `closure` is used here to follow the terminology of the domain. See for instance [HLT1993].

Here, we implement a simple algorithm to determine this set. Roughly, for each vertex $u \in S$, the algorithm first performs a breadth first search from u to get distances, and then identifies the vertices of G lying on a shortest path from u to any $v \in S$ using a reversal traversal from vertices in S. This algorithm has time complexity in $O(|S|(n+m))$ for `SparseGraph`, $O(|S|(n+m)+n^2)$ for `DenseGraph` and space complexity in $O(n+m)$.

INPUT:

- `G` – a Sage graph
- `S` – a subset of vertices of G

EXAMPLES:

The vertices of the Petersen graph can be obtained by a geodetic closure of four of its vertices:

```
sage: from sage.graphs.convexity_properties import geodetic_closure
sage: G = graphs.PetersenGraph()
sage: geodetic_closure(G, [0, 2, 8, 9])
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

```
>>> from sage.all import *
>>> from sage.graphs.convexity_properties import geodetic_closure
>>> G = graphs.PetersenGraph()
>>> geodetic_closure(G, [Integer(0), Integer(2), Integer(8), Integer(9)])
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

The vertices of a 2D grid can be obtained by a geodetic closure of two vertices:

```
sage: G = graphs.Grid2dGraph(4, 4)
sage: c = G.geodetic_closure([(0, 0), (3, 3)])
sage: len(c) == G.order()
True
```

```
>>> from sage.all import *
>>> G = graphs.Grid2dGraph(Integer(4), Integer(4))
>>> c = G.geodetic_closure([(Integer(0), Integer(0)), (Integer(3),␣
↪Integer(3))])
>>> len(c) == G.order()
True
```

If two vertices belong to different connected components of a graph, their geodetic closure is trivial:

```
sage: G = Graph([(0, 1), (2, 3)])
sage: geodetic_closure(G, [0, 2])
[0, 2]
```

```
>>> from sage.all import *
>>> G = Graph([(Integer(0), Integer(1)), (Integer(2), Integer(3))])
>>> geodetic_closure(G, [Integer(0), Integer(2)])
[0, 2]
```

The geodetic closure does not satisfy the closure function property that $f(f(X))$ should be equal to $f(X)$:

```
sage: G = graphs.DiamondGraph()
sage: G.subdivide_edge((1, 2), 1)
sage: geodetic_closure(G, [0, 3])
[0, 1, 2, 3]
sage: geodetic_closure(G, geodetic_closure(G, [0, 3]))
[0, 1, 2, 3, 4]
```

```
>>> from sage.all import *
>>> G = graphs.DiamondGraph()
>>> G.subdivide_edge((Integer(1), Integer(2)), Integer(1))
>>> geodetic_closure(G, [Integer(0), Integer(3)])
[0, 1, 2, 3]
>>> geodetic_closure(G, geodetic_closure(G, [Integer(0), Integer(3)]))
[0, 1, 2, 3, 4]
```

gomory_hu_tree(*algorithm*, *solver=None*, *verbose=None*, *immutable=0*, *integrality_tolerance=None*)

Return a Gomory-Hu tree of `self`.

Given a tree T with labeled edges representing capacities, it is very easy to determine the maximum flow between any pair of vertices : it is the minimal label on the edges of the unique path between them.

Given a graph G, a Gomory-Hu tree T of G is a tree with the same set of vertices, and such that the maximum flow between any two vertices is the same in G as in T. See the Wikipedia article Gomory–Hu_tree. Note that, in general, a graph admits more than one Gomory-Hu tree.

See also 15.4 (Gomory-Hu trees) from [Sch2003].

INPUT:

- `algorithm` – select the algorithm used by *`edge_cut()`* method. Refer to its documentation for allowed values and default behaviour.
- `solver` – string (default: `None`); specifies a Mixed Integer Linear Programming (MILP) solver to be used. If set to `None`, the default one is used. For more information on MILP solvers and which default solver is used, see the method `solve` of the class `MixedIntegerLinearProgram`.

  Only useful when `algorithm == "LP"`.
- `verbose` – integer (default: 0); sets the level of verbosity. Set to 0 by default, which means quiet.

  Only useful when `algorithm == "LP"`.
- `immutable` – boolean (default: `None`); whether to create a mutable/immutable graph. `immutable=None` (default) means that the graph and Gomory-Hu tree will behave the same way.
- `integrality_tolerance` – float; parameter for use with MILP solvers over an inexact base ring; see `MixedIntegerLinearProgram.get_values()`.

Only useful when `algorithm == "LP"`.

OUTPUT: a graph with labeled edges

EXAMPLES:

Taking the Petersen graph:

```
sage: g = graphs.PetersenGraph()
sage: t = g.gomory_hu_tree()
```

```
>>> from sage.all import *
>>> g = graphs.PetersenGraph()
>>> t = g.gomory_hu_tree()
```

Obviously, this graph is a tree:

```
sage: t.is_tree()
True
```

```
>>> from sage.all import *
>>> t.is_tree()
True
```

Note that if the original graph is not connected, then the Gomory-Hu tree is in fact a forest:

```
sage: (2*g).gomory_hu_tree().is_forest()
True
sage: (2*g).gomory_hu_tree().is_connected()
False
```

```
>>> from sage.all import *
>>> (Integer(2)*g).gomory_hu_tree().is_forest()
True
>>> (Integer(2)*g).gomory_hu_tree().is_connected()
False
```

On the other hand, such a tree has lost nothing of the initial graph connectedness:

```
sage: all(t.flow(u,v) == g.flow(u,v) for u,v in Subsets(g.
↪vertices(sort=False), 2))
True
```

```
>>> from sage.all import *
>>> all(t.flow(u,v) == g.flow(u,v) for u,v in Subsets(g.vertices(sort=False),␣
↪Integer(2)))
True
```

Just to make sure, we can check that the same is true for two vertices in a random graph:

```
sage: g = graphs.RandomGNP(20,.3)
sage: t = g.gomory_hu_tree()
sage: g.flow(0,1) == t.flow(0,1)
True
```

```
>>> from sage.all import *
>>> g = graphs.RandomGNP(Integer(20),RealNumber('.3'))
>>> t = g.gomory_hu_tree()
>>> g.flow(Integer(0),Integer(1)) == t.flow(Integer(0),Integer(1))
True
```

And also the min cut:

```
sage: g.edge_connectivity() == min(t.edge_labels()) or not g.is_connected()
True
```

```
>>> from sage.all import *
>>> g.edge_connectivity() == min(t.edge_labels()) or not g.is_connected()
True
```

If the graph has an edge whose removal increases the number of connected components, the flow between the parts separated by this edge is one:

```
sage: g = Graph()
sage: g.add_clique([1, 2, 3, 4])
sage: g.add_clique([5, 6, 7, 8])
sage: g.add_edge(1, 5)
sage: t = g.gomory_hu_tree()
sage: t.edge_label(1, 5)
1
sage: g.flow(randint(1, 4), randint(5, 8)) == t.edge_label(1, 5)
True
sage: for u, v in g.edge_iterator(labels=False):
....:     g.set_edge_label(u, v, 3)
sage: t = g.gomory_hu_tree()
sage: t.edge_label(1, 5)
3
sage: g.flow(randint(1, 4), randint(5, 8)) == t.edge_label(1, 5)
True
```

```
>>> from sage.all import *
>>> g = Graph()
>>> g.add_clique([Integer(1), Integer(2), Integer(3), Integer(4)])
>>> g.add_clique([Integer(5), Integer(6), Integer(7), Integer(8)])
>>> g.add_edge(Integer(1), Integer(5))
>>> t = g.gomory_hu_tree()
>>> t.edge_label(Integer(1), Integer(5))
1
>>> g.flow(randint(Integer(1), Integer(4)), randint(Integer(5), Integer(8)))␣
↪== t.edge_label(Integer(1), Integer(5))
True
>>> for u, v in g.edge_iterator(labels=False):
...     g.set_edge_label(u, v, Integer(3))
>>> t = g.gomory_hu_tree()
>>> t.edge_label(Integer(1), Integer(5))
3
>>> g.flow(randint(Integer(1), Integer(4)), randint(Integer(5), Integer(8)))␣
```

(continues on next page)

(continued from previous page)

```
↪== t.edge_label(Integer(1), Integer(5))
True
```

graph6_string()

Return the graph6 representation of the graph as an ASCII string.

This is only valid for simple (no loops, no multiple edges) graphs on at most $2^{18} - 1 = 262143$ vertices.

Note

As the graph6 format only handles graphs with vertex set $\{0, ..., n-1\}$, a *relabelled copy* will be encoded, if necessary.

See also

- *dig6_string()* – a similar string format for directed graphs

EXAMPLES:

```
sage: G = graphs.KrackhardtKiteGraph()
sage: G.graph6_string()
'IvUqwK@?G'
```

```
>>> from sage.all import *
>>> G = graphs.KrackhardtKiteGraph()
>>> G.graph6_string()
'IvUqwK@?G'
```

has_perfect_matching(*G*, *algorithm*, *solver='Edmonds'*, *verbose=None*, *integrality_tolerance=0*)

Return whether the graph has a perfect matching.

INPUT:

- `algorithm` – string (default: `'Edmonds'`)
  - `'Edmonds'` uses Edmonds' algorithm as implemented in NetworkX to find a matching of maximal cardinality, then check whether this cardinality is half the number of vertices of the graph.
  - `'LP_matching'` uses a Linear Program to find a matching of maximal cardinality, then check whether this cardinality is half the number of vertices of the graph.
  - `'LP'` uses a Linear Program formulation of the perfect matching problem: put a binary variable `b[e]` on each edge e, and for each vertex v, require that the sum of the values of the edges incident to v is 1.
- `solver` – string (default: `None`); specifies a Mixed Integer Linear Programming (MILP) solver to be used. If set to `None`, the default one is used. For more information on MILP solvers and which default solver is used, see the method `solve` of the class `MixedIntegerLinearProgram`.
- `verbose` – integer (default: 0); sets the level of verbosity: set to 0 by default, which means quiet (only useful when `algorithm == "LP_matching"` or `algorithm == "LP"`)
- `integrality_tolerance` – float; parameter for use with MILP solvers over an inexact base ring; see `MixedIntegerLinearProgram.get_values()`.

OUTPUT: boolean

EXAMPLES:

```
sage: graphs.PetersenGraph().has_perfect_matching()                          #␣
↪needs networkx
True
sage: graphs.WheelGraph(6).has_perfect_matching()                            #␣
↪needs networkx
True
sage: graphs.WheelGraph(5).has_perfect_matching()                            #␣
↪needs networkx
False
sage: graphs.PetersenGraph().has_perfect_matching(algorithm='LP_matching')   #␣
↪needs sage.numerical.mip
True
sage: graphs.WheelGraph(6).has_perfect_matching(algorithm='LP_matching')     #␣
↪needs sage.numerical.mip
True
sage: graphs.WheelGraph(5).has_perfect_matching(algorithm='LP_matching')
False
sage: graphs.PetersenGraph().has_perfect_matching(algorithm='LP_matching')   #␣
↪needs sage.numerical.mip
True
sage: graphs.WheelGraph(6).has_perfect_matching(algorithm='LP_matching')     #␣
↪needs sage.numerical.mip
True
sage: graphs.WheelGraph(5).has_perfect_matching(algorithm='LP_matching')
False
```

```
>>> from sage.all import *
>>> graphs.PetersenGraph().has_perfect_matching()                            #␣
↪needs networkx
True
>>> graphs.WheelGraph(Integer(6)).has_perfect_matching()                      ␣
↪     # needs networkx
True
>>> graphs.WheelGraph(Integer(5)).has_perfect_matching()                      ␣
↪     # needs networkx
False
>>> graphs.PetersenGraph().has_perfect_matching(algorithm='LP_matching')   #␣
↪needs sage.numerical.mip
True
>>> graphs.WheelGraph(Integer(6)).has_perfect_matching(algorithm='LP_matching
↪')     # needs sage.numerical.mip
True
>>> graphs.WheelGraph(Integer(5)).has_perfect_matching(algorithm='LP_matching
↪')
False
>>> graphs.PetersenGraph().has_perfect_matching(algorithm='LP_matching')   #␣
↪needs sage.numerical.mip
True
>>> graphs.WheelGraph(Integer(6)).has_perfect_matching(algorithm='LP_matching
↪')     # needs sage.numerical.mip
```

(continues on next page)

(continued from previous page)

```
True
>>> graphs.WheelGraph(Integer(5)).has_perfect_matching(algorithm='LP_matching
↪')
False
```

hyperbolicity(*G*, *algorithm='BCCM'*, *approximation_factor=None*, *additive_gap=None*, *verbose=False*)

Return the hyperbolicity of the graph or an approximation of this value.

The hyperbolicity of a graph has been defined by Gromov [Gro1987] as follows: Let a, b, c, d be vertices of the graph, let $S_1 = dist(a, b) + dist(b, c)$, $S_2 = dist(a, c) + dist(b, d)$, and $S_3 = dist(a, d) + dist(b, c)$, and let M_1 and M_2 be the two largest values among S_1, S_2, and S_3. We have $hyp(a, b, c, d) = |M_1 - M_2|$, and the hyperbolicity of the graph is the maximum over all possible 4-tuples (a, b, c, d) divided by 2. The worst case time complexity is in $O(n^4)$.

See the documentation of *sage.graphs.hyperbolicity* for more information.

INPUT:

- `G` – a connected Graph
- `algorithm` – (default: `'BCCM'`) specifies the algorithm to use among:
  - `'basic'` is an exhaustive algorithm considering all possible 4-tuples and so have time complexity in $O(n^4)$.
  - `'CCL'` is an exact algorithm proposed in [CCL2015]. It considers the 4-tuples in an ordering allowing to cut the search space as soon as a new lower bound is found (see the module's documentation). This algorithm can be turned into a approximation algorithm.
  - `'CCL+FA'` or `'CCL+'` uses the notion of far-apart pairs as proposed in [Sot2011] to significantly reduce the overall computation time of the `'CCL'` algorithm.
  - `'BCCM'` is an exact algorithm proposed in [BCCM2015]. It improves `'CCL+FA'` by cutting several 4-tuples (for more information, see the module's documentation).
  - `'dom'` is an approximation with additive constant four. It computes the hyperbolicity of the vertices of a dominating set of the graph. This is sometimes slower than `'CCL'` and sometimes faster. Try it to know if it is interesting for you. The `additive_gap` and `approximation_factor` parameters cannot be used in combination with this method and so are ignored.
- `approximation_factor` – (default: `None`) when the approximation factor is set to some value (larger than 1.0), the function stop computations as soon as the ratio between the upper bound and the best found solution is less than the approximation factor. When the approximation factor is 1.0, the problem is solved optimally. This parameter is used only when the chosen algorithm is `'CCL'`, `'CCL+FA'`, or `'BCCM'`.
- `additive_gap` – (default: `None`) when set to a positive number, the function stop computations as soon as the difference between the upper bound and the best found solution is less than additive gap. When the gap is 0.0, the problem is solved optimally. This parameter is used only when the chosen algorithm is `'CCL'` or `'CCL+FA'`, or `'BCCM'`.
- `verbose` – boolean (default: `False`); set to `True` to display some information during execution: new upper and lower bounds, etc.

OUTPUT:

This function returns the tuple (delta, certificate, delta_UB), where:

- `delta` – the hyperbolicity of the graph (half-integer value)

- `certificate` – is the list of the 4 vertices for which the maximum value has been computed, and so the hyperbolicity of the graph
- `delta_UB` – is an upper bound for `delta`. When `delta == delta_UB`, the returned solution is optimal. Otherwise, the approximation factor if `delta_UB/delta`.

EXAMPLES:

Hyperbolicity of a 3×3 grid:

```
sage: from sage.graphs.hyperbolicity import hyperbolicity
sage: G = graphs.Grid2dGraph(3, 3)
sage: L,C,U = hyperbolicity(G, algorithm='BCCM'); L,sorted(C),U
(2, [(0, 0), (0, 2), (2, 0), (2, 2)], 2)
sage: L,C,U = hyperbolicity(G, algorithm='CCL'); L,sorted(C),U
(2, [(0, 0), (0, 2), (2, 0), (2, 2)], 2)
sage: L,C,U = hyperbolicity(G, algorithm='basic'); L,sorted(C),U
(2, [(0, 0), (0, 2), (2, 0), (2, 2)], 2)
```

```
>>> from sage.all import *
>>> from sage.graphs.hyperbolicity import hyperbolicity
>>> G = graphs.Grid2dGraph(Integer(3), Integer(3))
>>> L,C,U = hyperbolicity(G, algorithm='BCCM'); L,sorted(C),U
(2, [(0, 0), (0, 2), (2, 0), (2, 2)], 2)
>>> L,C,U = hyperbolicity(G, algorithm='CCL'); L,sorted(C),U
(2, [(0, 0), (0, 2), (2, 0), (2, 2)], 2)
>>> L,C,U = hyperbolicity(G, algorithm='basic'); L,sorted(C),U
(2, [(0, 0), (0, 2), (2, 0), (2, 2)], 2)
```

Hyperbolicity of a PetersenGraph:

```
sage: from sage.graphs.hyperbolicity import hyperbolicity
sage: G = graphs.PetersenGraph()
sage: L,C,U = hyperbolicity(G, algorithm='BCCM'); L,sorted(C),U
(1/2, [6, 7, 8, 9], 1/2)
sage: L,C,U = hyperbolicity(G, algorithm='CCL'); L,sorted(C),U
(1/2, [0, 1, 2, 3], 1/2)
sage: L,C,U = hyperbolicity(G, algorithm='CCL+'); L,sorted(C),U
(1/2, [0, 1, 2, 3], 1/2)
sage: L,C,U = hyperbolicity(G, algorithm='CCL+FA'); L,sorted(C),U
(1/2, [0, 1, 2, 3], 1/2)
sage: L,C,U = hyperbolicity(G, algorithm='basic'); L,sorted(C),U
(1/2, [0, 1, 2, 3], 1/2)
sage: L,C,U = hyperbolicity(G, algorithm='dom'); L,U
(0, 1)
sage: sorted(C)  # random
[0, 1, 2, 6]
```

```
>>> from sage.all import *
>>> from sage.graphs.hyperbolicity import hyperbolicity
>>> G = graphs.PetersenGraph()
>>> L,C,U = hyperbolicity(G, algorithm='BCCM'); L,sorted(C),U
(1/2, [6, 7, 8, 9], 1/2)
>>> L,C,U = hyperbolicity(G, algorithm='CCL'); L,sorted(C),U
(1/2, [0, 1, 2, 3], 1/2)
```

(continues on next page)

(continued from previous page)

```
>>> L,C,U = hyperbolicity(G, algorithm='CCL+'); L,sorted(C),U
(1/2, [0, 1, 2, 3], 1/2)
>>> L,C,U = hyperbolicity(G, algorithm='CCL+FA'); L,sorted(C),U
(1/2, [0, 1, 2, 3], 1/2)
>>> L,C,U = hyperbolicity(G, algorithm='basic'); L,sorted(C),U
(1/2, [0, 1, 2, 3], 1/2)
>>> L,C,U = hyperbolicity(G, algorithm='dom'); L,U
(0, 1)
>>> sorted(C)  # random
[0, 1, 2, 6]
```

Asking for an approximation in a grid graph:

```
sage: from sage.graphs.hyperbolicity import hyperbolicity
sage: G = graphs.Grid2dGraph(2, 10)
sage: L,C,U = hyperbolicity(G, algorithm='CCL', approximation_factor=1.5); L,U
(1, 3/2)
sage: L,C,U = hyperbolicity(G, algorithm='CCL+', approximation_factor=1.5); L,
↪U
(1, 1)
sage: L,C,U = hyperbolicity(G, algorithm='CCL', approximation_factor=4); L,U
(1, 4)
sage: L,C,U = hyperbolicity(G, algorithm='CCL', additive_gap=2); L,U
(1, 3)
sage: L,C,U = hyperbolicity(G, algorithm='dom'); L,U
(1, 5)
```

```
>>> from sage.all import *
>>> from sage.graphs.hyperbolicity import hyperbolicity
>>> G = graphs.Grid2dGraph(Integer(2), Integer(10))
>>> L,C,U = hyperbolicity(G, algorithm='CCL', approximation_factor=RealNumber(
↪'1.5')); L,U
(1, 3/2)
>>> L,C,U = hyperbolicity(G, algorithm='CCL+', approximation_
↪factor=RealNumber('1.5')); L,U
(1, 1)
>>> L,C,U = hyperbolicity(G, algorithm='CCL', approximation_
↪factor=Integer(4)); L,U
(1, 4)
>>> L,C,U = hyperbolicity(G, algorithm='CCL', additive_gap=Integer(2)); L,U
(1, 3)
>>> L,C,U = hyperbolicity(G, algorithm='dom'); L,U
(1, 5)
```

Asking for an approximation in a cycle graph:

```
sage: from sage.graphs.hyperbolicity import hyperbolicity
sage: G = graphs.CycleGraph(10)
sage: L,C,U = hyperbolicity(G, algorithm='CCL', approximation_factor=1.5); L,U
(2, 5/2)
sage: L,C,U = hyperbolicity(G, algorithm='CCL+FA', approximation_factor=1.5);␣
↪L,U
```

(continues on next page)

(continued from previous page)

```
(2, 5/2)
sage: L,C,U = hyperbolicity(G, algorithm='CCL+FA', additive_gap=1); L,U
(2, 5/2)
```

```
>>> from sage.all import *
>>> from sage.graphs.hyperbolicity import hyperbolicity
>>> G = graphs.CycleGraph(Integer(10))
>>> L,C,U = hyperbolicity(G, algorithm='CCL', approximation_factor=RealNumber(
↪'1.5')); L,U
(2, 5/2)
>>> L,C,U = hyperbolicity(G, algorithm='CCL+FA', approximation_
↪factor=RealNumber('1.5')); L,U
(2, 5/2)
>>> L,C,U = hyperbolicity(G, algorithm='CCL+FA', additive_gap=Integer(1)); L,U
(2, 5/2)
```

Comparison of results:

```
sage: from sage.graphs.hyperbolicity import hyperbolicity
sage: for i in range(10):                # long time
↪  # needs networkx
....:     G = graphs.RandomBarabasiAlbert(100,2)
....:     d1,_,_ = hyperbolicity(G, algorithm='basic')
....:     d2,_,_ = hyperbolicity(G, algorithm='CCL')
....:     d3,_,_ = hyperbolicity(G, algorithm='CCL+')
....:     d4,_,_ = hyperbolicity(G, algorithm='CCL+FA')
....:     d5,_,_ = hyperbolicity(G, algorithm='BCCM')
....:     l3,_,u3 = hyperbolicity(G, approximation_factor=2)
....:     if (not d1==d2==d3==d4==d5) or l3>d1 or u3<d1:
....:        print("That's not good!")

sage: from sage.graphs.hyperbolicity import hyperbolicity
sage: import random
sage: random.seed()
sage: for i in range(10):                # long time
↪  # needs networkx
....:     n = random.randint(2, 20)
....:     m = random.randint(0, n*(n-1) / 2)
....:     G = graphs.RandomGNM(n, m)
....:     for cc in G.connected_components_subgraphs():
....:         d1,_,_ = hyperbolicity(cc, algorithm='basic')
....:         d2,_,_ = hyperbolicity(cc, algorithm='CCL')
....:         d3,_,_ = hyperbolicity(cc, algorithm='CCL+')
....:         d4,_,_ = hyperbolicity(cc, algorithm='CCL+FA')
....:         d5,_,_ = hyperbolicity(cc, algorithm='BCCM')
....:         l3,_,u3 = hyperbolicity(cc, approximation_factor=2)
....:         if (not d1==d2==d3==d4==d5) or l3>d1 or u3<d1:
....:             print("Error in graph ", cc.edges(sort=True))
```

```
>>> from sage.all import *
>>> from sage.graphs.hyperbolicity import hyperbolicity
>>> for i in range(Integer(10)):                # long time
```

(continues on next page)

(continued from previous page)

```
↪          # needs networkx
...      G = graphs.RandomBarabasiAlbert(Integer(100),Integer(2))
...      d1,_,_ = hyperbolicity(G, algorithm='basic')
...      d2,_,_ = hyperbolicity(G, algorithm='CCL')
...      d3,_,_ = hyperbolicity(G, algorithm='CCL+')
...      d4,_,_ = hyperbolicity(G, algorithm='CCL+FA')
...      d5,_,_ = hyperbolicity(G, algorithm='BCCM')
...      l3,_,u3 = hyperbolicity(G, approximation_factor=Integer(2))
...      if (not d1==d2==d3==d4==d5) or l3>d1 or u3<d1:
...         print("That's not good!")

>>> from sage.graphs.hyperbolicity import hyperbolicity
>>> import random
>>> random.seed()
>>> for i in range(Integer(10)):                # long time                    ␣
↪          # needs networkx
...      n = random.randint(Integer(2), Integer(20))
...      m = random.randint(Integer(0), n*(n-Integer(1)) / Integer(2))
...      G = graphs.RandomGNM(n, m)
...      for cc in G.connected_components_subgraphs():
...          d1,_,_ = hyperbolicity(cc, algorithm='basic')
...          d2,_,_ = hyperbolicity(cc, algorithm='CCL')
...          d3,_,_ = hyperbolicity(cc, algorithm='CCL+')
...          d4,_,_ = hyperbolicity(cc, algorithm='CCL+FA')
...          d5,_,_ = hyperbolicity(cc, algorithm='BCCM')
...          l3,_,u3 = hyperbolicity(cc, approximation_factor=Integer(2))
...          if (not d1==d2==d3==d4==d5) or l3>d1 or u3<d1:
...              print("Error in graph ", cc.edges(sort=True))
```

The hyperbolicity of a graph is the maximum value over all its biconnected components:

```
sage: from sage.graphs.hyperbolicity import hyperbolicity
sage: G = graphs.PetersenGraph() * 2
sage: G.add_edge(0, 11)
sage: L,C,U = hyperbolicity(G); L,sorted(C),U
(1/2, [6, 7, 8, 9], 1/2)
```

```
>>> from sage.all import *
>>> from sage.graphs.hyperbolicity import hyperbolicity
>>> G = graphs.PetersenGraph() * Integer(2)
>>> G.add_edge(Integer(0), Integer(11))
>>> L,C,U = hyperbolicity(G); L,sorted(C),U
(1/2, [6, 7, 8, 9], 1/2)
```

ihara_zeta_function_inverse()

Compute the inverse of the Ihara zeta function of the graph.

This is a polynomial in one variable with integer coefficients. The Ihara zeta function itself is the inverse of this polynomial.

See the Wikipedia article Ihara zeta function for more information.

ALGORITHM:

This is computed here as the (reversed) characteristic polynomial of a square matrix of size twice the number of edges, related to the adjacency matrix of the line graph, see for example Proposition 9 in [SS2008] and Def. 4.1 in [Ter2011].

The graph is first replaced by its 2-core, as this does not change the Ihara zeta function.

EXAMPLES:

```
sage: G = graphs.CompleteGraph(4)
sage: factor(G.ihara_zeta_function_inverse())                                  #␣
↪needs sage.libs.pari sage.modules
(2*t - 1) * (t + 1)^2 * (t - 1)^3 * (2*t^2 + t + 1)^3

sage: G = graphs.CompleteGraph(5)
sage: factor(G.ihara_zeta_function_inverse())                                  #␣
↪needs sage.libs.pari sage.modules
(-1) * (3*t - 1) * (t + 1)^5 * (t - 1)^6 * (3*t^2 + t + 1)^4

sage: G = graphs.PetersenGraph()
sage: factor(G.ihara_zeta_function_inverse())                                  #␣
↪needs sage.libs.pari sage.modules
(-1) * (2*t - 1) * (t + 1)^5 * (t - 1)^6 * (2*t^2 + 2*t + 1)^4
* (2*t^2 - t + 1)^5

sage: G = graphs.RandomTree(10)
sage: G.ihara_zeta_function_inverse()                                          #␣
↪needs sage.libs.pari sage.modules
1
```

```
>>> from sage.all import *
>>> G = graphs.CompleteGraph(Integer(4))
>>> factor(G.ihara_zeta_function_inverse())                                    #␣
↪needs sage.libs.pari sage.modules
(2*t - 1) * (t + 1)^2 * (t - 1)^3 * (2*t^2 + t + 1)^3

>>> G = graphs.CompleteGraph(Integer(5))
>>> factor(G.ihara_zeta_function_inverse())                                    #␣
↪needs sage.libs.pari sage.modules
(-1) * (3*t - 1) * (t + 1)^5 * (t - 1)^6 * (3*t^2 + t + 1)^4

>>> G = graphs.PetersenGraph()
>>> factor(G.ihara_zeta_function_inverse())                                    #␣
↪needs sage.libs.pari sage.modules
(-1) * (2*t - 1) * (t + 1)^5 * (t - 1)^6 * (2*t^2 + 2*t + 1)^4
* (2*t^2 - t + 1)^5

>>> G = graphs.RandomTree(Integer(10))
>>> G.ihara_zeta_function_inverse()                                            #␣
↪needs sage.libs.pari sage.modules
1
```

REFERENCES:

[HST2001]

independent_set(*algorithm*, *value_only='Cliquer'*, *reduction_rules=False*, *solver=True*, *verbose=None*, *integrality_tolerance=0*)

Return a maximum independent set.

An independent set of a graph is a set of pairwise non-adjacent vertices. A maximum independent set is an independent set of maximum cardinality. It induces an empty subgraph.

Equivalently, an independent set is defined as the complement of a vertex cover.

For more information, see the Wikipedia article Independent_set_(graph_theory) and the Wikipedia article Vertex_cover.

INPUT:

- `algorithm` – the algorithm to be used
  - If `algorithm = "Cliquer"` (default), the problem is solved using Cliquer [NO2003].

    (see the *Cliquer modules*)
  - If `algorithm = "MILP"`, the problem is solved through a Mixed Integer Linear Program.

    (see `MixedIntegerLinearProgram`)
- If `algorithm = "mcqd"`, uses the MCQD solver (http://www.sicmm.org/~konc/maxclique/). Note that the MCQD package must be installed.
- `value_only` – boolean (default: `False`); if set to `True`, only the size of a maximum independent set is returned. Otherwise, a maximum independent set is returned as a list of vertices.
- `reduction_rules` – (default: `True`) specify if the reductions rules from kernelization must be applied as pre-processing or not. See [ACFLSS04] for more details. Note that depending on the instance, it might be faster to disable reduction rules.
- `solver` – string (default: `None`); specifies a Mixed Integer Linear Programming (MILP) solver to be used. If set to `None`, the default one is used. For more information on MILP solvers and which default solver is used, see the method `solve` of the class `MixedIntegerLinearProgram`.
- `verbose` – integer (default: 0); sets the level of verbosity. Set to 0 by default, which means quiet.
- `integrality_tolerance` – float; parameter for use with MILP solvers over an inexact base ring; see `MixedIntegerLinearProgram.get_values()`.

Note

While Cliquer/MCAD are usually (and by far) the most efficient implementations, the MILP formulation sometimes proves faster on very "symmetrical" graphs.

EXAMPLES:

Using Cliquer:

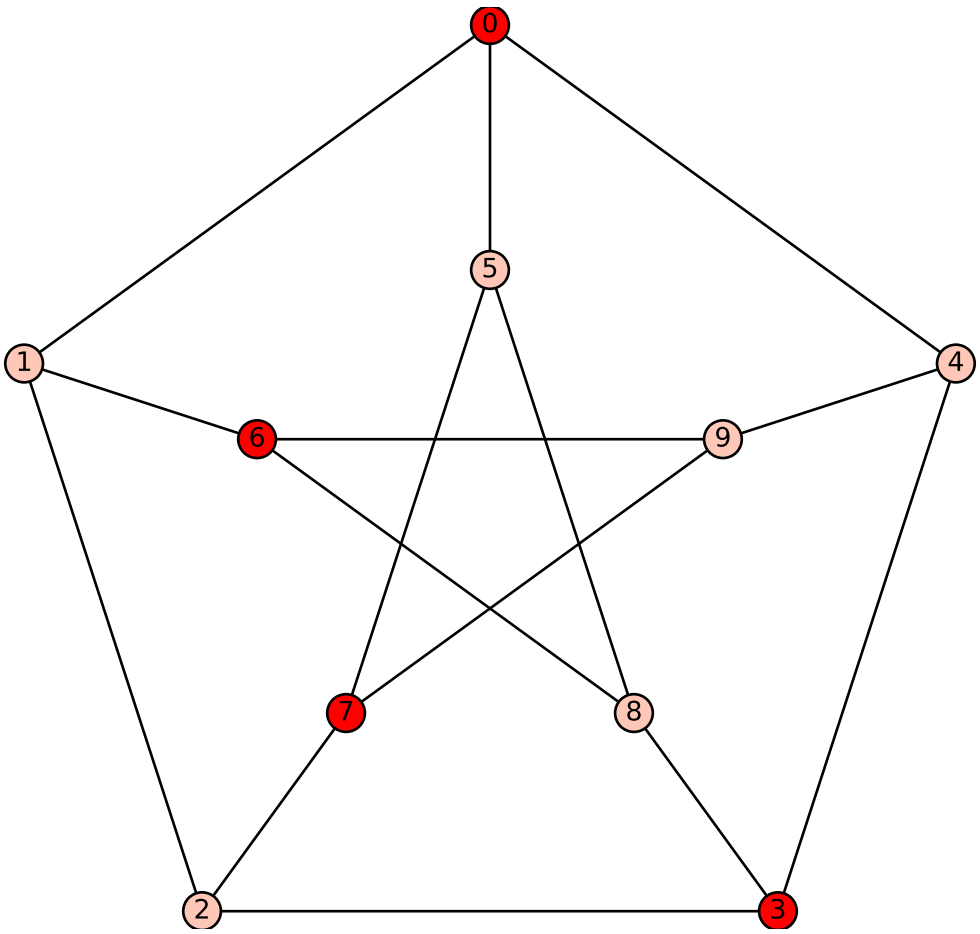
```
sage: C = graphs.PetersenGraph()
sage: C.independent_set()
[0, 3, 6, 7]
```

```
>>> from sage.all import *
>>> C = graphs.PetersenGraph()
>>> C.independent_set()
[0, 3, 6, 7]
```

As a linear program:

```
sage: C = graphs.PetersenGraph()
sage: len(C.independent_set(algorithm='MILP'))                                  #␣
↪needs sage.numerical.mip
4
```

```
>>> from sage.all import *
>>> C = graphs.PetersenGraph()
>>> len(C.independent_set(algorithm='MILP'))                                    #␣
↪needs sage.numerical.mip
4
```



independent_set_of_representatives(*family*, *solver*, *verbose=None*, *integrality_tolerance=0*)

Return an independent set of representatives.

Given a graph G and a family $F = \{F_i : i \in [1, ..., k]\}$ of subsets of `g.vertices(sort=False)`, an Independent Set of Representatives (ISR) is an assignation of a vertex $v_i \in F_i$ to each set F_i such that $v_i! = v_j$ if $i < j$ (they are representatives) and the set $\cup_i v_i$ is an independent set in G.

It generalizes, for example, graph coloring and graph list coloring.

(See [ABZ2007] for more information.)

INPUT:

- `family` – list of lists defining the family F (actually, a Family of subsets of `G.vertices(sort=False)`)
- `solver` – string (default: `None`); specifies a Mixed Integer Linear Programming (MILP) solver to be used. If set to `None`, the default one is used. For more information on MILP solvers and which default solver is used, see the method `solve` of the class `MixedIntegerLinearProgram`.
- `verbose` – integer (default: 0); sets the level of verbosity. Set to 0 by default, which means quiet.
- `integrality_tolerance` – float; parameter for use with MILP solvers over an inexact base ring; see `MixedIntegerLinearProgram.get_values()`.

OUTPUT:

- A list whose i^{th} element is the representative of the ith element of the `family` list. If there is no ISR, `None` is returned.

EXAMPLES:

For a bipartite graph missing one edge, the solution is as expected:

```
sage: g = graphs.CompleteBipartiteGraph(3,3)
sage: g.delete_edge(1,4)
sage: g.independent_set_of_representatives([[0,1,2],[3,4,5]])
↪# needs sage.numerical.mip
[1, 4]
```

```
>>> from sage.all import *
>>> g = graphs.CompleteBipartiteGraph(Integer(3),Integer(3))
>>> g.delete_edge(Integer(1),Integer(4))
>>> g.independent_set_of_representatives([[Integer(0),Integer(1),Integer(2)],
↪[Integer(3),Integer(4),Integer(5)]])                # needs sage.numerical.
↪mip
[1, 4]
```

The Petersen Graph is 3-colorable, which can be expressed as an independent set of representatives problem : take 3 disjoint copies of the Petersen Graph, each one representing one color. Then take as a partition of the set of vertices the family defined by the three copies of each vertex. The ISR of such a family defines a 3-coloring:

```
sage: g = 3 * graphs.PetersenGraph()
sage: n = g.order() / 3
sage: f = [[i, i + n, i + 2*n] for i in range(n)]
sage: isr = g.independent_set_of_representatives(f)
sage: c = [integer_floor(i / n) for i in isr]
sage: color_classes = [[], [], []]
sage: for v, i in enumerate(c):
....:   color_classes[i].append(v)
sage: for classs in color_classes:
....:   g.subgraph(classs).size() == 0
True
True
True
```

```
>>> from sage.all import *
>>> g = Integer(3) * graphs.PetersenGraph()
```

(continues on next page)

(continued from previous page)

```
>>> n = g.order() / Integer(3)
>>> f = [[i, i + n, i + Integer(2)*n] for i in range(n)]
>>> isr = g.independent_set_of_representatives(f)
>>> c = [integer_floor(i / n) for i in isr]
>>> color_classes = [[], [], []]
>>> for v, i in enumerate(c):
...   color_classes[i].append(v)
>>> for classs in color_classes:
...   g.subgraph(classs).size() == Integer(0)
True
True
True
```

is_antipodal()

Check whether this graph is antipodal.

A graph G of diameter d is said to be antipodal if its distance-d graph is a disjoint union of cliques.

EXAMPLES:

```
sage: G = graphs.JohnsonGraph(10, 5)
sage: G.is_antipodal()
True
sage: H = G.folded_graph()
sage: H.is_antipodal()
False
```

```
>>> from sage.all import *
>>> G = graphs.JohnsonGraph(Integer(10), Integer(5))
>>> G.is_antipodal()
True
>>> H = G.folded_graph()
>>> H.is_antipodal()
False
```

REFERENCES:

See [BCN1989] p. 438 or [Sam2012] for this definition of antipodal graphs.

is_apex()

Test if the graph is apex.

A graph is apex if it can be made planar by the removal of a single vertex. The deleted vertex is called an apex of the graph, and a graph may have more than one apex. For instance, in the minimal nonplanar graphs K_5 or $K_{3,3}$, every vertex is an apex. The apex graphs include graphs that are themselves planar, in which case again every vertex is an apex. The null graph is also counted as an apex graph even though it has no vertex to remove. If the graph is not connected, we say that it is apex if it has at most one non planar connected component and that this component is apex. See the Wikipedia article Apex_graph for more information.

See also

- *apex_vertices()*
- *is_planar()*

EXAMPLES:

K_5 and $K_{3,3}$ are apex graphs, and each of their vertices is an apex:

```
sage: G = graphs.CompleteGraph(5)
sage: G.is_apex()
True
sage: G = graphs.CompleteBipartiteGraph(3,3)
sage: G.is_apex()
True
```

```
>>> from sage.all import *
>>> G = graphs.CompleteGraph(Integer(5))
>>> G.is_apex()
True
>>> G = graphs.CompleteBipartiteGraph(Integer(3),Integer(3))
>>> G.is_apex()
True
```

The Petersen graph is not apex:

```
sage: G = graphs.PetersenGraph()
sage: G.is_apex()
False
```

```
>>> from sage.all import *
>>> G = graphs.PetersenGraph()
>>> G.is_apex()
False
```

A graph is apex if all its connected components are apex, but at most one is not planar:

```
sage: M = graphs.Grid2dGraph(3,3)
sage: K5 = graphs.CompleteGraph(5)
sage: (M+K5).is_apex()
True
sage: (M+K5+K5).is_apex()
False
```

```
>>> from sage.all import *
>>> M = graphs.Grid2dGraph(Integer(3),Integer(3))
>>> K5 = graphs.CompleteGraph(Integer(5))
>>> (M+K5).is_apex()
True
>>> (M+K5+K5).is_apex()
False
```

is_arc_transitive()

Check if `self` is an arc-transitive graph.

A graph is arc-transitive if its automorphism group acts transitively on its pairs of adjacent vertices.

Equivalently, if there exists for any pair of edges $uv, u'v' \in E(G)$ an automorphism ϕ_1 of G such that $\phi_1(u) = u'$ and $\phi_1(v) = v'$, as well as another automorphism ϕ_2 of G such that $\phi_2(u) = v'$ and $\phi_2(v) = u'$

See also

- Wikipedia article arc-transitive_graph
- *is_edge_transitive()*
- *is_half_transitive()*
- *is_semi_symmetric()*

EXAMPLES:

```
sage: P = graphs.PetersenGraph()
sage: P.is_arc_transitive()                                                 #␣
↪needs sage.libs.gap
True
sage: G = graphs.GrayGraph()
sage: G.is_arc_transitive()                                                 #␣
↪needs sage.libs.gap
False
```

```
>>> from sage.all import *
>>> P = graphs.PetersenGraph()
>>> P.is_arc_transitive()                                                   #␣
↪needs sage.libs.gap
True
>>> G = graphs.GrayGraph()
>>> G.is_arc_transitive()                                                   #␣
↪needs sage.libs.gap
False
```

is_asteroidal_triple_free(*G*, *certificate=False*)

Test if the input graph is asteroidal triple-free.

An independent set of three vertices such that each pair is joined by a path that avoids the neighborhood of the third one is called an *asteroidal triple*. A graph is asteroidal triple-free (AT-free) if it contains no asteroidal triples. See the *module's documentation* for more details.

This method returns `True` if the graph is AT-free and `False` otherwise.

INPUT:

- `G` – a Graph
- `certificate` – boolean (default: `False`); by default, this method returns `True` if the graph is asteroidal triple-free and `False` otherwise. When `certificate==True`, this method returns in addition a list of three vertices forming an asteroidal triple if such a triple is found, and the empty list otherwise.

EXAMPLES:

The complete graph is AT-free, as well as its line graph:

```
sage: G = graphs.CompleteGraph(5)
sage: G.is_asteroidal_triple_free()
True
sage: G.is_asteroidal_triple_free(certificate=True)
(True, [])
```

(continues on next page)

(continued from previous page)

```
sage: LG = G.line_graph()
sage: LG.is_asteroidal_triple_free()
True
sage: LLG = LG.line_graph()
sage: LLG.is_asteroidal_triple_free()
False
```

```
>>> from sage.all import *
>>> G = graphs.CompleteGraph(Integer(5))
>>> G.is_asteroidal_triple_free()
True
>>> G.is_asteroidal_triple_free(certificate=True)
(True, [])
>>> LG = G.line_graph()
>>> LG.is_asteroidal_triple_free()
True
>>> LLG = LG.line_graph()
>>> LLG.is_asteroidal_triple_free()
False
```

The PetersenGraph is not AT-free:

```
sage: from sage.graphs.asteroidal_triples import *
sage: G = graphs.PetersenGraph()
sage: G.is_asteroidal_triple_free()
False
sage: G.is_asteroidal_triple_free(certificate=True)
(False, [0, 2, 6])
```

```
>>> from sage.all import *
>>> from sage.graphs.asteroidal_triples import *
>>> G = graphs.PetersenGraph()
>>> G.is_asteroidal_triple_free()
False
>>> G.is_asteroidal_triple_free(certificate=True)
(False, [0, 2, 6])
```

is_bicritical(*G*, *matching*, *algorithm=None*, *coNP_certificate='Edmonds'*, *solver=False*, *verbose=None*, *integrality_tolerance=0*)

Check if the graph is bicritical.

A nontrivial graph G is *bicritical* if $G - u - v$ has a perfect matching for any two distinct vertices u and v of G. Bicritical graphs are special kind of matching covered graphs. Each maximal barrier of a bicritical graph is a singleton. Thus, for a bicritical graph, the canonical partition of the vertex set is the set of sets where each set is an individual vertex. Three-connected bicritical graphs, aka *bricks*, play an important role in the theory of matching covered graphs.

This method implements the algorithm proposed in [LZ2001] and we assume that a connected graph of order two is bicritical, whereas a disconnected graph of the same order is not. The time complexity of the algorithm is $\mathcal{O}(|V| \cdot |E|)$, if a perfect matching of the graph is given, where $|V|$ and $|E|$ are the order and the size of the graph respectively. Otherwise, time complexity may be dominated by the time needed to compute a maximum matching of the graph.

Note that a `ValueError` is returned if the graph has loops or if the graph is trivial, i.e., it has at most one vertex.

INPUT:

- `matching` – (default: `None`); a perfect matching of the graph, that can be given using any valid input format of *Graph*.

  If set to `None`, a matching is computed using the other parameters.

- `algorithm` – string (default: `'Edmonds'`); the algorithm to be used to compute a maximum matching of the graph among

  - `'Edmonds'` selects Edmonds' algorithm as implemented in NetworkX,
  - `'LP'` uses a Linear Program formulation of the matching problem.

- `coNP_certificate` – boolean (default: `False`); if set to `True` a set of pair of vertices (say u and v) is returned such that $G - u - v$ does not have a perfect matching if G is not bicritical or otherwise `None` is returned.

- `solver` – string (default: `None`); specify a Mixed Integer Linear Programming (MILP) solver to be used. If set to `None`, the default one is used. For more information on MILP solvers and which default solver is used, see the method `solve` of the class `MixedIntegerLinearProgram`.

- `verbose` – integer (default: `0`); sets the level of verbosity: set to 0 by default, which means quiet (only useful when `algorithm == 'LP'`).

- `integrality_tolerance` – float; parameter for use with MILP solvers over an inexact base ring; see `MixedIntegerLinearProgram.get_values()`.

OUTPUT:

- A boolean indicating whether the graph is bicritical or not.
- If `coNP_certificate` is set to `True`, a set of pair of vertices is returned in case the graph is not bicritical otherwise `None` is returned.

EXAMPLES:

The Petersen graph is bicritical:

```
sage: G = graphs.PetersenGraph()
sage: G.is_bicritical()
True
```

```
>>> from sage.all import *
>>> G = graphs.PetersenGraph()
>>> G.is_bicritical()
True
```

A graph (without a self-loop) is bicritical if and only if the underlying simple graph is bicritical:

```
sage: G = graphs.PetersenGraph()
sage: G.allow_multiple_edges(True)
sage: G.add_edge(0, 5)
sage: G.is_bicritical()
True
```

```
>>> from sage.all import *
>>> G = graphs.PetersenGraph()
>>> G.allow_multiple_edges(True)
>>> G.add_edge(Integer(0), Integer(5))
>>> G.is_bicritical()
True
```

A nontrivial circular ladder graph whose order is not divisible by 4 is bicritical:

```
sage: G = graphs.CircularLadderGraph(5)
sage: G.is_bicritical()
True
```

```
>>> from sage.all import *
>>> G = graphs.CircularLadderGraph(Integer(5))
>>> G.is_bicritical()
True
```

The graph obtained by splicing two bicritical graph is also bicritical. For instance, K_4 with one extra (multiple) edge (say $G := K_4^+$) is bicritical. Let $H := K_4^+ \odot K_4^+$ such that H is free of multiple edge. The graph H is also bicritical:

```
sage: G = graphs.CompleteGraph(4)
sage: G.allow_multiple_edges(True)
sage: G.add_edge(0, 1)
sage: G.is_bicritical()
True
sage: H = Graph()
sage: H.add_edges([
....:     (0, 1), (0, 2), (0, 3), (0, 4), (1, 2),
....:     (1, 5), (2, 5), (3, 4), (3, 5), (4, 5)
....: ])
sage: H.is_bicritical()
True
```

```
>>> from sage.all import *
>>> G = graphs.CompleteGraph(Integer(4))
>>> G.allow_multiple_edges(True)
>>> G.add_edge(Integer(0), Integer(1))
>>> G.is_bicritical()
True
>>> H = Graph()
>>> H.add_edges([
...     (Integer(0), Integer(1)), (Integer(0), Integer(2)), (Integer(0),␣
↪Integer(3)), (Integer(0), Integer(4)), (Integer(1), Integer(2)),
...     (Integer(1), Integer(5)), (Integer(2), Integer(5)), (Integer(3),␣
↪Integer(4)), (Integer(3), Integer(5)), (Integer(4), Integer(5))
... ])
>>> H.is_bicritical()
True
```

A graph (of order more than two) with more that one component is not bicritical:

```
sage: G = graphs.CycleGraph(4)
sage: G += graphs.CycleGraph(6)
sage: G.number_of_connected_components()
2
sage: G.is_bicritical()
False
```

```
>>> from sage.all import *
>>> G = graphs.CycleGraph(Integer(4))
>>> G += graphs.CycleGraph(Integer(6))
>>> G.number_of_connected_components()
2
>>> G.is_bicritical()
False
```

A graph (of order more than two) with a cut-vertex is not bicritical:

```
sage: G = graphs.CycleGraph(6)
sage: G.add_edges([(5, 6), (5, 7), (6, 7)])
sage: G.is_cut_vertex(5)
True
sage: G.has_perfect_matching()
True
sage: G.is_bicritical()
False
```

```
>>> from sage.all import *
>>> G = graphs.CycleGraph(Integer(6))
>>> G.add_edges([(Integer(5), Integer(6)), (Integer(5), Integer(7)),␣
↪(Integer(6), Integer(7))])
>>> G.is_cut_vertex(Integer(5))
True
>>> G.has_perfect_matching()
True
>>> G.is_bicritical()
False
```

A connected graph of order two is assumed to be bicritical, whereas the disconnected graph of the same order is not:

```
sage: G = graphs.CompleteBipartiteGraph(1, 1)
sage: G.is_bicritical()
True
sage: G = graphs.CompleteBipartiteGraph(2, 0)
sage: G.is_bicritical()
False
```

```
>>> from sage.all import *
>>> G = graphs.CompleteBipartiteGraph(Integer(1), Integer(1))
>>> G.is_bicritical()
True
>>> G = graphs.CompleteBipartiteGraph(Integer(2), Integer(0))
```

(continues on next page)

(continued from previous page)

```
>>> G.is_bicritical()
False
```

A bipartite graph of order three or more is not bicritical:

```
sage: G = graphs.CompleteBipartiteGraph(3, 3)
sage: G.has_perfect_matching()
True
sage: G.is_bicritical()
False
```

```
>>> from sage.all import *
>>> G = graphs.CompleteBipartiteGraph(Integer(3), Integer(3))
>>> G.has_perfect_matching()
True
>>> G.is_bicritical()
False
```

One may specify a matching:

```
sage: G = graphs.WheelGraph(10)
sage: M = G.matching()
sage: G.is_bicritical(matching=M)
True
sage: H = graphs.HexahedralGraph()
sage: N = H.matching()
sage: H.is_bicritical(matching=N)
False
```

```
>>> from sage.all import *
>>> G = graphs.WheelGraph(Integer(10))
>>> M = G.matching()
>>> G.is_bicritical(matching=M)
True
>>> H = graphs.HexahedralGraph()
>>> N = H.matching()
>>> H.is_bicritical(matching=N)
False
```

One may ask for a co-$\mathcal{NP}$ certificate:

```
sage: G = graphs.CompleteGraph(14)
sage: G.is_bicritical(coNP_certificate=True)
(True, None)
sage: H = graphs.CircularLadderGraph(20)
sage: M = H.matching()
sage: H.is_bicritical(matching=M, coNP_certificate=True)
(False, {0, 2})
```

```
>>> from sage.all import *
>>> G = graphs.CompleteGraph(Integer(14))
>>> G.is_bicritical(coNP_certificate=True)
```

(continues on next page)

(continued from previous page)

```
(True, None)
>>> H = graphs.CircularLadderGraph(Integer(20))
>>> M = H.matching()
>>> H.is_bicritical(matching=M, coNP_certificate=True)
(False, {0, 2})
```

REFERENCES:

- [LM2024]
- [LZ2001]

See also

is_factor_critical(), *is_matching_covered()*

AUTHORS:

- Janmenjaya Panda (2024-06-17)

is_block_graph()

Return whether this graph is a block graph.

A block graph is a connected graph in which every biconnected component (block) is a clique.

See also

- Wikipedia article Block_graph for more details on these graphs
- *RandomBlockGraph()* – generator of random block graphs
- *blocks_and_cut_vertices()*
- *blocks_and_cuts_tree()*

EXAMPLES:

```
sage: G = graphs.RandomBlockGraph(6, 2, kmax=4)
sage: G.is_block_graph()
True
sage: from sage.graphs.isgci import graph_classes
sage: G in graph_classes.Block
True
sage: graphs.CompleteGraph(4).is_block_graph()
True
sage: graphs.RandomTree(6).is_block_graph()
True
sage: graphs.PetersenGraph().is_block_graph()
False
sage: Graph(4).is_block_graph()
False
```

```
>>> from sage.all import *
>>> G = graphs.RandomBlockGraph(Integer(6), Integer(2), kmax=Integer(4))
>>> G.is_block_graph()
True
>>> from sage.graphs.isgci import graph_classes
>>> G in graph_classes.Block
True
>>> graphs.CompleteGraph(Integer(4)).is_block_graph()
True
>>> graphs.RandomTree(Integer(6)).is_block_graph()
True
>>> graphs.PetersenGraph().is_block_graph()
False
>>> Graph(Integer(4)).is_block_graph()
False
```

is_cactus()

Check whether the graph is cactus graph.

A graph is called *cactus graph* if it is connected and every pair of simple cycles have at most one common vertex.

There are other definitions, see the Wikipedia article Cactus_graph.

EXAMPLES:

```
sage: g = Graph({1: [2], 2: [3, 4], 3: [4, 5, 6, 7], 8: [3, 5], 9: [6, 7]})
sage: g.is_cactus()
True

sage: c6 = graphs.CycleGraph(6)
sage: naphthalene = c6 + c6
sage: naphthalene.is_cactus()  # Not connected
False
sage: naphthalene.merge_vertices([0, 6])
sage: naphthalene.is_cactus()
True
sage: naphthalene.merge_vertices([1, 7])
sage: naphthalene.is_cactus()
False
```

```
>>> from sage.all import *
>>> g = Graph({Integer(1): [Integer(2)], Integer(2): [Integer(3), Integer(4)],
↪ Integer(3): [Integer(4), Integer(5), Integer(6), Integer(7)], Integer(8):␣
↪[Integer(3), Integer(5)], Integer(9): [Integer(6), Integer(7)]})
>>> g.is_cactus()
True

>>> c6 = graphs.CycleGraph(Integer(6))
>>> naphthalene = c6 + c6
>>> naphthalene.is_cactus()  # Not connected
False
>>> naphthalene.merge_vertices([Integer(0), Integer(6)])
>>> naphthalene.is_cactus()
```

(continues on next page)

(continued from previous page)

```
True
>>> naphthalene.merge_vertices([Integer(1), Integer(7)])
>>> naphthalene.is_cactus()
False
```

is_cartesian_product(*g*, *certificate=False*, *relabeling=False*, *immutable=None*)

Test whether the graph is a Cartesian product.

INPUT:

- `certificate` – boolean (default: `False`); if `certificate = False` (default) the method only returns `True` or `False` answers. If `certificate = True`, the `True` answers are replaced by the list of the factors of the graph.
- `relabeling` – boolean (default: `False`); if `relabeling = True` (implies `certificate = True`), the method also returns a dictionary associating to each vertex its natural coordinates as a vertex of a product graph. If g is not a Cartesian product, `None` is returned instead.
- `immutable` – boolean (default: `None`); whether to create a mutable/immutable graph. `immutable=None` (default) means that the graph and its factors will behave the same way.

See also

- *cartesian_product()*
- *graph_products* – a module on graph products

Note

This algorithm may run faster whenever the graph's vertices are integers (see *relabel()*). Give it a try if it is too slow !

EXAMPLES:

The Petersen graph is prime:

```
sage: from sage.graphs.graph_decompositions.graph_products import is_
↪cartesian_product
sage: g = graphs.PetersenGraph()
sage: is_cartesian_product(g)
False
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_decompositions.graph_products import is_cartesian_
↪product
>>> g = graphs.PetersenGraph()
>>> is_cartesian_product(g)
False
```

A 2d grid is the product of paths:

```
sage: g = graphs.Grid2dGraph(5,5)
sage: p1, p2 = is_cartesian_product(g, certificate = True)
sage: p1.is_isomorphic(graphs.PathGraph(5))
True
sage: p2.is_isomorphic(graphs.PathGraph(5))
True
```

```
>>> from sage.all import *
>>> g = graphs.Grid2dGraph(Integer(5),Integer(5))
>>> p1, p2 = is_cartesian_product(g, certificate = True)
>>> p1.is_isomorphic(graphs.PathGraph(Integer(5)))
True
>>> p2.is_isomorphic(graphs.PathGraph(Integer(5)))
True
```

Forgetting the graph's labels, then finding them back:

```
sage: g.relabel()
sage: b,D = g.is_cartesian_product(g, relabeling=True)
sage: b
True
sage: D  # random isomorphism
{0: (20, 0), 1: (20, 1), 2: (20, 2), 3: (20, 3), 4: (20, 4),
 5: (15, 0), 6: (15, 1), 7: (15, 2), 8: (15, 3), 9: (15, 4),
 10: (10, 0), 11: (10, 1), 12: (10, 2), 13: (10, 3), 14: (10, 4),
 15: (5, 0), 16: (5, 1), 17: (5, 2), 18: (5, 3), 19: (5, 4),
 20: (0, 0), 21: (0, 1), 22: (0, 2), 23: (0, 3), 24: (0, 4)}
```

```
>>> from sage.all import *
>>> g.relabel()
>>> b,D = g.is_cartesian_product(g, relabeling=True)
>>> b
True
>>> D  # random isomorphism
{0: (20, 0), 1: (20, 1), 2: (20, 2), 3: (20, 3), 4: (20, 4),
 5: (15, 0), 6: (15, 1), 7: (15, 2), 8: (15, 3), 9: (15, 4),
 10: (10, 0), 11: (10, 1), 12: (10, 2), 13: (10, 3), 14: (10, 4),
 15: (5, 0), 16: (5, 1), 17: (5, 2), 18: (5, 3), 19: (5, 4),
 20: (0, 0), 21: (0, 1), 22: (0, 2), 23: (0, 3), 24: (0, 4)}
```

And of course, we find the factors back when we build a graph from a product:

```
sage: g = graphs.PetersenGraph().cartesian_product(graphs.CycleGraph(3))
sage: g1, g2 = is_cartesian_product(g, certificate = True)
sage: any( x.is_isomorphic(graphs.PetersenGraph()) for x in [g1,g2])
True
sage: any( x.is_isomorphic(graphs.CycleGraph(3)) for x in [g1,g2])
True
```

```
>>> from sage.all import *
>>> g = graphs.PetersenGraph().cartesian_product(graphs.
↪CycleGraph(Integer(3)))
```

(continues on next page)

(continued from previous page)

```
>>> g1, g2 = is_cartesian_product(g, certificate = True)
>>> any( x.is_isomorphic(graphs.PetersenGraph()) for x in [g1,g2])
True
>>> any( x.is_isomorphic(graphs.CycleGraph(Integer(3))) for x in [g1,g2])
True
```

is_chordal_bipartite(*certificate=False*)

Check whether the given graph is chordal bipartite.

A graph G is chordal bipartite if it is bipartite and has no induced cycle of length at least 6.

An edge $(x, y) \in E$ is bisimplical if $N(x) \cup N(y)$ induces a complete bipartite subgraph of G.

A Perfect Edge Without Vertex Elimination Ordering of a bipartite graph $G = (X, Y, E)$ is an ordering $e_1, ..., e_m$ of its edge set such that for all $1 \leq i \leq m$, e_i is a bisimplical edge of $G_i = (X, Y, E_i)$ where E_i consists of the edges $e_i, e_{i+1}..., e_m$.

A graph G is chordal bipartite if and only if it has a Perfect Edge Without Vertex Elimination Ordering. See Lemma 4 in [KKD1995].

INPUT:

- `certificate` – boolean (default: `False`); whether to return a certificate

OUTPUT:

When `certificate` is set to `False` (default) this method only returns `True` or `False` answers. When `certificate` is set to `True`, the method either returns:

- `(True, pewveo)` when the graph is chordal bipartite, where `pewveo` is a Perfect Edge Without Vertex Elimination Ordering of edges.
- `(False, cycle)` when the graph is not chordal bipartite, where `cycle` is an odd cycle or a chordless cycle of length at least 6.

ALGORITHM:

This algorithm is based on these facts. The first one is trivial.

- A reduced adjacnecy matrix (*reduced_adjacency_matrix()*) of a bipartite graph has no cycle submatrix if and only if the graph is chordal bipartite, where cycle submatrix is 0-1 $n \times n$ matrix $n \geq 3$ with exactly two 1's in each row and column and no proper submatrix satisfy this property.
- A doubly lexical ordering (`doubly_lexical_ordering()`) of a 0-1 matrix is Γ-free (`is_Gamma_free()`) if and only if the matrix has no cycle submatrix. See Theorem 5.4 in [Lub1987].

Hence, checking a doubly lexical ordering of a reduced adjacency matrix of a bipartite graph is Γ-free leads to detecting the graph is chordal bipartite. Also, this matrix contains a certificate. Hence, if G is chordal bipartite, we find a Perfect Edge Without Vertex Elimination Ordering of edges by Lemma 10 in [KKD1995]. Otherwise, we can find a cycle submatrix by Theorem 5.2 in [Lub1987]. The time complexity of this algorithm is $O(n^3)$.

EXAMPLES:

A non-bipartite graph is not chordal bipartite:

```
sage: g = graphs.CycleGraph(5)
sage: g.is_chordal_bipartite()
False
```

(continues on next page)

(continued from previous page)

```
sage: _, cycle = g.is_chordal_bipartite(certificate=True)
sage: len(cycle) % 2 == 1
True
```

```
>>> from sage.all import *
>>> g = graphs.CycleGraph(Integer(5))
>>> g.is_chordal_bipartite()
False
>>> _, cycle = g.is_chordal_bipartite(certificate=True)
>>> len(cycle) % Integer(2) == Integer(1)
True
```

A 6-cycle graph is not chordal bipartite:

```
sage: g = graphs.CycleGraph(6)
sage: g.is_chordal_bipartite()
False
sage: _, cycle = g.is_chordal_bipartite(certificate=True)
sage: len(cycle) == 6
True
```

```
>>> from sage.all import *
>>> g = graphs.CycleGraph(Integer(6))
>>> g.is_chordal_bipartite()
False
>>> _, cycle = g.is_chordal_bipartite(certificate=True)
>>> len(cycle) == Integer(6)
True
```

A $2 \times n$ grid graph is chordal bipartite:

```
sage: g = graphs.Grid2dGraph(2, 6)
sage: result, pewveo = g.is_chordal_bipartite(certificate=True)
sage: result
True
```

```
>>> from sage.all import *
>>> g = graphs.Grid2dGraph(Integer(2), Integer(6))
>>> result, pewveo = g.is_chordal_bipartite(certificate=True)
>>> result
True
```

Let us check the certificate given by Sage is indeed a perfect edge without vertex elimination ordering:

```
sage: for e in pewveo:
....:     a = g.subgraph(vertices=g.neighbors(e[0]) + g.neighbors(e[1]))
....:     b = BipartiteGraph(a).complement_bipartite()
....:     if b.edges():
....:         raise ValueError("this should never happen")
....:     g.delete_edge(e)
```

```
>>> from sage.all import *
>>> for e in pewveo:
...     a = g.subgraph(vertices=g.neighbors(e[Integer(0)]) + g.
↪neighbors(e[Integer(1)]))
...     b = BipartiteGraph(a).complement_bipartite()
...     if b.edges():
...         raise ValueError("this should never happen")
...     g.delete_edge(e)
```

Let us check the certificate given by Sage is indeed a chordless cycle of length at least 6:

```
sage: g = graphs.Grid2dGraph(3, 6)
sage: result, cycle = g.is_chordal_bipartite(certificate=True)
sage: result
False
sage: l = len(cycle); l >= 6
True
sage: for i in range(len(cycle)):
....:     if not g.has_edge(cycle[i], cycle[(i+1)%l]):
....:         raise ValueError("this should never happen")
sage: h = g.subgraph(vertices=cycle)
sage: h.is_cycle()
True
```

```
>>> from sage.all import *
>>> g = graphs.Grid2dGraph(Integer(3), Integer(6))
>>> result, cycle = g.is_chordal_bipartite(certificate=True)
>>> result
False
>>> l = len(cycle); l >= Integer(6)
True
>>> for i in range(len(cycle)):
...     if not g.has_edge(cycle[i], cycle[(i+Integer(1))%l]):
...         raise ValueError("this should never happen")
>>> h = g.subgraph(vertices=cycle)
>>> h.is_cycle()
True
```

is_circumscribable(*solver='ppl'*, *verbose=0*)

Test whether the graph is the graph of a circumscribed polyhedron.

A polyhedron is circumscribed if all of its facets are tangent to a sphere. By a theorem of Rivin ([HRS1993]), this can be checked by solving a linear program that assigns weights between 0 and 1/2 on each edge of the polyhedron, so that the weights on any face add to exactly one and the weights on any non-facial cycle add to more than one. If and only if this can be done, the polyhedron can be circumscribed.

INPUT:

- `solver` – (default: `'ppl'`) specify a Linear Program (LP) solver to be used. If set to `None`, the default one is used. For more information on LP solvers and which default solver is used, see the method `solve` of the class `MixedIntegerLinearProgram`.
- `verbose` – integer (default: 0); sets the level of verbosity. Set to 0 by default, which means quiet.

EXAMPLES:

```
sage: C = graphs.CubeGraph(3)
sage: C.is_circumscribable()                                                #␣
↪needs sage.numerical.mip
True

sage: O = graphs.OctahedralGraph()
sage: O.is_circumscribable()                                                #␣
↪needs sage.numerical.mip
True

sage: TT = polytopes.truncated_tetrahedron().graph()                        #␣
↪needs sage.geometry.polyhedron
sage: TT.is_circumscribable()                                               #␣
↪needs sage.geometry.polyhedron sage.numerical.mip
False
```

```
>>> from sage.all import *
>>> C = graphs.CubeGraph(Integer(3))
>>> C.is_circumscribable()                                                  #␣
↪needs sage.numerical.mip
True

>>> O = graphs.OctahedralGraph()
>>> O.is_circumscribable()                                                  #␣
↪needs sage.numerical.mip
True

>>> TT = polytopes.truncated_tetrahedron().graph()                          #␣
↪needs sage.geometry.polyhedron
>>> TT.is_circumscribable()                                                 #␣
↪needs sage.geometry.polyhedron sage.numerical.mip
False
```

Stellating in a face of the octahedral graph is not circumscribable:

```
sage: f = set(flatten(choice(O.faces())))
sage: O.add_edges([[6, i] for i in f])
sage: O.is_circumscribable()                                                #␣
↪needs sage.numerical.mip
False
```

```
>>> from sage.all import *
>>> f = set(flatten(choice(O.faces())))
>>> O.add_edges([[Integer(6), i] for i in f])
>>> O.is_circumscribable()                                                  #␣
↪needs sage.numerical.mip
False
```

See also

- *is_polyhedral()*

- *is_inscribable()*

Todo

Allow the use of other, inexact but faster solvers.

is_cograph()

Check whether the graph is cograph.

A cograph is defined recursively: the single-vertex graph is cograph, complement of cograph is cograph, and disjoint union of two cographs is cograph. There are many other characterizations, see the Wikipedia article Cograph.

EXAMPLES:

```
sage: graphs.HouseXGraph().is_cograph()
True
sage: graphs.HouseGraph().is_cograph()                                    #␣
↪needs sage.modules
False
```

```
>>> from sage.all import *
>>> graphs.HouseXGraph().is_cograph()
True
>>> graphs.HouseGraph().is_cograph()                                      #␣
↪needs sage.modules
False
```

Todo

Implement faster recognition algorithm, as for instance the linear time recognition algorithm using LexBFS proposed in [Bre2008].

is_comparability(*g*, *algorithm='greedy'*, *certificate=False*, *check=True*, *solver=None*, *verbose=0*)

Check whether the graph is a comparability graph.

INPUT:

- `g` – a graph
- `algorithm` – string (default: `'greedy'`); choose the implementation used to do the test
  - `'greedy'` – a greedy algorithm (see the documentation of the *comparability module*)
  - `'MILP'` – a Mixed Integer Linear Program formulation of the problem. Beware, for this implementation is unable to return negative certificates ! When `certificate = True`, negative certificates are always equal to `None`. `True` certificates are valid, though.
- `certificate` – boolean (default: `False`); whether to return a certificate. *Yes*-answers the certificate is a transitive orientation of G, and a *no* certificates is an odd cycle of sequentially forcing edges.
- `check` – boolean (default: `True`); whether to check that the yes-certificates are indeed transitive. As it is very quick compared to the rest of the operation, it is enabled by default.

- `solver` – string (default: `None`); specifies a Mixed Integer Linear Programming (MILP) solver to be used. If set to `None`, the default one is used. For more information on MILP solvers and which default solver is used, see the method `solve` of the class `MixedIntegerLinearProgram`.
- `verbose` – integer (default: 0); sets the level of verbosity. Set to 0 by default, which means quiet.

EXAMPLES:

```
sage: from sage.graphs.comparability import is_comparability
sage: g = graphs.PetersenGraph()
sage: is_comparability(g)
False
sage: is_comparability(graphs.CompleteGraph(5), certificate=True)
(True, Digraph on 5 vertices)
```

```
>>> from sage.all import *
>>> from sage.graphs.comparability import is_comparability
>>> g = graphs.PetersenGraph()
>>> is_comparability(g)
False
>>> is_comparability(graphs.CompleteGraph(Integer(5)), certificate=True)
(True, Digraph on 5 vertices)
```

is_directed()

Since graph is undirected, returns False.

EXAMPLES:

```
sage: Graph().is_directed()
False
```

```
>>> from sage.all import *
>>> Graph().is_directed()
False
```

is_distance_regular(*G*, *parameters=False*)

Test if the graph is distance-regular.

A graph G is distance-regular if for any integers j, k the value of $|\{x : d_G(x,u) = j, x \in V(G)\} \cap \{y : d_G(y,v) = j, y \in V(G)\}|$ is constant for any two vertices $u, v \in V(G)$ at distance i from each other. In particular G is regular, of degree b_0 (see below), as one can take $u = v$.

Equivalently a graph is distance-regular if there exist integers b_i, c_i such that for any two vertices u, v at distance i we have

- $b_i = |\{x : d_G(x,u) = i+1, x \in V(G)\} \cap N_G(v)\}|,\ 0 \leq i \leq d-1$
- $c_i = |\{x : d_G(x,u) = i-1, x \in V(G)\} \cap N_G(v)\}|,\ 1 \leq i \leq d,$

where d is the diameter of the graph. For more information on distance-regular graphs, see the Wikipedia article Distance-regular_graph.

INPUT:

- `parameters` – boolean (default: `False`); if set to `True`, the function returns the pair `(b, c)` of lists of integers instead of a boolean answer (see the definition above)

See also

- *is_regular()*
- *is_strongly_regular()*

EXAMPLES:

```
sage: g = graphs.PetersenGraph()
sage: g.is_distance_regular()
True
sage: g.is_distance_regular(parameters = True)
([3, 2, None], [None, 1, 1])
```

```
>>> from sage.all import *
>>> g = graphs.PetersenGraph()
>>> g.is_distance_regular()
True
>>> g.is_distance_regular(parameters = True)
([3, 2, None], [None, 1, 1])
```

Cube graphs, which are not strongly regular, are a bit more interesting:

```
sage: graphs.CubeGraph(4).is_distance_regular()
True
sage: graphs.OddGraph(5).is_distance_regular()
True
```

```
>>> from sage.all import *
>>> graphs.CubeGraph(Integer(4)).is_distance_regular()
True
>>> graphs.OddGraph(Integer(5)).is_distance_regular()
True
```

Disconnected graph:

```
sage: (2*graphs.CubeGraph(4)).is_distance_regular()
True
```

```
>>> from sage.all import *
>>> (Integer(2)*graphs.CubeGraph(Integer(4))).is_distance_regular()
True
```

is_dominating(*G*, *dom*, *focus=None*)

Check whether `dom` is a dominating set of `G`.

We say that a set D of vertices of a graph G dominates a set S if every vertex of S either belongs to D or is adjacent to a vertex of D. Also, D is a dominating set of G if it dominates $V(G)$.

INPUT:

- `dom` – iterable of vertices of `G`; the vertices of the supposed dominating set
- `focus` – iterable of vertices of `G` (default: `None`); if specified, this method checks instead if `dom` dominates the vertices in `focus`

EXAMPLES:

```
sage: g = graphs.CycleGraph(5)
sage: g.is_dominating([0,1], [4, 2])
True

sage: g.is_dominating([0,1])
False
```

```
>>> from sage.all import *
>>> g = graphs.CycleGraph(Integer(5))
>>> g.is_dominating([Integer(0),Integer(1)], [Integer(4), Integer(2)])
True

>>> g.is_dominating([Integer(0),Integer(1)])
False
```

is_edge_transitive()

Check if `self` is an edge transitive graph.

A graph is edge-transitive if its automorphism group acts transitively on its edge set.

Equivalently, if there exists for any pair of edges $uv, u'v' \in E(G)$ an automorphism ϕ of G such that $\phi(uv) = u'v'$ (note this does not necessarily mean that $\phi(u) = u'$ and $\phi(v) = v'$).

See also

- Wikipedia article Edge-transitive_graph
- *is_arc_transitive()*
- *is_half_transitive()*
- *is_semi_symmetric()*

EXAMPLES:

```
sage: P = graphs.PetersenGraph()
sage: P.is_edge_transitive()                                                #␣
↪needs sage.libs.gap
True
sage: C = graphs.CubeGraph(3)
sage: C.is_edge_transitive()                                                #␣
↪needs sage.libs.gap
True
sage: G = graphs.GrayGraph()
sage: G.is_edge_transitive()                                                #␣
↪needs sage.libs.gap
True
sage: P = graphs.PathGraph(4)
sage: P.is_edge_transitive()                                                #␣
↪needs sage.libs.gap
False
```

```
>>> from sage.all import *
>>> P = graphs.PetersenGraph()
>>> P.is_edge_transitive()                                                    #␣
↪needs sage.libs.gap
True
>>> C = graphs.CubeGraph(Integer(3))
>>> C.is_edge_transitive()                                                    #␣
↪needs sage.libs.gap
True
>>> G = graphs.GrayGraph()
>>> G.is_edge_transitive()                                                    #␣
↪needs sage.libs.gap
True
>>> P = graphs.PathGraph(Integer(4))
>>> P.is_edge_transitive()                                                    #␣
↪needs sage.libs.gap
False
```

is_even_hole_free(*certificate=False*)

Test whether `self` contains an induced even hole.

A Hole is a cycle of length at least 4 (included). It is said to be even (resp. odd) if its length is even (resp. odd).

Even-hole-free graphs always contain a bisimplicial vertex, which ensures that their chromatic number is at most twice their clique number [ACHRS2008].

INPUT:

- `certificate` – boolean (default: `False`); when `certificate = False`, this method only returns `True` or `False`. If `certificate = True`, the subgraph found is returned instead of `False`.

EXAMPLES:

Is the Petersen Graph even-hole-free

```
sage: g = graphs.PetersenGraph()
sage: g.is_even_hole_free()                                                   #␣
↪needs sage.modules
False
```

```
>>> from sage.all import *
>>> g = graphs.PetersenGraph()
>>> g.is_even_hole_free()                                                     #␣
↪needs sage.modules
False
```

As any chordal graph is hole-free, interval graphs behave the same way:

```
sage: g = graphs.RandomIntervalGraph(20)
sage: g.is_even_hole_free()                                                   #␣
↪needs sage.modules
True
```

```
>>> from sage.all import *
>>> g = graphs.RandomIntervalGraph(Integer(20))
>>> g.is_even_hole_free()                                                    #␣
↪needs sage.modules
True
```

It is clear, though, that a random Bipartite Graph which is not a forest has an even hole:

```
sage: g = graphs.RandomBipartite(10, 10, .5)                                #␣
↪needs numpy
sage: g.is_even_hole_free() and not g.is_forest()                           #␣
↪needs numpy sage.modules
False
```

```
>>> from sage.all import *
>>> g = graphs.RandomBipartite(Integer(10), Integer(10), RealNumber('.5'))    ␣
↪                              # needs numpy
>>> g.is_even_hole_free() and not g.is_forest()                             #␣
↪needs numpy sage.modules
False
```

We can check the certificate returned is indeed an even cycle:

```
sage: if not g.is_forest():                                                 #␣
↪needs numpy sage.modules
....:     cycle = g.is_even_hole_free(certificate=True)
....:     if cycle.order() % 2 == 1:
....:         print("Error !")
....:     if not cycle.is_isomorphic(
....:            graphs.CycleGraph(cycle.order())):
....:         print("Error !")
...
sage: print("Everything is Fine !")
Everything is Fine !
```

```
>>> from sage.all import *
>>> if not g.is_forest():                                                   #␣
↪needs numpy sage.modules
...     cycle = g.is_even_hole_free(certificate=True)
...     if cycle.order() % Integer(2) == Integer(1):
...         print("Error !")
...     if not cycle.is_isomorphic(
...            graphs.CycleGraph(cycle.order())):
...         print("Error !")
...
>>> print("Everything is Fine !")
Everything is Fine !
```

is_factor_critical(*G*, *matching*, *algorithm=None*, *solver='Edmonds'*, *verbose=None*, *integrality_tolerance=0*)

Check whether the graph is factor-critical.

A graph of order n is *factor-critical* if every subgraph of $n-1$ vertices have a perfect matching, hence n must be odd. See Wikipedia article Factor-critical_graph for more details.

This method implements the algorithm proposed in [LR2004] and we assume that a graph of order one is factor-critical. The time complexity of the algorithm is linear if a near perfect matching is given as input (i.e., a matching such that all vertices but one are incident to an edge of the matching). Otherwise, the time complexity is dominated by the time needed to compute a maximum matching of the graph.

INPUT:

- `matching` – (default: `None`); a near perfect matching of the graph, that is a matching such that all vertices of the graph but one are incident to an edge of the matching. It can be given using any valid input format of *Graph*.

  If set to `None`, a matching is computed using the other parameters.

- `algorithm` – string (default: `'Edmonds'`); the algorithm to use to compute a maximum matching of the graph among

  - `'Edmonds'` selects Edmonds' algorithm as implemented in NetworkX
  - `'LP'` uses a Linear Program formulation of the matching problem

- `solver` – string (default: `None`); specifies a Mixed Integer Linear Programming (MILP) solver to be used. If set to `None`, the default one is used. For more information on MILP solvers and which default solver is used, see the method `solve` of the class `MixedIntegerLinearProgram`.

- `verbose` – integer (default: 0); sets the level of verbosity: set to 0 by default, which means quiet (only useful when `algorithm == "LP"`)

- `integrality_tolerance` – float; parameter for use with MILP solvers over an inexact base ring; see `MixedIntegerLinearProgram.get_values()`.

EXAMPLES:

Odd length cycles and odd cliques of order at least 3 are factor-critical graphs:

```
sage: [graphs.CycleGraph(2*i + 1).is_factor_critical() for i in range(5)]    #␣
↪needs networkx
[True, True, True, True, True]
sage: [graphs.CompleteGraph(2*i + 1).is_factor_critical() for i in range(5)] ␣
↪           # needs networkx
[True, True, True, True, True]
```

```
>>> from sage.all import *
>>> [graphs.CycleGraph(Integer(2)*i + Integer(1)).is_factor_critical() for i␣
↪in range(Integer(5))]    # needs networkx
[True, True, True, True, True]
>>> [graphs.CompleteGraph(Integer(2)*i + Integer(1)).is_factor_critical() for␣
↪i in range(Integer(5))]             # needs networkx
[True, True, True, True, True]
```

More generally, every Hamiltonian graph with an odd number of vertices is factor-critical:

```
sage: G = graphs.RandomGNP(15, .2)
sage: G.add_path([0..14])
sage: G.add_edge(14, 0)
sage: G.is_hamiltonian()
True
sage: G.is_factor_critical()                                                #␣
↪needs networkx
True
```

```
>>> from sage.all import *
>>> G = graphs.RandomGNP(Integer(15), RealNumber('.2'))
>>> G.add_path((ellipsis_range(Integer(0),Ellipsis,Integer(14))))
>>> G.add_edge(Integer(14), Integer(0))
>>> G.is_hamiltonian()
True
>>> G.is_factor_critical()                                                    #␣
↪needs networkx
True
```

Friendship graphs are non-Hamiltonian factor-critical graphs:

```
sage: all(graphs.FriendshipGraph(i).is_factor_critical() for i in range(1,␣
↪5))             # needs networkx
True
```

```
>>> from sage.all import *
>>> all(graphs.FriendshipGraph(i).is_factor_critical() for i in␣
↪range(Integer(1), Integer(5)))             # needs networkx
True
```

Bipartite graphs are not factor-critical:

```
sage: G = graphs.RandomBipartite(randint(1, 10), randint(1, 10), .5)         #␣
↪needs numpy
sage: G.is_factor_critical()                                                 #␣
↪needs numpy
False
```

```
>>> from sage.all import *
>>> G = graphs.RandomBipartite(randint(Integer(1), Integer(10)),␣
↪randint(Integer(1), Integer(10)), RealNumber('.5'))        # needs numpy
>>> G.is_factor_critical()                                                   #␣
↪needs numpy
False
```

Graphs with even order are not factor critical:

```
sage: G = graphs.RandomGNP(10, .5)
sage: G.is_factor_critical()
False
```

```
>>> from sage.all import *
>>> G = graphs.RandomGNP(Integer(10), RealNumber('.5'))
>>> G.is_factor_critical()
False
```

One can specify a matching:

```
sage: F = graphs.FriendshipGraph(4)
sage: M = F.matching()                                                       #␣
↪needs networkx
sage: F.is_factor_critical(matching=M)                                       #␣
```

(continues on next page)

(continued from previous page)

```
↪needs networkx
True
sage: F.is_factor_critical(matching=Graph(M))                                  #␣
↪needs networkx
True
```

```
>>> from sage.all import *
>>> F = graphs.FriendshipGraph(Integer(4))
>>> M = F.matching()                                                           #␣
↪needs networkx
>>> F.is_factor_critical(matching=M)                                           #␣
↪needs networkx
True
>>> F.is_factor_critical(matching=Graph(M))                                    #␣
↪needs networkx
True
```

is_forest(*certificate=False*, *output='vertex'*)

Test if the graph is a forest, i.e. a disjoint union of trees.

INPUT:

- `certificate` – boolean (default: `False`); whether to return a certificate. The method only returns boolean answers when `certificate = False` (default). When it is set to `True`, it either answers `(True, None)` when the graph is a forest or `(False, cycle)` when it contains a cycle.
- `output` – either `'vertex'` (default) or `'edge'`; whether the certificate is given as a list of vertices (`output = 'vertex'`) or a list of edges (`output = 'edge'`).

EXAMPLES:

```
sage: seven_acre_wood = sum(graphs.trees(7), Graph())
sage: seven_acre_wood.is_forest()
True
```

```
>>> from sage.all import *
>>> seven_acre_wood = sum(graphs.trees(Integer(7)), Graph())
>>> seven_acre_wood.is_forest()
True
```

With certificates:

```
sage: g = graphs.RandomTree(30)
sage: g.is_forest(certificate=True)
(True, None)
sage: (2*g + graphs.PetersenGraph() + g).is_forest(certificate=True)
(False, [64, 69, 67, 65, 60])
```

```
>>> from sage.all import *
>>> g = graphs.RandomTree(Integer(30))
>>> g.is_forest(certificate=True)
(True, None)
>>> (Integer(2)*g + graphs.PetersenGraph() + g).is_forest(certificate=True)
(False, [64, 69, 67, 65, 60])
```

is_half_transitive()

Check if `self` is a half-transitive graph.

A graph is half-transitive if it is both vertex and edge transitive but not arc-transitive.

See also

- Wikipedia article half-transitive_graph
- *is_edge_transitive()*
- *is_arc_transitive()*
- *is_semi_symmetric()*

EXAMPLES:

The Petersen Graph is not half-transitive:

```
sage: P = graphs.PetersenGraph()
sage: P.is_half_transitive()                                                #␣
↪needs sage.libs.gap
False
```

```
>>> from sage.all import *
>>> P = graphs.PetersenGraph()
>>> P.is_half_transitive()                                                  #␣
↪needs sage.libs.gap
False
```

The smallest half-transitive graph is the Holt Graph:

```
sage: H = graphs.HoltGraph()
sage: H.is_half_transitive()                                                #␣
↪needs sage.libs.gap
True
```

```
>>> from sage.all import *
>>> H = graphs.HoltGraph()
>>> H.is_half_transitive()                                                  #␣
↪needs sage.libs.gap
True
```

is_inscribable(*solver='ppl'*, *verbose=0*)

Test whether the graph is the graph of an inscribed polyhedron.

A polyhedron is inscribed if all of its vertices are on a sphere. This is dual to the notion of circumscribed polyhedron: A Polyhedron is inscribed if and only if its polar dual is circumscribed and hence a graph is inscribable if and only if its planar dual is circumscribable.

INPUT:

- `solver` – (default: `'ppl'`) specify a Linear Program (LP) solver to be used. If set to `None`, the default one is used. For more information on LP solvers and which default solver is used, see the method `solve` of the class `MixedIntegerLinearProgram`.

- `verbose` – integer (default: 0); sets the level of verbosity. Set to 0 by default, which means quiet.

EXAMPLES:

```
sage: H = graphs.HerschelGraph()
sage: H.is_inscribable()             # long time (> 1 sec)                    #␣
↪needs sage.numerical.mip
False
sage: H.planar_dual().is_inscribable()      # long time (> 1 sec)             #␣
↪needs sage.numerical.mip
True

sage: C = graphs.CubeGraph(3)
sage: C.is_inscribable()                                                      #␣
↪needs sage.numerical.mip
True
```

```
>>> from sage.all import *
>>> H = graphs.HerschelGraph()
>>> H.is_inscribable()             # long time (> 1 sec)                    #␣
↪needs sage.numerical.mip
False
>>> H.planar_dual().is_inscribable()      # long time (> 1 sec)             #␣
↪needs sage.numerical.mip
True

>>> C = graphs.CubeGraph(Integer(3))
>>> C.is_inscribable()                                                      #␣
↪needs sage.numerical.mip
True
```

Cutting off a vertex from the cube yields an uninscribable graph:

```
sage: C = graphs.CubeGraph(3)
sage: v = next(C.vertex_iterator())
sage: triangle = [_ + v for _ in C.neighbors(v)]
sage: C.add_edges(Combinations(triangle, 2))
sage: C.add_edges(zip(triangle, C.neighbors(v)))
sage: C.delete_vertex(v)
sage: C.is_inscribable()                                                      #␣
↪needs sage.numerical.mip
False
```

```
>>> from sage.all import *
>>> C = graphs.CubeGraph(Integer(3))
>>> v = next(C.vertex_iterator())
>>> triangle = [_ + v for _ in C.neighbors(v)]
>>> C.add_edges(Combinations(triangle, Integer(2)))
>>> C.add_edges(zip(triangle, C.neighbors(v)))
>>> C.delete_vertex(v)
>>> C.is_inscribable()                                                      #␣
↪needs sage.numerical.mip
False
```

Breaking a face of the cube yields an uninscribable graph:

```
sage: C = graphs.CubeGraph(3)
sage: face = choice(C.faces())
sage: C.add_edge([face[0][0], face[2][0]])
sage: C.is_inscribable()                                                    #␣
↪needs sage.numerical.mip
False
```

```
>>> from sage.all import *
>>> C = graphs.CubeGraph(Integer(3))
>>> face = choice(C.faces())
>>> C.add_edge([face[Integer(0)][Integer(0)], face[Integer(2)][Integer(0)]])
>>> C.is_inscribable()                                                      #␣
↪needs sage.numerical.mip
False
```

See also

- *is_polyhedral()*
- *is_circumscribable()*

is_line_graph(*g*, *certificate=False*)

Check whether the graph *g* is a line graph.

INPUT:

- `certificate` – boolean; whether to return a certificate along with the boolean result. Here is what happens when `certificate = True`:
  - If the graph is not a line graph, the method returns a pair `(b, subgraph)` where `b` is `False` and `subgraph` is a subgraph isomorphic to one of the 9 forbidden induced subgraphs of a line graph.
  - If the graph is a line graph, the method returns a triple `(b,R,isom)` where `b` is `True`, `R` is a graph whose line graph is the graph given as input, and `isom` is a map associating an edge of `R` to each vertex of the graph.

Note

This method wastes a bit of time when the input graph is not connected. If you have performance in mind, it is probably better to only feed it with connected graphs only.

See also

- The *line_graph* module.
- *line_graph_forbidden_subgraphs()* – the forbidden subgraphs of a line graph.
- *line_graph()*

EXAMPLES:

A complete graph is always the line graph of a star:

```
sage: graphs.CompleteGraph(5).is_line_graph()
True
```

```
>>> from sage.all import *
>>> graphs.CompleteGraph(Integer(5)).is_line_graph()
True
```

The Petersen Graph not being claw-free, it is not a line graph:

```
sage: graphs.PetersenGraph().is_line_graph()
False
```

```
>>> from sage.all import *
>>> graphs.PetersenGraph().is_line_graph()
False
```

This is indeed the subgraph returned:

```
sage: C = graphs.PetersenGraph().is_line_graph(certificate=True)[1]
↪  # needs sage.modules
sage: C.is_isomorphic(graphs.ClawGraph())
↪  # needs sage.modules
True
```

```
>>> from sage.all import *
>>> C = graphs.PetersenGraph().is_line_graph(certificate=True)[Integer(1)]
↪          # needs sage.modules
>>> C.is_isomorphic(graphs.ClawGraph())
↪# needs sage.modules
True
```

The house graph is a line graph:

```
sage: g = graphs.HouseGraph()
sage: g.is_line_graph()
True
```

```
>>> from sage.all import *
>>> g = graphs.HouseGraph()
>>> g.is_line_graph()
True
```

But what is the graph whose line graph is the house ?:

```
sage: is_line, R, isom = g.is_line_graph(certificate=True)
sage: R.sparse6_string()
':DaHI~'
sage: R.show()
↪  # needs sage.plot
sage: isom
{0: (0, 1), 1: (0, 2), 2: (1, 3), 3: (2, 3), 4: (3, 4)}
```

```
>>> from sage.all import *
>>> is_line, R, isom = g.is_line_graph(certificate=True)
>>> R.sparse6_string()
':DaHI~'
>>> R.show()
↪# needs sage.plot
>>> isom
{0: (0, 1), 1: (0, 2), 2: (1, 3), 3: (2, 3), 4: (3, 4)}
```

is_long_antihole_free(*g*, *certificate=False*)

Test whether the given graph contains an induced subgraph that is isomorphic to the complement of a cycle of length at least 5.

INPUT:

- `certificate` – boolean (default: `False`)

  Whether to return a certificate. When `certificate = True`, then the function returns

  - `(False, Antihole)` if `g` contains an induced complement of a cycle of length at least 5 returned as `Antihole`.
  - `(True, [])` if `g` does not contain an induced complement of a cycle of length at least 5. For this case it is not known how to provide a certificate.

  When `certificate = False`, the function returns just `True` or `False` accordingly.

ALGORITHM:

This algorithm tries to find a cycle in the graph of all induced $\overline{P_4}$ of g, where two copies $\overline{P}$ and $\overline{P'}$ of $\overline{P_4}$ are adjacent if there exists a (not necessarily induced) copy of $\overline{P_5} = u_1u_2u_3u_4u_5$ such that $\overline{P} = u_1u_2u_3u_4$ and $\overline{P'} = u_2u_3u_4u_5$.

This is done through a depth-first-search. For efficiency, the auxiliary graph is constructed on-the-fly and never stored in memory.

The run time of this algorithm is $O(n + m^2)$ for `SparseGraph` and $O(n^2 \log m + m^2)$ for `DenseGraph` [NP2007] (where n is the number of vertices and m is the number of edges of the graph).

EXAMPLES:

The Petersen Graph contains an antihole:

```
sage: g = graphs.PetersenGraph()
sage: g.is_long_antihole_free()
False
```

```
>>> from sage.all import *
>>> g = graphs.PetersenGraph()
>>> g.is_long_antihole_free()
False
```

The complement of a cycle is an antihole:

```
sage: g = graphs.CycleGraph(6).complement()
sage: r,a = g.is_long_antihole_free(certificate=True)
sage: r
False
```

(continues on next page)

(continued from previous page)

```
sage: a.complement().is_isomorphic(graphs.CycleGraph(6))
True
```

```
>>> from sage.all import *
>>> g = graphs.CycleGraph(Integer(6)).complement()
>>> r,a = g.is_long_antihole_free(certificate=True)
>>> r
False
>>> a.complement().is_isomorphic(graphs.CycleGraph(Integer(6)))
True
```

is_long_hole_free(*g*, *certificate=False*)

Test whether `g` contains an induced cycle of length at least 5.

INPUT:

- `certificate` – boolean (default: `False`)

  Whether to return a certificate. When `certificate = True`, then the function returns

  - `(True, [])` if `g` does not contain such a cycle. For this case, it is not known how to provide a certificate.
  - `(False, Hole)` if `g` contains an induced cycle of length at least 5. `Hole` returns this cycle.

  If `certificate = False`, the function returns just `True` or `False` accordingly.

ALGORITHM:

This algorithm tries to find a cycle in the graph of all induced P_4 of g, where two copies P and P' of P_4 are adjacent if there exists a (not necessarily induced) copy of $P_5 = u_1u_2u_3u_4u_5$ such that $P = u_1u_2u_3u_4$ and $P' = u_2u_3u_4u_5$.

This is done through a depth-first-search. For efficiency, the auxiliary graph is constructed on-the-fly and never stored in memory.

The run time of this algorithm is $O(n+m^2)$ for `SparseGraph` and $O(n^2+m^2)$ for `DenseGraph` [NP2007] (where n is the number of vertices and m is the number of edges of the graph).

EXAMPLES:

The Petersen Graph contains a hole:

```
sage: g = graphs.PetersenGraph()
sage: g.is_long_hole_free()
False
```

```
>>> from sage.all import *
>>> g = graphs.PetersenGraph()
>>> g.is_long_hole_free()
False
```

The following graph contains a hole, which we want to display:

```
sage: g = graphs.FlowerSnark()
sage: r,h = g.is_long_hole_free(certificate=True)
sage: r
```

(continues on next page)

(continued from previous page)

```
False
sage: Graph(h).is_isomorphic(graphs.CycleGraph(h.order()))
True
```

```
>>> from sage.all import *
>>> g = graphs.FlowerSnark()
>>> r,h = g.is_long_hole_free(certificate=True)
>>> r
False
>>> Graph(h).is_isomorphic(graphs.CycleGraph(h.order()))
True
```

is_matching_covered(*G*, *matching*, *algorithm=None*, *coNP_certificate='Edmonds'*, *solver=False*, *verbose=None*, *integrality_tolerance=0*)

Check if the graph is matching covered.

A connected nontrivial graph wherein each edge participates in some perfect matching is called a *matching covered graph*.

If a perfect matching of the graph is provided, for bipartite graph, this method implements a linear time algorithm as proposed in [LM2024] that is based on the following theorem:

Given a connected bipartite graph $G[A, B]$ with a perfect matching M. Construct a directed graph D from G such that $V(D) := V(G)$ and for each edge in G direct the corresponding edge from A to B in D, if it is in M or otherwise direct it from B to A. The graph G is matching covered if and only if D is strongly connected.

For nonbipartite graph, if a perfect matching of the graph is provided, this method implements an $\mathcal{O}(|V|\cdot|E|)$ algorithm, where $|V|$ and $|E|$ are the order and the size of the graph respectively. This implementation is inspired by the $M\text{-}alternating\ tree\ search$ method explained in [LZ2001]. For nonbipartite graph, the implementation is based on the following theorem:

Given a nonbipartite graph G with a perfect matching M. The graph G is matching covered if and only if for each edge uv not in M, there exists an $M\text{-}alternating$ odd length uv-path starting and ending with edges not in M.

The time complexity may be dominated by the time needed to compute a maximum matching of the graph, in case a perfect matching is not provided. Also, note that for a disconnected or a trivial or a graph with a loop, a `ValueError` is returned.

INPUT:

- `matching` – (default: `None`); a perfect matching of the graph, that can be given using any valid input format of *Graph*.

  If set to `None`, a matching is computed using the other parameters.

- `algorithm` – string (default: `'Edmonds'`); the algorithm to be used to compute a maximum matching of the graph among

  - `'Edmonds'` selects Edmonds' algorithm as implemented in NetworkX,
  - `'LP'` uses a Linear Program formulation of the matching problem.

- `coNP_certificate` – boolean (default: `False`); if set to `True` an edge of the graph, that does not participate in any perfect matching, is returned if G is not matching covered or otherwise `None` is returned.

- `solver` – string (default: `None`); specify a Mixed Integer Linear Programming (MILP) solver to be used. If set to `None`, the default one is used. For more information on MILP solvers and which default solver is used, see the method `solve` of the class `MixedIntegerLinearProgram`.
- `verbose` – integer (default: `0`); sets the level of verbosity: set to 0 by default, which means quiet (only useful when `algorithm == 'LP'`).
- `integrality_tolerance` – float; parameter for use with MILP solvers over an inexact base ring; see `MixedIntegerLinearProgram.get_values()`.

OUTPUT:

- A boolean indicating whether the graph is matching covered or not.
- If `coNP_certificate` is set to `True`, an edge is returned in case the graph is not matching covered otherwise `None` is returned.

EXAMPLES:

The Petersen graph is matching covered:

```
sage: G = graphs.PetersenGraph()
sage: G.is_matching_covered()
True
```

```
>>> from sage.all import *
>>> G = graphs.PetersenGraph()
>>> G.is_matching_covered()
True
```

A graph (without a self-loop) is matching covered if and only if the underlying simple graph is matching covered:

```
sage: G = graphs.PetersenGraph()
sage: G.allow_multiple_edges(True)
sage: G.add_edge(0, 5)
sage: G.is_matching_covered()
True
```

```
>>> from sage.all import *
>>> G = graphs.PetersenGraph()
>>> G.allow_multiple_edges(True)
>>> G.add_edge(Integer(0), Integer(5))
>>> G.is_matching_covered()
True
```

A corollary to Tutte's fundamental result [Tut1947], as a strengthening of Petersen's Theorem, states that every 2-connected cubic graph is matching covered:

```
sage: G = Graph()
sage: G.add_edges([
....:     (0, 1), (0, 2), (0, 3),
....:     (1, 2), (1, 4), (2, 4),
....:     (3, 5), (3, 6), (4, 7),
....:     (5, 6), (5, 7), (6, 7)
....: ])
sage: G.vertex_connectivity()
```

(continues on next page)

(continued from previous page)

```
2
sage: degree_sequence = G.degree_sequence()
sage: min(degree_sequence) == max(degree_sequence) == 3
True
sage: G.is_matching_covered()
True
```

```
>>> from sage.all import *
>>> G = Graph()
>>> G.add_edges([
...     (Integer(0), Integer(1)), (Integer(0), Integer(2)), (Integer(0),␣
↪Integer(3)),
...     (Integer(1), Integer(2)), (Integer(1), Integer(4)), (Integer(2),␣
↪Integer(4)),
...     (Integer(3), Integer(5)), (Integer(3), Integer(6)), (Integer(4),␣
↪Integer(7)),
...     (Integer(5), Integer(6)), (Integer(5), Integer(7)), (Integer(6),␣
↪Integer(7))
... ])
>>> G.vertex_connectivity()
2
>>> degree_sequence = G.degree_sequence()
>>> min(degree_sequence) == max(degree_sequence) == Integer(3)
True
>>> G.is_matching_covered()
True
```

A connected bipartite graph $G[A, B]$, with $|A| = |B| \geq 2$, is matching covered if and only if $|N(X)| \geq |X| + 1$, for all $X \subset A$ such that $1 \leq |X| \leq |A| - 1$. For instance, the Hexahedral graph is matching covered, but not the path graphs on even number of vertices, even though they have a perfect matching:

```
sage: G = graphs.HexahedralGraph()
sage: G.is_bipartite()
True
sage: G.is_matching_covered()
True
sage: P = graphs.PathGraph(10)
sage: P.is_bipartite()
True
sage: M = Graph(P.matching())
sage: set(P) == set(M)
True
sage: P.is_matching_covered()
False
```

```
>>> from sage.all import *
>>> G = graphs.HexahedralGraph()
>>> G.is_bipartite()
True
>>> G.is_matching_covered()
True
>>> P = graphs.PathGraph(Integer(10))
```

(continues on next page)

(continued from previous page)

```
>>> P.is_bipartite()
True
>>> M = Graph(P.matching())
>>> set(P) == set(M)
True
>>> P.is_matching_covered()
False
```

A connected bipartite graph $G[A, B]$ of order six or more is matching covered if and only if $G - a - b$ has a perfect matching for some vertex a in A and some vertex b in B:

```
sage: G = graphs.CircularLadderGraph(8)
sage: G.is_bipartite()
True
sage: G.is_matching_covered()
True
sage: A, B = G.bipartite_sets()
sage: # needs random
sage: import random
sage: a = random.choice(list(A))
sage: b = random.choice(list(B))
sage: G.delete_vertices([a, b])
sage: M = Graph(G.matching())
sage: set(M) == set(G)
True
sage: cycle1 = graphs.CycleGraph(4)
sage: cycle2 = graphs.CycleGraph(6)
sage: cycle2.relabel(lambda v: v + 4)
sage: H = Graph()
sage: H.add_edges(cycle1.edges() + cycle2.edges())
sage: H.add_edge(3, 4)
sage: H.is_bipartite()
True
sage: H.is_matching_covered()
False
sage: H.delete_vertices([3, 4])
sage: N = Graph(H.matching())
sage: set(N) == set(H)
False
```

```
>>> from sage.all import *
>>> G = graphs.CircularLadderGraph(Integer(8))
>>> G.is_bipartite()
True
>>> G.is_matching_covered()
True
>>> A, B = G.bipartite_sets()
>>> # needs random
>>> import random
>>> a = random.choice(list(A))
>>> b = random.choice(list(B))
>>> G.delete_vertices([a, b])
```

(continues on next page)

(continued from previous page)

```
>>> M = Graph(G.matching())
>>> set(M) == set(G)
True
>>> cycle1 = graphs.CycleGraph(Integer(4))
>>> cycle2 = graphs.CycleGraph(Integer(6))
>>> cycle2.relabel(lambda v: v + Integer(4))
>>> H = Graph()
>>> H.add_edges(cycle1.edges() + cycle2.edges())
>>> H.add_edge(Integer(3), Integer(4))
>>> H.is_bipartite()
True
>>> H.is_matching_covered()
False
>>> H.delete_vertices([Integer(3), Integer(4)])
>>> N = Graph(H.matching())
>>> set(N) == set(H)
False
```

One may specify a matching:

```
sage: G = graphs.WheelGraph(20)
sage: M = Graph(G.matching())
sage: G.is_matching_covered(matching=M)
True
sage: J = graphs.CycleGraph(4)
sage: J.add_edge(0, 2)
sage: N = J.matching()
sage: J.is_matching_covered(matching=N)
False
```

```
>>> from sage.all import *
>>> G = graphs.WheelGraph(Integer(20))
>>> M = Graph(G.matching())
>>> G.is_matching_covered(matching=M)
True
>>> J = graphs.CycleGraph(Integer(4))
>>> J.add_edge(Integer(0), Integer(2))
>>> N = J.matching()
>>> J.is_matching_covered(matching=N)
False
```

One may ask for a co-$\mathcal{NP}$ certificate:

```
sage: G = graphs.CompleteGraph(14)
sage: G.is_matching_covered(coNP_certificate=True)
(True, None)
sage: H = graphs.PathGraph(20)
sage: M = H.matching()
sage: H.is_matching_covered(matching=M, coNP_certificate=True)
(False, (2, 1, None))
```

```
>>> from sage.all import *
>>> G = graphs.CompleteGraph(Integer(14))
>>> G.is_matching_covered(coNP_certificate=True)
(True, None)
>>> H = graphs.PathGraph(Integer(20))
>>> M = H.matching()
>>> H.is_matching_covered(matching=M, coNP_certificate=True)
(False, (2, 1, None))
```

REFERENCES:

- [LM2024]
- [LZ2001]
- [Tut1947]

See also

is_factor_critical(), *is_bicritical()*

AUTHORS:

- Janmenjaya Panda (2024-06-23)

is_module(*vertices*)

Check whether `vertices` is a module of `self`.

A subset M of the vertices of a graph is a module if for every vertex v outside of M, either all vertices of M are neighbors of v or all vertices of M are not neighbors of v.

INPUT:

- `vertices` – iterable; a subset of vertices of `self`

EXAMPLES:

The whole graph, the empty set and singletons are trivial modules:

```
sage: G = graphs.PetersenGraph()
sage: G.is_module([])
True
sage: G.is_module([G.random_vertex()])
True
sage: G.is_module(G)
True
```

```
>>> from sage.all import *
>>> G = graphs.PetersenGraph()
>>> G.is_module([])
True
>>> G.is_module([G.random_vertex()])
True
>>> G.is_module(G)
True
```

Prime graphs only have trivial modules:

```
sage: G = graphs.PathGraph(5)
sage: G.is_prime()
True
sage: all(not G.is_module(S) for S in subsets(G)
....:                         if len(S) > 1 and len(S) < G.order())
True
```

```
>>> from sage.all import *
>>> G = graphs.PathGraph(Integer(5))
>>> G.is_prime()
True
>>> all(not G.is_module(S) for S in subsets(G)
...                         if len(S) > Integer(1) and len(S) < G.order())
True
```

For edgeless graphs and complete graphs, all subsets are modules:

```
sage: G = Graph(5)
sage: all(G.is_module(S) for S in subsets(G))
True
sage: G = graphs.CompleteGraph(5)
sage: all(G.is_module(S) for S in subsets(G))
True
```

```
>>> from sage.all import *
>>> G = Graph(Integer(5))
>>> all(G.is_module(S) for S in subsets(G))
True
>>> G = graphs.CompleteGraph(Integer(5))
>>> all(G.is_module(S) for S in subsets(G))
True
```

The modules of a graph and of its complements are the same:

```
sage: G = graphs.TuranGraph(10, 3)
sage: G.is_module([0,1,2])
True
sage: G.complement().is_module([0,1,2])
True
sage: G.is_module([3,4,5])
True
sage: G.complement().is_module([3,4,5])
True
sage: G.is_module([2,3,4])
False
sage: G.complement().is_module([2,3,4])
False
sage: G.is_module([3,4,5,6,7,8,9])
True
sage: G.complement().is_module([3,4,5,6,7,8,9])
True
```

```
>>> from sage.all import *
>>> G = graphs.TuranGraph(Integer(10), Integer(3))
>>> G.is_module([Integer(0),Integer(1),Integer(2)])
True
>>> G.complement().is_module([Integer(0),Integer(1),Integer(2)])
True
>>> G.is_module([Integer(3),Integer(4),Integer(5)])
True
>>> G.complement().is_module([Integer(3),Integer(4),Integer(5)])
True
>>> G.is_module([Integer(2),Integer(3),Integer(4)])
False
>>> G.complement().is_module([Integer(2),Integer(3),Integer(4)])
False
>>> G.is_module([Integer(3),Integer(4),Integer(5),Integer(6),Integer(7),
↪Integer(8),Integer(9)])
True
>>> G.complement().is_module([Integer(3),Integer(4),Integer(5),Integer(6),
↪Integer(7),Integer(8),Integer(9)])
True
```

Elements of `vertices` must be in `self`:

```
sage: G = graphs.PetersenGraph()
sage: G.is_module(['Terry'])
Traceback (most recent call last):
...
LookupError: vertex (Terry) is not a vertex of the graph
sage: G.is_module([1, 'Graham'])
Traceback (most recent call last):
...
LookupError: vertex (Graham) is not a vertex of the graph
```

```
>>> from sage.all import *
>>> G = graphs.PetersenGraph()
>>> G.is_module(['Terry'])
Traceback (most recent call last):
...
LookupError: vertex (Terry) is not a vertex of the graph
>>> G.is_module([Integer(1), 'Graham'])
Traceback (most recent call last):
...
LookupError: vertex (Graham) is not a vertex of the graph
```

is_odd_hole_free(*certificate=False*)

Test whether `self` contains an induced odd hole.

A Hole is a cycle of length at least 4 (included). It is said to be even (resp. odd) if its length is even (resp. odd).

It is interesting to notice that while it is polynomial to check whether a graph has an odd hole or an odd antihole [CCLSV2005], it is not known whether testing for one of these two cases independently is polynomial too.

INPUT:

- `certificate` – boolean (default: `False`); when `certificate = False`, this method only returns `True` or `False`. If `certificate = True`, the subgraph found is returned instead of `False`.

EXAMPLES:

Is the Petersen Graph odd-hole-free

```
sage: g = graphs.PetersenGraph()
sage: g.is_odd_hole_free()                                                  #␣
↪needs sage.modules
False
```

```
>>> from sage.all import *
>>> g = graphs.PetersenGraph()
>>> g.is_odd_hole_free()                                                  #␣
↪needs sage.modules
False
```

Which was to be expected, as its girth is 5

```
sage: g.girth()
5
```

```
>>> from sage.all import *
>>> g.girth()
5
```

We can check the certificate returned is indeed a 5-cycle:

```
sage: cycle = g.is_odd_hole_free(certificate=True)                          #␣
↪needs sage.modules
sage: cycle.is_isomorphic(graphs.CycleGraph(5))                             #␣
↪needs sage.modules
True
```

```
>>> from sage.all import *
>>> cycle = g.is_odd_hole_free(certificate=True)                          #␣
↪needs sage.modules
>>> cycle.is_isomorphic(graphs.CycleGraph(Integer(5)))                      ␣
↪     # needs sage.modules
True
```

As any chordal graph is hole-free, no interval graph has an odd hole:

```
sage: g = graphs.RandomIntervalGraph(20)
sage: g.is_odd_hole_free()                                                  #␣
↪needs sage.modules
True
```

```
>>> from sage.all import *
>>> g = graphs.RandomIntervalGraph(Integer(20))
>>> g.is_odd_hole_free()                                                  #␣
↪needs sage.modules
True
```

is_overfull()

Test whether the current graph is overfull.

A graph G on n vertices and m edges is said to be overfull if:

- n is odd
- It satisfies $2m > (n-1)\Delta(G)$, where $\Delta(G)$ denotes the maximum degree among all vertices in G.

An overfull graph must have a chromatic index of $\Delta(G)+1$.

EXAMPLES:

A complete graph of order $n > 1$ is overfull if and only if n is odd:

```
sage: graphs.CompleteGraph(6).is_overfull()
False
sage: graphs.CompleteGraph(7).is_overfull()
True
sage: graphs.CompleteGraph(1).is_overfull()
False
```

```
>>> from sage.all import *
>>> graphs.CompleteGraph(Integer(6)).is_overfull()
False
>>> graphs.CompleteGraph(Integer(7)).is_overfull()
True
>>> graphs.CompleteGraph(Integer(1)).is_overfull()
False
```

The claw graph is not overfull:

```
sage: from sage.graphs.graph_coloring import edge_coloring
sage: g = graphs.ClawGraph()
sage: g
Claw graph: Graph on 4 vertices
sage: edge_coloring(g, value_only=True)                                 #␣
↪needs sage.numerical_mip
3
sage: g.is_overfull()
False
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_coloring import edge_coloring
>>> g = graphs.ClawGraph()
>>> g
Claw graph: Graph on 4 vertices
>>> edge_coloring(g, value_only=True)                                 #␣
↪needs sage.numerical_mip
3
>>> g.is_overfull()
False
```

The Holt graph is an example of a overfull graph:

```
sage: G = graphs.HoltGraph()
sage: G.is_overfull()
True
```

```
>>> from sage.all import *
>>> G = graphs.HoltGraph()
>>> G.is_overfull()
True
```

Checking that all complete graphs K_n for even $0 \leq n \leq 100$ are not overfull:

```
sage: def check_overfull_Kn_even(n):
....:     i = 0
....:     while i <= n:
....:         if graphs.CompleteGraph(i).is_overfull():
....:             print("A complete graph of even order cannot be overfull.")
....:             return
....:         i += 2
....:     print("Complete graphs of even order up to %s are not overfull." %␣
↪n)
...
sage: check_overfull_Kn_even(100)  # long time
Complete graphs of even order up to 100 are not overfull.
```

```
>>> from sage.all import *
>>> def check_overfull_Kn_even(n):
...     i = Integer(0)
...     while i <= n:
...         if graphs.CompleteGraph(i).is_overfull():
...             print("A complete graph of even order cannot be overfull.")
...             return
...         i += Integer(2)
...     print("Complete graphs of even order up to %s are not overfull." % n)
...
>>> check_overfull_Kn_even(Integer(100))  # long time
Complete graphs of even order up to 100 are not overfull.
```

The null graph, i.e. the graph with no vertices, is not overfull:

```
sage: Graph().is_overfull()
False
sage: graphs.CompleteGraph(0).is_overfull()
False
```

```
>>> from sage.all import *
>>> Graph().is_overfull()
False
>>> graphs.CompleteGraph(Integer(0)).is_overfull()
False
```

Checking that all complete graphs K_n for odd $1 < n \leq 100$ are overfull:

```
sage: def check_overfull_Kn_odd(n):
....:     i = 3
....:     while i <= n:
....:         if not graphs.CompleteGraph(i).is_overfull():
....:             print("A complete graph of odd order > 1 must be overfull.")
....:             return
....:         i += 2
....:     print("Complete graphs of odd order > 1 up to %s are overfull." % n)
...
sage: check_overfull_Kn_odd(100)  # long time
Complete graphs of odd order > 1 up to 100 are overfull.
```

```
>>> from sage.all import *
>>> def check_overfull_Kn_odd(n):
...     i = Integer(3)
...     while i <= n:
...         if not graphs.CompleteGraph(i).is_overfull():
...             print("A complete graph of odd order > 1 must be overfull.")
...             return
...         i += Integer(2)
...     print("Complete graphs of odd order > 1 up to %s are overfull." % n)
...
>>> check_overfull_Kn_odd(Integer(100))  # long time
Complete graphs of odd order > 1 up to 100 are overfull.
```

The Petersen Graph, though, is not overfull while its chromatic index is $\Delta + 1$:

```
sage: g = graphs.PetersenGraph()
sage: g.is_overfull()
False
sage: from sage.graphs.graph_coloring import edge_coloring
sage: max(g.degree()) + 1 ==  edge_coloring(g, value_only=True)             #␣
↪needs sage.numerical_mip
True
```

```
>>> from sage.all import *
>>> g = graphs.PetersenGraph()
>>> g.is_overfull()
False
>>> from sage.graphs.graph_coloring import edge_coloring
>>> max(g.degree()) + Integer(1) ==  edge_coloring(g, value_only=True)       ␣
↪     # needs sage.numerical_mip
True
```

is_partial_cube(*G*, *certificate=False*)

Test whether the given graph is a partial cube.

A partial cube is a graph that can be isometrically embedded into a hypercube, i.e., its vertices can be labelled with (0,1)-vectors of some fixed length such that the distance between any two vertices in the graph equals the Hamming distance of their labels.

Originally written by D. Eppstein for the PADS library (http://www.ics.uci.edu/~eppstein/PADS/), see also [Epp2008]. The algorithm runs in $O(n^2)$ time, where n is the number of vertices. See the documentation of *partial_cube* for an overview of the algorithm.

INPUT:

- `certificate` – boolean (default: `False`); this function returns `True` or `False` according to the graph, when `certificate = False`. When `certificate = True` and the graph is a partial cube, the function returns `(True, mapping)`, where `mapping` is an isometric mapping of the vertices of the graph to the vertices of a hypercube ((0, 1)-strings of a fixed length). When `certificate = True` and the graph is not a partial cube, `(False, None)` is returned.

EXAMPLES:

The Petersen graph is not a partial cube:

```
sage: g = graphs.PetersenGraph()
sage: g.is_partial_cube()
False
```

```
>>> from sage.all import *
>>> g = graphs.PetersenGraph()
>>> g.is_partial_cube()
False
```

All prisms are partial cubes:

```
sage: g = graphs.CycleGraph(10).cartesian_product(graphs.CompleteGraph(2))
sage: g.is_partial_cube()
True
```

```
>>> from sage.all import *
>>> g = graphs.CycleGraph(Integer(10)).cartesian_product(graphs.
↪CompleteGraph(Integer(2)))
>>> g.is_partial_cube()
True
```

is_path()

Check whether `self` is a path.

A connected graph of order $n \geq 2$ is a path if it is a tree (see *is_tree()*) with $n - 2$ vertices of degree 2 and two of degree 1. By convention, a graph of order 1 without loops is a path, but the empty graph is not a path.

EXAMPLES:

sage: G = graphs.PathGraph(5) sage: G.is_path() True sage: H = graphs.CycleGraph(5) sage: H.is_path() False sage: D = graphs.PathGraph(5).disjoint_union(graphs.CycleGraph(5)) sage: D.is_path() False sage: E = graphs.EmptyGraph() sage: E.is_path() False sage: O = Graph([[1], []]) sage: O.is_path() True sage: O.allow_loops(True) sage: O.add_edge(1, 1) sage: O.is_path() False

is_perfect(*certificate=False*)

Test whether the graph is perfect.

A graph G is said to be perfect if $\chi(H) = \omega(H)$ hold for any induced subgraph $H \subseteq_i G$ (and so for G itself, too), where $\chi(H)$ represents the chromatic number of H, and $\omega(H)$ its clique number. The Strong Perfect Graph Theorem [CRST2006] gives another characterization of perfect graphs:

A graph is perfect if and only if it contains no odd hole (cycle on an odd number k of vertices, $k > 3$) nor any odd antihole (complement of a hole) as an induced subgraph.

INPUT:

- `certificate` – boolean (default: `False`); whether to return a certificate

OUTPUT:

When `certificate = False`, this function returns a boolean value. When `certificate = True`, it returns a subgraph of `self` isomorphic to an odd hole or an odd antihole if any, and `None` otherwise.

EXAMPLES:

A Bipartite Graph is always perfect

```
sage: g = graphs.RandomBipartite(8,4,.5)                                    #␣
↪needs numpy
sage: g.is_perfect()                                                        #␣
↪needs numpy sage.modules
True
```

```
>>> from sage.all import *
>>> g = graphs.RandomBipartite(Integer(8),Integer(4),RealNumber('.5'))      ␣
↪                              # needs numpy
>>> g.is_perfect()                                                          #␣
↪needs numpy sage.modules
True
```

So is the line graph of a bipartite graph:

```
sage: g = graphs.RandomBipartite(4,3,0.7)                                   #␣
↪needs numpy
sage: g.line_graph().is_perfect()    # long time                            #␣
↪needs numpy sage.modules
True
```

```
>>> from sage.all import *
>>> g = graphs.RandomBipartite(Integer(4),Integer(3),RealNumber('0.7'))     ␣
↪                              # needs numpy
>>> g.line_graph().is_perfect()    # long time                              #␣
↪needs numpy sage.modules
True
```

As well as the Cartesian product of two complete graphs:

```
sage: g = graphs.CompleteGraph(3).cartesian_product(graphs.CompleteGraph(3))
sage: g.is_perfect()                                                        #␣
↪needs sage.modules
True
```

```
>>> from sage.all import *
>>> g = graphs.CompleteGraph(Integer(3)).cartesian_product(graphs.
↪CompleteGraph(Integer(3)))
>>> g.is_perfect()                                                          #␣
↪needs sage.modules
True
```

Interval Graphs, which are chordal graphs, too

```
sage: g =  graphs.RandomIntervalGraph(7)
sage: g.is_perfect()                                                    #␣
↪needs sage.modules
True
```

```
>>> from sage.all import *
>>> g =  graphs.RandomIntervalGraph(Integer(7))
>>> g.is_perfect()                                                      #␣
↪needs sage.modules
True
```

The PetersenGraph, which is triangle-free and has chromatic number 3 is obviously not perfect:

```
sage: g = graphs.PetersenGraph()
sage: g.is_perfect()                                                    #␣
↪needs sage.modules
False
```

```
>>> from sage.all import *
>>> g = graphs.PetersenGraph()
>>> g.is_perfect()                                                      #␣
↪needs sage.modules
False
```

We can obtain an induced 5-cycle as a certificate:

```
sage: g.is_perfect(certificate=True)                                    #␣
↪needs sage.modules
Subgraph of (Petersen graph): Graph on 5 vertices
```

```
>>> from sage.all import *
>>> g.is_perfect(certificate=True)                                      #␣
↪needs sage.modules
Subgraph of (Petersen graph): Graph on 5 vertices
```

is_permutation(*g*, *algorithm='greedy'*, *certificate=False*, *check=True*, *solver=None*, *verbose=0*)

Check whether the graph is a permutation graph.

For more information on permutation graphs, refer to the documentation of the *comparability module*.

INPUT:

- `g` – a graph
- `algorithm` – string (default: `'greedy'`); choose the implementation used for the subcalls to *is_comparability()*
  - `'greedy'` – a greedy algorithm (see the documentation of the *comparability module*)
  - `'MILP'` – a Mixed Integer Linear Program formulation of the problem. Beware, for this implementation is unable to return negative certificates ! When `certificate = True`, negative certificates are always equal to `None`. `True` certificates are valid, though.
- `certificate` – boolean (default: `False`); whether to return a certificate for the answer given. For `True` answers the certificate is a permutation, for `False` answers it is a no-certificate for the test of comparability or co-comparability.

- `check` – boolean (default: `True`); whether to check that the permutations returned indeed create the expected Permutation graph. Pretty cheap compared to the rest, hence a good investment. It is enabled by default.
- `solver` – string (default: `None`); specifies a Mixed Integer Linear Programming (MILP) solver to be used. If set to `None`, the default one is used. For more information on MILP solvers and which default solver is used, see the method `solve` of the class `MixedIntegerLinearProgram`.
- `verbose` – integer (default: 0); sets the level of verbosity. Set to 0 by default, which means quiet.

Note

As the `True` certificate is a `Permutation` object, the segment intersection model of the permutation graph can be visualized through a call to `Permutation.show`.

EXAMPLES:

A permutation realizing the bull graph:

```
sage: from sage.graphs.comparability import is_permutation
sage: g = graphs.BullGraph()
sage: _ , certif = is_permutation(g, certificate=True)
sage: h = graphs.PermutationGraph(*certif)
sage: h.is_isomorphic(g)
True
```

```
>>> from sage.all import *
>>> from sage.graphs.comparability import is_permutation
>>> g = graphs.BullGraph()
>>> _ , certif = is_permutation(g, certificate=True)
>>> h = graphs.PermutationGraph(*certif)
>>> h.is_isomorphic(g)
True
```

Plotting the realization as an intersection graph of segments:

```
sage: true, perm = is_permutation(g, certificate=True)
sage: p1 = Permutation([nn+1 for nn in perm[0]])
sage: p2 = Permutation([nn+1 for nn in perm[1]])
sage: p = p2 * p1.inverse()
sage: p.show(representation='braid')
 ↪  # needs sage.plot
```

```
>>> from sage.all import *
>>> true, perm = is_permutation(g, certificate=True)
>>> p1 = Permutation([nn+Integer(1) for nn in perm[Integer(0)]])
>>> p2 = Permutation([nn+Integer(1) for nn in perm[Integer(1)]])
>>> p = p2 * p1.inverse()
>>> p.show(representation='braid')
↪# needs sage.plot
```

is_polyhedral()

Check whether the graph is the graph of the polyhedron.

By a theorem of Steinitz (Satz 43, p. 77 of [St1922]), graphs of three-dimensional polyhedra are exactly the simple 3-vertex-connected planar graphs.

EXAMPLES:

```
sage: C = graphs.CubeGraph(3)
sage: C.is_polyhedral()
True
sage: K33=graphs.CompleteBipartiteGraph(3, 3)
sage: K33.is_polyhedral()
False
sage: graphs.CycleGraph(17).is_polyhedral()
False
sage: [i for i in range(9) if graphs.CompleteGraph(i).is_polyhedral()]
[4]
```

```
>>> from sage.all import *
>>> C = graphs.CubeGraph(Integer(3))
>>> C.is_polyhedral()
True
>>> K33=graphs.CompleteBipartiteGraph(Integer(3), Integer(3))
>>> K33.is_polyhedral()
False
>>> graphs.CycleGraph(Integer(17)).is_polyhedral()
False
>>> [i for i in range(Integer(9)) if graphs.CompleteGraph(i).is_polyhedral()]
[4]
```

See also

- *vertex_connectivity()*
- *is_planar()*
- *is_circumscribable()*
- *is_inscribable()*
- Wikipedia article Polyhedral_graph

is_prime(*algorithm=None*)

Test whether the current graph is prime.

A graph is prime if all its modules are trivial (i.e. empty, all of the graph or singletons) – see *modular_decomposition()*. This method computes the modular decomposition tree using *modular_decomposition()*.

INPUT:

- `algorithm` – string (default: `None`); the algorithm used to compute the modular decomposition tree; the value is forwarded directly to *modular_decomposition()*.

EXAMPLES:

The Petersen Graph and the Bull Graph are both prime:

```
sage: graphs.PetersenGraph().is_prime()
True
sage: graphs.BullGraph().is_prime()
True
```

```
>>> from sage.all import *
>>> graphs.PetersenGraph().is_prime()
True
>>> graphs.BullGraph().is_prime()
True
```

Though quite obviously, the disjoint union of them is not:

```
sage: (graphs.PetersenGraph() + graphs.BullGraph()).is_prime()
False
```

```
>>> from sage.all import *
>>> (graphs.PetersenGraph() + graphs.BullGraph()).is_prime()
False
```

is_projective_planar(*return_map=False*)

Check whether `self` is projective planar.

A graph is projective planar if it can be embedded in the projective plane. The approach is to check that the graph does not contain any of the known forbidden minors.

INPUT:

- `return_map` – boolean (default: `False`); whether to return a map indicating one of the forbidden graph minors if in fact the graph is not projective planar, or only `True`/`False`.

OUTPUT:

Return `True` if the graph is projective planar and `False` if not. If the parameter `return_map` is `True` and the graph is not projective planar, then the method returns `False` and a map from *minor()* indicating one of the forbidden graph minors.

EXAMPLES:

The Petersen graph is a known projective planar graph:

```
sage: P = graphs.PetersenGraph()
sage: P.is_projective_planar()
True
```

```
>>> from sage.all import *
>>> P = graphs.PetersenGraph()
>>> P.is_projective_planar()
True
```

$K_{4,4}$ has a projective plane crossing number of 2. One of the minimal forbidden minors is $K_{4,4} - e$, so we get a one-to-one dictionary from *minor()*:

```
sage: K44 = graphs.CompleteBipartiteGraph(4, 4)
sage: is_planar, minor_map = K44.is_projective_planar(return_map=True)
sage: is_planar
```

(continues on next page)

(continued from previous page)

```
False
sage: sorted(tuple(v) for v in minor_map.values())
[(0,), (1,), (2,), (3,), (4,), (5,), (6,), (7,)]
```

```
>>> from sage.all import *
>>> K44 = graphs.CompleteBipartiteGraph(Integer(4), Integer(4))
>>> is_planar, minor_map = K44.is_projective_planar(return_map=True)
>>> is_planar
False
>>> sorted(tuple(v) for v in minor_map.values())
[(0,), (1,), (2,), (3,), (4,), (5,), (6,), (7,)]
```

See also

- *minor()*

is_redundant(*G*, *dom*, *focus=None*)

Check whether `dom` has redundant vertices.

For a graph G and sets D and S of vertices, we say that a vertex $v \in D$ is *redundant* in S if v has no private neighbor with respect to D in S. In other words, there is no vertex in S that is dominated by v but not by $D \setminus \{v\}$.

INPUT:

- `dom` – iterable of vertices of `G`; where we look for redundant vertices
- `focus` – iterable of vertices of `G` (default: `None`); if specified, this method checks instead whether `dom` has a redundant vertex in `focus`

Warning

The assumption is made that `focus` (if provided) does not contain repeated vertices.

EXAMPLES:

```
sage: G = graphs.CubeGraph(3)
sage: G.is_redundant(['000', '101'], ['011'])
True
sage: G.is_redundant(['000', '101'])
False
```

```
>>> from sage.all import *
>>> G = graphs.CubeGraph(Integer(3))
>>> G.is_redundant(['000', '101'], ['011'])
True
>>> G.is_redundant(['000', '101'])
False
```

is_semi_symmetric()

Check if `self` is semi-symmetric.

A graph is semi-symmetric if it is regular, edge-transitive but not vertex-transitive.

See also

- Wikipedia article Semi-symmetric_graph
- *is_edge_transitive()*
- *is_arc_transitive()*
- *is_half_transitive()*

EXAMPLES:

The Petersen graph is not semi-symmetric:

```
sage: P = graphs.PetersenGraph()
sage: P.is_semi_symmetric()                                                 #␣
↪needs sage.libs.gap
False
```

```
>>> from sage.all import *
>>> P = graphs.PetersenGraph()
>>> P.is_semi_symmetric()                                                   #␣
↪needs sage.libs.gap
False
```

The Gray graph is the smallest possible cubic semi-symmetric graph:

```
sage: G = graphs.GrayGraph()
sage: G.is_semi_symmetric()                                                 #␣
↪needs sage.libs.gap
True
```

```
>>> from sage.all import *
>>> G = graphs.GrayGraph()
>>> G.is_semi_symmetric()                                                   #␣
↪needs sage.libs.gap
True
```

Another well known semi-symmetric graph is the Ljubljana graph:

```
sage: L = graphs.LjubljanaGraph()
sage: L.is_semi_symmetric()                                                 #␣
↪needs sage.libs.gap
True
```

```
>>> from sage.all import *
>>> L = graphs.LjubljanaGraph()
>>> L.is_semi_symmetric()                                                   #␣
↪needs sage.libs.gap
True
```

is_split()

Return `True` if the graph is a Split graph, `False` otherwise.

A Graph G is said to be a split graph if its vertices $V(G)$ can be partitioned into two sets K and I such that the vertices of K induce a complete graph, and those of I are an independent set.

There is a simple test to check whether a graph is a split graph (see, for instance, the book "Graph Classes, a survey" [BLS1999] page 203) :

Given the degree sequence $d_1 \geq ... \geq d_n$ of G, a graph is a split graph if and only if :

$$\sum_{i=1}^{\omega} d_i = \omega(\omega - 1) + \sum_{i=\omega+1}^{n} d_i$$

where $\omega = max\{i : d_i \geq i - 1\}$.

EXAMPLES:

Split graphs are, in particular, chordal graphs. Hence, The Petersen graph can not be split:

```
sage: graphs.PetersenGraph().is_split()
False
```

```
>>> from sage.all import *
>>> graphs.PetersenGraph().is_split()
False
```

We can easily build some "random" split graph by creating a complete graph, and adding vertices only connected to some random vertices of the clique:

```
sage: g = graphs.CompleteGraph(10)
sage: sets = Subsets(Set(range(10)))
sage: for i in range(10, 25):
....:     g.add_edges([(i,k) for k in sets.random_element()])
sage: g.is_split()
True
```

```
>>> from sage.all import *
>>> g = graphs.CompleteGraph(Integer(10))
>>> sets = Subsets(Set(range(Integer(10))))
>>> for i in range(Integer(10), Integer(25)):
...     g.add_edges([(i,k) for k in sets.random_element()])
>>> g.is_split()
True
```

Another characterisation of split graph states that a graph is a split graph if and only if does not contain the 4-cycle, 5-cycle or $2K_2$ as an induced subgraph. Hence for the above graph we have:

```
sage: forbidden_subgraphs = [graphs.CycleGraph(4),
....:                        graphs.CycleGraph(5),
....:                        2 * graphs.CompleteGraph(2)]
sage: sum(g.subgraph_search_count(H, induced=True)                    #␣
↪needs sage.modules
....:     for H in forbidden_subgraphs)
0
```

```
>>> from sage.all import *
>>> forbidden_subgraphs = [graphs.CycleGraph(Integer(4)),
...                        graphs.CycleGraph(Integer(5)),
...                        Integer(2) * graphs.CompleteGraph(Integer(2))]
>>> sum(g.subgraph_search_count(H, induced=True)                              #␣
↪needs sage.modules
...     for H in forbidden_subgraphs)
0
```

is_strongly_regular(*g*, *parameters=False*)

Check whether the graph is strongly regular.

A simple graph G is said to be strongly regular with parameters (n, k, λ, μ) if and only if:

- G has n vertices
- G is k-regular
- Any two adjacent vertices of G have λ common neighbors
- Any two non-adjacent vertices of G have μ common neighbors

By convention, the complete graphs, the graphs with no edges and the empty graph are not strongly regular.

See the Wikipedia article Strongly regular graph.

INPUT:

- `parameters` – boolean (default: `False`); whether to return the quadruple (n, k, λ, μ). If `parameters = False` (default), this method only returns `True` and `False` answers. If `parameters = True`, the `True` answers are replaced by quadruples (n, k, λ, μ). See definition above.

EXAMPLES:

Petersen's graph is strongly regular:

```
sage: g = graphs.PetersenGraph()
sage: g.is_strongly_regular()
True
sage: g.is_strongly_regular(parameters=True)
(10, 3, 0, 1)
```

```
>>> from sage.all import *
>>> g = graphs.PetersenGraph()
>>> g.is_strongly_regular()
True
>>> g.is_strongly_regular(parameters=True)
(10, 3, 0, 1)
```

And Clebsch's graph is too:

```
sage: g = graphs.ClebschGraph()
sage: g.is_strongly_regular()
True
sage: g.is_strongly_regular(parameters=True)
(16, 5, 0, 2)
```

```
>>> from sage.all import *
>>> g = graphs.ClebschGraph()
>>> g.is_strongly_regular()
True
>>> g.is_strongly_regular(parameters=True)
(16, 5, 0, 2)
```

But Chvatal's graph is not:

```
sage: g = graphs.ChvatalGraph()
sage: g.is_strongly_regular()
False
```

```
>>> from sage.all import *
>>> g = graphs.ChvatalGraph()
>>> g.is_strongly_regular()
False
```

Complete graphs are not strongly regular. (Issue #14297)

```
sage: g = graphs.CompleteGraph(5)
sage: g.is_strongly_regular()
False
```

```
>>> from sage.all import *
>>> g = graphs.CompleteGraph(Integer(5))
>>> g.is_strongly_regular()
False
```

Completements of complete graphs are not strongly regular:

```
sage: g = graphs.CompleteGraph(5).complement()
sage: g.is_strongly_regular()
False
```

```
>>> from sage.all import *
>>> g = graphs.CompleteGraph(Integer(5)).complement()
>>> g.is_strongly_regular()
False
```

The empty graph is not strongly regular:

```
sage: g = graphs.EmptyGraph()
sage: g.is_strongly_regular()
False
```

```
>>> from sage.all import *
>>> g = graphs.EmptyGraph()
>>> g.is_strongly_regular()
False
```

If the input graph has loops or multiedges an exception is raised:

```
sage: Graph([(1,1),(2,2)],loops=True).is_strongly_regular()
Traceback (most recent call last):
...
ValueError: This method is not known to work on graphs with
loops. Perhaps this method can be updated to handle them, but in the
meantime if you want to use it please disallow loops using
allow_loops().

sage: Graph([(1,2),(1,2)],multiedges=True).is_strongly_regular()
Traceback (most recent call last):
...
ValueError: This method is not known to work on graphs with
multiedges. Perhaps this method can be updated to handle them, but in
the meantime if you want to use it please disallow multiedges using
allow_multiple_edges().
```

```
>>> from sage.all import *
>>> Graph([(Integer(1),Integer(1)),(Integer(2),Integer(2))],loops=True).is_
↪strongly_regular()
Traceback (most recent call last):
...
ValueError: This method is not known to work on graphs with
loops. Perhaps this method can be updated to handle them, but in the
meantime if you want to use it please disallow loops using
allow_loops().

>>> Graph([(Integer(1),Integer(2)),(Integer(1),Integer(2))],multiedges=True).
↪is_strongly_regular()
Traceback (most recent call last):
...
ValueError: This method is not known to work on graphs with
multiedges. Perhaps this method can be updated to handle them, but in
the meantime if you want to use it please disallow multiedges using
allow_multiple_edges().
```

is_tree(*certificate=False*, *output='vertex'*)

Test if the graph is a tree.

The empty graph is defined to be not a tree.

INPUT:

- `certificate` – boolean (default: `False`); whether to return a certificate. The method only returns boolean answers when `certificate = False` (default). When it is set to `True`, it either answers `(True, None)` when the graph is a tree or `(False, cycle)` when it contains a cycle. It returns `(False, None)` when the graph is empty or not connected.
- `output` – either `'vertex'` (default) or `'edge'`; whether the certificate is given as a list of vertices (`output = 'vertex'`) or a list of edges (`output = 'edge'`).

When the certificate cycle is given as a list of edges, the edges are given as (v_i, v_{i+1}, l) where $v_1, v_2, \ldots, v_n$ are the vertices of the cycles (in their cyclic order).

EXAMPLES:

```
sage: all(T.is_tree() for T in graphs.trees(15))
True
```

```
>>> from sage.all import *
>>> all(T.is_tree() for T in graphs.trees(Integer(15)))
True
```

With certificates:

```
sage: g = graphs.RandomTree(30)
sage: g.is_tree(certificate=True)
(True, None)
sage: g.add_edge(10,-1)
sage: g.add_edge(11,-1)
sage: isit, cycle = g.is_tree(certificate=True)
sage: isit
False
sage: -1 in cycle
True
```

```
>>> from sage.all import *
>>> g = graphs.RandomTree(Integer(30))
>>> g.is_tree(certificate=True)
(True, None)
>>> g.add_edge(Integer(10),-Integer(1))
>>> g.add_edge(Integer(11),-Integer(1))
>>> isit, cycle = g.is_tree(certificate=True)
>>> isit
False
>>> -Integer(1) in cycle
True
```

One can also ask for the certificate as a list of edges:

```
sage: g = graphs.CycleGraph(4)
sage: g.is_tree(certificate=True, output='edge')
(False, [(3, 2, None), (2, 1, None), (1, 0, None), (0, 3, None)])
```

```
>>> from sage.all import *
>>> g = graphs.CycleGraph(Integer(4))
>>> g.is_tree(certificate=True, output='edge')
(False, [(3, 2, None), (2, 1, None), (1, 0, None), (0, 3, None)])
```

This is useful for graphs with multiple edges:

```
sage: G = Graph([(1, 2, 'a'), (1, 2, 'b')], multiedges=True)
sage: G.is_tree(certificate=True)
(False, [1, 2])
sage: G.is_tree(certificate=True, output='edge')
(False, [(1, 2, 'b'), (2, 1, 'a')])
```

```
>>> from sage.all import *
>>> G = Graph([(Integer(1), Integer(2), 'a'), (Integer(1), Integer(2), 'b')],␣
↪multiedges=True)
>>> G.is_tree(certificate=True)
(False, [1, 2])
>>> G.is_tree(certificate=True, output='edge')
(False, [(1, 2, 'b'), (2, 1, 'a')])
```

is_triangle_free(*algorithm='dense_graph'*, *certificate=False*)

Check whether `self` is triangle-free.

INPUT:

- `algorithm` – (default: `'dense_graph'`) specifies the algorithm to use among:
  - `'matrix'` – tests if the trace of the adjacency matrix is positive
  - `'bitset'` – encodes adjacencies into bitsets and uses fast bitset operations to test if the input graph contains a triangle. This method is generally faster than standard matrix multiplication.
  - `'dense_graph'` – use the implementation of *sage.graphs.base.static_dense_graph*
- `certificate` – boolean (default: `False`); whether to return a triangle if one is found. This parameter is ignored when `algorithm` is `'matrix'`.

EXAMPLES:

The Petersen Graph is triangle-free:

```
sage: g = graphs.PetersenGraph()
sage: g.is_triangle_free()
True
```

```
>>> from sage.all import *
>>> g = graphs.PetersenGraph()
>>> g.is_triangle_free()
True
```

or a complete Bipartite Graph:

```
sage: G = graphs.CompleteBipartiteGraph(5,6)
sage: G.is_triangle_free(algorithm='matrix')                                #␣
↪needs sage.modules
True
sage: G.is_triangle_free(algorithm='bitset')
True
sage: G.is_triangle_free(algorithm='dense_graph')
True
```

```
>>> from sage.all import *
>>> G = graphs.CompleteBipartiteGraph(Integer(5),Integer(6))
>>> G.is_triangle_free(algorithm='matrix')                                  #␣
↪needs sage.modules
True
>>> G.is_triangle_free(algorithm='bitset')
True
```

(continues on next page)

(continued from previous page)

```
>>> G.is_triangle_free(algorithm='dense_graph')
True
```

a tripartite graph, though, contains many triangles:

```
sage: G = (3 * graphs.CompleteGraph(5)).complement()
sage: G.is_triangle_free(algorithm='matrix')                                #␣
↪needs sage.modules
False
sage: G.is_triangle_free(algorithm='bitset')
False
sage: G.is_triangle_free(algorithm='dense_graph')
False
```

```
>>> from sage.all import *
>>> G = (Integer(3) * graphs.CompleteGraph(Integer(5))).complement()
>>> G.is_triangle_free(algorithm='matrix')                                  #␣
↪needs sage.modules
False
>>> G.is_triangle_free(algorithm='bitset')
False
>>> G.is_triangle_free(algorithm='dense_graph')
False
```

Asking for a certificate:

```
sage: K4 = graphs.CompleteGraph(4)
sage: K4.is_triangle_free(algorithm='dense_graph', certificate=True)
(False, [0, 1, 2])
sage: K4.is_triangle_free(algorithm='bitset', certificate=True)
(False, [0, 1, 2])
```

```
>>> from sage.all import *
>>> K4 = graphs.CompleteGraph(Integer(4))
>>> K4.is_triangle_free(algorithm='dense_graph', certificate=True)
(False, [0, 1, 2])
>>> K4.is_triangle_free(algorithm='bitset', certificate=True)
(False, [0, 1, 2])
```

is_triconnected(G)

Check whether the graph is triconnected.

A triconnected graph is a connected graph on 3 or more vertices that is not broken into disconnected pieces by deleting any pair of vertices.

EXAMPLES:

The Petersen graph is triconnected:

```
sage: G = graphs.PetersenGraph()
sage: G.is_triconnected()
True
```

```
>>> from sage.all import *
>>> G = graphs.PetersenGraph()
>>> G.is_triconnected()
True
```

But a 2D grid is not:

```
sage: G = graphs.Grid2dGraph(3, 3)
sage: G.is_triconnected()
False
```

```
>>> from sage.all import *
>>> G = graphs.Grid2dGraph(Integer(3), Integer(3))
>>> G.is_triconnected()
False
```

By convention, a cycle of order 3 is triconnected:

```
sage: G = graphs.CycleGraph(3)
sage: G.is_triconnected()
True
```

```
>>> from sage.all import *
>>> G = graphs.CycleGraph(Integer(3))
>>> G.is_triconnected()
True
```

But cycles of order 4 and more are not:

```
sage: [graphs.CycleGraph(i).is_triconnected() for i in range(4, 8)]
[False, False, False, False]
```

```
>>> from sage.all import *
>>> [graphs.CycleGraph(i).is_triconnected() for i in range(Integer(4),␣
↪Integer(8))]
[False, False, False, False]
```

Comparing different methods on random graphs that are not always triconnected:

```
sage: G = graphs.RandomBarabasiAlbert(50, 3)                                    ␣
↪  # needs networkx
sage: G.is_triconnected() == G.vertex_connectivity(k=3)                         ␣
↪  # needs networkx
True
```

```
>>> from sage.all import *
>>> G = graphs.RandomBarabasiAlbert(Integer(50), Integer(3))                    ␣
↪                  # needs networkx
>>> G.is_triconnected() == G.vertex_connectivity(k=Integer(3))                  ␣
↪          # needs networkx
True
```

See also

- *is_connected()*
- *is_biconnected()*
- *spqr_tree()*
- Wikipedia article SPQR_tree

is_weakly_chordal(*g*, *certificate=False*)

Test whether the given graph is weakly chordal, i.e., the graph and its complement have no induced cycle of length at least 5.

INPUT:

- `certificate` – boolean (default: `False`); whether to return a certificate. If `certificate = False`, return `True` or `False` according to the graph. If `certificate = True`, return
  - `(False, forbidden_subgraph)` when the graph contains a forbidden subgraph H, this graph is returned.
  - **`(True, [])` when the graph is weakly chordal.**
    For this case, it is not known how to provide a certificate.

ALGORITHM:

This algorithm checks whether the graph `g` or its complement contain an induced cycle of length at least 5.

Using is_long_hole_free() and is_long_antihole_free() yields a run time of $O(n + m^2)$ for `SparseGraph` and $O(n^2 + m^2)$ for `DenseGraph` (where n is the number of vertices and m is the number of edges of the graph).

EXAMPLES:

The Petersen Graph is not weakly chordal and contains a hole:

```
sage: g = graphs.PetersenGraph()
sage: r,s = g.is_weakly_chordal(certificate=True)
sage: r
False
sage: l = s.order()
sage: s.is_isomorphic(graphs.CycleGraph(l))
True
```

```
>>> from sage.all import *
>>> g = graphs.PetersenGraph()
>>> r,s = g.is_weakly_chordal(certificate=True)
>>> r
False
>>> l = s.order()
>>> s.is_isomorphic(graphs.CycleGraph(l))
True
```

join(*other*, *labels='pairs'*, *immutable=None*)

Return the join of `self` and `other`.

INPUT:

- `labels` – (defaults to 'pairs'); if set to 'pairs', each element v in the first graph will be named $(0, v)$ and each element u in `other` will be named $(1, u)$ in the result. If set to 'integers', the elements of the result will be relabeled with consecutive integers.
- `immutable` – boolean (default: `None`); whether to create a mutable/immutable join. `immutable=None` (default) means that the graphs and their join will behave the same way.

See also

- *union()*
- *disjoint_union()*

EXAMPLES:

```
sage: G = graphs.CycleGraph(3)
sage: H = Graph(2)
sage: J = G.join(H); J
Cycle graph join : Graph on 5 vertices
sage: J.vertices(sort=True)
[(0, 0), (0, 1), (0, 2), (1, 0), (1, 1)]
sage: J = G.join(H, labels='integers'); J
Cycle graph join : Graph on 5 vertices
sage: J.vertices(sort=True)
[0, 1, 2, 3, 4]
sage: J.edges(sort=True)
[(0, 1, None), (0, 2, None), (0, 3, None), (0, 4, None), (1, 2, None), (1, 3,␣
↪None), (1, 4, None), (2, 3, None), (2, 4, None)]
```

```
>>> from sage.all import *
>>> G = graphs.CycleGraph(Integer(3))
>>> H = Graph(Integer(2))
>>> J = G.join(H); J
Cycle graph join : Graph on 5 vertices
>>> J.vertices(sort=True)
[(0, 0), (0, 1), (0, 2), (1, 0), (1, 1)]
>>> J = G.join(H, labels='integers'); J
Cycle graph join : Graph on 5 vertices
>>> J.vertices(sort=True)
[0, 1, 2, 3, 4]
>>> J.edges(sort=True)
[(0, 1, None), (0, 2, None), (0, 3, None), (0, 4, None), (1, 2, None), (1, 3,␣
↪None), (1, 4, None), (2, 3, None), (2, 4, None)]
```

```
sage: G = Graph(3)
sage: G.name("Graph on 3 vertices")
sage: H = Graph(2)
sage: H.name("Graph on 2 vertices")
sage: J = G.join(H); J
Graph on 3 vertices join Graph on 2 vertices: Graph on 5 vertices
sage: J.vertices(sort=True)
[(0, 0), (0, 1), (0, 2), (1, 0), (1, 1)]
sage: J = G.join(H, labels='integers'); J
```

(continues on next page)

(continued from previous page)

```
Graph on 3 vertices join Graph on 2 vertices: Graph on 5 vertices
sage: J.edges(sort=True)
[(0, 3, None), (0, 4, None), (1, 3, None), (1, 4, None), (2, 3, None), (2, 4,␣
↪None)]
```

```
>>> from sage.all import *
>>> G = Graph(Integer(3))
>>> G.name("Graph on 3 vertices")
>>> H = Graph(Integer(2))
>>> H.name("Graph on 2 vertices")
>>> J = G.join(H); J
Graph on 3 vertices join Graph on 2 vertices: Graph on 5 vertices
>>> J.vertices(sort=True)
[(0, 0), (0, 1), (0, 2), (1, 0), (1, 1)]
>>> J = G.join(H, labels='integers'); J
Graph on 3 vertices join Graph on 2 vertices: Graph on 5 vertices
>>> J.edges(sort=True)
[(0, 3, None), (0, 4, None), (1, 3, None), (1, 4, None), (2, 3, None), (2, 4,␣
↪None)]
```

kirchhoff_symanzik_polynomial(*name='t'*)

Return the Kirchhoff-Symanzik polynomial of a graph.

This is a polynomial in variables t_e (each of them representing an edge of the graph G) defined as a sum over all spanning trees:

$$\Psi_G(t) = \sum_{\substack{T \subseteq V \\ \text{a spanning tree}}} \prod_{e \notin E(T)} t_e$$

This is also called the first Symanzik polynomial or the Kirchhoff polynomial.

INPUT:

- `name` – name of the variables (default: `'t'`)

OUTPUT: a polynomial with integer coefficients

ALGORITHM:

This is computed here using a determinant, as explained in Section 3.1 of [Mar2009a].

As an intermediate step, one computes a cycle basis $\mathcal{C}$ of G and a rectangular $|\mathcal{C}| \times |E(G)|$ matrix with entries in $\{-1, 0, 1\}$, which describes which edge belong to which cycle of $\mathcal{C}$ and their respective orientations.

More precisely, after fixing an arbitrary orientation for each edge $e \in E(G)$ and each cycle $C \in \mathcal{C}$, one gets a sign for every incident pair (edge, cycle) which is 1 if the orientation coincide and -1 otherwise.

EXAMPLES:

For the cycle of length 5:

```
sage: G = graphs.CycleGraph(5)
sage: G.kirchhoff_symanzik_polynomial()                                    #␣
↪needs networkx sage.modules
t0 + t1 + t2 + t3 + t4
```

```
>>> from sage.all import *
>>> G = graphs.CycleGraph(Integer(5))
>>> G.kirchhoff_symanzik_polynomial()                                    #␣
↪needs networkx sage.modules
t0 + t1 + t2 + t3 + t4
```

One can use another letter for variables:

```
sage: G.kirchhoff_symanzik_polynomial(name='u')                          #␣
↪needs networkx sage.modules
u0 + u1 + u2 + u3 + u4
```

```
>>> from sage.all import *
>>> G.kirchhoff_symanzik_polynomial(name='u')                            #␣
↪needs networkx sage.modules
u0 + u1 + u2 + u3 + u4
```

For the 'coffee bean' graph:

```
sage: G = Graph([(0,1,'a'),(0,1,'b'),(0,1,'c')], multiedges=True)
sage: G.kirchhoff_symanzik_polynomial()                                  #␣
↪needs networkx sage.modules
t0*t1 + t0*t2 + t1*t2
```

```
>>> from sage.all import *
>>> G = Graph([(Integer(0),Integer(1),'a'),(Integer(0),Integer(1),'b'),
↪(Integer(0),Integer(1),'c')], multiedges=True)
>>> G.kirchhoff_symanzik_polynomial()                                    #␣
↪needs networkx sage.modules
t0*t1 + t0*t2 + t1*t2
```

For the 'parachute' graph:

```
sage: G = Graph([(0,2,'a'),(0,2,'b'),(0,1,'c'),(1,2,'d')], multiedges=True)
sage: G.kirchhoff_symanzik_polynomial()                                  #␣
↪needs networkx sage.modules
t0*t1 + t0*t2 + t1*t2 + t1*t3 + t2*t3
```

```
>>> from sage.all import *
>>> G = Graph([(Integer(0),Integer(2),'a'),(Integer(0),Integer(2),'b'),
↪(Integer(0),Integer(1),'c'),(Integer(1),Integer(2),'d')], multiedges=True)
>>> G.kirchhoff_symanzik_polynomial()                                    #␣
↪needs networkx sage.modules
t0*t1 + t0*t2 + t1*t2 + t1*t3 + t2*t3
```

For the complete graph with 4 vertices:

```
sage: G = graphs.CompleteGraph(4)
sage: G.kirchhoff_symanzik_polynomial()                                  #␣
↪needs networkx sage.modules
t0*t1*t3 + t0*t2*t3 + t1*t2*t3 + t0*t1*t4 + t0*t2*t4 + t1*t2*t4
+ t1*t3*t4 + t2*t3*t4 + t0*t1*t5 + t0*t2*t5 + t1*t2*t5 + t0*t3*t5
+ t2*t3*t5 + t0*t4*t5 + t1*t4*t5 + t3*t4*t5
```

```
>>> from sage.all import *
>>> G = graphs.CompleteGraph(Integer(4))
>>> G.kirchhoff_symanzik_polynomial()                                          #␣
↪needs networkx sage.modules
t0*t1*t3 + t0*t2*t3 + t1*t2*t3 + t0*t1*t4 + t0*t2*t4 + t1*t2*t4
+ t1*t3*t4 + t2*t3*t4 + t0*t1*t5 + t0*t2*t5 + t1*t2*t5 + t0*t3*t5
+ t2*t3*t5 + t0*t4*t5 + t1*t4*t5 + t3*t4*t5
```

REFERENCES:

[Bro2011]

least_effective_resistance(*nonedgesonly=True*)

Return a list of pairs of nodes with the least effective resistance.

The resistance distance between vertices i and j of a simple connected graph G is defined as the effective resistance between the two vertices on an electrical network constructed from G replacing each edge of the graph by a unit (1 ohm) resistor.

INPUT:

- `nonedgesonly` – boolean (default: `True`); if `True`, assign zero resistance to pairs of adjacent vertices

OUTPUT: list

EXAMPLES:

Pairs of non-adjacent nodes with least effective resistance in a straight linear 2-tree on 6 vertices:

```
sage: G = Graph([(0,1),(0,2),(1,2),(1,3),(3,5),(2,4),(2,3),(3,4),(4,5)])
sage: G.least_effective_resistance()                                        #␣
↪needs sage.modules
[(1, 4)]
```

```
>>> from sage.all import *
>>> G = Graph([(Integer(0),Integer(1)),(Integer(0),Integer(2)),(Integer(1),
↪Integer(2)),(Integer(1),Integer(3)),(Integer(3),Integer(5)),(Integer(2),
↪Integer(4)),(Integer(2),Integer(3)),(Integer(3),Integer(4)),(Integer(4),
↪Integer(5))])
>>> G.least_effective_resistance()                                          #␣
↪needs sage.modules
[(1, 4)]
```

Pairs of (adjacent or non-adjacent) nodes with least effective resistance in a straight linear 2-tree on 6 vertices

```
sage: G.least_effective_resistance(nonedgesonly=False)                      #␣
↪needs sage.modules
[(2, 3)]
```

```
>>> from sage.all import *
>>> G.least_effective_resistance(nonedgesonly=False)                        #␣
↪needs sage.modules
[(2, 3)]
```

Pairs of non-adjacent nodes with least effective resistance in a fan on 6 vertices counting only non-adjacent vertex pairs

```
sage: H = Graph([(0,1),(0,2),(0,3),(0,4),(0,5),(0,6),(1,2),(2,3),(3,4),(4,5)])
sage: H.least_effective_resistance()                                        #␣
↪needs sage.modules
[(2, 4)]
```

```
>>> from sage.all import *
>>> H = Graph([(Integer(0),Integer(1)),(Integer(0),Integer(2)),(Integer(0),
↪Integer(3)),(Integer(0),Integer(4)),(Integer(0),Integer(5)),(Integer(0),
↪Integer(6)),(Integer(1),Integer(2)),(Integer(2),Integer(3)),(Integer(3),
↪Integer(4)),(Integer(4),Integer(5))])
>>> H.least_effective_resistance()                                          #␣
↪needs sage.modules
[(2, 4)]
```

See also

- *effective_resistance_matrix()* – a similar method giving a matrix full of all effective resistances
- *effective_resistance()* – computes effective resistance for a single node pair
- See Wikipedia article Resistance_distance for more details.

lex_M(*triangulation=False*, *labels=False*, *initial_vertex=None*, *algorithm=None*)

Return an ordering of the vertices according the LexM graph traversal.

LexM is a lexicographic ordering scheme that is a special type of breadth-first-search. LexM can also produce a triangulation of the given graph. This functionality is implemented in this method. For more details on the algorithms used see Sections 4 (`'lex_M_slow'`) and 5.3 (`'lex_M_fast'`) of [RTL76].

Note

This method works only for undirected graphs.

INPUT:

- `triangulation` – boolean (default: `False`); whether to return a list of edges that need to be added in order to triangulate the graph
- `labels` – boolean (default: `False`); whether to return the labels assigned to each vertex
- `initial_vertex` – (default: `None`); the first vertex to consider
- `algorithm` – string (default: `None`); one of the following algorithms:
  - `'lex_M_slow'`: slower implementation of LexM traversal
  - `'lex_M_fast'`: faster implementation of LexM traversal (works only when `labels` is set to `False`)
  - `None`: Sage chooses the best algorithm: `'lex_M_slow'` if `labels` is set to `True`, `'lex_M_fast'` otherwise.

OUTPUT:

Depending on the values of the parameters `triangulation` and `labels` the method will return one or more of the following (in that order):

- an ordering of vertices of the graph according to LexM ordering scheme
- the labels assigned to each vertex
- a list of edges that when added to the graph will triangulate it

EXAMPLES:

LexM produces an ordering of the vertices:

```
sage: g = graphs.CompleteGraph(6)
sage: ord = g.lex_M(algorithm='lex_M_fast')
sage: len(ord) == g.order()
True
sage: set(ord) == set(g.vertices(sort=False))
True
sage: ord = g.lex_M(algorithm='lex_M_slow')
sage: len(ord) == g.order()
True
sage: set(ord) == set(g.vertices(sort=False))
True
```

```
>>> from sage.all import *
>>> g = graphs.CompleteGraph(Integer(6))
>>> ord = g.lex_M(algorithm='lex_M_fast')
>>> len(ord) == g.order()
True
>>> set(ord) == set(g.vertices(sort=False))
True
>>> ord = g.lex_M(algorithm='lex_M_slow')
>>> len(ord) == g.order()
True
>>> set(ord) == set(g.vertices(sort=False))
True
```

Both algorithms produce a valid LexM ordering α (i.e the neighbors of $\alpha(i)$ in $G[\{\alpha(i), ..., \alpha(n)\}]$ induce a clique):

```
sage: from sage.graphs.traversals import is_valid_lex_M_order
sage: G = graphs.PetersenGraph()
sage: ord, F = G.lex_M(triangulation=True, algorithm='lex_M_slow')
sage: is_valid_lex_M_order(G, ord, F)
True
sage: ord, F = G.lex_M(triangulation=True, algorithm='lex_M_fast')
sage: is_valid_lex_M_order(G, ord, F)
True
```

```
>>> from sage.all import *
>>> from sage.graphs.traversals import is_valid_lex_M_order
>>> G = graphs.PetersenGraph()
>>> ord, F = G.lex_M(triangulation=True, algorithm='lex_M_slow')
>>> is_valid_lex_M_order(G, ord, F)
True
```

(continues on next page)

(continued from previous page)

```
>>> ord, F = G.lex_M(triangulation=True, algorithm='lex_M_fast')
>>> is_valid_lex_M_order(G, ord, F)
True
```

LexM produces a triangulation of given graph:

```
sage: G = graphs.PetersenGraph()
sage: _, F = G.lex_M(triangulation=True)
sage: H = Graph(F, format='list_of_edges')
sage: H.is_chordal()
True
```

```
>>> from sage.all import *
>>> G = graphs.PetersenGraph()
>>> _, F = G.lex_M(triangulation=True)
>>> H = Graph(F, format='list_of_edges')
>>> H.is_chordal()
True
```

LexM ordering of the 3-sun graph:

```
sage: g = Graph([(1, 2), (1, 3), (2, 3), (2, 4), (2, 5), (3, 5), (3, 6), (4,␣
↪5), (5, 6)])
sage: g.lex_M()
[6, 4, 5, 3, 2, 1]
```

```
>>> from sage.all import *
>>> g = Graph([(Integer(1), Integer(2)), (Integer(1), Integer(3)),␣
↪(Integer(2), Integer(3)), (Integer(2), Integer(4)), (Integer(2),␣
↪Integer(5)), (Integer(3), Integer(5)), (Integer(3), Integer(6)),␣
↪(Integer(4), Integer(5)), (Integer(5), Integer(6))])
>>> g.lex_M()
[6, 4, 5, 3, 2, 1]
```

The ordering depends on the initial vertex:

```
sage: G = graphs.HouseGraph()
sage: G.lex_M(algorithm='lex_M_slow', initial_vertex=0)
[4, 3, 2, 1, 0]
sage: G.lex_M(algorithm='lex_M_slow', initial_vertex=2)
[1, 4, 3, 0, 2]
sage: G.lex_M(algorithm='lex_M_fast', initial_vertex=0)
[4, 3, 2, 1, 0]
sage: G.lex_M(algorithm='lex_M_fast', initial_vertex=2)
[1, 4, 3, 0, 2]
```

```
>>> from sage.all import *
>>> G = graphs.HouseGraph()
>>> G.lex_M(algorithm='lex_M_slow', initial_vertex=Integer(0))
[4, 3, 2, 1, 0]
>>> G.lex_M(algorithm='lex_M_slow', initial_vertex=Integer(2))
[1, 4, 3, 0, 2]
```

(continues on next page)

(continued from previous page)

```
>>> G.lex_M(algorithm='lex_M_fast', initial_vertex=Integer(0))
[4, 3, 2, 1, 0]
>>> G.lex_M(algorithm='lex_M_fast', initial_vertex=Integer(2))
[1, 4, 3, 0, 2]
```

lovasz_theta(*graph*)

Return the value of Lovász theta-function of graph.

For a graph G this function is denoted by $\theta(G)$, and it can be computed in polynomial time. Mathematically, its most important property is the following:

$$\alpha(G) \leq \theta(G) \leq \chi(\overline{G})$$

with $\alpha(G)$ and $\chi(\overline{G})$ being, respectively, the maximum size of an *independent set* set of G and the *chromatic number* of the *complement* $\overline{G}$ of G.

For more information, see the Wikipedia article Lovász_number.

Note

- Implemented for undirected graphs only. Use `to_undirected` to convert a digraph to an undirected graph.
- This function requires the optional package `csdp`, which you can install with `sage -i csdp`.

EXAMPLES:

```
sage: C = graphs.PetersenGraph()
sage: C.lovasz_theta()                                  # optional - csdp
4.0
sage: graphs.CycleGraph(5).lovasz_theta()               # optional - csdp
2.236068
```

```
>>> from sage.all import *
>>> C = graphs.PetersenGraph()
>>> C.lovasz_theta()                                  # optional - csdp
4.0
>>> graphs.CycleGraph(Integer(5)).lovasz_theta()          # optional - csdp
2.236068
```

magnitude_function()

Return the magnitude function of the graph as a rational function.

This is defined as the sum of all coefficients in the inverse of the matrix Z whose coefficient $Z_{i,j}$ indexed by a pair of vertices (i, j) is $q^d(i, j)$ where d is the distance function in the graph.

By convention, if the distance from i to j is infinite (for two vertices not path connected) then $Z_{i,j} = 0$.

The value of the magnitude function at $q = 0$ is the cardinality of the graph. The magnitude function of a disjoint union is the sum of the magnitudes functions of the connected components. The magnitude function of a Cartesian product is the product of the magnitudes functions of the factors.

EXAMPLES:

```
sage: g = Graph({1:[], 2:[]})
sage: g.magnitude_function()                                                #␣
↪needs sage.modules
2

sage: g = graphs.CycleGraph(4)
sage: g.magnitude_function()                                                #␣
↪needs sage.modules
4/(q^2 + 2*q + 1)

sage: g = graphs.CycleGraph(5)
sage: m = g.magnitude_function(); m                                         #␣
↪needs sage.modules
5/(2*q^2 + 2*q + 1)
```

```
>>> from sage.all import *
>>> g = Graph({Integer(1):[], Integer(2):[]})
>>> g.magnitude_function()                                                  #␣
↪needs sage.modules
2

>>> g = graphs.CycleGraph(Integer(4))
>>> g.magnitude_function()                                                  #␣
↪needs sage.modules
4/(q^2 + 2*q + 1)

>>> g = graphs.CycleGraph(Integer(5))
>>> m = g.magnitude_function(); m                                           #␣
↪needs sage.modules
5/(2*q^2 + 2*q + 1)
```

One can expand the magnitude as a power series in q as follows:

```
sage: q = QQ[['q']].gen()
sage: m(q)                                                                  #␣
↪needs sage.modules
5 - 10*q + 10*q^2 - 20*q^4 + 40*q^5 - 40*q^6 + ...
```

```
>>> from sage.all import *
>>> q = QQ[['q']].gen()
>>> m(q)                                                                    #␣
↪needs sage.modules
5 - 10*q + 10*q^2 - 20*q^4 + 40*q^5 - 40*q^6 + ...
```

One can also use the substitution $q = exp(-t)$ to obtain the magnitude function as a function of t:

```
sage: g = graphs.CycleGraph(6)
sage: m = g.magnitude_function()                                            #␣
↪needs sage.modules
sage: t = var('t')                                                          #␣
↪needs sage.modules sage.symbolic
sage: m(exp(-t))                                                            #␣
```

(continues on next page)

(continued from previous page)

```
↪needs sage.modules sage.symbolic
6/(2*e^(-t) + 2*e^(-2*t) + e^(-3*t) + 1)
```

```
>>> from sage.all import *
>>> g = graphs.CycleGraph(Integer(6))
>>> m = g.magnitude_function()                                    #␣
↪needs sage.modules
>>> t = var('t')                                                  #␣
↪needs sage.modules sage.symbolic
>>> m(exp(-t))                                                    #␣
↪needs sage.modules sage.symbolic
6/(2*e^(-t) + 2*e^(-2*t) + e^(-3*t) + 1)
```

REFERENCES:

matching(*G*, *value_only*, *algorithm=False*, *by_weight='Edmonds'*, *weight_function=False*, *check_weight=None*, *solver=True*, *verbose=None*, *integrality_tolerance=0*)

Return a maximum weighted matching of the graph represented by the list of its edges.

For more information, see the Wikipedia article Matching_(graph_theory).

Given a graph G such that each edge e has a weight w_e, a maximum matching is a subset S of the edges of G of maximum weight such that no two edges of S are incident with each other.

As an optimization problem, it can be expressed as:

$$\text{Maximize : } \sum_{e \in G.edges()} w_e b_e$$

$$\text{Such that : } \forall v \in G, \sum_{(u,v) \in G.edges()} b_{(u,v)} \leq 1$$

$$\forall x \in G, b_x \text{ is a binary variable}$$

INPUT:

- `value_only` – boolean (default: `False`); when set to `True`, only the cardinal (or the weight) of the matching is returned
- `algorithm` – string (default: `'Edmonds'`)
  - `'Edmonds'` selects Edmonds' algorithm as implemented in NetworkX
  - `'LP'` uses a Linear Program formulation of the matching problem
- `by_weight` – boolean (default: `False`); if `True`, computes a weighted matching where each edge has the weight given by `weight_function`, or its label if `weight_function` is `None`. If `False`, each edge has weight 1.
- `weight_function` – function (default: `None`); a function that takes as input an edge `(u, v, l)` and outputs its weight. If not `None`, `by_weight` is automatically set to `True`. If `None` and `by_weight` is `True`, we use the edge label `l`, if `l` is not `None`, else 1 as a weight.
- `check_weight` – boolean (default: `True`); whether to check that the `weight_function` outputs a number for each edge
- `solver` – string (default: `None`); specifies a Mixed Integer Linear Programming (MILP) solver to be used. If set to `None`, the default one is used. For more information on MILP solvers and which default solver is used, see the method `solve` of the class `MixedIntegerLinearProgram`.

- `verbose` – integer (default: 0); sets the level of verbosity: set to 0 by default, which means quiet (only useful when `algorithm == "LP"`)
- `integrality_tolerance` – float; parameter for use with MILP solvers over an inexact base ring; see `MixedIntegerLinearProgram.get_values()`.

OUTPUT:

- When `value_only=False` (default), this method returns an *EdgesView* containing the edges of a maximum matching of G.
- When `value_only=True`, this method returns the sum of the weights (default: `1`) of the edges of a maximum matching of G. The type of the output may vary according to the type of the edge labels and the algorithm used.

ALGORITHM:

The problem is solved using Edmond's algorithm implemented in NetworkX, or using Linear Programming depending on the value of `algorithm`.

EXAMPLES:

Maximum matching in a Pappus Graph:

```
sage: g = graphs.PappusGraph()
sage: g.matching(value_only=True)
↪# needs sage.networkx
9
```

```
>>> from sage.all import *
>>> g = graphs.PappusGraph()
>>> g.matching(value_only=True)                                                #␣
↪needs sage.networkx
9
```

Same test with the Linear Program formulation:

```
sage: g = graphs.PappusGraph()
sage: g.matching(algorithm='LP', value_only=True)
↪# needs sage.numerical.mip
9
```

```
>>> from sage.all import *
>>> g = graphs.PappusGraph()
>>> g.matching(algorithm='LP', value_only=True)                                #␣
↪needs sage.numerical.mip
9
```

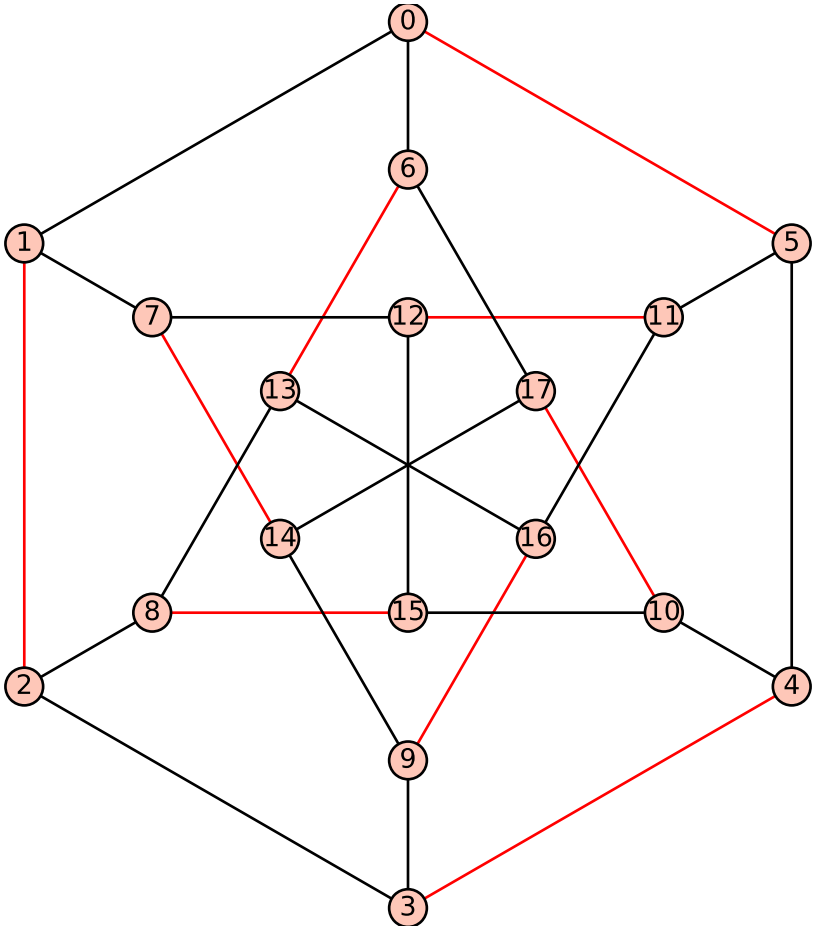
matching_polynomial(*G*, *complement=True*, *name=None*)

Compute the matching polynomial of the graph G.

If $p(G, k)$ denotes the number of k-matchings (matchings with k edges) in G, then the matching polynomial is defined as [God1993]:

$$\mu(x) = \sum_{k \geq 0} (-1)^k p(G, k) x^{n-2k}$$

INPUT:


0
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17

- `complement` – boolean (default: `True`); whether to use Godsil's duality theorem to compute the matching polynomial from that of the graphs complement (see ALGORITHM)
- `name` – (optional) string for the variable name in the polynomial

Note

The `complement` option uses matching polynomials of complete graphs, which are cached. So if you are crazy enough to try computing the matching polynomial on a graph with millions of vertices, you might not want to use this option, since it will end up caching millions of polynomials of degree in the millions.

ALGORITHM:

The algorithm used is a recursive one, based on the following observation [God1993]:

- If e is an edge of G, G' is the result of deleting the edge e, and G'' is the result of deleting each vertex in e, then the matching polynomial of G is equal to that of G' minus that of G''.

  (the algorithm actually computes the *signless* matching polynomial, for which the recursion is the same when one replaces the subtraction by an addition. It is then converted into the matching polynomial and returned)

Depending on the value of `complement`, Godsil's duality theorem [God1993] can also be used to compute $\mu(x)$:

$$\mu(\overline{G}, x) = \sum_{k \geq 0} p(G, k)\mu(K_{n-2k}, x)$$

Where $\overline{G}$ is the complement of G, and K_n the complete graph on n vertices.

EXAMPLES:

```
sage: g = graphs.PetersenGraph()
sage: g.matching_polynomial()
x^10 - 15*x^8 + 75*x^6 - 145*x^4 + 90*x^2 - 6
sage: g.matching_polynomial(complement=False)
x^10 - 15*x^8 + 75*x^6 - 145*x^4 + 90*x^2 - 6
sage: g.matching_polynomial(name='tom')
tom^10 - 15*tom^8 + 75*tom^6 - 145*tom^4 + 90*tom^2 - 6
sage: g = Graph()
sage: L = [graphs.RandomGNP(8, .3) for i in range(1, 6)]
sage: prod([h.matching_polynomial() for h in L]) == sum(L, g).matching_
↪polynomial()  # long time (up to 10s on sage.math, 2011)
True
```

```
>>> from sage.all import *
>>> g = graphs.PetersenGraph()
>>> g.matching_polynomial()
x^10 - 15*x^8 + 75*x^6 - 145*x^4 + 90*x^2 - 6
>>> g.matching_polynomial(complement=False)
x^10 - 15*x^8 + 75*x^6 - 145*x^4 + 90*x^2 - 6
>>> g.matching_polynomial(name='tom')
tom^10 - 15*tom^8 + 75*tom^6 - 145*tom^4 + 90*tom^2 - 6
>>> g = Graph()
>>> L = [graphs.RandomGNP(Integer(8), RealNumber('.3')) for i in␣
```

(continues on next page)

(continued from previous page)

```
↪range(Integer(1), Integer(6))]
>>> prod([h.matching_polynomial() for h in L]) == sum(L, g).matching_
↪polynomial()  # long time (up to 10s on sage.math, 2011)
True
```

```
sage: for i in range(1, 12):  # long time (10s on sage.math, 2011)
....:     for t in graphs.trees(i):
....:         if t.matching_polynomial() != t.characteristic_polynomial():
....:             raise RuntimeError('bug for a tree A of size {0}'.format(i))
....:         c = t.complement()
....:         if c.matching_polynomial(complement=False) != c.matching_
↪polynomial():
....:             raise RuntimeError('bug for a tree B of size {0}'.format(i))
```

```
>>> from sage.all import *
>>> for i in range(Integer(1), Integer(12)):  # long time (10s on sage.math,␣
↪2011)
...     for t in graphs.trees(i):
...         if t.matching_polynomial() != t.characteristic_polynomial():
...             raise RuntimeError('bug for a tree A of size {0}'.format(i))
...         c = t.complement()
...         if c.matching_polynomial(complement=False) != c.matching_
↪polynomial():
...             raise RuntimeError('bug for a tree B of size {0}'.format(i))
```

```
sage: from sage.graphs.matchpoly import matching_polynomial
sage: matching_polynomial(graphs.CompleteGraph(0))
1
sage: matching_polynomial(graphs.CompleteGraph(1))
x
sage: matching_polynomial(graphs.CompleteGraph(2))
x^2 - 1
sage: matching_polynomial(graphs.CompleteGraph(3))
x^3 - 3*x
sage: matching_polynomial(graphs.CompleteGraph(4))
x^4 - 6*x^2 + 3
sage: matching_polynomial(graphs.CompleteGraph(5))
x^5 - 10*x^3 + 15*x
sage: matching_polynomial(graphs.CompleteGraph(6))
x^6 - 15*x^4 + 45*x^2 - 15
sage: matching_polynomial(graphs.CompleteGraph(7))
x^7 - 21*x^5 + 105*x^3 - 105*x
sage: matching_polynomial(graphs.CompleteGraph(8))
x^8 - 28*x^6 + 210*x^4 - 420*x^2 + 105
sage: matching_polynomial(graphs.CompleteGraph(9))
x^9 - 36*x^7 + 378*x^5 - 1260*x^3 + 945*x
sage: matching_polynomial(graphs.CompleteGraph(10))
x^10 - 45*x^8 + 630*x^6 - 3150*x^4 + 4725*x^2 - 945
sage: matching_polynomial(graphs.CompleteGraph(11))
x^11 - 55*x^9 + 990*x^7 - 6930*x^5 + 17325*x^3 - 10395*x
sage: matching_polynomial(graphs.CompleteGraph(12))
```

(continues on next page)

(continued from previous page)

```
x^12 - 66*x^10 + 1485*x^8 - 13860*x^6 + 51975*x^4 - 62370*x^2 + 10395
sage: matching_polynomial(graphs.CompleteGraph(13))
x^13 - 78*x^11 + 2145*x^9 - 25740*x^7 + 135135*x^5 - 270270*x^3 + 135135*x
```

```
>>> from sage.all import *
>>> from sage.graphs.matchpoly import matching_polynomial
>>> matching_polynomial(graphs.CompleteGraph(Integer(0)))
1
>>> matching_polynomial(graphs.CompleteGraph(Integer(1)))
x
>>> matching_polynomial(graphs.CompleteGraph(Integer(2)))
x^2 - 1
>>> matching_polynomial(graphs.CompleteGraph(Integer(3)))
x^3 - 3*x
>>> matching_polynomial(graphs.CompleteGraph(Integer(4)))
x^4 - 6*x^2 + 3
>>> matching_polynomial(graphs.CompleteGraph(Integer(5)))
x^5 - 10*x^3 + 15*x
>>> matching_polynomial(graphs.CompleteGraph(Integer(6)))
x^6 - 15*x^4 + 45*x^2 - 15
>>> matching_polynomial(graphs.CompleteGraph(Integer(7)))
x^7 - 21*x^5 + 105*x^3 - 105*x
>>> matching_polynomial(graphs.CompleteGraph(Integer(8)))
x^8 - 28*x^6 + 210*x^4 - 420*x^2 + 105
>>> matching_polynomial(graphs.CompleteGraph(Integer(9)))
x^9 - 36*x^7 + 378*x^5 - 1260*x^3 + 945*x
>>> matching_polynomial(graphs.CompleteGraph(Integer(10)))
x^10 - 45*x^8 + 630*x^6 - 3150*x^4 + 4725*x^2 - 945
>>> matching_polynomial(graphs.CompleteGraph(Integer(11)))
x^11 - 55*x^9 + 990*x^7 - 6930*x^5 + 17325*x^3 - 10395*x
>>> matching_polynomial(graphs.CompleteGraph(Integer(12)))
x^12 - 66*x^10 + 1485*x^8 - 13860*x^6 + 51975*x^4 - 62370*x^2 + 10395
>>> matching_polynomial(graphs.CompleteGraph(Integer(13)))
x^13 - 78*x^11 + 2145*x^9 - 25740*x^7 + 135135*x^5 - 270270*x^3 + 135135*x
```

```
sage: G = Graph({0:[1,2], 1:[2]})
sage: matching_polynomial(G)
x^3 - 3*x
sage: G = Graph({0:[1,2]})
sage: matching_polynomial(G)
x^3 - 2*x
sage: G = Graph({0:[1], 2:[]})
sage: matching_polynomial(G)
x^3 - x
sage: G = Graph({0:[], 1:[], 2:[]})
sage: matching_polynomial(G)
x^3
```

```
>>> from sage.all import *
>>> G = Graph({Integer(0):[Integer(1),Integer(2)], Integer(1):[Integer(2)]})
>>> matching_polynomial(G)
```

(continues on next page)

(continued from previous page)

```
x^3 - 3*x
>>> G = Graph({Integer(0):[Integer(1),Integer(2)]})
>>> matching_polynomial(G)
x^3 - 2*x
>>> G = Graph({Integer(0):[Integer(1)], Integer(2):[]})
>>> matching_polynomial(G)
x^3 - x
>>> G = Graph({Integer(0):[], Integer(1):[], Integer(2):[]})
>>> matching_polynomial(G)
x^3
```

```
sage: matching_polynomial(graphs.CompleteGraph(0), complement=False)
1
sage: matching_polynomial(graphs.CompleteGraph(1), complement=False)
x
sage: matching_polynomial(graphs.CompleteGraph(2), complement=False)
x^2 - 1
sage: matching_polynomial(graphs.CompleteGraph(3), complement=False)
x^3 - 3*x
sage: matching_polynomial(graphs.CompleteGraph(4), complement=False)
x^4 - 6*x^2 + 3
sage: matching_polynomial(graphs.CompleteGraph(5), complement=False)
x^5 - 10*x^3 + 15*x
sage: matching_polynomial(graphs.CompleteGraph(6), complement=False)
x^6 - 15*x^4 + 45*x^2 - 15
sage: matching_polynomial(graphs.CompleteGraph(7), complement=False)
x^7 - 21*x^5 + 105*x^3 - 105*x
sage: matching_polynomial(graphs.CompleteGraph(8), complement=False)
x^8 - 28*x^6 + 210*x^4 - 420*x^2 + 105
sage: matching_polynomial(graphs.CompleteGraph(9), complement=False)
x^9 - 36*x^7 + 378*x^5 - 1260*x^3 + 945*x
sage: matching_polynomial(graphs.CompleteGraph(10), complement=False)
x^10 - 45*x^8 + 630*x^6 - 3150*x^4 + 4725*x^2 - 945
sage: matching_polynomial(graphs.CompleteGraph(11), complement=False)
x^11 - 55*x^9 + 990*x^7 - 6930*x^5 + 17325*x^3 - 10395*x
sage: matching_polynomial(graphs.CompleteGraph(12), complement=False)
x^12 - 66*x^10 + 1485*x^8 - 13860*x^6 + 51975*x^4 - 62370*x^2 + 10395
sage: matching_polynomial(graphs.CompleteGraph(13), complement=False)
x^13 - 78*x^11 + 2145*x^9 - 25740*x^7 + 135135*x^5 - 270270*x^3 + 135135*x
```

```
>>> from sage.all import *
>>> matching_polynomial(graphs.CompleteGraph(Integer(0)), complement=False)
1
>>> matching_polynomial(graphs.CompleteGraph(Integer(1)), complement=False)
x
>>> matching_polynomial(graphs.CompleteGraph(Integer(2)), complement=False)
x^2 - 1
>>> matching_polynomial(graphs.CompleteGraph(Integer(3)), complement=False)
x^3 - 3*x
>>> matching_polynomial(graphs.CompleteGraph(Integer(4)), complement=False)
x^4 - 6*x^2 + 3
```

(continues on next page)

(continued from previous page)

```
>>> matching_polynomial(graphs.CompleteGraph(Integer(5)), complement=False)
x^5 - 10*x^3 + 15*x
>>> matching_polynomial(graphs.CompleteGraph(Integer(6)), complement=False)
x^6 - 15*x^4 + 45*x^2 - 15
>>> matching_polynomial(graphs.CompleteGraph(Integer(7)), complement=False)
x^7 - 21*x^5 + 105*x^3 - 105*x
>>> matching_polynomial(graphs.CompleteGraph(Integer(8)), complement=False)
x^8 - 28*x^6 + 210*x^4 - 420*x^2 + 105
>>> matching_polynomial(graphs.CompleteGraph(Integer(9)), complement=False)
x^9 - 36*x^7 + 378*x^5 - 1260*x^3 + 945*x
>>> matching_polynomial(graphs.CompleteGraph(Integer(10)), complement=False)
x^10 - 45*x^8 + 630*x^6 - 3150*x^4 + 4725*x^2 - 945
>>> matching_polynomial(graphs.CompleteGraph(Integer(11)), complement=False)
x^11 - 55*x^9 + 990*x^7 - 6930*x^5 + 17325*x^3 - 10395*x
>>> matching_polynomial(graphs.CompleteGraph(Integer(12)), complement=False)
x^12 - 66*x^10 + 1485*x^8 - 13860*x^6 + 51975*x^4 - 62370*x^2 + 10395
>>> matching_polynomial(graphs.CompleteGraph(Integer(13)), complement=False)
x^13 - 78*x^11 + 2145*x^9 - 25740*x^7 + 135135*x^5 - 270270*x^3 + 135135*x
```

maximum_average_degree(*value_only=True*, *solver=None*, *verbose=0*)

Return the Maximum Average Degree (MAD) of the current graph.

The Maximum Average Degree (MAD) of a graph is defined as the average degree of its densest subgraph. More formally, `Mad(G) = \max_{H\subseteq G} Ad(H)`, where $Ad(G)$ denotes the average degree of G.

This can be computed in polynomial time.

INPUT:

- `value_only` – boolean (default: `True`)
  - If `value_only=True`, only the numerical value of the MAD is returned.
  - Else, the subgraph of G realizing the MAD is returned.
- `solver` – (default: `None`) specify a Linear Program (LP) solver to be used. If set to `None`, the default one is used. For more information on LP solvers and which default solver is used, see the method `solve` of the class `MixedIntegerLinearProgram`.
- `verbose` – integer (default: 0); sets the level of verbosity. Set to 0 by default, which means quiet.

EXAMPLES:

In any graph, the Mad is always larger than the average degree:

```
sage: g = graphs.RandomGNP(20,.3)
sage: mad_g = g.maximum_average_degree()                                    #␣
↪needs sage.numerical.mip
sage: g.average_degree() <= mad_g                                           #␣
↪needs sage.numerical.mip
True
```

```
>>> from sage.all import *
>>> g = graphs.RandomGNP(Integer(20),RealNumber('.3'))
>>> mad_g = g.maximum_average_degree()                                      #␣
↪needs sage.numerical.mip
```

(continues on next page)

(continued from previous page)

```
>>> g.average_degree() <= mad_g                                                #␣
↪needs sage.numerical.mip
True
```

Unlike the average degree, the Mad of the disjoint union of two graphs is the maximum of the Mad of each graphs:

```
sage: h = graphs.RandomGNP(20,.3)
sage: mad_h = h.maximum_average_degree()                                       #␣
↪needs sage.numerical.mip
sage: (g+h).maximum_average_degree() == max(mad_g, mad_h)                      #␣
↪needs sage.numerical.mip
True
```

```
>>> from sage.all import *
>>> h = graphs.RandomGNP(Integer(20),RealNumber('.3'))
>>> mad_h = h.maximum_average_degree()                                         #␣
↪needs sage.numerical.mip
>>> (g+h).maximum_average_degree() == max(mad_g, mad_h)                        #␣
↪needs sage.numerical.mip
True
```

The subgraph of a regular graph realizing the maximum average degree is always the whole graph

```
sage: g = graphs.CompleteGraph(5)
sage: mad_g = g.maximum_average_degree(value_only=False)                       #␣
↪needs sage.numerical.mip
sage: g.is_isomorphic(mad_g)                                                   #␣
↪needs sage.numerical.mip
True
```

```
>>> from sage.all import *
>>> g = graphs.CompleteGraph(Integer(5))
>>> mad_g = g.maximum_average_degree(value_only=False)                         #␣
↪needs sage.numerical.mip
>>> g.is_isomorphic(mad_g)                                                     #␣
↪needs sage.numerical.mip
True
```

This also works for complete bipartite graphs

```
sage: g = graphs.CompleteBipartiteGraph(3,4)
sage: mad_g = g.maximum_average_degree(value_only=False)                       #␣
↪needs sage.numerical.mip
sage: g.is_isomorphic(mad_g)                                                   #␣
↪needs sage.numerical.mip
True
```

```
>>> from sage.all import *
>>> g = graphs.CompleteBipartiteGraph(Integer(3),Integer(4))
>>> mad_g = g.maximum_average_degree(value_only=False)                         #␣
↪needs sage.numerical.mip
```

(continues on next page)

(continued from previous page)

```
>>> g.is_isomorphic(mad_g)                                                    #␣
↪needs sage.numerical.mip
True
```

maximum_cardinality_search(*G*, *reverse=False*, *tree=False*, *initial_vertex=None*)

Return an ordering of the vertices according a maximum cardinality search.

Maximum cardinality search (MCS) is a graph traversal introduced in [TY1984]. It starts by assigning an arbitrary vertex (or the specified `initial_vertex`) of G the last position in the ordering α. Every vertex keeps a weight equal to the number of its already processed neighbors (i.e., already added to α), and a vertex of largest such number is chosen at each step i to be placed in position $n - i$ in α. This ordering can be computed in time $O(n + m)$.

Time complexity is $O(n + m)$ for `SparseGraph` and $O(n^2)$ for `DenseGraph` where n is the number of vertices and m is the number of edges.

When the graph is chordal, the ordering returned by MCS is a *perfect elimination ordering*, like `lex_BFS()`. So this ordering can be used to recognize chordal graphs. See [He2006] for more details.

Note

The current implementation is for connected graphs only.

INPUT:

- `G` – a Sage graph
- `reverse` – boolean (default: `False`); whether to return the vertices in discovery order, or the reverse
- `tree` – boolean (default: `False`); whether to also return the discovery directed tree (each vertex being linked to the one that saw it for the first time)
- `initial_vertex` – (default: `None`) the first vertex to consider

OUTPUT:

By default, return the ordering α as a list. When `tree` is `True`, the method returns a tuple (α, T), where T is a directed tree with the same set of vertices as G and a directed edge from u to v if u was the first vertex to see v.

EXAMPLES:

When specified, the `initial_vertex` is placed at the end of the ordering, unless parameter `reverse` is `True`, in which case it is placed at the beginning:

```
sage: G = graphs.PathGraph(4)
sage: G.maximum_cardinality_search(initial_vertex=0)
[3, 2, 1, 0]
sage: G.maximum_cardinality_search(initial_vertex=1)
[3, 2, 0, 1]
sage: G.maximum_cardinality_search(initial_vertex=2)
[0, 3, 1, 2]
sage: G.maximum_cardinality_search(initial_vertex=3)
[0, 1, 2, 3]
sage: G.maximum_cardinality_search(initial_vertex=3, reverse=True)
[3, 2, 1, 0]
```

```
>>> from sage.all import *
>>> G = graphs.PathGraph(Integer(4))
>>> G.maximum_cardinality_search(initial_vertex=Integer(0))
[3, 2, 1, 0]
>>> G.maximum_cardinality_search(initial_vertex=Integer(1))
[3, 2, 0, 1]
>>> G.maximum_cardinality_search(initial_vertex=Integer(2))
[0, 3, 1, 2]
>>> G.maximum_cardinality_search(initial_vertex=Integer(3))
[0, 1, 2, 3]
>>> G.maximum_cardinality_search(initial_vertex=Integer(3), reverse=True)
[3, 2, 1, 0]
```

Returning the discovery tree:

```
sage: G = graphs.PathGraph(4)
sage: _, T = G.maximum_cardinality_search(tree=True, initial_vertex=0)
sage: T.order(), T.size()
(4, 3)
sage: T.edges(labels=False, sort=True)
[(1, 0), (2, 1), (3, 2)]
sage: _, T = G.maximum_cardinality_search(tree=True, initial_vertex=3)
sage: T.edges(labels=False, sort=True)
[(0, 1), (1, 2), (2, 3)]
```

```
>>> from sage.all import *
>>> G = graphs.PathGraph(Integer(4))
>>> _, T = G.maximum_cardinality_search(tree=True, initial_vertex=Integer(0))
>>> T.order(), T.size()
(4, 3)
>>> T.edges(labels=False, sort=True)
[(1, 0), (2, 1), (3, 2)]
>>> _, T = G.maximum_cardinality_search(tree=True, initial_vertex=Integer(3))
>>> T.edges(labels=False, sort=True)
[(0, 1), (1, 2), (2, 3)]
```

maximum_cardinality_search_M(*G*, *initial_vertex=None*)

Return the ordering and the edges of the triangulation produced by MCS-M.

Maximum cardinality search M (MCS-M) is an extension of MCS (*maximum_cardinality_search()*) in the same way that Lex-M (*lex_M()*) is an extension of Lex-BFS (*lex_BFS()*). That is, in MCS-M when u receives number i at step $n - i + 1$, it increments the weight of all unnumbered vertices v for which there exists a path between u and v consisting only of unnumbered vertices with weight strictly less than $w^-(u)$ and $w^-(v)$, where w^- is the number of times a vertex has been reached during previous iterations. See [BBHP2004] for the details of this $O(nm)$ time algorithm.

If G is not connected, the orderings of each of its connected components are added consecutively. Furthermore, if G has k connected components C_i for $0 \leq i < k$, X contains at least one vertex of C_i for each $i \geq 1$. Hence, $|X| \geq k - 1$. In particular, some isolated vertices (i.e., of degree 0) can appear in X as for such a vertex x, we have that $G \setminus N(x) = G$ is not connected.

INPUT:

- `G` – a Sage graph
- `initial_vertex` – (default: `None`) the first vertex to consider

OUTPUT: a tuple (α, F, X), where

- α is the resulting ordering of the vertices. If an initial vertex is specified, it gets the last position in the ordering α.
- F is the list of edges of a minimal triangulation of G according α
- X is a list of vertices such that for each $x \in X$, the neighborhood of x in G is a separator (i.e., $G \setminus N(x)$ is not connected). Note that we may have $N(x) = \emptyset$ if G is not connected and x has degree 0.

EXAMPLES:

Chordal graphs have a perfect elimination ordering, and so the set F of edges of the triangulation is empty:

```
sage: G = graphs.RandomChordalGraph(20)
sage: alpha, F, X = G.maximum_cardinality_search_M(); F
[]
```

```
>>> from sage.all import *
>>> G = graphs.RandomChordalGraph(Integer(20))
>>> alpha, F, X = G.maximum_cardinality_search_M(); F
[]
```

The cycle of order 4 is not chordal and so the triangulation has one edge:

```
sage: G = graphs.CycleGraph(4)
sage: alpha, F, X = G.maximum_cardinality_search_M(); len(F)
1
```

```
>>> from sage.all import *
>>> G = graphs.CycleGraph(Integer(4))
>>> alpha, F, X = G.maximum_cardinality_search_M(); len(F)
1
```

The number of edges needed to triangulate of a cycle graph or order n is $n - 3$, independently of the initial vertex:

```
sage: n = randint(3, 20)
sage: C = graphs.CycleGraph(n)
sage: _, F, X = C.maximum_cardinality_search_M()
sage: len(F) == n - 3
True
sage: _, F, X = C.maximum_cardinality_search_M(initial_vertex=C.random_
↪vertex())
sage: len(F) == n - 3
True
```

```
>>> from sage.all import *
>>> n = randint(Integer(3), Integer(20))
>>> C = graphs.CycleGraph(n)
>>> _, F, X = C.maximum_cardinality_search_M()
>>> len(F) == n - Integer(3)
True
>>> _, F, X = C.maximum_cardinality_search_M(initial_vertex=C.random_vertex())
>>> len(F) == n - Integer(3)
True
```

When an initial vertex is specified, it gets the last position in the ordering:

```
sage: G = graphs.PathGraph(4)
sage: G.maximum_cardinality_search_M(initial_vertex=0)
([3, 2, 1, 0], [], [2, 3])
sage: G.maximum_cardinality_search_M(initial_vertex=1)
([3, 2, 0, 1], [], [2, 3])
sage: G.maximum_cardinality_search_M(initial_vertex=2)
([0, 1, 3, 2], [], [0, 1])
sage: G.maximum_cardinality_search_M(initial_vertex=3)
([0, 1, 2, 3], [], [0, 1])
```

```
>>> from sage.all import *
>>> G = graphs.PathGraph(Integer(4))
>>> G.maximum_cardinality_search_M(initial_vertex=Integer(0))
([3, 2, 1, 0], [], [2, 3])
>>> G.maximum_cardinality_search_M(initial_vertex=Integer(1))
([3, 2, 0, 1], [], [2, 3])
>>> G.maximum_cardinality_search_M(initial_vertex=Integer(2))
([0, 1, 3, 2], [], [0, 1])
>>> G.maximum_cardinality_search_M(initial_vertex=Integer(3))
([0, 1, 2, 3], [], [0, 1])
```

When G is not connected, the orderings of each of its connected components are added consecutively, the vertices of the component containing the initial vertex occupying the last positions:

```
sage: G = graphs.CycleGraph(4) * 2
sage: G.maximum_cardinality_search_M()[0]
[5, 4, 6, 7, 2, 3, 1, 0]
sage: G.maximum_cardinality_search_M(initial_vertex=7)[0]
[2, 1, 3, 0, 5, 6, 4, 7]
```

```
>>> from sage.all import *
>>> G = graphs.CycleGraph(Integer(4)) * Integer(2)
>>> G.maximum_cardinality_search_M()[Integer(0)]
[5, 4, 6, 7, 2, 3, 1, 0]
>>> G.maximum_cardinality_search_M(initial_vertex=Integer(7))[Integer(0)]
[2, 1, 3, 0, 5, 6, 4, 7]
```

Furthermore, if G has k connected components, X contains at least one vertex per connected component, except for the first one, and so at least $k-1$ vertices:

```
sage: for k in range(1, 5):
....:     _, _, X = Graph(k).maximum_cardinality_search_M()
....:     if len(X) < k - 1:
....:         raise ValueError("something goes wrong")
sage: G = graphs.RandomGNP(10, .2)
sage: cc = G.connected_components(sort=False)
sage: _, _, X = G.maximum_cardinality_search_M()
sage: len(X) >= len(cc) - 1
True
```

```
>>> from sage.all import *
>>> for k in range(Integer(1), Integer(5)):
...     _, _, X = Graph(k).maximum_cardinality_search_M()
...     if len(X) < k - Integer(1):
...         raise ValueError("something goes wrong")
>>> G = graphs.RandomGNP(Integer(10), RealNumber('.2'))
>>> cc = G.connected_components(sort=False)
>>> _, _, X = G.maximum_cardinality_search_M()
>>> len(X) >= len(cc) - Integer(1)
True
```

In the example of [BPS2010], the triangulation has 3 edges:

```
sage: G = Graph({'a': ['b', 'k'], 'b': ['c'], 'c': ['d', 'j', 'k'],
....:            'd': ['e', 'f', 'j', 'k'], 'e': ['g'],
....:            'f': ['g', 'j', 'k'], 'g': ['j', 'k'], 'h': ['i', 'j'],
....:            'i': ['k'], 'j': ['k']})
sage: _, F, _ = G.maximum_cardinality_search_M(initial_vertex='a')
sage: len(F)
3
```

```
>>> from sage.all import *
>>> G = Graph({'a': ['b', 'k'], 'b': ['c'], 'c': ['d', 'j', 'k'],
...            'd': ['e', 'f', 'j', 'k'], 'e': ['g'],
...            'f': ['g', 'j', 'k'], 'g': ['j', 'k'], 'h': ['i', 'j'],
...            'i': ['k'], 'j': ['k']})
>>> _, F, _ = G.maximum_cardinality_search_M(initial_vertex='a')
>>> len(F)
3
```

minimal_dominating_sets(*G*, *to_dominate=None*, *work_on_copy=True*, *k=1*)

Return an iterator over the minimal dominating sets of a graph.

INPUT:

- `G` – a graph
- `to_dominate` – vertex iterable or `None` (default: `None`); the set of vertices to be dominated
- `work_on_copy` – boolean (default: `True`); whether or not to work on a copy of the input graph; if set to `False`, the input graph will be modified (relabeled)
- `k` – nonnegative integer (default: 1); the domination distance

OUTPUT:

An iterator over the inclusion-minimal sets of vertices of `G`. If `to_dominate` is provided, return an iterator over the inclusion-minimal sets of vertices that dominate the vertices of `to_dominate`.

ALGORITHM: The algorithm described in [BDHPR2019].

AUTHOR: Jean-Florent Raymond (2019-03-04) – initial version.

EXAMPLES:

```
sage: G = graphs.ButterflyGraph()
sage: ll = list(G.minimal_dominating_sets())
```

(continues on next page)

(continued from previous page)

```
sage: pp = [{0, 1}, {1, 3}, {0, 2}, {2, 3}, {4}]
sage: len(ll) == len(pp) and all(x in pp for x in ll) and all(x in ll for x␣
↪in pp)
True

sage: ll = list(G.minimal_dominating_sets([0,3]))
sage: pp = [{0}, {3}, {4}]
sage: len(ll) == len(pp) and all(x in pp for x in ll) and all(x in ll for x␣
↪in pp)
True

sage: ll = list(G.minimal_dominating_sets([4]))
sage: pp = [{4}, {0}, {1}, {2}, {3}]
sage: len(ll) == len(pp) and all(x in pp for x in ll) and all(x in ll for x␣
↪in pp)
True
```

```
>>> from sage.all import *
>>> G = graphs.ButterflyGraph()
>>> ll = list(G.minimal_dominating_sets())
>>> pp = [{Integer(0), Integer(1)}, {Integer(1), Integer(3)}, {Integer(0),␣
↪Integer(2)}, {Integer(2), Integer(3)}, {Integer(4)}]
>>> len(ll) == len(pp) and all(x in pp for x in ll) and all(x in ll for x in␣
↪pp)
True

>>> ll = list(G.minimal_dominating_sets([Integer(0),Integer(3)]))
>>> pp = [{Integer(0)}, {Integer(3)}, {Integer(4)}]
>>> len(ll) == len(pp) and all(x in pp for x in ll) and all(x in ll for x in␣
↪pp)
True

>>> ll = list(G.minimal_dominating_sets([Integer(4)]))
>>> pp = [{Integer(4)}, {Integer(0)}, {Integer(1)}, {Integer(2)}, {Integer(3)}
↪]
>>> len(ll) == len(pp) and all(x in pp for x in ll) and all(x in ll for x in␣
↪pp)
True
```

```
sage: ll = list(graphs.PetersenGraph().minimal_dominating_sets())
sage: pp = [{0, 2, 6},
....:       {0, 9, 3},
....:       {0, 8, 7},
....:       {1, 3, 7},
....:       {1, 4, 5},
....:       {8, 1, 9},
....:       {8, 2, 4},
....:       {9, 2, 5},
....:       {3, 5, 6},
....:       {4, 6, 7},
....:       {0, 8, 2, 9},
```

(continues on next page)

(continued from previous page)

```
....:        {0, 3, 6, 7},
....:        {1, 3, 5, 9},
....:        {8, 1, 4, 7},
....:        {2, 4, 5, 6},
....:        {0, 1, 2, 3, 4},
....:        {0, 1, 2, 5, 7},
....:        {0, 1, 4, 6, 9},
....:        {0, 1, 5, 6, 8},
....:        {0, 8, 3, 4, 5},
....:        {0, 9, 4, 5, 7},
....:        {8, 1, 2, 3, 6},
....:        {1, 2, 9, 6, 7},
....:        {9, 2, 3, 4, 7},
....:        {8, 2, 3, 5, 7},
....:        {8, 9, 3, 4, 6},
....:        {8, 9, 5, 6, 7}]
sage: len(ll) == len(pp) and all(x in pp for x in ll) and all(x in ll for x␣
↪in pp)
True
```

```
>>> from sage.all import *
>>> ll = list(graphs.PetersenGraph().minimal_dominating_sets())
>>> pp = [{Integer(0), Integer(2), Integer(6)},
...       {Integer(0), Integer(9), Integer(3)},
...       {Integer(0), Integer(8), Integer(7)},
...       {Integer(1), Integer(3), Integer(7)},
...       {Integer(1), Integer(4), Integer(5)},
...       {Integer(8), Integer(1), Integer(9)},
...       {Integer(8), Integer(2), Integer(4)},
...       {Integer(9), Integer(2), Integer(5)},
...       {Integer(3), Integer(5), Integer(6)},
...       {Integer(4), Integer(6), Integer(7)},
...       {Integer(0), Integer(8), Integer(2), Integer(9)},
...       {Integer(0), Integer(3), Integer(6), Integer(7)},
...       {Integer(1), Integer(3), Integer(5), Integer(9)},
...       {Integer(8), Integer(1), Integer(4), Integer(7)},
...       {Integer(2), Integer(4), Integer(5), Integer(6)},
...       {Integer(0), Integer(1), Integer(2), Integer(3), Integer(4)},
...       {Integer(0), Integer(1), Integer(2), Integer(5), Integer(7)},
...       {Integer(0), Integer(1), Integer(4), Integer(6), Integer(9)},
...       {Integer(0), Integer(1), Integer(5), Integer(6), Integer(8)},
...       {Integer(0), Integer(8), Integer(3), Integer(4), Integer(5)},
...       {Integer(0), Integer(9), Integer(4), Integer(5), Integer(7)},
...       {Integer(8), Integer(1), Integer(2), Integer(3), Integer(6)},
...       {Integer(1), Integer(2), Integer(9), Integer(6), Integer(7)},
...       {Integer(9), Integer(2), Integer(3), Integer(4), Integer(7)},
...       {Integer(8), Integer(2), Integer(3), Integer(5), Integer(7)},
...       {Integer(8), Integer(9), Integer(3), Integer(4), Integer(6)},
...       {Integer(8), Integer(9), Integer(5), Integer(6), Integer(7)}]
>>> len(ll) == len(pp) and all(x in pp for x in ll) and all(x in ll for x in␣
↪pp)
True
```

Listing minimal distance-k dominating sets:

```
sage: G = graphs.Grid2dGraph(2, 3)
sage: list(G.minimal_dominating_sets(k=0))
[{(0, 0), (0, 1), (0, 2), (1, 0), (1, 1), (1, 2)}]
sage: list(G.minimal_dominating_sets(k=1))
[{(0, 0), (0, 2), (1, 1)},
 {(0, 1), (1, 1)},
 {(0, 0), (0, 1), (0, 2)},
 {(0, 2), (1, 0)},
 {(0, 0), (1, 2)},
 {(0, 1), (1, 0), (1, 2)},
 {(1, 0), (1, 1), (1, 2)}]
sage: list(G.minimal_dominating_sets(k=2))
[{(0, 0), (1, 2)},
 {(0, 2), (1, 2)},
 {(1, 0), (1, 2)},
 {(0, 1)},
 {(0, 0), (0, 2)},
 {(0, 2), (1, 0)},
 {(0, 0), (1, 0)},
 {(1, 1)}]
sage: list(G.minimal_dominating_sets(k=3))
[{(0, 0)}, {(0, 1)}, {(0, 2)}, {(1, 0)}, {(1, 1)}, {(1, 2)}]
```

```
>>> from sage.all import *
>>> G = graphs.Grid2dGraph(Integer(2), Integer(3))
>>> list(G.minimal_dominating_sets(k=Integer(0)))
[{(0, 0), (0, 1), (0, 2), (1, 0), (1, 1), (1, 2)}]
>>> list(G.minimal_dominating_sets(k=Integer(1)))
[{(0, 0), (0, 2), (1, 1)},
 {(0, 1), (1, 1)},
 {(0, 0), (0, 1), (0, 2)},
 {(0, 2), (1, 0)},
 {(0, 0), (1, 2)},
 {(0, 1), (1, 0), (1, 2)},
 {(1, 0), (1, 1), (1, 2)}]
>>> list(G.minimal_dominating_sets(k=Integer(2)))
[{(0, 0), (1, 2)},
 {(0, 2), (1, 2)},
 {(1, 0), (1, 2)},
 {(0, 1)},
 {(0, 0), (0, 2)},
 {(0, 2), (1, 0)},
 {(0, 0), (1, 0)},
 {(1, 1)}]
>>> list(G.minimal_dominating_sets(k=Integer(3)))
[{(0, 0)}, {(0, 1)}, {(0, 2)}, {(1, 0)}, {(1, 1)}, {(1, 2)}]
```

When parameter `work_on_copy` is `False`, the input graph is modified (relabeled):

```
sage: G = Graph([('A', 'B')])
sage: _ = list(G.minimal_dominating_sets(work_on_copy=True))
```

(continues on next page)

(continued from previous page)

```
sage: set(G) == {'A', 'B'}
True
sage: _ = list(G.minimal_dominating_sets(work_on_copy=False))
sage: set(G) == {'A', 'B'}
False
sage: set(G) == {0, 1}
True
```

```
>>> from sage.all import *
>>> G = Graph([('A', 'B')])
>>> _ = list(G.minimal_dominating_sets(work_on_copy=True))
>>> set(G) == {'A', 'B'}
True
>>> _ = list(G.minimal_dominating_sets(work_on_copy=False))
>>> set(G) == {'A', 'B'}
False
>>> set(G) == {Integer(0), Integer(1)}
True
```

minimal_separators(*G*, *forbidden_vertices=None*)

Return an iterator over the minimal separators of `G`.

A separator in a graph is a set of vertices whose removal increases the number of connected components. In other words, a separator is a vertex cut. This method implements the algorithm proposed in [BBC2000]. It computes the set S of minimal separators of a graph in $O(n^3)$ time per separator, and so overall in $O(n^3|S|)$ time.

Warning

Note that all separators are recorded during the execution of the algorithm and so the memory consumption of this method might be huge.

INPUT:

- `G` – an undirected graph
- `forbidden_vertices` – list (default: `None`); set of vertices to avoid during the search

EXAMPLES:

```
sage: P = graphs.PathGraph(5)
sage: sorted(sorted(sep) for sep in P.minimal_separators())
[[1], [2], [3]]
sage: C = graphs.CycleGraph(6)
sage: sorted(sorted(sep) for sep in C.minimal_separators())
[[0, 2], [0, 3], [0, 4], [1, 3], [1, 4], [1, 5], [2, 4], [2, 5], [3, 5]]
sage: sorted(sorted(sep) for sep in C.minimal_separators(forbidden_
↪vertices=[0]))
[[2], [3], [4]]
sage: sorted(sorted(sep) for sep in (P + C).minimal_separators())
[[1], [2], [3], [5, 7], [5, 8], [5, 9], [6, 8],
 [6, 9], [6, 10], [7, 9], [7, 10], [8, 10]]
sage: sorted(sorted(sep) for sep in (P + C).minimal_separators(forbidden_
```

(continues on next page)

(continued from previous page)

```
↪vertices=[10]))
[[1], [2], [3], [6], [7], [8]]

sage: G = graphs.RandomGNP(10, .3)
sage: all(G.is_vertex_cut(sep) for sep in G.minimal_separators())
True
```

```
>>> from sage.all import *
>>> P = graphs.PathGraph(Integer(5))
>>> sorted(sorted(sep) for sep in P.minimal_separators())
[[1], [2], [3]]
>>> C = graphs.CycleGraph(Integer(6))
>>> sorted(sorted(sep) for sep in C.minimal_separators())
[[0, 2], [0, 3], [0, 4], [1, 3], [1, 4], [1, 5], [2, 4], [2, 5], [3, 5]]
>>> sorted(sorted(sep) for sep in C.minimal_separators(forbidden_
↪vertices=[Integer(0)]))
[[2], [3], [4]]
>>> sorted(sorted(sep) for sep in (P + C).minimal_separators())
[[1], [2], [3], [5, 7], [5, 8], [5, 9], [6, 8],
 [6, 9], [6, 10], [7, 9], [7, 10], [8, 10]]
>>> sorted(sorted(sep) for sep in (P + C).minimal_separators(forbidden_
↪vertices=[Integer(10)]))
[[1], [2], [3], [6], [7], [8]]

>>> G = graphs.RandomGNP(Integer(10), RealNumber('.3'))
>>> all(G.is_vertex_cut(sep) for sep in G.minimal_separators())
True
```

Example of [BPS2010]:

```
sage: G = Graph({'a': ['b', 'k'], 'b': ['c'], 'c': ['d', 'j', 'k'],
....:            'd': ['e', 'f', 'j', 'k'], 'e': ['g'],
....:            'f': ['g', 'j', 'k'], 'g': ['j', 'k'], 'h': ['i', 'j'],
....:            'i': ['k'], 'j': ['k']})
sage: sorted(sorted(sep) for sep in G.minimal_separators())
[['a', 'c'], ['b', 'k'], ['c', 'k'], ['d', 'j', 'k'],
 ['h', 'k'], ['i', 'j'], ['j', 'k']]
```

```
>>> from sage.all import *
>>> G = Graph({'a': ['b', 'k'], 'b': ['c'], 'c': ['d', 'j', 'k'],
...            'd': ['e', 'f', 'j', 'k'], 'e': ['g'],
...            'f': ['g', 'j', 'k'], 'g': ['j', 'k'], 'h': ['i', 'j'],
...            'i': ['k'], 'j': ['k']})
>>> sorted(sorted(sep) for sep in G.minimal_separators())
[['a', 'c'], ['b', 'k'], ['c', 'k'], ['d', 'j', 'k'],
 ['h', 'k'], ['i', 'j'], ['j', 'k']]
```

minimum_outdegree_orientation(*G*, *by_weight*, *weight_function=False*, *check_weight=None*, *solver=True*, *verbose=None*, *integrality_tolerance=0*)

Return an orientation of G with the smallest possible maximum outdegree.

Given a Graph G, it is polynomial to compute an orientation D of the edges of G such that the maximum out-degree in D is minimized. This problem, though, is NP-complete in the weighted case [AMOZ2006].

INPUT:

- `G` – an undirected graph
- `by_weight` – boolean (default: `False`); if `True`, uses the weight given by `weight_function`, or the edge label if `weight_function` is `None`, to compute the orientation. If `False`, gives a weight of 1 to all the edges.
- `weight_function` – function (default: `None`); a function that takes as input an edge `(u, v, l)` and outputs its weight. If not `None`, `by_weight` is automatically set to `True`. If `None` and `by_weight` is `True`, we use the edge label `l`, if `l` is not `None`, else 1 as a weight.
- `check_weight` – boolean (default: `True`); whether to check that the `weight_function` outputs a number for each edge
- `solver` – string (default: `None`); specifies a Mixed Integer Linear Programming (MILP) solver to be used. If set to `None`, the default one is used. For more information on MILP solvers and which default solver is used, see the method `solve` of the class `MixedIntegerLinearProgram`.
- `verbose` – integer (default: 0); sets the level of verbosity. Set to 0 by default, which means quiet.
- `integrality_tolerance` – float; parameter for use with MILP solvers over an inexact base ring; see `MixedIntegerLinearProgram.get_values()`.

EXAMPLES:

Given a complete bipartite graph $K_{n,m}$, the maximum out-degree of an optimal orientation is $\left\lceil \frac{nm}{n+m} \right\rceil$:

```
sage: g = graphs.CompleteBipartiteGraph(3,4)
sage: o = g.minimum_outdegree_orientation()                                 ␣
↪  # needs sage.numerical.mip
sage: max(o.out_degree()) == integer_ceil((4*3)/(3+4))                      ␣
↪  # needs sage.numerical.mip
True
```

```
>>> from sage.all import *
>>> g = graphs.CompleteBipartiteGraph(Integer(3),Integer(4))
>>> o = g.minimum_outdegree_orientation()
↪# needs sage.numerical.mip
>>> max(o.out_degree()) == integer_ceil((Integer(4)*Integer(3))/
↪(Integer(3)+Integer(4)))                          # needs sage.numerical.mip
True
```

Show the influence of edge labels on the solution:

```
sage: g = graphs.PetersenGraph()
sage: o = g.minimum_outdegree_orientation(by_weight=False)
sage: max(o.out_degree())
2
sage: _ = [g.set_edge_label(u, v, 1) for u, v in g.edge_
↪iterator(labels=False)]
sage: o = g.minimum_outdegree_orientation(by_weight=True)
sage: max(o.out_degree())
2
sage: g.set_edge_label(0, 1, 100)
sage: o = g.minimum_outdegree_orientation(by_weight=True)
sage: max(o.out_degree())
3
```

```
>>> from sage.all import *
>>> g = graphs.PetersenGraph()
>>> o = g.minimum_outdegree_orientation(by_weight=False)
>>> max(o.out_degree())
2
>>> _ = [g.set_edge_label(u, v, Integer(1)) for u, v in g.edge_
↪iterator(labels=False)]
>>> o = g.minimum_outdegree_orientation(by_weight=True)
>>> max(o.out_degree())
2
>>> g.set_edge_label(Integer(0), Integer(1), Integer(100))
>>> o = g.minimum_outdegree_orientation(by_weight=True)
>>> max(o.out_degree())
3
```

A `weight_function` can read the weight from anywhere in the edge, and setting it implies `by_weight=True`:

```
sage: g = graphs.PetersenGraph()
sage: _ = [g.set_edge_label(u, v, {'w': 1})
....:      for u, v in g.edge_iterator(labels=False)]
sage: g.set_edge_label(0, 1, {'w': 100})
sage: o = g.minimum_outdegree_orientation(                                  ␣
↪  # needs sage.numerical.mip
....:         weight_function=lambda e: e[2]['w'])
sage: max(o.out_degree())                                                   ␣
↪  # needs sage.numerical.mip
3
```

```
>>> from sage.all import *
>>> g = graphs.PetersenGraph()
>>> _ = [g.set_edge_label(u, v, {'w': Integer(1)})
...      for u, v in g.edge_iterator(labels=False)]
>>> g.set_edge_label(Integer(0), Integer(1), {'w': Integer(100)})
>>> o = g.minimum_outdegree_orientation(
↪# needs sage.numerical.mip
...         weight_function=lambda e: e[Integer(2)]['w'])
>>> max(o.out_degree())
↪# needs sage.numerical.mip
3
```

The `use_edge_labels` keyword is deprecated in favour of `by_weight`:

```
sage: g = graphs.PetersenGraph()
sage: o = g.minimum_outdegree_orientation(use_edge_labels=True)             ␣
↪  # needs sage.numerical.mip
doctest:warning...
DeprecationWarning: use the option 'by_weight' instead of 'use_edge_labels'
See https://github.com/sagemath/sage/issues/42760 for details.
sage: max(o.out_degree())                                                   ␣
↪  # needs sage.numerical.mip
2
```

```
>>> from sage.all import *
>>> g = graphs.PetersenGraph()
>>> o = g.minimum_outdegree_orientation(use_edge_labels=True)
 ↪# needs sage.numerical.mip
doctest:warning...
DeprecationWarning: use the option 'by_weight' instead of 'use_edge_labels'
See https://github.com/sagemath/sage/issues/42760 for details.
>>> max(o.out_degree())
 ↪# needs sage.numerical.mip
2
```

minor(*H*, *solver*, *verbose=None*, *induced=0*, *integrality_tolerance=False*)

Return the vertices of a minor isomorphic to H in the current graph.

We say that a graph G has a H-minor (or that it has a graph isomorphic to H as a minor), if for all $h \in H$, there exist disjoint sets $S_h \subseteq V(G)$ such that once the vertices of each S_h have been merged to create a new graph G', this new graph contains H as a subgraph.

When parameter `induced` is `True`, this method returns an induced minor isomorphic to H, if it exists.

We say that a graph G has an induced H-minor (or that it has a graph isomorphic to H as an induced minor), if H can be obtained from an induced subgraph of G by contracting edges. Otherwise, G is said to be H-induced minor-free.

For more information, see the Wikipedia article Minor_(graph_theory).

INPUT:

- `H` – the minor to find for in the current graph
- `solver` – string (default: `None`); specifies a Mixed Integer Linear Programming (MILP) solver to be used. If set to `None`, the default one is used. For more information on MILP solvers and which default solver is used, see the method `solve` of the class `MixedIntegerLinearProgram`.
- `verbose` – integer (default: 0); sets the level of verbosity. Set to 0 by default, which means quiet.
- `integrality_tolerance` – float; parameter for use with MILP solvers over an inexact base ring; see `MixedIntegerLinearProgram.get_values()`.
- `induced` – boolean (default: `False`); if `True`, returns an induced minor isomorphic to H if it exists, and raises a `ValueError` otherwise.

OUTPUT:

A dictionary associating to each vertex of H the set of vertices in the current graph representing it.

ALGORITHM:

Mixed Integer Linear Programming

COMPLEXITY:

Theoretically, when H is fixed, testing for the existence of a H-minor is polynomial. The known algorithms are highly exponential in H, though.

Note

This function can be expected to be *very* slow, especially where the minor does not exist.

EXAMPLES:

Trying to find a minor isomorphic to K_4 in the 4×4 grid:

```
sage: g = graphs.GridGraph([4,4])
sage: h = graphs.CompleteGraph(4)
sage: L = g.minor(h)
sage: gg = g.subgraph(flatten(L.values(), max_level = 1))
sage: _ = [gg.merge_vertices(l) for l in L.values() if len(l)>1]
sage: gg.is_isomorphic(h)
True
```

```
>>> from sage.all import *
>>> g = graphs.GridGraph([Integer(4),Integer(4)])
>>> h = graphs.CompleteGraph(Integer(4))
>>> L = g.minor(h)
>>> gg = g.subgraph(flatten(L.values(), max_level = Integer(1)))
>>> _ = [gg.merge_vertices(l) for l in L.values() if len(l)>Integer(1)]
>>> gg.is_isomorphic(h)
True
```

We can also try to prove this way that the Petersen graph is not planar, as it has a K_5 minor:

```
sage: g = graphs.PetersenGraph()
sage: K5_minor = g.minor(graphs.CompleteGraph(5))    # long time              #␣
↪needs sage.numerical.mip
```

```
>>> from sage.all import *
>>> g = graphs.PetersenGraph()
>>> K5_minor = g.minor(graphs.CompleteGraph(Integer(5)))    # long time       ␣
↪     # needs sage.numerical.mip
```

And even a $K_{3,3}$ minor:

```
sage: K33_minor = g.minor(graphs.CompleteBipartiteGraph(3,3))        # long␣
↪time, needs sage.numerical.mip
```

```
>>> from sage.all import *
>>> K33_minor = g.minor(graphs.CompleteBipartiteGraph(Integer(3),Integer(3)))␣
↪      # long time, needs sage.numerical.mip
```

(It is much faster to use the linear-time test of planarity in this situation, though.)

As there is no cycle in a tree, looking for a K_3 minor is useless. This function will raise an exception in this case:

```
sage: g = graphs.RandomGNP(20, .5)
sage: g = g.subgraph(edges=g.min_spanning_tree())
sage: g.is_tree()
```

(continues on next page)

(continued from previous page)

```
True
sage: L = g.minor(graphs.CompleteGraph(3))                                  #␣
↪needs sage.numerical.mip
Traceback (most recent call last):
...
ValueError: This graph has no minor isomorphic to H !
```

```
>>> from sage.all import *
>>> g = graphs.RandomGNP(Integer(20), RealNumber('.5'))
>>> g = g.subgraph(edges=g.min_spanning_tree())
>>> g.is_tree()
True
>>> L = g.minor(graphs.CompleteGraph(Integer(3)))                           ␣
↪     # needs sage.numerical.mip
Traceback (most recent call last):
...
ValueError: This graph has no minor isomorphic to H !
```

Trying to find an induced minor isomorphic to C_5 in a graph containing an induced C_6:

```
sage: g = graphs.CycleGraph(6)
sage: for i in range(randint(10, 30)):
....:     g.add_edge(randint(0, 5), g.add_vertex())
sage: h = graphs.CycleGraph(5)
sage: L = g.minor(h, induced=True)
sage: gg = g.subgraph(flatten(L.values(), max_level=1))
sage: _ = [gg.merge_vertices(l) for l in L.values() if len(l) > 1]
sage: gg.is_isomorphic(h)
True
```

```
>>> from sage.all import *
>>> g = graphs.CycleGraph(Integer(6))
>>> for i in range(randint(Integer(10), Integer(30))):
...     g.add_edge(randint(Integer(0), Integer(5)), g.add_vertex())
>>> h = graphs.CycleGraph(Integer(5))
>>> L = g.minor(h, induced=True)
>>> gg = g.subgraph(flatten(L.values(), max_level=Integer(1)))
>>> _ = [gg.merge_vertices(l) for l in L.values() if len(l) > Integer(1)]
>>> gg.is_isomorphic(h)
True
```

modular_decomposition(*algorithm=None*, *style='tuple'*)

Return the modular decomposition of the current graph.

A module of an undirected graph is a subset of vertices such that every vertex outside the module is either connected to all members of the module or to none of them. Every graph that has a nontrivial module can be partitioned into modules, and the increasingly fine partitions into modules form a tree. The `modular_decomposition` method returns that tree.

INPUT:

- `algorithm` – string (default: `None`); the algorithm to use among:
  - `None` or `'corneil_habib_paul_tedder'` – will use the Corneil-Habib-Paul-Tedder algorithm

from [TCHP2008], its complexity is linear in the number of vertices and edges.

- `'habib_maurer'` – will use the Habib-Maurer algorithm from [HM1979], its complexity is cubic in the number of vertices.

- `style` – string (default: `'tuple'`); specifies the output format:
  - `'tuple'` – as nested tuples
  - `'tree'` – as `LabelledRootedTree`

OUTPUT:

The modular decomposition tree, either as nested tuples (if `style='tuple'`) or as an object of `LabelledRootedTree` (if `style='tree'`)

Crash course on modular decomposition:

A module M of a graph G is a proper subset of its vertices such that for all $u \in V(G) - M, v, w \in M$ the relation $u \sim v \Leftrightarrow u \sim w$ holds, where $\sim$ denotes the adjacency relation in G. Equivalently, $M \subset V(G)$ is a module if all its vertices have the same adjacency relations with each vertex outside of the module (vertex by vertex).

Hence, for a set like a module, it is very easy to encode the information of the adjacencies between the vertices inside and outside the module – we can actually add a new vertex v_M to our graph representing our module M, and let v_M be adjacent to $u \in V(G) - M$ if and only if some $v \in M$ (and hence all the vertices contained in the module) is adjacent to u. We can now independently (and recursively) study the structure of our module M and the new graph $G - M + \{v_M\}$, without any loss of information.

Here are two very simple modules :

- A connected component C (or the union of some –but not all– of them) of a disconnected graph G, for instance, is a module, as no vertex of C has a neighbor outside of it.
- An anticomponent C (or the union of some –but not all– of them) of a non-anticonnected graph G, for the same reason (it is just the complement of the previous graph !).

These modules being of special interest, the disjoint union of graphs is called a Parallel composition, and the complement of a disjoint union is called a Series composition. A graph whose only modules are singletons is called Prime.

For more information on modular decomposition, in particular for an explanation of the terms “Parallel,” “Prime” and “Series,” see the Wikipedia article Modular_decomposition.

You may also be interested in the survey from Michel Habib and Christophe Paul entitled “A survey on Algorithmic aspects of modular decomposition” [HP2010].

EXAMPLES:

The Bull Graph is prime:

```
sage: graphs.BullGraph().modular_decomposition()
(PRIME, [1, 2, 0, 3, 4])
```

```
>>> from sage.all import *
>>> graphs.BullGraph().modular_decomposition()
(PRIME, [1, 2, 0, 3, 4])
```

The Petersen Graph too:

```
sage: graphs.PetersenGraph().modular_decomposition()
(PRIME, [1, 4, 5, 0, 6, 2, 3, 9, 7, 8])
```

```
>>> from sage.all import *
>>> graphs.PetersenGraph().modular_decomposition()
(PRIME, [1, 4, 5, 0, 6, 2, 3, 9, 7, 8])
```

Graph from the Wikipedia article Modular_decomposition:

```
sage: G = Graph('Jv\\zoKF@wN?', format='graph6')
sage: G.relabel([1..11])
sage: G.modular_decomposition()
(PRIME,
 [(SERIES, [4, (PARALLEL, [2, 3])]),
  1,
  5,
  (PARALLEL, [6, 7]),
  (SERIES, [(PARALLEL, [10, 11]), 9, 8])])
```

```
>>> from sage.all import *
>>> G = Graph('Jv\\zoKF@wN?', format='graph6')
>>> G.relabel((ellipsis_range(Integer(1),Ellipsis,Integer(11))))
>>> G.modular_decomposition()
(PRIME,
 [(SERIES, [4, (PARALLEL, [2, 3])]),
  1,
  5,
  (PARALLEL, [6, 7]),
  (SERIES, [(PARALLEL, [10, 11]), 9, 8])])
```

This a clique on 5 vertices with 2 pendant edges, though, has a more interesting decomposition:

```
sage: g = graphs.CompleteGraph(5)
sage: g.add_edge(0,5)
sage: g.add_edge(0,6)
sage: g.modular_decomposition()
(SERIES, [(PARALLEL, [(SERIES, [3, 4, 2, 1]), 5, 6]), 0])
```

```
>>> from sage.all import *
>>> g = graphs.CompleteGraph(Integer(5))
>>> g.add_edge(Integer(0),Integer(5))
>>> g.add_edge(Integer(0),Integer(6))
>>> g.modular_decomposition()
(SERIES, [(PARALLEL, [(SERIES, [3, 4, 2, 1]), 5, 6]), 0])
```

Turán graphs are co-graphs:

```
sage: graphs.TuranGraph(11, 3).modular_decomposition()
(SERIES,
 [(PARALLEL, [7, 8, 9, 10]), (PARALLEL, [3, 4, 5, 6]), (PARALLEL, [0, 1, 2])])
```

```
>>> from sage.all import *
>>> graphs.TuranGraph(Integer(11), Integer(3)).modular_decomposition()
(SERIES,
 [(PARALLEL, [7, 8, 9, 10]), (PARALLEL, [3, 4, 5, 6]), (PARALLEL, [0, 1, 2])])
```

We can choose output to be a LabelledRootedTree:

```
sage: g.modular_decomposition(style='tree')
SERIES[0[], PARALLEL[5[], 6[], SERIES[1[], 2[], 3[], 4[]]]]
sage: ascii_art(g.modular_decomposition(algorithm="habib_maurer",style='tree
↪'))
  __SERIES
 /       /
0   ___PARALLEL
   / /      /
  5 6   __SERIES
       / / / /
      1 2 3 4
```

```
>>> from sage.all import *
>>> g.modular_decomposition(style='tree')
SERIES[0[], PARALLEL[5[], 6[], SERIES[1[], 2[], 3[], 4[]]]]
>>> ascii_art(g.modular_decomposition(algorithm="habib_maurer",style='tree'))
  __SERIES
 /       /
0   ___PARALLEL
   / /      /
  5 6   __SERIES
       / / / /
      1 2 3 4
```

ALGORITHM:

This function can use either the algorithm of D. Corneil, M. Habib, C. Paul and M. Tedder [TCHP2008] or the algorithm of M. Habib and M. Maurer [HM1979].

See also

- *is_prime()* – tests whether a graph is prime
- LabelledRootedTree.
- *corneil_habib_paul_tedder_algorithm()*
- *habib_maurer_algorithm()*

Note

A buggy implementation of the linear time algorithm from [TCHP2008] was removed in Sage 9.7, see Issue #25872. A new implementation was reintroduced in Sage 10.6 after some corrections to the original algorithm, see Issue #39038.

most_common_neighbors(*nonedgesonly=True*)

Return vertex pairs with maximal number of common neighbors.

This method is only valid for simple (no loops, no multiple edges) graphs with order ≥ 2

INPUT:

- `nonedgesonly` – boolean (default: `True`); if `True`, assigns 0 value to adjacent vertices

OUTPUT: list of tuples of edge pairs

EXAMPLES:

The maximum common neighbor (non-adjacent) pairs for a straight linear 2-tree

```
sage: G1 = Graph([(0,1),(0,2),(1,2),(1,3),(3,5),(2,4),(2,3),(3,4),(4,5)])
sage: G1.most_common_neighbors()                                            #␣
↪needs sage.modules
[(0, 3), (1, 4), (2, 5)]
```

```
>>> from sage.all import *
>>> G1 = Graph([(Integer(0),Integer(1)),(Integer(0),Integer(2)),(Integer(1),
↪Integer(2)),(Integer(1),Integer(3)),(Integer(3),Integer(5)),(Integer(2),
↪Integer(4)),(Integer(2),Integer(3)),(Integer(3),Integer(4)),(Integer(4),
↪Integer(5))])
>>> G1.most_common_neighbors()                                              #␣
↪needs sage.modules
[(0, 3), (1, 4), (2, 5)]
```

If we include non-adjacent pairs

```
sage: G1.most_common_neighbors(nonedgesonly=False)                          #␣
↪needs sage.modules
[(0, 3), (1, 2), (1, 4), (2, 3), (2, 5), (3, 4)]
```

```
>>> from sage.all import *
>>> G1.most_common_neighbors(nonedgesonly=False)                            #␣
↪needs sage.modules
[(0, 3), (1, 2), (1, 4), (2, 3), (2, 5), (3, 4)]
```

The common neighbors matrix for a fan on 6 vertices counting only non-adjacent vertex pairs

```
sage: H = Graph([(0,1),(0,2),(0,3),(0,4),(0,5),(0,6),(1,2),(2,3),(3,4),(4,5)])
sage: H.most_common_neighbors()                                             #␣
↪needs sage.modules
[(1, 3), (2, 4), (3, 5)]
```

```
>>> from sage.all import *
>>> H = Graph([(Integer(0),Integer(1)),(Integer(0),Integer(2)),(Integer(0),
↪Integer(3)),(Integer(0),Integer(4)),(Integer(0),Integer(5)),(Integer(0),
↪Integer(6)),(Integer(1),Integer(2)),(Integer(2),Integer(3)),(Integer(3),
↪Integer(4)),(Integer(4),Integer(5))])
>>> H.most_common_neighbors()                                               #␣
↪needs sage.modules
[(1, 3), (2, 4), (3, 5)]
```

See also

- *common_neighbors_matrix()* – a similar method giving a matrix of number of common neighbors

orient(*G*, *f*, *weighted=None*, *data_structure=None*, *sparse=None*, *immutable=None*, *hash_labels=None*)

Return an oriented version of G according the input function f.

INPUT:

- `G` – an undirected graph
- `f` – a function that inputs an edge and outputs an orientation of this edge
- `weighted` – boolean (default: `None`); weightedness for the oriented digraph. By default (`None`), the graph and its orientation will behave the same.
- `sparse` – boolean (default: `None`); `sparse=True` is an alias for `data_structure="sparse"`, and `sparse=False` is an alias for `data_structure="dense"`. Only used when `data_structure=None`.
- `data_structure` – string (default: `None`); one of `'sparse'`, `'static_sparse'`, or `'dense'`. See the documentation of *DiGraph*.
- `immutable` – boolean (default: `None`); whether to create a mutable/immutable digraph. Only used when `data_structure=None`.
  - `immutable=None` (default) means that the graph and its orientation will behave the same way.
  - `immutable=True` is a shortcut for `data_structure='static_sparse'`
  - `immutable=False` means that the created digraph is mutable. When used to orient an immutable graph, the data structure used is `'sparse'` unless anything else is specified.
- `hash_labels` – boolean (default: `None`); whether to include edge labels during hashing of the oriented digraph. This parameter defaults to `True` if the graph is weighted. This parameter is ignored when parameter `immutable` is not `True`. Beware that trying to hash unhashable labels will raise an error.

OUTPUT: a *DiGraph* object

Note

This method behaves similarly to method *copy()*. That is, the returned digraph uses the same data structure by default, unless the user asks to use another data structure, and the attributes of the input graph are copied.

EXAMPLES:

```
sage: G = graphs.CycleGraph(4); G
Cycle graph: Graph on 4 vertices
sage: D = G.orient(lambda e:e if e[0] < e[1] else (e[1], e[0], e[2])); D
Orientation of Cycle graph: Digraph on 4 vertices
sage: sorted(D.edges(labels=False))
[(0, 1), (0, 3), (1, 2), (2, 3)]
```

```
>>> from sage.all import *
>>> G = graphs.CycleGraph(Integer(4)); G
Cycle graph: Graph on 4 vertices
>>> D = G.orient(lambda e:e if e[Integer(0)] < e[Integer(1)] else␣
↪(e[Integer(1)], e[Integer(0)], e[Integer(2)])); D
Orientation of Cycle graph: Digraph on 4 vertices
>>> sorted(D.edges(labels=False))
[(0, 1), (0, 3), (1, 2), (2, 3)]
```

orientations(*G*, *data_structure=None*, *sparse=None*)

Return an iterator over orientations of G.

An *orientation* of an undirected graph is a directed graph such that every edge is assigned a direction. Hence there are 2^s oriented digraphs for a simple graph with s edges.

INPUT:

- `G` – an undirected graph
- `data_structure` – one of `'sparse'`, `'static_sparse'`, or `'dense'`; see the documentation of *Graph* or *DiGraph*; default is the data structure of G
- `sparse` – boolean (default: `None`); `sparse=True` is an alias for `data_structure="sparse"`, and `sparse=False` is an alias for `data_structure="dense"`. By default (`None`), guess the most suitable data structure.

Warning

This always considers multiple edges of graphs as distinguishable, and hence, may have repeated digraphs.

See also

- *strong_orientation()*
- *strong_orientations_iterator()*
- *nauty_directg()*
- *random_orientation()*

EXAMPLES:

```
sage: G = Graph([[1,2,3], [(1, 2, 'a'), (1, 3, 'b')]], format='vertices_and_
↪edges')
sage: it = G.orientations()
sage: D = next(it)
sage: D.edges(sort=True)
[(1, 2, 'a'), (1, 3, 'b')]
sage: D = next(it)
sage: D.edges(sort=True)
[(1, 2, 'a'), (3, 1, 'b')]
```

```
>>> from sage.all import *
>>> G = Graph([[Integer(1),Integer(2),Integer(3)], [(Integer(1), Integer(2),
↪'a'), (Integer(1), Integer(3), 'b')]], format='vertices_and_edges')
>>> it = G.orientations()
>>> D = next(it)
>>> D.edges(sort=True)
[(1, 2, 'a'), (1, 3, 'b')]
>>> D = next(it)
>>> D.edges(sort=True)
[(1, 2, 'a'), (3, 1, 'b')]
```

pathwidth(*G*, *k=None*, *certificate=False*, *algorithm='BAB'*, *verbose=False*, *max_prefix_length=20*, *max_prefix_number=1000000*, *immutable=None*, *solver=None*)

Compute the pathwidth of `G` (and provides a decomposition).

INPUT:

- `G` – a graph
- `k` – integer (default: `None`); the width to be considered. When `k` is an integer, the method checks that the graph has pathwidth $\leq k$. If `k` is `None` (default), the method computes the optimal pathwidth.
- `certificate` – boolean (default: `False`); whether to return the path-decomposition itself
- `algorithm` – string (default: `'BAB'`); algorithm to use among:
  - `'BAB'` – use a branch-and-bound algorithm. This algorithm has no size restriction but could take a very long time on large graphs. It can also be used to test is the input graph has pathwidth $\leq k$, in which cas it will return the first found solution with width $\leq k$ is `certificate==True`.
  - `exponential` – use an exponential time and space algorithm. This algorithm only works of graphs on less than 32 vertices
  - `MILP` – use a mixed integer linear programming formulation. This algorithm has no size restriction but could take a very long time
- `verbose` – boolean (default: `False`); whether to display information on the computations
- `max_prefix_length` – integer (default: 20); limits the length of the stored prefixes to prevent storing too many prefixes. This parameter is used only when `algorithm=="BAB"`.
- `max_prefix_number` – integer (default: 10**6); upper bound on the number of stored prefixes used to prevent using too much memory. This parameter is used only when `algorithm=="BAB"`.
- `immutable` – boolean (default: `None`); whether to create a mutable/immutable graph. `immutable=None` (default) means that the graph and its path decomposition will behave the same way.
- `solver` – string (default: `None`); specifies a Mixed Integer Linear Programming (MILP) solver to be used. If set to `None`, the default one is used. For more information on MILP solvers and which default solver is used, see the method `solve` of the class `MixedIntegerLinearProgram`.

OUTPUT:

Return the pathwidth of `self`. When `k` is specified, it returns `False` when no path-decomposition of width $\leq k$ exists or `True` otherwise. When `certificate=True`, the path-decomposition is also returned.

See also

- *`Graph.treewidth()`* – computes the treewidth of a graph
- *`vertex_separation()`* – computes the vertex separation of a (di)graph

EXAMPLES:

The pathwidth of a cycle is equal to 2:

```
sage: g = graphs.CycleGraph(6)
sage: g.pathwidth()
2
sage: pw, decomp = g.pathwidth(certificate=True)
```

(continues on next page)

(continued from previous page)

```
sage: sorted(decomp, key=str)
[{0, 1, 5}, {1, 2, 5}, {2, 3, 4}, {2, 4, 5}]
```

```
>>> from sage.all import *
>>> g = graphs.CycleGraph(Integer(6))
>>> g.pathwidth()
2
>>> pw, decomp = g.pathwidth(certificate=True)
>>> sorted(decomp, key=str)
[{0, 1, 5}, {1, 2, 5}, {2, 3, 4}, {2, 4, 5}]
```

The pathwidth of a Petersen graph is 5:

```
sage: g = graphs.PetersenGraph()
sage: g.pathwidth()
5
sage: g.pathwidth(k=2)
False
sage: g.pathwidth(k=6)
True
sage: g.pathwidth(k=6, certificate=True)
(True, Graph on 5 vertices)
```

```
>>> from sage.all import *
>>> g = graphs.PetersenGraph()
>>> g.pathwidth()
5
>>> g.pathwidth(k=Integer(2))
False
>>> g.pathwidth(k=Integer(6))
True
>>> g.pathwidth(k=Integer(6), certificate=True)
(True, Graph on 5 vertices)
```

perfect_matchings(*G*, *labels=False*)

Return an iterator over all perfect matchings of the graph.

ALGORITHM:

Choose a vertex v, then recurse through all edges incident to v, removing one edge at a time whenever an edge is added to a matching.

INPUT:

- `labels` – boolean (default: `False`); when `True`, the edges in each perfect matching are triples (containing the label as the third element), otherwise the edges are pairs.

See also

matching()

EXAMPLES:

```
sage: G=graphs.GridGraph([2,3])
sage: for m in G.perfect_matchings():
....:     print(sorted(m))
[((0, 0), (0, 1)), ((0, 2), (1, 2)), ((1, 0), (1, 1))]
[((0, 0), (1, 0)), ((0, 1), (0, 2)), ((1, 1), (1, 2))]
[((0, 0), (1, 0)), ((0, 1), (1, 1)), ((0, 2), (1, 2))]

sage: G = graphs.CompleteGraph(4)
sage: for m in G.perfect_matchings(labels=True):
....:     print(sorted(m))
[(0, 1, None), (2, 3, None)]
[(0, 2, None), (1, 3, None)]
[(0, 3, None), (1, 2, None)]

sage: G = Graph([[1,-1,'a'], [2,-2, 'b'], [1,-2,'x'], [2,-1,'y']])
sage: sorted(sorted(m) for m in G.perfect_matchings(labels=True))
[[(-2, 1, 'x'), (-1, 2, 'y')], [(-2, 2, 'b'), (-1, 1, 'a')]]

sage: G = graphs.CompleteGraph(8)
sage: mpc = G.matching_polynomial().coefficients(sparse=False)[0]           #␣
↪needs sage.libs.flint
sage: len(list(G.perfect_matchings())) == mpc                               #␣
↪needs sage.libs.flint
True

sage: G = graphs.PetersenGraph().copy(immutable=True)
sage: [sorted(m) for m in G.perfect_matchings()]
[[(0, 1), (2, 3), (4, 9), (5, 7), (6, 8)],
    [(0, 1), (2, 7), (3, 4), (5, 8), (6, 9)],
    [(0, 4), (1, 2), (3, 8), (5, 7), (6, 9)],
    [(0, 4), (1, 6), (2, 3), (5, 8), (7, 9)],
    [(0, 5), (1, 2), (3, 4), (6, 8), (7, 9)],
    [(0, 5), (1, 6), (2, 7), (3, 8), (4, 9)]]

sage: list(Graph().perfect_matchings())
[[]]

sage: G = graphs.CompleteGraph(5)
sage: list(G.perfect_matchings())
[]
```

```
>>> from sage.all import *
>>> G=graphs.GridGraph([Integer(2),Integer(3)])
>>> for m in G.perfect_matchings():
...     print(sorted(m))
[((0, 0), (0, 1)), ((0, 2), (1, 2)), ((1, 0), (1, 1))]
[((0, 0), (1, 0)), ((0, 1), (0, 2)), ((1, 1), (1, 2))]
[((0, 0), (1, 0)), ((0, 1), (1, 1)), ((0, 2), (1, 2))]

>>> G = graphs.CompleteGraph(Integer(4))
>>> for m in G.perfect_matchings(labels=True):
...     print(sorted(m))
```

(continues on next page)

(continued from previous page)

```
[(0, 1, None), (2, 3, None)]
[(0, 2, None), (1, 3, None)]
[(0, 3, None), (1, 2, None)]

>>> G = Graph([[Integer(1),-Integer(1),'a'], [Integer(2),-Integer(2), 'b'],␣
↪[Integer(1),-Integer(2),'x'], [Integer(2),-Integer(1),'y']])
>>> sorted(sorted(m) for m in G.perfect_matchings(labels=True))
[[(-2, 1, 'x'), (-1, 2, 'y')], [(-2, 2, 'b'), (-1, 1, 'a')]]

>>> G = graphs.CompleteGraph(Integer(8))
>>> mpc = G.matching_polynomial().coefficients(sparse=False)[Integer(0)]      ␣
↪     # needs sage.libs.flint
>>> len(list(G.perfect_matchings())) == mpc                                   #␣
↪needs sage.libs.flint
True

>>> G = graphs.PetersenGraph().copy(immutable=True)
>>> [sorted(m) for m in G.perfect_matchings()]
[[(0, 1), (2, 3), (4, 9), (5, 7), (6, 8)],
    [(0, 1), (2, 7), (3, 4), (5, 8), (6, 9)],
    [(0, 4), (1, 2), (3, 8), (5, 7), (6, 9)],
    [(0, 4), (1, 6), (2, 3), (5, 8), (7, 9)],
    [(0, 5), (1, 2), (3, 4), (6, 8), (7, 9)],
    [(0, 5), (1, 6), (2, 7), (3, 8), (4, 9)]]

>>> list(Graph().perfect_matchings())
[[]]

>>> G = graphs.CompleteGraph(Integer(5))
>>> list(G.perfect_matchings())
[]
```

periphery(*by_weight=False*, *algorithm=None*, *weight_function=None*, *check_weight=True*)

Return the set of vertices in the periphery of the graph.

The periphery is the set of vertices whose eccentricity is equal to the diameter of the graph, i.e., achieving the maximum eccentricity.

For more information and examples on how to use input variables, see *shortest_paths()* and *eccentricity()*

INPUT:

- `by_weight` – boolean (default: `False`); if `True`, edge weights are taken into account; if `False`, all edges have weight 1
- `algorithm` – string (default: `None`); see method *eccentricity()* for the list of available algorithms
- `weight_function` – function (default: `None`); a function that takes as input an edge `(u, v, l)` and outputs its weight. If not `None`, `by_weight` is automatically set to `True`. If `None` and `by_weight` is `True`, we use the edge label `l` as a weight, if `l` is not `None`, else `1` as a weight.
- `check_weight` – boolean (default: `True`); if `True`, we check that the `weight_function` outputs a number for each edge

EXAMPLES:

```
sage: G = graphs.DiamondGraph()
sage: G.periphery()
[0, 3]
sage: P = graphs.PetersenGraph()
sage: P.subgraph(P.periphery()) == P
True
sage: S = graphs.StarGraph(19)
sage: S.periphery()
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19]
sage: G = Graph()
sage: G.periphery()
[]
sage: G.add_vertex()
0
sage: G.periphery()
[0]
```

```
>>> from sage.all import *
>>> G = graphs.DiamondGraph()
>>> G.periphery()
[0, 3]
>>> P = graphs.PetersenGraph()
>>> P.subgraph(P.periphery()) == P
True
>>> S = graphs.StarGraph(Integer(19))
>>> S.periphery()
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19]
>>> G = Graph()
>>> G.periphery()
[]
>>> G.add_vertex()
0
>>> G.periphery()
[0]
```

private_neighbors(*G*, *vertex*, *dom*)

Return the private neighbors of a vertex with respect to other vertices.

A private neighbor of a vertex v with respect to a vertex subset D is a closed neighbor of v that is not dominated by a vertex of $D \setminus \{v\}$.

INPUT:

- `vertex` – a vertex of `G`
- `dom` – iterable of vertices of `G`; the vertices possibly stealing private neighbors from `vertex`

OUTPUT:

Return the closed neighbors of `vertex` that are not closed neighbors of any other vertex of `dom`.

EXAMPLES:

```
sage: g = graphs.PathGraph(5)
sage: list(g.private_neighbors(1, [1, 3, 4]))
[1, 0]
```

(continues on next page)

(continued from previous page)

```
sage: list(g.private_neighbors(1, [3, 4]))
[1, 0]

sage: list(g.private_neighbors(1, [3, 4, 0]))
[]
```

```
>>> from sage.all import *
>>> g = graphs.PathGraph(Integer(5))
>>> list(g.private_neighbors(Integer(1), [Integer(1), Integer(3),␣
↪Integer(4)]))
[1, 0]

>>> list(g.private_neighbors(Integer(1), [Integer(3), Integer(4)]))
[1, 0]

>>> list(g.private_neighbors(Integer(1), [Integer(3), Integer(4),␣
↪Integer(0)]))
[]
```

radius(*by_weight=False*, *algorithm='DHV'*, *weight_function=None*, *check_weight=True*)

Return the radius of the graph.

The radius is defined to be the minimum eccentricity of any vertex, where the eccentricity is the maximum distance to any other vertex. For more information and examples on how to use input variables, see *shortest_paths()* and *eccentricity()*

INPUT:

- `by_weight` – boolean (default: `False`); if `True`, edge weights are taken into account; if `False`, all edges have weight 1
- `algorithm` – string (default: `'DHV'`)
  - `'DHV'` – radius computation is done using the algorithm proposed in [Dragan2018]. Works for graph with nonnegative edge weights For more information see method *sage.graphs.distances_all_pairs.radius_DHV()* and *sage.graphs.base.boost_graph.radius_DHV()*.
  - see method *eccentricity()* for the list of remaining algorithms
- `weight_function` – function (default: `None`); a function that takes as input an edge `(u, v, l)` and outputs its weight. If not `None`, `by_weight` is automatically set to `True`. If `None` and `by_weight` is `True`, we use the edge label `l` as a weight, if `l` is not `None`, else `1` as a weight.
- `check_weight` – boolean (default: `True`); if `True`, we check that the `weight_function` outputs a number for each edge

EXAMPLES:

The more symmetric a graph is, the smaller (diameter - radius) is:

```
sage: G = graphs.BarbellGraph(9, 3)
sage: G.radius()
3
sage: G.diameter()
6
```

```
>>> from sage.all import *
>>> G = graphs.BarbellGraph(Integer(9), Integer(3))
>>> G.radius()
3
>>> G.diameter()
6
```

```
sage: G = graphs.OctahedralGraph()
sage: G.radius()
2
sage: G.diameter()
2
```

```
>>> from sage.all import *
>>> G = graphs.OctahedralGraph()
>>> G.radius()
2
>>> G.diameter()
2
```

random_orientation(*G*)

Return a random orientation of a graph G.

An *orientation* of an undirected graph is a directed graph such that every edge is assigned a direction. Hence there are 2^m oriented digraphs for a simple graph with m edges.

INPUT:

- `G` – a Graph

EXAMPLES:

```
sage: from sage.graphs.orientations import random_orientation
sage: G = graphs.PetersenGraph()
sage: D = random_orientation(G)
sage: D.order() == G.order(), D.size() == G.size()
(True, True)
```

```
>>> from sage.all import *
>>> from sage.graphs.orientations import random_orientation
>>> G = graphs.PetersenGraph()
>>> D = random_orientation(G)
>>> D.order() == G.order(), D.size() == G.size()
(True, True)
```

See also

- *orientations()*
- *strong_orientation()*
- *strong_orientations_iterator()*
- *nauty_directg()*

random_spanning_tree(*G*, *output_as_graph=False*, *by_weight=False*, *weight_function=None*, *check_weight=True*)

Return a random spanning tree of the graph.

This uses the Aldous-Broder algorithm ([Bro1989], [Ald1990]) to generate a random spanning tree with the uniform distribution, as follows.

Start from any vertex. Perform a random walk by choosing at every step one neighbor uniformly at random. Every time a new vertex j is met, add the edge (i, j) to the spanning tree, where i is the previous vertex in the random walk.

When `by_weight` is `True` or a weight function is given, the selection of the neighbor is done proportionaly to the edge weights.

INPUT:

- `G` – an undirected graph
- `output_as_graph` – boolean (default: `False`); whether to return a list of edges or a graph
- `by_weight` – boolean (default: `False`); if `True`, the edges in the graph are weighted, otherwise all edges have weight 1
- `weight_function` – function (default: `None`); a function that takes as input an edge `(u, v, l)` and outputs its weight. If not `None`, `by_weight` is automatically set to `True`. If `None` and `by_weight` is `True`, we use the edge label `l` , if `l` is not `None`, else `1` as a weight. The `weight_function` can be used to transform the label into a weight (note that, if the weight returned is not convertible to a float, an error is raised)
- `check_weight` – boolean (default: `True`); whether to check that the `weight_function` outputs a number for each edge

See also

number_of_spanning_trees() and *spanning_trees()*

EXAMPLES:

```
sage: G = graphs.TietzeGraph()
sage: G.random_spanning_tree(output_as_graph=True)
Graph on 12 vertices
sage: rg = G.random_spanning_tree(); rg # random
[(0, 9),
(9, 11),
(0, 8),
(8, 7),
(7, 6),
(7, 2),
(2, 1),
(1, 5),
(9, 10),
(5, 4),
(2, 3)]
sage: Graph(rg).is_tree()
True
```

```
>>> from sage.all import *
>>> G = graphs.TietzeGraph()
>>> G.random_spanning_tree(output_as_graph=True)
Graph on 12 vertices
>>> rg = G.random_spanning_tree(); rg # random
[(0, 9),
(9, 11),
(0, 8),
(8, 7),
(7, 6),
(7, 2),
(2, 1),
(1, 5),
(9, 10),
(5, 4),
(2, 3)]
>>> Graph(rg).is_tree()
True
```

A visual example for the grid graph:

```
sage: G = graphs.Grid2dGraph(6, 6)
sage: pos = G.get_pos()
sage: T = G.random_spanning_tree(True)
sage: T.set_pos(pos)
sage: T.show(vertex_labels=False)
↪  # needs sage.plot
```

```
>>> from sage.all import *
>>> G = graphs.Grid2dGraph(Integer(6), Integer(6))
>>> pos = G.get_pos()
>>> T = G.random_spanning_tree(True)
>>> T.set_pos(pos)
>>> T.show(vertex_labels=False)
↪# needs sage.plot
```

We can also use edge weights to change the probability of returning a spanning tree:

```
sage: def foo(G, k):
....:     S = set()
....:     for _ in range(k):
....:         E = G.random_spanning_tree(by_weight=True)
....:         S.add(Graph(E).graph6_string())
....:     return S
sage: K3 = graphs.CompleteGraph(3)
sage: for u, v in K3.edges(sort=True, labels=False):
....:     K3.set_edge_label(u, v, randint(1, 2))
sage: foo(K3, 100) == {'BW', 'Bg', 'Bo'}  # random
True
sage: K4 = graphs.CompleteGraph(4)
sage: for u, v in K4.edges(sort=True, labels=False):
....:     K4.set_edge_label(u, v, randint(1, 2))
sage: print(len(foo(K4, 100)))  # random
```

(continues on next page)

(continued from previous page)

```
16
```

```
>>> from sage.all import *
>>> def foo(G, k):
...     S = set()
...     for _ in range(k):
...         E = G.random_spanning_tree(by_weight=True)
...         S.add(Graph(E).graph6_string())
...     return S
>>> K3 = graphs.CompleteGraph(Integer(3))
>>> for u, v in K3.edges(sort=True, labels=False):
...     K3.set_edge_label(u, v, randint(Integer(1), Integer(2)))
>>> foo(K3, Integer(100)) == {'BW', 'Bg', 'Bo'}  # random
True
>>> K4 = graphs.CompleteGraph(Integer(4))
>>> for u, v in K4.edges(sort=True, labels=False):
...     K4.set_edge_label(u, v, randint(Integer(1), Integer(2)))
>>> print(len(foo(K4, Integer(100))))  # random
16
```

Check that the spanning tree returned when using weights is a tree:

```
sage: # needs networkx
sage: G = graphs.RandomBarabasiAlbert(50, 2)
sage: for u, v in G.edge_iterator(labels=False):
....:     G.set_edge_label(u, v, randint(1, 10))
sage: T = G.random_spanning_tree(by_weight=True, output_as_graph=True)
sage: T.is_tree()
True
```

```
>>> from sage.all import *
>>> # needs networkx
>>> G = graphs.RandomBarabasiAlbert(Integer(50), Integer(2))
>>> for u, v in G.edge_iterator(labels=False):
...     G.set_edge_label(u, v, randint(Integer(1), Integer(10)))
>>> T = G.random_spanning_tree(by_weight=True, output_as_graph=True)
>>> T.is_tree()
True
```

rank_decomposition(*G*, *verbose=False*, *immutable=None*)

Compute an optimal rank-decomposition of the given graph.

This function is available as a method of the *Graph* class. See *rank_decomposition*.

INPUT:

- `verbose` – boolean (default: `False`); whether to display progress information while computing the decomposition
- `immutable` – boolean (default: `None`); whether to create a mutable/immutable graph. `immutable=None` (default) means that the graph and its decomposition tree will behave the same way.

OUTPUT:

A pair `(rankwidth, decomposition_tree)`, where `rankwidth` is a numerical value and `decomposition_tree` is a ternary tree describing the decomposition (cf. the module's documentation).

EXAMPLES:

```
sage: # needs rankwidth
sage: from sage.graphs.graph_decompositions.rankwidth import rank_
↪decomposition
sage: g = graphs.PetersenGraph()
sage: rank_decomposition(g)
(3, Graph on 19 vertices)
```

```
>>> from sage.all import *
>>> # needs rankwidth
>>> from sage.graphs.graph_decompositions.rankwidth import rank_decomposition
>>> g = graphs.PetersenGraph()
>>> rank_decomposition(g)
(3, Graph on 19 vertices)
```

On more than 32 vertices:

```
sage: # needs rankwidth
sage: g = graphs.RandomGNP(40, .5)
sage: rank_decomposition(g)
Traceback (most recent call last):
...
RuntimeError: the rank decomposition cannot be computed on graphs of >= 32␣
↪vertices
```

```
>>> from sage.all import *
>>> # needs rankwidth
>>> g = graphs.RandomGNP(Integer(40), RealNumber('.5'))
>>> rank_decomposition(g)
Traceback (most recent call last):
...
RuntimeError: the rank decomposition cannot be computed on graphs of >= 32␣
↪vertices
```

The empty graph:

```
sage: # needs rankwidth
sage: g = Graph()
sage: rank_decomposition(g)
(0, Graph on 0 vertices)
```

```
>>> from sage.all import *
>>> # needs rankwidth
>>> g = Graph()
>>> rank_decomposition(g)
(0, Graph on 0 vertices)
```

Check the behavior of parameter `immutable`:

```
sage: # needs rankwidth
sage: G = Graph()
sage: rank_decomposition(G)[1].is_immutable()
False
sage: rank_decomposition(G, immutable=True)[1].is_immutable()
True
sage: G = Graph(immutable=True)
sage: rank_decomposition(G)[1].is_immutable()
True
sage: rank_decomposition(G, immutable=False)[1].is_immutable()
False
```

```
>>> from sage.all import *
>>> # needs rankwidth
>>> G = Graph()
>>> rank_decomposition(G)[Integer(1)].is_immutable()
False
>>> rank_decomposition(G, immutable=True)[Integer(1)].is_immutable()
True
>>> G = Graph(immutable=True)
>>> rank_decomposition(G)[Integer(1)].is_immutable()
True
>>> rank_decomposition(G, immutable=False)[Integer(1)].is_immutable()
False
```

seidel_adjacency_matrix(*vertices*, *base_ring=None*, ***kwds*)

Return the Seidel adjacency matrix of `self`.

Returns $J - I - 2A$, for A the (ordinary) *adjacency matrix* of `self`, I the identity matrix, and J the all-1 matrix. It is closely related to *twograph()*.

By default, the matrix returned is over the integers.

INPUT:

- `vertices` – list of vertices (default: `None`); the ordering of the vertices defining how they should appear in the matrix. By default, the ordering given by *vertices()* is used.
- `base_ring` – a ring (default: `None`); the base ring of the matrix space to use
- `**kwds` – other keywords to pass to `matrix()`

EXAMPLES:

```
sage: G = graphs.CycleGraph(5)
sage: G = G.disjoint_union(graphs.CompleteGraph(1))
sage: G.seidel_adjacency_matrix().minpoly()                                 #␣
↪needs sage.libs.pari sage.modules
x^2 - 5
```

```
>>> from sage.all import *
>>> G = graphs.CycleGraph(Integer(5))
>>> G = G.disjoint_union(graphs.CompleteGraph(Integer(1)))
>>> G.seidel_adjacency_matrix().minpoly()                                   #␣
↪needs sage.libs.pari sage.modules
x^2 - 5
```

Selecting the base ring:

```
sage: G.seidel_adjacency_matrix()[0, 0].parent()                                #␣
↪needs sage.modules
Integer Ring
sage: G.seidel_adjacency_matrix(base_ring=RDF)[0, 0].parent()                   #␣
↪needs sage.modules
Real Double Field
```

```
>>> from sage.all import *
>>> G.seidel_adjacency_matrix()[Integer(0), Integer(0)].parent()               ␣
↪              # needs sage.modules
Integer Ring
>>> G.seidel_adjacency_matrix(base_ring=RDF)[Integer(0), Integer(0)].parent()␣
↪              # needs sage.modules
Real Double Field
```

seidel_switching(*s*, *inplace=True*, *immutable=None*)

Return the Seidel switching of `self` w.r.t. subset of vertices `s`.

Returns the graph obtained by Seidel switching of `self` with respect to the subset of vertices `s`. This is the graph given by Seidel adjacency matrix DSD, for S the Seidel adjacency matrix of `self`, and D the diagonal matrix with -1s at positions corresponding to `s`, and 1s elsewhere.

INPUT:

- `s` – list of vertices of `self`
- `inplace` – boolean (default: `True`); whether to do the modification inplace, or to return a copy of the graph after switching
- `immutable` – boolean (default: `None`); whether to create a mutable/immutable graph. `immutable=None` (default) means that the graph and its Seidel switching will behave the same way. This parameter is ignored when `inplace` is `True`.

EXAMPLES:

```
sage: G = graphs.CycleGraph(5)
sage: G = G.disjoint_union(graphs.CompleteGraph(1))
sage: G.seidel_switching([(0,1),(1,0),(0,0)])
sage: G.seidel_adjacency_matrix().minpoly()                                     #␣
↪needs sage.libs.pari sage.modules
x^2 - 5
sage: G.is_connected()
True
```

```
>>> from sage.all import *
>>> G = graphs.CycleGraph(Integer(5))
>>> G = G.disjoint_union(graphs.CompleteGraph(Integer(1)))
>>> G.seidel_switching([(Integer(0),Integer(1)),(Integer(1),Integer(0)),
↪(Integer(0),Integer(0))])
>>> G.seidel_adjacency_matrix().minpoly()                                       #␣
↪needs sage.libs.pari sage.modules
x^2 - 5
>>> G.is_connected()
True
```

slice_decomposition(*G*, *initial_vertex=None*)

Compute a slice decomposition of the simple undirected graph

INPUT:

- `G` – a Sage graph.
- `initial_vertex` – (default: `None`); the first vertex to consider.

OUTPUT:

An object of type *`SliceDecomposition`* that represents a slice decomposition of `G`

Note

Loops and multiple edges are ignored during the computation of the slice decomposition.

ALGORITHM:

The method use the algorithm based on "partition refinement" described in [HMPV2000] and [TCHP2008]. The time complexity of this algorithm is in $O(n+m)$, and our implementation follows that complexity for `SparseGraph`. For `DenseGraph`, the complexity is $O(n^2)$.

EXAMPLES:

Slice decomposition of the Petersen Graph:

```
sage: G = graphs.PetersenGraph()
sage: SD = G.slice_decomposition(); SD
[0[1[4[5]]] [2[6]] [3] [9] [7] [8]]
```

```
>>> from sage.all import *
>>> G = graphs.PetersenGraph()
>>> SD = G.slice_decomposition(); SD
[0[1[4[5]]] [2[6]] [3] [9] [7] [8]]
```

The graph can have loops or multiple edges but they are ignored:

```
sage: H = Graph(G,loops=True,multiedges=True)
sage: H.add_edges([(4, 4), (2, 2), (1, 6)])
sage: SD2 = H.slice_decomposition()
sage: SD2 == SD
True
sage: SD2.underlying_graph() == G.to_simple(immutable=True)
True
```

```
>>> from sage.all import *
>>> H = Graph(G,loops=True,multiedges=True)
>>> H.add_edges([(Integer(4), Integer(4)), (Integer(2), Integer(2)),␣
↪(Integer(1), Integer(6))])
>>> SD2 = H.slice_decomposition()
>>> SD2 == SD
True
>>> SD2.underlying_graph() == G.to_simple(immutable=True)
True
```

The tree corresponding to the slice decomposition can be displayed using `view`:

```
sage: from sage.graphs.graph_latex import check_tkz_graph
sage: check_tkz_graph()  # random - depends on Tex installation
sage: view(G)  # not tested
sage: latex(G)  # to obtain the corresponding LaTeX code
\begin{tikzpicture}
...
\end{tikzpicture}
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_latex import check_tkz_graph
>>> check_tkz_graph()  # random - depends on Tex installation
>>> view(G)  # not tested
>>> latex(G)  # to obtain the corresponding LaTeX code
\begin{tikzpicture}
...
\end{tikzpicture}
```

Slice decompositions are only defined for undirected graphs:

```
sage: from sage.graphs.graph_decompositions.slice_decomposition import slice_
↪decomposition
sage: slice_decomposition(DiGraph())
Traceback (most recent call last):
...
ValueError: parameter G must be an undirected graph
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_decompositions.slice_decomposition import slice_
↪decomposition
>>> slice_decomposition(DiGraph())
Traceback (most recent call last):
...
ValueError: parameter G must be an undirected graph
```

spanning_trees(*g*, *labels=False*)

Return an iterator over all spanning trees of the graph g.

A disconnected graph has no spanning tree.

Uses the Read-Tarjan backtracking algorithm [RT1975a].

INPUT:

- `labels` – boolean (default: `False`); whether to return edges labels in the spanning trees or not

EXAMPLES:

```
sage: G = Graph([(1,2),(1,2),(1,3),(1,3),(2,3),(1,4)], multiedges=True)
sage: len(list(G.spanning_trees()))
8
sage: G.number_of_spanning_trees()
↪  # needs sage.modules
8
sage: G = Graph([(1,2),(2,3),(3,1),(3,4),(4,5),(4,5),(4,6)], multiedges=True)
sage: len(list(G.spanning_trees()))
```

(continues on next page)

(continued from previous page)

```
6
sage: G.number_of_spanning_trees()                                              ␣
↪  # needs sage.modules
6
```

```
>>> from sage.all import *
>>> G = Graph([(Integer(1),Integer(2)),(Integer(1),Integer(2)),(Integer(1),
↪Integer(3)),(Integer(1),Integer(3)),(Integer(2),Integer(3)),(Integer(1),
↪Integer(4))], multiedges=True)
>>> len(list(G.spanning_trees()))
8
>>> G.number_of_spanning_trees()
↪# needs sage.modules
8
>>> G = Graph([(Integer(1),Integer(2)),(Integer(2),Integer(3)),(Integer(3),
↪Integer(1)),(Integer(3),Integer(4)),(Integer(4),Integer(5)),(Integer(4),
↪Integer(5)),(Integer(4),Integer(6))], multiedges=True)
>>> len(list(G.spanning_trees()))
6
>>> G.number_of_spanning_trees()
↪# needs sage.modules
6
```

See also

- *number_of_spanning_trees()* – counts the number of spanning trees
- *random_spanning_tree()* – returns a random spanning tree

sparse6_string()

Return the sparse6 representation of the graph as an ASCII string.

Only valid for undirected graphs on 0 to 262143 vertices, but loops and multiple edges are permitted.

Note

As the sparse6 format only handles graphs whose vertex set is $\{0, ..., n-1\}$, a *relabelled copy* of your graph will be encoded if necessary.

EXAMPLES:

```
sage: G = graphs.BullGraph()
sage: G.sparse6_string()
':Da@en'
```

```
>>> from sage.all import *
>>> G = graphs.BullGraph()
>>> G.sparse6_string()
':Da@en'
```

```
sage: G = Graph(loops=True, multiedges=True, data_structure='sparse')
sage: Graph(':?', data_structure='sparse') == G
True
```

```
>>> from sage.all import *
>>> G = Graph(loops=True, multiedges=True, data_structure='sparse')
>>> Graph(':?', data_structure='sparse') == G
True
```

spqr_tree(*G*, *algorithm='Hopcroft_Tarjan'*, *solver=None*, *verbose=0*, *integrality_tolerance=0.001*)

Return an SPQR-tree representing the triconnected components of the graph.

An SPQR-tree is a tree data structure used to represent the triconnected components of a biconnected (multi)graph and the 2-vertex cuts separating them. A node of a SPQR-tree, and the graph associated with it, can be one of the following four types:

- `'S'` – the associated graph is a cycle with at least three vertices `'S'` stands for `series`
- `'P'` – the associated graph is a dipole graph, a multigraph with two vertices and three or more edges. `'P'` stands for `parallel`
- `'Q'` – the associated graph has a single real edge. This trivial case is necessary to handle the graph that has only one edge
- `'R'` – the associated graph is a 3-connected graph that is not a cycle or dipole. `'R'` stands for `rigid`

This method decomposes a biconnected graph into cycles, cocycles, and 3-connected blocks summed over cocycles, and arranges them as a SPQR-tree. More precisely, it splits the graph at each of its 2-vertex cuts, giving a unique decomposition into 3-connected blocks, cycles and cocycles. The cocycles are dipole graphs with one edge per real edge between the included vertices and one additional (virtual) edge per connected component resulting from deletion of the vertices in the cut. See the Wikipedia article SPQR_tree.

INPUT:

- `G` – the input graph
- `algorithm` – string (default: `'Hopcroft_Tarjan'`); the algorithm to use among:
  - `'Hopcroft_Tarjan'` – default; use the algorithm proposed by Hopcroft and Tarjan in [Hopcroft1973] and later corrected by Gutwenger and Mutzel in [Gut2001]. See *TriconnectivitySPQR*.
  - `'cleave'` – using method *cleave()*
- `solver` – string (default: `None`); specifies a Mixed Integer Linear Programming (MILP) solver to be used. If set to `None`, the default one is used. For more information on MILP solvers and which default solver is used, see the method `solve` of the class `MixedIntegerLinearProgram`.
- `verbose` – integer (default: 0); sets the level of verbosity. Set to 0 by default, which means quiet.
- `integrality_tolerance` – float; parameter for use with MILP solvers over an inexact base ring; see `MixedIntegerLinearProgram.get_values()`.

OUTPUT: `SPQR-tree` a tree whose vertices are labeled with the block's type and the subgraph of three-blocks in the decomposition.

EXAMPLES:

```
sage: from sage.graphs.connectivity import spqr_tree
sage: G = Graph(2)
```

(continues on next page)

(continued from previous page)

```
sage: for i in range(3):
....:     G.add_clique([0, 1, G.add_vertex(), G.add_vertex()])
sage: Tree = spqr_tree(G)
sage: Tree.order()
4
sage: K4 = graphs.CompleteGraph(4)
sage: all(u[1].is_isomorphic(K4) for u in Tree if u[0] == 'R')
True
sage: from sage.graphs.connectivity import spqr_tree_to_graph
sage: G.is_isomorphic(spqr_tree_to_graph(Tree))
True

sage: G = Graph(2)
sage: for i in range(3):
....:     G.add_path([0, G.add_vertex(), G.add_vertex(), 1])
sage: Tree = spqr_tree(G)
sage: Tree.order()
4
sage: C4 = graphs.CycleGraph(4)
sage: all(u[1].is_isomorphic(C4) for u in Tree if u[0] == 'S')
True
sage: G.is_isomorphic(spqr_tree_to_graph(Tree))
True

sage: G.allow_multiple_edges(True)
sage: G.add_edges(G.edge_iterator())
sage: Tree = spqr_tree(G)
sage: Tree.order()
13
sage: all(u[1].is_isomorphic(C4) for u in Tree if u[0] == 'S')
True
sage: G.is_isomorphic(spqr_tree_to_graph(Tree))
True

sage: G = graphs.CycleGraph(6)
sage: Tree = spqr_tree(G)
sage: Tree.order()
1
sage: G.is_isomorphic(spqr_tree_to_graph(Tree))
True
sage: G.add_edge(0, 3)
sage: Tree = spqr_tree(G)
sage: Tree.order()
3
sage: G.is_isomorphic(spqr_tree_to_graph(Tree))
True

sage: G = Graph('LlCG{O@?GBoMw?')
sage: T = spqr_tree(G, algorithm='Hopcroft_Tarjan')
sage: G.is_isomorphic(spqr_tree_to_graph(T))
True
sage: T2 = spqr_tree(G, algorithm='cleave')
```

(continues on next page)

(continued from previous page)

```
↪  # needs sage.numerical.mip
sage: G.is_isomorphic(spqr_tree_to_graph(T2))                                  ␣
↪  # needs sage.numerical.mip
True

sage: G = Graph([(0, 1)], multiedges=True)
sage: T = spqr_tree(G, algorithm='cleave')                                     ␣
↪  # needs sage.numerical.mip
sage: T.vertices(sort=True)                                                    ␣
↪  # needs sage.numerical.mip
[('Q', Multi-graph on 2 vertices)]
sage: G.is_isomorphic(spqr_tree_to_graph(T))                                   ␣
↪  # needs sage.numerical.mip
True
sage: T = spqr_tree(G, algorithm='Hopcroft_Tarjan')
sage: T.vertices(sort=True)
[('Q', Multi-graph on 2 vertices)]
sage: G.add_edge(0, 1)
sage: spqr_tree(G, algorithm='cleave').vertices(sort=True)                     ␣
↪  # needs sage.numerical.mip
[('P', Multi-graph on 2 vertices)]

sage: from collections import Counter
sage: G = graphs.PetersenGraph()
sage: T = G.spqr_tree(algorithm='Hopcroft_Tarjan')
sage: Counter(u[0] for u in T)
Counter({'R': 1})
sage: T = G.spqr_tree(algorithm='cleave')                                      ␣
↪  # needs sage.numerical.mip
sage: Counter(u[0] for u in T)                                                 ␣
↪  # needs sage.numerical.mip
Counter({'R': 1})
sage: for u,v in list(G.edges(labels=False, sort=False)):
....:     G.add_path([u, G.add_vertex(), G.add_vertex(), v])
sage: T = G.spqr_tree(algorithm='Hopcroft_Tarjan')
sage: sorted(Counter(u[0] for u in T).items())
[('P', 15), ('R', 1), ('S', 15)]
sage: T = G.spqr_tree(algorithm='cleave')                                      ␣
↪  # needs sage.numerical.mip
sage: G.is_isomorphic(spqr_tree_to_graph(T))                                   ␣
↪  # needs sage.numerical.mip
True
sage: for u,v in list(G.edges(labels=False, sort=False)):
....:     G.add_path([u, G.add_vertex(), G.add_vertex(), v])
sage: T = G.spqr_tree(algorithm='Hopcroft_Tarjan')
sage: sorted(Counter(u[0] for u in T).items())
[('P', 60), ('R', 1), ('S', 75)]
sage: T = G.spqr_tree(algorithm='cleave')       # long time                    ␣
↪  # needs sage.numerical.mip
sage: G.is_isomorphic(spqr_tree_to_graph(T))    # long time                    ␣
↪  # needs sage.numerical.mip
True
```

```
>>> from sage.all import *
>>> from sage.graphs.connectivity import spqr_tree
>>> G = Graph(Integer(2))
>>> for i in range(Integer(3)):
...     G.add_clique([Integer(0), Integer(1), G.add_vertex(), G.add_vertex()])
>>> Tree = spqr_tree(G)
>>> Tree.order()
4
>>> K4 = graphs.CompleteGraph(Integer(4))
>>> all(u[Integer(1)].is_isomorphic(K4) for u in Tree if u[Integer(0)] == 'R')
True
>>> from sage.graphs.connectivity import spqr_tree_to_graph
>>> G.is_isomorphic(spqr_tree_to_graph(Tree))
True

>>> G = Graph(Integer(2))
>>> for i in range(Integer(3)):
...     G.add_path([Integer(0), G.add_vertex(), G.add_vertex(), Integer(1)])
>>> Tree = spqr_tree(G)
>>> Tree.order()
4
>>> C4 = graphs.CycleGraph(Integer(4))
>>> all(u[Integer(1)].is_isomorphic(C4) for u in Tree if u[Integer(0)] == 'S')
True
>>> G.is_isomorphic(spqr_tree_to_graph(Tree))
True

>>> G.allow_multiple_edges(True)
>>> G.add_edges(G.edge_iterator())
>>> Tree = spqr_tree(G)
>>> Tree.order()
13
>>> all(u[Integer(1)].is_isomorphic(C4) for u in Tree if u[Integer(0)] == 'S')
True
>>> G.is_isomorphic(spqr_tree_to_graph(Tree))
True

>>> G = graphs.CycleGraph(Integer(6))
>>> Tree = spqr_tree(G)
>>> Tree.order()
1
>>> G.is_isomorphic(spqr_tree_to_graph(Tree))
True
>>> G.add_edge(Integer(0), Integer(3))
>>> Tree = spqr_tree(G)
>>> Tree.order()
3
>>> G.is_isomorphic(spqr_tree_to_graph(Tree))
True

>>> G = Graph('LlCG{O@?GBoMw?')
>>> T = spqr_tree(G, algorithm='Hopcroft_Tarjan')
>>> G.is_isomorphic(spqr_tree_to_graph(T))
```

(continues on next page)

(continued from previous page)

```
True
>>> T2 = spqr_tree(G, algorithm='cleave')
↪# needs sage.numerical.mip
>>> G.is_isomorphic(spqr_tree_to_graph(T2))
↪# needs sage.numerical.mip
True

>>> G = Graph([(Integer(0), Integer(1))], multiedges=True)
>>> T = spqr_tree(G, algorithm='cleave')
↪# needs sage.numerical.mip
>>> T.vertices(sort=True)
↪# needs sage.numerical.mip
[('Q', Multi-graph on 2 vertices)]
>>> G.is_isomorphic(spqr_tree_to_graph(T))
↪# needs sage.numerical.mip
True
>>> T = spqr_tree(G, algorithm='Hopcroft_Tarjan')
>>> T.vertices(sort=True)
[('Q', Multi-graph on 2 vertices)]
>>> G.add_edge(Integer(0), Integer(1))
>>> spqr_tree(G, algorithm='cleave').vertices(sort=True)
↪# needs sage.numerical.mip
[('P', Multi-graph on 2 vertices)]

>>> from collections import Counter
>>> G = graphs.PetersenGraph()
>>> T = G.spqr_tree(algorithm='Hopcroft_Tarjan')
>>> Counter(u[Integer(0)] for u in T)
Counter({'R': 1})
>>> T = G.spqr_tree(algorithm='cleave')
↪# needs sage.numerical.mip
>>> Counter(u[Integer(0)] for u in T)
↪         # needs sage.numerical.mip
Counter({'R': 1})
>>> for u,v in list(G.edges(labels=False, sort=False)):
...     G.add_path([u, G.add_vertex(), G.add_vertex(), v])
>>> T = G.spqr_tree(algorithm='Hopcroft_Tarjan')
>>> sorted(Counter(u[Integer(0)] for u in T).items())
[('P', 15), ('R', 1), ('S', 15)]
>>> T = G.spqr_tree(algorithm='cleave')
↪# needs sage.numerical.mip
>>> G.is_isomorphic(spqr_tree_to_graph(T))
↪# needs sage.numerical.mip
True
>>> for u,v in list(G.edges(labels=False, sort=False)):
...     G.add_path([u, G.add_vertex(), G.add_vertex(), v])
>>> T = G.spqr_tree(algorithm='Hopcroft_Tarjan')
>>> sorted(Counter(u[Integer(0)] for u in T).items())
[('P', 60), ('R', 1), ('S', 75)]
>>> T = G.spqr_tree(algorithm='cleave')       # long time
↪# needs sage.numerical.mip
>>> G.is_isomorphic(spqr_tree_to_graph(T))    # long time
```

(continues on next page)

(continued from previous page)

```
↪# needs sage.numerical.mip
True
```

strong_orientation(G)

Return a strongly connected orientation of the graph G.

An orientation of an undirected graph is a digraph obtained by giving an unique direction to each of its edges. An orientation is said to be strong if there is a directed path between each pair of vertices. See also the Wikipedia article Strongly_connected_component.

If the graph is 2-edge-connected, a strongly connected orientation can be found in linear time. If the given graph is not 2-connected, the orientation returned will ensure that each 2-connected component has a strongly connected orientation.

INPUT:

- `G` – an undirected graph

OUTPUT: a digraph representing an orientation of the current graph

Note

- This method assumes that the input the graph is connected.
- The time complexity is $O(n+m)$ for `SparseGraph` and $O(n^2)$ for `DenseGraph` .

See also

- *orientations()*
- *strong_orientations_iterator()*
- *nauty_directg()*
- *random_orientation()*

EXAMPLES:

For a 2-regular graph, a strong orientation gives to each vertex an out-degree equal to 1:

```
sage: g = graphs.CycleGraph(5)
sage: g.strong_orientation().out_degree()
[1, 1, 1, 1, 1]
```

```
>>> from sage.all import *
>>> g = graphs.CycleGraph(Integer(5))
>>> g.strong_orientation().out_degree()
[1, 1, 1, 1, 1]
```

The Petersen Graph is 2-edge connected. It then has a strongly connected orientation:

```
sage: g = graphs.PetersenGraph()
sage: o = g.strong_orientation()
```

(continues on next page)

(continued from previous page)

```
sage: len(o.strongly_connected_components())
1
```

```
>>> from sage.all import *
>>> g = graphs.PetersenGraph()
>>> o = g.strong_orientation()
>>> len(o.strongly_connected_components())
1
```

The same goes for the CubeGraph in any dimension:

```
sage: all(len(graphs.CubeGraph(i).strong_orientation().strongly_connected_
↪components()) == 1
....:     for i in range(2,6))
True
```

```
>>> from sage.all import *
>>> all(len(graphs.CubeGraph(i).strong_orientation().strongly_connected_
↪components()) == Integer(1)
...     for i in range(Integer(2),Integer(6)))
True
```

A multigraph also has a strong orientation:

```
sage: g = Graph([(0, 1), (0, 2), (1, 2)] * 2, multiedges=True)
sage: g.strong_orientation()
Multi-digraph on 3 vertices
```

```
>>> from sage.all import *
>>> g = Graph([(Integer(0), Integer(1)), (Integer(0), Integer(2)),␣
↪(Integer(1), Integer(2))] * Integer(2), multiedges=True)
>>> g.strong_orientation()
Multi-digraph on 3 vertices
```

strong_orientations_iterator(G)

Return an iterator over all strong orientations of a graph G.

A strong orientation of a graph is an orientation of its edges such that the obtained digraph is strongly connected (i.e. there exist a directed path between each pair of vertices). According to Robbins' theorem (see the Wikipedia article Robbins_theorem), the graphs that have strong orientations are exactly the 2-edge-connected graphs (i.e., the bridgeless graphs).

ALGORITHM:

It is an adaptation of the algorithm published in [CGMRV16]. It runs in $O(mn)$ amortized time, where m is the number of edges and n is the number of vertices. The amortized time can be improved to $O(m)$ with a more involved method. In this function, first the graph is preprocessed and a spanning tree is generated. Then every orientation of the non-tree edges of the graph can be extended to at least one new strong orientation by orienting properly the edges of the spanning tree (this property is proved in [CGMRV16]). Therefore, this function generates all partial orientations of the non-tree edges and then launches a helper function corresponding to the generation algorithm described in [CGMRV16]. In order to avoid trivial symmetries, the orientation of an arbitrary edge is fixed before the start of the enumeration process.

INPUT:

- G – an undirected graph

OUTPUT: an iterator which will produce all strong orientations of this graph

Note

Works only for simple graphs (no multiple edges). To avoid symmetries an orientation of an arbitrary edge is fixed.

See also

- *orientations()*
- *strong_orientation()*
- *nauty_directg()*
- *random_orientation()*

EXAMPLES:

A cycle has one possible (non-symmetric) strong orientation:

```
sage: g = graphs.CycleGraph(4)
sage: it = g.strong_orientations_iterator()
sage: len(list(it))
1
```

```
>>> from sage.all import *
>>> g = graphs.CycleGraph(Integer(4))
>>> it = g.strong_orientations_iterator()
>>> len(list(it))
1
```

A tree cannot be strongly oriented:

```
sage: g = graphs.RandomTree(10)
sage: len(list(g.strong_orientations_iterator()))
0
```

```
>>> from sage.all import *
>>> g = graphs.RandomTree(Integer(10))
>>> len(list(g.strong_orientations_iterator()))
0
```

Neither can be a disconnected graph:

```
sage: g = graphs.CompleteGraph(6)
sage: g.add_vertex(7)
sage: len(list(g.strong_orientations_iterator()))
0
```

```
>>> from sage.all import *
>>> g = graphs.CompleteGraph(Integer(6))
>>> g.add_vertex(Integer(7))
>>> len(list(g.strong_orientations_iterator()))
0
```

to_directed(*data_structure=None*, *sparse=None*, *immutable=None*)

Return a directed version of the graph.

A single edge becomes two edges, one in each direction.

INPUT:

- `data_structure` – one of `'sparse'`, `'static_sparse'`, or `'dense'`. See the documentation of *Graph* or *DiGraph*.
- `sparse` – boolean (default: `None`); `sparse=True` is an alias for `data_structure="sparse"`, and `sparse=False` is an alias for `data_structure="dense"`.
- `immutable` – boolean (default: `None`); whether to create a mutable/immutable digraph. Only used when `data_structure=None`.
  - `immutable=None` (default) means that the graph and its directed version will behave the same way.
  - `immutable=True` is a shortcut for `data_structure='static_sparse'`
  - `immutable=False` means that the created digraph is mutable. When used with an immutable graph, the data structure used is `'sparse'` unless anything else is specified.

EXAMPLES:

```
sage: graphs.PetersenGraph().to_directed()
Petersen graph: Digraph on 10 vertices
```

```
>>> from sage.all import *
>>> graphs.PetersenGraph().to_directed()
Petersen graph: Digraph on 10 vertices
```

to_undirected(*immutable=None*)

Since the graph is already undirected, simply returns a copy of itself.

INPUT:

- `immutable` – boolean (default: `None`); whether to create a mutable/immutable graph. `immutable=None` (default) means that the graph and the returned copy behave the same way.

EXAMPLES:

```
sage: graphs.PetersenGraph().to_undirected()
Petersen graph: Graph on 10 vertices
```

```
>>> from sage.all import *
>>> graphs.PetersenGraph().to_undirected()
Petersen graph: Graph on 10 vertices
```

topological_minor(*H*, *vertices*, *paths=False*, *solver=False*, *verbose=None*, *immutable=0*, *integrality_tolerance=None*)

Return a topological H-minor from `self` if one exists.

We say that a graph G has a topological H-minor (or that it has a graph isomorphic to H as a topological minor), if G contains a subdivision of a graph isomorphic to H (i.e. obtained from H through arbitrary subdivision of its edges) as a subgraph.

For more information, see the Wikipedia article Minor_(graph_theory).

INPUT:

- `H` – the topological minor to find in the current graph
- `solver` – string (default: `None`); specifies a Mixed Integer Linear Programming (MILP) solver to be used. If set to `None`, the default one is used. For more information on MILP solvers and which default solver is used, see the method `solve` of the class `MixedIntegerLinearProgram`.
- `verbose` – integer (default: 0); sets the level of verbosity. Set to 0 by default, which means quiet.
- `immutable` – boolean (default: `None`); whether to create a mutable/immutable graph. `immutable=None` (default) means that the graph and its topological minor will behave the same way.
- `integrality_tolerance` – float; parameter for use with MILP solvers over an inexact base ring; see `MixedIntegerLinearProgram.get_values()`.

OUTPUT:

The topological H-minor found is returned as a subgraph M of `self`, such that the vertex v of M that represents a vertex $h \in H$ has `h` as a label (see *get_vertex* and *set_vertex*), and such that every edge of M has as a label the edge of H it (partially) represents.

If no topological minor is found, this method returns `False`.

ALGORITHM:

Mixed Integer Linear Programming.

COMPLEXITY:

Theoretically, when H is fixed, testing for the existence of a topological H-minor is polynomial. The known algorithms are highly exponential in H, though.

Note

This function can be expected to be *very* slow, especially where the topological minor does not exist.

(CPLEX seems to be *much* more efficient than GLPK on this kind of problem)

EXAMPLES:

Petersen's graph has a topological K_4-minor:

```
sage: g = graphs.PetersenGraph()
sage: g.topological_minor(graphs.CompleteGraph(4))                              #␣
↪needs sage.numerical.mip
Subgraph of (Petersen graph): Graph on ...
```

```
>>> from sage.all import *
>>> g = graphs.PetersenGraph()
>>> g.topological_minor(graphs.CompleteGraph(Integer(4)))                       ␣
↪     # needs sage.numerical.mip
Subgraph of (Petersen graph): Graph on ...
```

And a topological $K_{3,3}$-minor:

```
sage: g.topological_minor(graphs.CompleteBipartiteGraph(3,3))                #␣
↪needs sage.numerical.mip
Subgraph of (Petersen graph): Graph on ...
```

```
>>> from sage.all import *
>>> g.topological_minor(graphs.CompleteBipartiteGraph(Integer(3),Integer(3)))␣
↪              # needs sage.numerical.mip
Subgraph of (Petersen graph): Graph on ...
```

And of course, a tree has no topological C_3-minor:

```
sage: g = graphs.RandomGNP(15,.3)
sage: g = g.subgraph(edges=g.min_spanning_tree())
sage: g.topological_minor(graphs.CycleGraph(3))                              #␣
↪needs sage.numerical.mip
False
```

```
>>> from sage.all import *
>>> g = graphs.RandomGNP(Integer(15),RealNumber('.3'))
>>> g = g.subgraph(edges=g.min_spanning_tree())
>>> g.topological_minor(graphs.CycleGraph(Integer(3)))                       ␣
↪     # needs sage.numerical.mip
False
```

treelength(*G*, *k=None*, *certificate=False*)

Compute the treelength of G (and provide a decomposition).

The *length* of a tree decomposition, as proposed in [DG2006], is the maximum *diameter* in G of its bags, where the diameter of a bag X_i is the largest distance in G between the vertices in X_i (i.e., $\max_{u,v \in X_i} \text{dist}_G(u,v)$). The *treelength* $tl(G)$ of a graph G is the minimum length among all possible tree decompositions of G. See the documentation of the *tree_decomposition* module for more details.

INPUT:

- `G` – a sage Graph
- `k` – integer (default: `None`); indicates the length to be considered. When k is an integer, the method checks that the graph has treelength $\leq k$. If k is `None` (default), the method computes the optimal treelength.
- `certificate` – boolean (default: `False`); whether to also return the tree-decomposition itself

OUTPUT:

`G.treelength()` returns the treelength of G. When k is specified, it returns `False` when no tree-decomposition of length $\leq k$ exists or `True` otherwise. When `certificate=True`, the tree-decomposition is also returned.

ALGORITHM:

This method virtually explores the graph of all pairs `(vertex_cut, connected_component)`, where `vertex_cut` is a vertex cut of the graph of length $\leq k$, and `connected_component` is a connected component of the graph induced by `G - vertex_cut`.

We deduce that the pair `(vertex_cut, connected_component)` is feasible with treelength k if `connected_component` is empty, or if a vertex `v` from `vertex_cut` can be replaced with a vertex from `connected_component`, such that the pair `(vertex_cut + v, connected_component - v)` is feasible.

In practice, this method decomposes the graph by its clique minimal separators into atoms, computes the treelength of each of atom and returns the maximum value over all the atoms. Indeed, we have that $tl(G) = \max_{X \in A} tl(G[X])$ where A is the set of atoms of the decomposition by clique separators of G. When `certificate == True`, the tree-decompositions of the atoms are connected to each others by adding edges with respect to the clique separators.

See also

- *treewidth()* computes the treewidth of a graph.
- *path_decomposition()* computes the pathwidth of a graph.
- module *vertex_separation*.
- *atoms_and_clique_separators()*

EXAMPLES:

The PetersenGraph has treelength 2:

```
sage: G = graphs.PetersenGraph()
sage: G.treelength()
2
```

```
>>> from sage.all import *
>>> G = graphs.PetersenGraph()
>>> G.treelength()
2
```

Disconnected graphs have infinite treelength:

```
sage: G = Graph(2)
sage: G.treelength()
+Infinity
sage: G.treelength(k=+Infinity)
True
sage: G.treelength(k=2)
False
sage: G.treelength(certificate=True)
Traceback (most recent call last):
...
ValueError: the tree decomposition of a disconnected graph is not defined
```

```
>>> from sage.all import *
>>> G = Graph(Integer(2))
>>> G.treelength()
+Infinity
>>> G.treelength(k=+Infinity)
True
>>> G.treelength(k=Integer(2))
False
>>> G.treelength(certificate=True)
Traceback (most recent call last):
```

(continues on next page)

(continued from previous page)

```
...
ValueError: the tree decomposition of a disconnected graph is not defined
```

Chordal graphs have treelength 1:

```
sage: G = graphs.RandomChordalGraph(30)
sage: while not G.is_connected():
....:     G = graphs.RandomChordalGraph(30)
sage: G.treelength()
1
```

```
>>> from sage.all import *
>>> G = graphs.RandomChordalGraph(Integer(30))
>>> while not G.is_connected():
...     G = graphs.RandomChordalGraph(Integer(30))
>>> G.treelength()
1
```

Cycles have treelength $\lceil n/3 \rceil$:

```
sage: [graphs.CycleGraph(n).treelength() for n in range(3, 11)]
[1, 2, 2, 2, 3, 3, 3, 4]
```

```
>>> from sage.all import *
>>> [graphs.CycleGraph(n).treelength() for n in range(Integer(3),␣
↪Integer(11))]
[1, 2, 2, 2, 3, 3, 3, 4]
```

treewidth(*g*, *k=None*, *kmin=None*, *certificate=False*, *algorithm=None*, *nice=False*)

Compute the treewidth of g (and provide a decomposition).

INPUT:

- `g` – a Sage Graph
- `k` – integer (default: `None`); indicates the width to be considered. When `k` is an integer, the method checks that the graph has treewidth $\leq k$. If `k` is `None` (default), the method computes the optimal treewidth.
- `kmin` – integer (default: `None`); when specified, search for a tree-decomposition of width at least `kmin`. This parameter is useful when the graph can be decomposed into atoms. This parameter is ignored when `k` is not `None` or when `algorithm == 'tdlib'`.
- `certificate` – boolean (default: `False`); whether to return the tree-decomposition itself
- `algorithm` – whether to use `'sage'` or `'tdlib'` (requires the installation of the tdlib: Algorithms for computing tree decompositions of graphs package). The default behaviour is to use 'tdlib' if it is available, and Sage's own algorithm when it is not.
- `nice` – boolean (default: `False`); whether or not to return the nice tree decomposition, provided `certificate` is `True`

OUTPUT:

`g.treewidth()` returns treewidth of the graph `g`.

When `k` is specified, it returns `False` if there is no tree decomposition of width $\leq k$, and `True` otherwise.

When `certificate=True`, the tree decomposition is returned.

When `nice=True`, the nice tree decomposition is returned. See *make_nice_tree_decomposition()* for details.

ALGORITHM:

This function virtually explores the graph of all pairs `(vertex_cut, cc)`, where `vertex_cut` is a vertex cut of the graph of cardinality $\leq k+1$, and `connected_component` is a connected component of the graph induced by `G-vertex_cut`.

We deduce that the pair `(vertex_cut, cc)` is feasible with treewidth k if `cc` is empty, or if a vertex `v` from `vertex_cut` can be replaced with a vertex from `cc`, such that the pair `(vertex_cut+v,cc-v)` is feasible.

Note

The implementation would be much faster if `cc`, the argument of the recursive function, was a bitset. It would also be very nice to not copy the graph in order to compute connected components, for this is really a waste of time.

See also

- The utility functions in the *tree_decomposition* module, especially *label_nice_tree_decomposition()*.
- *path_decomposition()* computes the pathwidth of a graph.
- module *vertex_separation*.

EXAMPLES:

The PetersenGraph has treewidth 4:

```
sage: petersen = graphs.PetersenGraph()
sage: petersen.treewidth(algorithm='sage')
4
sage: petersen.treewidth(algorithm='sage', certificate=True)
Tree decomposition: Graph on 6 vertices
```

```
>>> from sage.all import *
>>> petersen = graphs.PetersenGraph()
>>> petersen.treewidth(algorithm='sage')
4
>>> petersen.treewidth(algorithm='sage', certificate=True)
Tree decomposition: Graph on 6 vertices
```

The PetersenGraph has treewidth 4 (with `tdlib`):

```
sage: petersen = graphs.PetersenGraph()
sage: petersen.treewidth(algorithm='tdlib')                    # optional -␣
↪tdlib
4
sage: petersen.treewidth(algorithm='tdlib', certificate=True)  # optional -␣
```

(continues on next page)

(continued from previous page)

```
↪tdlib
Tree decomposition: Graph on 6 vertices
```

```
>>> from sage.all import *
>>> petersen = graphs.PetersenGraph()
>>> petersen.treewidth(algorithm='tdlib')                    # optional -␣
↪tdlib
4
>>> petersen.treewidth(algorithm='tdlib', certificate=True)  # optional -␣
↪tdlib
Tree decomposition: Graph on 6 vertices
```

Nice tree decomposition of the PetersenGraph has 28 nodes:

```
sage: petersen = graphs.PetersenGraph()
sage: petersen.treewidth(algorithm='sage', certificate=True, nice=True)
Nice tree decomposition of Tree decomposition: Graph on 28 vertices
sage: petersen.treewidth(algorithm='tdlib', certificate=True, nice=True)  #␣
↪optional - tdlib
Nice tree decomposition of Tree decomposition: Graph on 28 vertices
```

```
>>> from sage.all import *
>>> petersen = graphs.PetersenGraph()
>>> petersen.treewidth(algorithm='sage', certificate=True, nice=True)
Nice tree decomposition of Tree decomposition: Graph on 28 vertices
>>> petersen.treewidth(algorithm='tdlib', certificate=True, nice=True)  #␣
↪optional - tdlib
Nice tree decomposition of Tree decomposition: Graph on 28 vertices
```

The treewidth of a 2-dimensional grid is its smallest side:

```
sage: graphs.Grid2dGraph(2,5).treewidth()
2
sage: graphs.Grid2dGraph(3,5).treewidth()
3
```

```
>>> from sage.all import *
>>> graphs.Grid2dGraph(Integer(2),Integer(5)).treewidth()
2
>>> graphs.Grid2dGraph(Integer(3),Integer(5)).treewidth()
3
```

When parameter `kmin` is specified, the method searches for a tree-decomposition of width at least `kmin`:

```
sage: g = graphs.PetersenGraph()
sage: g.treewidth()
4
sage: g.treewidth(kmin=2, algorithm='sage')
4
sage: g.treewidth(kmin=g.order(), certificate=True, algorithm='sage')
Tree decomposition: Graph on 1 vertex
```

```
>>> from sage.all import *
>>> g = graphs.PetersenGraph()
>>> g.treewidth()
4
>>> g.treewidth(kmin=Integer(2), algorithm='sage')
4
>>> g.treewidth(kmin=g.order(), certificate=True, algorithm='sage')
Tree decomposition: Graph on 1 vertex
```

tutte_polynomial(*G*, *edge_selector=None*, *cache=None*)

Return the Tutte polynomial of the graph G.

INPUT:

- `edge_selector` – method (optional); this argument allows the user to specify his own heuristic for selecting edges used in the deletion contraction recurrence
- `cache` – (optional) dictionary to cache the Tutte polynomials generated in the recursive process. One will be created automatically if not provided.

EXAMPLES:

The Tutte polynomial of any tree of order n is x^{n-1}:

```
sage: all(T.tutte_polynomial() == x**9 for T in graphs.trees(10))
↪  # needs sage.symbolic
True
```

```
>>> from sage.all import *
>>> all(T.tutte_polynomial() == x**Integer(9) for T in graphs.
↪trees(Integer(10)))                # needs sage.symbolic
True
```

The Tutte polynomial of the Petersen graph is:

```
sage: P = graphs.PetersenGraph()
sage: P.tutte_polynomial()
x^9 + 6*x^8 + 21*x^7 + 56*x^6 + 12*x^5*y + y^6 + 114*x^5 + 70*x^4*y
+ 30*x^3*y^2 + 15*x^2*y^3 + 10*x*y^4 + 9*y^5 + 170*x^4 + 170*x^3*y
+ 105*x^2*y^2 + 65*x*y^3 + 35*y^4 + 180*x^3 + 240*x^2*y + 171*x*y^2
+ 75*y^3 + 120*x^2 + 168*x*y + 84*y^2 + 36*x + 36*y
```

```
>>> from sage.all import *
>>> P = graphs.PetersenGraph()
>>> P.tutte_polynomial()
x^9 + 6*x^8 + 21*x^7 + 56*x^6 + 12*x^5*y + y^6 + 114*x^5 + 70*x^4*y
+ 30*x^3*y^2 + 15*x^2*y^3 + 10*x*y^4 + 9*y^5 + 170*x^4 + 170*x^3*y
+ 105*x^2*y^2 + 65*x*y^3 + 35*y^4 + 180*x^3 + 240*x^2*y + 171*x*y^2
+ 75*y^3 + 120*x^2 + 168*x*y + 84*y^2 + 36*x + 36*y
```

The Tutte polynomial of a connected graph G evaluated at (1,1) is the number of spanning trees of G:

```
sage: G = graphs.RandomGNP(10,0.6)
sage: while not G.is_connected():
....:     G = graphs.RandomGNP(10,0.6)
```

(continues on next page)

(continued from previous page)

```
sage: G.tutte_polynomial()(1,1) == G.number_of_spanning_trees()
↪  # needs sage.modules
True
```

```
>>> from sage.all import *
>>> G = graphs.RandomGNP(Integer(10),RealNumber('0.6'))
>>> while not G.is_connected():
...     G = graphs.RandomGNP(Integer(10),RealNumber('0.6'))
>>> G.tutte_polynomial()(Integer(1),Integer(1)) == G.number_of_spanning_
↪trees()                  # needs sage.modules
True
```

Given that $T(x, y)$ is the Tutte polynomial of a graph G with n vertices and c connected components, then $(-1)^{n-c}x^kT(1-x, 0)$ is the chromatic polynomial of G.

```
sage: G = graphs.OctahedralGraph()
sage: T = G.tutte_polynomial()
sage: R = PolynomialRing(ZZ, 'x')
sage: R((-1)^5*x*T(1-x,0)).factor()
↪  # needs sage.symbolic
(x - 2) * (x - 1) * x * (x^3 - 9*x^2 + 29*x - 32)
sage: G.chromatic_polynomial().factor()
↪  # needs sage.libs.flint
(x - 2) * (x - 1) * x * (x^3 - 9*x^2 + 29*x - 32)
```

```
>>> from sage.all import *
>>> G = graphs.OctahedralGraph()
>>> T = G.tutte_polynomial()
>>> R = PolynomialRing(ZZ, 'x')
>>> R((-Integer(1))**Integer(5)*x*T(Integer(1)-x,Integer(0))).factor()
↪                                    # needs sage.symbolic
(x - 2) * (x - 1) * x * (x^3 - 9*x^2 + 29*x - 32)
>>> G.chromatic_polynomial().factor()
↪# needs sage.libs.flint
(x - 2) * (x - 1) * x * (x^3 - 9*x^2 + 29*x - 32)
```

tutte_symmetric_function(*R=None*, *t=None*)

Return the Tutte symmetric function of `self`.

Let G be a graph. The Tutte symmetric function XB_G of the graph G was introduced in [Sta1998]. We present the equivalent definition given in [CS2022].

$$XB_G = \sum_{\pi \vdash V} (1+t)^{e(\pi)} \tilde{m}_{\lambda(\pi)},$$

where the sum ranges over all set-partitions π of the vertex set V, $\lambda(\pi)$ is the partition determined by the sizes of the blocks of π, and $e(\pi)$ is the number of edges whose endpoints lie in the same block of π. In particular, the coefficients of XB_G when expanded in terms of augmented monomial symmetric functions are polynomials in t with non-negative integer coefficients.

For an integer partition $\lambda = 1^{r_1}2^{r_2}\cdots$ expressed in the exponential notation, the augmented monomial symmetric function is defined as

$$\tilde{m}_\lambda = \left(\prod_i r_i!\right) m_\lambda.$$

INPUT:

- `R` – (default: the parent of `t`) the base ring for the symmetric functions
- `t` – (default: t in $\mathbf{Z}[t]$) the parameter t

EXAMPLES:

```
sage: p = SymmetricFunctions(ZZ).p()                                    #␣
↪needs sage.combinat sage.modules
sage: G = Graph([[1,2],[2,3],[3,4],[4,1],[1,3]])
sage: XB_G = G.tutte_symmetric_function(); XB_G                          #␣
↪needs sage.combinat sage.modules
24*m[1, 1, 1, 1] + (10*t+12)*m[2, 1, 1] + (4*t^2+10*t+6)*m[2, 2]
 + (2*t^3+8*t^2+10*t+4)*m[3, 1]
 + (t^5+5*t^4+10*t^3+10*t^2+5*t+1)*m[4]
sage: p(XB_G)                                                           #␣
↪needs sage.combinat sage.modules
p[1, 1, 1, 1] + 5*t*p[2, 1, 1] + 2*t^2*p[2, 2]
 + (2*t^3+8*t^2)*p[3, 1] + (t^5+5*t^4+8*t^3)*p[4]
```

```
>>> from sage.all import *
>>> p = SymmetricFunctions(ZZ).p()                                      #␣
↪needs sage.combinat sage.modules
>>> G = Graph([[Integer(1),Integer(2)],[Integer(2),Integer(3)],[Integer(3),
↪Integer(4)],[Integer(4),Integer(1)],[Integer(1),Integer(3)]])
>>> XB_G = G.tutte_symmetric_function(); XB_G                           #␣
↪needs sage.combinat sage.modules
24*m[1, 1, 1, 1] + (10*t+12)*m[2, 1, 1] + (4*t^2+10*t+6)*m[2, 2]
 + (2*t^3+8*t^2+10*t+4)*m[3, 1]
 + (t^5+5*t^4+10*t^3+10*t^2+5*t+1)*m[4]
>>> p(XB_G)                                                             #␣
↪needs sage.combinat sage.modules
p[1, 1, 1, 1] + 5*t*p[2, 1, 1] + 2*t^2*p[2, 2]
 + (2*t^3+8*t^2)*p[3, 1] + (t^5+5*t^4+8*t^3)*p[4]
```

Graphs are allowed to have multiedges and loops:

```
sage: G = Graph([[1,2],[2,3],[2,3]], multiedges = True)
sage: XB_G = G.tutte_symmetric_function(); XB_G                          #␣
↪needs sage.combinat sage.modules
6*m[1, 1, 1] + (t^2+3*t+3)*m[2, 1] + (t^3+3*t^2+3*t+1)*m[3]
```

```
>>> from sage.all import *
>>> G = Graph([[Integer(1),Integer(2)],[Integer(2),Integer(3)],[Integer(2),
↪Integer(3)]], multiedges = True)
>>> XB_G = G.tutte_symmetric_function(); XB_G                           #␣
↪needs sage.combinat sage.modules
6*m[1, 1, 1] + (t^2+3*t+3)*m[2, 1] + (t^3+3*t^2+3*t+1)*m[3]
```

We check that at $t = -1$, we recover the usual chromatic symmetric function:

```
sage: G = Graph([[1,2],[1,2],[2,3],[3,4],[4,5]], multiedges=True)
sage: XB_G = G.tutte_symmetric_function(t=-1); XB_G                      #␣
↪needs sage.combinat sage.modules
```

(continues on next page)

(continued from previous page)

```
120*m[1, 1, 1, 1, 1] + 36*m[2, 1, 1, 1] + 12*m[2, 2, 1]
 + 2*m[3, 1, 1] + m[3, 2]
sage: X_G = G.chromatic_symmetric_function(); X_G                          #␣
↪needs sage.combinat sage.modules
p[1, 1, 1, 1, 1] - 4*p[2, 1, 1, 1] + 3*p[2, 2, 1] + 3*p[3, 1, 1]
 - 2*p[3, 2] - 2*p[4, 1] + p[5]
sage: XB_G == X_G                                                          #␣
↪needs sage.combinat sage.modules
True
```

```
>>> from sage.all import *
>>> G = Graph([[Integer(1),Integer(2)],[Integer(1),Integer(2)],[Integer(2),
↪Integer(3)],[Integer(3),Integer(4)],[Integer(4),Integer(5)]],␣
↪multiedges=True)
>>> XB_G = G.tutte_symmetric_function(t=-Integer(1)); XB_G                  ␣
↪     # needs sage.combinat sage.modules
120*m[1, 1, 1, 1, 1] + 36*m[2, 1, 1, 1] + 12*m[2, 2, 1]
 + 2*m[3, 1, 1] + m[3, 2]
>>> X_G = G.chromatic_symmetric_function(); X_G                            #␣
↪needs sage.combinat sage.modules
p[1, 1, 1, 1, 1] - 4*p[2, 1, 1, 1] + 3*p[2, 2, 1] + 3*p[3, 1, 1]
 - 2*p[3, 2] - 2*p[4, 1] + p[5]
>>> XB_G == X_G                                                            #␣
↪needs sage.combinat sage.modules
True
```

two_factor_petersen(*solver*, *verbose=None*, *integrality_tolerance=0*)

Return a decomposition of the graph into 2-factors.

Petersen's 2-factor decomposition theorem asserts that any $2r$-regular graph G can be decomposed into 2-factors. Equivalently, it means that the edges of any $2r$-regular graphs can be partitioned in r sets $C_1, \ldots, C_r$ such that for all i, the set C_i is a disjoint union of cycles (a 2-regular graph).

As any graph of maximal degree Δ can be completed into a regular graph of degree $2\lceil\frac{\Delta}{2}\rceil$, this result also means that the edges of any graph of degree Δ can be partitioned in $r = 2\lceil\frac{\Delta}{2}\rceil$ sets $C_1, \ldots, C_r$ such that for all i, the set C_i is a graph of maximal degree 2 (a disjoint union of paths and cycles).

INPUT:

- `solver` – string (default: `None`); specifies a Mixed Integer Linear Programming (MILP) solver to be used. If set to `None`, the default one is used. For more information on MILP solvers and which default solver is used, see the method `solve` of the class `MixedIntegerLinearProgram`.
- `verbose` – integer (default: 0); sets the level of verbosity. Set to 0 by default, which means quiet.
- `integrality_tolerance` – float; parameter for use with MILP solvers over an inexact base ring; see `MixedIntegerLinearProgram.get_values()`.

EXAMPLES:

The Complete Graph on 7 vertices is a 6-regular graph, so it can be edge-partitionned into 2-regular graphs:

```
sage: g = graphs.CompleteGraph(7)
sage: classes = g.two_factor_petersen()                                    #␣
↪needs sage.numerical.mip
sage: for c in classes:                                                    #␣
```

(continues on next page)

(continued from previous page)

```
↪needs sage.numerical.mip
....:     gg = Graph()
....:     gg.add_edges(c)
....:     print(max(gg.degree())<=2)
True
True
True
sage: Set(set(classes[0])                                                    #␣
↪needs sage.numerical.mip
....:     | set(classes[1])
....:     | set(classes[2])).cardinality() == g.size()
True
```

```
>>> from sage.all import *
>>> g = graphs.CompleteGraph(Integer(7))
>>> classes = g.two_factor_petersen()                                        #␣
↪needs sage.numerical.mip
>>> for c in classes:                                                        #␣
↪needs sage.numerical.mip
...     gg = Graph()
...     gg.add_edges(c)
...     print(max(gg.degree())<=Integer(2))
True
True
True
>>> Set(set(classes[Integer(0)])                                             ␣
↪     # needs sage.numerical.mip
...     | set(classes[Integer(1)])
...     | set(classes[Integer(2)])).cardinality() == g.size()
True
```

```
sage: g = graphs.CirculantGraph(24, [7, 11])
sage: cl = g.two_factor_petersen()                                           #␣
↪needs sage.numerical.mip
sage: g.plot(edge_colors={'black':cl[0], 'red':cl[1]})                       #␣
↪needs sage.numerical.mip sage.plot
Graphics object consisting of 73 graphics primitives
```

```
>>> from sage.all import *
>>> g = graphs.CirculantGraph(Integer(24), [Integer(7), Integer(11)])
>>> cl = g.two_factor_petersen()                                             #␣
↪needs sage.numerical.mip
>>> g.plot(edge_colors={'black':cl[Integer(0)], 'red':cl[Integer(1)]})       ␣
↪              # needs sage.numerical.mip sage.plot
Graphics object consisting of 73 graphics primitives
```

twograph()

Return the two-graph of `self`.

Returns the `two-graph` with the triples $T = \{t \in \binom{V}{3} : \left|\binom{t}{2} \cap E\right| \text{odd}\}$ where V and E are vertices and edges of `self`, respectively.

EXAMPLES:

```
sage: p = graphs.PetersenGraph()
sage: p.twograph()                                                          #␣
↪needs sage.modules
Incidence structure with 10 points and 60 blocks
sage: p = graphs.chang_graphs()
sage: T8 = graphs.CompleteGraph(8).line_graph()
sage: C = T8.seidel_switching([(0,1,None), (2,3,None), (4,5,None), (6,7,
↪None)],
....:                         inplace=False)
sage: T8.twograph() == C.twograph()                                         #␣
↪needs sage.modules
True
sage: T8.is_isomorphic(C)
False
```

```
>>> from sage.all import *
>>> p = graphs.PetersenGraph()
>>> p.twograph()                                                            #␣
↪needs sage.modules
Incidence structure with 10 points and 60 blocks
>>> p = graphs.chang_graphs()
>>> T8 = graphs.CompleteGraph(Integer(8)).line_graph()
>>> C = T8.seidel_switching([(Integer(0),Integer(1),None), (Integer(2),
↪Integer(3),None), (Integer(4),Integer(5),None), (Integer(6),Integer(7),
↪None)],
...                         inplace=False)
>>> T8.twograph() == C.twograph()                                           #␣
↪needs sage.modules
True
>>> T8.is_isomorphic(C)
False
```

See also

- `descendant()` – computes the descendant graph of the two-graph of `self` at a vertex
- `twograph_descendant()` – ditto, but much faster.

vertex_cover(*algorithm*, *value_only='Cliquer'*, *reduction_rules=False*, *solver=True*, *verbose=None*, *integrality_tolerance=0*)

Return a minimum vertex cover of `self` represented by a set of vertices.

A minimum vertex cover of a graph is a set S of vertices such that each edge is incident to at least one element of S, and such that S is of minimum cardinality. For more information, see the Wikipedia article Vertex_cover.

Equivalently, a vertex cover is defined as the complement of an independent set.

As an optimization problem, it can be expressed as follows:

$$\begin{aligned} \text{Minimize : } & \sum_{v \in G} b_v \\ \text{Such that : } & \forall (u,v) \in G.edges(), b_u + b_v \geq 1 \\ & \forall x \in G, b_x \text{ is a binary variable} \end{aligned}$$

INPUT:

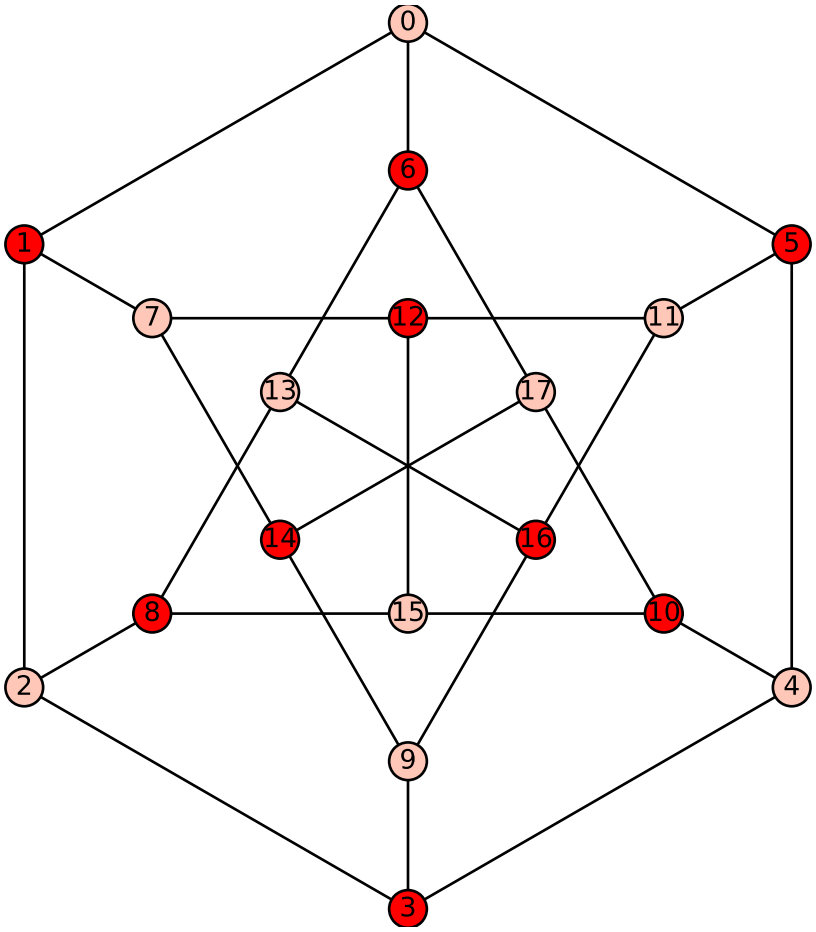
- `algorithm` – string (default: `'Cliquer'`); indicating which algorithm to use. It can be one of those values.
  - `'Cliquer'` will compute a minimum vertex cover using the Cliquer package
  - `'MILP'` will compute a minimum vertex cover through a mixed integer linear program
  - `'mcqd'` will use the MCQD solver (http://www.sicmm.org/~konc/maxclique/). Note that the MCQD package must be installed.
- `value_only` – boolean (default: `False`); if set to `True`, only the size of a minimum vertex cover is returned. Otherwise, a minimum vertex cover is returned as a list of vertices.
- `reduction_rules` – (default: `True`) specify if the reductions rules from kernelization must be applied as pre-processing or not. See [ACFLSS04] for more details. Note that depending on the instance, it might be faster to disable reduction rules.
- `solver` – string (default: `None`); specifies a Mixed Integer Linear Programming (MILP) solver to be used. If set to `None`, the default one is used. For more information on MILP solvers and which default solver is used, see the method `solve` of the class `MixedIntegerLinearProgram`.
- `verbose` – integer (default: 0); sets the level of verbosity. Set to 0 by default, which means quiet.
- `integrality_tolerance` – float; parameter for use with MILP solvers over an inexact base ring; see `MixedIntegerLinearProgram.get_values()`.

EXAMPLES:

On the Pappus graph:

```
sage: g = graphs.PappusGraph()
sage: g.vertex_cover(value_only=True)
9
```

```
>>> from sage.all import *
>>> g = graphs.PappusGraph()
>>> g.vertex_cover(value_only=True)
9
```



vertex_isoperimetric_number(*g*)

Return the vertex-isoperimetric number of the graph.

The vertex-isoperimetric number of a graph $G = (V, E)$ is also sometimes called the *magnifying constant*. It is defined as the minimum of $|N(S)|/|S|$ where $|N(S)|$ is the vertex boundary of S and the minimum is taken over the subsets S of vertices of size at most half of the vertices.

See also

Alternative but similar quantities can be obtained via the methods *cheeger_constant()* and *edge_isoperimetric_number()*.

EXAMPLES:

The vertex-isoperimetric number of a complete graph on n vertices is $\lceil n/2 \rceil / \lfloor n/2 \rfloor$:

```
sage: [graphs.CompleteGraph(k).vertex_isoperimetric_number() for k in range(2,
↪15)]
[1, 2, 1, 3/2, 1, 4/3, 1, 5/4, 1, 6/5, 1, 7/6, 1]
```

```
>>> from sage.all import *
>>> [graphs.CompleteGraph(k).vertex_isoperimetric_number() for k in␣
↪range(Integer(2),Integer(15))]
[1, 2, 1, 3/2, 1, 4/3, 1, 5/4, 1, 6/5, 1, 7/6, 1]
```

The vertex-isoperimetric number of a cycle on n vertices is $2/\lfloor n/2 \rfloor$:

```
sage: [graphs.CycleGraph(k).vertex_isoperimetric_number() for k in range(2,
↪15)]
[1, 2, 1, 1, 2/3, 2/3, 1/2, 1/2, 2/5, 2/5, 1/3, 1/3, 2/7]
```

```
>>> from sage.all import *
>>> [graphs.CycleGraph(k).vertex_isoperimetric_number() for k in␣
↪range(Integer(2),Integer(15))]
[1, 2, 1, 1, 2/3, 2/3, 1/2, 1/2, 2/5, 2/5, 1/3, 1/3, 2/7]
```

And the vertex-isoperimetric number of a disconnected graph is 0:

```
sage: Graph([[1,2,3],[(1,2)]]).vertex_isoperimetric_number()
0
```

```
>>> from sage.all import *
>>> Graph([[Integer(1),Integer(2),Integer(3)],[(Integer(1),Integer(2))]]).
↪vertex_isoperimetric_number()
0
```

The vertex-isoperimetric number is independent of edge multiplicity:

```
sage: G = graphs.CycleGraph(6)
sage: G.vertex_isoperimetric_number()
2/3
sage: G.allow_multiple_edges(True)
sage: G.add_edges(G.edges(sort=False))
sage: G.vertex_isoperimetric_number()
2/3
```

```
>>> from sage.all import *
>>> G = graphs.CycleGraph(Integer(6))
>>> G.vertex_isoperimetric_number()
2/3
>>> G.allow_multiple_edges(True)
>>> G.add_edges(G.edges(sort=False))
>>> G.vertex_isoperimetric_number()
2/3
```

write_to_eps(*filename*, ***options*)

Write a plot of the graph to `filename` in `eps` format.

INPUT:

- `filename` – string
- `**options` – same layout options as *layout()*

EXAMPLES:

```
sage: P = graphs.PetersenGraph()
sage: P.write_to_eps(tmp_filename(ext='.eps'))
```

```
>>> from sage.all import *
>>> P = graphs.PetersenGraph()
>>> P.write_to_eps(tmp_filename(ext='.eps'))
```

It is relatively simple to include this file in a LaTeX document. `\usepackage{graphics}` must appear in the preamble, and `\includegraphics{filename}` will include the file. To compile the document to `pdf` with `pdflatex` or `xelatex` the file needs first to be converted to `pdf`, for example with `ps2pdf filename.eps filename.pdf`.

1.3 Directed graphs

This module implements functions and operations involving directed graphs. Here is what they can do

Graph basic operations:

layout_acyclic_dummy()	Compute a (dummy) ranked layout so that all edges point upward.
layout_acyclic()	Compute a ranked layout so that all edges point upward.
reverse()	Return a copy of digraph with edges reversed in direction.
reverse_edge()	Reverse an edge.
reverse_edges()	Reverse a list of edges.
out_degree_sequence()	Return the outdegree sequence.
out_degree_iterator()	Same as degree_iterator, but for out degree.
out_degree()	Same as degree, but for out degree.
in_degree_sequence()	Return the indegree sequence of this digraph.
in_degree_iterator()	Same as degree_iterator, but for in degree.
in_degree()	Same as degree, but for in-degree.
neighbors_out()	Return the list of the out-neighbors of a given vertex.
neighbor_out_iterator()	Return an iterator over the out-neighbors of a given vertex.
neighbors_in()	Return the list of the in-neighbors of a given vertex.
neighbor_in_iterator()	Return an iterator over the in-neighbors of vertex.
outgoing_edges()	Return a list of edges departing from vertices.
outgoing_edge_iterator()	Return an iterator over all departing edges from vertices
incoming_edges()	Return a list of edges arriving at vertices.
incoming_edge_iterator()	Return an iterator over all arriving edges from vertices
sources()	Return the list of all sources (vertices without incoming edges) of this digraph.
sinks()	Return the list of all sinks (vertices without outgoing edges) of this digraph.
to_undirected()	Return an undirected version of the graph.
to_directed()	Since the graph is already directed, simply returns a copy of itself.
is_directed()	Since digraph is directed, returns True.
dig6_string()	Return the `dig6` representation of the digraph as an ASCII string.

Distances:

eccentricity()	Return the eccentricity of vertex (or vertices) v.
radius()	Return the radius of the DiGraph.
diameter()	Return the diameter of the DiGraph.
center()	Return the set of vertices in the center of the DiGraph.
periphery()	Return the set of vertices in the periphery of the DiGraph.

Paths and cycles:

all_cycles_iterator()	Return an iterator over all the cycles of self starting with one of the given vertices.
all_simple_cycles()	Return a list of all simple cycles of self.

Representation theory:

path_semigroup()	Return the (partial) semigroup formed by the paths of the digraph.
auslander_reiten_quiver()	Return the Auslander-Reiten quiver of self.

Connectivity:

is_strongly_connected()	Check whether the current DiGraph is strongly connected.
strongly_connected_components_digraph()	Return the digraph of the strongly connected components
strongly_connected_components_subgraphs()	Return the strongly connected components as a list of subgraphs.
strongly_connected_component_containing_vertex()	Return the strongly connected component containing a given vertex
strongly_connected_components()	Return the list of strongly connected components.
strong_articulation_points()	Return the strong articulation points of this digraph.

Acyclicity:

is_directed_acyclic()	Check whether the digraph is acyclic or not.
is_transitive()	Check whether the digraph is transitive or not.
is_aperiodic()	Check whether the digraph is aperiodic or not.
is_tournament()	Check whether the digraph is a tournament.
period()	Return the period of the digraph.
level_sets()	Return the level set decomposition of the digraph.
topological_sort_generator()	Return a list of all topological sorts of the digraph if it is acyclic.
topological_sort()	Return a topological sort of the digraph if it is acyclic.
longest_dag_path()	Return a longest path in this directed acyclic graph.

Hard stuff:

feedback_edge_set()	Compute the minimum feedback edge (arc) set of a digraph

Miscellaneous:

flow_polytope()	Compute the flow polytope of a digraph
degree_polynomial()	Return the generating polynomial of degrees of vertices in `self`.
out_branchings()	Return an iterator over the out branchings rooted at given vertex in `self`.
in_branchings()	Return an iterator over the in branchings rooted at given vertex in `self`.

1.3.1 Methods

class sage.graphs.digraph.**DiGraph**(*data=None*, *pos=None*, *loops=None*, *format=None*, *weighted=None*, *data_structure='sparse'*, *vertex_labels=True*, *name=None*, *multiedges=None*, *convert_empty_dict_labels_to_None=None*, *sparse=True*, *immutable=False*, *hash_labels=None*)

Bases: *GenericGraph*

Directed graph.

A digraph or directed graph is a set of vertices connected by oriented edges. See also the Wikipedia article Directed_graph. For a collection of pre-defined digraphs, see the *digraph_generators* module.

A *DiGraph* object has many methods whose list can be obtained by typing `g.<tab>` (i.e. hit the `Tab` key) or by reading the documentation of *digraph*, *generic_graph*, and *graph*.

INPUT:

By default, a *DiGraph* object is simple (i.e. no *loops* nor *multiple edges*) and unweighted. This can be easily tuned with the appropriate flags (see below).

- `data` – can be any of the following (see the `format` argument):
  1. `DiGraph()` – build a digraph on 0 vertices
  2. `DiGraph(5)` – return an edgeless digraph on the 5 vertices 0,…,4
  3. `DiGraph([list_of_vertices, list_of_edges])` – return a digraph with given vertices/edges

     To bypass auto-detection, prefer the more explicit `DiGraph([V, E], format='vertices_and_edges')`.
  4. `DiGraph(list_of_edges)` – return a digraph with a given list of edges (see documentation of *add_edges()*).

     To bypass auto-detection, prefer the more explicit `DiGraph(L, format='list_of_edges')`.
  5. `DiGraph({1: [2,3,4], 3: [4]})` – return a digraph by associating to each vertex the list of its out-neighbors.

     To bypass auto-detection, prefer the more explicit `DiGraph(D, format='dict_of_lists')`.
  6. `DiGraph({1: {2: 'a', 3: 'b'}, 3: {2: 'c'}})` – return a digraph by associating a list of out-neighbors to each vertex and providing its edge label.

     To bypass auto-detection, prefer the more explicit `DiGraph(D, format='dict_of_dicts')`.

     For digraphs with multiple edges, you can provide a list of labels instead, e.g.: `DiGraph({1: {2: ['a1', 'a2'], 3:['b']}, 3:{2:['c']}})`.
  7. `DiGraph(a_matrix)` – return a digraph with given (weighted) adjacency matrix (see documentation of *adjacency_matrix()*).

     To bypass auto-detection, prefer the more explicit `DiGraph(M, format='adjacency_matrix')`. To take weights into account, use `format='weighted_adjacency_matrix'` instead.

8. `DiGraph(a_nonsquare_matrix)` – return a digraph with given incidence matrix (see documentation of *incidence_matrix()*).

   To bypass auto-detection, prefer the more explicit `DiGraph(M, format='incidence_matrix')`.

9. `DiGraph([V, f])` – return a digraph with a vertex set `V` and an edge u, v whenever $f(u, v)$ is `True`. Example: `DiGraph([ [1..10], lambda x,y: abs(x - y).is_square()])`

10. `DiGraph('FOC@?OC@_?')` – return a digraph from a `dig6` string (see documentation of *dig6_string()*).

11. `DiGraph(another_digraph)` – return a digraph from a Sage (di)graph, pygraphviz digraph, NetworkX digraph, or igraph digraph.

- `pos` – dictionary (default: `None`); a positioning dictionary. For example, the spring layout from NetworkX for the 5-cycle is:

```
{0: [-0.91679746, 0.88169588],
 1: [ 0.47294849, 1.125     ],
 2: [ 1.125     ,-0.12867615],
 3: [ 0.12743933,-1.125     ],
 4: [-1.125     ,-0.50118505]}
```

- `name` – string (default: `None`); gives the graph a name (e.g., name='complete')
- `loops` – boolean (default: `None`); whether to allow loops (ignored if data is an instance of the DiGraph class)
- `multiedges` – boolean (default: `None`); whether to allow multiple edges (ignored if data is an instance of the DiGraph class)
- `weighted` – boolean (default: `None`); whether digraph thinks of itself as weighted or not. See `self.weighted()`
- `format` – string (default: `None`); if set to `None`, *DiGraph* tries to guess input's format. To avoid this possibly time-consuming step, one of the following values can be specified (see description above): `'int'`, `'dig6'`, `'rule'`, `'list_of_edges'`, `'dict_of_lists'`, `'dict_of_dicts'`, `'adjacency_matrix'`, `'weighted_adjacency_matrix'`, `'incidence_matrix'`, `"NX"`, `'igraph'`.
- `sparse` – boolean (default: `True`); `sparse=True` is an alias for `data_structure="sparse"`, and `sparse=False` is an alias for `data_structure="dense"`
- `data_structure` – string (default: `'sparse'`); one of the following (for more information, see *overview*):
  - `'dense'` – selects the *dense_graph* backend
  - `'sparse'` – selects the *sparse_graph* backend
  - `'static_sparse'` – selects the *static_sparse_backend* (this backend is faster than the sparse backend and smaller in memory, and it is immutable, so that the resulting graphs can be used as dictionary keys).
- `immutable` – boolean (default: `False`); whether to create a immutable digraph. Note that `immutable=True` is actually a shortcut for `data_structure='static_sparse'`.
- `hash_labels` – boolean (default: `None`); whether to include edge labels during hashing. This parameter defaults to `True` if the digraph is weighted. This parameter is ignored if the digraph is mutable. Beware that trying to hash unhashable labels will raise an error.
- `vertex_labels` – boolean (default: `True`); whether to allow any object as a vertex (slower), or only the integers $0, ..., n-1$, where n is the number of vertices.

- `convert_empty_dict_labels_to_None` – boolean (default: `None`); this arguments sets the default edge labels used by NetworkX (empty dictionaries) to be replaced by `None`, the default Sage edge label. It is set to `True` iff a NetworkX graph is on the input.

EXAMPLES:

1. A dictionary of dictionaries:

```
sage: g = DiGraph({0: {1: 'x', 2: 'z', 3: 'a'}, 2: {5: 'out'}}); g
Digraph on 5 vertices
```

```
>>> from sage.all import *
>>> g = DiGraph({Integer(0): {Integer(1): 'x', Integer(2): 'z', Integer(3): 'a
↪'}, Integer(2): {Integer(5): 'out'}}); g
Digraph on 5 vertices
```

   The labels ('x', 'z', 'a', 'out') are labels for edges. For example, 'out' is the label for the edge from 2 to 5. Labels can be used as weights, if all the labels share some common parent.

2. A dictionary of lists (or iterables):

```
sage: g = DiGraph({0: [1, 2, 3], 2: [4]}); g
Digraph on 5 vertices
sage: g = DiGraph({0: (1, 2, 3), 2: (4,)}); g
Digraph on 5 vertices
```

```
>>> from sage.all import *
>>> g = DiGraph({Integer(0): [Integer(1), Integer(2), Integer(3)],␣
↪Integer(2): [Integer(4)]}); g
Digraph on 5 vertices
>>> g = DiGraph({Integer(0): (Integer(1), Integer(2), Integer(3)),␣
↪Integer(2): (Integer(4),)}); g
Digraph on 5 vertices
```

3. A list of vertices and a function describing adjacencies. Note that the list of vertices and the function must be enclosed in a list (i.e., `[list of vertices, function]`).

   We construct a graph on the integers 1 through 12 such that there is a directed edge from i to j if and only if i divides j:

```
sage: g = DiGraph([[1..12], lambda i,j: i != j and i.divides(j)])
sage: g.vertices(sort=True)
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]
sage: g.adjacency_matrix()                                                  #␣
↪needs sage.modules
[0 1 1 1 1 1 1 1 1 1 1 1]
[0 0 0 1 0 1 0 1 0 1 0 1]
[0 0 0 0 0 1 0 0 1 0 0 1]
[0 0 0 0 0 0 0 1 0 0 0 1]
[0 0 0 0 0 0 0 0 0 1 0 0]
[0 0 0 0 0 0 0 0 0 0 0 1]
[0 0 0 0 0 0 0 0 0 0 0 0]
[0 0 0 0 0 0 0 0 0 0 0 0]
[0 0 0 0 0 0 0 0 0 0 0 0]
[0 0 0 0 0 0 0 0 0 0 0 0]
```

(continues on next page)

(continued from previous page)

```
[0 0 0 0 0 0 0 0 0 0 0 0]
[0 0 0 0 0 0 0 0 0 0 0 0]
```

```
>>> from sage.all import *
>>> g = DiGraph([(ellipsis_range(Integer(1),Ellipsis,Integer(12))), lambda i,
↪j: i != j and i.divides(j)])
>>> g.vertices(sort=True)
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]
>>> g.adjacency_matrix()                                                    #␣
↪needs sage.modules
[0 1 1 1 1 1 1 1 1 1 1 1]
[0 0 0 1 0 1 0 1 0 1 0 1]
[0 0 0 0 0 1 0 0 1 0 0 1]
[0 0 0 0 0 0 0 1 0 0 0 1]
[0 0 0 0 0 0 0 0 0 1 0 0]
[0 0 0 0 0 0 0 0 0 0 0 1]
[0 0 0 0 0 0 0 0 0 0 0 0]
[0 0 0 0 0 0 0 0 0 0 0 0]
[0 0 0 0 0 0 0 0 0 0 0 0]
[0 0 0 0 0 0 0 0 0 0 0 0]
[0 0 0 0 0 0 0 0 0 0 0 0]
[0 0 0 0 0 0 0 0 0 0 0 0]
```

4. A Sage matrix: Note: If format is not specified, then Sage assumes a square matrix is an adjacency matrix, and a nonsquare matrix is an incidence matrix.

   - an adjacency matrix:

```
sage: M = Matrix([[0, 1, 1, 1, 0], [0, 0, 0, 0, 0],                        ␣
↪  # needs sage.modules
....:             [0, 0, 0, 0, 1], [0, 0, 0, 0, 0], [0, 0, 0, 0, 0]]); M
[0 1 1 1 0]
[0 0 0 0 0]
[0 0 0 0 1]
[0 0 0 0 0]
[0 0 0 0 0]
sage: DiGraph(M)                                                           ␣
↪  # needs sage.modules
Digraph on 5 vertices

sage: M = Matrix([[0,1,-1], [-1,0,-1/2], [1,1/2,0]]); M                    ␣
↪  # needs sage.modules
[   0    1   -1]
[  -1    0 -1/2]
[   1  1/2    0]
sage: G = DiGraph(M, sparse=True, weighted=True); G                        ␣
↪  # needs sage.modules
Digraph on 3 vertices
sage: G.weighted()                                                         ␣
↪  # needs sage.modules
True
```

```
>>> from sage.all import *
>>> M = Matrix([[Integer(0), Integer(1), Integer(1), Integer(1),␣
↪Integer(0)], [Integer(0), Integer(0), Integer(0), Integer(0),␣
↪Integer(0)],                          # needs sage.modules
...             [Integer(0), Integer(0), Integer(0), Integer(0),␣
↪Integer(1)], [Integer(0), Integer(0), Integer(0), Integer(0),␣
↪Integer(0)], [Integer(0), Integer(0), Integer(0), Integer(0),␣
↪Integer(0)]]); M
[0 1 1 1 0]
[0 0 0 0 0]
[0 0 0 0 1]
[0 0 0 0 0]
[0 0 0 0 0]
>>> DiGraph(M)
↪# needs sage.modules
Digraph on 5 vertices

>>> M = Matrix([[Integer(0),Integer(1),-Integer(1)], [-Integer(1),
↪Integer(0),-Integer(1)/Integer(2)], [Integer(1),Integer(1)/Integer(2),
↪Integer(0)]]); M                      # needs sage.modules
[   0    1   -1]
[  -1    0 -1/2]
[   1  1/2    0]
>>> G = DiGraph(M, sparse=True, weighted=True); G
↪# needs sage.modules
Digraph on 3 vertices
>>> G.weighted()
↪# needs sage.modules
True
```

- an incidence matrix:

```
sage: M = Matrix(6, [-1,0,0,0,1, 1,-1,0,0,0, 0,1,-1,0,0,                  ␣
↪  # needs sage.modules
....:                0,0,1,-1,0, 0,0,0,1,-1, 0,0,0,0,0]); M
[-1  0  0  0  1]
[ 1 -1  0  0  0]
[ 0  1 -1  0  0]
[ 0  0  1 -1  0]
[ 0  0  0  1 -1]
[ 0  0  0  0  0]
sage: DiGraph(M)                                                          ␣
↪  # needs sage.modules
Digraph on 6 vertices
```

```
>>> from sage.all import *
>>> M = Matrix(Integer(6), [-Integer(1),Integer(0),Integer(0),Integer(0),
↪Integer(1), Integer(1),-Integer(1),Integer(0),Integer(0),Integer(0),␣
↪Integer(0),Integer(1),-Integer(1),Integer(0),Integer(0),                 ␣
↪    # needs sage.modules
...                Integer(0),Integer(0),Integer(1),-Integer(1),
↪Integer(0), Integer(0),Integer(0),Integer(0),Integer(1),-Integer(1),␣
↪Integer(0),Integer(0),Integer(0),Integer(0),Integer(0)]); M
```

(continues on next page)

(continued from previous page)

```
[-1  0  0  0  1]
[ 1 -1  0  0  0]
[ 0  1 -1  0  0]
[ 0  0  1 -1  0]
[ 0  0  0  1 -1]
[ 0  0  0  0  0]
>>> DiGraph(M)
 ↪# needs sage.modules
Digraph on 6 vertices
```

5. A `dig6` string: Sage automatically recognizes whether a string is in `dig6` format, which is a directed version of `graph6`:

```
sage: D = DiGraph('IRAaDCIIOWEOKcPWAo')
sage: D
Digraph on 10 vertices

sage: D = DiGraph('IRAaDCIIOEOKcPWAo')
Traceback (most recent call last):
...
RuntimeError: the string (IRAaDCIIOEOKcPWAo) seems corrupt: for n = 10, the␣
 ↪string is too short

sage: D = DiGraph("IRAaDCI'OWEOKcPWAo")
Traceback (most recent call last):
...
RuntimeError: the string seems corrupt: valid characters are
?@ABCDEFGHIJKLMNOPQRSTUVWXYZ[\]^_`abcdefghijklmnopqrstuvwxyz{|}~
```

```
>>> from sage.all import *
>>> D = DiGraph('IRAaDCIIOWEOKcPWAo')
>>> D
Digraph on 10 vertices

>>> D = DiGraph('IRAaDCIIOEOKcPWAo')
Traceback (most recent call last):
...
RuntimeError: the string (IRAaDCIIOEOKcPWAo) seems corrupt: for n = 10, the␣
 ↪string is too short

>>> D = DiGraph("IRAaDCI'OWEOKcPWAo")
Traceback (most recent call last):
...
RuntimeError: the string seems corrupt: valid characters are
?@ABCDEFGHIJKLMNOPQRSTUVWXYZ[\]^_`abcdefghijklmnopqrstuvwxyz{|}~
```

6. A NetworkX MultiDiGraph:

```
sage: import networkx                                                    #␣
 ↪needs networkx
sage: g = networkx.MultiDiGraph({0: [1, 2, 3], 2: [4]})                  #␣
 ↪needs networkx
```

(continues on next page)

(continued from previous page)

```
sage: DiGraph(g)                                                              #␣
↪needs networkx
Multi-digraph on 5 vertices
```

```
>>> from sage.all import *
>>> import networkx                                                           #␣
↪needs networkx
>>> g = networkx.MultiDiGraph({Integer(0): [Integer(1), Integer(2),␣
↪Integer(3)], Integer(2): [Integer(4)]})                    # needs networkx
>>> DiGraph(g)                                                               #␣
↪needs networkx
Multi-digraph on 5 vertices
```

7. A NetworkX digraph:

```
sage: import networkx                                                         #␣
↪needs networkx
sage: g = networkx.DiGraph({0: [1, 2, 3], 2: [4]})                           #␣
↪needs networkx
sage: DiGraph(g)                                                              #␣
↪needs networkx
Digraph on 5 vertices
```

```
>>> from sage.all import *
>>> import networkx                                                           #␣
↪needs networkx
>>> g = networkx.DiGraph({Integer(0): [Integer(1), Integer(2), Integer(3)],␣
↪Integer(2): [Integer(4)]})                                 # needs networkx
>>> DiGraph(g)                                                               #␣
↪needs networkx
Digraph on 5 vertices
```

8. An igraph directed Graph (see also *igraph_graph()*):

```
sage: import igraph                                          # optional - python_
↪igraph
sage: g = igraph.Graph([(0,1),(0,2)], directed=True)  # optional - python_
↪igraph
sage: DiGraph(g)                                             # optional - python_
↪igraph
Digraph on 3 vertices
```

```
>>> from sage.all import *
>>> import igraph                                          # optional - python_igraph
>>> g = igraph.Graph([(Integer(0),Integer(1)),(Integer(0),Integer(2))],␣
↪directed=True)  # optional - python_igraph
>>> DiGraph(g)                                             # optional - python_igraph
Digraph on 3 vertices
```

If `vertex_labels` is `True`, the names of the vertices are given by the vertex attribute `'name'`, if available:

```
sage: # optional - python_igraph
sage: g = igraph.Graph([(0,1),(0,2)], directed=True, vertex_attrs={'name':['a
↪','b','c']})
sage: DiGraph(g).vertices(sort=True)
['a', 'b', 'c']
sage: g = igraph.Graph([(0,1),(0,2)], directed=True, vertex_attrs={'label':['a
↪','b','c']})
sage: DiGraph(g).vertices(sort=True)
[0, 1, 2]
```

```
>>> from sage.all import *
>>> # optional - python_igraph
>>> g = igraph.Graph([(Integer(0),Integer(1)),(Integer(0),Integer(2))],␣
↪directed=True, vertex_attrs={'name':['a','b','c']})
>>> DiGraph(g).vertices(sort=True)
['a', 'b', 'c']
>>> g = igraph.Graph([(Integer(0),Integer(1)),(Integer(0),Integer(2))],␣
↪directed=True, vertex_attrs={'label':['a','b','c']})
>>> DiGraph(g).vertices(sort=True)
[0, 1, 2]
```

If the igraph Graph has edge attributes, they are used as edge labels:

```
sage: g = igraph.Graph([(0, 1), (0, 2)], directed=True,                    #␣
↪optional - python_igraph
....:                  edge_attrs={'name':['a', 'b'], 'weight':[1, 3]})
sage: DiGraph(g).edges(sort=True)                                          #␣
↪optional - python_igraph
[(0, 1, {'name': 'a', 'weight': 1}), (0, 2, {'name': 'b', 'weight': 3})]
```

```
>>> from sage.all import *
>>> g = igraph.Graph([(Integer(0), Integer(1)), (Integer(0), Integer(2))],␣
↪directed=True,                  # optional - python_igraph
...                  edge_attrs={'name':['a', 'b'], 'weight':[Integer(1),␣
↪Integer(3)]})
>>> DiGraph(g).edges(sort=True)                                            #␣
↪optional - python_igraph
[(0, 1, {'name': 'a', 'weight': 1}), (0, 2, {'name': 'b', 'weight': 3})]
```

auslander_reiten_quiver()

Return the Auslander-Reiten quiver of `self`.

See also

`AuslanderReitenQuiver`

EXAMPLES:

```
sage: D = DiGraph([[1,2,'a'], [1,2,'b']], multiedges=True)
sage: D.auslander_reiten_quiver()
Auslander-Reiten quiver of Multi-digraph on 2 vertices
```

```
>>> from sage.all import *
>>> D = DiGraph([[Integer(1),Integer(2),'a'], [Integer(1),Integer(2),'b']],␣
↪multiedges=True)
>>> D.auslander_reiten_quiver()
Auslander-Reiten quiver of Multi-digraph on 2 vertices
```

center(*by_weight=False*, *algorithm=None*, *weight_function=None*, *check_weight=True*)

Return the set of vertices in the center of the DiGraph.

The center is the set of vertices whose eccentricity is equal to the radius of the DiGraph, i.e., achieving the minimum eccentricity.

For more information and examples on how to use input variables, see *shortest_paths()* and *eccentricity()*

INPUT:

- `by_weight` – boolean (default: `False`); if `True`, edge weights are taken into account; if `False`, all edges have weight 1
- `algorithm` – string (default: `None`); see method *eccentricity()* for the list of available algorithms
- `weight_function` – function (default: `None`); a function that takes as input an edge `(u, v, l)` and outputs its weight. If not `None`, `by_weight` is automatically set to `True`. If `None` and `by_weight` is `True`, we use the edge label `l` as a weight, if `l` is not `None`, else `1` as a weight.
- `check_weight` – boolean (default: `True`); if `True`, we check that the `weight_function` outputs a number for each edge

EXAMPLES:

Every vertex is a center in a Circuit-DiGraph:

```
sage: G = digraphs.Circuit(9)
sage: G.center()
[0, 1, 2, 3, 4, 5, 6, 7, 8]
```

```
>>> from sage.all import *
>>> G = digraphs.Circuit(Integer(9))
>>> G.center()
[0, 1, 2, 3, 4, 5, 6, 7, 8]
```

Center can be the whole graph:

```
sage: G.subgraph(G.center()) == G
True
```

```
>>> from sage.all import *
>>> G.subgraph(G.center()) == G
True
```

Some other graphs:

```
sage: G = digraphs.Path(5)
sage: G.center()
[0]
sage: G = DiGraph([(0,1,2), (1,2,3), (2,0,2)])
```

(continues on next page)

(continued from previous page)

```
sage: G.center(by_weight=True)
[2]
```

```
>>> from sage.all import *
>>> G = digraphs.Path(Integer(5))
>>> G.center()
[0]
>>> G = DiGraph([(Integer(0),Integer(1),Integer(2)), (Integer(1),Integer(2),
↪Integer(3)), (Integer(2),Integer(0),Integer(2))])
>>> G.center(by_weight=True)
[2]
```

degree_polynomial()

Return the generating polynomial of degrees of vertices in `self`.

This is the sum

$$\sum_{v \in G} x^{\text{in}(v)} y^{\text{out}(v)},$$

where `in(v)` and `out(v)` are the number of incoming and outgoing edges at vertex v in the digraph G.

Because this polynomial is multiplicative for Cartesian product of digraphs, it is useful to help see if the digraph can be isomorphic to a Cartesian product.

See also

n_vertices() for the value at $(x, y) = (1, 1)$

EXAMPLES:

```
sage: G = posets.PentagonPoset().hasse_diagram()                    #␣
↪needs sage.modules
sage: G.degree_polynomial()                                         #␣
↪needs sage.modules
x^2 + 3*x*y + y^2

sage: G = posets.BooleanLattice(4).hasse_diagram()
sage: G.degree_polynomial().factor()                                #␣
↪needs sage.libs.pari
(x + y)^4
```

```
>>> from sage.all import *
>>> G = posets.PentagonPoset().hasse_diagram()                      #␣
↪needs sage.modules
>>> G.degree_polynomial()                                           #␣
↪needs sage.modules
x^2 + 3*x*y + y^2

>>> G = posets.BooleanLattice(Integer(4)).hasse_diagram()
>>> G.degree_polynomial().factor()                                  #␣
↪needs sage.libs.pari
(x + y)^4
```

diameter(*by_weight=False*, *algorithm=None*, *weight_function=None*, *check_weight=True*)

Return the diameter of the DiGraph.

The diameter is defined to be the maximum distance between two vertices. It is infinite if the DiGraph is not strongly connected.

For more information and examples on how to use input variables, see *shortest_paths()* and *eccentricity()*

INPUT:

- `by_weight` – boolean (default: `False`); if `True`, edge weights are taken into account; if `False`, all edges have weight 1
- `algorithm` – string (default: `None`); one of the following algorithms:
  - `'BFS'`: the computation is done through a BFS centered on each vertex successively. Works only if `by_weight==False`. It computes all the eccentricities and return the maximum value.
  - `'Floyd-Warshall-Cython'`: a Cython implementation of the Floyd-Warshall algorithm. Works only if `by_weight==False`. It computes all the eccentricities and return the maximum value.
  - `'Floyd-Warshall-Python'`: a Python implementation of the Floyd-Warshall algorithm. Works also with weighted graphs, even with negative weights (but no negative cycle is allowed). It computes all the eccentricities and return the maximum value.
  - `'Dijkstra_NetworkX'`: the Dijkstra algorithm, implemented in NetworkX. It works with weighted graphs, but no negative weight is allowed. It computes all the eccentricities and return the maximum value.
  - `'DiFUB'`, `'2Dsweep'`: these algorithms are implemented in *sage.graphs.distances_all_pairs.diameter()* and *sage.graphs.base.boost_graph.diameter()*. `'2Dsweep'` returns lower bound on the diameter, while `'DiFUB'` returns the exact computed diameter. They also work with negative weight, if there is no negative cycle. See the functions documentation for more information.
  - `'standard'`: the standard algorithm is implemented in *sage.graphs.distances_all_pairs.diameter()*. It works only if `by_weight==False`. See the function documentation for more information. It computes all the eccentricities and return the maximum value.
  - `'Dijkstra_Boost'`: the Dijkstra algorithm, implemented in Boost (works only with positive weights). It computes all the eccentricities and return the maximum value.
  - `'Johnson_Boost'`: the Johnson algorithm, implemented in Boost (works also with negative weights, if there is no negative cycle). It computes all the eccentricities and return the maximum value.
  - `None` (default): Sage chooses the best algorithm: `'DiFUB'`.
- `weight_function` – function (default: `None`); a function that takes as input an edge `(u, v, l)` and outputs its weight. If not `None`, `by_weight` is automatically set to `True`. If `None` and `by_weight` is `True`, we use the edge label `l`, if `l` is not `None`, else `1` as weight.
- `check_weight` – boolean (default: `True`); if `True`, we check that the `weight_function` outputs a number for each edge

EXAMPLES:

```
sage: G = digraphs.DeBruijn(5,4)
sage: G.diameter()
4
```

(continues on next page)

(continued from previous page)

```
sage: G = digraphs.GeneralizedDeBruijn(9, 3)
sage: G.diameter()
2
```

```
>>> from sage.all import *
>>> G = digraphs.DeBruijn(Integer(5),Integer(4))
>>> G.diameter()
4
>>> G = digraphs.GeneralizedDeBruijn(Integer(9), Integer(3))
>>> G.diameter()
2
```

dig6_string()

Return the `dig6` representation of the digraph as an ASCII string.

This is only valid for simple (no multiple edges) digraphs on at most $2^{18} - 1 = 262143$ vertices.

Note

As the `dig6` format only handles graphs with vertex set $\{0, \ldots, n-1\}$, a *relabelled copy* will be encoded, if necessary.

See also

- *graph6_string()* – a similar string format for undirected graphs

EXAMPLES:

```
sage: D = DiGraph({0: [1, 2], 1: [2], 2: [3], 3: [0]})
sage: D.dig6_string()
'CW`_'
sage: L = DiGraph({0: [1, 2], 1: [2], 2: [3], 3: [3]})
sage: L.dig6_string()
'CW`C'
```

```
>>> from sage.all import *
>>> D = DiGraph({Integer(0): [Integer(1), Integer(2)], Integer(1):␣
↪[Integer(2)], Integer(2): [Integer(3)], Integer(3): [Integer(0)]})
>>> D.dig6_string()
'CW`_'
>>> L = DiGraph({Integer(0): [Integer(1), Integer(2)], Integer(1):␣
↪[Integer(2)], Integer(2): [Integer(3)], Integer(3): [Integer(3)]})
>>> L.dig6_string()
'CW`C'
```

eccentricity(*v=None*, *by_weight=False*, *algorithm=None*, *weight_function=None*, *check_weight=True*, *dist_dict=None*, *with_labels=False*)

Return the eccentricity of vertex (or vertices) `v`.

The eccentricity of a vertex is the maximum distance to any other vertex.

For more information and examples on how to use input variables, see *shortest_path_all_pairs()*, *shortest_path_lengths()* and *shortest_paths()*

INPUT:

- `v` – either a single vertex or a list of vertices. If it is not specified, then it is taken to be all vertices
- `by_weight` – boolean (default: `False`); if `True`, edge weights are taken into account; if `False`, all edges have weight 1
- `algorithm` – string (default: `None`); one of the following algorithms:
  - `'BFS'` – the computation is done through a BFS centered on each vertex successively. Works only if `by_weight==False`
  - `'Floyd-Warshall-Cython'` – a Cython implementation of the Floyd-Warshall algorithm. Works only if `by_weight==False` and `v is None` or `v` should contain all vertices of `self`.
  - `'Floyd-Warshall-Python'` – a Python implementation of the Floyd-Warshall algorithm. Works also with weighted graphs, even with negative weights (but no negative cycle is allowed). However, `v` must be `None` or `v` should contain all vertices of `self`.
  - `'Dijkstra_NetworkX'` – the Dijkstra algorithm, implemented in NetworkX. It works with weighted graphs, but no negative weight is allowed.
  - `'Dijkstra_Boost'` – the Dijkstra algorithm, implemented in Boost (works only with positive weights)
  - `'Johnson_Boost'` – the Johnson algorithm, implemented in Boost (works also with negative weights, if there is no negative cycle). Works only if `v is None` or `v` should contain all vertices of `self`.
  - `'From_Dictionary'` – uses the (already computed) distances, that are provided by input variable `dist_dict`
  - `None` (default): Sage chooses the best algorithm: `'From_Dictionary'` if `dist_dict` is not None, `'BFS'` for unweighted graphs, `'Dijkstra_Boost'` if all weights are positive, `'Johnson_Boost'` otherwise.
- `weight_function` – function (default: `None`); a function that takes as input an edge `(u, v, l)` and outputs its weight. If not `None`, `by_weight` is automatically set to `True`. If `None` and `by_weight` is `True`, we use the edge label `l`, if `l` is not `None`, else `1` as a weight.
- `check_weight` – boolean (default: `True`); if `True`, we check that the `weight_function` outputs a number for each edge
- `dist_dict` – dictionary (default: `None`); a dict of dicts of distances (used only if `algorithm=='From_Dictionary'`)
- `with_labels` – boolean (default: `False`); whether to return a list or a dictionary keyed by vertices

EXAMPLES:

```
sage: G = graphs.KrackhardtKiteGraph().to_directed()
sage: G.eccentricity()
[4, 4, 4, 4, 4, 3, 3, 2, 3, 4]
sage: G.vertices(sort=True)
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
sage: G.eccentricity(7)
2
sage: G.eccentricity([7,8,9])
[2, 3, 4]
```

(continues on next page)

(continued from previous page)

```
sage: G.eccentricity([7,8,9], with_labels=True) == {8: 3, 9: 4, 7: 2}
True
sage: G = DiGraph(3)
sage: G.eccentricity(with_labels=True)
{0: +Infinity, 1: +Infinity, 2: +Infinity}
sage: G = DiGraph({0:[]})
sage: G.eccentricity(with_labels=True)
{0: 0}
sage: G = DiGraph([(0,1,2), (1,2,3), (2,0,2)])
sage: G.eccentricity(algorithm='BFS')
[2, 2, 2]
sage: G.eccentricity(algorithm='Floyd-Warshall-Cython')
[2, 2, 2]
sage: G.eccentricity(by_weight=True, algorithm='Dijkstra_NetworkX')          #␣
↪needs networkx
[5, 5, 4]
sage: G.eccentricity(by_weight=True, algorithm='Dijkstra_Boost')
[5, 5, 4]
sage: G.eccentricity(by_weight=True, algorithm='Johnson_Boost')
[5, 5, 4]
sage: G.eccentricity(by_weight=True, algorithm='Floyd-Warshall-Python')
[5, 5, 4]
sage: G.eccentricity(dist_dict=G.shortest_path_all_pairs(by_weight=True)[0])
[5, 5, 4]
```

```
>>> from sage.all import *
>>> G = graphs.KrackhardtKiteGraph().to_directed()
>>> G.eccentricity()
[4, 4, 4, 4, 4, 3, 3, 2, 3, 4]
>>> G.vertices(sort=True)
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
>>> G.eccentricity(Integer(7))
2
>>> G.eccentricity([Integer(7),Integer(8),Integer(9)])
[2, 3, 4]
>>> G.eccentricity([Integer(7),Integer(8),Integer(9)], with_labels=True) ==
↪{Integer(8): Integer(3), Integer(9): Integer(4), Integer(7): Integer(2)}
True
>>> G = DiGraph(Integer(3))
>>> G.eccentricity(with_labels=True)
{0: +Infinity, 1: +Infinity, 2: +Infinity}
>>> G = DiGraph({Integer(0):[]})
>>> G.eccentricity(with_labels=True)
{0: 0}
>>> G = DiGraph([(Integer(0),Integer(1),Integer(2)), (Integer(1),Integer(2),
↪Integer(3)), (Integer(2),Integer(0),Integer(2))])
>>> G.eccentricity(algorithm='BFS')
[2, 2, 2]
>>> G.eccentricity(algorithm='Floyd-Warshall-Cython')
[2, 2, 2]
>>> G.eccentricity(by_weight=True, algorithm='Dijkstra_NetworkX')          #␣
↪needs networkx
```

(continues on next page)

(continued from previous page)

```
[5, 5, 4]
>>> G.eccentricity(by_weight=True, algorithm='Dijkstra_Boost')
[5, 5, 4]
>>> G.eccentricity(by_weight=True, algorithm='Johnson_Boost')
[5, 5, 4]
>>> G.eccentricity(by_weight=True, algorithm='Floyd-Warshall-Python')
[5, 5, 4]
>>> G.eccentricity(dist_dict=G.shortest_path_all_pairs(by_
↪weight=True)[Integer(0)])
[5, 5, 4]
```

feedback_edge_set(*constraint_generation*, *value_only=True*, *solver=False*, *verbose=None*, *integrality_tolerance=0*)

Compute the minimum feedback edge set of a digraph (also called feedback arc set).

The minimum feedback edge set of a digraph is a set of edges that intersect all the circuits of the digraph. Equivalently, a minimum feedback arc set of a DiGraph is a set S of arcs such that the digraph $G - S$ is acyclic. For more information, see the Wikipedia article Feedback_arc_set.

INPUT:

- `value_only` – boolean (default: `False`)
  - When set to `True`, only the minimum cardinal of a minimum edge set is returned.
  - When set to `False`, the `Set` of edges of a minimal edge set is returned.
- `constraint_generation` – boolean (default: `True`); whether to use constraint generation when solving the Mixed Integer Linear Program.
- `solver` – string (default: `None`); specifies a Mixed Integer Linear Programming (MILP) solver to be used. If set to `None`, the default one is used. For more information on MILP solvers and which default solver is used, see the method `solve` of the class `MixedIntegerLinearProgram`.
- `verbose` – integer (default: 0); sets the level of verbosity. Set to 0 by default, which means quiet.
- `integrality_tolerance` – float; parameter for use with MILP solvers over an inexact base ring; see `MixedIntegerLinearProgram.get_values()`.

ALGORITHM:

This problem is solved using Linear Programming, in two different ways. The first one is to solve the following formulation:

$$\text{Minimize : } \sum_{(u,v)\in G} b_{(u,v)}$$

$$\text{Such that :}$$

$$\forall (u,v) \in G, d_u - d_v + n \cdot b_{(u,v)} \geq 0$$

$$\forall u \in G, 0 \leq d_u \leq |G|$$

An explanation:

An acyclic digraph can be seen as a poset, and every poset has a linear extension. This means that in any acyclic digraph the vertices can be ordered with a total order $<$ in such a way that if $(u, v) \in G$, then $u < v$.

Thus, this linear program is built in order to assign to each vertex v a number $d_v \in [0, \ldots, n-1]$ such that if there exists an edge $(u, v) \in G$ such that $d_v < d_u$, then the edge (u, v) is removed.

The number of edges removed is then minimized, which is the objective.

(Constraint Generation)

If the parameter `constraint_generation` is enabled, a more efficient formulation is used :

$$\text{Minimize : } \sum_{(u,v)\in G} b_{(u,v)}$$

Such that :

$$\forall C \text{ circuits } \subseteq G, \sum_{uv\in C} b_{(u,v)} \geq 1$$

As the number of circuits contained in a graph is exponential, this LP is solved through constraint generation. This means that the solver is sequentially asked to solved the problem, knowing only a portion of the circuits contained in G, each time adding to the list of its constraints the circuit which its last answer had left intact.

EXAMPLES:

If the digraph is created from a graph, and hence is symmetric (if uv is an edge, then vu is an edge too), then obviously the cardinality of its feedback arc set is the number of edges in the first graph:

```
sage: cycle = graphs.CycleGraph(5)
sage: dcycle = DiGraph(cycle)
sage: cycle.size()
5
sage: dcycle.feedback_edge_set(value_only=True)                                 #␣
↪needs sage.numerical.mip
5
```

```
>>> from sage.all import *
>>> cycle = graphs.CycleGraph(Integer(5))
>>> dcycle = DiGraph(cycle)
>>> cycle.size()
5
>>> dcycle.feedback_edge_set(value_only=True)                                   #␣
↪needs sage.numerical.mip
5
```

And in this situation, for any edge uv of the first graph, uv of vu is in the returned feedback arc set:

```
sage: g = graphs.RandomGNP(5,.3)
sage: while not g.n_edges():
....:     g = graphs.RandomGNP(5,.3)
sage: dg = DiGraph(g)
sage: feedback = dg.feedback_edge_set()
↪# needs sage.numerical.mip
sage: u,v,l = next(g.edge_iterator())
sage: (u,v) in feedback or (v,u) in feedback
↪# needs sage.numerical.mip
True
```

```
>>> from sage.all import *
>>> g = graphs.RandomGNP(Integer(5),RealNumber('.3'))
>>> while not g.n_edges():
...     g = graphs.RandomGNP(Integer(5),RealNumber('.3'))
>>> dg = DiGraph(g)
```

(continues on next page)

(continued from previous page)

```
>>> feedback = dg.feedback_edge_set()                                    #␣
↪needs sage.numerical.mip
>>> u,v,l = next(g.edge_iterator())
>>> (u,v) in feedback or (v,u) in feedback                               #␣
↪needs sage.numerical.mip
True
```

flow_polytope(*edges=None*, *ends=None*, *backend=None*)

Return the flow polytope of a digraph.

The flow polytope of a directed graph is the polytope consisting of all nonnegative flows on the graph with a given set S of sources and a given set T of sinks.

A *flow* on a directed graph G with a given set S of sources and a given set T of sinks means an assignment of a nonnegative real to each edge of G such that the flow is conserved in each vertex outside of S and T, and there is a unit of flow entering each vertex in S and a unit of flow leaving each vertex in T. These flows clearly form a polytope in the space of all assignments of reals to the edges of G.

The polytope is empty unless the sets S and T are equinumerous.

By default, S is taken to be the set of all sources (i.e., vertices of indegree 0) of G, and T is taken to be the set of all sinks (i.e., vertices of outdegree 0) of G. If a different choice of S and T is desired, it can be specified using the optional `ends` parameter.

The polytope is returned as a polytope in $\mathbf{R}^m$, where m is the number of edges of the digraph `self`. The k-th coordinate of a point in the polytope is the real assigned to the k-th edge of `self`. The order of the edges is the one returned by `self.edges(sort=True)`. If a different order is desired, it can be specified using the optional `edges` parameter.

The faces and volume of these polytopes are of interest. Examples of these polytopes are the Chan-Robbins-Yuen polytope and the Pitman-Stanley polytope [PS2002].

INPUT:

- `edges` – list (default: `None`); a list of edges of `self`. If not specified, the list of all edges of `self` is used with the default ordering of `self.edges(sort=True)`. This determines which coordinate of a point in the polytope will correspond to which edge of `self`. It is also possible to specify a list which contains not all edges of `self`; this results in a polytope corresponding to the flows which are 0 on all remaining edges. Notice that the edges entered here must be in the precisely same format as outputted by `self.edges()`; so, if `self.edges()` outputs an edge in the form `(1, 3, None)`, then `(1, 3)` will not do!
- `ends` – (default: `(self.sources(), self.sinks())`) a pair (S, T) of an iterable S and an iterable T
- `backend` – string or `None` (default); the backend to use; see the `Polyhedron constructor`

Note

Flow polytopes can also be built through the `polytopes.<tab>` object:

```
sage: polytopes.flow_polytope(digraphs.Path(5))                          #␣
↪needs sage.geometry.polyhedron
A 0-dimensional polyhedron in QQ^4 defined as the convex hull of 1 vertex
```

```
>>> from sage.all import *
>>> polytopes.flow_polytope(digraphs.Path(Integer(5)))                   ␣
↪    # needs sage.geometry.polyhedron
A 0-dimensional polyhedron in QQ^4 defined as the convex hull of 1 vertex
```

EXAMPLES:

A commutative square:

```
sage: G = DiGraph({1: [2, 3], 2: [4], 3: [4]})
sage: fl = G.flow_polytope(); fl                                                #␣
↪needs sage.geometry.polyhedron
A 1-dimensional polyhedron in QQ^4 defined as the convex hull
of 2 vertices
sage: fl.vertices()                                                             #␣
↪needs sage.geometry.polyhedron
(A vertex at (0, 1, 0, 1), A vertex at (1, 0, 1, 0))
```

```
>>> from sage.all import *
>>> G = DiGraph({Integer(1): [Integer(2), Integer(3)], Integer(2):␣
↪[Integer(4)], Integer(3): [Integer(4)]})
>>> fl = G.flow_polytope(); fl                                                  #␣
↪needs sage.geometry.polyhedron
A 1-dimensional polyhedron in QQ^4 defined as the convex hull
of 2 vertices
>>> fl.vertices()                                                               #␣
↪needs sage.geometry.polyhedron
(A vertex at (0, 1, 0, 1), A vertex at (1, 0, 1, 0))
```

Using a different order for the edges of the graph:

```
sage: ordered_edges = G.edges(sort=True, key=lambda x: x[0] - x[1])
sage: fl = G.flow_polytope(edges=ordered_edges); fl                             #␣
↪needs sage.geometry.polyhedron
A 1-dimensional polyhedron in QQ^4 defined as the convex hull of 2 vertices
sage: fl.vertices()                                                             #␣
↪needs sage.geometry.polyhedron
(A vertex at (0, 1, 1, 0), A vertex at (1, 0, 0, 1))
```

```
>>> from sage.all import *
>>> ordered_edges = G.edges(sort=True, key=lambda x: x[Integer(0)] -␣
↪x[Integer(1)])
>>> fl = G.flow_polytope(edges=ordered_edges); fl                               #␣
↪needs sage.geometry.polyhedron
A 1-dimensional polyhedron in QQ^4 defined as the convex hull of 2 vertices
>>> fl.vertices()                                                               #␣
↪needs sage.geometry.polyhedron
(A vertex at (0, 1, 1, 0), A vertex at (1, 0, 0, 1))
```

A tournament on 4 vertices:

```
sage: H = digraphs.TransitiveTournament(4)
sage: fl = H.flow_polytope(); fl                                                #␣
↪needs sage.geometry.polyhedron
A 3-dimensional polyhedron in QQ^6 defined as the convex hull
of 4 vertices
```

(continues on next page)

(continued from previous page)

```
sage: fl.vertices()                                                          #␣
↪needs sage.geometry.polyhedron
(A vertex at (0, 0, 1, 0, 0, 0),
 A vertex at (0, 1, 0, 0, 0, 1),
 A vertex at (1, 0, 0, 0, 1, 0),
 A vertex at (1, 0, 0, 1, 0, 1))
```

```
>>> from sage.all import *
>>> H = digraphs.TransitiveTournament(Integer(4))
>>> fl = H.flow_polytope(); fl                                               #␣
↪needs sage.geometry.polyhedron
A 3-dimensional polyhedron in QQ^6 defined as the convex hull
of 4 vertices
>>> fl.vertices()                                                            #␣
↪needs sage.geometry.polyhedron
(A vertex at (0, 0, 1, 0, 0, 0),
 A vertex at (0, 1, 0, 0, 0, 1),
 A vertex at (1, 0, 0, 0, 1, 0),
 A vertex at (1, 0, 0, 1, 0, 1))
```

Restricting to a subset of the edges:

```
sage: fl = H.flow_polytope(edges=[(0, 1, None), (1, 2, None),                #␣
↪needs sage.geometry.polyhedron
....:                             (2, 3, None), (0, 3, None)]); fl
A 1-dimensional polyhedron in QQ^4 defined as the convex hull
of 2 vertices
sage: fl.vertices()                                                          #␣
↪needs sage.geometry.polyhedron
(A vertex at (0, 0, 0, 1), A vertex at (1, 1, 1, 0))
```

```
>>> from sage.all import *
>>> fl = H.flow_polytope(edges=[(Integer(0), Integer(1), None), (Integer(1),␣
↪Integer(2), None),                # needs sage.geometry.polyhedron
...                               (Integer(2), Integer(3), None), (Integer(0),␣
↪Integer(3), None)]); fl
A 1-dimensional polyhedron in QQ^4 defined as the convex hull
of 2 vertices
>>> fl.vertices()                                                            #␣
↪needs sage.geometry.polyhedron
(A vertex at (0, 0, 0, 1), A vertex at (1, 1, 1, 0))
```

Using a different choice of sources and sinks:

```
sage: fl = H.flow_polytope(ends=([1], [3])); fl
A 1-dimensional polyhedron in QQ^6 defined as the convex hull
of 2 vertices
sage: fl.vertices()
(A vertex at (0, 0, 0, 1, 0, 1), A vertex at (0, 0, 0, 0, 1, 0))
sage: fl = H.flow_polytope(ends=([0, 1], [3])); fl
The empty polyhedron in QQ^6
sage: fl = H.flow_polytope(ends=([3], [0])); fl
```

(continues on next page)

(continued from previous page)

```
The empty polyhedron in QQ^6
sage: fl = H.flow_polytope(ends=([0, 1], [2, 3])); fl
A 3-dimensional polyhedron in QQ^6 defined as the convex hull
of 5 vertices
sage: fl.vertices()
(A vertex at (0, 0, 1, 1, 0, 0),
 A vertex at (0, 1, 0, 0, 1, 0),
 A vertex at (1, 0, 0, 2, 0, 1),
 A vertex at (1, 0, 0, 1, 1, 0),
 A vertex at (0, 1, 0, 1, 0, 1))
sage: fl = H.flow_polytope(edges=[(0, 1, None), (1, 2, None),
....:                             (2, 3, None), (0, 2, None),
....:                             (1, 3, None)],
....:                      ends=([0, 1], [2, 3])); fl
A 2-dimensional polyhedron in QQ^5 defined as the convex hull
of 4 vertices
sage: fl.vertices()
(A vertex at (0, 0, 0, 1, 1),
 A vertex at (1, 2, 1, 0, 0),
 A vertex at (1, 1, 0, 0, 1),
 A vertex at (0, 1, 1, 1, 0))
```

```
>>> from sage.all import *
>>> fl = H.flow_polytope(ends=([Integer(1)], [Integer(3)])); fl
A 1-dimensional polyhedron in QQ^6 defined as the convex hull
of 2 vertices
>>> fl.vertices()
(A vertex at (0, 0, 0, 1, 0, 1), A vertex at (0, 0, 0, 0, 1, 0))
>>> fl = H.flow_polytope(ends=([Integer(0), Integer(1)], [Integer(3)])); fl
The empty polyhedron in QQ^6
>>> fl = H.flow_polytope(ends=([Integer(3)], [Integer(0)])); fl
The empty polyhedron in QQ^6
>>> fl = H.flow_polytope(ends=([Integer(0), Integer(1)], [Integer(2),␣
↪Integer(3)])); fl
A 3-dimensional polyhedron in QQ^6 defined as the convex hull
of 5 vertices
>>> fl.vertices()
(A vertex at (0, 0, 1, 1, 0, 0),
 A vertex at (0, 1, 0, 0, 1, 0),
 A vertex at (1, 0, 0, 2, 0, 1),
 A vertex at (1, 0, 0, 1, 1, 0),
 A vertex at (0, 1, 0, 1, 0, 1))
>>> fl = H.flow_polytope(edges=[(Integer(0), Integer(1), None), (Integer(1),␣
↪Integer(2), None),
...                             (Integer(2), Integer(3), None), (Integer(0),␣
↪Integer(2), None),
...                             (Integer(1), Integer(3), None)],
...                      ends=([Integer(0), Integer(1)], [Integer(2),␣
↪Integer(3)])); fl
A 2-dimensional polyhedron in QQ^5 defined as the convex hull
of 4 vertices
>>> fl.vertices()
```

(continues on next page)

(continued from previous page)

```
(A vertex at (0, 0, 0, 1, 1),
 A vertex at (1, 2, 1, 0, 0),
 A vertex at (1, 1, 0, 0, 1),
 A vertex at (0, 1, 1, 1, 0))
```

A digraph with one source and two sinks:

```
sage: Y = DiGraph({1: [2], 2: [3, 4]})
sage: Y.flow_polytope()                                                    #␣
↪needs sage.geometry.polyhedron
The empty polyhedron in QQ^3
```

```
>>> from sage.all import *
>>> Y = DiGraph({Integer(1): [Integer(2)], Integer(2): [Integer(3),␣
↪Integer(4)]})
>>> Y.flow_polytope()                                                      #␣
↪needs sage.geometry.polyhedron
The empty polyhedron in QQ^3
```

A digraph with one vertex and no edge:

```
sage: Z = DiGraph({1: []})
sage: Z.flow_polytope()                                                    #␣
↪needs sage.geometry.polyhedron
A 0-dimensional polyhedron in QQ^0 defined as the convex hull
of 1 vertex
```

```
>>> from sage.all import *
>>> Z = DiGraph({Integer(1): []})
>>> Z.flow_polytope()                                                      #␣
↪needs sage.geometry.polyhedron
A 0-dimensional polyhedron in QQ^0 defined as the convex hull
of 1 vertex
```

A digraph with multiple edges (Issue #28837):

```
sage: G = DiGraph([(0, 1), (0,1)], multiedges=True); G
Multi-digraph on 2 vertices
sage: P = G.flow_polytope(); P                                             #␣
↪needs sage.geometry.polyhedron
A 1-dimensional polyhedron in QQ^2 defined as the convex hull of 2 vertices
sage: P.vertices()                                                         #␣
↪needs sage.geometry.polyhedron
(A vertex at (1, 0), A vertex at (0, 1))
sage: P.lines()                                                            #␣
↪needs sage.geometry.polyhedron
()
```

```
>>> from sage.all import *
>>> G = DiGraph([(Integer(0), Integer(1)), (Integer(0),Integer(1))],␣
↪multiedges=True); G
Multi-digraph on 2 vertices
```

(continues on next page)

(continued from previous page)

```
>>> P = G.flow_polytope(); P                                                    #␣
↪needs sage.geometry.polyhedron
A 1-dimensional polyhedron in QQ^2 defined as the convex hull of 2 vertices
>>> P.vertices()                                                               #␣
↪needs sage.geometry.polyhedron
(A vertex at (1, 0), A vertex at (0, 1))
>>> P.lines()                                                                  #␣
↪needs sage.geometry.polyhedron
()
```

in_branchings(*source*, *spanning=True*)

Return an iterator over the in branchings rooted at given vertex in `self`.

An in-branching is a directed tree rooted at `source` whose arcs are directed to source from leaves. An in-branching is spanning if it contains all vertices of the digraph.

If no spanning in branching rooted at `source` exist, raises `ValueError` or return non spanning in branching rooted at `source`, depending on the value of `spanning`.

INPUT:

- `source` – vertex used as the source for all in branchings
- `spanning` – boolean (default: `True`); if `False` return maximum in branching to `source`. Otherwise, return spanning in branching if exists.

OUTPUT: an iterator over the in branchings rooted in the given source

See also

- *out_branchings()* – iterator over out-branchings rooted at given vertex.
- *spanning_trees()* – returns all spanning trees.
- *number_of_spanning_trees()* – counts the number of spanning trees.

ALGORITHM:

Recursively computes all in branchings.

At each step:

0. clean the graph (see below)
1. pick an edge e incoming to source
2. find all in branchings that do not contain e by first removing it
3. find all in branchings that do contain e by first merging the end vertices of e

Cleaning the graph implies to remove loops and replace multiedges by a single one with an appropriate label since these lead to similar steps of computation.

EXAMPLES:

A bidirectional 4-cycle:

```
sage: G = DiGraph({1:[2,3], 2:[1,4], 3:[1,4], 4:[2,3]}, format='dict_of_lists
↪')
sage: list(G.in_branchings(1))
[Digraph on 4 vertices,
 Digraph on 4 vertices,
 Digraph on 4 vertices,
 Digraph on 4 vertices]
```

```
>>> from sage.all import *
>>> G = DiGraph({Integer(1):[Integer(2),Integer(3)], Integer(2):[Integer(1),
↪Integer(4)], Integer(3):[Integer(1),Integer(4)], Integer(4):[Integer(2),
↪Integer(3)]}, format='dict_of_lists')
>>> list(G.in_branchings(Integer(1)))
[Digraph on 4 vertices,
 Digraph on 4 vertices,
 Digraph on 4 vertices,
 Digraph on 4 vertices]
```

With the Petersen graph turned into a symmetric directed graph:

```
sage: G = graphs.PetersenGraph().to_directed()
sage: len(list(G.in_branchings(0)))
2000
```

```
>>> from sage.all import *
>>> G = graphs.PetersenGraph().to_directed()
>>> len(list(G.in_branchings(Integer(0))))
2000
```

With a non connected `DiGraph` and `spanning = True`:

```
sage: G = graphs.PetersenGraph().to_directed() + graphs.PetersenGraph().to_
↪directed()
sage: G.in_branchings(0)
Traceback (most recent call last):
...
ValueError: no spanning in branching to vertex (0) exist
```

```
>>> from sage.all import *
>>> G = graphs.PetersenGraph().to_directed() + graphs.PetersenGraph().to_
↪directed()
>>> G.in_branchings(Integer(0))
Traceback (most recent call last):
...
ValueError: no spanning in branching to vertex (0) exist
```

With a non connected `DiGraph` and `spanning = False`:

```
sage: g=DiGraph([(1,0), (1,0), (2,1), (3,4)],multiedges=True)
sage: list(g.in_branchings(0,spanning=False))
[Digraph on 3 vertices, Digraph on 3 vertices]
```

```
>>> from sage.all import *
>>> g=DiGraph([(Integer(1),Integer(0)), (Integer(1),Integer(0)), (Integer(2),
↪Integer(1)), (Integer(3),Integer(4))],multiedges=True)
>>> list(g.in_branchings(Integer(0),spanning=False))
[Digraph on 3 vertices, Digraph on 3 vertices]
```

With multiedges:

```
sage: G = DiGraph({0:[1,1,1], 1:[2,2]}, format='dict_of_lists',␣
↪multiedges=True)
sage: len(list(G.in_branchings(2)))
6
```

```
>>> from sage.all import *
>>> G = DiGraph({Integer(0):[Integer(1),Integer(1),Integer(1)],␣
↪Integer(1):[Integer(2),Integer(2)]}, format='dict_of_lists',␣
↪multiedges=True)
>>> len(list(G.in_branchings(Integer(2))))
6
```

With a DiGraph already being a spanning in branching:

```
sage: G = DiGraph({0:[], 1:[0], 2:[0], 3:[1], 4:[1], 5:[2]}, format='dict_of_
↪lists')
sage: next(G.in_branchings(0)) == G
True
```

```
>>> from sage.all import *
>>> G = DiGraph({Integer(0):[], Integer(1):[Integer(0)],␣
↪Integer(2):[Integer(0)], Integer(3):[Integer(1)], Integer(4):[Integer(1)],␣
↪Integer(5):[Integer(2)]}, format='dict_of_lists')
>>> next(G.in_branchings(Integer(0))) == G
True
```

in_degree(*vertices=None*, *labels=False*)

Same as degree, but for in degree.

EXAMPLES:

```
sage: D = DiGraph({0: [1,2,3], 1: [0,2], 2: [3], 3: [4], 4: [0,5], 5: [1]})
sage: D.in_degree(vertices=[0, 1, 2], labels=True)
{0: 2, 1: 2, 2: 2}
sage: D.in_degree()
[2, 2, 2, 2, 1, 1]
sage: G = graphs.PetersenGraph().to_directed()
sage: G.in_degree(0)
3
```

```
>>> from sage.all import *
>>> D = DiGraph({Integer(0): [Integer(1),Integer(2),Integer(3)], Integer(1):␣
↪[Integer(0),Integer(2)], Integer(2): [Integer(3)], Integer(3): [Integer(4)],
↪ Integer(4): [Integer(0),Integer(5)], Integer(5): [Integer(1)]})
>>> D.in_degree(vertices=[Integer(0), Integer(1), Integer(2)], labels=True)
```

(continues on next page)

(continued from previous page)

```
{0: 2, 1: 2, 2: 2}
>>> D.in_degree()
[2, 2, 2, 2, 1, 1]
>>> G = graphs.PetersenGraph().to_directed()
>>> G.in_degree(Integer(0))
3
```

in_degree_iterator(*vertices=None*, *labels=False*)

Same as degree_iterator, but for in degree.

EXAMPLES:

```
sage: D = graphs.Grid2dGraph(2,4).to_directed()
sage: sorted(D.in_degree_iterator())
[2, 2, 2, 2, 3, 3, 3, 3]
sage: sorted(D.in_degree_iterator(labels=True))
[((0, 0), 2),
 ((0, 1), 3),
 ((0, 2), 3),
 ((0, 3), 2),
 ((1, 0), 2),
 ((1, 1), 3),
 ((1, 2), 3),
 ((1, 3), 2)]
```

```
>>> from sage.all import *
>>> D = graphs.Grid2dGraph(Integer(2),Integer(4)).to_directed()
>>> sorted(D.in_degree_iterator())
[2, 2, 2, 2, 3, 3, 3, 3]
>>> sorted(D.in_degree_iterator(labels=True))
[((0, 0), 2),
 ((0, 1), 3),
 ((0, 2), 3),
 ((0, 3), 2),
 ((1, 0), 2),
 ((1, 1), 3),
 ((1, 2), 3),
 ((1, 3), 2)]
```

in_degree_sequence()

Return the in-degree sequence.

EXAMPLES:

The in-degree sequences of two digraphs:

```
sage: g = DiGraph({1: [2, 5, 6], 2: [3, 6], 3: [4, 6], 4: [6], 5: [4, 6]})
sage: g.in_degree_sequence()
[5, 2, 1, 1, 1, 0]
```

```
>>> from sage.all import *
>>> g = DiGraph({Integer(1): [Integer(2), Integer(5), Integer(6)],␣
↪Integer(2): [Integer(3), Integer(6)], Integer(3): [Integer(4), Integer(6)],␣
```

(continues on next page)

(continued from previous page)

```
↪Integer(4): [Integer(6)], Integer(5): [Integer(4), Integer(6)]})
>>> g.in_degree_sequence()
[5, 2, 1, 1, 1, 0]
```

```
sage: V = [2, 3, 5, 7, 8, 9, 10, 11]
sage: E = [[], [8, 10], [11], [8, 11], [9], [], [], [2, 9, 10]]
sage: g = DiGraph(dict(zip(V, E)))
sage: g.in_degree_sequence()
[2, 2, 2, 2, 1, 0, 0, 0]
```

```
>>> from sage.all import *
>>> V = [Integer(2), Integer(3), Integer(5), Integer(7), Integer(8),␣
↪Integer(9), Integer(10), Integer(11)]
>>> E = [[], [Integer(8), Integer(10)], [Integer(11)], [Integer(8),␣
↪Integer(11)], [Integer(9)], [], [], [Integer(2), Integer(9), Integer(10)]]
>>> g = DiGraph(dict(zip(V, E)))
>>> g.in_degree_sequence()
[2, 2, 2, 2, 1, 0, 0, 0]
```

incoming_edge_iterator(*vertices*, *labels=True*)

Return an iterator over all arriving edges from vertices.

INPUT:

- `vertices` – a vertex or a list of vertices
- `labels` – boolean (default: `True`); whether to return edges as pairs of vertices, or as triples containing the labels

EXAMPLES:

```
sage: D = DiGraph({0: [1,2,3], 1: [0,2], 2: [3], 3: [4], 4: [0,5], 5: [1]})
sage: for a in D.incoming_edge_iterator([0]):
....:     print(a)
(1, 0, None)
(4, 0, None)
```

```
>>> from sage.all import *
>>> D = DiGraph({Integer(0): [Integer(1),Integer(2),Integer(3)], Integer(1):␣
↪[Integer(0),Integer(2)], Integer(2): [Integer(3)], Integer(3): [Integer(4)],
↪ Integer(4): [Integer(0),Integer(5)], Integer(5): [Integer(1)]})
>>> for a in D.incoming_edge_iterator([Integer(0)]):
...     print(a)
(1, 0, None)
(4, 0, None)
```

incoming_edges(*vertices*, *labels=True*)

Return a list of edges arriving at vertices.

INPUT:

- `vertices` – a vertex or a list of vertices
- `labels` – boolean (default: `True`); whether to return edges as pairs of vertices, or as triples containing the labels

EXAMPLES:

```
sage: D = DiGraph({0: [1,2,3], 1: [0,2], 2: [3], 3: [4], 4: [0,5], 5: [1]})
sage: D.incoming_edges([0])
[(1, 0, None), (4, 0, None)]
```

```
>>> from sage.all import *
>>> D = DiGraph({Integer(0): [Integer(1),Integer(2),Integer(3)], Integer(1):␣
↪[Integer(0),Integer(2)], Integer(2): [Integer(3)], Integer(3): [Integer(4)],
↪ Integer(4): [Integer(0),Integer(5)], Integer(5): [Integer(1)]})
>>> D.incoming_edges([Integer(0)])
[(1, 0, None), (4, 0, None)]
```

is_aperiodic()

Return whether the current `DiGraph` is aperiodic.

A directed graph is aperiodic if there is no integer $k > 1$ that divides the length of every cycle in the graph. See the Wikipedia article Aperiodic_graph for more information.

EXAMPLES:

The following graph has period `2`, so it is not aperiodic:

```
sage: g = DiGraph({0: [1], 1: [0]})
sage: g.is_aperiodic()
False
```

```
>>> from sage.all import *
>>> g = DiGraph({Integer(0): [Integer(1)], Integer(1): [Integer(0)]})
>>> g.is_aperiodic()
False
```

The following graph has a cycle of length 2 and a cycle of length 3, so it is aperiodic:

```
sage: g = DiGraph({0: [1, 4], 1: [2], 2: [0], 4: [0]})
sage: g.is_aperiodic()
True
```

```
>>> from sage.all import *
>>> g = DiGraph({Integer(0): [Integer(1), Integer(4)], Integer(1):␣
↪[Integer(2)], Integer(2): [Integer(0)], Integer(4): [Integer(0)]})
>>> g.is_aperiodic()
True
```

See also

period()

is_directed()

Since digraph is directed, return `True`.

EXAMPLES:

```
sage: DiGraph().is_directed()
True
```

```
>>> from sage.all import *
>>> DiGraph().is_directed()
True
```

is_directed_acyclic(*certificate=False*)

Check whether the digraph is acyclic or not.

A directed graph is acyclic if for any vertex v, there is no directed path that starts and ends at v. Every directed acyclic graph (DAG) corresponds to a partial ordering of its vertices, however multiple dags may lead to the same partial ordering.

INPUT:

- `certificate` – boolean (default: `False`); whether to return a certificate

OUTPUT:

- When `certificate=False`, returns a boolean value
- When `certificate=True`:
  - If the graph is acyclic, returns a pair `(True, ordering)` where `ordering` is a list of the vertices such that u appears before v in `ordering` if uv is an edge.
  - Else, returns a pair `(False, cycle)` where `cycle` is a list of vertices representing a circuit in the graph.

EXAMPLES:

At first, the following graph is acyclic:

```
sage: D = DiGraph({0:[1, 2, 3], 4:[2, 5], 1:[8], 2:[7], 3:[7], 5:[6,7], 7:[8],
↪ 6:[9], 8:[10], 9:[10]})
sage: D.plot(layout='circular').show()                                    #␣
↪needs sage.plot
sage: D.is_directed_acyclic()
True
```

```
>>> from sage.all import *
>>> D = DiGraph({Integer(0):[Integer(1), Integer(2), Integer(3)],␣
↪Integer(4):[Integer(2), Integer(5)], Integer(1):[Integer(8)],␣
↪Integer(2):[Integer(7)], Integer(3):[Integer(7)], Integer(5):[Integer(6),
↪Integer(7)], Integer(7):[Integer(8)], Integer(6):[Integer(9)],␣
↪Integer(8):[Integer(10)], Integer(9):[Integer(10)]})
>>> D.plot(layout='circular').show()                                      #␣
↪needs sage.plot
>>> D.is_directed_acyclic()
True
```

Adding an edge from 9 to 7 does not change it:

```
sage: D.add_edge(9, 7)
sage: D.is_directed_acyclic()
True
```

```
>>> from sage.all import *
>>> D.add_edge(Integer(9), Integer(7))
>>> D.is_directed_acyclic()
True
```

We can obtain as a proof an ordering of the vertices such that u appears before v if uv is an edge of the graph:

```
sage: D.is_directed_acyclic(certificate=True)
(True, [4, 5, 6, 9, 0, 1, 2, 3, 7, 8, 10])
```

```
>>> from sage.all import *
>>> D.is_directed_acyclic(certificate=True)
(True, [4, 5, 6, 9, 0, 1, 2, 3, 7, 8, 10])
```

Adding an edge from 7 to 4, though, makes a difference:

```
sage: D.add_edge(7, 4)
sage: D.is_directed_acyclic()
False
```

```
>>> from sage.all import *
>>> D.add_edge(Integer(7), Integer(4))
>>> D.is_directed_acyclic()
False
```

Indeed, it creates a circuit $7, 4, 5$:

```
sage: D.is_directed_acyclic(certificate=True)
(False, [7, 4, 5])
```

```
>>> from sage.all import *
>>> D.is_directed_acyclic(certificate=True)
(False, [7, 4, 5])
```

Checking acyclic graphs are indeed acyclic

```
sage: def random_acyclic(n, p):
....:   g = graphs.RandomGNP(n, p)
....:   h = DiGraph()
....:   h.add_edges(((u, v) if u < v else (v, u)) for u, v in g.edge_
↪iterator(labels=False))
....:   return h
...
sage: all(random_acyclic(100, .2).is_directed_acyclic()     # long time
....:       for i in range(50))
True
```

```
>>> from sage.all import *
>>> def random_acyclic(n, p):
...   g = graphs.RandomGNP(n, p)
...   h = DiGraph()
...   h.add_edges(((u, v) if u < v else (v, u)) for u, v in g.edge_
```

(continues on next page)

(continued from previous page)

```
↪iterator(labels=False))
...   return h
...
>>> all(random_acyclic(Integer(100), RealNumber('.2')).is_directed_acyclic() ␣
↪  # long time
...       for i in range(Integer(50)))
True
```

is_strongly_connected(*G*)

Check whether the current `DiGraph` is strongly connected.

EXAMPLES:

The circuit is obviously strongly connected:

```
sage: from sage.graphs.connectivity import is_strongly_connected
sage: g = digraphs.Circuit(5)
sage: is_strongly_connected(g)
True
sage: g.is_strongly_connected()
True
```

```
>>> from sage.all import *
>>> from sage.graphs.connectivity import is_strongly_connected
>>> g = digraphs.Circuit(Integer(5))
>>> is_strongly_connected(g)
True
>>> g.is_strongly_connected()
True
```

But a transitive triangle is not:

```
sage: g = DiGraph({0: [1, 2], 1: [2]})
sage: is_strongly_connected(g)
False
```

```
>>> from sage.all import *
>>> g = DiGraph({Integer(0): [Integer(1), Integer(2)], Integer(1):␣
↪[Integer(2)]})
>>> is_strongly_connected(g)
False
```

is_tournament()

Check whether the digraph is a tournament.

A tournament is a digraph in which each pair of distinct vertices is connected by a single arc.

EXAMPLES:

```
sage: g = digraphs.RandomTournament(6)
sage: g.is_tournament()
True
sage: u,v = next(g.edge_iterator(labels=False))
sage: g.add_edge(v, u)
```

(continues on next page)

(continued from previous page)

```
sage: g.is_tournament()
False
sage: g.add_edges([(u, v), (v, u)])
sage: g.is_tournament()
False
```

```
>>> from sage.all import *
>>> g = digraphs.RandomTournament(Integer(6))
>>> g.is_tournament()
True
>>> u,v = next(g.edge_iterator(labels=False))
>>> g.add_edge(v, u)
>>> g.is_tournament()
False
>>> g.add_edges([(u, v), (v, u)])
>>> g.is_tournament()
False
```

See also

- Wikipedia article Tournament_(graph_theory)
- *RandomTournament()*
- *TransitiveTournament()*

is_transitive(*g*, *certificate=False*)

Check whether the digraph is transitive.

A digraph is transitive if for any pair of vertices $u, v \in G$ linked by a uv-path the edge uv belongs to G.

INPUT:

- `g` – a digraph
- `certificate` – boolean (default: `False`); whether to return a certificate for negative answers
  - If `certificate = False` (default), this method returns `True` or `False` according to the graph.
  - If `certificate = True`, this method either returns `True` answers or yield a pair of vertices uv such that there exists a uv-path in G but $uv \notin G$.

EXAMPLES:

```
sage: digraphs.Circuit(4).is_transitive()
False
sage: digraphs.Circuit(4).is_transitive(certificate=True)
(0, 2)
sage: digraphs.RandomDirectedGNP(30,.2).is_transitive()
False
sage: D = digraphs.DeBruijn(5, 2)
↪  # needs sage.combinat
sage: D.is_transitive()
↪  # needs sage.combinat
```

(continues on next page)

(continued from previous page)

```
False
sage: cert = D.is_transitive(certificate=True)
↪  # needs sage.combinat
sage: D.has_edge(*cert)
↪  # needs sage.combinat
False
sage: bool(D.shortest_path(*cert))
↪  # needs sage.combinat
True
sage: digraphs.RandomDirectedGNP(20,.2).transitive_closure().is_transitive()
↪  # needs networkx
True
```

```
>>> from sage.all import *
>>> digraphs.Circuit(Integer(4)).is_transitive()
False
>>> digraphs.Circuit(Integer(4)).is_transitive(certificate=True)
(0, 2)
>>> digraphs.RandomDirectedGNP(Integer(30),RealNumber('.2')).is_transitive()
False
>>> D = digraphs.DeBruijn(Integer(5), Integer(2))
↪                  # needs sage.combinat
>>> D.is_transitive()
↪# needs sage.combinat
False
>>> cert = D.is_transitive(certificate=True)
↪# needs sage.combinat
>>> D.has_edge(*cert)
↪# needs sage.combinat
False
>>> bool(D.shortest_path(*cert))
↪# needs sage.combinat
True
>>> digraphs.RandomDirectedGNP(Integer(20),RealNumber('.2')).transitive_
↪closure().is_transitive()     # needs networkx
True
```

layout_acyclic(*rankdir='up'*, ***options*)

Return a ranked layout so that all edges point upward.

To this end, the heights of the vertices are set according to the level set decomposition of the graph (see *level_sets()*).

This is achieved by calling `graphviz` and `dot2tex` if available (see *layout_graphviz()*), and using a spring layout with fixed vertical placement of the vertices otherwise (see *layout_acyclic_dummy()* and *layout_ranked()*).

Non acyclic graphs are partially supported by `graphviz`, which then chooses some edges to point down.

INPUT:

- `rankdir` – string (default: `'up'`); indicates which direction the edges should point toward among `'up'`, `'down'`, `'left'`, or `'right'`
- `**options` – passed down to *layout_ranked()* or *layout_graphviz()*

EXAMPLES:

```
sage: H = DiGraph({0: [1, 2], 1: [3], 2: [3], 3: [], 5: [1, 6], 6: [2, 3]})
```

```
>>> from sage.all import *
>>> H = DiGraph({Integer(0): [Integer(1), Integer(2)], Integer(1):␣
↪[Integer(3)], Integer(2): [Integer(3)], Integer(3): [], Integer(5):␣
↪[Integer(1), Integer(6)], Integer(6): [Integer(2), Integer(3)]})
```

The actual layout computed depends on whether dot2tex and graphviz are installed, so we don't test its relative values:

```
sage: H.layout_acyclic()
{0: [..., ...], 1: [..., ...], 2: [..., ...], 3: [..., ...], 5: [..., ...],␣
↪6: [..., ...]}

sage: H = DiGraph({0: [1]})
sage: pos = H.layout_acyclic(rankdir='up')
sage: pos[1][1] > pos[0][1] + .5
True
sage: pos = H.layout_acyclic(rankdir='down')
sage: pos[1][1] < pos[0][1] - .5
True
sage: pos = H.layout_acyclic(rankdir='right')
sage: pos[1][0] > pos[0][0] + .5
True
sage: pos = H.layout_acyclic(rankdir='left')
sage: pos[1][0] < pos[0][0] - .5
True
```

```
>>> from sage.all import *
>>> H.layout_acyclic()
{0: [..., ...], 1: [..., ...], 2: [..., ...], 3: [..., ...], 5: [..., ...],␣
↪6: [..., ...]}

>>> H = DiGraph({Integer(0): [Integer(1)]})
>>> pos = H.layout_acyclic(rankdir='up')
>>> pos[Integer(1)][Integer(1)] > pos[Integer(0)][Integer(1)] + RealNumber('.5
↪')
True
>>> pos = H.layout_acyclic(rankdir='down')
>>> pos[Integer(1)][Integer(1)] < pos[Integer(0)][Integer(1)] - RealNumber('.5
↪')
True
>>> pos = H.layout_acyclic(rankdir='right')
>>> pos[Integer(1)][Integer(0)] > pos[Integer(0)][Integer(0)] + RealNumber('.5
↪')
True
>>> pos = H.layout_acyclic(rankdir='left')
>>> pos[Integer(1)][Integer(0)] < pos[Integer(0)][Integer(0)] - RealNumber('.5
↪')
True
```

layout_acyclic_dummy(*heights=None*, *rankdir='up'*, ***options*)

Return a ranked layout so that all edges point upward.

To this end, the heights of the vertices are set according to the level set decomposition of the graph (see *level_sets()*). This is achieved by a spring layout with fixed vertical placement of the vertices otherwise (see *layout_acyclic_dummy()* and *layout_ranked()*).

INPUT:

- `rankdir` – string (default: `'up'`); indicates which direction the edges should point toward among `'up'`, `'down'`, `'left'`, or `'right'`
- `**options` – passed down to *layout_ranked()*

EXAMPLES:

```
sage: H = DiGraph({0: [1, 2], 1: [3], 2: [3], 3: [], 5: [1, 6], 6: [2, 3]})
sage: H.layout_acyclic_dummy()
{0: [1.0..., 0], 1: [1.0..., 1], 2: [1.5..., 2], 3: [1.5..., 3], 5: [2.0...,␣
↪0], 6: [2.0..., 1]}

sage: H = DiGraph({0: [1]})
sage: H.layout_acyclic_dummy(rankdir='up')
{0: [0.5..., 0], 1: [0.5..., 1]}
sage: H.layout_acyclic_dummy(rankdir='down')
{0: [0.5..., 1], 1: [0.5..., 0]}
sage: H.layout_acyclic_dummy(rankdir='left')
{0: [1, 0.5...], 1: [0, 0.5...]}
sage: H.layout_acyclic_dummy(rankdir='right')
{0: [0, 0.5...], 1: [1, 0.5...]}
sage: H = DiGraph({0: [1, 2], 1: [3], 2: [3], 3: [1], 5: [1, 6], 6: [2, 3]})
sage: H.layout_acyclic_dummy()
Traceback (most recent call last):
...
ValueError: `self` should be an acyclic graph
```

```
>>> from sage.all import *
>>> H = DiGraph({Integer(0): [Integer(1), Integer(2)], Integer(1):␣
↪[Integer(3)], Integer(2): [Integer(3)], Integer(3): [], Integer(5):␣
↪[Integer(1), Integer(6)], Integer(6): [Integer(2), Integer(3)]})
>>> H.layout_acyclic_dummy()
{0: [1.0..., 0], 1: [1.0..., 1], 2: [1.5..., 2], 3: [1.5..., 3], 5: [2.0...,␣
↪0], 6: [2.0..., 1]}

>>> H = DiGraph({Integer(0): [Integer(1)]})
>>> H.layout_acyclic_dummy(rankdir='up')
{0: [0.5..., 0], 1: [0.5..., 1]}
>>> H.layout_acyclic_dummy(rankdir='down')
{0: [0.5..., 1], 1: [0.5..., 0]}
>>> H.layout_acyclic_dummy(rankdir='left')
{0: [1, 0.5...], 1: [0, 0.5...]}
>>> H.layout_acyclic_dummy(rankdir='right')
{0: [0, 0.5...], 1: [1, 0.5...]}
>>> H = DiGraph({Integer(0): [Integer(1), Integer(2)], Integer(1):␣
↪[Integer(3)], Integer(2): [Integer(3)], Integer(3): [Integer(1)],␣
↪Integer(5): [Integer(1), Integer(6)], Integer(6): [Integer(2), Integer(3)]})
```

(continues on next page)

(continued from previous page)

```
>>> H.layout_acyclic_dummy()
Traceback (most recent call last):
...
ValueError: `self` should be an acyclic graph
```

level_sets()

Return the level set decomposition of the digraph.

OUTPUT: list of non empty lists of vertices of this graph

The level set decomposition of the digraph is a list l such that the level $l[i]$ contains all the vertices having all their predecessors in the levels $l[j]$ for $j < i$, and at least one in level $l[i-1]$ (unless $i = 0$).

The level decomposition contains exactly the vertices not occurring in any cycle of the graph. In particular, the graph is acyclic if and only if the decomposition forms a set partition of its vertices, and we recover the usual level set decomposition of the corresponding poset.

EXAMPLES:

```
sage: H = DiGraph({0: [1, 2], 1: [3], 2: [3], 3: [], 5: [1, 6], 6: [2, 3]})
sage: H.level_sets()
[[0, 5], [1, 6], [2], [3]]

sage: H = DiGraph({0: [1, 2], 1: [3], 2: [3], 3: [1], 5: [1, 6], 6: [2, 3]})
sage: H.level_sets()
[[0, 5], [6], [2]]
```

```
>>> from sage.all import *
>>> H = DiGraph({Integer(0): [Integer(1), Integer(2)], Integer(1):␣
↪[Integer(3)], Integer(2): [Integer(3)], Integer(3): [], Integer(5):␣
↪[Integer(1), Integer(6)], Integer(6): [Integer(2), Integer(3)]})
>>> H.level_sets()
[[0, 5], [1, 6], [2], [3]]

>>> H = DiGraph({Integer(0): [Integer(1), Integer(2)], Integer(1):␣
↪[Integer(3)], Integer(2): [Integer(3)], Integer(3): [Integer(1)],␣
↪Integer(5): [Integer(1), Integer(6)], Integer(6): [Integer(2), Integer(3)]})
>>> H.level_sets()
[[0, 5], [6], [2]]
```

This routine is mostly used for Hasse diagrams of posets:

```
sage: from sage.combinat.posets.hasse_diagram import HasseDiagram
sage: H = HasseDiagram({0: [1, 2], 1: [3], 2: [3], 3: []})
sage: [len(x) for x in H.level_sets()]
[1, 2, 1]
```

```
>>> from sage.all import *
>>> from sage.combinat.posets.hasse_diagram import HasseDiagram
>>> H = HasseDiagram({Integer(0): [Integer(1), Integer(2)], Integer(1):␣
↪[Integer(3)], Integer(2): [Integer(3)], Integer(3): []})
>>> [len(x) for x in H.level_sets()]
[1, 2, 1]
```

```
sage: from sage.combinat.posets.hasse_diagram import HasseDiagram
sage: H = HasseDiagram({0: [1, 2], 1: [3], 2: [4], 3: [4]})
sage: [len(x) for x in H.level_sets()]
[1, 2, 1, 1]
```

```
>>> from sage.all import *
>>> from sage.combinat.posets.hasse_diagram import HasseDiagram
>>> H = HasseDiagram({Integer(0): [Integer(1), Integer(2)], Integer(1):␣
↪[Integer(3)], Integer(2): [Integer(4)], Integer(3): [Integer(4)]})
>>> [len(x) for x in H.level_sets()]
[1, 2, 1, 1]
```

Complexity: $O(n+m)$ in time and $O(n)$ in memory (besides the storage of the graph itself), where n and m are respectively the number of vertices and edges (assuming that appending to a list is constant time, which it is not quite).

longest_dag_path(*source=None*, *target=None*, *by_weight=False*, *weight_function=None*, *check_weight=True*)

Return the longest path in this DAG, by edge count or total weight.

This method uses dynamic programming over a topological ordering of the vertices and runs in linear time $O(V + E)$.

Warning

This method raises a `ValueError` if the digraph contains a directed cycle. For general digraphs the longest-path problem is NP-hard.

INPUT:

- `source` – (default: `None`) a vertex; if given, only paths *starting* at `source` are considered
- `target` – (default: `None`) a vertex; if given, only paths *ending* at `target` are considered
- `by_weight` – boolean (default: `False`); if `True`, the edges in the path are weighted by the function `weight_function`
- `weight_function` – function (default: `None`); a function that takes as input an edge `(u, v, label)` and outputs its weight; if `None`, the edge label is used directly as a numeric weight (see *GenericGraph._get_weight_function*)
- `check_weight` – boolean (default: `True`); if `True`, the `weight_function` is applied to all edges and an exception is raised if a non-numeric value is found

EXAMPLES:

```
sage: D = DiGraph([(0, 1), (1, 2), (2, 3)])
sage: D.longest_dag_path()
(3, [0, 1, 2, 3])
```

```
>>> from sage.all import *
>>> D = DiGraph([(Integer(0), Integer(1)), (Integer(1), Integer(2)),␣
↪(Integer(2), Integer(3))])
>>> D.longest_dag_path()
(3, [0, 1, 2, 3])
```

A diamond DAG has two equal-length paths:

```
sage: D = DiGraph([(0, 1), (0, 2), (1, 3), (2, 3)])
sage: length, path = D.longest_dag_path()
sage: length
2
sage: path[0] == 0 and path[-1] == 3
True
```

```
>>> from sage.all import *
>>> D = DiGraph([(Integer(0), Integer(1)), (Integer(0), Integer(2)),␣
↪(Integer(1), Integer(3)), (Integer(2), Integer(3))])
>>> length, path = D.longest_dag_path()
>>> length
2
>>> path[Integer(0)] == Integer(0) and path[-Integer(1)] == Integer(3)
True
```

Restricting by source and target:

```
sage: D = DiGraph([(0, 1), (1, 2), (0, 2), (2, 3)])
sage: D.longest_dag_path(source=0, target=2)
(2, [0, 1, 2])
```

```
>>> from sage.all import *
>>> D = DiGraph([(Integer(0), Integer(1)), (Integer(1), Integer(2)),␣
↪(Integer(0), Integer(2)), (Integer(2), Integer(3))])
>>> D.longest_dag_path(source=Integer(0), target=Integer(2))
(2, [0, 1, 2])
```

Using edge weights via `by_weight` — the direct edge is heavier:

```
sage: D = DiGraph()
sage: D.add_edges([(0, 1, 3), (1, 2, 1), (0, 2, 5)])
sage: D.longest_dag_path(by_weight=True)
(5, [0, 2])
```

```
>>> from sage.all import *
>>> D = DiGraph()
>>> D.add_edges([(Integer(0), Integer(1), Integer(3)), (Integer(1),␣
↪Integer(2), Integer(1)), (Integer(0), Integer(2), Integer(5))])
>>> D.longest_dag_path(by_weight=True)
(5, [0, 2])
```

Using edge weights — the two-hop path is heavier:

```
sage: D = DiGraph()
sage: D.add_edges([(0, 1, 4), (1, 2, 4), (0, 2, 6)])
sage: D.longest_dag_path(by_weight=True)
(8, [0, 1, 2])
```

```
>>> from sage.all import *
>>> D = DiGraph()
```

(continues on next page)

(continued from previous page)

```
>>> D.add_edges([(Integer(0), Integer(1), Integer(4)), (Integer(1),␣
↪Integer(2), Integer(4)), (Integer(0), Integer(2), Integer(6))])
>>> D.longest_dag_path(by_weight=True)
(8, [0, 1, 2])
```

A custom `weight_function`:

```
sage: D = DiGraph()
sage: D.add_edges([(0, 1, {'t': 4}), (1, 2, {'t': 4}),
....:              (0, 2, {'t': 6})])
sage: D.longest_dag_path(by_weight=True,
....:                    weight_function=lambda e: e[2]['t'])
(8, [0, 1, 2])
```

```
>>> from sage.all import *
>>> D = DiGraph()
>>> D.add_edges([(Integer(0), Integer(1), {'t': Integer(4)}), (Integer(1),␣
↪Integer(2), {'t': Integer(4)}),
...              (Integer(0), Integer(2), {'t': Integer(6)})])
>>> D.longest_dag_path(by_weight=True,
...                    weight_function=lambda e: e[Integer(2)]['t'])
(8, [0, 1, 2])
```

Negative edge weights are supported; with `source` and `target` given the method returns the maximum-weight path between them even if that weight is negative:

```
sage: D = DiGraph()
sage: D.add_edges([(0, 1, -1), (1, 2, -1), (0, 2, -3)])
sage: D.longest_dag_path(source=0, target=2, by_weight=True)
(-2, [0, 1, 2])
```

```
>>> from sage.all import *
>>> D = DiGraph()
>>> D.add_edges([(Integer(0), Integer(1), -Integer(1)), (Integer(1),␣
↪Integer(2), -Integer(1)), (Integer(0), Integer(2), -Integer(3))])
>>> D.longest_dag_path(source=Integer(0), target=Integer(2), by_weight=True)
(-2, [0, 1, 2])
```

Vertices connected in the *wrong* direction are not reachable:

```
sage: D = DiGraph([(0, 1), (1, 2), (0, 2), (2, 3)])
sage: D.longest_dag_path(source=2, target=0)
(0, [])
```

```
>>> from sage.all import *
>>> D = DiGraph([(Integer(0), Integer(1)), (Integer(1), Integer(2)),␣
↪(Integer(0), Integer(2)), (Integer(2), Integer(3))])
>>> D.longest_dag_path(source=Integer(2), target=Integer(0))
(0, [])
```

neighbor_in_iterator(*vertex*)

Return an iterator over the in-neighbors of `vertex`.

A vertex u is an in-neighbor of a vertex v if uv in an edge.

EXAMPLES:

```
sage: D = DiGraph({0: [1,2,3], 1: [0,2], 2: [3], 3: [4], 4: [0,5], 5: [1]})
sage: for a in D.neighbor_in_iterator(0):
....:     print(a)
1
4
```

```
>>> from sage.all import *
>>> D = DiGraph({Integer(0): [Integer(1),Integer(2),Integer(3)], Integer(1):␣
↪[Integer(0),Integer(2)], Integer(2): [Integer(3)], Integer(3): [Integer(4)],
↪ Integer(4): [Integer(0),Integer(5)], Integer(5): [Integer(1)]})
>>> for a in D.neighbor_in_iterator(Integer(0)):
...     print(a)
1
4
```

neighbor_out_iterator(*vertex*)

Return an iterator over the out-neighbors of a given vertex.

A vertex u is an out-neighbor of a vertex v if vu in an edge.

EXAMPLES:

```
sage: D = DiGraph({0: [1,2,3], 1: [0,2], 2: [3], 3: [4], 4: [0,5], 5: [1]})
sage: for a in D.neighbor_out_iterator(0):
....:     print(a)
1
2
3
```

```
>>> from sage.all import *
>>> D = DiGraph({Integer(0): [Integer(1),Integer(2),Integer(3)], Integer(1):␣
↪[Integer(0),Integer(2)], Integer(2): [Integer(3)], Integer(3): [Integer(4)],
↪ Integer(4): [Integer(0),Integer(5)], Integer(5): [Integer(1)]})
>>> for a in D.neighbor_out_iterator(Integer(0)):
...     print(a)
1
2
3
```

neighbors_in(*vertex*)

Return the list of the in-neighbors of a given vertex.

A vertex u is an in-neighbor of a vertex v if uv in an edge.

EXAMPLES:

```
sage: D = DiGraph({0: [1,2,3], 1: [0,2], 2: [3], 3: [4], 4: [0,5], 5: [1]})
sage: D.neighbors_in(0)
[1, 4]
```

```
>>> from sage.all import *
>>> D = DiGraph({Integer(0): [Integer(1),Integer(2),Integer(3)], Integer(1):␣
↪[Integer(0),Integer(2)], Integer(2): [Integer(3)], Integer(3): [Integer(4)],
↪ Integer(4): [Integer(0),Integer(5)], Integer(5): [Integer(1)]})
>>> D.neighbors_in(Integer(0))
[1, 4]
```

neighbors_out(*vertex*)

Return the list of the out-neighbors of a given vertex.

A vertex u is an out-neighbor of a vertex v if vu in an edge.

EXAMPLES:

```
sage: D = DiGraph({0: [1,2,3], 1: [0,2], 2: [3], 3: [4], 4: [0,5], 5: [1]})
sage: D.neighbors_out(0)
[1, 2, 3]
```

```
>>> from sage.all import *
>>> D = DiGraph({Integer(0): [Integer(1),Integer(2),Integer(3)], Integer(1):␣
↪[Integer(0),Integer(2)], Integer(2): [Integer(3)], Integer(3): [Integer(4)],
↪ Integer(4): [Integer(0),Integer(5)], Integer(5): [Integer(1)]})
>>> D.neighbors_out(Integer(0))
[1, 2, 3]
```

out_branchings(*source*, *spanning=True*)

Return an iterator over the out branchings rooted at given vertex in `self`.

An out-branching is a directed tree rooted at `source` whose arcs are directed from source to leaves. An out-branching is spanning if it contains all vertices of the digraph.

If no spanning out branching rooted at `source` exist, raises `ValueError` or return non spanning out branching rooted at `source`, depending on the value of `spanning`.

INPUT:

- `source` – vertex used as the source for all out branchings
- `spanning` – boolean (default: `True`); if `False` return maximum out branching from `source`. Otherwise, return spanning out branching if exists.

OUTPUT: an iterator over the out branchings rooted in the given source

See also

- *in_branchings()* – iterator over in-branchings rooted at given vertex.
- *spanning_trees()* – returns all spanning trees.
- *number_of_spanning_trees()* – counts the number of spanning trees.

ALGORITHM:

Recursively computes all out branchings.

At each step:

0. clean the graph (see below)

1. pick an edge e out of source
2. find all out branchings that do not contain e by first removing it
3. find all out branchings that do contain e by first merging the end vertices of e

Cleaning the graph implies to remove loops and replace multiedges by a single one with an appropriate label since these lead to similar steps of computation.

EXAMPLES:

A bidirectional 4-cycle:

```
sage: G = DiGraph({1:[2,3], 2:[1,4], 3:[1,4], 4:[2,3]}, format='dict_of_lists
↪')
sage: list(G.out_branchings(1))
[Digraph on 4 vertices,
 Digraph on 4 vertices,
 Digraph on 4 vertices,
 Digraph on 4 vertices]
```

```
>>> from sage.all import *
>>> G = DiGraph({Integer(1):[Integer(2),Integer(3)], Integer(2):[Integer(1),
↪Integer(4)], Integer(3):[Integer(1),Integer(4)], Integer(4):[Integer(2),
↪Integer(3)]}, format='dict_of_lists')
>>> list(G.out_branchings(Integer(1)))
[Digraph on 4 vertices,
 Digraph on 4 vertices,
 Digraph on 4 vertices,
 Digraph on 4 vertices]
```

With the Petersen graph turned into a symmetric directed graph:

```
sage: G = graphs.PetersenGraph().to_directed()
sage: len(list(G.out_branchings(0)))
2000
```

```
>>> from sage.all import *
>>> G = graphs.PetersenGraph().to_directed()
>>> len(list(G.out_branchings(Integer(0))))
2000
```

With a non connected `DiGraph` and `spanning = True`:

```
sage: G = graphs.PetersenGraph().to_directed() + graphs.PetersenGraph().to_
↪directed()
sage: G.out_branchings(0, spanning=True)
Traceback (most recent call last):
...
ValueError: no spanning out branching from vertex (0) exist
```

```
>>> from sage.all import *
>>> G = graphs.PetersenGraph().to_directed() + graphs.PetersenGraph().to_
↪directed()
>>> G.out_branchings(Integer(0), spanning=True)
```

(continues on next page)

(continued from previous page)

```
Traceback (most recent call last):
...
ValueError: no spanning out branching from vertex (0) exist
```

With a non connected `DiGraph` and `spanning = False`:

```
sage: g=DiGraph([(0,1), (0,1), (1,2), (3,4)],multiedges=True)
sage: list(g.out_branchings(0, spanning=False))
[Digraph on 3 vertices, Digraph on 3 vertices]
```

```
>>> from sage.all import *
>>> g=DiGraph([(Integer(0),Integer(1)), (Integer(0),Integer(1)), (Integer(1),
↪Integer(2)), (Integer(3),Integer(4))],multiedges=True)
>>> list(g.out_branchings(Integer(0), spanning=False))
[Digraph on 3 vertices, Digraph on 3 vertices]
```

With multiedges:

```
sage: G = DiGraph({0:[1,1,1], 1:[2,2]}, format='dict_of_lists',␣
↪multiedges=True)
sage: len(list(G.out_branchings(0)))
6
```

```
>>> from sage.all import *
>>> G = DiGraph({Integer(0):[Integer(1),Integer(1),Integer(1)],␣
↪Integer(1):[Integer(2),Integer(2)]}, format='dict_of_lists',␣
↪multiedges=True)
>>> len(list(G.out_branchings(Integer(0))))
6
```

With a DiGraph already being a spanning out branching:

```
sage: G = DiGraph({0:[1,2], 1:[3,4], 2:[5], 3:[], 4:[], 5:[]}, format='dict_
↪of_lists')
sage: next(G.out_branchings(0)) == G
True
```

```
>>> from sage.all import *
>>> G = DiGraph({Integer(0):[Integer(1),Integer(2)], Integer(1):[Integer(3),
↪Integer(4)], Integer(2):[Integer(5)], Integer(3):[], Integer(4):[],␣
↪Integer(5):[]}, format='dict_of_lists')
>>> next(G.out_branchings(Integer(0))) == G
True
```

out_degree(*vertices=None*, *labels=False*)

Same as degree, but for out degree.

EXAMPLES:

```
sage: D = DiGraph({0: [1,2,3], 1: [0,2], 2: [3], 3: [4], 4: [0,5], 5: [1]})
sage: D.out_degree(vertices=[0, 1 ,2], labels=True)
{0: 3, 1: 2, 2: 1}
```

(continues on next page)

(continued from previous page)

```
sage: D.out_degree()
[3, 2, 1, 1, 2, 1]
sage: D.out_degree(2)
1
```

```
>>> from sage.all import *
>>> D = DiGraph({Integer(0): [Integer(1),Integer(2),Integer(3)], Integer(1):␣
↪[Integer(0),Integer(2)], Integer(2): [Integer(3)], Integer(3): [Integer(4)],
↪ Integer(4): [Integer(0),Integer(5)], Integer(5): [Integer(1)]})
>>> D.out_degree(vertices=[Integer(0), Integer(1) ,Integer(2)], labels=True)
{0: 3, 1: 2, 2: 1}
>>> D.out_degree()
[3, 2, 1, 1, 2, 1]
>>> D.out_degree(Integer(2))
1
```

out_degree_iterator(*vertices=None*, *labels=False*)

Same as degree_iterator, but for out degree.

EXAMPLES:

```
sage: D = graphs.Grid2dGraph(2,4).to_directed()
sage: sorted(D.out_degree_iterator())
[2, 2, 2, 2, 3, 3, 3, 3]
sage: sorted(D.out_degree_iterator(labels=True))
[((0, 0), 2),
 ((0, 1), 3),
 ((0, 2), 3),
 ((0, 3), 2),
 ((1, 0), 2),
 ((1, 1), 3),
 ((1, 2), 3),
 ((1, 3), 2)]
```

```
>>> from sage.all import *
>>> D = graphs.Grid2dGraph(Integer(2),Integer(4)).to_directed()
>>> sorted(D.out_degree_iterator())
[2, 2, 2, 2, 3, 3, 3, 3]
>>> sorted(D.out_degree_iterator(labels=True))
[((0, 0), 2),
 ((0, 1), 3),
 ((0, 2), 3),
 ((0, 3), 2),
 ((1, 0), 2),
 ((1, 1), 3),
 ((1, 2), 3),
 ((1, 3), 2)]
```

out_degree_sequence()

Return the outdegree sequence of this digraph.

EXAMPLES:

The outdegree sequences of two digraphs:

```
sage: g = DiGraph({1: [2, 5, 6], 2: [3, 6], 3: [4, 6], 4: [6], 5: [4, 6]})
sage: g.out_degree_sequence()
[3, 2, 2, 2, 1, 0]
```

```
>>> from sage.all import *
>>> g = DiGraph({Integer(1): [Integer(2), Integer(5), Integer(6)],␣
↪Integer(2): [Integer(3), Integer(6)], Integer(3): [Integer(4), Integer(6)],␣
↪Integer(4): [Integer(6)], Integer(5): [Integer(4), Integer(6)]})
>>> g.out_degree_sequence()
[3, 2, 2, 2, 1, 0]
```

```
sage: V = [2, 3, 5, 7, 8, 9, 10, 11]
sage: E = [[], [8, 10], [11], [8, 11], [9], [], [], [2, 9, 10]]
sage: g = DiGraph(dict(zip(V, E)))
sage: g.out_degree_sequence()
[3, 2, 2, 1, 1, 0, 0, 0]
```

```
>>> from sage.all import *
>>> V = [Integer(2), Integer(3), Integer(5), Integer(7), Integer(8),␣
↪Integer(9), Integer(10), Integer(11)]
>>> E = [[], [Integer(8), Integer(10)], [Integer(11)], [Integer(8),␣
↪Integer(11)], [Integer(9)], [], [], [Integer(2), Integer(9), Integer(10)]]
>>> g = DiGraph(dict(zip(V, E)))
>>> g.out_degree_sequence()
[3, 2, 2, 1, 1, 0, 0, 0]
```

outgoing_edge_iterator(*vertices*, *labels=True*)

Return an iterator over all departing edges from vertices.

INPUT:

- `vertices` – a vertex or a list of vertices
- `labels` – boolean (default: `True`); whether to return edges as pairs of vertices, or as triples containing the labels

EXAMPLES:

```
sage: D = DiGraph({0: [1,2,3], 1: [0,2], 2: [3], 3: [4], 4: [0,5], 5: [1]})
sage: for a in D.outgoing_edge_iterator([0]):
....:     print(a)
(0, 1, None)
(0, 2, None)
(0, 3, None)
```

```
>>> from sage.all import *
>>> D = DiGraph({Integer(0): [Integer(1),Integer(2),Integer(3)], Integer(1):␣
↪[Integer(0),Integer(2)], Integer(2): [Integer(3)], Integer(3): [Integer(4)],
↪ Integer(4): [Integer(0),Integer(5)], Integer(5): [Integer(1)]})
>>> for a in D.outgoing_edge_iterator([Integer(0)]):
...     print(a)
(0, 1, None)
(0, 2, None)
(0, 3, None)
```

outgoing_edges(*vertices*, *labels=True*)

Return a list of edges departing from vertices.

INPUT:

- `vertices` – a vertex or a list of vertices
- `labels` – boolean (default: `True`); whether to return edges as pairs of vertices, or as triples containing the labels

EXAMPLES:

```
sage: D = DiGraph({0: [1,2,3], 1: [0,2], 2: [3], 3: [4], 4: [0,5], 5: [1]})
sage: D.outgoing_edges([0])
[(0, 1, None), (0, 2, None), (0, 3, None)]
```

```
>>> from sage.all import *
>>> D = DiGraph({Integer(0): [Integer(1),Integer(2),Integer(3)], Integer(1):␣
↪[Integer(0),Integer(2)], Integer(2): [Integer(3)], Integer(3): [Integer(4)],
↪ Integer(4): [Integer(0),Integer(5)], Integer(5): [Integer(1)]})
>>> D.outgoing_edges([Integer(0)])
[(0, 1, None), (0, 2, None), (0, 3, None)]
```

path_semigroup()

The partial semigroup formed by the paths of this quiver.

EXAMPLES:

```
sage: Q = DiGraph({1: {2: ['a', 'c']}, 2: {3: ['b']}})
sage: F = Q.path_semigroup(); F                                                 #␣
↪needs sage.libs.flint
Partial semigroup formed by the directed paths of Multi-digraph on 3 vertices
sage: list(F)                                                                   #␣
↪needs sage.libs.flint
[e_1, e_2, e_3, a, c, b, a*b, c*b]
```

```
>>> from sage.all import *
>>> Q = DiGraph({Integer(1): {Integer(2): ['a', 'c']}, Integer(2):
↪{Integer(3): ['b']}})
>>> F = Q.path_semigroup(); F                                                   #␣
↪needs sage.libs.flint
Partial semigroup formed by the directed paths of Multi-digraph on 3 vertices
>>> list(F)                                                                     #␣
↪needs sage.libs.flint
[e_1, e_2, e_3, a, c, b, a*b, c*b]
```

period()

Return the period of the current `DiGraph`.

The period of a directed graph is the largest integer that divides the length of every cycle in the graph. See the Wikipedia article Aperiodic_graph for more information.

EXAMPLES:

The following graph has period `2`:

```
sage: g = DiGraph({0: [1], 1: [0]})
sage: g.period()
2
```

```
>>> from sage.all import *
>>> g = DiGraph({Integer(0): [Integer(1)], Integer(1): [Integer(0)]})
>>> g.period()
2
```

The following graph has a cycle of length 2 and a cycle of length 3, so it has period 1:

```
sage: g = DiGraph({0: [1, 4], 1: [2], 2: [0], 4: [0]})
sage: g.period()
1
```

```
>>> from sage.all import *
>>> g = DiGraph({Integer(0): [Integer(1), Integer(4)], Integer(1):␣
↪[Integer(2)], Integer(2): [Integer(0)], Integer(4): [Integer(0)]})
>>> g.period()
1
```

Here is an example of computing the period of a digraph which is not strongly connected. By definition, it is the `gcd()` of the periods of its strongly connected components:

```
sage: g = DiGraph({-1: [-2], -2: [-3], -3: [-1],
....:     1: [2], 2: [1]})
sage: g.period()
1
sage: sorted([s.period() for s
....:         in g.strongly_connected_components_subgraphs()])
[2, 3]
```

```
>>> from sage.all import *
>>> g = DiGraph({-Integer(1): [-Integer(2)], -Integer(2): [-Integer(3)], -
↪Integer(3): [-Integer(1)],
...     Integer(1): [Integer(2)], Integer(2): [Integer(1)]})
>>> g.period()
1
>>> sorted([s.period() for s
...         in g.strongly_connected_components_subgraphs()])
[2, 3]
```

ALGORITHM:

See the networkX implementation of `is_aperiodic`, that is based on breadth first search.

See also

is_aperiodic()

periphery(*by_weight=False*, *algorithm=None*, *weight_function=None*, *check_weight=True*)

Return the set of vertices in the periphery of the DiGraph.

The periphery is the set of vertices whose eccentricity is equal to the diameter of the DiGraph, i.e., achieving the maximum eccentricity.

For more information and examples on how to use input variables, see *shortest_paths()* and *eccentricity()*

INPUT:

- `by_weight` – boolean (default: `False`); if `True`, edge weights are taken into account; if `False`, all edges have weight 1
- `algorithm` – string (default: `None`); see method *eccentricity()* for the list of available algorithms
- `weight_function` – function (default: `None`); a function that takes as input an edge `(u, v, l)` and outputs its weight. If not `None`, `by_weight` is automatically set to `True`. If `None` and `by_weight` is `True`, we use the edge label `l` as a weight, if `l` is not `None`, else `1` as a weight.
- `check_weight` – boolean (default: `True`); if `True`, we check that the `weight_function` outputs a number for each edge

EXAMPLES:

```
sage: G = graphs.DiamondGraph().to_directed()
sage: G.periphery()
[0, 3]
sage: P = digraphs.Path(5)
sage: P.periphery()
[1, 2, 3, 4]
sage: G = digraphs.Complete(5)
sage: G.subgraph(G.periphery()) == G
True
```

```
>>> from sage.all import *
>>> G = graphs.DiamondGraph().to_directed()
>>> G.periphery()
[0, 3]
>>> P = digraphs.Path(Integer(5))
>>> P.periphery()
[1, 2, 3, 4]
>>> G = digraphs.Complete(Integer(5))
>>> G.subgraph(G.periphery()) == G
True
```

radius(*by_weight=False*, *algorithm=None*, *weight_function=None*, *check_weight=True*)

Return the radius of the DiGraph.

The radius is defined to be the minimum eccentricity of any vertex, where the eccentricity is the maximum distance to any other vertex. For more information and examples on how to use input variables, see *shortest_paths()* and *eccentricity()*

INPUT:

- `by_weight` – boolean (default: `False`); if `True`, edge weights are taken into account; if `False`, all edges have weight 1
- `algorithm` – string (default: `None`); see method *eccentricity()* for the list of available algorithms
- `weight_function` – function (default: `None`); a function that takes as input an edge `(u, v, l)` and outputs its weight. If not `None`, `by_weight` is automatically set to `True`. If `None` and `by_weight` is `True`, we use the edge label `l`, if `l` is not `None`, else `1` as a weight.

- `check_weight` – boolean (default: `True`); if `True`, we check that the `weight_function` outputs a number for each edge

EXAMPLES:

The more symmetric a DiGraph is, the smaller (diameter - radius) is:

```
sage: G = graphs.BarbellGraph(9, 3).to_directed()
sage: G.radius()
3
sage: G.diameter()
6
```

```
>>> from sage.all import *
>>> G = graphs.BarbellGraph(Integer(9), Integer(3)).to_directed()
>>> G.radius()
3
>>> G.diameter()
6
```

```
sage: G = digraphs.Circuit(9)
sage: G.radius()
8
sage: G.diameter()
8
```

```
>>> from sage.all import *
>>> G = digraphs.Circuit(Integer(9))
>>> G.radius()
8
>>> G.diameter()
8
```

reverse(*immutable=None*)

Return a copy of digraph with edges reversed in direction.

INPUT:

- `immutable` – boolean (default: `None`); whether to return an immutable digraph or not. By default (`None`), the returned digraph has the same setting than `self`. That is, if `self` is immutable, the returned digraph also is.

EXAMPLES:

```
sage: adj = {0: [1,2,3], 1: [0,2], 2: [3], 3: [4], 4: [0,5], 5: [1]}
sage: D = DiGraph(adj)
sage: R = D.reverse(); R
Reverse of (): Digraph on 6 vertices
sage: H = R.reverse()
sage: adj == H.to_dictionary()
True
```

```
>>> from sage.all import *
>>> adj = {Integer(0): [Integer(1),Integer(2),Integer(3)], Integer(1):␣
↪[Integer(0),Integer(2)], Integer(2): [Integer(3)], Integer(3): [Integer(4)],
```

(continues on next page)

(continued from previous page)

```
↪ Integer(4): [Integer(0),Integer(5)], Integer(5): [Integer(1)]}
>>> D = DiGraph(adj)
>>> R = D.reverse(); R
Reverse of (): Digraph on 6 vertices
>>> H = R.reverse()
>>> adj == H.to_dictionary()
True
```

reverse_edge(*u*, *v=None*, *label=None*, *inplace=True*, *multiedges=None*)

Reverse the edge from u to v.

INPUT:

- `inplace` – boolean (default: `True`); if `False`, a new digraph is created and returned as output, otherwise `self` is modified.
- `multiedges` – boolean (default: `None`); how to decide what should be done in case of doubt (for instance when edge $(1, 2)$ is to be reversed in a graph while $(2, 1)$ already exists):
  - If set to `True`, input graph will be forced to allow parallel edges if necessary and edge $(1, 2)$ will appear twice in the graph.
  - If set to `False`, only one edge $(1, 2)$ will remain in the graph after $(2, 1)$ is reversed. Besides, the label of edge $(1, 2)$ will be overwritten with the label of edge $(2, 1)$.

  The default behaviour (`multiedges = None`) will raise an exception each time a subjective decision (setting `multiedges` to `True` or `False`) is necessary to perform the operation.

The following forms are all accepted:

- D.reverse_edge(1, 2)
- D.reverse_edge((1, 2))
- D.reverse_edge([1, 2])
- D.reverse_edge(1, 2, 'label')
- D.reverse_edge((1, 2, 'label'))
- D.reverse_edge([1, 2, 'label'])
- D.reverse_edge((1, 2), label='label')

EXAMPLES:

If `inplace` is `True` (default), `self` is modified:

```
sage: D = DiGraph([(0, 1 ,2)])
sage: D.reverse_edge(0, 1)
sage: D.edges(sort=True)
[(1, 0, 2)]
```

```
>>> from sage.all import *
>>> D = DiGraph([(Integer(0), Integer(1) ,Integer(2))])
>>> D.reverse_edge(Integer(0), Integer(1))
>>> D.edges(sort=True)
[(1, 0, 2)]
```

If `inplace` is `False`, `self` is not modified and a new digraph is returned:

```
sage: D = DiGraph([(0, 1, 2)])
sage: re = D.reverse_edge(0, 1, inplace=False)
sage: re.edges(sort=True)
[(1, 0, 2)]
sage: D.edges(sort=True)
[(0, 1, 2)]
```

```
>>> from sage.all import *
>>> D = DiGraph([(Integer(0), Integer(1), Integer(2))])
>>> re = D.reverse_edge(Integer(0), Integer(1), inplace=False)
>>> re.edges(sort=True)
[(1, 0, 2)]
>>> D.edges(sort=True)
[(0, 1, 2)]
```

If `multiedges` is `True`, `self` will be forced to allow parallel edges when and only when it is necessary:

```
sage: D = DiGraph([(1, 2, 'A'), (2, 1, 'A'), (2, 3, None)])
sage: D.reverse_edge(1, 2, multiedges=True)
sage: D.edges(sort=True)
[(2, 1, 'A'), (2, 1, 'A'), (2, 3, None)]
sage: D.allows_multiple_edges()
True
```

```
>>> from sage.all import *
>>> D = DiGraph([(Integer(1), Integer(2), 'A'), (Integer(2), Integer(1), 'A'),
↪ (Integer(2), Integer(3), None)])
>>> D.reverse_edge(Integer(1), Integer(2), multiedges=True)
>>> D.edges(sort=True)
[(2, 1, 'A'), (2, 1, 'A'), (2, 3, None)]
>>> D.allows_multiple_edges()
True
```

Even if `multiedges` is `True`, `self` will not be forced to allow parallel edges when it is not necessary:

```
sage: D = DiGraph( [(1, 2, 'A'), (2, 1, 'A'), (2, 3, None)] )
sage: D.reverse_edge(2, 3, multiedges=True)
sage: D.edges(sort=True)
[(1, 2, 'A'), (2, 1, 'A'), (3, 2, None)]
sage: D.allows_multiple_edges()
False
```

```
>>> from sage.all import *
>>> D = DiGraph( [(Integer(1), Integer(2), 'A'), (Integer(2), Integer(1), 'A
↪'), (Integer(2), Integer(3), None)] )
>>> D.reverse_edge(Integer(2), Integer(3), multiedges=True)
>>> D.edges(sort=True)
[(1, 2, 'A'), (2, 1, 'A'), (3, 2, None)]
>>> D.allows_multiple_edges()
False
```

If user specifies `multiedges = False`, `self` will not be forced to allow parallel edges and a parallel edge will get deleted:

```
sage: D = DiGraph( [(1, 2, 'A'), (2, 1, 'A'), (2, 3, None)] )
sage: D.edges(sort=True)
[(1, 2, 'A'), (2, 1, 'A'), (2, 3, None)]
sage: D.reverse_edge(1, 2, multiedges=False)
sage: D.edges(sort=True)
[(2, 1, 'A'), (2, 3, None)]
```

```
>>> from sage.all import *
>>> D = DiGraph( [(Integer(1), Integer(2), 'A'), (Integer(2), Integer(1), 'A
↪'), (Integer(2), Integer(3), None)] )
>>> D.edges(sort=True)
[(1, 2, 'A'), (2, 1, 'A'), (2, 3, None)]
>>> D.reverse_edge(Integer(1), Integer(2), multiedges=False)
>>> D.edges(sort=True)
[(2, 1, 'A'), (2, 3, None)]
```

Note that in the following graph, specifying `multiedges = False` will result in overwriting the label of $(1, 2)$ with the label of $(2, 1)$:

```
sage: D = DiGraph( [(1, 2, 'B'), (2, 1, 'A'), (2, 3, None)] )
sage: D.edges(sort=True)
[(1, 2, 'B'), (2, 1, 'A'), (2, 3, None)]
sage: D.reverse_edge(2, 1, multiedges=False)
sage: D.edges(sort=True)
[(1, 2, 'A'), (2, 3, None)]
```

```
>>> from sage.all import *
>>> D = DiGraph( [(Integer(1), Integer(2), 'B'), (Integer(2), Integer(1), 'A
↪'), (Integer(2), Integer(3), None)] )
>>> D.edges(sort=True)
[(1, 2, 'B'), (2, 1, 'A'), (2, 3, None)]
>>> D.reverse_edge(Integer(2), Integer(1), multiedges=False)
>>> D.edges(sort=True)
[(1, 2, 'A'), (2, 3, None)]
```

If input edge in digraph has weight/label, then the weight/label should be preserved in the output digraph. User does not need to specify the weight/label when calling function:

```
sage: D = DiGraph([[0, 1, 2], [1, 2, 1]], weighted=True)
sage: D.reverse_edge(0, 1)
sage: D.edges(sort=True)
[(1, 0, 2), (1, 2, 1)]
sage: re = D.reverse_edge([1, 2], inplace=False)
sage: re.edges(sort=True)
[(1, 0, 2), (2, 1, 1)]
```

```
>>> from sage.all import *
>>> D = DiGraph([[Integer(0), Integer(1), Integer(2)], [Integer(1),␣
↪Integer(2), Integer(1)]], weighted=True)
>>> D.reverse_edge(Integer(0), Integer(1))
>>> D.edges(sort=True)
[(1, 0, 2), (1, 2, 1)]
```

(continues on next page)

(continued from previous page)

```
>>> re = D.reverse_edge([Integer(1), Integer(2)], inplace=False)
>>> re.edges(sort=True)
[(1, 0, 2), (2, 1, 1)]
```

If `self` has multiple copies (parallel edges) of the input edge, only 1 of the parallel edges is reversed:

```
sage: D = DiGraph([(0, 1, '01'), (0, 1, '01'), (0, 1, 'cat'), (1, 2, '12')],␣
↪weighted=True, multiedges=True)
sage: re = D.reverse_edge([0, 1, '01'], inplace=False)
sage: re.edges(sort=True)
[(0, 1, '01'), (0, 1, 'cat'), (1, 0, '01'), (1, 2, '12')]
```

```
>>> from sage.all import *
>>> D = DiGraph([(Integer(0), Integer(1), '01'), (Integer(0), Integer(1), '01
↪'), (Integer(0), Integer(1), 'cat'), (Integer(1), Integer(2), '12')],␣
↪weighted=True, multiedges=True)
>>> re = D.reverse_edge([Integer(0), Integer(1), '01'], inplace=False)
>>> re.edges(sort=True)
[(0, 1, '01'), (0, 1, 'cat'), (1, 0, '01'), (1, 2, '12')]
```

If `self` has multiple copies (parallel edges) of the input edge but with distinct labels and no input label is specified, only 1 of the parallel edges is reversed (the edge that is labeled by the first label on the list returned by *edge_label()*):

```
sage: D = DiGraph([(0, 1, 'A'), (0, 1, 'B'), (0, 1, 'mouse'), (0, 1, 'cat')],␣
↪multiedges=true)
sage: D.edge_label(0, 1)
['cat', 'mouse', 'B', 'A']
sage: D.reverse_edge(0, 1)
sage: D.edges(sort=True)
[(0, 1, 'A'), (0, 1, 'B'), (0, 1, 'mouse'), (1, 0, 'cat')]
```

```
>>> from sage.all import *
>>> D = DiGraph([(Integer(0), Integer(1), 'A'), (Integer(0), Integer(1), 'B'),
↪ (Integer(0), Integer(1), 'mouse'), (Integer(0), Integer(1), 'cat')],␣
↪multiedges=true)
>>> D.edge_label(Integer(0), Integer(1))
['cat', 'mouse', 'B', 'A']
>>> D.reverse_edge(Integer(0), Integer(1))
>>> D.edges(sort=True)
[(0, 1, 'A'), (0, 1, 'B'), (0, 1, 'mouse'), (1, 0, 'cat')]
```

Finally, an exception is raised when Sage does not know how to choose between allowing multiple edges and losing some data:

```
sage: D = DiGraph([(0, 1, 'A'), (1, 0, 'B')])
sage: D.reverse_edge(0, 1)
Traceback (most recent call last):
...
ValueError: reversing the given edge is about to create two parallel
edges but input digraph doesn't allow them - User needs to specify
multiedges is True or False.
```

```
>>> from sage.all import *
>>> D = DiGraph([(Integer(0), Integer(1), 'A'), (Integer(1), Integer(0), 'B
↪')])
>>> D.reverse_edge(Integer(0), Integer(1))
Traceback (most recent call last):
...
ValueError: reversing the given edge is about to create two parallel
edges but input digraph doesn't allow them - User needs to specify
multiedges is True or False.
```

The following syntax is supported, but note that you must use the `label` keyword:

```
sage: D = DiGraph()
sage: D.add_edge((1, 2), label='label')
sage: D.edges(sort=True)
[(1, 2, 'label')]
sage: D.reverse_edge((1, 2), label='label')
sage: D.edges(sort=True)
[(2, 1, 'label')]
sage: D.add_edge((1, 2), 'label')
sage: D.edges(sort=False)
[((1, 2), 'label', None), (2, 1, 'label')]
sage: D.reverse_edge((1, 2), 'label')
sage: D.edges(sort=False)
[('label', (1, 2), None), (2, 1, 'label')]
```

```
>>> from sage.all import *
>>> D = DiGraph()
>>> D.add_edge((Integer(1), Integer(2)), label='label')
>>> D.edges(sort=True)
[(1, 2, 'label')]
>>> D.reverse_edge((Integer(1), Integer(2)), label='label')
>>> D.edges(sort=True)
[(2, 1, 'label')]
>>> D.add_edge((Integer(1), Integer(2)), 'label')
>>> D.edges(sort=False)
[((1, 2), 'label', None), (2, 1, 'label')]
>>> D.reverse_edge((Integer(1), Integer(2)), 'label')
>>> D.edges(sort=False)
[('label', (1, 2), None), (2, 1, 'label')]
```

reverse_edges(*edges*, *inplace=True*, *multiedges=None*)

Reverse a list of edges.

INPUT:

- `edges` – list of edges in the DiGraph
- `inplace` – boolean (default: `True`); if `False`, a new digraph is created and returned as output, otherwise `self` is modified
- `multiedges` – boolean (default: `None`); if `True`, input graph will be forced to allow parallel edges when necessary (for more information see the documentation of *reverse_edge()*)

See also

reverse_edge() – reverses a single edge

EXAMPLES:

If `inplace` is `True` (default), `self` is modified:

```
sage: D = DiGraph({ 0: [1, 1, 3], 2: [3, 3], 4: [1, 5]}, multiedges=true)
sage: D.reverse_edges([[0, 1], [0, 3]])
sage: D.reverse_edges([(2, 3), (4, 5)])
sage: D.edges(sort=True)
[(0, 1, None), (1, 0, None), (2, 3, None), (3, 0, None),
 (3, 2, None), (4, 1, None), (5, 4, None)]
```

```
>>> from sage.all import *
>>> D = DiGraph({ Integer(0): [Integer(1), Integer(1), Integer(3)],␣
↪Integer(2): [Integer(3), Integer(3)], Integer(4): [Integer(1), Integer(5)]},
↪ multiedges=true)
>>> D.reverse_edges([[Integer(0), Integer(1)], [Integer(0), Integer(3)]])
>>> D.reverse_edges([(Integer(2), Integer(3)), (Integer(4), Integer(5))])
>>> D.edges(sort=True)
[(0, 1, None), (1, 0, None), (2, 3, None), (3, 0, None),
 (3, 2, None), (4, 1, None), (5, 4, None)]
```

If `inplace` is `False`, `self` is not modified and a new digraph is returned:

```
sage: D = DiGraph([(0, 1, 'A'), (1, 0, 'B'), (1, 2, 'C')])
sage: re = D.reverse_edges([(0, 1), (1, 2)],
....:                      inplace=False,
....:                      multiedges=True)
sage: re.edges(sort=True)
[(1, 0, 'A'), (1, 0, 'B'), (2, 1, 'C')]
sage: D.edges(sort=True)
[(0, 1, 'A'), (1, 0, 'B'), (1, 2, 'C')]
sage: D.allows_multiple_edges()
False
sage: re.allows_multiple_edges()
True
```

```
>>> from sage.all import *
>>> D = DiGraph([(Integer(0), Integer(1), 'A'), (Integer(1), Integer(0), 'B'),
↪ (Integer(1), Integer(2), 'C')])
>>> re = D.reverse_edges([(Integer(0), Integer(1)), (Integer(1), Integer(2))],
...                      inplace=False,
...                      multiedges=True)
>>> re.edges(sort=True)
[(1, 0, 'A'), (1, 0, 'B'), (2, 1, 'C')]
>>> D.edges(sort=True)
[(0, 1, 'A'), (1, 0, 'B'), (1, 2, 'C')]
>>> D.allows_multiple_edges()
False
```

(continues on next page)

(continued from previous page)

```
>>> re.allows_multiple_edges()
True
```

If `multiedges` is `True`, `self` will be forced to allow parallel edges when and only when it is necessary:

```
sage: D = DiGraph([(1, 2, 'A'), (2, 1, 'A'), (2, 3, None)])
sage: D.reverse_edges([(1, 2), (2, 3)], multiedges=True)
sage: D.edges(sort=True)
[(2, 1, 'A'), (2, 1, 'A'), (3, 2, None)]
sage: D.allows_multiple_edges()
True
```

```
>>> from sage.all import *
>>> D = DiGraph([(Integer(1), Integer(2), 'A'), (Integer(2), Integer(1), 'A'),
↪ (Integer(2), Integer(3), None)])
>>> D.reverse_edges([(Integer(1), Integer(2)), (Integer(2), Integer(3))],␣
↪multiedges=True)
>>> D.edges(sort=True)
[(2, 1, 'A'), (2, 1, 'A'), (3, 2, None)]
>>> D.allows_multiple_edges()
True
```

Even if `multiedges` is `True`, `self` will not be forced to allow parallel edges when it is not necessary:

```
sage: D = DiGraph([(1, 2, 'A'), (2, 1, 'A'), (2, 3, None)])
sage: D.reverse_edges([(2, 3)], multiedges=True)
sage: D.edges(sort=True)
[(1, 2, 'A'), (2, 1, 'A'), (3, 2, None)]
sage: D.allows_multiple_edges()
False
```

```
>>> from sage.all import *
>>> D = DiGraph([(Integer(1), Integer(2), 'A'), (Integer(2), Integer(1), 'A'),
↪ (Integer(2), Integer(3), None)])
>>> D.reverse_edges([(Integer(2), Integer(3))], multiedges=True)
>>> D.edges(sort=True)
[(1, 2, 'A'), (2, 1, 'A'), (3, 2, None)]
>>> D.allows_multiple_edges()
False
```

If `multiedges` is `False`, `self` will not be forced to allow parallel edges and an edge will get deleted:

```
sage: D = DiGraph([(1, 2), (2, 1)])
sage: D.edges(sort=True)
[(1, 2, None), (2, 1, None)]
sage: D.reverse_edges([(1, 2)], multiedges=False)
sage: D.edges(sort=True)
[(2, 1, None)]
```

```
>>> from sage.all import *
>>> D = DiGraph([(Integer(1), Integer(2)), (Integer(2), Integer(1))])
>>> D.edges(sort=True)
```

(continues on next page)

(continued from previous page)

```
[(1, 2, None), (2, 1, None)]
>>> D.reverse_edges([(Integer(1), Integer(2))], multiedges=False)
>>> D.edges(sort=True)
[(2, 1, None)]
```

If input edge in digraph has weight/label, then the weight/label should be preserved in the output digraph. User does not need to specify the weight/label when calling function:

```
sage: D = DiGraph([(0, 1, '01'), (1, 2, 1), (2, 3, '23')], weighted=True)
sage: D.reverse_edges([(0, 1, '01'), (1, 2), (2, 3)])
sage: D.edges(sort=True)
[(1, 0, '01'), (2, 1, 1), (3, 2, '23')]
```

```
>>> from sage.all import *
>>> D = DiGraph([(Integer(0), Integer(1), '01'), (Integer(1), Integer(2),␣
↪Integer(1)), (Integer(2), Integer(3), '23')], weighted=True)
>>> D.reverse_edges([(Integer(0), Integer(1), '01'), (Integer(1), Integer(2)),
↪ (Integer(2), Integer(3))])
>>> D.edges(sort=True)
[(1, 0, '01'), (2, 1, 1), (3, 2, '23')]
```

sinks()

Return a list of sinks of the digraph.

OUTPUT: list of the vertices of the digraph that have no edges beginning at them

EXAMPLES:

```
sage: G = DiGraph({1: {3: ['a']}, 2: {3: ['b']}})
sage: G.sinks()
[3]
sage: T = DiGraph({1: {}})
sage: T.sinks()
[1]
```

```
>>> from sage.all import *
>>> G = DiGraph({Integer(1): {Integer(3): ['a']}, Integer(2): {Integer(3): ['b
↪']}})
>>> G.sinks()
[3]
>>> T = DiGraph({Integer(1): {}})
>>> T.sinks()
[1]
```

sources()

Return a list of sources of the digraph.

OUTPUT: list of the vertices of the digraph that have no edges going into them

EXAMPLES:

```
sage: G = DiGraph({1: {3: ['a']}, 2: {3: ['b']}})
sage: G.sources()
[1, 2]
```

(continues on next page)

(continued from previous page)

```
sage: T = DiGraph({1: {}})
sage: T.sources()
[1]
```

```
>>> from sage.all import *
>>> G = DiGraph({Integer(1): {Integer(3): ['a']}, Integer(2): {Integer(3): ['b
↪']}})
>>> G.sources()
[1, 2]
>>> T = DiGraph({Integer(1): {}})
>>> T.sources()
[1]
```

strong_articulation_points(*G*)

Return the strong articulation points of this digraph.

A vertex is a strong articulation point if its deletion increases the number of strongly connected components. This method implements the algorithm described in [ILS2012]. The time complexity is dominated by the time complexity of the immediate dominators finding algorithm.

OUTPUT: the list of strong articulation points

EXAMPLES:

Two cliques sharing a vertex:

```
sage: from sage.graphs.connectivity import strong_articulation_points
sage: D = digraphs.Complete(4)
sage: D.add_clique([3, 4, 5, 6])
sage: strong_articulation_points(D)
[3]
sage: D.strong_articulation_points()
[3]
```

```
>>> from sage.all import *
>>> from sage.graphs.connectivity import strong_articulation_points
>>> D = digraphs.Complete(Integer(4))
>>> D.add_clique([Integer(3), Integer(4), Integer(5), Integer(6)])
>>> strong_articulation_points(D)
[3]
>>> D.strong_articulation_points()
[3]
```

Two cliques connected by some arcs:

```
sage: D = digraphs.Complete(4) * 2
sage: D.add_edges([(0, 4), (7, 3)])
sage: sorted(strong_articulation_points(D))
[0, 3, 4, 7]
sage: D.add_edge(1, 5)
sage: sorted(strong_articulation_points(D))
[3, 7]
sage: D.add_edge(6, 2)
```

(continues on next page)

(continued from previous page)

```
sage: strong_articulation_points(D)
[]
```

```
>>> from sage.all import *
>>> D = digraphs.Complete(Integer(4)) * Integer(2)
>>> D.add_edges([(Integer(0), Integer(4)), (Integer(7), Integer(3))])
>>> sorted(strong_articulation_points(D))
[0, 3, 4, 7]
>>> D.add_edge(Integer(1), Integer(5))
>>> sorted(strong_articulation_points(D))
[3, 7]
>>> D.add_edge(Integer(6), Integer(2))
>>> strong_articulation_points(D)
[]
```

See also

- *strongly_connected_components()*
- *dominator_tree()*

strongly_connected_component_containing_vertex(*G*, *v*)

Return the strongly connected component containing a given vertex.

INPUT:

- `G` – the input DiGraph
- `v` – a vertex

EXAMPLES:

In the symmetric digraph of a graph, the strongly connected components are the connected components:

```
sage: from sage.graphs.connectivity import strongly_connected_component_
↪containing_vertex
sage: g = graphs.PetersenGraph()
sage: d = DiGraph(g)
sage: strongly_connected_component_containing_vertex(d, 0)
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
sage: d.strongly_connected_component_containing_vertex(0)
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

```
>>> from sage.all import *
>>> from sage.graphs.connectivity import strongly_connected_component_
↪containing_vertex
>>> g = graphs.PetersenGraph()
>>> d = DiGraph(g)
>>> strongly_connected_component_containing_vertex(d, Integer(0))
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
>>> d.strongly_connected_component_containing_vertex(Integer(0))
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

```
sage: g = DiGraph([(0, 1), (1, 0), (1, 2), (2, 3), (3, 2)])
sage: strongly_connected_component_containing_vertex(g, 0)
[0, 1]
```

```
>>> from sage.all import *
>>> g = DiGraph([(Integer(0), Integer(1)), (Integer(1), Integer(0)),␣
↪(Integer(1), Integer(2)), (Integer(2), Integer(3)), (Integer(3),␣
↪Integer(2))])
>>> strongly_connected_component_containing_vertex(g, Integer(0))
[0, 1]
```

strongly_connected_components(*G*)

alias of *tarjan_strongly_connected_components()*.

strongly_connected_components_digraph(*G*, *keep_labels=False*)

Return the digraph of the strongly connected components.

The digraph of the strongly connected components of a graph G has a vertex per strongly connected component included in G. There is an edge from a component C_1 to a component C_2 if there is an edge in G from a vertex $u_1 \in C_1$ to a vertex $u_2 \in C_2$.

INPUT:

- `G` – the input DiGraph
- `keep_labels` – boolean (default: `False`); when `keep_labels=True`, the resulting digraph has an edge from a component C_i to a component C_j for each edge in G from a vertex $u_i \in C_i$ to a vertex $u_j \in C_j$. Hence the resulting digraph may have loops and multiple edges. However, edges in the result with same source, target, and label are not duplicated (see examples below). When `keep_labels=False`, the return digraph is simple, so without loops nor multiple edges, and edges are unlabelled.

EXAMPLES:

Such a digraph is always acyclic:

```
sage: from sage.graphs.connectivity import strongly_connected_components_
↪digraph
sage: g = digraphs.RandomDirectedGNP(15, .1)
sage: scc_digraph = strongly_connected_components_digraph(g)
sage: scc_digraph.is_directed_acyclic()
True
sage: scc_digraph = g.strongly_connected_components_digraph()
sage: scc_digraph.is_directed_acyclic()
True
```

```
>>> from sage.all import *
>>> from sage.graphs.connectivity import strongly_connected_components_digraph
>>> g = digraphs.RandomDirectedGNP(Integer(15), RealNumber('.1'))
>>> scc_digraph = strongly_connected_components_digraph(g)
>>> scc_digraph.is_directed_acyclic()
True
>>> scc_digraph = g.strongly_connected_components_digraph()
>>> scc_digraph.is_directed_acyclic()
True
```

The vertices of the digraph of strongly connected components are exactly the strongly connected components:

```
sage: g = digraphs.ButterflyGraph(2)
sage: scc_digraph = strongly_connected_components_digraph(g)
sage: g.is_directed_acyclic()
True
sage: V_scc = list(scc_digraph)
sage: all(Set(scc) in V_scc for scc in g.strongly_connected_components())
True
```

```
>>> from sage.all import *
>>> g = digraphs.ButterflyGraph(Integer(2))
>>> scc_digraph = strongly_connected_components_digraph(g)
>>> g.is_directed_acyclic()
True
>>> V_scc = list(scc_digraph)
>>> all(Set(scc) in V_scc for scc in g.strongly_connected_components())
True
```

The following digraph has three strongly connected components, and the digraph of those is a *Transitive-Tournament()*:

```
sage: g = DiGraph({0: {1: "01", 2: "02", 3: "03"}, 1: {2: "12"}, 2:{1: "21",␣
↪3: "23"}})
sage: scc_digraph = strongly_connected_components_digraph(g)
sage: scc_digraph.is_isomorphic(digraphs.TransitiveTournament(3))
True
```

```
>>> from sage.all import *
>>> g = DiGraph({Integer(0): {Integer(1): "01", Integer(2): "02", Integer(3):
↪"03"}, Integer(1): {Integer(2): "12"}, Integer(2):{Integer(1): "21",␣
↪Integer(3): "23"}})
>>> scc_digraph = strongly_connected_components_digraph(g)
>>> scc_digraph.is_isomorphic(digraphs.TransitiveTournament(Integer(3)))
True
```

By default, the labels are discarded, and the result has no loops nor multiple edges. If `keep_labels` is `True`, then the labels are kept, and the result is a multi digraph, possibly with multiple edges and loops. However, edges in the result with same source, target, and label are not duplicated (see the edges from 0 to the strongly connected component $\{1, 2\}$ below):

```
sage: g = DiGraph({0: {1: "0-12", 2: "0-12", 3: "0-3"}, 1: {2: "1-2", 3: "1-3
↪"}, 2: {1: "2-1", 3: "2-3"}})
sage: g.order(), g.size()
(4, 7)
sage: scc_digraph = strongly_connected_components_digraph(g, keep_labels=True)
sage: (scc_digraph.order(), scc_digraph.size())
(3, 6)
sage: set(g.edge_labels()) == set(scc_digraph.edge_labels())
True
```

```
>>> from sage.all import *
>>> g = DiGraph({Integer(0): {Integer(1): "0-12", Integer(2): "0-12",␣
↪Integer(3): "0-3"}, Integer(1): {Integer(2): "1-2", Integer(3): "1-3"},␣
```

(continues on next page)

(continued from previous page)

```
↪Integer(2): {Integer(1): "2-1", Integer(3): "2-3"}})
>>> g.order(), g.size()
(4, 7)
>>> scc_digraph = strongly_connected_components_digraph(g, keep_labels=True)
>>> (scc_digraph.order(), scc_digraph.size())
(3, 6)
>>> set(g.edge_labels()) == set(scc_digraph.edge_labels())
True
```

strongly_connected_components_subgraphs(*G*, *immutable=None*)

Return the strongly connected components as a list of subgraphs.

INPUT:

- `G` – a directed graph
- `immutable` – boolean (default: `None`); whether to create a mutable/immutable graphs. `immutable=None` (default) means that the graph and the subgraphs of its connected components will behave the same way.

EXAMPLES:

In the symmetric digraph of a graph, the strongly connected components are the connected components:

```
sage: from sage.graphs.connectivity import strongly_connected_components_
↪subgraphs
sage: g = graphs.PetersenGraph()
sage: d = DiGraph(g)
sage: strongly_connected_components_subgraphs(d)
[Subgraph of (Petersen graph): Digraph on 10 vertices]
sage: d.strongly_connected_components_subgraphs()
[Subgraph of (Petersen graph): Digraph on 10 vertices]
```

```
>>> from sage.all import *
>>> from sage.graphs.connectivity import strongly_connected_components_
↪subgraphs
>>> g = graphs.PetersenGraph()
>>> d = DiGraph(g)
>>> strongly_connected_components_subgraphs(d)
[Subgraph of (Petersen graph): Digraph on 10 vertices]
>>> d.strongly_connected_components_subgraphs()
[Subgraph of (Petersen graph): Digraph on 10 vertices]
```

```
sage: g = DiGraph([(0, 1), (1, 0), (1, 2), (2, 3), (3, 2)])
sage: strongly_connected_components_subgraphs(g)
[Subgraph of (): Digraph on 2 vertices, Subgraph of (): Digraph on 2 vertices]
```

```
>>> from sage.all import *
>>> g = DiGraph([(Integer(0), Integer(1)), (Integer(1), Integer(0)),␣
↪(Integer(1), Integer(2)), (Integer(2), Integer(3)), (Integer(3),␣
↪Integer(2))])
>>> strongly_connected_components_subgraphs(g)
[Subgraph of (): Digraph on 2 vertices, Subgraph of (): Digraph on 2 vertices]
```

to_directed(*immutable=None*)

Since the graph is already directed, simply returns a copy of itself.

INPUT:

- `immutable` – boolean (default: `None`); whether to create a mutable/immutable graph. `immutable=None` (default) means that the digraph and the returned copy behave the same way.

EXAMPLES:

```
sage: DiGraph({0: [1, 2, 3], 4: [5, 1]}).to_directed()
Digraph on 6 vertices
```

```
>>> from sage.all import *
>>> DiGraph({Integer(0): [Integer(1), Integer(2), Integer(3)], Integer(4):␣
↪[Integer(5), Integer(1)]}).to_directed()
Digraph on 6 vertices
```

to_undirected(*data_structure=None*, *sparse=None*, *immutable=None*)

Return an undirected version of the graph.

Every directed edge becomes an edge.

INPUT:

- `data_structure` – string (default: `None`); one of `'sparse'`, `'static_sparse'`, or `'dense'`. See the documentation of *Graph* or *DiGraph*.
- `sparse` – boolean (default: `None`); `sparse=True` is an alias for `data_structure="sparse"`, and `sparse=False` is an alias for `data_structure="dense"`.
- `immutable` – boolean (default: `None`); whether to create a mutable/immutable digraph. Only used when `data_structure=None`.
  - `immutable=None` (default) means that the graph and its directed version will behave the same way.
  - `immutable=True` is a shortcut for `data_structure='static_sparse'`
  - `immutable=False` means that the created digraph is mutable. When used with an immutable graph, the data structure used is `'sparse'` unless anything else is specified.

EXAMPLES:

```
sage: D = DiGraph({0: [1, 2], 1: [0]})
sage: G = D.to_undirected()
sage: D.edges(sort=True, labels=False)
[(0, 1), (0, 2), (1, 0)]
sage: G.edges(sort=True, labels=False)
[(0, 1), (0, 2)]
```

```
>>> from sage.all import *
>>> D = DiGraph({Integer(0): [Integer(1), Integer(2)], Integer(1):␣
↪[Integer(0)]})
>>> G = D.to_undirected()
>>> D.edges(sort=True, labels=False)
[(0, 1), (0, 2), (1, 0)]
>>> G.edges(sort=True, labels=False)
[(0, 1), (0, 2)]
```

topological_sort(*implementation='default'*)

Return a topological sort of the digraph if it is acyclic.

If the digraph contains a directed cycle, a `TypeError` is raised. As topological sorts are not necessarily unique, different implementations may yield different results.

A topological sort is an ordering of the vertices of the digraph such that each vertex comes before all of its successors. That is, if u comes before v in the sort, then there may be a directed path from u to v, but there will be no directed path from v to u.

INPUT:

- `implementation` – string (default: `'default'`); either use the default Cython implementation, or the default NetworkX library (`implementation = "NetworkX"`)

See also

- *is_directed_acyclic()* – tests whether a directed graph is acyclic (can also join a certificate; a topological sort or a circuit in the graph)

EXAMPLES:

```
sage: D = DiGraph({0: [1, 2, 3], 4: [2, 5], 1: [8], 2: [7], 3: [7],
....:   5: [6, 7], 7: [8], 6: [9], 8: [10], 9: [10]})
sage: D.plot(layout='circular').show()                                    #␣
↪needs sage.plot
sage: D.topological_sort()
[4, 5, 6, 9, 0, 1, 2, 3, 7, 8, 10]
```

```
>>> from sage.all import *
>>> D = DiGraph({Integer(0): [Integer(1), Integer(2), Integer(3)],␣
↪Integer(4): [Integer(2), Integer(5)], Integer(1): [Integer(8)], Integer(2):␣
↪[Integer(7)], Integer(3): [Integer(7)],
...   Integer(5): [Integer(6), Integer(7)], Integer(7): [Integer(8)],␣
↪Integer(6): [Integer(9)], Integer(8): [Integer(10)], Integer(9):␣
↪[Integer(10)]})
>>> D.plot(layout='circular').show()                                      #␣
↪needs sage.plot
>>> D.topological_sort()
[4, 5, 6, 9, 0, 1, 2, 3, 7, 8, 10]
```

```
sage: D.add_edge(9, 7)
sage: D.topological_sort()
[4, 5, 6, 9, 0, 1, 2, 3, 7, 8, 10]
```

```
>>> from sage.all import *
>>> D.add_edge(Integer(9), Integer(7))
>>> D.topological_sort()
[4, 5, 6, 9, 0, 1, 2, 3, 7, 8, 10]
```

Using the NetworkX implementation

```
sage: s = list(D.topological_sort(implementation="NetworkX")); s  # random  #␣
↪needs networkx
[0, 4, 1, 3, 2, 5, 6, 9, 7, 8, 10]
sage: all(s.index(u) < s.index(v)                                          #␣
↪needs networkx
....:     for u, v in D.edges(sort=False, labels=False))
True
```

```
>>> from sage.all import *
>>> s = list(D.topological_sort(implementation="NetworkX")); s  # random  #␣
↪needs networkx
[0, 4, 1, 3, 2, 5, 6, 9, 7, 8, 10]
>>> all(s.index(u) < s.index(v)                                            #␣
↪needs networkx
...     for u, v in D.edges(sort=False, labels=False))
True
```

```
sage: D.add_edge(7, 4)
sage: D.topological_sort()
Traceback (most recent call last):
...
TypeError: digraph is not acyclic; there is no topological sort
```

```
>>> from sage.all import *
>>> D.add_edge(Integer(7), Integer(4))
>>> D.topological_sort()
Traceback (most recent call last):
...
TypeError: digraph is not acyclic; there is no topological sort
```

topological_sort_generator()

Return an iterator over all topological sorts of the digraph if it is acyclic.

If the digraph contains a directed cycle, a `TypeError` is raised.

A topological sort is an ordering of the vertices of the digraph such that each vertex comes before all of its successors. That is, if u comes before v in the sort, then there may be a directed path from u to v, but there will be no directed path from v to u. See also *topological_sort()*.

AUTHORS:

- Mike Hansen - original implementation
- Robert L. Miller: wrapping, documentation

REFERENCE:

- [1] Pruesse, Gara and Ruskey, Frank. Generating Linear Extensions Fast. SIAM J. Comput., Vol. 23 (1994), no. 2, pp. 373-386.

EXAMPLES:

```
sage: D = DiGraph({0: [1, 2], 1: [3], 2: [3, 4]})
sage: D.plot(layout='circular').show()                                     #␣
↪needs sage.plot
sage: list(D.topological_sort_generator())                                 #␣
```

(continues on next page)

(continued from previous page)

```
↪needs sage.modules sage.rings.finite_rings
[[0, 1, 2, 3, 4], [0, 2, 1, 3, 4], [0, 2, 1, 4, 3],
 [0, 2, 4, 1, 3], [0, 1, 2, 4, 3]]
```

```
>>> from sage.all import *
>>> D = DiGraph({Integer(0): [Integer(1), Integer(2)], Integer(1):␣
↪[Integer(3)], Integer(2): [Integer(3), Integer(4)]})
>>> D.plot(layout='circular').show()                                    #␣
↪needs sage.plot
>>> list(D.topological_sort_generator())                                #␣
↪needs sage.modules sage.rings.finite_rings
[[0, 1, 2, 3, 4], [0, 2, 1, 3, 4], [0, 2, 1, 4, 3],
 [0, 2, 4, 1, 3], [0, 1, 2, 4, 3]]
```

```
sage: for sort in D.topological_sort_generator():                       #␣
↪needs sage.modules sage.rings.finite_rings
....:     for u, v in D.edge_iterator(labels=False):
....:         if sort.index(u) > sort.index(v):
....:             print("this should never happen")
```

```
>>> from sage.all import *
>>> for sort in D.topological_sort_generator():                         #␣
↪needs sage.modules sage.rings.finite_rings
...     for u, v in D.edge_iterator(labels=False):
...         if sort.index(u) > sort.index(v):
...             print("this should never happen")
```

1.4 Bipartite graphs

This module implements bipartite graphs.

AUTHORS:

- Robert L. Miller (2008-01-20): initial version
- Ryan W. Hinton (2010-03-04): overrides for adding and deleting vertices and edges
- Enjeck M. Cleopatra (2022): fixes incorrect partite sets and adds graph creation from graph6 string

class sage.graphs.bipartite_graph.**BipartiteGraph**(*data=None*, *partition=None*, *check=True*, *hash_labels=None*, **args*, ***kwds*)

Bases: *Graph*

Bipartite graph.

INPUT:

- `data` – can be any of the following:
  1. Empty or `None` (creates an empty graph).
  2. An arbitrary graph.
  3. A reduced adjacency matrix.

A reduced adjacency matrix contains only the non-redundant portion of the full adjacency matrix for the bipartite graph. Specifically, for zero matrices of the appropriate size, for the reduced adjacency matrix `H`, the full adjacency matrix is `[[0, H'], [H, 0]]`. The columns correspond to vertices on the left, and the rows correspond to vertices on the right.

4. A file in alist format.

   The alist file format is described at http://www.inference.phy.cam.ac.uk/mackay/codes/alist.html

5. A `graph6` string (see documentation of *`graph6_string()`*).

6. From a NetworkX bipartite graph.

- `partition` – (default: `None`) a tuple defining vertices of the left and right partition of the graph. Partitions will be determined automatically if `partition` is `None`.
- `check` – boolean (default: `True`); if `True`, an invalid input partition raises an exception. In the other case offending edges simply won't be included.
- `loops` – ignored; bipartite graphs cannot have loops
- `multiedges` – boolean (default: `None`); whether to allow multiple edges
- `weighted` – boolean (default: `None`); whether graph thinks of itself as weighted or not. See `self.weighted()`
- `hash_labels` – boolean (default: `None`); whether to include edge labels during hashing. This parameter defaults to `True` if the graph is weighted. This parameter is ignored if the graph is mutable. Beware that trying to hash unhashable labels will raise an error.

Note

All remaining arguments are passed to the `Graph` constructor

EXAMPLES:

1. No inputs or `None` for the input creates an empty graph:

```
sage: B = BipartiteGraph()
sage: type(B)
<class 'sage.graphs.bipartite_graph.BipartiteGraph'>
sage: B.order()
0
sage: B == BipartiteGraph(None)
True
```

```
>>> from sage.all import *
>>> B = BipartiteGraph()
>>> type(B)
<class 'sage.graphs.bipartite_graph.BipartiteGraph'>
>>> B.order()
0
>>> B == BipartiteGraph(None)
True
```

2. From a graph: without any more information, finds a bipartition:

```
sage: B = BipartiteGraph(graphs.CycleGraph(4))
sage: B = BipartiteGraph(graphs.CycleGraph(5))
Traceback (most recent call last):
...
ValueError: input graph is not bipartite
sage: G = Graph({0: [5, 6], 1: [4, 5], 2: [4, 6], 3: [4, 5, 6]})
sage: B = BipartiteGraph(G)
sage: B == G
True
sage: B.left
{0, 1, 2, 3}
sage: B.right
{4, 5, 6}
sage: B = BipartiteGraph({0: [5, 6], 1: [4, 5], 2: [4, 6], 3: [4, 5, 6]})
sage: B == G
True
sage: B.left
{0, 1, 2, 3}
sage: B.right
{4, 5, 6}
```

```
>>> from sage.all import *
>>> B = BipartiteGraph(graphs.CycleGraph(Integer(4)))
>>> B = BipartiteGraph(graphs.CycleGraph(Integer(5)))
Traceback (most recent call last):
...
ValueError: input graph is not bipartite
>>> G = Graph({Integer(0): [Integer(5), Integer(6)], Integer(1): [Integer(4),␣
↪Integer(5)], Integer(2): [Integer(4), Integer(6)], Integer(3): [Integer(4),␣
↪Integer(5), Integer(6)]})
>>> B = BipartiteGraph(G)
>>> B == G
True
>>> B.left
{0, 1, 2, 3}
>>> B.right
{4, 5, 6}
>>> B = BipartiteGraph({Integer(0): [Integer(5), Integer(6)], Integer(1):␣
↪[Integer(4), Integer(5)], Integer(2): [Integer(4), Integer(6)], Integer(3):␣
↪[Integer(4), Integer(5), Integer(6)]})
>>> B == G
True
>>> B.left
{0, 1, 2, 3}
>>> B.right
{4, 5, 6}
```

3. If a `Graph` or `DiGraph` is used as data, you can specify a partition using `partition` argument. Note that if such graph is not bipartite, then Sage will raise an error. However, if one specifies `check=False`, the offending edges are simply deleted (along with those vertices not appearing in either list). We also lump creating one bipartite graph from another into this category:

```
sage: P = graphs.PetersenGraph()
sage: partition = [list(range(5)), list(range(5, 10))]
sage: B = BipartiteGraph(P, partition)
Traceback (most recent call last):
...
TypeError: input graph is not bipartite with respect to the given partition

sage: B = BipartiteGraph(P, partition, check=False)
sage: B.left
{0, 1, 2, 3, 4}
sage: B.show()
↪  # needs sage.plot
```

```
>>> from sage.all import *
>>> P = graphs.PetersenGraph()
>>> partition = [list(range(Integer(5))), list(range(Integer(5),␣
↪Integer(10)))]
>>> B = BipartiteGraph(P, partition)
Traceback (most recent call last):
...
TypeError: input graph is not bipartite with respect to the given partition

>>> B = BipartiteGraph(P, partition, check=False)
>>> B.left
{0, 1, 2, 3, 4}
>>> B.show()
↪# needs sage.plot
```

```
sage: G = Graph({0: [5, 6], 1: [4, 5], 2: [4, 6], 3: [4, 5, 6]})
sage: B = BipartiteGraph(G)
sage: B2 = BipartiteGraph(B)
sage: B == B2
True
sage: B3 = BipartiteGraph(G, [list(range(4)), list(range(4, 7))])
sage: B3
Bipartite graph on 7 vertices
sage: B3 == B2
True
```

```
>>> from sage.all import *
>>> G = Graph({Integer(0): [Integer(5), Integer(6)], Integer(1): [Integer(4),␣
↪Integer(5)], Integer(2): [Integer(4), Integer(6)], Integer(3): [Integer(4),␣
↪Integer(5), Integer(6)]})
>>> B = BipartiteGraph(G)
>>> B2 = BipartiteGraph(B)
>>> B == B2
True
>>> B3 = BipartiteGraph(G, [list(range(Integer(4))), list(range(Integer(4),␣
↪Integer(7)))])
>>> B3
Bipartite graph on 7 vertices
>>> B3 == B2
```

(continues on next page)

(continued from previous page)

```
True
```

```
sage: G = Graph({0: [], 1: [], 2: []})
sage: part = (list(range(2)), [2])
sage: B = BipartiteGraph(G, part)
sage: B2 = BipartiteGraph(B)
sage: B == B2
True
```

```
>>> from sage.all import *
>>> G = Graph({Integer(0): [], Integer(1): [], Integer(2): []})
>>> part = (list(range(Integer(2))), [Integer(2)])
>>> B = BipartiteGraph(G, part)
>>> B2 = BipartiteGraph(B)
>>> B == B2
True
```

```
sage: d = DiGraph(6)
sage: d.add_edge(0, 1)
sage: part=[[1, 2, 3], [0, 4, 5]]
sage: b = BipartiteGraph(d, part)
sage: b.left
{1, 2, 3}
sage: b.right
{0, 4, 5}
```

```
>>> from sage.all import *
>>> d = DiGraph(Integer(6))
>>> d.add_edge(Integer(0), Integer(1))
>>> part=[[Integer(1), Integer(2), Integer(3)], [Integer(0), Integer(4),␣
↪Integer(5)]]
>>> b = BipartiteGraph(d, part)
>>> b.left
{1, 2, 3}
>>> b.right
{0, 4, 5}
```

4. From a reduced adjacency matrix:

```
sage: M = Matrix([(1,1,1,0,0,0,0), (1,0,0,1,1,0,0),
....:             (0,1,0,1,0,1,0), (1,1,0,1,0,0,1)])
sage: M
[1 1 1 0 0 0 0]
[1 0 0 1 1 0 0]
[0 1 0 1 0 1 0]
[1 1 0 1 0 0 1]
sage: H = BipartiteGraph(M); H
Bipartite graph on 11 vertices
sage: H.edges(sort=True)
[(0, 7, None),
 (0, 8, None),
```

(continues on next page)

(continued from previous page)

```
 (0, 10, None),
 (1, 7, None),
 (1, 9, None),
 (1, 10, None),
 (2, 7, None),
 (3, 8, None),
 (3, 9, None),
 (3, 10, None),
 (4, 8, None),
 (5, 9, None),
 (6, 10, None)]
```

```
>>> from sage.all import *
>>> M = Matrix([(Integer(1),Integer(1),Integer(1),Integer(0),Integer(0),
↪Integer(0),Integer(0)), (Integer(1),Integer(0),Integer(0),Integer(1),
↪Integer(1),Integer(0),Integer(0)),
...              (Integer(0),Integer(1),Integer(0),Integer(1),Integer(0),
↪Integer(1),Integer(0)), (Integer(1),Integer(1),Integer(0),Integer(1),
↪Integer(0),Integer(0),Integer(1))])
>>> M
[1 1 1 0 0 0 0]
[1 0 0 1 1 0 0]
[0 1 0 1 0 1 0]
[1 1 0 1 0 0 1]
>>> H = BipartiteGraph(M); H
Bipartite graph on 11 vertices
>>> H.edges(sort=True)
[(0, 7, None),
 (0, 8, None),
 (0, 10, None),
 (1, 7, None),
 (1, 9, None),
 (1, 10, None),
 (2, 7, None),
 (3, 8, None),
 (3, 9, None),
 (3, 10, None),
 (4, 8, None),
 (5, 9, None),
 (6, 10, None)]
```

```
sage: M = Matrix([(1, 1, 2, 0, 0), (0, 2, 1, 1, 1), (0, 1, 2, 1, 1)])          ␣
↪  # needs sage.modules
sage: B = BipartiteGraph(M, multiedges=True, sparse=True)                       ␣
↪  # needs sage.modules
sage: B.edges(sort=True)                                                        ␣
↪  # needs sage.modules
[(0, 5, None),
 (1, 5, None),
 (1, 6, None),
 (1, 6, None),
```

(continues on next page)

(continued from previous page)

```
 (1, 7, None),
 (2, 5, None),
 (2, 5, None),
 (2, 6, None),
 (2, 7, None),
 (2, 7, None),
 (3, 6, None),
 (3, 7, None),
 (4, 6, None),
 (4, 7, None)]
```

```
>>> from sage.all import *
>>> M = Matrix([(Integer(1), Integer(1), Integer(2), Integer(0), Integer(0)),␣
↪(Integer(0), Integer(2), Integer(1), Integer(1), Integer(1)), (Integer(0),␣
↪Integer(1), Integer(2), Integer(1), Integer(1))])           # needs sage.
↪modules
>>> B = BipartiteGraph(M, multiedges=True, sparse=True)
↪# needs sage.modules
>>> B.edges(sort=True)
↪# needs sage.modules
[(0, 5, None),
 (1, 5, None),
 (1, 6, None),
 (1, 6, None),
 (1, 7, None),
 (2, 5, None),
 (2, 5, None),
 (2, 6, None),
 (2, 7, None),
 (2, 7, None),
 (3, 6, None),
 (3, 7, None),
 (4, 6, None),
 (4, 7, None)]
```

```
sage: F.<a> = GF(4)
sage: MS = MatrixSpace(F, 2, 3)
sage: M = MS.matrix([[0, 1, a + 1], [a, 1, 1]])
sage: B = BipartiteGraph(M, weighted=True, sparse=True)
sage: B.edges(sort=True)
[(0, 4, a), (1, 3, 1), (1, 4, 1), (2, 3, a + 1), (2, 4, 1)]
sage: B.weighted()
True
```

```
>>> from sage.all import *
>>> F = GF(Integer(4), names=('a',)); (a,) = F._first_ngens(1)
>>> MS = MatrixSpace(F, Integer(2), Integer(3))
>>> M = MS.matrix([[Integer(0), Integer(1), a + Integer(1)], [a, Integer(1),␣
↪Integer(1)]])
>>> B = BipartiteGraph(M, weighted=True, sparse=True)
>>> B.edges(sort=True)
```

(continues on next page)

(continued from previous page)

```
[(0, 4, a), (1, 3, 1), (1, 4, 1), (2, 3, a + 1), (2, 4, 1)]
>>> B.weighted()
True
```

5. From an alist file:

```
sage: import tempfile
sage: with tempfile.NamedTemporaryFile(mode='w+t', delete_on_close=False) as␣
↪f:
....:     _ = f.write("7 4 \n 3 4 \n 3 3 1 3 1 1 1 \n\
....:                  3 3 3 4 \n 1 2 4 \n 1 3 4 \n 1 0 0 \n\
....:                  2 3 4 \n 2 0 0 \n 3 0 0 \n 4 0 0 \n\
....:                  1 2 3 0 \n 1 4 5 0 \n 2 4 6 0 \n\
....:                  1 2 4 7 \n")
....:     f.close()
....:     B = BipartiteGraph(f.name)
sage: B.is_isomorphic(H)
True
```

```
>>> from sage.all import *
>>> import tempfile
>>> with tempfile.NamedTemporaryFile(mode='w+t', delete_on_close=False) as f:
...     _ = f.write("7 4 \n 3 4 \n 3 3 1 3 1 1 1 \n\
...                  3 3 3 4 \n 1 2 4 \n 1 3 4 \n 1 0 0 \n\
...                  2 3 4 \n 2 0 0 \n 3 0 0 \n 4 0 0 \n\
...                  1 2 3 0 \n 1 4 5 0 \n 2 4 6 0 \n\
...                  1 2 4 7 \n")
...     f.close()
...     B = BipartiteGraph(f.name)
>>> B.is_isomorphic(H)
True
```

6. From a `graph6` string:

```
sage: B = BipartiteGraph('Bo')
sage: B
Bipartite graph on 3 vertices
sage: B.left
{0}
sage: B.right
{1, 2}
```

```
>>> from sage.all import *
>>> B = BipartiteGraph('Bo')
>>> B
Bipartite graph on 3 vertices
>>> B.left
{0}
>>> B.right
{1, 2}
```

```
sage: B = BipartiteGraph('F?^T_\n', format='graph6')
sage: B.vertices(sort=True)
[0, 1, 2, 3, 4, 5, 6]
sage: B.edges(sort=True)
[(0, 5, None), (0, 6, None), (1, 4, None), (1, 5, None), (2, 4, None),
 (2, 6, None), (3, 4, None), (3, 5, None), (3, 6, None)]
sage: B.left
{0, 1, 2, 3}
sage: B.right
{4, 5, 6}
```

```
>>> from sage.all import *
>>> B = BipartiteGraph('F?^T_\n', format='graph6')
>>> B.vertices(sort=True)
[0, 1, 2, 3, 4, 5, 6]
>>> B.edges(sort=True)
[(0, 5, None), (0, 6, None), (1, 4, None), (1, 5, None), (2, 4, None),
 (2, 6, None), (3, 4, None), (3, 5, None), (3, 6, None)]
>>> B.left
{0, 1, 2, 3}
>>> B.right
{4, 5, 6}
```

::

sage: B = BipartiteGraph('Bo', partition=[[0], [1, 2]]) sage: B.left {0} sage: B.right {1, 2}

```
sage: B = BipartiteGraph('F?^T_\n', partition=[[0, 1, 2], [3, 4, 5, 6]])
Traceback (most recent call last):
...
TypeError: input graph is not bipartite with respect to the given partition

sage: B = BipartiteGraph('F?^T_\n', partition=[[0, 1, 2], [3, 4, 5, 6]],␣
↪check=False)
sage: B.left
{0, 1, 2}
sage: B.show()                                                          ␣
↪ # needs sage.plot
```

```
>>> from sage.all import *
>>> B = BipartiteGraph('F?^T_\n', partition=[[Integer(0), Integer(1),␣
↪Integer(2)], [Integer(3), Integer(4), Integer(5), Integer(6)]])
Traceback (most recent call last):
...
TypeError: input graph is not bipartite with respect to the given partition

>>> B = BipartiteGraph('F?^T_\n', partition=[[Integer(0), Integer(1),␣
↪Integer(2)], [Integer(3), Integer(4), Integer(5), Integer(6)]], check=False)
>>> B.left
{0, 1, 2}
>>> B.show()
↪# needs sage.plot
```

7. From a NetworkX bipartite graph:

```
sage: # needs networkx
sage: import networkx
sage: G = graphs.OctahedralGraph()
sage: N = networkx.make_clique_bipartite(G.networkx_graph())
sage: B = BipartiteGraph(N)
```

```
>>> from sage.all import *
>>> # needs networkx
>>> import networkx
>>> G = graphs.OctahedralGraph()
>>> N = networkx.make_clique_bipartite(G.networkx_graph())
>>> B = BipartiteGraph(N)
```

add_edge(*u*, *v=None*, *label=None*)

Add an edge from u to v.

INPUT:

- `u` – the tail of an edge
- `v` – (default: `None`) the head of an edge. If `v=None`, then attempt to understand `u` as a edge tuple
- `label` – (default: `None`) the label of the edge `(u, v)`

The following forms are all accepted:

- `G.add_edge(1, 2)`
- `G.add_edge((1, 2))`
- `G.add_edges([(1, 2)])`
- `G.add_edge(1, 2, 'label')`
- `G.add_edge((1, 2, 'label'))`
- `G.add_edges([(1, 2, 'label')])`

See *add_edge()* for more detail.

This method simply checks that the edge endpoints are in different partitions. If a new vertex is to be created, it will be added to the proper partition. If both vertices are created, the first one will be added to the left partition, the second to the right partition. If both vertices are in the same partition but different connected components, one of the components will be "flipped", i.e. each vertex will be put into whichever partition it's not currently in. This will allow for the graph to remain bipartite, without changing the edges or vertices.

add_edges(*edges*, *loops=True*)

Add edges from an iterable container.

INPUT:

- `edges` – an iterable of edges, given either as `(u, v)` or `(u, v, label)`
- `loops` – ignored

See *add_edges()* for more detail.

This method simply checks that the edge endpoints are in different partitions. If a new vertex is to be created, it will be added to the proper partition. If both vertices are created, the first one will be added to the left partition, the second to the right partition. If both vertices are in the same partition but different connected

components, one of the components will be "flipped", i.e. each vertex will be put into whichever partition it's not currently in. This will allow for the graph to remain bipartite, without changing the edges or vertices.

EXAMPLES:

```
sage: bg = BipartiteGraph()
sage: bg.add_vertices([0, 1, 2], left=[True, False, True])
sage: bg.add_edges([(0, 1), (2, 1)])
sage: bg.add_edges([[0, 2]])
Traceback (most recent call last):
...
ValueError: the specified set of edges cannot be added
while still preserving the bipartition property
sage: G = BipartiteGraph()
sage: G.add_edges([(0, 1), (3, 2), (1, 2)])
sage: G.bipartition()
({0, 2}, {1, 3})
```

```
>>> from sage.all import *
>>> bg = BipartiteGraph()
>>> bg.add_vertices([Integer(0), Integer(1), Integer(2)], left=[True, False,␣
↪True])
>>> bg.add_edges([(Integer(0), Integer(1)), (Integer(2), Integer(1))])
>>> bg.add_edges([[Integer(0), Integer(2)]])
Traceback (most recent call last):
...
ValueError: the specified set of edges cannot be added
while still preserving the bipartition property
>>> G = BipartiteGraph()
>>> G.add_edges([(Integer(0), Integer(1)), (Integer(3), Integer(2)),␣
↪(Integer(1), Integer(2))])
>>> G.bipartition()
({0, 2}, {1, 3})
```

Loops will raise an error:

```
sage: bg.add_edges([[0, 3], [3, 3]])
Traceback (most recent call last):
...
ValueError: the specified set of edges cannot be added
while still preserving the bipartition property
```

```
>>> from sage.all import *
>>> bg.add_edges([[Integer(0), Integer(3)], [Integer(3), Integer(3)]])
Traceback (most recent call last):
...
ValueError: the specified set of edges cannot be added
while still preserving the bipartition property
```

Adding edges is fine as long as there exists a valid bipartition. Otherwise an error is raised without modifyiong the graph:

```
sage: G = BipartiteGraph()
sage: G.add_edges([(0, 1), (2, 3)])
```

(continues on next page)

(continued from previous page)

```
sage: G.bipartition()
({0, 2}, {1, 3})
sage: G.add_edges([(0,2), (0,3)])
Traceback (most recent call last):
...
ValueError: the specified set of edges cannot be added
while still preserving the bipartition property
sage: G.bipartition()
({0, 2}, {1, 3})
sage: G.edges(labels=False, sort=True)
[(0, 1), (2, 3)]
```

```
>>> from sage.all import *
>>> G = BipartiteGraph()
>>> G.add_edges([(Integer(0), Integer(1)), (Integer(2), Integer(3))])
>>> G.bipartition()
({0, 2}, {1, 3})
>>> G.add_edges([(Integer(0),Integer(2)), (Integer(0),Integer(3))])
Traceback (most recent call last):
...
ValueError: the specified set of edges cannot be added
while still preserving the bipartition property
>>> G.bipartition()
({0, 2}, {1, 3})
>>> G.edges(labels=False, sort=True)
[(0, 1), (2, 3)]
```

add_vertex(*name=None*, *left=False*, *right=False*)

Create an isolated vertex. If the vertex already exists, then nothing is done.

INPUT:

- `name` – (default: `None`) name of the new vertex. If no name is specified, then the vertex will be represented by the least nonnegative integer not already representing a vertex. Name must be an immutable object and cannot be `None`.
- `left` – boolean (default: `False`); if `True`, puts the new vertex in the left partition
- `right` – boolean (default: `False`); if `True`, puts the new vertex in the right partition

Obviously, `left` and `right` are mutually exclusive.

As it is implemented now, if a graph G has a large number of vertices with numeric labels, then `G.add_vertex()` could potentially be slow, if name is `None`.

OUTPUT:

- If `name` is `None`, the new vertex name is returned. `None` otherwise.

EXAMPLES:

```
sage: G = BipartiteGraph()
sage: G.add_vertex(left=True)
0
sage: G.add_vertex(right=True)
1
```

(continues on next page)

(continued from previous page)

```
sage: G.vertices(sort=True)
[0, 1]
sage: G.left
{0}
sage: G.right
{1}
```

```
>>> from sage.all import *
>>> G = BipartiteGraph()
>>> G.add_vertex(left=True)
0
>>> G.add_vertex(right=True)
1
>>> G.vertices(sort=True)
[0, 1]
>>> G.left
{0}
>>> G.right
{1}
```

add_vertices(*vertices*, *left=False*, *right=False*)

Add vertices to the bipartite graph from an iterable container of vertices.

Vertices that already exist in the graph will not be added again.

INPUT:

- `vertices` – sequence of vertices to add
- `left` – (default: `False`) either `True` or sequence of same length as `vertices` with boolean elements
- `right` – (default: `False`) either `True` or sequence of the same length as `vertices` with boolean elements

Only one of `left` and `right` keywords should be provided. See the examples below.

EXAMPLES:

```
sage: bg = BipartiteGraph()
sage: bg.add_vertices([0, 1, 2], left=True)
sage: bg.add_vertices([3, 4, 5], left=[True, False, True])
sage: bg.add_vertices([6, 7, 8], right=[True, False, True])
sage: bg.add_vertices([9, 10, 11], right=True)
sage: bg.left
{0, 1, 2, 3, 5, 7}
sage: bg.right
{4, 6, 8, 9, 10, 11}
```

```
>>> from sage.all import *
>>> bg = BipartiteGraph()
>>> bg.add_vertices([Integer(0), Integer(1), Integer(2)], left=True)
>>> bg.add_vertices([Integer(3), Integer(4), Integer(5)], left=[True, False,␣
↪True])
>>> bg.add_vertices([Integer(6), Integer(7), Integer(8)], right=[True, False,␣
↪True])
```

(continues on next page)

(continued from previous page)

```
>>> bg.add_vertices([Integer(9), Integer(10), Integer(11)], right=True)
>>> bg.left
{0, 1, 2, 3, 5, 7}
>>> bg.right
{4, 6, 8, 9, 10, 11}
```

allow_loops(*new*, *check=True*)

Change whether loops are permitted in the (di)graph.

> **Note**
>
> This method overwrite the *allow_loops()* method to ensure that loops are forbidden in *BipartiteGraph*.

INPUT:

- new – boolean

EXAMPLES:

```
sage: B = BipartiteGraph()
sage: B.allow_loops(True)
Traceback (most recent call last):
...
ValueError: loops are not allowed in bipartite graphs
```

```
>>> from sage.all import *
>>> B = BipartiteGraph()
>>> B.allow_loops(True)
Traceback (most recent call last):
...
ValueError: loops are not allowed in bipartite graphs
```

bipartition()

Return the underlying bipartition of the bipartite graph.

EXAMPLES:

```
sage: B = BipartiteGraph(graphs.CycleGraph(4))
sage: B.bipartition()
({0, 2}, {1, 3})
```

```
>>> from sage.all import *
>>> B = BipartiteGraph(graphs.CycleGraph(Integer(4)))
>>> B.bipartition()
({0, 2}, {1, 3})
```

canonical_label(*partition=None*, *certificate=False*, *edge_labels=False*, *algorithm=None*, *return_graph=True*, *immutable=None*)

Return the canonical graph.

A canonical graph is the representative graph of an isomorphism class by some canonization function c. If G and H are graphs, then $G \cong c(G)$, and $c(G) == c(H)$ if and only if $G \cong H$.

See the Wikipedia article Graph_canonization for more information.

INPUT:

- `partition` – if given, the canonical label with respect to this set partition will be computed. The default is the unit set partition.
- `certificate` – boolean (default: `False`); when set to `True`, a dictionary mapping from the vertices of the (di)graph to its canonical label will also be returned
- `edge_labels` – boolean (default: `False`); when set to `True`, allows only permutations respecting edge labels
- `algorithm` – string (default: `None`); the algorithm to use. Currently available:
  - `'bliss'`: use the optional package bliss (http://www.tcs.tkk.fi/Software/bliss/index.html);
  - `'sage'`: always use Sage's implementation.
  - `None` (default): use bliss when available and possible

  Note

  Make sure you always compare canonical forms obtained by the same algorithm.

- `return_graph` – boolean (default: `True`); when set to `False`, returns the list of edges of the canonical graph instead of the canonical graph. Only available when `'bliss'` is explicitly set as algorithm.
- `immutable` – boolean (default: `None`); whether to create a mutable/immutable (di)graph. `immutable=None` (default) means that the (di)graph and its canonical (di)graph will behave the same way.

EXAMPLES:

```
sage: B = BipartiteGraph( [(0, 4), (0, 5), (0, 6), (0, 8), (1, 5),
....:                      (1, 7), (1, 8), (2, 6), (2, 7), (2, 8),
....:                      (3, 4), (3, 7), (3, 8), (4, 9), (5, 9),
....:                      (6, 9), (7, 9)] )
sage: C = B.canonical_label(partition=(B.left,B.right), algorithm='sage')
sage: C
Bipartite graph on 10 vertices
sage: C.left
{0, 1, 2, 3, 4}
sage: C.right
{5, 6, 7, 8, 9}
```

```
>>> from sage.all import *
>>> B = BipartiteGraph( [(Integer(0), Integer(4)), (Integer(0), Integer(5)),␣
↪(Integer(0), Integer(6)), (Integer(0), Integer(8)), (Integer(1),␣
↪Integer(5)),
...                      (Integer(1), Integer(7)), (Integer(1), Integer(8)),␣
↪(Integer(2), Integer(6)), (Integer(2), Integer(7)), (Integer(2),␣
↪Integer(8)),
...                      (Integer(3), Integer(4)), (Integer(3), Integer(7)),␣
↪(Integer(3), Integer(8)), (Integer(4), Integer(9)), (Integer(5),␣
↪Integer(9)),
...                      (Integer(6), Integer(9)), (Integer(7), Integer(9))] )
```

(continues on next page)

(continued from previous page)

```
>>> C = B.canonical_label(partition=(B.left,B.right), algorithm='sage')
>>> C
Bipartite graph on 10 vertices
>>> C.left
{0, 1, 2, 3, 4}
>>> C.right
{5, 6, 7, 8, 9}
```

```
sage: B = BipartiteGraph( [(0, 4), (0, 5), (0, 6), (0, 8), (1, 5),
....:                      (1, 7), (1, 8), (2, 6), (2, 7), (2, 8),
....:                      (3, 4), (3, 7), (3, 8), (4, 9), (5, 9),
....:                      (6, 9), (7, 9)] )
sage: C, cert = B.canonical_label(partition=(B.left, B.right),
....:                             certificate=True, algorithm='sage')
sage: C
Bipartite graph on 10 vertices
sage: C.left
{0, 1, 2, 3, 4}
sage: C.right
{5, 6, 7, 8, 9}
sage: cert == {0: 3, 1: 0, 2: 1, 3: 2, 4: 5, 5: 7, 6: 6, 7: 8, 8: 9, 9: 4}
True
```

```
>>> from sage.all import *
>>> B = BipartiteGraph( [(Integer(0), Integer(4)), (Integer(0), Integer(5)),␣
↪(Integer(0), Integer(6)), (Integer(0), Integer(8)), (Integer(1),␣
↪Integer(5)),
...                      (Integer(1), Integer(7)), (Integer(1), Integer(8)),␣
↪(Integer(2), Integer(6)), (Integer(2), Integer(7)), (Integer(2),␣
↪Integer(8)),
...                      (Integer(3), Integer(4)), (Integer(3), Integer(7)),␣
↪(Integer(3), Integer(8)), (Integer(4), Integer(9)), (Integer(5),␣
↪Integer(9)),
...                      (Integer(6), Integer(9)), (Integer(7), Integer(9))] )
>>> C, cert = B.canonical_label(partition=(B.left, B.right),
...                             certificate=True, algorithm='sage')
>>> C
Bipartite graph on 10 vertices
>>> C.left
{0, 1, 2, 3, 4}
>>> C.right
{5, 6, 7, 8, 9}
>>> cert == {Integer(0): Integer(3), Integer(1): Integer(0), Integer(2):␣
↪Integer(1), Integer(3): Integer(2), Integer(4): Integer(5), Integer(5):␣
↪Integer(7), Integer(6): Integer(6), Integer(7): Integer(8), Integer(8):␣
↪Integer(9), Integer(9): Integer(4)}
True
```

```
sage: G = Graph({0: [5, 6], 1: [4, 5], 2: [4, 6], 3: [4, 5, 6]})
sage: B = BipartiteGraph(G)
sage: C = B.canonical_label(partition=(B.left, B.right),
```

(continues on next page)

(continued from previous page)

```
....:                         edge_labels=True, algorithm='sage')
sage: C.left
{0, 1, 2, 3}
sage: C.right
{4, 5, 6}
```

```
>>> from sage.all import *
>>> G = Graph({Integer(0): [Integer(5), Integer(6)], Integer(1): [Integer(4),␣
↪Integer(5)], Integer(2): [Integer(4), Integer(6)], Integer(3): [Integer(4),␣
↪Integer(5), Integer(6)]})
>>> B = BipartiteGraph(G)
>>> C = B.canonical_label(partition=(B.left, B.right),
...                       edge_labels=True, algorithm='sage')
>>> C.left
{0, 1, 2, 3}
>>> C.right
{4, 5, 6}
```

See also

canonical_label()

complement()

Return a complement of this graph.

Given a simple *BipartiteGraph* $G = (L, R, E)$ with vertex set $L \cup R$ and edge set E, this method returns a *Graph* $H = (V, F)$, where $V = L \cup R$ and F is the set of edges of a complete graph of order $|V|$ minus the edges in E.

Warning

This method returns the complement of a bipartite graph $G = (V = L \cup R, E)$ with respect the complete graph of order $|V|$. If looking for the complement with respect the complete bipartite graph $K = (L, R, L \times R)$, use method *complement_bipartite()*.

See also

complement_bipartite()

EXAMPLES:

```
sage: B = BipartiteGraph({1: [2, 4], 3: [4, 5]})
sage: G = B.complement(); G
Graph on 5 vertices
sage: G.edges(sort=True, labels=False)
[(1, 3), (1, 5), (2, 3), (2, 4), (2, 5), (4, 5)]
sage: B.size() + G.size() == graphs.CompleteGraph(B.order()).size()
True
```

```
>>> from sage.all import *
>>> B = BipartiteGraph({Integer(1): [Integer(2), Integer(4)], Integer(3):␣
↪[Integer(4), Integer(5)]})
>>> G = B.complement(); G
Graph on 5 vertices
>>> G.edges(sort=True, labels=False)
[(1, 3), (1, 5), (2, 3), (2, 4), (2, 5), (4, 5)]
>>> B.size() + G.size() == graphs.CompleteGraph(B.order()).size()
True
```

complement_bipartite()

Return the bipartite complement of this bipartite graph.

Given a simple *BipartiteGraph* $G = (L, R, E)$ with vertex set $L \cup R$ and edge set E, this method returns a *BipartiteGraph* $H = (L \cup R, F)$, where F is the set of edges of a complete bipartite graph between vertex sets L and R minus the edges in E.

See also

complement()

EXAMPLES:

sage: B = BipartiteGraph({0: [1, 2, 3]}) sage: C = B.complement_bipartite() sage: C Bipartite graph on 4 vertices sage: C.is_bipartite() True sage: B.left == C.left and B.right == C.right True sage: C.size() == len(B.left)*len(B.right) - B.size() True sage: G = B.complement() sage: G.is_bipartite() False

delete_vertex(*vertex*, *in_order=False*)

Delete vertex, removing all incident edges.

Deleting a non-existent vertex will raise an exception.

INPUT:

- `vertex` – a vertex to delete
- `in_order` – boolean (default: `False`); if `True`, deletes the i-th vertex in the sorted list of vertices, i.e. `G.vertices(sort=True)[i]`

EXAMPLES:

```
sage: B = BipartiteGraph(graphs.CycleGraph(4))
sage: B
Bipartite cycle graph: graph on 4 vertices
sage: B.delete_vertex(0)
sage: B
Bipartite cycle graph: graph on 3 vertices
sage: B.left
{2}
sage: B.edges(sort=True)
[(1, 2, None), (2, 3, None)]
sage: B.delete_vertex(3)
sage: B.right
{1}
```

(continues on next page)

(continued from previous page)

```
sage: B.edges(sort=True)
[(1, 2, None)]
sage: B.delete_vertex(0)
Traceback (most recent call last):
...
ValueError: vertex (0) not in the graph
```

```
>>> from sage.all import *
>>> B = BipartiteGraph(graphs.CycleGraph(Integer(4)))
>>> B
Bipartite cycle graph: graph on 4 vertices
>>> B.delete_vertex(Integer(0))
>>> B
Bipartite cycle graph: graph on 3 vertices
>>> B.left
{2}
>>> B.edges(sort=True)
[(1, 2, None), (2, 3, None)]
>>> B.delete_vertex(Integer(3))
>>> B.right
{1}
>>> B.edges(sort=True)
[(1, 2, None)]
>>> B.delete_vertex(Integer(0))
Traceback (most recent call last):
...
ValueError: vertex (0) not in the graph
```

```
sage: g = Graph({'a': ['b'], 'c': ['b']})
sage: bg = BipartiteGraph(g)  # finds bipartition
sage: bg.vertices(sort=True)
['a', 'b', 'c']
sage: bg.delete_vertex('a')
sage: bg.edges(sort=True)
[('b', 'c', None)]
sage: bg.vertices(sort=True)
['b', 'c']
sage: bg2 = BipartiteGraph(g)
sage: bg2.delete_vertex(0, in_order=True)
sage: bg2 == bg
True
```

```
>>> from sage.all import *
>>> g = Graph({'a': ['b'], 'c': ['b']})
>>> bg = BipartiteGraph(g)  # finds bipartition
>>> bg.vertices(sort=True)
['a', 'b', 'c']
>>> bg.delete_vertex('a')
>>> bg.edges(sort=True)
[('b', 'c', None)]
>>> bg.vertices(sort=True)
```

(continues on next page)

(continued from previous page)

```
['b', 'c']
>>> bg2 = BipartiteGraph(g)
>>> bg2.delete_vertex(Integer(0), in_order=True)
>>> bg2 == bg
True
```

delete_vertices(*vertices*)

Remove vertices from the bipartite graph taken from an iterable sequence of vertices.

Deleting a non-existent vertex will raise an exception.

INPUT:

- `vertices` – a sequence of vertices to remove

EXAMPLES:

```
sage: B = BipartiteGraph(graphs.CycleGraph(4))
sage: B
Bipartite cycle graph: graph on 4 vertices
sage: B.delete_vertices([0, 3])
sage: B
Bipartite cycle graph: graph on 2 vertices
sage: B.left
{2}
sage: B.right
{1}
sage: B.edges(sort=True)
[(1, 2, None)]
sage: B.delete_vertices([0])
Traceback (most recent call last):
...
ValueError: vertex (0) not in the graph
```

```
>>> from sage.all import *
>>> B = BipartiteGraph(graphs.CycleGraph(Integer(4)))
>>> B
Bipartite cycle graph: graph on 4 vertices
>>> B.delete_vertices([Integer(0), Integer(3)])
>>> B
Bipartite cycle graph: graph on 2 vertices
>>> B.left
{2}
>>> B.right
{1}
>>> B.edges(sort=True)
[(1, 2, None)]
>>> B.delete_vertices([Integer(0)])
Traceback (most recent call last):
...
ValueError: vertex (0) not in the graph
```

is_bipartite(*certificate=False*)

Check whether the graph is bipartite.

This method always returns `True` as first value, plus a certificate when `certificate == True`.

INPUT:

- `certificate` – boolean (default: `False`); whether to return a certificate. If set to `True`, the certificate returned is a proper 2-coloring of the vertices.

See also

is_bipartite()

EXAMPLES:

```
sage: g = BipartiteGraph(graphs.RandomBipartite(3, 3, .5))                    #␣
↪needs numpy
sage: g.is_bipartite()                                                        #␣
↪needs numpy
True
sage: g.is_bipartite(certificate=True)  # random                              #␣
↪needs numpy
(True, {(0, 0): 0, (0, 1): 0, (0, 2): 0, (1, 0): 1, (1, 1): 1, (1, 2): 1})
```

```
>>> from sage.all import *
>>> g = BipartiteGraph(graphs.RandomBipartite(Integer(3), Integer(3),␣
↪RealNumber('.5')))                    # needs numpy
>>> g.is_bipartite()                                                          #␣
↪needs numpy
True
>>> g.is_bipartite(certificate=True)  # random                               #␣
↪needs numpy
(True, {(0, 0): 0, (0, 1): 0, (0, 2): 0, (1, 0): 1, (1, 1): 1, (1, 2): 1})
```

load_afile(*fname*, *immutable=False*)

Load into the current object the bipartite graph specified in the given file name.

INPUT:

- `fname` – string; file name in alist format
- `immutable` – boolean (default: `False`); whether to load the graph with an immutable backend

This file should follow David MacKay's alist format, see http://www.inference.phy.cam.ac.uk/mackay/codes/data.html for examples and definition of the format.

EXAMPLES:

```
sage: import tempfile
sage: with tempfile.NamedTemporaryFile(mode='w+t', delete_on_close=False) as␣
↪f:
....:     _ = f.write("7 4 \n 3 4 \n 3 3 1 3 1 1 1 \n\
....:                  3 3 3 4 \n 1 2 4 \n 1 3 4 \n\
....:                  1 0 0 \n 2 3 4 \n 2 0 0 \n 3 0 0 \n\
....:                  4 0 0 \n 1 2 3 0 \n 1 4 5 0 \n\
....:                  2 4 6 0 \n 1 2 4 7 \n")
....:     f.close()
....:     B = BipartiteGraph()
```

(continues on next page)

(continued from previous page)

```
....:     B2 = BipartiteGraph(f.name)
....:     B.load_afile(f.name)
Bipartite graph on 11 vertices
sage: B.edges(sort=True)
[(0, 7, None),
 (0, 8, None),
 (0, 10, None),
 (1, 7, None),
 (1, 9, None),
 (1, 10, None),
 (2, 7, None),
 (3, 8, None),
 (3, 9, None),
 (3, 10, None),
 (4, 8, None),
 (5, 9, None),
 (6, 10, None)]
 sage: B2 == B
 True
```

```
>>> from sage.all import *
>>> import tempfile
>>> with tempfile.NamedTemporaryFile(mode='w+t', delete_on_close=False) as f:
...     _ = f.write("7 4 \n 3 4 \n 3 3 1 3 1 1 1 \n\
...                  3 3 3 4 \n 1 2 4 \n 1 3 4 \n\
...                  1 0 0 \n 2 3 4 \n 2 0 0 \n 3 0 0 \n\
...                  4 0 0 \n 1 2 3 0 \n 1 4 5 0 \n\
...                  2 4 6 0 \n 1 2 4 7 \n")
...     f.close()
...     B = BipartiteGraph()
...     B2 = BipartiteGraph(f.name)
...     B.load_afile(f.name)
Bipartite graph on 11 vertices
>>> B.edges(sort=True)
[(0, 7, None),
 (0, 8, None),
 (0, 10, None),
 (1, 7, None),
 (1, 9, None),
 (1, 10, None),
 (2, 7, None),
 (3, 8, None),
 (3, 9, None),
 (3, 10, None),
 (4, 8, None),
 (5, 9, None),
 (6, 10, None)]
>>> B2 == B
 True
```

matching(*value_only*, *algorithm=False*, *by_weight=None*, *weight_function=False*, *check_weight=None*, *solver=True*, *verbose=None*, *integrality_tolerance=0*)

Return a maximum matching of the graph represented by the list of its edges.

Given a graph G such that each edge e has a weight w_e, a maximum matching is a subset S of the edges of G of maximum weight such that no two edges of S are incident with each other.

INPUT:

- `value_only` – boolean (default: `False`); when set to `True`, only the cardinal (or the weight) of the matching is returned
- `algorithm` – string (default: `'Hopcroft-Karp'` if `by_weight==False`, otherwise `'Edmonds'`); algorithm to use among:
  - `'Hopcroft-Karp'` selects the default bipartite graph algorithm as implemented in NetworkX
  - `'Eppstein'` selects Eppstein's algorithm as implemented in NetworkX
  - `'Edmonds'` selects Edmonds' algorithm as implemented in NetworkX
  - `'LP'` uses a Linear Program formulation of the matching problem
- `by_weight` – boolean (default: `False`); if `True`, computes a weighted matching where each edge has the weight given by `weight_function`, or its label if `weight_function` is `None`. If `False`, each edge has weight 1. Only available when `algorithm` is `'Edmonds'` or `'LP'`.
- `weight_function` – function (default: `None`); a function that takes as input an edge `(u, v, l)` and outputs its weight. If not `None`, `by_weight` is automatically set to `True`. If `None` and `by_weight` is `True`, we use the edge label `l`, if `l` is not `None`, else 1 as a weight.
- `check_weight` – boolean (default: `True`); whether to check that the `weight_function` outputs a number for each edge
- `solver` – string (default: `None`); specifies a Mixed Integer Linear Programming (MILP) solver to be used. If set to `None`, the default one is used. For more information on MILP solvers and which default solver is used, see the method `solve` of the class `MixedIntegerLinearProgram`.
- `verbose` – integer (default: 0); sets the level of verbosity. Set to 0 by default, which means quiet.
- `integrality_tolerance` – float; parameter for use with MILP solvers over an inexact base ring; see `MixedIntegerLinearProgram.get_values()`.

See also

- Wikipedia article Matching_(graph_theory)
- *matching()*

EXAMPLES:

Maximum matching in a cycle graph:

```
sage: G = BipartiteGraph(graphs.CycleGraph(10))
sage: G.matching()                                                          #␣
↪needs networkx
[(0, 1, None), (2, 3, None), (4, 5, None), (6, 7, None), (8, 9, None)]
```

```
>>> from sage.all import *
>>> G = BipartiteGraph(graphs.CycleGraph(Integer(10)))
>>> G.matching()                                                            #␣
```

(continues on next page)

(continued from previous page)

```
↪needs networkx
[(0, 1, None), (2, 3, None), (4, 5, None), (6, 7, None), (8, 9, None)]
```

The size of a maximum matching in a complete bipartite graph using Eppstein:

```
sage: G = BipartiteGraph(graphs.CompleteBipartiteGraph(4,5))
sage: G.matching(algorithm='Eppstein', value_only=True)                    #␣
↪needs networkx
4
```

```
>>> from sage.all import *
>>> G = BipartiteGraph(graphs.CompleteBipartiteGraph(Integer(4),Integer(5)))
>>> G.matching(algorithm='Eppstein', value_only=True)                      #␣
↪needs networkx
4
```

matching_polynomial(*algorithm='Godsil'*, *name=None*)

Compute the matching polynomial.

The *matching polynomial* is defined as in [God1993], where $p(G, k)$ denotes the number of k-matchings (matchings with k edges) in G :

$$\mu(x) = \sum_{k \geq 0} (-1)^k p(G, k) x^{n-2k}$$

INPUT:

- `algorithm` – string (default: `'Godsil'`); either "Godsil" or "rook"; "rook" is usually faster for larger graphs
- `name` – string (default: `None`); name of the variable in the polynomial, set to x when `name` is `None`

EXAMPLES:

```
sage: BipartiteGraph(graphs.CubeGraph(3)).matching_polynomial()            #␣
↪needs sage.libs.flint
x^8 - 12*x^6 + 42*x^4 - 44*x^2 + 9
```

```
>>> from sage.all import *
>>> BipartiteGraph(graphs.CubeGraph(Integer(3))).matching_polynomial()      ␣
↪     # needs sage.libs.flint
x^8 - 12*x^6 + 42*x^4 - 44*x^2 + 9
```

```
sage: x = polygen(QQ)
sage: g = BipartiteGraph(graphs.CompleteBipartiteGraph(16, 16))
sage: bool(factorial(16) * laguerre(16, x^2)                               #␣
↪needs sage.symbolic
....:       == g.matching_polynomial(algorithm='rook'))
True
```

```
>>> from sage.all import *
>>> x = polygen(QQ)
>>> g = BipartiteGraph(graphs.CompleteBipartiteGraph(Integer(16),␣
```

(continues on next page)

(continued from previous page)

```
↪Integer(16)))
>>> bool(factorial(Integer(16)) * laguerre(Integer(16), x**Integer(2))
↪                          # needs sage.symbolic
...       == g.matching_polynomial(algorithm='rook'))
True
```

Compute the matching polynomial of a line with 60 vertices:

```
sage: from sage.functions.orthogonal_polys import chebyshev_U
↪needs sage.symbolic
sage: g = next(graphs.trees(60))
sage: (chebyshev_U(60, x/2)
↪needs sage.symbolic
....:    == BipartiteGraph(g).matching_polynomial(algorithm='rook'))
True
```

```
>>> from sage.all import *
>>> from sage.functions.orthogonal_polys import chebyshev_U
↪needs sage.symbolic
>>> g = next(graphs.trees(Integer(60)))
>>> (chebyshev_U(Integer(60), x/Integer(2))
↪              # needs sage.symbolic
...    == BipartiteGraph(g).matching_polynomial(algorithm='rook'))
True
```

The matching polynomial of a tree is equal to its characteristic polynomial:

```
sage: g = graphs.RandomTree(20)
sage: p = g.characteristic_polynomial()
↪needs sage.modules
sage: p == BipartiteGraph(g).matching_polynomial(algorithm='rook')
↪needs sage.modules
True
```

```
>>> from sage.all import *
>>> g = graphs.RandomTree(Integer(20))
>>> p = g.characteristic_polynomial()
↪needs sage.modules
>>> p == BipartiteGraph(g).matching_polynomial(algorithm='rook')
↪needs sage.modules
True
```

perfect_matchings(*labels=False*)

Return an iterator over all perfect matchings of the bipartite graph.

ALGORITHM:

Choose a vertex v in the right set of vertices, then recurse through all edges incident to v, removing one edge at a time whenever an edge is added to a matching.

INPUT:

- `labels` – boolean (default: `False`); when `True`, the edges in each perfect matching are triples (containing the label as the third element), otherwise the edges are pairs.

See also

perfect_matchings() matching()

EXAMPLES:

```
sage: B = BipartiteGraph({0: [5, 7], 1: [4, 6, 7], 2: [4, 5, 8],
....:                     3: [4, 5, 6], 6: [9], 8: [9]})
sage: len(list(B.perfect_matchings()))
6
sage: G = Graph(B.edges(sort=False))
sage: len(list(G.perfect_matchings()))
6
```

```
>>> from sage.all import *
>>> B = BipartiteGraph({Integer(0): [Integer(5), Integer(7)], Integer(1):␣
↪[Integer(4), Integer(6), Integer(7)], Integer(2): [Integer(4), Integer(5),␣
↪Integer(8)],
...                     Integer(3): [Integer(4), Integer(5), Integer(6)],␣
↪Integer(6): [Integer(9)], Integer(8): [Integer(9)]})
>>> len(list(B.perfect_matchings()))
6
>>> G = Graph(B.edges(sort=False))
>>> len(list(G.perfect_matchings()))
6
```

The algorithm ensures that for any edge of a perfect matching, the first vertex is on the left set of vertices and the second vertex in the right set:

```
sage: B = BipartiteGraph({0: [5, 7], 1: [4, 6, 7], 2: [4, 5, 8],
....:                     3: [4, 5, 6], 6: [9], 8: [9]})
sage: m = next(B.perfect_matchings(labels=False))
sage: B.left
{0, 1, 2, 3, 9}
sage: B.right
{4, 5, 6, 7, 8}
sage: sorted(m)
[(0, 7), (1, 4), (2, 5), (3, 6), (9, 8)]
sage: all((u in B.left and v in B.right) for u, v in m)
True
```

```
>>> from sage.all import *
>>> B = BipartiteGraph({Integer(0): [Integer(5), Integer(7)], Integer(1):␣
↪[Integer(4), Integer(6), Integer(7)], Integer(2): [Integer(4), Integer(5),␣
↪Integer(8)],
...                     Integer(3): [Integer(4), Integer(5), Integer(6)],␣
↪Integer(6): [Integer(9)], Integer(8): [Integer(9)]})
>>> m = next(B.perfect_matchings(labels=False))
>>> B.left
{0, 1, 2, 3, 9}
>>> B.right
{4, 5, 6, 7, 8}
```

(continues on next page)

(continued from previous page)

```
>>> sorted(m)
[(0, 7), (1, 4), (2, 5), (3, 6), (9, 8)]
>>> all((u in B.left and v in B.right) for u, v in m)
True
```

Multiple edges are taken into account:

```
sage: B = BipartiteGraph({0: [5, 7], 1: [4, 6, 7], 2: [4, 5, 8],
....:                     3: [4, 5, 6], 6: [9], 8: [9]})
sage: B.allow_multiple_edges(True)
sage: B.add_edge(0, 7)
sage: len(list(B.perfect_matchings()))
10
```

```
>>> from sage.all import *
>>> B = BipartiteGraph({Integer(0): [Integer(5), Integer(7)], Integer(1):␣
↪[Integer(4), Integer(6), Integer(7)], Integer(2): [Integer(4), Integer(5),␣
↪Integer(8)],
...                     Integer(3): [Integer(4), Integer(5), Integer(6)],␣
↪Integer(6): [Integer(9)], Integer(8): [Integer(9)]})
>>> B.allow_multiple_edges(True)
>>> B.add_edge(Integer(0), Integer(7))
>>> len(list(B.perfect_matchings()))
10
```

Empty graph:

```
sage: list(BipartiteGraph().perfect_matchings())
[[]]
```

```
>>> from sage.all import *
>>> list(BipartiteGraph().perfect_matchings())
[[]]
```

Bipartite graph without perfect matching:

```
sage: B = BipartiteGraph(graphs.CompleteBipartiteGraph(3, 4))
sage: list(B.perfect_matchings())
[]
```

```
>>> from sage.all import *
>>> B = BipartiteGraph(graphs.CompleteBipartiteGraph(Integer(3), Integer(4)))
>>> list(B.perfect_matchings())
[]
```

Check that the number of perfect matchings of a complete bipartite graph is consistent with the matching polynomial:

```
sage: B = BipartiteGraph(graphs.CompleteBipartiteGraph(4, 4))
sage: len(list(B.perfect_matchings()))
24
sage: B.matching_polynomial(algorithm='rook')(0)                                #␣
```

(continues on next page)

(continued from previous page)

```
↪needs sage.modules
24
```

```
>>> from sage.all import *
>>> B = BipartiteGraph(graphs.CompleteBipartiteGraph(Integer(4), Integer(4)))
>>> len(list(B.perfect_matchings()))
24
>>> B.matching_polynomial(algorithm='rook')(Integer(0))                    ␣
↪     # needs sage.modules
24
```

plot(**args*, ***kwds*)

Override Graph's plot function, to illustrate the bipartite nature.

EXAMPLES:

```
sage: B = BipartiteGraph(graphs.CycleGraph(20))
sage: B.plot()                                                             #␣
↪needs sage.plot
Graphics object consisting of 41 graphics primitives
```

```
>>> from sage.all import *
>>> B = BipartiteGraph(graphs.CycleGraph(Integer(20)))
>>> B.plot()                                                               #␣
↪needs sage.plot
Graphics object consisting of 41 graphics primitives
```

project_left(*immutable=None*)

Project `self` onto left vertices. Edges are 2-paths in the original.

INPUT:

- `immutable` – boolean (default: `None`); whether to create a mutable/immutable graph. `immutable=None` (default) means that the bipartite graph and its projection will behave the same way.

EXAMPLES:

```
sage: B = BipartiteGraph(graphs.CycleGraph(20))
sage: G = B.project_left()
sage: G.order(), G.size()
(10, 10)
```

```
>>> from sage.all import *
>>> B = BipartiteGraph(graphs.CycleGraph(Integer(20)))
>>> G = B.project_left()
>>> G.order(), G.size()
(10, 10)
```

project_right(*immutable=None*)

Project `self` onto right vertices. Edges are 2-paths in the original.

INPUT:

- `immutable` – boolean (default: `None`); whether to create a mutable/immutable graph. `immutable=None` (default) means that the bipartite graph and its projection will behave the same way.

EXAMPLES:

```
sage: B = BipartiteGraph(graphs.CycleGraph(20))
sage: G = B.project_right()
sage: G.order(), G.size()
(10, 10)
```

```
>>> from sage.all import *
>>> B = BipartiteGraph(graphs.CycleGraph(Integer(20)))
>>> G = B.project_right()
>>> G.order(), G.size()
(10, 10)
```

reduced_adjacency_matrix(*sparse*, *base_ring=True*, ***kwds*)

Return the reduced adjacency matrix for the given graph.

A reduced adjacency matrix contains only the non-redundant portion of the full adjacency matrix for the bipartite graph. Specifically, for zero matrices of the appropriate size, for the reduced adjacency matrix `H`, the full adjacency matrix is `[[0, H'], [H, 0]]`.

By default, the matrix returned is over the integers.

INPUT:

- `sparse` – boolean (default: `True`); whether to return a sparse matrix
- `base_ring` – a ring (default: `None`); the base ring of the matrix space to use. By default, the base ring is `ZZ` if the graph is not weighted and otherwise the same ring as the (first) weights.
- `**kwds` – other keywords to pass to `matrix()`

EXAMPLES:

Bipartite graphs that are not weighted will return a matrix over ZZ, unless a base ring is specified:

```
sage: M = Matrix([(1,1,1,0,0,0,0), (1,0,0,1,1,0,0),
....:             (0,1,0,1,0,1,0), (1,1,0,1,0,0,1)])
sage: B = BipartiteGraph(M)
sage: N = B.reduced_adjacency_matrix(); N
[1 1 1 0 0 0]
[1 0 0 1 1 0]
[0 1 0 1 0 1]
[1 1 0 1 0 0]
sage: N == M
True
sage: N[0,0].parent()
Integer Ring
sage: N2 = B.reduced_adjacency_matrix(base_ring=RDF); N2
[1.0 1.0 1.0 0.0 0.0 0.0]
[1.0 0.0 0.0 1.0 1.0 0.0]
[0.0 1.0 0.0 1.0 0.0 1.0]
[1.0 1.0 0.0 1.0 0.0 0.0]
sage: N2[0, 0].parent()
Real Double Field
```

```
>>> from sage.all import *
>>> M = Matrix([(Integer(1),Integer(1),Integer(1),Integer(0),Integer(0),
```

(continues on next page)

(continued from previous page)

```
↪Integer(0),Integer(0)), (Integer(1),Integer(0),Integer(0),Integer(1),
↪Integer(1),Integer(0),Integer(0)),
...              (Integer(0),Integer(1),Integer(0),Integer(1),Integer(0),
↪Integer(1),Integer(0)), (Integer(1),Integer(1),Integer(0),Integer(1),
↪Integer(0),Integer(0),Integer(1))])
>>> B = BipartiteGraph(M)
>>> N = B.reduced_adjacency_matrix(); N
[1 1 1 0 0 0 0]
[1 0 0 1 1 0 0]
[0 1 0 1 0 1 0]
[1 1 0 1 0 0 1]
>>> N == M
True
>>> N[Integer(0),Integer(0)].parent()
Integer Ring
>>> N2 = B.reduced_adjacency_matrix(base_ring=RDF); N2
[1.0 1.0 1.0 0.0 0.0 0.0 0.0]
[1.0 0.0 0.0 1.0 1.0 0.0 0.0]
[0.0 1.0 0.0 1.0 0.0 1.0 0.0]
[1.0 1.0 0.0 1.0 0.0 0.0 1.0]
>>> N2[Integer(0), Integer(0)].parent()
Real Double Field
```

Multi-edge graphs also return a matrix over ZZ, unless a base ring is specified:

```
sage: M = Matrix([(1,1,2,0,0), (0,2,1,1,1), (0,1,2,1,1)])
sage: B = BipartiteGraph(M, multiedges=True, sparse=True)
sage: N = B.reduced_adjacency_matrix()
sage: N == M
True
sage: N[0,0].parent()
Integer Ring
sage: N2 = B.reduced_adjacency_matrix(base_ring=RDF)
sage: N2[0, 0].parent()
Real Double Field
```

```
>>> from sage.all import *
>>> M = Matrix([(Integer(1),Integer(1),Integer(2),Integer(0),Integer(0)),␣
↪(Integer(0),Integer(2),Integer(1),Integer(1),Integer(1)), (Integer(0),
↪Integer(1),Integer(2),Integer(1),Integer(1))])
>>> B = BipartiteGraph(M, multiedges=True, sparse=True)
>>> N = B.reduced_adjacency_matrix()
>>> N == M
True
>>> N[Integer(0),Integer(0)].parent()
Integer Ring
>>> N2 = B.reduced_adjacency_matrix(base_ring=RDF)
>>> N2[Integer(0), Integer(0)].parent()
Real Double Field
```

Weighted graphs will return a matrix over the ring given by their (first) weights, unless a base ring is specified:

```
sage: F.<a> = GF(4)
sage: MS = MatrixSpace(F, 2, 3)
sage: M = MS.matrix([[0, 1, a+1], [a, 1, 1]])
sage: B = BipartiteGraph(M, weighted=True, sparse=True)
sage: N = B.reduced_adjacency_matrix(sparse=False)
sage: N == M
True
sage: N[0,0].parent()
Finite Field in a of size 2^2
sage: N2 = B.reduced_adjacency_matrix(base_ring=F)
sage: N2[0, 0].parent()
Finite Field in a of size 2^2
```

```
>>> from sage.all import *
>>> F = GF(Integer(4), names=('a',)); (a,) = F._first_ngens(1)
>>> MS = MatrixSpace(F, Integer(2), Integer(3))
>>> M = MS.matrix([[Integer(0), Integer(1), a+Integer(1)], [a, Integer(1),␣
↪Integer(1)]])
>>> B = BipartiteGraph(M, weighted=True, sparse=True)
>>> N = B.reduced_adjacency_matrix(sparse=False)
>>> N == M
True
>>> N[Integer(0),Integer(0)].parent()
Finite Field in a of size 2^2
>>> N2 = B.reduced_adjacency_matrix(base_ring=F)
>>> N2[Integer(0), Integer(0)].parent()
Finite Field in a of size 2^2
```

relabel(*perm=None*, *inplace=True*, *return_map=False*, *check_input=True*, *complete_partial_function=True*, *immutable=None*)

Relabel the vertices of `self`.

INPUT:

- `perm` – a function, dictionary, iterable, permutation, or `None` (default: `None`)
- `inplace` – boolean (default: `True`)
- `return_map` – boolean (default: `False`)
- `check_input` – boolean; whether to test input for correctness. *This can potentially be very time-consuming !*
- `complete_partial_function` – boolean; whether to automatically complete the permutation if some elements of the graph are not associated with any new name. In this case, those elements are not relabeled *This can potentially be very time-consuming !*.
- `immutable` – boolean; with `inplace=False`, whether to create a mutable/immutable relabelled copy. `immutable=None` (default) means that the graph and its copy will behave the same way.

If `perm` is a function `f`, then each vertex `v` is relabeled to `f(v)`.

If `perm` is a dictionary `d`, then each vertex `v` (which should be a key of `d`) is relabeled to `d[v]`.

If `perm` is a list (or more generally, any iterable) of length `n`, then the first vertex returned by `G.vertices(sort=True)` is relabeled to `l[0]`, the second to `l[1]`, ...

If `perm` is a permutation, then each vertex `v` is relabeled to `perm(v)`.

If `perm` is `None`, the graph is relabeled to be on the vertices $\{0, 1, ..., n-1\}$. This is *not* any kind of canonical labeling, but it is consistent (relabeling twice will give the same result).

If `inplace` is `True`, the graph is modified in place and `None` is returned. Otherwise a relabeled copy of the graph is returned.

If `return_map` is `True` a dictionary representing the relabelling map is returned (incompatible with `inplace==False`).

EXAMPLES:

```
sage: G = BipartiteGraph(graphs.PathGraph(3))
sage: G.am()                                                                #␣
↪needs sage.modules
[0 1 0]
[1 0 1]
[0 1 0]
```

```
>>> from sage.all import *
>>> G = BipartiteGraph(graphs.PathGraph(Integer(3)))
>>> G.am()                                                                  #␣
↪needs sage.modules
[0 1 0]
[1 0 1]
[0 1 0]
```

Relabeling using a dictionary. Note that the dictionary does not define the new label of vertex 0:

```
sage: G.relabel({1:2,2:1}, inplace=False).am()                              #␣
↪needs sage.modules
[0 0 1]
[0 0 1]
[1 1 0]
```

```
>>> from sage.all import *
>>> G.relabel({Integer(1):Integer(2),Integer(2):Integer(1)}, inplace=False).
↪am()                                  # needs sage.modules
[0 0 1]
[0 0 1]
[1 1 0]
```

This is because the method automatically "extends" the relabeling to the missing vertices (whose label will not change). Checking that all vertices have an image can require some time, and this feature can be disabled (at your own risk):

```
sage: G.relabel({1:2,2:1}, inplace=False,                                   #␣
↪needs sage.modules
....:           complete_partial_function=False).am()
Traceback (most recent call last):
...
KeyError: 0
```

```
>>> from sage.all import *
>>> G.relabel({Integer(1):Integer(2),Integer(2):Integer(1)}, inplace=False,  ␣
```

(continues on next page)

(continued from previous page)

```
↪                                  # needs sage.modules
...           complete_partial_function=False).am()
Traceback (most recent call last):
...
KeyError: 0
```

Relabeling using a list:

```
sage: G.relabel([0,2,1], inplace=False).am()                                  #␣
↪needs sage.modules
[0 0 1]
[0 0 1]
[1 1 0]
```

```
>>> from sage.all import *
>>> G.relabel([Integer(0),Integer(2),Integer(1)], inplace=False).am()        ␣
↪                          # needs sage.modules
[0 0 1]
[0 0 1]
[1 1 0]
```

Relabeling using an iterable:

```
sage: G.relabel(iter((0,2,1)), inplace=False).am()                            #␣
↪needs sage.modules
[0 0 1]
[0 0 1]
[1 1 0]
```

```
>>> from sage.all import *
>>> G.relabel(iter((Integer(0),Integer(2),Integer(1))), inplace=False).am()  ␣
↪                          # needs sage.modules
[0 0 1]
[0 0 1]
[1 1 0]
```

Relabeling using a Sage permutation:

```
sage: G = BipartiteGraph(graphs.PathGraph(3))
sage: from sage.groups.perm_gps.permgroup_named import SymmetricGroup        #␣
↪needs sage.groups
sage: S = SymmetricGroup(3)                                                   #␣
↪needs sage.groups
sage: gamma = S('(1,2)')                                                      #␣
↪needs sage.groups
sage: G.relabel(gamma, inplace=False).am()                                    #␣
↪needs sage.groups sage.modules
[0 0 1]
[0 0 1]
[1 1 0]

sage: C4 = BipartiteGraph(graphs.CycleGraph(4))
```

(continues on next page)

(continued from previous page)

```
sage: C4.relabel({u: str(u) for u in C4}, inplace=True)
sage: ar = C4.automorphism_group().random_element()
sage: R = C4.relabel(ar, inplace=False)
sage: perm = C4.is_isomorphic(R, certificate=True)[1]
sage: perm == {u: ar(u) for u in C4}
True
```

```
>>> from sage.all import *
>>> G = BipartiteGraph(graphs.PathGraph(Integer(3)))
>>> from sage.groups.perm_gps.permgroup_named import SymmetricGroup        #␣
↪needs sage.groups
>>> S = SymmetricGroup(Integer(3))                                          ␣
↪     # needs sage.groups
>>> gamma = S('(1,2)')                                                      #␣
↪needs sage.groups
>>> G.relabel(gamma, inplace=False).am()                                    #␣
↪needs sage.groups sage.modules
[0 0 1]
[0 0 1]
[1 1 0]

>>> C4 = BipartiteGraph(graphs.CycleGraph(Integer(4)))
>>> C4.relabel({u: str(u) for u in C4}, inplace=True)
>>> ar = C4.automorphism_group().random_element()
>>> R = C4.relabel(ar, inplace=False)
>>> perm = C4.is_isomorphic(R, certificate=True)[Integer(1)]
>>> perm == {u: ar(u) for u in C4}
True
```

Relabeling using an injective function:

```
sage: G.edges(sort=True)
[(0, 1, None), (1, 2, None)]
sage: H = G.relabel(lambda i: i+10, inplace=False)
sage: H.vertices(sort=True)
[10, 11, 12]
sage: H.edges(sort=True)
[(10, 11, None), (11, 12, None)]
```

```
>>> from sage.all import *
>>> G.edges(sort=True)
[(0, 1, None), (1, 2, None)]
>>> H = G.relabel(lambda i: i+Integer(10), inplace=False)
>>> H.vertices(sort=True)
[10, 11, 12]
>>> H.edges(sort=True)
[(10, 11, None), (11, 12, None)]
```

Relabeling using a non injective function has no meaning:

```
sage: G.edges(sort=True)
[(0, 1, None), (1, 2, None)]
```

(continues on next page)

(continued from previous page)

```
sage: G.relabel(lambda i: 0, inplace=False)
Traceback (most recent call last):
...
NotImplementedError: Non injective relabeling
```

```
>>> from sage.all import *
>>> G.edges(sort=True)
[(0, 1, None), (1, 2, None)]
>>> G.relabel(lambda i: Integer(0), inplace=False)
Traceback (most recent call last):
...
NotImplementedError: Non injective relabeling
```

But this test can be disabled, which can lead to … problems:

```
sage: G.edges(sort=True)
[(0, 1, None), (1, 2, None)]
sage: G.relabel(lambda i: 0, check_input = False)
sage: G.edges(sort=True)
[]
```

```
>>> from sage.all import *
>>> G.edges(sort=True)
[(0, 1, None), (1, 2, None)]
>>> G.relabel(lambda i: Integer(0), check_input = False)
>>> G.edges(sort=True)
[]
```

Recovering the relabeling with `return_map`:

```
sage: G = BipartiteGraph(graphs.CubeGraph(3))
sage: G.relabel(range(8), return_map=True)
{'000': 0,
 '001': 1,
 '010': 2,
 '011': 3,
 '100': 4,
 '101': 5,
 '110': 6,
 '111': 7}
```

```
>>> from sage.all import *
>>> G = BipartiteGraph(graphs.CubeGraph(Integer(3)))
>>> G.relabel(range(Integer(8)), return_map=True)
{'000': 0,
 '001': 1,
 '010': 2,
 '011': 3,
 '100': 4,
 '101': 5,
 '110': 6,
 '111': 7}
```

When no permutation is given, the relabeling is done to integers from 0 to N-1 but in an arbitrary order:

```
sage: G = BipartiteGraph(graphs.CubeGraph(3))
sage: G.vertices(sort=True)
['000', '001', '010', '011', '100', '101', '110', '111']
sage: G.relabel()
sage: G.vertices(sort=True)
[0, 1, 2, 3, 4, 5, 6, 7]
```

```
>>> from sage.all import *
>>> G = BipartiteGraph(graphs.CubeGraph(Integer(3)))
>>> G.vertices(sort=True)
['000', '001', '010', '011', '100', '101', '110', '111']
>>> G.relabel()
>>> G.vertices(sort=True)
[0, 1, 2, 3, 4, 5, 6, 7]
```

In the above case, the mapping is arbitrary but consistent:

```
sage: map1 = G.relabel(inplace=False, return_map=True)
sage: map2 = G.relabel(inplace=False, return_map=True)
sage: map1 == map2
True
```

```
>>> from sage.all import *
>>> map1 = G.relabel(inplace=False, return_map=True)
>>> map2 = G.relabel(inplace=False, return_map=True)
>>> map1 == map2
True
```

```
sage: G = BipartiteGraph(graphs.PathGraph(3))
sage: G.relabel(lambda i: i+10, return_map=True)
{0: 10, 1: 11, 2: 12}
```

```
>>> from sage.all import *
>>> G = BipartiteGraph(graphs.PathGraph(Integer(3)))
>>> G.relabel(lambda i: i+Integer(10), return_map=True)
{0: 10, 1: 11, 2: 12}
```

save_afile(*fname*)

Save the graph to file in alist format.

Saves this graph to file in David MacKay's alist format, see http://www.inference.phy.cam.ac.uk/mackay/codes/data.html for examples and definition of the format.

EXAMPLES:

```
sage: M = Matrix([(1,1,1,0,0,0,0), (1,0,0,1,1,0,0),
....:             (0,1,0,1,0,1,0), (1,1,0,1,0,0,1)])
sage: M
[1 1 1 0 0 0 0]
[1 0 0 1 1 0 0]
[0 1 0 1 0 1 0]
```

(continues on next page)

(continued from previous page)

```
[1 1 0 1 0 0 1]
sage: b = BipartiteGraph(M)
sage: import tempfile
sage: with tempfile.NamedTemporaryFile(delete_on_close=False) as f:
....:     b.save_afile(f.name)
....:     b2 = BipartiteGraph(f.name)
sage: b.is_isomorphic(b2)
True
```

```
>>> from sage.all import *
>>> M = Matrix([(Integer(1),Integer(1),Integer(1),Integer(0),Integer(0),
↪Integer(0),Integer(0)), (Integer(1),Integer(0),Integer(0),Integer(1),
↪Integer(1),Integer(0),Integer(0)),
...              (Integer(0),Integer(1),Integer(0),Integer(1),Integer(0),
↪Integer(1),Integer(0)), (Integer(1),Integer(1),Integer(0),Integer(1),
↪Integer(0),Integer(0),Integer(1))])
>>> M
[1 1 1 0 0 0 0]
[1 0 0 1 1 0 0]
[0 1 0 1 0 1 0]
[1 1 0 1 0 0 1]
>>> b = BipartiteGraph(M)
>>> import tempfile
>>> with tempfile.NamedTemporaryFile(delete_on_close=False) as f:
...     b.save_afile(f.name)
...     b2 = BipartiteGraph(f.name)
>>> b.is_isomorphic(b2)
True
```

to_undirected()

Return an undirected Graph (without bipartite constraint) of the given object.

EXAMPLES:

```
sage: BipartiteGraph(graphs.CycleGraph(6)).to_undirected()
Cycle graph: Graph on 6 vertices
```

```
>>> from sage.all import *
>>> BipartiteGraph(graphs.CycleGraph(Integer(6))).to_undirected()
Cycle graph: Graph on 6 vertices
```

vertex_cover(*algorithm*, *value_only='Konig'*, *reduction_rules=False*, *solver=True*, *verbose=None*, *integrality_tolerance=0*)

Return a minimum vertex cover of `self` represented by a set of vertices.

A minimum vertex cover of a graph is a set S of vertices such that each edge is incident to at least one element of S, and such that S is of minimum cardinality. For more information, see Wikipedia article Vertex_cover.

Equivalently, a vertex cover is defined as the complement of an independent set.

As an optimization problem, it can be expressed as follows:

$$\begin{aligned}\text{Minimize : } & \sum_{v \in G} b_v \\ \text{Such that : } & \forall (u,v) \in G.edges(sort = True), b_u + b_v \geq 1 \\ & \forall x \in G, b_x \text{ is a binary variable}\end{aligned}$$

INPUT:

- `algorithm` – string (default: `'Konig'`); algorithm to use among:
  - `'Konig'` will compute a minimum vertex cover using Konig's algorithm (Wikipedia article Kőnig%27s_theorem_(graph_theory))
  - `'Cliquer'` will compute a minimum vertex cover using the Cliquer package
  - `'MILP'` will compute a minimum vertex cover through a mixed integer linear program
  - `'mcqd'` will use the MCQD solver (http://www.sicmm.org/~konc/maxclique/), and the MCQD package must be installed
- `value_only` – boolean (default: `False`); if set to `True`, only the size of a minimum vertex cover is returned. Otherwise, a minimum vertex cover is returned as a list of vertices.
- `reduction_rules` – (default: `True`) specify if the reductions rules from kernelization must be applied as pre-processing or not. See [ACFLSS04] for more details. Note that depending on the instance, it might be faster to disable reduction rules. This parameter is currently ignored when `algorithm == "Konig"`.
- `solver` – string (default: `None`); specifies a Mixed Integer Linear Programming (MILP) solver to be used. If set to `None`, the default one is used. For more information on MILP solvers and which default solver is used, see the method `solve` of the class `MixedIntegerLinearProgram`.
- `verbose` – integer (default: 0); sets the level of verbosity. Set to 0 by default, which means quiet.
- `integrality_tolerance` – float; parameter for use with MILP solvers over an inexact base ring; see `MixedIntegerLinearProgram.get_values()`.

EXAMPLES:

On the Cycle Graph:

```
sage: B = BipartiteGraph(graphs.CycleGraph(6))
sage: len(B.vertex_cover())                                                 #␣
↪needs networkx
3
sage: B.vertex_cover(value_only=True)                                       #␣
↪needs networkx
3
```

```
>>> from sage.all import *
>>> B = BipartiteGraph(graphs.CycleGraph(Integer(6)))
>>> len(B.vertex_cover())                                                   #␣
↪needs networkx
3
>>> B.vertex_cover(value_only=True)                                         #␣
↪needs networkx
3
```

The two algorithms should return the same result:

```
sage: # needs networkx numpy
sage: g = BipartiteGraph(graphs.RandomBipartite(10, 10, .5))
sage: vc1 = g.vertex_cover(algorithm='Konig')
sage: vc2 = g.vertex_cover(algorithm='Cliquer')
sage: len(vc1) == len(vc2)
True
```

```
>>> from sage.all import *
>>> # needs networkx numpy
>>> g = BipartiteGraph(graphs.RandomBipartite(Integer(10), Integer(10),␣
↪RealNumber('.5')))
>>> vc1 = g.vertex_cover(algorithm='Konig')
>>> vc2 = g.vertex_cover(algorithm='Cliquer')
>>> len(vc1) == len(vc2)
True
```

1.5 Matching covered graphs

This module implements functions and operations pertaining to matching covered graphs.

A *matching* in a graph is a set of pairwise nonadjacent links (nonloop edges). In other words, a matching in a graph is the edge set of an 1-regular subgraph. A matching is called a *perfect matching* if it the subgraph generated by a set of matching edges spans the graph, i.e. it's the edge set of an 1-regular spanning subgraph. A connected nontrivial graph is called *matching covered* if each edge participates in some perfect matching.

Barriers and canonical partition

canonical_partition()	Return the canonical partition of the (matching covered) graph.
maximal_barrier()	Return the (unique) maximal barrier containing the vertex.

Bricks, braces and tight cut decomposition

is_brace()	Check if the (matching covered) graph is a brace.
is_brick()	Check if the (matching covered) graph is a brick.

Miscellaneous methods

get_matching()	Return an *EdgesView* of `self._matching`.
update_matching()	Update the perfect matching captured in `self._matching`.

Overwritten methods

add_edge()	Add an edge from vertex `u` to vertex `v`.
add_edges()	Add edges from an iterable container.
add_vertex()	Add a vertex to the (matching covered) graph.
add_vertices()	Add vertices to the (matching covered) graph from an iterable container of vertices.
allow_loops()	Change whether loops are allowed in (matching covered) graphs.
allows_loops()	Return whether loops are permitted in (matching covered) graphs.
delete_vertex()	Delete a vertex, removing all incident edges.
delete_vertices()	Delete specified vertices form `self`.
has_loops()	Check whether there are loops in the (matching covered) graph.
has_perfect_matching()	Check whether the graph has a perfect matching.
is_biconnected()	Check whether the (matching covered) graph is biconnected.
loop_vertices()	Return a list of vertices with loops.
loops()	Return a list of all loops in the (matching covered) graph.
number_of_loops()	Return the number of edges that are loops.
remove_loops()	Remove loops on vertices in `vertices`.
subdivide_edge()	Subdivide an edge k times.
subdivide_edges()	Subdivide k times edges from an iterable container.

REFERENCES:

- This methods of this module has been adopted and inspired by the book of Lucchesi and Murty — *Perfect Matchings: a theory of matching covered graphs* [LM2024].

AUTHORS:

- Janmenjaya Panda (2024-06-14): initial version

Todo

The following methods are to be incorporated in *MatchingCoveredGraph*:

`__hash__()`	Compute a hash for `self`, if `self` is immutable.
`_subgraph_by_deleting()`	Return the matching covered subgraph containing the provided vertices and edges.

Overwritten Methods

`add_clique()`	Add a clique to the graph with the provided vertices.
`add_cycle()`	Add a cycle to the graph with the provided vertices.
`add_path()`	Add a path to the graph with the provided vertices.
`cartesian_product()`	Return the Cartesian product of `self` and `other`.
`clear()`	Empties the graph of vertices and edges and removes name, associated objects, and position information.
`complement()`	Return the complement of the graph.
`contract_edge()`	Contract an edge from `u` to `v`.
`contract_edges()`	Contract edges from an iterable container.
`degree_constrained_subgraph()`	Return a degree-constrained matching covered subgraph.
`delete_edge()`	Delete the edge from `u` to `v`.
`delete_edges()`	Delete edges from an iterable container.
`delete_multiedge()`	Delete all edges from `u` to `v`.

disjoint_union()	Return the disjoint union of self and other.
disjunctive_product()	Return the disjunctive product of self and other.
is_block_graph()	Check whether the matching covered graph is a block graph.
is_cograph()	Check whether the matching covered graph is cograph.
is_forest()	Check if the matching covered graph is a forest, i.e. a disjoint union of trees.
is_matching_covered()	Check if the graph is matching covered.
is_path()	Check whether the graph is a path.
is_subgraph()	Check whether the matching covered graph is a subgraph of other.
is_tree()	Check whether the matching covered graph is a tree.
join()	Return the join of self and other.
lexicographic_product()	Return the lexicographic product of self and other.
load_afile()	Load the matching covered graph specified in the given file into the current object.
merge_vertices()	Merge vertices.
minor()	Return the vertices of a minor isomorphic to H in the current graph.
random_subgraph()	Return a random matching covered subgraph containing each vertex with probability p.
save_afile()	Save the graph to file in alist format.
strong_product()	Return the strong product of self and other.
subgraph()	Return the matching covered subgraph containing the given vertices and edges.
subgraph_search()	Return a copy of (matching covered) G in self.
subgraph_search_count()	Return the number of labelled occurrences of (matching covered) G in self.
subgraph_search_iterator()	Return an iterator over the labelled copies of (matching covered) G in self.
tensor_product()	Return the tensor product of self and other.
to_undirected()	Return an undirected Graph instance of the matching covered graph.
topological_minor()	Return a topological H-minor from self if one exists.
transitive_closure()	Return the transitive closure of the matching covered graph.
transitive_reduction()	Return a transitive reduction of the matching covered graph.
union()	Return the union of self and other.

Bricks, braces and tight cut decomposition

is_brace()	Check if the matching covered graph is a brace.
is_brick()	Check if the matching covered graph is a brick.

Removability and ear decomposition

add_ear()	Add an ear to the graph with the provided end vertices number of internal vertices.
efficient_ear_decomposition()	Return a matching covered ear decomposition computed at the fastest possible time.
is_removable_double_ear()	Check whether the pair of ears form a removable double ear.
is_removable_doubleton()	Check whether the pair of edges constitute a removable doubleton.
is_removable_ear()	Check whether the ear is removable.
is_removable_edge()	Check whether the edge is removable.
optimal_ear_decomposition()	Return an optimal ear decomposition.
removable_double_ears()	Return a list of removable double ears.
removable_doubletons()	Return a list of removable doubletons.
removable_ears()	Return a list of removable ears.
removable_edges()	Return a *EdgesView* of removable edges.
retract()	Compute the retract of the (matching covered) graph.

Generating bricks and braces

brace_generation_sequence()	Return a McCuaig brace generation sequence of the (provided) brace.
brick_generation_sequence()	Return a Norine-Thomas brick generation sequence of the (provided) brick.
is_mccuaig_brace()	Check if the brace is a McCuaig brace.
is_norine_thomas_brick()	Check if the brick is a Norine-Thomas brick.

1.5.1 Methods

class sage.graphs.matching_covered_graph.**MatchingCoveredGraph**(*data=None*, *matching=None*, *algorithm='Edmonds'*, *solver=None*, *verbose=0*, *integrality_tolerance=0.001*, **args*, ***kwds*)

Bases: *Graph*

Matching covered graph

INPUT:

- data – can be any of the following:
  - Empty or None (throws a ValueError as the graph must be nontrival).
  - An arbitrary graph.

- `matching` – (default: `None`); a perfect matching of the graph, that can be given using any valid input format of *Graph*.

  If set to `None`, a matching is computed using the other parameters.

- `algorithm` – string (default: `'Edmonds'`); the algorithm to be used to compute a maximum matching of the graph among

  - `'Edmonds'` selects Edmonds' algorithm as implemented in NetworkX,
  - `'LP'` uses a Linear Program formulation of the matching problem.

- `solver` – string (default: `None`); specify a Mixed Integer Linear Programming (MILP) solver to be used. If set to `None`, the default one is used. For more information on MILP solvers and which default solver is used, see the method `solve` of the class `MixedIntegerLinearProgram`.

- `verbose` – integer (default: `0`); sets the level of verbosity: set to 0 by default, which means quiet (only useful when `algorithm == 'LP'`).

- `integrality_tolerance` – float; parameter for use with MILP solvers over an inexact base ring; see `MixedIntegerLinearProgram.get_values()`.

OUTPUT:

- An object of the class *MatchingCoveredGraph* if the input is valid and the graph is matching covered, or otherwise an error is thrown.

Note

All remaining arguments are passed to the `Graph` constructor

EXAMPLES:

Generating an object of the class `MatchingCoveredGraph` from the provided instance of `Graph` without providing any other information:

```
sage: G = MatchingCoveredGraph(graphs.PetersenGraph())
sage: G
Matching covered petersen graph: graph on 10 vertices
sage: sorted(G.get_matching())
[(0, 5, None), (1, 6, None), (2, 7, None), (3, 8, None), (4, 9, None)]

sage: G = graphs.StaircaseGraph(4)
sage: H = MatchingCoveredGraph(G)
sage: H
Matching covered staircase graph: graph on 8 vertices
sage: H == G
True
sage: sorted(H.get_matching())
[(0, 1, None), (2, 7, None), (3, 6, None), (4, 5, None)]

sage: G = Graph({0: [1, 2, 3, 4], 1: [2, 5],
....:            2: [5], 3: [4, 5], 4: [5]})
sage: H = MatchingCoveredGraph(G)
sage: H
Matching covered graph on 6 vertices
sage: H == G
```

(continues on next page)

(continued from previous page)

```
True
sage: sorted(H.get_matching())
[(0, 4, None), (1, 2, None), (3, 5, None)]

sage: # needs networkx
sage: import networkx
sage: G = Graph(networkx.complete_bipartite_graph(12, 12))
sage: H = MatchingCoveredGraph(G)
sage: H
Matching covered graph on 24 vertices
sage: H == G
True
sage: sorted(H.get_matching())
[(0, 15, None), (1, 14, None), (2, 13, None), (3, 12, None),
 (4, 23, None), (5, 22, None), (6, 21, None), (7, 20, None),
 (8, 19, None), (9, 18, None), (10, 17, None), (11, 16, None)]

sage: G = Graph('E|fG', sparse=True)
sage: H = MatchingCoveredGraph(G)
sage: H
Matching covered graph on 6 vertices
sage: H == G
True
sage: sorted(H.get_matching())
[(0, 5, None), (1, 2, None), (3, 4, None)]

sage: M = Matrix([(0,1,0,0,1,1,0,0,0,0),
....:             (1,0,1,0,0,0,1,0,0,0),
....:             (0,1,0,1,0,0,0,1,0,0),
....:             (0,0,1,0,1,0,0,0,1,0),
....:             (1,0,0,1,0,0,0,0,0,1),
....:             (1,0,0,0,0,0,0,1,1,0),
....:             (0,1,0,0,0,0,0,0,1,1),
....:             (0,0,1,0,0,1,0,0,0,1),
....:             (0,0,0,1,0,1,1,0,0,0),
....:             (0,0,0,0,1,0,1,1,0,0)])
sage: M
[0 1 0 0 1 1 0 0 0 0]
[1 0 1 0 0 0 1 0 0 0]
[0 1 0 1 0 0 0 1 0 0]
[0 0 1 0 1 0 0 0 1 0]
[1 0 0 1 0 0 0 0 0 1]
[1 0 0 0 0 0 0 1 1 0]
[0 1 0 0 0 0 0 0 1 1]
[0 0 1 0 0 1 0 0 0 1]
[0 0 0 1 0 1 1 0 0 0]
[0 0 0 0 1 0 1 1 0 0]
sage: G = Graph(M)
sage: H = MatchingCoveredGraph(G)
sage: H == G
True
sage: sorted(H.get_matching())
```

(continues on next page)

(continued from previous page)

```
[(0, 5, None), (1, 6, None), (2, 7, None), (3, 8, None), (4, 9, None)]

sage: M = Matrix([(-1, 0, 0, 0, 1, 0, 0, 0, 0, 0,-1, 0, 0, 0, 0),
....:             ( 1,-1, 0, 0, 0, 0, 0, 0, 0, 0, 0,-1, 0, 0, 0),
....:             ( 0, 1,-1, 0, 0, 0, 0, 0, 0, 0, 0, 0,-1, 0, 0),
....:             ( 0, 0, 1,-1, 0, 0, 0, 0, 0, 0, 0, 0, 0,-1, 0),
....:             ( 0, 0, 0, 1,-1, 0, 0, 0, 0, 0, 0, 0, 0, 0,-1),
....:             ( 0, 0, 0, 0, 0,-1, 0, 0, 0, 1, 1, 0, 0, 0, 0),
....:             ( 0, 0, 0, 0, 0, 0, 0, 1,-1, 0, 0, 1, 0, 0, 0),
....:             ( 0, 0, 0, 0, 0, 1,-1, 0, 0, 0, 0, 0, 1, 0, 0),
....:             ( 0, 0, 0, 0, 0, 0, 0, 0, 1,-1, 0, 0, 0, 1, 0),
....:             ( 0, 0, 0, 0, 0, 0, 1,-1, 0, 0, 0, 0, 0, 0, 1)])
sage: M
[-1  0  0  0  1  0  0  0  0  0 -1  0  0  0  0]
[ 1 -1  0  0  0  0  0  0  0  0  0 -1  0  0  0]
[ 0  1 -1  0  0  0  0  0  0  0  0  0 -1  0  0]
[ 0  0  1 -1  0  0  0  0  0  0  0  0  0 -1  0]
[ 0  0  0  1 -1  0  0  0  0  0  0  0  0  0 -1]
[ 0  0  0  0  0 -1  0  0  0  1  1  0  0  0  0]
[ 0  0  0  0  0  0  0  1 -1  0  0  1  0  0  0]
[ 0  0  0  0  0  1 -1  0  0  0  0  0  1  0  0]
[ 0  0  0  0  0  0  0  0  1 -1  0  0  0  1  0]
[ 0  0  0  0  0  0  1 -1  0  0  0  0  0  0  1]
sage: G = Graph(M)
sage: H = MatchingCoveredGraph(G)
sage: H == G
True
sage: sorted(H.get_matching())
[(0, 5, None), (1, 6, None), (2, 7, None), (3, 8, None), (4, 9, None)]

sage: G = Graph([(0, 1), (0, 3), (0, 4), (1, 2), (1, 5), (2, 3),
....:            (2, 6), (3, 7), (4, 5), (4, 7), (5, 6), (6, 7)])
sage: H = MatchingCoveredGraph(G)
sage: H == G
True
sage: sorted(H.get_matching())
[(0, 4, None), (1, 5, None), (2, 6, None), (3, 7, None)]

sage: # optional - python_igraph
sage: import igraph
sage: G = Graph(igraph.Graph([(0, 1), (0, 3), (1, 2), (2, 3)]))
sage: H = MatchingCoveredGraph(G)
sage: H
Matching covered graph on 4 vertices
sage: sorted(H.get_matching())
[(0, 3, {}), (1, 2, {})]
```

```
>>> from sage.all import *
>>> G = MatchingCoveredGraph(graphs.PetersenGraph())
>>> G
Matching covered petersen graph: graph on 10 vertices
>>> sorted(G.get_matching())
```

(continues on next page)

(continued from previous page)

```
[(0, 5, None), (1, 6, None), (2, 7, None), (3, 8, None), (4, 9, None)]

>>> G = graphs.StaircaseGraph(Integer(4))
>>> H = MatchingCoveredGraph(G)
>>> H
Matching covered staircase graph: graph on 8 vertices
>>> H == G
True
>>> sorted(H.get_matching())
[(0, 1, None), (2, 7, None), (3, 6, None), (4, 5, None)]

>>> G = Graph({Integer(0): [Integer(1), Integer(2), Integer(3), Integer(4)],␣
↪Integer(1): [Integer(2), Integer(5)],
...            Integer(2): [Integer(5)], Integer(3): [Integer(4), Integer(5)],␣
↪Integer(4): [Integer(5)]})
>>> H = MatchingCoveredGraph(G)
>>> H
Matching covered graph on 6 vertices
>>> H == G
True
>>> sorted(H.get_matching())
[(0, 4, None), (1, 2, None), (3, 5, None)]

>>> # needs networkx
>>> import networkx
>>> G = Graph(networkx.complete_bipartite_graph(Integer(12), Integer(12)))
>>> H = MatchingCoveredGraph(G)
>>> H
Matching covered graph on 24 vertices
>>> H == G
True
>>> sorted(H.get_matching())
[(0, 15, None), (1, 14, None), (2, 13, None), (3, 12, None),
 (4, 23, None), (5, 22, None), (6, 21, None), (7, 20, None),
 (8, 19, None), (9, 18, None), (10, 17, None), (11, 16, None)]

>>> G = Graph('E|fG', sparse=True)
>>> H = MatchingCoveredGraph(G)
>>> H
Matching covered graph on 6 vertices
>>> H == G
True
>>> sorted(H.get_matching())
[(0, 5, None), (1, 2, None), (3, 4, None)]

>>> M = Matrix([(Integer(0),Integer(1),Integer(0),Integer(0),Integer(1),
↪Integer(1),Integer(0),Integer(0),Integer(0),Integer(0)),
...             (Integer(1),Integer(0),Integer(1),Integer(0),Integer(0),
↪Integer(0),Integer(1),Integer(0),Integer(0),Integer(0)),
...             (Integer(0),Integer(1),Integer(0),Integer(1),Integer(0),
↪Integer(0),Integer(0),Integer(1),Integer(0),Integer(0)),
...             (Integer(0),Integer(0),Integer(1),Integer(0),Integer(1),
```

(continues on next page)

(continued from previous page)

```
↪Integer(0),Integer(0),Integer(0),Integer(1),Integer(0)),
...              (Integer(1),Integer(0),Integer(0),Integer(1),Integer(0),
↪Integer(0),Integer(0),Integer(0),Integer(0),Integer(1)),
...              (Integer(1),Integer(0),Integer(0),Integer(0),Integer(0),
↪Integer(0),Integer(0),Integer(1),Integer(1),Integer(0)),
...              (Integer(0),Integer(1),Integer(0),Integer(0),Integer(0),
↪Integer(0),Integer(0),Integer(0),Integer(1),Integer(1)),
...              (Integer(0),Integer(0),Integer(1),Integer(0),Integer(0),
↪Integer(1),Integer(0),Integer(0),Integer(0),Integer(1)),
...              (Integer(0),Integer(0),Integer(0),Integer(1),Integer(0),
↪Integer(1),Integer(1),Integer(0),Integer(0),Integer(0)),
...              (Integer(0),Integer(0),Integer(0),Integer(0),Integer(1),
↪Integer(0),Integer(1),Integer(1),Integer(0),Integer(0))])
>>> M
[0 1 0 0 1 1 0 0 0 0]
[1 0 1 0 0 0 1 0 0 0]
[0 1 0 1 0 0 0 1 0 0]
[0 0 1 0 1 0 0 0 1 0]
[1 0 0 1 0 0 0 0 0 1]
[1 0 0 0 0 0 0 1 1 0]
[0 1 0 0 0 0 0 0 1 1]
[0 0 1 0 0 1 0 0 0 1]
[0 0 0 1 0 1 1 0 0 0]
[0 0 0 0 1 0 1 1 0 0]
>>> G = Graph(M)
>>> H = MatchingCoveredGraph(G)
>>> H == G
True
>>> sorted(H.get_matching())
[(0, 5, None), (1, 6, None), (2, 7, None), (3, 8, None), (4, 9, None)]

>>> M = Matrix([(-Integer(1), Integer(0), Integer(0), Integer(0), Integer(1),␣
↪Integer(0), Integer(0), Integer(0), Integer(0), Integer(0),-Integer(1),␣
↪Integer(0), Integer(0), Integer(0), Integer(0)),
...              ( Integer(1),-Integer(1), Integer(0), Integer(0), Integer(0),␣
↪Integer(0), Integer(0), Integer(0), Integer(0), Integer(0), Integer(0),-
↪Integer(1), Integer(0), Integer(0), Integer(0)),
...              ( Integer(0), Integer(1),-Integer(1), Integer(0), Integer(0),␣
↪Integer(0), Integer(0), Integer(0), Integer(0), Integer(0), Integer(0),␣
↪Integer(0),-Integer(1), Integer(0), Integer(0)),
...              ( Integer(0), Integer(0), Integer(1),-Integer(1), Integer(0),␣
↪Integer(0), Integer(0), Integer(0), Integer(0), Integer(0), Integer(0),␣
↪Integer(0), Integer(0),-Integer(1), Integer(0)),
...              ( Integer(0), Integer(0), Integer(0), Integer(1),-Integer(1),␣
↪Integer(0), Integer(0), Integer(0), Integer(0), Integer(0), Integer(0),␣
↪Integer(0), Integer(0), Integer(0),-Integer(1)),
...              ( Integer(0), Integer(0), Integer(0), Integer(0), Integer(0),-
↪Integer(1), Integer(0), Integer(0), Integer(0), Integer(1), Integer(1),␣
↪Integer(0), Integer(0), Integer(0), Integer(0)),
...              ( Integer(0), Integer(0), Integer(0), Integer(0), Integer(0),␣
↪Integer(0), Integer(0), Integer(1),-Integer(1), Integer(0), Integer(0),␣
↪Integer(1), Integer(0), Integer(0), Integer(0)),
```

(continues on next page)

(continued from previous page)

```
...              ( Integer(0), Integer(0), Integer(0), Integer(0), Integer(0),␣
↪Integer(1),-Integer(1), Integer(0), Integer(0), Integer(0), Integer(0),␣
↪Integer(0), Integer(1), Integer(0), Integer(0)),
...              ( Integer(0), Integer(0), Integer(0), Integer(0), Integer(0),␣
↪Integer(0), Integer(0), Integer(0), Integer(1),-Integer(1), Integer(0),␣
↪Integer(0), Integer(0), Integer(1), Integer(0)),
...              ( Integer(0), Integer(0), Integer(0), Integer(0), Integer(0),␣
↪Integer(0), Integer(1),-Integer(1), Integer(0), Integer(0), Integer(0),␣
↪Integer(0), Integer(0), Integer(0), Integer(1))])
>>> M
[-1  0  0  0  1  0  0  0  0  0 -1  0  0  0  0]
[ 1 -1  0  0  0  0  0  0  0  0  0 -1  0  0  0]
[ 0  1 -1  0  0  0  0  0  0  0  0  0 -1  0  0]
[ 0  0  1 -1  0  0  0  0  0  0  0  0  0 -1  0]
[ 0  0  0  1 -1  0  0  0  0  0  0  0  0  0 -1]
[ 0  0  0  0  0 -1  0  0  0  1  1  0  0  0  0]
[ 0  0  0  0  0  0  0  1 -1  0  0  1  0  0  0]
[ 0  0  0  0  0  1 -1  0  0  0  0  0  1  0  0]
[ 0  0  0  0  0  0  0  0  1 -1  0  0  0  1  0]
[ 0  0  0  0  0  0  1 -1  0  0  0  0  0  0  1]
>>> G = Graph(M)
>>> H = MatchingCoveredGraph(G)
>>> H == G
True
>>> sorted(H.get_matching())
[(0, 5, None), (1, 6, None), (2, 7, None), (3, 8, None), (4, 9, None)]

>>> G = Graph([(Integer(0), Integer(1)), (Integer(0), Integer(3)), (Integer(0),␣
↪Integer(4)), (Integer(1), Integer(2)), (Integer(1), Integer(5)), (Integer(2),␣
↪Integer(3)),
...             (Integer(2), Integer(6)), (Integer(3), Integer(7)), (Integer(4),␣
↪Integer(5)), (Integer(4), Integer(7)), (Integer(5), Integer(6)), (Integer(6),␣
↪Integer(7))])
>>> H = MatchingCoveredGraph(G)
>>> H == G
True
>>> sorted(H.get_matching())
[(0, 4, None), (1, 5, None), (2, 6, None), (3, 7, None)]

>>> # optional - python_igraph
>>> import igraph
>>> G = Graph(igraph.Graph([(Integer(0), Integer(1)), (Integer(0), Integer(3)),␣
↪(Integer(1), Integer(2)), (Integer(2), Integer(3))]))
>>> H = MatchingCoveredGraph(G)
>>> H
Matching covered graph on 4 vertices
>>> sorted(H.get_matching())
[(0, 3, {}), (1, 2, {})]
```

One may specify a perfect matching:

```
sage: P = graphs.PetersenGraph()
```

(continues on next page)

(continued from previous page)

```
sage: M = P.matching()
sage: sorted(M)
[(0, 5, None), (1, 6, None), (2, 7, None), (3, 8, None), (4, 9, None)]
sage: G = MatchingCoveredGraph(P, matching=M)
sage: G
Matching covered petersen graph: graph on 10 vertices
sage: P == G
True
sage: sorted(G.get_matching())
[(0, 5, None), (1, 6, None), (2, 7, None), (3, 8, None), (4, 9, None)]
sage: sorted(G.get_matching()) == sorted(M)
True

sage: G = graphs.TruncatedBiwheelGraph(14)
sage: M = G.matching()
sage: sorted(M)
[(0, 27, None), (1, 26, None), (2, 3, None), (4, 5, None),
 (6, 7, None), (8, 9, None), (10, 11, None), (12, 13, None),
 (14, 15, None), (16, 17, None), (18, 19, None), (20, 21, None),
 (22, 23, None), (24, 25, None)]
sage: H = MatchingCoveredGraph(G, M)
sage: H
Matching covered truncated biwheel graph: graph on 28 vertices
sage: H == G
True
sage: sorted(H.get_matching()) == sorted(M)
True
```

```
>>> from sage.all import *
>>> P = graphs.PetersenGraph()
>>> M = P.matching()
>>> sorted(M)
[(0, 5, None), (1, 6, None), (2, 7, None), (3, 8, None), (4, 9, None)]
>>> G = MatchingCoveredGraph(P, matching=M)
>>> G
Matching covered petersen graph: graph on 10 vertices
>>> P == G
True
>>> sorted(G.get_matching())
[(0, 5, None), (1, 6, None), (2, 7, None), (3, 8, None), (4, 9, None)]
>>> sorted(G.get_matching()) == sorted(M)
True

>>> G = graphs.TruncatedBiwheelGraph(Integer(14))
>>> M = G.matching()
>>> sorted(M)
[(0, 27, None), (1, 26, None), (2, 3, None), (4, 5, None),
 (6, 7, None), (8, 9, None), (10, 11, None), (12, 13, None),
 (14, 15, None), (16, 17, None), (18, 19, None), (20, 21, None),
 (22, 23, None), (24, 25, None)]
>>> H = MatchingCoveredGraph(G, M)
>>> H
```

(continues on next page)

(continued from previous page)

```
Matching covered truncated biwheel graph: graph on 28 vertices
>>> H == G
True
>>> sorted(H.get_matching()) == sorted(M)
True
```

One may specify some keyword arguments:

```
sage: G = Graph([(0, 1, 5)], {'weighted': True})
sage: kwds = {
....:   'loops': False,
....:   'multiedges': True,
....:   'pos': {0: (0, 0), 1: (1, 1)}
....: }
sage: H = MatchingCoveredGraph(G, **kwds)
sage: H
Matching covered multi-graph on 2 vertices
sage: H.add_edge(0, 1)
sage: H.edges()
[(0, 1, None), (0, 1, 5)]
```

```
>>> from sage.all import *
>>> G = Graph([(Integer(0), Integer(1), Integer(5))], {'weighted': True})
>>> kwds = {
...   'loops': False,
...   'multiedges': True,
...   'pos': {Integer(0): (Integer(0), Integer(0)), Integer(1): (Integer(1),␣
↪Integer(1))}
... }
>>> H = MatchingCoveredGraph(G, **kwds)
>>> H
Matching covered multi-graph on 2 vertices
>>> H.add_edge(Integer(0), Integer(1))
>>> H.edges()
[(0, 1, None), (0, 1, 5)]
```

add_edge(*u*, *v=None*, *label=None*)

Add an edge from vertex `u` to vertex `v`.

> **Note**
>
> This method overwrites the *add_edge()* method to ensure that resultant graph is also matching covered.

INPUT:

The following forms are all accepted:

- G.add_edge(1, 2)
- G.add_edge((1, 2))
- G.add_edges([(1, 2)])
- G.add_edge(1, 2, 'label')

- G.add_edge((1, 2, 'label'))
- G.add_edges([(1, 2, 'label')])

OUTPUT:

- If an edge is provided with a valid format, but addition of the edge leaves the resulting graph not being matching covered, a `ValueError` is returned without any alteration to the existing matching covered graph. If the addition of the edge preserves the property of matching covered, then the graph is updated and nothing is returned.
- If the edge is provided in an invalid format, a `ValueError` is returned.

WARNING:

The following intuitive input results in nonintuitive output, even though the resulting graph behind the intuition might be matching covered:

```
sage: P = graphs.WheelGraph(6)
sage: G = MatchingCoveredGraph(P)
sage: G.add_edge((1, 4), 'label')
Traceback (most recent call last):
...
ValueError: the graph obtained after the addition of edge
(((1, 4), 'label', None)) is not matching covered
sage: G.edges(sort=False)
[(0, 1, None), (0, 2, None), (0, 3, None), (0, 4, None),
 (0, 5, None), (1, 2, None), (1, 5, None), (2, 3, None),
 (3, 4, None), (4, 5, None)]
```

```
>>> from sage.all import *
>>> P = graphs.WheelGraph(Integer(6))
>>> G = MatchingCoveredGraph(P)
>>> G.add_edge((Integer(1), Integer(4)), 'label')
Traceback (most recent call last):
...
ValueError: the graph obtained after the addition of edge
(((1, 4), 'label', None)) is not matching covered
>>> G.edges(sort=False)
[(0, 1, None), (0, 2, None), (0, 3, None), (0, 4, None),
 (0, 5, None), (1, 2, None), (1, 5, None), (2, 3, None),
 (3, 4, None), (4, 5, None)]
```

The key word `label` must be used:

```
sage: W = graphs.WheelGraph(6)
sage: G = MatchingCoveredGraph(W)
sage: G.add_edge((1, 4), label='label')
sage: G.edges(sort=False)  # No alteration to the existing graph
[(0, 1, None), (0, 2, None), (0, 3, None), (0, 4, None),
 (0, 5, None), (1, 2, None), (1, 4, 'label'), (1, 5, None),
 (2, 3, None), (3, 4, None), (4, 5, None)]
```

```
>>> from sage.all import *
>>> W = graphs.WheelGraph(Integer(6))
>>> G = MatchingCoveredGraph(W)
```

(continues on next page)

(continued from previous page)

```
>>> G.add_edge((Integer(1), Integer(4)), label='label')
>>> G.edges(sort=False)  # No alteration to the existing graph
[(0, 1, None), (0, 2, None), (0, 3, None), (0, 4, None),
 (0, 5, None), (1, 2, None), (1, 4, 'label'), (1, 5, None),
 (2, 3, None), (3, 4, None), (4, 5, None)]
```

An expression, analogous to the syntax mentioned above may be used:

```
sage: S = graphs.StaircaseGraph(4)
sage: G = MatchingCoveredGraph(S)
sage: G.add_edge(0, 5)
sage: G.edges(sort=False)
[(0, 1, None), (0, 3, None), (0, 5, None), (0, 6, None),
 (1, 2, None), (1, 4, None), (2, 5, None), (2, 7, None),
 (3, 4, None), (3, 6, None), (4, 5, None), (5, 7, None),
 (6, 7, None)]
sage: G.add_edge((2, 3))
sage: G.edges(sort=False)
[(0, 1, None), (0, 3, None), (0, 5, None), (0, 6, None),
 (1, 2, None), (1, 4, None), (2, 3, None), (2, 5, None),
 (2, 7, None), (3, 4, None), (3, 6, None), (4, 5, None),
 (5, 7, None), (6, 7, None)]
sage: G.add_edges([(0, 4)])
sage: G.edges(sort=False)
[(0, 1, None), (0, 3, None), (0, 4, None), (0, 5, None),
 (0, 6, None), (1, 2, None), (1, 4, None), (2, 3, None),
 (2, 5, None), (2, 7, None), (3, 4, None), (3, 6, None),
 (4, 5, None), (5, 7, None), (6, 7, None)]
sage: G.add_edge(2, 4, 'label')
sage: G.edges(sort=False)
[(0, 1, None), (0, 3, None), (0, 4, None), (0, 5, None),
 (0, 6, None), (1, 2, None), (1, 4, None), (2, 3, None),
 (2, 4, 'label'), (2, 5, None), (2, 7, None), (3, 4, None),
 (3, 6, None), (4, 5, None), (5, 7, None), (6, 7, None)]
sage: G.add_edge((4, 6, 'label'))
sage: G.edges(sort=False)
[(0, 1, None), (0, 3, None), (0, 4, None), (0, 5, None),
 (0, 6, None), (1, 2, None), (1, 4, None), (2, 3, None),
 (2, 4, 'label'), (2, 5, None), (2, 7, None), (3, 4, None),
 (3, 6, None), (4, 5, None), (4, 6, 'label'), (5, 7, None),
 (6, 7, None)]
sage: G.add_edges([(4, 7, 'label')])
sage: G.edges(sort=False)
[(0, 1, None), (0, 3, None), (0, 4, None), (0, 5, None),
 (0, 6, None), (1, 2, None), (1, 4, None), (2, 3, None),
 (2, 4, 'label'), (2, 5, None), (2, 7, None), (3, 4, None),
 (3, 6, None), (4, 5, None), (4, 6, 'label'), (4, 7, 'label'),
 (5, 7, None), (6, 7, None)]
```

```
>>> from sage.all import *
>>> S = graphs.StaircaseGraph(Integer(4))
>>> G = MatchingCoveredGraph(S)
```

(continues on next page)

(continued from previous page)

```
>>> G.add_edge(Integer(0), Integer(5))
>>> G.edges(sort=False)
[(0, 1, None), (0, 3, None), (0, 5, None), (0, 6, None),
 (1, 2, None), (1, 4, None), (2, 5, None), (2, 7, None),
 (3, 4, None), (3, 6, None), (4, 5, None), (5, 7, None),
 (6, 7, None)]
>>> G.add_edge((Integer(2), Integer(3)))
>>> G.edges(sort=False)
[(0, 1, None), (0, 3, None), (0, 5, None), (0, 6, None),
 (1, 2, None), (1, 4, None), (2, 3, None), (2, 5, None),
 (2, 7, None), (3, 4, None), (3, 6, None), (4, 5, None),
 (5, 7, None), (6, 7, None)]
>>> G.add_edges([(Integer(0), Integer(4))])
>>> G.edges(sort=False)
[(0, 1, None), (0, 3, None), (0, 4, None), (0, 5, None),
 (0, 6, None), (1, 2, None), (1, 4, None), (2, 3, None),
 (2, 5, None), (2, 7, None), (3, 4, None), (3, 6, None),
 (4, 5, None), (5, 7, None), (6, 7, None)]
>>> G.add_edge(Integer(2), Integer(4), 'label')
>>> G.edges(sort=False)
[(0, 1, None), (0, 3, None), (0, 4, None), (0, 5, None),
 (0, 6, None), (1, 2, None), (1, 4, None), (2, 3, None),
 (2, 4, 'label'), (2, 5, None), (2, 7, None), (3, 4, None),
 (3, 6, None), (4, 5, None), (5, 7, None), (6, 7, None)]
>>> G.add_edge((Integer(4), Integer(6), 'label'))
>>> G.edges(sort=False)
[(0, 1, None), (0, 3, None), (0, 4, None), (0, 5, None),
 (0, 6, None), (1, 2, None), (1, 4, None), (2, 3, None),
 (2, 4, 'label'), (2, 5, None), (2, 7, None), (3, 4, None),
 (3, 6, None), (4, 5, None), (4, 6, 'label'), (5, 7, None),
 (6, 7, None)]
>>> G.add_edges([(Integer(4), Integer(7), 'label')])
>>> G.edges(sort=False)
[(0, 1, None), (0, 3, None), (0, 4, None), (0, 5, None),
 (0, 6, None), (1, 2, None), (1, 4, None), (2, 3, None),
 (2, 4, 'label'), (2, 5, None), (2, 7, None), (3, 4, None),
 (3, 6, None), (4, 5, None), (4, 6, 'label'), (4, 7, 'label'),
 (5, 7, None), (6, 7, None)]
```

Note that the `weight` of the edge shall be input as the `label`:

```
sage: G.add_edge((1, 3), label=5)
sage: G.edges()
[(0, 1, None), (0, 3, None), (0, 4, None), (0, 5, None),
 (0, 6, None), (1, 2, None), (1, 3, 5), (1, 4, None),
 (2, 3, None), (2, 4, 'label'), (2, 5, None), (2, 7, None),
 (3, 4, None), (3, 6, None), (4, 5, None), (4, 6, 'label'),
 (4, 7, 'label'), (5, 7, None), (6, 7, None)]
sage: G.add_edge((2, 4, 6), label=6)
Traceback (most recent call last):
...
ValueError: the graph obtained after the addition of edge
```

(continues on next page)

(continued from previous page)

```
(((2, 4, 6), None, 6)) is not matching covered
```

```
>>> from sage.all import *
>>> G.add_edge((Integer(1), Integer(3)), label=Integer(5))
>>> G.edges()
[(0, 1, None), (0, 3, None), (0, 4, None), (0, 5, None),
 (0, 6, None), (1, 2, None), (1, 3, 5), (1, 4, None),
 (2, 3, None), (2, 4, 'label'), (2, 5, None), (2, 7, None),
 (3, 4, None), (3, 6, None), (4, 5, None), (4, 6, 'label'),
 (4, 7, 'label'), (5, 7, None), (6, 7, None)]
>>> G.add_edge((Integer(2), Integer(4), Integer(6)), label=Integer(6))
Traceback (most recent call last):
...
ValueError: the graph obtained after the addition of edge
(((2, 4, 6), None, 6)) is not matching covered
```

Vertex name cannot be `None`, so:

```
sage: W = graphs.WheelGraph(6)
sage: H = MatchingCoveredGraph(W)
sage: H.add_edge(None, 1)
Traceback (most recent call last):
...
ValueError: the graph obtained after the addition of edge
((None, 1, None)) is not matching covered
sage: H.edges(sort=False)  # No alteration to the existing graph
[(0, 1, None), (0, 2, None), (0, 3, None), (0, 4, None),
 (0, 5, None), (1, 2, None), (1, 5, None), (2, 3, None),
 (3, 4, None), (4, 5, None)]
sage: H.add_edge(None, None)
Traceback (most recent call last):
...
ValueError: the graph obtained after the addition of edge
((None, None, None)) is not matching covered
sage: H.edges(sort=False)  # No alteration to the existing graph
[(0, 1, None), (0, 2, None), (0, 3, None), (0, 4, None),
 (0, 5, None), (1, 2, None), (1, 5, None), (2, 3, None),
 (3, 4, None), (4, 5, None)]
```

```
>>> from sage.all import *
>>> W = graphs.WheelGraph(Integer(6))
>>> H = MatchingCoveredGraph(W)
>>> H.add_edge(None, Integer(1))
Traceback (most recent call last):
...
ValueError: the graph obtained after the addition of edge
((None, 1, None)) is not matching covered
>>> H.edges(sort=False)  # No alteration to the existing graph
[(0, 1, None), (0, 2, None), (0, 3, None), (0, 4, None),
 (0, 5, None), (1, 2, None), (1, 5, None), (2, 3, None),
 (3, 4, None), (4, 5, None)]
>>> H.add_edge(None, None)
```

(continues on next page)

(continued from previous page)

```
Traceback (most recent call last):
...
ValueError: the graph obtained after the addition of edge
((None, None, None)) is not matching covered
>>> H.edges(sort=False)  # No alteration to the existing graph
[(0, 1, None), (0, 2, None), (0, 3, None), (0, 4, None),
 (0, 5, None), (1, 2, None), (1, 5, None), (2, 3, None),
 (3, 4, None), (4, 5, None)]
```

EXAMPLES:

Adding an already existing edge:

```
sage: P = graphs.PetersenGraph()
sage: G = MatchingCoveredGraph(P)
sage: G.add_edge(next(G.edge_iterator()))
sage: P == G
True
sage: G.size()
15
sage: G.allow_multiple_edges(True)
sage: G.add_edge(0, 1)
sage: G.size()
16
```

```
>>> from sage.all import *
>>> P = graphs.PetersenGraph()
>>> G = MatchingCoveredGraph(P)
>>> G.add_edge(next(G.edge_iterator()))
>>> P == G
True
>>> G.size()
15
>>> G.allow_multiple_edges(True)
>>> G.add_edge(Integer(0), Integer(1))
>>> G.size()
16
```

Adding an edge such that the resulting graph is matching covered:

```
sage: P = graphs.PetersenGraph()
sage: G = MatchingCoveredGraph(P)
sage: G.add_edge(1, 4)
sage: G.edges(sort=False)
[(0, 1, None), (0, 4, None), (0, 5, None), (1, 2, None),
 (1, 4, None), (1, 6, None), (2, 3, None), (2, 7, None),
 (3, 4, None), (3, 8, None), (4, 9, None), (5, 7, None),
 (5, 8, None), (6, 8, None), (6, 9, None), (7, 9, None)]
```

```
>>> from sage.all import *
>>> P = graphs.PetersenGraph()
>>> G = MatchingCoveredGraph(P)
```

(continues on next page)

(continued from previous page)

```
>>> G.add_edge(Integer(1), Integer(4))
>>> G.edges(sort=False)
[(0, 1, None), (0, 4, None), (0, 5, None), (1, 2, None),
 (1, 4, None), (1, 6, None), (2, 3, None), (2, 7, None),
 (3, 4, None), (3, 8, None), (4, 9, None), (5, 7, None),
 (5, 8, None), (6, 8, None), (6, 9, None), (7, 9, None)]
```

Adding an edge with both the incident vertices being existent such that the resulting graph is not matching covered:

```
sage: C = graphs.CycleGraph(4)
sage: G = MatchingCoveredGraph(C)
sage: G.add_edge(0, 2)
Traceback (most recent call last):
...
ValueError: the graph obtained after the addition of edge
((0, 2, None)) is not matching covered
sage: G.edges(sort=False) # No alteration to the existing graph
[(0, 1, None), (0, 3, None), (1, 2, None), (2, 3, None)]
```

```
>>> from sage.all import *
>>> C = graphs.CycleGraph(Integer(4))
>>> G = MatchingCoveredGraph(C)
>>> G.add_edge(Integer(0), Integer(2))
Traceback (most recent call last):
...
ValueError: the graph obtained after the addition of edge
((0, 2, None)) is not matching covered
>>> G.edges(sort=False) # No alteration to the existing graph
[(0, 1, None), (0, 3, None), (1, 2, None), (2, 3, None)]
```

Adding an edge with exactly one incident vertex that is nonexistent throws a `ValueError` exception, as the resulting graph would have an odd order:

```
sage: C = graphs.CycleGraph(4)
sage: G = MatchingCoveredGraph(C)
sage: G.add_edge(0, 4)
Traceback (most recent call last):
...
ValueError: the graph obtained after the addition of edge
((0, 4, None)) is not matching covered
sage: G.edges(sort=False) # No alteration to the existing graph
[(0, 1, None), (0, 3, None), (1, 2, None), (2, 3, None)]
```

```
>>> from sage.all import *
>>> C = graphs.CycleGraph(Integer(4))
>>> G = MatchingCoveredGraph(C)
>>> G.add_edge(Integer(0), Integer(4))
Traceback (most recent call last):
...
ValueError: the graph obtained after the addition of edge
((0, 4, None)) is not matching covered
```

(continues on next page)

(continued from previous page)

```
>>> G.edges(sort=False) # No alteration to the existing graph
[(0, 1, None), (0, 3, None), (1, 2, None), (2, 3, None)]
```

Adding an edge with both the incident vertices that is nonexistent throws a `ValueError` exception, as the resulting graph would have been disconnected:

```
sage: C = graphs.CycleGraph(4)
sage: G = MatchingCoveredGraph(C)
sage: G.add_edge(4, 5)
Traceback (most recent call last):
...
ValueError: the graph obtained after the addition of edge
((4, 5, None)) is not matching covered
sage: G.edges(sort=False) # No alteration to the existing graph
[(0, 1, None), (0, 3, None), (1, 2, None), (2, 3, None)]
```

```
>>> from sage.all import *
>>> C = graphs.CycleGraph(Integer(4))
>>> G = MatchingCoveredGraph(C)
>>> G.add_edge(Integer(4), Integer(5))
Traceback (most recent call last):
...
ValueError: the graph obtained after the addition of edge
((4, 5, None)) is not matching covered
>>> G.edges(sort=False) # No alteration to the existing graph
[(0, 1, None), (0, 3, None), (1, 2, None), (2, 3, None)]
```

Adding a self-loop:

```
sage: H = graphs.HeawoodGraph()
sage: G = MatchingCoveredGraph(H)
sage: v = next(G.vertex_iterator())
sage: G.add_edge(v, v)
Traceback (most recent call last):
...
ValueError: loops are not allowed in matching covered graphs
```

```
>>> from sage.all import *
>>> H = graphs.HeawoodGraph()
>>> G = MatchingCoveredGraph(H)
>>> v = next(G.vertex_iterator())
>>> G.add_edge(v, v)
Traceback (most recent call last):
...
ValueError: loops are not allowed in matching covered graphs
```

add_edges(*edges*, *loops=False*)

Add edges from an iterable container.

Note

This method overwrites the *add_edges()* method to ensure that resultant graph is also matching covered.

INPUT:

- `edges` – an iterable of edges, given either as `(u, v)` or `(u, v, 'label')`. If an edge is provided in the format `(u, v)`, the label is set to `None`.
- `loops` – boolean (default: `False`); note that this shall always be set to either `False` or `None` (since matching covered graphs are free of loops), in which case all the loops `(v, v, 'label')` are removed from the iterator. If `loops` is set to `True`, a `ValueError` is thrown.
- Please note that all the loops present in the iterator are ignored, provided that `loops` is set to `False` or `None`.

OUTPUT:

- If `loops` is set to `True`, a `ValueError` is returned.
- If `edges` is provided with a valid format, but addition of the edges leave the resulting graph not being matching covered, a `ValueError` is returned without any alteration to the existing matching covered graph. If the addition of the edges preserves the property of matching covered, then the graph is updated and nothing is returned.
- If `edges` is provided in an invalid format, a `ValueError` is returned.

EXAMPLES:

Providing with an empty list of edges:

```
sage: C = graphs.CycleGraph(6)
sage: G = MatchingCoveredGraph(C)
sage: G.add_edges([])
sage: G == C
True
```

```
>>> from sage.all import *
>>> C = graphs.CycleGraph(Integer(6))
>>> G = MatchingCoveredGraph(C)
>>> G.add_edges([])
>>> G == C
True
```

Adding some edges, the incident vertices of each of which are existent, such that the resulting graph is matching covered:

```
sage: S = graphs.StaircaseGraph(4)
sage: G = MatchingCoveredGraph(S)
sage: F = [(0, 4), (2, 4), (4, 6), (4, 7)]
sage: G.add_edges(F)
sage: G.edges(sort=True, sort_vertices=True)
[(0, 1, None), (0, 3, None), (0, 4, None), (0, 6, None),
 (1, 2, None), (1, 4, None), (2, 4, None), (2, 5, None),
 (2, 7, None), (3, 4, None), (3, 6, None), (4, 5, None),
 (4, 6, None), (4, 7, None), (5, 7, None), (6, 7, None)]
```

```
>>> from sage.all import *
>>> S = graphs.StaircaseGraph(Integer(4))
>>> G = MatchingCoveredGraph(S)
>>> F = [(Integer(0), Integer(4)), (Integer(2), Integer(4)), (Integer(4),␣
↪Integer(6)), (Integer(4), Integer(7))]
>>> G.add_edges(F)
>>> G.edges(sort=True, sort_vertices=True)
[(0, 1, None), (0, 3, None), (0, 4, None), (0, 6, None),
 (1, 2, None), (1, 4, None), (2, 4, None), (2, 5, None),
 (2, 7, None), (3, 4, None), (3, 6, None), (4, 5, None),
 (4, 6, None), (4, 7, None), (5, 7, None), (6, 7, None)]
```

Adding some edges, at least one of the incident vertices of some of which are nonexistent such that the resulting graph is matching covered:

```
sage: C = graphs.CycleGraph(8)
sage: G = MatchingCoveredGraph(C)
sage: F = [(0, 9), (1, 8), (2, 9), (3, 8),
....:      (4, 9), (5, 8), (6, 9), (7, 8)]
sage: G.add_edges(F)
sage: G.edges(sort=True, sort_vertices=True)
[(0, 1, None), (0, 7, None), (0, 9, None), (1, 2, None),
 (1, 8, None), (2, 3, None), (2, 9, None), (3, 4, None),
 (3, 8, None), (4, 5, None), (4, 9, None), (5, 6, None),
 (5, 8, None), (6, 7, None), (6, 9, None), (7, 8, None)]
sage: G.is_isomorphic(graphs.BiwheelGraph(5))
True
```

```
>>> from sage.all import *
>>> C = graphs.CycleGraph(Integer(8))
>>> G = MatchingCoveredGraph(C)
>>> F = [(Integer(0), Integer(9)), (Integer(1), Integer(8)), (Integer(2),␣
↪Integer(9)), (Integer(3), Integer(8)),
...       (Integer(4), Integer(9)), (Integer(5), Integer(8)), (Integer(6),␣
↪Integer(9)), (Integer(7), Integer(8))]
>>> G.add_edges(F)
>>> G.edges(sort=True, sort_vertices=True)
[(0, 1, None), (0, 7, None), (0, 9, None), (1, 2, None),
 (1, 8, None), (2, 3, None), (2, 9, None), (3, 4, None),
 (3, 8, None), (4, 5, None), (4, 9, None), (5, 6, None),
 (5, 8, None), (6, 7, None), (6, 9, None), (7, 8, None)]
>>> G.is_isomorphic(graphs.BiwheelGraph(Integer(5)))
True
```

Adding a removable double ear to a matching covered graph:

```
sage: H = graphs.HexahedralGraph()
sage: G = MatchingCoveredGraph(H)
sage: F = {(0, 8, None), (1, 10), (4, 11, 'label'),
....:      (5, 9), (8, 9), (10, 11)}
sage: G.add_edges(F)
sage: G.edges(sort=True, sort_vertices=True)
[(0, 1, None), (0, 3, None), (0, 4, None), (0, 8, None),
```

(continues on next page)

(continued from previous page)

```
 (1, 2, None), (1, 5, None), (1, 10, None), (2, 3, None),
 (2, 6, None), (3, 7, None), (4, 5, None), (4, 7, None),
 (4, 11, 'label'), (5, 6, None), (5, 9, None), (6, 7, None),
 (8, 9, None), (10, 11, None)]
```

```
>>> from sage.all import *
>>> H = graphs.HexahedralGraph()
>>> G = MatchingCoveredGraph(H)
>>> F = {(Integer(0), Integer(8), None), (Integer(1), Integer(10)),␣
↪(Integer(4), Integer(11), 'label'),
...       (Integer(5), Integer(9)), (Integer(8), Integer(9)), (Integer(10),␣
↪Integer(11))}
>>> G.add_edges(F)
>>> G.edges(sort=True, sort_vertices=True)
[(0, 1, None), (0, 3, None), (0, 4, None), (0, 8, None),
 (1, 2, None), (1, 5, None), (1, 10, None), (2, 3, None),
 (2, 6, None), (3, 7, None), (4, 5, None), (4, 7, None),
 (4, 11, 'label'), (5, 6, None), (5, 9, None), (6, 7, None),
 (8, 9, None), (10, 11, None)]
```

Adding some edges, the incident vertices of each of which are existent, such that the resulting graph is NOT matching covered:

```
sage: C = graphs.CycleGraph(6)
sage: G = MatchingCoveredGraph(C)
sage: F = [(0, 2), (3, 5)]
sage: G.add_edges(F)
Traceback (most recent call last):
...
ValueError: the resulting graph after the addition ofthe edges is not␣
↪matching covered
```

```
>>> from sage.all import *
>>> C = graphs.CycleGraph(Integer(6))
>>> G = MatchingCoveredGraph(C)
>>> F = [(Integer(0), Integer(2)), (Integer(3), Integer(5))]
>>> G.add_edges(F)
Traceback (most recent call last):
...
ValueError: the resulting graph after the addition ofthe edges is not␣
↪matching covered
```

Adding some edges, at least one of the incident vertices of some of which are nonexistent such that the resulting graph is NOT matching covered:

```
sage: H = graphs.HexahedralGraph()
sage: G = MatchingCoveredGraph(H)
sage: F = [(3, 8), (6, 9), (8, 9)]
sage: G.add_edges(F)
Traceback (most recent call last):
...
ValueError: the resulting graph after the addition ofthe edges is not␣
```

(continues on next page)

(continued from previous page)

```
↪matching covered
sage: I = [(0, 8), (1, 9)]
sage: G.add_edges(I)
Traceback (most recent call last):
...
ValueError: the resulting graph after the addition ofthe edges is not␣
↪matching covered
sage: J = [(u, 8) for u in range(8)]
sage: G.add_edges(J)
Traceback (most recent call last):
...
ValueError: odd order is not allowed for matching covered graphs
```

```
>>> from sage.all import *
>>> H = graphs.HexahedralGraph()
>>> G = MatchingCoveredGraph(H)
>>> F = [(Integer(3), Integer(8)), (Integer(6), Integer(9)), (Integer(8),␣
↪Integer(9))]
>>> G.add_edges(F)
Traceback (most recent call last):
...
ValueError: the resulting graph after the addition ofthe edges is not␣
↪matching covered
>>> I = [(Integer(0), Integer(8)), (Integer(1), Integer(9))]
>>> G.add_edges(I)
Traceback (most recent call last):
...
ValueError: the resulting graph after the addition ofthe edges is not␣
↪matching covered
>>> J = [(u, Integer(8)) for u in range(Integer(8))]
>>> G.add_edges(J)
Traceback (most recent call last):
...
ValueError: odd order is not allowed for matching covered graphs
```

Setting the parameter `loops` to either `False` or `None`:

```
sage: W = graphs.WheelGraph(6)
sage: G = MatchingCoveredGraph(W)
sage: F = [(0, 0), (1, 3), (2, 4)]
sage: G.add_edges(edges=F, loops=False)
sage: G.edges(sort=True, sort_vertices=True)
[(0, 1, None), (0, 2, None), (0, 3, None), (0, 4, None),
 (0, 5, None), (1, 2, None), (1, 3, None), (1, 5, None),
 (2, 3, None), (2, 4, None), (3, 4, None), (4, 5, None)]
sage: J = [(1, 1), (3, 5)]
sage: G.add_edges(edges=J, loops=True)
Traceback (most recent call last):
...
ValueError: loops are not allowed in matching covered graphs
sage: G.edges(sort=True, sort_vertices=True)
[(0, 1, None), (0, 2, None), (0, 3, None), (0, 4, None),
```

(continues on next page)

(continued from previous page)

```
 (0, 5, None), (1, 2, None), (1, 3, None), (1, 5, None),
 (2, 3, None), (2, 4, None), (3, 4, None), (4, 5, None)]
```

```
>>> from sage.all import *
>>> W = graphs.WheelGraph(Integer(6))
>>> G = MatchingCoveredGraph(W)
>>> F = [(Integer(0), Integer(0)), (Integer(1), Integer(3)), (Integer(2),␣
↪Integer(4))]
>>> G.add_edges(edges=F, loops=False)
>>> G.edges(sort=True, sort_vertices=True)
[(0, 1, None), (0, 2, None), (0, 3, None), (0, 4, None),
 (0, 5, None), (1, 2, None), (1, 3, None), (1, 5, None),
 (2, 3, None), (2, 4, None), (3, 4, None), (4, 5, None)]
>>> J = [(Integer(1), Integer(1)), (Integer(3), Integer(5))]
>>> G.add_edges(edges=J, loops=True)
Traceback (most recent call last):
...
ValueError: loops are not allowed in matching covered graphs
>>> G.edges(sort=True, sort_vertices=True)
[(0, 1, None), (0, 2, None), (0, 3, None), (0, 4, None),
 (0, 5, None), (1, 2, None), (1, 3, None), (1, 5, None),
 (2, 3, None), (2, 4, None), (3, 4, None), (4, 5, None)]
```

Setting the parameter `loops` to `True`:

```
sage: P = graphs.PetersenGraph()
sage: G = MatchingCoveredGraph(P)
sage: F = [(0, 0), (0, 2), (0, 3)]
sage: G.add_edges(edges=F, loops=True)
Traceback (most recent call last):
...
ValueError: loops are not allowed in matching covered graphs
```

```
>>> from sage.all import *
>>> P = graphs.PetersenGraph()
>>> G = MatchingCoveredGraph(P)
>>> F = [(Integer(0), Integer(0)), (Integer(0), Integer(2)), (Integer(0),␣
↪Integer(3))]
>>> G.add_edges(edges=F, loops=True)
Traceback (most recent call last):
...
ValueError: loops are not allowed in matching covered graphs
```

Adding a multiple edge:

```
sage: S = graphs.StaircaseGraph(4)
sage: G = MatchingCoveredGraph(S)
sage: G.allow_multiple_edges(True)
sage: F = [(0, 1, 'label'), (0, 4), (1, 2)]
sage: G.add_edges(F)
sage: G.edges(sort=False)
[(0, 1, None), (0, 1, 'label'), (0, 3, None), (0, 4, None),
```

(continues on next page)

(continued from previous page)

```
 (0, 6, None), (1, 2, None), (1, 2, None), (1, 4, None),
 (2, 5, None), (2, 7, None), (3, 4, None), (3, 6, None),
 (4, 5, None), (5, 7, None), (6, 7, None)]
sage: H = [(0, 1)] * 4
sage: G.add_edges(H)
sage: G.edge_label(0, 1)
[None, None, None, None, None, 'label']
```

```
>>> from sage.all import *
>>> S = graphs.StaircaseGraph(Integer(4))
>>> G = MatchingCoveredGraph(S)
>>> G.allow_multiple_edges(True)
>>> F = [(Integer(0), Integer(1), 'label'), (Integer(0), Integer(4)),␣
↪(Integer(1), Integer(2))]
>>> G.add_edges(F)
>>> G.edges(sort=False)
[(0, 1, None), (0, 1, 'label'), (0, 3, None), (0, 4, None),
 (0, 6, None), (1, 2, None), (1, 2, None), (1, 4, None),
 (2, 5, None), (2, 7, None), (3, 4, None), (3, 6, None),
 (4, 5, None), (5, 7, None), (6, 7, None)]
>>> H = [(Integer(0), Integer(1))] * Integer(4)
>>> G.add_edges(H)
>>> G.edge_label(Integer(0), Integer(1))
[None, None, None, None, None, 'label']
```

add_vertex(*name=None*)

Add a vertex to the (matching covered) graph.

> **Note**
>
> This method overwrites the *add_vertex()* method to ensure that isolated vertices are forbidden in *MatchingCoveredGraph*.

INPUT:

- `name` – an immutable object (default: `None`); when no name is specified (default), then the new vertex will be represented by the least integer not already representing a vertex. `name` must be an immutable object (e.g., an integer, a tuple, etc.).

OUTPUT:

- If `name` specifies an existing vertex, then nothing is done. Otherwise a `ValueError` is returned with no change to the existing (matching covered) graph is returned since matching covered graphs are free of isolated vertices.

EXAMPLES:

Adding an existing vertex:

```
sage: P = graphs.PetersenGraph()
sage: P
Petersen graph: Graph on 10 vertices
sage: G = MatchingCoveredGraph(P)
```

(continues on next page)

(continued from previous page)

```
sage: G
Matching covered petersen graph: graph on 10 vertices
sage: u = next(G.vertex_iterator())
sage: G.add_vertex(u)
sage: G
Matching covered petersen graph: graph on 10 vertices
```

```
>>> from sage.all import *
>>> P = graphs.PetersenGraph()
>>> P
Petersen graph: Graph on 10 vertices
>>> G = MatchingCoveredGraph(P)
>>> G
Matching covered petersen graph: graph on 10 vertices
>>> u = next(G.vertex_iterator())
>>> G.add_vertex(u)
>>> G
Matching covered petersen graph: graph on 10 vertices
```

Adding a new/ non-existing vertex:

```
sage: G.add_vertex()
Traceback (most recent call last):
...
ValueError: isolated vertices are not allowed in matching covered graphs
sage: u = 100
sage: G.add_vertex(u)
Traceback (most recent call last):
...
ValueError: isolated vertices are not allowed in matching covered graphs
```

```
>>> from sage.all import *
>>> G.add_vertex()
Traceback (most recent call last):
...
ValueError: isolated vertices are not allowed in matching covered graphs
>>> u = Integer(100)
>>> G.add_vertex(u)
Traceback (most recent call last):
...
ValueError: isolated vertices are not allowed in matching covered graphs
```

add_vertices(*vertices*)

Add vertices to the (matching covered) graph from an iterable container of vertices.

> **Note**
>
> This method overwrites the *add_vertices()* method to ensure that isolated vertices are forbidden in *MatchingCoveredGraph*.

INPUT:

- `vertices` – iterator container of vertex labels. A new label is created, used and returned in the output list for all `None` values in `vertices`.

OUTPUT:

- If all of the vertices are existing vertices of the (matching covered) graph, then nothing is done; otherwise a `ValueError` is returned with no change to the existing (matching covered) graph since matching covered graphs are free of isolated vertices.

EXAMPLES:

Adding a list of already existing vertices:

```
sage: T = graphs.TruncatedBiwheelGraph(15)
sage: T
Truncated biwheel graph: Graph on 30 vertices
sage: G = MatchingCoveredGraph(T)
sage: G
Matching covered truncated biwheel graph: graph on 30 vertices
sage: S = [0, 1, 2, 3]  # We choose 4 existing vertices
sage: G.add_vertices(S)
sage: G
Matching covered truncated biwheel graph: graph on 30 vertices
```

```
>>> from sage.all import *
>>> T = graphs.TruncatedBiwheelGraph(Integer(15))
>>> T
Truncated biwheel graph: Graph on 30 vertices
>>> G = MatchingCoveredGraph(T)
>>> G
Matching covered truncated biwheel graph: graph on 30 vertices
>>> S = [Integer(0), Integer(1), Integer(2), Integer(3)]  # We choose 4␣
↪existing vertices
>>> G.add_vertices(S)
>>> G
Matching covered truncated biwheel graph: graph on 30 vertices
```

Adding a list of vertices in which at least one is non-existent or `None` or possibly both:

```
sage: T = graphs.CompleteGraph(2)
sage: T
Complete graph: Graph on 2 vertices
sage: G = MatchingCoveredGraph(T)
sage: G
Matching covered complete graph: graph on 2 vertices
sage: S1 = [2, 3, 4]
sage: G.add_vertices(S1)
Traceback (most recent call last):
...
ValueError: isolated vertices are not allowed in matching covered graphs
sage: S2 = [None, None]
sage: G.add_vertices(S2)
Traceback (most recent call last):
...
ValueError: isolated vertices are not allowed in matching covered graphs
```

(continues on next page)

(continued from previous page)

```
sage: S3 = [2, None, None, 5]
sage: G.add_vertices(S3)
Traceback (most recent call last):
...
ValueError: isolated vertices are not allowed in matching covered graphs
```

```
>>> from sage.all import *
>>> T = graphs.CompleteGraph(Integer(2))
>>> T
Complete graph: Graph on 2 vertices
>>> G = MatchingCoveredGraph(T)
>>> G
Matching covered complete graph: graph on 2 vertices
>>> S1 = [Integer(2), Integer(3), Integer(4)]
>>> G.add_vertices(S1)
Traceback (most recent call last):
...
ValueError: isolated vertices are not allowed in matching covered graphs
>>> S2 = [None, None]
>>> G.add_vertices(S2)
Traceback (most recent call last):
...
ValueError: isolated vertices are not allowed in matching covered graphs
>>> S3 = [Integer(2), None, None, Integer(5)]
>>> G.add_vertices(S3)
Traceback (most recent call last):
...
ValueError: isolated vertices are not allowed in matching covered graphs
```

allow_loops(*new*, *check=True*)

Change whether loops are allowed in (matching covered) graphs.

> **Note**
>
> This method overwrites the *allow_loops()* method to ensure that loops are forbidden in *MatchingCoveredGraph*.

INPUT:

- `new` – boolean
- `check` – boolean (default: `True`); whether to remove existing loops from the graph when the new status is `False`. It is an argument in *allow_loops()* method and is not used in this overwritten one.

OUTPUT:

- A `ValueError` is returned with no change to the existing (matching covered) graph if `new` is `True` since a matching covered graph, by definition, is free of self-loops. If `new` is set to `False`, there is no output.

EXAMPLES:

Petersen graph is matching covered:

```
sage: P = graphs.PetersenGraph()
sage: P.is_matching_covered()
True
sage: G = MatchingCoveredGraph(P)
sage: G.allow_loops(True)
Traceback (most recent call last):
...
ValueError: loops are not allowed in matching covered graphs
```

```
>>> from sage.all import *
>>> P = graphs.PetersenGraph()
>>> P.is_matching_covered()
True
>>> G = MatchingCoveredGraph(P)
>>> G.allow_loops(True)
Traceback (most recent call last):
...
ValueError: loops are not allowed in matching covered graphs
```

See also

- *allows_loops()*
- *has_loops()*
- *loop_edges()*
- *loop_vertices()*
- *loops()*
- *number_of_loops()*
- *remove_loops()*

allows_loops()

Return whether loops are permitted in (matching covered) graphs.

Note

This method overwrites the *allows_loops()* method to show that loops are forbidden in *MatchingCoveredGraph*.

OUTPUT:

- A boolean value `False` is returned, since matching covered graphs, by definition, are free of loops.

EXAMPLES:

Petersen graph is matching covered:

```
sage: P = graphs.PetersenGraph()
sage: P.is_matching_covered()
True
```

(continues on next page)

(continued from previous page)

```
sage: G = MatchingCoveredGraph(P)
sage: G.allows_loops()
False
```

```
>>> from sage.all import *
>>> P = graphs.PetersenGraph()
>>> P.is_matching_covered()
True
>>> G = MatchingCoveredGraph(P)
>>> G.allows_loops()
False
```

See also

- *allow_loops()*
- *has_loops()*
- *loop_edges()*
- *loop_vertices()*
- *loops()*
- *number_of_loops()*
- *remove_loops()*

canonical_partition()

Return the canonical partition of the (matching covered) graph.

For a matching covered graph G, a subset B of the vertex set V is a barrier if $|B| = o(G - B)$, where $|B|$ denotes the cardinality of the set B and $o(G - B)$ denotes the number of odd components in the graph $G - B$. And a barrier B is a maximal barrier if C is not a barrier for each C such that $B \subset C \subseteq V$.

Note that in a matching covered graph, each vertex belongs to a unique maximal barrier. The maximal barriers of a matching covered graph partitions its vertex set and the partition of the vertex set of a matching covered graph into its maximal barriers is called as its *canonical partition*.

OUTPUT:

- A list of sets that constitute a (canonical) partition of the vertex set, wherein each set is a (unique) maximal barrier of the (matching covered) graph.

EXAMPLES:

Show the maximal barrier of the graph $K_4 \odot K_{3,3}$:

```
sage: G = Graph([
....:     (0, 2), (0, 3), (0, 4), (1, 2),
....:     (1, 3), (1, 4), (2, 5), (3, 6),
....:     (4, 7), (5, 6), (5, 7), (6, 7)
....: ])
sage: H = MatchingCoveredGraph(G)
sage: H.canonical_partition()
[{0}, {1}, {2, 3, 4}, {5}, {6}, {7}]
```

```
>>> from sage.all import *
>>> G = Graph([
...     (Integer(0), Integer(2)), (Integer(0), Integer(3)), (Integer(0),␣
↪Integer(4)), (Integer(1), Integer(2)),
...     (Integer(1), Integer(3)), (Integer(1), Integer(4)), (Integer(2),␣
↪Integer(5)), (Integer(3), Integer(6)),
...     (Integer(4), Integer(7)), (Integer(5), Integer(6)), (Integer(5),␣
↪Integer(7)), (Integer(6), Integer(7))
... ])
>>> H = MatchingCoveredGraph(G)
>>> H.canonical_partition()
[{0}, {1}, {2, 3, 4}, {5}, {6}, {7}]
```

For a bicritical graph (for instance, the Petersen graph), the canonical partition constitutes of only singleton sets each containing an individual vertex:

```
sage: P = graphs.PetersenGraph()
sage: G = MatchingCoveredGraph(P)
sage: G.canonical_partition()
[{0}, {1}, {2}, {3}, {4}, {5}, {6}, {7}, {8}, {9}]
```

```
>>> from sage.all import *
>>> P = graphs.PetersenGraph()
>>> G = MatchingCoveredGraph(P)
>>> G.canonical_partition()
[{0}, {1}, {2}, {3}, {4}, {5}, {6}, {7}, {8}, {9}]
```

For a bipartite matching covered graph (for instance, the Hexahedral graph), the canonical partition consists of two sets each of which corresponds to the individual color class:

```
sage: H = graphs.HexahedralGraph()
sage: G = MatchingCoveredGraph(H)
sage: G.canonical_partition()
[{0, 2, 5, 7}, {1, 3, 4, 6}]
sage: B = BipartiteGraph(H)
sage: list(B.bipartition()) == G.canonical_partition()
True
```

```
>>> from sage.all import *
>>> H = graphs.HexahedralGraph()
>>> G = MatchingCoveredGraph(H)
>>> G.canonical_partition()
[{0, 2, 5, 7}, {1, 3, 4, 6}]
>>> B = BipartiteGraph(H)
>>> list(B.bipartition()) == G.canonical_partition()
True
```

REFERENCES:

- [LM2024]

See also

- *is_bicritical()*
- *is_matching_covered()*
- *maximal_barrier()*

delete_vertex(*vertex*, *in_order=False*)

Delete a vertex, removing all incident edges.

Note

This method overwrites the *delete_vertex()* method to ensure that an odd order is forbidden in *MatchingCoveredGraph*.

INPUT:

- `vertex` – a vertex that is to be deleted.
- `in_order` – boolean (default: `False`); if `True`, this deletes the i-th vertex in the sorted list of vertices, i.e. `G.vertices(sort=True)[i]`

OUTPUT:

- Deleting a non-existent vertex raises a `ValueError` exception; also a (different) `ValueError` is returned on deleting an existing vertex since matching covered graphs are of even order. In both cases no modifications are made to the existing (matching covered) graph.

EXAMPLES:

Deleting a non-existent vertex:

```
sage: W = graphs.WheelGraph(12)
sage: G = MatchingCoveredGraph(W)
sage: u = 100
sage: G.delete_vertex(u)
Traceback (most recent call last):
...
ValueError: vertex (100) not in the graph
sage: G.delete_vertex(vertex=u, in_order=True)
Traceback (most recent call last):
...
ValueError: vertex (100) not in the graph
```

```
>>> from sage.all import *
>>> W = graphs.WheelGraph(Integer(12))
>>> G = MatchingCoveredGraph(W)
>>> u = Integer(100)
>>> G.delete_vertex(u)
Traceback (most recent call last):
...
ValueError: vertex (100) not in the graph
>>> G.delete_vertex(vertex=u, in_order=True)
Traceback (most recent call last):
```

(continues on next page)

(continued from previous page)

```
...
ValueError: vertex (100) not in the graph
```

Deleting an existing vertex:

```
sage: W = graphs.WheelGraph(12)
sage: G = MatchingCoveredGraph(W)
sage: u = next(G.vertex_iterator())
sage: G.delete_vertex(u)
Traceback (most recent call last):
...
ValueError: odd order is not allowed for matching covered graphs
sage: G.delete_vertex(vertex=u, in_order=True)
Traceback (most recent call last):
...
ValueError: odd order is not allowed for matching covered graphs
```

```
>>> from sage.all import *
>>> W = graphs.WheelGraph(Integer(12))
>>> G = MatchingCoveredGraph(W)
>>> u = next(G.vertex_iterator())
>>> G.delete_vertex(u)
Traceback (most recent call last):
...
ValueError: odd order is not allowed for matching covered graphs
>>> G.delete_vertex(vertex=u, in_order=True)
Traceback (most recent call last):
...
ValueError: odd order is not allowed for matching covered graphs
```

delete_vertices(*vertices*)

Delete specified vertices form `self`.

This method deletes the vertices from the iterable container `vertices` from `self` along with incident edges. An error is raised if the resulting graph is not matching covered.

Note

This method overwrites the *delete_vertices()* method to ensure that an odd order is forbidden in *MatchingCoveredGraph*.

INPUT:

- `vertices` – a list/ set of vertices that are to be deleted.

OUTPUT:

- Deleting a non-existent vertex will raise a `ValueError` exception, in which case none of the vertices in `vertices` is deleted.
- If all of the vertices in the list/ set provided exist in the graph, but the resulting graph after deletion of all of those is not matching covered, then a `ValueError` exception is raised without any alterations to the existing (matching covered) graph, otherwise the vertices are deleted and nothing is returned.

EXAMPLES:

Providing with an empty list of vertices:

```
sage: C = graphs.CycleGraph(6)
sage: G = MatchingCoveredGraph(C)
sage: G.delete_vertices([])
sage: G == C
True
```

```
>>> from sage.all import *
>>> C = graphs.CycleGraph(Integer(6))
>>> G = MatchingCoveredGraph(C)
>>> G.delete_vertices([])
>>> G == C
True
```

Removing all the existent vertices:

```
sage: M = graphs.MoebiusLadderGraph(10)
sage: G = MatchingCoveredGraph(M)
sage: S = list(G.vertices())
sage: G.delete_vertices(S)
Traceback (most recent call last):
...
ValueError: the resulting graph after the removal of the vertices
is trivial, therefore is not matching covered
```

```
>>> from sage.all import *
>>> M = graphs.MoebiusLadderGraph(Integer(10))
>>> G = MatchingCoveredGraph(M)
>>> S = list(G.vertices())
>>> G.delete_vertices(S)
Traceback (most recent call last):
...
ValueError: the resulting graph after the removal of the vertices
is trivial, therefore is not matching covered
```

Providing with a list of vertices with at least one non-existent vertex:

```
sage: S = graphs.StaircaseGraph(4)
sage: S
Staircase graph: Graph on 8 vertices
sage: G = MatchingCoveredGraph(S)
sage: G
Matching covered staircase graph: graph on 8 vertices
sage: T = list(range(5, 20, 2))
sage: G.delete_vertices(T)
Traceback (most recent call last):
...
ValueError: vertex (9) not in the graph
```

```
>>> from sage.all import *
>>> S = graphs.StaircaseGraph(Integer(4))
```

(continues on next page)

(continued from previous page)

```
>>> S
Staircase graph: Graph on 8 vertices
>>> G = MatchingCoveredGraph(S)
>>> G
Matching covered staircase graph: graph on 8 vertices
>>> T = list(range(Integer(5), Integer(20), Integer(2)))
>>> G.delete_vertices(T)
Traceback (most recent call last):
...
ValueError: vertex (9) not in the graph
```

Removing an odd no. of distinct vertices from a matching covered graph:

```
sage: P = graphs.PetersenGraph()
sage: G = MatchingCoveredGraph(P)
sage: S = [0, 1, 2, 10, 10, 100]
sage: G.delete_vertices(S)
Traceback (most recent call last):
...
ValueError: an odd no. of distinct vertices can not be
removed from a matching covered graph
```

```
>>> from sage.all import *
>>> P = graphs.PetersenGraph()
>>> G = MatchingCoveredGraph(P)
>>> S = [Integer(0), Integer(1), Integer(2), Integer(10), Integer(10),␣
↪Integer(100)]
>>> G.delete_vertices(S)
Traceback (most recent call last):
...
ValueError: an odd no. of distinct vertices can not be
removed from a matching covered graph
```

Providing with a list of existent vertices whose deletion results in a graph which is not matching covered:

```
sage: S = graphs.StaircaseGraph(4)
sage: S
Staircase graph: Graph on 8 vertices
sage: G = MatchingCoveredGraph(S)
sage: G
Matching covered staircase graph: graph on 8 vertices
sage: T = [1, 4]
sage: G.delete_vertices(T)
Traceback (most recent call last):
...
ValueError: the resulting graph after the removal of
the vertices is not matching covered
```

```
>>> from sage.all import *
>>> S = graphs.StaircaseGraph(Integer(4))
>>> S
Staircase graph: Graph on 8 vertices
```

(continues on next page)

(continued from previous page)

```
>>> G = MatchingCoveredGraph(S)
>>> G
Matching covered staircase graph: graph on 8 vertices
>>> T = [Integer(1), Integer(4)]
>>> G.delete_vertices(T)
Traceback (most recent call last):
...
ValueError: the resulting graph after the removal of
the vertices is not matching covered
```

Providing with a list of existent vertices after the deletion of which the resulting graph is still matching covered; note that in the following example, after the deletion of two vertices from a staircase graph, the resulting graph is NOT a staircase graph (see Issue #38768):

```
sage: S = graphs.StaircaseGraph(4)
sage: S
Staircase graph: Graph on 8 vertices
sage: G = MatchingCoveredGraph(S)
sage: G
Matching covered staircase graph: graph on 8 vertices
sage: T = [6, 7]
sage: G.delete_vertices(T)
sage: G  # Matching covered graph on 6 vertices
Matching covered staircase graph: graph on 6 vertices
```

```
>>> from sage.all import *
>>> S = graphs.StaircaseGraph(Integer(4))
>>> S
Staircase graph: Graph on 8 vertices
>>> G = MatchingCoveredGraph(S)
>>> G
Matching covered staircase graph: graph on 8 vertices
>>> T = [Integer(6), Integer(7)]
>>> G.delete_vertices(T)
>>> G  # Matching covered graph on 6 vertices
Matching covered staircase graph: graph on 6 vertices
```

get_matching()

Return an *EdgesView* of `self._matching`.

OUTPUT:

- This method returns *EdgesView* of the edges of a perfect matching of the (matching covered) graph.

EXAMPLES:

If one specifies a perfect matching while initializing the object, the value of `self._matching` is the same matching:

```
sage: P = graphs.PetersenGraph()
sage: M = [(0, 1), (2, 3), (4, 9), (5, 7), (6, 8)]
sage: G = MatchingCoveredGraph(P, M)
sage: sorted(G.get_matching())
[(0, 1), (2, 3), (4, 9), (5, 7), (6, 8)]
```

(continues on next page)

(continued from previous page)

```
sage: M == sorted(G.get_matching())
True
```

```
>>> from sage.all import *
>>> P = graphs.PetersenGraph()
>>> M = [(Integer(0), Integer(1)), (Integer(2), Integer(3)), (Integer(4),␣
↪Integer(9)), (Integer(5), Integer(7)), (Integer(6), Integer(8))]
>>> G = MatchingCoveredGraph(P, M)
>>> sorted(G.get_matching())
[(0, 1), (2, 3), (4, 9), (5, 7), (6, 8)]
>>> M == sorted(G.get_matching())
True
```

If no matching is specified while initializing a matching covered graph, a perfect matching is computed *matching()* and that is captured as `self._matching`:

```
sage: P = graphs.PetersenGraph()
sage: M = P.matching()
sage: G = MatchingCoveredGraph(P)
sage: sorted(G.get_matching())
[(0, 5, None), (1, 6, None), (2, 7, None), (3, 8, None), (4, 9, None)]
sage: sorted(M) == sorted(G.get_matching())
True
```

```
>>> from sage.all import *
>>> P = graphs.PetersenGraph()
>>> M = P.matching()
>>> G = MatchingCoveredGraph(P)
>>> sorted(G.get_matching())
[(0, 5, None), (1, 6, None), (2, 7, None), (3, 8, None), (4, 9, None)]
>>> sorted(M) == sorted(G.get_matching())
True
```

has_loops()

Check whether there are loops in the (matching covered) graph.

Note

This method overwrites the *has_loops()* method in order to return `False` as matching covered graphs are always free of looped edges.

OUTPUT:

- A boolean `False` is returned since matching covered graphs, by definition, are free of self-loops.

EXAMPLES:

A matching covered graph, for instance the Petersen graph, is always free of loops:

```
sage: P = graphs.PetersenGraph()
sage: G = MatchingCoveredGraph(P)
sage: G
```

(continues on next page)

(continued from previous page)

```
Matching covered petersen graph: graph on 10 vertices
sage: G.has_loops()
False
sage: G.allows_loops()
False
sage: G.add_edge(0, 0)
Traceback (most recent call last):
...
ValueError: loops are not allowed in matching covered graphs
```

```
>>> from sage.all import *
>>> P = graphs.PetersenGraph()
>>> G = MatchingCoveredGraph(P)
>>> G
Matching covered petersen graph: graph on 10 vertices
>>> G.has_loops()
False
>>> G.allows_loops()
False
>>> G.add_edge(Integer(0), Integer(0))
Traceback (most recent call last):
...
ValueError: loops are not allowed in matching covered graphs
```

A matching covered graph may support multiple edges, still no loops are allowed:

```
sage: K = graphs.CompleteGraph(2)
sage: G = MatchingCoveredGraph(K)
sage: G.allow_multiple_edges(True)
sage: G
Matching covered complete graph: multi-graph on 2 vertices
sage: G.add_edge(0, 1, 'label')
sage: G.add_edge(0, 0)
Traceback (most recent call last):
...
ValueError: loops are not allowed in matching covered graphs
sage: G.edges(sort=False)
[(0, 1, None), (0, 1, 'label')]
sage: G.allows_loops()
False
sage: G.has_loops()
False
sage: G.allow_loops(True)
Traceback (most recent call last):
...
ValueError: loops are not allowed in matching covered graphs
```

```
>>> from sage.all import *
>>> K = graphs.CompleteGraph(Integer(2))
>>> G = MatchingCoveredGraph(K)
>>> G.allow_multiple_edges(True)
>>> G
```

(continues on next page)

(continued from previous page)

```
Matching covered complete graph: multi-graph on 2 vertices
>>> G.add_edge(Integer(0), Integer(1), 'label')
>>> G.add_edge(Integer(0), Integer(0))
Traceback (most recent call last):
...
ValueError: loops are not allowed in matching covered graphs
>>> G.edges(sort=False)
[(0, 1, None), (0, 1, 'label')]
>>> G.allows_loops()
False
>>> G.has_loops()
False
>>> G.allow_loops(True)
Traceback (most recent call last):
...
ValueError: loops are not allowed in matching covered graphs
```

See also

- *allow_loops()*
- *allows_loops()*
- *loop_edges()*
- *loop_vertices()*
- *loops()*
- *number_of_loops()*
- *remove_loops()*

has_perfect_matching(*G*, *algorithm*, *solver='Edmonds'*, *verbose=None*, *integrality_tolerance=0*)

Check whether the graph has a perfect matching.

Note

This method overwrites the *has_perfect_matching()* method in order to return `True` (provided the input arguments are valid) as matching covered graphs always admit a perfect matching.

INPUT:

- `algorithm` – string (default: `'Edmonds'`)
  - `'Edmonds'` uses Edmonds' algorithm as implemented in NetworkX to find a matching of maximal cardinality, then check whether this cardinality is half the number of vertices of the graph.
  - `'LP_matching'` uses a Linear Program to find a matching of maximal cardinality, then check whether this cardinality is half the number of vertices of the graph.
  - `'LP'` uses a Linear Program formulation of the perfect matching problem: put a binary variable `b[e]` on each edge e, and for each vertex v, require that the sum of the values of the edges incident to v is 1.

- `solver` – string (default: `None`); specifies a Mixed Integer Linear Programming (MILP) solver to be used. If set to `None`, the default one is used. For more information on MILP solvers and which default solver is used, see the method `solve` of the class `MixedIntegerLinearProgram`.
- `verbose` – integer (default: 0); sets the level of verbosity: set to 0 by default, which means quiet (only useful when `algorithm == "LP_matching"` or `algorithm == "LP"`)
- `integrality_tolerance` – float; parameter for use with MILP solvers over an inexact base ring; see `MixedIntegerLinearProgram.get_values()`.

OUTPUT:

- If the input arguments are valid, a boolean (`True`) is returned as a maximum matching of a matching covered graph is always a perfect matching, otherwise a `ValueError` is raised.

EXAMPLES:

Note that regardless of the algorithm (as long as the input arguments are in valid format), the method always returns the boolean `True`:

```
sage: P = graphs.PetersenGraph()
sage: P.has_perfect_matching()  # Calls Graph.has_perfect_matching()
True
sage: G = MatchingCoveredGraph(P)
sage: G.has_perfect_matching()  # Calls MatchingCoveredGraph.has_perfect_
↪matching()
True
sage: W = graphs.WheelGraph(6)
sage: H = MatchingCoveredGraph(W)
sage: H.has_perfect_matching(algorithm='LP_matching')
True
```

```
>>> from sage.all import *
>>> P = graphs.PetersenGraph()
>>> P.has_perfect_matching()  # Calls Graph.has_perfect_matching()
True
>>> G = MatchingCoveredGraph(P)
>>> G.has_perfect_matching()  # Calls MatchingCoveredGraph.has_perfect_
↪matching()
True
>>> W = graphs.WheelGraph(Integer(6))
>>> H = MatchingCoveredGraph(W)
>>> H.has_perfect_matching(algorithm='LP_matching')
True
```

Providing with an algorithm, that is not one of `'Edmonds'`, `'LP_matching'` or `'LP'`:

```
sage: S = graphs.StaircaseGraph(4)
sage: J = MatchingCoveredGraph(S)
sage: J.has_perfect_matching(algorithm='algorithm')
Traceback (most recent call last):
...
ValueError: algorithm must be set to 'Edmonds',
'LP_matching' or 'LP'
```

```
>>> from sage.all import *
>>> S = graphs.StaircaseGraph(Integer(4))
>>> J = MatchingCoveredGraph(S)
>>> J.has_perfect_matching(algorithm='algorithm')
Traceback (most recent call last):
...
ValueError: algorithm must be set to 'Edmonds',
'LP_matching' or 'LP'
```

is_biconnected()

Check whether the (matching covered) graph is biconnected.

A biconnected graph is a connected graph on two or more vertices that is not broken into disconnected pieces by deleting any single vertex. By definition of matching covered graphs, it follows that a graph, that is matching covered, is biconnected.

Observe that K_2 (upto multiple edges) is biconnected. A matching covered graph G, that is not K_2 (upto multiple edges), has at least four vertices and four edges. Consider any two adjacent edges (e and f) of G. Take a perfect matching M of G containing the edge e and a different perfect matching N of G containing the edge f. Observe that the symmetric difference of M and N has a cycle containing both of the edges e and f. Thus, each edge of G is contained in some cycle of G. Therefore, G is biconnected.

Note

This method overwrites the *is_biconnected()* method in order to return `True` as matching covered graphs are biconnected.

EXAMPLES:

The complete graph on two vertices is the smallest biconnected graph that is matching covered:

```
sage: K2 = graphs.CompleteGraph(2)
sage: G = MatchingCoveredGraph(K2)
sage: G.is_biconnected()
True
```

```
>>> from sage.all import *
>>> K2 = graphs.CompleteGraph(Integer(2))
>>> G = MatchingCoveredGraph(K2)
>>> G.is_biconnected()
True
```

Petersen graph is matching covered and biconnected:

```
sage: P = graphs.PetersenGraph()
sage: G = MatchingCoveredGraph(P)
sage: G.is_biconnected()
True
```

```
>>> from sage.all import *
>>> P = graphs.PetersenGraph()
>>> G = MatchingCoveredGraph(P)
```

(continues on next page)

(continued from previous page)

```
>>> G.is_biconnected()
True
```

See also

- *is_connected()*

is_brace(*coNP_certificate=False*)

Check if the (matching covered) graph is a brace.

A matching covered graph which is free of nontrivial tight cuts is called a *brace* if it is bipartite. Let $G := (A \cup B, E)$ be a bipartite matching covered graph on six or more vertices. The following statements are equivalent [LM2024]:

1. G is a brace (aka free of nontrivial tight cuts).
2. $G - a_1 - a_2 - b_1 - b_2$ has a perfect matching for any two distinct vertices a_1 and a_2 in A and any two distinct vertices b_1 and b_2 in B.
3. G is two extendable (any two nonadjacent distinct edges can be extended to some perfect matching of G).
4. $|N(X)| \geq |X|+2$, for all $X \subset A$ such that $0 < |X| < |A|-1$, where $N(S) := \{b \mid (a, b) \in E \wedge a \in S\}$ is called the neighboring set of S.
5. $G - a - b$ is matching covered, for some perfect matching M of G and for each edge ab in M.

We shall be using the 5th characterization mentioned above in order to determine whether the provided bipartite matching covered graph is a brace or not using *M*-alternating tree search [LZ2001].

INPUT:

- `coNP_certificate` – boolean (default: `False`)

OUTPUT:

- If the input matching covered graph is not bipartite, a `ValueError` is returned.
- If the input bipartite matching covered graph is a brace, a boolean `True` is returned if `coNP_certificate` is set to `False` otherwise a 5-tuple `(True, None, None, None, None)` is returned.
- If the input bipartite matching covered graph is not a brace, a boolean `False` is returned if `coNP_certificate` is set to `False` otherwise a 5-tuple of

  1. a boolean `False`,
  2. a list of edges constituting a nontrivial tight cut (which is a nontrivial barrier cut)
  3. a set of vertices of one of the shores of the nontrivial tight cut
  4. a string 'nontrivial tight cut'
  5. a set of vertices showing the respective barrier

  is returned.

EXAMPLES:

The complete graph on two vertices K_2 is the smallest brace:

```
sage: K = graphs.CompleteGraph(2)
sage: G = MatchingCoveredGraph(K)
sage: G.is_brace()
True
```

```
>>> from sage.all import *
>>> K = graphs.CompleteGraph(Integer(2))
>>> G = MatchingCoveredGraph(K)
>>> G.is_brace()
True
```

The cycle graph on four vertices C_4 is a brace:

```
sage: C = graphs.CycleGraph(4)
sage: G = MatchingCoveredGraph(C)
sage: G.is_brace()
True
```

```
>>> from sage.all import *
>>> C = graphs.CycleGraph(Integer(4))
>>> G = MatchingCoveredGraph(C)
>>> G.is_brace()
True
```

Each graph that is isomorphic to a biwheel is a brace:

```
sage: B = graphs.BiwheelGraph(15)
sage: G = MatchingCoveredGraph(B)
sage: G.is_brace()
True
```

```
>>> from sage.all import *
>>> B = graphs.BiwheelGraph(Integer(15))
>>> G = MatchingCoveredGraph(B)
>>> G.is_brace()
True
```

A circular ladder graph of order eight or more on $2n$ vertices for an even n is a brace:

```
sage: n = 10
sage: CL = graphs.CircularLadderGraph(n)
sage: G = MatchingCoveredGraph(CL)
sage: G.is_brace()
True
```

```
>>> from sage.all import *
>>> n = Integer(10)
>>> CL = graphs.CircularLadderGraph(n)
>>> G = MatchingCoveredGraph(CL)
>>> G.is_brace()
True
```

A moebius ladder graph of order six or more on $2n$ vertices for an odd n is a brace:

```
sage: n = 11
sage: ML = graphs.MoebiusLadderGraph(n)
sage: G = MatchingCoveredGraph(ML)
sage: G.is_brace()
True
```

```
>>> from sage.all import *
>>> n = Integer(11)
>>> ML = graphs.MoebiusLadderGraph(n)
>>> G = MatchingCoveredGraph(ML)
>>> G.is_brace()
True
```

Note that the union of the above mentioned four families of braces, that are:

1. the biwheel graph `BiwheelGraph(n)`,
2. the circular ladder graph `CircularLadderGraph(n)` for even `n`,
3. the moebius ladder graph `MoebiusLadderGraph(n)` for odd `n`,

is referred to as the *McCuaig family of braces*.

The only simple brace of order six is the complete graph of the same order, that is $K_{3,3}$:

```
sage: L = list(graphs(6,
....:          lambda G: G.size() <= 15 and
....:                    G.is_bipartite())
....: )
sage: L = list(G for G in L if G.is_connected() and
....:                          G.is_matching_covered()
....: )
sage: M = list(MatchingCoveredGraph(G) for G in L)
sage: B = list(G for G in M if G.is_brace())
sage: K = graphs.CompleteBipartiteGraph(3, 3)
sage: G = MatchingCoveredGraph(K)
sage: next(iter(B)).is_isomorphic(G)
True
```

```
>>> from sage.all import *
>>> L = list(graphs(Integer(6),
...          lambda G: G.size() <= Integer(15) and
...                    G.is_bipartite())
... )
>>> L = list(G for G in L if G.is_connected() and
...                          G.is_matching_covered()
... )
>>> M = list(MatchingCoveredGraph(G) for G in L)
>>> B = list(G for G in M if G.is_brace())
>>> K = graphs.CompleteBipartiteGraph(Integer(3), Integer(3))
>>> G = MatchingCoveredGraph(K)
>>> next(iter(B)).is_isomorphic(G)
True
```

The nonplanar $K_{3,3}$-free brace Heawood graph is the unique cubic graph of girth six with the fewest number of vertices (that is 14). Note that by $K_{3,3}$-free, it shows that the Heawood graph does not contain a subgraph

that is isomorphic to a graph obtained by bisubdivision of $K_{3,3}$:

```
sage: K = graphs.CompleteBipartiteGraph(3, 3)
sage: J = graphs.HeawoodGraph()
sage: H = MatchingCoveredGraph(J)
sage: H.is_brace() and not H.is_planar() and \
....: H.is_regular(k=3) and H.girth() == 6
True
```

```
>>> from sage.all import *
>>> K = graphs.CompleteBipartiteGraph(Integer(3), Integer(3))
>>> J = graphs.HeawoodGraph()
>>> H = MatchingCoveredGraph(J)
>>> H.is_brace() and not H.is_planar() and H.is_regular(k=Integer(3)) and H.
↪girth() == Integer(6)
True
```

Braces of order six or more are 3-connected:

```
sage: H = graphs.HexahedralGraph()
sage: G = MatchingCoveredGraph(H)
sage: G.is_brace() and G.is_triconnected()
True
```

```
>>> from sage.all import *
>>> H = graphs.HexahedralGraph()
>>> G = MatchingCoveredGraph(H)
>>> G.is_brace() and G.is_triconnected()
True
```

Braces of order four or more are 2-extendable:

```
sage: H = graphs.EllinghamHorton54Graph()
sage: G = MatchingCoveredGraph(H)
sage: G.is_brace()
True
sage: e = next(G.edge_iterator(labels=False)); f = None
sage: for f in G.edge_iterator(labels=False):
....:     if not (set(e) & set(f)):
....:         break
sage: S = [u for x in [e, f] for u in set(x)]
sage: J = H.copy(); J.delete_vertices(S)
sage: M = Graph(J.matching())
sage: M.add_edges([e, f])
sage: if all(d == 1 for d in M.degree()) and \
....:    G.order() == M.order() and \
....:    G.order() == 2*M.size():
....:      print(f'graph {G} is 2-extendable')
graph Ellingham-Horton 54-graph is 2-extendable
```

```
>>> from sage.all import *
>>> H = graphs.EllinghamHorton54Graph()
>>> G = MatchingCoveredGraph(H)
```

(continues on next page)

(continued from previous page)

```
>>> G.is_brace()
True
>>> e = next(G.edge_iterator(labels=False)); f = None
>>> for f in G.edge_iterator(labels=False):
...     if not (set(e) & set(f)):
...         break
>>> S = [u for x in [e, f] for u in set(x)]
>>> J = H.copy(); J.delete_vertices(S)
>>> M = Graph(J.matching())
>>> M.add_edges([e, f])
>>> if all(d == Integer(1) for d in M.degree()) and    G.order() == M.order()␣
↪and    G.order() == Integer(2)*M.size():
...     print(f'graph {G} is 2-extendable')
graph Ellingham-Horton 54-graph is 2-extendable
```

Every edge in a brace of order at least six is removable:

```
sage: H = graphs.CircularLadderGraph(8)
sage: G = MatchingCoveredGraph(H)
sage: # len(G.removble_edges()) == G.size()
# True
```

```
>>> from sage.all import *
>>> H = graphs.CircularLadderGraph(Integer(8))
>>> G = MatchingCoveredGraph(H)
>>> # len(G.removble_edges()) == G.size()
# True
```

Every brace of order eight has the hexahedral graph as a spanning subgraph:

```
sage: H = graphs.HexahedralGraph()
sage: L = list(graphs(8,
....:          lambda G: G.size() <= 28 and
....:                    G.is_bipartite())
....: )
sage: L = list(G for G in L if G.is_connected() and
....:                          G.is_matching_covered()
....: )
sage: M = list(MatchingCoveredGraph(G) for G in L)
sage: B = list(G for G in M if G.is_brace())
sage: C = list(G for G in M if Graph(G).subgraph_search(H) is not None)
sage: B == C
True
```

```
>>> from sage.all import *
>>> H = graphs.HexahedralGraph()
>>> L = list(graphs(Integer(8),
...          lambda G: G.size() <= Integer(28) and
...                    G.is_bipartite())
... )
>>> L = list(G for G in L if G.is_connected() and
...                          G.is_matching_covered()
```

(continues on next page)

(continued from previous page)

```
... )
>>> M = list(MatchingCoveredGraph(G) for G in L)
>>> B = list(G for G in M if G.is_brace())
>>> C = list(G for G in M if Graph(G).subgraph_search(H) is not None)
>>> B == C
True
```

For every brace $G[A, B]$ of order at least six, the graph $G - a_1 - a_2 - b_1 - b_2$ has a perfect matching for any two distinct vertices a_1 and a_2 in A and any two distinct vertices b_1 and b_2 in B:

```
sage: H =  graphs.CompleteBipartiteGraph(10, 10)
sage: G = MatchingCoveredGraph(H)
sage: G.is_brace()
True
sage: S = [0, 1, 10, 12]
sage: G.delete_vertices(S)
sage: G.has_perfect_matching()
True
```

```
>>> from sage.all import *
>>> H =  graphs.CompleteBipartiteGraph(Integer(10), Integer(10))
>>> G = MatchingCoveredGraph(H)
>>> G.is_brace()
True
>>> S = [Integer(0), Integer(1), Integer(10), Integer(12)]
>>> G.delete_vertices(S)
>>> G.has_perfect_matching()
True
```

For a brace $G[A, B]$ of order six or more, $|N(X)| \geq |X| + 2$, for all $X \subset A$ such that $0 < |X| < |A| - 1$, where $N(S) := \{b | (a, b) \in E \wedge a \in S\}$ is called the neighboring set of S:

```
sage: H = graphs.MoebiusLadderGraph(15)
sage: G = MatchingCoveredGraph(H)
sage: G.is_brace()
True
sage: A, _ = G.bipartite_sets()
sage: # needs random
sage: X = random.sample(list(A), random.randint(1, len(A) - 1))
sage: N = {v for u in X for v in G.neighbor_iterator(u)}
sage: len(N) >= len(X) + 2
True
```

```
>>> from sage.all import *
>>> H = graphs.MoebiusLadderGraph(Integer(15))
>>> G = MatchingCoveredGraph(H)
>>> G.is_brace()
True
>>> A, _ = G.bipartite_sets()
>>> # needs random
>>> X = random.sample(list(A), random.randint(Integer(1), len(A) -␣
↪Integer(1)))
```

(continues on next page)

(continued from previous page)

```
>>> N = {v for u in X for v in G.neighbor_iterator(u)}
>>> len(N) >= len(X) + Integer(2)
True
```

For a brace G of order four or more with a perfect matching M, the graph $G - a - b$ is matching covered for each edge (a, b) in M:

```
sage: H = graphs.HeawoodGraph()
sage: G = MatchingCoveredGraph(H)
sage: G.is_brace()
True
sage: M = G.get_matching()
sage: L = []
sage: for a, b, *_ in M:
....:     J = G.copy(); J.delete_vertices([a, b])
....:     if J.is_matching_covered():
....:         L.append(J)
sage: len(L) == len(M)
True
```

```
>>> from sage.all import *
>>> H = graphs.HeawoodGraph()
>>> G = MatchingCoveredGraph(H)
>>> G.is_brace()
True
>>> M = G.get_matching()
>>> L = []
>>> for a, b, *_ in M:
...     J = G.copy(); J.delete_vertices([a, b])
...     if J.is_matching_covered():
...         L.append(J)
>>> len(L) == len(M)
True
```

A cycle graph of order six or more is a bipartite matching covered graph, but is not a brace:

```
sage: C = graphs.CycleGraph(10)
sage: G = MatchingCoveredGraph(C)
sage: G.is_brace()
False
```

```
>>> from sage.all import *
>>> C = graphs.CycleGraph(Integer(10))
>>> G = MatchingCoveredGraph(C)
>>> G.is_brace()
False
```

A ladder graph of order six or more is a bipartite matching covered graph, that is not a brace. The tight cut decomposition of a ladder graph produces a list graphs the underlying graph of each of which is isomorphic to a 4-cycle:

```
sage: L = graphs.LadderGraph(10)
sage: G = MatchingCoveredGraph(L)
sage: G.is_brace()
False
```

```
>>> from sage.all import *
>>> L = graphs.LadderGraph(Integer(10))
>>> G = MatchingCoveredGraph(L)
>>> G.is_brace()
False
```

One may set the `coNP_certificate` to be `True`:

```
sage: H = graphs.HexahedralGraph()
sage: G = MatchingCoveredGraph(H)
sage: G.is_brace(coNP_certificate=True)
(True, None, None, None, None)
sage: C = graphs.CycleGraph(6)
sage: D = MatchingCoveredGraph(C)
sage: is_brace, nontrivial_tight_cut, nontrivial_odd_component, \
....: nontrivial_tight_cut_variant, cut_identifier = \
....: D.is_brace(coNP_certificate=True)
sage: is_brace is False
True
sage: J = C.subgraph(vertices=nontrivial_odd_component)
sage: J.is_isomorphic(graphs.PathGraph(3))
True
sage: len(nontrivial_tight_cut) == 2
True
sage: nontrivial_tight_cut_variant
'nontrivial barrier cut'
sage: # Corresponding barrier
sage: cut_identifier == {a for u, v, *_ in nontrivial_tight_cut for a in [u,␣
↪v] \
....: if a not in nontrivial_odd_component}
True
sage: for u, v, *_ in nontrivial_tight_cut:
....:     assert (u in nontrivial_odd_component and v not in nontrivial_odd_
↪component)
sage: L = graphs.LadderGraph(3) # A ladder graph with two constituent braces
sage: G = MatchingCoveredGraph(L)
sage: is_brace, nontrivial_tight_cut, nontrivial_odd_component, cut_variant,␣
↪cut_identifier = \
....: G.is_brace(coNP_certificate=True)
sage: is_brace is False
True
sage: G1 = L.copy()
sage: G1.merge_vertices(list(nontrivial_odd_component))
sage: G1.to_simple().is_isomorphic(graphs.CycleGraph(4))
True
sage: G2 = L.copy()
sage: G2.merge_vertices([v for v in G if v not in nontrivial_odd_component])
sage: G2.to_simple().is_isomorphic(graphs.CycleGraph(4))
```

(continues on next page)

(continued from previous page)

```
True
sage: cut_variant
'nontrivial barrier cut'
sage: cut_identifier == {a for u, v, *_ in nontrivial_tight_cut for a in [u,␣
↪v] \
....: if a not in nontrivial_odd_component}
True
sage: H = graphs.CompleteBipartiteGraph(3, 3)
sage: H.delete_edge(0, 3)
sage: G = MatchingCoveredGraph(H)
sage: G.is_brace(coNP_certificate=True)
(False,
 [(4, 1, None), (5, 1, None), (4, 2, None), (5, 2, None)],
 {0, 4, 5},
 'nontrivial barrier cut',
 {1, 2})
```

```
>>> from sage.all import *
>>> H = graphs.HexahedralGraph()
>>> G = MatchingCoveredGraph(H)
>>> G.is_brace(coNP_certificate=True)
(True, None, None, None, None)
>>> C = graphs.CycleGraph(Integer(6))
>>> D = MatchingCoveredGraph(C)
>>> is_brace, nontrivial_tight_cut, nontrivial_odd_component, nontrivial_
↪tight_cut_variant, cut_identifier = D.is_brace(coNP_certificate=True)
>>> is_brace is False
True
>>> J = C.subgraph(vertices=nontrivial_odd_component)
>>> J.is_isomorphic(graphs.PathGraph(Integer(3)))
True
>>> len(nontrivial_tight_cut) == Integer(2)
True
>>> nontrivial_tight_cut_variant
'nontrivial barrier cut'
>>> # Corresponding barrier
>>> cut_identifier == {a for u, v, *_ in nontrivial_tight_cut for a in [u, v]␣
↪if a not in nontrivial_odd_component}
True
>>> for u, v, *_ in nontrivial_tight_cut:
...     assert (u in nontrivial_odd_component and v not in nontrivial_odd_
↪component)
>>> L = graphs.LadderGraph(Integer(3)) # A ladder graph with two constituent␣
↪braces
>>> G = MatchingCoveredGraph(L)
>>> is_brace, nontrivial_tight_cut, nontrivial_odd_component, cut_variant,␣
↪cut_identifier = G.is_brace(coNP_certificate=True)
>>> is_brace is False
True
>>> G1 = L.copy()
>>> G1.merge_vertices(list(nontrivial_odd_component))
>>> G1.to_simple().is_isomorphic(graphs.CycleGraph(Integer(4)))
```

(continues on next page)

(continued from previous page)

```
True
>>> G2 = L.copy()
>>> G2.merge_vertices([v for v in G if v not in nontrivial_odd_component])
>>> G2.to_simple().is_isomorphic(graphs.CycleGraph(Integer(4)))
True
>>> cut_variant
'nontrivial barrier cut'
>>> cut_identifier == {a for u, v, *_ in nontrivial_tight_cut for a in [u, v]␣
↪if a not in nontrivial_odd_component}
True
>>> H = graphs.CompleteBipartiteGraph(Integer(3), Integer(3))
>>> H.delete_edge(Integer(0), Integer(3))
>>> G = MatchingCoveredGraph(H)
>>> G.is_brace(coNP_certificate=True)
(False,
 [(4, 1, None), (5, 1, None), (4, 2, None), (5, 2, None)],
 {0, 4, 5},
 'nontrivial barrier cut',
 {1, 2})
```

If the input matching covered graph is nonbipartite, a `ValueError` is thrown:

```
sage: K4 = graphs.CompleteGraph(4)
sage: G = MatchingCoveredGraph(K4)
sage: G.is_brace()
Traceback (most recent call last):
...
ValueError: the input graph is not bipartite
sage: P = graphs.PetersenGraph()
sage: H = MatchingCoveredGraph(P)
sage: H.is_brace(coNP_certificate=True)
Traceback (most recent call last):
...
ValueError: the input graph is not bipartite
```

```
>>> from sage.all import *
>>> K4 = graphs.CompleteGraph(Integer(4))
>>> G = MatchingCoveredGraph(K4)
>>> G.is_brace()
Traceback (most recent call last):
...
ValueError: the input graph is not bipartite
>>> P = graphs.PetersenGraph()
>>> H = MatchingCoveredGraph(P)
>>> H.is_brace(coNP_certificate=True)
Traceback (most recent call last):
...
ValueError: the input graph is not bipartite
```

See also

- *is_brick()*

is_brick(*coNP_certificate=False*)

Check if the (matching covered) graph is a brick.

A matching covered graph which is free of nontrivial tight cuts is called a *brick* if it is nonbipartite. A nonbipartite matching covered graph is a brick if and only if it is 3-connected and bicritical [LM2024].

INPUT:

- `coNP_certificate` – boolean (default: `False`)

OUTPUT:

- If the input matching covered graph is bipartite, a `ValueError` is returned.
- If the input nonbipartite matching covered graph is a brick, a boolean `True` is returned if `coNP_certificate` is set to `False`, otherwise a 5-tuple `(True, None, None, None, None)` is returned.
- If the input nonbipartite matching covered graph is not a brick, a boolean `False` is returned if `coNP_certificate` is set to `False`.
- If `coNP_certificate` is set to `True` and the input nonbipartite graph is not a brick, a 5-tuple of
  1. a boolean `False`,
  2. a list of lists of edges, each list constituting a nontrivial tight cut collectively representing a laminar tight cut,
  3. a list of set of vertices of one of the shores of those respective nontrivial tight cuts:
     1. In case of nontrivial barrier cuts, each of the shores is a nontrivial odd component with respect to a nontrivial barrier, thus the returned list forms mutually exclusive collection of (odd) sets.
     2. Otherwise each of the nontrivial tight cuts is a 2-separation cut, each of the shores form a subset sequence, with the i th shore being a proper subset of the $i+1$ th shore.
  4. a string showing whether the nontrivial tight cuts are barrier cuts (if the string is 'nontrivial barrier cut'), or 2-separation cuts (if the string is 'nontrivial 2-separation cut')
  5. a set of vertices showing the respective barrier if the nontrivial tight cuts are barrier cuts, or otherwise a set of two vertices constituting the corresponding two vertex cut (in this case the nontrivial tight cuts are 2-separation cuts)

  is returned.

EXAMPLES:

The complete graph on four vertices K_4 is the smallest brick:

```
sage: K = graphs.CompleteGraph(4)
sage: G = MatchingCoveredGraph(K)
sage: G.is_brick()
True
```

```
>>> from sage.all import *
>>> K = graphs.CompleteGraph(Integer(4))
>>> G = MatchingCoveredGraph(K)
```

(continues on next page)

(continued from previous page)

```
>>> G.is_brick()
True
```

The triangular circular ladder (a graph on six vertices), aka $\overline{C_6}$ is a brick:

```
sage: C6Bar = graphs.CircularLadderGraph(3)
sage: G = MatchingCoveredGraph(C6Bar)
sage: G.is_brick()
True
```

```
>>> from sage.all import *
>>> C6Bar = graphs.CircularLadderGraph(Integer(3))
>>> G = MatchingCoveredGraph(C6Bar)
>>> G.is_brick()
True
```

Each of Petersen graph, Bicorn graph, Tricorn graph, Cubeplex graph, Twinplex graph, Wagner graph is a brick:

```
sage: MatchingCoveredGraph(graphs.PetersenGraph()).is_brick() and \
....: MatchingCoveredGraph(graphs.StaircaseGraph(4)).is_brick() and \
....: MatchingCoveredGraph(graphs.TricornGraph()).is_brick() and \
....: MatchingCoveredGraph(graphs.CubeplexGraph()).is_brick() and \
....: MatchingCoveredGraph(graphs.TwinplexGraph()).is_brick() and \
....: MatchingCoveredGraph(graphs.WagnerGraph()).is_brick()
True
```

```
>>> from sage.all import *
>>> MatchingCoveredGraph(graphs.PetersenGraph()).is_brick() and␣
↪MatchingCoveredGraph(graphs.StaircaseGraph(Integer(4))).is_brick() and␣
↪MatchingCoveredGraph(graphs.TricornGraph()).is_brick() and␣
↪MatchingCoveredGraph(graphs.CubeplexGraph()).is_brick() and␣
↪MatchingCoveredGraph(graphs.TwinplexGraph()).is_brick() and␣
↪MatchingCoveredGraph(graphs.WagnerGraph()).is_brick()
True
```

The Murty graph is the smallest simple brick that is not odd-intercyclic:

```
sage: M = graphs.MurtyGraph()
sage: G = MatchingCoveredGraph(M)
sage: G.is_brick()
True
```

```
>>> from sage.all import *
>>> M = graphs.MurtyGraph()
>>> G = MatchingCoveredGraph(M)
>>> G.is_brick()
True
```

A circular ladder graph of order six or more on $2n$ vertices for an odd n is a brick:

```
sage: n = 11
sage: CL = graphs.CircularLadderGraph(n)
sage: G = MatchingCoveredGraph(CL)
sage: G.is_brick()
True
```

```
>>> from sage.all import *
>>> n = Integer(11)
>>> CL = graphs.CircularLadderGraph(n)
>>> G = MatchingCoveredGraph(CL)
>>> G.is_brick()
True
```

A moebius ladder graph of order eight or more on $2n$ vertices for an even n is a brick:

```
sage: n = 10
sage: ML = graphs.MoebiusLadderGraph(n)
sage: G = MatchingCoveredGraph(ML)
sage: G.is_brick()
True
```

```
>>> from sage.all import *
>>> n = Integer(10)
>>> ML = graphs.MoebiusLadderGraph(n)
>>> G = MatchingCoveredGraph(ML)
>>> G.is_brick()
True
```

A wheel graph of an even order is a brick:

```
sage: W = graphs.WheelGraph(10)
sage: G = MatchingCoveredGraph(W)
sage: G.is_brick()
True
```

```
>>> from sage.all import *
>>> W = graphs.WheelGraph(Integer(10))
>>> G = MatchingCoveredGraph(W)
>>> G.is_brick()
True
```

A graph that is isomorphic to a truncated biwheel graph is a brick:

```
sage: TB = graphs.TruncatedBiwheelGraph(15)
sage: G = MatchingCoveredGraph(TB)
sage: G.is_brick()
True
```

```
>>> from sage.all import *
>>> TB = graphs.TruncatedBiwheelGraph(Integer(15))
>>> G = MatchingCoveredGraph(TB)
>>> G.is_brick()
True
```

Each of the graphs in the staircase graph family with order eight or more is a brick:

```
sage: ST = graphs.StaircaseGraph(9)
sage: G = MatchingCoveredGraph(ST)
sage: G.is_brick()
True
```

```
>>> from sage.all import *
>>> ST = graphs.StaircaseGraph(Integer(9))
>>> G = MatchingCoveredGraph(ST)
>>> G.is_brick()
True
```

Bricks are 3-connected:

```
sage: P = graphs.PetersenGraph()
sage: G = MatchingCoveredGraph(P)
sage: G.is_brick()
True
sage: G.is_triconnected()
True
```

```
>>> from sage.all import *
>>> P = graphs.PetersenGraph()
>>> G = MatchingCoveredGraph(P)
>>> G.is_brick()
True
>>> G.is_triconnected()
True
```

Bricks are bicritical:

```
sage: P = graphs.PetersenGraph()
sage: G = MatchingCoveredGraph(P)
sage: G.is_brick()
True
sage: G.is_bicritical()
True
```

```
>>> from sage.all import *
>>> P = graphs.PetersenGraph()
>>> G = MatchingCoveredGraph(P)
>>> G.is_brick()
True
>>> G.is_bicritical()
True
```

Examples of nonbipartite matching covered graphs that are not bricks:

```
sage: H = Graph([
....:     (0, 3), (0, 4), (0, 7),
....:     (1, 3), (1, 5), (1, 7),
....:     (2, 3), (2, 6), (2, 7),
```

(continues on next page)

(continued from previous page)

```
....:     (4, 5), (4, 6), (5, 6)
....: ])
sage: G = MatchingCoveredGraph(H)
sage: G.is_bipartite()
False
sage: G.is_bicritical()
False
sage: G.is_triconnected()
True
sage: G.is_brick()
False
sage: H = Graph([
....:     (0, 1), (0, 2), (0, 3), (0, 4), (1, 2),
....:     (1, 5), (2, 5), (3, 4), (3, 5), (4, 5)
....: ])
sage: G = MatchingCoveredGraph(H)
sage: G.is_bipartite()
False
sage: G.is_bicritical()
True
sage: G.is_triconnected()
False
sage: G.is_brick()
False
```

```
>>> from sage.all import *
>>> H = Graph([
...     (Integer(0), Integer(3)), (Integer(0), Integer(4)), (Integer(0),␣
↪Integer(7)),
...     (Integer(1), Integer(3)), (Integer(1), Integer(5)), (Integer(1),␣
↪Integer(7)),
...     (Integer(2), Integer(3)), (Integer(2), Integer(6)), (Integer(2),␣
↪Integer(7)),
...     (Integer(4), Integer(5)), (Integer(4), Integer(6)), (Integer(5),␣
↪Integer(6))
... ])
>>> G = MatchingCoveredGraph(H)
>>> G.is_bipartite()
False
>>> G.is_bicritical()
False
>>> G.is_triconnected()
True
>>> G.is_brick()
False
>>> H = Graph([
...     (Integer(0), Integer(1)), (Integer(0), Integer(2)), (Integer(0),␣
↪Integer(3)), (Integer(0), Integer(4)), (Integer(1), Integer(2)),
...     (Integer(1), Integer(5)), (Integer(2), Integer(5)), (Integer(3),␣
↪Integer(4)), (Integer(3), Integer(5)), (Integer(4), Integer(5))
... ])
>>> G = MatchingCoveredGraph(H)
```

(continues on next page)

(continued from previous page)

```
>>> G.is_bipartite()
False
>>> G.is_bicritical()
True
>>> G.is_triconnected()
False
>>> G.is_brick()
False
```

One may set the `coNP_certificate` to be `True`:

```
sage: K4 = graphs.CompleteGraph(4)
sage: G = MatchingCoveredGraph(K4)
sage: G.is_brick(coNP_certificate=True)
(True, None, None, None, None)
sage: # K(4) □ K(3, 3) is nonbipartite but not a brick
sage: H = graphs.MurtyGraph(); H.delete_edge(0, 1)
sage: G = MatchingCoveredGraph(H)
sage: G.is_brick(coNP_certificate=True)
(False, [[(5, 2, None), (6, 3, None), (7, 4, None)]], [{5, 6, 7}],
 'nontrivial barrier cut', {2, 3, 4})
sage: H = Graph([
....:     (0, 12), (0, 13), (0, 15), (1, 4), (1, 13), (1, 14),
....:     (1, 19), (2, 4), (2, 13), (2, 14), (2, 17), (3, 9),
....:     (3, 13), (3, 16), (3, 21), (4, 6), (4, 7), (5, 7),
....:     (5, 8), (5, 12), (6, 8), (6, 11), (7, 10), (8, 9),
....:     (9, 10), (10, 11), (11, 12), (14, 15), (14, 16), (15, 16),
....:     (17, 18), (17, 21), (18, 19), (18, 20), (19, 20), (20, 21)
....: ])
sage: G = MatchingCoveredGraph(H)
sage: G.is_brick(coNP_certificate=True)
(False,
 [[(12, 0, None), (4, 1, None), (4, 2, None), (9, 3, None)],
  [(19, 1, None), (17, 2, None), (21, 3, None)],
  [(15, 0, None), (14, 1, None), (14, 2, None), (16, 3, None)]],
 [{4, 5, 6, 7, 8, 9, 10, 11, 12}, {17, 18, 19, 20, 21}, {14, 15, 16}],
 'nontrivial barrier cut', {0, 1, 2, 3})
sage: J = Graph([
....:     (0, 1), (0, 2), (0, 3), (0, 4), (0, 5),
....:     (0, 6), (0, 7), (0, 8), (0, 9), (0, 10),
....:     (1, 2), (1, 11), (2, 11), (3, 4), (3, 11),
....:     (4, 11), (5, 6), (5, 11), (6, 11), (7, 8),
....:     (7, 11), (8, 11), (9, 10), (9, 11), (10, 11)
....: ])
sage: G = MatchingCoveredGraph(J)
sage: G.is_brick(coNP_certificate=True)
(False,
 [[(0, 3, None),
   (0, 4, None),
   (0, 5, None),
   (0, 6, None),
   (0, 7, None),
```

(continues on next page)

(continued from previous page)

```
   (0, 8, None),
   (0, 9, None),
   (0, 10, None),
   (1, 11, None),
   (2, 11, None)],
  [(0, 5, None),
   (0, 6, None),
   (0, 7, None),
   (0, 8, None),
   (0, 9, None),
   (0, 10, None),
   (1, 11, None),
   (2, 11, None),
   (3, 11, None),
   (4, 11, None)],
  [(0, 7, None),
   (0, 8, None),
   (0, 9, None),
   (0, 10, None),
   (1, 11, None),
   (2, 11, None),
   (3, 11, None),
   (4, 11, None),
   (5, 11, None),
   (6, 11, None)],
  [(0, 9, None),
   (0, 10, None),
   (1, 11, None),
   (2, 11, None),
   (3, 11, None),
   (4, 11, None),
   (5, 11, None),
   (6, 11, None),
   (7, 11, None),
   (8, 11, None)]],
 [{0, 1, 2},
  {0, 1, 2, 3, 4},
  {0, 1, 2, 3, 4, 5, 6},
  {0, 1, 2, 3, 4, 5, 6, 7, 8}],
 'nontrivial 2-separation cut',
 {0, 11})
```

```
>>> from sage.all import *
>>> K4 = graphs.CompleteGraph(Integer(4))
>>> G = MatchingCoveredGraph(K4)
>>> G.is_brick(coNP_certificate=True)
(True, None, None, None, None)
>>> # K(4) □ K(3, 3) is nonbipartite but not a brick
>>> H = graphs.MurtyGraph(); H.delete_edge(Integer(0), Integer(1))
>>> G = MatchingCoveredGraph(H)
>>> G.is_brick(coNP_certificate=True)
(False, [[(5, 2, None), (6, 3, None), (7, 4, None)]], [{5, 6, 7}],
```

(continues on next page)

(continued from previous page)

```
 'nontrivial barrier cut', {2, 3, 4})
>>> H = Graph([
...     (Integer(0), Integer(12)), (Integer(0), Integer(13)), (Integer(0),␣
↪Integer(15)), (Integer(1), Integer(4)), (Integer(1), Integer(13)),␣
↪(Integer(1), Integer(14)),
...     (Integer(1), Integer(19)), (Integer(2), Integer(4)), (Integer(2),␣
↪Integer(13)), (Integer(2), Integer(14)), (Integer(2), Integer(17)),␣
↪(Integer(3), Integer(9)),
...     (Integer(3), Integer(13)), (Integer(3), Integer(16)), (Integer(3),␣
↪Integer(21)), (Integer(4), Integer(6)), (Integer(4), Integer(7)),␣
↪(Integer(5), Integer(7)),
...     (Integer(5), Integer(8)), (Integer(5), Integer(12)), (Integer(6),␣
↪Integer(8)), (Integer(6), Integer(11)), (Integer(7), Integer(10)),␣
↪(Integer(8), Integer(9)),
...     (Integer(9), Integer(10)), (Integer(10), Integer(11)), (Integer(11),␣
↪Integer(12)), (Integer(14), Integer(15)), (Integer(14), Integer(16)),␣
↪(Integer(15), Integer(16)),
...     (Integer(17), Integer(18)), (Integer(17), Integer(21)), (Integer(18),␣
↪Integer(19)), (Integer(18), Integer(20)), (Integer(19), Integer(20)),␣
↪(Integer(20), Integer(21))
... ])
>>> G = MatchingCoveredGraph(H)
>>> G.is_brick(coNP_certificate=True)
(False,
 [[(12, 0, None), (4, 1, None), (4, 2, None), (9, 3, None)],
  [(19, 1, None), (17, 2, None), (21, 3, None)],
  [(15, 0, None), (14, 1, None), (14, 2, None), (16, 3, None)]],
 [{4, 5, 6, 7, 8, 9, 10, 11, 12}, {17, 18, 19, 20, 21}, {14, 15, 16}],
 'nontrivial barrier cut', {0, 1, 2, 3})
>>> J = Graph([
...     (Integer(0), Integer(1)), (Integer(0), Integer(2)), (Integer(0),␣
↪Integer(3)), (Integer(0), Integer(4)), (Integer(0), Integer(5)),
...     (Integer(0), Integer(6)), (Integer(0), Integer(7)), (Integer(0),␣
↪Integer(8)), (Integer(0), Integer(9)), (Integer(0), Integer(10)),
...     (Integer(1), Integer(2)), (Integer(1), Integer(11)), (Integer(2),␣
↪Integer(11)), (Integer(3), Integer(4)), (Integer(3), Integer(11)),
...     (Integer(4), Integer(11)), (Integer(5), Integer(6)), (Integer(5),␣
↪Integer(11)), (Integer(6), Integer(11)), (Integer(7), Integer(8)),
...     (Integer(7), Integer(11)), (Integer(8), Integer(11)), (Integer(9),␣
↪Integer(10)), (Integer(9), Integer(11)), (Integer(10), Integer(11))
... ])
>>> G = MatchingCoveredGraph(J)
>>> G.is_brick(coNP_certificate=True)
(False,
 [[(0, 3, None),
   (0, 4, None),
   (0, 5, None),
   (0, 6, None),
   (0, 7, None),
   (0, 8, None),
   (0, 9, None),
   (0, 10, None),
```

(continues on next page)

(continued from previous page)

```
   (1, 11, None),
   (2, 11, None)],
  [(0, 5, None),
   (0, 6, None),
   (0, 7, None),
   (0, 8, None),
   (0, 9, None),
   (0, 10, None),
   (1, 11, None),
   (2, 11, None),
   (3, 11, None),
   (4, 11, None)],
  [(0, 7, None),
   (0, 8, None),
   (0, 9, None),
   (0, 10, None),
   (1, 11, None),
   (2, 11, None),
   (3, 11, None),
   (4, 11, None),
   (5, 11, None),
   (6, 11, None)],
  [(0, 9, None),
   (0, 10, None),
   (1, 11, None),
   (2, 11, None),
   (3, 11, None),
   (4, 11, None),
   (5, 11, None),
   (6, 11, None),
   (7, 11, None),
   (8, 11, None)]],
 [{0, 1, 2},
  {0, 1, 2, 3, 4},
  {0, 1, 2, 3, 4, 5, 6},
  {0, 1, 2, 3, 4, 5, 6, 7, 8}],
 'nontrivial 2-separation cut',
 {0, 11})
```

If the input matching covered graph is bipartite, a `ValueError` is thrown:

```
sage: H = graphs.HexahedralGraph()
sage: G = MatchingCoveredGraph(H)
sage: G.is_brick()
Traceback (most recent call last):
...
ValueError: the input graph is bipartite
sage: J = graphs.HeawoodGraph()
sage: G = MatchingCoveredGraph(J)
sage: G.is_brick(coNP_certificate=True)
Traceback (most recent call last):
...
```

(continues on next page)

(continued from previous page)

```
ValueError: the input graph is bipartite
```

```
>>> from sage.all import *
>>> H = graphs.HexahedralGraph()
>>> G = MatchingCoveredGraph(H)
>>> G.is_brick()
Traceback (most recent call last):
...
ValueError: the input graph is bipartite
>>> J = graphs.HeawoodGraph()
>>> G = MatchingCoveredGraph(J)
>>> G.is_brick(coNP_certificate=True)
Traceback (most recent call last):
...
ValueError: the input graph is bipartite
```

See also

- *is_bicritical()*
- *is_brace()*

loop_edges(*labels=True*)

Return a list of all loops in the (matching covered) graph.

Note

This method overwrites the *loop_edges()* method in order to return an empty list as matching covered graphs are free of looped edges.

INPUT:

- `labels` – boolean (default: `True`); whether returned edges have labels (`(u,v,l)`) or not (`(u,v)`).

OUTPUT:

- A list capturing the edges that are loops in the matching covered graph; note that, the list is empty since matching covered graphs do not contain any looped edges.

EXAMPLES:

A matching covered graph, for instance the Heawood graph, by definition, is always free of loops:

```
sage: H = graphs.HeawoodGraph()
sage: G = MatchingCoveredGraph(H)
sage: G
Matching covered heawood graph: graph on 14 vertices
sage: G.add_edge(0, 0)
Traceback (most recent call last):
...
ValueError: loops are not allowed in matching covered graphs
sage: G.loops()
```

(continues on next page)

(continued from previous page)

```
[]
sage: G.loop_edges()
[]
```

```
>>> from sage.all import *
>>> H = graphs.HeawoodGraph()
>>> G = MatchingCoveredGraph(H)
>>> G
Matching covered heawood graph: graph on 14 vertices
>>> G.add_edge(Integer(0), Integer(0))
Traceback (most recent call last):
...
ValueError: loops are not allowed in matching covered graphs
>>> G.loops()
[]
>>> G.loop_edges()
[]
```

A matching covered graph may support multiple edges, still no loops are allowed:

```
sage: C = graphs.CycleGraph(4)
sage: G = MatchingCoveredGraph(C)
sage: G.allow_multiple_edges(True)
sage: G
Matching covered cycle graph: multi-graph on 4 vertices
sage: G.add_edge(0, 1, 'label')
sage: G.add_edge(0, 0)
Traceback (most recent call last):
...
ValueError: loops are not allowed in matching covered graphs
sage: G.edges(sort=False)
[(0, 1, None), (0, 1, 'label'), (0, 3, None), (1, 2, None), (2, 3, None)]
sage: G.loops()
[]
sage: G.loop_edges()
[]
```

```
>>> from sage.all import *
>>> C = graphs.CycleGraph(Integer(4))
>>> G = MatchingCoveredGraph(C)
>>> G.allow_multiple_edges(True)
>>> G
Matching covered cycle graph: multi-graph on 4 vertices
>>> G.add_edge(Integer(0), Integer(1), 'label')
>>> G.add_edge(Integer(0), Integer(0))
Traceback (most recent call last):
...
ValueError: loops are not allowed in matching covered graphs
>>> G.edges(sort=False)
[(0, 1, None), (0, 1, 'label'), (0, 3, None), (1, 2, None), (2, 3, None)]
>>> G.loops()
[]
```

(continues on next page)

(continued from previous page)

```
>>> G.loop_edges()
[]
```

One may set the `label` to either `True` or `False`:

```
sage: G.loop_edges(labels=False)
[]
sage: G.loops(labels=True)
[]
```

```
>>> from sage.all import *
>>> G.loop_edges(labels=False)
[]
>>> G.loops(labels=True)
[]
```

See also

- *allow_loops()*
- *allows_loops()*
- *has_loops()*
- *loop_vertices()*
- *loops()*
- *number_of_loops()*
- *remove_loops()*

loop_vertices()

Return a list of vertices with loops.

Note

This method overwrites the *loop_vertices()* method in order to return an empty list as matching covered graphs are free of vertices that have looped edges.

OUTPUT:

- A list capturing the vertices that have loops in the matching covered graph; note that, the list is empty since matching covered graphs do not contain any looped edges.

EXAMPLES:

A matching covered graph, for instance the Möbius graph of order 8, by definition, is always free of loops:

```
sage: M = graphs.MoebiusLadderGraph(4)
sage: G = MatchingCoveredGraph(M)
sage: G
Matching covered moebius ladder graph: graph on 8 vertices
```

(continues on next page)

(continued from previous page)

```
sage: G.add_edge(0, 0)
Traceback (most recent call last):
...
ValueError: loops are not allowed in matching covered graphs
sage: G.loop_vertices()
[]
```

```
>>> from sage.all import *
>>> M = graphs.MoebiusLadderGraph(Integer(4))
>>> G = MatchingCoveredGraph(M)
>>> G
Matching covered moebius ladder graph: graph on 8 vertices
>>> G.add_edge(Integer(0), Integer(0))
Traceback (most recent call last):
...
ValueError: loops are not allowed in matching covered graphs
>>> G.loop_vertices()
[]
```

A matching covered graph may support multiple edges, still no loops are allowed:

```
sage: S = graphs.StaircaseGraph(4)
sage: G = MatchingCoveredGraph(S)
sage: G.allow_multiple_edges(True)
sage: G
Matching covered staircase graph: multi-graph on 8 vertices
sage: G.add_edge(0, 1, 'label')
sage: G.add_edge(0, 0)
Traceback (most recent call last):
...
ValueError: loops are not allowed in matching covered graphs
sage: G.edges(sort=False)
[(0, 1, None), (0, 1, 'label'), (0, 3, None), (0, 6, None),
 (1, 2, None), (1, 4, None), (2, 5, None), (2, 7, None),
 (3, 4, None), (3, 6, None), (4, 5, None), (5, 7, None),
 (6, 7, None)]
sage: G.loop_vertices()
[]
```

```
>>> from sage.all import *
>>> S = graphs.StaircaseGraph(Integer(4))
>>> G = MatchingCoveredGraph(S)
>>> G.allow_multiple_edges(True)
>>> G
Matching covered staircase graph: multi-graph on 8 vertices
>>> G.add_edge(Integer(0), Integer(1), 'label')
>>> G.add_edge(Integer(0), Integer(0))
Traceback (most recent call last):
...
ValueError: loops are not allowed in matching covered graphs
>>> G.edges(sort=False)
[(0, 1, None), (0, 1, 'label'), (0, 3, None), (0, 6, None),
```

(continues on next page)

(continued from previous page)

```
 (1, 2, None), (1, 4, None), (2, 5, None), (2, 7, None),
 (3, 4, None), (3, 6, None), (4, 5, None), (5, 7, None),
 (6, 7, None)]
>>> G.loop_vertices()
[]
```

See also

- *allow_loops()*
- *allows_loops()*
- *has_loops()*
- *loop_edges()*
- *loops()*
- *number_of_loops()*
- *remove_loops()*

loops(*labels=True*)

alias of *loop_edges()*.

maximal_barrier(*vertex*)

Return the (unique) maximal barrier containing the vertex.

For a matching covered graph G, a subset B of the vertex set V is a barrier if $|B| = o(G - B)$, where $|B|$ denotes the cardinality of the set B and $o(G - B)$ denotes the number of odd components in the graph $G - B$. And a barrier B is a maximal barrier if C is not a barrier for each C such that $B \subset C \subseteq V$.

In a matching covered graph, each vertex belongs to a unique maximal barrier, which is a consequence of the following theorem.

Theorem [LM2024]:

Let u and v be any two vertices in a matchable graph G. Then the graph $G - u - v$ is matchable if and only if there is no barrier of G which contains both u and v.

And in order to find the vertices that do not lie in the maximal barrier containing the provided vertex in linear time we take inspiration of the M alternating tree search method [LR2004].

INPUT:

- `vertex` – a vertex of the graph

OUTPUT:

- A `ValueError` is returned if `vertex` is not a vertex of the graph, otherwise a set of vertices that constitute the (unique) maximal barrier containing the vertex is returned.

EXAMPLES:

The graph $K_4 \odot K_{3,3}$ is matching covered. Show the set of vertices in the (unique) maximal barrier containing the vertex 2:

```
sage: G = Graph([
....:     (0, 2), (0, 3), (0, 4), (1, 2),
....:     (1, 3), (1, 4), (2, 5), (3, 6),
....:     (4, 7), (5, 6), (5, 7), (6, 7)
....: ])
sage: H = MatchingCoveredGraph(G)
sage: B = H.maximal_barrier(2)
sage: B
{2, 3, 4}
```

```
>>> from sage.all import *
>>> G = Graph([
...     (Integer(0), Integer(2)), (Integer(0), Integer(3)), (Integer(0),␣
↪Integer(4)), (Integer(1), Integer(2)),
...     (Integer(1), Integer(3)), (Integer(1), Integer(4)), (Integer(2),␣
↪Integer(5)), (Integer(3), Integer(6)),
...     (Integer(4), Integer(7)), (Integer(5), Integer(6)), (Integer(5),␣
↪Integer(7)), (Integer(6), Integer(7))
... ])
>>> H = MatchingCoveredGraph(G)
>>> B = H.maximal_barrier(Integer(2))
>>> B
{2, 3, 4}
```

Let B be a maximal barrier of a matching covered graph G (which is, of course, a matchable graph). The graph, $J := G - B$ has no even component:

```
sage: J = G.copy()
sage: J.delete_vertices(B)
sage: all(len(K)%2 != 0 for K in J.connected_components(sort=True))
True
```

```
>>> from sage.all import *
>>> J = G.copy()
>>> J.delete_vertices(B)
>>> all(len(K)%Integer(2) != Integer(0) for K in J.connected_
↪components(sort=True))
True
```

Let B be a maximal barrier in a matching covered graph G and let M be a perfect matching of G. If K is an odd component of $J := G - B$, then $M \cap \partial_G(K)$ has precisely one edge and if v is the end of that edge in $V(K)$, then $M \cap E(K)$ is a perfect matching of $K - v$:

```
sage: K = J.subgraph(vertices=(J.connected_components(sort=True))[0])
sage: # Let F := \partial_G(K) and T := M \cap F
sage: F = [edge for edge in G.edge_iterator()
....:      if (edge[0] in K and edge[1] not in K)
....:      or (edge[0] not in K and edge[1] in K)
....: ]
sage: M = H.get_matching()
sage: T = [edge for edge in F if edge in M]
sage: len(T) == 1
True
```

(continues on next page)

(continued from previous page)

```
sage: v = T[0][0] if T[0][0] in K else T[0][1]
sage: # Let N := M \cap E(K) and L := K - v
sage: N = Graph([edge for edge in K.edge_iterator() if edge in M])
sage: L = K.copy()
sage: L.delete_vertex(v)
sage: # Check if N is a perfect matching of L
sage: L.order() == 2*N.size()
True
```

```
>>> from sage.all import *
>>> K = J.subgraph(vertices=(J.connected_components(sort=True))[Integer(0)])
>>> # Let F := \partial_G(K) and T := M \cap F
>>> F = [edge for edge in G.edge_iterator()
...      if (edge[Integer(0)] in K and edge[Integer(1)] not in K)
...      or (edge[Integer(0)] not in K and edge[Integer(1)] in K)
... ]
>>> M = H.get_matching()
>>> T = [edge for edge in F if edge in M]
>>> len(T) == Integer(1)
True
>>> v = T[Integer(0)][Integer(0)] if T[Integer(0)][Integer(0)] in K else␣
↪T[Integer(0)][Integer(1)]
>>> # Let N := M \cap E(K) and L := K - v
>>> N = Graph([edge for edge in K.edge_iterator() if edge in M])
>>> L = K.copy()
>>> L.delete_vertex(v)
>>> # Check if N is a perfect matching of L
>>> L.order() == Integer(2)*N.size()
True
```

Let B be a maximal barrier of a matching covered graph G (which is, of course, a matchable graph). The graph induced by each component of $G - B$ is factor critical:

```
sage: all((K.subgraph(vertices=connected_component)).is_factor_critical()
....:     for connected_component in K.connected_components(sort=True)
....: )
True
```

```
>>> from sage.all import *
>>> all((K.subgraph(vertices=connected_component)).is_factor_critical()
...     for connected_component in K.connected_components(sort=True)
... )
True
```

For a bicritical graph (for instance, the Petersen graph), for each vertex the maximal barrier is a singleton set containing only that vertex:

```
sage: P = graphs.PetersenGraph()
sage: G = MatchingCoveredGraph(P)
sage: u = 0
sage: set([u]) == G.maximal_barrier(u)
True
```

```
>>> from sage.all import *
>>> P = graphs.PetersenGraph()
>>> G = MatchingCoveredGraph(P)
>>> u = Integer(0)
>>> set([u]) == G.maximal_barrier(u)
True
```

In a bipartite matching covered graph (for instance, the Hexahedral graph), for a vertex, the maximal barrier is the set of vertices of the color class that the particular vertex belongs to. In other words, there are precisely two maximal barriers in a bipartite matching covered graph, that is, the vertex sets of the individual color class:

```
sage: G = graphs.HexahedralGraph()
sage: H = MatchingCoveredGraph(G)
sage: A, _ = H.bipartite_sets()
sage: # needs random
sage: import random
sage: a = random.choice(list(A))
sage: A == H.maximal_barrier(a)
True
```

```
>>> from sage.all import *
>>> G = graphs.HexahedralGraph()
>>> H = MatchingCoveredGraph(G)
>>> A, _ = H.bipartite_sets()
>>> # needs random
>>> import random
>>> a = random.choice(list(A))
>>> A == H.maximal_barrier(a)
True
```

Maximal barriers of matching covered graph constitute a partition of its vertex set:

```
sage: S = set()
sage: for v in H:
....:     B = tuple(sorted(list(H.maximal_barrier(v))))
....:     S.add(B)
sage: S = list(S)
sage: # Check that S is a partition of the vertex set of H
sage: # Part 1: Check if S spans the vertex set of H
sage: sorted([u for B in S for u in B]) == sorted(list(H))
True
sage: # Part 2: Check if each maximal barrier in S is disjoint
sage: is_disjoint = True
sage: for i in range(len(S)):
....:     for j in range(i+1, len(S)):
....:         c = [v for v in S[i] if v in S[j]]
....:         is_disjoint = (len(c) == 0)
sage: is_disjoint
True
```

```
>>> from sage.all import *
>>> S = set()
```

(continues on next page)

(continued from previous page)

```
>>> for v in H:
...     B = tuple(sorted(list(H.maximal_barrier(v))))
...     S.add(B)
>>> S = list(S)
>>> # Check that S is a partition of the vertex set of H
>>> # Part 1: Check if S spans the vertex set of H
>>> sorted([u for B in S for u in B]) == sorted(list(H))
True
>>> # Part 2: Check if each maximal barrier in S is disjoint
>>> is_disjoint = True
>>> for i in range(len(S)):
...     for j in range(i+Integer(1), len(S)):
...         c = [v for v in S[i] if v in S[j]]
...         is_disjoint = (len(c) == Integer(0))
>>> is_disjoint
True
```

REFERENCES:

- [LZ2004]
- [LM2024]

See also

- *is_bicritical()*
- *is_matching_covered()*
- *canonical_partition()*

number_of_loops()

Return the number of edges that are loops.

Note

This method overwrites the *number_of_loops()* method in order to return 0 as matching covered graphs are free of looped edges.

OUTPUT:

- An integer, 0 is returned, since matching covered graphs do not contain zero loops.

EXAMPLES:

A matching covered graph, for instance the Truncated biwheel graph, by definition, is always free of loops:

```
sage: T = graphs.TruncatedBiwheelGraph(5)
sage: G = MatchingCoveredGraph(T)
sage: G
Matching covered truncated biwheel graph: graph on 10 vertices
sage: G.add_edge(0, 0)
Traceback (most recent call last):
```

(continues on next page)

(continued from previous page)

```
...
ValueError: loops are not allowed in matching covered graphs
sage: G.loop_vertices()
[]
sage: G.number_of_loops()
0
```

```
>>> from sage.all import *
>>> T = graphs.TruncatedBiwheelGraph(Integer(5))
>>> G = MatchingCoveredGraph(T)
>>> G
Matching covered truncated biwheel graph: graph on 10 vertices
>>> G.add_edge(Integer(0), Integer(0))
Traceback (most recent call last):
...
ValueError: loops are not allowed in matching covered graphs
>>> G.loop_vertices()
[]
>>> G.number_of_loops()
0
```

A matching covered graph may support multiple edges, still no loops are allowed:

```
sage: B = graphs.BiwheelGraph(4)
sage: G = MatchingCoveredGraph(B)
sage: G.allow_multiple_edges(True)
sage: G
Matching covered biwheel graph: multi-graph on 8 vertices
sage: G.add_edge(0, 1, 'label')
sage: G.add_edge(0, 0)
Traceback (most recent call last):
...
ValueError: loops are not allowed in matching covered graphs
sage: G.edges(sort=False)
[(0, 1, None), (0, 1, 'label'), (0, 5, None), (0, 7, None),
 (1, 2, None), (1, 6, None), (2, 3, None), (2, 7, None),
 (3, 4, None), (3, 6, None), (4, 5, None), (4, 7, None),
 (5, 6, None)]
sage: G.loop_vertices()
[]
sage: G.number_of_loops()
0
```

```
>>> from sage.all import *
>>> B = graphs.BiwheelGraph(Integer(4))
>>> G = MatchingCoveredGraph(B)
>>> G.allow_multiple_edges(True)
>>> G
Matching covered biwheel graph: multi-graph on 8 vertices
>>> G.add_edge(Integer(0), Integer(1), 'label')
>>> G.add_edge(Integer(0), Integer(0))
Traceback (most recent call last):
```

(continues on next page)

(continued from previous page)

```
...
ValueError: loops are not allowed in matching covered graphs
>>> G.edges(sort=False)
[(0, 1, None), (0, 1, 'label'), (0, 5, None), (0, 7, None),
 (1, 2, None), (1, 6, None), (2, 3, None), (2, 7, None),
 (3, 4, None), (3, 6, None), (4, 5, None), (4, 7, None),
 (5, 6, None)]
>>> G.loop_vertices()
[]
>>> G.number_of_loops()
0
```

See also

- *allow_loops()*
- *allows_loops()*
- *has_loops()*
- *loop_edges()*
- *loop_vertices()*
- *loops()*
- *remove_loops()*

remove_loops(*vertices=None*)

Remove loops on vertices in `vertices`.

Note

This method overwrites the *remove_loops()* method in order to return without any alteration as matching covered graphs are free of looped edges.

INPUT:

- `vertices` – (default: `None`) iterator container of vertex labels corresponding to which the looped edges are to be removed. If `vertices` is `None`, remove all loops.

OUTPUT:

- Nothing is returned, as a matching covered graph is already devoid of any loops.

EXAMPLES:

A matching covered graph, for instance the Wheel graph of order six, is always free of loops:

```
sage: W = graphs.WheelGraph(6)
sage: G = MatchingCoveredGraph(W)
sage: G
Matching covered wheel graph: graph on 6 vertices
sage: G.add_edge(0, 0)
Traceback (most recent call last):
```

(continues on next page)

(continued from previous page)

```
...
ValueError: loops are not allowed in matching covered graphs
sage: G.remove_loops()
sage: G.edges(sort=True)
[(0, 1, None), (0, 2, None), (0, 3, None), (0, 4, None),
 (0, 5, None), (1, 2, None), (1, 5, None), (2, 3, None),
 (3, 4, None), (4, 5, None)]
```

```
>>> from sage.all import *
>>> W = graphs.WheelGraph(Integer(6))
>>> G = MatchingCoveredGraph(W)
>>> G
Matching covered wheel graph: graph on 6 vertices
>>> G.add_edge(Integer(0), Integer(0))
Traceback (most recent call last):
...
ValueError: loops are not allowed in matching covered graphs
>>> G.remove_loops()
>>> G.edges(sort=True)
[(0, 1, None), (0, 2, None), (0, 3, None), (0, 4, None),
 (0, 5, None), (1, 2, None), (1, 5, None), (2, 3, None),
 (3, 4, None), (4, 5, None)]
```

A matching covered graph may support multiple edges, still no loops are allowed:

```
sage: K = graphs.CompleteGraph(2)
sage: G = MatchingCoveredGraph(K)
sage: G.allow_multiple_edges(True)
sage: G
Matching covered complete graph: multi-graph on 2 vertices
sage: G.add_edge(0, 1, 'label')
sage: G.add_edge(0, 0)
Traceback (most recent call last):
...
ValueError: loops are not allowed in matching covered graphs
sage: G.edges(sort=False)
[(0, 1, None), (0, 1, 'label')]
sage: G.remove_loops(vertices=[0, 1])
sage: G.edges(sort=False)
[(0, 1, None), (0, 1, 'label')]
sage: G.remove_loops(vertices=[0..100])
```

```
>>> from sage.all import *
>>> K = graphs.CompleteGraph(Integer(2))
>>> G = MatchingCoveredGraph(K)
>>> G.allow_multiple_edges(True)
>>> G
Matching covered complete graph: multi-graph on 2 vertices
>>> G.add_edge(Integer(0), Integer(1), 'label')
>>> G.add_edge(Integer(0), Integer(0))
Traceback (most recent call last):
...
```

(continues on next page)

(continued from previous page)

```
ValueError: loops are not allowed in matching covered graphs
>>> G.edges(sort=False)
[(0, 1, None), (0, 1, 'label')]
>>> G.remove_loops(vertices=[Integer(0), Integer(1)])
>>> G.edges(sort=False)
[(0, 1, None), (0, 1, 'label')]
>>> G.remove_loops(vertices=(ellipsis_range(Integer(0),Ellipsis,
↪Integer(100))))
```

Note that the parameter `vertices` must be either `None` or an iterable:

```
sage: G.remove_loops(vertices='')
sage: G.edges(sort=False)
[(0, 1, None), (0, 1, 'label')]
sage: G.remove_loops(vertices=None)
sage: G.edges(sort=False)
[(0, 1, None), (0, 1, 'label')]
sage: G.remove_loops(vertices=0)
Traceback (most recent call last):
...
TypeError: 'Integer' object is not iterable
sage: G.remove_loops(vertices=False)
Traceback (most recent call last):
...
TypeError: 'bool' object is not iterable
```

```
>>> from sage.all import *
>>> G.remove_loops(vertices='')
>>> G.edges(sort=False)
[(0, 1, None), (0, 1, 'label')]
>>> G.remove_loops(vertices=None)
>>> G.edges(sort=False)
[(0, 1, None), (0, 1, 'label')]
>>> G.remove_loops(vertices=Integer(0))
Traceback (most recent call last):
...
TypeError: 'Integer' object is not iterable
>>> G.remove_loops(vertices=False)
Traceback (most recent call last):
...
TypeError: 'bool' object is not iterable
```

See also

- *allow_loops()*
- *allows_loops()*
- *has_loops()*
- *loop_edges()*
- *loop_vertices()*

- *loops()*
- *number_of_loops()*

subdivide_edge(**args*)

Subdivide an edge k times.

Note

This method overwrites the *subdivide_edge()* method to ensure that resultant graph is also matching covered.

INPUT:

The following forms are all accepted to subdivide 8 times the edge between vertices 1 and 2 labeled with `'label'`.

- `G.subdivide_edge(1, 2, 8 )`
- `G.subdivide_edge((1, 2), 8 )`
- `G.subdivide_edge(1, 2, 'label', 8 )`
- `G.subdivide_edge((1, 2, 'label'), 8 )`

Note

- If the given edge is labelled with l, all the edges created by the subdivision will have the same label.
- If no label given, the label obtained from *edge_label()* will be used. If multiple such labels are present, the first one in the returned list will be used.
- The number of subdivisions must be a nonnegative even integer in order to ensure the resultant graph is matching covered.
- In the context of matching covered graphs, *bisubdividing* an edge *t* times is defined as subdividing the edge *2t* times, for some nonnegative integer *t*.
- For two input arguments, the first one must be of the form (u, v) or $(u, v, label)$

OUTPUT:

- If an existent edge is provided with a valid format and the parameter `k` in the argument is a nonnegative even integer then the graph is updated and nothing is returned, otherwise a `ValueError` is returned if `k` is not a nonnegative even integer,
- If the graph does not contain the edge provided, a `ValueError` is returned. Also, a `ValueError` is thrown in case the provided arguments are not valid.

EXAMPLES:

Subdividing 4 times an edge of the Petersen graph. Please note that the perfect matching captured at `self.get_matching()` also gets updated:

```
sage: P = graphs.PetersenGraph()
sage: G = MatchingCoveredGraph(P)
sage: from collections import Counter
```

(continues on next page)

(continued from previous page)

```
sage: V, E = set(G.vertices()), list(G.edges(sort=True, sort_vertices=True))
sage: M = set(G.get_matching())
sage: G.subdivide_edge(0, 1, 4)
sage: W, F = set(G.vertices()), list(G.edges(sort=True, sort_vertices=True))
sage: N = set(G.get_matching())
sage: sorted(W - V)
[10, 11, 12, 13]
sage: sorted(list((Counter(F) - Counter(E)).elements())), \
....: sorted(list((Counter(E) - Counter(F)).elements()))
([(0, 10, None), (1, 13, None), (10, 11, None), (11, 12, None),
 (12, 13, None)], [(0, 1, None)])
sage: if (0, 1, None) in M:
....:     assert sorted(N - M), sorted(M - N) == \
....:         ([(0, 10, None), (1, 13, None), (11, 12, None)], [(0, 1, None)])
....: else:
....:     assert sorted(N - M), sorted(M - N) == \
....:         ([(10, 11, None), (12, 13, None)], [])
```

```
>>> from sage.all import *
>>> P = graphs.PetersenGraph()
>>> G = MatchingCoveredGraph(P)
>>> from collections import Counter
>>> V, E = set(G.vertices()), list(G.edges(sort=True, sort_vertices=True))
>>> M = set(G.get_matching())
>>> G.subdivide_edge(Integer(0), Integer(1), Integer(4))
>>> W, F = set(G.vertices()), list(G.edges(sort=True, sort_vertices=True))
>>> N = set(G.get_matching())
>>> sorted(W - V)
[10, 11, 12, 13]
>>> sorted(list((Counter(F) - Counter(E)).elements())),␣
↪sorted(list((Counter(E) - Counter(F)).elements()))
([(0, 10, None), (1, 13, None), (10, 11, None), (11, 12, None),
 (12, 13, None)], [(0, 1, None)])
>>> if (Integer(0), Integer(1), None) in M:
...     assert sorted(N - M), sorted(M - N) ==         ([(Integer(0),␣
↪Integer(10), None), (Integer(1), Integer(13), None), (Integer(11),␣
↪Integer(12), None)], [(Integer(0), Integer(1), None)])
... else:
...     assert sorted(N - M), sorted(M - N) ==         ([(Integer(10),␣
↪Integer(11), None), (Integer(12), Integer(13), None)], [])
```

Subdividing a multiple edge/ some multiple edges:

```
sage: K = graphs.CycleGraph(4)
sage: K.allow_multiple_edges(1)
sage: K.add_edges([(0, 1, 2), (0, 1, 3), (0, 1, 0.5)])
sage: K.delete_edge(0, 1, None)
sage: G = MatchingCoveredGraph(K)
sage: from collections import Counter
sage: V, E = set(G.vertices()), list(G.edges(sort=True, sort_vertices=True))
sage: G.edge_label(0, 1)
[2, 3, 0.500000000000000]
```

(continues on next page)

(continued from previous page)

```
sage: G.subdivide_edge((0, 1), 6)  # the edge: (0, 1, 2)
sage: G.subdivide_edge((0, 1, 3), 2)  # the edge: (0, 1, 3)
sage: G.subdivide_edge(1, 2, None, 2)  # the edge: (1, 2, None)
sage: W, F = set(G.vertices()), list(G.edges(sort=True, sort_vertices=True))
sage: sorted(W - V)
[4, 5, 6, 7, 8, 9, 10, 11, 12, 13]
sage: sorted(list((Counter(F) - Counter(E)).elements())), \
....: sorted(list((Counter(E) - Counter(F)).elements()))
([(0, 4, 2), (0, 10, 3), (1, 9, 2), (1, 11, 3), (1, 12, None), (2, 13, None),
  (4, 5, 2), (5, 6, 2), (6, 7, 2), (7, 8, 2), (8, 9, 2), (10, 11, 3),
  (12, 13, None)], [(0, 1, 2), (0, 1, 3), (1, 2, None)])
```

```
>>> from sage.all import *
>>> K = graphs.CycleGraph(Integer(4))
>>> K.allow_multiple_edges(Integer(1))
>>> K.add_edges([(Integer(0), Integer(1), Integer(2)), (Integer(0),␣
↪Integer(1), Integer(3)), (Integer(0), Integer(1), RealNumber('0.5'))])
>>> K.delete_edge(Integer(0), Integer(1), None)
>>> G = MatchingCoveredGraph(K)
>>> from collections import Counter
>>> V, E = set(G.vertices()), list(G.edges(sort=True, sort_vertices=True))
>>> G.edge_label(Integer(0), Integer(1))
[2, 3, 0.500000000000000]
>>> G.subdivide_edge((Integer(0), Integer(1)), Integer(6))  # the edge: (0, 1,
↪ 2)
>>> G.subdivide_edge((Integer(0), Integer(1), Integer(3)), Integer(2))  # the␣
↪edge: (0, 1, 3)
>>> G.subdivide_edge(Integer(1), Integer(2), None, Integer(2))  # the edge:␣
↪(1, 2, None)
>>> W, F = set(G.vertices()), list(G.edges(sort=True, sort_vertices=True))
>>> sorted(W - V)
[4, 5, 6, 7, 8, 9, 10, 11, 12, 13]
>>> sorted(list((Counter(F) - Counter(E)).elements())),␣
↪sorted(list((Counter(E) - Counter(F)).elements()))
([(0, 4, 2), (0, 10, 3), (1, 9, 2), (1, 11, 3), (1, 12, None), (2, 13, None),
  (4, 5, 2), (5, 6, 2), (6, 7, 2), (7, 8, 2), (8, 9, 2), (10, 11, 3),
  (12, 13, None)], [(0, 1, 2), (0, 1, 3), (1, 2, None)])
```

Setting `k` to 0 does not change the graph:

```
sage: H = G.copy()
sage: G.subdivide_edge(0, 4, 0)  # the edge: (0, 4, None)
sage: H == G and H.edges(sort=True) == G.edges(sort=True)
True
```

```
>>> from sage.all import *
>>> H = G.copy()
>>> G.subdivide_edge(Integer(0), Integer(4), Integer(0))  # the edge: (0, 4,␣
↪None)
>>> H == G and H.edges(sort=True) == G.edges(sort=True)
True
```

If too many or too less arguments are given, an exception is raised:

```
sage: G.subdivide_edge(0, 1, 'label', 4, 6)
Traceback (most recent call last):
...
ValueError: this method takes at least 2 and at most 4 arguments
sage: G.subdivide_edge((0, 1, 'label', 4))
Traceback (most recent call last):
...
ValueError: this method takes at least 2 and at most 4 arguments
```

```
>>> from sage.all import *
>>> G.subdivide_edge(Integer(0), Integer(1), 'label', Integer(4), Integer(6))
Traceback (most recent call last):
...
ValueError: this method takes at least 2 and at most 4 arguments
>>> G.subdivide_edge((Integer(0), Integer(1), 'label', Integer(4)))
Traceback (most recent call last):
...
ValueError: this method takes at least 2 and at most 4 arguments
```

A `ValueError` is returned for k being in an invalid format or being not an even nonnegative, or for nonexistent or invalid edges:

```
sage: G.subdivide_edge(0, 4, 'label')  # No. of subdivision: 'label'
Traceback (most recent call last):
...
TypeError: '<' not supported between instances of 'str' and 'int'
sage: G.subdivide_edge(0, 4, 3)  # No. of subdivisions: 3
Traceback (most recent call last):
...
ValueError: the number of subdivisions must be a nonnegative even integer,
but found 3
sage: G.subdivide_edge(0, 4, -1)  # No. of subdivisions: -1
Traceback (most recent call last):
...
ValueError: the number of subdivisions must be a nonnegative even integer,
but found -1
sage: G.subdivide_edge(0, 4, 0.5)  # No. of subdivisions: 0.5
Traceback (most recent call last):
...
ValueError: the number of subdivisions must be a nonnegative even integer,
but found 0.500000000000000
sage: G.subdivide_edge(0, 5, 4)
Traceback (most recent call last):
...
ValueError: the given edge (0, 5, None) does not exist
sage: G.subdivide_edge((0, 5), 4)
Traceback (most recent call last):
...
ValueError: the given edge (0, 5, None) does not exist
sage: G.subdivide_edge((0, 1, 'label', None), 4)
Traceback (most recent call last):
```

(continues on next page)

(continued from previous page)

```
...
ValueError: for two input arguments, the first one must be of the form
(u, v) or (u, v, l), but found: (0, 1, 'label', None)
sage: G.subdivide_edge((0), 4)
Traceback (most recent call last):
...
TypeError: object of type 'sage.rings.integer.Integer' has no len()
```

```
>>> from sage.all import *
>>> G.subdivide_edge(Integer(0), Integer(4), 'label')  # No. of subdivision:
↪'label'
Traceback (most recent call last):
...
TypeError: '<' not supported between instances of 'str' and 'int'
>>> G.subdivide_edge(Integer(0), Integer(4), Integer(3))  # No. of␣
↪subdivisions: 3
Traceback (most recent call last):
...
ValueError: the number of subdivisions must be a nonnegative even integer,
but found 3
>>> G.subdivide_edge(Integer(0), Integer(4), -Integer(1))  # No. of␣
↪subdivisions: -1
Traceback (most recent call last):
...
ValueError: the number of subdivisions must be a nonnegative even integer,
but found -1
>>> G.subdivide_edge(Integer(0), Integer(4), RealNumber('0.5'))  # No. of␣
↪subdivisions: 0.5
Traceback (most recent call last):
...
ValueError: the number of subdivisions must be a nonnegative even integer,
but found 0.500000000000000
>>> G.subdivide_edge(Integer(0), Integer(5), Integer(4))
Traceback (most recent call last):
...
ValueError: the given edge (0, 5, None) does not exist
>>> G.subdivide_edge((Integer(0), Integer(5)), Integer(4))
Traceback (most recent call last):
...
ValueError: the given edge (0, 5, None) does not exist
>>> G.subdivide_edge((Integer(0), Integer(1), 'label', None), Integer(4))
Traceback (most recent call last):
...
ValueError: for two input arguments, the first one must be of the form
(u, v) or (u, v, l), but found: (0, 1, 'label', None)
>>> G.subdivide_edge((Integer(0)), Integer(4))
Traceback (most recent call last):
...
TypeError: object of type 'sage.rings.integer.Integer' has no len()
```

See also

- *subdivide_edges()*

subdivide_edges(*edges*, *k*)

Subdivide k times edges from an iterable container.

For more information on the behaviour of this method, please refer to the documentation of *subdivide_edge()*.

Note

This method overwrites the *subdivide_edges()* method to ensure that resultant graph is also matching covered.

INPUT:

- `edges` – an iterable of edges, each of which given either as `(u, v)` or `(u, v, 'label')`
- `k` – a nonnegative even integer; common length of the subdivisions

Note

- If the given edge is labelled with l, all the edges created by the subdivision will have the same label.
- If no label given, the label obtained from *edge_label()* will be used. If multiple such labels are present, the first one in the returned list will be used.
- The number of subdivisions must be a nonnegative even integer in order to ensure the resultant graph is matching covered.
- In the context of matching covered graphs, *bisubdividing* an edge t times is defined as subdividing the edge $2t$ times, for some nonnegative integer t.
- Please note that if a single edge is present in the iterable, it is considered only one time for the subdivision operation.

OUTPUT:

- If existent edges are provided with a valid format and the parameter `k` in the argument is a nonnegative even integer then the graph is updated and nothing is returned, otherwise a `ValueError` is returned if `k` is not a nonnegative even integer,
- If the graph does not contain at least one of the edges provided, a `ValueError` is returned. Also, a `ValueError` is thrown in case the provided arguments are not valid.

EXAMPLES:

Subdividing each edge of K_4 some even number of times. Please note that the perfect matching captured at `self.get_matching()` also gets updated:

```
sage: C = graphs.CycleGraph(4)
sage: G = MatchingCoveredGraph(C)
sage: from collections import Counter
sage: V, E = set(G.vertices()), list(G.edges(sort=True, sort_vertices=True))
```

(continues on next page)

(continued from previous page)

```
sage: M = set(G.get_matching())
sage: G.subdivide_edges(E, 2)
sage: W, F = set(G.vertices()), list(G.edges(sort=True, sort_vertices=True))
sage: N = set(G.get_matching())
sage: sorted(W - V)
[4, 5, 6, 7, 8, 9, 10, 11]
sage: sorted(list((Counter(F) - Counter(E)).elements())), \
....: sorted(list((Counter(E) - Counter(F)).elements()))
([(0, 4, None), (0, 6, None), (1, 5, None), (1, 8, None),
  (2, 9, None), (2, 10, None), (3, 7, None), (3, 11, None),
  (4, 5, None), (6, 7, None), (8, 9, None), (10, 11, None)],
 [(0, 1, None), (0, 3, None), (1, 2, None), (2, 3, None)])
sage: if (0, 1, None) in M:
....:     assert sorted(N - M), sorted(M - N) == \
....:         ([(0, 8, None), (1, 9, None), (2, 4, None), (3, 5, None), \
....:       (6, 7, None), (10, 11, None)], [(0, 1, None), (2, 3, None)])
....: else:
....:     assert sorted(N - M), sorted(M - N) == \
....:         ([(0, 6, None), (1, 10, None), (2, 11, None), (3, 7, None), \
....:           (4, 5, None), (8, 9, None)], [(0, 3, None), (1, 2, None)])
```

```
>>> from sage.all import *
>>> C = graphs.CycleGraph(Integer(4))
>>> G = MatchingCoveredGraph(C)
>>> from collections import Counter
>>> V, E = set(G.vertices()), list(G.edges(sort=True, sort_vertices=True))
>>> M = set(G.get_matching())
>>> G.subdivide_edges(E, Integer(2))
>>> W, F = set(G.vertices()), list(G.edges(sort=True, sort_vertices=True))
>>> N = set(G.get_matching())
>>> sorted(W - V)
[4, 5, 6, 7, 8, 9, 10, 11]
>>> sorted(list((Counter(F) - Counter(E)).elements())),␣
↪sorted(list((Counter(E) - Counter(F)).elements()))
([(0, 4, None), (0, 6, None), (1, 5, None), (1, 8, None),
  (2, 9, None), (2, 10, None), (3, 7, None), (3, 11, None),
  (4, 5, None), (6, 7, None), (8, 9, None), (10, 11, None)],
 [(0, 1, None), (0, 3, None), (1, 2, None), (2, 3, None)])
>>> if (Integer(0), Integer(1), None) in M:
...     assert sorted(N - M), sorted(M - N) ==          ([(Integer(0),␣
↪Integer(8), None), (Integer(1), Integer(9), None), (Integer(2), Integer(4),␣
↪None), (Integer(3), Integer(5), None),        (Integer(6), Integer(7), None),
↪  (Integer(10), Integer(11), None)], [(Integer(0), Integer(1), None),␣
↪(Integer(2), Integer(3), None)])
... else:
...     assert sorted(N - M), sorted(M - N) ==          ([(Integer(0),␣
↪Integer(6), None), (Integer(1), Integer(10), None), (Integer(2),␣
↪Integer(11), None), (Integer(3), Integer(7), None),            (Integer(4),␣
↪Integer(5), None), (Integer(8), Integer(9), None)], [(Integer(0),␣
↪Integer(3), None), (Integer(1), Integer(2), None)])
```

If a single is present multiple times in the iterable, if that many (or more) edges are present in the graph, each

of those edges gets subdivided:

```
sage: C = graphs.CycleGraph(4)
sage: G = MatchingCoveredGraph(C)
sage: G.allow_multiple_edges(True)
sage: G.add_edges([(0, 1)] * 3)
sage: from collections import Counter
sage: V, E = set(G.vertices()), list(G.edges(sort=True, sort_vertices=True))
sage: M = list(G.get_matching())
sage: G.subdivide_edges([(0, 1), (0, 1, None)], 2)
sage: W, F = set(G.vertices()), list(G.edges(sort=True, sort_vertices=True))
sage: N = set(G.get_matching())
sage: sorted(W - V)
[4, 5, 6, 7]
sage: sorted(list((Counter(F) - Counter(E)).elements())), \
....: sorted(list((Counter(E) - Counter(F)).elements()))
([(0, 4, None), (0, 6, None), (1, 5, None), (1, 7, None),
  (4, 5, None), (6, 7, None)], [(0, 1, None), (0, 1, None)])
sage: if (0, 1, None) in M:
....:     assert sorted(N - M), sorted(M - N) == \
....:         ([(0, 4, None), (1, 5, None)], [(0, 1, None)])
....:     assert sorted(N - M), sorted(M - N) == \
....:         ([(4, 5, None)], [])
```

```
>>> from sage.all import *
>>> C = graphs.CycleGraph(Integer(4))
>>> G = MatchingCoveredGraph(C)
>>> G.allow_multiple_edges(True)
>>> G.add_edges([(Integer(0), Integer(1))] * Integer(3))
>>> from collections import Counter
>>> V, E = set(G.vertices()), list(G.edges(sort=True, sort_vertices=True))
>>> M = list(G.get_matching())
>>> G.subdivide_edges([(Integer(0), Integer(1)), (Integer(0), Integer(1),␣
↪None)], Integer(2))
>>> W, F = set(G.vertices()), list(G.edges(sort=True, sort_vertices=True))
>>> N = set(G.get_matching())
>>> sorted(W - V)
[4, 5, 6, 7]
>>> sorted(list((Counter(F) - Counter(E)).elements())),␣
↪sorted(list((Counter(E) - Counter(F)).elements()))
([(0, 4, None), (0, 6, None), (1, 5, None), (1, 7, None),
  (4, 5, None), (6, 7, None)], [(0, 1, None), (0, 1, None)])
>>> if (Integer(0), Integer(1), None) in M:
...     assert sorted(N - M), sorted(M - N) ==          ([(Integer(0),␣
↪Integer(4), None), (Integer(1), Integer(5), None)], [(Integer(0),␣
↪Integer(1), None)])
...     assert sorted(N - M), sorted(M - N) ==          ([(Integer(4),␣
↪Integer(5), None)], [])
```

Subdividing edges with at least one of which is a multiple edge:

```
sage: T = graphs.TricornGraph()
sage: G = MatchingCoveredGraph(T)
```

(continues on next page)

(continued from previous page)

```
sage: G.allow_multiple_edges(True)
sage: G.add_edges([
....:     (0, 1, 2), (0, 1, 3),
....:     (2, 3, 0.5), (2, 3, 4)
....: ])
sage: G.delete_edge(0, 1, None)
sage: from collections import Counter
sage: V, E = set(G.vertices()), list(G.edges(sort=True, sort_vertices=True))
sage: G.subdivide_edges([(0, 1), (1, 2), (2, 3)], 4)
....: # edges considered: (0, 1, 2), (1, 2, None), (2, 3, None)
sage: W, F = set(G.vertices()), list(G.edges(sort=True, sort_vertices=True))
sage: sorted(W - V)
[10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21]
sage: sorted(list((Counter(F) - Counter(E)).elements())), \
....: sorted(list((Counter(E) - Counter(F)).elements()))
([(0, 10, 2), (1, 13, 2), (1, 14, None), (2, 17, None),
  (2, 18, None), (3, 21, None), (10, 11, 2), (11, 12, 2),
  (12, 13, 2), (14, 15, None), (15, 16, None), (16, 17, None),
  (18, 19, None), (19, 20, None), (20, 21, None)],
 [(0, 1, 2), (1, 2, None), (2, 3, None)])
```

```
>>> from sage.all import *
>>> T = graphs.TricornGraph()
>>> G = MatchingCoveredGraph(T)
>>> G.allow_multiple_edges(True)
>>> G.add_edges([
...     (Integer(0), Integer(1), Integer(2)), (Integer(0), Integer(1),␣
↪Integer(3)),
...     (Integer(2), Integer(3), RealNumber('0.5')), (Integer(2), Integer(3),␣
↪Integer(4))
... ])
>>> G.delete_edge(Integer(0), Integer(1), None)
>>> from collections import Counter
>>> V, E = set(G.vertices()), list(G.edges(sort=True, sort_vertices=True))
>>> G.subdivide_edges([(Integer(0), Integer(1)), (Integer(1), Integer(2)),␣
↪(Integer(2), Integer(3))], Integer(4))
... # edges considered: (0, 1, 2), (1, 2, None), (2, 3, None)
>>> W, F = set(G.vertices()), list(G.edges(sort=True, sort_vertices=True))
>>> sorted(W - V)
[10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21]
>>> sorted(list((Counter(F) - Counter(E)).elements())),␣
↪sorted(list((Counter(E) - Counter(F)).elements()))
([(0, 10, 2), (1, 13, 2), (1, 14, None), (2, 17, None),
  (2, 18, None), (3, 21, None), (10, 11, 2), (11, 12, 2),
  (12, 13, 2), (14, 15, None), (15, 16, None), (16, 17, None),
  (18, 19, None), (19, 20, None), (20, 21, None)],
 [(0, 1, 2), (1, 2, None), (2, 3, None)])
```

Setting `k` to 0 does not change the graph:

```
sage: H = G.copy()
sage: G.subdivide_edges([(0, 10), (1, 13), (2, 17)], 0)
```

(continues on next page)

(continued from previous page)

```
sage: H == G and H.edges(sort=True) == G.edges(sort=True)
True
```

```
>>> from sage.all import *
>>> H = G.copy()
>>> G.subdivide_edges([(Integer(0), Integer(10)), (Integer(1), Integer(13)),␣
↪(Integer(2), Integer(17))], Integer(0))
>>> H == G and H.edges(sort=True) == G.edges(sort=True)
True
```

Subdividing edges with at least one of which is nonexistent:

```
sage: G.subdivide_edges([(4, 5), (1, 5)], 4)
....: # edges considered: (4, 5, None), (1, 5, None)
Traceback (most recent call last):
...
ValueError: the given edge (1, 5, None) does not exist
sage: G.subdivide_edges([(0, 4), (0, 4, None)], 4)
Traceback (most recent call last):
...
ValueError: input contains 2 copies of the edge (0, 4, None),
but the graph contains 1
```

```
>>> from sage.all import *
>>> G.subdivide_edges([(Integer(4), Integer(5)), (Integer(1), Integer(5))],␣
↪Integer(4))
... # edges considered: (4, 5, None), (1, 5, None)
Traceback (most recent call last):
...
ValueError: the given edge (1, 5, None) does not exist
>>> G.subdivide_edges([(Integer(0), Integer(4)), (Integer(0), Integer(4),␣
↪None)], Integer(4))
Traceback (most recent call last):
...
ValueError: input contains 2 copies of the edge (0, 4, None),
but the graph contains 1
```

When `k` is not a nonnegative even integer:

```
sage: G.subdivide_edges([(0, 1), (1, 2), (2, 3)], 3)
Traceback (most recent call last):
...
ValueError: the number of subdivisions must be a nonnegative even integer,␣
↪but found 3
```

```
>>> from sage.all import *
>>> G.subdivide_edges([(Integer(0), Integer(1)), (Integer(1), Integer(2)),␣
↪(Integer(2), Integer(3))], Integer(3))
Traceback (most recent call last):
...
ValueError: the number of subdivisions must be a nonnegative even integer,␣
↪but found 3
```

Providing a noniterable object as `edges`:

```
sage: G.subdivide_edges(G.order(), 4)
Traceback (most recent call last):
...
ValueError: expected an iterable of edges, but got a non-iterable object
```

```
>>> from sage.all import *
>>> G.subdivide_edges(G.order(), Integer(4))
Traceback (most recent call last):
...
ValueError: expected an iterable of edges, but got a non-iterable object
```

Providing arguments in an invalid format:

```
sage: G.subdivide_edges([(0, ), (0, 1, 'label')], 4)
Traceback (most recent call last):
...
ValueError: need more than 1 value to unpack for edge: (0,)
sage: G.subdivide_edges([(0, 1, 2, 4), (0, 1, 'label')], 4)
Traceback (most recent call last):
...
ValueError: too many values to unpack (expected 2) for edge: (0, 1, 2, 4)
sage: G.subdivide_edges([0, (0, 1, 'label')], 4)
Traceback (most recent call last):
...
TypeError: input edge 0 is of unknown type
```

```
>>> from sage.all import *
>>> G.subdivide_edges([(Integer(0), ), (Integer(0), Integer(1), 'label')],␣
↪Integer(4))
Traceback (most recent call last):
...
ValueError: need more than 1 value to unpack for edge: (0,)
>>> G.subdivide_edges([(Integer(0), Integer(1), Integer(2), Integer(4)),␣
↪(Integer(0), Integer(1), 'label')], Integer(4))
Traceback (most recent call last):
...
ValueError: too many values to unpack (expected 2) for edge: (0, 1, 2, 4)
>>> G.subdivide_edges([Integer(0), (Integer(0), Integer(1), 'label')],␣
↪Integer(4))
Traceback (most recent call last):
...
TypeError: input edge 0 is of unknown type
```

See also

- *subdivide_edge()*

update_matching(*matching*)

Update the perfect matching captured in `self._matching`.

INPUT:

- `matching` – a perfect matching of the graph, that can be given using any valid input format of *Graph*.

OUTPUT:

- If `matching` is a valid perfect matching of the graph, then `self._matching` gets updated to this provided matching, or otherwise an exception is returned without any alterations to `self._matching`.

EXAMPLES:

Providing with a valid perfect matching of the graph:

```
sage: P = graphs.PetersenGraph()
sage: G = MatchingCoveredGraph(P)
sage: sorted(G.get_matching())
[(0, 5, None), (1, 6, None), (2, 7, None), (3, 8, None), (4, 9, None)]
sage: M = [(0, 1), (2, 3), (4, 9), (5, 7), (6, 8)]
sage: G.update_matching(M)
sage: sorted(G.get_matching())
[(0, 1, None), (2, 3, None), (4, 9, None), (5, 7, None), (6, 8, None)]
```

```
>>> from sage.all import *
>>> P = graphs.PetersenGraph()
>>> G = MatchingCoveredGraph(P)
>>> sorted(G.get_matching())
[(0, 5, None), (1, 6, None), (2, 7, None), (3, 8, None), (4, 9, None)]
>>> M = [(Integer(0), Integer(1)), (Integer(2), Integer(3)), (Integer(4),␣
↪Integer(9)), (Integer(5), Integer(7)), (Integer(6), Integer(8))]
>>> G.update_matching(M)
>>> sorted(G.get_matching())
[(0, 1, None), (2, 3, None), (4, 9, None), (5, 7, None), (6, 8, None)]
```

1.6 View classes

This module implements views for (di)graphs. A view is a read-only iterable container enabling operations like `for e in E` and `e in E`. It is updated as the graph is updated. Hence, the graph should not be updated while iterating through a view. Views can be iterated multiple times.

Todo

- View of neighborhood to get open/close neighborhood of a vertex/set of vertices

1.6.1 Classes

class sage.graphs.views.**EdgesView**

Bases: `object`

EdgesView class.

This class implements a read-only iterable container of edges enabling operations like `for e in E` and `e in E`. An *EdgesView* can be iterated multiple times, and checking membership is done in constant time. It avoids the construction of edge lists and so consumes little memory. It is updated as the graph is updated. Hence, the graph should not be updated while iterating through an *EdgesView*.

INPUT:

- `G` – a (di)graph
- `vertices` – list (default: `None`); an iterable container of vertices or `None`. When set, consider only edges incident to specified vertices.
- `vertices2` – list (default: `None`); an iterable container of vertices or `None`. When set, consider only edges incident to specified vertices. More precisely:
  - When both `vertices` and `vertices2` are set, consider only edges `(u, v, l)` with `u` in `vertices` and `v` in `vertices2`.
  - When `vertices` is `None` and `vertices2` is set, consider only edges `(u, v, l)` with `v` in `vertices2`.
- `labels` – boolean (default: `True`); if `False`, each edge is simply a pair `(u, v)` of vertices
- `ignore_direction` – boolean (default: `False`); only applies to directed graphs. If `True`, searches across edges in either direction.
- `sort` – boolean (default: `False`); whether to sort edges according the ordering specified with parameter `key`. If `False` (default), edges are not sorted. This is the fastest and less memory consuming method for iterating over edges.
- `key` – a function (default: `None`); a function that takes an edge (a pair or a triple, according to the `labels` keyword) as its one argument and returns a value that can be used for comparisons in the sorting algorithm. This parameter is ignored when `sort = False`.
- `sort_vertices` – boolean (default: `True`); whether to sort the ends of the edges; not sorting the ends is faster; only applicable to undirected graphs when `sort` is `False`

Warning

Since any object may be a vertex, there is no guarantee that any two vertices will be comparable, and thus no guarantee how two edges may compare. With default objects for vertices (all integers), or when all the vertices are of the same simple type, then there should not be a problem with how the vertices will be sorted. However, if you need to guarantee a total order for the sorting of the edges, use the `key` argument, as illustrated in the examples below.

EXAMPLES:

```
sage: from sage.graphs.views import EdgesView
sage: G = Graph([(0, 1, 'C'), (0, 2, 'A'), (1, 2, 'B')])
sage: E = EdgesView(G, sort=True); E
[(0, 1, 'C'), (0, 2, 'A'), (1, 2, 'B')]
sage: (1, 2) in E
False
sage: (1, 2, 'B') in E
True
sage: E = EdgesView(G, labels=False, sort=True); E
[(0, 1), (0, 2), (1, 2)]
sage: (1, 2) in E
True
sage: (1, 2, 'B') in E
False
sage: [e for e in E]
[(0, 1), (0, 2), (1, 2)]
```

```
>>> from sage.all import *
>>> from sage.graphs.views import EdgesView
>>> G = Graph([(Integer(0), Integer(1), 'C'), (Integer(0), Integer(2), 'A'),␣
↪(Integer(1), Integer(2), 'B')])
>>> E = EdgesView(G, sort=True); E
[(0, 1, 'C'), (0, 2, 'A'), (1, 2, 'B')]
>>> (Integer(1), Integer(2)) in E
False
>>> (Integer(1), Integer(2), 'B') in E
True
>>> E = EdgesView(G, labels=False, sort=True); E
[(0, 1), (0, 2), (1, 2)]
>>> (Integer(1), Integer(2)) in E
True
>>> (Integer(1), Integer(2), 'B') in E
False
>>> [e for e in E]
[(0, 1), (0, 2), (1, 2)]
```

An *EdgesView* can be iterated multiple times:

```
sage: G = graphs.CycleGraph(3)
sage: print(E)
[(0, 1), (0, 2), (1, 2)]
sage: print(E)
[(0, 1), (0, 2), (1, 2)]
sage: for e in E:
....:     for ee in E:
....:         print((e, ee))
((0, 1), (0, 1))
((0, 1), (0, 2))
((0, 1), (1, 2))
((0, 2), (0, 1))
((0, 2), (0, 2))
((0, 2), (1, 2))
((1, 2), (0, 1))
((1, 2), (0, 2))
((1, 2), (1, 2))
```

```
>>> from sage.all import *
>>> G = graphs.CycleGraph(Integer(3))
>>> print(E)
[(0, 1), (0, 2), (1, 2)]
>>> print(E)
[(0, 1), (0, 2), (1, 2)]
>>> for e in E:
...     for ee in E:
...         print((e, ee))
((0, 1), (0, 1))
((0, 1), (0, 2))
((0, 1), (1, 2))
((0, 2), (0, 1))
((0, 2), (0, 2))
```

(continues on next page)

(continued from previous page)

```
((0, 2), (1, 2))
((1, 2), (0, 1))
((1, 2), (0, 2))
((1, 2), (1, 2))
```

We can check if a view is empty:

```
sage: E = EdgesView(graphs.CycleGraph(3), sort=False)
sage: if E:
....:     print('not empty')
not empty
sage: E = EdgesView(Graph(), sort=False)
sage: if not E:
....:     print('empty')
empty
```

```
>>> from sage.all import *
>>> E = EdgesView(graphs.CycleGraph(Integer(3)), sort=False)
>>> if E:
...     print('not empty')
not empty
>>> E = EdgesView(Graph(), sort=False)
>>> if not E:
...     print('empty')
empty
```

When `sort` is `True`, edges are sorted by default in the default fashion:

```
sage: G = Graph([(0, 1, 'C'), (0, 2, 'A'), (1, 2, 'B')])
sage: E = EdgesView(G, sort=True); E
[(0, 1, 'C'), (0, 2, 'A'), (1, 2, 'B')]
```

```
>>> from sage.all import *
>>> G = Graph([(Integer(0), Integer(1), 'C'), (Integer(0), Integer(2), 'A'),␣
↪(Integer(1), Integer(2), 'B')])
>>> E = EdgesView(G, sort=True); E
[(0, 1, 'C'), (0, 2, 'A'), (1, 2, 'B')]
```

This can be overridden by specifying a key function. This first example just ignores the labels in the third component of the triple:

```
sage: G = Graph([(0, 1, 'C'), (0, 2, 'A'), (1, 2, 'B')])
sage: E = EdgesView(G, sort=True, key=lambda x: (x[1], -x[0])); E
[(0, 1, 'C'), (1, 2, 'B'), (0, 2, 'A')]
```

```
>>> from sage.all import *
>>> G = Graph([(Integer(0), Integer(1), 'C'), (Integer(0), Integer(2), 'A'),␣
↪(Integer(1), Integer(2), 'B')])
>>> E = EdgesView(G, sort=True, key=lambda x: (x[Integer(1)], -x[Integer(0)])); E
[(0, 1, 'C'), (1, 2, 'B'), (0, 2, 'A')]
```

We can also sort according to the labels:

```
sage: G = Graph([(0, 1, 'C'), (0, 2, 'A'), (1, 2, 'B')])
sage: E = EdgesView(G, sort=True, key=lambda x: x[2]); E
[(0, 2, 'A'), (1, 2, 'B'), (0, 1, 'C')]
```

```
>>> from sage.all import *
>>> G = Graph([(Integer(0), Integer(1), 'C'), (Integer(0), Integer(2), 'A'),␣
↪(Integer(1), Integer(2), 'B')])
>>> E = EdgesView(G, sort=True, key=lambda x: x[Integer(2)]); E
[(0, 2, 'A'), (1, 2, 'B'), (0, 1, 'C')]
```

Not sorting the ends of the vertices:

```
sage: G = Graph()
sage: G.add_edges([[1,2], [2,3], [0,3]])
sage: E = EdgesView(G, sort=False, sort_vertices=False); E
[(3, 0, None), (2, 1, None), (3, 2, None)]
```

```
>>> from sage.all import *
>>> G = Graph()
>>> G.add_edges([[Integer(1),Integer(2)], [Integer(2),Integer(3)], [Integer(0),
↪Integer(3)]])
>>> E = EdgesView(G, sort=False, sort_vertices=False); E
[(3, 0, None), (2, 1, None), (3, 2, None)]
```

With a directed graph:

```
sage: G = digraphs.DeBruijn(2, 2)                                              #␣
↪needs sage.combinat
sage: E = EdgesView(G, labels=False, sort=True); E                             #␣
↪needs sage.combinat
[('00', '00'), ('00', '01'), ('01', '10'), ('01', '11'),
 ('10', '00'), ('10', '01'), ('11', '10'), ('11', '11')]
sage: E = EdgesView(G, labels=False, sort=True, key=lambda e:(e[1], e[0])); E  #␣
↪needs sage.combinat
[('00', '00'), ('10', '00'), ('00', '01'), ('10', '01'),
 ('01', '10'), ('11', '10'), ('01', '11'), ('11', '11')]
```

```
>>> from sage.all import *
>>> G = digraphs.DeBruijn(Integer(2), Integer(2))                              ␣
↪              # needs sage.combinat
>>> E = EdgesView(G, labels=False, sort=True); E                               #␣
↪needs sage.combinat
[('00', '00'), ('00', '01'), ('01', '10'), ('01', '11'),
 ('10', '00'), ('10', '01'), ('11', '10'), ('11', '11')]
>>> E = EdgesView(G, labels=False, sort=True, key=lambda e:(e[Integer(1)],␣
↪e[Integer(0)])); E   # needs sage.combinat
[('00', '00'), ('10', '00'), ('00', '01'), ('10', '01'),
 ('01', '10'), ('11', '10'), ('01', '11'), ('11', '11')]
```

We can consider only edges incident to a specified set of vertices:

```
sage: G = graphs.CycleGraph(5)
sage: E = EdgesView(G, vertices=[0, 1], labels=False, sort=True); E
```

(continues on next page)

(continued from previous page)

```
[(0, 1), (0, 4), (1, 2)]
sage: E = EdgesView(G, vertices=[0], labels=False, sort=True); E
[(0, 1), (0, 4)]
sage: E = EdgesView(G, vertices=None, labels=False, sort=True); E
[(0, 1), (0, 4), (1, 2), (2, 3), (3, 4)]

sage: G = digraphs.Circuit(5)
sage: E = EdgesView(G, vertices=[0, 1], labels=False, sort=True); E
[(0, 1), (1, 2)]
```

```
>>> from sage.all import *
>>> G = graphs.CycleGraph(Integer(5))
>>> E = EdgesView(G, vertices=[Integer(0), Integer(1)], labels=False, sort=True);␣
↪E
[(0, 1), (0, 4), (1, 2)]
>>> E = EdgesView(G, vertices=[Integer(0)], labels=False, sort=True); E
[(0, 1), (0, 4)]
>>> E = EdgesView(G, vertices=None, labels=False, sort=True); E
[(0, 1), (0, 4), (1, 2), (2, 3), (3, 4)]

>>> G = digraphs.Circuit(Integer(5))
>>> E = EdgesView(G, vertices=[Integer(0), Integer(1)], labels=False, sort=True);␣
↪E
[(0, 1), (1, 2)]
```

We can ignore the direction of the edges of a directed graph, in which case we search across edges in either direction:

```
sage: G = digraphs.Circuit(5)
sage: E = EdgesView(G, vertices=[0, 1], labels=False, sort=True, ignore_
↪direction=False); E
[(0, 1), (1, 2)]
sage: (1, 0) in E
False
sage: E = EdgesView(G, vertices=[0, 1], labels=False, sort=True, ignore_
↪direction=True); E
[(0, 1), (0, 1), (1, 2), (4, 0)]
sage: (1, 0) in E
True
sage: G.has_edge(1, 0)
False
```

```
>>> from sage.all import *
>>> G = digraphs.Circuit(Integer(5))
>>> E = EdgesView(G, vertices=[Integer(0), Integer(1)], labels=False, sort=True,␣
↪ignore_direction=False); E
[(0, 1), (1, 2)]
>>> (Integer(1), Integer(0)) in E
False
>>> E = EdgesView(G, vertices=[Integer(0), Integer(1)], labels=False, sort=True,␣
↪ignore_direction=True); E
[(0, 1), (0, 1), (1, 2), (4, 0)]
>>> (Integer(1), Integer(0)) in E
```

(continues on next page)

(continued from previous page)

```
True
>>> G.has_edge(Integer(1), Integer(0))
False
```

We can consider only the edges between two specified sets of vertices:

```
sage: G = Graph([(0, 1), (1, 2)])
sage: E = EdgesView(G, vertices=[0], vertices2=[1], labels=False)
sage: (0, 1) in E and (1, 2) not in E
True
sage: E = EdgesView(G, vertices=[0], labels=False)
sage: (0, 1) in E and (1, 2) not in E
True
sage: E = EdgesView(G, vertices2=[1], labels=False)
sage: (0, 1) in E and (1, 2) in E
True
sage: D = DiGraph([(0, 1), (1, 2)])
sage: E = EdgesView(D, vertices=[0], vertices2=[1], labels=False)
sage: (0, 1) in E and (1, 2) not in E
True
sage: EdgesView(D, vertices=[1], vertices2=[0], labels=False)
[]
sage: E = EdgesView(D, vertices=[1], vertices2=[0], labels=False, ignore_
↪direction=True)
sage: (0, 1) in E and (1, 2) not in E
True
sage: E = EdgesView(D, vertices=[0], labels=False)
sage: (0, 1) in E and (1, 2) not in E
True
sage: E = EdgesView(D, vertices2=[1], labels=False)
sage: (0, 1) in E and (1, 2) not in E
True
sage: E = EdgesView(D, vertices2=[1], labels=False, ignore_direction=True)
sage: (0, 1) in E and (1, 2) in E
True
```

```
>>> from sage.all import *
>>> G = Graph([(Integer(0), Integer(1)), (Integer(1), Integer(2))])
>>> E = EdgesView(G, vertices=[Integer(0)], vertices2=[Integer(1)], labels=False)
>>> (Integer(0), Integer(1)) in E and (Integer(1), Integer(2)) not in E
True
>>> E = EdgesView(G, vertices=[Integer(0)], labels=False)
>>> (Integer(0), Integer(1)) in E and (Integer(1), Integer(2)) not in E
True
>>> E = EdgesView(G, vertices2=[Integer(1)], labels=False)
>>> (Integer(0), Integer(1)) in E and (Integer(1), Integer(2)) in E
True
>>> D = DiGraph([(Integer(0), Integer(1)), (Integer(1), Integer(2))])
>>> E = EdgesView(D, vertices=[Integer(0)], vertices2=[Integer(1)], labels=False)
>>> (Integer(0), Integer(1)) in E and (Integer(1), Integer(2)) not in E
True
>>> EdgesView(D, vertices=[Integer(1)], vertices2=[Integer(0)], labels=False)
```

(continues on next page)

(continued from previous page)

```
[]
>>> E = EdgesView(D, vertices=[Integer(1)], vertices2=[Integer(0)], labels=False,␣
↪ignore_direction=True)
>>> (Integer(0), Integer(1)) in E and (Integer(1), Integer(2)) not in E
True
>>> E = EdgesView(D, vertices=[Integer(0)], labels=False)
>>> (Integer(0), Integer(1)) in E and (Integer(1), Integer(2)) not in E
True
>>> E = EdgesView(D, vertices2=[Integer(1)], labels=False)
>>> (Integer(0), Integer(1)) in E and (Integer(1), Integer(2)) not in E
True
>>> E = EdgesView(D, vertices2=[Integer(1)], labels=False, ignore_direction=True)
>>> (Integer(0), Integer(1)) in E and (Integer(1), Integer(2)) in E
True
```

A view is updated as the graph is updated:

```
sage: G = Graph()
sage: E = EdgesView(G, vertices=[0, 3], labels=False, sort=True); E
[]
sage: G.add_edges([(0, 1), (1, 2)])
sage: E
[(0, 1)]
sage: G.add_edge(2, 3)
sage: E
[(0, 1), (2, 3)]
```

```
>>> from sage.all import *
>>> G = Graph()
>>> E = EdgesView(G, vertices=[Integer(0), Integer(3)], labels=False, sort=True);␣
↪E
[]
>>> G.add_edges([(Integer(0), Integer(1)), (Integer(1), Integer(2))])
>>> E
[(0, 1)]
>>> G.add_edge(Integer(2), Integer(3))
>>> E
[(0, 1), (2, 3)]
```

Hence, the graph should not be updated while iterating through a view:

```
sage: G = Graph([('a', 'b'), ('b', 'c')])
sage: E = EdgesView(G, labels=False, sort=False); E
[('a', 'b'), ('b', 'c')]
sage: for u, v in E:
....:     G.add_edge(u + u, v + v)
Traceback (most recent call last):
...
RuntimeError: dictionary changed size during iteration
```

```
>>> from sage.all import *
>>> G = Graph([('a', 'b'), ('b', 'c')])
```

(continues on next page)

(continued from previous page)

```
>>> E = EdgesView(G, labels=False, sort=False); E
[('a', 'b'), ('b', 'c')]
>>> for u, v in E:
...     G.add_edge(u + u, v + v)
Traceback (most recent call last):
...
RuntimeError: dictionary changed size during iteration
```

Two *EdgesView* are considered equal if they report either both directed, or both undirected edges, they have the same settings for `ignore_direction`, they have the same settings for `labels`, and they report the same edges in the same order:

```
sage: G = graphs.HouseGraph()
sage: EG = EdgesView(G, sort=False)
sage: H = Graph(EG)
sage: EH = EdgesView(H, sort=False)
sage: EG == EH
True
sage: G.add_edge(0, 10)
sage: EG = EdgesView(G, sort=False)
sage: EG == EH
False
sage: H.add_edge(0, 10)
sage: EH = EdgesView(H, sort=False)
sage: EG == EH
True
sage: H = G.strong_orientation()
sage: EH = EdgesView(H, sort=False)
sage: EG == EH
False
```

```
>>> from sage.all import *
>>> G = graphs.HouseGraph()
>>> EG = EdgesView(G, sort=False)
>>> H = Graph(EG)
>>> EH = EdgesView(H, sort=False)
>>> EG == EH
True
>>> G.add_edge(Integer(0), Integer(10))
>>> EG = EdgesView(G, sort=False)
>>> EG == EH
False
>>> H.add_edge(Integer(0), Integer(10))
>>> EH = EdgesView(H, sort=False)
>>> EG == EH
True
>>> H = G.strong_orientation()
>>> EH = EdgesView(H, sort=False)
>>> EG == EH
False
```

The sum of two *EdgesView* is a list containing the edges in both *EdgesView*:

```
sage: E1 = EdgesView(Graph([(0, 1)]), labels=False, sort=False)
sage: E2 = EdgesView(Graph([(2, 3)]), labels=False, sort=False)
sage: E1 + E2
[(0, 1), (2, 3)]
sage: E2 + E1
[(2, 3), (0, 1)]
```

```
>>> from sage.all import *
>>> E1 = EdgesView(Graph([(Integer(0), Integer(1))]), labels=False, sort=False)
>>> E2 = EdgesView(Graph([(Integer(2), Integer(3))]), labels=False, sort=False)
>>> E1 + E2
[(0, 1), (2, 3)]
>>> E2 + E1
[(2, 3), (0, 1)]
```

Recall that a *EdgesView* is read-only and that this method returns a list:

```
sage: E1 += E2
sage: type(E1) is list
True
```

```
>>> from sage.all import *
>>> E1 += E2
>>> type(E1) is list
True
```

It is also possible to get the sum a *EdgesView* with itself n times:

```
sage: E = EdgesView(Graph([(0, 1), (2, 3)]), labels=False, sort=True)
sage: E * 3
[(0, 1), (2, 3), (0, 1), (2, 3), (0, 1), (2, 3)]
sage: 3 * E
[(0, 1), (2, 3), (0, 1), (2, 3), (0, 1), (2, 3)]
```

```
>>> from sage.all import *
>>> E = EdgesView(Graph([(Integer(0), Integer(1)), (Integer(2), Integer(3))]),␣
↪labels=False, sort=True)
>>> E * Integer(3)
[(0, 1), (2, 3), (0, 1), (2, 3), (0, 1), (2, 3)]
>>> Integer(3) * E
[(0, 1), (2, 3), (0, 1), (2, 3), (0, 1), (2, 3)]
```

Recall that a *EdgesView* is read-only and that this method returns a list:

```
sage: E *= 2
sage: type(E) is list
True
```

```
>>> from sage.all import *
>>> E *= Integer(2)
>>> type(E) is list
True
```

We can ask for the i-th edge, or a slice of the edges as a list:

```
sage: E = EdgesView(graphs.HouseGraph(), labels=False, sort=True)
sage: E[0]
(0, 1)
sage: E[2]
(1, 3)
sage: E[-1]
(3, 4)
sage: E[1:-1]
[(0, 2), (1, 3), (2, 3), (2, 4)]
sage: E[::-1]
[(3, 4), (2, 4), (2, 3), (1, 3), (0, 2), (0, 1)]
```

```
>>> from sage.all import *
>>> E = EdgesView(graphs.HouseGraph(), labels=False, sort=True)
>>> E[Integer(0)]
(0, 1)
>>> E[Integer(2)]
(1, 3)
>>> E[-Integer(1)]
(3, 4)
>>> E[Integer(1):-Integer(1)]
[(0, 2), (1, 3), (2, 3), (2, 4)]
>>> E[::-Integer(1)]
[(3, 4), (2, 4), (2, 3), (1, 3), (0, 2), (0, 1)]
```

CHAPTER

TWO

CONSTRUCTORS AND DATABASES

2.1 Common graphs

All graphs in Sage can be built through the `graphs` object. In order to build a complete graph on 15 elements, one can do:

```
sage: g = graphs.CompleteGraph(15)
```

```
>>> from sage.all import *
>>> g = graphs.CompleteGraph(Integer(15))
```

sage.graphs.graph_generators.graphs = <sage.graphs.graph_generators.GraphGenerators object>

A class consisting of constructors for several common graphs, as well as orderly generation of isomorphism class representatives. See the *module's help* for a list of supported constructors.

A list of all graphs and graph structures (other than isomorphism class representatives) in this database is available via tab completion. Type "graphs." and then hit the `Tab` key to see which graphs are available.

The docstrings include educational information about each named graph with the hopes that this class can be used as a reference.

For all the constructors in this class (except the octahedral, dodecahedral, random and empty graphs), the position dictionary is filled to override the spring-layout algorithm.

ORDERLY GENERATION:

```
graphs(vertices, property=lambda x: True, augment='edges', size=None)
```

This syntax accesses the generator of isomorphism class representatives. Iterates over distinct, exhaustive representatives.

Also: see the use of the nauty package for generating graphs at the *nauty_geng()* method.

INPUT:

- `vertices` – a natural number or `None` to infinitely generate bigger and bigger graphs
- `property` – (default: `lambda x: True`) any property to be tested on graphs before generation, but note that in general the graphs produced are not the same as those produced by using the property function to filter a list of graphs produced by using the `lambda x: True` default. The generation process assumes the property has certain characteristics set by the `augment` argument, and only in the case of inherited properties such that all subgraphs of the relevant kind (for `augment='edges'` or `augment='vertices'`) of a graph with the property also possess the property will there be no missing graphs. (The `property` argument is ignored if `degree_sequence` is specified.)

- `augment` – (default: `'edges'`) possible values:
  - `'edges'` – augments a fixed number of vertices by adding one edge. In this case, all graphs on *exactly* `n=vertices` are generated. If for any graph G satisfying the property, every subgraph, obtained from G by deleting one edge but not the vertices incident to that edge, satisfies the property, then this will generate all graphs with that property. If this does not hold, then all the graphs generated will satisfy the property, but there will be some missing.
  - `'vertices'` – augments by adding a vertex and edges incident to that vertex. In this case, all graphs *up to* `n=vertices` are generated. If for any graph G satisfying the property, every subgraph, obtained from G by deleting one vertex and only edges incident to that vertex, satisfies the property, then this will generate all graphs with that property. If this does not hold, then all the graphs generated will satisfy the property, but there will be some missing.
- `size` – (default: `None`) the size of the graph to be generated
- `degree_sequence` – (default: `None`) a sequence of nonnegative integers, or `None`. If specified, the generated graphs will have these integers for degrees. In this case, property and size are both ignored.
- `loops` – boolean (default: `False`); whether to allow loops in the graph or not
- `sparse` – (default: `True`) whether to use a sparse or dense data structure. See the documentation of *Graph*.
- `copy` – boolean (default: `True`); whether to return copies. If set to `False` the method returns the graph it is working on. The second alternative is faster, but modifying any of the graph instances returned by the method may break the function's behaviour, as it is using these graphs to compute the next ones: only use `copy=False` when you stick to *reading* the graphs returned.

  This parameter is ignored when `immutable` is set to `True`, in which case returned graphs are always copies.
- `immutable` – boolean (default: `False`); whether to return immutable or mutable graphs. When set to `True`, this parameter implies `copy=True`.

EXAMPLES:

Print graphs on 3 or less vertices:

```
sage: for G in graphs(3, augment='vertices'):
....:     print(G)
Graph on 0 vertices
Graph on 1 vertex
Graph on 2 vertices
Graph on 3 vertices
Graph on 3 vertices
Graph on 3 vertices
Graph on 2 vertices
Graph on 3 vertices
```

```
>>> from sage.all import *
>>> for G in graphs(Integer(3), augment='vertices'):
...     print(G)
Graph on 0 vertices
Graph on 1 vertex
Graph on 2 vertices
Graph on 3 vertices
Graph on 3 vertices
Graph on 3 vertices
Graph on 2 vertices
Graph on 3 vertices
```

Print graphs on 3 vertices.

```
sage: for G in graphs(3):
....:     print(G)
Graph on 3 vertices
Graph on 3 vertices
Graph on 3 vertices
Graph on 3 vertices
```

```
>>> from sage.all import *
>>> for G in graphs(Integer(3)):
...     print(G)
Graph on 3 vertices
Graph on 3 vertices
Graph on 3 vertices
Graph on 3 vertices
```

Generate all graphs with 5 vertices and 4 edges.

```
sage: L = graphs(5, size=4)
sage: len(list(L))
6
```

```
>>> from sage.all import *
>>> L = graphs(Integer(5), size=Integer(4))
>>> len(list(L))
6
```

Generate all graphs with 5 vertices and up to 4 edges.

```
sage: L = list(graphs(5, lambda G: G.size() <= 4))
sage: len(L)
14
sage: graphs_list.show_graphs(L)         # long time                              #␣
↪needs sage.plot
```

```
>>> from sage.all import *
>>> L = list(graphs(Integer(5), lambda G: G.size() <= Integer(4)))
>>> len(L)
14
>>> graphs_list.show_graphs(L)         # long time                               #␣
↪needs sage.plot
```

Generate all graphs with up to 5 vertices and up to 4 edges.

```
sage: L = list(graphs(5, lambda G: G.size() <= 4, augment='vertices'))
sage: len(L)
31
sage: graphs_list.show_graphs(L)         # long time                              #␣
↪needs sage.plot
```

```
>>> from sage.all import *
>>> L = list(graphs(Integer(5), lambda G: G.size() <= Integer(4), augment=
```

(continues on next page)

(continued from previous page)

```
↪'vertices'))
>>> len(L)
31
>>> graphs_list.show_graphs(L)         # long time                                #␣
↪needs sage.plot
```

Generate all graphs with degree at most 2, up to 6 vertices.

```
sage: property = lambda G: ( max([G.degree(v) for v in G] + [0]) <= 2 )
sage: L = list(graphs(6, property, augment='vertices'))
sage: len(L)
45
```

```
>>> from sage.all import *
>>> property = lambda G: ( max([G.degree(v) for v in G] + [Integer(0)]) <=␣
↪Integer(2) )
>>> L = list(graphs(Integer(6), property, augment='vertices'))
>>> len(L)
45
```

Generate all bipartite graphs on up to 7 vertices: (see OEIS sequence A033995)

```
sage: L = list( graphs(7, lambda G: G.is_bipartite(), augment='vertices') )
sage: [len([g for g in L if g.order() == i]) for i in [1..7]]
[1, 2, 3, 7, 13, 35, 88]
```

```
>>> from sage.all import *
>>> L = list( graphs(Integer(7), lambda G: G.is_bipartite(), augment='vertices') )
>>> [len([g for g in L if g.order() == i]) for i in (ellipsis_range(Integer(1),
↪Ellipsis,Integer(7)))]
[1, 2, 3, 7, 13, 35, 88]
```

Generate all bipartite graphs on exactly 7 vertices:

```
sage: L = list( graphs(7, lambda G: G.is_bipartite()) )
sage: len(L)
88
```

```
>>> from sage.all import *
>>> L = list( graphs(Integer(7), lambda G: G.is_bipartite()) )
>>> len(L)
88
```

Generate all bipartite graphs on exactly 8 vertices:

```
sage: L = list( graphs(8, lambda G: G.is_bipartite()) ) # long time
sage: len(L)                                            # long time
303
```

```
>>> from sage.all import *
>>> L = list( graphs(Integer(8), lambda G: G.is_bipartite()) ) # long time
```

(continues on next page)

(continued from previous page)

```
>>> len(L)                                                    # long time
303
```

Remember that the property argument does not behave as a filter, except for appropriately inheritable properties:

```
sage: property = lambda G: G.is_vertex_transitive()
sage: len(list(graphs(4, property)))                                    #␣
↪needs sage.groups
1
sage: sum(1 for g in graphs(4) if property(g))                          #␣
↪needs sage.groups
4

sage: property = lambda G: G.is_bipartite()
sage: len(list(graphs(4, property)))
7
sage: sum(1 for g in graphs(4) if property(g))
7
```

```
>>> from sage.all import *
>>> property = lambda G: G.is_vertex_transitive()
>>> len(list(graphs(Integer(4), property)))                                ␣
↪     # needs sage.groups
1
>>> sum(Integer(1) for g in graphs(Integer(4)) if property(g))             ␣
↪              # needs sage.groups
4

>>> property = lambda G: G.is_bipartite()
>>> len(list(graphs(Integer(4), property)))
7
>>> sum(Integer(1) for g in graphs(Integer(4)) if property(g))
7
```

Generate graphs on the fly: (see OEIS sequence A000088)

```
sage: for i in range(7):
....:     print(len(list(graphs(i))))
1
1
2
4
11
34
156
```

```
>>> from sage.all import *
>>> for i in range(Integer(7)):
...     print(len(list(graphs(i))))
1
1
2
```

(continues on next page)

(continued from previous page)

```
4
11
34
156
```

Generate all simple graphs, allowing loops: (see OEIS sequence A000666)

```
sage: L = list(graphs(5,augment='vertices',loops=True))                # long time
sage: for i in [0..5]:  # long time
....:     print((i, len([g for g in L if g.order() == i])))
(0, 1)
(1, 2)
(2, 6)
(3, 20)
(4, 90)
(5, 544)
```

```
>>> from sage.all import *
>>> L = list(graphs(Integer(5),augment='vertices',loops=True))                #␣
↪long time
>>> for i in (ellipsis_range(Integer(0),Ellipsis,Integer(5))):  # long time
...     print((i, len([g for g in L if g.order() == i])))
(0, 1)
(1, 2)
(2, 6)
(3, 20)
(4, 90)
(5, 544)
```

Generate all graphs with a specified degree sequence (see OEIS sequence A002851):

```
sage: for i in [4,6,8]:  # long time (4s on sage.math, 2012)
....:     print((i, len([g for g in graphs(i, degree_sequence=[3]*i) if g.is_
↪connected()])))
(4, 1)
(6, 2)
(8, 5)
sage: for i in [4,6,8]:  # long time (7s on sage.math, 2012)
....:     print((i, len([g for g in graphs(i, augment='vertices', degree_
↪sequence=[3]*i) if g.is_connected()])))
(4, 1)
(6, 2)
(8, 5)
```

```
>>> from sage.all import *
>>> for i in [Integer(4),Integer(6),Integer(8)]:  # long time (4s on sage.math,␣
↪2012)
...     print((i, len([g for g in graphs(i, degree_sequence=[Integer(3)]*i) if g.
↪is_connected()])))
(4, 1)
(6, 2)
(8, 5)
```

(continues on next page)

(continued from previous page)

```
>>> for i in [Integer(4),Integer(6),Integer(8)]:  # long time (7s on sage.math,␣
↪2012)
...     print((i, len([g for g in graphs(i, augment='vertices', degree_
↪sequence=[Integer(3)]*i) if g.is_connected()])))
(4, 1)
(6, 2)
(8, 5)
```

```
sage: print((10, len([g for g in graphs(10,degree_sequence=[3]*10) if g.is_
↪connected()]))) # not tested
(10, 19)
```

```
>>> from sage.all import *
>>> print((Integer(10), len([g for g in graphs(Integer(10),degree_
↪sequence=[Integer(3)]*Integer(10)) if g.is_connected()]))) # not tested
(10, 19)
```

Make sure that the graphs are really independent and the generator survives repeated vertex removal (Issue #8458):

```
sage: for G in graphs(3):
....:     G.delete_vertex(0)
....:     print(G.order())
2
2
2
2
```

```
>>> from sage.all import *
>>> for G in graphs(Integer(3)):
...     G.delete_vertex(Integer(0))
...     print(G.order())
2
2
2
2
```

Returned graphs can be mutable or immutable:

```
sage: G = next(graphs(3, immutable=False))
sage: G.delete_vertex(0)
sage: G = next(graphs(3, immutable=True))
sage: G.delete_vertex(0)
Traceback (most recent call last):
...
TypeError: this graph is immutable and so cannot be changed
sage: G = next(graphs(4, degree_sequence=[3]*4))
sage: G.delete_vertex(0)
sage: G = next(graphs(4, degree_sequence=[3]*4, immutable=True))
sage: G.delete_vertex(0)
Traceback (most recent call last):
...
TypeError: this graph is immutable and so cannot be changed
```

```
>>> from sage.all import *
>>> G = next(graphs(Integer(3), immutable=False))
>>> G.delete_vertex(Integer(0))
>>> G = next(graphs(Integer(3), immutable=True))
>>> G.delete_vertex(Integer(0))
Traceback (most recent call last):
...
TypeError: this graph is immutable and so cannot be changed
>>> G = next(graphs(Integer(4), degree_sequence=[Integer(3)]*Integer(4)))
>>> G.delete_vertex(Integer(0))
>>> G = next(graphs(Integer(4), degree_sequence=[Integer(3)]*Integer(4),␣
↪immutable=True))
>>> G.delete_vertex(Integer(0))
Traceback (most recent call last):
...
TypeError: this graph is immutable and so cannot be changed
```

REFERENCE:

- Brendan D. McKay, Isomorph-Free Exhaustive generation. *Journal of Algorithms*, Volume 26, Issue 2, February 1998, pages 306-324.

To get a path with 4 vertices, and the house graph:

```
sage: p = graphs.PathGraph(4)
sage: h = graphs.HouseGraph()
```

```
>>> from sage.all import *
>>> p = graphs.PathGraph(Integer(4))
>>> h = graphs.HouseGraph()
```

More interestingly, one can get the list of all graphs that Sage knows how to build by typing `graphs.` in Sage and then hitting `Tab`.

Basic structures

BullGraph	*DiamondGraph*	*HouseXGraph*
ButterflyGraph	*GemGraph*	*LadderGraph*
CircularLadderGraph	*DartGraph*	*LollipopGraph*
ClawGraph	*ForkGraph*	*MoebiusLadderGraph*
CycleGraph	*DipoleGraph*	*PathGraph*
CompleteBipartiteGraph	*EmptyGraph*	*StarGraph*
CompleteGraph	*Grid2dGraph*	*TadpoleGraph*
CompleteMultipartiteGraph	*GridGraph*	*ToroidalGrid2dGraph*
CorrelationGraph	*HouseGraph*	*Toroidal6RegularGrid2dGraph*

Small Graphs

A small graph is just a single graph and has no parameter influencing the number of edges or vertices.

Balaban10Cage	*GossetGraph*	*MeredithGraph*
Balaban11Cage	*graph_3O73*	*MoebiusKantorGraph*

continues on next page

Table 1 – continued from previous page

BidiakisCube	*GrayGraph*	*MoserSpindle*
BiggsSmithGraph	*GritsenkoGraph*	*MurtyGraph*
BlanusaFirstSnarkGraph	*GrotzschGraph*	*NauruGraph*
BlanusaSecondSnarkGraph	*HallJankoGraph*	*PappusGraph*
BrinkmannGraph	*HarborthGraph*	*PoussinGraph*
BrouwerHaemersGraph	*HarriesGraph*	*PerkelGraph*
BuckyBall	*HarriesWongGraph*	*PetersenGraph*
CameronGraph	*HeawoodGraph*	*RobertsonGraph*
Cell600	*HerschelGraph*	*SchlaefliGraph*
Cell120	*HigmanSimsGraph*	*shortened_00_11_binary_Golay_code_graph*
ChvatalGraph	*HoffmanGraph*	*shortened_000_111_extended_binary_Golay_code_graph*
ClebschGraph	*HoffmanSingletonGraph*	*ShrikhandeGraph*
cocliques_HoffmannSingleton	*HoltGraph*	*SimsGewirtzGraph*
ConwaySmith_for_3S7	*HortonGraph*	*SousselierGraph*
CoxeterGraph	*IoninKharaghani765Graph*	*SylvesterGraph*
CubeplexGraph	*IvanovIvanovFaradjevGraph*	*SzekeresSnarkGraph*
DesarguesGraph	*J2Graph*	*ThomsenGraph*
DejterGraph	*JankoKharaghaniGraph*	*TietzeGraph*
distance_3_doubly_truncated_Golay_code_graph	*JankoKharaghaniTonchevGraph*	*TricornGraph*
DoubleStarSnark	*KittellGraph*	*TruncatedIcosidodecahedralGraph*
DoublyTruncatedWittGraph	*KrackhardtKiteGraph*	*TruncatedTetrahedralGraph*
DurerGraph	*Klein3RegularGraph*	*TruncatedWittGraph*
DyckGraph	*Klein7RegularGraph*	*Tutte12Cage*
EllinghamHorton54Graph	*LargeWittGraph*	*TutteCoxeterGraph*
EllinghamHorton78Graph	*LeonardGraph*	*TutteGraph*
ErreraGraph	*LjubljanaGraph*	*TwinplexGraph*
F26AGraph	*vanLintSchrijverGraph*	*U42Graph216*
FlowerSnark	*LivingstoneGraph*	*U42Graph540*
FolkmanGraph	*locally_GQ42_distance_transitive_graph*	*WagnerGraph*
FosterGraph	*LocalMcLaughlinGraph*	*WatkinsSnarkGraph*
FosterGraph3S6	*M22Graph*	*WellsGraph*
FranklinGraph	*MarkstroemGraph*	*WienerArayaGraph*
FruchtGraph	*MathonStronglyRegularGraph*	*SuzukiGraph*
GoldnerHararyGraph	*McGeeGraph*	
GolombGraph	*McLaughlinGraph*	

Platonic solids (ordered ascending by number of vertices)

TetrahedralGraph	*HexahedralGraph*	*DodecahedralGraph*
OctahedralGraph	*IcosahedralGraph*	

Families of graphs

A family of graphs is a set of graphs, which can be indexed by fixed number of parameters, e.g. two integer parameters. (A method whose name starts with a small letter does not return a single graph object but a graph iterator or a list of

graphs or …)

AlternatingFormsGraph	GeneralisedDodecagonGraph	NKStarGraph
AztecDiamondGraph	GeneralisedHexagonGraph	NStarGraph
BarbellGraph	GeneralisedOctagonGraph	OddGraph
BilinearFormsGraph	GeneralizedPetersenGraph	PaleyGraph
BiwheelGraph	GeneralizedSierpinskiGraph	PasechnikGraph
BubbleSortGraph	GoethalsSeidelGraph	petersen_family
CaiFurerImmermanGraph	GrassmannGraph	p2_forbidden_minors
chang_graphs	HalfCube	planar_graphs
CirculantGraph	HammingGraph	plantri_gen
cographs	HanoiTowerGraph	quadrangulations
cospectral_graphs	HararyGraph	RingedTree
CubeGraph	HermitianFormsGraph	SierpinskiGasketGraph
CubeConnectedCycle	HyperStarGraph	SquaredSkewHadamardMatrix-Graph
distance_regular_graph	JohnsonGraph	SwitchedSquaredSke-wHadamardMatrixGraph
DorogovtsevGoltsevMendes-Graph	KneserGraph	StaircaseGraph
DoubleGrassmannGraph	LCFGraph	strongly_regular_graph
DoubleOddGraph	line_graph_forbidden_sub-graphs	TruncatedBiwheelGraph
EgawaGraph	MathonPseudocyclicMerging-Graph	triangulations
FoldedCubeGraph	MathonPseudocyclicStrong-lyRegularGraph	TuranGraph
FriendshipGraph	MuzychukS6Graph	UstimenkoGraph
fullerenes	MycielskiGraph	WheelGraph
FurerGadget	MycielskiStep	WindmillGraph
fusenes	nauty_geng	
FuzzyBallGraph	nauty_genbg	

Graphs defined by systems of equations

Akq	LUWGraph
Dkq	WengerGraph

Graphs from classical geometries over finite fields

A number of classes of graphs related to geometries over finite fields and quadrics and Hermitean varieties there.

AffineOrthogonalPolarGraph	*SymplecticDualPolarGraph*	*HaemersGraph*
AhrensSzekeresGeneralizedQuadrangleGraph	*SymplecticPolarGraph*	*CossidentePenttilaGraph*
NonisotropicOrthogonalPolarGraph	*TaylorTwographDescendantSRG*	*UnitaryDualPolarGraph*
NonisotropicUnitaryPolarGraph	*TaylorTwographSRG*	*UnitaryPolarGraph*
OrthogonalDualPolarGraph	*T2starGeneralizedQuadrangleGraph*	
OrthogonalPolarGraph	*Nowhere0WordsTwoWeightCodeGraph*	

Chessboard Graphs

BishopGraph	*KnightGraph*	*RookGraph*
KingGraph	*QueenGraph*	

Intersection graphs

These graphs are generated by geometric representations. The objects of the representation correspond to the graph vertices and the intersections of objects yield the graph edges.

IntersectionGraph	*OrthogonalArrayBlockGraph*	*ToleranceGraph*
IntervalGraph	*PermutationGraph*	

Random graphs

RandomBarabasiAlbert	*RandomGNM*	*RandomProperIntervalGraph*
RandomBicubicPlanar	*RandomGNP*	*RandomRegular*
RandomBipartite	*RandomHolmeKim*	*RandomShell*
RandomRegularBipartite	*RandomIntervalGraph*	*RandomToleranceGraph*
RandomBlockGraph	*RandomKTree*	*RandomTriangulation*
RandomBoundedToleranceGraph	*RandomPartialKTree*	*RandomUnitDiskGraph*
RandomChordalGraph	*RandomNewmanWattsStrogatz*	

Trees

BalancedTree	*RandomLobster*	*trees*
FibonacciTree	*RandomTree*	*nauty_gentreeg*
Caterpillar	*RandomTreePowerlaw*	

Graphs with a given degree sequence

DegreeSequence	*DegreeSequenceConfigurationModel*	*DegreeSequenceTree*
DegreeSequenceBipartite	*DegreeSequenceExpected*	

Miscellaneous

WorldMap	*AfricaMap*
EuropeMap	*USAMap*

AUTHORS:

- Robert Miller (2006-11-05): initial version, empty, random, petersen
- Emily Kirkman (2006-11-12): basic structures, node positioning for all constructors
- Emily Kirkman (2006-11-19): docstrings, examples
- William Stein (2006-12-05): Editing.
- Robert Miller (2007-01-16): Cube generation and plotting
- Emily Kirkman (2007-01-16): more basic structures, docstrings
- Emily Kirkman (2007-02-14): added more named graphs
- Robert Miller (2007-06-08-11): Platonic solids, random graphs, graphs with a given degree sequence, random directed graphs
- Robert Miller (2007-10-24): Isomorph free exhaustive generation
- Nathann Cohen (2009-08-12): WorldMap
- Michael Yurko (2009-9-01): added hyperstar, (n,k)-star, n-star, and bubblesort graphs
- Anders Jonsson (2009-10-15): added generalized Petersen graphs
- Harald Schilly and Yann Laigle-Chapuy (2010-03-24): added Fibonacci Tree
- Jason Grout (2010-06-04): cospectral_graphs
- Edward Scheinerman (2010-08-11): RandomTree
- Ed Scheinerman (2010-08-21): added Grotzsch graph and Mycielski graphs
- Ed Scheinerman (2010-11-15): added RandomTriangulation
- Minh Van Nguyen (2010-11-26): added more named graphs
- Keshav Kini (2011-02-16): added Shrikhande and Dyck graphs
- David Coudert (2012-02-10): new RandomGNP generator
- David Coudert (2012-08-02): added chessboard graphs: Queen, King, Knight, Bishop, and Rook graphs
- Nico Van Cleemput (2013-05-26): added fullerenes
- Nico Van Cleemput (2013-07-01): added benzenoids
- Birk Eisermann (2013-07-29): new section 'intersection graphs', added (random, bounded) tolerance graphs
- Marco Cognetta (2016-03-03): added TuranGraph
- Janmenjaya Panda (2024-05-26): added MoebiusLadderGraph
- Janmenjaya Panda (2024-06-09): added StaircaseGraph, BiwheelGraph and TruncatedBiwheelGraph

2.1.1 Functions and methods

class sage.graphs.graph_generators.**GraphGenerators**

Bases: object

A class consisting of constructors for several common graphs, as well as orderly generation of isomorphism class representatives. See the *module's help* for a list of supported constructors.

A list of all graphs and graph structures (other than isomorphism class representatives) in this database is available via tab completion. Type "graphs." and then hit the Tab key to see which graphs are available.

The docstrings include educational information about each named graph with the hopes that this class can be used as a reference.

For all the constructors in this class (except the octahedral, dodecahedral, random and empty graphs), the position dictionary is filled to override the spring-layout algorithm.

ORDERLY GENERATION:

```
graphs(vertices, property=lambda x: True, augment='edges', size=None)
```

This syntax accesses the generator of isomorphism class representatives. Iterates over distinct, exhaustive representatives.

Also: see the use of the nauty package for generating graphs at the *nauty_geng()* method.

INPUT:

- vertices – a natural number or None to infinitely generate bigger and bigger graphs
- property – (default: lambda x: True) any property to be tested on graphs before generation, but note that in general the graphs produced are not the same as those produced by using the property function to filter a list of graphs produced by using the lambda x: True default. The generation process assumes the property has certain characteristics set by the augment argument, and only in the case of inherited properties such that all subgraphs of the relevant kind (for augment='edges' or augment='vertices') of a graph with the property also possess the property will there be no missing graphs. (The property argument is ignored if degree_sequence is specified.)
- augment – (default: 'edges') possible values:
  - 'edges' – augments a fixed number of vertices by adding one edge. In this case, all graphs on *exactly* n=vertices are generated. If for any graph G satisfying the property, every subgraph, obtained from G by deleting one edge but not the vertices incident to that edge, satisfies the property, then this will generate all graphs with that property. If this does not hold, then all the graphs generated will satisfy the property, but there will be some missing.
  - 'vertices' – augments by adding a vertex and edges incident to that vertex. In this case, all graphs *up to* n=vertices are generated. If for any graph G satisfying the property, every subgraph, obtained from G by deleting one vertex and only edges incident to that vertex, satisfies the property, then this will generate all graphs with that property. If this does not hold, then all the graphs generated will satisfy the property, but there will be some missing.
- size – (default: None) the size of the graph to be generated
- degree_sequence – (default: None) a sequence of nonnegative integers, or None. If specified, the generated graphs will have these integers for degrees. In this case, property and size are both ignored.
- loops – boolean (default: False); whether to allow loops in the graph or not
- sparse – (default: True) whether to use a sparse or dense data structure. See the documentation of *Graph*.
- copy – boolean (default: True); whether to return copies. If set to False the method returns the graph it is working on. The second alternative is faster, but modifying any of the graph instances returned by the

method may break the function's behaviour, as it is using these graphs to compute the next ones: only use `copy=False` when you stick to *reading* the graphs returned.

This parameter is ignored when `immutable` is set to `True`, in which case returned graphs are always copies.

- `immutable` – boolean (default: `False`); whether to return immutable or mutable graphs. When set to `True`, this parameter implies `copy=True`.

EXAMPLES:

Print graphs on 3 or less vertices:

```
sage: for G in graphs(3, augment='vertices'):
....:     print(G)
Graph on 0 vertices
Graph on 1 vertex
Graph on 2 vertices
Graph on 3 vertices
Graph on 3 vertices
Graph on 3 vertices
Graph on 2 vertices
Graph on 3 vertices
```

```
>>> from sage.all import *
>>> for G in graphs(Integer(3), augment='vertices'):
...     print(G)
Graph on 0 vertices
Graph on 1 vertex
Graph on 2 vertices
Graph on 3 vertices
Graph on 3 vertices
Graph on 3 vertices
Graph on 2 vertices
Graph on 3 vertices
```

Print graphs on 3 vertices.

```
sage: for G in graphs(3):
....:     print(G)
Graph on 3 vertices
Graph on 3 vertices
Graph on 3 vertices
Graph on 3 vertices
```

```
>>> from sage.all import *
>>> for G in graphs(Integer(3)):
...     print(G)
Graph on 3 vertices
Graph on 3 vertices
Graph on 3 vertices
Graph on 3 vertices
```

Generate all graphs with 5 vertices and 4 edges.

```
sage: L = graphs(5, size=4)
sage: len(list(L))
6
```

```
>>> from sage.all import *
>>> L = graphs(Integer(5), size=Integer(4))
>>> len(list(L))
6
```

Generate all graphs with 5 vertices and up to 4 edges.

```
sage: L = list(graphs(5, lambda G: G.size() <= 4))
sage: len(L)
14
sage: graphs_list.show_graphs(L)         # long time                                #␣
↪needs sage.plot
```

```
>>> from sage.all import *
>>> L = list(graphs(Integer(5), lambda G: G.size() <= Integer(4)))
>>> len(L)
14
>>> graphs_list.show_graphs(L)         # long time                                #␣
↪needs sage.plot
```

Generate all graphs with up to 5 vertices and up to 4 edges.

```
sage: L = list(graphs(5, lambda G: G.size() <= 4, augment='vertices'))
sage: len(L)
31
sage: graphs_list.show_graphs(L)         # long time                                #␣
↪needs sage.plot
```

```
>>> from sage.all import *
>>> L = list(graphs(Integer(5), lambda G: G.size() <= Integer(4), augment=
↪'vertices'))
>>> len(L)
31
>>> graphs_list.show_graphs(L)         # long time                                #␣
↪needs sage.plot
```

Generate all graphs with degree at most 2, up to 6 vertices.

```
sage: property = lambda G: ( max([G.degree(v) for v in G] + [0]) <= 2 )
sage: L = list(graphs(6, property, augment='vertices'))
sage: len(L)
45
```

```
>>> from sage.all import *
>>> property = lambda G: ( max([G.degree(v) for v in G] + [Integer(0)]) <=␣
↪Integer(2) )
>>> L = list(graphs(Integer(6), property, augment='vertices'))
>>> len(L)
45
```

Generate all bipartite graphs on up to 7 vertices: (see OEIS sequence A033995)

```
sage: L = list( graphs(7, lambda G: G.is_bipartite(), augment='vertices') )
sage: [len([g for g in L if g.order() == i]) for i in [1..7]]
[1, 2, 3, 7, 13, 35, 88]
```

```
>>> from sage.all import *
>>> L = list( graphs(Integer(7), lambda G: G.is_bipartite(), augment='vertices') )
>>> [len([g for g in L if g.order() == i]) for i in (ellipsis_range(Integer(1),
↪Ellipsis,Integer(7)))]
[1, 2, 3, 7, 13, 35, 88]
```

Generate all bipartite graphs on exactly 7 vertices:

```
sage: L = list( graphs(7, lambda G: G.is_bipartite()) )
sage: len(L)
88
```

```
>>> from sage.all import *
>>> L = list( graphs(Integer(7), lambda G: G.is_bipartite()) )
>>> len(L)
88
```

Generate all bipartite graphs on exactly 8 vertices:

```
sage: L = list( graphs(8, lambda G: G.is_bipartite()) ) # long time
sage: len(L)                                            # long time
303
```

```
>>> from sage.all import *
>>> L = list( graphs(Integer(8), lambda G: G.is_bipartite()) ) # long time
>>> len(L)                                                    # long time
303
```

Remember that the property argument does not behave as a filter, except for appropriately inheritable properties:

```
sage: property = lambda G: G.is_vertex_transitive()
sage: len(list(graphs(4, property)))                                          #␣
↪needs sage.groups
1
sage: sum(1 for g in graphs(4) if property(g))                                #␣
↪needs sage.groups
4

sage: property = lambda G: G.is_bipartite()
sage: len(list(graphs(4, property)))
7
sage: sum(1 for g in graphs(4) if property(g))
7
```

```
>>> from sage.all import *
>>> property = lambda G: G.is_vertex_transitive()
>>> len(list(graphs(Integer(4), property)))                                   ␣
```

(continues on next page)

(continued from previous page)

```
↪     # needs sage.groups
1
>>> sum(Integer(1) for g in graphs(Integer(4)) if property(g))                  ␣
↪              # needs sage.groups
4

>>> property = lambda G: G.is_bipartite()
>>> len(list(graphs(Integer(4), property)))
7
>>> sum(Integer(1) for g in graphs(Integer(4)) if property(g))
7
```

Generate graphs on the fly: (see OEIS sequence A000088)

```
sage: for i in range(7):
....:     print(len(list(graphs(i))))
1
1
2
4
11
34
156
```

```
>>> from sage.all import *
>>> for i in range(Integer(7)):
...     print(len(list(graphs(i))))
1
1
2
4
11
34
156
```

Generate all simple graphs, allowing loops: (see OEIS sequence A000666)

```
sage: L = list(graphs(5,augment='vertices',loops=True))                # long time
sage: for i in [0..5]:  # long time
....:     print((i, len([g for g in L if g.order() == i])))
(0, 1)
(1, 2)
(2, 6)
(3, 20)
(4, 90)
(5, 544)
```

```
>>> from sage.all import *
>>> L = list(graphs(Integer(5),augment='vertices',loops=True))                 #␣
↪long time
>>> for i in (ellipsis_range(Integer(0),Ellipsis,Integer(5))):  # long time
...     print((i, len([g for g in L if g.order() == i])))
```

(continues on next page)

(continued from previous page)

```
(0, 1)
(1, 2)
(2, 6)
(3, 20)
(4, 90)
(5, 544)
```

Generate all graphs with a specified degree sequence (see OEIS sequence A002851):

```
sage: for i in [4,6,8]:  # long time (4s on sage.math, 2012)
....:     print((i, len([g for g in graphs(i, degree_sequence=[3]*i) if g.is_
↪connected()])))
(4, 1)
(6, 2)
(8, 5)
sage: for i in [4,6,8]:  # long time (7s on sage.math, 2012)
....:     print((i, len([g for g in graphs(i, augment='vertices', degree_
↪sequence=[3]*i) if g.is_connected()])))
(4, 1)
(6, 2)
(8, 5)
```

```
>>> from sage.all import *
>>> for i in [Integer(4),Integer(6),Integer(8)]:  # long time (4s on sage.math,␣
↪2012)
...     print((i, len([g for g in graphs(i, degree_sequence=[Integer(3)]*i) if g.
↪is_connected()])))
(4, 1)
(6, 2)
(8, 5)
>>> for i in [Integer(4),Integer(6),Integer(8)]:  # long time (7s on sage.math,␣
↪2012)
...     print((i, len([g for g in graphs(i, augment='vertices', degree_
↪sequence=[Integer(3)]*i) if g.is_connected()])))
(4, 1)
(6, 2)
(8, 5)
```

```
sage: print((10, len([g for g in graphs(10,degree_sequence=[3]*10) if g.is_
↪connected()]))) # not tested
(10, 19)
```

```
>>> from sage.all import *
>>> print((Integer(10), len([g for g in graphs(Integer(10),degree_
↪sequence=[Integer(3)]*Integer(10)) if g.is_connected()]))) # not tested
(10, 19)
```

Make sure that the graphs are really independent and the generator survives repeated vertex removal (Issue #8458):

```
sage: for G in graphs(3):
....:     G.delete_vertex(0)
....:     print(G.order())
```

(continues on next page)

(continued from previous page)

```
2
2
2
2
```

```
>>> from sage.all import *
>>> for G in graphs(Integer(3)):
...     G.delete_vertex(Integer(0))
...     print(G.order())
2
2
2
2
```

Returned graphs can be mutable or immutable:

```
sage: G = next(graphs(3, immutable=False))
sage: G.delete_vertex(0)
sage: G = next(graphs(3, immutable=True))
sage: G.delete_vertex(0)
Traceback (most recent call last):
...
TypeError: this graph is immutable and so cannot be changed
sage: G = next(graphs(4, degree_sequence=[3]*4))
sage: G.delete_vertex(0)
sage: G = next(graphs(4, degree_sequence=[3]*4, immutable=True))
sage: G.delete_vertex(0)
Traceback (most recent call last):
...
TypeError: this graph is immutable and so cannot be changed
```

```
>>> from sage.all import *
>>> G = next(graphs(Integer(3), immutable=False))
>>> G.delete_vertex(Integer(0))
>>> G = next(graphs(Integer(3), immutable=True))
>>> G.delete_vertex(Integer(0))
Traceback (most recent call last):
...
TypeError: this graph is immutable and so cannot be changed
>>> G = next(graphs(Integer(4), degree_sequence=[Integer(3)]*Integer(4)))
>>> G.delete_vertex(Integer(0))
>>> G = next(graphs(Integer(4), degree_sequence=[Integer(3)]*Integer(4),␣
↪immutable=True))
>>> G.delete_vertex(Integer(0))
Traceback (most recent call last):
...
TypeError: this graph is immutable and so cannot be changed
```

REFERENCE:

- Brendan D. McKay, Isomorph-Free Exhaustive generation. *Journal of Algorithms*, Volume 26, Issue 2, February 1998, pages 306-324.

static AffineOrthogonalPolarGraph(d, q, *sign='+'*, *immutable=False*)

Return the affine polar graph $VO^+(d,q), VO^-(d,q)$ or $VO(d,q)$.

Affine Polar graphs are built from a d-dimensional vector space over F_q, and a quadratic form which is hyperbolic, elliptic or parabolic according to the value of `sign`.

Note that $VO^+(d,q), VO^-(d,q)$ are strongly regular graphs, while $VO(d,q)$ is not.

For more information on Affine Polar graphs, see Affine Polar Graphs page of Andries Brouwer's website.

INPUT:

- `d` – integer; `d` must be even if `sign is not None`, and odd otherwise
- `q` – integer; a power of a prime number, as F_q must exist
- `sign` – string (default: `'+'`); must be equal to `'+'`, `'-'`, or `None` to compute (respectively) $VO^+(d,q), VO^-(d,q)$ or $VO(d,q)$
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

Note

The graph $VO^\epsilon(d,q)$ is the graph induced by the non-neighbors of a vertex in an *Orthogonal Polar Graph* $O^\epsilon(d+2,q)$.

EXAMPLES:

The *Brouwer-Haemers graph* is isomorphic to $VO^-(4,3)$:

```
sage: g = graphs.AffineOrthogonalPolarGraph(4,3,"-")
↪  # needs sage.libs.gap
sage: g.is_isomorphic(graphs.BrouwerHaemersGraph())
↪  # needs sage.libs.gap
True
```

```
>>> from sage.all import *
>>> g = graphs.AffineOrthogonalPolarGraph(Integer(4),Integer(3),"-")
↪                  # needs sage.libs.gap
>>> g.is_isomorphic(graphs.BrouwerHaemersGraph())
↪# needs sage.libs.gap
True
```

Some examples from Brouwer's table or strongly regular graphs:

```
sage: g = graphs.AffineOrthogonalPolarGraph(6,2,"-"); g
Affine Polar Graph VO^-(6,2): Graph on 64 vertices
sage: g.is_strongly_regular(parameters=True)
(64, 27, 10, 12)
sage: g = graphs.AffineOrthogonalPolarGraph(6,2,"+"); g
Affine Polar Graph VO^+(6,2): Graph on 64 vertices
sage: g.is_strongly_regular(parameters=True)
(64, 35, 18, 20)
```

```
>>> from sage.all import *
>>> g = graphs.AffineOrthogonalPolarGraph(Integer(6),Integer(2),"-"); g
```

(continues on next page)

(continued from previous page)

```
Affine Polar Graph VO^-(6,2): Graph on 64 vertices
>>> g.is_strongly_regular(parameters=True)
(64, 27, 10, 12)
>>> g = graphs.AffineOrthogonalPolarGraph(Integer(6),Integer(2),"+"); g
Affine Polar Graph VO^+(6,2): Graph on 64 vertices
>>> g.is_strongly_regular(parameters=True)
(64, 35, 18, 20)
```

When `sign is None`:

```
sage: g = graphs.AffineOrthogonalPolarGraph(5,2,None); g
Affine Polar Graph VO^-(5,2): Graph on 32 vertices
sage: g.is_strongly_regular(parameters=True)
False
sage: g.is_regular()
True
sage: g.is_vertex_transitive()
True
```

```
>>> from sage.all import *
>>> g = graphs.AffineOrthogonalPolarGraph(Integer(5),Integer(2),None); g
Affine Polar Graph VO^-(5,2): Graph on 32 vertices
>>> g.is_strongly_regular(parameters=True)
False
>>> g.is_regular()
True
>>> g.is_vertex_transitive()
True
```

static AfricaMap(*continental=False*, *year=2018*, *immutable=False*)

Return African states as a graph of common border.

"African state" here is defined as an independent state having the capital city in Africa. The graph has an edge between those countries that have common *land* border.

INPUT:

- `continental` – boolean (default: `False`); whether to only return states in the continental Africa or all African states
- `year` – integer (default: 2018); reserved for future use
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: Africa = graphs.AfricaMap(); Africa
Africa Map: Graph on 54 vertices
sage: sorted(Africa.neighbors('Libya'))
['Algeria', 'Chad', 'Egypt', 'Niger', 'Sudan', 'Tunisia']

sage: cont_Africa = graphs.AfricaMap(continental=True)
sage: cont_Africa.order()
48
sage: 'Madagaskar' in cont_Africa
```

(continues on next page)

(continued from previous page)

```
False
sage: cont_Africa.is_connected()
True
```

```
>>> from sage.all import *
>>> Africa = graphs.AfricaMap(); Africa
Africa Map: Graph on 54 vertices
>>> sorted(Africa.neighbors('Libya'))
['Algeria', 'Chad', 'Egypt', 'Niger', 'Sudan', 'Tunisia']

>>> cont_Africa = graphs.AfricaMap(continental=True)
>>> cont_Africa.order()
48
>>> 'Madagaskar' in cont_Africa
False
>>> cont_Africa.is_connected()
True
```

static AhrensSzekeresGeneralizedQuadrangleGraph(*q*, *dual=False*, *immutable=False*)

Return the collinearity graph of the generalized quadrangle $AS(q)$, or of its dual

Let q be an odd prime power. $AS(q)$ is a generalized quadrangle (Wikipedia article Generalized_quadrangle) of order $(q-1, q+1)$, see 3.1.5 in [PT2009]. Its points are elements of F_q^3, and lines are sets of size q of the form

- $\{(\sigma, a, b) \mid \sigma \in F_q\}$
- $\{(a, \sigma, b) \mid \sigma \in F_q\}$
- $\{(c\sigma^2 - b\sigma + a, -2c\sigma + b, \sigma) \mid \sigma \in F_q\}$,

where a, b, c are arbitrary elements of F_q.

INPUT:

- `q` – a power of an odd prime number
- `dual` – boolean (default: `False`); whether to return the collinearity graph of $AS(q)$ or of the dual $AS(q)$ (when `True`)
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.AhrensSzekeresGeneralizedQuadrangleGraph(5); g
AS(5); GQ(4, 6): Graph on 125 vertices
sage: g.is_strongly_regular(parameters=True)
(125, 28, 3, 7)
sage: g = graphs.AhrensSzekeresGeneralizedQuadrangleGraph(5, dual=True); g
AS(5)*; GQ(6, 4): Graph on 175 vertices
sage: g.is_strongly_regular(parameters=True)
(175, 30, 5, 5)
```

```
>>> from sage.all import *
>>> g = graphs.AhrensSzekeresGeneralizedQuadrangleGraph(Integer(5)); g
AS(5); GQ(4, 6): Graph on 125 vertices
```

(continues on next page)

(continued from previous page)

```
>>> g.is_strongly_regular(parameters=True)
(125, 28, 3, 7)
>>> g = graphs.AhrensSzekeresGeneralizedQuadrangleGraph(Integer(5),␣
↪dual=True); g
AS(5)*; GQ(6, 4): Graph on 175 vertices
>>> g.is_strongly_regular(parameters=True)
(175, 30, 5, 5)
```

static Akq(*k*, *q*, *A=None*, *B=None*, *immutable=False*)

Return the graph $A(k,q)$ described in [LW2026].

The point set and line set are $\mathbf{F}_q^k$. A point $(p_1,\ldots,p_k)$ is adjacent to a line $(l_1,\ldots,l_k)$ if

$$p_i + l_i = \begin{cases} p_{i-1}l_1, & i \text{ even}, \\ p_1 l_{i-1}, & i \text{ odd}, \end{cases} \qquad 2 \le i \le k.$$

INPUT:

- `k` – integer at least 2
- `q` – prime power
- `A` – iterable, single value, or `None` (default: `None`); allowed first coordinates on the point side
- `B` – iterable, single value, or `None` (default: `None`); allowed first coordinates on the line side
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

OUTPUT:

A Sage *Graph*.

EXAMPLES:

```
sage: G = graphs.Akq(5, 3)
sage: G.order(), G.size(), G.girth()
(486, 729, 12)
```

```
>>> from sage.all import *
>>> G = graphs.Akq(Integer(5), Integer(3))
>>> G.order(), G.size(), G.girth()
(486, 729, 12)
```

REFERENCES:

- [LW2026]

static AlternatingFormsGraph(*n*, *q*, *immutable=False*)

Return the alternating forms graph with the given parameters.

This builds a graph whose vertices are all $n`x`n$ skew-symmetric matrices over $GF(q)$ with zero diagonal. Two vertices are adjacent if and only if the difference of the two matrices has rank 2.

This graph is distance-regular with classical parameters $(\lfloor \frac{n}{2} \rfloor, q^2, q^2-1, q^{2\lceil \frac{n}{2} \rceil - 1})$.

INPUT:

- `n` – integer
- `q` – a prime power

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: G = graphs.AlternatingFormsGraph(5, 2)  # long time
sage: G.is_distance_regular(True)  # long time
([155, 112, None], [None, 1, 20])
```

```
>>> from sage.all import *
>>> G = graphs.AlternatingFormsGraph(Integer(5), Integer(2))  # long time
>>> G.is_distance_regular(True)  # long time
([155, 112, None], [None, 1, 20])
```

REFERENCES:

See [BCN1989] pp. 282-284 for a rather detailed discussion, otherwise see [VDKT2016] p. 22.

static AztecDiamondGraph(*n*, *immutable=False*)

Return the Aztec Diamond graph of order `n`.

See the Wikipedia article Aztec_diamond for more information.

INPUT:

- `n` – integer ≥ 0; the order Aztec Diamond graph
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: graphs.AztecDiamondGraph(2)
Aztec Diamond graph of order 2: Graph on 12 vertices

sage: [graphs.AztecDiamondGraph(i).n_vertices() for i in range(8)]
[0, 4, 12, 24, 40, 60, 84, 112]

sage: [graphs.AztecDiamondGraph(i).n_edges() for i in range(8)]
[0, 4, 16, 36, 64, 100, 144, 196]

sage: G = graphs.AztecDiamondGraph(3)
sage: sum(1 for p in G.perfect_matchings())
64
```

```
>>> from sage.all import *
>>> graphs.AztecDiamondGraph(Integer(2))
Aztec Diamond graph of order 2: Graph on 12 vertices

>>> [graphs.AztecDiamondGraph(i).n_vertices() for i in range(Integer(8))]
[0, 4, 12, 24, 40, 60, 84, 112]

>>> [graphs.AztecDiamondGraph(i).n_edges() for i in range(Integer(8))]
[0, 4, 16, 36, 64, 100, 144, 196]

>>> G = graphs.AztecDiamondGraph(Integer(3))
>>> sum(Integer(1) for p in G.perfect_matchings())
64
```

static Balaban10Cage(*embedding=1*, *immutable=False*)

Return the Balaban 10-cage.

The Balaban 10-cage is a 3-regular graph with 70 vertices and 105 edges. See the Wikipedia article Balaban_10-cage.

The default embedding gives a deeper understanding of the graph's automorphism group. It is divided into 4 layers (each layer being a set of points at equal distance from the drawing's center). From outside to inside:

- L1: The outer layer (vertices which are the furthest from the origin) is actually the disjoint union of two cycles of length 10.
- L2: The second layer is an independent set of 20 vertices.
- L3: The third layer is a matching on 10 vertices.
- L4: The inner layer (vertices which are the closest from the origin) is also the disjoint union of two cycles of length 10.

This graph is not vertex-transitive, and its vertices are partitioned into 3 orbits: L2, L3, and the union of L1 of L4 whose elements are equivalent.

INPUT:

- `embedding` – integer (default: 1); two embeddings are available, and can be selected by setting `embedding` to be either 1 or 2
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.Balaban10Cage()
sage: g.girth()
10
sage: g.chromatic_number()
2
sage: g.diameter()
6
sage: g.is_hamiltonian()                                                    ␣
↪  # needs sage.numerical.mip
True
sage: g.show(figsize=[10,10])            # long time                        ␣
↪  # needs sage.plot
```

```
>>> from sage.all import *
>>> g = graphs.Balaban10Cage()
>>> g.girth()
10
>>> g.chromatic_number()
2
>>> g.diameter()
6
>>> g.is_hamiltonian()
↪# needs sage.numerical.mip
True
>>> g.show(figsize=[Integer(10),Integer(10)])           # long time          ␣
↪                  # needs sage.plot
```

static Balaban11Cage(*embedding=1*, *immutable=False*)

Return the Balaban 11-cage.

For more information, see the Wikipedia article Balaban_11-cage.

INPUT:

- `embedding` – integer (default: 1); three embeddings are available, and can be selected by setting `embedding` to be 1, 2, or 3
  - The first embedding is the one appearing on page 9 of the Fifth Annual Graph Drawing Contest report [EMMN1998]. It separates vertices based on their eccentricity (see *`eccentricity()`*).
  - The second embedding has been produced just for Sage and is meant to emphasize the automorphism group's 6 orbits.
  - The last embedding is the default one produced by the *`LCFGraph()`* constructor.
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

Note

The vertex labeling changes according to the value of `embedding=1`.

EXAMPLES:

Basic properties:

```
sage: g = graphs.Balaban11Cage()
sage: g.order()
112
sage: g.size()
168
sage: g.girth()
11
sage: g.diameter()
8
sage: g.automorphism_group().cardinality()
 ↪  # needs sage.groups
64
```

```
>>> from sage.all import *
>>> g = graphs.Balaban11Cage()
>>> g.order()
112
>>> g.size()
168
>>> g.girth()
11
>>> g.diameter()
8
>>> g.automorphism_group().cardinality()
↪# needs sage.groups
64
```

Our many embeddings:

```
sage: g1 = graphs.Balaban11Cage(embedding=1)
sage: g2 = graphs.Balaban11Cage(embedding=2)
sage: g3 = graphs.Balaban11Cage(embedding=3)
sage: g1.show(figsize=[10,10])          # long time                    ␣
↪  # needs sage.plot
sage: g2.show(figsize=[10,10])          # long time                    ␣
↪  # needs sage.plot
sage: g3.show(figsize=[10,10])          # long time                    ␣
↪  # needs sage.plot
```

```
>>> from sage.all import *
>>> g1 = graphs.Balaban11Cage(embedding=Integer(1))
>>> g2 = graphs.Balaban11Cage(embedding=Integer(2))
>>> g3 = graphs.Balaban11Cage(embedding=Integer(3))
>>> g1.show(figsize=[Integer(10),Integer(10)])          # long time          ␣
↪                  # needs sage.plot
>>> g2.show(figsize=[Integer(10),Integer(10)])          # long time          ␣
↪                  # needs sage.plot
>>> g3.show(figsize=[Integer(10),Integer(10)])          # long time          ␣
↪                  # needs sage.plot
```

Proof that the embeddings are the same graph:

```
sage: g1.is_isomorphic(g2)  # g2 and g3 are obviously isomorphic
True
```

```
>>> from sage.all import *
>>> g1.is_isomorphic(g2)  # g2 and g3 are obviously isomorphic
True
```

static BalancedTree(*r*, *h*, *immutable=False*)

Return the perfectly balanced tree of height $h \geq 1$, whose root has degree $r \geq 2$.

The number of vertices of this graph is $1 + r + r^2 + \cdots + r^h$, that is, $\frac{r^{h+1}-1}{r-1}$. The number of edges is one less than the number of vertices.

INPUT:

- `r` – positive integer ≥ 2; the degree of the root node
- `h` – positive integer ≥ 1; the height of the balanced tree
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

OUTPUT:

The perfectly balanced tree of height $h \geq 1$ and whose root has degree $r \geq 2$.

EXAMPLES:

A balanced tree whose root node has degree $r = 2$, and of height $h = 1$, has order 3 and size 2:

```
sage: G = graphs.BalancedTree(2, 1); G
Balanced tree: Graph on 3 vertices
sage: G.order()
3
sage: G.size()
```

(continues on next page)

(continued from previous page)

```
2
sage: r = 2; h = 1
sage: v = 1 + r
sage: v; v - 1
3
2
```

```
>>> from sage.all import *
>>> G = graphs.BalancedTree(Integer(2), Integer(1)); G
Balanced tree: Graph on 3 vertices
>>> G.order()
3
>>> G.size()
2
>>> r = Integer(2); h = Integer(1)
>>> v = Integer(1) + r
>>> v; v - Integer(1)
3
2
```

Plot a balanced tree of height 5, whose root node has degree $r = 3$:

```
sage: G = graphs.BalancedTree(3, 5)
sage: G.plot()                              # long time
↪  # needs sage.plot
Graphics object consisting of 728 graphics primitives
```

```
>>> from sage.all import *
>>> G = graphs.BalancedTree(Integer(3), Integer(5))
>>> G.plot()                              # long time
↪# needs sage.plot
Graphics object consisting of 728 graphics primitives
```

A tree is bipartite. If its vertex set is finite, then it is planar:

```
sage: # needs networkx
sage: r = randint(2, 5); h = randint(1, 7)
sage: T = graphs.BalancedTree(r, h)
sage: T.is_bipartite()
True
sage: T.is_planar()
True
sage: v = (r^(h + 1) - 1) / (r - 1)
sage: T.order() == v
True
sage: T.size() == v - 1
True
```

```
>>> from sage.all import *
>>> # needs networkx
>>> r = randint(Integer(2), Integer(5)); h = randint(Integer(1), Integer(7))
>>> T = graphs.BalancedTree(r, h)
```

(continues on next page)

(continued from previous page)

```
>>> T.is_bipartite()
True
>>> T.is_planar()
True
>>> v = (r**(h + Integer(1)) - Integer(1)) / (r - Integer(1))
>>> T.order() == v
True
>>> T.size() == v - Integer(1)
True
```

static BarbellGraph(*n1*, *n2*, *immutable=False*)

Return a barbell graph with $2n_1 + n_2$ nodes.

The argument n_1 must be greater than or equal to 2.

A barbell graph is a basic structure that consists of a path graph of order n_2 connecting two complete graphs of order n_1 each.

INPUT:

- `n1` – integer ≥ 2; the order of each of the two complete graphs
- `n2` – nonnegative integer; the order of the path graph connecting the two complete graphs
- `immutable` – boolean (default: `False`); whether to return immutable or mutable graphs

OUTPUT:

A barbell graph of order $2 * n_1 + n_2$. A `ValueError` is returned if $n_1 < 2$ or $n_2 < 0$.

PLOTTING:

Upon construction, the position dictionary is filled to override the spring-layout algorithm. By convention, each barbell graph will be displayed with the two complete graphs in the lower-left and upper-right corners, with the path graph connecting diagonally between the two. Thus the n_1-th node will be drawn at a 45 degree angle from the horizontal right center of the first complete graph, and the $n_1 + n_2 + 1$-th node will be drawn 45 degrees below the left horizontal center of the second complete graph.

EXAMPLES:

Construct and show a barbell graph `Bar = 4, Bells = 9`:

```
sage: g = graphs.BarbellGraph(9, 4); g
Barbell graph: Graph on 22 vertices
sage: g.show()                          # long time                    ␣
↪  # needs sage.plot
```

```
>>> from sage.all import *
>>> g = graphs.BarbellGraph(Integer(9), Integer(4)); g
Barbell graph: Graph on 22 vertices
>>> g.show()                           # long time
↪# needs sage.plot
```

An $n_1 \geq 2$, $n_2 \geq 0$ barbell graph has order $2 * n_1 + n_2$. It has the complete graph on n_1 vertices as a subgraph. It also has the path graph on n_2 vertices as a subgraph.

```
sage: n1 = randint(2, 2*10^2)
sage: n2 = randint(0, 2*10^2)
```

(continues on next page)

(continued from previous page)

```
sage: g = graphs.BarbellGraph(n1, n2)
sage: v = 2*n1 + n2
sage: g.order() == v
True
sage: K_n1 = graphs.CompleteGraph(n1)
sage: P_n2 = graphs.PathGraph(n2)

sage: s_K = g.subgraph_search(K_n1, induced=True)
sage: s_P = g.subgraph_search(P_n2, induced=True)
sage: K_n1.is_isomorphic(s_K)
True
sage: P_n2.is_isomorphic(s_P)
True
```

```
>>> from sage.all import *
>>> n1 = randint(Integer(2), Integer(2)*Integer(10)**Integer(2))
>>> n2 = randint(Integer(0), Integer(2)*Integer(10)**Integer(2))
>>> g = graphs.BarbellGraph(n1, n2)
>>> v = Integer(2)*n1 + n2
>>> g.order() == v
True
>>> K_n1 = graphs.CompleteGraph(n1)
>>> P_n2 = graphs.PathGraph(n2)

>>> s_K = g.subgraph_search(K_n1, induced=True)
>>> s_P = g.subgraph_search(P_n2, induced=True)
>>> K_n1.is_isomorphic(s_K)
True
>>> P_n2.is_isomorphic(s_P)
True
```

static BidiakisCube(*immutable=False*)

Return the Bidiakis cube.

For more information, see the Wikipedia article Bidiakis_cube.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

The Bidiakis cube is a 3-regular graph having 12 vertices and 18 edges. This means that each vertex has a degree of 3:

```
sage: g = graphs.BidiakisCube(); g
Bidiakis cube: Graph on 12 vertices
sage: g.show()                          # long time
↪  # needs sage.plot
sage: g.order()
12
sage: g.size()
18
sage: g.is_regular(3)
```

(continues on next page)

(continued from previous page)

```
True
```

```
>>> from sage.all import *
>>> g = graphs.BidiakisCube(); g
Bidiakis cube: Graph on 12 vertices
>>> g.show()                              # long time
↪# needs sage.plot
>>> g.order()
12
>>> g.size()
18
>>> g.is_regular(Integer(3))
True
```

It is a Hamiltonian graph with diameter 3 and girth 4:

```
sage: g.is_hamiltonian()                                                    ␣
↪  # needs sage.numerical.mip
True
sage: g.diameter()
3
sage: g.girth()
4
```

```
>>> from sage.all import *
>>> g.is_hamiltonian()
↪# needs sage.numerical.mip
True
>>> g.diameter()
3
>>> g.girth()
4
```

It is a planar graph with characteristic polynomial $(x - 3)(x - 2)(x^4)(x + 1)(x + 2)(x^2 + x - 4)^2$ and chromatic number 3:

```
sage: g.is_planar()
True
sage: char_poly = g.characteristic_polynomial()                             ␣
↪  # needs sage.modules
sage: x = char_poly.parent()('x')                                           ␣
↪  # needs sage.modules
sage: char_poly == (x - 3) * (x - 2) * (x^4) * (x + 1) * (x + 2) * (x^2 + x -␣
↪4)^2          # needs sage.modules
True
sage: g.chromatic_number()                                                  ␣
↪  # needs sage.modules
3
```

```
>>> from sage.all import *
>>> g.is_planar()
True
```

(continues on next page)

(continued from previous page)

```
>>> char_poly = g.characteristic_polynomial()
↪# needs sage.modules
>>> x = char_poly.parent()('x')
↪# needs sage.modules
>>> char_poly == (x - Integer(3)) * (x - Integer(2)) * (x**Integer(4)) * (x +␣
↪Integer(1)) * (x + Integer(2)) * (x**Integer(2) + x -␣
↪Integer(4))**Integer(2)           # needs sage.modules
True
>>> g.chromatic_number()
↪# needs sage.modules
3
```

static BiggsSmithGraph(*embedding=1*, *immutable=False*)

Return the Biggs-Smith graph.

For more information, see the Wikipedia article Biggs-Smith_graph.

INPUT:

- `embedding` – integer (default: 1); two embeddings are available, and can be selected by setting `embedding` to be 1 or 2
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

Basic properties:

```
sage: g = graphs.BiggsSmithGraph()
sage: g.order()
102
sage: g.size()
153
sage: g.girth()
9
sage: g.diameter()
7
sage: g.automorphism_group().cardinality()      # long time
2448
sage: g.show(figsize=[10, 10])          # long time                          ␣
↪  # needs sage.plot
```

```
>>> from sage.all import *
>>> g = graphs.BiggsSmithGraph()
>>> g.order()
102
>>> g.size()
153
>>> g.girth()
9
>>> g.diameter()
7
>>> g.automorphism_group().cardinality()      # long time
2448
```

(continues on next page)

(continued from previous page)

```
>>> g.show(figsize=[Integer(10), Integer(10)])          # long time          ␣
↪                  # needs sage.plot
```

The other embedding:

```
sage: graphs.BiggsSmithGraph(embedding=2).show()        # long time
```

```
>>> from sage.all import *
>>> graphs.BiggsSmithGraph(embedding=Integer(2)).show()        # long time
```

static BilinearFormsGraph(*d*, *e*, *q*, *immutable=False*)

Return a bilinear forms graph with the given parameters.

This builds a graph whose vertices are all $d`x`e$ matrices over $GF(q)$. Two vertices are adjacent if the difference of the two matrices has rank 1.

The graph is distance-regular with classical parameters $(\min(d, e), q, q-1, q^{\max(d,e)}-1)$.

INPUT:

- `d`, `e` – integers; dimension of the matrices
- `q` – integer; a prime power
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: G = graphs.BilinearFormsGraph(3, 3, 2)
sage: G.is_distance_regular(True)
([49, 36, 16, None], [None, 1, 6, 28])
sage: G = graphs.BilinearFormsGraph(3,3,3)      # not tested (20 s)          ␣
↪  # needs sage.rings.finite_rings
sage: G.order()                          # not tested (due to above)          ␣
↪  # needs sage.rings.finite_rings
19683
sage: G = graphs.BilinearFormsGraph(3, 4, 2)    # long time                  ␣
↪  # needs sage.rings.finite_rings
sage: G.is_distance_regular(True)       # long time                          ␣
↪  # needs sage.rings.finite_rings
([105, 84, 48, None], [None, 1, 6, 28])
```

```
>>> from sage.all import *
>>> G = graphs.BilinearFormsGraph(Integer(3), Integer(3), Integer(2))
>>> G.is_distance_regular(True)
([49, 36, 16, None], [None, 1, 6, 28])
>>> G = graphs.BilinearFormsGraph(Integer(3),Integer(3),Integer(3))      #␣
↪not tested (20 s)             # needs sage.rings.finite_rings
>>> G.order()                           # not tested (due to above)
↪# needs sage.rings.finite_rings
19683
>>> G = graphs.BilinearFormsGraph(Integer(3), Integer(4), Integer(2))    #␣
↪long time                     # needs sage.rings.finite_rings
>>> G.is_distance_regular(True)       # long time
```

(continues on next page)

(continued from previous page)

```
↪# needs sage.rings.finite_rings
([105, 84, 48, None], [None, 1, 6, 28])
```

REFERENCES:

See [BCN1989] pp. 280-282 for a rather detailed discussion, otherwise see [VDKT2016] p. 21.

static BishopGraph(*dim_list*, *radius=None*, *relabel=False*, *immutable=False*)

Return the d-dimensional Bishop Graph with prescribed dimensions.

The 2-dimensional Bishop Graph of parameters n and m is a graph with nm vertices in which each vertex represents a square in an $n \times m$ chessboard, and each edge corresponds to a legal move by a bishop.

The d-dimensional Bishop Graph with $d >= 2$ has for vertex set the cells of a d-dimensional grid with prescribed dimensions, and each edge corresponds to a legal move by a bishop in any pairs of dimensions.

The Bishop Graph is not connected.

INPUT:

- `dim_list` – iterable (list, set, dict); provides the dimensions $n_1, n_2, \ldots, n_d$, with $n_i \geq 1$, of the chessboard
- `radius` – integer (default: `None`); by setting the radius to a positive integer, one may decrease the power of the bishop to at most `radius` steps.
- `relabel` – boolean (default: `False`); indicates whether the vertices must be relabeled as integers
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

The (n,m)-Bishop Graph is not connected:

```
sage: G = graphs.BishopGraph( [3, 4] )
sage: G.is_connected()
False
```

```
>>> from sage.all import *
>>> G = graphs.BishopGraph( [Integer(3), Integer(4)] )
>>> G.is_connected()
False
```

The Bishop Graph can be obtained from Knight Graphs:

```
sage: for d in range(3,12):   # long time
....:     H = Graph()
....:     for r in range(1,d+1):
....:         B = graphs.BishopGraph([d,d],radius=r)
....:         H.add_edges( graphs.KnightGraph([d,d],one=r,two=r).
↪edges(sort=False) )
....:         if not B.is_isomorphic(H):
....:            print("that's not good!")
```

```
>>> from sage.all import *
>>> for d in range(Integer(3),Integer(12)):   # long time
...     H = Graph()
```

(continues on next page)

(continued from previous page)

```
...     for r in range(Integer(1),d+Integer(1)):
...         B = graphs.BishopGraph([d,d],radius=r)
...         H.add_edges( graphs.KnightGraph([d,d],one=r,two=r).
↪edges(sort=False) )
...         if not B.is_isomorphic(H):
...             print("that's not good!")
```

static BiwheelGraph(*n*, *immutable=False*)

Return a biwheel graph with $2n$ nodes

For $n \geq 4$, the biwheel graph of order $2n$ is the planar bipartite graph obtained from the cycle graph of order $2n-2$, i.e., `graphs.CycleGraph(2*n - 2)` (called the rim of the biwheel graph) by introducing two new nodes $2n-2$ and $2n-1$ (called the *hubs* of the biwheel graph), and then joining the node $2n-2$ with the odd indexed nodes up to $2n-3$ and joining the node $2n-1$ with the even indexed nodes up to $2n-4$.

PLOTTING:

Upon construction, the position dictionary is filled to override the spring-layout algorithm. By convention, each biwheel graph will be displayed with the first (0) node at the right if n is even or otherwise at an angle $\pi/(2n-2)$ with respect to the origin, with the rest of the nodes up to $2n-3$ following in a counterclockwise manner. Note that the last two nodes, i.e., the hubs $2n-2$ and $2n-1$ will be displayed at the coordinates $(-1/3, 0)$ and $(1/3, 0)$ respectively.

INPUT:

- `n` – an integer at least 4; number of nodes is $2n$
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

OUTPUT:

- `G` – a biwheel graph of order $2n$; note that a `ValueError` is returned if $n < 4$

EXAMPLES:

Construct and show a biwheel graph with 10 nodes:

```
sage: g = graphs.BiwheelGraph(5)
sage: g.show()                              # long time
↪  # needs sage.plot
sage: g.is_planar()
True
sage: g.is_bipartite()
True
```

```
>>> from sage.all import *
>>> g = graphs.BiwheelGraph(Integer(5))
>>> g.show()                              # long time
↪# needs sage.plot
>>> g.is_planar()
True
>>> g.is_bipartite()
True
```

Create several biwheel graphs in a Sage graphics array:

```
sage: g = []
sage: j = []
sage: for i in range(9):
....:     k = graphs.BiwheelGraph(i+4)
....:     g.append(k)
sage: for i in range(3):
....:     n = []
....:     for m in range(3):
....:         n.append(g[3*i + m].plot(vertex_size=50 - 4*(3*i+m), vertex_
↪labels=False))
....:     j.append(n)
sage: G = graphics_array(j)
sage: G.show()                              # long time
```

```
>>> from sage.all import *
>>> g = []
>>> j = []
>>> for i in range(Integer(9)):
...     k = graphs.BiwheelGraph(i+Integer(4))
...     g.append(k)
>>> for i in range(Integer(3)):
...     n = []
...     for m in range(Integer(3)):
...         n.append(g[Integer(3)*i + m].plot(vertex_size=Integer(50) -␣
↪Integer(4)*(Integer(3)*i+m), vertex_labels=False))
...     j.append(n)
>>> G = graphics_array(j)
>>> G.show()                              # long time
```

REFERENCES:

- [LM2024]

See also

WheelGraph(), *TruncatedBiwheelGraph()*

AUTHORS:

- Janmenjaya Panda (2024-06-09)

static BlanusaFirstSnarkGraph(*immutable=False*)

Return the first Blanusa Snark Graph.

The Blanusa graphs are two snarks on 18 vertices and 27 edges. For more information on them, see the Wikipedia article Blanusa_snarks.

See also

- *BlanusaSecondSnarkGraph()*.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.BlanusaFirstSnarkGraph()
sage: g.order()
18
sage: g.size()
27
sage: g.diameter()
4
sage: g.girth()
5
sage: g.automorphism_group().cardinality()
↪  # needs sage.groups
8
```

```
>>> from sage.all import *
>>> g = graphs.BlanusaFirstSnarkGraph()
>>> g.order()
18
>>> g.size()
27
>>> g.diameter()
4
>>> g.girth()
5
>>> g.automorphism_group().cardinality()
↪# needs sage.groups
8
```

static BlanusaSecondSnarkGraph(*immutable=False*)

Return the second Blanusa Snark Graph.

The Blanusa graphs are two snarks on 18 vertices and 27 edges. For more information on them, see the Wikipedia article Blanusa_snarks.

See also

- *BlanusaFirstSnarkGraph()*.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.BlanusaSecondSnarkGraph()
sage: g.order()
18
sage: g.size()
27
sage: g.diameter()
4
```

(continues on next page)

(continued from previous page)

```
sage: g.girth()
5
sage: g.automorphism_group().cardinality()                                  ␣
↪  # needs sage.groups
4
```

```
>>> from sage.all import *
>>> g = graphs.BlanusaSecondSnarkGraph()
>>> g.order()
18
>>> g.size()
27
>>> g.diameter()
4
>>> g.girth()
5
>>> g.automorphism_group().cardinality()
↪# needs sage.groups
4
```

static BrinkmannGraph(*immutable=False*)

Return the Brinkmann graph.

For more information, see the Wikipedia article Brinkmann_graph.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

The Brinkmann graph is a 4-regular graph having 21 vertices and 42 edges. This means that each vertex has degree 4:

```
sage: G = graphs.BrinkmannGraph(); G
Brinkmann graph: Graph on 21 vertices
sage: G.show()                              # long time                     ␣
↪  # needs sage.plot
sage: G.order()
21
sage: G.size()
42
sage: G.is_regular(4)
True
```

```
>>> from sage.all import *
>>> G = graphs.BrinkmannGraph(); G
Brinkmann graph: Graph on 21 vertices
>>> G.show()                            # long time
↪# needs sage.plot
>>> G.order()
21
>>> G.size()
42
```

(continues on next page)

(continued from previous page)

```
>>> G.is_regular(Integer(4))
True
```

It is an Eulerian graph with radius 3, diameter 3, and girth 5:

```
sage: G.is_eulerian()
True
sage: G.radius()
3
sage: G.diameter()
3
sage: G.girth()
5
```

```
>>> from sage.all import *
>>> G.is_eulerian()
True
>>> G.radius()
3
>>> G.diameter()
3
>>> G.girth()
5
```

The Brinkmann graph is also Hamiltonian with chromatic number 4:

```
sage: G.is_hamiltonian()
↪  # needs sage.numerical.mip
True
sage: G.chromatic_number()
4
```

```
>>> from sage.all import *
>>> G.is_hamiltonian()
↪# needs sage.numerical.mip
True
>>> G.chromatic_number()
4
```

Its automorphism group is isomorphic to D_7:

```
sage: ag = G.automorphism_group()
↪  # needs sage.groups
sage: ag.is_isomorphic(DihedralGroup(7))
↪  # needs sage.groups
True
```

```
>>> from sage.all import *
>>> ag = G.automorphism_group()
↪# needs sage.groups
>>> ag.is_isomorphic(DihedralGroup(Integer(7)))
```

(continues on next page)

(continued from previous page)

```
↪          # needs sage.groups
True
```

static BrouwerHaemersGraph(*immutable=False*)

Return the Brouwer-Haemers Graph.

The Brouwer-Haemers is the only strongly regular graph of parameters $(81, 20, 1, 6)$. It is build in Sage as the Affine Orthogonal graph $VO^-(6, 3)$. For more information on this graph, see its corresponding page on Andries Brouwer's website.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.BrouwerHaemersGraph(); g                                   ␣
↪  # needs sage.modules
Brouwer-Haemers: Graph on 81 vertices
```

```
>>> from sage.all import *
>>> g = graphs.BrouwerHaemersGraph(); g
↪# needs sage.modules
Brouwer-Haemers: Graph on 81 vertices
```

It is indeed strongly regular with parameters $(81, 20, 1, 6)$:

```
sage: g.is_strongly_regular(parameters=True)     # long time                 ␣
↪  # needs sage.modules sage.rings.finite_rings
(81, 20, 1, 6)
```

```
>>> from sage.all import *
>>> g.is_strongly_regular(parameters=True)     # long time
↪# needs sage.modules sage.rings.finite_rings
(81, 20, 1, 6)
```

Its has as eigenvalues $20, 2$ and -7:

```
sage: set(g.spectrum()) == {20,2,-7}                                        ␣
↪  # needs sage.modules sage.rings.finite_rings
True
```

```
>>> from sage.all import *
>>> set(g.spectrum()) == {Integer(20),Integer(2),-Integer(7)}               ␣
↪                            # needs sage.modules sage.rings.finite_rings
True
```

static BubbleSortGraph(*n*, *immutable=False*)

Return the bubble sort graph $B(n)$.

The vertices of the bubble sort graph are the set of permutations on n symbols. Two vertices are adjacent if one can be obtained from the other by swapping the labels in the i-th and $(i+1)$-th positions for $1 \leq i \leq n-1$. In total, $B(n)$ has order $n!$. Swapping two labels as described previously corresponds to multiplying on the right the permutation corresponding to the node by an elementary transposition in the `SymmetricGroup`.

The bubble sort graph is the underlying graph of the `permutahedron()`.

INPUT:

- `n` – positive integer. The number of symbols to permute
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

OUTPUT:

The bubble sort graph $B(n)$ on n symbols. If $n < 1$, a `ValueError` is returned.

EXAMPLES:

```
sage: g = graphs.BubbleSortGraph(4); g
Bubble sort: Graph on 24 vertices
sage: g.plot()                              # long time
↪  # needs sage.plot
Graphics object consisting of 61 graphics primitives
```

```
>>> from sage.all import *
>>> g = graphs.BubbleSortGraph(Integer(4)); g
Bubble sort: Graph on 24 vertices
>>> g.plot()                              # long time
↪# needs sage.plot
Graphics object consisting of 61 graphics primitives
```

The bubble sort graph on $n = 1$ symbol is the trivial graph K_1:

```
sage: graphs.BubbleSortGraph(1)
Bubble sort: Graph on 1 vertex
```

```
>>> from sage.all import *
>>> graphs.BubbleSortGraph(Integer(1))
Bubble sort: Graph on 1 vertex
```

If $n \geq 1$, then the order of $B(n)$ is $n!$:

```
sage: n = randint(1, 8)
sage: g = graphs.BubbleSortGraph(n)
sage: g.order() == factorial(n)
True
```

```
>>> from sage.all import *
>>> n = randint(Integer(1), Integer(8))
>>> g = graphs.BubbleSortGraph(n)
>>> g.order() == factorial(n)
True
```

See also

- `permutahedron()`

AUTHORS:

- Michael Yurko (2009-09-01)

static BuckyBall(*immutable=False*)

Return the Bucky Ball graph.

This graph is a 3-regular 60-vertex planar graph. Its vertices and edges correspond precisely to the carbon atoms and bonds in buckminsterfullerene. When embedded on a sphere, its 12 pentagon and 20 hexagon faces are arranged exactly as the sections of a soccer ball.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

The Bucky Ball is planar:

```
sage: g = graphs.BuckyBall()
sage: g.is_planar()
True
```

```
>>> from sage.all import *
>>> g = graphs.BuckyBall()
>>> g.is_planar()
True
```

The Bucky Ball can also be created by extracting the 1-skeleton of the Bucky Ball polyhedron, but this is much slower:

```
sage: g = polytopes.buckyball().vertex_graph()
sage: g.remove_loops()
sage: h = graphs.BuckyBall()
sage: g.is_isomorphic(h)
True
```

```
>>> from sage.all import *
>>> g = polytopes.buckyball().vertex_graph()
>>> g.remove_loops()
>>> h = graphs.BuckyBall()
>>> g.is_isomorphic(h)
True
```

The graph is returned along with an attractive embedding:

```
sage: g = graphs.BuckyBall()  # long time
sage: g.plot(vertex_labels=False, vertex_size=10).show()        # long time,␣
↪needs sage.plot
```

```
>>> from sage.all import *
>>> g = graphs.BuckyBall()  # long time
>>> g.plot(vertex_labels=False, vertex_size=Integer(10)).show()        # long␣
↪time, needs sage.plot
```

static BullGraph(*immutable=False*)

Return a bull graph with 5 nodes.

A bull graph is named for its shape. It's a triangle with horns. See the Wikipedia article Bull_graph for more information.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

PLOTTING:

Upon construction, the position dictionary is filled to override the spring-layout algorithm. By convention, the bull graph is drawn as a triangle with the first node (0) on the bottom. The second and third nodes (1 and 2) complete the triangle. Node 3 is the horn connected to 1 and node 4 is the horn connected to node 2.

EXAMPLES:

Construct and show a bull graph:

```
sage: g = graphs.BullGraph(); g
Bull graph: Graph on 5 vertices
sage: g.show()                              # long time
↪  # needs sage.plot
```

```
>>> from sage.all import *
>>> g = graphs.BullGraph(); g
Bull graph: Graph on 5 vertices
>>> g.show()                              # long time
↪# needs sage.plot
```

The bull graph has 5 vertices and 5 edges. Its radius is 2, its diameter 3, and its girth 3. The bull graph is planar with chromatic number 3 and chromatic index also 3:

```
sage: g.order(); g.size()
5
5
sage: g.radius(); g.diameter(); g.girth()
2
3
3
sage: g.chromatic_number()
3
```

```
>>> from sage.all import *
>>> g.order(); g.size()
5
5
>>> g.radius(); g.diameter(); g.girth()
2
3
3
>>> g.chromatic_number()
3
```

The bull graph has chromatic polynomial $x(x-2)(x-1)^3$ and Tutte polynomial $x^4+x^3+x^2y$. Its characteristic polynomial is $x(x^2-x-3)(x^2+x-1)$, which follows from the definition of characteristic polynomials for graphs, i.e. $\det(xI-A)$, where x is a variable, A the adjacency matrix of the graph, and I the identity matrix of the same dimensions as A:

```
sage: chrompoly = g.chromatic_polynomial()
sage: x = chrompoly.parent()('x')
sage: x * (x - 2) * (x - 1)^3 == chrompoly
True

sage: charpoly = g.characteristic_polynomial()
sage: M = g.adjacency_matrix(); M
[0 1 1 0 0]
[1 0 1 1 0]
[1 1 0 0 1]
[0 1 0 0 0]
[0 0 1 0 0]
sage: Id = identity_matrix(ZZ, M.nrows())
sage: D = x*Id - M
sage: D.determinant() == charpoly                                          ␣
↪  # needs sage.symbolic
True
sage: x * (x^2 - x - 3) * (x^2 + x - 1) == charpoly
True
```

```
>>> from sage.all import *
>>> chrompoly = g.chromatic_polynomial()
>>> x = chrompoly.parent()('x')
>>> x * (x - Integer(2)) * (x - Integer(1))**Integer(3) == chrompoly
True

>>> charpoly = g.characteristic_polynomial()
>>> M = g.adjacency_matrix(); M
[0 1 1 0 0]
[1 0 1 1 0]
[1 1 0 0 1]
[0 1 0 0 0]
[0 0 1 0 0]
>>> Id = identity_matrix(ZZ, M.nrows())
>>> D = x*Id - M
>>> D.determinant() == charpoly
↪# needs sage.symbolic
True
>>> x * (x**Integer(2) - x - Integer(3)) * (x**Integer(2) + x - Integer(1))␣
↪== charpoly
True
```

static ButterflyGraph(*immutable=False*)

Return the butterfly graph.

Let C_3 be the cycle graph on 3 vertices. The butterfly or bowtie graph is obtained by joining two copies of C_3 at a common vertex, resulting in a graph that is isomorphic to the friendship graph F_2. See the Wikipedia article Butterfly_graph for more information.

See also

- *FriendshipGraph()*

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

The butterfly graph is a planar graph on 5 vertices and having 6 edges:

```
sage: G = graphs.ButterflyGraph(); G
Butterfly graph: Graph on 5 vertices
sage: G.show()                              # long time                          ␣
↪  # needs sage.plot
sage: G.is_planar()
True
sage: G.order()
5
sage: G.size()
6
```

```
>>> from sage.all import *
>>> G = graphs.ButterflyGraph(); G
Butterfly graph: Graph on 5 vertices
>>> G.show()                             # long time
↪# needs sage.plot
>>> G.is_planar()
True
>>> G.order()
5
>>> G.size()
6
```

It has diameter 2, girth 3, and radius 1:

```
sage: G.diameter()
2
sage: G.girth()
3
sage: G.radius()
1
```

```
>>> from sage.all import *
>>> G.diameter()
2
>>> G.girth()
3
>>> G.radius()
1
```

The butterfly graph is Eulerian, with chromatic number 3:

```
sage: G.is_eulerian()
True
sage: G.chromatic_number()
3
```

```
>>> from sage.all import *
>>> G.is_eulerian()
True
>>> G.chromatic_number()
3
```

static CaiFurerImmermanGraph(*G*, *twisted=False*, *immutable=None*)

Return the a Cai-Furer-Immerman graph from G, possibly a twisted one, and a partition of its nodes.

A Cai-Furer-Immerman graph from/on G is a graph created by applying the transformation described in [CFI1992] on a graph G, that is substituting every vertex v in G with a Furer gadget $F(v)$ of order d equal to the degree of the vertex, and then substituting every edge (v, u) in G with a pair of edges, one connecting the two "a" nodes of $F(v)$ and $F(u)$ and the other their two "b" nodes. The returned coloring of the vertices is made by the union of the colorings of each single Furer gadget, individualised for each vertex of G. To understand better what these "a" and "b" nodes are, see the documentation on Furer gadgets.

Furthermore, this method can apply what is described in the paper mentioned above as a "twist" on an edge, that is taking only one of the pairs of edges introduced in the new graph and swap two of their extremes, making each edge go from an "a" node to a "b" node. This is only doable if the original graph G is connected.

A CaiFurerImmerman graph on a graph with no balanced vertex separators smaller than s and its twisted version cannot be distinguished by k-WL for any k < s.

INPUT:

- `G` – an undirected graph on which to construct the Cai-Furer-Immerman graph
- `twisted` – a boolean indicating if the version to construct is a twisted one or not

OUTPUT:

- `H` – the Cai-Furer-Immerman graph on `G`
- `coloring` – list of list of vertices, representing the partition induced by the coloring on `H`
- `immutable` – boolean (default: `None`); whether to create a mutable/immutable graph. `immutable=None` (default) means that the input graph G and the returned graph will behave the same way.

EXAMPLES:

CaiFurerImmerman graph with no balanced vertex separator smaller than 2

```
sage: G = graphs.CycleGraph(4)
sage: CFI, p = graphs.CaiFurerImmermanGraph(G)
sage: sorted(CFI, key=str)
[(0, ()), (0, (0, 'a')), (0, (0, 'b')), (0, (0, 1)), (0, (1, 'a')),
 (0, (1, 'b')), (1, ()), (1, (0, 'a')), (1, (0, 'b')), (1, (0, 1)),
 (1, (1, 'a')), (1, (1, 'b')), (2, ()), (2, (0, 'a')), (2, (0, 'b')),
 (2, (0, 1)), (2, (1, 'a')), (2, (1, 'b')), (3, ()), (3, (0, 'a')),
 (3, (0, 'b')), (3, (0, 1)), (3, (1, 'a')), (3, (1, 'b'))]
sage: sorted(CFI.edge_iterator(), key=str)
[((0, ()), (0, (0, 'b')), None),
 ((0, ()), (0, (1, 'b')), None),
 ((0, (0, 'a')), (1, (0, 'a')), None),
 ((0, (0, 'b')), (1, (0, 'b')), None),
 ((0, (0, 1)), (0, (0, 'a')), None),
 ((0, (0, 1)), (0, (1, 'a')), None),
 ((0, (1, 'a')), (3, (0, 'a')), None),
```

(continues on next page)

(continued from previous page)

```
 ((0, (1, 'b')), (3, (0, 'b')), None),
 ((1, ()), (1, (0, 'b')), None),
 ((1, ()), (1, (1, 'b')), None),
 ((1, (0, 1)), (1, (0, 'a')), None),
 ((1, (0, 1)), (1, (1, 'a')), None),
 ((1, (1, 'a')), (2, (0, 'a')), None),
 ((1, (1, 'b')), (2, (0, 'b')), None),
 ((2, ()), (2, (0, 'b')), None),
 ((2, ()), (2, (1, 'b')), None),
 ((2, (0, 1)), (2, (0, 'a')), None),
 ((2, (0, 1)), (2, (1, 'a')), None),
 ((2, (1, 'a')), (3, (1, 'a')), None),
 ((2, (1, 'b')), (3, (1, 'b')), None),
 ((3, ()), (3, (0, 'b')), None),
 ((3, ()), (3, (1, 'b')), None),
 ((3, (0, 1)), (3, (0, 'a')), None),
 ((3, (0, 1)), (3, (1, 'a')), None)]
```

```
>>> from sage.all import *
>>> G = graphs.CycleGraph(Integer(4))
>>> CFI, p = graphs.CaiFurerImmermanGraph(G)
>>> sorted(CFI, key=str)
[(0, ()), (0, (0, 'a')), (0, (0, 'b')), (0, (0, 1)), (0, (1, 'a')),
 (0, (1, 'b')), (1, ()), (1, (0, 'a')), (1, (0, 'b')), (1, (0, 1)),
 (1, (1, 'a')), (1, (1, 'b')), (2, ()), (2, (0, 'a')), (2, (0, 'b')),
 (2, (0, 1)), (2, (1, 'a')), (2, (1, 'b')), (3, ()), (3, (0, 'a')),
 (3, (0, 'b')), (3, (0, 1)), (3, (1, 'a')), (3, (1, 'b'))]
>>> sorted(CFI.edge_iterator(), key=str)
[((0, ()), (0, (0, 'b')), None),
 ((0, ()), (0, (1, 'b')), None),
 ((0, (0, 'a')), (1, (0, 'a')), None),
 ((0, (0, 'b')), (1, (0, 'b')), None),
 ((0, (0, 1)), (0, (0, 'a')), None),
 ((0, (0, 1)), (0, (1, 'a')), None),
 ((0, (1, 'a')), (3, (0, 'a')), None),
 ((0, (1, 'b')), (3, (0, 'b')), None),
 ((1, ()), (1, (0, 'b')), None),
 ((1, ()), (1, (1, 'b')), None),
 ((1, (0, 1)), (1, (0, 'a')), None),
 ((1, (0, 1)), (1, (1, 'a')), None),
 ((1, (1, 'a')), (2, (0, 'a')), None),
 ((1, (1, 'b')), (2, (0, 'b')), None),
 ((2, ()), (2, (0, 'b')), None),
 ((2, ()), (2, (1, 'b')), None),
 ((2, (0, 1)), (2, (0, 'a')), None),
 ((2, (0, 1)), (2, (1, 'a')), None),
 ((2, (1, 'a')), (3, (1, 'a')), None),
 ((2, (1, 'b')), (3, (1, 'b')), None),
 ((3, ()), (3, (0, 'b')), None),
 ((3, ()), (3, (1, 'b')), None),
 ((3, (0, 1)), (3, (0, 'a')), None),
 ((3, (0, 1)), (3, (1, 'a')), None)]
```

static CameronGraph(*immutable=False*)

Return the Cameron graph.

The Cameron graph is strongly regular with parameters $v = 231, k = 30, \lambda = 9, \mu = 3$.

For more information on the Cameron graph, see https://www.win.tue.nl/~aeb/graphs/Cameron.html.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.CameronGraph()
sage: g.order()
231
sage: g.size()
3465
sage: g.is_strongly_regular(parameters=True)     # long time
(231, 30, 9, 3)
```

```
>>> from sage.all import *
>>> g = graphs.CameronGraph()
>>> g.order()
231
>>> g.size()
3465
>>> g.is_strongly_regular(parameters=True)     # long time
(231, 30, 9, 3)
```

static Caterpillar(*spine*, *immutable=False*)

Return the caterpillar tree with given spine sequence.

A caterpillar tree consists of leaves attached to a path (the "spine").

INPUT:

- **spine – list of nonnegative integers in the form** $[a_1, a_2, \ldots, a_n]$, where a_i is the number of leaves adjacent to the i-th vertex on the spine (except for the first and last vertex, which have $a_1 + 1$ and $a_n + 1$ leaf-neighbors, respectively)
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

OUTPUT:

A caterpillar tree of diameter $n + 1$ on $n + 2 + \sum_{i=1}^{n} a_i$ vertices, n of which are not leaves.

PLOTTING: Upon construction, the position dictionary is filled to override the spring-layout algorithm if the returned graph does not have too many vertices. The spine vertices are positioned on a straight line together with two leaves at its ends. Every edge in the drawing has unit length.

EXAMPLES:

Caterpillars with all-zero spine sequence are paths:

```
sage: graphs.Caterpillar([]).is_isomorphic(graphs.PathGraph(2))
True
sage: graphs.Caterpillar([0]).is_isomorphic(graphs.PathGraph(3))
True
```

(continues on next page)

(continued from previous page)

```
sage: graphs.Caterpillar([0, 0]).is_isomorphic(graphs.PathGraph(4))
True
```

```
>>> from sage.all import *
>>> graphs.Caterpillar([]).is_isomorphic(graphs.PathGraph(Integer(2)))
True
>>> graphs.Caterpillar([Integer(0)]).is_isomorphic(graphs.
↪PathGraph(Integer(3)))
True
>>> graphs.Caterpillar([Integer(0), Integer(0)]).is_isomorphic(graphs.
↪PathGraph(Integer(4)))
True
```

Caterpillars with singleton spine are stars:

```
sage: graphs.Caterpillar([1]).is_isomorphic(graphs.StarGraph(3))
True
sage: graphs.Caterpillar([2]).is_isomorphic(graphs.StarGraph(4))
True
sage: graphs.Caterpillar([3]).is_isomorphic(graphs.StarGraph(5))
True
```

```
>>> from sage.all import *
>>> graphs.Caterpillar([Integer(1)]).is_isomorphic(graphs.
↪StarGraph(Integer(3)))
True
>>> graphs.Caterpillar([Integer(2)]).is_isomorphic(graphs.
↪StarGraph(Integer(4)))
True
>>> graphs.Caterpillar([Integer(3)]).is_isomorphic(graphs.
↪StarGraph(Integer(5)))
True
```

Distinct spine sequences can yield isomorphic caterpillars:

```
sage: graphs.Caterpillar([1,1,2]).is_isomorphic(graphs.Caterpillar([2,1,1]))
True
```

```
>>> from sage.all import *
>>> graphs.Caterpillar([Integer(1),Integer(1),Integer(2)]).is_
↪isomorphic(graphs.Caterpillar([Integer(2),Integer(1),Integer(1)]))
True
```

static Cell120(*immutable=False*)

Return the 120-Cell graph.

This is the adjacency graph of the 120-cell. It has 600 vertices and 1200 edges. For more information, see the Wikipedia article 120-cell.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: # long time, needs sage.rings.number_field
sage: g = graphs.Cell120()
sage: g.size()
1200
sage: g.is_regular(4)
True
sage: g.is_vertex_transitive()
True
```

```
>>> from sage.all import *
>>> # long time, needs sage.rings.number_field
>>> g = graphs.Cell120()
>>> g.size()
1200
>>> g.is_regular(Integer(4))
True
>>> g.is_vertex_transitive()
True
```

static Cell600(*embedding=1*, *immutable=False*)

Return the 600-Cell graph.

This is the adjacency graph of the 600-cell. It has 120 vertices and 720 edges. For more information, see the Wikipedia article 600-cell.

INPUT:

- `embedding` – integer (default: 1); two different embeddings for a plot
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: # long time, needs sage.rings.number_field
sage: g = graphs.Cell600()
sage: g.size()
720
sage: g.is_regular(12)
True
sage: g.is_vertex_transitive()
True
```

```
>>> from sage.all import *
>>> # long time, needs sage.rings.number_field
>>> g = graphs.Cell600()
>>> g.size()
720
>>> g.is_regular(Integer(12))
True
>>> g.is_vertex_transitive()
True
```

static ChessboardGraphGenerator(*dim_list*, *rook=True*, *rook_radius=None*, *bishop=True*, *bishop_radius=None*, *knight=True*, *knight_x=1*, *knight_y=2*, *relabel=False*, *immutable=False*)

Return a Graph built on a d-dimensional chessboard with prescribed dimensions and interconnections.

This function allows to generate many kinds of graphs corresponding to legal movements on a d-dimensional chessboard: Queen Graph, King Graph, Knight Graphs, Bishop Graph, and many generalizations. It also allows to avoid redundant code.

INPUT:

- `dim_list` – iterable (list, set, dict); provides the dimensions $n_1, n_2, \ldots, n_d$, with $n_i \geq 1$, of the chessboard
- `rook` – boolean (default: `True`); indicates whether the chess piece is able to move as a rook, that is at any distance along a dimension
- `rook_radius` – integer (default: `None`); restriction on the rook-like movements to distance at most `rook_radius`
- `bishop` – boolean (default: `True`); indicates whether the chess piece is able to move like a bishop, that is along diagonals
- `bishop_radius` – integer (default: `None`); restriction on the bishop-like movements to distance at most `bishop_radius`
- `knight` – boolean (default: `True`); indicating whether the chess piece is able to move like a knight
- `knight_x` – integer (default: 1); indicates the number on steps the chess piece moves in one dimension when moving like a knight
- `knight_y` – integer (default: 2); indicates the number on steps the chess piece moves in the second dimension when moving like a knight
- `relabel` – boolean (default: `False`); indicates whether the vertices must be relabeled as integers
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

OUTPUT:

- A Graph build on a d-dimensional chessboard with prescribed dimensions, and with edges according given parameters.
- A string encoding the dimensions. This is mainly useful for providing names to graphs.

EXAMPLES:

A $(2, 2)$-King Graph is isomorphic to the complete graph on 4 vertices:

```
sage: G, _ = graphs.ChessboardGraphGenerator( [2,2] )
sage: G.is_isomorphic( graphs.CompleteGraph(4) )
True
```

```
>>> from sage.all import *
>>> G, _ = graphs.ChessboardGraphGenerator( [Integer(2),Integer(2)] )
>>> G.is_isomorphic( graphs.CompleteGraph(Integer(4)) )
True
```

A Rook's Graph in 2 dimensions is isomorphic to the Cartesian product of 2 complete graphs:

```
sage: G, _ = graphs.ChessboardGraphGenerator([3,4], rook=True, rook_
↪radius=None, bishop=False, knight=False)
sage: H = (graphs.CompleteGraph(3)).cartesian_product(graphs.CompleteGraph(4))
```

(continues on next page)

(continued from previous page)

```
sage: G.is_isomorphic(H)
True
```

```
>>> from sage.all import *
>>> G, _ = graphs.ChessboardGraphGenerator([Integer(3),Integer(4)], rook=True,
↪ rook_radius=None, bishop=False, knight=False)
>>> H = (graphs.CompleteGraph(Integer(3))).cartesian_product(graphs.
↪CompleteGraph(Integer(4)))
>>> G.is_isomorphic(H)
True
```

static ChvatalGraph(*immutable=False*)

Return the Chvatal graph.

Chvatal graph is one of the few known graphs to satisfy Grunbaum's conjecture that for every m, n, there is an m-regular, m-chromatic graph of girth at least n. For more information, see the Wikipedia article Chv%C3%A1tal_graph.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

The Chvatal graph has 12 vertices and 24 edges. It is a 4-regular, 4-chromatic graph with radius 2, diameter 2, and girth 4:

```
sage: G = graphs.ChvatalGraph(); G
Chvatal graph: Graph on 12 vertices
sage: G.order(); G.size()
12
24
sage: G.degree()
[4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4]
sage: G.chromatic_number()
4
sage: G.radius(); G.diameter(); G.girth()
2
2
4
```

```
>>> from sage.all import *
>>> G = graphs.ChvatalGraph(); G
Chvatal graph: Graph on 12 vertices
>>> G.order(); G.size()
12
24
>>> G.degree()
[4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4]
>>> G.chromatic_number()
4
>>> G.radius(); G.diameter(); G.girth()
2
```

(continues on next page)

(continued from previous page)

```
2
4
```

static CirculantGraph(*n*, *adjacency*, *immutable=False*, *name=None*)

Return a circulant graph with n nodes.

A circulant graph has the property that the vertex i is connected with the vertices $i + j$ and $i - j$ for each j in `adjacency`.

INPUT:

- `n` – number of vertices in the graph
- `adjacency` – the list of j values
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph
- `name` – string (default: `None`); used as the name of the returned graph when set

PLOTTING: Upon construction, the position dictionary is filled to override the spring-layout algorithm. By convention, each circulant graph will be displayed with the first (0) node at the top, with the rest following in a counterclockwise manner.

Filling the position dictionary in advance adds $O(n)$ to the constructor.

See also

- *`is_circulant()`* – checks whether a (di)graph is circulant, and/or returns all possible sets of parameters.

EXAMPLES: Compare plotting using the predefined layout and networkx:

```
sage: # needs networkx
sage: import networkx
sage: n = networkx.cycle_graph(23)
sage: spring23 = Graph(n)
sage: posdict23 = graphs.CirculantGraph(23,2)
sage: spring23.show()                    # long time
sage: posdict23.show()  # long time
```

```
>>> from sage.all import *
>>> # needs networkx
>>> import networkx
>>> n = networkx.cycle_graph(Integer(23))
>>> spring23 = Graph(n)
>>> posdict23 = graphs.CirculantGraph(Integer(23),Integer(2))
>>> spring23.show()                    # long time
>>> posdict23.show()  # long time
```

We next view many cycle graphs as a Sage graphics array. First we use the `CirculantGraph` constructor, which fills in the position dictionary:

```
sage: g = []
sage: j = []
```

(continues on next page)

(continued from previous page)

```
sage: for i in range(9):
....:     k = graphs.CirculantGraph(i+4, i+1)
....:     g.append(k)
sage: for i in range(3):
....:     n = []
....:     for m in range(3):
....:         n.append(g[3*i + m].plot(vertex_size=50, vertex_labels=False))
....:     j.append(n)
sage: G = graphics_array(j)
sage: G.show()                          # long time
```

```
>>> from sage.all import *
>>> g = []
>>> j = []
>>> for i in range(Integer(9)):
...     k = graphs.CirculantGraph(i+Integer(4), i+Integer(1))
...     g.append(k)
>>> for i in range(Integer(3)):
...     n = []
...     for m in range(Integer(3)):
...         n.append(g[Integer(3)*i + m].plot(vertex_size=Integer(50), vertex_
↪labels=False))
...     j.append(n)
>>> G = graphics_array(j)
>>> G.show()                          # long time
```

Compare to plotting with the spring-layout algorithm:

```
sage: # needs networkx sage.plot
sage: g = []
sage: j = []
sage: for i in range(9):
....:     spr = networkx.cycle_graph(i+3)
....:     k = Graph(spr)
....:     g.append(k)
sage: for i in range(3):
....:  n = []
....:  for m in range(3):
....:      n.append(g[3*i + m].plot(vertex_size=50, vertex_labels=False))
....:  j.append(n)
sage: G = graphics_array(j)
sage: G.show()                          # long time
```

```
>>> from sage.all import *
>>> # needs networkx sage.plot
>>> g = []
>>> j = []
>>> for i in range(Integer(9)):
...     spr = networkx.cycle_graph(i+Integer(3))
...     k = Graph(spr)
...     g.append(k)
>>> for i in range(Integer(3)):
```

(continues on next page)

(continued from previous page)

```
...   n = []
...   for m in range(Integer(3)):
...       n.append(g[Integer(3)*i + m].plot(vertex_size=Integer(50), vertex_
↪labels=False))
...   j.append(n)
>>> G = graphics_array(j)
>>> G.show()                            # long time
```

Passing a 1 into adjacency should give the cycle.

```
sage: graphs.CirculantGraph(6,1) == graphs.CycleGraph(6)
True
sage: graphs.CirculantGraph(7,[1,3]).edges(sort=True, labels=false)
[(0, 1),
(0, 3),
(0, 4),
(0, 6),
(1, 2),
(1, 4),
(1, 5),
(2, 3),
(2, 5),
(2, 6),
(3, 4),
(3, 6),
(4, 5),
(5, 6)]
```

```
>>> from sage.all import *
>>> graphs.CirculantGraph(Integer(6),Integer(1)) == graphs.
↪CycleGraph(Integer(6))
True
>>> graphs.CirculantGraph(Integer(7),[Integer(1),Integer(3)]).edges(sort=True,
↪ labels=false)
[(0, 1),
(0, 3),
(0, 4),
(0, 6),
(1, 2),
(1, 4),
(1, 5),
(2, 3),
(2, 5),
(2, 6),
(3, 4),
(3, 6),
(4, 5),
(5, 6)]
```

static CircularLadderGraph(*n*, *immutable=False*)

Return a circular ladder graph with $2 * n$ nodes.

A Circular ladder graph is a ladder graph that is connected at the ends, i.e.: a ladder bent around so that top

meets bottom. Thus it can be described as two parallel cycle graphs connected at each corresponding node pair.

PLOTTING: Upon construction, the position dictionary is filled to override the spring-layout algorithm. By convention, the circular ladder graph is displayed as an inner and outer cycle pair, with the first n nodes drawn on the inner circle. The first (0) node is drawn at the top of the inner-circle, moving clockwise after that. The outer circle is drawn with the $(n+1)$-th node at the top, then counterclockwise as well. When $n == 2$, we rotate the outer circle by an angle of $\pi/8$ to ensure that all edges are visible (otherwise the 4 vertices of the graph would be placed on a single line).

INPUT:

- `n` – nonnegative integer
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

Construct and show a circular ladder graph with 26 nodes:

```
sage: g = graphs.CircularLadderGraph(13)
sage: g.show()                              # long time                 ␣
↪  # needs sage.plot
```

```
>>> from sage.all import *
>>> g = graphs.CircularLadderGraph(Integer(13))
>>> g.show()                              # long time
↪# needs sage.plot
```

Create several circular ladder graphs in a Sage graphics array:

```
sage: g = []
sage: j = []
sage: for i in range(9):
....:     k = graphs.CircularLadderGraph(i+3)
....:     g.append(k)
sage: for i in range(3):                                                  ␣
↪  # needs sage.plot
....:     n = []
....:     for m in range(3):
....:         n.append(g[3*i + m].plot(vertex_size=50, vertex_labels=False))
....:     j.append(n)
sage: G = graphics_array(j)                                               ␣
↪  # needs sage.plot
sage: G.show()                              # long time                 ␣
↪  # needs sage.plot
```

```
>>> from sage.all import *
>>> g = []
>>> j = []
>>> for i in range(Integer(9)):
...     k = graphs.CircularLadderGraph(i+Integer(3))
...     g.append(k)
>>> for i in range(Integer(3)):                                           ␣
↪          # needs sage.plot
...     n = []
```

(continues on next page)

(continued from previous page)

```
...     for m in range(Integer(3)):
...         n.append(g[Integer(3)*i + m].plot(vertex_size=Integer(50), vertex_
↪labels=False))
...     j.append(n)
>>> G = graphics_array(j)
↪# needs sage.plot
>>> G.show()                            # long time
↪# needs sage.plot
```

static ClawGraph(*immutable=False*)

Return a claw graph.

A claw graph is named for its shape. It is actually a complete bipartite graph with `(n1, n2) = (1, 3)`.

PLOTTING: See *CompleteBipartiteGraph()*.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

Show a Claw graph:

```
sage: (graphs.ClawGraph()).show()        # long time                          ␣
↪  # needs sage.plot
```

```
>>> from sage.all import *
>>> (graphs.ClawGraph()).show()        # long time
↪# needs sage.plot
```

Inspect a Claw graph:

```
sage: G = graphs.ClawGraph()
sage: G
Claw graph: Graph on 4 vertices
```

```
>>> from sage.all import *
>>> G = graphs.ClawGraph()
>>> G
Claw graph: Graph on 4 vertices
```

static ClebschGraph(*immutable=False*)

Return the Clebsch graph.

See the Wikipedia article Clebsch_graph for more information.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.ClebschGraph()
sage: g.automorphism_group().cardinality()                                    ␣
↪  # needs sage.groups
```

(continues on next page)

(continued from previous page)

```
1920
sage: g.girth()
4
sage: g.chromatic_number()
4
sage: g.diameter()
2
sage: g.show(figsize=[10, 10])          # long time
↪  # needs sage.plot
```

```
>>> from sage.all import *
>>> g = graphs.ClebschGraph()
>>> g.automorphism_group().cardinality()
↪# needs sage.groups
1920
>>> g.girth()
4
>>> g.chromatic_number()
4
>>> g.diameter()
2
>>> g.show(figsize=[Integer(10), Integer(10)])          # long time
↪                  # needs sage.plot
```

static CompleteBipartiteGraph(*p*, *q*, *set_position=True*, *immutable=False*, *name=None*)

Return a Complete Bipartite Graph on $p + q$ vertices.

A Complete Bipartite Graph is a graph with its vertices partitioned into two groups, $V_1 = \{0, ..., p-1\}$ and $V_2 = \{p, ..., p+q-1\}$. Each $u \in V_1$ is connected to every $v \in V_2$.

INPUT:

- `p`, `q` – number of vertices in each side
- `set_position` – boolean (default: `True`); if set to `True`, we assign positions to the vertices so that the set of cardinality p is on the line $y = 1$ and the set of cardinality q is on the line $y = 0$.
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph
- `name` – string (default: `None`); used as the name of the returned graph when set

PLOTTING: Upon construction, the position dictionary is filled to override the spring-layout algorithm. By convention, each complete bipartite graph will be displayed with the first p nodes on the top row (at $y = 1$) from left to right. The remaining q nodes appear at $y = 0$, also from left to right. The shorter row (partition with fewer nodes) is stretched to the same length as the longer row, unless the shorter row has 1 node; in which case it is centered. The x values in the plot are in domain $[0, \max(p, q)]$.

In the Complete Bipartite graph, there is a visual difference in using the spring-layout algorithm vs. the position dictionary used in this constructor. The position dictionary flattens the graph and separates the partitioned nodes, making it clear which nodes an edge is connected to. The Complete Bipartite graph plotted with the spring-layout algorithm tends to center the nodes in p (see `spring_med` in examples below), thus overlapping its nodes and edges, making it typically hard to decipher.

Filling the position dictionary in advance adds $O(n)$ to the constructor. Feel free to race the constructors below in the examples section. The much larger difference is the time added by the spring-layout algorithm when plotting. (Also shown in the example below). The spring model is typically described as $O(n^3)$, as appears to be the case in the NetworkX source code.

EXAMPLES:

Two ways of constructing the complete bipartite graph, using different layout algorithms:

```
sage: # needs networkx
sage: import networkx
sage: n = networkx.complete_bipartite_graph(389, 157)    # long time
sage: spring_big = Graph(n)              # long time
sage: posdict_big = graphs.CompleteBipartiteGraph(389, 157)         # long time
```

```
>>> from sage.all import *
>>> # needs networkx
>>> import networkx
>>> n = networkx.complete_bipartite_graph(Integer(389), Integer(157))    #␣
↪long time
>>> spring_big = Graph(n)              # long time
>>> posdict_big = graphs.CompleteBipartiteGraph(Integer(389), Integer(157))  ␣
↪     # long time
```

Compare the plotting:

```
sage: n = networkx.complete_bipartite_graph(11, 17)                          ␣
↪  # needs networkx
sage: spring_med = Graph(n)                                                  ␣
↪  # needs networkx
sage: posdict_med = graphs.CompleteBipartiteGraph(11, 17)
```

```
>>> from sage.all import *
>>> n = networkx.complete_bipartite_graph(Integer(11), Integer(17))          ␣
↪                  # needs networkx
>>> spring_med = Graph(n)
↪# needs networkx
>>> posdict_med = graphs.CompleteBipartiteGraph(Integer(11), Integer(17))
```

Notice here how the spring-layout tends to center the nodes of $n1$:

```
sage: spring_med.show()                  # long time                         ␣
↪  # needs networkx
sage: posdict_med.show()                 # long time                         ␣
↪  # needs sage.plot
```

```
>>> from sage.all import *
>>> spring_med.show()                  # long time
↪# needs networkx
>>> posdict_med.show()                 # long time
↪# needs sage.plot
```

View many complete bipartite graphs with a Sage Graphics Array, with this constructor (i.e., the position dictionary filled):

```
sage: g = []
sage: j = []
sage: for i in range(9):
....:     k = graphs.CompleteBipartiteGraph(i+1,4)
```

(continues on next page)

(continued from previous page)

```
....:     g.append(k)
sage: for i in range(3):                                                    ␣
↪  # needs sage.plot
....:     n = []
....:     for m in range(3):
....:         n.append(g[3*i + m].plot(vertex_size=50, vertex_labels=False))
....:     j.append(n)
sage: G = graphics_array(j)                                                 ␣
↪  # needs sage.plot
sage: G.show()                              # long time                     ␣
↪  # needs sage.plot
```

```
>>> from sage.all import *
>>> g = []
>>> j = []
>>> for i in range(Integer(9)):
...     k = graphs.CompleteBipartiteGraph(i+Integer(1),Integer(4))
...     g.append(k)
>>> for i in range(Integer(3)):                                             ␣
↪         # needs sage.plot
...     n = []
...     for m in range(Integer(3)):
...         n.append(g[Integer(3)*i + m].plot(vertex_size=Integer(50), vertex_
↪labels=False))
...     j.append(n)
>>> G = graphics_array(j)
↪# needs sage.plot
>>> G.show()                                # long time
↪# needs sage.plot
```

We compare to plotting with the spring-layout algorithm:

```
sage: # needs networkx sage.plot
sage: g = []
sage: j = []
sage: for i in range(9):
....:     spr = networkx.complete_bipartite_graph(i+1,4)
....:     k = Graph(spr)
....:     g.append(k)
sage: for i in range(3):
....:     n = []
....:     for m in range(3):
....:         n.append(g[3*i + m].plot(vertex_size=50, vertex_labels=False))
....:     j.append(n)
sage: G = graphics_array(j)
sage: G.show()                              # long time
```

```
>>> from sage.all import *
>>> # needs networkx sage.plot
>>> g = []
>>> j = []
>>> for i in range(Integer(9)):
```

(continues on next page)

(continued from previous page)

```
...       spr = networkx.complete_bipartite_graph(i+Integer(1),Integer(4))
...       k = Graph(spr)
...       g.append(k)
>>> for i in range(Integer(3)):
...       n = []
...       for m in range(Integer(3)):
...           n.append(g[Integer(3)*i + m].plot(vertex_size=Integer(50), vertex_
↪labels=False))
...       j.append(n)
>>> G = graphics_array(j)
>>> G.show()                              # long time
```

Issue #12155:

```
sage: graphs.CompleteBipartiteGraph(5,6).complement()
complement(Complete bipartite graph of order 5+6): Graph on 11 vertices
```

```
>>> from sage.all import *
>>> graphs.CompleteBipartiteGraph(Integer(5),Integer(6)).complement()
complement(Complete bipartite graph of order 5+6): Graph on 11 vertices
```

static CompleteGraph(*n*, *immutable=False*)

Return a complete graph on n nodes.

A Complete Graph is a graph in which all nodes are connected to all other nodes.

INPUT:

- `n` – nonnegative integer; the number of vertices of the complete graph
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

PLOTTING: Upon construction, the position dictionary is filled to override the spring-layout algorithm. By convention, each complete graph will be displayed with the first (0) node at the top, with the rest following in a counterclockwise manner.

In the complete graph, there is a big difference visually in using the spring-layout algorithm vs. the position dictionary used in this constructor. The position dictionary flattens the graph, making it clear which nodes an edge is connected to. But the complete graph offers a good example of how the spring-layout works. The edges push outward (everything is connected), causing the graph to appear as a 3-dimensional pointy ball. (See examples below).

EXAMPLES:

We view many Complete graphs with a Sage Graphics Array, first with this constructor (i.e., the position dictionary filled):

```
sage: g = []
sage: j = []
sage: for i in range(9):
....:     k = graphs.CompleteGraph(i+3)
....:     g.append(k)
sage: for i in range(3):
....:     n = []
....:     for m in range(3):
....:         n.append(g[3*i + m].plot(vertex_size=50, vertex_labels=False))
```

(continues on next page)

(continued from previous page)

```
....:     j.append(n)
sage: G = graphics_array(j)
sage: G.show()                          # long time
```

```
>>> from sage.all import *
>>> g = []
>>> j = []
>>> for i in range(Integer(9)):
...     k = graphs.CompleteGraph(i+Integer(3))
...     g.append(k)
>>> for i in range(Integer(3)):
...     n = []
...     for m in range(Integer(3)):
...         n.append(g[Integer(3)*i + m].plot(vertex_size=Integer(50), vertex_
↪labels=False))
...     j.append(n)
>>> G = graphics_array(j)
>>> G.show()                          # long time
```

We compare to plotting with the spring-layout algorithm:

```
sage: # needs networkx sage.plot
sage: import networkx
sage: g = []
sage: j = []
sage: for i in range(9):
....:     spr = networkx.complete_graph(i+3)
....:     k = Graph(spr)
....:     g.append(k)
sage: for i in range(3):
....:     n = []
....:     for m in range(3):
....:         n.append(g[3*i + m].plot(vertex_size=50, vertex_labels=False))
....:     j.append(n)
sage: G = graphics_array(j)
sage: G.show()                          # long time
```

```
>>> from sage.all import *
>>> # needs networkx sage.plot
>>> import networkx
>>> g = []
>>> j = []
>>> for i in range(Integer(9)):
...     spr = networkx.complete_graph(i+Integer(3))
...     k = Graph(spr)
...     g.append(k)
>>> for i in range(Integer(3)):
...     n = []
...     for m in range(Integer(3)):
...         n.append(g[Integer(3)*i + m].plot(vertex_size=Integer(50), vertex_
↪labels=False))
...     j.append(n)
```

(continues on next page)

(continued from previous page)

```
>>> G = graphics_array(j)
>>> G.show()                              # long time
```

Compare the constructors (results will vary):

```
sage: # needs networkx
sage: import networkx
sage: t = cputime()
sage: n = networkx.complete_graph(389); spring389 = Graph(n)
sage: cputime(t)  # random
0.59203700000000126
sage: t = cputime()
sage: posdict389 = graphs.CompleteGraph(389)
sage: cputime(t)  # random
0.6680419999999998
```

```
>>> from sage.all import *
>>> # needs networkx
>>> import networkx
>>> t = cputime()
>>> n = networkx.complete_graph(Integer(389)); spring389 = Graph(n)
>>> cputime(t)  # random
0.59203700000000126
>>> t = cputime()
>>> posdict389 = graphs.CompleteGraph(Integer(389))
>>> cputime(t)  # random
0.6680419999999998
```

We compare plotting:

```
sage: # needs networkx
sage: import networkx
sage: n = networkx.complete_graph(23)
sage: spring23 = Graph(n)
sage: posdict23 = graphs.CompleteGraph(23)
sage: spring23.show()                    # long time                    ␣
↪  # needs sage.plot
sage: posdict23.show()                   # long time                    ␣
↪  # needs sage.plot
```

```
>>> from sage.all import *
>>> # needs networkx
>>> import networkx
>>> n = networkx.complete_graph(Integer(23))
>>> spring23 = Graph(n)
>>> posdict23 = graphs.CompleteGraph(Integer(23))
>>> spring23.show()                      # long time
↪# needs sage.plot
>>> posdict23.show()                     # long time
↪# needs sage.plot
```

static CompleteMultipartiteGraph(*L*, *immutable=False*)

Return a complete multipartite graph.

INPUT:

- `L` – list of integers; the respective sizes of the components
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

PLOTTING: Produce a layout of the vertices so that vertices in the same vertex set are adjacent and clearly separated from vertices in other vertex sets.

This is done by calculating the vertices of an r-gon then calculating the slope between adjacent vertices. We then 'walk' around the r-gon placing graph vertices in regular intervals between adjacent vertices of the r-gon.

Makes a nicely organized graph like in this picture: https://commons.wikimedia.org/wiki/File:Turan_13-4.svg

EXAMPLES:

A complete tripartite graph with sets of sizes $5, 6, 8$:

```
sage: g = graphs.CompleteMultipartiteGraph([5, 6, 8]); g
Multipartite Graph with set sizes [5, 6, 8]: Graph on 19 vertices
```

```
>>> from sage.all import *
>>> g = graphs.CompleteMultipartiteGraph([Integer(5), Integer(6),␣
↪Integer(8)]); g
Multipartite Graph with set sizes [5, 6, 8]: Graph on 19 vertices
```

It clearly has a chromatic number of 3:

```
sage: g.chromatic_number()
3
```

```
>>> from sage.all import *
>>> g.chromatic_number()
3
```

static ConwaySmith_for_3S7(*immutable=False*)

Return the Conway-Smith graph related to $3Sym(7)$.

This is a distance-regular graph with intersection array $[10, 6, 4, 1; 1, 2, 6, 10]$.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: G = graphs.ConwaySmith_for_3S7()                                          ␣
↪  # needs sage.modules sage.rings.finite_rings sage.rings.number_field
sage: G.is_distance_regular(True)                                               ␣
↪  # needs sage.modules sage.rings.finite_rings sage.rings.number_field
([10, 6, 4, 1, None], [None, 1, 2, 6, 10])
```

```
>>> from sage.all import *
>>> G = graphs.ConwaySmith_for_3S7()
↪# needs sage.modules sage.rings.finite_rings sage.rings.number_field
>>> G.is_distance_regular(True)
↪# needs sage.modules sage.rings.finite_rings sage.rings.number_field
([10, 6, 4, 1, None], [None, 1, 2, 6, 10])
```

REFERENCES:

A description and construction of this graph can be found in [BCN1989] p. 399.

static CorrelationGraph(*seqs*, *alpha*, *include_anticorrelation*, *immutable=False*)

Return a correlation graph with a node per sequence in `seqs`.

Edges are added between nodes where the corresponding sequences have a correlation coefficient greater than alpha.

If `include_anticorrelation` is `True`, then edges are also added between nodes with correlation coefficient less than `-alpha`.

INPUT:

- `seqs` – list of sequences, that is a list of lists
- `alpha` – float; threshold on the correlation coefficient between two sequences for adding an edge
- `include_anticorrelation` – boolean; whether to add edges between nodes with correlation coefficient less than `-alpha` or not
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: # needs numpy
sage: from sage.graphs.generators.basic import CorrelationGraph
sage: data = [[1,2,3], [4,5,6], [7,8,9999]]
sage: CG1 = CorrelationGraph(data, 0.9, False)
sage: CG2 = CorrelationGraph(data, 0.9, True)
sage: CG3 = CorrelationGraph(data, 0.1, True)
sage: CG1.edges(sort=False)
[(0, 0, None), (0, 1, None), (1, 1, None), (2, 2, None)]
sage: CG2.edges(sort=False)
[(0, 0, None), (0, 1, None), (1, 1, None), (2, 2, None)]
sage: CG3.edges(sort=False)
[(0, 0, None), (0, 1, None), (0, 2, None), (1, 1, None), (1, 2, None), (2, 2,␣
↪None)]
```

```
>>> from sage.all import *
>>> # needs numpy
>>> from sage.graphs.generators.basic import CorrelationGraph
>>> data = [[Integer(1),Integer(2),Integer(3)], [Integer(4),Integer(5),
↪Integer(6)], [Integer(7),Integer(8),Integer(9999)]]
>>> CG1 = CorrelationGraph(data, RealNumber('0.9'), False)
>>> CG2 = CorrelationGraph(data, RealNumber('0.9'), True)
>>> CG3 = CorrelationGraph(data, RealNumber('0.1'), True)
>>> CG1.edges(sort=False)
[(0, 0, None), (0, 1, None), (1, 1, None), (2, 2, None)]
>>> CG2.edges(sort=False)
[(0, 0, None), (0, 1, None), (1, 1, None), (2, 2, None)]
>>> CG3.edges(sort=False)
[(0, 0, None), (0, 1, None), (0, 2, None), (1, 1, None), (1, 2, None), (2, 2,␣
↪None)]
```

static CossidentePenttilaGraph(*q*, *immutable=False*)

Return the Cossidente-Penttila $((q^3+1)(q+1)/2, (q^2+1)(q-1)/2, (q-3)/2, (q-1)^2/2)$-strongly regular graph

For each odd prime power q, one can partition the points of the $O_6^-(q)$-generalized quadrangle $GQ(q, q^2)$ into two parts, so that on any of them the induced subgraph of the point graph of the GQ has parameters as above [CP2005].

Directly following the construction in [CP2005] is not efficient, as one then needs to construct the dual $GQ(q^2, q)$. Thus we describe here a more efficient approach that we came up with, following a suggestion by T.Penttila. Namely, this partition is invariant under the subgroup $H = \Omega_3(q^2) < O_6^-(q)$. We build the appropriate H, which leaves the form $B(X, Y, Z) = XY + Z^2$ invariant, and pick up two orbits of H on the F_q-points. One them is B-isotropic, and we take the representative $(1 : 0 : 0)$. The other one corresponds to the points of $PG(2, q^2)$ that have all the lines on them either missing the conic specified by B, or intersecting the conic in two points. We take $(1 : 1 : e)$ as the representative. It suffices to pick e so that $e^2 + 1$ is not a square in F_{q^2}. Indeed, The conic can be viewed as the union of $\{(0 : 1 : 0)\}$ and $\{(1 : -t^2 : t) | t \in F_{q^2}\}$. The coefficients of a generic line on $(1 : 1 : e)$ are $[1 : -1 - eb : b]$, for $-1 \neq eb$. Thus, to make sure the intersection with the conic is always even, we need that the discriminant of $1 + (1 + eb)t^2 + tb = 0$ never vanishes, and this is if and only if $e^2 + 1$ is not a square. Further, we need to adjust B, by multiplying it by appropriately chosen ν, so that $(1 : 1 : e)$ becomes isotropic under the relative trace norm $\nu B(X, Y, Z) + (\nu B(X, Y, Z))^q$. The latter is used then to define the graph.

INPUT:

- `q` – an odd prime power
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

For $q = 3$ one gets Sims-Gewirtz graph.

```
sage: G = graphs.CossidentePenttilaGraph(3)      # optional - gap_package_grape
sage: G.is_strongly_regular(parameters=True)     # optional - gap_package_grape
(56, 10, 0, 2)
```

```
>>> from sage.all import *
>>> G = graphs.CossidentePenttilaGraph(Integer(3))      # optional - gap_
↪package_grape
>>> G.is_strongly_regular(parameters=True)     # optional - gap_package_grape
(56, 10, 0, 2)
```

For $q > 3$ one gets new graphs.

```
sage: G = graphs.CossidentePenttilaGraph(5)      # optional - gap_package_grape
sage: G.is_strongly_regular(parameters=True)     # optional - gap_package_grape
(378, 52, 1, 8)
```

```
>>> from sage.all import *
>>> G = graphs.CossidentePenttilaGraph(Integer(5))      # optional - gap_
↪package_grape
>>> G.is_strongly_regular(parameters=True)     # optional - gap_package_grape
(378, 52, 1, 8)
```

static CoxeterGraph(*immutable=False*)

Return the Coxeter graph.

See the Wikipedia article Coxeter_graph.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.CoxeterGraph()
sage: g.automorphism_group().cardinality()                                  ␣
↪  # needs sage.groups
336
sage: g.girth()
7
sage: g.chromatic_number()
3
sage: g.diameter()
4
sage: g.show(figsize=[10, 10])           # long time                        ␣
↪  # needs sage.plot
```

```
>>> from sage.all import *
>>> g = graphs.CoxeterGraph()
>>> g.automorphism_group().cardinality()
↪# needs sage.groups
336
>>> g.girth()
7
>>> g.chromatic_number()
3
>>> g.diameter()
4
>>> g.show(figsize=[Integer(10), Integer(10)])          # long time          ␣
↪                  # needs sage.plot
```

static CubeConnectedCycle(*d*, *immutable=False*)

Return the cube-connected cycle of dimension d.

The cube-connected cycle of order d is the d-dimensional hypercube with each of its vertices replaced by a cycle of length d. This graph has order $d \times 2^d$. The construction is as follows: Construct vertex (x, y) for $0 \leq x < 2^d$, $0 \leq y < d$. For each vertex, (x, y), add an edge between it and $(x, (y-1) \mod d)$, $(x, (y+1) \mod d)$, and $(x \oplus 2^y, y)$, where $\oplus$ is the bitwise xor operator.

For $d = 1$ and 2, the cube-connected cycle graph contains self-loops or multiple edges between a pair of vertices, but for all other d, it is simple.

INPUT:

- `d` – positive integer; the dimension of the desired hypercube as well as the length of the cycle to be placed at each vertex of the d-dimensional hypercube
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

The order of the graph is $d \times 2^d$

```
sage: d = 3
sage: g = graphs.CubeConnectedCycle(d)
sage: len(g) == d*2**d
True
```

```
>>> from sage.all import *
>>> d = Integer(3)
>>> g = graphs.CubeConnectedCycle(d)
>>> len(g) == d*Integer(2)**d
True
```

The diameter of cube-connected cycles for $d > 3$ is $2d + \lfloor \frac{d}{2} \rfloor - 2$

```
sage: d = 4
sage: g = graphs.CubeConnectedCycle(d)
sage: g.diameter() == 2*d+d//2-2
True
```

```
>>> from sage.all import *
>>> d = Integer(4)
>>> g = graphs.CubeConnectedCycle(d)
>>> g.diameter() == Integer(2)*d+d//Integer(2)-Integer(2)
True
```

All vertices have degree 3 when $d > 1$

```
sage: g = graphs.CubeConnectedCycle(5)
sage: all(g.degree(v) == 3 for v in g)
True
```

```
>>> from sage.all import *
>>> g = graphs.CubeConnectedCycle(Integer(5))
>>> all(g.degree(v) == Integer(3) for v in g)
True
```

static CubeGraph(*n*, *embedding=1*, *immutable=False*)

Return the n-cube graph, also called the hypercube in n dimensions.

The hypercube in n dimension is build upon the binary strings on n bits, two of them being adjacent if they differ in exactly one bit. Hence, the distance between two vertices in the hypercube is the Hamming distance.

INPUT:

- `n` – integer; the dimension of the cube graph
- `embedding` – integer (default: 1); two embeddings of the n-cube are available:
  - `1` – the n-cube is projected inside a regular $2n$-gonal polygon by a skew orthogonal projection. See the Wikipedia article Hypercube for more details.
  - `2` – orthogonal projection of the n-cube. This orientation shows columns of independent vertices such that the neighbors of a vertex are located in the columns on the left and on the right. The number of vertices in each column represents rows in Pascal's triangle. See for instance the Wikipedia article 10-cube for more details.
  - `3` – oblique projection of the n-cube. Oblique projection involves aligning one face parallel to the viewer and projecting at a specified angle, maintaining equal size for edges parallel to one axis while applying fixed foreshortening to others. This method simplifies the representation of a four-dimensional hypercube onto a two-dimensional plane, offering a geometrically consistent visualization.
  - `None` or `0` – no embedding is provided

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

The distance between 0100110 and 1011010 is 5, as expected:

```
sage: g = graphs.CubeGraph(7)
sage: g.distance('0100110','1011010')
5
```

```
>>> from sage.all import *
>>> g = graphs.CubeGraph(Integer(7))
>>> g.distance('0100110','1011010')
5
```

Plot several n-cubes in a Sage Graphics Array:

```
sage: g = []
sage: j = []
sage: for i in range(6):
....:  k = graphs.CubeGraph(i+1)
....:  g.append(k)
...
sage: for i in range(2):
....:  n = []
....:  for m in range(3):
....:      n.append(g[3*i + m].plot(vertex_size=50, vertex_labels=False))
....:  j.append(n)
...
sage: G = graphics_array(j)
sage: G.show(figsize=[6,4])              # long time
```

```
>>> from sage.all import *
>>> g = []
>>> j = []
>>> for i in range(Integer(6)):
...  k = graphs.CubeGraph(i+Integer(1))
...  g.append(k)
...
>>> for i in range(Integer(2)):
...  n = []
...  for m in range(Integer(3)):
...      n.append(g[Integer(3)*i + m].plot(vertex_size=Integer(50), vertex_
↪labels=False))
...  j.append(n)
...
>>> G = graphics_array(j)
>>> G.show(figsize=[Integer(6),Integer(4)])             # long time
```

Use the plot options to display larger n-cubes:

```
sage: g = graphs.CubeGraph(9, embedding=1)
sage: g.show(figsize=[12,12],vertex_labels=False, vertex_size=20)       #␣
↪long time, needs sage.plot
```

(continues on next page)

(continued from previous page)

```
sage: g = graphs.CubeGraph(9, embedding=2)
sage: g.show(figsize=[12,12],vertex_labels=False, vertex_size=20)       #␣
↪long time, needs sage.plot
sage: g = graphs.CubeGraph(9, embedding=3)
sage: g.show(figsize=[12,12],vertex_labels=False, vertex_size=20)       #␣
↪long time, needs sage.plot
```

```
>>> from sage.all import *
>>> g = graphs.CubeGraph(Integer(9), embedding=Integer(1))
>>> g.show(figsize=[Integer(12),Integer(12)],vertex_labels=False, vertex_
↪size=Integer(20))       # long time, needs sage.plot
>>> g = graphs.CubeGraph(Integer(9), embedding=Integer(2))
>>> g.show(figsize=[Integer(12),Integer(12)],vertex_labels=False, vertex_
↪size=Integer(20))       # long time, needs sage.plot
>>> g = graphs.CubeGraph(Integer(9), embedding=Integer(3))
>>> g.show(figsize=[Integer(12),Integer(12)],vertex_labels=False, vertex_
↪size=Integer(20))       # long time, needs sage.plot
```

AUTHORS:

- Robert Miller
- David Coudert

static CubeplexGraph(*embedding='LM'*, *immutable=False*)

Return the Cubeplex graph.

The Cubeplex graph is the cubic hamiltonian graph of order 12 that corresponds to the graph labeled as Γ_1 in Fischer and Little [FiLi2001]. It has LCF notation $[-6, -5, -3, -6, 3, 5, -6, -3, 5, -6, -5, 3]$.

The Fischer-Little Theorem [FiLi2001] may be stated as follows [LM2024]:

A near-bipartite graph is non-Pfaffian if and only if it contains one of the graphs $K_{3,3}$, Γ_1 and Γ_2 as an S-minor.

Norine and Thomas [NT2007] use the term `Cubeplex` to describe one of the 12-vertex cubic graphs, Γ_1 and Γ_2, as defined by Fischer and Little [FiLi2001]. However, the figure in their paper that supposedly provides embeddings for the graphs labeled Cubeplex and Twinplex actually shows both embeddings corresponding to Fischer and Little's Γ_1, which is the Cubeplex graph. Followingly, for `embedding='NT'`, we present only the embedding that is shown by the labeling `Cubeplex` in the paper of Norine and Thomas [NT2007].

PLOTTING:

Upon construction, the position dictionary is filled to override the spring-layout algorithm. For different values of the parameter `embedding`, the Cubeplex graph is displayed as it is mentioned in the respective paper/ book.

INPUT:

- `embedding` – string (default: `'LM'`)
  - `'LM'` displays the embedding as shown for Γ_1 by Lucchesi and Murty [LM2024]
  - `'FL'` displays the embedding as shown for Γ_1 by Fischer and Little [FiLi2001]
  - `'NT'` displays the embedding as shown for the `Cubeplex` by Norine and Thomas [NT2007]
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

OUTPUT:

- `G` – the Cubeplex graph; note that a `ValueError` is returned if `embedding` is none of `'FT'`, `'NT'` or `'LM'`

EXAMPLES:

Construct and show the Cubeplex graph:

```
sage: g = graphs.CubeplexGraph()
sage: g.name()
'Cubeplex Graph'
sage: g.order()
12
sage: g.size()
18
sage: g.girth()
4
sage: g.diameter()
3
sage: g.is_hamiltonian()
True
sage: g.crossing_number()
1
sage: g.show()                              # long time                    ␣
↪  # needs sage.plot
```

```
>>> from sage.all import *
>>> g = graphs.CubeplexGraph()
>>> g.name()
'Cubeplex Graph'
>>> g.order()
12
>>> g.size()
18
>>> g.girth()
4
>>> g.diameter()
3
>>> g.is_hamiltonian()
True
>>> g.crossing_number()
1
>>> g.show()                              # long time
↪# needs sage.plot
```

See also

TwinplexGraph()

AUTHORS:

- Janmenjaya Panda (2024-08-03)

static CycleGraph(*n*, *immutable=False*)

Return a cycle graph with n nodes.

A cycle graph is a basic structure which is also typically called an n-gon.

INPUT:

- `n` – nonnegative integer; the number of vertices of the cycle graph
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

PLOTTING: Upon construction, the position dictionary is filled to override the spring-layout algorithm. By convention, each cycle graph will be displayed with the first (0) node at the top, with the rest following in a counterclockwise manner.

The cycle graph is a good opportunity to compare efficiency of filling a position dictionary vs. using the spring-layout algorithm for plotting. Because the cycle graph is very symmetric, the resulting plots should be similar (in cases of small n).

Filling the position dictionary in advance adds $O(n)$ to the constructor.

EXAMPLES:

Compare plotting using the predefined layout and networkx:

```
sage: # needs networkx sage.plot
sage: import networkx
sage: n = networkx.cycle_graph(23)
sage: spring23 = Graph(n)
sage: posdict23 = graphs.CycleGraph(23)
sage: spring23.show()                    # long time
sage: posdict23.show()                   # long time
```

```
>>> from sage.all import *
>>> # needs networkx sage.plot
>>> import networkx
>>> n = networkx.cycle_graph(Integer(23))
>>> spring23 = Graph(n)
>>> posdict23 = graphs.CycleGraph(Integer(23))
>>> spring23.show()                    # long time
>>> posdict23.show()                   # long time
```

We next view many cycle graphs as a Sage graphics array. First we use the `CycleGraph` constructor, which fills in the position dictionary:

```
sage: # needs networkx sage.plot
sage: g = []
sage: j = []
sage: for i in range(9):
....:     k = graphs.CycleGraph(i+3)
....:     g.append(k)
sage: for i in range(3):
....:     n = []
....:     for m in range(3):
....:         n.append(g[3*i + m].plot(vertex_size=50, vertex_labels=False))
....:     j.append(n)
sage: G = graphics_array(j)
sage: G.show()                           # long time
```

```
>>> from sage.all import *
>>> # needs networkx sage.plot
>>> g = []
>>> j = []
>>> for i in range(Integer(9)):
...     k = graphs.CycleGraph(i+Integer(3))
...     g.append(k)
>>> for i in range(Integer(3)):
...     n = []
...     for m in range(Integer(3)):
...         n.append(g[Integer(3)*i + m].plot(vertex_size=Integer(50), vertex_
↪labels=False))
...     j.append(n)
>>> G = graphics_array(j)
>>> G.show()                            # long time
```

Compare to plotting with the spring-layout algorithm:

```
sage: # needs networkx sage.plot
sage: g = []
sage: j = []
sage: for i in range(9):
....:     spr = networkx.cycle_graph(i+3)
....:     k = Graph(spr)
....:     g.append(k)
sage: for i in range(3):
....:     n = []
....:     for m in range(3):
....:         n.append(g[3*i + m].plot(vertex_size=50, vertex_labels=False))
....:     j.append(n)
sage: G = graphics_array(j)
sage: G.show()                             # long time
```

```
>>> from sage.all import *
>>> # needs networkx sage.plot
>>> g = []
>>> j = []
>>> for i in range(Integer(9)):
...     spr = networkx.cycle_graph(i+Integer(3))
...     k = Graph(spr)
...     g.append(k)
>>> for i in range(Integer(3)):
...     n = []
...     for m in range(Integer(3)):
...         n.append(g[Integer(3)*i + m].plot(vertex_size=Integer(50), vertex_
↪labels=False))
...     j.append(n)
>>> G = graphics_array(j)
>>> G.show()                            # long time
```

static DartGraph(*immutable=False*)

Return a dart graph with 5 nodes.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

PLOTTING: Upon construction, the position dictionary is filled to override the spring-layout algorithm. By convention, the dart graph is drawn as a dart, with the sharp part on the bottom.

EXAMPLES:

Construct and show a dart graph:

```
sage: g = graphs.DartGraph()
sage: g.show()                              # long time
↪  # needs sage.plot
```

```
>>> from sage.all import *
>>> g = graphs.DartGraph()
>>> g.show()                              # long time
↪# needs sage.plot
```

static DegreeSequence(*deg_sequence*, *immutable=False*)

Return a graph with the given degree sequence.

This method raises a NetworkX error if the proposed degree sequence cannot be that of a graph.

Graph returned is the one returned by the Havel-Hakimi algorithm, which constructs a simple graph by connecting vertices of highest degree to other vertices of highest degree, resorting the remaining vertices by degree and repeating the process. See Theorem 1.4 in [CL1996].

INPUT:

- `deg_sequence` – list of integers with each entry corresponding to the degree of a different vertex
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: G = graphs.DegreeSequence([3,3,3,3])
↪  # needs networkx
sage: G.edges(sort=True, labels=False)
↪  # needs networkx
[(0, 1), (0, 2), (0, 3), (1, 2), (1, 3), (2, 3)]
sage: G.show()                              # long time
↪  # needs networkx sage.plot
```

```
>>> from sage.all import *
>>> G = graphs.DegreeSequence([Integer(3),Integer(3),Integer(3),Integer(3)])
↪                                     # needs networkx
>>> G.edges(sort=True, labels=False)
↪# needs networkx
[(0, 1), (0, 2), (0, 3), (1, 2), (1, 3), (2, 3)]
>>> G.show()                              # long time
↪# needs networkx sage.plot
```

```
sage: G = graphs.DegreeSequence([3,3,3,3,3,3,3,3,3,3,3,3,3,3,3,3,3,3,3,3,3,
↪3])  # needs networkx
sage: G.show()                              # long time
↪  # needs networkx sage.plot
```

```
>>> from sage.all import *
>>> G = graphs.DegreeSequence([Integer(3),Integer(3),Integer(3),Integer(3),
↪Integer(3),Integer(3),Integer(3),Integer(3),Integer(3),Integer(3),
↪Integer(3),Integer(3),Integer(3),Integer(3),Integer(3),Integer(3),
↪Integer(3),Integer(3),Integer(3),Integer(3),Integer(3),Integer(3)])   #␣
↪needs networkx
>>> G.show()                              # long time
↪# needs networkx sage.plot
```

```
sage: G = graphs.DegreeSequence([4,4,4,4,4,4,4,4])                    ␣
↪  # needs networkx
sage: G.show()                               # long time               ␣
↪  # needs networkx sage.plot
```

```
>>> from sage.all import *
>>> G = graphs.DegreeSequence([Integer(4),Integer(4),Integer(4),Integer(4),
↪Integer(4),Integer(4),Integer(4),Integer(4)])
↪# needs networkx
>>> G.show()                              # long time
↪# needs networkx sage.plot
```

```
sage: G = graphs.DegreeSequence([1,2,3,4,3,4,3,2,3,2,1])              ␣
↪  # needs networkx
sage: G.show()                               # long time               ␣
↪  # needs networkx sage.plot
```

```
>>> from sage.all import *
>>> G = graphs.DegreeSequence([Integer(1),Integer(2),Integer(3),Integer(4),
↪Integer(3),Integer(4),Integer(3),Integer(2),Integer(3),Integer(2),
↪Integer(1)])                          # needs networkx
>>> G.show()                              # long time
↪# needs networkx sage.plot
```

static DegreeSequenceBipartite(*s1*, *s2*, *immutable=False*)

Return a bipartite graph whose two sets have the given degree sequences.

Given two different sequences of degrees s_1 and s_2, this functions returns (if possible) a bipartite graph on sets A and B such that the vertices in A have s_1 as their degree sequence, while s_2 is the degree sequence of the vertices in B.

INPUT:

- `s1` – list of integers corresponding to the degree sequence of the first set of vertices
- `s2` – list of integers corresponding to the degree sequence of the second set of vertices
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

ALGORITHM:

This function works through the computation of the matrix given by the Gale-Ryser theorem, which is in this case the adjacency matrix of the bipartite graph.

EXAMPLES:

If we are given as sequences `[2,2,2,2,2]` and `[5,5]` we are given as expected the complete bipartite graph $K_{2,5}$:

```
sage: g = graphs.DegreeSequenceBipartite([2,2,2,2,2],[5,5])
↪  # needs sage.combinat sage.modules
sage: g.is_isomorphic(graphs.CompleteBipartiteGraph(5,2))
↪  # needs sage.combinat sage.modules
True
```

```
>>> from sage.all import *
>>> g = graphs.DegreeSequenceBipartite([Integer(2),Integer(2),Integer(2),
↪Integer(2),Integer(2)],[Integer(5),Integer(5)])                    # needs
↪sage.combinat sage.modules
>>> g.is_isomorphic(graphs.CompleteBipartiteGraph(Integer(5),Integer(2)))
↪                  # needs sage.combinat sage.modules
True
```

Some sequences being incompatible if, for example, their sums are different, the functions raises a `ValueError` when no graph corresponding to the degree sequences exists:

```
sage: g = graphs.DegreeSequenceBipartite([2,2,2,2,1],[5,5])
↪  # needs sage.combinat sage.modules
Traceback (most recent call last):
...
ValueError: there exists no bipartite graph corresponding to the given degree
↪sequences
```

```
>>> from sage.all import *
>>> g = graphs.DegreeSequenceBipartite([Integer(2),Integer(2),Integer(2),
↪Integer(2),Integer(1)],[Integer(5),Integer(5)])                    # needs
↪sage.combinat sage.modules
Traceback (most recent call last):
...
ValueError: there exists no bipartite graph corresponding to the given degree
↪sequences
```

static DegreeSequenceConfigurationModel(*deg_sequence*, *seed=None*, *immutable=False*)

Return a random pseudograph with the given degree sequence.

This method raises a NetworkX error if the proposed degree sequence cannot be that of a graph with multiple edges and loops.

One requirement is that the sum of the degrees must be even, since every edge must be incident with two vertices.

INPUT:

- `deg_sequence` – list of integers with each entry corresponding to the expected degree of a different vertex
- `seed` – (optional) a `random.Random` seed or a Python `int` for the random number generator
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: G = graphs.DegreeSequenceConfigurationModel([1,1])
↪  # needs networkx
sage: G.adjacency_matrix()
```

(continues on next page)

(continued from previous page)

```
↪  # needs networkx sage.modules
[0 1]
[1 0]
```

```
>>> from sage.all import *
>>> G = graphs.DegreeSequenceConfigurationModel([Integer(1),Integer(1)])
↪                  # needs networkx
>>> G.adjacency_matrix()
↪# needs networkx sage.modules
[0 1]
[1 0]
```

The output is allowed to contain both loops and multiple edges:

```
sage: # needs networkx
sage: deg_sequence = [3,3,3,3,3,3,3,3,3,3,3,3,3,3,3,3,3,3,3,3]
sage: G = graphs.DegreeSequenceConfigurationModel(deg_sequence)
sage: G.order(), G.size()
(20, 30)
sage: G.has_loops() or G.has_multiple_edges()  # random
True
sage: G.show()                              # long time
↪  # needs sage.plot
```

```
>>> from sage.all import *
>>> # needs networkx
>>> deg_sequence = [Integer(3),Integer(3),Integer(3),Integer(3),Integer(3),
↪Integer(3),Integer(3),Integer(3),Integer(3),Integer(3),Integer(3),
↪Integer(3),Integer(3),Integer(3),Integer(3),Integer(3),Integer(3),
↪Integer(3),Integer(3),Integer(3)]
>>> G = graphs.DegreeSequenceConfigurationModel(deg_sequence)
>>> G.order(), G.size()
(20, 30)
>>> G.has_loops() or G.has_multiple_edges()  # random
True
>>> G.show()                              # long time
↪# needs sage.plot
```

REFERENCE:

[New2003]

static DegreeSequenceExpected(*deg_sequence*, *seed=None*, *immutable=False*)

Return a random graph with expected given degree sequence.

This method raises a NetworkX error if the proposed degree sequence cannot be that of a graph.

One requirement is that the sum of the degrees must be even, since every edge must be incident with two vertices.

INPUT:

- `deg_sequence` – list of integers with each entry corresponding to the expected degree of a different vertex
- `seed` – (optional) a `random.Random` seed or a Python `int` for the random number generator

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: G = graphs.DegreeSequenceExpected([1,2,3,2,3]); G
↪  # needs networkx
Looped graph on 5 vertices
sage: G.show()                          # long time
↪  # needs networkx sage.plot
```

```
>>> from sage.all import *
>>> G = graphs.DegreeSequenceExpected([Integer(1),Integer(2),Integer(3),
↪Integer(2),Integer(3)]); G                          # needs networkx
Looped graph on 5 vertices
>>> G.show()                          # long time
↪# needs networkx sage.plot
```

REFERENCE:

[CL2002]

static DegreeSequenceTree(*deg_sequence*, *immutable=False*)

Return a tree with the given degree sequence.

This method raises a NetworkX error if the proposed degree sequence cannot be that of a tree.

Since every tree has one more vertex than edge, the degree sequence must satisfy `len(deg_sequence) - sum(deg_sequence)/2 == 1`.

INPUT:

- `deg_sequence` – list of integers with each entry corresponding to the expected degree of a different vertex
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: G = graphs.DegreeSequenceTree([3,1,3,3,1,1,1,2,1]); G
↪  # needs networkx
Graph on 9 vertices
sage: G.show()                          # long time
↪  # needs networkx sage.plot
```

```
>>> from sage.all import *
>>> G = graphs.DegreeSequenceTree([Integer(3),Integer(1),Integer(3),
↪Integer(3),Integer(1),Integer(1),Integer(1),Integer(2),Integer(1)]); G
↪               # needs networkx
Graph on 9 vertices
>>> G.show()                          # long time
↪# needs networkx sage.plot
```

static DejterGraph(*immutable=False*)

Return the Dejter graph.

The Dejter graph is obtained from the binary 7-cube by deleting a copy of the Hamming code of length 7. It is 6-regular, with 112 vertices and 336 edges. For more information, see the Wikipedia article Dejter_graph.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.DejterGraph(); g
↪  # needs sage.rings.finite_rings
Dejter Graph: Graph on 112 vertices
sage: g.is_regular(k=6)
↪  # needs sage.rings.finite_rings
True
sage: g.girth()
↪  # needs sage.rings.finite_rings
4
```

```
>>> from sage.all import *
>>> g = graphs.DejterGraph(); g
↪# needs sage.rings.finite_rings
Dejter Graph: Graph on 112 vertices
>>> g.is_regular(k=Integer(6))
↪          # needs sage.rings.finite_rings
True
>>> g.girth()
↪# needs sage.rings.finite_rings
4
```

static DesarguesGraph(*immutable=False*)

Return the Desargues graph.

PLOTTING: The layout chosen is the same as on the cover of [Har1969].

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: D = graphs.DesarguesGraph()
sage: L = graphs.LCFGraph(20,[5,-5,9,-9],5)
sage: D.is_isomorphic(L)
True
sage: D.show()                              # long time
↪  # needs sage.plot
```

```
>>> from sage.all import *
>>> D = graphs.DesarguesGraph()
>>> L = graphs.LCFGraph(Integer(20),[Integer(5),-Integer(5),Integer(9),-
↪Integer(9)],Integer(5))
>>> D.is_isomorphic(L)
True
>>> D.show()                              # long time
↪# needs sage.plot
```

static DiamondGraph(*immutable=False*)

Return a diamond graph with 4 nodes.

A diamond graph is a square with one pair of diagonal nodes connected.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

PLOTTING: Upon construction, the position dictionary is filled to override the spring-layout algorithm. By convention, the diamond graph is drawn as a diamond, with the first node on top, second on the left, third on the right, and fourth on the bottom; with the second and third node connected.

EXAMPLES:

Construct and show a diamond graph:

```
sage: g = graphs.DiamondGraph()
sage: g.show()                              # long time                    ␣
↪  # needs sage.plot
```

```
>>> from sage.all import *
>>> g = graphs.DiamondGraph()
>>> g.show()                              # long time
↪# needs sage.plot
```

static DipoleGraph(*n*, *immutable=False*)

Return a dipole graph with n edges.

A dipole graph is a multigraph consisting of 2 vertices connected with n parallel edges.

INPUT:

- `n` – integer ≥ 0; the number of parallel edges connecting the 2 vertices
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

Construct and show a dipole graph with 13 edges:

```
sage: g = graphs.DipoleGraph(13); g
Dipole graph: Multi-graph on 2 vertices
sage: g.show()                              # long time                    ␣
↪  # needs sage.plot
```

```
>>> from sage.all import *
>>> g = graphs.DipoleGraph(Integer(13)); g
Dipole graph: Multi-graph on 2 vertices
>>> g.show()                              # long time
↪# needs sage.plot
```

static Dkq(*k*, *q*, *A=None*, *B=None*, *immutable=False*)

Return the graph $D(k, q)$ described in [LW2026].

The point set and line set are $\mathbf{F}_q^k$. A point $(p_1, \ldots, p_k)$ is adjacent to a line $(l_1, \ldots, l_k)$ if

$$p_2 + l_2 = p_1 l_1,$$

and, when $k \geq 3$,

$$p_3 + l_3 = p_1 l_2.$$

For $4 \leq i \leq k$, the remaining equations are

$$p_i + l_i = \begin{cases} p_{i-2}l_1, & i \equiv 0, 1 \pmod 4, \\ p_1 l_{i-2}, & i \equiv 2, 3 \pmod 4. \end{cases}$$

INPUT:

- `k` – integer at least 2
- `q` – prime power
- `A` – iterable, single value, or `None` (default: `None`); allowed first coordinates on the point side
- `B` – iterable, single value, or `None` (default: `None`); allowed first coordinates on the line side
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

OUTPUT:

A Sage *Graph*.

EXAMPLES:

```
sage: G = graphs.Dkq(5, 3)
sage: G.order(), G.size(), G.girth()
(486, 729, 12)
```

```
>>> from sage.all import *
>>> G = graphs.Dkq(Integer(5), Integer(3))
>>> G.order(), G.size(), G.girth()
(486, 729, 12)
```

REFERENCES:

- [LW2026]

static DodecahedralGraph(*immutable=False*)

Return a Dodecahedral graph (with 20 nodes).

The dodecahedral graph is cubic symmetric, so the spring-layout algorithm will be very effective for display. It is dual to the icosahedral graph.

PLOTTING: The Dodecahedral graph should be viewed in 3 dimensions. We choose to use a planar embedding of the graph. We hope to add rotatable, 3-dimensional viewing in the future. In such a case, an argument will be added to select the desired layout.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

Construct and show a Dodecahedral graph:

```
sage: g = graphs.DodecahedralGraph()
sage: g.show()                              # long time
↪  # needs sage.plot
```

```
>>> from sage.all import *
>>> g = graphs.DodecahedralGraph()
>>> g.show()                              # long time
↪# needs sage.plot
```

Create several dodecahedral graphs in a Sage graphics array They will be drawn differently due to the use of the spring-layout algorithm:

```
sage: g = []
sage: j = []
sage: for i in range(9):
....:     k = graphs.DodecahedralGraph()
....:     g.append(k)
sage: for i in range(3):
....:     n = []
....:     for m in range(3):
....:         n.append(g[3*i + m].plot(vertex_size=50, vertex_labels=False))
....:     j.append(n)
sage: G = graphics_array(j)
sage: G.show()                          # long time
```

```
>>> from sage.all import *
>>> g = []
>>> j = []
>>> for i in range(Integer(9)):
...     k = graphs.DodecahedralGraph()
...     g.append(k)
>>> for i in range(Integer(3)):
...     n = []
...     for m in range(Integer(3)):
...         n.append(g[Integer(3)*i + m].plot(vertex_size=Integer(50), vertex_
↪labels=False))
...     j.append(n)
>>> G = graphics_array(j)
>>> G.show()                          # long time
```

static DorogovtsevGoltsevMendesGraph(*n*, *immutable=False*)

Construct the n-th generation of the Dorogovtsev-Goltsev-Mendes graph.

See [DGM2002] for more details.

INPUT:

- `n` – nonnegative integer; index of the generation
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: G = graphs.DorogovtsevGoltsevMendesGraph(8)                              ␣
↪  # needs networkx
sage: G.size()                                                                 ␣
↪  # needs networkx
6561
```

```
>>> from sage.all import *
>>> G = graphs.DorogovtsevGoltsevMendesGraph(Integer(8))                       ␣
↪          # needs networkx
>>> G.size()
↪# needs networkx
6561
```

static DoubleGeneralizedPetersenGraph(*n*, *k*, *immutable=False*)

Return a double generalized Petersen graph with $4n$ nodes.

The double generalized Petersen graphs is a family of graphs proposed in [ZF2012] as a variant of generalized Petersen graphs. The variables n, k are integers such that $n > 2$ and $0 < k \leq \lfloor (n-1)/2 \rfloor$.

INPUT:

- `n` – the number of nodes is $4 * n$
- `k` – integer such that $0 < k \leq \lfloor (n-1)/2 \rfloor$ determining how vertices on second and third inner rims are connected
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

PLOTTING: Upon construction, the position dictionary is filled to override the spring-layout algorithm. By convention, the double generalized Petersen graphs are displayed as 4 concentric cycles, with the first n nodes drawn on the outer circle. The first (0) node is drawn at the top of the outer-circle, moving counterclockwise after that. The second circle is drawn with the (n)th node at the top, then counterclockwise as well. The tird cycle is drawn with the (2n)th node at the top, then counterclockwise. And the fourth cycle is drawn with the (3n)th node at the top, then again counterclockwise.

EXAMPLES:

When n is even the resulting graph will be isomorphic to a double generalized Petersen graph with $k' = n/2 - k$:

```
sage: g = graphs.DoubleGeneralizedPetersenGraph(10, 2)
sage: g2 = graphs.DoubleGeneralizedPetersenGraph(10, 3)
sage: g.is_isomorphic(g2)
True
```

```
>>> from sage.all import *
>>> g = graphs.DoubleGeneralizedPetersenGraph(Integer(10), Integer(2))
>>> g2 = graphs.DoubleGeneralizedPetersenGraph(Integer(10), Integer(3))
>>> g.is_isomorphic(g2)
True
```

static DoubleGrassmannGraph(*q*, *e*, *immutable=False*)

Return the bipartite double of the distance-e graph of the Grassmann graph $J_q(n, e)$.

This graph can also be described as follows: Let V be the vector space of dimension n over $GF(q)$. The vertex set is the set of $e + 1$ or e subspaces of V. Two vertices are adjacent if one subspace is contained in the other.

This graph is distance-transitive.

INPUT:

- `q` – a prime power
- `e` – integer
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: G = graphs.DoubleGrassmannGraph(2,1)
↪  # needs sage.modules
sage: G.diameter()
```

(continues on next page)

(continued from previous page)

```
↪  # needs sage.modules
3
sage: G.is_distance_regular(True)                                              ␣
↪  # needs sage.modules
([3, 2, 2, None], [None, 1, 1, 3])
```

```
>>> from sage.all import *
>>> G = graphs.DoubleGrassmannGraph(Integer(2),Integer(1))                     ␣
↪                  # needs sage.modules
>>> G.diameter()
↪# needs sage.modules
3
>>> G.is_distance_regular(True)
↪# needs sage.modules
([3, 2, 2, None], [None, 1, 1, 3])
```

REFERENCES:

See [BCN1989] pp. 272, 273 or [VDKT2016] p. 25.

static DoubleOddGraph(*n*, *immutable=False*)

Return the double odd graph on $2n + 1$ points.

The graph is obtained using the subsets of size n and $n + 1$ of $1, 2, ..., 2n + 1$ as vertices. Two vertices are adjacent if one is included in the other.

The graph is distance-transitive.

INPUT:

- `n` – integer; must be greater than 0
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: G = graphs.DoubleOddGraph(5)
sage: G.is_distance_regular(True)
([6, 5, 5, 4, 4, 3, 3, 2, 2, 1, 1, None],
 [None, 1, 1, 2, 2, 3, 3, 4, 4, 5, 5, 6])
sage: G = graphs.DoubleOddGraph(3)
sage: G.diameter()
7
sage: G.is_distance_regular(True)
([4, 3, 3, 2, 2, 1, 1, None], [None, 1, 1, 2, 2, 3, 3, 4])
```

```
>>> from sage.all import *
>>> G = graphs.DoubleOddGraph(Integer(5))
>>> G.is_distance_regular(True)
([6, 5, 5, 4, 4, 3, 3, 2, 2, 1, 1, None],
 [None, 1, 1, 2, 2, 3, 3, 4, 4, 5, 5, 6])
>>> G = graphs.DoubleOddGraph(Integer(3))
>>> G.diameter()
7
>>> G.is_distance_regular(True)
([4, 3, 3, 2, 2, 1, 1, None], [None, 1, 1, 2, 2, 3, 3, 4])
```

REFERENCES:

See [BCN1989] pp. 259-261 or [VDKT2016] p. 25.

static DoubleStarSnark(*immutable=False*)

Return the double star snark.

The double star snark is a 3-regular graph on 30 vertices. See the Wikipedia article Double-star_snark.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.DoubleStarSnark()
sage: g.order()
30
sage: g.size()
45
sage: g.chromatic_number()
3
sage: g.is_hamiltonian()                                                    ␣
↪  # needs sage.numerical.mip
False
sage: g.automorphism_group().cardinality()                                  ␣
↪  # needs sage.groups
80
sage: g.show()                                                              ␣
↪  # needs sage.plot
```

```
>>> from sage.all import *
>>> g = graphs.DoubleStarSnark()
>>> g.order()
30
>>> g.size()
45
>>> g.chromatic_number()
3
>>> g.is_hamiltonian()
↪# needs sage.numerical.mip
False
>>> g.automorphism_group().cardinality()
↪# needs sage.groups
80
>>> g.show()
↪# needs sage.plot
```

static DoublyTruncatedWittGraph(*immutable=False*)

Return the doubly truncated Witt graph.

This builds the truncated Witt graph, then removes all vertices whose codeword start with a 1.

The graph is distance-regular with intersection array $[7, 6, 4, 4; 1, 1, 1, 6]$.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: G = graphs.DoublyTruncatedWittGraph()
↪  # needs sage.libs.pari sage.modules
sage: G.is_distance_regular(True)
↪  # needs sage.libs.pari sage.modules
([7, 6, 4, 4, None], [None, 1, 1, 1, 6])
```

```
>>> from sage.all import *
>>> G = graphs.DoublyTruncatedWittGraph()
↪# needs sage.libs.pari sage.modules
>>> G.is_distance_regular(True)
↪# needs sage.libs.pari sage.modules
([7, 6, 4, 4, None], [None, 1, 1, 1, 6])
```

REFERENCES:

A description and construction of this graph can be found in [BCN1989] p. 368.

static DurerGraph(*immutable=False*)

Return the Dürer graph.

For more information, see the Wikipedia article D%C3%BCrer_graph.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

The Dürer graph is named after Albrecht Dürer. It is a planar graph with 12 vertices and 18 edges:

```
sage: G = graphs.DurerGraph(); G
Durer graph: Graph on 12 vertices
sage: G.is_planar()
True
sage: G.order()
12
sage: G.size()
18
```

```
>>> from sage.all import *
>>> G = graphs.DurerGraph(); G
Durer graph: Graph on 12 vertices
>>> G.is_planar()
True
>>> G.order()
12
>>> G.size()
18
```

The Dürer graph has chromatic number 3, diameter 4, and girth 3:

```
sage: G.chromatic_number()
3
sage: G.diameter()
```

(continues on next page)

(continued from previous page)

```
4
sage: G.girth()
3
```

```
>>> from sage.all import *
>>> G.chromatic_number()
3
>>> G.diameter()
4
>>> G.girth()
3
```

Its automorphism group is isomorphic to D_6:

```
sage: ag = G.automorphism_group()
↪  # needs sage.groups
sage: ag.is_isomorphic(DihedralGroup(6))
↪  # needs sage.groups
True
```

```
>>> from sage.all import *
>>> ag = G.automorphism_group()
↪# needs sage.groups
>>> ag.is_isomorphic(DihedralGroup(Integer(6)))
↪          # needs sage.groups
True
```

static DyckGraph(*immutable=False*)

Return the Dyck graph.

For more information, see the MathWorld article on the Dyck graph or the Wikipedia article Dyck_graph.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

The Dyck graph was defined by Walther von Dyck in 1881. It has 32 vertices and 48 edges, and is a cubic graph (regular of degree 3):

```
sage: G = graphs.DyckGraph(); G
Dyck graph: Graph on 32 vertices
sage: G.order()
32
sage: G.size()
48
sage: G.is_regular()
True
sage: G.is_regular(3)
True
```

```
>>> from sage.all import *
>>> G = graphs.DyckGraph(); G
```

(continues on next page)

(continued from previous page)

```
Dyck graph: Graph on 32 vertices
>>> G.order()
32
>>> G.size()
48
>>> G.is_regular()
True
>>> G.is_regular(Integer(3))
True
```

It is non-planar and Hamiltonian, as well as bipartite (making it a bicubic graph):

```
sage: G.is_planar()
False
sage: G.is_hamiltonian()
↪  # needs sage.numerical.mip
True
sage: G.is_bipartite()
True
```

```
>>> from sage.all import *
>>> G.is_planar()
False
>>> G.is_hamiltonian()
↪# needs sage.numerical.mip
True
>>> G.is_bipartite()
True
```

It has radius 5, diameter 5, and girth 6:

```
sage: G.radius()
5
sage: G.diameter()
5
sage: G.girth()
6
```

```
>>> from sage.all import *
>>> G.radius()
5
>>> G.diameter()
5
>>> G.girth()
6
```

Its chromatic number is 2 and its automorphism group is of order 192:

```
sage: G.chromatic_number()
2
sage: G.automorphism_group().cardinality()
↪  # needs sage.groups
```

(continues on next page)

(continued from previous page)

```
192
```

```
>>> from sage.all import *
>>> G.chromatic_number()
2
>>> G.automorphism_group().cardinality()
↪# needs sage.groups
192
```

It is a non-integral graph as it has irrational eigenvalues:

```
sage: G.characteristic_polynomial().factor()                                ␣
↪  # needs sage.libs.pari sage.modules
(x - 3) * (x + 3) * (x - 1)^9 * (x + 1)^9 * (x^2 - 5)^6
```

```
>>> from sage.all import *
>>> G.characteristic_polynomial().factor()
↪# needs sage.libs.pari sage.modules
(x - 3) * (x + 3) * (x - 1)^9 * (x + 1)^9 * (x^2 - 5)^6
```

It is a toroidal graph, and its embedding on a torus is dual to an embedding of the Shrikhande graph (*ShrikhandeGraph*).

static EgawaGraph(*p*, *s*, *immutable=False*)

Return the Egawa graph with parameters p, s.

Egawa graphs are a peculiar family of graphs devised by Yoshimi Egawa in [Ega1981] . The Shrikhande graph is a special case of this family of graphs, with parameters $(1, 0)$. All the graphs in this family are not recognizable by 1-WL (Weisfeiler Lehamn algorithm of the first order) and 2-WL, that is their orbits are not correctly returned by k-WL for k lower than 3.

Furthermore, all the graphs in this family are distance-regular, but they are not distance-transitive if $p \neq 0$.

The Egawa graph with parameters $(0, s)$ is isomorphic to the Hamming graph with parameters $(s, 4)$, when the underlying set of the Hamming graph is $[0, 1, 2, 3]$

INPUT:

- `p` – power to which the graph named Y in the reference provided above will be raised
- `s` – power to which the graph named X in the reference provided above will be raised
- `immutable` – boolean (default: `False`); whether to return immutable or mutable graphs

OUTPUT:

- `G` – the Egawa graph with parameters (p,s)

EXAMPLES:

Every Egawa graph is distance regular.

```
sage: g = graphs.EgawaGraph(1, 2)
sage: g.is_distance_regular()
True
```

```
>>> from sage.all import *
>>> g = graphs.EgawaGraph(Integer(1), Integer(2))
>>> g.is_distance_regular()
True
```

An Egawa graph with parameters (0,s) is isomorphic to the Hamming graph with parameters (s, 4).

```
sage: g = graphs.EgawaGraph(0, 4)
sage: g.is_isomorphic(graphs.HammingGraph(4,4))
True
```

```
>>> from sage.all import *
>>> g = graphs.EgawaGraph(Integer(0), Integer(4))
>>> g.is_isomorphic(graphs.HammingGraph(Integer(4),Integer(4)))
True
```

static EllinghamHorton54Graph(*immutable=False*)

Return the Ellingham-Horton 54-graph.

For more information, see the Wikipedia article Ellingham-Horton_graph.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

This graph is 3-regular:

```
sage: g = graphs.EllinghamHorton54Graph()
sage: g.is_regular(k=3)
True
```

```
>>> from sage.all import *
>>> g = graphs.EllinghamHorton54Graph()
>>> g.is_regular(k=Integer(3))
True
```

It is 3-connected and bipartite:

```
sage: g.vertex_connectivity()   # not tested - too long
3
sage: g.is_bipartite()
True
```

```
>>> from sage.all import *
>>> g.vertex_connectivity()   # not tested - too long
3
>>> g.is_bipartite()
True
```

It is not Hamiltonian:

```
sage: g.is_hamiltonian()                # not tested                    ␣
↪  # needs sage.numerical.mip
False
```

```
>>> from sage.all import *
>>> g.is_hamiltonian()                # not tested
↪# needs sage.numerical.mip
False
```

… and it has a nice drawing

```
sage: g.show(figsize=[10, 10])  # not tested - too long
```

```
>>> from sage.all import *
>>> g.show(figsize=[Integer(10), Integer(10)])  # not tested - too long
```

static EllinghamHorton78Graph(*immutable=False*)

Return the Ellingham-Horton 78-graph.

For more information, see the Wikipedia article Ellingham%E2%80%93Horton_graph

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

This graph is 3-regular:

```
sage: g = graphs.EllinghamHorton78Graph()
sage: g.is_regular(k=3)
True
```

```
>>> from sage.all import *
>>> g = graphs.EllinghamHorton78Graph()
>>> g.is_regular(k=Integer(3))
True
```

It is 3-connected and bipartite:

```
sage: g.vertex_connectivity()   # not tested (too long)
3
sage: g.is_bipartite()
True
```

```
>>> from sage.all import *
>>> g.vertex_connectivity()   # not tested (too long)
3
>>> g.is_bipartite()
True
```

It is not Hamiltonian:

```
sage: g.is_hamiltonian()                 # not tested
↪  # needs sage.numerical.mip
False
```

```
>>> from sage.all import *
>>> g.is_hamiltonian()                 # not tested
↪# needs sage.numerical.mip
False
```

… and it has a nice drawing

```
sage: g.show(figsize=[10,10])    # not tested (too long)
```

```
>>> from sage.all import *
>>> g.show(figsize=[Integer(10),Integer(10)])    # not tested (too long)
```

static EmptyGraph(*immutable=False*)

Return an empty graph (0 nodes and 0 edges).

This is useful for constructing graphs by adding edges and vertices individually or in a loop.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

PLOTTING: When plotting, this graph will use the default spring-layout algorithm, unless a position dictionary is specified.

EXAMPLES:

Add one vertex to an empty graph and then show:

```
sage: empty1 = graphs.EmptyGraph()
sage: empty1.add_vertex()
0
sage: empty1.show()                      # long time
↪  # needs sage.plot
```

```
>>> from sage.all import *
>>> empty1 = graphs.EmptyGraph()
>>> empty1.add_vertex()
0
>>> empty1.show()                      # long time
↪# needs sage.plot
```

Use for loops to build a graph from an empty graph:

```
sage: empty2 = graphs.EmptyGraph()
sage: for i in range(5):
....:     empty2.add_vertex()  # add 5 nodes, labeled 0-4
0
1
2
3
4
```

(continues on next page)

(continued from previous page)

```
sage: for i in range(3):
....:     empty2.add_edge(i,i+1)  # add edges {[0:1],[1:2],[2:3]}
sage: for i in range(1, 4):
....:     empty2.add_edge(4,i)  # add edges {[1:4],[2:4],[3:4]}
sage: empty2.show()                      # long time
↪  # needs sage.plot
```

```
>>> from sage.all import *
>>> empty2 = graphs.EmptyGraph()
>>> for i in range(Integer(5)):
...     empty2.add_vertex()  # add 5 nodes, labeled 0-4
0
1
2
3
4
>>> for i in range(Integer(3)):
...     empty2.add_edge(i,i+Integer(1))  # add edges {[0:1],[1:2],[2:3]}
>>> for i in range(Integer(1), Integer(4)):
...     empty2.add_edge(Integer(4),i)  # add edges {[1:4],[2:4],[3:4]}
>>> empty2.show()                      # long time
↪# needs sage.plot
```

static ErreraGraph(*immutable=False*)

Return the Errera graph.

For more information, see the Wikipedia article Errera_graph.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

The Errera graph is named after Alfred Errera. It is a planar graph on 17 vertices and having 45 edges:

```
sage: G = graphs.ErreraGraph(); G
Errera graph: Graph on 17 vertices
sage: G.is_planar()
True
sage: G.order()
17
sage: G.size()
45
```

```
>>> from sage.all import *
>>> G = graphs.ErreraGraph(); G
Errera graph: Graph on 17 vertices
>>> G.is_planar()
True
>>> G.order()
17
>>> G.size()
45
```

The Errera graph is Hamiltonian with radius 3, diameter 4, girth 3, and chromatic number 4:

```
sage: G.is_hamiltonian()
↪  # needs sage.numerical.mip
True
sage: G.radius()
3
sage: G.diameter()
4
sage: G.girth()
3
sage: G.chromatic_number()
4
```

```
>>> from sage.all import *
>>> G.is_hamiltonian()
↪# needs sage.numerical.mip
True
>>> G.radius()
3
>>> G.diameter()
4
>>> G.girth()
3
>>> G.chromatic_number()
4
```

Each vertex degree is either 5 or 6. That is, if f counts the number of vertices of degree 5 and s counts the number of vertices of degree 6, then $f + s$ is equal to the order of the Errera graph:

```
sage: D = G.degree_sequence()
sage: D.count(5) + D.count(6) == G.order()
True
```

```
>>> from sage.all import *
>>> D = G.degree_sequence()
>>> D.count(Integer(5)) + D.count(Integer(6)) == G.order()
True
```

The automorphism group of the Errera graph is isomorphic to the dihedral group of order 20:

```
sage: ag = G.automorphism_group()
↪  # needs sage.groups
sage: ag.is_isomorphic(DihedralGroup(10))
↪  # needs sage.groups
True
```

```
>>> from sage.all import *
>>> ag = G.automorphism_group()
↪# needs sage.groups
>>> ag.is_isomorphic(DihedralGroup(Integer(10)))
↪         # needs sage.groups
True
```

static EuropeMap(*continental=False*, *year=2018*, *immutable=False*)

Return European states as a graph of common border.

"European state" here is defined as an independent state having the capital city in Europe. The graph has an edge between those countries that have common *land* border.

INPUT:

- `continental` – boolean (default: `False`); whether to only return states in the continental Europe or all European states
- `year` – integer (default: 2018); reserved for future use
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: Europe = graphs.EuropeMap(); Europe
Europe Map: Graph on 44 vertices
sage: Europe.neighbors('Ireland')
['United Kingdom']

sage: cont_Europe = graphs.EuropeMap(continental=True)
sage: cont_Europe.order()
40
sage: 'Iceland' in cont_Europe
False
sage: cont_Europe.is_connected()
True
sage: cont_Europe.is_immutable()
False
sage: cont_Europe = graphs.EuropeMap(continental=True, immutable=True)
sage: cont_Europe.is_immutable()
True
```

```
>>> from sage.all import *
>>> Europe = graphs.EuropeMap(); Europe
Europe Map: Graph on 44 vertices
>>> Europe.neighbors('Ireland')
['United Kingdom']

>>> cont_Europe = graphs.EuropeMap(continental=True)
>>> cont_Europe.order()
40
>>> 'Iceland' in cont_Europe
False
>>> cont_Europe.is_connected()
True
>>> cont_Europe.is_immutable()
False
>>> cont_Europe = graphs.EuropeMap(continental=True, immutable=True)
>>> cont_Europe.is_immutable()
True
```

static F26AGraph(*immutable=False*)

Return the F26A graph.

The F26A graph is a symmetric bipartite cubic graph with 26 vertices and 39 edges. For more information, see the Wikipedia article F26A_graph.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.F26AGraph(); g
F26A Graph: Graph on 26 vertices
sage: g.order(), g.size()
(26, 39)
sage: g.automorphism_group().cardinality()
78
sage: g.girth()
6
sage: g.is_bipartite()
True
sage: g.characteristic_polynomial().factor()
(x - 3) * (x + 3) * (x^4 - 5*x^2 + 3)^6
```

```
>>> from sage.all import *
>>> g = graphs.F26AGraph(); g
F26A Graph: Graph on 26 vertices
>>> g.order(), g.size()
(26, 39)
>>> g.automorphism_group().cardinality()
78
>>> g.girth()
6
>>> g.is_bipartite()
True
>>> g.characteristic_polynomial().factor()
(x - 3) * (x + 3) * (x^4 - 5*x^2 + 3)^6
```

static FibonacciTree(*n*, *immutable=False*)

Return the graph of the Fibonacci Tree F_n.

The Fibonacci tree F_n is recursively defined as the tree with a root vertex and two attached child trees F_{n-1} and F_{n-2}, where F_1 is just one vertex and F_0 is empty.

INPUT:

- `n` – the recursion depth of the Fibonacci Tree
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.FibonacciTree(3)                                           ␣
↪  # needs sage.libs.pari
sage: g.is_tree()                                                           ␣
↪  # needs sage.libs.pari
True
```

```
>>> from sage.all import *
>>> g = graphs.FibonacciTree(Integer(3))
↪          # needs sage.libs.pari
>>> g.is_tree()
↪# needs sage.libs.pari
True
```

```
sage: l1 = [graphs.FibonacciTree(_).order() + 1 for _ in range(6)]
↪  # needs sage.libs.pari
sage: l2 = list(fibonacci_sequence(2,8))
↪  # needs sage.libs.pari
sage: l1 == l2
↪  # needs sage.libs.pari
True
```

```
>>> from sage.all import *
>>> l1 = [graphs.FibonacciTree(_).order() + Integer(1) for _ in
↪range(Integer(6))]              # needs sage.libs.pari
>>> l2 = list(fibonacci_sequence(Integer(2),Integer(8)))
↪                  # needs sage.libs.pari
>>> l1 == l2
↪# needs sage.libs.pari
True
```

AUTHORS:

- Harald Schilly and Yann Laigle-Chapuy (2010-03-25)

static FlowerSnark(*immutable=False*)

Return a Flower Snark.

A flower snark has 20 vertices. It is part of the class of biconnected cubic graphs with edge chromatic number = 4, known as snarks. (i.e.: the Petersen graph). All snarks are not Hamiltonian, non-planar and have Petersen graph graph minors. See the Wikipedia article Flower_snark.

PLOTTING: Upon construction, the position dictionary is filled to override the spring-layout algorithm. By convention, the nodes are drawn 0-14 on the outer circle, and 15-19 in an inner pentagon.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES: Inspect a flower snark:

```
sage: F = graphs.FlowerSnark()
sage: F
Flower Snark: Graph on 20 vertices
sage: F.graph6_string()
'ShCGHC@?GGg@?@?Gp?K??C?CA?G?_G?Cc'
```

```
>>> from sage.all import *
>>> F = graphs.FlowerSnark()
>>> F
Flower Snark: Graph on 20 vertices
>>> F.graph6_string()
'ShCGHC@?GGg@?@?Gp?K??C?CA?G?_G?Cc'
```

Now show it:

```
sage: F.show()                                # long time                    ␣
↪  # needs sage.plot
```

```
>>> from sage.all import *
>>> F.show()                                # long time
↪# needs sage.plot
```

static FoldedCubeGraph(*n*, *immutable=False*)

Return the folded cube graph of order 2^{n-1}.

The folded cube graph on 2^{n-1} vertices can be obtained from a cube graph on 2^n vertices by merging together opposed vertices. Alternatively, it can be obtained from a cube graph on 2^{n-1} vertices by adding an edge between opposed vertices. This second construction is the one produced by this method.

See the Wikipedia article Folded_cube_graph for more information.

INPUT:

- `n` – integer ≥ 2; the dimension of the folded cube
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

The folded cube graph of order five is the Clebsch graph:

```
sage: fc = graphs.FoldedCubeGraph(5)
sage: clebsch = graphs.ClebschGraph()
sage: fc.is_isomorphic(clebsch)
True
```

```
>>> from sage.all import *
>>> fc = graphs.FoldedCubeGraph(Integer(5))
>>> clebsch = graphs.ClebschGraph()
>>> fc.is_isomorphic(clebsch)
True
```

static FolkmanGraph(*immutable=False*)

Return the Folkman graph.

See the Wikipedia article Folkman_graph.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.FolkmanGraph()
sage: g.order()
20
sage: g.size()
40
sage: g.diameter()
4
sage: g.girth()
```

(continues on next page)

(continued from previous page)

```
4
sage: g.charpoly().factor()
(x - 4) * (x + 4) * x^10 * (x^2 - 6)^4
sage: g.chromatic_number()
2
sage: g.is_eulerian()
True
sage: g.is_hamiltonian()                                                    ␣
↪  # needs sage.numerical_mip
True
sage: g.is_vertex_transitive()
False
sage: g.is_bipartite()
True
```

```
>>> from sage.all import *
>>> g = graphs.FolkmanGraph()
>>> g.order()
20
>>> g.size()
40
>>> g.diameter()
4
>>> g.girth()
4
>>> g.charpoly().factor()
(x - 4) * (x + 4) * x^10 * (x^2 - 6)^4
>>> g.chromatic_number()
2
>>> g.is_eulerian()
True
>>> g.is_hamiltonian()
↪# needs sage.numerical_mip
True
>>> g.is_vertex_transitive()
False
>>> g.is_bipartite()
True
```

static ForkGraph(*immutable=False*)

Return a fork graph with 5 nodes.

A fork graph, sometimes also called chair graph, is 5 vertex tree.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

PLOTTING: Upon construction, the position dictionary is filled to override the spring-layout algorithm. By convention, the fork graph is drawn as a fork, with the sharp part on the bottom.

EXAMPLES:

Construct and show a fork graph:

```
sage: g = graphs.ForkGraph()
sage: g.show()                              # long time
↪  # needs sage.plot
```

```
>>> from sage.all import *
>>> g = graphs.ForkGraph()
>>> g.show()                              # long time
↪# needs sage.plot
```

static FosterGraph(*immutable=False*)

Return the Foster graph.

See the Wikipedia article Foster_graph.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.FosterGraph()
sage: g.order()
90
sage: g.size()
135
sage: g.diameter()
8
sage: g.girth()
10
sage: g.automorphism_group().cardinality()
4320
sage: g.is_hamiltonian()
↪  # needs sage.numerical_mip
True
```

```
>>> from sage.all import *
>>> g = graphs.FosterGraph()
>>> g.order()
90
>>> g.size()
135
>>> g.diameter()
8
>>> g.girth()
10
>>> g.automorphism_group().cardinality()
4320
>>> g.is_hamiltonian()
↪# needs sage.numerical_mip
True
```

static FosterGraph3S6(*immutable=False*)

Return the Foster graph for $3.Sym(6)$.

This graph is distance-regular with intersection array $[6, 4, 2, 1; 1, 1, 4, 6]$.

The graph is also distance transitive.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: G = graphs.FosterGraph3S6()
↪  # needs sage.libs.gap
sage: G.is_distance_regular(True)
↪  # needs sage.libs.gap
([6, 4, 2, 1, None], [None, 1, 1, 4, 6])
```

```
>>> from sage.all import *
>>> G = graphs.FosterGraph3S6()
↪# needs sage.libs.gap
>>> G.is_distance_regular(True)
↪# needs sage.libs.gap
([6, 4, 2, 1, None], [None, 1, 1, 4, 6])
```

REFERENCES:

A description and construction of this graph can be found in [BCN1989] p. 397.

static FranklinGraph(*immutable=False*)

Return the Franklin graph.

For more information, see the Wikipedia article Franklin_graph.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

The Franklin graph is named after Philip Franklin. It is a 3-regular graph on 12 vertices and having 18 edges:

```
sage: G = graphs.FranklinGraph(); G
Franklin graph: Graph on 12 vertices
sage: G.is_regular(3)
True
sage: G.order()
12
sage: G.size()
18
```

```
>>> from sage.all import *
>>> G = graphs.FranklinGraph(); G
Franklin graph: Graph on 12 vertices
>>> G.is_regular(Integer(3))
True
>>> G.order()
12
>>> G.size()
18
```

The Franklin graph is a Hamiltonian, bipartite graph with radius 3, diameter 3, and girth 4:

```
sage: G.is_hamiltonian()                                                    ␣
↪  # needs sage.numerical_mip
True
sage: G.is_bipartite()
True
sage: G.radius()
3
sage: G.diameter()
3
sage: G.girth()
4
```

```
>>> from sage.all import *
>>> G.is_hamiltonian()
↪# needs sage.numerical_mip
True
>>> G.is_bipartite()
True
>>> G.radius()
3
>>> G.diameter()
3
>>> G.girth()
4
```

It is a perfect, triangle-free graph having chromatic number 2:

```
sage: G.is_perfect()
True
sage: G.is_triangle_free()
True
sage: G.chromatic_number()
2
```

```
>>> from sage.all import *
>>> G.is_perfect()
True
>>> G.is_triangle_free()
True
>>> G.chromatic_number()
2
```

static FriendshipGraph(*n*, *immutable=False*)

Return the friendship graph F_n.

The friendship graph is also known as the Dutch windmill graph. Let C_3 be the cycle graph on 3 vertices. Then F_n is constructed by joining $n \geq 1$ copies of C_3 at a common vertex. If $n = 1$, then F_1 is isomorphic to C_3 (the triangle graph). If $n = 2$, then F_2 is the butterfly graph, otherwise known as the bowtie graph. For more information, see the Wikipedia article Friendship_graph.

INPUT:

- `n` – positive integer; the number of copies of C_3 to use in constructing F_n
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

OUTPUT:

- The friendship graph F_n obtained from n copies of the cycle graph C_3.

See also

- *ButterflyGraph()*

EXAMPLES:

The first few friendship graphs.

```
sage: A = []; B = []
sage: for i in range(9):
....:     g = graphs.FriendshipGraph(i + 1)
....:     A.append(g)
sage: for i in range(3):
....:     n = []
....:     for j in range(3):
....:         n.append(A[3*i + j].plot(vertex_size=20, vertex_labels=False))
....:     B.append(n)
sage: G = graphics_array(B)
sage: G.show()                          # long time
```

```
>>> from sage.all import *
>>> A = []; B = []
>>> for i in range(Integer(9)):
...     g = graphs.FriendshipGraph(i + Integer(1))
...     A.append(g)
>>> for i in range(Integer(3)):
...     n = []
...     for j in range(Integer(3)):
...         n.append(A[Integer(3)*i + j].plot(vertex_size=Integer(20), vertex_
↪labels=False))
...     B.append(n)
>>> G = graphics_array(B)
>>> G.show()                          # long time
```

For $n = 1$, the friendship graph F_1 is isomorphic to the cycle graph C_3, whose visual representation is a triangle.

```
sage: G = graphs.FriendshipGraph(1); G
Friendship graph: Graph on 3 vertices
sage: G.show()                          # long time
↪  # needs sage.plot
sage: G.is_isomorphic(graphs.CycleGraph(3))
True
```

```
>>> from sage.all import *
>>> G = graphs.FriendshipGraph(Integer(1)); G
Friendship graph: Graph on 3 vertices
>>> G.show()                          # long time
↪# needs sage.plot
```

(continues on next page)

(continued from previous page)

```
>>> G.is_isomorphic(graphs.CycleGraph(Integer(3)))
True
```

For $n = 2$, the friendship graph F_2 is isomorphic to the butterfly graph, otherwise known as the bowtie graph.

```
sage: G = graphs.FriendshipGraph(2); G
Friendship graph: Graph on 5 vertices
sage: G.is_isomorphic(graphs.ButterflyGraph())
True
```

```
>>> from sage.all import *
>>> G = graphs.FriendshipGraph(Integer(2)); G
Friendship graph: Graph on 5 vertices
>>> G.is_isomorphic(graphs.ButterflyGraph())
True
```

If $n \geq 2$, then the friendship graph F_n has $2n + 1$ vertices and $3n$ edges. It has radius 1, diameter 2, girth 3, and chromatic number 3. Furthermore, F_n is planar and Eulerian.

```
sage: n = randint(2, 10^3)
sage: G = graphs.FriendshipGraph(n)
sage: G.order() == 2*n + 1
True
sage: G.size() == 3*n
True
sage: G.radius()
1
sage: G.diameter()
2
sage: G.girth()
3
sage: G.chromatic_number()
3
sage: G.is_planar()
True
sage: G.is_eulerian()
True
```

```
>>> from sage.all import *
>>> n = randint(Integer(2), Integer(10)**Integer(3))
>>> G = graphs.FriendshipGraph(n)
>>> G.order() == Integer(2)*n + Integer(1)
True
>>> G.size() == Integer(3)*n
True
>>> G.radius()
1
>>> G.diameter()
2
>>> G.girth()
3
>>> G.chromatic_number()
```

(continues on next page)

(continued from previous page)

```
3
>>> G.is_planar()
True
>>> G.is_eulerian()
True
```

static FruchtGraph(*immutable=False*)

Return a Frucht Graph.

A Frucht graph has 12 nodes and 18 edges. It is the smallest cubic identity graph. It is planar and Hamiltonian. See the Wikipedia article Frucht_graph.

PLOTTING: Upon construction, the position dictionary is filled to override the spring-layout algorithm. By convention, the first seven nodes are on the outer circle, with the next four on an inner circle and the last in the center.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: FRUCHT = graphs.FruchtGraph()
sage: FRUCHT
Frucht graph: Graph on 12 vertices
sage: FRUCHT.graph6_string()
'KhCKM?_EGK?L'
sage: (graphs.FruchtGraph()).show()      # long time
```

```
>>> from sage.all import *
>>> FRUCHT = graphs.FruchtGraph()
>>> FRUCHT
Frucht graph: Graph on 12 vertices
>>> FRUCHT.graph6_string()
'KhCKM?_EGK?L'
>>> (graphs.FruchtGraph()).show()      # long time
```

static FurerGadget(*k*, *prefix=None*, *immutable=False*)

Return a Furer gadget of order `k` and their coloring.

Construct the Furer gadget described in [CFI1992], a graph composed by a middle layer of $2^(k-1)$ nodes and two sets of nodes $(a_0, ..., a_{k-1})$ and $(b_0, ..., b_{k-1})$. Each node in the middle is connected to either a_i or b_i, for each i in [0,k[. To read about the complete construction, see [CFI1992]. The returned coloring colors the middle section with one color, and then each pair (a_i, b_i) with another color. Since this method is mainly used to create Furer gadgets for the Cai-Furer-Immerman construction, returning gadgets that don't always have the same vertex labels is important, that's why there is a parameter to manually set a prefix to be appended to each vertex label.

INPUT:

- `k` – the order of the returned Furer gadget, greater than 0
- `prefix` – prefix of to be appended to each vertex label, so as to individualise the returned Furer gadget; must be comparable for equality and hashable
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

OUTPUT:

- `G` – the Furer gadget of order `k`
- `coloring` – list of list of vertices, representing the partition induced by the coloring of `G`'s vertices

EXAMPLES:

Furer gadget of order 3, without any prefix.

```
sage: G, p = graphs.FurerGadget(3)
sage: sorted(G, key=str)
[(), (0, 'a'), (0, 'b'), (0, 1), (0, 2),
 (1, 'a'), (1, 'b'), (1, 2), (2, 'a'), (2, 'b')]
sage: sorted(G.edge_iterator(), key=str)
[((), (0, 'b'), None), ((), (1, 'b'), None),
 ((), (2, 'b'), None), ((0, 'b'), (1, 2), None),
 ((0, 1), (0, 'a'), None), ((0, 1), (1, 'a'), None),
 ((0, 1), (2, 'b'), None), ((0, 2), (0, 'a'), None),
 ((0, 2), (1, 'b'), None), ((0, 2), (2, 'a'), None),
 ((1, 2), (1, 'a'), None), ((1, 2), (2, 'a'), None)]
```

```
>>> from sage.all import *
>>> G, p = graphs.FurerGadget(Integer(3))
>>> sorted(G, key=str)
[(), (0, 'a'), (0, 'b'), (0, 1), (0, 2),
 (1, 'a'), (1, 'b'), (1, 2), (2, 'a'), (2, 'b')]
>>> sorted(G.edge_iterator(), key=str)
[((), (0, 'b'), None), ((), (1, 'b'), None),
 ((), (2, 'b'), None), ((0, 'b'), (1, 2), None),
 ((0, 1), (0, 'a'), None), ((0, 1), (1, 'a'), None),
 ((0, 1), (2, 'b'), None), ((0, 2), (0, 'a'), None),
 ((0, 2), (1, 'b'), None), ((0, 2), (2, 'a'), None),
 ((1, 2), (1, 'a'), None), ((1, 2), (2, 'a'), None)]
```

Furer gadget of order 3, with a prefix.

```
sage: G, p = graphs.FurerGadget(3, 'Prefix')
sage: sorted(G, key=str)
[('Prefix', ()), ('Prefix', (0, 'a')), ('Prefix', (0, 'b')),
 ('Prefix', (0, 1)), ('Prefix', (0, 2)), ('Prefix', (1, 'a')),
 ('Prefix', (1, 'b')), ('Prefix', (1, 2)), ('Prefix', (2, 'a')),
 ('Prefix', (2, 'b'))]
sage: sorted(G.edge_iterator(), key=str)
[(('Prefix', ()), ('Prefix', (0, 'b')), None),
 (('Prefix', ()), ('Prefix', (1, 'b')), None),
 (('Prefix', ()), ('Prefix', (2, 'b')), None),
 (('Prefix', (0, 'b')), ('Prefix', (1, 2)), None),
 (('Prefix', (0, 1)), ('Prefix', (0, 'a')), None),
 (('Prefix', (0, 1)), ('Prefix', (1, 'a')), None),
 (('Prefix', (0, 1)), ('Prefix', (2, 'b')), None),
 (('Prefix', (0, 2)), ('Prefix', (0, 'a')), None),
 (('Prefix', (0, 2)), ('Prefix', (1, 'b')), None),
 (('Prefix', (0, 2)), ('Prefix', (2, 'a')), None),
 (('Prefix', (1, 2)), ('Prefix', (1, 'a')), None),
 (('Prefix', (1, 2)), ('Prefix', (2, 'a')), None)]
```

```
>>> from sage.all import *
>>> G, p = graphs.FurerGadget(Integer(3), 'Prefix')
>>> sorted(G, key=str)
[('Prefix', ()), ('Prefix', (0, 'a')), ('Prefix', (0, 'b')),
 ('Prefix', (0, 1)), ('Prefix', (0, 2)), ('Prefix', (1, 'a')),
 ('Prefix', (1, 'b')), ('Prefix', (1, 2)), ('Prefix', (2, 'a')),
 ('Prefix', (2, 'b'))]
>>> sorted(G.edge_iterator(), key=str)
[(('Prefix', ()), ('Prefix', (0, 'b')), None),
 (('Prefix', ()), ('Prefix', (1, 'b')), None),
 (('Prefix', ()), ('Prefix', (2, 'b')), None),
 (('Prefix', (0, 'b')), ('Prefix', (1, 2)), None),
 (('Prefix', (0, 1)), ('Prefix', (0, 'a')), None),
 (('Prefix', (0, 1)), ('Prefix', (1, 'a')), None),
 (('Prefix', (0, 1)), ('Prefix', (2, 'b')), None),
 (('Prefix', (0, 2)), ('Prefix', (0, 'a')), None),
 (('Prefix', (0, 2)), ('Prefix', (1, 'b')), None),
 (('Prefix', (0, 2)), ('Prefix', (2, 'a')), None),
 (('Prefix', (1, 2)), ('Prefix', (1, 'a')), None),
 (('Prefix', (1, 2)), ('Prefix', (2, 'a')), None)]
```

static FuzzyBallGraph(*partition*, *q*, *immutable=False*)

Construct a Fuzzy Ball graph with the integer partition `partition` and `q` extra vertices.

Let q be an integer and let $m_1, m_2, ..., m_k$ be a set of positive integers. Let $n = q + m_1 + ... + m_k$. The Fuzzy Ball graph with partition $m_1, m_2, ..., m_k$ and q extra vertices is the graph constructed from the graph $G = K_n$ by attaching, for each $i = 1, 2, ..., k$, a new vertex a_i to m_i distinct vertices of G.

For given positive integers k and m and nonnegative integer q, the set of graphs `FuzzyBallGraph(p, q)` for all partitions p of m with k parts are cospectral with respect to the normalized Laplacian.

INPUT:

- `partition` – non empty list of positive integers
- `q` – nonnegative integer; number of extra vertices
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: F = graphs.FuzzyBallGraph([3,1],2); F
Fuzzy-Ball([3, 1], 2): Graph on 8 vertices
sage: F.adjacency_matrix(vertices=list(F))
↪  # needs sage.modules
[0 0 1 1 1 0 0 0]
[0 0 0 0 0 1 0 0]
[1 0 0 1 1 1 1 1]
[1 0 1 0 1 1 1 1]
[1 0 1 1 0 1 1 1]
[0 1 1 1 1 0 1 1]
[0 0 1 1 1 1 0 1]
[0 0 1 1 1 1 1 0]
```

```
>>> from sage.all import *
>>> F = graphs.FuzzyBallGraph([Integer(3),Integer(1)],Integer(2)); F
```

(continues on next page)

(continued from previous page)

```
Fuzzy-Ball([3, 1], 2): Graph on 8 vertices
>>> F.adjacency_matrix(vertices=list(F))
↪# needs sage.modules
[0 0 1 1 1 0 0 0]
[0 0 0 0 0 1 0 0]
[1 0 0 1 1 1 1 1]
[1 0 1 0 1 1 1 1]
[1 0 1 1 0 1 1 1]
[0 1 1 1 1 0 1 1]
[0 0 1 1 1 1 0 1]
[0 0 1 1 1 1 1 0]
```

Pick positive integers m and k and a nonnegative integer q. All the FuzzyBallGraphs constructed from partitions of m with k parts should be cospectral with respect to the normalized Laplacian:

```
sage: m = 4; q = 2; k = 2
sage: g_list = [graphs.FuzzyBallGraph(p,q)                                    ␣
↪  # needs sage.combinat sage.modules
....:           for p in Partitions(m, length=k)]
sage: set(g.laplacian_matrix(normalized=True,   # long time (7s on sage.math,␣
↪2011), needs sage.combinat sage.modules
....:                        vertices=list(g)).charpoly()
....:     for g in g_list)
{x^8 - 8*x^7 + 4079/150*x^6 - 68689/1350*x^5 + 610783/10800*x^4
  - 120877/3240*x^3 + 1351/100*x^2 - 931/450*x}
```

```
>>> from sage.all import *
>>> m = Integer(4); q = Integer(2); k = Integer(2)
>>> g_list = [graphs.FuzzyBallGraph(p,q)
↪# needs sage.combinat sage.modules
...           for p in Partitions(m, length=k)]
>>> set(g.laplacian_matrix(normalized=True,   # long time (7s on sage.math,␣
↪2011), needs sage.combinat sage.modules
...                         vertices=list(g)).charpoly()
...     for g in g_list)
{x^8 - 8*x^7 + 4079/150*x^6 - 68689/1350*x^5 + 610783/10800*x^4
  - 120877/3240*x^3 + 1351/100*x^2 - 931/450*x}
```

static GemGraph(*immutable=False*)

Return a gem graph with 5 nodes.

A gem graph is a fan graph (4,1).

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

PLOTTING: Upon construction, the position dictionary is filled to override the spring-layout algorithm. By convention, the gem graph is drawn as a gem, with the sharp part on the bottom.

EXAMPLES:

Construct and show a gem graph:

```
sage: g = graphs.GemGraph()
sage: g.show()                              # long time                    ␣
↪  # needs sage.plot
```

```
>>> from sage.all import *
>>> g = graphs.GemGraph()
>>> g.show()                              # long time
↪# needs sage.plot
```

static GeneralisedDodecagonGraph(*s*, *t*, *immutable=False*)

Return the point-graph of a generalised dodecagon of order (s, t).

INPUT:

- `s`, `t` – integers; order of the generalised dodecagon
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: # optional - gap_package_atlasrep internet
sage: G = graphs.GeneralisedDodecagonGraph(1, 5)
sage: G.is_distance_regular(True)
([6, 5, 5, 5, 5, 5, None], [None, 1, 1, 1, 1, 1, 6])
sage: H = graphs.GeneralisedDodecagonGraph(5, 1)
sage: H.order()
23436
sage: H.is_distance_regular(True)        # not tested (6 min)
([10, 5, 5, 5, 5, 5, None], [None, 1, 1, 1, 1, 1, 2])
```

```
>>> from sage.all import *
>>> # optional - gap_package_atlasrep internet
>>> G = graphs.GeneralisedDodecagonGraph(Integer(1), Integer(5))
>>> G.is_distance_regular(True)
([6, 5, 5, 5, 5, 5, None], [None, 1, 1, 1, 1, 1, 6])
>>> H = graphs.GeneralisedDodecagonGraph(Integer(5), Integer(1))
>>> H.order()
23436
>>> H.is_distance_regular(True)        # not tested (6 min)
([10, 5, 5, 5, 5, 5, None], [None, 1, 1, 1, 1, 1, 2])
```

Note

This function indirectly uses the GAP's AtlasRep package. Thus you may need an internet connection and the optional Sage's package `gap_packages`.

REFERENCES:

See [BCN1989] pp. 200-205 for a discussion of distance-regular graphs from generalised polygons.

static GeneralisedHexagonGraph(*s*, *t*, *immutable=False*)

Return the point-graph of a generalised hexagon of order (s, t).

INPUT:

- s, t – integers; order of the generalised hexagon
- immutable – boolean (default: False); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: G = graphs.GeneralisedHexagonGraph(5, 5)          # optional - gap_
↪package_atlasrep internet
sage: G.is_distance_regular(True)                       # optional - gap_
↪package_atlasrep internet
([30, 25, 25, None], [None, 1, 1, 6])
sage: G = graphs.GeneralisedHexagonGraph(7, 1)
sage: G.is_distance_regular(True)
([14, 7, 7, None], [None, 1, 1, 2])
sage: graphs.GeneralisedHexagonGraph(1, 1)
Cycle graph: Graph on 6 vertices
```

```
>>> from sage.all import *
>>> G = graphs.GeneralisedHexagonGraph(Integer(5), Integer(5))          #␣
↪optional - gap_package_atlasrep internet
>>> G.is_distance_regular(True)                       # optional - gap_
↪package_atlasrep internet
([30, 25, 25, None], [None, 1, 1, 6])
>>> G = graphs.GeneralisedHexagonGraph(Integer(7), Integer(1))
>>> G.is_distance_regular(True)
([14, 7, 7, None], [None, 1, 1, 2])
>>> graphs.GeneralisedHexagonGraph(Integer(1), Integer(1))
Cycle graph: Graph on 6 vertices
```

Note

This function uses the GAP's AtlasRep package to build GHs of order (q, q), (q, q^3) or (q^3, q). For those graphs you need an internet connection and Sage's optional package gap_packages.

REFERENCES:

See [BCN1989] pp. 200-205 for a discussion of distance-regular graphs from generalised polygons.

static GeneralisedOctagonGraph(*s*, *t*, *immutable=False*)

Return the point-graph of a generalised octagon of order (s, t).

INPUT:

- s, t – integers; order of the generalised octagon
- immutable – boolean (default: False); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: G = graphs.GeneralisedOctagonGraph(1, 4)          # optional - database_
↪graphs
sage: G.is_distance_regular(True)                       # optional - database_
↪graphs
([5, 4, 4, 4, None], [None, 1, 1, 1, 5])
sage: G = graphs.GeneralisedOctagonGraph(2, 4)          # optional - gap_
```

(continues on next page)

(continued from previous page)

```
↪package_atlasrep internet
sage: G.is_distance_regular(True)                    # optional - gap_
↪package_atlasrep internet
([10, 8, 8, 8, None], [None, 1, 1, 1, 5])
sage: G = graphs.GeneralisedOctagonGraph(5, 1)       # optional - database_
↪graphs
sage: G.is_distance_regular(True)                    # optional - database_
↪graphs
([10, 5, 5, 5, None], [None, 1, 1, 1, 2])
```

```
>>> from sage.all import *
>>> G = graphs.GeneralisedOctagonGraph(Integer(1), Integer(4))          #␣
↪optional - database_graphs
>>> G.is_distance_regular(True)                      # optional - database_
↪graphs
([5, 4, 4, 4, None], [None, 1, 1, 1, 5])
>>> G = graphs.GeneralisedOctagonGraph(Integer(2), Integer(4))          #␣
↪optional - gap_package_atlasrep internet
>>> G.is_distance_regular(True)                      # optional - gap_
↪package_atlasrep internet
([10, 8, 8, 8, None], [None, 1, 1, 1, 5])
>>> G = graphs.GeneralisedOctagonGraph(Integer(5), Integer(1))          #␣
↪optional - database_graphs
>>> G.is_distance_regular(True)                      # optional - database_
↪graphs
([10, 5, 5, 5, None], [None, 1, 1, 1, 2])
```

Note

This function uses the GAP's AtlasRep package to build the graphs of order $(2, 4)$ or $(4, 2)$. For those graphs you need an internet connection and Sage's optional package `gap_packages`.

REFERENCES:

See [BCN1989] pp. 200-205 for a discussion of distance-regular graphs from generalised polygons.

static GeneralizedPetersenGraph(*n*, *k*, *immutable=False*, *name=None*)

Return a generalized Petersen graph with $2n$ nodes. The variables n, k are integers such that $n > 2$ and $0 < k \leq \lfloor (n-1)/2 \rfloor$

For $k = 1$ the result is a graph isomorphic to the circular ladder graph with the same n. The regular Petersen Graph has $n = 5$ and $k = 2$. Other named graphs that can be described using this notation include the Desargues graph and the Möbius-Kantor graph.

INPUT:

- `n` – the number of nodes is $2 * n$
- `k` – integer ($0 < k \leq \lfloor (n-1)/2 \rfloor$); decides how inner vertices are connected
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph
- `name` – string (default: `None`); used as the name of the returned graph when set

PLOTTING: Upon construction, the position dictionary is filled to override the spring-layout algorithm. By convention, the generalized Petersen graphs are displayed as an inner and outer cycle pair, with the first n nodes drawn on the outer circle. The first (0) node is drawn at the top of the outer-circle, moving counterclockwise after that. The inner circle is drawn with the (n)th node at the top, then counterclockwise as well.

EXAMPLES: For $k = 1$ the resulting graph will be isomorphic to a circular ladder graph.

```
sage: g = graphs.GeneralizedPetersenGraph(13,1)
sage: g2 = graphs.CircularLadderGraph(13)
sage: g.is_isomorphic(g2)
True
```

```
>>> from sage.all import *
>>> g = graphs.GeneralizedPetersenGraph(Integer(13),Integer(1))
>>> g2 = graphs.CircularLadderGraph(Integer(13))
>>> g.is_isomorphic(g2)
True
```

The Desargues graph:

```
sage: g = graphs.GeneralizedPetersenGraph(10,3)
sage: g.girth()
6
sage: g.is_bipartite()
True
```

```
>>> from sage.all import *
>>> g = graphs.GeneralizedPetersenGraph(Integer(10),Integer(3))
>>> g.girth()
6
>>> g.is_bipartite()
True
```

AUTHORS:

- Anders Jonsson (2009-10-15)

static GeneralizedSierpinskiGraph(*G*, *k*, *stretch=None*, *immutable=False*)

Return the generalized Sierpinski graph of G of dimension k.

Generalized Sierpinski graphs have been introduced in [GKP2011] to generalize the notion of Sierpinski graphs [KM1997].

Given a graph $G = (V, E)$ of order n and a parameter k, the generalized Sierpinski graph of G of dimension k, denoted by $S(G, k)$, can be constructed recursively from G as follows. $S(G, 1)$ is isomorphic to G. To construct $S(G, k)$ for $k > 1$, copy n times $S(G, k-1)$, once per vertex $u \in V$, and add u at the beginning of the labels of each vertex in the copy of $S(G, k-1)$ corresponding to vertex u. Then for any edge $\{u, v\} \in E$, add an edge between vertex $(u, v, \ldots, v)$ and vertex $(v, u, \ldots, u)$.

INPUT:

- `G` – a sage Graph
- `k` – integer; the dimension
- `stretch` – integer (default: `None`); stretching factor used to determine the positions of the vertices of the output graph. By default (`None`), this value is set to twice the maximum Euclidean distance between the vertices of G. This parameter is used only when the vertices of G have positions.

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

See also

- *SierpinskiGasketGraph()*
- *HanoiTowerGraph()*

EXAMPLES:

The generalized Sierpinski graph of dimension 1 of any graph G is isomorphic to G:

```
sage: G = graphs.RandomGNP(10, .5)
sage: S = graphs.GeneralizedSierpinskiGraph(G, 1)
sage: S.is_isomorphic(G)
True
```

```
>>> from sage.all import *
>>> G = graphs.RandomGNP(Integer(10), RealNumber('.5'))
>>> S = graphs.GeneralizedSierpinskiGraph(G, Integer(1))
>>> S.is_isomorphic(G)
True
```

When G is a clique of order 3, the generalized Sierpinski graphs of G are isomorphic to Hanoi Tower graphs:

```
sage: k = randint(1, 5)
sage: S = graphs.GeneralizedSierpinskiGraph(graphs.CompleteGraph(3), k)
↪  # needs sage.modules
sage: H = graphs.HanoiTowerGraph(3, k)
sage: S.is_isomorphic(H)
↪  # needs sage.modules
True
```

```
>>> from sage.all import *
>>> k = randint(Integer(1), Integer(5))
>>> S = graphs.GeneralizedSierpinskiGraph(graphs.CompleteGraph(Integer(3)),
↪k)          # needs sage.modules
>>> H = graphs.HanoiTowerGraph(Integer(3), k)
>>> S.is_isomorphic(H)
↪# needs sage.modules
True
```

The generalized Sierpinski graph of dimension k of any graph G with n vertices and m edges has n^k vertices and $m\sum_{i=0}^{k-1} n^i$ edges:

```
sage: n = randint(2, 6)
sage: k = randint(1, 5)
sage: G = graphs.RandomGNP(n, .5)
sage: m = G.size()
sage: S = graphs.GeneralizedSierpinskiGraph(G, k)
sage: S.order() == n**k
True
sage: S.size() == m*sum([n**i for i in range(k)])
```

(continues on next page)

(continued from previous page)

```
True
sage: G = graphs.CompleteGraph(n)
sage: S = graphs.GeneralizedSierpinskiGraph(G, k)
sage: S.order() == n**k
True
sage: S.size() == (n*(n - 1)/2)*sum([n**i for i in range(k)])
True
```

```
>>> from sage.all import *
>>> n = randint(Integer(2), Integer(6))
>>> k = randint(Integer(1), Integer(5))
>>> G = graphs.RandomGNP(n, RealNumber('.5'))
>>> m = G.size()
>>> S = graphs.GeneralizedSierpinskiGraph(G, k)
>>> S.order() == n**k
True
>>> S.size() == m*sum([n**i for i in range(k)])
True
>>> G = graphs.CompleteGraph(n)
>>> S = graphs.GeneralizedSierpinskiGraph(G, k)
>>> S.order() == n**k
True
>>> S.size() == (n*(n - Integer(1))/Integer(2))*sum([n**i for i in range(k)])
True
```

The positions of the vertices of the output graph are determined from the positions of the vertices of G, if any:

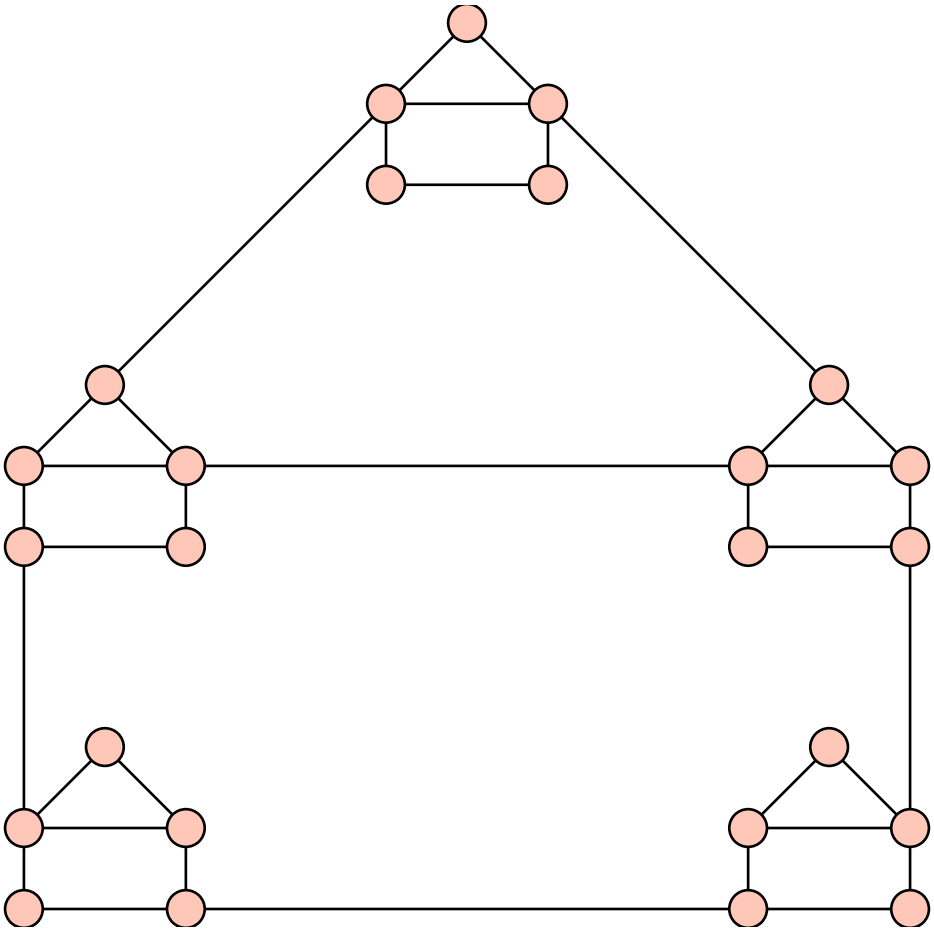
```
sage: G = graphs.HouseGraph()
sage: G.get_pos() is not None
True
sage: H = graphs.GeneralizedSierpinskiGraph(G, 2)
↪  # needs sage.symbolic
sage: H.get_pos() is not None
↪  # needs sage.symbolic
True
sage: G = Graph([(0, 1)])
sage: G.get_pos() is not None
False
sage: H = graphs.GeneralizedSierpinskiGraph(G, 2)
↪  # needs sage.symbolic
sage: H.get_pos() is not None
↪  # needs sage.symbolic
False
```

```
>>> from sage.all import *
>>> G = graphs.HouseGraph()
>>> G.get_pos() is not None
True
>>> H = graphs.GeneralizedSierpinskiGraph(G, Integer(2))
↪          # needs sage.symbolic
>>> H.get_pos() is not None
```

(continues on next page)

(continued from previous page)

```
↪# needs sage.symbolic
True
>>> G = Graph([(Integer(0), Integer(1))])
>>> G.get_pos() is not None
False
>>> H = graphs.GeneralizedSierpinskiGraph(G, Integer(2))                ␣
↪          # needs sage.symbolic
>>> H.get_pos() is not None
↪# needs sage.symbolic
False
```



static GoethalsSeidelGraph(*k*, *r*, *immutable=False*)

Return the graph Goethals-Seidel(k, r).

The graph Goethals-Seidel(k, r) comes from a construction presented in Theorem 2.4 of [GS1970]. It relies on a `(v,k)-BIBD` with r blocks and a `hadamard_matrix()` of order $r+1$. The result is a *sage.graphs.strongly_regular_db.strongly_regular_graph()* on $v(r+1)$ vertices with degree $k = (n+r-1)/2$.

It appears under this name in Andries Brouwer's database of strongly regular graphs.

INPUT:

- `k`, `r` – integers

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

See also

- *is_goethals_seidel()*

EXAMPLES:

```
sage: graphs.GoethalsSeidelGraph(3,3)                                          ␣
↪  # needs sage.combinat sage.modules
Graph on 28 vertices
sage: graphs.GoethalsSeidelGraph(3,3).is_strongly_regular(parameters=True)     ␣
↪  # needs sage.combinat sage.modules
(28, 15, 6, 10)
```

```
>>> from sage.all import *
>>> graphs.GoethalsSeidelGraph(Integer(3),Integer(3))                          ␣
↪                  # needs sage.combinat sage.modules
Graph on 28 vertices
>>> graphs.GoethalsSeidelGraph(Integer(3),Integer(3)).is_strongly_
↪regular(parameters=True)      # needs sage.combinat sage.modules
(28, 15, 6, 10)
```

static GoldnerHararyGraph(*immutable=False*)

Return the Goldner-Harary graph.

For more information, see the Wikipedia article Goldner%E2%80%93Harary_graph.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

The Goldner-Harary graph is named after A. Goldner and Frank Harary. It is a planar graph having 11 vertices and 27 edges:

```
sage: G = graphs.GoldnerHararyGraph(); G
Goldner-Harary graph: Graph on 11 vertices
sage: G.is_planar()
True
sage: G.order()
11
sage: G.size()
27
```

```
>>> from sage.all import *
>>> G = graphs.GoldnerHararyGraph(); G
Goldner-Harary graph: Graph on 11 vertices
>>> G.is_planar()
True
>>> G.order()
11
```

(continues on next page)

(continued from previous page)

```
>>> G.size()
27
```

The Goldner-Harary graph is chordal with radius 2, diameter 2, and girth 3:

```
sage: G.is_chordal()
True
sage: G.radius()
2
sage: G.diameter()
2
sage: G.girth()
3
```

```
>>> from sage.all import *
>>> G.is_chordal()
True
>>> G.radius()
2
>>> G.diameter()
2
>>> G.girth()
3
```

Its chromatic number is 4 and its automorphism group is isomorphic to the dihedral group D_6:

```
sage: G.chromatic_number()
4
sage: ag = G.automorphism_group()
 ↪  # needs sage.groups
sage: ag.is_isomorphic(DihedralGroup(6))
 ↪  # needs sage.groups
True
```

```
>>> from sage.all import *
>>> G.chromatic_number()
4
>>> ag = G.automorphism_group()
↪# needs sage.groups
>>> ag.is_isomorphic(DihedralGroup(Integer(6)))
↪          # needs sage.groups
True
```

static GolombGraph(*immutable=False*)

Return the Golomb graph.

See the Wikipedia article Golomb_graph for more information.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

The Golomb graph is a planar and Hamiltonian graph with 10 vertices and 18 edges. It has chromatic number 4, diameter 3, radius 2 and girth 3. It can be drawn in the plane as a unit distance graph:

```
sage: G = graphs.GolombGraph(); G
↪  # needs sage.symbolic
Golomb graph: Graph on 10 vertices
sage: pos = G.get_pos()
↪  # needs sage.symbolic
sage: def dist2(u, v):
....:     return (u[0]-v[0])**2 + (u[1]-v[1])**2
sage: all(dist2(pos[u], pos[v]) == 1 for u, v in G.edge_
↪iterator(labels=None))  # needs sage.symbolic
True
```

```
>>> from sage.all import *
>>> G = graphs.GolombGraph(); G
↪# needs sage.symbolic
Golomb graph: Graph on 10 vertices
>>> pos = G.get_pos()
↪# needs sage.symbolic
>>> def dist2(u, v):
...     return (u[Integer(0)]-v[Integer(0)])**Integer(2) + (u[Integer(1)]-
↪v[Integer(1)])**Integer(2)
>>> all(dist2(pos[u], pos[v]) == Integer(1) for u, v in G.edge_
↪iterator(labels=None))  # needs sage.symbolic
True
```

static GossetGraph(*immutable=False*)

Return the Gosset graph.

The Gosset graph is the skeleton of the `Gosset_3_21()` polytope. It has with 56 vertices and degree 27. For more information, see the Wikipedia article Gosset_graph.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.GossetGraph(); g
Gosset Graph: Graph on 56 vertices
sage: g.order(), g.size()
(56, 756)
```

```
>>> from sage.all import *
>>> g = graphs.GossetGraph(); g
Gosset Graph: Graph on 56 vertices
>>> g.order(), g.size()
(56, 756)
```

static GrassmannGraph(*q*, *n*, *input_e*, *immutable=False*)

Return the Grassmann graph with parameters (q, n, e).

This builds the Grassmann graph $J_q(n, e)$. That is, for a vector space $V = \mathbb{F}(q)^n$ the output is the graph on the subspaces of dimension e where two subspaces are adjacent if their intersection has dimension $e - 1$.

This graph is distance-regular with classical parameters $(\min(e, n-e), q, q, \begin{bmatrix} n-e+1 \\ 1 \end{bmatrix}_q - 1)$

INPUT:

- `q` – a prime power
- `n`, `e` – integers with $n > e + 1$
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: G = graphs.GrassmannGraph(2, 4, 2)
↪  # needs sage.modules sage.rings.finite_rings
sage: G.is_distance_regular(True)
↪  # needs sage.modules sage.rings.finite_rings
([18, 8, None], [None, 1, 9])
```

```
>>> from sage.all import *
>>> G = graphs.GrassmannGraph(Integer(2), Integer(4), Integer(2))
↪                          # needs sage.modules sage.rings.finite_rings
>>> G.is_distance_regular(True)
↪# needs sage.modules sage.rings.finite_rings
([18, 8, None], [None, 1, 9])
```

REFERENCES:

See [BCN1989] pp. 268-272 or [VDKT2016] p. 21.

static GrayGraph(*embedding=1*, *immutable=False*)

Return the Gray graph.

See the Wikipedia article Gray_graph.

INPUT:

- `embedding` – integer (default: 1); two embeddings are available, and can be selected by setting `embedding` to 1 or 2
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.GrayGraph()
sage: g.order()
54
sage: g.size()
81
sage: g.girth()
8
sage: g.diameter()
6
sage: g.show(figsize=[10, 10])          # long time
↪  # needs sage.plot
sage: graphs.GrayGraph(embedding=2).show(figsize=[10, 10])      # long time,
↪needs sage.plot
```

```
>>> from sage.all import *
>>> g = graphs.GrayGraph()
```

(continues on next page)

(continued from previous page)

```
>>> g.order()
54
>>> g.size()
81
>>> g.girth()
8
>>> g.diameter()
6
>>> g.show(figsize=[Integer(10), Integer(10)])          # long time
↪                  # needs sage.plot
>>> graphs.GrayGraph(embedding=Integer(2)).show(figsize=[Integer(10),
↪Integer(10)])      # long time, needs sage.plot
```

static Grid2dGraph(*p*, *q*, *set_positions=True*, *immutable=False*, *name=None*)

Return a 2-dimensional grid graph with $p \times q$ nodes (p rows and q columns).

A 2d grid graph resembles a 2 dimensional grid. All inner nodes are connected to their 4 neighbors. Outer (non-corner) nodes are connected to their 3 neighbors. Corner nodes are connected to their 2 neighbors.

INPUT:

- `p`, `q` – two positive integers
- `set_positions` – boolean (default: `True`); whether to set the position of the nodes
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph
- `name` – string (default: `None`); used as the name of the returned graph when set

PLOTTING: Upon construction, the position dictionary is filled to override the spring-layout algorithm. By convention, nodes are labelled in (row, column) pairs with $(0, 0)$ in the top left corner. Edges will always be horizontal and vertical - another advantage of filling the position dictionary.

EXAMPLES:

Construct and show a grid 2d graph Rows = 5, Columns = 7:

```
sage: g = graphs.Grid2dGraph(5,7)
sage: g.show()                              # long time
↪  # needs sage.plot
```

```
>>> from sage.all import *
>>> g = graphs.Grid2dGraph(Integer(5),Integer(7))
>>> g.show()                              # long time
↪# needs sage.plot
```

static GridGraph(*dim_list*, *immutable=False*)

Return an n-dimensional grid graph.

INPUT:

- `dim_list` – list of integers representing the number of nodes to extend in each dimension
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

PLOTTING: When plotting, this graph will use the default spring-layout algorithm, unless a position dictionary is specified.

EXAMPLES:

```
sage: G = graphs.GridGraph([2,3,4])
sage: G.show()                              # long time
↪  # needs sage.plot
```

```
>>> from sage.all import *
>>> G = graphs.GridGraph([Integer(2),Integer(3),Integer(4)])
>>> G.show()                              # long time
↪# needs sage.plot
```

```
sage: C = graphs.CubeGraph(4)
sage: G = graphs.GridGraph([2,2,2,2])
sage: C.show()                              # long time
↪  # needs sage.plot
sage: G.show()                              # long time
↪  # needs sage.plot
```

```
>>> from sage.all import *
>>> C = graphs.CubeGraph(Integer(4))
>>> G = graphs.GridGraph([Integer(2),Integer(2),Integer(2),Integer(2)])
>>> C.show()                              # long time
↪# needs sage.plot
>>> G.show()                              # long time
↪# needs sage.plot
```

static GritsenkoGraph(*immutable=False*)

Return SRG(65, 32, 15, 16) constructed by Gritsenko.

We took the adjacency matrix from O. Gritsenko's [Gri2021] and extracted orbits of the automorphism group on the edges.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: H = graphs.GritsenkoGraph(); H
↪  # needs sage.groups
Gritsenko strongly regular graph: Graph on 65 vertices
sage: H.is_strongly_regular(parameters=True)
↪  # needs sage.groups
(65, 32, 15, 16)
```

```
>>> from sage.all import *
>>> H = graphs.GritsenkoGraph(); H
↪# needs sage.groups
Gritsenko strongly regular graph: Graph on 65 vertices
>>> H.is_strongly_regular(parameters=True)
↪# needs sage.groups
(65, 32, 15, 16)
```

static GrotzschGraph(*immutable=False*)

Return the Grötzsch graph.

The Grötzsch graph is an example of a triangle-free graph with chromatic number equal to 4. For more information, see the Wikipedia article Gr%C3%B6tzsch_graph.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

The Grötzsch graph is named after Herbert Grötzsch. It is a Hamiltonian graph with 11 vertices and 20 edges:

```
sage: G = graphs.GrotzschGraph(); G
Grotzsch graph: Graph on 11 vertices
sage: G.is_hamiltonian()                                                    ␣
↪  # needs sage.numerical.mip
True
sage: G.order()
11
sage: G.size()
20
```

```
>>> from sage.all import *
>>> G = graphs.GrotzschGraph(); G
Grotzsch graph: Graph on 11 vertices
>>> G.is_hamiltonian()
↪# needs sage.numerical.mip
True
>>> G.order()
11
>>> G.size()
20
```

The Grötzsch graph is triangle-free and having radius 2, diameter 2, and girth 4:

```
sage: G.is_triangle_free()
True
sage: G.radius()
2
sage: G.diameter()
2
sage: G.girth()
4
```

```
>>> from sage.all import *
>>> G.is_triangle_free()
True
>>> G.radius()
2
>>> G.diameter()
2
>>> G.girth()
4
```

Its chromatic number is 4 and its automorphism group is isomorphic to the dihedral group D_5:

```
sage: G.chromatic_number()
4
sage: ag = G.automorphism_group()
↪  # needs sage.groups
sage: ag.is_isomorphic(DihedralGroup(5))
↪  # needs sage.groups
True
```

```
>>> from sage.all import *
>>> G.chromatic_number()
4
>>> ag = G.automorphism_group()
↪# needs sage.groups
>>> ag.is_isomorphic(DihedralGroup(Integer(5)))
↪          # needs sage.groups
True
```

static HaemersGraph(*q*, *hyperoval=None*, *hyperoval_matching=None*, *field=None*, *check_hyperoval=True*, *immutable=False*)

Return the Haemers graph obtained from $T_2^*(q)^*$.

Let q be a power of 2. In Sect. 8.A of [BL1984] one finds a construction of a strongly regular graph with parameters $(q^2(q+2), q^2+q-1, q-2, q)$ from the graph of $T_2^*(q)^*$, constructed by *T2starGeneralizedQuadrangleGraph()*, by redefining adjacencies in the way specified by an arbitrary `hyperoval_matching` of the points (i.e. partitioning into size two parts) of `hyperoval` defining $T_2^*(q)^*$.

While [BL1984] gives the construction in geometric terms, it can be formulated, and is implemented, in graph-theoretic ones, of re-adjusting the edges. Namely, $G = T_2^*(q)^*$ has a partition into $q+2$ independent sets I_k of size q^2 each. Each vertex in I_j is adjacent to q vertices from I_k. Each I_k is paired to some $I_{k'}$, according to `hyperoval_matching`. One adds edges (s,t) for $s,t \in I_k$ whenever s and t are adjacent to some $u \in I_{k'}$, and removes all the edges between I_k and $I_{k'}$.

INPUT:

- `q` – a power of two
- `hyperoval_matching` – if `None` (default), pair each i-th point of `hyperoval` with $(i+1)$-th. Otherwise, specifies the pairing in the format $((i_1, i_1'), (i_2, i_2'), ...)$.
- `hyperoval` – a hyperoval defining $T_2^*(q)^*$. If `None` (default), the classical hyperoval obtained from a conic is used. See the documentation of *T2starGeneralizedQuadrangleGraph()*, for more information.
- `field` – an instance of a finite field of order q, must be provided if `hyperoval` is provided
- `check_hyperoval` – boolean (default: `True`); whether to check `hyperoval` for correctness or not
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

using the built-in constructions:

```
sage: g = graphs.HaemersGraph(4); g
Haemers(4): Graph on 96 vertices
sage: g.is_strongly_regular(parameters=True)
(96, 19, 2, 4)
```

```
>>> from sage.all import *
>>> g = graphs.HaemersGraph(Integer(4)); g
Haemers(4): Graph on 96 vertices
>>> g.is_strongly_regular(parameters=True)
(96, 19, 2, 4)
```

supplying your own hyperoval_matching:

```
sage: g = graphs.HaemersGraph(4, hyperoval_matching=((0,5),(1,4),(2,3))); g
Haemers(4): Graph on 96 vertices
sage: g.is_strongly_regular(parameters=True)
(96, 19, 2, 4)
```

```
>>> from sage.all import *
>>> g = graphs.HaemersGraph(Integer(4), hyperoval_matching=((Integer(0),
↪Integer(5)),(Integer(1),Integer(4)),(Integer(2),Integer(3)))); g
Haemers(4): Graph on 96 vertices
>>> g.is_strongly_regular(parameters=True)
(96, 19, 2, 4)
```

static HalfCube(*n*, *immutable=False*)

Return the halved cube in n dimensions.

The graph is distance-regular with classical parameters $(\lfloor \frac{n}{2} \rfloor, 1, 2, 2\lceil \frac{n}{2} \rceil - 1)$.

INPUT:

- `n` – integer; must be greater than 2
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: G = graphs.HalfCube(8)
sage: G.is_distance_regular(True)
([28, 15, 6, 1, None], [None, 1, 6, 15, 28])
sage: G = graphs.HalfCube(4)
sage: G.is_distance_regular(True)
([6, 1, None], [None, 1, 6])
```

```
>>> from sage.all import *
>>> G = graphs.HalfCube(Integer(8))
>>> G.is_distance_regular(True)
([28, 15, 6, 1, None], [None, 1, 6, 15, 28])
>>> G = graphs.HalfCube(Integer(4))
>>> G.is_distance_regular(True)
([6, 1, None], [None, 1, 6])
```

REFERENCES:

See [BCN1989] pp. 264, 265 or [VDKT2016] p. 21. This construction can be found on Wikipedia article Halved_cube_graph#Equivalent_constructions

static HallJankoGraph(*from_string=True*, *immutable=False*)

Return the Hall-Janko graph.

For more information on the Hall-Janko graph, see the Wikipedia article Hall-Janko_graph.

The construction used to generate this graph in Sage is by a 100-point permutation representation of the Janko group J_2, as described in version 3 of the ATLAS of Finite Group representations, in particular on the page ATLAS: J2 – Permutation representation on 100 points.

INPUT:

- `from_string` – boolean (default: `True`); whether to build the graph from its sparse6 string or through GAP. The two methods return the same graph though doing it through GAP takes more time.
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.HallJankoGraph()
sage: g.is_regular(36)
True
sage: g.is_vertex_transitive()
↪  # needs sage.groups
True
```

```
>>> from sage.all import *
>>> g = graphs.HallJankoGraph()
>>> g.is_regular(Integer(36))
True
>>> g.is_vertex_transitive()
↪# needs sage.groups
True
```

Is it really strongly regular with parameters 14, 12?

```
sage: nu = set(g.neighbors(0))
sage: for v in range(1, 100):
....:     if v in nu:
....:         expected = 14
....:     else:
....:         expected = 12
....:     nv = set(g.neighbors(v))
....:     nv.discard(0)
....:     if len(nu & nv) != expected:
....:         print("Something is wrong here!!!")
....:         break
```

```
>>> from sage.all import *
>>> nu = set(g.neighbors(Integer(0)))
>>> for v in range(Integer(1), Integer(100)):
...     if v in nu:
...         expected = Integer(14)
...     else:
...         expected = Integer(12)
...     nv = set(g.neighbors(v))
...     nv.discard(Integer(0))
...     if len(nu & nv) != expected:
...         print("Something is wrong here!!!")
...         break
```

Some other properties that we know how to check:

```
sage: g.diameter()
2
sage: g.girth()
3
sage: factor(g.characteristic_polynomial())
↪  # needs sage.libs.pari sage.modules
(x - 36) * (x - 6)^36 * (x + 4)^63
```

```
>>> from sage.all import *
>>> g.diameter()
2
>>> g.girth()
3
>>> factor(g.characteristic_polynomial())
↪# needs sage.libs.pari sage.modules
(x - 36) * (x - 6)^36 * (x + 4)^63
```

static HammingGraph(*n*, *q*, *X=None*, *immutable=False*)

Return the Hamming graph with parameters n, q over X.

Hamming graphs are graphs over the cartesian product of n copies of X, where $q = |X|$, where the vertices, labelled with the corresponding tuple in X^n, are connected if the Hamming distance between their labels is 1. All Hamming graphs are regular, vertex-transitive and distance-regular.

Hamming graphs with parameters $(1, q)$ represent the complete graph with q vertices over the set X.

INPUT:

- `n` – power to which `X` will be raised to provide vertices for the Hamming graph
- `q` – cardinality of `X`
- `X` – list of labels representing the vertices of the underlying graph the Hamming graph will be based on; if `None` (or left unused), the list $[0, ..., q-1]$ will be used
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

OUTPUT:

- `G` – the Hamming graph with parameters (n, q, X)

EXAMPLES:

Every Hamming graph is distance-regular, regular and vertex-transitive:

```
sage: g = graphs.HammingGraph(3, 7)
sage: g.is_distance_regular()
True
sage: g.is_regular()
True
sage: g.is_vertex_transitive()
↪  # needs sage.groups
True
```

```
>>> from sage.all import *
>>> g = graphs.HammingGraph(Integer(3), Integer(7))
>>> g.is_distance_regular()
```

(continues on next page)

(continued from previous page)

```
True
>>> g.is_regular()
True
>>> g.is_vertex_transitive()
↪# needs sage.groups
True
```

A Hamming graph with parameters $(1, q)$ is isomorphic to the Complete graph with parameter q:

```
sage: g = graphs.HammingGraph(1, 23)
sage: g.is_isomorphic(graphs.CompleteGraph(23))
True
```

```
>>> from sage.all import *
>>> g = graphs.HammingGraph(Integer(1), Integer(23))
>>> g.is_isomorphic(graphs.CompleteGraph(Integer(23)))
True
```

If a parameter q is provided which is not equal to X's cardinality, an exception is raised:

```
sage: X = ['a','b','c','d','e']
sage: g = graphs.HammingGraph(2, 3, X)
Traceback (most recent call last):
...
ValueError: q must be the cardinality of X
```

```
>>> from sage.all import *
>>> X = ['a','b','c','d','e']
>>> g = graphs.HammingGraph(Integer(2), Integer(3), X)
Traceback (most recent call last):
...
ValueError: q must be the cardinality of X
```

REFERENCES:

For a more accurate description, see the following wikipedia page: Wikipedia article Hamming_graph

static HanoiTowerGraph(*pegs*, *disks*, *labels=True*, *positions=True*, *immutable=False*)

Return the graph whose vertices are the states of the Tower of Hanoi puzzle, with edges representing legal moves between states.

INPUT:

- `pegs` – the number of pegs in the puzzle, 2 or greater
- `disks` – the number of disks in the puzzle, 1 or greater
- `labels` – (default: `True`) if `True` the graph contains more meaningful labels, see explanation below. For large instances, turn off labels for much faster creation of the graph.
- `positions` – (default: `True`) if `True` the graph contains layout information. This creates a planar layout for the case of three pegs. For large instances, turn off layout information for much faster creation of the graph.
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

OUTPUT:

The Tower of Hanoi puzzle has a certain number of identical pegs and a certain number of disks, each of a different radius. Initially the disks are all on a single peg, arranged in order of their radii, with the largest on the bottom.

The goal of the puzzle is to move the disks to any other peg, arranged in the same order. The one constraint is that the disks resident on any one peg must always be arranged with larger radii lower down.

The vertices of this graph represent all the possible states of this puzzle. Each state of the puzzle is a tuple with length equal to the number of disks, ordered by largest disk first. The entry of the tuple is the peg where that disk resides. Since disks on a given peg must go down in size as we go up the peg, this totally describes the state of the puzzle.

For example `(2,0,0)` means the large disk is on peg 2, the medium disk is on peg 0, and the small disk is on peg 0 (and we know the small disk must be above the medium disk). We encode these tuples as integers with a base equal to the number of pegs, and low-order digits to the right.

Two vertices are adjacent if we can change the puzzle from one state to the other by moving a single disk. For example, `(2,0,0)` is adjacent to `(2,0,1)` since we can move the small disk off peg 0 and onto (the empty) peg 1. So the solution to a 3-disk puzzle (with at least two pegs) can be expressed by the shortest path between `(0,0,0)` and `(1,1,1)`. For more on this representation of the graph, or its properties, see [AD2010].

For greatest speed we create graphs with integer vertices, where we encode the tuples as integers with a base equal to the number of pegs, and low-order digits to the right. So for example, in a 3-peg puzzle with 5 disks, the state `(1,2,0,1,1)` is encoded as $1 * 3^4 + 2 * 3^3 + 0 * 3^2 + 1 * 3^1 + 1 * 3^0 = 139$.

For smaller graphs, the labels that are the tuples are informative, but slow down creation of the graph. Likewise computing layout information also incurs a significant speed penalty. For maximum speed, turn off labels and layout and decode the vertices explicitly as needed. The `sage.rings.integer.Integer.digits()` with the `padsto` option is a quick way to do this, though you may want to reverse the list that is output.

See also

- *GeneralizedSierpinskiGraph()*

PLOTTING:

The layout computed when `positions = True` will look especially good for the three-peg case, when the graph is known to be planar. Except for two small cases on 4 pegs, the graph is otherwise not planar, and likely there is a better way to layout the vertices.

EXAMPLES:

A classic puzzle uses 3 pegs. We solve the 5 disk puzzle using integer labels and report the minimum number of moves required. Note that $3^5 - 1$ is the state where all 5 disks are on peg 2.

```
sage: H = graphs.HanoiTowerGraph(3, 5, labels=False, positions=False)
sage: H.distance(0, 3^5-1)
31
```

```
>>> from sage.all import *
>>> H = graphs.HanoiTowerGraph(Integer(3), Integer(5), labels=False,␣
↪positions=False)
>>> H.distance(Integer(0), Integer(3)**Integer(5)-Integer(1))
31
```

A slightly larger instance.

```
sage: H = graphs.HanoiTowerGraph(4, 6, labels=False, positions=False)
sage: H.n_vertices()
4096
sage: H.distance(0, 4^6-1)
17
```

```
>>> from sage.all import *
>>> H = graphs.HanoiTowerGraph(Integer(4), Integer(6), labels=False,␣
↪positions=False)
>>> H.n_vertices()
4096
>>> H.distance(Integer(0), Integer(4)**Integer(6)-Integer(1))
17
```

For a small graph, labels and layout information can be useful. Here we explicitly list a solution as a list of states.

```
sage: H = graphs.HanoiTowerGraph(3, 3, labels=True, positions=True)
sage: H.shortest_path((0,0,0), (1,1,1))
[(0, 0, 0), (0, 0, 1), (0, 2, 1), (0, 2, 2), (1, 2, 2), (1, 2, 0), (1, 1, 0),␣
↪(1, 1, 1)]
```

```
>>> from sage.all import *
>>> H = graphs.HanoiTowerGraph(Integer(3), Integer(3), labels=True,␣
↪positions=True)
>>> H.shortest_path((Integer(0),Integer(0),Integer(0)), (Integer(1),
↪Integer(1),Integer(1)))
[(0, 0, 0), (0, 0, 1), (0, 2, 1), (0, 2, 2), (1, 2, 2), (1, 2, 0), (1, 1, 0),␣
↪(1, 1, 1)]
```

Some facts about this graph with p pegs and d disks:

- only automorphisms are the "obvious" ones – renumber the pegs
- chromatic number is less than or equal to p
- independence number is p^{d-1}

```
sage: H = graphs.HanoiTowerGraph(3, 4, labels=False, positions=False)
sage: H.automorphism_group().is_isomorphic(SymmetricGroup(3))                 ␣
↪  # needs sage.groups
True
sage: H.chromatic_number()
3
sage: len(H.independent_set()) == 3^(4-1)
True
```

```
>>> from sage.all import *
>>> H = graphs.HanoiTowerGraph(Integer(3), Integer(4), labels=False,␣
↪positions=False)
>>> H.automorphism_group().is_isomorphic(SymmetricGroup(Integer(3)))          ␣
↪          # needs sage.groups
True
>>> H.chromatic_number()
```

(continues on next page)

(continued from previous page)

```
3
>>> len(H.independent_set()) == Integer(3)**(Integer(4)-Integer(1))
True
```

AUTHOR:

- Rob Beezer, (2009-12-26), with assistance from Su Doree

static HararyGraph(*k*, *n*, *immutable=False*)

Return the Harary graph on n vertices and connectivity k, where $2 \leq k < n$.

A k-connected graph G on n vertices requires the minimum degree $\delta(G) \geq k$, so the minimum number of edges G should have is $\lceil kn/2 \rceil$. Harary graphs achieve this lower bound, that is, Harary graphs are minimal k-connected graphs on n vertices.

The construction provided uses the method CirculantGraph. For more details, see the book [West2001] or the MathWorld article on Harary graphs.

INPUT:

- `k` – integer; connectivity of the graph
- `n` – integer; number of vertices of the graph
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

Harary graphs $H_{k,n}$:

```
sage: h = graphs.HararyGraph(5,9); h
Harary graph 5, 9: Graph on 9 vertices
sage: h.order()
9
sage: h.size()
23
sage: h.vertex_connectivity()
↪  # needs sage.numerical.mip
5
```

```
>>> from sage.all import *
>>> h = graphs.HararyGraph(Integer(5),Integer(9)); h
Harary graph 5, 9: Graph on 9 vertices
>>> h.order()
9
>>> h.size()
23
>>> h.vertex_connectivity()
↪# needs sage.numerical.mip
5
```

static HarborthGraph(*immutable=False*)

Return the Harborth Graph.

The Harborth graph has 104 edges and 52 vertices, and is the smallest known example of a 4-regular matchstick graph. For more information, see the Wikipedia article Harborth_graph.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.HarborthGraph(); g
Harborth Graph: Graph on 52 vertices
sage: g.is_regular(4)
True
```

```
>>> from sage.all import *
>>> g = graphs.HarborthGraph(); g
Harborth Graph: Graph on 52 vertices
>>> g.is_regular(Integer(4))
True
```

static HarriesGraph(*embedding=1*, *immutable=False*)

Return the Harries Graph.

The Harries graph is a Hamiltonian 3-regular graph on 70 vertices. See the Wikipedia article Harries_graph.

The default embedding here is to emphasize the graph's 4 orbits. This graph actually has a funny construction. The following procedure gives an idea of it, though not all the adjacencies are being properly defined.

1. Take two disjoint copies of a *Petersen graph*. Their vertices will form an orbit of the final graph.
2. Subdivide all the edges once, to create 15+15=30 new vertices, which together form another orbit.
3. Create 15 vertices, each of them linked to 2 corresponding vertices of the previous orbit, one in each of the two subdivided Petersen graphs. At the end of this step all vertices from the previous orbit have degree 3, and the only vertices of degree 2 in the graph are those that were just created.
4. Create 5 vertices connected only to the ones from the previous orbit so that the graph becomes 3-regular.

INPUT:

- `embedding` – integer (default: 1); two embeddings are available, and can be selected by setting `embedding` to 1 or 2
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.HarriesGraph()
sage: g.order()
70
sage: g.size()
105
sage: g.girth()
10
sage: g.diameter()
6
sage: g.show(figsize=[10, 10])          # long time
↪  # needs sage.plot
sage: graphs.HarriesGraph(embedding=2).show(figsize=[10, 10])    # long time,
↪needs sage.plot
```

```
>>> from sage.all import *
>>> g = graphs.HarriesGraph()
```

(continues on next page)

(continued from previous page)

```
>>> g.order()
70
>>> g.size()
105
>>> g.girth()
10
>>> g.diameter()
6
>>> g.show(figsize=[Integer(10), Integer(10)])          # long time
↪                  # needs sage.plot
>>> graphs.HarriesGraph(embedding=Integer(2)).show(figsize=[Integer(10),
↪Integer(10)])    # long time, needs sage.plot
```

static HarriesWongGraph(*embedding=1*, *immutable=False*)

Return the Harries-Wong Graph.

See the Wikipedia article Harries-Wong_graph.

About the default embedding:

The default embedding is an attempt to emphasize the graph's 8 (!!!) different orbits. In order to understand this better, one can picture the graph as being built in the following way.

1. One first creates a 3-dimensional cube (8 vertices, 12 edges), whose vertices define the first orbit of the final graph.
2. The edges of this graph are subdivided once, to create 12 new vertices which define a second orbit.
3. The edges of the graph are subdivided once more, to create 24 new vertices giving a third orbit.
4. 4 vertices are created and made adjacent to the vertices of the second orbit so that they have degree 3. These 4 vertices also define a new orbit.
5. In order to make the vertices from the third orbit 3-regular (they all miss one edge), one creates a binary tree on 1 + 3 + 6 + 12 vertices. The leaves of this new tree are made adjacent to the 12 vertices of the third orbit, and the graph is now 3-regular. This binary tree contributes 4 new orbits to the Harries-Wong graph.

INPUT:

- `embedding` – integer (default: 1); two embeddings are available, and can be selected by setting `embedding` to 1 or 2
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.HarriesWongGraph()
sage: g.order()
70
sage: g.size()
105
sage: g.girth()
10
sage: g.diameter()
6
sage: orbits = g.automorphism_group(orbits=True)[-1]     # long time
↪  # needs sage.groups
```

(continues on next page)

(continued from previous page)

```
sage: g.show(figsize=[15, 15], partition=orbits)        # long time
↪  # needs sage.groups sage.plot
```

```
>>> from sage.all import *
>>> g = graphs.HarriesWongGraph()
>>> g.order()
70
>>> g.size()
105
>>> g.girth()
10
>>> g.diameter()
6
>>> orbits = g.automorphism_group(orbits=True)[-Integer(1)]    # long time
↪         # needs sage.groups
>>> g.show(figsize=[Integer(15), Integer(15)], partition=orbits)        #
↪long time             # needs sage.groups sage.plot
```

Alternative embedding:

```
sage: graphs.HarriesWongGraph(embedding=2).show()        # long time
↪  # needs sage.plot
```

```
>>> from sage.all import *
>>> graphs.HarriesWongGraph(embedding=Integer(2)).show()        # long time
↪         # needs sage.plot
```

static HeawoodGraph(*immutable=False*)

Return a Heawood graph.

The Heawood graph is a cage graph that has 14 nodes. It is a cubic symmetric graph. (See also the Möbius-Kantor graph, *MoebiusKantorGraph()*). It is nonplanar and Hamiltonian. It has diameter 3, radius 3, girth 6, and chromatic number 2. It is 4-transitive but not 5-transitive. See the Wikipedia article Heawood_graph.

PLOTTING: Upon construction, the position dictionary is filled to override the spring-layout algorithm. By convention, the nodes are positioned in a circular layout with the first node appearing at the top, and then continuing counterclockwise.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: H = graphs.HeawoodGraph()
sage: H
Heawood graph: Graph on 14 vertices
sage: H.graph6_string()
'MhEGHC@AI?_PC@_G_'
sage: (graphs.HeawoodGraph()).show()     # long time
↪  # needs sage.plot
```

```
>>> from sage.all import *
>>> H = graphs.HeawoodGraph()
>>> H
Heawood graph: Graph on 14 vertices
>>> H.graph6_string()
'MhEGHC@AI?_PC@_G_'
>>> (graphs.HeawoodGraph()).show()     # long time
↪# needs sage.plot
```

static HermitianFormsGraph(*n*, *r*, *immutable=False*)

Return the Hermitian forms graph with the given parameters.

We build a graph whose vertices are all $n \times n$ Hermitian matrices over `GF(r^2)`. Two vertices are adjacent if the difference of the two vertices has rank 1.

This graph is distance-regular with classical parameters $(n, -r, -r-1, -(-r)^d - 1)$.

INPUT:

- `n` – integer
- `r` – a prime power
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: G = graphs.HermitianFormsGraph(2, 2)
sage: G.is_distance_regular(True)
([5, 4, None], [None, 1, 2])
sage: G = graphs.HermitianFormsGraph(3, 3)      # not tested (2 min)
sage: G.order()                                 # not tested (because of the above)
19683
```

```
>>> from sage.all import *
>>> G = graphs.HermitianFormsGraph(Integer(2), Integer(2))
>>> G.is_distance_regular(True)
([5, 4, None], [None, 1, 2])
>>> G = graphs.HermitianFormsGraph(Integer(3), Integer(3))      # not tested␣
↪(2 min)
>>> G.order()                           # not tested (because of the above)
19683
```

REFERENCES:

See [BCN1989] p. 285 or [VDKT2016] p. 22.

static HerschelGraph(*immutable=False*)

Return the Herschel graph.

For more information, see the Wikipedia article Herschel_graph.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

The Herschel graph is named after Alexander Stewart Herschel. It is a planar, bipartite graph with 11 vertices and 18 edges:

```
sage: G = graphs.HerschelGraph(); G
Herschel graph: Graph on 11 vertices
sage: G.is_planar()
True
sage: G.is_bipartite()
True
sage: G.order()
11
sage: G.size()
18
```

```
>>> from sage.all import *
>>> G = graphs.HerschelGraph(); G
Herschel graph: Graph on 11 vertices
>>> G.is_planar()
True
>>> G.is_bipartite()
True
>>> G.order()
11
>>> G.size()
18
```

The Herschel graph is a perfect graph with radius 3, diameter 4, and girth 4:

```
sage: G.is_perfect()
True
sage: G.radius()
3
sage: G.diameter()
4
sage: G.girth()
4
```

```
>>> from sage.all import *
>>> G.is_perfect()
True
>>> G.radius()
3
>>> G.diameter()
4
>>> G.girth()
4
```

Its chromatic number is 2 and its automorphism group is isomorphic to the dihedral group D_6:

```
sage: G.chromatic_number()
2
sage: ag = G.automorphism_group()
↪  # needs sage.groups
sage: ag.is_isomorphic(DihedralGroup(6))
↪  # needs sage.groups
True
```

```
>>> from sage.all import *
>>> G.chromatic_number()
2
>>> ag = G.automorphism_group()
↪# needs sage.groups
>>> ag.is_isomorphic(DihedralGroup(Integer(6)))
↪          # needs sage.groups
True
```

static HexahedralGraph(*immutable=False*)

Return a hexahedral graph (with 8 nodes).

A regular hexahedron is a 6-sided cube. The hexahedral graph corresponds to the connectivity of the vertices of the hexahedron. This graph is equivalent to a 3-cube.

PLOTTING: The Hexahedral graph should be viewed in 3 dimensions. We choose to use a planar embedding of the graph. We hope to add rotatable, 3-dimensional viewing in the future. In such a case, an argument will be added to select the desired layout.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

Construct and show a Hexahedral graph:

```
sage: g = graphs.HexahedralGraph()
sage: g.show()                              # long time
↪  # needs sage.plot
```

```
>>> from sage.all import *
>>> g = graphs.HexahedralGraph()
>>> g.show()                              # long time
↪# needs sage.plot
```

Create several hexahedral graphs in a Sage graphics array. They will be drawn differently due to the use of the spring-layout algorithm:

```
sage: g = []
sage: j = []
sage: for i in range(9):
....:     k = graphs.HexahedralGraph()
....:     g.append(k)
sage: for i in range(3):
....:     n = []
....:     for m in range(3):
....:         n.append(g[3*i + m].plot(vertex_size=50, vertex_labels=False))
....:     j.append(n)
sage: G = graphics_array(j)
sage: G.show()                              # long time
```

```
>>> from sage.all import *
>>> g = []
>>> j = []
```

(continues on next page)

(continued from previous page)

```
>>> for i in range(Integer(9)):
...     k = graphs.HexahedralGraph()
...     g.append(k)
>>> for i in range(Integer(3)):
...     n = []
...     for m in range(Integer(3)):
...         n.append(g[Integer(3)*i + m].plot(vertex_size=Integer(50), vertex_
↪labels=False))
...     j.append(n)
>>> G = graphics_array(j)
>>> G.show()                          # long time
```

static HigmanSimsGraph(*relabel=True*, *immutable=False*)

Return the Higman-Sims graph.

The Higman-Sims graph is a remarkable strongly regular graph of degree 22 on 100 vertices. For example, it can be split into two sets of 50 vertices each, so that each half induces a subgraph isomorphic to the Hoffman-Singleton graph (*HoffmanSingletonGraph()*). This can be done in 352 ways (see Higman-Sims graph by Andries E. Brouwer, accessed 24 October 2009.)

Its most famous property is that the automorphism group has an index 2 subgroup which is one of the 26 sporadic groups [HS1968].

The construction used here follows [Haf2004].

See also the Wikipedia article Higman–Sims_graph.

INPUT:

- `relabel` – boolean (default: `True`); whether to relabel the vertices with consecutive integers. If `False` the labels are strings that are three digits long. "xyz" means the vertex is in group x (zero through three), pentagon or pentagram y (zero through four), and is vertex z (zero through four) of that pentagon or pentagram. See [Haf2004] for more.
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

OUTPUT: the Higman-Sims graph

EXAMPLES:

A split into the first 50 and last 50 vertices will induce two copies of the Hoffman-Singleton graph, and we illustrate another such split, which is obvious based on the construction used:

```
sage: H = graphs.HigmanSimsGraph()
sage: A = H.subgraph(range(0,50))
sage: B = H.subgraph(range(50,100))
sage: K = graphs.HoffmanSingletonGraph()
sage: K.is_isomorphic(A) and K.is_isomorphic(B)
True
sage: C = H.subgraph(range(25,75))
sage: D = H.subgraph(list(range(0,25))+list(range(75,100)))
sage: K.is_isomorphic(C) and K.is_isomorphic(D)
True
```

```
>>> from sage.all import *
>>> H = graphs.HigmanSimsGraph()
>>> A = H.subgraph(range(Integer(0),Integer(50)))
```

(continues on next page)

(continued from previous page)

```
>>> B = H.subgraph(range(Integer(50),Integer(100)))
>>> K = graphs.HoffmanSingletonGraph()
>>> K.is_isomorphic(A) and K.is_isomorphic(B)
True
>>> C = H.subgraph(range(Integer(25),Integer(75)))
>>> D = H.subgraph(list(range(Integer(0),Integer(25)))+list(range(Integer(75),
↪Integer(100))))
>>> K.is_isomorphic(C) and K.is_isomorphic(D)
True
```

The automorphism group contains only one nontrivial proper normal subgroup, which is of index 2 and is simple. It is known as the Higman-Sims group:

```
sage: H = graphs.HigmanSimsGraph()
sage: G = H.automorphism_group()
sage: g = G.order(); g
88704000
sage: K = G.normal_subgroups()[1]
sage: K.is_simple()
True
sage: g//K.order()
2
```

```
>>> from sage.all import *
>>> H = graphs.HigmanSimsGraph()
>>> G = H.automorphism_group()
>>> g = G.order(); g
88704000
>>> K = G.normal_subgroups()[Integer(1)]
>>> K.is_simple()
True
>>> g//K.order()
2
```

AUTHOR:

- Rob Beezer (2009-10-24)

static HoffmanGraph(*immutable=False*)

Return the Hoffman Graph.

See the Wikipedia article Hoffman_graph.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.HoffmanGraph()
sage: g.is_bipartite()
True
sage: g.is_hamiltonian()                # long time                    ␣
↪  # needs sage.numerical.mip
True
```

(continues on next page)

(continued from previous page)

```
sage: g.radius()
3
sage: g.diameter()
4
sage: g.automorphism_group().cardinality()                                  ␣
↪  # needs sage.groups
48
```

```
>>> from sage.all import *
>>> g = graphs.HoffmanGraph()
>>> g.is_bipartite()
True
>>> g.is_hamiltonian()                 # long time
↪# needs sage.numerical.mip
True
>>> g.radius()
3
>>> g.diameter()
4
>>> g.automorphism_group().cardinality()
↪# needs sage.groups
48
```

static HoffmanSingletonGraph(*immutable=False*)

Return the Hoffman-Singleton graph.

The Hoffman-Singleton graph is the Moore graph of degree 7, diameter 2 and girth 5. The Hoffman-Singleton theorem states that any Moore graph with girth 5 must have degree 2, 3, 7 or 57. The first three respectively are the pentagon, the Petersen graph, and the Hoffman-Singleton graph. The existence of a Moore graph with girth 5 and degree 57 is still open.

A Moore graph is a graph with diameter d and girth $2d+1$. This implies that the graph is regular, and distance regular.

For more details, see [GR2001] and the Wikipedia article Hoffman–Singleton_graph.

PLOTTING: Upon construction, the position dictionary is filled to override the spring-layout algorithm. A novel algorithm written by Tom Boothby gives a random layout which is pleasing to the eye.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: HS = graphs.HoffmanSingletonGraph()
sage: Set(HS.degree())
{7}
sage: HS.girth()
5
sage: HS.diameter()
2
sage: HS.n_vertices()
50
```

```
>>> from sage.all import *
>>> HS = graphs.HoffmanSingletonGraph()
>>> Set(HS.degree())
{7}
>>> HS.girth()
5
>>> HS.diameter()
2
>>> HS.n_vertices()
50
```

Note that you get a different layout each time you create the graph.

```
sage: HS.layout()[1]  # random
(-0.844..., 0.535...)
sage: HS = graphs.HoffmanSingletonGraph()
sage: HS.layout()[1]  # random
(-0.904..., 0.425...)
```

```
>>> from sage.all import *
>>> HS.layout()[Integer(1)]  # random
(-0.844..., 0.535...)
>>> HS = graphs.HoffmanSingletonGraph()
>>> HS.layout()[Integer(1)]  # random
(-0.904..., 0.425...)
```

static HoltGraph(*immutable=False*)

Return the Holt graph (also called the Doyle graph).

See the Wikipedia article Holt_graph.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.HoltGraph();g
Holt graph: Graph on 27 vertices
sage: g.is_regular()
True
sage: g.is_vertex_transitive()
↪  # needs sage.groups
True
sage: g.chromatic_number()
3
sage: g.is_hamiltonian()                # long time
↪  # needs sage.numerical.mip
True
sage: g.radius()
3
sage: g.diameter()
3
sage: g.girth()
5
```

(continues on next page)

(continued from previous page)

```
sage: g.automorphism_group().cardinality()
↪  # needs sage.groups
54
```

```
>>> from sage.all import *
>>> g = graphs.HoltGraph();g
Holt graph: Graph on 27 vertices
>>> g.is_regular()
True
>>> g.is_vertex_transitive()
↪# needs sage.groups
True
>>> g.chromatic_number()
3
>>> g.is_hamiltonian()                # long time
↪# needs sage.numerical.mip
True
>>> g.radius()
3
>>> g.diameter()
3
>>> g.girth()
5
>>> g.automorphism_group().cardinality()
↪# needs sage.groups
54
```

static HortonGraph(*immutable=False*)

Return the Horton Graph.

The Horton graph is a cubic 3-connected non-hamiltonian graph. For more information, see the Wikipedia article Horton_graph.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.HortonGraph()
sage: g.order()
96
sage: g.size()
144
sage: g.radius()
10
sage: g.diameter()
10
sage: g.girth()
6
sage: g.automorphism_group().cardinality()
96
sage: g.chromatic_number()
2
```

(continues on next page)

(continued from previous page)

```
sage: g.is_hamiltonian()                # not tested (veeeery long)
↪  # needs sage.numerical.mip
False
```

```
>>> from sage.all import *
>>> g = graphs.HortonGraph()
>>> g.order()
96
>>> g.size()
144
>>> g.radius()
10
>>> g.diameter()
10
>>> g.girth()
6
>>> g.automorphism_group().cardinality()
96
>>> g.chromatic_number()
2
>>> g.is_hamiltonian()                # not tested (veeeery long)
↪# needs sage.numerical.mip
False
```

static HouseGraph(*immutable=False*)

Return a house graph with 5 nodes.

A house graph is named for its shape. It is a triangle (roof) over a square (walls).

PLOTTING: Upon construction, the position dictionary is filled to override the spring-layout algorithm. By convention, the house graph is drawn with the first node in the lower-left corner of the house, the second in the lower-right corner of the house. The third node is in the upper-left corner connecting the roof to the wall, and the fourth is in the upper-right corner connecting the roof to the wall. The fifth node is the top of the roof, connected only to the third and fourth.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

Construct and show a house graph:

```
sage: g = graphs.HouseGraph()
sage: g.show()                          # long time
↪  # needs sage.plot
```

```
>>> from sage.all import *
>>> g = graphs.HouseGraph()
>>> g.show()                          # long time
↪# needs sage.plot
```

static HouseXGraph(*immutable=False*)

Return a house X graph with 5 nodes.

A house X graph is a house graph with two additional edges. The upper-right corner is connected to the lower-left. And the upper-left corner is connected to the lower-right.

PLOTTING: Upon construction, the position dictionary is filled to override the spring-layout algorithm. By convention, the house X graph is drawn with the first node in the lower-left corner of the house, the second in the lower-right corner of the house. The third node is in the upper-left corner connecting the roof to the wall, and the fourth is in the upper-right corner connecting the roof to the wall. The fifth node is the top of the roof, connected only to the third and fourth.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

Construct and show a house X graph:

```
sage: g = graphs.HouseXGraph()
sage: g.show()                              # long time
↪  # needs sage.plot
```

```
>>> from sage.all import *
>>> g = graphs.HouseXGraph()
>>> g.show()                              # long time
↪# needs sage.plot
```

static HyperStarGraph(*n*, *k*, *immutable=False*)

Return the hyper-star graph $HS(n, k)$.

The vertices of the hyper-star graph are the set of binary strings of length n which contain k 1s. Two vertices, u and v, are adjacent only if u can be obtained from v by swapping the first bit with a different symbol in another position. For instance, vertex `'011100'` of $HS(6, 3)$ is adjacent to vertices `'101100'`, `'110100'` and `'111000'`. See [LKOL2002] for more details.

INPUT:

- `n` – nonnegative integer; length of the binary strings
- `k` – nonnegative integer; number of 1s per binary string
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.HyperStarGraph(6,3)
sage: sorted(g.neighbors('011100'))
['101100', '110100', '111000']
sage: g.plot()                              # long time
↪  # needs sage.plot
Graphics object consisting of 51 graphics primitives
```

```
>>> from sage.all import *
>>> g = graphs.HyperStarGraph(Integer(6),Integer(3))
>>> sorted(g.neighbors('011100'))
['101100', '110100', '111000']
>>> g.plot()                              # long time
↪# needs sage.plot
Graphics object consisting of 51 graphics primitives
```

AUTHORS:

- Michael Yurko (2009-09-01)

static IGraph(*n*, *j*, *k*, *immutable=False*)

Return an I-graph with $2n$ nodes.

The I-Graph family as been proposed in [BCMS1988] as a generalization of the generalized Petersen graphs. The variables n, j, k are integers such that $n > 2$ and $0 < j, k \leq \lfloor (n-1)/2 \rfloor$. When $j = 1$ the resulting graph is isomorphic to the generalized Petersen graph with the same n and k.

INPUT:

- `n` – the number of nodes is $2 * n$
- `j` – integer such that $0 < j \leq \lfloor (n-1)/2 \rfloor$ determining how outer vertices are connected
- `k` – integer such that $0 < k \leq \lfloor (n-1)/2 \rfloor$ determining how inner vertices are connected
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

PLOTTING: Upon construction, the position dictionary is filled to override the spring-layout algorithm. By convention, the I-graphs are displayed as an inner and outer cycle pair, with the first n nodes drawn on the outer circle. The first (0) node is drawn at the top of the outer-circle, moving counterclockwise after that. The inner circle is drawn with the (n)th node at the top, then counterclockwise as well.

EXAMPLES:

When $j = 1$ the resulting graph will be isomorphic to a generalized Petersen graph:

```
sage: g = graphs.IGraph(7,1,2)
sage: g2 = graphs.GeneralizedPetersenGraph(7,2)
sage: g.is_isomorphic(g2)
True
```

```
>>> from sage.all import *
>>> g = graphs.IGraph(Integer(7),Integer(1),Integer(2))
>>> g2 = graphs.GeneralizedPetersenGraph(Integer(7),Integer(2))
>>> g.is_isomorphic(g2)
True
```

The IGraph with parameters (n, j, k) is isomorphic to the IGraph with parameters (n, k, j):

```
sage: g = graphs.IGraph(7, 2, 3)
sage: h = graphs.IGraph(7, 3, 2)
sage: g.is_isomorphic(h)
True
```

```
>>> from sage.all import *
>>> g = graphs.IGraph(Integer(7), Integer(2), Integer(3))
>>> h = graphs.IGraph(Integer(7), Integer(3), Integer(2))
>>> g.is_isomorphic(h)
True
```

static IcosahedralGraph(*immutable=False*)

Return an Icosahedral graph (with 12 nodes).

The regular icosahedron is a 20-sided triangular polyhedron. The icosahedral graph corresponds to the connectivity of the vertices of the icosahedron. It is dual to the dodecahedral graph. The icosahedron is symmetric, so the spring-layout algorithm will be very effective for display.

PLOTTING: The Icosahedral graph should be viewed in 3 dimensions. We choose to use a planar embedding of the graph. We hope to add rotatable, 3-dimensional viewing in the future. In such a case, an argument will be added to select the desired layout.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

Construct and show an Octahedral graph:

```
sage: g = graphs.IcosahedralGraph()
sage: g.show()                              # long time                    ␣
↪  # needs sage.plot
```

```
>>> from sage.all import *
>>> g = graphs.IcosahedralGraph()
>>> g.show()                              # long time
↪# needs sage.plot
```

Create several icosahedral graphs in a Sage graphics array. They will be drawn differently due to the use of the spring-layout algorithm:

```
sage: g = []
sage: j = []
sage: for i in range(9):
....:     k = graphs.IcosahedralGraph()
....:     g.append(k)
sage: for i in range(3):
....:     n = []
....:     for m in range(3):
....:         n.append(g[3*i + m].plot(vertex_size=50, vertex_labels=False))
....:     j.append(n)
sage: G = graphics_array(j)
sage: G.show()                              # long time
```

```
>>> from sage.all import *
>>> g = []
>>> j = []
>>> for i in range(Integer(9)):
...     k = graphs.IcosahedralGraph()
...     g.append(k)
>>> for i in range(Integer(3)):
...     n = []
...     for m in range(Integer(3)):
...         n.append(g[Integer(3)*i + m].plot(vertex_size=Integer(50), vertex_
↪labels=False))
...     j.append(n)
>>> G = graphics_array(j)
>>> G.show()                              # long time
```

static IntersectionGraph(*S*, *immutable=False*)

Return the intersection graph of the family S.

The intersection graph of a family S is a graph G with $V(G) = S$ such that two elements $s_1, s_2 \in S$ are adjacent in G if and only if $s_1 \cap s_2 \neq \emptyset$.

INPUT:

- `S` – list of sets/tuples/iterables

> **Note**
>
> The elements of S must be finite, hashable, and the elements of any $s \in S$ must be hashable too.

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: graphs.IntersectionGraph([(1,2,3),(3,4,5),(5,6,7)])
Intersection Graph: Graph on 3 vertices
```

```
>>> from sage.all import *
>>> graphs.IntersectionGraph([(Integer(1),Integer(2),Integer(3)),(Integer(3),
↪Integer(4),Integer(5)),(Integer(5),Integer(6),Integer(7))])
Intersection Graph: Graph on 3 vertices
```

static IntervalGraph(*intervals*, *points_ordered=False*, *immutable=False*)

Return the graph corresponding to the given intervals.

An interval graph is built from a list $(a_i, b_i)_{1 \leq i \leq n}$ of intervals : to each interval of the list is associated one vertex, two vertices being adjacent if the two corresponding (closed) intervals intersect.

INPUT:

- `intervals` – the list of pairs (a_i, b_i) defining the graph
- `points_ordered` – states whether every interval (a_i, b_i) of $intervals$ satisfies $a_i < b_i$. If satisfied then setting `points_ordered` to `True` will speed up the creation of the graph.
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

> **Note**
>
> - The vertices are named 0, 1, 2, and so on. The intervals used to create the graph are saved with the graph and can be recovered using `get_vertex()` or `get_vertices()`.

EXAMPLES:

The following line creates the sequence of intervals $(i, i+2)$ for i in $[0, ..., 8]$:

```
sage: intervals = [(i,i+2) for i in range(9)]
```

```
>>> from sage.all import *
>>> intervals = [(i,i+Integer(2)) for i in range(Integer(9))]
```

In the corresponding graph

```
sage: g = graphs.IntervalGraph(intervals)
sage: g.get_vertex(3)
(3, 5)
sage: neigh = g.neighbors(3)
sage: for v in neigh: print(g.get_vertex(v))
(1, 3)
(2, 4)
(4, 6)
(5, 7)
```

```
>>> from sage.all import *
>>> g = graphs.IntervalGraph(intervals)
>>> g.get_vertex(Integer(3))
(3, 5)
>>> neigh = g.neighbors(Integer(3))
>>> for v in neigh: print(g.get_vertex(v))
(1, 3)
(2, 4)
(4, 6)
(5, 7)
```

The is_interval() method verifies that this graph is an interval graph.

```
sage: g.is_interval()
True
```

```
>>> from sage.all import *
>>> g.is_interval()
True
```

The intervals in the list need not be distinct.

```
sage: intervals = [ (1,2), (1,2), (1,2), (2,3), (3,4) ]
sage: g = graphs.IntervalGraph(intervals,True)
sage: g.clique_maximum()
[0, 1, 2, 3]
sage: g.get_vertices()
{0: (1, 2), 1: (1, 2), 2: (1, 2), 3: (2, 3), 4: (3, 4)}
```

```
>>> from sage.all import *
>>> intervals = [ (Integer(1),Integer(2)), (Integer(1),Integer(2)),␣
↪(Integer(1),Integer(2)), (Integer(2),Integer(3)), (Integer(3),Integer(4)) ]
>>> g = graphs.IntervalGraph(intervals,True)
>>> g.clique_maximum()
[0, 1, 2, 3]
>>> g.get_vertices()
{0: (1, 2), 1: (1, 2), 2: (1, 2), 3: (2, 3), 4: (3, 4)}
```

The endpoints of the intervals are not ordered we get the same graph (except for the vertex labels).

```
sage: rev_intervals = [ (2,1), (2,1), (2,1), (3,2), (4,3) ]
sage: h = graphs.IntervalGraph(rev_intervals,False)
```

(continues on next page)

(continued from previous page)

```
sage: h.get_vertices()
{0: (2, 1), 1: (2, 1), 2: (2, 1), 3: (3, 2), 4: (4, 3)}
sage: g.edges(sort=True) == h.edges(sort=True)
True
```

```
>>> from sage.all import *
>>> rev_intervals = [ (Integer(2),Integer(1)), (Integer(2),Integer(1)),␣
↪(Integer(2),Integer(1)), (Integer(3),Integer(2)), (Integer(4),Integer(3)) ]
>>> h = graphs.IntervalGraph(rev_intervals,False)
>>> h.get_vertices()
{0: (2, 1), 1: (2, 1), 2: (2, 1), 3: (3, 2), 4: (4, 3)}
>>> g.edges(sort=True) == h.edges(sort=True)
True
```

static IoninKharaghani765Graph(*immutable=False*)

Return a $(765, 192, 48, 48)$-strongly regular graph.

Existence of a strongly regular graph with these parameters was claimed in [IK2003]. Implementing the construction in the latter did not work, however. This function implements the following instructions, shared by Yury Ionin and Hadi Kharaghani.

Let A be the affine plane over the field $GF(3) = \{-1, 0, 1\}$. Let

$$\begin{aligned}\phi_1(x, y) &= x\\ \phi_2(x, y) &= y\\ \phi_3(x, y) &= x + y\\ \phi_4(x, y) &= x - y\end{aligned}$$

For $i = 1, 2, 3, 4$ and $j \in GF(3)$, let $L_{i,j}$ be the line in A defined by $\phi_i(x, y) = j$. Let $\mathcal{M}$ be the set of all 12 lines $L_{i,j}$, plus the empty set. Let π be the permutation defined on $\mathcal{M}$ by $\pi(L_{i,j}) = L_{i,j+1}$ and $\pi(\emptyset) = \emptyset$, so that π has three orbits of cardinality 3 and one of cardinality 1.

Let $A = (p_1, ..., p_9)$ with $p_1 = (-1, 1)$, $p_2 = (-1, 0)$, $p_3 = (-1, 1)$, $p_4 = (0, -1)$, $p_5 = (0, 0)$, $p_6 = (0, 1)$, $p_7 = (1, -1)$, $p_8 = (1, 0)$, $p_9 = (1, 1)$. Note that $p_i + p_{10-i} = (0, 0)$. For any subset X of A, let $M(X)$ be the $(0, 1)$-matrix of order 9 whose (i, j)-entry equals 1 if and only if $p_{10-i} - p_j \in X$. Note that M is a symmetric matrix.

An MF-tuple is an ordered quintuple $(X_1, X_2, X_3, X_4, X_5)$ of subsets of A, of which one is the empty set and the other four are pairwise non-parallel lines. Such a quintuple generates the following block matrix:

$$N(X_1, X_2, X_3, X_4, X_5) = \begin{pmatrix} M(X_1) & M(X_2) & M(X_3) & M(X_4) & M(X_5)\\ M(X_2) & M(X_3) & M(X_4) & M(X_5) & M(X_1)\\ M(X_3) & M(X_4) & M(X_5) & M(X_1) & M(X_2)\\ M(X_4) & M(X_5) & M(X_1) & M(X_2) & M(X_3)\\ M(X_5) & M(X_1) & M(X_2) & M(X_3) & M(X_4)\end{pmatrix}$$

Observe that if $(X_1, X_2, X_3, X_4, X_5)$ is an MF-tuple, then $N(X_1, X_2, X_3, X_4, X_5)$ is the symmetric incidence matrix of a symmetric $(45, 12, 3)$-design.

Let $\mathcal{F}$ be the set of all MF-tuples and let σ be the following permutation of $\mathcal{F}$:

$$\begin{aligned}\sigma(X_1, X_2, X_3, X_4, X_5) &= (X_2, X_3, X_4, X_5, X_1)\\ \pi(X_1, X_2, X_3, X_4, X_5) &= (\pi(X_1), \pi(X_2), \pi(X_3), \pi(X_4), \pi(X_5))\end{aligned}$$

Observe that σ and π commute, and generate a (cyclic) group G of order 15. We will from now on identify G with the (cyclic) multiplicative group of the field $GF(16)$ equal to $\{\omega^0, ..., \omega^{14}\}$. Let $W = [w_{ij}]$ be the following matrix of order 17 over $GF(16) = \{a_1, ..., a_16\}$:

$$w_{ij} = \begin{cases} a_i + a_j & \text{if } 1 \leq i \leq 16, 1 \leq j \leq 16, \\ 1 & \text{if } i = 17, j \neq 17, \\ 1 & \text{if } i \neq 17, j = 17, \\ 0 & \text{if } i = j = 17 \end{cases}$$

The diagonal entries of W are equal to 0, each off-diagonal entry can be represented as ω^k with $0 \leq k \leq 14$. Matrix W is a symmetric $BGW(17, 16, 15; G)$.

Fix an MF-tuple $(X_1, X_2, X_3, X_4, X_5)$ and let S be the block matrix obtained from W by replacing every diagonal entry of W by the zero matrix of order 45, and every off-diagonal entry ω^k by the matrix $N(\sigma^k(X_1, X_2, X_3, X_4, X_5))$ (through the association of ω^k with an element of G). Then S is a symmetric incidence matrix of a symmetric $(765, 192, 48)$-design with zero diagonal, and therefore S is an adjacency matrix of a strongly regular graph with parameters $(765, 192, 48, 48)$.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.IoninKharaghani765Graph(); g                                  ␣
↪  # needs sage.modules sage.rings.finite_rings
Ionin-Kharaghani: Graph on 765 vertices
```

```
>>> from sage.all import *
>>> g = graphs.IoninKharaghani765Graph(); g
↪# needs sage.modules sage.rings.finite_rings
Ionin-Kharaghani: Graph on 765 vertices
```

Todo

An update to [IK2003] meant to fix the problem encountered became available 2016/02/24, see http://www.cs.uleth.ca/~hadi/research/IoninKharaghani.pdf

static IvanovIvanovFaradjevGraph(*immutable=False*)

Return the IvanovIvanovFaradjev graph.

The graph is distance-transitive with automorphism group $3.M_{22}$.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: G = graphs.IvanovIvanovFaradjevGraph()             # optional - internet␣
↪gap_package_atlasrep
sage: G.is_distance_regular(True)                        # optional - internet␣
↪gap_package_atlasrep
([7, 6, 4, 4, 4, 1, 1, 1, None], [None, 1, 1, 1, 2, 4, 4, 6, 7])
```

```
>>> from sage.all import *
>>> G = graphs.IvanovIvanovFaradjevGraph()             # optional - internet␣
↪gap_package_atlasrep
>>> G.is_distance_regular(True)                        # optional - internet␣
↪gap_package_atlasrep
([7, 6, 4, 4, 4, 1, 1, 1, None], [None, 1, 1, 1, 2, 4, 4, 6, 7])
```

REFERENCES:

A description and construction of this graph can be found in [BCN1989] p. 369.

static J2Graph(*immutable=False*)

Return the distance-transitive graph with automorphism group J_2.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: G = graphs.J2Graph()                              # optional - internet␣
↪gap_package_atlasrep
sage: G.is_distance_regular(True)                       # optional - internet␣
↪gap_package_atlasrep
([10, 8, 8, 2, None], [None, 1, 1, 4, 5])
```

```
>>> from sage.all import *
>>> G = graphs.J2Graph()                               # optional - internet␣
↪gap_package_atlasrep
>>> G.is_distance_regular(True)                        # optional - internet␣
↪gap_package_atlasrep
([10, 8, 8, 2, None], [None, 1, 1, 4, 5])
```

REFERENCES:

A description and construction of this graph can be found in [BCN1989] p. 408.

static JankoKharaghaniGraph(*v*, *immutable=False*)

Return a $(936, 375, 150, 150)$-srg or a $(1800, 1029, 588, 588)$-srg.

This functions returns a strongly regular graph for the two sets of parameters shown to be realizable in [JK2002]. The paper also uses a construction from [GM1987].

INPUT:

- `v` – integer; one of 936 or 1800
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.JankoKharaghaniGraph(936)      # long time                ␣
↪  # needs sage.libs.pari
sage: g.is_strongly_regular(parameters=True)    # long time                ␣
↪  # needs sage.libs.pari
(936, 375, 150, 150)

sage: g = graphs.JankoKharaghaniGraph(1800)     # not tested (30s)
```

(continues on next page)

(continued from previous page)

```
sage: g.is_strongly_regular(parameters=True)     # not tested (30s)
(1800, 1029, 588, 588)
```

```
>>> from sage.all import *
>>> g = graphs.JankoKharaghaniGraph(Integer(936))       # long time
↪          # needs sage.libs.pari
>>> g.is_strongly_regular(parameters=True)     # long time
↪# needs sage.libs.pari
(936, 375, 150, 150)

>>> g = graphs.JankoKharaghaniGraph(Integer(1800))      # not tested (30s)
>>> g.is_strongly_regular(parameters=True)     # not tested (30s)
(1800, 1029, 588, 588)
```

static JankoKharaghaniTonchevGraph(*immutable=False*)

Return a $(324, 153, 72, 72)$-strongly regular graph from [JKT2001].

Build the graph using the description given in [JKT2001], taking sets B1 and B163 in the text as adjacencies of vertices 1 and 163, respectively, and taking the edge orbits of the group G provided.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: Gamma = graphs.JankoKharaghaniTonchevGraph()       # long time
↪  # needs sage.libs.gap
sage: Gamma.is_strongly_regular(parameters=True)         # long time
↪  # needs sage.libs.gap
(324, 153, 72, 72)
```

```
>>> from sage.all import *
>>> Gamma = graphs.JankoKharaghaniTonchevGraph()       # long time
↪# needs sage.libs.gap
>>> Gamma.is_strongly_regular(parameters=True)         # long time
↪# needs sage.libs.gap
(324, 153, 72, 72)
```

static JohnsonGraph(n, k, *immutable=False*)

Return the Johnson graph with parameters n, k.

Johnson graphs are a special class of undirected graphs defined from systems of sets. The vertices of the Johnson graph $J(n, k)$ are the k-element subsets of an n-element set; two vertices are adjacent when they meet in a $(k-1)$-element set. See the Wikipedia article Johnson_graph for more information.

INPUT:

- `n` – nonnegative integer; number of elements of the groundset
- `k` – nonnegative integer; number of elements of each subset
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

The Johnson graph is a Hamiltonian graph:

```
sage: g = graphs.JohnsonGraph(7, 3)
sage: g.is_hamiltonian()                                                    ␣
↪  # needs sage.numerical.mip
True
```

```
>>> from sage.all import *
>>> g = graphs.JohnsonGraph(Integer(7), Integer(3))
>>> g.is_hamiltonian()
↪# needs sage.numerical.mip
True
```

Every Johnson graph is vertex transitive:

```
sage: g = graphs.JohnsonGraph(6, 4)
sage: g.is_vertex_transitive()                                              ␣
↪  # needs sage.groups
True
```

```
>>> from sage.all import *
>>> g = graphs.JohnsonGraph(Integer(6), Integer(4))
>>> g.is_vertex_transitive()
↪# needs sage.groups
True
```

The complement of the Johnson graph $J(n, 2)$ is isomorphic to the Kneser Graph $K(n, 2)$. In particular the complement of $J(5, 2)$ is isomorphic to the Petersen graph.:

```
sage: g = graphs.JohnsonGraph(5,2)
sage: g.complement().is_isomorphic(graphs.PetersenGraph())
True
```

```
>>> from sage.all import *
>>> g = graphs.JohnsonGraph(Integer(5),Integer(2))
>>> g.complement().is_isomorphic(graphs.PetersenGraph())
True
```

static KingGraph(*dim_list*, *radius=None*, *relabel=False*, *immutable=False*)

Return the d-dimensional King Graph with prescribed dimensions.

The 2-dimensional King Graph of parameters n and m is a graph with nm vertices in which each vertex represents a square in an $n \times m$ chessboard, and each edge corresponds to a legal move by a king.

The d-dimensional King Graph with $d >= 2$ has for vertex set the cells of a d-dimensional grid with prescribed dimensions, and each edge corresponds to a legal move by a king in either one or two dimensions.

All 2-dimensional King Graphs are Hamiltonian, biconnected, and have chromatic number 4 as soon as both dimensions are larger or equal to 2.

INPUT:

- `dim_list` – iterable (list, set, dict); provides the dimensions $n_1, n_2, \ldots, n_d$, with $n_i \geq 1$, of the chessboard
- `radius` – integer (default: `None`); by setting the radius to a positive integer, one may increase the power of the king to at least `radius` steps. When the radius equals the higher size of the dimensions, the resulting graph is a Queen Graph.

- `relabel` – boolean (default: `False`); indicates whether the vertices must be relabeled as integers
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

The $(2, 2)$-King Graph is isomorphic to the complete graph on 4 vertices:

```
sage: G = graphs.QueenGraph( [2, 2] )
sage: G.is_isomorphic( graphs.CompleteGraph(4) )
True
```

```
>>> from sage.all import *
>>> G = graphs.QueenGraph( [Integer(2), Integer(2)] )
>>> G.is_isomorphic( graphs.CompleteGraph(Integer(4)) )
True
```

The King Graph with large enough radius is isomorphic to a Queen Graph:

```
sage: G = graphs.KingGraph( [5, 4], radius=5 )
sage: H = graphs.QueenGraph( [4, 5] )
sage: G.is_isomorphic( H )
True
```

```
>>> from sage.all import *
>>> G = graphs.KingGraph( [Integer(5), Integer(4)], radius=Integer(5) )
>>> H = graphs.QueenGraph( [Integer(4), Integer(5)] )
>>> G.is_isomorphic( H )
True
```

Also `True` in higher dimensions:

```
sage: G = graphs.KingGraph( [2, 5, 4], radius=5 )
sage: H = graphs.QueenGraph( [4, 5, 2] )
sage: G.is_isomorphic( H )
True
```

```
>>> from sage.all import *
>>> G = graphs.KingGraph( [Integer(2), Integer(5), Integer(4)],␣
↪radius=Integer(5) )
>>> H = graphs.QueenGraph( [Integer(4), Integer(5), Integer(2)] )
>>> G.is_isomorphic( H )
True
```

static KittellGraph(*immutable=False*)

Return the Kittell Graph.

For more information, see the Wolfram page about the Kittel Graph.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.KittellGraph()
sage: g.order()
23
sage: g.size()
63
sage: g.radius()
3
sage: g.diameter()
4
sage: g.girth()
3
sage: g.chromatic_number()
4
```

```
>>> from sage.all import *
>>> g = graphs.KittellGraph()
>>> g.order()
23
>>> g.size()
63
>>> g.radius()
3
>>> g.diameter()
4
>>> g.girth()
3
>>> g.chromatic_number()
4
```

static Klein3RegularGraph(*immutable=False*)

Return the Klein 3-regular graph.

The cubic Klein graph has 56 vertices and can be embedded on a surface of genus 3. It is the dual of *Klein7RegularGraph()*. For more information, see the Wikipedia article Klein_graphs.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.Klein3RegularGraph(); g
Klein 3-regular Graph: Graph on 56 vertices
sage: g.order(), g.size()
(56, 84)
sage: g.girth()
7
sage: g.automorphism_group().cardinality()
↪  # needs sage.groups
336
sage: g.chromatic_number()
3
```

```
>>> from sage.all import *
>>> g = graphs.Klein3RegularGraph(); g
Klein 3-regular Graph: Graph on 56 vertices
>>> g.order(), g.size()
(56, 84)
>>> g.girth()
7
>>> g.automorphism_group().cardinality()
↪# needs sage.groups
336
>>> g.chromatic_number()
3
```

static Klein7RegularGraph(*immutable=False*)

Return the Klein 7-regular graph.

The 7-valent Klein graph has 24 vertices and can be embedded on a surface of genus 3. It is the dual of *Klein3RegularGraph()*. For more information, see the Wikipedia article Klein_graphs.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.Klein7RegularGraph(); g
Klein 7-regular Graph: Graph on 24 vertices
sage: g.order(), g.size()
(24, 84)
sage: g.girth()
3
sage: g.automorphism_group().cardinality()
↪  # needs sage.groups
336
sage: g.chromatic_number()
4
```

```
>>> from sage.all import *
>>> g = graphs.Klein7RegularGraph(); g
Klein 7-regular Graph: Graph on 24 vertices
>>> g.order(), g.size()
(24, 84)
>>> g.girth()
3
>>> g.automorphism_group().cardinality()
↪# needs sage.groups
336
>>> g.chromatic_number()
4
```

static KneserGraph(*n*, *k*, *immutable=False*, *name=None*)

Return the Kneser Graph with parameters n, k.

The Kneser Graph with parameters n, k is the graph whose vertices are the k-subsets of $[0, 1, \ldots, n-1]$, and such that two vertices are adjacent if their corresponding sets are disjoint.

For example, the Petersen Graph can be defined as the Kneser Graph with parameters $5, 2$.

INPUT:

- `n` – positive integer; number of elements of the groundset
- `k` – positive integer; number of elements of each subset
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph
- `name` – string (default: `None`); used as the name of the returned graph when set

EXAMPLES:

```
sage: KG = graphs.KneserGraph(5,2)
sage: sorted(KG.vertex_iterator(), key=str)
[{1, 2}, {1, 3}, {1, 4}, {1, 5}, {2, 3}, {2, 4}, {2, 5},
 {3, 4}, {3, 5}, {4, 5}]
sage: P = graphs.PetersenGraph()
sage: P.is_isomorphic(KG)
True
```

```
>>> from sage.all import *
>>> KG = graphs.KneserGraph(Integer(5),Integer(2))
>>> sorted(KG.vertex_iterator(), key=str)
[{1, 2}, {1, 3}, {1, 4}, {1, 5}, {2, 3}, {2, 4}, {2, 5},
 {3, 4}, {3, 5}, {4, 5}]
>>> P = graphs.PetersenGraph()
>>> P.is_isomorphic(KG)
True
```

static KnightGraph(*dim_list*, *one=1*, *two=2*, *relabel=False*, *immutable=False*)

Return the d-dimensional Knight Graph with prescribed dimensions.

The 2-dimensional Knight Graph of parameters n and m is a graph with nm vertices in which each vertex represents a square in an $n \times m$ chessboard, and each edge corresponds to a legal move by a knight.

The d-dimensional Knight Graph with $d >= 2$ has for vertex set the cells of a d-dimensional grid with prescribed dimensions, and each edge corresponds to a legal move by a knight in any pairs of dimensions.

The (n, n)-Knight Graph is Hamiltonian for even $n > 4$.

INPUT:

- `dim_list` – iterable (list, set, dict); provides the dimensions $n_1, n_2, \ldots, n_d$, with $n_i \geq 1$, of the chessboard
- `one` – integer (default: 1); indicates the number of steps in the first dimension
- `two` – integer (default: 2); indicates the number of steps in the second dimension
- `relabel` – boolean (default: `False`); indicates whether the vertices must be relabeled as integers
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

The $(3, 3)$-Knight Graph has an isolated vertex:

```
sage: G = graphs.KnightGraph( [3, 3] )
sage: G.degree( (1,1) )
0
```

```
>>> from sage.all import *
>>> G = graphs.KnightGraph( [Integer(3), Integer(3)] )
>>> G.degree( (Integer(1),Integer(1)) )
0
```

The $(3, 3)$-Knight Graph minus vertex (1,1) is a cycle of order 8:

```
sage: G = graphs.KnightGraph( [3, 3] )
sage: G.delete_vertex( (1,1) )
sage: G.is_isomorphic( graphs.CycleGraph(8) )
True
```

```
>>> from sage.all import *
>>> G = graphs.KnightGraph( [Integer(3), Integer(3)] )
>>> G.delete_vertex( (Integer(1),Integer(1)) )
>>> G.is_isomorphic( graphs.CycleGraph(Integer(8)) )
True
```

The $(6, 6)$-Knight Graph is Hamiltonian:

```
sage: G = graphs.KnightGraph( [6, 6] )
sage: G.is_hamiltonian()
↪  # needs sage.numerical.mip
True
```

```
>>> from sage.all import *
>>> G = graphs.KnightGraph( [Integer(6), Integer(6)] )
>>> G.is_hamiltonian()
↪# needs sage.numerical.mip
True
```

static KrackhardtKiteGraph(*immutable=False*)

Return a Krackhardt kite graph with 10 nodes.

The Krackhardt kite graph was originally developed by David Krackhardt for the purpose of studying social networks (see [Kre2002] and the Wikipedia article Krackhardt_kite_graph). It is used to show the distinction between degree centrality, betweenness centrality, and closeness centrality. For more information read the plotting section below in conjunction with the example.

PLOTTING: Upon construction, the position dictionary is filled to override the spring-layout algorithm. By convention, the graph is drawn left to right, in top to bottom row sequence of [2, 3, 2, 1, 1, 1] nodes on each row. This places the fourth node (3) in the center of the kite, with the highest degree. But the fourth node only connects nodes that are otherwise connected, or those in its clique (i.e.: Degree Centrality). The eighth (7) node is where the kite meets the tail. It has degree = 3, less than the average, but is the only connection between the kite and tail (i.e.: Betweenness Centrality). The sixth and seventh nodes (5 and 6) are drawn in the third row and have degree = 5. These nodes have the shortest path to all other nodes in the graph (i.e.: Closeness Centrality). Please execute the example for visualization.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

Construct and show a Krackhardt kite graph

```
sage: g = graphs.KrackhardtKiteGraph()
sage: g.show()                              # long time                    ␣
↪  # needs sage.plot
```

```
>>> from sage.all import *
>>> g = graphs.KrackhardtKiteGraph()
>>> g.show()                              # long time
↪# needs sage.plot
```

static LCFGraph(*n*, *shift_list*, *repeats*, *immutable=False*, *name=None*)

Return the cubic graph specified in LCF notation.

LCF (Lederberg-Coxeter-Fruchte) notation is a concise way of describing cubic Hamiltonian graphs. The way a graph is constructed is as follows. Since there is a Hamiltonian cycle, we first create a cycle on n nodes. The variable `shift_list` = $[s_0, s_1, ..., s_{k-1}]$ describes edges to be created by the following scheme: for each $i \in \{0, 1, \ldots, k-1\}$, connect vertex i to vertex $(i + s_i) \pmod n$. Then, `repeats` specifies the number of times to repeat this process, where on the j-th repeat we connect vertex $(i + jk) \pmod n$ to vertex $(i + jk + s_i) \pmod n$.

For more details, see the Wikipedia article LCF_notation and [Fru1977], [Gru2003] pp. 357-365, and [Led1965].

INPUT:

- `n` – the number of nodes
- `shift_list` – a list of integer shifts mod n
- `repeats` – the number of times to repeat the process
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph
- `name` – string (default: `None`); used as the name of the returned graph when set

EXAMPLES:

```
sage: G = graphs.LCFGraph(4, [2,-2], 2)
sage: G.is_isomorphic(graphs.TetrahedralGraph())
True
```

```
>>> from sage.all import *
>>> G = graphs.LCFGraph(Integer(4), [Integer(2),-Integer(2)], Integer(2))
>>> G.is_isomorphic(graphs.TetrahedralGraph())
True
```

```
sage: G = graphs.LCFGraph(20, [10,7,4,-4,-7,10,-4,7,-7,4], 2)
sage: G.is_isomorphic(graphs.DodecahedralGraph())
True
```

```
>>> from sage.all import *
>>> G = graphs.LCFGraph(Integer(20), [Integer(10),Integer(7),Integer(4),-
↪Integer(4),-Integer(7),Integer(10),-Integer(4),Integer(7),-Integer(7),
↪Integer(4)], Integer(2))
>>> G.is_isomorphic(graphs.DodecahedralGraph())
True
```

```
sage: G = graphs.LCFGraph(14, [5,-5], 7)
sage: G.is_isomorphic(graphs.HeawoodGraph())
True
```

```
>>> from sage.all import *
>>> G = graphs.LCFGraph(Integer(14), [Integer(5),-Integer(5)], Integer(7))
>>> G.is_isomorphic(graphs.HeawoodGraph())
True
```

The largest cubic nonplanar graph of diameter three:

```
sage: G = graphs.LCFGraph(20, [-10,-7,-5,4,7,-10,-7,-4,5,7,
....:                          -10,-7,6,-5,7,-10,-7,5,-6,7], 1)
sage: G.degree()
[3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3]
sage: G.diameter()
3
sage: G.show()                              # long time
↪  # needs sage.plot
```

```
>>> from sage.all import *
>>> G = graphs.LCFGraph(Integer(20), [-Integer(10),-Integer(7),-Integer(5),
↪Integer(4),Integer(7),-Integer(10),-Integer(7),-Integer(4),Integer(5),
↪Integer(7),
...                           -Integer(10),-Integer(7),Integer(6),-Integer(5),
↪Integer(7),-Integer(10),-Integer(7),Integer(5),-Integer(6),Integer(7)],␣
↪Integer(1))
>>> G.degree()
[3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3]
>>> G.diameter()
3
>>> G.show()                               # long time
↪# needs sage.plot
```

PLOTTING: LCF Graphs are plotted as an n-cycle with edges in the middle, as described above.

static LUWGraph(*ring*, *equations*, *name=None*, *A=None*, *B=None*, *point_coordinate_sets=None*, *line_coordinate_sets=None*, *immutable=False*)

Build a graph directly from a given set of equations.

This is the graph-returning constructor used by `graphs.LUWGraph`. The construction is surveyed in [LW2026].

INPUT:

- `ring` – finite commutative ring
- `equations` – nonempty iterable of Sage polynomials
- `name` – string (default: `None`); graph name
- `A` – iterable, single value, or `None` (default: `None`); allowed first coordinates on the point side
- `B` – iterable, single value, or `None` (default: `None`); allowed first coordinates on the line side
- `point_coordinate_sets` – iterable of coordinate sets or `None` (default: `None`)
- `line_coordinate_sets` – iterable of coordinate sets or `None` (default: `None`)

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

OUTPUT:

A Sage *Graph*.

EXAMPLES:

```
sage: F = GF(3)
sage: R = PolynomialRing(F, names=("p1", "l1", "p2", "l2", "p3", "l3"))
sage: p1, l1, p2, l2, p3, l3 = R.gens()
sage: G = graphs.LUWGraph(F, [p1*l1, p1*l2, p3*l1])
sage: G.order(), G.size()
(162, 243)
```

```
>>> from sage.all import *
>>> F = GF(Integer(3))
>>> R = PolynomialRing(F, names=("p1", "l1", "p2", "l2", "p3", "l3"))
>>> p1, l1, p2, l2, p3, l3 = R.gens()
>>> G = graphs.LUWGraph(F, [p1*l1, p1*l2, p3*l1])
>>> G.order(), G.size()
(162, 243)
```

REFERENCES:

- [LW2026]

static LadderGraph(*n*, *immutable=False*)

Return a ladder graph with $2 * n$ nodes.

A ladder graph is a basic structure that is typically displayed as a ladder, i.e.: two parallel path graphs connected at each corresponding node pair.

PLOTTING: Upon construction, the position dictionary is filled to override the spring-layout algorithm. By convention, each ladder graph will be displayed horizontally, with the first n nodes displayed left to right on the top horizontal line.

INPUT:

- `n` – a nonnegative integer; number of nodes is $2n$
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

Construct and show a ladder graph with 14 nodes:

```
sage: g = graphs.LadderGraph(7)
sage: g.show()                              # long time
↪  # needs sage.plot
```

```
>>> from sage.all import *
>>> g = graphs.LadderGraph(Integer(7))
>>> g.show()                              # long time
↪# needs sage.plot
```

Create several ladder graphs in a Sage graphics array:

```
sage: g = []
sage: j = []
sage: for i in range(9):
....:     k = graphs.LadderGraph(i+2)
....:     g.append(k)
sage: for i in range(3):
....:     n = []
....:     for m in range(3):
....:         n.append(g[3*i + m].plot(vertex_size=50, vertex_labels=False))
....:     j.append(n)
sage: G = graphics_array(j)
sage: G.show()                          # long time
```

```
>>> from sage.all import *
>>> g = []
>>> j = []
>>> for i in range(Integer(9)):
...     k = graphs.LadderGraph(i+Integer(2))
...     g.append(k)
>>> for i in range(Integer(3)):
...     n = []
...     for m in range(Integer(3)):
...         n.append(g[Integer(3)*i + m].plot(vertex_size=Integer(50), vertex_
↪labels=False))
...     j.append(n)
>>> G = graphics_array(j)
>>> G.show()                          # long time
```

static LargeWittGraph(*immutable=False*)

Return the large Witt graph.

This is a distance-regular graph with intersection array $[30, 28, 24; 1, 3, 15]$.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.LargeWittGraph()                                              ␣
↪  # needs sage.libs.pari sage.modules
sage: g.is_distance_regular(True)                                              ␣
↪  # needs sage.libs.pari sage.modules
([30, 28, 24, None], [None, 1, 3, 15])
```

```
>>> from sage.all import *
>>> g = graphs.LargeWittGraph()
↪# needs sage.libs.pari sage.modules
>>> g.is_distance_regular(True)
↪# needs sage.libs.pari sage.modules
([30, 28, 24, None], [None, 1, 3, 15])
```

REFERENCES:

A description of this graph can be found in [BCN1989] p. 366. This construction is taken from http://

mathworld.wolfram.com/LargeWittGraph.html

static LeonardGraph(*immutable=False*)

Return the Leonard graph.

The graph is distance-regular with intersection array $[12, 11, 10, 7; 1, 2, 5, 12]$.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: G = graphs.LeonardGraph()
↪ # needs sage.combinat sage.modules
sage: G.is_distance_regular(True)
↪ # needs sage.combinat sage.modules
([12, 11, 10, 7, None], [None, 1, 2, 5, 12])
```

```
>>> from sage.all import *
>>> G = graphs.LeonardGraph()
↪# needs sage.combinat sage.modules
>>> G.is_distance_regular(True)
↪# needs sage.combinat sage.modules
([12, 11, 10, 7, None], [None, 1, 2, 5, 12])
```

REFERENCES:

For a description of this graph see [BCN1989] p. 371.

static LivingstoneGraph(*immutable=False*)

Return the Livingstone Graph.

The Livingstone graph is a distance-transitive graph on 266 vertices whose automorphism group is the `J1 group`. For more information, see the Wikipedia article Livingstone_graph.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: # optional - internet
sage: g = graphs.LivingstoneGraph()
sage: g.order()
266
sage: g.size()
1463
sage: g.girth()
5
sage: g.is_vertex_transitive()
True
sage: g.is_distance_regular()
True
```

```
>>> from sage.all import *
>>> # optional - internet
>>> g = graphs.LivingstoneGraph()
```

(continues on next page)

(continued from previous page)

```
>>> g.order()
266
>>> g.size()
1463
>>> g.girth()
5
>>> g.is_vertex_transitive()
True
>>> g.is_distance_regular()
True
```

static LjubljanaGraph(*embedding=1*, *immutable=False*)

Return the Ljubljana Graph.

The Ljubljana graph is a bipartite 3-regular graph on 112 vertices and 168 edges. It is not vertex-transitive as it has two orbits which are also independent sets of size 56. See the Wikipedia article Ljubljana_graph.

The default embedding is obtained from the Heawood graph.

INPUT:

- `embedding` – integer (default: 1); two embeddings are available, and can be selected by setting `embedding` to 1 or 2
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.LjubljanaGraph()
sage: g.order()
112
sage: g.size()
168
sage: g.girth()
10
sage: g.diameter()
8
sage: g.show(figsize=[10, 10])          # long time                          ␣
↪  # needs sage.plot
sage: graphs.LjubljanaGraph(embedding=2).show(figsize=[10, 10])          #␣
↪long time, needs sage.plot
```

```
>>> from sage.all import *
>>> g = graphs.LjubljanaGraph()
>>> g.order()
112
>>> g.size()
168
>>> g.girth()
10
>>> g.diameter()
8
>>> g.show(figsize=[Integer(10), Integer(10)])          # long time          ␣
↪                  # needs sage.plot
```

(continues on next page)

(continued from previous page)

```
>>> graphs.LjubljanaGraph(embedding=Integer(2)).show(figsize=[Integer(10),␣
↪Integer(10)])          # long time, needs sage.plot
```

static LocalMcLaughlinGraph(*immutable=False*)

Return the local McLaughlin graph.

The local McLaughlin graph is a strongly regular graph with parameters $(162, 56, 10, 24)$. It can be obtained from *McLaughlinGraph()* by considering the stabilizer of a point: one of its orbits has cardinality 162.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.LocalMcLaughlinGraph(); g      # long time, optional - gap_
↪package_design
Local McLaughlin Graph: Graph on 162 vertices
sage: g.is_strongly_regular(parameters=True)    # long time, optional - gap_
↪package_design
(162, 56, 10, 24)
```

```
>>> from sage.all import *
>>> g = graphs.LocalMcLaughlinGraph(); g      # long time, optional - gap_
↪package_design
Local McLaughlin Graph: Graph on 162 vertices
>>> g.is_strongly_regular(parameters=True)    # long time, optional - gap_
↪package_design
(162, 56, 10, 24)
```

static LollipopGraph(*n1*, *n2*, *immutable=False*)

Return a lollipop graph with $n_1 + n_2$ nodes.

A lollipop graph is a path graph (order n_2) connected to a complete graph (order n_1). (A barbell graph minus one of the bells).

PLOTTING: Upon construction, the position dictionary is filled to override the spring-layout algorithm. By convention, the complete graph will be drawn in the lower-left corner with the n_1-th node at a 45 degree angle above the right horizontal center of the complete graph, leading directly into the path graph.

INPUT:

- `n1` – integer ≥ 0; the order of the complete graph
- `n2` – integer ≥ 0; the order of the path graph
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

Construct and show a lollipop graph Candy = 13, Stick = 4:

```
sage: g = graphs.LollipopGraph(13,4); g
Lollipop graph: Graph on 17 vertices
sage: g.show()                          # long time                          ␣
↪  # needs sage.plot
```

```
>>> from sage.all import *
>>> g = graphs.LollipopGraph(Integer(13),Integer(4)); g
Lollipop graph: Graph on 17 vertices
>>> g.show()                              # long time
↪# needs sage.plot
```

static M22Graph(*immutable=False*)

Return the M22 graph.

The M_{22} graph is the unique strongly regular graph with parameters $v = 77, k = 16, \lambda = 0, \mu = 4$.

For more information on the M_{22} graph, see https://www.win.tue.nl/~aeb/graphs/M22.html.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.M22Graph()
sage: g.order()
77
sage: g.size()
616
sage: g.is_strongly_regular(parameters=True)
(77, 16, 0, 4)
```

```
>>> from sage.all import *
>>> g = graphs.M22Graph()
>>> g.order()
77
>>> g.size()
616
>>> g.is_strongly_regular(parameters=True)
(77, 16, 0, 4)
```

static MarkstroemGraph(*immutable=False*)

Return the Markström Graph.

The Markström Graph is a cubic planar graph with no cycles of length 4 nor 8, but containing cycles of length 16. For more information, see the Wolfram page about the Markström Graph.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.MarkstroemGraph()
sage: g.order()
24
sage: g.size()
36
sage: g.is_planar()
True
sage: g.is_regular(3)
True
```

(continues on next page)

(continued from previous page)

```
sage: g.subgraph_search(graphs.CycleGraph(4)) is None
↪  # needs sage.modules
True
sage: g.subgraph_search(graphs.CycleGraph(8)) is None
↪  # needs sage.modules
True
sage: g.subgraph_search(graphs.CycleGraph(16))
↪  # needs sage.modules
Subgraph of (Markstroem Graph): Graph on 16 vertices
```

```
>>> from sage.all import *
>>> g = graphs.MarkstroemGraph()
>>> g.order()
24
>>> g.size()
36
>>> g.is_planar()
True
>>> g.is_regular(Integer(3))
True
>>> g.subgraph_search(graphs.CycleGraph(Integer(4))) is None
↪          # needs sage.modules
True
>>> g.subgraph_search(graphs.CycleGraph(Integer(8))) is None
↪          # needs sage.modules
True
>>> g.subgraph_search(graphs.CycleGraph(Integer(16)))
↪          # needs sage.modules
Subgraph of (Markstroem Graph): Graph on 16 vertices
```

static MathonPseudocyclicMergingGraph(*M*, *t*, *immutable=False*)

Mathon's merging of classes in a pseudo-cyclic 3-class association scheme.

Construct strongly regular graphs from p.97 of [BL1984].

INPUT:

- `M` – the list of matrices in a pseudo-cyclic 3-class association scheme; the identity matrix must be the first entry
- `t` – integer; the number of the graph, from 0 to 2
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

See also

- *is_muzychuk_S6()*

static MathonPseudocyclicStronglyRegularGraph(*t*, *G=None*, *L=None*, *immutable=False*)

Return a strongly regular graph on $(4t+1)(4t-1)^2$ vertices from [Mat1978].

Let $4t-1$ be a prime power, and $4t+1$ be such that there exists a strongly regular graph G with parameters $(4t+1, 2t, t-1, t)$. In particular, $4t+1$ must be a sum of two squares [Mat1978]. With this input, Mathon [Mat1978] gives a construction of a strongly regular graph with parameters $(4\mu+1, 2\mu, \mu-1, \mu)$, where

$\mu = t(4t(4t-1)-1)$. The construction is optionally parametrised by an a skew-symmetric Latin square of order $4t+1$, with entries in $-2t, ..., -1, 0, 1, ..., 2t$.

Our implementation follows a description given in [ST1981].

INPUT:

- `t` – positive integer
- `G` – if `None` (default), try to construct the necessary graph with parameters $(4t+1, 2t, t-1, t)$, otherwise use the user-supplied one, with vertices labelled from 0 to $4t$.
- `L` – if `None` (default), construct a necessary skew Latin square, otherwise use the user-supplied one. Here non-isomorphic Latin squares – one constructed from $Z/9Z$, and the other from $(Z/3Z)^2$ – lead to non-isomorphic graphs.
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

See also

- *is_mathon_PC_srg()*

EXAMPLES:

Using default `G` and `L`.

```
sage: from sage.graphs.generators.families import␣
↪MathonPseudocyclicStronglyRegularGraph
sage: G = MathonPseudocyclicStronglyRegularGraph(1); G                ␣
↪  # needs sage.modules sage.rings.finite_rings
Mathon's PC SRG on 45 vertices: Graph on 45 vertices
sage: G.is_strongly_regular(parameters=True)                          ␣
↪  # needs sage.modules sage.rings.finite_rings
(45, 22, 10, 11)
```

```
>>> from sage.all import *
>>> from sage.graphs.generators.families import␣
↪MathonPseudocyclicStronglyRegularGraph
>>> G = MathonPseudocyclicStronglyRegularGraph(Integer(1)); G           ␣
↪         # needs sage.modules sage.rings.finite_rings
Mathon's PC SRG on 45 vertices: Graph on 45 vertices
>>> G.is_strongly_regular(parameters=True)
↪# needs sage.modules sage.rings.finite_rings
(45, 22, 10, 11)
```

Supplying `G` and `L` (constructed from the automorphism group of `G`). The entries of L can't be tested directly because there's some unpredictability in the way that GAP chooses a representative in `NormalSubgroups()`, the function that underlies our own `normal_subgroups()` method:

```
sage: G = graphs.PaleyGraph(9)
sage: a = G.automorphism_group(partition=[sorted(G)])
sage: it = (x for x in a.normal_subgroups() if x.order() == 9)
sage: subg = next(iter(it))
sage: r = [matrix(libgap.PermutationMat(libgap(z), 9).sage())
....:      for z in subg]
```

(continues on next page)

(continued from previous page)

```
sage: ff = list(map(lambda y: (y[0]-1,y[1]-1),
....:           Permutation(map(lambda x: 1+r.index(x^-1), r)).cycle_
↪tuples()[1:]))
sage: L = sum(i*(r[a]-r[b]) for i,(a,b) in zip(range(1,len(ff)+1), ff))
sage: G.relabel(range(9))
sage: G3x3 = graphs.MathonPseudocyclicStronglyRegularGraph(2, G=G, L=L)
sage: G3x3.is_strongly_regular(parameters=True)
(441, 220, 109, 110)
sage: G3x3.automorphism_group(algorithm='bliss').order()  # optional - bliss
27
sage: G9 = graphs.MathonPseudocyclicStronglyRegularGraph(2)
sage: G9.is_strongly_regular(parameters=True)
(441, 220, 109, 110)
sage: G9.automorphism_group(algorithm='bliss').order()  # optional - bliss
9
```

```
>>> from sage.all import *
>>> G = graphs.PaleyGraph(Integer(9))
>>> a = G.automorphism_group(partition=[sorted(G)])
>>> it = (x for x in a.normal_subgroups() if x.order() == Integer(9))
>>> subg = next(iter(it))
>>> r = [matrix(libgap.PermutationMat(libgap(z), Integer(9)).sage())
...       for z in subg]
>>> ff = list(map(lambda y: (y[Integer(0)]-Integer(1),y[Integer(1)]-
↪Integer(1)),
...           Permutation(map(lambda x: Integer(1)+r.index(x**-Integer(1)),␣
↪r)).cycle_tuples()[Integer(1):]))
>>> L = sum(i*(r[a]-r[b]) for i,(a,b) in zip(range(Integer(1),
↪len(ff)+Integer(1)), ff))
>>> G.relabel(range(Integer(9)))
>>> G3x3 = graphs.MathonPseudocyclicStronglyRegularGraph(Integer(2), G=G, L=L)
>>> G3x3.is_strongly_regular(parameters=True)
(441, 220, 109, 110)
>>> G3x3.automorphism_group(algorithm='bliss').order()  # optional - bliss
27
>>> G9 = graphs.MathonPseudocyclicStronglyRegularGraph(Integer(2))
>>> G9.is_strongly_regular(parameters=True)
(441, 220, 109, 110)
>>> G9.automorphism_group(algorithm='bliss').order()  # optional - bliss
9
```

static MathonStronglyRegularGraph(*t*, *immutable=False*)

Return one of Mathon's graphs on 784 vertices.

INPUT:

- `t` – integer; the number of the graph, from 0 to 2
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: # long time, needs sage.libs.gap
sage: from sage.graphs.generators.smallgraphs import␣
```

(continues on next page)

(continued from previous page)

```
↪MathonStronglyRegularGraph
sage: G = MathonStronglyRegularGraph(0)
sage: G.is_strongly_regular(parameters=True)
(784, 243, 82, 72)
```

```
>>> from sage.all import *
>>> # long time, needs sage.libs.gap
>>> from sage.graphs.generators.smallgraphs import MathonStronglyRegularGraph
>>> G = MathonStronglyRegularGraph(Integer(0))
>>> G.is_strongly_regular(parameters=True)
(784, 243, 82, 72)
```

static McGeeGraph(*embedding=2*, *immutable=False*)

Return the McGee Graph.

See the Wikipedia article McGee_graph.

INPUT:

- `embedding` – integer (default: 2); two embeddings are available, and can be selected by setting `embedding` to 1 or 2
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.McGeeGraph()
sage: g.order()
24
sage: g.size()
36
sage: g.girth()
7
sage: g.diameter()
4
sage: g.show()                                                              ␣
↪  # needs sage.plot
sage: graphs.McGeeGraph(embedding=1).show()      # long time                ␣
↪  # needs sage.plot
```

```
>>> from sage.all import *
>>> g = graphs.McGeeGraph()
>>> g.order()
24
>>> g.size()
36
>>> g.girth()
7
>>> g.diameter()
4
>>> g.show()
↪# needs sage.plot
>>> graphs.McGeeGraph(embedding=Integer(1)).show()      # long time          ␣
↪          # needs sage.plot
```

static McLaughlinGraph(*immutable=False*)

Return the McLaughlin Graph.

The McLaughlin Graph is the unique strongly regular graph of parameters $(275, 112, 30, 56)$.

For more information on the McLaughlin Graph, see its web page on Andries Brouwer's website which gives the definition that this method implements.

Note

To create this graph you must have the gap_packages spkg installed.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.McLaughlinGraph()              # optional - gap_package_
↪design
sage: g.is_strongly_regular(parameters=True)    # optional - gap_package_
↪design
(275, 112, 30, 56)
sage: set(g.spectrum()) == {112, 2, -28}        # optional - gap_package_
↪design
True
```

```
>>> from sage.all import *
>>> g = graphs.McLaughlinGraph()              # optional - gap_package_design
>>> g.is_strongly_regular(parameters=True)    # optional - gap_package_design
(275, 112, 30, 56)
>>> set(g.spectrum()) == {Integer(112), Integer(2), -Integer(28)}        #␣
↪optional - gap_package_design
True
```

static MeredithGraph(*immutable=False*)

Return the Meredith Graph.

The Meredith Graph is a 4-regular 4-connected non-hamiltonian graph. For more information on the Meredith Graph, see the Wikipedia article Meredith_graph.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.MeredithGraph()
sage: g.is_regular(4)
True
sage: g.order()
70
sage: g.size()
140
sage: g.radius()
7
```

(continues on next page)

(continued from previous page)

```
sage: g.diameter()
8
sage: g.girth()
4
sage: g.chromatic_number()
3
sage: g.is_hamiltonian()                 # long time
↪  # needs sage.numerical.mip
False
```

```
>>> from sage.all import *
>>> g = graphs.MeredithGraph()
>>> g.is_regular(Integer(4))
True
>>> g.order()
70
>>> g.size()
140
>>> g.radius()
7
>>> g.diameter()
8
>>> g.girth()
4
>>> g.chromatic_number()
3
>>> g.is_hamiltonian()                 # long time
↪# needs sage.numerical.mip
False
```

static MoebiusKantorGraph(*immutable=False*)

Return a Möbius-Kantor Graph.

A Möbius-Kantor graph is a cubic symmetric graph. (See also the Heawood graph). It has 16 nodes and 24 edges. It is nonplanar and Hamiltonian. It has diameter 4, girth 6, and chromatic number 2. It is identical to the Generalized Petersen graph, P[8, 3].

For more details, see Möbius-Kantor Graph - from Wolfram MathWorld.

PLOTTING: See the plotting section for the generalized Petersen graphs.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: MK = graphs.MoebiusKantorGraph()
sage: MK
Moebius-Kantor Graph: Graph on 16 vertices
sage: MK.graph6_string()
'OhCGKE?O@?ACAC@I?Q_AS'
sage: (graphs.MoebiusKantorGraph()).show()      # long time
↪  # needs sage.plot
```

```
>>> from sage.all import *
>>> MK = graphs.MoebiusKantorGraph()
>>> MK
Moebius-Kantor Graph: Graph on 16 vertices
>>> MK.graph6_string()
'OhCGKE?O@?ACAC@I?Q_AS'
>>> (graphs.MoebiusKantorGraph()).show()      # long time
↪# needs sage.plot
```

static MoebiusLadderGraph(*n*, *immutable=False*)

Return a Möbius ladder graph with $2n$ nodes

A Möbius ladder graph of order $2n$ is a ladder graph of the same order that is connected at the ends with a single twist, i.e., a ladder graph bent around so that top meets bottom with a single twist. Alternatively, it can be described as a single cycle graph (of order $2n$) with the addition of edges (called $rungs$) joining the antipodal pairs of nodes. Also, note that the Möbius ladder graph `graphs.MoebiusLadderGraph(n)` is precisely the same graph as the circulant graph `graphs.CirculantGraph(2 * n, [1, n])`.

PLOTTING:

Upon construction, the position dictionary is filled to override the spring-layout algorithm. By convention, each Möbius ladder graph will be displayed with the first (0) node at the top, with the rest following in a counterclockwise manner.

INPUT:

- `n` – a nonnegative integer; number of nodes is $2n$
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

OUTPUT:

- `G` – a Möbius ladder graph of order $2n$; note that a `ValueError` is returned if $n < 0$

EXAMPLES:

Construct and show a Möbius ladder graph with 26 nodes:

```
sage: g = graphs.MoebiusLadderGraph(13)
sage: g.show()                              # long time
↪  # needs sage.plot
```

```
>>> from sage.all import *
>>> g = graphs.MoebiusLadderGraph(Integer(13))
>>> g.show()                              # long time
↪# needs sage.plot
```

Create several Möbius ladder graphs in a Sage graphics array:

```
sage: g = []
sage: j = []
sage: for i in range(9):
....:     k = graphs.MoebiusLadderGraph(i+3)
....:     g.append(k)
sage: for i in range(3):
....:     n = []
....:     for m in range(3):
```

(continues on next page)

(continued from previous page)

```
....:         n.append(g[3*i + m].plot(vertex_size=50, vertex_labels=False))
....:     j.append(n)
sage: G = graphics_array(j)
sage: G.show()                              # long time
```

```
>>> from sage.all import *
>>> g = []
>>> j = []
>>> for i in range(Integer(9)):
...     k = graphs.MoebiusLadderGraph(i+Integer(3))
...     g.append(k)
>>> for i in range(Integer(3)):
...     n = []
...     for m in range(Integer(3)):
...         n.append(g[Integer(3)*i + m].plot(vertex_size=Integer(50), vertex_
↪labels=False))
...     j.append(n)
>>> G = graphics_array(j)
>>> G.show()                              # long time
```

REFERENCES:

- Wikipedia article Möbius_ladder

See also

LadderGraph(), *CircularLadderGraph()*, *CirculantGraph()*

AUTHORS:

- Janmenjaya Panda (2024-05-26)

static MoserSpindle(*immutable=False*)

Return the Moser spindle.

For more information, see the Wikipedia article Moser_spindle.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

The Moser spindle is a planar graph having 7 vertices and 11 edges:

```
sage: G = graphs.MoserSpindle(); G
Moser spindle: Graph on 7 vertices
sage: G.is_planar()
True
sage: G.order()
7
sage: G.size()
11
```

```
>>> from sage.all import *
>>> G = graphs.MoserSpindle(); G
Moser spindle: Graph on 7 vertices
>>> G.is_planar()
True
>>> G.order()
7
>>> G.size()
11
```

It is a Hamiltonian graph with radius 2, diameter 2, and girth 3:

```
sage: G.is_hamiltonian()
↪  # needs sage.numerical.mip
True
sage: G.radius()
2
sage: G.diameter()
2
sage: G.girth()
3
```

```
>>> from sage.all import *
>>> G.is_hamiltonian()
↪# needs sage.numerical.mip
True
>>> G.radius()
2
>>> G.diameter()
2
>>> G.girth()
3
```

The Moser spindle can be drawn in the plane as a unit distance graph, has chromatic number 4, and its automorphism group is isomorphic to the dihedral group D_4:

```
sage: pos = G.get_pos()
sage: all(sum((ui-vi)**2 for ui, vi in zip(pos[u], pos[v])) == 1
....:         for u, v in G.edge_iterator(labels=None))
True
sage: G.chromatic_number()
4
sage: ag = G.automorphism_group()
sage: ag.is_isomorphic(DihedralGroup(4))
True
```

```
>>> from sage.all import *
>>> pos = G.get_pos()
>>> all(sum((ui-vi)**Integer(2) for ui, vi in zip(pos[u], pos[v])) ==
↪Integer(1)
...         for u, v in G.edge_iterator(labels=None))
True
>>> G.chromatic_number()
```

(continues on next page)

(continued from previous page)

```
4
>>> ag = G.automorphism_group()
>>> ag.is_isomorphic(DihedralGroup(Integer(4)))
True
```

static MurtyGraph(*immutable=False*)

Return the Murty graph.

Consider the complete bipartite graph $K_{3,3}$. There is a set of three black vertices and a set of three white vertices. Now, consider splicing the complete graph K_4 with one of the black vertices, this generates the graph $K_4 \odot K_{3,3}$. The Murty graph is obtained from $K_4 \odot K_{3,3}$ with the addition of an edge joining the remaining two black vertices. The Murty graph is free of conformal bicycles; in other words, the Murty graph is an example of a graph that is Birkhoff-von Neumann as well as PM-compact.

This is the smallest brick that is Birkhoff-von Neumann, aka a solid brick, but is not odd-intercyclic. It is in this context that Prof. U.S.R. Murty first stumbled upon this graph, and it also appears in the work of Carvalho, Lucchesi, and Murty [CLM2006].

PLOTTING:

Upon construction, the position dictionary is filled to override the spring-layout algorithm. By convention, the Murty graph is displayed as mentioned in the paper [CKWL2019], with the first two (noncubic) vertices on the top row, the second three vertices (that form a stable set) in the middle row, and the remaining three vertices (that form a triangle) at the bottom.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

OUTPUT:

- `G` – the Murty graph

EXAMPLES:

Construct and show the Murty graph:

```
sage: g = graphs.MurtyGraph()
sage: g.name()
'Murty Graph'
sage: g.order()
8
sage: g.size()
13
sage: g.girth()
3
sage: g.diameter()
2
sage: g.is_hamiltonian()
True
sage: g.show()                                # long time
↪  # needs sage.plot
```

```
>>> from sage.all import *
>>> g = graphs.MurtyGraph()
>>> g.name()
'Murty Graph'
```

(continues on next page)

(continued from previous page)

```
>>> g.order()
8
>>> g.size()
13
>>> g.girth()
3
>>> g.diameter()
2
>>> g.is_hamiltonian()
True
>>> g.show()                              # long time
↪# needs sage.plot
```

REFERENCES:

- [CKWL2019]
- [CLM2006]
- [LM2024]

AUTHORS:

- Janmenjaya Panda (2024-08-03)

static MuzychukS6Graph(*n*, *d*, *Phi='fixed'*, *Sigma='fixed'*, *verbose=False*, *immutable=False*)

Return a strongly regular graph of S6 type from [Muz2007] on $n^d((n^d-1)/(n-1)+1)$ vertices.

The construction depends upon a number of parameters, two of them, n and d, mandatory, and Φ and Σ mappings defined in [Muz2007]. These graphs have parameters $(mn^d, n^{d-1}(m-1)-1, \mu-2, \mu)$, where $\mu = \frac{n^{d-1}-1}{n-1}n^{d-1}$ and $m := \frac{n^d-1}{n-1}+1$.

Some details on Φ and Σ are as follows. Let L be the complete graph on $M := \{0, ..., m-1\}$ with the matching $\{(2i, 2i+1) | i = 0, ..., m/2\}$ removed. Then one arbitrarily chooses injections Φ_i from the edges of L on $i \in M$ into sets of parallel classes of affine d-dimensional designs; our implementation uses the designs of hyperplanes in d-dimensional affine geometries over $GF(n)$. Finally, for each edge ij of L one arbitrarily chooses bijections Σ_{ij} between Φ_i and Φ_j. More details, in particular how these choices lead to non-isomorphic graphs, are in [Muz2007].

INPUT:

- `n` – integer; a prime power
- `d` – integer; must be odd if n is odd
- `Phi` is an optional parameter of the construction; it must be either
  - `'fixed'` – this will generate fixed default Φ_i, for $i \in M$, or
  - `'random'` – Φ_i are generated at random, or
  - A dictionary describing the functions Φ_i; for $i \in M$, Phi[(i, T)] in M, for each edge T of L on i. Also, each Φ_i must be injective.
- `Sigma` is an optional parameter of the construction; it must be either
  - `'fixed'` – this will generate a fixed default Σ, or
  - `'random'` – Σ is generated at random
- `verbose` – boolean (default: `False`); if `True`, print progress information

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

See also

- *is_muzychuk_S6()*

Todo

Implement the possibility to explicitly supply the parameter Σ of the construction.

EXAMPLES:

```
sage: graphs.MuzychukS6Graph(3, 3).is_strongly_regular(parameters=True)
(378, 116, 34, 36)
sage: phi = {(2,(0,2)):0, (1,(1,3)):1, (0,(0,3)):1, (2,(1,2)):1,
....:         (1,(1,2)):0, (0,(0,2)):0, (3,(0,3)):0, (3,(1,3)):1}
sage: graphs.MuzychukS6Graph(2, 2, Phi=phi).is_strongly_
↪regular(parameters=True)
(16, 5, 0, 2)
```

```
>>> from sage.all import *
>>> graphs.MuzychukS6Graph(Integer(3), Integer(3)).is_strongly_
↪regular(parameters=True)
(378, 116, 34, 36)
>>> phi = {(Integer(2),(Integer(0),Integer(2))):Integer(0), (Integer(1),
↪(Integer(1),Integer(3))):Integer(1), (Integer(0),(Integer(0),
↪Integer(3))):Integer(1), (Integer(2),(Integer(1),Integer(2))):Integer(1),
...         (Integer(1),(Integer(1),Integer(2))):Integer(0), (Integer(0),
↪(Integer(0),Integer(2))):Integer(0), (Integer(3),(Integer(0),
↪Integer(3))):Integer(0), (Integer(3),(Integer(1),Integer(3))):Integer(1)}
>>> graphs.MuzychukS6Graph(Integer(2), Integer(2), Phi=phi).is_strongly_
↪regular(parameters=True)
(16, 5, 0, 2)
```

static MycielskiGraph(*k=1*, *relabel=True*, *immutable=False*)

Return the k-th Mycielski Graph.

The graph M_k is triangle-free and has chromatic number equal to k. These graphs show, constructively, that there are triangle-free graphs with arbitrarily high chromatic number.

The Mycielski graphs are built recursively starting with M_0, an empty graph; M_1, a single vertex graph; and M_2 is the graph K_2. M_{k+1} is then built from M_k as follows:

If the vertices of M_k are $v_1, \ldots, v_n$, then the vertices of M_{k+1} are $v_1, \ldots, v_n, w_1, \ldots, w_n, z$. Vertices $v_1, \ldots, v_n$ induce a copy of M_k. Vertices $w_1, \ldots, w_n$ are an independent set. Vertex z is adjacent to all the w_i-vertices. Finally, vertex w_i is adjacent to vertex v_j iff v_i is adjacent to v_j.

For more details, see the Wikipedia article Mycielskian.

INPUT:

- `k` – number of steps in the construction process

- `relabel` – relabel the vertices so their names are the integers `range(n)` where `n` is the number of vertices in the graph
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

The Mycielski graph M_k is triangle-free and has chromatic number equal to k.

```
sage: g = graphs.MycielskiGraph(5)
sage: g.is_triangle_free()
True
sage: g.chromatic_number()
5
```

```
>>> from sage.all import *
>>> g = graphs.MycielskiGraph(Integer(5))
>>> g.is_triangle_free()
True
>>> g.chromatic_number()
5
```

The graphs M_4 is (isomorphic to) the Grotzsch graph.

```
sage: g = graphs.MycielskiGraph(4)
sage: g.is_isomorphic(graphs.GrotzschGraph())
True
```

```
>>> from sage.all import *
>>> g = graphs.MycielskiGraph(Integer(4))
>>> g.is_isomorphic(graphs.GrotzschGraph())
True
```

static MycielskiStep(*g*, *immutable=None*)

Perform one iteration of the Mycielski construction.

See the documentation for `MycielskiGraph` which uses this method. We expose it to all users in case they may find it useful.

INPUT:

- `g` – a graph
- `immutable` – boolean (default: `None`); whether to return a mutable or an immutable graph. `immutable=None` (default) means that the input and returned graphs behave the same way.

EXAMPLES:

One iteration of the Mycielski step applied to the 5-cycle yields a graph isomorphic to the Grotzsch graph

```
sage: g = graphs.CycleGraph(5)
sage: h = graphs.MycielskiStep(g)
sage: h.is_isomorphic(graphs.GrotzschGraph())
True
```

```
>>> from sage.all import *
>>> g = graphs.CycleGraph(Integer(5))
```

(continues on next page)

(continued from previous page)

```
>>> h = graphs.MycielskiStep(g)
>>> h.is_isomorphic(graphs.GrotzschGraph())
True
```

static NKStarGraph(*n*, *k*, *immutable=False*)

Return the (n, k)-star graph.

The vertices of the (n, k)-star graph are the set of all arrangements of n symbols into labels of length k. There are two adjacency rules for the (n, k)-star graph. First, two vertices are adjacent if one can be obtained from the other by swapping the first symbol with another symbol. Second, two vertices are adjacent if one can be obtained from the other by swapping the first symbol with an external symbol (a symbol not used in the original label).

INPUT:

- `n` – integer; number of symbols
- `k` – integer; length of the labels of the vertices
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.NKStarGraph(4,2)
sage: g.plot()                          # long time
↪  # needs sage.plot
Graphics object consisting of 31 graphics primitives
```

```
>>> from sage.all import *
>>> g = graphs.NKStarGraph(Integer(4),Integer(2))
>>> g.plot()                            # long time
↪# needs sage.plot
Graphics object consisting of 31 graphics primitives
```

REFERENCES:

[CC1995]

AUTHORS:

- Michael Yurko (2009-09-01)

static NStarGraph(*n*, *immutable=False*)

Return the n-star graph.

The vertices of the n-star graph are the set of permutations on n symbols. There is an edge between two vertices if their labels differ only in the first and one other position.

INPUT:

- `n` – integer; number of symbols
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.NStarGraph(4)
sage: g.plot()                          # long time
↪  # needs sage.plot
Graphics object consisting of 61 graphics primitives
```

```
>>> from sage.all import *
>>> g = graphs.NStarGraph(Integer(4))
>>> g.plot()                              # long time
↪# needs sage.plot
Graphics object consisting of 61 graphics primitives
```

REFERENCES:

[AHK1994]

AUTHORS:

- Michael Yurko (2009-09-01)

static NauruGraph(*embedding=2*, *immutable=False*)

Return the Nauru Graph.

See the Wikipedia article Nauru_graph.

INPUT:

- `embedding` – integer (default: 2); two embeddings are available, and can be selected by setting `embedding` to 1 or 2
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.NauruGraph()
sage: g.order()
24
sage: g.size()
36
sage: g.girth()
6
sage: g.diameter()
4
sage: g.show()                                                              ␣
↪  # needs sage.plot
sage: graphs.NauruGraph(embedding=1).show()      # long time                ␣
↪  # needs sage.plot
```

```
>>> from sage.all import *
>>> g = graphs.NauruGraph()
>>> g.order()
24
>>> g.size()
36
>>> g.girth()
6
>>> g.diameter()
4
>>> g.show()
↪# needs sage.plot
>>> graphs.NauruGraph(embedding=Integer(1)).show()      # long time          ␣
↪          # needs sage.plot
```

static NonisotropicOrthogonalPolarGraph(*m*, *q*, *sign='+'*, *perp=None*, *immutable=False*)

Return the Graph $NO_m^{\epsilon,\perp}(q)$.

Let the vectorspace of dimension m over F_q be endowed with a nondegenerate quadratic form F, of type `sign` for m even.

- m even: assume further that $q = 2$ or 3. Returns the graph of the points (in the underlying projective space) x satisfying $F(x) = 1$, with adjacency given by orthogonality w.r.t. F. Parameter `perp` is ignored.
- m odd: if `perp` is not `None`, then we assume that $q = 5$ and return the graph of the points x satisfying $F(x) = \pm 1$ if `sign="+"`, respectively $F(x) \in \{2, 3\}$ if `sign="-"`, with adjacency given by orthogonality w.r.t. F (cf. Sect 7.D of [BL1984]). Otherwise return the graph of nongenerate hyperplanes of type `sign`, adjacent whenever the intersection is degenerate (cf. Sect. 7.C of [BL1984]). Note that for $q = 2$ one will get a complete graph.

For more information, see Sect. 9.9 of [BH2012] and [BL1984]. Note that the page of Andries Brouwer's website uses different notation.

INPUT:

- `m` – integer; half the dimension of the underlying vectorspace
- `q` – a power of a prime number, the size of the underlying field
- `sign` – string (default: `'+'`); must be either `'+'` or `'-'`
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

$NO^-(4, 2)$ is isomorphic to Petersen graph:

```
sage: g = graphs.NonisotropicOrthogonalPolarGraph(4,2,'-'); g
↪  # needs sage.libs.gap
NO^-(4, 2): Graph on 10 vertices
sage: g.is_strongly_regular(parameters=True)
↪  # needs sage.libs.gap
(10, 3, 0, 1)
```

```
>>> from sage.all import *
>>> g = graphs.NonisotropicOrthogonalPolarGraph(Integer(4),Integer(2),'-'); g
↪                  # needs sage.libs.gap
NO^-(4, 2): Graph on 10 vertices
>>> g.is_strongly_regular(parameters=True)
↪# needs sage.libs.gap
(10, 3, 0, 1)
```

$NO^-(6, 2)$ and $NO^+(6, 2)$:

```
sage: g = graphs.NonisotropicOrthogonalPolarGraph(6,2,'-')
sage: g.is_strongly_regular(parameters=True)
(36, 15, 6, 6)
sage: g = graphs.NonisotropicOrthogonalPolarGraph(6,2,'+'); g
NO^+(6, 2): Graph on 28 vertices
sage: g.is_strongly_regular(parameters=True)
(28, 15, 6, 10)
```

```
>>> from sage.all import *
>>> g = graphs.NonisotropicOrthogonalPolarGraph(Integer(6),Integer(2),'-')
>>> g.is_strongly_regular(parameters=True)
(36, 15, 6, 6)
>>> g = graphs.NonisotropicOrthogonalPolarGraph(Integer(6),Integer(2),'+'); g
NO^+(6, 2): Graph on 28 vertices
>>> g.is_strongly_regular(parameters=True)
(28, 15, 6, 10)
```

$NO^+(8,2)$:

```
sage: g = graphs.NonisotropicOrthogonalPolarGraph(8,2,'+')
↪  # needs sage.libs.gap
sage: g.is_strongly_regular(parameters=True)
↪  # needs sage.libs.gap
(120, 63, 30, 36)
```

```
>>> from sage.all import *
>>> g = graphs.NonisotropicOrthogonalPolarGraph(Integer(8),Integer(2),'+')
↪                  # needs sage.libs.gap
>>> g.is_strongly_regular(parameters=True)
↪# needs sage.libs.gap
(120, 63, 30, 36)
```

Wilbrink's graphs for $q = 5$:

```
sage: g = graphs.NonisotropicOrthogonalPolarGraph(5,5,perp=1)
sage: g.is_strongly_regular(parameters=True)     # long time
(325, 60, 15, 10)
sage: g = graphs.NonisotropicOrthogonalPolarGraph(5,5,'-',perp=1)
sage: g.is_strongly_regular(parameters=True)     # long time
(300, 65, 10, 15)
```

```
>>> from sage.all import *
>>> g = graphs.NonisotropicOrthogonalPolarGraph(Integer(5),Integer(5),
↪perp=Integer(1))
>>> g.is_strongly_regular(parameters=True)     # long time
(325, 60, 15, 10)
>>> g = graphs.NonisotropicOrthogonalPolarGraph(Integer(5),Integer(5),'-',
↪perp=Integer(1))
>>> g.is_strongly_regular(parameters=True)     # long time
(300, 65, 10, 15)
```

Wilbrink's graphs:

```
sage: g = graphs.NonisotropicOrthogonalPolarGraph(5,4,'+')
sage: g.is_strongly_regular(parameters=True)
(136, 75, 42, 40)
sage: g = graphs.NonisotropicOrthogonalPolarGraph(5,4,'-')
sage: g.is_strongly_regular(parameters=True)
(120, 51, 18, 24)
sage: g = graphs.NonisotropicOrthogonalPolarGraph(7,4,'+'); g         # not␣
↪tested (long time)
```

(continues on next page)

(continued from previous page)

```
NO^+(7, 4): Graph on 2080 vertices
sage: g.is_strongly_regular(parameters=True)                              # not␣
↪tested (long time)
(2080, 1071, 558, 544)
```

```
>>> from sage.all import *
>>> g = graphs.NonisotropicOrthogonalPolarGraph(Integer(5),Integer(4),'+')
>>> g.is_strongly_regular(parameters=True)
(136, 75, 42, 40)
>>> g = graphs.NonisotropicOrthogonalPolarGraph(Integer(5),Integer(4),'-')
>>> g.is_strongly_regular(parameters=True)
(120, 51, 18, 24)
>>> g = graphs.NonisotropicOrthogonalPolarGraph(Integer(7),Integer(4),'+'); g␣
↪       # not tested (long time)
NO^+(7, 4): Graph on 2080 vertices
>>> g.is_strongly_regular(parameters=True)                               # not␣
↪tested (long time)
(2080, 1071, 558, 544)
```

static NonisotropicUnitaryPolarGraph(*m*, *q*, *immutable=False*)

Return the Graph $NU(m, q)$.

This returns the graph on nonisotropic, with respect to a nondegenerate Hermitean form, points of the $(m-1)$-dimensional projective space over F_q, with points adjacent whenever they lie on a tangent (to the set of isotropic points) line. For more information, see Sect. 9.9 of [BH2012] and series C14 in [Hub1975].

INPUT:

- `m`, `q` – integers; q must be a prime power
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.NonisotropicUnitaryPolarGraph(5,2); g                   ␣
↪  # needs sage.libs.gap
NU(5, 2): Graph on 176 vertices
sage: g.is_strongly_regular(parameters=True)                             ␣
↪  # needs sage.libs.gap
(176, 135, 102, 108)
```

```
>>> from sage.all import *
>>> g = graphs.NonisotropicUnitaryPolarGraph(Integer(5),Integer(2)); g      ␣
↪                  # needs sage.libs.gap
NU(5, 2): Graph on 176 vertices
>>> g.is_strongly_regular(parameters=True)
↪# needs sage.libs.gap
(176, 135, 102, 108)
```

static Nowhere0WordsTwoWeightCodeGraph(*q*, *hyperoval=None*, *field=None*, *check_hyperoval=True*, *immutable=False*)

Return the subgraph of nowhere 0 words from two-weight code of projective plane hyperoval.

Let $q = 2^k$ and $\Pi = PG(2, q)$. Fix a hyperoval $O \subset \Pi$. Let $V = F_q^3$ and C the two-weight 3-dimensional

linear code over F_q with words $c(v)$ obtained from $v \in V$ by computing

$$c(v) = (\langle v, o_1 \rangle, ..., \langle v, o_{q+2} \rangle), o_j \in O.$$

C contains $q(q-1)^2/2$ words without 0 entries. The subgraph of the strongly regular graph of C induced on the latter words is also strongly regular, assuming $q > 4$. This is a construction due to A.E.Brouwer [Bro2016], and leads to graphs with parameters also given by a construction in [HHL2009]. According to [Bro2016], these two constructions are likely to produce isomorphic graphs.

INPUT:

- `q` – a power of two
- `hyperoval` – a hyperoval (i.e. a complete 2-arc; a set of points in the plane meeting every line in 0 or 2 points) in $PG(2, q)$ over the field `field`. Each point of `hyperoval` must be a length 3 vector over `field` with 1st non-0 coordinate equal to 1. By default, `hyperoval` and `field` are not specified, and constructed on the fly. In particular, `hyperoval` we build is the classical one, i.e. a conic with the point of intersection of its tangent lines.
- `field` – an instance of a finite field of order q; must be provided if `hyperoval` is provided
- `check_hyperoval` – boolean (default: `True`); whether to check `hyperoval` for correctness or not
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

See also

- *is_nowhere0_twoweight()*

EXAMPLES:

using the built-in construction:

```
sage: g = graphs.Nowhere0WordsTwoWeightCodeGraph(8); g
Nowhere0WordsTwoWeightCodeGraph(8): Graph on 196 vertices
sage: g.is_strongly_regular(parameters=True)
(196, 60, 14, 20)
sage: g = graphs.Nowhere0WordsTwoWeightCodeGraph(16)  # not tested (long time)
sage: g.is_strongly_regular(parameters=True)          # not tested (long time)
(1800, 728, 268, 312)
```

```
>>> from sage.all import *
>>> g = graphs.Nowhere0WordsTwoWeightCodeGraph(Integer(8)); g
Nowhere0WordsTwoWeightCodeGraph(8): Graph on 196 vertices
>>> g.is_strongly_regular(parameters=True)
(196, 60, 14, 20)
>>> g = graphs.Nowhere0WordsTwoWeightCodeGraph(Integer(16))  # not tested␣
↪(long time)
>>> g.is_strongly_regular(parameters=True)          # not tested (long time)
(1800, 728, 268, 312)
```

supplying your own hyperoval:

```
sage: F = GF(8)
sage: O = [vector(F,(0,0,1)),vector(F,(0,1,0))] + [vector(F, (1,x^2,x))
....:                                               for x in F]
```

(continues on next page)

(continued from previous page)

```
sage: g = graphs.Nowhere0WordsTwoWeightCodeGraph(8,hyperoval=O,field=F); g
Nowhere0WordsTwoWeightCodeGraph(8): Graph on 196 vertices
sage: g.is_strongly_regular(parameters=True)
(196, 60, 14, 20)
```

```
>>> from sage.all import *
>>> F = GF(Integer(8))
>>> O = [vector(F,(Integer(0),Integer(0),Integer(1))),vector(F,(Integer(0),
↪Integer(1),Integer(0)))] + [vector(F, (Integer(1),x**Integer(2),x))
...                                                 for x in F]
>>> g = graphs.Nowhere0WordsTwoWeightCodeGraph(Integer(8),hyperoval=O,
↪field=F); g
Nowhere0WordsTwoWeightCodeGraph(8): Graph on 196 vertices
>>> g.is_strongly_regular(parameters=True)
(196, 60, 14, 20)
```

static OctahedralGraph(*immutable=False*)

Return an Octahedral graph (with 6 nodes).

The regular octahedron is an 8-sided polyhedron with triangular faces. The octahedral graph corresponds to the connectivity of the vertices of the octahedron. It is the line graph of the tetrahedral graph. The octahedral is symmetric, so the spring-layout algorithm will be very effective for display.

PLOTTING: The Octahedral graph should be viewed in 3 dimensions. We choose to use a planar embedding of the graph. We hope to add rotatable, 3-dimensional viewing in the future. In such a case, an argument will be added to select the desired layout.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

Construct and show an Octahedral graph:

```
sage: g = graphs.OctahedralGraph()
sage: g.show()                              # long time                    ␣
↪  # needs sage.plot
```

```
>>> from sage.all import *
>>> g = graphs.OctahedralGraph()
>>> g.show()                              # long time
↪# needs sage.plot
```

Create several octahedral graphs in a Sage graphics array They will be drawn differently due to the use of the spring-layout algorithm:

```
sage: g = []
sage: j = []
sage: for i in range(9):
....:     k = graphs.OctahedralGraph()
....:     g.append(k)
sage: for i in range(3):
....:     n = []
....:     for m in range(3):
```

(continues on next page)

(continued from previous page)

```
....:         n.append(g[3*i + m].plot(vertex_size=50, vertex_labels=False))
....:     j.append(n)
sage: G = graphics_array(j)
sage: G.show()                          # long time
```

```
>>> from sage.all import *
>>> g = []
>>> j = []
>>> for i in range(Integer(9)):
...     k = graphs.OctahedralGraph()
...     g.append(k)
>>> for i in range(Integer(3)):
...     n = []
...     for m in range(Integer(3)):
...         n.append(g[Integer(3)*i + m].plot(vertex_size=Integer(50), vertex_
↪labels=False))
...     j.append(n)
>>> G = graphics_array(j)
>>> G.show()                          # long time
```

static OddGraph(*n*, *immutable=False*)

Return the Odd Graph with parameter n.

The Odd Graph with parameter n is defined as the Kneser Graph with parameters $2n-1, n-1$. Equivalently, the Odd Graph is the graph whose vertices are the $n-1$-subsets of $[0, 1, \ldots, 2(n-1)]$, and such that two vertices are adjacent if their corresponding sets are disjoint.

For example, the Petersen Graph can be defined as the Odd Graph with parameter 3.

INPUT:

- `n` – integer; the groundset has $2n-1$ elements
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: OG = graphs.OddGraph(3)
sage: sorted(OG.vertex_iterator(), key=str)
[{1, 2}, {1, 3}, {1, 4}, {1, 5}, {2, 3}, {2, 4}, {2, 5},
 {3, 4}, {3, 5}, {4, 5}]
sage: P = graphs.PetersenGraph()
sage: P.is_isomorphic(OG)
True
```

```
>>> from sage.all import *
>>> OG = graphs.OddGraph(Integer(3))
>>> sorted(OG.vertex_iterator(), key=str)
[{1, 2}, {1, 3}, {1, 4}, {1, 5}, {2, 3}, {2, 4}, {2, 5},
 {3, 4}, {3, 5}, {4, 5}]
>>> P = graphs.PetersenGraph()
>>> P.is_isomorphic(OG)
True
```

static OrthogonalArrayBlockGraph(k, n, *OA=None*, *immutable=False*)

Return the graph of an $OA(k,n)$.

The intersection graph of the blocks of a transversal design with parameters (k,n), or $TD(k,n)$ for short, is a strongly regular graph (unless it is a complete graph). Its parameters (v,k',λ,μ) are determined by the parameters k,n via:

$$v=n^2, k'=k(n-1), \lambda=(k-1)(k-2)+n-2, \mu=k(k-1)$$

As transversal designs and orthogonal arrays (OA for short) are equivalent objects, this graph can also be built from the blocks of an $OA(k,n)$, two of them being adjacent if one of their coordinates match.

For more information on these graphs, see Andries Brouwer's page on Orthogonal Array graphs.

Warning

- Brouwer's website uses the notation $OA(n,k)$ instead of $OA(k,n)$
- For given parameters k and n there can be many $OA(k,n)$: the graphs returned are not uniquely defined by their parameters (see the examples below).
- If the function is called only with the parameter `k` and `n` the results might be different with two versions of Sage, or even worse : some could not be available anymore.

See also

`sage.combinat.designs.orthogonal_arrays`

INPUT:

- `k`, `n` – integers
- `OA` – an orthogonal array; if set to `None` (default) then `orthogonal_array()` is called to compute an $OA(k,n)$
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: G = graphs.OrthogonalArrayBlockGraph(5,5); G
↪  # needs sage.schemes
OA(5,5): Graph on 25 vertices
sage: G.is_strongly_regular(parameters=True)
↪  # needs sage.schemes
(25, 20, 15, 20)
sage: G = graphs.OrthogonalArrayBlockGraph(4,10); G
OA(4,10): Graph on 100 vertices
sage: G.is_strongly_regular(parameters=True)
(100, 36, 14, 12)
```

```
>>> from sage.all import *
>>> G = graphs.OrthogonalArrayBlockGraph(Integer(5),Integer(5)); G
↪                  # needs sage.schemes
OA(5,5): Graph on 25 vertices
```

(continues on next page)

(continued from previous page)

```
>>> G.is_strongly_regular(parameters=True)
↪# needs sage.schemes
(25, 20, 15, 20)
>>> G = graphs.OrthogonalArrayBlockGraph(Integer(4),Integer(10)); G
OA(4,10): Graph on 100 vertices
>>> G.is_strongly_regular(parameters=True)
(100, 36, 14, 12)
```

Two graphs built from different orthogonal arrays are also different:

```
sage: k = 4; n = 10
sage: OAa = designs.orthogonal_arrays.build(k,n)
sage: OAb = [[(x+1)%n for x in R] for R in OAa]
sage: set(map(tuple,OAa)) == set(map(tuple,OAb))
False
sage: Ga = graphs.OrthogonalArrayBlockGraph(k, n, OAa)
sage: Gb = graphs.OrthogonalArrayBlockGraph(k, n, OAb)
sage: Ga == Gb
False
```

```
>>> from sage.all import *
>>> k = Integer(4); n = Integer(10)
>>> OAa = designs.orthogonal_arrays.build(k,n)
>>> OAb = [[(x+Integer(1))%n for x in R] for R in OAa]
>>> set(map(tuple,OAa)) == set(map(tuple,OAb))
False
>>> Ga = graphs.OrthogonalArrayBlockGraph(k, n, OAa)
>>> Gb = graphs.OrthogonalArrayBlockGraph(k, n, OAb)
>>> Ga == Gb
False
```

As `OAb` was obtained from `OAa` by a relabelling the two graphs are isomorphic:

```
sage: Ga.is_isomorphic(Gb)
↪  # needs sage.modules
True
```

```
>>> from sage.all import *
>>> Ga.is_isomorphic(Gb)
↪# needs sage.modules
True
```

But there are examples of $OA(k,n)$ for which the resulting graphs are not isomorphic:

```
sage: oa0 = [[0, 0, 1], [0, 1, 3], [0, 2, 0], [0, 3, 2],
....:        [1, 0, 3], [1, 1, 1], [1, 2, 2], [1, 3, 0],
....:        [2, 0, 0], [2, 1, 2], [2, 2, 1], [2, 3, 3],
....:        [3, 0, 2], [3, 1, 0], [3, 2, 3], [3, 3, 1]]
sage: oa1 = [[0, 0, 1], [0, 1, 0], [0, 2, 3], [0, 3, 2],
....:        [1, 0, 3], [1, 1, 2], [1, 2, 0], [1, 3, 1],
....:        [2, 0, 0], [2, 1, 1], [2, 2, 2], [2, 3, 3],
....:        [3, 0, 2], [3, 1, 3], [3, 2, 1], [3, 3, 0]]
```

(continues on next page)

(continued from previous page)

```
sage: g0 = graphs.OrthogonalArrayBlockGraph(3, 4, oa0)
↪  # needs sage.modules
sage: g1 = graphs.OrthogonalArrayBlockGraph(3, 4, oa1)
↪  # needs sage.modules
sage: g0.is_isomorphic(g1)
↪  # needs sage.modules
False
```

```
>>> from sage.all import *
>>> oa0 = [[Integer(0), Integer(0), Integer(1)], [Integer(0), Integer(1),
↪Integer(3)], [Integer(0), Integer(2), Integer(0)], [Integer(0), Integer(3),
↪Integer(2)],
...         [Integer(1), Integer(0), Integer(3)], [Integer(1), Integer(1),
↪Integer(1)], [Integer(1), Integer(2), Integer(2)], [Integer(1), Integer(3),
↪Integer(0)],
...         [Integer(2), Integer(0), Integer(0)], [Integer(2), Integer(1),
↪Integer(2)], [Integer(2), Integer(2), Integer(1)], [Integer(2), Integer(3),
↪Integer(3)],
...         [Integer(3), Integer(0), Integer(2)], [Integer(3), Integer(1),
↪Integer(0)], [Integer(3), Integer(2), Integer(3)], [Integer(3), Integer(3),
↪Integer(1)]]
>>> oa1 = [[Integer(0), Integer(0), Integer(1)], [Integer(0), Integer(1),
↪Integer(0)], [Integer(0), Integer(2), Integer(3)], [Integer(0), Integer(3),
↪Integer(2)],
...         [Integer(1), Integer(0), Integer(3)], [Integer(1), Integer(1),
↪Integer(2)], [Integer(1), Integer(2), Integer(0)], [Integer(1), Integer(3),
↪Integer(1)],
...         [Integer(2), Integer(0), Integer(0)], [Integer(2), Integer(1),
↪Integer(1)], [Integer(2), Integer(2), Integer(2)], [Integer(2), Integer(3),
↪Integer(3)],
...         [Integer(3), Integer(0), Integer(2)], [Integer(3), Integer(1),
↪Integer(3)], [Integer(3), Integer(2), Integer(1)], [Integer(3), Integer(3),
↪Integer(0)]]
>>> g0 = graphs.OrthogonalArrayBlockGraph(Integer(3), Integer(4), oa0)
↪                  # needs sage.modules
>>> g1 = graphs.OrthogonalArrayBlockGraph(Integer(3), Integer(4), oa1)
↪                  # needs sage.modules
>>> g0.is_isomorphic(g1)
↪# needs sage.modules
False
```

But nevertheless isospectral:

```
sage: g0.spectrum()
↪  # needs sage.modules sage.rings.number_field
[9, 1, 1, 1, 1, 1, 1, 1, 1, 1, -3, -3, -3, -3, -3, -3]
sage: g1.spectrum()
↪  # needs sage.modules sage.rings.number_field
[9, 1, 1, 1, 1, 1, 1, 1, 1, 1, -3, -3, -3, -3, -3, -3]
```

```
>>> from sage.all import *
>>> g0.spectrum()
```

(continues on next page)

(continued from previous page)

```
↪# needs sage.modules sage.rings.number_field
[9, 1, 1, 1, 1, 1, 1, 1, 1, -3, -3, -3, -3, -3, -3]
>>> g1.spectrum()
↪# needs sage.modules sage.rings.number_field
[9, 1, 1, 1, 1, 1, 1, 1, 1, -3, -3, -3, -3, -3, -3]
```

Note that the graph `g0` is actually isomorphic to the affine polar graph $VO^+(4,2)$:

```
sage: graphs.AffineOrthogonalPolarGraph(4,2,'+').is_isomorphic(g0)
↪  # needs sage.libs.gap sage.modules
True
```

```
>>> from sage.all import *
>>> graphs.AffineOrthogonalPolarGraph(Integer(4),Integer(2),'+').is_
↪isomorphic(g0)              # needs sage.libs.gap sage.modules
True
```

static OrthogonalDualPolarGraph(*e*, *d*, *q*, *immutable=False*)

Return the dual polar graph on $GO^e(n,q)$ of diameter d.

The value of n is determined by d and e.

The graph is distance-regular with classical parameters $(d, q, 0, q^e)$.

INPUT:

- `e` – integer; type of the orthogonal polar space to consider; must be $-1, 0$ or 1
- `d` – integer; diameter of the graph
- `q` – integer; prime power; order of the finite field over which to build the polar space
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: G = graphs.OrthogonalDualPolarGraph(1,3,2)
sage: G.is_distance_regular(True)
([7, 6, 4, None], [None, 1, 3, 7])
sage: G = graphs.OrthogonalDualPolarGraph(0,3,3)         # long time
sage: G.is_distance_regular(True)                        # long time
([39, 36, 27, None], [None, 1, 4, 13])
sage: G.order()                                          # long time
1120
```

```
>>> from sage.all import *
>>> G = graphs.OrthogonalDualPolarGraph(Integer(1),Integer(3),Integer(2))
>>> G.is_distance_regular(True)
([7, 6, 4, None], [None, 1, 3, 7])
>>> G = graphs.OrthogonalDualPolarGraph(Integer(0),Integer(3),Integer(3))
↪   # long time
>>> G.is_distance_regular(True)                          # long time
([39, 36, 27, None], [None, 1, 4, 13])
>>> G.order()                                            # long time
1120
```

REFERENCES:

See [BCN1989] pp. 274-279 or [VDKT2016] p. 22.

static OrthogonalPolarGraph(*m*, *q*, *sign='+'*, *immutable=False*)

Return the Orthogonal Polar Graph $O^\epsilon(m,q)$.

For more information on Orthogonal Polar graphs, see the page of Andries Brouwer's website.

INPUT:

- `m`, `q` – integers; q must be a prime power
- `sign` – string (default: `'+'`); must be `'+'` or `'-'` if m is even, `'+'` (default) otherwise
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: G = graphs.OrthogonalPolarGraph(6,3,"+"); G
Orthogonal Polar Graph O^+(6, 3): Graph on 130 vertices
sage: G.is_strongly_regular(parameters=True)
(130, 48, 20, 16)
sage: G = graphs.OrthogonalPolarGraph(6,3,"-"); G
Orthogonal Polar Graph O^-(6, 3): Graph on 112 vertices
sage: G.is_strongly_regular(parameters=True)
(112, 30, 2, 10)
sage: G = graphs.OrthogonalPolarGraph(5,3); G
Orthogonal Polar Graph O(5, 3): Graph on 40 vertices
sage: G.is_strongly_regular(parameters=True)
(40, 12, 2, 4)
sage: G = graphs.OrthogonalPolarGraph(8,2,"+"); G
Orthogonal Polar Graph O^+(8, 2): Graph on 135 vertices
sage: G.is_strongly_regular(parameters=True)
(135, 70, 37, 35)
sage: G = graphs.OrthogonalPolarGraph(8,2,"-"); G
Orthogonal Polar Graph O^-(8, 2): Graph on 119 vertices
sage: G.is_strongly_regular(parameters=True)
(119, 54, 21, 27)
```

```
>>> from sage.all import *
>>> G = graphs.OrthogonalPolarGraph(Integer(6),Integer(3),"+"); G
Orthogonal Polar Graph O^+(6, 3): Graph on 130 vertices
>>> G.is_strongly_regular(parameters=True)
(130, 48, 20, 16)
>>> G = graphs.OrthogonalPolarGraph(Integer(6),Integer(3),"-"); G
Orthogonal Polar Graph O^-(6, 3): Graph on 112 vertices
>>> G.is_strongly_regular(parameters=True)
(112, 30, 2, 10)
>>> G = graphs.OrthogonalPolarGraph(Integer(5),Integer(3)); G
Orthogonal Polar Graph O(5, 3): Graph on 40 vertices
>>> G.is_strongly_regular(parameters=True)
(40, 12, 2, 4)
>>> G = graphs.OrthogonalPolarGraph(Integer(8),Integer(2),"+"); G
Orthogonal Polar Graph O^+(8, 2): Graph on 135 vertices
>>> G.is_strongly_regular(parameters=True)
(135, 70, 37, 35)
```

(continues on next page)

(continued from previous page)

```
>>> G = graphs.OrthogonalPolarGraph(Integer(8),Integer(2),"-"); G
Orthogonal Polar Graph O^-(8, 2): Graph on 119 vertices
>>> G.is_strongly_regular(parameters=True)
(119, 54, 21, 27)
```

static PaleyGraph(*q*, *immutable=False*)

Paley graph with q vertices.

Parameter q must be the power of a prime number and congruent to 1 mod 4.

INPUT:

- `q` – integer; number of vertices
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: G = graphs.PaleyGraph(9); G
↪  # needs sage.rings.finite_rings
Paley graph with parameter 9: Graph on 9 vertices
sage: G.is_regular()
↪  # needs sage.rings.finite_rings
True
```

```
>>> from sage.all import *
>>> G = graphs.PaleyGraph(Integer(9)); G
↪         # needs sage.rings.finite_rings
Paley graph with parameter 9: Graph on 9 vertices
>>> G.is_regular()
↪# needs sage.rings.finite_rings
True
```

A Paley graph is always self-complementary:

```
sage: G.is_self_complementary()
↪  # needs sage.rings.finite_rings
True
```

```
>>> from sage.all import *
>>> G.is_self_complementary()
↪# needs sage.rings.finite_rings
True
```

static PappusGraph(*immutable=False*)

Return the Pappus graph, a graph on 18 vertices.

The Pappus graph is cubic, symmetric, and distance-regular.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: G = graphs.PappusGraph()
sage: G.show()                              # long time                    ␣
↪  # needs sage.plot
sage: L = graphs.LCFGraph(18, [5,7,-7,7,-7,-5], 3)
sage: L.show()                              # long time                    ␣
↪  # needs sage.plot
sage: G.is_isomorphic(L)
True
```

```
>>> from sage.all import *
>>> G = graphs.PappusGraph()
>>> G.show()                              # long time
↪# needs sage.plot
>>> L = graphs.LCFGraph(Integer(18), [Integer(5),Integer(7),-Integer(7),
↪Integer(7),-Integer(7),-Integer(5)], Integer(3))
>>> L.show()                              # long time
↪# needs sage.plot
>>> G.is_isomorphic(L)
True
```

static PasechnikGraph(*n*, *immutable=False*)

Pasechnik strongly regular graph on $(4n-1)^2$ vertices.

A strongly regular graph with parameters of the orthogonal array graph *OrthogonalArrayBlockGraph()*, also known as pseudo Latin squares graph $L_{2n-1}(4n-1)$, constructed from a skew Hadamard matrix of order $4n$ following [Pas1992].

See also

- *is_orthogonal_array_block_graph()*

INPUT:

- `n` – integer; the returned graph has $(4n-1)^2$ vertices
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: graphs.PasechnikGraph(4).is_strongly_regular(parameters=True)          ␣
↪  # needs sage.combinat sage.modules
(225, 98, 43, 42)
sage: graphs.PasechnikGraph(5).is_strongly_regular(parameters=True)     #␣
↪long time, needs sage.combinat sage.modules
(361, 162, 73, 72)
sage: graphs.PasechnikGraph(9).is_strongly_regular(parameters=True)  # not␣
↪tested
(1225, 578, 273, 272)
```

```
>>> from sage.all import *
>>> graphs.PasechnikGraph(Integer(4)).is_strongly_regular(parameters=True)   ␣
↪         # needs sage.combinat sage.modules
(225, 98, 43, 42)
```

(continues on next page)

(continued from previous page)

```
>>> graphs.PasechnikGraph(Integer(5)).is_strongly_regular(parameters=True)
↪ # long time, needs sage.combinat sage.modules
(361, 162, 73, 72)
>>> graphs.PasechnikGraph(Integer(9)).is_strongly_regular(parameters=True)  #
↪not tested
(1225, 578, 273, 272)
```

static PathGraph(*n*, *pos=None*, *immutable=False*, *name=None*)

Return a path graph with n nodes.

A path graph is a graph where all inner nodes are connected to their two neighbors and the two end-nodes are connected to their one inner neighbors (i.e.: a cycle graph without the first and last node connected).

INPUT:

- `n` – nonnegative integer; number of nodes of the path graph
- `pos` – string (default: `None`); indicates the embedding to use between 'circle', 'line' or the default algorithm. See the plotting section below for more detail.
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph
- `name` – string (default: `None`); used as the name of the returned graph when set

PLOTTING: Upon construction, the position dictionary is filled to override the spring-layout algorithm. By convention, the graph may be drawn in one of two ways: The 'line' argument will draw the graph in a horizontal line (left to right) if there are less than 11 nodes. Otherwise the 'line' argument will append horizontal lines of length 10 nodes below, alternating left to right and right to left. The 'circle' argument will cause the graph to be drawn in a cycle-shape, with the first node at the top and then about the circle in a clockwise manner. By default (without an appropriate string argument) the graph will be drawn as a 'circle' if $10 < n < 41$ and as a 'line' for all other n.

EXAMPLES:

Show default drawing by size: 'line': $n \leq 10$:

```
sage: p = graphs.PathGraph(10)
sage: p.show()                              # long time
↪  # needs sage.plot
```

```
>>> from sage.all import *
>>> p = graphs.PathGraph(Integer(10))
>>> p.show()                              # long time
↪# needs sage.plot
```

'circle': $10 < n < 41$:

```
sage: q = graphs.PathGraph(25)
sage: q.show()                              # long time
↪  # needs sage.plot
```

```
>>> from sage.all import *
>>> q = graphs.PathGraph(Integer(25))
>>> q.show()                              # long time
↪# needs sage.plot
```

'line': $n \geq 41$:

```
sage: r = graphs.PathGraph(55)
sage: r.show()                              # long time
↪  # needs sage.plot
```

```
>>> from sage.all import *
>>> r = graphs.PathGraph(Integer(55))
>>> r.show()                              # long time
↪# needs sage.plot
```

Override the default drawing:

```
sage: s = graphs.PathGraph(5,'circle')
sage: s.show()                              # long time
↪  # needs sage.plot
```

```
>>> from sage.all import *
>>> s = graphs.PathGraph(Integer(5),'circle')
>>> s.show()                              # long time
↪# needs sage.plot
```

static PerkelGraph(*immutable=False*)

Return the Perkel Graph.

The Perkel Graph is a 6-regular graph with 57 vertices and 171 edges. It is the unique distance-regular graph with intersection array $(6, 5, 2; 1, 1, 3)$. For more information, see the Wikipedia article Perkel_graph or https://www.win.tue.nl/~aeb/graphs/Perkel.html.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.PerkelGraph(); g
Perkel Graph: Graph on 57 vertices
sage: g.is_distance_regular(parameters=True)
([6, 5, 2, None], [None, 1, 1, 3])
```

```
>>> from sage.all import *
>>> g = graphs.PerkelGraph(); g
Perkel Graph: Graph on 57 vertices
>>> g.is_distance_regular(parameters=True)
([6, 5, 2, None], [None, 1, 1, 3])
```

static PermutationGraph(*second_permutation*, *first_permutation=None*, *immutable=False*)

Build a permutation graph from one permutation or from two lists.

Definition:

If σ is a permutation of $\{1, 2, \ldots, n\}$, then the permutation graph of σ is the graph on vertex set $\{1, 2, \ldots, n\}$ in which two vertices i and j satisfying $i < j$ are connected by an edge if and only if $\sigma^{-1}(i) > \sigma^{-1}(j)$. A visual way to construct this graph is as follows:

> Take two horizontal lines in the euclidean plane, and mark points $1, ..., n$ from left to right on the first of them. On the second one, still from left to right, mark n points $\sigma(1), \sigma(2), \ldots, \sigma(n)$. Now, link by a segment the two points marked with 1, then link together the points marked with 2, and

so on. The permutation graph of σ is the intersection graph of those segments: there exists a vertex in this graph for each element from 1 to n, two vertices i, j being adjacent if the segments i and j cross each other.

The set of edges of the permutation graph can thus be identified with the set of inversions of the inverse of the given permutation σ.

A more general notion of permutation graph can be defined as follows: If S is a set, and $(a_1, a_2, \ldots, a_n)$ and $(b_1, b_2, \ldots, b_n)$ are two lists of elements of S, each of which lists contains every element of S exactly once, then the permutation graph defined by these two lists is the graph on the vertex set S in which two vertices i and j are connected by an edge if and only if the order in which these vertices appear in the list $(a_1, a_2, \ldots, a_n)$ is the opposite of the order in which they appear in the list $(b_1, b_2, \ldots, b_n)$. When $(a_1, a_2, \ldots, a_n) = (1, 2, \ldots, n)$, this graph is the permutation graph of the permutation $(b_1, b_2, \ldots, b_n) \in S_n$. Notice that S does not have to be a set of integers here, but can be a set of strings, tuples, or anything else. We can still use the above visual description to construct the permutation graph, but now we have to mark points $a_1, a_2, \ldots, a_n$ from left to right on the first horizontal line and points $b_1, b_2, \ldots, b_n$ from left to right on the second horizontal line.

INPUT:

- `second_permutation` – the unique permutation/list defining the graph, or the second of the two (if the graph is to be built from two permutations/lists)
- `first_permutation` – (optional) the first of the two permutations/lists from which the graph should be built, if it is to be built from two permutations/lists

  When `first_permutation is None` (default), it is set to be equal to `sorted(second_permutation)`, which yields the expected ordering when the elements of the graph are integers.
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

See also

- Recognition of Permutation graphs in the *comparability module*.
- Drawings of permutation graphs as intersection graphs of segments is possible through the `show()` method of `Permutation` objects.

  The correct argument to use in this case is `show(representation = "braid")`.
- `inversions()`

EXAMPLES:

```
sage: p = Permutations(5).random_element()
sage: PG = graphs.PermutationGraph(p)
sage: edges = PG.edges(sort=True, labels=False)
sage: set(edges) == set(p.inverse().inversions())
True

sage: PG = graphs.PermutationGraph([3,4,5,1,2])
sage: sorted(PG.edges(sort=True))
[(1, 3, None),
 (1, 4, None),
 (1, 5, None),
 (2, 3, None),
 (2, 4, None),
```

(continues on next page)

(continued from previous page)

```
 (2, 5, None)]
sage: PG = graphs.PermutationGraph([3,4,5,1,2], [1,4,2,5,3])
sage: sorted(PG.edges(sort=True))
[(1, 3, None),
 (1, 4, None),
 (1, 5, None),
 (2, 3, None),
 (2, 5, None),
 (3, 4, None),
 (3, 5, None)]
sage: PG = graphs.PermutationGraph([1,4,2,5,3], [3,4,5,1,2])
sage: sorted(PG.edges(sort=True))
[(1, 3, None),
 (1, 4, None),
 (1, 5, None),
 (2, 3, None),
 (2, 5, None),
 (3, 4, None),
 (3, 5, None)]

sage: PG = graphs.PermutationGraph(Permutation([1,3,2]), Permutation([1,2,3]))
sage: sorted(PG.edges(sort=True))
[(2, 3, None)]

sage: graphs.PermutationGraph([]).edges(sort=True)
[]
sage: graphs.PermutationGraph([], []).edges(sort=True)
[]

sage: PG = graphs.PermutationGraph("graph", "phrag")
sage: sorted(PG.edges(sort=True))
[('a', 'g', None),
 ('a', 'h', None),
 ('a', 'p', None),
 ('g', 'h', None),
 ('g', 'p', None),
 ('g', 'r', None),
 ('h', 'r', None),
 ('p', 'r', None)]
```

```
>>> from sage.all import *
>>> p = Permutations(Integer(5)).random_element()
>>> PG = graphs.PermutationGraph(p)
>>> edges = PG.edges(sort=True, labels=False)
>>> set(edges) == set(p.inverse().inversions())
True

>>> PG = graphs.PermutationGraph([Integer(3),Integer(4),Integer(5),Integer(1),
↪Integer(2)])
>>> sorted(PG.edges(sort=True))
[(1, 3, None),
 (1, 4, None),
```

(continues on next page)

(continued from previous page)

```
 (1, 5, None),
 (2, 3, None),
 (2, 4, None),
 (2, 5, None)]
>>> PG = graphs.PermutationGraph([Integer(3),Integer(4),Integer(5),Integer(1),
↪Integer(2)], [Integer(1),Integer(4),Integer(2),Integer(5),Integer(3)])
>>> sorted(PG.edges(sort=True))
[(1, 3, None),
 (1, 4, None),
 (1, 5, None),
 (2, 3, None),
 (2, 5, None),
 (3, 4, None),
 (3, 5, None)]
>>> PG = graphs.PermutationGraph([Integer(1),Integer(4),Integer(2),Integer(5),
↪Integer(3)], [Integer(3),Integer(4),Integer(5),Integer(1),Integer(2)])
>>> sorted(PG.edges(sort=True))
[(1, 3, None),
 (1, 4, None),
 (1, 5, None),
 (2, 3, None),
 (2, 5, None),
 (3, 4, None),
 (3, 5, None)]

>>> PG = graphs.PermutationGraph(Permutation([Integer(1),Integer(3),
↪Integer(2)]), Permutation([Integer(1),Integer(2),Integer(3)]))
>>> sorted(PG.edges(sort=True))
[(2, 3, None)]

>>> graphs.PermutationGraph([]).edges(sort=True)
[]
>>> graphs.PermutationGraph([], []).edges(sort=True)
[]

>>> PG = graphs.PermutationGraph("graph", "phrag")
>>> sorted(PG.edges(sort=True))
[('a', 'g', None),
 ('a', 'h', None),
 ('a', 'p', None),
 ('g', 'h', None),
 ('g', 'p', None),
 ('g', 'r', None),
 ('h', 'r', None),
 ('p', 'r', None)]
```

static PetersenGraph(*immutable=False*)

Return the Petersen Graph.

The Petersen Graph is a named graph that consists of 10 vertices and 15 edges, usually drawn as a five-point star embedded in a pentagon.

The Petersen Graph is a common counterexample. For example, it is not Hamiltonian.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

PLOTTING: See the plotting section for the generalized Petersen graphs.

EXAMPLES: We compare below the Petersen graph with the default spring-layout versus a planned position dictionary of (x, y) tuples:

```
sage: petersen_spring = Graph({0:[1,4,5], 1:[0,2,6], 2:[1,3,7],
....:                          3:[2,4,8], 4:[0,3,9], 5:[0,7,8],
....:                          6:[1,8,9], 7:[2,5,9], 8:[3,5,6],
....:                          9:[4,6,7]})
sage: petersen_spring.show()             # long time
↪  # needs sage.plot
sage: petersen_database = graphs.PetersenGraph()
sage: petersen_database.show()           # long time
↪  # needs sage.plot
```

```
>>> from sage.all import *
>>> petersen_spring = Graph({Integer(0):[Integer(1),Integer(4),Integer(5)],
↪Integer(1):[Integer(0),Integer(2),Integer(6)], Integer(2):[Integer(1),
↪Integer(3),Integer(7)],
...                          Integer(3):[Integer(2),Integer(4),Integer(8)],
↪Integer(4):[Integer(0),Integer(3),Integer(9)], Integer(5):[Integer(0),
↪Integer(7),Integer(8)],
...                          Integer(6):[Integer(1),Integer(8),Integer(9)],
↪Integer(7):[Integer(2),Integer(5),Integer(9)], Integer(8):[Integer(3),
↪Integer(5),Integer(6)],
...                          Integer(9):[Integer(4),Integer(6),Integer(7)]})
>>> petersen_spring.show()              # long time
↪# needs sage.plot
>>> petersen_database = graphs.PetersenGraph()
>>> petersen_database.show()            # long time
↪# needs sage.plot
```

static PoussinGraph(*immutable=False*)

Return the Poussin Graph.

For more information on the Poussin Graph, see its corresponding Wolfram page.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.PoussinGraph()
sage: g.order()
15
sage: g.is_planar()
True
```

```
>>> from sage.all import *
>>> g = graphs.PoussinGraph()
>>> g.order()
```

(continues on next page)

(continued from previous page)

```
15
>>> g.is_planar()
True
```

static QueenGraph(*dim_list*, *radius=None*, *relabel=False*, *immutable=False*)

Return the d-dimensional Queen Graph with prescribed dimensions.

The 2-dimensional Queen Graph of parameters n and m is a graph with nm vertices in which each vertex represents a square in an $n \times m$ chessboard, and each edge corresponds to a legal move by a queen.

The d-dimensional Queen Graph with $d >= 2$ has for vertex set the cells of a d-dimensional grid with prescribed dimensions, and each edge corresponds to a legal move by a queen in either one or two dimensions.

All 2-dimensional Queen Graphs are Hamiltonian and biconnected. The chromatic number of a (n, n)-Queen Graph is at least n, and it is exactly n when $n \equiv 1, 5 \bmod 6$.

INPUT:

- `dim_list` – iterable (list, set, dict); provides the dimensions $n_1, n_2, \ldots, n_d$, with $n_i \geq 1$, of the chessboard
- `radius` – integer (default: `None`); by setting the radius to a positive integer, one may reduce the visibility of the queen to at most `radius` steps. When radius is 1, the resulting graph is a King Graph.
- `relabel` – boolean (default: `False`); indicates whether the vertices must be relabeled as integers
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

The $(2, 2)$-Queen Graph is isomorphic to the complete graph on 4 vertices:

```
sage: G = graphs.QueenGraph([2, 2])
sage: G.is_isomorphic(graphs.CompleteGraph(4))
True
```

```
>>> from sage.all import *
>>> G = graphs.QueenGraph([Integer(2), Integer(2)])
>>> G.is_isomorphic(graphs.CompleteGraph(Integer(4)))
True
```

The Queen Graph with radius 1 is isomorphic to the King Graph:

```
sage: G = graphs.QueenGraph([4, 5], radius=1)
sage: H = graphs.KingGraph([5, 4])
sage: G.is_isomorphic(H)
True
```

```
>>> from sage.all import *
>>> G = graphs.QueenGraph([Integer(4), Integer(5)], radius=Integer(1))
>>> H = graphs.KingGraph([Integer(5), Integer(4)])
>>> G.is_isomorphic(H)
True
```

Also `True` in higher dimensions:

```
sage: G = graphs.QueenGraph([3, 4, 5], radius=1)
sage: H = graphs.KingGraph([5, 3, 4])
sage: G.is_isomorphic(H)
True
```

```
>>> from sage.all import *
>>> G = graphs.QueenGraph([Integer(3), Integer(4), Integer(5)],␣
↪radius=Integer(1))
>>> H = graphs.KingGraph([Integer(5), Integer(3), Integer(4)])
>>> G.is_isomorphic(H)
True
```

The Queen Graph can be obtained from the Rook Graph and the Bishop Graph:

```
sage: for d in range(3,12):   # long time
....:     for r in range(1,d+1):
....:         G = graphs.QueenGraph([d,d],radius=r)
....:         H = graphs.RookGraph([d,d],radius=r)
....:         B = graphs.BishopGraph([d,d],radius=r)
....:         H.add_edges(B.edges(sort=False))
....:         if not G.is_isomorphic(H):
....:             print("that's not good!")
```

```
>>> from sage.all import *
>>> for d in range(Integer(3),Integer(12)):   # long time
...     for r in range(Integer(1),d+Integer(1)):
...         G = graphs.QueenGraph([d,d],radius=r)
...         H = graphs.RookGraph([d,d],radius=r)
...         B = graphs.BishopGraph([d,d],radius=r)
...         H.add_edges(B.edges(sort=False))
...         if not G.is_isomorphic(H):
...             print("that's not good!")
```

static RandomBarabasiAlbert(*n*, *m*, *seed=None*, *immutable=False*)

Return a random graph created using the Barabasi-Albert preferential attachment model.

A graph with m vertices and no edges is initialized, and a graph of n vertices is grown by attaching new vertices each with m edges that are attached to existing vertices, preferentially with high degree.

INPUT:

- `n` – number of vertices in the graph
- `m` – number of edges to attach from each new node
- `seed` – a `random.Random` seed or a Python `int` for the random number generator (default: `None`)
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

We show the edge list of a random graph on 6 nodes with $m = 2$:

```
sage: G = graphs.RandomBarabasiAlbert(6,2)                                  ␣
↪  # needs networkx
sage: G.order(), G.size()                                                   ␣
```

(continues on next page)

(continued from previous page)

```
↪  # needs networkx
(6, 8)
sage: G.degree_sequence()  # random                                          ␣
↪  # needs networkx
[4, 3, 3, 2, 2, 2]
```

```
>>> from sage.all import *
>>> G = graphs.RandomBarabasiAlbert(Integer(6),Integer(2))                  ␣
↪                  # needs networkx
>>> G.order(), G.size()
↪# needs networkx
(6, 8)
>>> G.degree_sequence()  # random
↪# needs networkx
[4, 3, 3, 2, 2, 2]
```

We plot a random graph on 12 nodes with $m = 3$:

```
sage: ba = graphs.RandomBarabasiAlbert(12,3)                               ␣
↪  # needs networkx
sage: ba.show()                          # long time                       ␣
↪  # needs networkx sage.plot
```

```
>>> from sage.all import *
>>> ba = graphs.RandomBarabasiAlbert(Integer(12),Integer(3))                ␣
↪                  # needs networkx
>>> ba.show()                           # long time
↪# needs networkx sage.plot
```

We view many random graphs using a graphics array:

```
sage: # needs networkx sage.plot
sage: g = []
sage: j = []
sage: for i in range(1,10):
....:     k = graphs.RandomBarabasiAlbert(i+3, 3)
....:     g.append(k)
sage: for i in range(3):
....:     n = []
....:     for m in range(3):
....:         n.append(g[3*i + m].plot(vertex_size=50, vertex_labels=False))
....:     j.append(n)
sage: G = graphics_array(j)
sage: G.show()                           # long time
```

```
>>> from sage.all import *
>>> # needs networkx sage.plot
>>> g = []
>>> j = []
>>> for i in range(Integer(1),Integer(10)):
...     k = graphs.RandomBarabasiAlbert(i+Integer(3), Integer(3))
...     g.append(k)
```

(continues on next page)

(continued from previous page)

```
>>> for i in range(Integer(3)):
...     n = []
...     for m in range(Integer(3)):
...         n.append(g[Integer(3)*i + m].plot(vertex_size=Integer(50), vertex_
↪labels=False))
...     j.append(n)
>>> G = graphics_array(j)
>>> G.show()                              # long time
```

When $m = 1$, the generated graph is a tree:

```
sage: graphs.RandomBarabasiAlbert(6, 1).is_tree()                              ␣
↪  # needs networkx
True
```

```
>>> from sage.all import *
>>> graphs.RandomBarabasiAlbert(Integer(6), Integer(1)).is_tree()             ␣
↪                  # needs networkx
True
```

static RandomBicubicPlanar(*n*, *seed=None*, *immutable=False*)

Return the graph of a random bipartite cubic map with $3n$ edges.

INPUT:

- `n` – integer (at least 1)
- `seed` – a `random.Random` seed or a Python `int` for the random number generator (default: `None`)
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

OUTPUT:

a graph with multiple edges (no embedding is provided)

The algorithm used is described in [Sch1999]. This samples a random rooted bipartite cubic map, chosen uniformly at random.

First one creates a random binary tree with n vertices. Next one turns this into a blossoming tree (at random) and reads the contour word of this blossoming tree.

Then one performs a rotation on this word so that this becomes a balanced word. There are three ways to do that, one is picked at random. Then a graph is build from the balanced word by iterated closure (adding edges).

In the returned graph, the three edges incident to any given vertex are colored by the integers 0, 1 and 2.

See also

the auxiliary function *blossoming_contour()*

EXAMPLES:

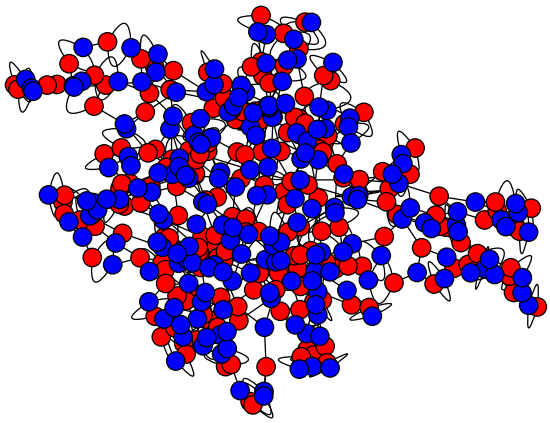
```
sage: n = randint(200, 300)
sage: G = graphs.RandomBicubicPlanar(n)
sage: G.order() == 2*n
```

(continues on next page)

(continued from previous page)

```
True
sage: G.size() == 3*n
True
sage: G.is_bipartite() and G.is_planar() and G.is_regular(3)
True
sage: dic = {'red': [v for v in G.vertices(sort=False) if v[0] == 'n'],
....:        'blue': [v for v in G.vertices(sort=False) if v[0] != 'n']}
sage: G.plot(vertex_labels=False, vertex_size=20, vertex_colors=dic)
↪  # needs sage.plot
Graphics object consisting of ... graphics primitives
```

```
>>> from sage.all import *
>>> n = randint(Integer(200), Integer(300))
>>> G = graphs.RandomBicubicPlanar(n)
>>> G.order() == Integer(2)*n
True
>>> G.size() == Integer(3)*n
True
>>> G.is_bipartite() and G.is_planar() and G.is_regular(Integer(3))
True
>>> dic = {'red': [v for v in G.vertices(sort=False) if v[Integer(0)] == 'n'],
...        'blue': [v for v in G.vertices(sort=False) if v[Integer(0)] != 'n
↪']}
>>> G.plot(vertex_labels=False, vertex_size=Integer(20), vertex_colors=dic)
↪         # needs sage.plot
Graphics object consisting of ... graphics primitives
```



static RandomBipartite(*n1*, *n2*, *p*, *set_position=False*, *seed=None*, *immutable=False*)

Return a bipartite graph with $n1 + n2$ vertices such that any edge from $[n1]$ to $[n2]$ exists with probability p.

INPUT:

- `n1`, `n2` – cardinalities of the two sets
- `p` – probability for an edge to exist
- `set_position` – boolean (default: `False`); if set to `True`, we assign positions to the vertices so that the set of cardinality $n1$ is on the line $y = 1$ and the set of cardinality $n2$ is on the line $y = 0$

- `seed` – a `random.Random` seed or a Python `int` for the random number generator (default: `None`)
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.RandomBipartite(5, 2, 0.5)
↪  # needs numpy
sage: g.vertices(sort=True)
↪  # needs numpy
[(0, 0), (0, 1), (0, 2), (0, 3), (0, 4), (1, 0), (1, 1)]
```

```
>>> from sage.all import *
>>> g = graphs.RandomBipartite(Integer(5), Integer(2), RealNumber('0.5'))
↪                                  # needs numpy
>>> g.vertices(sort=True)
↪# needs numpy
[(0, 0), (0, 1), (0, 2), (0, 3), (0, 4), (1, 0), (1, 1)]
```

static RandomBlockGraph(*m*, *k*, *kmax=None*, *incidence_structure=False*, *seed=None*, *immutable=False*)

Return a Random Block Graph.

A block graph is a connected graph in which every biconnected component (block) is a clique.

See also

- Wikipedia article Block_graph for more details on these graphs
- *is_block_graph()* – test if a graph is a block graph
- *blocks_and_cut_vertices()*
- *blocks_and_cuts_tree()*
- *IncidenceStructure()*

INPUT:

- `m` – integer; number of blocks (at least one)
- `k` – integer; minimum number of vertices of a block (at least two)
- `kmax` – integer (default: `None`); by default, each block has k vertices. When the parameter $kmax$ is specified (with $kmax \geq k$), the number of vertices of each block is randomly chosen between k and $kmax$.
- `incidence_structure` – boolean (default: `False`); when set to `True`, the incidence structure of the graphs is returned instead of the graph itself, that is the list of the lists of vertices in each block. This is useful for the creation of some hypergraphs.
- `seed` – a `random.Random` seed or a Python `int` for the random number generator (default: `None`)
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

OUTPUT:

A Graph when `incidence_structure==False` (default), and otherwise an incidence structure.

EXAMPLES:

A block graph with a single block is a clique:

```
sage: B = graphs.RandomBlockGraph(1, 4)
sage: B.is_clique()
True
```

```
>>> from sage.all import *
>>> B = graphs.RandomBlockGraph(Integer(1), Integer(4))
>>> B.is_clique()
True
```

A block graph with blocks of order 2 is a tree:

```
sage: B = graphs.RandomBlockGraph(10, 2)
sage: B.is_tree()
True
```

```
>>> from sage.all import *
>>> B = graphs.RandomBlockGraph(Integer(10), Integer(2))
>>> B.is_tree()
True
```

Every biconnected component of a block graph is a clique:

```
sage: B = graphs.RandomBlockGraph(5, 3, kmax=6)
sage: blocks,cuts = B.blocks_and_cut_vertices()
sage: all(B.is_clique(block) for block in blocks)
True
```

```
>>> from sage.all import *
>>> B = graphs.RandomBlockGraph(Integer(5), Integer(3), kmax=Integer(6))
>>> blocks,cuts = B.blocks_and_cut_vertices()
>>> all(B.is_clique(block) for block in blocks)
True
```

A block graph with blocks of order k has $m * (k-1) + 1$ vertices:

```
sage: m, k = 6, 4
sage: B = graphs.RandomBlockGraph(m, k)
sage: B.order() == m*(k-1)+1
True
```

```
>>> from sage.all import *
>>> m, k = Integer(6), Integer(4)
>>> B = graphs.RandomBlockGraph(m, k)
>>> B.order() == m*(k-Integer(1))+Integer(1)
True
```

Test recognition methods:

```
sage: B = graphs.RandomBlockGraph(6, 2, kmax=6)
sage: B.is_block_graph()
True
sage: B in graph_classes.Block
True
```

```
>>> from sage.all import *
>>> B = graphs.RandomBlockGraph(Integer(6), Integer(2), kmax=Integer(6))
>>> B.is_block_graph()
True
>>> B in graph_classes.Block
True
```

Asking for the incidence structure:

```
sage: m, k = 6, 4
sage: IS = graphs.RandomBlockGraph(m, k, incidence_structure=True)
sage: from sage.combinat.designs.incidence_structures import␣
↪IncidenceStructure
sage: IncidenceStructure(IS)                                                ␣
↪  # needs sage.modules
Incidence structure with 19 points and 6 blocks
sage: m*(k-1)+1
19
```

```
>>> from sage.all import *
>>> m, k = Integer(6), Integer(4)
>>> IS = graphs.RandomBlockGraph(m, k, incidence_structure=True)
>>> from sage.combinat.designs.incidence_structures import IncidenceStructure
>>> IncidenceStructure(IS)
↪# needs sage.modules
Incidence structure with 19 points and 6 blocks
>>> m*(k-Integer(1))+Integer(1)
19
```

static RandomBoundedToleranceGraph(*n*, *seed=None*, *immutable=False*)

Return a random bounded tolerance graph.

The random tolerance graph is built from a random bounded tolerance representation by using the function $ToleranceGraph$. This representation is a list $((l_0, r_0, t_0), (l_1, r_1, t_1), ..., (l_k, r_k, t_k))$ where $k = n - 1$ and $I_i = (l_i, r_i)$ denotes a random interval and t_i a random positive value less than or equal to the length of the interval I_i. The width of the representation is limited to $n^2 * 2^n$.

Note

The tolerance representation used to create the graph can be recovered using `get_vertex()` or `get_vertices()`.

INPUT:

- `n` – number of vertices of the random graph
- `seed` – a `random.Random` seed or a Python `int` for the random number generator (default: `None`)
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

Every (bounded) tolerance graph is perfect. Hence, the chromatic number is equal to the clique number

```
sage: g = graphs.RandomBoundedToleranceGraph(8)
sage: g.clique_number() == g.chromatic_number()
True
```

```
>>> from sage.all import *
>>> g = graphs.RandomBoundedToleranceGraph(Integer(8))
>>> g.clique_number() == g.chromatic_number()
True
```

static RandomChordalGraph(*n*, *algorithm='growing'*, *k=None*, *l=None*, *f=None*, *s=None*, *seed=None*, *immutable=False*)

Return a random chordal graph of order `n`.

A Graph G is said to be chordal if it contains no induced hole (a cycle of length at least 4). Equivalently, G is chordal if it has a perfect elimination orderings, if each minimal separator is a clique, or if it is the intersection graphs of subtrees of a tree. See the Wikipedia article Chordal_graph.

This generator implements the algorithms proposed in [SHET2018] for generating random chordal graphs as the intersection graph of n subtrees of a tree of order n.

The returned graph is not necessarily connected.

INPUT:

- `n` – integer; the number of nodes of the graph
- `algorithm` – string (default: `'growing'`); the choice of the algorithm for randomly selecting n subtrees of a random tree of order n. Possible choices are:
  - `'growing'` – for each subtree T_i, the algorithm picks a size k_i randomly from $[1, k]$. Then a random node of T is chosen as the first node of T_i. In each of the subsequent $k_i - 1$ iterations, it picks a random node in the neighborhood of T_i and adds it to T_i.
  - `'connecting'` – for each subtree T_i, it first selects k_i nodes of T, where k_i is a random integer from a Poisson distribution with mean l. T_i is then generated to be the minimal subtree containing the selected k_i nodes. This implies that a subtree will most likely have many more nodes than those selected initially, and this must be taken into consideration when choosing l.
  - `'pruned'` – for each subtree T_i, it randomly selects a fraction f of the edges on the tree and removes them. The number of edges to delete, say l, is calculated as $\lfloor (n-1)f \rfloor$, which will leave $l+1$ subtrees in total. Then, it determines the sizes of the $l+1$ subtrees and stores the distinct values. Finally, it picks a random size k_i from the set of largest $100(1-s)\%$ of distinct values, and randomly chooses a subtree with size k_i.
- `k` – integer (default: `None`); maximum size of a subtree. If not specified (`None`), the maximum size is set to $\sqrt{n}$. This parameter is used only when `algorithm="growing"`. See *growing_subtrees()* for more details.
- `l` – a strictly positive real number (default: `None`); mean of a Poisson distribution. If not specified, the mean in set to $\log_2 n$. This parameter is used only when `algorithm="connecting"`. See *connecting_nodes()* for more details.
- `f` – a rational number (default: `None`); the edge deletion fraction. This value must be chosen in $[0..1]$. If not specified, this parameter is set to $\frac{1}{n-1}$. This parameter is used only when `algorithm="pruned"`. See *pruned_tree()* for more details.
- `s` – a real number between 0 and 1 (default: `None`); selection barrier for the size of trees. If not specified, this parameter is set to 0.5. This parameter is used only when `algorithm="pruned"`. See *pruned_tree()* for more details.

- `seed` – a `random.Random` seed or a Python `int` for the random number generator (default: `None`)
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: from sage.graphs.generators.random import RandomChordalGraph
sage: T = RandomChordalGraph(20, algorithm='growing', k=5)
sage: T.is_chordal()
True
sage: T = RandomChordalGraph(20, algorithm='connecting', l=3)
↪  # needs numpy
sage: T.is_chordal()
↪  # needs numpy
True
sage: T = RandomChordalGraph(20, algorithm='pruned', f=1/3, s=.5)
sage: T.is_chordal()
True
```

```
>>> from sage.all import *
>>> from sage.graphs.generators.random import RandomChordalGraph
>>> T = RandomChordalGraph(Integer(20), algorithm='growing', k=Integer(5))
>>> T.is_chordal()
True
>>> T = RandomChordalGraph(Integer(20), algorithm='connecting', l=Integer(3))
↪                  # needs numpy
>>> T.is_chordal()
↪# needs numpy
True
>>> T = RandomChordalGraph(Integer(20), algorithm='pruned', f=Integer(1)/
↪Integer(3), s=RealNumber('.5'))
>>> T.is_chordal()
True
```

See also

- *growing_subtrees()*
- *connecting_nodes()*
- *pruned_tree()*
- Wikipedia article Chordal_graph
- *is_chordal()*
- *IntersectionGraph()*

static RandomGNM(*n*, *m*, *dense=False*, *seed=None*, *immutable=False*)

Return a graph randomly picked out of all graphs on n vertices with m edges.

INPUT:

- `n` – number of vertices
- `m` – number of edges

- `dense` – whether to use NetworkX's `dense_gnm_random_graph()` or `gnm_random_graph()`
- `seed` – a `random.Random` seed or a Python `int` for the random number generator (default: `None`)
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

We show the edge list of a random graph on 5 nodes with 10 edges:

```
sage: graphs.RandomGNM(5, 10).edges(sort=True, labels=False)
↪  # needs networkx
[(0, 1), (0, 2), (0, 3), (0, 4), (1, 2), (1, 3), (1, 4), (2, 3), (2, 4), (3,
↪4)]
```

```
>>> from sage.all import *
>>> graphs.RandomGNM(Integer(5), Integer(10)).edges(sort=True, labels=False)
↪                  # needs networkx
[(0, 1), (0, 2), (0, 3), (0, 4), (1, 2), (1, 3), (1, 4), (2, 3), (2, 4), (3,
↪4)]
```

We plot a random graph on 12 nodes and 12 edges:

```
sage: gnm = graphs.RandomGNM(12, 12)
↪  # needs networkx
sage: gnm.show()                          # long time
↪  # needs networkx sage.plot
```

```
>>> from sage.all import *
>>> gnm = graphs.RandomGNM(Integer(12), Integer(12))
↪                  # needs networkx
>>> gnm.show()                          # long time
↪# needs networkx sage.plot
```

We view many random graphs using a graphics array:

```
sage: # needs networkx sage.plot
sage: g = []
sage: j = []
sage: for i in range(9):
....:     k = graphs.RandomGNM(i+3, i^2-i)
....:     g.append(k)
sage: for i in range(3):
....:     n = []
....:     for m in range(3):
....:         n.append(g[3*i + m].plot(vertex_size=50, vertex_labels=False))
....:     j.append(n)
sage: G = graphics_array(j)
sage: G.show()                            # long time
```

```
>>> from sage.all import *
>>> # needs networkx sage.plot
>>> g = []
>>> j = []
>>> for i in range(Integer(9)):
```

(continues on next page)

(continued from previous page)

```
...       k = graphs.RandomGNM(i+Integer(3), i**Integer(2)-i)
...       g.append(k)
>>> for i in range(Integer(3)):
...       n = []
...       for m in range(Integer(3)):
...           n.append(g[Integer(3)*i + m].plot(vertex_size=Integer(50), vertex_
↪labels=False))
...       j.append(n)
>>> G = graphics_array(j)
>>> G.show()                              # long time
```

static RandomGNP(*n*, *p*, *seed=None*, *fast=True*, *algorithm='Sage'*, *immutable=False*)

Return a random graph on n nodes. Each edge is inserted independently with probability p.

INPUT:

- `n` – number of nodes of the graph
- `p` – probability of an edge
- `seed` – a `random.Random` seed or a Python `int` for the random number generator (default: `None`)
- `fast` – boolean (default: `True`) to use the algorithm with time complexity in $O(n + m)$ proposed in [BB2005a]. It is designed for generating large sparse graphs. It is faster than other algorithms for *LARGE* instances (try it to know whether it is useful for you).
- `algorithm` – (default: `'Sage'`) this function uses the algorithm implemented in `sage.graphs.graph_generators_pyx.pyx`. When `algorithm='networkx'`, this function calls the NetworkX function `fast_gnp_random_graph`, unless `fast=False`, then `gnp_random_graph`. Try them to know which algorithm is the best for you. The `fast` parameter is not taken into account by the 'Sage' algorithm so far.
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

REFERENCES:

- [ER1959]
- [Gil1959]

PLOTTING: When plotting, this graph will use the default spring-layout algorithm, unless a position dictionary is specified.

EXAMPLES: We show the edge list of a random graph on 6 nodes with probability $p = .4$:

```
sage: set_random_seed(0)
sage: graphs.RandomGNP(6, .4).edges(sort=true, labels=False)
[(0, 3), (1, 2), (2, 3), (2, 4)]
```

```
>>> from sage.all import *
>>> set_random_seed(Integer(0))
>>> graphs.RandomGNP(Integer(6), RealNumber('.4')).edges(sort=true,␣
↪labels=False)
[(0, 3), (1, 2), (2, 3), (2, 4)]
```

We plot a random graph on 12 nodes with probability $p = .71$:

```
sage: gnp = graphs.RandomGNP(12,.71)
sage: gnp.show()                          # long time
↪  # needs sage.plot
```

```
>>> from sage.all import *
>>> gnp = graphs.RandomGNP(Integer(12),RealNumber('.71'))
>>> gnp.show()                          # long time
↪# needs sage.plot
```

We view many random graphs using a graphics array:

```
sage: g = []
sage: j = []
sage: for i in range(9):
....:     k = graphs.RandomGNP(i+3,.43)
....:     g.append(k)
sage: for i in range(3):
↪  # needs sage.plot
....:     n = []
....:     for m in range(3):
....:         n.append(g[3*i + m].plot(vertex_size=50, vertex_labels=False))
....:     j.append(n)
sage: G = graphics_array(j)
↪  # needs sage.plot
sage: G.show()                          # long time
↪  # needs sage.plot
sage: graphs.RandomGNP(4,1)
Complete graph: Graph on 4 vertices
```

```
>>> from sage.all import *
>>> g = []
>>> j = []
>>> for i in range(Integer(9)):
...     k = graphs.RandomGNP(i+Integer(3),RealNumber('.43'))
...     g.append(k)
>>> for i in range(Integer(3)):
↪         # needs sage.plot
...     n = []
...     for m in range(Integer(3)):
...         n.append(g[Integer(3)*i + m].plot(vertex_size=Integer(50), vertex_
↪labels=False))
...     j.append(n)
>>> G = graphics_array(j)
↪# needs sage.plot
>>> G.show()                          # long time
↪# needs sage.plot
>>> graphs.RandomGNP(Integer(4),Integer(1))
Complete graph: Graph on 4 vertices
```

static RandomHolmeKim(*n*, *m*, *p*, *seed=None*, *immutable=False*)

Return a random graph generated by the Holme and Kim algorithm for graphs with power law degree distribution and approximate average clustering.

INPUT:

- `n` – number of vertices
- `m` – number of random edges to add for each new node
- `p` – probability of adding a triangle after adding a random edge
- `seed` – a `random.Random` seed or a Python `int` for the random number generator (default: `None`)
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

From the NetworkX documentation: the average clustering has a hard time getting above a certain cutoff that depends on m. This cutoff is often quite low. Note that the transitivity (fraction of triangles to possible triangles) seems to go down with network size. It is essentially the Barabasi-Albert growth model with an extra step that each random edge is followed by a chance of making an edge to one of its neighbors too (and thus a triangle). This algorithm improves on B-A in the sense that it enables a higher average clustering to be attained if desired. It seems possible to have a disconnected graph with this algorithm since the initial m nodes may not be all linked to a new node on the first iteration like the BA model.

EXAMPLES:

```
sage: G = graphs.RandomHolmeKim(12, 3, .3)                                  ␣
↪  # needs networkx
sage: G.show()                              # long time                     ␣
↪  # needs networkx sage.plot
```

```
>>> from sage.all import *
>>> G = graphs.RandomHolmeKim(Integer(12), Integer(3), RealNumber('.3'))    ␣
↪                                  # needs networkx
>>> G.show()                                # long time
↪# needs networkx sage.plot
```

REFERENCE:

[HK2002a]

static RandomIntervalGraph(*n*, *seed=None*, *immutable=False*)

Return a random interval graph.

An interval graph is built from a list $(a_i, b_i)_{1 \leq i \leq n}$ of intervals : to each interval of the list is associated one vertex, two vertices being adjacent if the two corresponding intervals intersect.

A random interval graph of order n is generated by picking random values for the (a_i, b_j), each of the two coordinates being generated from the uniform distribution on the interval $[0, 1]$.

This definitions follows [BF2001].

Note

The vertices are named 0, 1, 2, and so on. The intervals used to create the graph are saved with the graph and can be recovered using `get_vertex()` or `get_vertices()`.

See also

- *sage.graphs.generators.intersection.IntervalGraph()*

- *sage.graphs.generators.random.RandomProperIntervalGraph()*

INPUT:

- `n` – integer; the number of vertices in the random graph
- `seed` – a `random.Random` seed or a Python `int` for the random number generator (default: `None`)
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

As for any interval graph, the chromatic number is equal to the clique number

```
sage: g = graphs.RandomIntervalGraph(8)
sage: g.clique_number() == g.chromatic_number()
True
```

```
>>> from sage.all import *
>>> g = graphs.RandomIntervalGraph(Integer(8))
>>> g.clique_number() == g.chromatic_number()
True
```

static RandomKTree(*n*, *k*, *seed=None*, *immutable=False*)

Return a random k-tree on n nodes numbered 0 through $n-1$.

ALGORITHM:

The algorithm first generates a complete graph on $k+1$ vertices. Vertices are subsequently generated by randomly choosing one of the existing cliques in the graph, and creating a new clique by replacing one of the vertices in the selected clique with a newly created one.

INPUT:

- `n` – number of vertices in the k-tree
- `k` – within a clique each vertex is connected to k vertices. k also corresponds to the treewidth of the k-tree
- `seed` – a `random.Random` seed or a Python `int` for the random number generator (default: `None`)
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: G = graphs.RandomKTree(50, 5)
sage: G.treewidth()
5
sage: G.show()  # not tested
```

```
>>> from sage.all import *
>>> G = graphs.RandomKTree(Integer(50), Integer(5))
>>> G.treewidth()
5
>>> G.show()  # not tested
```

static RandomLobster(*n*, *p*, *q*, *seed=None*, *immutable=False*)

Return a random lobster.

A lobster is a tree that reduces to a caterpillar when pruning all leaf vertices. A caterpillar is a tree that reduces to a path when pruning all leaf vertices ($q = 0$).

INPUT:

- `n` – expected number of vertices in the backbone
- `p` – probability of adding an edge to the backbone
- `q` – probability of adding an edge (claw) to the arms
- `seed` – a `random.Random` seed or a Python `int` for the random number generator (default: `None`)
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

We check a random graph with 12 backbone nodes and probabilities $p = 0.7$ and $q = 0.3$:

```
sage: # needs networkx
sage: G = graphs.RandomLobster(12, 0.7, 0.3)
sage: leaves = [v for v in G.vertices(sort=False) if G.degree(v) == 1]
sage: G.delete_vertices(leaves)                                    # caterpillar
sage: leaves = [v for v in G.vertices(sort=False) if G.degree(v) == 1]
sage: G.delete_vertices(leaves)                                    # path
sage: s = G.degree_sequence()
sage: if G:
....:     if G.n_vertices() == 1:
....:         assert s == [0]
....:     else:
....:         assert s[-2:] == [1, 1]
....:     assert all(d == 2 for d in s[:-2])
```

```
>>> from sage.all import *
>>> # needs networkx
>>> G = graphs.RandomLobster(Integer(12), RealNumber('0.7'), RealNumber('0.3
↪'))
>>> leaves = [v for v in G.vertices(sort=False) if G.degree(v) == Integer(1)]
>>> G.delete_vertices(leaves)                                    # caterpillar
>>> leaves = [v for v in G.vertices(sort=False) if G.degree(v) == Integer(1)]
>>> G.delete_vertices(leaves)                                    # path
>>> s = G.degree_sequence()
>>> if G:
...     if G.n_vertices() == Integer(1):
...         assert s == [Integer(0)]
...     else:
...         assert s[-Integer(2):] == [Integer(1), Integer(1)]
...     assert all(d == Integer(2) for d in s[:-Integer(2)])
```

```
sage: G = graphs.RandomLobster(9, .6, .3)                                  ␣
↪  # needs networkx
sage: G.show()                          # long time                        ␣
↪  # needs networkx sage.plot
```

```
>>> from sage.all import *
>>> G = graphs.RandomLobster(Integer(9), RealNumber('.6'), RealNumber('.3')) ␣
```

(continues on next page)

(continued from previous page)

```
↪                                          # needs networkx
>>> G.show()                               # long time
↪# needs networkx sage.plot
```

static RandomNewmanWattsStrogatz(*n*, *k*, *p*, *seed=None*, *immutable=False*)

Return a Newman-Watts-Strogatz small world random graph on n vertices.

From the NetworkX documentation: first create a ring over n nodes. Then each node in the ring is connected with its k nearest neighbors. Then shortcuts are created by adding new edges as follows: for each edge $u - v$ in the underlying "n-ring with k nearest neighbors"; with probability p add a new edge $u - w$ with randomly-chosen existing node w. In contrast with `networkx.watts_strogatz_graph()`, no edges are removed.

INPUT:

- `n` – number of vertices
- `k` – each vertex is connected to its k nearest neighbors
- `p` – the probability of adding a new edge for each edge
- `seed` – a `random.Random` seed or a Python `int` for the random number generator (default: `None`)
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

We check that the generated graph contains a cycle of order n:

```
sage: # needs networkx
sage: G = graphs.RandomNewmanWattsStrogatz(7, 2, 0.2)
sage: G.order()
7
sage: C7 = graphs.CycleGraph(7)
sage: G.subgraph_search(C7)
Subgraph of (): Graph on 7 vertices
sage: G.diameter() <= C7.diameter()
True
```

```
>>> from sage.all import *
>>> # needs networkx
>>> G = graphs.RandomNewmanWattsStrogatz(Integer(7), Integer(2), RealNumber(
↪'0.2'))
>>> G.order()
7
>>> C7 = graphs.CycleGraph(Integer(7))
>>> G.subgraph_search(C7)
Subgraph of (): Graph on 7 vertices
>>> G.diameter() <= C7.diameter()
True
```

```
sage: G = graphs.RandomNewmanWattsStrogatz(12, 2, .3)                    ␣
↪  # needs networkx
sage: G.show()                              # long time                  ␣
↪  # needs networkx sage.plot
```

```
>>> from sage.all import *
>>> G = graphs.RandomNewmanWattsStrogatz(Integer(12), Integer(2), RealNumber(
↪'.3'))                                  # needs networkx
>>> G.show()                                # long time
↪# needs networkx sage.plot
```

REFERENCE:

[NWS2002]

static RandomPartialKTree(*n*, *k*, *x*, *seed=None*, *immutable=False*)

Return a random partial k-tree on n nodes.

A partial k-tree is defined as a subgraph of a k-tree. This can also be described as a graph with treewidth at most k.

INPUT:

- `n` – number of vertices in the k-tree
- `k` – within a clique each vertex is connected to k vertices. k also corresponds to the treewidth of the k-tree
- `x` – how many edges are deleted from the k-tree
- `seed` – a `random.Random` seed or a Python `int` for the random number generator (default: `None`)
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: G = graphs.RandomPartialKTree(50,5,2)
sage: G.treewidth()
5
sage: G.show()  # not tested
```

```
>>> from sage.all import *
>>> G = graphs.RandomPartialKTree(Integer(50),Integer(5),Integer(2))
>>> G.treewidth()
5
>>> G.show()  # not tested
```

static RandomProperIntervalGraph(*n*, *seed=None*, *immutable=False*)

Return a random proper interval graph.

An interval graph is built from a list $(a_i, b_i)_{1 \leq i \leq n}$ of intervals : to each interval of the list is associated one vertex, two vertices being adjacent if the two corresponding (closed) intervals intersect. An interval graph is proper if no interval of the list properly contains another interval. Observe that proper interval graphs coincide with unit interval graphs. See the Wikipedia article Interval_graph for more details.

This method implements the random proper interval graph generator proposed in [SYKU2010] which outputs graphs with uniform probability. The time complexity of this generator is in $O(n^3)$.

Note

The vertices are named 0, 1, 2, and so on. The intervals used to create the graph are saved with the graph and can be recovered using `get_vertex()` or `get_vertices()`.

See also

- *sage.graphs.generators.intersection.IntervalGraph()*
- *sage.graphs.generators.random.RandomIntervalGraph()*

INPUT:

- `n` – positive integer; the number of vertices of the graph
- `seed` – a `random.Random` seed or a Python `int` for the random number generator (default: `None`)
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: from sage.graphs.generators.random import RandomProperIntervalGraph
sage: G = RandomProperIntervalGraph(10)
sage: G.is_interval()
True
```

```
>>> from sage.all import *
>>> from sage.graphs.generators.random import RandomProperIntervalGraph
>>> G = RandomProperIntervalGraph(Integer(10))
>>> G.is_interval()
True
```

static RandomRegular(*d*, *n*, *seed=None*, *immutable=False*)

Return a random d-regular graph on n vertices, or `False` on failure.

Since every edge is incident to two vertices, $n \times d$ must be even.

INPUT:

- `d` – degree
- `n` – number of vertices
- `seed` – a `random.Random` seed or a Python `int` for the random number generator (default: `None`)
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

We check that a random graph with 8 nodes each of degree 3 is 3-regular:

```
sage: G = graphs.RandomRegular(3, 8)
↪  # needs networkx
sage: G.is_regular(k=3)
↪  # needs networkx
True
sage: G.degree_histogram()
↪  # needs networkx
[0, 0, 0, 8]
```

```
>>> from sage.all import *
>>> G = graphs.RandomRegular(Integer(3), Integer(8))
↪                  # needs networkx
```

(continues on next page)

(continued from previous page)

```
>>> G.is_regular(k=Integer(3))
↪          # needs networkx
True
>>> G.degree_histogram()
↪# needs networkx
[0, 0, 0, 8]
```

```
sage: G = graphs.RandomRegular(3, 20)
↪  # needs networkx
sage: if G:                                # random output          # long time,
↪needs networkx sage.plot
....:     G.show()
```

```
>>> from sage.all import *
>>> G = graphs.RandomRegular(Integer(3), Integer(20))
↪                  # needs networkx
>>> if G:                                # random output          # long time,
↪needs networkx sage.plot
...     G.show()
```

REFERENCES:

- [KV2003]
- [SW1999]

static RandomRegularBipartite(*n1*, *n2*, *d1*, *set_position=False*, *seed=None*, *immutable=False*)

Return a random regular bipartite graph on $n1 + n2$ vertices.

The bipartite graph has $n1 * d1$ edges. Hence, $n2$ must divide $n1 * d1$. Each vertex of the set of cardinality $n1$ has degree $d1$ (which can be at most $n2$) and each vertex in the set of cardinality $n2$ has degree $(n1 * d1)/n2$. The bipartite graph has no multiple edges.

This generator implements an algorithm inspired by that of [MW1990] for the uniform generation of random regular bipartite graphs. It performs well when $d1 = o(n2^{1/3})$ or $(n2 - d1 = o(n2^{1/3}))$. In other cases, the running time can be huge. Note that the currently implemented algorithm does not generate uniformly random graphs.

INPUT:

- `n1`, `n2` – number of vertices in each side
- `d1` – degree of the vertices in the set of cardinality $n1$
- `set_position` – boolean (default: `False`); if set to `True`, we assign positions to the vertices so that the set of cardinality $n1$ is on the line $y = 1$ and the set of cardinality $n2$ is on the line $y = 0$.
- `seed` – a `random.Random` seed or a Python `int` for the random number generator (default: `None`)
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.RandomRegularBipartite(4, 6, 3)
sage: g.order(), g.size()
(10, 12)
sage: set(g.degree())
```

(continues on next page)

(continued from previous page)

```
{2, 3}

sage: graphs.RandomRegularBipartite(1, 2, 2, set_position=True).get_pos()
{0: (1, 1.0), 1: (0, 0), 2: (2.0, 0.0)}
sage: graphs.RandomRegularBipartite(2, 1, 1, set_position=True).get_pos()
{0: (0, 1), 1: (2.0, 1.0), 2: (1, 0.0)}
sage: graphs.RandomRegularBipartite(2, 3, 3, set_position=True).get_pos()
{0: (0, 1), 1: (3.0, 1.0), 2: (0, 0), 3: (1.5, 0.0), 4: (3.0, 0.0)}
sage: graphs.RandomRegularBipartite(2, 3, 3, set_position=False).get_pos()
```

```
>>> from sage.all import *
>>> g = graphs.RandomRegularBipartite(Integer(4), Integer(6), Integer(3))
>>> g.order(), g.size()
(10, 12)
>>> set(g.degree())
{2, 3}

>>> graphs.RandomRegularBipartite(Integer(1), Integer(2), Integer(2), set_
↪position=True).get_pos()
{0: (1, 1.0), 1: (0, 0), 2: (2.0, 0.0)}
>>> graphs.RandomRegularBipartite(Integer(2), Integer(1), Integer(1), set_
↪position=True).get_pos()
{0: (0, 1), 1: (2.0, 1.0), 2: (1, 0.0)}
>>> graphs.RandomRegularBipartite(Integer(2), Integer(3), Integer(3), set_
↪position=True).get_pos()
{0: (0, 1), 1: (3.0, 1.0), 2: (0, 0), 3: (1.5, 0.0), 4: (3.0, 0.0)}
>>> graphs.RandomRegularBipartite(Integer(2), Integer(3), Integer(3), set_
↪position=False).get_pos()
```

static RandomShell(*constructor*, *seed=None*, *immutable=False*)

Return a random shell graph for the constructor given.

INPUT:

- `constructor` – list of 3-tuples (n, m, d), each representing a shell, where:
  - `n` – the number of vertices in the shell
  - `m` – the number of edges in the shell
  - `d` – the ratio of inter (next) shell edges to intra shell edges
- `seed` – a `random.Random` seed or a Python `int` for the random number generator (default: `None`)
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: G = graphs.RandomShell([(10,20,0.8),(20,40,0.8)])
↪  # needs networkx
sage: G.order(), G.size()
↪  # needs networkx
(30, 52)
sage: G.show()                          # long time
↪  # needs networkx sage.plot
```

```
>>> from sage.all import *
>>> G = graphs.RandomShell([(Integer(10),Integer(20),RealNumber('0.8')),
↪(Integer(20),Integer(40),RealNumber('0.8'))])                          #␣
↪needs networkx
>>> G.order(), G.size()
↪# needs networkx
(30, 52)
>>> G.show()                              # long time
↪# needs networkx sage.plot
```

static RandomToleranceGraph(*n*, *seed=None*, *immutable=False*)

Return a random tolerance graph.

The random tolerance graph is built from a random tolerance representation by using the function *ToleranceGraph()*. This representation is a list $((l_0, r_0, t_0), (l_1, r_1, t_1), ..., (l_k, r_k, t_k))$ where $k = n - 1$ and $I_i = (l_i, r_i)$ denotes a random interval and t_i a random positive value. The width of the representation is limited to $n^2 * 2^n$.

Note

The vertices are named $0, 1, \cdots, n-1$. The tolerance representation used to create the graph is saved with the graph and can be recovered using *get_vertex()* or *get_vertices()*.

INPUT:

- `n` – number of vertices of the random graph
- `seed` – a `random.Random` seed or a Python `int` for the random number generator (default: `None`)
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

Every tolerance graph is perfect. Hence, the chromatic number is equal to the clique number

```
sage: g = graphs.RandomToleranceGraph(8)
sage: g.clique_number() == g.chromatic_number()
True
```

```
>>> from sage.all import *
>>> g = graphs.RandomToleranceGraph(Integer(8))
>>> g.clique_number() == g.chromatic_number()
True
```

static RandomTree(*n*, *seed=None*, *immutable=False*)

Return a random tree on n nodes numbered 0 through $n - 1$.

By Cayley's theorem, there are n^{n-2} trees with vertex set $\{0, 1, \ldots, n-1\}$. This constructor chooses one of these uniformly at random.

ALGORITHM:

The algorithm works by generating an $(n-2)$-long random sequence of numbers chosen independently and uniformly from $\{0, 1, \ldots, n-1\}$ and then applies an inverse Prufer transformation.

INPUT:

- `n` – number of vertices in the tree
- `seed` – a `random.Random` seed or a Python `int` for the random number generator (default: `None`)
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: G = graphs.RandomTree(10)
sage: G.is_tree()
True
sage: G.show()                              # long time
↪  # needs sage.plot
```

```
>>> from sage.all import *
>>> G = graphs.RandomTree(Integer(10))
>>> G.is_tree()
True
>>> G.show()                             # long time
↪# needs sage.plot
```

static RandomTreePowerlaw(*n*, *gamma=3*, *tries=1000*, *seed=None*, *immutable=False*)

Return a tree with a power law degree distribution, or `False` on failure.

From the NetworkX documentation: a trial power law degree sequence is chosen and then elements are swapped with new elements from a power law distribution until the sequence makes a tree (size = order - 1).

INPUT:

- `n` – number of vertices
- `gamma` – exponent of power law distribution
- `tries` – number of attempts to adjust sequence to make a tree
- `seed` – a `random.Random` seed or a Python `int` for the random number generator (default: `None`)
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

We check that the generated graph is a tree:

```
sage: G = graphs.RandomTreePowerlaw(10, 3)
↪  # needs networkx
sage: G.is_tree()
↪  # needs networkx
True
sage: G.order(), G.size()
↪  # needs networkx
(10, 9)
```

```
>>> from sage.all import *
>>> G = graphs.RandomTreePowerlaw(Integer(10), Integer(3))
↪                  # needs networkx
>>> G.is_tree()
↪# needs networkx
True
>>> G.order(), G.size()
```

(continues on next page)

(continued from previous page)

```
↪# needs networkx
(10, 9)
```

```
sage: G = graphs.RandomTreePowerlaw(15, 2)                                  ␣
↪  # needs networkx
sage: if G:                                 # random output          # long time,␣
↪needs networkx sage.plot
....:     G.show()
```

```
>>> from sage.all import *
>>> G = graphs.RandomTreePowerlaw(Integer(15), Integer(2))                  ␣
↪                  # needs networkx
>>> if G:                                 # random output          # long time,␣
↪needs networkx sage.plot
...     G.show()
```

static RandomTriangulation(*n*, *set_position=False*, *k=3*, *seed=None*, *immutable=False*)

Return a random inner triangulation of an outer face of degree `k` with `n` vertices in total.

An inner triangulation is a plane graph all of whose faces (except the outer/unbounded face) are triangles (3-cycles).

INPUT:

- `n` – the number of vertices of the graph
- `k` – the size of the outer face
- `set_position` – boolean (default: `False`); if set to `True`, this will compute coordinates for a planar drawing of the graph
- `seed` – a `random.Random` seed or a Python `int` for the random number generator (default: `None`)
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

OUTPUT:

A random graph chosen uniformly among the inner triangulations of a *rooted* k-gon with n vertices (including the k vertices from the outer face). This is a planar graph and comes with a combinatorial embedding. The vertices of the root edge are labelled `-1` and `-2` and the outer face is the face returned by *`faces()`* in which `-1` and `-2` are consecutive vertices in this order.

Because some triangulations have nontrivial automorphism groups, this may not be equal to the uniform distribution among inner triangulations of unrooted k-gons.

ALGORITHM:

The algorithm is taken from [PS2006], Section 5.

Starting from a planar k-gonal forest (represented by its contour as a sequence of vertices), one performs local closures, until no one is possible. A local closure amounts to replace in the cyclic contour word a sequence `in1, in2, in3, lf, in3` by `in1, in3`.

At every step of the algorithm, newly created edges are recorded in a graph, which will be returned at the end. The combinatorial embedding is also computed and recorded in the output graph.

See also

triangulations(), RandomTwoSphere().

EXAMPLES:

```
sage: G = graphs.RandomTriangulation(6, True); G
Graph on 6 vertices
sage: G.is_planar()
True
sage: G.girth()
3
sage: G.plot(vertex_size=0, vertex_labels=False)
↪  # needs sage.plot
Graphics object consisting of 13 graphics primitives

sage: H = graphs.RandomTriangulation(7, k=5)
sage: sorted(len(f) for f in H.faces())
[3, 3, 3, 3, 3, 3, 3, 5]
```

```
>>> from sage.all import *
>>> G = graphs.RandomTriangulation(Integer(6), True); G
Graph on 6 vertices
>>> G.is_planar()
True
>>> G.girth()
3
>>> G.plot(vertex_size=Integer(0), vertex_labels=False)
↪          # needs sage.plot
Graphics object consisting of 13 graphics primitives

>>> H = graphs.RandomTriangulation(Integer(7), k=Integer(5))
>>> sorted(len(f) for f in H.faces())
[3, 3, 3, 3, 3, 3, 3, 5]
```

static RandomUnitDiskGraph(*n*, *radius=0.1*, *side=1*, *seed=None*, *immutable=False*)

Return a random unit disk graph of order n.

A unit disk graph is the intersection graph of a family of unit disks in the Euclidean plane. That is a graph with one vertex per disk of the family and an edge between two vertices whenever they lie within a unit distance of each other. See the Wikipedia article Unit_disk_graph for more details.

INPUT:

- `n` – number of nodes
- `radius` – float (default: 0.1); two vertices at distance less than `radius` are connected by an edge
- `side` – float (default: `1`); indicate the side of the area in which the points are drawn
- `seed` – seed of the random number generator
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

When using twice the same seed, the vertices get the same positions:

```
sage: # needs scipy
sage: from sage.misc.randstate import current_randstate
sage: seed = current_randstate().seed()
sage: G = graphs.RandomUnitDiskGraph(20, radius=.5, side=1, seed=seed)
sage: H = graphs.RandomUnitDiskGraph(20, radius=.2, side=1, seed=seed)
sage: H.is_subgraph(G, induced=False)
True
sage: H.size() <= G.size()
True
sage: Gpos = G.get_pos()
sage: Hpos = H.get_pos()
sage: all(Gpos[u] == Hpos[u] for u in G)
True
```

```
>>> from sage.all import *
>>> # needs scipy
>>> from sage.misc.randstate import current_randstate
>>> seed = current_randstate().seed()
>>> G = graphs.RandomUnitDiskGraph(Integer(20), radius=RealNumber('.5'),␣
↪side=Integer(1), seed=seed)
>>> H = graphs.RandomUnitDiskGraph(Integer(20), radius=RealNumber('.2'),␣
↪side=Integer(1), seed=seed)
>>> H.is_subgraph(G, induced=False)
True
>>> H.size() <= G.size()
True
>>> Gpos = G.get_pos()
>>> Hpos = H.get_pos()
>>> all(Gpos[u] == Hpos[u] for u in G)
True
```

When the radius is more than $\sqrt{2\text{side}}$, the graph is a clique:

```
sage: G = graphs.RandomUnitDiskGraph(10, radius=2, side=1)                    ␣
↪  # needs scipy
sage: G.is_clique()                                                           ␣
↪  # needs scipy
True
```

```
>>> from sage.all import *
>>> G = graphs.RandomUnitDiskGraph(Integer(10), radius=Integer(2),␣
↪side=Integer(1))                        # needs scipy
>>> G.is_clique()
↪# needs scipy
True
```

static RingedTree(*k*, *vertex_labels=True*, *immutable=False*)

Return the ringed tree on k-levels.

A ringed tree of level k is a binary tree with k levels (counting the root as a level), in which all vertices at the same level are connected by a ring.

More precisely, in each layer of the binary tree (i.e. a layer is the set of vertices $[2^i...2^{i+1}-1]$) two vertices u, v are adjacent if $u = v + 1$ or if $u = 2^i$ and $v = 2^{i+1} - 1$.

Ringed trees are defined in [CFHM2013].

INPUT:

- `k` – the number of levels of the ringed tree
- `vertex_labels` – boolean; whether to label vertices as binary words (default) or as integers
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: # needs networkx
sage: G = graphs.RingedTree(5)
sage: P = G.plot(vertex_labels=False, vertex_size=10)
↪  # needs sage.plot
sage: P.show()                          # long time
↪  # needs sage.plot
sage: G.vertices(sort=True)
['', '0', '00', '000', '0000', '0001', '001', '0010', '0011', '01',
 '010', '0100', '0101', '011', '0110', '0111', '1', '10', '100',
 '1000', '1001', '101', '1010', '1011', '11', '110', '1100', '1101',
 '111', '1110', '1111']
```

```
>>> from sage.all import *
>>> # needs networkx
>>> G = graphs.RingedTree(Integer(5))
>>> P = G.plot(vertex_labels=False, vertex_size=Integer(10))
↪         # needs sage.plot
>>> P.show()                          # long time
↪# needs sage.plot
>>> G.vertices(sort=True)
['', '0', '00', '000', '0000', '0001', '001', '0010', '0011', '01',
 '010', '0100', '0101', '011', '0110', '0111', '1', '10', '100',
 '1000', '1001', '101', '1010', '1011', '11', '110', '1100', '1101',
 '111', '1110', '1111']
```

static RobertsonGraph(*immutable=False*)

Return the Robertson graph.

See the Wikipedia article Robertson_graph.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.RobertsonGraph()
sage: g.order()
19
sage: g.size()
38
sage: g.diameter()
3
sage: g.girth()
5
sage: g.charpoly().factor()
```

(continues on next page)

(continued from previous page)

```
(x - 4) * (x - 1)^2 * (x^2 + x - 5) * (x^2 + x - 1)
 * (x^2 - 3)^2 * (x^2 + x - 4)^2 * (x^2 + x - 3)^2
sage: g.chromatic_number()
3
sage: g.is_hamiltonian()
↪  # needs sage.numerical.mip
True
sage: g.is_vertex_transitive()
False
```

```
>>> from sage.all import *
>>> g = graphs.RobertsonGraph()
>>> g.order()
19
>>> g.size()
38
>>> g.diameter()
3
>>> g.girth()
5
>>> g.charpoly().factor()
(x - 4) * (x - 1)^2 * (x^2 + x - 5) * (x^2 + x - 1)
 * (x^2 - 3)^2 * (x^2 + x - 4)^2 * (x^2 + x - 3)^2
>>> g.chromatic_number()
3
>>> g.is_hamiltonian()
↪# needs sage.numerical.mip
True
>>> g.is_vertex_transitive()
False
```

static RookGraph(*dim_list*, *radius=None*, *relabel=False*, *immutable=False*)

Return the d-dimensional Rook's Graph with prescribed dimensions.

The 2-dimensional Rook's Graph of parameters n and m is a graph with nm vertices in which each vertex represents a square in an $n \times m$ chessboard, and each edge corresponds to a legal move by a rook.

The d-dimensional Rook Graph with $d >= 2$ has for vertex set the cells of a d-dimensional grid with prescribed dimensions, and each edge corresponds to a legal move by a rook in any of the dimensions.

The Rook's Graph for an $n \times m$ chessboard may also be defined as the Cartesian product of two complete graphs $K_n \Box K_m$.

INPUT:

- `dim_list` – iterable (list, set, dict); provides the dimensions $n_1, n_2, \ldots, n_d$, with $n_i \geq 1$, of the chessboard
- `radius` – integer (default: `None`); by setting the radius to a positive integer, one may decrease the power of the rook to at most `radius` steps. When the radius is 1, the resulting graph is a d-dimensional grid.
- `relabel` – boolean (default: `False`); indicates whether the vertices must be relabeled as integers
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

The (n, m)-Rook's Graph is isomorphic to the Cartesian product of two complete graphs:

```
sage: G = graphs.RookGraph( [3, 4] )
sage: H = ( graphs.CompleteGraph(3) ).cartesian_product( graphs.
↪CompleteGraph(4) )
sage: G.is_isomorphic( H )
True
```

```
>>> from sage.all import *
>>> G = graphs.RookGraph( [Integer(3), Integer(4)] )
>>> H = ( graphs.CompleteGraph(Integer(3)) ).cartesian_product( graphs.
↪CompleteGraph(Integer(4)) )
>>> G.is_isomorphic( H )
True
```

When the radius is 1, the Rook's Graph is a grid:

```
sage: G = graphs.RookGraph( [3, 3, 4], radius=1 )
sage: H = graphs.GridGraph( [3, 4, 3] )
sage: G.is_isomorphic( H )
True
```

```
>>> from sage.all import *
>>> G = graphs.RookGraph( [Integer(3), Integer(3), Integer(4)],␣
↪radius=Integer(1) )
>>> H = graphs.GridGraph( [Integer(3), Integer(4), Integer(3)] )
>>> G.is_isomorphic( H )
True
```

static RoseWindowGraph(*n*, *a*, *r*, *immutable=False*)

Return a rose window graph with $2n$ nodes.

The rose window graphs is a family of tetravalent graphs introduced in [Wilson2008]. The parameters n, a and r are integers such that $n > 2$, $1 \leq a, r < n$, and $r \neq n/2$.

INPUT:

- `n` – the number of nodes is $2 * n$
- `a` – integer such that $1 \leq a < n$ determining a-spoke edges
- `r` – integer such that $1 \leq r < n$ and $r \neq n/2$ determining how inner vertices are connected
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

PLOTTING: Upon construction, the position dictionary is filled to override the spring-layout algorithm. By convention, the rose window graphs are displayed as an inner and outer cycle pair, with the first n nodes drawn on the outer circle. The first (0) node is drawn at the top of the outer-circle, moving counterclockwise after that. The inner circle is drawn with the (n)th node at the top, then counterclockwise as well. Vertices in the outer circle are connected in the circular manner, vertices in the inner circle are connected when their label have difference $r \pmod n$. Vertices on the outer rim are connected with the vertices on the inner rim when they are at the same position and when they are a apart.

EXAMPLES:

The vertices of a rose window graph have all degree 4:

```
sage: G = graphs.RoseWindowGraph(5, 1, 2)
sage: all(G.degree(u) == 4 for u in G)
True
```

```
>>> from sage.all import *
>>> G = graphs.RoseWindowGraph(Integer(5), Integer(1), Integer(2))
>>> all(G.degree(u) == Integer(4) for u in G)
True
```

The smallest rose window graph as parameters $(3, 2, 1)$:

```
sage: G = graphs.RoseWindowGraph(3, 2, 1)
sage: all(G.degree(u) == 4 for u in G)
True
```

```
>>> from sage.all import *
>>> G = graphs.RoseWindowGraph(Integer(3), Integer(2), Integer(1))
>>> all(G.degree(u) == Integer(4) for u in G)
True
```

static SchlaefliGraph(*immutable=False*)

Return the Schläfli graph.

The Schläfli graph is the only strongly regular graphs of parameters $(27, 16, 10, 8)$ (see [GR2001]).

For more information, see the Wikipedia article Schläfli_graph.

See also

Graph.is_strongly_regular() – tests whether a graph is strongly regular and/or returns its parameters.

Todo

Find a beautiful layout for this beautiful graph.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

Checking that the method actually returns the Schläfli graph:

```
sage: S = graphs.SchlaefliGraph()
sage: S.is_strongly_regular(parameters=True)
(27, 16, 10, 8)
```

```
>>> from sage.all import *
>>> S = graphs.SchlaefliGraph()
>>> S.is_strongly_regular(parameters=True)
(27, 16, 10, 8)
```

The graph is vertex-transitive:

```
sage: S.is_vertex_transitive()                                              ␣
↪  # needs sage.groups
True
```

```
>>> from sage.all import *
>>> S.is_vertex_transitive()
↪# needs sage.groups
True
```

The neighborhood of each vertex is isomorphic to the complement of the Clebsch graph:

```
sage: neighborhood = S.subgraph(vertices=S.neighbors(0))
sage: graphs.ClebschGraph().complement().is_isomorphic(neighborhood)
True
```

```
>>> from sage.all import *
>>> neighborhood = S.subgraph(vertices=S.neighbors(Integer(0)))
>>> graphs.ClebschGraph().complement().is_isomorphic(neighborhood)
True
```

static ShrikhandeGraph(*immutable=False*)

Return the Shrikhande graph.

For more information, see the MathWorld article on the Shrikhande graph or the Wikipedia article Shrikhande_graph.

See also

Graph.is_strongly_regular() – tests whether a graph is strongly regular and/or returns its parameters.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

The Shrikhande graph was defined by S. S. Shrikhande in 1959. It has 16 vertices and 48 edges, and is strongly regular of degree 6 with parameters $(2, 2)$:

```
sage: G = graphs.ShrikhandeGraph(); G
Shrikhande graph: Graph on 16 vertices
sage: G.order()
16
sage: G.size()
48
sage: G.is_regular(6)
True
sage: set([ len([x for x in G.neighbors(i) if x in G.neighbors(j)])
....:       for i in range(G.order())
....:       for j in range(i) ])
{2}
```

```
>>> from sage.all import *
>>> G = graphs.ShrikhandeGraph(); G
Shrikhande graph: Graph on 16 vertices
>>> G.order()
16
>>> G.size()
48
>>> G.is_regular(Integer(6))
True
>>> set([ len([x for x in G.neighbors(i) if x in G.neighbors(j)])
...      for i in range(G.order())
...      for j in range(i) ])
{2}
```

It is non-planar, and both Hamiltonian and Eulerian:

```
sage: G.is_planar()
False
sage: G.is_hamiltonian()
↪  # needs sage.numerical.mip
True
sage: G.is_eulerian()
True
```

```
>>> from sage.all import *
>>> G.is_planar()
False
>>> G.is_hamiltonian()
↪# needs sage.numerical.mip
True
>>> G.is_eulerian()
True
```

It has radius 2, diameter 2, and girth 3:

```
sage: G.radius()
2
sage: G.diameter()
2
sage: G.girth()
3
```

```
>>> from sage.all import *
>>> G.radius()
2
>>> G.diameter()
2
>>> G.girth()
3
```

Its chromatic number is 4 and its automorphism group is of order 192:

```
sage: G.chromatic_number()
4
sage: G.automorphism_group().cardinality()
↪  # needs sage.groups
192
```

```
>>> from sage.all import *
>>> G.chromatic_number()
4
>>> G.automorphism_group().cardinality()
↪# needs sage.groups
192
```

It is an integral graph since it has only integral eigenvalues:

```
sage: G.characteristic_polynomial().factor()
↪  # needs sage.libs.pari sage.modules
(x - 6) * (x - 2)^6 * (x + 2)^9
```

```
>>> from sage.all import *
>>> G.characteristic_polynomial().factor()
↪# needs sage.libs.pari sage.modules
(x - 6) * (x - 2)^6 * (x + 2)^9
```

It is a toroidal graph, and its embedding on a torus is dual to an embedding of the Dyck graph (*DyckGraph*).

static SierpinskiGasketGraph(*n*, *immutable=False*)

Return the Sierpinski Gasket graph of generation n.

All vertices but 3 have valence 4.

INPUT:

- `n` – integer
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

OUTPUT:

a graph S_n with $3(3^{n-1}+1)/2$ vertices and 3^n edges, closely related to the famous Sierpinski triangle fractal.

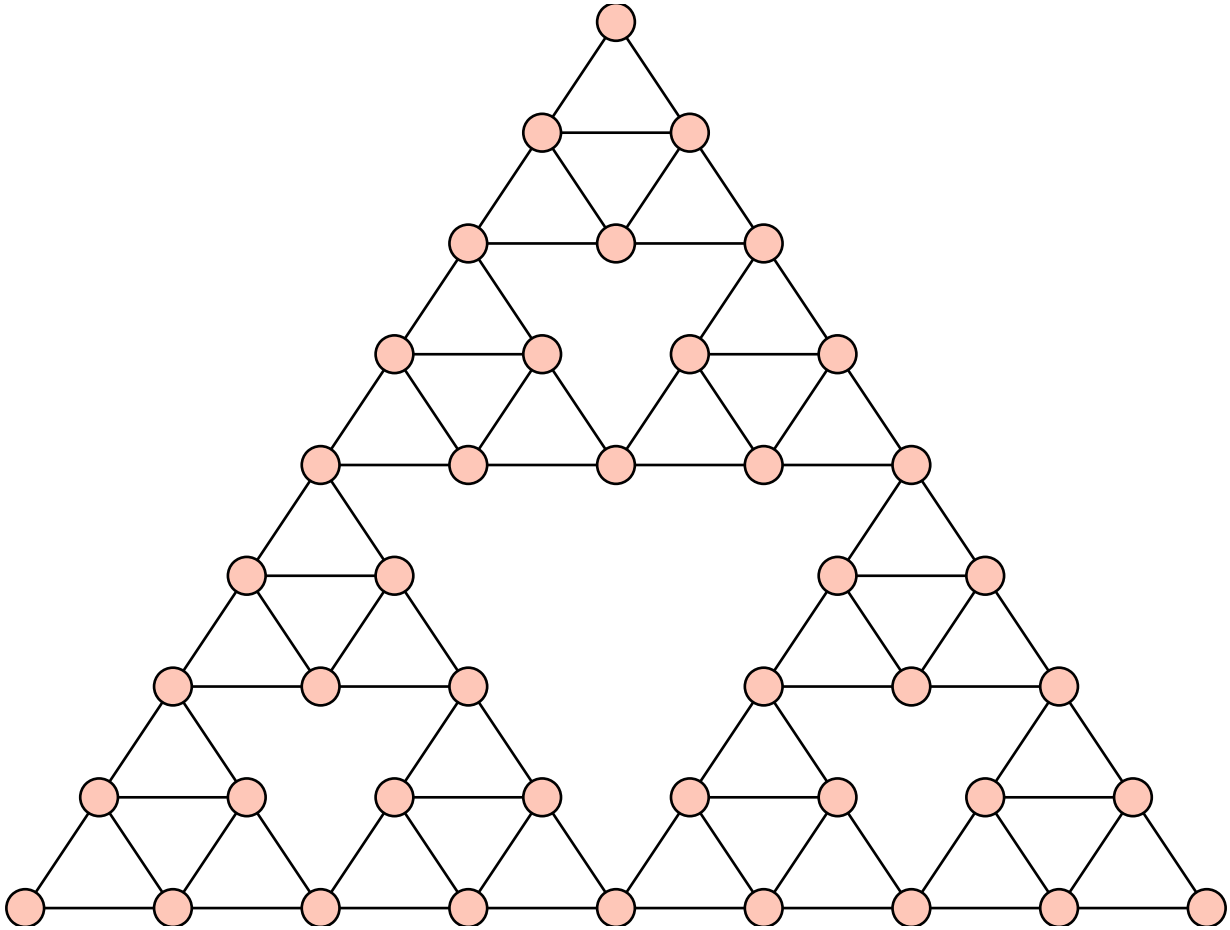
All these graphs have a triangular shape, and three special vertices at top, bottom left and bottom right. These are the only vertices of valence 2, all the other ones having valence 4.

The graph S_1 (generation 1) is a triangle.

The graph S_{n+1} is obtained from the disjoint union of three copies A,B,C of S_n by identifying pairs of vertices: the top vertex of A with the bottom left vertex of B, the bottom right vertex of B with the top vertex of C, and the bottom left vertex of C with the bottom right vertex of A.

See also

- *HanoiTowerGraph()*. There is another family of graphs called Sierpinski graphs, where all vertices but 3 have valence 3. They are available using `graphs.HanoiTowerGraph(3, n)`.
- *GeneralizedSierpinskiGraph()*



EXAMPLES:

```
sage: s4 = graphs.SierpinskiGasketGraph(4); s4
Graph on 42 vertices
sage: s4.size()
81
sage: s4.degree_histogram()
[0, 0, 3, 0, 39]
sage: s4.is_hamiltonian()
True
```

```
>>> from sage.all import *
>>> s4 = graphs.SierpinskiGasketGraph(Integer(4)); s4
Graph on 42 vertices
>>> s4.size()
81
>>> s4.degree_histogram()
[0, 0, 3, 0, 39]
>>> s4.is_hamiltonian()
True
```

REFERENCES:

[LLWC2011]

static SimsGewirtzGraph(*immutable=False*)

Return the Sims-Gewirtz Graph.

This graph is obtained from the Higman Sims graph by considering the graph induced by the vertices at distance two from the vertices of an (any) edge. It is the only strongly regular graph with parameters $v = 56$, $k = 10$, $\lambda = 0$, $\mu = 2$

For more information on the Sylvester graph, see https://www.win.tue.nl/~aeb/graphs/Sims-Gewirtz.html or its Wikipedia article Gewirtz_graph.

See also

- *HigmanSimsGraph()*.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.SimsGewirtzGraph(); g
Sims-Gewirtz Graph: Graph on 56 vertices
sage: g.order()
56
sage: g.size()
280
sage: g.is_strongly_regular(parameters = True)
(56, 10, 0, 2)
```

```
>>> from sage.all import *
>>> g = graphs.SimsGewirtzGraph(); g
Sims-Gewirtz Graph: Graph on 56 vertices
>>> g.order()
56
>>> g.size()
280
>>> g.is_strongly_regular(parameters = True)
(56, 10, 0, 2)
```

static SousselierGraph(*immutable=False*)

Return the Sousselier Graph.

The Sousselier graph is a hypohamiltonian graph on 16 vertices and 27 edges. For more information, see Wikipedia article Sousselier_graph or the corresponding French Wikipedia page.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.SousselierGraph()
sage: g.order()
16
sage: g.size()
27
sage: g.radius()
2
sage: g.diameter()
3
sage: g.automorphism_group().cardinality()
↪  # needs sage.groups
2
sage: g.is_hamiltonian()
↪  # needs sage.numerical.mip
False
sage: g.delete_vertex(g.random_vertex())
sage: g.is_hamiltonian()
↪  # needs sage.numerical.mip
True
```

```
>>> from sage.all import *
>>> g = graphs.SousselierGraph()
>>> g.order()
16
>>> g.size()
27
>>> g.radius()
2
>>> g.diameter()
3
>>> g.automorphism_group().cardinality()
↪# needs sage.groups
```

(continues on next page)

(continued from previous page)

```
2
>>> g.is_hamiltonian()
↪# needs sage.numerical.mip
False
>>> g.delete_vertex(g.random_vertex())
>>> g.is_hamiltonian()
↪# needs sage.numerical.mip
True
```

static SquaredSkewHadamardMatrixGraph(*n*, *immutable=False*)

Pseudo-$OA(2n, 4n-1)$-graph from a skew Hadamard matrix of order $4n$.

A strongly regular graph with parameters of the orthogonal array graph *OrthogonalArrayBlockGraph()*, also known as pseudo Latin squares graph $L_{2n}(4n-1)$, constructed from a skew Hadamard matrix of order $4n$, due to Goethals and Seidel, see [BL1984].

See also

- *is_orthogonal_array_block_graph()*

INPUT:

- `n` – integer
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: G = graphs.SquaredSkewHadamardMatrixGraph(4)
sage: G.is_strongly_regular(parameters=True)
(225, 112, 55, 56)
sage: G = graphs.SquaredSkewHadamardMatrixGraph(5)
sage: G.is_strongly_regular(parameters=True)     # long time
(361, 180, 89, 90)
sage: G = graphs.SquaredSkewHadamardMatrixGraph(9)
sage: G.is_strongly_regular(parameters=True)     # not tested
(1225, 612, 305, 306)
```

```
>>> from sage.all import *
>>> G = graphs.SquaredSkewHadamardMatrixGraph(Integer(4))
>>> G.is_strongly_regular(parameters=True)
(225, 112, 55, 56)
>>> G = graphs.SquaredSkewHadamardMatrixGraph(Integer(5))
>>> G.is_strongly_regular(parameters=True)     # long time
(361, 180, 89, 90)
>>> G = graphs.SquaredSkewHadamardMatrixGraph(Integer(9))
>>> G.is_strongly_regular(parameters=True)     # not tested
(1225, 612, 305, 306)
```

static StaircaseGraph(*n*, *immutable=False*)

Return a staircase graph with $2n$ nodes

For $n \geq 3$, the staircase graph of order $2n$ is the graph obtained from the ladder graph of order $2n-2$, i.e., `graphs.LadderGraph(n - 1)` by introducing two new nodes $2n-2$ and $2n-1$, and then joining the

node $2n-2$ with 0 and $n-1$, the node $2n-1$ with $n-2$ and $2n-3$, and the nodes $2n-2$ and $2n-1$ with each other.

Note that `graphs.StaircaseGraph(4)` is also known as the `Bicorn graph`. It is the only brick that has a unique b-invariant edge.

PLOTTING:

Upon construction, the position dictionary is filled to override the spring-layout algorithm. By convention, each staircase graph will be displayed horizontally, with the first $n-1$ nodes displayed from left to right on the top horizontal line, the second $n-1$ nodes displayed from left to right on the middle horizontal line, and the last two nodes displayed at the bottom two corners.

INPUT:

- `n` – an integer at least 3; number of nodes is $2n$
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

OUTPUT:

- `G` – a staircase graph of order $2n$; note that a `ValueError` is returned if $n < 3$

EXAMPLES:

Construct and show a staircase graph with 10 nodes:

```
sage: g = graphs.StaircaseGraph(5)
sage: g.show()                              # long time
↪  # needs sage.plot
```

```
>>> from sage.all import *
>>> g = graphs.StaircaseGraph(Integer(5))
>>> g.show()                              # long time
↪# needs sage.plot
```

Construct and show the Bicorn graph. Note that the edge $(1, 4)$ is the unique b-invariant edge:

```
sage: bicornGraph = graphs.StaircaseGraph(4)
sage: bicornGraph.show()                    # long time
↪  # needs sage.plot
```

```
>>> from sage.all import *
>>> bicornGraph = graphs.StaircaseGraph(Integer(4))
>>> bicornGraph.show()                    # long time
↪# needs sage.plot
```

Create several staircase graphs in a Sage graphics array:

```
sage: g = []
sage: j = []
sage: for i in range(9):
....:     k = graphs.StaircaseGraph(i+3)
....:     g.append(k)
sage: for i in range(3):
....:     n = []
....:     for m in range(3):
....:         n.append(g[3*i + m].plot(vertex_size=50 - 4*(3*i+m), vertex_
```

(continues on next page)

(continued from previous page)

```
↪labels=False))
....:     j.append(n)
sage: G = graphics_array(j)
sage: G.show()                              # long time
```

```
>>> from sage.all import *
>>> g = []
>>> j = []
>>> for i in range(Integer(9)):
...     k = graphs.StaircaseGraph(i+Integer(3))
...     g.append(k)
>>> for i in range(Integer(3)):
...     n = []
...     for m in range(Integer(3)):
...         n.append(g[Integer(3)*i + m].plot(vertex_size=Integer(50) -␣
↪Integer(4)*(Integer(3)*i+m), vertex_labels=False))
...     j.append(n)
>>> G = graphics_array(j)
>>> G.show()                              # long time
```

REFERENCES:

- [LM2024]

See also

LadderGraph()

AUTHORS:

- Janmenjaya Panda (2024-06-09)

static StarGraph(*n*, *immutable=False*)

Return a star graph with $n + 1$ nodes.

A Star graph is a basic structure where one node is connected to all other nodes.

INPUT:

- `n` – a nonnegative integer; number of nodes is $n + 1$
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

PLOTTING: Upon construction, the position dictionary is filled to override the spring-layout algorithm. By convention, each star graph will be displayed with the first (0) node in the center, the second node (1) at the top, with the rest following in a counterclockwise manner. (0) is the node connected to all other nodes.

The star graph is a good opportunity to compare efficiency of filling a position dictionary vs. using the spring-layout algorithm for plotting. As far as display, the spring-layout should push all other nodes away from the (0) node, and thus look very similar to this constructor's positioning.

EXAMPLES:

```
sage: import networkx                                                          ␣
↪  # needs networkx
```

```
>>> from sage.all import *
>>> import networkx
↪# needs networkx
```

Compare the plots:

```
sage: # needs networkx sage.plot
sage: n = networkx.star_graph(23)
sage: spring23 = Graph(n)
sage: posdict23 = graphs.StarGraph(23)
sage: spring23.show()                    # long time
sage: posdict23.show()  # long time
```

```
>>> from sage.all import *
>>> # needs networkx sage.plot
>>> n = networkx.star_graph(Integer(23))
>>> spring23 = Graph(n)
>>> posdict23 = graphs.StarGraph(Integer(23))
>>> spring23.show()                    # long time
>>> posdict23.show()  # long time
```

View many star graphs as a Sage Graphics Array

With this constructor (i.e., the position dictionary filled)

```
sage: g = []
sage: j = []
sage: for i in range(9):
....:     k = graphs.StarGraph(i+3)
....:     g.append(k)
sage: for i in range(3):
....:     n = []
....:     for m in range(3):
....:         n.append(g[3*i + m].plot(vertex_size=50, vertex_labels=False))
....:     j.append(n)
sage: G = graphics_array(j)
sage: G.show()                           # long time
```

```
>>> from sage.all import *
>>> g = []
>>> j = []
>>> for i in range(Integer(9)):
...     k = graphs.StarGraph(i+Integer(3))
...     g.append(k)
>>> for i in range(Integer(3)):
...     n = []
...     for m in range(Integer(3)):
...         n.append(g[Integer(3)*i + m].plot(vertex_size=Integer(50), vertex_
↪labels=False))
...     j.append(n)
>>> G = graphics_array(j)
>>> G.show()                           # long time
```

Compared to plotting with the spring-layout algorithm

```
sage: # needs networkx sage.plot
sage: g = []
sage: j = []
sage: for i in range(9):
....:     spr = networkx.star_graph(i+3)
....:     k = Graph(spr)
....:     g.append(k)
sage: for i in range(3):
....:     n = []
....:     for m in range(3):
....:         n.append(g[3*i + m].plot(vertex_size=50, vertex_labels=False))
....:     j.append(n)
sage: G = graphics_array(j)
sage: G.show()                          # long time
```

```
>>> from sage.all import *
>>> # needs networkx sage.plot
>>> g = []
>>> j = []
>>> for i in range(Integer(9)):
...     spr = networkx.star_graph(i+Integer(3))
...     k = Graph(spr)
...     g.append(k)
>>> for i in range(Integer(3)):
...     n = []
...     for m in range(Integer(3)):
...         n.append(g[Integer(3)*i + m].plot(vertex_size=Integer(50), vertex_
↪labels=False))
...     j.append(n)
>>> G = graphics_array(j)
>>> G.show()                          # long time
```

static SuzukiGraph(*immutable=False*)

Return the Suzuki Graph.

The Suzuki graph has 1782 vertices, and is strongly regular with parameters $(1782, 416, 100, 96)$. Known as S.15 in [Hub1975].

Note

It takes approximately 50 seconds to build this graph. Do not be too impatient.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.SuzukiGraph(); g  # optional internet # not tested
Suzuki graph: Graph on 1782 vertices
sage: g.is_strongly_regular(parameters=True)  # optional internet # not tested
(1782, 416, 100, 96)
```

```
>>> from sage.all import *
>>> g = graphs.SuzukiGraph(); g  # optional internet # not tested
Suzuki graph: Graph on 1782 vertices
>>> g.is_strongly_regular(parameters=True)  # optional internet # not tested
(1782, 416, 100, 96)
```

static SwitchedSquaredSkewHadamardMatrixGraph(*n*, *immutable=False*)

A strongly regular graph in Seidel switching class of *SquaredSkewHadamardMatrixGraph()*.

A strongly regular graph in the *Seidel switching* class of the disjoint union of a 1-vertex graph and the one produced by *Pseudo-L_{2n}(4n-1)*

In this case, the other possible parameter set of a strongly regular graph in the Seidel switching class of the latter graph (see [BH2012]) coincides with the set of parameters of the complement of the graph returned by this function.

See also

- *is_switch_skewhad()*

INPUT:

- `n` – integer
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.SwitchedSquaredSkewHadamardMatrixGraph(4)
sage: g.is_strongly_regular(parameters=True)
(226, 105, 48, 49)
sage: from sage.combinat.designs.twographs import twograph_descendant
sage: twograph_descendant(g, 0).is_strongly_regular(parameters=True)
(225, 112, 55, 56)
sage: gc = g.complement()
sage: twograph_descendant(gc, 0).is_strongly_regular(parameters=True)
(225, 112, 55, 56)
```

```
>>> from sage.all import *
>>> g = graphs.SwitchedSquaredSkewHadamardMatrixGraph(Integer(4))
>>> g.is_strongly_regular(parameters=True)
(226, 105, 48, 49)
>>> from sage.combinat.designs.twographs import twograph_descendant
>>> twograph_descendant(g, Integer(0)).is_strongly_regular(parameters=True)
(225, 112, 55, 56)
>>> gc = g.complement()
>>> twograph_descendant(gc, Integer(0)).is_strongly_regular(parameters=True)
(225, 112, 55, 56)
```

static SylvesterGraph(*immutable=False*)

Return the Sylvester Graph.

This graph is obtained from the Hoffman Singleton graph by considering the graph induced by the vertices at distance two from the vertices of an (any) edge.

For more information on the Sylvester graph, see https://www.win.tue.nl/~aeb/graphs/Sylvester.html.

See also

- *HoffmanSingletonGraph()*.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.SylvesterGraph(); g
Sylvester Graph: Graph on 36 vertices
sage: g.order()
36
sage: g.size()
90
sage: g.is_regular(k=5)
True
```

```
>>> from sage.all import *
>>> g = graphs.SylvesterGraph(); g
Sylvester Graph: Graph on 36 vertices
>>> g.order()
36
>>> g.size()
90
>>> g.is_regular(k=Integer(5))
True
```

static SymplecticDualPolarGraph(*m*, *q*, *immutable=False*)

Return the Symplectic Dual Polar Graph $DSp(m, q)$.

For more information on Symplectic Dual Polar graphs, see [BCN1989] and Sect. 2.3.1 of [Coh1981].

INPUT:

- `m`, `q` – integers; q must be a prime power, and m must be even
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: G = graphs.SymplecticDualPolarGraph(6,3); G       # not tested (long␣
↪time)
Symplectic Dual Polar Graph DSp(6, 3): Graph on 1120 vertices
sage: G.is_distance_regular(parameters=True)            # not tested (long␣
↪time)
([39, 36, 27, None], [None, 1, 4, 13])
```

```
>>> from sage.all import *
>>> G = graphs.SymplecticDualPolarGraph(Integer(6),Integer(3)); G       # not␣
↪tested (long time)
Symplectic Dual Polar Graph DSp(6, 3): Graph on 1120 vertices
```

(continues on next page)

(continued from previous page)

```
>>> G.is_distance_regular(parameters=True)           # not tested (long time)
([39, 36, 27, None], [None, 1, 4, 13])
```

static SymplecticPolarGraph(*d*, *q*, *algorithm=None*, *immutable=False*)

Return the Symplectic Polar Graph $Sp(d, q)$.

The Symplectic Polar Graph $Sp(d, q)$ is built from a projective space of dimension $d-1$ over a field F_q, and a symplectic form f. Two vertices u, v are made adjacent if $f(u, v) = 0$.

See the page on symplectic graphs on Andries Brouwer's website.

INPUT:

- `d`, `q` – integers; note that only even values of d are accepted by the function
- `algorithm` – string (default: `None`); if set to `'gap'`, then the computation is carried via GAP library interface, computing totally singular subspaces, which is faster for $q > 3$. Otherwise it is done directly.
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

Computation of the spectrum of $Sp(6, 2)$:

```
sage: g = graphs.SymplecticPolarGraph(6, 2)
sage: g.is_strongly_regular(parameters=True)
(63, 30, 13, 15)
sage: set(g.spectrum()) == {-5, 3, 30}
↪  # needs sage.rings.number_field
True
```

```
>>> from sage.all import *
>>> g = graphs.SymplecticPolarGraph(Integer(6), Integer(2))
>>> g.is_strongly_regular(parameters=True)
(63, 30, 13, 15)
>>> set(g.spectrum()) == {-Integer(5), Integer(3), Integer(30)}
↪                          # needs sage.rings.number_field
True
```

The parameters of $Sp(4, q)$ are the same as of $O(5, q)$, but they are not isomorphic if q is odd:

```
sage: G = graphs.SymplecticPolarGraph(4, 3)
sage: G.is_strongly_regular(parameters=True)
(40, 12, 2, 4)

sage: O = graphs.OrthogonalPolarGraph(5, 3)
sage: O.is_strongly_regular(parameters=True)
(40, 12, 2, 4)
sage: O.is_isomorphic(G)
False
sage: S = graphs.SymplecticPolarGraph(6, 4, algorithm='gap')     # not tested
↪(long time)
sage: S.is_strongly_regular(parameters=True)                     # not tested
↪(long time)
(1365, 340, 83, 85)
```

```
>>> from sage.all import *
>>> G = graphs.SymplecticPolarGraph(Integer(4), Integer(3))
>>> G.is_strongly_regular(parameters=True)
(40, 12, 2, 4)

>>> O = graphs.OrthogonalPolarGraph(Integer(5), Integer(3))
>>> O.is_strongly_regular(parameters=True)
(40, 12, 2, 4)
>>> O.is_isomorphic(G)
False
>>> S = graphs.SymplecticPolarGraph(Integer(6), Integer(4), algorithm='gap') ␣
↪  # not tested (long time)
>>> S.is_strongly_regular(parameters=True)                    # not tested␣
↪(long time)
(1365, 340, 83, 85)
```

static SzekeresSnarkGraph(*immutable=False*)

Return the Szekeres Snark Graph.

The Szekeres graph is a snark with 50 vertices and 75 edges. For more information on this graph, see the Wikipedia article Szekeres_snark.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.SzekeresSnarkGraph()
sage: g.order()
50
sage: g.size()
75
sage: g.chromatic_number()
3
```

```
>>> from sage.all import *
>>> g = graphs.SzekeresSnarkGraph()
>>> g.order()
50
>>> g.size()
75
>>> g.chromatic_number()
3
```

static T2starGeneralizedQuadrangleGraph(*q*, *dual=False*, *hyperoval=None*, *field=None*, *check_hyperoval=True*, *immutable=False*)

Return the collinearity graph of the generalized quadrangle $T_2^*(q)$, or of its dual

Let $q = 2^k$ and $\Theta = PG(3, q)$. $T_2^*(q)$ is a generalized quadrangle (Wikipedia article Generalized_quadrangle) of order $(q-1, q+1)$, see 3.1.3 in [PT2009]. Fix a plane $\Pi \subset \Theta$ and a hyperoval $O \subset \Pi$. The points of $T_2^*(q) := T_2^*(O)$ are the points of Θ outside Π, and the lines are the lines of Θ outside Π that meet Π in a point of O.

INPUT:

- `q` – a power of two
- `dual` – boolean (default: `False`); whether to return the graph of $T_2^*(O)$ or of the dual $T_2^*(O)$ (when `True`)
- `hyperoval` – a hyperoval (i.e. a complete 2-arc; a set of points in the plane meeting every line in 0 or 2 points) in the plane of points with 0th coordinate 0 in $PG(3, q)$ over the field `field`. Each point of `hyperoval` must be a length 4 vector over `field` with 1st non-0 coordinate equal to 1. By default, `hyperoval` and `field` are not specified, and constructed on the fly. In particular, `hyperoval` we build is the classical one, i.e. a conic with the point of intersection of its tangent lines.
- `field` – an instance of a finite field of order q, must be provided if `hyperoval` is provided
- `check_hyperoval` – boolean (default: `True`); whether to check `hyperoval` for correctness or not
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

using the built-in construction:

```
sage: g = graphs.T2starGeneralizedQuadrangleGraph(4); g
T2*(O,4); GQ(3, 5): Graph on 64 vertices
sage: g.is_strongly_regular(parameters=True)
(64, 18, 2, 6)
sage: g = graphs.T2starGeneralizedQuadrangleGraph(4, dual=True); g
T2*(O,4)*; GQ(5, 3): Graph on 96 vertices
sage: g.is_strongly_regular(parameters=True)
(96, 20, 4, 4)
```

```
>>> from sage.all import *
>>> g = graphs.T2starGeneralizedQuadrangleGraph(Integer(4)); g
T2*(O,4); GQ(3, 5): Graph on 64 vertices
>>> g.is_strongly_regular(parameters=True)
(64, 18, 2, 6)
>>> g = graphs.T2starGeneralizedQuadrangleGraph(Integer(4), dual=True); g
T2*(O,4)*; GQ(5, 3): Graph on 96 vertices
>>> g.is_strongly_regular(parameters=True)
(96, 20, 4, 4)
```

supplying your own hyperoval:

```
sage: F = GF(4,'b')
sage: O = [vector(F,(0,0,0,1)),vector(F,(0,0,1,0))] + [vector(F, (0,1,x^2,x))
....:                                                  for x in F]
sage: g = graphs.T2starGeneralizedQuadrangleGraph(4, hyperoval=O, field=F); g
T2*(O,4); GQ(3, 5): Graph on 64 vertices
sage: g.is_strongly_regular(parameters=True)
(64, 18, 2, 6)
```

```
>>> from sage.all import *
>>> F = GF(Integer(4),'b')
>>> O = [vector(F,(Integer(0),Integer(0),Integer(0),Integer(1))),vector(F,
↪(Integer(0),Integer(0),Integer(1),Integer(0)))] + [vector(F, (Integer(0),
↪Integer(1),x**Integer(2),x))
...                                                 for x in F]
```

(continues on next page)

(continued from previous page)

```
>>> g = graphs.T2starGeneralizedQuadrangleGraph(Integer(4), hyperoval=O,␣
↪field=F); g
T2*(O,4); GQ(3, 5): Graph on 64 vertices
>>> g.is_strongly_regular(parameters=True)
(64, 18, 2, 6)
```

static TabacjnGraph(*n*, *a*, *b*, *r*, *immutable=False*)

Return a Tabačjn graph with $2n$ nodes.

The Tabačjn graphs is a family of pentavalent bicirculants graphs proposed in [AHKOS2014] as a generalization of generalized Petersen graphs. The parameters n, a, b, r are integers such that $n \geq 3$, $1 \leq a, b, r \leq n-1$, with $a \neq b$ and $r \neq n/2$.

INPUT:

- `n` – the number of nodes is $2 * n$
- `a` – integer such that $0 < a < n$ and $a \neq b$, that determines a-spoke edges
- `b` – integer such that $0 < b < n$ and $b \neq a$, that determines b-spoke edges
- `r` – integer such that $0 < r < n$ and $r \neq n/2$ determining how inner vertices are connected
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

PLOTTING: Upon construction, the position dictionary is filled to override the spring-layout algorithm. By convention, the rose window graphs are displayed as an inner and outer cycle pair, with the first n nodes drawn on the outer circle. The first (0) node is drawn at the top of the outer-circle, moving counterclockwise after that. The inner circle is drawn with the (n)th node at the top, then counterclockwise as well. Vertices in the outer circle are connected in the circular manner, vertices in the inner circle are connected when their label have difference $r \pmod n$. Vertices on the outer rim are connected with the vertices on the inner rim when they are at the same position and when they are a and b apart.

EXAMPLES:

```
sage: G = graphs.TabacjnGraph(3, 1, 2, 1)
sage: G.degree()
[5, 5, 5, 5, 5, 5]
sage: G.is_isomorphic(graphs.CompleteGraph(6))
True
sage: G = graphs.TabacjnGraph(6, 1, 5, 2)
sage: I = graphs.IcosahedralGraph()
sage: G.is_isomorphic(I)
True
```

```
>>> from sage.all import *
>>> G = graphs.TabacjnGraph(Integer(3), Integer(1), Integer(2), Integer(1))
>>> G.degree()
[5, 5, 5, 5, 5, 5]
>>> G.is_isomorphic(graphs.CompleteGraph(Integer(6)))
True
>>> G = graphs.TabacjnGraph(Integer(6), Integer(1), Integer(5), Integer(2))
>>> I = graphs.IcosahedralGraph()
>>> G.is_isomorphic(I)
True
```

static TadpoleGraph(*n1*, *n2*, *immutable=False*)

Return a tadpole graph with $n_1 + n_2$ nodes.

A tadpole graph is a path graph (order n_2) connected to a cycle graph (order n_1).

PLOTTING: Upon construction, the position dictionary is filled to override the spring-layout algorithm. By convention, the cycle graph will be drawn in the lower-left corner with the n_1-th node at a 45 degree angle above the right horizontal center of the cycle graph, leading directly into the path graph.

INPUT:

- `n1` – integer ≥ 3; the order of the cycle graph
- `n2` – integer ≥ 0; the order of the path graph
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

Construct and show a tadpole graph Cycle = 13, Stick = 4:

```
sage: g = graphs.TadpoleGraph(13, 4); g
Tadpole graph: Graph on 17 vertices
sage: g.show()                              # long time                    ␣
↪  # needs sage.plot
```

```
>>> from sage.all import *
>>> g = graphs.TadpoleGraph(Integer(13), Integer(4)); g
Tadpole graph: Graph on 17 vertices
>>> g.show()                              # long time
↪# needs sage.plot
```

static TaylorTwographDescendantSRG(*q*, *clique_partition=False*, *immutable=False*)

Return the descendant graph of the Taylor's two-graph for $U_3(q)$, q odd.

This is a strongly regular graph with parameters $(v, k, \lambda, \mu) = (q^3, (q^2+1)(q-1)/2, (q-1)^3/4-1, (q^2+1)(q-1)/4)$ obtained as a two-graph descendant of the `Taylor's two-graph` T. This graph admits a partition into cliques of size q, which are useful in *TaylorTwographSRG()*, a strongly regular graph on q^3+1 vertices in the Seidel switching class of T, for which we need $(q^2+1)/2$ cliques. The cliques are the q^2 lines on v_0 of the projective plane containing the unital for $U_3(q)$, and intersecting the unital (i.e. the vertices of the graph and the point we remove) in $q+1$ points. This is all taken from §7E of [BL1984].

INPUT:

- `q` – a power of an odd prime number
- `clique_partition` – boolean (default: `False`); when set to `True`, return q^2-1 cliques of size q with empty pairwise intersection. (Removing all of them leaves a clique, too), and the point removed from the unital.
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.TaylorTwographDescendantSRG(3); g
Taylor two-graph descendant SRG: Graph on 27 vertices
sage: g.is_strongly_regular(parameters=True)
(27, 10, 1, 5)
sage: from sage.combinat.designs.twographs import taylor_twograph
sage: T = taylor_twograph(3)                              # long time
```

(continues on next page)

(continued from previous page)

```
sage: g.is_isomorphic(T.descendant(T.ground_set()[1]))  # long time
True
sage: g = graphs.TaylorTwographDescendantSRG(5)         # not tested (long␣
↪time)
sage: g.is_strongly_regular(parameters=True)            # not tested (long␣
↪time)
(125, 52, 15, 26)
```

```
>>> from sage.all import *
>>> g = graphs.TaylorTwographDescendantSRG(Integer(3)); g
Taylor two-graph descendant SRG: Graph on 27 vertices
>>> g.is_strongly_regular(parameters=True)
(27, 10, 1, 5)
>>> from sage.combinat.designs.twographs import taylor_twograph
>>> T = taylor_twograph(Integer(3))                              # long time
>>> g.is_isomorphic(T.descendant(T.ground_set()[Integer(1)]))  # long time
True
>>> g = graphs.TaylorTwographDescendantSRG(Integer(5))         # not tested␣
↪(long time)
>>> g.is_strongly_regular(parameters=True)            # not tested (long time)
(125, 52, 15, 26)
```

static TaylorTwographSRG(*q*, *immutable=False*)

Return a strongly regular graph from the Taylor's two-graph for $U_3(q)$, q odd

This is a strongly regular graph with parameters $(v, k, \lambda, \mu) = (q^3+1, q(q^2+1)/2, (q^2+3)(q-1)/4, (q^2+1)(q+1)/4)$ in the Seidel switching class of `Taylor two-graph`. Details are in §7E of [BL1984].

INPUT:

- `q` – a power of an odd prime number
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

See also

- *TaylorTwographDescendantSRG()*

EXAMPLES:

```
sage: t = graphs.TaylorTwographSRG(3); t                                  ␣
↪  # needs sage.rings.finite_rings
Taylor two-graph SRG: Graph on 28 vertices
sage: t.is_strongly_regular(parameters=True)                              ␣
↪  # needs sage.rings.finite_rings
(28, 15, 6, 10)
```

```
>>> from sage.all import *
>>> t = graphs.TaylorTwographSRG(Integer(3)); t                           ␣
↪         # needs sage.rings.finite_rings
Taylor two-graph SRG: Graph on 28 vertices
>>> t.is_strongly_regular(parameters=True)
```

(continues on next page)

(continued from previous page)

```
↪# needs sage.rings.finite_rings
(28, 15, 6, 10)
```

static TetrahedralGraph(*immutable=False*)

Return a tetrahedral graph (with 4 nodes).

A tetrahedron is a 4-sided triangular pyramid. The tetrahedral graph corresponds to the connectivity of the vertices of the tetrahedron. This graph is equivalent to a wheel graph with 4 nodes and also a complete graph on four nodes. (See examples below).

PLOTTING: The Tetrahedral graph should be viewed in 3 dimensions. We choose to use a planar embedding of the graph. We hope to add rotatable, 3-dimensional viewing in the future. In such a case, an argument will be added to select the desired layout.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

Construct and show a Tetrahedral graph:

```
sage: g = graphs.TetrahedralGraph()
sage: g.show()                              # long time                    ␣
↪  # needs sage.plot
```

```
>>> from sage.all import *
>>> g = graphs.TetrahedralGraph()
>>> g.show()                              # long time
↪# needs sage.plot
```

The following example requires networkx:

```
sage: import networkx as NX                                                ␣
↪  # needs networkx
```

```
>>> from sage.all import *
>>> import networkx as NX
↪# needs networkx
```

Compare this Tetrahedral, Wheel(4), Complete(4), and the Tetrahedral plotted with the spring-layout algorithm below in a Sage graphics array:

```
sage: # needs networkx sage.plot
sage: tetra_pos = graphs.TetrahedralGraph()
sage: tetra_spring = Graph(NX.tetrahedral_graph())
sage: wheel = graphs.WheelGraph(4)
sage: complete = graphs.CompleteGraph(4)
sage: g = [tetra_pos, tetra_spring, wheel, complete]
sage: j = []
sage: for i in range(2):
....:     n = []
....:     for m in range(2):
....:         n.append(g[i + m].plot(vertex_size=50, vertex_labels=False))
....:     j.append(n)
```

(continues on next page)

(continued from previous page)

```
sage: G = graphics_array(j)
sage: G.show()                              # long time
```

```
>>> from sage.all import *
>>> # needs networkx sage.plot
>>> tetra_pos = graphs.TetrahedralGraph()
>>> tetra_spring = Graph(NX.tetrahedral_graph())
>>> wheel = graphs.WheelGraph(Integer(4))
>>> complete = graphs.CompleteGraph(Integer(4))
>>> g = [tetra_pos, tetra_spring, wheel, complete]
>>> j = []
>>> for i in range(Integer(2)):
...     n = []
...     for m in range(Integer(2)):
...         n.append(g[i + m].plot(vertex_size=Integer(50), vertex_
↪labels=False))
...     j.append(n)
>>> G = graphics_array(j)
>>> G.show()                              # long time
```

static ThomsenGraph(*immutable=False*)

Return the Thomsen Graph.

The Thomsen Graph is actually a complete bipartite graph with $(n1, n2) = (3, 3)$. It is also called the Utility graph.

PLOTTING: See CompleteBipartiteGraph.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: T = graphs.ThomsenGraph()
sage: T
Thomsen graph: Graph on 6 vertices
sage: T.graph6_string()
'EFz_'
sage: (graphs.ThomsenGraph()).show()     # long time
↪  # needs sage.plot
```

```
>>> from sage.all import *
>>> T = graphs.ThomsenGraph()
>>> T
Thomsen graph: Graph on 6 vertices
>>> T.graph6_string()
'EFz_'
>>> (graphs.ThomsenGraph()).show()     # long time
↪# needs sage.plot
```

static TietzeGraph(*immutable=False*)

Return the Tietze Graph.

For more information on the Tietze Graph, see the Wikipedia article Tietze%27s_graph.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.TietzeGraph()
sage: g.order()
12
sage: g.size()
18
sage: g.diameter()
3
sage: g.girth()
3
sage: g.automorphism_group().cardinality()                                  ␣
↪  # needs sage.groups
12
sage: g.automorphism_group().is_isomorphic(groups.permutation.Dihedral(6))  ␣
↪  # needs sage.groups
True
```

```
>>> from sage.all import *
>>> g = graphs.TietzeGraph()
>>> g.order()
12
>>> g.size()
18
>>> g.diameter()
3
>>> g.girth()
3
>>> g.automorphism_group().cardinality()
↪# needs sage.groups
12
>>> g.automorphism_group().is_isomorphic(groups.permutation.
↪Dihedral(Integer(6)))       # needs sage.groups
True
```

static ToleranceGraph(*tolrep*, *immutable=False*, *name=None*)

Return the graph generated by the tolerance representation `tolrep`.

The tolerance representation `tolrep` is described by the list $((l_0, r_0, t_0), (l_1, r_1, t_1), \ldots, (l_k, r_k, t_k))$ where $I_i = (l_i, r_i)$ denotes a closed interval on the real line with $l_i < r_i$ and t_i a strictly positive value, called tolerance. This representation generates the tolerance graph with the vertex set $\{0, 1, \ldots, k\}$ and the edge set $\{(i, j) : |I_i \cap I_j| \geq \min\{t_i, t_j\}\}$ where $|I_i \cap I_j|$ denotes the length of the intersection of I_i and I_j.

INPUT:

- `tolrep` – list of triples (l_i, r_i, t_i) where (l_i, r_i) denotes a closed interval on the real line and t_i a positive value
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph
- `name` – string (default: `None`); used as the name of the returned graph when set

Note

The vertices are named $0, 1, \ldots, k$. The tolerance representation used to create the graph is saved with the graph and can be recovered using `get_vertex()` or `get_vertices()`.

EXAMPLES:

The following code creates a tolerance representation `tolrep`, generates its tolerance graph `g`, and applies some checks:

```
sage: tolrep = [(1,4,3),(1,2,1),(2,3,1),(0,3,3)]
sage: g = graphs.ToleranceGraph(tolrep)
sage: g.get_vertex(3)
(0, 3, 3)
sage: neigh = g.neighbors(3)
sage: for v in neigh: print(g.get_vertex(v))
(1, 2, 1)
(2, 3, 1)
sage: g.is_interval()
False
sage: g.is_weakly_chordal()
True
```

```
>>> from sage.all import *
>>> tolrep = [(Integer(1),Integer(4),Integer(3)),(Integer(1),Integer(2),
...Integer(1)),(Integer(2),Integer(3),Integer(1)),(Integer(0),Integer(3),
...Integer(3))]
>>> g = graphs.ToleranceGraph(tolrep)
>>> g.get_vertex(Integer(3))
(0, 3, 3)
>>> neigh = g.neighbors(Integer(3))
>>> for v in neigh: print(g.get_vertex(v))
(1, 2, 1)
(2, 3, 1)
>>> g.is_interval()
False
>>> g.is_weakly_chordal()
True
```

The intervals in the list need not be distinct

```
sage: tolrep2 = [(0,4,5),(1,2,1),(2,3,1),(0,4,5)]
sage: g2 = graphs.ToleranceGraph(tolrep2)
sage: g2.get_vertices()
{0: (0, 4, 5), 1: (1, 2, 1), 2: (2, 3, 1), 3: (0, 4, 5)}
sage: g2.is_isomorphic(g)
True
```

```
>>> from sage.all import *
>>> tolrep2 = [(Integer(0),Integer(4),Integer(5)),(Integer(1),Integer(2),
...Integer(1)),(Integer(2),Integer(3),Integer(1)),(Integer(0),Integer(4),
...Integer(5))]
>>> g2 = graphs.ToleranceGraph(tolrep2)
```

(continues on next page)

(continued from previous page)

```
>>> g2.get_vertices()
{0: (0, 4, 5), 1: (1, 2, 1), 2: (2, 3, 1), 3: (0, 4, 5)}
>>> g2.is_isomorphic(g)
True
```

Real values are also allowed

```
sage: tolrep = [(0.1,3.3,4.4),(1.1,2.5,1.1),(1.4,4.4,3.3)]
sage: g = graphs.ToleranceGraph(tolrep)
sage: g.is_isomorphic(graphs.PathGraph(3))
True
```

```
>>> from sage.all import *
>>> tolrep = [(RealNumber('0.1'),RealNumber('3.3'),RealNumber('4.4')),
↪(RealNumber('1.1'),RealNumber('2.5'),RealNumber('1.1')),(RealNumber('1.4'),
↪RealNumber('4.4'),RealNumber('3.3'))]
>>> g = graphs.ToleranceGraph(tolrep)
>>> g.is_isomorphic(graphs.PathGraph(Integer(3)))
True
```

static Toroidal6RegularGrid2dGraph(*p*, *q*, *immutable=False*)

Return a toroidal 6-regular grid.

The toroidal 6-regular grid is a 6-regular graph on $p \times q$ vertices and its elements have coordinates (i, j) for $i \in \{0...p-1\}$ and $j \in \{0...q-1\}$.

Its edges are those of the *ToroidalGrid2dGraph()*, to which are added the edges between (i, j) and $((i+1)\%p, (j+1)\%q)$.

INPUT:

- `p`, `q` – integers
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

The toroidal 6-regular grid on 25 elements:

```
sage: g = graphs.Toroidal6RegularGrid2dGraph(5,5)
sage: g.is_regular(k=6)
True
sage: g.is_vertex_transitive()                                              ␣
↪  # needs sage.groups
True
sage: g.line_graph().is_vertex_transitive()                                 ␣
↪  # needs sage.groups
True
sage: g.automorphism_group().cardinality()                                  ␣
↪  # needs sage.groups
300
sage: g.is_hamiltonian()                                                    ␣
↪  # needs sage.numerical.mip
True
```

```
>>> from sage.all import *
>>> g = graphs.Toroidal6RegularGrid2dGraph(Integer(5),Integer(5))
>>> g.is_regular(k=Integer(6))
True
>>> g.is_vertex_transitive()
↪# needs sage.groups
True
>>> g.line_graph().is_vertex_transitive()
↪# needs sage.groups
True
>>> g.automorphism_group().cardinality()
↪# needs sage.groups
300
>>> g.is_hamiltonian()
↪# needs sage.numerical.mip
True
```

static ToroidalGrid2dGraph(*p*, *q*, *immutable=False*)

Return a toroidal 2-dimensional grid graph with $p \times q$ nodes (p rows and q columns).

The toroidal 2-dimensional grid with parameters p, q is the 2-dimensional grid graph with identical parameters to which are added the edges $((i, 0), (i, q-1))$ and $((0, i), (p-1, i))$.

INPUT:

- `p, q` – nonnegative integers; the sides of the toroidal grid
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

The toroidal 2-dimensional grid is a regular graph, while the usual 2-dimensional grid is not

```
sage: tgrid = graphs.ToroidalGrid2dGraph(8,9)
sage: print(tgrid)
Toroidal 2D Grid Graph with parameters 8,9
sage: grid = graphs.Grid2dGraph(8,9)
sage: grid.is_regular()
False
sage: tgrid.is_regular()
True
```

```
>>> from sage.all import *
>>> tgrid = graphs.ToroidalGrid2dGraph(Integer(8),Integer(9))
>>> print(tgrid)
Toroidal 2D Grid Graph with parameters 8,9
>>> grid = graphs.Grid2dGraph(Integer(8),Integer(9))
>>> grid.is_regular()
False
>>> tgrid.is_regular()
True
```

static TricornGraph(*immutable=False*)

Return the Tricorn graph.

The Tricorn graph is obtained by splicing a complete graph K_4 with the the triangular circular ladder graph $\overline{C_6}$. (Note that this generates a unique graph as both of the graphs K_4 and $\overline{C_6}$ are vertex-transitive). It is a

nonsolid brick. This matching covered graph is one of the ten extremal cubic bricks. (A matching covered graph G is *extremal* if $\Phi(G) = dim(\mathcal{L}\backslash(G))$, where $\Phi(G)$ denotes the number of perfect matchings of G, and $dim(\mathcal{L}\backslash(G))$ stands for the dimension of the linear space of G).

The Tricorn graph has no removable doubletons and has precisely three removable edges. The wheel graph W_5 and the complete graph K_4 are matching minors of the Tricorn graph.

As per a theorem of Lovász [Lov1983], each non bipartite matching covered graph has a conformal subgraph which is either a bi-subdivision of K_4 or of $\overline{C_6}$ or both. In their paper, Kothari and Murty [KM2015] characterized those planar bricks that are free of $\overline{C_6}$ (that is, the planar bricks that do not contain a bi-subdivision of $\overline{C_6}$ as a conformal subgraph). Besides two infinite families of matching covered graphs (odd wheel graphs and staircase graphs of order $4k$), the Tricorn graph is the only exception brick that is simple, planar and free of $\overline{C_6}$.

PLOTTING:

Upon construction, the position dictionary is filled to override the spring-layout algorithm. By convention, the Tricorn graph is displayed as mentioned in the book [LM2024], with the central vertex being the 0-th one. Rest of the nine vertices are shown in groups of three, one on the top, rest two on the bottom left and on the bottom right corners respectively.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

OUTPUT:

- `G` – the Tricorn graph

EXAMPLES:

Construct and show the Tricorn graph; note that the edges $(2, 3)$, $(5, 6)$ and $(8, 9)$ are the only removable edges of the Tricorn graph:

```
sage: g = graphs.TricornGraph()
sage: g.name()
'Tricorn Graph'
sage: g.order()
10
sage: g.size()
15
sage: g.girth()
3
sage: g.diameter()
3
sage: g.is_hamiltonian()
True
sage: g.show()                                # long time
↪  # needs sage.plot
```

```
>>> from sage.all import *
>>> g = graphs.TricornGraph()
>>> g.name()
'Tricorn Graph'
>>> g.order()
10
>>> g.size()
15
```

(continues on next page)

(continued from previous page)

```
>>> g.girth()
3
>>> g.diameter()
3
>>> g.is_hamiltonian()
True
>>> g.show()                              # long time
↪# needs sage.plot
```

REFERENCES:

- [KM2015]
- [LM2024]
- [Lov1983]

AUTHORS:

- Janmenjaya Panda (2024-08-02)

static TruncatedBiwheelGraph(*n*, *immutable=False*)

Return a truncated biwheel graph with $2n$ nodes

For $n \geq 3$, the truncated biwheel graph of order $2n$ is the graph obtained from the path graph of order $2n-2$, i.e., `graphs.PathGraph(2*n - 2)` by introducing two new nodes $2n-2$ and $2n-1$, and then joining the node $2n-2$ with the odd indexed nodes up to $2n-3$, joining the node $2n-1$ with the even indexed nodes up to $2n-4$ and adding the edges $(0, 2n-2)$ and $(2n-3, 2n-1)$.

PLOTTING:

Upon construction, the position dictionary is filled to override the spring-layout algorithm. By convention, each truncated biwheel graph will be displayed horizontally, with the first $2n-2$ nodes displayed from left to right on the middle horizontal line and the nodes $2n-2$ and $2n-1$ displayed at the top and the bottom central positions respectively.

INPUT:

- `n` – an integer at least 3; number of nodes is $2n$
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

OUTPUT:

- `G` – a truncated biwheel graph of order $2n$; note that a `ValueError` is returned if $n < 3$

EXAMPLES:

Construct and show a truncated biwheel graph with 10 nodes:

```
sage: g = graphs.TruncatedBiwheelGraph(5)
sage: g.show()                              # long time
↪  # needs sage.plot
```

```
>>> from sage.all import *
>>> g = graphs.TruncatedBiwheelGraph(Integer(5))
>>> g.show()                              # long time
↪# needs sage.plot
```

Create several truncated biwheel graphs in a Sage graphics array:

```
sage: g = []
sage: j = []
sage: for i in range(9):
....:     k = graphs.TruncatedBiwheelGraph(i+3)
....:     g.append(k)
sage: for i in range(3):
....:     n = []
....:     for m in range(3):
....:         n.append(g[3*i + m].plot(vertex_size=50 - 4*(3*i+m), vertex_
↪labels=False))
....:     j.append(n)
sage: G = graphics_array(j)
sage: G.show()                          # long time
```

```
>>> from sage.all import *
>>> g = []
>>> j = []
>>> for i in range(Integer(9)):
...     k = graphs.TruncatedBiwheelGraph(i+Integer(3))
...     g.append(k)
>>> for i in range(Integer(3)):
...     n = []
...     for m in range(Integer(3)):
...         n.append(g[Integer(3)*i + m].plot(vertex_size=Integer(50) -␣
↪Integer(4)*(Integer(3)*i+m), vertex_labels=False))
...     j.append(n)
>>> G = graphics_array(j)
>>> G.show()                          # long time
```

REFERENCES:

- [LM2024]

See also

WheelGraph(), *BiwheelGraph()*

AUTHORS:

- Janmenjaya Panda (2024-06-09)

static TruncatedIcosidodecahedralGraph(*immutable=False*)

Return the truncated icosidodecahedron.

The truncated icosidodecahedron is an Archimedean solid with 30 square faces, 20 regular hexagonal faces, 12 regular decagonal faces, 120 vertices and 180 edges. For more information, see the Wikipedia article Truncated_icosidodecahedron.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

Unfortunately, this graph can not be constructed currently, due to numerical issues:

```
sage: g = graphs.TruncatedIcosidodecahedralGraph(); g
↪  # needs sage.geometry.polyhedron sage.groups sage.rings.number_field
Traceback (most recent call last):
...
ValueError: *Error: Numerical inconsistency is found.  Use the GMP exact
↪arithmetic.
sage: g.order(), g.size()                # not tested
↪  # needs sage.geometry.polyhedron sage.groups sage.rings.number_field
(120, 180)
```

```
>>> from sage.all import *
>>> g = graphs.TruncatedIcosidodecahedralGraph(); g
↪# needs sage.geometry.polyhedron sage.groups sage.rings.number_field
Traceback (most recent call last):
...
ValueError: *Error: Numerical inconsistency is found.  Use the GMP exact
↪arithmetic.
>>> g.order(), g.size()                # not tested
↪# needs sage.geometry.polyhedron sage.groups sage.rings.number_field
(120, 180)
```

static TruncatedTetrahedralGraph(*immutable=False*)

Return the truncated tetrahedron.

The truncated tetrahedron is an Archimedean solid with 12 vertices and 18 edges. For more information, see the Wikipedia article Truncated_tetrahedron.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.TruncatedTetrahedralGraph(); g
Truncated Tetrahedron: Graph on 12 vertices
sage: g.order(), g.size()
(12, 18)
sage: g.is_isomorphic(polytopes.simplex(3).truncation().graph())
↪  # needs sage.geometry.polyhedron
True
```

```
>>> from sage.all import *
>>> g = graphs.TruncatedTetrahedralGraph(); g
Truncated Tetrahedron: Graph on 12 vertices
>>> g.order(), g.size()
(12, 18)
>>> g.is_isomorphic(polytopes.simplex(Integer(3)).truncation().graph())
↪         # needs sage.geometry.polyhedron
True
```

static TruncatedWittGraph(*immutable=False*)

Return the truncated Witt graph.

This builds the large Witt graph, then removes all vertices whose codeword start with a 1.

The graph is distance-regular with intersection array $[15, 14, 12; 1, 1, 9]$.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: # long time, needs sage.libs.pari sage.modules
sage: G = graphs.TruncatedWittGraph()
sage: G.is_distance_regular(True)
([15, 14, 12, None], [None, 1, 1, 9])
```

```
>>> from sage.all import *
>>> # long time, needs sage.libs.pari sage.modules
>>> G = graphs.TruncatedWittGraph()
>>> G.is_distance_regular(True)
([15, 14, 12, None], [None, 1, 1, 9])
```

REFERENCES:

A description and construction of this graph can be found in [BCN1989] p. 367.

static TuranGraph(*n*, *r*, *immutable=False*)

Return the Turan graph with parameters n, r.

Turan graphs are complete multipartite graphs with n vertices and r subsets, denoted $T(n, r)$, with the property that the sizes of the subsets are as close to equal as possible. The graph $T(n, r)$ will have $n \pmod r$ subsets of size $\lfloor n/r \rfloor$ and $r - (n \pmod r)$ subsets of size $\lceil n/r \rceil$. See the Wikipedia article Turan_graph for more information.

INPUT:

- `n` – integer; the number of vertices in the graph
- `r` – integer; the number of partitions of the graph
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

The Turan graph is a complete multipartite graph:

```
sage: g = graphs.TuranGraph(13, 4)
sage: k = graphs.CompleteMultipartiteGraph([3,3,3,4])
sage: g.is_isomorphic(k)
True
```

```
>>> from sage.all import *
>>> g = graphs.TuranGraph(Integer(13), Integer(4))
>>> k = graphs.CompleteMultipartiteGraph([Integer(3),Integer(3),Integer(3),
↪Integer(4)])
>>> g.is_isomorphic(k)
True
```

The Turan graph $T(n, r)$ has $\frac{(r-1)(n^2-s^2)}{2r} + \frac{s(s-1)}{2}$ edges, where $s = n \pmod r$ (Issue #34249):

```
sage: n = 12
sage: r = 8
sage: g = graphs.TuranGraph(n, r)
```

(continues on next page)

(continued from previous page)

```
sage: def count(n, r):
....:     s = n % r
....:     return (r - 1) * (n**2 - s**2) / (2*r) + s*(s - 1)/2
sage: g.size() == count(n, r)
True
sage: n = randint(3, 100)
sage: r = randint(2, n - 1)
sage: g = graphs.TuranGraph(n, r)
sage: g.size() == count(n, r)
True
```

```
>>> from sage.all import *
>>> n = Integer(12)
>>> r = Integer(8)
>>> g = graphs.TuranGraph(n, r)
>>> def count(n, r):
...     s = n % r
...     return (r - Integer(1)) * (n**Integer(2) - s**Integer(2)) /␣
↪(Integer(2)*r) + s*(s - Integer(1))/Integer(2)
>>> g.size() == count(n, r)
True
>>> n = randint(Integer(3), Integer(100))
>>> r = randint(Integer(2), n - Integer(1))
>>> g = graphs.TuranGraph(n, r)
>>> g.size() == count(n, r)
True
```

static Tutte12Cage(*immutable=False*)

Return the Tutte 12-Cage.

See the Wikipedia article Tutte_12-cage.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.Tutte12Cage()
sage: g.order()
126
sage: g.size()
189
sage: g.girth()
12
sage: g.diameter()
6
sage: g.show()                                                              ␣
↪  # needs sage.plot
```

```
>>> from sage.all import *
>>> g = graphs.Tutte12Cage()
>>> g.order()
```

(continues on next page)

(continued from previous page)

```
126
>>> g.size()
189
>>> g.girth()
12
>>> g.diameter()
6
>>> g.show()
↪# needs sage.plot
```

static TutteCoxeterGraph(*embedding=2*, *immutable=False*)

Return the Tutte-Coxeter graph.

See the Wikipedia article Tutte-Coxeter_graph.

INPUT:

- `embedding` – integer (default: 2); two embeddings are available, and can be selected by setting `embedding` to 1 or 2
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.TutteCoxeterGraph()
sage: g.order()
30
sage: g.size()
45
sage: g.girth()
8
sage: g.diameter()
4
sage: g.show()                                                              ␣
↪  # needs sage.plot
sage: graphs.TutteCoxeterGraph(embedding=1).show()      # long time         ␣
↪  # needs sage.plot
```

```
>>> from sage.all import *
>>> g = graphs.TutteCoxeterGraph()
>>> g.order()
30
>>> g.size()
45
>>> g.girth()
8
>>> g.diameter()
4
>>> g.show()
↪# needs sage.plot
>>> graphs.TutteCoxeterGraph(embedding=Integer(1)).show()      # long time   ␣
↪         # needs sage.plot
```

static TutteGraph(*immutable=False*)

Return the Tutte Graph.

The Tutte graph is a 3-regular, 3-connected, and planar non-hamiltonian graph. For more information on the Tutte Graph, see the Wikipedia article Tutte_graph.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.TutteGraph()
sage: g.order()
46
sage: g.size()
69
sage: g.is_planar()
True
sage: g.vertex_connectivity()            # long time
↪  # needs sage.numerical.mip
3
sage: g.girth()
4
sage: g.automorphism_group().cardinality()
↪  # needs sage.groups
3
sage: g.is_hamiltonian()
↪  # needs sage.numerical.mip
False
```

```
>>> from sage.all import *
>>> g = graphs.TutteGraph()
>>> g.order()
46
>>> g.size()
69
>>> g.is_planar()
True
>>> g.vertex_connectivity()            # long time
↪# needs sage.numerical.mip
3
>>> g.girth()
4
>>> g.automorphism_group().cardinality()
↪# needs sage.groups
3
>>> g.is_hamiltonian()
↪# needs sage.numerical.mip
False
```

static TwinplexGraph(*embedding='LM'*, *immutable=False*)

Return the Twinplex graph.

The Twinplex graph is a cubic hamiltonian graph of order 12 with the graph crossing number 2 and has a girth 5 (that is the maximal girth among all cubic graphs on 12 vertices [CHNP2020]). It corresponds to the graph labeled as Γ_2 by Fischer and Little [FiLi2001]. The Twinplex graph has LCF notation $[-5, -4, 4, -4, 4, 5, -4, 5, -4, 4, -5, 4]$.

The Fischer-Little Theorem [FiLi2001] may be stated as follows [LM2024]:

A near-bipartite graph is non-Pfaffian if and only if it contains one of the graphs $K_{3,3}$, Γ_1 and Γ_2 as an S-minor.

Norine and Thomas [NT2007] use the term `Twinplex` to describe one of the 12-vertex cubic graphs, Γ_1 and Γ_2, as defined by Fischer and Little [FiLi2001]. However, the figure in their paper that supposedly provides embeddings for the graphs labeled Cubeplex and Twinplex actually shows both embeddings corresponding to Fischer and Little's Γ_1, which is the Cubeplex graph. Followingly, for `embedding='NT'`, we present a correct version of the Twinplex graph with a slight modification of the embedding that is labeled as `Twinplex` in the paper of Norine and Thomas [NT2007].

PLOTTING:

Upon construction, the position dictionary is filled to override the spring-layout algorithm. For different values of the parameter `embedding`, the Twinplex graph is displayed as it is mentioned in the respective paper/ book. Note that for `embedding='NT'`, a correct embedding of the Twinplex graph is displayed with a minor modification to the (incorrect) embedding shown in the paper [NT2007].

INPUT:

- `embedding` – string (default: `'LM'`)
  - `'LM'` displays the embedding as shown for Γ_2 by Lucchesi and Murty [LM2024]
  - `'FL'` displays the embedding as shown for Γ_2 by Fischer and Little [FiLi2001]
  - `'NT'` displays the correct embedding with a minor modification to the one shown as the (incorrect) `Twinplex` by Norine and Thomas [NT2007]
  - `'RST'` displays the embedding as shown for the `Twinplex` by Robertson, Seymour and Thomas [RST2019]
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

OUTPUT:

- `G` – the Twinplex graph; note that a `ValueError` is returned if `embedding` is none of `'FT'`, `'NT'`, `'RST'` or `'LM'`

EXAMPLES:

Construct and show the Twinplex graph:

```
sage: g = graphs.TwinplexGraph()
sage: g.name()
'Twinplex Graph'
sage: g.order()
12
sage: g.size()
18
sage: g.girth()
5
sage: g.diameter()
3
sage: g.is_hamiltonian()
True
sage: g.crossing_number()
2
sage: g.show()                                # long time
↪  # needs sage.plot
```

```
>>> from sage.all import *
>>> g = graphs.TwinplexGraph()
>>> g.name()
'Twinplex Graph'
>>> g.order()
12
>>> g.size()
18
>>> g.girth()
5
>>> g.diameter()
3
>>> g.is_hamiltonian()
True
>>> g.crossing_number()
2
>>> g.show()                              # long time
↪# needs sage.plot
```

See also

CubeplexGraph()

AUTHORS:

- Janmenjaya Panda (2024-08-03)

static U42Graph216(*immutable=False*)

Return a (216,40,4,8)-strongly regular graph from [CRS2016].

Build the graph, interpreting the $U_4(2)$-action considered in [CRS2016] as the one on the hyperbolic lines of the corresponding unitary polar space, and then doing the unique merging of the orbitals leading to a graph with the parameters in question.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: G=graphs.U42Graph216()                    # optional - gap_package_grape
sage: G.is_strongly_regular(parameters=True)    # optional - gap_package_grape
(216, 40, 4, 8)
```

```
>>> from sage.all import *
>>> G=graphs.U42Graph216()                    # optional - gap_package_grape
>>> G.is_strongly_regular(parameters=True)    # optional - gap_package_grape
(216, 40, 4, 8)
```

static U42Graph540(*immutable=False*)

Return a (540,187,58,68)-strongly regular graph from [CRS2016].

Build the graph, interpreting the $U_4(2)$-action considered in [CRS2016] as the action of $U_4(2) = Sp_4(3) < U_4(3)$ on the nonsingular, w.r.t. to the Hermitean form stabilised by $U_4(3)$, points of the 3-dimensional

projective space over $GF(9)$. There are several possible mergings of orbitals, some leading to non-isomorphic graphs with the same parameters. We found the merging here using [FK1991].

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: G = graphs.U42Graph540()                    # optional - gap_package_grape
sage: G.is_strongly_regular(parameters=True)      # optional - gap_package_grape
(540, 187, 58, 68)
```

```
>>> from sage.all import *
>>> G = graphs.U42Graph540()                     # optional - gap_package_grape
>>> G.is_strongly_regular(parameters=True)       # optional - gap_package_grape
(540, 187, 58, 68)
```

static USAMap(*continental=False*, *immutable=False*)

Return states of USA as a graph of common border.

The graph has an edge between those states that have common *land* border line or point. Hence for example Colorado and Arizona are marked as neighbors, but Michigan and Minnesota are not.

INPUT:

- `continental` – boolean (default: `False`); whether to exclude Alaska and Hawaii
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

How many states are neighbor's neighbor for Pennsylvania:

```
sage: USA = graphs.USAMap()
sage: distance = USA.shortest_path_lengths('Pennsylvania')
sage: len([n2 for n2, d in distance.items() if d == 2])
7
```

```
>>> from sage.all import *
>>> USA = graphs.USAMap()
>>> distance = USA.shortest_path_lengths('Pennsylvania')
>>> len([n2 for n2, d in distance.items() if d == Integer(2)])
7
```

Diameter for continental USA:

```
sage: USAcont = graphs.USAMap(continental=True)
sage: USAcont.diameter()
11
```

```
>>> from sage.all import *
>>> USAcont = graphs.USAMap(continental=True)
>>> USAcont.diameter()
11
```

static UnitaryDualPolarGraph(*m*, *q*, *immutable=False*)

Return the Dual Unitary Polar Graph $U(m, q)$.

For more information on Unitary Dual Polar graphs, see [BCN1989] and Sect. 2.3.1 of [Coh1981].

INPUT:

- `m`, `q` – integers; q must be a prime power
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

The point graph of a generalized quadrangle (see Wikipedia article Generalized_quadrangle, [PT2009]) of order (8,4):

```
sage: G = graphs.UnitaryDualPolarGraph(5,2); G  # long time
↪  # needs sage.libs.gap
Unitary Dual Polar Graph DU(5, 2); GQ(8, 4): Graph on 297 vertices
sage: G.is_strongly_regular(parameters=True)    # long time
↪  # needs sage.libs.gap
(297, 40, 7, 5)
```

```
>>> from sage.all import *
>>> G = graphs.UnitaryDualPolarGraph(Integer(5),Integer(2)); G  # long time
↪                  # needs sage.libs.gap
Unitary Dual Polar Graph DU(5, 2); GQ(8, 4): Graph on 297 vertices
>>> G.is_strongly_regular(parameters=True)    # long time
↪# needs sage.libs.gap
(297, 40, 7, 5)
```

Another way to get the generalized quadrangle of order (2,4):

```
sage: G = graphs.UnitaryDualPolarGraph(4,2); G
↪  # needs sage.libs.gap
Unitary Dual Polar Graph DU(4, 2); GQ(2, 4): Graph on 27 vertices
sage: G.is_isomorphic(graphs.OrthogonalPolarGraph(6,2,'-'))
↪  # needs sage.libs.gap
True
```

```
>>> from sage.all import *
>>> G = graphs.UnitaryDualPolarGraph(Integer(4),Integer(2)); G
↪                  # needs sage.libs.gap
Unitary Dual Polar Graph DU(4, 2); GQ(2, 4): Graph on 27 vertices
>>> G.is_isomorphic(graphs.OrthogonalPolarGraph(Integer(6),Integer(2),'-'))
↪                  # needs sage.libs.gap
True
```

A bigger graph:

```
sage: G = graphs.UnitaryDualPolarGraph(6,2); G   # not tested (long time)
Unitary Dual Polar Graph DU(6, 2): Graph on 891 vertices
sage: G.is_distance_regular(parameters=True)     # not tested (long time)
([42, 40, 32, None], [None, 1, 5, 21])
```

```
>>> from sage.all import *
>>> G = graphs.UnitaryDualPolarGraph(Integer(6),Integer(2)); G    # not tested␣
↪(long time)
Unitary Dual Polar Graph DU(6, 2): Graph on 891 vertices
>>> G.is_distance_regular(parameters=True)      # not tested (long time)
([42, 40, 32, None], [None, 1, 5, 21])
```

static UnitaryPolarGraph(*m*, *q*, *algorithm='gap'*, *immutable=False*)

Return the Unitary Polar Graph $U(m, q)$.

For more information on Unitary Polar graphs, see the page of Andries Brouwer's website.

INPUT:

- `m`, `q` – integers; q must be a prime power
- `algorithm` – string (default: `'gap'`); if set to `'gap'` then the computation is carried via GAP library interface, computing totally singular subspaces, which is faster for large examples (especially with $q > 2$). Otherwise it is done directly.
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: G = graphs.UnitaryPolarGraph(4,2); G
Unitary Polar Graph U(4, 2); GQ(4, 2): Graph on 45 vertices
sage: G.is_strongly_regular(parameters=True)
(45, 12, 3, 3)
sage: graphs.UnitaryPolarGraph(5,2).is_strongly_regular(parameters=True)
(165, 36, 3, 9)
sage: graphs.UnitaryPolarGraph(6,2)      # not tested (long time)
Unitary Polar Graph U(6, 2): Graph on 693 vertices
```

```
>>> from sage.all import *
>>> G = graphs.UnitaryPolarGraph(Integer(4),Integer(2)); G
Unitary Polar Graph U(4, 2); GQ(4, 2): Graph on 45 vertices
>>> G.is_strongly_regular(parameters=True)
(45, 12, 3, 3)
>>> graphs.UnitaryPolarGraph(Integer(5),Integer(2)).is_strongly_
↪regular(parameters=True)
(165, 36, 3, 9)
>>> graphs.UnitaryPolarGraph(Integer(6),Integer(2))      # not tested (long␣
↪time)
Unitary Polar Graph U(6, 2): Graph on 693 vertices
```

static UstimenkoGraph(*m*, *q*, *immutable=False*)

Return the Ustimenko graph with parameters (m, q).

This is the distance 1 or 2 graph of the dual polar graph $C_{m-1}(q)$. The graph is distance-regular with parameters $(d, q^2, \binom{3}{1}_q - 1, \binom{m+1}{1}_q - 1)$, where $\binom{n}{k}_q$ is the q-binomial coefficient.

INPUT:

- `m`, `q` – integers; q must be a prime power and $m > 1$
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: G = graphs.UstimenkoGraph(4, 2)
↪  # needs sage.libs.gap
sage: G.is_distance_regular(True)
↪  # needs sage.libs.gap
([70, 32, None], [None, 1, 35])
```

```
>>> from sage.all import *
>>> G = graphs.UstimenkoGraph(Integer(4), Integer(2))
↪                  # needs sage.libs.gap
>>> G.is_distance_regular(True)
↪# needs sage.libs.gap
([70, 32, None], [None, 1, 35])
```

REFERENCES:

See [BCN1989] p. 279 or [VDKT2016] p. 22.

static WagnerGraph(*immutable=False*)

Return the Wagner Graph.

See the Wikipedia article Wagner_graph.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.WagnerGraph()
sage: g.order()
8
sage: g.size()
12
sage: g.girth()
4
sage: g.diameter()
2
sage: g.show()
↪  # needs sage.plot
```

```
>>> from sage.all import *
>>> g = graphs.WagnerGraph()
>>> g.order()
8
>>> g.size()
12
>>> g.girth()
4
>>> g.diameter()
2
>>> g.show()
↪# needs sage.plot
```

static WatkinsSnarkGraph(*immutable=False*)

Return the Watkins Snark Graph.

The Watkins Graph is a snark with 50 vertices and 75 edges. For more information, see the Wikipedia article Watkins_snark.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.WatkinsSnarkGraph()
sage: g.order()
50
sage: g.size()
75
sage: g.chromatic_number()
3
```

```
>>> from sage.all import *
>>> g = graphs.WatkinsSnarkGraph()
>>> g.order()
50
>>> g.size()
75
>>> g.chromatic_number()
3
```

static WellsGraph(*immutable=False*)

Return the Wells graph.

For more information on the Wells graph (also called Armanios-Wells graph), see this page.

The implementation follows the construction given on page 266 of [BCN1989]. This requires to create intermediate graphs and run a small isomorphism test, while everything could be replaced by a pre-computed list of edges. I believe that it is better to keep "the recipe" in the code, however, as it is quite unlikely that this could become the most time-consuming operation in any sensible algorithm, and …. "preserves knowledge", which is what open-source software is meant to do.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.WellsGraph(); g
Wells graph: Graph on 32 vertices
sage: g.order()
32
sage: g.size()
80
sage: g.girth()
5
sage: g.diameter()
4
sage: g.chromatic_number()
4
sage: g.is_regular(k=5)
True
```

```
>>> from sage.all import *
>>> g = graphs.WellsGraph(); g
Wells graph: Graph on 32 vertices
>>> g.order()
32
>>> g.size()
80
>>> g.girth()
5
>>> g.diameter()
4
>>> g.chromatic_number()
4
>>> g.is_regular(k=Integer(5))
True
```

static WengerGraph(*m*, *q*, *A=None*, *B=None*, *immutable=False*)

Return the Wenger graph $W_m(q)$ described in [LW2026].

The point set and line set are $\mathbf{F}_q^{m+1}$. A point $(p_1, \ldots, p_{m+1})$ is adjacent to a line $(l_1, \ldots, l_{m+1})$ if

$$p_{i+1} + l_{i+1} = p_1 l_i, \qquad 1 \leq i \leq m.$$

INPUT:

- `m` – positive integer
- `q` – prime power
- `A` – iterable, single value, or `None` (default: `None`); allowed first coordinates on the point side
- `B` – iterable, single value, or `None` (default: `None`); allowed first coordinates on the line side
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

OUTPUT:

A Sage *Graph*.

EXAMPLES:

```
sage: G = graphs.WengerGraph(2, 3)
sage: G.order(), G.size()
(54, 81)
```

```
>>> from sage.all import *
>>> G = graphs.WengerGraph(Integer(2), Integer(3))
>>> G.order(), G.size()
(54, 81)
```

REFERENCES:

- [LW2026]

static WheelGraph(*n*, *immutable=False*)

Return a Wheel graph with n nodes.

A Wheel graph is a basic structure where one node is connected to all other nodes and those (outer) nodes are connected cyclically.

PLOTTING: Upon construction, the position dictionary is filled to override the spring-layout algorithm. By convention, each wheel graph will be displayed with the first (0) node in the center, the second node at the top, and the rest following in a counterclockwise manner.

With the wheel graph, we see that it doesn't take a very large n at all for the spring-layout to give a counter-intuitive display. (See Graphics Array examples below).

INPUT:

- `n` – integer; the number of vertices
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

We view many wheel graphs with a Sage Graphics Array, first with this constructor (i.e., the position dictionary filled):

```
sage: g = []
sage: j = []
sage: for i in range(9):
....:  k = graphs.WheelGraph(i+3)
....:  g.append(k)
...
sage: for i in range(3):
....:  n = []
....:  for m in range(3):
....:      n.append(g[3*i + m].plot(vertex_size=50, vertex_labels=False))
....:  j.append(n)
...
sage: G = graphics_array(j)
sage: G.show()                          # long time
```

```
>>> from sage.all import *
>>> g = []
>>> j = []
>>> for i in range(Integer(9)):
...  k = graphs.WheelGraph(i+Integer(3))
...  g.append(k)
...
>>> for i in range(Integer(3)):
...  n = []
...  for m in range(Integer(3)):
...      n.append(g[Integer(3)*i + m].plot(vertex_size=Integer(50), vertex_
↪labels=False))
...  j.append(n)
...
>>> G = graphics_array(j)
>>> G.show()                          # long time
```

Next, using the spring-layout algorithm:

```
sage: # needs networkx sage.plot
sage: import networkx
sage: g = []
sage: j = []
```

(continues on next page)

(continued from previous page)

```
sage: for i in range(9):
....:  spr = networkx.wheel_graph(i+3)
....:  k = Graph(spr)
....:  g.append(k)
...
sage: for i in range(3):
....:  n = []
....:  for m in range(3):
....:      n.append(g[3*i + m].plot(vertex_size=50, vertex_labels=False))
....:  j.append(n)
...
sage: G = graphics_array(j)
sage: G.show()                          # long time
```

```
>>> from sage.all import *
>>> # needs networkx sage.plot
>>> import networkx
>>> g = []
>>> j = []
>>> for i in range(Integer(9)):
...  spr = networkx.wheel_graph(i+Integer(3))
...  k = Graph(spr)
...  g.append(k)
...
>>> for i in range(Integer(3)):
...  n = []
...  for m in range(Integer(3)):
...      n.append(g[Integer(3)*i + m].plot(vertex_size=Integer(50), vertex_
↪labels=False))
...  j.append(n)
...
>>> G = graphics_array(j)
>>> G.show()                          # long time
```

Compare the plotting:

```
sage: # needs networkx sage.plot
sage: n = networkx.wheel_graph(23)
sage: spring23 = Graph(n)
sage: posdict23 = graphs.WheelGraph(23)
sage: spring23.show()                   # long time
sage: posdict23.show()  # long time
```

```
>>> from sage.all import *
>>> # needs networkx sage.plot
>>> n = networkx.wheel_graph(Integer(23))
>>> spring23 = Graph(n)
>>> posdict23 = graphs.WheelGraph(Integer(23))
>>> spring23.show()                   # long time
>>> posdict23.show()  # long time
```

static WienerArayaGraph(*immutable=False*)

Return the Wiener-Araya Graph.

The Wiener-Araya Graph is a planar hypohamiltonian graph on 42 vertices and 67 edges. For more information, see the Wolfram Page on the Wiener-Araya Graph or Wikipedia article Wiener-Araya_graph.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.WienerArayaGraph()
sage: g.order()
42
sage: g.size()
67
sage: g.girth()
4
sage: g.is_planar()
True
sage: g.is_hamiltonian()                # not tested (30s)
↪  # needs sage.numerical.mip
False
sage: g.delete_vertex(g.random_vertex())
sage: g.is_hamiltonian()
↪  # needs sage.numerical.mip
True
```

```
>>> from sage.all import *
>>> g = graphs.WienerArayaGraph()
>>> g.order()
42
>>> g.size()
67
>>> g.girth()
4
>>> g.is_planar()
True
>>> g.is_hamiltonian()                # not tested (30s)
↪# needs sage.numerical.mip
False
>>> g.delete_vertex(g.random_vertex())
>>> g.is_hamiltonian()
↪# needs sage.numerical.mip
True
```

static WindmillGraph(*k*, *n*, *immutable=False*)

Return the Windmill graph $Wd(k, n)$.

The windmill graph $Wd(k, n)$ is an undirected graph constructed for $k \geq 2$ and $n \geq 2$ by joining n copies of the complete graph K_k at a shared vertex. It has $(k-1)n+1$ vertices and $nk(k-1)/2$ edges, girth 3 (if $k > 2$), radius 1 and diameter 2. It has vertex connectivity 1 because its central vertex is an articulation point; however, like the complete graphs from which it is formed, it is $(k-1)$-edge-connected. It is trivially perfect and a block graph.

See also

- Wikipedia article Windmill_graph
- *StarGraph()*
- *FriendshipGraph()*

INPUT:

- `k` – integer; the number of vertices of the complete graphs
- `n` – integer; the number of copies of the complete graph
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

The Windmill graph $Wd(2, n)$ is a star graph:

```
sage: n = 5
sage: W = graphs.WindmillGraph(2, n)
sage: W.is_isomorphic( graphs.StarGraph(n) )
True
```

```
>>> from sage.all import *
>>> n = Integer(5)
>>> W = graphs.WindmillGraph(Integer(2), n)
>>> W.is_isomorphic( graphs.StarGraph(n) )
True
```

The Windmill graph $Wd(3, n)$ is the Friendship graph F_n:

```
sage: n = 5
sage: W = graphs.WindmillGraph(3, n)
sage: W.is_isomorphic( graphs.FriendshipGraph(n) )
True
```

```
>>> from sage.all import *
>>> n = Integer(5)
>>> W = graphs.WindmillGraph(Integer(3), n)
>>> W.is_isomorphic( graphs.FriendshipGraph(n) )
True
```

The Windmill graph $Wd(3, 2)$ is the Butterfly graph:

```
sage: W = graphs.WindmillGraph(3, 2)
sage: W.is_isomorphic( graphs.ButterflyGraph() )
True
```

```
>>> from sage.all import *
>>> W = graphs.WindmillGraph(Integer(3), Integer(2))
>>> W.is_isomorphic( graphs.ButterflyGraph() )
True
```

The Windmill graph $Wd(k, n)$ has chromatic number k:

```
sage: n,k = 5,6
sage: W = graphs.WindmillGraph(k, n)
sage: W.chromatic_number() == k
True
```

```
>>> from sage.all import *
>>> n,k = Integer(5),Integer(6)
>>> W = graphs.WindmillGraph(k, n)
>>> W.chromatic_number() == k
True
```

static WorldMap(*immutable=False*)

Return the Graph of all the countries, in which two countries are adjacent in the graph if they have a common boundary.

This graph has been built from the data available in The CIA World Factbook [CIA] (2009-08-21).

The returned graph `G` has a member `G.gps_coordinates` equal to a dictionary containing the GPS coordinates of each country's capital city.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.WorldMap()
sage: g.has_edge("France", "Italy")
True
sage: g.gps_coordinates["Bolivia"]
[[17, 'S'], [65, 'W']]
sage: g.connected_component_containing_vertex('Ireland', sort=True)
['Ireland', 'United Kingdom']
```

```
>>> from sage.all import *
>>> g = graphs.WorldMap()
>>> g.has_edge("France", "Italy")
True
>>> g.gps_coordinates["Bolivia"]
[[17, 'S'], [65, 'W']]
>>> g.connected_component_containing_vertex('Ireland', sort=True)
['Ireland', 'United Kingdom']
```

basic = <module 'sage.graphs.generators.basic' from '/run/media/hsy/3000gb/sagemath/sage/src/sage/graphs/generators/basic.py'>

static chang_graphs(*immutable=False*)

Return the three Chang graphs.

Three of the four strongly regular graphs of parameters $(28, 12, 6, 4)$ are called the Chang graphs. The fourth is the line graph of K_8. For more information about the Chang graphs, see the Wikipedia article Chang_graphs or https://www.win.tue.nl/~aeb/graphs/Chang.html.

INPUT:

- `immutable` – boolean (default: `False`); whether to return immutable or mutable graphs

EXAMPLES: check that we get 4 non-isomorphic s.r.g.'s with the same parameters:

```
sage: chang_graphs = graphs.chang_graphs()
sage: K8 = graphs.CompleteGraph(8)
sage: T8 = K8.line_graph()
sage: four_srg = chang_graphs + [T8]
sage: for g in four_srg:
....:     print(g.is_strongly_regular(parameters=True))
(28, 12, 6, 4)
(28, 12, 6, 4)
(28, 12, 6, 4)
(28, 12, 6, 4)
sage: from itertools import combinations
sage: for g1,g2 in combinations(four_srg,2):
....:     assert not g1.is_isomorphic(g2)
```

```
>>> from sage.all import *
>>> chang_graphs = graphs.chang_graphs()
>>> K8 = graphs.CompleteGraph(Integer(8))
>>> T8 = K8.line_graph()
>>> four_srg = chang_graphs + [T8]
>>> for g in four_srg:
...     print(g.is_strongly_regular(parameters=True))
(28, 12, 6, 4)
(28, 12, 6, 4)
(28, 12, 6, 4)
(28, 12, 6, 4)
>>> from itertools import combinations
>>> for g1,g2 in combinations(four_srg,Integer(2)):
...     assert not g1.is_isomorphic(g2)
```

Construct the Chang graphs by Seidel switching:

```
sage: c3c5 = graphs.CycleGraph(3).disjoint_union(graphs.CycleGraph(5))
sage: c8 = graphs.CycleGraph(8)
sage: s = [K8.subgraph_search(c8).edges(sort=False),
↪  # needs sage.modules
....:      [(0,1,None),(2,3,None),(4,5,None),(6,7,None)],
....:      K8.subgraph_search(c3c5).edges(sort=False)]
sage: [T8.seidel_switching(x, inplace=False).is_isomorphic(G)
↪  # needs sage.modules
....:  for x, G in zip(s, chang_graphs)]
[True, True, True]
```

```
>>> from sage.all import *
>>> c3c5 = graphs.CycleGraph(Integer(3)).disjoint_union(graphs.
↪CycleGraph(Integer(5)))
>>> c8 = graphs.CycleGraph(Integer(8))
>>> s = [K8.subgraph_search(c8).edges(sort=False),
↪# needs sage.modules
...      [(Integer(0),Integer(1),None),(Integer(2),Integer(3),None),
↪(Integer(4),Integer(5),None),(Integer(6),Integer(7),None)],
...      K8.subgraph_search(c3c5).edges(sort=False)]
```

(continues on next page)

(continued from previous page)

```
>>> [T8.seidel_switching(x, inplace=False).is_isomorphic(G)
↪# needs sage.modules
...  for x, G in zip(s, chang_graphs)]
[True, True, True]
```

chessboard = <module 'sage.graphs.generators.chessboard' from '/run/media/hsy/3000gb/sagemath/sage/src/sage/graphs/generators/chessboard.py'>

classical_geometries = <module 'sage.graphs.generators.classical_geometries' from '/run/media/hsy/3000gb/sagemath/sage/src/sage/graphs/generators/classical_geometries.py'>

static cocliques_HoffmannSingleton(*immutable=False*)

Return the graph obtained from the cocliques of the Hoffmann-Singleton graph.

This is a distance-regular graph with intersection array $[15, 14, 10, 3; 1, 5, 12, 15]$.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: G = graphs.cocliques_HoffmannSingleton()
sage: G.is_distance_regular(True)
([15, 14, 10, 3, None], [None, 1, 5, 12, 15])
```

```
>>> from sage.all import *
>>> G = graphs.cocliques_HoffmannSingleton()
>>> G.is_distance_regular(True)
([15, 14, 10, 3, None], [None, 1, 5, 12, 15])
```

REFERENCES:

The construction of this graph can be found in [BCN1989] p. 392.

static cographs(*n*, *as_graph=True*, *immutable=False*)

Return an iterator over the cographs of order n.

A cograph is a P_4-free graph, that is a graph without induced path of order 4. Any cograph may be constructed, starting from the single vertex graph, by a sequence of *sage.graphs.graph.Graph.join()* and *sage.graphs.generic_graph.GenericGraph.disjoint_union()* operations. See the Wikipedia article Cograph for more details.

This method implements the generator of all cographs of order n proposed in [JPD2018]. The algorithm generates one by one every cotree with n nodes, and each cotree is generated by using its previous cotree. The time to construct the first cotree is $O(n)$ and the time spent between two consecutive outputs is $O(n)$. Hence, the overall complexity of the algorithm is $O(n * M_n)$, where n is the number of nodes and M_n is the total number of cographs with n nodes (see OEIS sequence A000084).

INPUT:

- `n` – integer larger or equal to 1
- `as_graph` – boolean (default: `True`); whether to return graphs or the tree data structure encoding the graph
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph. This parameter is used only when `as_graph is True`.

EXAMPLES:

The output can be either cotrees or graphs:

```
sage: for t in graphs.cographs(3, as_graph=False):
....:     print(t)
[ J ( 0 ) ( 1 ) ( 2 ) ]
[ J [ U ( 0 ) ( 1 ) ( 2 ) ] ]
[ J ( 0 ) [ U ( 1 ) ( 2 ) ] ]
[ J [ U ( 0 ) [ J ( 1 ) ( 2 ) ] ] ]
sage: for g in graphs.cographs(3, as_graph=True):
....:     print(g.edges(labels=False, sort=True))
[(0, 1), (0, 2), (1, 2)]
[]
[(0, 1), (0, 2)]
[(1, 2)]
```

```
>>> from sage.all import *
>>> for t in graphs.cographs(Integer(3), as_graph=False):
...     print(t)
[ J ( 0 ) ( 1 ) ( 2 ) ]
[ J [ U ( 0 ) ( 1 ) ( 2 ) ] ]
[ J ( 0 ) [ U ( 1 ) ( 2 ) ] ]
[ J [ U ( 0 ) [ J ( 1 ) ( 2 ) ] ] ]
>>> for g in graphs.cographs(Integer(3), as_graph=True):
...     print(g.edges(labels=False, sort=True))
[(0, 1), (0, 2), (1, 2)]
[]
[(0, 1), (0, 2)]
[(1, 2)]
```

Check that we agree with OEIS sequence A000084:

```
sage: [sum(1 for _ in graphs.cographs(n, as_graph=False))
....:  for n in range(1, 8)]
[1, 2, 4, 10, 24, 66, 180]
```

```
>>> from sage.all import *
>>> [sum(Integer(1) for _ in graphs.cographs(n, as_graph=False))
...  for n in range(Integer(1), Integer(8))]
[1, 2, 4, 10, 24, 66, 180]
```

cographs_module = <module 'sage.graphs.cographs' from '/run/media/hsy/3000gb/sagemath/sage/src/sage/graphs/cographs.py'>

cospectral_graphs(*vertices*, *matrix_function=None*, *graphs=None*, *immutable=False*)

Find all sets of graphs on `vertices` vertices (with possible restrictions) which are cospectral with respect to a constructed matrix.

INPUT:

- `vertices` – the number of vertices in the graphs to be tested
- `matrix_function` – a function taking a graph and giving back a matrix. This defaults to the adjacency matrix. The spectra examined are the spectra of these matrices.

- `graphs` – one of three things:
  - `None` – default; test all graphs having `vertices` vertices
  - a function taking a graph and returning `True` or `False` - test only the graphs on `vertices` vertices for which the function returns `True`
  - a list of graphs (or other iterable object) – these graphs are tested for cospectral sets. In this case, `vertices` is ignored.
- `immutable` – boolean (default: `False`); whether to return immutable or mutable graphs

OUTPUT:

A list of lists of graphs. Each sublist will be a list of cospectral graphs (lists of cardinality 1 being omitted).

See also

Graph.is_strongly_regular() – tests whether a graph is strongly regular and/or returns its parameters.

EXAMPLES:

```
sage: g = graphs.cospectral_graphs(5)                                          #␣
↪needs sage.modules
sage: sorted(sorted(g.graph6_string() for g in glist) for glist in g)          #␣
↪needs sage.modules
[['Dr?', 'Ds_']]
sage: g[0][1].am().charpoly()==g[0][1].am().charpoly()                         #␣
↪needs sage.modules
True
```

```
>>> from sage.all import *
>>> g = graphs.cospectral_graphs(Integer(5))                                    ␣
↪     # needs sage.modules
>>> sorted(sorted(g.graph6_string() for g in glist) for glist in g)          #␣
↪needs sage.modules
[['Dr?', 'Ds_']]
>>> g[Integer(0)][Integer(1)].am().charpoly()==g[Integer(0)][Integer(1)].am().
↪charpoly()                        # needs sage.modules
True
```

There are two sets of cospectral graphs on six vertices with no isolated vertices:

```
sage: g = graphs.cospectral_graphs(6, graphs=lambda x: min(x.degree())>0)
sage: sorted(sorted(g.graph6_string() for g in glist) for glist in g)
[['Ep__', 'Er?G'], ['ExGg', 'ExoG']]
sage: g[0][1].am().charpoly()==g[0][1].am().charpoly()
True
sage: g[1][1].am().charpoly()==g[1][1].am().charpoly()
True
```

```
>>> from sage.all import *
>>> g = graphs.cospectral_graphs(Integer(6), graphs=lambda x: min(x.degree())>
```

(continues on next page)

(continued from previous page)

```
↪Integer(0))
>>> sorted(sorted(g.graph6_string() for g in glist) for glist in g)
[['Ep__', 'Er?G'], ['ExGg', 'ExoG']]
>>> g[Integer(0)][Integer(1)].am().charpoly()==g[Integer(0)][Integer(1)].am().
↪charpoly()
True
>>> g[Integer(1)][Integer(1)].am().charpoly()==g[Integer(1)][Integer(1)].am().
↪charpoly()
True
```

There is one pair of cospectral trees on eight vertices:

```
sage: g = graphs.cospectral_graphs(6, graphs=graphs.trees(8))                #␣
↪needs sage.modules
sage: sorted(sorted(g.graph6_string() for g in glist) for glist in g)        #␣
↪needs sage.modules
[['GiPC?C', 'GiQCC?']]
sage: g[0][1].am().charpoly()==g[0][1].am().charpoly()                        #␣
↪needs sage.modules
True
```

```
>>> from sage.all import *
>>> g = graphs.cospectral_graphs(Integer(6), graphs=graphs.trees(Integer(8)))␣
↪              # needs sage.modules
>>> sorted(sorted(g.graph6_string() for g in glist) for glist in g)        #␣
↪needs sage.modules
[['GiPC?C', 'GiQCC?']]
>>> g[Integer(0)][Integer(1)].am().charpoly()==g[Integer(0)][Integer(1)].am().
↪charpoly()                      # needs sage.modules
True
```

There are two sets of cospectral graphs (with respect to the Laplacian matrix) on six vertices:

```
sage: g = graphs.cospectral_graphs(6, matrix_function=lambda g: g.laplacian_
↪matrix())
sage: sorted(sorted(g.graph6_string() for g in glist) for glist in g)
[['Edq_', 'ErcG'], ['Exoo', 'EzcG']]
sage: g[0][1].laplacian_matrix().charpoly()==g[0][1].laplacian_matrix().
↪charpoly()
True
sage: g[1][1].laplacian_matrix().charpoly()==g[1][1].laplacian_matrix().
↪charpoly()
True
```

```
>>> from sage.all import *
>>> g = graphs.cospectral_graphs(Integer(6), matrix_function=lambda g: g.
↪laplacian_matrix())
>>> sorted(sorted(g.graph6_string() for g in glist) for glist in g)
[['Edq_', 'ErcG'], ['Exoo', 'EzcG']]
>>> g[Integer(0)][Integer(1)].laplacian_matrix().
↪charpoly()==g[Integer(0)][Integer(1)].laplacian_matrix().charpoly()
True
```

(continues on next page)

(continued from previous page)

```
>>> g[Integer(1)][Integer(1)].laplacian_matrix().
↪charpoly()==g[Integer(1)][Integer(1)].laplacian_matrix().charpoly()
True
```

To find cospectral graphs with respect to the normalized Laplacian, assuming the graphs do not have an isolated vertex, it is enough to check the spectrum of the matrix $D^{-1}A$, where D is the diagonal matrix of vertex degrees, and A is the adjacency matrix. We find two such cospectral graphs (for the normalized Laplacian) on five vertices:

```
sage: def DinverseA(g):
....:     A = g.adjacency_matrix().change_ring(QQ)
....:     for i in range(g.order()):
....:         A.rescale_row(i, 1 / len(A.nonzero_positions_in_row(i)))
....:     return A
sage: g = graphs.cospectral_graphs(5, matrix_function=DinverseA,            #␣
↪needs sage.libs.pari sage.modules
....:                              graphs=lambda g: min(g.degree()) > 0)
sage: sorted(sorted(g.graph6_string() for g in glist) for glist in g)       #␣
↪needs sage.modules
[['Dlg', 'Ds_']]
sage: (g[0][1].laplacian_matrix(normalized=True).charpoly()                 #␣
↪needs sage.modules sage.symbolic
....:     == g[0][1].laplacian_matrix(normalized=True).charpoly())
True
```

```
>>> from sage.all import *
>>> def DinverseA(g):
...     A = g.adjacency_matrix().change_ring(QQ)
...     for i in range(g.order()):
...         A.rescale_row(i, Integer(1) / len(A.nonzero_positions_in_row(i)))
...     return A
>>> g = graphs.cospectral_graphs(Integer(5), matrix_function=DinverseA,      ␣
↪      # needs sage.libs.pari sage.modules
...                               graphs=lambda g: min(g.degree()) >␣
↪Integer(0))
>>> sorted(sorted(g.graph6_string() for g in glist) for glist in g)       #␣
↪needs sage.modules
[['Dlg', 'Ds_']]
>>> (g[Integer(0)][Integer(1)].laplacian_matrix(normalized=True).charpoly()  ␣
↪              # needs sage.modules sage.symbolic
...     == g[Integer(0)][Integer(1)].laplacian_matrix(normalized=True).
↪charpoly())
True
```

degree_sequence = <module 'sage.graphs.generators.degree_sequence' from '/run/media/hsy/3000gb/sagemath/sage/src/sage/graphs/generators/degree_sequence.py'>

static distance_3_doubly_truncated_Golay_code_graph(*immutable=False*)

Return a distance-regular graph with intersection array $[9, 8, 6, 3; 1, 1, 3, 8]$.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: # long time, needs sage.modules sage.rings.finite_rings
sage: G = graphs.distance_3_doubly_truncated_Golay_code_graph()
sage: G.is_distance_regular(True)       # long time (due to above)
([9, 8, 6, 3, None], [None, 1, 1, 3, 8])
```

```
>>> from sage.all import *
>>> # long time, needs sage.modules sage.rings.finite_rings
>>> G = graphs.distance_3_doubly_truncated_Golay_code_graph()
>>> G.is_distance_regular(True)       # long time (due to above)
([9, 8, 6, 3, None], [None, 1, 1, 3, 8])
```

ALGORITHM:

Compute the binary Golay code and truncate it twice. Compute its coset graph. Take a vertex and compute the set of vertices at distance 3 from the vertex chosen. This set constitutes the set of vertices of our distance-regular graph. Moreover we have an edge (u, v) if the coset graph contains such edge.

REFERENCES:

Description and construction of this graph are taken from [BCN1989] p. 364.

distance_regular = <module 'sage.graphs.generators.distance_regular' from '/run/media/hsy/3000gb/sagemath/sage/build/sage-distro/src/sage/graphs/generators/distance_regular.cpython-314-x86_64-linux-gnu.so'>

static distance_regular_graph(*arr*, *existence=False*, *check=True*, *immutable=False*)

Return a distance-regular graph with the intersection array given.

INPUT:

- `arr` – list; intersection array of the graph
- `existence` – boolean (optional); instead of building the graph return:
  - `True` – if a graph with the given intersection array exists;
  - `False` – if there is no graph with the given intersection array;
  - `Unknown` – if Sage doesn't know if such a graph exists
- `check` – boolean (default: `True`); if `True`, then checks that the result of this function has the given intersection array
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: graphs.distance_regular_graph([21,20,16,1,2,12], existence=True)
True
sage: G = graphs.distance_regular_graph([12,11,10,7,1,2,5,12], check=False)
↪  # needs sage.combinat sage.modules
sage: G.is_distance_regular(True)
↪  # needs sage.combinat sage.modules
([12, 11, 10, 7, None], [None, 1, 2, 5, 12])
```

```
>>> from sage.all import *
>>> graphs.distance_regular_graph([Integer(21),Integer(20),Integer(16),
```

(continues on next page)

(continued from previous page)

```
↪Integer(1),Integer(2),Integer(12)], existence=True)
True
>>> G = graphs.distance_regular_graph([Integer(12),Integer(11),Integer(10),
↪Integer(7),Integer(1),Integer(2),Integer(5),Integer(12)], check=False)
↪# needs sage.combinat sage.modules
>>> G.is_distance_regular(True)
↪# needs sage.combinat sage.modules
([12, 11, 10, 7, None], [None, 1, 2, 5, 12])
```

REFERENCES:

See [BCN1989] and [VDKT2016].

families = <module 'sage.graphs.generators.families' from '/run/media/hsy/3000gb/sagemath/sage/src/sage/graphs/generators/families.py'>

fullerenes(*order*, *ipr=False*, *immutable=False*)

Return a generator which creates fullerene graphs using the buckygen generator (see [BGM2012]).

INPUT:

- `order` – a positive even integer smaller than or equal to 254 This specifies the number of vertices in the generated fullerenes
- `ipr` – boolean (default: `False`); if `True` only fullerenes that satisfy the Isolated Pentagon Rule are generated. This means that no pentagonal faces share an edge.
- `immutable` – boolean (default: `False`); whether to return immutable or mutable graphs

OUTPUT:

A generator which will produce the fullerene graphs as Sage graphs with an embedding set. These will be simple graphs: no loops, no multiple edges, no directed edges.

See also

- *set_embedding()*, *get_embedding()* – get/set methods for embeddings.

EXAMPLES:

There are 1812 isomers of C_{60}, i.e., 1812 fullerene graphs on 60 vertices:

```
sage: gen = graphs.fullerenes(60)  # optional - buckygen
sage: len(list(gen))               # optional - buckygen
1812
```

```
>>> from sage.all import *
>>> gen = graphs.fullerenes(Integer(60))  # optional - buckygen
>>> len(list(gen))                # optional - buckygen
1812
```

However, there is only one IPR fullerene graph on 60 vertices: the famous Buckminster Fullerene:

```
sage: gen = graphs.fullerenes(60, ipr=True)  # optional - buckygen
sage: next(gen)                              # optional - buckygen
```

(continues on next page)

(continued from previous page)

```
Graph on 60 vertices
sage: next(gen)                                  # optional - buckygen
Traceback (most recent call last):
...
StopIteration
```

```
>>> from sage.all import *
>>> gen = graphs.fullerenes(Integer(60), ipr=True)  # optional - buckygen
>>> next(gen)                                    # optional - buckygen
Graph on 60 vertices
>>> next(gen)                                    # optional - buckygen
Traceback (most recent call last):
...
StopIteration
```

The unique fullerene graph on 20 vertices is isomorphic to the dodecahedron graph.

```
sage: # optional - buckygen
sage: gen = graphs.fullerenes(20)
sage: g = next(gen)
sage: g.is_isomorphic(graphs.DodecahedralGraph())
True
sage: g.get_embedding()
{1: [2, 3, 4],
 2: [1, 5, 6],
 3: [1, 7, 8],
 4: [1, 9, 10],
 5: [2, 10, 11],
 6: [2, 12, 7],
 7: [3, 6, 13],
 8: [3, 14, 9],
 9: [4, 8, 15],
 10: [4, 16, 5],
 11: [5, 17, 12],
 12: [6, 11, 18],
 13: [7, 18, 14],
 14: [8, 13, 19],
 15: [9, 19, 16],
 16: [10, 15, 17],
 17: [11, 16, 20],
 18: [12, 20, 13],
 19: [14, 20, 15],
 20: [17, 19, 18]}
sage: g.plot3d(layout='spring')
Graphics3d Object
```

```
>>> from sage.all import *
>>> # optional - buckygen
>>> gen = graphs.fullerenes(Integer(20))
>>> g = next(gen)
>>> g.is_isomorphic(graphs.DodecahedralGraph())
True
```

(continues on next page)

(continued from previous page)

```
>>> g.get_embedding()
{1: [2, 3, 4],
 2: [1, 5, 6],
 3: [1, 7, 8],
 4: [1, 9, 10],
 5: [2, 10, 11],
 6: [2, 12, 7],
 7: [3, 6, 13],
 8: [3, 14, 9],
 9: [4, 8, 15],
 10: [4, 16, 5],
 11: [5, 17, 12],
 12: [6, 11, 18],
 13: [7, 18, 14],
 14: [8, 13, 19],
 15: [9, 19, 16],
 16: [10, 15, 17],
 17: [11, 16, 20],
 18: [12, 20, 13],
 19: [14, 20, 15],
 20: [17, 19, 18]}
>>> g.plot3d(layout='spring')
Graphics3d Object
```

fusenes(*hexagon_count*, *benzenoids=False*, *immutable=False*)

Return a generator which creates fusenes and benzenoids using the benzene generator (see [BCH2002]). Fusenes are planar polycyclic hydrocarbons with all bounded faces hexagons. Benzenoids are fusenes that are subgraphs of the hexagonal lattice.

INPUT:

- `hexagon_count` – positive integer smaller than or equal to 30; this specifies the number of hexagons in the generated benzenoids
- `benzenoids` – boolean (default: `False`); if `True` only benzenoids are generated
- `immutable` – boolean (default: `False`); whether to return immutable or mutable graphs

OUTPUT:

A generator which will produce the fusenes as Sage graphs with an embedding set. These will be simple graphs: no loops, no multiple edges, no directed edges.

See also

- *set_embedding()*, *get_embedding()* – get/set methods for embeddings.

EXAMPLES:

There is a unique fusene with 2 hexagons:

```
sage: gen = graphs.fusenes(2)  # optional - benzene
sage: len(list(gen))           # optional - benzene
1
```

```
>>> from sage.all import *
>>> gen = graphs.fusenes(Integer(2))  # optional - benzene
>>> len(list(gen))           # optional - benzene
1
```

This fusene is naphthalene ($C_{10}H_8$). In the fusene graph the H-atoms are not stored, so this is a graph on just 10 vertices:

```
sage: gen = graphs.fusenes(2)  # optional - benzene
sage: next(gen)                # optional - benzene
Graph on 10 vertices
sage: next(gen)                # optional - benzene
Traceback (most recent call last):
...
StopIteration
```

```
>>> from sage.all import *
>>> gen = graphs.fusenes(Integer(2))  # optional - benzene
>>> next(gen)                # optional - benzene
Graph on 10 vertices
>>> next(gen)                # optional - benzene
Traceback (most recent call last):
...
StopIteration
```

There are 6505 benzenoids with 9 hexagons:

```
sage: gen = graphs.fusenes(9, benzenoids=True)  # optional - benzene
sage: len(list(gen))                            # optional - benzene
6505
```

```
>>> from sage.all import *
>>> gen = graphs.fusenes(Integer(9), benzenoids=True)  # optional - benzene
>>> len(list(gen))                           # optional - benzene
6505
```

gen_trees = <module 'sage.graphs.generators.trees' from '/run/media/hsy/3000gb/sagemath/sage/build/sage-distro/src/sage/graphs/generators/trees.cpython-314-x86_64-linux-gnu.so'>

static graph_3O73(*immutable=False*)

Return the graph related to the group $3O(7,3)$.

This graph is distance-regular with intersection array $[117, 80, 24, 1; 1, 12, 80, 117]$.

The graph is also distance transitive with $3.O(7,3)$ as automorphism group

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: G = graphs.graph_3O73()                              # optional - internet␣
↪gap_package_atlasrep
```

(continues on next page)

(continued from previous page)

```
sage: G.is_distance_regular(True)                          # optional - internet␣
↪gap_package_atlasrep
([117, 80, 24, 1, None], [None, 1, 12, 80, 117])
```

```
>>> from sage.all import *
>>> G = graphs.graph_3073()                                # optional - internet␣
↪gap_package_atlasrep
>>> G.is_distance_regular(True)                            # optional - internet␣
↪gap_package_atlasrep
([117, 80, 24, 1, None], [None, 1, 12, 80, 117])
```

REFERENCES:

A description and construction of this graph can be found in [BCN1989] p. 400.

intersection = <module 'sage.graphs.generators.intersection' from '/run/media/hsy/3000gb/sagemath/sage/src/sage/graphs/generators/intersection.py'>

static line_graph_forbidden_subgraphs(*immutable=False*)

Return the 9 forbidden subgraphs of a line graph.

See the Wikipedia article Line_graph for more information.

The graphs are returned in the ordering given by the Wikipedia drawing, read from left to right and from top to bottom.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: graphs.line_graph_forbidden_subgraphs()
[Claw graph: Graph on 4 vertices,
Graph on 6 vertices,
Graph on 6 vertices,
Graph on 5 vertices,
Graph on 6 vertices,
Graph on 6 vertices,
Graph on 6 vertices,
Graph on 6 vertices,
Graph on 5 vertices]
```

```
>>> from sage.all import *
>>> graphs.line_graph_forbidden_subgraphs()
[Claw graph: Graph on 4 vertices,
Graph on 6 vertices,
Graph on 6 vertices,
Graph on 5 vertices,
Graph on 6 vertices,
Graph on 6 vertices,
Graph on 6 vertices,
Graph on 6 vertices,
Graph on 5 vertices]
```

static locally_GQ42_distance_transitive_graph(*immutable=False*)

Return the unique amply regular graph with $\mu = 6$ which is locally a generalised quadrangle.

This graph is distance-regular with intersection array $[45, 32, 12, 1; 1, 6, 32, 45]$.

This graph is also distance-transitive.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: G = graphs.locally_GQ42_distance_transitive_graph()        # optional -␣
↪internet gap_package_atlasrep
sage: G.is_distance_regular(True)                                # optional -␣
↪internet gap_package_atlasrep
([45, 32, 12, 1, None], [None, 1, 6, 32, 45])
```

```
>>> from sage.all import *
>>> G = graphs.locally_GQ42_distance_transitive_graph()        # optional -␣
↪internet gap_package_atlasrep
>>> G.is_distance_regular(True)                                # optional -␣
↪internet gap_package_atlasrep
([45, 32, 12, 1, None], [None, 1, 6, 32, 45])
```

REFERENCES:

A description of this graph can be found in [BCN1989] p.399. This construction is due to Dima Pasechnik.

luw_graphs = <module 'sage.graphs.generators.luw_graphs' from '/run/media/hsy/3000gb/sagemath/sage/src/sage/graphs/generators/luw_graphs.py'>

nauty_genbg(*options=''*, *debug=False*, *immutable=False*)

Return a generator which creates bipartite graphs from nauty's `genbgL` program.

INPUT:

- `options` – string (default: `""`); a string passed to `genbgL` as if it was run at a system command line. At a minimum, you *must* pass the number of vertices you desire in each side. Sage expects the bipartite graphs to be in nauty's "graph6" format, do not set an option to change this default or results will be unpredictable.
- `debug` – boolean (default: `False`); if `True` the first line of `geng`'s output to standard error is captured and the first call to the generator's `next()` function will return this line as a string. A line leading with ">A" indicates a successful initiation of the program with some information on the arguments, while a line beginning with ">E" indicates an error with the input.
- `immutable` – boolean (default: `False`); whether to return immutable or mutable graphs

The possible options, obtained as output of `genbgL --help`:

```
    n1       : the number of vertices in the first class.
               We must have n1=1..30.
    n2       : the number of vertices in the second class.
               We must have n2=0..64 and n1+n2=1..64.
mine:maxe    : <int>:<int> a range for the number of edges
                <int>:0 means '<int> or more' except in the case 0:0
  res/mod    : only generate subset res out of subsets 0..mod-1
```

(continues on next page)

(continued from previous page)

```
    -c       : only write connected graphs
    -z       : all the vertices in the second class must have
               different neighbourhoods
    -F       : the vertices in the second class must have at least
               two neighbours of degree at least 2
    -L       : there is no vertex in the first class whose removal
               leaves the vertices in the second class unreachable
               from each other
    -Y<int>  : two vertices in the second class must have at least
               <int> common neighbours
    -Z<int>  : two vertices in the second class must have at most
               <int> common neighbours
    -A       : no vertex in the second class has a neighbourhood
               which is a subset of another vertex's neighbourhood
               in the second class
    -D<int>  : specify an upper bound for the maximum degree
               Example: -D6. You can also give separate maxima for
               the two parts, for example: -D5:6
    -d<int>  : specify a lower bound for the minimum degree
               Again, you can specify it separately for the two parts,
               for example -d1:2
    -v       : display counts by number of edges to stderr
    -l       : canonically label output graphs
```

Options which cause `genbgL` to use an output format different than the `graph6` format are not listed above (`-s`, `-a`) as they will confuse the creation of a Sage graph. Option `-q` which suppress auxiliary output (except from `-v`) should never be used as we are unable to recover the partition of the vertices of the bipartite graph without the auxiliary output. Hence the partition of the vertices of returned bipartite graphs might not respect the requirement.

The res/mod option can be useful when using the output in a routine run several times in parallel.

OUTPUT:

A generator which will produce the graphs as *BipartiteGraph*. These will be simple bipartite graphs: no loops, no multiple edges, no directed edges.

EXAMPLES:

The generator can be used to construct bipartite graphs for testing, one at a time (usually inside a loop). Or it can be used to create an entire list all at once if there is sufficient memory to contain it:

```
sage: gen = graphs.nauty_genbg("1 1")
sage: next(gen)
Bipartite graph on 2 vertices
sage: next(gen)
Bipartite graph on 2 vertices
sage: next(gen)
Traceback (most recent call last):
...
StopIteration
```

```
>>> from sage.all import *
>>> gen = graphs.nauty_genbg("1 1")
```

(continues on next page)

(continued from previous page)

```
>>> next(gen)
Bipartite graph on 2 vertices
>>> next(gen)
Bipartite graph on 2 vertices
>>> next(gen)
Traceback (most recent call last):
...
StopIteration
```

Connected bipartite graphs of order 6 with different number of vertices in each side:

```
sage: gen = graphs.nauty_genbg("1 5 -c")
sage: len(list(gen))
1
sage: gen = graphs.nauty_genbg("2 4 -c")
sage: len(list(gen))
6
sage: gen = graphs.nauty_genbg("3 3 -c")
sage: len(list(gen))
13
```

```
>>> from sage.all import *
>>> gen = graphs.nauty_genbg("1 5 -c")
>>> len(list(gen))
1
>>> gen = graphs.nauty_genbg("2 4 -c")
>>> len(list(gen))
6
>>> gen = graphs.nauty_genbg("3 3 -c")
>>> len(list(gen))
13
```

Use *nauty_geng()* instead if you want the list of all bipartite graphs of order n. For instance, the list of all connected bipartite graphs of order 6, which agrees with OEIS sequence A005142:

```
sage: gen = graphs.nauty_geng("-b -c 6")
sage: len(list(gen))
17
```

```
>>> from sage.all import *
>>> gen = graphs.nauty_geng("-b -c 6")
>>> len(list(gen))
17
```

The `debug` switch can be used to examine `genbgL`'s reaction to the input in the `options` string. A message starting with ">A" indicates success and a message starting with ">E" indicates a failure:

```
sage: gen = graphs.nauty_genbg("2 3", debug=True)
sage: print(next(gen))
>A ...genbg... n=2+3 e=0:6 d=0:0 D=3:2
sage: gen = graphs.nauty_genbg("-c2 3", debug=True)
sage: next(gen)
```

(continues on next page)

(continued from previous page)

```
'>E Usage: ...genbg... [-c -ugs -vq -lzF] [-Z#] [-D#] [-A] [-d#|-d#:#] [-D#|-D
↪#:#] n1 n2...
```

```
>>> from sage.all import *
>>> gen = graphs.nauty_genbg("2 3", debug=True)
>>> print(next(gen))
>A ...genbg... n=2+3 e=0:6 d=0:0 D=3:2
>>> gen = graphs.nauty_genbg("-c2 3", debug=True)
>>> next(gen)
'>E Usage: ...genbg... [-c -ugs -vq -lzF] [-Z#] [-D#] [-A] [-d#|-d#:#] [-D#|-D
↪#:#] n1 n2...
```

Check that the partition of the bipartite graph is consistent:

```
sage: gen = graphs.nauty_genbg("3 3")
sage: left = set(range(3))
sage: for g in gen:
....:     if g.left != left:
....:         raise ValueError('wrong partition')
```

```
>>> from sage.all import *
>>> gen = graphs.nauty_genbg("3 3")
>>> left = set(range(Integer(3)))
>>> for g in gen:
...     if g.left != left:
...         raise ValueError('wrong partition')
```

nauty_geng(*options=''*, *debug=False*, *immutable=False*)

Return a generator which creates graphs from nauty's geng program.

INPUT:

- `options` – string (default: `''`); a string passed to `geng` as if it was run at a system command line. At a minimum, you *must* pass the number of vertices you desire. Sage expects the graphs to be in nauty's "graph6" format, do not set an option to change this default or results will be unpredictable.
- `debug` – boolean (default: `False`); if `True` the first line of `geng`'s output to standard error is captured and the first call to the generator's `next()` function will return this line as a string. A line leading with ">A" indicates a successful initiation of the program with some information on the arguments, while a line beginning with ">E" indicates an error with the input.
- `immutable` – boolean (default: `False`); whether to return immutable or mutable graphs

The possible options, obtained as output of `geng --help`:

```
     n       : the number of vertices
mine:maxe    : <int>:<int> a range for the number of edges
                  <int>:0 means '<int> or more' except in the case 0:0
  res/mod : only generate subset res out of subsets 0..mod-1

    -c       : only write connected graphs
    -C       : only write biconnected graphs
    -t       : only generate triangle-free graphs
    -f       : only generate 4-cycle-free graphs
```

(continues on next page)

(continued from previous page)

```
    -b       : only generate bipartite graphs
                   (-t, -f and -b can be used in any combination)
    -m       : save memory at the expense of time (only makes a
                   difference in the absence of -b, -t, -f and n <= 28).
    -d<int>  : a lower bound for the minimum degree
    -D<int>  : a upper bound for the maximum degree
    -v       : display counts by number of edges
    -l       : canonically label output graphs

    -q       : suppress auxiliary output (except from -v)
```

Options which cause `geng` to use an output format different than the graph6 format are not listed above (-u, -g, -s, -y, -h) as they will confuse the creation of a Sage graph. The res/mod option can be useful when using the output in a routine run several times in parallel.

OUTPUT:

A generator which will produce the graphs as Sage graphs. These will be simple graphs: no loops, no multiple edges, no directed edges.

See also

Graph.is_strongly_regular() – tests whether a graph is strongly regular and/or returns its parameters.

EXAMPLES:

The generator can be used to construct graphs for testing, one at a time (usually inside a loop). Or it can be used to create an entire list all at once if there is sufficient memory to contain it.

```
sage: gen = graphs.nauty_geng("2")
sage: next(gen)
Graph on 2 vertices
sage: next(gen)
Graph on 2 vertices
sage: next(gen)
Traceback (most recent call last):
...
StopIteration
```

```
>>> from sage.all import *
>>> gen = graphs.nauty_geng("2")
>>> next(gen)
Graph on 2 vertices
>>> next(gen)
Graph on 2 vertices
>>> next(gen)
Traceback (most recent call last):
...
StopIteration
```

A list of all graphs on 7 vertices. This agrees with OEIS sequence A000088.

```
sage: gen = graphs.nauty_geng("7")
sage: len(list(gen))
1044
```

```
>>> from sage.all import *
>>> gen = graphs.nauty_geng("7")
>>> len(list(gen))
1044
```

A list of just the connected graphs on 7 vertices. This agrees with OEIS sequence A001349.

```
sage: gen = graphs.nauty_geng("7 -c")
sage: len(list(gen))
853
```

```
>>> from sage.all import *
>>> gen = graphs.nauty_geng("7 -c")
>>> len(list(gen))
853
```

A list of connected degree exactly 2 graphs on 5 vertices.

```
sage: gen = graphs.nauty_geng("5 -c -d2 -D2")
sage: len(list(gen))
1
```

```
>>> from sage.all import *
>>> gen = graphs.nauty_geng("5 -c -d2 -D2")
>>> len(list(gen))
1
```

The `debug` switch can be used to examine `geng`'s reaction to the input in the `options` string. We illustrate success. (A failure will be a string beginning with ">E".) Passing the "-q" switch to `geng` will suppress the indicator of a successful initiation, and so the first returned value might be an empty string if `debug` is `True`:

```
sage: gen = graphs.nauty_geng("4", debug=True)
sage: print(next(gen))
>A ...geng -d0D3 n=4 e=0-6
sage: gen = graphs.nauty_geng("4 -q", debug=True)
sage: next(gen)
''
```

```
>>> from sage.all import *
>>> gen = graphs.nauty_geng("4", debug=True)
>>> print(next(gen))
>A ...geng -d0D3 n=4 e=0-6
>>> gen = graphs.nauty_geng("4 -q", debug=True)
>>> next(gen)
''
```

nauty_genktreeg(*options=''*, *debug=False*, *immutable=False*)

Return a generator which creates all k-trees using nauty.

A k-tree is an undirected graph formed by starting with a complete graph on $k+1$ vertices and then repeatedly add vertices in such a way that each added vertex v has exactly k neighbors U such that, together, the $k+1$ vertices formed by v and U form a clique. See the Wikipedia article K-tree for more details.

INPUT:

- `options` – string (default: `""`); a string passed to `genktreeg` as if it was run at a system command line. At a minimum, you *must* pass the number of vertices you desire. Sage expects the graphs to be in nauty's "graph6" format, do not set an option to change this default or results will be unpredictable.
- `debug` – boolean (default: `False`); if `True` the first line of `genktreeg`'s output to standard error is captured and the first call to the generator's `next()` function will return this line as a string. A line leading with ">A" indicates a successful initiation of the program with some information on the arguments, while a line beginning with ">E" indicates an error with the input.
- `immutable` – boolean (default: `False`); whether to return immutable or mutable graphs

The possible options, obtained as output of `genktreeg --help`:

```
   n        : the number of vertices
  -k<int>   : the value of `k`(default: 2)
res/mod     : only generate subset res out of subsets 0..mod-1
  -l        : canonically label output graphs
```

Options which cause `genktreeg` to use an output format different than the graph6 format are not listed above (-u, -s, -h) as they will confuse the creation of a Sage graph. The res/mod option can be useful when using the output in a routine run several times in parallel.

OUTPUT:

A generator which will produce the graphs as Sage graphs. These will be simple graphs: no loops, no multiple edges, no directed edges.

EXAMPLES:

A k-tree is a maximal graph with treewidth k:

```
sage: # needs nauty
sage: gen = graphs.nauty_genktreeg("10 -k4")
sage: G = next(gen); G
Graph on 10 vertices
sage: G.treewidth()
4
```

```
>>> from sage.all import *
>>> # needs nauty
>>> gen = graphs.nauty_genktreeg("10 -k4")
>>> G = next(gen); G
Graph on 10 vertices
>>> G.treewidth()
4
```

A list of all 2-trees with 6, 7 and 8 vertices. This agrees with OEIS sequence A054581:

```
sage: # needs nauty
sage: gen = graphs.nauty_genktreeg("6")
sage: len(list(gen))
5
```

(continues on next page)

(continued from previous page)

```
sage: gen = graphs.nauty_genktreeg("7")
sage: len(list(gen))
12
sage: gen = graphs.nauty_genktreeg("8")
sage: len(list(gen))
39
```

```
>>> from sage.all import *
>>> # needs nauty
>>> gen = graphs.nauty_genktreeg("6")
>>> len(list(gen))
5
>>> gen = graphs.nauty_genktreeg("7")
>>> len(list(gen))
12
>>> gen = graphs.nauty_genktreeg("8")
>>> len(list(gen))
39
```

The `debug` switch can be used to examine `geng`'s reaction to the input in the `options` string. We illustrate success. (A failure will be a string beginning with ">E".) Passing the "-q" switch to `geng` will suppress the indicator of a successful initiation, and so the first returned value might be an empty string if `debug` is `True`:

```
sage: gen = graphs.nauty_genktreeg("7", debug=True)                    #␣
↪needs nauty
sage: print(next(gen))                                                 #␣
↪needs nauty
>A ...genktreeg k=2 n=7
```

```
>>> from sage.all import *
>>> gen = graphs.nauty_genktreeg("7", debug=True)                      #␣
↪needs nauty
>>> print(next(gen))                                                   #␣
↪needs nauty
>A ...genktreeg k=2 n=7
```

static nauty_gentreeg(*options=''*, *debug=False*, *immutable=False*)

Return a generator which creates non-isomorphic trees from nauty's gentreeg program.

INPUT:

- `options` – string (default: `""`); a string passed to `gentreeg` as if it was run at a system command line. At a minimum, you *must* pass the number of vertices you desire. Sage expects the graphs to be in nauty's "sparse6" format, do not set an option to change this default or results will be unpredictable.
- `debug` – boolean (default: `False`); if `True` the first line of `gentreeg`'s output to standard error is captured and the first call to the generator's `next()` function will return this line as a string. A line leading with ">A" indicates a successful initiation of the program with some information on the arguments, while a line beginning with ">E" indicates an error with the input.
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

The possible options, obtained as output of `gentreeg -help`:

```
   n            : the number of vertices. Must be in range 1..128
res/mod         : only generate subset res out of subsets 0..mod-1
  -D<int>       : an upper bound for the maximum degree
  -Z<int>:<int> : bounds on the diameter
  -q            : suppress auxiliary output
```

Options which cause `gentreeg` to use an output format different than the sparse6 format are not listed above (-p, -l, -u) as they will confuse the creation of a Sage graph. The res/mod option can be useful when using the output in a routine run several times in parallel.

OUTPUT:

A generator which will produce the graphs as Sage graphs. These will be simple graphs: no loops, no multiple edges, no directed edges.

See also

trees() – another generator of trees

EXAMPLES:

The generator can be used to construct trees for testing, one at a time (usually inside a loop). Or it can be used to create an entire list all at once if there is sufficient memory to contain it:

```
sage: gen = graphs.nauty_gentreeg("4")
sage: next(gen)
Graph on 4 vertices
sage: next(gen)
Graph on 4 vertices
sage: next(gen)
Traceback (most recent call last):
...
StopIteration
```

```
>>> from sage.all import *
>>> gen = graphs.nauty_gentreeg("4")
>>> next(gen)
Graph on 4 vertices
>>> next(gen)
Graph on 4 vertices
>>> next(gen)
Traceback (most recent call last):
...
StopIteration
```

The number of trees on the first few vertex counts. This agrees with OEIS sequence A000055:

```
sage: [len(list(graphs.nauty_gentreeg(str(i)))) for i in range(1, 15)]
[1, 1, 1, 2, 3, 6, 11, 23, 47, 106, 235, 551, 1301, 3159]
```

```
>>> from sage.all import *
>>> [len(list(graphs.nauty_gentreeg(str(i)))) for i in range(Integer(1),␣
```

(continues on next page)

(continued from previous page)

```
↪Integer(15))]
[1, 1, 1, 2, 3, 6, 11, 23, 47, 106, 235, 551, 1301, 3159]
```

The `debug` switch can be used to examine `gentreeg`'s reaction to the input in the `options` string. We illustrate success. (A failure will be a string beginning with ">E".) Passing the "-q" switch to `gentreeg` will suppress the indicator of a successful initiation, and so the first returned value might be an empty string if `debug` is `True`:

```
sage: gen = graphs.nauty_gentreeg("4", debug=True)
sage: print(next(gen))
>A ...gentreeg ...
sage: gen = graphs.nauty_gentreeg("4 -q", debug=True)
sage: next(gen)
''
```

```
>>> from sage.all import *
>>> gen = graphs.nauty_gentreeg("4", debug=True)
>>> print(next(gen))
>A ...gentreeg ...
>>> gen = graphs.nauty_gentreeg("4 -q", debug=True)
>>> next(gen)
''
```

static p2_forbidden_minors(*immutable=False*)

Return an array containing the 35 minimal forbidden excluded minors of the projective plane.

We constructed the graphs given in Theorem 6.5.1 of [MT2001], which is a result of Archdeacon and encoded them in graph6 format. The order of the graphs is the same as they appear in [WA2025].

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

static petersen_family(*generate=False*, *immutable=False*)

Return the Petersen family.

The Petersen family is a collection of 7 graphs which are the forbidden minors of the linklessly embeddable graphs. For more information see the Wikipedia article Petersen_family.

INPUT:

- `generate` – boolean; whether to generate the family from the $\Delta - Y$ transformations. When set to `False` (default) a hardcoded version of the graphs (with a prettier layout) is returned.
- `immutable` – boolean (default: `False`); whether to return immutable or mutable graphs

EXAMPLES:

```
sage: graphs.petersen_family()
[Petersen graph: Graph on 10 vertices,
 Complete graph: Graph on 6 vertices,
 Multipartite Graph with set sizes [3, 3, 1]: Graph on 7 vertices,
 Graph on 8 vertices,
 Graph on 9 vertices,
 Graph on 7 vertices,
 Graph on 8 vertices]
```

```
>>> from sage.all import *
>>> graphs.petersen_family()
[Petersen graph: Graph on 10 vertices,
 Complete graph: Graph on 6 vertices,
 Multipartite Graph with set sizes [3, 3, 1]: Graph on 7 vertices,
 Graph on 8 vertices,
 Graph on 9 vertices,
 Graph on 7 vertices,
 Graph on 8 vertices]
```

The two different inputs generate the same graphs:

```
sage: F1 = graphs.petersen_family(generate=False)
sage: F2 = graphs.petersen_family(generate=True)                    ␣
↪  # needs sage.modules
sage: F1 = [g.canonical_label().graph6_string() for g in F1]
sage: F2 = [g.canonical_label().graph6_string() for g in F2]        ␣
↪  # needs sage.modules
sage: set(F1) == set(F2)                                            ␣
↪  # needs sage.modules
True
```

```
>>> from sage.all import *
>>> F1 = graphs.petersen_family(generate=False)
>>> F2 = graphs.petersen_family(generate=True)
↪# needs sage.modules
>>> F1 = [g.canonical_label().graph6_string() for g in F1]
>>> F2 = [g.canonical_label().graph6_string() for g in F2]
↪# needs sage.modules
>>> set(F1) == set(F2)
↪# needs sage.modules
True
```

planar_graphs(*order*, *minimum_degree=None*, *minimum_connectivity=None*, *exact_connectivity=False*, *minimum_edges=None*, *maximum_edges=None*, *maximum_face_size=None*, *only_bipartite=False*, *dual=False*, *immutable=False*)

An iterator over connected planar graphs using the plantri generator.

This uses the plantri generator (see [BM2007]) which is available through the optional package plantri.

Note

The non-3-connected graphs will be returned several times, with all its possible embeddings.

INPUT:

- `order` – positive integer smaller than or equal to 64; this specifies the number of vertices in the generated graphs
- `minimum_degree` – (default: `None`) a value ≥ 1 and ≤ 5, or `None`. This specifies the minimum degree of the generated graphs. If this is `None` and the order is 1, then this is set to 0. If this is `None` and the minimum connectivity is specified, then this is set to the same value as the minimum connectivity. If the minimum connectivity is also equal to `None`, then this is set to 1.

- `minimum_connectivity` – (default: `None`) a value ≥ 1 and ≤ 4, or `None`. This specifies the minimum connectivity of the generated graphs. If this is `None` and the minimum degree is specified, then this is set to the minimum of the minimum degree and 3. If the minimum degree is also equal to `None`, then this is set to 1.
- `exact_connectivity` – (default: `False`) if `True` only graphs with exactly the specified connectivity will be generated. This option requires `minimum_connectivity` to be set to 1 or 2.
- `minimum_edges` – integer (default: `None`); lower bound on the number of edges
- `maximum_edges` – integer (default: `None`); upper bound on the number of edges
- `maximum_face_size` – integer (default: `None`); upper bound on the size of a face and so on the maximum degree of the dual graph
- `only_bipartite` – (default: `False`) if `True` only bipartite graphs will be generated. This option cannot be used for graphs with a minimum degree larger than 3.
- `dual` – (default: `False`) if `True` return instead the planar duals of the generated graphs
- `immutable` – boolean (default: `False`); whether to return immutable or mutable graphs

OUTPUT:

An iterator which will produce all planar graphs with the given number of vertices as Sage graphs with an embedding set. These will be simple graphs (no loops, no multiple edges, no directed edges) unless the option `dual=True` is used.

See also

- *`set_embedding()`*, *`get_embedding()`* – get/set methods for embeddings.

EXAMPLES:

There are 6 planar graphs on 4 vertices:

```
sage: gen = graphs.planar_graphs(4)  # optional - plantri
sage: len(list(gen))                 # optional - plantri
6
```

```
>>> from sage.all import *
>>> gen = graphs.planar_graphs(Integer(4))  # optional - plantri
>>> len(list(gen))                 # optional - plantri
6
```

Three of these planar graphs are bipartite:

```
sage: gen = graphs.planar_graphs(4, only_bipartite=True)  # optional - plantri
sage: len(list(gen))                                      # optional - plantri
3
```

```
>>> from sage.all import *
>>> gen = graphs.planar_graphs(Integer(4), only_bipartite=True)  # optional -␣
↪plantri
>>> len(list(gen))                                          # optional - plantri
3
```

Setting `dual=True` gives the planar dual graphs:

```
sage: gen = graphs.planar_graphs(4, dual=True)  # optional - plantri
sage: [u for u in list(gen)]                    # optional - plantri
[Graph on 4 vertices,
Multi-graph on 3 vertices,
Multi-graph on 2 vertices,
Looped multi-graph on 2 vertices,
Looped multi-graph on 1 vertex,
Looped multi-graph on 1 vertex]
```

```
>>> from sage.all import *
>>> gen = graphs.planar_graphs(Integer(4), dual=True)  # optional - plantri
>>> [u for u in list(gen)]                    # optional - plantri
[Graph on 4 vertices,
Multi-graph on 3 vertices,
Multi-graph on 2 vertices,
Looped multi-graph on 2 vertices,
Looped multi-graph on 1 vertex,
Looped multi-graph on 1 vertex]
```

The cycle of length 4 is the only 2-connected bipartite planar graph on 4 vertices:

```
sage: l = list(graphs.planar_graphs(4, minimum_connectivity=2, only_
↪bipartite=True))  # optional - plantri
sage: l[0].get_embedding()                                                    ␣
↪        # optional - plantri
{1: [2, 3],
 2: [1, 4],
 3: [1, 4],
 4: [2, 3]}
```

```
>>> from sage.all import *
>>> l = list(graphs.planar_graphs(Integer(4), minimum_connectivity=Integer(2),
↪ only_bipartite=True))  # optional - plantri
>>> l[Integer(0)].get_embedding()                                             ␣
↪               # optional - plantri
{1: [2, 3],
 2: [1, 4],
 3: [1, 4],
 4: [2, 3]}
```

There is one planar graph with one vertex. This graph obviously has minimum degree equal to 0:

```
sage: list(graphs.planar_graphs(1))                    # optional - plantri
[Graph on 1 vertex]
sage: list(graphs.planar_graphs(1, minimum_degree=1))  # optional - plantri
[]
```

```
>>> from sage.all import *
>>> list(graphs.planar_graphs(Integer(1)))                    # optional -␣
↪plantri
[Graph on 1 vertex]
```

(continues on next page)

(continued from previous page)

```
>>> list(graphs.planar_graphs(Integer(1), minimum_degree=Integer(1)))  #␣
↪optional - plantri
[]
```

Specifying lower and upper bounds on the number of edges:

```
sage: # optional - plantri
sage: len(list(graphs.planar_graphs(4)))
6
sage: len(list(graphs.planar_graphs(4, minimum_edges=4)))
4
sage: len(list(graphs.planar_graphs(4, maximum_edges=4)))
4
sage: len(list(graphs.planar_graphs(4, minimum_edges=4, maximum_edges=4)))
2
```

```
>>> from sage.all import *
>>> # optional - plantri
>>> len(list(graphs.planar_graphs(Integer(4))))
6
>>> len(list(graphs.planar_graphs(Integer(4), minimum_edges=Integer(4))))
4
>>> len(list(graphs.planar_graphs(Integer(4), maximum_edges=Integer(4))))
4
>>> len(list(graphs.planar_graphs(Integer(4), minimum_edges=Integer(4),␣
↪maximum_edges=Integer(4))))
2
```

Specifying the maximum size of a face:

```
sage: len(list(graphs.planar_graphs(4, maximum_face_size=3)))  # optional -␣
↪plantri
1
sage: len(list(graphs.planar_graphs(4, maximum_face_size=4)))  # optional -␣
↪plantri
3
```

```
>>> from sage.all import *
>>> len(list(graphs.planar_graphs(Integer(4), maximum_face_size=Integer(3))))␣
↪ # optional - plantri
1
>>> len(list(graphs.planar_graphs(Integer(4), maximum_face_size=Integer(4))))␣
↪ # optional - plantri
3
```

plantri_gen(*options=''*, *immutable=False*)

Iterator over planar graphs created using the `plantri` generator.

`plantri` is a (optional) program that generates certain types of graphs that are embedded on the sphere. It outputs exactly one member of each isomorphism class, using an amount of memory almost independent of the number of graphs produced. Isomorphisms are defined with respect to the embeddings, so in some cases outputs may be isomorphic as abstract graphs.

This method allows for passing command directly to `plantry`, similarly to method *nauty_geng()*, provide that the output format is not changed.

INPUT:

- `options` – string (default: `""`); a string passed to `plantri` as if it was run at a system command line. At a minimum, you *must* pass the number of vertices you desire. Sage expects the output of plantri to be in "planar code" format, so do not set an option to change this default or results will be unpredictable.
- `immutable` – boolean (default: `False`); whether to return immutable or mutable graphs

The possible options are:

```
n       : the number of vertices (the only compulsory parameter).
          This number must be in range `3\cdots 64`.
          It can also be given as ``n`` followed by suffix ``d``,
          where the suffix ``d`` means
          "dual", in which case it is converted by adding 4 then
          dividing by 2, i.e., `(28+4)/2 = 16`. In the case of
          triangulations, this calculation yields the number of
          faces, which is the number of vertices in the dual cubic
          graph.

-d      : output the dual instead of the original graph.
          Note that it is applied only at the output stage. All
          other switches refer to the original graph before the dual
          is taken.

-o      : Normally, one member of each isomorphism class is written.
          If this switch is given, one member of each O-P
          isomorphism class is written.

-V      : output only graphs with non-trivial group. If -o is
          given the O-P group is used, the full group otherwise.

-m<int> : lower bound on the minimum degree. The default is -m3.
          In the dual graph, this means a lower bound on the minimum
          face size.

-c<int> : lower bound on the connectivity. The default is -c3.

-x      : when used in combination with -cN, the connectivity must
          be exactly N rather than at least N.

-e      : used to specify bounds on the number of edges.
          There are four possible forms:
              -e<int>        exactly <int> edges
              -e:<int>       at most <int> edges
              -e<int>:       at least <int> edges
              -e<int>:<int>  between <int> and <int> edges

-f<int> : upper bound on the size of a face, and so on the maximum
          degree of the dual.

-b but not -p : select eulerian triangulations, where "eulerian"
```

(continues on next page)

(continued from previous page)

```
                means that every vertex has even degree.
                This parameter can be used in combination with
                parameters -c and -x.

-p but not -b : select general planar simple graphs.
                This parameter can be used in combination with
                parameters -m, -c, -x, -e and -f.

-bp or -pb    : select general planar simple bipartite graphs.
                This parameter can be used in combination with
                parameters -m, -c, -x, -e and -f, except -c4, -m4,
                -m5 and -f3.

-P<int> : select triangulations of a disk. These are embedded simple
          graphs with a distinguished "outer" face. The outer face
          can be of any size (here called the disk size) but the
          other faces must be triangles.  The argument <int> to -P
          is the disk size. If no argument (or 0) is given, all disk
          sizes are permitted.
          This parameter can be used in combination with
          parameters -m, -c, and -x.

-q      : select simple quadrangulations. These are planar simple
          graphs for which every face has length 4.
          This parameter can be used in combination with parameters
          -c and -m.

-A      : select Appolonian networks. These are simple planar
          triangulations that can be formed starting with `K_4` then
          repeatedly dividing a face into three by addition of a new
          vertex. They all have minimum degree and connectivity
          equal to 3.

res/mod : only generate subset res out of subsets 0..mod-1.
          The set of objects is divided into mod disjoint classes
          and only the res-th class is generated.
```

If -b, -q, -p, -P and -A are absent, the graphs found are triangulations only restricted by connectivity and minimum degree. In this case, there is the possibility of connectivity lower than 3.

Other options listed in the `plantri` guide might cause unpredictable behavior, in particular those changing the output format of `plantri` as they will confuse the creation of a Sage graph.

OUTPUT:

An iterator which yields the graphs generated by `plantri` as Sage *Graph*.

See also

- *planar_graphs()* – iterator over connected planar graphs using the `plantri` generator
- *triangulations()* – iterator over connected planar triangulations using the `plantri` generator
- *quadrangulations()* – iterator over connected planar quadrangulations using the `plantri`

generator

EXAMPLES:

The generator can be used to construct graphs for testing, one at a time (usually inside a loop). Or it can be used to create an entire list all at once if there is sufficient memory to contain it:

```
sage: # optional - plantri
sage: gen = graphs.plantri_gen("6")
sage: next(gen)
Graph on 6 vertices
sage: next(gen)
Graph on 6 vertices
sage: next(gen)
Traceback (most recent call last):
...
StopIteration
```

```
>>> from sage.all import *
>>> # optional - plantri
>>> gen = graphs.plantri_gen("6")
>>> next(gen)
Graph on 6 vertices
>>> next(gen)
Graph on 6 vertices
>>> next(gen)
Traceback (most recent call last):
...
StopIteration
```

An overview of the number of quadrangulations on up to 12 vertices. This agrees with OEIS sequence A113201:

```
sage: for i in range(4, 13):                          # optional - plantri
....:     cmd = '-qm2c2 {}'.format(i)
....:     L = len(list(graphs.plantri_gen(cmd)))
....:     print("{:2d}   {:3d}".format(i, L))
 4     1
 5     1
 6     2
 7     3
 8     9
 9    18
10    62
11   198
12   803
```

```
>>> from sage.all import *
>>> for i in range(Integer(4), Integer(13)):                          #␣
↪optional - plantri
...     cmd = '-qm2c2 {}'.format(i)
...     L = len(list(graphs.plantri_gen(cmd)))
```

(continues on next page)

(continued from previous page)

```
...     print("{:2d}    {:3d}".format(i, L))
 4      1
 5      1
 6      2
 7      3
 8      9
 9     18
10     62
11    198
12    803
```

platonic_solids = <module 'sage.graphs.generators.platonic_solids' from '/run/media/hsy/3000gb/sagemath/sage/src/sage/graphs/generators/platonic_solids.py'>

quadrangulations(*order*, *minimum_degree=None*, *minimum_connectivity=None*, *no_nonfacial_quadrangles=False*, *dual=False*, *immutable=False*)

An iterator over planar quadrangulations using the plantri generator.

This uses the plantri generator (see [BM2007]) which is available through the optional package plantri.

INPUT:

- `order` – positive integer smaller than or equal to 64; this specifies the number of vertices in the generated quadrangulations
- `minimum_degree` – (default: `None`) a value ≥ 2 and ≤ 3, or `None`. This specifies the minimum degree of the generated quadrangulations. If this is `None` and the minimum connectivity is specified, then this is set to the same value as the minimum connectivity. If the minimum connectivity is also equal to `None`, then this is set to 2.
- `minimum_connectivity` – (default: `None`) a value ≥ 2 and ≤ 3, or `None`. This specifies the minimum connectivity of the generated quadrangulations. If this is `None` and the option `no_nonfacial_quadrangles` is set to `True`, then this is set to 3. Otherwise if this is `None` and the minimum degree is specified, then this is set to the minimum degree. If the minimum degree is also equal to `None`, then this is set to 3.
- `no_nonfacial_quadrangles` – (default: `False`) if `True` only quadrangulations with no non-facial quadrangles are generated. This option cannot be used if `minimum_connectivity` is set to 2.
- `dual` – (default: `False`) if `True` return instead the planar duals of the generated graphs
- `immutable` – boolean (default: `False`); whether to return immutable or mutable graphs

OUTPUT:

An iterator which will produce all planar quadrangulations with the given number of vertices as Sage graphs with an embedding set. These will be simple graphs (no loops, no multiple edges, no directed edges).

See also

- *`set_embedding()`*, *`get_embedding()`* – get/set methods for embeddings.

EXAMPLES:

The cube is the only 3-connected planar quadrangulation on 8 vertices:

```
sage: # optional - plantri
sage: gen = graphs.quadrangulations(8, minimum_connectivity=3)
sage: g = next(gen)
sage: g.is_isomorphic(graphs.CubeGraph(3))
True
sage: next(gen)
Traceback (most recent call last):
...
StopIteration
```

```
>>> from sage.all import *
>>> # optional - plantri
>>> gen = graphs.quadrangulations(Integer(8), minimum_connectivity=Integer(3))
>>> g = next(gen)
>>> g.is_isomorphic(graphs.CubeGraph(Integer(3)))
True
>>> next(gen)
Traceback (most recent call last):
...
StopIteration
```

An overview of the number of quadrangulations on up to 12 vertices. This agrees with OEIS sequence A113201:

```
sage: for i in range(4,13):                              # optional - plantri
....:     L =  len(list(graphs.quadrangulations(i)))
....:     print("{:2d}   {:3d}".format(i,L))
 4     1
 5     1
 6     2
 7     3
 8     9
 9    18
10    62
11   198
12   803
```

```
>>> from sage.all import *
>>> for i in range(Integer(4),Integer(13)):                              #␣
↪optional - plantri
...     L =  len(list(graphs.quadrangulations(i)))
...     print("{:2d}   {:3d}".format(i,L))
 4     1
 5     1
 6     2
 7     3
 8     9
 9    18
10    62
11   198
12   803
```

There are 2 planar quadrangulation on 12 vertices that do not have a non-facial quadrangle:

```
sage: len([g for g in graphs.quadrangulations(12, no_nonfacial_
↪quadrangles=True)])  # optional - plantri
2
```

```
>>> from sage.all import *
>>> len([g for g in graphs.quadrangulations(Integer(12), no_nonfacial_
↪quadrangles=True)])  # optional - plantri
2
```

Setting `dual=True` gives the planar dual graphs:

```
sage: [len(g) for g in graphs.quadrangulations(12, no_nonfacial_
↪quadrangles=True, dual=True)]  # optional - plantri
[10, 10]
```

```
>>> from sage.all import *
>>> [len(g) for g in graphs.quadrangulations(Integer(12), no_nonfacial_
↪quadrangles=True, dual=True)]  # optional - plantri
[10, 10]
```

random = <module 'sage.graphs.generators.random' from '/run/media/hsy/3000gb/sagemath/sage/src/sage/graphs/generators/random.py'>

static shortened_000_111_extended_binary_Golay_code_graph(*immutable=False*)

Return a distance-regular graph with intersection array $[21, 20, 16, 9, 2, 1; 1, 2, 3, 16, 20, 21]$.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: # long time, needs sage.modules sage.rings.finite_rings
sage: G = graphs.shortened_000_111_extended_binary_Golay_code_graph()
sage: G.is_distance_regular(True)
([21, 20, 16, 9, 2, 1, None], [None, 1, 2, 3, 16, 20, 21])
```

```
>>> from sage.all import *
>>> # long time, needs sage.modules sage.rings.finite_rings
>>> G = graphs.shortened_000_111_extended_binary_Golay_code_graph()
>>> G.is_distance_regular(True)
([21, 20, 16, 9, 2, 1, None], [None, 1, 2, 3, 16, 20, 21])
```

ALGORITHM:

The vertices and edges of this graph have been precomputed and pickled, so truthfully, we just unpickle them and pass them to the Graph constructor. But the algorithm used to compute those vertices and edges in the first place is,

1. Compute the extended binary Golay code.
2. Compute its subcode whose codewords start with 000 or 111.
3. Remove the first 3 entries from all the codewords from the new linear code and compute its coset graph.

This construction is tested in `generators_test.py`, where the result is compared with the result from this method.

REFERENCES:

The description and construction of this graph can be found in [BCN1989], page 365.

static shortened_00_11_binary_Golay_code_graph(*immutable=False*)

Return a distance-regular graph with intersection array $[21, 20, 16, 6, 2, 1; 1, 2, 6, 16, 20, 21]$.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: # long time, needs sage.modules sage.rings.finite_rings
sage: G = graphs.shortened_00_11_binary_Golay_code_graph()      # 9 s
sage: G.is_distance_regular(True)
([21, 20, 16, 6, 2, 1, None], [None, 1, 2, 6, 16, 20, 21])
```

```
>>> from sage.all import *
>>> # long time, needs sage.modules sage.rings.finite_rings
>>> G = graphs.shortened_00_11_binary_Golay_code_graph()      # 9 s
>>> G.is_distance_regular(True)
([21, 20, 16, 6, 2, 1, None], [None, 1, 2, 6, 16, 20, 21])
```

ALGORITHM:

Compute the binary Golay code. Compute the subcode whose codewords start with 00 or 11. Remove the first two entries from all codewords of the newly found linear code and compute its coset graph.

REFERENCES:

Description and construction of this graph can be found in [BCN1989] p. 365.

smallgraphs = <module 'sage.graphs.generators.smallgraphs' from '/run/media/hsy/3000gb/sagemath/sage/src/sage/graphs/generators/smallgraphs.py'>

strongly_regular_db = <module 'sage.graphs.strongly_regular_db' from '/run/media/hsy/3000gb/sagemath/sage/build/sage-distro/src/sage/graphs/strongly_regular_db.cpython-314-x86_64-linux-gnu.so'>

static strongly_regular_graph(*v*, *k*, *l*, *mu=-1*, *existence=False*, *check=True*, *immutable=False*)

Return a (v, k, λ, μ)-strongly regular graph.

This function relies partly on Andries Brouwer's database of strongly regular graphs. See the documentation of *sage.graphs.strongly_regular_db* for more information.

INPUT:

- `v`, `k`, `l`, `mu` – `integers` – note that `mu`, if unspecified, is automatically determined from `v`, `k`, `l`
- `existence` – boolean (default: `False`); instead of building the graph, return:
  - `True` – meaning that a (v, k, λ, μ)-strongly regular graph exists
  - `Unknown` – meaning that Sage does not know if such a strongly regular graph exists (see `sage.misc.unknown`)
  - `False` – meaning that no such strongly regular graph exists
- `check` – boolean (default: `True`); whether to check that output is correct before returning it. As this is expected to be useless, you may want to disable it whenever you want speed.

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph. This parameter is only only when `existence` is `False`.

EXAMPLES:

Petersen's graph from its set of parameters:

```
sage: graphs.strongly_regular_graph(10,3,0,1,existence=True)
↪  # needs sage.libs.pari
True
sage: graphs.strongly_regular_graph(10,3,0,1)
complement(Johnson graph with parameters 5,2): Graph on 10 vertices
```

```
>>> from sage.all import *
>>> graphs.strongly_regular_graph(Integer(10),Integer(3),Integer(0),
↪Integer(1),existence=True)                    # needs sage.libs.pari
True
>>> graphs.strongly_regular_graph(Integer(10),Integer(3),Integer(0),
↪Integer(1))
complement(Johnson graph with parameters 5,2): Graph on 10 vertices
```

Now without specifying μ:

```
sage: graphs.strongly_regular_graph(10,3,0)
complement(Johnson graph with parameters 5,2): Graph on 10 vertices
```

```
>>> from sage.all import *
>>> graphs.strongly_regular_graph(Integer(10),Integer(3),Integer(0))
complement(Johnson graph with parameters 5,2): Graph on 10 vertices
```

An obviously infeasible set of parameters:

```
sage: graphs.strongly_regular_graph(5,5,5,5,existence=True)
False
sage: graphs.strongly_regular_graph(5,5,5,5)
Traceback (most recent call last):
...
ValueError: There exists no (5, 5, 5, 5)-strongly regular graph
```

```
>>> from sage.all import *
>>> graphs.strongly_regular_graph(Integer(5),Integer(5),Integer(5),Integer(5),
↪existence=True)
False
>>> graphs.strongly_regular_graph(Integer(5),Integer(5),Integer(5),Integer(5))
Traceback (most recent call last):
...
ValueError: There exists no (5, 5, 5, 5)-strongly regular graph
```

A set of parameters proved in a paper to be infeasible:

```
sage: graphs.strongly_regular_graph(324,57,0,12,existence=True)
↪  # needs sage.combinat sage.modules
False
sage: graphs.strongly_regular_graph(324,57,0,12)
```

(continues on next page)

(continued from previous page)

```
↪  # needs sage.combinat sage.modules
Traceback (most recent call last):
...
EmptySetError: Andries Brouwer's database reports that no (324, 57, 0,
12)-strongly regular graph exists. Comments: <a
href="srgtabrefs.html#GavrilyukMakhnev05">Gavrilyuk & Makhnev</a> ...
```

```
>>> from sage.all import *
>>> graphs.strongly_regular_graph(Integer(324),Integer(57),Integer(0),
↪Integer(12),existence=True)                  # needs sage.combinat sage.
↪modules
False
>>> graphs.strongly_regular_graph(Integer(324),Integer(57),Integer(0),
↪Integer(12))                                 # needs sage.combinat sage.
↪modules
Traceback (most recent call last):
...
EmptySetError: Andries Brouwer's database reports that no (324, 57, 0,
12)-strongly regular graph exists. Comments: <a
href="srgtabrefs.html#GavrilyukMakhnev05">Gavrilyuk & Makhnev</a> ...
```

A set of parameters unknown to be realizable in Andries Brouwer's database:

```
sage: graphs.strongly_regular_graph(324,95,22,30,existence=True)
↪  # needs sage.combinat
Unknown
sage: graphs.strongly_regular_graph(324,95,22,30)
↪  # needs sage.combinat
Traceback (most recent call last):
...
RuntimeError: Andries Brouwer's database reports that no
(324, 95, 22, 30)-strongly regular graph is known to exist.
Comments:
```

```
>>> from sage.all import *
>>> graphs.strongly_regular_graph(Integer(324),Integer(95),Integer(22),
↪Integer(30),existence=True)                  # needs sage.combinat
Unknown
>>> graphs.strongly_regular_graph(Integer(324),Integer(95),Integer(22),
↪Integer(30))                                 # needs sage.combinat
Traceback (most recent call last):
...
RuntimeError: Andries Brouwer's database reports that no
(324, 95, 22, 30)-strongly regular graph is known to exist.
Comments:
```

A large unknown set of parameters (not in Andries Brouwer's database):

```
sage: graphs.strongly_regular_graph(1394,175,0,25,existence=True)
↪  # needs sage.combinat
Unknown
sage: graphs.strongly_regular_graph(1394,175,0,25)
```

(continues on next page)

(continued from previous page)

```
↪  # needs sage.combinat
Traceback (most recent call last):
...
RuntimeError: Sage cannot figure out if a (1394, 175, 0, 25)-strongly
regular graph exists.
```

```
>>> from sage.all import *
>>> graphs.strongly_regular_graph(Integer(1394),Integer(175),Integer(0),
↪Integer(25),existence=True)               # needs sage.combinat
Unknown
>>> graphs.strongly_regular_graph(Integer(1394),Integer(175),Integer(0),
↪Integer(25))                              # needs sage.combinat
Traceback (most recent call last):
...
RuntimeError: Sage cannot figure out if a (1394, 175, 0, 25)-strongly
regular graph exists.
```

Test the Claw bound (see 3.D of [BL1984]):

```
sage: graphs.strongly_regular_graph(2058,242,91,20,existence=True)
False
```

```
>>> from sage.all import *
>>> graphs.strongly_regular_graph(Integer(2058),Integer(242),Integer(91),
↪Integer(20),existence=True)
False
```

static trees(*n*, *immutable=False*)

Return a generator of the distinct trees on a fixed number of vertices.

INPUT:

- `n` – integer; the size of the trees created
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

OUTPUT:

A generator which creates an exhaustive, duplicate-free listing of the connected free (unlabeled) trees with `n` vertices. A tree is a graph with no cycles.

ALGORITHM:

Uses the algorithm that generates each new tree in constant time described in [WROM1986]. See the documentation for, and implementation of, the *sage.graphs.generators.trees* module.

EXAMPLES:

We create an iterator, then loop over its elements:

```
sage: tree_iterator = graphs.trees(7)
sage: for T in tree_iterator:
....:     print(T.degree_sequence())
[2, 2, 2, 2, 2, 1, 1]
[3, 2, 2, 2, 1, 1, 1]
[3, 2, 2, 2, 1, 1, 1]
```

(continues on next page)

(continued from previous page)

```
[4, 2, 2, 1, 1, 1, 1]
[3, 3, 2, 1, 1, 1, 1]
[3, 3, 2, 1, 1, 1, 1]
[4, 3, 1, 1, 1, 1, 1]
[3, 2, 2, 2, 1, 1, 1]
[4, 2, 2, 1, 1, 1, 1]
[5, 2, 1, 1, 1, 1, 1]
[6, 1, 1, 1, 1, 1, 1]
```

```
>>> from sage.all import *
>>> tree_iterator = graphs.trees(Integer(7))
>>> for T in tree_iterator:
...     print(T.degree_sequence())
[2, 2, 2, 2, 2, 1, 1]
[3, 2, 2, 2, 1, 1, 1]
[3, 2, 2, 2, 1, 1, 1]
[4, 2, 2, 1, 1, 1, 1]
[3, 3, 2, 1, 1, 1, 1]
[3, 3, 2, 1, 1, 1, 1]
[4, 3, 1, 1, 1, 1, 1]
[3, 2, 2, 2, 1, 1, 1]
[4, 2, 2, 1, 1, 1, 1]
[5, 2, 1, 1, 1, 1, 1]
[6, 1, 1, 1, 1, 1, 1]
```

The number of trees on the first few vertex counts. This is sequence A000055 in Sloane's OEIS:

```
sage: [len(list(graphs.trees(i))) for i in range(15)]
[1, 1, 1, 1, 2, 3, 6, 11, 23, 47, 106, 235, 551, 1301, 3159]
```

```
>>> from sage.all import *
>>> [len(list(graphs.trees(i))) for i in range(Integer(15))]
[1, 1, 1, 1, 2, 3, 6, 11, 23, 47, 106, 235, 551, 1301, 3159]
```

triangulations(*order*, *minimum_degree=None*, *minimum_connectivity=None*, *exact_connectivity=False*, *only_eulerian=False*, *dual=False*, *immutable=False*)

An iterator over connected planar triangulations using the plantri generator.

This uses the plantri generator (see [BM2007]) which is available through the optional package plantri.

INPUT:

- `order` – positive integer smaller than or equal to 64; this specifies the number of vertices in the generated triangulations
- `minimum_degree` – (default: `None`) a value ≥ 3 and ≤ 5, or `None`. This specifies the minimum degree of the generated triangulations. If this is `None` and the minimum connectivity is specified, then this is set to the same value as the minimum connectivity. If the minimum connectivity is also equal to `None`, then this is set to 3.
- `minimum_connectivity` – (default: `None`) a value ≥ 3 and ≤ 5, or `None`. This specifies the minimum connectivity of the generated triangulations. If this is `None` and the minimum degree is specified, then this is set to the minimum of the minimum degree and 3. If the minimum degree is also equal to `None`, then this is set to 3.

- `exact_connectivity` – (default: `False`) if `True` only triangulations with exactly the specified connectivity will be generated. This option cannot be used with `minimum_connectivity=3`, or if the minimum connectivity is not explicitly set.
- `only_eulerian` – (default: `False`) if `True` only Eulerian triangulations will be generated. This option cannot be used if the minimum degree is explicitly set to anything else than 4.
- `dual` – (default: `False`) if `True` return instead the planar duals of the generated graphs
- `immutable` – boolean (default: `False`); whether to return immutable or mutable graphs

OUTPUT:

An iterator which will produce all planar triangulations with the given number of vertices as Sage graphs with an embedding set. These will be simple graphs (no loops, no multiple edges, no directed edges).

See also

- *set_embedding()*, *get_embedding()* – get/set methods for embeddings.
- *RandomTriangulation()* – build a random triangulation.

EXAMPLES:

The unique planar embedding of the K_4 is the only planar triangulations on 4 vertices:

```
sage: gen = graphs.triangulations(4)     # optional - plantri
sage: [g.get_embedding() for g in gen]   # optional - plantri
[{1: [2, 3, 4], 2: [1, 4, 3], 3: [1, 2, 4], 4: [1, 3, 2]}]
```

```
>>> from sage.all import *
>>> gen = graphs.triangulations(Integer(4))     # optional - plantri
>>> [g.get_embedding() for g in gen]   # optional - plantri
[{1: [2, 3, 4], 2: [1, 4, 3], 3: [1, 2, 4], 4: [1, 3, 2]}]
```

but, of course, this graph is not Eulerian:

```
sage: gen = graphs.triangulations(4, only_eulerian=True)   # optional - plantri
sage: len(list(gen))                                       # optional - plantri
0
```

```
>>> from sage.all import *
>>> gen = graphs.triangulations(Integer(4), only_eulerian=True)   # optional -␣
↪plantri
>>> len(list(gen))                                          # optional - plantri
0
```

The unique Eulerian triangulation on 6 vertices is isomorphic to the octahedral graph.

```
sage: gen = graphs.triangulations(6, only_eulerian=True)   # optional - plantri
sage: g = next(gen)                                        # optional - plantri
sage: g.is_isomorphic(graphs.OctahedralGraph())            # optional - plantri
True
```

```
>>> from sage.all import *
>>> gen = graphs.triangulations(Integer(6), only_eulerian=True)  # optional -␣
↪plantri
>>> g = next(gen)                                               # optional - plantri
>>> g.is_isomorphic(graphs.OctahedralGraph())                   # optional - plantri
True
```

The minimum degree of a triangulation is 3, so the method can not output a triangle:

```
sage: list(graphs.triangulations(3))                            # optional - plantri
[]
```

```
>>> from sage.all import *
>>> list(graphs.triangulations(Integer(3)))                     # optional -␣
↪plantri
[]
```

An overview of the number of 5-connected triangulations on up to 22 vertices. This agrees with OEIS sequence A081621:

```
sage: for i in range(12, 23):                                                  #␣
↪optional - plantri
....:     L = len(list(graphs.triangulations(i, minimum_connectivity=5)))
....:     print("{}   {:3d}".format(i,L))
12     1
13     0
14     1
15     1
16     3
17     4
18    12
19    23
20    71
21   187
22   627
```

```
>>> from sage.all import *
>>> for i in range(Integer(12), Integer(23)):                                  ␣
↪            # optional - plantri
...     L = len(list(graphs.triangulations(i, minimum_
↪connectivity=Integer(5))))
...     print("{}   {:3d}".format(i,L))
12     1
13     0
14     1
15     1
16     3
17     4
18    12
19    23
20    71
21   187
22   627
```

The minimum connectivity can be at most the minimum degree:

```
sage: gen = next(graphs.triangulations(10, minimum_degree=3,     # optional -␣
↪plantri
....:                                  minimum_connectivity=5))
Traceback (most recent call last):
...
ValueError: Minimum connectivity can be at most the minimum degree.
```

```
>>> from sage.all import *
>>> gen = next(graphs.triangulations(Integer(10), minimum_degree=Integer(3), ␣
↪   # optional - plantri
...                                  minimum_connectivity=Integer(5)))
Traceback (most recent call last):
...
ValueError: Minimum connectivity can be at most the minimum degree.
```

There are 5 triangulations with 9 vertices and minimum degree equal to 4 that are 3-connected, but only one of them is not 4-connected:

```
sage: len([g for g in graphs.triangulations(9, minimum_degree=4,         #␣
↪optional - plantri
....:                                       minimum_connectivity=3)])
5
sage: len([g for g in graphs.triangulations(9, minimum_degree=4,         #␣
↪optional - plantri
....:                                       minimum_connectivity=3,
....:                                       exact_connectivity=True)])
1
```

```
>>> from sage.all import *
>>> len([g for g in graphs.triangulations(Integer(9), minimum_
↪degree=Integer(4),         # optional - plantri
...                                       minimum_connectivity=Integer(3))])
5
>>> len([g for g in graphs.triangulations(Integer(9), minimum_
↪degree=Integer(4),         # optional - plantri
...                                       minimum_connectivity=Integer(3),
...                                       exact_connectivity=True)])
1
```

Setting `dual=True` gives the planar dual graphs:

```
sage: [len(g) for g in graphs.triangulations(9, minimum_degree=4,         #␣
↪optional plantri
....:                                        minimum_connectivity=3,␣
↪dual=True)]
[14, 14, 14, 14, 14]
```

```
>>> from sage.all import *
>>> [len(g) for g in graphs.triangulations(Integer(9), minimum_
↪degree=Integer(4),         # optional plantri
...                                        minimum_connectivity=Integer(3),␣
```

(continues on next page)

(continued from previous page)

```
↪dual=True)]
[14, 14, 14, 14, 14]
```

static vanLintSchrijverGraph(*immutable=False*)

Return the van Lint-Schrijver graph.

The graph is distance-regular with intersection array $[6, 5, 5, 4; 1, 1, 2, 6]$.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: G = graphs.vanLintSchrijverGraph()                                    ␣
↪ # needs sage.modules
sage: G.is_distance_regular(True)                                           ␣
↪ # needs sage.modules
([6, 5, 5, 4, None], [None, 1, 1, 2, 6])
```

```
>>> from sage.all import *
>>> G = graphs.vanLintSchrijverGraph()
↪# needs sage.modules
>>> G.is_distance_regular(True)
↪# needs sage.modules
([6, 5, 5, 4, None], [None, 1, 1, 2, 6])
```

REFERENCES:

For a description of this graph see [BCN1989] p. 373.

world_map = <module 'sage.graphs.generators.world_map' from '/run/media/hsy/3000gb/sagemath/sage/src/sage/graphs/generators/world_map.py'>

sage.graphs.graph_generators.**canaug_traverse_edge**(*g*, *aut_gens*, *property*, *dig=False*, *loops=False*, *sparse=True*)

Main function for exhaustive generation. Recursive traversal of a canonically generated tree of isomorph free graphs satisfying a given property.

INPUT:

- `g` – current position on the tree
- `aut_gens` – list of generators of Aut(g), in list notation
- `property` – check before traversing below g

EXAMPLES:

```
sage: from sage.graphs.graph_generators import canaug_traverse_edge
sage: G = Graph(3)
sage: list(canaug_traverse_edge(G, [], lambda x: True))
[Graph on 3 vertices, ... Graph on 3 vertices]
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_generators import canaug_traverse_edge
>>> G = Graph(Integer(3))
```

(continues on next page)

(continued from previous page)

```
>>> list(canaug_traverse_edge(G, [], lambda x: True))
[Graph on 3 vertices, ... Graph on 3 vertices]
```

The best way to access this function is through the graphs() iterator:

Print graphs on 3 or less vertices.

```
sage: for G in graphs(3):
....:     print(G)
Graph on 3 vertices
Graph on 3 vertices
Graph on 3 vertices
Graph on 3 vertices
```

```
>>> from sage.all import *
>>> for G in graphs(Integer(3)):
...     print(G)
Graph on 3 vertices
Graph on 3 vertices
Graph on 3 vertices
Graph on 3 vertices
```

Print digraphs on 3 or less vertices.

```
sage: for G in digraphs(3):
....:     print(G)
Digraph on 3 vertices
Digraph on 3 vertices
...
Digraph on 3 vertices
Digraph on 3 vertices
```

```
>>> from sage.all import *
>>> for G in digraphs(Integer(3)):
...     print(G)
Digraph on 3 vertices
Digraph on 3 vertices
...
Digraph on 3 vertices
Digraph on 3 vertices
```

sage.graphs.graph_generators.**canaug_traverse_vert**(*g*, *aut_gens*, *max_verts*, *property*, *dig=False*, *loops=False*, *sparse=True*)

Main function for exhaustive generation. Recursive traversal of a canonically generated tree of isomorph free (di)graphs satisfying a given property.

INPUT:

- `g` – current position on the tree
- `aut_gens` – list of generators of Aut(g), in list notation
- `max_verts` – when to retreat
- `property` – check before traversing below g

- `degree_sequence` – specify a degree sequence to try to obtain

EXAMPLES:

```
sage: from sage.graphs.graph_generators import canaug_traverse_vert
sage: list(canaug_traverse_vert(Graph(), [], 3, lambda x: True))
[Graph on 0 vertices, ... Graph on 3 vertices]
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_generators import canaug_traverse_vert
>>> list(canaug_traverse_vert(Graph(), [], Integer(3), lambda x: True))
[Graph on 0 vertices, ... Graph on 3 vertices]
```

The best way to access this function is through the graphs() iterator:

Print graphs on 3 or less vertices.

```
sage: for G in graphs(3, augment='vertices'):
....:     print(G)
Graph on 0 vertices
Graph on 1 vertex
Graph on 2 vertices
Graph on 3 vertices
Graph on 3 vertices
Graph on 3 vertices
Graph on 2 vertices
Graph on 3 vertices
```

```
>>> from sage.all import *
>>> for G in graphs(Integer(3), augment='vertices'):
...     print(G)
Graph on 0 vertices
Graph on 1 vertex
Graph on 2 vertices
Graph on 3 vertices
Graph on 3 vertices
Graph on 3 vertices
Graph on 2 vertices
Graph on 3 vertices
```

Print digraphs on 2 or less vertices.

```
sage: for D in digraphs(2, augment='vertices'):
....:     print(D)
Digraph on 0 vertices
Digraph on 1 vertex
Digraph on 2 vertices
Digraph on 2 vertices
Digraph on 2 vertices
```

```
>>> from sage.all import *
>>> for D in digraphs(Integer(2), augment='vertices'):
...     print(D)
Digraph on 0 vertices
```

(continues on next page)

(continued from previous page)

```
Digraph on 1 vertex
Digraph on 2 vertices
Digraph on 2 vertices
Digraph on 2 vertices
```

sage.graphs.graph_generators.**check_aut**(*aut_gens*, *cut_vert*, *n*)

Helper function for exhaustive generation.

At the start, check_aut is given a set of generators for the automorphism group, aut_gens. We already know we are looking for an element of the auto- morphism group that sends cut_vert to n, and check_aut generates these for the canaug_traverse function.

EXAMPLES:

Note that the last two entries indicate that none of the automorphism group has yet been searched - we are starting at the identity [0, 1, 2, 3] and so far that is all we have seen. We return automorphisms mapping 2 to 3:

```
sage: from sage.graphs.graph_generators import check_aut
sage: list( check_aut( [ [0, 3, 2, 1], [1, 0, 3, 2], [2, 1, 0, 3] ], 2, 3))
[[1, 0, 3, 2], [1, 2, 3, 0]]
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_generators import check_aut
>>> list( check_aut( [ [Integer(0), Integer(3), Integer(2), Integer(1)],␣
↪[Integer(1), Integer(0), Integer(3), Integer(2)], [Integer(2), Integer(1),␣
↪Integer(0), Integer(3)] ], Integer(2), Integer(3)))
[[1, 0, 3, 2], [1, 2, 3, 0]]
```

sage.graphs.graph_generators.**check_aut_edge**(*aut_gens*, *cut_edge*, *i*, *j*, *n*, *dig=False*)

Helper function for exhaustive generation.

At the start, check_aut_edge is given a set of generators for the automorphism group, aut_gens. We already know we are looking for an element of the auto- morphism group that sends cut_edge to {i, j}, and check_aut generates these for the canaug_traverse function.

EXAMPLES:

Note that the last two entries indicate that none of the automorphism group has yet been searched - we are starting at the identity [0, 1, 2, 3] and so far that is all we have seen. We return automorphisms mapping 2 to 3:

```
sage: from sage.graphs.graph_generators import check_aut
sage: list( check_aut( [ [0, 3, 2, 1], [1, 0, 3, 2], [2, 1, 0, 3] ], 2, 3))
[[1, 0, 3, 2], [1, 2, 3, 0]]
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_generators import check_aut
>>> list( check_aut( [ [Integer(0), Integer(3), Integer(2), Integer(1)],␣
↪[Integer(1), Integer(0), Integer(3), Integer(2)], [Integer(2), Integer(1),␣
↪Integer(0), Integer(3)] ], Integer(2), Integer(3)))
[[1, 0, 3, 2], [1, 2, 3, 0]]
```

2.2 Common digraphs

All digraphs in Sage can be built through the `digraphs` object. In order to build a circuit on 15 elements, one can do:

```
sage: g = digraphs.Circuit(15)
```

```
>>> from sage.all import *
>>> g = digraphs.Circuit(Integer(15))
```

To get a circulant graph on 10 vertices in which a vertex i has $i+2$ and $i+3$ as outneighbors:

```
sage: p = digraphs.Circulant(10,[2,3])
```

```
>>> from sage.all import *
>>> p = digraphs.Circulant(Integer(10),[Integer(2),Integer(3)])
```

More interestingly, one can get the list of all digraphs that Sage knows how to build by typing `digraphs.` in Sage and then hitting `Tab`.

ButterflyGraph()	Return a n-dimensional butterfly graph.
Circuit()	Return the circuit on n vertices.
Circulant()	Return a circulant digraph on n vertices from a set of integers.
Complete()	Return a complete digraph on n vertices.
DeBruijn()	Return the De Bruijn digraph with parameters k, n.
GeneralizedDeBruijn()	Return the generalized de Bruijn digraph of order n and degree d.
ImaseItoh()	Return the digraph of Imase and Itoh of order n and degree d.
Kautz()	Return the Kautz digraph of degree d and diameter D.
nauty_directg()	Return an iterator yielding digraphs using nauty's `directg` program.
nauty_posetg()	Return an iterator yielding Hasse diagrams of posets using nauty's `genposetg` program.
Paley()	Return a Paley digraph on q vertices.
Path()	Return a directed path on n vertices.
RandomDirectedAcyclicGraph()	Return a random (weighted) directed acyclic graph of order n.
RandomDirectedGNC()	Return a random growing network with copying (GNC) digraph with n vertices.
RandomDirectedGNM()	Return a random labelled digraph on n nodes and m arcs.
RandomDirectedGNP()	Return a random digraph on n nodes.
RandomDirectedGN()	Return a random growing network (GN) digraph with n vertices.
RandomDirectedGNR()	Return a random growing network with redirection (GNR) digraph.
RandomSemiComplete()	Return a random semi-complete digraph of order n.
RandomTournament()	Return a random tournament on n vertices.
TransitiveTournament()	Return a transitive tournament on n vertices.
tournaments_nauty()	Iterator over all tournaments on n vertices using Nauty.

AUTHORS:

- Robert L. Miller (2006)
- Emily A. Kirkman (2006)
- Michael C. Yurko (2009)
- David Coudert (2012)
- Janmenjaya Panda (2024)

2.2.1 Functions and methods

class sage.graphs.digraph_generators.**DiGraphGenerators**

Bases: object

A class consisting of constructors for several common digraphs, including orderly generation of isomorphism class representatives.

A list of all graphs and graph structures in this database is available via tab completion. Type `digraphs.` and then hit `Tab` to see which digraphs are available.

The docstrings include educational information about each named digraph with the hopes that this class can be used as a reference.

The constructors currently in this class include:

```
Random Directed Graphs:
    - RandomDirectedAcyclicGraph
    - RandomDirectedGN
    - RandomDirectedGNC
    - RandomDirectedGNP
    - RandomDirectedGNM
    - RandomDirectedGNR
    - RandomTournament
    - RandomSemiComplete

Families of Graphs:
    - Complete
    - DeBruijn
    - GeneralizedDeBruijn
    - Kautz
    - Path
    - ImaseItoh
    - RandomTournament
    - TransitiveTournament
    - tournaments_nauty
```

ORDERLY GENERATION: digraphs(vertices, property=lambda x: True, augment='edges', size=None)

Accesses the generator of isomorphism class representatives [McK1998]. Iterates over distinct, exhaustive representatives.

INPUT:

- `vertices` – natural number or `None` to infinitely generate bigger and bigger digraphs
- `property` – any property to be tested on digraphs before generation
- `augment` – choices:
  - `'vertices'` – augments by adding a vertex, and edges incident to that vertex. In this case, all digraphs on *up to* n=vertices are generated. If for any digraph G satisfying the property, every subgraph, obtained from G by deleting one vertex and only edges incident to that vertex, satisfies the property, then this will generate all digraphs with that property. If this does not hold, then all the digraphs generated will satisfy the property, but there will be some missing.
  - `'edges'` – augments a fixed number of vertices by adding one edge. In this case, all digraphs on *exactly* n=vertices are generated. If for any graph G satisfying the property, every subgraph, obtained from G by deleting one edge but not the vertices incident to that edge, satisfies the property, then this will generate

all digraphs with that property. If this does not hold, then all the digraphs generated will satisfy the property, but there will be some missing.

- `implementation` – which underlying implementation to use (see DiGraph?)
- `sparse` – boolean (default: `True`); whether to use a sparse or dense data structure. See the documentation of *Graph*.
- `copy` – boolean (default: `True`); whether to make copies of the digraphs before returning them. If set to `False` the method returns the digraph it is working on. The second alternative is faster, but modifying any of the digraph instances returned by the method may break the function's behaviour, as it is using these digraphs to compute the next ones: only use `copy = False` when you stick to *reading* the digraphs returned.

  This parameter is ignored when `immutable` is set to `True`, in which case returned graphs are always copies.
- `immutable` – boolean (default: `False`); whether to return immutable or mutable digraphs. When set to `True`, this parameter implies `copy=True`.

EXAMPLES:

Print digraphs on 2 or less vertices:

```
sage: for D in digraphs(2, augment='vertices'):
....:     print(D)
Digraph on 0 vertices
Digraph on 1 vertex
Digraph on 2 vertices
Digraph on 2 vertices
Digraph on 2 vertices
```

```
>>> from sage.all import *
>>> for D in digraphs(Integer(2), augment='vertices'):
...     print(D)
Digraph on 0 vertices
Digraph on 1 vertex
Digraph on 2 vertices
Digraph on 2 vertices
Digraph on 2 vertices
```

Print digraphs on 3 vertices:

```
sage: for D in digraphs(3):
....:     print(D)
Digraph on 3 vertices
Digraph on 3 vertices
...
Digraph on 3 vertices
Digraph on 3 vertices
```

```
>>> from sage.all import *
>>> for D in digraphs(Integer(3)):
...     print(D)
Digraph on 3 vertices
Digraph on 3 vertices
...
Digraph on 3 vertices
Digraph on 3 vertices
```

Generate all digraphs with 4 vertices and 3 edges:

```
sage: L = digraphs(4, size=3)
sage: len(list(L))
13
```

```
>>> from sage.all import *
>>> L = digraphs(Integer(4), size=Integer(3))
>>> len(list(L))
13
```

Generate all digraphs with 4 vertices and up to 3 edges:

```
sage: L = list(digraphs(4, lambda G: G.size() <= 3))
sage: len(L)
20
sage: graphs_list.show_graphs(L)         # long time                              #␣
↪needs sage.plot
```

```
>>> from sage.all import *
>>> L = list(digraphs(Integer(4), lambda G: G.size() <= Integer(3)))
>>> len(L)
20
>>> graphs_list.show_graphs(L)         # long time                               #␣
↪needs sage.plot
```

Generate all digraphs with degree at most 2, up to 5 vertices:

```
sage: property = lambda G: (max([G.degree(v) for v in G] + [0]) <= 2)
sage: L = list(digraphs(5, property, augment='vertices'))
sage: len(L)
75
```

```
>>> from sage.all import *
>>> property = lambda G: (max([G.degree(v) for v in G] + [Integer(0)]) <=␣
↪Integer(2))
>>> L = list(digraphs(Integer(5), property, augment='vertices'))
>>> len(L)
75
```

Generate digraphs on the fly (see http://oeis.org/classic/A000273):

```
sage: for i in range(5):
....:     print(len(list(digraphs(i))))
1
1
3
16
218
```

```
>>> from sage.all import *
>>> for i in range(Integer(5)):
...     print(len(list(digraphs(i))))
```

(continues on next page)

(continued from previous page)

```
1
1
3
16
218
```

ButterflyGraph(*n*, *vertices='strings'*, *immutable=False*)

Return a n-dimensional butterfly graph.

The vertices consist of pairs (v, i), where v is an n-dimensional tuple (vector) with binary entries (or a string representation of such) and i is an integer in $[0..n]$. A directed edge goes from (v, i) to $(w, i + 1)$ if v and w are identical except for possibly when $v[i] \neq w[i]$.

A butterfly graph has $(2^n)(n + 1)$ vertices and $n2^{n+1}$ edges.

INPUT:

- `n` – nonnegative integer; the dimension of the butterfly graph
- `vertices` – string (default: `'strings'`); specifies whether the vertices are zero-one strings (default) or tuples over GF(2) (`vertices='vectors'`)
- `immutable` – boolean (default: `False`); whether to return an immutable or mutable digraph

EXAMPLES:

```
sage: digraphs.ButterflyGraph(2).edges(sort=True, labels=False)
[(('00', 0), ('00', 1)),
 (('00', 0), ('10', 1)),
 (('00', 1), ('00', 2)),
 (('00', 1), ('01', 2)),
 (('01', 0), ('01', 1)),
 (('01', 0), ('11', 1)),
 (('01', 1), ('00', 2)),
 (('01', 1), ('01', 2)),
 (('10', 0), ('00', 1)),
 (('10', 0), ('10', 1)),
 (('10', 1), ('10', 2)),
 (('10', 1), ('11', 2)),
 (('11', 0), ('01', 1)),
 (('11', 0), ('11', 1)),
 (('11', 1), ('10', 2)),
 (('11', 1), ('11', 2))]
sage: digraphs.ButterflyGraph(2, vertices='vectors').edges(sort=True,       #␣
↪needs sage.modules sage.rings.finite_rings
....:                                                      labels=False)
[(((0, 0), 0), ((0, 0), 1)),
 (((0, 0), 0), ((1, 0), 1)),
 (((0, 0), 1), ((0, 0), 2)),
 (((0, 0), 1), ((0, 1), 2)),
 (((0, 1), 0), ((0, 1), 1)),
 (((0, 1), 0), ((1, 1), 1)),
 (((0, 1), 1), ((0, 0), 2)),
 (((0, 1), 1), ((0, 1), 2)),
 (((1, 0), 0), ((0, 0), 1)),
```

(continues on next page)

(continued from previous page)

```
 (((1, 0), 0), ((1, 0), 1)),
 (((1, 0), 1), ((1, 0), 2)),
 (((1, 0), 1), ((1, 1), 2)),
 (((1, 1), 0), ((0, 1), 1)),
 (((1, 1), 0), ((1, 1), 1)),
 (((1, 1), 1), ((1, 0), 2)),
 (((1, 1), 1), ((1, 1), 2))]
sage: pos = digraphs.ButterflyGraph(2).get_pos()
sage: pos['11', 0]
(0, 0)
```

```
>>> from sage.all import *
>>> digraphs.ButterflyGraph(Integer(2)).edges(sort=True, labels=False)
[(('00', 0), ('00', 1)),
 (('00', 0), ('10', 1)),
 (('00', 1), ('00', 2)),
 (('00', 1), ('01', 2)),
 (('01', 0), ('01', 1)),
 (('01', 0), ('11', 1)),
 (('01', 1), ('00', 2)),
 (('01', 1), ('01', 2)),
 (('10', 0), ('00', 1)),
 (('10', 0), ('10', 1)),
 (('10', 1), ('10', 2)),
 (('10', 1), ('11', 2)),
 (('11', 0), ('01', 1)),
 (('11', 0), ('11', 1)),
 (('11', 1), ('10', 2)),
 (('11', 1), ('11', 2))]
>>> digraphs.ButterflyGraph(Integer(2), vertices='vectors').edges(sort=True, ␣
↪     # needs sage.modules sage.rings.finite_rings
...                                                          labels=False)
[(((0, 0), 0), ((0, 0), 1)),
 (((0, 0), 0), ((1, 0), 1)),
 (((0, 0), 1), ((0, 0), 2)),
 (((0, 0), 1), ((0, 1), 2)),
 (((0, 1), 0), ((0, 1), 1)),
 (((0, 1), 0), ((1, 1), 1)),
 (((0, 1), 1), ((0, 0), 2)),
 (((0, 1), 1), ((0, 1), 2)),
 (((1, 0), 0), ((0, 0), 1)),
 (((1, 0), 0), ((1, 0), 1)),
 (((1, 0), 1), ((1, 0), 2)),
 (((1, 0), 1), ((1, 1), 2)),
 (((1, 1), 0), ((0, 1), 1)),
 (((1, 1), 0), ((1, 1), 1)),
 (((1, 1), 1), ((1, 0), 2)),
 (((1, 1), 1), ((1, 1), 2))]
>>> pos = digraphs.ButterflyGraph(Integer(2)).get_pos()
>>> pos['11', Integer(0)]
(0, 0)
```

Circuit(*n*, *immutable=False*)

Return the circuit on n vertices.

The circuit is an oriented *CycleGraph()*.

INPUT:

- `n` – integer; number of vertices
- `immutable` – boolean (default: `False`); whether to return an immutable or mutable digraph

EXAMPLES:

A circuit is the smallest strongly connected digraph:

```
sage: circuit = digraphs.Circuit(15)
sage: len(circuit.strongly_connected_components()) == 1
True
```

```
>>> from sage.all import *
>>> circuit = digraphs.Circuit(Integer(15))
>>> len(circuit.strongly_connected_components()) == Integer(1)
True
```

Circulant(*n*, *integers*, *immutable=False*)

Return a circulant digraph on n vertices from a set of integers.

A circulant digraph of order n has an arc from vertex i to vertex $i + j \pmod n$, for each j in `integers`.

INPUT:

- `n` – integer; number of vertices
- `integers` – iterable container (list, set, etc.) of integers such that there is an edge from i to j if and only if $(j - i) \pmod n$ is an integer
- `immutable` – boolean (default: `False`); whether to return an immutable or mutable digraph

EXAMPLES:

Construct and show the circulant graph [3, 5, 7], a digraph on 13 vertices:

```
sage: g = digraphs.Circulant(13, [3, 5, 7])
sage: g.show()                              # long time
↪  # needs sage.plot
```

```
>>> from sage.all import *
>>> g = digraphs.Circulant(Integer(13), [Integer(3), Integer(5), Integer(7)])
>>> g.show()                              # long time
↪# needs sage.plot
```

The Koh-Tindell digraph [LM2024] is the circulant digraph of order 7 with parameters $[1, 5]$. This 2-diregular digraph is vertex-transitive but not arc-transitive. The associated bipartite digraph of the Koh-Tindell digraph is a Pfaffian orientation of the Heawood graph. Construct and show the Koh-Tindell digraph:

```
sage: kohTindellDigraph = digraphs.Circulant(7, [1, 5])
sage: kohTindellDigraph.show()              # long time
↪  # needs sage.plot
```

```
>>> from sage.all import *
>>> kohTindellDigraph = digraphs.Circulant(Integer(7), [Integer(1),␣
↪Integer(5)])
>>> kohTindellDigraph.show()           # long time
↪# needs sage.plot
```

Complete(*n*, *loops=False*, *immutable=False*)

Return the complete digraph on n vertices.

INPUT:

- `n` – integer; number of vertices
- `loops` – boolean (default: `False`); whether to add loops or not, i.e., edges from u to itself
- `immutable` – boolean (default: `False`); whether to return an immutable or mutable digraph

See also

- *RandomSemiComplete()*
- *RandomTournament()*

EXAMPLES:

```
sage: n = 10
sage: G = digraphs.Complete(n); G
Complete digraph: Digraph on 10 vertices
sage: G.size() == n*(n-1)
True
sage: G = digraphs.Complete(n, loops=True); G
Complete digraph with loops: Looped digraph on 10 vertices
sage: G.size() == n*n
True
sage: digraphs.Complete(-1)
Traceback (most recent call last):
...
ValueError: the number of vertices cannot be strictly negative
```

```
>>> from sage.all import *
>>> n = Integer(10)
>>> G = digraphs.Complete(n); G
Complete digraph: Digraph on 10 vertices
>>> G.size() == n*(n-Integer(1))
True
>>> G = digraphs.Complete(n, loops=True); G
Complete digraph with loops: Looped digraph on 10 vertices
>>> G.size() == n*n
True
>>> digraphs.Complete(-Integer(1))
Traceback (most recent call last):
...
ValueError: the number of vertices cannot be strictly negative
```

DeBruijn(*k*, *n*, *vertices='strings'*, *immutable=False*)

Return the De Bruijn digraph with parameters k, n.

The De Bruijn digraph with parameters k, n is built upon a set of vertices equal to the set of words of length n from a dictionary of k letters.

In this digraph, there is an arc w_1w_2 if w_2 can be obtained from w_1 by removing the leftmost letter and adding a new letter at its right end. For more information, see the Wikipedia article De_Bruijn_graph.

INPUT:

- **k – two possibilities for this parameter :**
  - An integer equal to the cardinality of the alphabet to use, that is, the degree of the digraph to be produced.
  - An iterable object to be used as the set of letters. The degree of the resulting digraph is the cardinality of the set of letters.
- `n` – integer; length of words in the De Bruijn digraph when `vertices == 'strings'`, and also the diameter of the digraph
- `vertices` – string (default: `'strings'`); whether the vertices are words over an alphabet (default) or integers (`vertices='string'`)
- `immutable` – boolean (default: `False`); whether to return an immutable or mutable digraph

EXAMPLES:

de Bruijn digraph of degree 2 and diameter 2:

```
sage: db = digraphs.DeBruijn(2, 2); db
De Bruijn digraph (k=2, n=2): Looped digraph on 4 vertices
sage: db.order(), db.size()
(4, 8)
sage: db.diameter()
2
```

```
>>> from sage.all import *
>>> db = digraphs.DeBruijn(Integer(2), Integer(2)); db
De Bruijn digraph (k=2, n=2): Looped digraph on 4 vertices
>>> db.order(), db.size()
(4, 8)
>>> db.diameter()
2
```

Building a de Bruijn digraph on a different alphabet:

```
sage: g = digraphs.DeBruijn(['a', 'b'], 2)
sage: g.vertices(sort=True)
['aa', 'ab', 'ba', 'bb']
sage: g.is_isomorphic(db)
True
sage: g = digraphs.DeBruijn(['AA', 'BB'], 2)
sage: g.vertices(sort=True)
['AA,AA', 'AA,BB', 'BB,AA', 'BB,BB']
sage: g.is_isomorphic(db)
True
```

```
>>> from sage.all import *
>>> g = digraphs.DeBruijn(['a', 'b'], Integer(2))
>>> g.vertices(sort=True)
['aa', 'ab', 'ba', 'bb']
>>> g.is_isomorphic(db)
True
>>> g = digraphs.DeBruijn(['AA', 'BB'], Integer(2))
>>> g.vertices(sort=True)
['AA,AA', 'AA,BB', 'BB,AA', 'BB,BB']
>>> g.is_isomorphic(db)
True
```

GeneralizedDeBruijn(*n*, *d*, *immutable=False*, *name=None*)

Return the generalized de Bruijn digraph of order n and degree d.

The generalized de Bruijn digraph was defined in [RPK1980] [RPK1983]. It has vertex set $V = \{0, 1, ..., n-1\}$ and there is an arc from vertex $u \in V$ to all vertices $v \in V$ such that $v \equiv (u * d + a) \mod n$ with $0 \leq a < d$.

When $n = d^D$, the generalized de Bruijn digraph is isomorphic to the de Bruijn digraph of degree d and diameter D.

INPUT:

- `n` – integer; number of vertices of the digraph (must be at least one)
- `d` – integer; degree of the digraph (must be at least one)
- `immutable` – boolean (default: `False`); whether to return an immutable or mutable digraph
- `name` – string (default: `None`); when set, the specified name is used instead of the default one

See also

- *sage.graphs.generic_graph.GenericGraph.is_circulant()* – checks whether a (di)graph is circulant, and/or returns all possible sets of parameters.

EXAMPLES:

```
sage: GB = digraphs.GeneralizedDeBruijn(8, 2)
sage: GB.is_isomorphic(digraphs.DeBruijn(2, 3), certificate=True)           #␣
↪needs sage.combinat
(True, {0: '000', 1: '001', 2: '010', 3: '011',
        4: '100', 5: '101', 6: '110', 7: '111'})
```

```
>>> from sage.all import *
>>> GB = digraphs.GeneralizedDeBruijn(Integer(8), Integer(2))
>>> GB.is_isomorphic(digraphs.DeBruijn(Integer(2), Integer(3)),␣
↪certificate=True)           # needs sage.combinat
(True, {0: '000', 1: '001', 2: '010', 3: '011',
        4: '100', 5: '101', 6: '110', 7: '111'})
```

ImaseItoh(*n*, *d*, *immutable=False*, *name=None*)

Return the Imase-Itoh digraph of order n and degree d.

The Imase-Itoh digraph was defined in [II1983]. It has vertex set $V = \{0, 1, ..., n-1\}$ and there is an arc from vertex $u \in V$ to all vertices $v \in V$ such that $v \equiv (-u * d - a - 1) \mod n$ with $0 \leq a < d$.

When $n = d^D$, the Imase-Itoh digraph is isomorphic to the de Bruijn digraph of degree d and diameter D. When $n = d^{D-1}(d+1)$, the Imase-Itoh digraph is isomorphic to the Kautz digraph [Kau1968] of degree d and diameter D.

INPUT:

- `n` – integer; number of vertices of the digraph (must be greater than or equal to two)
- `d` – integer; degree of the digraph (must be greater than or equal to one)
- `immutable` – boolean (default: `False`); whether to return an immutable or mutable digraph
- `name` – string (default: `None`); when set, the specified name is used instead of the default one

EXAMPLES:

```
sage: II = digraphs.ImaseItoh(8, 2)
sage: II.is_isomorphic(digraphs.DeBruijn(2, 3), certificate=True)           #␣
↪needs sage.combinat
(True, {0: '010', 1: '011', 2: '000', 3: '001',
        4: '110', 5: '111', 6: '100', 7: '101'})

sage: II = digraphs.ImaseItoh(12, 2)
sage: b,D = II.is_isomorphic(digraphs.Kautz(2, 3), certificate=True)        #␣
↪needs sage.combinat
sage: b                                                                     #␣
↪needs sage.combinat
True
sage: D   # random isomorphism                                              #␣
↪needs sage.combinat
{0: '202', 1: '201', 2: '210', 3: '212', 4: '121',
 5: '120', 6: '102', 7: '101', 8: '010', 9: '012',
 10: '021', 11: '020'}
```

```
>>> from sage.all import *
>>> II = digraphs.ImaseItoh(Integer(8), Integer(2))
>>> II.is_isomorphic(digraphs.DeBruijn(Integer(2), Integer(3)),␣
↪certificate=True)           # needs sage.combinat
(True, {0: '010', 1: '011', 2: '000', 3: '001',
        4: '110', 5: '111', 6: '100', 7: '101'})

>>> II = digraphs.ImaseItoh(Integer(12), Integer(2))
>>> b,D = II.is_isomorphic(digraphs.Kautz(Integer(2), Integer(3)),␣
↪certificate=True)        # needs sage.combinat
>>> b                                                                       #␣
↪needs sage.combinat
True
>>> D   # random isomorphism                                               #␣
↪needs sage.combinat
{0: '202', 1: '201', 2: '210', 3: '212', 4: '121',
 5: '120', 6: '102', 7: '101', 8: '010', 9: '012',
 10: '021', 11: '020'}
```

Kautz(*k*, *D*, *vertices='strings'*, *immutable=False*)

Return the Kautz digraph of degree d and diameter D.

The Kautz digraph has been defined in [Kau1968]. The Kautz digraph of degree d and diameter D has $d^{D-1}(d+1)$ vertices. This digraph is built from a set of vertices equal to the set of words of length D over an alphabet of $d+1$ letters such that consecutive letters are different. There is an arc from vertex u to vertex v if v can be obtained from u by removing the leftmost letter and adding a new letter, distinct from the rightmost letter of u, at the right end.

The Kautz digraph of degree d and diameter D is isomorphic to the Imase-Itoh digraph [II1983] of degree d and order $d^{D-1}(d+1)$.

See the Wikipedia article Kautz_graph for more information.

INPUT:

- `k` – two possibilities for this parameter. In either case the degree must be at least one:
  - An integer equal to the degree of the digraph to be produced, that is, the cardinality of the alphabet to be used minus one.
  - An iterable object to be used as the set of letters. The degree of the resulting digraph is the cardinality of the set of letters minus one.
- `D` – integer; diameter of the digraph, and length of a vertex label when `vertices == 'strings'` (must be at least one)
- `vertices` – string (default: `'strings'`); whether the vertices are words over an alphabet (default) or integers (`vertices='strings'`)
- `immutable` – boolean (default: `False`); whether to return an immutable or mutable digraph

EXAMPLES:

```
sage: K = digraphs.Kautz(2, 3)
sage: b, D = K.is_isomorphic(digraphs.ImaseItoh(12, 2), certificate=True)
sage: b
True
sage: D  # random isomorphism
{'010': 8, '012': 9, '020': 11, '021': 10, '101': 7,  '102': 6,
 '120': 5, '121': 4, '201': 1, '202': 0, '210': 2, '212': 3}

sage: K = digraphs.Kautz([1,'a','B'], 2)                                    #␣
↪needs sage.combinat
sage: K.edges(sort=True)                                                    #␣
↪needs sage.combinat
[('1B', 'B1', '1'), ('1B', 'Ba', 'a'), ('1a', 'a1', '1'),
 ('1a', 'aB', 'B'), ('B1', '1B', 'B'), ('B1', '1a', 'a'),
 ('Ba', 'a1', '1'), ('Ba', 'aB', 'B'), ('a1', '1B', 'B'),
 ('a1', '1a', 'a'), ('aB', 'B1', '1'), ('aB', 'Ba', 'a')]

sage: K = digraphs.Kautz([1,'aA','BB'], 2)                                  #␣
↪needs sage.combinat
sage: K.edges(sort=True)                                                    #␣
↪needs sage.combinat
[('1,BB', 'BB,1', '1'), ('1,BB', 'BB,aA', 'aA'),
 ('1,aA', 'aA,1', '1'), ('1,aA', 'aA,BB', 'BB'),
 ('BB,1', '1,BB', 'BB'), ('BB,1', '1,aA', 'aA'),
 ('BB,aA', 'aA,1', '1'), ('BB,aA', 'aA,BB', 'BB'),
```

(continues on next page)

(continued from previous page)

```
 ('aA,1', '1,BB', 'BB'), ('aA,1', '1,aA', 'aA'),
 ('aA,BB', 'BB,1', '1'), ('aA,BB', 'BB,aA', 'aA')]
```

```
>>> from sage.all import *
>>> K = digraphs.Kautz(Integer(2), Integer(3))
>>> b, D = K.is_isomorphic(digraphs.ImaseItoh(Integer(12), Integer(2)),␣
↪certificate=True)
>>> b
True
>>> D  # random isomorphism
{'010': 8, '012': 9, '020': 11, '021': 10, '101': 7,  '102': 6,
 '120': 5, '121': 4, '201': 1, '202': 0, '210': 2, '212': 3}

>>> K = digraphs.Kautz([Integer(1),'a','B'], Integer(2))                    ␣
↪              # needs sage.combinat
>>> K.edges(sort=True)                                                      #␣
↪needs sage.combinat
[('1B', 'B1', '1'), ('1B', 'Ba', 'a'), ('1a', 'a1', '1'),
 ('1a', 'aB', 'B'), ('B1', '1B', 'B'), ('B1', '1a', 'a'),
 ('Ba', 'a1', '1'), ('Ba', 'aB', 'B'), ('a1', '1B', 'B'),
 ('a1', '1a', 'a'), ('aB', 'B1', '1'), ('aB', 'Ba', 'a')]

>>> K = digraphs.Kautz([Integer(1),'aA','BB'], Integer(2))                  ␣
↪              # needs sage.combinat
>>> K.edges(sort=True)                                                      #␣
↪needs sage.combinat
[('1,BB', 'BB,1', '1'), ('1,BB', 'BB,aA', 'aA'),
 ('1,aA', 'aA,1', '1'), ('1,aA', 'aA,BB', 'BB'),
 ('BB,1', '1,BB', 'BB'), ('BB,1', '1,aA', 'aA'),
 ('BB,aA', 'aA,1', '1'), ('BB,aA', 'aA,BB', 'BB'),
 ('aA,1', '1,BB', 'BB'), ('aA,1', '1,aA', 'aA'),
 ('aA,BB', 'BB,1', '1'), ('aA,BB', 'BB,aA', 'aA')]
```

Paley(*q*, *immutable=False*)

Return a Paley digraph on q vertices.

Parameter q must be the power of a prime number and congruent to 3 mod 4.

INPUT:

- `q` – integer; the number of vertices of the digraph
- `immutable` – boolean (default: `False`); whether to return an immutable or mutable digraph

See also

- Wikipedia article Paley_graph
- *PaleyGraph()*

EXAMPLES:

A Paley digraph has $n * (n-1)/2$ edges, its underlying graph is a clique, and so it is a tournament:

```
sage: g = digraphs.Paley(7); g                                                  #␣
↪needs sage.rings.finite_rings
Paley digraph with parameter 7: Digraph on 7 vertices
sage: g.size() == g.order() * (g.order() - 1) / 2                               #␣
↪needs sage.rings.finite_rings
True
sage: g.to_undirected().is_clique()                                             #␣
↪needs sage.rings.finite_rings
True
```

```
>>> from sage.all import *
>>> g = digraphs.Paley(Integer(7)); g                                           ␣
↪     # needs sage.rings.finite_rings
Paley digraph with parameter 7: Digraph on 7 vertices
>>> g.size() == g.order() * (g.order() - Integer(1)) / Integer(2)               ␣
↪              # needs sage.rings.finite_rings
True
>>> g.to_undirected().is_clique()                                              #␣
↪needs sage.rings.finite_rings
True
```

A Paley digraph is always self-complementary:

```
sage: g.complement().is_isomorphic(g)                                           #␣
↪needs sage.rings.finite_rings
True
```

```
>>> from sage.all import *
>>> g.complement().is_isomorphic(g)                                            #␣
↪needs sage.rings.finite_rings
True
```

Path(*n*, *immutable=False*)

Return a directed path on n vertices.

INPUT:

- `n` – integer; number of vertices in the path
- `immutable` – boolean (default: `False`); whether to return an immutable or mutable digraph

EXAMPLES:

```
sage: g = digraphs.Path(5)
sage: g.vertices(sort=True)
[0, 1, 2, 3, 4]
sage: g.size()
4
sage: g.automorphism_group().cardinality()                                      #␣
↪needs sage.groups
1
```

```
>>> from sage.all import *
>>> g = digraphs.Path(Integer(5))
```

(continues on next page)

(continued from previous page)

```
>>> g.vertices(sort=True)
[0, 1, 2, 3, 4]
>>> g.size()
4
>>> g.automorphism_group().cardinality()                                    #␣
↪needs sage.groups
1
```

RandomDirectedAcyclicGraph(*n*, *p*, *weight_max=None*, *immutable=False*)

Return a random (weighted) directed acyclic graph of order n.

The method starts with the sink vertex and adds vertices one at a time. A vertex is connected only to previously defined vertices, and the probability of each possible connection is given by the probability p. The weight of an edge is a random integer between `1` and `weight_max`.

INPUT:

- `n` – number of nodes of the graph
- `p` – probability of an edge
- `weight_max` – (default: `None`) by default, the returned DAG is unweighted. When `weight_max` is set to a positive integer, edges are assigned a random integer weight between `1` and `weight_max`.
- `immutable` – boolean (default: `False`); whether to return an immutable or mutable digraph

EXAMPLES:

```
sage: D = digraphs.RandomDirectedAcyclicGraph(5, .5); D
RandomDAG(5, 0.500000000000000): Digraph on 5 vertices
sage: D.is_directed_acyclic()
True
sage: D = digraphs.RandomDirectedAcyclicGraph(5, .5, weight_max=3); D
RandomWeightedDAG(5, 0.500000000000000, 3): Digraph on 5 vertices
sage: D.is_directed_acyclic()
True
```

```
>>> from sage.all import *
>>> D = digraphs.RandomDirectedAcyclicGraph(Integer(5), RealNumber('.5')); D
RandomDAG(5, 0.500000000000000): Digraph on 5 vertices
>>> D.is_directed_acyclic()
True
>>> D = digraphs.RandomDirectedAcyclicGraph(Integer(5), RealNumber('.5'),␣
↪weight_max=Integer(3)); D
RandomWeightedDAG(5, 0.500000000000000, 3): Digraph on 5 vertices
>>> D.is_directed_acyclic()
True
```

RandomDirectedGN(*n*, *kernel=None*, *seed=None*, *immutable=False*)

Return a random growing network (GN) digraph with n vertices.

The digraph is constructed by adding vertices with a link to one previously added vertex. The vertex to link to is chosen with a preferential attachment model, i.e. probability is proportional to degree. The default attachment kernel is a linear function of degree. The digraph is always a tree, so in particular it is a directed acyclic graph. See [KR2001b] for more details.

INPUT:

- `n` – integer; number of vertices
- `kernel` – the attachment kernel (default: identity function)
- `seed` – a `random.Random` seed or a Python `int` for the random number generator (default: `None`)
- `immutable` – boolean (default: `False`); whether to return an immutable or mutable digraph

EXAMPLES:

```
sage: # needs networkx
sage: D = digraphs.RandomDirectedGN(25)
sage: D.n_vertices()
25
sage: D.n_edges()
24
sage: D.is_connected()
True
sage: D.parent() is DiGraph
True
sage: D.show()                          # long time
```

```
>>> from sage.all import *
>>> # needs networkx
>>> D = digraphs.RandomDirectedGN(Integer(25))
>>> D.n_vertices()
25
>>> D.n_edges()
24
>>> D.is_connected()
True
>>> D.parent() is DiGraph
True
>>> D.show()                          # long time
```

RandomDirectedGNC(*n*, *seed=None*, *immutable=False*)

Return a random growing network with copying (GNC) digraph with n vertices.

The digraph is constructed by adding vertices with a link to one previously added vertex. The vertex to link to is chosen with a preferential attachment model, i.e. probability is proportional to degree. The new vertex is also linked to all of the previously added vertex's successors. See [KR2005] for more details.

INPUT:

- `n` – integer; number of vertices
- `seed` – a `random.Random` seed or a Python `int` for the random number generator (default: `None`)
- `immutable` – boolean (default: `False`); whether to return an immutable or mutable digraph

EXAMPLES:

```
sage: # needs networkx
sage: D = digraphs.RandomDirectedGNC(25)
sage: D.is_directed_acyclic()
True
sage: D.topological_sort()
```

(continues on next page)

(continued from previous page)

```
[24, 23, ..., 1, 0]
sage: D.show()                          # long time
```

```
>>> from sage.all import *
>>> # needs networkx
>>> D = digraphs.RandomDirectedGNC(Integer(25))
>>> D.is_directed_acyclic()
True
>>> D.topological_sort()
[24, 23, ..., 1, 0]
>>> D.show()                          # long time
```

RandomDirectedGNM(*n*, *m*, *loops=False*, *immutable=False*)

Return a random labelled digraph on n nodes and m arcs.

INPUT:

- `n` – integer; number of vertices
- `m` – integer; number of edges
- `loops` – boolean (default: `False`); whether to allow loops
- `immutable` – boolean (default: `False`); whether to return an immutable or mutable digraph

PLOTTING: When plotting, this graph will use the default spring-layout algorithm, unless a position dictionary is specified.

EXAMPLES:

```
sage: D = digraphs.RandomDirectedGNM(10, 5)
sage: D.n_vertices()
10
sage: D.n_edges()
5
```

```
>>> from sage.all import *
>>> D = digraphs.RandomDirectedGNM(Integer(10), Integer(5))
>>> D.n_vertices()
10
>>> D.n_edges()
5
```

With loops:

```
sage: D = digraphs.RandomDirectedGNM(10, 100, loops = True)
sage: D.n_vertices()
10
sage: D.loops()
[(0, 0, None), (1, 1, None), (2, 2, None), (3, 3, None),
 (4, 4, None), (5, 5, None), (6, 6, None), (7, 7, None),
 (8, 8, None), (9, 9, None)]
```

```
>>> from sage.all import *
>>> D = digraphs.RandomDirectedGNM(Integer(10), Integer(100), loops = True)
```

(continues on next page)

(continued from previous page)

```
>>> D.n_vertices()
10
>>> D.loops()
[(0, 0, None), (1, 1, None), (2, 2, None), (3, 3, None),
 (4, 4, None), (5, 5, None), (6, 6, None), (7, 7, None),
 (8, 8, None), (9, 9, None)]
```

RandomDirectedGNP(*n*, *p*, *loops=False*, *seed=None*, *immutable=False*)

Return a random digraph on n nodes.

Each edge is inserted independently with probability p. See [ER1959] and [Gil1959] for more details.

INPUT:

- `n` – integer; number of nodes of the digraph
- `p` – float; probability of an edge
- `loops` – boolean (default: `False`); whether the random digraph may have loops
- `seed` – integer (default: `None`); seed for random number generator
- `immutable` – boolean (default: `False`); whether to return an immutable or mutable digraph

PLOTTING: When plotting, this graph will use the default spring-layout algorithm, unless a position dictionary is specified.

EXAMPLES:

```
sage: D = digraphs.RandomDirectedGNP(10, .2)
sage: D.n_vertices()
10
sage: D.parent() is DiGraph
True
```

```
>>> from sage.all import *
>>> D = digraphs.RandomDirectedGNP(Integer(10), RealNumber('.2'))
>>> D.n_vertices()
10
>>> D.parent() is DiGraph
True
```

RandomDirectedGNR(*n*, *p*, *seed=None*, *immutable=False*)

Return a random growing network with redirection (GNR) digraph with n vertices and redirection probability p.

The digraph is constructed by adding vertices with a link to one previously added vertex. The vertex to link to is chosen uniformly. With probability p, the arc is instead redirected to the successor vertex. The digraph is always a tree. See [KR2001b] for more details.

INPUT:

- `n` – integer; number of vertices
- `p` – redirection probability
- `seed` – a `random.Random` seed or a Python `int` for the random number generator (default: `None`)
- `immutable` – boolean (default: `False`); whether to return an immutable or mutable digraph

EXAMPLES:

```
sage: # needs networkx
sage: D = digraphs.RandomDirectedGNR(25, .2)
sage: D.is_directed_acyclic()
True
sage: D.to_undirected().is_tree()
True
sage: D.show()                          # long time                          #␣
↪needs sage.plot
```

```
>>> from sage.all import *
>>> # needs networkx
>>> D = digraphs.RandomDirectedGNR(Integer(25), RealNumber('.2'))
>>> D.is_directed_acyclic()
True
>>> D.to_undirected().is_tree()
True
>>> D.show()                          # long time                          #␣
↪needs sage.plot
```

RandomSemiComplete(*n*, *immutable=False*)

Return a random semi-complete digraph on n vertices.

A directed graph $G = (V, E)$ is *semi-complete* if for any pair of vertices u and v, there is *at least* one arc between them.

To generate randomly a semi-complete digraph, we have to ensure, for any pair of distinct vertices u and v, that with probability $1/3$ we have only arc uv, with probability $1/3$ we have only arc vu, and with probability $1/3$ we have both arc uv and arc vu. We do so by selecting a random integer $coin$ in $[1, 3]$. When $coin == 1$ we select only arc uv, when $coin == 3$ we select only arc vu, and when $coin == 2$ we select both arcs. In other words, we select arc uv when $coin \leq 2$ and arc vu when $coin \geq 2$.

INPUT:

- `n` – integer; the number of nodes
- `immutable` – boolean (default: `False`); whether to return an immutable or mutable digraph

See also

- *Complete()*
- *RandomTournament()*

EXAMPLES:

```
sage: SC = digraphs.RandomSemiComplete(10); SC
Random Semi-Complete digraph: Digraph on 10 vertices
sage: SC.size() >= binomial(10, 2)                                          #␣
↪needs sage.symbolic
True
sage: digraphs.RandomSemiComplete(-1)
Traceback (most recent call last):
```

(continues on next page)

(continued from previous page)

```
...
ValueError: the number of vertices cannot be strictly negative
```

```
>>> from sage.all import *
>>> SC = digraphs.RandomSemiComplete(Integer(10)); SC
Random Semi-Complete digraph: Digraph on 10 vertices
>>> SC.size() >= binomial(Integer(10), Integer(2))
↪              # needs sage.symbolic
True
>>> digraphs.RandomSemiComplete(-Integer(1))
Traceback (most recent call last):
...
ValueError: the number of vertices cannot be strictly negative
```

RandomTournament(*n*, *immutable=False*)

Return a random tournament on n vertices.

For every pair of vertices, the tournament has an edge from i to j with probability $1/2$, otherwise it has an edge from j to i.

INPUT:

- `n` – integer; number of vertices
- `immutable` – boolean (default: `False`); whether to return an immutable or mutable digraph

EXAMPLES:

```
sage: T = digraphs.RandomTournament(10); T
Random Tournament: Digraph on 10 vertices
sage: T.size() == binomial(10, 2)
↪needs sage.symbolic
True
sage: T.is_tournament()
True
sage: digraphs.RandomTournament(-1)
Traceback (most recent call last):
...
ValueError: the number of vertices cannot be strictly negative
```

```
>>> from sage.all import *
>>> T = digraphs.RandomTournament(Integer(10)); T
Random Tournament: Digraph on 10 vertices
>>> T.size() == binomial(Integer(10), Integer(2))
↪              # needs sage.symbolic
True
>>> T.is_tournament()
True
>>> digraphs.RandomTournament(-Integer(1))
Traceback (most recent call last):
...
ValueError: the number of vertices cannot be strictly negative
```

See also

- Wikipedia article Tournament_(graph_theory)
- *is_tournament()*
- *TransitiveTournament()*
- *Complete()*
- *RandomSemiComplete()*

StronglyRegular(*n*, *immutable=False*)

Return a Strongly Regular digraph with n vertices.

The adjacency matrix of the graph is constructed from a skew Hadamard matrix of order $n+1$. These graphs were first constructed in [Duv1988].

INPUT:

- `n` – integer; the number of vertices of the digraph
- `immutable` – boolean (default: `False`); whether to return an immutable or mutable digraph

See also

- sage.combinat.matrices.hadamard_matrix.skew_hadamard_matrix()
- *Paley()*

EXAMPLES:

A Strongly Regular digraph satisfies the condition $AJ = JA = kJ$ where A is the adjacency matrix:

```
sage: g = digraphs.StronglyRegular(7); g
Strongly regular digraph: Digraph on 7 vertices
sage: A = g.adjacency_matrix()*ones_matrix(7)
sage: B = ones_matrix(7)*g.adjacency_matrix()
sage: A == B == A[0, 0]*ones_matrix(7)
True
```

```
>>> from sage.all import *
>>> g = digraphs.StronglyRegular(Integer(7)); g
Strongly regular digraph: Digraph on 7 vertices
>>> A = g.adjacency_matrix()*ones_matrix(Integer(7))
>>> B = ones_matrix(Integer(7))*g.adjacency_matrix()
>>> A == B == A[Integer(0), Integer(0)]*ones_matrix(Integer(7))
True
```

TransitiveTournament(*n*, *immutable=False*)

Return a transitive tournament on n vertices.

In this tournament there is an edge from i to j if $i < j$.

See the Wikipedia article Tournament_(graph_theory) for more information.

INPUT:

- `n` – integer; number of vertices in the tournament
- `immutable` – boolean (default: `False`); whether to return an immutable or mutable digraph

EXAMPLES:

```
sage: g = digraphs.TransitiveTournament(5)
sage: g.vertices(sort=True)
[0, 1, 2, 3, 4]
sage: g.size()
10
sage: g.automorphism_group().cardinality()                                  #␣
↪needs sage.groups
1
```

```
>>> from sage.all import *
>>> g = digraphs.TransitiveTournament(Integer(5))
>>> g.vertices(sort=True)
[0, 1, 2, 3, 4]
>>> g.size()
10
>>> g.automorphism_group().cardinality()                                    #␣
↪needs sage.groups
1
```

See also

- Wikipedia article Tournament_(graph_theory)
- *is_tournament()*
- *is_transitive()*
- *RandomTournament()*

nauty_directg(*graphs*, *options=''*, *debug=False*, *immutable=False*)

Return an iterator yielding digraphs using nauty's `directg` program.

Description from `directg --help`: Read undirected graphs and orient their edges in all possible ways. Edges can be oriented in either or both directions (3 possibilities). Isomorphic directed graphs derived from the same input are suppressed. If the input graphs are non-isomorphic then the output graphs are also.

INPUT:

- `graphs` – a *Graph* or an iterable containing *Graph*. The graph6 string of these graphs is used as an input for `directg`.
- `options` – string passed to `directg` as if it was run at a system command line. Available options from `directg --help`:

```
-e<int> | -e<int>:<int>  specify a value or range of the total number of␣
↪arcs
-o       orient each edge in only one direction, never both
-a       only make acyclic orientations (implies -o)
-f<int>  Use only the subgroup that fixes the first <int> vertices setwise
-V       only output graphs with nontrivial groups (including exchange of
```

(continues on next page)

(continued from previous page)

```
        isolated vertices).  The -f option is respected.
-s<int>/<int>  Make only a fraction of the orientations: The first␣
↪integer is
        the part number (first is 0) and the second is the number of
        parts. Splitting is done per input graph independently.
```

- `debug` – boolean (default: `False`); if `True` `directg` standard error and standard output are displayed
- `immutable` – boolean (default: `False`); whether to return immutable or mutable digraphs

EXAMPLES:

```
sage: gen = graphs.nauty_geng("-c 3")
sage: dgs = list(digraphs.nauty_directg(gen))
sage: len(dgs)
13
sage: dgs[0]
Digraph on 3 vertices
sage: dgs[0]._bit_vector()
'001001000'
sage: len(list(digraphs.nauty_directg(graphs.PetersenGraph(), options='-o')))
324
```

```
>>> from sage.all import *
>>> gen = graphs.nauty_geng("-c 3")
>>> dgs = list(digraphs.nauty_directg(gen))
>>> len(dgs)
13
>>> dgs[Integer(0)]
Digraph on 3 vertices
>>> dgs[Integer(0)]._bit_vector()
'001001000'
>>> len(list(digraphs.nauty_directg(graphs.PetersenGraph(), options='-o')))
324
```

Generate non-isomorphic acyclic orientations:

```
sage: K = graphs.CompleteGraph(4)
sage: all(d.is_directed_acyclic() for d in digraphs.nauty_directg(K, options=
↪'-a'))
True
sage: sum(1 for _ in digraphs.nauty_directg(K, options='-a'))
1
sage: S = graphs.StarGraph(4)
sage: all(d.is_directed_acyclic() for d in digraphs.nauty_directg(S, options=
↪'-a'))
True
sage: sum(1 for _ in digraphs.nauty_directg(S, options='-a'))
5
```

```
>>> from sage.all import *
>>> K = graphs.CompleteGraph(Integer(4))
>>> all(d.is_directed_acyclic() for d in digraphs.nauty_directg(K, options='-a
```

(continues on next page)

(continued from previous page)

```
↪'))
True
>>> sum(Integer(1) for _ in digraphs.nauty_directg(K, options='-a'))
1
>>> S = graphs.StarGraph(Integer(4))
>>> all(d.is_directed_acyclic() for d in digraphs.nauty_directg(S, options='-a
↪'))
True
>>> sum(Integer(1) for _ in digraphs.nauty_directg(S, options='-a'))
5
```

See also

- *orientations()*
- *strong_orientation()*
- *strong_orientations_iterator()*
- *random_orientation()*

nauty_posetg(*options=''*, *debug=False*, *immutable=False*)

Return a generator which creates all posets using `nauty`.

Here a poset is seen through its Hasse diagram, which is an acyclic and transitively reduced digraph.

INPUT:

- `options` – string (default: `""`); a string passed to `genposetg` as if it was run at a system command line. At a minimum, you *must* pass the number of vertices you desire and a choice between `o` and `t` for the output order.
- `debug` – boolean (default: `False`); if `True` the first line of `genposetg`'s output to standard error is captured and the first call to the generator's `next()` function will return this line as a string. A line leading with ">A" indicates a successful initiation of the program with some information on the arguments, while a line beginning with ">E" indicates an error with the input.
- `immutable` – boolean (default: `False`); whether to return immutable or mutable posets

The possible options, obtained as output of `genposetg --help`:

```
n: the number of vertices, between 0 and 16
o: digraph6 output in arbitrary order
t: digraph6 output in topological order
```

EXAMPLES:

```
sage: gen = digraphs.nauty_posetg("5 o")
sage: len(list(gen))
63
```

```
>>> from sage.all import *
>>> gen = digraphs.nauty_posetg("5 o")
>>> len(list(gen))
63
```

This coincides with OEIS sequence A000112.

tournaments_nauty(*n*, *min_out_degree=None*, *max_out_degree=None*, *strongly_connected=False*, *debug=False*, *options=''*, *immutable=False*)

Iterator over all tournaments on n vertices using Nauty.

INPUT:

- `n` – integer; number of vertices
- `min_out_degree`, `max_out_degree` – integers; if set to `None` (default), then the min/max out-degree is not constrained
- `debug` – boolean (default: `False`); if `True` the first line of gentourng's output to standard error is captured and the first call to the generator's `next()` function will return this line as a string. A line leading with ">A" indicates a successful initiation of the program with some information on the arguments, while a line beginning with ">E" indicates an error with the input.
- `options` – string; anything else that should be forwarded as input to Nauty's gentourng. See its documentation for more information : https://pallini.di.uniroma1.it.
- `immutable` – boolean (default: `False`); whether to return immutable or mutable digraphs

EXAMPLES:

```
sage: for g in digraphs.tournaments_nauty(4):
....:     print(g.edges(sort=True, labels = False))
[(1, 0), (2, 0), (2, 1), (3, 0), (3, 1), (3, 2)]
[(1, 0), (1, 3), (2, 0), (2, 1), (3, 0), (3, 2)]
[(0, 2), (1, 0), (2, 1), (3, 0), (3, 1), (3, 2)]
[(0, 2), (0, 3), (1, 0), (2, 1), (3, 1), (3, 2)]
sage: tournaments = digraphs.tournaments_nauty
sage: [len(list(tournaments(x))) for x in range(1,8)]
[1, 1, 2, 4, 12, 56, 456]
sage: [len(list(tournaments(x, strongly_connected = True))) for x in range(1,
↪9)]
[1, 0, 1, 1, 6, 35, 353, 6008]
```

```
>>> from sage.all import *
>>> for g in digraphs.tournaments_nauty(Integer(4)):
...     print(g.edges(sort=True, labels = False))
[(1, 0), (2, 0), (2, 1), (3, 0), (3, 1), (3, 2)]
[(1, 0), (1, 3), (2, 0), (2, 1), (3, 0), (3, 2)]
[(0, 2), (1, 0), (2, 1), (3, 0), (3, 1), (3, 2)]
[(0, 2), (0, 3), (1, 0), (2, 1), (3, 1), (3, 2)]
>>> tournaments = digraphs.tournaments_nauty
>>> [len(list(tournaments(x))) for x in range(Integer(1),Integer(8))]
[1, 1, 2, 4, 12, 56, 456]
>>> [len(list(tournaments(x, strongly_connected = True))) for x in␣
↪range(Integer(1),Integer(9))]
[1, 0, 1, 1, 6, 35, 353, 6008]
```

2.3 Common graphs and digraphs generators (Cython)

AUTHORS:

- David Coudert (2012)

sage.graphs.graph_generators_pyx.**RandomGNP**(*n*, *p*, *directed=False*, *loops=False*, *seed=None*, *immutable=False*)

Return a random graph or a digraph on n nodes.

Each edge is inserted independently with probability p.

INPUT:

- `n` – number of nodes of the digraph
- `p` – probability of an edge
- `directed` – boolean (default: `False`); whether the random graph is directed or undirected (default)
- `loops` – boolean (default: `False`); whether the random digraph may have loops or not. This value is used only when `directed == True`
- `seed` – a `random.Random` seed or a Python `int` for the random number generator (default: `None`)
- `immutable` – boolean (default: `False`); whether to return an immutable or mutable (di)graph.

REFERENCES:

- [ER1959]
- [Gil1959]

EXAMPLES:

```
sage: from sage.graphs.graph_generators_pyx import RandomGNP
sage: D = RandomGNP(10, .2, directed=True, seed=0)
sage: D.n_vertices()
10
sage: D.edges(sort=True, labels=False)
[(0, 3), (0, 6), (1, 7), (1, 9), (4, 6), (4, 7), (5, 4), (5, 6),
 (5, 8), (5, 9), (6, 3), (7, 2), (7, 9), (8, 5), (9, 1), (9, 5)]
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_generators_pyx import RandomGNP
>>> D = RandomGNP(Integer(10), RealNumber('.2'), directed=True, seed=Integer(0))
>>> D.n_vertices()
10
>>> D.edges(sort=True, labels=False)
[(0, 3), (0, 6), (1, 7), (1, 9), (4, 6), (4, 7), (5, 4), (5, 6),
 (5, 8), (5, 9), (6, 3), (7, 2), (7, 9), (8, 5), (9, 1), (9, 5)]
```

2.4 Graph database

This module implements classes (*GraphDatabase*, *GraphQuery*, *GenericGraphQuery*) for interfacing with the `sqlite` database `graphs.db`.

The *GraphDatabase* class interfaces with the `sqlite` database `graphs.db`. It is an immutable database that inherits from `SQLDatabase` (see `sage.databases.sql_db`).

The database contains all unlabeled graphs with 7 or fewer nodes. This class will also interface with the optional database package containing all unlabeled graphs with 8 or fewer nodes. The database(s) consists of five tables, and has the structure given by the function *graph_db_info()* (For a full description including column data types, create a *GraphDatabase* instance and call the method get_skeleton()).

AUTHORS:

- Emily A. Kirkman (2008-09-20): first version of interactive queries, cleaned up code and generalized many elements to sage.databases.sql_db.py
- Emily A. Kirkman (2007-07-23): inherits GenericSQLDatabase, also added classes: GraphQuery and GenericGraphQuery
- Emily A. Kirkman (2007-05-11): initial sqlite version
- Emily A. Kirkman (2007-02-13): initial version (non-sqlite)

REFERENCES:

- Data provided by Jason Grout (Brigham Young University). [Online] Available: http://artsci.drake.edu/grout/graphs/

class sage.graphs.graph_database.**GenericGraphQuery**(*query_string*, *database=None*, *param_tuple=None*)

Bases: SQLQuery

A query for a *GraphDatabase*.

INPUT:

- query_string – string representing the SQL query
- database – (default: None) the *GraphDatabase* instance to query (if None then a new instance is created)
- param_tuple – tuple of strings (default: None); what to replace question marks in query_string with (optional, but a good idea)

Note

This query class is generally intended for developers and more advanced users. It allows you to execute any query, and so may be considered unsafe.

EXAMPLES:

See *GraphDatabase* class docstrings or enter:

```
sage: G = GraphDatabase()
sage: G.get_skeleton()
{...
```

```
>>> from sage.all import *
>>> G = GraphDatabase()
>>> G.get_skeleton()
{...
```

to see the underlying structure of the database. Also see sage.databases.sql_db.SQLQuery in sage.databases.sql_db for more info and a tutorial.

A piece of advice about '?' and param_tuple: it is generally considered safer to query with a '?' in place of each value parameter, and using a second argument (a tuple of strings) in a call to the `sqlite` database. Successful use of the `param_tuple` argument is exemplified:

```
sage: G = GraphDatabase()
sage: q = 'select graph_id,graph6,num_vertices,num_edges from graph_data where␣
↪graph_id<=(?) and num_vertices=(?)'
sage: param = (22,5)
sage: Q = SQLQuery(G, q, param)
sage: Q.show()
graph_id              graph6                num_vertices          num_edges
------------------------------------------------------------------------------
18                    D??                   5                     0
19                    D?C                   5                     1
20                    D?K                   5                     2
21                    D@O                   5                     2
22                    D?[                   5                     3
```

```
>>> from sage.all import *
>>> G = GraphDatabase()
>>> q = 'select graph_id,graph6,num_vertices,num_edges from graph_data where␣
↪graph_id<=(?) and num_vertices=(?)'
>>> param = (Integer(22),Integer(5))
>>> Q = SQLQuery(G, q, param)
>>> Q.show()
graph_id              graph6                num_vertices          num_edges
------------------------------------------------------------------------------
18                    D??                   5                     0
19                    D?C                   5                     1
20                    D?K                   5                     2
21                    D@O                   5                     2
22                    D?[                   5                     3
```

class sage.graphs.graph_database.**GraphDatabase**

Bases: SQLDatabase

Graph Database.

This class interfaces with the `sqlite` database `graphs.db`. It is an immutable database that inherits from SQLDatabase (see `sage.databases.sql_db`). The display functions and get_graphs_list create their own queries, but it is also possible to query the database by constructing either a SQLQuery.

The database contains all unlabeled graphs with 7 or fewer nodes. This class will also interface with the optional database package containing all unlabeled graphs with 8 or fewer nodes. The database consists of five tables. For a full table and column structure, call *graph_db_info()*.

The tables are associated by the unique primary key `graph_id` (int).

To query this database, we create a *GraphQuery*. This can be done directly with the *query()* method or by initializing one of:

- *GenericGraphQuery* – allows direct entry of a query string and tuple of parameters. This is the route for more advanced users that are familiar with SQL
- *GraphQuery* – a wrapper of SQLQuery, a general database/query wrapper of SQLite for new users

REFERENCES:

- Data provided by Jason Grout (Brigham Young University). [Online] Available: http://artsci.drake.edu/grout/graphs/

EXAMPLES:

```
sage: G = GraphDatabase()
sage: G.get_skeleton()
{'aut_grp': {'aut_grp_size': {'index': True,
   'primary_key': False,
   'sql': 'INTEGER',
   'unique': False},
  'edge_transitive': {'index': True,
   'primary_key': False,
   'sql': 'BOOLEAN',
   'unique': False},
  'graph_id': {'index': False,
   'primary_key': False,
   'sql': 'INTEGER',
   'unique': False},
  'num_fixed_points': {'index': True,
   'primary_key': False,
   'sql': 'INTEGER',
   'unique': False},
  'num_orbits': {'index': True,
   'primary_key': False,
   'sql': 'INTEGER',
   'unique': False},
  'vertex_transitive': {'index': True,
   'primary_key': False,
   'sql': 'BOOLEAN',
   'unique': False}},
 'degrees': {'average_degree': {'index': True,
   'primary_key': False,
   'sql': 'REAL',
   'unique': False},
  'degree_sequence': {'index': False,
   'primary_key': False,
   'sql': 'INTEGER',
   'unique': False},
  'degrees_sd': {'index': True,
   'primary_key': False,
   'sql': 'REAL',
   'unique': False},
  'graph_id': {'index': False,
   'primary_key': False,
   'sql': 'INTEGER',
   'unique': False},
  'max_degree': {'index': True,
   'primary_key': False,
   'sql': 'INTEGER',
   'unique': False},
  'min_degree': {'index': True,
   'primary_key': False,
   'sql': 'INTEGER',
```

(continues on next page)

(continued from previous page)

```
   'unique': False},
  'regular': {'index': True,
   'primary_key': False,
   'sql': 'BOOLEAN',
   'unique': False}},
 'graph_data': {'complement_graph6': {'index': True,
   'primary_key': False,
   'sql': 'TEXT',
   'unique': False},
  'eulerian': {'index': True,
   'primary_key': False,
   'sql': 'BOOLEAN',
   'unique': False},
  'graph6': {'index': True,
   'primary_key': False,
   'sql': 'TEXT',
   'unique': False},
  'graph_id': {'index': True,
   'primary_key': False,
   'sql': 'INTEGER',
   'unique': True},
  'lovasz_number': {'index': True,
   'primary_key': False,
   'sql': 'REAL',
   'unique': False},
  'num_cycles': {'index': True,
   'primary_key': False,
   'sql': 'INTEGER',
   'unique': False},
  'num_edges': {'index': True,
   'primary_key': False,
   'sql': 'INTEGER',
   'unique': False},
  'num_hamiltonian_cycles': {'index': True,
   'primary_key': False,
   'sql': 'INTEGER',
   'unique': False},
  'num_vertices': {'index': True,
   'primary_key': False,
   'sql': 'INTEGER',
   'unique': False},
  'perfect': {'index': True,
   'primary_key': False,
   'sql': 'BOOLEAN',
   'unique': False},
  'planar': {'index': True,
   'primary_key': False,
   'sql': 'BOOLEAN',
   'unique': False}},
 'misc': {'clique_number': {'index': True,
   'primary_key': False,
   'sql': 'INTEGER',
```

(continues on next page)

(continued from previous page)

```
   'unique': False},
  'diameter': {'index': True,
   'primary_key': False,
   'sql': 'INTEGER',
   'unique': False},
  'edge_connectivity': {'index': True,
   'primary_key': False,
   'sql': 'BOOLEAN',
   'unique': False},
  'girth': {'index': True,
   'primary_key': False,
   'sql': 'INTEGER',
   'unique': False},
  'graph_id': {'index': False,
   'primary_key': False,
   'sql': 'INTEGER',
   'unique': False},
  'independence_number': {'index': True,
   'primary_key': False,
   'sql': 'INTEGER',
   'unique': False},
  'induced_subgraphs': {'index': True,
   'primary_key': False,
   'sql': 'TEXT',
   'unique': False},
  'min_vertex_cover_size': {'index': True,
   'primary_key': False,
   'sql': 'INTEGER',
   'unique': False},
  'num_components': {'index': True,
   'primary_key': False,
   'sql': 'INTEGER',
   'unique': False},
  'num_cut_vertices': {'index': True,
   'primary_key': False,
   'sql': 'INTEGER',
   'unique': False},
  'num_spanning_trees': {'index': True,
   'primary_key': False,
   'sql': 'INTEGER',
   'unique': False},
  'radius': {'index': True,
   'primary_key': False,
   'sql': 'INTEGER',
   'unique': False},
  'vertex_connectivity': {'index': True,
   'primary_key': False,
   'sql': 'BOOLEAN',
   'unique': False}},
 'spectrum': {'eigenvalues_sd': {'index': True,
   'primary_key': False,
   'sql': 'REAL',
```

(continues on next page)

(continued from previous page)

```
   'unique': False},
  'energy': {'index': True,
   'primary_key': False,
   'sql': 'REAL',
   'unique': False},
  'graph_id': {'index': False,
   'primary_key': False,
   'sql': 'INTEGER',
   'unique': False},
  'max_eigenvalue': {'index': True,
   'primary_key': False,
   'sql': 'REAL',
   'unique': False},
  'min_eigenvalue': {'index': True,
   'primary_key': False,
   'sql': 'REAL',
   'unique': False},
  'spectrum': {'index': False,
   'primary_key': False,
   'sql': 'TEXT',
   'unique': False}}}
```

```
>>> from sage.all import *
>>> G = GraphDatabase()
>>> G.get_skeleton()
{'aut_grp': {'aut_grp_size': {'index': True,
   'primary_key': False,
   'sql': 'INTEGER',
   'unique': False},
  'edge_transitive': {'index': True,
   'primary_key': False,
   'sql': 'BOOLEAN',
   'unique': False},
  'graph_id': {'index': False,
   'primary_key': False,
   'sql': 'INTEGER',
   'unique': False},
  'num_fixed_points': {'index': True,
   'primary_key': False,
   'sql': 'INTEGER',
   'unique': False},
  'num_orbits': {'index': True,
   'primary_key': False,
   'sql': 'INTEGER',
   'unique': False},
  'vertex_transitive': {'index': True,
   'primary_key': False,
   'sql': 'BOOLEAN',
   'unique': False}},
 'degrees': {'average_degree': {'index': True,
   'primary_key': False,
   'sql': 'REAL',
```

(continues on next page)

(continued from previous page)

```
   'unique': False},
  'degree_sequence': {'index': False,
   'primary_key': False,
   'sql': 'INTEGER',
   'unique': False},
  'degrees_sd': {'index': True,
   'primary_key': False,
   'sql': 'REAL',
   'unique': False},
  'graph_id': {'index': False,
   'primary_key': False,
   'sql': 'INTEGER',
   'unique': False},
  'max_degree': {'index': True,
   'primary_key': False,
   'sql': 'INTEGER',
   'unique': False},
  'min_degree': {'index': True,
   'primary_key': False,
   'sql': 'INTEGER',
   'unique': False},
  'regular': {'index': True,
   'primary_key': False,
   'sql': 'BOOLEAN',
   'unique': False}},
 'graph_data': {'complement_graph6': {'index': True,
   'primary_key': False,
   'sql': 'TEXT',
   'unique': False},
  'eulerian': {'index': True,
   'primary_key': False,
   'sql': 'BOOLEAN',
   'unique': False},
  'graph6': {'index': True,
   'primary_key': False,
   'sql': 'TEXT',
   'unique': False},
  'graph_id': {'index': True,
   'primary_key': False,
   'sql': 'INTEGER',
   'unique': True},
  'lovasz_number': {'index': True,
   'primary_key': False,
   'sql': 'REAL',
   'unique': False},
  'num_cycles': {'index': True,
   'primary_key': False,
   'sql': 'INTEGER',
   'unique': False},
  'num_edges': {'index': True,
   'primary_key': False,
   'sql': 'INTEGER',
```

(continues on next page)

(continued from previous page)

```
   'unique': False},
  'num_hamiltonian_cycles': {'index': True,
   'primary_key': False,
   'sql': 'INTEGER',
   'unique': False},
  'num_vertices': {'index': True,
   'primary_key': False,
   'sql': 'INTEGER',
   'unique': False},
  'perfect': {'index': True,
   'primary_key': False,
   'sql': 'BOOLEAN',
   'unique': False},
  'planar': {'index': True,
   'primary_key': False,
   'sql': 'BOOLEAN',
   'unique': False}},
 'misc': {'clique_number': {'index': True,
   'primary_key': False,
   'sql': 'INTEGER',
   'unique': False},
  'diameter': {'index': True,
   'primary_key': False,
   'sql': 'INTEGER',
   'unique': False},
  'edge_connectivity': {'index': True,
   'primary_key': False,
   'sql': 'BOOLEAN',
   'unique': False},
  'girth': {'index': True,
   'primary_key': False,
   'sql': 'INTEGER',
   'unique': False},
  'graph_id': {'index': False,
   'primary_key': False,
   'sql': 'INTEGER',
   'unique': False},
  'independence_number': {'index': True,
   'primary_key': False,
   'sql': 'INTEGER',
   'unique': False},
  'induced_subgraphs': {'index': True,
   'primary_key': False,
   'sql': 'TEXT',
   'unique': False},
  'min_vertex_cover_size': {'index': True,
   'primary_key': False,
   'sql': 'INTEGER',
   'unique': False},
  'num_components': {'index': True,
   'primary_key': False,
   'sql': 'INTEGER',
```

(continues on next page)

(continued from previous page)

```
   'unique': False},
  'num_cut_vertices': {'index': True,
   'primary_key': False,
   'sql': 'INTEGER',
   'unique': False},
  'num_spanning_trees': {'index': True,
   'primary_key': False,
   'sql': 'INTEGER',
   'unique': False},
  'radius': {'index': True,
   'primary_key': False,
   'sql': 'INTEGER',
   'unique': False},
  'vertex_connectivity': {'index': True,
   'primary_key': False,
   'sql': 'BOOLEAN',
   'unique': False}},
 'spectrum': {'eigenvalues_sd': {'index': True,
   'primary_key': False,
   'sql': 'REAL',
   'unique': False},
  'energy': {'index': True,
   'primary_key': False,
   'sql': 'REAL',
   'unique': False},
  'graph_id': {'index': False,
   'primary_key': False,
   'sql': 'INTEGER',
   'unique': False},
  'max_eigenvalue': {'index': True,
   'primary_key': False,
   'sql': 'REAL',
   'unique': False},
  'min_eigenvalue': {'index': True,
   'primary_key': False,
   'sql': 'REAL',
   'unique': False},
  'spectrum': {'index': False,
   'primary_key': False,
   'sql': 'TEXT',
   'unique': False}}}
```

interactive_query(*display_cols*, ***kwds*)

Generate an interact shell to query the database.

> **Warning**
>
> This is no longer implemented since the switch to Python3.

This method generates an interact shell that allows the user to manipulate query parameters and see the updated results.

Todo

This function could use improvement. Add full options of typical *GraphQuery* (i.e.: have it accept list input); and update options in interact to make it less annoying to put in operators.

EXAMPLES:

```
sage: D = GraphDatabase()
sage: D.interactive_query(display_cols=['graph6', 'num_vertices', 'degree_
↪sequence'], num_edges=5, max_degree=3)                                    #␣
↪needs sage.symbolic
Traceback (most recent call last):
...
NotImplementedError: not available in Jupyter notebook
```

```
>>> from sage.all import *
>>> D = GraphDatabase()
>>> D.interactive_query(display_cols=['graph6', 'num_vertices', 'degree_
↪sequence'], num_edges=Integer(5), max_degree=Integer(3))                  ␣
↪                # needs sage.symbolic
Traceback (most recent call last):
...
NotImplementedError: not available in Jupyter notebook
```

query(*query_dict=None*, *display_cols=None*, ***kwds*)

Create a GraphQuery on this database.

For full class details, type `GraphQuery?` and press `Shift` + `Enter`.

EXAMPLES:

```
sage: D = GraphDatabase()
sage: q = D.query(display_cols=['graph6', 'num_vertices', 'degree_sequence'],␣
↪num_edges=['<=', 5])          # needs sage.symbolic
sage: q.show()                                                             #␣
↪needs sage.symbolic
Graph6                Num Vertices          Degree Sequence
-------------------------------------------------------------
@                     1                     [0]
A?                    2                     [0, 0]
A_                    2                     [1, 1]
B?                    3                     [0, 0, 0]
BG                    3                     [0, 1, 1]
BW                    3                     [1, 1, 2]
Bw                    3                     [2, 2, 2]
C?                    4                     [0, 0, 0, 0]
C@                    4                     [0, 0, 1, 1]
CB                    4                     [0, 1, 1, 2]
CF                    4                     [1, 1, 1, 3]
CJ                    4                     [0, 2, 2, 2]
CK                    4                     [1, 1, 1, 1]
CL                    4                     [1, 1, 2, 2]
CN                    4                     [1, 2, 2, 3]
```

(continues on next page)

(continued from previous page)

C]	4	[2, 2, 2, 2]
C^	4	[2, 2, 3, 3]
D??	5	[0, 0, 0, 0, 0]
D?C	5	[0, 0, 0, 1, 1]
D?K	5	[0, 0, 1, 1, 2]
D?[	5	[0, 1, 1, 1, 3]
D?{	5	[1, 1, 1, 1, 4]
D@K	5	[0, 0, 2, 2, 2]
D@O	5	[0, 1, 1, 1, 1]
D@S	5	[0, 1, 1, 2, 2]
D@[	5	[0, 1, 2, 2, 3]
D@s	5	[1, 1, 1, 2, 3]
D@{	5	[1, 1, 2, 2, 4]
DBW	5	[0, 2, 2, 2, 2]
DB[	5	[0, 2, 2, 3, 3]
DBg	5	[1, 1, 2, 2, 2]
DBk	5	[1, 1, 2, 3, 3]
DIk	5	[1, 2, 2, 2, 3]
DK[	5	[1, 2, 2, 2, 3]
DLo	5	[2, 2, 2, 2, 2]
D_K	5	[1, 1, 1, 1, 2]
D`K	5	[1, 1, 2, 2, 2]
E???	6	[0, 0, 0, 0, 0, 0]
E??G	6	[0, 0, 0, 0, 1, 1]
E??W	6	[0, 0, 0, 1, 1, 2]
E??w	6	[0, 0, 1, 1, 1, 3]
E?@w	6	[0, 1, 1, 1, 1, 4]
E?Bw	6	[1, 1, 1, 1, 1, 5]
E?CW	6	[0, 0, 0, 2, 2, 2]
E?C_	6	[0, 0, 1, 1, 1, 1]
E?Cg	6	[0, 0, 1, 1, 2, 2]
E?Cw	6	[0, 0, 1, 2, 2, 3]
E?Dg	6	[0, 1, 1, 1, 2, 3]
E?Dw	6	[0, 1, 1, 2, 2, 4]
E?Fg	6	[1, 1, 1, 1, 2, 4]
E?Ko	6	[0, 0, 2, 2, 2, 2]
E?Kw	6	[0, 0, 2, 2, 3, 3]
E?LO	6	[0, 1, 1, 2, 2, 2]
E?LW	6	[0, 1, 1, 2, 3, 3]
E?N?	6	[1, 1, 1, 1, 2, 2]
E?NG	6	[1, 1, 1, 1, 3, 3]
E@FG	6	[1, 1, 1, 2, 2, 3]
E@HW	6	[0, 1, 2, 2, 2, 3]
E@N?	6	[1, 1, 2, 2, 2, 2]
E@Ow	6	[0, 1, 2, 2, 2, 3]
E@Q?	6	[1, 1, 1, 1, 1, 1]
E@QW	6	[1, 1, 1, 2, 2, 3]
E@T_	6	[0, 2, 2, 2, 2, 2]
E@YO	6	[1, 1, 2, 2, 2, 2]
EG?W	6	[0, 1, 1, 1, 1, 2]
EGCW	6	[0, 1, 1, 2, 2, 2]
E_?w	6	[1, 1, 1, 1, 1, 3]

(continues on next page)

(continued from previous page)

```
E_Cg                    6                    [1, 1, 1, 1, 2, 2]
E_Cw                    6                    [1, 1, 1, 2, 2, 3]
E_Ko                    6                    [1, 1, 2, 2, 2, 2]
F????                   7                    [0, 0, 0, 0, 0, 0, 0]
F???G                   7                    [0, 0, 0, 0, 0, 1, 1]
F???W                   7                    [0, 0, 0, 0, 1, 1, 2]
F???w                   7                    [0, 0, 0, 1, 1, 1, 3]
F??@w                   7                    [0, 0, 1, 1, 1, 1, 4]
F??Bw                   7                    [0, 1, 1, 1, 1, 1, 5]
F??GW                   7                    [0, 0, 0, 0, 2, 2, 2]
F??G_                   7                    [0, 0, 0, 1, 1, 1, 1]
F??Gg                   7                    [0, 0, 0, 1, 1, 2, 2]
F??Gw                   7                    [0, 0, 0, 1, 2, 2, 3]
F??Hg                   7                    [0, 0, 1, 1, 1, 2, 3]
F??Hw                   7                    [0, 0, 1, 1, 2, 2, 4]
F??Jg                   7                    [0, 1, 1, 1, 1, 2, 4]
F??Wo                   7                    [0, 0, 0, 2, 2, 2, 2]
F??Ww                   7                    [0, 0, 0, 2, 2, 3, 3]
F??XO                   7                    [0, 0, 1, 1, 2, 2, 2]
F??XW                   7                    [0, 0, 1, 1, 2, 3, 3]
F??Z?                   7                    [0, 1, 1, 1, 1, 2, 2]
F??ZG                   7                    [0, 1, 1, 1, 1, 3, 3]
F??^?                   7                    [1, 1, 1, 1, 1, 2, 3]
F?CJG                   7                    [0, 1, 1, 1, 2, 2, 3]
F?CPW                   7                    [0, 0, 1, 2, 2, 2, 3]
F?CZ?                   7                    [0, 1, 1, 2, 2, 2, 2]
F?C_w                   7                    [0, 0, 1, 2, 2, 2, 3]
F?Ca?                   7                    [0, 1, 1, 1, 1, 1, 1]
F?CaW                   7                    [0, 1, 1, 1, 2, 2, 3]
F?Ch_                   7                    [0, 0, 2, 2, 2, 2, 2]
F?CqO                   7                    [0, 1, 1, 2, 2, 2, 2]
F?LCG                   7                    [1, 1, 1, 1, 2, 2, 2]
F@??W                   7                    [0, 0, 1, 1, 1, 1, 2]
F@?GW                   7                    [0, 0, 1, 1, 2, 2, 2]
FG??w                   7                    [0, 1, 1, 1, 1, 1, 3]
FG?Gg                   7                    [0, 1, 1, 1, 1, 2, 2]
FG?Gw                   7                    [0, 1, 1, 1, 2, 2, 3]
FG?Wo                   7                    [0, 1, 1, 2, 2, 2, 2]
FK??W                   7                    [1, 1, 1, 1, 1, 1, 2]
FK?GW                   7                    [1, 1, 1, 1, 2, 2, 2]
F_?@w                   7                    [1, 1, 1, 1, 1, 1, 4]
F_?Hg                   7                    [1, 1, 1, 1, 1, 2, 3]
F_?XO                   7                    [1, 1, 1, 1, 2, 2, 2]
```

```
>>> from sage.all import *
>>> D = GraphDatabase()
>>> q = D.query(display_cols=['graph6', 'num_vertices', 'degree_sequence'],␣
↪num_edges=['<=', Integer(5)])          # needs sage.symbolic
>>> q.show()                                                              #␣
↪needs sage.symbolic
Graph6               Num Vertices          Degree Sequence
------------------------------------------------------------
```

(continues on next page)

(continued from previous page)

```
@                       1                       [0]
A?                      2                       [0, 0]
A_                      2                       [1, 1]
B?                      3                       [0, 0, 0]
BG                      3                       [0, 1, 1]
BW                      3                       [1, 1, 2]
Bw                      3                       [2, 2, 2]
C?                      4                       [0, 0, 0, 0]
C@                      4                       [0, 0, 1, 1]
CB                      4                       [0, 1, 1, 2]
CF                      4                       [1, 1, 1, 3]
CJ                      4                       [0, 2, 2, 2]
CK                      4                       [1, 1, 1, 1]
CL                      4                       [1, 1, 2, 2]
CN                      4                       [1, 2, 2, 3]
C]                      4                       [2, 2, 2, 2]
C^                      4                       [2, 2, 3, 3]
D??                     5                       [0, 0, 0, 0, 0]
D?C                     5                       [0, 0, 0, 1, 1]
D?K                     5                       [0, 0, 1, 1, 2]
D?[                     5                       [0, 1, 1, 1, 3]
D?{                     5                       [1, 1, 1, 1, 4]
D@K                     5                       [0, 0, 2, 2, 2]
D@O                     5                       [0, 1, 1, 1, 1]
D@S                     5                       [0, 1, 1, 2, 2]
D@[                     5                       [0, 1, 2, 2, 3]
D@s                     5                       [1, 1, 1, 2, 3]
D@{                     5                       [1, 1, 2, 2, 4]
DBW                     5                       [0, 2, 2, 2, 2]
DB[                     5                       [0, 2, 2, 3, 3]
DBg                     5                       [1, 1, 2, 2, 2]
DBk                     5                       [1, 1, 2, 3, 3]
DIk                     5                       [1, 2, 2, 2, 3]
DK[                     5                       [1, 2, 2, 2, 3]
DLo                     5                       [2, 2, 2, 2, 2]
D_K                     5                       [1, 1, 1, 1, 2]
D`K                     5                       [1, 1, 2, 2, 2]
E????                   6                       [0, 0, 0, 0, 0, 0]
E??G                    6                       [0, 0, 0, 0, 1, 1]
E??W                    6                       [0, 0, 0, 1, 1, 2]
E??w                    6                       [0, 0, 1, 1, 1, 3]
E?@w                    6                       [0, 1, 1, 1, 1, 4]
E?Bw                    6                       [1, 1, 1, 1, 1, 5]
E?CW                    6                       [0, 0, 0, 2, 2, 2]
E?C_                    6                       [0, 0, 1, 1, 1, 1]
E?Cg                    6                       [0, 0, 1, 1, 2, 2]
E?Cw                    6                       [0, 0, 1, 2, 2, 3]
E?Dg                    6                       [0, 1, 1, 1, 2, 3]
E?Dw                    6                       [0, 1, 1, 2, 2, 4]
E?Fg                    6                       [1, 1, 1, 1, 2, 4]
E?Ko                    6                       [0, 0, 2, 2, 2, 2]
E?Kw                    6                       [0, 0, 2, 2, 3, 3]
```

(continues on next page)

(continued from previous page)

```
E?LO                6   [0, 1, 1, 2, 2, 2]
E?LW                6   [0, 1, 1, 2, 3, 3]
E?N?                6   [1, 1, 1, 1, 2, 2]
E?NG                6   [1, 1, 1, 1, 3, 3]
E@FG                6   [1, 1, 1, 2, 2, 3]
E@HW                6   [0, 1, 2, 2, 2, 3]
E@N?                6   [1, 1, 2, 2, 2, 2]
E@Ow                6   [0, 1, 2, 2, 2, 3]
E@Q?                6   [1, 1, 1, 1, 1, 1]
E@QW                6   [1, 1, 1, 2, 2, 3]
E@T_                6   [0, 2, 2, 2, 2, 2]
E@YO                6   [1, 1, 2, 2, 2, 2]
EG?W                6   [0, 1, 1, 1, 1, 2]
EGCW                6   [0, 1, 1, 2, 2, 2]
E_?w                6   [1, 1, 1, 1, 1, 3]
E_Cg                6   [1, 1, 1, 1, 2, 2]
E_Cw                6   [1, 1, 1, 2, 2, 3]
E_Ko                6   [1, 1, 2, 2, 2, 2]
F????               7   [0, 0, 0, 0, 0, 0, 0]
F???G               7   [0, 0, 0, 0, 0, 1, 1]
F???W               7   [0, 0, 0, 0, 1, 1, 2]
F???w               7   [0, 0, 0, 1, 1, 1, 3]
F??@w               7   [0, 0, 1, 1, 1, 1, 4]
F??Bw               7   [0, 1, 1, 1, 1, 1, 5]
F??GW               7   [0, 0, 0, 0, 2, 2, 2]
F??G_               7   [0, 0, 0, 1, 1, 1, 1]
F??Gg               7   [0, 0, 0, 1, 1, 2, 2]
F??Gw               7   [0, 0, 0, 1, 2, 2, 3]
F??Hg               7   [0, 0, 1, 1, 1, 2, 3]
F??Hw               7   [0, 0, 1, 1, 2, 2, 4]
F??Jg               7   [0, 1, 1, 1, 1, 2, 4]
F??Wo               7   [0, 0, 0, 2, 2, 2, 2]
F??Ww               7   [0, 0, 0, 2, 2, 3, 3]
F??XO               7   [0, 0, 1, 1, 2, 2, 2]
F??XW               7   [0, 0, 1, 1, 2, 3, 3]
F??Z?               7   [0, 1, 1, 1, 1, 2, 2]
F??ZG               7   [0, 1, 1, 1, 1, 3, 3]
F??^?               7   [1, 1, 1, 1, 1, 2, 3]
F?CJG               7   [0, 1, 1, 1, 2, 2, 3]
F?CPW               7   [0, 0, 1, 2, 2, 2, 3]
F?CZ?               7   [0, 1, 1, 2, 2, 2, 2]
F?C_w               7   [0, 0, 1, 2, 2, 2, 3]
F?Ca?               7   [0, 1, 1, 1, 1, 1, 1]
F?CaW               7   [0, 1, 1, 1, 2, 2, 3]
F?Ch_               7   [0, 0, 2, 2, 2, 2, 2]
F?CqO               7   [0, 1, 1, 2, 2, 2, 2]
F?LCG               7   [1, 1, 1, 1, 2, 2, 2]
F@??W               7   [0, 0, 1, 1, 1, 1, 2]
F@?GW               7   [0, 0, 1, 1, 2, 2, 2]
FG??w               7   [0, 1, 1, 1, 1, 1, 3]
FG?Gg               7   [0, 1, 1, 1, 1, 2, 2]
FG?Gw               7   [0, 1, 1, 1, 2, 2, 3]
```

(continues on next page)

(continued from previous page)

```
FG?Wo                    7                    [0, 1, 1, 2, 2, 2, 2]
FK??W                    7                    [1, 1, 1, 1, 1, 1, 2]
FK?GW                    7                    [1, 1, 1, 1, 2, 2, 2]
F_?@w                    7                    [1, 1, 1, 1, 1, 1, 4]
F_?Hg                    7                    [1, 1, 1, 1, 1, 2, 3]
F_?XO                    7                    [1, 1, 1, 1, 2, 2, 2]
```

class sage.graphs.graph_database.**GraphQuery**(*graph_db=None*, *query_dict=None*, *display_cols=None*, *immutable=False*, ***kwds*)

Bases: *GenericGraphQuery*

A query for an instance of *GraphDatabase*.

This class nicely wraps the sage.databases.sql_db.SQLQuery class located in sage.databases.sql_db to make the query constraints intuitive and with as many pre-definitions as possible. (i.e.: since it has to be a *GraphDatabase*, we already know the table structure and types; and since it is immutable, we can treat these as a guarantee).

Note

sage.databases.sql_db.SQLQuery functions are available for *GraphQuery*. See sage.databases.sql_db for more details.

INPUT:

- graph_db – *GraphDatabase* (default: None); instance to apply the query to (If None, then a new instance is created)
- query_dict – dictionary (default: None); a dictionary specifying the query itself. Format is: {'table_name': 'tblname', 'display_cols': ['col1', 'col2'], 'expression': [col, operator, value]}. If not None, query_dict will take precedence over all other arguments.
- display_cols – list of strings (default: None); a list of column names (strings) to display in the result when running or showing a query
- immutable – boolean (default: False); whether to return immutable or mutable graphs
- kwds – the columns of the database are all keywords. For a database table/column structure dictionary, call *graph_db_info()*. Keywords accept both single values and lists of length 2. The list allows the user to specify an expression other than equality. Valid expressions are strings, and for numeric values (i.e. Reals and Integers) are: '=','','','=','='. String values also accept 'regexp' as an expression argument. The only keyword exception to this format is induced_subgraphs, which accepts one of the following options:
  - ['one_of', String, ..., String] – will search for graphs containing a subgraph isomorphic to *any* of the graph6 strings in the list
  - ['all_of', String, ..., String] – will search for graphs containing a subgraph isomorphic to *each* of the graph6 strings in the list

EXAMPLES:

```
sage: Q = GraphQuery(display_cols=['graph6', 'num_vertices', 'degree_sequence'],␣
↪num_edges=['<=', 5], min_degree=1)
sage: Q.number_of()
35
sage: Q.show()
```

(continues on next page)

(continued from previous page)

```
Graph6                Num Vertices          Degree Sequence
-------------------------------------------------------------
A_                    2                     [1, 1]
BW                    3                     [1, 1, 2]
CF                    4                     [1, 1, 1, 3]
CK                    4                     [1, 1, 1, 1]
CL                    4                     [1, 1, 2, 2]
CN                    4                     [1, 2, 2, 3]
D?{                   5                     [1, 1, 1, 1, 4]
D@s                   5                     [1, 1, 1, 2, 3]
D@{                   5                     [1, 1, 2, 2, 4]
DBg                   5                     [1, 1, 2, 2, 2]
DBk                   5                     [1, 1, 2, 3, 3]
DIk                   5                     [1, 2, 2, 2, 3]
DK[                   5                     [1, 2, 2, 2, 3]
D_K                   5                     [1, 1, 1, 1, 2]
D`K                   5                     [1, 1, 2, 2, 2]
E?Bw                  6                     [1, 1, 1, 1, 1, 5]
E?Fg                  6                     [1, 1, 1, 1, 2, 4]
E?N?                  6                     [1, 1, 1, 1, 2, 2]
E?NG                  6                     [1, 1, 1, 1, 3, 3]
E@FG                  6                     [1, 1, 1, 2, 2, 3]
E@N?                  6                     [1, 1, 2, 2, 2, 2]
E@Q?                  6                     [1, 1, 1, 1, 1, 1]
E@QW                  6                     [1, 1, 1, 2, 2, 3]
E@YO                  6                     [1, 1, 2, 2, 2, 2]
E_?w                  6                     [1, 1, 1, 1, 1, 3]
E_Cg                  6                     [1, 1, 1, 1, 2, 2]
E_Cw                  6                     [1, 1, 1, 2, 2, 3]
E_Ko                  6                     [1, 1, 2, 2, 2, 2]
F??^?                 7                     [1, 1, 1, 1, 1, 2, 3]
F?LCG                 7                     [1, 1, 1, 1, 2, 2, 2]
FK??W                 7                     [1, 1, 1, 1, 1, 1, 2]
FK?GW                 7                     [1, 1, 1, 1, 2, 2, 2]
F_?@w                 7                     [1, 1, 1, 1, 1, 1, 4]
F_?Hg                 7                     [1, 1, 1, 1, 1, 2, 3]
F_?XO                 7                     [1, 1, 1, 1, 2, 2, 2]
```

```
>>> from sage.all import *
>>> Q = GraphQuery(display_cols=['graph6', 'num_vertices', 'degree_sequence'],␣
↪num_edges=['<=', Integer(5)], min_degree=Integer(1))
>>> Q.number_of()
35
>>> Q.show()
Graph6                Num Vertices          Degree Sequence
-------------------------------------------------------------
A_                    2                     [1, 1]
BW                    3                     [1, 1, 2]
CF                    4                     [1, 1, 1, 3]
CK                    4                     [1, 1, 1, 1]
CL                    4                     [1, 1, 2, 2]
CN                    4                     [1, 2, 2, 3]
```

(continues on next page)

(continued from previous page)

```
D?{                    5                    [1, 1, 1, 1, 4]
D@s                    5                    [1, 1, 1, 2, 3]
D@{                    5                    [1, 1, 2, 2, 4]
DBg                    5                    [1, 1, 2, 2, 2]
DBk                    5                    [1, 1, 2, 3, 3]
DIk                    5                    [1, 2, 2, 2, 3]
DK[                    5                    [1, 2, 2, 2, 3]
D_K                    5                    [1, 1, 1, 1, 2]
D`K                    5                    [1, 1, 2, 2, 2]
E?Bw                   6                    [1, 1, 1, 1, 1, 5]
E?Fg                   6                    [1, 1, 1, 1, 2, 4]
E?N?                   6                    [1, 1, 1, 1, 2, 2]
E?NG                   6                    [1, 1, 1, 1, 3, 3]
E@FG                   6                    [1, 1, 1, 2, 2, 3]
E@N?                   6                    [1, 1, 2, 2, 2, 2]
E@Q?                   6                    [1, 1, 1, 1, 1, 1]
E@QW                   6                    [1, 1, 1, 2, 2, 3]
E@YO                   6                    [1, 1, 2, 2, 2, 2]
E_?w                   6                    [1, 1, 1, 1, 1, 3]
E_Cg                   6                    [1, 1, 1, 1, 2, 2]
E_Cw                   6                    [1, 1, 1, 2, 2, 3]
E_Ko                   6                    [1, 1, 2, 2, 2, 2]
F??^?                  7                    [1, 1, 1, 1, 1, 2, 3]
F?LCG                  7                    [1, 1, 1, 1, 2, 2, 2]
FK??W                  7                    [1, 1, 1, 1, 1, 1, 2]
FK?GW                  7                    [1, 1, 1, 1, 2, 2, 2]
F_?@w                  7                    [1, 1, 1, 1, 1, 1, 4]
F_?Hg                  7                    [1, 1, 1, 1, 1, 2, 3]
F_?XO                  7                    [1, 1, 1, 1, 2, 2, 2]
```

Check the behavior of parameter `immutable`:

```
sage: Q = GraphQuery(display_cols=['graph6'], num_vertices=3)
sage: any(g.is_immutable() for g in Q)
False
sage: Q = GraphQuery(display_cols=['graph6'], num_vertices=3, immutable=True)
sage: all(g.is_immutable() for g in Q)
True
```

```
>>> from sage.all import *
>>> Q = GraphQuery(display_cols=['graph6'], num_vertices=Integer(3))
>>> any(g.is_immutable() for g in Q)
False
>>> Q = GraphQuery(display_cols=['graph6'], num_vertices=Integer(3),␣
↪immutable=True)
>>> all(g.is_immutable() for g in Q)
True
```

get_graphs_list(*immutable=None*)

Return a list of Sage Graph objects that satisfy the query.

INPUT:

- `immutable` – boolean (default: `None`); whether to create mutable/immutable graphs. By default

(`immutable=None`), follow the behavior of `self`.

EXAMPLES:

```
sage: Q = GraphQuery(display_cols=['graph6', 'num_vertices', 'degree_sequence
↪'], num_edges=['<=', 5], min_degree=1)
sage: L = Q.get_graphs_list()
sage: L[0]
Graph on 2 vertices
sage: len(L)
35
```

```
>>> from sage.all import *
>>> Q = GraphQuery(display_cols=['graph6', 'num_vertices', 'degree_sequence'],
↪ num_edges=['<=', Integer(5)], min_degree=Integer(1))
>>> L = Q.get_graphs_list()
>>> L[Integer(0)]
Graph on 2 vertices
>>> len(L)
35
```

Check the behavior of parameter `immutable`:

```
sage: Q = GraphQuery(display_cols=['graph6'], num_vertices=3)
sage: any(g.is_immutable() for g in Q.get_graphs_list())
False
sage: all(g.is_immutable() for g in Q.get_graphs_list(immutable=True))
True
sage: Q = GraphQuery(display_cols=['graph6'], num_vertices=3, immutable=True)
sage: all(g.is_immutable() for g in Q.get_graphs_list())
True
sage: any(g.is_immutable() for g in Q.get_graphs_list(immutable=False))
False
```

```
>>> from sage.all import *
>>> Q = GraphQuery(display_cols=['graph6'], num_vertices=Integer(3))
>>> any(g.is_immutable() for g in Q.get_graphs_list())
False
>>> all(g.is_immutable() for g in Q.get_graphs_list(immutable=True))
True
>>> Q = GraphQuery(display_cols=['graph6'], num_vertices=Integer(3),␣
↪immutable=True)
>>> all(g.is_immutable() for g in Q.get_graphs_list())
True
>>> any(g.is_immutable() for g in Q.get_graphs_list(immutable=False))
False
```

number_of()

Return the number of graphs in the database that satisfy the query.

EXAMPLES:

```
sage: Q = GraphQuery(display_cols=['graph6', 'num_vertices', 'degree_sequence
↪'] ,num_edges=['<=', 5], min_degree=1)
```

(continues on next page)

(continued from previous page)

```
sage: Q.number_of()
35
```

```
>>> from sage.all import *
>>> Q = GraphQuery(display_cols=['graph6', 'num_vertices', 'degree_sequence'],␣
↪num_edges=['<=', Integer(5)], min_degree=Integer(1))
>>> Q.number_of()
35
```

query_iterator(*immutable=None*)

Return an iterator over the results list of the *GraphQuery*.

INPUT:

- `immutable` – boolean (default: `None`); whether to create mutable/immutable graphs. By default (`immutable=None`), follow the behavior of `self`.

EXAMPLES:

```
sage: Q = GraphQuery(display_cols=['graph6'], num_vertices=7, diameter=5)
sage: for g in Q:
....:     print(g.graph6_string())
F?`po
F?gqg
F@?]O
F@OKg
F@R@o
FA_pW
FEOhW
FGC{o
FIAHo
sage: Q = GraphQuery(display_cols=['graph6'], num_vertices=7, diameter=5)
sage: it = iter(Q)
sage: while True:
....:     try: print(next(it).graph6_string())
....:     except StopIteration: break
F?`po
F?gqg
F@?]O
F@OKg
F@R@o
FA_pW
FEOhW
FGC{o
FIAHo
```

```
>>> from sage.all import *
>>> Q = GraphQuery(display_cols=['graph6'], num_vertices=Integer(7),␣
↪diameter=Integer(5))
>>> for g in Q:
...     print(g.graph6_string())
F?`po
F?gqg
```

(continues on next page)

(continued from previous page)

```
F@?]O
F@OKg
F@R@o
FA_pW
FEOhW
FGC{o
FIAHo
>>> Q = GraphQuery(display_cols=['graph6'], num_vertices=Integer(7),␣
↪diameter=Integer(5))
>>> it = iter(Q)
>>> while True:
...     try: print(next(it).graph6_string())
...     except StopIteration: break
F?`po
F?gqg
F@?]O
F@OKg
F@R@o
FA_pW
FEOhW
FGC{o
FIAHo
```

Check the behavior of parameter `immutable`:

```
sage: Q = GraphQuery(display_cols=['graph6'], num_vertices=3)
sage: any(g.is_immutable() for g in Q.query_iterator())
False
sage: all(g.is_immutable() for g in Q.query_iterator(immutable=True))
True
sage: Q = GraphQuery(display_cols=['graph6'], num_vertices=3, immutable=True)
sage: all(g.is_immutable() for g in Q.query_iterator())
True
sage: any(g.is_immutable() for g in Q.query_iterator(immutable=False))
False
```

```
>>> from sage.all import *
>>> Q = GraphQuery(display_cols=['graph6'], num_vertices=Integer(3))
>>> any(g.is_immutable() for g in Q.query_iterator())
False
>>> all(g.is_immutable() for g in Q.query_iterator(immutable=True))
True
>>> Q = GraphQuery(display_cols=['graph6'], num_vertices=Integer(3),␣
↪immutable=True)
>>> all(g.is_immutable() for g in Q.query_iterator())
True
>>> any(g.is_immutable() for g in Q.query_iterator(immutable=False))
False
```

show(*max_field_size=20*, *with_picture=False*)

Display the results of a query in table format.

INPUT:

- `max_field_size` – integer (default: 20); width of fields in command prompt version
- `with_picture` – boolean (default: `False`); whether or not to display results with a picture of the graph (available only in the notebook)

EXAMPLES:

```
sage: G = GraphDatabase()
sage: Q = GraphQuery(G, display_cols=['graph6','num_vertices','aut_grp_size'],
↪ num_vertices=4, aut_grp_size=4)
sage: Q.show()
Graph6                Num Vertices          Aut Grp Size
------------------------------------------------------------
C@                    4                     4
C^                    4                     4
```

```
>>> from sage.all import *
>>> G = GraphDatabase()
>>> Q = GraphQuery(G, display_cols=['graph6','num_vertices','aut_grp_size'],␣
↪num_vertices=Integer(4), aut_grp_size=Integer(4))
>>> Q.show()
Graph6                Num Vertices          Aut Grp Size
------------------------------------------------------------
C@                    4                     4
C^                    4                     4
```

```
sage: R = GraphQuery(G, display_cols=['graph6','num_vertices','degree_sequence
↪'], num_vertices=4)
sage: R.show()
Graph6                Num Vertices          Degree Sequence
------------------------------------------------------------
C?                    4                     [0, 0, 0, 0]
C@                    4                     [0, 0, 1, 1]
CB                    4                     [0, 1, 1, 2]
CF                    4                     [1, 1, 1, 3]
CJ                    4                     [0, 2, 2, 2]
CK                    4                     [1, 1, 1, 1]
CL                    4                     [1, 1, 2, 2]
CN                    4                     [1, 2, 2, 3]
C]                    4                     [2, 2, 2, 2]
C^                    4                     [2, 2, 3, 3]
C~                    4                     [3, 3, 3, 3]
```

```
>>> from sage.all import *
>>> R = GraphQuery(G, display_cols=['graph6','num_vertices','degree_sequence
↪'], num_vertices=Integer(4))
>>> R.show()
Graph6                Num Vertices          Degree Sequence
------------------------------------------------------------
C?                    4                     [0, 0, 0, 0]
C@                    4                     [0, 0, 1, 1]
CB                    4                     [0, 1, 1, 2]
CF                    4                     [1, 1, 1, 3]
CJ                    4                     [0, 2, 2, 2]
```

(continues on next page)

(continued from previous page)

```
CK                    4                     [1, 1, 1, 1]
CL                    4                     [1, 1, 2, 2]
CN                    4                     [1, 2, 2, 3]
C]                    4                     [2, 2, 2, 2]
C^                    4                     [2, 2, 3, 3]
C~                    4                     [3, 3, 3, 3]
```

Show the pictures (in notebook mode only):

```
sage: S = GraphQuery(G, display_cols=['graph6','aut_grp_size'], num_
↪vertices=4)
sage: S.show(with_picture=True)
Traceback (most recent call last):
...
NotImplementedError: Cannot display plot on command line.
```

```
>>> from sage.all import *
>>> S = GraphQuery(G, display_cols=['graph6','aut_grp_size'], num_
↪vertices=Integer(4))
>>> S.show(with_picture=True)
Traceback (most recent call last):
...
NotImplementedError: Cannot display plot on command line.
```

Note that pictures can be turned off:

```
sage: S.show(with_picture=False)
Graph6                Aut Grp Size
---------------------------------------
C?                    24
C@                    4
CB                    2
CF                    6
CJ                    6
CK                    8
CL                    2
CN                    2
C]                    8
C^                    4
C~                    24
```

```
>>> from sage.all import *
>>> S.show(with_picture=False)
Graph6                Aut Grp Size
---------------------------------------
C?                    24
C@                    4
CB                    2
CF                    6
CJ                    6
CK                    8
CL                    2
```

(continues on next page)

(continued from previous page)

```
CN                    2
C]                    8
C^                    4
C~                    24
```

Show your own query (note that the output is not reformatted for generic queries):

```
sage: (GenericGraphQuery('select degree_sequence from degrees where max_
↪degree=2 and min_degree >= 1', G)).show()
degree_sequence
--------------------
211
222
2211
2222
21111
22211
22211
22222
221111
221111
222211
222211
222211
222222
222222
2111111
2221111
2221111
2221111
2222211
2222211
2222211
2222211
2222222
2222222
```

```
>>> from sage.all import *
>>> (GenericGraphQuery('select degree_sequence from degrees where max_
↪degree=2 and min_degree >= 1', G)).show()
degree_sequence
--------------------
211
222
2211
2222
21111
22211
22211
22222
221111
221111
```

(continues on next page)

(continued from previous page)

```
222211
222211
222211
222222
222222
2111111
2221111
2221111
2221111
2222211
2222211
2222211
2222211
2222222
2222222
```

sage.graphs.graph_database.**data_to_degseq**(*data*, *graph6=None*)

Convert a database integer data type to a degree sequence list.

INPUT:

- `data` – integer data type (one digit per vertex representing its degree, sorted high to low) to be converted to a degree sequence list
- `graph6` – string (default: `None`); the `graph6` identifier is required for all graphs with no edges, so that the correct number of zeros is returned.

EXAMPLES:

```
sage: from sage.graphs.graph_database import data_to_degseq
sage: data_to_degseq(3221)
[1, 2, 2, 3]
sage: data_to_degseq(0, 'D??')
[0, 0, 0, 0, 0]
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_database import data_to_degseq
>>> data_to_degseq(Integer(3221))
[1, 2, 2, 3]
>>> data_to_degseq(Integer(0), 'D??')
[0, 0, 0, 0, 0]
```

sage.graphs.graph_database.**degseq_to_data**(*degree_sequence*)

Convert a degree sequence list to a sorted (max-min) integer data type.

The input degree sequence list (of Integers) is converted to a sorted (max-min) integer data type, as used for faster access in the underlying database.

INPUT:

- `degree_sequence` – list of integers; input degree sequence list

EXAMPLES:

```
sage: from sage.graphs.graph_database import degseq_to_data
sage: degseq_to_data([2,2,3,1])
3221
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_database import degseq_to_data
>>> degseq_to_data([Integer(2),Integer(2),Integer(3),Integer(1)])
3221
```

sage.graphs.graph_database.**graph6_to_plot**(*graph6*)

Return a Graphics object from a graph6 string.

This method constructs a graph from a graph6 string and returns a sage.plot.graphics.Graphics object with arguments preset for the sage.plot.graphics.Graphics.show() method.

INPUT:

- graph6 – a graph6 string

EXAMPLES:

```
sage: from sage.graphs.graph_database import graph6_to_plot
sage: type(graph6_to_plot('D??'))                                           #␣
↪needs sage.plot
<class 'sage.plot.graphics.Graphics'>
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_database import graph6_to_plot
>>> type(graph6_to_plot('D??'))                                             #␣
↪needs sage.plot
<class 'sage.plot.graphics.Graphics'>
```

sage.graphs.graph_database.**graph_db_info**(*tablename=None*)

Return a dictionary of allowed table and column names.

INPUT:

- tablename – restricts the output to a single table

EXAMPLES:

```
sage: sorted(graph_db_info())
['aut_grp', 'degrees', 'graph_data', 'misc', 'spectrum']
```

```
>>> from sage.all import *
>>> sorted(graph_db_info())
['aut_grp', 'degrees', 'graph_data', 'misc', 'spectrum']
```

```
sage: graph_db_info(tablename='graph_data')
['complement_graph6',
 'eulerian',
 'graph6',
 'lovasz_number',
 'num_cycles',
 'num_edges',
```

(continues on next page)

(continued from previous page)

```
 'num_hamiltonian_cycles',
 'num_vertices',
 'perfect',
 'planar']
```

```
>>> from sage.all import *
>>> graph_db_info(tablename='graph_data')
['complement_graph6',
 'eulerian',
 'graph6',
 'lovasz_number',
 'num_cycles',
 'num_edges',
 'num_hamiltonian_cycles',
 'num_vertices',
 'perfect',
 'planar']
```

sage.graphs.graph_database.**subgraphs_to_query**(*subgraphs*, *db*)

Return a GraphQuery object required for the induced_subgraphs parameter.

This method constructs and returns a *GraphQuery* object respecting the special input required for the `induced_subgraphs` parameter.

INPUT:

- `subgraphs` – list of strings; the list should be of one of the following two formats:
  - `['one_of', String, ..., String]` – will search for graphs containing a subgraph isomorphic to *any* of the `graph6` strings in the list
  - `['all_of', String, ..., String]` – will search for graphs containing a subgraph isomorphic to *each* of the `graph6` strings in the list
- db – a *GraphDatabase*

Note

This is a helper method called by the *GraphQuery* constructor to handle this special format. This method should not be used on its own because it doesn't set any display columns in the query string, causing a failure to fetch the data when run.

EXAMPLES:

```
sage: from sage.graphs.graph_database import subgraphs_to_query
sage: gd = GraphDatabase()
sage: q = subgraphs_to_query(['all_of', 'A?', 'B?', 'C?'], gd)
sage: q.get_query_string()
'SELECT ,,,,,  FROM misc WHERE ( ( misc.induced_subgraphs regexp ? ) AND (
misc.induced_subgraphs regexp ? ) ) AND ( misc.induced_subgraphs regexp ? )'
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_database import subgraphs_to_query
```

(continues on next page)

(continued from previous page)

```
>>> gd = GraphDatabase()
>>> q = subgraphs_to_query(['all_of', 'A?', 'B?', 'C?'], gd)
>>> q.get_query_string()
'SELECT ,,,,,  FROM misc WHERE ( ( misc.induced_subgraphs regexp ? ) AND (
misc.induced_subgraphs regexp ? ) ) AND ( misc.induced_subgraphs regexp ? )'
```

2.5 Database of strongly regular graphs

This module manages a database associating to a set of four integers (v, k, λ, μ) a strongly regular graphs with these parameters, when one exists.

Using Andries Brouwer's database of strongly regular graphs, it can also return non-existence results. Note that some constructions are missing, and that some strongly regular graphs that exist in the database cannot be automatically built by Sage. Help us if you know any. An outline of the implementation can be found in [CP2016].

Note

Any missing/incorrect information in the database must be reported to Andries E. Brouwer directly, in order to have a unique and updated source of information.

REFERENCES:

[BL1984]

2.5.1 Functions

`sage.graphs.strongly_regular_db.`**`SRG_100_44_18_20`**(*immutable=False*)

Return a $(100, 44, 18, 20)$-strongly regular graph.

This graph is built as a Cayley graph, using the construction for Δ_1 with group H_3 presented in Table 8.1 of [JK2003]

EXAMPLES:

```
sage: from sage.graphs.strongly_regular_db import SRG_100_44_18_20
sage: G = SRG_100_44_18_20()                    # long time                    #␣
↪needs sage.groups
sage: G.is_strongly_regular(parameters=True)    # long time                    #␣
↪needs sage.groups
(100, 44, 18, 20)
```

```
>>> from sage.all import *
>>> from sage.graphs.strongly_regular_db import SRG_100_44_18_20
>>> G = SRG_100_44_18_20()                    # long time                    #␣
↪needs sage.groups
>>> G.is_strongly_regular(parameters=True)    # long time                    #␣
↪needs sage.groups
(100, 44, 18, 20)
```

`sage.graphs.strongly_regular_db.`**`SRG_100_45_20_20`**(*immutable=False*)

Return a $(100, 45, 20, 20)$-strongly regular graph.

This graph is built as a Cayley graph, using the construction for Γ_3 with group H_3 presented in Table 8.1 of [JK2003].

EXAMPLES:

```
sage: from sage.graphs.strongly_regular_db import SRG_100_45_20_20
sage: G = SRG_100_45_20_20()                    # long time                    #␣
↪needs sage.groups
sage: G.is_strongly_regular(parameters=True)    # long time                    #␣
↪needs sage.groups
(100, 45, 20, 20)
```

```
>>> from sage.all import *
>>> from sage.graphs.strongly_regular_db import SRG_100_45_20_20
>>> G = SRG_100_45_20_20()                    # long time                    #␣
↪needs sage.groups
>>> G.is_strongly_regular(parameters=True)    # long time                    #␣
↪needs sage.groups
(100, 45, 20, 20)
```

sage.graphs.strongly_regular_db.**SRG_105_32_4_12**(*immutable=False*)

Return a $(105, 32, 4, 12)$-strongly regular graph.

The vertices are the flags of the projective plane of order 4. Two flags (a, A) and (b, B) are adjacent if the point a is on the line B or the point b is on the line A, and $a \neq b$, $A \neq B$. See Theorem 2.7 in [GS1970], and [Coo2006].

EXAMPLES:

```
sage: from sage.graphs.strongly_regular_db import SRG_105_32_4_12
sage: G = SRG_105_32_4_12(); G                                                 #␣
↪needs sage.rings.finite_rings
Aut L(3,4) on flags: Graph on 105 vertices
sage: G.is_strongly_regular(parameters=True)                                   #␣
↪needs sage.rings.finite_rings
(105, 32, 4, 12)
```

```
>>> from sage.all import *
>>> from sage.graphs.strongly_regular_db import SRG_105_32_4_12
>>> G = SRG_105_32_4_12(); G                                                   #␣
↪needs sage.rings.finite_rings
Aut L(3,4) on flags: Graph on 105 vertices
>>> G.is_strongly_regular(parameters=True)                                     #␣
↪needs sage.rings.finite_rings
(105, 32, 4, 12)
```

sage.graphs.strongly_regular_db.**SRG_120_63_30_36**(*immutable=False*)

Return a $(120, 63, 30, 36)$-strongly regular graph.

It is the distance-2 graph of *JohnsonGraph(10,3)*.

EXAMPLES:

```
sage: from sage.graphs.strongly_regular_db import SRG_120_63_30_36
sage: G =  SRG_120_63_30_36()
sage: G.is_strongly_regular(parameters=True)
(120, 63, 30, 36)
```

```
>>> from sage.all import *
>>> from sage.graphs.strongly_regular_db import SRG_120_63_30_36
>>> G =  SRG_120_63_30_36()
>>> G.is_strongly_regular(parameters=True)
(120, 63, 30, 36)
```

sage.graphs.strongly_regular_db.**SRG_120_77_52_44**(*immutable=False*)

Return a $(120, 77, 52, 44)$-strongly regular graph.

To build this graph, we first build a $2-(21, 7, 12)$ design, by removing two points from the `WittDesign()` on 23 points. We then build the intersection graph of blocks with intersection size 3.

EXAMPLES:

```
sage: from sage.graphs.strongly_regular_db import SRG_120_77_52_44
sage: G = SRG_120_77_52_44()                    # optional - gap_package_design
sage: G.is_strongly_regular(parameters=True)    # optional - gap_package_design
(120, 77, 52, 44)
```

```
>>> from sage.all import *
>>> from sage.graphs.strongly_regular_db import SRG_120_77_52_44
>>> G = SRG_120_77_52_44()                    # optional - gap_package_design
>>> G.is_strongly_regular(parameters=True)    # optional - gap_package_design
(120, 77, 52, 44)
```

sage.graphs.strongly_regular_db.**SRG_126_25_8_4**(*immutable=False*)

Return a $(126, 25, 8, 4)$-strongly regular graph.

It is the distance-(1 or 4) graph of *JohnsonGraph(9,4)*.

EXAMPLES:

```
sage: from sage.graphs.strongly_regular_db import SRG_126_25_8_4
sage: G =  SRG_126_25_8_4()
sage: G.is_strongly_regular(parameters=True)
(126, 25, 8, 4)
```

```
>>> from sage.all import *
>>> from sage.graphs.strongly_regular_db import SRG_126_25_8_4
>>> G =  SRG_126_25_8_4()
>>> G.is_strongly_regular(parameters=True)
(126, 25, 8, 4)
```

sage.graphs.strongly_regular_db.**SRG_126_50_13_24**(*immutable=False*)

Return a $(126, 50, 13, 24)$-strongly regular graph.

This graph is a subgraph of *SRG_175_72_20_36()*. This construction, due to Goethals, is given in §10B.(vii) of [BL1984].

EXAMPLES:

```
sage: from sage.graphs.strongly_regular_db import SRG_126_50_13_24
sage: G = SRG_126_50_13_24(); G
Goethals graph: Graph on 126 vertices
sage: G.is_strongly_regular(parameters=True)
(126, 50, 13, 24)
```

```
>>> from sage.all import *
>>> from sage.graphs.strongly_regular_db import SRG_126_50_13_24
>>> G = SRG_126_50_13_24(); G
Goethals graph: Graph on 126 vertices
>>> G.is_strongly_regular(parameters=True)
(126, 50, 13, 24)
```

sage.graphs.strongly_regular_db.**SRG_1288_792_476_504**(*immutable=False*)

Return a $(1288, 792, 476, 504)$-strongly regular graph.

This graph is built on the words of weight 12 in the binary `GolayCode`, obtained as `GolayCode(GF(2), False)`. Two of them are then made adjacent if their symmetric difference has weight 12 (cf [BE1992]).

See also

strongly_regular_from_two_weight_code() – build a strongly regular graph from a two-weight code.

EXAMPLES:

```
sage: from sage.graphs.strongly_regular_db import SRG_1288_792_476_504
sage: G = SRG_1288_792_476_504()                # long time                    #␣
↪needs sage.rings.finite_rings
sage: G.is_strongly_regular(parameters=True)    # long time                    #␣
↪needs sage.rings.finite_rings
(1288, 792, 476, 504)
```

```
>>> from sage.all import *
>>> from sage.graphs.strongly_regular_db import SRG_1288_792_476_504
>>> G = SRG_1288_792_476_504()                # long time                    #␣
↪needs sage.rings.finite_rings
>>> G.is_strongly_regular(parameters=True)    # long time                    #␣
↪needs sage.rings.finite_rings
(1288, 792, 476, 504)
```

sage.graphs.strongly_regular_db.**SRG_144_39_6_12**(*immutable=False*)

Return a $(144, 39, 6, 12)$-strongly regular graph.

This graph is obtained as an orbit of length 2808 on sets of cardinality 2 (among 2 such orbits) of the group $PGL_3(3)$ acting on the (right) cosets of a subgroup of order 39.

EXAMPLES:

```
sage: from sage.graphs.strongly_regular_db import SRG_144_39_6_12
sage: G = SRG_144_39_6_12()                                                 #␣
↪needs sage.libs.gap
sage: G.is_strongly_regular(parameters=True)                                #␣
↪needs sage.libs.gap
(144, 39, 6, 12)
```

```
>>> from sage.all import *
>>> from sage.graphs.strongly_regular_db import SRG_144_39_6_12
>>> G = SRG_144_39_6_12()                                                   #␣
```

(continues on next page)

(continued from previous page)

```
↪needs sage.libs.gap
>>> G.is_strongly_regular(parameters=True)                                    #␣
↪needs sage.libs.gap
(144, 39, 6, 12)
```

sage.graphs.strongly_regular_db.**SRG_175_72_20_36**(*immutable=False*)

Return a $(175, 72, 20, 36)$-strongly regular graph.

This graph is obtained from the line graph of `HoffmanSingletonGraph()`. Setting two vertices to be adjacent if their distance in the line graph is exactly 2 yields the graph. For more information, see 10.B.(iv) in [BL1984] and https://www.win.tue.nl/~aeb/graphs/McL.html.

EXAMPLES:

```
sage: from sage.graphs.strongly_regular_db import SRG_175_72_20_36
sage: G = SRG_175_72_20_36()
sage: G.is_strongly_regular(parameters=True)
(175, 72, 20, 36)
```

```
>>> from sage.all import *
>>> from sage.graphs.strongly_regular_db import SRG_175_72_20_36
>>> G = SRG_175_72_20_36()
>>> G.is_strongly_regular(parameters=True)
(175, 72, 20, 36)
```

sage.graphs.strongly_regular_db.**SRG_176_105_68_54**(*immutable=False*)

Return a $(176, 105, 68, 54)$-strongly regular graph.

To build this graph, we first build a $2-(22, 7, 16)$ design, by removing one point from the `WittDesign()` on 23 points. We then build the intersection graph of blocks with intersection size 3. Known as S.7 in [Hub1975].

EXAMPLES:

```
sage: from sage.graphs.strongly_regular_db import SRG_176_105_68_54
sage: G = SRG_176_105_68_54()                    # optional - gap_package_design
sage: G.is_strongly_regular(parameters=True)     # optional - gap_package_design
(176, 105, 68, 54)
```

```
>>> from sage.all import *
>>> from sage.graphs.strongly_regular_db import SRG_176_105_68_54
>>> G = SRG_176_105_68_54()                    # optional - gap_package_design
>>> G.is_strongly_regular(parameters=True)     # optional - gap_package_design
(176, 105, 68, 54)
```

sage.graphs.strongly_regular_db.**SRG_176_49_12_14**(*immutable=False*)

Return a $(176, 49, 12, 14)$-strongly regular graph.

This graph is built from the symmetric Higman-Sims design. In [Bro1982], it is explained that there exists an involution σ exchanging the points and blocks of the Higman-Sims design, such that each point is mapped on a block that contains it (i.e. σ is a 'polarity with all universal points'). The graph is then built by making two vertices u, v adjacent whenever $v \in \sigma(u)$.

EXAMPLES:

```
sage: from sage.graphs.strongly_regular_db import SRG_176_49_12_14
sage: G = SRG_176_49_12_14()                    # long time, optional - gap_
↪package_design
sage: G.is_strongly_regular(parameters=True)    # long time, optional - gap_
↪package_design
(176, 49, 12, 14)
```

```
>>> from sage.all import *
>>> from sage.graphs.strongly_regular_db import SRG_176_49_12_14
>>> G = SRG_176_49_12_14()                    # long time, optional - gap_package_
↪design
>>> G.is_strongly_regular(parameters=True)    # long time, optional - gap_package_
↪design
(176, 49, 12, 14)
```

sage.graphs.strongly_regular_db.**SRG_176_90_38_54**(*immutable=False*)

Return a $(176, 90, 38, 54)$-strongly regular graph.

This graph is obtained from *SRG_175_72_20_36()* by attaching a isolated vertex and doing Seidel switching with respect to disjoint union of 18 maximum cliques, following a construction by W.Haemers given in Sect.10.B.(vi) of [BL1984].

EXAMPLES:

```
sage: from sage.graphs.strongly_regular_db import SRG_176_90_38_54
sage: G = SRG_176_90_38_54(); G
a Seidel switching of Distance graph for distance 2 in : Graph on 176 vertices
sage: G.is_strongly_regular(parameters=True)
(176, 90, 38, 54)
```

```
>>> from sage.all import *
>>> from sage.graphs.strongly_regular_db import SRG_176_90_38_54
>>> G = SRG_176_90_38_54(); G
a Seidel switching of Distance graph for distance 2 in : Graph on 176 vertices
>>> G.is_strongly_regular(parameters=True)
(176, 90, 38, 54)
```

sage.graphs.strongly_regular_db.**SRG_196_91_42_42**(*immutable=False*)

Return a $(196, 91, 42, 42)$-strongly regular graph.

This strongly regular graph is built following the construction provided in Corollary 8.2.27 of [IS2006].

EXAMPLES:

```
sage: from sage.graphs.strongly_regular_db import SRG_196_91_42_42
sage: G = SRG_196_91_42_42()
sage: G.is_strongly_regular(parameters=True)
(196, 91, 42, 42)
```

```
>>> from sage.all import *
>>> from sage.graphs.strongly_regular_db import SRG_196_91_42_42
>>> G = SRG_196_91_42_42()
>>> G.is_strongly_regular(parameters=True)
(196, 91, 42, 42)
```

sage.graphs.strongly_regular_db.**SRG_210_99_48_45**(*immutable=False*)

Return a strongly regular graph with parameters $(210, 99, 48, 45)$.

This graph is from Example 4.2 in [KPRWZ2010]. One considers the action of the symmetric group S_7 on the 210 digraphs isomorphic to the disjoint union of K_1 and the circulant 6-vertex digraph `digraphs.Circulant(6, [1,4])`. It has 16 orbitals; the package [FK1991] found a megring of them, explicitly described in [KPRWZ2010], resulting in this graph.

EXAMPLES:

```
sage: from sage.graphs.strongly_regular_db import SRG_210_99_48_45
sage: g = SRG_210_99_48_45()                                                #␣
↪needs sage.libs.gap
sage: g.is_strongly_regular(parameters=True)                                #␣
↪needs sage.libs.gap
(210, 99, 48, 45)
```

```
>>> from sage.all import *
>>> from sage.graphs.strongly_regular_db import SRG_210_99_48_45
>>> g = SRG_210_99_48_45()                                                  #␣
↪needs sage.libs.gap
>>> g.is_strongly_regular(parameters=True)                                  #␣
↪needs sage.libs.gap
(210, 99, 48, 45)
```

sage.graphs.strongly_regular_db.**SRG_220_84_38_28**(*immutable=False*)

Return a $(220, 84, 38, 28)$-strongly regular graph.

This graph is obtained from the *intersection_graph()* of a `BIBD_45_9_8()`. This construction appears in VII.11.2 from [DesignHandbook]

EXAMPLES:

```
sage: from sage.graphs.strongly_regular_db import SRG_220_84_38_28
sage: g = SRG_220_84_38_28()
sage: g.is_strongly_regular(parameters=True)
(220, 84, 38, 28)
```

```
>>> from sage.all import *
>>> from sage.graphs.strongly_regular_db import SRG_220_84_38_28
>>> g = SRG_220_84_38_28()
>>> g.is_strongly_regular(parameters=True)
(220, 84, 38, 28)
```

sage.graphs.strongly_regular_db.**SRG_243_110_37_60**(*immutable=False*)

Return a $(243, 110, 37, 60)$-strongly regular graph.

Consider the orthogonal complement of the ternary `GolayCode`, obtained as `GolayCode(GF(3), False)`, which has 243 words. On them we define a graph, in which two words are adjacent whenever their Hamming distance is 9. This construction appears in [GS1975].

Note

A strongly regular graph with the same parameters is also obtained from the database of 2-weight codes.

EXAMPLES:

```
sage: from sage.graphs.strongly_regular_db import SRG_243_110_37_60
sage: G = SRG_243_110_37_60()                                                  #␣
↪needs sage.modules sage.rings.finite_rings
sage: G.is_strongly_regular(parameters=True)                                   #␣
↪needs sage.modules sage.rings.finite_rings
(243, 110, 37, 60)
```

```
>>> from sage.all import *
>>> from sage.graphs.strongly_regular_db import SRG_243_110_37_60
>>> G = SRG_243_110_37_60()                                                    #␣
↪needs sage.modules sage.rings.finite_rings
>>> G.is_strongly_regular(parameters=True)                                     #␣
↪needs sage.modules sage.rings.finite_rings
(243, 110, 37, 60)
```

sage.graphs.strongly_regular_db.**SRG_253_140_87_65**(*immutable=False*)

Return a $(253, 140, 87, 65)$-strongly regular graph.

To build this graph, we first build the `WittDesign()` on 23 points which is a $2-(23,7,21)$ design. We then build the intersection graph of blocks with intersection size 3. Known as S.6 in [Hub1975].

EXAMPLES:

```
sage: from sage.graphs.strongly_regular_db import SRG_253_140_87_65
sage: G = SRG_253_140_87_65()                    # optional - gap_package_design
sage: G.is_strongly_regular(parameters=True)     # optional - gap_package_design
(253, 140, 87, 65)
```

```
>>> from sage.all import *
>>> from sage.graphs.strongly_regular_db import SRG_253_140_87_65
>>> G = SRG_253_140_87_65()                    # optional - gap_package_design
>>> G.is_strongly_regular(parameters=True)     # optional - gap_package_design
(253, 140, 87, 65)
```

sage.graphs.strongly_regular_db.**SRG_276_140_58_84**(*immutable=False*)

Return a $(276, 140, 58, 84)$-strongly regular graph.

The graph is built from *McLaughlinGraph()*, with an added isolated vertex. We then perform a *seidel_switching()* on a set of 28 disjoint 5-cliques, which exist by cf. [HT1996].

EXAMPLES:

```
sage: from sage.graphs.strongly_regular_db import SRG_276_140_58_84
sage: g = SRG_276_140_58_84()                    # long time, optional - gap_
↪package_design
sage: g.is_strongly_regular(parameters=True)     # long time, optional - gap_
↪package_design
(276, 140, 58, 84)
```

```
>>> from sage.all import *
>>> from sage.graphs.strongly_regular_db import SRG_276_140_58_84
>>> g = SRG_276_140_58_84()                    # long time, optional - gap_package_
↪design
>>> g.is_strongly_regular(parameters=True)     # long time, optional - gap_package_
↪design
(276, 140, 58, 84)
```

sage.graphs.strongly_regular_db.**SRG_280_117_44_52**(*immutable=False*)

Return a strongly regular graph with parameters $(280, 117, 44, 52)$.

This graph is built according to a very pretty construction of Mathon and Rosa [MR1985]:

> The vertices of the graph G are all partitions of a set of 9 elements into $\{\{a, b, c\}, \{d, e, f\}, \{g, h, i\}\}$. The cross-intersection of two such partitions $P = \{P_1, P_2, P_3\}$ and $P' = \{P'_1, P'_2, P'_3\}$ being defined as $\{P_i \cap P'_j : 1 \leq i, j \leq 3\}$, two vertices of G are set to be adjacent if the cross-intersection of their respective partitions does not contain exactly 7 nonempty sets.

EXAMPLES:

```
sage: from sage.graphs.strongly_regular_db import SRG_280_117_44_52
sage: g=SRG_280_117_44_52()
sage: g.is_strongly_regular(parameters=True)
(280, 117, 44, 52)
```

```
>>> from sage.all import *
>>> from sage.graphs.strongly_regular_db import SRG_280_117_44_52
>>> g=SRG_280_117_44_52()
>>> g.is_strongly_regular(parameters=True)
(280, 117, 44, 52)
```

sage.graphs.strongly_regular_db.**SRG_280_135_70_60**(*immutable=False*)

Return a strongly regular graph with parameters $(280, 135, 70, 60)$.

This graph is built from the action of J_2 on the cosets of a $3.PGL(2, 9)$-subgroup.

EXAMPLES:

```
sage: from sage.graphs.strongly_regular_db import SRG_280_135_70_60
sage: g=SRG_280_135_70_60()                    # long time, optional - internet
sage: g.is_strongly_regular(parameters=True)   # long time, optional - internet
(280, 135, 70, 60)
```

```
>>> from sage.all import *
>>> from sage.graphs.strongly_regular_db import SRG_280_135_70_60
>>> g=SRG_280_135_70_60()                    # long time, optional - internet
>>> g.is_strongly_regular(parameters=True)   # long time, optional - internet
(280, 135, 70, 60)
```

sage.graphs.strongly_regular_db.**SRG_416_100_36_20**(*immutable=False*)

Return a $(416, 100, 36, 20)$-strongly regular graph.

This graph is obtained as an orbit on sets of cardinality 2 (among 2 that exists) of the group $G_2(4)$. This graph is isomorphic to the subgraph of the from *Suzuki Graph* induced on the neighbors of a vertex. Known as S.14 in [Hub1975].

EXAMPLES:

```
sage: from sage.graphs.strongly_regular_db import SRG_416_100_36_20
sage: g = SRG_416_100_36_20()                    # long time, optional - internet,␣
↪needs sage.libs.gap
sage: g.is_strongly_regular(parameters=True)     # long time, optional - internet,␣
↪needs sage.libs.gap
(416, 100, 36, 20)
```

```
>>> from sage.all import *
>>> from sage.graphs.strongly_regular_db import SRG_416_100_36_20
>>> g = SRG_416_100_36_20()                    # long time, optional - internet,␣
↪needs sage.libs.gap
>>> g.is_strongly_regular(parameters=True)     # long time, optional - internet,␣
↪needs sage.libs.gap
(416, 100, 36, 20)
```

sage.graphs.strongly_regular_db.**SRG_560_208_72_80**(*immutable=False*)

Return a $(560, 208, 72, 80)$-strongly regular graph.

This graph is obtained as the union of 4 orbits of sets of cardinality 2 (among the 13 that exist) of the group $Sz(8)$.

EXAMPLES:

```
sage: from sage.graphs.strongly_regular_db import SRG_560_208_72_80
sage: g = SRG_560_208_72_80()                    # not tested (~2s)              #␣
↪needs sage.libs.gap
sage: g.is_strongly_regular(parameters=True)     # not tested (~2s)              #␣
↪needs sage.libs.gap
(560, 208, 72, 80)
```

```
>>> from sage.all import *
>>> from sage.graphs.strongly_regular_db import SRG_560_208_72_80
>>> g = SRG_560_208_72_80()                    # not tested (~2s)              #␣
↪needs sage.libs.gap
>>> g.is_strongly_regular(parameters=True)     # not tested (~2s)              #␣
↪needs sage.libs.gap
(560, 208, 72, 80)
```

sage.graphs.strongly_regular_db.**SRG_630_85_20_10**(*immutable=False*)

Return a $(630, 85, 20, 10)$-strongly regular graph.

This graph is the line graph of $pg(5, 18, 2)$; its point graph is *SRG_175_72_20_36()*. One selects a subset of 630 maximum cliques in the latter following a construction by W.Haemers given in Sect.10.B.(v) of [BL1984].

EXAMPLES:

```
sage: from sage.graphs.strongly_regular_db import SRG_630_85_20_10
sage: G = SRG_630_85_20_10()                     # long time                     #␣
↪needs sage.groups
sage: G.is_strongly_regular(parameters=True)     # long time                     #␣
↪needs sage.groups
(630, 85, 20, 10)
```

```
>>> from sage.all import *
>>> from sage.graphs.strongly_regular_db import SRG_630_85_20_10
>>> G = SRG_630_85_20_10()                    # long time                    #␣
↪needs sage.groups
>>> G.is_strongly_regular(parameters=True)    # long time                    #␣
↪needs sage.groups
(630, 85, 20, 10)
```

sage.graphs.strongly_regular_db.**SRG_from_RSHCD**(*v*, *k*, *l*, *mu*, *existence=False*, *check=True*, *immutable=False*)

Return a (v, k, l, mu)-strongly regular graph from a RSHCD.

This construction appears in 8.D of [BL1984]. For more information, see `regular_symmetric_hadamard_matrix_with_constant_diagonal()`.

INPUT:

- `v`, `k`, `l`, `mu` – integers
- `existence` – boolean; whether to return a graph or to test if Sage can build such a graph
- `check` – boolean (default: `True`); whether to check that output is correct before returning it. As this is expected to be useless, you may want to disable it whenever you want speed.
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph. This parameter is only only when `existence` is `False`.

EXAMPLES:

some graphs

```
sage: from sage.graphs.strongly_regular_db import SRG_from_RSHCD
sage: SRG_from_RSHCD(784, 0, 14, 38, existence=True)                        #␣
↪needs sage.combinat sage.modules
False
sage: SRG_from_RSHCD(784, 377, 180, 182, existence=True)                    #␣
↪needs sage.combinat sage.modules
True
sage: SRG_from_RSHCD(144, 65, 28, 30)                                       #␣
↪needs sage.combinat sage.modules
Graph on 144 vertices
```

```
>>> from sage.all import *
>>> from sage.graphs.strongly_regular_db import SRG_from_RSHCD
>>> SRG_from_RSHCD(Integer(784), Integer(0), Integer(14), Integer(38),␣
↪existence=True)                           # needs sage.combinat sage.modules
False
>>> SRG_from_RSHCD(Integer(784), Integer(377), Integer(180), Integer(182),␣
↪existence=True)                           # needs sage.combinat sage.modules
True
>>> SRG_from_RSHCD(Integer(144), Integer(65), Integer(28), Integer(30))      ␣
↪                                  # needs sage.combinat sage.modules
Graph on 144 vertices
```

an example with vertex-transitive automorphism group, found during the implementation of the case $v = 324$

```
sage: # long time, needs sage.combinat sage.modules
sage: G = SRG_from_RSHCD(324,152,70,72)
sage: a = G.automorphism_group()
sage: a.order()
2592
sage: len(a.orbits())
1
```

```
>>> from sage.all import *
>>> # long time, needs sage.combinat sage.modules
>>> G = SRG_from_RSHCD(Integer(324),Integer(152),Integer(70),Integer(72))
>>> a = G.automorphism_group()
>>> a.order()
2592
>>> len(a.orbits())
1
```

sage.graphs.strongly_regular_db.**apparently_feasible_parameters**(*n*)

Return a list of a priori feasible parameters (v, k, λ, μ), with $0 < \mu < k$.

Note that some of those that it returns may also be infeasible for more involved reasons. The condition $0 < \mu < k$ makes sure we skip trivial cases of complete multipartite graphs and their complements.

INPUT:

- `n` – integer; return all a-priori feasible tuples (v, k, λ, μ) for $v < n$

EXAMPLES:

All sets of parameters with $v < 20$ which pass basic arithmetic tests are feasible:

```
sage: from sage.graphs.strongly_regular_db import apparently_feasible_parameters
sage: small_feasible = apparently_feasible_parameters(20); small_feasible
{(5, 2, 0, 1),
 (9, 4, 1, 2),
 (10, 3, 0, 1),
 (10, 6, 3, 4),
 (13, 6, 2, 3),
 (15, 6, 1, 3),
 (15, 8, 4, 4),
 (16, 5, 0, 2),
 (16, 6, 2, 2),
 (16, 9, 4, 6),
 (16, 10, 6, 6),
 (17, 8, 3, 4)}
sage: all(graphs.strongly_regular_graph(*x,existence=True) is True              #␣
↪needs sage.libs.pari
....:     for x in small_feasible)
True
```

```
>>> from sage.all import *
>>> from sage.graphs.strongly_regular_db import apparently_feasible_parameters
>>> small_feasible = apparently_feasible_parameters(Integer(20)); small_feasible
{(5, 2, 0, 1),
 (9, 4, 1, 2),
```

(continues on next page)

(continued from previous page)

```
 (10, 3, 0, 1),
 (10, 6, 3, 4),
 (13, 6, 2, 3),
 (15, 6, 1, 3),
 (15, 8, 4, 4),
 (16, 5, 0, 2),
 (16, 6, 2, 2),
 (16, 9, 4, 6),
 (16, 10, 6, 6),
 (17, 8, 3, 4)}
>>> all(graphs.strongly_regular_graph(*x,existence=True) is True              #␣
↪needs sage.libs.pari
...     for x in small_feasible)
True
```

But that becomes wrong for $v < 60$ (because of the non-existence of a $(49, 16, 3, 6)$-strongly regular graph):

```
sage: small_feasible = apparently_feasible_parameters(60)
sage: all(graphs.strongly_regular_graph(*x,existence=True) is True              #␣
↪needs sage.libs.pari
....:     for x in small_feasible)
False
```

```
>>> from sage.all import *
>>> small_feasible = apparently_feasible_parameters(Integer(60))
>>> all(graphs.strongly_regular_graph(*x,existence=True) is True              #␣
↪needs sage.libs.pari
...     for x in small_feasible)
False
```

sage.graphs.strongly_regular_db.**eigenmatrix**(*v*, *k*, *l*, *mu*)

Return the first eigenmatrix of a (v, k, l, mu)-strongly regular graph.

The adjacency matrix A of an s.r.g. commutes with the adjacency matrix $A' = J - A - I$ of its complement (here J is all-1 matrix, and I the identity matrix). Thus, they can be simultaneously diagonalized and so A and A' share eigenspaces.

The eigenvalues of J are v with multiplicity 1, and 0 with multiplicity $v-1$. Thus the eigenvalue of A' corresponding to the 1-dimension k-eigenspace of A is $v-k-1$. Respectively, the eigenvalues of A' corresponding to t-eigenspace of A, with t unequal to k, equals $-t-1$. The 1st eigenmatrix P of the C-algebra $C[A]$ generated by A encodes this eigenvalue information in its three columns; the 2nd (resp. 3rd) column contains distinct eigenvalues of A (resp. of A'), and the 1st column contains the corresponding eigenvalues of I. The matrix vP^{-1} is called the 2nd eigenvalue matrix of $C[A]$.

The most interesting feature of vP^{-1} is that it is the 1st eigenmatrix of the dual of $C[A]$ if the dual is generated by the adjacency matrix of a strongly regular graph. See [BH2012] and [BI1984] for details.

If the set of parameters is not feasible, or if they correspond to a conference graph, the function returns `None`. Its output is stable, assuming that the eigenvalues r,s used satisfy r>s; this holds for the current implementation of eigenvalues().

INPUT:

- `v`, `k`, `l`, `mu` – integers

EXAMPLES:

Petersen's graph's C-algebra does not have a dual coming from an s.r.g.:

```
sage: from sage.graphs.strongly_regular_db import eigenmatrix
sage: P = eigenmatrix(10,3,0,1); P                                              #␣
↪needs sage.modules
[ 1  3  6]
[ 1  1 -2]
[ 1 -2  1]
sage: 10*P^-1                                                                   #␣
↪needs sage.modules
[   1    5    4]
[   1  5/3 -8/3]
[   1 -5/3  2/3]
```

```
>>> from sage.all import *
>>> from sage.graphs.strongly_regular_db import eigenmatrix
>>> P = eigenmatrix(Integer(10),Integer(3),Integer(0),Integer(1)); P           ␣
↪                                  # needs sage.modules
[ 1  3  6]
[ 1  1 -2]
[ 1 -2  1]
>>> Integer(10)*P**-Integer(1)                                                 ␣
↪                # needs sage.modules
[   1    5    4]
[   1  5/3 -8/3]
[   1 -5/3  2/3]
```

The line graph of $K_{3,3}$ is self-dual:

```
sage: P = eigenmatrix(9,4,1,2); P                                               #␣
↪needs sage.modules
[ 1  4  4]
[ 1  1 -2]
[ 1 -2  1]
sage: 9*P^-1                                                                    #␣
↪needs sage.modules
[ 1  4  4]
[ 1  1 -2]
[ 1 -2  1]
```

```
>>> from sage.all import *
>>> P = eigenmatrix(Integer(9),Integer(4),Integer(1),Integer(2)); P           ␣
↪                                  # needs sage.modules
[ 1  4  4]
[ 1  1 -2]
[ 1 -2  1]
>>> Integer(9)*P**-Integer(1)                                                  ␣
↪                # needs sage.modules
[ 1  4  4]
[ 1  1 -2]
[ 1 -2  1]
```

A strongly regular graph with a non-isomorphic dual coming from another strongly regular graph:

```
sage: graphs.strongly_regular_graph(243,220,199,200, existence=True)              #␣
↪needs sage.combinat
True
sage: graphs.strongly_regular_graph(243,110,37,60, existence=True)               #␣
↪needs sage.combinat
True
sage: P = eigenmatrix(243,220,199,200); P
[  1 220  22]
[  1   4  -5]
[  1  -5   4]
sage: 243*P^-1
[  1 110 132]
[  1   2  -3]
[  1 -25  24]
sage: 243*P^-1==eigenmatrix(243,110,37,60)
True
```

```
>>> from sage.all import *
>>> graphs.strongly_regular_graph(Integer(243),Integer(220),Integer(199),
↪Integer(200), existence=True)            # needs sage.combinat
True
>>> graphs.strongly_regular_graph(Integer(243),Integer(110),Integer(37),
↪Integer(60), existence=True)              # needs sage.combinat
True
>>> P = eigenmatrix(Integer(243),Integer(220),Integer(199),Integer(200)); P
[  1 220  22]
[  1   4  -5]
[  1  -5   4]
>>> Integer(243)*P**-Integer(1)
[  1 110 132]
[  1   2  -3]
[  1 -25  24]
>>> Integer(243)*P**-Integer(1)==eigenmatrix(Integer(243),Integer(110),
↪Integer(37),Integer(60))
True
```

sage.graphs.strongly_regular_db.**is_GQqmqp**(k, l, mu)

Test whether some $GQ(q-1,q+1)$ or $GQ(q+1,q-1)$-graph is (v,k,λ,μ)-srg.

INPUT:

- `v`, `k`, `l`, `mu` – integers

OUTPUT:

A tuple `t` such that `t[0](*t[1:])` builds the requested graph if one exists, and `None` otherwise.

EXAMPLES:

```
sage: from sage.graphs.strongly_regular_db import is_GQqmqp
sage: t = is_GQqmqp(27,10,1,5); t
(<function AhrensSzekeresGeneralizedQuadrangleGraph at ...>, 3, False)
sage: g = t[0](*t[1:]); g
AS(3); GQ(2, 4): Graph on 27 vertices
sage: t = is_GQqmqp(45,12,3,3); t
```

(continues on next page)

(continued from previous page)

```
(<function AhrensSzekeresGeneralizedQuadrangleGraph at ...>, 3, True)
sage: g = t[0](*t[1:]); g
AS(3)*; GQ(4, 2): Graph on 45 vertices
sage: g.is_strongly_regular(parameters=True)
(45, 12, 3, 3)
sage: t = is_GQqmqp(16,6,2,2); t
(<function T2starGeneralizedQuadrangleGraph at ...>, 2, True)
sage: g = t[0](*t[1:]); g
T2*(O,2)*; GQ(3, 1): Graph on 16 vertices
sage: g.is_strongly_regular(parameters=True)
(16, 6, 2, 2)
sage: t = is_GQqmqp(64,18,2,6); t
(<function T2starGeneralizedQuadrangleGraph at ...>, 4, False)
sage: g = t[0](*t[1:]); g
T2*(O,4); GQ(3, 5): Graph on 64 vertices
sage: g.is_strongly_regular(parameters=True)
(64, 18, 2, 6)
```

```
>>> from sage.all import *
>>> from sage.graphs.strongly_regular_db import is_GQqmqp
>>> t = is_GQqmqp(Integer(27),Integer(10),Integer(1),Integer(5)); t
(<function AhrensSzekeresGeneralizedQuadrangleGraph at ...>, 3, False)
>>> g = t[Integer(0)](*t[Integer(1):]); g
AS(3); GQ(2, 4): Graph on 27 vertices
>>> t = is_GQqmqp(Integer(45),Integer(12),Integer(3),Integer(3)); t
(<function AhrensSzekeresGeneralizedQuadrangleGraph at ...>, 3, True)
>>> g = t[Integer(0)](*t[Integer(1):]); g
AS(3)*; GQ(4, 2): Graph on 45 vertices
>>> g.is_strongly_regular(parameters=True)
(45, 12, 3, 3)
>>> t = is_GQqmqp(Integer(16),Integer(6),Integer(2),Integer(2)); t
(<function T2starGeneralizedQuadrangleGraph at ...>, 2, True)
>>> g = t[Integer(0)](*t[Integer(1):]); g
T2*(O,2)*; GQ(3, 1): Graph on 16 vertices
>>> g.is_strongly_regular(parameters=True)
(16, 6, 2, 2)
>>> t = is_GQqmqp(Integer(64),Integer(18),Integer(2),Integer(6)); t
(<function T2starGeneralizedQuadrangleGraph at ...>, 4, False)
>>> g = t[Integer(0)](*t[Integer(1):]); g
T2*(O,4); GQ(3, 5): Graph on 64 vertices
>>> g.is_strongly_regular(parameters=True)
(64, 18, 2, 6)
```

sage.graphs.strongly_regular_db.**is_NO_F2**(k, l, *mu*)

Test whether some NO^e,perp(2n,2) graph is (v, k, λ, μ)-strongly regular.

For more information, see *sage.graphs.graph_generators.GraphGenerators.NonisotropicOrthogonalPolarGraph()*.

INPUT:

- v, k, l, mu – integers

OUTPUT:

A tuple `t` such that `t[0](*t[1:])` builds the requested graph if one exists, and `None` otherwise.

EXAMPLES:

```
sage: from sage.graphs.strongly_regular_db import is_NO_F2
sage: t = is_NO_F2(10, 3, 0, 1); t                                          #␣
↪needs sage.libs.pari
(<function NonisotropicOrthogonalPolarGraph at ...>, 4, 2, '-')
sage: g = t[0](*t[1:]); g                                                   #␣
↪needs sage.libs.pari
NO^-(4, 2): Graph on 10 vertices
sage: g.is_strongly_regular(parameters=True)                                #␣
↪needs sage.libs.pari
(10, 3, 0, 1)
```

```
>>> from sage.all import *
>>> from sage.graphs.strongly_regular_db import is_NO_F2
>>> t = is_NO_F2(Integer(10), Integer(3), Integer(0), Integer(1)); t          ␣
↪                                  # needs sage.libs.pari
(<function NonisotropicOrthogonalPolarGraph at ...>, 4, 2, '-')
>>> g = t[Integer(0)](*t[Integer(1):]); g                                    ␣
↪              # needs sage.libs.pari
NO^-(4, 2): Graph on 10 vertices
>>> g.is_strongly_regular(parameters=True)                                  #␣
↪needs sage.libs.pari
(10, 3, 0, 1)
```

`sage.graphs.strongly_regular_db.`**`is_NO_F3`**`(`k, l, *mu*`)`

Test whether some NO^e,perp(2n,3) graph is (v, k, λ, μ)-strongly regular.

For more information, see *sage.graphs.graph_generators.GraphGenerators.NonisotropicOrthogonalPolarGraph()*.

INPUT:

- `v`, `k`, `l`, `mu` – integers

OUTPUT:

A tuple `t` such that `t[0](*t[1:])` builds the requested graph if one exists, and `None` otherwise.

EXAMPLES:

```
sage: from sage.graphs.strongly_regular_db import is_NO_F3
sage: t = is_NO_F3(15, 6, 1, 3); t                                          #␣
↪needs sage.libs.pari
(<function NonisotropicOrthogonalPolarGraph at ...>, 4, 3, '-')
sage: g = t[0](*t[1:]); g                                                   #␣
↪needs sage.libs.pari
NO^-(4, 3): Graph on 15 vertices
sage: g.is_strongly_regular(parameters=True)                                #␣
↪needs sage.libs.pari
(15, 6, 1, 3)
```

```
>>> from sage.all import *
>>> from sage.graphs.strongly_regular_db import is_NO_F3
```

(continues on next page)

(continued from previous page)

```
>>> t = is_NO_F3(Integer(15), Integer(6), Integer(1), Integer(3)); t
↪                                  # needs sage.libs.pari
(<function NonisotropicOrthogonalPolarGraph at ...>, 4, 3, '-')
>>> g = t[Integer(0)](*t[Integer(1):]); g
↪               # needs sage.libs.pari
NO^-(4, 3): Graph on 15 vertices
>>> g.is_strongly_regular(parameters=True)
↪needs sage.libs.pari
(15, 6, 1, 3)
```

sage.graphs.strongly_regular_db.**is_NOodd**(k, l, mu)

Test whether some NO^e(2n+1,q) graph is (v, k, λ, μ)-strongly regular.

Here $q > 2$, for in the case $q = 2$ this graph is complete. For more information, see *sage.graphs.graph_generators.GraphGenerators.NonisotropicOrthogonalPolarGraph()* and Sect. 7.C of [BL1984].

INPUT:

- `v`, `k`, `l`, `mu` – integers

OUTPUT:

A tuple `t` such that `t[0](*t[1:])` builds the requested graph if one exists, and `None` otherwise.

EXAMPLES:

```
sage: from sage.graphs.strongly_regular_db import is_NOodd
sage: t = is_NOodd(120, 51, 18, 24); t
↪needs sage.libs.pari
(<function NonisotropicOrthogonalPolarGraph at ...>, 5, 4, '-')
sage: g = t[0](*t[1:]); g
↪needs sage.libs.pari
NO^-(5, 4): Graph on 120 vertices
sage: g.is_strongly_regular(parameters=True)
↪needs sage.libs.pari
(120, 51, 18, 24)
```

```
>>> from sage.all import *
>>> from sage.graphs.strongly_regular_db import is_NOodd
>>> t = is_NOodd(Integer(120), Integer(51), Integer(18), Integer(24)); t
↪                                  # needs sage.libs.pari
(<function NonisotropicOrthogonalPolarGraph at ...>, 5, 4, '-')
>>> g = t[Integer(0)](*t[Integer(1):]); g
↪               # needs sage.libs.pari
NO^-(5, 4): Graph on 120 vertices
>>> g.is_strongly_regular(parameters=True)
↪needs sage.libs.pari
(120, 51, 18, 24)
```

sage.graphs.strongly_regular_db.**is_NOperp_F5**(k, l, mu)

Test whether some NO^e,perp(2n+1,5) graph is (v, k, λ, μ)-strongly regular.

For more information, see *sage.graphs.graph_generators.GraphGenerators.NonisotropicOrthogonalPolarGraph()* and Sect. 7.D of [BL1984].

INPUT:

- v, k, l, mu – integers

OUTPUT:

A tuple t such that t[0](*t[1:]) builds the requested graph if one exists, and None otherwise.

EXAMPLES:

```
sage: from sage.graphs.strongly_regular_db import is_NOperp_F5
sage: t = is_NOperp_F5(10, 3, 0, 1); t                                          #␣
↪needs sage.libs.pari
(<function NonisotropicOrthogonalPolarGraph at ...>, 3, 5, '-', 1)
sage: g = t[0](*t[1:]); g                                                       #␣
↪needs sage.libs.pari
NO^-,perp(3, 5): Graph on 10 vertices
sage: g.is_strongly_regular(parameters=True)                                    #␣
↪needs sage.libs.pari
(10, 3, 0, 1)
```

```
>>> from sage.all import *
>>> from sage.graphs.strongly_regular_db import is_NOperp_F5
>>> t = is_NOperp_F5(Integer(10), Integer(3), Integer(0), Integer(1)); t       ␣
↪                                 # needs sage.libs.pari
(<function NonisotropicOrthogonalPolarGraph at ...>, 3, 5, '-', 1)
>>> g = t[Integer(0)](*t[Integer(1):]); g                                       ␣
↪              # needs sage.libs.pari
NO^-,perp(3, 5): Graph on 10 vertices
>>> g.is_strongly_regular(parameters=True)                                    #␣
↪needs sage.libs.pari
(10, 3, 0, 1)
```

sage.graphs.strongly_regular_db.**is_NU**(k, l, mu)

Test whether some NU(n,q)-graph, is (v, k, λ, μ)-strongly regular.

Note that n>2; for n=2 there is no s.r.g. For more information, see *sage.graphs.graph_generators.GraphGenerators.NonisotropicUnitaryPolarGraph()* and series C14 in [Hub1975].

INPUT:

- v, k, l, mu – integers

OUTPUT:

A tuple t such that t[0](*t[1:]) builds the requested graph if one exists, and None otherwise.

EXAMPLES:

```
sage: from sage.graphs.strongly_regular_db import is_NU
sage: t = is_NU(40, 27, 18, 18); t                                              #␣
↪needs sage.libs.pari
(<function NonisotropicUnitaryPolarGraph at ...>, 4, 2)
sage: g = t[0](*t[1:]); g                                                       #␣
↪needs sage.libs.pari
NU(4, 2): Graph on 40 vertices
sage: g.is_strongly_regular(parameters=True)                                    #␣
```

(continues on next page)

(continued from previous page)

```
↪needs sage.libs.pari
(40, 27, 18, 18)
```

```
>>> from sage.all import *
>>> from sage.graphs.strongly_regular_db import is_NU
>>> t = is_NU(Integer(40), Integer(27), Integer(18), Integer(18)); t
↪                                  # needs sage.libs.pari
(<function NonisotropicUnitaryPolarGraph at ...>, 4, 2)
>>> g = t[Integer(0)](*t[Integer(1):]); g
↪               # needs sage.libs.pari
NU(4, 2): Graph on 40 vertices
>>> g.is_strongly_regular(parameters=True)
↪needs sage.libs.pari
(40, 27, 18, 18)
```

sage.graphs.strongly_regular_db.**is_RSHCD**(*v*, *k*, *l*, *mu*)

Test whether some RSHCD graph is (v, k, λ, μ)-strongly regular.

For more information, see *SRG_from_RSHCD()*.

INPUT:

- v, k, l, mu – integers

OUTPUT:

A tuple t such that t[0](*t[1:]) builds the requested graph if one exists, and None otherwise.

EXAMPLES:

```
sage: from sage.graphs.strongly_regular_db import is_RSHCD
sage: t = is_RSHCD(64,27,10,12); t
↪needs sage.combinat sage.modules
(<cyfunction SRG_from_RSHCD at ...>, 64, 27, 10, 12)
sage: g = t[0](*t[1:]); g
↪needs sage.combinat sage.modules
Graph on 64 vertices
sage: g.is_strongly_regular(parameters=True)
↪needs sage.combinat sage.modules
(64, 27, 10, 12)
```

```
>>> from sage.all import *
>>> from sage.graphs.strongly_regular_db import is_RSHCD
>>> t = is_RSHCD(Integer(64),Integer(27),Integer(10),Integer(12)); t
↪                                  # needs sage.combinat sage.modules
(<cyfunction SRG_from_RSHCD at ...>, 64, 27, 10, 12)
>>> g = t[Integer(0)](*t[Integer(1):]); g
↪               # needs sage.combinat sage.modules
Graph on 64 vertices
>>> g.is_strongly_regular(parameters=True)
↪needs sage.combinat sage.modules
(64, 27, 10, 12)
```

sage.graphs.strongly_regular_db.**is_affine_polar**(*k*, *l*, *mu*)

Test whether some Affine Polar graph is (v, k, λ, μ)-strongly regular.

For more information, see https://www.win.tue.nl/~aeb/graphs/VO.html.

INPUT:

- v, k, l, mu – integers

OUTPUT:

A tuple t such that t[0](*t[1:]) builds the requested graph if one exists, and None otherwise.

EXAMPLES:

```
sage: from sage.graphs.strongly_regular_db import is_affine_polar
sage: t = is_affine_polar(81,32,13,12); t                                   #␣
↪needs sage.rings.finite_rings
(..., 4, 3, '+')
sage: g = t[0](*t[1:]); g                                                   #␣
↪needs sage.rings.finite_rings
Affine Polar Graph VO^+(4,3): Graph on 81 vertices
sage: g.is_strongly_regular(parameters=True)                                #␣
↪needs sage.rings.finite_rings
(81, 32, 13, 12)

sage: t = is_affine_polar(5,5,5,5); t
```

```
>>> from sage.all import *
>>> from sage.graphs.strongly_regular_db import is_affine_polar
>>> t = is_affine_polar(Integer(81),Integer(32),Integer(13),Integer(12)); t      ␣
↪                                  # needs sage.rings.finite_rings
(..., 4, 3, '+')
>>> g = t[Integer(0)](*t[Integer(1):]); g                                        ␣
↪              # needs sage.rings.finite_rings
Affine Polar Graph VO^+(4,3): Graph on 81 vertices
>>> g.is_strongly_regular(parameters=True)                                    #␣
↪needs sage.rings.finite_rings
(81, 32, 13, 12)

>>> t = is_affine_polar(Integer(5),Integer(5),Integer(5),Integer(5)); t
```

sage.graphs.strongly_regular_db.**is_complete_multipartite**(*k*, *l*, *mu*)

Test whether some complete multipartite graph is (v, k, λ, μ)-strongly regular.

Any complete multipartite graph with parts of the same size is strongly regular.

INPUT:

- v, k, l, mu – integers

OUTPUT:

A tuple t such that t[0](*t[1:]) builds the requested graph if one exists, and None otherwise.

EXAMPLES:

```
sage: from sage.graphs.strongly_regular_db import is_complete_multipartite
sage: t = is_complete_multipartite(12,8,4,8); t
(<...CompleteMultipartiteSRG...>,
 3,
 4)
```

(continues on next page)

(continued from previous page)

```
sage: g = t[0](*t[1:]); g
Multipartite Graph with set sizes [4, 4, 4]: Graph on 12 vertices
sage: g.is_strongly_regular(parameters=True)
(12, 8, 4, 8)
```

```
>>> from sage.all import *
>>> from sage.graphs.strongly_regular_db import is_complete_multipartite
>>> t = is_complete_multipartite(Integer(12),Integer(8),Integer(4),Integer(8)); t
(<...CompleteMultipartiteSRG...>,
 3,
 4)
>>> g = t[Integer(0)](*t[Integer(1):]); g
Multipartite Graph with set sizes [4, 4, 4]: Graph on 12 vertices
>>> g.is_strongly_regular(parameters=True)
(12, 8, 4, 8)
```

sage.graphs.strongly_regular_db.**is_cossidente_penttila**(*k*, *l*, *mu*)

Test whether some CossidentePenttilaGraph graph is (v, k, λ, μ)-strongly regular.

For more information, see *CossidentePenttilaGraph()*.

INPUT:

- v, k, l, mu – integers

OUTPUT:

A tuple t such that t[0](*t[1:]) builds the requested graph if one exists, and None otherwise.

EXAMPLES:

```
sage: from sage.graphs.strongly_regular_db import is_cossidente_penttila
sage: t =  is_cossidente_penttila(378, 52, 1, 8); t                             #␣
↪needs sage.libs.pari
(<function CossidentePenttilaGraph at ...>, 5)
sage: g = t[0](*t[1:]); g                       # optional - gap_package_design,␣
↪needs sage.libs.pari
CossidentePenttila(5): Graph on 378 vertices
sage: g.is_strongly_regular(parameters=True)    # optional - gap_package_design,␣
↪needs sage.libs.pari
(378, 52, 1, 8)
```

```
>>> from sage.all import *
>>> from sage.graphs.strongly_regular_db import is_cossidente_penttila
>>> t =  is_cossidente_penttila(Integer(378), Integer(52), Integer(1),␣
↪Integer(8)); t                                 # needs sage.libs.pari
(<function CossidentePenttilaGraph at ...>, 5)
>>> g = t[Integer(0)](*t[Integer(1):]); g                       # optional - gap_
↪package_design, needs sage.libs.pari
CossidentePenttila(5): Graph on 378 vertices
>>> g.is_strongly_regular(parameters=True)    # optional - gap_package_design,␣
↪needs sage.libs.pari
(378, 52, 1, 8)
```

sage.graphs.strongly_regular_db.**is_goethals_seidel**(k, l, mu)

Test whether some *GoethalsSeidelGraph()* graph is (v, k, λ, μ)-strongly regular.

INPUT:

- v, k, l, mu – integers

OUTPUT:

A tuple t such that t[0](*t[1:]) builds the requested graph if one exists, and None otherwise.

EXAMPLES:

```
sage: from sage.graphs.strongly_regular_db import is_goethals_seidel
sage: t = is_goethals_seidel(28, 15, 6, 10); t                                  #␣
↪needs sage.combinat sage.modules
(<function GoethalsSeidelGraph at ...>, 3, 3)
sage: g = t[0](*t[1:]); g                                                       #␣
↪needs sage.combinat sage.modules
Graph on 28 vertices
sage: g.is_strongly_regular(parameters=True)                                    #␣
↪needs sage.combinat sage.modules
(28, 15, 6, 10)

sage: t = is_goethals_seidel(256, 135, 70, 72); t                               #␣
↪needs sage.combinat sage.modules
(<function GoethalsSeidelGraph at ...>, 2, 15)
sage: g = t[0](*t[1:]); g                                                       #␣
↪needs sage.combinat sage.modules
Graph on 256 vertices
sage: g.is_strongly_regular(parameters=True)                                    #␣
↪needs sage.combinat sage.modules
(256, 135, 70, 72)

sage: t = is_goethals_seidel(5,5,5,5); t                                        #␣
↪needs sage.combinat sage.modules
```

```
>>> from sage.all import *
>>> from sage.graphs.strongly_regular_db import is_goethals_seidel
>>> t = is_goethals_seidel(Integer(28), Integer(15), Integer(6), Integer(10)); t ␣
↪                                  # needs sage.combinat sage.modules
(<function GoethalsSeidelGraph at ...>, 3, 3)
>>> g = t[Integer(0)](*t[Integer(1):]); g                                        ␣
↪                # needs sage.combinat sage.modules
Graph on 28 vertices
>>> g.is_strongly_regular(parameters=True)                                      #␣
↪needs sage.combinat sage.modules
(28, 15, 6, 10)

>>> t = is_goethals_seidel(Integer(256), Integer(135), Integer(70), Integer(72));␣
↪t                                 # needs sage.combinat sage.modules
(<function GoethalsSeidelGraph at ...>, 2, 15)
>>> g = t[Integer(0)](*t[Integer(1):]); g                                        ␣
↪                # needs sage.combinat sage.modules
Graph on 256 vertices
```

(continues on next page)

(continued from previous page)

```
>>> g.is_strongly_regular(parameters=True)                                    #␣
↪needs sage.combinat sage.modules
(256, 135, 70, 72)

>>> t = is_goethals_seidel(Integer(5),Integer(5),Integer(5),Integer(5)); t       ␣
↪                                  # needs sage.combinat sage.modules
```

sage.graphs.strongly_regular_db.**is_haemers**(*k*, *l*, *mu*)

Test whether some HaemersGraph graph is (v, k, λ, μ)-strongly regular.

For more information, see *HaemersGraph()*.

INPUT:

- v, k, l, mu – integers

OUTPUT:

A tuple t such that t[0](*t[1:]) builds the requested graph if one exists, and None otherwise.

EXAMPLES:

```
sage: from sage.graphs.strongly_regular_db import is_haemers
sage: t = is_haemers(96, 19, 2, 4); t                                        #␣
↪needs sage.libs.pari
(<function HaemersGraph at ...>, 4)
sage: g = t[0](*t[1:]); g                                                    #␣
↪needs sage.libs.pari
Haemers(4): Graph on 96 vertices
sage: g.is_strongly_regular(parameters=True)                                 #␣
↪needs sage.libs.pari
(96, 19, 2, 4)
```

```
>>> from sage.all import *
>>> from sage.graphs.strongly_regular_db import is_haemers
>>> t = is_haemers(Integer(96), Integer(19), Integer(2), Integer(4)); t          ␣
↪                                  # needs sage.libs.pari
(<function HaemersGraph at ...>, 4)
>>> g = t[Integer(0)](*t[Integer(1):]); g                                        ␣
↪              # needs sage.libs.pari
Haemers(4): Graph on 96 vertices
>>> g.is_strongly_regular(parameters=True)                                    #␣
↪needs sage.libs.pari
(96, 19, 2, 4)
```

sage.graphs.strongly_regular_db.**is_johnson**(*k*, *l*, *mu*)

Test whether some Johnson graph is (v, k, λ, μ)-strongly regular.

INPUT:

- v, k, l, mu – integers

OUTPUT:

A tuple t such that t[0](*t[1:]) builds the requested graph if one exists, and None otherwise.

EXAMPLES:

```
sage: from sage.graphs.strongly_regular_db import is_johnson
sage: t = is_johnson(10,6,3,4); t
(..., 5, 2)
sage: g = t[0](*t[1:]); g
Johnson graph with parameters 5,2: Graph on 10 vertices
sage: g.is_strongly_regular(parameters=True)
(10, 6, 3, 4)

sage: t = is_johnson(5,5,5,5); t
```

```
>>> from sage.all import *
>>> from sage.graphs.strongly_regular_db import is_johnson
>>> t = is_johnson(Integer(10),Integer(6),Integer(3),Integer(4)); t
(..., 5, 2)
>>> g = t[Integer(0)](*t[Integer(1):]); g
Johnson graph with parameters 5,2: Graph on 10 vertices
>>> g.is_strongly_regular(parameters=True)
(10, 6, 3, 4)

>>> t = is_johnson(Integer(5),Integer(5),Integer(5),Integer(5)); t
```

sage.graphs.strongly_regular_db.**is_mathon_PC_srg**(k, l, mu)

Test whether some Mathon's Pseudocyclic s.r.g. is (v, k, λ, μ)-strongly regular.

INPUT:

- `v`, `k`, `l`, `mu` – integers

OUTPUT:

A tuple `t` such that `t[0](*t[1:])` builds the requested graph if one exists, and `None` otherwise.

Todo

The current implementation only gives a subset of all possible graphs that can be obtained using this construction. A full implementation should rely on a database of conference matrices (or, equivalently, on a database of s.r.g.'s with parameters $(4t+1, 2t, t-1, t)$. Currently we make an extra assumption that $4t+1$ is a prime power. The first case where we miss a construction is $t = 11$, where we could (recursively) use the graph for $t = 1$ to construct a graph on 83205 vertices.

EXAMPLES:

```
sage: from sage.graphs.strongly_regular_db import is_mathon_PC_srg
sage: t = is_mathon_PC_srg(45,22,10,11); t                                  #␣
↪needs sage.libs.pari
(..., 1)
sage: g = t[0](*t[1:]); g                                                   #␣
↪needs sage.libs.pari
Mathon's PC SRG on 45 vertices: Graph on 45 vertices
sage: g.is_strongly_regular(parameters=True)                                #␣
↪needs sage.libs.pari
(45, 22, 10, 11)
```

```
>>> from sage.all import *
>>> from sage.graphs.strongly_regular_db import is_mathon_PC_srg
>>> t = is_mathon_PC_srg(Integer(45),Integer(22),Integer(10),Integer(11)); t
↪                                  # needs sage.libs.pari
(..., 1)
>>> g = t[Integer(0)](*t[Integer(1):]); g
↪              # needs sage.libs.pari
Mathon's PC SRG on 45 vertices: Graph on 45 vertices
>>> g.is_strongly_regular(parameters=True)
↪needs sage.libs.pari
(45, 22, 10, 11)
```

sage.graphs.strongly_regular_db.**is_muzychuk_S6**(*k*, *l*, *mu*)

Test whether some Muzychuk S6 graph is (v, k, l, mu)-strongly regular.

Tests whether a *MuzychukS6Graph()* has parameters (v, k, l, mu).

INPUT:

- v, k, l, mu – integers

OUTPUT:

A tuple t such that t[0](*t[1:]) builds the required graph if it exists, and None otherwise.

EXAMPLES:

```
sage: from sage.graphs.strongly_regular_db import is_muzychuk_S6
sage: t = is_muzychuk_S6(378, 116, 34, 36)
sage: G = t[0](*t[1:]); G
Muzychuk S6 graph with parameters (3,3): Graph on 378 vertices
sage: G.is_strongly_regular(parameters=True)
(378, 116, 34, 36)
sage: t = is_muzychuk_S6(5, 5, 5, 5); t
```

```
>>> from sage.all import *
>>> from sage.graphs.strongly_regular_db import is_muzychuk_S6
>>> t = is_muzychuk_S6(Integer(378), Integer(116), Integer(34), Integer(36))
>>> G = t[Integer(0)](*t[Integer(1):]); G
Muzychuk S6 graph with parameters (3,3): Graph on 378 vertices
>>> G.is_strongly_regular(parameters=True)
(378, 116, 34, 36)
>>> t = is_muzychuk_S6(Integer(5), Integer(5), Integer(5), Integer(5)); t
```

sage.graphs.strongly_regular_db.**is_nowhere0_twoweight**(*v*, *k*, *l*, *mu*)

Test whether some graph of nowhere 0 words is (v, k, λ, μ)-strongly regular.

Test whether a *Nowhere0WordsTwoWeightCodeGraph()* is (v, k, λ, μ)-strongly regular.

INPUT:

- v, k, l, mu – integers

OUTPUT:

A tuple t such that t[0](*t[1:]) builds the requested graph if the parameters match, and None otherwise.

EXAMPLES:

```
sage: graphs.strongly_regular_graph(196, 60, 14, 20)                    #␣
↪needs sage.combinat sage.modules
Nowhere0WordsTwoWeightCodeGraph(8): Graph on 196 vertices
```

```
>>> from sage.all import *
>>> graphs.strongly_regular_graph(Integer(196), Integer(60), Integer(14),␣
↪Integer(20))                              # needs sage.combinat sage.modules
Nowhere0WordsTwoWeightCodeGraph(8): Graph on 196 vertices
```

sage.graphs.strongly_regular_db.**is_orthogonal_array_block_graph**(*k*, *l*, *mu*)

Test whether some (pseudo)Orthogonal Array graph is (v, k, λ, μ)-strongly regular.

We know how to construct graphs with parameters of an Orthogonal Array ($OA(m, n)$), also known as Latin squares graphs $L_m(n)$, in several cases where no orthogonal array is known, or even in some cases for which they are known not to exist.

Such graphs are usually called pseudo-Latin squares graphs. Namely, Sage can construct a graph with parameters of an $OA(m, n)$-graph whenever there exists a skew-Hadamard matrix of order $n + 1$, and $m = (n + 1)/2$ or $m = (n - 1)/2$. The construction in the former case is due to Goethals-Seidel [BL1984], and in the latter case due to Pasechnik [Pas1992].

INPUT:

- `v`, `k`, `l`, `mu` – integers

OUTPUT:

A tuple `t` such that `t[0](*t[1:])` builds the requested graph if one exists, and `None` otherwise.

EXAMPLES:

```
sage: from sage.graphs.strongly_regular_db import is_orthogonal_array_block_graph
sage: t = is_orthogonal_array_block_graph(64, 35, 18, 20); t
(..., 5, 8)
sage: g = t[0](*t[1:]); g
OA(5,8): Graph on 64 vertices
sage: g.is_strongly_regular(parameters=True)
(64, 35, 18, 20)
sage: t = is_orthogonal_array_block_graph(225,98,43,42); t
(..., 4)
sage: g = t[0](*t[1:]); g
Pasechnik Graph_4: Graph on 225 vertices
sage: g.is_strongly_regular(parameters=True)
(225, 98, 43, 42)
sage: t = is_orthogonal_array_block_graph(225,112,55,56); t
(..., 4)
sage: g = t[0](*t[1:]); g
skewhad^2_4: Graph on 225 vertices
sage: g.is_strongly_regular(parameters=True)
(225, 112, 55, 56)

sage: t = is_orthogonal_array_block_graph(5,5,5,5); t                   #␣
↪needs sage.combinat sage.modules
```

```
>>> from sage.all import *
>>> from sage.graphs.strongly_regular_db import is_orthogonal_array_block_graph
```

(continues on next page)

(continued from previous page)

```
>>> t = is_orthogonal_array_block_graph(Integer(64), Integer(35), Integer(18),␣
↪Integer(20)); t
(..., 5, 8)
>>> g = t[Integer(0)](*t[Integer(1):]); g
OA(5,8): Graph on 64 vertices
>>> g.is_strongly_regular(parameters=True)
(64, 35, 18, 20)
>>> t = is_orthogonal_array_block_graph(Integer(225),Integer(98),Integer(43),
↪Integer(42)); t
(..., 4)
>>> g = t[Integer(0)](*t[Integer(1):]); g
Pasechnik Graph_4: Graph on 225 vertices
>>> g.is_strongly_regular(parameters=True)
(225, 98, 43, 42)
>>> t = is_orthogonal_array_block_graph(Integer(225),Integer(112),Integer(55),
↪Integer(56)); t
(..., 4)
>>> g = t[Integer(0)](*t[Integer(1):]); g
skewhad^2_4: Graph on 225 vertices
>>> g.is_strongly_regular(parameters=True)
(225, 112, 55, 56)

>>> t = is_orthogonal_array_block_graph(Integer(5),Integer(5),Integer(5),
↪Integer(5)); t                              # needs sage.combinat sage.modules
```

sage.graphs.strongly_regular_db.**is_orthogonal_polar**(*k*, *l*, *mu*)

Test whether some Orthogonal Polar graph is (v, k, λ, μ)-strongly regular.

For more information, see https://www.win.tue.nl/~aeb/graphs/srghub.html.

INPUT:

- `v`, `k`, `l`, `mu` – integers

OUTPUT:

A tuple `t` such that `t[0](*t[1:])` builds the requested graph if one exists, and `None` otherwise.

EXAMPLES:

```
sage: from sage.graphs.strongly_regular_db import is_orthogonal_polar
sage: t = is_orthogonal_polar(85, 20, 3, 5); t
(<function OrthogonalPolarGraph at ...>, 5, 4, '')
sage: g = t[0](*t[1:]); g                                                    #␣
↪needs sage.rings.finite_rings
Orthogonal Polar Graph O(5, 4): Graph on 85 vertices
sage: g.is_strongly_regular(parameters=True)                                 #␣
↪needs sage.rings.finite_rings
(85, 20, 3, 5)

sage: t = is_orthogonal_polar(5,5,5,5); t                                    #␣
↪needs sage.rings.finite_rings
```

```
>>> from sage.all import *
>>> from sage.graphs.strongly_regular_db import is_orthogonal_polar
```

(continues on next page)

(continued from previous page)

```
>>> t = is_orthogonal_polar(Integer(85), Integer(20), Integer(3), Integer(5)); t
(<function OrthogonalPolarGraph at ...>, 5, 4, '')
>>> g = t[Integer(0)](*t[Integer(1):]); g                                    ␣
↪              # needs sage.rings.finite_rings
Orthogonal Polar Graph O(5, 4): Graph on 85 vertices
>>> g.is_strongly_regular(parameters=True)                                  #␣
↪needs sage.rings.finite_rings
(85, 20, 3, 5)

>>> t = is_orthogonal_polar(Integer(5),Integer(5),Integer(5),Integer(5)); t  ␣
↪                                # needs sage.rings.finite_rings
```

sage.graphs.strongly_regular_db.**is_paley**(*k*, *l*, *mu*)

Test whether some Paley graph is (v, k, λ, μ)-strongly regular.

INPUT:

- `v`, `k`, `l`, `mu` – integers

OUTPUT:

A tuple `t` such that `t[0](*t[1:])` builds the requested graph if one exists, and `None` otherwise.

EXAMPLES:

```
sage: from sage.graphs.strongly_regular_db import is_paley
sage: t = is_paley(13,6,2,3); t
(..., 13)
sage: g = t[0](*t[1:]); g                                                  #␣
↪needs sage.rings.finite_rings
Paley graph with parameter 13: Graph on 13 vertices
sage: g.is_strongly_regular(parameters=True)                               #␣
↪needs sage.rings.finite_rings
(13, 6, 2, 3)
sage: t = is_paley(5,5,5,5); t
```

```
>>> from sage.all import *
>>> from sage.graphs.strongly_regular_db import is_paley
>>> t = is_paley(Integer(13),Integer(6),Integer(2),Integer(3)); t
(..., 13)
>>> g = t[Integer(0)](*t[Integer(1):]); g                                    ␣
↪              # needs sage.rings.finite_rings
Paley graph with parameter 13: Graph on 13 vertices
>>> g.is_strongly_regular(parameters=True)                                  #␣
↪needs sage.rings.finite_rings
(13, 6, 2, 3)
>>> t = is_paley(Integer(5),Integer(5),Integer(5),Integer(5)); t
```

sage.graphs.strongly_regular_db.**is_polhill**(*k*, *l*, *mu*)

Test whether some graph from [Pol2009] is $(1024, k, \lambda, \mu)$-strongly regular.

Note

This function does not actually explore *all* strongly regular graphs produced in [Pol2009], but only those on 1024 vertices.

John Polhill offered his help if we attempt to write a code to guess, given (v, k, λ, μ), which of his construction must be applied to find the graph.

INPUT:

- `v`, `k`, `l`, `mu` – integers

OUTPUT:

A tuple `t` such that `t[0](*t[1:])` builds the requested graph if the parameters match, and `None` otherwise.

EXAMPLES:

```
sage: from sage.graphs.strongly_regular_db import is_polhill
sage: t = is_polhill(1024, 231,  38,  56); t
(<cyfunction is_polhill.<locals>.additive_cayley at ...>, ...)
sage: g = t[0](*t[1:]); g               # not tested (too long)
Graph on 1024 vertices
sage: g.is_strongly_regular(parameters=True)    # not tested (too long)
(1024, 231, 38, 56)
sage: t = is_polhill(1024, 264,  56,  72); t
(<cyfunction is_polhill.<locals>.additive_cayley at ...>, ...)
sage: t = is_polhill(1024, 297,  76,  90); t
(<cyfunction is_polhill.<locals>.additive_cayley_complement at ...>, ...)
sage: t = is_polhill(1024, 330,  98, 110); t
(<cyfunction is_polhill.<locals>.additive_cayley_complement at ...>, ...)
sage: t = is_polhill(1024, 462, 206, 210); t
(<cyfunction is_polhill.<locals>.additive_cayley at ...>, ...)
```

```
>>> from sage.all import *
>>> from sage.graphs.strongly_regular_db import is_polhill
>>> t = is_polhill(Integer(1024), Integer(231),  Integer(38),  Integer(56)); t
(<cyfunction is_polhill.<locals>.additive_cayley at ...>, ...)
>>> g = t[Integer(0)](*t[Integer(1):]); g               # not tested (too long)
Graph on 1024 vertices
>>> g.is_strongly_regular(parameters=True)    # not tested (too long)
(1024, 231, 38, 56)
>>> t = is_polhill(Integer(1024), Integer(264),  Integer(56),  Integer(72)); t
(<cyfunction is_polhill.<locals>.additive_cayley at ...>, ...)
>>> t = is_polhill(Integer(1024), Integer(297),  Integer(76),  Integer(90)); t
(<cyfunction is_polhill.<locals>.additive_cayley_complement at ...>, ...)
>>> t = is_polhill(Integer(1024), Integer(330),  Integer(98), Integer(110)); t
(<cyfunction is_polhill.<locals>.additive_cayley_complement at ...>, ...)
>>> t = is_polhill(Integer(1024), Integer(462), Integer(206), Integer(210)); t
(<cyfunction is_polhill.<locals>.additive_cayley at ...>, ...)
```

`sage.graphs.strongly_regular_db.`**`is_steiner`**`(k, l, mu)`

Test whether some Steiner graph is (v, k, λ, μ)-strongly regular.

A Steiner graph is the intersection graph of a Steiner set system. For more information, see https://www.win.tue.nl/~aeb/graphs/S.html.

INPUT:

- v, k, l, mu – integers

OUTPUT:

A tuple t such that t[0](*t[1:]) builds the requested graph if one exists, and None otherwise.

EXAMPLES:

```
sage: from sage.graphs.strongly_regular_db import is_steiner
sage: t = is_steiner(26,15,8,9); t
(..., 13, 3)
sage: g = t[0](*t[1:]); g
Intersection Graph: Graph on 26 vertices
sage: g.is_strongly_regular(parameters=True)
(26, 15, 8, 9)

sage: t = is_steiner(5,5,5,5); t
```

```
>>> from sage.all import *
>>> from sage.graphs.strongly_regular_db import is_steiner
>>> t = is_steiner(Integer(26),Integer(15),Integer(8),Integer(9)); t
(..., 13, 3)
>>> g = t[Integer(0)](*t[Integer(1):]); g
Intersection Graph: Graph on 26 vertices
>>> g.is_strongly_regular(parameters=True)
(26, 15, 8, 9)

>>> t = is_steiner(Integer(5),Integer(5),Integer(5),Integer(5)); t
```

sage.graphs.strongly_regular_db.**is_switch_OA_srg**(*v*, *k*, *l*, *mu*)

Test whether some *switch* $OA(k,n)+*$ is (v,k,λ,μ)-strongly regular.

The "switch* $OA(k,n)+*$ graphs appear on Andries Brouwer's database and are built by adding an isolated vertex to a *OrthogonalArrayBlockGraph()*, and a *Seidel switching* a set of disjoint n-cocliques.

INPUT:

- v, k, l, mu – integers

OUTPUT:

A tuple t such that t[0](*t[1:]) builds the requested graph if the parameters match, and None otherwise.

EXAMPLES:

```
sage: graphs.strongly_regular_graph(170, 78, 35, 36)  # indirect doctest        #␣
↪needs sage.combinat sage.modules
Graph on 170 vertices
```

```
>>> from sage.all import *
>>> graphs.strongly_regular_graph(Integer(170), Integer(78), Integer(35),␣
↪Integer(36))  # indirect doctest        # needs sage.combinat sage.modules
Graph on 170 vertices
```

sage.graphs.strongly_regular_db.**is_switch_skewhad**(*v*, *k*, *l*, *mu*)

Test whether some switch skewhad^2+* is (v,k,λ,μ)-strongly regular.

The `switch skewhad^2+*` graphs appear on Andries Brouwer's database and are built by adding an isolated vertex to the complement of *SquaredSkewHadamardMatrixGraph()*, and a *Seidel switching* a set of disjoint n-cocliques.

INPUT:

- `v`, `k`, `l`, `mu` – integers

OUTPUT:

A tuple `t` such that `t[0](*t[1:])` builds the requested graph if the parameters match, and `None` otherwise.

EXAMPLES:

```
sage: graphs.strongly_regular_graph(226, 105, 48, 49)                          #␣
↪needs sage.combinat sage.modules
switch skewhad^2+*_4: Graph on 226 vertices
```

```
>>> from sage.all import *
>>> graphs.strongly_regular_graph(Integer(226), Integer(105), Integer(48),␣
↪Integer(49))                              # needs sage.combinat sage.modules
switch skewhad^2+*_4: Graph on 226 vertices
```

`sage.graphs.strongly_regular_db.`**`is_taylor_twograph_srg`**`(k, l, mu)`

Test whether some Taylor two-graph SRG is (v, k, λ, μ)-strongly regular.

For more information, see §7E of [BL1984].

INPUT:

- `v`, `k`, `l`, `mu` – integers

OUTPUT:

A tuple `t` such that `t[0](*t[1:])` builds the requested graph *TaylorTwographSRG* if the parameters match, and `None` otherwise.

EXAMPLES:

```
sage: from sage.graphs.strongly_regular_db import is_taylor_twograph_srg
sage: t = is_taylor_twograph_srg(28, 15, 6, 10); t
(<function TaylorTwographSRG at ...>, 3)
sage: g = t[0](*t[1:]); g
Taylor two-graph SRG: Graph on 28 vertices
sage: g.is_strongly_regular(parameters=True)
(28, 15, 6, 10)
sage: t = is_taylor_twograph_srg(5,5,5,5); t
```

```
>>> from sage.all import *
>>> from sage.graphs.strongly_regular_db import is_taylor_twograph_srg
>>> t = is_taylor_twograph_srg(Integer(28), Integer(15), Integer(6), Integer(10));
↪ t
(<function TaylorTwographSRG at ...>, 3)
>>> g = t[Integer(0)](*t[Integer(1):]); g
Taylor two-graph SRG: Graph on 28 vertices
>>> g.is_strongly_regular(parameters=True)
(28, 15, 6, 10)
>>> t = is_taylor_twograph_srg(Integer(5),Integer(5),Integer(5),Integer(5)); t
```

sage.graphs.strongly_regular_db.**is_twograph_descendant_of_srg**(*k0*, *l*, *mu*)

Test whether some descendant graph of a s.r.g. is (v, k_0, λ, μ)-s.r.g.

We check whether there can exist $(v+1, k, \lambda^*, \mu^*)$-s.r.g. G so that `self` is a descendant graph of the regular two-graph specified by G. Specifically, we must have that $v+1 = 2(2k - \lambda^* - \mu^*)$, and $k_0 = 2(k - \mu^*)$, $\lambda = k + \lambda^* - 2\mu^*$, $\mu = k - \mu^*$, which give 2 independent linear conditions, say $k - \mu^* = \mu$ and $\lambda^* - \mu^* = \lambda - \mu$. Further, there is a quadratic relation $2k^2 - (v + 1 + 4\mu)k + 2v\mu = 0$.

If we can construct such G then we return a function to build a (v, k_0, λ, μ)-s.r.g. For more information, see 10.3 in https://www.win.tue.nl/~aeb/2WF02/spectra.pdf

INPUT:

- `v`, `k0`, `l`, `mu` – integers

OUTPUT:

A tuple `t` such that `t[0](*t[1:])` builds the requested graph if one exists and is known, and `None` otherwise.

EXAMPLES:

```
sage: from sage.graphs.strongly_regular_db import is_twograph_descendant_of_srg
sage: t = is_twograph_descendant_of_srg(27, 10, 1, 5); t                    #␣
↪needs sage.rings.finite_rings
(<...is_twograph_descendant_of_srg...>, 8, 2)
sage: g = t[0](*t[1:]); g                                                   #␣
↪needs sage.rings.finite_rings
descendant of complement(Johnson graph with parameters 8,2) at {0, 1}: Graph on␣
↪27 vertices
sage: g.is_strongly_regular(parameters=True)                                #␣
↪needs sage.rings.finite_rings
(27, 10, 1, 5)
sage: t = is_twograph_descendant_of_srg(5,5,5,5); t
```

```
>>> from sage.all import *
>>> from sage.graphs.strongly_regular_db import is_twograph_descendant_of_srg
>>> t = is_twograph_descendant_of_srg(Integer(27), Integer(10), Integer(1),␣
↪Integer(5)); t                          # needs sage.rings.finite_rings
(<...is_twograph_descendant_of_srg...>, 8, 2)
>>> g = t[Integer(0)](*t[Integer(1):]); g                                    ␣
↪              # needs sage.rings.finite_rings
descendant of complement(Johnson graph with parameters 8,2) at {0, 1}: Graph on␣
↪27 vertices
>>> g.is_strongly_regular(parameters=True)                                  #␣
↪needs sage.rings.finite_rings
(27, 10, 1, 5)
>>> t = is_twograph_descendant_of_srg(Integer(5),Integer(5),Integer(5),
↪Integer(5)); t
```

sage.graphs.strongly_regular_db.**is_unitary_dual_polar**(*k*, *l*, *mu*)

Test whether some Unitary Dual Polar graph is (v, k, λ, μ)-strongly regular.

This must be the U_5(q) on totally isotropic lines. For more information, see https://www.win.tue.nl/~aeb/graphs/srghub.html.

INPUT:

- `v`, `k`, `l`, `mu` – integers

OUTPUT:

A tuple `t` such that `t[0](*t[1:])` builds the requested graph if one exists, and `None` otherwise.

EXAMPLES:

```
sage: from sage.graphs.strongly_regular_db import is_unitary_dual_polar
sage: t = is_unitary_dual_polar(297, 40, 7, 5); t
(<function UnitaryDualPolarGraph at ...>, 5, 2)
sage: g = t[0](*t[1:]); g
Unitary Dual Polar Graph DU(5, 2); GQ(8, 4): Graph on 297 vertices
sage: g.is_strongly_regular(parameters=True)
(297, 40, 7, 5)
sage: t = is_unitary_dual_polar(5,5,5,5); t
```

```
>>> from sage.all import *
>>> from sage.graphs.strongly_regular_db import is_unitary_dual_polar
>>> t = is_unitary_dual_polar(Integer(297), Integer(40), Integer(7), Integer(5));␣
↪t
(<function UnitaryDualPolarGraph at ...>, 5, 2)
>>> g = t[Integer(0)](*t[Integer(1):]); g
Unitary Dual Polar Graph DU(5, 2); GQ(8, 4): Graph on 297 vertices
>>> g.is_strongly_regular(parameters=True)
(297, 40, 7, 5)
>>> t = is_unitary_dual_polar(Integer(5),Integer(5),Integer(5),Integer(5)); t
```

`sage.graphs.strongly_regular_db.`**`is_unitary_polar`**`(k, l, mu)`

Test whether some Unitary Polar graph is (v, k, λ, μ)-strongly regular.

For more information, see https://www.win.tue.nl/~aeb/graphs/srghub.html.

INPUT:

- `v`, `k`, `l`, `mu` – integers

OUTPUT:

A tuple `t` such that `t[0](*t[1:])` builds the requested graph if one exists, and `None` otherwise.

EXAMPLES:

```
sage: from sage.graphs.strongly_regular_db import is_unitary_polar
sage: t = is_unitary_polar(45, 12, 3, 3); t                                    #␣
↪needs sage.libs.pari
(<function UnitaryPolarGraph at ...>, 4, 2)
sage: g = t[0](*t[1:]); g                                                      #␣
↪needs sage.libs.pari
Unitary Polar Graph U(4, 2); GQ(4, 2): Graph on 45 vertices
sage: g.is_strongly_regular(parameters=True)                                   #␣
↪needs sage.libs.pari
(45, 12, 3, 3)

sage: t = is_unitary_polar(5,5,5,5); t                                         #␣
↪needs sage.libs.pari
```

```
>>> from sage.all import *
>>> from sage.graphs.strongly_regular_db import is_unitary_polar
```

(continues on next page)

(continued from previous page)

```
>>> t = is_unitary_polar(Integer(45), Integer(12), Integer(3), Integer(3)); t
↪                                  # needs sage.libs.pari
(<function UnitaryPolarGraph at ...>, 4, 2)
>>> g = t[Integer(0)](*t[Integer(1):]); g
↪               # needs sage.libs.pari
Unitary Polar Graph U(4, 2); GQ(4, 2): Graph on 45 vertices
>>> g.is_strongly_regular(parameters=True)
↪needs sage.libs.pari
(45, 12, 3, 3)

>>> t = is_unitary_polar(Integer(5),Integer(5),Integer(5),Integer(5)); t
↪                                  # needs sage.libs.pari
```

sage.graphs.strongly_regular_db.**latin_squares_graph_parameters**(*v*, *k*, *l*, *mu*)

Check whether (v,k,l,mu)-strongly regular graph has parameters of an $L_g(n)$ s.r.g.

Also known as pseudo-OA(n,g) case, i.e. s.r.g. with parameters of an OA(n,g)-graph. Return g and n, if they exist. See Sect. 9.1 of [BH2012] for details.

INPUT:

- `v`, `k`, `l`, `mu` – - (integrs) parameters of the graph

OUTPUT:

- `(g, n)` – parameters of an $L_g(n)$ graph, or `None`

sage.graphs.strongly_regular_db.**strongly_regular_from_two_intersection_set**(*M*, *immutable=False*)

Return a strongly regular graph from a 2-intersection set.

A set of points in the projective geometry $PG(k,q)$ is said to be a 2-intersection set if it intersects every hyperplane in either h_1 or h_2 points, where $h_1, h_2 \in NN$.

From a 2-intersection set S can be defined a strongly-regular graph in the following way:

- Place the points of S on a hyperplane H in $PG(k+1,q)$
- Define the graph G on all points of $PG(k+1,q) \setminus H$
- Make two points of $V(G) = PG(k+1,q) \setminus H$ adjacent if the line going through them intersects S

For more information, see e.g. [CD2013] where this explanation has been taken from.

INPUT:

- `M` – a $|S| \times k$ matrix with entries in F_q representing the points of the 2-intersection set. We assume that the first nonzero entry of each row is equal to 1, that is, they give points in homogeneous coordinates.

The implementation does not check that S is actually a 2-intersection set.

EXAMPLES:

```
sage: from sage.graphs.strongly_regular_db import strongly_regular_from_two_
↪intersection_set
sage: S = Matrix([(0,0,1),(0,1,0)] + [(1,x^2,x) for x in GF(4,'b')])
↪needs sage.modules sage.rings.finite_rings
sage: g = strongly_regular_from_two_intersection_set(S); g
↪needs sage.modules sage.rings.finite_rings
```

(continues on next page)

(continued from previous page)

```
two-intersection set in PG(3,4): Graph on 64 vertices
sage: g.is_strongly_regular(parameters=True)                                    #␣
↪needs sage.modules sage.rings.finite_rings
(64, 18, 2, 6)
```

```
>>> from sage.all import *
>>> from sage.graphs.strongly_regular_db import strongly_regular_from_two_
↪intersection_set
>>> S = Matrix([(Integer(0),Integer(0),Integer(1)),(Integer(0),Integer(1),
↪Integer(0))] + [(Integer(1),x**Integer(2),x) for x in GF(Integer(4),'b')])      ␣
↪      # needs sage.modules sage.rings.finite_rings
>>> g = strongly_regular_from_two_intersection_set(S); g                        #␣
↪needs sage.modules sage.rings.finite_rings
two-intersection set in PG(3,4): Graph on 64 vertices
>>> g.is_strongly_regular(parameters=True)                                      #␣
↪needs sage.modules sage.rings.finite_rings
(64, 18, 2, 6)
```

sage.graphs.strongly_regular_db.**strongly_regular_from_two_weight_code**(*L*, *immutable=False*)

Return a strongly regular graph from a two-weight code.

A code is said to be a *two-weight* code the weight of its nonzero codewords (i.e. their number of nonzero coordinates) can only be one of two integer values w_1, w_2. It is said to be *projective* if the minimum weight of the dual code is ≥ 3. A strongly regular graph can be built from a two-weight projective code with weights w_1, w_2 (assuming $w_1 < w_2$) by adding an edge between any two codewords whose difference has weight w_1. For more information, see [LS1981] or [Del1972].

INPUT:

- `L` – a two-weight linear code, or its generating matrix

EXAMPLES:

```
sage: from sage.graphs.strongly_regular_db import strongly_regular_from_two_
↪weight_code
sage: x = ("100022021001111",
....:      "010011211122000",
....:      "001021112100011",
....:      "000110120222220")
sage: M = Matrix(GF(3),[list(l) for l in x])                                   #␣
↪needs sage.modules sage.rings.finite_rings
sage: G = strongly_regular_from_two_weight_code(LinearCode(M))                 #␣
↪needs sage.modules sage.rings.finite_rings
sage: G.is_strongly_regular(parameters=True)                                   #␣
↪needs sage.modules sage.rings.finite_rings
(81, 50, 31, 30)
```

```
>>> from sage.all import *
>>> from sage.graphs.strongly_regular_db import strongly_regular_from_two_weight_
↪code
>>> x = ("100022021001111",
...      "010011211122000",
...      "001021112100011",
```

(continues on next page)

(continued from previous page)

```
...       "000110120222220")
>>> M = Matrix(GF(Integer(3)),[list(l) for l in x])                    ␣
↪     # needs sage.modules sage.rings.finite_rings
>>> G = strongly_regular_from_two_weight_code(LinearCode(M))          #␣
↪needs sage.modules sage.rings.finite_rings
>>> G.is_strongly_regular(parameters=True)                            #␣
↪needs sage.modules sage.rings.finite_rings
(81, 50, 31, 30)
```

sage.graphs.strongly_regular_db.**strongly_regular_graph**(*v*, *k*, *l*, *mu=-1*, *existence=False*, *check=True*, *immutable=False*)

Return a (v, k, λ, μ)-strongly regular graph.

This function relies partly on Andries Brouwer's database of strongly regular graphs. See the documentation of *sage.graphs.strongly_regular_db* for more information.

INPUT:

- `v`, `k`, `l`, `mu` – `integers` – note that `mu`, if unspecified, is automatically determined from `v`, `k`, `l`
- `existence` – boolean (default: `False`); instead of building the graph, return:
  - `True` – meaning that a (v, k, λ, μ)-strongly regular graph exists
  - `Unknown` – meaning that Sage does not know if such a strongly regular graph exists (see `sage.misc.unknown`)
  - `False` – meaning that no such strongly regular graph exists
- `check` – boolean (default: `True`); whether to check that output is correct before returning it. As this is expected to be useless, you may want to disable it whenever you want speed.
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph. This parameter is only only when `existence` is `False`.

EXAMPLES:

Petersen's graph from its set of parameters:

```
sage: graphs.strongly_regular_graph(10,3,0,1,existence=True)           #␣
↪needs sage.libs.pari
True
sage: graphs.strongly_regular_graph(10,3,0,1)
complement(Johnson graph with parameters 5,2): Graph on 10 vertices
```

```
>>> from sage.all import *
>>> graphs.strongly_regular_graph(Integer(10),Integer(3),Integer(0),Integer(1),
↪existence=True)                    # needs sage.libs.pari
True
>>> graphs.strongly_regular_graph(Integer(10),Integer(3),Integer(0),Integer(1))
complement(Johnson graph with parameters 5,2): Graph on 10 vertices
```

Now without specifying μ:

```
sage: graphs.strongly_regular_graph(10,3,0)
complement(Johnson graph with parameters 5,2): Graph on 10 vertices
```

```
>>> from sage.all import *
>>> graphs.strongly_regular_graph(Integer(10),Integer(3),Integer(0))
complement(Johnson graph with parameters 5,2): Graph on 10 vertices
```

An obviously infeasible set of parameters:

```
sage: graphs.strongly_regular_graph(5,5,5,5,existence=True)
False
sage: graphs.strongly_regular_graph(5,5,5,5)
Traceback (most recent call last):
...
ValueError: There exists no (5, 5, 5, 5)-strongly regular graph
```

```
>>> from sage.all import *
>>> graphs.strongly_regular_graph(Integer(5),Integer(5),Integer(5),Integer(5),
↪existence=True)
False
>>> graphs.strongly_regular_graph(Integer(5),Integer(5),Integer(5),Integer(5))
Traceback (most recent call last):
...
ValueError: There exists no (5, 5, 5, 5)-strongly regular graph
```

A set of parameters proved in a paper to be infeasible:

```
sage: graphs.strongly_regular_graph(324,57,0,12,existence=True)                #␣
↪needs sage.combinat sage.modules
False
sage: graphs.strongly_regular_graph(324,57,0,12)                               #␣
↪needs sage.combinat sage.modules
Traceback (most recent call last):
...
EmptySetError: Andries Brouwer's database reports that no (324, 57, 0,
12)-strongly regular graph exists. Comments: <a
href="srgtabrefs.html#GavrilyukMakhnev05">Gavrilyuk & Makhnev</a> ...
```

```
>>> from sage.all import *
>>> graphs.strongly_regular_graph(Integer(324),Integer(57),Integer(0),Integer(12),
↪existence=True)                 # needs sage.combinat sage.modules
False
>>> graphs.strongly_regular_graph(Integer(324),Integer(57),Integer(0),
↪Integer(12))                                  # needs sage.combinat sage.modules
Traceback (most recent call last):
...
EmptySetError: Andries Brouwer's database reports that no (324, 57, 0,
12)-strongly regular graph exists. Comments: <a
href="srgtabrefs.html#GavrilyukMakhnev05">Gavrilyuk & Makhnev</a> ...
```

A set of parameters unknown to be realizable in Andries Brouwer's database:

```
sage: graphs.strongly_regular_graph(324,95,22,30,existence=True)               #␣
↪needs sage.combinat
Unknown
sage: graphs.strongly_regular_graph(324,95,22,30)                              #␣
```

(continues on next page)

(continued from previous page)

```
↪needs sage.combinat
Traceback (most recent call last):
...
RuntimeError: Andries Brouwer's database reports that no
(324, 95, 22, 30)-strongly regular graph is known to exist.
Comments:
```

```
>>> from sage.all import *
>>> graphs.strongly_regular_graph(Integer(324),Integer(95),Integer(22),
↪Integer(30),existence=True)                # needs sage.combinat
Unknown
>>> graphs.strongly_regular_graph(Integer(324),Integer(95),Integer(22),
↪Integer(30))                               # needs sage.combinat
Traceback (most recent call last):
...
RuntimeError: Andries Brouwer's database reports that no
(324, 95, 22, 30)-strongly regular graph is known to exist.
Comments:
```

A large unknown set of parameters (not in Andries Brouwer's database):

```
sage: graphs.strongly_regular_graph(1394,175,0,25,existence=True)                #␣
↪needs sage.combinat
Unknown
sage: graphs.strongly_regular_graph(1394,175,0,25)                               #␣
↪needs sage.combinat
Traceback (most recent call last):
...
RuntimeError: Sage cannot figure out if a (1394, 175, 0, 25)-strongly
regular graph exists.
```

```
>>> from sage.all import *
>>> graphs.strongly_regular_graph(Integer(1394),Integer(175),Integer(0),
↪Integer(25),existence=True)                # needs sage.combinat
Unknown
>>> graphs.strongly_regular_graph(Integer(1394),Integer(175),Integer(0),
↪Integer(25))                               # needs sage.combinat
Traceback (most recent call last):
...
RuntimeError: Sage cannot figure out if a (1394, 175, 0, 25)-strongly
regular graph exists.
```

Test the Claw bound (see 3.D of [BL1984]):

```
sage: graphs.strongly_regular_graph(2058,242,91,20,existence=True)
False
```

```
>>> from sage.all import *
>>> graphs.strongly_regular_graph(Integer(2058),Integer(242),Integer(91),
↪Integer(20),existence=True)
False
```

sage.graphs.strongly_regular_db.**strongly_regular_graph_lazy**(*v*, *k*, *l*, *mu=-1*, *existence=False*)

Return a promise to build an (v, k, l, mu)-srg.

Return a promise to build an (v, k, l, mu)-srg as a tuple t, with $t[0]$ a function to evaluate on $*t[1:]$.

Input as in *strongly_regular_graph()*, although without *check*.

2.6 Graphs defined by systems of equations

This module implements a class of bipartite graphs defined by triangular systems of equations, popularized by Lazebnik, Ustimenko, and Woldar. More precisely, let R be a finite commutative ring. The graph has point part $P = R^n$ and line part $L = R^n$. For $2 \le i \le n$, let f_i be a polynomial function in $2i-2$ variables. A point $(p_1, p_2, \dots, p_n)$ is adjacent to a line $(l_1, l_2, \dots, l_n)$ if

$$p_i + l_i = f_i(p_1, l_1, \dots, p_{i-1}, l_{i-1}), \qquad 2 \le i \le n.$$

The class *LUWGraphDescriptor* stores a validated set of equations which define the graph. This lets users experiment with large examples without paying the cost of building the full graph. Currently, the descriptor-returning constructors are *define_luw_graph()*, *define_Akq()*, *define_Dkq()*, and *define_WengerGraph()*. The graph-returning constructors *Akq()*, *Dkq()*, *LUWGraph()*, and *WengerGraph()* are exposed through *sage.graphs.graph_generators*, and so are available as `graphs.LUWGraph(...)`, `graphs.WengerGraph(...)`, and so on.

The general construction and the graph families implemented here are surveyed in [LW2026].

EXAMPLES:

The graph constructor is available directly from `graphs`:

```
sage: F = GF(3)
sage: R = PolynomialRing(F, names=("p1", "l1", "p2", "l2"))
sage: p1, l1, p2, l2 = R.gens()
sage: G = graphs.LUWGraph(F, [p1*l1, p1*l2])
sage: G.order(), G.size(), G.girth()
(54, 81, 8)
```

```
>>> from sage.all import *
>>> F = GF(Integer(3))
>>> R = PolynomialRing(F, names=("p1", "l1", "p2", "l2"))
>>> p1, l1, p2, l2 = R.gens()
>>> G = graphs.LUWGraph(F, [p1*l1, p1*l2])
>>> G.order(), G.size(), G.girth()
(54, 81, 8)
```

Use *define_luw_graph()* when the algebraic description should be kept as a *LUWGraphDescriptor* without immediately constructing the full graph.

AUTHORS:

- Vladislav Taranchuk (2026-04): initial version

sage.graphs.generators.luw_graphs.**Akq**(*k*, *q*, *A=None*, *B=None*, *immutable=False*)

Return the graph $A(k, q)$ described in [LW2026].

The point set and line set are $\mathbf{F}_q^k$. A point $(p_1, \dots, p_k)$ is adjacent to a line $(l_1, \dots, l_k)$ if

$$p_i + l_i = \begin{cases} p_{i-1} l_1, & i \text{ even}, \\ p_1 l_{i-1}, & i \text{ odd}, \end{cases} \qquad 2 \le i \le k.$$

INPUT:

- k – integer at least 2
- q – prime power
- A – iterable, single value, or None (default: None); allowed first coordinates on the point side
- B – iterable, single value, or None (default: None); allowed first coordinates on the line side
- immutable – boolean (default: False); whether to return an immutable or a mutable graph

OUTPUT:

A Sage *Graph*.

EXAMPLES:

```
sage: G = graphs.Akq(5, 3)
sage: G.order(), G.size(), G.girth()
(486, 729, 12)
```

```
>>> from sage.all import *
>>> G = graphs.Akq(Integer(5), Integer(3))
>>> G.order(), G.size(), G.girth()
(486, 729, 12)
```

REFERENCES:

- [LW2026]

sage.graphs.generators.luw_graphs.**Dkq**(*k*, *q*, *A=None*, *B=None*, *immutable=False*)

Return the graph $D(k, q)$ described in [LW2026].

The point set and line set are $\mathbf{F}_q^k$. A point $(p_1, \ldots, p_k)$ is adjacent to a line $(l_1, \ldots, l_k)$ if

$$p_2 + l_2 = p_1 l_1,$$

and, when $k \geq 3$,

$$p_3 + l_3 = p_1 l_2.$$

For $4 \leq i \leq k$, the remaining equations are

$$p_i + l_i = \begin{cases} p_{i-2} l_1, & i \equiv 0, 1 \pmod 4, \\ p_1 l_{i-2}, & i \equiv 2, 3 \pmod 4. \end{cases}$$

INPUT:

- k – integer at least 2
- q – prime power
- A – iterable, single value, or None (default: None); allowed first coordinates on the point side
- B – iterable, single value, or None (default: None); allowed first coordinates on the line side
- immutable – boolean (default: False); whether to return an immutable or a mutable graph

OUTPUT:

A Sage *Graph*.

EXAMPLES:

```
sage: G = graphs.Dkq(5, 3)
sage: G.order(), G.size(), G.girth()
(486, 729, 12)
```

```
>>> from sage.all import *
>>> G = graphs.Dkq(Integer(5), Integer(3))
>>> G.order(), G.size(), G.girth()
(486, 729, 12)
```

REFERENCES:

- [LW2026]

sage.graphs.generators.luw_graphs.**LUWGraph**(*ring*, *equations*, *name=None*, *A=None*, *B=None*, *point_coordinate_sets=None*, *line_coordinate_sets=None*, *immutable=False*)

Build a graph directly from a given set of equations.

This is the graph-returning constructor used by `graphs.LUWGraph`. The construction is surveyed in [LW2026].

INPUT:

- `ring` – finite commutative ring
- `equations` – nonempty iterable of Sage polynomials
- `name` – string (default: `None`); graph name
- `A` – iterable, single value, or `None` (default: `None`); allowed first coordinates on the point side
- `B` – iterable, single value, or `None` (default: `None`); allowed first coordinates on the line side
- `point_coordinate_sets` – iterable of coordinate sets or `None` (default: `None`)
- `line_coordinate_sets` – iterable of coordinate sets or `None` (default: `None`)
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

OUTPUT:

A Sage *Graph*.

EXAMPLES:

```
sage: F = GF(3)
sage: R = PolynomialRing(F, names=("p1", "l1", "p2", "l2", "p3", "l3"))
sage: p1, l1, p2, l2, p3, l3 = R.gens()
sage: G = graphs.LUWGraph(F, [p1*l1, p1*l2, p3*l1])
sage: G.order(), G.size()
(162, 243)
```

```
>>> from sage.all import *
>>> F = GF(Integer(3))
>>> R = PolynomialRing(F, names=("p1", "l1", "p2", "l2", "p3", "l3"))
>>> p1, l1, p2, l2, p3, l3 = R.gens()
>>> G = graphs.LUWGraph(F, [p1*l1, p1*l2, p3*l1])
>>> G.order(), G.size()
(162, 243)
```

REFERENCES:

- [LW2026]

class sage.graphs.generators.luw_graphs.**LUWGraphDescriptor**(*ring*, *equations*, *name*, *point_coordinate_sets*, *line_coordinate_sets*)

Bases: object

A validated algebraic description of a bipartite graph.

The graph itself is built from the stored equations and coordinate sets. See [LW2026] for a survey of these graphs.

EXAMPLES:

```
sage: from sage.graphs.generators.luw_graphs import define_luw_graph
sage: F = GF(5)
sage: R = PolynomialRing(F, names=("p1", "l1"))
sage: p1, l1 = R.gens()
sage: D = define_luw_graph(F, [p1*l1])
sage: D.order()
50
sage: D.size()
125
```

```
>>> from sage.all import *
>>> from sage.graphs.generators.luw_graphs import define_luw_graph
>>> F = GF(Integer(5))
>>> R = PolynomialRing(F, names=("p1", "l1"))
>>> p1, l1 = R.gens()
>>> D = define_luw_graph(F, [p1*l1])
>>> D.order()
50
>>> D.size()
125
```

REFERENCES:

- [LW2026]

adjacency_matrix(*vertices=None*)

Return the adjacency matrix without first constructing the graph.

INPUT:

- `vertices` – list, tuple, or `None` (default: `None`); vertex order

When `vertices` is `None`, the vertex order agrees with Sage's default graph adjacency-matrix order, namely sorted vertex labels.

EXAMPLES:

```
sage: from sage.graphs.generators.luw_graphs import define_luw_graph
sage: F = GF(3)
sage: R = PolynomialRing(F, names=("p1", "l1", "p2", "l2"))
sage: p1, l1, p2, l2 = R.gens()
sage: D = define_luw_graph(F, [p1*l1, p1*l2 + p2])
sage: D.adjacency_matrix().dimensions()
(54, 54)
```

(continues on next page)

(continued from previous page)

```
sage: D.adjacency_matrix() == D.graph().adjacency_matrix()
True
```

```
>>> from sage.all import *
>>> from sage.graphs.generators.luw_graphs import define_luw_graph
>>> F = GF(Integer(3))
>>> R = PolynomialRing(F, names=("p1", "l1", "p2", "l2"))
>>> p1, l1, p2, l2 = R.gens()
>>> D = define_luw_graph(F, [p1*l1, p1*l2 + p2])
>>> D.adjacency_matrix().dimensions()
(54, 54)
>>> D.adjacency_matrix() == D.graph().adjacency_matrix()
True
```

A custom vertex order can also be supplied:

```
sage: vertices = list(D.graph().vertices(sort=False))
sage: D.adjacency_matrix(vertices=vertices) == \
....:     D.graph().adjacency_matrix(vertices=vertices)
True
```

```
>>> from sage.all import *
>>> vertices = list(D.graph().vertices(sort=False))
>>> D.adjacency_matrix(vertices=vertices) ==     D.graph().adjacency_
↪matrix(vertices=vertices)
True
```

ball(*start_vertex*, *depth*, *immutable=False*)

Build the ball of radius `depth` around `start_vertex`.

The construction stops early if a new layer adds no vertices.

INPUT:

- `start_vertex` – pair `(side, coordinates)` where `side` is either `"P"` or `"L"`
- `depth` – nonnegative integer; radius of the ball
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: from sage.graphs.generators.luw_graphs import define_luw_graph
sage: F = GF(3)
sage: R = PolynomialRing(F, names=("p1", "l1"))
sage: p1, l1 = R.gens()
sage: D = define_luw_graph(F, [p1*l1])
sage: B = D.ball(("P", (0, 0)), 1)
sage: B.order(), B.size()
(4, 3)
sage: D.ball(("P", (0, 0)), 1, immutable=True).is_immutable()
True
sage: D = define_luw_graph(F, [p1*l1], A=[0], B=[0])
sage: B = D.ball(("P", (0, 0)), 5)
```

(continues on next page)

(continued from previous page)

```
sage: B.order(), B._luw_graph_ball_metadata["component_recovered"]
(2, True)
```

```
>>> from sage.all import *
>>> from sage.graphs.generators.luw_graphs import define_luw_graph
>>> F = GF(Integer(3))
>>> R = PolynomialRing(F, names=("p1", "l1"))
>>> p1, l1 = R.gens()
>>> D = define_luw_graph(F, [p1*l1])
>>> B = D.ball(("P", (Integer(0), Integer(0))), Integer(1))
>>> B.order(), B.size()
(4, 3)
>>> D.ball(("P", (Integer(0), Integer(0))), Integer(1), immutable=True).is_
↪immutable()
True
>>> D = define_luw_graph(F, [p1*l1], A=[Integer(0)], B=[Integer(0)])
>>> B = D.ball(("P", (Integer(0), Integer(0))), Integer(5))
>>> B.order(), B._luw_graph_ball_metadata["component_recovered"]
(2, True)
```

base_ring()

Return the underlying finite ring.

EXAMPLES:

```
sage: from sage.graphs.generators.luw_graphs import define_luw_graph
sage: Z4 = Integers(4)
sage: R = PolynomialRing(Z4, names=("p1", "l1"))
sage: p1, l1 = R.gens()
sage: define_luw_graph(Z4, [p1*l1]).base_ring()
Ring of integers modulo 4
```

```
>>> from sage.all import *
>>> from sage.graphs.generators.luw_graphs import define_luw_graph
>>> Z4 = Integers(Integer(4))
>>> R = PolynomialRing(Z4, names=("p1", "l1"))
>>> p1, l1 = R.gens()
>>> define_luw_graph(Z4, [p1*l1]).base_ring()
Ring of integers modulo 4
```

dimension()

Return the number of point or line coordinates.

EXAMPLES:

```
sage: from sage.graphs.generators.luw_graphs import define_luw_graph
sage: F = GF(3)
sage: R = PolynomialRing(F, names=("p1", "l1"))
sage: p1, l1 = R.gens()
sage: define_luw_graph(F, [p1*l1]).dimension()
2
```

```
>>> from sage.all import *
>>> from sage.graphs.generators.luw_graphs import define_luw_graph
>>> F = GF(Integer(3))
>>> R = PolynomialRing(F, names=("p1", "l1"))
>>> p1, l1 = R.gens()
>>> define_luw_graph(F, [p1*l1]).dimension()
2
```

equations()

Return the validated defining polynomials.

EXAMPLES:

```
sage: from sage.graphs.generators.luw_graphs import define_Akq
sage: define_Akq(3, 3).equations()
(p1*l1, p1*l2)
```

```
>>> from sage.all import *
>>> from sage.graphs.generators.luw_graphs import define_Akq
>>> define_Akq(Integer(3), Integer(3)).equations()
(p1*l1, p1*l2)
```

graph(*immutable=False*)

Build the full graph corresponding to `self`.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: from sage.graphs.generators.luw_graphs import define_WengerGraph
sage: G = define_WengerGraph(2, 3).graph()
sage: G.order(), G.size()
(54, 81)
sage: define_WengerGraph(2, 3).graph(immutable=True).is_immutable()
True
```

```
>>> from sage.all import *
>>> from sage.graphs.generators.luw_graphs import define_WengerGraph
>>> G = define_WengerGraph(Integer(2), Integer(3)).graph()
>>> G.order(), G.size()
(54, 81)
>>> define_WengerGraph(Integer(2), Integer(3)).graph(immutable=True).is_
↪immutable()
True
```

lift(*new_functions*, *name=None*)

Return the lifted algebraic definition obtained by appending equations.

INPUT:

- `new_functions` – iterable of Sage polynomials to append
- `name` – string (default: `None`); name of the new definition

EXAMPLES:

```
sage: from sage.graphs.generators.luw_graphs import define_luw_graph
sage: F = GF(3)
sage: R = PolynomialRing(F, names=("p1", "l1", "p2", "l2"))
sage: p1, l1, p2, l2 = R.gens()
sage: D = define_luw_graph(F, [p1*l1])
sage: D2 = D.lift([p1*l2])
sage: D2.equations()
(p1*l1, p1*l2)
```

```
>>> from sage.all import *
>>> from sage.graphs.generators.luw_graphs import define_luw_graph
>>> F = GF(Integer(3))
>>> R = PolynomialRing(F, names=("p1", "l1", "p2", "l2"))
>>> p1, l1, p2, l2 = R.gens()
>>> D = define_luw_graph(F, [p1*l1])
>>> D2 = D.lift([p1*l2])
>>> D2.equations()
(p1*l1, p1*l2)
```

line_count()

Return the number of line vertices.

EXAMPLES:

```
sage: from sage.graphs.generators.luw_graphs import define_luw_graph
sage: F = GF(3)
sage: R = PolynomialRing(F, names=("p1", "l1"))
sage: p1, l1 = R.gens()
sage: define_luw_graph(F, [p1*l1], B=[1, 2]).line_count()
6
```

```
>>> from sage.all import *
>>> from sage.graphs.generators.luw_graphs import define_luw_graph
>>> F = GF(Integer(3))
>>> R = PolynomialRing(F, names=("p1", "l1"))
>>> p1, l1 = R.gens()
>>> define_luw_graph(F, [p1*l1], B=[Integer(1), Integer(2)]).line_count()
6
```

line_first_coordinates()

Return the allowed first coordinates on the line side.

EXAMPLES:

```
sage: from sage.graphs.generators.luw_graphs import define_luw_graph
sage: F = GF(3)
sage: R = PolynomialRing(F, names=("p1", "l1"))
sage: p1, l1 = R.gens()
sage: define_luw_graph(F, [p1*l1], B=[1, 2]).line_first_coordinates()
(1, 2)
```

```
>>> from sage.all import *
>>> from sage.graphs.generators.luw_graphs import define_luw_graph
```

(continues on next page)

(continued from previous page)

```
>>> F = GF(Integer(3))
>>> R = PolynomialRing(F, names=("p1", "l1"))
>>> p1, l1 = R.gens()
>>> define_luw_graph(F, [p1*l1], B=[Integer(1), Integer(2)]).line_first_
↪coordinates()
(1, 2)
```

neighbors(*vertex*)

Return the neighbors of `vertex` without building the full graph.

INPUT:

- `vertex` – pair `(side, coordinates)` where `side` is either `"P"` or `"L"`

EXAMPLES:

```
sage: from sage.graphs.generators.luw_graphs import define_luw_graph
sage: F = GF(3)
sage: R = PolynomialRing(F, names=("p1", "l1"))
sage: p1, l1 = R.gens()
sage: D = define_luw_graph(F, [p1*l1])
sage: v = ("P", (F(0), F(0)))
sage: D.neighbors(v)
(('L', (0, 0)), ('L', (1, 0)), ('L', (2, 0)))
sage: G = D.graph()
sage: set(D.neighbors(v)) == set(G.neighbors(v))
True
```

```
>>> from sage.all import *
>>> from sage.graphs.generators.luw_graphs import define_luw_graph
>>> F = GF(Integer(3))
>>> R = PolynomialRing(F, names=("p1", "l1"))
>>> p1, l1 = R.gens()
>>> D = define_luw_graph(F, [p1*l1])
>>> v = ("P", (F(Integer(0)), F(Integer(0))))
>>> D.neighbors(v)
(('L', (0, 0)), ('L', (1, 0)), ('L', (2, 0)))
>>> G = D.graph()
>>> set(D.neighbors(v)) == set(G.neighbors(v))
True
```

order()

Return the number of vertices of the graph.

EXAMPLES:

```
sage: from sage.graphs.generators.luw_graphs import define_luw_graph
sage: F = GF(3)
sage: R = PolynomialRing(F, names=("p1", "l1"))
sage: p1, l1 = R.gens()
sage: define_luw_graph(F, [p1*l1]).order()
18
```

```
>>> from sage.all import *
>>> from sage.graphs.generators.luw_graphs import define_luw_graph
>>> F = GF(Integer(3))
>>> R = PolynomialRing(F, names=("p1", "l1"))
>>> p1, l1 = R.gens()
>>> define_luw_graph(F, [p1*l1]).order()
18
```

point_count()

Return the number of point vertices.

EXAMPLES:

```
sage: from sage.graphs.generators.luw_graphs import define_luw_graph
sage: F = GF(3)
sage: R = PolynomialRing(F, names=("p1", "l1"))
sage: p1, l1 = R.gens()
sage: define_luw_graph(F, [p1*l1], A=[0, 1]).point_count()
6
```

```
>>> from sage.all import *
>>> from sage.graphs.generators.luw_graphs import define_luw_graph
>>> F = GF(Integer(3))
>>> R = PolynomialRing(F, names=("p1", "l1"))
>>> p1, l1 = R.gens()
>>> define_luw_graph(F, [p1*l1], A=[Integer(0), Integer(1)]).point_count()
6
```

point_first_coordinates()

Return the allowed first coordinates on the point side.

EXAMPLES:

```
sage: from sage.graphs.generators.luw_graphs import define_luw_graph
sage: F = GF(3)
sage: R = PolynomialRing(F, names=("p1", "l1"))
sage: p1, l1 = R.gens()
sage: define_luw_graph(F, [p1*l1], A=[0, 1]).point_first_coordinates()
(0, 1)
```

```
>>> from sage.all import *
>>> from sage.graphs.generators.luw_graphs import define_luw_graph
>>> F = GF(Integer(3))
>>> R = PolynomialRing(F, names=("p1", "l1"))
>>> p1, l1 = R.gens()
>>> define_luw_graph(F, [p1*l1], A=[Integer(0), Integer(1)]).point_first_
↪coordinates()
(0, 1)
```

project_down(*x*, *name=None*)

Return the projection down obtained by removing the last `x` equations.

INPUT:

- `x` – positive integer; number of final equations to remove

- `name` – string (default: `None`); name of the new definition

EXAMPLES:

```
sage: from sage.graphs.generators.luw_graphs import define_luw_graph
sage: F = GF(3)
sage: R = PolynomialRing(F, names=("p1", "l1", "p2", "l2"))
sage: p1, l1, p2, l2 = R.gens()
sage: D = define_luw_graph(F, [p1*l1, p1*l2])
sage: D.project_down(1).equations()
(p1*l1,)
```

```
>>> from sage.all import *
>>> from sage.graphs.generators.luw_graphs import define_luw_graph
>>> F = GF(Integer(3))
>>> R = PolynomialRing(F, names=("p1", "l1", "p2", "l2"))
>>> p1, l1, p2, l2 = R.gens()
>>> D = define_luw_graph(F, [p1*l1, p1*l2])
>>> D.project_down(Integer(1)).equations()
(p1*l1,)
```

size()

Return the number of edges of the graph.

EXAMPLES:

```
sage: from sage.graphs.generators.luw_graphs import define_luw_graph
sage: F = GF(3)
sage: R = PolynomialRing(F, names=("p1", "l1"))
sage: p1, l1 = R.gens()
sage: define_luw_graph(F, [p1*l1]).size()
27
```

```
>>> from sage.all import *
>>> from sage.graphs.generators.luw_graphs import define_luw_graph
>>> F = GF(Integer(3))
>>> R = PolynomialRing(F, names=("p1", "l1"))
>>> p1, l1 = R.gens()
>>> define_luw_graph(F, [p1*l1]).size()
27
```

sage.graphs.generators.luw_graphs.**WengerGraph**(*m*, *q*, *A=None*, *B=None*, *immutable=False*)

Return the Wenger graph $W_m(q)$ described in [LW2026].

The point set and line set are $\mathbf{F}_q^{m+1}$. A point $(p_1, \ldots, p_{m+1})$ is adjacent to a line $(l_1, \ldots, l_{m+1})$ if

$$p_{i+1} + l_{i+1} = p_1 l_i, \qquad 1 \le i \le m.$$

INPUT:

- `m` – positive integer
- `q` – prime power
- `A` – iterable, single value, or `None` (default: `None`); allowed first coordinates on the point side
- `B` – iterable, single value, or `None` (default: `None`); allowed first coordinates on the line side

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

OUTPUT:

A Sage *Graph*.

EXAMPLES:

```
sage: G = graphs.WengerGraph(2, 3)
sage: G.order(), G.size()
(54, 81)
```

```
>>> from sage.all import *
>>> G = graphs.WengerGraph(Integer(2), Integer(3))
>>> G.order(), G.size()
(54, 81)
```

REFERENCES:

- [LW2026]

sage.graphs.generators.luw_graphs.**define_Akq**(*k*, *q*, *A=None*, *B=None*)

Return a descriptor for the graph $A(k, q)$ described in [LW2026].

The point set and line set are $\mathbf{F}_q^k$. A point $(p_1, \ldots, p_k)$ is adjacent to a line $(l_1, \ldots, l_k)$ if

$$p_i + l_i = \begin{cases} p_{i-1} l_1, & i \text{ even}, \\ p_1 l_{i-1}, & i \text{ odd}, \end{cases} \qquad 2 \le i \le k.$$

INPUT:

- `k` – integer at least 2
- `q` – prime power
- `A` – iterable, single value, or `None` (default: `None`); allowed first coordinates on the point side
- `B` – iterable, single value, or `None` (default: `None`); allowed first coordinates on the line side

OUTPUT:

A *LUWGraphDescriptor*.

EXAMPLES:

```
sage: from sage.graphs.generators.luw_graphs import define_Akq
sage: D = define_Akq(5, 3)
sage: D.dimension(), D.order(), D.size()
(5, 486, 729)
sage: D.graph().order()
486
```

```
>>> from sage.all import *
>>> from sage.graphs.generators.luw_graphs import define_Akq
>>> D = define_Akq(Integer(5), Integer(3))
>>> D.dimension(), D.order(), D.size()
(5, 486, 729)
>>> D.graph().order()
486
```

REFERENCES:

- [LW2026]

sage.graphs.generators.luw_graphs.**define_Dkq**(*k*, *q*, *A=None*, *B=None*)

Return a descriptor for the graph $D(k, q)$ described in [LW2026].

The point set and line set are $\mathbf{F}_q^k$. A point $(p_1, \ldots, p_k)$ is adjacent to a line $(l_1, \ldots, l_k)$ if

$$p_2 + l_2 = p_1 l_1,$$

and, when $k \geq 3$,

$$p_3 + l_3 = p_1 l_2.$$

For $4 \leq i \leq k$, the remaining equations are

$$p_i + l_i = \begin{cases} p_{i-2} l_1, & i \equiv 0, 1 \pmod 4, \\ p_1 l_{i-2}, & i \equiv 2, 3 \pmod 4. \end{cases}$$

INPUT:

- `k` – integer at least 2
- `q` – prime power
- `A` – iterable, single value, or `None` (default: `None`); allowed first coordinates on the point side
- `B` – iterable, single value, or `None` (default: `None`); allowed first coordinates on the line side

OUTPUT:

A *LUWGraphDescriptor*.

EXAMPLES:

```
sage: from sage.graphs.generators.luw_graphs import define_Dkq
sage: D = define_Dkq(5, 3)
sage: D.dimension(), D.order(), D.size()
(5, 486, 729)
sage: D.graph().size()
729
```

```
>>> from sage.all import *
>>> from sage.graphs.generators.luw_graphs import define_Dkq
>>> D = define_Dkq(Integer(5), Integer(3))
>>> D.dimension(), D.order(), D.size()
(5, 486, 729)
>>> D.graph().size()
729
```

REFERENCES:

- [LW2026]

sage.graphs.generators.luw_graphs.**define_WengerGraph**(*m*, *q*, *A=None*, *B=None*)

Return a descriptor for the Wenger graph $W_m(q)$ described in [LW2026].

The point set and line set are $\mathbf{F}_q^{m+1}$. A point $(p_1, \ldots, p_{m+1})$ is adjacent to a line $(l_1, \ldots, l_{m+1})$ if

$$p_{i+1} + l_{i+1} = p_1 l_i, \qquad 1 \leq i \leq m.$$

INPUT:

- `m` – positive integer
- `q` – prime power
- `A` – iterable, single value, or `None` (default: `None`); allowed first coordinates on the point side
- `B` – iterable, single value, or `None` (default: `None`); allowed first coordinates on the line side

OUTPUT:

A *LUWGraphDescriptor*.

EXAMPLES:

```
sage: from sage.graphs.generators.luw_graphs import define_WengerGraph
sage: D = define_WengerGraph(2, 3)
sage: D.dimension(), D.order(), D.size()
(3, 54, 81)
sage: D.graph().order()
54
```

```
>>> from sage.all import *
>>> from sage.graphs.generators.luw_graphs import define_WengerGraph
>>> D = define_WengerGraph(Integer(2), Integer(3))
>>> D.dimension(), D.order(), D.size()
(3, 54, 81)
>>> D.graph().order()
54
```

REFERENCES:

- [LW2026]

sage.graphs.generators.luw_graphs.**define_luw_graph**(*ring*, *equations*, *name=None*, *A=None*, *B=None*, *point_coordinate_sets=None*, *line_coordinate_sets=None*)

Return a validated algebraic descriptor for an LUW graph.

The general construction is surveyed in [LW2026].

INPUT:

- `ring` – finite commutative ring
- `equations` – nonempty iterable of Sage polynomials

  The polynomial variables are interpreted by their Sage generator names. Users should label Python variables in the same order as the polynomial ring generators, or bind them by name, for example with `R.gens_dict()`. If `p1` is accidentally bound to the generator named `p2`, the polynomial will be interpreted as using `p2`.
- `name` – string (default: `None`); name for the resulting definition
- `A` – iterable, single value, or `None` (default: `None`); allowed first coordinates on the point side
- `B` – iterable, single value, or `None` (default: `None`); allowed first coordinates on the line side
- `point_coordinate_sets` – iterable of coordinate sets or `None` (default: `None`)
- `line_coordinate_sets` – iterable of coordinate sets or `None` (default: `None`)

OUTPUT:

A *LUWGraphDescriptor*.

EXAMPLES:

```
sage: from sage.graphs.generators.luw_graphs import define_luw_graph
sage: F = GF(3)
sage: R = PolynomialRing(F, names=("p1", "l1", "p2", "l2"))
sage: p1, l1, p2, l2 = R.gens()
sage: D = define_luw_graph(F, [p1*l1, p1*l2], A=[0, 1], B=[1, 2])
sage: D.point_first_coordinates()
(0, 1)
sage: D.line_first_coordinates()
(1, 2)
sage: Z4 = Integers(4)
sage: R = PolynomialRing(Z4, names=("p1", "l1"))
sage: p1, l1 = R.gens()
sage: define_luw_graph(Z4, [p1*l1]).order()
32
```

```
>>> from sage.all import *
>>> from sage.graphs.generators.luw_graphs import define_luw_graph
>>> F = GF(Integer(3))
>>> R = PolynomialRing(F, names=("p1", "l1", "p2", "l2"))
>>> p1, l1, p2, l2 = R.gens()
>>> D = define_luw_graph(F, [p1*l1, p1*l2], A=[Integer(0), Integer(1)],␣
↪B=[Integer(1), Integer(2)])
>>> D.point_first_coordinates()
(0, 1)
>>> D.line_first_coordinates()
(1, 2)
>>> Z4 = Integers(Integer(4))
>>> R = PolynomialRing(Z4, names=("p1", "l1"))
>>> p1, l1 = R.gens()
>>> define_luw_graph(Z4, [p1*l1]).order()
32
```

REFERENCES:

- [LW2026]

2.7 Basic graphs

The methods defined here appear in *sage.graphs.graph_generators*.

sage.graphs.generators.basic.**BullGraph**(*immutable=False*)

Return a bull graph with 5 nodes.

A bull graph is named for its shape. It's a triangle with horns. See the Wikipedia article Bull_graph for more information.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

PLOTTING:

Upon construction, the position dictionary is filled to override the spring-layout algorithm. By convention, the bull graph is drawn as a triangle with the first node (0) on the bottom. The second and third nodes (1 and 2) complete the triangle. Node 3 is the horn connected to 1 and node 4 is the horn connected to node 2.

EXAMPLES:

Construct and show a bull graph:

```
sage: g = graphs.BullGraph(); g
Bull graph: Graph on 5 vertices
sage: g.show()                              # long time                              #␣
↪needs sage.plot
```

```
>>> from sage.all import *
>>> g = graphs.BullGraph(); g
Bull graph: Graph on 5 vertices
>>> g.show()                              # long time                              #␣
↪needs sage.plot
```

The bull graph has 5 vertices and 5 edges. Its radius is 2, its diameter 3, and its girth 3. The bull graph is planar with chromatic number 3 and chromatic index also 3:

```
sage: g.order(); g.size()
5
5
sage: g.radius(); g.diameter(); g.girth()
2
3
3
sage: g.chromatic_number()
3
```

```
>>> from sage.all import *
>>> g.order(); g.size()
5
5
>>> g.radius(); g.diameter(); g.girth()
2
3
3
>>> g.chromatic_number()
3
```

The bull graph has chromatic polynomial $x(x-2)(x-1)^3$ and Tutte polynomial $x^4+x^3+x^2y$. Its characteristic polynomial is $x(x^2-x-3)(x^2+x-1)$, which follows from the definition of characteristic polynomials for graphs, i.e. $\det(xI-A)$, where x is a variable, A the adjacency matrix of the graph, and I the identity matrix of the same dimensions as A:

```
sage: chrompoly = g.chromatic_polynomial()
sage: x = chrompoly.parent()('x')
sage: x * (x - 2) * (x - 1)^3 == chrompoly
True

sage: charpoly = g.characteristic_polynomial()
sage: M = g.adjacency_matrix(); M
[0 1 1 0 0]
[1 0 1 1 0]
[1 1 0 0 1]
```

(continues on next page)

(continued from previous page)

```
[0 1 0 0 0]
[0 0 1 0 0]
sage: Id = identity_matrix(ZZ, M.nrows())
sage: D = x*Id - M
sage: D.determinant() == charpoly                                           #␣
↪needs sage.symbolic
True
sage: x * (x^2 - x - 3) * (x^2 + x - 1) == charpoly
True
```

```
>>> from sage.all import *
>>> chrompoly = g.chromatic_polynomial()
>>> x = chrompoly.parent()('x')
>>> x * (x - Integer(2)) * (x - Integer(1))**Integer(3) == chrompoly
True

>>> charpoly = g.characteristic_polynomial()
>>> M = g.adjacency_matrix(); M
[0 1 1 0 0]
[1 0 1 1 0]
[1 1 0 0 1]
[0 1 0 0 0]
[0 0 1 0 0]
>>> Id = identity_matrix(ZZ, M.nrows())
>>> D = x*Id - M
>>> D.determinant() == charpoly                                             #␣
↪needs sage.symbolic
True
>>> x * (x**Integer(2) - x - Integer(3)) * (x**Integer(2) + x - Integer(1)) ==␣
↪charpoly
True
```

sage.graphs.generators.basic.**ButterflyGraph**(*immutable=False*)

Return the butterfly graph.

Let C_3 be the cycle graph on 3 vertices. The butterfly or bowtie graph is obtained by joining two copies of C_3 at a common vertex, resulting in a graph that is isomorphic to the friendship graph F_2. See the Wikipedia article Butterfly_graph for more information.

See also

- *FriendshipGraph()*

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

The butterfly graph is a planar graph on 5 vertices and having 6 edges:

```
sage: G = graphs.ButterflyGraph(); G
Butterfly graph: Graph on 5 vertices
sage: G.show()                              # long time                              #␣
↪needs sage.plot
sage: G.is_planar()
True
sage: G.order()
5
sage: G.size()
6
```

```
>>> from sage.all import *
>>> G = graphs.ButterflyGraph(); G
Butterfly graph: Graph on 5 vertices
>>> G.show()                              # long time                              #␣
↪needs sage.plot
>>> G.is_planar()
True
>>> G.order()
5
>>> G.size()
6
```

It has diameter 2, girth 3, and radius 1:

```
sage: G.diameter()
2
sage: G.girth()
3
sage: G.radius()
1
```

```
>>> from sage.all import *
>>> G.diameter()
2
>>> G.girth()
3
>>> G.radius()
1
```

The butterfly graph is Eulerian, with chromatic number 3:

```
sage: G.is_eulerian()
True
sage: G.chromatic_number()
3
```

```
>>> from sage.all import *
>>> G.is_eulerian()
True
>>> G.chromatic_number()
3
```

sage.graphs.generators.basic.**CircularLadderGraph**(*n*, *immutable=False*)

Return a circular ladder graph with $2 * n$ nodes.

A Circular ladder graph is a ladder graph that is connected at the ends, i.e.: a ladder bent around so that top meets bottom. Thus it can be described as two parallel cycle graphs connected at each corresponding node pair.

PLOTTING: Upon construction, the position dictionary is filled to override the spring-layout algorithm. By convention, the circular ladder graph is displayed as an inner and outer cycle pair, with the first n nodes drawn on the inner circle. The first (0) node is drawn at the top of the inner-circle, moving clockwise after that. The outer circle is drawn with the $(n + 1)$-th node at the top, then counterclockwise as well. When $n == 2$, we rotate the outer circle by an angle of $\pi/8$ to ensure that all edges are visible (otherwise the 4 vertices of the graph would be placed on a single line).

INPUT:

- `n` – nonnegative integer
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

Construct and show a circular ladder graph with 26 nodes:

```
sage: g = graphs.CircularLadderGraph(13)
sage: g.show()                              # long time                          #␣
↪needs sage.plot
```

```
>>> from sage.all import *
>>> g = graphs.CircularLadderGraph(Integer(13))
>>> g.show()                              # long time                          #␣
↪needs sage.plot
```

Create several circular ladder graphs in a Sage graphics array:

```
sage: g = []
sage: j = []
sage: for i in range(9):
....:     k = graphs.CircularLadderGraph(i+3)
....:     g.append(k)
sage: for i in range(3):                                                       #␣
↪needs sage.plot
....:     n = []
....:     for m in range(3):
....:         n.append(g[3*i + m].plot(vertex_size=50, vertex_labels=False))
....:     j.append(n)
sage: G = graphics_array(j)                                                    #␣
↪needs sage.plot
sage: G.show()                              # long time                          #␣
↪needs sage.plot
```

```
>>> from sage.all import *
>>> g = []
>>> j = []
>>> for i in range(Integer(9)):
...     k = graphs.CircularLadderGraph(i+Integer(3))
...     g.append(k)
```

(continues on next page)

(continued from previous page)

```
>>> for i in range(Integer(3)):                                                 ␣
↪     # needs sage.plot
...     n = []
...     for m in range(Integer(3)):
...         n.append(g[Integer(3)*i + m].plot(vertex_size=Integer(50), vertex_
↪labels=False))
...     j.append(n)
>>> G = graphics_array(j)                                                      #␣
↪needs sage.plot
>>> G.show()                            # long time                           #␣
↪needs sage.plot
```

sage.graphs.generators.basic.**ClawGraph**(*immutable=False*)

Return a claw graph.

A claw graph is named for its shape. It is actually a complete bipartite graph with `(n1, n2) = (1, 3)`.

PLOTTING: See *CompleteBipartiteGraph()*.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

Show a Claw graph:

```
sage: (graphs.ClawGraph()).show()        # long time                          #␣
↪needs sage.plot
```

```
>>> from sage.all import *
>>> (graphs.ClawGraph()).show()        # long time                           #␣
↪needs sage.plot
```

Inspect a Claw graph:

```
sage: G = graphs.ClawGraph()
sage: G
Claw graph: Graph on 4 vertices
```

```
>>> from sage.all import *
>>> G = graphs.ClawGraph()
>>> G
Claw graph: Graph on 4 vertices
```

sage.graphs.generators.basic.**CompleteBipartiteGraph**(*p*, *q*, *set_position=True*, *immutable=False*, *name=None*)

Return a Complete Bipartite Graph on $p + q$ vertices.

A Complete Bipartite Graph is a graph with its vertices partitioned into two groups, $V_1 = \{0, ..., p-1\}$ and $V_2 = \{p, ..., p+q-1\}$. Each $u \in V_1$ is connected to every $v \in V_2$.

INPUT:

- `p`, `q` – number of vertices in each side

- `set_position` – boolean (default: `True`); if set to `True`, we assign positions to the vertices so that the set of cardinality p is on the line $y = 1$ and the set of cardinality q is on the line $y = 0$.
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph
- `name` – string (default: `None`); used as the name of the returned graph when set

PLOTTING: Upon construction, the position dictionary is filled to override the spring-layout algorithm. By convention, each complete bipartite graph will be displayed with the first p nodes on the top row (at $y = 1$) from left to right. The remaining q nodes appear at $y = 0$, also from left to right. The shorter row (partition with fewer nodes) is stretched to the same length as the longer row, unless the shorter row has 1 node; in which case it is centered. The x values in the plot are in domain $[0, \max(p, q)]$.

In the Complete Bipartite graph, there is a visual difference in using the spring-layout algorithm vs. the position dictionary used in this constructor. The position dictionary flattens the graph and separates the partitioned nodes, making it clear which nodes an edge is connected to. The Complete Bipartite graph plotted with the spring-layout algorithm tends to center the nodes in p (see `spring_med` in examples below), thus overlapping its nodes and edges, making it typically hard to decipher.

Filling the position dictionary in advance adds $O(n)$ to the constructor. Feel free to race the constructors below in the examples section. The much larger difference is the time added by the spring-layout algorithm when plotting. (Also shown in the example below). The spring model is typically described as $O(n^3)$, as appears to be the case in the NetworkX source code.

EXAMPLES:

Two ways of constructing the complete bipartite graph, using different layout algorithms:

```
sage: # needs networkx
sage: import networkx
sage: n = networkx.complete_bipartite_graph(389, 157)   # long time
sage: spring_big = Graph(n)                # long time
sage: posdict_big = graphs.CompleteBipartiteGraph(389, 157)         # long time
```

```
>>> from sage.all import *
>>> # needs networkx
>>> import networkx
>>> n = networkx.complete_bipartite_graph(Integer(389), Integer(157))    # long␣
↪time
>>> spring_big = Graph(n)                # long time
>>> posdict_big = graphs.CompleteBipartiteGraph(Integer(389), Integer(157))      ␣
↪ # long time
```

Compare the plotting:

```
sage: n = networkx.complete_bipartite_graph(11, 17)                             #␣
↪needs networkx
sage: spring_med = Graph(n)                                                     #␣
↪needs networkx
sage: posdict_med = graphs.CompleteBipartiteGraph(11, 17)
```

```
>>> from sage.all import *
>>> n = networkx.complete_bipartite_graph(Integer(11), Integer(17))              ␣
↪              # needs networkx
>>> spring_med = Graph(n)                                                      #␣
↪needs networkx
>>> posdict_med = graphs.CompleteBipartiteGraph(Integer(11), Integer(17))
```

Notice here how the spring-layout tends to center the nodes of $n1$:

```
sage: spring_med.show()                  # long time                              #␣
↪needs networkx
sage: posdict_med.show()                 # long time                              #␣
↪needs sage.plot
```

```
>>> from sage.all import *
>>> spring_med.show()                  # long time                              #␣
↪needs networkx
>>> posdict_med.show()                 # long time                              #␣
↪needs sage.plot
```

View many complete bipartite graphs with a Sage Graphics Array, with this constructor (i.e., the position dictionary filled):

```
sage: g = []
sage: j = []
sage: for i in range(9):
....:     k = graphs.CompleteBipartiteGraph(i+1,4)
....:     g.append(k)
sage: for i in range(3):                                                        #␣
↪needs sage.plot
....:     n = []
....:     for m in range(3):
....:         n.append(g[3*i + m].plot(vertex_size=50, vertex_labels=False))
....:     j.append(n)
sage: G = graphics_array(j)                                                     #␣
↪needs sage.plot
sage: G.show()                           # long time                              #␣
↪needs sage.plot
```

```
>>> from sage.all import *
>>> g = []
>>> j = []
>>> for i in range(Integer(9)):
...     k = graphs.CompleteBipartiteGraph(i+Integer(1),Integer(4))
...     g.append(k)
>>> for i in range(Integer(3)):                                                 ␣
↪     # needs sage.plot
...     n = []
...     for m in range(Integer(3)):
...         n.append(g[Integer(3)*i + m].plot(vertex_size=Integer(50), vertex_
↪labels=False))
...     j.append(n)
>>> G = graphics_array(j)                                                       #␣
↪needs sage.plot
>>> G.show()                           # long time                              #␣
↪needs sage.plot
```

We compare to plotting with the spring-layout algorithm:

```
sage: # needs networkx sage.plot
```

(continues on next page)

(continued from previous page)

```
sage: g = []
sage: j = []
sage: for i in range(9):
....:     spr = networkx.complete_bipartite_graph(i+1,4)
....:     k = Graph(spr)
....:     g.append(k)
sage: for i in range(3):
....:     n = []
....:     for m in range(3):
....:         n.append(g[3*i + m].plot(vertex_size=50, vertex_labels=False))
....:     j.append(n)
sage: G = graphics_array(j)
sage: G.show()                          # long time
```

```
>>> from sage.all import *
>>> # needs networkx sage.plot
>>> g = []
>>> j = []
>>> for i in range(Integer(9)):
...     spr = networkx.complete_bipartite_graph(i+Integer(1),Integer(4))
...     k = Graph(spr)
...     g.append(k)
>>> for i in range(Integer(3)):
...     n = []
...     for m in range(Integer(3)):
...         n.append(g[Integer(3)*i + m].plot(vertex_size=Integer(50), vertex_
↪labels=False))
...     j.append(n)
>>> G = graphics_array(j)
>>> G.show()                          # long time
```

Issue #12155:

```
sage: graphs.CompleteBipartiteGraph(5,6).complement()
complement(Complete bipartite graph of order 5+6): Graph on 11 vertices
```

```
>>> from sage.all import *
>>> graphs.CompleteBipartiteGraph(Integer(5),Integer(6)).complement()
complement(Complete bipartite graph of order 5+6): Graph on 11 vertices
```

sage.graphs.generators.basic.**CompleteGraph**(*n*, *immutable=False*)

Return a complete graph on n nodes.

A Complete Graph is a graph in which all nodes are connected to all other nodes.

INPUT:

- `n` – nonnegative integer; the number of vertices of the complete graph
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

PLOTTING: Upon construction, the position dictionary is filled to override the spring-layout algorithm. By convention, each complete graph will be displayed with the first (0) node at the top, with the rest following in a counterclockwise manner.

In the complete graph, there is a big difference visually in using the spring-layout algorithm vs. the position dictionary used in this constructor. The position dictionary flattens the graph, making it clear which nodes an edge is connected to. But the complete graph offers a good example of how the spring-layout works. The edges push outward (everything is connected), causing the graph to appear as a 3-dimensional pointy ball. (See examples below).

EXAMPLES:

We view many Complete graphs with a Sage Graphics Array, first with this constructor (i.e., the position dictionary filled):

```
sage: g = []
sage: j = []
sage: for i in range(9):
....:     k = graphs.CompleteGraph(i+3)
....:     g.append(k)
sage: for i in range(3):
....:     n = []
....:     for m in range(3):
....:         n.append(g[3*i + m].plot(vertex_size=50, vertex_labels=False))
....:     j.append(n)
sage: G = graphics_array(j)
sage: G.show()                          # long time
```

```
>>> from sage.all import *
>>> g = []
>>> j = []
>>> for i in range(Integer(9)):
...     k = graphs.CompleteGraph(i+Integer(3))
...     g.append(k)
>>> for i in range(Integer(3)):
...     n = []
...     for m in range(Integer(3)):
...         n.append(g[Integer(3)*i + m].plot(vertex_size=Integer(50), vertex_
↪labels=False))
...     j.append(n)
>>> G = graphics_array(j)
>>> G.show()                          # long time
```

We compare to plotting with the spring-layout algorithm:

```
sage: # needs networkx sage.plot
sage: import networkx
sage: g = []
sage: j = []
sage: for i in range(9):
....:     spr = networkx.complete_graph(i+3)
....:     k = Graph(spr)
....:     g.append(k)
sage: for i in range(3):
....:     n = []
....:     for m in range(3):
....:         n.append(g[3*i + m].plot(vertex_size=50, vertex_labels=False))
....:     j.append(n)
```

(continues on next page)

(continued from previous page)

```
sage: G = graphics_array(j)
sage: G.show()                              # long time
```

```
>>> from sage.all import *
>>> # needs networkx sage.plot
>>> import networkx
>>> g = []
>>> j = []
>>> for i in range(Integer(9)):
...     spr = networkx.complete_graph(i+Integer(3))
...     k = Graph(spr)
...     g.append(k)
>>> for i in range(Integer(3)):
...     n = []
...     for m in range(Integer(3)):
...         n.append(g[Integer(3)*i + m].plot(vertex_size=Integer(50), vertex_
↪labels=False))
...     j.append(n)
>>> G = graphics_array(j)
>>> G.show()                              # long time
```

Compare the constructors (results will vary):

```
sage: # needs networkx
sage: import networkx
sage: t = cputime()
sage: n = networkx.complete_graph(389); spring389 = Graph(n)
sage: cputime(t)  # random
0.59203700000000126
sage: t = cputime()
sage: posdict389 = graphs.CompleteGraph(389)
sage: cputime(t)  # random
0.6680419999999998
```

```
>>> from sage.all import *
>>> # needs networkx
>>> import networkx
>>> t = cputime()
>>> n = networkx.complete_graph(Integer(389)); spring389 = Graph(n)
>>> cputime(t)  # random
0.59203700000000126
>>> t = cputime()
>>> posdict389 = graphs.CompleteGraph(Integer(389))
>>> cputime(t)  # random
0.6680419999999998
```

We compare plotting:

```
sage: # needs networkx
sage: import networkx
sage: n = networkx.complete_graph(23)
sage: spring23 = Graph(n)
```

(continues on next page)

(continued from previous page)

```
sage: posdict23 = graphs.CompleteGraph(23)
sage: spring23.show()                    # long time                          #␣
↪needs sage.plot
sage: posdict23.show()                   # long time                          #␣
↪needs sage.plot
```

```
>>> from sage.all import *
>>> # needs networkx
>>> import networkx
>>> n = networkx.complete_graph(Integer(23))
>>> spring23 = Graph(n)
>>> posdict23 = graphs.CompleteGraph(Integer(23))
>>> spring23.show()                    # long time                          #␣
↪needs sage.plot
>>> posdict23.show()                   # long time                          #␣
↪needs sage.plot
```

sage.graphs.generators.basic.**CompleteMultipartiteGraph**(*L*, *immutable=False*)

Return a complete multipartite graph.

INPUT:

- `L` – list of integers; the respective sizes of the components
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

PLOTTING: Produce a layout of the vertices so that vertices in the same vertex set are adjacent and clearly separated from vertices in other vertex sets.

This is done by calculating the vertices of an r-gon then calculating the slope between adjacent vertices. We then 'walk' around the r-gon placing graph vertices in regular intervals between adjacent vertices of the r-gon.

Makes a nicely organized graph like in this picture: https://commons.wikimedia.org/wiki/File:Turan_13-4.svg

EXAMPLES:

A complete tripartite graph with sets of sizes $5, 6, 8$:

```
sage: g = graphs.CompleteMultipartiteGraph([5, 6, 8]); g
Multipartite Graph with set sizes [5, 6, 8]: Graph on 19 vertices
```

```
>>> from sage.all import *
>>> g = graphs.CompleteMultipartiteGraph([Integer(5), Integer(6), Integer(8)]); g
Multipartite Graph with set sizes [5, 6, 8]: Graph on 19 vertices
```

It clearly has a chromatic number of 3:

```
sage: g.chromatic_number()
3
```

```
>>> from sage.all import *
>>> g.chromatic_number()
3
```

sage.graphs.generators.basic.**CorrelationGraph**(*seqs*, *alpha*, *include_anticorrelation*, *immutable=False*)

Return a correlation graph with a node per sequence in `seqs`.

Edges are added between nodes where the corresponding sequences have a correlation coefficient greater than alpha.

If `include_anticorrelation` is `True`, then edges are also added between nodes with correlation coefficient less than `-alpha`.

INPUT:

- `seqs` – list of sequences, that is a list of lists
- `alpha` – float; threshold on the correlation coefficient between two sequences for adding an edge
- `include_anticorrelation` – boolean; whether to add edges between nodes with correlation coefficient less than `-alpha` or not
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: # needs numpy
sage: from sage.graphs.generators.basic import CorrelationGraph
sage: data = [[1,2,3], [4,5,6], [7,8,9999]]
sage: CG1 = CorrelationGraph(data, 0.9, False)
sage: CG2 = CorrelationGraph(data, 0.9, True)
sage: CG3 = CorrelationGraph(data, 0.1, True)
sage: CG1.edges(sort=False)
[(0, 0, None), (0, 1, None), (1, 1, None), (2, 2, None)]
sage: CG2.edges(sort=False)
[(0, 0, None), (0, 1, None), (1, 1, None), (2, 2, None)]
sage: CG3.edges(sort=False)
[(0, 0, None), (0, 1, None), (0, 2, None), (1, 1, None), (1, 2, None), (2, 2,␣
↪None)]
```

```
>>> from sage.all import *
>>> # needs numpy
>>> from sage.graphs.generators.basic import CorrelationGraph
>>> data = [[Integer(1),Integer(2),Integer(3)], [Integer(4),Integer(5),
↪Integer(6)], [Integer(7),Integer(8),Integer(9999)]]
>>> CG1 = CorrelationGraph(data, RealNumber('0.9'), False)
>>> CG2 = CorrelationGraph(data, RealNumber('0.9'), True)
>>> CG3 = CorrelationGraph(data, RealNumber('0.1'), True)
>>> CG1.edges(sort=False)
[(0, 0, None), (0, 1, None), (1, 1, None), (2, 2, None)]
>>> CG2.edges(sort=False)
[(0, 0, None), (0, 1, None), (1, 1, None), (2, 2, None)]
>>> CG3.edges(sort=False)
[(0, 0, None), (0, 1, None), (0, 2, None), (1, 1, None), (1, 2, None), (2, 2,␣
↪None)]
```

sage.graphs.generators.basic.**CycleGraph**(*n*, *immutable=False*)

Return a cycle graph with n nodes.

A cycle graph is a basic structure which is also typically called an n-gon.

INPUT:

- `n` – nonnegative integer; the number of vertices of the cycle graph

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

PLOTTING: Upon construction, the position dictionary is filled to override the spring-layout algorithm. By convention, each cycle graph will be displayed with the first (0) node at the top, with the rest following in a counterclockwise manner.

The cycle graph is a good opportunity to compare efficiency of filling a position dictionary vs. using the spring-layout algorithm for plotting. Because the cycle graph is very symmetric, the resulting plots should be similar (in cases of small n).

Filling the position dictionary in advance adds $O(n)$ to the constructor.

EXAMPLES:

Compare plotting using the predefined layout and networkx:

```
sage: # needs networkx sage.plot
sage: import networkx
sage: n = networkx.cycle_graph(23)
sage: spring23 = Graph(n)
sage: posdict23 = graphs.CycleGraph(23)
sage: spring23.show()                    # long time
sage: posdict23.show()                   # long time
```

```
>>> from sage.all import *
>>> # needs networkx sage.plot
>>> import networkx
>>> n = networkx.cycle_graph(Integer(23))
>>> spring23 = Graph(n)
>>> posdict23 = graphs.CycleGraph(Integer(23))
>>> spring23.show()                    # long time
>>> posdict23.show()                   # long time
```

We next view many cycle graphs as a Sage graphics array. First we use the `CycleGraph` constructor, which fills in the position dictionary:

```
sage: # needs networkx sage.plot
sage: g = []
sage: j = []
sage: for i in range(9):
....:     k = graphs.CycleGraph(i+3)
....:     g.append(k)
sage: for i in range(3):
....:     n = []
....:     for m in range(3):
....:         n.append(g[3*i + m].plot(vertex_size=50, vertex_labels=False))
....:     j.append(n)
sage: G = graphics_array(j)
sage: G.show()                           # long time
```

```
>>> from sage.all import *
>>> # needs networkx sage.plot
>>> g = []
>>> j = []
>>> for i in range(Integer(9)):
...     k = graphs.CycleGraph(i+Integer(3))
```

(continues on next page)

(continued from previous page)

```
...       g.append(k)
>>> for i in range(Integer(3)):
...       n = []
...       for m in range(Integer(3)):
...           n.append(g[Integer(3)*i + m].plot(vertex_size=Integer(50), vertex_
↪labels=False))
...       j.append(n)
>>> G = graphics_array(j)
>>> G.show()                            # long time
```

Compare to plotting with the spring-layout algorithm:

```
sage: # needs networkx sage.plot
sage: g = []
sage: j = []
sage: for i in range(9):
....:     spr = networkx.cycle_graph(i+3)
....:     k = Graph(spr)
....:     g.append(k)
sage: for i in range(3):
....:     n = []
....:     for m in range(3):
....:         n.append(g[3*i + m].plot(vertex_size=50, vertex_labels=False))
....:     j.append(n)
sage: G = graphics_array(j)
sage: G.show()                             # long time
```

```
>>> from sage.all import *
>>> # needs networkx sage.plot
>>> g = []
>>> j = []
>>> for i in range(Integer(9)):
...       spr = networkx.cycle_graph(i+Integer(3))
...       k = Graph(spr)
...       g.append(k)
>>> for i in range(Integer(3)):
...       n = []
...       for m in range(Integer(3)):
...           n.append(g[Integer(3)*i + m].plot(vertex_size=Integer(50), vertex_
↪labels=False))
...       j.append(n)
>>> G = graphics_array(j)
>>> G.show()                            # long time
```

sage.graphs.generators.basic.**DartGraph**(*immutable=False*)

Return a dart graph with 5 nodes.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

PLOTTING: Upon construction, the position dictionary is filled to override the spring-layout algorithm. By convention, the dart graph is drawn as a dart, with the sharp part on the bottom.

EXAMPLES:

Construct and show a dart graph:

```
sage: g = graphs.DartGraph()
sage: g.show()                                # long time                    #␣
↪needs sage.plot
```

```
>>> from sage.all import *
>>> g = graphs.DartGraph()
>>> g.show()                                # long time                    #␣
↪needs sage.plot
```

sage.graphs.generators.basic.**DiamondGraph**(*immutable=False*)

Return a diamond graph with 4 nodes.

A diamond graph is a square with one pair of diagonal nodes connected.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

PLOTTING: Upon construction, the position dictionary is filled to override the spring-layout algorithm. By convention, the diamond graph is drawn as a diamond, with the first node on top, second on the left, third on the right, and fourth on the bottom; with the second and third node connected.

EXAMPLES:

Construct and show a diamond graph:

```
sage: g = graphs.DiamondGraph()
sage: g.show()                                # long time                    #␣
↪needs sage.plot
```

```
>>> from sage.all import *
>>> g = graphs.DiamondGraph()
>>> g.show()                                # long time                    #␣
↪needs sage.plot
```

sage.graphs.generators.basic.**EmptyGraph**(*immutable=False*)

Return an empty graph (0 nodes and 0 edges).

This is useful for constructing graphs by adding edges and vertices individually or in a loop.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

PLOTTING: When plotting, this graph will use the default spring-layout algorithm, unless a position dictionary is specified.

EXAMPLES:

Add one vertex to an empty graph and then show:

```
sage: empty1 = graphs.EmptyGraph()
sage: empty1.add_vertex()
0
sage: empty1.show()                           # long time                    #␣
↪needs sage.plot
```

```
>>> from sage.all import *
>>> empty1 = graphs.EmptyGraph()
>>> empty1.add_vertex()
0
>>> empty1.show()                          # long time                                #␣
↪needs sage.plot
```

Use for loops to build a graph from an empty graph:

```
sage: empty2 = graphs.EmptyGraph()
sage: for i in range(5):
....:     empty2.add_vertex()  # add 5 nodes, labeled 0-4
0
1
2
3
4
sage: for i in range(3):
....:     empty2.add_edge(i,i+1)  # add edges {[0:1],[1:2],[2:3]}
sage: for i in range(1, 4):
....:     empty2.add_edge(4,i)  # add edges {[1:4],[2:4],[3:4]}
sage: empty2.show()                       # long time                              #␣
↪needs sage.plot
```

```
>>> from sage.all import *
>>> empty2 = graphs.EmptyGraph()
>>> for i in range(Integer(5)):
...     empty2.add_vertex()  # add 5 nodes, labeled 0-4
0
1
2
3
4
>>> for i in range(Integer(3)):
...     empty2.add_edge(i,i+Integer(1))  # add edges {[0:1],[1:2],[2:3]}
>>> for i in range(Integer(1), Integer(4)):
...     empty2.add_edge(Integer(4),i)  # add edges {[1:4],[2:4],[3:4]}
>>> empty2.show()                       # long time                                #␣
↪needs sage.plot
```

sage.graphs.generators.basic.**ForkGraph**(*immutable=False*)

Return a fork graph with 5 nodes.

A fork graph, sometimes also called chair graph, is 5 vertex tree.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

PLOTTING: Upon construction, the position dictionary is filled to override the spring-layout algorithm. By convention, the fork graph is drawn as a fork, with the sharp part on the bottom.

EXAMPLES:

Construct and show a fork graph:

```
sage: g = graphs.ForkGraph()
sage: g.show()                              # long time                          #␣
↪needs sage.plot
```

```
>>> from sage.all import *
>>> g = graphs.ForkGraph()
>>> g.show()                              # long time                          #␣
↪needs sage.plot
```

sage.graphs.generators.basic.**GemGraph**(*immutable=False*)

Return a gem graph with 5 nodes.

A gem graph is a fan graph (4,1).

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

PLOTTING: Upon construction, the position dictionary is filled to override the spring-layout algorithm. By convention, the gem graph is drawn as a gem, with the sharp part on the bottom.

EXAMPLES:

Construct and show a gem graph:

```
sage: g = graphs.GemGraph()
sage: g.show()                              # long time                          #␣
↪needs sage.plot
```

```
>>> from sage.all import *
>>> g = graphs.GemGraph()
>>> g.show()                              # long time                          #␣
↪needs sage.plot
```

sage.graphs.generators.basic.**Grid2dGraph**(*p*, *q*, *set_positions=True*, *immutable=False*, *name=None*)

Return a 2-dimensional grid graph with $p \times q$ nodes (p rows and q columns).

A 2d grid graph resembles a 2 dimensional grid. All inner nodes are connected to their 4 neighbors. Outer (non-corner) nodes are connected to their 3 neighbors. Corner nodes are connected to their 2 neighbors.

INPUT:

- `p`, `q` – two positive integers
- `set_positions` – boolean (default: `True`); whether to set the position of the nodes
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph
- `name` – string (default: `None`); used as the name of the returned graph when set

PLOTTING: Upon construction, the position dictionary is filled to override the spring-layout algorithm. By convention, nodes are labelled in (row, column) pairs with $(0, 0)$ in the top left corner. Edges will always be horizontal and vertical - another advantage of filling the position dictionary.

EXAMPLES:

Construct and show a grid 2d graph Rows = 5, Columns = 7:

```
sage: g = graphs.Grid2dGraph(5,7)
sage: g.show()                              # long time                          #␣
↪needs sage.plot
```

```
>>> from sage.all import *
>>> g = graphs.Grid2dGraph(Integer(5),Integer(7))
>>> g.show()                                # long time                          #␣
↪needs sage.plot
```

sage.graphs.generators.basic.**GridGraph**(*dim_list*, *immutable=False*)

Return an n-dimensional grid graph.

INPUT:

- `dim_list` – list of integers representing the number of nodes to extend in each dimension
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

PLOTTING: When plotting, this graph will use the default spring-layout algorithm, unless a position dictionary is specified.

EXAMPLES:

```
sage: G = graphs.GridGraph([2,3,4])
sage: G.show()                              # long time                          #␣
↪needs sage.plot
```

```
>>> from sage.all import *
>>> G = graphs.GridGraph([Integer(2),Integer(3),Integer(4)])
>>> G.show()                                # long time                          #␣
↪needs sage.plot
```

```
sage: C = graphs.CubeGraph(4)
sage: G = graphs.GridGraph([2,2,2,2])
sage: C.show()                              # long time                          #␣
↪needs sage.plot
sage: G.show()                              # long time                          #␣
↪needs sage.plot
```

```
>>> from sage.all import *
>>> C = graphs.CubeGraph(Integer(4))
>>> G = graphs.GridGraph([Integer(2),Integer(2),Integer(2),Integer(2)])
>>> C.show()                                # long time                          #␣
↪needs sage.plot
>>> G.show()                                # long time                          #␣
↪needs sage.plot
```

sage.graphs.generators.basic.**HouseGraph**(*immutable=False*)

Return a house graph with 5 nodes.

A house graph is named for its shape. It is a triangle (roof) over a square (walls).

PLOTTING: Upon construction, the position dictionary is filled to override the spring-layout algorithm. By convention, the house graph is drawn with the first node in the lower-left corner of the house, the second in the lower-right corner of the house. The third node is in the upper-left corner connecting the roof to the wall, and the

fourth is in the upper-right corner connecting the roof to the wall. The fifth node is the top of the roof, connected only to the third and fourth.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

Construct and show a house graph:

```
sage: g = graphs.HouseGraph()
sage: g.show()                              # long time                          #␣
↪needs sage.plot
```

```
>>> from sage.all import *
>>> g = graphs.HouseGraph()
>>> g.show()                              # long time                          #␣
↪needs sage.plot
```

sage.graphs.generators.basic.**HouseXGraph**(*immutable=False*)

Return a house X graph with 5 nodes.

A house X graph is a house graph with two additional edges. The upper-right corner is connected to the lower-left. And the upper-left corner is connected to the lower-right.

PLOTTING: Upon construction, the position dictionary is filled to override the spring-layout algorithm. By convention, the house X graph is drawn with the first node in the lower-left corner of the house, the second in the lower-right corner of the house. The third node is in the upper-left corner connecting the roof to the wall, and the fourth is in the upper-right corner connecting the roof to the wall. The fifth node is the top of the roof, connected only to the third and fourth.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

Construct and show a house X graph:

```
sage: g = graphs.HouseXGraph()
sage: g.show()                              # long time                          #␣
↪needs sage.plot
```

```
>>> from sage.all import *
>>> g = graphs.HouseXGraph()
>>> g.show()                              # long time                          #␣
↪needs sage.plot
```

sage.graphs.generators.basic.**LadderGraph**(*n*, *immutable=False*)

Return a ladder graph with $2 * n$ nodes.

A ladder graph is a basic structure that is typically displayed as a ladder, i.e.: two parallel path graphs connected at each corresponding node pair.

PLOTTING: Upon construction, the position dictionary is filled to override the spring-layout algorithm. By convention, each ladder graph will be displayed horizontally, with the first n nodes displayed left to right on the top horizontal line.

INPUT:

- `n` – a nonnegative integer; number of nodes is $2n$
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

Construct and show a ladder graph with 14 nodes:

```
sage: g = graphs.LadderGraph(7)
sage: g.show()                              # long time                              #␣
↪needs sage.plot
```

```
>>> from sage.all import *
>>> g = graphs.LadderGraph(Integer(7))
>>> g.show()                              # long time                              #␣
↪needs sage.plot
```

Create several ladder graphs in a Sage graphics array:

```
sage: g = []
sage: j = []
sage: for i in range(9):
....:     k = graphs.LadderGraph(i+2)
....:     g.append(k)
sage: for i in range(3):
....:     n = []
....:     for m in range(3):
....:         n.append(g[3*i + m].plot(vertex_size=50, vertex_labels=False))
....:     j.append(n)
sage: G = graphics_array(j)
sage: G.show()                              # long time
```

```
>>> from sage.all import *
>>> g = []
>>> j = []
>>> for i in range(Integer(9)):
...     k = graphs.LadderGraph(i+Integer(2))
...     g.append(k)
>>> for i in range(Integer(3)):
...     n = []
...     for m in range(Integer(3)):
...         n.append(g[Integer(3)*i + m].plot(vertex_size=Integer(50), vertex_
↪labels=False))
...     j.append(n)
>>> G = graphics_array(j)
>>> G.show()                              # long time
```

`sage.graphs.generators.basic.`**`MoebiusLadderGraph`**(*n*, *immutable=False*)

Return a Möbius ladder graph with $2n$ nodes

A Möbius ladder graph of order $2n$ is a ladder graph of the same order that is connected at the ends with a single twist, i.e., a ladder graph bent around so that top meets bottom with a single twist. Alternatively, it can be described as a single cycle graph (of order $2n$) with the addition of edges (called $rungs$) joining the antipodal pairs of nodes. Also, note that the Möbius ladder graph `graphs.MoebiusLadderGraph(n)` is precisely the same graph as the circulant graph `graphs.CirculantGraph(2 * n, [1, n])`.

PLOTTING:

Upon construction, the position dictionary is filled to override the spring-layout algorithm. By convention, each Möbius ladder graph will be displayed with the first (0) node at the top, with the rest following in a counterclockwise manner.

INPUT:

- `n` – a nonnegative integer; number of nodes is $2n$
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

OUTPUT:

- `G` – a Möbius ladder graph of order $2n$; note that a `ValueError` is returned if $n < 0$

EXAMPLES:

Construct and show a Möbius ladder graph with 26 nodes:

```
sage: g = graphs.MoebiusLadderGraph(13)
sage: g.show()                              # long time                          #␣
↪needs sage.plot
```

```
>>> from sage.all import *
>>> g = graphs.MoebiusLadderGraph(Integer(13))
>>> g.show()                              # long time                          #␣
↪needs sage.plot
```

Create several Möbius ladder graphs in a Sage graphics array:

```
sage: g = []
sage: j = []
sage: for i in range(9):
....:     k = graphs.MoebiusLadderGraph(i+3)
....:     g.append(k)
sage: for i in range(3):
....:     n = []
....:     for m in range(3):
....:         n.append(g[3*i + m].plot(vertex_size=50, vertex_labels=False))
....:     j.append(n)
sage: G = graphics_array(j)
sage: G.show()                              # long time
```

```
>>> from sage.all import *
>>> g = []
>>> j = []
>>> for i in range(Integer(9)):
...     k = graphs.MoebiusLadderGraph(i+Integer(3))
...     g.append(k)
>>> for i in range(Integer(3)):
...     n = []
...     for m in range(Integer(3)):
...         n.append(g[Integer(3)*i + m].plot(vertex_size=Integer(50), vertex_
↪labels=False))
...     j.append(n)
```

(continues on next page)

(continued from previous page)

```
>>> G = graphics_array(j)
>>> G.show()                              # long time
```

REFERENCES:

- Wikipedia article Möbius_ladder

See also

LadderGraph(), *CircularLadderGraph()*, *CirculantGraph()*

AUTHORS:

- Janmenjaya Panda (2024-05-26)

sage.graphs.generators.basic.**PathGraph**(*n*, *pos=None*, *immutable=False*, *name=None*)

Return a path graph with n nodes.

A path graph is a graph where all inner nodes are connected to their two neighbors and the two end-nodes are connected to their one inner neighbors (i.e.: a cycle graph without the first and last node connected).

INPUT:

- `n` – nonnegative integer; number of nodes of the path graph
- `pos` – string (default: `None`); indicates the embedding to use between 'circle', 'line' or the default algorithm. See the plotting section below for more detail.
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph
- `name` – string (default: `None`); used as the name of the returned graph when set

PLOTTING: Upon construction, the position dictionary is filled to override the spring-layout algorithm. By convention, the graph may be drawn in one of two ways: The 'line' argument will draw the graph in a horizontal line (left to right) if there are less than 11 nodes. Otherwise the 'line' argument will append horizontal lines of length 10 nodes below, alternating left to right and right to left. The 'circle' argument will cause the graph to be drawn in a cycle-shape, with the first node at the top and then about the circle in a clockwise manner. By default (without an appropriate string argument) the graph will be drawn as a 'circle' if $10 < n < 41$ and as a 'line' for all other n.

EXAMPLES:

Show default drawing by size: 'line': $n \leq 10$:

```
sage: p = graphs.PathGraph(10)
sage: p.show()                              # long time                              #␣
↪needs sage.plot
```

```
>>> from sage.all import *
>>> p = graphs.PathGraph(Integer(10))
>>> p.show()                              # long time                              #␣
↪needs sage.plot
```

'circle': $10 < n < 41$:

```
sage: q = graphs.PathGraph(25)
sage: q.show()                              # long time                              #␣
↪needs sage.plot
```

```
>>> from sage.all import *
>>> q = graphs.PathGraph(Integer(25))
>>> q.show()                              # long time                          #␣
↪needs sage.plot
```

'line': $n \geq 41$:

```
sage: r = graphs.PathGraph(55)
sage: r.show()                             # long time                          #␣
↪needs sage.plot
```

```
>>> from sage.all import *
>>> r = graphs.PathGraph(Integer(55))
>>> r.show()                              # long time                          #␣
↪needs sage.plot
```

Override the default drawing:

```
sage: s = graphs.PathGraph(5,'circle')
sage: s.show()                             # long time                          #␣
↪needs sage.plot
```

```
>>> from sage.all import *
>>> s = graphs.PathGraph(Integer(5),'circle')
>>> s.show()                              # long time                          #␣
↪needs sage.plot
```

sage.graphs.generators.basic.**StarGraph**(*n*, *immutable=False*)

Return a star graph with $n + 1$ nodes.

A Star graph is a basic structure where one node is connected to all other nodes.

INPUT:

- `n` – a nonnegative integer; number of nodes is $n + 1$
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

PLOTTING: Upon construction, the position dictionary is filled to override the spring-layout algorithm. By convention, each star graph will be displayed with the first (0) node in the center, the second node (1) at the top, with the rest following in a counterclockwise manner. (0) is the node connected to all other nodes.

The star graph is a good opportunity to compare efficiency of filling a position dictionary vs. using the spring-layout algorithm for plotting. As far as display, the spring-layout should push all other nodes away from the (0) node, and thus look very similar to this constructor's positioning.

EXAMPLES:

```
sage: import networkx                                                          #␣
↪needs networkx
```

```
>>> from sage.all import *
>>> import networkx                                                            #␣
↪needs networkx
```

Compare the plots:

```
sage: # needs networkx sage.plot
sage: n = networkx.star_graph(23)
sage: spring23 = Graph(n)
sage: posdict23 = graphs.StarGraph(23)
sage: spring23.show()                    # long time
sage: posdict23.show()  # long time
```

```
>>> from sage.all import *
>>> # needs networkx sage.plot
>>> n = networkx.star_graph(Integer(23))
>>> spring23 = Graph(n)
>>> posdict23 = graphs.StarGraph(Integer(23))
>>> spring23.show()                    # long time
>>> posdict23.show()  # long time
```

View many star graphs as a Sage Graphics Array

With this constructor (i.e., the position dictionary filled)

```
sage: g = []
sage: j = []
sage: for i in range(9):
....:     k = graphs.StarGraph(i+3)
....:     g.append(k)
sage: for i in range(3):
....:     n = []
....:     for m in range(3):
....:         n.append(g[3*i + m].plot(vertex_size=50, vertex_labels=False))
....:     j.append(n)
sage: G = graphics_array(j)
sage: G.show()                           # long time
```

```
>>> from sage.all import *
>>> g = []
>>> j = []
>>> for i in range(Integer(9)):
...     k = graphs.StarGraph(i+Integer(3))
...     g.append(k)
>>> for i in range(Integer(3)):
...     n = []
...     for m in range(Integer(3)):
...         n.append(g[Integer(3)*i + m].plot(vertex_size=Integer(50), vertex_
↪labels=False))
...     j.append(n)
>>> G = graphics_array(j)
>>> G.show()                           # long time
```

Compared to plotting with the spring-layout algorithm

```
sage: # needs networkx sage.plot
sage: g = []
sage: j = []
sage: for i in range(9):
```

(continues on next page)

(continued from previous page)

```
....:     spr = networkx.star_graph(i+3)
....:     k = Graph(spr)
....:     g.append(k)
sage: for i in range(3):
....:     n = []
....:     for m in range(3):
....:         n.append(g[3*i + m].plot(vertex_size=50, vertex_labels=False))
....:     j.append(n)
sage: G = graphics_array(j)
sage: G.show()                          # long time
```

```
>>> from sage.all import *
>>> # needs networkx sage.plot
>>> g = []
>>> j = []
>>> for i in range(Integer(9)):
...     spr = networkx.star_graph(i+Integer(3))
...     k = Graph(spr)
...     g.append(k)
>>> for i in range(Integer(3)):
...     n = []
...     for m in range(Integer(3)):
...         n.append(g[Integer(3)*i + m].plot(vertex_size=Integer(50), vertex_
↪labels=False))
...     j.append(n)
>>> G = graphics_array(j)
>>> G.show()                          # long time
```

sage.graphs.generators.basic.**Toroidal6RegularGrid2dGraph**(*p*, *q*, *immutable=False*)

Return a toroidal 6-regular grid.

The toroidal 6-regular grid is a 6-regular graph on $p \times q$ vertices and its elements have coordinates (i, j) for $i \in \{0...p-1\}$ and $j \in \{0...q-1\}$.

Its edges are those of the *ToroidalGrid2dGraph()*, to which are added the edges between (i, j) and $((i+1)\%p, (j+1)\%q)$.

INPUT:

- `p`, `q` – integers
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

The toroidal 6-regular grid on 25 elements:

```
sage: g = graphs.Toroidal6RegularGrid2dGraph(5,5)
sage: g.is_regular(k=6)
True
sage: g.is_vertex_transitive()                                              #␣
↪needs sage.groups
True
sage: g.line_graph().is_vertex_transitive()                                 #␣
↪needs sage.groups
```

(continues on next page)

(continued from previous page)

```
True
sage: g.automorphism_group().cardinality()                                  #␣
↪needs sage.groups
300
sage: g.is_hamiltonian()                                                    #␣
↪needs sage.numerical.mip
True
```

```
>>> from sage.all import *
>>> g = graphs.Toroidal6RegularGrid2dGraph(Integer(5),Integer(5))
>>> g.is_regular(k=Integer(6))
True
>>> g.is_vertex_transitive()                                                #␣
↪needs sage.groups
True
>>> g.line_graph().is_vertex_transitive()                                   #␣
↪needs sage.groups
True
>>> g.automorphism_group().cardinality()                                    #␣
↪needs sage.groups
300
>>> g.is_hamiltonian()                                                      #␣
↪needs sage.numerical.mip
True
```

sage.graphs.generators.basic.**ToroidalGrid2dGraph**(*p*, *q*, *immutable=False*)

Return a toroidal 2-dimensional grid graph with $p \times q$ nodes (p rows and q columns).

The toroidal 2-dimensional grid with parameters p, q is the 2-dimensional grid graph with identical parameters to which are added the edges $((i, 0), (i, q-1))$ and $((0, i), (p-1, i))$.

INPUT:

- `p, q` – nonnegative integers; the sides of the toroidal grid
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

The toroidal 2-dimensional grid is a regular graph, while the usual 2-dimensional grid is not

```
sage: tgrid = graphs.ToroidalGrid2dGraph(8,9)
sage: print(tgrid)
Toroidal 2D Grid Graph with parameters 8,9
sage: grid = graphs.Grid2dGraph(8,9)
sage: grid.is_regular()
False
sage: tgrid.is_regular()
True
```

```
>>> from sage.all import *
>>> tgrid = graphs.ToroidalGrid2dGraph(Integer(8),Integer(9))
>>> print(tgrid)
Toroidal 2D Grid Graph with parameters 8,9
```

(continues on next page)

(continued from previous page)

```
>>> grid = graphs.Grid2dGraph(Integer(8),Integer(9))
>>> grid.is_regular()
False
>>> tgrid.is_regular()
True
```

2.8 Chessboard graphs

The methods defined here appear in *sage.graphs.graph_generators*.

- *BishopGraph*
- *KingGraph*
- *KnightGraph*
- *QueenGraph*
- *RookGraph*

AUTHORS:

- David Coudert (2012)

sage.graphs.generators.chessboard.**BishopGraph**(*dim_list*, *radius=None*, *relabel=False*, *immutable=False*)

Return the d-dimensional Bishop Graph with prescribed dimensions.

The 2-dimensional Bishop Graph of parameters n and m is a graph with nm vertices in which each vertex represents a square in an $n \times m$ chessboard, and each edge corresponds to a legal move by a bishop.

The d-dimensional Bishop Graph with $d >= 2$ has for vertex set the cells of a d-dimensional grid with prescribed dimensions, and each edge corresponds to a legal move by a bishop in any pairs of dimensions.

The Bishop Graph is not connected.

INPUT:

- `dim_list` – iterable (list, set, dict); provides the dimensions $n_1, n_2, \ldots, n_d$, with $n_i \geq 1$, of the chessboard
- `radius` – integer (default: `None`); by setting the radius to a positive integer, one may decrease the power of the bishop to at most `radius` steps.
- `relabel` – boolean (default: `False`); indicates whether the vertices must be relabeled as integers
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

The (n,m)-Bishop Graph is not connected:

```
sage: G = graphs.BishopGraph( [3, 4] )
sage: G.is_connected()
False
```

```
>>> from sage.all import *
>>> G = graphs.BishopGraph( [Integer(3), Integer(4)] )
>>> G.is_connected()
False
```

The Bishop Graph can be obtained from Knight Graphs:

```
sage: for d in range(3,12):   # long time
....:     H = Graph()
....:     for r in range(1,d+1):
....:         B = graphs.BishopGraph([d,d],radius=r)
....:         H.add_edges( graphs.KnightGraph([d,d],one=r,two=r).
↪edges(sort=False) )
....:         if not B.is_isomorphic(H):
....:            print("that's not good!")
```

```
>>> from sage.all import *
>>> for d in range(Integer(3),Integer(12)):   # long time
...     H = Graph()
...     for r in range(Integer(1),d+Integer(1)):
...         B = graphs.BishopGraph([d,d],radius=r)
...         H.add_edges( graphs.KnightGraph([d,d],one=r,two=r).edges(sort=False) )
...         if not B.is_isomorphic(H):
...            print("that's not good!")
```

sage.graphs.generators.chessboard.**ChessboardGraphGenerator**(*dim_list*, *rook=True*, *rook_radius=None*, *bishop=True*, *bishop_radius=None*, *knight=True*, *knight_x=1*, *knight_y=2*, *relabel=False*, *immutable=False*)

Return a Graph built on a d-dimensional chessboard with prescribed dimensions and interconnections.

This function allows to generate many kinds of graphs corresponding to legal movements on a d-dimensional chessboard: Queen Graph, King Graph, Knight Graphs, Bishop Graph, and many generalizations. It also allows to avoid redundant code.

INPUT:

- `dim_list` – iterable (list, set, dict); provides the dimensions $n_1, n_2, \ldots, n_d$, with $n_i \geq 1$, of the chessboard
- `rook` – boolean (default: `True`); indicates whether the chess piece is able to move as a rook, that is at any distance along a dimension
- `rook_radius` – integer (default: `None`); restriction on the rook-like movements to distance at most `rook_radius`
- `bishop` – boolean (default: `True`); indicates whether the chess piece is able to move like a bishop, that is along diagonals
- `bishop_radius` – integer (default: `None`); restriction on the bishop-like movements to distance at most `bishop_radius`
- `knight` – boolean (default: `True`); indicating whether the chess piece is able to move like a knight
- `knight_x` – integer (default: 1); indicates the number on steps the chess piece moves in one dimension when moving like a knight
- `knight_y` – integer (default: 2); indicates the number on steps the chess piece moves in the second dimension when moving like a knight
- `relabel` – boolean (default: `False`); indicates whether the vertices must be relabeled as integers
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

OUTPUT:

- A Graph build on a d-dimensional chessboard with prescribed dimensions, and with edges according given parameters.
- A string encoding the dimensions. This is mainly useful for providing names to graphs.

EXAMPLES:

A $(2, 2)$-King Graph is isomorphic to the complete graph on 4 vertices:

```
sage: G, _ = graphs.ChessboardGraphGenerator( [2,2] )
sage: G.is_isomorphic( graphs.CompleteGraph(4) )
True
```

```
>>> from sage.all import *
>>> G, _ = graphs.ChessboardGraphGenerator( [Integer(2),Integer(2)] )
>>> G.is_isomorphic( graphs.CompleteGraph(Integer(4)) )
True
```

A Rook's Graph in 2 dimensions is isomorphic to the Cartesian product of 2 complete graphs:

```
sage: G, _ = graphs.ChessboardGraphGenerator([3,4], rook=True, rook_radius=None,␣
↪bishop=False, knight=False)
sage: H = (graphs.CompleteGraph(3)).cartesian_product(graphs.CompleteGraph(4))
sage: G.is_isomorphic(H)
True
```

```
>>> from sage.all import *
>>> G, _ = graphs.ChessboardGraphGenerator([Integer(3),Integer(4)], rook=True,␣
↪rook_radius=None, bishop=False, knight=False)
>>> H = (graphs.CompleteGraph(Integer(3))).cartesian_product(graphs.
↪CompleteGraph(Integer(4)))
>>> G.is_isomorphic(H)
True
```

sage.graphs.generators.chessboard.**KingGraph**(*dim_list*, *radius=None*, *relabel=False*, *immutable=False*)

Return the d-dimensional King Graph with prescribed dimensions.

The 2-dimensional King Graph of parameters n and m is a graph with nm vertices in which each vertex represents a square in an $n \times m$ chessboard, and each edge corresponds to a legal move by a king.

The d-dimensional King Graph with $d >= 2$ has for vertex set the cells of a d-dimensional grid with prescribed dimensions, and each edge corresponds to a legal move by a king in either one or two dimensions.

All 2-dimensional King Graphs are Hamiltonian, biconnected, and have chromatic number 4 as soon as both dimensions are larger or equal to 2.

INPUT:

- `dim_list` – iterable (list, set, dict); provides the dimensions $n_1, n_2, \ldots, n_d$, with $n_i \geq 1$, of the chessboard
- `radius` – integer (default: `None`); by setting the radius to a positive integer, one may increase the power of the king to at least `radius` steps. When the radius equals the higher size of the dimensions, the resulting graph is a Queen Graph.
- `relabel` – boolean (default: `False`); indicates whether the vertices must be relabeled as integers
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

The $(2, 2)$-King Graph is isomorphic to the complete graph on 4 vertices:

```
sage: G = graphs.QueenGraph( [2, 2] )
sage: G.is_isomorphic( graphs.CompleteGraph(4) )
True
```

```
>>> from sage.all import *
>>> G = graphs.QueenGraph( [Integer(2), Integer(2)] )
>>> G.is_isomorphic( graphs.CompleteGraph(Integer(4)) )
True
```

The King Graph with large enough radius is isomorphic to a Queen Graph:

```
sage: G = graphs.KingGraph( [5, 4], radius=5 )
sage: H = graphs.QueenGraph( [4, 5] )
sage: G.is_isomorphic( H )
True
```

```
>>> from sage.all import *
>>> G = graphs.KingGraph( [Integer(5), Integer(4)], radius=Integer(5) )
>>> H = graphs.QueenGraph( [Integer(4), Integer(5)] )
>>> G.is_isomorphic( H )
True
```

Also `True` in higher dimensions:

```
sage: G = graphs.KingGraph( [2, 5, 4], radius=5 )
sage: H = graphs.QueenGraph( [4, 5, 2] )
sage: G.is_isomorphic( H )
True
```

```
>>> from sage.all import *
>>> G = graphs.KingGraph( [Integer(2), Integer(5), Integer(4)], radius=Integer(5)␣
↪)
>>> H = graphs.QueenGraph( [Integer(4), Integer(5), Integer(2)] )
>>> G.is_isomorphic( H )
True
```

sage.graphs.generators.chessboard.**KnightGraph**(*dim_list*, *one=1*, *two=2*, *relabel=False*, *immutable=False*)

Return the d-dimensional Knight Graph with prescribed dimensions.

The 2-dimensional Knight Graph of parameters n and m is a graph with nm vertices in which each vertex represents a square in an $n \times m$ chessboard, and each edge corresponds to a legal move by a knight.

The d-dimensional Knight Graph with $d >= 2$ has for vertex set the cells of a d-dimensional grid with prescribed dimensions, and each edge corresponds to a legal move by a knight in any pairs of dimensions.

The (n, n)-Knight Graph is Hamiltonian for even $n > 4$.

INPUT:

- `dim_list` – iterable (list, set, dict); provides the dimensions $n_1, n_2, \ldots, n_d$, with $n_i \geq 1$, of the chessboard
- `one` – integer (default: 1); indicates the number of steps in the first dimension
- `two` – integer (default: 2); indicates the number of steps in the second dimension

- `relabel` – boolean (default: `False`); indicates whether the vertices must be relabeled as integers
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

The $(3, 3)$-Knight Graph has an isolated vertex:

```
sage: G = graphs.KnightGraph( [3, 3] )
sage: G.degree( (1,1) )
0
```

```
>>> from sage.all import *
>>> G = graphs.KnightGraph( [Integer(3), Integer(3)] )
>>> G.degree( (Integer(1),Integer(1)) )
0
```

The $(3, 3)$-Knight Graph minus vertex (1,1) is a cycle of order 8:

```
sage: G = graphs.KnightGraph( [3, 3] )
sage: G.delete_vertex( (1,1) )
sage: G.is_isomorphic( graphs.CycleGraph(8) )
True
```

```
>>> from sage.all import *
>>> G = graphs.KnightGraph( [Integer(3), Integer(3)] )
>>> G.delete_vertex( (Integer(1),Integer(1)) )
>>> G.is_isomorphic( graphs.CycleGraph(Integer(8)) )
True
```

The $(6, 6)$-Knight Graph is Hamiltonian:

```
sage: G = graphs.KnightGraph( [6, 6] )
sage: G.is_hamiltonian()                                                    #␣
↪needs sage.numerical.mip
True
```

```
>>> from sage.all import *
>>> G = graphs.KnightGraph( [Integer(6), Integer(6)] )
>>> G.is_hamiltonian()                                                      #␣
↪needs sage.numerical.mip
True
```

`sage.graphs.generators.chessboard.`**`QueenGraph`**(*dim_list*, *radius=None*, *relabel=False*, *immutable=False*)

Return the d-dimensional Queen Graph with prescribed dimensions.

The 2-dimensional Queen Graph of parameters n and m is a graph with nm vertices in which each vertex represents a square in an $n \times m$ chessboard, and each edge corresponds to a legal move by a queen.

The d-dimensional Queen Graph with $d >= 2$ has for vertex set the cells of a d-dimensional grid with prescribed dimensions, and each edge corresponds to a legal move by a queen in either one or two dimensions.

All 2-dimensional Queen Graphs are Hamiltonian and biconnected. The chromatic number of a (n, n)-Queen Graph is at least n, and it is exactly n when $n \equiv 1, 5 \bmod 6$.

INPUT:

- `dim_list` – iterable (list, set, dict); provides the dimensions $n_1, n_2, \ldots, n_d$, with $n_i \geq 1$, of the chessboard

- `radius` – integer (default: `None`); by setting the radius to a positive integer, one may reduce the visibility of the queen to at most `radius` steps. When radius is 1, the resulting graph is a King Graph.
- `relabel` – boolean (default: `False`); indicates whether the vertices must be relabeled as integers
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

The $(2, 2)$-Queen Graph is isomorphic to the complete graph on 4 vertices:

```
sage: G = graphs.QueenGraph([2, 2])
sage: G.is_isomorphic(graphs.CompleteGraph(4))
True
```

```
>>> from sage.all import *
>>> G = graphs.QueenGraph([Integer(2), Integer(2)])
>>> G.is_isomorphic(graphs.CompleteGraph(Integer(4)))
True
```

The Queen Graph with radius 1 is isomorphic to the King Graph:

```
sage: G = graphs.QueenGraph([4, 5], radius=1)
sage: H = graphs.KingGraph([5, 4])
sage: G.is_isomorphic(H)
True
```

```
>>> from sage.all import *
>>> G = graphs.QueenGraph([Integer(4), Integer(5)], radius=Integer(1))
>>> H = graphs.KingGraph([Integer(5), Integer(4)])
>>> G.is_isomorphic(H)
True
```

Also `True` in higher dimensions:

```
sage: G = graphs.QueenGraph([3, 4, 5], radius=1)
sage: H = graphs.KingGraph([5, 3, 4])
sage: G.is_isomorphic(H)
True
```

```
>>> from sage.all import *
>>> G = graphs.QueenGraph([Integer(3), Integer(4), Integer(5)], radius=Integer(1))
>>> H = graphs.KingGraph([Integer(5), Integer(3), Integer(4)])
>>> G.is_isomorphic(H)
True
```

The Queen Graph can be obtained from the Rook Graph and the Bishop Graph:

```
sage: for d in range(3,12):   # long time
....:     for r in range(1,d+1):
....:         G = graphs.QueenGraph([d,d],radius=r)
....:         H = graphs.RookGraph([d,d],radius=r)
....:         B = graphs.BishopGraph([d,d],radius=r)
....:         H.add_edges(B.edges(sort=False))
....:         if not G.is_isomorphic(H):
....:             print("that's not good!")
```

```
>>> from sage.all import *
>>> for d in range(Integer(3),Integer(12)):   # long time
...     for r in range(Integer(1),d+Integer(1)):
...         G = graphs.QueenGraph([d,d],radius=r)
...         H = graphs.RookGraph([d,d],radius=r)
...         B = graphs.BishopGraph([d,d],radius=r)
...         H.add_edges(B.edges(sort=False))
...         if not G.is_isomorphic(H):
...             print("that's not good!")
```

sage.graphs.generators.chessboard.**RookGraph**(*dim_list*, *radius=None*, *relabel=False*, *immutable=False*)

Return the d-dimensional Rook's Graph with prescribed dimensions.

The 2-dimensional Rook's Graph of parameters n and m is a graph with nm vertices in which each vertex represents a square in an $n \times m$ chessboard, and each edge corresponds to a legal move by a rook.

The d-dimensional Rook Graph with $d >= 2$ has for vertex set the cells of a d-dimensional grid with prescribed dimensions, and each edge corresponds to a legal move by a rook in any of the dimensions.

The Rook's Graph for an $n \times m$ chessboard may also be defined as the Cartesian product of two complete graphs $K_n \Box K_m$.

INPUT:

- `dim_list` – iterable (list, set, dict); provides the dimensions $n_1, n_2, \ldots, n_d$, with $n_i \geq 1$, of the chessboard
- `radius` – integer (default: `None`); by setting the radius to a positive integer, one may decrease the power of the rook to at most `radius` steps. When the radius is 1, the resulting graph is a d-dimensional grid.
- `relabel` – boolean (default: `False`); indicates whether the vertices must be relabeled as integers
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

The (n, m)-Rook's Graph is isomorphic to the Cartesian product of two complete graphs:

```
sage: G = graphs.RookGraph( [3, 4] )
sage: H = ( graphs.CompleteGraph(3) ).cartesian_product( graphs.CompleteGraph(4) )
sage: G.is_isomorphic( H )
True
```

```
>>> from sage.all import *
>>> G = graphs.RookGraph( [Integer(3), Integer(4)] )
>>> H = ( graphs.CompleteGraph(Integer(3)) ).cartesian_product( graphs.
↪CompleteGraph(Integer(4)) )
>>> G.is_isomorphic( H )
True
```

When the radius is 1, the Rook's Graph is a grid:

```
sage: G = graphs.RookGraph( [3, 3, 4], radius=1 )
sage: H = graphs.GridGraph( [3, 4, 3] )
sage: G.is_isomorphic( H )
True
```

```
>>> from sage.all import *
>>> G = graphs.RookGraph( [Integer(3), Integer(3), Integer(4)], radius=Integer(1)␣
↪)
>>> H = graphs.GridGraph( [Integer(3), Integer(4), Integer(3)] )
>>> G.is_isomorphic( H )
True
```

2.9 Families of graphs derived from classical geometries over finite fields

These include graphs of polar spaces, affine polar graphs, graphs related to Hermitean unitals, graphs on nonisotropic points, etc.

The methods defined here appear in *sage.graphs.graph_generators*.

sage.graphs.generators.classical_geometries.**AffineOrthogonalPolarGraph**(*d*, *q*, *sign='+'*, *immutable=False*)

Return the affine polar graph $VO^+(d,q), VO^-(d,q)$ or $VO(d,q)$.

Affine Polar graphs are built from a d-dimensional vector space over F_q, and a quadratic form which is hyperbolic, elliptic or parabolic according to the value of `sign`.

Note that $VO^+(d,q), VO^-(d,q)$ are strongly regular graphs, while $VO(d,q)$ is not.

For more information on Affine Polar graphs, see Affine Polar Graphs page of Andries Brouwer's website.

INPUT:

- `d` – integer; `d` must be even if `sign is not None`, and odd otherwise
- `q` – integer; a power of a prime number, as F_q must exist
- `sign` – string (default: `'+'`); must be equal to `'+'`, `'-'`, or `None` to compute (respectively) $VO^+(d,q), VO^-(d,q)$ or $VO(d,q)$
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

Note

The graph $VO^\epsilon(d,q)$ is the graph induced by the non-neighbors of a vertex in an *Orthogonal Polar Graph* $O^\epsilon(d+2,q)$.

EXAMPLES:

The *Brouwer-Haemers graph* is isomorphic to $VO^-(4,3)$:

```
sage: g = graphs.AffineOrthogonalPolarGraph(4,3,"-")                    #␣
↪needs sage.libs.gap
sage: g.is_isomorphic(graphs.BrouwerHaemersGraph())                     #␣
↪needs sage.libs.gap
True
```

```
>>> from sage.all import *
>>> g = graphs.AffineOrthogonalPolarGraph(Integer(4),Integer(3),"-")      ␣
↪              # needs sage.libs.gap
```

(continues on next page)

(continued from previous page)

```
>>> g.is_isomorphic(graphs.BrouwerHaemersGraph())                    #␣
↪needs sage.libs.gap
True
```

Some examples from Brouwer's table or strongly regular graphs:

```
sage: g = graphs.AffineOrthogonalPolarGraph(6,2,"-"); g
Affine Polar Graph VO^-(6,2): Graph on 64 vertices
sage: g.is_strongly_regular(parameters=True)
(64, 27, 10, 12)
sage: g = graphs.AffineOrthogonalPolarGraph(6,2,"+"); g
Affine Polar Graph VO^+(6,2): Graph on 64 vertices
sage: g.is_strongly_regular(parameters=True)
(64, 35, 18, 20)
```

```
>>> from sage.all import *
>>> g = graphs.AffineOrthogonalPolarGraph(Integer(6),Integer(2),"-"); g
Affine Polar Graph VO^-(6,2): Graph on 64 vertices
>>> g.is_strongly_regular(parameters=True)
(64, 27, 10, 12)
>>> g = graphs.AffineOrthogonalPolarGraph(Integer(6),Integer(2),"+"); g
Affine Polar Graph VO^+(6,2): Graph on 64 vertices
>>> g.is_strongly_regular(parameters=True)
(64, 35, 18, 20)
```

When `sign is None`:

```
sage: g = graphs.AffineOrthogonalPolarGraph(5,2,None); g
Affine Polar Graph VO^-(5,2): Graph on 32 vertices
sage: g.is_strongly_regular(parameters=True)
False
sage: g.is_regular()
True
sage: g.is_vertex_transitive()
True
```

```
>>> from sage.all import *
>>> g = graphs.AffineOrthogonalPolarGraph(Integer(5),Integer(2),None); g
Affine Polar Graph VO^-(5,2): Graph on 32 vertices
>>> g.is_strongly_regular(parameters=True)
False
>>> g.is_regular()
True
>>> g.is_vertex_transitive()
True
```

sage.graphs.generators.classical_geometries.**AhrensSzekeresGeneralizedQuadrangleGraph**(*q*, *dual=False*, *immutable=False*)

Return the collinearity graph of the generalized quadrangle $AS(q)$, or of its dual

Let q be an odd prime power. $AS(q)$ is a generalized quadrangle (Wikipedia article Generalized_quadrangle) of

order $(q-1, q+1)$, see 3.1.5 in [PT2009]. Its points are elements of F_q^3, and lines are sets of size q of the form

- $\{(\sigma, a, b) \mid \sigma \in F_q\}$
- $\{(a, \sigma, b) \mid \sigma \in F_q\}$
- $\{(c\sigma^2 - b\sigma + a, -2c\sigma + b, \sigma) \mid \sigma \in F_q\}$,

where a, b, c are arbitrary elements of F_q.

INPUT:

- `q` – a power of an odd prime number
- `dual` – boolean (default: `False`); whether to return the collinearity graph of $AS(q)$ or of the dual $AS(q)$ (when `True`)
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.AhrensSzekeresGeneralizedQuadrangleGraph(5); g
AS(5); GQ(4, 6): Graph on 125 vertices
sage: g.is_strongly_regular(parameters=True)
(125, 28, 3, 7)
sage: g = graphs.AhrensSzekeresGeneralizedQuadrangleGraph(5, dual=True); g
AS(5)*; GQ(6, 4): Graph on 175 vertices
sage: g.is_strongly_regular(parameters=True)
(175, 30, 5, 5)
```

```
>>> from sage.all import *
>>> g = graphs.AhrensSzekeresGeneralizedQuadrangleGraph(Integer(5)); g
AS(5); GQ(4, 6): Graph on 125 vertices
>>> g.is_strongly_regular(parameters=True)
(125, 28, 3, 7)
>>> g = graphs.AhrensSzekeresGeneralizedQuadrangleGraph(Integer(5), dual=True); g
AS(5)*; GQ(6, 4): Graph on 175 vertices
>>> g.is_strongly_regular(parameters=True)
(175, 30, 5, 5)
```

sage.graphs.generators.classical_geometries.**CossidentePenttilaGraph**(*q*, *immutable=False*)

Return the Cossidente-Penttila $((q^3+1)(q+1)/2, (q^2+1)(q-1)/2, (q-3)/2, (q-1)^2/2)$-strongly regular graph

For each odd prime power q, one can partition the points of the $O_6^-(q)$-generalized quadrangle $GQ(q, q^2)$ into two parts, so that on any of them the induced subgraph of the point graph of the GQ has parameters as above [CP2005].

Directly following the construction in [CP2005] is not efficient, as one then needs to construct the dual $GQ(q^2, q)$. Thus we describe here a more efficient approach that we came up with, following a suggestion by T.Penttila. Namely, this partition is invariant under the subgroup $H = \Omega_3(q^2) < O_6^-(q)$. We build the appropriate H, which leaves the form $B(X, Y, Z) = XY + Z^2$ invariant, and pick up two orbits of H on the F_q-points. One them is B-isotropic, and we take the representative $(1:0:0)$. The other one corresponds to the points of $PG(2, q^2)$ that have all the lines on them either missing the conic specified by B, or intersecting the conic in two points. We take $(1:1:e)$ as the representative. It suffices to pick e so that e^2+1 is not a square in F_{q^2}. Indeed, The conic can be viewed as the union of $\{(0:1:0)\}$ and $\{(1:-t^2:t) | t \in F_{q^2}\}$. The coefficients of a generic line on $(1:1:e)$ are $[1:-1-eb:b]$, for $-1 \neq eb$. Thus, to make sure the intersection with the conic is always even, we need that the discriminant of $1 + (1+eb)t^2 + tb = 0$ never vanishes, and this is if and only if e^2+1 is not a square. Further, we need to adjust B, by multiplying it by appropriately chosen ν, so that $(1:1:e)$ becomes isotropic under the relative trace norm $\nu B(X, Y, Z) + (\nu B(X, Y, Z))^q$. The latter is used then to define the graph.

INPUT:

- `q` – an odd prime power
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

For $q = 3$ one gets Sims-Gewirtz graph.

```
sage: G = graphs.CossidentePenttilaGraph(3)      # optional - gap_package_grape
sage: G.is_strongly_regular(parameters=True)     # optional - gap_package_grape
(56, 10, 0, 2)
```

```
>>> from sage.all import *
>>> G = graphs.CossidentePenttilaGraph(Integer(3))      # optional - gap_package_
↪grape
>>> G.is_strongly_regular(parameters=True)     # optional - gap_package_grape
(56, 10, 0, 2)
```

For $q > 3$ one gets new graphs.

```
sage: G = graphs.CossidentePenttilaGraph(5)      # optional - gap_package_grape
sage: G.is_strongly_regular(parameters=True)     # optional - gap_package_grape
(378, 52, 1, 8)
```

```
>>> from sage.all import *
>>> G = graphs.CossidentePenttilaGraph(Integer(5))      # optional - gap_package_
↪grape
>>> G.is_strongly_regular(parameters=True)     # optional - gap_package_grape
(378, 52, 1, 8)
```

sage.graphs.generators.classical_geometries.**HaemersGraph**(*q*, *hyperoval=None*, *hyperoval_matching=None*, *field=None*, *check_hyperoval=True*, *immutable=False*)

Return the Haemers graph obtained from $T_2^*(q)^*$.

Let q be a power of 2. In Sect. 8.A of [BL1984] one finds a construction of a strongly regular graph with parameters $(q^2(q+2), q^2+q-1, q-2, q)$ from the graph of $T_2^*(q)^*$, constructed by *T2starGeneralizedQuadrangleGraph()*, by redefining adjacencies in the way specified by an arbitrary `hyperoval_matching` of the points (i.e. partitioning into size two parts) of `hyperoval` defining $T_2^*(q)^*$.

While [BL1984] gives the construction in geometric terms, it can be formulated, and is implemented, in graph-theoretic ones, of re-adjusting the edges. Namely, $G = T_2^*(q)^*$ has a partition into $q+2$ independent sets I_k of size q^2 each. Each vertex in I_j is adjacent to q vertices from I_k. Each I_k is paired to some $I_{k'}$, according to `hyperoval_matching`. One adds edges (s,t) for $s,t \in I_k$ whenever s and t are adjacent to some $u \in I_{k'}$, and removes all the edges between I_k and $I_{k'}$.

INPUT:

- `q` – a power of two
- `hyperoval_matching` – if `None` (default), pair each i-th point of `hyperoval` with $(i+1)$-th. Otherwise, specifies the pairing in the format $((i_1, i'_1), (i_2, i'_2), ...)$.
- `hyperoval` – a hyperoval defining $T_2^*(q)^*$. If `None` (default), the classical hyperoval obtained from a conic is used. See the documentation of *T2starGeneralizedQuadrangleGraph()*, for more information.

- `field` – an instance of a finite field of order q, must be provided if `hyperoval` is provided
- `check_hyperoval` – boolean (default: `True`); whether to check `hyperoval` for correctness or not
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

using the built-in constructions:

```
sage: g = graphs.HaemersGraph(4); g
Haemers(4): Graph on 96 vertices
sage: g.is_strongly_regular(parameters=True)
(96, 19, 2, 4)
```

```
>>> from sage.all import *
>>> g = graphs.HaemersGraph(Integer(4)); g
Haemers(4): Graph on 96 vertices
>>> g.is_strongly_regular(parameters=True)
(96, 19, 2, 4)
```

supplying your own hyperoval_matching:

```
sage: g = graphs.HaemersGraph(4, hyperoval_matching=((0,5),(1,4),(2,3))); g
Haemers(4): Graph on 96 vertices
sage: g.is_strongly_regular(parameters=True)
(96, 19, 2, 4)
```

```
>>> from sage.all import *
>>> g = graphs.HaemersGraph(Integer(4), hyperoval_matching=((Integer(0),
↪Integer(5)),(Integer(1),Integer(4)),(Integer(2),Integer(3)))); g
Haemers(4): Graph on 96 vertices
>>> g.is_strongly_regular(parameters=True)
(96, 19, 2, 4)
```

sage.graphs.generators.classical_geometries.**NonisotropicOrthogonalPolarGraph**(*m*, *q*, *sign='+'*, *perp=None*, *immutable=False*)

Return the Graph $NO^{\epsilon,\perp}_m(q)$.

Let the vectorspace of dimension m over F_q be endowed with a nondegenerate quadratic form F, of type `sign` for m even.

- m even: assume further that $q = 2$ or 3. Returns the graph of the points (in the underlying projective space) x satisfying $F(x) = 1$, with adjacency given by orthogonality w.r.t. F. Parameter `perp` is ignored.
- m odd: if `perp` is not `None`, then we assume that $q = 5$ and return the graph of the points x satisfying $F(x) = \pm 1$ if `sign="+"`, respectively $F(x) \in \{2, 3\}$ if `sign="-"`, with adjacency given by orthogonality w.r.t. F (cf. Sect 7.D of [BL1984]). Otherwise return the graph of nongenerate hyperplanes of type `sign`, adjacent whenever the intersection is degenerate (cf. Sect. 7.C of [BL1984]). Note that for $q = 2$ one will get a complete graph.

For more information, see Sect. 9.9 of [BH2012] and [BL1984]. Note that the page of Andries Brouwer's website uses different notation.

INPUT:

- `m` – integer; half the dimension of the underlying vectorspace
- `q` – a power of a prime number, the size of the underlying field
- `sign` – string (default: `'+'`); must be either `'+'` or `'-'`
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

$NO^-(4,2)$ is isomorphic to Petersen graph:

```
sage: g = graphs.NonisotropicOrthogonalPolarGraph(4,2,'-'); g                  #␣
↪needs sage.libs.gap
NO^-(4, 2): Graph on 10 vertices
sage: g.is_strongly_regular(parameters=True)                                   #␣
↪needs sage.libs.gap
(10, 3, 0, 1)
```

```
>>> from sage.all import *
>>> g = graphs.NonisotropicOrthogonalPolarGraph(Integer(4),Integer(2),'-'); g    ␣
↪              # needs sage.libs.gap
NO^-(4, 2): Graph on 10 vertices
>>> g.is_strongly_regular(parameters=True)                                     #␣
↪needs sage.libs.gap
(10, 3, 0, 1)
```

$NO^-(6,2)$ and $NO^+(6,2)$:

```
sage: g = graphs.NonisotropicOrthogonalPolarGraph(6,2,'-')
sage: g.is_strongly_regular(parameters=True)
(36, 15, 6, 6)
sage: g = graphs.NonisotropicOrthogonalPolarGraph(6,2,'+'); g
NO^+(6, 2): Graph on 28 vertices
sage: g.is_strongly_regular(parameters=True)
(28, 15, 6, 10)
```

```
>>> from sage.all import *
>>> g = graphs.NonisotropicOrthogonalPolarGraph(Integer(6),Integer(2),'-')
>>> g.is_strongly_regular(parameters=True)
(36, 15, 6, 6)
>>> g = graphs.NonisotropicOrthogonalPolarGraph(Integer(6),Integer(2),'+'); g
NO^+(6, 2): Graph on 28 vertices
>>> g.is_strongly_regular(parameters=True)
(28, 15, 6, 10)
```

$NO^+(8,2)$:

```
sage: g = graphs.NonisotropicOrthogonalPolarGraph(8,2,'+')                     #␣
↪needs sage.libs.gap
sage: g.is_strongly_regular(parameters=True)                                   #␣
↪needs sage.libs.gap
(120, 63, 30, 36)
```

```
>>> from sage.all import *
>>> g = graphs.NonisotropicOrthogonalPolarGraph(Integer(8),Integer(2),'+')
↪              # needs sage.libs.gap
>>> g.is_strongly_regular(parameters=True)                                    #␣
↪needs sage.libs.gap
(120, 63, 30, 36)
```

Wilbrink's graphs for $q = 5$:

```
sage: g = graphs.NonisotropicOrthogonalPolarGraph(5,5,perp=1)
sage: g.is_strongly_regular(parameters=True)     # long time
(325, 60, 15, 10)
sage: g = graphs.NonisotropicOrthogonalPolarGraph(5,5,'-',perp=1)
sage: g.is_strongly_regular(parameters=True)     # long time
(300, 65, 10, 15)
```

```
>>> from sage.all import *
>>> g = graphs.NonisotropicOrthogonalPolarGraph(Integer(5),Integer(5),
↪perp=Integer(1))
>>> g.is_strongly_regular(parameters=True)     # long time
(325, 60, 15, 10)
>>> g = graphs.NonisotropicOrthogonalPolarGraph(Integer(5),Integer(5),'-',
↪perp=Integer(1))
>>> g.is_strongly_regular(parameters=True)     # long time
(300, 65, 10, 15)
```

Wilbrink's graphs:

```
sage: g = graphs.NonisotropicOrthogonalPolarGraph(5,4,'+')
sage: g.is_strongly_regular(parameters=True)
(136, 75, 42, 40)
sage: g = graphs.NonisotropicOrthogonalPolarGraph(5,4,'-')
sage: g.is_strongly_regular(parameters=True)
(120, 51, 18, 24)
sage: g = graphs.NonisotropicOrthogonalPolarGraph(7,4,'+'); g         # not tested␣
↪(long time)
NO^+(7, 4): Graph on 2080 vertices
sage: g.is_strongly_regular(parameters=True)                          # not tested␣
↪(long time)
(2080, 1071, 558, 544)
```

```
>>> from sage.all import *
>>> g = graphs.NonisotropicOrthogonalPolarGraph(Integer(5),Integer(4),'+')
>>> g.is_strongly_regular(parameters=True)
(136, 75, 42, 40)
>>> g = graphs.NonisotropicOrthogonalPolarGraph(Integer(5),Integer(4),'-')
>>> g.is_strongly_regular(parameters=True)
(120, 51, 18, 24)
>>> g = graphs.NonisotropicOrthogonalPolarGraph(Integer(7),Integer(4),'+'); g    ␣
↪    # not tested (long time)
NO^+(7, 4): Graph on 2080 vertices
>>> g.is_strongly_regular(parameters=True)                          # not tested␣
```

(continues on next page)

(continued from previous page)

```
↪(long time)
(2080, 1071, 558, 544)
```

sage.graphs.generators.classical_geometries.**NonisotropicUnitaryPolarGraph**(*m*, *q*, *immutable=False*)

Return the Graph $NU(m, q)$.

This returns the graph on nonisotropic, with respect to a nondegenerate Hermitean form, points of the $(m-1)$-dimensional projective space over F_q, with points adjacent whenever they lie on a tangent (to the set of isotropic points) line. For more information, see Sect. 9.9 of [BH2012] and series C14 in [Hub1975].

INPUT:

- `m`, `q` – integers; q must be a prime power
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.NonisotropicUnitaryPolarGraph(5,2); g                    #␣
↪needs sage.libs.gap
NU(5, 2): Graph on 176 vertices
sage: g.is_strongly_regular(parameters=True)                              #␣
↪needs sage.libs.gap
(176, 135, 102, 108)
```

```
>>> from sage.all import *
>>> g = graphs.NonisotropicUnitaryPolarGraph(Integer(5),Integer(2)); g          ␣
↪              # needs sage.libs.gap
NU(5, 2): Graph on 176 vertices
>>> g.is_strongly_regular(parameters=True)                                  #␣
↪needs sage.libs.gap
(176, 135, 102, 108)
```

sage.graphs.generators.classical_geometries.**Nowhere0WordsTwoWeightCodeGraph**(*q*, *hyperoval=None*, *field=None*, *check_hyperoval=True*, *immutable=False*)

Return the subgraph of nowhere 0 words from two-weight code of projective plane hyperoval.

Let $q = 2^k$ and $\Pi = PG(2, q)$. Fix a hyperoval $O \subset \Pi$. Let $V = F_q^3$ and C the two-weight 3-dimensional linear code over F_q with words $c(v)$ obtained from $v \in V$ by computing

$$c(v) = (\langle v, o_1\rangle, ..., \langle v, o_{q+2}\rangle), o_j \in O.$$

C contains $q(q-1)^2/2$ words without 0 entries. The subgraph of the strongly regular graph of C induced on the latter words is also strongly regular, assuming $q > 4$. This is a construction due to A.E.Brouwer [Bro2016], and leads to graphs with parameters also given by a construction in [HHL2009]. According to [Bro2016], these two constructions are likely to produce isomorphic graphs.

INPUT:

- `q` – a power of two

- `hyperoval` – a hyperoval (i.e. a complete 2-arc; a set of points in the plane meeting every line in 0 or 2 points) in $PG(2, q)$ over the field `field`. Each point of `hyperoval` must be a length 3 vector over `field` with 1st non-0 coordinate equal to 1. By default, `hyperoval` and `field` are not specified, and constructed on the fly. In particular, `hyperoval` we build is the classical one, i.e. a conic with the point of intersection of its tangent lines.
- `field` – an instance of a finite field of order q; must be provided if `hyperoval` is provided
- `check_hyperoval` – boolean (default: `True`); whether to check `hyperoval` for correctness or not
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

See also

- *is_nowhere0_twoweight()*

EXAMPLES:

using the built-in construction:

```
sage: g = graphs.Nowhere0WordsTwoWeightCodeGraph(8); g
Nowhere0WordsTwoWeightCodeGraph(8): Graph on 196 vertices
sage: g.is_strongly_regular(parameters=True)
(196, 60, 14, 20)
sage: g = graphs.Nowhere0WordsTwoWeightCodeGraph(16)  # not tested (long time)
sage: g.is_strongly_regular(parameters=True)          # not tested (long time)
(1800, 728, 268, 312)
```

```
>>> from sage.all import *
>>> g = graphs.Nowhere0WordsTwoWeightCodeGraph(Integer(8)); g
Nowhere0WordsTwoWeightCodeGraph(8): Graph on 196 vertices
>>> g.is_strongly_regular(parameters=True)
(196, 60, 14, 20)
>>> g = graphs.Nowhere0WordsTwoWeightCodeGraph(Integer(16))  # not tested (long␣
↪time)
>>> g.is_strongly_regular(parameters=True)          # not tested (long time)
(1800, 728, 268, 312)
```

supplying your own hyperoval:

```
sage: F = GF(8)
sage: O = [vector(F,(0,0,1)),vector(F,(0,1,0))] + [vector(F, (1,x^2,x))
....:                                               for x in F]
sage: g = graphs.Nowhere0WordsTwoWeightCodeGraph(8,hyperoval=O,field=F); g
Nowhere0WordsTwoWeightCodeGraph(8): Graph on 196 vertices
sage: g.is_strongly_regular(parameters=True)
(196, 60, 14, 20)
```

```
>>> from sage.all import *
>>> F = GF(Integer(8))
>>> O = [vector(F,(Integer(0),Integer(0),Integer(1))),vector(F,(Integer(0),
↪Integer(1),Integer(0)))] + [vector(F, (Integer(1),x**Integer(2),x))
...                                                 for x in F]
>>> g = graphs.Nowhere0WordsTwoWeightCodeGraph(Integer(8),hyperoval=O,field=F); g
```

(continues on next page)

(continued from previous page)

```
Nowhere0WordsTwoWeightCodeGraph(8): Graph on 196 vertices
>>> g.is_strongly_regular(parameters=True)
(196, 60, 14, 20)
```

sage.graphs.generators.classical_geometries.**OrthogonalDualPolarGraph**(*e*, *d*, *q*, *immutable=False*)

Return the dual polar graph on $GO^e(n, q)$ of diameter d.

The value of n is determined by d and e.

The graph is distance-regular with classical parameters $(d, q, 0, q^e)$.

INPUT:

- `e` – integer; type of the orthogonal polar space to consider; must be $-1, 0$ or 1
- `d` – integer; diameter of the graph
- `q` – integer; prime power; order of the finite field over which to build the polar space
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: G = graphs.OrthogonalDualPolarGraph(1,3,2)
sage: G.is_distance_regular(True)
([7, 6, 4, None], [None, 1, 3, 7])
sage: G = graphs.OrthogonalDualPolarGraph(0,3,3)         # long time
sage: G.is_distance_regular(True)                        # long time
([39, 36, 27, None], [None, 1, 4, 13])
sage: G.order()                                          # long time
1120
```

```
>>> from sage.all import *
>>> G = graphs.OrthogonalDualPolarGraph(Integer(1),Integer(3),Integer(2))
>>> G.is_distance_regular(True)
([7, 6, 4, None], [None, 1, 3, 7])
>>> G = graphs.OrthogonalDualPolarGraph(Integer(0),Integer(3),Integer(3))
↪# long time
>>> G.is_distance_regular(True)                          # long time
([39, 36, 27, None], [None, 1, 4, 13])
>>> G.order()                                            # long time
1120
```

REFERENCES:

See [BCN1989] pp. 274-279 or [VDKT2016] p. 22.

sage.graphs.generators.classical_geometries.**OrthogonalPolarGraph**(*m*, *q*, *sign='+'*, *immutable=False*)

Return the Orthogonal Polar Graph $O^\epsilon(m, q)$.

For more information on Orthogonal Polar graphs, see the page of Andries Brouwer's website.

INPUT:

- `m`, `q` – integers; q must be a prime power
- `sign` – string (default: `'+'`); must be `'+'` or `'-'` if m is even, `'+'` (default) otherwise

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: G = graphs.OrthogonalPolarGraph(6,3,"+"); G
Orthogonal Polar Graph O^+(6, 3): Graph on 130 vertices
sage: G.is_strongly_regular(parameters=True)
(130, 48, 20, 16)
sage: G = graphs.OrthogonalPolarGraph(6,3,"-"); G
Orthogonal Polar Graph O^-(6, 3): Graph on 112 vertices
sage: G.is_strongly_regular(parameters=True)
(112, 30, 2, 10)
sage: G = graphs.OrthogonalPolarGraph(5,3); G
Orthogonal Polar Graph O(5, 3): Graph on 40 vertices
sage: G.is_strongly_regular(parameters=True)
(40, 12, 2, 4)
sage: G = graphs.OrthogonalPolarGraph(8,2,"+"); G
Orthogonal Polar Graph O^+(8, 2): Graph on 135 vertices
sage: G.is_strongly_regular(parameters=True)
(135, 70, 37, 35)
sage: G = graphs.OrthogonalPolarGraph(8,2,"-"); G
Orthogonal Polar Graph O^-(8, 2): Graph on 119 vertices
sage: G.is_strongly_regular(parameters=True)
(119, 54, 21, 27)
```

```
>>> from sage.all import *
>>> G = graphs.OrthogonalPolarGraph(Integer(6),Integer(3),"+"); G
Orthogonal Polar Graph O^+(6, 3): Graph on 130 vertices
>>> G.is_strongly_regular(parameters=True)
(130, 48, 20, 16)
>>> G = graphs.OrthogonalPolarGraph(Integer(6),Integer(3),"-"); G
Orthogonal Polar Graph O^-(6, 3): Graph on 112 vertices
>>> G.is_strongly_regular(parameters=True)
(112, 30, 2, 10)
>>> G = graphs.OrthogonalPolarGraph(Integer(5),Integer(3)); G
Orthogonal Polar Graph O(5, 3): Graph on 40 vertices
>>> G.is_strongly_regular(parameters=True)
(40, 12, 2, 4)
>>> G = graphs.OrthogonalPolarGraph(Integer(8),Integer(2),"+"); G
Orthogonal Polar Graph O^+(8, 2): Graph on 135 vertices
>>> G.is_strongly_regular(parameters=True)
(135, 70, 37, 35)
>>> G = graphs.OrthogonalPolarGraph(Integer(8),Integer(2),"-"); G
Orthogonal Polar Graph O^-(8, 2): Graph on 119 vertices
>>> G.is_strongly_regular(parameters=True)
(119, 54, 21, 27)
```

sage.graphs.generators.classical_geometries.**SymplecticDualPolarGraph**(*m*, *q*, *immutable=False*)

Return the Symplectic Dual Polar Graph $DSp(m, q)$.

For more information on Symplectic Dual Polar graphs, see [BCN1989] and Sect. 2.3.1 of [Coh1981].

INPUT:

- `m`, `q` – integers; q must be a prime power, and m must be even

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: G = graphs.SymplecticDualPolarGraph(6,3); G       # not tested (long time)
Symplectic Dual Polar Graph DSp(6, 3): Graph on 1120 vertices
sage: G.is_distance_regular(parameters=True)            # not tested (long time)
([39, 36, 27, None], [None, 1, 4, 13])
```

```
>>> from sage.all import *
>>> G = graphs.SymplecticDualPolarGraph(Integer(6),Integer(3)); G       # not␣
↪tested (long time)
Symplectic Dual Polar Graph DSp(6, 3): Graph on 1120 vertices
>>> G.is_distance_regular(parameters=True)            # not tested (long time)
([39, 36, 27, None], [None, 1, 4, 13])
```

sage.graphs.generators.classical_geometries.**SymplecticPolarGraph**(*d*, *q*, *algorithm=None*, *immutable=False*)

Return the Symplectic Polar Graph $Sp(d, q)$.

The Symplectic Polar Graph $Sp(d, q)$ is built from a projective space of dimension $d - 1$ over a field F_q, and a symplectic form f. Two vertices u, v are made adjacent if $f(u, v) = 0$.

See the page on symplectic graphs on Andries Brouwer's website.

INPUT:

- `d`, `q` – integers; note that only even values of d are accepted by the function
- `algorithm` – string (default: `None`); if set to `'gap'`, then the computation is carried via GAP library interface, computing totally singular subspaces, which is faster for $q > 3$. Otherwise it is done directly.
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

Computation of the spectrum of $Sp(6, 2)$:

```
sage: g = graphs.SymplecticPolarGraph(6, 2)
sage: g.is_strongly_regular(parameters=True)
(63, 30, 13, 15)
sage: set(g.spectrum()) == {-5, 3, 30}                                  #␣
↪needs sage.rings.number_field
True
```

```
>>> from sage.all import *
>>> g = graphs.SymplecticPolarGraph(Integer(6), Integer(2))
>>> g.is_strongly_regular(parameters=True)
(63, 30, 13, 15)
>>> set(g.spectrum()) == {-Integer(5), Integer(3), Integer(30)}           ␣
↪                       # needs sage.rings.number_field
True
```

The parameters of $Sp(4, q)$ are the same as of $O(5, q)$, but they are not isomorphic if q is odd:

```
sage: G = graphs.SymplecticPolarGraph(4, 3)
sage: G.is_strongly_regular(parameters=True)
```

(continues on next page)

(continued from previous page)

```
(40, 12, 2, 4)

sage: O = graphs.OrthogonalPolarGraph(5, 3)
sage: O.is_strongly_regular(parameters=True)
(40, 12, 2, 4)
sage: O.is_isomorphic(G)
False
sage: S = graphs.SymplecticPolarGraph(6, 4, algorithm='gap')     # not tested␣
↪(long time)
sage: S.is_strongly_regular(parameters=True)                     # not tested␣
↪(long time)
(1365, 340, 83, 85)
```

```
>>> from sage.all import *
>>> G = graphs.SymplecticPolarGraph(Integer(4), Integer(3))
>>> G.is_strongly_regular(parameters=True)
(40, 12, 2, 4)

>>> O = graphs.OrthogonalPolarGraph(Integer(5), Integer(3))
>>> O.is_strongly_regular(parameters=True)
(40, 12, 2, 4)
>>> O.is_isomorphic(G)
False
>>> S = graphs.SymplecticPolarGraph(Integer(6), Integer(4), algorithm='gap')    #␣
↪not tested (long time)
>>> S.is_strongly_regular(parameters=True)                    # not tested (long␣
↪time)
(1365, 340, 83, 85)
```

sage.graphs.generators.classical_geometries.**T2starGeneralizedQuadrangleGraph**(*q*, *dual=False*, *hyperoval=None*, *field=None*, *check_hyperoval=True*, *immutable=False*)

Return the collinearity graph of the generalized quadrangle $T_2^*(q)$, or of its dual

Let $q = 2^k$ and $\Theta = PG(3, q)$. $T_2^*(q)$ is a generalized quadrangle (Wikipedia article Generalized_quadrangle) of order $(q-1, q+1)$, see 3.1.3 in [PT2009]. Fix a plane $\Pi \subset \Theta$ and a hyperoval $O \subset \Pi$. The points of $T_2^*(q) := T_2^*(O)$ are the points of Θ outside Π, and the lines are the lines of Θ outside Π that meet Π in a point of O.

INPUT:

- `q` – a power of two
- `dual` – boolean (default: `False`); whether to return the graph of $T_2^*(O)$ or of the dual $T_2^*(O)$ (when `True`)
- `hyperoval` – a hyperoval (i.e. a complete 2-arc; a set of points in the plane meeting every line in 0 or 2 points) in the plane of points with 0th coordinate 0 in $PG(3, q)$ over the field `field`. Each point of `hyperoval` must be a length 4 vector over `field` with 1st non-0 coordinate equal to 1. By default, `hyperoval` and

field are not specified, and constructed on the fly. In particular, hyperoval we build is the classical one, i.e. a conic with the point of intersection of its tangent lines.

- field – an instance of a finite field of order q, must be provided if hyperoval is provided
- check_hyperoval – boolean (default: True); whether to check hyperoval for correctness or not
- immutable – boolean (default: False); whether to return an immutable or a mutable graph

EXAMPLES:

using the built-in construction:

```
sage: g = graphs.T2starGeneralizedQuadrangleGraph(4); g
T2*(O,4); GQ(3, 5): Graph on 64 vertices
sage: g.is_strongly_regular(parameters=True)
(64, 18, 2, 6)
sage: g = graphs.T2starGeneralizedQuadrangleGraph(4, dual=True); g
T2*(O,4)*; GQ(5, 3): Graph on 96 vertices
sage: g.is_strongly_regular(parameters=True)
(96, 20, 4, 4)
```

```
>>> from sage.all import *
>>> g = graphs.T2starGeneralizedQuadrangleGraph(Integer(4)); g
T2*(O,4); GQ(3, 5): Graph on 64 vertices
>>> g.is_strongly_regular(parameters=True)
(64, 18, 2, 6)
>>> g = graphs.T2starGeneralizedQuadrangleGraph(Integer(4), dual=True); g
T2*(O,4)*; GQ(5, 3): Graph on 96 vertices
>>> g.is_strongly_regular(parameters=True)
(96, 20, 4, 4)
```

supplying your own hyperoval:

```
sage: F = GF(4,'b')
sage: O = [vector(F,(0,0,0,1)),vector(F,(0,0,1,0))] + [vector(F, (0,1,x^2,x))
....:                                                  for x in F]
sage: g = graphs.T2starGeneralizedQuadrangleGraph(4, hyperoval=O, field=F); g
T2*(O,4); GQ(3, 5): Graph on 64 vertices
sage: g.is_strongly_regular(parameters=True)
(64, 18, 2, 6)
```

```
>>> from sage.all import *
>>> F = GF(Integer(4),'b')
>>> O = [vector(F,(Integer(0),Integer(0),Integer(0),Integer(1))),vector(F,
↪(Integer(0),Integer(0),Integer(1),Integer(0)))] + [vector(F, (Integer(0),
↪Integer(1),x**Integer(2),x))
...                                                 for x in F]
>>> g = graphs.T2starGeneralizedQuadrangleGraph(Integer(4), hyperoval=O, field=F);
↪ g
T2*(O,4); GQ(3, 5): Graph on 64 vertices
>>> g.is_strongly_regular(parameters=True)
(64, 18, 2, 6)
```

sage.graphs.generators.classical_geometries.**TaylorTwographDescendantSRG**(*q*, *clique_partition=False*, *immutable=False*)

Return the descendant graph of the Taylor's two-graph for $U_3(q)$, q odd.

This is a strongly regular graph with parameters $(v, k, \lambda, \mu) = (q^3, (q^2+1)(q-1)/2, (q-1)^3/4 - 1, (q^2+1)(q-1)/4)$ obtained as a two-graph descendant of the `Taylor's two-graph` T. This graph admits a partition into cliques of size q, which are useful in *`TaylorTwographSRG()`*, a strongly regular graph on q^3+1 vertices in the Seidel switching class of T, for which we need $(q^2+1)/2$ cliques. The cliques are the q^2 lines on v_0 of the projective plane containing the unital for $U_3(q)$, and intersecting the unital (i.e. the vertices of the graph and the point we remove) in $q+1$ points. This is all taken from §7E of [BL1984].

INPUT:

- `q` – a power of an odd prime number
- `clique_partition` – boolean (default: `False`); when set to `True`, return q^2-1 cliques of size q with empty pairwise intersection. (Removing all of them leaves a clique, too), and the point removed from the unital.
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.TaylorTwographDescendantSRG(3); g
Taylor two-graph descendant SRG: Graph on 27 vertices
sage: g.is_strongly_regular(parameters=True)
(27, 10, 1, 5)
sage: from sage.combinat.designs.twographs import taylor_twograph
sage: T = taylor_twograph(3)                                # long time
sage: g.is_isomorphic(T.descendant(T.ground_set()[1]))      # long time
True
sage: g = graphs.TaylorTwographDescendantSRG(5)             # not tested (long time)
sage: g.is_strongly_regular(parameters=True)                # not tested (long time)
(125, 52, 15, 26)
```

```
>>> from sage.all import *
>>> g = graphs.TaylorTwographDescendantSRG(Integer(3)); g
Taylor two-graph descendant SRG: Graph on 27 vertices
>>> g.is_strongly_regular(parameters=True)
(27, 10, 1, 5)
>>> from sage.combinat.designs.twographs import taylor_twograph
>>> T = taylor_twograph(Integer(3))                                # long time
>>> g.is_isomorphic(T.descendant(T.ground_set()[Integer(1)]))      # long time
True
>>> g = graphs.TaylorTwographDescendantSRG(Integer(5))             # not tested (long␣
↪time)
>>> g.is_strongly_regular(parameters=True)                # not tested (long time)
(125, 52, 15, 26)
```

`sage.graphs.generators.classical_geometries.`**`TaylorTwographSRG`**(*q*, *immutable=False*)

Return a strongly regular graph from the Taylor's two-graph for $U_3(q)$, q odd

This is a strongly regular graph with parameters $(v, k, \lambda, \mu) = (q^3+1, q(q^2+1)/2, (q^2+3)(q-1)/4, (q^2+1)(q+1)/4)$ in the Seidel switching class of `Taylor two-graph`. Details are in §7E of [BL1984].

INPUT:

- `q` – a power of an odd prime number
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

See also

- *TaylorTwographDescendantSRG()*

EXAMPLES:

```
sage: t = graphs.TaylorTwographSRG(3); t                                    #␣
↪needs sage.rings.finite_rings
Taylor two-graph SRG: Graph on 28 vertices
sage: t.is_strongly_regular(parameters=True)                                #␣
↪needs sage.rings.finite_rings
(28, 15, 6, 10)
```

```
>>> from sage.all import *
>>> t = graphs.TaylorTwographSRG(Integer(3)); t                                ␣
↪     # needs sage.rings.finite_rings
Taylor two-graph SRG: Graph on 28 vertices
>>> t.is_strongly_regular(parameters=True)                                  #␣
↪needs sage.rings.finite_rings
(28, 15, 6, 10)
```

sage.graphs.generators.classical_geometries.**UnitaryDualPolarGraph**(*m*, *q*, *immutable=False*)

Return the Dual Unitary Polar Graph $U(m, q)$.

For more information on Unitary Dual Polar graphs, see [BCN1989] and Sect. 2.3.1 of [Coh1981].

INPUT:

- `m`, `q` – integers; q must be a prime power
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

The point graph of a generalized quadrangle (see Wikipedia article Generalized_quadrangle, [PT2009]) of order (8,4):

```
sage: G = graphs.UnitaryDualPolarGraph(5,2); G  # long time                 #␣
↪needs sage.libs.gap
Unitary Dual Polar Graph DU(5, 2); GQ(8, 4): Graph on 297 vertices
sage: G.is_strongly_regular(parameters=True)    # long time                 #␣
↪needs sage.libs.gap
(297, 40, 7, 5)
```

```
>>> from sage.all import *
>>> G = graphs.UnitaryDualPolarGraph(Integer(5),Integer(2)); G  # long time      ␣
↪              # needs sage.libs.gap
Unitary Dual Polar Graph DU(5, 2); GQ(8, 4): Graph on 297 vertices
>>> G.is_strongly_regular(parameters=True)    # long time                   #␣
↪needs sage.libs.gap
(297, 40, 7, 5)
```

Another way to get the generalized quadrangle of order (2,4):

```
sage: G = graphs.UnitaryDualPolarGraph(4,2); G                                  #␣
↪needs sage.libs.gap
Unitary Dual Polar Graph DU(4, 2); GQ(2, 4): Graph on 27 vertices
sage: G.is_isomorphic(graphs.OrthogonalPolarGraph(6,2,'-'))                     #␣
↪needs sage.libs.gap
True
```

```
>>> from sage.all import *
>>> G = graphs.UnitaryDualPolarGraph(Integer(4),Integer(2)); G                  ␣
↪              # needs sage.libs.gap
Unitary Dual Polar Graph DU(4, 2); GQ(2, 4): Graph on 27 vertices
>>> G.is_isomorphic(graphs.OrthogonalPolarGraph(Integer(6),Integer(2),'-'))     ␣
↪              # needs sage.libs.gap
True
```

A bigger graph:

```
sage: G = graphs.UnitaryDualPolarGraph(6,2); G    # not tested (long time)
Unitary Dual Polar Graph DU(6, 2): Graph on 891 vertices
sage: G.is_distance_regular(parameters=True)      # not tested (long time)
([42, 40, 32, None], [None, 1, 5, 21])
```

```
>>> from sage.all import *
>>> G = graphs.UnitaryDualPolarGraph(Integer(6),Integer(2)); G    # not tested␣
↪(long time)
Unitary Dual Polar Graph DU(6, 2): Graph on 891 vertices
>>> G.is_distance_regular(parameters=True)      # not tested (long time)
([42, 40, 32, None], [None, 1, 5, 21])
```

sage.graphs.generators.classical_geometries.**UnitaryPolarGraph**(*m*, *q*, *algorithm='gap'*, *immutable=False*)

Return the Unitary Polar Graph $U(m, q)$.

For more information on Unitary Polar graphs, see the page of Andries Brouwer's website.

INPUT:

- `m`, `q` – integers; q must be a prime power
- `algorithm` – string (default: `'gap'`); if set to `'gap'` then the computation is carried via GAP library interface, computing totally singular subspaces, which is faster for large examples (especially with $q > 2$). Otherwise it is done directly.
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: G = graphs.UnitaryPolarGraph(4,2); G
Unitary Polar Graph U(4, 2); GQ(4, 2): Graph on 45 vertices
sage: G.is_strongly_regular(parameters=True)
(45, 12, 3, 3)
sage: graphs.UnitaryPolarGraph(5,2).is_strongly_regular(parameters=True)
(165, 36, 3, 9)
sage: graphs.UnitaryPolarGraph(6,2)      # not tested (long time)
Unitary Polar Graph U(6, 2): Graph on 693 vertices
```

```
>>> from sage.all import *
>>> G = graphs.UnitaryPolarGraph(Integer(4),Integer(2)); G
Unitary Polar Graph U(4, 2); GQ(4, 2): Graph on 45 vertices
>>> G.is_strongly_regular(parameters=True)
(45, 12, 3, 3)
>>> graphs.UnitaryPolarGraph(Integer(5),Integer(2)).is_strongly_
↪regular(parameters=True)
(165, 36, 3, 9)
>>> graphs.UnitaryPolarGraph(Integer(6),Integer(2))     # not tested (long time)
Unitary Polar Graph U(6, 2): Graph on 693 vertices
```

2.10 Graphs with a given degree sequence

The methods defined here appear in *sage.graphs.graph_generators*.

sage.graphs.generators.degree_sequence.**DegreeSequence**(*deg_sequence*, *immutable=False*)

Return a graph with the given degree sequence.

This method raises a NetworkX error if the proposed degree sequence cannot be that of a graph.

Graph returned is the one returned by the Havel-Hakimi algorithm, which constructs a simple graph by connecting vertices of highest degree to other vertices of highest degree, resorting the remaining vertices by degree and repeating the process. See Theorem 1.4 in [CL1996].

INPUT:

- `deg_sequence` – list of integers with each entry corresponding to the degree of a different vertex
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: G = graphs.DegreeSequence([3,3,3,3])                                    #␣
↪needs networkx
sage: G.edges(sort=True, labels=False)                                        #␣
↪needs networkx
[(0, 1), (0, 2), (0, 3), (1, 2), (1, 3), (2, 3)]
sage: G.show()                            # long time                         #␣
↪needs networkx sage.plot
```

```
>>> from sage.all import *
>>> G = graphs.DegreeSequence([Integer(3),Integer(3),Integer(3),Integer(3)])     ␣
↪                                 # needs networkx
>>> G.edges(sort=True, labels=False)                                          #␣
↪needs networkx
[(0, 1), (0, 2), (0, 3), (1, 2), (1, 3), (2, 3)]
>>> G.show()                              # long time                         #␣
↪needs networkx sage.plot
```

```
sage: G = graphs.DegreeSequence([3,3,3,3,3,3,3,3,3,3,3,3,3,3,3,3,3,3,3,3,3,3])  #␣
↪needs networkx
sage: G.show()                            # long time                         #␣
↪needs networkx sage.plot
```

```
>>> from sage.all import *
>>> G = graphs.DegreeSequence([Integer(3),Integer(3),Integer(3),Integer(3),
↪Integer(3),Integer(3),Integer(3),Integer(3),Integer(3),Integer(3),Integer(3),
↪Integer(3),Integer(3),Integer(3),Integer(3),Integer(3),Integer(3),Integer(3),
↪Integer(3),Integer(3),Integer(3),Integer(3)])  # needs networkx
>>> G.show()                            # long time                              #␣
↪needs networkx sage.plot
```

```
sage: G = graphs.DegreeSequence([4,4,4,4,4,4,4,4])                               #␣
↪needs networkx
sage: G.show()                              # long time                          #␣
↪needs networkx sage.plot
```

```
>>> from sage.all import *
>>> G = graphs.DegreeSequence([Integer(4),Integer(4),Integer(4),Integer(4),
↪Integer(4),Integer(4),Integer(4),Integer(4)])                                   #␣
↪needs networkx
>>> G.show()                            # long time                              #␣
↪needs networkx sage.plot
```

```
sage: G = graphs.DegreeSequence([1,2,3,4,3,4,3,2,3,2,1])                         #␣
↪needs networkx
sage: G.show()                              # long time                          #␣
↪needs networkx sage.plot
```

```
>>> from sage.all import *
>>> G = graphs.DegreeSequence([Integer(1),Integer(2),Integer(3),Integer(4),
↪Integer(3),Integer(4),Integer(3),Integer(2),Integer(3),Integer(2),Integer(1)]) ␣
↪                      # needs networkx
>>> G.show()                            # long time                              #␣
↪needs networkx sage.plot
```

sage.graphs.generators.degree_sequence.**DegreeSequenceBipartite**(*s1*, *s2*, *immutable=False*)

Return a bipartite graph whose two sets have the given degree sequences.

Given two different sequences of degrees s_1 and s_2, this functions returns (if possible) a bipartite graph on sets A and B such that the vertices in A have s_1 as their degree sequence, while s_2 is the degree sequence of the vertices in B.

INPUT:

- `s1` – list of integers corresponding to the degree sequence of the first set of vertices
- `s2` – list of integers corresponding to the degree sequence of the second set of vertices
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

ALGORITHM:

This function works through the computation of the matrix given by the Gale-Ryser theorem, which is in this case the adjacency matrix of the bipartite graph.

EXAMPLES:

If we are given as sequences `[2,2,2,2,2]` and `[5,5]` we are given as expected the complete bipartite graph $K_{2,5}$:

```
sage: g = graphs.DegreeSequenceBipartite([2,2,2,2,2],[5,5])                    #␣
↪needs sage.combinat sage.modules
sage: g.is_isomorphic(graphs.CompleteBipartiteGraph(5,2))                      #␣
↪needs sage.combinat sage.modules
True
```

```
>>> from sage.all import *
>>> g = graphs.DegreeSequenceBipartite([Integer(2),Integer(2),Integer(2),
↪Integer(2),Integer(2)],[Integer(5),Integer(5)])                    # needs␣
↪sage.combinat sage.modules
>>> g.is_isomorphic(graphs.CompleteBipartiteGraph(Integer(5),Integer(2)))      ␣
↪              # needs sage.combinat sage.modules
True
```

Some sequences being incompatible if, for example, their sums are different, the functions raises a `ValueError` when no graph corresponding to the degree sequences exists:

```
sage: g = graphs.DegreeSequenceBipartite([2,2,2,2,1],[5,5])                    #␣
↪needs sage.combinat sage.modules
Traceback (most recent call last):
...
ValueError: there exists no bipartite graph corresponding to the given degree␣
↪sequences
```

```
>>> from sage.all import *
>>> g = graphs.DegreeSequenceBipartite([Integer(2),Integer(2),Integer(2),
↪Integer(2),Integer(1)],[Integer(5),Integer(5)])                    # needs␣
↪sage.combinat sage.modules
Traceback (most recent call last):
...
ValueError: there exists no bipartite graph corresponding to the given degree␣
↪sequences
```

sage.graphs.generators.degree_sequence.**DegreeSequenceConfigurationModel**(*deg_sequence*, *seed=None*, *immutable=False*)

Return a random pseudograph with the given degree sequence.

This method raises a NetworkX error if the proposed degree sequence cannot be that of a graph with multiple edges and loops.

One requirement is that the sum of the degrees must be even, since every edge must be incident with two vertices.

INPUT:

- `deg_sequence` – list of integers with each entry corresponding to the expected degree of a different vertex
- `seed` – (optional) a `random.Random` seed or a Python `int` for the random number generator
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: G = graphs.DegreeSequenceConfigurationModel([1,1])                       #␣
↪needs networkx
sage: G.adjacency_matrix()                                                     #␣
```

(continues on next page)

(continued from previous page)

```
↪needs networkx sage.modules
[0 1]
[1 0]
```

```
>>> from sage.all import *
>>> G = graphs.DegreeSequenceConfigurationModel([Integer(1),Integer(1)])␣
↪              # needs networkx
>>> G.adjacency_matrix()                                                #␣
↪needs networkx sage.modules
[0 1]
[1 0]
```

The output is allowed to contain both loops and multiple edges:

```
sage: # needs networkx
sage: deg_sequence = [3,3,3,3,3,3,3,3,3,3,3,3,3,3,3,3,3,3,3,3]
sage: G = graphs.DegreeSequenceConfigurationModel(deg_sequence)
sage: G.order(), G.size()
(20, 30)
sage: G.has_loops() or G.has_multiple_edges()  # random
True
sage: G.show()                           # long time                          #␣
↪needs sage.plot
```

```
>>> from sage.all import *
>>> # needs networkx
>>> deg_sequence = [Integer(3),Integer(3),Integer(3),Integer(3),Integer(3),
↪Integer(3),Integer(3),Integer(3),Integer(3),Integer(3),Integer(3),Integer(3),
↪Integer(3),Integer(3),Integer(3),Integer(3),Integer(3),Integer(3),Integer(3),
↪Integer(3)]
>>> G = graphs.DegreeSequenceConfigurationModel(deg_sequence)
>>> G.order(), G.size()
(20, 30)
>>> G.has_loops() or G.has_multiple_edges()  # random
True
>>> G.show()                           # long time                          #␣
↪needs sage.plot
```

REFERENCE:

[New2003]

sage.graphs.generators.degree_sequence.**DegreeSequenceExpected**(*deg_sequence*, *seed=None*, *immutable=False*)

Return a random graph with expected given degree sequence.

This method raises a NetworkX error if the proposed degree sequence cannot be that of a graph.

One requirement is that the sum of the degrees must be even, since every edge must be incident with two vertices.

INPUT:

- `deg_sequence` – list of integers with each entry corresponding to the expected degree of a different vertex
- `seed` – (optional) a `random.Random` seed or a Python `int` for the random number generator

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: G = graphs.DegreeSequenceExpected([1,2,3,2,3]); G                    #␣
↪needs networkx
Looped graph on 5 vertices
sage: G.show()                              # long time                    #␣
↪needs networkx sage.plot
```

```
>>> from sage.all import *
>>> G = graphs.DegreeSequenceExpected([Integer(1),Integer(2),Integer(3),
↪Integer(2),Integer(3)]); G                          # needs networkx
Looped graph on 5 vertices
>>> G.show()                              # long time                     #␣
↪needs networkx sage.plot
```

REFERENCE:

[CL2002]

`sage.graphs.generators.degree_sequence.`**`DegreeSequenceTree`**(*deg_sequence*, *immutable=False*)

Return a tree with the given degree sequence.

This method raises a NetworkX error if the proposed degree sequence cannot be that of a tree.

Since every tree has one more vertex than edge, the degree sequence must satisfy `len(deg_sequence) - sum(deg_sequence)/2 == 1`.

INPUT:

- `deg_sequence` – list of integers with each entry corresponding to the expected degree of a different vertex
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: G = graphs.DegreeSequenceTree([3,1,3,3,1,1,1,2,1]); G                #␣
↪needs networkx
Graph on 9 vertices
sage: G.show()                              # long time                    #␣
↪needs networkx sage.plot
```

```
>>> from sage.all import *
>>> G = graphs.DegreeSequenceTree([Integer(3),Integer(1),Integer(3),Integer(3),
↪Integer(1),Integer(1),Integer(1),Integer(2),Integer(1)]); G
↪# needs networkx
Graph on 9 vertices
>>> G.show()                              # long time                     #␣
↪needs networkx sage.plot
```

2.11 Database of distance regular graphs

In this module we construct several distance regular graphs and group them in a function that maps intersection arrays to graphs.

For a survey on distance-regular graph see [BCN1989] or [VDKT2016].

EXAMPLES:

```
sage: G = graphs.cocliques_HoffmannSingleton()
sage: G.is_distance_regular()
True
sage: H = graphs.distance_regular_graph([15, 14, 10, 3, 1, 5, 12, 15])
sage: H == G
True
sage: G = graphs.distance_regular_graph([27, 10, 1, 1, 10, 27])
sage: G.is_distance_regular(True)
([27, 10, 1, None], [None, 1, 10, 27])
```

```
>>> from sage.all import *
>>> G = graphs.cocliques_HoffmannSingleton()
>>> G.is_distance_regular()
True
>>> H = graphs.distance_regular_graph([Integer(15), Integer(14), Integer(10),␣
↪Integer(3), Integer(1), Integer(5), Integer(12), Integer(15)])
>>> H == G
True
>>> G = graphs.distance_regular_graph([Integer(27), Integer(10), Integer(1),␣
↪Integer(1), Integer(10), Integer(27)])
>>> G.is_distance_regular(True)
([27, 10, 1, None], [None, 1, 10, 27])
```

AUTHORS:

- Ivo Maffei (2020-07-28): initial version

sage.graphs.generators.distance_regular.**AlternatingFormsGraph**(*n*, *q*, *immutable=False*)

Return the alternating forms graph with the given parameters.

This builds a graph whose vertices are all $n`x`n$ skew-symmetric matrices over $GF(q)$ with zero diagonal. Two vertices are adjacent if and only if the difference of the two matrices has rank 2.

This graph is distance-regular with classical parameters $(\lfloor \frac{n}{2} \rfloor, q^2, q^2-1, q^{2\lceil \frac{n}{2} \rceil - 1})$.

INPUT:

- `n` – integer
- `q` – a prime power
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: G = graphs.AlternatingFormsGraph(5, 2)  # long time
sage: G.is_distance_regular(True)  # long time
([155, 112, None], [None, 1, 20])
```

```
>>> from sage.all import *
>>> G = graphs.AlternatingFormsGraph(Integer(5), Integer(2))  # long time
>>> G.is_distance_regular(True)  # long time
([155, 112, None], [None, 1, 20])
```

REFERENCES:

See [BCN1989] pp. 282-284 for a rather detailed discussion, otherwise see [VDKT2016] p. 22.

sage.graphs.generators.distance_regular.**BilinearFormsGraph**(*d*, *e*, *q*, *immutable=False*)

Return a bilinear forms graph with the given parameters.

This builds a graph whose vertices are all $d`x`e$ matrices over $GF(q)$. Two vertices are adjacent if the difference of the two matrices has rank 1.

The graph is distance-regular with classical parameters $(\min(d, e), q, q-1, q^{\max(d,e)}-1)$.

INPUT:

- `d`, `e` – integers; dimension of the matrices
- `q` – integer; a prime power
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: G = graphs.BilinearFormsGraph(3, 3, 2)
sage: G.is_distance_regular(True)
([49, 36, 16, None], [None, 1, 6, 28])
sage: G = graphs.BilinearFormsGraph(3,3,3)      # not tested (20 s)              #␣
↪needs sage.rings.finite_rings
sage: G.order()                           # not tested (due to above)            #␣
↪needs sage.rings.finite_rings
19683
sage: G = graphs.BilinearFormsGraph(3, 4, 2)    # long time                      #␣
↪needs sage.rings.finite_rings
sage: G.is_distance_regular(True)       # long time                              #␣
↪needs sage.rings.finite_rings
([105, 84, 48, None], [None, 1, 6, 28])
```

```
>>> from sage.all import *
>>> G = graphs.BilinearFormsGraph(Integer(3), Integer(3), Integer(2))
>>> G.is_distance_regular(True)
([49, 36, 16, None], [None, 1, 6, 28])
>>> G = graphs.BilinearFormsGraph(Integer(3),Integer(3),Integer(3))      # not␣
↪tested (20 s)             # needs sage.rings.finite_rings
>>> G.order()                           # not tested (due to above)             #␣
↪needs sage.rings.finite_rings
19683
>>> G = graphs.BilinearFormsGraph(Integer(3), Integer(4), Integer(2))    # long␣
↪time                      # needs sage.rings.finite_rings
>>> G.is_distance_regular(True)       # long time                              #␣
↪needs sage.rings.finite_rings
([105, 84, 48, None], [None, 1, 6, 28])
```

REFERENCES:

See [BCN1989] pp. 280-282 for a rather detailed discussion, otherwise see [VDKT2016] p. 21.

sage.graphs.generators.distance_regular.**ConwaySmith_for_3S7**(*immutable=False*)

Return the Conway-Smith graph related to $3Sym(7)$.

This is a distance-regular graph with intersection array $[10, 6, 4, 1; 1, 2, 6, 10]$.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: G = graphs.ConwaySmith_for_3S7()                                          #␣
↪needs sage.modules sage.rings.finite_rings sage.rings.number_field
sage: G.is_distance_regular(True)                                               #␣
↪needs sage.modules sage.rings.finite_rings sage.rings.number_field
([10, 6, 4, 1, None], [None, 1, 2, 6, 10])
```

```
>>> from sage.all import *
>>> G = graphs.ConwaySmith_for_3S7()                                            #␣
↪needs sage.modules sage.rings.finite_rings sage.rings.number_field
>>> G.is_distance_regular(True)                                                 #␣
↪needs sage.modules sage.rings.finite_rings sage.rings.number_field
([10, 6, 4, 1, None], [None, 1, 2, 6, 10])
```

REFERENCES:

A description and construction of this graph can be found in [BCN1989] p. 399.

sage.graphs.generators.distance_regular.**DoubleGrassmannGraph**(*q*, *e*, *immutable=False*)

Return the bipartite double of the distance-e graph of the Grassmann graph $J_q(n, e)$.

This graph can also be described as follows: Let V be the vector space of dimension n over $GF(q)$. The vertex set is the set of $e + 1$ or e subspaces of V. Two vertices are adjacent if one subspace is contained in the other.

This graph is distance-transitive.

INPUT:

- `q` – a prime power
- `e` – integer
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: G = graphs.DoubleGrassmannGraph(2,1)                                      #␣
↪needs sage.modules
sage: G.diameter()                                                              #␣
↪needs sage.modules
3
sage: G.is_distance_regular(True)                                               #␣
↪needs sage.modules
([3, 2, 2, None], [None, 1, 1, 3])
```

```
>>> from sage.all import *
>>> G = graphs.DoubleGrassmannGraph(Integer(2),Integer(1))                      ␣
↪              # needs sage.modules
>>> G.diameter()                                                                #␣
↪needs sage.modules
3
>>> G.is_distance_regular(True)                                                 #␣
↪needs sage.modules
([3, 2, 2, None], [None, 1, 1, 3])
```

REFERENCES:

See [BCN1989] pp. 272, 273 or [VDKT2016] p. 25.

sage.graphs.generators.distance_regular.**DoubleOddGraph**(*n*, *immutable=False*)

Return the double odd graph on $2n + 1$ points.

The graph is obtained using the subsets of size n and $n+1$ of $1, 2, ..., 2n + 1$ as vertices. Two vertices are adjacent if one is included in the other.

The graph is distance-transitive.

INPUT:

- `n` – integer; must be greater than 0
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: G = graphs.DoubleOddGraph(5)
sage: G.is_distance_regular(True)
([6, 5, 5, 4, 4, 3, 3, 2, 2, 1, 1, None],
 [None, 1, 1, 2, 2, 3, 3, 4, 4, 5, 5, 6])
sage: G = graphs.DoubleOddGraph(3)
sage: G.diameter()
7
sage: G.is_distance_regular(True)
([4, 3, 3, 2, 2, 1, 1, None], [None, 1, 1, 2, 2, 3, 3, 4])
```

```
>>> from sage.all import *
>>> G = graphs.DoubleOddGraph(Integer(5))
>>> G.is_distance_regular(True)
([6, 5, 5, 4, 4, 3, 3, 2, 2, 1, 1, None],
 [None, 1, 1, 2, 2, 3, 3, 4, 4, 5, 5, 6])
>>> G = graphs.DoubleOddGraph(Integer(3))
>>> G.diameter()
7
>>> G.is_distance_regular(True)
([4, 3, 3, 2, 2, 1, 1, None], [None, 1, 1, 2, 2, 3, 3, 4])
```

REFERENCES:

See [BCN1989] pp. 259-261 or [VDKT2016] p. 25.

sage.graphs.generators.distance_regular.**DoublyTruncatedWittGraph**(*immutable=False*)

Return the doubly truncated Witt graph.

This builds the truncated Witt graph, then removes all vertices whose codeword start with a 1.

The graph is distance-regular with intersection array $[7, 6, 4, 4; 1, 1, 1, 6]$.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: G = graphs.DoublyTruncatedWittGraph()                                    #␣
↪needs sage.libs.pari sage.modules
sage: G.is_distance_regular(True)                                              #␣
↪needs sage.libs.pari sage.modules
([7, 6, 4, 4, None], [None, 1, 1, 1, 6])
```

```
>>> from sage.all import *
>>> G = graphs.DoublyTruncatedWittGraph()                                    #␣
↪needs sage.libs.pari sage.modules
>>> G.is_distance_regular(True)                                              #␣
↪needs sage.libs.pari sage.modules
([7, 6, 4, 4, None], [None, 1, 1, 1, 6])
```

REFERENCES:

A description and construction of this graph can be found in [BCN1989] p. 368.

sage.graphs.generators.distance_regular.**FosterGraph3S6**(*immutable=False*)

Return the Foster graph for $3.Sym(6)$.

This graph is distance-regular with intersection array $[6, 4, 2, 1; 1, 1, 4, 6]$.

The graph is also distance transitive.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: G = graphs.FosterGraph3S6()                                            #␣
↪needs sage.libs.gap
sage: G.is_distance_regular(True)                                            #␣
↪needs sage.libs.gap
([6, 4, 2, 1, None], [None, 1, 1, 4, 6])
```

```
>>> from sage.all import *
>>> G = graphs.FosterGraph3S6()                                              #␣
↪needs sage.libs.gap
>>> G.is_distance_regular(True)                                              #␣
↪needs sage.libs.gap
([6, 4, 2, 1, None], [None, 1, 1, 4, 6])
```

REFERENCES:

A description and construction of this graph can be found in [BCN1989] p. 397.

sage.graphs.generators.distance_regular.**GeneralisedDodecagonGraph**(*s*, *t*, *immutable=False*)

Return the point-graph of a generalised dodecagon of order (s, t).

INPUT:

- `s`, `t` – integers; order of the generalised dodecagon
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: # optional - gap_package_atlasrep internet
sage: G = graphs.GeneralisedDodecagonGraph(1, 5)
sage: G.is_distance_regular(True)
([6, 5, 5, 5, 5, 5, None], [None, 1, 1, 1, 1, 1, 6])
sage: H = graphs.GeneralisedDodecagonGraph(5, 1)
sage: H.order()
23436
```

(continues on next page)

(continued from previous page)

```
sage: H.is_distance_regular(True)       # not tested (6 min)
([10, 5, 5, 5, 5, 5, None], [None, 1, 1, 1, 1, 1, 2])
```

```
>>> from sage.all import *
>>> # optional - gap_package_atlasrep internet
>>> G = graphs.GeneralisedDodecagonGraph(Integer(1), Integer(5))
>>> G.is_distance_regular(True)
([6, 5, 5, 5, 5, 5, None], [None, 1, 1, 1, 1, 1, 6])
>>> H = graphs.GeneralisedDodecagonGraph(Integer(5), Integer(1))
>>> H.order()
23436
>>> H.is_distance_regular(True)       # not tested (6 min)
([10, 5, 5, 5, 5, 5, None], [None, 1, 1, 1, 1, 1, 2])
```

Note

This function indirectly uses the GAP's AtlasRep package. Thus you may need an internet connection and the optional Sage's package `gap_packages`.

REFERENCES:

See [BCN1989] pp. 200-205 for a discussion of distance-regular graphs from generalised polygons.

`sage.graphs.generators.distance_regular.`**`GeneralisedHexagonGraph`**(*s*, *t*, *immutable=False*)

Return the point-graph of a generalised hexagon of order (s, t).

INPUT:

- `s`, `t` – integers; order of the generalised hexagon
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: G = graphs.GeneralisedHexagonGraph(5, 5)          # optional - gap_package_
↪atlasrep internet
sage: G.is_distance_regular(True)                       # optional - gap_package_
↪atlasrep internet
([30, 25, 25, None], [None, 1, 1, 6])
sage: G = graphs.GeneralisedHexagonGraph(7, 1)
sage: G.is_distance_regular(True)
([14, 7, 7, None], [None, 1, 1, 2])
sage: graphs.GeneralisedHexagonGraph(1, 1)
Cycle graph: Graph on 6 vertices
```

```
>>> from sage.all import *
>>> G = graphs.GeneralisedHexagonGraph(Integer(5), Integer(5))          #␣
↪optional - gap_package_atlasrep internet
>>> G.is_distance_regular(True)                       # optional - gap_package_
↪atlasrep internet
([30, 25, 25, None], [None, 1, 1, 6])
>>> G = graphs.GeneralisedHexagonGraph(Integer(7), Integer(1))
>>> G.is_distance_regular(True)
```

(continues on next page)

(continued from previous page)

```
([14, 7, 7, None], [None, 1, 1, 2])
>>> graphs.GeneralisedHexagonGraph(Integer(1), Integer(1))
Cycle graph: Graph on 6 vertices
```

Note

This function uses the GAP's AtlasRep package to build GHs of order (q, q), (q, q^3) or (q^3, q). For those graphs you need an internet connection and Sage's optional package `gap_packages`.

REFERENCES:

See [BCN1989] pp. 200-205 for a discussion of distance-regular graphs from generalised polygons.

`sage.graphs.generators.distance_regular.`**`GeneralisedOctagonGraph`**(*s*, *t*, *immutable=False*)

Return the point-graph of a generalised octagon of order (s, t).

INPUT:

- `s, t` – integers; order of the generalised octagon
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: G = graphs.GeneralisedOctagonGraph(1, 4)          # optional - database_
↪graphs
sage: G.is_distance_regular(True)                       # optional - database_
↪graphs
([5, 4, 4, 4, None], [None, 1, 1, 1, 5])
sage: G = graphs.GeneralisedOctagonGraph(2, 4)          # optional - gap_package_
↪atlasrep internet
sage: G.is_distance_regular(True)                       # optional - gap_package_
↪atlasrep internet
([10, 8, 8, 8, None], [None, 1, 1, 1, 5])
sage: G = graphs.GeneralisedOctagonGraph(5, 1)          # optional - database_
↪graphs
sage: G.is_distance_regular(True)                       # optional - database_
↪graphs
([10, 5, 5, 5, None], [None, 1, 1, 1, 2])
```

```
>>> from sage.all import *
>>> G = graphs.GeneralisedOctagonGraph(Integer(1), Integer(4))          #␣
↪optional - database_graphs
>>> G.is_distance_regular(True)                       # optional - database_graphs
([5, 4, 4, 4, None], [None, 1, 1, 1, 5])
>>> G = graphs.GeneralisedOctagonGraph(Integer(2), Integer(4))          #␣
↪optional - gap_package_atlasrep internet
>>> G.is_distance_regular(True)                       # optional - gap_package_
↪atlasrep internet
([10, 8, 8, 8, None], [None, 1, 1, 1, 5])
>>> G = graphs.GeneralisedOctagonGraph(Integer(5), Integer(1))          #␣
↪optional - database_graphs
```

(continues on next page)

(continued from previous page)

```
>>> G.is_distance_regular(True)                          # optional - database_graphs
([10, 5, 5, 5, None], [None, 1, 1, 1, 2])
```

Note

This function uses the GAP's AtlasRep package to build the graphs of order $(2, 4)$ or $(4, 2)$. For those graphs you need an internet connection and Sage's optional package `gap_packages`.

REFERENCES:

See [BCN1989] pp. 200-205 for a discussion of distance-regular graphs from generalised polygons.

`sage.graphs.generators.distance_regular.`**`GrassmannGraph`**(*q*, *n*, *input_e*, *immutable=False*)

Return the Grassmann graph with parameters (q, n, e).

This builds the Grassmann graph $J_q(n, e)$. That is, for a vector space $V = \mathbb{F}(q)^n$ the output is the graph on the subspaces of dimension e where two subspaces are adjacent if their intersection has dimension $e - 1$.

This graph is distance-regular with classical parameters $(\min(e, n - e), q, q, \begin{bmatrix} n-e+1 \\ 1 \end{bmatrix}_q - 1)$

INPUT:

- `q` – a prime power
- `n`, `e` – integers with $n > e + 1$
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: G = graphs.GrassmannGraph(2, 4, 2)                                    #␣
↪needs sage.modules sage.rings.finite_rings
sage: G.is_distance_regular(True)                                           #␣
↪needs sage.modules sage.rings.finite_rings
([18, 8, None], [None, 1, 9])
```

```
>>> from sage.all import *
>>> G = graphs.GrassmannGraph(Integer(2), Integer(4), Integer(2))                ␣
↪                       # needs sage.modules sage.rings.finite_rings
>>> G.is_distance_regular(True)                                              #␣
↪needs sage.modules sage.rings.finite_rings
([18, 8, None], [None, 1, 9])
```

REFERENCES:

See [BCN1989] pp. 268-272 or [VDKT2016] p. 21.

`sage.graphs.generators.distance_regular.`**`HalfCube`**(*n*, *immutable=False*)

Return the halved cube in n dimensions.

The graph is distance-regular with classical parameters $(\lfloor \frac{n}{2} \rfloor, 1, 2, 2\lceil \frac{n}{2} \rceil - 1)$.

INPUT:

- `n` – integer; must be greater than 2
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: G = graphs.HalfCube(8)
sage: G.is_distance_regular(True)
([28, 15, 6, 1, None], [None, 1, 6, 15, 28])
sage: G = graphs.HalfCube(4)
sage: G.is_distance_regular(True)
([6, 1, None], [None, 1, 6])
```

```
>>> from sage.all import *
>>> G = graphs.HalfCube(Integer(8))
>>> G.is_distance_regular(True)
([28, 15, 6, 1, None], [None, 1, 6, 15, 28])
>>> G = graphs.HalfCube(Integer(4))
>>> G.is_distance_regular(True)
([6, 1, None], [None, 1, 6])
```

REFERENCES:

See [BCN1989] pp. 264, 265 or [VDKT2016] p. 21. This construction can be found on Wikipedia article Halved_cube_graph#Equivalent_constructions

sage.graphs.generators.distance_regular.**HermitianFormsGraph**(*n*, *r*, *immutable=False*)

Return the Hermitian forms graph with the given parameters.

We build a graph whose vertices are all $n \times n$ Hermitian matrices over `GF(r^2)`. Two vertices are adjacent if the difference of the two vertices has rank 1.

This graph is distance-regular with classical parameters $(n, -r, -r-1, -(-r)^d - 1)$.

INPUT:

- `n` – integer
- `r` – a prime power
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: G = graphs.HermitianFormsGraph(2, 2)
sage: G.is_distance_regular(True)
([5, 4, None], [None, 1, 2])
sage: G = graphs.HermitianFormsGraph(3, 3)      # not tested (2 min)
sage: G.order()                                 # not tested (because of the above)
19683
```

```
>>> from sage.all import *
>>> G = graphs.HermitianFormsGraph(Integer(2), Integer(2))
>>> G.is_distance_regular(True)
([5, 4, None], [None, 1, 2])
>>> G = graphs.HermitianFormsGraph(Integer(3), Integer(3))      # not tested (2␣
↪min)
>>> G.order()                                 # not tested (because of the above)
19683
```

REFERENCES:

See [BCN1989] p. 285 or [VDKT2016] p. 22.

sage.graphs.generators.distance_regular.**IvanovIvanovFaradjevGraph**(*immutable=False*)

Return the IvanovIvanovFaradjev graph.

The graph is distance-transitive with automorphism group $3.M_{22}$.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: G = graphs.IvanovIvanovFaradjevGraph()             # optional - internet gap_
↪package_atlasrep
sage: G.is_distance_regular(True)                        # optional - internet gap_
↪package_atlasrep
([7, 6, 4, 4, 4, 1, 1, 1, None], [None, 1, 1, 1, 2, 4, 4, 6, 7])
```

```
>>> from sage.all import *
>>> G = graphs.IvanovIvanovFaradjevGraph()             # optional - internet gap_
↪package_atlasrep
>>> G.is_distance_regular(True)                        # optional - internet gap_
↪package_atlasrep
([7, 6, 4, 4, 4, 1, 1, 1, None], [None, 1, 1, 1, 2, 4, 4, 6, 7])
```

REFERENCES:

A description and construction of this graph can be found in [BCN1989] p. 369.

sage.graphs.generators.distance_regular.**J2Graph**(*immutable=False*)

Return the distance-transitive graph with automorphism group J_2.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: G = graphs.J2Graph()                               # optional - internet gap_
↪package_atlasrep
sage: G.is_distance_regular(True)                        # optional - internet gap_
↪package_atlasrep
([10, 8, 8, 2, None], [None, 1, 1, 4, 5])
```

```
>>> from sage.all import *
>>> G = graphs.J2Graph()                               # optional - internet gap_
↪package_atlasrep
>>> G.is_distance_regular(True)                        # optional - internet gap_
↪package_atlasrep
([10, 8, 8, 2, None], [None, 1, 1, 4, 5])
```

REFERENCES:

A description and construction of this graph can be found in [BCN1989] p. 408.

sage.graphs.generators.distance_regular.**LargeWittGraph**(*immutable=False*)

Return the large Witt graph.

This is a distance-regular graph with intersection array $[30, 28, 24; 1, 3, 15]$.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.LargeWittGraph()                                          #␣
↪needs sage.libs.pari sage.modules
sage: g.is_distance_regular(True)                                          #␣
↪needs sage.libs.pari sage.modules
([30, 28, 24, None], [None, 1, 3, 15])
```

```
>>> from sage.all import *
>>> g = graphs.LargeWittGraph()                                            #␣
↪needs sage.libs.pari sage.modules
>>> g.is_distance_regular(True)                                            #␣
↪needs sage.libs.pari sage.modules
([30, 28, 24, None], [None, 1, 3, 15])
```

REFERENCES:

A description of this graph can be found in [BCN1989] p. 366. This construction is taken from http://mathworld.wolfram.com/LargeWittGraph.html

`sage.graphs.generators.distance_regular.`**`LeonardGraph`**(*immutable=False*)

Return the Leonard graph.

The graph is distance-regular with intersection array $[12, 11, 10, 7; 1, 2, 5, 12]$.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: G = graphs.LeonardGraph()                                            #␣
↪needs sage.combinat sage.modules
sage: G.is_distance_regular(True)                                          #␣
↪needs sage.combinat sage.modules
([12, 11, 10, 7, None], [None, 1, 2, 5, 12])
```

```
>>> from sage.all import *
>>> G = graphs.LeonardGraph()                                              #␣
↪needs sage.combinat sage.modules
>>> G.is_distance_regular(True)                                            #␣
↪needs sage.combinat sage.modules
([12, 11, 10, 7, None], [None, 1, 2, 5, 12])
```

REFERENCES:

For a description of this graph see [BCN1989] p. 371.

`sage.graphs.generators.distance_regular.`**`TruncatedWittGraph`**(*immutable=False*)

Return the truncated Witt graph.

This builds the large Witt graph, then removes all vertices whose codeword start with a 1.

The graph is distance-regular with intersection array $[15, 14, 12; 1, 1, 9]$.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: # long time, needs sage.libs.pari sage.modules
sage: G = graphs.TruncatedWittGraph()
sage: G.is_distance_regular(True)
([15, 14, 12, None], [None, 1, 1, 9])
```

```
>>> from sage.all import *
>>> # long time, needs sage.libs.pari sage.modules
>>> G = graphs.TruncatedWittGraph()
>>> G.is_distance_regular(True)
([15, 14, 12, None], [None, 1, 1, 9])
```

REFERENCES:

A description and construction of this graph can be found in [BCN1989] p. 367.

sage.graphs.generators.distance_regular.**UstimenkoGraph**(*m*, *q*, *immutable=False*)

Return the Ustimenko graph with parameters (m, q).

This is the distance 1 or 2 graph of the dual polar graph $C_{m-1}(q)$. The graph is distance-regular with parameters $(d, q^2, \binom{3}{1}_q - 1, \binom{m+1}{1}_q - 1)$, where $\binom{n}{k}_q$ is the q-binomial coefficient.

INPUT:

- `m`, `q` – integers; q must be a prime power and $m > 1$
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: G = graphs.UstimenkoGraph(4, 2)                                       #␣
↪needs sage.libs.gap
sage: G.is_distance_regular(True)                                           #␣
↪needs sage.libs.gap
([70, 32, None], [None, 1, 35])
```

```
>>> from sage.all import *
>>> G = graphs.UstimenkoGraph(Integer(4), Integer(2))                        ␣
↪              # needs sage.libs.gap
>>> G.is_distance_regular(True)                                            #␣
↪needs sage.libs.gap
([70, 32, None], [None, 1, 35])
```

REFERENCES:

See [BCN1989] p. 279 or [VDKT2016] p. 22.

sage.graphs.generators.distance_regular.**cocliques_HoffmannSingleton**(*immutable=False*)

Return the graph obtained from the cocliques of the Hoffmann-Singleton graph.

This is a distance-regular graph with intersection array $[15, 14, 10, 3; 1, 5, 12, 15]$.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: G = graphs.cocliques_HoffmannSingleton()
sage: G.is_distance_regular(True)
([15, 14, 10, 3, None], [None, 1, 5, 12, 15])
```

```
>>> from sage.all import *
>>> G = graphs.cocliques_HoffmannSingleton()
>>> G.is_distance_regular(True)
([15, 14, 10, 3, None], [None, 1, 5, 12, 15])
```

REFERENCES:

The construction of this graph can be found in [BCN1989] p. 392.

sage.graphs.generators.distance_regular.**distance_3_doubly_truncated_Golay_code_graph**(*immutable=False*)

Return a distance-regular graph with intersection array $[9, 8, 6, 3; 1, 1, 3, 8]$.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: # long time, needs sage.modules sage.rings.finite_rings
sage: G = graphs.distance_3_doubly_truncated_Golay_code_graph()
sage: G.is_distance_regular(True)       # long time (due to above)
([9, 8, 6, 3, None], [None, 1, 1, 3, 8])
```

```
>>> from sage.all import *
>>> # long time, needs sage.modules sage.rings.finite_rings
>>> G = graphs.distance_3_doubly_truncated_Golay_code_graph()
>>> G.is_distance_regular(True)       # long time (due to above)
([9, 8, 6, 3, None], [None, 1, 1, 3, 8])
```

ALGORITHM:

Compute the binary Golay code and truncate it twice. Compute its coset graph. Take a vertex and compute the set of vertices at distance 3 from the vertex chosen. This set constitutes the set of vertices of our distance-regular graph. Moreover we have an edge (u, v) if the coset graph contains such edge.

REFERENCES:

Description and construction of this graph are taken from [BCN1989] p. 364.

sage.graphs.generators.distance_regular.**distance_regular_graph**(*arr*, *existence=False*, *check=True*, *immutable=False*)

Return a distance-regular graph with the intersection array given.

INPUT:

- `arr` – list; intersection array of the graph
- `existence` – boolean (optional); instead of building the graph return:
  - `True` – if a graph with the given intersection array exists;
  - `False` – if there is no graph with the given intersection array;
  - `Unknown` – if Sage doesn't know if such a graph exists

- `check` – boolean (default: `True`); if `True`, then checks that the result of this function has the given intersection array
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: graphs.distance_regular_graph([21,20,16,1,2,12], existence=True)
True
sage: G = graphs.distance_regular_graph([12,11,10,7,1,2,5,12], check=False)      #␣
↪needs sage.combinat sage.modules
sage: G.is_distance_regular(True)                                                #␣
↪needs sage.combinat sage.modules
([12, 11, 10, 7, None], [None, 1, 2, 5, 12])
```

```
>>> from sage.all import *
>>> graphs.distance_regular_graph([Integer(21),Integer(20),Integer(16),Integer(1),
↪Integer(2),Integer(12)], existence=True)
True
>>> G = graphs.distance_regular_graph([Integer(12),Integer(11),Integer(10),
↪Integer(7),Integer(1),Integer(2),Integer(5),Integer(12)], check=False)      #␣
↪needs sage.combinat sage.modules
>>> G.is_distance_regular(True)                                                 #␣
↪needs sage.combinat sage.modules
([12, 11, 10, 7, None], [None, 1, 2, 5, 12])
```

REFERENCES:

See [BCN1989] and [VDKT2016].

`sage.graphs.generators.distance_regular.`**`graph_3O73`**(*immutable=False*)

Return the graph related to the group $3O(7,3)$.

This graph is distance-regular with intersection array $[117, 80, 24, 1; 1, 12, 80, 117]$.

The graph is also distance transitive with $3.O(7,3)$ as automorphism group

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: G = graphs.graph_3O73()                                  # optional - internet gap_
↪package_atlasrep
sage: G.is_distance_regular(True)                              # optional - internet gap_
↪package_atlasrep
([117, 80, 24, 1, None], [None, 1, 12, 80, 117])
```

```
>>> from sage.all import *
>>> G = graphs.graph_3O73()                                 # optional - internet gap_
↪package_atlasrep
>>> G.is_distance_regular(True)                             # optional - internet gap_
↪package_atlasrep
([117, 80, 24, 1, None], [None, 1, 12, 80, 117])
```

REFERENCES:

A description and construction of this graph can be found in [BCN1989] p. 400.

sage.graphs.generators.distance_regular.**graph_from_GQ_spread**(*s*, *t*, *immutable=False*)

Return the point graph of the generalised quadrangle with order (s, t) after removing one of its spreads.

These graphs are antipodal covers of complete graphs and, in particular, distance-regular graphs of diameter 3.

INPUT:

- `s`, `t` – integers; order of the generalised quadrangle
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: from sage.graphs.generators.distance_regular import \
....: graph_from_GQ_spread
sage: G = graph_from_GQ_spread(2, 2)                                    #␣
↪needs sage.libs.pari
sage: G.is_distance_regular(True)                                       #␣
↪needs sage.libs.pari
([4, 2, 1, None], [None, 1, 1, 4])
sage: G = graph_from_GQ_spread(4, 16)                                   #␣
↪needs sage.libs.pari
sage: G.is_distance_regular(True)                                       #␣
↪needs sage.libs.pari
([64, 60, 1, None], [None, 1, 15, 64])
```

```
>>> from sage.all import *
>>> from sage.graphs.generators.distance_regular import graph_from_GQ_spread
>>> G = graph_from_GQ_spread(Integer(2), Integer(2))                    ␣
↪             # needs sage.libs.pari
>>> G.is_distance_regular(True)                                        #␣
↪needs sage.libs.pari
([4, 2, 1, None], [None, 1, 1, 4])
>>> G = graph_from_GQ_spread(Integer(4), Integer(16))                   ␣
↪             # needs sage.libs.pari
>>> G.is_distance_regular(True)                                        #␣
↪needs sage.libs.pari
([64, 60, 1, None], [None, 1, 15, 64])
```

REFERENCES:

The graphs constructed here follow [BCN1989] pp. 385, 386.

sage.graphs.generators.distance_regular.**graph_with_classical_parameters**(*d*, *b*, *alpha_in*, *beta_in*, *gamma*, *immutable=False*)

Return the graph with the classical parameters given.

The last parameter `gamma` is meant to be an element of the enum `ClassicalParametersGraph` used to identify the family of graphs to construct. In particular this function doesn't build any sporadic graph. To build such a graph use *sage.graphs.generators.distance_regular.distance_regular_graph()*.

INPUT:

- `d`, `b`, `alpha_in`, `beta_in` – numbers; the parameters of the graph; `d` and `b` must be integers
- `gamma` – element of the enum `ClassicalParametersGraph`
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: from sage.graphs.generators.distance_regular import *
sage: graph_with_classical_parameters(3, 1, 1, 3, 1)
Johnson graph with parameters 6,3: Graph on 20 vertices
```

```
>>> from sage.all import *
>>> from sage.graphs.generators.distance_regular import *
>>> graph_with_classical_parameters(Integer(3), Integer(1), Integer(1),␣
↪Integer(3), Integer(1))
Johnson graph with parameters 6,3: Graph on 20 vertices
```

The last parameter is very important as it takes precedence. This function will not check that the other four parameters match the correct family. Use *sage.graphs.generators.distance_regular.is_classical_parameters_graph()* to check the parameters:

```
sage: from sage.graphs.generators.distance_regular import *
sage: graph_with_classical_parameters(3, 1, 1, 3, 2)
Hamming Graph with parameters 3,4: Graph on 64 vertices
sage: G = _; G.is_distance_regular(True)
([9, 6, 3, None], [None, 1, 2, 3])
sage: is_classical_parameters_graph([9, 6, 3, 1, 2, 3])                    #␣
↪needs sage.combinat
(3, 1, 0, 3, 2)
```

```
>>> from sage.all import *
>>> from sage.graphs.generators.distance_regular import *
>>> graph_with_classical_parameters(Integer(3), Integer(1), Integer(1),␣
↪Integer(3), Integer(2))
Hamming Graph with parameters 3,4: Graph on 64 vertices
>>> G = _; G.is_distance_regular(True)
([9, 6, 3, None], [None, 1, 2, 3])
>>> is_classical_parameters_graph([Integer(9), Integer(6), Integer(3), Integer(1),
↪ Integer(2), Integer(3)])                          # needs sage.combinat
(3, 1, 0, 3, 2)
```

Two families of graphs are not implemented yet:

```
sage: from sage.graphs.generators.distance_regular import *
sage: graph_with_classical_parameters(3, 16, 15, 511, 17)
Traceback (most recent call last):
...
NotImplementedError: Graph would be too big
sage: graph_with_classical_parameters(3, 16, 30, 1022, 16)
Traceback (most recent call last):
...
NotImplementedError: Graph would be too big
```

```
>>> from sage.all import *
>>> from sage.graphs.generators.distance_regular import *
>>> graph_with_classical_parameters(Integer(3), Integer(16), Integer(15),␣
↪Integer(511), Integer(17))
Traceback (most recent call last):
```

(continues on next page)

(continued from previous page)

```
...
NotImplementedError: Graph would be too big
>>> graph_with_classical_parameters(Integer(3), Integer(16), Integer(30),␣
↪Integer(1022), Integer(16))
Traceback (most recent call last):
...
NotImplementedError: Graph would be too big
```

REFERENCES:

See [BCN1989] chapter 9 for a discussion of distance-regular graphs with classical parameters. See also [VDKT2016] section 3.1.1.

sage.graphs.generators.distance_regular.**is_classical_parameters_graph**(*array*)

Return a tuple of parameters representing the array given. If such no tuple can be produced, it returns `False`.

Given an intersection array, if it represents a family of distance-regular graphs with classical parameters, then this function returns a tuple consisting of the parameters (d, b, α, β) and a fourth parameter which is the enum `CalssicalParametersGraph` indicating the family with the given intersection array. If the array does not belong to any classical parameter graph, then this function returns `False`. If the array belongs to a sporadic graph rather than a family of graphs, then the function returns `False`. This is to reduce the overlap with `sage.graphs.generators.distance_regular._sporadic_graph_database`.

Note

The array given as an input is expected to be an intersection array. If this is not the case, then some exception may be raised.

INPUT:

- `array` – list; an intersection array

OUTPUT:

`False` or a tuple `(d, b, alpha, beta, gamma)`.

EXAMPLES:

```
sage: from sage.graphs.generators.distance_regular import \
....: is_classical_parameters_graph
sage: G = graphs.HammingGraph(5, 4)
sage: G.is_distance_regular(True)
([15, 12, 9, 6, 3, None], [None, 1, 2, 3, 4, 5])
sage: is_classical_parameters_graph([15, 12, 9, 6, 3, 1, 2, 3, 4, 5])           #␣
↪needs sage.combinat
(5, 1, 0, 3, 2)
```

```
>>> from sage.all import *
>>> from sage.graphs.generators.distance_regular import is_classical_parameters_
↪graph
>>> G = graphs.HammingGraph(Integer(5), Integer(4))
>>> G.is_distance_regular(True)
([15, 12, 9, 6, 3, None], [None, 1, 2, 3, 4, 5])
>>> is_classical_parameters_graph([Integer(15), Integer(12), Integer(9),␣
```

(continues on next page)

(continued from previous page)

```
↪Integer(6), Integer(3), Integer(1), Integer(2), Integer(3), Integer(4),␣
↪Integer(5)])           # needs sage.combinat
(5, 1, 0, 3, 2)
```

REFERENCES:

See [BCN1989] chapter 9 for a discussion of distance-regular graphs with classical parameters. See [BCN1989] chapter 6.2 for a method to compute the classical parameters of a graph. See also [VDKT2016] section 3.1.1.

sage.graphs.generators.distance_regular.**is_from_GQ_spread**(*arr*)

Return a pair (s, t) if the graph obtained from a GQ of order (s, t) with a spread has the intersection array passed. We also require that such GQ can be built by Sage. If no such pair exists, then return `False`.

INPUT:

- `arr` – list; an intersection array

EXAMPLES:

```
sage: from sage.graphs.generators.distance_regular import \
....: is_from_GQ_spread, graph_from_GQ_spread
sage: is_from_GQ_spread([125, 120, 1, 1, 24, 125])                    #␣
↪needs sage.libs.pari
(5, 25)
sage: G = graph_from_GQ_spread(5, 25)                                 #␣
↪needs sage.libs.pari
sage: G.is_distance_regular(True)                                     #␣
↪needs sage.libs.pari
([125, 120, 1, None], [None, 1, 24, 125])
```

```
>>> from sage.all import *
>>> from sage.graphs.generators.distance_regular import is_from_GQ_spread, graph_
↪from_GQ_spread
>>> is_from_GQ_spread([Integer(125), Integer(120), Integer(1), Integer(1),␣
↪Integer(24), Integer(125)])                          # needs sage.libs.pari
(5, 25)
>>> G = graph_from_GQ_spread(Integer(5), Integer(25))                    ␣
↪             # needs sage.libs.pari
>>> G.is_distance_regular(True)                                        #␣
↪needs sage.libs.pari
([125, 120, 1, None], [None, 1, 24, 125])
```

REFERENCES:

The graphs we are looking for are antipodal covers of complete graphs. See [BCN1989] pp. 385, 386 for a discussion on these particular case.

sage.graphs.generators.distance_regular.**is_near_polygon**(*array*)

Return a tuple of parameters which identify the near polygon graph with the given intersection array. If such tuple doesn't exist, return `False`.

Note that `array` may be the intersection array of a near polygon, but if such graph has diameter less than 3, then this function will return `False`.

INPUT:

- `array` – list; intersection array

OUTPUT:

The tuple has the form `(id, params)` where `id` is a value of the enum $NearPolygonGraph$ which identify a family of graphs and `params` are all parameters needed to construct the final graph.

EXAMPLES:

```
sage: from sage.graphs.generators.distance_regular import (
....: is_near_polygon, near_polygon_graph)
sage: is_near_polygon([7, 6, 6, 5, 5, 4, 1, 1, 2, 2, 3, 3])                    #␣
↪needs sage.combinat
(2, 7)
sage: near_polygon_graph(2, 7)
Odd Graph with parameter 7: Graph on 1716 vertices
sage: _.is_distance_regular(True)
([7, 6, 6, 5, 5, 4, None], [None, 1, 1, 2, 2, 3, 3])
```

```
>>> from sage.all import *
>>> from sage.graphs.generators.distance_regular import (
... is_near_polygon, near_polygon_graph)
>>> is_near_polygon([Integer(7), Integer(6), Integer(6), Integer(5), Integer(5),␣
↪Integer(4), Integer(1), Integer(1), Integer(2), Integer(2), Integer(3),␣
↪Integer(3)])                    # needs sage.combinat
(2, 7)
>>> near_polygon_graph(Integer(2), Integer(7))
Odd Graph with parameter 7: Graph on 1716 vertices
>>> _.is_distance_regular(True)
([7, 6, 6, 5, 5, 4, None], [None, 1, 1, 2, 2, 3, 3])
```

REFERENCES:

See [BCN1989] pp. 198-206 for some theory about near polygons as well as a list of known examples.

`sage.graphs.generators.distance_regular.`**`is_pseudo_partition_graph`**`(arr)`

Return (m, a) if the intersection array given satisfies: $b_i = (m - i)(1 + a(m - 1 - i))$ for $0 \leq i < d$ $c_i = i(1 + a(i - 1))$ for $0 \leq i < d$ $c_d = (2d + 2 - m)d(1 + a(d - 1))$ where d is the diameter of the graph.

If such pair (m, a) doesn't exist or the diameter is less than 3, then this function returns `False`.

These graphs are called pseudo partition graphs in [BCN1989] chapter 6.3.

INPUT:

- `arr` – list; intersection array

OUTPUT:

A pair (m, a) of integers or `False` if such pair doesn't exist.

EXAMPLES:

```
sage: from sage.graphs.generators.distance_regular import *
sage: is_pseudo_partition_graph([36, 25, 16, 1, 4, 18])
(6, 1)
sage: pseudo_partition_graph(6, 1)  # long time
Folded Johnson graph with parameters 12,6: Graph on 462 vertices
sage: _.is_distance_regular(True)  # long time
([36, 25, 16, None], [None, 1, 4, 18])
```

```
>>> from sage.all import *
>>> from sage.graphs.generators.distance_regular import *
>>> is_pseudo_partition_graph([Integer(36), Integer(25), Integer(16), Integer(1),␣
↪Integer(4), Integer(18)])
(6, 1)
>>> pseudo_partition_graph(Integer(6), Integer(1))  # long time
Folded Johnson graph with parameters 12,6: Graph on 462 vertices
>>> _.is_distance_regular(True)  # long time
([36, 25, 16, None], [None, 1, 4, 18])
```

REFERENCE:

See [BCN1989] pp. 197, 198 or [VDKT2016] pp. 38, 39.

sage.graphs.generators.distance_regular.**locally_GQ42_distance_transitive_graph**(*immutable=False*)

Return the unique amply regular graph with $\mu = 6$ which is locally a generalised quadrangle.

This graph is distance-regular with intersection array $[45, 32, 12, 1; 1, 6, 32, 45]$.

This graph is also distance-transitive.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: G = graphs.locally_GQ42_distance_transitive_graph()       # optional -␣
↪internet gap_package_atlasrep
sage: G.is_distance_regular(True)                               # optional -␣
↪internet gap_package_atlasrep
([45, 32, 12, 1, None], [None, 1, 6, 32, 45])
```

```
>>> from sage.all import *
>>> G = graphs.locally_GQ42_distance_transitive_graph()       # optional -␣
↪internet gap_package_atlasrep
>>> G.is_distance_regular(True)                               # optional -␣
↪internet gap_package_atlasrep
([45, 32, 12, 1, None], [None, 1, 6, 32, 45])
```

REFERENCES:

A description of this graph can be found in [BCN1989] p.399. This construction is due to Dima Pasechnik.

sage.graphs.generators.distance_regular.**near_polygon_graph**(*family*, *params*, *immutable=False*)

Return the near polygon graph with the given parameters.

The input is expected to be the result of the function *sage.graphs.generators.distance_regular.is_near_polygon()*.

INPUT:

- `family` – integer; an element of the enum `NearPolygonGraph`
- `params` – integer or tuple; the parameters needed to construct a graph of the family `family`
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: from sage.graphs.generators.distance_regular import is_near_polygon, near_
↪polygon_graph
sage: near_polygon_graph(*is_near_polygon(                                    #␣
↪needs sage.combinat
....:     [6, 5, 5, 4, 4, 3, 3, 2, 2, 1, 1, 1, 1, 2, 2, 3, 3, 4, 4, 5, 5, 6]))
Bipartite double of Odd graph on a set of 11 elements: Graph on 924 vertices
sage: G=_; G.is_distance_regular(True)                                        #␣
↪needs sage.combinat
([6, 5, 5, 4, 4, 3, 3, 2, 2, 1, 1, None],
 [None, 1, 1, 2, 2, 3, 3, 4, 4, 5, 5, 6])
```

```
>>> from sage.all import *
>>> from sage.graphs.generators.distance_regular import is_near_polygon, near_
↪polygon_graph
>>> near_polygon_graph(*is_near_polygon(                                      #␣
↪needs sage.combinat
...     [Integer(6), Integer(5), Integer(5), Integer(4), Integer(4), Integer(3),␣
↪Integer(3), Integer(2), Integer(2), Integer(1), Integer(1), Integer(1),␣
↪Integer(1), Integer(2), Integer(2), Integer(3), Integer(3), Integer(4),␣
↪Integer(4), Integer(5), Integer(5), Integer(6)]))
Bipartite double of Odd graph on a set of 11 elements: Graph on 924 vertices
>>> G=_; G.is_distance_regular(True)                                          #␣
↪needs sage.combinat
([6, 5, 5, 4, 4, 3, 3, 2, 2, 1, 1, None],
 [None, 1, 1, 2, 2, 3, 3, 4, 4, 5, 5, 6])
```

REFERENCES:

See [BCN1989] pp. 198-206 for some theory about near polygons as well as a list of known examples.

sage.graphs.generators.distance_regular.**pseudo_partition_graph**(*m*, *a*, *immutable=False*)

Return a pseudo partition graph with the given parameters.

A graph is a pseudo partition graph if it is distance-regular with diameter at least 3 and whose intersection numbers satisfy: $b_i = (m-i)(1+a(m-1-i))$ for $0 \leq i < d$ $c_i = i(1+a(i-1))$ for $0 \leq i < d$ $c_d = (2d+2-m)d(1+a(d-1))$ where d is the diameter of the graph.

INPUT:

- `m`, `a` – integers; parameters of the graph
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: from sage.graphs.generators.distance_regular import *
sage: pseudo_partition_graph(6, 1)  # long time
Folded Johnson graph with parameters 12,6: Graph on 462 vertices
```

```
>>> from sage.all import *
>>> from sage.graphs.generators.distance_regular import *
>>> pseudo_partition_graph(Integer(6), Integer(1))  # long time
Folded Johnson graph with parameters 12,6: Graph on 462 vertices
```

Not all graphs built with this function are pseudo partition graphs as intended by *sage.graphs.generators.distance_regular.is_pseudo_partition_graph()*, since that function requires the diameter to be at least

3:

```
sage: from sage.graphs.generators.distance_regular import *
sage: pseudo_partition_graph(3, 1)
Folded Johnson graph with parameters 6,3: Graph on 10 vertices
sage: G=_; G.is_distance_regular(True)
([9, None], [None, 1])
sage: is_pseudo_partition_graph([9, 1])
False
```

```
>>> from sage.all import *
>>> from sage.graphs.generators.distance_regular import *
>>> pseudo_partition_graph(Integer(3), Integer(1))
Folded Johnson graph with parameters 6,3: Graph on 10 vertices
>>> G=_; G.is_distance_regular(True)
([9, None], [None, 1])
>>> is_pseudo_partition_graph([Integer(9), Integer(1)])
False
```

REFERENCES:

See [BCN1989] pp. 197, 198 or [VDKT2016] pp. 38, 39 for a discussion of known pseudo partition graphs.

sage.graphs.generators.distance_regular.**shortened_000_111_extended_binary_Golay_code_graph**(*immutable=Fal*

Return a distance-regular graph with intersection array $[21, 20, 16, 9, 2, 1; 1, 2, 3, 16, 20, 21]$.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: # long time, needs sage.modules sage.rings.finite_rings
sage: G = graphs.shortened_000_111_extended_binary_Golay_code_graph()
sage: G.is_distance_regular(True)
([21, 20, 16, 9, 2, 1, None], [None, 1, 2, 3, 16, 20, 21])
```

```
>>> from sage.all import *
>>> # long time, needs sage.modules sage.rings.finite_rings
>>> G = graphs.shortened_000_111_extended_binary_Golay_code_graph()
>>> G.is_distance_regular(True)
([21, 20, 16, 9, 2, 1, None], [None, 1, 2, 3, 16, 20, 21])
```

ALGORITHM:

The vertices and edges of this graph have been precomputed and pickled, so truthfully, we just unpickle them and pass them to the Graph constructor. But the algorithm used to compute those vertices and edges in the first place is,

1. Compute the extended binary Golay code.
2. Compute its subcode whose codewords start with 000 or 111.
3. Remove the first 3 entries from all the codewords from the new linear code and compute its coset graph.

This construction is tested in `generators_test.py`, where the result is compared with the result from this method.

REFERENCES:

The description and construction of this graph can be found in [BCN1989], page 365.

sage.graphs.generators.distance_regular.**shortened_00_11_binary_Golay_code_graph**(*immutable=False*)

Return a distance-regular graph with intersection array $[21, 20, 16, 6, 2, 1; 1, 2, 6, 16, 20, 21]$.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: # long time, needs sage.modules sage.rings.finite_rings
sage: G = graphs.shortened_00_11_binary_Golay_code_graph()      # 9 s
sage: G.is_distance_regular(True)
([21, 20, 16, 6, 2, 1, None], [None, 1, 2, 6, 16, 20, 21])
```

```
>>> from sage.all import *
>>> # long time, needs sage.modules sage.rings.finite_rings
>>> G = graphs.shortened_00_11_binary_Golay_code_graph()      # 9 s
>>> G.is_distance_regular(True)
([21, 20, 16, 6, 2, 1, None], [None, 1, 2, 6, 16, 20, 21])
```

ALGORITHM:

Compute the binary Golay code. Compute the subcode whose codewords start with 00 or 11. Remove the first two entries from all codewords of the newly found linear code and compute its coset graph.

REFERENCES:

Description and construction of this graph can be found in [BCN1989] p. 365.

sage.graphs.generators.distance_regular.**vanLintSchrijverGraph**(*immutable=False*)

Return the van Lint-Schrijver graph.

The graph is distance-regular with intersection array $[6, 5, 5, 4; 1, 1, 2, 6]$.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: G = graphs.vanLintSchrijverGraph()                                #␣
↪needs sage.modules
sage: G.is_distance_regular(True)                                       #␣
↪needs sage.modules
([6, 5, 5, 4, None], [None, 1, 1, 2, 6])
```

```
>>> from sage.all import *
>>> G = graphs.vanLintSchrijverGraph()                                #␣
↪needs sage.modules
>>> G.is_distance_regular(True)                                       #␣
↪needs sage.modules
([6, 5, 5, 4, None], [None, 1, 1, 2, 6])
```

REFERENCES:

For a description of this graph see [BCN1989] p. 373.

2.12 Various families of graphs

The methods defined here appear in *sage.graphs.graph_generators*.

sage.graphs.generators.families.**AztecDiamondGraph**(*n*, *immutable=False*)

Return the Aztec Diamond graph of order `n`.

See the Wikipedia article Aztec_diamond for more information.

INPUT:

- `n` – integer ≥ 0; the order Aztec Diamond graph
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: graphs.AztecDiamondGraph(2)
Aztec Diamond graph of order 2: Graph on 12 vertices

sage: [graphs.AztecDiamondGraph(i).n_vertices() for i in range(8)]
[0, 4, 12, 24, 40, 60, 84, 112]

sage: [graphs.AztecDiamondGraph(i).n_edges() for i in range(8)]
[0, 4, 16, 36, 64, 100, 144, 196]

sage: G = graphs.AztecDiamondGraph(3)
sage: sum(1 for p in G.perfect_matchings())
64
```

```
>>> from sage.all import *
>>> graphs.AztecDiamondGraph(Integer(2))
Aztec Diamond graph of order 2: Graph on 12 vertices

>>> [graphs.AztecDiamondGraph(i).n_vertices() for i in range(Integer(8))]
[0, 4, 12, 24, 40, 60, 84, 112]

>>> [graphs.AztecDiamondGraph(i).n_edges() for i in range(Integer(8))]
[0, 4, 16, 36, 64, 100, 144, 196]

>>> G = graphs.AztecDiamondGraph(Integer(3))
>>> sum(Integer(1) for p in G.perfect_matchings())
64
```

sage.graphs.generators.families.**BarbellGraph**(*n1*, *n2*, *immutable=False*)

Return a barbell graph with $2n_1 + n_2$ nodes.

The argument n_1 must be greater than or equal to 2.

A barbell graph is a basic structure that consists of a path graph of order n_2 connecting two complete graphs of order n_1 each.

INPUT:

- `n1` – integer ≥ 2; the order of each of the two complete graphs

- `n2` – nonnegative integer; the order of the path graph connecting the two complete graphs
- `immutable` – boolean (default: `False`); whether to return immutable or mutable graphs

OUTPUT:

A barbell graph of order $2 * n_1 + n_2$. A `ValueError` is returned if $n_1 < 2$ or $n_2 < 0$.

PLOTTING:

Upon construction, the position dictionary is filled to override the spring-layout algorithm. By convention, each barbell graph will be displayed with the two complete graphs in the lower-left and upper-right corners, with the path graph connecting diagonally between the two. Thus the n_1-th node will be drawn at a 45 degree angle from the horizontal right center of the first complete graph, and the $n_1 + n_2 + 1$-th node will be drawn 45 degrees below the left horizontal center of the second complete graph.

EXAMPLES:

Construct and show a barbell graph `Bar = 4, Bells = 9`:

```
sage: g = graphs.BarbellGraph(9, 4); g
Barbell graph: Graph on 22 vertices
sage: g.show()                              # long time                       #␣
↪needs sage.plot
```

```
>>> from sage.all import *
>>> g = graphs.BarbellGraph(Integer(9), Integer(4)); g
Barbell graph: Graph on 22 vertices
>>> g.show()                              # long time                       #␣
↪needs sage.plot
```

An $n_1 \geq 2, n_2 \geq 0$ barbell graph has order $2 * n_1 + n_2$. It has the complete graph on n_1 vertices as a subgraph. It also has the path graph on n_2 vertices as a subgraph.

```
sage: n1 = randint(2, 2*10^2)
sage: n2 = randint(0, 2*10^2)
sage: g = graphs.BarbellGraph(n1, n2)
sage: v = 2*n1 + n2
sage: g.order() == v
True
sage: K_n1 = graphs.CompleteGraph(n1)
sage: P_n2 = graphs.PathGraph(n2)

sage: s_K = g.subgraph_search(K_n1, induced=True)
sage: s_P = g.subgraph_search(P_n2, induced=True)
sage: K_n1.is_isomorphic(s_K)
True
sage: P_n2.is_isomorphic(s_P)
True
```

```
>>> from sage.all import *
>>> n1 = randint(Integer(2), Integer(2)*Integer(10)**Integer(2))
>>> n2 = randint(Integer(0), Integer(2)*Integer(10)**Integer(2))
>>> g = graphs.BarbellGraph(n1, n2)
>>> v = Integer(2)*n1 + n2
>>> g.order() == v
```

(continues on next page)

(continued from previous page)

```
True
>>> K_n1 = graphs.CompleteGraph(n1)
>>> P_n2 = graphs.PathGraph(n2)

>>> s_K = g.subgraph_search(K_n1, induced=True)
>>> s_P = g.subgraph_search(P_n2, induced=True)
>>> K_n1.is_isomorphic(s_K)
True
>>> P_n2.is_isomorphic(s_P)
True
```

sage.graphs.generators.families.**BiwheelGraph**(*n*, *immutable=False*)

Return a biwheel graph with $2n$ nodes

For $n \geq 4$, the biwheel graph of order $2n$ is the planar bipartite graph obtained from the cycle graph of order $2n-2$, i.e., `graphs.CycleGraph(2*n - 2)` (called the rim of the biwheel graph) by introducing two new nodes $2n-2$ and $2n-1$ (called the *hubs* of the biwheel graph), and then joining the node $2n-2$ with the odd indexed nodes up to $2n-3$ and joining the node $2n-1$ with the even indexed nodes up to $2n-4$.

PLOTTING:

Upon construction, the position dictionary is filled to override the spring-layout algorithm. By convention, each biwheel graph will be displayed with the first (0) node at the right if n is even or otherwise at an angle $\pi/(2n-2)$ with respect to the origin, with the rest of the nodes up to $2n-3$ following in a counterclockwise manner. Note that the last two nodes, i.e., the hubs $2n-2$ and $2n-1$ will be displayed at the coordinates $(-1/3, 0)$ and $(1/3, 0)$ respectively.

INPUT:

- `n` – an integer at least 4; number of nodes is $2n$
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

OUTPUT:

- `G` – a biwheel graph of order $2n$; note that a `ValueError` is returned if $n < 4$

EXAMPLES:

Construct and show a biwheel graph with 10 nodes:

```
sage: g = graphs.BiwheelGraph(5)
sage: g.show()                              # long time                              #␣
↪needs sage.plot
sage: g.is_planar()
True
sage: g.is_bipartite()
True
```

```
>>> from sage.all import *
>>> g = graphs.BiwheelGraph(Integer(5))
>>> g.show()                                # long time                              #␣
↪needs sage.plot
>>> g.is_planar()
True
>>> g.is_bipartite()
True
```

Create several biwheel graphs in a Sage graphics array:

```
sage: g = []
sage: j = []
sage: for i in range(9):
....:     k = graphs.BiwheelGraph(i+4)
....:     g.append(k)
sage: for i in range(3):
....:     n = []
....:     for m in range(3):
....:         n.append(g[3*i + m].plot(vertex_size=50 - 4*(3*i+m), vertex_
↪labels=False))
....:     j.append(n)
sage: G = graphics_array(j)
sage: G.show()                            # long time
```

```
>>> from sage.all import *
>>> g = []
>>> j = []
>>> for i in range(Integer(9)):
...     k = graphs.BiwheelGraph(i+Integer(4))
...     g.append(k)
>>> for i in range(Integer(3)):
...     n = []
...     for m in range(Integer(3)):
...         n.append(g[Integer(3)*i + m].plot(vertex_size=Integer(50) -␣
↪Integer(4)*(Integer(3)*i+m), vertex_labels=False))
...     j.append(n)
>>> G = graphics_array(j)
>>> G.show()                            # long time
```

REFERENCES:

- [LM2024]

See also

WheelGraph(), *TruncatedBiwheelGraph()*

AUTHORS:

- Janmenjaya Panda (2024-06-09)

sage.graphs.generators.families.**BubbleSortGraph**(*n*, *immutable=False*)

Return the bubble sort graph $B(n)$.

The vertices of the bubble sort graph are the set of permutations on n symbols. Two vertices are adjacent if one can be obtained from the other by swapping the labels in the i-th and $(i+1)$-th positions for $1 \leq i \leq n-1$. In total, $B(n)$ has order $n!$. Swapping two labels as described previously corresponds to multiplying on the right the permutation corresponding to the node by an elementary transposition in the `SymmetricGroup`.

The bubble sort graph is the underlying graph of the `permutahedron()`.

INPUT:

- `n` – positive integer. The number of symbols to permute

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

OUTPUT:

The bubble sort graph $B(n)$ on n symbols. If $n < 1$, a `ValueError` is returned.

EXAMPLES:

```
sage: g = graphs.BubbleSortGraph(4); g
Bubble sort: Graph on 24 vertices
sage: g.plot()                              # long time                              #␣
↪needs sage.plot
Graphics object consisting of 61 graphics primitives
```

```
>>> from sage.all import *
>>> g = graphs.BubbleSortGraph(Integer(4)); g
Bubble sort: Graph on 24 vertices
>>> g.plot()                                # long time                              #␣
↪needs sage.plot
Graphics object consisting of 61 graphics primitives
```

The bubble sort graph on $n = 1$ symbol is the trivial graph K_1:

```
sage: graphs.BubbleSortGraph(1)
Bubble sort: Graph on 1 vertex
```

```
>>> from sage.all import *
>>> graphs.BubbleSortGraph(Integer(1))
Bubble sort: Graph on 1 vertex
```

If $n \geq 1$, then the order of $B(n)$ is $n!$:

```
sage: n = randint(1, 8)
sage: g = graphs.BubbleSortGraph(n)
sage: g.order() == factorial(n)
True
```

```
>>> from sage.all import *
>>> n = randint(Integer(1), Integer(8))
>>> g = graphs.BubbleSortGraph(n)
>>> g.order() == factorial(n)
True
```

See also

- `permutahedron()`

AUTHORS:

- Michael Yurko (2009-09-01)

`sage.graphs.generators.families.`**`CaiFurerImmermanGraph`**(*G*, *twisted=False*, *immutable=None*)

Return the a Cai-Furer-Immerman graph from G, possibly a twisted one, and a partition of its nodes.

A Cai-Furer-Immerman graph from/on G is a graph created by applying the transformation described in [CFI1992] on a graph G, that is substituting every vertex v in G with a Furer gadget $F(v)$ of order d equal to the degree of the vertex, and then substituting every edge (v, u) in G with a pair of edges, one connecting the two "a" nodes of $F(v)$ and $F(u)$ and the other their two "b" nodes. The returned coloring of the vertices is made by the union of the colorings of each single Furer gadget, individualised for each vertex of G. To understand better what these "a" and "b" nodes are, see the documentation on Furer gadgets.

Furthermore, this method can apply what is described in the paper mentioned above as a "twist" on an edge, that is taking only one of the pairs of edges introduced in the new graph and swap two of their extremes, making each edge go from an "a" node to a "b" node. This is only doable if the original graph G is connected.

A CaiFurerImmerman graph on a graph with no balanced vertex separators smaller than s and its twisted version cannot be distinguished by k-WL for any k < s.

INPUT:

- `G` – an undirected graph on which to construct the Cai-Furer-Immerman graph
- `twisted` – a boolean indicating if the version to construct is a twisted one or not

OUTPUT:

- `H` – the Cai-Furer-Immerman graph on `G`
- `coloring` – list of list of vertices, representing the partition induced by the coloring on `H`
- `immutable` – boolean (default: `None`); whether to create a mutable/immutable graph. `immutable=None` (default) means that the input graph G and the returned graph will behave the same way.

EXAMPLES:

CaiFurerImmerman graph with no balanced vertex separator smaller than 2

```
sage: G = graphs.CycleGraph(4)
sage: CFI, p = graphs.CaiFurerImmermanGraph(G)
sage: sorted(CFI, key=str)
[(0, ()), (0, (0, 'a')), (0, (0, 'b')), (0, (0, 1)), (0, (1, 'a')),
 (0, (1, 'b')), (1, ()), (1, (0, 'a')), (1, (0, 'b')), (1, (0, 1)),
 (1, (1, 'a')), (1, (1, 'b')), (2, ()), (2, (0, 'a')), (2, (0, 'b')),
 (2, (0, 1)), (2, (1, 'a')), (2, (1, 'b')), (3, ()), (3, (0, 'a')),
 (3, (0, 'b')), (3, (0, 1)), (3, (1, 'a')), (3, (1, 'b'))]
sage: sorted(CFI.edge_iterator(), key=str)
[((0, ()), (0, (0, 'b')), None),
 ((0, ()), (0, (1, 'b')), None),
 ((0, (0, 'a')), (1, (0, 'a')), None),
 ((0, (0, 'b')), (1, (0, 'b')), None),
 ((0, (0, 1)), (0, (0, 'a')), None),
 ((0, (0, 1)), (0, (1, 'a')), None),
 ((0, (1, 'a')), (3, (0, 'a')), None),
 ((0, (1, 'b')), (3, (0, 'b')), None),
 ((1, ()), (1, (0, 'b')), None),
 ((1, ()), (1, (1, 'b')), None),
 ((1, (0, 1)), (1, (0, 'a')), None),
 ((1, (0, 1)), (1, (1, 'a')), None),
 ((1, (1, 'a')), (2, (0, 'a')), None),
 ((1, (1, 'b')), (2, (0, 'b')), None),
 ((2, ()), (2, (0, 'b')), None),
 ((2, ()), (2, (1, 'b')), None),
 ((2, (0, 1)), (2, (0, 'a')), None),
```

(continues on next page)

(continued from previous page)

```
 ((2, (0, 1)), (2, (1, 'a')), None),
 ((2, (1, 'a')), (3, (1, 'a')), None),
 ((2, (1, 'b')), (3, (1, 'b')), None),
 ((3, ()), (3, (0, 'b')), None),
 ((3, ()), (3, (1, 'b')), None),
 ((3, (0, 1)), (3, (0, 'a')), None),
 ((3, (0, 1)), (3, (1, 'a')), None)]
```

```
>>> from sage.all import *
>>> G = graphs.CycleGraph(Integer(4))
>>> CFI, p = graphs.CaiFurerImmermanGraph(G)
>>> sorted(CFI, key=str)
[(0, ()), (0, (0, 'a')), (0, (0, 'b')), (0, (0, 1)), (0, (1, 'a')),
 (0, (1, 'b')), (1, ()), (1, (0, 'a')), (1, (0, 'b')), (1, (0, 1)),
 (1, (1, 'a')), (1, (1, 'b')), (2, ()), (2, (0, 'a')), (2, (0, 'b')),
 (2, (0, 1)), (2, (1, 'a')), (2, (1, 'b')), (3, ()), (3, (0, 'a')),
 (3, (0, 'b')), (3, (0, 1)), (3, (1, 'a')), (3, (1, 'b'))]
>>> sorted(CFI.edge_iterator(), key=str)
[((0, ()), (0, (0, 'b')), None),
 ((0, ()), (0, (1, 'b')), None),
 ((0, (0, 'a')), (1, (0, 'a')), None),
 ((0, (0, 'b')), (1, (0, 'b')), None),
 ((0, (0, 1)), (0, (0, 'a')), None),
 ((0, (0, 1)), (0, (1, 'a')), None),
 ((0, (1, 'a')), (3, (0, 'a')), None),
 ((0, (1, 'b')), (3, (0, 'b')), None),
 ((1, ()), (1, (0, 'b')), None),
 ((1, ()), (1, (1, 'b')), None),
 ((1, (0, 1)), (1, (0, 'a')), None),
 ((1, (0, 1)), (1, (1, 'a')), None),
 ((1, (1, 'a')), (2, (0, 'a')), None),
 ((1, (1, 'b')), (2, (0, 'b')), None),
 ((2, ()), (2, (0, 'b')), None),
 ((2, ()), (2, (1, 'b')), None),
 ((2, (0, 1)), (2, (0, 'a')), None),
 ((2, (0, 1)), (2, (1, 'a')), None),
 ((2, (1, 'a')), (3, (1, 'a')), None),
 ((2, (1, 'b')), (3, (1, 'b')), None),
 ((3, ()), (3, (0, 'b')), None),
 ((3, ()), (3, (1, 'b')), None),
 ((3, (0, 1)), (3, (0, 'a')), None),
 ((3, (0, 1)), (3, (1, 'a')), None)]
```

sage.graphs.generators.families.**CirculantGraph**(*n*, *adjacency*, *immutable=False*, *name=None*)

Return a circulant graph with n nodes.

A circulant graph has the property that the vertex i is connected with the vertices $i + j$ and $i - j$ for each j in `adjacency`.

INPUT:

- `n` – number of vertices in the graph
- `adjacency` – the list of j values

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph
- `name` – string (default: `None`); used as the name of the returned graph when set

PLOTTING: Upon construction, the position dictionary is filled to override the spring-layout algorithm. By convention, each circulant graph will be displayed with the first (0) node at the top, with the rest following in a counterclockwise manner.

Filling the position dictionary in advance adds $O(n)$ to the constructor.

See also

- *`is_circulant()`* – checks whether a (di)graph is circulant, and/or returns all possible sets of parameters.

EXAMPLES: Compare plotting using the predefined layout and networkx:

```
sage: # needs networkx
sage: import networkx
sage: n = networkx.cycle_graph(23)
sage: spring23 = Graph(n)
sage: posdict23 = graphs.CirculantGraph(23,2)
sage: spring23.show()                    # long time
sage: posdict23.show()  # long time
```

```
>>> from sage.all import *
>>> # needs networkx
>>> import networkx
>>> n = networkx.cycle_graph(Integer(23))
>>> spring23 = Graph(n)
>>> posdict23 = graphs.CirculantGraph(Integer(23),Integer(2))
>>> spring23.show()                    # long time
>>> posdict23.show()  # long time
```

We next view many cycle graphs as a Sage graphics array. First we use the `CirculantGraph` constructor, which fills in the position dictionary:

```
sage: g = []
sage: j = []
sage: for i in range(9):
....:     k = graphs.CirculantGraph(i+4, i+1)
....:     g.append(k)
sage: for i in range(3):
....:     n = []
....:     for m in range(3):
....:         n.append(g[3*i + m].plot(vertex_size=50, vertex_labels=False))
....:     j.append(n)
sage: G = graphics_array(j)
sage: G.show()                           # long time
```

```
>>> from sage.all import *
>>> g = []
>>> j = []
```

(continues on next page)

(continued from previous page)

```
>>> for i in range(Integer(9)):
...     k = graphs.CirculantGraph(i+Integer(4), i+Integer(1))
...     g.append(k)
>>> for i in range(Integer(3)):
...     n = []
...     for m in range(Integer(3)):
...         n.append(g[Integer(3)*i + m].plot(vertex_size=Integer(50), vertex_
↪labels=False))
...     j.append(n)
>>> G = graphics_array(j)
>>> G.show()                            # long time
```

Compare to plotting with the spring-layout algorithm:

```
sage: # needs networkx sage.plot
sage: g = []
sage: j = []
sage: for i in range(9):
....:     spr = networkx.cycle_graph(i+3)
....:     k = Graph(spr)
....:     g.append(k)
sage: for i in range(3):
....:  n = []
....:  for m in range(3):
....:      n.append(g[3*i + m].plot(vertex_size=50, vertex_labels=False))
....:  j.append(n)
sage: G = graphics_array(j)
sage: G.show()                             # long time
```

```
>>> from sage.all import *
>>> # needs networkx sage.plot
>>> g = []
>>> j = []
>>> for i in range(Integer(9)):
...     spr = networkx.cycle_graph(i+Integer(3))
...     k = Graph(spr)
...     g.append(k)
>>> for i in range(Integer(3)):
...  n = []
...  for m in range(Integer(3)):
...      n.append(g[Integer(3)*i + m].plot(vertex_size=Integer(50), vertex_
↪labels=False))
...  j.append(n)
>>> G = graphics_array(j)
>>> G.show()                            # long time
```

Passing a 1 into adjacency should give the cycle.

```
sage: graphs.CirculantGraph(6,1) == graphs.CycleGraph(6)
True
sage: graphs.CirculantGraph(7,[1,3]).edges(sort=True, labels=false)
[(0, 1),
```

(continues on next page)

(continued from previous page)

```
(0, 3),
(0, 4),
(0, 6),
(1, 2),
(1, 4),
(1, 5),
(2, 3),
(2, 5),
(2, 6),
(3, 4),
(3, 6),
(4, 5),
(5, 6)]
```

```
>>> from sage.all import *
>>> graphs.CirculantGraph(Integer(6),Integer(1)) == graphs.CycleGraph(Integer(6))
True
>>> graphs.CirculantGraph(Integer(7),[Integer(1),Integer(3)]).edges(sort=True,␣
↪labels=false)
[(0, 1),
(0, 3),
(0, 4),
(0, 6),
(1, 2),
(1, 4),
(1, 5),
(2, 3),
(2, 5),
(2, 6),
(3, 4),
(3, 6),
(4, 5),
(5, 6)]
```

sage.graphs.generators.families.**CubeConnectedCycle**(d, *immutable=False*)

Return the cube-connected cycle of dimension d.

The cube-connected cycle of order d is the d-dimensional hypercube with each of its vertices replaced by a cycle of length d. This graph has order $d \times 2^d$. The construction is as follows: Construct vertex (x, y) for $0 \leq x < 2^d$, $0 \leq y < d$. For each vertex, (x, y), add an edge between it and $(x, (y - 1) \mod d))$, $(x, (y + 1) \mod d)$, and $(x \oplus 2^y, y)$, where $\oplus$ is the bitwise xor operator.

For $d = 1$ and 2, the cube-connected cycle graph contains self-loops or multiple edges between a pair of vertices, but for all other d, it is simple.

INPUT:

- `d` – positive integer; the dimension of the desired hypercube as well as the length of the cycle to be placed at each vertex of the d-dimensional hypercube
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

The order of the graph is $d \times 2^d$

```
sage: d = 3
sage: g = graphs.CubeConnectedCycle(d)
sage: len(g) == d*2**d
True
```

```
>>> from sage.all import *
>>> d = Integer(3)
>>> g = graphs.CubeConnectedCycle(d)
>>> len(g) == d*Integer(2)**d
True
```

The diameter of cube-connected cycles for $d > 3$ is $2d + \lfloor \frac{d}{2} \rfloor - 2$

```
sage: d = 4
sage: g = graphs.CubeConnectedCycle(d)
sage: g.diameter() == 2*d+d//2-2
True
```

```
>>> from sage.all import *
>>> d = Integer(4)
>>> g = graphs.CubeConnectedCycle(d)
>>> g.diameter() == Integer(2)*d+d//Integer(2)-Integer(2)
True
```

All vertices have degree 3 when $d > 1$

```
sage: g = graphs.CubeConnectedCycle(5)
sage: all(g.degree(v) == 3 for v in g)
True
```

```
>>> from sage.all import *
>>> g = graphs.CubeConnectedCycle(Integer(5))
>>> all(g.degree(v) == Integer(3) for v in g)
True
```

sage.graphs.generators.families.**CubeGraph**(*n*, *embedding=1*, *immutable=False*)

Return the n-cube graph, also called the hypercube in n dimensions.

The hypercube in n dimension is build upon the binary strings on n bits, two of them being adjacent if they differ in exactly one bit. Hence, the distance between two vertices in the hypercube is the Hamming distance.

INPUT:

- `n` – integer; the dimension of the cube graph
- `embedding` – integer (default: 1); two embeddings of the n-cube are available:
  - `1` – the n-cube is projected inside a regular $2n$-gonal polygon by a skew orthogonal projection. See the Wikipedia article Hypercube for more details.
  - `2` – orthogonal projection of the n-cube. This orientation shows columns of independent vertices such that the neighbors of a vertex are located in the columns on the left and on the right. The number of vertices in each column represents rows in Pascal's triangle. See for instance the Wikipedia article 10-cube for more details.

  - `3` – oblique projection of the n-cube. Oblique projection involves aligning one face parallel to the viewer and projecting at a specified angle, maintaining equal size for edges parallel to one axis while applying fixed foreshortening to others. This method simplifies the representation of a four-dimensional hypercube onto a two-dimensional plane, offering a geometrically consistent visualization.
  - `None` or `0` – no embedding is provided

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

The distance between 0100110 and 1011010 is 5, as expected:

```
sage: g = graphs.CubeGraph(7)
sage: g.distance('0100110','1011010')
5
```

```
>>> from sage.all import *
>>> g = graphs.CubeGraph(Integer(7))
>>> g.distance('0100110','1011010')
5
```

Plot several n-cubes in a Sage Graphics Array:

```
sage: g = []
sage: j = []
sage: for i in range(6):
....:  k = graphs.CubeGraph(i+1)
....:  g.append(k)
...
sage: for i in range(2):
....:  n = []
....:  for m in range(3):
....:      n.append(g[3*i + m].plot(vertex_size=50, vertex_labels=False))
....:  j.append(n)
...
sage: G = graphics_array(j)
sage: G.show(figsize=[6,4])              # long time
```

```
>>> from sage.all import *
>>> g = []
>>> j = []
>>> for i in range(Integer(6)):
...  k = graphs.CubeGraph(i+Integer(1))
...  g.append(k)
...
>>> for i in range(Integer(2)):
...  n = []
...  for m in range(Integer(3)):
...      n.append(g[Integer(3)*i + m].plot(vertex_size=Integer(50), vertex_
↪labels=False))
...  j.append(n)
...
>>> G = graphics_array(j)
>>> G.show(figsize=[Integer(6),Integer(4)])             # long time
```

Use the plot options to display larger n-cubes:

```
sage: g = graphs.CubeGraph(9, embedding=1)
sage: g.show(figsize=[12,12],vertex_labels=False, vertex_size=20)       # long␣
↪time, needs sage.plot
sage: g = graphs.CubeGraph(9, embedding=2)
sage: g.show(figsize=[12,12],vertex_labels=False, vertex_size=20)       # long␣
↪time, needs sage.plot
sage: g = graphs.CubeGraph(9, embedding=3)
sage: g.show(figsize=[12,12],vertex_labels=False, vertex_size=20)       # long␣
↪time, needs sage.plot
```

```
>>> from sage.all import *
>>> g = graphs.CubeGraph(Integer(9), embedding=Integer(1))
>>> g.show(figsize=[Integer(12),Integer(12)],vertex_labels=False, vertex_
↪size=Integer(20))       # long time, needs sage.plot
>>> g = graphs.CubeGraph(Integer(9), embedding=Integer(2))
>>> g.show(figsize=[Integer(12),Integer(12)],vertex_labels=False, vertex_
↪size=Integer(20))       # long time, needs sage.plot
>>> g = graphs.CubeGraph(Integer(9), embedding=Integer(3))
>>> g.show(figsize=[Integer(12),Integer(12)],vertex_labels=False, vertex_
↪size=Integer(20))       # long time, needs sage.plot
```

AUTHORS:

- Robert Miller
- David Coudert

`sage.graphs.generators.families.`**`DipoleGraph`**(*n*, *immutable=False*)

Return a dipole graph with n edges.

A dipole graph is a multigraph consisting of 2 vertices connected with n parallel edges.

INPUT:

- `n` – integer ≥ 0; the number of parallel edges connecting the 2 vertices
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

Construct and show a dipole graph with 13 edges:

```
sage: g = graphs.DipoleGraph(13); g
Dipole graph: Multi-graph on 2 vertices
sage: g.show()                          # long time                          #␣
↪needs sage.plot
```

```
>>> from sage.all import *
>>> g = graphs.DipoleGraph(Integer(13)); g
Dipole graph: Multi-graph on 2 vertices
>>> g.show()                          # long time                          #␣
↪needs sage.plot
```

`sage.graphs.generators.families.`**`DorogovtsevGoltsevMendesGraph`**(*n*, *immutable=False*)

Construct the n-th generation of the Dorogovtsev-Goltsev-Mendes graph.

See [DGM2002] for more details.

INPUT:

- `n` – nonnegative integer; index of the generation
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: G = graphs.DorogovtsevGoltsevMendesGraph(8)                              #␣
↪needs networkx
sage: G.size()                                                                 #␣
↪needs networkx
6561
```

```
>>> from sage.all import *
>>> G = graphs.DorogovtsevGoltsevMendesGraph(Integer(8))                        ␣
↪     # needs networkx
>>> G.size()                                                                  #␣
↪needs networkx
6561
```

sage.graphs.generators.families.**DoubleGeneralizedPetersenGraph**(*n*, *k*, *immutable=False*)

Return a double generalized Petersen graph with $4n$ nodes.

The double generalized Petersen graphs is a family of graphs proposed in [ZF2012] as a variant of generalized Petersen graphs. The variables n, k are integers such that $n > 2$ and $0 < k \leq \lfloor (n-1)/2 \rfloor$.

INPUT:

- `n` – the number of nodes is $4 * n$
- `k` – integer such that $0 < k \leq \lfloor (n-1)/2 \rfloor$ determining how vertices on second and third inner rims are connected
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

PLOTTING: Upon construction, the position dictionary is filled to override the spring-layout algorithm. By convention, the double generalized Petersen graphs are displayed as 4 concentric cycles, with the first n nodes drawn on the outer circle. The first (0) node is drawn at the top of the outer-circle, moving counterclockwise after that. The second circle is drawn with the (n)th node at the top, then counterclockwise as well. The tird cycle is drawn with the (2n)th node at the top, then counterclockwise. And the fourth cycle is drawn with the (3n)th node at the top, then again counterclockwise.

EXAMPLES:

When n is even the resulting graph will be isomorphic to a double generalized Petersen graph with $k' = n/2 - k$:

```
sage: g = graphs.DoubleGeneralizedPetersenGraph(10, 2)
sage: g2 = graphs.DoubleGeneralizedPetersenGraph(10, 3)
sage: g.is_isomorphic(g2)
True
```

```
>>> from sage.all import *
>>> g = graphs.DoubleGeneralizedPetersenGraph(Integer(10), Integer(2))
>>> g2 = graphs.DoubleGeneralizedPetersenGraph(Integer(10), Integer(3))
>>> g.is_isomorphic(g2)
True
```

sage.graphs.generators.families.**EgawaGraph**(*p*, *s*, *immutable=False*)

Return the Egawa graph with parameters p, s.

Egawa graphs are a peculiar family of graphs devised by Yoshimi Egawa in [Ega1981] . The Shrikhande graph is a special case of this family of graphs, with parameters $(1, 0)$. All the graphs in this family are not recognizable by 1-WL (Weisfeiler Lehamn algorithm of the first order) and 2-WL, that is their orbits are not correctly returned by k-WL for k lower than 3.

Furthermore, all the graphs in this family are distance-regular, but they are not distance-transitive if $p \neq 0$.

The Egawa graph with parameters $(0, s)$ is isomorphic to the Hamming graph with parameters $(s, 4)$, when the underlying set of the Hamming graph is $[0, 1, 2, 3]$

INPUT:

- `p` – power to which the graph named Y in the reference provided above will be raised
- `s` – power to which the graph named X in the reference provided above will be raised
- `immutable` – boolean (default: `False`); whether to return immutable or mutable graphs

OUTPUT:

- `G` – the Egawa graph with parameters (p,s)

EXAMPLES:

Every Egawa graph is distance regular.

```
sage: g = graphs.EgawaGraph(1, 2)
sage: g.is_distance_regular()
True
```

```
>>> from sage.all import *
>>> g = graphs.EgawaGraph(Integer(1), Integer(2))
>>> g.is_distance_regular()
True
```

An Egawa graph with parameters (0,s) is isomorphic to the Hamming graph with parameters (s, 4).

```
sage: g = graphs.EgawaGraph(0, 4)
sage: g.is_isomorphic(graphs.HammingGraph(4,4))
True
```

```
>>> from sage.all import *
>>> g = graphs.EgawaGraph(Integer(0), Integer(4))
>>> g.is_isomorphic(graphs.HammingGraph(Integer(4),Integer(4)))
True
```

sage.graphs.generators.families.**FoldedCubeGraph**(*n*, *immutable=False*)

Return the folded cube graph of order 2^{n-1}.

The folded cube graph on 2^{n-1} vertices can be obtained from a cube graph on 2^n vertices by merging together opposed vertices. Alternatively, it can be obtained from a cube graph on 2^{n-1} vertices by adding an edge between opposed vertices. This second construction is the one produced by this method.

See the Wikipedia article Folded_cube_graph for more information.

INPUT:

- `n` – integer ≥ 2; the dimension of the folded cube

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

The folded cube graph of order five is the Clebsch graph:

```
sage: fc = graphs.FoldedCubeGraph(5)
sage: clebsch = graphs.ClebschGraph()
sage: fc.is_isomorphic(clebsch)
True
```

```
>>> from sage.all import *
>>> fc = graphs.FoldedCubeGraph(Integer(5))
>>> clebsch = graphs.ClebschGraph()
>>> fc.is_isomorphic(clebsch)
True
```

`sage.graphs.generators.families.`**`FriendshipGraph`**(*n*, *immutable=False*)

Return the friendship graph F_n.

The friendship graph is also known as the Dutch windmill graph. Let C_3 be the cycle graph on 3 vertices. Then F_n is constructed by joining $n \geq 1$ copies of C_3 at a common vertex. If $n = 1$, then F_1 is isomorphic to C_3 (the triangle graph). If $n = 2$, then F_2 is the butterfly graph, otherwise known as the bowtie graph. For more information, see the Wikipedia article Friendship_graph.

INPUT:

- `n` – positive integer; the number of copies of C_3 to use in constructing F_n
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

OUTPUT:

- The friendship graph F_n obtained from n copies of the cycle graph C_3.

See also

- *ButterflyGraph()*

EXAMPLES:

The first few friendship graphs.

```
sage: A = []; B = []
sage: for i in range(9):
....:     g = graphs.FriendshipGraph(i + 1)
....:     A.append(g)
sage: for i in range(3):
....:     n = []
....:     for j in range(3):
....:         n.append(A[3*i + j].plot(vertex_size=20, vertex_labels=False))
....:     B.append(n)
sage: G = graphics_array(B)
sage: G.show()                          # long time
```

```
>>> from sage.all import *
>>> A = []; B = []
>>> for i in range(Integer(9)):
...     g = graphs.FriendshipGraph(i + Integer(1))
...     A.append(g)
>>> for i in range(Integer(3)):
...     n = []
...     for j in range(Integer(3)):
...         n.append(A[Integer(3)*i + j].plot(vertex_size=Integer(20), vertex_
↪labels=False))
...     B.append(n)
>>> G = graphics_array(B)
>>> G.show()                            # long time
```

For $n = 1$, the friendship graph F_1 is isomorphic to the cycle graph C_3, whose visual representation is a triangle.

```
sage: G = graphs.FriendshipGraph(1); G
Friendship graph: Graph on 3 vertices
sage: G.show()                           # long time                              #␣
↪needs sage.plot
sage: G.is_isomorphic(graphs.CycleGraph(3))
True
```

```
>>> from sage.all import *
>>> G = graphs.FriendshipGraph(Integer(1)); G
Friendship graph: Graph on 3 vertices
>>> G.show()                            # long time                              #␣
↪needs sage.plot
>>> G.is_isomorphic(graphs.CycleGraph(Integer(3)))
True
```

For $n = 2$, the friendship graph F_2 is isomorphic to the butterfly graph, otherwise known as the bowtie graph.

```
sage: G = graphs.FriendshipGraph(2); G
Friendship graph: Graph on 5 vertices
sage: G.is_isomorphic(graphs.ButterflyGraph())
True
```

```
>>> from sage.all import *
>>> G = graphs.FriendshipGraph(Integer(2)); G
Friendship graph: Graph on 5 vertices
>>> G.is_isomorphic(graphs.ButterflyGraph())
True
```

If $n \geq 2$, then the friendship graph F_n has $2n + 1$ vertices and $3n$ edges. It has radius 1, diameter 2, girth 3, and chromatic number 3. Furthermore, F_n is planar and Eulerian.

```
sage: n = randint(2, 10^3)
sage: G = graphs.FriendshipGraph(n)
sage: G.order() == 2*n + 1
True
sage: G.size() == 3*n
True
```

(continues on next page)

(continued from previous page)

```
sage: G.radius()
1
sage: G.diameter()
2
sage: G.girth()
3
sage: G.chromatic_number()
3
sage: G.is_planar()
True
sage: G.is_eulerian()
True
```

```
>>> from sage.all import *
>>> n = randint(Integer(2), Integer(10)**Integer(3))
>>> G = graphs.FriendshipGraph(n)
>>> G.order() == Integer(2)*n + Integer(1)
True
>>> G.size() == Integer(3)*n
True
>>> G.radius()
1
>>> G.diameter()
2
>>> G.girth()
3
>>> G.chromatic_number()
3
>>> G.is_planar()
True
>>> G.is_eulerian()
True
```

sage.graphs.generators.families.**FurerGadget**(*k*, *prefix=None*, *immutable=False*)

Return a Furer gadget of order `k` and their coloring.

Construct the Furer gadget described in [CFI1992], a graph composed by a middle layer of $2^(k-1)$ nodes and two sets of nodes $(a_0, ..., a_{k-1})$ and $(b_0, ..., b_{k-1})$. Each node in the middle is connected to either a_i or b_i, for each i in [0,k[. To read about the complete construction, see [CFI1992]. The returned coloring colors the middle section with one color, and then each pair (a_i, b_i) with another color. Since this method is mainly used to create Furer gadgets for the Cai-Furer-Immerman construction, returning gadgets that don't always have the same vertex labels is important, that's why there is a parameter to manually set a prefix to be appended to each vertex label.

INPUT:

- `k` – the order of the returned Furer gadget, greater than 0
- `prefix` – prefix of to be appended to each vertex label, so as to individualise the returned Furer gadget; must be comparable for equality and hashable
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

OUTPUT:

- `G` – the Furer gadget of order `k`

- `coloring` – list of list of vertices, representing the partition induced by the coloring of `G`'s vertices

EXAMPLES:

Furer gadget of order 3, without any prefix.

```
sage: G, p = graphs.FurerGadget(3)
sage: sorted(G, key=str)
[(), (0, 'a'), (0, 'b'), (0, 1), (0, 2),
 (1, 'a'), (1, 'b'), (1, 2), (2, 'a'), (2, 'b')]
sage: sorted(G.edge_iterator(), key=str)
[((), (0, 'b'), None), ((), (1, 'b'), None),
 ((), (2, 'b'), None), ((0, 'b'), (1, 2), None),
 ((0, 1), (0, 'a'), None), ((0, 1), (1, 'a'), None),
 ((0, 1), (2, 'b'), None), ((0, 2), (0, 'a'), None),
 ((0, 2), (1, 'b'), None), ((0, 2), (2, 'a'), None),
 ((1, 2), (1, 'a'), None), ((1, 2), (2, 'a'), None)]
```

```
>>> from sage.all import *
>>> G, p = graphs.FurerGadget(Integer(3))
>>> sorted(G, key=str)
[(), (0, 'a'), (0, 'b'), (0, 1), (0, 2),
 (1, 'a'), (1, 'b'), (1, 2), (2, 'a'), (2, 'b')]
>>> sorted(G.edge_iterator(), key=str)
[((), (0, 'b'), None), ((), (1, 'b'), None),
 ((), (2, 'b'), None), ((0, 'b'), (1, 2), None),
 ((0, 1), (0, 'a'), None), ((0, 1), (1, 'a'), None),
 ((0, 1), (2, 'b'), None), ((0, 2), (0, 'a'), None),
 ((0, 2), (1, 'b'), None), ((0, 2), (2, 'a'), None),
 ((1, 2), (1, 'a'), None), ((1, 2), (2, 'a'), None)]
```

Furer gadget of order 3, with a prefix.

```
sage: G, p = graphs.FurerGadget(3, 'Prefix')
sage: sorted(G, key=str)
[('Prefix', ()), ('Prefix', (0, 'a')), ('Prefix', (0, 'b')),
 ('Prefix', (0, 1)), ('Prefix', (0, 2)), ('Prefix', (1, 'a')),
 ('Prefix', (1, 'b')), ('Prefix', (1, 2)), ('Prefix', (2, 'a')),
 ('Prefix', (2, 'b'))]
sage: sorted(G.edge_iterator(), key=str)
[(('Prefix', ()), ('Prefix', (0, 'b')), None),
 (('Prefix', ()), ('Prefix', (1, 'b')), None),
 (('Prefix', ()), ('Prefix', (2, 'b')), None),
 (('Prefix', (0, 'b')), ('Prefix', (1, 2)), None),
 (('Prefix', (0, 1)), ('Prefix', (0, 'a')), None),
 (('Prefix', (0, 1)), ('Prefix', (1, 'a')), None),
 (('Prefix', (0, 1)), ('Prefix', (2, 'b')), None),
 (('Prefix', (0, 2)), ('Prefix', (0, 'a')), None),
 (('Prefix', (0, 2)), ('Prefix', (1, 'b')), None),
 (('Prefix', (0, 2)), ('Prefix', (2, 'a')), None),
 (('Prefix', (1, 2)), ('Prefix', (1, 'a')), None),
 (('Prefix', (1, 2)), ('Prefix', (2, 'a')), None)]
```

```
>>> from sage.all import *
```

(continues on next page)

(continued from previous page)

```
>>> G, p = graphs.FurerGadget(Integer(3), 'Prefix')
>>> sorted(G, key=str)
[('Prefix', ()), ('Prefix', (0, 'a')), ('Prefix', (0, 'b')),
 ('Prefix', (0, 1)), ('Prefix', (0, 2)), ('Prefix', (1, 'a')),
 ('Prefix', (1, 'b')), ('Prefix', (1, 2)), ('Prefix', (2, 'a')),
 ('Prefix', (2, 'b'))]
>>> sorted(G.edge_iterator(), key=str)
[(('Prefix', ()), ('Prefix', (0, 'b')), None),
 (('Prefix', ()), ('Prefix', (1, 'b')), None),
 (('Prefix', ()), ('Prefix', (2, 'b')), None),
 (('Prefix', (0, 'b')), ('Prefix', (1, 2)), None),
 (('Prefix', (0, 1)), ('Prefix', (0, 'a')), None),
 (('Prefix', (0, 1)), ('Prefix', (1, 'a')), None),
 (('Prefix', (0, 1)), ('Prefix', (2, 'b')), None),
 (('Prefix', (0, 2)), ('Prefix', (0, 'a')), None),
 (('Prefix', (0, 2)), ('Prefix', (1, 'b')), None),
 (('Prefix', (0, 2)), ('Prefix', (2, 'a')), None),
 (('Prefix', (1, 2)), ('Prefix', (1, 'a')), None),
 (('Prefix', (1, 2)), ('Prefix', (2, 'a')), None)]
```

sage.graphs.generators.families.**FuzzyBallGraph**(*partition*, *q*, *immutable=False*)

Construct a Fuzzy Ball graph with the integer partition `partition` and `q` extra vertices.

Let q be an integer and let $m_1, m_2, ..., m_k$ be a set of positive integers. Let $n = q + m_1 + ... + m_k$. The Fuzzy Ball graph with partition $m_1, m_2, ..., m_k$ and q extra vertices is the graph constructed from the graph $G = K_n$ by attaching, for each $i = 1, 2, ..., k$, a new vertex a_i to m_i distinct vertices of G.

For given positive integers k and m and nonnegative integer q, the set of graphs `FuzzyBallGraph(p, q)` for all partitions p of m with k parts are cospectral with respect to the normalized Laplacian.

INPUT:

- `partition` – non empty list of positive integers
- `q` – nonnegative integer; number of extra vertices
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: F = graphs.FuzzyBallGraph([3,1],2); F
Fuzzy-Ball([3, 1], 2): Graph on 8 vertices
sage: F.adjacency_matrix(vertices=list(F))                                  #␣
↪needs sage.modules
[0 0 1 1 1 0 0 0]
[0 0 0 0 0 1 0 0]
[1 0 0 1 1 1 1 1]
[1 0 1 0 1 1 1 1]
[1 0 1 1 0 1 1 1]
[0 1 1 1 1 0 1 1]
[0 0 1 1 1 1 0 1]
[0 0 1 1 1 1 1 0]
```

```
>>> from sage.all import *
>>> F = graphs.FuzzyBallGraph([Integer(3),Integer(1)],Integer(2)); F
```

(continues on next page)

(continued from previous page)

```
Fuzzy-Ball([3, 1], 2): Graph on 8 vertices
>>> F.adjacency_matrix(vertices=list(F))                                    #␣
↪needs sage.modules
[0 0 1 1 1 0 0 0]
[0 0 0 0 0 1 0 0]
[1 0 0 1 1 1 1 1]
[1 0 1 0 1 1 1 1]
[1 0 1 1 0 1 1 1]
[0 1 1 1 1 0 1 1]
[0 0 1 1 1 1 0 1]
[0 0 1 1 1 1 1 0]
```

Pick positive integers m and k and a nonnegative integer q. All the FuzzyBallGraphs constructed from partitions of m with k parts should be cospectral with respect to the normalized Laplacian:

```
sage: m = 4; q = 2; k = 2
sage: g_list = [graphs.FuzzyBallGraph(p,q)                                  #␣
↪needs sage.combinat sage.modules
....:           for p in Partitions(m, length=k)]
sage: set(g.laplacian_matrix(normalized=True,    # long time (7s on sage.math,␣
↪2011), needs sage.combinat sage.modules
....:                        vertices=list(g)).charpoly()
....:     for g in g_list)
{x^8 - 8*x^7 + 4079/150*x^6 - 68689/1350*x^5 + 610783/10800*x^4
  - 120877/3240*x^3 + 1351/100*x^2 - 931/450*x}
```

```
>>> from sage.all import *
>>> m = Integer(4); q = Integer(2); k = Integer(2)
>>> g_list = [graphs.FuzzyBallGraph(p,q)                                    #␣
↪needs sage.combinat sage.modules
...           for p in Partitions(m, length=k)]
>>> set(g.laplacian_matrix(normalized=True,    # long time (7s on sage.math, 2011),
↪ needs sage.combinat sage.modules
...                         vertices=list(g)).charpoly()
...     for g in g_list)
{x^8 - 8*x^7 + 4079/150*x^6 - 68689/1350*x^5 + 610783/10800*x^4
  - 120877/3240*x^3 + 1351/100*x^2 - 931/450*x}
```

sage.graphs.generators.families.**GeneralizedPetersenGraph**(*n*, *k*, *immutable=False*, *name=None*)

Return a generalized Petersen graph with $2n$ nodes. The variables n, k are integers such that $n > 2$ and $0 < k \leq \lfloor (n-1)/2 \rfloor$

For $k = 1$ the result is a graph isomorphic to the circular ladder graph with the same n. The regular Petersen Graph has $n = 5$ and $k = 2$. Other named graphs that can be described using this notation include the Desargues graph and the Möbius-Kantor graph.

INPUT:

- `n` – the number of nodes is $2 * n$
- `k` – integer ($0 < k \leq \lfloor (n-1)/2 \rfloor$); decides how inner vertices are connected
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph
- `name` – string (default: `None`); used as the name of the returned graph when set

PLOTTING: Upon construction, the position dictionary is filled to override the spring-layout algorithm. By convention, the generalized Petersen graphs are displayed as an inner and outer cycle pair, with the first n nodes drawn on the outer circle. The first (0) node is drawn at the top of the outer-circle, moving counterclockwise after that. The inner circle is drawn with the (n)th node at the top, then counterclockwise as well.

EXAMPLES: For $k = 1$ the resulting graph will be isomorphic to a circular ladder graph.

```
sage: g = graphs.GeneralizedPetersenGraph(13,1)
sage: g2 = graphs.CircularLadderGraph(13)
sage: g.is_isomorphic(g2)
True
```

```
>>> from sage.all import *
>>> g = graphs.GeneralizedPetersenGraph(Integer(13),Integer(1))
>>> g2 = graphs.CircularLadderGraph(Integer(13))
>>> g.is_isomorphic(g2)
True
```

The Desargues graph:

```
sage: g = graphs.GeneralizedPetersenGraph(10,3)
sage: g.girth()
6
sage: g.is_bipartite()
True
```

```
>>> from sage.all import *
>>> g = graphs.GeneralizedPetersenGraph(Integer(10),Integer(3))
>>> g.girth()
6
>>> g.is_bipartite()
True
```

AUTHORS:

- Anders Jonsson (2009-10-15)

sage.graphs.generators.families.**GeneralizedSierpinskiGraph**(*G*, *k*, *stretch=None*, *immutable=False*)

Return the generalized Sierpinski graph of G of dimension k.

Generalized Sierpinski graphs have been introduced in [GKP2011] to generalize the notion of Sierpinski graphs [KM1997].

Given a graph $G = (V, E)$ of order n and a parameter k, the generalized Sierpinski graph of G of dimension k, denoted by $S(G, k)$, can be constructed recursively from G as follows. $S(G, 1)$ is isomorphic to G. To construct $S(G, k)$ for $k > 1$, copy n times $S(G, k - 1)$, once per vertex $u \in V$, and add u at the beginning of the labels of each vertex in the copy of $S(G, k - 1)$ corresponding to vertex u. Then for any edge $\{u, v\} \in E$, add an edge between vertex $(u, v, \ldots, v)$ and vertex $(v, u, \ldots, u)$.

INPUT:

- `G` – a sage Graph
- `k` – integer; the dimension
- `stretch` – integer (default: `None`); stretching factor used to determine the positions of the vertices of the output graph. By default (`None`), this value is set to twice the maximum Euclidean distance between the

vertices of G. This parameter is used only when the vertices of G have positions.

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

See also

- *SierpinskiGasketGraph()*
- *HanoiTowerGraph()*

EXAMPLES:

The generalized Sierpinski graph of dimension 1 of any graph G is isomorphic to G:

```
sage: G = graphs.RandomGNP(10, .5)
sage: S = graphs.GeneralizedSierpinskiGraph(G, 1)
sage: S.is_isomorphic(G)
True
```

```
>>> from sage.all import *
>>> G = graphs.RandomGNP(Integer(10), RealNumber('.5'))
>>> S = graphs.GeneralizedSierpinskiGraph(G, Integer(1))
>>> S.is_isomorphic(G)
True
```

When G is a clique of order 3, the generalized Sierpinski graphs of G are isomorphic to Hanoi Tower graphs:

```
sage: k = randint(1, 5)
sage: S = graphs.GeneralizedSierpinskiGraph(graphs.CompleteGraph(3), k)          #␣
↪needs sage.modules
sage: H = graphs.HanoiTowerGraph(3, k)
sage: S.is_isomorphic(H)                                                         #␣
↪needs sage.modules
True
```

```
>>> from sage.all import *
>>> k = randint(Integer(1), Integer(5))
>>> S = graphs.GeneralizedSierpinskiGraph(graphs.CompleteGraph(Integer(3)), k)    ␣
↪     # needs sage.modules
>>> H = graphs.HanoiTowerGraph(Integer(3), k)
>>> S.is_isomorphic(H)                                                           #␣
↪needs sage.modules
True
```

The generalized Sierpinski graph of dimension k of any graph G with n vertices and m edges has n^k vertices and $m\sum_{i=0}^{k-1} n^i$ edges:

```
sage: n = randint(2, 6)
sage: k = randint(1, 5)
sage: G = graphs.RandomGNP(n, .5)
sage: m = G.size()
sage: S = graphs.GeneralizedSierpinskiGraph(G, k)
sage: S.order() == n**k
```

(continues on next page)

(continued from previous page)

```
True
sage: S.size() == m*sum([n**i for i in range(k)])
True
sage: G = graphs.CompleteGraph(n)
sage: S = graphs.GeneralizedSierpinskiGraph(G, k)
sage: S.order() == n**k
True
sage: S.size() == (n*(n - 1)/2)*sum([n**i for i in range(k)])
True
```

```
>>> from sage.all import *
>>> n = randint(Integer(2), Integer(6))
>>> k = randint(Integer(1), Integer(5))
>>> G = graphs.RandomGNP(n, RealNumber('.5'))
>>> m = G.size()
>>> S = graphs.GeneralizedSierpinskiGraph(G, k)
>>> S.order() == n**k
True
>>> S.size() == m*sum([n**i for i in range(k)])
True
>>> G = graphs.CompleteGraph(n)
>>> S = graphs.GeneralizedSierpinskiGraph(G, k)
>>> S.order() == n**k
True
>>> S.size() == (n*(n - Integer(1))/Integer(2))*sum([n**i for i in range(k)])
True
```

The positions of the vertices of the output graph are determined from the positions of the vertices of G, if any:

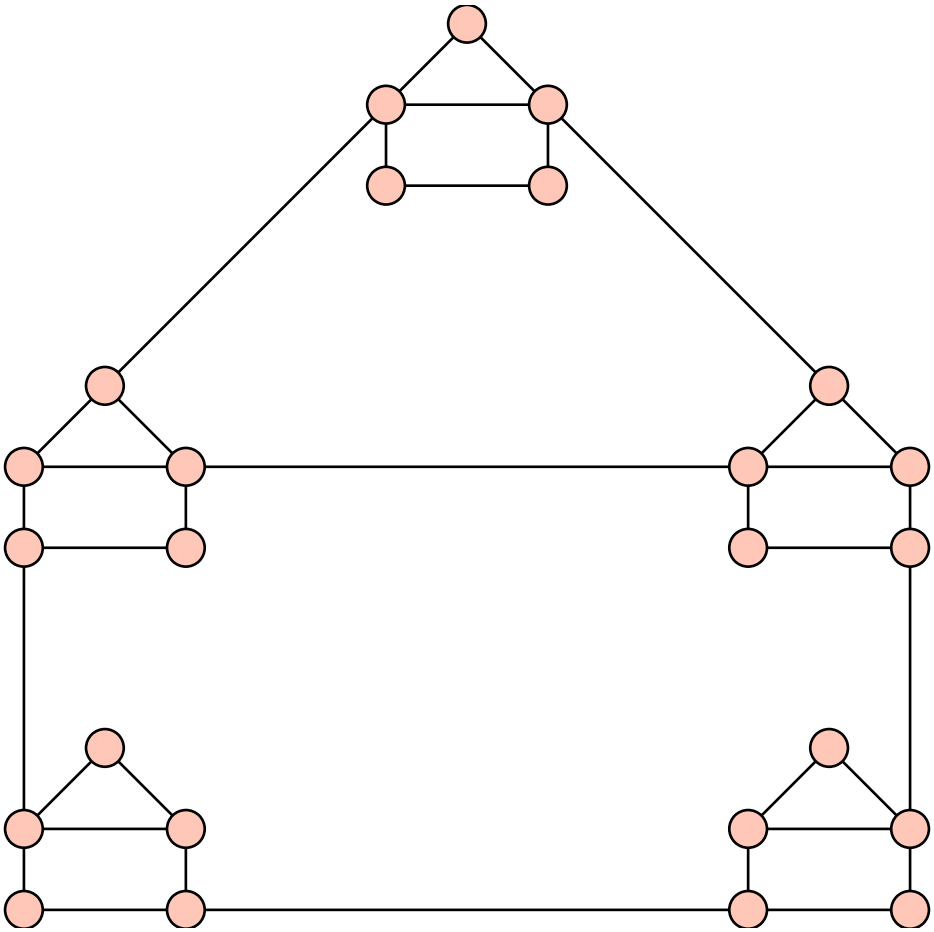
```
sage: G = graphs.HouseGraph()
sage: G.get_pos() is not None
True
sage: H = graphs.GeneralizedSierpinskiGraph(G, 2)                    #␣
↪needs sage.symbolic
sage: H.get_pos() is not None                                        #␣
↪needs sage.symbolic
True
sage: G = Graph([(0, 1)])
sage: G.get_pos() is not None
False
sage: H = graphs.GeneralizedSierpinskiGraph(G, 2)                    #␣
↪needs sage.symbolic
sage: H.get_pos() is not None                                        #␣
↪needs sage.symbolic
False
```

```
>>> from sage.all import *
>>> G = graphs.HouseGraph()
>>> G.get_pos() is not None
True
>>> H = graphs.GeneralizedSierpinskiGraph(G, Integer(2))                ␣
↪     # needs sage.symbolic
```

(continues on next page)

(continued from previous page)

```
>>> H.get_pos() is not None                                                    #␣
↪needs sage.symbolic
True
>>> G = Graph([(Integer(0), Integer(1))])
>>> G.get_pos() is not None
False
>>> H = graphs.GeneralizedSierpinskiGraph(G, Integer(2))                       ␣
↪     # needs sage.symbolic
>>> H.get_pos() is not None                                                    #␣
↪needs sage.symbolic
False
```



sage.graphs.generators.families.**GoethalsSeidelGraph**(*k*, *r*, *immutable=False*)

Return the graph Goethals-Seidel(k, r).

The graph Goethals-Seidel(k, r) comes from a construction presented in Theorem 2.4 of [GS1970]. It relies on a `(v,k)-BIBD` with r blocks and a `hadamard_matrix()` of order $r + 1$. The result is a *sage.graphs.strongly_regular_db.strongly_regular_graph()* on $v(r + 1)$ vertices with degree $k = (n + r - 1)/2$.

It appears under this name in Andries Brouwer's database of strongly regular graphs.

INPUT:

- `k`, `r` – integers

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

See also

- *is_goethals_seidel()*

EXAMPLES:

```
sage: graphs.GoethalsSeidelGraph(3,3)                                          #␣
↪needs sage.combinat sage.modules
Graph on 28 vertices
sage: graphs.GoethalsSeidelGraph(3,3).is_strongly_regular(parameters=True)     #␣
↪needs sage.combinat sage.modules
(28, 15, 6, 10)
```

```
>>> from sage.all import *
>>> graphs.GoethalsSeidelGraph(Integer(3),Integer(3))                          ␣
↪              # needs sage.combinat sage.modules
Graph on 28 vertices
>>> graphs.GoethalsSeidelGraph(Integer(3),Integer(3)).is_strongly_
↪regular(parameters=True)      # needs sage.combinat sage.modules
(28, 15, 6, 10)
```

sage.graphs.generators.families.**HammingGraph**(*n*, *q*, *X=None*, *immutable=False*)

Return the Hamming graph with parameters n, q over X.

Hamming graphs are graphs over the cartesian product of n copies of X, where $q = |X|$, where the vertices, labelled with the corresponding tuple in X^n, are connected if the Hamming distance between their labels is 1. All Hamming graphs are regular, vertex-transitive and distance-regular.

Hamming graphs with parameters $(1, q)$ represent the complete graph with q vertices over the set X.

INPUT:

- `n` – power to which `X` will be raised to provide vertices for the Hamming graph
- `q` – cardinality of `X`
- `X` – list of labels representing the vertices of the underlying graph the Hamming graph will be based on; if `None` (or left unused), the list $[0, ..., q-1]$ will be used
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

OUTPUT:

- `G` – the Hamming graph with parameters (n, q, X)

EXAMPLES:

Every Hamming graph is distance-regular, regular and vertex-transitive:

```
sage: g = graphs.HammingGraph(3, 7)
sage: g.is_distance_regular()
True
sage: g.is_regular()
True
sage: g.is_vertex_transitive()                                                 #␣
```

(continues on next page)

(continued from previous page)

```
↪needs sage.groups
True
```

```
>>> from sage.all import *
>>> g = graphs.HammingGraph(Integer(3), Integer(7))
>>> g.is_distance_regular()
True
>>> g.is_regular()
True
>>> g.is_vertex_transitive()                                                #␣
↪needs sage.groups
True
```

A Hamming graph with parameters $(1, q)$ is isomorphic to the Complete graph with parameter q:

```
sage: g = graphs.HammingGraph(1, 23)
sage: g.is_isomorphic(graphs.CompleteGraph(23))
True
```

```
>>> from sage.all import *
>>> g = graphs.HammingGraph(Integer(1), Integer(23))
>>> g.is_isomorphic(graphs.CompleteGraph(Integer(23)))
True
```

If a parameter q is provided which is not equal to X's cardinality, an exception is raised:

```
sage: X = ['a','b','c','d','e']
sage: g = graphs.HammingGraph(2, 3, X)
Traceback (most recent call last):
...
ValueError: q must be the cardinality of X
```

```
>>> from sage.all import *
>>> X = ['a','b','c','d','e']
>>> g = graphs.HammingGraph(Integer(2), Integer(3), X)
Traceback (most recent call last):
...
ValueError: q must be the cardinality of X
```

REFERENCES:

For a more accurate description, see the following wikipedia page: Wikipedia article Hamming_graph

sage.graphs.generators.families.**HanoiTowerGraph**(*pegs*, *disks*, *labels=True*, *positions=True*, *immutable=False*)

Return the graph whose vertices are the states of the Tower of Hanoi puzzle, with edges representing legal moves between states.

INPUT:

- `pegs` – the number of pegs in the puzzle, 2 or greater
- `disks` – the number of disks in the puzzle, 1 or greater

- `labels` – (default: `True`) if `True` the graph contains more meaningful labels, see explanation below. For large instances, turn off labels for much faster creation of the graph.
- `positions` – (default: `True`) if `True` the graph contains layout information. This creates a planar layout for the case of three pegs. For large instances, turn off layout information for much faster creation of the graph.
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

OUTPUT:

The Tower of Hanoi puzzle has a certain number of identical pegs and a certain number of disks, each of a different radius. Initially the disks are all on a single peg, arranged in order of their radii, with the largest on the bottom.

The goal of the puzzle is to move the disks to any other peg, arranged in the same order. The one constraint is that the disks resident on any one peg must always be arranged with larger radii lower down.

The vertices of this graph represent all the possible states of this puzzle. Each state of the puzzle is a tuple with length equal to the number of disks, ordered by largest disk first. The entry of the tuple is the peg where that disk resides. Since disks on a given peg must go down in size as we go up the peg, this totally describes the state of the puzzle.

For example `(2,0,0)` means the large disk is on peg 2, the medium disk is on peg 0, and the small disk is on peg 0 (and we know the small disk must be above the medium disk). We encode these tuples as integers with a base equal to the number of pegs, and low-order digits to the right.

Two vertices are adjacent if we can change the puzzle from one state to the other by moving a single disk. For example, `(2,0,0)` is adjacent to `(2,0,1)` since we can move the small disk off peg 0 and onto (the empty) peg 1. So the solution to a 3-disk puzzle (with at least two pegs) can be expressed by the shortest path between `(0,0,0)` and `(1,1,1)`. For more on this representation of the graph, or its properties, see [AD2010].

For greatest speed we create graphs with integer vertices, where we encode the tuples as integers with a base equal to the number of pegs, and low-order digits to the right. So for example, in a 3-peg puzzle with 5 disks, the state `(1,2,0,1,1)` is encoded as $1 * 3^4 + 2 * 3^3 + 0 * 3^2 + 1 * 3^1 + 1 * 3^0 = 139$.

For smaller graphs, the labels that are the tuples are informative, but slow down creation of the graph. Likewise computing layout information also incurs a significant speed penalty. For maximum speed, turn off labels and layout and decode the vertices explicitly as needed. The `sage.rings.integer.Integer.digits()` with the `padsto` option is a quick way to do this, though you may want to reverse the list that is output.

See also

- *GeneralizedSierpinskiGraph()*

PLOTTING:

The layout computed when `positions = True` will look especially good for the three-peg case, when the graph is known to be planar. Except for two small cases on 4 pegs, the graph is otherwise not planar, and likely there is a better way to layout the vertices.

EXAMPLES:

A classic puzzle uses 3 pegs. We solve the 5 disk puzzle using integer labels and report the minimum number of moves required. Note that $3^5 - 1$ is the state where all 5 disks are on peg 2.

```
sage: H = graphs.HanoiTowerGraph(3, 5, labels=False, positions=False)
sage: H.distance(0, 3^5-1)
31
```

```
>>> from sage.all import *
>>> H = graphs.HanoiTowerGraph(Integer(3), Integer(5), labels=False,␣
↪positions=False)
>>> H.distance(Integer(0), Integer(3)**Integer(5)-Integer(1))
31
```

A slightly larger instance.

```
sage: H = graphs.HanoiTowerGraph(4, 6, labels=False, positions=False)
sage: H.n_vertices()
4096
sage: H.distance(0, 4^6-1)
17
```

```
>>> from sage.all import *
>>> H = graphs.HanoiTowerGraph(Integer(4), Integer(6), labels=False,␣
↪positions=False)
>>> H.n_vertices()
4096
>>> H.distance(Integer(0), Integer(4)**Integer(6)-Integer(1))
17
```

For a small graph, labels and layout information can be useful. Here we explicitly list a solution as a list of states.

```
sage: H = graphs.HanoiTowerGraph(3, 3, labels=True, positions=True)
sage: H.shortest_path((0,0,0), (1,1,1))
[(0, 0, 0), (0, 0, 1), (0, 2, 1), (0, 2, 2), (1, 2, 2), (1, 2, 0), (1, 1, 0), (1,␣
↪1, 1)]
```

```
>>> from sage.all import *
>>> H = graphs.HanoiTowerGraph(Integer(3), Integer(3), labels=True,␣
↪positions=True)
>>> H.shortest_path((Integer(0),Integer(0),Integer(0)), (Integer(1),Integer(1),
↪Integer(1)))
[(0, 0, 0), (0, 0, 1), (0, 2, 1), (0, 2, 2), (1, 2, 2), (1, 2, 0), (1, 1, 0), (1,␣
↪1, 1)]
```

Some facts about this graph with p pegs and d disks:

- only automorphisms are the "obvious" ones – renumber the pegs
- chromatic number is less than or equal to p
- independence number is p^{d-1}

```
sage: H = graphs.HanoiTowerGraph(3, 4, labels=False, positions=False)
sage: H.automorphism_group().is_isomorphic(SymmetricGroup(3))                    #␣
↪needs sage.groups
True
sage: H.chromatic_number()
3
sage: len(H.independent_set()) == 3^(4-1)
True
```

```
>>> from sage.all import *
>>> H = graphs.HanoiTowerGraph(Integer(3), Integer(4), labels=False,␣
↪positions=False)
>>> H.automorphism_group().is_isomorphic(SymmetricGroup(Integer(3)))          ␣
↪     # needs sage.groups
True
>>> H.chromatic_number()
3
>>> len(H.independent_set()) == Integer(3)**(Integer(4)-Integer(1))
True
```

AUTHOR:

- Rob Beezer, (2009-12-26), with assistance from Su Doree

sage.graphs.generators.families.**HararyGraph**(*k*, *n*, *immutable=False*)

Return the Harary graph on n vertices and connectivity k, where $2 \leq k < n$.

A k-connected graph G on n vertices requires the minimum degree $\delta(G) \geq k$, so the minimum number of edges G should have is $\lceil kn/2 \rceil$. Harary graphs achieve this lower bound, that is, Harary graphs are minimal k-connected graphs on n vertices.

The construction provided uses the method CirculantGraph. For more details, see the book [West2001] or the MathWorld article on Harary graphs.

INPUT:

- `k` – integer; connectivity of the graph
- `n` – integer; number of vertices of the graph
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

Harary graphs $H_{k,n}$:

```
sage: h = graphs.HararyGraph(5,9); h
Harary graph 5, 9: Graph on 9 vertices
sage: h.order()
9
sage: h.size()
23
sage: h.vertex_connectivity()                                               #␣
↪needs sage.numerical.mip
5
```

```
>>> from sage.all import *
>>> h = graphs.HararyGraph(Integer(5),Integer(9)); h
Harary graph 5, 9: Graph on 9 vertices
>>> h.order()
9
>>> h.size()
23
>>> h.vertex_connectivity()                                                 #␣
↪needs sage.numerical.mip
5
```

sage.graphs.generators.families.**HyperStarGraph**(*n*, *k*, *immutable=False*)

Return the hyper-star graph $HS(n, k)$.

The vertices of the hyper-star graph are the set of binary strings of length n which contain k 1s. Two vertices, u and v, are adjacent only if u can be obtained from v by swapping the first bit with a different symbol in another position. For instance, vertex `'011100'` of $HS(6, 3)$ is adjacent to vertices `'101100'`, `'110100'` and `'111000'`. See [LKOL2002] for more details.

INPUT:

- `n` – nonnegative integer; length of the binary strings
- `k` – nonnegative integer; number of 1s per binary string
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.HyperStarGraph(6,3)
sage: sorted(g.neighbors('011100'))
['101100', '110100', '111000']
sage: g.plot()                              # long time                              #␣
↪needs sage.plot
Graphics object consisting of 51 graphics primitives
```

```
>>> from sage.all import *
>>> g = graphs.HyperStarGraph(Integer(6),Integer(3))
>>> sorted(g.neighbors('011100'))
['101100', '110100', '111000']
>>> g.plot()                              # long time                              #␣
↪needs sage.plot
Graphics object consisting of 51 graphics primitives
```

AUTHORS:

- Michael Yurko (2009-09-01)

sage.graphs.generators.families.**IGraph**(*n*, *j*, *k*, *immutable=False*)

Return an I-graph with $2n$ nodes.

The I-Graph family as been proposed in [BCMS1988] as a generalization of the generalized Petersen graphs. The variables n, j, k are integers such that $n > 2$ and $0 < j, k \leq \lfloor (n-1)/2 \rfloor$. When $j = 1$ the resulting graph is isomorphic to the generalized Petersen graph with the same n and k.

INPUT:

- `n` – the number of nodes is $2 * n$
- `j` – integer such that $0 < j \leq \lfloor (n-1)/2 \rfloor$ determining how outer vertices are connected
- `k` – integer such that $0 < k \leq \lfloor (n-1)/2 \rfloor$ determining how inner vertices are connected
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

PLOTTING: Upon construction, the position dictionary is filled to override the spring-layout algorithm. By convention, the I-graphs are displayed as an inner and outer cycle pair, with the first n nodes drawn on the outer circle. The first (0) node is drawn at the top of the outer-circle, moving counterclockwise after that. The inner circle is drawn with the (n)th node at the top, then counterclockwise as well.

EXAMPLES:

When $j = 1$ the resulting graph will be isomorphic to a generalized Petersen graph:

```
sage: g = graphs.IGraph(7,1,2)
sage: g2 = graphs.GeneralizedPetersenGraph(7,2)
sage: g.is_isomorphic(g2)
True
```

```
>>> from sage.all import *
>>> g = graphs.IGraph(Integer(7),Integer(1),Integer(2))
>>> g2 = graphs.GeneralizedPetersenGraph(Integer(7),Integer(2))
>>> g.is_isomorphic(g2)
True
```

The IGraph with parameters (n, j, k) is isomorphic to the IGraph with parameters (n, k, j):

```
sage: g = graphs.IGraph(7, 2, 3)
sage: h = graphs.IGraph(7, 3, 2)
sage: g.is_isomorphic(h)
True
```

```
>>> from sage.all import *
>>> g = graphs.IGraph(Integer(7), Integer(2), Integer(3))
>>> h = graphs.IGraph(Integer(7), Integer(3), Integer(2))
>>> g.is_isomorphic(h)
True
```

sage.graphs.generators.families.**JohnsonGraph**(*n*, *k*, *immutable=False*)

Return the Johnson graph with parameters n, k.

Johnson graphs are a special class of undirected graphs defined from systems of sets. The vertices of the Johnson graph $J(n, k)$ are the k-element subsets of an n-element set; two vertices are adjacent when they meet in a $(k - 1)$-element set. See the Wikipedia article Johnson_graph for more information.

INPUT:

- `n` – nonnegative integer; number of elements of the groundset
- `k` – nonnegative integer; number of elements of each subset
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

The Johnson graph is a Hamiltonian graph:

```
sage: g = graphs.JohnsonGraph(7, 3)
sage: g.is_hamiltonian()                                                    #␣
↪needs sage.numerical.mip
True
```

```
>>> from sage.all import *
>>> g = graphs.JohnsonGraph(Integer(7), Integer(3))
>>> g.is_hamiltonian()                                                      #␣
↪needs sage.numerical.mip
True
```

Every Johnson graph is vertex transitive:

```
sage: g = graphs.JohnsonGraph(6, 4)
sage: g.is_vertex_transitive()                                              #␣
↪needs sage.groups
True
```

```
>>> from sage.all import *
>>> g = graphs.JohnsonGraph(Integer(6), Integer(4))
>>> g.is_vertex_transitive()                                                #␣
↪needs sage.groups
True
```

The complement of the Johnson graph $J(n, 2)$ is isomorphic to the Kneser Graph $K(n, 2)$. In particular the complement of $J(5, 2)$ is isomorphic to the Petersen graph.:

```
sage: g = graphs.JohnsonGraph(5,2)
sage: g.complement().is_isomorphic(graphs.PetersenGraph())
True
```

```
>>> from sage.all import *
>>> g = graphs.JohnsonGraph(Integer(5),Integer(2))
>>> g.complement().is_isomorphic(graphs.PetersenGraph())
True
```

sage.graphs.generators.families.**KneserGraph**(*n*, *k*, *immutable=False*, *name=None*)

Return the Kneser Graph with parameters n, k.

The Kneser Graph with parameters n, k is the graph whose vertices are the k-subsets of $[0, 1, \ldots, n-1]$, and such that two vertices are adjacent if their corresponding sets are disjoint.

For example, the Petersen Graph can be defined as the Kneser Graph with parameters $5, 2$.

INPUT:

- `n` – positive integer; number of elements of the groundset
- `k` – positive integer; number of elements of each subset
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph
- `name` – string (default: `None`); used as the name of the returned graph when set

EXAMPLES:

```
sage: KG = graphs.KneserGraph(5,2)
sage: sorted(KG.vertex_iterator(), key=str)
[{1, 2}, {1, 3}, {1, 4}, {1, 5}, {2, 3}, {2, 4}, {2, 5},
 {3, 4}, {3, 5}, {4, 5}]
sage: P = graphs.PetersenGraph()
sage: P.is_isomorphic(KG)
True
```

```
>>> from sage.all import *
>>> KG = graphs.KneserGraph(Integer(5),Integer(2))
>>> sorted(KG.vertex_iterator(), key=str)
[{1, 2}, {1, 3}, {1, 4}, {1, 5}, {2, 3}, {2, 4}, {2, 5},
 {3, 4}, {3, 5}, {4, 5}]
```

(continues on next page)

(continued from previous page)

```
>>> P = graphs.PetersenGraph()
>>> P.is_isomorphic(KG)
True
```

sage.graphs.generators.families.**LCFGraph**(*n*, *shift_list*, *repeats*, *immutable=False*, *name=None*)

Return the cubic graph specified in LCF notation.

LCF (Lederberg-Coxeter-Fruchte) notation is a concise way of describing cubic Hamiltonian graphs. The way a graph is constructed is as follows. Since there is a Hamiltonian cycle, we first create a cycle on n nodes. The variable `shift_list` = $[s_0, s_1, ..., s_{k-1}]$ describes edges to be created by the following scheme: for each $i \in \{0, 1, \ldots, k-1\}$, connect vertex i to vertex $(i + s_i) \pmod n$. Then, `repeats` specifies the number of times to repeat this process, where on the j-th repeat we connect vertex $(i+jk) \pmod n$ to vertex $(i+jk+s_i) \pmod n$.

For more details, see the Wikipedia article LCF_notation and [Fru1977], [Gru2003] pp. 357-365, and [Led1965].

INPUT:

- `n` – the number of nodes
- `shift_list` – a list of integer shifts mod n
- `repeats` – the number of times to repeat the process
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph
- `name` – string (default: `None`); used as the name of the returned graph when set

EXAMPLES:

```
sage: G = graphs.LCFGraph(4, [2,-2], 2)
sage: G.is_isomorphic(graphs.TetrahedralGraph())
True
```

```
>>> from sage.all import *
>>> G = graphs.LCFGraph(Integer(4), [Integer(2),-Integer(2)], Integer(2))
>>> G.is_isomorphic(graphs.TetrahedralGraph())
True
```

```
sage: G = graphs.LCFGraph(20, [10,7,4,-4,-7,10,-4,7,-7,4], 2)
sage: G.is_isomorphic(graphs.DodecahedralGraph())
True
```

```
>>> from sage.all import *
>>> G = graphs.LCFGraph(Integer(20), [Integer(10),Integer(7),Integer(4),-
↪Integer(4),-Integer(7),Integer(10),-Integer(4),Integer(7),-Integer(7),
↪Integer(4)], Integer(2))
>>> G.is_isomorphic(graphs.DodecahedralGraph())
True
```

```
sage: G = graphs.LCFGraph(14, [5,-5], 7)
sage: G.is_isomorphic(graphs.HeawoodGraph())
True
```

```
>>> from sage.all import *
>>> G = graphs.LCFGraph(Integer(14), [Integer(5),-Integer(5)], Integer(7))
```

(continues on next page)

(continued from previous page)

```
>>> G.is_isomorphic(graphs.HeawoodGraph())
True
```

The largest cubic nonplanar graph of diameter three:

```
sage: G = graphs.LCFGraph(20, [-10,-7,-5,4,7,-10,-7,-4,5,7,
....:                          -10,-7,6,-5,7,-10,-7,5,-6,7], 1)
sage: G.degree()
[3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3]
sage: G.diameter()
3
sage: G.show()                              # long time                              #␣
↪needs sage.plot
```

```
>>> from sage.all import *
>>> G = graphs.LCFGraph(Integer(20), [-Integer(10),-Integer(7),-Integer(5),
↪Integer(4),Integer(7),-Integer(10),-Integer(7),-Integer(4),Integer(5),
↪Integer(7),
...                           -Integer(10),-Integer(7),Integer(6),-Integer(5),
↪Integer(7),-Integer(10),-Integer(7),Integer(5),-Integer(6),Integer(7)],␣
↪Integer(1))
>>> G.degree()
[3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3]
>>> G.diameter()
3
>>> G.show()                              # long time                              #␣
↪needs sage.plot
```

PLOTTING: LCF Graphs are plotted as an n-cycle with edges in the middle, as described above.

`sage.graphs.generators.families.`**`LollipopGraph`**(*n1*, *n2*, *immutable=False*)

Return a lollipop graph with $n_1 + n_2$ nodes.

A lollipop graph is a path graph (order n_2) connected to a complete graph (order n_1). (A barbell graph minus one of the bells).

PLOTTING: Upon construction, the position dictionary is filled to override the spring-layout algorithm. By convention, the complete graph will be drawn in the lower-left corner with the n_1-th node at a 45 degree angle above the right horizontal center of the complete graph, leading directly into the path graph.

INPUT:

- `n1` – integer ≥ 0; the order of the complete graph
- `n2` – integer ≥ 0; the order of the path graph
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

Construct and show a lollipop graph Candy = 13, Stick = 4:

```
sage: g = graphs.LollipopGraph(13,4); g
Lollipop graph: Graph on 17 vertices
sage: g.show()                              # long time                              #␣
↪needs sage.plot
```

```
>>> from sage.all import *
>>> g = graphs.LollipopGraph(Integer(13),Integer(4)); g
Lollipop graph: Graph on 17 vertices
>>> g.show()                              # long time                              #␣
↪needs sage.plot
```

sage.graphs.generators.families.**MathonPseudocyclicMergingGraph**(*M*, *t*, *immutable=False*)

Mathon's merging of classes in a pseudo-cyclic 3-class association scheme.

Construct strongly regular graphs from p.97 of [BL1984].

INPUT:

- `M` – the list of matrices in a pseudo-cyclic 3-class association scheme; the identity matrix must be the first entry
- `t` – integer; the number of the graph, from 0 to 2
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

See also

- *is_muzychuk_S6()*

sage.graphs.generators.families.**MathonPseudocyclicStronglyRegularGraph**(*t*, *G=None*, *L=None*, *immutable=False*)

Return a strongly regular graph on $(4t+1)(4t-1)^2$ vertices from [Mat1978].

Let $4t-1$ be a prime power, and $4t+1$ be such that there exists a strongly regular graph G with parameters $(4t+1, 2t, t-1, t)$. In particular, $4t+1$ must be a sum of two squares [Mat1978]. With this input, Mathon [Mat1978] gives a construction of a strongly regular graph with parameters $(4\mu+1, 2\mu, \mu-1, \mu)$, where $\mu = t(4t(4t-1)-1)$. The construction is optionally parametrised by an a skew-symmetric Latin square of order $4t+1$, with entries in $-2t, ..., -1, 0, 1, ..., 2t$.

Our implementation follows a description given in [ST1981].

INPUT:

- `t` – positive integer
- `G` – if `None` (default), try to construct the necessary graph with parameters $(4t+1, 2t, t-1, t)$, otherwise use the user-supplied one, with vertices labelled from 0 to $4t$.
- `L` – if `None` (default), construct a necessary skew Latin square, otherwise use the user-supplied one. Here non-isomorphic Latin squares – one constructed from $Z/9Z$, and the other from $(Z/3Z)^2$ – lead to non-isomorphic graphs.
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

See also

- *is_mathon_PC_srg()*

EXAMPLES:

Using default `G` and `L`.

```
sage: from sage.graphs.generators.families import␣
↪MathonPseudocyclicStronglyRegularGraph
sage: G = MathonPseudocyclicStronglyRegularGraph(1); G                          #␣
↪needs sage.modules sage.rings.finite_rings
Mathon's PC SRG on 45 vertices: Graph on 45 vertices
sage: G.is_strongly_regular(parameters=True)                                    #␣
↪needs sage.modules sage.rings.finite_rings
(45, 22, 10, 11)
```

```
>>> from sage.all import *
>>> from sage.graphs.generators.families import␣
↪MathonPseudocyclicStronglyRegularGraph
>>> G = MathonPseudocyclicStronglyRegularGraph(Integer(1)); G                    ␣
↪     # needs sage.modules sage.rings.finite_rings
Mathon's PC SRG on 45 vertices: Graph on 45 vertices
>>> G.is_strongly_regular(parameters=True)                                      #␣
↪needs sage.modules sage.rings.finite_rings
(45, 22, 10, 11)
```

Supplying `G` and `L` (constructed from the automorphism group of `G`). The entries of L can't be tested directly because there's some unpredictability in the way that GAP chooses a representative in `NormalSubgroups()`, the function that underlies our own `normal_subgroups()` method:

```
sage: G = graphs.PaleyGraph(9)
sage: a = G.automorphism_group(partition=[sorted(G)])
sage: it = (x for x in a.normal_subgroups() if x.order() == 9)
sage: subg = next(iter(it))
sage: r = [matrix(libgap.PermutationMat(libgap(z), 9).sage())
....:       for z in subg]
sage: ff = list(map(lambda y: (y[0]-1,y[1]-1),
....:           Permutation(map(lambda x: 1+r.index(x^-1), r)).cycle_tuples()[1:]))
sage: L = sum(i*(r[a]-r[b]) for i,(a,b) in zip(range(1,len(ff)+1), ff))
sage: G.relabel(range(9))
sage: G3x3 = graphs.MathonPseudocyclicStronglyRegularGraph(2, G=G, L=L)
sage: G3x3.is_strongly_regular(parameters=True)
(441, 220, 109, 110)
sage: G3x3.automorphism_group(algorithm='bliss').order()  # optional - bliss
27
sage: G9 = graphs.MathonPseudocyclicStronglyRegularGraph(2)
sage: G9.is_strongly_regular(parameters=True)
(441, 220, 109, 110)
sage: G9.automorphism_group(algorithm='bliss').order()  # optional - bliss
9
```

```
>>> from sage.all import *
>>> G = graphs.PaleyGraph(Integer(9))
>>> a = G.automorphism_group(partition=[sorted(G)])
>>> it = (x for x in a.normal_subgroups() if x.order() == Integer(9))
>>> subg = next(iter(it))
>>> r = [matrix(libgap.PermutationMat(libgap(z), Integer(9)).sage())
...       for z in subg]
>>> ff = list(map(lambda y: (y[Integer(0)]-Integer(1),y[Integer(1)]-Integer(1)),
```

(continues on next page)

(continued from previous page)

```
...          Permutation(map(lambda x: Integer(1)+r.index(x**-Integer(1)), r)).
↪cycle_tuples()[Integer(1):]))
>>> L = sum(i*(r[a]-r[b]) for i,(a,b) in zip(range(Integer(1),len(ff)+Integer(1)),
↪ ff))
>>> G.relabel(range(Integer(9)))
>>> G3x3 = graphs.MathonPseudocyclicStronglyRegularGraph(Integer(2), G=G, L=L)
>>> G3x3.is_strongly_regular(parameters=True)
(441, 220, 109, 110)
>>> G3x3.automorphism_group(algorithm='bliss').order()  # optional - bliss
27
>>> G9 = graphs.MathonPseudocyclicStronglyRegularGraph(Integer(2))
>>> G9.is_strongly_regular(parameters=True)
(441, 220, 109, 110)
>>> G9.automorphism_group(algorithm='bliss').order()  # optional - bliss
9
```

sage.graphs.generators.families.**MuzychukS6Graph**(*n*, *d*, *Phi='fixed'*, *Sigma='fixed'*, *verbose=False*, *immutable=False*)

Return a strongly regular graph of S6 type from [Muz2007] on $n^d((n^d-1)/(n-1)+1)$ vertices.

The construction depends upon a number of parameters, two of them, n and d, mandatory, and Φ and Σ mappings defined in [Muz2007]. These graphs have parameters $(mn^d, n^{d-1}(m-1)-1, \mu-2, \mu)$, where $\mu = \frac{n^{d-1}-1}{n-1}n^{d-1}$ and $m := \frac{n^d-1}{n-1}+1$.

Some details on Φ and Σ are as follows. Let L be the complete graph on $M := \{0, ..., m-1\}$ with the matching $\{(2i, 2i+1) | i = 0, ..., m/2\}$ removed. Then one arbitrarily chooses injections Φ_i from the edges of L on $i \in M$ into sets of parallel classes of affine d-dimensional designs; our implementation uses the designs of hyperplanes in d-dimensional affine geometries over $GF(n)$. Finally, for each edge ij of L one arbitrarily chooses bijections Σ_{ij} between Φ_i and Φ_j. More details, in particular how these choices lead to non-isomorphic graphs, are in [Muz2007].

INPUT:

- `n` – integer; a prime power
- `d` – integer; must be odd if n is odd
- `Phi` is an optional parameter of the construction; it must be either
  - `'fixed'` – this will generate fixed default Φ_i, for $i \in M$, or
  - `'random'` – Φ_i are generated at random, or
  - A dictionary describing the functions Φ_i; for $i \in M$, Phi[(i, T)] in M, for each edge T of L on i. Also, each Φ_i must be injective.
- `Sigma` is an optional parameter of the construction; it must be either
  - `'fixed'` – this will generate a fixed default Σ, or
  - `'random'` – Σ is generated at random
- `verbose` – boolean (default: `False`); if `True`, print progress information
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

See also

- *is_muzychuk_S6()*

Todo

Implement the possibility to explicitly supply the parameter Σ of the construction.

EXAMPLES:

```
sage: graphs.MuzychukS6Graph(3, 3).is_strongly_regular(parameters=True)
(378, 116, 34, 36)
sage: phi = {(2,(0,2)):0, (1,(1,3)):1, (0,(0,3)):1, (2,(1,2)):1,
....:        (1,(1,2)):0, (0,(0,2)):0, (3,(0,3)):0, (3,(1,3)):1}
sage: graphs.MuzychukS6Graph(2, 2, Phi=phi).is_strongly_regular(parameters=True)
(16, 5, 0, 2)
```

```
>>> from sage.all import *
>>> graphs.MuzychukS6Graph(Integer(3), Integer(3)).is_strongly_
↪regular(parameters=True)
(378, 116, 34, 36)
>>> phi = {(Integer(2),(Integer(0),Integer(2))):Integer(0), (Integer(1),
↪(Integer(1),Integer(3))):Integer(1), (Integer(0),(Integer(0),
↪Integer(3))):Integer(1), (Integer(2),(Integer(1),Integer(2))):Integer(1),
...        (Integer(1),(Integer(1),Integer(2))):Integer(0), (Integer(0),
↪(Integer(0),Integer(2))):Integer(0), (Integer(3),(Integer(0),
↪Integer(3))):Integer(0), (Integer(3),(Integer(1),Integer(3))):Integer(1)}
>>> graphs.MuzychukS6Graph(Integer(2), Integer(2), Phi=phi).is_strongly_
↪regular(parameters=True)
(16, 5, 0, 2)
```

sage.graphs.generators.families.**MycielskiGraph**(*k=1*, *relabel=True*, *immutable=False*)

Return the k-th Mycielski Graph.

The graph M_k is triangle-free and has chromatic number equal to k. These graphs show, constructively, that there are triangle-free graphs with arbitrarily high chromatic number.

The Mycielski graphs are built recursively starting with M_0, an empty graph; M_1, a single vertex graph; and M_2 is the graph K_2. M_{k+1} is then built from M_k as follows:

If the vertices of M_k are $v_1, \ldots, v_n$, then the vertices of M_{k+1} are $v_1, \ldots, v_n, w_1, \ldots, w_n, z$. Vertices $v_1, \ldots, v_n$ induce a copy of M_k. Vertices $w_1, \ldots, w_n$ are an independent set. Vertex z is adjacent to all the w_i-vertices. Finally, vertex w_i is adjacent to vertex v_j iff v_i is adjacent to v_j.

For more details, see the Wikipedia article Mycielskian.

INPUT:

- `k` – number of steps in the construction process
- `relabel` – relabel the vertices so their names are the integers `range(n)` where `n` is the number of vertices in the graph
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

The Mycielski graph M_k is triangle-free and has chromatic number equal to k.

```
sage: g = graphs.MycielskiGraph(5)
sage: g.is_triangle_free()
True
sage: g.chromatic_number()
5
```

```
>>> from sage.all import *
>>> g = graphs.MycielskiGraph(Integer(5))
>>> g.is_triangle_free()
True
>>> g.chromatic_number()
5
```

The graphs M_4 is (isomorphic to) the Grotzsch graph.

```
sage: g = graphs.MycielskiGraph(4)
sage: g.is_isomorphic(graphs.GrotzschGraph())
True
```

```
>>> from sage.all import *
>>> g = graphs.MycielskiGraph(Integer(4))
>>> g.is_isomorphic(graphs.GrotzschGraph())
True
```

sage.graphs.generators.families.**MycielskiStep**(*g*, *immutable=None*)

Perform one iteration of the Mycielski construction.

See the documentation for `MycielskiGraph` which uses this method. We expose it to all users in case they may find it useful.

INPUT:

- `g` – a graph
- `immutable` – boolean (default: `None`); whether to return a mutable or an immutable graph. `immutable=None` (default) means that the input and returned graphs behave the same way.

EXAMPLES:

One iteration of the Mycielski step applied to the 5-cycle yields a graph isomorphic to the Grotzsch graph

```
sage: g = graphs.CycleGraph(5)
sage: h = graphs.MycielskiStep(g)
sage: h.is_isomorphic(graphs.GrotzschGraph())
True
```

```
>>> from sage.all import *
>>> g = graphs.CycleGraph(Integer(5))
>>> h = graphs.MycielskiStep(g)
>>> h.is_isomorphic(graphs.GrotzschGraph())
True
```

sage.graphs.generators.families.**NKStarGraph**(*n*, *k*, *immutable=False*)

Return the (n, k)-star graph.

The vertices of the (n, k)-star graph are the set of all arrangements of n symbols into labels of length k. There are two adjacency rules for the (n, k)-star graph. First, two vertices are adjacent if one can be obtained from the other by swapping the first symbol with another symbol. Second, two vertices are adjacent if one can be obtained from the other by swapping the first symbol with an external symbol (a symbol not used in the original label).

INPUT:

- `n` – integer; number of symbols
- `k` – integer; length of the labels of the vertices
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.NKStarGraph(4,2)
sage: g.plot()                          # long time                          #␣
↪needs sage.plot
Graphics object consisting of 31 graphics primitives
```

```
>>> from sage.all import *
>>> g = graphs.NKStarGraph(Integer(4),Integer(2))
>>> g.plot()                            # long time                          #␣
↪needs sage.plot
Graphics object consisting of 31 graphics primitives
```

REFERENCES:

[CC1995]

AUTHORS:

- Michael Yurko (2009-09-01)

sage.graphs.generators.families.**NStarGraph**(*n*, *immutable=False*)

Return the n-star graph.

The vertices of the n-star graph are the set of permutations on n symbols. There is an edge between two vertices if their labels differ only in the first and one other position.

INPUT:

- `n` – integer; number of symbols
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.NStarGraph(4)
sage: g.plot()                          # long time                          #␣
↪needs sage.plot
Graphics object consisting of 61 graphics primitives
```

```
>>> from sage.all import *
>>> g = graphs.NStarGraph(Integer(4))
>>> g.plot()                            # long time                          #␣
↪needs sage.plot
Graphics object consisting of 61 graphics primitives
```

REFERENCES:

[AHK1994]

AUTHORS:

- Michael Yurko (2009-09-01)

sage.graphs.generators.families.**OddGraph**(*n*, *immutable=False*)

Return the Odd Graph with parameter n.

The Odd Graph with parameter n is defined as the Kneser Graph with parameters $2n-1, n-1$. Equivalently, the Odd Graph is the graph whose vertices are the $n-1$-subsets of $[0, 1, \ldots, 2(n-1)]$, and such that two vertices are adjacent if their corresponding sets are disjoint.

For example, the Petersen Graph can be defined as the Odd Graph with parameter 3.

INPUT:

- `n` – integer; the groundset has $2n-1$ elements
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: OG = graphs.OddGraph(3)
sage: sorted(OG.vertex_iterator(), key=str)
[{1, 2}, {1, 3}, {1, 4}, {1, 5}, {2, 3}, {2, 4}, {2, 5},
 {3, 4}, {3, 5}, {4, 5}]
sage: P = graphs.PetersenGraph()
sage: P.is_isomorphic(OG)
True
```

```
>>> from sage.all import *
>>> OG = graphs.OddGraph(Integer(3))
>>> sorted(OG.vertex_iterator(), key=str)
[{1, 2}, {1, 3}, {1, 4}, {1, 5}, {2, 3}, {2, 4}, {2, 5},
 {3, 4}, {3, 5}, {4, 5}]
>>> P = graphs.PetersenGraph()
>>> P.is_isomorphic(OG)
True
```

sage.graphs.generators.families.**PaleyGraph**(*q*, *immutable=False*)

Paley graph with q vertices.

Parameter q must be the power of a prime number and congruent to 1 mod 4.

INPUT:

- `q` – integer; number of vertices
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: G = graphs.PaleyGraph(9); G                                              #␣
↪needs sage.rings.finite_rings
Paley graph with parameter 9: Graph on 9 vertices
sage: G.is_regular()                                                           #␣
↪needs sage.rings.finite_rings
True
```

```
>>> from sage.all import *
>>> G = graphs.PaleyGraph(Integer(9)); G
↪     # needs sage.rings.finite_rings
Paley graph with parameter 9: Graph on 9 vertices
>>> G.is_regular()
↪needs sage.rings.finite_rings
True
```

A Paley graph is always self-complementary:

```
sage: G.is_self_complementary()
↪needs sage.rings.finite_rings
True
```

```
>>> from sage.all import *
>>> G.is_self_complementary()
↪needs sage.rings.finite_rings
True
```

sage.graphs.generators.families.**PasechnikGraph**(*n*, *immutable=False*)

Pasechnik strongly regular graph on $(4n-1)^2$ vertices.

A strongly regular graph with parameters of the orthogonal array graph *OrthogonalArrayBlockGraph()*, also known as pseudo Latin squares graph $L_{2n-1}(4n-1)$, constructed from a skew Hadamard matrix of order $4n$ following [Pas1992].

See also

- *is_orthogonal_array_block_graph()*

INPUT:

- `n` – integer; the returned graph has $(4n-1)^2$ vertices
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: graphs.PasechnikGraph(4).is_strongly_regular(parameters=True)
↪needs sage.combinat sage.modules
(225, 98, 43, 42)
sage: graphs.PasechnikGraph(5).is_strongly_regular(parameters=True)     # long
↪time, needs sage.combinat sage.modules
(361, 162, 73, 72)
sage: graphs.PasechnikGraph(9).is_strongly_regular(parameters=True)  # not tested
(1225, 578, 273, 272)
```

```
>>> from sage.all import *
>>> graphs.PasechnikGraph(Integer(4)).is_strongly_regular(parameters=True)
↪     # needs sage.combinat sage.modules
(225, 98, 43, 42)
>>> graphs.PasechnikGraph(Integer(5)).is_strongly_regular(parameters=True)
↪long time, needs sage.combinat sage.modules
```

(continues on next page)

(continued from previous page)

```
(361, 162, 73, 72)
>>> graphs.PasechnikGraph(Integer(9)).is_strongly_regular(parameters=True)  # not␣
↪tested
(1225, 578, 273, 272)
```

sage.graphs.generators.families.**RingedTree**(*k*, *vertex_labels=True*, *immutable=False*)

Return the ringed tree on k-levels.

A ringed tree of level k is a binary tree with k levels (counting the root as a level), in which all vertices at the same level are connected by a ring.

More precisely, in each layer of the binary tree (i.e. a layer is the set of vertices $[2^i...2^{i+1}-1]$) two vertices u, v are adjacent if $u = v + 1$ or if $u = 2^i$ and $v = 2^{i+1} - 1$.

Ringed trees are defined in [CFHM2013].

INPUT:

- `k` – the number of levels of the ringed tree
- `vertex_labels` – boolean; whether to label vertices as binary words (default) or as integers
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: # needs networkx
sage: G = graphs.RingedTree(5)
sage: P = G.plot(vertex_labels=False, vertex_size=10)                          #␣
↪needs sage.plot
sage: P.show()                            # long time                          #␣
↪needs sage.plot
sage: G.vertices(sort=True)
['', '0', '00', '000', '0000', '0001', '001', '0010', '0011', '01',
 '010', '0100', '0101', '011', '0110', '0111', '1', '10', '100',
 '1000', '1001', '101', '1010', '1011', '11', '110', '1100', '1101',
 '111', '1110', '1111']
```

```
>>> from sage.all import *
>>> # needs networkx
>>> G = graphs.RingedTree(Integer(5))
>>> P = G.plot(vertex_labels=False, vertex_size=Integer(10))                    ␣
↪     # needs sage.plot
>>> P.show()                              # long time                          #␣
↪needs sage.plot
>>> G.vertices(sort=True)
['', '0', '00', '000', '0000', '0001', '001', '0010', '0011', '01',
 '010', '0100', '0101', '011', '0110', '0111', '1', '10', '100',
 '1000', '1001', '101', '1010', '1011', '11', '110', '1100', '1101',
 '111', '1110', '1111']
```

sage.graphs.generators.families.**RoseWindowGraph**(*n*, *a*, *r*, *immutable=False*)

Return a rose window graph with $2n$ nodes.

The rose window graphs is a family of tetravalent graphs introduced in [Wilson2008]. The parameters n, a and r are integers such that $n > 2$, $1 \leq a, r < n$, and $r \neq n/2$.

INPUT:

- `n` – the number of nodes is $2 * n$
- `a` – integer such that $1 \leq a < n$ determining a-spoke edges
- `r` – integer such that $1 \leq r < n$ and $r \neq n/2$ determining how inner vertices are connected
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

PLOTTING: Upon construction, the position dictionary is filled to override the spring-layout algorithm. By convention, the rose window graphs are displayed as an inner and outer cycle pair, with the first n nodes drawn on the outer circle. The first (0) node is drawn at the top of the outer-circle, moving counterclockwise after that. The inner circle is drawn with the (n)th node at the top, then counterclockwise as well. Vertices in the outer circle are connected in the circular manner, vertices in the inner circle are connected when their label have difference $r \pmod n$. Vertices on the outer rim are connected with the vertices on the inner rim when they are at the same position and when they are a apart.

EXAMPLES:

The vertices of a rose window graph have all degree 4:

```
sage: G = graphs.RoseWindowGraph(5, 1, 2)
sage: all(G.degree(u) == 4 for u in G)
True
```

```
>>> from sage.all import *
>>> G = graphs.RoseWindowGraph(Integer(5), Integer(1), Integer(2))
>>> all(G.degree(u) == Integer(4) for u in G)
True
```

The smallest rose window graph as parameters $(3, 2, 1)$:

```
sage: G = graphs.RoseWindowGraph(3, 2, 1)
sage: all(G.degree(u) == 4 for u in G)
True
```

```
>>> from sage.all import *
>>> G = graphs.RoseWindowGraph(Integer(3), Integer(2), Integer(1))
>>> all(G.degree(u) == Integer(4) for u in G)
True
```

`sage.graphs.generators.families.`**`SierpinskiGasketGraph`**(*n*, *immutable=False*)

Return the Sierpinski Gasket graph of generation n.

All vertices but 3 have valence 4.

INPUT:

- `n` – integer
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

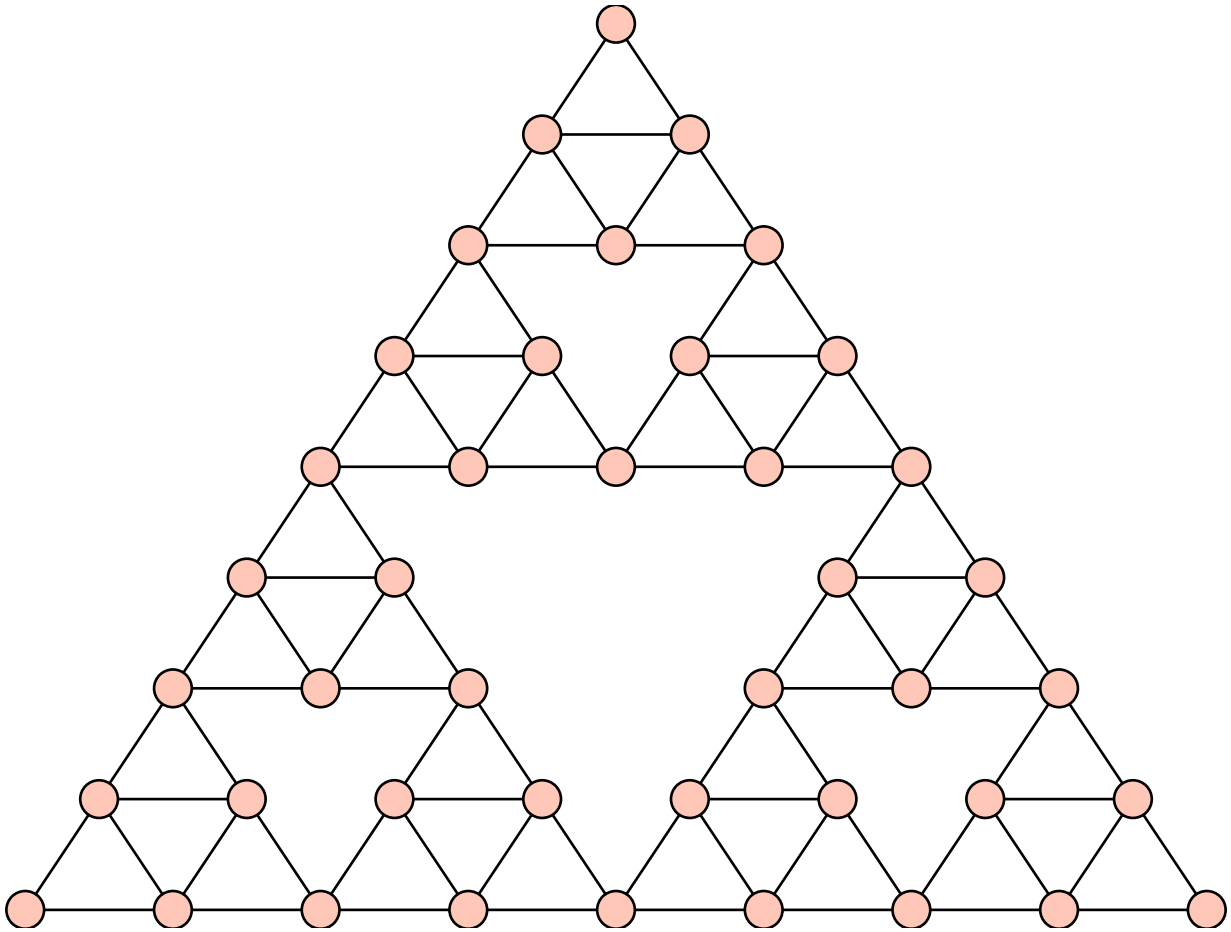
OUTPUT:

a graph S_n with $3(3^{n-1}+1)/2$ vertices and 3^n edges, closely related to the famous Sierpinski triangle fractal.

All these graphs have a triangular shape, and three special vertices at top, bottom left and bottom right. These are the only vertices of valence 2, all the other ones having valence 4.

The graph S_1 (generation 1) is a triangle.

The graph S_{n+1} is obtained from the disjoint union of three copies A,B,C of S_n by identifying pairs of vertices: the top vertex of A with the bottom left vertex of B, the bottom right vertex of B with the top vertex of C, and the bottom left vertex of C with the bottom right vertex of A.



See also

- *HanoiTowerGraph()*. There is another family of graphs called Sierpinski graphs, where all vertices but 3 have valence 3. They are available using `graphs.HanoiTowerGraph(3, n)`.
- *GeneralizedSierpinskiGraph()*

EXAMPLES:

```
sage: s4 = graphs.SierpinskiGasketGraph(4); s4
Graph on 42 vertices
sage: s4.size()
81
sage: s4.degree_histogram()
[0, 0, 3, 0, 39]
sage: s4.is_hamiltonian()
True
```

```
>>> from sage.all import *
>>> s4 = graphs.SierpinskiGasketGraph(Integer(4)); s4
Graph on 42 vertices
>>> s4.size()
81
>>> s4.degree_histogram()
[0, 0, 3, 0, 39]
>>> s4.is_hamiltonian()
True
```

REFERENCES:

[LLWC2011]

sage.graphs.generators.families.**SquaredSkewHadamardMatrixGraph**(*n*, *immutable=False*)

Pseudo-$OA(2n, 4n-1)$-graph from a skew Hadamard matrix of order $4n$.

A strongly regular graph with parameters of the orthogonal array graph *OrthogonalArrayBlockGraph()*, also known as pseudo Latin squares graph $L_{2n}(4n-1)$, constructed from a skew Hadamard matrix of order $4n$, due to Goethals and Seidel, see [BL1984].

See also

- *is_orthogonal_array_block_graph()*

INPUT:

- `n` – integer
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: G = graphs.SquaredSkewHadamardMatrixGraph(4)
sage: G.is_strongly_regular(parameters=True)
(225, 112, 55, 56)
sage: G = graphs.SquaredSkewHadamardMatrixGraph(5)
sage: G.is_strongly_regular(parameters=True)     # long time
(361, 180, 89, 90)
sage: G = graphs.SquaredSkewHadamardMatrixGraph(9)
sage: G.is_strongly_regular(parameters=True)     # not tested
(1225, 612, 305, 306)
```

```
>>> from sage.all import *
>>> G = graphs.SquaredSkewHadamardMatrixGraph(Integer(4))
>>> G.is_strongly_regular(parameters=True)
(225, 112, 55, 56)
>>> G = graphs.SquaredSkewHadamardMatrixGraph(Integer(5))
>>> G.is_strongly_regular(parameters=True)     # long time
(361, 180, 89, 90)
>>> G = graphs.SquaredSkewHadamardMatrixGraph(Integer(9))
>>> G.is_strongly_regular(parameters=True)     # not tested
(1225, 612, 305, 306)
```

sage.graphs.generators.families.**StaircaseGraph**(*n*, *immutable=False*)

Return a staircase graph with $2n$ nodes

For $n \geq 3$, the staircase graph of order $2n$ is the graph obtained from the ladder graph of order $2n-2$, i.e., `graphs.LadderGraph(n - 1)` by introducing two new nodes $2n-2$ and $2n-1$, and then joining the node $2n-2$ with 0 and $n-1$, the node $2n-1$ with $n-2$ and $2n-3$, and the nodes $2n-2$ and $2n-1$ with each other.

Note that `graphs.StaircaseGraph(4)` is also known as the `Bicorn graph`. It is the only brick that has a unique b-invariant edge.

PLOTTING:

Upon construction, the position dictionary is filled to override the spring-layout algorithm. By convention, each staircase graph will be displayed horizontally, with the first $n-1$ nodes displayed from left to right on the top horizontal line, the second $n-1$ nodes displayed from left to right on the middle horizontal line, and the last two nodes displayed at the bottom two corners.

INPUT:

- `n` – an integer at least 3; number of nodes is $2n$
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

OUTPUT:

- `G` – a staircase graph of order $2n$; note that a `ValueError` is returned if $n < 3$

EXAMPLES:

Construct and show a staircase graph with 10 nodes:

```
sage: g = graphs.StaircaseGraph(5)
sage: g.show()                              # long time                          #␣
↪needs sage.plot
```

```
>>> from sage.all import *
>>> g = graphs.StaircaseGraph(Integer(5))
>>> g.show()                              # long time                          #␣
↪needs sage.plot
```

Construct and show the Bicorn graph. Note that the edge $(1, 4)$ is the unique b-invariant edge:

```
sage: bicornGraph = graphs.StaircaseGraph(4)
sage: bicornGraph.show()                    # long time                          #␣
↪needs sage.plot
```

```
>>> from sage.all import *
>>> bicornGraph = graphs.StaircaseGraph(Integer(4))
>>> bicornGraph.show()                    # long time                          #␣
↪needs sage.plot
```

Create several staircase graphs in a Sage graphics array:

```
sage: g = []
sage: j = []
sage: for i in range(9):
....:     k = graphs.StaircaseGraph(i+3)
```

(continues on next page)

(continued from previous page)

```
....:     g.append(k)
sage: for i in range(3):
....:     n = []
....:     for m in range(3):
....:         n.append(g[3*i + m].plot(vertex_size=50 - 4*(3*i+m), vertex_
↪labels=False))
....:     j.append(n)
sage: G = graphics_array(j)
sage: G.show()                              # long time
```

```
>>> from sage.all import *
>>> g = []
>>> j = []
>>> for i in range(Integer(9)):
...     k = graphs.StaircaseGraph(i+Integer(3))
...     g.append(k)
>>> for i in range(Integer(3)):
...     n = []
...     for m in range(Integer(3)):
...         n.append(g[Integer(3)*i + m].plot(vertex_size=Integer(50) -␣
↪Integer(4)*(Integer(3)*i+m), vertex_labels=False))
...     j.append(n)
>>> G = graphics_array(j)
>>> G.show()                              # long time
```

REFERENCES:

- [LM2024]

See also

LadderGraph()

AUTHORS:

- Janmenjaya Panda (2024-06-09)

sage.graphs.generators.families.**SwitchedSquaredSkewHadamardMatrixGraph**(*n*, *immutable=False*)

A strongly regular graph in Seidel switching class of *SquaredSkewHadamardMatrixGraph()*.

A strongly regular graph in the *Seidel switching* class of the disjoint union of a 1-vertex graph and the one produced by *Pseudo-L_{2n}(4n-1)*

In this case, the other possible parameter set of a strongly regular graph in the Seidel switching class of the latter graph (see [BH2012]) coincides with the set of parameters of the complement of the graph returned by this function.

See also

- *is_switch_skewhad()*

INPUT:

- n – integer

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.SwitchedSquaredSkewHadamardMatrixGraph(4)
sage: g.is_strongly_regular(parameters=True)
(226, 105, 48, 49)
sage: from sage.combinat.designs.twographs import twograph_descendant
sage: twograph_descendant(g, 0).is_strongly_regular(parameters=True)
(225, 112, 55, 56)
sage: gc = g.complement()
sage: twograph_descendant(gc, 0).is_strongly_regular(parameters=True)
(225, 112, 55, 56)
```

```
>>> from sage.all import *
>>> g = graphs.SwitchedSquaredSkewHadamardMatrixGraph(Integer(4))
>>> g.is_strongly_regular(parameters=True)
(226, 105, 48, 49)
>>> from sage.combinat.designs.twographs import twograph_descendant
>>> twograph_descendant(g, Integer(0)).is_strongly_regular(parameters=True)
(225, 112, 55, 56)
>>> gc = g.complement()
>>> twograph_descendant(gc, Integer(0)).is_strongly_regular(parameters=True)
(225, 112, 55, 56)
```

sage.graphs.generators.families.**TabacjnGraph**(*n*, *a*, *b*, *r*, *immutable=False*)

Return a Tabačjn graph with $2n$ nodes.

The Tabačjn graphs is a family of pentavalent bicirculants graphs proposed in [AHKOS2014] as a generalization of generalized Petersen graphs. The parameters n, a, b, r are integers such that $n \geq 3$, $1 \leq a, b, r \leq n-1$, with $a \neq b$ and $r \neq n/2$.

INPUT:

- `n` – the number of nodes is $2 * n$
- `a` – integer such that $0 < a < n$ and $a \neq b$, that determines a-spoke edges
- `b` – integer such that $0 < b < n$ and $b \neq a$, that determines b-spoke edges
- `r` – integer such that $0 < r < n$ and $r \neq n/2$ determining how inner vertices are connected
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

PLOTTING: Upon construction, the position dictionary is filled to override the spring-layout algorithm. By convention, the rose window graphs are displayed as an inner and outer cycle pair, with the first n nodes drawn on the outer circle. The first (0) node is drawn at the top of the outer-circle, moving counterclockwise after that. The inner circle is drawn with the (n)th node at the top, then counterclockwise as well. Vertices in the outer circle are connected in the circular manner, vertices in the inner circle are connected when their label have difference $r \pmod n$. Vertices on the outer rim are connected with the vertices on the inner rim when they are at the same position and when they are a and b apart.

EXAMPLES:

```
sage: G = graphs.TabacjnGraph(3, 1, 2, 1)
sage: G.degree()
[5, 5, 5, 5, 5, 5]
sage: G.is_isomorphic(graphs.CompleteGraph(6))
```

(continues on next page)

(continued from previous page)

```
True
sage: G = graphs.TabacjnGraph(6, 1, 5, 2)
sage: I = graphs.IcosahedralGraph()
sage: G.is_isomorphic(I)
True
```

```
>>> from sage.all import *
>>> G = graphs.TabacjnGraph(Integer(3), Integer(1), Integer(2), Integer(1))
>>> G.degree()
[5, 5, 5, 5, 5, 5]
>>> G.is_isomorphic(graphs.CompleteGraph(Integer(6)))
True
>>> G = graphs.TabacjnGraph(Integer(6), Integer(1), Integer(5), Integer(2))
>>> I = graphs.IcosahedralGraph()
>>> G.is_isomorphic(I)
True
```

sage.graphs.generators.families.**TadpoleGraph**(*n1*, *n2*, *immutable=False*)

Return a tadpole graph with $n_1 + n_2$ nodes.

A tadpole graph is a path graph (order n_2) connected to a cycle graph (order n_1).

PLOTTING: Upon construction, the position dictionary is filled to override the spring-layout algorithm. By convention, the cycle graph will be drawn in the lower-left corner with the n_1-th node at a 45 degree angle above the right horizontal center of the cycle graph, leading directly into the path graph.

INPUT:

- `n1` – integer ≥ 3; the order of the cycle graph
- `n2` – integer ≥ 0; the order of the path graph
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

Construct and show a tadpole graph Cycle = 13, Stick = 4:

```
sage: g = graphs.TadpoleGraph(13, 4); g
Tadpole graph: Graph on 17 vertices
sage: g.show()                          # long time                          #␣
↪needs sage.plot
```

```
>>> from sage.all import *
>>> g = graphs.TadpoleGraph(Integer(13), Integer(4)); g
Tadpole graph: Graph on 17 vertices
>>> g.show()                          # long time                          #␣
↪needs sage.plot
```

sage.graphs.generators.families.**TruncatedBiwheelGraph**(*n*, *immutable=False*)

Return a truncated biwheel graph with $2n$ nodes

For $n \geq 3$, the truncated biwheel graph of order $2n$ is the graph obtained from the path graph of order $2n-2$, i.e., `graphs.PathGraph(2*n - 2)` by introducing two new nodes $2n-2$ and $2n-1$, and then joining the node $2n-2$ with the odd indexed nodes up to $2n-3$, joining the node $2n-1$ with the even indexed nodes up to $2n-4$ and adding the edges $(0, 2n-2)$ and $(2n-3, 2n-1)$.

PLOTTING:

Upon construction, the position dictionary is filled to override the spring-layout algorithm. By convention, each truncated biwheel graph will be displayed horizontally, with the first $2n-2$ nodes displayed from left to right on the middle horizontal line and the nodes $2n-2$ and $2n-1$ displayed at the top and the bottom central positions respectively.

INPUT:

- `n` – an integer at least 3; number of nodes is $2n$
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

OUTPUT:

- `G` – a truncated biwheel graph of order $2n$; note that a `ValueError` is returned if $n < 3$

EXAMPLES:

Construct and show a truncated biwheel graph with 10 nodes:

```
sage: g = graphs.TruncatedBiwheelGraph(5)
sage: g.show()                              # long time                          #␣
↪needs sage.plot
```

```
>>> from sage.all import *
>>> g = graphs.TruncatedBiwheelGraph(Integer(5))
>>> g.show()                              # long time                          #␣
↪needs sage.plot
```

Create several truncated biwheel graphs in a Sage graphics array:

```
sage: g = []
sage: j = []
sage: for i in range(9):
....:     k = graphs.TruncatedBiwheelGraph(i+3)
....:     g.append(k)
sage: for i in range(3):
....:     n = []
....:     for m in range(3):
....:         n.append(g[3*i + m].plot(vertex_size=50 - 4*(3*i+m), vertex_
↪labels=False))
....:     j.append(n)
sage: G = graphics_array(j)
sage: G.show()                              # long time
```

```
>>> from sage.all import *
>>> g = []
>>> j = []
>>> for i in range(Integer(9)):
...     k = graphs.TruncatedBiwheelGraph(i+Integer(3))
...     g.append(k)
>>> for i in range(Integer(3)):
...     n = []
...     for m in range(Integer(3)):
...         n.append(g[Integer(3)*i + m].plot(vertex_size=Integer(50) -␣
↪Integer(4)*(Integer(3)*i+m), vertex_labels=False))
```

(continues on next page)

(continued from previous page)

```
...     j.append(n)
>>> G = graphics_array(j)
>>> G.show()                              # long time
```

REFERENCES:

- [LM2024]

See also

WheelGraph(), *BiwheelGraph()*

AUTHORS:

- Janmenjaya Panda (2024-06-09)

sage.graphs.generators.families.**TuranGraph**(*n*, *r*, *immutable=False*)

Return the Turan graph with parameters n, r.

Turan graphs are complete multipartite graphs with n vertices and r subsets, denoted $T(n, r)$, with the property that the sizes of the subsets are as close to equal as possible. The graph $T(n, r)$ will have $n \pmod r$ subsets of size $\lfloor n/r \rfloor$ and $r - (n \pmod r)$ subsets of size $\lceil n/r \rceil$. See the Wikipedia article Turan_graph for more information.

INPUT:

- `n` – integer; the number of vertices in the graph
- `r` – integer; the number of partitions of the graph
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

The Turan graph is a complete multipartite graph:

```
sage: g = graphs.TuranGraph(13, 4)
sage: k = graphs.CompleteMultipartiteGraph([3,3,3,4])
sage: g.is_isomorphic(k)
True
```

```
>>> from sage.all import *
>>> g = graphs.TuranGraph(Integer(13), Integer(4))
>>> k = graphs.CompleteMultipartiteGraph([Integer(3),Integer(3),Integer(3),
↪Integer(4)])
>>> g.is_isomorphic(k)
True
```

The Turan graph $T(n, r)$ has $\frac{(r-1)(n^2-s^2)}{2r} + \frac{s(s-1)}{2}$ edges, where $s = n \pmod r$ (Issue #34249):

```
sage: n = 12
sage: r = 8
sage: g = graphs.TuranGraph(n, r)
sage: def count(n, r):
....:     s = n % r
....:     return (r - 1) * (n**2 - s**2) / (2*r) + s*(s - 1)/2
sage: g.size() == count(n, r)
```

(continues on next page)

(continued from previous page)

```
True
sage: n = randint(3, 100)
sage: r = randint(2, n - 1)
sage: g = graphs.TuranGraph(n, r)
sage: g.size() == count(n, r)
True
```

```
>>> from sage.all import *
>>> n = Integer(12)
>>> r = Integer(8)
>>> g = graphs.TuranGraph(n, r)
>>> def count(n, r):
...     s = n % r
...     return (r - Integer(1)) * (n**Integer(2) - s**Integer(2)) /␣
↪(Integer(2)*r) + s*(s - Integer(1))/Integer(2)
>>> g.size() == count(n, r)
True
>>> n = randint(Integer(3), Integer(100))
>>> r = randint(Integer(2), n - Integer(1))
>>> g = graphs.TuranGraph(n, r)
>>> g.size() == count(n, r)
True
```

sage.graphs.generators.families.**WheelGraph**(*n*, *immutable=False*)

Return a Wheel graph with n nodes.

A Wheel graph is a basic structure where one node is connected to all other nodes and those (outer) nodes are connected cyclically.

PLOTTING: Upon construction, the position dictionary is filled to override the spring-layout algorithm. By convention, each wheel graph will be displayed with the first (0) node in the center, the second node at the top, and the rest following in a counterclockwise manner.

With the wheel graph, we see that it doesn't take a very large n at all for the spring-layout to give a counter-intuitive display. (See Graphics Array examples below).

INPUT:

- `n` – integer; the number of vertices
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

We view many wheel graphs with a Sage Graphics Array, first with this constructor (i.e., the position dictionary filled):

```
sage: g = []
sage: j = []
sage: for i in range(9):
....:     k = graphs.WheelGraph(i+3)
....:     g.append(k)
...
sage: for i in range(3):
....:     n = []
....:     for m in range(3):
```

(continues on next page)

(continued from previous page)

```
....:       n.append(g[3*i + m].plot(vertex_size=50, vertex_labels=False))
....:   j.append(n)
...
sage: G = graphics_array(j)
sage: G.show()                              # long time
```

```
>>> from sage.all import *
>>> g = []
>>> j = []
>>> for i in range(Integer(9)):
...   k = graphs.WheelGraph(i+Integer(3))
...   g.append(k)
...
>>> for i in range(Integer(3)):
...   n = []
...   for m in range(Integer(3)):
...       n.append(g[Integer(3)*i + m].plot(vertex_size=Integer(50), vertex_
↪labels=False))
...   j.append(n)
...
>>> G = graphics_array(j)
>>> G.show()                              # long time
```

Next, using the spring-layout algorithm:

```
sage: # needs networkx sage.plot
sage: import networkx
sage: g = []
sage: j = []
sage: for i in range(9):
....:   spr = networkx.wheel_graph(i+3)
....:   k = Graph(spr)
....:   g.append(k)
...
sage: for i in range(3):
....:   n = []
....:   for m in range(3):
....:       n.append(g[3*i + m].plot(vertex_size=50, vertex_labels=False))
....:   j.append(n)
...
sage: G = graphics_array(j)
sage: G.show()                              # long time
```

```
>>> from sage.all import *
>>> # needs networkx sage.plot
>>> import networkx
>>> g = []
>>> j = []
>>> for i in range(Integer(9)):
...   spr = networkx.wheel_graph(i+Integer(3))
...   k = Graph(spr)
...   g.append(k)
```

(continues on next page)

(continued from previous page)

```
...
>>> for i in range(Integer(3)):
...  n = []
...  for m in range(Integer(3)):
...      n.append(g[Integer(3)*i + m].plot(vertex_size=Integer(50), vertex_
↪labels=False))
...  j.append(n)
...
>>> G = graphics_array(j)
>>> G.show()                          # long time
```

Compare the plotting:

```
sage: # needs networkx sage.plot
sage: n = networkx.wheel_graph(23)
sage: spring23 = Graph(n)
sage: posdict23 = graphs.WheelGraph(23)
sage: spring23.show()                    # long time
sage: posdict23.show()  # long time
```

```
>>> from sage.all import *
>>> # needs networkx sage.plot
>>> n = networkx.wheel_graph(Integer(23))
>>> spring23 = Graph(n)
>>> posdict23 = graphs.WheelGraph(Integer(23))
>>> spring23.show()                    # long time
>>> posdict23.show()  # long time
```

sage.graphs.generators.families.**WindmillGraph**(*k*, *n*, *immutable=False*)

Return the Windmill graph $Wd(k, n)$.

The windmill graph $Wd(k, n)$ is an undirected graph constructed for $k \geq 2$ and $n \geq 2$ by joining n copies of the complete graph K_k at a shared vertex. It has $(k-1)n+1$ vertices and $nk(k-1)/2$ edges, girth 3 (if $k > 2$), radius 1 and diameter 2. It has vertex connectivity 1 because its central vertex is an articulation point; however, like the complete graphs from which it is formed, it is $(k-1)$-edge-connected. It is trivially perfect and a block graph.

See also

- Wikipedia article Windmill_graph
- *StarGraph()*
- *FriendshipGraph()*

INPUT:

- `k` – integer; the number of vertices of the complete graphs
- `n` – integer; the number of copies of the complete graph
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

The Windmill graph $Wd(2, n)$ is a star graph:

```
sage: n = 5
sage: W = graphs.WindmillGraph(2, n)
sage: W.is_isomorphic( graphs.StarGraph(n) )
True
```

```
>>> from sage.all import *
>>> n = Integer(5)
>>> W = graphs.WindmillGraph(Integer(2), n)
>>> W.is_isomorphic( graphs.StarGraph(n) )
True
```

The Windmill graph $Wd(3, n)$ is the Friendship graph F_n:

```
sage: n = 5
sage: W = graphs.WindmillGraph(3, n)
sage: W.is_isomorphic( graphs.FriendshipGraph(n) )
True
```

```
>>> from sage.all import *
>>> n = Integer(5)
>>> W = graphs.WindmillGraph(Integer(3), n)
>>> W.is_isomorphic( graphs.FriendshipGraph(n) )
True
```

The Windmill graph $Wd(3, 2)$ is the Butterfly graph:

```
sage: W = graphs.WindmillGraph(3, 2)
sage: W.is_isomorphic( graphs.ButterflyGraph() )
True
```

```
>>> from sage.all import *
>>> W = graphs.WindmillGraph(Integer(3), Integer(2))
>>> W.is_isomorphic( graphs.ButterflyGraph() )
True
```

The Windmill graph $Wd(k, n)$ has chromatic number k:

```
sage: n,k = 5,6
sage: W = graphs.WindmillGraph(k, n)
sage: W.chromatic_number() == k
True
```

```
>>> from sage.all import *
>>> n,k = Integer(5),Integer(6)
>>> W = graphs.WindmillGraph(k, n)
>>> W.chromatic_number() == k
True
```

sage.graphs.generators.families.**chang_graphs**(*immutable=False*)

Return the three Chang graphs.

Three of the four strongly regular graphs of parameters $(28, 12, 6, 4)$ are called the Chang graphs. The fourth is the line graph of K_8. For more information about the Chang graphs, see the Wikipedia article Chang_graphs or https://www.win.tue.nl/~aeb/graphs/Chang.html.

INPUT:

- `immutable` – boolean (default: `False`); whether to return immutable or mutable graphs

EXAMPLES: check that we get 4 non-isomorphic s.r.g.'s with the same parameters:

```
sage: chang_graphs = graphs.chang_graphs()
sage: K8 = graphs.CompleteGraph(8)
sage: T8 = K8.line_graph()
sage: four_srg = chang_graphs + [T8]
sage: for g in four_srg:
....:     print(g.is_strongly_regular(parameters=True))
(28, 12, 6, 4)
(28, 12, 6, 4)
(28, 12, 6, 4)
(28, 12, 6, 4)
sage: from itertools import combinations
sage: for g1,g2 in combinations(four_srg,2):
....:     assert not g1.is_isomorphic(g2)
```

```
>>> from sage.all import *
>>> chang_graphs = graphs.chang_graphs()
>>> K8 = graphs.CompleteGraph(Integer(8))
>>> T8 = K8.line_graph()
>>> four_srg = chang_graphs + [T8]
>>> for g in four_srg:
...     print(g.is_strongly_regular(parameters=True))
(28, 12, 6, 4)
(28, 12, 6, 4)
(28, 12, 6, 4)
(28, 12, 6, 4)
>>> from itertools import combinations
>>> for g1,g2 in combinations(four_srg,Integer(2)):
...     assert not g1.is_isomorphic(g2)
```

Construct the Chang graphs by Seidel switching:

```
sage: c3c5 = graphs.CycleGraph(3).disjoint_union(graphs.CycleGraph(5))
sage: c8 = graphs.CycleGraph(8)
sage: s = [K8.subgraph_search(c8).edges(sort=False),                    #␣
↪needs sage.modules
....:      [(0,1,None),(2,3,None),(4,5,None),(6,7,None)],
....:      K8.subgraph_search(c3c5).edges(sort=False)]
sage: [T8.seidel_switching(x, inplace=False).is_isomorphic(G)           #␣
↪needs sage.modules
....:  for x, G in zip(s, chang_graphs)]
[True, True, True]
```

```
>>> from sage.all import *
>>> c3c5 = graphs.CycleGraph(Integer(3)).disjoint_union(graphs.
↪CycleGraph(Integer(5)))
```

(continues on next page)

(continued from previous page)

```
>>> c8 = graphs.CycleGraph(Integer(8))
>>> s = [K8.subgraph_search(c8).edges(sort=False),                          #␣
↪needs sage.modules
...      [(Integer(0),Integer(1),None),(Integer(2),Integer(3),None),(Integer(4),
↪Integer(5),None),(Integer(6),Integer(7),None)],
...      K8.subgraph_search(c3c5).edges(sort=False)]
>>> [T8.seidel_switching(x, inplace=False).is_isomorphic(G)                 #␣
↪needs sage.modules
...  for x, G in zip(s, chang_graphs)]
[True, True, True]
```

sage.graphs.generators.families.**line_graph_forbidden_subgraphs**(*immutable=False*)

Return the 9 forbidden subgraphs of a line graph.

See the Wikipedia article Line_graph for more information.

The graphs are returned in the ordering given by the Wikipedia drawing, read from left to right and from top to bottom.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: graphs.line_graph_forbidden_subgraphs()
[Claw graph: Graph on 4 vertices,
Graph on 6 vertices,
Graph on 6 vertices,
Graph on 5 vertices,
Graph on 6 vertices,
Graph on 6 vertices,
Graph on 6 vertices,
Graph on 6 vertices,
Graph on 5 vertices]
```

```
>>> from sage.all import *
>>> graphs.line_graph_forbidden_subgraphs()
[Claw graph: Graph on 4 vertices,
Graph on 6 vertices,
Graph on 6 vertices,
Graph on 5 vertices,
Graph on 6 vertices,
Graph on 6 vertices,
Graph on 6 vertices,
Graph on 6 vertices,
Graph on 5 vertices]
```

sage.graphs.generators.families.**p2_forbidden_minors**(*immutable=False*)

Return an array containing the 35 minimal forbidden excluded minors of the projective plane.

We constructed the graphs given in Theorem 6.5.1 of [MT2001], which is a result of Archdeacon and encoded them in graph6 format. The order of the graphs is the same as they appear in [WA2025].

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

`sage.graphs.generators.families.`**`petersen_family`**(*generate=False*, *immutable=False*)

Return the Petersen family.

The Petersen family is a collection of 7 graphs which are the forbidden minors of the linklessly embeddable graphs. For more information see the Wikipedia article Petersen_family.

INPUT:

- `generate` – boolean; whether to generate the family from the $\Delta - Y$ transformations. When set to `False` (default) a hardcoded version of the graphs (with a prettier layout) is returned.
- `immutable` – boolean (default: `False`); whether to return immutable or mutable graphs

EXAMPLES:

```
sage: graphs.petersen_family()
[Petersen graph: Graph on 10 vertices,
 Complete graph: Graph on 6 vertices,
 Multipartite Graph with set sizes [3, 3, 1]: Graph on 7 vertices,
 Graph on 8 vertices,
 Graph on 9 vertices,
 Graph on 7 vertices,
 Graph on 8 vertices]
```

```
>>> from sage.all import *
>>> graphs.petersen_family()
[Petersen graph: Graph on 10 vertices,
 Complete graph: Graph on 6 vertices,
 Multipartite Graph with set sizes [3, 3, 1]: Graph on 7 vertices,
 Graph on 8 vertices,
 Graph on 9 vertices,
 Graph on 7 vertices,
 Graph on 8 vertices]
```

The two different inputs generate the same graphs:

```
sage: F1 = graphs.petersen_family(generate=False)
sage: F2 = graphs.petersen_family(generate=True)                    #␣
↪needs sage.modules
sage: F1 = [g.canonical_label().graph6_string() for g in F1]
sage: F2 = [g.canonical_label().graph6_string() for g in F2]        #␣
↪needs sage.modules
sage: set(F1) == set(F2)                                            #␣
↪needs sage.modules
True
```

```
>>> from sage.all import *
>>> F1 = graphs.petersen_family(generate=False)
>>> F2 = graphs.petersen_family(generate=True)                      #␣
↪needs sage.modules
>>> F1 = [g.canonical_label().graph6_string() for g in F1]
>>> F2 = [g.canonical_label().graph6_string() for g in F2]          #␣
↪needs sage.modules
>>> set(F1) == set(F2)                                              #␣
```

(continues on next page)

(continued from previous page)

```
↪needs sage.modules
True
```

2.13 Intersection graphs

The methods defined here appear in *sage.graphs.graph_generators*.

sage.graphs.generators.intersection.**IntersectionGraph**(*S*, *immutable=False*)

Return the intersection graph of the family S.

The intersection graph of a family S is a graph G with $V(G) = S$ such that two elements $s_1, s_2 \in S$ are adjacent in G if and only if $s_1 \cap s_2 \neq \emptyset$.

INPUT:

- `S` – list of sets/tuples/iterables

> **Note**
>
> The elements of S must be finite, hashable, and the elements of any $s \in S$ must be hashable too.

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: graphs.IntersectionGraph([(1,2,3),(3,4,5),(5,6,7)])
Intersection Graph: Graph on 3 vertices
```

```
>>> from sage.all import *
>>> graphs.IntersectionGraph([(Integer(1),Integer(2),Integer(3)),(Integer(3),
↪Integer(4),Integer(5)),(Integer(5),Integer(6),Integer(7))])
Intersection Graph: Graph on 3 vertices
```

sage.graphs.generators.intersection.**IntervalGraph**(*intervals*, *points_ordered=False*, *immutable=False*)

Return the graph corresponding to the given intervals.

An interval graph is built from a list $(a_i, b_i)_{1 \leq i \leq n}$ of intervals : to each interval of the list is associated one vertex, two vertices being adjacent if the two corresponding (closed) intervals intersect.

INPUT:

- `intervals` – the list of pairs (a_i, b_i) defining the graph
- `points_ordered` – states whether every interval (a_i, b_i) of $intervals$ satisfies $a_i < b_i$. If satisfied then setting `points_ordered` to `True` will speed up the creation of the graph.
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

Note

- The vertices are named 0, 1, 2, and so on. The intervals used to create the graph are saved with the graph and can be recovered using `get_vertex()` or `get_vertices()`.

EXAMPLES:

The following line creates the sequence of intervals $(i, i+2)$ for i in $[0, ..., 8]$:

```
sage: intervals = [(i,i+2) for i in range(9)]
```

```
>>> from sage.all import *
>>> intervals = [(i,i+Integer(2)) for i in range(Integer(9))]
```

In the corresponding graph

```
sage: g = graphs.IntervalGraph(intervals)
sage: g.get_vertex(3)
(3, 5)
sage: neigh = g.neighbors(3)
sage: for v in neigh: print(g.get_vertex(v))
(1, 3)
(2, 4)
(4, 6)
(5, 7)
```

```
>>> from sage.all import *
>>> g = graphs.IntervalGraph(intervals)
>>> g.get_vertex(Integer(3))
(3, 5)
>>> neigh = g.neighbors(Integer(3))
>>> for v in neigh: print(g.get_vertex(v))
(1, 3)
(2, 4)
(4, 6)
(5, 7)
```

The is_interval() method verifies that this graph is an interval graph.

```
sage: g.is_interval()
True
```

```
>>> from sage.all import *
>>> g.is_interval()
True
```

The intervals in the list need not be distinct.

```
sage: intervals = [ (1,2), (1,2), (1,2), (2,3), (3,4) ]
sage: g = graphs.IntervalGraph(intervals,True)
sage: g.clique_maximum()
[0, 1, 2, 3]
```

(continues on next page)

(continued from previous page)

```
sage: g.get_vertices()
{0: (1, 2), 1: (1, 2), 2: (1, 2), 3: (2, 3), 4: (3, 4)}
```

```
>>> from sage.all import *
>>> intervals = [ (Integer(1),Integer(2)), (Integer(1),Integer(2)), (Integer(1),
↪Integer(2)), (Integer(2),Integer(3)), (Integer(3),Integer(4)) ]
>>> g = graphs.IntervalGraph(intervals,True)
>>> g.clique_maximum()
[0, 1, 2, 3]
>>> g.get_vertices()
{0: (1, 2), 1: (1, 2), 2: (1, 2), 3: (2, 3), 4: (3, 4)}
```

The endpoints of the intervals are not ordered we get the same graph (except for the vertex labels).

```
sage: rev_intervals = [ (2,1), (2,1), (2,1), (3,2), (4,3) ]
sage: h = graphs.IntervalGraph(rev_intervals,False)
sage: h.get_vertices()
{0: (2, 1), 1: (2, 1), 2: (2, 1), 3: (3, 2), 4: (4, 3)}
sage: g.edges(sort=True) == h.edges(sort=True)
True
```

```
>>> from sage.all import *
>>> rev_intervals = [ (Integer(2),Integer(1)), (Integer(2),Integer(1)),␣
↪(Integer(2),Integer(1)), (Integer(3),Integer(2)), (Integer(4),Integer(3)) ]
>>> h = graphs.IntervalGraph(rev_intervals,False)
>>> h.get_vertices()
{0: (2, 1), 1: (2, 1), 2: (2, 1), 3: (3, 2), 4: (4, 3)}
>>> g.edges(sort=True) == h.edges(sort=True)
True
```

sage.graphs.generators.intersection.**OrthogonalArrayBlockGraph**(*k*, *n*, *OA=None*, *immutable=False*)

Return the graph of an $OA(k,n)$.

The intersection graph of the blocks of a transversal design with parameters (k,n), or $TD(k,n)$ for short, is a strongly regular graph (unless it is a complete graph). Its parameters (v,k',λ,μ) are determined by the parameters k,n via:

$$v=n^2, k'=k(n-1), \lambda=(k-1)(k-2)+n-2, \mu=k(k-1)$$

As transversal designs and orthogonal arrays (OA for short) are equivalent objects, this graph can also be built from the blocks of an $OA(k,n)$, two of them being adjacent if one of their coordinates match.

For more information on these graphs, see Andries Brouwer's page on Orthogonal Array graphs.

Warning

- Brouwer's website uses the notation $OA(n,k)$ instead of $OA(k,n)$
- For given parameters k and n there can be many $OA(k,n)$: the graphs returned are not uniquely defined by their parameters (see the examples below).

- If the function is called only with the parameter `k` and `n` the results might be different with two versions of Sage, or even worse : some could not be available anymore.

See also

`sage.combinat.designs.orthogonal_arrays`

INPUT:

- `k`, `n` – integers
- `OA` – an orthogonal array; if set to `None` (default) then `orthogonal_array()` is called to compute an $OA(k,n)$
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: G = graphs.OrthogonalArrayBlockGraph(5,5); G                              #␣
↪needs sage.schemes
OA(5,5): Graph on 25 vertices
sage: G.is_strongly_regular(parameters=True)                                    #␣
↪needs sage.schemes
(25, 20, 15, 20)
sage: G = graphs.OrthogonalArrayBlockGraph(4,10); G
OA(4,10): Graph on 100 vertices
sage: G.is_strongly_regular(parameters=True)
(100, 36, 14, 12)
```

```
>>> from sage.all import *
>>> G = graphs.OrthogonalArrayBlockGraph(Integer(5),Integer(5)); G              ␣
↪              # needs sage.schemes
OA(5,5): Graph on 25 vertices
>>> G.is_strongly_regular(parameters=True)                                     #␣
↪needs sage.schemes
(25, 20, 15, 20)
>>> G = graphs.OrthogonalArrayBlockGraph(Integer(4),Integer(10)); G
OA(4,10): Graph on 100 vertices
>>> G.is_strongly_regular(parameters=True)
(100, 36, 14, 12)
```

Two graphs built from different orthogonal arrays are also different:

```
sage: k = 4; n = 10
sage: OAa = designs.orthogonal_arrays.build(k,n)
sage: OAb = [[(x+1)%n for x in R] for R in OAa]
sage: set(map(tuple,OAa)) == set(map(tuple,OAb))
False
sage: Ga = graphs.OrthogonalArrayBlockGraph(k, n, OAa)
sage: Gb = graphs.OrthogonalArrayBlockGraph(k, n, OAb)
sage: Ga == Gb
False
```

```
>>> from sage.all import *
>>> k = Integer(4); n = Integer(10)
>>> OAa = designs.orthogonal_arrays.build(k,n)
>>> OAb = [[(x+Integer(1))%n for x in R] for R in OAa]
>>> set(map(tuple,OAa)) == set(map(tuple,OAb))
False
>>> Ga = graphs.OrthogonalArrayBlockGraph(k, n, OAa)
>>> Gb = graphs.OrthogonalArrayBlockGraph(k, n, OAb)
>>> Ga == Gb
False
```

As `OAb` was obtained from `OAa` by a relabelling the two graphs are isomorphic:

```
sage: Ga.is_isomorphic(Gb)                                                  #␣
↪needs sage.modules
True
```

```
>>> from sage.all import *
>>> Ga.is_isomorphic(Gb)                                                    #␣
↪needs sage.modules
True
```

But there are examples of $OA(k, n)$ for which the resulting graphs are not isomorphic:

```
sage: oa0 = [[0, 0, 1], [0, 1, 3], [0, 2, 0], [0, 3, 2],
....:        [1, 0, 3], [1, 1, 1], [1, 2, 2], [1, 3, 0],
....:        [2, 0, 0], [2, 1, 2], [2, 2, 1], [2, 3, 3],
....:        [3, 0, 2], [3, 1, 0], [3, 2, 3], [3, 3, 1]]
sage: oa1 = [[0, 0, 1], [0, 1, 0], [0, 2, 3], [0, 3, 2],
....:        [1, 0, 3], [1, 1, 2], [1, 2, 0], [1, 3, 1],
....:        [2, 0, 0], [2, 1, 1], [2, 2, 2], [2, 3, 3],
....:        [3, 0, 2], [3, 1, 3], [3, 2, 1], [3, 3, 0]]
sage: g0 = graphs.OrthogonalArrayBlockGraph(3, 4, oa0)                      #␣
↪needs sage.modules
sage: g1 = graphs.OrthogonalArrayBlockGraph(3, 4, oa1)                      #␣
↪needs sage.modules
sage: g0.is_isomorphic(g1)                                                  #␣
↪needs sage.modules
False
```

```
>>> from sage.all import *
>>> oa0 = [[Integer(0), Integer(0), Integer(1)], [Integer(0), Integer(1),␣
↪Integer(3)], [Integer(0), Integer(2), Integer(0)], [Integer(0), Integer(3),␣
↪Integer(2)],
...         [Integer(1), Integer(0), Integer(3)], [Integer(1), Integer(1),␣
↪Integer(1)], [Integer(1), Integer(2), Integer(2)], [Integer(1), Integer(3),␣
↪Integer(0)],
...         [Integer(2), Integer(0), Integer(0)], [Integer(2), Integer(1),␣
↪Integer(2)], [Integer(2), Integer(2), Integer(1)], [Integer(2), Integer(3),␣
↪Integer(3)],
...         [Integer(3), Integer(0), Integer(2)], [Integer(3), Integer(1),␣
↪Integer(0)], [Integer(3), Integer(2), Integer(3)], [Integer(3), Integer(3),␣
↪Integer(1)]]
```

(continues on next page)

(continued from previous page)

```
>>> oa1 = [[Integer(0), Integer(0), Integer(1)], [Integer(0), Integer(1),␣
↪Integer(0)], [Integer(0), Integer(2), Integer(3)], [Integer(0), Integer(3),␣
↪Integer(2)],
...         [Integer(1), Integer(0), Integer(3)], [Integer(1), Integer(1),␣
↪Integer(2)], [Integer(1), Integer(2), Integer(0)], [Integer(1), Integer(3),␣
↪Integer(1)],
...         [Integer(2), Integer(0), Integer(0)], [Integer(2), Integer(1),␣
↪Integer(1)], [Integer(2), Integer(2), Integer(2)], [Integer(2), Integer(3),␣
↪Integer(3)],
...         [Integer(3), Integer(0), Integer(2)], [Integer(3), Integer(1),␣
↪Integer(3)], [Integer(3), Integer(2), Integer(1)], [Integer(3), Integer(3),␣
↪Integer(0)]]
>>> g0 = graphs.OrthogonalArrayBlockGraph(Integer(3), Integer(4), oa0)           ␣
↪              # needs sage.modules
>>> g1 = graphs.OrthogonalArrayBlockGraph(Integer(3), Integer(4), oa1)           ␣
↪              # needs sage.modules
>>> g0.is_isomorphic(g1)                                                        #␣
↪needs sage.modules
False
```

But nevertheless isospectral:

```
sage: g0.spectrum()                                                             #␣
↪needs sage.modules sage.rings.number_field
[9, 1, 1, 1, 1, 1, 1, 1, 1, 1, -3, -3, -3, -3, -3, -3]
sage: g1.spectrum()                                                             #␣
↪needs sage.modules sage.rings.number_field
[9, 1, 1, 1, 1, 1, 1, 1, 1, 1, -3, -3, -3, -3, -3, -3]
```

```
>>> from sage.all import *
>>> g0.spectrum()                                                               #␣
↪needs sage.modules sage.rings.number_field
[9, 1, 1, 1, 1, 1, 1, 1, 1, 1, -3, -3, -3, -3, -3, -3]
>>> g1.spectrum()                                                               #␣
↪needs sage.modules sage.rings.number_field
[9, 1, 1, 1, 1, 1, 1, 1, 1, 1, -3, -3, -3, -3, -3, -3]
```

Note that the graph `g0` is actually isomorphic to the affine polar graph $VO^+(4,2)$:

```
sage: graphs.AffineOrthogonalPolarGraph(4,2,'+').is_isomorphic(g0)              #␣
↪needs sage.libs.gap sage.modules
True
```

```
>>> from sage.all import *
>>> graphs.AffineOrthogonalPolarGraph(Integer(4),Integer(2),'+').is_
↪isomorphic(g0)              # needs sage.libs.gap sage.modules
True
```

sage.graphs.generators.intersection.**PermutationGraph**(*second_permutation*, *first_permutation=None*, *immutable=False*)

Build a permutation graph from one permutation or from two lists.

Definition:

If σ is a permutation of $\{1, 2, \ldots, n\}$, then the permutation graph of σ is the graph on vertex set $\{1, 2, \ldots, n\}$ in which two vertices i and j satisfying $i < j$ are connected by an edge if and only if $\sigma^{-1}(i) > \sigma^{-1}(j)$. A visual way to construct this graph is as follows:

> Take two horizontal lines in the euclidean plane, and mark points $1, ..., n$ from left to right on the first of them. On the second one, still from left to right, mark n points $\sigma(1), \sigma(2), \ldots, \sigma(n)$. Now, link by a segment the two points marked with 1, then link together the points marked with 2, and so on. The permutation graph of σ is the intersection graph of those segments: there exists a vertex in this graph for each element from 1 to n, two vertices i, j being adjacent if the segments i and j cross each other.

The set of edges of the permutation graph can thus be identified with the set of inversions of the inverse of the given permutation σ.

A more general notion of permutation graph can be defined as follows: If S is a set, and $(a_1, a_2, \ldots, a_n)$ and $(b_1, b_2, \ldots, b_n)$ are two lists of elements of S, each of which lists contains every element of S exactly once, then the permutation graph defined by these two lists is the graph on the vertex set S in which two vertices i and j are connected by an edge if and only if the order in which these vertices appear in the list $(a_1, a_2, \ldots, a_n)$ is the opposite of the order in which they appear in the list $(b_1, b_2, \ldots, b_n)$. When $(a_1, a_2, \ldots, a_n) = (1, 2, \ldots, n)$, this graph is the permutation graph of the permutation $(b_1, b_2, \ldots, b_n) \in S_n$. Notice that S does not have to be a set of integers here, but can be a set of strings, tuples, or anything else. We can still use the above visual description to construct the permutation graph, but now we have to mark points $a_1, a_2, \ldots, a_n$ from left to right on the first horizontal line and points $b_1, b_2, \ldots, b_n$ from left to right on the second horizontal line.

INPUT:

- `second_permutation` – the unique permutation/list defining the graph, or the second of the two (if the graph is to be built from two permutations/lists)
- `first_permutation` – (optional) the first of the two permutations/lists from which the graph should be built, if it is to be built from two permutations/lists

  When `first_permutation is None` (default), it is set to be equal to `sorted(second_permutation)`, which yields the expected ordering when the elements of the graph are integers.
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

See also

- Recognition of Permutation graphs in the *comparability module*.
- Drawings of permutation graphs as intersection graphs of segments is possible through the `show()` method of `Permutation` objects.

  The correct argument to use in this case is `show(representation = "braid")`.
- `inversions()`

EXAMPLES:

```
sage: p = Permutations(5).random_element()
sage: PG = graphs.PermutationGraph(p)
sage: edges = PG.edges(sort=True, labels=False)
sage: set(edges) == set(p.inverse().inversions())
True

sage: PG = graphs.PermutationGraph([3,4,5,1,2])
sage: sorted(PG.edges(sort=True))
```

(continues on next page)

(continued from previous page)

```
[(1, 3, None),
 (1, 4, None),
 (1, 5, None),
 (2, 3, None),
 (2, 4, None),
 (2, 5, None)]
sage: PG = graphs.PermutationGraph([3,4,5,1,2], [1,4,2,5,3])
sage: sorted(PG.edges(sort=True))
[(1, 3, None),
 (1, 4, None),
 (1, 5, None),
 (2, 3, None),
 (2, 5, None),
 (3, 4, None),
 (3, 5, None)]
sage: PG = graphs.PermutationGraph([1,4,2,5,3], [3,4,5,1,2])
sage: sorted(PG.edges(sort=True))
[(1, 3, None),
 (1, 4, None),
 (1, 5, None),
 (2, 3, None),
 (2, 5, None),
 (3, 4, None),
 (3, 5, None)]

sage: PG = graphs.PermutationGraph(Permutation([1,3,2]), Permutation([1,2,3]))
sage: sorted(PG.edges(sort=True))
[(2, 3, None)]

sage: graphs.PermutationGraph([]).edges(sort=True)
[]
sage: graphs.PermutationGraph([], []).edges(sort=True)
[]

sage: PG = graphs.PermutationGraph("graph", "phrag")
sage: sorted(PG.edges(sort=True))
[('a', 'g', None),
 ('a', 'h', None),
 ('a', 'p', None),
 ('g', 'h', None),
 ('g', 'p', None),
 ('g', 'r', None),
 ('h', 'r', None),
 ('p', 'r', None)]
```

```
>>> from sage.all import *
>>> p = Permutations(Integer(5)).random_element()
>>> PG = graphs.PermutationGraph(p)
>>> edges = PG.edges(sort=True, labels=False)
>>> set(edges) == set(p.inverse().inversions())
True
```

(continues on next page)

(continued from previous page)

```
>>> PG = graphs.PermutationGraph([Integer(3),Integer(4),Integer(5),Integer(1),
↪Integer(2)])
>>> sorted(PG.edges(sort=True))
[(1, 3, None),
 (1, 4, None),
 (1, 5, None),
 (2, 3, None),
 (2, 4, None),
 (2, 5, None)]
>>> PG = graphs.PermutationGraph([Integer(3),Integer(4),Integer(5),Integer(1),
↪Integer(2)], [Integer(1),Integer(4),Integer(2),Integer(5),Integer(3)])
>>> sorted(PG.edges(sort=True))
[(1, 3, None),
 (1, 4, None),
 (1, 5, None),
 (2, 3, None),
 (2, 5, None),
 (3, 4, None),
 (3, 5, None)]
>>> PG = graphs.PermutationGraph([Integer(1),Integer(4),Integer(2),Integer(5),
↪Integer(3)], [Integer(3),Integer(4),Integer(5),Integer(1),Integer(2)])
>>> sorted(PG.edges(sort=True))
[(1, 3, None),
 (1, 4, None),
 (1, 5, None),
 (2, 3, None),
 (2, 5, None),
 (3, 4, None),
 (3, 5, None)]

>>> PG = graphs.PermutationGraph(Permutation([Integer(1),Integer(3),Integer(2)]),␣
↪Permutation([Integer(1),Integer(2),Integer(3)]))
>>> sorted(PG.edges(sort=True))
[(2, 3, None)]

>>> graphs.PermutationGraph([]).edges(sort=True)
[]
>>> graphs.PermutationGraph([], []).edges(sort=True)
[]

>>> PG = graphs.PermutationGraph("graph", "phrag")
>>> sorted(PG.edges(sort=True))
[('a', 'g', None),
 ('a', 'h', None),
 ('a', 'p', None),
 ('g', 'h', None),
 ('g', 'p', None),
 ('g', 'r', None),
 ('h', 'r', None),
 ('p', 'r', None)]
```

sage.graphs.generators.intersection.**ToleranceGraph**(*tolrep*, *immutable=False*, *name=None*)

Return the graph generated by the tolerance representation `tolrep`.

The tolerance representation `tolrep` is described by the list $((l_0, r_0, t_0), (l_1, r_1, t_1), \ldots, (l_k, r_k, t_k))$ where $I_i = (l_i, r_i)$ denotes a closed interval on the real line with $l_i < r_i$ and t_i a strictly positive value, called tolerance. This representation generates the tolerance graph with the vertex set $\{0, 1, \ldots, k\}$ and the edge set $\{(i, j) : |I_i \cap I_j| \geq \min\{t_i, t_j\}\}$ where $|I_i \cap I_j|$ denotes the length of the intersection of I_i and I_j.

INPUT:

- `tolrep` – list of triples (l_i, r_i, t_i) where (l_i, r_i) denotes a closed interval on the real line and t_i a positive value
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph
- `name` – string (default: `None`); used as the name of the returned graph when set

Note

The vertices are named $0, 1, \ldots, k$. The tolerance representation used to create the graph is saved with the graph and can be recovered using `get_vertex()` or `get_vertices()`.

EXAMPLES:

The following code creates a tolerance representation `tolrep`, generates its tolerance graph `g`, and applies some checks:

```
sage: tolrep = [(1,4,3),(1,2,1),(2,3,1),(0,3,3)]
sage: g = graphs.ToleranceGraph(tolrep)
sage: g.get_vertex(3)
(0, 3, 3)
sage: neigh = g.neighbors(3)
sage: for v in neigh: print(g.get_vertex(v))
(1, 2, 1)
(2, 3, 1)
sage: g.is_interval()
False
sage: g.is_weakly_chordal()
True
```

```
>>> from sage.all import *
>>> tolrep = [(Integer(1),Integer(4),Integer(3)),(Integer(1),Integer(2),
↪Integer(1)),(Integer(2),Integer(3),Integer(1)),(Integer(0),Integer(3),
↪Integer(3))]
>>> g = graphs.ToleranceGraph(tolrep)
>>> g.get_vertex(Integer(3))
(0, 3, 3)
>>> neigh = g.neighbors(Integer(3))
>>> for v in neigh: print(g.get_vertex(v))
(1, 2, 1)
(2, 3, 1)
>>> g.is_interval()
False
>>> g.is_weakly_chordal()
True
```

The intervals in the list need not be distinct

```
sage: tolrep2 = [(0,4,5),(1,2,1),(2,3,1),(0,4,5)]
sage: g2 = graphs.ToleranceGraph(tolrep2)
sage: g2.get_vertices()
{0: (0, 4, 5), 1: (1, 2, 1), 2: (2, 3, 1), 3: (0, 4, 5)}
sage: g2.is_isomorphic(g)
True
```

```
>>> from sage.all import *
>>> tolrep2 = [(Integer(0),Integer(4),Integer(5)),(Integer(1),Integer(2),
↪Integer(1)),(Integer(2),Integer(3),Integer(1)),(Integer(0),Integer(4),
↪Integer(5))]
>>> g2 = graphs.ToleranceGraph(tolrep2)
>>> g2.get_vertices()
{0: (0, 4, 5), 1: (1, 2, 1), 2: (2, 3, 1), 3: (0, 4, 5)}
>>> g2.is_isomorphic(g)
True
```

Real values are also allowed

```
sage: tolrep = [(0.1,3.3,4.4),(1.1,2.5,1.1),(1.4,4.4,3.3)]
sage: g = graphs.ToleranceGraph(tolrep)
sage: g.is_isomorphic(graphs.PathGraph(3))
True
```

```
>>> from sage.all import *
>>> tolrep = [(RealNumber('0.1'),RealNumber('3.3'),RealNumber('4.4')),(RealNumber(
↪'1.1'),RealNumber('2.5'),RealNumber('1.1')),(RealNumber('1.4'),RealNumber('4.4
↪'),RealNumber('3.3'))]
>>> g = graphs.ToleranceGraph(tolrep)
>>> g.is_isomorphic(graphs.PathGraph(Integer(3)))
True
```

2.14 1-skeletons of Platonic solids

The methods defined here appear in *sage.graphs.graph_generators*.

sage.graphs.generators.platonic_solids.**DodecahedralGraph**(*immutable=False*)

Return a Dodecahedral graph (with 20 nodes).

The dodecahedral graph is cubic symmetric, so the spring-layout algorithm will be very effective for display. It is dual to the icosahedral graph.

PLOTTING: The Dodecahedral graph should be viewed in 3 dimensions. We choose to use a planar embedding of the graph. We hope to add rotatable, 3-dimensional viewing in the future. In such a case, an argument will be added to select the desired layout.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

Construct and show a Dodecahedral graph:

```
sage: g = graphs.DodecahedralGraph()
sage: g.show()                              # long time                          #␣
↪needs sage.plot
```

```
>>> from sage.all import *
>>> g = graphs.DodecahedralGraph()
>>> g.show()                              # long time                           #␣
↪needs sage.plot
```

Create several dodecahedral graphs in a Sage graphics array They will be drawn differently due to the use of the spring-layout algorithm:

```
sage: g = []
sage: j = []
sage: for i in range(9):
....:     k = graphs.DodecahedralGraph()
....:     g.append(k)
sage: for i in range(3):
....:     n = []
....:     for m in range(3):
....:         n.append(g[3*i + m].plot(vertex_size=50, vertex_labels=False))
....:     j.append(n)
sage: G = graphics_array(j)
sage: G.show()                              # long time
```

```
>>> from sage.all import *
>>> g = []
>>> j = []
>>> for i in range(Integer(9)):
...     k = graphs.DodecahedralGraph()
...     g.append(k)
>>> for i in range(Integer(3)):
...     n = []
...     for m in range(Integer(3)):
...         n.append(g[Integer(3)*i + m].plot(vertex_size=Integer(50), vertex_
↪labels=False))
...     j.append(n)
>>> G = graphics_array(j)
>>> G.show()                              # long time
```

sage.graphs.generators.platonic_solids.**HexahedralGraph**(*immutable=False*)

Return a hexahedral graph (with 8 nodes).

A regular hexahedron is a 6-sided cube. The hexahedral graph corresponds to the connectivity of the vertices of the hexahedron. This graph is equivalent to a 3-cube.

PLOTTING: The Hexahedral graph should be viewed in 3 dimensions. We choose to use a planar embedding of the graph. We hope to add rotatable, 3-dimensional viewing in the future. In such a case, an argument will be added to select the desired layout.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

Construct and show a Hexahedral graph:

```
sage: g = graphs.HexahedralGraph()
sage: g.show()                              # long time                              #␣
↪needs sage.plot
```

```
>>> from sage.all import *
>>> g = graphs.HexahedralGraph()
>>> g.show()                              # long time                              #␣
↪needs sage.plot
```

Create several hexahedral graphs in a Sage graphics array. They will be drawn differently due to the use of the spring-layout algorithm:

```
sage: g = []
sage: j = []
sage: for i in range(9):
....:     k = graphs.HexahedralGraph()
....:     g.append(k)
sage: for i in range(3):
....:     n = []
....:     for m in range(3):
....:         n.append(g[3*i + m].plot(vertex_size=50, vertex_labels=False))
....:     j.append(n)
sage: G = graphics_array(j)
sage: G.show()                              # long time
```

```
>>> from sage.all import *
>>> g = []
>>> j = []
>>> for i in range(Integer(9)):
...     k = graphs.HexahedralGraph()
...     g.append(k)
>>> for i in range(Integer(3)):
...     n = []
...     for m in range(Integer(3)):
...         n.append(g[Integer(3)*i + m].plot(vertex_size=Integer(50), vertex_
↪labels=False))
...     j.append(n)
>>> G = graphics_array(j)
>>> G.show()                              # long time
```

sage.graphs.generators.platonic_solids.**IcosahedralGraph**(*immutable=False*)

Return an Icosahedral graph (with 12 nodes).

The regular icosahedron is a 20-sided triangular polyhedron. The icosahedral graph corresponds to the connectivity of the vertices of the icosahedron. It is dual to the dodecahedral graph. The icosahedron is symmetric, so the spring-layout algorithm will be very effective for display.

PLOTTING: The Icosahedral graph should be viewed in 3 dimensions. We choose to use a planar embedding of the graph. We hope to add rotatable, 3-dimensional viewing in the future. In such a case, an argument will be added to select the desired layout.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

Construct and show an Octahedral graph:

```
sage: g = graphs.IcosahedralGraph()
sage: g.show()                              # long time                              #␣
↪needs sage.plot
```

```
>>> from sage.all import *
>>> g = graphs.IcosahedralGraph()
>>> g.show()                              # long time                              #␣
↪needs sage.plot
```

Create several icosahedral graphs in a Sage graphics array. They will be drawn differently due to the use of the spring-layout algorithm:

```
sage: g = []
sage: j = []
sage: for i in range(9):
....:     k = graphs.IcosahedralGraph()
....:     g.append(k)
sage: for i in range(3):
....:     n = []
....:     for m in range(3):
....:         n.append(g[3*i + m].plot(vertex_size=50, vertex_labels=False))
....:     j.append(n)
sage: G = graphics_array(j)
sage: G.show()                              # long time
```

```
>>> from sage.all import *
>>> g = []
>>> j = []
>>> for i in range(Integer(9)):
...     k = graphs.IcosahedralGraph()
...     g.append(k)
>>> for i in range(Integer(3)):
...     n = []
...     for m in range(Integer(3)):
...         n.append(g[Integer(3)*i + m].plot(vertex_size=Integer(50), vertex_
↪labels=False))
...     j.append(n)
>>> G = graphics_array(j)
>>> G.show()                              # long time
```

sage.graphs.generators.platonic_solids.**OctahedralGraph**(*immutable=False*)

Return an Octahedral graph (with 6 nodes).

The regular octahedron is an 8-sided polyhedron with triangular faces. The octahedral graph corresponds to the connectivity of the vertices of the octahedron. It is the line graph of the tetrahedral graph. The octahedral is symmetric, so the spring-layout algorithm will be very effective for display.

PLOTTING: The Octahedral graph should be viewed in 3 dimensions. We choose to use a planar embedding of the graph. We hope to add rotatable, 3-dimensional viewing in the future. In such a case, an argument will be added to select the desired layout.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

Construct and show an Octahedral graph:

```
sage: g = graphs.OctahedralGraph()
sage: g.show()                              # long time                              #␣
↪needs sage.plot
```

```
>>> from sage.all import *
>>> g = graphs.OctahedralGraph()
>>> g.show()                              # long time                              #␣
↪needs sage.plot
```

Create several octahedral graphs in a Sage graphics array They will be drawn differently due to the use of the spring-layout algorithm:

```
sage: g = []
sage: j = []
sage: for i in range(9):
....:     k = graphs.OctahedralGraph()
....:     g.append(k)
sage: for i in range(3):
....:     n = []
....:     for m in range(3):
....:         n.append(g[3*i + m].plot(vertex_size=50, vertex_labels=False))
....:     j.append(n)
sage: G = graphics_array(j)
sage: G.show()                              # long time
```

```
>>> from sage.all import *
>>> g = []
>>> j = []
>>> for i in range(Integer(9)):
...     k = graphs.OctahedralGraph()
...     g.append(k)
>>> for i in range(Integer(3)):
...     n = []
...     for m in range(Integer(3)):
...         n.append(g[Integer(3)*i + m].plot(vertex_size=Integer(50), vertex_
↪labels=False))
...     j.append(n)
>>> G = graphics_array(j)
>>> G.show()                              # long time
```

`sage.graphs.generators.platonic_solids.`**`TetrahedralGraph`**(*immutable=False*)

Return a tetrahedral graph (with 4 nodes).

A tetrahedron is a 4-sided triangular pyramid. The tetrahedral graph corresponds to the connectivity of the vertices of the tetrahedron. This graph is equivalent to a wheel graph with 4 nodes and also a complete graph on four nodes. (See examples below).

PLOTTING: The Tetrahedral graph should be viewed in 3 dimensions. We choose to use a planar embedding of the graph. We hope to add rotatable, 3-dimensional viewing in the future. In such a case, an argument will be added to select the desired layout.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

Construct and show a Tetrahedral graph:

```
sage: g = graphs.TetrahedralGraph()
sage: g.show()                              # long time                              #␣
↪needs sage.plot
```

```
>>> from sage.all import *
>>> g = graphs.TetrahedralGraph()
>>> g.show()                              # long time                              #␣
↪needs sage.plot
```

The following example requires networkx:

```
sage: import networkx as NX                                                        #␣
↪needs networkx
```

```
>>> from sage.all import *
>>> import networkx as NX                                                          #␣
↪needs networkx
```

Compare this Tetrahedral, Wheel(4), Complete(4), and the Tetrahedral plotted with the spring-layout algorithm below in a Sage graphics array:

```
sage: # needs networkx sage.plot
sage: tetra_pos = graphs.TetrahedralGraph()
sage: tetra_spring = Graph(NX.tetrahedral_graph())
sage: wheel = graphs.WheelGraph(4)
sage: complete = graphs.CompleteGraph(4)
sage: g = [tetra_pos, tetra_spring, wheel, complete]
sage: j = []
sage: for i in range(2):
....:     n = []
....:     for m in range(2):
....:         n.append(g[i + m].plot(vertex_size=50, vertex_labels=False))
....:     j.append(n)
sage: G = graphics_array(j)
sage: G.show()                              # long time
```

```
>>> from sage.all import *
>>> # needs networkx sage.plot
>>> tetra_pos = graphs.TetrahedralGraph()
>>> tetra_spring = Graph(NX.tetrahedral_graph())
>>> wheel = graphs.WheelGraph(Integer(4))
>>> complete = graphs.CompleteGraph(Integer(4))
>>> g = [tetra_pos, tetra_spring, wheel, complete]
>>> j = []
>>> for i in range(Integer(2)):
...     n = []
```

(continues on next page)

(continued from previous page)

```
...       for m in range(Integer(2)):
...           n.append(g[i + m].plot(vertex_size=Integer(50), vertex_labels=False))
...       j.append(n)
>>> G = graphics_array(j)
>>> G.show()                            # long time
```

2.15 Random graphs

The methods defined here appear in *sage.graphs.graph_generators*.

sage.graphs.generators.random.**RandomBarabasiAlbert**(*n*, *m*, *seed=None*, *immutable=False*)

Return a random graph created using the Barabasi-Albert preferential attachment model.

A graph with m vertices and no edges is initialized, and a graph of n vertices is grown by attaching new vertices each with m edges that are attached to existing vertices, preferentially with high degree.

INPUT:

- `n` – number of vertices in the graph
- `m` – number of edges to attach from each new node
- `seed` – a `random.Random` seed or a Python `int` for the random number generator (default: `None`)
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

We show the edge list of a random graph on 6 nodes with $m = 2$:

```
sage: G = graphs.RandomBarabasiAlbert(6,2)                                    #␣
↪needs networkx
sage: G.order(), G.size()                                                     #␣
↪needs networkx
(6, 8)
sage: G.degree_sequence()  # random                                           #␣
↪needs networkx
[4, 3, 3, 2, 2, 2]
```

```
>>> from sage.all import *
>>> G = graphs.RandomBarabasiAlbert(Integer(6),Integer(2))                     ␣
↪              # needs networkx
>>> G.order(), G.size()                                                      #␣
↪needs networkx
(6, 8)
>>> G.degree_sequence()  # random                                            #␣
↪needs networkx
[4, 3, 3, 2, 2, 2]
```

We plot a random graph on 12 nodes with $m = 3$:

```
sage: ba = graphs.RandomBarabasiAlbert(12,3)                                  #␣
↪needs networkx
sage: ba.show()                           # long time                         #␣
↪needs networkx sage.plot
```

```
>>> from sage.all import *
>>> ba = graphs.RandomBarabasiAlbert(Integer(12),Integer(3))                    ␣
↪              # needs networkx
>>> ba.show()                              # long time                          #␣
↪needs networkx sage.plot
```

We view many random graphs using a graphics array:

```
sage: # needs networkx sage.plot
sage: g = []
sage: j = []
sage: for i in range(1,10):
....:     k = graphs.RandomBarabasiAlbert(i+3, 3)
....:     g.append(k)
sage: for i in range(3):
....:     n = []
....:     for m in range(3):
....:         n.append(g[3*i + m].plot(vertex_size=50, vertex_labels=False))
....:     j.append(n)
sage: G = graphics_array(j)
sage: G.show()                            # long time
```

```
>>> from sage.all import *
>>> # needs networkx sage.plot
>>> g = []
>>> j = []
>>> for i in range(Integer(1),Integer(10)):
...     k = graphs.RandomBarabasiAlbert(i+Integer(3), Integer(3))
...     g.append(k)
>>> for i in range(Integer(3)):
...     n = []
...     for m in range(Integer(3)):
...         n.append(g[Integer(3)*i + m].plot(vertex_size=Integer(50), vertex_
↪labels=False))
...     j.append(n)
>>> G = graphics_array(j)
>>> G.show()                              # long time
```

When $m = 1$, the generated graph is a tree:

```
sage: graphs.RandomBarabasiAlbert(6, 1).is_tree()                               #␣
↪needs networkx
True
```

```
>>> from sage.all import *
>>> graphs.RandomBarabasiAlbert(Integer(6), Integer(1)).is_tree()               ␣
↪              # needs networkx
True
```

sage.graphs.generators.random.**RandomBicubicPlanar**(*n*, *seed=None*, *immutable=False*)

Return the graph of a random bipartite cubic map with $3n$ edges.

INPUT:

- `n` – integer (at least 1)
- `seed` – a `random.Random` seed or a Python `int` for the random number generator (default: `None`)
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

OUTPUT:

a graph with multiple edges (no embedding is provided)

The algorithm used is described in [Sch1999]. This samples a random rooted bipartite cubic map, chosen uniformly at random.

First one creates a random binary tree with n vertices. Next one turns this into a blossoming tree (at random) and reads the contour word of this blossoming tree.

Then one performs a rotation on this word so that this becomes a balanced word. There are three ways to do that, one is picked at random. Then a graph is build from the balanced word by iterated closure (adding edges).

In the returned graph, the three edges incident to any given vertex are colored by the integers 0, 1 and 2.

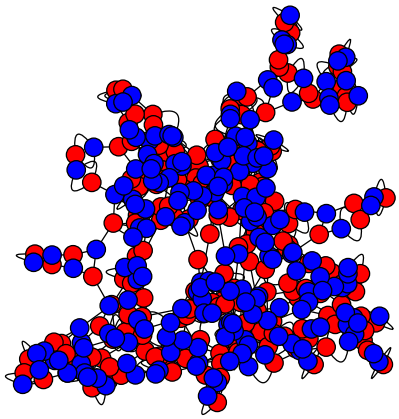
See also

the auxiliary function *blossoming_contour()*

EXAMPLES:

```
sage: n = randint(200, 300)
sage: G = graphs.RandomBicubicPlanar(n)
sage: G.order() == 2*n
True
sage: G.size() == 3*n
True
sage: G.is_bipartite() and G.is_planar() and G.is_regular(3)
True
sage: dic = {'red': [v for v in G.vertices(sort=False) if v[0] == 'n'],
....:        'blue': [v for v in G.vertices(sort=False) if v[0] != 'n']}
sage: G.plot(vertex_labels=False, vertex_size=20, vertex_colors=dic)            #␣
↪needs sage.plot
Graphics object consisting of ... graphics primitives
```

```
>>> from sage.all import *
>>> n = randint(Integer(200), Integer(300))
>>> G = graphs.RandomBicubicPlanar(n)
>>> G.order() == Integer(2)*n
True
>>> G.size() == Integer(3)*n
True
>>> G.is_bipartite() and G.is_planar() and G.is_regular(Integer(3))
True
>>> dic = {'red': [v for v in G.vertices(sort=False) if v[Integer(0)] == 'n'],
...        'blue': [v for v in G.vertices(sort=False) if v[Integer(0)] != 'n']}
>>> G.plot(vertex_labels=False, vertex_size=Integer(20), vertex_colors=dic)      ␣
↪     # needs sage.plot
Graphics object consisting of ... graphics primitives
```



sage.graphs.generators.random.**RandomBipartite**(*n1*, *n2*, *p*, *set_position=False*, *seed=None*, *immutable=False*)

Return a bipartite graph with $n1 + n2$ vertices such that any edge from $[n1]$ to $[n2]$ exists with probability p.

INPUT:

- `n1`, `n2` – cardinalities of the two sets
- `p` – probability for an edge to exist
- `set_position` – boolean (default: `False`); if set to `True`, we assign positions to the vertices so that the set of cardinality $n1$ is on the line $y = 1$ and the set of cardinality $n2$ is on the line $y = 0$
- `seed` – a `random.Random` seed or a Python `int` for the random number generator (default: `None`)
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.RandomBipartite(5, 2, 0.5)                                    #␣
↪needs numpy
sage: g.vertices(sort=True)                                                    #␣
↪needs numpy
[(0, 0), (0, 1), (0, 2), (0, 3), (0, 4), (1, 0), (1, 1)]
```

```
>>> from sage.all import *
>>> g = graphs.RandomBipartite(Integer(5), Integer(2), RealNumber('0.5'))      ␣
↪                              # needs numpy
>>> g.vertices(sort=True)                                                      #␣
↪needs numpy
[(0, 0), (0, 1), (0, 2), (0, 3), (0, 4), (1, 0), (1, 1)]
```

sage.graphs.generators.random.**RandomBlockGraph**(*m*, *k*, *kmax=None*, *incidence_structure=False*, *seed=None*, *immutable=False*)

Return a Random Block Graph.

A block graph is a connected graph in which every biconnected component (block) is a clique.

See also

- Wikipedia article Block_graph for more details on these graphs
- *is_block_graph()* – test if a graph is a block graph
- *blocks_and_cut_vertices()*
- *blocks_and_cuts_tree()*
- *IncidenceStructure()*

INPUT:

- `m` – integer; number of blocks (at least one)
- `k` – integer; minimum number of vertices of a block (at least two)
- `kmax` – integer (default: `None`); by default, each block has k vertices. When the parameter $kmax$ is specified (with $kmax \geq k$), the number of vertices of each block is randomly chosen between k and $kmax$.
- `incidence_structure` – boolean (default: `False`); when set to `True`, the incidence structure of the graphs is returned instead of the graph itself, that is the list of the lists of vertices in each block. This is useful for the creation of some hypergraphs.
- `seed` – a `random.Random` seed or a Python `int` for the random number generator (default: `None`)
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

OUTPUT:

A Graph when `incidence_structure==False` (default), and otherwise an incidence structure.

EXAMPLES:

A block graph with a single block is a clique:

```
sage: B = graphs.RandomBlockGraph(1, 4)
sage: B.is_clique()
True
```

```
>>> from sage.all import *
>>> B = graphs.RandomBlockGraph(Integer(1), Integer(4))
>>> B.is_clique()
True
```

A block graph with blocks of order 2 is a tree:

```
sage: B = graphs.RandomBlockGraph(10, 2)
sage: B.is_tree()
True
```

```
>>> from sage.all import *
>>> B = graphs.RandomBlockGraph(Integer(10), Integer(2))
>>> B.is_tree()
True
```

Every biconnected component of a block graph is a clique:

```
sage: B = graphs.RandomBlockGraph(5, 3, kmax=6)
sage: blocks,cuts = B.blocks_and_cut_vertices()
sage: all(B.is_clique(block) for block in blocks)
True
```

```
>>> from sage.all import *
>>> B = graphs.RandomBlockGraph(Integer(5), Integer(3), kmax=Integer(6))
>>> blocks,cuts = B.blocks_and_cut_vertices()
>>> all(B.is_clique(block) for block in blocks)
True
```

A block graph with blocks of order k has $m * (k - 1) + 1$ vertices:

```
sage: m, k = 6, 4
sage: B = graphs.RandomBlockGraph(m, k)
sage: B.order() == m*(k-1)+1
True
```

```
>>> from sage.all import *
>>> m, k = Integer(6), Integer(4)
>>> B = graphs.RandomBlockGraph(m, k)
>>> B.order() == m*(k-Integer(1))+Integer(1)
True
```

Test recognition methods:

```
sage: B = graphs.RandomBlockGraph(6, 2, kmax=6)
sage: B.is_block_graph()
True
sage: B in graph_classes.Block
True
```

```
>>> from sage.all import *
>>> B = graphs.RandomBlockGraph(Integer(6), Integer(2), kmax=Integer(6))
>>> B.is_block_graph()
True
>>> B in graph_classes.Block
True
```

Asking for the incidence structure:

```
sage: m, k = 6, 4
sage: IS = graphs.RandomBlockGraph(m, k, incidence_structure=True)
sage: from sage.combinat.designs.incidence_structures import IncidenceStructure
sage: IncidenceStructure(IS)                                                #␣
↪needs sage.modules
Incidence structure with 19 points and 6 blocks
sage: m*(k-1)+1
19
```

```
>>> from sage.all import *
>>> m, k = Integer(6), Integer(4)
```

(continues on next page)

(continued from previous page)

```
>>> IS = graphs.RandomBlockGraph(m, k, incidence_structure=True)
>>> from sage.combinat.designs.incidence_structures import IncidenceStructure
>>> IncidenceStructure(IS)                                                    #␣
↪needs sage.modules
Incidence structure with 19 points and 6 blocks
>>> m*(k-Integer(1))+Integer(1)
19
```

sage.graphs.generators.random.**RandomBoundedToleranceGraph**(*n*, *seed=None*, *immutable=False*)

Return a random bounded tolerance graph.

The random tolerance graph is built from a random bounded tolerance representation by using the function $ToleranceGraph$. This representation is a list $((l_0, r_0, t_0), (l_1, r_1, t_1), ..., (l_k, r_k, t_k))$ where $k = n - 1$ and $I_i = (l_i, r_i)$ denotes a random interval and t_i a random positive value less than or equal to the length of the interval I_i. The width of the representation is limited to $n^2 * 2^n$.

Note

The tolerance representation used to create the graph can be recovered using `get_vertex()` or `get_vertices()`.

INPUT:

- `n` – number of vertices of the random graph
- `seed` – a `random.Random` seed or a Python `int` for the random number generator (default: `None`)
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

Every (bounded) tolerance graph is perfect. Hence, the chromatic number is equal to the clique number

```
sage: g = graphs.RandomBoundedToleranceGraph(8)
sage: g.clique_number() == g.chromatic_number()
True
```

```
>>> from sage.all import *
>>> g = graphs.RandomBoundedToleranceGraph(Integer(8))
>>> g.clique_number() == g.chromatic_number()
True
```

sage.graphs.generators.random.**RandomChordalGraph**(*n*, *algorithm='growing'*, *k=None*, *l=None*, *f=None*, *s=None*, *seed=None*, *immutable=False*)

Return a random chordal graph of order `n`.

A Graph G is said to be chordal if it contains no induced hole (a cycle of length at least 4). Equivalently, G is chordal if it has a perfect elimination orderings, if each minimal separator is a clique, or if it is the intersection graphs of subtrees of a tree. See the Wikipedia article Chordal_graph.

This generator implements the algorithms proposed in [SHET2018] for generating random chordal graphs as the intersection graph of n subtrees of a tree of order n.

The returned graph is not necessarily connected.

INPUT:

- `n` – integer; the number of nodes of the graph
- `algorithm` – string (default: `'growing'`); the choice of the algorithm for randomly selecting n subtrees of a random tree of order n. Possible choices are:
  - `'growing'` – for each subtree T_i, the algorithm picks a size k_i randomly from $[1, k]$. Then a random node of T is chosen as the first node of T_i. In each of the subsequent $k_i - 1$ iterations, it picks a random node in the neighborhood of T_i and adds it to T_i.
  - `'connecting'` – for each subtree T_i, it first selects k_i nodes of T, where k_i is a random integer from a Poisson distribution with mean l. T_i is then generated to be the minimal subtree containing the selected k_i nodes. This implies that a subtree will most likely have many more nodes than those selected initially, and this must be taken into consideration when choosing l.
  - `'pruned'` – for each subtree T_i, it randomly selects a fraction f of the edges on the tree and removes them. The number of edges to delete, say l, is calculated as $\lfloor (n-1)f \rfloor$, which will leave $l+1$ subtrees in total. Then, it determines the sizes of the $l+1$ subtrees and stores the distinct values. Finally, it picks a random size k_i from the set of largest $100(1-s)\%$ of distinct values, and randomly chooses a subtree with size k_i.
- `k` – integer (default: `None`); maximum size of a subtree. If not specified (`None`), the maximum size is set to $\sqrt{n}$. This parameter is used only when `algorithm="growing"`. See *growing_subtrees()* for more details.
- `l` – a strictly positive real number (default: `None`); mean of a Poisson distribution. If not specified, the mean in set to $\log_2 n$. This parameter is used only when `algorithm="connecting"`. See *connecting_nodes()* for more details.
- `f` – a rational number (default: `None`); the edge deletion fraction. This value must be chosen in $[0..1]$. If not specified, this parameter is set to $\frac{1}{n-1}$. This parameter is used only when `algorithm="pruned"`. See *pruned_tree()* for more details.
- `s` – a real number between 0 and 1 (default: `None`); selection barrier for the size of trees. If not specified, this parameter is set to 0.5. This parameter is used only when `algorithm="pruned"`. See *pruned_tree()* for more details.
- `seed` – a `random.Random` seed or a Python `int` for the random number generator (default: `None`)
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: from sage.graphs.generators.random import RandomChordalGraph
sage: T = RandomChordalGraph(20, algorithm='growing', k=5)
sage: T.is_chordal()
True
sage: T = RandomChordalGraph(20, algorithm='connecting', l=3)                  #␣
↪needs numpy
sage: T.is_chordal()                                                           #␣
↪needs numpy
True
sage: T = RandomChordalGraph(20, algorithm='pruned', f=1/3, s=.5)
sage: T.is_chordal()
True
```

```
>>> from sage.all import *
>>> from sage.graphs.generators.random import RandomChordalGraph
>>> T = RandomChordalGraph(Integer(20), algorithm='growing', k=Integer(5))
>>> T.is_chordal()
```

(continues on next page)

(continued from previous page)

```
True
>>> T = RandomChordalGraph(Integer(20), algorithm='connecting', l=Integer(3))
↪              # needs numpy
>>> T.is_chordal()                                                    #␣
↪needs numpy
True
>>> T = RandomChordalGraph(Integer(20), algorithm='pruned', f=Integer(1)/
↪Integer(3), s=RealNumber('.5'))
>>> T.is_chordal()
True
```

See also

- *growing_subtrees()*
- *connecting_nodes()*
- *pruned_tree()*
- Wikipedia article Chordal_graph
- *is_chordal()*
- *IntersectionGraph()*

sage.graphs.generators.random.**RandomGNM**(*n*, *m*, *dense=False*, *seed=None*, *immutable=False*)

Return a graph randomly picked out of all graphs on n vertices with m edges.

INPUT:

- `n` – number of vertices
- `m` – number of edges
- `dense` – whether to use NetworkX's `dense_gnm_random_graph()` or `gnm_random_graph()`
- `seed` – a `random.Random` seed or a Python `int` for the random number generator (default: `None`)
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

We show the edge list of a random graph on 5 nodes with 10 edges:

```
sage: graphs.RandomGNM(5, 10).edges(sort=True, labels=False)                #␣
↪needs networkx
[(0, 1), (0, 2), (0, 3), (0, 4), (1, 2), (1, 3), (1, 4), (2, 3), (2, 4), (3, 4)]
```

```
>>> from sage.all import *
>>> graphs.RandomGNM(Integer(5), Integer(10)).edges(sort=True, labels=False)
↪              # needs networkx
[(0, 1), (0, 2), (0, 3), (0, 4), (1, 2), (1, 3), (1, 4), (2, 3), (2, 4), (3, 4)]
```

We plot a random graph on 12 nodes and 12 edges:

```
sage: gnm = graphs.RandomGNM(12, 12)                                    #␣
↪needs networkx
sage: gnm.show()                          # long time                   #␣
↪needs networkx sage.plot
```

```
>>> from sage.all import *
>>> gnm = graphs.RandomGNM(Integer(12), Integer(12))                     ␣
↪              # needs networkx
>>> gnm.show()                           # long time                   #␣
↪needs networkx sage.plot
```

We view many random graphs using a graphics array:

```
sage: # needs networkx sage.plot
sage: g = []
sage: j = []
sage: for i in range(9):
....:     k = graphs.RandomGNM(i+3, i^2-i)
....:     g.append(k)
sage: for i in range(3):
....:     n = []
....:     for m in range(3):
....:         n.append(g[3*i + m].plot(vertex_size=50, vertex_labels=False))
....:     j.append(n)
sage: G = graphics_array(j)
sage: G.show()                            # long time
```

```
>>> from sage.all import *
>>> # needs networkx sage.plot
>>> g = []
>>> j = []
>>> for i in range(Integer(9)):
...     k = graphs.RandomGNM(i+Integer(3), i**Integer(2)-i)
...     g.append(k)
>>> for i in range(Integer(3)):
...     n = []
...     for m in range(Integer(3)):
...         n.append(g[Integer(3)*i + m].plot(vertex_size=Integer(50), vertex_
↪labels=False))
...     j.append(n)
>>> G = graphics_array(j)
>>> G.show()                             # long time
```

sage.graphs.generators.random.**RandomGNP**(*n*, *p*, *seed=None*, *fast=True*, *algorithm='Sage'*, *immutable=False*)

Return a random graph on n nodes. Each edge is inserted independently with probability p.

INPUT:

- `n` – number of nodes of the graph
- `p` – probability of an edge
- `seed` – a `random.Random` seed or a Python `int` for the random number generator (default: `None`)

- `fast` – boolean (default: `True`) to use the algorithm with time complexity in $O(n + m)$ proposed in [BB2005a]. It is designed for generating large sparse graphs. It is faster than other algorithms for *LARGE* instances (try it to know whether it is useful for you).
- `algorithm` – (default: `'Sage'`) this function uses the algorithm implemented in `sage.graphs.graph_generators_pyx.pyx`. When `algorithm='networkx'`, this function calls the NetworkX function `fast_gnp_random_graph`, unless `fast=False`, then `gnp_random_graph`. Try them to know which algorithm is the best for you. The `fast` parameter is not taken into account by the 'Sage' algorithm so far.
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

REFERENCES:

- [ER1959]
- [Gil1959]

PLOTTING: When plotting, this graph will use the default spring-layout algorithm, unless a position dictionary is specified.

EXAMPLES: We show the edge list of a random graph on 6 nodes with probability $p = .4$:

```
sage: set_random_seed(0)
sage: graphs.RandomGNP(6, .4).edges(sort=true, labels=False)
[(0, 3), (1, 2), (2, 3), (2, 4)]
```

```
>>> from sage.all import *
>>> set_random_seed(Integer(0))
>>> graphs.RandomGNP(Integer(6), RealNumber('.4')).edges(sort=true, labels=False)
[(0, 3), (1, 2), (2, 3), (2, 4)]
```

We plot a random graph on 12 nodes with probability $p = .71$:

```
sage: gnp = graphs.RandomGNP(12,.71)
sage: gnp.show()                          # long time                          #␣
↪needs sage.plot
```

```
>>> from sage.all import *
>>> gnp = graphs.RandomGNP(Integer(12),RealNumber('.71'))
>>> gnp.show()                          # long time                          #␣
↪needs sage.plot
```

We view many random graphs using a graphics array:

```
sage: g = []
sage: j = []
sage: for i in range(9):
....:     k = graphs.RandomGNP(i+3,.43)
....:     g.append(k)
sage: for i in range(3):                                                      #␣
↪needs sage.plot
....:     n = []
....:     for m in range(3):
....:         n.append(g[3*i + m].plot(vertex_size=50, vertex_labels=False))
....:     j.append(n)
sage: G = graphics_array(j)                                                   #␣
↪needs sage.plot
```

(continues on next page)

(continued from previous page)

```
sage: G.show()                              # long time                              #␣
↪needs sage.plot
sage: graphs.RandomGNP(4,1)
Complete graph: Graph on 4 vertices
```

```
>>> from sage.all import *
>>> g = []
>>> j = []
>>> for i in range(Integer(9)):
...     k = graphs.RandomGNP(i+Integer(3),RealNumber('.43'))
...     g.append(k)
>>> for i in range(Integer(3)):                                                    ␣
↪     # needs sage.plot
...     n = []
...     for m in range(Integer(3)):
...         n.append(g[Integer(3)*i + m].plot(vertex_size=Integer(50), vertex_
↪labels=False))
...     j.append(n)
>>> G = graphics_array(j)                                                         #␣
↪needs sage.plot
>>> G.show()                              # long time                             #␣
↪needs sage.plot
>>> graphs.RandomGNP(Integer(4),Integer(1))
Complete graph: Graph on 4 vertices
```

sage.graphs.generators.random.**RandomHolmeKim**(*n*, *m*, *p*, *seed=None*, *immutable=False*)

Return a random graph generated by the Holme and Kim algorithm for graphs with power law degree distribution and approximate average clustering.

INPUT:

- `n` – number of vertices
- `m` – number of random edges to add for each new node
- `p` – probability of adding a triangle after adding a random edge
- `seed` – a `random.Random` seed or a Python `int` for the random number generator (default: `None`)
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

From the NetworkX documentation: the average clustering has a hard time getting above a certain cutoff that depends on m. This cutoff is often quite low. Note that the transitivity (fraction of triangles to possible triangles) seems to go down with network size. It is essentially the Barabasi-Albert growth model with an extra step that each random edge is followed by a chance of making an edge to one of its neighbors too (and thus a triangle). This algorithm improves on B-A in the sense that it enables a higher average clustering to be attained if desired. It seems possible to have a disconnected graph with this algorithm since the initial m nodes may not be all linked to a new node on the first iteration like the BA model.

EXAMPLES:

```
sage: G = graphs.RandomHolmeKim(12, 3, .3)                                         #␣
↪needs networkx
sage: G.show()                              # long time                            #␣
↪needs networkx sage.plot
```

```
>>> from sage.all import *
>>> G = graphs.RandomHolmeKim(Integer(12), Integer(3), RealNumber('.3'))          ␣
↪                              # needs networkx
>>> G.show()                             # long time                              #␣
↪needs networkx sage.plot
```

REFERENCE:

[HK2002a]

sage.graphs.generators.random.**RandomIntervalGraph**(*n*, *seed=None*, *immutable=False*)

Return a random interval graph.

An interval graph is built from a list $(a_i, b_i)_{1 \leq i \leq n}$ of intervals : to each interval of the list is associated one vertex, two vertices being adjacent if the two corresponding intervals intersect.

A random interval graph of order n is generated by picking random values for the (a_i, b_j), each of the two coordinates being generated from the uniform distribution on the interval $[0, 1]$.

This definitions follows [BF2001].

Note

The vertices are named 0, 1, 2, and so on. The intervals used to create the graph are saved with the graph and can be recovered using `get_vertex()` or `get_vertices()`.

See also

- *sage.graphs.generators.intersection.IntervalGraph()*
- *sage.graphs.generators.random.RandomProperIntervalGraph()*

INPUT:

- `n` – integer; the number of vertices in the random graph
- `seed` – a `random.Random` seed or a Python `int` for the random number generator (default: `None`)
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

As for any interval graph, the chromatic number is equal to the clique number

```
sage: g = graphs.RandomIntervalGraph(8)
sage: g.clique_number() == g.chromatic_number()
True
```

```
>>> from sage.all import *
>>> g = graphs.RandomIntervalGraph(Integer(8))
>>> g.clique_number() == g.chromatic_number()
True
```

sage.graphs.generators.random.**RandomKTree**(*n*, *k*, *seed=None*, *immutable=False*)

Return a random k-tree on n nodes numbered 0 through $n - 1$.

ALGORITHM:

The algorithm first generates a complete graph on $k+1$ vertices. Vertices are subsequently generated by randomly choosing one of the existing cliques in the graph, and creating a new clique by replacing one of the vertices in the selected clique with a newly created one.

INPUT:

- `n` – number of vertices in the k-tree
- `k` – within a clique each vertex is connected to k vertices. k also corresponds to the treewidth of the k-tree
- `seed` – a `random.Random` seed or a Python `int` for the random number generator (default: `None`)
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: G = graphs.RandomKTree(50, 5)
sage: G.treewidth()
5
sage: G.show()  # not tested
```

```
>>> from sage.all import *
>>> G = graphs.RandomKTree(Integer(50), Integer(5))
>>> G.treewidth()
5
>>> G.show()  # not tested
```

sage.graphs.generators.random.**RandomNewmanWattsStrogatz**(*n*, *k*, *p*, *seed=None*, *immutable=False*)

Return a Newman-Watts-Strogatz small world random graph on n vertices.

From the NetworkX documentation: first create a ring over n nodes. Then each node in the ring is connected with its k nearest neighbors. Then shortcuts are created by adding new edges as follows: for each edge $u-v$ in the underlying "n-ring with k nearest neighbors"; with probability p add a new edge $u-w$ with randomly-chosen existing node w. In contrast with `networkx.watts_strogatz_graph()`, no edges are removed.

INPUT:

- `n` – number of vertices
- `k` – each vertex is connected to its k nearest neighbors
- `p` – the probability of adding a new edge for each edge
- `seed` – a `random.Random` seed or a Python `int` for the random number generator (default: `None`)
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

We check that the generated graph contains a cycle of order n:

```
sage: # needs networkx
sage: G = graphs.RandomNewmanWattsStrogatz(7, 2, 0.2)
sage: G.order()
7
sage: C7 = graphs.CycleGraph(7)
sage: G.subgraph_search(C7)
Subgraph of (): Graph on 7 vertices
```

(continues on next page)

(continued from previous page)

```
sage: G.diameter() <= C7.diameter()
True
```

```
>>> from sage.all import *
>>> # needs networkx
>>> G = graphs.RandomNewmanWattsStrogatz(Integer(7), Integer(2), RealNumber('0.2
↪'))
>>> G.order()
7
>>> C7 = graphs.CycleGraph(Integer(7))
>>> G.subgraph_search(C7)
Subgraph of (): Graph on 7 vertices
>>> G.diameter() <= C7.diameter()
True
```

```
sage: G = graphs.RandomNewmanWattsStrogatz(12, 2, .3)                    #␣
↪needs networkx
sage: G.show()                          # long time                      #␣
↪needs networkx sage.plot
```

```
>>> from sage.all import *
>>> G = graphs.RandomNewmanWattsStrogatz(Integer(12), Integer(2), RealNumber('.3
↪'))                          # needs networkx
>>> G.show()                          # long time                        #␣
↪needs networkx sage.plot
```

REFERENCE:

[NWS2002]

sage.graphs.generators.random.**RandomPartialKTree**(*n*, *k*, *x*, *seed=None*, *immutable=False*)

Return a random partial k-tree on n nodes.

A partial k-tree is defined as a subgraph of a k-tree. This can also be described as a graph with treewidth at most k.

INPUT:

- `n` – number of vertices in the k-tree
- `k` – within a clique each vertex is connected to k vertices. k also corresponds to the treewidth of the k-tree
- `x` – how many edges are deleted from the k-tree
- `seed` – a `random.Random` seed or a Python `int` for the random number generator (default: `None`)
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: G = graphs.RandomPartialKTree(50,5,2)
sage: G.treewidth()
5
sage: G.show()  # not tested
```

```
>>> from sage.all import *
>>> G = graphs.RandomPartialKTree(Integer(50),Integer(5),Integer(2))
>>> G.treewidth()
5
>>> G.show()  # not tested
```

sage.graphs.generators.random.**RandomProperIntervalGraph**(*n*, *seed=None*, *immutable=False*)

Return a random proper interval graph.

An interval graph is built from a list $(a_i, b_i)_{1 \leq i \leq n}$ of intervals : to each interval of the list is associated one vertex, two vertices being adjacent if the two corresponding (closed) intervals intersect. An interval graph is proper if no interval of the list properly contains another interval. Observe that proper interval graphs coincide with unit interval graphs. See the Wikipedia article Interval_graph for more details.

This method implements the random proper interval graph generator proposed in [SYKU2010] which outputs graphs with uniform probability. The time complexity of this generator is in $O(n^3)$.

Note

The vertices are named 0, 1, 2, and so on. The intervals used to create the graph are saved with the graph and can be recovered using `get_vertex()` or `get_vertices()`.

See also

- *sage.graphs.generators.intersection.IntervalGraph()*
- *sage.graphs.generators.random.RandomIntervalGraph()*

INPUT:

- `n` – positive integer; the number of vertices of the graph
- `seed` – a `random.Random` seed or a Python `int` for the random number generator (default: `None`)
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: from sage.graphs.generators.random import RandomProperIntervalGraph
sage: G = RandomProperIntervalGraph(10)
sage: G.is_interval()
True
```

```
>>> from sage.all import *
>>> from sage.graphs.generators.random import RandomProperIntervalGraph
>>> G = RandomProperIntervalGraph(Integer(10))
>>> G.is_interval()
True
```

sage.graphs.generators.random.**RandomRegular**(*d*, *n*, *seed=None*, *immutable=False*)

Return a random d-regular graph on n vertices, or `False` on failure.

Since every edge is incident to two vertices, $n \times d$ must be even.

INPUT:

- `d` – degree
- `n` – number of vertices
- `seed` – a `random.Random` seed or a Python `int` for the random number generator (default: `None`)
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

We check that a random graph with 8 nodes each of degree 3 is 3-regular:

```
sage: G = graphs.RandomRegular(3, 8)                                    #␣
↪needs networkx
sage: G.is_regular(k=3)                                                 #␣
↪needs networkx
True
sage: G.degree_histogram()                                              #␣
↪needs networkx
[0, 0, 0, 8]
```

```
>>> from sage.all import *
>>> G = graphs.RandomRegular(Integer(3), Integer(8))                    ␣
↪              # needs networkx
>>> G.is_regular(k=Integer(3))                                          ␣
↪     # needs networkx
True
>>> G.degree_histogram()                                               #␣
↪needs networkx
[0, 0, 0, 8]
```

```
sage: G = graphs.RandomRegular(3, 20)                                   #␣
↪needs networkx
sage: if G:                                 # random output          # long time,␣
↪needs networkx sage.plot
....:     G.show()
```

```
>>> from sage.all import *
>>> G = graphs.RandomRegular(Integer(3), Integer(20))                   ␣
↪              # needs networkx
>>> if G:                                  # random output          # long time, needs␣
↪networkx sage.plot
...     G.show()
```

REFERENCES:

- [KV2003]
- [SW1999]

`sage.graphs.generators.random.`**`RandomRegularBipartite`**(*n1*, *n2*, *d1*, *set_position=False*, *seed=None*, *immutable=False*)

Return a random regular bipartite graph on $n1 + n2$ vertices.

The bipartite graph has $n1 * d1$ edges. Hence, $n2$ must divide $n1 * d1$. Each vertex of the set of cardinality $n1$ has degree $d1$ (which can be at most $n2$) and each vertex in the set of cardinality $n2$ has degree $(n1 * d1)/n2$. The bipartite graph has no multiple edges.

This generator implements an algorithm inspired by that of [MW1990] for the uniform generation of random regular bipartite graphs. It performs well when $d1 = o(n2^{1/3})$ or $(n2 - d1 = o(n2^{1/3}))$. In other cases, the running time can be huge. Note that the currently implemented algorithm does not generate uniformly random graphs.

INPUT:

- `n1`, `n2` – number of vertices in each side
- `d1` – degree of the vertices in the set of cardinality $n1$
- `set_position` – boolean (default: `False`); if set to `True`, we assign positions to the vertices so that the set of cardinality $n1$ is on the line $y = 1$ and the set of cardinality $n2$ is on the line $y = 0$.
- `seed` – a `random.Random` seed or a Python `int` for the random number generator (default: `None`)
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.RandomRegularBipartite(4, 6, 3)
sage: g.order(), g.size()
(10, 12)
sage: set(g.degree())
{2, 3}

sage: graphs.RandomRegularBipartite(1, 2, 2, set_position=True).get_pos()
{0: (1, 1.0), 1: (0, 0), 2: (2.0, 0.0)}
sage: graphs.RandomRegularBipartite(2, 1, 1, set_position=True).get_pos()
{0: (0, 1), 1: (2.0, 1.0), 2: (1, 0.0)}
sage: graphs.RandomRegularBipartite(2, 3, 3, set_position=True).get_pos()
{0: (0, 1), 1: (3.0, 1.0), 2: (0, 0), 3: (1.5, 0.0), 4: (3.0, 0.0)}
sage: graphs.RandomRegularBipartite(2, 3, 3, set_position=False).get_pos()
```

```
>>> from sage.all import *
>>> g = graphs.RandomRegularBipartite(Integer(4), Integer(6), Integer(3))
>>> g.order(), g.size()
(10, 12)
>>> set(g.degree())
{2, 3}

>>> graphs.RandomRegularBipartite(Integer(1), Integer(2), Integer(2), set_
↪position=True).get_pos()
{0: (1, 1.0), 1: (0, 0), 2: (2.0, 0.0)}
>>> graphs.RandomRegularBipartite(Integer(2), Integer(1), Integer(1), set_
↪position=True).get_pos()
{0: (0, 1), 1: (2.0, 1.0), 2: (1, 0.0)}
>>> graphs.RandomRegularBipartite(Integer(2), Integer(3), Integer(3), set_
↪position=True).get_pos()
{0: (0, 1), 1: (3.0, 1.0), 2: (0, 0), 3: (1.5, 0.0), 4: (3.0, 0.0)}
>>> graphs.RandomRegularBipartite(Integer(2), Integer(3), Integer(3), set_
↪position=False).get_pos()
```

`sage.graphs.generators.random.`**`RandomShell`**(*constructor*, *seed=None*, *immutable=False*)

Return a random shell graph for the constructor given.

INPUT:

- `constructor` – list of 3-tuples (n, m, d), each representing a shell, where:
  - `n` – the number of vertices in the shell
  - `m` – the number of edges in the shell
  - `d` – the ratio of inter (next) shell edges to intra shell edges
- `seed` – a `random.Random` seed or a Python `int` for the random number generator (default: `None`)
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: G = graphs.RandomShell([(10,20,0.8),(20,40,0.8)])                    #␣
↪needs networkx
sage: G.order(), G.size()                                                  #␣
↪needs networkx
(30, 52)
sage: G.show()                              # long time                    #␣
↪needs networkx sage.plot
```

```
>>> from sage.all import *
>>> G = graphs.RandomShell([(Integer(10),Integer(20),RealNumber('0.8')),
↪(Integer(20),Integer(40),RealNumber('0.8'))])                          # needs␣
↪networkx
>>> G.order(), G.size()                                                    #␣
↪needs networkx
(30, 52)
>>> G.show()                              # long time                     #␣
↪needs networkx sage.plot
```

`sage.graphs.generators.random.`**`RandomToleranceGraph`**(*n*, *seed=None*, *immutable=False*)

Return a random tolerance graph.

The random tolerance graph is built from a random tolerance representation by using the function *`ToleranceGraph()`*. This representation is a list $((l_0, r_0, t_0), (l_1, r_1, t_1), ..., (l_k, r_k, t_k))$ where $k = n - 1$ and $I_i = (l_i, r_i)$ denotes a random interval and t_i a random positive value. The width of the representation is limited to $n^2 * 2^n$.

Note

The vertices are named $0, 1, \cdots, n-1$. The tolerance representation used to create the graph is saved with the graph and can be recovered using *`get_vertex()`* or *`get_vertices()`*.

INPUT:

- `n` – number of vertices of the random graph
- `seed` – a `random.Random` seed or a Python `int` for the random number generator (default: `None`)
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

Every tolerance graph is perfect. Hence, the chromatic number is equal to the clique number

```
sage: g = graphs.RandomToleranceGraph(8)
sage: g.clique_number() == g.chromatic_number()
True
```

```
>>> from sage.all import *
>>> g = graphs.RandomToleranceGraph(Integer(8))
>>> g.clique_number() == g.chromatic_number()
True
```

sage.graphs.generators.random.**RandomTriangulation**(*n*, *set_position=False*, *k=3*, *seed=None*, *immutable=False*)

Return a random inner triangulation of an outer face of degree `k` with `n` vertices in total.

An inner triangulation is a plane graph all of whose faces (except the outer/unbounded face) are triangles (3-cycles).

INPUT:

- `n` – the number of vertices of the graph
- `k` – the size of the outer face
- `set_position` – boolean (default: `False`); if set to `True`, this will compute coordinates for a planar drawing of the graph
- `seed` – a `random.Random` seed or a Python `int` for the random number generator (default: `None`)
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

OUTPUT:

A random graph chosen uniformly among the inner triangulations of a *rooted* k-gon with n vertices (including the k vertices from the outer face). This is a planar graph and comes with a combinatorial embedding. The vertices of the root edge are labelled `-1` and `-2` and the outer face is the face returned by *faces()* in which `-1` and `-2` are consecutive vertices in this order.

Because some triangulations have nontrivial automorphism groups, this may not be equal to the uniform distribution among inner triangulations of unrooted k-gons.

ALGORITHM:

The algorithm is taken from [PS2006], Section 5.

Starting from a planar k-gonal forest (represented by its contour as a sequence of vertices), one performs local closures, until no one is possible. A local closure amounts to replace in the cyclic contour word a sequence `in1, in2, in3, lf, in3` by `in1, in3`.

At every step of the algorithm, newly created edges are recorded in a graph, which will be returned at the end. The combinatorial embedding is also computed and recorded in the output graph.

See also

triangulations(), RandomTwoSphere().

EXAMPLES:

```
sage: G = graphs.RandomTriangulation(6, True); G
Graph on 6 vertices
sage: G.is_planar()
```

(continues on next page)

(continued from previous page)

```
True
sage: G.girth()
3
sage: G.plot(vertex_size=0, vertex_labels=False)                                #␣
↪needs sage.plot
Graphics object consisting of 13 graphics primitives

sage: H = graphs.RandomTriangulation(7, k=5)
sage: sorted(len(f) for f in H.faces())
[3, 3, 3, 3, 3, 3, 3, 5]
```

```
>>> from sage.all import *
>>> G = graphs.RandomTriangulation(Integer(6), True); G
Graph on 6 vertices
>>> G.is_planar()
True
>>> G.girth()
3
>>> G.plot(vertex_size=Integer(0), vertex_labels=False)                         ␣
↪     # needs sage.plot
Graphics object consisting of 13 graphics primitives

>>> H = graphs.RandomTriangulation(Integer(7), k=Integer(5))
>>> sorted(len(f) for f in H.faces())
[3, 3, 3, 3, 3, 3, 3, 5]
```

sage.graphs.generators.random.**RandomUnitDiskGraph**(*n*, *radius=0.1*, *side=1*, *seed=None*, *immutable=False*)

Return a random unit disk graph of order n.

A unit disk graph is the intersection graph of a family of unit disks in the Euclidean plane. That is a graph with one vertex per disk of the family and an edge between two vertices whenever they lie within a unit distance of each other. See the Wikipedia article Unit_disk_graph for more details.

INPUT:

- `n` – number of nodes
- `radius` – float (default: 0.1); two vertices at distance less than `radius` are connected by an edge
- `side` – float (default: `1`); indicate the side of the area in which the points are drawn
- `seed` – seed of the random number generator
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

When using twice the same seed, the vertices get the same positions:

```
sage: # needs scipy
sage: from sage.misc.randstate import current_randstate
sage: seed = current_randstate().seed()
sage: G = graphs.RandomUnitDiskGraph(20, radius=.5, side=1, seed=seed)
sage: H = graphs.RandomUnitDiskGraph(20, radius=.2, side=1, seed=seed)
sage: H.is_subgraph(G, induced=False)
```

(continues on next page)

(continued from previous page)

```
True
sage: H.size() <= G.size()
True
sage: Gpos = G.get_pos()
sage: Hpos = H.get_pos()
sage: all(Gpos[u] == Hpos[u] for u in G)
True
```

```
>>> from sage.all import *
>>> # needs scipy
>>> from sage.misc.randstate import current_randstate
>>> seed = current_randstate().seed()
>>> G = graphs.RandomUnitDiskGraph(Integer(20), radius=RealNumber('.5'),␣
↪side=Integer(1), seed=seed)
>>> H = graphs.RandomUnitDiskGraph(Integer(20), radius=RealNumber('.2'),␣
↪side=Integer(1), seed=seed)
>>> H.is_subgraph(G, induced=False)
True
>>> H.size() <= G.size()
True
>>> Gpos = G.get_pos()
>>> Hpos = H.get_pos()
>>> all(Gpos[u] == Hpos[u] for u in G)
True
```

When the radius is more than $\sqrt{2\text{side}}$, the graph is a clique:

```
sage: G = graphs.RandomUnitDiskGraph(10, radius=2, side=1)                    #␣
↪needs scipy
sage: G.is_clique()                                                           #␣
↪needs scipy
True
```

```
>>> from sage.all import *
>>> G = graphs.RandomUnitDiskGraph(Integer(10), radius=Integer(2),␣
↪side=Integer(1))                        # needs scipy
>>> G.is_clique()                                                             #␣
↪needs scipy
True
```

sage.graphs.generators.random.**blossoming_contour**(*t*, *shift=0*, *seed=None*)

Return a random blossoming of a binary tree t, as a contour word.

This is doing several things simultaneously:

- complete the binary tree, by adding leaves labelled `xb`,
- add a vertex labelled `n` at the middle of every inner edge, with a leaf labelled `x` either on the left or on the right (at random),
- number all vertices (but not leaves) by integers starting from $shift$,
- compute the counter-clockwise contour word of the result.

Initial vertices receive the label `i`.

This is an auxiliary function, used for the generation of random planar bicubic maps.

INPUT:

- `t` – a binary tree (non-empty)
- `shift` – integer (default: 0); used as a starting index

OUTPUT: contour word of a random blossoming of t

EXAMPLES:

```
sage: from sage.graphs.generators.random import blossoming_contour
sage: print(blossoming_contour(BinaryTrees(1).an_element()))
[('i', 0), ('xb',), ('i', 0), ('xb',), ('i', 0)]

sage: t = BinaryTrees(2).random_element()                                   #␣
↪needs sage.combinat
sage: print(blossoming_contour(t))  # random                                #␣
↪needs sage.combinat
[('i', 0), ('xb',), ('i', 0), ('n', 2), ('i', 1), ('xb',), ('i', 1),
('xb',), ('i', 1), ('n', 2), ('x',), ('n', 2), ('i', 0)]

sage: w = blossoming_contour(BinaryTrees(3).random_element()); len(w)       #␣
↪needs sage.combinat
21
sage: w.count(('xb',))                                                      #␣
↪needs sage.combinat
4
sage: w.count(('x',))                                                       #␣
↪needs sage.combinat
2
```

```
>>> from sage.all import *
>>> from sage.graphs.generators.random import blossoming_contour
>>> print(blossoming_contour(BinaryTrees(Integer(1)).an_element()))
[('i', 0), ('xb',), ('i', 0), ('xb',), ('i', 0)]

>>> t = BinaryTrees(Integer(2)).random_element()                            ␣
↪     # needs sage.combinat
>>> print(blossoming_contour(t))  # random                                  #␣
↪needs sage.combinat
[('i', 0), ('xb',), ('i', 0), ('n', 2), ('i', 1), ('xb',), ('i', 1),
('xb',), ('i', 1), ('n', 2), ('x',), ('n', 2), ('i', 0)]

>>> w = blossoming_contour(BinaryTrees(Integer(3)).random_element()); len(w)  ␣
↪     # needs sage.combinat
21
>>> w.count(('xb',))                                                        #␣
↪needs sage.combinat
4
>>> w.count(('x',))                                                         #␣
↪needs sage.combinat
2
```

sage.graphs.generators.random.**connecting_nodes**(T, l)

Return a list of the vertex sets of n randomly chosen subtrees of T.

This method is part of *RandomChordalGraph()*.

ALGORITHM:

For each subtree T_i, we first select k_i nodes of T, where k_i is a random integer from a Poisson distribution with mean l. T_i is then generated to be the minimal subtree that contains the selected k_i nodes. This implies that a subtree will most likely have many more nodes than those selected initially, and this must be taken into consideration when choosing l.

See [SHET2018] for more details.

INPUT:

- `T` – a tree
- `l` – a strictly positive real number; mean of a Poisson distribution

EXAMPLES:

```
sage: from sage.graphs.generators.random import connecting_nodes
sage: T = graphs.RandomTree(10)
sage: S = connecting_nodes(T, 5)                                                #␣
↪needs numpy
sage: len(S)                                                                    #␣
↪needs numpy
10
```

```
>>> from sage.all import *
>>> from sage.graphs.generators.random import connecting_nodes
>>> T = graphs.RandomTree(Integer(10))
>>> S = connecting_nodes(T, Integer(5))                                          ␣
↪     # needs numpy
>>> len(S)                                                                     #␣
↪needs numpy
10
```

sage.graphs.generators.random.**growing_subtrees**(T, k)

Return a list of the vertex sets of n randomly chosen subtrees of T.

For a tree of order n, the collection contains n subtrees with maximum order k and average order $\frac{k+1}{2}$.

This method is part of *RandomChordalGraph()*.

ALGORITHM:

For each subtree T_i, the algorithm picks a size k_i randomly from $[1, k]$. Then a random node of T is chosen as the first node of T_i. In each of the subsequent $k_i - 1$ iterations, it picks a random node in the neighborhood of T_i and adds it to T_i.

See [SHET2018] for more details.

INPUT:

- `T` – a tree
- `k` – a strictly positive integer; maximum size of a subtree

EXAMPLES:

```
sage: from sage.graphs.generators.random import growing_subtrees
sage: T = graphs.RandomTree(10)
sage: S = growing_subtrees(T, 5)
sage: len(S)
10
```

```
>>> from sage.all import *
>>> from sage.graphs.generators.random import growing_subtrees
>>> T = graphs.RandomTree(Integer(10))
>>> S = growing_subtrees(T, Integer(5))
>>> len(S)
10
```

sage.graphs.generators.random.**pruned_tree**(T, f, s)

Return a list of the vertex sets of n randomly chosen subtrees of T.

This method is part of *RandomChordalGraph()*.

ALGORITHM:

For each subtree T_i, it randomly selects a fraction f of the edges on the tree and removes them. The number of edges to delete, say l, is calculated as $\lfloor((n-1)f\rfloor$, which will leave $l+1$ subtrees in total. Then, it determines the sizes of the $l+1$ subtrees and stores the distinct values. Finally, it picks a random size k_i from the set of largest $100(1-s)\%$ of distinct values, and randomly chooses a subtree with size k_i.

See [SHET2018] for more details.

INPUT:

- `T` – a tree
- `f` – a rational number; the edge deletion fraction. This value must be chosen in $[0..1]$
- `s` – a real number between 0 and 1; selection barrier for the size of trees

EXAMPLES:

```
sage: from sage.graphs.generators.random import pruned_tree
sage: T = graphs.RandomTree(11)
sage: S = pruned_tree(T, 1/10, 0.5)
sage: len(S)
11
```

```
>>> from sage.all import *
>>> from sage.graphs.generators.random import pruned_tree
>>> T = graphs.RandomTree(Integer(11))
>>> S = pruned_tree(T, Integer(1)/Integer(10), RealNumber('0.5'))
>>> len(S)
11
```

2.16 Various small graphs

The methods defined here appear in *sage.graphs.graph_generators*.

sage.graphs.generators.smallgraphs.**Balaban10Cage**(*embedding=1*, *immutable=False*)

Return the Balaban 10-cage.

The Balaban 10-cage is a 3-regular graph with 70 vertices and 105 edges. See the Wikipedia article Balaban_10-cage.

The default embedding gives a deeper understanding of the graph's automorphism group. It is divided into 4 layers (each layer being a set of points at equal distance from the drawing's center). From outside to inside:

- L1: The outer layer (vertices which are the furthest from the origin) is actually the disjoint union of two cycles of length 10.
- L2: The second layer is an independent set of 20 vertices.
- L3: The third layer is a matching on 10 vertices.
- L4: The inner layer (vertices which are the closest from the origin) is also the disjoint union of two cycles of length 10.

This graph is not vertex-transitive, and its vertices are partitioned into 3 orbits: L2, L3, and the union of L1 of L4 whose elements are equivalent.

INPUT:

- `embedding` – integer (default: 1); two embeddings are available, and can be selected by setting `embedding` to be either 1 or 2
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.Balaban10Cage()
sage: g.girth()
10
sage: g.chromatic_number()
2
sage: g.diameter()
6
sage: g.is_hamiltonian()                                                    #␣
↪needs sage.numerical.mip
True
sage: g.show(figsize=[10,10])            # long time                        #␣
↪needs sage.plot
```

```
>>> from sage.all import *
>>> g = graphs.Balaban10Cage()
>>> g.girth()
10
>>> g.chromatic_number()
2
>>> g.diameter()
6
>>> g.is_hamiltonian()                                                      #␣
↪needs sage.numerical.mip
True
>>> g.show(figsize=[Integer(10),Integer(10)])           # long time          ␣
↪              # needs sage.plot
```

sage.graphs.generators.smallgraphs.**Balaban11Cage**(*embedding=1*, *immutable=False*)

Return the Balaban 11-cage.

For more information, see the Wikipedia article Balaban_11-cage.

INPUT:

- `embedding` – integer (default: 1); three embeddings are available, and can be selected by setting `embedding` to be 1, 2, or 3
  - The first embedding is the one appearing on page 9 of the Fifth Annual Graph Drawing Contest report [EMMN1998]. It separates vertices based on their eccentricity (see *`eccentricity()`*).
  - The second embedding has been produced just for Sage and is meant to emphasize the automorphism group's 6 orbits.
  - The last embedding is the default one produced by the *`LCFGraph()`* constructor.
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

Note

The vertex labeling changes according to the value of `embedding=1`.

EXAMPLES:

Basic properties:

```
sage: g = graphs.Balaban11Cage()
sage: g.order()
112
sage: g.size()
168
sage: g.girth()
11
sage: g.diameter()
8
sage: g.automorphism_group().cardinality()                                  #␣
↪needs sage.groups
64
```

```
>>> from sage.all import *
>>> g = graphs.Balaban11Cage()
>>> g.order()
112
>>> g.size()
168
>>> g.girth()
11
>>> g.diameter()
8
>>> g.automorphism_group().cardinality()                                    #␣
↪needs sage.groups
64
```

Our many embeddings:

```
sage: g1 = graphs.Balaban11Cage(embedding=1)
sage: g2 = graphs.Balaban11Cage(embedding=2)
sage: g3 = graphs.Balaban11Cage(embedding=3)
```

(continues on next page)

(continued from previous page)

```
sage: g1.show(figsize=[10,10])          # long time                          #␣
↪needs sage.plot
sage: g2.show(figsize=[10,10])          # long time                          #␣
↪needs sage.plot
sage: g3.show(figsize=[10,10])          # long time                          #␣
↪needs sage.plot
```

```
>>> from sage.all import *
>>> g1 = graphs.Balaban11Cage(embedding=Integer(1))
>>> g2 = graphs.Balaban11Cage(embedding=Integer(2))
>>> g3 = graphs.Balaban11Cage(embedding=Integer(3))
>>> g1.show(figsize=[Integer(10),Integer(10)])          # long time          ␣
↪              # needs sage.plot
>>> g2.show(figsize=[Integer(10),Integer(10)])          # long time          ␣
↪              # needs sage.plot
>>> g3.show(figsize=[Integer(10),Integer(10)])          # long time          ␣
↪              # needs sage.plot
```

Proof that the embeddings are the same graph:

```
sage: g1.is_isomorphic(g2)  # g2 and g3 are obviously isomorphic
True
```

```
>>> from sage.all import *
>>> g1.is_isomorphic(g2)  # g2 and g3 are obviously isomorphic
True
```

sage.graphs.generators.smallgraphs.**BidiakisCube**(*immutable=False*)

Return the Bidiakis cube.

For more information, see the Wikipedia article Bidiakis_cube.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

The Bidiakis cube is a 3-regular graph having 12 vertices and 18 edges. This means that each vertex has a degree of 3:

```
sage: g = graphs.BidiakisCube(); g
Bidiakis cube: Graph on 12 vertices
sage: g.show()                          # long time                          #␣
↪needs sage.plot
sage: g.order()
12
sage: g.size()
18
sage: g.is_regular(3)
True
```

```
>>> from sage.all import *
>>> g = graphs.BidiakisCube(); g
```

(continues on next page)

(continued from previous page)

```
Bidiakis cube: Graph on 12 vertices
>>> g.show()                              # long time                              #␣
↪needs sage.plot
>>> g.order()
12
>>> g.size()
18
>>> g.is_regular(Integer(3))
True
```

It is a Hamiltonian graph with diameter 3 and girth 4:

```
sage: g.is_hamiltonian()                                                          #␣
↪needs sage.numerical.mip
True
sage: g.diameter()
3
sage: g.girth()
4
```

```
>>> from sage.all import *
>>> g.is_hamiltonian()                                                            #␣
↪needs sage.numerical.mip
True
>>> g.diameter()
3
>>> g.girth()
4
```

It is a planar graph with characteristic polynomial $(x-3)(x-2)(x^4)(x+1)(x+2)(x^2+x-4)^2$ and chromatic number 3:

```
sage: g.is_planar()
True
sage: char_poly = g.characteristic_polynomial()                                   #␣
↪needs sage.modules
sage: x = char_poly.parent()('x')                                                 #␣
↪needs sage.modules
sage: char_poly == (x - 3) * (x - 2) * (x^4) * (x + 1) * (x + 2) * (x^2 + x - 4)^
↪2          # needs sage.modules
True
sage: g.chromatic_number()                                                        #␣
↪needs sage.modules
3
```

```
>>> from sage.all import *
>>> g.is_planar()
True
>>> char_poly = g.characteristic_polynomial()                                     #␣
↪needs sage.modules
>>> x = char_poly.parent()('x')                                                   #␣
↪needs sage.modules
```

(continues on next page)

(continued from previous page)

```
>>> char_poly == (x - Integer(3)) * (x - Integer(2)) * (x**Integer(4)) * (x +␣
↪Integer(1)) * (x + Integer(2)) * (x**Integer(2) + x - Integer(4))**Integer(2)  ␣
↪       # needs sage.modules
True
>>> g.chromatic_number()                                                        #␣
↪needs sage.modules
3
```

sage.graphs.generators.smallgraphs.**BiggsSmithGraph**(*embedding=1*, *immutable=False*)

Return the Biggs-Smith graph.

For more information, see the Wikipedia article Biggs-Smith_graph.

INPUT:

- `embedding` – integer (default: 1); two embeddings are available, and can be selected by setting `embedding` to be 1 or 2
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

Basic properties:

```
sage: g = graphs.BiggsSmithGraph()
sage: g.order()
102
sage: g.size()
153
sage: g.girth()
9
sage: g.diameter()
7
sage: g.automorphism_group().cardinality()      # long time
2448
sage: g.show(figsize=[10, 10])          # long time                            #␣
↪needs sage.plot
```

```
>>> from sage.all import *
>>> g = graphs.BiggsSmithGraph()
>>> g.order()
102
>>> g.size()
153
>>> g.girth()
9
>>> g.diameter()
7
>>> g.automorphism_group().cardinality()      # long time
2448
>>> g.show(figsize=[Integer(10), Integer(10)])          # long time             ␣
↪             # needs sage.plot
```

The other embedding:

```
sage: graphs.BiggsSmithGraph(embedding=2).show()        # long time
```

```
>>> from sage.all import *
>>> graphs.BiggsSmithGraph(embedding=Integer(2)).show()        # long time
```

sage.graphs.generators.smallgraphs.**BlanusaFirstSnarkGraph**(*immutable=False*)

Return the first Blanusa Snark Graph.

The Blanusa graphs are two snarks on 18 vertices and 27 edges. For more information on them, see the Wikipedia article Blanusa_snarks.

See also

- *BlanusaSecondSnarkGraph()*.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.BlanusaFirstSnarkGraph()
sage: g.order()
18
sage: g.size()
27
sage: g.diameter()
4
sage: g.girth()
5
sage: g.automorphism_group().cardinality()                                  #␣
↪needs sage.groups
8
```

```
>>> from sage.all import *
>>> g = graphs.BlanusaFirstSnarkGraph()
>>> g.order()
18
>>> g.size()
27
>>> g.diameter()
4
>>> g.girth()
5
>>> g.automorphism_group().cardinality()                                  #␣
↪needs sage.groups
8
```

sage.graphs.generators.smallgraphs.**BlanusaSecondSnarkGraph**(*immutable=False*)

Return the second Blanusa Snark Graph.

The Blanusa graphs are two snarks on 18 vertices and 27 edges. For more information on them, see the Wikipedia article Blanusa_snarks.

See also

- *BlanusaFirstSnarkGraph()*.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.BlanusaSecondSnarkGraph()
sage: g.order()
18
sage: g.size()
27
sage: g.diameter()
4
sage: g.girth()
5
sage: g.automorphism_group().cardinality()                                  #␣
↪needs sage.groups
4
```

```
>>> from sage.all import *
>>> g = graphs.BlanusaSecondSnarkGraph()
>>> g.order()
18
>>> g.size()
27
>>> g.diameter()
4
>>> g.girth()
5
>>> g.automorphism_group().cardinality()                                    #␣
↪needs sage.groups
4
```

sage.graphs.generators.smallgraphs.**BrinkmannGraph**(*immutable=False*)

Return the Brinkmann graph.

For more information, see the Wikipedia article Brinkmann_graph.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

The Brinkmann graph is a 4-regular graph having 21 vertices and 42 edges. This means that each vertex has degree 4:

```
sage: G = graphs.BrinkmannGraph(); G
Brinkmann graph: Graph on 21 vertices
sage: G.show()                              # long time                     #␣
↪needs sage.plot
```

(continues on next page)

(continued from previous page)

```
sage: G.order()
21
sage: G.size()
42
sage: G.is_regular(4)
True
```

```
>>> from sage.all import *
>>> G = graphs.BrinkmannGraph(); G
Brinkmann graph: Graph on 21 vertices
>>> G.show()                              # long time                              #␣
↪needs sage.plot
>>> G.order()
21
>>> G.size()
42
>>> G.is_regular(Integer(4))
True
```

It is an Eulerian graph with radius 3, diameter 3, and girth 5:

```
sage: G.is_eulerian()
True
sage: G.radius()
3
sage: G.diameter()
3
sage: G.girth()
5
```

```
>>> from sage.all import *
>>> G.is_eulerian()
True
>>> G.radius()
3
>>> G.diameter()
3
>>> G.girth()
5
```

The Brinkmann graph is also Hamiltonian with chromatic number 4:

```
sage: G.is_hamiltonian()                                                          #␣
↪needs sage.numerical.mip
True
sage: G.chromatic_number()
4
```

```
>>> from sage.all import *
>>> G.is_hamiltonian()                                                            #␣
↪needs sage.numerical.mip
True
```

(continues on next page)

(continued from previous page)

```
>>> G.chromatic_number()
4
```

Its automorphism group is isomorphic to D_7:

```
sage: ag = G.automorphism_group()                                              #␣
↪needs sage.groups
sage: ag.is_isomorphic(DihedralGroup(7))                                       #␣
↪needs sage.groups
True
```

```
>>> from sage.all import *
>>> ag = G.automorphism_group()                                                #␣
↪needs sage.groups
>>> ag.is_isomorphic(DihedralGroup(Integer(7)))                                 ␣
↪     # needs sage.groups
True
```

sage.graphs.generators.smallgraphs.**BrouwerHaemersGraph**(*immutable=False*)

Return the Brouwer-Haemers Graph.

The Brouwer-Haemers is the only strongly regular graph of parameters $(81, 20, 1, 6)$. It is build in Sage as the Affine Orthogonal graph $VO^-(6, 3)$. For more information on this graph, see its corresponding page on Andries Brouwer's website.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.BrouwerHaemersGraph(); g                                      #␣
↪needs sage.modules
Brouwer-Haemers: Graph on 81 vertices
```

```
>>> from sage.all import *
>>> g = graphs.BrouwerHaemersGraph(); g                                        #␣
↪needs sage.modules
Brouwer-Haemers: Graph on 81 vertices
```

It is indeed strongly regular with parameters $(81, 20, 1, 6)$:

```
sage: g.is_strongly_regular(parameters=True)     # long time                   #␣
↪needs sage.modules sage.rings.finite_rings
(81, 20, 1, 6)
```

```
>>> from sage.all import *
>>> g.is_strongly_regular(parameters=True)     # long time                     #␣
↪needs sage.modules sage.rings.finite_rings
(81, 20, 1, 6)
```

Its has as eigenvalues $20, 2$ and -7:

```
sage: set(g.spectrum()) == {20,2,-7}                                    #␣
↪needs sage.modules sage.rings.finite_rings
True
```

```
>>> from sage.all import *
>>> set(g.spectrum()) == {Integer(20),Integer(2),-Integer(7)}           ␣
↪                      # needs sage.modules sage.rings.finite_rings
True
```

sage.graphs.generators.smallgraphs.**BuckyBall**(*immutable=False*)

Return the Bucky Ball graph.

This graph is a 3-regular 60-vertex planar graph. Its vertices and edges correspond precisely to the carbon atoms and bonds in buckminsterfullerene. When embedded on a sphere, its 12 pentagon and 20 hexagon faces are arranged exactly as the sections of a soccer ball.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

The Bucky Ball is planar:

```
sage: g = graphs.BuckyBall()
sage: g.is_planar()
True
```

```
>>> from sage.all import *
>>> g = graphs.BuckyBall()
>>> g.is_planar()
True
```

The Bucky Ball can also be created by extracting the 1-skeleton of the Bucky Ball polyhedron, but this is much slower:

```
sage: g = polytopes.buckyball().vertex_graph()
sage: g.remove_loops()
sage: h = graphs.BuckyBall()
sage: g.is_isomorphic(h)
True
```

```
>>> from sage.all import *
>>> g = polytopes.buckyball().vertex_graph()
>>> g.remove_loops()
>>> h = graphs.BuckyBall()
>>> g.is_isomorphic(h)
True
```

The graph is returned along with an attractive embedding:

```
sage: g = graphs.BuckyBall()  # long time
sage: g.plot(vertex_labels=False, vertex_size=10).show()        # long time,␣
↪needs sage.plot
```

```
>>> from sage.all import *
>>> g = graphs.BuckyBall()  # long time
>>> g.plot(vertex_labels=False, vertex_size=Integer(10)).show()         # long␣
↪time, needs sage.plot
```

sage.graphs.generators.smallgraphs.**CameronGraph**(*immutable=False*)

Return the Cameron graph.

The Cameron graph is strongly regular with parameters $v = 231, k = 30, \lambda = 9, \mu = 3$.

For more information on the Cameron graph, see https://www.win.tue.nl/~aeb/graphs/Cameron.html.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.CameronGraph()
sage: g.order()
231
sage: g.size()
3465
sage: g.is_strongly_regular(parameters=True)     # long time
(231, 30, 9, 3)
```

```
>>> from sage.all import *
>>> g = graphs.CameronGraph()
>>> g.order()
231
>>> g.size()
3465
>>> g.is_strongly_regular(parameters=True)     # long time
(231, 30, 9, 3)
```

sage.graphs.generators.smallgraphs.**Cell120**(*immutable=False*)

Return the 120-Cell graph.

This is the adjacency graph of the 120-cell. It has 600 vertices and 1200 edges. For more information, see the Wikipedia article 120-cell.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: # long time, needs sage.rings.number_field
sage: g = graphs.Cell120()
sage: g.size()
1200
sage: g.is_regular(4)
True
sage: g.is_vertex_transitive()
True
```

```
>>> from sage.all import *
>>> # long time, needs sage.rings.number_field
>>> g = graphs.Cell120()
>>> g.size()
1200
>>> g.is_regular(Integer(4))
True
>>> g.is_vertex_transitive()
True
```

sage.graphs.generators.smallgraphs.**Cell600**(*embedding=1*, *immutable=False*)

Return the 600-Cell graph.

This is the adjacency graph of the 600-cell. It has 120 vertices and 720 edges. For more information, see the Wikipedia article 600-cell.

INPUT:

- `embedding` – integer (default: 1); two different embeddings for a plot
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: # long time, needs sage.rings.number_field
sage: g = graphs.Cell600()
sage: g.size()
720
sage: g.is_regular(12)
True
sage: g.is_vertex_transitive()
True
```

```
>>> from sage.all import *
>>> # long time, needs sage.rings.number_field
>>> g = graphs.Cell600()
>>> g.size()
720
>>> g.is_regular(Integer(12))
True
>>> g.is_vertex_transitive()
True
```

sage.graphs.generators.smallgraphs.**ChvatalGraph**(*immutable=False*)

Return the Chvatal graph.

Chvatal graph is one of the few known graphs to satisfy Grunbaum's conjecture that for every m, n, there is an m-regular, m-chromatic graph of girth at least n. For more information, see the Wikipedia article Chv%C3%A1tal_graph.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

The Chvatal graph has 12 vertices and 24 edges. It is a 4-regular, 4-chromatic graph with radius 2, diameter 2, and girth 4:

```
sage: G = graphs.ChvatalGraph(); G
Chvatal graph: Graph on 12 vertices
sage: G.order(); G.size()
12
24
sage: G.degree()
[4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4]
sage: G.chromatic_number()
4
sage: G.radius(); G.diameter(); G.girth()
2
2
4
```

```
>>> from sage.all import *
>>> G = graphs.ChvatalGraph(); G
Chvatal graph: Graph on 12 vertices
>>> G.order(); G.size()
12
24
>>> G.degree()
[4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4]
>>> G.chromatic_number()
4
>>> G.radius(); G.diameter(); G.girth()
2
2
4
```

sage.graphs.generators.smallgraphs.**ClebschGraph**(*immutable=False*)

Return the Clebsch graph.

See the Wikipedia article Clebsch_graph for more information.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.ClebschGraph()
sage: g.automorphism_group().cardinality()                                  #␣
↪needs sage.groups
1920
sage: g.girth()
4
sage: g.chromatic_number()
4
sage: g.diameter()
2
sage: g.show(figsize=[10, 10])          # long time                         #␣
↪needs sage.plot
```

```
>>> from sage.all import *
>>> g = graphs.ClebschGraph()
>>> g.automorphism_group().cardinality()                                    #␣
↪needs sage.groups
1920
>>> g.girth()
4
>>> g.chromatic_number()
4
>>> g.diameter()
2
>>> g.show(figsize=[Integer(10), Integer(10)])          # long time          ␣
↪              # needs sage.plot
```

sage.graphs.generators.smallgraphs.**CoxeterGraph**(*immutable=False*)

Return the Coxeter graph.

See the Wikipedia article Coxeter_graph.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.CoxeterGraph()
sage: g.automorphism_group().cardinality()                                  #␣
↪needs sage.groups
336
sage: g.girth()
7
sage: g.chromatic_number()
3
sage: g.diameter()
4
sage: g.show(figsize=[10, 10])          # long time                         #␣
↪needs sage.plot
```

```
>>> from sage.all import *
>>> g = graphs.CoxeterGraph()
>>> g.automorphism_group().cardinality()                                    #␣
↪needs sage.groups
336
>>> g.girth()
7
>>> g.chromatic_number()
3
>>> g.diameter()
4
>>> g.show(figsize=[Integer(10), Integer(10)])          # long time          ␣
↪              # needs sage.plot
```

sage.graphs.generators.smallgraphs.**CubeplexGraph**(*embedding='LM'*, *immutable=False*)

Return the Cubeplex graph.

The Cubeplex graph is the cubic hamiltonian graph of order 12 that corresponds to the graph labeled as Γ_1 in Fischer and Little [FiLi2001]. It has LCF notation $[-6, -5, -3, -6, 3, 5, -6, -3, 5, -6, -5, 3]$.

The Fischer-Little Theorem [FiLi2001] may be stated as follows [LM2024]:

A near-bipartite graph is non-Pfaffian if and only if it contains one of the graphs $K_{3,3}$, Γ_1 and Γ_2 as an S-minor.

Norine and Thomas [NT2007] use the term `Cubeplex` to describe one of the 12-vertex cubic graphs, Γ_1 and Γ_2, as defined by Fischer and Little [FiLi2001]. However, the figure in their paper that supposedly provides embeddings for the graphs labeled Cubeplex and Twinplex actually shows both embeddings corresponding to Fischer and Little's Γ_1, which is the Cubeplex graph. Followingly, for `embedding='NT'`, we present only the embedding that is shown by the labeling `Cubeplex` in the paper of Norine and Thomas [NT2007].

PLOTTING:

Upon construction, the position dictionary is filled to override the spring-layout algorithm. For different values of the parameter `embedding`, the Cubeplex graph is displayed as it is mentioned in the respective paper/ book.

INPUT:

- `embedding` – string (default: `'LM'`)
  - `'LM'` displays the embedding as shown for Γ_1 by Lucchesi and Murty [LM2024]
  - `'FL'` displays the embedding as shown for Γ_1 by Fischer and Little [FiLi2001]
  - `'NT'` displays the embedding as shown for the `Cubeplex` by Norine and Thomas [NT2007]
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

OUTPUT:

- `G` – the Cubeplex graph; note that a `ValueError` is returned if `embedding` is none of `'FT'`, `'NT'` or `'LM'`

EXAMPLES:

Construct and show the Cubeplex graph:

```
sage: g = graphs.CubeplexGraph()
sage: g.name()
'Cubeplex Graph'
sage: g.order()
12
sage: g.size()
18
sage: g.girth()
4
sage: g.diameter()
3
sage: g.is_hamiltonian()
True
sage: g.crossing_number()
1
sage: g.show()                              # long time                              #␣
↪needs sage.plot
```

```
>>> from sage.all import *
>>> g = graphs.CubeplexGraph()
>>> g.name()
'Cubeplex Graph'
>>> g.order()
```

(continues on next page)

(continued from previous page)

```
12
>>> g.size()
18
>>> g.girth()
4
>>> g.diameter()
3
>>> g.is_hamiltonian()
True
>>> g.crossing_number()
1
>>> g.show()                              # long time                              #␣
↪needs sage.plot
```

See also

TwinplexGraph()

AUTHORS:

- Janmenjaya Panda (2024-08-03)

sage.graphs.generators.smallgraphs.**DejterGraph**(*immutable=False*)

Return the Dejter graph.

The Dejter graph is obtained from the binary 7-cube by deleting a copy of the Hamming code of length 7. It is 6-regular, with 112 vertices and 336 edges. For more information, see the Wikipedia article Dejter_graph.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.DejterGraph(); g                                                #␣
↪needs sage.rings.finite_rings
Dejter Graph: Graph on 112 vertices
sage: g.is_regular(k=6)                                                          #␣
↪needs sage.rings.finite_rings
True
sage: g.girth()                                                                  #␣
↪needs sage.rings.finite_rings
4
```

```
>>> from sage.all import *
>>> g = graphs.DejterGraph(); g                                                  #␣
↪needs sage.rings.finite_rings
Dejter Graph: Graph on 112 vertices
>>> g.is_regular(k=Integer(6))                                                    ␣
↪     # needs sage.rings.finite_rings
True
>>> g.girth()                                                                    #␣
↪needs sage.rings.finite_rings
4
```

sage.graphs.generators.smallgraphs.**DesarguesGraph**(*immutable=False*)

Return the Desargues graph.

PLOTTING: The layout chosen is the same as on the cover of [Har1969].

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: D = graphs.DesarguesGraph()
sage: L = graphs.LCFGraph(20,[5,-5,9,-9],5)
sage: D.is_isomorphic(L)
True
sage: D.show()                              # long time                              #␣
↪needs sage.plot
```

```
>>> from sage.all import *
>>> D = graphs.DesarguesGraph()
>>> L = graphs.LCFGraph(Integer(20),[Integer(5),-Integer(5),Integer(9),-
↪Integer(9)],Integer(5))
>>> D.is_isomorphic(L)
True
>>> D.show()                              # long time                              #␣
↪needs sage.plot
```

sage.graphs.generators.smallgraphs.**DoubleStarSnark**(*immutable=False*)

Return the double star snark.

The double star snark is a 3-regular graph on 30 vertices. See the Wikipedia article Double-star_snark.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.DoubleStarSnark()
sage: g.order()
30
sage: g.size()
45
sage: g.chromatic_number()
3
sage: g.is_hamiltonian()                                                           #␣
↪needs sage.numerical.mip
False
sage: g.automorphism_group().cardinality()                                         #␣
↪needs sage.groups
80
sage: g.show()                                                                     #␣
↪needs sage.plot
```

```
>>> from sage.all import *
>>> g = graphs.DoubleStarSnark()
>>> g.order()
```

(continues on next page)

(continued from previous page)

```
30
>>> g.size()
45
>>> g.chromatic_number()
3
>>> g.is_hamiltonian()                                                    #␣
↪needs sage.numerical.mip
False
>>> g.automorphism_group().cardinality()                                  #␣
↪needs sage.groups
80
>>> g.show()                                                              #␣
↪needs sage.plot
```

sage.graphs.generators.smallgraphs.**DurerGraph**(*immutable=False*)

Return the Dürer graph.

For more information, see the Wikipedia article D%C3%BCrer_graph.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

The Dürer graph is named after Albrecht Dürer. It is a planar graph with 12 vertices and 18 edges:

```
sage: G = graphs.DurerGraph(); G
Durer graph: Graph on 12 vertices
sage: G.is_planar()
True
sage: G.order()
12
sage: G.size()
18
```

```
>>> from sage.all import *
>>> G = graphs.DurerGraph(); G
Durer graph: Graph on 12 vertices
>>> G.is_planar()
True
>>> G.order()
12
>>> G.size()
18
```

The Dürer graph has chromatic number 3, diameter 4, and girth 3:

```
sage: G.chromatic_number()
3
sage: G.diameter()
4
sage: G.girth()
3
```

```
>>> from sage.all import *
>>> G.chromatic_number()
3
>>> G.diameter()
4
>>> G.girth()
3
```

Its automorphism group is isomorphic to D_6:

```
sage: ag = G.automorphism_group()                                              #␣
↪needs sage.groups
sage: ag.is_isomorphic(DihedralGroup(6))                                       #␣
↪needs sage.groups
True
```

```
>>> from sage.all import *
>>> ag = G.automorphism_group()                                                #␣
↪needs sage.groups
>>> ag.is_isomorphic(DihedralGroup(Integer(6)))                                 ␣
↪     # needs sage.groups
True
```

sage.graphs.generators.smallgraphs.**DyckGraph**(*immutable=False*)

Return the Dyck graph.

For more information, see the MathWorld article on the Dyck graph or the Wikipedia article Dyck_graph.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

The Dyck graph was defined by Walther von Dyck in 1881. It has 32 vertices and 48 edges, and is a cubic graph (regular of degree 3):

```
sage: G = graphs.DyckGraph(); G
Dyck graph: Graph on 32 vertices
sage: G.order()
32
sage: G.size()
48
sage: G.is_regular()
True
sage: G.is_regular(3)
True
```

```
>>> from sage.all import *
>>> G = graphs.DyckGraph(); G
Dyck graph: Graph on 32 vertices
>>> G.order()
32
>>> G.size()
48
```

(continues on next page)

(continued from previous page)

```
>>> G.is_regular()
True
>>> G.is_regular(Integer(3))
True
```

It is non-planar and Hamiltonian, as well as bipartite (making it a bicubic graph):

```
sage: G.is_planar()
False
sage: G.is_hamiltonian()                                                    #␣
↪needs sage.numerical.mip
True
sage: G.is_bipartite()
True
```

```
>>> from sage.all import *
>>> G.is_planar()
False
>>> G.is_hamiltonian()                                                      #␣
↪needs sage.numerical.mip
True
>>> G.is_bipartite()
True
```

It has radius 5, diameter 5, and girth 6:

```
sage: G.radius()
5
sage: G.diameter()
5
sage: G.girth()
6
```

```
>>> from sage.all import *
>>> G.radius()
5
>>> G.diameter()
5
>>> G.girth()
6
```

Its chromatic number is 2 and its automorphism group is of order 192:

```
sage: G.chromatic_number()
2
sage: G.automorphism_group().cardinality()                                  #␣
↪needs sage.groups
192
```

```
>>> from sage.all import *
>>> G.chromatic_number()
2
```

(continues on next page)

(continued from previous page)

```
>>> G.automorphism_group().cardinality()                                    #␣
↪needs sage.groups
192
```

It is a non-integral graph as it has irrational eigenvalues:

```
sage: G.characteristic_polynomial().factor()                                #␣
↪needs sage.libs.pari sage.modules
(x - 3) * (x + 3) * (x - 1)^9 * (x + 1)^9 * (x^2 - 5)^6
```

```
>>> from sage.all import *
>>> G.characteristic_polynomial().factor()                                  #␣
↪needs sage.libs.pari sage.modules
(x - 3) * (x + 3) * (x - 1)^9 * (x + 1)^9 * (x^2 - 5)^6
```

It is a toroidal graph, and its embedding on a torus is dual to an embedding of the Shrikhande graph (*ShrikhandeGraph*).

sage.graphs.generators.smallgraphs.**EllinghamHorton54Graph**(*immutable=False*)

Return the Ellingham-Horton 54-graph.

For more information, see the Wikipedia article Ellingham-Horton_graph.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

This graph is 3-regular:

```
sage: g = graphs.EllinghamHorton54Graph()
sage: g.is_regular(k=3)
True
```

```
>>> from sage.all import *
>>> g = graphs.EllinghamHorton54Graph()
>>> g.is_regular(k=Integer(3))
True
```

It is 3-connected and bipartite:

```
sage: g.vertex_connectivity()   # not tested - too long
3
sage: g.is_bipartite()
True
```

```
>>> from sage.all import *
>>> g.vertex_connectivity()   # not tested - too long
3
>>> g.is_bipartite()
True
```

It is not Hamiltonian:

```
sage: g.is_hamiltonian()                # not tested                                #␣
↪needs sage.numerical.mip
False
```

```
>>> from sage.all import *
>>> g.is_hamiltonian()                # not tested                                #␣
↪needs sage.numerical.mip
False
```

… and it has a nice drawing

```
sage: g.show(figsize=[10, 10])  # not tested - too long
```

```
>>> from sage.all import *
>>> g.show(figsize=[Integer(10), Integer(10)])  # not tested - too long
```

sage.graphs.generators.smallgraphs.**EllinghamHorton78Graph**(*immutable=False*)

Return the Ellingham-Horton 78-graph.

For more information, see the Wikipedia article Ellingham%E2%80%93Horton_graph

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

This graph is 3-regular:

```
sage: g = graphs.EllinghamHorton78Graph()
sage: g.is_regular(k=3)
True
```

```
>>> from sage.all import *
>>> g = graphs.EllinghamHorton78Graph()
>>> g.is_regular(k=Integer(3))
True
```

It is 3-connected and bipartite:

```
sage: g.vertex_connectivity()   # not tested (too long)
3
sage: g.is_bipartite()
True
```

```
>>> from sage.all import *
>>> g.vertex_connectivity()   # not tested (too long)
3
>>> g.is_bipartite()
True
```

It is not Hamiltonian:

```
sage: g.is_hamiltonian()                # not tested                                #␣
↪needs sage.numerical.mip
False
```

```
>>> from sage.all import *
>>> g.is_hamiltonian()                # not tested                                #␣
↪needs sage.numerical.mip
False
```

… and it has a nice drawing

```
sage: g.show(figsize=[10,10])    # not tested (too long)
```

```
>>> from sage.all import *
>>> g.show(figsize=[Integer(10),Integer(10)])    # not tested (too long)
```

sage.graphs.generators.smallgraphs.**ErreraGraph**(*immutable=False*)

Return the Errera graph.

For more information, see the Wikipedia article Errera_graph.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

The Errera graph is named after Alfred Errera. It is a planar graph on 17 vertices and having 45 edges:

```
sage: G = graphs.ErreraGraph(); G
Errera graph: Graph on 17 vertices
sage: G.is_planar()
True
sage: G.order()
17
sage: G.size()
45
```

```
>>> from sage.all import *
>>> G = graphs.ErreraGraph(); G
Errera graph: Graph on 17 vertices
>>> G.is_planar()
True
>>> G.order()
17
>>> G.size()
45
```

The Errera graph is Hamiltonian with radius 3, diameter 4, girth 3, and chromatic number 4:

```
sage: G.is_hamiltonian()                                                          #␣
↪needs sage.numerical.mip
True
sage: G.radius()
3
```

(continues on next page)

(continued from previous page)

```
sage: G.diameter()
4
sage: G.girth()
3
sage: G.chromatic_number()
4
```

```
>>> from sage.all import *
>>> G.is_hamiltonian()                                                    #␣
↪needs sage.numerical.mip
True
>>> G.radius()
3
>>> G.diameter()
4
>>> G.girth()
3
>>> G.chromatic_number()
4
```

Each vertex degree is either 5 or 6. That is, if f counts the number of vertices of degree 5 and s counts the number of vertices of degree 6, then $f + s$ is equal to the order of the Errera graph:

```
sage: D = G.degree_sequence()
sage: D.count(5) + D.count(6) == G.order()
True
```

```
>>> from sage.all import *
>>> D = G.degree_sequence()
>>> D.count(Integer(5)) + D.count(Integer(6)) == G.order()
True
```

The automorphism group of the Errera graph is isomorphic to the dihedral group of order 20:

```
sage: ag = G.automorphism_group()                                        #␣
↪needs sage.groups
sage: ag.is_isomorphic(DihedralGroup(10))                                #␣
↪needs sage.groups
True
```

```
>>> from sage.all import *
>>> ag = G.automorphism_group()                                          #␣
↪needs sage.groups
>>> ag.is_isomorphic(DihedralGroup(Integer(10)))                          ␣
↪     # needs sage.groups
True
```

sage.graphs.generators.smallgraphs.**F26AGraph**(*immutable=False*)

Return the F26A graph.

The F26A graph is a symmetric bipartite cubic graph with 26 vertices and 39 edges. For more information, see the Wikipedia article F26A_graph.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.F26AGraph(); g
F26A Graph: Graph on 26 vertices
sage: g.order(), g.size()
(26, 39)
sage: g.automorphism_group().cardinality()
78
sage: g.girth()
6
sage: g.is_bipartite()
True
sage: g.characteristic_polynomial().factor()
(x - 3) * (x + 3) * (x^4 - 5*x^2 + 3)^6
```

```
>>> from sage.all import *
>>> g = graphs.F26AGraph(); g
F26A Graph: Graph on 26 vertices
>>> g.order(), g.size()
(26, 39)
>>> g.automorphism_group().cardinality()
78
>>> g.girth()
6
>>> g.is_bipartite()
True
>>> g.characteristic_polynomial().factor()
(x - 3) * (x + 3) * (x^4 - 5*x^2 + 3)^6
```

sage.graphs.generators.smallgraphs.**FlowerSnark**(*immutable=False*)

Return a Flower Snark.

A flower snark has 20 vertices. It is part of the class of biconnected cubic graphs with edge chromatic number = 4, known as snarks. (i.e.: the Petersen graph). All snarks are not Hamiltonian, non-planar and have Petersen graph graph minors. See the Wikipedia article Flower_snark.

PLOTTING: Upon construction, the position dictionary is filled to override the spring-layout algorithm. By convention, the nodes are drawn 0-14 on the outer circle, and 15-19 in an inner pentagon.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES: Inspect a flower snark:

```
sage: F = graphs.FlowerSnark()
sage: F
Flower Snark: Graph on 20 vertices
sage: F.graph6_string()
'ShCGHC@?GGg@?@?Gp?K??C?CA?G?_G?Cc'
```

```
>>> from sage.all import *
>>> F = graphs.FlowerSnark()
>>> F
Flower Snark: Graph on 20 vertices
>>> F.graph6_string()
'ShCGHC@?GGg@?@?Gp?K??C?CA?G?_G?Cc'
```

Now show it:

```
sage: F.show()                              # long time                              #␣
↪needs sage.plot
```

```
>>> from sage.all import *
>>> F.show()                              # long time                              #␣
↪needs sage.plot
```

sage.graphs.generators.smallgraphs.**FolkmanGraph**(*immutable=False*)

Return the Folkman graph.

See the Wikipedia article Folkman_graph.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.FolkmanGraph()
sage: g.order()
20
sage: g.size()
40
sage: g.diameter()
4
sage: g.girth()
4
sage: g.charpoly().factor()
(x - 4) * (x + 4) * x^10 * (x^2 - 6)^4
sage: g.chromatic_number()
2
sage: g.is_eulerian()
True
sage: g.is_hamiltonian()                                                           #␣
↪needs sage.numerical_mip
True
sage: g.is_vertex_transitive()
False
sage: g.is_bipartite()
True
```

```
>>> from sage.all import *
>>> g = graphs.FolkmanGraph()
>>> g.order()
20
```

(continues on next page)

(continued from previous page)

```
>>> g.size()
40
>>> g.diameter()
4
>>> g.girth()
4
>>> g.charpoly().factor()
(x - 4) * (x + 4) * x^10 * (x^2 - 6)^4
>>> g.chromatic_number()
2
>>> g.is_eulerian()
True
>>> g.is_hamiltonian()                                                    #␣
↪needs sage.numerical_mip
True
>>> g.is_vertex_transitive()
False
>>> g.is_bipartite()
True
```

sage.graphs.generators.smallgraphs.**FosterGraph**(*immutable=False*)

Return the Foster graph.

See the Wikipedia article Foster_graph.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.FosterGraph()
sage: g.order()
90
sage: g.size()
135
sage: g.diameter()
8
sage: g.girth()
10
sage: g.automorphism_group().cardinality()
4320
sage: g.is_hamiltonian()                                                  #␣
↪needs sage.numerical_mip
True
```

```
>>> from sage.all import *
>>> g = graphs.FosterGraph()
>>> g.order()
90
>>> g.size()
135
>>> g.diameter()
8
```

(continues on next page)

(continued from previous page)

```
>>> g.girth()
10
>>> g.automorphism_group().cardinality()
4320
>>> g.is_hamiltonian()                                                    #␣
↪needs sage.numerical_mip
True
```

sage.graphs.generators.smallgraphs.**FranklinGraph**(*immutable=False*)

Return the Franklin graph.

For more information, see the Wikipedia article Franklin_graph.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

The Franklin graph is named after Philip Franklin. It is a 3-regular graph on 12 vertices and having 18 edges:

```
sage: G = graphs.FranklinGraph(); G
Franklin graph: Graph on 12 vertices
sage: G.is_regular(3)
True
sage: G.order()
12
sage: G.size()
18
```

```
>>> from sage.all import *
>>> G = graphs.FranklinGraph(); G
Franklin graph: Graph on 12 vertices
>>> G.is_regular(Integer(3))
True
>>> G.order()
12
>>> G.size()
18
```

The Franklin graph is a Hamiltonian, bipartite graph with radius 3, diameter 3, and girth 4:

```
sage: G.is_hamiltonian()                                                #␣
↪needs sage.numerical_mip
True
sage: G.is_bipartite()
True
sage: G.radius()
3
sage: G.diameter()
3
sage: G.girth()
4
```

```
>>> from sage.all import *
>>> G.is_hamiltonian()                                                    #␣
↪needs sage.numerical_mip
True
>>> G.is_bipartite()
True
>>> G.radius()
3
>>> G.diameter()
3
>>> G.girth()
4
```

It is a perfect, triangle-free graph having chromatic number 2:

```
sage: G.is_perfect()
True
sage: G.is_triangle_free()
True
sage: G.chromatic_number()
2
```

```
>>> from sage.all import *
>>> G.is_perfect()
True
>>> G.is_triangle_free()
True
>>> G.chromatic_number()
2
```

sage.graphs.generators.smallgraphs.**FruchtGraph**(*immutable=False*)

Return a Frucht Graph.

A Frucht graph has 12 nodes and 18 edges. It is the smallest cubic identity graph. It is planar and Hamiltonian. See the Wikipedia article Frucht_graph.

PLOTTING: Upon construction, the position dictionary is filled to override the spring-layout algorithm. By convention, the first seven nodes are on the outer circle, with the next four on an inner circle and the last in the center.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: FRUCHT = graphs.FruchtGraph()
sage: FRUCHT
Frucht graph: Graph on 12 vertices
sage: FRUCHT.graph6_string()
'KhCKM?_EGK?L'
sage: (graphs.FruchtGraph()).show()      # long time
```

```
>>> from sage.all import *
>>> FRUCHT = graphs.FruchtGraph()
>>> FRUCHT
```

(continues on next page)

(continued from previous page)

```
Frucht graph: Graph on 12 vertices
>>> FRUCHT.graph6_string()
'KhCKM?_EGK?L'
>>> (graphs.FruchtGraph()).show()      # long time
```

sage.graphs.generators.smallgraphs.**GoldnerHararyGraph**(*immutable=False*)

Return the Goldner-Harary graph.

For more information, see the Wikipedia article Goldner%E2%80%93Harary_graph.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

The Goldner-Harary graph is named after A. Goldner and Frank Harary. It is a planar graph having 11 vertices and 27 edges:

```
sage: G = graphs.GoldnerHararyGraph(); G
Goldner-Harary graph: Graph on 11 vertices
sage: G.is_planar()
True
sage: G.order()
11
sage: G.size()
27
```

```
>>> from sage.all import *
>>> G = graphs.GoldnerHararyGraph(); G
Goldner-Harary graph: Graph on 11 vertices
>>> G.is_planar()
True
>>> G.order()
11
>>> G.size()
27
```

The Goldner-Harary graph is chordal with radius 2, diameter 2, and girth 3:

```
sage: G.is_chordal()
True
sage: G.radius()
2
sage: G.diameter()
2
sage: G.girth()
3
```

```
>>> from sage.all import *
>>> G.is_chordal()
True
>>> G.radius()
2
```

(continues on next page)

(continued from previous page)

```
>>> G.diameter()
2
>>> G.girth()
3
```

Its chromatic number is 4 and its automorphism group is isomorphic to the dihedral group D_6:

```
sage: G.chromatic_number()
4
sage: ag = G.automorphism_group()                                              #␣
↪needs sage.groups
sage: ag.is_isomorphic(DihedralGroup(6))                                       #␣
↪needs sage.groups
True
```

```
>>> from sage.all import *
>>> G.chromatic_number()
4
>>> ag = G.automorphism_group()                                                #␣
↪needs sage.groups
>>> ag.is_isomorphic(DihedralGroup(Integer(6)))                                ␣
↪     # needs sage.groups
True
```

sage.graphs.generators.smallgraphs.**GolombGraph**(*immutable=False*)

Return the Golomb graph.

See the Wikipedia article Golomb_graph for more information.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

The Golomb graph is a planar and Hamiltonian graph with 10 vertices and 18 edges. It has chromatic number 4, diameter 3, radius 2 and girth 3. It can be drawn in the plane as a unit distance graph:

```
sage: G = graphs.GolombGraph(); G                                              #␣
↪needs sage.symbolic
Golomb graph: Graph on 10 vertices
sage: pos = G.get_pos()                                                        #␣
↪needs sage.symbolic
sage: def dist2(u, v):
....:     return (u[0]-v[0])**2 + (u[1]-v[1])**2
sage: all(dist2(pos[u], pos[v]) == 1 for u, v in G.edge_iterator(labels=None))  #␣
↪needs sage.symbolic
True
```

```
>>> from sage.all import *
>>> G = graphs.GolombGraph(); G                                                #␣
↪needs sage.symbolic
Golomb graph: Graph on 10 vertices
>>> pos = G.get_pos()                                                          #␣
```

(continues on next page)

(continued from previous page)

```
↪needs sage.symbolic
>>> def dist2(u, v):
...     return (u[Integer(0)]-v[Integer(0)])**Integer(2) + (u[Integer(1)]-
↪v[Integer(1)])**Integer(2)
>>> all(dist2(pos[u], pos[v]) == Integer(1) for u, v in G.edge_
↪iterator(labels=None))  # needs sage.symbolic
True
```

sage.graphs.generators.smallgraphs.**GossetGraph**(*immutable=False*)

Return the Gosset graph.

The Gosset graph is the skeleton of the `Gosset_3_21()` polytope. It has with 56 vertices and degree 27. For more information, see the Wikipedia article Gosset_graph.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.GossetGraph(); g
Gosset Graph: Graph on 56 vertices
sage: g.order(), g.size()
(56, 756)
```

```
>>> from sage.all import *
>>> g = graphs.GossetGraph(); g
Gosset Graph: Graph on 56 vertices
>>> g.order(), g.size()
(56, 756)
```

sage.graphs.generators.smallgraphs.**GrayGraph**(*embedding=1*, *immutable=False*)

Return the Gray graph.

See the Wikipedia article Gray_graph.

INPUT:

- `embedding` – integer (default: 1); two embeddings are available, and can be selected by setting `embedding` to 1 or 2
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.GrayGraph()
sage: g.order()
54
sage: g.size()
81
sage: g.girth()
8
sage: g.diameter()
6
sage: g.show(figsize=[10, 10])          # long time                              #␣
↪needs sage.plot
```

(continues on next page)

(continued from previous page)

```
sage: graphs.GrayGraph(embedding=2).show(figsize=[10, 10])       # long time,␣
↪needs sage.plot
```

```
>>> from sage.all import *
>>> g = graphs.GrayGraph()
>>> g.order()
54
>>> g.size()
81
>>> g.girth()
8
>>> g.diameter()
6
>>> g.show(figsize=[Integer(10), Integer(10)])          # long time              ␣
↪              # needs sage.plot
>>> graphs.GrayGraph(embedding=Integer(2)).show(figsize=[Integer(10),␣
↪Integer(10)])       # long time, needs sage.plot
```

sage.graphs.generators.smallgraphs.**GritsenkoGraph**(*immutable=False*)

Return SRG(65, 32, 15, 16) constructed by Gritsenko.

We took the adjacency matrix from O. Gritsenko's [Gri2021] and extracted orbits of the automorphism group on the edges.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: H = graphs.GritsenkoGraph(); H                                          #␣
↪needs sage.groups
Gritsenko strongly regular graph: Graph on 65 vertices
sage: H.is_strongly_regular(parameters=True)                                  #␣
↪needs sage.groups
(65, 32, 15, 16)
```

```
>>> from sage.all import *
>>> H = graphs.GritsenkoGraph(); H                                          #␣
↪needs sage.groups
Gritsenko strongly regular graph: Graph on 65 vertices
>>> H.is_strongly_regular(parameters=True)                                  #␣
↪needs sage.groups
(65, 32, 15, 16)
```

sage.graphs.generators.smallgraphs.**GrotzschGraph**(*immutable=False*)

Return the Grötzsch graph.

The Grötzsch graph is an example of a triangle-free graph with chromatic number equal to 4. For more information, see the Wikipedia article Gr%C3%B6tzsch_graph.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

The Grötzsch graph is named after Herbert Grötzsch. It is a Hamiltonian graph with 11 vertices and 20 edges:

```
sage: G = graphs.GrotzschGraph(); G
Grotzsch graph: Graph on 11 vertices
sage: G.is_hamiltonian()                                                    #␣
↪needs sage.numerical.mip
True
sage: G.order()
11
sage: G.size()
20
```

```
>>> from sage.all import *
>>> G = graphs.GrotzschGraph(); G
Grotzsch graph: Graph on 11 vertices
>>> G.is_hamiltonian()                                                      #␣
↪needs sage.numerical.mip
True
>>> G.order()
11
>>> G.size()
20
```

The Grötzsch graph is triangle-free and having radius 2, diameter 2, and girth 4:

```
sage: G.is_triangle_free()
True
sage: G.radius()
2
sage: G.diameter()
2
sage: G.girth()
4
```

```
>>> from sage.all import *
>>> G.is_triangle_free()
True
>>> G.radius()
2
>>> G.diameter()
2
>>> G.girth()
4
```

Its chromatic number is 4 and its automorphism group is isomorphic to the dihedral group D_5:

```
sage: G.chromatic_number()
4
sage: ag = G.automorphism_group()                                           #␣
↪needs sage.groups
sage: ag.is_isomorphic(DihedralGroup(5))                                    #␣
↪needs sage.groups
True
```

```
>>> from sage.all import *
>>> G.chromatic_number()
4
>>> ag = G.automorphism_group()                                                #␣
↪needs sage.groups
>>> ag.is_isomorphic(DihedralGroup(Integer(5)))                                ␣
↪     # needs sage.groups
True
```

sage.graphs.generators.smallgraphs.**HallJankoGraph**(*from_string=True*, *immutable=False*)

Return the Hall-Janko graph.

For more information on the Hall-Janko graph, see the Wikipedia article Hall-Janko_graph.

The construction used to generate this graph in Sage is by a 100-point permutation representation of the Janko group J_2, as described in version 3 of the ATLAS of Finite Group representations, in particular on the page ATLAS: J2 – Permutation representation on 100 points.

INPUT:

- `from_string` – boolean (default: `True`); whether to build the graph from its sparse6 string or through GAP. The two methods return the same graph though doing it through GAP takes more time.
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.HallJankoGraph()
sage: g.is_regular(36)
True
sage: g.is_vertex_transitive()                                              #␣
↪needs sage.groups
True
```

```
>>> from sage.all import *
>>> g = graphs.HallJankoGraph()
>>> g.is_regular(Integer(36))
True
>>> g.is_vertex_transitive()                                                #␣
↪needs sage.groups
True
```

Is it really strongly regular with parameters 14, 12?

```
sage: nu = set(g.neighbors(0))
sage: for v in range(1, 100):
....:     if v in nu:
....:         expected = 14
....:     else:
....:         expected = 12
....:     nv = set(g.neighbors(v))
....:     nv.discard(0)
....:     if len(nu & nv) != expected:
....:         print("Something is wrong here!!!")
....:         break
```

```
>>> from sage.all import *
>>> nu = set(g.neighbors(Integer(0)))
>>> for v in range(Integer(1), Integer(100)):
...     if v in nu:
...         expected = Integer(14)
...     else:
...         expected = Integer(12)
...     nv = set(g.neighbors(v))
...     nv.discard(Integer(0))
...     if len(nu & nv) != expected:
...         print("Something is wrong here!!!")
...         break
```

Some other properties that we know how to check:

```
sage: g.diameter()
2
sage: g.girth()
3
sage: factor(g.characteristic_polynomial())                                    #␣
↪needs sage.libs.pari sage.modules
(x - 36) * (x - 6)^36 * (x + 4)^63
```

```
>>> from sage.all import *
>>> g.diameter()
2
>>> g.girth()
3
>>> factor(g.characteristic_polynomial())                                      #␣
↪needs sage.libs.pari sage.modules
(x - 36) * (x - 6)^36 * (x + 4)^63
```

sage.graphs.generators.smallgraphs.**HarborthGraph**(*immutable=False*)

Return the Harborth Graph.

The Harborth graph has 104 edges and 52 vertices, and is the smallest known example of a 4-regular matchstick graph. For more information, see the Wikipedia article Harborth_graph.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.HarborthGraph(); g
Harborth Graph: Graph on 52 vertices
sage: g.is_regular(4)
True
```

```
>>> from sage.all import *
>>> g = graphs.HarborthGraph(); g
Harborth Graph: Graph on 52 vertices
>>> g.is_regular(Integer(4))
True
```

sage.graphs.generators.smallgraphs.**HarriesGraph**(*embedding=1*, *immutable=False*)

Return the Harries Graph.

The Harries graph is a Hamiltonian 3-regular graph on 70 vertices. See the Wikipedia article Harries_graph.

The default embedding here is to emphasize the graph's 4 orbits. This graph actually has a funny construction. The following procedure gives an idea of it, though not all the adjacencies are being properly defined.

1. Take two disjoint copies of a *Petersen graph*. Their vertices will form an orbit of the final graph.
2. Subdivide all the edges once, to create 15+15=30 new vertices, which together form another orbit.
3. Create 15 vertices, each of them linked to 2 corresponding vertices of the previous orbit, one in each of the two subdivided Petersen graphs. At the end of this step all vertices from the previous orbit have degree 3, and the only vertices of degree 2 in the graph are those that were just created.
4. Create 5 vertices connected only to the ones from the previous orbit so that the graph becomes 3-regular.

INPUT:

- `embedding` – integer (default: 1); two embeddings are available, and can be selected by setting `embedding` to 1 or 2
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.HarriesGraph()
sage: g.order()
70
sage: g.size()
105
sage: g.girth()
10
sage: g.diameter()
6
sage: g.show(figsize=[10, 10])          # long time                              #␣
↪needs sage.plot
sage: graphs.HarriesGraph(embedding=2).show(figsize=[10, 10])    # long time,␣
↪needs sage.plot
```

```
>>> from sage.all import *
>>> g = graphs.HarriesGraph()
>>> g.order()
70
>>> g.size()
105
>>> g.girth()
10
>>> g.diameter()
6
>>> g.show(figsize=[Integer(10), Integer(10)])          # long time              ␣
↪              # needs sage.plot
>>> graphs.HarriesGraph(embedding=Integer(2)).show(figsize=[Integer(10),␣
↪Integer(10)])    # long time, needs sage.plot
```

sage.graphs.generators.smallgraphs.**HarriesWongGraph**(*embedding=1*, *immutable=False*)

Return the Harries-Wong Graph.

See the Wikipedia article Harries-Wong_graph.

About the default embedding:

The default embedding is an attempt to emphasize the graph's 8 (!!!) different orbits. In order to understand this better, one can picture the graph as being built in the following way.

1. One first creates a 3-dimensional cube (8 vertices, 12 edges), whose vertices define the first orbit of the final graph.
2. The edges of this graph are subdivided once, to create 12 new vertices which define a second orbit.
3. The edges of the graph are subdivided once more, to create 24 new vertices giving a third orbit.
4. 4 vertices are created and made adjacent to the vertices of the second orbit so that they have degree 3. These 4 vertices also define a new orbit.
5. In order to make the vertices from the third orbit 3-regular (they all miss one edge), one creates a binary tree on 1 + 3 + 6 + 12 vertices. The leaves of this new tree are made adjacent to the 12 vertices of the third orbit, and the graph is now 3-regular. This binary tree contributes 4 new orbits to the Harries-Wong graph.

INPUT:

- `embedding` – integer (default: 1); two embeddings are available, and can be selected by setting `embedding` to 1 or 2
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.HarriesWongGraph()
sage: g.order()
70
sage: g.size()
105
sage: g.girth()
10
sage: g.diameter()
6
sage: orbits = g.automorphism_group(orbits=True)[-1]    # long time             #␣
↪needs sage.groups
sage: g.show(figsize=[15, 15], partition=orbits)        # long time             #␣
↪needs sage.groups sage.plot
```

```
>>> from sage.all import *
>>> g = graphs.HarriesWongGraph()
>>> g.order()
70
>>> g.size()
105
>>> g.girth()
10
>>> g.diameter()
6
>>> orbits = g.automorphism_group(orbits=True)[-Integer(1)]    # long time       ␣
↪     # needs sage.groups
>>> g.show(figsize=[Integer(15), Integer(15)], partition=orbits)        # long␣
↪time             # needs sage.groups sage.plot
```

Alternative embedding:

```
sage: graphs.HarriesWongGraph(embedding=2).show()       # long time              #␣
↪needs sage.plot
```

```
>>> from sage.all import *
>>> graphs.HarriesWongGraph(embedding=Integer(2)).show()       # long time       ␣
↪     # needs sage.plot
```

sage.graphs.generators.smallgraphs.**HeawoodGraph**(*immutable=False*)

Return a Heawood graph.

The Heawood graph is a cage graph that has 14 nodes. It is a cubic symmetric graph. (See also the Möbius-Kantor graph, *MoebiusKantorGraph()*). It is nonplanar and Hamiltonian. It has diameter 3, radius 3, girth 6, and chromatic number 2. It is 4-transitive but not 5-transitive. See the Wikipedia article Heawood_graph.

PLOTTING: Upon construction, the position dictionary is filled to override the spring-layout algorithm. By convention, the nodes are positioned in a circular layout with the first node appearing at the top, and then continuing counterclockwise.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: H = graphs.HeawoodGraph()
sage: H
Heawood graph: Graph on 14 vertices
sage: H.graph6_string()
'MhEGHC@AI?_PC@_G_'
sage: (graphs.HeawoodGraph()).show()     # long time                             #␣
↪needs sage.plot
```

```
>>> from sage.all import *
>>> H = graphs.HeawoodGraph()
>>> H
Heawood graph: Graph on 14 vertices
>>> H.graph6_string()
'MhEGHC@AI?_PC@_G_'
>>> (graphs.HeawoodGraph()).show()     # long time                             #␣
↪needs sage.plot
```

sage.graphs.generators.smallgraphs.**HerschelGraph**(*immutable=False*)

Return the Herschel graph.

For more information, see the Wikipedia article Herschel_graph.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

The Herschel graph is named after Alexander Stewart Herschel. It is a planar, bipartite graph with 11 vertices and 18 edges:

```
sage: G = graphs.HerschelGraph(); G
Herschel graph: Graph on 11 vertices
sage: G.is_planar()
True
sage: G.is_bipartite()
True
sage: G.order()
11
sage: G.size()
18
```

```
>>> from sage.all import *
>>> G = graphs.HerschelGraph(); G
Herschel graph: Graph on 11 vertices
>>> G.is_planar()
True
>>> G.is_bipartite()
True
>>> G.order()
11
>>> G.size()
18
```

The Herschel graph is a perfect graph with radius 3, diameter 4, and girth 4:

```
sage: G.is_perfect()
True
sage: G.radius()
3
sage: G.diameter()
4
sage: G.girth()
4
```

```
>>> from sage.all import *
>>> G.is_perfect()
True
>>> G.radius()
3
>>> G.diameter()
4
>>> G.girth()
4
```

Its chromatic number is 2 and its automorphism group is isomorphic to the dihedral group D_6:

```
sage: G.chromatic_number()
2
sage: ag = G.automorphism_group()                                    #␣
↪needs sage.groups
sage: ag.is_isomorphic(DihedralGroup(6))                             #␣
↪needs sage.groups
True
```

```
>>> from sage.all import *
>>> G.chromatic_number()
2
>>> ag = G.automorphism_group()                                              #␣
↪needs sage.groups
>>> ag.is_isomorphic(DihedralGroup(Integer(6)))                                ␣
↪     # needs sage.groups
True
```

sage.graphs.generators.smallgraphs.**HigmanSimsGraph**(*relabel=True*, *immutable=False*)

Return the Higman-Sims graph.

The Higman-Sims graph is a remarkable strongly regular graph of degree 22 on 100 vertices. For example, it can be split into two sets of 50 vertices each, so that each half induces a subgraph isomorphic to the Hoffman-Singleton graph (*HoffmanSingletonGraph()*). This can be done in 352 ways (see Higman-Sims graph by Andries E. Brouwer, accessed 24 October 2009.)

Its most famous property is that the automorphism group has an index 2 subgroup which is one of the 26 sporadic groups [HS1968].

The construction used here follows [Haf2004].

See also the Wikipedia article Higman–Sims_graph.

INPUT:

- `relabel` – boolean (default: `True`); whether to relabel the vertices with consecutive integers. If `False` the labels are strings that are three digits long. "xyz" means the vertex is in group x (zero through three), pentagon or pentagram y (zero through four), and is vertex z (zero through four) of that pentagon or pentagram. See [Haf2004] for more.
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

OUTPUT: the Higman-Sims graph

EXAMPLES:

A split into the first 50 and last 50 vertices will induce two copies of the Hoffman-Singleton graph, and we illustrate another such split, which is obvious based on the construction used:

```
sage: H = graphs.HigmanSimsGraph()
sage: A = H.subgraph(range(0,50))
sage: B = H.subgraph(range(50,100))
sage: K = graphs.HoffmanSingletonGraph()
sage: K.is_isomorphic(A) and K.is_isomorphic(B)
True
sage: C = H.subgraph(range(25,75))
sage: D = H.subgraph(list(range(0,25))+list(range(75,100)))
sage: K.is_isomorphic(C) and K.is_isomorphic(D)
True
```

```
>>> from sage.all import *
>>> H = graphs.HigmanSimsGraph()
>>> A = H.subgraph(range(Integer(0),Integer(50)))
>>> B = H.subgraph(range(Integer(50),Integer(100)))
>>> K = graphs.HoffmanSingletonGraph()
>>> K.is_isomorphic(A) and K.is_isomorphic(B)
```

(continues on next page)

(continued from previous page)

```
True
>>> C = H.subgraph(range(Integer(25),Integer(75)))
>>> D = H.subgraph(list(range(Integer(0),Integer(25)))+list(range(Integer(75),
↪Integer(100))))
>>> K.is_isomorphic(C) and K.is_isomorphic(D)
True
```

The automorphism group contains only one nontrivial proper normal subgroup, which is of index 2 and is simple. It is known as the Higman-Sims group:

```
sage: H = graphs.HigmanSimsGraph()
sage: G = H.automorphism_group()
sage: g = G.order(); g
88704000
sage: K = G.normal_subgroups()[1]
sage: K.is_simple()
True
sage: g//K.order()
2
```

```
>>> from sage.all import *
>>> H = graphs.HigmanSimsGraph()
>>> G = H.automorphism_group()
>>> g = G.order(); g
88704000
>>> K = G.normal_subgroups()[Integer(1)]
>>> K.is_simple()
True
>>> g//K.order()
2
```

AUTHOR:

- Rob Beezer (2009-10-24)

sage.graphs.generators.smallgraphs.**HoffmanGraph**(*immutable=False*)

Return the Hoffman Graph.

See the Wikipedia article Hoffman_graph.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.HoffmanGraph()
sage: g.is_bipartite()
True
sage: g.is_hamiltonian()                # long time                              #␣
↪needs sage.numerical.mip
True
sage: g.radius()
3
sage: g.diameter()
```

(continues on next page)

(continued from previous page)

```
4
sage: g.automorphism_group().cardinality()                                  #␣
↪needs sage.groups
48
```

```
>>> from sage.all import *
>>> g = graphs.HoffmanGraph()
>>> g.is_bipartite()
True
>>> g.is_hamiltonian()                # long time                          #␣
↪needs sage.numerical.mip
True
>>> g.radius()
3
>>> g.diameter()
4
>>> g.automorphism_group().cardinality()                                    #␣
↪needs sage.groups
48
```

sage.graphs.generators.smallgraphs.**HoffmanSingletonGraph**(*immutable=False*)

Return the Hoffman-Singleton graph.

The Hoffman-Singleton graph is the Moore graph of degree 7, diameter 2 and girth 5. The Hoffman-Singleton theorem states that any Moore graph with girth 5 must have degree 2, 3, 7 or 57. The first three respectively are the pentagon, the Petersen graph, and the Hoffman-Singleton graph. The existence of a Moore graph with girth 5 and degree 57 is still open.

A Moore graph is a graph with diameter d and girth $2d + 1$. This implies that the graph is regular, and distance regular.

For more details, see [GR2001] and the Wikipedia article Hoffman–Singleton_graph.

PLOTTING: Upon construction, the position dictionary is filled to override the spring-layout algorithm. A novel algorithm written by Tom Boothby gives a random layout which is pleasing to the eye.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: HS = graphs.HoffmanSingletonGraph()
sage: Set(HS.degree())
{7}
sage: HS.girth()
5
sage: HS.diameter()
2
sage: HS.n_vertices()
50
```

```
>>> from sage.all import *
>>> HS = graphs.HoffmanSingletonGraph()
>>> Set(HS.degree())
```

(continues on next page)

(continued from previous page)

```
{7}
>>> HS.girth()
5
>>> HS.diameter()
2
>>> HS.n_vertices()
50
```

Note that you get a different layout each time you create the graph.

```
sage: HS.layout()[1]  # random
(-0.844..., 0.535...)
sage: HS = graphs.HoffmanSingletonGraph()
sage: HS.layout()[1]  # random
(-0.904..., 0.425...)
```

```
>>> from sage.all import *
>>> HS.layout()[Integer(1)]  # random
(-0.844..., 0.535...)
>>> HS = graphs.HoffmanSingletonGraph()
>>> HS.layout()[Integer(1)]  # random
(-0.904..., 0.425...)
```

sage.graphs.generators.smallgraphs.**HoltGraph**(*immutable=False*)

Return the Holt graph (also called the Doyle graph).

See the Wikipedia article Holt_graph.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.HoltGraph();g
Holt graph: Graph on 27 vertices
sage: g.is_regular()
True
sage: g.is_vertex_transitive()                                              #␣
↪needs sage.groups
True
sage: g.chromatic_number()
3
sage: g.is_hamiltonian()                  # long time                       #␣
↪needs sage.numerical.mip
True
sage: g.radius()
3
sage: g.diameter()
3
sage: g.girth()
5
sage: g.automorphism_group().cardinality()                                  #␣
```

(continues on next page)

(continued from previous page)

```
↪needs sage.groups
54
```

```
>>> from sage.all import *
>>> g = graphs.HoltGraph();g
Holt graph: Graph on 27 vertices
>>> g.is_regular()
True
>>> g.is_vertex_transitive()                                                    #␣
↪needs sage.groups
True
>>> g.chromatic_number()
3
>>> g.is_hamiltonian()                 # long time                              #␣
↪needs sage.numerical.mip
True
>>> g.radius()
3
>>> g.diameter()
3
>>> g.girth()
5
>>> g.automorphism_group().cardinality()                                        #␣
↪needs sage.groups
54
```

sage.graphs.generators.smallgraphs.**HortonGraph**(*immutable=False*)

Return the Horton Graph.

The Horton graph is a cubic 3-connected non-hamiltonian graph. For more information, see the Wikipedia article Horton_graph.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.HortonGraph()
sage: g.order()
96
sage: g.size()
144
sage: g.radius()
10
sage: g.diameter()
10
sage: g.girth()
6
sage: g.automorphism_group().cardinality()
96
sage: g.chromatic_number()
2
sage: g.is_hamiltonian()                # not tested (veeeery long)             #␣
```

(continues on next page)

(continued from previous page)

```
↪needs sage.numerical.mip
False
```

```
>>> from sage.all import *
>>> g = graphs.HortonGraph()
>>> g.order()
96
>>> g.size()
144
>>> g.radius()
10
>>> g.diameter()
10
>>> g.girth()
6
>>> g.automorphism_group().cardinality()
96
>>> g.chromatic_number()
2
>>> g.is_hamiltonian()                # not tested (veeeery long)              #␣
↪needs sage.numerical.mip
False
```

sage.graphs.generators.smallgraphs.**IoninKharaghani765Graph**(*immutable=False*)

Return a $(765, 192, 48, 48)$-strongly regular graph.

Existence of a strongly regular graph with these parameters was claimed in [IK2003]. Implementing the construction in the latter did not work, however. This function implements the following instructions, shared by Yury Ionin and Hadi Kharaghani.

Let A be the affine plane over the field $GF(3) = \{-1, 0, 1\}$. Let

$$\begin{aligned}\phi_1(x, y) &= x\\ \phi_2(x, y) &= y\\ \phi_3(x, y) &= x + y\\ \phi_4(x, y) &= x - y\end{aligned}$$

For $i = 1, 2, 3, 4$ and $j \in GF(3)$, let $L_{i,j}$ be the line in A defined by $\phi_i(x, y) = j$. Let $\mathcal{M}$ be the set of all 12 lines $L_{i,j}$, plus the empty set. Let π be the permutation defined on $\mathcal{M}$ by $\pi(L_{i,j}) = L_{i,j+1}$ and $\pi(\emptyset) = \emptyset$, so that π has three orbits of cardinality 3 and one of cardinality 1.

Let $A = (p_1, ..., p_9)$ with $p_1 = (-1, 1)$, $p_2 = (-1, 0)$, $p_3 = (-1, 1)$, $p_4 = (0, -1)$, $p_5 = (0, 0)$, $p_6 = (0, 1)$, $p_7 = (1, -1)$, $p_8 = (1, 0)$, $p_9 = (1, 1)$. Note that $p_i + p_{10-i} = (0, 0)$. For any subset X of A, let $M(X)$ be the $(0, 1)$-matrix of order 9 whose (i, j)-entry equals 1 if and only if $p_{10-i} - p_j \in X$. Note that M is a symmetric matrix.

An MF-tuple is an ordered quintuple $(X_1, X_2, X_3, X_4, X_5)$ of subsets of A, of which one is the empty set and the other four are pairwise non-parallel lines. Such a quintuple generates the following block matrix:

$$N(X_1, X_2, X_3, X_4, X_5) = \begin{pmatrix} M(X_1) & M(X_2) & M(X_3) & M(X_4) & M(X_5)\\ M(X_2) & M(X_3) & M(X_4) & M(X_5) & M(X_1)\\ M(X_3) & M(X_4) & M(X_5) & M(X_1) & M(X_2)\\ M(X_4) & M(X_5) & M(X_1) & M(X_2) & M(X_3)\\ M(X_5) & M(X_1) & M(X_2) & M(X_3) & M(X_4)\end{pmatrix}$$

Observe that if $(X_1, X_2, X_3, X_4, X_5)$ is an MF-tuple, then $N(X_1, X_2, X_3, X_4, X_5)$ is the symmetric incidence matrix of a symmetric $(45, 12, 3)$-design.

Let $\mathcal{F}$ be the set of all MF-tuples and let σ be the following permutation of $\mathcal{F}$:

$$\sigma(X_1, X_2, X_3, X_4, X_5) = (X_2, X_3, X_4, X_5, X_1)$$
$$\pi(X_1, X_2, X_3, X_4, X_5) = (\pi(X_1), \pi(X_2), \pi(X_3), \pi(X_4), \pi(X_5))$$

Observe that σ and π commute, and generate a (cyclic) group G of order 15. We will from now on identify G with the (cyclic) multiplicative group of the field $GF(16)$ equal to $\{\omega^0, ..., \omega^{14}\}$. Let $W = [w_{ij}]$ be the following matrix of order 17 over $GF(16) = \{a_1, ..., a_16\}$:

$$w_{ij} = \begin{cases} a_i + a_j & \text{if } 1 \leq i \leq 16, 1 \leq j \leq 16, \\ 1 & \text{if } i = 17, j \neq 17, \\ 1 & \text{if } i \neq 17, j = 17, \\ 0 & \text{if } i = j = 17 \end{cases}$$

The diagonal entries of W are equal to 0, each off-diagonal entry can be represented as ω^k with $0 \leq k \leq 14$. Matrix W is a symmetric $BGW(17, 16, 15; G)$.

Fix an MF-tuple $(X_1, X_2, X_3, X_4, X_5)$ and let S be the block matrix obtained from W by replacing every diagonal entry of W by the zero matrix of order 45, and every off-diagonal entry ω^k by the matrix $N(\sigma^k(X_1, X_2, X_3, X_4, X_5))$ (through the association of ω^k with an element of G). Then S is a symmetric incidence matrix of a symmetric $(765, 192, 48)$-design with zero diagonal, and therefore S is an adjacency matrix of a strongly regular graph with parameters $(765, 192, 48, 48)$.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.IoninKharaghani765Graph(); g                                  #␣
↪needs sage.modules sage.rings.finite_rings
Ionin-Kharaghani: Graph on 765 vertices
```

```
>>> from sage.all import *
>>> g = graphs.IoninKharaghani765Graph(); g                                    #␣
↪needs sage.modules sage.rings.finite_rings
Ionin-Kharaghani: Graph on 765 vertices
```

Todo

An update to [IK2003] meant to fix the problem encountered became available 2016/02/24, see http://www.cs.uleth.ca/~hadi/research/IoninKharaghani.pdf

`sage.graphs.generators.smallgraphs.`**`JankoKharaghaniGraph`**`(v, immutable=False)`

Return a $(936, 375, 150, 150)$-srg or a $(1800, 1029, 588, 588)$-srg.

This functions returns a strongly regular graph for the two sets of parameters shown to be realizable in [JK2002]. The paper also uses a construction from [GM1987].

INPUT:

- `v` – integer; one of 936 or 1800
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.JankoKharaghaniGraph(936)       # long time                    #␣
↪needs sage.libs.pari
sage: g.is_strongly_regular(parameters=True)     # long time                    #␣
↪needs sage.libs.pari
(936, 375, 150, 150)

sage: g = graphs.JankoKharaghaniGraph(1800)      # not tested (30s)
sage: g.is_strongly_regular(parameters=True)     # not tested (30s)
(1800, 1029, 588, 588)
```

```
>>> from sage.all import *
>>> g = graphs.JankoKharaghaniGraph(Integer(936))       # long time              ␣
↪     # needs sage.libs.pari
>>> g.is_strongly_regular(parameters=True)     # long time                    #␣
↪needs sage.libs.pari
(936, 375, 150, 150)

>>> g = graphs.JankoKharaghaniGraph(Integer(1800))      # not tested (30s)
>>> g.is_strongly_regular(parameters=True)     # not tested (30s)
(1800, 1029, 588, 588)
```

sage.graphs.generators.smallgraphs.**JankoKharaghaniTonchevGraph**(*immutable=False*)

Return a $(324, 153, 72, 72)$-strongly regular graph from [JKT2001].

Build the graph using the description given in [JKT2001], taking sets B1 and B163 in the text as adjacencies of vertices 1 and 163, respectively, and taking the edge orbits of the group G provided.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: Gamma = graphs.JankoKharaghaniTonchevGraph()       # long time            #␣
↪needs sage.libs.gap
sage: Gamma.is_strongly_regular(parameters=True)         # long time            #␣
↪needs sage.libs.gap
(324, 153, 72, 72)
```

```
>>> from sage.all import *
>>> Gamma = graphs.JankoKharaghaniTonchevGraph()       # long time            #␣
↪needs sage.libs.gap
>>> Gamma.is_strongly_regular(parameters=True)         # long time            #␣
↪needs sage.libs.gap
(324, 153, 72, 72)
```

sage.graphs.generators.smallgraphs.**KittellGraph**(*immutable=False*)

Return the Kittell Graph.

For more information, see the Wolfram page about the Kittel Graph.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.KittellGraph()
sage: g.order()
23
sage: g.size()
63
sage: g.radius()
3
sage: g.diameter()
4
sage: g.girth()
3
sage: g.chromatic_number()
4
```

```
>>> from sage.all import *
>>> g = graphs.KittellGraph()
>>> g.order()
23
>>> g.size()
63
>>> g.radius()
3
>>> g.diameter()
4
>>> g.girth()
3
>>> g.chromatic_number()
4
```

sage.graphs.generators.smallgraphs.**Klein3RegularGraph**(*immutable=False*)

Return the Klein 3-regular graph.

The cubic Klein graph has 56 vertices and can be embedded on a surface of genus 3. It is the dual of *Klein7RegularGraph()*. For more information, see the Wikipedia article Klein_graphs.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.Klein3RegularGraph(); g
Klein 3-regular Graph: Graph on 56 vertices
sage: g.order(), g.size()
(56, 84)
sage: g.girth()
7
sage: g.automorphism_group().cardinality()                                  #␣
↪needs sage.groups
336
sage: g.chromatic_number()
3
```

```
>>> from sage.all import *
>>> g = graphs.Klein3RegularGraph(); g
Klein 3-regular Graph: Graph on 56 vertices
>>> g.order(), g.size()
(56, 84)
>>> g.girth()
7
>>> g.automorphism_group().cardinality()                                    #␣
↪needs sage.groups
336
>>> g.chromatic_number()
3
```

sage.graphs.generators.smallgraphs.**Klein7RegularGraph**(*immutable=False*)

Return the Klein 7-regular graph.

The 7-valent Klein graph has 24 vertices and can be embedded on a surface of genus 3. It is the dual of *Klein3RegularGraph()*. For more information, see the Wikipedia article Klein_graphs.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.Klein7RegularGraph(); g
Klein 7-regular Graph: Graph on 24 vertices
sage: g.order(), g.size()
(24, 84)
sage: g.girth()
3
sage: g.automorphism_group().cardinality()                                  #␣
↪needs sage.groups
336
sage: g.chromatic_number()
4
```

```
>>> from sage.all import *
>>> g = graphs.Klein7RegularGraph(); g
Klein 7-regular Graph: Graph on 24 vertices
>>> g.order(), g.size()
(24, 84)
>>> g.girth()
3
>>> g.automorphism_group().cardinality()                                    #␣
↪needs sage.groups
336
>>> g.chromatic_number()
4
```

sage.graphs.generators.smallgraphs.**KrackhardtKiteGraph**(*immutable=False*)

Return a Krackhardt kite graph with 10 nodes.

The Krackhardt kite graph was originally developed by David Krackhardt for the purpose of studying social networks (see [Kre2002] and the Wikipedia article Krackhardt_kite_graph). It is used to show the distinction between

degree centrality, betweenness centrality, and closeness centrality. For more information read the plotting section below in conjunction with the example.

PLOTTING: Upon construction, the position dictionary is filled to override the spring-layout algorithm. By convention, the graph is drawn left to right, in top to bottom row sequence of [2, 3, 2, 1, 1, 1] nodes on each row. This places the fourth node (3) in the center of the kite, with the highest degree. But the fourth node only connects nodes that are otherwise connected, or those in its clique (i.e.: Degree Centrality). The eighth (7) node is where the kite meets the tail. It has degree = 3, less than the average, but is the only connection between the kite and tail (i.e.: Betweenness Centrality). The sixth and seventh nodes (5 and 6) are drawn in the third row and have degree = 5. These nodes have the shortest path to all other nodes in the graph (i.e.: Closeness Centrality). Please execute the example for visualization.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

Construct and show a Krackhardt kite graph

```
sage: g = graphs.KrackhardtKiteGraph()
sage: g.show()                              # long time                          #␣
↪needs sage.plot
```

```
>>> from sage.all import *
>>> g = graphs.KrackhardtKiteGraph()
>>> g.show()                              # long time                          #␣
↪needs sage.plot
```

sage.graphs.generators.smallgraphs.**LivingstoneGraph**(*immutable=False*)

Return the Livingstone Graph.

The Livingstone graph is a distance-transitive graph on 266 vertices whose automorphism group is the `J1 group`. For more information, see the Wikipedia article Livingstone_graph.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: # optional - internet
sage: g = graphs.LivingstoneGraph()
sage: g.order()
266
sage: g.size()
1463
sage: g.girth()
5
sage: g.is_vertex_transitive()
True
sage: g.is_distance_regular()
True
```

```
>>> from sage.all import *
>>> # optional - internet
>>> g = graphs.LivingstoneGraph()
```

(continues on next page)

(continued from previous page)

```
>>> g.order()
266
>>> g.size()
1463
>>> g.girth()
5
>>> g.is_vertex_transitive()
True
>>> g.is_distance_regular()
True
```

sage.graphs.generators.smallgraphs.**LjubljanaGraph**(*embedding=1*, *immutable=False*)

Return the Ljubljana Graph.

The Ljubljana graph is a bipartite 3-regular graph on 112 vertices and 168 edges. It is not vertex-transitive as it has two orbits which are also independent sets of size 56. See the Wikipedia article Ljubljana_graph.

The default embedding is obtained from the Heawood graph.

INPUT:

- `embedding` – integer (default: 1); two embeddings are available, and can be selected by setting `embedding` to 1 or 2
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.LjubljanaGraph()
sage: g.order()
112
sage: g.size()
168
sage: g.girth()
10
sage: g.diameter()
8
sage: g.show(figsize=[10, 10])          # long time                                #␣
↪needs sage.plot
sage: graphs.LjubljanaGraph(embedding=2).show(figsize=[10, 10])          # long␣
↪time, needs sage.plot
```

```
>>> from sage.all import *
>>> g = graphs.LjubljanaGraph()
>>> g.order()
112
>>> g.size()
168
>>> g.girth()
10
>>> g.diameter()
8
>>> g.show(figsize=[Integer(10), Integer(10)])          # long time              ␣
↪              # needs sage.plot
```

(continues on next page)

(continued from previous page)

```
>>> graphs.LjubljanaGraph(embedding=Integer(2)).show(figsize=[Integer(10),␣
↪Integer(10)])          # long time, needs sage.plot
```

sage.graphs.generators.smallgraphs.**LocalMcLaughlinGraph**(*immutable=False*)

Return the local McLaughlin graph.

The local McLaughlin graph is a strongly regular graph with parameters $(162, 56, 10, 24)$. It can be obtained from *McLaughlinGraph()* by considering the stabilizer of a point: one of its orbits has cardinality 162.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.LocalMcLaughlinGraph(); g      # long time, optional - gap_
↪package_design
Local McLaughlin Graph: Graph on 162 vertices
sage: g.is_strongly_regular(parameters=True)    # long time, optional - gap_
↪package_design
(162, 56, 10, 24)
```

```
>>> from sage.all import *
>>> g = graphs.LocalMcLaughlinGraph(); g      # long time, optional - gap_package_
↪design
Local McLaughlin Graph: Graph on 162 vertices
>>> g.is_strongly_regular(parameters=True)    # long time, optional - gap_package_
↪design
(162, 56, 10, 24)
```

sage.graphs.generators.smallgraphs.**M22Graph**(*immutable=False*)

Return the M22 graph.

The M_{22} graph is the unique strongly regular graph with parameters $v = 77, k = 16, \lambda = 0, \mu = 4$.

For more information on the M_{22} graph, see https://www.win.tue.nl/~aeb/graphs/M22.html.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.M22Graph()
sage: g.order()
77
sage: g.size()
616
sage: g.is_strongly_regular(parameters=True)
(77, 16, 0, 4)
```

```
>>> from sage.all import *
>>> g = graphs.M22Graph()
>>> g.order()
77
>>> g.size()
```

(continues on next page)

(continued from previous page)

```
616
>>> g.is_strongly_regular(parameters=True)
(77, 16, 0, 4)
```

sage.graphs.generators.smallgraphs.**MarkstroemGraph**(*immutable=False*)

Return the Markström Graph.

The Markström Graph is a cubic planar graph with no cycles of length 4 nor 8, but containing cycles of length 16. For more information, see the Wolfram page about the Markström Graph.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.MarkstroemGraph()
sage: g.order()
24
sage: g.size()
36
sage: g.is_planar()
True
sage: g.is_regular(3)
True
sage: g.subgraph_search(graphs.CycleGraph(4)) is None                    #␣
↪needs sage.modules
True
sage: g.subgraph_search(graphs.CycleGraph(8)) is None                    #␣
↪needs sage.modules
True
sage: g.subgraph_search(graphs.CycleGraph(16))                           #␣
↪needs sage.modules
Subgraph of (Markstroem Graph): Graph on 16 vertices
```

```
>>> from sage.all import *
>>> g = graphs.MarkstroemGraph()
>>> g.order()
24
>>> g.size()
36
>>> g.is_planar()
True
>>> g.is_regular(Integer(3))
True
>>> g.subgraph_search(graphs.CycleGraph(Integer(4))) is None                ␣
↪     # needs sage.modules
True
>>> g.subgraph_search(graphs.CycleGraph(Integer(8))) is None                ␣
↪     # needs sage.modules
True
>>> g.subgraph_search(graphs.CycleGraph(Integer(16)))                       ␣
↪     # needs sage.modules
Subgraph of (Markstroem Graph): Graph on 16 vertices
```

sage.graphs.generators.smallgraphs.**MathonStronglyRegularGraph**(*t*, *immutable=False*)

Return one of Mathon's graphs on 784 vertices.

INPUT:

- `t` – integer; the number of the graph, from 0 to 2
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: # long time, needs sage.libs.gap
sage: from sage.graphs.generators.smallgraphs import MathonStronglyRegularGraph
sage: G = MathonStronglyRegularGraph(0)
sage: G.is_strongly_regular(parameters=True)
(784, 243, 82, 72)
```

```
>>> from sage.all import *
>>> # long time, needs sage.libs.gap
>>> from sage.graphs.generators.smallgraphs import MathonStronglyRegularGraph
>>> G = MathonStronglyRegularGraph(Integer(0))
>>> G.is_strongly_regular(parameters=True)
(784, 243, 82, 72)
```

sage.graphs.generators.smallgraphs.**McGeeGraph**(*embedding=2*, *immutable=False*)

Return the McGee Graph.

See the Wikipedia article McGee_graph.

INPUT:

- `embedding` – integer (default: 2); two embeddings are available, and can be selected by setting `embedding` to 1 or 2
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.McGeeGraph()
sage: g.order()
24
sage: g.size()
36
sage: g.girth()
7
sage: g.diameter()
4
sage: g.show()                                                              #␣
↪needs sage.plot
sage: graphs.McGeeGraph(embedding=1).show()      # long time                #␣
↪needs sage.plot
```

```
>>> from sage.all import *
>>> g = graphs.McGeeGraph()
>>> g.order()
24
>>> g.size()
```

(continues on next page)

(continued from previous page)

```
36
>>> g.girth()
7
>>> g.diameter()
4
>>> g.show()                                                                  #␣
↪needs sage.plot
>>> graphs.McGeeGraph(embedding=Integer(1)).show()      # long time              ␣
↪     # needs sage.plot
```

sage.graphs.generators.smallgraphs.**McLaughlinGraph**(*immutable=False*)

Return the McLaughlin Graph.

The McLaughlin Graph is the unique strongly regular graph of parameters $(275, 112, 30, 56)$.

For more information on the McLaughlin Graph, see its web page on Andries Brouwer's website which gives the definition that this method implements.

Note

To create this graph you must have the gap_packages spkg installed.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.McLaughlinGraph()                # optional - gap_package_design
sage: g.is_strongly_regular(parameters=True)      # optional - gap_package_design
(275, 112, 30, 56)
sage: set(g.spectrum()) == {112, 2, -28}          # optional - gap_package_design
True
```

```
>>> from sage.all import *
>>> g = graphs.McLaughlinGraph()                # optional - gap_package_design
>>> g.is_strongly_regular(parameters=True)      # optional - gap_package_design
(275, 112, 30, 56)
>>> set(g.spectrum()) == {Integer(112), Integer(2), -Integer(28)}          #␣
↪optional - gap_package_design
True
```

sage.graphs.generators.smallgraphs.**MeredithGraph**(*immutable=False*)

Return the Meredith Graph.

The Meredith Graph is a 4-regular 4-connected non-hamiltonian graph. For more information on the Meredith Graph, see the Wikipedia article Meredith_graph.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.MeredithGraph()
sage: g.is_regular(4)
True
sage: g.order()
70
sage: g.size()
140
sage: g.radius()
7
sage: g.diameter()
8
sage: g.girth()
4
sage: g.chromatic_number()
3
sage: g.is_hamiltonian()                # long time                              #␣
↪needs sage.numerical.mip
False
```

```
>>> from sage.all import *
>>> g = graphs.MeredithGraph()
>>> g.is_regular(Integer(4))
True
>>> g.order()
70
>>> g.size()
140
>>> g.radius()
7
>>> g.diameter()
8
>>> g.girth()
4
>>> g.chromatic_number()
3
>>> g.is_hamiltonian()                # long time                              #␣
↪needs sage.numerical.mip
False
```

sage.graphs.generators.smallgraphs.**MoebiusKantorGraph**(*immutable=False*)

Return a Möbius-Kantor Graph.

A Möbius-Kantor graph is a cubic symmetric graph. (See also the Heawood graph). It has 16 nodes and 24 edges. It is nonplanar and Hamiltonian. It has diameter 4, girth 6, and chromatic number 2. It is identical to the Generalized Petersen graph, P[8, 3].

For more details, see Möbius-Kantor Graph - from Wolfram MathWorld.

PLOTTING: See the plotting section for the generalized Petersen graphs.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: MK = graphs.MoebiusKantorGraph()
sage: MK
Moebius-Kantor Graph: Graph on 16 vertices
sage: MK.graph6_string()
'OhCGKE?O@?ACAC@I?Q_AS'
sage: (graphs.MoebiusKantorGraph()).show()      # long time                    #␣
↪needs sage.plot
```

```
>>> from sage.all import *
>>> MK = graphs.MoebiusKantorGraph()
>>> MK
Moebius-Kantor Graph: Graph on 16 vertices
>>> MK.graph6_string()
'OhCGKE?O@?ACAC@I?Q_AS'
>>> (graphs.MoebiusKantorGraph()).show()      # long time                    #␣
↪needs sage.plot
```

sage.graphs.generators.smallgraphs.**MoserSpindle**(*immutable=False*)

Return the Moser spindle.

For more information, see the Wikipedia article Moser_spindle.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

The Moser spindle is a planar graph having 7 vertices and 11 edges:

```
sage: G = graphs.MoserSpindle(); G
Moser spindle: Graph on 7 vertices
sage: G.is_planar()
True
sage: G.order()
7
sage: G.size()
11
```

```
>>> from sage.all import *
>>> G = graphs.MoserSpindle(); G
Moser spindle: Graph on 7 vertices
>>> G.is_planar()
True
>>> G.order()
7
>>> G.size()
11
```

It is a Hamiltonian graph with radius 2, diameter 2, and girth 3:

```
sage: G.is_hamiltonian()                                                    #␣
↪needs sage.numerical.mip
True
sage: G.radius()
```

(continues on next page)

(continued from previous page)

```
2
sage: G.diameter()
2
sage: G.girth()
3
```

```
>>> from sage.all import *
>>> G.is_hamiltonian()                                                    #␣
↪needs sage.numerical.mip
True
>>> G.radius()
2
>>> G.diameter()
2
>>> G.girth()
3
```

The Moser spindle can be drawn in the plane as a unit distance graph, has chromatic number 4, and its automorphism group is isomorphic to the dihedral group D_4:

```
sage: pos = G.get_pos()
sage: all(sum((ui-vi)**2 for ui, vi in zip(pos[u], pos[v])) == 1
....:         for u, v in G.edge_iterator(labels=None))
True
sage: G.chromatic_number()
4
sage: ag = G.automorphism_group()
sage: ag.is_isomorphic(DihedralGroup(4))
True
```

```
>>> from sage.all import *
>>> pos = G.get_pos()
>>> all(sum((ui-vi)**Integer(2) for ui, vi in zip(pos[u], pos[v])) == Integer(1)
...         for u, v in G.edge_iterator(labels=None))
True
>>> G.chromatic_number()
4
>>> ag = G.automorphism_group()
>>> ag.is_isomorphic(DihedralGroup(Integer(4)))
True
```

sage.graphs.generators.smallgraphs.**MurtyGraph**(*immutable=False*)

Return the Murty graph.

Consider the complete bipartite graph $K_{3,3}$. There is a set of three black vertices and a set of three white vertices. Now, consider splicing the complete graph K_4 with one of the black vertices, this generates the graph $K_4 \odot K_{3,3}$. The Murty graph is obtained from $K_4 \odot K_{3,3}$ with the addition of an edge joining the remaining two black vertices. The Murty graph is free of conformal bicycles; in other words, the Murty graph is an example of a graph that is Birkhoff-von Neumann as well as PM-compact.

This is the smallest brick that is Birkhoff-von Neumann, aka a solid brick, but is not odd-intercyclic. It is in this context that Prof. U.S.R. Murty first stumbled upon this graph, and it also appears in the work of Carvalho, Lucchesi, and Murty [CLM2006].

PLOTTING:

Upon construction, the position dictionary is filled to override the spring-layout algorithm. By convention, the Murty graph is displayed as mentioned in the paper [CKWL2019], with the first two (noncubic) vertices on the top row, the second three vertices (that form a stable set) in the middle row, and the remaining three vertices (that form a triangle) at the bottom.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

OUTPUT:

- `G` – the Murty graph

EXAMPLES:

Construct and show the Murty graph:

```
sage: g = graphs.MurtyGraph()
sage: g.name()
'Murty Graph'
sage: g.order()
8
sage: g.size()
13
sage: g.girth()
3
sage: g.diameter()
2
sage: g.is_hamiltonian()
True
sage: g.show()                              # long time                    #␣
↪needs sage.plot
```

```
>>> from sage.all import *
>>> g = graphs.MurtyGraph()
>>> g.name()
'Murty Graph'
>>> g.order()
8
>>> g.size()
13
>>> g.girth()
3
>>> g.diameter()
2
>>> g.is_hamiltonian()
True
>>> g.show()                            # long time                    #␣
↪needs sage.plot
```

REFERENCES:

- [CKWL2019]
- [CLM2006]
- [LM2024]

AUTHORS:

- Janmenjaya Panda (2024-08-03)

sage.graphs.generators.smallgraphs.**NauruGraph**(*embedding=2*, *immutable=False*)

Return the Nauru Graph.

See the Wikipedia article Nauru_graph.

INPUT:

- `embedding` – integer (default: 2); two embeddings are available, and can be selected by setting `embedding` to 1 or 2
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.NauruGraph()
sage: g.order()
24
sage: g.size()
36
sage: g.girth()
6
sage: g.diameter()
4
sage: g.show()                                                              #␣
↪needs sage.plot
sage: graphs.NauruGraph(embedding=1).show()      # long time                #␣
↪needs sage.plot
```

```
>>> from sage.all import *
>>> g = graphs.NauruGraph()
>>> g.order()
24
>>> g.size()
36
>>> g.girth()
6
>>> g.diameter()
4
>>> g.show()                                                                #␣
↪needs sage.plot
>>> graphs.NauruGraph(embedding=Integer(1)).show()      # long time          ␣
↪     # needs sage.plot
```

sage.graphs.generators.smallgraphs.**PappusGraph**(*immutable=False*)

Return the Pappus graph, a graph on 18 vertices.

The Pappus graph is cubic, symmetric, and distance-regular.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: G = graphs.PappusGraph()
sage: G.show()                              # long time                          #␣
↪needs sage.plot
sage: L = graphs.LCFGraph(18, [5,7,-7,7,-7,-5], 3)
sage: L.show()                              # long time                          #␣
↪needs sage.plot
sage: G.is_isomorphic(L)
True
```

```
>>> from sage.all import *
>>> G = graphs.PappusGraph()
>>> G.show()                              # long time                          #␣
↪needs sage.plot
>>> L = graphs.LCFGraph(Integer(18), [Integer(5),Integer(7),-Integer(7),
↪Integer(7),-Integer(7),-Integer(5)], Integer(3))
>>> L.show()                              # long time                          #␣
↪needs sage.plot
>>> G.is_isomorphic(L)
True
```

sage.graphs.generators.smallgraphs.**PerkelGraph**(*immutable=False*)

Return the Perkel Graph.

The Perkel Graph is a 6-regular graph with 57 vertices and 171 edges. It is the unique distance-regular graph with intersection array $(6, 5, 2; 1, 1, 3)$. For more information, see the Wikipedia article Perkel_graph or https://www.win.tue.nl/~aeb/graphs/Perkel.html.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.PerkelGraph(); g
Perkel Graph: Graph on 57 vertices
sage: g.is_distance_regular(parameters=True)
([6, 5, 2, None], [None, 1, 1, 3])
```

```
>>> from sage.all import *
>>> g = graphs.PerkelGraph(); g
Perkel Graph: Graph on 57 vertices
>>> g.is_distance_regular(parameters=True)
([6, 5, 2, None], [None, 1, 1, 3])
```

sage.graphs.generators.smallgraphs.**PetersenGraph**(*immutable=False*)

Return the Petersen Graph.

The Petersen Graph is a named graph that consists of 10 vertices and 15 edges, usually drawn as a five-point star embedded in a pentagon.

The Petersen Graph is a common counterexample. For example, it is not Hamiltonian.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

PLOTTING: See the plotting section for the generalized Petersen graphs.

EXAMPLES: We compare below the Petersen graph with the default spring-layout versus a planned position dictionary of (x, y) tuples:

```
sage: petersen_spring = Graph({0:[1,4,5], 1:[0,2,6], 2:[1,3,7],
....:                          3:[2,4,8], 4:[0,3,9], 5:[0,7,8],
....:                          6:[1,8,9], 7:[2,5,9], 8:[3,5,6],
....:                          9:[4,6,7]})
sage: petersen_spring.show()            # long time                          #␣
↪needs sage.plot
sage: petersen_database = graphs.PetersenGraph()
sage: petersen_database.show()          # long time                          #␣
↪needs sage.plot
```

```
>>> from sage.all import *
>>> petersen_spring = Graph({Integer(0):[Integer(1),Integer(4),Integer(5)],␣
↪Integer(1):[Integer(0),Integer(2),Integer(6)], Integer(2):[Integer(1),
↪Integer(3),Integer(7)],
...                          Integer(3):[Integer(2),Integer(4),Integer(8)],␣
↪Integer(4):[Integer(0),Integer(3),Integer(9)], Integer(5):[Integer(0),
↪Integer(7),Integer(8)],
...                          Integer(6):[Integer(1),Integer(8),Integer(9)],␣
↪Integer(7):[Integer(2),Integer(5),Integer(9)], Integer(8):[Integer(3),
↪Integer(5),Integer(6)],
...                          Integer(9):[Integer(4),Integer(6),Integer(7)]})
>>> petersen_spring.show()            # long time                          #␣
↪needs sage.plot
>>> petersen_database = graphs.PetersenGraph()
>>> petersen_database.show()          # long time                          #␣
↪needs sage.plot
```

sage.graphs.generators.smallgraphs.**PoussinGraph**(*immutable=False*)

Return the Poussin Graph.

For more information on the Poussin Graph, see its corresponding Wolfram page.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.PoussinGraph()
sage: g.order()
15
sage: g.is_planar()
True
```

```
>>> from sage.all import *
>>> g = graphs.PoussinGraph()
>>> g.order()
15
>>> g.is_planar()
True
```

sage.graphs.generators.smallgraphs.**RobertsonGraph**(*immutable=False*)

Return the Robertson graph.

See the Wikipedia article Robertson_graph.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.RobertsonGraph()
sage: g.order()
19
sage: g.size()
38
sage: g.diameter()
3
sage: g.girth()
5
sage: g.charpoly().factor()
(x - 4) * (x - 1)^2 * (x^2 + x - 5) * (x^2 + x - 1)
 * (x^2 - 3)^2 * (x^2 + x - 4)^2 * (x^2 + x - 3)^2
sage: g.chromatic_number()
3
sage: g.is_hamiltonian()                                                    #␣
↪needs sage.numerical.mip
True
sage: g.is_vertex_transitive()
False
```

```
>>> from sage.all import *
>>> g = graphs.RobertsonGraph()
>>> g.order()
19
>>> g.size()
38
>>> g.diameter()
3
>>> g.girth()
5
>>> g.charpoly().factor()
(x - 4) * (x - 1)^2 * (x^2 + x - 5) * (x^2 + x - 1)
 * (x^2 - 3)^2 * (x^2 + x - 4)^2 * (x^2 + x - 3)^2
>>> g.chromatic_number()
3
>>> g.is_hamiltonian()                                                      #␣
↪needs sage.numerical.mip
True
>>> g.is_vertex_transitive()
False
```

`sage.graphs.generators.smallgraphs.`**`SchlaefliGraph`**(*immutable=False*)

Return the Schläfli graph.

The Schläfli graph is the only strongly regular graphs of parameters $(27, 16, 10, 8)$ (see [GR2001]).

For more information, see the Wikipedia article Schläfli_graph.

See also

Graph.is_strongly_regular() – tests whether a graph is strongly regular and/or returns its parameters.

Todo

Find a beautiful layout for this beautiful graph.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

Checking that the method actually returns the Schläfli graph:

```
sage: S = graphs.SchlaefliGraph()
sage: S.is_strongly_regular(parameters=True)
(27, 16, 10, 8)
```

```
>>> from sage.all import *
>>> S = graphs.SchlaefliGraph()
>>> S.is_strongly_regular(parameters=True)
(27, 16, 10, 8)
```

The graph is vertex-transitive:

```
sage: S.is_vertex_transitive()                                                  #␣
↪needs sage.groups
True
```

```
>>> from sage.all import *
>>> S.is_vertex_transitive()                                                    #␣
↪needs sage.groups
True
```

The neighborhood of each vertex is isomorphic to the complement of the Clebsch graph:

```
sage: neighborhood = S.subgraph(vertices=S.neighbors(0))
sage: graphs.ClebschGraph().complement().is_isomorphic(neighborhood)
True
```

```
>>> from sage.all import *
>>> neighborhood = S.subgraph(vertices=S.neighbors(Integer(0)))
>>> graphs.ClebschGraph().complement().is_isomorphic(neighborhood)
True
```

sage.graphs.generators.smallgraphs.**ShrikhandeGraph**(*immutable=False*)

Return the Shrikhande graph.

For more information, see the MathWorld article on the Shrikhande graph or the Wikipedia article Shrikhande_graph.

See also

Graph.is_strongly_regular() – tests whether a graph is strongly regular and/or returns its parameters.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

The Shrikhande graph was defined by S. S. Shrikhande in 1959. It has 16 vertices and 48 edges, and is strongly regular of degree 6 with parameters $(2, 2)$:

```
sage: G = graphs.ShrikhandeGraph(); G
Shrikhande graph: Graph on 16 vertices
sage: G.order()
16
sage: G.size()
48
sage: G.is_regular(6)
True
sage: set([ len([x for x in G.neighbors(i) if x in G.neighbors(j)])
....:       for i in range(G.order())
....:       for j in range(i) ])
{2}
```

```
>>> from sage.all import *
>>> G = graphs.ShrikhandeGraph(); G
Shrikhande graph: Graph on 16 vertices
>>> G.order()
16
>>> G.size()
48
>>> G.is_regular(Integer(6))
True
>>> set([ len([x for x in G.neighbors(i) if x in G.neighbors(j)])
...       for i in range(G.order())
...       for j in range(i) ])
{2}
```

It is non-planar, and both Hamiltonian and Eulerian:

```
sage: G.is_planar()
False
sage: G.is_hamiltonian()                                                    #␣
↪needs sage.numerical.mip
True
sage: G.is_eulerian()
True
```

```
>>> from sage.all import *
>>> G.is_planar()
False
>>> G.is_hamiltonian()                                                      #␣
```

(continues on next page)

(continued from previous page)

```
↪needs sage.numerical.mip
True
>>> G.is_eulerian()
True
```

It has radius 2, diameter 2, and girth 3:

```
sage: G.radius()
2
sage: G.diameter()
2
sage: G.girth()
3
```

```
>>> from sage.all import *
>>> G.radius()
2
>>> G.diameter()
2
>>> G.girth()
3
```

Its chromatic number is 4 and its automorphism group is of order 192:

```
sage: G.chromatic_number()
4
sage: G.automorphism_group().cardinality()                                  #␣
↪needs sage.groups
192
```

```
>>> from sage.all import *
>>> G.chromatic_number()
4
>>> G.automorphism_group().cardinality()                                  #␣
↪needs sage.groups
192
```

It is an integral graph since it has only integral eigenvalues:

```
sage: G.characteristic_polynomial().factor()                                #␣
↪needs sage.libs.pari sage.modules
(x - 6) * (x - 2)^6 * (x + 2)^9
```

```
>>> from sage.all import *
>>> G.characteristic_polynomial().factor()                                #␣
↪needs sage.libs.pari sage.modules
(x - 6) * (x - 2)^6 * (x + 2)^9
```

It is a toroidal graph, and its embedding on a torus is dual to an embedding of the Dyck graph (*DyckGraph*).

sage.graphs.generators.smallgraphs.**SimsGewirtzGraph**(*immutable=False*)

Return the Sims-Gewirtz Graph.

This graph is obtained from the Higman Sims graph by considering the graph induced by the vertices at distance two from the vertices of an (any) edge. It is the only strongly regular graph with parameters $v = 56$, $k = 10$, $\lambda = 0$, $\mu = 2$

For more information on the Sylvester graph, see https://www.win.tue.nl/~aeb/graphs/Sims-Gewirtz.html or its Wikipedia article Gewirtz_graph.

See also

- *HigmanSimsGraph()*.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.SimsGewirtzGraph(); g
Sims-Gewirtz Graph: Graph on 56 vertices
sage: g.order()
56
sage: g.size()
280
sage: g.is_strongly_regular(parameters = True)
(56, 10, 0, 2)
```

```
>>> from sage.all import *
>>> g = graphs.SimsGewirtzGraph(); g
Sims-Gewirtz Graph: Graph on 56 vertices
>>> g.order()
56
>>> g.size()
280
>>> g.is_strongly_regular(parameters = True)
(56, 10, 0, 2)
```

sage.graphs.generators.smallgraphs.**SousselierGraph**(*immutable=False*)

Return the Sousselier Graph.

The Sousselier graph is a hypohamiltonian graph on 16 vertices and 27 edges. For more information, see Wikipedia article Sousselier_graph or the corresponding French Wikipedia page.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.SousselierGraph()
sage: g.order()
16
sage: g.size()
27
sage: g.radius()
2
```

(continues on next page)

(continued from previous page)

```
sage: g.diameter()
3
sage: g.automorphism_group().cardinality()                                  #␣
↪needs sage.groups
2
sage: g.is_hamiltonian()                                                    #␣
↪needs sage.numerical.mip
False
sage: g.delete_vertex(g.random_vertex())
sage: g.is_hamiltonian()                                                    #␣
↪needs sage.numerical.mip
True
```

```
>>> from sage.all import *
>>> g = graphs.SousselierGraph()
>>> g.order()
16
>>> g.size()
27
>>> g.radius()
2
>>> g.diameter()
3
>>> g.automorphism_group().cardinality()                                    #␣
↪needs sage.groups
2
>>> g.is_hamiltonian()                                                      #␣
↪needs sage.numerical.mip
False
>>> g.delete_vertex(g.random_vertex())
>>> g.is_hamiltonian()                                                      #␣
↪needs sage.numerical.mip
True
```

sage.graphs.generators.smallgraphs.**SuzukiGraph**(*immutable=False*)

Return the Suzuki Graph.

The Suzuki graph has 1782 vertices, and is strongly regular with parameters $(1782, 416, 100, 96)$. Known as S.15 in [Hub1975].

Note

It takes approximately 50 seconds to build this graph. Do not be too impatient.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.SuzukiGraph(); g  # optional internet # not tested
Suzuki graph: Graph on 1782 vertices
```

(continues on next page)

(continued from previous page)

```
sage: g.is_strongly_regular(parameters=True)  # optional internet # not tested
(1782, 416, 100, 96)
```

```
>>> from sage.all import *
>>> g = graphs.SuzukiGraph(); g  # optional internet # not tested
Suzuki graph: Graph on 1782 vertices
>>> g.is_strongly_regular(parameters=True)  # optional internet # not tested
(1782, 416, 100, 96)
```

sage.graphs.generators.smallgraphs.**SylvesterGraph**(*immutable=False*)

Return the Sylvester Graph.

This graph is obtained from the Hoffman Singleton graph by considering the graph induced by the vertices at distance two from the vertices of an (any) edge.

For more information on the Sylvester graph, see https://www.win.tue.nl/~aeb/graphs/Sylvester.html.

See also

- *HoffmanSingletonGraph()*.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.SylvesterGraph(); g
Sylvester Graph: Graph on 36 vertices
sage: g.order()
36
sage: g.size()
90
sage: g.is_regular(k=5)
True
```

```
>>> from sage.all import *
>>> g = graphs.SylvesterGraph(); g
Sylvester Graph: Graph on 36 vertices
>>> g.order()
36
>>> g.size()
90
>>> g.is_regular(k=Integer(5))
True
```

sage.graphs.generators.smallgraphs.**SzekeresSnarkGraph**(*immutable=False*)

Return the Szekeres Snark Graph.

The Szekeres graph is a snark with 50 vertices and 75 edges. For more information on this graph, see the Wikipedia article Szekeres_snark.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.SzekeresSnarkGraph()
sage: g.order()
50
sage: g.size()
75
sage: g.chromatic_number()
3
```

```
>>> from sage.all import *
>>> g = graphs.SzekeresSnarkGraph()
>>> g.order()
50
>>> g.size()
75
>>> g.chromatic_number()
3
```

sage.graphs.generators.smallgraphs.**ThomsenGraph**(*immutable=False*)

Return the Thomsen Graph.

The Thomsen Graph is actually a complete bipartite graph with $(n1, n2) = (3, 3)$. It is also called the Utility graph.

PLOTTING: See CompleteBipartiteGraph.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: T = graphs.ThomsenGraph()
sage: T
Thomsen graph: Graph on 6 vertices
sage: T.graph6_string()
'EFz_'
sage: (graphs.ThomsenGraph()).show()     # long time                              #␣
↪needs sage.plot
```

```
>>> from sage.all import *
>>> T = graphs.ThomsenGraph()
>>> T
Thomsen graph: Graph on 6 vertices
>>> T.graph6_string()
'EFz_'
>>> (graphs.ThomsenGraph()).show()     # long time                                #␣
↪needs sage.plot
```

sage.graphs.generators.smallgraphs.**TietzeGraph**(*immutable=False*)

Return the Tietze Graph.

For more information on the Tietze Graph, see the Wikipedia article Tietze%27s_graph.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.TietzeGraph()
sage: g.order()
12
sage: g.size()
18
sage: g.diameter()
3
sage: g.girth()
3
sage: g.automorphism_group().cardinality()                                  #␣
↪needs sage.groups
12
sage: g.automorphism_group().is_isomorphic(groups.permutation.Dihedral(6))  #␣
↪needs sage.groups
True
```

```
>>> from sage.all import *
>>> g = graphs.TietzeGraph()
>>> g.order()
12
>>> g.size()
18
>>> g.diameter()
3
>>> g.girth()
3
>>> g.automorphism_group().cardinality()                                    #␣
↪needs sage.groups
12
>>> g.automorphism_group().is_isomorphic(groups.permutation.Dihedral(Integer(6)))␣
↪     # needs sage.groups
True
```

sage.graphs.generators.smallgraphs.**TricornGraph**(*immutable=False*)

Return the Tricorn graph.

The Tricorn graph is obtained by splicing a complete graph K_4 with the the triangular circular ladder graph $\overline{C_6}$. (Note that this generates a unique graph as both of the graphs K_4 and $\overline{C_6}$ are vertex-transitive). It is a nonsolid brick. This matching covered graph is one of the ten extremal cubic bricks. (A matching covered graph G is *extremal* if $\Phi(G) = dim(\mathcal{L}\backslash(G))$, where $\Phi(G)$ denotes the number of perfect matchings of G, and $dim(\mathcal{L}\backslash(G))$ stands for the dimension of the linear space of G).

The Tricorn graph has no removable doubletons and has precisely three removable edges. The wheel graph W_5 and the complete graph K_4 are matching minors of the Tricorn graph.

As per a theorem of Lovász [Lov1983], each non bipartite matching covered graph has a conformal subgraph which is either a bi-subdivision of K_4 or of $\overline{C_6}$ or both. In their paper, Kothari and Murty [KM2015] characterized those planar bricks that are free of $\overline{C_6}$ (that is, the planar bricks that do not contain a bi-subdivision of $\overline{C_6}$ as a conformal subgraph). Besides two infinite families of matching covered graphs (odd wheel graphs and staircase graphs of order *4k*), the Tricorn graph is the only exception brick that is simple, planar and free of $\overline{C_6}$.

PLOTTING:

Upon construction, the position dictionary is filled to override the spring-layout algorithm. By convention, the Tricorn graph is displayed as mentioned in the book [LM2024], with the central vertex being the 0-th one. Rest of the nine vertices are shown in groups of three, one on the top, rest two on the bottom left and on the bottom right corners respectively.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

OUTPUT:

- `G` – the Tricorn graph

EXAMPLES:

Construct and show the Tricorn graph; note that the edges $(2, 3)$, $(5, 6)$ and $(8, 9)$ are the only removable edges of the Tricorn graph:

```
sage: g = graphs.TricornGraph()
sage: g.name()
'Tricorn Graph'
sage: g.order()
10
sage: g.size()
15
sage: g.girth()
3
sage: g.diameter()
3
sage: g.is_hamiltonian()
True
sage: g.show()                              # long time                              #␣
↪needs sage.plot
```

```
>>> from sage.all import *
>>> g = graphs.TricornGraph()
>>> g.name()
'Tricorn Graph'
>>> g.order()
10
>>> g.size()
15
>>> g.girth()
3
>>> g.diameter()
3
>>> g.is_hamiltonian()
True
>>> g.show()                            # long time                              #␣
↪needs sage.plot
```

REFERENCES:

- [KM2015]
- [LM2024]
- [Lov1983]

AUTHORS:

- Janmenjaya Panda (2024-08-02)

sage.graphs.generators.smallgraphs.**TruncatedIcosidodecahedralGraph**(*immutable=False*)

Return the truncated icosidodecahedron.

The truncated icosidodecahedron is an Archimedean solid with 30 square faces, 20 regular hexagonal faces, 12 regular decagonal faces, 120 vertices and 180 edges. For more information, see the Wikipedia article Truncated_icosidodecahedron.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

Unfortunately, this graph can not be constructed currently, due to numerical issues:

```
sage: g = graphs.TruncatedIcosidodecahedralGraph(); g                          #␣
↪needs sage.geometry.polyhedron sage.groups sage.rings.number_field
Traceback (most recent call last):
...
ValueError: *Error: Numerical inconsistency is found.  Use the GMP exact␣
↪arithmetic.
sage: g.order(), g.size()               # not tested                           #␣
↪needs sage.geometry.polyhedron sage.groups sage.rings.number_field
(120, 180)
```

```
>>> from sage.all import *
>>> g = graphs.TruncatedIcosidodecahedralGraph(); g                            #␣
↪needs sage.geometry.polyhedron sage.groups sage.rings.number_field
Traceback (most recent call last):
...
ValueError: *Error: Numerical inconsistency is found.  Use the GMP exact␣
↪arithmetic.
>>> g.order(), g.size()               # not tested                             #␣
↪needs sage.geometry.polyhedron sage.groups sage.rings.number_field
(120, 180)
```

sage.graphs.generators.smallgraphs.**TruncatedTetrahedralGraph**(*immutable=False*)

Return the truncated tetrahedron.

The truncated tetrahedron is an Archimedean solid with 12 vertices and 18 edges. For more information, see the Wikipedia article Truncated_tetrahedron.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.TruncatedTetrahedralGraph(); g
Truncated Tetrahedron: Graph on 12 vertices
sage: g.order(), g.size()
(12, 18)
sage: g.is_isomorphic(polytopes.simplex(3).truncation().graph())               #␣
↪needs sage.geometry.polyhedron
True
```

```
>>> from sage.all import *
>>> g = graphs.TruncatedTetrahedralGraph(); g
Truncated Tetrahedron: Graph on 12 vertices
>>> g.order(), g.size()
(12, 18)
>>> g.is_isomorphic(polytopes.simplex(Integer(3)).truncation().graph())
↪     # needs sage.geometry.polyhedron
True
```

sage.graphs.generators.smallgraphs.**Tutte12Cage**(*immutable=False*)

Return the Tutte 12-Cage.

See the Wikipedia article Tutte_12-cage.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.Tutte12Cage()
sage: g.order()
126
sage: g.size()
189
sage: g.girth()
12
sage: g.diameter()
6
sage: g.show()
↪needs sage.plot
```

```
>>> from sage.all import *
>>> g = graphs.Tutte12Cage()
>>> g.order()
126
>>> g.size()
189
>>> g.girth()
12
>>> g.diameter()
6
>>> g.show()
↪needs sage.plot
```

sage.graphs.generators.smallgraphs.**TutteCoxeterGraph**(*embedding=2*, *immutable=False*)

Return the Tutte-Coxeter graph.

See the Wikipedia article Tutte-Coxeter_graph.

INPUT:

- `embedding` – integer (default: 2); two embeddings are available, and can be selected by setting `embedding` to 1 or 2
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.TutteCoxeterGraph()
sage: g.order()
30
sage: g.size()
45
sage: g.girth()
8
sage: g.diameter()
4
sage: g.show()                                                              #␣
↪needs sage.plot
sage: graphs.TutteCoxeterGraph(embedding=1).show()      # long time         #␣
↪needs sage.plot
```

```
>>> from sage.all import *
>>> g = graphs.TutteCoxeterGraph()
>>> g.order()
30
>>> g.size()
45
>>> g.girth()
8
>>> g.diameter()
4
>>> g.show()                                                                #␣
↪needs sage.plot
>>> graphs.TutteCoxeterGraph(embedding=Integer(1)).show()      # long time      ␣
↪     # needs sage.plot
```

sage.graphs.generators.smallgraphs.**TutteGraph**(*immutable=False*)

Return the Tutte Graph.

The Tutte graph is a 3-regular, 3-connected, and planar non-hamiltonian graph. For more information on the Tutte Graph, see the Wikipedia article Tutte_graph.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.TutteGraph()
sage: g.order()
46
sage: g.size()
69
sage: g.is_planar()
True
sage: g.vertex_connectivity()           # long time                         #␣
↪needs sage.numerical.mip
3
sage: g.girth()
4
```

(continues on next page)

(continued from previous page)

```
sage: g.automorphism_group().cardinality()                                  #␣
↪needs sage.groups
3
sage: g.is_hamiltonian()                                                    #␣
↪needs sage.numerical.mip
False
```

```
>>> from sage.all import *
>>> g = graphs.TutteGraph()
>>> g.order()
46
>>> g.size()
69
>>> g.is_planar()
True
>>> g.vertex_connectivity()           # long time                          #␣
↪needs sage.numerical.mip
3
>>> g.girth()
4
>>> g.automorphism_group().cardinality()                                    #␣
↪needs sage.groups
3
>>> g.is_hamiltonian()                                                      #␣
↪needs sage.numerical.mip
False
```

sage.graphs.generators.smallgraphs.**TwinplexGraph**(*embedding='LM'*, *immutable=False*)

Return the Twinplex graph.

The Twinplex graph is a cubic hamiltonian graph of order 12 with the graph crossing number 2 and has a girth 5 (that is the maximal girth among all cubic graphs on 12 vertices [CHNP2020]). It corresponds to the graph labeled as Γ_2 by Fischer and Little [FiLi2001]. The Twinplex graph has LCF notation $[-5, -4, 4, -4, 4, 5, -4, 5, -4, 4, -5, 4]$.

The Fischer-Little Theorem [FiLi2001] may be stated as follows [LM2024]:

A near-bipartite graph is non-Pfaffian if and only if it contains one of the graphs $K_{3,3}$, Γ_1 and Γ_2 as an S-minor.

Norine and Thomas [NT2007] use the term `Twinplex` to describe one of the 12-vertex cubic graphs, Γ_1 and Γ_2, as defined by Fischer and Little [FiLi2001]. However, the figure in their paper that supposedly provides embeddings for the graphs labeled Cubeplex and Twinplex actually shows both embeddings corresponding to Fischer and Little's Γ_1, which is the Cubeplex graph. Followingly, for `embedding='NT'`, we present a correct version of the Twinplex graph with a slight modification of the embedding that is labeled as `Twinplex` in the paper of Norine and Thomas [NT2007].

PLOTTING:

Upon construction, the position dictionary is filled to override the spring-layout algorithm. For different values of the parameter `embedding`, the Twinplex graph is displayed as it is mentioned in the respective paper/ book. Note that for `embedding='NT'`, a correct embedding of the Twinplex graph is displayed with a minor modification to the (incorrect) embedding shown in the paper [NT2007].

INPUT:

- `embedding` – string (default: `'LM'`)
  - `'LM'` displays the embedding as shown for Γ_2 by Lucchesi and Murty [LM2024]

  - `'FL'` displays the embedding as shown for Γ_2 by Fischer and Little [FiLi2001]
  - `'NT'` displays the correct embedding with a minor modification to the one shown as the (incorrect) `Twinplex` by Norine and Thomas [NT2007]
  - `'RST'` displays the embedding as shown for the `Twinplex` by Robertson, Seymour and Thomas [RST2019]
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

OUTPUT:

- `G` – the Twinplex graph; note that a `ValueError` is returned if `embedding` is none of `'FT'`, `'NT'`, `'RST'` or `'LM'`

EXAMPLES:

Construct and show the Twinplex graph:

```
sage: g = graphs.TwinplexGraph()
sage: g.name()
'Twinplex Graph'
sage: g.order()
12
sage: g.size()
18
sage: g.girth()
5
sage: g.diameter()
3
sage: g.is_hamiltonian()
True
sage: g.crossing_number()
2
sage: g.show()                              # long time                              #␣
↪needs sage.plot
```

```
>>> from sage.all import *
>>> g = graphs.TwinplexGraph()
>>> g.name()
'Twinplex Graph'
>>> g.order()
12
>>> g.size()
18
>>> g.girth()
5
>>> g.diameter()
3
>>> g.is_hamiltonian()
True
>>> g.crossing_number()
2
>>> g.show()                              # long time                              #␣
↪needs sage.plot
```

See also

CubeplexGraph()

AUTHORS:

- Janmenjaya Panda (2024-08-03)

sage.graphs.generators.smallgraphs.**U42Graph216**(*immutable=False*)

Return a (216,40,4,8)-strongly regular graph from [CRS2016].

Build the graph, interpreting the $U_4(2)$-action considered in [CRS2016] as the one on the hyperbolic lines of the corresponding unitary polar space, and then doing the unique merging of the orbitals leading to a graph with the parameters in question.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: G=graphs.U42Graph216()                    # optional - gap_package_grape
sage: G.is_strongly_regular(parameters=True)    # optional - gap_package_grape
(216, 40, 4, 8)
```

```
>>> from sage.all import *
>>> G=graphs.U42Graph216()                    # optional - gap_package_grape
>>> G.is_strongly_regular(parameters=True)    # optional - gap_package_grape
(216, 40, 4, 8)
```

sage.graphs.generators.smallgraphs.**U42Graph540**(*immutable=False*)

Return a (540,187,58,68)-strongly regular graph from [CRS2016].

Build the graph, interpreting the $U_4(2)$-action considered in [CRS2016] as the action of $U_4(2) = Sp_4(3) < U_4(3)$ on the nonsingular, w.r.t. to the Hermitean form stabilised by $U_4(3)$, points of the 3-dimensional projective space over $GF(9)$. There are several possible mergings of orbitals, some leading to non-isomorphic graphs with the same parameters. We found the merging here using [FK1991].

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: G = graphs.U42Graph540()                  # optional - gap_package_grape
sage: G.is_strongly_regular(parameters=True)    # optional - gap_package_grape
(540, 187, 58, 68)
```

```
>>> from sage.all import *
>>> G = graphs.U42Graph540()                  # optional - gap_package_grape
>>> G.is_strongly_regular(parameters=True)    # optional - gap_package_grape
(540, 187, 58, 68)
```

sage.graphs.generators.smallgraphs.**WagnerGraph**(*immutable=False*)

Return the Wagner Graph.

See the Wikipedia article Wagner_graph.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.WagnerGraph()
sage: g.order()
8
sage: g.size()
12
sage: g.girth()
4
sage: g.diameter()
2
sage: g.show()                                                              #␣
↪needs sage.plot
```

```
>>> from sage.all import *
>>> g = graphs.WagnerGraph()
>>> g.order()
8
>>> g.size()
12
>>> g.girth()
4
>>> g.diameter()
2
>>> g.show()                                                                #␣
↪needs sage.plot
```

`sage.graphs.generators.smallgraphs.`**`WatkinsSnarkGraph`**(*immutable=False*)

Return the Watkins Snark Graph.

The Watkins Graph is a snark with 50 vertices and 75 edges. For more information, see the Wikipedia article Watkins_snark.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.WatkinsSnarkGraph()
sage: g.order()
50
sage: g.size()
75
sage: g.chromatic_number()
3
```

```
>>> from sage.all import *
>>> g = graphs.WatkinsSnarkGraph()
>>> g.order()
50
>>> g.size()
75
```

(continues on next page)

(continued from previous page)

```
>>> g.chromatic_number()
3
```

sage.graphs.generators.smallgraphs.**WellsGraph**(*immutable=False*)

Return the Wells graph.

For more information on the Wells graph (also called Armanios-Wells graph), see this page.

The implementation follows the construction given on page 266 of [BCN1989]. This requires to create intermediate graphs and run a small isomorphism test, while everything could be replaced by a pre-computed list of edges. I believe that it is better to keep "the recipe" in the code, however, as it is quite unlikely that this could become the most time-consuming operation in any sensible algorithm, and …. "preserves knowledge", which is what open-source software is meant to do.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.WellsGraph(); g
Wells graph: Graph on 32 vertices
sage: g.order()
32
sage: g.size()
80
sage: g.girth()
5
sage: g.diameter()
4
sage: g.chromatic_number()
4
sage: g.is_regular(k=5)
True
```

```
>>> from sage.all import *
>>> g = graphs.WellsGraph(); g
Wells graph: Graph on 32 vertices
>>> g.order()
32
>>> g.size()
80
>>> g.girth()
5
>>> g.diameter()
4
>>> g.chromatic_number()
4
>>> g.is_regular(k=Integer(5))
True
```

sage.graphs.generators.smallgraphs.**WienerArayaGraph**(*immutable=False*)

Return the Wiener-Araya Graph.

The Wiener-Araya Graph is a planar hypohamiltonian graph on 42 vertices and 67 edges. For more information, see the Wolfram Page on the Wiener-Araya Graph or Wikipedia article Wiener-Araya_graph.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.WienerArayaGraph()
sage: g.order()
42
sage: g.size()
67
sage: g.girth()
4
sage: g.is_planar()
True
sage: g.is_hamiltonian()                # not tested (30s)                    #␣
↪needs sage.numerical.mip
False
sage: g.delete_vertex(g.random_vertex())
sage: g.is_hamiltonian()                                                      #␣
↪needs sage.numerical.mip
True
```

```
>>> from sage.all import *
>>> g = graphs.WienerArayaGraph()
>>> g.order()
42
>>> g.size()
67
>>> g.girth()
4
>>> g.is_planar()
True
>>> g.is_hamiltonian()                # not tested (30s)                    #␣
↪needs sage.numerical.mip
False
>>> g.delete_vertex(g.random_vertex())
>>> g.is_hamiltonian()                                                      #␣
↪needs sage.numerical.mip
True
```

2.17 Generation of trees

This module gathers methods related to the generation of trees. All these methods appear in *sage.graphs.graph_generators*.

BalancedTree()	Return the perfectly balanced tree of height $h \geq 1$, whose root has degree $r \geq 2$.
DegreeSequenceTree()	Return a tree with the given degree sequence.
FibonacciTree()	Return the graph of the Fibonacci Tree F_n.
RandomLobster()	Return a random lobster.
RandomTree()	Return a random tree on n nodes numbered 0 through $n-1$.
RandomTreePowerlaw()	Return a tree with a power law degree distribution, or `False` on failure.
trees()	Return a generator of the distinct trees on a fixed number of vertices.
nauty_gentreeg()	Return a generator which creates non-isomorphic trees from nauty's gentreeg program.

There are different ways to enumerate all non-isomorphic trees of order n. We can use the implementation of the algorithm proposed in [WROM1986]:

```
sage: gen = graphs.trees(10)
sage: T = next(gen); T
Graph on 10 vertices
sage: T.is_tree()
True
```

```
>>> from sage.all import *
>>> gen = graphs.trees(Integer(10))
>>> T = next(gen); T
Graph on 10 vertices
>>> T.is_tree()
True
```

We can also use nauty's gentreeg:

```
sage: gen = graphs.nauty_gentreeg("10")
sage: T = next(gen); T
Graph on 10 vertices
sage: T.is_tree()
True
```

```
>>> from sage.all import *
>>> gen = graphs.nauty_gentreeg("10")
>>> T = next(gen); T
Graph on 10 vertices
>>> T.is_tree()
True
```

Note that nauty's gentreeg can only be used to generate trees with $1 \leq n \leq 128$ vertices:

```
sage: next(graphs.nauty_gentreeg("0"))
Traceback (most recent call last):
...
ValueError: wrong format of parameter options
sage: next(graphs.nauty_gentreeg("1"))
Graph on 1 vertex
sage: next(graphs.nauty_gentreeg("128"))
Graph on 128 vertices
sage: next(graphs.nauty_gentreeg("129"))
```

(continues on next page)

(continued from previous page)

```
Traceback (most recent call last):
...
ValueError: wrong format of parameter options
```

```
>>> from sage.all import *
>>> next(graphs.nauty_gentreeg("0"))
Traceback (most recent call last):
...
ValueError: wrong format of parameter options
>>> next(graphs.nauty_gentreeg("1"))
Graph on 1 vertex
>>> next(graphs.nauty_gentreeg("128"))
Graph on 128 vertices
>>> next(graphs.nauty_gentreeg("129"))
Traceback (most recent call last):
...
ValueError: wrong format of parameter options
```

We don't have this limitation with method *trees()*:

```
sage: next(graphs.trees(0))
Graph on 0 vertices
sage: next(graphs.trees(129))
Graph on 129 vertices
sage: next(graphs.trees(1000))
Graph on 1000 vertices
```

```
>>> from sage.all import *
>>> next(graphs.trees(Integer(0)))
Graph on 0 vertices
>>> next(graphs.trees(Integer(129)))
Graph on 129 vertices
>>> next(graphs.trees(Integer(1000)))
Graph on 1000 vertices
```

Nauty's gentreeg can be used to generate trees with bounded maximum degree:

```
sage: gen = graphs.nauty_gentreeg("8 -D3")
sage: all(max(g.degree()) <= 3 for g in gen)
True
sage: len(list(graphs.nauty_gentreeg("8 -D2")))
1
```

```
>>> from sage.all import *
>>> gen = graphs.nauty_gentreeg("8 -D3")
>>> all(max(g.degree()) <= Integer(3) for g in gen)
True
>>> len(list(graphs.nauty_gentreeg("8 -D2")))
1
```

Nauty's gentreeg can be used to generate trees with bounded diameter:

```
sage: all(g.diameter() == 3 for g in graphs.nauty_gentreeg("8 -Z3"))
True
sage: len(list(graphs.nauty_gentreeg("8 -Z3")))
3
```

```
>>> from sage.all import *
>>> all(g.diameter() == Integer(3) for g in graphs.nauty_gentreeg("8 -Z3"))
True
>>> len(list(graphs.nauty_gentreeg("8 -Z3")))
3
```

The number of trees on the first few vertex counts agrees with OEIS sequence A000055:

```
sage: [len(list(graphs.trees(i))) for i in range(15)]
[1, 1, 1, 1, 2, 3, 6, 11, 23, 47, 106, 235, 551, 1301, 3159]
sage: [len(list(graphs.nauty_gentreeg(str(i)))) for i in range(1, 15)]
[1, 1, 1, 2, 3, 6, 11, 23, 47, 106, 235, 551, 1301, 3159]
```

```
>>> from sage.all import *
>>> [len(list(graphs.trees(i))) for i in range(Integer(15))]
[1, 1, 1, 1, 2, 3, 6, 11, 23, 47, 106, 235, 551, 1301, 3159]
>>> [len(list(graphs.nauty_gentreeg(str(i)))) for i in range(Integer(1), Integer(15))]
[1, 1, 1, 2, 3, 6, 11, 23, 47, 106, 235, 551, 1301, 3159]
```

2.17.1 Methods

sage.graphs.generators.trees.**BalancedTree**(*r*, *h*, *immutable=False*)

Return the perfectly balanced tree of height $h \geq 1$, whose root has degree $r \geq 2$.

The number of vertices of this graph is $1 + r + r^2 + \cdots + r^h$, that is, $\frac{r^{h+1}-1}{r-1}$. The number of edges is one less than the number of vertices.

INPUT:

- `r` – positive integer ≥ 2; the degree of the root node
- `h` – positive integer ≥ 1; the height of the balanced tree
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

OUTPUT:

The perfectly balanced tree of height $h \geq 1$ and whose root has degree $r \geq 2$.

EXAMPLES:

A balanced tree whose root node has degree $r = 2$, and of height $h = 1$, has order 3 and size 2:

```
sage: G = graphs.BalancedTree(2, 1); G
Balanced tree: Graph on 3 vertices
sage: G.order()
3
sage: G.size()
2
sage: r = 2; h = 1
sage: v = 1 + r
```

(continues on next page)

(continued from previous page)

```
sage: v; v - 1
3
2
```

```
>>> from sage.all import *
>>> G = graphs.BalancedTree(Integer(2), Integer(1)); G
Balanced tree: Graph on 3 vertices
>>> G.order()
3
>>> G.size()
2
>>> r = Integer(2); h = Integer(1)
>>> v = Integer(1) + r
>>> v; v - Integer(1)
3
2
```

Plot a balanced tree of height 5, whose root node has degree $r = 3$:

```
sage: G = graphs.BalancedTree(3, 5)
sage: G.plot()                          # long time                          #␣
↪needs sage.plot
Graphics object consisting of 728 graphics primitives
```

```
>>> from sage.all import *
>>> G = graphs.BalancedTree(Integer(3), Integer(5))
>>> G.plot()                           # long time                          #␣
↪needs sage.plot
Graphics object consisting of 728 graphics primitives
```

A tree is bipartite. If its vertex set is finite, then it is planar:

```
sage: # needs networkx
sage: r = randint(2, 5); h = randint(1, 7)
sage: T = graphs.BalancedTree(r, h)
sage: T.is_bipartite()
True
sage: T.is_planar()
True
sage: v = (r^(h + 1) - 1) / (r - 1)
sage: T.order() == v
True
sage: T.size() == v - 1
True
```

```
>>> from sage.all import *
>>> # needs networkx
>>> r = randint(Integer(2), Integer(5)); h = randint(Integer(1), Integer(7))
>>> T = graphs.BalancedTree(r, h)
>>> T.is_bipartite()
True
>>> T.is_planar()
```

(continues on next page)

(continued from previous page)

```
True
>>> v = (r**(h + Integer(1)) - Integer(1)) / (r - Integer(1))
>>> T.order() == v
True
>>> T.size() == v - Integer(1)
True
```

sage.graphs.generators.trees.**Caterpillar**(*spine*, *immutable=False*)

Return the caterpillar tree with given spine sequence.

A caterpillar tree consists of leaves attached to a path (the "spine").

INPUT:

- **spine – list of nonnegative integers in the form** $[a_1, a_2, \ldots, a_n]$, where a_i is the number of leaves adjacent to the i-th vertex on the spine (except for the first and last vertex, which have $a_1 + 1$ and $a_n + 1$ leaf-neighbors, respectively)
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

OUTPUT:

A caterpillar tree of diameter $n + 1$ on $n + 2 + \sum_{i=1}^{n} a_i$ vertices, n of which are not leaves.

PLOTTING: Upon construction, the position dictionary is filled to override the spring-layout algorithm if the returned graph does not have too many vertices. The spine vertices are positioned on a straight line together with two leaves at its ends. Every edge in the drawing has unit length.

EXAMPLES:

Caterpillars with all-zero spine sequence are paths:

```
sage: graphs.Caterpillar([]).is_isomorphic(graphs.PathGraph(2))
True
sage: graphs.Caterpillar([0]).is_isomorphic(graphs.PathGraph(3))
True
sage: graphs.Caterpillar([0, 0]).is_isomorphic(graphs.PathGraph(4))
True
```

```
>>> from sage.all import *
>>> graphs.Caterpillar([]).is_isomorphic(graphs.PathGraph(Integer(2)))
True
>>> graphs.Caterpillar([Integer(0)]).is_isomorphic(graphs.PathGraph(Integer(3)))
True
>>> graphs.Caterpillar([Integer(0), Integer(0)]).is_isomorphic(graphs.
↪PathGraph(Integer(4)))
True
```

Caterpillars with singleton spine are stars:

```
sage: graphs.Caterpillar([1]).is_isomorphic(graphs.StarGraph(3))
True
sage: graphs.Caterpillar([2]).is_isomorphic(graphs.StarGraph(4))
True
sage: graphs.Caterpillar([3]).is_isomorphic(graphs.StarGraph(5))
True
```

```
>>> from sage.all import *
>>> graphs.Caterpillar([Integer(1)]).is_isomorphic(graphs.StarGraph(Integer(3)))
True
>>> graphs.Caterpillar([Integer(2)]).is_isomorphic(graphs.StarGraph(Integer(4)))
True
>>> graphs.Caterpillar([Integer(3)]).is_isomorphic(graphs.StarGraph(Integer(5)))
True
```

Distinct spine sequences can yield isomorphic caterpillars:

```
sage: graphs.Caterpillar([1,1,2]).is_isomorphic(graphs.Caterpillar([2,1,1]))
True
```

```
>>> from sage.all import *
>>> graphs.Caterpillar([Integer(1),Integer(1),Integer(2)]).is_isomorphic(graphs.
↪Caterpillar([Integer(2),Integer(1),Integer(1)]))
True
```

sage.graphs.generators.trees.**FibonacciTree**(*n*, *immutable=False*)

Return the graph of the Fibonacci Tree F_n.

The Fibonacci tree F_n is recursively defined as the tree with a root vertex and two attached child trees F_{n-1} and F_{n-2}, where F_1 is just one vertex and F_0 is empty.

INPUT:

- `n` – the recursion depth of the Fibonacci Tree
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.FibonacciTree(3)                                              #␣
↪needs sage.libs.pari
sage: g.is_tree()                                                              #␣
↪needs sage.libs.pari
True
```

```
>>> from sage.all import *
>>> g = graphs.FibonacciTree(Integer(3))                                        ␣
↪     # needs sage.libs.pari
>>> g.is_tree()                                                               #␣
↪needs sage.libs.pari
True
```

```
sage: l1 = [graphs.FibonacciTree(_).order() + 1 for _ in range(6)]             #␣
↪needs sage.libs.pari
sage: l2 = list(fibonacci_sequence(2,8))                                       #␣
↪needs sage.libs.pari
sage: l1 == l2                                                                 #␣
↪needs sage.libs.pari
True
```

```
>>> from sage.all import *
>>> l1 = [graphs.FibonacciTree(_).order() + Integer(1) for _ in␣
↪range(Integer(6))]              # needs sage.libs.pari
>>> l2 = list(fibonacci_sequence(Integer(2),Integer(8)))                    ␣
↪              # needs sage.libs.pari
>>> l1 == l2                                                               #␣
↪needs sage.libs.pari
True
```

AUTHORS:

- Harald Schilly and Yann Laigle-Chapuy (2010-03-25)

sage.graphs.generators.trees.**RandomLobster**(*n*, *p*, *q*, *seed=None*, *immutable=False*)

Return a random lobster.

A lobster is a tree that reduces to a caterpillar when pruning all leaf vertices. A caterpillar is a tree that reduces to a path when pruning all leaf vertices ($q = 0$).

INPUT:

- `n` – expected number of vertices in the backbone
- `p` – probability of adding an edge to the backbone
- `q` – probability of adding an edge (claw) to the arms
- `seed` – a `random.Random` seed or a Python `int` for the random number generator (default: `None`)
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

We check a random graph with 12 backbone nodes and probabilities $p = 0.7$ and $q = 0.3$:

```
sage: # needs networkx
sage: G = graphs.RandomLobster(12, 0.7, 0.3)
sage: leaves = [v for v in G.vertices(sort=False) if G.degree(v) == 1]
sage: G.delete_vertices(leaves)                                  # caterpillar
sage: leaves = [v for v in G.vertices(sort=False) if G.degree(v) == 1]
sage: G.delete_vertices(leaves)                                  # path
sage: s = G.degree_sequence()
sage: if G:
....:     if G.n_vertices() == 1:
....:         assert s == [0]
....:     else:
....:         assert s[-2:] == [1, 1]
....:     assert all(d == 2 for d in s[:-2])
```

```
>>> from sage.all import *
>>> # needs networkx
>>> G = graphs.RandomLobster(Integer(12), RealNumber('0.7'), RealNumber('0.3'))
>>> leaves = [v for v in G.vertices(sort=False) if G.degree(v) == Integer(1)]
>>> G.delete_vertices(leaves)                                  # caterpillar
>>> leaves = [v for v in G.vertices(sort=False) if G.degree(v) == Integer(1)]
>>> G.delete_vertices(leaves)                                  # path
>>> s = G.degree_sequence()
>>> if G:
```

(continues on next page)

(continued from previous page)

```
...     if G.n_vertices() == Integer(1):
...         assert s == [Integer(0)]
...     else:
...         assert s[-Integer(2):] == [Integer(1), Integer(1)]
...     assert all(d == Integer(2) for d in s[:-Integer(2)])
```

```
sage: G = graphs.RandomLobster(9, .6, .3)                                   #␣
↪needs networkx
sage: G.show()                              # long time                     #␣
↪needs networkx sage.plot
```

```
>>> from sage.all import *
>>> G = graphs.RandomLobster(Integer(9), RealNumber('.6'), RealNumber('.3'))     ␣
↪                                  # needs networkx
>>> G.show()                                # long time                    #␣
↪needs networkx sage.plot
```

sage.graphs.generators.trees.**RandomTree**(*n*, *seed=None*, *immutable=False*)

Return a random tree on n nodes numbered 0 through $n-1$.

By Cayley's theorem, there are n^{n-2} trees with vertex set $\{0, 1, \ldots, n-1\}$. This constructor chooses one of these uniformly at random.

ALGORITHM:

The algorithm works by generating an $(n-2)$-long random sequence of numbers chosen independently and uniformly from $\{0, 1, \ldots, n-1\}$ and then applies an inverse Prufer transformation.

INPUT:

- `n` – number of vertices in the tree
- `seed` – a `random.Random` seed or a Python `int` for the random number generator (default: `None`)
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: G = graphs.RandomTree(10)
sage: G.is_tree()
True
sage: G.show()                              # long time                     #␣
↪needs sage.plot
```

```
>>> from sage.all import *
>>> G = graphs.RandomTree(Integer(10))
>>> G.is_tree()
True
>>> G.show()                                # long time                    #␣
↪needs sage.plot
```

sage.graphs.generators.trees.**RandomTreePowerlaw**(*n*, *gamma=3*, *tries=1000*, *seed=None*, *immutable=False*)

Return a tree with a power law degree distribution, or `False` on failure.

From the NetworkX documentation: a trial power law degree sequence is chosen and then elements are swapped with new elements from a power law distribution until the sequence makes a tree (size = order - 1).

INPUT:

- `n` – number of vertices
- `gamma` – exponent of power law distribution
- `tries` – number of attempts to adjust sequence to make a tree
- `seed` – a `random.Random` seed or a Python `int` for the random number generator (default: `None`)
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

We check that the generated graph is a tree:

```
sage: G = graphs.RandomTreePowerlaw(10, 3)                                  #␣
↪needs networkx
sage: G.is_tree()                                                           #␣
↪needs networkx
True
sage: G.order(), G.size()                                                   #␣
↪needs networkx
(10, 9)
```

```
>>> from sage.all import *
>>> G = graphs.RandomTreePowerlaw(Integer(10), Integer(3))                  ␣
↪              # needs networkx
>>> G.is_tree()                                                           #␣
↪needs networkx
True
>>> G.order(), G.size()                                                   #␣
↪needs networkx
(10, 9)
```

```
sage: G = graphs.RandomTreePowerlaw(15, 2)                                  #␣
↪needs networkx
sage: if G:                                 # random output          # long time,␣
↪needs networkx sage.plot
....:     G.show()
```

```
>>> from sage.all import *
>>> G = graphs.RandomTreePowerlaw(Integer(15), Integer(2))                  ␣
↪              # needs networkx
>>> if G:                                 # random output          # long time, needs␣
↪networkx sage.plot
...     G.show()
```

class sage.graphs.generators.trees.**TreeIterator**

Bases: `object`

This class iterates over all trees with n vertices (up to isomorphism).

INPUT:

- `n` – integer; the size of the trees created

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: from sage.graphs.generators.trees import TreeIterator
sage: def check_trees(n):
....:     trees = []
....:     for t in TreeIterator(n):
....:         if not t.is_tree():
....:             return False
....:         if t.n_vertices() != n:
....:             return False
....:         if t.n_edges() != n - 1:
....:             return False
....:         for tree in trees:
....:             if tree.is_isomorphic(t):
....:                 return False
....:         trees.append(t)
....:     return True
sage: check_trees(10)
True
```

```
>>> from sage.all import *
>>> from sage.graphs.generators.trees import TreeIterator
>>> def check_trees(n):
...     trees = []
...     for t in TreeIterator(n):
...         if not t.is_tree():
...             return False
...         if t.n_vertices() != n:
...             return False
...         if t.n_edges() != n - Integer(1):
...             return False
...         for tree in trees:
...             if tree.is_isomorphic(t):
...                 return False
...         trees.append(t)
...     return True
>>> check_trees(Integer(10))
True
```

```
sage: from sage.graphs.generators.trees import TreeIterator
sage: count = 0
sage: for t in TreeIterator(15):
....:     count += 1
sage: count
7741
```

```
>>> from sage.all import *
>>> from sage.graphs.generators.trees import TreeIterator
>>> count = Integer(0)
>>> for t in TreeIterator(Integer(15)):
...     count += Integer(1)
```

(continues on next page)

(continued from previous page)

```
>>> count
7741
```

sage.graphs.generators.trees.**nauty_gentreeg**(*options=''*, *debug=False*, *immutable=False*)

Return a generator which creates non-isomorphic trees from nauty's gentreeg program.

INPUT:

- `options` – string (default: `""`); a string passed to `gentreeg` as if it was run at a system command line. At a minimum, you *must* pass the number of vertices you desire. Sage expects the graphs to be in nauty's "sparse6" format, do not set an option to change this default or results will be unpredictable.
- `debug` – boolean (default: `False`); if `True` the first line of `gentreeg`'s output to standard error is captured and the first call to the generator's `next()` function will return this line as a string. A line leading with ">A" indicates a successful initiation of the program with some information on the arguments, while a line beginning with ">E" indicates an error with the input.
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

The possible options, obtained as output of `gentreeg -help`:

```
   n            : the number of vertices. Must be in range 1..128
res/mod         : only generate subset res out of subsets 0..mod-1
  -D<int>       : an upper bound for the maximum degree
  -Z<int>:<int> : bounds on the diameter
  -q            : suppress auxiliary output
```

Options which cause `gentreeg` to use an output format different than the sparse6 format are not listed above (-p, -l, -u) as they will confuse the creation of a Sage graph. The res/mod option can be useful when using the output in a routine run several times in parallel.

OUTPUT:

A generator which will produce the graphs as Sage graphs. These will be simple graphs: no loops, no multiple edges, no directed edges.

See also

trees() – another generator of trees

EXAMPLES:

The generator can be used to construct trees for testing, one at a time (usually inside a loop). Or it can be used to create an entire list all at once if there is sufficient memory to contain it:

```
sage: gen = graphs.nauty_gentreeg("4")
sage: next(gen)
Graph on 4 vertices
sage: next(gen)
Graph on 4 vertices
sage: next(gen)
Traceback (most recent call last):
...
StopIteration
```

```
>>> from sage.all import *
>>> gen = graphs.nauty_gentreeg("4")
>>> next(gen)
Graph on 4 vertices
>>> next(gen)
Graph on 4 vertices
>>> next(gen)
Traceback (most recent call last):
...
StopIteration
```

The number of trees on the first few vertex counts. This agrees with OEIS sequence A000055:

```
sage: [len(list(graphs.nauty_gentreeg(str(i)))) for i in range(1, 15)]
[1, 1, 1, 2, 3, 6, 11, 23, 47, 106, 235, 551, 1301, 3159]
```

```
>>> from sage.all import *
>>> [len(list(graphs.nauty_gentreeg(str(i)))) for i in range(Integer(1),␣
↪Integer(15))]
[1, 1, 1, 2, 3, 6, 11, 23, 47, 106, 235, 551, 1301, 3159]
```

The `debug` switch can be used to examine `gentreeg`'s reaction to the input in the `options` string. We illustrate success. (A failure will be a string beginning with ">E".) Passing the "-q" switch to `gentreeg` will suppress the indicator of a successful initiation, and so the first returned value might be an empty string if `debug` is `True`:

```
sage: gen = graphs.nauty_gentreeg("4", debug=True)
sage: print(next(gen))
>A ...gentreeg ...
sage: gen = graphs.nauty_gentreeg("4 -q", debug=True)
sage: next(gen)
''
```

```
>>> from sage.all import *
>>> gen = graphs.nauty_gentreeg("4", debug=True)
>>> print(next(gen))
>A ...gentreeg ...
>>> gen = graphs.nauty_gentreeg("4 -q", debug=True)
>>> next(gen)
''
```

`sage.graphs.generators.trees.`**`trees`**(*n*, *immutable=False*)

Return a generator of the distinct trees on a fixed number of vertices.

INPUT:

- `n` – integer; the size of the trees created
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

OUTPUT:

A generator which creates an exhaustive, duplicate-free listing of the connected free (unlabeled) trees with `n` vertices. A tree is a graph with no cycles.

ALGORITHM:

Uses the algorithm that generates each new tree in constant time described in [WROM1986]. See the documentation for, and implementation of, the *sage.graphs.generators.trees* module.

EXAMPLES:

We create an iterator, then loop over its elements:

```
sage: tree_iterator = graphs.trees(7)
sage: for T in tree_iterator:
....:     print(T.degree_sequence())
[2, 2, 2, 2, 2, 1, 1]
[3, 2, 2, 2, 1, 1, 1]
[3, 2, 2, 2, 1, 1, 1]
[4, 2, 2, 1, 1, 1, 1]
[3, 3, 2, 1, 1, 1, 1]
[3, 3, 2, 1, 1, 1, 1]
[4, 3, 1, 1, 1, 1, 1]
[3, 2, 2, 2, 1, 1, 1]
[4, 2, 2, 1, 1, 1, 1]
[5, 2, 1, 1, 1, 1, 1]
[6, 1, 1, 1, 1, 1, 1]
```

```
>>> from sage.all import *
>>> tree_iterator = graphs.trees(Integer(7))
>>> for T in tree_iterator:
...     print(T.degree_sequence())
[2, 2, 2, 2, 2, 1, 1]
[3, 2, 2, 2, 1, 1, 1]
[3, 2, 2, 2, 1, 1, 1]
[4, 2, 2, 1, 1, 1, 1]
[3, 3, 2, 1, 1, 1, 1]
[3, 3, 2, 1, 1, 1, 1]
[4, 3, 1, 1, 1, 1, 1]
[3, 2, 2, 2, 1, 1, 1]
[4, 2, 2, 1, 1, 1, 1]
[5, 2, 1, 1, 1, 1, 1]
[6, 1, 1, 1, 1, 1, 1]
```

The number of trees on the first few vertex counts. This is sequence A000055 in Sloane's OEIS:

```
sage: [len(list(graphs.trees(i))) for i in range(15)]
[1, 1, 1, 1, 2, 3, 6, 11, 23, 47, 106, 235, 551, 1301, 3159]
```

```
>>> from sage.all import *
>>> [len(list(graphs.trees(i))) for i in range(Integer(15))]
[1, 1, 1, 1, 2, 3, 6, 11, 23, 47, 106, 235, 551, 1301, 3159]
```

2.18 Graphs from the World Map

The methods defined here appear in *sage.graphs.graph_generators*.

sage.graphs.generators.world_map.**AfricaMap**(*continental=False*, *year=2018*, *immutable=False*)

Return African states as a graph of common border.

"African state" here is defined as an independent state having the capital city in Africa. The graph has an edge between those countries that have common *land* border.

INPUT:

- `continental` – boolean (default: `False`); whether to only return states in the continental Africa or all African states
- `year` – integer (default: 2018); reserved for future use
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: Africa = graphs.AfricaMap(); Africa
Africa Map: Graph on 54 vertices
sage: sorted(Africa.neighbors('Libya'))
['Algeria', 'Chad', 'Egypt', 'Niger', 'Sudan', 'Tunisia']

sage: cont_Africa = graphs.AfricaMap(continental=True)
sage: cont_Africa.order()
48
sage: 'Madagaskar' in cont_Africa
False
sage: cont_Africa.is_connected()
True
```

```
>>> from sage.all import *
>>> Africa = graphs.AfricaMap(); Africa
Africa Map: Graph on 54 vertices
>>> sorted(Africa.neighbors('Libya'))
['Algeria', 'Chad', 'Egypt', 'Niger', 'Sudan', 'Tunisia']

>>> cont_Africa = graphs.AfricaMap(continental=True)
>>> cont_Africa.order()
48
>>> 'Madagaskar' in cont_Africa
False
>>> cont_Africa.is_connected()
True
```

sage.graphs.generators.world_map.**EuropeMap**(*continental=False*, *year=2018*, *immutable=False*)

Return European states as a graph of common border.

"European state" here is defined as an independent state having the capital city in Europe. The graph has an edge between those countries that have common *land* border.

INPUT:

- `continental` – boolean (default: `False`); whether to only return states in the continental Europe or all European states
- `year` – integer (default: 2018); reserved for future use
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: Europe = graphs.EuropeMap(); Europe
Europe Map: Graph on 44 vertices
sage: Europe.neighbors('Ireland')
['United Kingdom']

sage: cont_Europe = graphs.EuropeMap(continental=True)
sage: cont_Europe.order()
40
sage: 'Iceland' in cont_Europe
False
sage: cont_Europe.is_connected()
True
sage: cont_Europe.is_immutable()
False
sage: cont_Europe = graphs.EuropeMap(continental=True, immutable=True)
sage: cont_Europe.is_immutable()
True
```

```
>>> from sage.all import *
>>> Europe = graphs.EuropeMap(); Europe
Europe Map: Graph on 44 vertices
>>> Europe.neighbors('Ireland')
['United Kingdom']

>>> cont_Europe = graphs.EuropeMap(continental=True)
>>> cont_Europe.order()
40
>>> 'Iceland' in cont_Europe
False
>>> cont_Europe.is_connected()
True
>>> cont_Europe.is_immutable()
False
>>> cont_Europe = graphs.EuropeMap(continental=True, immutable=True)
>>> cont_Europe.is_immutable()
True
```

sage.graphs.generators.world_map.**USAMap**(*continental=False*, *immutable=False*)

Return states of USA as a graph of common border.

The graph has an edge between those states that have common *land* border line or point. Hence for example Colorado and Arizona are marked as neighbors, but Michigan and Minnesota are not.

INPUT:

- `continental` – boolean (default: `False`); whether to exclude Alaska and Hawaii
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

How many states are neighbor's neighbor for Pennsylvania:

```
sage: USA = graphs.USAMap()
sage: distance = USA.shortest_path_lengths('Pennsylvania')
```

(continues on next page)

(continued from previous page)

```
sage: len([n2 for n2, d in distance.items() if d == 2])
7
```

```
>>> from sage.all import *
>>> USA = graphs.USAMap()
>>> distance = USA.shortest_path_lengths('Pennsylvania')
>>> len([n2 for n2, d in distance.items() if d == Integer(2)])
7
```

Diameter for continental USA:

```
sage: USAcont = graphs.USAMap(continental=True)
sage: USAcont.diameter()
11
```

```
>>> from sage.all import *
>>> USAcont = graphs.USAMap(continental=True)
>>> USAcont.diameter()
11
```

sage.graphs.generators.world_map.**WorldMap**(*immutable=False*)

Return the Graph of all the countries, in which two countries are adjacent in the graph if they have a common boundary.

This graph has been built from the data available in The CIA World Factbook [CIA] (2009-08-21).

The returned graph `G` has a member `G.gps_coordinates` equal to a dictionary containing the GPS coordinates of each country's capital city.

INPUT:

- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

```
sage: g = graphs.WorldMap()
sage: g.has_edge("France", "Italy")
True
sage: g.gps_coordinates["Bolivia"]
[[17, 'S'], [65, 'W']]
sage: g.connected_component_containing_vertex('Ireland', sort=True)
['Ireland', 'United Kingdom']
```

```
>>> from sage.all import *
>>> g = graphs.WorldMap()
>>> g.has_edge("France", "Italy")
True
>>> g.gps_coordinates["Bolivia"]
[[17, 'S'], [65, 'W']]
>>> g.connected_component_containing_vertex('Ireland', sort=True)
['Ireland', 'United Kingdom']
```

2.19 ISGCI: Information System on Graph Classes and their Inclusions

This module implements an interface to the ISGCI database in Sage.

This database gathers information on graph classes and their inclusions in each other. It also contains information on the complexity of several computational problems.

It is available on the GraphClasses.org website maintained by H.N. de Ridder et al.

2.19.1 How to use it?

Presently, it is possible to use this database through the variables and methods present in the *graph_classes* object. For instance:

```
sage: Trees = graph_classes.Tree
sage: Chordal = graph_classes.Chordal
```

```
>>> from sage.all import *
>>> Trees = graph_classes.Tree
>>> Chordal = graph_classes.Chordal
```

Inclusions

It is then possible to check the inclusion of classes inside of others, if the information is available in the database:

```
sage: Trees <= Chordal
True
```

```
>>> from sage.all import *
>>> Trees <= Chordal
True
```

And indeed, trees are chordal graphs.

The ISGCI database is not all-knowing, and so comparing two classes can return `True`, `False`, or the `UnknownClass` instance `Unknown`.

An *unknown* answer to `A <= B` only means that ISGCI cannot deduce from the information in its database that `A` is a subclass of `B` nor that it is not. For instance, ISGCI does not know at the moment that some chordal graphs are not trees:

```
sage: graph_classes.Chordal <= graph_classes.Tree
Unknown
```

```
>>> from sage.all import *
>>> graph_classes.Chordal <= graph_classes.Tree
Unknown
```

Descriptions

Given a graph class, one can obtain its associated information in the ISGCI database with the *description()* method:

```
sage: Chordal.description()
Class of graphs : Chordal
--------------------------
id                            :  gc_32
name                          :  chordal
...
Problems :
-----------
3-Colourability               :  Linear
Clique                        :  Polynomial
Clique cover                  :  Polynomial
...
```

```
>>> from sage.all import *
>>> Chordal.description()
Class of graphs : Chordal
--------------------------
id                            :  gc_32
name                          :  chordal
...
Problems :
-----------
3-Colourability               :  Linear
Clique                        :  Polynomial
Clique cover                  :  Polynomial
...
```

It is possible to obtain the complete list of the classes stored in ISGCI by calling the *show_all()* method (beware – long output):

```
sage: graph_classes.show_all()
id        | name                                    | type                 |␣
↪smallgraph
-------------------------------------------------------------------------------------
↪---------------------------------
gc_309    | $K_4$--minor--free                      | base                 |
gc_541    | $N^*$                                   | base                 |
gc_215    | $N^*$--perfect                          | base                 |
gc_5      | $P_4$--bipartite                        | base                 |
gc_3      | $P_4$--brittle                          | base                 |
gc_6      | $P_4$--comparability                    | base                 |
gc_7      | $P_4$--extendible                       | base                 |
...
```

```
>>> from sage.all import *
>>> graph_classes.show_all()
id        | name                                    | type                 |␣
↪smallgraph
-------------------------------------------------------------------------------------
↪---------------------------------
gc_309    | $K_4$--minor--free                      | base                 |
gc_541    | $N^*$                                   | base                 |
```

(continues on next page)

(continued from previous page)

```
gc_215     | $N^*$--perfect                          | base                |
gc_5       | $P_4$--bipartite                        | base                |
gc_3       | $P_4$--brittle                          | base                |
gc_6       | $P_4$--comparability                    | base                |
gc_7       | $P_4$--extendible                       | base                |
...
```

Until a proper search method is implemented, this lets one find classes which do not appear in *graph_classes.**.

To retrieve a class of graph from its ISGCI ID one may use the *get_class()* method:

```
sage: GC = graph_classes.get_class("gc_5")
sage: GC
$P_4$--bipartite graphs
```

```
>>> from sage.all import *
>>> GC = graph_classes.get_class("gc_5")
>>> GC
$P_4$--bipartite graphs
```

Recognition of graphs

The graph classes represented by the ISGCI database can alternatively be used to access recognition algorithms. For instance, in order to check that a given graph is a tree one has the following the options

```
sage: graphs.PathGraph(5) in graph_classes.Tree
True
```

```
>>> from sage.all import *
>>> graphs.PathGraph(Integer(5)) in graph_classes.Tree
True
```

or:

```
sage: graphs.PathGraph(5).is_tree()
True
```

```
>>> from sage.all import *
>>> graphs.PathGraph(Integer(5)).is_tree()
True
```

Furthermore, all ISGCI graph classes which are defined by the exclusion of a finite sequence of induced subgraphs benefit from a generic recognition algorithm. For instance

```
sage: g = graphs.PetersenGraph()
sage: g in graph_classes.ClawFree
False
sage: g.line_graph() in graph_classes.ClawFree
True
```

```
>>> from sage.all import *
>>> g = graphs.PetersenGraph()
```

(continues on next page)

(continued from previous page)

```
>>> g in graph_classes.ClawFree
False
>>> g.line_graph() in graph_classes.ClawFree
True
```

Or directly from ISGCI

```
sage: gc = graph_classes.get_class("gc_441")
sage: gc
diamond--free graphs
sage: graphs.PetersenGraph() in gc
True
```

```
>>> from sage.all import *
>>> gc = graph_classes.get_class("gc_441")
>>> gc
diamond--free graphs
>>> graphs.PetersenGraph() in gc
True
```

2.19.2 Predefined classes

graph_classes currently predefines the following graph classes

Class	Related methods
Apex	*is_apex()*, *apex_vertices()*
AT_free	*is_asteroidal_triple_free()*
Biconnected	*is_biconnected()*, *blocks_and_cut_vertices()*, *blocks_and_cuts_tree()*
BinaryTrees	*BalancedTree()*, *is_tree()*
Bipartite	*BalancedTree()*, *is_bipartite()*
Block	*is_block_graph()*, *blocks_and_cut_vertices()*, *RandomBlockGraph()*
Chordal	*is_chordal()*
Claw-Free	*ClawGraph()*
Comparability	
Gallai	*is_gallai_tree()*
Grid	*Grid2dGraph()*, *GridGraph()*
Interval	*RandomIntervalGraph()*, *IntervalGraph()*, *is_interval()*
Line	*line_graph_forbidden_subgraphs()*, *is_line_graph()*
Modular	*modular_decomposition()*
Outerplanar	*is_circular_planar()*
Perfect	*is_perfect()*
Planar	*is_planar()*
Polyhedral	*is_polyhedral()*
Split	*is_split()*
Tree	*trees()*, *is_tree()*
UnitDisk	*IntervalGraph()*
UnitInterval	*is_interval()*

2.19.3 Sage's view of ISGCI

The database is stored by Sage in two ways.

The classes: the list of all graph classes and their properties is stored in a huge dictionary (see *classes()*). Below is what Sage knows of gc_249:

```
sage: graph_classes.classes()['gc_249']        # random
{'problem':
    {'Independent set': 'Polynomial',
     'Treewidth': 'Unknown',
     'Weighted independent set': 'Polynomial',
     'Cliquewidth expression': 'NP-complete',
     'Weighted clique': 'Polynomial',
     'Clique cover': 'Unknown',
     'Domination': 'NP-complete',
     'Clique': 'Polynomial',
     'Colourability': 'NP-complete',
     'Cliquewidth': 'Unbounded',
     '3-Colourability': 'NP-complete',
     'Recognition': 'Linear'},
 'type': 'base',
 'id': 'gc_249',
 'name': 'line'}
```

```
>>> from sage.all import *
>>> graph_classes.classes()['gc_249']        # random
{'problem':
    {'Independent set': 'Polynomial',
     'Treewidth': 'Unknown',
     'Weighted independent set': 'Polynomial',
     'Cliquewidth expression': 'NP-complete',
     'Weighted clique': 'Polynomial',
     'Clique cover': 'Unknown',
     'Domination': 'NP-complete',
     'Clique': 'Polynomial',
     'Colourability': 'NP-complete',
     'Cliquewidth': 'Unbounded',
     '3-Colourability': 'NP-complete',
     'Recognition': 'Linear'},
 'type': 'base',
 'id': 'gc_249',
 'name': 'line'}
```

The class inclusion digraph: Sage remembers the class inclusions through the inclusion digraph (see *inclusion_digraph()*). Its nodes are ID of ISGCI classes:

```
sage: d = graph_classes.inclusion_digraph()
sage: d.vertices(sort=True)[-10:]
['gc_990', 'gc_991', 'gc_992', 'gc_993', 'gc_994', 'gc_995', 'gc_996', 'gc_997', 'gc_
↪998', 'gc_999']
```

```
>>> from sage.all import *
>>> d = graph_classes.inclusion_digraph()
```

(continues on next page)

(continued from previous page)

```
>>> d.vertices(sort=True)[-Integer(10):]
['gc_990', 'gc_991', 'gc_992', 'gc_993', 'gc_994', 'gc_995', 'gc_996', 'gc_997', 'gc_
↪998', 'gc_999']
```

An arc from `gc1` to `gc2` means that `gc1` is a superclass of `gc2`. This being said, not all edges are stored ! To ensure that a given class is included in another one, we have to check whether there is in the digraph a `path` from the first one to the other:

```
sage: bip_id = graph_classes.Bipartite._gc_id
sage: perfect_id = graph_classes.Perfect._gc_id
sage: d.has_edge(perfect_id, bip_id)
False
sage: d.distance(perfect_id, bip_id)
2
```

```
>>> from sage.all import *
>>> bip_id = graph_classes.Bipartite._gc_id
>>> perfect_id = graph_classes.Perfect._gc_id
>>> d.has_edge(perfect_id, bip_id)
False
>>> d.distance(perfect_id, bip_id)
2
```

Hence bipartite graphs are perfect graphs. We can see how ISGCI obtains this result

```
sage: p = d.shortest_path(perfect_id, bip_id)
sage: len(p) - 1
2
sage: print(p)                    # random
['gc_56', 'gc_76', 'gc_69']
sage: for c in p:
....:     print(graph_classes.get_class(c))
perfect graphs
...
bipartite graphs
```

```
>>> from sage.all import *
>>> p = d.shortest_path(perfect_id, bip_id)
>>> len(p) - Integer(1)
2
>>> print(p)                    # random
['gc_56', 'gc_76', 'gc_69']
>>> for c in p:
...     print(graph_classes.get_class(c))
perfect graphs
...
bipartite graphs
```

What ISGCI knows is that perfect graphs contain unimodular graph which contain bipartite graphs. Therefore bipartite graphs are perfect !

Note

The inclusion digraph is **NOT ACYCLIC**. Indeed, several entries exist in the ISGCI database which represent the same graph class, for instance Perfect graphs and Berge graphs:

```
sage: graph_classes.inclusion_digraph().is_directed_acyclic()
False
sage: Berge = graph_classes.get_class("gc_274"); Berge
Berge graphs
sage: Perfect = graph_classes.get_class("gc_56"); Perfect
perfect graphs
sage: Berge <= Perfect
True
sage: Perfect <= Berge
True
sage: Perfect == Berge
True
```

```
>>> from sage.all import *
>>> graph_classes.inclusion_digraph().is_directed_acyclic()
False
>>> Berge = graph_classes.get_class("gc_274"); Berge
Berge graphs
>>> Perfect = graph_classes.get_class("gc_56"); Perfect
perfect graphs
>>> Berge <= Perfect
True
>>> Perfect <= Berge
True
>>> Perfect == Berge
True
```

2.19.4 Information for developers

- The database is loaded not *so* large, but it is still preferable to only load it on demand. This is achieved through the cached methods *classes()* and *inclusion_digraph()*.
- Upon the first access to the database, the information is extracted from the XML file and stored in the cache of three methods:
  - `sage.graphs.isgci._classes` (dictionary)
  - `sage.graphs.isgci._inclusions` (list of dictionaries)
  - `sage.graphs.isgci._inclusion_digraph` (DiGraph)

  Note that the digraph is only built if necessary (for instance if the user tries to compare two classes).

Todo

Technical things:

- Query the database for non-inclusion results so that comparisons can return `False`, and implement strict inclusions.
- Implement a proper search method for the classes not listed in *graph_classes*

See also

show_all().

- Some of the graph classes appearing in *graph_classes* already have a recognition algorithm implemented in Sage. It would be so nice to be able to write `g in Trees`, `g in Perfect`, `g in Chordal`, ... :-)

Long-term stuff:

- Implement simple accessors for all the information in the ISGCI database (as can be done from the website)
- Implement intersection of graph classes
- Write generic recognition algorithms for specific classes (when a graph class is defined by the exclusion of subgraphs, one can write a generic algorithm checking the existence of each of the graphs, and this method already exists in Sage).
- Improve the performance of Sage's graph library by letting it take advantage of the properties of graph classes. For example, *Graph.independent_set()* could use the library to detect that a given graph is, say, a tree or a planar graph, and use a specialized algorithm for finding an independent set.

2.19.5 AUTHORS:

- H.N. de Ridder et al. (ISGCI database)
- Nathann Cohen (Sage implementation)

2.19.6 Methods

class sage.graphs.isgci.**GraphClass**(*name*, *gc_id*, *recognition_function=None*)

Bases: SageObject, CachedRepresentation

An instance of this class represents a Graph Class, matching some entry in the ISGCI database.

EXAMPLES:

Testing the inclusion of two classes:

```
sage: Chordal = graph_classes.Chordal
sage: Trees = graph_classes.Tree
sage: Trees <= Chordal
True
sage: Chordal <= Trees
Unknown
```

```
>>> from sage.all import *
>>> Chordal = graph_classes.Chordal
>>> Trees = graph_classes.Tree
>>> Trees <= Chordal
True
>>> Chordal <= Trees
Unknown
```

description()

Print the information of ISGCI about the current class.

EXAMPLES:

```
sage: graph_classes.Chordal.description()
Class of graphs : Chordal
--------------------------
id                            :  gc_32
name                          :  chordal
...
Problems :
-----------
3-Colourability               :  Linear
Clique                        :  Polynomial
Clique cover                  :  Polynomial
...
Recognition                   :  Linear
...
```

```
>>> from sage.all import *
>>> graph_classes.Chordal.description()
Class of graphs : Chordal
--------------------------
id                            :  gc_32
name                          :  chordal
...
Problems :
-----------
3-Colourability               :  Linear
Clique                        :  Polynomial
Clique cover                  :  Polynomial
...
Recognition                   :  Linear
...
```

forbidden_subgraphs()

Return the list of forbidden induced subgraphs defining the class.

If the graph class is not defined by a *finite* list of forbidden induced subgraphs, `None` is returned instead.

EXAMPLES:

```
sage: graph_classes.Perfect.forbidden_subgraphs()
sage: gc = graph_classes.get_class('gc_62')
sage: gc
claw--free graphs
sage: gc.forbidden_subgraphs()
[Graph on 4 vertices]
sage: gc.forbidden_subgraphs()[0].is_isomorphic(graphs.ClawGraph())
True
```

```
>>> from sage.all import *
>>> graph_classes.Perfect.forbidden_subgraphs()
>>> gc = graph_classes.get_class('gc_62')
>>> gc
claw--free graphs
>>> gc.forbidden_subgraphs()
```

(continues on next page)

(continued from previous page)

```
[Graph on 4 vertices]
>>> gc.forbidden_subgraphs()[Integer(0)].is_isomorphic(graphs.ClawGraph())
True
```

class sage.graphs.isgci.**GraphClasses**

Bases: UniqueRepresentation

classes()

Return the graph classes, as a dictionary.

Upon the first call, this loads the database from the local XML file. Subsequent calls are cached.

EXAMPLES:

```
sage: t = graph_classes.classes()
sage: type(t)
<... 'dict'>
sage: sorted(t["gc_151"].keys())
['id', 'name',... 'problem',... 'type']
sage: t["gc_151"]['name']
'cograph'
sage: t["gc_151"]['problem']['Clique']
{'complexity': 'Linear'}
```

```
>>> from sage.all import *
>>> t = graph_classes.classes()
>>> type(t)
<... 'dict'>
>>> sorted(t["gc_151"].keys())
['id', 'name',... 'problem',... 'type']
>>> t["gc_151"]['name']
'cograph'
>>> t["gc_151"]['problem']['Clique']
{'complexity': 'Linear'}
```

get_class(*id*)

Return the class corresponding to the given id in the ISGCI database.

INPUT:

- id – string; the desired class' ID

See also

show_all()

EXAMPLES:

With an existing id:

```
sage: Cographs = graph_classes.get_class("gc_151")
sage: Cographs
cograph graphs
```

```
>>> from sage.all import *
>>> Cographs = graph_classes.get_class("gc_151")
>>> Cographs
cograph graphs
```

With a wrong id:

```
sage: graph_classes.get_class(-1)
Traceback (most recent call last):
...
ValueError: The given class id does not exist in the ISGCI database. Is the␣
↪db too old ? You can update it with graph_classes.update_db().
```

```
>>> from sage.all import *
>>> graph_classes.get_class(-Integer(1))
Traceback (most recent call last):
...
ValueError: The given class id does not exist in the ISGCI database. Is the␣
↪db too old ? You can update it with graph_classes.update_db().
```

inclusion_digraph()

Return the class inclusion digraph.

Upon the first call, this loads the database from the local XML file. Subsequent calls are cached.

EXAMPLES:

```
sage: g = graph_classes.inclusion_digraph(); g
Digraph on ... vertices
```

```
>>> from sage.all import *
>>> g = graph_classes.inclusion_digraph(); g
Digraph on ... vertices
```

inclusions()

Return the graph class inclusions.

OUTPUT: list of dictionaries

Upon the first call, this loads the database from the local XML file. Subsequent calls are cached.

EXAMPLES:

```
sage: t = graph_classes.inclusions()
sage: type(t)
<... 'list'>
sage: t[0]
{'sub': 'gc_1', 'super': 'gc_2'}
```

```
>>> from sage.all import *
>>> t = graph_classes.inclusions()
>>> type(t)
<... 'list'>
>>> t[Integer(0)]
{'sub': 'gc_1', 'super': 'gc_2'}
```

show_all()

Print all graph classes stored in ISGCI.

EXAMPLES:

```
sage: graph_classes.show_all()
id        | name                                    | type                 |␣
↪smallgraph
------------------------------------------------------------------------------
↪----------------------------------------
gc_309    | $K_4$-minor-free                        | base                 |
gc_541    | $N^*$                                   | base                 |
gc_215    | $N^*$-perfect                           | base                 |
gc_5      | $P_4$-bipartite                         | base                 |
gc_3      | $P_4$-brittle                           | base                 |
gc_6      | $P_4$-comparability                     | base                 |
gc_7      | $P_4$-extendible                        | base                 |
...
```

```
>>> from sage.all import *
>>> graph_classes.show_all()
id        | name                                    | type                 |␣
↪smallgraph
------------------------------------------------------------------------------
↪----------------------------------------
gc_309    | $K_4$-minor-free                        | base                 |
gc_541    | $N^*$                                   | base                 |
gc_215    | $N^*$-perfect                           | base                 |
gc_5      | $P_4$-bipartite                         | base                 |
gc_3      | $P_4$-brittle                           | base                 |
gc_6      | $P_4$-comparability                     | base                 |
gc_7      | $P_4$-extendible                        | base                 |
...
```

smallgraphs()

Return a dictionary associating a graph to a graph description string.

Upon the first call, this loads the database from the local XML files. Subsequent calls are cached.

EXAMPLES:

```
sage: t = graph_classes.smallgraphs()
sage: t['2C_4']
Graph on 8 vertices
sage: t['2K_3 + e']
Graph on 6 vertices
sage: t['fish']
Graph on 6 vertices
sage: t['bull']
Graph on 5 vertices
```

```
>>> from sage.all import *
>>> t = graph_classes.smallgraphs()
>>> t['2C_4']
```

(continues on next page)

(continued from previous page)

```
Graph on 8 vertices
>>> t['2K_3 + e']
Graph on 6 vertices
>>> t['fish']
Graph on 6 vertices
>>> t['bull']
Graph on 5 vertices
```

update_db()

Update the ISGCI database by downloading the latest version from internet.

This method downloads the ISGCI database from the website GraphClasses.org. It then extracts the zip file and parses its XML content. The XML file is saved in the directory controlled by the `DatabaseGraphs` class (usually, `$HOME/.sage/db`).

EXAMPLES:

```
sage: graph_classes.update_db()  # optional - internet
Database downloaded
```

```
>>> from sage.all import *
>>> graph_classes.update_db()  # optional - internet
Database downloaded
```

CHAPTER

THREE

LOW-LEVEL IMPLEMENTATION

3.1 Overview of (di)graph data structures

This module contains no code, and describes Sage's data structures for graphs and digraphs. They can be used directly at Cython/C level, or through the *Graph* and *DiGraph* classes (except one)

3.1.1 Data structures

Four data structures are natively available for (di)graphs in Sage:

- *sparse_graph* (default) – for sparse (di)graphs, with a $\log(n)$ edge test, and easy enumeration of neighbors. It is the most general-purpose data structure, though it can have a high memory cost in practice.
  - Supports: Addition/removal of edges/vertices, multiple edges, edge labels and loops.
- *dense_graph* – for dense (di)graphs, with a $O(1)$ edge test, and slow enumeration of neighbors.
  - Supports: addition/removal of edges/vertices, and loops.
  - Does not support: multiple edges and edge labels.
- *static_sparse_graph* – for sparse (di)graphs and very intensive computations (at C-level). It is faster than *sparse_graph* in practice and *much* lighter in memory.
  - Supports: multiple edges, edge labels and loops
  - Does not support: addition/removal of edges/vertices.
- *static_dense_graph* – for dense (di)graphs and very intensive computations (at C-level). It is mostly a wrapper over bitsets.
  - Supports: addition/removal of edges/vertices, and loops.
  - Does not support: multiple edges and edge labels.

For more information, see the data structures' respective pages.

3.1.2 The backends

The *Graph* and *DiGraph* objects delegate the storage of vertices and edges to other objects: the *graph backends*:

```
sage: Graph()._backend
<sage.graphs.base.sparse_graph.SparseGraphBackend object at ...>
```

```
>>> from sage.all import *
>>> Graph()._backend
<sage.graphs.base.sparse_graph.SparseGraphBackend object at ...>
```

A (di)graph backend is a simpler (di)graph class having only the most elementary methods (e.g.: add/remove vertices/edges). Its vertices can be arbitrary hashable objects.

The only backend available in Sage is *CGraphBackend*.

3.1.3 CGraph and CGraphBackend

CGraphBackend is the backend of all native data structures that can be used by *Graph* and *DiGraph*. It is extended by:

- *DenseGraphBackend*
- *SparseGraphBackend*
- *StaticSparseBackend*

While a *CGraphBackend* deals with arbitrary (hashable) vertices, it contains a `._cg` attribute of type *CGraph* which only deals with integer vertices.

The *CGraph* data structures available in Sage are:

- *DenseGraph*
- *SparseGraph*
- *StaticSparseCGraph*

See the *c_graph* module for more information.

3.2 Fast compiled graphs

This is a Cython implementation of the base class for sparse and dense graphs in Sage. It is not intended for use on its own. Specific graph types should extend this base class and implement missing functionalities. Whenever possible, specific methods should also be overridden with implementations that suit the graph type under consideration.

For an overview of graph data structures in sage, see *overview*.

3.2.1 Data structure

The class `CGraph` maintains the following variables:

- `cdef int num_verts`
- `cdef int num_arcs`
- `cdef int *in_degrees`
- `cdef int *out_degrees`
- `cdef bitset_t active_vertices`

The bitset `active_vertices` is a list of all available vertices for use, but only the ones which are set are considered to actually be in the graph. The variables `num_verts` and `num_arcs` are self-explanatory. Note that `num_verts` is the number of bits set in `active_vertices`, not the full length of the bitset. The arrays `in_degrees` and `out_degrees` are of the same length as the bitset.

For more information about active vertices, see the documentation for the method *realloc*.

class sage.graphs.base.c_graph.**CGraph**

Bases: `object`

Compiled sparse and dense graphs.

add_arc(*u*, *v*)

Add arc `(u, v)` to the graph.

INPUT:

- `u`, `v` – nonnegative integers; must be in self

EXAMPLES:

On the *CGraph* level, this always produces an error, as there are no vertices:

```
sage: from sage.graphs.base.c_graph import CGraph
sage: G = CGraph()
sage: G.add_arc(0, 1)
Traceback (most recent call last):
...
LookupError: vertex (0) is not a vertex of the graph
```

```
>>> from sage.all import *
>>> from sage.graphs.base.c_graph import CGraph
>>> G = CGraph()
>>> G.add_arc(Integer(0), Integer(1))
Traceback (most recent call last):
...
LookupError: vertex (0) is not a vertex of the graph
```

It works, once there are vertices and `add_arc_unsafe` is implemented:

```
sage: from sage.graphs.base.dense_graph import DenseGraph
sage: G = DenseGraph(5)
sage: G.add_arc(0, 1)
sage: G.add_arc(4, 7)
Traceback (most recent call last):
...
LookupError: vertex (7) is not a vertex of the graph
sage: G.has_arc(1, 0)
False
sage: G.has_arc(0, 1)
True

sage: from sage.graphs.base.sparse_graph import SparseGraph
sage: G = SparseGraph(5)
sage: G.add_arc(0,1)
sage: G.add_arc(4,7)
Traceback (most recent call last):
...
LookupError: vertex (7) is not a vertex of the graph
sage: G.has_arc(1,0)
False
sage: G.has_arc(0,1)
True
```

```
>>> from sage.all import *
>>> from sage.graphs.base.dense_graph import DenseGraph
>>> G = DenseGraph(Integer(5))
```

(continues on next page)

(continued from previous page)

```
>>> G.add_arc(Integer(0), Integer(1))
>>> G.add_arc(Integer(4), Integer(7))
Traceback (most recent call last):
...
LookupError: vertex (7) is not a vertex of the graph
>>> G.has_arc(Integer(1), Integer(0))
False
>>> G.has_arc(Integer(0), Integer(1))
True

>>> from sage.graphs.base.sparse_graph import SparseGraph
>>> G = SparseGraph(Integer(5))
>>> G.add_arc(Integer(0),Integer(1))
>>> G.add_arc(Integer(4),Integer(7))
Traceback (most recent call last):
...
LookupError: vertex (7) is not a vertex of the graph
>>> G.has_arc(Integer(1),Integer(0))
False
>>> G.has_arc(Integer(0),Integer(1))
True
```

add_vertex(*k=-1*)

Add vertex `k` to the graph.

INPUT:

- `k` – nonnegative integer or `-1` (default: `-1`); if $k = -1$, a new vertex is added and the integer used is returned. That is, for $k = -1$, this function will find the first available vertex that is not in `self` and add that vertex to this graph.

OUTPUT:

- `-1` – indicates that no vertex was added because the current allocation is already full or the vertex is out of range
- nonnegative integer – this vertex is now guaranteed to be in the graph

See also

- `add_vertex_unsafe` – add a vertex to a graph. This method is potentially unsafe. You should instead use *add_vertex()*
- `add_vertices` – add a bunch of vertices to a graph

EXAMPLES:

Adding vertices to a sparse graph:

```
sage: from sage.graphs.base.sparse_graph import SparseGraph
sage: G = SparseGraph(3, extra_vertices=3)
sage: G.add_vertex(3)
3
sage: G.add_arc(2, 5)
```

(continues on next page)

(continued from previous page)

```
Traceback (most recent call last):
...
LookupError: vertex (5) is not a vertex of the graph
sage: G.add_arc(1, 3)
sage: G.has_arc(1, 3)
True
sage: G.has_arc(2, 3)
False
```

```
>>> from sage.all import *
>>> from sage.graphs.base.sparse_graph import SparseGraph
>>> G = SparseGraph(Integer(3), extra_vertices=Integer(3))
>>> G.add_vertex(Integer(3))
3
>>> G.add_arc(Integer(2), Integer(5))
Traceback (most recent call last):
...
LookupError: vertex (5) is not a vertex of the graph
>>> G.add_arc(Integer(1), Integer(3))
>>> G.has_arc(Integer(1), Integer(3))
True
>>> G.has_arc(Integer(2), Integer(3))
False
```

Adding vertices to a dense graph:

```
sage: from sage.graphs.base.dense_graph import DenseGraph
sage: G = DenseGraph(3, extra_vertices=3)
sage: G.add_vertex(3)
3
sage: G.add_arc(2,5)
Traceback (most recent call last):
...
LookupError: vertex (5) is not a vertex of the graph
sage: G.add_arc(1, 3)
sage: G.has_arc(1, 3)
True
sage: G.has_arc(2, 3)
False
```

```
>>> from sage.all import *
>>> from sage.graphs.base.dense_graph import DenseGraph
>>> G = DenseGraph(Integer(3), extra_vertices=Integer(3))
>>> G.add_vertex(Integer(3))
3
>>> G.add_arc(Integer(2),Integer(5))
Traceback (most recent call last):
...
LookupError: vertex (5) is not a vertex of the graph
>>> G.add_arc(Integer(1), Integer(3))
>>> G.has_arc(Integer(1), Integer(3))
True
```

(continues on next page)

(continued from previous page)

```
>>> G.has_arc(Integer(2), Integer(3))
False
```

Repeatedly adding a vertex using $k = -1$ will allocate more memory as required:

```
sage: from sage.graphs.base.sparse_graph import SparseGraph
sage: G = SparseGraph(3, extra_vertices=0)
sage: G.verts()
[0, 1, 2]
sage: for i in range(10):
....:     _ = G.add_vertex(-1);
...
sage: G.verts()
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]
```

```
>>> from sage.all import *
>>> from sage.graphs.base.sparse_graph import SparseGraph
>>> G = SparseGraph(Integer(3), extra_vertices=Integer(0))
>>> G.verts()
[0, 1, 2]
>>> for i in range(Integer(10)):
...     _ = G.add_vertex(-Integer(1));
...
>>> G.verts()
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]
```

```
sage: from sage.graphs.base.dense_graph import DenseGraph
sage: G = DenseGraph(3, extra_vertices=0)
sage: G.verts()
[0, 1, 2]
sage: for i in range(12):
....:     _ = G.add_vertex(-1);
...
sage: G.verts()
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14]
```

```
>>> from sage.all import *
>>> from sage.graphs.base.dense_graph import DenseGraph
>>> G = DenseGraph(Integer(3), extra_vertices=Integer(0))
>>> G.verts()
[0, 1, 2]
>>> for i in range(Integer(12)):
...     _ = G.add_vertex(-Integer(1));
...
>>> G.verts()
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14]
```

add_vertices(*verts*)

Add vertices from the iterable `verts`.

INPUT:

- `verts` – an iterable of vertices; value -1 has a special meaning – for each such value an unused vertex

name is found, used to create a new vertex and returned.

OUTPUT:

List of generated labels if there is any -1 in `verts`. `None` otherwise.

See also

- *add_vertex()* – add a vertex to a graph

EXAMPLES:

Adding vertices for sparse graphs:

```
sage: from sage.graphs.base.sparse_graph import SparseGraph
sage: S = SparseGraph(nverts=4, extra_vertices=4)
sage: S.verts()
[0, 1, 2, 3]
sage: S.add_vertices([3, -1, 4, 9])
[5]
sage: S.verts()
[0, 1, 2, 3, 4, 5, 9]
sage: S.realloc(20)
sage: S.verts()
[0, 1, 2, 3, 4, 5, 9]
```

```
>>> from sage.all import *
>>> from sage.graphs.base.sparse_graph import SparseGraph
>>> S = SparseGraph(nverts=Integer(4), extra_vertices=Integer(4))
>>> S.verts()
[0, 1, 2, 3]
>>> S.add_vertices([Integer(3), -Integer(1), Integer(4), Integer(9)])
[5]
>>> S.verts()
[0, 1, 2, 3, 4, 5, 9]
>>> S.realloc(Integer(20))
>>> S.verts()
[0, 1, 2, 3, 4, 5, 9]
```

Adding vertices for dense graphs:

```
sage: from sage.graphs.base.dense_graph import DenseGraph
sage: D = DenseGraph(nverts=4, extra_vertices=4)
sage: D.verts()
[0, 1, 2, 3]
sage: D.add_vertices([3, -1, 4, 9])
[5]
sage: D.verts()
[0, 1, 2, 3, 4, 5, 9]
sage: D.realloc(20)
sage: D.verts()
[0, 1, 2, 3, 4, 5, 9]
```

```
>>> from sage.all import *
>>> from sage.graphs.base.dense_graph import DenseGraph
>>> D = DenseGraph(nverts=Integer(4), extra_vertices=Integer(4))
>>> D.verts()
[0, 1, 2, 3]
>>> D.add_vertices([Integer(3), -Integer(1), Integer(4), Integer(9)])
[5]
>>> D.verts()
[0, 1, 2, 3, 4, 5, 9]
>>> D.realloc(Integer(20))
>>> D.verts()
[0, 1, 2, 3, 4, 5, 9]
```

all_arcs(*u*, *v*)

Give the labels of all arcs `(u, v)`. An unlabeled arc is interpreted as having label 0.

EXAMPLES:

```
sage: from sage.graphs.base.sparse_graph import SparseGraph
sage: G = SparseGraph(5)
sage: G.add_arc_label(1,2,1)
sage: G.add_arc_label(1,2,2)
sage: G.add_arc_label(1,2,2)
sage: G.add_arc_label(1,2,2)
sage: G.add_arc_label(1,2,3)
sage: G.add_arc_label(1,2,3)
sage: G.add_arc_label(1,2,4)
sage: G.all_arcs(1,2)
[4, 3, 3, 2, 2, 2, 1]
```

```
>>> from sage.all import *
>>> from sage.graphs.base.sparse_graph import SparseGraph
>>> G = SparseGraph(Integer(5))
>>> G.add_arc_label(Integer(1),Integer(2),Integer(1))
>>> G.add_arc_label(Integer(1),Integer(2),Integer(2))
>>> G.add_arc_label(Integer(1),Integer(2),Integer(2))
>>> G.add_arc_label(Integer(1),Integer(2),Integer(2))
>>> G.add_arc_label(Integer(1),Integer(2),Integer(3))
>>> G.add_arc_label(Integer(1),Integer(2),Integer(3))
>>> G.add_arc_label(Integer(1),Integer(2),Integer(4))
>>> G.all_arcs(Integer(1),Integer(2))
[4, 3, 3, 2, 2, 2, 1]
```

arc_label(*u*, *v*)

Retrieves the first label found associated with `(u, v)`.

INPUT:

- `u`, `v` – nonnegative integers; must be in self

OUTPUT: one of

- positive integer – indicates that there is a label on `(u, v)`
- `0` – either the arc `(u, v)` is unlabeled, or there is no arc at all

EXAMPLES:

```
sage: from sage.graphs.base.sparse_graph import SparseGraph
sage: G = SparseGraph(5)
sage: G.add_arc_label(3,4,7)
sage: G.arc_label(3,4)
7
```

```
>>> from sage.all import *
>>> from sage.graphs.base.sparse_graph import SparseGraph
>>> G = SparseGraph(Integer(5))
>>> G.add_arc_label(Integer(3),Integer(4),Integer(7))
>>> G.arc_label(Integer(3),Integer(4))
7
```

To this function, an unlabeled arc is indistinguishable from a non-arc:

```
sage: G.add_arc_label(1,0)
sage: G.arc_label(1,0)
0
sage: G.arc_label(1,1)
0
```

```
>>> from sage.all import *
>>> G.add_arc_label(Integer(1),Integer(0))
>>> G.arc_label(Integer(1),Integer(0))
0
>>> G.arc_label(Integer(1),Integer(1))
0
```

This function only returns the *first* label it finds from `u` to `v`:

```
sage: G.add_arc_label(1,2,1)
sage: G.add_arc_label(1,2,2)
sage: G.arc_label(1,2)
2
```

```
>>> from sage.all import *
>>> G.add_arc_label(Integer(1),Integer(2),Integer(1))
>>> G.add_arc_label(Integer(1),Integer(2),Integer(2))
>>> G.arc_label(Integer(1),Integer(2))
2
```

check_vertex(*n*)

Check that n is a vertex of `self`.

This method is different from *has_vertex()*. The current method raises an error if n is not a vertex of this graph. On the other hand, *has_vertex()* returns a boolean to signify whether or not n is a vertex of this graph.

INPUT:

- `n` – nonnegative integer representing a vertex

OUTPUT: raise an error if n is not a vertex of this graph

See also

- *has_vertex()* – determine whether this graph has a specific vertex

EXAMPLES:

```
sage: from sage.graphs.base.sparse_graph import SparseGraph
sage: S = SparseGraph(nverts=10, expected_degree=3, extra_vertices=10)
sage: S.check_vertex(4)
sage: S.check_vertex(12)
Traceback (most recent call last):
...
LookupError: vertex (12) is not a vertex of the graph
sage: S.check_vertex(24)
Traceback (most recent call last):
...
LookupError: vertex (24) is not a vertex of the graph
sage: S.check_vertex(-19)
Traceback (most recent call last):
...
LookupError: vertex (-19) is not a vertex of the graph
```

```
>>> from sage.all import *
>>> from sage.graphs.base.sparse_graph import SparseGraph
>>> S = SparseGraph(nverts=Integer(10), expected_degree=Integer(3), extra_
↪vertices=Integer(10))
>>> S.check_vertex(Integer(4))
>>> S.check_vertex(Integer(12))
Traceback (most recent call last):
...
LookupError: vertex (12) is not a vertex of the graph
>>> S.check_vertex(Integer(24))
Traceback (most recent call last):
...
LookupError: vertex (24) is not a vertex of the graph
>>> S.check_vertex(-Integer(19))
Traceback (most recent call last):
...
LookupError: vertex (-19) is not a vertex of the graph
```

```
sage: from sage.graphs.base.dense_graph import DenseGraph
sage: D = DenseGraph(nverts=10, extra_vertices=10)
sage: D.check_vertex(4)
sage: D.check_vertex(12)
Traceback (most recent call last):
...
LookupError: vertex (12) is not a vertex of the graph
sage: D.check_vertex(24)
Traceback (most recent call last):
...
LookupError: vertex (24) is not a vertex of the graph
```

(continues on next page)

(continued from previous page)

```
sage: D.check_vertex(-19)
Traceback (most recent call last):
...
LookupError: vertex (-19) is not a vertex of the graph
```

```
>>> from sage.all import *
>>> from sage.graphs.base.dense_graph import DenseGraph
>>> D = DenseGraph(nverts=Integer(10), extra_vertices=Integer(10))
>>> D.check_vertex(Integer(4))
>>> D.check_vertex(Integer(12))
Traceback (most recent call last):
...
LookupError: vertex (12) is not a vertex of the graph
>>> D.check_vertex(Integer(24))
Traceback (most recent call last):
...
LookupError: vertex (24) is not a vertex of the graph
>>> D.check_vertex(-Integer(19))
Traceback (most recent call last):
...
LookupError: vertex (-19) is not a vertex of the graph
```

current_allocation()

Report the number of vertices allocated.

OUTPUT:

- The number of vertices allocated. This number is usually different from the order of a graph. We may have allocated enough memory for a graph to hold $n > 0$ vertices, but the order (actual number of vertices) of the graph could be less than n.

EXAMPLES:

```
sage: from sage.graphs.base.sparse_graph import SparseGraph
sage: S = SparseGraph(nverts=4, extra_vertices=4)
sage: S.current_allocation()
8
sage: S.add_vertex(6)
6
sage: S.current_allocation()
8
sage: S.add_vertex(10)
10
sage: S.current_allocation()
16
sage: S.add_vertex(40)
Traceback (most recent call last):
...
RuntimeError: requested vertex is past twice the allocated range: use realloc
sage: S.realloc(50)
sage: S.add_vertex(40)
40
sage: S.current_allocation()
```

(continues on next page)

(continued from previous page)

```
50
sage: S.realloc(30)
-1
sage: S.current_allocation()
50
sage: S.del_vertex(40)
sage: S.realloc(30)
sage: S.current_allocation()
30
```

```
>>> from sage.all import *
>>> from sage.graphs.base.sparse_graph import SparseGraph
>>> S = SparseGraph(nverts=Integer(4), extra_vertices=Integer(4))
>>> S.current_allocation()
8
>>> S.add_vertex(Integer(6))
6
>>> S.current_allocation()
8
>>> S.add_vertex(Integer(10))
10
>>> S.current_allocation()
16
>>> S.add_vertex(Integer(40))
Traceback (most recent call last):
...
RuntimeError: requested vertex is past twice the allocated range: use realloc
>>> S.realloc(Integer(50))
>>> S.add_vertex(Integer(40))
40
>>> S.current_allocation()
50
>>> S.realloc(Integer(30))
-1
>>> S.current_allocation()
50
>>> S.del_vertex(Integer(40))
>>> S.realloc(Integer(30))
>>> S.current_allocation()
30
```

The actual number of vertices in a graph might be less than the number of vertices allocated for the graph:

```
sage: from sage.graphs.base.dense_graph import DenseGraph
sage: G = DenseGraph(nverts=3, extra_vertices=2)
sage: order = len(G.verts())
sage: order
3
sage: G.current_allocation()
5
sage: order < G.current_allocation()
True
```

```
>>> from sage.all import *
>>> from sage.graphs.base.dense_graph import DenseGraph
>>> G = DenseGraph(nverts=Integer(3), extra_vertices=Integer(2))
>>> order = len(G.verts())
>>> order
3
>>> G.current_allocation()
5
>>> order < G.current_allocation()
True
```

del_all_arcs(*u*, *v*)

Delete all arcs from `u` to `v`.

INPUT:

- `u` – integer; the tail of an arc
- `v` – integer; the head of an arc

EXAMPLES:

On the *CGraph* level, this always produces an error, as there are no vertices:

```
sage: from sage.graphs.base.c_graph import CGraph
sage: G = CGraph()
sage: G.del_all_arcs(0,1)
Traceback (most recent call last):
...
LookupError: vertex (0) is not a vertex of the graph
```

```
>>> from sage.all import *
>>> from sage.graphs.base.c_graph import CGraph
>>> G = CGraph()
>>> G.del_all_arcs(Integer(0),Integer(1))
Traceback (most recent call last):
...
LookupError: vertex (0) is not a vertex of the graph
```

It works, once there are vertices and `del_arc_unsafe` is implemented:

```
sage: from sage.graphs.base.sparse_graph import SparseGraph
sage: G = SparseGraph(5)
sage: G.add_arc_label(0,1,0)
sage: G.add_arc_label(0,1,1)
sage: G.add_arc_label(0,1,2)
sage: G.add_arc_label(0,1,3)
sage: G.del_all_arcs(0,1)
sage: G.has_arc(0,1)
False
sage: G.arc_label(0,1)
0
sage: G.del_all_arcs(0,1)

sage: from sage.graphs.base.dense_graph import DenseGraph
```

(continues on next page)

(continued from previous page)

```
sage: G = DenseGraph(5)
sage: G.add_arc(0, 1)
sage: G.has_arc(0, 1)
True
sage: G.del_all_arcs(0, 1)
sage: G.has_arc(0, 1)
False
```

```
>>> from sage.all import *
>>> from sage.graphs.base.sparse_graph import SparseGraph
>>> G = SparseGraph(Integer(5))
>>> G.add_arc_label(Integer(0),Integer(1),Integer(0))
>>> G.add_arc_label(Integer(0),Integer(1),Integer(1))
>>> G.add_arc_label(Integer(0),Integer(1),Integer(2))
>>> G.add_arc_label(Integer(0),Integer(1),Integer(3))
>>> G.del_all_arcs(Integer(0),Integer(1))
>>> G.has_arc(Integer(0),Integer(1))
False
>>> G.arc_label(Integer(0),Integer(1))
0
>>> G.del_all_arcs(Integer(0),Integer(1))

>>> from sage.graphs.base.dense_graph import DenseGraph
>>> G = DenseGraph(Integer(5))
>>> G.add_arc(Integer(0), Integer(1))
>>> G.has_arc(Integer(0), Integer(1))
True
>>> G.del_all_arcs(Integer(0), Integer(1))
>>> G.has_arc(Integer(0), Integer(1))
False
```

del_arc_label(*u*, *v*, *l*)

Delete an arc `(u, v)` with label `l`.

INPUT:

- `u`, `v` – nonnegative integers; must be in self
- `l` – positive integer label, or zero for no label

EXAMPLES:

```
sage: from sage.graphs.base.sparse_graph import SparseGraph
sage: G = SparseGraph(5)
sage: G.add_arc_label(0,1,0)
sage: G.add_arc_label(0,1,1)
sage: G.add_arc_label(0,1,2)
sage: G.add_arc_label(0,1,2)
sage: G.add_arc_label(0,1,3)
sage: G.del_arc_label(0,1,2)
sage: G.all_arcs(0,1)
[0, 3, 2, 1]
sage: G.del_arc_label(0,1,0)
```

(continues on next page)

(continued from previous page)

```
sage: G.all_arcs(0,1)
[3, 2, 1]
```

```
>>> from sage.all import *
>>> from sage.graphs.base.sparse_graph import SparseGraph
>>> G = SparseGraph(Integer(5))
>>> G.add_arc_label(Integer(0),Integer(1),Integer(0))
>>> G.add_arc_label(Integer(0),Integer(1),Integer(1))
>>> G.add_arc_label(Integer(0),Integer(1),Integer(2))
>>> G.add_arc_label(Integer(0),Integer(1),Integer(2))
>>> G.add_arc_label(Integer(0),Integer(1),Integer(3))
>>> G.del_arc_label(Integer(0),Integer(1),Integer(2))
>>> G.all_arcs(Integer(0),Integer(1))
[0, 3, 2, 1]
>>> G.del_arc_label(Integer(0),Integer(1),Integer(0))
>>> G.all_arcs(Integer(0),Integer(1))
[3, 2, 1]
```

del_vertex(*v*)

Delete the vertex v, along with all edges incident to it.

If v is not in `self`, fails silently.

INPUT:

- `v` – nonnegative integer representing a vertex

See also

- `del_vertex_unsafe` – delete a vertex from a graph. This method is potentially unsafe. Use *del_vertex()* instead

EXAMPLES:

Deleting vertices of sparse graphs:

```
sage: from sage.graphs.base.sparse_graph import SparseGraph
sage: G = SparseGraph(3)
sage: G.add_arc(0, 1)
sage: G.add_arc(0, 2)
sage: G.add_arc(1, 2)
sage: G.add_arc(2, 0)
sage: G.del_vertex(2)
sage: for i in range(2):
....:     for j in range(2):
....:         if G.has_arc(i, j):
....:             print("{} {}".format(i,j))
0 1
sage: G = SparseGraph(3)
sage: G.add_arc(0, 1)
sage: G.add_arc(0, 2)
sage: G.add_arc(1, 2)
```

(continues on next page)

(continued from previous page)

```
sage: G.add_arc(2, 0)
sage: G.del_vertex(1)
sage: for i in range(3):
....:     for j in range(3):
....:         if G.has_arc(i, j):
....:             print("{} {}".format(i,j))
0 2
2 0
```

```
>>> from sage.all import *
>>> from sage.graphs.base.sparse_graph import SparseGraph
>>> G = SparseGraph(Integer(3))
>>> G.add_arc(Integer(0), Integer(1))
>>> G.add_arc(Integer(0), Integer(2))
>>> G.add_arc(Integer(1), Integer(2))
>>> G.add_arc(Integer(2), Integer(0))
>>> G.del_vertex(Integer(2))
>>> for i in range(Integer(2)):
...     for j in range(Integer(2)):
...         if G.has_arc(i, j):
...             print("{} {}".format(i,j))
0 1
>>> G = SparseGraph(Integer(3))
>>> G.add_arc(Integer(0), Integer(1))
>>> G.add_arc(Integer(0), Integer(2))
>>> G.add_arc(Integer(1), Integer(2))
>>> G.add_arc(Integer(2), Integer(0))
>>> G.del_vertex(Integer(1))
>>> for i in range(Integer(3)):
...     for j in range(Integer(3)):
...         if G.has_arc(i, j):
...             print("{} {}".format(i,j))
0 2
2 0
```

Deleting vertices of dense graphs:

```
sage: from sage.graphs.base.dense_graph import DenseGraph
sage: G = DenseGraph(4)
sage: G.add_arc(0, 1); G.add_arc(0, 2)
sage: G.add_arc(3, 1); G.add_arc(3, 2)
sage: G.add_arc(1, 2)
sage: G.verts()
[0, 1, 2, 3]
sage: G.del_vertex(3); G.verts()
[0, 1, 2]
sage: for i in range(3):
....:     for j in range(3):
....:         if G.has_arc(i, j):
....:             print("{} {}".format(i,j))
0 1
0 2
```

(continues on next page)

(continued from previous page)

```
1 2
```

```
>>> from sage.all import *
>>> from sage.graphs.base.dense_graph import DenseGraph
>>> G = DenseGraph(Integer(4))
>>> G.add_arc(Integer(0), Integer(1)); G.add_arc(Integer(0), Integer(2))
>>> G.add_arc(Integer(3), Integer(1)); G.add_arc(Integer(3), Integer(2))
>>> G.add_arc(Integer(1), Integer(2))
>>> G.verts()
[0, 1, 2, 3]
>>> G.del_vertex(Integer(3)); G.verts()
[0, 1, 2]
>>> for i in range(Integer(3)):
...     for j in range(Integer(3)):
...         if G.has_arc(i, j):
...             print("{} {}".format(i,j))
0 1
0 2
1 2
```

If the vertex to be deleted is not in this graph, then fail silently:

```
sage: from sage.graphs.base.sparse_graph import SparseGraph
sage: G = SparseGraph(3)
sage: G.verts()
[0, 1, 2]
sage: G.has_vertex(3)
False
sage: G.del_vertex(3)
sage: G.verts()
[0, 1, 2]
```

```
>>> from sage.all import *
>>> from sage.graphs.base.sparse_graph import SparseGraph
>>> G = SparseGraph(Integer(3))
>>> G.verts()
[0, 1, 2]
>>> G.has_vertex(Integer(3))
False
>>> G.del_vertex(Integer(3))
>>> G.verts()
[0, 1, 2]
```

```
sage: from sage.graphs.base.dense_graph import DenseGraph
sage: G = DenseGraph(5)
sage: G.verts()
[0, 1, 2, 3, 4]
sage: G.has_vertex(6)
False
sage: G.del_vertex(6)
sage: G.verts()
[0, 1, 2, 3, 4]
```

```
>>> from sage.all import *
>>> from sage.graphs.base.dense_graph import DenseGraph
>>> G = DenseGraph(Integer(5))
>>> G.verts()
[0, 1, 2, 3, 4]
>>> G.has_vertex(Integer(6))
False
>>> G.del_vertex(Integer(6))
>>> G.verts()
[0, 1, 2, 3, 4]
```

has_arc(*u*, *v*)

Check if the arc `(u, v)` is in this graph.

INPUT:

- `u` – integer; the tail of an arc
- `v` – integer; the head of an arc

OUTPUT:

- Print a `Not Implemented!` message. This method is not implemented at the *CGraph* level. A child class should provide a suitable implementation.

EXAMPLES:

On the *CGraph* this always returns `False`:

```
sage: from sage.graphs.base.c_graph import CGraph
sage: G = CGraph()
sage: G.has_arc(0, 1)
False
```

```
>>> from sage.all import *
>>> from sage.graphs.base.c_graph import CGraph
>>> G = CGraph()
>>> G.has_arc(Integer(0), Integer(1))
False
```

It works once `has_arc_unsafe` is implemented:

```
sage: from sage.graphs.base.dense_graph import DenseGraph
sage: G = DenseGraph(5)
sage: G.add_arc(0, 1)
sage: G.has_arc(1, 0)
False
sage: G.has_arc(0, 1)
True

sage: from sage.graphs.base.sparse_graph import SparseGraph
sage: G = SparseGraph(5)
sage: G.add_arc_label(0,1)
sage: G.has_arc(1,0)
False
```

(continues on next page)

(continued from previous page)

```
sage: G.has_arc(0,1)
True
```

```
>>> from sage.all import *
>>> from sage.graphs.base.dense_graph import DenseGraph
>>> G = DenseGraph(Integer(5))
>>> G.add_arc(Integer(0), Integer(1))
>>> G.has_arc(Integer(1), Integer(0))
False
>>> G.has_arc(Integer(0), Integer(1))
True

>>> from sage.graphs.base.sparse_graph import SparseGraph
>>> G = SparseGraph(Integer(5))
>>> G.add_arc_label(Integer(0),Integer(1))
>>> G.has_arc(Integer(1),Integer(0))
False
>>> G.has_arc(Integer(0),Integer(1))
True
```

has_arc_label(*u*, *v*, *l*)

Indicates whether there is an arc `(u, v)` with label `l`.

INPUT:

- `u`, `v` – nonnegative integers; must be in self
- `l` – positive integer label, or zero for no label

EXAMPLES:

```
sage: from sage.graphs.base.sparse_graph import SparseGraph
sage: G = SparseGraph(5)
sage: G.add_arc_label(0,1,0)
sage: G.add_arc_label(0,1,1)
sage: G.add_arc_label(0,1,2)
sage: G.add_arc_label(0,1,2)
sage: G.has_arc_label(0,1,1)
True
sage: G.has_arc_label(0,1,2)
True
sage: G.has_arc_label(0,1,3)
False
```

```
>>> from sage.all import *
>>> from sage.graphs.base.sparse_graph import SparseGraph
>>> G = SparseGraph(Integer(5))
>>> G.add_arc_label(Integer(0),Integer(1),Integer(0))
>>> G.add_arc_label(Integer(0),Integer(1),Integer(1))
>>> G.add_arc_label(Integer(0),Integer(1),Integer(2))
>>> G.add_arc_label(Integer(0),Integer(1),Integer(2))
>>> G.has_arc_label(Integer(0),Integer(1),Integer(1))
True
```

(continues on next page)

(continued from previous page)

```
>>> G.has_arc_label(Integer(0),Integer(1),Integer(2))
True
>>> G.has_arc_label(Integer(0),Integer(1),Integer(3))
False
```

has_vertex(*n*)

Determine whether the vertex n is in `self`.

This method is different from *check_vertex()*. The current method returns a boolean to signify whether or not n is a vertex of this graph. On the other hand, *check_vertex()* raises an error if n is not a vertex of this graph.

INPUT:

- `n` – nonnegative integer representing a vertex

OUTPUT: `True` if n is a vertex of this graph; `False` otherwise

See also

- *check_vertex()* – raise an error if this graph does not contain a specific vertex.

EXAMPLES:

Upon initialization, a *SparseGraph* or *DenseGraph* has the first `nverts` vertices:

```
sage: from sage.graphs.base.sparse_graph import SparseGraph
sage: S = SparseGraph(nverts=10, expected_degree=3, extra_vertices=10)
sage: S.has_vertex(6)
True
sage: S.has_vertex(12)
False
sage: S.has_vertex(24)
False
sage: S.has_vertex(-19)
False
```

```
>>> from sage.all import *
>>> from sage.graphs.base.sparse_graph import SparseGraph
>>> S = SparseGraph(nverts=Integer(10), expected_degree=Integer(3), extra_
↪vertices=Integer(10))
>>> S.has_vertex(Integer(6))
True
>>> S.has_vertex(Integer(12))
False
>>> S.has_vertex(Integer(24))
False
>>> S.has_vertex(-Integer(19))
False
```

```
sage: from sage.graphs.base.dense_graph import DenseGraph
sage: D = DenseGraph(nverts=10, extra_vertices=10)
```

(continues on next page)

(continued from previous page)

```
sage: D.has_vertex(6)
True
sage: D.has_vertex(12)
False
sage: D.has_vertex(24)
False
sage: D.has_vertex(-19)
False
```

```
>>> from sage.all import *
>>> from sage.graphs.base.dense_graph import DenseGraph
>>> D = DenseGraph(nverts=Integer(10), extra_vertices=Integer(10))
>>> D.has_vertex(Integer(6))
True
>>> D.has_vertex(Integer(12))
False
>>> D.has_vertex(Integer(24))
False
>>> D.has_vertex(-Integer(19))
False
```

in_neighbors(*v*)

Return the list of in-neighbors of the vertex `v`.

INPUT:

- `v` – integer representing a vertex of this graph

OUTPUT:

- Raise `NotImplementedError`. This method is not implemented at the *CGraph* level. A child class should provide a suitable implementation.

Note

Due to the implementation of SparseGraph, this method is much more expensive than out_neighbors_unsafe for SparseGraph's.

EXAMPLES:

On the *CGraph* level, this always produces an error, as there are no vertices:

```
sage: from sage.graphs.base.c_graph import CGraph
sage: G = CGraph()
sage: G.in_neighbors(0)
Traceback (most recent call last):
...
LookupError: vertex (0) is not a vertex of the graph
```

```
>>> from sage.all import *
>>> from sage.graphs.base.c_graph import CGraph
>>> G = CGraph()
>>> G.in_neighbors(Integer(0))
```

(continues on next page)

(continued from previous page)

```
Traceback (most recent call last):
...
LookupError: vertex (0) is not a vertex of the graph
```

It works, once there are vertices and `out_neighbors_unsafe` is implemented:

```
sage: from sage.graphs.base.dense_graph import DenseGraph
sage: G = DenseGraph(5)
sage: G.add_arc(0, 1)
sage: G.add_arc(3, 1)
sage: G.add_arc(1, 3)
sage: G.in_neighbors(1)
[0, 3]
sage: G.in_neighbors(3)
[1]

sage: from sage.graphs.base.sparse_graph import SparseGraph
sage: G = SparseGraph(5)
sage: G.add_arc(0,1)
sage: G.add_arc(3,1)
sage: G.add_arc(1,3)
sage: G.in_neighbors(1)
[0, 3]
sage: G.in_neighbors(3)
[1]
```

```
>>> from sage.all import *
>>> from sage.graphs.base.dense_graph import DenseGraph
>>> G = DenseGraph(Integer(5))
>>> G.add_arc(Integer(0), Integer(1))
>>> G.add_arc(Integer(3), Integer(1))
>>> G.add_arc(Integer(1), Integer(3))
>>> G.in_neighbors(Integer(1))
[0, 3]
>>> G.in_neighbors(Integer(3))
[1]

>>> from sage.graphs.base.sparse_graph import SparseGraph
>>> G = SparseGraph(Integer(5))
>>> G.add_arc(Integer(0),Integer(1))
>>> G.add_arc(Integer(3),Integer(1))
>>> G.add_arc(Integer(1),Integer(3))
>>> G.in_neighbors(Integer(1))
[0, 3]
>>> G.in_neighbors(Integer(3))
[1]
```

out_neighbors(*u*)

Return the list of out-neighbors of the vertex `u`.

INPUT:

- `u` – integer representing a vertex of this graph

EXAMPLES:

On the *CGraph* level, this always produces an error, as there are no vertices:

```
sage: from sage.graphs.base.c_graph import CGraph
sage: G = CGraph()
sage: G.out_neighbors(0)
Traceback (most recent call last):
...
LookupError: vertex (0) is not a vertex of the graph
```

```
>>> from sage.all import *
>>> from sage.graphs.base.c_graph import CGraph
>>> G = CGraph()
>>> G.out_neighbors(Integer(0))
Traceback (most recent call last):
...
LookupError: vertex (0) is not a vertex of the graph
```

It works, once there are vertices and `out_neighbors_unsafe` is implemented:

```
sage: from sage.graphs.base.dense_graph import DenseGraph
sage: G = DenseGraph(5)
sage: G.add_arc(0, 1)
sage: G.add_arc(1, 2)
sage: G.add_arc(1, 3)
sage: G.out_neighbors(0)
[1]
sage: G.out_neighbors(1)
[2, 3]

sage: from sage.graphs.base.sparse_graph import SparseGraph
sage: G = SparseGraph(5)
sage: G.add_arc(0,1)
sage: G.add_arc(1,2)
sage: G.add_arc(1,3)
sage: G.out_neighbors(0)
[1]
sage: G.out_neighbors(1)
[2, 3]
```

```
>>> from sage.all import *
>>> from sage.graphs.base.dense_graph import DenseGraph
>>> G = DenseGraph(Integer(5))
>>> G.add_arc(Integer(0), Integer(1))
>>> G.add_arc(Integer(1), Integer(2))
>>> G.add_arc(Integer(1), Integer(3))
>>> G.out_neighbors(Integer(0))
[1]
>>> G.out_neighbors(Integer(1))
[2, 3]

>>> from sage.graphs.base.sparse_graph import SparseGraph
>>> G = SparseGraph(Integer(5))
```

(continues on next page)

(continued from previous page)

```
>>> G.add_arc(Integer(0),Integer(1))
>>> G.add_arc(Integer(1),Integer(2))
>>> G.add_arc(Integer(1),Integer(3))
>>> G.out_neighbors(Integer(0))
[1]
>>> G.out_neighbors(Integer(1))
[2, 3]
```

realloc(*total*)

Reallocate the number of vertices to use, without actually adding any.

INPUT:

- `total` – integer; the total size to make the array of vertices

OUTPUT:

- Raise a `NotImplementedError`. This method is not implemented in this base class. A child class should provide a suitable implementation.

See also

- *realloc* – a `realloc` implementation for sparse graphs.
- *realloc* – a `realloc` implementation for dense graphs.

EXAMPLES:

First, note that *realloc()* is implemented for *SparseGraph* and *DenseGraph* differently, and is not implemented at the *CGraph* level:

```
sage: from sage.graphs.base.c_graph import CGraph
sage: G = CGraph()
sage: G.realloc(20)
Traceback (most recent call last):
...
NotImplementedError
```

```
>>> from sage.all import *
>>> from sage.graphs.base.c_graph import CGraph
>>> G = CGraph()
>>> G.realloc(Integer(20))
Traceback (most recent call last):
...
NotImplementedError
```

The `realloc` implementation for sparse graphs:

```
sage: from sage.graphs.base.sparse_graph import SparseGraph
sage: S = SparseGraph(nverts=4, extra_vertices=4)
sage: S.current_allocation()
8
sage: S.add_vertex(6)
```

(continues on next page)

(continued from previous page)

```
6
sage: S.current_allocation()
8
sage: S.add_vertex(10)
10
sage: S.current_allocation()
16
sage: S.add_vertex(40)
Traceback (most recent call last):
...
RuntimeError: requested vertex is past twice the allocated range: use realloc
sage: S.realloc(50)
sage: S.add_vertex(40)
40
sage: S.current_allocation()
50
sage: S.realloc(30)
-1
sage: S.current_allocation()
50
sage: S.del_vertex(40)
sage: S.realloc(30)
sage: S.current_allocation()
30
```

```
>>> from sage.all import *
>>> from sage.graphs.base.sparse_graph import SparseGraph
>>> S = SparseGraph(nverts=Integer(4), extra_vertices=Integer(4))
>>> S.current_allocation()
8
>>> S.add_vertex(Integer(6))
6
>>> S.current_allocation()
8
>>> S.add_vertex(Integer(10))
10
>>> S.current_allocation()
16
>>> S.add_vertex(Integer(40))
Traceback (most recent call last):
...
RuntimeError: requested vertex is past twice the allocated range: use realloc
>>> S.realloc(Integer(50))
>>> S.add_vertex(Integer(40))
40
>>> S.current_allocation()
50
>>> S.realloc(Integer(30))
-1
>>> S.current_allocation()
50
>>> S.del_vertex(Integer(40))
```

(continues on next page)

(continued from previous page)

```
>>> S.realloc(Integer(30))
>>> S.current_allocation()
30
```

The `realloc` implementation for dense graphs:

```
sage: from sage.graphs.base.dense_graph import DenseGraph
sage: D = DenseGraph(nverts=4, extra_vertices=4)
sage: D.current_allocation()
8
sage: D.add_vertex(6)
6
sage: D.current_allocation()
8
sage: D.add_vertex(10)
10
sage: D.current_allocation()
16
sage: D.add_vertex(40)
Traceback (most recent call last):
...
RuntimeError: requested vertex is past twice the allocated range: use realloc
sage: D.realloc(50)
sage: D.add_vertex(40)
40
sage: D.current_allocation()
50
sage: D.realloc(30)
-1
sage: D.current_allocation()
50
sage: D.del_vertex(40)
sage: D.realloc(30)
sage: D.current_allocation()
30
```

```
>>> from sage.all import *
>>> from sage.graphs.base.dense_graph import DenseGraph
>>> D = DenseGraph(nverts=Integer(4), extra_vertices=Integer(4))
>>> D.current_allocation()
8
>>> D.add_vertex(Integer(6))
6
>>> D.current_allocation()
8
>>> D.add_vertex(Integer(10))
10
>>> D.current_allocation()
16
>>> D.add_vertex(Integer(40))
Traceback (most recent call last):
...
```

(continues on next page)

(continued from previous page)

```
RuntimeError: requested vertex is past twice the allocated range: use realloc
>>> D.realloc(Integer(50))
>>> D.add_vertex(Integer(40))
40
>>> D.current_allocation()
50
>>> D.realloc(Integer(30))
-1
>>> D.current_allocation()
50
>>> D.del_vertex(Integer(40))
>>> D.realloc(Integer(30))
>>> D.current_allocation()
30
```

verts()

Return a list of the vertices in `self`.

EXAMPLES:

```
sage: from sage.graphs.base.sparse_graph import SparseGraph
sage: S = SparseGraph(nverts=4, extra_vertices=4)
sage: S.verts()
[0, 1, 2, 3]
sage: S.add_vertices([3,5,7,9])
sage: S.verts()
[0, 1, 2, 3, 5, 7, 9]
sage: S.realloc(20)
sage: S.verts()
[0, 1, 2, 3, 5, 7, 9]
```

```
>>> from sage.all import *
>>> from sage.graphs.base.sparse_graph import SparseGraph
>>> S = SparseGraph(nverts=Integer(4), extra_vertices=Integer(4))
>>> S.verts()
[0, 1, 2, 3]
>>> S.add_vertices([Integer(3),Integer(5),Integer(7),Integer(9)])
>>> S.verts()
[0, 1, 2, 3, 5, 7, 9]
>>> S.realloc(Integer(20))
>>> S.verts()
[0, 1, 2, 3, 5, 7, 9]
```

```
sage: from sage.graphs.base.dense_graph import DenseGraph
sage: G = DenseGraph(3, extra_vertices=2)
sage: G.verts()
[0, 1, 2]
sage: G.del_vertex(0)
sage: G.verts()
[1, 2]
```

```
>>> from sage.all import *
>>> from sage.graphs.base.dense_graph import DenseGraph
>>> G = DenseGraph(Integer(3), extra_vertices=Integer(2))
>>> G.verts()
[0, 1, 2]
>>> G.del_vertex(Integer(0))
>>> G.verts()
[1, 2]
```

class sage.graphs.base.c_graph.**CGraphBackend**

Bases: *GenericGraphBackend*

Base class for sparse and dense graph backends.

```
sage: from sage.graphs.base.c_graph import CGraphBackend
```

```
>>> from sage.all import *
>>> from sage.graphs.base.c_graph import CGraphBackend
```

This class is extended by *SparseGraphBackend* and *DenseGraphBackend*, which are fully functional backends. This class is mainly just for vertex functions, which are the same for both. A *CGraphBackend* will not work on its own:

```
sage: from sage.graphs.base.c_graph import CGraphBackend
sage: CGB = CGraphBackend()
sage: CGB.degree(0, True)
Traceback (most recent call last):
...
NotImplementedError: a derived class must return ``self._cg``
```

```
>>> from sage.all import *
>>> from sage.graphs.base.c_graph import CGraphBackend
>>> CGB = CGraphBackend()
>>> CGB.degree(Integer(0), True)
Traceback (most recent call last):
...
NotImplementedError: a derived class must return ``self._cg``
```

The appropriate way to use these backends is via Sage graphs:

```
sage: G = Graph(30)
sage: G.add_edges([(0,1), (0,3), (4,5), (9, 23)])
sage: G.edges(sort=True, labels=False)
[(0, 1), (0, 3), (4, 5), (9, 23)]
```

```
>>> from sage.all import *
>>> G = Graph(Integer(30))
>>> G.add_edges([(Integer(0),Integer(1)), (Integer(0),Integer(3)), (Integer(4),
↪Integer(5)), (Integer(9), Integer(23))])
>>> G.edges(sort=True, labels=False)
[(0, 1), (0, 3), (4, 5), (9, 23)]
```

This class handles the labels of vertices and edges. For vertices it uses two dictionaries `vertex_labels` and `vertex_ints`. They are just opposite of each other: `vertex_ints` makes a translation from label to integers

(that are internally used) and `vertex_labels` make the translation from internally used integers to actual labels. This class tries hard to avoid translation if possible. This will work only if the graph is built on integers from 0 to $n - 1$ and the vertices are basically added in increasing order.

See also

- *SparseGraphBackend* – backend for sparse graphs.
- *DenseGraphBackend* – backend for dense graphs.

add_edge(*u*, *v*, *l*, *directed*)

Add the edge `(u,v)` to `self`.

INPUT:

- `u`, `v` – the vertices of the edge
- `l` – the edge label
- `directed` – if `False`, also add `(v,u)`

Note

The input `l` is ignored if the backend does not support labels.

EXAMPLES:

```
sage: D = sage.graphs.base.sparse_graph.SparseGraphBackend(9)
sage: D.add_edge(0,1,None,False)
sage: list(D.iterator_edges(range(9), True))
[(0, 1, None)]
```

```
>>> from sage.all import *
>>> D = sage.graphs.base.sparse_graph.SparseGraphBackend(Integer(9))
>>> D.add_edge(Integer(0),Integer(1),None,False)
>>> list(D.iterator_edges(range(Integer(9)), True))
[(0, 1, None)]
```

```
sage: D = sage.graphs.base.dense_graph.DenseGraphBackend(9)
sage: D.add_edge(0, 1, None, False)
sage: list(D.iterator_edges(range(9), True))
[(0, 1, None)]
```

```
>>> from sage.all import *
>>> D = sage.graphs.base.dense_graph.DenseGraphBackend(Integer(9))
>>> D.add_edge(Integer(0), Integer(1), None, False)
>>> list(D.iterator_edges(range(Integer(9)), True))
[(0, 1, None)]
```

add_edges(*edges*, *directed*, *remove_loops=False*)

Add edges from a list.

INPUT:

- `edges` – the edges to be added; can either be of the form `(u,v)` or `(u,v,l)`
- `directed` – if `False`, add `(v,u)` as well as `(u,v)`
- `remove_loops` – if `True`, remove loops

EXAMPLES:

```
sage: D = sage.graphs.base.sparse_graph.SparseGraphBackend(9)
sage: D.add_edges([(0,1), (2,3), (4,5), (5,6)], False)
sage: list(D.iterator_edges(range(9), True))
[(0, 1, None),
 (2, 3, None),
 (4, 5, None),
 (5, 6, None)]
```

```
>>> from sage.all import *
>>> D = sage.graphs.base.sparse_graph.SparseGraphBackend(Integer(9))
>>> D.add_edges([(Integer(0),Integer(1)), (Integer(2),Integer(3)),␣
↪(Integer(4),Integer(5)), (Integer(5),Integer(6))], False)
>>> list(D.iterator_edges(range(Integer(9)), True))
[(0, 1, None),
 (2, 3, None),
 (4, 5, None),
 (5, 6, None)]
```

add_vertex(*name*)

Add a vertex to `self`.

INPUT:

- `name` – the vertex to be added (must be hashable). If `None`, a new name is created

OUTPUT:

- If `name = None`, the new vertex name is returned. `None` otherwise.

See also

- *add_vertices()* – add a bunch of vertices of this graph
- *has_vertex()* – returns whether or not this graph has a specific vertex

EXAMPLES:

```
sage: D = sage.graphs.base.dense_graph.DenseGraphBackend(9)
sage: D.add_vertex(10)
sage: D.add_vertex([])
Traceback (most recent call last):
...
TypeError: ...unhashable type: 'list'...
```

```
>>> from sage.all import *
>>> D = sage.graphs.base.dense_graph.DenseGraphBackend(Integer(9))
>>> D.add_vertex(Integer(10))
```

(continues on next page)

(continued from previous page)

```
>>> D.add_vertex([])
Traceback (most recent call last):
...
TypeError: ...unhashable type: 'list'...
```

```
sage: S = sage.graphs.base.sparse_graph.SparseGraphBackend(9)
sage: S.add_vertex(10)
sage: S.add_vertex([])
Traceback (most recent call last):
...
TypeError: ...unhashable type: 'list'...
```

```
>>> from sage.all import *
>>> S = sage.graphs.base.sparse_graph.SparseGraphBackend(Integer(9))
>>> S.add_vertex(Integer(10))
>>> S.add_vertex([])
Traceback (most recent call last):
...
TypeError: ...unhashable type: 'list'...
```

add_vertices(*vertices*)

Add vertices to `self`.

INPUT:

- `vertices` – iterator of vertex labels; a new name is created, used and returned in the output list for all `None` values in `vertices`

OUTPUT:

Generated names of new vertices if there is at least one `None` value present in `vertices`. `None` otherwise.

See also

- *add_vertex()* – add a vertex to this graph

EXAMPLES:

```
sage: D = sage.graphs.base.sparse_graph.SparseGraphBackend(1)
sage: D.add_vertices([1, 2, 3])
sage: D.add_vertices([None] * 4)
[4, 5, 6, 7]
```

```
>>> from sage.all import *
>>> D = sage.graphs.base.sparse_graph.SparseGraphBackend(Integer(1))
>>> D.add_vertices([Integer(1), Integer(2), Integer(3)])
>>> D.add_vertices([None] * Integer(4))
[4, 5, 6, 7]
```

```
sage: G = sage.graphs.base.sparse_graph.SparseGraphBackend(0)
sage: G.add_vertices([0, 1])
```

(continues on next page)

(continued from previous page)

```
sage: list(G.iterator_verts(None))
[0, 1]
sage: list(G.iterator_edges([0, 1], True))
[]
```

```
>>> from sage.all import *
>>> G = sage.graphs.base.sparse_graph.SparseGraphBackend(Integer(0))
>>> G.add_vertices([Integer(0), Integer(1)])
>>> list(G.iterator_verts(None))
[0, 1]
>>> list(G.iterator_edges([Integer(0), Integer(1)], True))
[]
```

```
sage: import sage.graphs.base.dense_graph
sage: D = sage.graphs.base.dense_graph.DenseGraphBackend(9)
sage: D.add_vertices([10, 11, 12])
```

```
>>> from sage.all import *
>>> import sage.graphs.base.dense_graph
>>> D = sage.graphs.base.dense_graph.DenseGraphBackend(Integer(9))
>>> D.add_vertices([Integer(10), Integer(11), Integer(12)])
```

bidirectional_dijkstra(*x*, *y*, *weight_function=None*, *distance_flag=False*)

Return the shortest path or distance from `x` to `y` using a bidirectional version of Dijkstra's algorithm.

INPUT:

- `x` – the starting vertex in the shortest path from `x` to `y`
- `y` – the end vertex in the shortest path from `x` to `y`
- `weight_function` – function (default: `None`); a function that inputs an edge `(u, v, l)` and outputs its weight. If `None`, we use the edge label `l` as a weight, if `l` is not `None`, else `1` as a weight.
- `distance_flag` – boolean (default: `False`); when set to `True`, the shortest path distance from `x` to `y` is returned instead of the path.

OUTPUT:

- A list of vertices in the shortest path from `x` to `y` or distance from `x` to `y` is returned depending upon the value of parameter `distance_flag`

EXAMPLES:

```
sage: G = Graph(graphs.PetersenGraph())
sage: for (u, v) in G.edges(sort=True, labels=None):
....:     G.set_edge_label(u, v, 1)
sage: G.shortest_path(0, 1, by_weight=True)
[0, 1]
sage: G.shortest_path_length(0, 1, by_weight=True)
1
sage: G = DiGraph([(1, 2, {'weight':1}), (1, 3, {'weight':5}), (2, 3, {'weight
↪':1})])
sage: G.shortest_path(1, 3, weight_function=lambda e:e[2]['weight'])
[1, 2, 3]
```

(continues on next page)

(continued from previous page)

```
sage: G.shortest_path_length(1, 3, weight_function=lambda e:e[2]['weight'])
2
```

```
>>> from sage.all import *
>>> G = Graph(graphs.PetersenGraph())
>>> for (u, v) in G.edges(sort=True, labels=None):
...     G.set_edge_label(u, v, Integer(1))
>>> G.shortest_path(Integer(0), Integer(1), by_weight=True)
[0, 1]
>>> G.shortest_path_length(Integer(0), Integer(1), by_weight=True)
1
>>> G = DiGraph([(Integer(1), Integer(2), {'weight':Integer(1)}), (Integer(1),
↪ Integer(3), {'weight':Integer(5)}), (Integer(2), Integer(3), {'weight
↪':Integer(1)})])
>>> G.shortest_path(Integer(1), Integer(3), weight_function=lambda␣
↪e:e[Integer(2)]['weight'])
[1, 2, 3]
>>> G.shortest_path_length(Integer(1), Integer(3), weight_function=lambda␣
↪e:e[Integer(2)]['weight'])
2
```

bidirectional_dijkstra_special(*x*, *y*, *weight_function=None*, *exclude_vertices=None*, *exclude_edges=None*, *include_vertices=None*, *distance_flag=False*, *reduced_weight=None*)

Return the shortest path or distance from `x` to `y` using a bidirectional version of Dijkstra's algorithm.

This method is an extension of *`bidirectional_dijkstra()`* method enabling to exclude vertices and/or edges from the search for the shortest path between `x` and `y`.

This method also has `include_vertices` option enabling to include the vertices which will be used to search for the shortest path between `x` and `y`.

INPUT:

- `x` – the starting vertex in the shortest path from `x` to `y`
- `y` – the end vertex in the shortest path from `x` to `y`
- `exclude_vertices` – iterable container (default: `None`); iterable of vertices to exclude from the graph while calculating the shortest path from `x` to `y`
- `exclude_edges` – iterable container (default: `None`); iterable of edges to exclude from the graph while calculating the shortest path from `x` to `y`
- `include_vertices` – iterable container (default: `None`); iterable of vertices to consider in the graph while calculating the shortest path from `x` to `y`
- `weight_function` – function (default: `None`); a function that inputs an edge `(u, v, l)` and outputs its weight. If `None`, we use the edge label `l` as a weight, if `l` is not `None`, else `1` as a weight.
- `distance_flag` – boolean (default: `False`); when set to `True`, the shortest path distance from `x` to `y` is returned instead of the path.
- `reduced_weight` – dictionary (default: `None`); a dictionary that takes as input an edge `(u, v)` and outputs its reduced weight

OUTPUT:

- A list of vertices in the shortest path from `x` to `y` or distance from `x` to `y` is returned depending upon the value of parameter `distance_flag`

EXAMPLES:

```
sage: G = Graph([(1, 2, 20), (2, 3, 10), (3, 4, 30), (1, 5, 20), (5, 6, 10),␣
↪(6, 4, 50), (4, 7, 5)])
sage: G._backend.bidirectional_dijkstra_special(1, 4, weight_function=lambda␣
↪e:e[2])
[1, 2, 3, 4]
sage: G._backend.bidirectional_dijkstra_special(1, 4, weight_function=lambda␣
↪e:e[2], exclude_vertices=[2], exclude_edges=[(3, 4)])
[1, 5, 6, 4]
sage: G._backend.bidirectional_dijkstra_special(1, 4, weight_function=lambda␣
↪e:e[2], exclude_vertices=[2, 7])
[1, 5, 6, 4]
sage: G._backend.bidirectional_dijkstra_special(1, 4, weight_function=lambda␣
↪e:e[2],  exclude_edges=[(5, 6)])
[1, 2, 3, 4]
sage: G._backend.bidirectional_dijkstra_special(1, 4, weight_function=lambda␣
↪e:e[2],  include_vertices=[1, 5, 6, 4])
[1, 5, 6, 4]
```

```
>>> from sage.all import *
>>> G = Graph([(Integer(1), Integer(2), Integer(20)), (Integer(2), Integer(3),
↪ Integer(10)), (Integer(3), Integer(4), Integer(30)), (Integer(1),␣
↪Integer(5), Integer(20)), (Integer(5), Integer(6), Integer(10)),␣
↪(Integer(6), Integer(4), Integer(50)), (Integer(4), Integer(7),␣
↪Integer(5))])
>>> G._backend.bidirectional_dijkstra_special(Integer(1), Integer(4), weight_
↪function=lambda e:e[Integer(2)])
[1, 2, 3, 4]
>>> G._backend.bidirectional_dijkstra_special(Integer(1), Integer(4), weight_
↪function=lambda e:e[Integer(2)], exclude_vertices=[Integer(2)], exclude_
↪edges=[(Integer(3), Integer(4))])
[1, 5, 6, 4]
>>> G._backend.bidirectional_dijkstra_special(Integer(1), Integer(4), weight_
↪function=lambda e:e[Integer(2)], exclude_vertices=[Integer(2), Integer(7)])
[1, 5, 6, 4]
>>> G._backend.bidirectional_dijkstra_special(Integer(1), Integer(4), weight_
↪function=lambda e:e[Integer(2)],  exclude_edges=[(Integer(5), Integer(6))])
[1, 2, 3, 4]
>>> G._backend.bidirectional_dijkstra_special(Integer(1), Integer(4), weight_
↪function=lambda e:e[Integer(2)],  include_vertices=[Integer(1), Integer(5),␣
↪Integer(6), Integer(4)])
[1, 5, 6, 4]
```

breadth_first_search(*v*, *reverse=False*, *ignore_direction=False*, *report_distance=False*, *edges=False*, *forbidden_vertices=None*)

Return a breadth-first search from vertex `v`.

INPUT:

- `v` – a vertex from which to start the breadth-first search
- `reverse` – boolean (default: `False`); this is only relevant to digraphs. If this is a digraph, consider the

reversed graph in which the out-neighbors become the in-neighbors and vice versa.

- `ignore_direction` – boolean (default: `False`); this is only relevant to digraphs. If this is a digraph, ignore all orientations and consider the graph as undirected.
- `report_distance` – boolean (default: `False`); if `True`, reports pairs `(vertex, distance)` where `distance` is the distance from the `start` nodes. If `False` only the vertices are reported.
- `edges` – boolean (default: `False`); whether to return the edges of the BFS tree in the order of visit or the vertices (default). Edges are directed in root to leaf orientation of the tree.

  Note that parameters `edges` and `report_distance` cannot be `True` simultaneously.
- `forbidden_vertices` – list (default: `None`); set of vertices to avoid during the search. The start vertex `v` cannot be in this set.

ALGORITHM:

Below is a general template for breadth-first search.

- **Input:** A directed or undirected graph $G = (V, E)$ of order $n > 0$. A vertex s from which to start the search. The vertices are numbered from 1 to $n = |V|$, i.e. $V = \{1, 2, \ldots, n\}$.
- **Output:** A list D of distances of all vertices from s. A tree T rooted at s.

1. $Q \leftarrow [s]$ # a queue of nodes to visit
2. $D \leftarrow [\infty, \infty, \ldots, \infty]$ # n copies of ∞
3. $D[s] \leftarrow 0$
4. $T \leftarrow [\,]$
5. while length$(Q) > 0$ do
    1. $v \leftarrow$ dequeue(Q)
    2. for each $w \in$ adj(v) do # for digraphs, use out-neighbor set oadj(v)
        1. if $D[w] = \infty$ then
            1. $D[w] \leftarrow D[v] + 1$
            2. enqueue(Q, w)
            3. append(T, vw)
6. return (D, T)

See also

- *breadth_first_search* – breadth-first search for generic graphs.
- *depth_first_search* – depth-first search for generic graphs.
- *depth_first_search()* – depth-first search for fast compiled graphs.

EXAMPLES:

Breadth-first search of the Petersen graph starting at vertex 0:

```
sage: G = Graph(graphs.PetersenGraph())
sage: list(G.breadth_first_search(0))
[0, 1, 4, 5, 2, 6, 3, 9, 7, 8]
```

```
>>> from sage.all import *
>>> G = Graph(graphs.PetersenGraph())
>>> list(G.breadth_first_search(Integer(0)))
[0, 1, 4, 5, 2, 6, 3, 9, 7, 8]
```

Visiting European countries using breadth-first search:

```
sage: G = graphs.EuropeMap(continental=True)
sage: list(G.breadth_first_search("Portugal"))
['Portugal', 'Spain', ..., 'Greece']
```

```
>>> from sage.all import *
>>> G = graphs.EuropeMap(continental=True)
>>> list(G.breadth_first_search("Portugal"))
['Portugal', 'Spain', ..., 'Greece']
```

Avoiding some countries:

sage: list(G.breadth_first_search("Portugal", ...: forbidden_vertices=["Germany","Italy"])) ['Portugal', 'Spain', ..., 'Sweden']

c_graph()

Return the `._cg` and `._cg_rev` attributes.

Note

The `._cg_rev` attribute has been removed and hence `None` is returned.

EXAMPLES:

```
sage: cg,cg_rev = graphs.PetersenGraph()._backend.c_graph()
sage: cg
<sage.graphs.base.sparse_graph.SparseGraph object at ...>
```

```
>>> from sage.all import *
>>> cg,cg_rev = graphs.PetersenGraph()._backend.c_graph()
>>> cg
<sage.graphs.base.sparse_graph.SparseGraph object at ...>
```

degree(*v*, *directed*)

Return the degree of the vertex `v`.

INPUT:

- `v` – a vertex of the graph
- `directed` – boolean; whether to take into account the orientation of this graph in counting the degree of `v`

OUTPUT: the degree of vertex `v`

EXAMPLES:

```
sage: from sage.graphs.base.sparse_graph import SparseGraphBackend
sage: B = SparseGraphBackend(7)
sage: B.degree(3, False)
0
```

```
>>> from sage.all import *
>>> from sage.graphs.base.sparse_graph import SparseGraphBackend
>>> B = SparseGraphBackend(Integer(7))
>>> B.degree(Integer(3), False)
0
```

del_edge(*u*, *v*, *l*, *directed*)

Delete edge `(u, v, l)`.

INPUT:

- `u`, `v` – the vertices of the edge
- `l` – the edge label
- `directed` – if `False`, also delete `(v, u, l)`

Note

The input `l` is ignored if the backend does not support labels.

EXAMPLES:

```
sage: D = sage.graphs.base.sparse_graph.SparseGraphBackend(9)
sage: D.add_edges([(0,1), (2,3), (4,5), (5,6)], False)
sage: list(D.iterator_edges(range(9), True))
[(0, 1, None),
 (2, 3, None),
 (4, 5, None),
 (5, 6, None)]
sage: D.del_edge(0,1,None,True)
sage: list(D.iterator_out_edges(range(9), True))
[(1, 0, None),
 (2, 3, None),
 (3, 2, None),
 (4, 5, None),
 (5, 4, None),
 (5, 6, None),
 (6, 5, None)]
```

```
>>> from sage.all import *
>>> D = sage.graphs.base.sparse_graph.SparseGraphBackend(Integer(9))
>>> D.add_edges([(Integer(0),Integer(1)), (Integer(2),Integer(3)),␣
↪(Integer(4),Integer(5)), (Integer(5),Integer(6))], False)
>>> list(D.iterator_edges(range(Integer(9)), True))
[(0, 1, None),
 (2, 3, None),
```

(continues on next page)

(continued from previous page)

```
 (4, 5, None),
 (5, 6, None)]
>>> D.del_edge(Integer(0),Integer(1),None,True)
>>> list(D.iterator_out_edges(range(Integer(9)), True))
[(1, 0, None),
 (2, 3, None),
 (3, 2, None),
 (4, 5, None),
 (5, 4, None),
 (5, 6, None),
 (6, 5, None)]
```

```
sage: D = sage.graphs.base.dense_graph.DenseGraphBackend(9)
sage: D.add_edges([(0, 1), (2, 3), (4, 5), (5, 6)], False)
sage: list(D.iterator_edges(range(9), True))
[(0, 1, None),
 (2, 3, None),
 (4, 5, None),
 (5, 6, None)]
sage: D.del_edge(0, 1, None, True)
sage: list(D.iterator_out_edges(range(9), True))
[(1, 0, None),
 (2, 3, None),
 (3, 2, None),
 (4, 5, None),
 (5, 4, None),
 (5, 6, None),
 (6, 5, None)]
```

```
>>> from sage.all import *
>>> D = sage.graphs.base.dense_graph.DenseGraphBackend(Integer(9))
>>> D.add_edges([(Integer(0), Integer(1)), (Integer(2), Integer(3)),␣
↪(Integer(4), Integer(5)), (Integer(5), Integer(6))], False)
>>> list(D.iterator_edges(range(Integer(9)), True))
[(0, 1, None),
 (2, 3, None),
 (4, 5, None),
 (5, 6, None)]
>>> D.del_edge(Integer(0), Integer(1), None, True)
>>> list(D.iterator_out_edges(range(Integer(9)), True))
[(1, 0, None),
 (2, 3, None),
 (3, 2, None),
 (4, 5, None),
 (5, 4, None),
 (5, 6, None),
 (6, 5, None)]
```

del_edges(*edges*, *directed*)

Delete edges from a list.

INPUT:

- `edges` – the edges to be added; can either be of the form `(u,v)` or `(u,v,l)`
- `directed` – if `False`, remove``(v,u)`` as well as `(u,v)`

EXAMPLES:

```
sage: D = sage.graphs.base.sparse_graph.SparseGraphBackend(9)
sage: D.add_edges([(0,1), (2,3), (4,5), (5,6)], False)
sage: D.del_edges([(0,1), (2,3), (4,5), (5,6)], False)
sage: list(D.iterator_edges(range(9), True))
[]
```

```
>>> from sage.all import *
>>> D = sage.graphs.base.sparse_graph.SparseGraphBackend(Integer(9))
>>> D.add_edges([(Integer(0),Integer(1)), (Integer(2),Integer(3)),␣
↪(Integer(4),Integer(5)), (Integer(5),Integer(6))], False)
>>> D.del_edges([(Integer(0),Integer(1)), (Integer(2),Integer(3)),␣
↪(Integer(4),Integer(5)), (Integer(5),Integer(6))], False)
>>> list(D.iterator_edges(range(Integer(9)), True))
[]
```

del_vertex(*v*)

Delete a vertex in `self`, failing silently if the vertex is not in the graph.

INPUT:

- `v` – vertex to be deleted

See also

- *del_vertices()* – delete a bunch of vertices from this graph

EXAMPLES:

```
sage: D = sage.graphs.base.dense_graph.DenseGraphBackend(9)
sage: D.del_vertex(0)
sage: D.has_vertex(0)
False
```

```
>>> from sage.all import *
>>> D = sage.graphs.base.dense_graph.DenseGraphBackend(Integer(9))
>>> D.del_vertex(Integer(0))
>>> D.has_vertex(Integer(0))
False
```

```
sage: S = sage.graphs.base.sparse_graph.SparseGraphBackend(9)
sage: S.del_vertex(0)
sage: S.has_vertex(0)
False
```

```
>>> from sage.all import *
>>> S = sage.graphs.base.sparse_graph.SparseGraphBackend(Integer(9))
>>> S.del_vertex(Integer(0))
```

(continues on next page)

(continued from previous page)

```
>>> S.has_vertex(Integer(0))
False
```

del_vertices(*vertices*)

Delete vertices from an iterable container.

INPUT:

- `vertices` – iterator of vertex labels

OUTPUT: same as for *del_vertex()*

See also

- *del_vertex()* – delete a vertex of this graph

EXAMPLES:

```
sage: import sage.graphs.base.dense_graph
sage: D = sage.graphs.base.dense_graph.DenseGraphBackend(9)
sage: D.del_vertices([7, 8])
sage: D.has_vertex(7)
False
sage: D.has_vertex(6)
True
```

```
>>> from sage.all import *
>>> import sage.graphs.base.dense_graph
>>> D = sage.graphs.base.dense_graph.DenseGraphBackend(Integer(9))
>>> D.del_vertices([Integer(7), Integer(8)])
>>> D.has_vertex(Integer(7))
False
>>> D.has_vertex(Integer(6))
True
```

```
sage: D = sage.graphs.base.sparse_graph.SparseGraphBackend(9)
sage: D.del_vertices([1, 2, 3])
sage: D.has_vertex(1)
False
sage: D.has_vertex(0)
True
```

```
>>> from sage.all import *
>>> D = sage.graphs.base.sparse_graph.SparseGraphBackend(Integer(9))
>>> D.del_vertices([Integer(1), Integer(2), Integer(3)])
>>> D.has_vertex(Integer(1))
False
>>> D.has_vertex(Integer(0))
True
```

depth_first_search(*v*, *reverse=False*, *ignore_direction=False*, *forbidden_vertices=None*)

Return a depth-first search from vertex `v`.

INPUT:

- `v` – a vertex from which to start the depth-first search
- `reverse` – boolean (default: `False`); this is only relevant to digraphs. If this is a digraph, consider the reversed graph in which the out-neighbors become the in-neighbors and vice versa.
- `ignore_direction` – boolean (default: `False`); this is only relevant to digraphs. If this is a digraph, ignore all orientations and consider the graph as undirected.
- `forbidden_vertices` – list (default: `None`); set of vertices to avoid during the search. The start vertex `v` cannot be in this set.

ALGORITHM:

Below is a general template for depth-first search.

- **Input:** A directed or undirected graph $G = (V, E)$ of order $n > 0$. A vertex s from which to start the search. The vertices are numbered from 1 to $n = |V|$, i.e. $V = \{1, 2, \ldots, n\}$.
- **Output:** A list D of distances of all vertices from s. A tree T rooted at s.

1. $S \leftarrow [s]$ # a stack of nodes to visit
2. $D \leftarrow [\infty, \infty, \ldots, \infty]$ # n copies of ∞
3. $D[s] \leftarrow 0$
4. $T \leftarrow [\,]$
5. while length$(S) > 0$ do
    1. $v \leftarrow$ pop(S)
    2. for each $w \in$ adj(v) do # for digraphs, use out-neighbor set oadj(v)
        1. if $D[w] = \infty$ then
            1. $D[w] \leftarrow D[v] + 1$
            2. push(S, w)
            3. append(T, vw)
6. return (D, T)

See also

- *breadth_first_search()* – breadth-first search for fast compiled graphs.
- *breadth_first_search* – breadth-first search for generic graphs.
- *depth_first_search* – depth-first search for generic graphs.

EXAMPLES:

Traversing the Petersen graph using depth-first search:

```
sage: G = graphs.PetersenGraph()
sage: list(G.depth_first_search(0))
[0, 5, 8, 6, 9, 7, 2, 3, 4, 1]
```

```
>>> from sage.all import *
>>> G = graphs.PetersenGraph()
>>> list(G.depth_first_search(Integer(0)))
[0, 5, 8, 6, 9, 7, 2, 3, 4, 1]
```

Visiting German cities using depth-first search:

```
sage: G = Graph({"Mannheim": ["Frankfurt","Karlsruhe"],
....: "Frankfurt": ["Mannheim","Wurzburg","Kassel"],
....: "Kassel": ["Frankfurt","Munchen"],
....: "Munchen": ["Kassel","Nurnberg","Augsburg"],
....: "Augsburg": ["Munchen","Karlsruhe"],
....: "Karlsruhe": ["Mannheim","Augsburg"],
....: "Wurzburg": ["Frankfurt","Erfurt","Nurnberg"],
....: "Nurnberg": ["Wurzburg","Stuttgart","Munchen"],
....: "Stuttgart": ["Nurnberg"], "Erfurt": ["Wurzburg"]})
sage: list(G.depth_first_search("Stuttgart"))
['Stuttgart', 'Nurnberg', ...]
```

```
>>> from sage.all import *
>>> G = Graph({"Mannheim": ["Frankfurt","Karlsruhe"],
... "Frankfurt": ["Mannheim","Wurzburg","Kassel"],
... "Kassel": ["Frankfurt","Munchen"],
... "Munchen": ["Kassel","Nurnberg","Augsburg"],
... "Augsburg": ["Munchen","Karlsruhe"],
... "Karlsruhe": ["Mannheim","Augsburg"],
... "Wurzburg": ["Frankfurt","Erfurt","Nurnberg"],
... "Nurnberg": ["Wurzburg","Stuttgart","Munchen"],
... "Stuttgart": ["Nurnberg"], "Erfurt": ["Wurzburg"]})
>>> list(G.depth_first_search("Stuttgart"))
['Stuttgart', 'Nurnberg', ...]
```

Avoiding some cities:

sage: list(G.depth_first_search("Stuttgart", ...: forbidden_vertices=["Frankfurt", "Munchen"])) ['Stuttgart', 'Nurnberg', 'Wurzburg', 'Erfurt']

has_vertex(*v*)

Check whether `v` is a vertex of `self`.

INPUT:

- `v` – any object

OUTPUT: `True` if `v` is a vertex of this graph; `False` otherwise

EXAMPLES:

```
sage: from sage.graphs.base.sparse_graph import SparseGraphBackend
sage: B = SparseGraphBackend(7)
sage: B.has_vertex(6)
True
sage: B.has_vertex(7)
False
```

```
>>> from sage.all import *
>>> from sage.graphs.base.sparse_graph import SparseGraphBackend
>>> B = SparseGraphBackend(Integer(7))
>>> B.has_vertex(Integer(6))
True
>>> B.has_vertex(Integer(7))
False
```

in_degree(*v*)

Return the in-degree of `v`.

INPUT:

- `v` – a vertex of the graph

EXAMPLES:

```
sage: D = DiGraph( { 0: [1,2,3], 1: [0,2], 2: [3], 3: [4], 4: [0,5], 5: [1] }␣
↪)
sage: D.out_degree(1)
2
```

```
>>> from sage.all import *
>>> D = DiGraph( { Integer(0): [Integer(1),Integer(2),Integer(3)],␣
↪Integer(1): [Integer(0),Integer(2)], Integer(2): [Integer(3)], Integer(3):␣
↪[Integer(4)], Integer(4): [Integer(0),Integer(5)], Integer(5): [Integer(1)]␣
↪} )
>>> D.out_degree(Integer(1))
2
```

is_connected(*forbidden_vertices=None*)

Check whether the graph is connected.

INPUT:

- `forbidden_vertices` – list (default: `None`); set of vertices to avoid during the search

EXAMPLES:

Petersen's graph is connected:

```
sage: DiGraph(graphs.PetersenGraph()).is_connected()
True
```

```
>>> from sage.all import *
>>> DiGraph(graphs.PetersenGraph()).is_connected()
True
```

While the disjoint union of two of them is not:

```
sage: DiGraph(2*graphs.PetersenGraph()).is_connected()
False
```

```
>>> from sage.all import *
>>> DiGraph(Integer(2)*graphs.PetersenGraph()).is_connected()
False
```

A graph with non-integer vertex labels:

```
sage: Graph(graphs.CubeGraph(3)).is_connected()
True
```

```
>>> from sage.all import *
>>> Graph(graphs.CubeGraph(Integer(3))).is_connected()
True
```

A graph with forbidden vertices:

```
sage: G = graphs.PathGraph(5)
sage: G._backend.is_connected()
True
sage: G._backend.is_connected(forbidden_vertices=[1])
False
sage: G._backend.is_connected(forbidden_vertices=[0, 1])
True
```

```
>>> from sage.all import *
>>> G = graphs.PathGraph(Integer(5))
>>> G._backend.is_connected()
True
>>> G._backend.is_connected(forbidden_vertices=[Integer(1)])
False
>>> G._backend.is_connected(forbidden_vertices=[Integer(0), Integer(1)])
True
```

is_directed_acyclic(*certificate=False*)

Check whether the graph is both directed and acyclic (possibly with a certificate)

INPUT:

- `certificate` – boolean (default: `False`); whether to return a certificate

OUTPUT:

When `certificate=False`, returns a boolean value. When `certificate=True` :

- If the graph is acyclic, returns a pair `(True, ordering)` where `ordering` is a list of the vertices such that `u` appears before `v` in `ordering` if `u, v` is an edge.
- Else, returns a pair `(False, cycle)` where `cycle` is a list of vertices representing a circuit in the graph.

ALGORITHM:

We pick a vertex at random, think hard and find out that if we are to remove the vertex from the graph we must remove all of its out-neighbors in the first place. So we put all of its out-neighbours in a stack, and repeat the same procedure with the vertex on top of the stack (when a vertex on top of the stack has no out-neighbors, we remove it immediately). Of course, for each vertex we only add its outneighbors to the end of the stack once : if for some reason the previous algorithm leads us to do it twice, it means we have found a circuit.

We keep track of the vertices whose out-neighborhood has been added to the stack once with a variable named `tried`.

There is no reason why the graph should be empty at the end of this procedure, so we run it again on the remaining vertices until none are left or a circuit is found.

Note

The graph is assumed to be directed. An exception is raised if it is not.

EXAMPLES:

At first, the following graph is acyclic:

```
sage: D = DiGraph({ 0:[1,2,3], 4:[2,5], 1:[8], 2:[7], 3:[7], 5:[6,7], 7:[8],␣
↪6:[9], 8:[10], 9:[10] })
sage: D.plot(layout='circular').show()                                    #␣
↪needs sage.plot
sage: D.is_directed_acyclic()
True
```

```
>>> from sage.all import *
>>> D = DiGraph({ Integer(0):[Integer(1),Integer(2),Integer(3)],␣
↪Integer(4):[Integer(2),Integer(5)], Integer(1):[Integer(8)],␣
↪Integer(2):[Integer(7)], Integer(3):[Integer(7)], Integer(5):[Integer(6),
↪Integer(7)], Integer(7):[Integer(8)], Integer(6):[Integer(9)],␣
↪Integer(8):[Integer(10)], Integer(9):[Integer(10)] })
>>> D.plot(layout='circular').show()                                      #␣
↪needs sage.plot
>>> D.is_directed_acyclic()
True
```

Adding an edge from 9 to 7 does not change it:

```
sage: D.add_edge(9,7)
sage: D.is_directed_acyclic()
True
```

```
>>> from sage.all import *
>>> D.add_edge(Integer(9),Integer(7))
>>> D.is_directed_acyclic()
True
```

We can obtain as a proof an ordering of the vertices such that u appears before v if uv is an edge of the graph:

```
sage: D.is_directed_acyclic(certificate = True)
(True, [4, 5, 6, 9, 0, 1, 2, 3, 7, 8, 10])
```

```
>>> from sage.all import *
>>> D.is_directed_acyclic(certificate = True)
(True, [4, 5, 6, 9, 0, 1, 2, 3, 7, 8, 10])
```

Adding an edge from 7 to 4, though, makes a difference:

```
sage: D.add_edge(7,4)
sage: D.is_directed_acyclic()
False
```

```
>>> from sage.all import *
>>> D.add_edge(Integer(7),Integer(4))
>>> D.is_directed_acyclic()
False
```

Indeed, it creates a circuit $7, 4, 5$:

```
sage: D.is_directed_acyclic(certificate = True)
(False, [7, 4, 5])
```

```
>>> from sage.all import *
>>> D.is_directed_acyclic(certificate = True)
(False, [7, 4, 5])
```

Checking acyclic graphs are indeed acyclic

```
sage: def random_acyclic(n, p):
....:   g = graphs.RandomGNP(n, p)
....:   h = DiGraph()
....:   h.add_edges([ ((u,v) if u<v else (v,u)) for u,v,_ in g.
↪edges(sort=True) ])
....:   return h
...
sage: all( random_acyclic(100, .2).is_directed_acyclic()     # long time
....:       for i in range(50))
True
```

```
>>> from sage.all import *
>>> def random_acyclic(n, p):
...   g = graphs.RandomGNP(n, p)
...   h = DiGraph()
...   h.add_edges([ ((u,v) if u<v else (v,u)) for u,v,_ in g.edges(sort=True)␣
↪])
...   return h
...
>>> all( random_acyclic(Integer(100), RealNumber('.2')).is_directed_acyclic()␣
↪   # long time
...       for i in range(Integer(50)))
True
```

is_strongly_connected()

Check whether the graph is strongly connected.

EXAMPLES:

The circuit on 3 vertices is obviously strongly connected:

```
sage: g = DiGraph({0: [1], 1: [2], 2: [0]})
sage: g.is_strongly_connected()
True
```

```
>>> from sage.all import *
>>> g = DiGraph({Integer(0): [Integer(1)], Integer(1): [Integer(2)],␣
```

(continues on next page)

(continued from previous page)

```
↪Integer(2): [Integer(0)]})
>>> g.is_strongly_connected()
True
```

But a transitive triangle is not:

```
sage: g = DiGraph({0: [1,2], 1: [2]})
sage: g.is_strongly_connected()
False
```

```
>>> from sage.all import *
>>> g = DiGraph({Integer(0): [Integer(1),Integer(2)], Integer(1):␣
↪[Integer(2)]})
>>> g.is_strongly_connected()
False
```

is_subgraph(*other*, *vertices*, *ignore_labels=False*)

Return whether the subgraph of `self` induced by `vertices` is a subgraph of `other`.

If `vertices` are the vertices of `self`, return whether `self` is a subgraph of `other`.

INPUT:

- `other` – a subclass of *CGraphBackend*
- `vertices` – a iterable over the vertex labels
- `ignore_labels` – boolean (default: `False`); whether to ignore the labels

EXAMPLES:

```
sage: G = sage.graphs.base.dense_graph.DenseGraphBackend(4, directed=True)
sage: H = sage.graphs.base.dense_graph.DenseGraphBackend(4, directed=True)
sage: G.add_edges([[0,1],[0,2],[0,3],[1,2]], True)
sage: H.add_edges([[0,1],[0,2],[0,3]], True)
sage: G.is_subgraph(H, range(4))
False
sage: H.is_subgraph(G, range(4))
True
sage: G.is_subgraph(H, [0,1,3])
True
```

```
>>> from sage.all import *
>>> G = sage.graphs.base.dense_graph.DenseGraphBackend(Integer(4),␣
↪directed=True)
>>> H = sage.graphs.base.dense_graph.DenseGraphBackend(Integer(4),␣
↪directed=True)
>>> G.add_edges([[Integer(0),Integer(1)],[Integer(0),Integer(2)],[Integer(0),
↪Integer(3)],[Integer(1),Integer(2)]], True)
>>> H.add_edges([[Integer(0),Integer(1)],[Integer(0),Integer(2)],[Integer(0),
↪Integer(3)]], True)
>>> G.is_subgraph(H, range(Integer(4)))
False
>>> H.is_subgraph(G, range(Integer(4)))
True
```

(continues on next page)

(continued from previous page)

```
>>> G.is_subgraph(H, [Integer(0),Integer(1),Integer(3)])
True
```

Ignore the labels or not:

```
sage: G = sage.graphs.base.sparse_graph.SparseGraphBackend(3, directed=True)
sage: G.multiple_edges(True)
sage: H = sage.graphs.base.sparse_graph.SparseGraphBackend(3, directed=True)
sage: H.multiple_edges(True)
sage: G.add_edges([[0,1,'a'], [0,1,'b'], [0,2,'c'], [0,2,'d'], [0,2,'e']],␣
↪True)
sage: H.add_edges([[0,1,'a'], [0,1,'foo'], [0,2,'c'], [0,2,'d'], [0,2,'e'],␣
↪[0,2,'e']], True)
sage: G.is_subgraph(H, range(3))
False
sage: G.is_subgraph(H, range(3), ignore_labels=True)
True
```

```
>>> from sage.all import *
>>> G = sage.graphs.base.sparse_graph.SparseGraphBackend(Integer(3),␣
↪directed=True)
>>> G.multiple_edges(True)
>>> H = sage.graphs.base.sparse_graph.SparseGraphBackend(Integer(3),␣
↪directed=True)
>>> H.multiple_edges(True)
>>> G.add_edges([[Integer(0),Integer(1),'a'], [Integer(0),Integer(1),'b'],␣
↪[Integer(0),Integer(2),'c'], [Integer(0),Integer(2),'d'], [Integer(0),
↪Integer(2),'e']], True)
>>> H.add_edges([[Integer(0),Integer(1),'a'], [Integer(0),Integer(1),'foo'],␣
↪[Integer(0),Integer(2),'c'], [Integer(0),Integer(2),'d'], [Integer(0),
↪Integer(2),'e'], [Integer(0),Integer(2),'e']], True)
>>> G.is_subgraph(H, range(Integer(3)))
False
>>> G.is_subgraph(H, range(Integer(3)), ignore_labels=True)
True
```

Multiplicities of edges are considered:

```
sage: G.is_subgraph(H, [0,2])
True
sage: H.is_subgraph(G, [0,2])
False
```

```
>>> from sage.all import *
>>> G.is_subgraph(H, [Integer(0),Integer(2)])
True
>>> H.is_subgraph(G, [Integer(0),Integer(2)])
False
```

iterator_edges(*vertices*, *labels*)

Iterate over the edges incident to a sequence of vertices.

Edges are assumed to be undirected.

Warning

This will try to sort the two ends of every edge.

INPUT:

- `vertices` – list of vertex labels
- `labels` – boolean, whether to return labels as well

EXAMPLES:

```
sage: G = sage.graphs.base.sparse_graph.SparseGraphBackend(9)
sage: G.add_edge(1,2,3,False)
sage: list(G.iterator_edges(range(9), False))
[(1, 2)]
sage: list(G.iterator_edges(range(9), True))
[(1, 2, 3)]
```

```
>>> from sage.all import *
>>> G = sage.graphs.base.sparse_graph.SparseGraphBackend(Integer(9))
>>> G.add_edge(Integer(1),Integer(2),Integer(3),False)
>>> list(G.iterator_edges(range(Integer(9)), False))
[(1, 2)]
>>> list(G.iterator_edges(range(Integer(9)), True))
[(1, 2, 3)]
```

iterator_in_edges(*vertices*, *labels*)

Iterate over the incoming edges incident to a sequence of vertices.

INPUT:

- `vertices` – list of vertex labels
- `labels` – boolean, whether to return labels as well

EXAMPLES:

```
sage: G = sage.graphs.base.sparse_graph.SparseGraphBackend(9)
sage: G.add_edge(1,2,3,True)
sage: list(G.iterator_in_edges([1], False))
[]
sage: list(G.iterator_in_edges([2], False))
[(1, 2)]
sage: list(G.iterator_in_edges([2], True))
[(1, 2, 3)]
```

```
>>> from sage.all import *
>>> G = sage.graphs.base.sparse_graph.SparseGraphBackend(Integer(9))
>>> G.add_edge(Integer(1),Integer(2),Integer(3),True)
>>> list(G.iterator_in_edges([Integer(1)], False))
[]
>>> list(G.iterator_in_edges([Integer(2)], False))
[(1, 2)]
>>> list(G.iterator_in_edges([Integer(2)], True))
[(1, 2, 3)]
```

iterator_in_nbrs(*v*)

Iterate over the incoming neighbors of v.

INPUT:

- v – a vertex of this graph

OUTPUT: an iterator over the in-neighbors of the vertex v

See also

- *iterator_nbrs()* – returns an iterator over the neighbors of a vertex
- *iterator_out_nbrs()* – returns an iterator over the out-neighbors of a vertex

EXAMPLES:

```
sage: P = DiGraph(graphs.PetersenGraph().to_directed())
sage: list(P._backend.iterator_in_nbrs(0))
[1, 4, 5]
```

```
>>> from sage.all import *
>>> P = DiGraph(graphs.PetersenGraph().to_directed())
>>> list(P._backend.iterator_in_nbrs(Integer(0)))
[1, 4, 5]
```

iterator_nbrs(*v*)

Iterate over the neighbors of v.

INPUT:

- v – a vertex of this graph

OUTPUT: an iterator over the neighbors the vertex v

See also

- *iterator_in_nbrs()* – returns an iterator over the in-neighbors of a vertex
- *iterator_out_nbrs()* – returns an iterator over the out-neighbors of a vertex
- *iterator_verts()* – returns an iterator over a given set of vertices

EXAMPLES:

```
sage: P = Graph(graphs.PetersenGraph())
sage: list(P._backend.iterator_nbrs(0))
[1, 4, 5]
sage: Q = DiGraph(P)
sage: list(Q._backend.iterator_nbrs(0))
[1, 4, 5]
```

```
>>> from sage.all import *
>>> P = Graph(graphs.PetersenGraph())
```

(continues on next page)

(continued from previous page)

```
>>> list(P._backend.iterator_nbrs(Integer(0)))
[1, 4, 5]
>>> Q = DiGraph(P)
>>> list(Q._backend.iterator_nbrs(Integer(0)))
[1, 4, 5]
```

iterator_out_edges(*vertices*, *labels*)

Iterate over the outbound edges incident to a sequence of vertices.

INPUT:

- `vertices` – list of vertex labels
- `labels` – boolean, whether to return labels as well

EXAMPLES:

```
sage: G = sage.graphs.base.sparse_graph.SparseGraphBackend(9)
sage: G.add_edge(1,2,3,True)
sage: list(G.iterator_out_edges([2], False))
[]
sage: list(G.iterator_out_edges([1], False))
[(1, 2)]
sage: list(G.iterator_out_edges([1], True))
[(1, 2, 3)]
```

```
>>> from sage.all import *
>>> G = sage.graphs.base.sparse_graph.SparseGraphBackend(Integer(9))
>>> G.add_edge(Integer(1),Integer(2),Integer(3),True)
>>> list(G.iterator_out_edges([Integer(2)], False))
[]
>>> list(G.iterator_out_edges([Integer(1)], False))
[(1, 2)]
>>> list(G.iterator_out_edges([Integer(1)], True))
[(1, 2, 3)]
```

iterator_out_nbrs(*v*)

Iterate over the outgoing neighbors of `v`.

INPUT:

- `v` – a vertex of this graph

OUTPUT: an iterator over the out-neighbors of the vertex `v`

See also

- *`iterator_nbrs()`* – returns an iterator over the neighbors of a vertex
- *`iterator_in_nbrs()`* – returns an iterator over the in-neighbors of a vertex

EXAMPLES:

```
sage: P = DiGraph(graphs.PetersenGraph().to_directed())
sage: list(P._backend.iterator_out_nbrs(0))
[1, 4, 5]
```

```
>>> from sage.all import *
>>> P = DiGraph(graphs.PetersenGraph().to_directed())
>>> list(P._backend.iterator_out_nbrs(Integer(0)))
[1, 4, 5]
```

iterator_unsorted_edges(*vertices*, *labels*)

Iterate over the edges incident to a sequence of vertices.

Edges are assumed to be undirected.

This does not sort the ends of each edge.

INPUT:

- `vertices` – list of vertex labels
- `labels` – boolean, whether to return labels as well

EXAMPLES:

```
sage: G = sage.graphs.base.sparse_graph.SparseGraphBackend(9)
sage: G.add_edge(1,2,3,False)
sage: list(G.iterator_unsorted_edges(range(9), False))
[(2, 1)]
sage: list(G.iterator_unsorted_edges(range(9), True))
[(2, 1, 3)]
```

```
>>> from sage.all import *
>>> G = sage.graphs.base.sparse_graph.SparseGraphBackend(Integer(9))
>>> G.add_edge(Integer(1),Integer(2),Integer(3),False)
>>> list(G.iterator_unsorted_edges(range(Integer(9)), False))
[(2, 1)]
>>> list(G.iterator_unsorted_edges(range(Integer(9)), True))
[(2, 1, 3)]
```

iterator_verts(*verts=None*)

Iterate over the vertices of `self` intersected with `verts`.

INPUT:

- `verts` – an iterable container of objects (default: `None`)

OUTPUT:

- If `verts=None`, return an iterator over all vertices of this graph
- If `verts` is a single vertex of the graph, treat it as the container `[verts]`
- If `verts` is a iterable container of vertices, find the intersection of `verts` with the vertex set of this graph and return an iterator over the resulting intersection

See also

- *iterator_nbrs()* – returns an iterator over the neighbors of a vertex.

EXAMPLES:

```
sage: P = Graph(graphs.PetersenGraph())
sage: list(P._backend.iterator_verts(P))
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
sage: list(P._backend.iterator_verts())
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
sage: list(P._backend.iterator_verts([1, 2, 3]))
[1, 2, 3]
sage: list(P._backend.iterator_verts([1, 2, 10]))
[1, 2]
```

```
>>> from sage.all import *
>>> P = Graph(graphs.PetersenGraph())
>>> list(P._backend.iterator_verts(P))
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
>>> list(P._backend.iterator_verts())
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
>>> list(P._backend.iterator_verts([Integer(1), Integer(2), Integer(3)]))
[1, 2, 3]
>>> list(P._backend.iterator_verts([Integer(1), Integer(2), Integer(10)]))
[1, 2]
```

loops(*new=None*)

Check whether loops are allowed in this graph.

INPUT:

- `new` – boolean (default: `None`); to set or `None` to get

OUTPUT:

- If `new=None`, return `True` if this graph allows self-loops or `False` if self-loops are not allowed
- If `new` is a boolean, set the self-loop permission of this graph according to the boolean value of `new`

EXAMPLES:

```
sage: G = Graph()
sage: G._backend.loops()
False
sage: G._backend.loops(True)
sage: G._backend.loops()
True
```

```
>>> from sage.all import *
>>> G = Graph()
>>> G._backend.loops()
False
>>> G._backend.loops(True)
>>> G._backend.loops()
True
```

n_edges(*directed*)

Return the number of edges in `self`.

INPUT:

- `directed` – boolean; whether to count `(u, v)` and `(v, u)` as one or two edges

OUTPUT:

- If `directed=True`, counts the number of directed edges in this graph. Otherwise, return the size of this graph.

See also

- *n_vertices()* – return the order of this graph.

EXAMPLES:

```
sage: G = Graph(graphs.PetersenGraph())
sage: G._backend.n_edges(False)
15
```

```
>>> from sage.all import *
>>> G = Graph(graphs.PetersenGraph())
>>> G._backend.n_edges(False)
15
```

n_vertices()

Return the number of vertices in `self`.

OUTPUT: the order of this graph

See also

- *n_edges()* – return the number of (directed) edges in this graph.

EXAMPLES:

```
sage: G = Graph(graphs.PetersenGraph())
sage: G._backend.n_vertices()
10
```

```
>>> from sage.all import *
>>> G = Graph(graphs.PetersenGraph())
>>> G._backend.n_vertices()
10
```

num_edges(*directed*)

alias of *n_edges()*.

num_verts()

alias of *n_vertices()*.

out_degree(*v*)

Return the out-degree of `v`.

INPUT:

- `v` – a vertex of the graph

EXAMPLES:

```
sage: D = DiGraph( { 0: [1,2,3], 1: [0,2], 2: [3], 3: [4], 4: [0,5], 5: [1] }␣
↪)
sage: D.out_degree(1)
2
```

```
>>> from sage.all import *
>>> D = DiGraph( { Integer(0): [Integer(1),Integer(2),Integer(3)],␣
↪Integer(1): [Integer(0),Integer(2)], Integer(2): [Integer(3)], Integer(3):␣
↪[Integer(4)], Integer(4): [Integer(0),Integer(5)], Integer(5): [Integer(1)]␣
↪} )
>>> D.out_degree(Integer(1))
2
```

relabel(*perm*, *directed*)

Relabel the graph according to `perm`.

INPUT:

- `perm` – anything which represents a permutation as `v --> perm[v]`, for example a dict or a list
- `directed` – ignored (this is here for compatibility with other backends)

EXAMPLES:

```
sage: G = Graph(graphs.PetersenGraph())
sage: G._backend.relabel(range(9,-1,-1), False)
sage: G.edges(sort=True)
[(0, 2, None),
 (0, 3, None),
 (0, 5, None),
 (1, 3, None),
 (1, 4, None),
 (1, 6, None),
 (2, 4, None),
 (2, 7, None),
 (3, 8, None),
 (4, 9, None),
 (5, 6, None),
 (5, 9, None),
 (6, 7, None),
 (7, 8, None),
 (8, 9, None)]
```

```
>>> from sage.all import *
>>> G = Graph(graphs.PetersenGraph())
>>> G._backend.relabel(range(Integer(9),-Integer(1),-Integer(1)), False)
>>> G.edges(sort=True)
```

(continues on next page)

(continued from previous page)

```
[(0, 2, None),
 (0, 3, None),
 (0, 5, None),
 (1, 3, None),
 (1, 4, None),
 (1, 6, None),
 (2, 4, None),
 (2, 7, None),
 (3, 8, None),
 (4, 9, None),
 (5, 6, None),
 (5, 9, None),
 (6, 7, None),
 (7, 8, None),
 (8, 9, None)]
```

shortest_path(*x*, *y*, *distance_flag=False*)

Return the shortest path or distance from `x` to `y`.

INPUT:

- `x` – the starting vertex in the shortest path from `x` to `y`
- `y` – the end vertex in the shortest path from `x` to `y`
- `distance_flag` – boolean (default: `False`); when set to `True`, the shortest path distance from `x` to `y` is returned instead of the path

OUTPUT:

- A list of vertices in the shortest path from `x` to `y` or distance from `x` to `y` is returned depending upon the value of parameter `distance_flag`

EXAMPLES:

```
sage: G = Graph(graphs.PetersenGraph())
sage: G.shortest_path(0, 1)
[0, 1]
sage: G.shortest_path_length(0, 1)
1
```

```
>>> from sage.all import *
>>> G = Graph(graphs.PetersenGraph())
>>> G.shortest_path(Integer(0), Integer(1))
[0, 1]
>>> G.shortest_path_length(Integer(0), Integer(1))
1
```

shortest_path_all_vertices(*v*, *cutoff=None*, *distance_flag=False*)

Return for each reachable vertex `u` a shortest `v`–`u` path or distance from `v` to `u`.

INPUT:

- `v` – a starting vertex in the shortest path
- `cutoff` – integer (default: `None`); maximal distance of returned paths (longer paths will not be returned), ignored when set to `None`

- `distance_flag` – boolean (default: `False`); when set to `True`, each vertex `u` connected to `v` is mapped to shortest path distance from `v` to `u` instead of the shortest path in the output dictionary.

OUTPUT:

- A dictionary which maps each vertex `u` connected to `v` to the shortest path list or distance from `v` to `u` depending upon the value of parameter `distance_flag`

Note

The weight of edges is not taken into account.

ALGORITHM:

This is just a breadth-first search.

EXAMPLES:

On the Petersen Graph:

```
sage: g = graphs.PetersenGraph()
sage: paths = g._backend.shortest_path_all_vertices(0)
sage: all((not paths[v] or len(paths[v])-1 == g.distance(0,v)) for v in g)
True
sage: g._backend.shortest_path_all_vertices(0, distance_flag=True)
{0: 0, 1: 1, 2: 2, 3: 2, 4: 1, 5: 1, 6: 2, 7: 2, 8: 2, 9: 2}
```

```
>>> from sage.all import *
>>> g = graphs.PetersenGraph()
>>> paths = g._backend.shortest_path_all_vertices(Integer(0))
>>> all((not paths[v] or len(paths[v])-Integer(1) == g.distance(Integer(0),
↪v)) for v in g)
True
>>> g._backend.shortest_path_all_vertices(Integer(0), distance_flag=True)
{0: 0, 1: 1, 2: 2, 3: 2, 4: 1, 5: 1, 6: 2, 7: 2, 8: 2, 9: 2}
```

On a disconnected graph:

```
sage: g = 2 * graphs.RandomGNP(20, .3)
sage: paths = g._backend.shortest_path_all_vertices(0)
sage: all((v not in paths and g.distance(0, v) == +Infinity) or len(paths[v])␣
↪- 1 == g.distance(0, v) for v in g)
True
```

```
>>> from sage.all import *
>>> g = Integer(2) * graphs.RandomGNP(Integer(20), RealNumber('.3'))
>>> paths = g._backend.shortest_path_all_vertices(Integer(0))
>>> all((v not in paths and g.distance(Integer(0), v) == +Infinity) or␣
↪len(paths[v]) - Integer(1) == g.distance(Integer(0), v) for v in g)
True
```

shortest_path_special(*x*, *y*, *exclude_vertices=None*, *exclude_edges=None*, *distance_flag=False*)

Return the shortest path or distance from `x` to `y`.

This method is an extension of *shortest_path()* method enabling to exclude vertices and/or edges from the search for the shortest path between `x` and `y`.

INPUT:

- `x` – the starting vertex in the shortest path from `x` to `y`
- `y` – the end vertex in the shortest path from `x` to `y`
- `exclude_vertices` – iterable container (default: `None`); iterable of vertices to exclude from the graph while calculating the shortest path from `x` to `y`
- `exclude_edges` – iterable container (default: `None`); iterable of edges to exclude from the graph while calculating the shortest path from `x` to `y`
- `distance_flag` – boolean (default: `False`); when set to `True`, the shortest path distance from `x` to `y` is returned instead of the path

OUTPUT:

- A list of vertices in the shortest path from `x` to `y` or distance from `x` to `y` is returned depending upon the value of parameter `distance_flag`

EXAMPLES:

```
sage: G = Graph([(1, 2), (2, 3), (3, 4), (1, 5), (5, 6), (6, 7), (7, 4)])
sage: G._backend.shortest_path_special(1, 4)
[1, 2, 3, 4]
sage: G._backend.shortest_path_special(1, 4, exclude_vertices=[5,7])
[1, 2, 3, 4]
sage: G._backend.shortest_path_special(1, 4, exclude_vertices=[2, 3])
[1, 5, 6, 7, 4]
sage: G._backend.shortest_path_special(1, 4, exclude_vertices=[2], exclude_
↪edges=[(5, 6)])
[]
sage: G._backend.shortest_path_special(1, 4, exclude_vertices=[2], exclude_
↪edges=[(2, 3)])
[1, 5, 6, 7, 4]
```

```
>>> from sage.all import *
>>> G = Graph([(Integer(1), Integer(2)), (Integer(2), Integer(3)),␣
↪(Integer(3), Integer(4)), (Integer(1), Integer(5)), (Integer(5),␣
↪Integer(6)), (Integer(6), Integer(7)), (Integer(7), Integer(4))])
>>> G._backend.shortest_path_special(Integer(1), Integer(4))
[1, 2, 3, 4]
>>> G._backend.shortest_path_special(Integer(1), Integer(4), exclude_
↪vertices=[Integer(5),Integer(7)])
[1, 2, 3, 4]
>>> G._backend.shortest_path_special(Integer(1), Integer(4), exclude_
↪vertices=[Integer(2), Integer(3)])
[1, 5, 6, 7, 4]
>>> G._backend.shortest_path_special(Integer(1), Integer(4), exclude_
↪vertices=[Integer(2)], exclude_edges=[(Integer(5), Integer(6))])
[]
>>> G._backend.shortest_path_special(Integer(1), Integer(4), exclude_
↪vertices=[Integer(2)], exclude_edges=[(Integer(2), Integer(3))])
[1, 5, 6, 7, 4]
```

shortest_path_to_set(*source*, *targets*, *by_weight=False*, *edge_weight=None*, *exclude_vertices=None*, *report_weight=False*)

Return the shortest path from `source` to any vertex in `targets`.

INPUT:

- source – the starting vertex.
- targets – iterable container; the set of end vertices.
- edge_weight – dictionary (default: None); a dictionary that takes as input an edge (u, v) and outputs its weight. If not None, by_weight is automatically set to True. If None and by_weight is True, we use the edge label l as a weight.
- by_weight – boolean (default: False); if True, the edges in the graph are weighted, otherwise all edges have weight 1.
- exclude_vertices – iterable container (default: None); iterable of vertices to exclude from the graph while calculating the shortest path from source to any vertex in targets.
- report_weight – boolean (default: False); if False, just a path is returned. Otherwise a tuple of path length and path is returned.

OUTPUT:

- A list of vertices in the shortest path from source to any vertex in targets or a tuple of path lengh and path is returned depending upon the value of parameter report_weight.

EXAMPLES:

```
sage: g = Graph([(1, 2, 10), (1, 3, 20), (1, 4, 30)])
sage: g._backend.shortest_path_to_set(1, {3, 4}, by_weight=True)
[1, 3]
sage: g = Graph([(1, 2, 10), (2, 3, 10), (1, 4, 20), (4, 5, 20), (1, 6, 30),␣
↪(6, 7, 30)])
sage: g._backend.shortest_path_to_set(1, {5, 7}, by_weight=True, exclude_
↪vertices=[4], report_weight=True)
(60.0, [1, 6, 7])
```

```
>>> from sage.all import *
>>> g = Graph([(Integer(1), Integer(2), Integer(10)), (Integer(1), Integer(3),
↪ Integer(20)), (Integer(1), Integer(4), Integer(30))])
>>> g._backend.shortest_path_to_set(Integer(1), {Integer(3), Integer(4)}, by_
↪weight=True)
[1, 3]
>>> g = Graph([(Integer(1), Integer(2), Integer(10)), (Integer(2), Integer(3),
↪ Integer(10)), (Integer(1), Integer(4), Integer(20)), (Integer(4),␣
↪Integer(5), Integer(20)), (Integer(1), Integer(6), Integer(30)),␣
↪(Integer(6), Integer(7), Integer(30))])
>>> g._backend.shortest_path_to_set(Integer(1), {Integer(5), Integer(7)}, by_
↪weight=True, exclude_vertices=[Integer(4)], report_weight=True)
(60.0, [1, 6, 7])
```

strongly_connected_component_containing_vertex(*v*)

Return the strongly connected component containing the given vertex.

INPUT:

- v – a vertex

EXAMPLES:

The digraph obtained from the PetersenGraph has an unique strongly connected component:

```
sage: g = DiGraph(graphs.PetersenGraph())
sage: g.strongly_connected_component_containing_vertex(0)
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

```
>>> from sage.all import *
>>> g = DiGraph(graphs.PetersenGraph())
>>> g.strongly_connected_component_containing_vertex(Integer(0))
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

In the Butterfly DiGraph, each vertex is a strongly connected component:

```
sage: g = digraphs.ButterflyGraph(3)
sage: all([v] == g.strongly_connected_component_containing_vertex(v) for v in␣
↪g)
True
```

```
>>> from sage.all import *
>>> g = digraphs.ButterflyGraph(Integer(3))
>>> all([v] == g.strongly_connected_component_containing_vertex(v) for v in g)
True
```

subgraph_given_vertices(*other*, *vertices*)

Initialize `other` to be the subgraph of `self` with given vertices.

INPUT:

- `other` – a (mutable) subclass of *CGraphBackend*
- `vertices` – list of vertex labels

Note

`other` is assumed to be the empty graph.

EXAMPLES:

Make a dense copy:

```
sage: G = sage.graphs.base.dense_graph.DenseGraphBackend(9, directed=True)
sage: G.loops(True)
sage: G.add_edges([[0,1], [1,2], [2,3], [3,4], [4,5], [5,6], [7,8], [3,3]],␣
↪True)
sage: H = sage.graphs.base.dense_graph.DenseGraphBackend(0, directed=True)
sage: H.loops(True)
sage: G.subgraph_given_vertices(H, range(9))
sage: list(H.iterator_out_edges(list(range(9)), False)) == list(G.iterator_
↪out_edges(list(range(9)), False))
True
```

```
>>> from sage.all import *
>>> G = sage.graphs.base.dense_graph.DenseGraphBackend(Integer(9),␣
↪directed=True)
>>> G.loops(True)
```

(continues on next page)

(continued from previous page)

```
>>> G.add_edges([[Integer(0),Integer(1)], [Integer(1),Integer(2)],␣
↪[Integer(2),Integer(3)], [Integer(3),Integer(4)], [Integer(4),Integer(5)],␣
↪[Integer(5),Integer(6)], [Integer(7),Integer(8)], [Integer(3),Integer(3)]],␣
↪True)
>>> H = sage.graphs.base.dense_graph.DenseGraphBackend(Integer(0),␣
↪directed=True)
>>> H.loops(True)
>>> G.subgraph_given_vertices(H, range(Integer(9)))
>>> list(H.iterator_out_edges(list(range(Integer(9))), False)) == list(G.
↪iterator_out_edges(list(range(Integer(9))), False))
True
```

Make a sparse copy:

```
sage: H = sage.graphs.base.sparse_graph.SparseGraphBackend(0, directed=True)
sage: H.loops(True)
sage: G.subgraph_given_vertices(H, range(9))
sage: sorted(list(H.iterator_out_edges(list(range(9)), False))) ==␣
↪sorted(list(G.iterator_out_edges(list(range(9)), False)))
True
```

```
>>> from sage.all import *
>>> H = sage.graphs.base.sparse_graph.SparseGraphBackend(Integer(0),␣
↪directed=True)
>>> H.loops(True)
>>> G.subgraph_given_vertices(H, range(Integer(9)))
>>> sorted(list(H.iterator_out_edges(list(range(Integer(9))), False))) ==␣
↪sorted(list(G.iterator_out_edges(list(range(Integer(9))), False)))
True
```

Initialize a proper subgraph:

```
sage: H = sage.graphs.base.sparse_graph.SparseGraphBackend(0, directed=True)
sage: H.loops(True)
sage: G.subgraph_given_vertices(H, [2,3,4,5])
sage: list(H.iterator_out_edges(list(range(9)), False))
[(2, 3), (3, 3), (3, 4), (4, 5)]
```

```
>>> from sage.all import *
>>> H = sage.graphs.base.sparse_graph.SparseGraphBackend(Integer(0),␣
↪directed=True)
>>> H.loops(True)
>>> G.subgraph_given_vertices(H, [Integer(2),Integer(3),Integer(4),
↪Integer(5)])
>>> list(H.iterator_out_edges(list(range(Integer(9))), False))
[(2, 3), (3, 3), (3, 4), (4, 5)]
```

Loops are removed, if the other graph does not allow loops:

```
sage: H = sage.graphs.base.sparse_graph.SparseGraphBackend(0, directed=True)
sage: H.loops(False)
sage: G.subgraph_given_vertices(H, [2,3,4,5])
```

(continues on next page)

(continued from previous page)

```
sage: list(H.iterator_out_edges(list(range(9)), False))
[(2, 3), (3, 4), (4, 5)]
```

```
>>> from sage.all import *
>>> H = sage.graphs.base.sparse_graph.SparseGraphBackend(Integer(0),␣
↪directed=True)
>>> H.loops(False)
>>> G.subgraph_given_vertices(H, [Integer(2),Integer(3),Integer(4),
↪Integer(5)])
>>> list(H.iterator_out_edges(list(range(Integer(9))), False))
[(2, 3), (3, 4), (4, 5)]
```

Multiple edges and labels are copied:

```
sage: G = sage.graphs.base.sparse_graph.SparseGraphBackend(4, directed=False)
sage: G.multiple_edges(True)
sage: G.add_edges([[0,1,'a'], [1,2,'b'], [2,3,'c'], [0,1,'d']], False)
sage: H = sage.graphs.base.sparse_graph.SparseGraphBackend(0, directed=False)
sage: H.multiple_edges(True)
sage: G.subgraph_given_vertices(H, [0,1,2])
sage: list(H.iterator_edges(list(range(4)), True))
[(0, 1, 'a'), (0, 1, 'd'), (1, 2, 'b')]
```

```
>>> from sage.all import *
>>> G = sage.graphs.base.sparse_graph.SparseGraphBackend(Integer(4),␣
↪directed=False)
>>> G.multiple_edges(True)
>>> G.add_edges([[Integer(0),Integer(1),'a'], [Integer(1),Integer(2),'b'],␣
↪[Integer(2),Integer(3),'c'], [Integer(0),Integer(1),'d']], False)
>>> H = sage.graphs.base.sparse_graph.SparseGraphBackend(Integer(0),␣
↪directed=False)
>>> H.multiple_edges(True)
>>> G.subgraph_given_vertices(H, [Integer(0),Integer(1),Integer(2)])
>>> list(H.iterator_edges(list(range(Integer(4))), True))
[(0, 1, 'a'), (0, 1, 'd'), (1, 2, 'b')]
```

Multiple edges are removed, if the other graph does not allow them:

```
sage: H = sage.graphs.base.sparse_graph.SparseGraphBackend(0, directed=False)
sage: H.multiple_edges(False)
sage: G.subgraph_given_vertices(H, [0,1,2])
sage: list(H.iterator_edges(list(range(4)), True))
[(0, 1, 'd'), (1, 2, 'b')]
```

```
>>> from sage.all import *
>>> H = sage.graphs.base.sparse_graph.SparseGraphBackend(Integer(0),␣
↪directed=False)
>>> H.multiple_edges(False)
>>> G.subgraph_given_vertices(H, [Integer(0),Integer(1),Integer(2)])
>>> list(H.iterator_edges(list(range(Integer(4))), True))
[(0, 1, 'd'), (1, 2, 'b')]
```

Labels are removed, if the other graph does not allow them:

```
sage: H = sage.graphs.base.dense_graph.DenseGraphBackend(0, directed=False)
sage: G.subgraph_given_vertices(H, [0,1,2])
sage: list(H.iterator_edges(list(range(4)), True))
[(0, 1, None), (1, 2, None)]
```

```
>>> from sage.all import *
>>> H = sage.graphs.base.dense_graph.DenseGraphBackend(Integer(0),␣
↪directed=False)
>>> G.subgraph_given_vertices(H, [Integer(0),Integer(1),Integer(2)])
>>> list(H.iterator_edges(list(range(Integer(4))), True))
[(0, 1, None), (1, 2, None)]
```

A directed subgraph of an undirected graph is taken by initializing with edges in both directions:

```
sage: G = sage.graphs.base.sparse_graph.SparseGraphBackend(4, directed=True)
sage: G.loops(True)
sage: G.multiple_edges(True)
sage: G.add_edges([[0,1,'a'], [1,2,'b'], [2,3,'c'], [0,1,'d'], [2,2,'e']],␣
↪False)
sage: H = sage.graphs.base.sparse_graph.SparseGraphBackend(0, directed=True)
sage: H.multiple_edges(True)
sage: H.loops(True)
sage: G.subgraph_given_vertices(H, [0,1,2])
sage: list(H.iterator_out_edges(list(range(4)), True))
[(0, 1, 'a'),
 (0, 1, 'd'),
 (1, 0, 'a'),
 (1, 0, 'd'),
 (1, 2, 'b'),
 (2, 1, 'b'),
 (2, 2, 'e')]
```

```
>>> from sage.all import *
>>> G = sage.graphs.base.sparse_graph.SparseGraphBackend(Integer(4),␣
↪directed=True)
>>> G.loops(True)
>>> G.multiple_edges(True)
>>> G.add_edges([[Integer(0),Integer(1),'a'], [Integer(1),Integer(2),'b'],␣
↪[Integer(2),Integer(3),'c'], [Integer(0),Integer(1),'d'], [Integer(2),
↪Integer(2),'e']], False)
>>> H = sage.graphs.base.sparse_graph.SparseGraphBackend(Integer(0),␣
↪directed=True)
>>> H.multiple_edges(True)
>>> H.loops(True)
>>> G.subgraph_given_vertices(H, [Integer(0),Integer(1),Integer(2)])
>>> list(H.iterator_out_edges(list(range(Integer(4))), True))
[(0, 1, 'a'),
 (0, 1, 'd'),
 (1, 0, 'a'),
 (1, 0, 'd'),
 (1, 2, 'b'),
```

(continues on next page)

(continued from previous page)

```
 (2, 1, 'b'),
 (2, 2, 'e')]
```

An undirected subgraph of a directeed graph is not defined:

```
sage: G = sage.graphs.base.sparse_graph.SparseGraphBackend(4, directed=True)
sage: G.add_edges([[0,1,'a'], [1,2,'b'], [2,3,'c']], False)
sage: H = sage.graphs.base.sparse_graph.SparseGraphBackend(0, directed=False)
sage: G.subgraph_given_vertices(H, [0,1,2])
Traceback (most recent call last):
...
ValueError: cannot obtain an undirected subgraph of a directed graph
```

```
>>> from sage.all import *
>>> G = sage.graphs.base.sparse_graph.SparseGraphBackend(Integer(4),␣
↪directed=True)
>>> G.add_edges([[Integer(0),Integer(1),'a'], [Integer(1),Integer(2),'b'],␣
↪[Integer(2),Integer(3),'c']], False)
>>> H = sage.graphs.base.sparse_graph.SparseGraphBackend(Integer(0),␣
↪directed=False)
>>> G.subgraph_given_vertices(H, [Integer(0),Integer(1),Integer(2)])
Traceback (most recent call last):
...
ValueError: cannot obtain an undirected subgraph of a directed graph
```

class sage.graphs.base.c_graph.**Search_iterator**

Bases: object

An iterator for traversing a (di)graph.

This class is commonly used to perform a depth-first or breadth-first search. The class does not build all at once in memory the whole list of visited vertices. The class maintains the following variables:

- `graph` – a graph whose vertices are to be iterated over
- `direction` – integer; this determines the position at which vertices to be visited are removed from the list. For breadth-first search (BFS), element removal follow a first-in first-out (FIFO) protocol, as signified by the value `direction=0`. We use a queue to maintain the list of vertices to visit in this case. For depth-first search (DFS), element removal follow a last-in first-out (LIFO) protocol, as signified by the value `direction=-1`. In this case, we use a stack to maintain the list of vertices to visit.
- `stack` – list of vertices to visit, used only when `direction=-1`
- `queue` – a queue of vertices to visit, used only when `direction=0`
- `seen` – list of vertices that are already visited
- `test_out` – boolean; whether we want to consider the out-neighbors of the graph to be traversed. For undirected graphs, we consider both the in- and out-neighbors. However, for digraphs we only traverse along out-neighbors.
- `test_in` – boolean; whether we want to consider the in-neighbors of the graph to be traversed. For undirected graphs, we consider both the in- and out-neighbors.

EXAMPLES:

```
sage: g = graphs.PetersenGraph()
sage: list(g.breadth_first_search(0))
[0, 1, 4, 5, 2, 6, 3, 9, 7, 8]
```

```
>>> from sage.all import *
>>> g = graphs.PetersenGraph()
>>> list(g.breadth_first_search(Integer(0)))
[0, 1, 4, 5, 2, 6, 3, 9, 7, 8]
```

3.3 Fast sparse graphs

For an overview of graph data structures in sage, see *overview*.

3.3.1 Usage Introduction

```
sage: from sage.graphs.base.sparse_graph import SparseGraph
```

```
>>> from sage.all import *
>>> from sage.graphs.base.sparse_graph import SparseGraph
```

Sparse graphs are initialized as follows:

```
sage: S = SparseGraph(nverts = 10, expected_degree = 3, extra_vertices = 10)
```

```
>>> from sage.all import *
>>> S = SparseGraph(nverts = Integer(10), expected_degree = Integer(3), extra_
↪vertices = Integer(10))
```

This example initializes a sparse graph with room for twenty vertices, the first ten of which are in the graph. In general, the first `nverts` are "active." For example, see that 9 is already in the graph:

```
sage: S.verts()
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
sage: S.add_vertex(9)
9
sage: S.verts()
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

```
>>> from sage.all import *
>>> S.verts()
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
>>> S.add_vertex(Integer(9))
9
>>> S.verts()
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

But 10 is not, until we add it:

```
sage: S.add_vertex(10)
10
```

(continues on next page)

(continued from previous page)

```
sage: S.verts()
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

```
>>> from sage.all import *
>>> S.add_vertex(Integer(10))
10
>>> S.verts()
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

You can begin working with unlabeled arcs right away as follows:

```
sage: S.add_arc(0,1)
sage: S.add_arc(1,2)
sage: S.add_arc(1,0)
sage: S.has_arc(7,3)
False
sage: S.has_arc(0,1)
True
sage: S.in_neighbors(1)
[0]
sage: S.out_neighbors(1)
[0, 2]
sage: S.del_all_arcs(0,1)
sage: S.all_arcs(0,1)
[]
sage: S.all_arcs(1,2)
[0]
sage: S.del_vertex(7)
sage: S.all_arcs(7,3)
Traceback (most recent call last):
...
LookupError: vertex (7) is not a vertex of the graph
```

```
>>> from sage.all import *
>>> S.add_arc(Integer(0),Integer(1))
>>> S.add_arc(Integer(1),Integer(2))
>>> S.add_arc(Integer(1),Integer(0))
>>> S.has_arc(Integer(7),Integer(3))
False
>>> S.has_arc(Integer(0),Integer(1))
True
>>> S.in_neighbors(Integer(1))
[0]
>>> S.out_neighbors(Integer(1))
[0, 2]
>>> S.del_all_arcs(Integer(0),Integer(1))
>>> S.all_arcs(Integer(0),Integer(1))
[]
>>> S.all_arcs(Integer(1),Integer(2))
[0]
>>> S.del_vertex(Integer(7))
>>> S.all_arcs(Integer(7),Integer(3))
```

(continues on next page)

(continued from previous page)

```
Traceback (most recent call last):
...
LookupError: vertex (7) is not a vertex of the graph
```

Sparse graphs support multiple edges and labeled edges, but requires that the labels be positive integers (the case label = 0 is treated as no label).

```
sage: S.add_arc_label(0,1,-1)
Traceback (most recent call last):
...
ValueError: Label (-1) must be a nonnegative integer.
sage: S.add_arc(0,1)
sage: S.arc_label(0,1)
0
```

```
>>> from sage.all import *
>>> S.add_arc_label(Integer(0),Integer(1),-Integer(1))
Traceback (most recent call last):
...
ValueError: Label (-1) must be a nonnegative integer.
>>> S.add_arc(Integer(0),Integer(1))
>>> S.arc_label(Integer(0),Integer(1))
0
```

Note that `arc_label` only returns the first edge label found in the specified place, and this can be in any order (if you want all arc labels, use `all_arcs`):

```
sage: S.add_arc_label(0,1,1)
sage: S.arc_label(0,1)
1
sage: S.all_arcs(0,1)
[0, 1]
```

```
>>> from sage.all import *
>>> S.add_arc_label(Integer(0),Integer(1),Integer(1))
>>> S.arc_label(Integer(0),Integer(1))
1
>>> S.all_arcs(Integer(0),Integer(1))
[0, 1]
```

Zero specifies only that there is no labeled arc:

```
sage: S.arc_label(1,2)
0
```

```
>>> from sage.all import *
>>> S.arc_label(Integer(1),Integer(2))
0
```

So do not be fooled:

```
sage: S.all_arcs(1,2)
[0]
sage: S.add_arc(1,2)
sage: S.arc_label(1,2)
0
```

```
>>> from sage.all import *
>>> S.all_arcs(Integer(1),Integer(2))
[0]
>>> S.add_arc(Integer(1),Integer(2))
>>> S.arc_label(Integer(1),Integer(2))
0
```

Instead, if you work with unlabeled edges, be sure to use the right functions:

```
sage: T = SparseGraph(nverts = 3, expected_degree = 2)
sage: T.add_arc(0,1)
sage: T.add_arc(1,2)
sage: T.add_arc(2,0)
sage: T.has_arc(0,1)
True
```

```
>>> from sage.all import *
>>> T = SparseGraph(nverts = Integer(3), expected_degree = Integer(2))
>>> T.add_arc(Integer(0),Integer(1))
>>> T.add_arc(Integer(1),Integer(2))
>>> T.add_arc(Integer(2),Integer(0))
>>> T.has_arc(Integer(0),Integer(1))
True
```

Sparse graphs are by their nature directed. As of this writing, you need to do operations in pairs to treat the undirected case (or use a backend or a Sage graph):

```
sage: T.has_arc(1,0)
False
```

```
>>> from sage.all import *
>>> T.has_arc(Integer(1),Integer(0))
False
```

Multiple unlabeled edges are also possible:

```
sage: for _ in range(10): S.add_arc(5,4)
sage: S.all_arcs(5,4)
[0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
```

```
>>> from sage.all import *
>>> for _ in range(Integer(10)): S.add_arc(Integer(5),Integer(4))
>>> S.all_arcs(Integer(5),Integer(4))
[0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
```

The curious developer is encouraged to check out the `unsafe` functions, which do not check input but which run in pure C.

3.3.2 Underlying Data Structure

The class `SparseGraph` contains the following variables which are inherited from `CGraph` (for explanation, refer to the documentation there):

```
cdef int num_verts
cdef int num_arcs
cdef int *in_degrees
cdef int *out_degrees
cdef bitset_t active_vertices
```

It also contains the following variables:

```
cdef int hash_length
cdef int hash_mask
cdef SparseGraphBTNode **vertices
cdef SparseGraphBTNode **vertices_rev
cdef bint _directed
```

For each vertex `u`, a hash table of length `hash_length` is instantiated. An arc `(u, v)` is stored at `u * hash_length + hash(v)` of the array `vertices`, where `hash` should be thought of as an arbitrary but fixed hash function which takes values in `0 <= hash < hash_length`. Each address may represent different arcs, say `(u, v1)` and `(u, v2)` where `hash(v1) == hash(v2)`. Thus, a binary tree structure is used at this step to speed access to individual arcs, whose nodes (each of which represents a pair `(u,v)`) are instances of the following type:

```
cdef struct SparseGraphBTNode:
    int vertex
    int number
    SparseGraphLLNode *labels
    SparseGraphBTNode *left
    SparseGraphBTNode *right
```

Which range of the `vertices` array the root of the tree is in determines `u`, and `vertex` stores `v`. The integer `number` stores only the number of unlabeled arcs from `u` to `v`.

Currently, labels are stored in a simple linked list, whose nodes are instances of the following type:

```
cdef struct SparseGraphLLNode:
    int label
    int number
    SparseGraphLLNode *next
```

The int `label` must be a positive integer, since 0 indicates no label, and negative numbers indicate errors. The int `number` is the number of arcs with the given label.

TODO: Optimally, edge labels would also be represented by a binary tree, which would help performance in graphs with many overlapping edges. Also, a more efficient binary tree structure could be used, although in practice the trees involved will usually have very small order, unless the degree of vertices becomes significantly larger than the `expected_degree` given, because this is the size of each hash table. Indeed, the expected size of the binary trees is $\frac{\text{actual degree}}{\text{expected degree}}$. Ryan Dingman, e.g., is working on a general-purpose Cython-based red black tree, which would be optimal for both of these uses.

class sage.graphs.base.sparse_graph.**SparseGraph**

Bases: *CGraph*

Compiled sparse graphs.

```
sage: from sage.graphs.base.sparse_graph import SparseGraph
```

```
>>> from sage.all import *
>>> from sage.graphs.base.sparse_graph import SparseGraph
```

Sparse graphs are initialized as follows:

```
sage: S = SparseGraph(nverts = 10, expected_degree = 3, extra_vertices = 10)
```

```
>>> from sage.all import *
>>> S = SparseGraph(nverts = Integer(10), expected_degree = Integer(3), extra_
↪vertices = Integer(10))
```

INPUT:

- `nverts` – nonnegative integer, the number of vertices
- `expected_degree` – nonnegative integer (default: 16); expected upper bound on degree of vertices
- `extra_vertices` – nonnegative integer (default: 0); how many extra vertices to allocate
- `verts` – (optional) list of vertices to add
- `arcs` – (optional) list of arcs to add

The first `nverts` are created as vertices of the graph, and the next `extra_vertices` can be freely added without reallocation. See top level documentation for more details. The input `verts` and `arcs` are mainly for use in pickling.

add_arc_label(*u*, *v*, *l=0*)

Add arc `(u, v)` to the graph with label `l`.

INPUT:

- `u`, `v` – nonnegative integers, must be in `self`
- `l` – positive integer label, or zero for no label

EXAMPLES:

```
sage: from sage.graphs.base.sparse_graph import SparseGraph
sage: G = SparseGraph(5)
sage: G.add_arc_label(0,1)
sage: G.add_arc_label(4,7)
Traceback (most recent call last):
...
LookupError: vertex (7) is not a vertex of the graph
sage: G.has_arc(1,0)
False
sage: G.has_arc(0,1)
True
sage: G.add_arc_label(1,2,2)
sage: G.arc_label(1,2)
2
```

```
>>> from sage.all import *
>>> from sage.graphs.base.sparse_graph import SparseGraph
```

(continues on next page)

(continued from previous page)

```
>>> G = SparseGraph(Integer(5))
>>> G.add_arc_label(Integer(0),Integer(1))
>>> G.add_arc_label(Integer(4),Integer(7))
Traceback (most recent call last):
...
LookupError: vertex (7) is not a vertex of the graph
>>> G.has_arc(Integer(1),Integer(0))
False
>>> G.has_arc(Integer(0),Integer(1))
True
>>> G.add_arc_label(Integer(1),Integer(2),Integer(2))
>>> G.arc_label(Integer(1),Integer(2))
2
```

in_degree(*v*)

Return the in-degree of v

INPUT:

- v – integer

EXAMPLES:

```
sage: from sage.graphs.base.sparse_graph import SparseGraph
sage: G = SparseGraph(5)
sage: G.add_arc(0,1)
sage: G.add_arc(1,2)
sage: G.add_arc(1,3)
sage: G.in_degree(0)
0
sage: G.in_degree(1)
1
```

```
>>> from sage.all import *
>>> from sage.graphs.base.sparse_graph import SparseGraph
>>> G = SparseGraph(Integer(5))
>>> G.add_arc(Integer(0),Integer(1))
>>> G.add_arc(Integer(1),Integer(2))
>>> G.add_arc(Integer(1),Integer(3))
>>> G.in_degree(Integer(0))
0
>>> G.in_degree(Integer(1))
1
```

is_directed()

Return whether the graph is directed.

EXAMPLES:

```
sage: from sage.graphs.base.sparse_graph import SparseGraph
sage: G = SparseGraph(5)
sage: G.is_directed()
True
sage: G = SparseGraph(5, directed=False)
```

(continues on next page)

(continued from previous page)

```
sage: G.is_directed()
False
```

```
>>> from sage.all import *
>>> from sage.graphs.base.sparse_graph import SparseGraph
>>> G = SparseGraph(Integer(5))
>>> G.is_directed()
True
>>> G = SparseGraph(Integer(5), directed=False)
>>> G.is_directed()
False
```

out_degree(*u*)

Return the out-degree of `v`

INPUT:

- `u` – integer

EXAMPLES:

```
sage: from sage.graphs.base.sparse_graph import SparseGraph
sage: G = SparseGraph(5)
sage: G.add_arc(0,1)
sage: G.add_arc(1,2)
sage: G.add_arc(1,3)
sage: G.out_degree(0)
1
sage: G.out_degree(1)
2
```

```
>>> from sage.all import *
>>> from sage.graphs.base.sparse_graph import SparseGraph
>>> G = SparseGraph(Integer(5))
>>> G.add_arc(Integer(0),Integer(1))
>>> G.add_arc(Integer(1),Integer(2))
>>> G.add_arc(Integer(1),Integer(3))
>>> G.out_degree(Integer(0))
1
>>> G.out_degree(Integer(1))
2
```

realloc(*total*)

Reallocate the number of vertices to use, without actually adding any.

INPUT:

- `total` – integer; the total size to make the array

Returns -1 and fails if reallocation would destroy any active vertices.

EXAMPLES:

```
sage: from sage.graphs.base.sparse_graph import SparseGraph
sage: S = SparseGraph(nverts=4, extra_vertices=4)
```

(continues on next page)

(continued from previous page)

```
sage: S.current_allocation()
8
sage: S.add_vertex(6)
6
sage: S.current_allocation()
8
sage: S.add_vertex(10)
10
sage: S.current_allocation()
16
sage: S.add_vertex(40)
Traceback (most recent call last):
...
RuntimeError: requested vertex is past twice the allocated range: use realloc
sage: S.realloc(50)
sage: S.add_vertex(40)
40
sage: S.current_allocation()
50
sage: S.realloc(30)
-1
sage: S.current_allocation()
50
sage: S.del_vertex(40)
sage: S.realloc(30)
sage: S.current_allocation()
30
```

```
>>> from sage.all import *
>>> from sage.graphs.base.sparse_graph import SparseGraph
>>> S = SparseGraph(nverts=Integer(4), extra_vertices=Integer(4))
>>> S.current_allocation()
8
>>> S.add_vertex(Integer(6))
6
>>> S.current_allocation()
8
>>> S.add_vertex(Integer(10))
10
>>> S.current_allocation()
16
>>> S.add_vertex(Integer(40))
Traceback (most recent call last):
...
RuntimeError: requested vertex is past twice the allocated range: use realloc
>>> S.realloc(Integer(50))
>>> S.add_vertex(Integer(40))
40
>>> S.current_allocation()
50
>>> S.realloc(Integer(30))
-1
```

(continues on next page)

(continued from previous page)

```
>>> S.current_allocation()
50
>>> S.del_vertex(Integer(40))
>>> S.realloc(Integer(30))
>>> S.current_allocation()
30
```

class sage.graphs.base.sparse_graph.**SparseGraphBackend**

Bases: *CGraphBackend*

Backend for Sage graphs using SparseGraphs.

```
sage: from sage.graphs.base.sparse_graph import SparseGraphBackend
```

```
>>> from sage.all import *
>>> from sage.graphs.base.sparse_graph import SparseGraphBackend
```

This class is only intended for use by the Sage Graph and DiGraph class. If you are interested in using a SparseGraph, you probably want to do something like the following example, which creates a Sage Graph instance which wraps a SparseGraph object:

```
sage: G = Graph(30, sparse=True)
sage: G.add_edges([(0,1), (0,3), (4,5), (9, 23)])
sage: G.edges(sort=True, labels=False)
[(0, 1), (0, 3), (4, 5), (9, 23)]
```

```
>>> from sage.all import *
>>> G = Graph(Integer(30), sparse=True)
>>> G.add_edges([(Integer(0),Integer(1)), (Integer(0),Integer(3)), (Integer(4),
↪Integer(5)), (Integer(9), Integer(23))])
>>> G.edges(sort=True, labels=False)
[(0, 1), (0, 3), (4, 5), (9, 23)]
```

Note that Sage graphs using the backend are more flexible than SparseGraphs themselves. This is because SparseGraphs (by design) do not deal with Python objects:

```
sage: G.add_vertex((0,1,2))
sage: sorted(list(G),
....:        key=lambda x: (isinstance(x, tuple), x))
[0,
...
 29,
 (0, 1, 2)]
sage: from sage.graphs.base.sparse_graph import SparseGraph
sage: SG = SparseGraph(30)
sage: SG.add_vertex((0,1,2))
Traceback (most recent call last):
...
TypeError: an integer is required
```

```
>>> from sage.all import *
>>> G.add_vertex((Integer(0),Integer(1),Integer(2)))
```

(continues on next page)

(continued from previous page)

```
>>> sorted(list(G),
...        key=lambda x: (isinstance(x, tuple), x))
[0,
...
 29,
 (0, 1, 2)]
>>> from sage.graphs.base.sparse_graph import SparseGraph
>>> SG = SparseGraph(Integer(30))
>>> SG.add_vertex((Integer(0),Integer(1),Integer(2)))
Traceback (most recent call last):
...
TypeError: an integer is required
```

get_edge_label(*u*, *v*)

Return the edge label for `(u, v)`.

INPUT:

- `u`, `v` – the vertices of the edge

EXAMPLES:

```
sage: D = sage.graphs.base.sparse_graph.SparseGraphBackend(9)
sage: D.add_edges([(0,1,1), (2,3,2), (4,5,3), (5,6,2)], False)
sage: list(D.iterator_edges(range(9), True))
[(0, 1, 1), (2, 3, 2), (4, 5, 3), (5, 6, 2)]
sage: D.get_edge_label(3,2)
2
```

```
>>> from sage.all import *
>>> D = sage.graphs.base.sparse_graph.SparseGraphBackend(Integer(9))
>>> D.add_edges([(Integer(0),Integer(1),Integer(1)), (Integer(2),Integer(3),
↪Integer(2)), (Integer(4),Integer(5),Integer(3)), (Integer(5),Integer(6),
↪Integer(2))], False)
>>> list(D.iterator_edges(range(Integer(9)), True))
[(0, 1, 1), (2, 3, 2), (4, 5, 3), (5, 6, 2)]
>>> D.get_edge_label(Integer(3),Integer(2))
2
```

has_edge(*u*, *v*, *l*)

Return whether this graph has edge `(u,v)` with label `l`. If `l` is `None`, return whether this graph has an edge `(u,v)` with any label.

INPUT:

- `u`, `v` – the vertices of the edge
- `l` – the edge label, or `None`

EXAMPLES:

```
sage: D = sage.graphs.base.sparse_graph.SparseGraphBackend(9)
sage: D.add_edges([(0,1), (2,3), (4,5), (5,6)], False)
sage: D.has_edge(0,1,None)
True
```

```
>>> from sage.all import *
>>> D = sage.graphs.base.sparse_graph.SparseGraphBackend(Integer(9))
>>> D.add_edges([(Integer(0),Integer(1)), (Integer(2),Integer(3)),␣
↪(Integer(4),Integer(5)), (Integer(5),Integer(6))], False)
>>> D.has_edge(Integer(0),Integer(1),None)
True
```

multiple_edges(*new*)

Get/set whether or not `self` allows multiple edges.

INPUT:

- `new` – boolean (to set) or `None` (to get)

EXAMPLES:

```
sage: G = sage.graphs.base.sparse_graph.SparseGraphBackend(9)
sage: G.multiple_edges(True)
sage: G.multiple_edges(None)
True
sage: G.multiple_edges(False)
sage: G.multiple_edges(None)
False
sage: G.add_edge(0,1,0,True)
sage: G.add_edge(0,1,0,True)
sage: list(G.iterator_out_edges(range(9), True))
[(0, 1, 0)]
```

```
>>> from sage.all import *
>>> G = sage.graphs.base.sparse_graph.SparseGraphBackend(Integer(9))
>>> G.multiple_edges(True)
>>> G.multiple_edges(None)
True
>>> G.multiple_edges(False)
>>> G.multiple_edges(None)
False
>>> G.add_edge(Integer(0),Integer(1),Integer(0),True)
>>> G.add_edge(Integer(0),Integer(1),Integer(0),True)
>>> list(G.iterator_out_edges(range(Integer(9)), True))
[(0, 1, 0)]
```

set_edge_label(*u*, *v*, *l*, *directed*)

Label the edge `(u,v)` by `l`.

INPUT:

- `u`, `v` – the vertices of the edge
- `l` – the edge label
- `directed` – if `False`, also set `(v,u)` with label `l`

EXAMPLES:

```
sage: G = sage.graphs.base.sparse_graph.SparseGraphBackend(9)
sage: G.add_edge(1,2,None,True)
```

(continues on next page)

(continued from previous page)

```
sage: G.set_edge_label(1,2,'a',True)
sage: list(G.iterator_out_edges(range(9), True))
[(1, 2, 'a')]
```

```
>>> from sage.all import *
>>> G = sage.graphs.base.sparse_graph.SparseGraphBackend(Integer(9))
>>> G.add_edge(Integer(1),Integer(2),None,True)
>>> G.set_edge_label(Integer(1),Integer(2),'a',True)
>>> list(G.iterator_out_edges(range(Integer(9)), True))
[(1, 2, 'a')]
```

Note that it fails silently if there is no edge there:

```
sage: G.set_edge_label(2,1,'b',True)
sage: list(G.iterator_out_edges(range(9), True))
[(1, 2, 'a')]
```

```
>>> from sage.all import *
>>> G.set_edge_label(Integer(2),Integer(1),'b',True)
>>> list(G.iterator_out_edges(range(Integer(9)), True))
[(1, 2, 'a')]
```

3.4 Fast dense graphs

For an overview of graph data structures in sage, see *overview*.

3.4.1 Usage Introduction

```
sage: from sage.graphs.base.dense_graph import DenseGraph
```

```
>>> from sage.all import *
>>> from sage.graphs.base.dense_graph import DenseGraph
```

Dense graphs are initialized as follows:

```
sage: D = DenseGraph(nverts=10, extra_vertices=10)
```

```
>>> from sage.all import *
>>> D = DenseGraph(nverts=Integer(10), extra_vertices=Integer(10))
```

This example initializes a dense graph with room for twenty vertices, the first ten of which are in the graph. In general, the first `nverts` are "active." For example, see that 9 is already in the graph:

```
sage: D.verts()
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
sage: D.add_vertex(9)
9
sage: D.verts()
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

```
>>> from sage.all import *
>>> D.verts()
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
>>> D.add_vertex(Integer(9))
9
>>> D.verts()
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

But 10 is not, until we add it:

```
sage: D.add_vertex(10)
10
sage: D.verts()
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

```
>>> from sage.all import *
>>> D.add_vertex(Integer(10))
10
>>> D.verts()
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

You can begin working right away as follows:

```
sage: D.add_arc(0, 1)
sage: D.add_arc(1, 2)
sage: D.add_arc(1, 0)
sage: D.has_arc(7, 3)
False
sage: D.has_arc(0, 1)
True
sage: D.in_neighbors(1)
[0]
sage: D.out_neighbors(1)
[0, 2]
sage: D.del_all_arcs(0, 1)
sage: D.has_arc(0, 1)
False
sage: D.has_arc(1, 2)
True
sage: D.del_vertex(7)
sage: D.has_arc(7, 3)
False
```

```
>>> from sage.all import *
>>> D.add_arc(Integer(0), Integer(1))
>>> D.add_arc(Integer(1), Integer(2))
>>> D.add_arc(Integer(1), Integer(0))
>>> D.has_arc(Integer(7), Integer(3))
False
>>> D.has_arc(Integer(0), Integer(1))
True
>>> D.in_neighbors(Integer(1))
[0]
```

(continues on next page)

(continued from previous page)

```
>>> D.out_neighbors(Integer(1))
[0, 2]
>>> D.del_all_arcs(Integer(0), Integer(1))
>>> D.has_arc(Integer(0), Integer(1))
False
>>> D.has_arc(Integer(1), Integer(2))
True
>>> D.del_vertex(Integer(7))
>>> D.has_arc(Integer(7), Integer(3))
False
```

Dense graphs do not support multiple or labeled edges.

```
sage: T = DenseGraph(nverts=3, extra_vertices=2)
sage: T.add_arc(0, 1)
sage: T.add_arc(1, 2)
sage: T.add_arc(2, 0)
sage: T.has_arc(0, 1)
True
```

```
>>> from sage.all import *
>>> T = DenseGraph(nverts=Integer(3), extra_vertices=Integer(2))
>>> T.add_arc(Integer(0), Integer(1))
>>> T.add_arc(Integer(1), Integer(2))
>>> T.add_arc(Integer(2), Integer(0))
>>> T.has_arc(Integer(0), Integer(1))
True
```

```
sage: for _ in range(10): D.add_arc(5, 4)
sage: D.has_arc(5,4 )
True
```

```
>>> from sage.all import *
>>> for _ in range(Integer(10)): D.add_arc(Integer(5), Integer(4))
>>> D.has_arc(Integer(5),Integer(4) )
True
```

Dense graphs are by their nature directed. As of this writing, you need to do operations in pairs to treat the undirected case (or use a backend or a Sage graph):

```
sage: T.has_arc(1, 0)
False
```

```
>>> from sage.all import *
>>> T.has_arc(Integer(1), Integer(0))
False
```

The curious developer is encouraged to check out the `unsafe` functions, which do not check input but which run in pure C.

3.4.2 Underlying Data Structure

The class `DenseGraph` contains the following variables which are inherited from `CGraph` (for explanation, refer to the documentation there):

```
cdef int num_verts
cdef int num_arcs
cdef int *in_degrees
cdef int *out_degrees
cdef bitset_t active_vertices
```

It also contains the following variables:

```
cdef binary_matrix_t edges
```

Note

As the edges are stored as the adjacency matrix of the graph, enumerating the edges of the graph has complexity $O(n^2)$ and enumerating the neighbors of a vertex has complexity $O(n)$ (where n in the size of the bitset active_vertices). So, the class `DenseGraph` should be used for graphs such that the number of edges is close to the square of the number of vertices.

class sage.graphs.base.dense_graph.**DenseGraph**

Bases: *CGraph*

Compiled dense graphs.

```
sage: from sage.graphs.base.dense_graph import DenseGraph
```

```
>>> from sage.all import *
>>> from sage.graphs.base.dense_graph import DenseGraph
```

Dense graphs are initialized as follows:

```
sage: D = DenseGraph(nverts=10, extra_vertices=10)
```

```
>>> from sage.all import *
>>> D = DenseGraph(nverts=Integer(10), extra_vertices=Integer(10))
```

INPUT:

- `nverts` – nonnegative integer; the number of vertices
- `extra_vertices` – nonnegative integer (default: 10); how many extra vertices to allocate
- `verts` – list (default: `None`); optional list of vertices to add
- `arcs` – list (default: `None`); optional list of arcs to add
- `directed` – boolean (default: `None`); whether the graph is directed

The first `nverts` are created as vertices of the graph, and the next `extra_vertices` can be freely added without reallocation. See top level documentation for more details. The input `verts` and `arcs` are mainly for use in pickling.

complement()

Replace the graph with its complement.

Note

Assumes that the graph has no loop.

EXAMPLES:

```
sage: from sage.graphs.base.dense_graph import DenseGraph
sage: G = DenseGraph(5)
sage: G.add_arc(0, 1)
sage: G.has_arc(0, 1)
True
sage: G.complement()
sage: G.has_arc(0, 1)
False
```

```
>>> from sage.all import *
>>> from sage.graphs.base.dense_graph import DenseGraph
>>> G = DenseGraph(Integer(5))
>>> G.add_arc(Integer(0), Integer(1))
>>> G.has_arc(Integer(0), Integer(1))
True
>>> G.complement()
>>> G.has_arc(Integer(0), Integer(1))
False
```

in_degree(*v*)

Return the in-degree of v

INPUT:

- v – integer

EXAMPLES:

```
sage: from sage.graphs.base.dense_graph import DenseGraph
sage: G = DenseGraph(5)
sage: G.add_arc(0,1)
sage: G.add_arc(1,2)
sage: G.add_arc(1,3)
sage: G.in_degree(0)
0
sage: G.in_degree(1)
1
```

```
>>> from sage.all import *
>>> from sage.graphs.base.dense_graph import DenseGraph
>>> G = DenseGraph(Integer(5))
>>> G.add_arc(Integer(0),Integer(1))
>>> G.add_arc(Integer(1),Integer(2))
>>> G.add_arc(Integer(1),Integer(3))
>>> G.in_degree(Integer(0))
0
>>> G.in_degree(Integer(1))
1
```

out_degree(*u*)

Return the out-degree of v

INPUT:

- u – integer

EXAMPLES:

```
sage: from sage.graphs.base.dense_graph import DenseGraph
sage: G = DenseGraph(5)
sage: G.add_arc(0,1)
sage: G.add_arc(1,2)
sage: G.add_arc(1,3)
sage: G.out_degree(0)
1
sage: G.out_degree(1)
2
```

```
>>> from sage.all import *
>>> from sage.graphs.base.dense_graph import DenseGraph
>>> G = DenseGraph(Integer(5))
>>> G.add_arc(Integer(0),Integer(1))
>>> G.add_arc(Integer(1),Integer(2))
>>> G.add_arc(Integer(1),Integer(3))
>>> G.out_degree(Integer(0))
1
>>> G.out_degree(Integer(1))
2
```

realloc(*total_verts*)

Reallocate the number of vertices to use, without actually adding any.

INPUT:

- total – integer; the total size to make the array

Returns -1 and fails if reallocation would destroy any active vertices.

EXAMPLES:

```
sage: from sage.graphs.base.dense_graph import DenseGraph
sage: D = DenseGraph(nverts=4, extra_vertices=4)
sage: D.current_allocation()
8
sage: D.add_vertex(6)
6
sage: D.current_allocation()
8
sage: D.add_vertex(10)
10
sage: D.current_allocation()
16
sage: D.add_vertex(40)
Traceback (most recent call last):
...
```

(continues on next page)

(continued from previous page)

```
RuntimeError: requested vertex is past twice the allocated range: use realloc
sage: D.realloc(50)
sage: D.add_vertex(40)
40
sage: D.current_allocation()
50
sage: D.realloc(30)
-1
sage: D.current_allocation()
50
sage: D.del_vertex(40)
sage: D.realloc(30)
sage: D.current_allocation()
30
```

```
>>> from sage.all import *
>>> from sage.graphs.base.dense_graph import DenseGraph
>>> D = DenseGraph(nverts=Integer(4), extra_vertices=Integer(4))
>>> D.current_allocation()
8
>>> D.add_vertex(Integer(6))
6
>>> D.current_allocation()
8
>>> D.add_vertex(Integer(10))
10
>>> D.current_allocation()
16
>>> D.add_vertex(Integer(40))
Traceback (most recent call last):
...
RuntimeError: requested vertex is past twice the allocated range: use realloc
>>> D.realloc(Integer(50))
>>> D.add_vertex(Integer(40))
40
>>> D.current_allocation()
50
>>> D.realloc(Integer(30))
-1
>>> D.current_allocation()
50
>>> D.del_vertex(Integer(40))
>>> D.realloc(Integer(30))
>>> D.current_allocation()
30
```

class sage.graphs.base.dense_graph.**DenseGraphBackend**

Bases: *CGraphBackend*

Backend for Sage graphs using DenseGraphs.

```
sage: from sage.graphs.base.dense_graph import DenseGraphBackend
```

```
>>> from sage.all import *
>>> from sage.graphs.base.dense_graph import DenseGraphBackend
```

This class is only intended for use by the Sage Graph and DiGraph class. If you are interested in using a DenseGraph, you probably want to do something like the following example, which creates a Sage Graph instance which wraps a `DenseGraph` object:

```
sage: G = Graph(30, sparse=False)
sage: G.add_edges([(0, 1), (0, 3), (4, 5), (9, 23)])
sage: G.edges(sort=True, labels=False)
[(0, 1), (0, 3), (4, 5), (9, 23)]
```

```
>>> from sage.all import *
>>> G = Graph(Integer(30), sparse=False)
>>> G.add_edges([(Integer(0), Integer(1)), (Integer(0), Integer(3)), (Integer(4),␣
↪Integer(5)), (Integer(9), Integer(23))])
>>> G.edges(sort=True, labels=False)
[(0, 1), (0, 3), (4, 5), (9, 23)]
```

Note that Sage graphs using the backend are more flexible than DenseGraphs themselves. This is because DenseGraphs (by design) do not deal with Python objects:

```
sage: G.add_vertex((0, 1, 2))
sage: sorted(list(G),
....:        key=lambda x: (isinstance(x, tuple), x))
[0,
...
 29,
 (0, 1, 2)]
sage: from sage.graphs.base.dense_graph import DenseGraph
sage: DG = DenseGraph(30)
sage: DG.add_vertex((0, 1, 2))
Traceback (most recent call last):
...
TypeError: an integer is required
```

```
>>> from sage.all import *
>>> G.add_vertex((Integer(0), Integer(1), Integer(2)))
>>> sorted(list(G),
...        key=lambda x: (isinstance(x, tuple), x))
[0,
...
 29,
 (0, 1, 2)]
>>> from sage.graphs.base.dense_graph import DenseGraph
>>> DG = DenseGraph(Integer(30))
>>> DG.add_vertex((Integer(0), Integer(1), Integer(2)))
Traceback (most recent call last):
...
TypeError: an integer is required
```

add_edges(*edges*, *directed*, *remove_loops=False*)

Add edges from a list.

INPUT:

- `edges` – an iterable of edges to be added; each edge can either be of the form `(u, v)` or `(u, v, l)`
- `directed` – if `False`, adds `(v, u)` as well as `(u, v)`
- `remove_loops` – if `True`, remove loops

EXAMPLES:

```
sage: D = sage.graphs.base.dense_graph.DenseGraphBackend(9)
sage: D.add_edges([(0, 1), (2, 3), (4, 5), (5, 6)], False)
sage: list(D.iterator_edges(range(9), True))
[(0, 1, None),
 (2, 3, None),
 (4, 5, None),
 (5, 6, None)]
```

```
>>> from sage.all import *
>>> D = sage.graphs.base.dense_graph.DenseGraphBackend(Integer(9))
>>> D.add_edges([(Integer(0), Integer(1)), (Integer(2), Integer(3)),␣
↪(Integer(4), Integer(5)), (Integer(5), Integer(6))], False)
>>> list(D.iterator_edges(range(Integer(9)), True))
[(0, 1, None),
 (2, 3, None),
 (4, 5, None),
 (5, 6, None)]
```

get_edge_label(*u*, *v*)

Return the edge label for `(u, v)`.

Always `None`, since dense graphs do not support edge labels.

INPUT:

- `u`, `v` – the vertices of the edge

EXAMPLES:

```
sage: D = sage.graphs.base.dense_graph.DenseGraphBackend(9)
sage: D.add_edges([(0, 1), (2, 3, 7), (4, 5), (5, 6)], False)
sage: list(D.iterator_edges(range(9), True))
[(0, 1, None),
 (2, 3, None),
 (4, 5, None),
 (5, 6, None)]
sage: D.del_edge(0, 1, None, True)
sage: list(D.iterator_out_edges(range(9), True))
[(1, 0, None),
 (2, 3, None),
 (3, 2, None),
 (4, 5, None),
 (5, 4, None),
 (5, 6, None),
 (6, 5, None)]
sage: D.get_edge_label(2, 3)
sage: D.get_edge_label(2, 4)
```

(continues on next page)

(continued from previous page)

```
Traceback (most recent call last):
...
LookupError: (2, 4) is not an edge of the graph
```

```
>>> from sage.all import *
>>> D = sage.graphs.base.dense_graph.DenseGraphBackend(Integer(9))
>>> D.add_edges([(Integer(0), Integer(1)), (Integer(2), Integer(3),␣
↪Integer(7)), (Integer(4), Integer(5)), (Integer(5), Integer(6))], False)
>>> list(D.iterator_edges(range(Integer(9)), True))
[(0, 1, None),
 (2, 3, None),
 (4, 5, None),
 (5, 6, None)]
>>> D.del_edge(Integer(0), Integer(1), None, True)
>>> list(D.iterator_out_edges(range(Integer(9)), True))
[(1, 0, None),
 (2, 3, None),
 (3, 2, None),
 (4, 5, None),
 (5, 4, None),
 (5, 6, None),
 (6, 5, None)]
>>> D.get_edge_label(Integer(2), Integer(3))
>>> D.get_edge_label(Integer(2), Integer(4))
Traceback (most recent call last):
...
LookupError: (2, 4) is not an edge of the graph
```

has_edge(*u*, *v*, *l*)

Check whether this graph has edge `(u, v)`.

Note

The input `l` is for consistency with other backends.

INPUT:

- `u`, `v` – the vertices of the edge
- `l` – the edge label (ignored)

EXAMPLES:

```
sage: D = sage.graphs.base.dense_graph.DenseGraphBackend(9)
sage: D.add_edges([(0, 1), (2, 3), (4, 5), (5, 6)], False)
sage: D.has_edge(0, 1, None)
True
```

```
>>> from sage.all import *
>>> D = sage.graphs.base.dense_graph.DenseGraphBackend(Integer(9))
>>> D.add_edges([(Integer(0), Integer(1)), (Integer(2), Integer(3)),␣
↪(Integer(4), Integer(5)), (Integer(5), Integer(6))], False)
```

(continues on next page)

(continued from previous page)

```
>>> D.has_edge(Integer(0), Integer(1), None)
True
```

multiple_edges(*new*)

Get/set whether or not `self` allows multiple edges.

INPUT:

- `new` – boolean (to set) or `None` (to get)

EXAMPLES:

```
sage: import sage.graphs.base.dense_graph
sage: G = sage.graphs.base.dense_graph.DenseGraphBackend(9)
sage: G.multiple_edges(True)
Traceback (most recent call last):
...
NotImplementedError: dense graphs do not support multiple edges
sage: G.multiple_edges(None)
False
```

```
>>> from sage.all import *
>>> import sage.graphs.base.dense_graph
>>> G = sage.graphs.base.dense_graph.DenseGraphBackend(Integer(9))
>>> G.multiple_edges(True)
Traceback (most recent call last):
...
NotImplementedError: dense graphs do not support multiple edges
>>> G.multiple_edges(None)
False
```

set_edge_label(*u*, *v*, *l*, *directed*)

Label the edge `(u, v)` by `l`.

INPUT:

- `u`, `v` – the vertices of the edge
- `l` – the edge label
- `directed` – if `False`, also set `(v, u)` with label `l`

EXAMPLES:

```
sage: import sage.graphs.base.dense_graph
sage: G = sage.graphs.base.dense_graph.DenseGraphBackend(9)
sage: G.set_edge_label(1, 2, 'a', True)
Traceback (most recent call last):
...
NotImplementedError: dense graphs do not support edge labels
```

```
>>> from sage.all import *
>>> import sage.graphs.base.dense_graph
>>> G = sage.graphs.base.dense_graph.DenseGraphBackend(Integer(9))
>>> G.set_edge_label(Integer(1), Integer(2), 'a', True)
```

(continues on next page)

(continued from previous page)

```
Traceback (most recent call last):
...
NotImplementedError: dense graphs do not support edge labels
```

3.5 Static dense graphs

This module gathers everything which is related to static dense graphs, i.e. :

- The vertices are integer from 0 to $n-1$
- No labels on vertices/edges
- No multiple edges
- No addition/removal of vertices

This being said, it is technically possible to add/remove edges. The data structure does not mind at all.

It is all based on the binary matrix data structure described in `data_structures/binary_matrix.pxd`, which is almost a copy of the bitset data structure. The only difference is that it differentiates the rows (the vertices) instead of storing the whole data in a long bitset, and we can use that.

For an overview of graph data structures in sage, see *overview*.

3.5.1 Index

Cython functions

`dense_graph_init`	Fill a binary matrix with the information from a Sage (di)graph.

Python functions

is_strongly_regular()	Check whether the graph is strongly regular
is_triangle_free()	Check whether G is triangle free
triangles_count()	Return the number of triangles containing v, for every v
connected_subgraph_iterator()	Iterator over the induced connected subgraphs of order at most k

3.5.2 Functions

sage.graphs.base.static_dense_graph.**connected_full_subgraphs**(*G*, *edges_only=False*, *labels=False*, *min_edges=None*, *max_edges=None*)

Return an iterator over the connected subgraphs of G with same vertex set.

This method implements a iterator over the connected subgraphs of the input (di)graph with the same ground set of vertices. That is, it iterates over every subgraph $H = (V_H, E_H)$ of $G = (V, E)$ such that $V_H = V$, $E_H \subseteq E$ and H is connected. Hence, this method may yield a huge number of graphs.

When the input (di)graph G is not connected, this method returns nothing.

As for method *connected_components()*, edge orientation is ignored. Hence, the directed graph with a single arc $0 \to 1$ is considered connected.

INPUT:

- G – a *Graph* or a *DiGraph*; loops and multiple edges are *not* allowed
- edges_only – boolean (default: False); whether to return (Di)Graph or list of vertices
- labels – boolean (default: False); whether to return labeled edges or not. This parameter is used only when edges_only is True.
- min_edges – integer (default: None); minimum number of edges of reported subgraphs. By default (None), this lower bound will be set to $n - 1$.
- max_edges – integer (default: None); maximum number of edges of reported subgraphs. By default (None), this lower bound will be set to the number of edges of the input (di)graph.

Note

Roughly, this method explores all possible subsets of neighbors of each vertex, which represents a huge number of subsets. We have thus chosen to limit the degree of the vertices of the graphs that can be considered, even if the graph has a single connected subgraph (e.g., a tree). It is therefore recommended to call this method on biconnected components, as done in *connected_subgraph_iterator()*.

EXAMPLES:

The complete graph of order 3 has 4 connected subgraphs:

```
sage: from sage.graphs.base.static_dense_graph import connected_full_subgraphs
sage: G = graphs.CompleteGraph(3)
sage: len(list(connected_full_subgraphs(G)))
4
```

```
>>> from sage.all import *
>>> from sage.graphs.base.static_dense_graph import connected_full_subgraphs
>>> G = graphs.CompleteGraph(Integer(3))
>>> len(list(connected_full_subgraphs(G)))
4
```

A cycle of order 5 has 6 connected subgraphs:

```
sage: from sage.graphs.base.static_dense_graph import connected_full_subgraphs
sage: G = graphs.CycleGraph(5)
sage: len(list(connected_full_subgraphs(G)))
6
```

```
>>> from sage.all import *
>>> from sage.graphs.base.static_dense_graph import connected_full_subgraphs
>>> G = graphs.CycleGraph(Integer(5))
>>> len(list(connected_full_subgraphs(G)))
6
```

The House graph has 18 connected subgraphs of order 5:

```
sage: from sage.graphs.base.static_dense_graph import connected_full_subgraphs
sage: G = graphs.HouseGraph()
sage: L = list(connected_full_subgraphs(G))
sage: len(L)
18
```

(continues on next page)

(continued from previous page)

```
sage: all(g.order() == 5 for g in L)
True
sage: all(g.is_connected() for g in L)
True
sage: F = frozenset(frozenset(g.edges(sort=False, labels=False)) for g in L)
sage: len(F)
18
```

```
>>> from sage.all import *
>>> from sage.graphs.base.static_dense_graph import connected_full_subgraphs
>>> G = graphs.HouseGraph()
>>> L = list(connected_full_subgraphs(G))
>>> len(L)
18
>>> all(g.order() == Integer(5) for g in L)
True
>>> all(g.is_connected() for g in L)
True
>>> F = frozenset(frozenset(g.edges(sort=False, labels=False)) for g in L)
>>> len(F)
18
```

Specifying bounds on the number of edges:

```
sage: from sage.graphs.base.static_dense_graph import connected_full_subgraphs
sage: G = graphs.HouseGraph()
sage: [g.size() for g in connected_full_subgraphs(G)]
[6, 5, 5, 5, 4, 4, 5, 4, 4, 4, 5, 4, 4, 4, 5, 4, 4, 4]
sage: [g.size() for g in connected_full_subgraphs(G, max_edges=4)]
[4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4]
sage: [g.size() for g in connected_full_subgraphs(G, min_edges=6)]
[6]
sage: [g.size() for g in connected_full_subgraphs(G, min_edges=5, max_edges=5)]
[5, 5, 5, 5, 5, 5]
```

```
>>> from sage.all import *
>>> from sage.graphs.base.static_dense_graph import connected_full_subgraphs
>>> G = graphs.HouseGraph()
>>> [g.size() for g in connected_full_subgraphs(G)]
[6, 5, 5, 5, 4, 4, 5, 4, 4, 4, 5, 4, 4, 4, 5, 4, 4, 4]
>>> [g.size() for g in connected_full_subgraphs(G, max_edges=Integer(4))]
[4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4]
>>> [g.size() for g in connected_full_subgraphs(G, min_edges=Integer(6))]
[6]
>>> [g.size() for g in connected_full_subgraphs(G, min_edges=Integer(5), max_
↪edges=Integer(5))]
[5, 5, 5, 5, 5, 5]
```

Asking for edges only:

```
sage: from sage.graphs.base.static_dense_graph import connected_full_subgraphs
sage: G = Graph([(0, 1, "01"), (0, 2, "02"), (1, 2, "12")])
```

(continues on next page)

(continued from previous page)

```
sage: it = connected_full_subgraphs(G, edges_only=True)
sage: next(it)
[(0, 1), (0, 2), (1, 2)]
sage: next(it)
[(0, 1), (0, 2)]
sage: it = connected_full_subgraphs(G, edges_only=True, labels=True)
sage: next(it)
[(0, 1, '01'), (0, 2, '02'), (1, 2, '12')]
sage: next(it)
[(0, 1, '01'), (0, 2, '02')]
```

```
>>> from sage.all import *
>>> from sage.graphs.base.static_dense_graph import connected_full_subgraphs
>>> G = Graph([(Integer(0), Integer(1), "01"), (Integer(0), Integer(2), "02"),␣
↪(Integer(1), Integer(2), "12")])
>>> it = connected_full_subgraphs(G, edges_only=True)
>>> next(it)
[(0, 1), (0, 2), (1, 2)]
>>> next(it)
[(0, 1), (0, 2)]
>>> it = connected_full_subgraphs(G, edges_only=True, labels=True)
>>> next(it)
[(0, 1, '01'), (0, 2, '02'), (1, 2, '12')]
>>> next(it)
[(0, 1, '01'), (0, 2, '02')]
```

Subgraphs of a digraph:

```
sage: from sage.graphs.base.static_dense_graph import connected_full_subgraphs
sage: G = digraphs.Complete(2)
sage: list(connected_full_subgraphs(G, edges_only=True))
[[(0, 1)], [(1, 0)], [(0, 1), (1, 0)]]
```

```
>>> from sage.all import *
>>> from sage.graphs.base.static_dense_graph import connected_full_subgraphs
>>> G = digraphs.Complete(Integer(2))
>>> list(connected_full_subgraphs(G, edges_only=True))
[[(0, 1)], [(1, 0)], [(0, 1), (1, 0)]]
```

sage.graphs.base.static_dense_graph.**connected_subgraph_iterator**(*G*, *k=None*, *vertices_only=False*, *edges_only=False*, *labels=False*, *induced=True*, *exactly_k=False*)

Return an iterator over the induced connected subgraphs of order at most k.

This method implements a iterator over the induced connected subgraphs of the input (di)graph. An induced subgraph of a graph is another graph, formed from a subset of the vertices of the graph and all of the edges connecting pairs of vertices in that subset (Wikipedia article Induced_subgraph).

As for method *connected_components()*, edge orientation is ignored. Hence, the directed graph with a single arc $0 \to 1$ is considered connected.

INPUT:

- G – a *Graph* or a *DiGraph*; loops and multiple edges are allowed
- k – (optional) integer; maximum order of the connected subgraphs to report; by default, the method iterates over all connected subgraphs (equivalent to k == n)
- vertices_only – boolean (default: False); whether to return (Di)Graph or list of vertices. This parameter is ignored when induced is True.
- edges_only – boolean (default: False); whether to return (Di)Graph or list of edges. When vertices_only is True, this parameter is ignored.
- labels – boolean (default: False); whether to return labeled edges or not. This parameter is used only when vertices_only is False and edges_only is True.
- induced – boolean (default: True); whether to return induced connected sub(di)graph only or also non-induced sub(di)graphs. This parameter can be set to False for simple (di)graphs only.
- exactly_k – boolean (default: False); True if we only return graphs of order k, False if we return graphs of order at most k.

EXAMPLES:

```
sage: G = DiGraph([(1, 2), (2, 3), (3, 4), (4, 2)])
sage: list(G.connected_subgraph_iterator())
[Subgraph of (): Digraph on 1 vertex,
 Subgraph of (): Digraph on 2 vertices,
 Subgraph of (): Digraph on 3 vertices,
 Subgraph of (): Digraph on 4 vertices,
 Subgraph of (): Digraph on 3 vertices,
 Subgraph of (): Digraph on 1 vertex,
 Subgraph of (): Digraph on 2 vertices,
 Subgraph of (): Digraph on 3 vertices,
 Subgraph of (): Digraph on 2 vertices,
 Subgraph of (): Digraph on 1 vertex,
 Subgraph of (): Digraph on 2 vertices,
 Subgraph of (): Digraph on 1 vertex]
sage: list(G.connected_subgraph_iterator(vertices_only=True))
[[1], [1, 2], [1, 2, 3], [1, 2, 3, 4], [1, 2, 4],
 [2], [2, 3], [2, 3, 4], [2, 4], [3], [3, 4], [4]]
sage: list(G.connected_subgraph_iterator(k=2))
[Subgraph of (): Digraph on 1 vertex,
 Subgraph of (): Digraph on 2 vertices,
 Subgraph of (): Digraph on 1 vertex,
 Subgraph of (): Digraph on 2 vertices,
 Subgraph of (): Digraph on 2 vertices,
 Subgraph of (): Digraph on 1 vertex,
 Subgraph of (): Digraph on 2 vertices,
 Subgraph of (): Digraph on 1 vertex]
sage: list(G.connected_subgraph_iterator(k=3, vertices_only=True, exactly_k=True))
[[1, 2, 3], [1, 2, 4], [2, 3, 4]]
sage: list(G.connected_subgraph_iterator(k=2, vertices_only=True))
[[1], [1, 2], [2], [2, 3], [2, 4], [3], [3, 4], [4]]

sage: G = DiGraph([(1, 2), (2, 1)])
sage: list(G.connected_subgraph_iterator())
[Subgraph of (): Digraph on 1 vertex,
 Subgraph of (): Digraph on 2 vertices,
```

(continues on next page)

(continued from previous page)

```
 Subgraph of (): Digraph on 1 vertex]
sage: list(G.connected_subgraph_iterator(vertices_only=True))
[[1], [1, 2], [2]]

sage: G = graphs.CompleteGraph(3)
sage: len(list(G.connected_subgraph_iterator()))
7
sage: len(list(G.connected_subgraph_iterator(vertices_only=True)))
7
sage: len(list(G.connected_subgraph_iterator(edges_only=True)))
7
sage: len(list(G.connected_subgraph_iterator(induced=False)))
10

sage: G = DiGraph([(0, 1), (1, 0), (1, 2), (2, 1)])
sage: len(list(G.connected_subgraph_iterator()))
6
sage: len(list(G.connected_subgraph_iterator(vertices_only=True)))
6
sage: len(list(G.connected_subgraph_iterator(edges_only=True)))
6
sage: len(list(G.connected_subgraph_iterator(induced=False)))
18
```

```
>>> from sage.all import *
>>> G = DiGraph([(Integer(1), Integer(2)), (Integer(2), Integer(3)), (Integer(3),␣
↪Integer(4)), (Integer(4), Integer(2))])
>>> list(G.connected_subgraph_iterator())
[Subgraph of (): Digraph on 1 vertex,
 Subgraph of (): Digraph on 2 vertices,
 Subgraph of (): Digraph on 3 vertices,
 Subgraph of (): Digraph on 4 vertices,
 Subgraph of (): Digraph on 3 vertices,
 Subgraph of (): Digraph on 1 vertex,
 Subgraph of (): Digraph on 2 vertices,
 Subgraph of (): Digraph on 3 vertices,
 Subgraph of (): Digraph on 2 vertices,
 Subgraph of (): Digraph on 1 vertex,
 Subgraph of (): Digraph on 2 vertices,
 Subgraph of (): Digraph on 1 vertex]
>>> list(G.connected_subgraph_iterator(vertices_only=True))
[[1], [1, 2], [1, 2, 3], [1, 2, 3, 4], [1, 2, 4],
 [2], [2, 3], [2, 3, 4], [2, 4], [3], [3, 4], [4]]
>>> list(G.connected_subgraph_iterator(k=Integer(2)))
[Subgraph of (): Digraph on 1 vertex,
 Subgraph of (): Digraph on 2 vertices,
 Subgraph of (): Digraph on 1 vertex,
 Subgraph of (): Digraph on 2 vertices,
 Subgraph of (): Digraph on 2 vertices,
 Subgraph of (): Digraph on 1 vertex,
 Subgraph of (): Digraph on 2 vertices,
 Subgraph of (): Digraph on 1 vertex]
```

(continues on next page)

(continued from previous page)

```
>>> list(G.connected_subgraph_iterator(k=Integer(3), vertices_only=True, exactly_
↪k=True))
[[1, 2, 3], [1, 2, 4], [2, 3, 4]]
>>> list(G.connected_subgraph_iterator(k=Integer(2), vertices_only=True))
[[1], [1, 2], [2], [2, 3], [2, 4], [3], [3, 4], [4]]

>>> G = DiGraph([(Integer(1), Integer(2)), (Integer(2), Integer(1))])
>>> list(G.connected_subgraph_iterator())
[Subgraph of (): Digraph on 1 vertex,
 Subgraph of (): Digraph on 2 vertices,
 Subgraph of (): Digraph on 1 vertex]
>>> list(G.connected_subgraph_iterator(vertices_only=True))
[[1], [1, 2], [2]]

>>> G = graphs.CompleteGraph(Integer(3))
>>> len(list(G.connected_subgraph_iterator()))
7
>>> len(list(G.connected_subgraph_iterator(vertices_only=True)))
7
>>> len(list(G.connected_subgraph_iterator(edges_only=True)))
7
>>> len(list(G.connected_subgraph_iterator(induced=False)))
10

>>> G = DiGraph([(Integer(0), Integer(1)), (Integer(1), Integer(0)), (Integer(1),␣
↪Integer(2)), (Integer(2), Integer(1))])
>>> len(list(G.connected_subgraph_iterator()))
6
>>> len(list(G.connected_subgraph_iterator(vertices_only=True)))
6
>>> len(list(G.connected_subgraph_iterator(edges_only=True)))
6
>>> len(list(G.connected_subgraph_iterator(induced=False)))
18
```

sage.graphs.base.static_dense_graph.**is_strongly_regular**(*g*, *parameters=False*)

Check whether the graph is strongly regular.

A simple graph G is said to be strongly regular with parameters (n, k, λ, μ) if and only if:

- G has n vertices
- G is k-regular
- Any two adjacent vertices of G have λ common neighbors
- Any two non-adjacent vertices of G have μ common neighbors

By convention, the complete graphs, the graphs with no edges and the empty graph are not strongly regular.

See the Wikipedia article Strongly regular graph.

INPUT:

- `parameters` – boolean (default: `False`); whether to return the quadruple (n, k, λ, μ). If `parameters = False` (default), this method only returns `True` and `False` answers. If `parameters = True`, the `True` answers are replaced by quadruples (n, k, λ, μ). See definition above.

EXAMPLES:

Petersen's graph is strongly regular:

```
sage: g = graphs.PetersenGraph()
sage: g.is_strongly_regular()
True
sage: g.is_strongly_regular(parameters=True)
(10, 3, 0, 1)
```

```
>>> from sage.all import *
>>> g = graphs.PetersenGraph()
>>> g.is_strongly_regular()
True
>>> g.is_strongly_regular(parameters=True)
(10, 3, 0, 1)
```

And Clebsch's graph is too:

```
sage: g = graphs.ClebschGraph()
sage: g.is_strongly_regular()
True
sage: g.is_strongly_regular(parameters=True)
(16, 5, 0, 2)
```

```
>>> from sage.all import *
>>> g = graphs.ClebschGraph()
>>> g.is_strongly_regular()
True
>>> g.is_strongly_regular(parameters=True)
(16, 5, 0, 2)
```

But Chvatal's graph is not:

```
sage: g = graphs.ChvatalGraph()
sage: g.is_strongly_regular()
False
```

```
>>> from sage.all import *
>>> g = graphs.ChvatalGraph()
>>> g.is_strongly_regular()
False
```

Complete graphs are not strongly regular. (Issue #14297)

```
sage: g = graphs.CompleteGraph(5)
sage: g.is_strongly_regular()
False
```

```
>>> from sage.all import *
>>> g = graphs.CompleteGraph(Integer(5))
>>> g.is_strongly_regular()
False
```

Completements of complete graphs are not strongly regular:

```
sage: g = graphs.CompleteGraph(5).complement()
sage: g.is_strongly_regular()
False
```

```
>>> from sage.all import *
>>> g = graphs.CompleteGraph(Integer(5)).complement()
>>> g.is_strongly_regular()
False
```

The empty graph is not strongly regular:

```
sage: g = graphs.EmptyGraph()
sage: g.is_strongly_regular()
False
```

```
>>> from sage.all import *
>>> g = graphs.EmptyGraph()
>>> g.is_strongly_regular()
False
```

If the input graph has loops or multiedges an exception is raised:

```
sage: Graph([(1,1),(2,2)],loops=True).is_strongly_regular()
Traceback (most recent call last):
...
ValueError: This method is not known to work on graphs with
loops. Perhaps this method can be updated to handle them, but in the
meantime if you want to use it please disallow loops using
allow_loops().

sage: Graph([(1,2),(1,2)],multiedges=True).is_strongly_regular()
Traceback (most recent call last):
...
ValueError: This method is not known to work on graphs with
multiedges. Perhaps this method can be updated to handle them, but in
the meantime if you want to use it please disallow multiedges using
allow_multiple_edges().
```

```
>>> from sage.all import *
>>> Graph([(Integer(1),Integer(1)),(Integer(2),Integer(2))],loops=True).is_
↪strongly_regular()
Traceback (most recent call last):
...
ValueError: This method is not known to work on graphs with
loops. Perhaps this method can be updated to handle them, but in the
meantime if you want to use it please disallow loops using
allow_loops().

>>> Graph([(Integer(1),Integer(2)),(Integer(1),Integer(2))],multiedges=True).is_
↪strongly_regular()
Traceback (most recent call last):
```

(continues on next page)

(continued from previous page)

```
...
ValueError: This method is not known to work on graphs with
multiedges. Perhaps this method can be updated to handle them, but in
the meantime if you want to use it please disallow multiedges using
allow_multiple_edges().
```

sage.graphs.base.static_dense_graph.**is_triangle_free**(*G*, *certificate=False*)

Check whether G is triangle free.

INPUT:

- `G` – a Sage graph
- `certificate` – boolean (default: `False`); whether to return a triangle if one is found

EXAMPLES:

```
sage: from sage.graphs.base.static_dense_graph import is_triangle_free
sage: is_triangle_free(graphs.PetersenGraph())
True
sage: K4 = graphs.CompleteGraph(4)
sage: is_triangle_free(K4)
False
sage: b, certif = is_triangle_free(K4, certificate=True)
sage: K4.subgraph(certif).is_clique()
True
```

```
>>> from sage.all import *
>>> from sage.graphs.base.static_dense_graph import is_triangle_free
>>> is_triangle_free(graphs.PetersenGraph())
True
>>> K4 = graphs.CompleteGraph(Integer(4))
>>> is_triangle_free(K4)
False
>>> b, certif = is_triangle_free(K4, certificate=True)
>>> K4.subgraph(certif).is_clique()
True
```

sage.graphs.base.static_dense_graph.**triangles_count**(*G*)

Return the number of triangles containing v, for every v.

INPUT:

- `G` – a simple Sage graph

EXAMPLES:

```
sage: from sage.graphs.base.static_dense_graph import triangles_count
sage: triangles_count(graphs.PetersenGraph())
{0: 0, 1: 0, 2: 0, 3: 0, 4: 0, 5: 0, 6: 0, 7: 0, 8: 0, 9: 0}
sage: sum(triangles_count(graphs.CompleteGraph(15)).values()) == 3 * binomial(15,␣
↪3)        # needs sage.symbolic
True
```

```
>>> from sage.all import *
>>> from sage.graphs.base.static_dense_graph import triangles_count
>>> triangles_count(graphs.PetersenGraph())
{0: 0, 1: 0, 2: 0, 3: 0, 4: 0, 5: 0, 6: 0, 7: 0, 8: 0, 9: 0}
>>> sum(triangles_count(graphs.CompleteGraph(Integer(15))).values()) ==␣
↪Integer(3) * binomial(Integer(15), Integer(3))        # needs sage.symbolic
True
```

3.6 Static sparse graphs

3.6.1 What is the point ?

This class implements a Cython (di)graph structure made for efficiency. The graphs are *static*, i.e. no add/remove vertex/edges methods are available, nor can they easily or efficiently be implemented within this data structure.

The data structure, however, is made to save the maximum amount of computations for graph algorithms whose main operation is to *list the out-neighbours of a vertex* (which is precisely what BFS, DFS, distance computations and the flow-related stuff waste their life on).

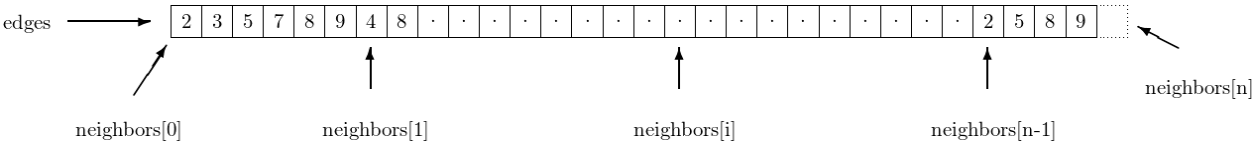
The code contained in this module is written C-style. The purpose is efficiency and simplicity.

For an overview of graph data structures in sage, see `overview`.

Author:

- Nathann Cohen (2011)

3.6.2 Data structure



The data structure is actually pretty simple and compact. `short_digraph` has five fields

- `n` – integer; the number of vertices in the graph
- `m` – integer; the number of edges in the graph
- `edges` – `uint32_t *`; array whose length is the number of edges of the graph
- `neighbors` – `uint32_t **`; this array has size $n + 1$, and describes how the data of `edges` should be read : the neighbors of vertex i are the elements of `edges` addressed by `neighbors[i]...neighbors[i+1]-1`. The element `neighbors[n]`, which corresponds to no vertex (they are numbered from 0 to $n - 1$) is present so that it remains easy to enumerate the neighbors of vertex $n - 1$: the last of them is the element addressed by `neighbors[n]-1`. The arrays `neighbors[i]` are guaranteed to be sorted so the time complexity for deciding if `g` has edge (u, v) is $O(\log m)$ using binary search.
- `edge_labels` – list; this cython list associates a label to each edge of the graph. If a given edge is represented by `edges[i]`, this its associated label can be found at `edge_labels[i]`. This object is usually NULL, unless the call to `init_short_digraph` explicitly requires the labels to be stored in the data structure.

In the example given above, vertex 0 has 2,3,5,7,8 and 9 as out-neighbors, but not 4, which is an out-neighbour of vertex 1. Vertex $n - 1$ has 2, 5, 8 and 9 as out-neighbors. `neighbors[n]` points toward the cell immediately *after* the end of `edges`, hence *outside of the allocated memory*. It is used to indicate the end of the outneighbors of vertex $n - 1$

Iterating over the edges

This is *the one thing* to have in mind when working with this data structure:

```
cdef list_edges(short_digraph g):
    cdef int i, j
    for i in range(g.n):
        for j in range(g.neighbors[i+1]-g.neighbors[i]):
            print("There is an edge from {} to {}".format(i, g.neighbors[i][j]))
```

Advantages

Two great points :

- The neighbors of a vertex are C types, and are contiguous in memory.
- Storing such graphs is incredibly cheaper than storing Python structures.

Well, I think it would be hard to have anything more efficient than that to enumerate out-neighbors in sparse graphs ! :-)

3.6.3 Technical details

- When creating a `short_digraph` from a `Graph` or `DiGraph` named `G`, the ith vertex corresponds *by default* to `list(G)[i]`. Using optional parameter `vertex_list`, you can specify the order of the vertices. Then i^{th} vertex will corresponds to `vertex_list[i]`.
- Some methods return `bitset_t` objects when lists could be expected. There is a very useful `bitset_list` function for this kind of problems :-)
- When the edges are labelled, most of the space taken by this graph is taken by edge labels. If no edge is labelled then this space is not allocated, but if *any* edge has a label then a (possibly empty) label is stored for each edge, which can double the memory needs.
- The data structure stores the number of edges, even though it appears that this number can be reconstructed with `g.neighbors[n]-g.neighbors[0]`. The trick is that not all elements of the `g.edges` array are necessarily used : when an undirected graph contains loops, only one entry of the array of size $2m$ is used to store it, instead of the expected two. Storing the number of edges is the only way to avoid an uselessly costly computation to obtain the number of edges of an undirected, looped, AND labelled graph (think of several loops on the same vertex with different labels).
- The codes of this module are well documented, and many answers can be found directly in the code.

3.6.4 Cython functions

`init_short_digraph(short_digraph g, G, edge_labelled, vertex_list)`	Initialize `short_digraph g` from a Sage (Di)Graph.
`int n_edges(short_digraph g)`	Return the number of edges in `g`
`int out_degree(short_digraph g, int i)`	Return the out-degree of vertex i in `g`
`has_edge(short_digraph g, int u, int v)`	Test the existence of an edge.
`edge_label(short_digraph g, int * edge)`	Return the label associated with a given edge
`init_empty_copy(short_digraph dst, short_digraph src)`	Allocate `dst` so that it can contain as many vertices and edges as `src`.
`init_reverse(short_digraph dst, short_digraph src)`	Initialize `dst` to a copy of `src` with all edges in the opposite direction.
`free_short_digraph(short_digraph g)`	Free the resources used by `g`

Connectivity

`can_be_reached_from(short_digraph g, int src, bitset_t reached)`

> Assuming `bitset_t reached` has size at least `g.n`, this method updates `reached` so that it represents the set of vertices that can be reached from `src` in `g`.

`strongly_connected_component_containing_vertex(short_digraph g, short_digraph g_reversed, int v, bitset_t scc)`

> Assuming `bitset_t reached` has size at least `g.n`, this method updates `scc` so that it represents the vertices of the strongly connected component containing `v` in `g`. The variable `g_reversed` is assumed to represent the reverse of `g`.

`tarjan_strongly_connected_components_C(short_digraph g, int *scc)`

> Assuming `scc` is already allocated and has size at least `g.n`, this method computes the strongly connected components of `g`, and outputs in `scc[v]` the number of the strongly connected component containing `v`. It returns the number of strongly connected components.

`strongly_connected_components_digraph_C(short_digraph g, int nscc, int *scc, short_digraph output):`

> Assuming `nscc` and `scc` are the outputs of `tarjan_strongly_connected_components_C` on `g`, this routine sets `output` to the strongly connected component digraph of `g`, that is, the vertices of `output` are the strongly connected components of `g` (numbers are provided by `scc`), and `output` contains an arc `(C1,C2)` if `g` has an arc from a vertex in `C1` to a vertex in `C2`.

3.6.5 What is this module used for ?

It is for instance used in the *sage.graphs.distances_all_pairs* module, and in the *strongly_connected_components()* method.

3.6.6 Python functions

These functions are available so that Python modules from Sage can call the Cython routines this module implements (as they cannot directly call methods with C arguments).

sage.graphs.base.static_sparse_graph.**spectral_radius**(*G*, *prec=1e-10*)

Return an interval of floating point number that encloses the spectral radius of this graph

The input graph `G` must be *strongly connected*.

INPUT:

- `prec` – (default: `1e-10`) an upper bound for the relative precision of the interval

The algorithm is iterative and uses an inequality valid for nonnegative matrices. Namely, if A is a nonnegative square matrix with Perron-Frobenius eigenvalue λ then the following inequality is valid for any vector x

$$\min_i \frac{(Ax)_i}{x_i} \leq \lambda \leq \max_i \frac{(Ax)_i}{x_i}$$

Note

The speed of convergence of the algorithm is governed by the spectral gap (the distance to the second largest modulus of other eigenvalues). If this gap is small, then this function might not be appropriate.

The algorithm is not smart and not parallel! It uses basic interval arithmetic and native floating point arithmetic.

EXAMPLES:

```
sage: from sage.graphs.base.static_sparse_graph import spectral_radius

sage: G = DiGraph([(0,0),(0,1),(1,0)], loops=True)
sage: phi = (RR(1) + RR(5).sqrt() ) / 2
sage: phi  # abs tol 1e-14
1.618033988749895
sage: e_min, e_max = spectral_radius(G, 1e-14)
sage: e_min, e_max     # abs tol 1e-14
(1.618033988749894, 1.618033988749896)
sage: (e_max - e_min)  # abs tol 1e-14
1e-14
sage: e_min < phi < e_max
True
```

```
>>> from sage.all import *
>>> from sage.graphs.base.static_sparse_graph import spectral_radius

>>> G = DiGraph([(Integer(0),Integer(0)),(Integer(0),Integer(1)),(Integer(1),
↪Integer(0))], loops=True)
>>> phi = (RR(Integer(1)) + RR(Integer(5)).sqrt() ) / Integer(2)
>>> phi  # abs tol 1e-14
```

(continues on next page)

(continued from previous page)

```
1.618033988749895
>>> e_min, e_max = spectral_radius(G, RealNumber('1e-14'))
>>> e_min, e_max     # abs tol 1e-14
(1.618033988749894, 1.618033988749896)
>>> (e_max - e_min)  # abs tol 1e-14
1e-14
>>> e_min < phi < e_max
True
```

This function also works for graphs:

```
sage: G = Graph([(0,1),(0,2),(1,2),(1,3),(2,4),(3,4)])
sage: e_min, e_max = spectral_radius(G, 1e-14)
sage: e = max(G.adjacency_matrix().charpoly().roots(AA, multiplicities=False))  #␣
↪needs sage.modules sage.rings.number_field
sage: e_min < e < e_max                                                         #␣
↪needs sage.modules sage.rings.number_field sage.symbolic
True

sage: G.spectral_radius()  # abs tol 1e-9
(2.48119430408, 2.4811943041)
```

```
>>> from sage.all import *
>>> G = Graph([(Integer(0),Integer(1)),(Integer(0),Integer(2)),(Integer(1),
↪Integer(2)),(Integer(1),Integer(3)),(Integer(2),Integer(4)),(Integer(3),
↪Integer(4))])
>>> e_min, e_max = spectral_radius(G, RealNumber('1e-14'))
>>> e = max(G.adjacency_matrix().charpoly().roots(AA, multiplicities=False))  #␣
↪needs sage.modules sage.rings.number_field
>>> e_min < e < e_max                                                         #␣
↪needs sage.modules sage.rings.number_field sage.symbolic
True

>>> G.spectral_radius()  # abs tol 1e-9
(2.48119430408, 2.4811943041)
```

A larger example:

```
sage: G = DiGraph()
sage: G.add_edges((i,i+1) for i in range(200))
sage: G.add_edge(200,0)
sage: G.add_edge(1,0)
sage: e_min, e_max = spectral_radius(G, 0.00001)
sage: p = G.adjacency_matrix(sparse=True).charpoly()
sage: p
x^201 - x^199 - 1
sage: r = p.roots(AA, multiplicities=False)[0]                                  #␣
↪needs sage.rings.number_field
sage: e_min < r < e_max                                                         #␣
↪needs sage.rings.number_field
True
```

```
>>> from sage.all import *
>>> G = DiGraph()
>>> G.add_edges((i,i+Integer(1)) for i in range(Integer(200)))
>>> G.add_edge(Integer(200),Integer(0))
>>> G.add_edge(Integer(1),Integer(0))
>>> e_min, e_max = spectral_radius(G, RealNumber('0.00001'))
>>> p = G.adjacency_matrix(sparse=True).charpoly()
>>> p
x^201 - x^199 - 1
>>> r = p.roots(AA, multiplicities=False)[Integer(0)]
↪     # needs sage.rings.number_field
>>> e_min < r < e_max
↪needs sage.rings.number_field
True
```

A much larger example:

```
sage: G = DiGraph(100000)
sage: r = list(range(100000))
sage: while not G.is_strongly_connected():
....:     shuffle(r)
....:     G.add_edges(enumerate(r), loops=False)
sage: spectral_radius(G, 1e-10)  # random  # long time
(1.9997956006500042, 1.9998043797692782)
```

```
>>> from sage.all import *
>>> G = DiGraph(Integer(100000))
>>> r = list(range(Integer(100000)))
>>> while not G.is_strongly_connected():
...     shuffle(r)
...     G.add_edges(enumerate(r), loops=False)
>>> spectral_radius(G, RealNumber('1e-10'))  # random  # long time
(1.9997956006500042, 1.9998043797692782)
```

The algorithm takes care of multiple edges:

```
sage: G = DiGraph(2,loops=True,multiedges=True)
sage: G.add_edges([(0,0),(0,0),(0,1),(1,0)])
sage: spectral_radius(G, 1e-14)  # abs tol 1e-14
(2.414213562373094, 2.414213562373095)
sage: max(G.adjacency_matrix().eigenvalues(AA))
↪needs sage.modules sage.rings.number_field
2.414213562373095?
```

```
>>> from sage.all import *
>>> G = DiGraph(Integer(2),loops=True,multiedges=True)
>>> G.add_edges([(Integer(0),Integer(0)),(Integer(0),Integer(0)),(Integer(0),
↪Integer(1)),(Integer(1),Integer(0))])
>>> spectral_radius(G, RealNumber('1e-14'))  # abs tol 1e-14
(2.414213562373094, 2.414213562373095)
>>> max(G.adjacency_matrix().eigenvalues(AA))
↪needs sage.modules sage.rings.number_field
2.414213562373095?
```

Some bipartite graphs:

```
sage: G = Graph([(0,1),(0,3),(2,3)])
sage: G.spectral_radius()  # abs tol 1e-10
(1.6180339887253428, 1.6180339887592732)

sage: G = DiGraph([(0,1),(0,3),(2,3),(3,0),(1,0),(1,2)])
sage: G.spectral_radius() # abs tol 1e-10
(1.5537739740270458, 1.553773974033029)

sage: G = graphs.CompleteBipartiteGraph(1,3)
sage: G.spectral_radius()  # abs tol 1e-10
(1.7320508075688772, 1.7320508075688774)
```

```
>>> from sage.all import *
>>> G = Graph([(Integer(0),Integer(1)),(Integer(0),Integer(3)),(Integer(2),
↪Integer(3))])
>>> G.spectral_radius()  # abs tol 1e-10
(1.6180339887253428, 1.6180339887592732)

>>> G = DiGraph([(Integer(0),Integer(1)),(Integer(0),Integer(3)),(Integer(2),
↪Integer(3)),(Integer(3),Integer(0)),(Integer(1),Integer(0)),(Integer(1),
↪Integer(2))])
>>> G.spectral_radius() # abs tol 1e-10
(1.5537739740270458, 1.553773974033029)

>>> G = graphs.CompleteBipartiteGraph(Integer(1),Integer(3))
>>> G.spectral_radius()  # abs tol 1e-10
(1.7320508075688772, 1.7320508075688774)
```

sage.graphs.base.static_sparse_graph.**strongly_connected_components_digraph**(G)

Return the digraph of the strongly connected components (SCCs).

This routine is used to test `strongly_connected_components_digraph_C`, but it is not used by the Sage digraph. It outputs a pair `[g_scc,scc]`, where `g_scc` is the SCC digraph of g, `scc` is a dictionary associating to each vertex `v` the number of the SCC of `v`, as it appears in `g_scc`.

EXAMPLES:

```
sage: from sage.graphs.base.static_sparse_graph import strongly_connected_
↪components_digraph
sage: strongly_connected_components_digraph(digraphs.Path(3))
(Digraph on 3 vertices, {0: 2, 1: 1, 2: 0})
sage: strongly_connected_components_digraph(DiGraph(4))
(Digraph on 4 vertices, {0: 0, 1: 1, 2: 2, 3: 3})
```

```
>>> from sage.all import *
>>> from sage.graphs.base.static_sparse_graph import strongly_connected_
↪components_digraph
>>> strongly_connected_components_digraph(digraphs.Path(Integer(3)))
(Digraph on 3 vertices, {0: 2, 1: 1, 2: 0})
>>> strongly_connected_components_digraph(DiGraph(Integer(4)))
(Digraph on 4 vertices, {0: 0, 1: 1, 2: 2, 3: 3})
```

sage.graphs.base.static_sparse_graph.**tarjan_strongly_connected_components**(G)

Return the lists of vertices in each strongly connected components (SCCs).

This method implements the Tarjan algorithm to compute the strongly connected components of the digraph. It returns a list of lists of vertices, each list of vertices representing a strongly connected component.

The basic idea of the algorithm is this: a depth-first search (DFS) begins from an arbitrary start node (and subsequent DFSes are conducted on any nodes that have not yet been found). As usual with DFSes, the search visits every node of the graph exactly once, declining to revisit any node that has already been explored. Thus, the collection of search trees is a spanning forest of the graph. The strongly connected components correspond to the subtrees of this spanning forest that have no edge directed outside the subtree.

To recover these components, during the DFS, we keep the index of a node, that is, the position in the DFS tree, and the lowlink: as soon as the subtree rooted at v has been fully explored, the lowlink of v is the smallest index reachable from v passing from descendants of v. If the subtree rooted at v has been fully explored, and the index of v equals the lowlink of v, that whole subtree is a new SCC.

For more information, see the Wikipedia article Tarjan%27s_strongly_connected_components_algorithm.

EXAMPLES:

```
sage: from sage.graphs.base.static_sparse_graph import tarjan_strongly_connected_
↪components
sage: tarjan_strongly_connected_components(digraphs.Path(3))
[[2], [1], [0]]
sage: D = DiGraph( { 0 : [1, 3], 1 : [2], 2 : [3], 4 : [5, 6], 5 : [6] } )
sage: D.connected_components(sort=True)
[[0, 1, 2, 3], [4, 5, 6]]
sage: D = DiGraph( { 0 : [1, 3], 1 : [2], 2 : [3], 4 : [5, 6], 5 : [6] } )
sage: D.strongly_connected_components()
[[3], [2], [1], [0], [6], [5], [4]]
sage: D.add_edge([2,0])
sage: D.strongly_connected_components()
[[3], [0, 1, 2], [6], [5], [4]]
sage: D = DiGraph([('a','b'), ('b','c'), ('c', 'd'), ('d', 'b'), ('c', 'e')])
sage: [sorted(scc) for scc in D.strongly_connected_components()]
[['e'], ['b', 'c', 'd'], ['a']]
```

```
>>> from sage.all import *
>>> from sage.graphs.base.static_sparse_graph import tarjan_strongly_connected_
↪components
>>> tarjan_strongly_connected_components(digraphs.Path(Integer(3)))
[[2], [1], [0]]
>>> D = DiGraph( { Integer(0) : [Integer(1), Integer(3)], Integer(1) :␣
↪[Integer(2)], Integer(2) : [Integer(3)], Integer(4) : [Integer(5), Integer(6)],␣
↪Integer(5) : [Integer(6)] } )
>>> D.connected_components(sort=True)
[[0, 1, 2, 3], [4, 5, 6]]
>>> D = DiGraph( { Integer(0) : [Integer(1), Integer(3)], Integer(1) :␣
↪[Integer(2)], Integer(2) : [Integer(3)], Integer(4) : [Integer(5), Integer(6)],␣
↪Integer(5) : [Integer(6)] } )
>>> D.strongly_connected_components()
[[3], [2], [1], [0], [6], [5], [4]]
>>> D.add_edge([Integer(2),Integer(0)])
>>> D.strongly_connected_components()
```

(continues on next page)

(continued from previous page)

```
[[3], [0, 1, 2], [6], [5], [4]]
>>> D = DiGraph([('a','b'), ('b','c'), ('c', 'd'), ('d', 'b'), ('c', 'e')])
>>> [sorted(scc) for scc in D.strongly_connected_components()]
[['e'], ['b', 'c', 'd'], ['a']]
```

sage.graphs.base.static_sparse_graph.**triangles_count**(G)

Return the number of triangles containing v, for every v.

INPUT:

- G – a graph

EXAMPLES:

```
sage: from sage.graphs.base.static_sparse_graph import triangles_count
sage: triangles_count(graphs.PetersenGraph())
{0: 0, 1: 0, 2: 0, 3: 0, 4: 0, 5: 0, 6: 0, 7: 0, 8: 0, 9: 0}
sage: sum(triangles_count(graphs.CompleteGraph(15)).values()) == 3*binomial(15,3)␣
↪          # needs sage.symbolic
True
```

```
>>> from sage.all import *
>>> from sage.graphs.base.static_sparse_graph import triangles_count
>>> triangles_count(graphs.PetersenGraph())
{0: 0, 1: 0, 2: 0, 3: 0, 4: 0, 5: 0, 6: 0, 7: 0, 8: 0, 9: 0}
>>> sum(triangles_count(graphs.CompleteGraph(Integer(15))).values()) ==␣
↪Integer(3)*binomial(Integer(15),Integer(3))           # needs sage.symbolic
True
```

3.7 Static sparse graph backend

This module implement a immutable sparse graph backend using the data structure from *sage.graphs.base.static_sparse_graph*. It supports both directed and undirected graphs, as well as vertex/edge labels, loops and multiple edges. As it uses a very compact C structure it should be very small in memory.

As it is a sparse data structure, you can expect it to be very efficient when you need to list the graph's edge, or those incident to a vertex, but an adjacency test can be much longer than in a dense data structure (i.e. like in *sage.graphs.base.static_dense_graph*)

For an overview of graph data structures in sage, see *overview*.

3.7.1 Two classes

This module implements two classes

- *StaticSparseCGraph* extends *CGraph* and is a Cython class that manages the definition/deallocation of the short_digraph structure. It does not know anything about labels on vertices.
- *StaticSparseBackend* extends *CGraphBackend* and is a Python class that does know about vertex labels and contains an instance of *StaticSparseCGraph* as an internal variable. The input/output of its methods are labeled vertices, which it translates to integer id before forwarding them to the *StaticSparseCGraph* instance.

3.7.2 Classes and methods

class sage.graphs.base.static_sparse_backend.**StaticSparseBackend**

Bases: *CGraphBackend*

A graph *backend* for static sparse graphs.

INPUT:

- `G` – a *Graph* or *DiGraph* object, or `None` (default: `None`)
- `loops` – boolean (default: `False`); whether loops are allowed
- `multiedges` – boolean (default: `False`); whether multiple edges are allowed
- `sort` – boolean (default: `True`); whether vertices are sorted before building the backend
- `vertex_list` – (optional) list of vertices; used when `G` is `None` to define vertex order explicitly
- `edges` – (optional) iterable of edges/arcs used when `G` is `None`
- `directed` – boolean (default: `None`); mandatory when `G` is `None`
- `edge_labelled` – boolean (default: `False`); whether labels are stored when `G` is `None`

EXAMPLES:

```
sage: D = sage.graphs.base.sparse_graph.SparseGraphBackend(9)
sage: D.add_edge(0, 1, None, False)
sage: list(D.iterator_edges(range(9), True))
[(0, 1, None)]
```

```
>>> from sage.all import *
>>> D = sage.graphs.base.sparse_graph.SparseGraphBackend(Integer(9))
>>> D.add_edge(Integer(0), Integer(1), None, False)
>>> list(D.iterator_edges(range(Integer(9)), True))
[(0, 1, None)]
```

```
sage: from sage.graphs.base.static_sparse_backend import StaticSparseBackend
sage: g = StaticSparseBackend(graphs.PetersenGraph())
sage: list(g.iterator_edges([0], 1))
[(0, 1, None), (0, 4, None), (0, 5, None)]
```

```
>>> from sage.all import *
>>> from sage.graphs.base.static_sparse_backend import StaticSparseBackend
>>> g = StaticSparseBackend(graphs.PetersenGraph())
>>> list(g.iterator_edges([Integer(0)], Integer(1)))
[(0, 1, None), (0, 4, None), (0, 5, None)]
```

```
sage: g = DiGraph(digraphs.DeBruijn(4, 3), data_structure='static_sparse')
sage: gi = DiGraph(g, data_structure='static_sparse')
sage: gi.edges(sort=True)[0]
('000', '000', '0')
sage: sorted(gi.edges_incident('111'))
[('111', '110', '0'),
('111', '111', '1'),
('111', '112', '2'),
('111', '113', '3')]
```

(continues on next page)

(continued from previous page)

```
sage: set(g.edges(sort=False)) == set(gi.edges(sort=False))                    #␣
↪needs sage.combinat
True
```

```
>>> from sage.all import *
>>> g = DiGraph(digraphs.DeBruijn(Integer(4), Integer(3)), data_structure='static_
↪sparse')
>>> gi = DiGraph(g, data_structure='static_sparse')
>>> gi.edges(sort=True)[Integer(0)]
('000', '000', '0')
>>> sorted(gi.edges_incident('111'))
[('111', '110', '0'),
('111', '111', '1'),
('111', '112', '2'),
('111', '113', '3')]

>>> set(g.edges(sort=False)) == set(gi.edges(sort=False))                    # needs␣
↪sage.combinat
True
```

```
sage: g = graphs.PetersenGraph()
sage: gi = Graph(g, data_structure='static_sparse')
sage: g == gi
True
sage: set(g.edges(sort=False)) == set(gi.edges(sort=False))
True
```

```
>>> from sage.all import *
>>> g = graphs.PetersenGraph()
>>> gi = Graph(g, data_structure='static_sparse')
>>> g == gi
True
>>> set(g.edges(sort=False)) == set(gi.edges(sort=False))
True
```

```
sage: gi = Graph({ 0: {1: 1}, 1: {2: 1}, 2: {3: 1}, 3: {4: 2}, 4: {0: 2}}, data_
↪structure='static_sparse')
sage: (0, 4, 2) in gi.edges(sort=False)
True
sage: gi.has_edge(0, 4)
True
```

```
>>> from sage.all import *
>>> gi = Graph({ Integer(0): {Integer(1): Integer(1)}, Integer(1): {Integer(2):␣
↪Integer(1)}, Integer(2): {Integer(3): Integer(1)}, Integer(3): {Integer(4):␣
↪Integer(2)}, Integer(4): {Integer(0): Integer(2)}}, data_structure='static_
↪sparse')
>>> (Integer(0), Integer(4), Integer(2)) in gi.edges(sort=False)
True
>>> gi.has_edge(Integer(0), Integer(4))
```

(continues on next page)

(continued from previous page)

```
True
```

```
sage: G = Graph({1:{2:28, 6:10}, 2:{3:16, 7:14}, 3:{4:12}, 4:{5:22, 7:18}, 5:
↪{6:25, 7:24}})
sage: GI = Graph({1:{2:28, 6:10}, 2:{3:16, 7:14}, 3:{4:12}, 4:{5:22, 7:18}, 5:
↪{6:25, 7:24}}, data_structure='static_sparse')
sage: G == GI
True
```

```
>>> from sage.all import *
>>> G = Graph({Integer(1):{Integer(2):Integer(28), Integer(6):Integer(10)},␣
↪Integer(2):{Integer(3):Integer(16), Integer(7):Integer(14)}, Integer(3):
↪{Integer(4):Integer(12)}, Integer(4):{Integer(5):Integer(22),␣
↪Integer(7):Integer(18)}, Integer(5):{Integer(6):Integer(25),␣
↪Integer(7):Integer(24)}})
>>> GI = Graph({Integer(1):{Integer(2):Integer(28), Integer(6):Integer(10)},␣
↪Integer(2):{Integer(3):Integer(16), Integer(7):Integer(14)}, Integer(3):
↪{Integer(4):Integer(12)}, Integer(4):{Integer(5):Integer(22),␣
↪Integer(7):Integer(18)}, Integer(5):{Integer(6):Integer(25),␣
↪Integer(7):Integer(24)}}, data_structure='static_sparse')
>>> G == GI
True
```

```
sage: G = graphs.OddGraph(4)
sage: d = G.diameter()
sage: H = G.distance_graph(list(range(d + 1)))
sage: HI = Graph(H, data_structure='static_sparse')
sage: HI.size() == len(HI.edges(sort=False))
True
```

```
>>> from sage.all import *
>>> G = graphs.OddGraph(Integer(4))
>>> d = G.diameter()
>>> H = G.distance_graph(list(range(d + Integer(1))))
>>> HI = Graph(H, data_structure='static_sparse')
>>> HI.size() == len(HI.edges(sort=False))
True
```

```
sage: g = Graph({1: {1: [1, 2, 3]}}, data_structure='static_sparse')
sage: g.size()
3
sage: g.order()
1
sage: g.vertices(sort=False)
[1]
sage: g.edges(sort=True)
[(1, 1, 1), (1, 1, 2), (1, 1, 3)]
```

```
>>> from sage.all import *
>>> g = Graph({Integer(1): {Integer(1): [Integer(1), Integer(2), Integer(3)]}},␣
```

(continues on next page)

(continued from previous page)

```
↪data_structure='static_sparse')
>>> g.size()
3
>>> g.order()
1
>>> g.vertices(sort=False)
[1]
>>> g.edges(sort=True)
[(1, 1, 1), (1, 1, 2), (1, 1, 3)]
```

Issue #15810 is fixed:

```
sage: DiGraph({1: {2: ['a', 'b'], 3: ['c']}, 2: {3: ['d']}}, immutable=True).is_
↪directed_acyclic()
True
```

```
>>> from sage.all import *
>>> DiGraph({Integer(1): {Integer(2): ['a', 'b'], Integer(3): ['c']}, Integer(2):
↪{Integer(3): ['d']}}, immutable=True).is_directed_acyclic()
True
```

Direct initialization from raw vertices/edges:

```
sage: from sage.graphs.base.static_sparse_backend import StaticSparseBackend
sage: b = StaticSparseBackend(vertex_list=['b', 'a'],
....:                         edges=[('b', 'a')],
....:                         directed=False, sort=False)
sage: list(b.iterator_edges(['b'], True))
[('b', 'a', None)]

sage: b = StaticSparseBackend(vertex_list=['d', 'b', 'a', 'c'],
....:                         edges=[('d', 'a', 3), ('b', 'a', 2), ('c', 'a', 1)],
....:                         directed=True, edge_labelled=True, sort=False)
sage: list(b.iterator_in_edges(['a'], True))
[('d', 'a', 3), ('b', 'a', 2), ('c', 'a', 1)]
```

```
>>> from sage.all import *
>>> from sage.graphs.base.static_sparse_backend import StaticSparseBackend
>>> b = StaticSparseBackend(vertex_list=['b', 'a'],
...                         edges=[('b', 'a')],
...                         directed=False, sort=False)
>>> list(b.iterator_edges(['b'], True))
[('b', 'a', None)]

>>> b = StaticSparseBackend(vertex_list=['d', 'b', 'a', 'c'],
...                         edges=[('d', 'a', Integer(3)), ('b', 'a', Integer(2)),␣
↪('c', 'a', Integer(1))],
...                         directed=True, edge_labelled=True, sort=False)
>>> list(b.iterator_in_edges(['a'], True))
[('d', 'a', 3), ('b', 'a', 2), ('c', 'a', 1)]
```

Error cases for direct initialization:

```
sage: StaticSparseBackend(vertex_list=[0], edges=[])
Traceback (most recent call last):
...
TypeError: missing required argument 'directed'
sage: StaticSparseBackend(vertex_list=[0, 0], edges=[], directed=False)
Traceback (most recent call last):
...
ValueError: vertex_list has duplicates
sage: StaticSparseBackend(vertex_list=[0], edges=[(0, 1)], directed=True)
Traceback (most recent call last):
...
ValueError: edge contains a vertex not in vertex_list
```

```
>>> from sage.all import *
>>> StaticSparseBackend(vertex_list=[Integer(0)], edges=[])
Traceback (most recent call last):
...
TypeError: missing required argument 'directed'
>>> StaticSparseBackend(vertex_list=[Integer(0), Integer(0)], edges=[],␣
↪directed=False)
Traceback (most recent call last):
...
ValueError: vertex_list has duplicates
>>> StaticSparseBackend(vertex_list=[Integer(0)], edges=[(Integer(0),␣
↪Integer(1))], directed=True)
Traceback (most recent call last):
...
ValueError: edge contains a vertex not in vertex_list
```

add_edge(*u*, *v*, *l*, *directed*)

Add an edge to the graph. No way.

add_edges(*edges*, *directed*, *remove_loops=False*)

Add edges to the graph. No way.

INPUT:

- `edges` – a list of edges (or not?)

add_vertex(*v*)

Add a vertex to the graph. No way

INPUT:

- `v` – a vertex (or not?)

add_vertices(*vertices*)

Add vertices to the graph. No way.

INPUT:

- `vertices` – a list of vertices (or not?)

allows_loops(*value=None*)

Return whether the graph allows loops.

INPUT:

- `value` – only useful for compatibility with other graph backends, where this method can be used to define this boolean. This method raises an exception if `value` is not equal to `None`.

degree(*v*, *directed*)

Return the degree of a vertex.

INPUT:

- `v` – a vertex
- `directed` – boolean; whether to take into account the orientation of this graph in counting the degree of `v`

EXAMPLES:

```
sage: g = Graph(graphs.PetersenGraph(), data_structure='static_sparse')
sage: g.degree(0)
3
```

```
>>> from sage.all import *
>>> g = Graph(graphs.PetersenGraph(), data_structure='static_sparse')
>>> g.degree(Integer(0))
3
```

Issue #17225 about the degree of a vertex with a loop:

```
sage: Graph({0: [0]}, immutable=True).degree(0)
2
sage: Graph({0: [0], 1: [0, 1, 1, 1]}, immutable=True).degree(1)
7
```

```
>>> from sage.all import *
>>> Graph({Integer(0): [Integer(0)]}, immutable=True).degree(Integer(0))
2
>>> Graph({Integer(0): [Integer(0)], Integer(1): [Integer(0), Integer(1),␣
↪Integer(1), Integer(1)]}, immutable=True).degree(Integer(1))
7
```

del_edge(*u*, *v*, *l*, *directed*)

Delete an edge of the graph. No way.

del_edges(*edges*, *directed*)

Delete edges of the graph. No way.

del_vertex(*v*)

Delete a vertex from the graph. No way

INPUT:

- `v` – a vertex (or not?)

del_vertices(*vertices*)

Delete vertices from the graph. No way

INPUT:

- `vertices` – a list of vertices (or not?)

get_edge_label(*u*, *v*)

Return the edge label for `(u, v)`.

INPUT:

- `u`, `v` – two vertices

has_edge(*u*, *v*, *l*)

Return whether this graph has edge `(u, v)` with label `l`.

If `l` is `None`, return whether this graph has an edge `(u, v)` with any label.

INPUT:

- `u`, `v` – two vertices
- `l` – a label

has_vertex(*v*)

Test if the vertex belongs to the graph.

INPUT:

- `v` – a vertex (or not?)

in_degree(*v*)

Return the in-degree of a vertex.

INPUT:

- `v` – a vertex

EXAMPLES:

```
sage: g = DiGraph(graphs.PetersenGraph(), data_structure='static_sparse')
sage: g.in_degree(0)
3
```

```
>>> from sage.all import *
>>> g = DiGraph(graphs.PetersenGraph(), data_structure='static_sparse')
>>> g.in_degree(Integer(0))
3
```

iterator_edges(*vertices*, *labels*)

Iterate over the graph's edges.

INPUT:

- `vertices` – list; only returns the edges incident to at least one vertex of `vertices`
- `labels` – boolean; whether to return edge labels too

iterator_in_edges(*vertices*, *labels*)

Iterate over the incoming edges incident to a sequence of vertices.

INPUT:

- `vertices` – list of vertices
- `labels` – whether to return labels too

iterator_in_nbrs(*v*)

Iterate over the in-neighbors of a vertex.

INPUT:

- `v` – a vertex

EXAMPLES:

```
sage: g = DiGraph(graphs.PetersenGraph(), data_structure='static_sparse')
sage: g.neighbors_in(0)
[1, 4, 5]
```

```
>>> from sage.all import *
>>> g = DiGraph(graphs.PetersenGraph(), data_structure='static_sparse')
>>> g.neighbors_in(Integer(0))
[1, 4, 5]
```

iterator_nbrs(*v*)

Iterate over the neighbors of a vertex.

INPUT:

- `v` – a vertex

EXAMPLES:

```
sage: g = Graph(graphs.PetersenGraph(), data_structure='static_sparse')
sage: g.neighbors(0)
[1, 4, 5]
```

```
>>> from sage.all import *
>>> g = Graph(graphs.PetersenGraph(), data_structure='static_sparse')
>>> g.neighbors(Integer(0))
[1, 4, 5]
```

iterator_out_edges(*vertices*, *labels*)

Iterate over the outbound edges incident to a sequence of vertices.

INPUT:

- `vertices` – list of vertices
- `labels` – whether to return labels too

iterator_out_nbrs(*v*)

Iterate over the out-neighbors of a vertex.

INPUT:

- `v` – a vertex

EXAMPLES:

```
sage: g = DiGraph(graphs.PetersenGraph(), data_structure='static_sparse')
sage: g.neighbors_out(0)
[1, 4, 5]
```

```
>>> from sage.all import *
>>> g = DiGraph(graphs.PetersenGraph(), data_structure='static_sparse')
>>> g.neighbors_out(Integer(0))
[1, 4, 5]
```

iterator_unsorted_edges(*vertices*, *labels*)

alias of *iterator_edges()*.

iterator_verts(*vertices*)

Iterate over the vertices.

INPUT:

- `vertices` – list of objects; the method will only return the elements of the graph which are contained in `vertices`. It's not very efficient. If `vertices` is equal to `None`, all the vertices are returned.

multiple_edges(*value=None*)

Return whether the graph allows multiple edges.

INPUT:

- `value` – only useful for compatibility with other graph backends, where this method can be used to define this boolean. This method raises an exception if `value` is not equal to `None`.

n_edges(*directed*)

Return the number of edges.

INPUT:

- `directed` – boolean; whether to consider the graph as directed or not

n_vertices()

Return the number of vertices.

num_edges(*directed*)

alias of *n_edges()*.

num_verts()

alias of *n_vertices()*.

out_degree(*v*)

Return the out-degree of a vertex.

INPUT:

- `v` – a vertex

EXAMPLES:

```
sage: g = DiGraph(graphs.PetersenGraph(), data_structure='static_sparse')
sage: g.out_degree(0)
3
```

```
>>> from sage.all import *
>>> g = DiGraph(graphs.PetersenGraph(), data_structure='static_sparse')
>>> g.out_degree(Integer(0))
3
```

relabel(*perm*, *directed*)

Relabel the graphs' vertices. No way.

set_edge_label(*u*, *v*, *l*, *directed*)

Set edge label. No way.

class sage.graphs.base.static_sparse_backend.**StaticSparseCGraph**

Bases: *CGraph*

CGraph class based on the sparse graph data structure *static sparse graphs*.

add_vertex(*k*)

Add a vertex to the graph. No way.

del_vertex(*k*)

Remove a vertex from the graph. No way.

has_arc(*u*, *v*)

Test if uv is an edge of the graph

INPUT:

- u, v – integers

has_vertex(*v*)

Test if a vertex belongs to the graph

INPUT:

- n – integer

in_degree(*u*)

Return the in-degree of a vertex

INPUT:

- u – a vertex

in_neighbors(*u*)

Return the in-neighbors of a vertex.

INPUT:

- u – a vertex

out_degree(*u*)

Return the out-degree of a vertex

INPUT:

- u – a vertex

out_neighbors(*u*)

List the out-neighbors of a vertex.

INPUT:

- u – a vertex

verts()

Return the list of vertices.

3.8 Backends for Sage (di)graphs

This module implements *GenericGraphBackend* (the base class for backends).

Any graph backend must redefine the following methods (for which *GenericGraphBackend* raises a NotImplementedError)

add_edge()	Add an edge (u, v) to self, with label l.
add_edges()	Add a sequence of edges to self.
add_vertex()	Add a labelled vertex to self.
add_vertices()	Add labelled vertices to self.
degree()	Return the total number of vertices incident to v.
in_degree()	Return the in-degree of v
out_degree()	Return the out-degree of v
del_edge()	Delete the edge (u, v) with label l.
del_vertex()	Delete a labelled vertex in self.
del_vertices()	Delete labelled vertices in self.
get_edge_label()	Return the edge label of (u, v).
has_edge()	True if self has an edge (u, v) with label l.
has_vertex()	True if self has a vertex with label v.
iterator_edges()	Iterate over the edges incident to a sequence of vertices.
iterator_in_edges()	Iterate over the incoming edges incident to a sequence of vertices.
iterator_out_edges()	Iterate over the outbound edges incident to a sequence of vertices.
iterator_nbrs()	Iterate over the vertices adjacent to v.
iterator_in_nbrs()	Iterate over the in-neighbors of vertex v.
iterator_out_nbrs()	Iterate over the out-neighbors of vertex v.
iterator_verts()	Iterate over the vertices v with labels in verts.
loops()	Get/set whether or not self allows loops.
multiple_edges()	Get/set whether or not self allows multiple edges.
name()	Get/set name of self.
n_edges()	The number of edges in self
n_vertices()	The number of vertices in self
relabel()	Relabel the vertices of self by a permutation.
set_edge_label()	Label the edge (u, v) by l.

For an overview of graph data structures in sage, see *overview*.

3.8.1 Classes and methods

class sage.graphs.base.graph_backends.**GenericGraphBackend**

Bases: SageObject

A generic wrapper for the backend of a graph.

Various graph classes use extensions of this class. Note, this graph has a number of placeholder functions, so the doctests are rather silly.

add_edge(*u*, *v*, *l*, *directed*)

Add an edge (u, v) to self, with label l.

If directed is True, this is interpreted as an arc from u to v.

INPUT:

- u, v – vertices

- `l` – edge label
- `directed` – boolean

add_edges(*edges*, *directed*)

Add a sequence of edges to `self`.

If `directed` is `True`, these are interpreted as arcs.

INPUT:

- `edges` – list/iterator of edges to be added
- `directed` – boolean

add_vertex(*name*)

Add a labelled vertex to `self`.

INPUT:

- `name` – vertex label

OUTPUT: if `name=None`, the new vertex name is returned, `None` otherwise

add_vertices(*vertices*)

Add labelled vertices to `self`.

INPUT:

- `vertices` – iterator of vertex labels; a new label is created, used and returned in the output list for all `None` values in `vertices`

OUTPUT:

Generated names of new vertices if there is at least one `None` value present in `vertices`. `None` otherwise.

EXAMPLES:

```
sage: G = sage.graphs.base.graph_backends.GenericGraphBackend()
sage: G.add_vertices([1,2,3])
Traceback (most recent call last):
...
NotImplementedError
```

```
>>> from sage.all import *
>>> G = sage.graphs.base.graph_backends.GenericGraphBackend()
>>> G.add_vertices([Integer(1),Integer(2),Integer(3)])
Traceback (most recent call last):
...
NotImplementedError
```

degree(*v*, *directed*)

Return the total number of vertices incident to v.

INPUT:

- `v` – a vertex label
- `directed` – boolean

OUTPUT: degree of v

del_edge(*u*, *v*, *l*, *directed*)

Delete the edge (u, v) with label l.

INPUT:

- `u`, `v` – vertices
- `l` – edge label
- `directed` – boolean

del_vertex(*v*)

Delete a labelled vertex in `self`.

INPUT:

- `v` – vertex label

del_vertices(*vertices*)

Delete labelled vertices in `self`.

INPUT:

- `vertices` – iterator of vertex labels

get_edge_label(*u*, *v*)

Return the edge label of (u, v).

INPUT:

- `u`, `v` – vertex labels

OUTPUT:

label of (u, v)

has_edge(*u*, *v*, *l*)

Check whether `self` has an edge (u, v) with label l.

INPUT:

- `u`, `v` – vertex labels
- `l` – label

OUTPUT: boolean

has_vertex(*v*)

Check whether `self` has a vertex with label v.

INPUT:

- `v` – vertex label

OUTPUT: boolean

in_degree(*v*)

Return the in-degree of v.

INPUT:

- `v` – a vertex label

iterator_edges(*vertices*, *labels*)

Iterate over the edges incident to a sequence of vertices.

Edges are assumed to be undirected.

This method returns an iterator over the edges (u, v) such that either u or v is in `vertices` and the edge (u, v) is in `self`.

INPUT:

- `vertices` – list of vertex labels
- `labels` – boolean

OUTPUT:

a generator which yields edges, with or without labels depending on the labels parameter.

iterator_in_edges(*vertices*, *labels*)

Iterate over the incoming edges incident to a sequence of vertices.

This method returns an iterator over the edges (u, v) such that v is in `vertices` and the edge (u, v) is in `self`.

INPUT:

- `vertices` – list of vertex labels
- `labels` – boolean

OUTPUT: a generator which yields edges, with or without labels depending on the labels parameter

iterator_in_nbrs(*v*)

Iterate over the in-neighbors of vertex v.

This method returns an iterator over the vertices u such that the edge (u, v) is in `self` (that is, predecessors of v).

INPUT:

- `v` – vertex label

OUTPUT: a generator which yields vertex labels

iterator_nbrs(*v*)

Iterate over the vertices adjacent to v.

This method returns an iterator over the vertices u such that either the edge (u, v) or the edge (v, u) is in `self` (that is, neighbors of v).

INPUT:

- `v` – vertex label

OUTPUT: a generator which yields vertex labels

iterator_out_edges(*vertices*, *labels*)

Iterate over the outbound edges incident to a sequence of vertices.

This method returns an iterator over the edges (v, u) such that v is in `vertices` and the edge (v, u) is in `self`.

INPUT:

- `vertices` – list of vertex labels
- `labels` – boolean

OUTPUT:

a generator which yields edges, with or without labels depending on the labels parameter.

iterator_out_nbrs(*v*)

Iterate over the out-neighbors of v.

This method returns an iterator over the vertices u such that the edge (v, u) is in `self` (that is, successors of v).

INPUT:

- `v` – vertex label

OUTPUT: a generator which yields vertex labels

iterator_verts(*verts*)

Iterate over the vertices v with labels in `verts`.

INPUT:

- `verts` – vertex labels

OUTPUT: a generator which yields vertices

loops(*new=None*)

Get/set whether or not `self` allows loops.

INPUT:

- `new` – can be a boolean (in which case it sets the value) or `None`, in which case the current value is returned. It is set to `None` by default.

multiple_edges(*new=None*)

Get/set whether or not `self` allows multiple edges.

INPUT:

- `new` – can be a boolean (in which case it sets the value) or `None`, in which case the current value is returned. It is set to `None` by default.

n_edges(*directed*)

Return the number of edges in `self`.

INPUT:

- `directed` – boolean

n_vertices()

Return the number of vertices in `self`.

name(*new=None*)

Get/set name of `self`.

INPUT:

- `new` – can be a string (in which case it sets the value) or `None`, in which case the current value is returned. It is set to `None` by default.

num_edges(*directed*)

alias of *n_edges()*.

num_verts()

alias of *n_vertices()*.

out_degree(*v*)

Return the out-degree of v.

INPUT:

- `v` – a vertex label

relabel(*perm*, *directed*)

Relabel the vertices of `self` by a permutation.

INPUT:

- `perm` – permutation
- `directed` – boolean

set_edge_label(*u*, *v*, *l*, *directed*)

Label the edge (u, v) by l.

INPUT:

- `u`, `v` – vertices
- `l` – edge label
- `directed` – boolean

`sage.graphs.base.graph_backends.`**unpickle_graph_backend**(*directed*, *vertices*, *edges*, *kwds*)

Return a backend from its pickled data.

This methods is defined because Python's pickling mechanism can only build objects from a pair `(f,args)` by running `f(*args)`. In particular, there is apparently no way to define a `**kwargs` (i.e. define the value of keyword arguments of `f`), which means that one must know the order of all arguments of `f` (here, `f` is *Graph* or *DiGraph*).

As a consequence, this means that the order cannot change in the future, which is something we cannot swear.

INPUT:

- `directed` – boolean
- `vertices` – list of vertices
- `edges` – list of edges
- `kwds` – any dictionary whose keywords will be forwarded to the graph constructor

This function builds a *Graph* or *DiGraph* from its data, and returns the `_backend` attribute of this object.

EXAMPLES:

```
sage: from sage.graphs.base.graph_backends import unpickle_graph_backend
sage: b = unpickle_graph_backend(0, [0, 1, 2, 3], [(0, 3, 'label'), (0, 0, 1)], {
↪'loops': True})
sage: b
<sage.graphs.base.sparse_graph.SparseGraphBackend object at ...>
sage: list(b.iterator_edges(range(4), True))
[(0, 0, 1), (0, 3, 'label')]
```

```
>>> from sage.all import *
>>> from sage.graphs.base.graph_backends import unpickle_graph_backend
>>> b = unpickle_graph_backend(Integer(0), [Integer(0), Integer(1), Integer(2),␣
↪Integer(3)], [(Integer(0), Integer(3), 'label'), (Integer(0), Integer(0),␣
↪Integer(1))], {'loops': True})
```

(continues on next page)

(continued from previous page)

```
>>> b
<sage.graphs.base.sparse_graph.SparseGraphBackend object at ...>
>>> list(b.iterator_edges(range(Integer(4)), True))
[(0, 0, 1), (0, 3, 'label')]
```

3.9 Interface to run Boost algorithms

Wrapper for a Boost graph. The Boost graphs are Cython C++ variables, and they cannot be converted to Python objects: as a consequence, only functions defined with cdef are able to create, read, modify, and delete these graphs.

A very important feature of Boost graph library is that all object are generic: for instance, adjacency lists can be stored using different data structures, and (most of) the functions work with all implementations provided. This feature is implemented in our interface using fused types: however, Cython's support for fused types is still experimental, and some features are missing. For instance, there cannot be nested generic function calls, and no variable can have a generic type, apart from the arguments of a generic function.

All the input functions use pointers, because otherwise we might have problems with `delete()`.

Basic Boost Graph operations:

clustering_coeff()	Return the clustering coefficient of all vertices in the graph.
edge_connectivity()	Return the edge connectivity of the graph.
dominator_tree()	Return a dominator tree of the graph.
bandwidth_heuristics()	Use heuristics to approximate the bandwidth of the graph.
min_spanning_tree()	Compute a minimum spanning tree of a (weighted) graph.
shortest_paths()	Use Dijkstra or Bellman-Ford algorithm to compute the single-source shortest paths.
johnson_shortest_paths()	Use Johnson algorithm to compute the all-pairs shortest paths.
floyd_warshall_shortest_paths()	Use Floyd-Warshall algorithm to compute the all-pairs shortest paths.
johnson_closeness_centrality()	Use Johnson algorithm to compute the closeness centrality of all vertices.
blocks_and_cut_vertices()	Use Tarjan's algorithm to compute the blocks and cut vertices of the graph.
min_cycle_basis()	Return a minimum weight cycle basis of the input graph.

3.9.1 Functions

sage.graphs.base.boost_graph.**bandwidth_heuristics**(*g*, *algorithm='cuthill_mckee'*)

Use Boost heuristics to approximate the bandwidth of the input graph.

The bandwidth $bw(M)$ of a matrix M is the smallest integer k such that all nonzero entries of M are at distance k from the diagonal. The bandwidth $bw(g)$ of an undirected graph g is the minimum bandwidth of the adjacency matrix of g, over all possible relabellings of its vertices (for more information, see the *bandwidth* module).

Unfortunately, exactly computing the bandwidth is NP-hard (and an exponential algorithm is implemented in Sagemath in routine *bandwidth()*). Here, we implement two heuristics to find good orderings: Cuthill-McKee, reverse Cuthill-McKee (also known as RCM) and King.

This function works only in undirected graphs, and its running time is $O(md_{max} \log d_{max})$ for the Cuthill-McKee ordering, and $O(md_{max}^2 \log d_{max})$ for the King ordering, where m is the number of edges, and d_{max} is the maximum degree in the graph.

INPUT:

- `g` – the input Sage graph
- `algorithm` – string (default: `'cuthill_mckee'`); the heuristic used to compute the ordering among `'cuthill_mckee'`, `'reverse_cuthill_mckee'` and `'king'`

OUTPUT:

A pair `[bandwidth, ordering]`, where `ordering` is the ordering of vertices, `bandwidth` is the bandwidth of that specific ordering (which is not necessarily the bandwidth of the graph, because this is a heuristic).

EXAMPLES:

```
sage: from sage.graphs.base.boost_graph import bandwidth_heuristics
sage: bandwidth_heuristics(graphs.PathGraph(10))
(1, [0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
sage: bandwidth_heuristics(graphs.GridGraph([3,3]))
(3, [(0, 0), (1, 0), (0, 1), (2, 0), (1, 1), (0, 2), (2, 1), (1, 2), (2, 2)])
sage: bandwidth_heuristics(graphs.GridGraph([3,3]), algorithm='reverse_cuthill_
↪mckee')
(3, [(2, 2), (1, 2), (2, 1), (0, 2), (1, 1), (2, 0), (0, 1), (1, 0), (0, 0)])
sage: bandwidth_heuristics(graphs.GridGraph([3,3]), algorithm='king')
(3, [(0, 0), (1, 0), (0, 1), (2, 0), (1, 1), (0, 2), (2, 1), (1, 2), (2, 2)])
```

```
>>> from sage.all import *
>>> from sage.graphs.base.boost_graph import bandwidth_heuristics
>>> bandwidth_heuristics(graphs.PathGraph(Integer(10)))
(1, [0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
>>> bandwidth_heuristics(graphs.GridGraph([Integer(3),Integer(3)]))
(3, [(0, 0), (1, 0), (0, 1), (2, 0), (1, 1), (0, 2), (2, 1), (1, 2), (2, 2)])
>>> bandwidth_heuristics(graphs.GridGraph([Integer(3),Integer(3)]), algorithm=
↪'reverse_cuthill_mckee')
(3, [(2, 2), (1, 2), (2, 1), (0, 2), (1, 1), (2, 0), (0, 1), (1, 0), (0, 0)])
>>> bandwidth_heuristics(graphs.GridGraph([Integer(3),Integer(3)]), algorithm=
↪'king')
(3, [(0, 0), (1, 0), (0, 1), (2, 0), (1, 1), (0, 2), (2, 1), (1, 2), (2, 2)])
```

sage.graphs.base.boost_graph.**blocks_and_cut_vertices**(*g*, *forbidden_vertices=None*)

Compute the blocks and cut vertices of the graph.

This method uses the implementation of Tarjan's algorithm available in the Boost library .

INPUT:

- `g` – the input Sage graph
- `forbidden_vertices` – list (default: `None`); set of vertices to avoid during the search

OUTPUT:

A 2-dimensional vector with m+1 rows (m is the number of biconnected components), where each of the first m rows correspond to vertices in a block, and the last row is the list of cut vertices.

See also

- *sage.graphs.generic_graph.GenericGraph.blocks_and_cut_vertices()*

EXAMPLES:

```
sage: from sage.graphs.base.boost_graph import blocks_and_cut_vertices
sage: g = graphs.KrackhardtKiteGraph()
sage: blocks_and_cut_vertices(g)
([[8, 9], [7, 8], [0, 1, 2, 3, 5, 4, 6, 7]], [8, 7])

sage: G = Graph([(0,1,{'name':'a','weight':1}),
....:            (0,2,{'name':'b','weight':3}),
....:            (1,2,{'name':'b','weight':1})])
sage: blocks_and_cut_vertices(G)
([[0, 1, 2]], [])
```

```
>>> from sage.all import *
>>> from sage.graphs.base.boost_graph import blocks_and_cut_vertices
>>> g = graphs.KrackhardtKiteGraph()
>>> blocks_and_cut_vertices(g)
([[8, 9], [7, 8], [0, 1, 2, 3, 5, 4, 6, 7]], [8, 7])

>>> G = Graph([(Integer(0),Integer(1),{'name':'a','weight':Integer(1)}),
...            (Integer(0),Integer(2),{'name':'b','weight':Integer(3)}),
...            (Integer(1),Integer(2),{'name':'b','weight':Integer(1)})])
>>> blocks_and_cut_vertices(G)
([[0, 1, 2]], [])
```

Check the behavior of parameter `forbidden_vertices`:

```
sage: G = graphs.WindmillGraph(4, 3)
sage: blocks_and_cut_vertices(G)
([[0, 1, 2, 3], [0, 4, 5, 6], [0, 7, 8, 9]], [0])
sage: blocks_and_cut_vertices(G, forbidden_vertices=[0])
([[1, 2, 3], [4, 5, 6], [7, 8, 9]], [])
sage: blocks_and_cut_vertices(G, forbidden_vertices=[1])
([[0, 2, 3], [0, 4, 5, 6], [0, 7, 8, 9]], [0])
sage: G = graphs.PathGraph(3)
sage: blocks_and_cut_vertices(G)
([[1, 2], [0, 1]], [1])
sage: blocks_and_cut_vertices(G, forbidden_vertices=[0])
([[1, 2]], [])
sage: blocks_and_cut_vertices(G, forbidden_vertices=[1])
([[0], [2]], [])
```

```
>>> from sage.all import *
>>> G = graphs.WindmillGraph(Integer(4), Integer(3))
>>> blocks_and_cut_vertices(G)
([[0, 1, 2, 3], [0, 4, 5, 6], [0, 7, 8, 9]], [0])
>>> blocks_and_cut_vertices(G, forbidden_vertices=[Integer(0)])
([[1, 2, 3], [4, 5, 6], [7, 8, 9]], [])
```

(continues on next page)

(continued from previous page)

```
>>> blocks_and_cut_vertices(G, forbidden_vertices=[Integer(1)])
([[0, 2, 3], [0, 4, 5, 6], [0, 7, 8, 9]], [0])
>>> G = graphs.PathGraph(Integer(3))
>>> blocks_and_cut_vertices(G)
([[1, 2], [0, 1]], [1])
>>> blocks_and_cut_vertices(G, forbidden_vertices=[Integer(0)])
([[1, 2]], [])
>>> blocks_and_cut_vertices(G, forbidden_vertices=[Integer(1)])
([[0], [2]], [])
```

sage.graphs.base.boost_graph.**clustering_coeff**(*g*, *vertices=None*)

Compute the clustering coefficient of the input graph, using Boost.

See also

sage.graphs.generic_graph.GenericGraph.clustering_coeff()

INPUT:

- `g` – the input Sage Graph
- `vertices` – list (default: `None`); the list of vertices to analyze (if `None`, compute the clustering coefficient of all vertices)

OUTPUT: a pair `(average_clustering_coefficient, clust_of_v)`, where `average_clustering_coefficient` is the average clustering of the vertices in variable `vertices`, `clust_of_v` is a dictionary that associates to each vertex its clustering coefficient. If `vertices` is `None`, all vertices are considered.

EXAMPLES:

Computing the clustering coefficient of a clique:

```
sage: from sage.graphs.base.boost_graph import clustering_coeff
sage: g = graphs.CompleteGraph(5)
sage: clustering_coeff(g)
(1.0, {0: 1.0, 1: 1.0, 2: 1.0, 3: 1.0, 4: 1.0})
sage: clustering_coeff(g, vertices = [0,1,2])
(1.0, {0: 1.0, 1: 1.0, 2: 1.0})
```

```
>>> from sage.all import *
>>> from sage.graphs.base.boost_graph import clustering_coeff
>>> g = graphs.CompleteGraph(Integer(5))
>>> clustering_coeff(g)
(1.0, {0: 1.0, 1: 1.0, 2: 1.0, 3: 1.0, 4: 1.0})
>>> clustering_coeff(g, vertices = [Integer(0),Integer(1),Integer(2)])
(1.0, {0: 1.0, 1: 1.0, 2: 1.0})
```

Of a non-clique graph with triangles:

```
sage: g = graphs.IcosahedralGraph()
sage: clustering_coeff(g, vertices=[1,2,3])
(0.5, {1: 0.5, 2: 0.5, 3: 0.5})
```

```
>>> from sage.all import *
>>> g = graphs.IcosahedralGraph()
>>> clustering_coeff(g, vertices=[Integer(1),Integer(2),Integer(3)])
(0.5, {1: 0.5, 2: 0.5, 3: 0.5})
```

With labels:

```
sage: g.relabel(list("abcdefghiklm"))
sage: clustering_coeff(g, vertices='abde')
(0.5, {'a': 0.5, 'b': 0.5, 'd': 0.5, 'e': 0.5})
```

```
>>> from sage.all import *
>>> g.relabel(list("abcdefghiklm"))
>>> clustering_coeff(g, vertices='abde')
(0.5, {'a': 0.5, 'b': 0.5, 'd': 0.5, 'e': 0.5})
```

sage.graphs.base.boost_graph.**diameter**(*G*, *algorithm=None*, *source=None*, *weight_function=None*, *check_weight=True*)

Return the diameter of G.

This method returns Infinity if the digraph is not strongly connected. It can also quickly return a lower bound on the diameter using the `2Dsweep` scheme.

INPUT:

- `G` – the input sage digraph
- `algorithm` – string (default: `None`); specifies the algorithm to use among:
  - `'2Dsweep'` – computes lower bound on the diameter of a weighted directed graph using the weighted version of the algorithm proposed in [Broder2000]. See the code's documentation for more details.
  - `'DiFUB'` – computes the diameter of a weighted directed graph using the weighted version of the algorithm proposed in [CGLM2012]. See the code's documentation for more details.
- `source` – (default: `None`) vertex from which to start the computation. If `source==None`, an arbitrary vertex of the graph is chosen. Raise an error if the initial vertex is not in G.
- `weight_function` – function (default: `None`); a function that associates a weight to each edge. If `None` (default), the weights of `G` are used, if `G.weighted()==True`, otherwise all edges have weight 1.
- `check_weight` – boolean (default: `True`); if `True`, we check that the `weight_function` outputs a number for each edge

EXAMPLES:

```
sage: from sage.graphs.base.boost_graph import diameter
sage: G = DiGraph([(0, 1, 2), (1, 0, -1)])
sage: diameter(G, algorithm='DiFUB')
1.0
sage: diameter(G, algorithm='DiFUB', weight_function=lambda e:e[2])
2.0
sage: G = DiGraph([(0, 1, -1), (1, 0, 2)])
sage: diameter(G, algorithm='DiFUB', weight_function=lambda e:e[2])
2.0
```

```
>>> from sage.all import *
>>> from sage.graphs.base.boost_graph import diameter
>>> G = DiGraph([(Integer(0), Integer(1), Integer(2)), (Integer(1), Integer(0), -
↪Integer(1))])
>>> diameter(G, algorithm='DiFUB')
1.0
>>> diameter(G, algorithm='DiFUB', weight_function=lambda e:e[Integer(2)])
2.0
>>> G = DiGraph([(Integer(0), Integer(1), -Integer(1)), (Integer(1), Integer(0),␣
↪Integer(2))])
>>> diameter(G, algorithm='DiFUB', weight_function=lambda e:e[Integer(2)])
2.0
```

sage.graphs.base.boost_graph.**diameter_DHV**(*g*, *weight_function=None*, *check_weight=True*)

Return the diameter of weighted graph g.

This method computes the diameter of undirected graph using the algorithm proposed in [Dragan2018].

This method returns Infinity if graph is not connected.

INPUT:

- `g` – the input Sage graph
- `weight_function` – function (default: `None`); a function that associates a weight to each edge. If `None` (default), the weights of `g` are used, if `g.weighted()==True`, otherwise all edges have weight 1.
- `check_weight` – boolean (default: `True`); if `True`, we check that the `weight_function` outputs a number for each edge

EXAMPLES:

```
sage: from sage.graphs.base.boost_graph import diameter_DHV
sage: G = graphs.ButterflyGraph()
sage: diameter_DHV(G)
2.0
```

```
>>> from sage.all import *
>>> from sage.graphs.base.boost_graph import diameter_DHV
>>> G = graphs.ButterflyGraph()
>>> diameter_DHV(G)
2.0
```

sage.graphs.base.boost_graph.**dominator_tree**(*g*, *root*, *return_dict=False*, *reverse=False*)

Use Boost to compute the dominator tree of `g`, rooted at `root`.

A node d dominates a node n if every path from the entry node `root` to n must go through d. The immediate dominator of a node n is the unique node that strictly dominates n but does not dominate any other node that dominates n. A dominator tree is a tree where each node's children are those nodes it immediately dominates. For more information, see the Wikipedia article Dominator_(graph_theory).

If the graph is connected and undirected, the parent of a vertex v is:

- the root if v is in the same biconnected component as the root;
- the first cut vertex in a path from v to the root, otherwise.

If the graph is not connected, the dominator tree of the whole graph is equal to the dominator tree of the connected component of the root.

If the graph is directed, computing a dominator tree is more complicated, and it needs time $O(m \log m)$, where m is the number of edges. The implementation provided by Boost is the most general one, so it needs time $O(m \log m)$ even for undirected graphs.

INPUT:

- `g` – the input Sage (Di)Graph
- `root` – the root of the dominator tree
- `return_dict` – boolean (default: `False`); if `True`, the function returns a dictionary associating to each vertex its parent in the dominator tree. If `False` (default), it returns the whole tree, as a `Graph` or a `DiGraph`.
- `reverse` – boolean (default: `False`); when set to `True`, computes the dominator tree in the reverse graph

OUTPUT:

The dominator tree, as a graph or as a dictionary, depending on the value of `return_dict`. If the output is a dictionary, it will contain `None` in correspondence of `root` and of vertices that are not reachable from `root`. If the output is a graph, it will not contain vertices that are not reachable from `root`.

EXAMPLES:

An undirected grid is biconnected, and its dominator tree is a star (everyone's parent is the root):

```
sage: g = graphs.GridGraph([2,2]).dominator_tree((0,0))
sage: g.to_dictionary()
{(0, 0): [(0, 1), (1, 0), (1, 1)], (0, 1): [(0, 0)], (1, 0): [(0, 0)], (1, 1):␣
↪[(0, 0)]}
```

```
>>> from sage.all import *
>>> g = graphs.GridGraph([Integer(2),Integer(2)]).dominator_tree((Integer(0),
↪Integer(0)))
>>> g.to_dictionary()
{(0, 0): [(0, 1), (1, 0), (1, 1)], (0, 1): [(0, 0)], (1, 0): [(0, 0)], (1, 1):␣
↪[(0, 0)]}
```

If the graph is made by two 3-cycles C_1, C_2 connected by an edge (v, w), with $v \in C_1$, $w \in C_2$, the cut vertices are v and w, the biconnected components are C_1, C_2, and the edge (v, w). If the root is in C_1, the parent of each vertex in C_1 is the root, the parent of w is v, and the parent of each vertex in C_2 is w:

```
sage: G = 2 * graphs.CycleGraph(3)
sage: v = 0
sage: w = 3
sage: G.add_edge(v,w)
sage: G.dominator_tree(1, return_dict=True)
{0: 1, 1: None, 2: 1, 3: 0, 4: 3, 5: 3}
```

```
>>> from sage.all import *
>>> G = Integer(2) * graphs.CycleGraph(Integer(3))
>>> v = Integer(0)
>>> w = Integer(3)
>>> G.add_edge(v,w)
>>> G.dominator_tree(Integer(1), return_dict=True)
{0: 1, 1: None, 2: 1, 3: 0, 4: 3, 5: 3}
```

An example with a directed graph:

```
sage: g = digraphs.Circuit(10).dominator_tree(5)
sage: g.to_dictionary()
{0: [1], 1: [2], 2: [3], 3: [4], 4: [], 5: [6], 6: [7], 7: [8], 8: [9], 9: [0]}
sage: g = digraphs.Circuit(10).dominator_tree(5, reverse=True)
sage: g.to_dictionary()
{0: [9], 1: [0], 2: [1], 3: [2], 4: [3], 5: [4], 6: [], 7: [6], 8: [7], 9: [8]}
```

```
>>> from sage.all import *
>>> g = digraphs.Circuit(Integer(10)).dominator_tree(Integer(5))
>>> g.to_dictionary()
{0: [1], 1: [2], 2: [3], 3: [4], 4: [], 5: [6], 6: [7], 7: [8], 8: [9], 9: [0]}
>>> g = digraphs.Circuit(Integer(10)).dominator_tree(Integer(5), reverse=True)
>>> g.to_dictionary()
{0: [9], 1: [0], 2: [1], 3: [2], 4: [3], 5: [4], 6: [], 7: [6], 8: [7], 9: [8]}
```

If the output is a dictionary:

```
sage: graphs.GridGraph([2,2]).dominator_tree((0,0), return_dict=True)
{(0, 0): None, (0, 1): (0, 0), (1, 0): (0, 0), (1, 1): (0, 0)}
```

```
>>> from sage.all import *
>>> graphs.GridGraph([Integer(2),Integer(2)]).dominator_tree((Integer(0),
↪Integer(0)), return_dict=True)
{(0, 0): None, (0, 1): (0, 0), (1, 0): (0, 0), (1, 1): (0, 0)}
```

sage.graphs.base.boost_graph.**eccentricity_DHV**(*g*, *vertex_list=None*, *weight_function=None*, *check_weight=True*)

Return the vector of eccentricities using the algorithm of [Dragan2018].

The array returned is of length n, and by default its i-th component is the eccentricity of the i-th vertex in `g.vertices(sort=True)`, if `vertex_list is None`, otherwise `ecc[i]` is the eccentricity of vertex `vertex_list[i]`.

The algorithm proposed in [Dragan2018] is based on the observation that for all nodes $v, w \in V$, we have $\max(ecc[v] - d(v,w), d(v,w)) \leq ecc[w] \leq ecc[v] + d(v,w)$. Also the algorithm iteratively improves upper and lower bounds on the eccentricity of each vertex until no further improvements can be done.

INPUT:

- `g` – the input Sage graph
- `vertex_list` – list (default: `None`); a list of n vertices specifying a mapping from $(0, \ldots, n-1)$ to vertex labels in g. When set, `ecc[i]` is the eccentricity of vertex `vertex_list[i]`. When `vertex_list` is `None`, `ecc[i]` is the eccentricity of vertex `g.vertices(sort=True)[i]`.
- `weight_function` – function (default: `None`); a function that associates a weight to each edge. If `None` (default), the weights of `g` are used, if `g.weighted()==True`, otherwise all edges have weight 1.
- `check_weight` – boolean (default: `True`); if `True`, we check that the `weight_function` outputs a number for each edge

EXAMPLES:

```
sage: from sage.graphs.base.boost_graph import eccentricity_DHV
sage: G = graphs.BullGraph()
sage: eccentricity_DHV(G)
[2.0, 2.0, 2.0, 3.0, 3.0]
```

```
>>> from sage.all import *
>>> from sage.graphs.base.boost_graph import eccentricity_DHV
>>> G = graphs.BullGraph()
>>> eccentricity_DHV(G)
[2.0, 2.0, 2.0, 3.0, 3.0]
```

sage.graphs.base.boost_graph.**edge_connectivity**(*g*)

Compute the edge connectivity of the input graph, using Boost.

OUTPUT: a pair `(ec, edges)`, where `ec` is the edge connectivity, `edges` is the list of edges in a minimum cut.

See also

sage.graphs.generic_graph.GenericGraph.edge_connectivity()

EXAMPLES:

Computing the edge connectivity of a clique:

```
sage: from sage.graphs.base.boost_graph import edge_connectivity
sage: g = graphs.CompleteGraph(5)
sage: edge_connectivity(g)
(4, [(0, 1), (0, 2), (0, 3), (0, 4)])
```

```
>>> from sage.all import *
>>> from sage.graphs.base.boost_graph import edge_connectivity
>>> g = graphs.CompleteGraph(Integer(5))
>>> edge_connectivity(g)
(4, [(0, 1), (0, 2), (0, 3), (0, 4)])
```

Vertex-labeled graphs:

```
sage: from sage.graphs.base.boost_graph import edge_connectivity
sage: g = graphs.GridGraph([2,2])
sage: edge_connectivity(g)
(2, [((0, 0), (0, 1)), ((0, 0), (1, 0))])
```

```
>>> from sage.all import *
>>> from sage.graphs.base.boost_graph import edge_connectivity
>>> g = graphs.GridGraph([Integer(2),Integer(2)])
>>> edge_connectivity(g)
(2, [((0, 0), (0, 1)), ((0, 0), (1, 0))])
```

sage.graphs.base.boost_graph.**floyd_warshall_shortest_paths**(*g*, *weight_function=None*, *distances=True*, *predecessors=False*)

Use Floyd-Warshall algorithm to solve the all-pairs-shortest-paths.

This routine outputs the distance between each pair of vertices and the predecessors matrix (depending on the values of boolean `distances` and `predecessors`) using a dictionary of dictionaries. This method should be preferred only if the graph is dense. If the graph is sparse the much faster johnson_shortest_paths should be used.

The time-complexity is $O(n^3 + nm)$, where n is the number of nodes and m the number of edges. The factor nm in the complexity is added only when `predecessors` is set to `True`.

INPUT:

- `g` – the input Sage graph
- `weight_function` – function (default: `None`); a function that associates a weight to each edge. If `None` (default), the weights of `g` are used, if `g.weighted()==True`, otherwise all edges have weight 1.
- `distances` – boolean (default: `True`); whether to return the dictionary of shortest distances
- `predecessors` – boolean (default: `False`); whether to return the predecessors matrix

OUTPUT:

Depending on the input, this function return the dictionary of predecessors, the dictionary of distances, or a pair of dictionaries `(distances, predecessors)` where `distance[u][v]` denotes the distance of a shortest path from u to v and `predecessors[u][v]` indicates the predecessor of w on a shortest path from u to v.

EXAMPLES:

Undirected graphs:

```
sage: from sage.graphs.base.boost_graph import floyd_warshall_shortest_paths
sage: g = Graph([(0,1,1),(1,2,2),(1,3,4),(2,3,1)], weighted=True)
sage: floyd_warshall_shortest_paths(g)
{0: {0: 0, 1: 1, 2: 3, 3: 4},
 1: {0: 1, 1: 0, 2: 2, 3: 3},
 2: {0: 3, 1: 2, 2: 0, 3: 1},
 3: {0: 4, 1: 3, 2: 1, 3: 0}}
sage: expected = {0: {0: None, 1: 0, 2: 1, 3: 2},
....:             1: {0: 1, 1: None, 2: 1, 3: 2},
....:             2: {0: 1, 1: 2, 2: None, 3: 2},
....:             3: {0: 1, 1: 2, 2: 3, 3: None}}
sage: floyd_warshall_shortest_paths(g, distances=False, predecessors=True) ==␣
↪expected
True
```

```
>>> from sage.all import *
>>> from sage.graphs.base.boost_graph import floyd_warshall_shortest_paths
>>> g = Graph([(Integer(0),Integer(1),Integer(1)),(Integer(1),Integer(2),
↪Integer(2)),(Integer(1),Integer(3),Integer(4)),(Integer(2),Integer(3),
↪Integer(1))], weighted=True)
>>> floyd_warshall_shortest_paths(g)
{0: {0: 0, 1: 1, 2: 3, 3: 4},
 1: {0: 1, 1: 0, 2: 2, 3: 3},
 2: {0: 3, 1: 2, 2: 0, 3: 1},
 3: {0: 4, 1: 3, 2: 1, 3: 0}}
>>> expected = {Integer(0): {Integer(0): None, Integer(1): Integer(0),␣
↪Integer(2): Integer(1), Integer(3): Integer(2)},
...             Integer(1): {Integer(0): Integer(1), Integer(1): None,␣
↪Integer(2): Integer(1), Integer(3): Integer(2)},
...             Integer(2): {Integer(0): Integer(1), Integer(1): Integer(2),␣
↪Integer(2): None, Integer(3): Integer(2)},
...             Integer(3): {Integer(0): Integer(1), Integer(1): Integer(2),␣
↪Integer(2): Integer(3), Integer(3): None}}
>>> floyd_warshall_shortest_paths(g, distances=False, predecessors=True) ==␣
↪expected
True
```

Directed graphs:

```
sage: g = DiGraph([(0,1,1),(1,2,-2),(1,3,4),(2,3,1)], weighted=True)
sage: floyd_warshall_shortest_paths(g)
{0: {0: 0, 1: 1, 2: -1, 3: 0},
 1: {1: 0, 2: -2, 3: -1},
 2: {2: 0, 3: 1},
 3: {3: 0}}
sage: g = DiGraph([(1,2,3),(2,3,2),(1,4,1),(4,2,1)], weighted=True)
sage: floyd_warshall_shortest_paths(g, distances=False, predecessors=True)
{1: {1: None, 2: 4, 3: 2, 4: 1},
 2: {2: None, 3: 2},
 3: {3: None},
 4: {2: 4, 3: 2, 4: None}}
```

```
>>> from sage.all import *
>>> g = DiGraph([(Integer(0),Integer(1),Integer(1)),(Integer(1),Integer(2),-
↪Integer(2)),(Integer(1),Integer(3),Integer(4)),(Integer(2),Integer(3),
↪Integer(1))], weighted=True)
>>> floyd_warshall_shortest_paths(g)
{0: {0: 0, 1: 1, 2: -1, 3: 0},
 1: {1: 0, 2: -2, 3: -1},
 2: {2: 0, 3: 1},
 3: {3: 0}}
>>> g = DiGraph([(Integer(1),Integer(2),Integer(3)),(Integer(2),Integer(3),
↪Integer(2)),(Integer(1),Integer(4),Integer(1)),(Integer(4),Integer(2),
↪Integer(1))], weighted=True)
>>> floyd_warshall_shortest_paths(g, distances=False, predecessors=True)
{1: {1: None, 2: 4, 3: 2, 4: 1},
 2: {2: None, 3: 2},
 3: {3: None},
 4: {2: 4, 3: 2, 4: None}}
```

sage.graphs.base.boost_graph.**johnson_closeness_centrality**(*g*, *weight_function=None*)

Use Johnson algorithm to compute the closeness centrality of all vertices.

This routine is preferable to *johnson_shortest_paths()* because it does not create a doubly indexed dictionary of distances, saving memory.

The time-complexity is $O(mn \log n)$, where n is the number of nodes and m is the number of edges.

INPUT:

- `g` – the input Sage graph
- `weight_function` – function (default: `None`); a function that associates a weight to each edge. If `None` (default), the weights of `g` are used, if `g.weighted()==True`, otherwise all edges have weight 1.

OUTPUT: a dictionary associating each vertex `v` to its closeness centrality

EXAMPLES:

Undirected graphs:

```
sage: from sage.graphs.base.boost_graph import johnson_closeness_centrality
sage: g = Graph([(0,1,1),(1,2,2),(1,3,4),(2,3,1)], weighted=True)
sage: johnson_closeness_centrality(g)
{0: 0.375, 1: 0.5, 2: 0.5, 3: 0.375}
```

```
>>> from sage.all import *
>>> from sage.graphs.base.boost_graph import johnson_closeness_centrality
>>> g = Graph([(Integer(0),Integer(1),Integer(1)),(Integer(1),Integer(2),
↪Integer(2)),(Integer(1),Integer(3),Integer(4)),(Integer(2),Integer(3),
↪Integer(1))], weighted=True)
>>> johnson_closeness_centrality(g)
{0: 0.375, 1: 0.5, 2: 0.5, 3: 0.375}
```

Directed graphs:

```
sage: from sage.graphs.base.boost_graph import johnson_closeness_centrality
sage: g = DiGraph([(0,1,1),(1,2,-2),(1,3,4),(2,3,1)], weighted=True)
sage: johnson_closeness_centrality(g)
{0: inf, 1: -0.4444444444444444, 2: 0.3333333333333333}
```

```
>>> from sage.all import *
>>> from sage.graphs.base.boost_graph import johnson_closeness_centrality
>>> g = DiGraph([(Integer(0),Integer(1),Integer(1)),(Integer(1),Integer(2),-
↪Integer(2)),(Integer(1),Integer(3),Integer(4)),(Integer(2),Integer(3),
↪Integer(1))], weighted=True)
>>> johnson_closeness_centrality(g)
{0: inf, 1: -0.4444444444444444, 2: 0.3333333333333333}
```

sage.graphs.base.boost_graph.**johnson_shortest_paths**(*g*, *weight_function=None*, *distances=True*, *predecessors=False*)

Use Johnson algorithm to solve the all-pairs-shortest-paths.

This routine outputs the distance between each pair of vertices and the predecessors matrix (depending on the values of boolean `distances` and `predecessors`) using a dictionary of dictionaries. It works on all kinds of graphs, but it is designed specifically for graphs with negative weights (otherwise there are more efficient algorithms, like Dijkstra).

The time-complexity is $O(mn \log n)$, where n is the number of nodes and m is the number of edges.

INPUT:

- `g` – the input Sage graph
- `weight_function` – function (default: `None`); a function that associates a weight to each edge. If `None` (default), the weights of `g` are used, if `g.weighted()==True`, otherwise all edges have weight 1.
- `distances` – boolean (default: `True`); whether to return the dictionary of shortest distances
- `predecessors` – boolean (default: `False`); whether to return the predecessors matrix

OUTPUT:

Depending on the input, this function return the dictionary of predecessors, the dictionary of distances, or a pair of dictionaries `(distances, predecessors)` where `distance[u][v]` denotes the distance of a shortest path from u to v and `predecessors[u][v]` indicates the predecessor of w on a shortest path from u to v.

EXAMPLES:

Undirected graphs:

```
sage: from sage.graphs.base.boost_graph import johnson_shortest_paths
sage: g = Graph([(0,1,1),(1,2,2),(1,3,4),(2,3,1)], weighted=True)
sage: johnson_shortest_paths(g)
```

(continues on next page)

(continued from previous page)

```
{0: {0: 0, 1: 1, 2: 3, 3: 4},
 1: {0: 1, 1: 0, 2: 2, 3: 3},
 2: {0: 3, 1: 2, 2: 0, 3: 1},
 3: {0: 4, 1: 3, 2: 1, 3: 0}}
sage: expected = {0: {0: None, 1: 0, 2: 1, 3: 2},
....:             1: {0: 1, 1: None, 2: 1, 3: 2},
....:             2: {0: 1, 1: 2, 2: None, 3: 2},
....:             3: {0: 1, 1: 2, 2: 3, 3: None}}
sage: johnson_shortest_paths(g, distances=False, predecessors=True) == expected
True
```

```
>>> from sage.all import *
>>> from sage.graphs.base.boost_graph import johnson_shortest_paths
>>> g = Graph([(Integer(0),Integer(1),Integer(1)),(Integer(1),Integer(2),
...Integer(2)),(Integer(1),Integer(3),Integer(4)),(Integer(2),Integer(3),
...Integer(1))], weighted=True)
>>> johnson_shortest_paths(g)
{0: {0: 0, 1: 1, 2: 3, 3: 4},
 1: {0: 1, 1: 0, 2: 2, 3: 3},
 2: {0: 3, 1: 2, 2: 0, 3: 1},
 3: {0: 4, 1: 3, 2: 1, 3: 0}}
>>> expected = {Integer(0): {Integer(0): None, Integer(1): Integer(0),␣
...Integer(2): Integer(1), Integer(3): Integer(2)},
...             Integer(1): {Integer(0): Integer(1), Integer(1): None,␣
...Integer(2): Integer(1), Integer(3): Integer(2)},
...             Integer(2): {Integer(0): Integer(1), Integer(1): Integer(2),␣
...Integer(2): None, Integer(3): Integer(2)},
...             Integer(3): {Integer(0): Integer(1), Integer(1): Integer(2),␣
...Integer(2): Integer(3), Integer(3): None}}
>>> johnson_shortest_paths(g, distances=False, predecessors=True) == expected
True
```

Directed graphs:

```
sage: g = DiGraph([(0,1,1),(1,2,-2),(1,3,4),(2,3,1)], weighted=True)
sage: johnson_shortest_paths(g)
{0: {0: 0, 1: 1, 2: -1, 3: 0},
 1: {1: 0, 2: -2, 3: -1},
 2: {2: 0, 3: 1},
 3: {3: 0}}
sage: g = DiGraph([(1,2,3),(2,3,2),(1,4,1),(4,2,1)], weighted=True)
sage: johnson_shortest_paths(g, distances=False, predecessors=True)
{1: {1: None, 2: 4, 3: 2, 4: 1},
 2: {2: None, 3: 2},
 3: {3: None},
 4: {2: 4, 3: 2, 4: None}}
```

```
>>> from sage.all import *
>>> g = DiGraph([(Integer(0),Integer(1),Integer(1)),(Integer(1),Integer(2),-
...Integer(2)),(Integer(1),Integer(3),Integer(4)),(Integer(2),Integer(3),
...Integer(1))], weighted=True)
>>> johnson_shortest_paths(g)
```

(continues on next page)

(continued from previous page)

```
{0: {0: 0, 1: 1, 2: -1, 3: 0},
 1: {1: 0, 2: -2, 3: -1},
 2: {2: 0, 3: 1},
 3: {3: 0}}
>>> g = DiGraph([(Integer(1),Integer(2),Integer(3)),(Integer(2),Integer(3),
↪Integer(2)),(Integer(1),Integer(4),Integer(1)),(Integer(4),Integer(2),
↪Integer(1))], weighted=True)
>>> johnson_shortest_paths(g, distances=False, predecessors=True)
{1: {1: None, 2: 4, 3: 2, 4: 1},
 2: {2: None, 3: 2},
 3: {3: None},
 4: {2: 4, 3: 2, 4: None}}
```

sage.graphs.base.boost_graph.**min_cycle_basis**(*g_sage*, *weight_function=None*, *by_weight=False*)

Return a minimum weight cycle basis of the input graph `g_sage`.

A cycle basis is a list of cycles (list of vertices forming a cycle) of `g_sage`. Note that the vertices are not necessarily returned in the order in which they appear in the cycle.

A minimum weight cycle basis is a cycle basis that minimizes the sum of the weights (length for unweighted graphs) of its cycles.

Not implemented for directed graphs and multigraphs.

INPUT:

- `g_sage` – a Sage Graph
- `weight_function` – function (default: `None`); a function that takes as input an edge `(u, v, l)` and outputs its weight. If not `None`, `by_weight` is automatically set to `True`. If `None` and `by_weight` is `True`, the weights of `g_sage` are used, if `g_sage.weighted()==True`, otherwise all edges have weight 1.
- `by_weight` – boolean (default: `False`); if `True`, the edges in the graph are weighted, otherwise all edges have weight 1

EXAMPLES:

```
sage: g = Graph([(1, 2, 3), (2, 3, 5), (3, 4, 8), (4, 1, 13), (1, 3, 250), (5, 6,␣
↪9), (6, 7, 17), (7, 5, 20)])
sage: sorted(g.minimum_cycle_basis(by_weight=True))
[[1, 2, 3], [1, 2, 3, 4], [5, 6, 7]]
sage: sorted(g.minimum_cycle_basis())
[[1, 2, 3], [1, 3, 4], [5, 6, 7]]
```

```
>>> from sage.all import *
>>> g = Graph([(Integer(1), Integer(2), Integer(3)), (Integer(2), Integer(3),␣
↪Integer(5)), (Integer(3), Integer(4), Integer(8)), (Integer(4), Integer(1),␣
↪Integer(13)), (Integer(1), Integer(3), Integer(250)), (Integer(5), Integer(6),␣
↪Integer(9)), (Integer(6), Integer(7), Integer(17)), (Integer(7), Integer(5),␣
↪Integer(20))])
>>> sorted(g.minimum_cycle_basis(by_weight=True))
[[1, 2, 3], [1, 2, 3, 4], [5, 6, 7]]
>>> sorted(g.minimum_cycle_basis())
[[1, 2, 3], [1, 3, 4], [5, 6, 7]]
```

See also

- Wikipedia article Cycle_basis

sage.graphs.base.boost_graph.**min_spanning_tree**(*g*, *weight_function=None*, *algorithm='Kruskal'*)

Use Boost to compute the minimum spanning tree of the input graph.

INPUT:

- `g` – the input Sage graph
- `weight_function` – function (default: `None`); a function that inputs an edge `e` and outputs its weight. An edge has the form `(u,v,l)`, where `u` and `v` are vertices, `l` is a label (that can be of any kind). The `weight_function` can be used to transform the label into a weight (see the example below). In particular:
  - if `weight_function` is not `None`, the weight of an edge `e` is `weight_function(e)`;
  - if `weight_function` is `None` (default) and `g` is weighted (that is, `g.weighted()==True`), for each edge `e=(u,v,l)`, we set weight `l`;
  - if `weight_function` is `None` and `g` is not weighted, we set all weights to 1 (hence, the output can be any spanning tree).

  Note that, if the weight is not convertible to a number with function `float()`, an error is raised (see tests below).
- `algorithm` – string (default: `'Kruskal'`); the algorithm to use among `'Kruskal'` and `'Prim'`

OUTPUT:

The edges of a minimum spanning tree of `g`, if one exists, otherwise the empty list.

See also

- *sage.graphs.generic_graph.GenericGraph.min_spanning_tree()*

EXAMPLES:

```
sage: from sage.graphs.base.boost_graph import min_spanning_tree
sage: min_spanning_tree(graphs.PathGraph(4))
[(0, 1, None), (1, 2, None), (2, 3, None)]

sage: G = Graph([(0,1,{'name':'a','weight':1}), (0,2,{'name':'b','weight':3}), (1,
↪2,{'name':'b','weight':1})])
sage: min_spanning_tree(G, weight_function=lambda e: e[2]['weight'])
[(0, 1, {'name': 'a', 'weight': 1}), (1, 2, {'name': 'b', 'weight': 1})]
```

```
>>> from sage.all import *
>>> from sage.graphs.base.boost_graph import min_spanning_tree
>>> min_spanning_tree(graphs.PathGraph(Integer(4)))
[(0, 1, None), (1, 2, None), (2, 3, None)]

>>> G = Graph([(Integer(0),Integer(1),{'name':'a','weight':Integer(1)}),␣
↪(Integer(0),Integer(2),{'name':'b','weight':Integer(3)}), (Integer(1),
↪Integer(2),{'name':'b','weight':Integer(1)})])
```

(continues on next page)

(continued from previous page)

```
>>> min_spanning_tree(G, weight_function=lambda e: e[Integer(2)]['weight'])
[(0, 1, {'name': 'a', 'weight': 1}), (1, 2, {'name': 'b', 'weight': 1})]
```

sage.graphs.base.boost_graph.**radius_DHV**(*g*, *weight_function=None*, *check_weight=True*)

Return the radius of weighted graph g.

This method computes the radius of undirected graph using the algorithm given in [Dragan2018].

This method returns Infinity if graph is not connected.

INPUT:

- `g` – the input Sage graph
- `weight_function` – function (default: `None`); a function that associates a weight to each edge. If `None` (default), the weights of `g` are used, if `g.weighted()==True`, otherwise all edges have weight 1.
- `check_weight` – boolean (default: `True`); if `True`, we check that the `weight_function` outputs a number for each edge

EXAMPLES:

```
sage: from sage.graphs.base.boost_graph import radius_DHV
sage: G = Graph([(0,1,1), (1,2,1), (0,2,3)])
sage: radius_DHV(G)
1.0
sage: G = graphs.PathGraph(7)
sage: radius_DHV(G) == G.radius(algorithm='Dijkstra_Boost')
True
```

```
>>> from sage.all import *
>>> from sage.graphs.base.boost_graph import radius_DHV
>>> G = Graph([(Integer(0),Integer(1),Integer(1)), (Integer(1),Integer(2),
↪Integer(1)), (Integer(0),Integer(2),Integer(3))])
>>> radius_DHV(G)
1.0
>>> G = graphs.PathGraph(Integer(7))
>>> radius_DHV(G) == G.radius(algorithm='Dijkstra_Boost')
True
```

sage.graphs.base.boost_graph.**shortest_paths**(*g*, *start*, *weight_function=None*, *algorithm=None*)

Compute the shortest paths from `start` to all other vertices.

This routine outputs all shortest paths from node `start` to any other node in the graph. The input graph can be weighted: if the algorithm is Dijkstra, no negative weights are allowed, while if the algorithm is Bellman-Ford, negative weights are allowed, but there must be no negative cycle (otherwise, the shortest paths might not exist).

However, Dijkstra algorithm is more efficient: for this reason, we suggest to use Bellman-Ford only if necessary (which is also the default option). Note that, if the graph is undirected, a negative edge automatically creates a negative cycle: for this reason, in this case, Dijkstra algorithm is always better.

The running-time is $O(n \log n + m)$ for Dijkstra algorithm and $O(mn)$ for Bellman-Ford algorithm, where n is the number of nodes and m is the number of edges.

INPUT:

- `g` – the input Sage graph
- `start` – the starting vertex to compute shortest paths

- `weight_function` – function (default: `None`); a function that associates a weight to each edge. If `None` (default), the weights of `g` are used, if `g.weighted()==True`, otherwise all edges have weight 1.
- `algorithm` – string (default: `None`); one of the following algorithms:
  - `'Dijkstra'`, `'Dijkstra_Boost'`: the Dijkstra algorithm implemented in Boost (works only with positive weights)
  - `'Bellman-Ford'`, `'Bellman-Ford_Boost'`: the Bellman-Ford algorithm implemented in Boost (works also with negative weights, if there is no negative cycle)

OUTPUT:

A pair of dictionaries `(distances, predecessors)` such that, for each vertex `v`, `distances[v]` is the distance from `start` to `v`, `predecessors[v]` is the last vertex in a shortest path from `start` to `v`.

EXAMPLES:

Undirected graphs:

```
sage: from sage.graphs.base.boost_graph import shortest_paths
sage: g = Graph([(0,1,1),(1,2,2),(1,3,4),(2,3,1)], weighted=True)
sage: shortest_paths(g, 1)
({0: 1, 1: 0, 2: 2, 3: 3}, {0: 1, 1: None, 2: 1, 3: 2})
sage: g = graphs.GridGraph([2,2])
sage: shortest_paths(g,(0,0),weight_function=lambda e:2)
({(0, 0): 0, (0, 1): 2, (1, 0): 2, (1, 1): 4},
 {(0, 0): None, (0, 1): (0, 0), (1, 0): (0, 0), (1, 1): (0, 1)})
```

```
>>> from sage.all import *
>>> from sage.graphs.base.boost_graph import shortest_paths
>>> g = Graph([(Integer(0),Integer(1),Integer(1)),(Integer(1),Integer(2),
...Integer(2)),(Integer(1),Integer(3),Integer(4)),(Integer(2),Integer(3),
...Integer(1))], weighted=True)
>>> shortest_paths(g, Integer(1))
({0: 1, 1: 0, 2: 2, 3: 3}, {0: 1, 1: None, 2: 1, 3: 2})
>>> g = graphs.GridGraph([Integer(2),Integer(2)])
>>> shortest_paths(g,(Integer(0),Integer(0)),weight_function=lambda e:Integer(2))
({(0, 0): 0, (0, 1): 2, (1, 0): 2, (1, 1): 4},
 {(0, 0): None, (0, 1): (0, 0), (1, 0): (0, 0), (1, 1): (0, 1)})
```

Directed graphs:

```
sage: g = DiGraph([(0,1,1),(1,2,2),(1,3,4),(2,3,1)], weighted=True)
sage: shortest_paths(g, 1)
({1: 0, 2: 2, 3: 3}, {1: None, 2: 1, 3: 2})
```

```
>>> from sage.all import *
>>> g = DiGraph([(Integer(0),Integer(1),Integer(1)),(Integer(1),Integer(2),
...Integer(2)),(Integer(1),Integer(3),Integer(4)),(Integer(2),Integer(3),
...Integer(1))], weighted=True)
>>> shortest_paths(g, Integer(1))
({1: 0, 2: 2, 3: 3}, {1: None, 2: 1, 3: 2})
```

sage.graphs.base.boost_graph.**shortest_paths_from_vertices**(*g*, *vertex_list=None*, *order=None*, *weight_function=None*, *algorithm=None*)

Compute the shortest paths to all vertices from each vertex in `vertex_list`.

The input graph can be weighted: if the algorithm is Dijkstra, no negative weights are allowed, while if the algorithm is Bellman-Ford, negative weights are allowed, but there must be no negative cycle (otherwise, the shortest paths might not exist).

However, Dijkstra algorithm is more efficient: for this reason, we suggest to use Bellman-Ford only if necessary (which is also the default option).

The running-time for each vertex is $O(n \log n+m)$ for Dijkstra algorithm and $O(mn)$ for Bellman-Ford algorithm, where n is the number of nodes and m is the number of edges.

INPUT:

- `g` – the input Sage graph
- `vertex_list` – list (default: `None`); list of vertices to compute shortest paths from. By default (`None`), compute shortest paths from all vertices.
- `order` – list (default: `None`); order of vertices of g
- `weight_function` – function (default: `None`); a function that associates a weight to each edge. If `None` (default), the weights of `g` are used, if `g.weighted()==True`, otherwise all edges have weight 1.
- `algorithm` – string (default: `None`); one of the following algorithms:
  - `'Dijkstra'`, `'Dijkstra_Boost'` – the Dijkstra algorithm implemented in Boost (works only with positive weights)
  - `'Bellman-Ford'`, `'Bellman-Ford_Boost'` – the Bellman-Ford algorithm implemented in Boost (works also with negative weights, if there is no negative cycle)

OUTPUT:

The type of output depends on the input. More precisely -

- A pair of dictionaries of list `(distances, predecessors)`, when `order is not None`, such that for each vertex `v` in `vertex_list`, `distances[v][i]` store the shortest distance between `v` and `order[i]` and `predecessors[v][i]` store the last vertex in the shortest path from `v` to `order[i]`.
- A pair of dictionaries of dictionaries `(distances, predecessors)` such that for each vertex `v` in `vertex_list`, `distances[v]` store the shortest distances of all the other vertices from `v`, `predecessors[v]` store the last vertices in the shortest path from `v` to all the other vertices.

EXAMPLES:

Undirected graphs:

```
sage: from sage.graphs.base.boost_graph import shortest_paths_from_vertices
sage: g = Graph([(0,1,1),(1,2,2),(1,3,4),(2,3,1)], weighted=True)
sage: shortest_paths_from_vertices(g,[1,2])
({1: {0: 1.0, 1: 0.0, 2: 2.0, 3: 3.0}, 2: {0: 3.0, 1: 2.0, 2: 0.0, 3: 1.0}},
 {1: {0: 1, 1: None, 2: 1, 3: 2}, 2: {0: 1, 1: 2, 2: None, 3: 2}})
```

```
>>> from sage.all import *
>>> from sage.graphs.base.boost_graph import shortest_paths_from_vertices
>>> g = Graph([(Integer(0),Integer(1),Integer(1)),(Integer(1),Integer(2),
↪Integer(2)),(Integer(1),Integer(3),Integer(4)),(Integer(2),Integer(3),
↪Integer(1))], weighted=True)
>>> shortest_paths_from_vertices(g,[Integer(1),Integer(2)])
```

(continues on next page)

(continued from previous page)

```
({1: {0: 1.0, 1: 0.0, 2: 2.0, 3: 3.0}, 2: {0: 3.0, 1: 2.0, 2: 0.0, 3: 1.0}},
 {1: {0: 1, 1: None, 2: 1, 3: 2}, 2: {0: 1, 1: 2, 2: None, 3: 2}})
```

Directed graphs:

```
sage: g = DiGraph([(0,1,1),(1,2,-1),(2,0,2),(2,3,1)], weighted=True)
sage: shortest_paths_from_vertices(g,1)
({1: {0: 1.0, 1: 0.0, 2: -1.0, 3: 0.0}}, {1: {0: 2, 1: None, 2: 1, 3: 2}})
sage: shortest_paths_from_vertices(g, 1, [0,1,2,3])
({1: [1.0, 0.0, -1.0, 0.0]}, {1: [2, None, 1, 2]})
```

```
>>> from sage.all import *
>>> g = DiGraph([(Integer(0),Integer(1),Integer(1)),(Integer(1),Integer(2),-
↪Integer(1)),(Integer(2),Integer(0),Integer(2)),(Integer(2),Integer(3),
↪Integer(1))], weighted=True)
>>> shortest_paths_from_vertices(g,Integer(1))
({1: {0: 1.0, 1: 0.0, 2: -1.0, 3: 0.0}}, {1: {0: 2, 1: None, 2: 1, 3: 2}})
>>> shortest_paths_from_vertices(g, Integer(1), [Integer(0),Integer(1),Integer(2),
↪Integer(3)])
({1: [1.0, 0.0, -1.0, 0.0]}, {1: [2, None, 1, 2]})
```

sage.graphs.base.boost_graph.**wiener_index**(*g*, *algorithm=None*, *weight_function=None*, *check_weight=True*)

Return the Wiener index of the graph.

The Wiener index of an undirected graph G is defined as $W(G) = \frac{1}{2}\sum_{u,v\in G} d(u,v)$ where $d(u,v)$ denotes the distance between vertices u and v (see [KRG1996]).

The Wiener index of a directed graph G is defined as the sum of the distances between each pairs of vertices, $W(G) = \sum_{u,v\in G} d(u,v)$.

INPUT:

- `g` – the input Sage graph
- `algorithm` – string (default: `None`); one of the following algorithms:
  - `'Dijkstra'`, `'Dijkstra_Boost'`: the Dijkstra algorithm implemented in Boost (works only with positive weights)
  - `'Bellman-Ford'`, `'Bellman-Ford_Boost'`: the Bellman-Ford algorithm implemented in Boost (works also with negative weights, if there is no negative cycle)
- `weight_function` – function (default: `None`); a function that associates a weight to each edge. If `None` (default), the weights of `g` are used, if `g.weighted()==True`, otherwise all edges have weight 1.
- `check_weight` – boolean (default: `True`); if `True`, we check that the `weight_function` outputs a number for each edge

EXAMPLES:

sage: from sage.graphs.base.boost_graph import wiener_index sage: g = Graph([(0,1,9), (1,2,7), (2,3,4), (3,0,3)]) sage: wiener_index(g) 8.0 sage: g.weighted(True) sage: wiener_index(g) 41.0

Wiener index of circuit digraphs:

```
sage: n = 10
sage: g = digraphs.Circuit(n)
```

(continues on next page)

(continued from previous page)

```
sage: w = lambda x: (x*x*(x-1))/2
sage: wiener_index(g) == w(n)
True
```

```
>>> from sage.all import *
>>> n = Integer(10)
>>> g = digraphs.Circuit(n)
>>> w = lambda x: (x*x*(x-Integer(1)))/Integer(2)
>>> wiener_index(g) == w(n)
True
```

Wiener index of a graph of order 1:

```
sage: wiener_index(Graph(1))
0
```

```
>>> from sage.all import *
>>> wiener_index(Graph(Integer(1)))
0
```

The Wiener index is not defined on the empty graph:

```
sage: wiener_index(Graph())
Traceback (most recent call last):
...
ValueError: Wiener index is not defined for the empty graph
```

```
>>> from sage.all import *
>>> wiener_index(Graph())
Traceback (most recent call last):
...
ValueError: Wiener index is not defined for the empty graph
```

CHAPTER

FOUR

HYPERGRAPHS

4.1 Hypergraph generators

This module implements generators of hypergraphs. All hypergraphs can be built through the `hypergraphs` object. For instance, to build a complete 3-uniform hypergraph on 5 points, one can do:

```
sage: H = hypergraphs.CompleteUniform(5, 3)
```

```
>>> from sage.all import *
>>> H = hypergraphs.CompleteUniform(Integer(5), Integer(3))
```

To enumerate hypergraphs with certain properties up to isomorphism, one can use method *nauty()*, which calls Brendan McKay's Nauty (http://cs.anu.edu.au/~bdm/nauty/):

```
sage: list(hypergraphs.nauty(2, 2, connected=True))
[((0,), (0, 1))]
```

```
>>> from sage.all import *
>>> list(hypergraphs.nauty(Integer(2), Integer(2), connected=True))
[((0,), (0, 1))]
```

This module contains the following hypergraph generators

nauty()	Enumerate hypergraphs up to isomorphism using Nauty.
CompleteUniform()	Return the complete k-uniform hypergraph on n points.
UniformRandomUniform()	Return a uniformly sampled k-uniform hypergraph on n points with m hyperedges.

4.1.1 Functions and methods

class sage.graphs.hypergraph_generators.**HypergraphGenerators**

Bases: `object`

A class consisting of constructors for common hypergraphs.

BinomialRandomUniform(n, k, p)

Return a random k-uniform hypergraph on n points, in which each edge is inserted independently with probability p.

- `n` – number of nodes of the graph
- `k` – uniformity

- p – probability of an edge

EXAMPLES:

```
sage: hypergraphs.BinomialRandomUniform(50, 3, 1).n_blocks()              #␣
↪needs numpy, long time
19600
sage: hypergraphs.BinomialRandomUniform(50, 3, 0).n_blocks()              #␣
↪needs numpy
0
```

```
>>> from sage.all import *
>>> hypergraphs.BinomialRandomUniform(Integer(50), Integer(3), Integer(1)).n_
↪blocks()              # needs numpy, long time
19600
>>> hypergraphs.BinomialRandomUniform(Integer(50), Integer(3), Integer(0)).n_
↪blocks()              # needs numpy
0
```

CompleteUniform(n, k)

Return the complete k-uniform hypergraph on n points.

INPUT:

- k, n – nonnegative integers with $k \leq n$

EXAMPLES:

```
sage: h = hypergraphs.CompleteUniform(5, 2); h
Incidence structure with 5 points and 10 blocks
sage: len(h.packing())                                                    #␣
↪needs sage.numerical.mip
2
```

```
>>> from sage.all import *
>>> h = hypergraphs.CompleteUniform(Integer(5), Integer(2)); h
Incidence structure with 5 points and 10 blocks
>>> len(h.packing())                                                      #␣
↪needs sage.numerical.mip
2
```

UniformRandomUniform(n, k, m)

Return a uniformly sampled k-uniform hypergraph on n points with m hyperedges.

- n – number of nodes of the graph
- k – uniformity
- m – number of edges

EXAMPLES:

```
sage: H = hypergraphs.UniformRandomUniform(52, 3, 17)
sage: H
Incidence structure with 52 points and 17 blocks
sage: H.is_connected()
False
```

```
>>> from sage.all import *
>>> H = hypergraphs.UniformRandomUniform(Integer(52), Integer(3), Integer(17))
>>> H
Incidence structure with 52 points and 17 blocks
>>> H.is_connected()
False
```

nauty(*number_of_sets*, *number_of_vertices*, *multiple_sets=False*, *vertex_min_degree=None*, *vertex_max_degree=None*, *set_max_size=None*, *set_min_size=None*, *regular=False*, *uniform=False*, *max_intersection=None*, *connected=False*, *debug=False*, *options=''*)

Enumerate hypergraphs up to isomorphism using Nauty.

INPUT:

- `number_of_sets` – integer; at most 64 minus `number_of_vertices`
- `number_of_vertices` – integer; at most 30
- `multiple_sets` – boolean (default: `False`); whether to allow several sets of the hypergraph to be equal
- `vertex_min_degree`, `vertex_max_degree` – integers (default: `None`); define the maximum and minimum degree of an element from the ground set (i.e. the number of sets which contain it)
- `set_min_size`, `set_max_size` – integers (default: `None`); define the maximum and minimum size of a set
- `regular` – integer (default: `False`); if set to an integer value k, requires the hypergraphs to be k-regular. It is actually a shortcut for the corresponding min/max values.
- `uniform` – integer (default: `False`); if set to an integer value k, requires the hypergraphs to be k-uniform. It is actually a shortcut for the corresponding min/max values.
- `max_intersection` – integer (default: `None`); constraints the maximum cardinality of the intersection of two sets from the hypergraphs
- `connected` – boolean (default: `False`); whether to require the hypergraphs to be connected
- `debug` – boolean (default: `False`); if `True` the first line of genbgL's output to standard error is captured and the first call to the generator's `next()` function will return this line as a string. A line leading with ">A" indicates a successful initiation of the program with some information on the arguments, while a line beginning with ">E" indicates an error with the input.
- `options` – string (default: `''`); anything else that should be forwarded as input to Nauty's genbgL. See its documentation for more information : http://cs.anu.edu.au/~bdm/nauty/.

> **Note**
>
> For genbgL the *first class* elements are vertices, and *second class* elements are the hypergraph's sets.

OUTPUT: a tuple of tuples

EXAMPLES:

Small hypergraphs:

```
sage: list(hypergraphs.nauty(4, 2))
[((), (0,), (1,), (0, 1))]
```

```
>>> from sage.all import *
>>> list(hypergraphs.nauty(Integer(4), Integer(2)))
[((), (0,), (1,), (0, 1))]
```

Only connected ones:

```
sage: list(hypergraphs.nauty(2, 2, connected=True))
[((0,), (0, 1))]
```

```
>>> from sage.all import *
>>> list(hypergraphs.nauty(Integer(2), Integer(2), connected=True))
[((0,), (0, 1))]
```

Non-empty sets only:

```
sage: list(hypergraphs.nauty(3, 2, set_min_size=1))
[((0,), (1,), (0, 1))]
```

```
>>> from sage.all import *
>>> list(hypergraphs.nauty(Integer(3), Integer(2), set_min_size=Integer(1)))
[((0,), (1,), (0, 1))]
```

The Fano Plane, as the only 3-uniform hypergraph with 7 sets and 7 vertices:

```
sage: fano = next(hypergraphs.nauty(7, 7, uniform=3, max_intersection=1))
sage: print(fano)
((0, 1, 2), (0, 3, 4), (0, 5, 6), (1, 3, 5), (2, 4, 5), (2, 3, 6), (1, 4, 6))
```

```
>>> from sage.all import *
>>> fano = next(hypergraphs.nauty(Integer(7), Integer(7), uniform=Integer(3),␣
↪max_intersection=Integer(1)))
>>> print(fano)
((0, 1, 2), (0, 3, 4), (0, 5, 6), (1, 3, 5), (2, 4, 5), (2, 3, 6), (1, 4, 6))
```

The Fano Plane, as the only 3-regular hypergraph with 7 sets and 7 vertices:

```
sage: fano = next(hypergraphs.nauty(7, 7, regular=3, max_intersection=1))
sage: print(fano)
((0, 1, 2), (0, 3, 4), (0, 5, 6), (1, 3, 5), (2, 4, 5), (2, 3, 6), (1, 4, 6))
```

```
>>> from sage.all import *
>>> fano = next(hypergraphs.nauty(Integer(7), Integer(7), regular=Integer(3),␣
↪max_intersection=Integer(1)))
>>> print(fano)
((0, 1, 2), (0, 3, 4), (0, 5, 6), (1, 3, 5), (2, 4, 5), (2, 3, 6), (1, 4, 6))
```

4.2 Incidence structures (i.e. hypergraphs, i.e. set systems)

An incidence structure is specified by a list of points, blocks, or an incidence matrix ([1],[2]). *IncidenceStructure* instances have the following methods:

[1] Block designs and incidence structures from wikipedia, Wikipedia article Block_design Wikipedia article Incidence_structure
[2] E. Assmus, J. Key, Designs and their codes, CUP, 1992.

automorphism_group()	Return the subgroup of the automorphism group of the incidence graph which respects the P B partition. It is (isomorphic to) the automorphism group of the block design, although the degrees differ.
block_sizes()	Return the set of block sizes.
blocks()	Return the list of blocks.
canonical_label()	Return a canonical label for the incidence structure.
coloring()	Compute a (weak) k-coloring of the hypergraph.
complement()	Return the complement of the incidence structure.
copy()	Return a copy of the incidence structure.
degree()	Return the degree of a point `p` (or a set of points).
degrees()	Return the degree of all sets of given size, or the degree of all points.
dual()	Return the dual of the incidence structure.
edge_coloring()	Compute a proper edge-coloring.
ground_set()	Return the ground set (i.e the list of points).
incidence_graph()	Return the incidence graph of the incidence structure.
incidence_matrix()	Return the incidence matrix A of the design. A is a $(v \times b)$ matrix defined by: `A[i,j] = 1` if `i` is in block `B_j` and 0 otherwise.
induced_substructure()	Return the substructure induced by a set of points.
intersection_graph()	Return the intersection graph of the incidence structure.
is_berge_cyclic()	Check whether `self` is a Berge-Cyclic uniform hypergraph.
is_connected()	Test whether the design is connected.
is_generalized_quadrangle()	Test if the incidence structure is a generalized quadrangle.
is_isomorphic()	Return whether the two incidence structures are isomorphic.
is_regular()	Test whether the incidence structure is r-regular.
is_resolvable()	Test whether the hypergraph is resolvable.
is_simple()	Test whether this design is simple (i.e. no repeated block).
is_spread()	Check whether the input is a spread for `self`.
is_t_design()	Test whether `self` is a $t-(v,k,l)$ design.
is_uniform()	Test whether the incidence structure is k-uniform
isomorphic_substructures_iterator()	Iterate over all copies of `H2` contained in `self`.
num_blocks()	Return the number of blocks.
num_points()	Return the size of the ground set.
packing()	Return a maximum packing.
rank()	Return the rank of the hypergraph (the maximum size of a block).
relabel()	Relabel the ground set.
trace()	Return the trace of a set of points.

REFERENCES:

AUTHORS:

- Peter Dobcsanyi and David Joyner (2007-2008)

  This is a significantly modified form of part of the module block_design.py (version 0.6) written by Peter Dobcsanyi peter@designtheory.org.

- Vincent Delecroix (2014): major rewrite

4.2.1 Methods

class sage.combinat.designs.incidence_structures.**IncidenceStructure**(*points=None*, *blocks=None*, *incidence_matrix=None*, *name=None*, *check=True*, *copy=True*)

Bases: SageObject

A base class for incidence structures (i.e. hypergraphs, i.e. set systems)

An incidence structure (i.e. hypergraph, i.e. set system) can be defined from a collection of blocks (i.e. sets, i.e. edges), optionally with an explicit ground set (i.e. point set, i.e. vertex set). Alternatively they can be defined from a binary incidence matrix.

INPUT:

- `points` – (i.e. ground set, i.e. vertex set) the underlying set. If `points` is an integer v, then the set is considered to be $\{0, ..., v-1\}$.

  Note

  The following syntax, where `points` is omitted, automatically defines the ground set as the union of the blocks:

```
sage: H = IncidenceStructure([['a','b','c'],['c','d','e']])
sage: sorted(H.ground_set())
['a', 'b', 'c', 'd', 'e']
```

```
>>> from sage.all import *
>>> H = IncidenceStructure([['a','b','c'],['c','d','e']])
>>> sorted(H.ground_set())
['a', 'b', 'c', 'd', 'e']
```

- `blocks` – (i.e. edges, i.e. sets) the blocks defining the incidence structure; can be any iterable
- `incidence_matrix` – a binary incidence matrix; each column represents a set
- `name` – string (such as "Fano plane")
- `check` – whether to check the input
- `copy` – (use with caution) if set to `False` then `blocks` must be a list of lists of integers. The list will not be copied but will be modified in place (each block is sorted, and the whole list is sorted). Your `blocks` object will become the *IncidenceStructure* instance's internal data.

EXAMPLES:

An incidence structure can be constructed by giving the number of points and the list of blocks:

```
sage: IncidenceStructure(7, [[0,1,2],[0,3,4],[0,5,6],[1,3,5],[1,4,6],[2,3,6],[2,4,
↪5]])
Incidence structure with 7 points and 7 blocks
```

```
>>> from sage.all import *
>>> IncidenceStructure(Integer(7), [[Integer(0),Integer(1),Integer(2)],
↪[Integer(0),Integer(3),Integer(4)],[Integer(0),Integer(5),Integer(6)],
```

(continues on next page)

(continued from previous page)

```
↪[Integer(1),Integer(3),Integer(5)],[Integer(1),Integer(4),Integer(6)],
↪[Integer(2),Integer(3),Integer(6)],[Integer(2),Integer(4),Integer(5)]])
Incidence structure with 7 points and 7 blocks
```

Only providing the set of blocks is sufficient. In this case, the ground set is defined as the union of the blocks:

```
sage: IncidenceStructure([[1,2,3],[2,3,4]])
Incidence structure with 4 points and 2 blocks
```

```
>>> from sage.all import *
>>> IncidenceStructure([[Integer(1),Integer(2),Integer(3)],[Integer(2),Integer(3),
↪Integer(4)]])
Incidence structure with 4 points and 2 blocks
```

Or by its adjacency matrix (a $\{0,1\}$-matrix in which rows are indexed by points and columns by blocks):

```
sage: m = matrix([[0,1,0],[0,0,1],[1,0,1],[1,1,1]])                    #␣
↪needs sage.modules
sage: IncidenceStructure(m)                                            #␣
↪needs sage.modules
Incidence structure with 4 points and 3 blocks
```

```
>>> from sage.all import *
>>> m = matrix([[Integer(0),Integer(1),Integer(0)],[Integer(0),Integer(0),
↪Integer(1)],[Integer(1),Integer(0),Integer(1)],[Integer(1),Integer(1),
↪Integer(1)]])                                  # needs sage.modules
>>> IncidenceStructure(m)                                                #␣
↪needs sage.modules
Incidence structure with 4 points and 3 blocks
```

The points can be any (hashable) object:

```
sage: V = [(0,'a'),(0,'b'),(1,'a'),(1,'b')]
sage: B = [(V[0],V[1],V[2]), (V[1],V[2]), (V[0],V[2])]
sage: I = IncidenceStructure(V, B)
sage: I.ground_set()
[(0, 'a'), (0, 'b'), (1, 'a'), (1, 'b')]
sage: I.blocks()
[[(0, 'a'), (0, 'b'), (1, 'a')], [(0, 'a'), (1, 'a')], [(0, 'b'), (1, 'a')]]
```

```
>>> from sage.all import *
>>> V = [(Integer(0),'a'),(Integer(0),'b'),(Integer(1),'a'),(Integer(1),'b')]
>>> B = [(V[Integer(0)],V[Integer(1)],V[Integer(2)]), (V[Integer(1)],
↪V[Integer(2)]), (V[Integer(0)],V[Integer(2)])]
>>> I = IncidenceStructure(V, B)
>>> I.ground_set()
[(0, 'a'), (0, 'b'), (1, 'a'), (1, 'b')]
>>> I.blocks()
[[(0, 'a'), (0, 'b'), (1, 'a')], [(0, 'a'), (1, 'a')], [(0, 'b'), (1, 'a')]]
```

The order of the points and blocks does not matter as they are sorted on input (see Issue #11333):

```
sage: A = IncidenceStructure([0,1,2], [[0],[0,2]])
sage: B = IncidenceStructure([1,0,2], [[0],[2,0]])
sage: B == A
True

sage: C = BlockDesign(2, [[0], [1,0]])
sage: D = BlockDesign(2, [[0,1], [0]])
sage: C == D
True
```

```
>>> from sage.all import *
>>> A = IncidenceStructure([Integer(0),Integer(1),Integer(2)], [[Integer(0)],
↪[Integer(0),Integer(2)]])
>>> B = IncidenceStructure([Integer(1),Integer(0),Integer(2)], [[Integer(0)],
↪[Integer(2),Integer(0)]])
>>> B == A
True

>>> C = BlockDesign(Integer(2), [[Integer(0)], [Integer(1),Integer(0)]])
>>> D = BlockDesign(Integer(2), [[Integer(0),Integer(1)], [Integer(0)]])
>>> C == D
True
```

If you care for speed, you can set `copy` to `False`, but in that case, your input must be a list of lists and the ground set must be $0, ..., v-1$:

```
sage: blocks = [[0,1],[2,0],[1,2]]  # a list of lists of integers
sage: I = IncidenceStructure(3, blocks, copy=False)
sage: I._blocks is blocks
True
```

```
>>> from sage.all import *
>>> blocks = [[Integer(0),Integer(1)],[Integer(2),Integer(0)],[Integer(1),
↪Integer(2)]]  # a list of lists of integers
>>> I = IncidenceStructure(Integer(3), blocks, copy=False)
>>> I._blocks is blocks
True
```

automorphism_group()

Return the subgroup of the automorphism group of the incidence graph which respects the P B partition. It is (isomorphic to) the automorphism group of the block design, although the degrees differ.

EXAMPLES:

```
sage: # needs sage.groups sage.rings.finite_rings
sage: P = designs.DesarguesianProjectivePlaneDesign(2); P
(7,3,1)-Balanced Incomplete Block Design
sage: G = P.automorphism_group()
sage: G.is_isomorphic(PGL(3,2))
True
sage: G
Permutation Group with generators [...]
```

(continues on next page)

(continued from previous page)

```
sage: G.cardinality()
168
```

```
>>> from sage.all import *
>>> # needs sage.groups sage.rings.finite_rings
>>> P = designs.DesarguesianProjectivePlaneDesign(Integer(2)); P
(7,3,1)-Balanced Incomplete Block Design
>>> G = P.automorphism_group()
>>> G.is_isomorphic(PGL(Integer(3),Integer(2)))
True
>>> G
Permutation Group with generators [...]
>>> G.cardinality()
168
```

A non self-dual example:

```
sage: IS = IncidenceStructure(list(range(4)), [[0,1,2,3],[1,2,3]])
sage: IS.automorphism_group().cardinality()                                  #␣
↪needs sage.groups
6
sage: IS.dual().automorphism_group().cardinality()                          #␣
↪needs sage.groups sage.modules
1
```

```
>>> from sage.all import *
>>> IS = IncidenceStructure(list(range(Integer(4))), [[Integer(0),Integer(1),
↪Integer(2),Integer(3)],[Integer(1),Integer(2),Integer(3)]])
>>> IS.automorphism_group().cardinality()                                   #␣
↪needs sage.groups
6
>>> IS.dual().automorphism_group().cardinality()                            #␣
↪needs sage.groups sage.modules
1
```

Examples with non-integer points:

```
sage: I = IncidenceStructure('abc', ('ab','ac','bc'))
sage: I.automorphism_group()                                                #␣
↪needs sage.groups
Permutation Group with generators [('b','c'), ('a','b')]
sage: IncidenceStructure([[(1,2),(3,4)]]).automorphism_group()              #␣
↪needs sage.groups
Permutation Group with generators [((1,2),(3,4))]
```

```
>>> from sage.all import *
>>> I = IncidenceStructure('abc', ('ab','ac','bc'))
>>> I.automorphism_group()                                                  #␣
↪needs sage.groups
Permutation Group with generators [('b','c'), ('a','b')]
>>> IncidenceStructure([[(Integer(1),Integer(2)),(Integer(3),Integer(4))]]).
```

(continues on next page)

(continued from previous page)

```
↪automorphism_group()              # needs sage.groups
Permutation Group with generators [((1,2),(3,4))]
```

block_sizes()

Return the set of block sizes.

EXAMPLES:

```
sage: BD = IncidenceStructure(8, [[0,1,3],[1,4,5,6],[1,2],[5,6,7]])
sage: BD.block_sizes()
[3, 2, 4, 3]
sage: BD = IncidenceStructure(7,[[0,1,2],[0,3,4],[0,5,6],[1,3,5],[1,4,6],[2,3,
↪6],[2,4,5]])
sage: BD.block_sizes()
[3, 3, 3, 3, 3, 3, 3]
```

```
>>> from sage.all import *
>>> BD = IncidenceStructure(Integer(8), [[Integer(0),Integer(1),Integer(3)],
↪[Integer(1),Integer(4),Integer(5),Integer(6)],[Integer(1),Integer(2)],
↪[Integer(5),Integer(6),Integer(7)]])
>>> BD.block_sizes()
[3, 2, 4, 3]
>>> BD = IncidenceStructure(Integer(7),[[Integer(0),Integer(1),Integer(2)],
↪[Integer(0),Integer(3),Integer(4)],[Integer(0),Integer(5),Integer(6)],
↪[Integer(1),Integer(3),Integer(5)],[Integer(1),Integer(4),Integer(6)],
↪[Integer(2),Integer(3),Integer(6)],[Integer(2),Integer(4),Integer(5)]])
>>> BD.block_sizes()
[3, 3, 3, 3, 3, 3, 3]
```

blocks()

Return the list of blocks.

EXAMPLES:

```
sage: BD = IncidenceStructure(7,[[0,1,2],[0,3,4],[0,5,6],[1,3,5],[1,4,6],[2,3,
↪6],[2,4,5]])
sage: BD.blocks()
[[0, 1, 2], [0, 3, 4], [0, 5, 6], [1, 3, 5], [1, 4, 6], [2, 3, 6], [2, 4, 5]]
```

```
>>> from sage.all import *
>>> BD = IncidenceStructure(Integer(7),[[Integer(0),Integer(1),Integer(2)],
↪[Integer(0),Integer(3),Integer(4)],[Integer(0),Integer(5),Integer(6)],
↪[Integer(1),Integer(3),Integer(5)],[Integer(1),Integer(4),Integer(6)],
↪[Integer(2),Integer(3),Integer(6)],[Integer(2),Integer(4),Integer(5)]])
>>> BD.blocks()
[[0, 1, 2], [0, 3, 4], [0, 5, 6], [1, 3, 5], [1, 4, 6], [2, 3, 6], [2, 4, 5]]
```

canonical_label()

Return a canonical label for the incidence structure.

A canonical label is relabeling of the points into integers $\{0, ..., n-1\}$ such that isomorphic incidence structures are relabelled to equal objects.

EXAMPLES:

```
sage: # needs sage.schemes
sage: fano1 = designs.balanced_incomplete_block_design(7,3)
sage: fano2 = designs.projective_plane(2)
sage: fano1 == fano2
False
sage: fano1.relabel(fano1.canonical_label())
sage: fano2.relabel(fano2.canonical_label())
sage: fano1 == fano2
True
```

```
>>> from sage.all import *
>>> # needs sage.schemes
>>> fano1 = designs.balanced_incomplete_block_design(Integer(7),Integer(3))
>>> fano2 = designs.projective_plane(Integer(2))
>>> fano1 == fano2
False
>>> fano1.relabel(fano1.canonical_label())
>>> fano2.relabel(fano2.canonical_label())
>>> fano1 == fano2
True
```

coloring(*k*, *solver=None*, *verbose=None*, *integrality_tolerance=0*)

Compute a (weak) k-coloring of the hypergraph.

A weak coloring of a hypergraph $\mathcal{H}$ is an assignment of colors to its vertices such that no set is monochromatic.

INPUT:

- `k` – integer; compute a coloring with k colors if an integer is provided, otherwise returns an optimal coloring (i.e. with the minimum possible number of colors).
- `solver` – (default: `None`) specify a Mixed Integer Linear Programming (MILP) solver to be used. If set to `None`, the default one is used. For more information on MILP solvers and which default solver is used, see the method `solve` of the class `MixedIntegerLinearProgram`.
- `verbose` – nonnegative integer (default: 0); set the level of verbosity you want from the linear program solver. Since the problem is NP-complete, its solving may take some time depending on the graph. A value of 0 means that there will be no message printed by the solver.
- `integrality_tolerance` – parameter for use with MILP solvers over an inexact base ring; see `MixedIntegerLinearProgram.get_values()`

EXAMPLES:

The Fano plane has chromatic number 3:

```
sage: len(designs.steiner_triple_system(7).coloring())                          #␣
↪needs sage.numerical.mip
3
```

```
>>> from sage.all import *
>>> len(designs.steiner_triple_system(Integer(7)).coloring())                   ␣
↪     # needs sage.numerical.mip
3
```

One admissible 3-coloring:

```
sage: designs.steiner_triple_system(7).coloring()    # not tested              #␣
↪needs sage.numerical.mip
[[0, 2, 5, 1], [4, 3], [6]]
```

```
>>> from sage.all import *
>>> designs.steiner_triple_system(Integer(7)).coloring()    # not tested      ␣
↪     # needs sage.numerical.mip
[[0, 2, 5, 1], [4, 3], [6]]
```

The chromatic number of a graph is equal to the chromatic number of its 2-uniform corresponding hypergraph:

```
sage: g = graphs.PetersenGraph()
sage: H = IncidenceStructure(g.edges(sort=True, labels=False))
sage: len(g.coloring())
3
sage: len(H.coloring())                                                        #␣
↪needs sage.numerical.mip
3
```

```
>>> from sage.all import *
>>> g = graphs.PetersenGraph()
>>> H = IncidenceStructure(g.edges(sort=True, labels=False))
>>> len(g.coloring())
3
>>> len(H.coloring())                                                          #␣
↪needs sage.numerical.mip
3
```

complement(*uniform=False*)

Return the complement of the incidence structure.

Two different definitions of "complement" are made available, according to the value of `uniform`.

INPUT:

- `uniform` – boolean
  - if set to `False` (default), returns the incidence structure whose blocks are the complements of all blocks of the incidence structure.
  - If set to `True` and the incidence structure is k-uniform, returns the incidence structure whose blocks are all k-sets of the ground set that do not appear in `self`.

EXAMPLES:

The complement of a `BalancedIncompleteBlockDesign` is also a 2-design:

```
sage: bibd = designs.balanced_incomplete_block_design(13,4)                    #␣
↪needs sage.schemes
sage: bibd.is_t_design(return_parameters=True)                                 #␣
↪needs sage.schemes
(True, (2, 13, 4, 1))
sage: bibd.complement().is_t_design(return_parameters=True)                    #␣
↪needs sage.schemes
(True, (2, 13, 9, 6))
```

```
>>> from sage.all import *
>>> bibd = designs.balanced_incomplete_block_design(Integer(13),Integer(4))
↪              # needs sage.schemes
>>> bibd.is_t_design(return_parameters=True)                                #␣
↪needs sage.schemes
(True, (2, 13, 4, 1))
>>> bibd.complement().is_t_design(return_parameters=True)                   #␣
↪needs sage.schemes
(True, (2, 13, 9, 6))
```

The "uniform" complement of a graph is a graph:

```
sage: g = graphs.PetersenGraph()
sage: G = IncidenceStructure(g.edges(sort=True, labels=False))
sage: H = G.complement(uniform=True)
sage: h = Graph(H.blocks())
sage: g == h
False
sage: g == h.complement()
True
```

```
>>> from sage.all import *
>>> g = graphs.PetersenGraph()
>>> G = IncidenceStructure(g.edges(sort=True, labels=False))
>>> H = G.complement(uniform=True)
>>> h = Graph(H.blocks())
>>> g == h
False
>>> g == h.complement()
True
```

copy()

Return a copy of the incidence structure.

EXAMPLES:

```
sage: IS = IncidenceStructure([[1,2,3,"e"]], name='Test')
sage: IS
Incidence structure with 4 points and 1 blocks
sage: copy(IS)
Incidence structure with 4 points and 1 blocks
sage: [1, 2, 3, 'e'] in copy(IS)
True
sage: copy(IS)._name
'Test'
```

```
>>> from sage.all import *
>>> IS = IncidenceStructure([[Integer(1),Integer(2),Integer(3),"e"]], name=
↪'Test')
>>> IS
Incidence structure with 4 points and 1 blocks
>>> copy(IS)
Incidence structure with 4 points and 1 blocks
```

(continues on next page)

(continued from previous page)

```
>>> [Integer(1), Integer(2), Integer(3), 'e'] in copy(IS)
True
>>> copy(IS)._name
'Test'
```

degree(*p=None*, *subset=False*)

Return the degree of a point `p` (or a set of points).

The degree of a point (or set of points) is the number of blocks that contain it.

INPUT:

- `p` – a point (or a set of points) of the incidence structure
- `subset` – boolean (default: `False`); whether to interpret the argument as a set of point or as a point (default)

EXAMPLES:

```
sage: designs.steiner_triple_system(9).degree(3)
4
sage: designs.steiner_triple_system(9).degree({1,2},subset=True)
1
```

```
>>> from sage.all import *
>>> designs.steiner_triple_system(Integer(9)).degree(Integer(3))
4
>>> designs.steiner_triple_system(Integer(9)).degree({Integer(1),Integer(2)},
↪subset=True)
1
```

degrees(*size=None*)

Return the degree of all sets of given size, or the degree of all points.

The degree of a point (or set of point) is the number of blocks that contain it.

INPUT:

- `size` – integer; return the degree of all subsets of points of cardinality `size`. When `size=None`, the function outputs the degree of all points.

> **Note**
>
> When `size=None` the output is indexed by the points. When `size=1` it is indexed by tuples of size 1. This is the same information, stored slightly differently.

OUTPUT: a dictionary whose values are degrees and keys are either:

- the points of the incidence structure if `size=None` (default)
- the subsets of size `size` of the points stored as tuples

EXAMPLES:

```
sage: IncidenceStructure([[1,2,3],[1,4]]).degrees(2)
{(1, 2): 1, (1, 3): 1, (1, 4): 1, (2, 3): 1, (2, 4): 0, (3, 4): 0}
```

```
>>> from sage.all import *
>>> IncidenceStructure([[Integer(1),Integer(2),Integer(3)],[Integer(1),
↪Integer(4)]]).degrees(Integer(2))
{(1, 2): 1, (1, 3): 1, (1, 4): 1, (2, 3): 1, (2, 4): 0, (3, 4): 0}
```

In a Steiner triple system, all pairs have degree 1:

```
sage: S13 = designs.steiner_triple_system(13)
sage: all(v == 1 for v in S13.degrees(2).values())
True
```

```
>>> from sage.all import *
>>> S13 = designs.steiner_triple_system(Integer(13))
>>> all(v == Integer(1) for v in S13.degrees(Integer(2)).values())
True
```

dual(*algorithm=None*)

Return the dual of the incidence structure.

INPUT:

- `algorithm` – whether to use Sage's implementation (`algorithm=None`, default) or use GAP's (`algorithm='gap'`)

> **Note**
>
> The `algorithm='gap'` option requires GAP's Design package (included in the `gap_packages` Sage spkg).

EXAMPLES:

The dual of a projective plane is a projective plane:

```
sage: PP = designs.DesarguesianProjectivePlaneDesign(4)                    #␣
↪needs sage.rings.finite_rings
sage: PP.dual().is_t_design(return_parameters=True)                        #␣
↪needs sage.modules sage.rings.finite_rings
(True, (2, 21, 5, 1))
```

```
>>> from sage.all import *
>>> PP = designs.DesarguesianProjectivePlaneDesign(Integer(4))                ␣
↪     # needs sage.rings.finite_rings
>>> PP.dual().is_t_design(return_parameters=True)                           #␣
↪needs sage.modules sage.rings.finite_rings
(True, (2, 21, 5, 1))
```

REFERENCE:

- Leonard Soicher, Design package manual

edge_coloring()

Compute a proper edge-coloring.

A proper edge-coloring is an assignment of colors to the sets of the incidence structure such that two sets with non-empty intersection receive different colors. The coloring returned minimizes the number of colors.

OUTPUT: a partition of the sets into color classes

EXAMPLES:

```
sage: H = Hypergraph([{1,2,3},{2,3,4},{3,4,5},{4,5,6}]); H
Incidence structure with 6 points and 4 blocks
sage: C = H.edge_coloring()
sage: C # random
[[[3, 4, 5]], [[2, 3, 4]], [[4, 5, 6], [1, 2, 3]]]
sage: Set(map(Set,sum(C,[]))) == Set(map(Set,H.blocks()))
True
```

```
>>> from sage.all import *
>>> H = Hypergraph([{Integer(1),Integer(2),Integer(3)},{Integer(2),Integer(3),
↪Integer(4)},{Integer(3),Integer(4),Integer(5)},{Integer(4),Integer(5),
↪Integer(6)}]); H
Incidence structure with 6 points and 4 blocks
>>> C = H.edge_coloring()
>>> C # random
[[[3, 4, 5]], [[2, 3, 4]], [[4, 5, 6], [1, 2, 3]]]
>>> Set(map(Set,sum(C,[]))) == Set(map(Set,H.blocks()))
True
```

ground_set()

Return the ground set (i.e the list of points).

EXAMPLES:

```
sage: IncidenceStructure(3, [[0,1],[0,2]]).ground_set()
[0, 1, 2]
```

```
>>> from sage.all import *
>>> IncidenceStructure(Integer(3), [[Integer(0),Integer(1)],[Integer(0),
↪Integer(2)]]).ground_set()
[0, 1, 2]
```

incidence_graph(*labels=False*)

Return the incidence graph of the incidence structure.

A point and a block are adjacent in this graph whenever they are incident.

INPUT:

- `labels` – boolean; whether to return a graph whose vertices are integers, or labelled elements
  - `labels is False` – default; in this case the first vertices of the graphs are the elements of *`ground_set()`*, and appear in the same order. Similarly, the following vertices represent the elements of *`blocks()`*, and appear in the same order.
  - `labels is True`, the points keep their original labels, and the blocks are `Set` objects.

    Note that the labelled incidence graph can be incorrect when blocks are repeated, and on some (rare) occasions when the elements of *`ground_set()`* mix `Set()` and non-`Set` objects.

EXAMPLES:

```
sage: BD = IncidenceStructure(7, [[0,1,2],[0,3,4],[0,5,6],[1,3,5],
....:                             [1,4,6],[2,3,6],[2,4,5]])
sage: BD.incidence_graph()                                                  #␣
↪needs sage.modules
Bipartite graph on 14 vertices
sage: A = BD.incidence_matrix()                                             #␣
↪needs sage.modules
sage: Graph(block_matrix([[A*0, A],                                         #␣
↪needs sage.modules
....:                     [A.transpose(),A*0]])) == BD.incidence_graph()
True
```

```
>>> from sage.all import *
>>> BD = IncidenceStructure(Integer(7), [[Integer(0),Integer(1),Integer(2)],
↪[Integer(0),Integer(3),Integer(4)],[Integer(0),Integer(5),Integer(6)],
↪[Integer(1),Integer(3),Integer(5)],
...                               [Integer(1),Integer(4),Integer(6)],
↪[Integer(2),Integer(3),Integer(6)],[Integer(2),Integer(4),Integer(5)]])
>>> BD.incidence_graph()                                                    #␣
↪needs sage.modules
Bipartite graph on 14 vertices
>>> A = BD.incidence_matrix()                                               #␣
↪needs sage.modules
>>> Graph(block_matrix([[A*Integer(0), A],                                    ␣
↪     # needs sage.modules
...                     [A.transpose(),A*Integer(0)]])) == BD.incidence_
↪graph()
True
```

incidence_matrix()

Return the incidence matrix A of the design. A is a $(v \times b)$ matrix defined by: `A[i,j] = 1` if `i` is in block `B_j` and 0 otherwise.

EXAMPLES:

```
sage: BD = IncidenceStructure(7, [[0,1,2],[0,3,4],[0,5,6],[1,3,5],
....:                             [1,4,6],[2,3,6],[2,4,5]])
sage: BD.block_sizes()
[3, 3, 3, 3, 3, 3, 3]
sage: BD.incidence_matrix()                                                 #␣
↪needs sage.modules
[1 1 1 0 0 0 0]
[1 0 0 1 1 0 0]
[1 0 0 0 0 1 1]
[0 1 0 1 0 1 0]
[0 1 0 0 1 0 1]
[0 0 1 1 0 0 1]
[0 0 1 0 1 1 0]

sage: I = IncidenceStructure('abc', ('ab','abc','ac','c'))
sage: I.incidence_matrix()                                                  #␣
↪needs sage.modules
[1 1 1 0]
```

(continues on next page)

(continued from previous page)

```
[1 1 0 0]
[0 1 1 1]
```

```
>>> from sage.all import *
>>> BD = IncidenceStructure(Integer(7), [[Integer(0),Integer(1),Integer(2)],
↪[Integer(0),Integer(3),Integer(4)],[Integer(0),Integer(5),Integer(6)],
↪[Integer(1),Integer(3),Integer(5)],
...                                [Integer(1),Integer(4),Integer(6)],
↪[Integer(2),Integer(3),Integer(6)],[Integer(2),Integer(4),Integer(5)]])
>>> BD.block_sizes()
[3, 3, 3, 3, 3, 3, 3]
>>> BD.incidence_matrix()                                                    #␣
↪needs sage.modules
[1 1 1 0 0 0 0]
[1 0 0 1 1 0 0]
[1 0 0 0 0 1 1]
[0 1 0 1 0 1 0]
[0 1 0 0 1 0 1]
[0 0 1 1 0 0 1]
[0 0 1 0 1 1 0]

>>> I = IncidenceStructure('abc', ('ab','abc','ac','c'))
>>> I.incidence_matrix()                                                    #␣
↪needs sage.modules
[1 1 1 0]
[1 1 0 0]
[0 1 1 1]
```

induced_substructure(*points*)

Return the substructure induced by a set of points.

The substructure induced in $\mathcal{H}$ by a set $X \subseteq V(\mathcal{H})$ of points is the incidence structure $\mathcal{H}_X$ defined on X whose sets are all $S \in \mathcal{H}$ such that $S \subseteq X$.

INPUT:

- `points` – set of points

Note

This method goes over all sets of `self` before building a new *`IncidenceStructure`* (which involves some relabelling and sorting). It probably should not be called in a performance-critical code.

EXAMPLES:

A Fano plane with one point removed:

```
sage: F = designs.steiner_triple_system(7)
sage: F.induced_substructure([0..5])
Incidence structure with 6 points and 4 blocks
```

```
sage: from sage.all import *
sage: F = designs.steiner_triple_system(Integer(7))
sage: F.induced_substructure((ellipsis_range(Integer(0),Ellipsis,Integer(5))))
Incidence structure with 6 points and 4 blocks
```

intersection_graph(*sizes=None*, *immutable=False*)

Return the intersection graph of the incidence structure.

The vertices of this graph are the *blocks()* of the incidence structure. Two of them are adjacent if the size of their intersection belongs to the set `sizes`.

INPUT:

- `sizes` – list/set of integers; for convenience, setting `sizes` to `5` has the same effect as `sizes=[5]`. When set to `None` (default), behaves as `sizes=PositiveIntegers()`.
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph

EXAMPLES:

The intersection graph of a `balanced_incomplete_block_design()` is a *strongly regular graph* (when it is not trivial):

```
sage: BIBD =  designs.balanced_incomplete_block_design(19,3)
sage: G = BIBD.intersection_graph(1)
sage: G.is_strongly_regular(parameters=True)
(57, 24, 11, 9)
```

```
sage: from sage.all import *
sage: BIBD =  designs.balanced_incomplete_block_design(Integer(19),Integer(3))
sage: G = BIBD.intersection_graph(Integer(1))
sage: G.is_strongly_regular(parameters=True)
(57, 24, 11, 9)
```

is_berge_cyclic()

Check whether `self` is a Berge-Cyclic uniform hypergraph.

A k-uniform Berge cycle (named after Claude Berge) of length ℓ is a cyclic list of distinct k-sets $F_1, \ldots, F_\ell$, $\ell > 1$, and distinct vertices $C = \{v_1, \ldots, v_\ell\}$ such that for each $1 \leq i \leq \ell$, F_i contains v_i and v_{i+1} (where $v_{l+1} = v_1$).

A uniform hypergraph is Berge-cyclic if its incidence graph is cyclic. It is called "Berge-acyclic" otherwise.

For more information, see [Fag1983] and Wikipedia article Hypergraph.

EXAMPLES:

```
sage: Hypergraph(5, [[1, 2, 3], [2, 3, 4]]).is_berge_cyclic()                #␣
↪needs sage.modules
True
sage: Hypergraph(6, [[1, 2, 3], [3, 4, 5]]).is_berge_cyclic()                #␣
↪needs sage.modules
False
```

```
sage: from sage.all import *
sage: Hypergraph(Integer(5), [[Integer(1), Integer(2), Integer(3)], [Integer(2),
↪ Integer(3), Integer(4)]]).is_berge_cyclic()                # needs sage.
```

(continues on next page)

(continued from previous page)

```
↪modules
True
>>> Hypergraph(Integer(6), [[Integer(1), Integer(2), Integer(3)], [Integer(3),
↪ Integer(4), Integer(5)]]).is_berge_cyclic()                # needs sage.
↪modules
False
```

is_connected()

Test whether the design is connected.

EXAMPLES:

```
sage: IncidenceStructure(3, [[0,1],[0,2]]).is_connected()
True
sage: IncidenceStructure(4, [[0,1],[2,3]]).is_connected()
False
```

```
>>> from sage.all import *
>>> IncidenceStructure(Integer(3), [[Integer(0),Integer(1)],[Integer(0),
↪Integer(2)]]).is_connected()
True
>>> IncidenceStructure(Integer(4), [[Integer(0),Integer(1)],[Integer(2),
↪Integer(3)]]).is_connected()
False
```

is_generalized_quadrangle(*verbose=False*, *parameters=False*)

Test if the incidence structure is a generalized quadrangle.

An incidence structure is a generalized quadrangle iff (see [BH2012], section 9.6):

- two blocks intersect on at most one point.
- For every point p not in a block B, there is a unique block B' intersecting both $\{p\}$ and B

It is a *regular* generalized quadrangle if furthermore:

- it is $s+1$-*uniform* for some positive integer s.
- it is $t+1$-*regular* for some positive integer t.

For more information, see the Wikipedia article Generalized_quadrangle.

Note

Some references (e.g. [PT2009] or Wikipedia article Generalized_quadrangle) only allow *regular* generalized quadrangles. To use such a definition, see the `parameters` optional argument described below, or the methods *is_regular()* and *is_uniform()*.

INPUT:

- `verbose` – boolean; whether to print an explanation when the instance is not a generalized quadrangle
- `parameters` – boolean (default: `False`); if set to `True`, the function returns a pair `(s,t)` instead of `True` answers. In this case, s and t are the integers defined above if they exist (each can be set to `False` otherwise).

EXAMPLES:

```
sage: h = designs.CremonaRichmondConfiguration()                    #␣
↪needs networkx
sage: h.is_generalized_quadrangle()                                 #␣
↪needs networkx
True
```

```
>>> from sage.all import *
>>> h = designs.CremonaRichmondConfiguration()                      #␣
↪needs networkx
>>> h.is_generalized_quadrangle()                                   #␣
↪needs networkx
True
```

This is actually a *regular* generalized quadrangle:

```
sage: h.is_generalized_quadrangle(parameters=True)                  #␣
↪needs networkx
(2, 2)
```

```
>>> from sage.all import *
>>> h.is_generalized_quadrangle(parameters=True)                    #␣
↪needs networkx
(2, 2)
```

is_isomorphic(*other*, *certificate=False*)

Return whether the two incidence structures are isomorphic.

INPUT:

- `other` – an incidence structure
- `certificate` – boolean (default: `False`); whether to return an isomorphism from `self` to `other` instead of a boolean answer

EXAMPLES:

```
sage: # needs sage.schemes
sage: fano1 = designs.balanced_incomplete_block_design(7,3)
sage: fano2 = designs.projective_plane(2)
sage: fano1.is_isomorphic(fano2)
True
sage: fano1.is_isomorphic(fano2,certificate=True)
{0: 0, 1: 1, 2: 2, 3: 6, 4: 4, 5: 3, 6: 5}
```

```
>>> from sage.all import *
>>> # needs sage.schemes
>>> fano1 = designs.balanced_incomplete_block_design(Integer(7),Integer(3))
>>> fano2 = designs.projective_plane(Integer(2))
>>> fano1.is_isomorphic(fano2)
True
>>> fano1.is_isomorphic(fano2,certificate=True)
{0: 0, 1: 1, 2: 2, 3: 6, 4: 4, 5: 3, 6: 5}
```

is_regular(*r=None*)

Test whether the incidence structure is r-regular.

An incidence structure is said to be r-regular if all its points are incident with exactly r blocks.

INPUT:

- `r` – integer

OUTPUT:

If `r` is defined, a boolean is returned. If `r` is set to `None` (default), the method returns either `False` or the integer `r` such that the incidence structure is r-regular.

Warning

In case of 0-regular incidence structure, beware that `if not H.is_regular()` is a satisfied condition.

EXAMPLES:

```
sage: designs.balanced_incomplete_block_design(7,3).is_regular()              #␣
↪needs sage.schemes
3
sage: designs.balanced_incomplete_block_design(7,3).is_regular(r=3)           #␣
↪needs sage.schemes
True
sage: designs.balanced_incomplete_block_design(7,3).is_regular(r=4)           #␣
↪needs sage.schemes
False
```

```
>>> from sage.all import *
>>> designs.balanced_incomplete_block_design(Integer(7),Integer(3)).is_
↪regular()             # needs sage.schemes
3
>>> designs.balanced_incomplete_block_design(Integer(7),Integer(3)).is_
↪regular(r=Integer(3))          # needs sage.schemes
True
>>> designs.balanced_incomplete_block_design(Integer(7),Integer(3)).is_
↪regular(r=Integer(4))          # needs sage.schemes
False
```

is_resolvable(*certificate*, *solver=False*, *verbose=None*, *check=0*, *integrality_tolerance=True*)

Test whether the hypergraph is resolvable.

A hypergraph is said to be resolvable if its sets can be partitionned into classes, each of which is a partition of the ground set.

Note

This problem is solved using an Integer Linear Program, and GLPK (the default LP solver) has been reported to be very slow on some instances. If you hit this wall, consider installing a more powerful MILP solver (CPLEX, Gurobi, …).

INPUT:

- `certificate` – boolean; whether to return the classes along with the binary answer (see examples below)

- `solver` – (default: `None`) specify a Mixed Integer Linear Programming (MILP) solver to be used. If set to `None`, the default one is used. For more information on MILP solvers and which default solver is used, see the method `solve` of the class `MixedIntegerLinearProgram`.
- `verbose` – integer (default: 0); sets the level of verbosity. Set to 0 by default, which means quiet.
- `check` – boolean (default: `True`); whether to check that output is correct before returning it. As this is expected to be useless, you may want to disable it whenever you want speed.
- `integrality_tolerance` – parameter for use with MILP solvers over an inexact base ring; see `MixedIntegerLinearProgram.get_values()`

EXAMPLES:

Some resolvable designs:

```
sage: TD = designs.transversal_design(2,2,resolvable=True)
sage: TD.is_resolvable()
True

sage: AG = designs.AffineGeometryDesign(3,1,GF(2))                              #␣
↪needs sage.combinat
sage: AG.is_resolvable()                                                        #␣
↪needs sage.combinat
True
```

```
>>> from sage.all import *
>>> TD = designs.transversal_design(Integer(2),Integer(2),resolvable=True)
>>> TD.is_resolvable()
True

>>> AG = designs.AffineGeometryDesign(Integer(3),Integer(1),GF(Integer(2)))  ␣
↪                        # needs sage.combinat
>>> AG.is_resolvable()                                                          #␣
↪needs sage.combinat
True
```

Their classes:

```
sage: b, cls = TD.is_resolvable(True)
sage: b
True
sage: cls # random
[[[0, 3], [1, 2]], [[1, 3], [0, 2]]]

sage: # needs sage.combinat
sage: b, cls = AG.is_resolvable(True)
sage: b
True
sage: cls  # random
[[[6, 7], [4, 5], [0, 1], [2, 3]],
 [[5, 7], [0, 4], [3, 6], [1, 2]],
 [[0, 2], [4, 7], [1, 3], [5, 6]],
 [[3, 4], [0, 7], [1, 5], [2, 6]],
 [[3, 7], [1, 6], [0, 5], [2, 4]],
```

(continues on next page)

(continued from previous page)

```
 [[0, 6], [2, 7], [1, 4], [3, 5]],
 [[4, 6], [0, 3], [2, 5], [1, 7]]]
```

```
>>> from sage.all import *
>>> b, cls = TD.is_resolvable(True)
>>> b
True
>>> cls # random
[[[0, 3], [1, 2]], [[1, 3], [0, 2]]]

>>> # needs sage.combinat
>>> b, cls = AG.is_resolvable(True)
>>> b
True
>>> cls  # random
[[[6, 7], [4, 5], [0, 1], [2, 3]],
 [[5, 7], [0, 4], [3, 6], [1, 2]],
 [[0, 2], [4, 7], [1, 3], [5, 6]],
 [[3, 4], [0, 7], [1, 5], [2, 6]],
 [[3, 7], [1, 6], [0, 5], [2, 4]],
 [[0, 6], [2, 7], [1, 4], [3, 5]],
 [[4, 6], [0, 3], [2, 5], [1, 7]]]
```

A non-resolvable design:

```
sage: Fano = designs.balanced_incomplete_block_design(7,3)                    #␣
↪needs sage.schemes
sage: Fano.is_resolvable()                                                    #␣
↪needs sage.schemes
False
sage: Fano.is_resolvable(True)                                                #␣
↪needs sage.schemes
(False, [])
```

```
>>> from sage.all import *
>>> Fano = designs.balanced_incomplete_block_design(Integer(7),Integer(3))    ␣
↪              # needs sage.schemes
>>> Fano.is_resolvable()                                                      #␣
↪needs sage.schemes
False
>>> Fano.is_resolvable(True)                                                  #␣
↪needs sage.schemes
(False, [])
```

is_simple()

Test whether this design is simple (i.e. no repeated block).

EXAMPLES:

```
sage: IncidenceStructure(3, [[0,1],[1,2],[0,2]]).is_simple()
True
sage: IncidenceStructure(3, [[0],[0]]).is_simple()
```

(continues on next page)

(continued from previous page)

```
False

sage: V = [(0,'a'),(0,'b'),(1,'a'),(1,'b')]
sage: B = [[V[0],V[1]], [V[1],V[2]]]
sage: I = IncidenceStructure(V, B)
sage: I.is_simple()
True
sage: I2 = IncidenceStructure(V, B*2)
sage: I2.is_simple()
False
```

```
>>> from sage.all import *
>>> IncidenceStructure(Integer(3), [[Integer(0),Integer(1)],[Integer(1),
↪Integer(2)],[Integer(0),Integer(2)]]).is_simple()
True
>>> IncidenceStructure(Integer(3), [[Integer(0)],[Integer(0)]]).is_simple()
False

>>> V = [(Integer(0),'a'),(Integer(0),'b'),(Integer(1),'a'),(Integer(1),'b')]
>>> B = [[V[Integer(0)],V[Integer(1)]], [V[Integer(1)],V[Integer(2)]]]
>>> I = IncidenceStructure(V, B)
>>> I.is_simple()
True
>>> I2 = IncidenceStructure(V, B*Integer(2))
>>> I2.is_simple()
False
```

is_spread(*spread*)

Check whether the input is a spread for `self`.

A spread of an incidence structure (P, B) is a subset of B which forms a partition of P.

INPUT:

- `spread` – iterable; defines the spread

EXAMPLES:

```
sage: E = IncidenceStructure([[1, 2, 3], [4, 5, 6], [1, 5, 6]])
sage: E.is_spread([[1, 2, 3], [4, 5, 6]])
True
sage: E.is_spread([1, 2, 3, 4, 5, 6])
Traceback (most recent call last):
...
TypeError: 'sage.rings.integer.Integer' object is not iterable
sage: E.is_spread([[1, 2, 3, 4], [5, 6]])
False
```

```
>>> from sage.all import *
>>> E = IncidenceStructure([[Integer(1), Integer(2), Integer(3)], [Integer(4),
↪ Integer(5), Integer(6)], [Integer(1), Integer(5), Integer(6)]])
>>> E.is_spread([[Integer(1), Integer(2), Integer(3)], [Integer(4),␣
↪Integer(5), Integer(6)]])
```

(continues on next page)

(continued from previous page)

```
True
>>> E.is_spread([Integer(1), Integer(2), Integer(3), Integer(4), Integer(5),␣
↪Integer(6)])
Traceback (most recent call last):
...
TypeError: 'sage.rings.integer.Integer' object is not iterable
>>> E.is_spread([[Integer(1), Integer(2), Integer(3), Integer(4)],␣
↪[Integer(5), Integer(6)]])
False
```

Order of blocks or of points within each block doesn't matter:

```
sage: E = IncidenceStructure([[1, 2, 3], [4, 5, 6], [1, 5, 6]])
sage: E.is_spread([[5, 6, 4], [3, 1, 2]])
True
```

```
>>> from sage.all import *
>>> E = IncidenceStructure([[Integer(1), Integer(2), Integer(3)], [Integer(4),
↪ Integer(5), Integer(6)], [Integer(1), Integer(5), Integer(6)]])
>>> E.is_spread([[Integer(5), Integer(6), Integer(4)], [Integer(3),␣
↪Integer(1), Integer(2)]])
True
```

is_t_design (*t=None*, *v=None*, *k=None*, *l=None*, *return_parameters=False*)

Test whether `self` is a $t-(v,k,l)$ design.

A $t-(v,k,\lambda)$ (sometimes called t-design for short) is a block design in which:

- the underlying set has cardinality v
- the blocks have size k
- each t-subset of points is covered by λ blocks

INPUT:

- `t`, `v`, `k`, `l` – integers; their value is set to `None` by default. The function tests whether the design is a $t-(v,k,l)$ design using the provided values and guesses the others. Note that `l` cannot be specified if `t` is not.
- `return_parameters` – boolean; whether to return the parameters of the t-design. If set to `True`, the function returns a pair `(boolean_answer,(t,v,k,l))`.

EXAMPLES:

```
sage: fano_blocks = [[0,1,2],[0,3,4],[0,5,6],[1,3,5],[1,4,6],[2,3,6],[2,4,5]]
sage: BD = IncidenceStructure(7, fano_blocks)
sage: BD.is_t_design()
True
sage: BD.is_t_design(return_parameters=True)
(True, (2, 7, 3, 1))
sage: BD.is_t_design(2, 7, 3, 1)
True
sage: BD.is_t_design(1, 7, 3, 3)
True
sage: BD.is_t_design(0, 7, 3, 7)
```

(continues on next page)

(continued from previous page)

```
True

sage: BD.is_t_design(0,6,3,7) or BD.is_t_design(0,7,4,7) or BD.is_t_design(0,
↪7,3,8)
False

sage: BD = designs.AffineGeometryDesign(3, 1, GF(2))                          #␣
↪needs sage.combinat
sage: BD.is_t_design(1)                                                       #␣
↪needs sage.combinat
True
sage: BD.is_t_design(2)                                                       #␣
↪needs sage.combinat
True
```

```
>>> from sage.all import *
>>> fano_blocks = [[Integer(0),Integer(1),Integer(2)],[Integer(0),Integer(3),
↪Integer(4)],[Integer(0),Integer(5),Integer(6)],[Integer(1),Integer(3),
↪Integer(5)],[Integer(1),Integer(4),Integer(6)],[Integer(2),Integer(3),
↪Integer(6)],[Integer(2),Integer(4),Integer(5)]]
>>> BD = IncidenceStructure(Integer(7), fano_blocks)
>>> BD.is_t_design()
True
>>> BD.is_t_design(return_parameters=True)
(True, (2, 7, 3, 1))
>>> BD.is_t_design(Integer(2), Integer(7), Integer(3), Integer(1))
True
>>> BD.is_t_design(Integer(1), Integer(7), Integer(3), Integer(3))
True
>>> BD.is_t_design(Integer(0), Integer(7), Integer(3), Integer(7))
True

>>> BD.is_t_design(Integer(0),Integer(6),Integer(3),Integer(7)) or BD.is_t_
↪design(Integer(0),Integer(7),Integer(4),Integer(7)) or BD.is_t_
↪design(Integer(0),Integer(7),Integer(3),Integer(8))
False

>>> BD = designs.AffineGeometryDesign(Integer(3), Integer(1), GF(Integer(2)))␣
↪                        # needs sage.combinat
>>> BD.is_t_design(Integer(1))                                                ␣
↪     # needs sage.combinat
True
>>> BD.is_t_design(Integer(2))                                                ␣
↪     # needs sage.combinat
True
```

Steiner triple and quadruple systems are other names for $2-(v,3,1)$ and $3-(v,4,1)$ designs:

```
sage: S3_9 = designs.steiner_triple_system(9)
sage: S3_9.is_t_design(2,9,3,1)
True
```

(continues on next page)

(continued from previous page)

```
sage: blocks = designs.steiner_quadruple_system(8)
sage: S4_8 = IncidenceStructure(8, blocks)
sage: S4_8.is_t_design(3,8,4,1)
True

sage: blocks = designs.steiner_quadruple_system(14)
sage: S4_14 = IncidenceStructure(14, blocks)
sage: S4_14.is_t_design(3,14,4,1)
True
```

```
>>> from sage.all import *
>>> S3_9 = designs.steiner_triple_system(Integer(9))
>>> S3_9.is_t_design(Integer(2),Integer(9),Integer(3),Integer(1))
True

>>> blocks = designs.steiner_quadruple_system(Integer(8))
>>> S4_8 = IncidenceStructure(Integer(8), blocks)
>>> S4_8.is_t_design(Integer(3),Integer(8),Integer(4),Integer(1))
True

>>> blocks = designs.steiner_quadruple_system(Integer(14))
>>> S4_14 = IncidenceStructure(Integer(14), blocks)
>>> S4_14.is_t_design(Integer(3),Integer(14),Integer(4),Integer(1))
True
```

Some examples of Witt designs that need the gap database:

```
sage: # optional - gap_package_design
sage: BD = designs.WittDesign(9)
sage: BD.is_t_design(2,9,3,1)
True
sage: W12 = designs.WittDesign(12)
sage: W12.is_t_design(5,12,6,1)
True
sage: W12.is_t_design(4)
True
```

```
>>> from sage.all import *
>>> # optional - gap_package_design
>>> BD = designs.WittDesign(Integer(9))
>>> BD.is_t_design(Integer(2),Integer(9),Integer(3),Integer(1))
True
>>> W12 = designs.WittDesign(Integer(12))
>>> W12.is_t_design(Integer(5),Integer(12),Integer(6),Integer(1))
True
>>> W12.is_t_design(Integer(4))
True
```

Further examples:

```
sage: D = IncidenceStructure(4,[[],[]])
sage: D.is_t_design(return_parameters=True)
```

(continues on next page)

(continued from previous page)

```
(True,  (0, 4, 0, 2))

sage: D = IncidenceStructure(4, [[0,1],[0,2],[0,3]])
sage: D.is_t_design(return_parameters=True)
(True, (0, 4, 2, 3))

sage: D = IncidenceStructure(4, [[0],[1],[2],[3]])
sage: D.is_t_design(return_parameters=True)
(True, (1, 4, 1, 1))

sage: D = IncidenceStructure(4,[[0,1],[2,3]])
sage: D.is_t_design(return_parameters=True)
(True, (1, 4, 2, 1))

sage: D = IncidenceStructure(4, [list(range(4))])
sage: D.is_t_design(return_parameters=True)
(True, (4, 4, 4, 1))
```

```
>>> from sage.all import *
>>> D = IncidenceStructure(Integer(4),[[],[]])
>>> D.is_t_design(return_parameters=True)
(True,  (0, 4, 0, 2))

>>> D = IncidenceStructure(Integer(4), [[Integer(0),Integer(1)],[Integer(0),
↪Integer(2)],[Integer(0),Integer(3)]])
>>> D.is_t_design(return_parameters=True)
(True, (0, 4, 2, 3))

>>> D = IncidenceStructure(Integer(4), [[Integer(0)],[Integer(1)],
↪[Integer(2)],[Integer(3)]])
>>> D.is_t_design(return_parameters=True)
(True, (1, 4, 1, 1))

>>> D = IncidenceStructure(Integer(4),[[Integer(0),Integer(1)],[Integer(2),
↪Integer(3)]])
>>> D.is_t_design(return_parameters=True)
(True, (1, 4, 2, 1))

>>> D = IncidenceStructure(Integer(4), [list(range(Integer(4)))])
>>> D.is_t_design(return_parameters=True)
(True, (4, 4, 4, 1))
```

is_uniform(*k=None*)

Test whether the incidence structure is k-uniform

An incidence structure is said to be k-uniform if all its blocks have size k.

INPUT:

- `k` – integer

OUTPUT:

If `k` is defined, a boolean is returned. If `k` is set to `None` (default), the method returns either `False` or the integer `k` such that the incidence structure is k-uniform.

Warning

In case of 0-uniform incidence structure, beware that `if not H.is_uniform()` is a satisfied condition.

EXAMPLES:

```
sage: designs.balanced_incomplete_block_design(7,3).is_uniform()            #␣
↪needs sage.schemes
3
sage: designs.balanced_incomplete_block_design(7,3).is_uniform(k=3)         #␣
↪needs sage.schemes
True
sage: designs.balanced_incomplete_block_design(7,3).is_uniform(k=4)         #␣
↪needs sage.schemes
False
```

```
>>> from sage.all import *
>>> designs.balanced_incomplete_block_design(Integer(7),Integer(3)).is_
↪uniform()            # needs sage.schemes
3
>>> designs.balanced_incomplete_block_design(Integer(7),Integer(3)).is_
↪uniform(k=Integer(3))          # needs sage.schemes
True
>>> designs.balanced_incomplete_block_design(Integer(7),Integer(3)).is_
↪uniform(k=Integer(4))          # needs sage.schemes
False
```

isomorphic_substructures_iterator(*H2*, *induced=False*)

Iterate over all copies of `H2` contained in `self`.

A hypergraph H_1 contains an isomorphic copy of a hypergraph H_2 if there exists an injection $f : V(H_2) \mapsto V(H_1)$ such that for any set $S_2 \in E(H_2)$ the set $S_1 = f(S2)$ belongs to $E(H_1)$.

It is an *induced* copy if no other set of $E(H_1)$ is contained in $f(V(H_2))$, i.e. $|E(H_2)| = \{S : S \in E(H_1)$ and $f(V(H_2))\}$.

This function lists all such injections. In particular, the number of copies of H in itself is equal to *the size of its automorphism group*.

See `subhypergraph_search` for more information.

INPUT:

- `H2` – an *IncidenceStructure* object
- `induced` – boolean (default: `False`); whether to require the copies to be induced

EXAMPLES:

How many distinct C_5 in Petersen's graph ?

```
sage: P = graphs.PetersenGraph()
sage: C = graphs.CycleGraph(5)
sage: IP = IncidenceStructure(P.edges(sort=True, labels=False))
sage: IC = IncidenceStructure(C.edges(sort=True, labels=False))
sage: sum(1 for _ in IP.isomorphic_substructures_iterator(IC))
120
```

```
>>> from sage.all import *
>>> P = graphs.PetersenGraph()
>>> C = graphs.CycleGraph(Integer(5))
>>> IP = IncidenceStructure(P.edges(sort=True, labels=False))
>>> IC = IncidenceStructure(C.edges(sort=True, labels=False))
>>> sum(Integer(1) for _ in IP.isomorphic_substructures_iterator(IC))
120
```

As the automorphism group of C_5 has size 10, the number of distinct unlabelled copies is 12. Let us check that all functions returned correspond to an actual C_5 subgraph:

```
sage: for f in IP.isomorphic_substructures_iterator(IC):
....:     assert all(P.has_edge(f[x],f[y]) for x,y in C.edges(sort=True,␣
↪labels=False))
```

```
>>> from sage.all import *
>>> for f in IP.isomorphic_substructures_iterator(IC):
...     assert all(P.has_edge(f[x],f[y]) for x,y in C.edges(sort=True,␣
↪labels=False))
```

The number of induced copies, in this case, is the same:

```
sage: sum(1 for _ in IP.isomorphic_substructures_iterator(IC,induced=True))
120
```

```
>>> from sage.all import *
>>> sum(Integer(1) for _ in IP.isomorphic_substructures_iterator(IC,
↪induced=True))
120
```

They begin to differ if we make one vertex universal:

```
sage: P.add_edges([(0,x) for x in P], loops=False)
sage: IP = IncidenceStructure(P.edges(sort=True, labels=False))
sage: IC = IncidenceStructure(C.edges(sort=True, labels=False))
sage: sum(1 for _ in IP.isomorphic_substructures_iterator(IC))
420
sage: sum(1 for _ in IP.isomorphic_substructures_iterator(IC,induced=True))
60
```

```
>>> from sage.all import *
>>> P.add_edges([(Integer(0),x) for x in P], loops=False)
>>> IP = IncidenceStructure(P.edges(sort=True, labels=False))
>>> IC = IncidenceStructure(C.edges(sort=True, labels=False))
>>> sum(Integer(1) for _ in IP.isomorphic_substructures_iterator(IC))
420
>>> sum(Integer(1) for _ in IP.isomorphic_substructures_iterator(IC,
↪induced=True))
60
```

The number of copies of H in itself is the size of its automorphism group:

```
sage: H = designs.projective_plane(3)                                          #␣
↪needs sage.schemes
sage: sum(1 for _ in H.isomorphic_substructures_iterator(H))                   #␣
↪needs sage.schemes
5616
sage: H.automorphism_group().cardinality()                                     #␣
↪needs sage.groups sage.schemes
5616
```

```
>>> from sage.all import *
>>> H = designs.projective_plane(Integer(3))                                   ␣
↪     # needs sage.schemes
>>> sum(Integer(1) for _ in H.isomorphic_substructures_iterator(H))            ␣
↪     # needs sage.schemes
5616
>>> H.automorphism_group().cardinality()                                      #␣
↪needs sage.groups sage.schemes
5616
```

n_blocks()

Return the number of blocks.

EXAMPLES:

```
sage: designs.DesarguesianProjectivePlaneDesign(2).n_blocks()
7
sage: B = IncidenceStructure(4, [[0,1],[0,2],[0,3],[1,2], [1,2,3]])
sage: B.n_blocks()
5
```

```
>>> from sage.all import *
>>> designs.DesarguesianProjectivePlaneDesign(Integer(2)).n_blocks()
7
>>> B = IncidenceStructure(Integer(4), [[Integer(0),Integer(1)],[Integer(0),
↪Integer(2)],[Integer(0),Integer(3)],[Integer(1),Integer(2)], [Integer(1),
↪Integer(2),Integer(3)]])
>>> B.n_blocks()
5
```

n_points()

Return the size of the ground set.

EXAMPLES:

```
sage: designs.DesarguesianProjectivePlaneDesign(2).n_points()
7
sage: B = IncidenceStructure(4, [[0,1],[0,2],[0,3],[1,2], [1,2,3]])
sage: B.n_points()
4
```

```
>>> from sage.all import *
>>> designs.DesarguesianProjectivePlaneDesign(Integer(2)).n_points()
7
```

(continues on next page)

(continued from previous page)

```
>>> B = IncidenceStructure(Integer(4), [[Integer(0),Integer(1)],[Integer(0),
↪Integer(2)],[Integer(0),Integer(3)],[Integer(1),Integer(2)], [Integer(1),
↪Integer(2),Integer(3)]])
>>> B.n_points()
4
```

num_blocks()

alias of *n_blocks()*.

num_points()

alias of *n_points()*.

packing(*solver*, *verbose=None*, *integrality_tolerance=0*)

Return a maximum packing.

A maximum packing in a hypergraph is collection of disjoint sets/blocks of maximal cardinality. This problem is NP-complete in general, and in particular on 3-uniform hypergraphs. It is solved here with an Integer Linear Program.

For more information, see the Wikipedia article Packing_in_a_hypergraph.

INPUT:

- `solver` – (default: `None`) specify a Mixed Integer Linear Programming (MILP) solver to be used. If set to `None`, the default one is used. For more information on LP solvers and which default solver is used, see the method `solve` of the class `MixedIntegerLinearProgram`.
- `verbose` – integer (default: 0); sets the level of verbosity. Set to 0 by default, which means quiet.
- `integrality_tolerance` – parameter for use with MILP solvers over an inexact base ring; see `MixedIntegerLinearProgram.get_values()`.

EXAMPLES:

```
sage: P = IncidenceStructure([[1,2],[3,4],[2,3]]).packing()                    #␣
↪needs sage.numerical.mip
sage: sorted(sorted(b) for b in P)                                             #␣
↪needs sage.numerical.mip
[[1, 2], [3, 4]]
sage: len(designs.steiner_triple_system(9).packing())                          #␣
↪needs sage.numerical.mip
3
```

```
>>> from sage.all import *
>>> P = IncidenceStructure([[Integer(1),Integer(2)],[Integer(3),Integer(4)],
↪[Integer(2),Integer(3)]]).packing()                  # needs sage.numerical.
↪mip
>>> sorted(sorted(b) for b in P)                                               #␣
↪needs sage.numerical.mip
[[1, 2], [3, 4]]
>>> len(designs.steiner_triple_system(Integer(9)).packing())                    ␣
↪     # needs sage.numerical.mip
3
```

rank()

Return the rank of the hypergraph (the maximum size of a block).

EXAMPLES:

```
sage: h = Hypergraph(8, [[0,1,3],[1,4,5,6],[1,2]])
sage: h.rank()
4
```

```
>>> from sage.all import *
>>> h = Hypergraph(Integer(8), [[Integer(0),Integer(1),Integer(3)],
↪[Integer(1),Integer(4),Integer(5),Integer(6)],[Integer(1),Integer(2)]])
>>> h.rank()
4
```

relabel(*perm=None*, *inplace=True*)

Relabel the ground set.

INPUT:

- `perm` – can be one of
  - a dictionary – then each point `p` (which should be a key of `perm`) is relabeled to `perm[p]`
  - an iterable of length `n` – the first point returned by *ground_set()* is relabeled to `perm[0]`, the second to `perm[1]`, ...
  - a callable – then each point `p` is relabeled to `perm(p)`
  - `None` – the incidence structure is relabeled to be on $\{0, 1, ..., n-1\}$ in the ordering given by *ground_set()*
- `inplace` – boolean (default: `False`); if `True` then return a relabeled graph and does not touch `self`

EXAMPLES:

```
sage: # needs sage.schemes
sage: TD = designs.transversal_design(5,5)
sage: TD.relabel({i: chr(97+i) for i in range(25)})
sage: TD.ground_set()
['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j', 'k', 'l', 'm',
 'n', 'o', 'p', 'q', 'r', 's', 't', 'u', 'v', 'w', 'x', 'y']
sage: TD.blocks()[:3]
[['a', 'f', 'k', 'p', 'u'], ['a', 'g', 'm', 's', 'y'], ['a', 'h', 'o', 'q', 'x
↪']]
```

```
>>> from sage.all import *
>>> # needs sage.schemes
>>> TD = designs.transversal_design(Integer(5),Integer(5))
>>> TD.relabel({i: chr(Integer(97)+i) for i in range(Integer(25))})
>>> TD.ground_set()
['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j', 'k', 'l', 'm',
 'n', 'o', 'p', 'q', 'r', 's', 't', 'u', 'v', 'w', 'x', 'y']
>>> TD.blocks()[:Integer(3)]
[['a', 'f', 'k', 'p', 'u'], ['a', 'g', 'm', 's', 'y'], ['a', 'h', 'o', 'q', 'x
↪']]
```

Relabel to integer points:

```
sage: TD.relabel()                                                          #␣
↪needs sage.schemes
sage: TD.blocks()[:3]                                                       #␣
↪needs sage.schemes
[[0, 5, 10, 15, 20], [0, 6, 12, 18, 24], [0, 7, 14, 16, 23]]
```

```
>>> from sage.all import *
>>> TD.relabel()                                                            #␣
↪needs sage.schemes
>>> TD.blocks()[:Integer(3)]                                                ␣
↪     # needs sage.schemes
[[0, 5, 10, 15, 20], [0, 6, 12, 18, 24], [0, 7, 14, 16, 23]]
```

trace(*points*, *min_size=1*, *multiset=True*)

Return the trace of a set of points.

Given an hypergraph $\mathcal{H}$, the *trace* of a set X of points in $\mathcal{H}$ is the hypergraph whose blocks are all non-empty $S \cap X$ where $S \in \mathcal{H}$.

INPUT:

- `points` – set of points
- `min_size` – integer (default: 1); minimum size of the sets to keep. By default all empty sets are discarded, i.e. `min_size=1`
- `multiset` – boolean (default: `True`); whether to keep multiple copies of the same set

Note

This method goes over all sets of `self` before building a new *IncidenceStructure* (which involves some relabelling and sorting). It probably should not be called in a performance-critical code.

EXAMPLES:

A Baer subplane of order 2 (i.e. a Fano plane) in a projective plane of order 4:

```
sage: # needs sage.schemes
sage: P4 = designs.projective_plane(4)
sage: F = designs.projective_plane(2)
sage: for x in Subsets(P4.ground_set(),7):
....:     if P4.trace(x,min_size=2).is_isomorphic(F):
....:         break
sage: subplane = P4.trace(x,min_size=2); subplane
Incidence structure with 7 points and 7 blocks
sage: subplane.is_isomorphic(F)
True
```

```
>>> from sage.all import *
>>> # needs sage.schemes
>>> P4 = designs.projective_plane(Integer(4))
>>> F = designs.projective_plane(Integer(2))
>>> for x in Subsets(P4.ground_set(),Integer(7)):
...     if P4.trace(x,min_size=Integer(2)).is_isomorphic(F):
```

(continues on next page)

(continued from previous page)

```
...         break
>>> subplane = P4.trace(x,min_size=Integer(2)); subplane
Incidence structure with 7 points and 7 blocks
>>> subplane.is_isomorphic(F)
True
```

CHAPTER

FIVE

LIBRARIES OF ALGORITHMS

5.1 Graph coloring

This module gathers all methods related to graph coloring. Here is what it can do :

Proper vertex coloring

all_graph_colorings()	Compute all n-colorings a graph
first_coloring()	Return the first vertex coloring found
number_of_n_colorings()	Compute the number of n-colorings of a graph
numbers_of_colorings()	Compute the number of colorings of a graph
chromatic_number()	Return the chromatic number of the graph
vertex_coloring()	Compute vertex colorings and chromatic numbers

Fractional relaxations

fractional_chromatic_number()	Return the fractional chromatic number of the graph
fractional_chromatic_index()	Return the fractional chromatic index of the graph

Other colorings

grundy_coloring()	Compute Grundy numbers and Grundy colorings
b_coloring()	Compute b-chromatic numbers and b-colorings
edge_coloring()	Compute chromatic index and edge colorings
round_robin()	Compute a round-robin coloring of the complete graph on n vertices
linear_arboricity()	Compute the linear arboricity of the given graph
acyclic_edge_coloring()	Compute an acyclic edge coloring of the current graph

AUTHORS:

- Tom Boothby (2008-02-21): Initial version
- Carlo Hamalainen (2009-03-28): minor change: switch to C++ DLX solver
- Nathann Cohen (2009-10-24): Coloring methods using linear programming

5.1.1 Methods

class sage.graphs.graph_coloring.**Test**

Bases: object

This class performs randomized testing for *all_graph_colorings()*.

Since everything else in this file is derived from *all_graph_colorings()*, this is a pretty good randomized tester for the entire file. Note that for a graph G, `G.chromatic_polynomial()` uses an entirely different algorithm, so we provide a good, independent test.

random(*tests=1000*)

Call `self.random_all_graph_colorings()`.

In the future, if other methods are added, it should call them, too.

random_all_graph_colorings(*tests=2*)

Verify the results of `all_graph_colorings()` in three ways:

1. all colorings are unique
2. number of m-colorings is $P(m)$ (where P is the chromatic polynomial of the graph being tested)
3. colorings are valid – that is, that no two vertices of the same color share an edge.

sage.graphs.graph_coloring.**acyclic_edge_coloring**(*g*, *hex_colors=False*, *value_only=False*, *k=0*, *solver=None*, *verbose=0*, *integrality_tolerance=0.001*)

Compute an acyclic edge coloring of the current graph.

An edge coloring of a graph is a assignment of colors to the edges of a graph such that :

- the coloring is proper (no adjacent edges share a color)
- For any two colors i, j, the union of the edges colored with i or j is a forest.

The least number of colors such that such a coloring exists for a graph G is written $\chi'_a(G)$, also called the acyclic chromatic index of G.

It is conjectured that this parameter cannot be too different from the obvious lower bound $\Delta(G) \leq \chi'_a(G)$, $\Delta(G)$ being the maximum degree of G, which is given by the first of the two constraints. Indeed, it is conjectured that $\Delta(G) \leq \chi'_a(G) \leq \Delta(G) + 2$.

INPUT:

- `hex_colors` – boolean (default: `False`):
  - If `hex_colors = True`, the function returns a dictionary associating to each color a list of edges (meant as an argument to the `edge_colors` keyword of the `plot` method).
  - If `hex_colors = False` (default value), returns a list of graphs corresponding to each color class.
- `value_only` – boolean (default: `False`):
  - If `value_only = True`, only returns the acyclic chromatic index as an integer value
  - If `value_only = False`, returns the color classes according to the value of `hex_colors`
- `k` – integer; the number of colors to use
  - If `k > 0`, computes an acyclic edge coloring using k colors.
  - If `k = 0` (default), computes a coloring of G into $\Delta(G) + 2$ colors, which is the conjectured general bound.
  - If `k = None`, computes a decomposition using the least possible number of colors.

- `solver` – string (default: `None`); specifies a Mixed Integer Linear Programming (MILP) solver to be used. If set to `None`, the default one is used. For more information on MILP solvers and which default solver is used, see the method `solve` of the class `MixedIntegerLinearProgram`.
- `verbose` – integer (default: 0); sets the level of verbosity. Set to 0 by default, which means quiet.
- `integrality_tolerance` – float; parameter for use with MILP solvers over an inexact base ring; see `MixedIntegerLinearProgram.get_values()`.

ALGORITHM:

Linear Programming

EXAMPLES:

The complete graph on 8 vertices cannot be acyclically edge-colored with less $\Delta + 1$ colors, but it can be colored with $\Delta + 2 = 9$:

```
sage: from sage.graphs.graph_coloring import acyclic_edge_coloring
sage: g = graphs.CompleteGraph(8)
sage: colors = acyclic_edge_coloring(g)                                    #␣
↪needs sage.numerical.mip
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_coloring import acyclic_edge_coloring
>>> g = graphs.CompleteGraph(Integer(8))
>>> colors = acyclic_edge_coloring(g)                                      #␣
↪needs sage.numerical.mip
```

Each color class is of course a matching

```
sage: all(max(gg.degree()) <= 1 for gg in colors)                          #␣
↪needs sage.numerical.mip
True
```

```
>>> from sage.all import *
>>> all(max(gg.degree()) <= Integer(1) for gg in colors)                    ␣
↪     # needs sage.numerical.mip
True
```

These matchings being a partition of the edge set:

```
sage: all(any(gg.has_edge(e) for gg in colors)                             #␣
↪needs sage.numerical.mip
....:     for e in g.edge_iterator(labels=False))
True
```

```
>>> from sage.all import *
>>> all(any(gg.has_edge(e) for gg in colors)                               #␣
↪needs sage.numerical.mip
...     for e in g.edge_iterator(labels=False))
True
```

Besides, the union of any two of them is a forest

```
sage: all(g1.union(g2).is_forest() for g1 in colors for g2 in colors)           #␣
↪needs sage.numerical.mip
True
```

```
>>> from sage.all import *
>>> all(g1.union(g2).is_forest() for g1 in colors for g2 in colors)           #␣
↪needs sage.numerical.mip
True
```

If one wants to acyclically color a cycle on 4 vertices, at least 3 colors will be necessary. The function raises an exception when asked to color it with only 2:

```
sage: g = graphs.CycleGraph(4)
sage: acyclic_edge_coloring(g, k=2)                                             #␣
↪needs sage.numerical.mip
Traceback (most recent call last):
...
ValueError: this graph cannot be colored with the given number of colors
```

```
>>> from sage.all import *
>>> g = graphs.CycleGraph(Integer(4))
>>> acyclic_edge_coloring(g, k=Integer(2))                                       ␣
↪     # needs sage.numerical.mip
Traceback (most recent call last):
...
ValueError: this graph cannot be colored with the given number of colors
```

The optimal coloring give us 3 classes:

```
sage: colors = acyclic_edge_coloring(g, k=None)                                 #␣
↪needs sage.numerical.mip
sage: len(colors)                                                               #␣
↪needs sage.numerical.mip
3
```

```
>>> from sage.all import *
>>> colors = acyclic_edge_coloring(g, k=None)                                 #␣
↪needs sage.numerical.mip
>>> len(colors)                                                               #␣
↪needs sage.numerical.mip
3
```

sage.graphs.graph_coloring.**all_graph_colorings**(*G*, *n*, *count_only=False*, *hex_colors=False*, *vertex_color_dict=False*, *color_classes=False*)

Compute all n-colorings of a graph.

This method casts the graph coloring problem into an exact cover problem, and passes this into an implementation of the Dancing Links algorithm described by Knuth (who attributes the idea to Hitotumatu and Noshita).

INPUT:

- `G` – a graph
- `n` – positive integer; the number of colors

- `count_only` – boolean (default: `False`); when set to `True`, it returns 1 for each coloring and ignores other parameters
- `hex_colors` – boolean (default: `False`); when set to `False`, colors are labeled [0, 1, ..., $n-1$], otherwise the RGB Hex labeling is used
- `vertex_color_dict` – boolean (default: `False`); when set to `True`, it returns a dictionary `{vertex: color}`, otherwise it returns a dictionary `{color: [list of vertices]}`
- `color_classes` – boolean (default: `False`); when set to `True`, the method returns only a list of the color classes and ignores parameters `hex_colors` and `vertex_color_dict`

Warning

This method considers only colorings using exactly n colors, even if a coloring using fewer colors can be found.

The construction works as follows. Columns:

- The first $|V|$ columns correspond to a vertex – a 1 in this column indicates that this vertex has a color.
- After those $|V|$ columns, we add $n * |E|$ columns – a 1 in these columns indicate that a particular edge is incident to a vertex with a certain color.

Rows:

- For each vertex, add n rows; one for each color c. Place a 1 in the column corresponding to the vertex, and a 1 in the appropriate column for each edge incident to the vertex, indicating that that edge is incident to the color c.
- If $n > 2$, the above construction cannot be exactly covered since each edge will be incident to only two vertices (and hence two colors) - so we add $n * |E|$ rows, each one containing a 1 for each of the $n * |E|$ columns. These get added to the cover solutions "for free" during the backtracking.

Note that this construction results in $n * |V| + 2 * n * |E| + n * |E|$ entries in the matrix. The Dancing Links algorithm uses a sparse representation, so if the graph is simple, $|E| \leq |V|^2$ and $n <= |V|$, this construction runs in $O(|V|^3)$ time. Back-conversion to a coloring solution is a simple scan of the solutions, which will contain $|V| + (n-2) * |E|$ entries, so runs in $O(|V|^3)$ time also. For most graphs, the conversion will be much faster – for example, a planar graph will be transformed for 4-coloring in linear time since $|E| = O(|V|)$.

REFERENCES:

http://www-cs-staff.stanford.edu/~uno/papers/dancing-color.ps.gz

EXAMPLES:

```
sage: from sage.graphs.graph_coloring import all_graph_colorings
sage: G = Graph({0: [1, 2, 3], 1: [2]})
sage: n = 0
sage: for C in all_graph_colorings(G, 3, hex_colors=True):                    #␣
↪needs sage.plot
....:     parts = [C[k] for k in C]
....:     for P in parts:
....:         l = len(P)
....:         for i in range(l):
....:             for j in range(i + 1, l):
....:                 if G.has_edge(P[i], P[j]):
....:                     raise RuntimeError("Coloring Failed.")
....:     n += 1
```

(continues on next page)

(continued from previous page)

```
sage: print("G has %s 3-colorings." % n)                                    #␣
↪needs sage.plot
G has 12 3-colorings.
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_coloring import all_graph_colorings
>>> G = Graph({Integer(0): [Integer(1), Integer(2), Integer(3)], Integer(1):␣
↪[Integer(2)]})
>>> n = Integer(0)
>>> for C in all_graph_colorings(G, Integer(3), hex_colors=True):                ␣
↪     # needs sage.plot
...     parts = [C[k] for k in C]
...     for P in parts:
...         l = len(P)
...         for i in range(l):
...             for j in range(i + Integer(1), l):
...                 if G.has_edge(P[i], P[j]):
...                     raise RuntimeError("Coloring Failed.")
...     n += Integer(1)
>>> print("G has %s 3-colorings." % n)                                       #␣
↪needs sage.plot
G has 12 3-colorings.
```

sage.graphs.graph_coloring.**b_coloring**(*g*, *k*, *value_only=True*, *solver=None*, *verbose=0*, *integrality_tolerance=0.001*)

Compute b-chromatic numbers and b-colorings.

This function computes a b-coloring with at most k colors that maximizes the number of colors, if such a coloring exists.

Definition :

Given a proper coloring of a graph G and a color class C such that none of its vertices have neighbors in all the other color classes, one can eliminate color class C assigning to each of its elements a missing color in its neighborhood.

Let a b-vertex be a vertex with neighbors in all other colorings. Then, one can repeat the above procedure until a coloring is obtained where every color class contains a b-vertex, in which case none of the color classes can be eliminated with the same idea. So, one can define a b-coloring as a proper coloring where each color class has a b-vertex.

In the worst case, after successive applications of the above procedure, one get a proper coloring that uses a number of colors equal to the b-chromatic number of G (denoted $\chi_b(G)$): the maximum k such that G admits a b-coloring with k colors.

A useful upper bound for calculating the b-chromatic number is the following. If G admits a b-coloring with k colors, then there are k vertices of degree at least $k-1$ (the b-vertices of each color class). So, if we set $m(G) = \max\{k | \text{there are } k \text{ vertices of degree at least } k-1\}$, we have that $\chi_b(G) \leq m(G)$.

Note

This method computes a b-coloring that uses at *MOST* k colors. If this method returns a value equal to k, it cannot be assumed that k is equal to $\chi_b(G)$. Meanwhile, if it returns any value $k' < k$, this is a certificate that the Grundy number of the given graph is k'.

As $\chi_b(G) \leq m(G)$, it can be assumed that $\chi_b(G) = k$ if `b_coloring(g, k)` returns k when $k = m(G)$.

INPUT:

- `k` – integer; maximum number of colors
- `value_only` – boolean (default: `True`); when set to `True`, only the number of colors is returned. Otherwise, the pair `(nb_colors, coloring)` is returned, where `coloring` is a dictionary associating its color (integer) to each vertex of the graph.
- `solver` – string (default: `None`); specifies a Mixed Integer Linear Programming (MILP) solver to be used. If set to `None`, the default one is used. For more information on MILP solvers and which default solver is used, see the method `solve` of the class `MixedIntegerLinearProgram`.
- `verbose` – integer (default: 0); sets the level of verbosity. Set to 0 by default, which means quiet.
- `integrality_tolerance` – float; parameter for use with MILP solvers over an inexact base ring; see `MixedIntegerLinearProgram.get_values()`.

ALGORITHM:

Integer Linear Program.

EXAMPLES:

The b-chromatic number of a P_5 is equal to 3:

```
sage: from sage.graphs.graph_coloring import b_coloring
sage: g = graphs.PathGraph(5)
sage: b_coloring(g, 5)                                                          #␣
↪needs sage.numerical.mip
3
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_coloring import b_coloring
>>> g = graphs.PathGraph(Integer(5))
>>> b_coloring(g, Integer(5))                                                   ␣
↪     # needs sage.numerical.mip
3
```

The b-chromatic number of the Petersen Graph is equal to 3:

```
sage: g = graphs.PetersenGraph()
sage: b_coloring(g, 5)                                                          #␣
↪needs sage.numerical.mip
3
```

```
>>> from sage.all import *
>>> g = graphs.PetersenGraph()
>>> b_coloring(g, Integer(5))                                                   ␣
↪     # needs sage.numerical.mip
3
```

It would have been sufficient to set the value of `k` to 4 in this case, as $4 = m(G)$.

`sage.graphs.graph_coloring.`**`chromatic_number`**`(`G`)`

Return the chromatic number of the graph.

The chromatic number is the minimal number of colors needed to color the vertices of the graph G.

EXAMPLES:

```
sage: from sage.graphs.graph_coloring import chromatic_number
sage: G = Graph({0: [1, 2, 3], 1: [2]})
sage: chromatic_number(G)
3

sage: G = graphs.PetersenGraph()
sage: G.chromatic_number()
3
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_coloring import chromatic_number
>>> G = Graph({Integer(0): [Integer(1), Integer(2), Integer(3)], Integer(1):␣
↪[Integer(2)]})
>>> chromatic_number(G)
3

>>> G = graphs.PetersenGraph()
>>> G.chromatic_number()
3
```

sage.graphs.graph_coloring.**edge_coloring**(*g*, *value_only=False*, *vizing=False*, *hex_colors=False*, *solver=None*, *verbose=0*, *integrality_tolerance=0.001*)

Compute chromatic index and edge colorings.

INPUT:

- `g` – a graph
- `value_only` – boolean (default: `False`):
  - When set to `True`, only the chromatic index is returned
  - When set to `False`, a partition of the edge set into matchings is returned if possible
- `vizing` – boolean (default: `False`):
  - When set to `True`, finds an edge coloring using the algorithm described at [MG1992]. This always results in a coloring with $\Delta + 1$ colors, where Δ is equal to the maximum degree in the graph, even if one of the colors is empty, for the sake of consistency.
  - When set to `False`, tries to find a Δ-edge-coloring using Mixed Integer Linear Programming (MILP). If impossible, returns a $(\Delta + 1)$-edge-coloring. Please note that determining if the chromatic index of a graph equals Δ is computationally difficult, and could take a long time.
- `hex_colors` – boolean (default: `False`); when set to `True`, the partition returned is a dictionary whose keys are colors and whose values are the color classes (ideal for plotting)
- `solver` – string (default: `None`); specifies a Mixed Integer Linear Programming (MILP) solver to be used. If set to `None`, the default one is used. For more information on MILP solvers and which default solver is used, see the method `solve` of the class `MixedIntegerLinearProgram`.
- `verbose` – integer (default: 0); sets the level of verbosity. Set to 0 by default, which means quiet.
- `integrality_tolerance` – float; parameter for use with MILP solvers over an inexact base ring; see `MixedIntegerLinearProgram.get_values()`.

OUTPUT:

In the following, Δ is equal to the maximum degree in the graph `g`.

- If `vizing=True` and `value_only=False`, return a partition of the edge set into $\Delta + 1$ matchings.
- If `vizing=False` and `value_only=True`, return the chromatic index.
- If `vizing=False` and `value_only=False`, return a partition of the edge set into the minimum number of matchings.
- If `vizing=True` and `value_only=True`, should return something, but mainly you are just trying to compute the maximum degree of the graph, and this is not the easiest way. By Vizing's theorem, a graph has a chromatic index equal to Δ or to $\Delta + 1$.

Note

In a few cases, it is possible to find very quickly the chromatic index of a graph, while it remains a tedious job to compute a corresponding coloring. For this reason, `value_only = True` can sometimes be much faster, and it is a bad idea to compute the whole coloring if you do not need it !

See also

- Wikipedia article Edge_coloring for further details on edge coloring
- *chromatic_index()*
- *fractional_chromatic_index()*
- *chromatic_number()*
- *sage.graphs.graph_coloring.vertex_coloring()*

EXAMPLES:

The Petersen graph has chromatic index 4:

```
sage: from sage.graphs.graph_coloring import edge_coloring
sage: g = graphs.PetersenGraph()
sage: edge_coloring(g, value_only=True, solver='GLPK')
4
sage: color_classes = edge_coloring(g, value_only=False, solver='GLPK')
sage: len(color_classes)
4
sage: len(set(frozenset(e) for C in color_classes for e in C)) == g.size()
True
sage: all(g.has_edge(e) for C in color_classes for e in C)
True
sage: all(len(Graph(C).matching()) == len(C) for C in color_classes)            #␣
↪needs networkx
True
sage: color_classes = edge_coloring(g, value_only=False,
....:                               hex_colors=True, solver='GLPK')
sage: sorted(color_classes.keys())
['#00ffff', '#7f00ff', '#7fff00', '#ff0000']
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_coloring import edge_coloring
```

(continues on next page)

(continued from previous page)

```
>>> g = graphs.PetersenGraph()
>>> edge_coloring(g, value_only=True, solver='GLPK')
4
>>> color_classes = edge_coloring(g, value_only=False, solver='GLPK')
>>> len(color_classes)
4
>>> len(set(frozenset(e) for C in color_classes for e in C)) == g.size()
True
>>> all(g.has_edge(e) for C in color_classes for e in C)
True
>>> all(len(Graph(C).matching()) == len(C) for C in color_classes)            #␣
↪needs networkx
True
>>> color_classes = edge_coloring(g, value_only=False,
...                               hex_colors=True, solver='GLPK')
>>> sorted(color_classes.keys())
['#00ffff', '#7f00ff', '#7fff00', '#ff0000']
```

Complete graphs are colored using the linear-time round-robin coloring:

```
sage: from sage.graphs.graph_coloring import edge_coloring
sage: len(edge_coloring(graphs.CompleteGraph(20)))                            #␣
↪needs sage.numerical.mip
19
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_coloring import edge_coloring
>>> len(edge_coloring(graphs.CompleteGraph(Integer(20))))                      ␣
↪     # needs sage.numerical.mip
19
```

The chromatic index of a non connected graph is the maximum over its connected components:

```
sage: g = graphs.CompleteGraph(4) + graphs.CompleteGraph(10)
sage: edge_coloring(g, value_only=True)                                       #␣
↪needs sage.numerical.mip
9
```

```
>>> from sage.all import *
>>> g = graphs.CompleteGraph(Integer(4)) + graphs.CompleteGraph(Integer(10))
>>> edge_coloring(g, value_only=True)                                         #␣
↪needs sage.numerical.mip
9
```

sage.graphs.graph_coloring.**first_coloring**(*G*, *n=0*, *hex_colors=False*)

Return the first vertex coloring found.

If a natural number n is provided, returns the first found coloring with at least n colors. That is, n is a lower bound on the number of colors to use.

INPUT:

- `n` – integer (default: 0); the minimal number of colors to try

- `hex_colors` – boolean (default: `False`); when set to `True`, the partition returned is a dictionary whose keys are colors and whose values are the color classes (ideal for plotting)

EXAMPLES:

```
sage: from sage.graphs.graph_coloring import first_coloring
sage: G = Graph({0: [1, 2, 3], 1: [2]})
sage: sorted(first_coloring(G, 3))
[[0], [1, 3], [2]]
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_coloring import first_coloring
>>> G = Graph({Integer(0): [Integer(1), Integer(2), Integer(3)], Integer(1):␣
↪[Integer(2)]})
>>> sorted(first_coloring(G, Integer(3)))
[[0], [1, 3], [2]]
```

sage.graphs.graph_coloring.**format_coloring**(*data*, *value_only=False*, *hex_colors=False*, *vertex_color_dict=False*)

Helper method for vertex and edge coloring methods.

INPUT:

- `data` – either a number when `value_only` is `True` or a list of color classes
- `value_only` – boolean (default: `False`); when set to `True`, it simply returns `data`
- `hex_colors` – boolean (default: `False`); when set to `False`, colors are labeled $[0, 1, \ldots, n-1]$, otherwise the RGB Hex labeling is used
- `vertex_color_dict` – boolean (default: `False`); when set to `True`, it returns a dictionary `{vertex: color}`, otherwise it returns a dictionary `{color: [list of vertices]}`

EXAMPLES:

```
sage: from sage.graphs.graph_coloring import format_coloring
sage: color_classes = [['a', 'b'], ['c'], ['d']]
sage: format_coloring(color_classes, value_only=True)
[['a', 'b'], ['c'], ['d']]
sage: format_coloring(len(color_classes), value_only=True)
3
sage: format_coloring(color_classes, value_only=False)
{0: ['a', 'b'], 1: ['c'], 2: ['d']}
sage: format_coloring(color_classes, value_only=False, hex_colors=True)          #␣
↪needs sage.plot
{'#0000ff': ['d'], '#00ff00': ['c'], '#ff0000': ['a', 'b']}
sage: format_coloring(color_classes, value_only=False, hex_colors=False,
....:                 vertex_color_dict=True)
{'a': 0, 'b': 0, 'c': 1, 'd': 2}
sage: format_coloring(color_classes, value_only=False, hex_colors=True,          #␣
↪needs sage.plot
....:                 vertex_color_dict=True)
{'a': '#ff0000', 'b': '#ff0000', 'c': '#00ff00', 'd': '#0000ff'}
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_coloring import format_coloring
```

(continues on next page)

(continued from previous page)

```
>>> color_classes = [['a', 'b'], ['c'], ['d']]
>>> format_coloring(color_classes, value_only=True)
[['a', 'b'], ['c'], ['d']]
>>> format_coloring(len(color_classes), value_only=True)
3
>>> format_coloring(color_classes, value_only=False)
{0: ['a', 'b'], 1: ['c'], 2: ['d']}
>>> format_coloring(color_classes, value_only=False, hex_colors=True)          #␣
↪needs sage.plot
{'#0000ff': ['d'], '#00ff00': ['c'], '#ff0000': ['a', 'b']}
>>> format_coloring(color_classes, value_only=False, hex_colors=False,
...                 vertex_color_dict=True)
{'a': 0, 'b': 0, 'c': 1, 'd': 2}
>>> format_coloring(color_classes, value_only=False, hex_colors=True,          #␣
↪needs sage.plot
...                 vertex_color_dict=True)
{'a': '#ff0000', 'b': '#ff0000', 'c': '#00ff00', 'd': '#0000ff'}
```

sage.graphs.graph_coloring.**fractional_chromatic_index**(*G*, *solver='PPL'*, *verbose_constraints=False*, *verbose=0*)

Return the fractional chromatic index of the graph.

The fractional chromatic index is a relaxed version of edge-coloring. An edge coloring of a graph being actually a covering of its edges into the smallest possible number of matchings, the fractional chromatic index of a graph G is the smallest real value $\chi_f(G)$ such that there exists a list of matchings $M_1, \ldots, M_k$ of G and coefficients $\alpha_1, \ldots, \alpha_k$ with the property that each edge is covered by the matchings in the following relaxed way

$$\forall e \in E(G), \sum_{e \in M_i} \alpha_i \geq 1.$$

For more information, see the Wikipedia article Fractional_coloring.

ALGORITHM:

The fractional chromatic index is computed through Linear Programming through its dual. The LP solved by sage is actually:

$$\text{Maximize : } \sum_{e \in E(G)} r_e$$

$$\text{Such that : }$$

$$\forall M \text{ matching } \subseteq G, \sum_{e \in M} r_v \leq 1$$

INPUT:

- `G` – a graph
- `solver` – (default: `'PPL'`) specify a Linear Program (LP) solver to be used. If set to `None`, the default one is used. For more information on LP solvers and which default solver is used, see the method `solve` of the class `MixedIntegerLinearProgram`.

Note

The default solver used here is `'PPL'` which provides exact results, i.e. a rational number, although this may be slower that using other solvers. Be aware that this method may loop endlessly when using some non exact solvers as reported in Issue #23658 and Issue #23798.

- `verbose_constraints` – boolean (default: `False`); whether to display which constraints are being generated
- `verbose` – integer (default: 0); sets the level of verbosity of the LP solver

EXAMPLES:

The fractional chromatic index of a C_5 is $5/2$:

```
sage: g = graphs.CycleGraph(5)
sage: g.fractional_chromatic_index()                                        #␣
↪needs sage.numerical.mip
5/2
```

```
>>> from sage.all import *
>>> g = graphs.CycleGraph(Integer(5))
>>> g.fractional_chromatic_index()                                          #␣
↪needs sage.numerical.mip
5/2
```

sage.graphs.graph_coloring.**fractional_chromatic_number**(*G*, *solver='PPL'*, *verbose=0*, *check_components=True*, *check_bipartite=True*)

Return the fractional chromatic number of the graph.

Fractional coloring is a relaxed version of vertex coloring with several equivalent definitions, such as the optimum value in a linear relaxation of the integer program that gives the usual chromatic number. It is also equal to the fractional clique number by LP-duality.

ALGORITHM:

The fractional chromatic number is computed via the usual Linear Program. The LP solved by sage is essentially,

$$\text{Minimize : } \sum_{I \in \mathcal{I}(G)} x_I$$

$$\text{Such that : }$$

$$\forall v \in V(G), \sum_{I \in \mathcal{I}(G),\, v \in I} x_v \geq 1$$

$$\forall I \in \mathcal{I}(G), x_I \geq 0$$

where $\mathcal{I}(G)$ is the set of maximal independent sets of G (see Section 2.1 of [CFKPR2010] to know why it is sufficient to consider maximal independent sets). As optional optimisations, we construct the LP on each biconnected component of G (and output the maximum value), and avoid using the LP if G is bipartite (as then the output must be 1 or 2).

Note

Computing the fractional chromatic number can be very slow. Since the variables of the LP are independent sets, in general the LP has size exponential in the order of the graph. In the current implementation a list of all maximal independent sets is created and stored, which can be both slow and memory-hungry.

INPUT:

- `G` – a graph
- `solver` – (default: `'PPL'`) specify a Linear Program (LP) solver to be used. If set to `None`, the default one is used. For more information on LP solvers and which default solver is used, see the method `solve` of the class `MixedIntegerLinearProgram`.

> **Note**
>
> The default solver used here is `'PPL'` which provides exact results, i.e. a rational number, although this may be slower that using other solvers.

- `verbose` – integer (default: 0); sets the level of verbosity of the LP solver
- `check_components` – boolean (default: `True`); whether the method is called on each biconnected component of G
- `check_bipartite` – boolean (default: `True`); whether the graph is checked for bipartiteness. If the graph is bipartite then we can avoid creating and solving the LP.

EXAMPLES:

The fractional chromatic number of a C_5 is $5/2$:

```
sage: g = graphs.CycleGraph(5)
sage: g.fractional_chromatic_number()                                          #␣
↪needs sage.numerical.mip
5/2
```

```
>>> from sage.all import *
>>> g = graphs.CycleGraph(Integer(5))
>>> g.fractional_chromatic_number()                                            #␣
↪needs sage.numerical.mip
5/2
```

sage.graphs.graph_coloring.**grundy_coloring**(*g*, *k*, *value_only=True*, *solver=None*, *verbose=0*, *integrality_tolerance=0.001*)

Compute Grundy numbers and Grundy colorings.

The method computes the worst-case of a first-fit coloring with less than k colors.

Definition:

A first-fit coloring is obtained by sequentially coloring the vertices of a graph, assigning them the smallest color not already assigned to one of its neighbors. The result is clearly a proper coloring, which usually requires much more colors than an optimal vertex coloring of the graph, and heavily depends on the ordering of the vertices.

The number of colors required by the worst-case application of this algorithm on a graph G is called the Grundy number, written $\Gamma(G)$.

Equivalent formulation:

Equivalently, a Grundy coloring is a proper vertex coloring such that any vertex colored with i has, for every $j < i$, a neighbor colored with j. This can define a Linear Program, which is used here to compute the Grundy number of a graph.

Note

This method computes a grundy coloring using at *MOST* k colors. If this method returns a value equal to k, it cannot be assumed that k is equal to $\Gamma(G)$. Meanwhile, if it returns any value $k' < k$, this is a certificate that the Grundy number of the given graph is k'.

As $\Gamma(G) \leq \Delta(G) + 1$, it can also be assumed that $\Gamma(G) = k$ if `grundy_coloring(g, k)` returns k when $k = \Delta(G) + 1$.

INPUT:

- `k` – integer; maximum number of colors
- `value_only` – boolean (default: `True`); when set to `True`, only the number of colors is returned. Otherwise, the pair `(nb_colors, coloring)` is returned, where `coloring` is a dictionary associating its color (integer) to each vertex of the graph.
- `solver` – string (default: `None`); specifies a Mixed Integer Linear Programming (MILP) solver to be used. If set to `None`, the default one is used. For more information on MILP solvers and which default solver is used, see the method `solve` of the class `MixedIntegerLinearProgram`.
- `verbose` – integer (default: 0); sets the level of verbosity. Set to 0 by default, which means quiet.
- `integrality_tolerance` – float; parameter for use with MILP solvers over an inexact base ring; see `MixedIntegerLinearProgram.get_values()`.

ALGORITHM:

Integer Linear Program.

EXAMPLES:

The Grundy number of a P_4 is equal to 3:

```
sage: from sage.graphs.graph_coloring import grundy_coloring
sage: g = graphs.PathGraph(4)
sage: grundy_coloring(g, 4)                                                 #␣
↪needs sage.numerical.mip
3
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_coloring import grundy_coloring
>>> g = graphs.PathGraph(Integer(4))
>>> grundy_coloring(g, Integer(4))                                          ␣
↪     # needs sage.numerical.mip
3
```

The Grundy number of the PetersenGraph is equal to 4:

```
sage: g = graphs.PetersenGraph()
sage: grundy_coloring(g, 5)                                                 #␣
↪needs sage.numerical.mip
4
```

```
>>> from sage.all import *
>>> g = graphs.PetersenGraph()
>>> grundy_coloring(g, Integer(5))                                          ␣
```

(continues on next page)

(continued from previous page)

```
↪      # needs sage.numerical.mip
4
```

It would have been sufficient to set the value of `k` to 4 in this case, as $4 = \Delta(G) + 1$.

sage.graphs.graph_coloring.**linear_arboricity**(*g*, *plus_one=None*, *hex_colors=False*, *value_only=False*, *solver=None*, *verbose=0*, *integrality_tolerance=0.001*)

Compute the linear arboricity of the given graph.

The linear arboricity of a graph G is the least number $la(G)$ such that the edges of G can be partitioned into linear forests (i.e. into forests of paths).

Obviously, $la(G) \geq \left\lceil \frac{\Delta(G)}{2} \right\rceil$.

It is conjectured in [Aki1980] that $la(G) \leq \left\lceil \frac{\Delta(G)+1}{2} \right\rceil$.

INPUT:

- `plus_one` – integer (default: `None`); whether to use $\left\lceil \frac{\Delta(G)}{2} \right\rceil$ or $\left\lceil \frac{\Delta(G)+1}{2} \right\rceil$ colors.
  - If `0`, computes a decomposition of G into $\left\lceil \frac{\Delta(G)}{2} \right\rceil$ forests of paths
  - If `1`, computes a decomposition of G into $\left\lceil \frac{\Delta(G)+1}{2} \right\rceil$ colors, which is the conjectured general bound.
  - If `plus_one = None` (default), computes a decomposition using the least possible number of colors.
- `hex_colors` – boolean (default: `False`):
  - If `hex_colors = True`, the function returns a dictionary associating to each color a list of edges (meant as an argument to the `edge_colors` keyword of the `plot` method).
  - If `hex_colors = False` (default value), returns a list of graphs corresponding to each color class.
- `value_only` – boolean (default: `False`):
  - If `value_only = True`, only returns the linear arboricity as an integer value.
  - If `value_only = False`, returns the color classes according to the value of `hex_colors`
- `solver` – string (default: `None`); specifies a Mixed Integer Linear Programming (MILP) solver to be used. If set to `None`, the default one is used. For more information on MILP solvers and which default solver is used, see the method `solve` of the class `MixedIntegerLinearProgram`.
- `verbose` – integer (default: 0); sets the level of verbosity. Set to 0 by default, which means quiet.
- `integrality_tolerance` – float; parameter for use with MILP solvers over an inexact base ring; see `MixedIntegerLinearProgram.get_values()`.

ALGORITHM:

Linear Programming

COMPLEXITY:

NP-Hard

EXAMPLES:

Obviously, a square grid has a linear arboricity of 2, as the set of horizontal lines and the set of vertical lines are an admissible partition:

```
sage: from sage.graphs.graph_coloring import linear_arboricity
sage: g = graphs.Grid2dGraph(4, 4)                                              #␣
↪needs sage.numerical.mip
sage: g1,g2 = linear_arboricity(g)                                              #␣
↪needs sage.numerical.mip
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_coloring import linear_arboricity
>>> g = graphs.Grid2dGraph(Integer(4), Integer(4))                              ␣
↪              # needs sage.numerical.mip
>>> g1,g2 = linear_arboricity(g)                                               #␣
↪needs sage.numerical.mip
```

Each graph is of course a forest:

```
sage: g1.is_forest() and g2.is_forest()                                         #␣
↪needs sage.numerical.mip
True
```

```
>>> from sage.all import *
>>> g1.is_forest() and g2.is_forest()                                           #␣
↪needs sage.numerical.mip
True
```

Of maximum degree 2:

```
sage: max(g1.degree()) <= 2 and max(g2.degree()) <= 2                           #␣
↪needs sage.numerical.mip
True
```

```
>>> from sage.all import *
>>> max(g1.degree()) <= Integer(2) and max(g2.degree()) <= Integer(2)           ␣
↪              # needs sage.numerical.mip
True
```

Which constitutes a partition of the whole edge set:

```
sage: all((g1.has_edge(e) or g2.has_edge(e))                                    #␣
↪needs sage.numerical.mip
....:     for e in g.edge_iterator(labels=None))
True
```

```
>>> from sage.all import *
>>> all((g1.has_edge(e) or g2.has_edge(e))                                      #␣
↪needs sage.numerical.mip
...     for e in g.edge_iterator(labels=None))
True
```

sage.graphs.graph_coloring.**number_of_n_colorings**(G, n)

Compute the number of n-colorings of a graph.

INPUT:

- G – a graph

- n – positive integer; the number of colors

EXAMPLES:

```
sage: from sage.graphs.graph_coloring import number_of_n_colorings
sage: G = Graph({0: [1, 2, 3], 1: [2]})
sage: number_of_n_colorings(G, 3)
12
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_coloring import number_of_n_colorings
>>> G = Graph({Integer(0): [Integer(1), Integer(2), Integer(3)], Integer(1):␣
↪[Integer(2)]})
>>> number_of_n_colorings(G, Integer(3))
12
```

sage.graphs.graph_coloring.**numbers_of_colorings**(G)

Compute the number of colorings of a graph.

Return the number of n-colorings of the graph G for all n from 0 to $|V|$.

EXAMPLES:

```
sage: from sage.graphs.graph_coloring import numbers_of_colorings
sage: G = Graph({0: [1, 2, 3], 1: [2]})
sage: numbers_of_colorings(G)
[0, 0, 0, 12, 24]
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_coloring import numbers_of_colorings
>>> G = Graph({Integer(0): [Integer(1), Integer(2), Integer(3)], Integer(1):␣
↪[Integer(2)]})
>>> numbers_of_colorings(G)
[0, 0, 0, 12, 24]
```

sage.graphs.graph_coloring.**round_robin**(n)

Compute a round-robin coloring of the complete graph on n vertices.

A round-robin coloring of the complete graph G on $2n$ vertices ($V = [0, \ldots, 2n-1]$) is a proper coloring of its edges such that the edges with color i are all the $(i+j, i-j)$ plus the edge $(2n-1, i)$.

If n is odd, one obtain a round-robin coloring of the complete graph through the round-robin coloring of the graph with $n+1$ vertices.

INPUT:

- n – the number of vertices in the complete graph

OUTPUT:

- A *CompleteGraph()* with labelled edges such that the label of each edge is its color.

EXAMPLES:

```
sage: from sage.graphs.graph_coloring import round_robin
sage: round_robin(3).edges(sort=True)
[(0, 1, 2), (0, 2, 1), (1, 2, 0)]
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_coloring import round_robin
>>> round_robin(Integer(3)).edges(sort=True)
[(0, 1, 2), (0, 2, 1), (1, 2, 0)]
```

```
sage: round_robin(4).edges(sort=True)
[(0, 1, 2), (0, 2, 1), (0, 3, 0), (1, 2, 0), (1, 3, 1), (2, 3, 2)]
```

```
>>> from sage.all import *
>>> round_robin(Integer(4)).edges(sort=True)
[(0, 1, 2), (0, 2, 1), (0, 3, 0), (1, 2, 0), (1, 3, 1), (2, 3, 2)]
```

For higher orders, the coloring is still proper and uses the expected number of colors:

```
sage: g = round_robin(9)
sage: sum(Set(e[2] for e in g.edges_incident(v)).cardinality() for v in g) == 2 *␣
↪g.size()
True
sage: Set(e[2] for e in g.edge_iterator()).cardinality()
9
```

```
>>> from sage.all import *
>>> g = round_robin(Integer(9))
>>> sum(Set(e[Integer(2)] for e in g.edges_incident(v)).cardinality() for v in g)␣
↪== Integer(2) * g.size()
True
>>> Set(e[Integer(2)] for e in g.edge_iterator()).cardinality()
9
```

```
sage: g = round_robin(10)
sage: sum(Set(e[2] for e in g.edges_incident(v)).cardinality() for v in g) == 2 *␣
↪g.size()
True
sage: Set(e[2] for e in g.edge_iterator()).cardinality()
9
```

```
>>> from sage.all import *
>>> g = round_robin(Integer(10))
>>> sum(Set(e[Integer(2)] for e in g.edges_incident(v)).cardinality() for v in g)␣
↪== Integer(2) * g.size()
True
>>> Set(e[Integer(2)] for e in g.edge_iterator()).cardinality()
9
```

sage.graphs.graph_coloring.**vertex_coloring**(*g*, *k=None*, *value_only=False*, *hex_colors=False*, *solver=None*, *verbose=0*, *integrality_tolerance=0.001*)

Compute Vertex colorings and chromatic numbers.

This function can compute the chromatic number of the given graph or test its k-colorability.

See the Wikipedia article Graph_coloring for further details on graph coloring.

INPUT:

- `g` – a graph
- `k` – integer (default: `None`); tests whether the graph is k-colorable. The function returns a partition of the vertex set in k independent sets if possible and `False` otherwise.
- `value_only` – boolean (default: `False`):
  - When set to `True`, only the chromatic number is returned.
  - When set to `False` (default), a partition of the vertex set into independent sets is returned if possible.
- `hex_colors` – boolean (default: `False`); when set to `True`, the partition returned is a dictionary whose keys are colors and whose values are the color classes (ideal for plotting).
- `solver` – string (default: `None`); specifies a Mixed Integer Linear Programming (MILP) solver to be used. If set to `None`, the default one is used. For more information on MILP solvers and which default solver is used, see the method `solve` of the class `MixedIntegerLinearProgram`.
- `verbose` – integer (default: 0); sets the level of verbosity. Set to 0 by default, which means quiet.
- `integrality_tolerance` – float; parameter for use with MILP solvers over an inexact base ring; see `MixedIntegerLinearProgram.get_values()`.

OUTPUT:

- If `k=None` and `value_only=False`, then return a partition of the vertex set into the minimum possible of independent sets.
- If `k=None` and `value_only=True`, return the chromatic number.
- If `k` is set and `value_only=None`, return `False` if the graph is not k-colorable, and a partition of the vertex set into k independent sets otherwise.
- If `k` is set and `value_only=True`, test whether the graph is k-colorable, and return `True` or `False` accordingly.

EXAMPLES:

```
sage: from sage.graphs.graph_coloring import vertex_coloring
sage: g = graphs.PetersenGraph()
sage: vertex_coloring(g, value_only=True)
↪# needs sage.numerical.mip
3
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_coloring import vertex_coloring
>>> g = graphs.PetersenGraph()
>>> vertex_coloring(g, value_only=True)                                    #␣
↪needs sage.numerical.mip
3
```

5.2 Interface with Cliquer (clique-related problems)

This module defines functions based on Cliquer, an exact branch-and-bound algorithm developed by Patric R. J. Ostergard and written by Sampo Niskanen.

AUTHORS:

- Nathann Cohen (2009-08-14): Initial version
- Jeroen Demeyer (2011-05-06): Make cliquer interruptible (Issue #11252)

- Nico Van Cleemput (2013-05-27): Handle the empty graph (Issue #14525)

REFERENCE:

[NO2003]

5.2.1 Methods

`sage.graphs.cliquer.`**`all_cliques`**(*graph*, *min_size=0*, *max_size=0*)

Iterator over the cliques in `graph`.

A clique is an induced complete subgraph. This method is an iterator over all the cliques with size in between `min_size` and `max_size`. By default, this method returns only maximum cliques. Each yielded clique is represented by a list of vertices.

Note

Currently only implemented for undirected graphs. Use *`to_undirected()`* to convert a digraph to an undirected graph.

INPUT:

- `min_size` – integer (default: 0); minimum size of reported cliques. When set to 0 (default), this method searches for maximum cliques. In such case, parameter `max_size` must also be set to 0.
- `max_size` – integer (default: 0); maximum size of reported cliques. When set to 0 (default), the maximum size of the cliques is unbounded. When `min_size` is set to 0, this parameter must be set to 0.

ALGORITHM:

This function is based on Cliquer [NO2003].

EXAMPLES:

```
sage: G = graphs.CompleteGraph(5)
sage: list(sage.graphs.cliquer.all_cliques(G))
[[0, 1, 2, 3, 4]]
sage: list(sage.graphs.cliquer.all_cliques(G, 2, 3))
[[3, 4],
 [2, 3],
 [2, 3, 4],
 [2, 4],
 [1, 2],
 [1, 2, 3],
 [1, 2, 4],
 [1, 3],
 [1, 3, 4],
 [1, 4],
 [0, 1],
 [0, 1, 2],
 [0, 1, 3],
 [0, 1, 4],
 [0, 2],
 [0, 2, 3],
 [0, 2, 4],
 [0, 3],
```

(continues on next page)

(continued from previous page)

```
 [0, 3, 4],
 [0, 4]]
sage: G.delete_edge([1,3])
sage: list(sage.graphs.cliquer.all_cliques(G))
[[0, 2, 3, 4], [0, 1, 2, 4]]
```

```
>>> from sage.all import *
>>> G = graphs.CompleteGraph(Integer(5))
>>> list(sage.graphs.cliquer.all_cliques(G))
[[0, 1, 2, 3, 4]]
>>> list(sage.graphs.cliquer.all_cliques(G, Integer(2), Integer(3)))
[[3, 4],
 [2, 3],
 [2, 3, 4],
 [2, 4],
 [1, 2],
 [1, 2, 3],
 [1, 2, 4],
 [1, 3],
 [1, 3, 4],
 [1, 4],
 [0, 1],
 [0, 1, 2],
 [0, 1, 3],
 [0, 1, 4],
 [0, 2],
 [0, 2, 3],
 [0, 2, 4],
 [0, 3],
 [0, 3, 4],
 [0, 4]]
>>> G.delete_edge([Integer(1),Integer(3)])
>>> list(sage.graphs.cliquer.all_cliques(G))
[[0, 2, 3, 4], [0, 1, 2, 4]]
```

Todo

Use the re-entrant functionality of Cliquer [NO2003] to avoid storing all cliques.

sage.graphs.cliquer.**all_max_clique**(*graph*)

Return the vertex sets of *ALL* the maximum complete subgraphs.

Returns the list of all maximum cliques, with each clique represented by a list of vertices. A clique is an induced complete subgraph, and a maximum clique is one of maximal order.

Note

Currently only implemented for undirected graphs. Use *to_undirected()* to convert a digraph to an undirected graph.

ALGORITHM:

This function is based on Cliquer [NO2003].

EXAMPLES:

```
sage: graphs.ChvatalGraph().cliques_maximum() # indirect doctest
[[0, 1], [0, 4], [0, 6], [0, 9], [1, 2], [1, 5], [1, 7], [2, 3],
 [2, 6], [2, 8], [3, 4], [3, 7], [3, 9], [4, 5], [4, 8], [5, 10],
 [5, 11], [6, 10], [6, 11], [7, 8], [7, 11], [8, 10], [9, 10], [9, 11]]
sage: G = Graph({0:[1,2,3], 1:[2], 3:[0,1]})
sage: G.show(figsize=[2,2])                                                    #␣
↪needs sage.plot
sage: G.cliques_maximum()
[[0, 1, 2], [0, 1, 3]]
sage: C = graphs.PetersenGraph()
sage: C.cliques_maximum()
[[0, 1], [0, 4], [0, 5], [1, 2], [1, 6], [2, 3], [2, 7], [3, 4],
 [3, 8], [4, 9], [5, 7], [5, 8], [6, 8], [6, 9], [7, 9]]
sage: C = Graph('DJ{')
sage: C.cliques_maximum()
[[1, 2, 3, 4]]
```

```
>>> from sage.all import *
>>> graphs.ChvatalGraph().cliques_maximum() # indirect doctest
[[0, 1], [0, 4], [0, 6], [0, 9], [1, 2], [1, 5], [1, 7], [2, 3],
 [2, 6], [2, 8], [3, 4], [3, 7], [3, 9], [4, 5], [4, 8], [5, 10],
 [5, 11], [6, 10], [6, 11], [7, 8], [7, 11], [8, 10], [9, 10], [9, 11]]
>>> G = Graph({Integer(0):[Integer(1),Integer(2),Integer(3)],␣
↪Integer(1):[Integer(2)], Integer(3):[Integer(0),Integer(1)]})
>>> G.show(figsize=[Integer(2),Integer(2)])                                    ␣
↪              # needs sage.plot
>>> G.cliques_maximum()
[[0, 1, 2], [0, 1, 3]]
>>> C = graphs.PetersenGraph()
>>> C.cliques_maximum()
[[0, 1], [0, 4], [0, 5], [1, 2], [1, 6], [2, 3], [2, 7], [3, 4],
 [3, 8], [4, 9], [5, 7], [5, 8], [6, 8], [6, 9], [7, 9]]
>>> C = Graph('DJ{')
>>> C.cliques_maximum()
[[1, 2, 3, 4]]
```

sage.graphs.cliquer.**clique_number**(*graph*)

Return the size of the largest clique of the graph (clique number).

Note

Currently only implemented for undirected graphs. Use *to_undirected()* to convert a digraph to an undi-

rected graph.

EXAMPLES:

```
sage: C = Graph('DJ{')
sage: C.clique_number()
4
sage: G = Graph({0:[1,2,3], 1:[2], 3:[0,1]})
sage: G.show(figsize=[2,2])                                                    #␣
↪needs sage.plot
sage: G.clique_number()
3
```

```
>>> from sage.all import *
>>> C = Graph('DJ{')
>>> C.clique_number()
4
>>> G = Graph({Integer(0):[Integer(1),Integer(2),Integer(3)],␣
↪Integer(1):[Integer(2)], Integer(3):[Integer(0),Integer(1)]})
>>> G.show(figsize=[Integer(2),Integer(2)])                                    ␣
↪              # needs sage.plot
>>> G.clique_number()
3
```

sage.graphs.cliquer.**max_clique**(*graph*)

Return the vertex set of a maximum complete subgraph.

Note

Currently only implemented for undirected graphs. Use *to_undirected()* to convert a digraph to an undirected graph.

EXAMPLES:

```
sage: C = graphs.PetersenGraph()
sage: from sage.graphs.cliquer import max_clique
sage: max_clique(C)
[7, 9]
```

```
>>> from sage.all import *
>>> C = graphs.PetersenGraph()
>>> from sage.graphs.cliquer import max_clique
>>> max_clique(C)
[7, 9]
```

5.3 Centrality

This module is meant for all functions related to centrality in networks.

centrality_betweenness()	Return the centrality betweenness of G
centrality_closeness_top_k()	Return the k most closeness central vertices of G
centrality_closeness_random_k()	Return an estimation of the closeness centrality of G.

5.3.1 Functions

sage.graphs.centrality.**centrality_betweenness**(*G*, *exact=False*, *normalize=True*)

Return the centrality betweenness of G.

The centrality betweenness of a vertex $v \in G$ is defined by:

$$c(v) = \sum_{s \neq v \neq t} \frac{\#\{\text{shortest } st - \text{paths containing v}\}}{\#\{\text{shortest } st - \text{paths}\}}$$

For more information, see the Wikipedia article Betweenness_centrality.

INPUT:

- `G` – a (di)graph
- `exact` – boolean (default: `False`); whether to compute over rationals or on `double` C variables
- `normalize` – boolean (default: `True`); whether to renormalize the values by dividing them by $\binom{n-1}{2}$ (for graphs) or $2\binom{n-1}{2}$ (for digraphs).

ALGORITHM:

To compute $c(v)$, we fix s and define $c_s(v)$ as the centrality of v *due to* s, obtained from the formula above by running the sum over t only. We obtain $c(v) = \sum_{s \neq v} c_s(v)$.

For every vertex s, we compute the value of $c_s(v)$ for all v, using the following remark (see [Brandes01]):

Let $v_1, \ldots, v_k$ be the out-neighbors of v such that $dist(s, v_i) = dist(s, v) + 1$. Then

$$c_s(v) = \sum_{1 \leq i \leq k} c_s(v_i) \frac{\#\{\text{shortest } sv_i - \text{paths}\}}{\#\{\text{shortest } sv - \text{paths}\}}$$

The number of shortest paths between s and every other vertex can be computed with a slightly modified BFS. While running this BFS we can also store the list of the vertices $v_1, \ldots, v_k$ associated with each v.

EXAMPLES:

```
sage: from sage.graphs.centrality import centrality_betweenness
sage: centrality_betweenness(digraphs.Circuit(6)) # abs tol 1e-10
{0: 0.5, 1: 0.5, 2: 0.5, 3: 0.5, 4: 0.5, 5: 0.5}
sage: centrality_betweenness(graphs.CycleGraph(6)) # abs tol 1e-10
{0: 0.2, 1: 0.2, 2: 0.2, 3: 0.2, 4: 0.2, 5: 0.2}
```

```
>>> from sage.all import *
>>> from sage.graphs.centrality import centrality_betweenness
>>> centrality_betweenness(digraphs.Circuit(Integer(6))) # abs tol 1e-10
{0: 0.5, 1: 0.5, 2: 0.5, 3: 0.5, 4: 0.5, 5: 0.5}
>>> centrality_betweenness(graphs.CycleGraph(Integer(6))) # abs tol 1e-10
{0: 0.2, 1: 0.2, 2: 0.2, 3: 0.2, 4: 0.2, 5: 0.2}
```

Exact computations:

```
sage: graphs.PetersenGraph().centrality_betweenness(exact=True)
{0: 1/12, 1: 1/12, 2: 1/12, 3: 1/12, 4: 1/12, 5: 1/12, 6: 1/12, 7: 1/12, 8: 1/12,␣
↪9: 1/12}
```

```
>>> from sage.all import *
>>> graphs.PetersenGraph().centrality_betweenness(exact=True)
{0: 1/12, 1: 1/12, 2: 1/12, 3: 1/12, 4: 1/12, 5: 1/12, 6: 1/12, 7: 1/12, 8: 1/12,␣
↪9: 1/12}
```

sage.graphs.centrality.**centrality_closeness_random_k**(*G*, *k=1*)

Return an estimation of the closeness centrality of G.

The algorithm first randomly selects a set S of k vertices. Then it computes shortest path distances from each vertex in S (using Dijkstra for weighted graph and breadth-first-search (BFS) for unweighted graph) and uses this knowledge to estimate the closeness centrality of all vertices.

For more information, see [EDI2014].

INPUT:

- `G` – an undirected connected Graph
- `k` – integer (default: 1); number of random nodes to choose

OUTPUT: a dictionary associating to each vertex its estimated closeness centrality

EXAMPLES:

Estimation of the closeness centrality of the Petersen Graph when $k == n$:

```
sage: from sage.graphs.centrality import centrality_closeness_random_k
sage: G = graphs.PetersenGraph()
sage: centrality_closeness_random_k(G, 10)
{0: 0.6,
 1: 0.6,
 2: 0.6,
 3: 0.6,
 4: 0.6,
 5: 0.6,
 6: 0.6,
 7: 0.6,
 8: 0.6,
 9: 0.6}
```

```
>>> from sage.all import *
>>> from sage.graphs.centrality import centrality_closeness_random_k
>>> G = graphs.PetersenGraph()
>>> centrality_closeness_random_k(G, Integer(10))
{0: 0.6,
 1: 0.6,
 2: 0.6,
 3: 0.6,
 4: 0.6,
 5: 0.6,
 6: 0.6,
```

(continues on next page)

(continued from previous page)

```
 7: 0.6,
 8: 0.6,
 9: 0.6}
```

sage.graphs.centrality.**centrality_closeness_top_k**(*G*, *k=1*, *verbose=0*)

Compute the `k` vertices with largest closeness centrality.

The algorithm is based on performing a breadth-first-search (BFS) from each vertex, and to use bounds in order to cut these BFSes as soon as possible. If `k` is small, it is much more efficient than computing all centralities with *centrality_closeness()*. Conversely, if `k` is close to the number of nodes, the running-time is approximately the same (it might even be a bit longer, because more computations are needed).

For more information, see [BCM15]. The algorithm does not work on weighted graphs.

INPUT:

- `G` – a Sage Graph or DiGraph;
- `k` – integer (default: 1); the algorithm will return the `k` vertices with largest closeness centrality. This value should be between 1 and the number of vertices with positive (out)degree, because the closeness centrality is not defined for vertices with (out)degree 0. If `k` is bigger than this value, the output will contain all vertices of positive (out)degree.
- `verbose` – integer (default: 0); define how "verbose" the algorithm should be. If 0, nothing is printed, if 1, we print only the performance ratio at the end of the algorithm, if 2, we print partial results every 1000 visits, if 3, we print partial results after every visit.

OUTPUT:

An ordered list of `k` pairs $(closv, v)$, where v is one of the `k` most central vertices, and $closv$ is its closeness centrality. If k is bigger than the number of vertices with positive (out)degree, the list might be smaller.

EXAMPLES:

```
sage: from sage.graphs.centrality import centrality_closeness_top_k
sage: g = graphs.PathGraph(10)
sage: centrality_closeness_top_k(g, 4, 1)
Final performance ratio: 0.711111111111...
[(0.36, 5),
 (0.36, 4),
 (0.3333333333333333, 6),
 (0.3333333333333333, 3)]
sage: g = digraphs.Path(10)
sage: centrality_closeness_top_k(g, 5, 1)
Final performance ratio: 0.422222222222...
[(0.2, 0),
 (0.19753086419753085, 1),
 (0.1944444444444442, 2),
 (0.19047619047619047, 3),
 (0.18518518518518517, 4)]
```

```
>>> from sage.all import *
>>> from sage.graphs.centrality import centrality_closeness_top_k
>>> g = graphs.PathGraph(Integer(10))
>>> centrality_closeness_top_k(g, Integer(4), Integer(1))
Final performance ratio: 0.711111111111...
```

(continues on next page)

(continued from previous page)

```
[(0.36, 5),
 (0.36, 4),
 (0.3333333333333333, 6),
 (0.3333333333333333, 3)]
>>> g = digraphs.Path(Integer(10))
>>> centrality_closeness_top_k(g, Integer(5), Integer(1))
Final performance ratio: 0.422222222222...
[(0.2, 0),
 (0.19753086419753085, 1),
 (0.1944444444444442, 2),
 (0.19047619047619047, 3),
 (0.18518518518518517, 4)]
```

5.4 Asteroidal triples

This module contains the following function:

is_asteroidal_triple_free()	Test if the input graph is asteroidal triple-free

5.4.1 Definition

Three independent vertices of a graph form an *asteroidal triple* if every two of them are connected by a path avoiding the neighborhood of the third one. A graph is *asteroidal triple-free* (*AT-free*, for short) if it contains no asteroidal triple [LB1962].

Use `graph_classes.AT_free.description()` to get some known properties of AT-free graphs, or visit this page.

5.4.2 Algorithm

This module implements the *Straightforward algorithm* recalled in [Koh2004] and due to [LB1962] for testing if a graph is AT-free or not. This algorithm has time complexity in $O(n^3)$ and space complexity in $O(n^2)$.

This algorithm uses the *connected structure* of the graph, stored into a $n \times n$ matrix M. This matrix is such that $M[u][v] == 0$ if $v \in (\{u\} \cup N(u))$, and otherwise $M[u][v]$ is the unique identifier (a strictly positive integer) of the connected component of $G \setminus (\{u\} \cup N(u))$ to which v belongs. This connected structure can be computed in time $O(n(n+m))$ using n BFS.

Now, a triple $u, v, w \in V$ is an asteroidal triple if and only if it satisfies $M[u][v] == M[u][w]$ and $M[v][u] == M[v][w]$ and $M[w][u] == M[w][v]$, assuming all these values are positive. Indeed, if $M[u][v] == M[u][w]$, v and w are in the same connected component of $G \setminus (\{u\} \cup N(u))$, and so there is a path between v and w avoiding the neighborhood of u. The algorithm iterates over all triples.

5.4.3 Functions

`sage.graphs.asteroidal_triples.`**`is_asteroidal_triple_free`**(*G*, *certificate=False*)

Test if the input graph is asteroidal triple-free.

An independent set of three vertices such that each pair is joined by a path that avoids the neighborhood of the third one is called an *asteroidal triple*. A graph is asteroidal triple-free (AT-free) if it contains no asteroidal triples. See the *module's documentation* for more details.

This method returns `True` if the graph is AT-free and `False` otherwise.

INPUT:

- `G` – a Graph
- `certificate` – boolean (default: `False`); by default, this method returns `True` if the graph is asteroidal triple-free and `False` otherwise. When `certificate==True`, this method returns in addition a list of three vertices forming an asteroidal triple if such a triple is found, and the empty list otherwise.

EXAMPLES:

The complete graph is AT-free, as well as its line graph:

```
sage: G = graphs.CompleteGraph(5)
sage: G.is_asteroidal_triple_free()
True
sage: G.is_asteroidal_triple_free(certificate=True)
(True, [])
sage: LG = G.line_graph()
sage: LG.is_asteroidal_triple_free()
True
sage: LLG = LG.line_graph()
sage: LLG.is_asteroidal_triple_free()
False
```

```
>>> from sage.all import *
>>> G = graphs.CompleteGraph(Integer(5))
>>> G.is_asteroidal_triple_free()
True
>>> G.is_asteroidal_triple_free(certificate=True)
(True, [])
>>> LG = G.line_graph()
>>> LG.is_asteroidal_triple_free()
True
>>> LLG = LG.line_graph()
>>> LLG.is_asteroidal_triple_free()
False
```

The PetersenGraph is not AT-free:

```
sage: from sage.graphs.asteroidal_triples import *
sage: G = graphs.PetersenGraph()
sage: G.is_asteroidal_triple_free()
False
sage: G.is_asteroidal_triple_free(certificate=True)
(False, [0, 2, 6])
```

```
>>> from sage.all import *
>>> from sage.graphs.asteroidal_triples import *
>>> G = graphs.PetersenGraph()
>>> G.is_asteroidal_triple_free()
False
>>> G.is_asteroidal_triple_free(certificate=True)
(False, [0, 2, 6])
```

5.5 Independent sets

This module implements the *IndependentSets* class which can be used to :

- List the independent sets (or cliques) of a graph
- Count them (which is obviously faster)
- Test whether a set of vertices is an independent set

It can also be restricted to focus on (inclusionwise) maximal independent sets. See the documentation of *IndependentSets* for actual examples.

5.5.1 Classes and methods

class sage.graphs.independent_sets.**IndependentSets**

Bases: object

The set of independent sets of a graph.

For more information on independent sets, see the Wikipedia article Independent_set_(graph_theory).

INPUT:

- G – a graph
- maximal – boolean (default: False); whether to only consider (inclusionwise) maximal independent sets
- complement – boolean (default: False); whether to consider the graph's complement (i.e. cliques instead of independent sets)

ALGORITHM:

The enumeration of independent sets is done naively : given an independent set, this implementation considers all ways to add a new vertex to it (while keeping it an independent set), and then creates new independent sets from all those that were created this way.

The implementation, however, is not recursive.

Note

This implementation of the enumeration of *maximal* independent sets is not much faster than NetworkX', which is surprising as it is written in Cython. This being said, the algorithm from NetworkX appears to be slightly different from this one, and that would be a good thing to explore if one wants to improve the implementation.

A simple generalization can also be done without too much modifications: iteration through independent sets with given size bounds (minimum and maximum number of vertices allowed).

EXAMPLES:

Listing all independent sets of the Claw graph:

```
sage: from sage.graphs.independent_sets import IndependentSets
sage: g = graphs.ClawGraph()
sage: I = IndependentSets(g)
sage: list(I)
[[0], [1], [1, 2], [1, 2, 3], [1, 3], [2], [2, 3], [3], []]
```

```
>>> from sage.all import *
>>> from sage.graphs.independent_sets import IndependentSets
>>> g = graphs.ClawGraph()
>>> I = IndependentSets(g)
>>> list(I)
[[0], [1], [1, 2], [1, 2, 3], [1, 3], [2], [2, 3], [3], []]
```

Count them:

```
sage: I.cardinality()
9
```

```
>>> from sage.all import *
>>> I.cardinality()
9
```

List only the maximal independent sets:

```
sage: Im = IndependentSets(g, maximal=True)
sage: list(Im)
[[0], [1, 2, 3]]
```

```
>>> from sage.all import *
>>> Im = IndependentSets(g, maximal=True)
>>> list(Im)
[[0], [1, 2, 3]]
```

And count them:

```
sage: Im.cardinality()
2
```

```
>>> from sage.all import *
>>> Im.cardinality()
2
```

One can easily count the number of independent sets of each cardinality:

```
sage: g = graphs.PetersenGraph()
sage: number_of = [0] * g.order()
sage: for x in IndependentSets(g):
....:     number_of[len(x)] += 1
sage: number_of
[1, 10, 30, 30, 5, 0, 0, 0, 0, 0]
```

```
>>> from sage.all import *
>>> g = graphs.PetersenGraph()
>>> number_of = [Integer(0)] * g.order()
>>> for x in IndependentSets(g):
...     number_of[len(x)] += Integer(1)
>>> number_of
[1, 10, 30, 30, 5, 0, 0, 0, 0, 0]
```

It is also possible to define an iterator over all independent sets of a given cardinality. Note, however, that Sage will generate them *all*, to return only those that satisfy the cardinality constraints. Getting the list of independent sets of size 4 in this way can thus take a very long time:

```
sage: is4 = (x for x in IndependentSets(g) if len(x) == 4)
sage: list(is4)
[[0, 2, 8, 9], [0, 3, 6, 7], [1, 3, 5, 9], [1, 4, 7, 8], [2, 4, 5, 6]]
```

```
>>> from sage.all import *
>>> is4 = (x for x in IndependentSets(g) if len(x) == Integer(4))
>>> list(is4)
[[0, 2, 8, 9], [0, 3, 6, 7], [1, 3, 5, 9], [1, 4, 7, 8], [2, 4, 5, 6]]
```

Given a subset of the vertices, it is possible to test whether it is an independent set:

```
sage: g = graphs.DurerGraph()
sage: I = IndependentSets(g)
sage: [0, 2] in I
True
sage: [0, 3, 5] in I
False
```

```
>>> from sage.all import *
>>> g = graphs.DurerGraph()
>>> I = IndependentSets(g)
>>> [Integer(0), Integer(2)] in I
True
>>> [Integer(0), Integer(3), Integer(5)] in I
False
```

If an element of the subset is not a vertex, then an error is raised:

```
sage: [0, 'a', 'b', 'c'] in I
Traceback (most recent call last):
...
ValueError: a is not a vertex of the graph
```

```
>>> from sage.all import *
>>> [Integer(0), 'a', 'b', 'c'] in I
Traceback (most recent call last):
...
ValueError: a is not a vertex of the graph
```

cardinality()

Compute and return the number of independent sets.

5.6 Chromatic polynomial

AUTHORS:

- Gordon Royle - original C implementation
- Robert Miller - transplant

REFERENCE:

See [Rea1968] and the Wikipedia article Chromatic_polynomial for more details on this notion in graphs.

sage.graphs.chrompoly.**chromatic_polynomial**(*G*, *return_tree_basis=False*, *algorithm='C'*, *cache=None*)

Compute the chromatic polynomial of the graph G.

The algorithm used is a recursive one, based on the following observations of Read [Rea1968]:

- The chromatic polynomial of a tree on n vertices is x(x-1)^(n-1).
- If e is an edge of G, G' is the result of deleting the edge e, and G" is the result of contracting e, then the chromatic polynomial of G is equal to that of G' minus that of G".

INPUT:

- `G` – a Sage graph
- `return_tree_basis` – boolean (default: `False`); not used yet
- `algorithm` – string (default: `'C'`); the algorithm to use among
  - `'C'`, an implementation in C by Robert Miller and Gordon Royle.
  - `'Python'`, an implementation in Python using caching to avoid recomputing the chromatic polynomial of a graph that has already been seen. This seems faster on some dense graphs.
- `cache` – dictionary (default: `None`); this parameter is used only for algorithm `'Python'`. It is a dictionary keyed by canonical labelings of graphs and used to cache the chromatic polynomials of the graphs generated by the algorithm. In other words, it avoids computing twice the chromatic polynomial of isometric graphs. One will be created automatically if not provided.

EXAMPLES:

```
sage: graphs.CycleGraph(4).chromatic_polynomial()
x^4 - 4*x^3 + 6*x^2 - 3*x
sage: graphs.CycleGraph(3).chromatic_polynomial()
x^3 - 3*x^2 + 2*x
sage: graphs.CubeGraph(3).chromatic_polynomial()
x^8 - 12*x^7 + 66*x^6 - 214*x^5 + 441*x^4 - 572*x^3 + 423*x^2 - 133*x
sage: graphs.PetersenGraph().chromatic_polynomial()
x^10 - 15*x^9 + 105*x^8 - 455*x^7 + 1353*x^6 - 2861*x^5 + 4275*x^4 - 4305*x^3 +␣
↪2606*x^2 - 704*x
sage: graphs.CompleteBipartiteGraph(3,3).chromatic_polynomial()
x^6 - 9*x^5 + 36*x^4 - 75*x^3 + 78*x^2 - 31*x
sage: for i in range(2,7):
....:     graphs.CompleteGraph(i).chromatic_polynomial().factor()
(x - 1) * x
(x - 2) * (x - 1) * x
(x - 3) * (x - 2) * (x - 1) * x
(x - 4) * (x - 3) * (x - 2) * (x - 1) * x
(x - 5) * (x - 4) * (x - 3) * (x - 2) * (x - 1) * x
sage: graphs.CycleGraph(5).chromatic_polynomial().factor()
(x - 2) * (x - 1) * x * (x^2 - 2*x + 2)
sage: graphs.OctahedralGraph().chromatic_polynomial().factor()
(x - 2) * (x - 1) * x * (x^3 - 9*x^2 + 29*x - 32)
sage: graphs.WheelGraph(5).chromatic_polynomial().factor()
(x - 2) * (x - 1) * x * (x^2 - 5*x + 7)
sage: graphs.WheelGraph(6).chromatic_polynomial().factor()
(x - 3) * (x - 2) * (x - 1) * x * (x^2 - 4*x + 5)
```

(continues on next page)

(continued from previous page)

```
sage: C(x)=graphs.LCFGraph(24, [12,7,-7], 8).chromatic_polynomial()  # long time␣
↪(6s on sage.math, 2011)
sage: C(2)  # long time
0
```

```
>>> from sage.all import *
>>> graphs.CycleGraph(Integer(4)).chromatic_polynomial()
x^4 - 4*x^3 + 6*x^2 - 3*x
>>> graphs.CycleGraph(Integer(3)).chromatic_polynomial()
x^3 - 3*x^2 + 2*x
>>> graphs.CubeGraph(Integer(3)).chromatic_polynomial()
x^8 - 12*x^7 + 66*x^6 - 214*x^5 + 441*x^4 - 572*x^3 + 423*x^2 - 133*x
>>> graphs.PetersenGraph().chromatic_polynomial()
x^10 - 15*x^9 + 105*x^8 - 455*x^7 + 1353*x^6 - 2861*x^5 + 4275*x^4 - 4305*x^3 +␣
↪2606*x^2 - 704*x
>>> graphs.CompleteBipartiteGraph(Integer(3),Integer(3)).chromatic_polynomial()
x^6 - 9*x^5 + 36*x^4 - 75*x^3 + 78*x^2 - 31*x
>>> for i in range(Integer(2),Integer(7)):
...     graphs.CompleteGraph(i).chromatic_polynomial().factor()
(x - 1) * x
(x - 2) * (x - 1) * x
(x - 3) * (x - 2) * (x - 1) * x
(x - 4) * (x - 3) * (x - 2) * (x - 1) * x
(x - 5) * (x - 4) * (x - 3) * (x - 2) * (x - 1) * x
>>> graphs.CycleGraph(Integer(5)).chromatic_polynomial().factor()
(x - 2) * (x - 1) * x * (x^2 - 2*x + 2)
>>> graphs.OctahedralGraph().chromatic_polynomial().factor()
(x - 2) * (x - 1) * x * (x^3 - 9*x^2 + 29*x - 32)
>>> graphs.WheelGraph(Integer(5)).chromatic_polynomial().factor()
(x - 2) * (x - 1) * x * (x^2 - 5*x + 7)
>>> graphs.WheelGraph(Integer(6)).chromatic_polynomial().factor()
(x - 3) * (x - 2) * (x - 1) * x * (x^2 - 4*x + 5)
>>> __tmp__=var("x"); C = symbolic_expression(graphs.LCFGraph(Integer(24),␣
↪[Integer(12),Integer(7),-Integer(7)], Integer(8)).chromatic_polynomial()  ).
↪function(x)# long time (6s on sage.math, 2011)
>>> C(Integer(2))  # long time
0
```

By definition, the chromatic number of a graph G is the least integer k such that the chromatic polynomial of G is strictly positive at k:

```
sage: G = graphs.PetersenGraph()
sage: P = G.chromatic_polynomial()
sage: min(i for i in range(11) if P(i) > 0) == G.chromatic_number()
True

sage: G = graphs.RandomGNP(10,0.7)
sage: P = G.chromatic_polynomial()
sage: min(i for i in range(11) if P(i) > 0) == G.chromatic_number()
True
```

```
>>> from sage.all import *
>>> G = graphs.PetersenGraph()
>>> P = G.chromatic_polynomial()
>>> min(i for i in range(Integer(11)) if P(i) > Integer(0)) == G.chromatic_
↪number()
True

>>> G = graphs.RandomGNP(Integer(10),RealNumber('0.7'))
>>> P = G.chromatic_polynomial()
>>> min(i for i in range(Integer(11)) if P(i) > Integer(0)) == G.chromatic_
↪number()
True
```

Check that algorithms `'C'` and `'Python'` return the same results:

```
sage: G = graphs.RandomGNP(8, randint(1, 9)*0.1)
sage: c = G.chromatic_polynomial(algorithm='C')
sage: p = G.chromatic_polynomial(algorithm='Python')
sage: c == p
True
```

```
>>> from sage.all import *
>>> G = graphs.RandomGNP(Integer(8), randint(Integer(1), Integer(9))*RealNumber(
↪'0.1'))
>>> c = G.chromatic_polynomial(algorithm='C')
>>> p = G.chromatic_polynomial(algorithm='Python')
>>> c == p
True
```

sage.graphs.chrompoly.**chromatic_polynomial_with_cache**(*G*, *cache=None*)

Return the chromatic polynomial of the graph `G`.

The algorithm used is here is the non recursive version of a recursive algorithm based on the following observations of Read:

- The chromatic polynomial of a tree on n vertices is $x(x-1)^{n-1}$.
- If e is an edge of G, G' is the result of deleting the edge e, and G'' is the result of contracting e, then the chromatic polynomial of G is equal to that of G' minus that of G''.
- If G is not connected, its the chromatic polynomial is the product of the chromatic polynomials of its connected components.

Since this method makes extensive use of canonical labelings, it is recommended to install optional package `bliss`.

INPUT:

- `G` – a Sage graph
- `cache` – dictionary (default: `None`); dictionary keyed by canonical labelings of graphs and used to cache the chromatic polynomials of the graphs generated by the algorithm. In other words, it avoids computing twice the chromatic polynomial of isometric graphs. One will be created automatically if not provided.

EXAMPLES:

```
sage: from sage.graphs.chrompoly import chromatic_polynomial_with_cache
sage: chromatic_polynomial_with_cache(graphs.CycleGraph(4))
```

(continues on next page)

(continued from previous page)

```
x^4 - 4*x^3 + 6*x^2 - 3*x
sage: chromatic_polynomial_with_cache(graphs.CycleGraph(3))
x^3 - 3*x^2 + 2*x
sage: chromatic_polynomial_with_cache(graphs.CubeGraph(3))
x^8 - 12*x^7 + 66*x^6 - 214*x^5 + 441*x^4 - 572*x^3 + 423*x^2 - 133*x
sage: chromatic_polynomial_with_cache(graphs.PetersenGraph())
x^10 - 15*x^9 + 105*x^8 - 455*x^7 + 1353*x^6 - 2861*x^5 + 4275*x^4 - 4305*x^3 +␣
↪2606*x^2 - 704*x
sage: chromatic_polynomial_with_cache(graphs.CompleteBipartiteGraph(3,3))
x^6 - 9*x^5 + 36*x^4 - 75*x^3 + 78*x^2 - 31*x
```

```
>>> from sage.all import *
>>> from sage.graphs.chrompoly import chromatic_polynomial_with_cache
>>> chromatic_polynomial_with_cache(graphs.CycleGraph(Integer(4)))
x^4 - 4*x^3 + 6*x^2 - 3*x
>>> chromatic_polynomial_with_cache(graphs.CycleGraph(Integer(3)))
x^3 - 3*x^2 + 2*x
>>> chromatic_polynomial_with_cache(graphs.CubeGraph(Integer(3)))
x^8 - 12*x^7 + 66*x^6 - 214*x^5 + 441*x^4 - 572*x^3 + 423*x^2 - 133*x
>>> chromatic_polynomial_with_cache(graphs.PetersenGraph())
x^10 - 15*x^9 + 105*x^8 - 455*x^7 + 1353*x^6 - 2861*x^5 + 4275*x^4 - 4305*x^3 +␣
↪2606*x^2 - 704*x
>>> chromatic_polynomial_with_cache(graphs.CompleteBipartiteGraph(Integer(3),
↪Integer(3)))
x^6 - 9*x^5 + 36*x^4 - 75*x^3 + 78*x^2 - 31*x
```

If a cache is provided, it is fed:

```
sage: cache = {}
sage: G = graphs.CycleGraph(4)
sage: p = chromatic_polynomial_with_cache(graphs.CycleGraph(4), cache=cache)
sage: key = frozenset(G.canonical_label().edges(labels=False, sort=False))
sage: cache[key]
x^4 - 4*x^3 + 6*x^2 - 3*x
```

```
>>> from sage.all import *
>>> cache = {}
>>> G = graphs.CycleGraph(Integer(4))
>>> p = chromatic_polynomial_with_cache(graphs.CycleGraph(Integer(4)),␣
↪cache=cache)
>>> key = frozenset(G.canonical_label().edges(labels=False, sort=False))
>>> cache[key]
x^4 - 4*x^3 + 6*x^2 - 3*x
```

5.7 Cographs

A cograph is a P_4-free graph, that is a graph without induced path of order 4. Any cograph may be constructed, starting from the single vertex graph, by a sequence of join and disjoint union operations. See the Wikipedia article Cograph for more details on this graph class, and OEIS sequence A000084 to know the number of cographs of order $n \geq 1$.

This module implements the following methods concerning cographs:

cographs()	Return an iterator over the cographs of order n.

5.7.1 Methods

class sage.graphs.cographs.**CoTree**(*name='root'*)

Bases: object

Generic cotree node.

This data structure is used for the generation of cographs in *cographs()*.

add_child(*node*)

Add cotree `node` in the list of children of `self`.

INPUT:

- `node` – a CoTree

EXAMPLES:

```
sage: from sage.graphs.cographs import CoTree
sage: T = CoTree('J')
sage: T.add_child(CoTree(1))
sage: T
[ J ( 1 ) ]
```

```
>>> from sage.all import *
>>> from sage.graphs.cographs import CoTree
>>> T = CoTree('J')
>>> T.add_child(CoTree(Integer(1)))
>>> T
[ J ( 1 ) ]
```

copy_tree(*T*)

Make T a copy of `self`.

INPUT:

- `T` – a CoTree

EXAMPLES:

```
sage: from sage.graphs.cographs import CoTree
sage: T = CoTree('J')
sage: T.add_child(CoTree(1))
sage: T.add_child(CoTree(2))
sage: T
[ J ( 1 ) ( 2 ) ]
sage: B =  CoTree('U')
sage: T.copy_tree(B)
sage: B
[ U ( 1 ) ( 2 ) ]
```

```
>>> from sage.all import *
>>> from sage.graphs.cographs import CoTree
```

(continues on next page)

(continued from previous page)

```
>>> T = CoTree('J')
>>> T.add_child(CoTree(Integer(1)))
>>> T.add_child(CoTree(Integer(2)))
>>> T
[ J ( 1 ) ( 2 ) ]
>>> B =  CoTree('U')
>>> T.copy_tree(B)
>>> B
[ U ( 1 ) ( 2 ) ]
```

reset_info()

Reset parameter `info` from all nodes of `self`.

EXAMPLES:

```
sage: from sage.graphs.cographs import CoTree
sage: T = CoTree(1)
sage: B = CoTree(2)
sage: T.add_child(B)
sage: C = CoTree(3)
sage: B.add_child(C)
sage: C.info = 'info'
sage: T.reset_info()
sage: C.info is None
True
```

```
>>> from sage.all import *
>>> from sage.graphs.cographs import CoTree
>>> T = CoTree(Integer(1))
>>> B = CoTree(Integer(2))
>>> T.add_child(B)
>>> C = CoTree(Integer(3))
>>> B.add_child(C)
>>> C.info = 'info'
>>> T.reset_info()
>>> C.info is None
True
```

sage.graphs.cographs.**change_label**(*tree*, *status*, *counter*)

Set the names of the nodes of `tree`.

This is a helper method to method *cographs()*.

The input `tree` is such that each node has as label its number of children. This method changes the label of each node so that a parallel node is labeled 'U', a series node is labeled 'J' and a leaf node gets a unique number.

INPUT:

- `tree` – the tree to relabel
- `status` – boolean; used to detect series (`True`) and parallel (`False`) internal nodes
- `counter` – list; the first integer of the list is used to assign a unique number to the leaves of the tree

EXAMPLES:

```
sage: next(graphs.cographs(4, as_graph=True)).vertices()  # indirect doctest
[0, 1, 2, 3]
```

```
>>> from sage.all import *
>>> next(graphs.cographs(Integer(4), as_graph=True)).vertices()  # indirect␣
↪doctest
[0, 1, 2, 3]
```

sage.graphs.cographs.**cographs**(*n*, *as_graph=True*, *immutable=False*)

Return an iterator over the cographs of order n.

A cograph is a P_4-free graph, that is a graph without induced path of order 4. Any cograph may be constructed, starting from the single vertex graph, by a sequence of *sage.graphs.graph.Graph.join()* and *sage.graphs.generic_graph.GenericGraph.disjoint_union()* operations. See the Wikipedia article Cograph for more details.

This method implements the generator of all cographs of order n proposed in [JPD2018]. The algorithm generates one by one every cotree with n nodes, and each cotree is generated by using its previous cotree. The time to construct the first cotree is $O(n)$ and the time spent between two consecutive outputs is $O(n)$. Hence, the overall complexity of the algorithm is $O(n * M_n)$, where n is the number of nodes and M_n is the total number of cographs with n nodes (see OEIS sequence A000084).

INPUT:

- `n` – integer larger or equal to 1
- `as_graph` – boolean (default: `True`); whether to return graphs or the tree data structure encoding the graph
- `immutable` – boolean (default: `False`); whether to return an immutable or a mutable graph. This parameter is used only when `as_graph is True`.

EXAMPLES:

The output can be either cotrees or graphs:

```
sage: for t in graphs.cographs(3, as_graph=False):
....:     print(t)
[ J ( 0 ) ( 1 ) ( 2 ) ]
[ J [ U ( 0 ) ( 1 ) ( 2 ) ] ]
[ J ( 0 ) [ U ( 1 ) ( 2 ) ] ]
[ J [ U ( 0 ) [ J ( 1 ) ( 2 ) ] ] ]
sage: for g in graphs.cographs(3, as_graph=True):
....:     print(g.edges(labels=False, sort=True))
[(0, 1), (0, 2), (1, 2)]
[]
[(0, 1), (0, 2)]
[(1, 2)]
```

```
>>> from sage.all import *
>>> for t in graphs.cographs(Integer(3), as_graph=False):
...     print(t)
[ J ( 0 ) ( 1 ) ( 2 ) ]
[ J [ U ( 0 ) ( 1 ) ( 2 ) ] ]
[ J ( 0 ) [ U ( 1 ) ( 2 ) ] ]
[ J [ U ( 0 ) [ J ( 1 ) ( 2 ) ] ] ]
>>> for g in graphs.cographs(Integer(3), as_graph=True):
```

(continues on next page)

(continued from previous page)

```
...     print(g.edges(labels=False, sort=True))
[(0, 1), (0, 2), (1, 2)]
[]
[(0, 1), (0, 2)]
[(1, 2)]
```

Check that we agree with OEIS sequence A000084:

```
sage: [sum(1 for _ in graphs.cographs(n, as_graph=False))
....:  for n in range(1, 8)]
[1, 2, 4, 10, 24, 66, 180]
```

```
>>> from sage.all import *
>>> [sum(Integer(1) for _ in graphs.cographs(n, as_graph=False))
...  for n in range(Integer(1), Integer(8))]
[1, 2, 4, 10, 24, 66, 180]
```

sage.graphs.cographs.**find_pivot**(T)

Search for a pivot node in T.

This is a helper method to method *cographs()*.

A node in T is a `pivot` if it is not a leaf, it does not induce a maximum partition and it is the first such node in the inverse postorder traversal.

INPUT:

- `T` – a `CoTree`

EXAMPLES:

```
sage: next(graphs.cographs(3, as_graph=True)).vertices()  # indirect doctest
[0, 1, 2]
```

```
>>> from sage.all import *
>>> next(graphs.cographs(Integer(3), as_graph=True)).vertices()  # indirect␣
↪doctest
[0, 1, 2]
```

sage.graphs.cographs.**next_tree**(T)

Check if there is another tree after T.

This is a helper method to method *cographs()*.

This methods returns `True` if there is a tree after T, and if so it modifies the input tree T that becomes the next tree.

INPUT:

- `T` – a `CoTree`

EXAMPLES:

```
sage: next(graphs.cographs(3, as_graph=True)).vertices()  # indirect doctest
[0, 1, 2]
```

```
>>> from sage.all import *
>>> next(graphs.cographs(Integer(3), as_graph=True)).vertices()  # indirect␣
↪doctest
[0, 1, 2]
```

sage.graphs.cographs.**rebuild_node**(*u*, *P*)

Replace the subtree rooted at u by a subtree induced by partition P.

This is a helper method to method *cographs()*.

INPUT:

- u – a CoTree
- P – a partition encoding the new structure of the children of u

EXAMPLES:

```
sage: next(graphs.cographs(3, as_graph=True)).vertices()  # indirect doctest
[0, 1, 2]
```

```
>>> from sage.all import *
>>> next(graphs.cographs(Integer(3), as_graph=True)).vertices()  # indirect␣
↪doctest
[0, 1, 2]
```

sage.graphs.cographs.**tree_to_graph**(*tree*, *immutable=False*)

Return the cograph represented by tree.

This is a helper method to method *cographs()*.

EXAMPLES:

```
sage: for t in graphs.cographs(3, as_graph=True):  # indirect doctest
....:     print(t.edges(labels=False, sort=True))
[(0, 1), (0, 2), (1, 2)]
[]
[(0, 1), (0, 2)]
[(1, 2)]
```

```
>>> from sage.all import *
>>> for t in graphs.cographs(Integer(3), as_graph=True):  # indirect doctest
...     print(t.edges(labels=False, sort=True))
[(0, 1), (0, 2), (1, 2)]
[]
[(0, 1), (0, 2)]
[(1, 2)]
```

5.8 Comparability and permutation graphs

This module implements method related to Wikipedia article Comparability_graph and Wikipedia article Permutation_graph, that is, for the moment, only recognition algorithms.

Most of the information found here can also be found in [ST1994] or [Sha1997].

The following methods are implemented in this module

is_comparability_MILP()	Check whether the graph is a comparability graph (MILP)
greedy_is_comparability()	Check whether the graph is a comparability graph (greedy algorithm)
greedy_is_comparability_with_certificate()	Check whether the graph is a comparability graph and returns certificates (greedy algorithm)
is_comparability()	Check whether the graph is a comparability graph
is_permutation()	Check whether the graph is a permutation graph.
is_transitive()	Check whether the digraph is transitive.

Author:

- Nathann Cohen 2012-04

5.8.1 Graph classes

Comparability graphs

A graph is a comparability graph if it can be obtained from a poset by adding an edge between any two elements that are comparable. Co-comparability graph are complements of such graphs, i.e. graphs built from a poset by adding an edge between any two incomparable elements.

For more information on comparability graphs, see the Wikipedia article Comparability_graph.

Permutation graphs

Definitions:

- A permutation $\pi = \pi_1\pi_2\dots\pi_n$ defines a graph on n vertices such that $i \sim j$ when π reverses i and j (i.e. when $i < j$ and $\pi_j < \pi_i$. A graph is a permutation graph whenever it can be built through this construction.
- A graph is a permutation graph if it can be build from two parallel lines are the intersection graph of segments intersecting both lines.
- A graph is a permutation graph if it is both a comparability graph and a co-comparability graph.

For more information on permutation graphs, see the Wikipedia article Permutation_graph.

5.8.2 Recognition algorithm for comparability graphs

Greedy algorithm

This algorithm attempts to build a transitive orientation of a given graph G, that is an orientation D such that for any directed uv-path of D there exists in D an edge uv. This already determines a notion of equivalence between some edges of G :

> In G, two edges uv and uv' (incident to a common vertex u) such that $vv' \notin G$ need necessarily be oriented *the same way* (that is that they should either both *leave* or both *enter* u). Indeed, if one enters G while the other leaves it, these two edges form a path of length two, which is not possible in any transitive orientation of G as $vv' \notin G$.

Hence, we can say that in this case a *directed edge* uv is equivalent to a *directed edge* uv' (to mean that if one belongs to the transitive orientation, the other one must be present too) in the same way that vu is equivalent to $v'u$. We can thus define equivalence classes on oriented edges, to represent set of edges that imply each other. We can thus define C^G_{uv} to be the equivalence class in G of the oriented edge uv.

Of course, if there exists a transitive orientation of a graph G, then no edge uv implies its contrary vu, i.e. it is necessary to ensure that $\forall uv \in G, vu \notin C^G_{uv}$. The key result on which the greedy algorithm is built is the following (see [ST1994]):

> **Theorem –** The following statements are equivalent :

- G is a comparability graph
- $\forall uv \in G, vu \notin C_{uv}^G$
- The edges of G can be partitioned into $B_1, ..., B_k$ where B_i is the equivalence class of some oriented edge in $G - B_1 - \cdots - B_{i-1}$

Hence, ensuring that a graph is a comparability graph can be done by checking that no equivalence class is contradictory. Building the orientation, however, requires to build equivalence classes step by step until an orientation has been found for all of them.

Mixed Integer Linear Program

A MILP formulation is available to check the other methods for correction. It is easily built :

> To each edge are associated two binary variables (one for each possible direction). We then ensure that each triangle is transitively oriented, and that each pair of incident edges uv, uv' such that $vv' \notin G$ do not create a 2-path.

Here is the formulation:

$$
\begin{aligned}
&\text{Maximize : Nothing}\\
&\text{Such that :}\\
&\qquad \forall uv \in G\\
&\qquad \cdot\ o_{uv} + o_{vu} = 1\\
&\qquad \forall u \in G, \forall v, v' \in N(v) \text{ such that } vv' \notin G\\
&\qquad \cdot\ o_{uv} + o_{v'u} - o_{v'v} \leq 1\\
&\qquad \cdot\ o_{uv'} + o_{vu} - o_{vv'} \leq 1\\
&\qquad \forall u \in G, \forall v, v' \in N(v) \text{ such that } vv' \in G\\
&\qquad \cdot\ o_{uv} + o_{v'u} \leq 1\\
&\qquad \cdot\ o_{uv'} + o_{vu} \leq 1\\
&\qquad o_{uv} \text{ is a binary variable}
\end{aligned}
$$

Note

The MILP formulation is usually much slower than the greedy algorithm. This MILP has been implemented to check the results of the greedy algorithm that has been implemented to check the results of a faster algorithm which has not been implemented yet.

5.8.3 Certificates

Comparability graphs

The *yes*-certificates that a graph is a comparability graphs are transitive orientations of it. The *no*-certificates, on the other hand, are odd cycles of such graph. These odd cycles have the property that around each vertex v of the cycle its two incident edges must have the same orientation (toward v, or outward v) in any transitive orientation of the graph. This is impossible whenever the cycle has odd length. Explanations are given in the "Greedy algorithm" part of the previous section.

Permutation graphs

Permutation graphs are precisely the intersection of comparability graphs and co-comparability graphs. Hence, negative certificates are precisely negative certificates of comparability or co-comparability. Positive certificates are a pair of permutations that can be used through `PermutationGraph()` (whose documentation says more about what these permutations represent).

5.8.4 Implementation details

Test that the equivalence classes are not self-contradictory

This is done by a call to *is_bipartite()*, and here is how :

> Around a vertex u, any two edges uv, uv' such that $vv' \notin G$ are equivalent. Hence, the equivalence class of edges around a vertex are precisely the connected components of the complement of the graph induced by the neighbors of u.
>
> In each equivalence class (around a given vertex u), the edges should all have the same orientation, i.e. all should go toward u at the same time, or leave it at the same time. To represent this, we create a graph with vertices for all equivalent classes around all vertices of G, and link (v, C) to (u, C') if $u \in C$ and $v \in C'$.
>
> A bipartite coloring of this graph with colors 0 and 1 tells us that the edges of an equivalence class C around u should be directed toward u if (u, C) is colored with 0, and outward if (u, C) is colored with 1.
>
> If the graph is not bipartite, this is the proof that some equivalence class is self-contradictory !

Note

The greedy algorithm implemented here is just there to check the correction of more complicated ones, and it is reaaaaaaaaaaalllly bad whenever you look at it with performance in mind.

5.8.5 Methods

sage.graphs.comparability.**greedy_is_comparability**(*g*, *no_certificate=False*, *equivalence_class=False*)

Check whether the graph is a comparability graph (greedy algorithm).

This method only returns no-certificates.

To understand how this method works, please consult the documentation of the *comparability module*.

INPUT:

- `g` – a graph
- `no_certificate` – whether to return a *no*-certificate when the graph is not a comparability graph. This certificate is an odd cycle of edges, each of which implies the next. It is set to `False` by default.
- `equivalence_class` – whether to return an equivalence class if the graph is a comparability graph

OUTPUT:

- If the graph is a comparability graph and `no_certificate = False`, this method returns `True` or `(True, an_equivalence_class)` according to the value of `equivalence_class`.
- If the graph is *not* a comparability graph, this method returns `False` or `(False, odd_cycle)` according to the value of `no_certificate`.

EXAMPLES:

The Petersen Graph is not transitively orientable:

```
sage: from sage.graphs.comparability import greedy_is_comparability as is_
↪comparability
sage: g = graphs.PetersenGraph()
sage: is_comparability(g)
False
sage: is_comparability(g, no_certificate=True)
(False, [2, 1, 0, 4, 3, 2])
```

```
>>> from sage.all import *
>>> from sage.graphs.comparability import greedy_is_comparability as is_
↪comparability
>>> g = graphs.PetersenGraph()
>>> is_comparability(g)
False
>>> is_comparability(g, no_certificate=True)
(False, [2, 1, 0, 4, 3, 2])
```

But the Bull graph is:

```
sage: g = graphs.BullGraph()
sage: is_comparability(g)
True
```

```
>>> from sage.all import *
>>> g = graphs.BullGraph()
>>> is_comparability(g)
True
```

sage.graphs.comparability.**greedy_is_comparability_with_certificate**(*g*, *certificate=False*)

Check whether the graph is a comparability graph and returns certificates (greedy algorithm).

This method can return certificates of both *yes* and *no* answers.

To understand how this method works, please consult the documentation of the *comparability module*.

INPUT:

- `g` – a graph
- `certificate` – boolean; whether to return a certificate. *Yes*-answers the certificate is a transitive orientation of G, and a *no* certificates is an odd cycle of sequentially forcing edges.

EXAMPLES:

The 5-cycle or the Petersen Graph are not transitively orientable:

```
sage: from sage.graphs.comparability import greedy_is_comparability_with_
↪certificate as is_comparability
sage: is_comparability(graphs.CycleGraph(5), certificate=True)
(False, [2, 1, 0, 4, 3, 2])
sage: g = graphs.PetersenGraph()
sage: is_comparability(g)
False
sage: is_comparability(g, certificate=True)
(False, [2, 1, 0, 4, 3, 2])
```

```
>>> from sage.all import *
>>> from sage.graphs.comparability import greedy_is_comparability_with_
↪certificate as is_comparability
>>> is_comparability(graphs.CycleGraph(Integer(5)), certificate=True)
(False, [2, 1, 0, 4, 3, 2])
>>> g = graphs.PetersenGraph()
>>> is_comparability(g)
False
```

(continues on next page)

(continued from previous page)

```
>>> is_comparability(g, certificate=True)
(False, [2, 1, 0, 4, 3, 2])
```

But the Bull graph is:

```
sage: g = graphs.BullGraph()
sage: is_comparability(g)
True
sage: is_comparability(g, certificate = True)
(True, Digraph on 5 vertices)
sage: is_comparability(g, certificate = True)[1].is_transitive()
True
```

```
>>> from sage.all import *
>>> g = graphs.BullGraph()
>>> is_comparability(g)
True
>>> is_comparability(g, certificate = True)
(True, Digraph on 5 vertices)
>>> is_comparability(g, certificate = True)[Integer(1)].is_transitive()
True
```

sage.graphs.comparability.**is_comparability**(*g*, *algorithm='greedy'*, *certificate=False*, *check=True*, *solver=None*, *verbose=0*)

Check whether the graph is a comparability graph.

INPUT:

- `g` – a graph
- `algorithm` – string (default: `'greedy'`); choose the implementation used to do the test
  - `'greedy'` – a greedy algorithm (see the documentation of the *comparability module*)
  - `'MILP'` – a Mixed Integer Linear Program formulation of the problem. Beware, for this implementation is unable to return negative certificates ! When `certificate = True`, negative certificates are always equal to `None`. `True` certificates are valid, though.
- `certificate` – boolean (default: `False`); whether to return a certificate. *Yes*-answers the certificate is a transitive orientation of G, and a *no* certificates is an odd cycle of sequentially forcing edges.
- `check` – boolean (default: `True`); whether to check that the yes-certificates are indeed transitive. As it is very quick compared to the rest of the operation, it is enabled by default.
- `solver` – string (default: `None`); specifies a Mixed Integer Linear Programming (MILP) solver to be used. If set to `None`, the default one is used. For more information on MILP solvers and which default solver is used, see the method `solve` of the class `MixedIntegerLinearProgram`.
- `verbose` – integer (default: 0); sets the level of verbosity. Set to 0 by default, which means quiet.

EXAMPLES:

```
sage: from sage.graphs.comparability import is_comparability
sage: g = graphs.PetersenGraph()
sage: is_comparability(g)
False
```

(continues on next page)

(continued from previous page)

```
sage: is_comparability(graphs.CompleteGraph(5), certificate=True)
(True, Digraph on 5 vertices)
```

```
>>> from sage.all import *
>>> from sage.graphs.comparability import is_comparability
>>> g = graphs.PetersenGraph()
>>> is_comparability(g)
False
>>> is_comparability(graphs.CompleteGraph(Integer(5)), certificate=True)
(True, Digraph on 5 vertices)
```

sage.graphs.comparability.**is_comparability_MILP**(*g*, *certificate=False*, *solver=None*, *verbose=0*)

Check whether the graph is a comparability graph (MILP).

INPUT:

- `g` – a graph
- `certificate` – boolean (default: `False`); whether to return a certificate for yes instances. This method cannot return negative certificates.
- `solver` – string (default: `None`); specifies a Mixed Integer Linear Programming (MILP) solver to be used. If set to `None`, the default one is used. For more information on MILP solvers and which default solver is used, see the method `solve` of the class `MixedIntegerLinearProgram`.
- `verbose` – integer (default: 0); sets the level of verbosity. Set to 0 by default, which means quiet.

EXAMPLES:

The 5-cycle or the Petersen Graph are not transitively orientable:

```
sage: from sage.graphs.comparability import is_comparability_MILP as is_
↪comparability
sage: is_comparability(graphs.CycleGraph(5), certificate=True)                  #␣
↪needs sage.numerical.mip
(False, None)
sage: g = graphs.PetersenGraph()
sage: is_comparability(g, certificate=True)                                     #␣
↪needs sage.numerical.mip
(False, None)
```

```
>>> from sage.all import *
>>> from sage.graphs.comparability import is_comparability_MILP as is_
↪comparability
>>> is_comparability(graphs.CycleGraph(Integer(5)), certificate=True)           ␣
↪     # needs sage.numerical.mip
(False, None)
>>> g = graphs.PetersenGraph()
>>> is_comparability(g, certificate=True)                                     #␣
↪needs sage.numerical.mip
(False, None)
```

But the Bull graph is:

```
sage: g = graphs.BullGraph()
sage: is_comparability(g)                                                   #␣
↪needs sage.numerical.mip
True
sage: is_comparability(g, certificate=True)                                 #␣
↪needs sage.numerical.mip
(True, Digraph on 5 vertices)
sage: is_comparability(g, certificate=True)[1].is_transitive()              #␣
↪needs sage.numerical.mip
True
```

```
>>> from sage.all import *
>>> g = graphs.BullGraph()
>>> is_comparability(g)                                                     #␣
↪needs sage.numerical.mip
True
>>> is_comparability(g, certificate=True)                                   #␣
↪needs sage.numerical.mip
(True, Digraph on 5 vertices)
>>> is_comparability(g, certificate=True)[Integer(1)].is_transitive()       ␣
↪     # needs sage.numerical.mip
True
```

sage.graphs.comparability.**is_permutation**(*g*, *algorithm='greedy'*, *certificate=False*, *check=True*, *solver=None*, *verbose=0*)

Check whether the graph is a permutation graph.

For more information on permutation graphs, refer to the documentation of the *comparability module*.

INPUT:

- `g` – a graph
- `algorithm` – string (default: `'greedy'`); choose the implementation used for the subcalls to *is_comparability()*
  - `'greedy'` – a greedy algorithm (see the documentation of the *comparability module*)
  - `'MILP'` – a Mixed Integer Linear Program formulation of the problem. Beware, for this implementation is unable to return negative certificates ! When `certificate = True`, negative certificates are always equal to `None`. `True` certificates are valid, though.
- `certificate` – boolean (default: `False`); whether to return a certificate for the answer given. For `True` answers the certificate is a permutation, for `False` answers it is a no-certificate for the test of comparability or co-comparability.
- `check` – boolean (default: `True`); whether to check that the permutations returned indeed create the expected Permutation graph. Pretty cheap compared to the rest, hence a good investment. It is enabled by default.
- `solver` – string (default: `None`); specifies a Mixed Integer Linear Programming (MILP) solver to be used. If set to `None`, the default one is used. For more information on MILP solvers and which default solver is used, see the method `solve` of the class `MixedIntegerLinearProgram`.
- `verbose` – integer (default: 0); sets the level of verbosity. Set to 0 by default, which means quiet.

Note

As the `True` certificate is a `Permutation` object, the segment intersection model of the permutation graph can be visualized through a call to `Permutation.show`.

EXAMPLES:

A permutation realizing the bull graph:

```
sage: from sage.graphs.comparability import is_permutation
sage: g = graphs.BullGraph()
sage: _ , certif = is_permutation(g, certificate=True)
sage: h = graphs.PermutationGraph(*certif)
sage: h.is_isomorphic(g)
True
```

```
>>> from sage.all import *
>>> from sage.graphs.comparability import is_permutation
>>> g = graphs.BullGraph()
>>> _ , certif = is_permutation(g, certificate=True)
>>> h = graphs.PermutationGraph(*certif)
>>> h.is_isomorphic(g)
True
```

Plotting the realization as an intersection graph of segments:

```
sage: true, perm = is_permutation(g, certificate=True)
sage: p1 = Permutation([nn+1 for nn in perm[0]])
sage: p2 = Permutation([nn+1 for nn in perm[1]])
sage: p = p2 * p1.inverse()
sage: p.show(representation='braid')                                    #␣
↪needs sage.plot
```

```
>>> from sage.all import *
>>> true, perm = is_permutation(g, certificate=True)
>>> p1 = Permutation([nn+Integer(1) for nn in perm[Integer(0)]])
>>> p2 = Permutation([nn+Integer(1) for nn in perm[Integer(1)]])
>>> p = p2 * p1.inverse()
>>> p.show(representation='braid')                                      #␣
↪needs sage.plot
```

`sage.graphs.comparability.`**`is_transitive`**(*g*, *certificate=False*)

Check whether the digraph is transitive.

A digraph is transitive if for any pair of vertices $u, v \in G$ linked by a uv-path the edge uv belongs to G.

INPUT:

- `g` – a digraph
- `certificate` – boolean (default: `False`); whether to return a certificate for negative answers
  - If `certificate = False` (default), this method returns `True` or `False` according to the graph.
  - If `certificate = True`, this method either returns `True` answers or yield a pair of vertices uv such that there exists a uv-path in G but $uv \notin G$.

EXAMPLES:

```
sage: digraphs.Circuit(4).is_transitive()
False
sage: digraphs.Circuit(4).is_transitive(certificate=True)
(0, 2)
sage: digraphs.RandomDirectedGNP(30,.2).is_transitive()
False
sage: D = digraphs.DeBruijn(5, 2)                                           #␣
↪needs sage.combinat
sage: D.is_transitive()                                                     #␣
↪needs sage.combinat
False
sage: cert = D.is_transitive(certificate=True)                              #␣
↪needs sage.combinat
sage: D.has_edge(*cert)                                                     #␣
↪needs sage.combinat
False
sage: bool(D.shortest_path(*cert))                                          #␣
↪needs sage.combinat
True
sage: digraphs.RandomDirectedGNP(20,.2).transitive_closure().is_transitive()    #␣
↪needs networkx
True
```

```
>>> from sage.all import *
>>> digraphs.Circuit(Integer(4)).is_transitive()
False
>>> digraphs.Circuit(Integer(4)).is_transitive(certificate=True)
(0, 2)
>>> digraphs.RandomDirectedGNP(Integer(30),RealNumber('.2')).is_transitive()
False
>>> D = digraphs.DeBruijn(Integer(5), Integer(2))                                 ␣
↪              # needs sage.combinat
>>> D.is_transitive()                                                        #␣
↪needs sage.combinat
False
>>> cert = D.is_transitive(certificate=True)                                 #␣
↪needs sage.combinat
>>> D.has_edge(*cert)                                                        #␣
↪needs sage.combinat
False
>>> bool(D.shortest_path(*cert))                                             #␣
↪needs sage.combinat
True
>>> digraphs.RandomDirectedGNP(Integer(20),RealNumber('.2')).transitive_closure().
↪is_transitive()     # needs networkx
True
```

5.9 Isoperimetric inequalities

This module contains various functions to compute isoperimetric numbers of a graph.

Authors:

- Peleg Michaeli
- Vincent Delecroix

sage.graphs.isoperimetric_inequalities.**cheeger_constant**(*g*)

Return the cheeger constant of the graph.

The Cheeger constant of a graph $G = (V, E)$ is the minimum of $|\partial S|/|Vol(S)|$ where $Vol(S)$ is the sum of degrees of element in S, ∂S is the edge boundary of S (number of edges with one end in S and one end in $V \setminus S$) and the minimum is taken over all non-empty subsets S of vertices so that $|Vol(S)| \leq |E|$.

See also

Alternative but similar quantities can be obtained via the methods *edge_isoperimetric_number()* and *vertex_isoperimetric_number()*.

EXAMPLES:

```
sage: graphs.PetersenGraph().cheeger_constant()
1/3
```

```
>>> from sage.all import *
>>> graphs.PetersenGraph().cheeger_constant()
1/3
```

The Cheeger constant of a cycle on n vertices is $1/\lfloor n/2 \rfloor$:

```
sage: [graphs.CycleGraph(k).cheeger_constant() for k in range(2,10)]
[1, 1, 1/2, 1/2, 1/3, 1/3, 1/4, 1/4]
```

```
>>> from sage.all import *
>>> [graphs.CycleGraph(k).cheeger_constant() for k in range(Integer(2),
↪Integer(10))]
[1, 1, 1/2, 1/2, 1/3, 1/3, 1/4, 1/4]
```

The Cheeger constant of a complete graph on n vertices is $\lceil n/2 \rceil/(n-1)$:

```
sage: [graphs.CompleteGraph(k).cheeger_constant() for k in range(2,10)]
[1, 1, 2/3, 3/4, 3/5, 2/3, 4/7, 5/8]
```

```
>>> from sage.all import *
>>> [graphs.CompleteGraph(k).cheeger_constant() for k in range(Integer(2),
↪Integer(10))]
[1, 1, 2/3, 3/4, 3/5, 2/3, 4/7, 5/8]
```

For complete bipartite:

```
sage: [graphs.CompleteBipartiteGraph(i,j).cheeger_constant() for i in range(2,7)␣
↪for j in range(2, i)]
[3/5, 1/2, 3/5, 5/9, 4/7, 5/9, 1/2, 5/9, 1/2, 5/9]
```

```
>>> from sage.all import *
>>> [graphs.CompleteBipartiteGraph(i,j).cheeger_constant() for i in␣
↪range(Integer(2),Integer(7)) for j in range(Integer(2), i)]
[3/5, 1/2, 3/5, 5/9, 4/7, 5/9, 1/2, 5/9, 1/2, 5/9]
```

More examples:

```
sage: G = Graph([(0, 1), (0, 3), (0, 8), (1, 4), (1, 6), (2, 4), (2, 7), (2, 9),
....:            (3, 6), (3, 8), (4, 9), (5, 6), (5, 7), (5, 8), (7, 9)])
sage: G.cheeger_constant()
1/6

sage: G = Graph([(0, 1), (0, 2), (1, 2), (1, 3), (1, 4), (1, 5), (2, 3), (3, 4),␣
↪(3, 5)])
sage: G.cheeger_constant()
1/2

sage: Graph([[1,2,3,4],[(1,2),(3,4)]]).cheeger_constant()
0
```

```
>>> from sage.all import *
>>> G = Graph([(Integer(0), Integer(1)), (Integer(0), Integer(3)), (Integer(0),␣
↪Integer(8)), (Integer(1), Integer(4)), (Integer(1), Integer(6)), (Integer(2),␣
↪Integer(4)), (Integer(2), Integer(7)), (Integer(2), Integer(9)),
...              (Integer(3), Integer(6)), (Integer(3), Integer(8)), (Integer(4),␣
↪Integer(9)), (Integer(5), Integer(6)), (Integer(5), Integer(7)), (Integer(5),␣
↪Integer(8)), (Integer(7), Integer(9))])
>>> G.cheeger_constant()
1/6

>>> G = Graph([(Integer(0), Integer(1)), (Integer(0), Integer(2)), (Integer(1),␣
↪Integer(2)), (Integer(1), Integer(3)), (Integer(1), Integer(4)), (Integer(1),␣
↪Integer(5)), (Integer(2), Integer(3)), (Integer(3), Integer(4)), (Integer(3),␣
↪Integer(5))])
>>> G.cheeger_constant()
1/2

>>> Graph([[Integer(1),Integer(2),Integer(3),Integer(4)],[(Integer(1),Integer(2)),
↪(Integer(3),Integer(4))]]).cheeger_constant()
0
```

sage.graphs.isoperimetric_inequalities.**edge_isoperimetric_number**(*g*)

Return the edge-isoperimetric number of the graph.

The edge-isoperimetric number of a graph $G = (V, E)$ is also sometimes called the *isoperimetric number*. It is defined as the minimum of $|\partial S|/|S|$ where ∂S is the edge boundary of S (number of edges with one end in S and one end in $V \setminus S$) and the minimum is taken over all subsets of vertices whose cardinality does not exceed half the size $|V|$ of the graph.

See also

Alternative but similar quantities can be obtained via the methods *cheeger_constant()* and *vertex_isoperimetric_number()*.

EXAMPLES:

The edge-isoperimetric number of a complete graph on n vertices is $\lceil n/2 \rceil$:

```
sage: [graphs.CompleteGraph(n).edge_isoperimetric_number() for n in range(2,10)]
[1, 2, 2, 3, 3, 4, 4, 5]
```

```
>>> from sage.all import *
>>> [graphs.CompleteGraph(n).edge_isoperimetric_number() for n in␣
↪range(Integer(2),Integer(10))]
[1, 2, 2, 3, 3, 4, 4, 5]
```

The edge-isoperimetric constant of a cycle on n vertices is $2/\lfloor n/2 \rfloor$:

```
sage: [graphs.CycleGraph(n).edge_isoperimetric_number() for n in range(2,15)]
[1, 2, 1, 1, 2/3, 2/3, 1/2, 1/2, 2/5, 2/5, 1/3, 1/3, 2/7]
```

```
>>> from sage.all import *
>>> [graphs.CycleGraph(n).edge_isoperimetric_number() for n in range(Integer(2),
↪Integer(15))]
[1, 2, 1, 1, 2/3, 2/3, 1/2, 1/2, 2/5, 2/5, 1/3, 1/3, 2/7]
```

In general, for d-regular graphs the edge-isoperimetric number is d times larger than the Cheeger constant of the graph:

```
sage: g = graphs.RandomRegular(3, 10)                                    #␣
↪needs networkx
sage: g.edge_isoperimetric_number() == g.cheeger_constant() * 3          #␣
↪needs networkx
True
```

```
>>> from sage.all import *
>>> g = graphs.RandomRegular(Integer(3), Integer(10))                     ␣
↪              # needs networkx
>>> g.edge_isoperimetric_number() == g.cheeger_constant() * Integer(3)    ␣
↪     # needs networkx
True
```

And the edge-isoperimetric constant of a disconnected graph is 0:

```
sage: Graph([[1,2,3,4],[(1,2),(3,4)]]).edge_isoperimetric_number()
0
```

```
>>> from sage.all import *
>>> Graph([[Integer(1),Integer(2),Integer(3),Integer(4)],[(Integer(1),Integer(2)),
↪(Integer(3),Integer(4))]]).edge_isoperimetric_number()
0
```

sage.graphs.isoperimetric_inequalities.**vertex_isoperimetric_number**(*g*)

Return the vertex-isoperimetric number of the graph.

The vertex-isoperimetric number of a graph $G = (V, E)$ is also sometimes called the *magnifying constant*. It is defined as the minimum of $|N(S)|/|S|$ where $|N(S)|$ is the vertex boundary of S and the minimum is taken over the subsets S of vertices of size at most half of the vertices.

See also

Alternative but similar quantities can be obtained via the methods *cheeger_constant()* and *edge_isoperimetric_number()*.

EXAMPLES:

The vertex-isoperimetric number of a complete graph on n vertices is $\lceil n/2 \rceil / \lfloor n/2 \rfloor$:

```
sage: [graphs.CompleteGraph(k).vertex_isoperimetric_number() for k in range(2,15)]
[1, 2, 1, 3/2, 1, 4/3, 1, 5/4, 1, 6/5, 1, 7/6, 1]
```

```
>>> from sage.all import *
>>> [graphs.CompleteGraph(k).vertex_isoperimetric_number() for k in␣
↪range(Integer(2),Integer(15))]
[1, 2, 1, 3/2, 1, 4/3, 1, 5/4, 1, 6/5, 1, 7/6, 1]
```

The vertex-isoperimetric number of a cycle on n vertices is $2/\lfloor n/2 \rfloor$:

```
sage: [graphs.CycleGraph(k).vertex_isoperimetric_number() for k in range(2,15)]
[1, 2, 1, 1, 2/3, 2/3, 1/2, 1/2, 2/5, 2/5, 1/3, 1/3, 2/7]
```

```
>>> from sage.all import *
>>> [graphs.CycleGraph(k).vertex_isoperimetric_number() for k in range(Integer(2),
↪Integer(15))]
[1, 2, 1, 1, 2/3, 2/3, 1/2, 1/2, 2/5, 2/5, 1/3, 1/3, 2/7]
```

And the vertex-isoperimetric number of a disconnected graph is 0:

```
sage: Graph([[1,2,3],[(1,2)]]).vertex_isoperimetric_number()
0
```

```
>>> from sage.all import *
>>> Graph([[Integer(1),Integer(2),Integer(3)],[(Integer(1),Integer(2))]]).vertex_
↪isoperimetric_number()
0
```

The vertex-isoperimetric number is independent of edge multiplicity:

```
sage: G = graphs.CycleGraph(6)
sage: G.vertex_isoperimetric_number()
2/3
sage: G.allow_multiple_edges(True)
sage: G.add_edges(G.edges(sort=False))
sage: G.vertex_isoperimetric_number()
2/3
```

```
>>> from sage.all import *
>>> G = graphs.CycleGraph(Integer(6))
>>> G.vertex_isoperimetric_number()
2/3
>>> G.allow_multiple_edges(True)
>>> G.add_edges(G.edges(sort=False))
>>> G.vertex_isoperimetric_number()
2/3
```

5.10 Line graphs

This module gather everything which is related to line graphs. Right now, this amounts to the following functions :

line_graph()	Return the line graph of a given graph
is_line_graph()	Check whether a graph is a line graph
root_graph()	Return the root graph corresponding to the given graph

Author:

- Nathann Cohen (01-2013), *root_graph()* method and module documentation. Written while listening to Nina Simone *"I wish I knew how it would feel to be free"*. Crazy good song. And *"Prendre ta douleur"*, too.
- David Coudert (10-2018), use maximal cliques iterator in *root_graph()*, and use *root_graph()* instead of forbidden subgraph search in *is_line_graph()* (Issue #26444).

5.10.1 Definition

Given a graph G, the *line graph* $L(G)$ of G is the graph such that

$$V(L(G)) = E(G)$$
$$E(L(G)) = \{(e, e') : \text{ and } e, e' \text{ have a common endpoint in } G\}$$

The definition is extended to directed graphs. In this situation, there is an arc (e, e') in $L(G)$ if the destination of e is the origin of e'.

For more information, see the Wikipedia article Line_graph.

5.10.2 Root graph

A graph whose line graph is LG is called the *root graph* of LG. The root graph of a (connected) graph is unique ([Whi1932], [Har1969]), except when $LG = K_3$, as both $L(K_3)$ and $L(K_{1,3})$ are equal to K_3.

Here is how we can *"see"* G by staring (very intently) at LG :

A graph LG is the line graph of G if there exists a collection $(S_v)_{v \in G}$ of subsets of $V(LG)$ such that :

- Every S_v is a complete subgraph of LG.
- Every $v \in LG$ belongs to exactly two sets of the family $(S_v)_{v \in G}$.
- Any two sets of $(S_v)_{v \in G}$ have at most one common elements
- For any edge $(u, v) \in LG$ there exists a set of $(S_v)_{v \in G}$ containing both u and v.

In this family, each set S_v represent a vertex of G, and contains "the set of edges incident to v in G". Two elements $S_v, S_{v'}$ have a nonempty intersection whenever vv' is an edge of G.

Hence, finding the root graph of LG is the job of finding this collection of sets.

In particular, what we know for sure is that a maximal clique S of size 2 or ≥ 4 in LG corresponds to a vertex of degree $|S|$ in G, whose incident edges are the elements of S itself.

The main problem lies with maximal cliques of size 3, i.e. triangles. Those we have to split into two categories, *even* and *odd* triangles :

> A triangle $\{e_1, e_2, e_3\} \subseteq V(LG)$ is said to be an *odd* triangle if there exists a vertex $e \in V(G)$ incident to exactly *one* or *all* of $\{e_1, e_2, e_3\}$, and it is said to be *even* otherwise.

The very good point of this definition is that an inclusionwise maximal clique which is an odd triangle will always correspond to a vertex of degree 3 in G, while an even triangle could result from either a vertex of degree 3 in G or a triangle in G. And in order to build the root graph we obviously have to decide *which*.

Beineke proves in [Bei1970] that the collection of sets we are looking for can be easily found. Indeed it turns out that it is the union of :

1. The family of all maximal cliques of LG of size 2 or ≥ 4, as well as all odd triangles.
2. The family of all pairs of adjacent vertices which appear in exactly *one* maximal clique which is an even triangle.

There are actually four special cases to which the decomposition above does not apply, i.e. graphs containing an edge which belongs to exactly two even triangles. We deal with those independently.

- The `Complete graph` K_3.
- The `Diamond graph` – the line graph of $K_{1,3}$ plus an edge.
- The `Wheel graph` on $4+1$ vertices – the line graph of the `Diamond graph`
- The `Octahedron` – the line graph of K_4.

This decomposition turns out to be very easy to implement :-)

Warning

Even though the root graph is *NOT UNIQUE* for the triangle, this method returns $K_{1,3}$ (and not K_3) in this case. Pay *very close* attention to that, for this answer is not theoretically correct : there is no unique answer in this case, and we deal with it by returning one of the two possible answers.

5.10.3 Functions

`sage.graphs.line_graph.`**`is_line_graph`**(*g*, *certificate=False*)

Check whether the graph g is a line graph.

INPUT:

- `certificate` – boolean; whether to return a certificate along with the boolean result. Here is what happens when `certificate = True`:
  - If the graph is not a line graph, the method returns a pair `(b, subgraph)` where `b` is `False` and `subgraph` is a subgraph isomorphic to one of the 9 forbidden induced subgraphs of a line graph.
  - If the graph is a line graph, the method returns a triple `(b,R,isom)` where `b` is `True`, `R` is a graph whose line graph is the graph given as input, and `isom` is a map associating an edge of `R` to each vertex of the graph.

Note

This method wastes a bit of time when the input graph is not connected. If you have performance in mind, it is probably better to only feed it with connected graphs only.

See also

- The *line_graph* module.
- *line_graph_forbidden_subgraphs()* – the forbidden subgraphs of a line graph.
- *line_graph()*

EXAMPLES:

A complete graph is always the line graph of a star:

```
sage: graphs.CompleteGraph(5).is_line_graph()
True
```

```
>>> from sage.all import *
>>> graphs.CompleteGraph(Integer(5)).is_line_graph()
True
```

The Petersen Graph not being claw-free, it is not a line graph:

```
sage: graphs.PetersenGraph().is_line_graph()
False
```

```
>>> from sage.all import *
>>> graphs.PetersenGraph().is_line_graph()
False
```

This is indeed the subgraph returned:

```
sage: C = graphs.PetersenGraph().is_line_graph(certificate=True)[1]              #␣
↪needs sage.modules
sage: C.is_isomorphic(graphs.ClawGraph())                                        #␣
↪needs sage.modules
True
```

```
>>> from sage.all import *
>>> C = graphs.PetersenGraph().is_line_graph(certificate=True)[Integer(1)]       ␣
↪     # needs sage.modules
>>> C.is_isomorphic(graphs.ClawGraph())                                          #␣
↪needs sage.modules
True
```

The house graph is a line graph:

```
sage: g = graphs.HouseGraph()
sage: g.is_line_graph()
True
```

```
>>> from sage.all import *
>>> g = graphs.HouseGraph()
>>> g.is_line_graph()
True
```

But what is the graph whose line graph is the house ?:

```
sage: is_line, R, isom = g.is_line_graph(certificate=True)
sage: R.sparse6_string()
':DaHI~'
sage: R.show()                                                              #␣
↪needs sage.plot
sage: isom
{0: (0, 1), 1: (0, 2), 2: (1, 3), 3: (2, 3), 4: (3, 4)}
```

```
>>> from sage.all import *
>>> is_line, R, isom = g.is_line_graph(certificate=True)
>>> R.sparse6_string()
':DaHI~'
>>> R.show()                                                                #␣
↪needs sage.plot
>>> isom
{0: (0, 1), 1: (0, 2), 2: (1, 3), 3: (2, 3), 4: (3, 4)}
```

sage.graphs.line_graph.**line_graph**(*g*, *labels=True*, *return_labels=False*, *immutable=None*)

Return the line graph of the (di)graph `g` (multiedges and loops allowed).

INPUT:

- `labels` – boolean (default: `True`); whether edge labels should be taken in consideration. If `labels=True`, the vertices of the line graph will be triples `(u,v,label)`, and pairs of vertices otherwise. In case of multiple edges, the vertices of the line graph will be triples `(u,v,an integer)`.
- `return_labels` – boolean (default: `False`); whether edge labels should be stored or not. If g has multiple edges and if `return_labels=True`, the method returns a list the first element of which is the line-graph of g and the second element is a dictionary {vertex of the line-graph: former corresponding edge with its original label}. If `return_labels=False`, the method returns only the line-graph.
- `immutable` – boolean (default: `None`); whether to create a mutable/immutable (di)graph. `immutable=None` (default) means that the (di)graph and its line (di)graph will behave the same way.

The line graph of an undirected graph G is an undirected simple graph H such that the vertices of H are the edges of G and two vertices e and f of H are adjacent if e and f share a common vertex in G. In other words, an edge in H represents a path of length 2 in G.

Loops are not adjacent to themselves.

The line graph of a directed graph G is a directed graph H such that the vertices of H are the edges of G and two vertices e and f of H are adjacent if e and f share a common vertex in G and the terminal vertex of e is the initial vertex of f. In other words, an edge in H represents a (directed) path of length 2 in G.

Note

As a *Graph* object only accepts hashable objects as vertices (and as the vertices of the line graph are the edges of the graph), this code will fail if edge labels are not hashable. You can also set the argument `labels=False` to ignore labels.

See also

- The *line_graph* module
- *line_graph_forbidden_subgraphs()* – the forbidden subgraphs of a line graph
- *is_line_graph()* – tests whether a graph is a line graph

EXAMPLES:

```
sage: g = graphs.CompleteGraph(4)
sage: h = g.line_graph()
sage: h.vertices(sort=True)
[(0, 1, None),
 (0, 2, None),
 (0, 3, None),
 (1, 2, None),
 (1, 3, None),
 (2, 3, None)]
sage: h.am()                                                                #␣
↪needs sage.modules
[0 1 1 1 1 0]
[1 0 1 1 0 1]
[1 1 0 0 1 1]
[1 1 0 0 1 1]
[1 0 1 1 0 1]
[0 1 1 1 1 0]
sage: h2 = g.line_graph(labels=False)
sage: h2.vertices(sort=True)
[(0, 1), (0, 2), (0, 3), (1, 2), (1, 3), (2, 3)]
sage: h2.am() == h.am()                                                     #␣
↪needs sage.modules
True
sage: g = DiGraph([[1..4], lambda i,j: i < j])
sage: h = g.line_graph()
sage: h.vertices(sort=True)
[(1, 2, None),
 (1, 3, None),
 (1, 4, None),
 (2, 3, None),
 (2, 4, None),
 (3, 4, None)]
sage: h.edges(sort=True)
[((1, 2, None), (2, 3, None), None),
 ((1, 2, None), (2, 4, None), None),
 ((1, 3, None), (3, 4, None), None),
 ((2, 3, None), (3, 4, None), None)]
```

```
>>> from sage.all import *
>>> g = graphs.CompleteGraph(Integer(4))
>>> h = g.line_graph()
>>> h.vertices(sort=True)
[(0, 1, None),
```

(continues on next page)

(continued from previous page)

```
 (0, 2, None),
 (0, 3, None),
 (1, 2, None),
 (1, 3, None),
 (2, 3, None)]
>>> h.am()                                                          # needs␣
↪sage.modules
[0 1 1 1 1 0]
[1 0 1 1 0 1]
[1 1 0 0 1 1]
[1 1 0 0 1 1]
[1 0 1 1 0 1]
[0 1 1 1 1 0]
>>> h2 = g.line_graph(labels=False)
>>> h2.vertices(sort=True)
[(0, 1), (0, 2), (0, 3), (1, 2), (1, 3), (2, 3)]
>>> h2.am() == h.am()                                               # needs␣
↪sage.modules
True
>>> g = DiGraph([(ellipsis_range(Integer(1),Ellipsis,Integer(4))), lambda i,j: i
↪< j])
>>> h = g.line_graph()
>>> h.vertices(sort=True)
[(1, 2, None),
 (1, 3, None),
 (1, 4, None),
 (2, 3, None),
 (2, 4, None),
 (3, 4, None)]
>>> h.edges(sort=True)
[((1, 2, None), (2, 3, None), None),
 ((1, 2, None), (2, 4, None), None),
 ((1, 3, None), (3, 4, None), None),
 ((2, 3, None), (3, 4, None), None)]
```

Examples with multiple edges:

```
sage: L = Graph([(0,1),(0,1),(1,2)],multiedges=True).line_graph()
sage: L.edges()
[((0, 1, 0), (0, 1, 1), None), ((0, 1, 1), (1, 2, 2), None),
((0, 1, 0), (1, 2, 2), None)]
sage: G = Graph([(0,1),(0,1,'a'),(0,1,'b'),(0,2),(1,2,'c')],
....: multiedges=True)
sage: L = G.line_graph(False,True)
sage: L[0].edges()
[((0, 1, 1), (0, 1, 2), None), ((0, 1, 0), (0, 1, 2), None),
((0, 1, 2), (0, 2, 3), None), ((0, 1, 2), (1, 2, 4), None), ((0, 1, 0),
(0, 1, 1), None), ((0, 1, 1), (0, 2, 3), None), ((0, 1, 1),
(1, 2, 4), None), ((0, 1, 0), (0, 2, 3), None), ((0, 1, 0),
(1, 2, 4), None), ((0, 2, 3), (1, 2, 4), None)]
sage: L[1]
{(0, 1, 0): (0, 1, None),
```

(continues on next page)

(continued from previous page)

```
(0, 1, 1): (0, 1, 'b'),
(0, 1, 2): (0, 1, 'a'),
(0, 2, 3): (0, 2, None),
(1, 2, 4): (1, 2, 'c')}
sage: g = DiGraph([(0,1),(0,1),(1,2)],multiedges=True)
sage: g.line_graph().edges()
[((0, 1, 1), (1, 2, 2), None), ((0, 1, 0), (1, 2, 2), None)]
```

```
>>> from sage.all import *
>>> L = Graph([(Integer(0),Integer(1)),(Integer(0),Integer(1)),(Integer(1),
↪Integer(2))],multiedges=True).line_graph()
>>> L.edges()
[((0, 1, 0), (0, 1, 1), None), ((0, 1, 1), (1, 2, 2), None),
((0, 1, 0), (1, 2, 2), None)]
>>> G = Graph([(Integer(0),Integer(1)),(Integer(0),Integer(1),'a'),(Integer(0),
↪Integer(1),'b'),(Integer(0),Integer(2)),(Integer(1),Integer(2),'c')],
... multiedges=True)
>>> L = G.line_graph(False,True)
>>> L[Integer(0)].edges()
[((0, 1, 1), (0, 1, 2), None), ((0, 1, 0), (0, 1, 2), None),
((0, 1, 2), (0, 2, 3), None), ((0, 1, 2), (1, 2, 4), None), ((0, 1, 0),
(0, 1, 1), None), ((0, 1, 1), (0, 2, 3), None), ((0, 1, 1),
(1, 2, 4), None), ((0, 1, 0), (0, 2, 3), None), ((0, 1, 0),
(1, 2, 4), None), ((0, 2, 3), (1, 2, 4), None)]
>>> L[Integer(1)]
{(0, 1, 0): (0, 1, None),
(0, 1, 1): (0, 1, 'b'),
(0, 1, 2): (0, 1, 'a'),
(0, 2, 3): (0, 2, None),
(1, 2, 4): (1, 2, 'c')}
>>> g = DiGraph([(Integer(0),Integer(1)),(Integer(0),Integer(1)),(Integer(1),
↪Integer(2))],multiedges=True)
>>> g.line_graph().edges()
[((0, 1, 1), (1, 2, 2), None), ((0, 1, 0), (1, 2, 2), None)]
```

An example with a loop:

```
sage: g = Graph([(0,0),(0,1),(0,2),(1,2)],multiedges=True,loops=True)
sage: L = g.line_graph()
sage: L.edges()
[((0, 0, None), (0, 1, None), None), ((0, 0, None), (0, 2, None), None),
((0, 1, None), (0, 2, None), None), ((0, 1, None), (1, 2, None), None),
((0, 2, None), (1, 2, None), None)]
```

```
>>> from sage.all import *
>>> g = Graph([(Integer(0),Integer(0)),(Integer(0),Integer(1)),(Integer(0),
↪Integer(2)),(Integer(1),Integer(2))],multiedges=True,loops=True)
>>> L = g.line_graph()
>>> L.edges()
[((0, 0, None), (0, 1, None), None), ((0, 0, None), (0, 2, None), None),
((0, 1, None), (0, 2, None), None), ((0, 1, None), (1, 2, None), None),
((0, 2, None), (1, 2, None), None)]
```

sage.graphs.line_graph.**root_graph**(*g*, *verbose=False*, *immutable=None*)

Return the root graph corresponding to the given graph `g`.

See the documentation of *sage.graphs.line_graph* to know how it works.

INPUT:

- `g` – a graph
- `verbose` – boolean (default: `False`); display some information about what is happening inside of the algorithm
- `immutable` – boolean (default: `None`); whether to create a mutable/immutable (di)graph. `immutable=None` (default) means that the (di)graph and its root (di)graph will behave the same way.

Warning

This code assumes that g is a line graph, and is a connected, undirected graph without multiple edges.

5.11 Spanning trees

This module is a collection of algorithms on spanning trees. Also included in the collection are algorithms for minimum spanning trees. See the book [JNC2010] for descriptions of spanning tree algorithms, including minimum spanning trees.

See also

- *GenericGraph.min_spanning_tree*.

Todo

- Parallel version of Boruvka's algorithm.

5.11.1 Methods

sage.graphs.spanning_tree.**boruvka**(*G*, *by_weight=True*, *weight_function=None*, *check_weight=True*, *check=False*)

Minimum spanning tree using Boruvka's algorithm.

This function assumes that we can only compute minimum spanning trees for undirected graphs. Such graphs can be weighted or unweighted, and they can have multiple edges (since we are computing the minimum spanning tree, only the minimum weight among all (u, v)-edges is considered, for each pair of vertices u, v).

INPUT:

- `G` – an undirected graph
- `by_weight` – boolean (default: `True`); if `True`, the edges in the graph are weighted. If `False`, all edges have weight 1
- `weight_function` – function (default: `None`); a function that takes as input an edge `(u, v, l)` and outputs its weight. If not `None`, `by_weight` is automatically set to `True`. If `None` and `by_weight` is `True`, we use the edge label `l`, if `l` is not `None`, else `1` as a weight.

- `check_weight` – boolean (default: `False`); whether to check that the `weight_function` outputs a number for each edge
- `check` – boolean (default: `False`); whether to first perform sanity checks on the input graph `G`. Default: `check=False`. If we toggle `check=True`, the following sanity checks are first performed on `G` prior to running Boruvka's algorithm on that input graph:
  - Is `G` the null graph or graph on one vertex?
  - Is `G` disconnected?
  - Is `G` a tree?

  By default, we turn off the sanity checks for performance reasons. This means that by default the function assumes that its input graph is connected, and has at least one vertex. Otherwise, you should set `check=True` to perform some sanity checks and preprocessing on the input graph.

OUTPUT:

The edges of a minimum spanning tree of `G`, if one exists, otherwise returns the empty list.

See also

- *min_spanning_tree()*

EXAMPLES:

An example from pages 727–728 in [Sah2000]:

```
sage: from sage.graphs.spanning_tree import boruvka
sage: G = Graph({1:{2:28, 6:10}, 2:{3:16, 7:14}, 3:{4:12}, 4:{5:22, 7:18}, 5:
↪{6:25, 7:24}})
sage: G.weighted(True)
sage: E = boruvka(G, check=True); E
[(1, 6, 10), (2, 7, 14), (3, 4, 12), (4, 5, 22), (5, 6, 25), (2, 3, 16)]
sage: boruvka(G, by_weight=True)
[(1, 6, 10), (2, 7, 14), (3, 4, 12), (4, 5, 22), (5, 6, 25), (2, 3, 16)]
sage: sorted(boruvka(G, by_weight=False))
[(1, 2, 28), (1, 6, 10), (2, 3, 16), (2, 7, 14), (3, 4, 12), (4, 5, 22)]
```

```
>>> from sage.all import *
>>> from sage.graphs.spanning_tree import boruvka
>>> G = Graph({Integer(1):{Integer(2):Integer(28), Integer(6):Integer(10)},␣
↪Integer(2):{Integer(3):Integer(16), Integer(7):Integer(14)}, Integer(3):
↪{Integer(4):Integer(12)}, Integer(4):{Integer(5):Integer(22),␣
↪Integer(7):Integer(18)}, Integer(5):{Integer(6):Integer(25),␣
↪Integer(7):Integer(24)}})
>>> G.weighted(True)
>>> E = boruvka(G, check=True); E
[(1, 6, 10), (2, 7, 14), (3, 4, 12), (4, 5, 22), (5, 6, 25), (2, 3, 16)]
>>> boruvka(G, by_weight=True)
[(1, 6, 10), (2, 7, 14), (3, 4, 12), (4, 5, 22), (5, 6, 25), (2, 3, 16)]
>>> sorted(boruvka(G, by_weight=False))
[(1, 2, 28), (1, 6, 10), (2, 3, 16), (2, 7, 14), (3, 4, 12), (4, 5, 22)]
```

An example with custom edge labels:

```
sage: G = Graph([[0,1,1],[1,2,1],[2,0,10]], weighted=True)
sage: weight = lambda e:3-e[0]-e[1]
sage: boruvka(G, weight_function=lambda e:3-e[0]-e[1], by_weight=True)
[(0, 2, 10), (1, 2, 1)]
sage: boruvka(G, weight_function=lambda e:float(1/e[2]), by_weight=True)
[(0, 2, 10), (0, 1, 1)]
```

```
>>> from sage.all import *
>>> G = Graph([[Integer(0),Integer(1),Integer(1)],[Integer(1),Integer(2),
↪Integer(1)],[Integer(2),Integer(0),Integer(10)]], weighted=True)
>>> weight = lambda e:Integer(3)-e[Integer(0)]-e[Integer(1)]
>>> boruvka(G, weight_function=lambda e:Integer(3)-e[Integer(0)]-e[Integer(1)],␣
↪by_weight=True)
[(0, 2, 10), (1, 2, 1)]
>>> boruvka(G, weight_function=lambda e:float(Integer(1)/e[Integer(2)]), by_
↪weight=True)
[(0, 2, 10), (0, 1, 1)]
```

An example of disconnected graph with `check` disabled:

```
sage: from sage.graphs.spanning_tree import boruvka
sage: G = Graph({1:{2:28}, 3:{4:16}}, weighted=True)
sage: boruvka(G, check=False)
[]
```

```
>>> from sage.all import *
>>> from sage.graphs.spanning_tree import boruvka
>>> G = Graph({Integer(1):{Integer(2):Integer(28)}, Integer(3):
↪{Integer(4):Integer(16)}}, weighted=True)
>>> boruvka(G, check=False)
[]
```

sage.graphs.spanning_tree.**edge_disjoint_spanning_trees**(*G*, *k*, *by_weight=False*, *weight_function=None*, *check_weight=True*)

Return k edge-disjoint spanning trees of minimum cost.

This method implements the Roskind-Tarjan algorithm for finding k minimum-cost edge-disjoint spanning trees in simple undirected graphs [RT1985]. When edge weights are taken into account, the algorithm ensures that the sum of the weights of the returned spanning trees is minimized. The time complexity of the algorithm is in $O(k^2n^2)$ for the unweighted case and otherwise in $O(m \log m + k^2n^2)$.

This method raises an error if the graph does not contain the requested number of spanning trees.

INPUT:

- `G` – a simple undirected graph
- `k` – the requested number of edge-disjoint spanning trees
- `by_weight` – boolean (default: `False`); if `True`, the edges in the graph are weighted, otherwise all edges have weight 1
- `weight_function` – function (default: `None`); a function that takes as input an edge `(u, v, l)` and outputs its weight. If not `None`, `by_weight` is automatically set to `True`. If `None` and `by_weight` is `True`, we use the edge label `l`, if `l` is not `None`, else `1` as a weight.

- `check_weight` – boolean (default: `True`); if `True`, we check that the `weight_function` outputs a number for each edge

EXAMPLES:

Example from [RT1985]:

```
sage: from sage.graphs.spanning_tree import edge_disjoint_spanning_trees
sage: G = Graph({'a': ['b', 'c', 'd', 'e'], 'b': ['c', 'e'], 'c': ['d'], 'd': ['e
↪']})
sage: F = edge_disjoint_spanning_trees(G, 2)
sage: F
[Graph on 5 vertices, Graph on 5 vertices]
sage: [f.is_tree() for f in F]
[True, True]
```

```
>>> from sage.all import *
>>> from sage.graphs.spanning_tree import edge_disjoint_spanning_trees
>>> G = Graph({'a': ['b', 'c', 'd', 'e'], 'b': ['c', 'e'], 'c': ['d'], 'd': ['e']}
↪)
>>> F = edge_disjoint_spanning_trees(G, Integer(2))
>>> F
[Graph on 5 vertices, Graph on 5 vertices]
>>> [f.is_tree() for f in F]
[True, True]
```

This method raises an error if the graph does not contain the required number of trees:

```
sage: edge_disjoint_spanning_trees(G, 3)
Traceback (most recent call last):
...
EmptySetError: this graph does not contain the required number of trees/
↪arborescences
```

```
>>> from sage.all import *
>>> edge_disjoint_spanning_trees(G, Integer(3))
Traceback (most recent call last):
...
EmptySetError: this graph does not contain the required number of trees/
↪arborescences
```

A clique of order n has $\lfloor n/2 \rfloor$ edge disjoint spanning trees:

```
sage: for n in range(1, 10):
....:     g = graphs.CompleteGraph(n)
....:     F = edge_disjoint_spanning_trees(g, n//2)
```

```
>>> from sage.all import *
>>> for n in range(Integer(1), Integer(10)):
...     g = graphs.CompleteGraph(n)
...     F = edge_disjoint_spanning_trees(g, n//Integer(2))
```

The sum of the weights of the returned spanning trees is minimum:

```
sage: g = graphs.CompleteGraph(5)
sage: for u, v in g.edges(sort=True, labels=False):
....:     g.set_edge_label(u, v, 1)
sage: g.set_edge_label(0, 1, 33)
sage: g.set_edge_label(1, 3, 33)
sage: F = edge_disjoint_spanning_trees(g, 2, by_weight=True)
sage: sum(F[0].edge_labels()) + sum(F[1].edge_labels())
8
```

```
>>> from sage.all import *
>>> g = graphs.CompleteGraph(Integer(5))
>>> for u, v in g.edges(sort=True, labels=False):
...     g.set_edge_label(u, v, Integer(1))
>>> g.set_edge_label(Integer(0), Integer(1), Integer(33))
>>> g.set_edge_label(Integer(1), Integer(3), Integer(33))
>>> F = edge_disjoint_spanning_trees(g, Integer(2), by_weight=True)
>>> sum(F[Integer(0)].edge_labels()) + sum(F[Integer(1)].edge_labels())
8
```

sage.graphs.spanning_tree.**filter_kruskal**(*G*, *threshold=10000*, *by_weight=True*, *weight_function=None*, *check_weight=True*, *check=False*)

Minimum spanning tree using Filter Kruskal algorithm.

This function implements the variant of Kruskal's algorithm proposed in [OSS2009]. Instead of directly sorting the whole set of edges, it partitions it in a similar way to quicksort and filter out edges that connect vertices of the same tree to reduce the cost of sorting.

This function assumes that we can only compute minimum spanning trees for undirected graphs. Such graphs can be weighted or unweighted, and they can have multiple edges (since we are computing the minimum spanning tree, only the minimum weight among all (u, v)-edges is considered, for each pair of vertices u, v).

INPUT:

- `G` – an undirected graph
- `threshold` – integer (default: 10000); maximum number of edges on which to run kruskal algorithm. Above that value, edges are partitioned into sets of size at most `threshold`
- `by_weight` – boolean (default: `True`); if `True`, the edges in the graph are weighted. If `False`, all edges have weight 1.
- `weight_function` – function (default: `None`); a function that takes as input an edge `(u, v, l)` and outputs its weight. If not `None`, `by_weight` is automatically set to `True`. If `None` and `by_weight` is `True`, we use the edge label `l`, if `l` is not `None`, else `1` as a weight.
- `check_weight` – boolean (default: `False`); whether to check that the `weight_function` outputs a number for each edge
- `check` – boolean (default: `False`); whether to first perform sanity checks on the input graph `G`. Default: `check=False`. If we toggle `check=True`, the following sanity checks are first performed on `G` prior to running Kruskal's algorithm on that input graph:
  - Is `G` the null graph?
  - Is `G` disconnected?
  - Is `G` a tree?
  - Does `G` have self-loops?

– Does `G` have multiple edges?

OUTPUT:

The edges of a minimum spanning tree of `G`, if one exists, otherwise returns the empty list.

See also

- *sage.graphs.generic_graph.GenericGraph.min_spanning_tree()*
- Wikipedia article Kruskal%27s_algorithm
- *kruskal()*
- *filter_kruskal_iterator()*

EXAMPLES:

```
sage: from sage.graphs.spanning_tree import filter_kruskal
sage: G = Graph({1:{2:28, 6:10}, 2:{3:16, 7:14}, 3:{4:12}, 4:{5:22, 7:18}, 5:
↪{6:25, 7:24}})
sage: G.weighted(True)
sage: filter_kruskal(G, check=True)
[(1, 6, 10), (3, 4, 12), (2, 7, 14), (2, 3, 16), (4, 5, 22), (5, 6, 25)]

sage: filter_kruskal(Graph(2), check=True)
[]
```

```
>>> from sage.all import *
>>> from sage.graphs.spanning_tree import filter_kruskal
>>> G = Graph({Integer(1):{Integer(2):Integer(28), Integer(6):Integer(10)},␣
↪Integer(2):{Integer(3):Integer(16), Integer(7):Integer(14)}, Integer(3):
↪{Integer(4):Integer(12)}, Integer(4):{Integer(5):Integer(22),␣
↪Integer(7):Integer(18)}, Integer(5):{Integer(6):Integer(25),␣
↪Integer(7):Integer(24)}})
>>> G.weighted(True)
>>> filter_kruskal(G, check=True)
[(1, 6, 10), (3, 4, 12), (2, 7, 14), (2, 3, 16), (4, 5, 22), (5, 6, 25)]

>>> filter_kruskal(Graph(Integer(2)), check=True)
[]
```

sage.graphs.spanning_tree.**filter_kruskal_iterator**(*G*, *threshold=10000*, *by_weight=True*, *weight_function=None*, *check_weight=True*, *check=False*)

Return an iterator implementation of Filter Kruskal's algorithm.

INPUT:

- `G` – an undirected graph
- `threshold` – integer (default: 10000); maximum number of edges on which to run kruskal algorithm. Above that value, edges are partitioned into sets of size at most `threshold`
- `by_weight` – boolean (default: `True`); if `True`, the edges in the graph are weighted. If `False`, all edges have weight 1.

- `weight_function` – function (default: `None`); a function that takes as input an edge `(u, v, l)` and outputs its weight. If not `None`, `by_weight` is automatically set to `True`. If `None` and `by_weight` is `True`, we use the edge label `l`, if `l` is not `None`, else `1` as a weight.
- `check_weight` – boolean (default: `False`); whether to check that the `weight_function` outputs a number for each edge
- `check` – boolean (default: `False`); whether to first perform sanity checks on the input graph `G`. Default: `check=False`. If we toggle `check=True`, the following sanity checks are first performed on `G` prior to running Kruskal's algorithm on that input graph:
  - Is `G` the null graph?
  - Is `G` disconnected?
  - Is `G` a tree?
  - Does `G` have self-loops?
  - Does `G` have multiple edges?

OUTPUT: the edges of a minimum spanning tree of `G`, one by one

See also

- *sage.graphs.generic_graph.GenericGraph.min_spanning_tree()*
- Wikipedia article Kruskal%27s_algorithm
- *kruskal()*
- *filter_kruskal()*

EXAMPLES:

The edges of a minimum spanning tree of `G`, if one exists, otherwise returns the empty list.

```
sage: from sage.graphs.spanning_tree import filter_kruskal_iterator
sage: G = Graph({1:{2:28, 6:10}, 2:{3:16, 7:14}, 3:{4:12}, 4:{5:22, 7:18}, 5:
↪{6:25, 7:24}})
sage: G.weighted(True)
sage: list(filter_kruskal_iterator(G, threshold=3, check=True))
[(1, 6, 10), (3, 4, 12), (2, 7, 14), (2, 3, 16), (4, 5, 22), (5, 6, 25)]
```

```
>>> from sage.all import *
>>> from sage.graphs.spanning_tree import filter_kruskal_iterator
>>> G = Graph({Integer(1):{Integer(2):Integer(28), Integer(6):Integer(10)},␣
↪Integer(2):{Integer(3):Integer(16), Integer(7):Integer(14)}, Integer(3):
↪{Integer(4):Integer(12)}, Integer(4):{Integer(5):Integer(22),␣
↪Integer(7):Integer(18)}, Integer(5):{Integer(6):Integer(25),␣
↪Integer(7):Integer(24)}})
>>> G.weighted(True)
>>> list(filter_kruskal_iterator(G, threshold=Integer(3), check=True))
[(1, 6, 10), (3, 4, 12), (2, 7, 14), (2, 3, 16), (4, 5, 22), (5, 6, 25)]
```

The weights of the spanning trees returned by *kruskal_iterator()* and *filter_kruskal_iterator()* are the same:

```
sage: # needs networkx
sage: from sage.graphs.spanning_tree import kruskal_iterator
sage: G = graphs.RandomBarabasiAlbert(50, 2)
sage: for u, v in G.edge_iterator(labels=False):
....:     G.set_edge_label(u, v, randint(1, 10))
sage: G.weighted(True)
sage: sum(e[2] for e in kruskal_iterator(G)) == sum(e[2]
....:     for e in filter_kruskal_iterator(G, threshold=20))
True
```

```
>>> from sage.all import *
>>> # needs networkx
>>> from sage.graphs.spanning_tree import kruskal_iterator
>>> G = graphs.RandomBarabasiAlbert(Integer(50), Integer(2))
>>> for u, v in G.edge_iterator(labels=False):
...     G.set_edge_label(u, v, randint(Integer(1), Integer(10)))
>>> G.weighted(True)
>>> sum(e[Integer(2)] for e in kruskal_iterator(G)) == sum(e[Integer(2)]
...     for e in filter_kruskal_iterator(G, threshold=Integer(20)))
True
```

sage.graphs.spanning_tree.**kruskal**(*G*, *by_weight=True*, *weight_function=None*, *check_weight=False*, *check=False*)

Minimum spanning tree using Kruskal's algorithm.

This function assumes that we can only compute minimum spanning trees for undirected graphs. Such graphs can be weighted or unweighted, and they can have multiple edges (since we are computing the minimum spanning tree, only the minimum weight among all (u, v)-edges is considered, for each pair of vertices u, v).

INPUT:

- `G` – an undirected graph
- `by_weight` – boolean (default: `True`); if `True`, the edges in the graph are weighted. If `False`, all edges have weight 1.
- `weight_function` – function (default: `None`); a function that takes as input an edge `(u, v, l)` and outputs its weight. If not `None`, `by_weight` is automatically set to `True`. If `None` and `by_weight` is `True`, we use the edge label `l`, if `l` is not `None`, else `1` as a weight.
- `check_weight` – boolean (default: `False`); whether to check that the `weight_function` outputs a number for each edge
- `check` – boolean (default: `False`); whether to first perform sanity checks on the input graph `G`. Default: `check=False`. If we toggle `check=True`, the following sanity checks are first performed on `G` prior to running Kruskal's algorithm on that input graph:
  - Is `G` the null graph?
  - Is `G` disconnected?
  - Is `G` a tree?
  - Does `G` have self-loops?
  - Does `G` have multiple edges?

  By default, we turn off the sanity checks for performance reasons. This means that by default the function assumes that its input graph is connected, and has at least one vertex. Otherwise, you should set `check=True`

to perform some sanity checks and preprocessing on the input graph. If G has multiple edges or self-loops, the algorithm still works, but the running-time can be improved if these edges are removed. To further improve the runtime of this function, you should call it directly instead of using it indirectly via *sage.graphs.generic_graph.GenericGraph.min_spanning_tree()*.

OUTPUT:

The edges of a minimum spanning tree of G, if one exists, otherwise returns the empty list.

See also

- *sage.graphs.generic_graph.GenericGraph.min_spanning_tree()*
- *kruskal_iterator()*
- *filter_kruskal()* and *filter_kruskal_iterator()*

EXAMPLES:

An example from pages 727–728 in [Sah2000].

```
sage: from sage.graphs.spanning_tree import kruskal
sage: G = Graph({1:{2:28, 6:10}, 2:{3:16, 7:14}, 3:{4:12}, 4:{5:22, 7:18}, 5:
↪{6:25, 7:24}})
sage: G.weighted(True)
sage: E = kruskal(G, check=True); E
[(1, 6, 10), (3, 4, 12), (2, 7, 14), (2, 3, 16), (4, 5, 22), (5, 6, 25)]
```

```
>>> from sage.all import *
>>> from sage.graphs.spanning_tree import kruskal
>>> G = Graph({Integer(1):{Integer(2):Integer(28), Integer(6):Integer(10)},␣
↪Integer(2):{Integer(3):Integer(16), Integer(7):Integer(14)}, Integer(3):
↪{Integer(4):Integer(12)}, Integer(4):{Integer(5):Integer(22),␣
↪Integer(7):Integer(18)}, Integer(5):{Integer(6):Integer(25),␣
↪Integer(7):Integer(24)}})
>>> G.weighted(True)
>>> E = kruskal(G, check=True); E
[(1, 6, 10), (3, 4, 12), (2, 7, 14), (2, 3, 16), (4, 5, 22), (5, 6, 25)]
```

Variants of the previous example.

```
sage: H = Graph(G.edges(sort=True, labels=False))
sage: kruskal(H, check=True)
[(1, 2, None), (1, 6, None), (2, 3, None), (2, 7, None), (3, 4, None), (4, 5,␣
↪None)]
sage: G.allow_loops(True)
sage: G.allow_multiple_edges(True)
sage: G
Looped multi-graph on 7 vertices
sage: for i in range(20):
....:     u = randint(1, 7)
....:     v = randint(1, 7)
....:     w = randint(0, 20)
....:     G.add_edge(u, v, w)
```

(continues on next page)

(continued from previous page)

```
sage: H = copy(G)
sage: H
Looped multi-graph on 7 vertices
sage: def sanitize(G):
....:     G.allow_loops(False)
....:     G.allow_multiple_edges(False, keep_label='min')
sage: sanitize(H)
sage: H
Graph on 7 vertices
sage: sum(e[2] for e in kruskal(G, check=True)) == sum(e[2] for e in kruskal(H,␣
↪check=True))
True
```

```
>>> from sage.all import *
>>> H = Graph(G.edges(sort=True, labels=False))
>>> kruskal(H, check=True)
[(1, 2, None), (1, 6, None), (2, 3, None), (2, 7, None), (3, 4, None), (4, 5,␣
↪None)]
>>> G.allow_loops(True)
>>> G.allow_multiple_edges(True)
>>> G
Looped multi-graph on 7 vertices
>>> for i in range(Integer(20)):
...     u = randint(Integer(1), Integer(7))
...     v = randint(Integer(1), Integer(7))
...     w = randint(Integer(0), Integer(20))
...     G.add_edge(u, v, w)
>>> H = copy(G)
>>> H
Looped multi-graph on 7 vertices
>>> def sanitize(G):
...     G.allow_loops(False)
...     G.allow_multiple_edges(False, keep_label='min')
>>> sanitize(H)
>>> H
Graph on 7 vertices
>>> sum(e[Integer(2)] for e in kruskal(G, check=True)) == sum(e[Integer(2)] for e␣
↪in kruskal(H, check=True))
True
```

An example from pages 599–601 in [GT2001].

```
sage: G = Graph({"SFO":{"BOS":2704, "ORD":1846, "DFW":1464, "LAX":337},
....: "BOS":{"ORD":867, "JFK":187, "MIA":1258},
....: "ORD":{"PVD":849, "JFK":740, "BWI":621, "DFW":802},
....: "DFW":{"JFK":1391, "MIA":1121, "LAX":1235},
....: "LAX":{"MIA":2342},
....: "PVD":{"JFK":144},
....: "JFK":{"MIA":1090, "BWI":184},
....: "BWI":{"MIA":946}})
sage: G.weighted(True)
sage: kruskal(G, check=True)
```

(continues on next page)

(continued from previous page)

```
[('JFK', 'PVD', 144),
 ('BWI', 'JFK', 184),
 ('BOS', 'JFK', 187),
 ('LAX', 'SFO', 337),
 ('BWI', 'ORD', 621),
 ('DFW', 'ORD', 802),
 ('BWI', 'MIA', 946),
 ('DFW', 'LAX', 1235)]
```

```
>>> from sage.all import *
>>> G = Graph({"SFO":{"BOS":Integer(2704), "ORD":Integer(1846), "DFW
↪":Integer(1464), "LAX":Integer(337)},
... "BOS":{"ORD":Integer(867), "JFK":Integer(187), "MIA":Integer(1258)},
... "ORD":{"PVD":Integer(849), "JFK":Integer(740), "BWI":Integer(621), "DFW
↪":Integer(802)},
... "DFW":{"JFK":Integer(1391), "MIA":Integer(1121), "LAX":Integer(1235)},
... "LAX":{"MIA":Integer(2342)},
... "PVD":{"JFK":Integer(144)},
... "JFK":{"MIA":Integer(1090), "BWI":Integer(184)},
... "BWI":{"MIA":Integer(946)}})
>>> G.weighted(True)
>>> kruskal(G, check=True)
[('JFK', 'PVD', 144),
 ('BWI', 'JFK', 184),
 ('BOS', 'JFK', 187),
 ('LAX', 'SFO', 337),
 ('BWI', 'ORD', 621),
 ('DFW', 'ORD', 802),
 ('BWI', 'MIA', 946),
 ('DFW', 'LAX', 1235)]
```

An example from pages 568–569 in [CLRS2001].

```
sage: G = Graph({"a":{"b":4, "h":8}, "b":{"c":8, "h":11},
....: "c":{"d":7, "f":4, "i":2}, "d":{"e":9, "f":14},
....: "e":{"f":10}, "f":{"g":2}, "g":{"h":1, "i":6}, "h":{"i":7}})
sage: G.weighted(True)
sage: T = Graph(kruskal(G, check=True), format='list_of_edges')
sage: sum(T.edge_labels())
37
sage: T.is_tree()
True
```

```
>>> from sage.all import *
>>> G = Graph({"a":{"b":Integer(4), "h":Integer(8)}, "b":{"c":Integer(8), "h
↪":Integer(11)},
... "c":{"d":Integer(7), "f":Integer(4), "i":Integer(2)}, "d":{"e":Integer(9), "f
↪":Integer(14)},
... "e":{"f":Integer(10)}, "f":{"g":Integer(2)}, "g":{"h":Integer(1), "i
↪":Integer(6)}, "h":{"i":Integer(7)}})
>>> G.weighted(True)
>>> T = Graph(kruskal(G, check=True), format='list_of_edges')
```

(continues on next page)

(continued from previous page)

```
>>> sum(T.edge_labels())
37
>>> T.is_tree()
True
```

An example with custom edge labels:

```
sage: G = Graph([[0,1,1],[1,2,1],[2,0,10]], weighted=True)
sage: weight = lambda e:3-e[0]-e[1]
sage: sorted(kruskal(G, check=True))
[(0, 1, 1), (1, 2, 1)]
sage: sorted(kruskal(G, weight_function=weight, check=True))
[(0, 2, 10), (1, 2, 1)]
sage: sorted(kruskal(G, weight_function=weight, check=False))
[(0, 2, 10), (1, 2, 1)]
```

```
>>> from sage.all import *
>>> G = Graph([[Integer(0),Integer(1),Integer(1)],[Integer(1),Integer(2),
↪Integer(1)],[Integer(2),Integer(0),Integer(10)]], weighted=True)
>>> weight = lambda e:Integer(3)-e[Integer(0)]-e[Integer(1)]
>>> sorted(kruskal(G, check=True))
[(0, 1, 1), (1, 2, 1)]
>>> sorted(kruskal(G, weight_function=weight, check=True))
[(0, 2, 10), (1, 2, 1)]
>>> sorted(kruskal(G, weight_function=weight, check=False))
[(0, 2, 10), (1, 2, 1)]
```

sage.graphs.spanning_tree.**kruskal_iterator**(*G*, *by_weight=True*, *weight_function=None*, *check_weight=False*, *check=False*)

Return an iterator implementation of Kruskal algorithm.

INPUT:

- `G` – an undirected graph
- `by_weight` – boolean (default: `True`); if `True`, the edges in the graph are weighted. If `False`, all edges have weight 1.
- `weight_function` – function (default: `None`); a function that takes as input an edge `(u, v, l)` and outputs its weight. If not `None`, `by_weight` is automatically set to `True`. If `None` and `by_weight` is `True`, we use the edge label `l`, if `l` is not `None`, else `1` as a weight.
- `check_weight` – boolean (default: `False`); whether to check that the `weight_function` outputs a number for each edge
- `check` – boolean (default: `False`); whether to first perform sanity checks on the input graph `G`. Default: `check=False`. If we toggle `check=True`, the following sanity checks are first performed on `G` prior to running Kruskal's algorithm on that input graph:
  - Is `G` the null graph?
  - Is `G` disconnected?
  - Is `G` a tree?
  - Does `G` have self-loops?
  - Does `G` have multiple edges?

By default, we turn off the sanity checks for performance reasons. This means that by default the function assumes that its input graph is connected, and has at least one vertex. Otherwise, you should set `check=True` to perform some sanity checks and preprocessing on the input graph. If `G` has multiple edges or self-loops, the algorithm still works, but the running-time can be improved if these edges are removed. To further improve the runtime of this function, you should call it directly instead of using it indirectly via *sage.graphs.generic_graph.GenericGraph.min_spanning_tree()*.

OUTPUT: the edges of a minimum spanning tree of `G`, one by one

See also

kruskal()

EXAMPLES:

```
sage: from sage.graphs.spanning_tree import kruskal_iterator
sage: G = Graph({1:{2:28, 6:10}, 2:{3:16, 7:14}, 3:{4:12}, 4:{5:22, 7:18}, 5:
↪{6:25, 7:24}})
sage: G.weighted(True)
sage: next(kruskal_iterator(G, check=True))
(1, 6, 10)
```

```
>>> from sage.all import *
>>> from sage.graphs.spanning_tree import kruskal_iterator
>>> G = Graph({Integer(1):{Integer(2):Integer(28), Integer(6):Integer(10)},␣
↪Integer(2):{Integer(3):Integer(16), Integer(7):Integer(14)}, Integer(3):
↪{Integer(4):Integer(12)}, Integer(4):{Integer(5):Integer(22),␣
↪Integer(7):Integer(18)}, Integer(5):{Integer(6):Integer(25),␣
↪Integer(7):Integer(24)}})
>>> G.weighted(True)
>>> next(kruskal_iterator(G, check=True))
(1, 6, 10)
```

sage.graphs.spanning_tree.**kruskal_iterator_from_edges**(*edges*, *union_find*, *by_weight=True*, *weight_function=None*, *check_weight=False*)

Return an iterator implementation of Kruskal algorithm on list of edges.

INPUT:

- `edges` – list of edges
- `union_find` – a `DisjointSet_of_hashables` encoding a forest
- `by_weight` – boolean (default: `True`); if `True`, the edges in the graph are weighted. If `False`, all edges have weight 1.
- `weight_function` – function (default: `None`); a function that takes as input an edge `(u, v, l)` and outputs its weight. If not `None`, `by_weight` is automatically set to `True`. If `None` and `by_weight` is `True`, we use the edge label `l`, if `l` is not `None`, else `1` as a weight.
- `check_weight` – boolean (default: `False`); whether to check that the `weight_function` outputs a number for each edge

OUTPUT: the edges of a minimum spanning tree of `G`, one by one

See also

- *kruskal()*
- *filter_kruskal()*

EXAMPLES:

```
sage: from sage.graphs.spanning_tree import kruskal_iterator_from_edges
sage: G = Graph({1:{2:28, 6:10}, 2:{3:16, 7:14}, 3:{4:12}, 4:{5:22, 7:18}, 5:
↪{6:25, 7:24}})
sage: G.weighted(True)
sage: union_set = DisjointSet(G)
sage: next(kruskal_iterator_from_edges(G.edges(sort=False), union_set, by_
↪weight=G.weighted()))
(1, 6, 10)
```

```
>>> from sage.all import *
>>> from sage.graphs.spanning_tree import kruskal_iterator_from_edges
>>> G = Graph({Integer(1):{Integer(2):Integer(28), Integer(6):Integer(10)},␣
↪Integer(2):{Integer(3):Integer(16), Integer(7):Integer(14)}, Integer(3):
↪{Integer(4):Integer(12)}, Integer(4):{Integer(5):Integer(22),␣
↪Integer(7):Integer(18)}, Integer(5):{Integer(6):Integer(25),␣
↪Integer(7):Integer(24)}})
>>> G.weighted(True)
>>> union_set = DisjointSet(G)
>>> next(kruskal_iterator_from_edges(G.edges(sort=False), union_set, by_weight=G.
↪weighted()))
(1, 6, 10)
```

Check that the method is robust to incomparable vertices:

```
sage: G = Graph([(1, 2, 10), (1, 'a', 1), ('a', 'b', 1), ('b', 2, 1)])
sage: union_set = DisjointSet(G)
sage: E = list(kruskal_iterator_from_edges(G.edges(sort=False), union_set, by_
↪weight=True))
sage: sum(w for _, _, w in E)
3
```

```
>>> from sage.all import *
>>> G = Graph([(Integer(1), Integer(2), Integer(10)), (Integer(1), 'a',␣
↪Integer(1)), ('a', 'b', Integer(1)), ('b', Integer(2), Integer(1))])
>>> union_set = DisjointSet(G)
>>> E = list(kruskal_iterator_from_edges(G.edges(sort=False), union_set, by_
↪weight=True))
>>> sum(w for _, _, w in E)
3
```

sage.graphs.spanning_tree.**random_spanning_tree**(*G*, *output_as_graph=False*, *by_weight=False*, *weight_function=None*, *check_weight=True*)

Return a random spanning tree of the graph.

This uses the Aldous-Broder algorithm ([Bro1989], [Ald1990]) to generate a random spanning tree with the uni-

form distribution, as follows.

Start from any vertex. Perform a random walk by choosing at every step one neighbor uniformly at random. Every time a new vertex j is met, add the edge (i, j) to the spanning tree, where i is the previous vertex in the random walk.

When `by_weight` is `True` or a weight function is given, the selection of the neighbor is done proportionaly to the edge weights.

INPUT:

- `G` – an undirected graph
- `output_as_graph` – boolean (default: `False`); whether to return a list of edges or a graph
- `by_weight` – boolean (default: `False`); if `True`, the edges in the graph are weighted, otherwise all edges have weight 1
- `weight_function` – function (default: `None`); a function that takes as input an edge `(u, v, l)` and outputs its weight. If not `None`, `by_weight` is automatically set to `True`. If `None` and `by_weight` is `True`, we use the edge label `l` , if `l` is not `None`, else `1` as a weight. The `weight_function` can be used to transform the label into a weight (note that, if the weight returned is not convertible to a float, an error is raised)
- `check_weight` – boolean (default: `True`); whether to check that the `weight_function` outputs a number for each edge

See also

number_of_spanning_trees() and *spanning_trees()*

EXAMPLES:

```
sage: G = graphs.TietzeGraph()
sage: G.random_spanning_tree(output_as_graph=True)
Graph on 12 vertices
sage: rg = G.random_spanning_tree(); rg # random
[(0, 9),
(9, 11),
(0, 8),
(8, 7),
(7, 6),
(7, 2),
(2, 1),
(1, 5),
(9, 10),
(5, 4),
(2, 3)]
sage: Graph(rg).is_tree()
True
```

```
>>> from sage.all import *
>>> G = graphs.TietzeGraph()
>>> G.random_spanning_tree(output_as_graph=True)
Graph on 12 vertices
>>> rg = G.random_spanning_tree(); rg # random
```

(continues on next page)

(continued from previous page)

```
[(0, 9),
(9, 11),
(0, 8),
(8, 7),
(7, 6),
(7, 2),
(2, 1),
(1, 5),
(9, 10),
(5, 4),
(2, 3)]
>>> Graph(rg).is_tree()
True
```

A visual example for the grid graph:

```
sage: G = graphs.Grid2dGraph(6, 6)
sage: pos = G.get_pos()
sage: T = G.random_spanning_tree(True)
sage: T.set_pos(pos)
sage: T.show(vertex_labels=False)                                              #␣
↪needs sage.plot
```

```
>>> from sage.all import *
>>> G = graphs.Grid2dGraph(Integer(6), Integer(6))
>>> pos = G.get_pos()
>>> T = G.random_spanning_tree(True)
>>> T.set_pos(pos)
>>> T.show(vertex_labels=False)                                                #␣
↪needs sage.plot
```

We can also use edge weights to change the probability of returning a spanning tree:

```
sage: def foo(G, k):
....:     S = set()
....:     for _ in range(k):
....:         E = G.random_spanning_tree(by_weight=True)
....:         S.add(Graph(E).graph6_string())
....:     return S
sage: K3 = graphs.CompleteGraph(3)
sage: for u, v in K3.edges(sort=True, labels=False):
....:     K3.set_edge_label(u, v, randint(1, 2))
sage: foo(K3, 100) == {'BW', 'Bg', 'Bo'}  # random
True
sage: K4 = graphs.CompleteGraph(4)
sage: for u, v in K4.edges(sort=True, labels=False):
....:     K4.set_edge_label(u, v, randint(1, 2))
sage: print(len(foo(K4, 100)))  # random
16
```

```
>>> from sage.all import *
>>> def foo(G, k):
```

(continues on next page)

(continued from previous page)

```
...       S = set()
...       for _ in range(k):
...           E = G.random_spanning_tree(by_weight=True)
...           S.add(Graph(E).graph6_string())
...       return S
>>> K3 = graphs.CompleteGraph(Integer(3))
>>> for u, v in K3.edges(sort=True, labels=False):
...     K3.set_edge_label(u, v, randint(Integer(1), Integer(2)))
>>> foo(K3, Integer(100)) == {'BW', 'Bg', 'Bo'}  # random
True
>>> K4 = graphs.CompleteGraph(Integer(4))
>>> for u, v in K4.edges(sort=True, labels=False):
...     K4.set_edge_label(u, v, randint(Integer(1), Integer(2)))
>>> print(len(foo(K4, Integer(100))))  # random
16
```

Check that the spanning tree returned when using weights is a tree:

```
sage: # needs networkx
sage: G = graphs.RandomBarabasiAlbert(50, 2)
sage: for u, v in G.edge_iterator(labels=False):
....:     G.set_edge_label(u, v, randint(1, 10))
sage: T = G.random_spanning_tree(by_weight=True, output_as_graph=True)
sage: T.is_tree()
True
```

```
>>> from sage.all import *
>>> # needs networkx
>>> G = graphs.RandomBarabasiAlbert(Integer(50), Integer(2))
>>> for u, v in G.edge_iterator(labels=False):
...     G.set_edge_label(u, v, randint(Integer(1), Integer(10)))
>>> T = G.random_spanning_tree(by_weight=True, output_as_graph=True)
>>> T.is_tree()
True
```

sage.graphs.spanning_tree.**spanning_trees**(*g*, *labels=False*)

Return an iterator over all spanning trees of the graph *g*.

A disconnected graph has no spanning tree.

Uses the Read-Tarjan backtracking algorithm [RT1975a].

INPUT:

- `labels` – boolean (default: `False`); whether to return edges labels in the spanning trees or not

EXAMPLES:

```
sage: G = Graph([(1,2),(1,2),(1,3),(1,3),(2,3),(1,4)], multiedges=True)
sage: len(list(G.spanning_trees()))
8
sage: G.number_of_spanning_trees()                                          #␣
↪needs sage.modules
8
```

(continues on next page)

(continued from previous page)

```
sage: G = Graph([(1,2),(2,3),(3,1),(3,4),(4,5),(4,5),(4,6)], multiedges=True)
sage: len(list(G.spanning_trees()))
6
sage: G.number_of_spanning_trees()                                          #␣
↪needs sage.modules
6
```

```
>>> from sage.all import *
>>> G = Graph([(Integer(1),Integer(2)),(Integer(1),Integer(2)),(Integer(1),
↪Integer(3)),(Integer(1),Integer(3)),(Integer(2),Integer(3)),(Integer(1),
↪Integer(4))], multiedges=True)
>>> len(list(G.spanning_trees()))
8
>>> G.number_of_spanning_trees()                                            #␣
↪needs sage.modules
8
>>> G = Graph([(Integer(1),Integer(2)),(Integer(2),Integer(3)),(Integer(3),
↪Integer(1)),(Integer(3),Integer(4)),(Integer(4),Integer(5)),(Integer(4),
↪Integer(5)),(Integer(4),Integer(6))], multiedges=True)
>>> len(list(G.spanning_trees()))
6
>>> G.number_of_spanning_trees()                                            #␣
↪needs sage.modules
6
```

See also

- *number_of_spanning_trees()* – counts the number of spanning trees
- *random_spanning_tree()* – returns a random spanning tree

5.12 PQ-Trees

This module implements PQ-Trees, a data structure use to represent all permutations of the columns of a matrix which satisfy the *consecutive ones property*:

> A binary matrix satisfies the *consecutive ones property* if the 1s are contiguous in each of its rows (or equivalently, if no row contains the regexp pattern 10^+1).
>
> Alternatively, one can say that a sequence of sets $S_1, ..., S_n$ satisfies the *consecutive ones property* if for any x the indices of the sets containing x is an interval of $[1, n]$.

This module is used for the recognition of Interval Graphs (see *is_interval()*).

P-tree and Q-tree

- A P-tree with children $c_1, ..., c_k$ (which can be P-trees, Q-trees, or actual sets of points) indicates that all $k!$ permutations of the children are allowed.

  Example: $\{1, 2\}, \{3, 4\}, \{5, 6\}$ (disjoint sets can be permuted in any way)

- A Q-tree with children $c_1, ..., c_k$ (which can be P-trees, Q-trees, or actual sets of points) indicates that only two permutations of its children are allowed: $c_1, ..., c_k$ or $c_k, ..., c_1$.

Example: $\{1,2\},\{2,3\},\{3,4\},\{4,5\},\{5,6\}$ (only two permutations of these sets have the *consecutive ones property*).

Computation of all possible orderings

1. In order to compute all permutations of a sequence of sets $S_1, ..., S_k$ satisfying the *consecutive ones property*, we initialize T as a P-tree whose children are all the $S_1, ..., S_k$, thus representing the set of all $k!$ permutations of them.

2. We select some element x and update the data structure T to restrict the permutations it describes to those that keep the occurrences of x on an interval of $[1, ..., k]$. This will result in a new P-tree whose children are:

   - all $\bar{c}_x$ sets S_i which do *not* contain x.
   - a new P-tree whose children are the c_x sets S_i containing x.

   This describes the set of all $c_x! \times \bar{c}'_x!$ permutations of $S_1, ..., S_k$ that keep the sets containing x on an interval.

3. We take a second element x' and update the data structure T to restrict the permutations it describes to those that keep x' on an interval of $[1, ..., k]$. The sets $S_1, ..., S_k$ belong to 4 categories:

   - The family S_{00} of sets which do not contain any of x, x'.
   - The family S_{01} of sets which contain x' but do not contain x.
   - The family S_{10} of sets which contain x but do not contain x'.
   - The family S_{11} of sets which contain x' and x'.

   With these notations, the permutations of $S_1, ..., S_k$ which keep the occurrences of x and x' on an interval are of two forms:

   - <some sets S_{00}>, <sets from S_{10}>, <sets from S_{11}>, <sets from S_{01}>, <other sets from S_{00}>
   - <some sets S_{00}>, <sets from S_{01}>, <sets from S_{11}>, <sets from S_{10}>, <other sets from S_{00}>

   These permutations can be modeled with the following PQ-tree:

   - A P-tree whose children are:
     - All sets from S_{00}
     - A Q-tree whose children are:
       * A P-tree with whose children are the sets from S_{10}
       * A P-tree with whose children are the sets from S_{11}
       * A P-tree with whose children are the sets from S_{01}

4. One at a time, we update the data structure with each element until they are all exhausted, or until we reach a proof that no permutation satisfying the *consecutive ones property* exists.

   Using these two types of tree, and exploring the different cases of intersection, it is possible to represent all the possible permutations of our sets satisfying our constraints, or to prove that no such ordering exists. This is the whole purpose of this module, and is explained with more details in many places, for example in the following document from Hajiaghayi [Haj2000].

Authors:

Nathann Cohen (initial implementation)

5.12.1 Methods and functions

class sage.graphs.pq_trees.**P**(*seq*)

Bases: *PQ*

A P-Tree is a PQ-Tree whose children can be permuted in any way.

For more information, see the documentation of *sage.graphs.pq_trees*.

cardinality()

Return the number of orderings allowed by the structure.

See also

orderings() – iterate over all admissible orderings

EXAMPLES:

```
sage: from sage.graphs.pq_trees import P, Q
sage: p = P([[0,3], [1,2], [2,3], [2,4], [4,0],[2,8], [2,9]])
sage: p.cardinality()
5040
sage: p.set_contiguous(3)
(1, True)
sage: p.cardinality()
1440
```

```
>>> from sage.all import *
>>> from sage.graphs.pq_trees import P, Q
>>> p = P([[Integer(0),Integer(3)], [Integer(1),Integer(2)], [Integer(2),
↪Integer(3)], [Integer(2),Integer(4)], [Integer(4),Integer(0)],[Integer(2),
↪Integer(8)], [Integer(2),Integer(9)]])
>>> p.cardinality()
5040
>>> p.set_contiguous(Integer(3))
(1, True)
>>> p.cardinality()
1440
```

orderings()

Iterate over all orderings of the sets allowed by the structure.

See also

cardinality() – return the number of orderings

EXAMPLES:

```
sage: from sage.graphs.pq_trees import P, Q
sage: p = P([[2,4], [1,2], [0,8], [0,5]])
sage: for o in p.orderings():
....:     print(o)
```

(continues on next page)

(continued from previous page)

```
({2, 4}, {1, 2}, {0, 8}, {0, 5})
({2, 4}, {1, 2}, {0, 5}, {0, 8})
({2, 4}, {0, 8}, {1, 2}, {0, 5})
({2, 4}, {0, 8}, {0, 5}, {1, 2})
...
```

```
>>> from sage.all import *
>>> from sage.graphs.pq_trees import P, Q
>>> p = P([[Integer(2),Integer(4)], [Integer(1),Integer(2)], [Integer(0),
↪Integer(8)], [Integer(0),Integer(5)]])
>>> for o in p.orderings():
...     print(o)
({2, 4}, {1, 2}, {0, 8}, {0, 5})
({2, 4}, {1, 2}, {0, 5}, {0, 8})
({2, 4}, {0, 8}, {1, 2}, {0, 5})
({2, 4}, {0, 8}, {0, 5}, {1, 2})
...
```

set_contiguous(*v*)

Update `self` so that the sets containing `v` are contiguous for any admissible permutation of its subtrees.

INPUT:

- `v` – an element of the ground set

OUTPUT:

According to the cases:

- `(EMPTY, ALIGNED)` if no set of the tree contains an occurrence of `v`
- `(FULL, ALIGNED)` if all the sets of the tree contain `v`
- `(PARTIAL, ALIGNED)` if some (but not all) of the sets contain `v`, all of which are aligned to the right of the ordering at the end when the function ends
- `(PARTIAL, UNALIGNED)` if some (but not all) of the sets contain `v`, though it is impossible to align them all to the right

In any case, the sets containing `v` are contiguous when this function ends. If there is no possibility of doing so, the function raises a `ValueError` exception.

EXAMPLES:

Ensuring the sets containing `0` are continuous:

```
sage: from sage.graphs.pq_trees import P, Q
sage: p = P([[0,3], [1,2], [2,3], [2,4], [4,0],[2,8], [2,9]])
sage: p.set_contiguous(0)
(1, True)
sage: print(p)
('P', [{1, 2}, {2, 3}, {2, 4}, {8, 2}, {9, 2}, ('P', [{0, 3}, {0, 4}])])
```

```
>>> from sage.all import *
>>> from sage.graphs.pq_trees import P, Q
>>> p = P([[Integer(0),Integer(3)], [Integer(1),Integer(2)], [Integer(2),
↪Integer(3)], [Integer(2),Integer(4)], [Integer(4),Integer(0)],[Integer(2),
```

(continues on next page)

(continued from previous page)

```
↪Integer(8)], [Integer(2),Integer(9)]])
>>> p.set_contiguous(Integer(0))
(1, True)
>>> print(p)
('P', [{1, 2}, {2, 3}, {2, 4}, {8, 2}, {9, 2}, ('P', [{0, 3}, {0, 4}])])
```

Impossible situation:

```
sage: p = P([[0,1], [1,2], [2,3], [3,0]])
sage: p.set_contiguous(0)
(1, True)
sage: p.set_contiguous(1)
(1, True)
sage: p.set_contiguous(2)
(1, True)
sage: p.set_contiguous(3)
Traceback (most recent call last):
...
ValueError: Impossible
```

```
>>> from sage.all import *
>>> p = P([[Integer(0),Integer(1)], [Integer(1),Integer(2)], [Integer(2),
↪Integer(3)], [Integer(3),Integer(0)]])
>>> p.set_contiguous(Integer(0))
(1, True)
>>> p.set_contiguous(Integer(1))
(1, True)
>>> p.set_contiguous(Integer(2))
(1, True)
>>> p.set_contiguous(Integer(3))
Traceback (most recent call last):
...
ValueError: Impossible
```

class sage.graphs.pq_trees.**PQ**(*seq*)

Bases: object

PQ-Trees

This class should not be instantiated by itself: it is extended by *P* and *Q*. See the documentation of *sage.graphs.pq_trees* for more information.

AUTHOR : Nathann Cohen

flatten()

Return a flattened copy of self.

If self has only one child, we may as well consider its child's children, as self encodes no information. This method recursively "flattens" trees having only on PQ-tree child, and returns it.

EXAMPLES:

```
sage: from sage.graphs.pq_trees import P, Q
sage: p = Q([P([[2,4], [2,8], [2,9]])])
```

(continues on next page)

(continued from previous page)

```
sage: p.flatten()
('P', [{2, 4}, {8, 2}, {9, 2}])
```

```
>>> from sage.all import *
>>> from sage.graphs.pq_trees import P, Q
>>> p = Q([P([[Integer(2),Integer(4)], [Integer(2),Integer(8)], [Integer(2),
↪Integer(9)]])])
>>> p.flatten()
('P', [{2, 4}, {8, 2}, {9, 2}])
```

number_of_children()

Return the number of children of `self`.

EXAMPLES:

```
sage: from sage.graphs.pq_trees import P, Q
sage: p = Q([[1,2], [2,3], P([[2,4], [2,8], [2,9]])])
sage: p.number_of_children()
3
```

```
>>> from sage.all import *
>>> from sage.graphs.pq_trees import P, Q
>>> p = Q([[Integer(1),Integer(2)], [Integer(2),Integer(3)], P([[Integer(2),
↪Integer(4)], [Integer(2),Integer(8)], [Integer(2),Integer(9)]])])
>>> p.number_of_children()
3
```

ordering()

Return the current ordering given by listing the leaves from left to right.

EXAMPLES:

```
sage: from sage.graphs.pq_trees import P, Q
sage: p = Q([[1,2], [2,3], P([[2,4], [2,8], [2,9]])])
sage: p.ordering()
[{1, 2}, {2, 3}, {2, 4}, {8, 2}, {9, 2}]
```

```
>>> from sage.all import *
>>> from sage.graphs.pq_trees import P, Q
>>> p = Q([[Integer(1),Integer(2)], [Integer(2),Integer(3)], P([[Integer(2),
↪Integer(4)], [Integer(2),Integer(8)], [Integer(2),Integer(9)]])])
>>> p.ordering()
[{1, 2}, {2, 3}, {2, 4}, {8, 2}, {9, 2}]
```

reverse()

Recursively reverse `self` and its children.

EXAMPLES:

```
sage: from sage.graphs.pq_trees import P, Q
sage: p = Q([[1,2], [2,3], P([[2,4], [2,8], [2,9]])])
sage: p.ordering()
```

(continues on next page)

(continued from previous page)

```
[{1, 2}, {2, 3}, {2, 4}, {8, 2}, {9, 2}]
sage: p.reverse()
sage: p.ordering()
[{9, 2}, {8, 2}, {2, 4}, {2, 3}, {1, 2}]
```

```
>>> from sage.all import *
>>> from sage.graphs.pq_trees import P, Q
>>> p = Q([[Integer(1),Integer(2)], [Integer(2),Integer(3)], P([[Integer(2),
↪Integer(4)], [Integer(2),Integer(8)], [Integer(2),Integer(9)]])])
>>> p.ordering()
[{1, 2}, {2, 3}, {2, 4}, {8, 2}, {9, 2}]
>>> p.reverse()
>>> p.ordering()
[{9, 2}, {8, 2}, {2, 4}, {2, 3}, {1, 2}]
```

simplify(*v*, *left=False*, *right=False*)

Return a simplified copy of `self` according to the element `v`.

If `self` is a partial P-tree for `v`, we would like to restrict the permutations of its children to permutations keeping the children containing `v` contiguous. This function also "locks" all the elements not containing `v` inside a P-tree, which is useful when one want to keep the elements containing `v` on one side (which is the case when this method is called).

INPUT:

- `left`, `right` – booleans; whether `v` is aligned to the right or to the left
- `v` – an element of the ground set

OUTPUT:

If `self` is a Q-Tree, the sequence of its children is returned. If `self` is a P-tree, 2 P-tree are returned, namely the two P-tree defined above and restricting the permutations, in the order implied by `left, right` (if `right =True`, the second P-tree will be the one gathering the elements containing `v`, if `left=True`, the opposite).

Note

This method is assumes that `self` is partial for `v`, and aligned to the side indicated by `left, right`.

EXAMPLES:

A P-Tree

```
sage: from sage.graphs.pq_trees import P, Q
sage: p = P([[2,4], [1,2], [0,8], [0,5]])
sage: p.simplify(0, right = True)
[('P', [{2, 4}, {1, 2}]), ('P', [{0, 8}, {0, 5}])]
```

```
>>> from sage.all import *
>>> from sage.graphs.pq_trees import P, Q
>>> p = P([[Integer(2),Integer(4)], [Integer(1),Integer(2)], [Integer(0),
↪Integer(8)], [Integer(0),Integer(5)]])
```

(continues on next page)

(continued from previous page)

```
>>> p.simplify(Integer(0), right = True)
[('P', [{2, 4}, {1, 2}]), ('P', [{0, 8}, {0, 5}])]
```

A Q-Tree

```
sage: q = Q([[2,4], [1,2], [0,8], [0,5]])
sage: q.simplify(0, right = True)
[{2, 4}, {1, 2}, {0, 8}, {0, 5}]
```

```
>>> from sage.all import *
>>> q = Q([[Integer(2),Integer(4)], [Integer(1),Integer(2)], [Integer(0),
↪Integer(8)], [Integer(0),Integer(5)]])
>>> q.simplify(Integer(0), right = True)
[{2, 4}, {1, 2}, {0, 8}, {0, 5}]
```

class sage.graphs.pq_trees.**Q**(*seq*)

Bases: *PQ*

A Q-Tree is a PQ-Tree whose children are ordered up to reversal.

For more information, see the documentation of *sage.graphs.pq_trees*.

cardinality()

Return the number of orderings allowed by the structure.

See also

orderings() – iterate over all admissible orderings

EXAMPLES:

```
sage: from sage.graphs.pq_trees import P, Q
sage: q = Q([[0,3], [1,2], [2,3], [2,4], [4,0],[2,8], [2,9]])
sage: q.cardinality()
2
```

```
>>> from sage.all import *
>>> from sage.graphs.pq_trees import P, Q
>>> q = Q([[Integer(0),Integer(3)], [Integer(1),Integer(2)], [Integer(2),
↪Integer(3)], [Integer(2),Integer(4)], [Integer(4),Integer(0)],[Integer(2),
↪Integer(8)], [Integer(2),Integer(9)]])
>>> q.cardinality()
2
```

orderings()

Iterate over all orderings of the sets allowed by the structure.

See also

cardinality() – return the number of orderings

EXAMPLES:

```
sage: from sage.graphs.pq_trees import P, Q
sage: q = Q([[2,4], [1,2], [0,8], [0,5]])
sage: for o in q.orderings():
....:     print(o)
({2, 4}, {1, 2}, {0, 8}, {0, 5})
({0, 5}, {0, 8}, {1, 2}, {2, 4})
```

```
>>> from sage.all import *
>>> from sage.graphs.pq_trees import P, Q
>>> q = Q([[Integer(2),Integer(4)], [Integer(1),Integer(2)], [Integer(0),
↪Integer(8)], [Integer(0),Integer(5)]])
>>> for o in q.orderings():
...     print(o)
({2, 4}, {1, 2}, {0, 8}, {0, 5})
({0, 5}, {0, 8}, {1, 2}, {2, 4})
```

set_contiguous(*v*)

Update `self` so that the sets containing `v` are contiguous for any admissible permutation of its subtrees.

INPUT:

- `v` – an element of the ground set

OUTPUT:

According to the cases:

- `(EMPTY, ALIGNED)` if no set of the tree contains an occurrence of `v`
- `(FULL, ALIGNED)` if all the sets of the tree contain `v`
- `(PARTIAL, ALIGNED)` if some (but not all) of the sets contain `v`, all of which are aligned to the right of the ordering at the end when the function ends
- `(PARTIAL, UNALIGNED)` if some (but not all) of the sets contain `v`, though it is impossible to align them all to the right

In any case, the sets containing `v` are contiguous when this function ends. If there is no possibility of doing so, the function raises a `ValueError` exception.

EXAMPLES:

Ensuring the sets containing `0` are continuous:

```
sage: from sage.graphs.pq_trees import P, Q
sage: q = Q([[2,3], Q([[3,0],[3,1]]), Q([[4,0],[4,5]])])
sage: q.set_contiguous(0)
(1, False)
sage: print(q)
('Q', [{2, 3}, {1, 3}, {0, 3}, {0, 4}, {4, 5}])
```

```
>>> from sage.all import *
>>> from sage.graphs.pq_trees import P, Q
>>> q = Q([[Integer(2),Integer(3)], Q([[Integer(3),Integer(0)],[Integer(3),
↪Integer(1)]]), Q([[Integer(4),Integer(0)],[Integer(4),Integer(5)]])])
>>> q.set_contiguous(Integer(0))
(1, False)
```

(continues on next page)

(continued from previous page)

```
>>> print(q)
('Q', [{2, 3}, {1, 3}, {0, 3}, {0, 4}, {4, 5}])
```

Impossible situation:

```
sage: p = Q([[0,1], [1,2], [2,0]])
sage: p.set_contiguous(0)
Traceback (most recent call last):
...
ValueError: Impossible
```

```
>>> from sage.all import *
>>> p = Q([[Integer(0),Integer(1)], [Integer(1),Integer(2)], [Integer(2),
↪Integer(0)]])
>>> p.set_contiguous(Integer(0))
Traceback (most recent call last):
...
ValueError: Impossible
```

sage.graphs.pq_trees.**reorder_sets**(*sets*)

Reorders a collection of sets such that each element appears on an interval.

Given a collection of sets $C = S_1, ..., S_k$ on a ground set X, this function attempts to reorder them in such a way that $\forall x \in X$ and $i < j$ with $x \in S_i, S_j$, then $x \in S_l$ for every $i < l < j$ if it exists.

INPUT:

- `sets` – list of instances of `list`, `Set` or `set`

ALGORITHM: PQ-Trees

EXAMPLES:

There is only one way (up to reversal) to represent contiguously the sequence of sets $\{i-1, i, i+1\}$:

```
sage: from sage.graphs.pq_trees import reorder_sets
sage: seq = [Set([i-1,i,i+1]) for i in range(1,15)]
```

```
>>> from sage.all import *
>>> from sage.graphs.pq_trees import reorder_sets
>>> seq = [Set([i-Integer(1),i,i+Integer(1)]) for i in range(Integer(1),
↪Integer(15))]
```

We apply a random permutation:

```
sage: p = Permutations(len(seq)).random_element()
sage: seq = [ seq[p(i+1)-1] for i in range(len(seq)) ]
sage: ordered = reorder_sets(seq)
sage: if not 0 in ordered[0]:
....:     ordered = ordered.reverse()
sage: print(ordered)
[{0, 1, 2}, {1, 2, 3}, {2, 3, 4}, {3, 4, 5}, {4, 5, 6}, {5, 6, 7},
 {8, 6, 7}, {8, 9, 7}, {8, 9, 10}, {9, 10, 11}, {10, 11, 12},
 {11, 12, 13}, {12, 13, 14}, {13, 14, 15}]
```

```
>>> from sage.all import *
>>> p = Permutations(len(seq)).random_element()
>>> seq = [ seq[p(i+Integer(1))-Integer(1)] for i in range(len(seq)) ]
>>> ordered = reorder_sets(seq)
>>> if not Integer(0) in ordered[Integer(0)]:
...     ordered = ordered.reverse()
>>> print(ordered)
[{0, 1, 2}, {1, 2, 3}, {2, 3, 4}, {3, 4, 5}, {4, 5, 6}, {5, 6, 7},
 {8, 6, 7}, {8, 9, 7}, {8, 9, 10}, {9, 10, 11}, {10, 11, 12},
 {11, 12, 13}, {12, 13, 14}, {13, 14, 15}]
```

5.13 Matching

This module implements the functions pertaining to matching of undirected graphs. A *matching* in a graph is a set of pairwise nonadjacent links (nonloop edges). In other words, a matching in a graph is the edge set of an 1-regular subgraph. A matching is called a *perfect matching* if it the subgraph generated by a set of matching edges spans the graph, i.e. it's the edge set of an 1-regular spanning subgraph.

The implemented methods are listed below:

has_perfect_matching()	Return whether the graph has a perfect matching
is_bicritical()	Check if the graph is bicritical
is_factor_critical()	Check whether the graph is factor-critical
is_matching_covered()	Check if the graph is matching covered
matching()	Return a maximum weighted matching of the graph represented by the list of its edges
perfect_matchings()	Return an iterator over all perfect matchings of the graph
M_alternating_even_mark()	Return the vertices reachable from the provided vertex via an even alternating path starting with a non-matching edge

AUTHORS:

- Robert L. Miller (2006-10-22): initial implementations
- Janmenjaya Panda (2024-06-17): added *M_alternating_even_mark()*, *is_bicritical()* and *is_matching_covered()*

5.13.1 Methods

sage.graphs.matching.**M_alternating_even_mark**(*G*, *vertex*, *matching*)

Return the vertices reachable from `vertex` via an even alternating path starting with a non-matching edge.

This method implements the algorithm proposed in [LR2004]. Note that the complexity of the algorithm is linear in number of edges.

INPUT:

- `vertex` – a vertex of the graph
- `matching` – a matching of the graph; it can be given using any valid input format of *Graph*

OUTPUT:

- `even` – the set of vertices each of which is reachable from the provided vertex through a path starting with an edge not in the matching and ending with an edge in the matching; note that a note that a `ValueError` is returned if the graph is not simple

EXAMPLES:

Show the list of required vertices for a graph G with a matching M for a vertex u:

```
sage: G = graphs.CycleGraph(3)
sage: M = G.matching()
sage: M
[(0, 2, None)]
sage: from sage.graphs.matching import M_alternating_even_mark
sage: S0 = M_alternating_even_mark(G, 0, M)
sage: S0
{0}
sage: S1 = M_alternating_even_mark(G, 1, M)
sage: S1
{0, 1, 2}
```

```
>>> from sage.all import *
>>> G = graphs.CycleGraph(Integer(3))
>>> M = G.matching()
>>> M
[(0, 2, None)]
>>> from sage.graphs.matching import M_alternating_even_mark
>>> S0 = M_alternating_even_mark(G, Integer(0), M)
>>> S0
{0}
>>> S1 = M_alternating_even_mark(G, Integer(1), M)
>>> S1
{0, 1, 2}
```

The result is equivalent for the underlying simple graph of the provided graph, if the other parameters provided are the same:

```
sage: G = graphs.CompleteBipartiteGraph(3, 3)
sage: G.allow_multiple_edges(True)
sage: G.add_edge(0, 3)
sage: M = G.matching()
sage: u = 0
sage: from sage.graphs.matching import M_alternating_even_mark
sage: S = M_alternating_even_mark(G, u, M)
sage: S
{0, 1, 2}
sage: T = M_alternating_even_mark(G.to_simple(), u, M)
sage: T
{0, 1, 2}
```

```
>>> from sage.all import *
>>> G = graphs.CompleteBipartiteGraph(Integer(3), Integer(3))
>>> G.allow_multiple_edges(True)
>>> G.add_edge(Integer(0), Integer(3))
>>> M = G.matching()
>>> u = Integer(0)
>>> from sage.graphs.matching import M_alternating_even_mark
>>> S = M_alternating_even_mark(G, u, M)
>>> S
```

(continues on next page)

(continued from previous page)

```
{0, 1, 2}
>>> T = M_alternating_even_mark(G.to_simple(), u, M)
>>> T
{0, 1, 2}
```

For a factor critical graph G (for instance, a wheel graph of an odd order) with a near perfect matching M and u being the (unique) M-exposed vertex, each vertex in G is reachable from u through an even length M-alternating path as described above:

```
sage: G = graphs.WheelGraph(11)
sage: M = Graph(G.matching())
sage: G.is_factor_critical(M)
True
sage: for v in G:
....:     if v not in M:
....:         break
....:
sage: from sage.graphs.matching import M_alternating_even_mark
sage: S = M_alternating_even_mark(G, v, M)
sage: S == set(G)
True
```

```
>>> from sage.all import *
>>> G = graphs.WheelGraph(Integer(11))
>>> M = Graph(G.matching())
>>> G.is_factor_critical(M)
True
>>> for v in G:
...     if v not in M:
...         break
....:
>>> from sage.graphs.matching import M_alternating_even_mark
>>> S = M_alternating_even_mark(G, v, M)
>>> S == set(G)
True
```

For a matching covered graph G (for instance, $K_4 \odot K_{3,3}$) with a perfect matching M and for some vertex u with v being its M-matched neighbor, each neighbor of v is reachable from u through an even length M-alternating path as described above:

```
sage: G = Graph()
sage: G.add_edges([
....:     (0, 2), (0, 3), (0, 4), (1, 2),
....:     (1, 3), (1, 4), (2, 5), (3, 6),
....:     (4, 7), (5, 6), (5, 7), (6, 7)
....: ])
sage: M = Graph(G.matching())
sage: G.is_matching_covered(M)
True
sage: u = 0
sage: v = next(M.neighbor_iterator(u))
sage: from sage.graphs.matching import M_alternating_even_mark
```

(continues on next page)

(continued from previous page)

```
sage: S = M_alternating_even_mark(G, u, M)
sage: (set(G.neighbor_iterator(v))).issubset(S)
True
```

```
>>> from sage.all import *
>>> G = Graph()
>>> G.add_edges([
...     (Integer(0), Integer(2)), (Integer(0), Integer(3)), (Integer(0),␣
↪Integer(4)), (Integer(1), Integer(2)),
...     (Integer(1), Integer(3)), (Integer(1), Integer(4)), (Integer(2),␣
↪Integer(5)), (Integer(3), Integer(6)),
...     (Integer(4), Integer(7)), (Integer(5), Integer(6)), (Integer(5),␣
↪Integer(7)), (Integer(6), Integer(7))
... ])
>>> M = Graph(G.matching())
>>> G.is_matching_covered(M)
True
>>> u = Integer(0)
>>> v = next(M.neighbor_iterator(u))
>>> from sage.graphs.matching import M_alternating_even_mark
>>> S = M_alternating_even_mark(G, u, M)
>>> (set(G.neighbor_iterator(v))).issubset(S)
True
```

For a bicritical graph G (for instance, the Petersen graph) with a perfect matching M and for some vertex u with its M-matched neighbor being v, each vertex of the graph distinct from v is reachable from u through an even length M-alternating path as described above:

```
sage: G = graphs.PetersenGraph()
sage: M = Graph(G.matching())
sage: G.is_bicritical(M)
True
sage: import random
sage: u = random.choice(list(G))                                            #␣
↪needs random
sage: v = next(M.neighbor_iterator(u))
sage: from sage.graphs.matching import M_alternating_even_mark
sage: S = M_alternating_even_mark(G, u, M)
sage: S == (set(G) - {v})
True
```

```
>>> from sage.all import *
>>> G = graphs.PetersenGraph()
>>> M = Graph(G.matching())
>>> G.is_bicritical(M)
True
>>> import random
>>> u = random.choice(list(G))                                          # needs␣
↪random
>>> v = next(M.neighbor_iterator(u))
>>> from sage.graphs.matching import M_alternating_even_mark
>>> S = M_alternating_even_mark(G, u, M)
```

(continues on next page)

(continued from previous page)

```
>>> S == (set(G) - {v})
True
```

REFERENCES:

- [LR2004]

See also

is_factor_critical(), *is_matching_covered()*, *is_bicritical()*

AUTHORS:

- Janmenjaya Panda (2024-06-17)

sage.graphs.matching.**has_perfect_matching**(*G*, *algorithm*, *solver='Edmonds'*, *verbose=None*, *integrality_tolerance=0*)

Return whether the graph has a perfect matching.

INPUT:

- `algorithm` – string (default: `'Edmonds'`)
  - `'Edmonds'` uses Edmonds' algorithm as implemented in NetworkX to find a matching of maximal cardinality, then check whether this cardinality is half the number of vertices of the graph.
  - `'LP_matching'` uses a Linear Program to find a matching of maximal cardinality, then check whether this cardinality is half the number of vertices of the graph.
  - `'LP'` uses a Linear Program formulation of the perfect matching problem: put a binary variable `b[e]` on each edge e, and for each vertex v, require that the sum of the values of the edges incident to v is 1.
- `solver` – string (default: `None`); specifies a Mixed Integer Linear Programming (MILP) solver to be used. If set to `None`, the default one is used. For more information on MILP solvers and which default solver is used, see the method `solve` of the class `MixedIntegerLinearProgram`.
- `verbose` – integer (default: 0); sets the level of verbosity: set to 0 by default, which means quiet (only useful when `algorithm == "LP_matching"` or `algorithm == "LP"`)
- `integrality_tolerance` – float; parameter for use with MILP solvers over an inexact base ring; see `MixedIntegerLinearProgram.get_values()`.

OUTPUT: boolean

EXAMPLES:

```
sage: graphs.PetersenGraph().has_perfect_matching()                    #␣
↪needs networkx
True
sage: graphs.WheelGraph(6).has_perfect_matching()                      #␣
↪needs networkx
True
sage: graphs.WheelGraph(5).has_perfect_matching()                      #␣
↪needs networkx
False
sage: graphs.PetersenGraph().has_perfect_matching(algorithm='LP_matching')  #␣
↪needs sage.numerical.mip
```

(continues on next page)

(continued from previous page)

```
True
sage: graphs.WheelGraph(6).has_perfect_matching(algorithm='LP_matching')     #␣
↪needs sage.numerical.mip
True
sage: graphs.WheelGraph(5).has_perfect_matching(algorithm='LP_matching')
False
sage: graphs.PetersenGraph().has_perfect_matching(algorithm='LP_matching')   #␣
↪needs sage.numerical.mip
True
sage: graphs.WheelGraph(6).has_perfect_matching(algorithm='LP_matching')     #␣
↪needs sage.numerical.mip
True
sage: graphs.WheelGraph(5).has_perfect_matching(algorithm='LP_matching')
False
```

```
>>> from sage.all import *
>>> graphs.PetersenGraph().has_perfect_matching()                            # needs␣
↪networkx
True
>>> graphs.WheelGraph(Integer(6)).has_perfect_matching()                          ␣
↪ # needs networkx
True
>>> graphs.WheelGraph(Integer(5)).has_perfect_matching()                          ␣
↪ # needs networkx
False
>>> graphs.PetersenGraph().has_perfect_matching(algorithm='LP_matching')   # needs␣
↪sage.numerical.mip
True
>>> graphs.WheelGraph(Integer(6)).has_perfect_matching(algorithm='LP_matching')   ␣
↪ # needs sage.numerical.mip
True
>>> graphs.WheelGraph(Integer(5)).has_perfect_matching(algorithm='LP_matching')
False
>>> graphs.PetersenGraph().has_perfect_matching(algorithm='LP_matching')   # needs␣
↪sage.numerical.mip
True
>>> graphs.WheelGraph(Integer(6)).has_perfect_matching(algorithm='LP_matching')   ␣
↪ # needs sage.numerical.mip
True
>>> graphs.WheelGraph(Integer(5)).has_perfect_matching(algorithm='LP_matching')
False
```

sage.graphs.matching.**is_bicritical**(*G*, *matching*, *algorithm=None*, *coNP_certificate='Edmonds'*, *solver=False*, *verbose=None*, *integrality_tolerance=0*)

Check if the graph is bicritical.

A nontrivial graph G is *bicritical* if $G - u - v$ has a perfect matching for any two distinct vertices u and v of G. Bicritical graphs are special kind of matching covered graphs. Each maximal barrier of a bicritical graph is a singleton. Thus, for a bicritical graph, the canonical partition of the vertex set is the set of sets where each set is an individual vertex. Three-connected bicritical graphs, aka *bricks*, play an important role in the theory of matching covered graphs.

This method implements the algorithm proposed in [LZ2001] and we assume that a connected graph of order

two is bicritical, whereas a disconnected graph of the same order is not. The time complexity of the algorithm is $\mathcal{O}(|V| \cdot |E|)$, if a perfect matching of the graph is given, where $|V|$ and $|E|$ are the order and the size of the graph respectively. Otherwise, time complexity may be dominated by the time needed to compute a maximum matching of the graph.

Note that a `ValueError` is returned if the graph has loops or if the graph is trivial, i.e., it has at most one vertex.

INPUT:

- `matching` – (default: `None`); a perfect matching of the graph, that can be given using any valid input format of *Graph*.

  If set to `None`, a matching is computed using the other parameters.

- `algorithm` – string (default: `'Edmonds'`); the algorithm to be used to compute a maximum matching of the graph among

  - `'Edmonds'` selects Edmonds' algorithm as implemented in NetworkX,
  - `'LP'` uses a Linear Program formulation of the matching problem.

- `coNP_certificate` – boolean (default: `False`); if set to `True` a set of pair of vertices (say u and v) is returned such that $G - u - v$ does not have a perfect matching if G is not bicritical or otherwise `None` is returned.

- `solver` – string (default: `None`); specify a Mixed Integer Linear Programming (MILP) solver to be used. If set to `None`, the default one is used. For more information on MILP solvers and which default solver is used, see the method `solve` of the class `MixedIntegerLinearProgram`.

- `verbose` – integer (default: `0`); sets the level of verbosity: set to 0 by default, which means quiet (only useful when `algorithm == 'LP'`).

- `integrality_tolerance` – float; parameter for use with MILP solvers over an inexact base ring; see `MixedIntegerLinearProgram.get_values()`.

OUTPUT:

- A boolean indicating whether the graph is bicritical or not.
- If `coNP_certificate` is set to `True`, a set of pair of vertices is returned in case the graph is not bicritical otherwise `None` is returned.

EXAMPLES:

The Petersen graph is bicritical:

```
sage: G = graphs.PetersenGraph()
sage: G.is_bicritical()
True
```

```
>>> from sage.all import *
>>> G = graphs.PetersenGraph()
>>> G.is_bicritical()
True
```

A graph (without a self-loop) is bicritical if and only if the underlying simple graph is bicritical:

```
sage: G = graphs.PetersenGraph()
sage: G.allow_multiple_edges(True)
sage: G.add_edge(0, 5)
sage: G.is_bicritical()
True
```

```
>>> from sage.all import *
>>> G = graphs.PetersenGraph()
>>> G.allow_multiple_edges(True)
>>> G.add_edge(Integer(0), Integer(5))
>>> G.is_bicritical()
True
```

A nontrivial circular ladder graph whose order is not divisible by 4 is bicritical:

```
sage: G = graphs.CircularLadderGraph(5)
sage: G.is_bicritical()
True
```

```
>>> from sage.all import *
>>> G = graphs.CircularLadderGraph(Integer(5))
>>> G.is_bicritical()
True
```

The graph obtained by splicing two bicritical graph is also bicritical. For instance, K_4 with one extra (multiple) edge (say $G := K_4^+$) is bicritical. Let $H := K_4^+ \odot K_4^+$ such that H is free of multiple edge. The graph H is also bicritical:

```
sage: G = graphs.CompleteGraph(4)
sage: G.allow_multiple_edges(True)
sage: G.add_edge(0, 1)
sage: G.is_bicritical()
True
sage: H = Graph()
sage: H.add_edges([
....:     (0, 1), (0, 2), (0, 3), (0, 4), (1, 2),
....:     (1, 5), (2, 5), (3, 4), (3, 5), (4, 5)
....: ])
sage: H.is_bicritical()
True
```

```
>>> from sage.all import *
>>> G = graphs.CompleteGraph(Integer(4))
>>> G.allow_multiple_edges(True)
>>> G.add_edge(Integer(0), Integer(1))
>>> G.is_bicritical()
True
>>> H = Graph()
>>> H.add_edges([
...     (Integer(0), Integer(1)), (Integer(0), Integer(2)), (Integer(0),␣
↪Integer(3)), (Integer(0), Integer(4)), (Integer(1), Integer(2)),
...     (Integer(1), Integer(5)), (Integer(2), Integer(5)), (Integer(3),␣
↪Integer(4)), (Integer(3), Integer(5)), (Integer(4), Integer(5))
... ])
>>> H.is_bicritical()
True
```

A graph (of order more than two) with more that one component is not bicritical:

```
sage: G = graphs.CycleGraph(4)
sage: G += graphs.CycleGraph(6)
sage: G.number_of_connected_components()
2
sage: G.is_bicritical()
False
```

```
>>> from sage.all import *
>>> G = graphs.CycleGraph(Integer(4))
>>> G += graphs.CycleGraph(Integer(6))
>>> G.number_of_connected_components()
2
>>> G.is_bicritical()
False
```

A graph (of order more than two) with a cut-vertex is not bicritical:

```
sage: G = graphs.CycleGraph(6)
sage: G.add_edges([(5, 6), (5, 7), (6, 7)])
sage: G.is_cut_vertex(5)
True
sage: G.has_perfect_matching()
True
sage: G.is_bicritical()
False
```

```
>>> from sage.all import *
>>> G = graphs.CycleGraph(Integer(6))
>>> G.add_edges([(Integer(5), Integer(6)), (Integer(5), Integer(7)), (Integer(6),␣
↪Integer(7))])
>>> G.is_cut_vertex(Integer(5))
True
>>> G.has_perfect_matching()
True
>>> G.is_bicritical()
False
```

A connected graph of order two is assumed to be bicritical, whereas the disconnected graph of the same order is not:

```
sage: G = graphs.CompleteBipartiteGraph(1, 1)
sage: G.is_bicritical()
True
sage: G = graphs.CompleteBipartiteGraph(2, 0)
sage: G.is_bicritical()
False
```

```
>>> from sage.all import *
>>> G = graphs.CompleteBipartiteGraph(Integer(1), Integer(1))
>>> G.is_bicritical()
True
>>> G = graphs.CompleteBipartiteGraph(Integer(2), Integer(0))
```

(continues on next page)

(continued from previous page)

```
>>> G.is_bicritical()
False
```

A bipartite graph of order three or more is not bicritical:

```
sage: G = graphs.CompleteBipartiteGraph(3, 3)
sage: G.has_perfect_matching()
True
sage: G.is_bicritical()
False
```

```
>>> from sage.all import *
>>> G = graphs.CompleteBipartiteGraph(Integer(3), Integer(3))
>>> G.has_perfect_matching()
True
>>> G.is_bicritical()
False
```

One may specify a matching:

```
sage: G = graphs.WheelGraph(10)
sage: M = G.matching()
sage: G.is_bicritical(matching=M)
True
sage: H = graphs.HexahedralGraph()
sage: N = H.matching()
sage: H.is_bicritical(matching=N)
False
```

```
>>> from sage.all import *
>>> G = graphs.WheelGraph(Integer(10))
>>> M = G.matching()
>>> G.is_bicritical(matching=M)
True
>>> H = graphs.HexahedralGraph()
>>> N = H.matching()
>>> H.is_bicritical(matching=N)
False
```

One may ask for a co-$\mathcal{NP}$ certificate:

```
sage: G = graphs.CompleteGraph(14)
sage: G.is_bicritical(coNP_certificate=True)
(True, None)
sage: H = graphs.CircularLadderGraph(20)
sage: M = H.matching()
sage: H.is_bicritical(matching=M, coNP_certificate=True)
(False, {0, 2})
```

```
>>> from sage.all import *
>>> G = graphs.CompleteGraph(Integer(14))
>>> G.is_bicritical(coNP_certificate=True)
```

(continues on next page)

(continued from previous page)

```
(True, None)
>>> H = graphs.CircularLadderGraph(Integer(20))
>>> M = H.matching()
>>> H.is_bicritical(matching=M, coNP_certificate=True)
(False, {0, 2})
```

REFERENCES:

- [LM2024]
- [LZ2001]

See also

is_factor_critical(), *is_matching_covered()*

AUTHORS:

- Janmenjaya Panda (2024-06-17)

sage.graphs.matching.**is_factor_critical**(*G*, *matching*, *algorithm=None*, *solver='Edmonds'*, *verbose=None*, *integrality_tolerance=0*)

Check whether the graph is factor-critical.

A graph of order n is *factor-critical* if every subgraph of $n-1$ vertices have a perfect matching, hence n must be odd. See Wikipedia article Factor-critical_graph for more details.

This method implements the algorithm proposed in [LR2004] and we assume that a graph of order one is factor-critical. The time complexity of the algorithm is linear if a near perfect matching is given as input (i.e., a matching such that all vertices but one are incident to an edge of the matching). Otherwise, the time complexity is dominated by the time needed to compute a maximum matching of the graph.

INPUT:

- `matching` – (default: `None`); a near perfect matching of the graph, that is a matching such that all vertices of the graph but one are incident to an edge of the matching. It can be given using any valid input format of *Graph*.

  If set to `None`, a matching is computed using the other parameters.

- `algorithm` – string (default: `'Edmonds'`); the algorithm to use to compute a maximum matching of the graph among

  - `'Edmonds'` selects Edmonds' algorithm as implemented in NetworkX
  - `'LP'` uses a Linear Program formulation of the matching problem

- `solver` – string (default: `None`); specifies a Mixed Integer Linear Programming (MILP) solver to be used. If set to `None`, the default one is used. For more information on MILP solvers and which default solver is used, see the method `solve` of the class `MixedIntegerLinearProgram`.

- `verbose` – integer (default: 0); sets the level of verbosity: set to 0 by default, which means quiet (only useful when `algorithm == "LP"`)

- `integrality_tolerance` – float; parameter for use with MILP solvers over an inexact base ring; see `MixedIntegerLinearProgram.get_values()`.

EXAMPLES:

Odd length cycles and odd cliques of order at least 3 are factor-critical graphs:

```
sage: [graphs.CycleGraph(2*i + 1).is_factor_critical() for i in range(5)]   #␣
↪needs networkx
[True, True, True, True, True]
sage: [graphs.CompleteGraph(2*i + 1).is_factor_critical() for i in range(5)]     ␣
↪       # needs networkx
[True, True, True, True, True]
```

```
>>> from sage.all import *
>>> [graphs.CycleGraph(Integer(2)*i + Integer(1)).is_factor_critical() for i in␣
↪range(Integer(5))]   # needs networkx
[True, True, True, True, True]
>>> [graphs.CompleteGraph(Integer(2)*i + Integer(1)).is_factor_critical() for i␣
↪in range(Integer(5))]             # needs networkx
[True, True, True, True, True]
```

More generally, every Hamiltonian graph with an odd number of vertices is factor-critical:

```
sage: G = graphs.RandomGNP(15, .2)
sage: G.add_path([0..14])
sage: G.add_edge(14, 0)
sage: G.is_hamiltonian()
True
sage: G.is_factor_critical()                                                  #␣
↪needs networkx
True
```

```
>>> from sage.all import *
>>> G = graphs.RandomGNP(Integer(15), RealNumber('.2'))
>>> G.add_path((ellipsis_range(Integer(0),Ellipsis,Integer(14))))
>>> G.add_edge(Integer(14), Integer(0))
>>> G.is_hamiltonian()
True
>>> G.is_factor_critical()                                                  # needs␣
↪networkx
True
```

Friendship graphs are non-Hamiltonian factor-critical graphs:

```
sage: all(graphs.FriendshipGraph(i).is_factor_critical() for i in range(1, 5))   ␣
↪         # needs networkx
True
```

```
>>> from sage.all import *
>>> all(graphs.FriendshipGraph(i).is_factor_critical() for i in range(Integer(1),␣
↪Integer(5)))             # needs networkx
True
```

Bipartite graphs are not factor-critical:

```
sage: G = graphs.RandomBipartite(randint(1, 10), randint(1, 10), .5)         #␣
↪needs numpy
sage: G.is_factor_critical()                                                 #␣
```

(continues on next page)

(continued from previous page)

```
↪needs numpy
False
```

```
>>> from sage.all import *
>>> G = graphs.RandomBipartite(randint(Integer(1), Integer(10)),␣
↪randint(Integer(1), Integer(10)), RealNumber('.5'))        # needs numpy
>>> G.is_factor_critical()                                              # needs␣
↪numpy
False
```

Graphs with even order are not factor critical:

```
sage: G = graphs.RandomGNP(10, .5)
sage: G.is_factor_critical()
False
```

```
>>> from sage.all import *
>>> G = graphs.RandomGNP(Integer(10), RealNumber('.5'))
>>> G.is_factor_critical()
False
```

One can specify a matching:

```
sage: F = graphs.FriendshipGraph(4)
sage: M = F.matching()                                                  #␣
↪needs networkx
sage: F.is_factor_critical(matching=M)                                  #␣
↪needs networkx
True
sage: F.is_factor_critical(matching=Graph(M))                           #␣
↪needs networkx
True
```

```
>>> from sage.all import *
>>> F = graphs.FriendshipGraph(Integer(4))
>>> M = F.matching()                                                    # needs␣
↪networkx
>>> F.is_factor_critical(matching=M)                                    # needs␣
↪networkx
True
>>> F.is_factor_critical(matching=Graph(M))                             # needs␣
↪networkx
True
```

sage.graphs.matching.**is_matching_covered**(*G*, *matching*, *algorithm=None*, *coNP_certificate='Edmonds'*, *solver=False*, *verbose=None*, *integrality_tolerance=0*)

Check if the graph is matching covered.

A connected nontrivial graph wherein each edge participates in some perfect matching is called a *matching covered graph*.

If a perfect matching of the graph is provided, for bipartite graph, this method implements a linear time algorithm as proposed in [LM2024] that is based on the following theorem:

Given a connected bipartite graph $G[A, B]$ with a perfect matching M. Construct a directed graph D from G such that $V(D) := V(G)$ and for each edge in G direct the corresponding edge from A to B in D, if it is in M or otherwise direct it from B to A. The graph G is matching covered if and only if D is strongly connected.

For nonbipartite graph, if a perfect matching of the graph is provided, this method implements an $\mathcal{O}(|V| \cdot |E|)$ algorithm, where $|V|$ and $|E|$ are the order and the size of the graph respectively. This implementation is inspired by the $M\text{-}alternating\ tree\ search$ method explained in [LZ2001]. For nonbipartite graph, the implementation is based on the following theorem:

Given a nonbipartite graph G with a perfect matching M. The graph G is matching covered if and only if for each edge uv not in M, there exists an $M\text{-}alternating$ odd length uv-path starting and ending with edges not in M.

The time complexity may be dominated by the time needed to compute a maximum matching of the graph, in case a perfect matching is not provided. Also, note that for a disconnected or a trivial or a graph with a loop, a `ValueError` is returned.

INPUT:

- `matching` – (default: `None`); a perfect matching of the graph, that can be given using any valid input format of *Graph*.

  If set to `None`, a matching is computed using the other parameters.

- `algorithm` – string (default: `'Edmonds'`); the algorithm to be used to compute a maximum matching of the graph among

  - `'Edmonds'` selects Edmonds' algorithm as implemented in NetworkX,
  - `'LP'` uses a Linear Program formulation of the matching problem.

- `coNP_certificate` – boolean (default: `False`); if set to `True` an edge of the graph, that does not participate in any perfect matching, is returned if G is not matching covered or otherwise `None` is returned.

- `solver` – string (default: `None`); specify a Mixed Integer Linear Programming (MILP) solver to be used. If set to `None`, the default one is used. For more information on MILP solvers and which default solver is used, see the method `solve` of the class `MixedIntegerLinearProgram`.

- `verbose` – integer (default: `0`); sets the level of verbosity: set to 0 by default, which means quiet (only useful when `algorithm == 'LP'`).

- `integrality_tolerance` – float; parameter for use with MILP solvers over an inexact base ring; see `MixedIntegerLinearProgram.get_values()`.

OUTPUT:

- A boolean indicating whether the graph is matching covered or not.
- If `coNP_certificate` is set to `True`, an edge is returned in case the graph is not matching covered otherwise `None` is returned.

EXAMPLES:

The Petersen graph is matching covered:

```
sage: G = graphs.PetersenGraph()
sage: G.is_matching_covered()
True
```

```
>>> from sage.all import *
>>> G = graphs.PetersenGraph()
>>> G.is_matching_covered()
True
```

A graph (without a self-loop) is matching covered if and only if the underlying simple graph is matching covered:

```
sage: G = graphs.PetersenGraph()
sage: G.allow_multiple_edges(True)
sage: G.add_edge(0, 5)
sage: G.is_matching_covered()
True
```

```
>>> from sage.all import *
>>> G = graphs.PetersenGraph()
>>> G.allow_multiple_edges(True)
>>> G.add_edge(Integer(0), Integer(5))
>>> G.is_matching_covered()
True
```

A corollary to Tutte's fundamental result [Tut1947], as a strengthening of Petersen's Theorem, states that every 2-connected cubic graph is matching covered:

```
sage: G = Graph()
sage: G.add_edges([
....:     (0, 1), (0, 2), (0, 3),
....:     (1, 2), (1, 4), (2, 4),
....:     (3, 5), (3, 6), (4, 7),
....:     (5, 6), (5, 7), (6, 7)
....: ])
sage: G.vertex_connectivity()
2
sage: degree_sequence = G.degree_sequence()
sage: min(degree_sequence) == max(degree_sequence) == 3
True
sage: G.is_matching_covered()
True
```

```
>>> from sage.all import *
>>> G = Graph()
>>> G.add_edges([
...     (Integer(0), Integer(1)), (Integer(0), Integer(2)), (Integer(0),␣
↪Integer(3)),
...     (Integer(1), Integer(2)), (Integer(1), Integer(4)), (Integer(2),␣
↪Integer(4)),
...     (Integer(3), Integer(5)), (Integer(3), Integer(6)), (Integer(4),␣
↪Integer(7)),
...     (Integer(5), Integer(6)), (Integer(5), Integer(7)), (Integer(6),␣
↪Integer(7))
... ])
>>> G.vertex_connectivity()
2
>>> degree_sequence = G.degree_sequence()
>>> min(degree_sequence) == max(degree_sequence) == Integer(3)
True
>>> G.is_matching_covered()
True
```

A connected bipartite graph $G[A, B]$, with $|A| = |B| \geq 2$, is matching covered if and only if $|N(X)| \geq |X| + 1$,

for all $X \subset A$ such that $1 \leq |X| \leq |A| - 1$. For instance, the Hexahedral graph is matching covered, but not the path graphs on even number of vertices, even though they have a perfect matching:

```
sage: G = graphs.HexahedralGraph()
sage: G.is_bipartite()
True
sage: G.is_matching_covered()
True
sage: P = graphs.PathGraph(10)
sage: P.is_bipartite()
True
sage: M = Graph(P.matching())
sage: set(P) == set(M)
True
sage: P.is_matching_covered()
False
```

```
>>> from sage.all import *
>>> G = graphs.HexahedralGraph()
>>> G.is_bipartite()
True
>>> G.is_matching_covered()
True
>>> P = graphs.PathGraph(Integer(10))
>>> P.is_bipartite()
True
>>> M = Graph(P.matching())
>>> set(P) == set(M)
True
>>> P.is_matching_covered()
False
```

A connected bipartite graph $G[A, B]$ of order six or more is matching covered if and only if $G - a - b$ has a perfect matching for some vertex a in A and some vertex b in B:

```
sage: G = graphs.CircularLadderGraph(8)
sage: G.is_bipartite()
True
sage: G.is_matching_covered()
True
sage: A, B = G.bipartite_sets()
sage: # needs random
sage: import random
sage: a = random.choice(list(A))
sage: b = random.choice(list(B))
sage: G.delete_vertices([a, b])
sage: M = Graph(G.matching())
sage: set(M) == set(G)
True
sage: cycle1 = graphs.CycleGraph(4)
sage: cycle2 = graphs.CycleGraph(6)
sage: cycle2.relabel(lambda v: v + 4)
sage: H = Graph()
```

(continues on next page)

(continued from previous page)

```
sage: H.add_edges(cycle1.edges() + cycle2.edges())
sage: H.add_edge(3, 4)
sage: H.is_bipartite()
True
sage: H.is_matching_covered()
False
sage: H.delete_vertices([3, 4])
sage: N = Graph(H.matching())
sage: set(N) == set(H)
False
```

```
>>> from sage.all import *
>>> G = graphs.CircularLadderGraph(Integer(8))
>>> G.is_bipartite()
True
>>> G.is_matching_covered()
True
>>> A, B = G.bipartite_sets()
>>> # needs random
>>> import random
>>> a = random.choice(list(A))
>>> b = random.choice(list(B))
>>> G.delete_vertices([a, b])
>>> M = Graph(G.matching())
>>> set(M) == set(G)
True
>>> cycle1 = graphs.CycleGraph(Integer(4))
>>> cycle2 = graphs.CycleGraph(Integer(6))
>>> cycle2.relabel(lambda v: v + Integer(4))
>>> H = Graph()
>>> H.add_edges(cycle1.edges() + cycle2.edges())
>>> H.add_edge(Integer(3), Integer(4))
>>> H.is_bipartite()
True
>>> H.is_matching_covered()
False
>>> H.delete_vertices([Integer(3), Integer(4)])
>>> N = Graph(H.matching())
>>> set(N) == set(H)
False
```

One may specify a matching:

```
sage: G = graphs.WheelGraph(20)
sage: M = Graph(G.matching())
sage: G.is_matching_covered(matching=M)
True
sage: J = graphs.CycleGraph(4)
sage: J.add_edge(0, 2)
sage: N = J.matching()
sage: J.is_matching_covered(matching=N)
False
```

```
>>> from sage.all import *
>>> G = graphs.WheelGraph(Integer(20))
>>> M = Graph(G.matching())
>>> G.is_matching_covered(matching=M)
True
>>> J = graphs.CycleGraph(Integer(4))
>>> J.add_edge(Integer(0), Integer(2))
>>> N = J.matching()
>>> J.is_matching_covered(matching=N)
False
```

One may ask for a co-$\mathcal{NP}$ certificate:

```
sage: G = graphs.CompleteGraph(14)
sage: G.is_matching_covered(coNP_certificate=True)
(True, None)
sage: H = graphs.PathGraph(20)
sage: M = H.matching()
sage: H.is_matching_covered(matching=M, coNP_certificate=True)
(False, (2, 1, None))
```

```
>>> from sage.all import *
>>> G = graphs.CompleteGraph(Integer(14))
>>> G.is_matching_covered(coNP_certificate=True)
(True, None)
>>> H = graphs.PathGraph(Integer(20))
>>> M = H.matching()
>>> H.is_matching_covered(matching=M, coNP_certificate=True)
(False, (2, 1, None))
```

REFERENCES:

- [LM2024]
- [LZ2001]
- [Tut1947]

See also

is_factor_critical(), *is_bicritical()*

AUTHORS:

- Janmenjaya Panda (2024-06-23)

sage.graphs.matching.**matching**(*G*, *value_only*, *algorithm=False*, *by_weight='Edmonds'*, *weight_function=False*, *check_weight=None*, *solver=True*, *verbose=None*, *integrality_tolerance=0*)

Return a maximum weighted matching of the graph represented by the list of its edges.

For more information, see the Wikipedia article Matching_(graph_theory).

Given a graph G such that each edge e has a weight w_e, a maximum matching is a subset S of the edges of G of maximum weight such that no two edges of S are incident with each other.

As an optimization problem, it can be expressed as:

$$\text{Maximize : } \sum_{e \in G.edges()} w_e b_e$$

$$\text{Such that : } \forall v \in G, \sum_{(u,v) \in G.edges()} b_{(u,v)} \leq 1$$

$$\forall x \in G, b_x \text{ is a binary variable}$$

INPUT:

- `value_only` – boolean (default: `False`); when set to `True`, only the cardinal (or the weight) of the matching is returned
- `algorithm` – string (default: `'Edmonds'`)
  - `'Edmonds'` selects Edmonds' algorithm as implemented in NetworkX
  - `'LP'` uses a Linear Program formulation of the matching problem
- `by_weight` – boolean (default: `False`); if `True`, computes a weighted matching where each edge has the weight given by `weight_function`, or its label if `weight_function` is `None`. If `False`, each edge has weight 1.
- `weight_function` – function (default: `None`); a function that takes as input an edge `(u, v, l)` and outputs its weight. If not `None`, `by_weight` is automatically set to `True`. If `None` and `by_weight` is `True`, we use the edge label `l`, if `l` is not `None`, else 1 as a weight.
- `check_weight` – boolean (default: `True`); whether to check that the `weight_function` outputs a number for each edge
- `solver` – string (default: `None`); specifies a Mixed Integer Linear Programming (MILP) solver to be used. If set to `None`, the default one is used. For more information on MILP solvers and which default solver is used, see the method `solve` of the class `MixedIntegerLinearProgram`.
- `verbose` – integer (default: 0); sets the level of verbosity: set to 0 by default, which means quiet (only useful when `algorithm == "LP"`)
- `integrality_tolerance` – float; parameter for use with MILP solvers over an inexact base ring; see `MixedIntegerLinearProgram.get_values()`.

OUTPUT:

- When `value_only=False` (default), this method returns an *EdgesView* containing the edges of a maximum matching of G.
- When `value_only=True`, this method returns the sum of the weights (default: `1`) of the edges of a maximum matching of G. The type of the output may vary according to the type of the edge labels and the algorithm used.

ALGORITHM:

The problem is solved using Edmond's algorithm implemented in NetworkX, or using Linear Programming depending on the value of `algorithm`.

EXAMPLES:

Maximum matching in a Pappus Graph:

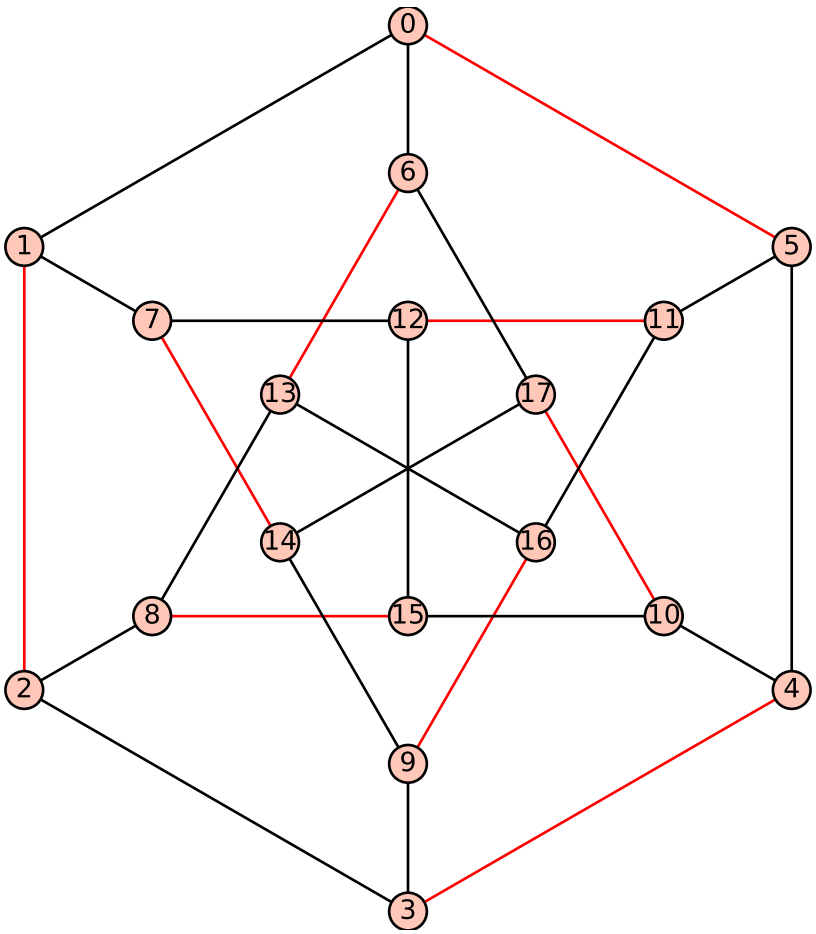
```
sage: g = graphs.PappusGraph()
sage: g.matching(value_only=True)                                          #␣
↪needs sage.networkx
9
```

```
>>> from sage.all import *
>>> g = graphs.PappusGraph()
>>> g.matching(value_only=True)                                                #␣
↪needs sage.networkx
9
```

Same test with the Linear Program formulation:

```
sage: g = graphs.PappusGraph()
sage: g.matching(algorithm='LP', value_only=True)                             #␣
↪needs sage.numerical.mip
9
```

```
>>> from sage.all import *
>>> g = graphs.PappusGraph()
>>> g.matching(algorithm='LP', value_only=True)                               #␣
↪needs sage.numerical.mip
9
```



sage.graphs.matching.**perfect_matchings**(*G*, *labels=False*)

Return an iterator over all perfect matchings of the graph.

ALGORITHM:

Choose a vertex v, then recurse through all edges incident to v, removing one edge at a time whenever an edge is added to a matching.

INPUT:

- `labels` – boolean (default: `False`); when `True`, the edges in each perfect matching are triples (containing the label as the third element), otherwise the edges are pairs.

See also

matching()

EXAMPLES:

```
sage: G=graphs.GridGraph([2,3])
sage: for m in G.perfect_matchings():
....:     print(sorted(m))
[((0, 0), (0, 1)), ((0, 2), (1, 2)), ((1, 0), (1, 1))]
[((0, 0), (1, 0)), ((0, 1), (0, 2)), ((1, 1), (1, 2))]
[((0, 0), (1, 0)), ((0, 1), (1, 1)), ((0, 2), (1, 2))]

sage: G = graphs.CompleteGraph(4)
sage: for m in G.perfect_matchings(labels=True):
....:     print(sorted(m))
[(0, 1, None), (2, 3, None)]
[(0, 2, None), (1, 3, None)]
[(0, 3, None), (1, 2, None)]

sage: G = Graph([[1,-1,'a'], [2,-2, 'b'], [1,-2,'x'], [2,-1,'y']])
sage: sorted(sorted(m) for m in G.perfect_matchings(labels=True))
[[(-2, 1, 'x'), (-1, 2, 'y')], [(-2, 2, 'b'), (-1, 1, 'a')]]

sage: G = graphs.CompleteGraph(8)
sage: mpc = G.matching_polynomial().coefficients(sparse=False)[0]           #␣
↪needs sage.libs.flint
sage: len(list(G.perfect_matchings())) == mpc                               #␣
↪needs sage.libs.flint
True

sage: G = graphs.PetersenGraph().copy(immutable=True)
sage: [sorted(m) for m in G.perfect_matchings()]
[[(0, 1), (2, 3), (4, 9), (5, 7), (6, 8)],
    [(0, 1), (2, 7), (3, 4), (5, 8), (6, 9)],
    [(0, 4), (1, 2), (3, 8), (5, 7), (6, 9)],
    [(0, 4), (1, 6), (2, 3), (5, 8), (7, 9)],
    [(0, 5), (1, 2), (3, 4), (6, 8), (7, 9)],
    [(0, 5), (1, 6), (2, 7), (3, 8), (4, 9)]]

sage: list(Graph().perfect_matchings())
[[]]

sage: G = graphs.CompleteGraph(5)
sage: list(G.perfect_matchings())
[]
```

```
>>> from sage.all import *
>>> G=graphs.GridGraph([Integer(2),Integer(3)])
>>> for m in G.perfect_matchings():
...     print(sorted(m))
[((0, 0), (0, 1)), ((0, 2), (1, 2)), ((1, 0), (1, 1))]
[((0, 0), (1, 0)), ((0, 1), (0, 2)), ((1, 1), (1, 2))]
[((0, 0), (1, 0)), ((0, 1), (1, 1)), ((0, 2), (1, 2))]

>>> G = graphs.CompleteGraph(Integer(4))
>>> for m in G.perfect_matchings(labels=True):
...     print(sorted(m))
[(0, 1, None), (2, 3, None)]
[(0, 2, None), (1, 3, None)]
[(0, 3, None), (1, 2, None)]

>>> G = Graph([[Integer(1),-Integer(1),'a'], [Integer(2),-Integer(2), 'b'],␣
↪[Integer(1),-Integer(2),'x'], [Integer(2),-Integer(1),'y']])
>>> sorted(sorted(m) for m in G.perfect_matchings(labels=True))
[[(-2, 1, 'x'), (-1, 2, 'y')], [(-2, 2, 'b'), (-1, 1, 'a')]]

>>> G = graphs.CompleteGraph(Integer(8))
>>> mpc = G.matching_polynomial().coefficients(sparse=False)[Integer(0)]          ␣
↪ # needs sage.libs.flint
>>> len(list(G.perfect_matchings())) == mpc                                  # needs␣
↪sage.libs.flint
True

>>> G = graphs.PetersenGraph().copy(immutable=True)
>>> [sorted(m) for m in G.perfect_matchings()]
[[(0, 1), (2, 3), (4, 9), (5, 7), (6, 8)],
    [(0, 1), (2, 7), (3, 4), (5, 8), (6, 9)],
    [(0, 4), (1, 2), (3, 8), (5, 7), (6, 9)],
    [(0, 4), (1, 6), (2, 3), (5, 8), (7, 9)],
    [(0, 5), (1, 2), (3, 4), (6, 8), (7, 9)],
    [(0, 5), (1, 6), (2, 7), (3, 8), (4, 9)]]

>>> list(Graph().perfect_matchings())
[[]]

>>> G = graphs.CompleteGraph(Integer(5))
>>> list(G.perfect_matchings())
[]
```

5.14 Matching polynomial

This module contains the following methods:

matching_polynomial()	Compute the matching polynomial of a given graph.
complete_poly()	Compute the matching polynomial of the complete graph on n vertices.

AUTHORS:

- Robert Miller, Tom Boothby - original implementation

REFERENCE:

[God1993]

5.14.1 Methods

sage.graphs.matchpoly.**complete_poly**(n)

Compute the matching polynomial of the complete graph on n vertices.

INPUT:

- n – order of the complete graph

Todo

This code could probably be made more efficient by using FLINT polynomials and being written in Cython, using an array of fmpz_poly_t pointers or something… Right now just about the whole complement optimization is written in Python, and could be easily sped up.

EXAMPLES:

```
sage: from sage.graphs.matchpoly import complete_poly
sage: f = complete_poly(10)
sage: f
x^10 - 45*x^8 + 630*x^6 - 3150*x^4 + 4725*x^2 - 945
sage: f = complete_poly(20)
sage: f[8]
1309458150
sage: f = complete_poly(1000)
sage: len(str(f))
406824
```

```
>>> from sage.all import *
>>> from sage.graphs.matchpoly import complete_poly
>>> f = complete_poly(Integer(10))
>>> f
x^10 - 45*x^8 + 630*x^6 - 3150*x^4 + 4725*x^2 - 945
>>> f = complete_poly(Integer(20))
>>> f[Integer(8)]
1309458150
>>> f = complete_poly(Integer(1000))
>>> len(str(f))
406824
```

sage.graphs.matchpoly.**matching_polynomial**(G, *complement=True*, *name=None*)

Compute the matching polynomial of the graph G.

If $p(G, k)$ denotes the number of k-matchings (matchings with k edges) in G, then the matching polynomial is defined as [God1993]:

$$\mu(x) = \sum_{k \geq 0} (-1)^k p(G, k) x^{n-2k}$$

INPUT:

- `complement` – boolean (default: `True`); whether to use Godsil's duality theorem to compute the matching polynomial from that of the graphs complement (see ALGORITHM)
- `name` – (optional) string for the variable name in the polynomial

Note

The `complement` option uses matching polynomials of complete graphs, which are cached. So if you are crazy enough to try computing the matching polynomial on a graph with millions of vertices, you might not want to use this option, since it will end up caching millions of polynomials of degree in the millions.

ALGORITHM:

The algorithm used is a recursive one, based on the following observation [God1993]:

- If e is an edge of G, G' is the result of deleting the edge e, and G'' is the result of deleting each vertex in e, then the matching polynomial of G is equal to that of G' minus that of G''.

  (the algorithm actually computes the *signless* matching polynomial, for which the recursion is the same when one replaces the subtraction by an addition. It is then converted into the matching polynomial and returned)

Depending on the value of `complement`, Godsil's duality theorem [God1993] can also be used to compute $\mu(x)$:

$$\mu(\overline{G}, x) = \sum_{k \geq 0} p(G, k)\mu(K_{n-2k}, x)$$

Where $\overline{G}$ is the complement of G, and K_n the complete graph on n vertices.

EXAMPLES:

```
sage: g = graphs.PetersenGraph()
sage: g.matching_polynomial()
x^10 - 15*x^8 + 75*x^6 - 145*x^4 + 90*x^2 - 6
sage: g.matching_polynomial(complement=False)
x^10 - 15*x^8 + 75*x^6 - 145*x^4 + 90*x^2 - 6
sage: g.matching_polynomial(name='tom')
tom^10 - 15*tom^8 + 75*tom^6 - 145*tom^4 + 90*tom^2 - 6
sage: g = Graph()
sage: L = [graphs.RandomGNP(8, .3) for i in range(1, 6)]
sage: prod([h.matching_polynomial() for h in L]) == sum(L, g).matching_
↪polynomial()  # long time (up to 10s on sage.math, 2011)
True
```

```
>>> from sage.all import *
>>> g = graphs.PetersenGraph()
>>> g.matching_polynomial()
x^10 - 15*x^8 + 75*x^6 - 145*x^4 + 90*x^2 - 6
>>> g.matching_polynomial(complement=False)
x^10 - 15*x^8 + 75*x^6 - 145*x^4 + 90*x^2 - 6
>>> g.matching_polynomial(name='tom')
tom^10 - 15*tom^8 + 75*tom^6 - 145*tom^4 + 90*tom^2 - 6
>>> g = Graph()
>>> L = [graphs.RandomGNP(Integer(8), RealNumber('.3')) for i in range(Integer(1),
↪ Integer(6))]
>>> prod([h.matching_polynomial() for h in L]) == sum(L, g).matching_polynomial()␣
```

(continues on next page)

(continued from previous page)

```
↪ # long time (up to 10s on sage.math, 2011)
True
```

```
sage: for i in range(1, 12):  # long time (10s on sage.math, 2011)
....:     for t in graphs.trees(i):
....:         if t.matching_polynomial() != t.characteristic_polynomial():
....:             raise RuntimeError('bug for a tree A of size {0}'.format(i))
....:         c = t.complement()
....:         if c.matching_polynomial(complement=False) != c.matching_
↪polynomial():
....:             raise RuntimeError('bug for a tree B of size {0}'.format(i))
```

```
>>> from sage.all import *
>>> for i in range(Integer(1), Integer(12)):  # long time (10s on sage.math, 2011)
...     for t in graphs.trees(i):
...         if t.matching_polynomial() != t.characteristic_polynomial():
...             raise RuntimeError('bug for a tree A of size {0}'.format(i))
...         c = t.complement()
...         if c.matching_polynomial(complement=False) != c.matching_polynomial():
...             raise RuntimeError('bug for a tree B of size {0}'.format(i))
```

```
sage: from sage.graphs.matchpoly import matching_polynomial
sage: matching_polynomial(graphs.CompleteGraph(0))
1
sage: matching_polynomial(graphs.CompleteGraph(1))
x
sage: matching_polynomial(graphs.CompleteGraph(2))
x^2 - 1
sage: matching_polynomial(graphs.CompleteGraph(3))
x^3 - 3*x
sage: matching_polynomial(graphs.CompleteGraph(4))
x^4 - 6*x^2 + 3
sage: matching_polynomial(graphs.CompleteGraph(5))
x^5 - 10*x^3 + 15*x
sage: matching_polynomial(graphs.CompleteGraph(6))
x^6 - 15*x^4 + 45*x^2 - 15
sage: matching_polynomial(graphs.CompleteGraph(7))
x^7 - 21*x^5 + 105*x^3 - 105*x
sage: matching_polynomial(graphs.CompleteGraph(8))
x^8 - 28*x^6 + 210*x^4 - 420*x^2 + 105
sage: matching_polynomial(graphs.CompleteGraph(9))
x^9 - 36*x^7 + 378*x^5 - 1260*x^3 + 945*x
sage: matching_polynomial(graphs.CompleteGraph(10))
x^10 - 45*x^8 + 630*x^6 - 3150*x^4 + 4725*x^2 - 945
sage: matching_polynomial(graphs.CompleteGraph(11))
x^11 - 55*x^9 + 990*x^7 - 6930*x^5 + 17325*x^3 - 10395*x
sage: matching_polynomial(graphs.CompleteGraph(12))
x^12 - 66*x^10 + 1485*x^8 - 13860*x^6 + 51975*x^4 - 62370*x^2 + 10395
sage: matching_polynomial(graphs.CompleteGraph(13))
x^13 - 78*x^11 + 2145*x^9 - 25740*x^7 + 135135*x^5 - 270270*x^3 + 135135*x
```

```
>>> from sage.all import *
>>> from sage.graphs.matchpoly import matching_polynomial
>>> matching_polynomial(graphs.CompleteGraph(Integer(0)))
1
>>> matching_polynomial(graphs.CompleteGraph(Integer(1)))
x
>>> matching_polynomial(graphs.CompleteGraph(Integer(2)))
x^2 - 1
>>> matching_polynomial(graphs.CompleteGraph(Integer(3)))
x^3 - 3*x
>>> matching_polynomial(graphs.CompleteGraph(Integer(4)))
x^4 - 6*x^2 + 3
>>> matching_polynomial(graphs.CompleteGraph(Integer(5)))
x^5 - 10*x^3 + 15*x
>>> matching_polynomial(graphs.CompleteGraph(Integer(6)))
x^6 - 15*x^4 + 45*x^2 - 15
>>> matching_polynomial(graphs.CompleteGraph(Integer(7)))
x^7 - 21*x^5 + 105*x^3 - 105*x
>>> matching_polynomial(graphs.CompleteGraph(Integer(8)))
x^8 - 28*x^6 + 210*x^4 - 420*x^2 + 105
>>> matching_polynomial(graphs.CompleteGraph(Integer(9)))
x^9 - 36*x^7 + 378*x^5 - 1260*x^3 + 945*x
>>> matching_polynomial(graphs.CompleteGraph(Integer(10)))
x^10 - 45*x^8 + 630*x^6 - 3150*x^4 + 4725*x^2 - 945
>>> matching_polynomial(graphs.CompleteGraph(Integer(11)))
x^11 - 55*x^9 + 990*x^7 - 6930*x^5 + 17325*x^3 - 10395*x
>>> matching_polynomial(graphs.CompleteGraph(Integer(12)))
x^12 - 66*x^10 + 1485*x^8 - 13860*x^6 + 51975*x^4 - 62370*x^2 + 10395
>>> matching_polynomial(graphs.CompleteGraph(Integer(13)))
x^13 - 78*x^11 + 2145*x^9 - 25740*x^7 + 135135*x^5 - 270270*x^3 + 135135*x
```

```
sage: G = Graph({0:[1,2], 1:[2]})
sage: matching_polynomial(G)
x^3 - 3*x
sage: G = Graph({0:[1,2]})
sage: matching_polynomial(G)
x^3 - 2*x
sage: G = Graph({0:[1], 2:[]})
sage: matching_polynomial(G)
x^3 - x
sage: G = Graph({0:[], 1:[], 2:[]})
sage: matching_polynomial(G)
x^3
```

```
>>> from sage.all import *
>>> G = Graph({Integer(0):[Integer(1),Integer(2)], Integer(1):[Integer(2)]})
>>> matching_polynomial(G)
x^3 - 3*x
>>> G = Graph({Integer(0):[Integer(1),Integer(2)]})
>>> matching_polynomial(G)
x^3 - 2*x
>>> G = Graph({Integer(0):[Integer(1)], Integer(2):[]})
```

(continues on next page)

(continued from previous page)

```
>>> matching_polynomial(G)
x^3 - x
>>> G = Graph({Integer(0):[], Integer(1):[], Integer(2):[]})
>>> matching_polynomial(G)
x^3
```

```
sage: matching_polynomial(graphs.CompleteGraph(0), complement=False)
1
sage: matching_polynomial(graphs.CompleteGraph(1), complement=False)
x
sage: matching_polynomial(graphs.CompleteGraph(2), complement=False)
x^2 - 1
sage: matching_polynomial(graphs.CompleteGraph(3), complement=False)
x^3 - 3*x
sage: matching_polynomial(graphs.CompleteGraph(4), complement=False)
x^4 - 6*x^2 + 3
sage: matching_polynomial(graphs.CompleteGraph(5), complement=False)
x^5 - 10*x^3 + 15*x
sage: matching_polynomial(graphs.CompleteGraph(6), complement=False)
x^6 - 15*x^4 + 45*x^2 - 15
sage: matching_polynomial(graphs.CompleteGraph(7), complement=False)
x^7 - 21*x^5 + 105*x^3 - 105*x
sage: matching_polynomial(graphs.CompleteGraph(8), complement=False)
x^8 - 28*x^6 + 210*x^4 - 420*x^2 + 105
sage: matching_polynomial(graphs.CompleteGraph(9), complement=False)
x^9 - 36*x^7 + 378*x^5 - 1260*x^3 + 945*x
sage: matching_polynomial(graphs.CompleteGraph(10), complement=False)
x^10 - 45*x^8 + 630*x^6 - 3150*x^4 + 4725*x^2 - 945
sage: matching_polynomial(graphs.CompleteGraph(11), complement=False)
x^11 - 55*x^9 + 990*x^7 - 6930*x^5 + 17325*x^3 - 10395*x
sage: matching_polynomial(graphs.CompleteGraph(12), complement=False)
x^12 - 66*x^10 + 1485*x^8 - 13860*x^6 + 51975*x^4 - 62370*x^2 + 10395
sage: matching_polynomial(graphs.CompleteGraph(13), complement=False)
x^13 - 78*x^11 + 2145*x^9 - 25740*x^7 + 135135*x^5 - 270270*x^3 + 135135*x
```

```
>>> from sage.all import *
>>> matching_polynomial(graphs.CompleteGraph(Integer(0)), complement=False)
1
>>> matching_polynomial(graphs.CompleteGraph(Integer(1)), complement=False)
x
>>> matching_polynomial(graphs.CompleteGraph(Integer(2)), complement=False)
x^2 - 1
>>> matching_polynomial(graphs.CompleteGraph(Integer(3)), complement=False)
x^3 - 3*x
>>> matching_polynomial(graphs.CompleteGraph(Integer(4)), complement=False)
x^4 - 6*x^2 + 3
>>> matching_polynomial(graphs.CompleteGraph(Integer(5)), complement=False)
x^5 - 10*x^3 + 15*x
>>> matching_polynomial(graphs.CompleteGraph(Integer(6)), complement=False)
x^6 - 15*x^4 + 45*x^2 - 15
>>> matching_polynomial(graphs.CompleteGraph(Integer(7)), complement=False)
```

(continues on next page)

(continued from previous page)

```
x^7 - 21*x^5 + 105*x^3 - 105*x
>>> matching_polynomial(graphs.CompleteGraph(Integer(8)), complement=False)
x^8 - 28*x^6 + 210*x^4 - 420*x^2 + 105
>>> matching_polynomial(graphs.CompleteGraph(Integer(9)), complement=False)
x^9 - 36*x^7 + 378*x^5 - 1260*x^3 + 945*x
>>> matching_polynomial(graphs.CompleteGraph(Integer(10)), complement=False)
x^10 - 45*x^8 + 630*x^6 - 3150*x^4 + 4725*x^2 - 945
>>> matching_polynomial(graphs.CompleteGraph(Integer(11)), complement=False)
x^11 - 55*x^9 + 990*x^7 - 6930*x^5 + 17325*x^3 - 10395*x
>>> matching_polynomial(graphs.CompleteGraph(Integer(12)), complement=False)
x^12 - 66*x^10 + 1485*x^8 - 13860*x^6 + 51975*x^4 - 62370*x^2 + 10395
>>> matching_polynomial(graphs.CompleteGraph(Integer(13)), complement=False)
x^13 - 78*x^11 + 2145*x^9 - 25740*x^7 + 135135*x^5 - 270270*x^3 + 135135*x
```

5.15 Morphisms

This module gathers methods related to homeomorphims, homomorphisms, isomorphisms, etc. in (di)graphs.

This module contains the following methods

is_homeomorphic()	Check whether G and H are homeomorphic.
reduced_homeomorphic_graph()	Return the smallest graph homeomorphic to G.
has_homomorphism_to()	Check whether there is a homomorphism between two graphs.

Todo

- Move methods related to graph automorphisms to this module
- Move methods related to graph isomorphisms to this module

5.15.1 Methods

sage.graphs.morphisms.**has_homomorphism_to**(*G*, *H*, *core*, *solver=False*, *verbose=None*, *integrality_tolerance=0*)

Check whether there is a homomorphism between two graphs.

A homomorphism from a graph G to a graph H is a function $\phi : V(G) \mapsto V(H)$ such that for any edge $uv \in E(G)$ the pair $\phi(u)\phi(v)$ is an edge of H.

Saying that a graph can be k-colored is equivalent to saying that it has a homomorphism to K_k, the complete graph of order k.

For more information, see the Wikipedia article Graph_homomorphism.

INPUT:

- G – the graph to map
- H – the graph to which G should be sent
- core – boolean (default: False); whether to minimize the size of the mapping's image (see examples below). This is set to False by default.

- `solver` – string (default: `None`); specifies a Mixed Integer Linear Programming (MILP) solver to be used. If set to `None`, the default one is used. For more information on MILP solvers and which default solver is used, see the method `solve` of the class `MixedIntegerLinearProgram`.
- `verbose` – integer (default: 0); sets the level of verbosity. Set to 0 by default, which means quiet.
- `integrality_tolerance` – float; parameter for use with MILP solvers over an inexact base ring; see `MixedIntegerLinearProgram.get_values()`.

OUTPUT:

This method returns `False` when the homomorphism does not exist, and returns the homomorphism otherwise as a dictionary associating a vertex of H to a vertex of G.

EXAMPLES:

Is Petersen's graph 3-colorable:

```
sage: P = graphs.PetersenGraph()
sage: P.has_homomorphism_to(graphs.CompleteGraph(3)) is not False                #␣
↪needs sage.numerical.mip
True
```

```
>>> from sage.all import *
>>> P = graphs.PetersenGraph()
>>> P.has_homomorphism_to(graphs.CompleteGraph(Integer(3))) is not False          ␣
↪     # needs sage.numerical.mip
True
```

An odd cycle admits a homomorphism to a smaller odd cycle, but not to an even cycle:

```
sage: g = graphs.CycleGraph(9)
sage: g.has_homomorphism_to(graphs.CycleGraph(5)) is not False                   #␣
↪needs sage.numerical.mip
True
sage: g.has_homomorphism_to(graphs.CycleGraph(7)) is not False                   #␣
↪needs sage.numerical.mip
True
sage: g.has_homomorphism_to(graphs.CycleGraph(4)) is not False                   #␣
↪needs sage.numerical.mip
False
```

```
>>> from sage.all import *
>>> g = graphs.CycleGraph(Integer(9))
>>> g.has_homomorphism_to(graphs.CycleGraph(Integer(5))) is not False            ␣
↪     # needs sage.numerical.mip
True
>>> g.has_homomorphism_to(graphs.CycleGraph(Integer(7))) is not False            ␣
↪     # needs sage.numerical.mip
True
>>> g.has_homomorphism_to(graphs.CycleGraph(Integer(4))) is not False            ␣
↪     # needs sage.numerical.mip
False
```

One can compute the core of a graph (with respect to homomorphism) with this method:

```
sage: g = graphs.CycleGraph(8)
sage: mapping = g.has_homomorphism_to(g, core=True)
sage: print(f"The size of the core is {len(set(mapping.values()))}")
The size of the core is 2
sage: g = graphs.CycleGraph(9)
sage: mapping = g.has_homomorphism_to(g, core=True)
sage: print(f"The size of the core is {len(set(mapping.values()))}")
The size of the core is 9
```

```
>>> from sage.all import *
>>> g = graphs.CycleGraph(Integer(8))
>>> mapping = g.has_homomorphism_to(g, core=True)
>>> print(f"The size of the core is {len(set(mapping.values()))}")
The size of the core is 2
>>> g = graphs.CycleGraph(Integer(9))
>>> mapping = g.has_homomorphism_to(g, core=True)
>>> print(f"The size of the core is {len(set(mapping.values()))}")
The size of the core is 9
```

The chromatic number of a graph is the order of the smallest clique to which it has an homomorphism:

```
sage: g = graphs.CycleGraph(9)
sage: g.chromatic_number()
3
sage: g.has_homomorphism_to(graphs.CompleteGraph(3)) is not False
True
sage: g.has_homomorphism_to(graphs.CompleteGraph(2)) is not False
False
sage: K6 = graphs.CompleteGraph(6)
sage: g.has_homomorphism_to(K6) is not False
True
sage: mapping = g.has_homomorphism_to(K6, core=True)
sage: print(f"The size of the core is {len(set(mapping.values()))}")
The size of the core is 3
```

```
>>> from sage.all import *
>>> g = graphs.CycleGraph(Integer(9))
>>> g.chromatic_number()
3
>>> g.has_homomorphism_to(graphs.CompleteGraph(Integer(3))) is not False
True
>>> g.has_homomorphism_to(graphs.CompleteGraph(Integer(2))) is not False
False
>>> K6 = graphs.CompleteGraph(Integer(6))
>>> g.has_homomorphism_to(K6) is not False
True
>>> mapping = g.has_homomorphism_to(K6, core=True)
>>> print(f"The size of the core is {len(set(mapping.values()))}")
The size of the core is 3
```

A circuit of order n admits a homomorphism to smaller circuit of order $p \leq n$ if p is a divisor of n:

```
sage: g = digraphs.Circuit(12)
sage: [i for i in range(2, g.order() + 1)                                    #␣
↪needs sage.numerical.mip
....:  if g.has_homomorphism_to(digraphs.Circuit(i)) is not False]
[2, 3, 4, 6, 12]
```

```
>>> from sage.all import *
>>> g = digraphs.Circuit(Integer(12))
>>> [i for i in range(Integer(2), g.order() + Integer(1))                    ␣
↪              # needs sage.numerical.mip
...  if g.has_homomorphism_to(digraphs.Circuit(i)) is not False]
[2, 3, 4, 6, 12]
```

sage.graphs.morphisms.**is_homeomorphic**(G, H)

Check whether `G` and `H` are homeomorphic.

Two graphs G and H are homeomorphic if there is an isomorphism from some subdivision of G to some subdivision of H. To check whether G and H are homeomorphic, it suffices to check whether their reduced homeomorphic (di)graphs are isomorphic. For more details, see the Wikipedia article Homeomorphism_(graph_theory).

INPUT:

- `G`, `H` – two (di)graphs

EXAMPLES:

```
sage: G = graphs.RandomGNP(10, .2)
sage: H = G.copy()
sage: for e in list(G.edges()):
....:     G.subdivide_edge(e, randint(0, 5))
....:     H.subdivide_edge(e, randint(0, 5))
sage: G.is_homeomorphic(H)
True
sage: G = graphs.RandomGNP(10, .2)
sage: G.allow_multiple_edges(True)
sage: G.add_edges(G.edges())
sage: H = G.copy()
sage: for e in list(G.edges()):
....:     G.subdivide_edge(e, randint(0, 5))
....:     H.subdivide_edge(e, randint(0, 5))
sage: G.is_homeomorphic(H)
True

sage: G = digraphs.RandomDirectedGNP(10, .2)
sage: H = G.copy()
sage: for e in list(G.edges()):
....:     G.subdivide_edge(e, randint(0, 5))
....:     H.subdivide_edge(e, randint(0, 5))
sage: G.is_homeomorphic(H)
True
sage: G = digraphs.RandomDirectedGNP(10, .2)
sage: G.allow_multiple_edges(True)
sage: G.add_edges(G.edges())
sage: H = G.copy()
```

(continues on next page)

(continued from previous page)

```
sage: for e in list(G.edges()):
....:     G.subdivide_edge(e, randint(0, 5))
....:     H.subdivide_edge(e, randint(0, 5))
sage: G.is_homeomorphic(H)
True

sage: G = digraphs.RandomDirectedGNP(10, .2)
sage: G.allow_loops(True)
sage: G.add_edges((u, u) for u in G if randint(0, 1))
sage: G.allow_multiple_edges(True)
sage: G.add_edges(G.edges())
sage: H = G.copy()
sage: for e in list(G.edges()):
....:     G.subdivide_edge(e, randint(0, 5))
....:     H.subdivide_edge(e, randint(0, 5))
sage: G.is_homeomorphic(H)
True
```

```
>>> from sage.all import *
>>> G = graphs.RandomGNP(Integer(10), RealNumber('.2'))
>>> H = G.copy()
>>> for e in list(G.edges()):
...     G.subdivide_edge(e, randint(Integer(0), Integer(5)))
...     H.subdivide_edge(e, randint(Integer(0), Integer(5)))
>>> G.is_homeomorphic(H)
True
>>> G = graphs.RandomGNP(Integer(10), RealNumber('.2'))
>>> G.allow_multiple_edges(True)
>>> G.add_edges(G.edges())
>>> H = G.copy()
>>> for e in list(G.edges()):
...     G.subdivide_edge(e, randint(Integer(0), Integer(5)))
...     H.subdivide_edge(e, randint(Integer(0), Integer(5)))
>>> G.is_homeomorphic(H)
True

>>> G = digraphs.RandomDirectedGNP(Integer(10), RealNumber('.2'))
>>> H = G.copy()
>>> for e in list(G.edges()):
...     G.subdivide_edge(e, randint(Integer(0), Integer(5)))
...     H.subdivide_edge(e, randint(Integer(0), Integer(5)))
>>> G.is_homeomorphic(H)
True
>>> G = digraphs.RandomDirectedGNP(Integer(10), RealNumber('.2'))
>>> G.allow_multiple_edges(True)
>>> G.add_edges(G.edges())
>>> H = G.copy()
>>> for e in list(G.edges()):
...     G.subdivide_edge(e, randint(Integer(0), Integer(5)))
...     H.subdivide_edge(e, randint(Integer(0), Integer(5)))
>>> G.is_homeomorphic(H)
True
```

(continues on next page)

(continued from previous page)

```
>>> G = digraphs.RandomDirectedGNP(Integer(10), RealNumber('.2'))
>>> G.allow_loops(True)
>>> G.add_edges((u, u) for u in G if randint(Integer(0), Integer(1)))
>>> G.allow_multiple_edges(True)
>>> G.add_edges(G.edges())
>>> H = G.copy()
>>> for e in list(G.edges()):
...     G.subdivide_edge(e, randint(Integer(0), Integer(5)))
...     H.subdivide_edge(e, randint(Integer(0), Integer(5)))
>>> G.is_homeomorphic(H)
True
```

sage.graphs.morphisms.**reduced_homeomorphic_graph**(*G*, *allow_multiple_edges=False*, *allow_loops=False*, *return_steps=False*, *immutable=None*)

Return the smallest graph homeomorphic to G.

Two graphs G and H are homeomorphic if there is an isomorphism from some subdivision of G to some subdivision of H. For more details, see the Wikipedia article Homeomorphism_(graph_theory).

By default (i.e., when `allow_multiple_edges == False` and `allow_loops == False`), given a graph G, a vertex u of degree two and its neighbors x and y, with $x \neq y$, this methods replaces the path (x, u, y) with the edge (x, y) unless the graph already has edge (x, y). This process is repeated for each vertex of degree two. The resulting graph H is the smallest graph that is homeomorphic to G.

When `allow_multiple_edges == True` and `allow_loops == False`, this method always replaces the path (x, u, y) with a new edge (x, y). Hence, the resulting graph may have several edges between x and y. This operation is performed only if $x \neq y$.

When `allow_loops == True`, this method also assumes that `allow_multiple_edges == True`. If a vertex u of degree two is connected by two edges to a vertex x, this method replaces the two edges by a loop edge on x.

For digraphs, the method considers the vertices with in and out degree one.

INPUT:

- `G` – a graph or a digraph
- `allow_multiple_edges` – boolean (default: `False`); whether to allow the creation of new multiple edges. This parameter is considered `True` when `allow_loops` is `True`.
- `allow_loops` – boolean (default: `False`); whether to allow the creation of new loops
- `return_steps` – boolean (default: `False`); whether to return the steps of the reduction as a list of triples (x, u, y) indicating that path (x, u, y) has been replaced by edge (x, y). The original graph can be reconstructed by using this list in reverse order.
- `immutable` – boolean (default: `None`); whether to create a mutable/immutable (di)graph. `immutable=None` (default) means that the (di)graph and its reduced (di)graph will behave the same way.

OUTPUT: When `return_steps` is `False`, this method returns the reduced graph. When `return_steps` is `True`, this method returns both the reduced graph and the ordered list of reduction operations. Each reduction operation is a triple (x, u, y) indicating that the path (x, u, y), with u of degree two (or with in and out degree one for digraphs), has been replaced by edge (x, y).

EXAMPLES:

Reduction of a Cycle Graph:

```
sage: G = graphs.CycleGraph(4)
sage: G.reduced_homeomorphic_graph()
Graph on 3 vertices
sage: G.reduced_homeomorphic_graph(allow_multiple_edges=True)
Multi-graph on 2 vertices
sage: G.reduced_homeomorphic_graph(allow_loops=True)
Looped multi-graph on 1 vertex
```

```
>>> from sage.all import *
>>> G = graphs.CycleGraph(Integer(4))
>>> G.reduced_homeomorphic_graph()
Graph on 3 vertices
>>> G.reduced_homeomorphic_graph(allow_multiple_edges=True)
Multi-graph on 2 vertices
>>> G.reduced_homeomorphic_graph(allow_loops=True)
Looped multi-graph on 1 vertex
```

Reduction of a Circuit:

```
sage: G = digraphs.Circuit(4)
sage: G.reduced_homeomorphic_graph()
Digraph on 2 vertices
sage: G.reduced_homeomorphic_graph(allow_multiple_edges=True)
Multi-digraph on 2 vertices
sage: G.reduced_homeomorphic_graph(allow_loops=True)
Looped multi-digraph on 1 vertex
```

```
>>> from sage.all import *
>>> G = digraphs.Circuit(Integer(4))
>>> G.reduced_homeomorphic_graph()
Digraph on 2 vertices
>>> G.reduced_homeomorphic_graph(allow_multiple_edges=True)
Multi-digraph on 2 vertices
>>> G.reduced_homeomorphic_graph(allow_loops=True)
Looped multi-digraph on 1 vertex
```

Check that the construction is reversible:

```
sage: def revert_steps(g, steps):
....:     h = g.copy(immutable=False)
....:     for P in reversed(steps):
....:         h.add_path(P)
....:         h.delete_edge(P[0], P[2])
....:     return h

sage: G = graphs.WindmillGraph(3, 5)
sage: G.order(), G.size()
(11, 15)
sage: H, steps = G.reduced_homeomorphic_graph(return_steps=True)
sage: H.order(), H.size()
(11, 15)
sage: G.is_isomorphic(revert_steps(H, steps))
True
```

(continues on next page)

(continued from previous page)

```
sage: H, steps = G.reduced_homeomorphic_graph(allow_multiple_edges=True, return_
↪steps=True)
sage: H.order(), H.size()
(6, 10)
sage: G.is_isomorphic(revert_steps(H, steps))
True
sage: H, steps = G.reduced_homeomorphic_graph(allow_loops=True, return_steps=True)
sage: H.order(), H.size()
(1, 5)
sage: len(H.loop_edges())
5
sage: G.is_isomorphic(revert_steps(H, steps))
True
```

```
>>> from sage.all import *
>>> def revert_steps(g, steps):
...     h = g.copy(immutable=False)
...     for P in reversed(steps):
...         h.add_path(P)
...         h.delete_edge(P[Integer(0)], P[Integer(2)])
...     return h

>>> G = graphs.WindmillGraph(Integer(3), Integer(5))
>>> G.order(), G.size()
(11, 15)
>>> H, steps = G.reduced_homeomorphic_graph(return_steps=True)
>>> H.order(), H.size()
(11, 15)
>>> G.is_isomorphic(revert_steps(H, steps))
True
>>> H, steps = G.reduced_homeomorphic_graph(allow_multiple_edges=True, return_
↪steps=True)
>>> H.order(), H.size()
(6, 10)
>>> G.is_isomorphic(revert_steps(H, steps))
True
>>> H, steps = G.reduced_homeomorphic_graph(allow_loops=True, return_steps=True)
>>> H.order(), H.size()
(1, 5)
>>> len(H.loop_edges())
5
>>> G.is_isomorphic(revert_steps(H, steps))
True
```

Random digraph:

```
sage: G = digraphs.RandomDirectedGNP(20, 0.05, loops=True)
sage: H, steps = G.reduced_homeomorphic_graph(return_steps=True)
sage: G.is_isomorphic(revert_steps(H, steps))
True
sage: H, steps = G.reduced_homeomorphic_graph(allow_multiple_edges=True, return_
↪steps=True)
```

(continues on next page)

(continued from previous page)

```
sage: G.is_isomorphic(revert_steps(H, steps))
True
sage: H, steps = G.reduced_homeomorphic_graph(allow_loops=True, return_steps=True)
sage: G.is_isomorphic(revert_steps(H, steps))
True
```

```
>>> from sage.all import *
>>> G = digraphs.RandomDirectedGNP(Integer(20), RealNumber('0.05'), loops=True)
>>> H, steps = G.reduced_homeomorphic_graph(return_steps=True)
>>> G.is_isomorphic(revert_steps(H, steps))
True
>>> H, steps = G.reduced_homeomorphic_graph(allow_multiple_edges=True, return_
↪steps=True)
>>> G.is_isomorphic(revert_steps(H, steps))
True
>>> H, steps = G.reduced_homeomorphic_graph(allow_loops=True, return_steps=True)
>>> G.is_isomorphic(revert_steps(H, steps))
True
```

5.16 Graph genus algorithms

This module integrates algorithms for the orientable genus of a simple connected graph. The built-in `'simple'` algorithm is always available. The optional graph_genus package provides the `'page'` and `'multi_genus'` algorithms.

The simple algorithm works by enumerating combinatorial embeddings of a graph, and computing the genus of these via the Euler characteristic. We view a combinatorial embedding of a graph as a pair of permutations v, e which act on a set B of $2|E(G)|$ "darts". The permutation e is an involution, and its orbits correspond to edges in the graph. Similarly, The orbits of v correspond to the vertices of the graph, and those of $f = ve$ correspond to faces of the embedded graph. The requirement that the group $< v, e >$ acts transitively on B is equivalent to the graph being connected. We can compute the genus of a graph by $2 - 2g = V - E + F$ where E, V, and F denote the number of orbits of e, v, and f respectively.

AUTHORS:

- Tom Boothby (2010): original simple backtracking algorithm
- Gunnar Brinkmann (2022): MultiGenus algorithm
- Alexander Metzger and Austin Ulrigg (2026): PAGE algorithm
- Alexander Metzger (2026): Sage integration of PAGE and MultiGenus

class sage.graphs.genus.**simple_connected_genus_backtracker**

Bases: object

A class which computes the genus of a DenseGraph through an extremely slow but relatively optimized algorithm. This is "only" exponential for graphs of bounded degree, and feels pretty snappy for 3-regular graphs. The generic runtime is

$$|V(G)| \prod_{v \in V(G)} (deg(v) - 1)!$$

which is $2^{|V(G)|}$ for 3-regular graphs, and can achieve $n(n-1)!^n$ for the complete graph on n vertices. We can handily compute the genus of K_6 in milliseconds on modern hardware, but K_7 may take a few days. Don't bother with K_8, or any graph with more than one vertex of degree 10 or worse, unless you can find an a priori lower bound on the genus and expect the graph to have that genus.

Warning

THIS MAY SEGFAULT OR HANG ON:

- DISCONNECTED GRAPHS
- DIRECTED GRAPHS
- LOOPED GRAPHS
- MULTIGRAPHS

EXAMPLES:

```
sage: import sage.graphs.genus
sage: G = graphs.CompleteGraph(6)
sage: G = Graph(G, sparse=False)
sage: bt = sage.graphs.genus.simple_connected_genus_backtracker(G._backend.c_
↪graph()[0])
sage: bt.genus() #long time
1
sage: bt.genus(cutoff=1)
1
sage: G = graphs.PetersenGraph()
sage: G = Graph(G, sparse=False)
sage: bt = sage.graphs.genus.simple_connected_genus_backtracker(G._backend.c_
↪graph()[0])
sage: bt.genus()
1
sage: G = graphs.FlowerSnark()
sage: G = Graph(G, sparse=False)
sage: bt = sage.graphs.genus.simple_connected_genus_backtracker(G._backend.c_
↪graph()[0])
sage: bt.genus()
2
```

```
>>> from sage.all import *
>>> import sage.graphs.genus
>>> G = graphs.CompleteGraph(Integer(6))
>>> G = Graph(G, sparse=False)
>>> bt = sage.graphs.genus.simple_connected_genus_backtracker(G._backend.c_
↪graph()[Integer(0)])
>>> bt.genus() #long time
1
>>> bt.genus(cutoff=Integer(1))
1
>>> G = graphs.PetersenGraph()
>>> G = Graph(G, sparse=False)
>>> bt = sage.graphs.genus.simple_connected_genus_backtracker(G._backend.c_
↪graph()[Integer(0)])
>>> bt.genus()
1
>>> G = graphs.FlowerSnark()
>>> G = Graph(G, sparse=False)
```

(continues on next page)

(continued from previous page)

```
>>> bt = sage.graphs.genus.simple_connected_genus_backtracker(G._backend.c_
↪graph()[Integer(0)])
>>> bt.genus()
2
```

genus (*style=1*, *cutoff=0*, *record_embedding=False*)

Compute the minimal or maximal genus of `self`'s graph.

Note, this is a remarkably naive algorithm for a very difficult problem. Most interesting cases will take millennia to finish, with the exception of graphs with max degree 3.

INPUT:

- `style` – integer (default: 1); find minimum genus if 1, maximum genus if 2
- `cutoff` – integer (default: 0); stop searching if search style is 1 and `genus` $\leq$ `cutoff`, or if style is 2 and `genus` $\geq$ `cutoff`. This is useful where the genus of the graph has a known bound.
- `record_embedding` – boolean (default: `False`); whether or not to remember the best embedding seen. This embedding can be retrieved with `self.get_embedding()`.

OUTPUT: the minimal or maximal genus for `self`'s graph

EXAMPLES:

```
sage: import sage.graphs.genus
sage: G = Graph(graphs.CompleteGraph(5), sparse=False)
sage: gb = sage.graphs.genus.simple_connected_genus_backtracker(G._backend.c_
↪graph()[0])
sage: gb.genus(cutoff=2, record_embedding=True)
2
sage: E = gb.get_embedding()
sage: gb.genus(record_embedding=False)
1
sage: gb.get_embedding() == E
True
sage: gb.genus(style=2, cutoff=5)
3
sage: G = Graph(sparse=False)
sage: gb = sage.graphs.genus.simple_connected_genus_backtracker(G._backend.c_
↪graph()[0])
sage: gb.genus()
0
```

```
>>> from sage.all import *
>>> import sage.graphs.genus
>>> G = Graph(graphs.CompleteGraph(Integer(5)), sparse=False)
>>> gb = sage.graphs.genus.simple_connected_genus_backtracker(G._backend.c_
↪graph()[Integer(0)])
>>> gb.genus(cutoff=Integer(2), record_embedding=True)
2
>>> E = gb.get_embedding()
>>> gb.genus(record_embedding=False)
1
>>> gb.get_embedding() == E
```

(continues on next page)

(continued from previous page)

```
True
>>> gb.genus(style=Integer(2), cutoff=Integer(5))
3
>>> G = Graph(sparse=False)
>>> gb = sage.graphs.genus.simple_connected_genus_backtracker(G._backend.c_
↪graph()[Integer(0)])
>>> gb.genus()
0
```

get_embedding()

Return an embedding for the graph.

If `min_genus_backtrack` has been called with `record_embedding = True`, then this will return the first minimal embedding that we found. Otherwise, this returns the first embedding considered.

EXAMPLES:

```
sage: import sage.graphs.genus
sage: G = Graph(graphs.CompleteGraph(5), sparse=False)
sage: gb = sage.graphs.genus.simple_connected_genus_backtracker(G._backend.c_
↪graph()[0])
sage: gb.genus(record_embedding=True)
1
sage: gb.get_embedding()
{0: [1, 2, 3, 4], 1: [0, 2, 3, 4], 2: [0, 1, 4, 3], 3: [0, 2, 1, 4], 4: [0, 3,
↪ 1, 2]}
sage: G = Graph(sparse=False)
sage: G.add_edge(0,1)
sage: gb = sage.graphs.genus.simple_connected_genus_backtracker(G._backend.c_
↪graph()[0])
sage: gb.get_embedding()
{0: [1], 1: [0]}
sage: G = Graph(sparse=False)
sage: gb = sage.graphs.genus.simple_connected_genus_backtracker(G._backend.c_
↪graph()[0])
sage: gb.get_embedding()
{}
```

```
>>> from sage.all import *
>>> import sage.graphs.genus
>>> G = Graph(graphs.CompleteGraph(Integer(5)), sparse=False)
>>> gb = sage.graphs.genus.simple_connected_genus_backtracker(G._backend.c_
↪graph()[Integer(0)])
>>> gb.genus(record_embedding=True)
1
>>> gb.get_embedding()
{0: [1, 2, 3, 4], 1: [0, 2, 3, 4], 2: [0, 1, 4, 3], 3: [0, 2, 1, 4], 4: [0, 3,
↪ 1, 2]}
>>> G = Graph(sparse=False)
>>> G.add_edge(Integer(0),Integer(1))
>>> gb = sage.graphs.genus.simple_connected_genus_backtracker(G._backend.c_
↪graph()[Integer(0)])
>>> gb.get_embedding()
```

(continues on next page)

(continued from previous page)

```
{0: [1], 1: [0]}
>>> G = Graph(sparse=False)
>>> gb = sage.graphs.genus.simple_connected_genus_backtracker(G._backend.c_
↪graph()[Integer(0)])
>>> gb.get_embedding()
{}
```

sage.graphs.genus.**simple_connected_graph_genus**(*G*, *set_embedding=False*, *check=True*, *minimal=True*, *algorithm=None*)

Compute the genus of a simple connected graph.

INPUT:

- `G` – a simple connected graph
- `set_embedding` – boolean (default: `False`); whether to store a minimum-genus combinatorial embedding on `G`
- `check` – boolean (default: `True`); whether to validate and simplify the graph before the computation
- `minimal` – boolean (default: `True`); whether to compute minimum genus. If `False`, only `algorithm='simple'` is supported.
- `algorithm` – string or `None` (default: `None`); one of `'page'`, `'multi_genus'`, or `'simple'`. If `None`, Sage uses `'page'` when the optional graph_genus package is installed and otherwise uses `'simple'`.

OUTPUT: integer; the orientable genus of `G`

EXAMPLES:

```
sage: import sage.graphs.genus
sage: from sage.graphs.genus import simple_connected_graph_genus as genus
sage: graphs.CompleteGraph(5).genus()
1
sage: graphs.CompleteGraph(5).genus(algorithm='simple')
1
sage: graphs.CompleteGraph(5).genus(algorithm='page')      # optional - graph_
↪genus
1
sage: graphs.CompleteGraph(5).genus(algorithm='multi_genus')  # optional - graph_
↪genus
1
sage: G = graphs.PetersenGraph()
sage: genus(G, algorithm='simple')
1
sage: genus(G, algorithm='page')                           # optional - graph_
↪genus
1
sage: genus(G, algorithm='multi_genus')                    # optional - graph_
↪genus
1
```

```
>>> from sage.all import *
>>> import sage.graphs.genus
>>> from sage.graphs.genus import simple_connected_graph_genus as genus
>>> graphs.CompleteGraph(Integer(5)).genus()
```

(continues on next page)

(continued from previous page)

```
1
>>> graphs.CompleteGraph(Integer(5)).genus(algorithm='simple')
1
>>> graphs.CompleteGraph(Integer(5)).genus(algorithm='page')      # optional -␣
↪graph_genus
1
>>> graphs.CompleteGraph(Integer(5)).genus(algorithm='multi_genus')  # optional -␣
↪graph_genus
1
>>> G = graphs.PetersenGraph()
>>> genus(G, algorithm='simple')
1
>>> genus(G, algorithm='page')                              # optional - graph_genus
1
>>> genus(G, algorithm='multi_genus')                       # optional - graph_genus
1
```

ALGORITHM:

The `'simple'` algorithm is always available and is the fallback default when the optional graph_genus package is not installed. It is a direct rotation-system enumeration; it is useful for maximum genus and as a compact reference implementation, but has much worse scaling.

The optional `'page'` algorithm is provided by `graph_genus` and is based on the cycle-fitting method of [MetUlr2026]. It is usually the best choice for low-degree sparse graphs. For example, PAGE solves 3-cages through many girth-9 examples in seconds where the simple enumerator takes days or does not finish. The optional `'multi_genus'` algorithm is also provided by `graph_genus` and wraps Brinkmann's implementation [Bri2022], which is often extremely fast on complete, complete multipartite, and many dense/small examples, but has fixed C integer-size limits.

5.17 Lovász theta-function of graphs

AUTHORS:

- Dima Pasechnik (2015-06-30): Initial version

REFERENCE:

[Lov1979]

5.17.1 Functions

`sage.graphs.lovasz_theta.`**`lovasz_theta`**(*graph*)

Return the value of Lovász theta-function of graph.

For a graph G this function is denoted by $\theta(G)$, and it can be computed in polynomial time. Mathematically, its most important property is the following:

$$\alpha(G) \leq \theta(G) \leq \chi(\overline{G})$$

with $\alpha(G)$ and $\chi(\overline{G})$ being, respectively, the maximum size of an *independent set* set of G and the *chromatic number* of the *complement* $\overline{G}$ of G.

For more information, see the Wikipedia article Lovász_number.

Note

- Implemented for undirected graphs only. Use `to_undirected` to convert a digraph to an undirected graph.
- This function requires the optional package `csdp`, which you can install with `sage -i csdp`.

EXAMPLES:

```
sage: C = graphs.PetersenGraph()
sage: C.lovasz_theta()                                  # optional - csdp
4.0
sage: graphs.CycleGraph(5).lovasz_theta()               # optional - csdp
2.236068
```

```
>>> from sage.all import *
>>> C = graphs.PetersenGraph()
>>> C.lovasz_theta()                                  # optional - csdp
4.0
>>> graphs.CycleGraph(Integer(5)).lovasz_theta()           # optional - csdp
2.236068
```

5.18 Schnyder's algorithm for straight-line planar embeddings

A module for computing the (x,y) coordinates for a straight-line planar embedding of any connected planar graph with at least three vertices. Uses Walter Schnyder's Algorithm from [Sch1990].

AUTHORS:

- Jonathan Bober, Emily Kirkman (2008-02-09) – initial version

class sage.graphs.schnyder.**TreeNode**(*parent=None*, *children=None*, *label=None*)

Bases: `object`

A class to represent each node in the trees used by `_realizer` and `_compute_coordinates` when finding a planar geometric embedding in the grid.

Each tree node is doubly linked to its parent and children.

INPUT:

- `parent` – the parent TreeNode of `self`
- `children` – list of TreeNode children of `self`
- `label` – the associated realizer vertex label

EXAMPLES:

```
sage: from sage.graphs.schnyder import TreeNode
sage: tn = TreeNode(label=5)
sage: tn2 = TreeNode(label=2,parent=tn)
sage: tn3 = TreeNode(label=3)
sage: tn.append_child(tn3)
sage: tn.compute_number_of_descendants()
2
```

(continues on next page)

(continued from previous page)

```
sage: tn.number_of_descendants
2
sage: tn3.number_of_descendants
1
sage: tn.compute_depth_of_self_and_children()
sage: tn3.depth
2
```

```
>>> from sage.all import *
>>> from sage.graphs.schnyder import TreeNode
>>> tn = TreeNode(label=Integer(5))
>>> tn2 = TreeNode(label=Integer(2),parent=tn)
>>> tn3 = TreeNode(label=Integer(3))
>>> tn.append_child(tn3)
>>> tn.compute_number_of_descendants()
2
>>> tn.number_of_descendants
2
>>> tn3.number_of_descendants
1
>>> tn.compute_depth_of_self_and_children()
>>> tn3.depth
2
```

append_child(*child*)

Add a child to list of children.

EXAMPLES:

```
sage: from sage.graphs.schnyder import TreeNode
sage: tn = TreeNode(label=5)
sage: tn2 = TreeNode(label=2,parent=tn)
sage: tn3 = TreeNode(label=3)
sage: tn.append_child(tn3)
sage: tn.compute_number_of_descendants()
2
sage: tn.number_of_descendants
2
sage: tn3.number_of_descendants
1
sage: tn.compute_depth_of_self_and_children()
sage: tn3.depth
2
```

```
>>> from sage.all import *
>>> from sage.graphs.schnyder import TreeNode
>>> tn = TreeNode(label=Integer(5))
>>> tn2 = TreeNode(label=Integer(2),parent=tn)
>>> tn3 = TreeNode(label=Integer(3))
>>> tn.append_child(tn3)
>>> tn.compute_number_of_descendants()
2
```

(continues on next page)

(continued from previous page)

```
>>> tn.number_of_descendants
2
>>> tn3.number_of_descendants
1
>>> tn.compute_depth_of_self_and_children()
>>> tn3.depth
2
```

compute_depth_of_self_and_children()

Compute the depth of `self` and all descendants.

For each TreeNode, sets result as `attribute self.depth`.

EXAMPLES:

```
sage: from sage.graphs.schnyder import TreeNode
sage: tn = TreeNode(label=5)
sage: tn2 = TreeNode(label=2,parent=tn)
sage: tn3 = TreeNode(label=3)
sage: tn.append_child(tn3)
sage: tn.compute_number_of_descendants()
2
sage: tn.number_of_descendants
2
sage: tn3.number_of_descendants
1
sage: tn.compute_depth_of_self_and_children()
sage: tn3.depth
2
```

```
>>> from sage.all import *
>>> from sage.graphs.schnyder import TreeNode
>>> tn = TreeNode(label=Integer(5))
>>> tn2 = TreeNode(label=Integer(2),parent=tn)
>>> tn3 = TreeNode(label=Integer(3))
>>> tn.append_child(tn3)
>>> tn.compute_number_of_descendants()
2
>>> tn.number_of_descendants
2
>>> tn3.number_of_descendants
1
>>> tn.compute_depth_of_self_and_children()
>>> tn3.depth
2
```

compute_number_of_descendants()

Compute the number of descendants of `self` and all descendants.

For each TreeNode, sets result as attribute `self.number_of_descendants`.

EXAMPLES:

```
sage: from sage.graphs.schnyder import TreeNode
sage: tn = TreeNode(label=5)
sage: tn2 = TreeNode(label=2,parent=tn)
sage: tn3 = TreeNode(label=3)
sage: tn.append_child(tn3)
sage: tn.compute_number_of_descendants()
2
sage: tn.number_of_descendants
2
sage: tn3.number_of_descendants
1
sage: tn.compute_depth_of_self_and_children()
sage: tn3.depth
2
```

```
>>> from sage.all import *
>>> from sage.graphs.schnyder import TreeNode
>>> tn = TreeNode(label=Integer(5))
>>> tn2 = TreeNode(label=Integer(2),parent=tn)
>>> tn3 = TreeNode(label=Integer(3))
>>> tn.append_child(tn3)
>>> tn.compute_number_of_descendants()
2
>>> tn.number_of_descendants
2
>>> tn3.number_of_descendants
1
>>> tn.compute_depth_of_self_and_children()
>>> tn3.depth
2
```

sage.graphs.schnyder.**minimal_schnyder_wood**(*graph*, *root_edge=None*, *minimal=True*, *check=True*)

Return the minimal Schnyder wood of a planar rooted triangulation.

INPUT:

- `graph` – a planar triangulation, given by a graph with an embedding
- `root_edge` – a pair of vertices (default: from `-1` to `-2`); the third boundary vertex is then determined using the orientation and will be labelled `-3`
- `minimal` – boolean (default: `True`); whether to return a minimal or a maximal Schnyder wood
- `check` – boolean (default: `True`); whether to check if the input is a planar triangulation

OUTPUT:

A planar graph, with edges oriented and colored. The three outer edges of the initial graph are removed. For the three outer vertices the list of the neighbors stored in the combinatorial embedding is in the order of the incident edges between the two incident (and removed) outer edges, and not a cyclic shift of it.

The algorithm is taken from [Bre2000] (section 4.2).

EXAMPLES:

```
sage: from sage.graphs.schnyder import minimal_schnyder_wood
sage: g = Graph([(0,-1),(0,-2),(0,-3),(-1,-2),(-2,-3),
```

(continues on next page)

(continued from previous page)

```
....:  (-3,-1)], format='list_of_edges')
sage: g.set_embedding({-1:[-2,0,-3],-2:[-3,0,-1],
....:  -3:[-1,0,-2],0:[-1,-2,-3]})
sage: newg = minimal_schnyder_wood(g)
sage: newg.edges(sort=True)
[(0, -3, 'red'), (0, -2, 'blue'), (0, -1, 'green')]
sage: newg.plot(color_by_label={'red':'red','blue':'blue',                    #␣
↪needs sage.plot
....:  'green':'green',None:'black'})
Graphics object consisting of 8 graphics primitives
```

```
>>> from sage.all import *
>>> from sage.graphs.schnyder import minimal_schnyder_wood
>>> g = Graph([(Integer(0),-Integer(1)),(Integer(0),-Integer(2)),(Integer(0),-
↪Integer(3)),(-Integer(1),-Integer(2)),(-Integer(2),-Integer(3)),
...  (-Integer(3),-Integer(1))], format='list_of_edges')
>>> g.set_embedding({-Integer(1):[-Integer(2),Integer(0),-Integer(3)],-
↪Integer(2):[-Integer(3),Integer(0),-Integer(1)],
...  -Integer(3):[-Integer(1),Integer(0),-Integer(2)],Integer(0):[-Integer(1),-
↪Integer(2),-Integer(3)]})
>>> newg = minimal_schnyder_wood(g)
>>> newg.edges(sort=True)
[(0, -3, 'red'), (0, -2, 'blue'), (0, -1, 'green')]
>>> newg.plot(color_by_label={'red':'red','blue':'blue',                     #␣
↪needs sage.plot
...  'green':'green',None:'black'})
Graphics object consisting of 8 graphics primitives
```

A larger example:

```
sage: g = Graph([(0,-1),(0,2),(0,1),(0,-3),(-1,-3),(-1,2),
....: (-1,-2),(1,2),(1,-3),(2,-2),(1,-2),(-2,-3)], format='list_of_edges')
sage: g.set_embedding({-1:[-2,2,0,-3],-2:[-3,1,2,-1],
....: -3:[-1,0,1,-2],0:[-1,2,1,-3],1:[-2,-3,0,2],2:[-1,-2,1,0]})
sage: newg = minimal_schnyder_wood(g)
sage: newg.edges(sort=True, key=lambda e:(str(e[0]),str(e[1])))
[(0, -1, 'green'),
 (0, -3, 'red'),
 (0, 2, 'blue'),
 (1, -2, 'blue'),
 (1, -3, 'red'),
 (1, 0, 'green'),
 (2, -1, 'green'),
 (2, -2, 'blue'),
 (2, 1, 'red')]
sage: newg2 = minimal_schnyder_wood(g, minimal=False)
sage: newg2.edges(sort=True, key=lambda e:(str(e[0]),str(e[1])))
[(0, -1, 'green'),
 (0, -3, 'red'),
 (0, 1, 'blue'),
 (1, -2, 'blue'),
 (1, -3, 'red'),
```

(continues on next page)

(continued from previous page)

```
 (1, 2, 'green'),
 (2, -1, 'green'),
 (2, -2, 'blue'),
 (2, 0, 'red')]
```

```
>>> from sage.all import *
>>> g = Graph([(Integer(0),-Integer(1)),(Integer(0),Integer(2)),(Integer(0),
↪Integer(1)),(Integer(0),-Integer(3)),(-Integer(1),-Integer(3)),(-Integer(1),
↪Integer(2)),
... (-Integer(1),-Integer(2)),(Integer(1),Integer(2)),(Integer(1),-Integer(3)),
↪(Integer(2),-Integer(2)),(Integer(1),-Integer(2)),(-Integer(2),-Integer(3))],␣
↪format='list_of_edges')
>>> g.set_embedding({-Integer(1):[-Integer(2),Integer(2),Integer(0),-Integer(3)],-
↪Integer(2):[-Integer(3),Integer(1),Integer(2),-Integer(1)],
... -Integer(3):[-Integer(1),Integer(0),Integer(1),-Integer(2)],Integer(0):[-
↪Integer(1),Integer(2),Integer(1),-Integer(3)],Integer(1):[-Integer(2),-
↪Integer(3),Integer(0),Integer(2)],Integer(2):[-Integer(1),-Integer(2),
↪Integer(1),Integer(0)]})
>>> newg = minimal_schnyder_wood(g)
>>> newg.edges(sort=True, key=lambda e:(str(e[Integer(0)]),str(e[Integer(1)])))
[(0, -1, 'green'),
 (0, -3, 'red'),
 (0, 2, 'blue'),
 (1, -2, 'blue'),
 (1, -3, 'red'),
 (1, 0, 'green'),
 (2, -1, 'green'),
 (2, -2, 'blue'),
 (2, 1, 'red')]
>>> newg2 = minimal_schnyder_wood(g, minimal=False)
>>> newg2.edges(sort=True, key=lambda e:(str(e[Integer(0)]),str(e[Integer(1)])))
[(0, -1, 'green'),
 (0, -3, 'red'),
 (0, 1, 'blue'),
 (1, -2, 'blue'),
 (1, -3, 'red'),
 (1, 2, 'green'),
 (2, -1, 'green'),
 (2, -2, 'blue'),
 (2, 0, 'red')]
```

5.19 Wrapper for Boyer's (C) planarity algorithm

sage.graphs.planarity.**is_planar**(*g*, *kuratowski=False*, *set_pos=False*, *set_embedding=False*, *immutable=None*)

Check whether `g` is planar using Boyer's planarity algorithm.

If `kuratowski` is `False`, returns `True` if `g` is planar, `False` otherwise. If `kuratowski` is `True`, returns a tuple, first entry is a boolean (whether or not the graph is planar) and second entry is a Kuratowski subgraph, i.e. an edge subdivision of K_5 or $K_{3,3}$ (if not planar) or `None` (if planar). Also, will set an `_embedding` attribute for the graph `g` if `set_embedding` is set to `True`.

INPUT:

- `kuratowski` – boolean (default: `False`); when set to `True`, return a tuple of a boolean and either `None` or a Kuratowski subgraph (i.e. an edge subdivision of K_5 or $K_{3,3}$). When set to `False`, returns `True` if `g` is planar, `False` otherwise.
- `set_pos` – boolean (default: `False`); whether to use Schnyder's algorithm to determine and set positions
- `set_embedding` – boolean (default: `False`); whether to record the combinatorial embedding returned (see *get_embedding()*)
- `immutable` – boolean (default: `None`); whether to create a mutable/immutable graph. `immutable=None` (default) means that the graph and the Kuratowski subgraph will behave the same way. This parameter is ignored when `kuratowski=False`.

EXAMPLES:

```
sage: G = graphs.DodecahedralGraph()
sage: from sage.graphs.planarity import is_planar
sage: is_planar(G)
True
sage: Graph('@').is_planar()
True
```

```
>>> from sage.all import *
>>> G = graphs.DodecahedralGraph()
>>> from sage.graphs.planarity import is_planar
>>> is_planar(G)
True
>>> Graph('@').is_planar()
True
```

5.20 Graph traversals

This module implements the following graph traversals

lex_BFS()	Perform a lexicographic breadth first search (LexBFS) on the graph.
lex_DFS()	Perform a lexicographic depth first search (LexDFS) on the graph.
lex_UP()	Perform a lexicographic UP search (LexUP) on the graph.
lex_DOWN()	Perform a lexicographic DOWN search (LexDOWN) on the graph.
lex_M()	Return an ordering of the vertices according the LexM graph traversal.
lex_M_slow()	Return an ordering of the vertices according the LexM graph traversal.
lex_M_fast()	Return an ordering of the vertices according the LexM graph traversal.
maximum_cardinality_search()	Return an ordering of the vertices according a maximum cardinality search.
maximum_cardinality_search_M()	Return the ordering and the edges of the triangulation produced by MCS-M.

ALGORITHM:

For *lex_BFS()* with `algorithm="slow"`, *lex_DFS()*, *lex_UP()* and *lex_DOWN()* the same generic implementation is used. It corresponds to an implementation the generic algorithm described in "Algorithm 1" of [Mil2017].

This algorithm maintains for each vertex left in the graph a lexicographic label corresponding to the vertices already removed. The vertex of maximal lexicographic label is then removed, and the lexicographic labels of its neighbors are

updated. Depending on how the update is done, it corresponds to LexBFS, LexUP, LexDFS or LexDOWN: during the i-th iteration of the algorithm $n-i$ (for LexBFS and LexDOWN) or i (for LexDFS and LexUP) is appended (for LexBFS and LexUP) or prepended (for LexDFS and LexDOWN) to the lexicographic labels of all neighbors of the selected vertex that are left in the graph.

The time complexity of the algorithm is $O(mn)$ for `SparseGraph` and $O(\max\{mn, n^2\})$ for `DenseGraph`, where n is the number of vertices and m is the number of edges.

See [CK2008] and [Mil2017] for more details on the algorithm and graphs searching.

5.20.1 Methods

`sage.graphs.traversals.`**`BFS_with_condition`**(*G*, *v*, *condition*, *backward=False*)

Perform a breadth-first-search under condition.

This returns an iterator over edges between the allowed vertices. Some of these edges are traversed forward during the search, but others are yielded but not traversed when the target vertex has already been met.

INPUT:

- `G` – a simple DiGraph without loops
- `v` – a vertex of `G` ; the vertex `v` is assumed to satisfy the condition
- `condition` – a condition on vertices for them to be accepted
- `backward` – optional boolean (default `False`) whether to move backwards along the edges

EXAMPLES:

```
sage: from sage.graphs.traversals import BFS_with_condition
sage: dg = DiGraph([[0,1],[1,3],[0,2],[2,3]])
sage: list(BFS_with_condition(dg, 0, lambda x: not x % 2))
[(0, 2)]
```

```
>>> from sage.all import *
>>> from sage.graphs.traversals import BFS_with_condition
>>> dg = DiGraph([[Integer(0),Integer(1)],[Integer(1),Integer(3)],[Integer(0),
↪Integer(2)],[Integer(2),Integer(3)]])
>>> list(BFS_with_condition(dg, Integer(0), lambda x: not x % Integer(2)))
[(0, 2)]
```

Note

This code was kindly provided by Nathann Cohen.

`sage.graphs.traversals.`**`is_valid_lex_M_order`**(*G*, *alpha*, *F*)

Check whether the ordering alpha and the triangulation F are valid for G.

Given the graph $G = (V, E)$ with vertex set V and edge set E, and the set F of edges of a triangulation of G, let $H = (V, E \cup F)$. By induction one can see that for every $i \in \{1, ..., n-1\}$ the neighbors of $\alpha(i)$ in $H[\{\alpha(i), ..., \alpha(n)\}]$ induce a clique. The ordering α is a perfect elimination ordering of H, so H is chordal. See [RTL76] for more details.

INPUT:

- `G` – a Graph

- `alpha` – list; an ordering of the vertices of G
- `F` – an iterable of edges given either as `(u, v)` or `(u, v, label)`, the edges of the triangulation of G

`sage.graphs.traversals.lex_BFS`(*G*, *reverse=False*, *tree=False*, *initial_vertex=None*, *algorithm='fast'*)

Perform a lexicographic breadth first search (LexBFS) on the graph.

INPUT:

- `G` – a sage graph
- `reverse` – boolean (default: `False`); whether to return the vertices in discovery order, or the reverse
- `tree` – boolean (default: `False`); whether to return the discovery directed tree (each vertex being linked to the one that saw it last)
- `initial_vertex` – (default: `None`) the first vertex to consider
- `algorithm` – string (default: `'fast'`); algorithm to use among:
  - `'slow'` – it use the generic algorithm for all the lexicographic searchs. See the documentation of the *traversals* module for more details.
  - `'fast'` – this algorithm uses the notion of *partition refinement* to determine the position of the vertices in the ordering. The time complexity of this algorithm is in $O(n+m)$, and our implementation follows that complexity for `SparseGraph`. For `DenseGraph`, the complexity is $O(n^2)$. See [HMPV2000] and [TCHP2008] for more details. This algorithm is also used to compute slice decompositions of undirected graphs, a more thorough description can be found in the documentation of the *slice_decomposition* module.

Note

Loops and multiple edges are ignored during the computation of `lex_BFS` and directed graphs are converted to undirected graphs.

See also

- Wikipedia article Lexicographic_breadth-first_search
- *slice_decomposition* module and *slice_decomposition()* – compute a slice decomposition of the graph using an extended lex BFS algorithm
- *lex_DFS()* – perform a lexicographic depth first search (LexDFS) on the graph
- *lex_UP()* – perform a lexicographic UP search (LexUP) on the graph
- *lex_DOWN()* – perform a lexicographic DOWN search (LexDOWN) on the graph

EXAMPLES:

A Lex BFS is obviously an ordering of the vertices:

```
sage: g = graphs.CompleteGraph(6)
sage: set(g.lex_BFS()) == set(g)
True
```

```
>>> from sage.all import *
>>> g = graphs.CompleteGraph(Integer(6))
>>> set(g.lex_BFS()) == set(g)
True
```

Lex BFS ordering of the 3-sun graph:

```
sage: g = Graph([(1, 2), (1, 3), (2, 3), (2, 4), (2, 5), (3, 5), (3, 6), (4, 5),␣
↪(5, 6)])
sage: g.lex_BFS()
[1, 2, 3, 5, 4, 6]
```

```
>>> from sage.all import *
>>> g = Graph([(Integer(1), Integer(2)), (Integer(1), Integer(3)), (Integer(2),␣
↪Integer(3)), (Integer(2), Integer(4)), (Integer(2), Integer(5)), (Integer(3),␣
↪Integer(5)), (Integer(3), Integer(6)), (Integer(4), Integer(5)), (Integer(5),␣
↪Integer(6))])
>>> g.lex_BFS()
[1, 2, 3, 5, 4, 6]
```

The method also works for directed graphs:

```
sage: G = DiGraph([(1, 2), (2, 3), (1, 3)])
sage: correct_anwsers = [[2, 1, 3], [2, 3, 1]]
sage: G.lex_BFS(initial_vertex=2, algorithm='slow') in correct_anwsers
True
sage: G.lex_BFS(initial_vertex=2, algorithm='fast') in correct_anwsers
True
```

```
>>> from sage.all import *
>>> G = DiGraph([(Integer(1), Integer(2)), (Integer(2), Integer(3)), (Integer(1),␣
↪Integer(3))])
>>> correct_anwsers = [[Integer(2), Integer(1), Integer(3)], [Integer(2),␣
↪Integer(3), Integer(1)]]
>>> G.lex_BFS(initial_vertex=Integer(2), algorithm='slow') in correct_anwsers
True
>>> G.lex_BFS(initial_vertex=Integer(2), algorithm='fast') in correct_anwsers
True
```

For a Chordal Graph, a reversed Lex BFS is a Perfect Elimination Order:

```
sage: g = graphs.PathGraph(3).lexicographic_product(graphs.CompleteGraph(2))
sage: g.lex_BFS(reverse=True)
[(2, 1), (2, 0), (1, 1), (1, 0), (0, 1), (0, 0)]
```

```
>>> from sage.all import *
>>> g = graphs.PathGraph(Integer(3)).lexicographic_product(graphs.
↪CompleteGraph(Integer(2)))
>>> g.lex_BFS(reverse=True)
[(2, 1), (2, 0), (1, 1), (1, 0), (0, 1), (0, 0)]
```

And the vertices at the end of the tree of discovery are, for chordal graphs, simplicial vertices (their neighborhood is a complete graph):

```
sage: g = graphs.ClawGraph().lexicographic_product(graphs.CompleteGraph(2))
sage: v = g.lex_BFS()[-1]
sage: peo, tree = g.lex_BFS(initial_vertex = v,  tree=True)
sage: leaves = [v for v in tree if tree.in_degree(v) ==0]
sage: all(g.subgraph(g.neighbors(v)).is_clique() for v in leaves)
True
```

```
>>> from sage.all import *
>>> g = graphs.ClawGraph().lexicographic_product(graphs.CompleteGraph(Integer(2)))
>>> v = g.lex_BFS()[-Integer(1)]
>>> peo, tree = g.lex_BFS(initial_vertex = v,  tree=True)
>>> leaves = [v for v in tree if tree.in_degree(v) ==Integer(0)]
>>> all(g.subgraph(g.neighbors(v)).is_clique() for v in leaves)
True
```

Different orderings for different traversals:

```
sage: G = digraphs.DeBruijn(2,3)
sage: G.lex_BFS(initial_vertex='000', algorithm='fast')
['000', '001', '100', '010', '011', '110', '101', '111']
sage: G.lex_BFS(initial_vertex='000', algorithm='slow')
['000', '001', '100', '010', '011', '110', '101', '111']
sage: G.lex_DFS(initial_vertex='000')
['000', '001', '100', '010', '101', '110', '011', '111']
sage: G.lex_UP(initial_vertex='000')
['000', '001', '010', '101', '110', '111', '011', '100']
sage: G.lex_DOWN(initial_vertex='000')
['000', '001', '100', '011', '010', '110', '111', '101']
```

```
>>> from sage.all import *
>>> G = digraphs.DeBruijn(Integer(2),Integer(3))
>>> G.lex_BFS(initial_vertex='000', algorithm='fast')
['000', '001', '100', '010', '011', '110', '101', '111']
>>> G.lex_BFS(initial_vertex='000', algorithm='slow')
['000', '001', '100', '010', '011', '110', '101', '111']
>>> G.lex_DFS(initial_vertex='000')
['000', '001', '100', '010', '101', '110', '011', '111']
>>> G.lex_UP(initial_vertex='000')
['000', '001', '010', '101', '110', '111', '011', '100']
>>> G.lex_DOWN(initial_vertex='000')
['000', '001', '100', '011', '010', '110', '111', '101']
```

sage.graphs.traversals.**lex_DFS**(*G*, *reverse=False*, *tree=False*, *initial_vertex=None*)

Perform a lexicographic depth first search (LexDFS) on the graph.

INPUT:

- `G` – a sage graph
- `reverse` – boolean (default: `False`); whether to return the vertices in discovery order, or the reverse
- `tree` – boolean (default: `False`); whether to return the discovery directed tree (each vertex being linked to the one that saw it last)
- `initial_vertex` – (default: `None`) the first vertex to consider

Note

Loops and multiple edges are ignored during the computation of `lex_DFS` and directed graphs are converted to undirected graphs.

See also

- *lex_BFS()* – perform a lexicographic breadth first search (LexBFS) on the graph
- *lex_UP()* – perform a lexicographic UP search (LexUP) on the graph
- *lex_DOWN()* – perform a lexicographic DOWN search (LexDOWN) on the graph

ALGORITHM:

See the documentation of the *traversals* module.

EXAMPLES:

A Lex DFS is obviously an ordering of the vertices:

```
sage: g = graphs.CompleteGraph(6)
sage: set(g.lex_DFS()) == set(g)
True
```

```
>>> from sage.all import *
>>> g = graphs.CompleteGraph(Integer(6))
>>> set(g.lex_DFS()) == set(g)
True
```

Lex DFS ordering of the 3-sun graph:

```
sage: g = Graph([(1, 2), (1, 3), (2, 3), (2, 4), (2, 5), (3, 5), (3, 6), (4, 5),␣
↪(5, 6)])
sage: g.lex_DFS()
[1, 2, 3, 5, 6, 4]
```

```
>>> from sage.all import *
>>> g = Graph([(Integer(1), Integer(2)), (Integer(1), Integer(3)), (Integer(2),␣
↪Integer(3)), (Integer(2), Integer(4)), (Integer(2), Integer(5)), (Integer(3),␣
↪Integer(5)), (Integer(3), Integer(6)), (Integer(4), Integer(5)), (Integer(5),␣
↪Integer(6))])
>>> g.lex_DFS()
[1, 2, 3, 5, 6, 4]
```

The method also works for directed graphs:

```
sage: G = DiGraph([(1, 2), (2, 3), (1, 3)])
sage: correct_anwsers = [[2, 1, 3], [2, 3, 1]]
sage: G.lex_DFS(initial_vertex=2) in correct_anwsers
True
```

```
>>> from sage.all import *
>>> G = DiGraph([(Integer(1), Integer(2)), (Integer(2), Integer(3)), (Integer(1),␣
↪Integer(3))])
>>> correct_anwsers = [[Integer(2), Integer(1), Integer(3)], [Integer(2),␣
↪Integer(3), Integer(1)]]
>>> G.lex_DFS(initial_vertex=Integer(2)) in correct_anwsers
True
```

Different orderings for different traversals:

```
sage: G = digraphs.DeBruijn(2,3)
sage: G.lex_BFS(initial_vertex='000')
['000', '001', '100', '010', '011', '110', '101', '111']
sage: G.lex_DFS(initial_vertex='000')
['000', '001', '100', '010', '101', '110', '011', '111']
sage: G.lex_UP(initial_vertex='000')
['000', '001', '010', '101', '110', '111', '011', '100']
sage: G.lex_DOWN(initial_vertex='000')
['000', '001', '100', '011', '010', '110', '111', '101']
```

```
>>> from sage.all import *
>>> G = digraphs.DeBruijn(Integer(2),Integer(3))
>>> G.lex_BFS(initial_vertex='000')
['000', '001', '100', '010', '011', '110', '101', '111']
>>> G.lex_DFS(initial_vertex='000')
['000', '001', '100', '010', '101', '110', '011', '111']
>>> G.lex_UP(initial_vertex='000')
['000', '001', '010', '101', '110', '111', '011', '100']
>>> G.lex_DOWN(initial_vertex='000')
['000', '001', '100', '011', '010', '110', '111', '101']
```

sage.graphs.traversals.**lex_DOWN**(*G*, *reverse=False*, *tree=False*, *initial_vertex=None*)

Perform a lexicographic DOWN search (LexDOWN) on the graph.

INPUT:

- `G` – a sage graph
- `reverse` – boolean (default: `False`); whether to return the vertices in discovery order, or the reverse
- `tree` – boolean (default: `False`); whether to return the discovery directed tree (each vertex being linked to the one that saw it)
- `initial_vertex` – (default: `None`) the first vertex to consider

Note

Loops and multiple edges are ignored during the computation of `lex_DOWN` and directed graphs are converted to undirected graphs.

See also

- *`lex_BFS()`* – perform a lexicographic breadth first search (LexBFS) on the graph

- *lex_DFS()* – perform a lexicographic depth first search (LexDFS) on the graph
- *lex_UP()* – perform a lexicographic UP search (LexUP) on the graph

ALGORITHM:

See the documentation of the *traversals* module.

EXAMPLES:

A Lex DOWN is obviously an ordering of the vertices:

```
sage: g = graphs.CompleteGraph(6)
sage: set(g.lex_DOWN()) == set(g)
True
```

```
>>> from sage.all import *
>>> g = graphs.CompleteGraph(Integer(6))
>>> set(g.lex_DOWN()) == set(g)
True
```

Lex DOWN ordering of the 3-sun graph:

```
sage: g = Graph([(1, 2), (1, 3), (2, 3), (2, 4), (2, 5), (3, 5), (3, 6), (4, 5),␣
↪(5, 6)])
sage: g.lex_DOWN()
[1, 2, 3, 4, 6, 5]
```

```
>>> from sage.all import *
>>> g = Graph([(Integer(1), Integer(2)), (Integer(1), Integer(3)), (Integer(2),␣
↪Integer(3)), (Integer(2), Integer(4)), (Integer(2), Integer(5)), (Integer(3),␣
↪Integer(5)), (Integer(3), Integer(6)), (Integer(4), Integer(5)), (Integer(5),␣
↪Integer(6))])
>>> g.lex_DOWN()
[1, 2, 3, 4, 6, 5]
```

The method also works for directed graphs:

```
sage: G = DiGraph([(1, 2), (2, 3), (1, 3)])
sage: correct_anwsers = [[2, 1, 3], [2, 3, 1]]
sage: G.lex_DOWN(initial_vertex=2) in correct_anwsers
True
```

```
>>> from sage.all import *
>>> G = DiGraph([(Integer(1), Integer(2)), (Integer(2), Integer(3)), (Integer(1),␣
↪Integer(3))])
>>> correct_anwsers = [[Integer(2), Integer(1), Integer(3)], [Integer(2),␣
↪Integer(3), Integer(1)]]
>>> G.lex_DOWN(initial_vertex=Integer(2)) in correct_anwsers
True
```

Different orderings for different traversals:

```
sage: G = digraphs.DeBruijn(2,3)
sage: G.lex_BFS(initial_vertex='000')
['000', '001', '100', '010', '011', '110', '101', '111']
sage: G.lex_DFS(initial_vertex='000')
['000', '001', '100', '010', '101', '110', '011', '111']
sage: G.lex_UP(initial_vertex='000')
['000', '001', '010', '101', '110', '111', '011', '100']
sage: G.lex_DOWN(initial_vertex='000')
['000', '001', '100', '011', '010', '110', '111', '101']
```

```
>>> from sage.all import *
>>> G = digraphs.DeBruijn(Integer(2),Integer(3))
>>> G.lex_BFS(initial_vertex='000')
['000', '001', '100', '010', '011', '110', '101', '111']
>>> G.lex_DFS(initial_vertex='000')
['000', '001', '100', '010', '101', '110', '011', '111']
>>> G.lex_UP(initial_vertex='000')
['000', '001', '010', '101', '110', '111', '011', '100']
>>> G.lex_DOWN(initial_vertex='000')
['000', '001', '100', '011', '010', '110', '111', '101']
```

sage.graphs.traversals.**lex_M**(*self*, *triangulation=False*, *labels=False*, *initial_vertex=None*, *algorithm=None*)

Return an ordering of the vertices according the LexM graph traversal.

LexM is a lexicographic ordering scheme that is a special type of breadth-first-search. LexM can also produce a triangulation of the given graph. This functionality is implemented in this method. For more details on the algorithms used see Sections 4 (`'lex_M_slow'`) and 5.3 (`'lex_M_fast'`) of [RTL76].

Note

This method works only for undirected graphs.

INPUT:

- `triangulation` – boolean (default: `False`); whether to return a list of edges that need to be added in order to triangulate the graph
- `labels` – boolean (default: `False`); whether to return the labels assigned to each vertex
- `initial_vertex` – (default: `None`); the first vertex to consider
- `algorithm` – string (default: `None`); one of the following algorithms:
  - `'lex_M_slow'`: slower implementation of LexM traversal
  - `'lex_M_fast'`: faster implementation of LexM traversal (works only when `labels` is set to `False`)
  - `None`: Sage chooses the best algorithm: `'lex_M_slow'` if `labels` is set to `True`, `'lex_M_fast'` otherwise.

OUTPUT:

Depending on the values of the parameters `triangulation` and `labels` the method will return one or more of the following (in that order):

- an ordering of vertices of the graph according to LexM ordering scheme
- the labels assigned to each vertex

- a list of edges that when added to the graph will triangulate it

EXAMPLES:

LexM produces an ordering of the vertices:

```
sage: g = graphs.CompleteGraph(6)
sage: ord = g.lex_M(algorithm='lex_M_fast')
sage: len(ord) == g.order()
True
sage: set(ord) == set(g.vertices(sort=False))
True
sage: ord = g.lex_M(algorithm='lex_M_slow')
sage: len(ord) == g.order()
True
sage: set(ord) == set(g.vertices(sort=False))
True
```

```
>>> from sage.all import *
>>> g = graphs.CompleteGraph(Integer(6))
>>> ord = g.lex_M(algorithm='lex_M_fast')
>>> len(ord) == g.order()
True
>>> set(ord) == set(g.vertices(sort=False))
True
>>> ord = g.lex_M(algorithm='lex_M_slow')
>>> len(ord) == g.order()
True
>>> set(ord) == set(g.vertices(sort=False))
True
```

Both algorithms produce a valid LexM ordering α (i.e the neighbors of $\alpha(i)$ in $G[\{\alpha(i), ..., \alpha(n)\}]$ induce a clique):

```
sage: from sage.graphs.traversals import is_valid_lex_M_order
sage: G = graphs.PetersenGraph()
sage: ord, F = G.lex_M(triangulation=True, algorithm='lex_M_slow')
sage: is_valid_lex_M_order(G, ord, F)
True
sage: ord, F = G.lex_M(triangulation=True, algorithm='lex_M_fast')
sage: is_valid_lex_M_order(G, ord, F)
True
```

```
>>> from sage.all import *
>>> from sage.graphs.traversals import is_valid_lex_M_order
>>> G = graphs.PetersenGraph()
>>> ord, F = G.lex_M(triangulation=True, algorithm='lex_M_slow')
>>> is_valid_lex_M_order(G, ord, F)
True
>>> ord, F = G.lex_M(triangulation=True, algorithm='lex_M_fast')
>>> is_valid_lex_M_order(G, ord, F)
True
```

LexM produces a triangulation of given graph:

```
sage: G = graphs.PetersenGraph()
sage: _, F = G.lex_M(triangulation=True)
sage: H = Graph(F, format='list_of_edges')
sage: H.is_chordal()
True
```

```
>>> from sage.all import *
>>> G = graphs.PetersenGraph()
>>> _, F = G.lex_M(triangulation=True)
>>> H = Graph(F, format='list_of_edges')
>>> H.is_chordal()
True
```

LexM ordering of the 3-sun graph:

```
sage: g = Graph([(1, 2), (1, 3), (2, 3), (2, 4), (2, 5), (3, 5), (3, 6), (4, 5),␣
↪(5, 6)])
sage: g.lex_M()
[6, 4, 5, 3, 2, 1]
```

```
>>> from sage.all import *
>>> g = Graph([(Integer(1), Integer(2)), (Integer(1), Integer(3)), (Integer(2),␣
↪Integer(3)), (Integer(2), Integer(4)), (Integer(2), Integer(5)), (Integer(3),␣
↪Integer(5)), (Integer(3), Integer(6)), (Integer(4), Integer(5)), (Integer(5),␣
↪Integer(6))])
>>> g.lex_M()
[6, 4, 5, 3, 2, 1]
```

The ordering depends on the initial vertex:

```
sage: G = graphs.HouseGraph()
sage: G.lex_M(algorithm='lex_M_slow', initial_vertex=0)
[4, 3, 2, 1, 0]
sage: G.lex_M(algorithm='lex_M_slow', initial_vertex=2)
[1, 4, 3, 0, 2]
sage: G.lex_M(algorithm='lex_M_fast', initial_vertex=0)
[4, 3, 2, 1, 0]
sage: G.lex_M(algorithm='lex_M_fast', initial_vertex=2)
[1, 4, 3, 0, 2]
```

```
>>> from sage.all import *
>>> G = graphs.HouseGraph()
>>> G.lex_M(algorithm='lex_M_slow', initial_vertex=Integer(0))
[4, 3, 2, 1, 0]
>>> G.lex_M(algorithm='lex_M_slow', initial_vertex=Integer(2))
[1, 4, 3, 0, 2]
>>> G.lex_M(algorithm='lex_M_fast', initial_vertex=Integer(0))
[4, 3, 2, 1, 0]
>>> G.lex_M(algorithm='lex_M_fast', initial_vertex=Integer(2))
[1, 4, 3, 0, 2]
```

sage.graphs.traversals.**lex_M_fast**(*G*, *triangulation=False*, *initial_vertex=None*)

Return an ordering of the vertices according the LexM graph traversal.

LexM is a lexicographic ordering scheme that is a special type of breadth-first-search. This function implements the algorithm described in Section 5.3 of [RTL76].

Note that instead of using labels $1, 2, \ldots, k$ and adding $1/2$, we use labels $2, 4, \ldots, k$ and add 1, thus avoiding to use floats or rationals.

Note

This method works only for undirected graphs.

INPUT:

- `G` – a sage graph
- `triangulation` – boolean (default: `False`); whether to return the triangulation of given graph produced by the method
- `initial_vertex` – (default: `None`) the first vertex to consider

OUTPUT:

This method will return an ordering of the vertices of `G` according to the LexM ordering scheme. Furthermore, if `triangulation` is set to `True` the method also returns a list of edges `F` such that when added to `G` the resulting graph is a triangulation of `G`.

EXAMPLES:

A LexM ordering is obviously an ordering of the vertices:

```
sage: from sage.graphs.traversals import lex_M_fast
sage: g = graphs.CompleteGraph(6)
sage: len(lex_M_fast(g)) == g.order()
True
```

```
>>> from sage.all import *
>>> from sage.graphs.traversals import lex_M_fast
>>> g = graphs.CompleteGraph(Integer(6))
>>> len(lex_M_fast(g)) == g.order()
True
```

LexM ordering of the 3-sun graph:

```
sage: from sage.graphs.traversals import lex_M_fast
sage: g = Graph([(1, 2), (1, 3), (2, 3), (2, 4), (2, 5), (3, 5), (3, 6), (4, 5),␣
↪(5, 6)])
sage: lex_M_fast(g)
[6, 4, 5, 3, 2, 1]
```

```
>>> from sage.all import *
>>> from sage.graphs.traversals import lex_M_fast
>>> g = Graph([(Integer(1), Integer(2)), (Integer(1), Integer(3)), (Integer(2),␣
↪Integer(3)), (Integer(2), Integer(4)), (Integer(2), Integer(5)), (Integer(3),␣
↪Integer(5)), (Integer(3), Integer(6)), (Integer(4), Integer(5)), (Integer(5),␣
↪Integer(6))])
>>> lex_M_fast(g)
[6, 4, 5, 3, 2, 1]
```

LexM produces a triangulation of given graph:

```
sage: from sage.graphs.traversals import lex_M_fast
sage: G = graphs.PetersenGraph()
sage: _, F = lex_M_fast(G, triangulation=True)
sage: H = G.copy()
sage: H.add_edges(F)
sage: H.is_chordal()
True
```

```
>>> from sage.all import *
>>> from sage.graphs.traversals import lex_M_fast
>>> G = graphs.PetersenGraph()
>>> _, F = lex_M_fast(G, triangulation=True)
>>> H = G.copy()
>>> H.add_edges(F)
>>> H.is_chordal()
True
```

sage.graphs.traversals.**lex_M_slow**(*G*, *triangulation=False*, *labels=False*, *initial_vertex=None*)

Return an ordering of the vertices according the LexM graph traversal.

LexM is a lexicographic ordering scheme that is a special type of breadth-first-search. This function implements the algorithm described in Section 4 of [RTL76].

During the search, the vertices are numbered from n to 1. Let $\alpha(i)$ denote the vertex numbered i and let $\alpha^{-1}(u)$ denote the number assigned to u. Each vertex u has also a label, denoted by $label(u)$, consisting of a list of numbers selected from $[1, n]$ and ordered in decreasing order. Given two labels $L_1 = [p_1, p_2, \ldots, p_k]$ and $L_1 = [q_1, q_2, \ldots, q_l]$, we define $L_1 < L_2$ if, for some j, $p_i == q_i$ for $i = 1, \ldots, j-1$ and $p_j < q_j$, or if $p_i == q_i$ for $i = 1, \ldots, k$ and $k < l$. Observe that this is exactly how Python compares two lists.

Note

This method works only for undirected graphs.

INPUT:

- `G` – a sage graph
- `triangulation` – boolean (default: `False`); whether to return the triangulation of the graph produced by the method
- `labels` – boolean (default: `False`); whether to return the labels assigned to each vertex
- `initial_vertex` – (default: `None`) the first vertex to consider. If not specified, an arbitrary vertex is chosen.

OUTPUT:

Depending on the values of the parameters `triangulation` and `labels` the method will return one or more of the following (in that order):

- the ordering of vertices of G
- the labels assigned to each vertex
- a list of edges that when added to G will produce a triangulation of G

EXAMPLES:

A LexM ordering is obviously an ordering of the vertices:

```
sage: from sage.graphs.traversals import lex_M_slow
sage: g = graphs.CompleteGraph(6)
sage: len(lex_M_slow(g)) == g.order()
True
```

```
>>> from sage.all import *
>>> from sage.graphs.traversals import lex_M_slow
>>> g = graphs.CompleteGraph(Integer(6))
>>> len(lex_M_slow(g)) == g.order()
True
```

LexM ordering and label assignments on the vertices of the 3-sun graph:

```
sage: from sage.graphs.traversals import lex_M_slow
sage: g = Graph([(1, 2), (1, 3), (2, 3), (2, 4), (2, 5), (3, 5), (3, 6), (4, 5),␣
↪(5, 6)])
sage: lex_M_slow(g, labels=True)
([6, 4, 5, 3, 2, 1],
 {1: [], 2: [5], 3: [5, 4], 4: [4, 2], 5: [4, 3], 6: [3, 2]})
```

```
>>> from sage.all import *
>>> from sage.graphs.traversals import lex_M_slow
>>> g = Graph([(Integer(1), Integer(2)), (Integer(1), Integer(3)), (Integer(2),␣
↪Integer(3)), (Integer(2), Integer(4)), (Integer(2), Integer(5)), (Integer(3),␣
↪Integer(5)), (Integer(3), Integer(6)), (Integer(4), Integer(5)), (Integer(5),␣
↪Integer(6))])
>>> lex_M_slow(g, labels=True)
([6, 4, 5, 3, 2, 1],
 {1: [], 2: [5], 3: [5, 4], 4: [4, 2], 5: [4, 3], 6: [3, 2]})
```

LexM produces a triangulation of given graph:

```
sage: from sage.graphs.traversals import lex_M_slow
sage: G = graphs.PetersenGraph()
sage: _, F = lex_M_slow(G, triangulation=True)
sage: H = G.copy()
sage: H.add_edges(F)
sage: H.is_chordal()
True
```

```
>>> from sage.all import *
>>> from sage.graphs.traversals import lex_M_slow
>>> G = graphs.PetersenGraph()
>>> _, F = lex_M_slow(G, triangulation=True)
>>> H = G.copy()
>>> H.add_edges(F)
>>> H.is_chordal()
True
```

sage.graphs.traversals.**lex_UP**(*G*, *reverse=False*, *tree=False*, *initial_vertex=None*)

Perform a lexicographic UP search (LexUP) on the graph.

INPUT:

- `G` – a sage graph
- `reverse` – boolean (default: `False`); whether to return the vertices in discovery order, or the reverse
- `tree` – boolean (default: `False`); whether to return the discovery directed tree (each vertex being linked to the one that saw it last)
- `initial_vertex` – (default: `None`) the first vertex to consider

Note

Loops and multiple edges are ignored during the computation of `lex_UP` and directed graphs are converted to undirected graphs.

See also

- *`lex_BFS()`* – perform a lexicographic breadth first search (LexBFS) on the graph
- *`lex_DFS()`* – perform a lexicographic depth first search (LexDFS) on the graph
- *`lex_DOWN()`* – perform a lexicographic DOWN search (LexDOWN) on the graph

ALGORITHM:

See the documentation of the *`traversals`* module.

EXAMPLES:

A Lex UP is obviously an ordering of the vertices:

```
sage: g = graphs.CompleteGraph(6)
sage: set(g.lex_UP()) == set(g)
True
```

```
>>> from sage.all import *
>>> g = graphs.CompleteGraph(Integer(6))
>>> set(g.lex_UP()) == set(g)
True
```

Lex UP ordering of the 3-sun graph:

```
sage: g = Graph([(1, 2), (1, 3), (2, 3), (2, 4), (2, 5), (3, 5), (3, 6), (4, 5),␣
↪(5, 6)])
sage: g.lex_UP()
[1, 2, 4, 5, 6, 3]
```

```
>>> from sage.all import *
>>> g = Graph([(Integer(1), Integer(2)), (Integer(1), Integer(3)), (Integer(2),␣
↪Integer(3)), (Integer(2), Integer(4)), (Integer(2), Integer(5)), (Integer(3),␣
↪Integer(5)), (Integer(3), Integer(6)), (Integer(4), Integer(5)), (Integer(5),␣
↪Integer(6))])
>>> g.lex_UP()
[1, 2, 4, 5, 6, 3]
```

The method also works for directed graphs:

```
sage: G = DiGraph([(1, 2), (2, 3), (1, 3)])
sage: correct_anwsers = [[2, 1, 3], [2, 3, 1]]
sage: G.lex_UP(initial_vertex=2) in correct_anwsers
True
```

```
>>> from sage.all import *
>>> G = DiGraph([(Integer(1), Integer(2)), (Integer(2), Integer(3)), (Integer(1),␣
↪Integer(3))])
>>> correct_anwsers = [[Integer(2), Integer(1), Integer(3)], [Integer(2),␣
↪Integer(3), Integer(1)]]
>>> G.lex_UP(initial_vertex=Integer(2)) in correct_anwsers
True
```

Different orderings for different traversals:

```
sage: G = digraphs.DeBruijn(2,3)
sage: G.lex_BFS(initial_vertex='000')
['000', '001', '100', '010', '011', '110', '101', '111']
sage: G.lex_DFS(initial_vertex='000')
['000', '001', '100', '010', '101', '110', '011', '111']
sage: G.lex_UP(initial_vertex='000')
['000', '001', '010', '101', '110', '111', '011', '100']
sage: G.lex_DOWN(initial_vertex='000')
['000', '001', '100', '011', '010', '110', '111', '101']
```

```
>>> from sage.all import *
>>> G = digraphs.DeBruijn(Integer(2),Integer(3))
>>> G.lex_BFS(initial_vertex='000')
['000', '001', '100', '010', '011', '110', '101', '111']
>>> G.lex_DFS(initial_vertex='000')
['000', '001', '100', '010', '101', '110', '011', '111']
>>> G.lex_UP(initial_vertex='000')
['000', '001', '010', '101', '110', '111', '011', '100']
>>> G.lex_DOWN(initial_vertex='000')
['000', '001', '100', '011', '010', '110', '111', '101']
```

sage.graphs.traversals.**maximum_cardinality_search**(*G*, *reverse=False*, *tree=False*, *initial_vertex=None*)

Return an ordering of the vertices according a maximum cardinality search.

Maximum cardinality search (MCS) is a graph traversal introduced in [TY1984]. It starts by assigning an arbitrary vertex (or the specified `initial_vertex`) of G the last position in the ordering α. Every vertex keeps a weight equal to the number of its already processed neighbors (i.e., already added to α), and a vertex of largest such number is chosen at each step i to be placed in position $n - i$ in α. This ordering can be computed in time $O(n + m)$.

Time complexity is $O(n + m)$ for `SparseGraph` and $O(n^2)$ for `DenseGraph` where n is the number of vertices and m is the number of edges.

When the graph is chordal, the ordering returned by MCS is a *perfect elimination ordering*, like *lex_BFS()*. So this ordering can be used to recognize chordal graphs. See [He2006] for more details.

Note

The current implementation is for connected graphs only.

INPUT:

- `G` – a Sage graph
- `reverse` – boolean (default: `False`); whether to return the vertices in discovery order, or the reverse
- `tree` – boolean (default: `False`); whether to also return the discovery directed tree (each vertex being linked to the one that saw it for the first time)
- `initial_vertex` – (default: `None`) the first vertex to consider

OUTPUT:

By default, return the ordering α as a list. When `tree` is `True`, the method returns a tuple (α, T), where T is a directed tree with the same set of vertices as G and a directed edge from u to v if u was the first vertex to see v.

EXAMPLES:

When specified, the `initial_vertex` is placed at the end of the ordering, unless parameter `reverse` is `True`, in which case it is placed at the beginning:

```
sage: G = graphs.PathGraph(4)
sage: G.maximum_cardinality_search(initial_vertex=0)
[3, 2, 1, 0]
sage: G.maximum_cardinality_search(initial_vertex=1)
[3, 2, 0, 1]
sage: G.maximum_cardinality_search(initial_vertex=2)
[0, 3, 1, 2]
sage: G.maximum_cardinality_search(initial_vertex=3)
[0, 1, 2, 3]
sage: G.maximum_cardinality_search(initial_vertex=3, reverse=True)
[3, 2, 1, 0]
```

```
>>> from sage.all import *
>>> G = graphs.PathGraph(Integer(4))
>>> G.maximum_cardinality_search(initial_vertex=Integer(0))
[3, 2, 1, 0]
>>> G.maximum_cardinality_search(initial_vertex=Integer(1))
[3, 2, 0, 1]
>>> G.maximum_cardinality_search(initial_vertex=Integer(2))
[0, 3, 1, 2]
>>> G.maximum_cardinality_search(initial_vertex=Integer(3))
[0, 1, 2, 3]
>>> G.maximum_cardinality_search(initial_vertex=Integer(3), reverse=True)
[3, 2, 1, 0]
```

Returning the discovery tree:

```
sage: G = graphs.PathGraph(4)
sage: _, T = G.maximum_cardinality_search(tree=True, initial_vertex=0)
sage: T.order(), T.size()
(4, 3)
```

(continues on next page)

(continued from previous page)

```
sage: T.edges(labels=False, sort=True)
[(1, 0), (2, 1), (3, 2)]
sage: _, T = G.maximum_cardinality_search(tree=True, initial_vertex=3)
sage: T.edges(labels=False, sort=True)
[(0, 1), (1, 2), (2, 3)]
```

```
>>> from sage.all import *
>>> G = graphs.PathGraph(Integer(4))
>>> _, T = G.maximum_cardinality_search(tree=True, initial_vertex=Integer(0))
>>> T.order(), T.size()
(4, 3)
>>> T.edges(labels=False, sort=True)
[(1, 0), (2, 1), (3, 2)]
>>> _, T = G.maximum_cardinality_search(tree=True, initial_vertex=Integer(3))
>>> T.edges(labels=False, sort=True)
[(0, 1), (1, 2), (2, 3)]
```

sage.graphs.traversals.**maximum_cardinality_search_M**(*G*, *initial_vertex=None*)

Return the ordering and the edges of the triangulation produced by MCS-M.

Maximum cardinality search M (MCS-M) is an extension of MCS (*maximum_cardinality_search()*) in the same way that Lex-M (*lex_M()*) is an extension of Lex-BFS (*lex_BFS()*). That is, in MCS-M when u receives number i at step $n - i + 1$, it increments the weight of all unnumbered vertices v for which there exists a path between u and v consisting only of unnumbered vertices with weight strictly less than $w^-(u)$ and $w^-(v)$, where w^- is the number of times a vertex has been reached during previous iterations. See [BBHP2004] for the details of this $O(nm)$ time algorithm.

If G is not connected, the orderings of each of its connected components are added consecutively. Furthermore, if G has k connected components C_i for $0 \leq i < k$, X contains at least one vertex of C_i for each $i \geq 1$. Hence, $|X| \geq k - 1$. In particular, some isolated vertices (i.e., of degree 0) can appear in X as for such a vertex x, we have that $G \setminus N(x) = G$ is not connected.

INPUT:

- `G` – a Sage graph
- `initial_vertex` – (default: `None`) the first vertex to consider

OUTPUT: a tuple (α, F, X), where

- α is the resulting ordering of the vertices. If an initial vertex is specified, it gets the last position in the ordering α.
- F is the list of edges of a minimal triangulation of G according α
- X is a list of vertices such that for each $x \in X$, the neighborhood of x in G is a separator (i.e., $G \setminus N(x)$ is not connected). Note that we may have $N(x) = \emptyset$ if G is not connected and x has degree 0.

EXAMPLES:

Chordal graphs have a perfect elimination ordering, and so the set F of edges of the triangulation is empty:

```
sage: G = graphs.RandomChordalGraph(20)
sage: alpha, F, X = G.maximum_cardinality_search_M(); F
[]
```

```
>>> from sage.all import *
>>> G = graphs.RandomChordalGraph(Integer(20))
>>> alpha, F, X = G.maximum_cardinality_search_M(); F
[]
```

The cycle of order 4 is not chordal and so the triangulation has one edge:

```
sage: G = graphs.CycleGraph(4)
sage: alpha, F, X = G.maximum_cardinality_search_M(); len(F)
1
```

```
>>> from sage.all import *
>>> G = graphs.CycleGraph(Integer(4))
>>> alpha, F, X = G.maximum_cardinality_search_M(); len(F)
1
```

The number of edges needed to triangulate of a cycle graph or order n is $n-3$, independently of the initial vertex:

```
sage: n = randint(3, 20)
sage: C = graphs.CycleGraph(n)
sage: _, F, X = C.maximum_cardinality_search_M()
sage: len(F) == n - 3
True
sage: _, F, X = C.maximum_cardinality_search_M(initial_vertex=C.random_vertex())
sage: len(F) == n - 3
True
```

```
>>> from sage.all import *
>>> n = randint(Integer(3), Integer(20))
>>> C = graphs.CycleGraph(n)
>>> _, F, X = C.maximum_cardinality_search_M()
>>> len(F) == n - Integer(3)
True
>>> _, F, X = C.maximum_cardinality_search_M(initial_vertex=C.random_vertex())
>>> len(F) == n - Integer(3)
True
```

When an initial vertex is specified, it gets the last position in the ordering:

```
sage: G = graphs.PathGraph(4)
sage: G.maximum_cardinality_search_M(initial_vertex=0)
([3, 2, 1, 0], [], [2, 3])
sage: G.maximum_cardinality_search_M(initial_vertex=1)
([3, 2, 0, 1], [], [2, 3])
sage: G.maximum_cardinality_search_M(initial_vertex=2)
([0, 1, 3, 2], [], [0, 1])
sage: G.maximum_cardinality_search_M(initial_vertex=3)
([0, 1, 2, 3], [], [0, 1])
```

```
>>> from sage.all import *
>>> G = graphs.PathGraph(Integer(4))
>>> G.maximum_cardinality_search_M(initial_vertex=Integer(0))
```

(continues on next page)

(continued from previous page)

```
([3, 2, 1, 0], [], [2, 3])
>>> G.maximum_cardinality_search_M(initial_vertex=Integer(1))
([3, 2, 0, 1], [], [2, 3])
>>> G.maximum_cardinality_search_M(initial_vertex=Integer(2))
([0, 1, 3, 2], [], [0, 1])
>>> G.maximum_cardinality_search_M(initial_vertex=Integer(3))
([0, 1, 2, 3], [], [0, 1])
```

When G is not connected, the orderings of each of its connected components are added consecutively, the vertices of the component containing the initial vertex occupying the last positions:

```
sage: G = graphs.CycleGraph(4) * 2
sage: G.maximum_cardinality_search_M()[0]
[5, 4, 6, 7, 2, 3, 1, 0]
sage: G.maximum_cardinality_search_M(initial_vertex=7)[0]
[2, 1, 3, 0, 5, 6, 4, 7]
```

```
>>> from sage.all import *
>>> G = graphs.CycleGraph(Integer(4)) * Integer(2)
>>> G.maximum_cardinality_search_M()[Integer(0)]
[5, 4, 6, 7, 2, 3, 1, 0]
>>> G.maximum_cardinality_search_M(initial_vertex=Integer(7))[Integer(0)]
[2, 1, 3, 0, 5, 6, 4, 7]
```

Furthermore, if G has k connected components, X contains at least one vertex per connected component, except for the first one, and so at least $k - 1$ vertices:

```
sage: for k in range(1, 5):
....:     _, _, X = Graph(k).maximum_cardinality_search_M()
....:     if len(X) < k - 1:
....:         raise ValueError("something goes wrong")
sage: G = graphs.RandomGNP(10, .2)
sage: cc = G.connected_components(sort=False)
sage: _, _, X = G.maximum_cardinality_search_M()
sage: len(X) >= len(cc) - 1
True
```

```
>>> from sage.all import *
>>> for k in range(Integer(1), Integer(5)):
...     _, _, X = Graph(k).maximum_cardinality_search_M()
...     if len(X) < k - Integer(1):
...         raise ValueError("something goes wrong")
>>> G = graphs.RandomGNP(Integer(10), RealNumber('.2'))
>>> cc = G.connected_components(sort=False)
>>> _, _, X = G.maximum_cardinality_search_M()
>>> len(X) >= len(cc) - Integer(1)
True
```

In the example of [BPS2010], the triangulation has 3 edges:

```
sage: G = Graph({'a': ['b', 'k'], 'b': ['c'], 'c': ['d', 'j', 'k'],
....:            'd': ['e', 'f', 'j', 'k'], 'e': ['g'],
```

(continues on next page)

(continued from previous page)

```
....:             'f': ['g', 'j', 'k'], 'g': ['j', 'k'], 'h': ['i', 'j'],
....:             'i': ['k'], 'j': ['k']})
sage: _, F, _ = G.maximum_cardinality_search_M(initial_vertex='a')
sage: len(F)
3
```

```
>>> from sage.all import *
>>> G = Graph({'a': ['b', 'k'], 'b': ['c'], 'c': ['d', 'j', 'k'],
...            'd': ['e', 'f', 'j', 'k'], 'e': ['g'],
...            'f': ['g', 'j', 'k'], 'g': ['j', 'k'], 'h': ['i', 'j'],
...            'i': ['k'], 'j': ['k']})
>>> _, F, _ = G.maximum_cardinality_search_M(initial_vertex='a')
>>> len(F)
3
```

5.21 Graph plotting

(For LaTeX drawings of graphs, see the `graph_latex` *module.)*

All graphs have an associated Sage graphics object, which you can display:

```
sage: G = graphs.WheelGraph(15)
sage: P = G.plot()
sage: P.show()  # long time
```

```
>>> from sage.all import *
>>> G = graphs.WheelGraph(Integer(15))
>>> P = G.plot()
>>> P.show()  # long time
```

When plotting a graph created using Sage's `Graph` command, node positions are determined using the spring-layout algorithm. Special graphs available from `graphs.*` have preset positions. For example, compare the two plots of the Petersen graph, as obtained using `Graph` or as obtained from that database:

```
sage: petersen_spring = Graph(':I`ES@obGkqegW~')
sage: petersen_spring.show()  # long time
```

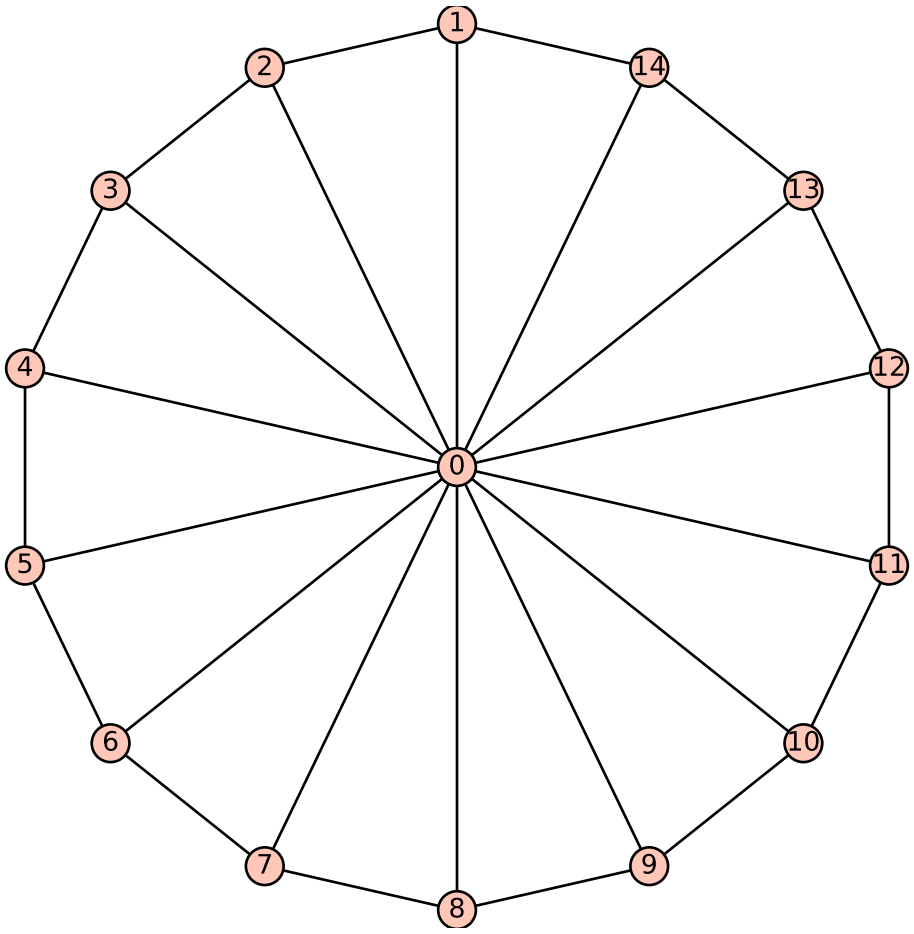
```
>>> from sage.all import *
>>> petersen_spring = Graph(':I`ES@obGkqegW~')
>>> petersen_spring.show()  # long time
```

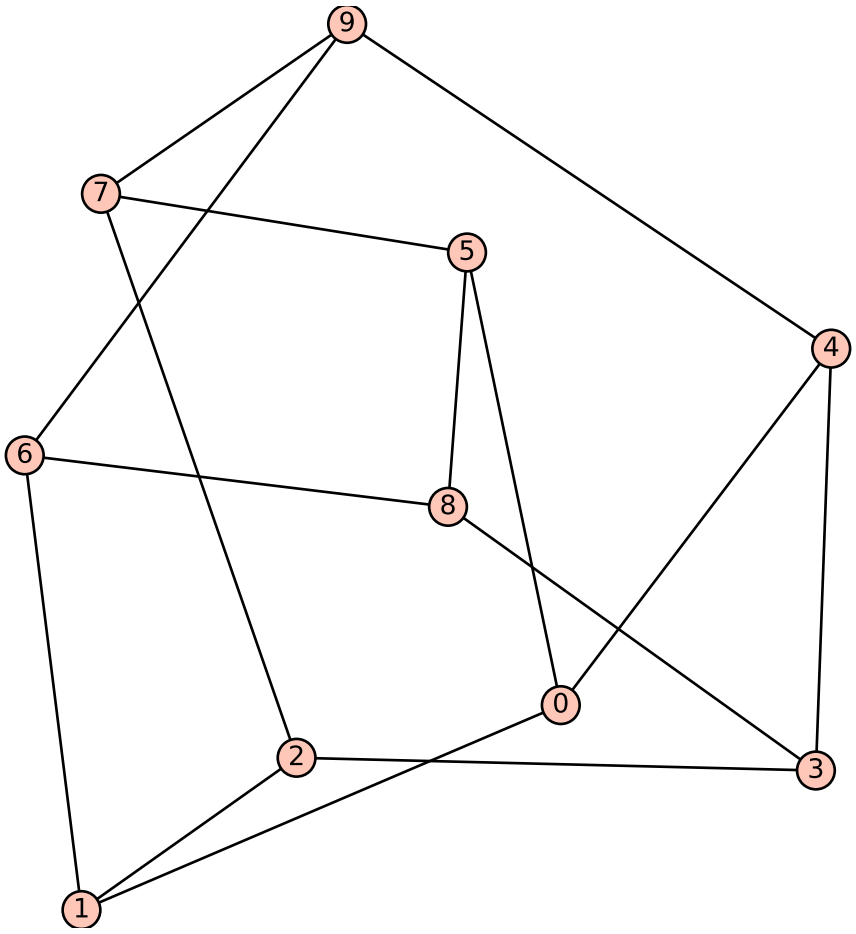
```
sage: petersen_database = graphs.PetersenGraph()
sage: petersen_database.show()  # long time
```

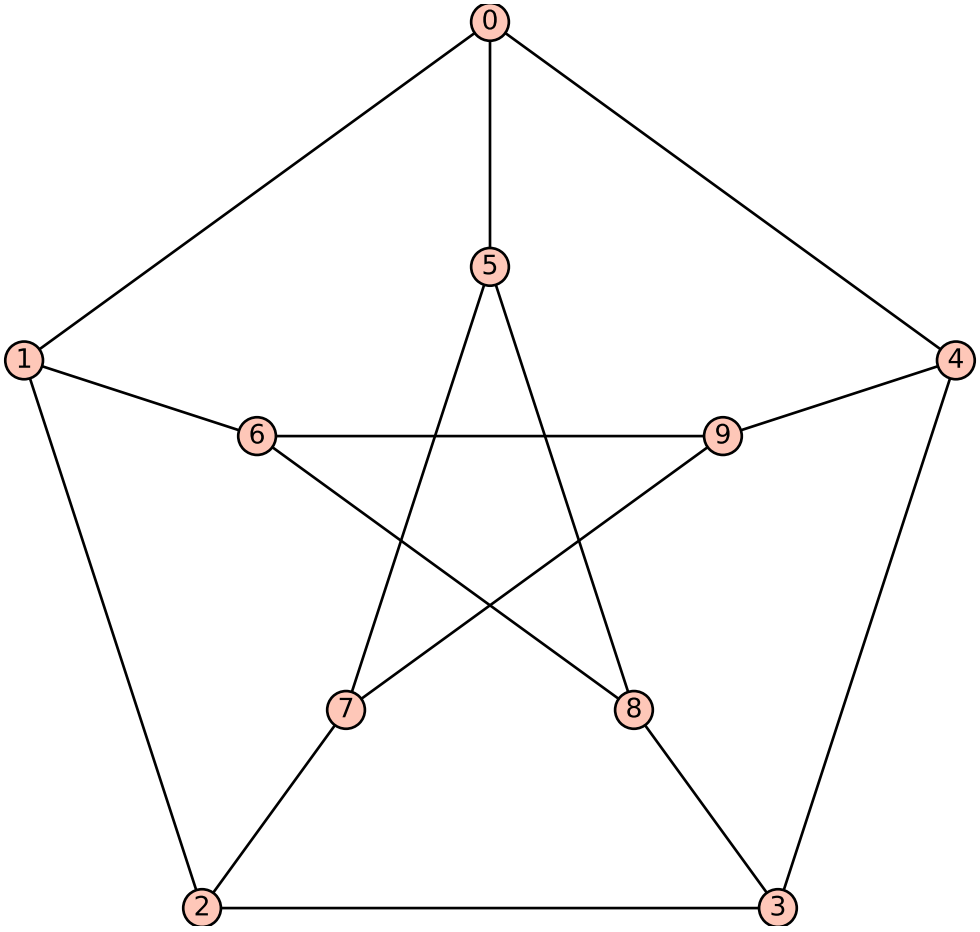
```
>>> from sage.all import *
>>> petersen_database = graphs.PetersenGraph()
>>> petersen_database.show()  # long time
```

All constructors in this database (except some random graphs) prefill the position dictionary, bypassing the spring-layout positioning algorithm.

Plot options


1
2
14
3
13
4
12
0
5
11
6
10
7
9
8


9
7
5
4
6
8
0
2
3
1


0
1
2
3
4
5
6
7
8
9

Here is the list of options accepted by *plot()* and the constructor of *GraphPlot*. Those two functions also accept all options of sage.plot.graphics.Graphics.show().

Parameter	Description	Default
layout	A layout algorithm – one of: • *'circular'* – plots the graph with vertices evenly distributed on a circle • *'spring'* – uses the traditional spring layout, using the graph's current positions as initial positions • *'ranked'* – plots the graph randomly according to the specified heights dictionary • *'graphviz'* – plots the graph using graphviz • *'planar'* – computes a planar layout using Schnyder's algorithm • *'tutte'* – uses the Tutte embedding algorithm. The graph must be a 3-connected, planar graph. • *'tree'* – the (di)graph must be a tree. One can specify the root of the tree using the tree_root parameter and the growth direction using the tree_orientation parameter. • *'forest'* – plots each connected component using the 'tree' layout • *'acyclic'* – plots an acyclic digraph so that all edges point upward	None
iterations	The number of times to execute the spring layout algorithm.	50
heights	A dictionary mapping heights to the list of vertices at this height.	None
spring	Use spring layout to finalize the current 'ranked' layout.	
tree_root	A vertex designation for drawing trees. A vertex of the tree to be used as the root for the layout='tree' option. If no root is specified, then one is chosen close to the center of the tree. Ignored unless layout='tree'.	
forest_roots	An iterable specifying which vertices to use as roots for the layout='forest' option. If no root is specified for a tree, then one is chosen close to the center of the tree. Ignored unless layout='forest'.	
tree_orientation	The direction of tree branches – 'up', 'down', 'left' or 'right'.	down
external_face	A list of the vertices of the external face of the graph if used for Tutte embedding layout; or an edge on the external face if used for the 'planar' layout.	
external_face_pos	A dictionary specifying the positions of the external face of the graph, used for Tutte embedding layout. If none specified, the external face is a regular polygon.	
save_pos	Whether or not to save the computed position for the graph.	
dim	The dimension of the layout – 2 or 3.	
prog	Which graphviz layout program to use – one of "circo", "dot", "fdp", "neato", or "twopi".	
by_component	Whether to do the spring layout by connected component – boolean.	
pos	The position dictionary of vertices. Ignored when layout is specified.	
vertex_labels	Vertex labels to draw. This can be True/False to indicate whether to print the vertex string representation of not, a dictionary keyed by vertices and associating to each vertex a label string, or a function taking as input a vertex and returning a label string.	True
vertex_label_shift	If layout is circular and we have vertex labels, will shift vertices away from center of circle in coordinate fashion (x, y).	None
vertex_color	Default color for vertices not listed in vertex_colors dictionary.	
vertex_colors	A dictionary specifying vertex colors: each key is a color recognizable by matplotlib, and each corresponding value is a list of vertices.	
vertex_size	The size to draw the vertices.	200

continues on next page

Table 1 – continued from previous page

Parameter	Description	Default
vertex_shape	The shape to draw the vertices, for example 'o' for circle or 's' for square. Whole list is available at https://matplotlib.org/api/markers_api.html. Currently unavailable for Multi-edged DiGraphs.	
edge_labels	Whether or not to draw edge labels.	False
edge_style	The linestyle of the edges. It should be one of "solid", "dashed", "dotted", "dashdot", or "-", "–", ":", "-.", respectively.	solid
edge_styles	A dictionary specifying edge styles: each key is an edge or a label (all same) and value is the linestyle of the edge. It should be one of "solid", "dashed", "dotted", "dashdot", or "-", "–", ":", "-.", respectively.	None
edge_thickness	The thickness of the edges.	1
edge_thicknesses	A dictionary specifying edge thicknesses: each key is an edge or a label (all same) and thickness of the corresponding edge.	None
edge_color	The default color for edges not listed in edge_colors.	black
edge_colors	A dictionary specifying edge colors: each key is a color recognized by matplotlib, and each corresponding value is a list of edges.	None
color_by_label	Whether to color the edges according to their labels (the colors are chosen along a rainbow). This also accepts a function or dictionary mapping labels to colors.	False
partition	A partition of the vertex set. If specified, plot will show each cell in a different color; vertex_colors takes precedence.	None
loop_size	The radius of the smallest loop.	0.075
arrowsize	Size of arrow tips.	
dist	The distance between multiedges.	0.075
max_dist	The max distance range to allow multiedges.	1.5
talk	Whether to display the vertices in talk mode (larger and white).	False
label_fontsize	font size of all labels	10
graph_border	Whether or not to draw a frame around the graph.	False
edge_labels_background	The color of the background of the edge labels.	white

Default options

This module defines two dictionaries containing default options for the *plot()* and *show()* methods. These two dictionaries are sage.graphs.graph_plot.DEFAULT_PLOT_OPTIONS and sage.graphs.graph_plot.DEFAULT_SHOW_OPTIONS, respectively.

Obviously, these values are overruled when arguments are given explicitly.

Here is how to define the default size of a graph drawing to be (6, 6). The first two calls to *show()* use this option, while the third does not (a value for figsize is explicitly given):

```
sage: import sage.graphs.graph_plot
sage: sage.graphs.graph_plot.DEFAULT_SHOW_OPTIONS['figsize'] = (6, 6)
sage: graphs.PetersenGraph().show()  # long time
sage: graphs.ChvatalGraph().show()  # long time
sage: graphs.PetersenGraph().show(figsize=(4, 4))  # long time
```

```
>>> from sage.all import *
>>> import sage.graphs.graph_plot
>>> sage.graphs.graph_plot.DEFAULT_SHOW_OPTIONS['figsize'] = (Integer(6), Integer(6))
>>> graphs.PetersenGraph().show()  # long time
```

(continues on next page)

(continued from previous page)

```
>>> graphs.ChvatalGraph().show()  # long time
>>> graphs.PetersenGraph().show(figsize=(Integer(4), Integer(4)))  # long time
```

We can now reset the default to its initial value, and now display graphs as previously:

```
sage: sage.graphs.graph_plot.DEFAULT_SHOW_OPTIONS['figsize'] = (4, 4)
sage: graphs.PetersenGraph().show()  # long time
sage: graphs.ChvatalGraph().show()  # long time
```

```
>>> from sage.all import *
>>> sage.graphs.graph_plot.DEFAULT_SHOW_OPTIONS['figsize'] = (Integer(4), Integer(4))
>>> graphs.PetersenGraph().show()  # long time
>>> graphs.ChvatalGraph().show()  # long time
```

Note

- While `DEFAULT_PLOT_OPTIONS` affects both `G.show()` and `G.plot()`, settings from `DEFAULT_SHOW_OPTIONS` only affects `G.show()`.
- In order to define a default value permanently, you can add a couple of lines to Sage's startup scripts. Example:

```
sage: import sage.graphs.graph_plot
sage: sage.graphs.graph_plot.DEFAULT_SHOW_OPTIONS['figsize'] = (4, 4)
```

```
>>> from sage.all import *
>>> import sage.graphs.graph_plot
>>> sage.graphs.graph_plot.DEFAULT_SHOW_OPTIONS['figsize'] = (Integer(4),␣
↪Integer(4))
```

Index of methods and functions

GraphPlot.set_pos()	Set the position plotting parameters for this GraphPlot.
GraphPlot.set_vertices()	Set the vertex plotting parameters for this GraphPlot.
GraphPlot.set_edges()	Set the edge (or arrow) plotting parameters for the GraphPlot object.
GraphPlot.show()	Show the (Di)Graph associated with this GraphPlot object.
GraphPlot.plot()	Return a graphics object representing the (di)graph.
GraphPlot.layout_tree()	Compute a nice layout of a tree.

class sage.graphs.graph_plot.**GraphPlot**(*graph*, *options*)

Bases: `SageObject`

Return a `GraphPlot` object, which stores all the parameters needed for plotting (Di)Graphs.

A `GraphPlot` has a plot and show function, as well as some functions to set parameters for vertices and edges. This constructor assumes default options are set. Defaults are shown in the example below.

EXAMPLES:

```
sage: from sage.graphs.graph_plot import GraphPlot
sage: options = {
....:     'vertex_size': 200,
```

(continues on next page)

(continued from previous page)

```
....:     'vertex_labels': True,
....:     'layout': None,
....:     'edge_style': 'solid',
....:     'edge_color': 'black',
....:     'edge_colors': None,
....:     'edge_labels': False,
....:     'iterations': 50,
....:     'tree_orientation': 'down',
....:     'heights': None,
....:     'graph_border': False,
....:     'talk': False,
....:     'color_by_label': False,
....:     'partition': None,
....:     'dist': .075,
....:     'max_dist': 1.5,
....:     'loop_size': .075,
....:     'edge_labels_background': 'transparent'}
sage: g = Graph({0: [1, 2], 2: [3], 4: [0, 1]})
sage: GP = GraphPlot(g, options)
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_plot import GraphPlot
>>> options = {
...     'vertex_size': Integer(200),
...     'vertex_labels': True,
...     'layout': None,
...     'edge_style': 'solid',
...     'edge_color': 'black',
...     'edge_colors': None,
...     'edge_labels': False,
...     'iterations': Integer(50),
...     'tree_orientation': 'down',
...     'heights': None,
...     'graph_border': False,
...     'talk': False,
...     'color_by_label': False,
...     'partition': None,
...     'dist': RealNumber('.075'),
...     'max_dist': RealNumber('1.5'),
...     'loop_size': RealNumber('.075'),
...     'edge_labels_background': 'transparent'}
>>> g = Graph({Integer(0): [Integer(1), Integer(2)], Integer(2): [Integer(3)],␣
↪Integer(4): [Integer(0), Integer(1)]})
>>> GP = GraphPlot(g, options)
```

layout_tree(*root*, *orientation*)

Compute a nice layout of a tree.

INPUT:

- `root` – the root vertex
- `orientation` – whether to place the root at the top or at the bottom:

- `orientation="down"` – children are placed below their parent
- `orientation="top"` – children are placed above their parent

EXAMPLES:

```
sage: from sage.graphs.graph_plot import GraphPlot
sage: G = graphs.HoffmanSingletonGraph()
sage: T = Graph()
sage: T.add_edges(G.min_spanning_tree(starting_vertex=0))
sage: T.show(layout='tree', tree_root=0)  # indirect doctest
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_plot import GraphPlot
>>> G = graphs.HoffmanSingletonGraph()
>>> T = Graph()
>>> T.add_edges(G.min_spanning_tree(starting_vertex=Integer(0)))
>>> T.show(layout='tree', tree_root=Integer(0))  # indirect doctest
```

plot(***kwds*)

Return a graphics object representing the (di)graph.

INPUT:

The options accepted by this method are to be found in the documentation of the *sage.graphs.graph_plot* module, and the `show()` method.

Note

See *the module's documentation* for information on default values of this method.

We can specify some pretty precise plotting of familiar graphs:

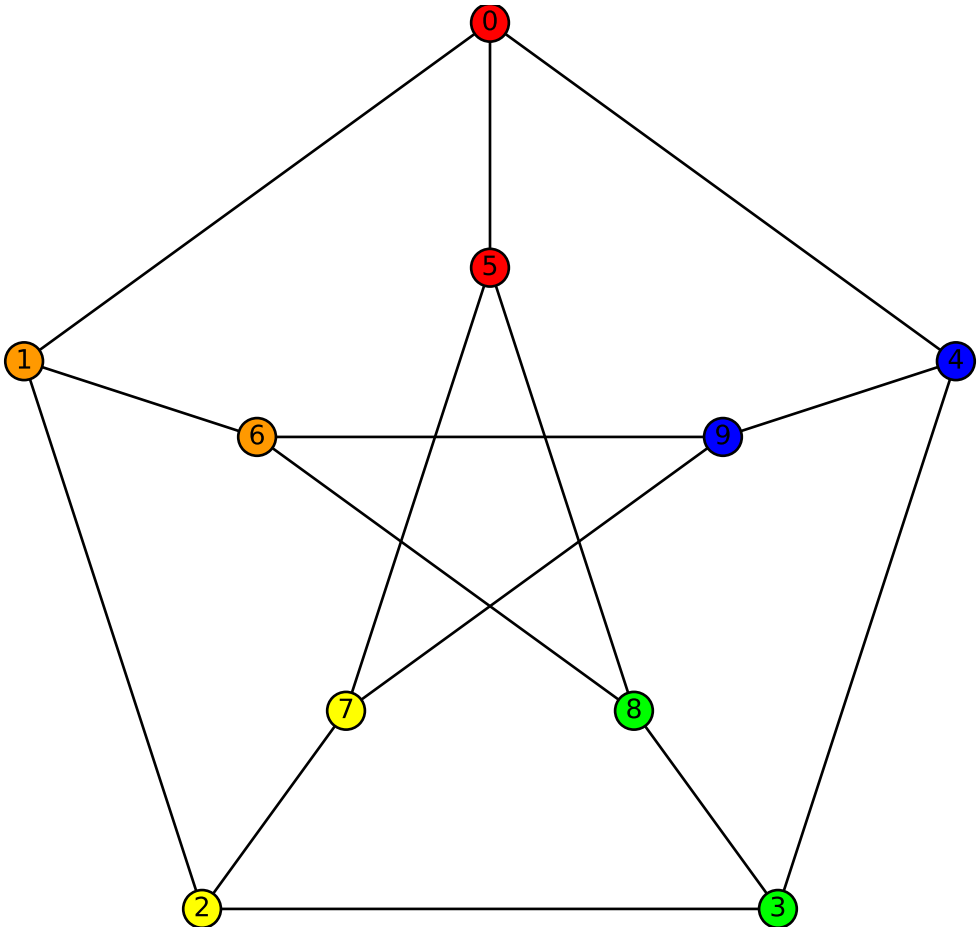
```
sage: from math import sin, cos, pi
sage: P = graphs.PetersenGraph()
sage: d = {'#FF0000': [0, 5], '#FF9900': [1, 6], '#FFFF00': [2, 7],
....:      '#00FF00': [3, 8], '#0000FF': [4,9]}
sage: pos_dict = {}
sage: for i in range(5):
....:  x = float(cos(pi/2 + ((2*pi)/5)*i))
....:  y = float(sin(pi/2 + ((2*pi)/5)*i))
....:  pos_dict[i] = [x,y]
...
sage: for i in range(5, 10):
....:  x = float(0.5*cos(pi/2 + ((2*pi)/5)*i))
....:  y = float(0.5*sin(pi/2 + ((2*pi)/5)*i))
....:  pos_dict[i] = [x,y]
...
sage: pl = P.graphplot(pos=pos_dict, vertex_colors=d)
sage: pl.show()
```

```
>>> from sage.all import *
>>> from math import sin, cos, pi
>>> P = graphs.PetersenGraph()
```

(continues on next page)

(continued from previous page)

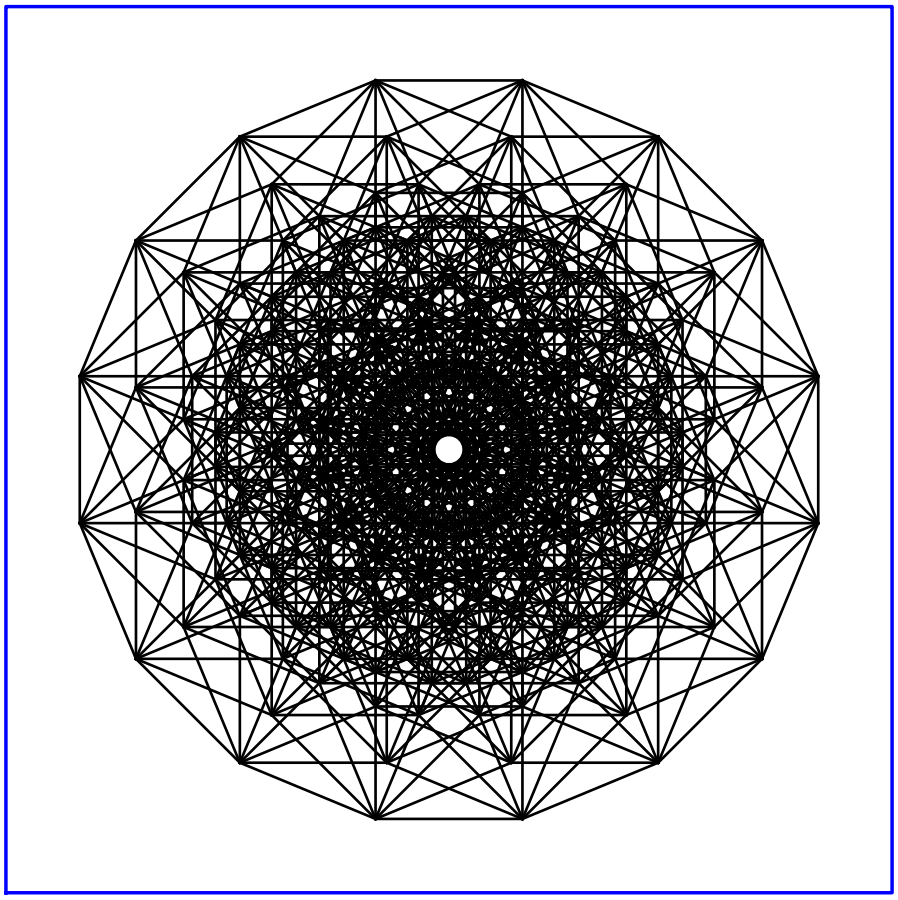
```
>>> d = {'#FF0000': [Integer(0), Integer(5)], '#FF9900': [Integer(1),␣
↪Integer(6)], '#FFFF00': [Integer(2), Integer(7)],
...       '#00FF00': [Integer(3), Integer(8)], '#0000FF': [Integer(4),
↪Integer(9)]}
>>> pos_dict = {}
>>> for i in range(Integer(5)):
...   x = float(cos(pi/Integer(2) + ((Integer(2)*pi)/Integer(5))*i))
...   y = float(sin(pi/Integer(2) + ((Integer(2)*pi)/Integer(5))*i))
...   pos_dict[i] = [x,y]
...
>>> for i in range(Integer(5), Integer(10)):
...   x = float(RealNumber('0.5')*cos(pi/Integer(2) + ((Integer(2)*pi)/
↪Integer(5))*i))
...   y = float(RealNumber('0.5')*sin(pi/Integer(2) + ((Integer(2)*pi)/
↪Integer(5))*i))
...   pos_dict[i] = [x,y]
...
>>> pl = P.graphplot(pos=pos_dict, vertex_colors=d)
>>> pl.show()
```



Here are some more common graphs with typical options:

```
sage: C = graphs.CubeGraph(8)
sage: P = C.graphplot(vertex_labels=False, vertex_size=0,
....:                 graph_border=True)
sage: P.show()
```

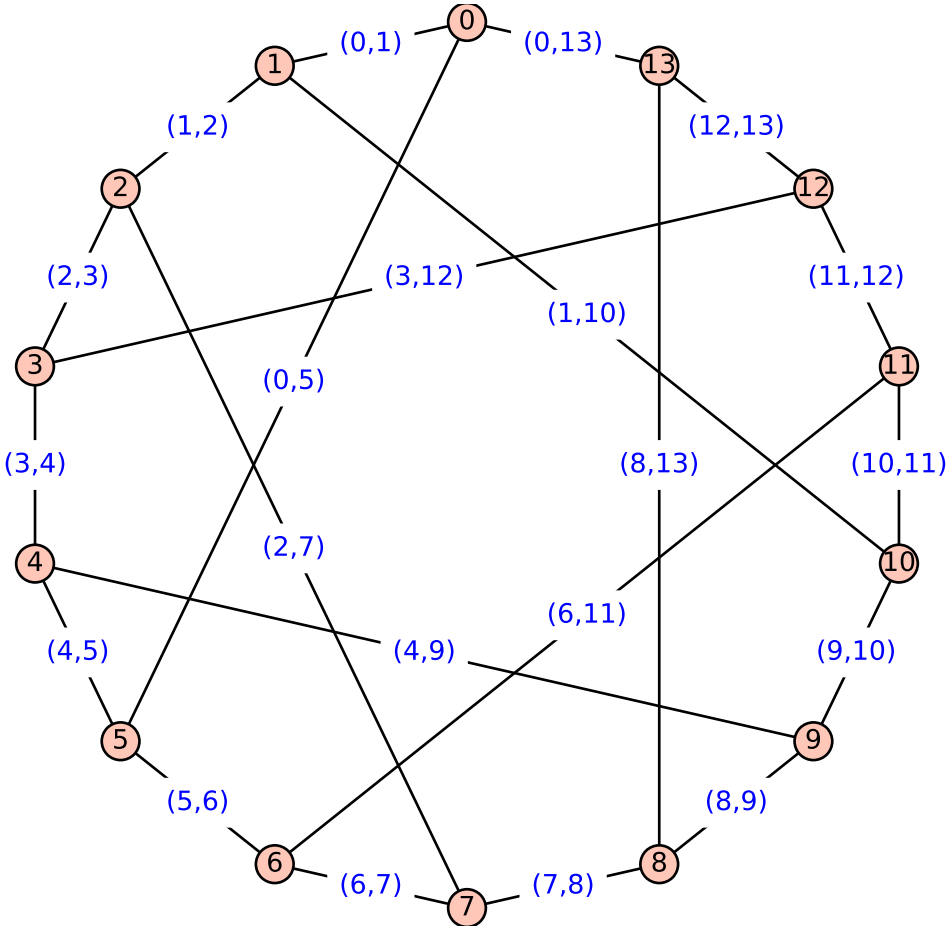
```
>>> from sage.all import *
>>> C = graphs.CubeGraph(Integer(8))
>>> P = C.graphplot(vertex_labels=False, vertex_size=Integer(0),
...                 graph_border=True)
>>> P.show()
```



```
sage: G = graphs.HeawoodGraph().copy(sparse=True)
sage: for u, v, l in G.edges(sort=True):
....:     G.set_edge_label(u, v, f'({u},{v})')
sage: G.graphplot(edge_labels=True).show()
```

```
>>> from sage.all import *
>>> G = graphs.HeawoodGraph().copy(sparse=True)
>>> for u, v, l in G.edges(sort=True):
...     G.set_edge_label(u, v, f'({u},{v})')
>>> G.graphplot(edge_labels=True).show()
```

The options for plotting also work with directed graphs:


0
1
13
2
12
3
11
4
10
5
9
6
8
7
(0,1)
(0,13)
(1,2)
(12,13)
(2,3)
(3,12)
(11,12)
(1,10)
(0,5)
(3,4)
(8,13)
(10,11)
(2,7)
(6,11)
(4,5)
(4,9)
(9,10)
(5,6)
(8,9)
(6,7)
(7,8)

```
sage: D = DiGraph({
....:     0: [1, 10, 19], 1: [8, 2], 2: [3, 6], 3: [19, 4],
....:     4: [17, 5], 5: [6, 15], 6: [7], 7: [8, 14], 8: [9],
....:     9: [10, 13], 10: [11], 11: [12, 18], 12: [16, 13],
....:     13: [14], 14: [15], 15: [16], 16: [17], 17: [18],
....:     18: [19], 19: []})
sage: for u, v, l in D.edges(sort=True):
....:     D.set_edge_label(u, v, f'({u},{v})')
sage: D.graphplot(edge_labels=True, layout='circular').show()
```

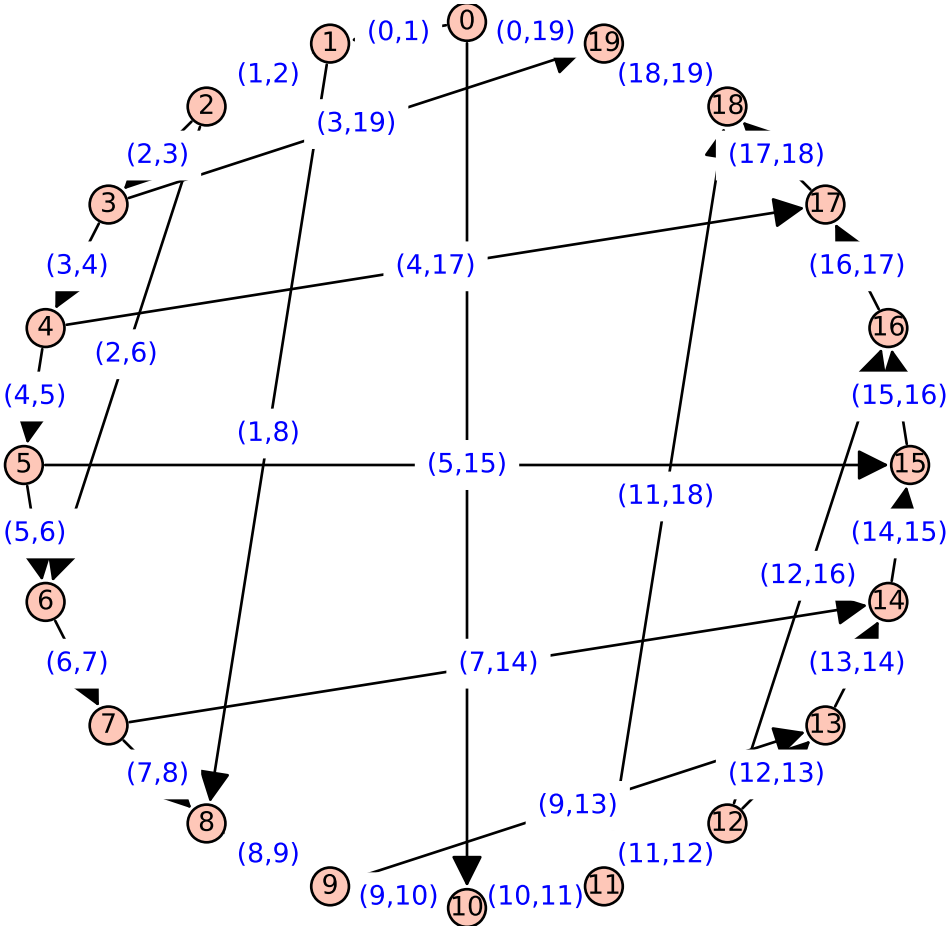
```
>>> from sage.all import *
>>> D = DiGraph({
...     Integer(0): [Integer(1), Integer(10), Integer(19)], Integer(1):␣
↪[Integer(8), Integer(2)], Integer(2): [Integer(3), Integer(6)], Integer(3):␣
↪[Integer(19), Integer(4)],
...     Integer(4): [Integer(17), Integer(5)], Integer(5): [Integer(6),␣
↪Integer(15)], Integer(6): [Integer(7)], Integer(7): [Integer(8),␣
↪Integer(14)], Integer(8): [Integer(9)],
...     Integer(9): [Integer(10), Integer(13)], Integer(10): [Integer(11)],␣
↪Integer(11): [Integer(12), Integer(18)], Integer(12): [Integer(16),␣
↪Integer(13)],
...     Integer(13): [Integer(14)], Integer(14): [Integer(15)], Integer(15):␣
↪[Integer(16)], Integer(16): [Integer(17)], Integer(17): [Integer(18)],
...     Integer(18): [Integer(19)], Integer(19): []})
>>> for u, v, l in D.edges(sort=True):
...     D.set_edge_label(u, v, f'({u},{v})')
>>> D.graphplot(edge_labels=True, layout='circular').show()
```

For graphs with `circular` layouts, one may shift the vertex labels by specifying coordinates to shift by:

```
sage: D = DiGraph({
....:     0: [1, 10, 19], 1: [8, 2], 2: [3, 6], 3: [19, 4],
....:     4: [17, 5], 5: [6, 15], 6: [7], 7: [8, 14], 8: [9],
....:     9: [10, 13], 10: [11], 11: [12, 18], 12: [16, 13],
....:     13: [14], 14: [15], 15: [16], 16: [17], 17: [18],
....:     18: [19], 19: []})
sage: for u, v, l in D.edges(sort=True):
....:     D.set_edge_label(u, v, f'({u},{v})')
sage: D.graphplot(edge_labels=True, layout='circular', vertex_label_shift=(15,
↪10)).show()
```

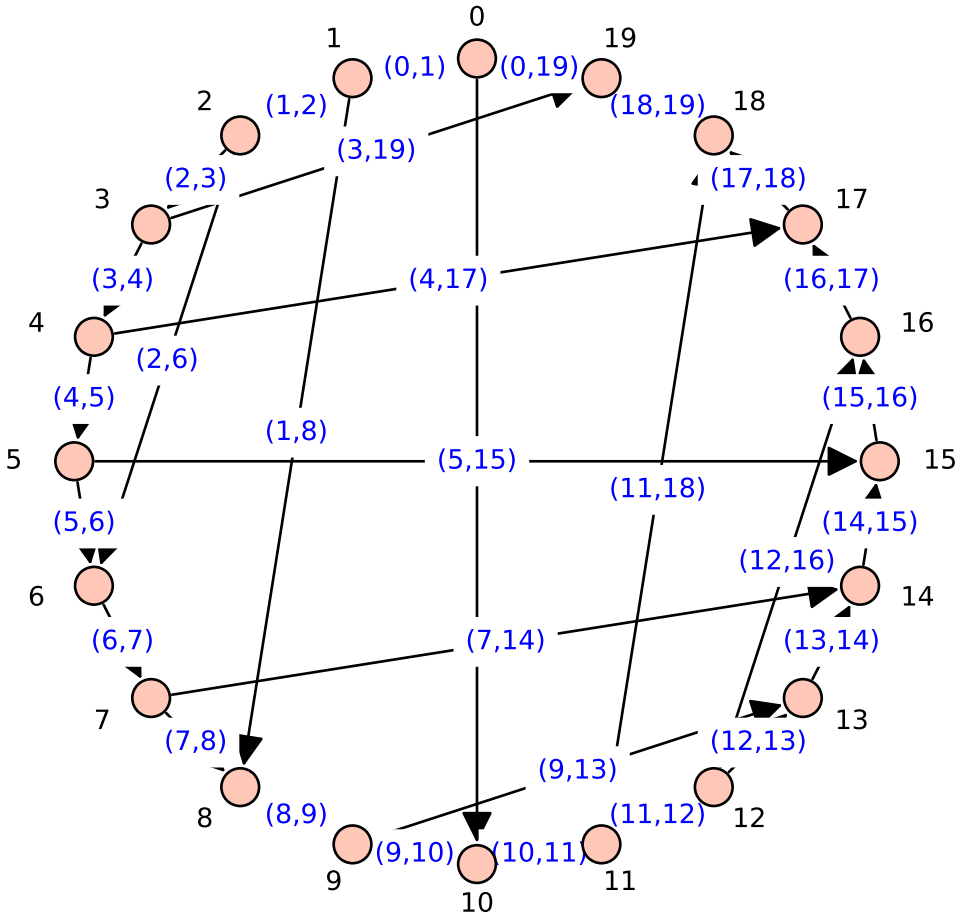
```
>>> from sage.all import *
>>> D = DiGraph({
...     Integer(0): [Integer(1), Integer(10), Integer(19)], Integer(1):␣
↪[Integer(8), Integer(2)], Integer(2): [Integer(3), Integer(6)], Integer(3):␣
↪[Integer(19), Integer(4)],
...     Integer(4): [Integer(17), Integer(5)], Integer(5): [Integer(6),␣
↪Integer(15)], Integer(6): [Integer(7)], Integer(7): [Integer(8),␣
↪Integer(14)], Integer(8): [Integer(9)],
...     Integer(9): [Integer(10), Integer(13)], Integer(10): [Integer(11)],␣
↪Integer(11): [Integer(12), Integer(18)], Integer(12): [Integer(16),␣
↪Integer(13)],
```

(continues on next page)


0
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
(0,1)
(0,19)
(1,2)
(18,19)
(3,19)
(2,3)
(17,18)
(3,4)
(4,17)
(16,17)
(2,6)
(4,5)
(15,16)
(1,8)
(5,15)
(11,18)
(5,6)
(14,15)
(12,16)
(6,7)
(7,14)
(13,14)
(7,8)
(12,13)
(9,13)
(8,9)
(11,12)
(9,10)
(10,11)

(continued from previous page)

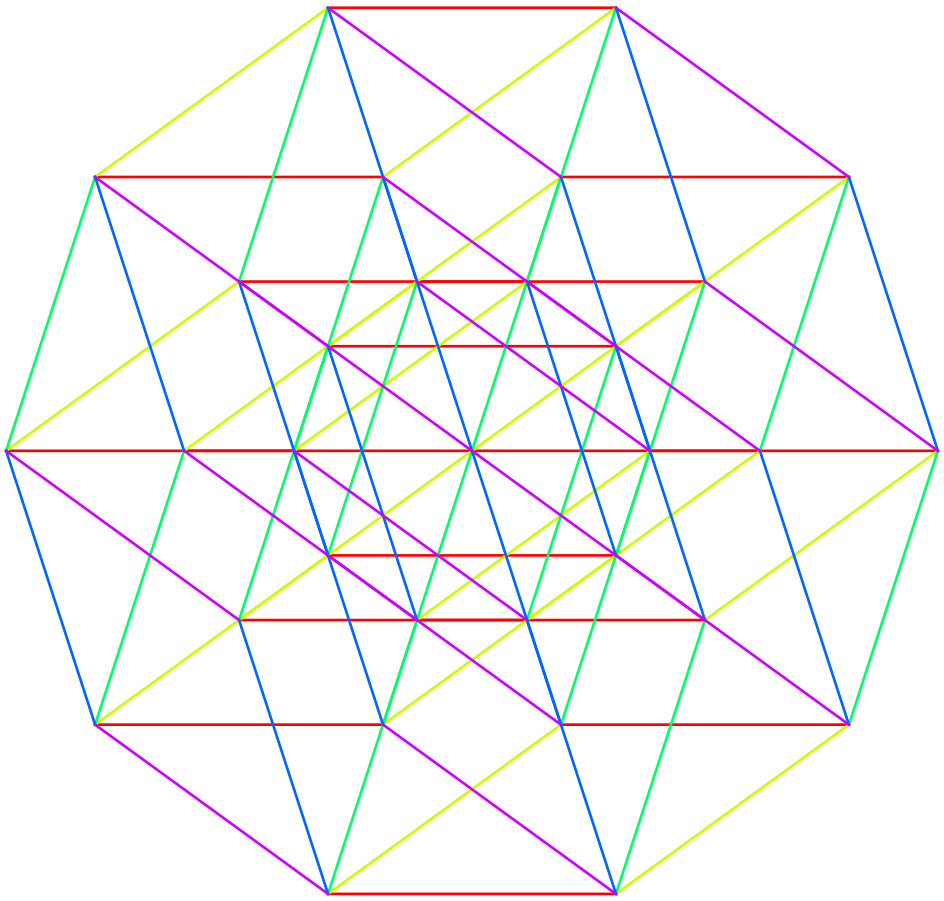
```
...       Integer(13): [Integer(14)], Integer(14): [Integer(15)], Integer(15):␣
↪[Integer(16)], Integer(16): [Integer(17)], Integer(17): [Integer(18)],
...       Integer(18): [Integer(19)], Integer(19): []})
>>> for u, v, l in D.edges(sort=True):
...     D.set_edge_label(u, v, f'({u},{v})')
>>> D.graphplot(edge_labels=True, layout='circular', vertex_label_
↪shift=(Integer(15),Integer(10))).show()
```



This example shows off the coloring of edges:

```
sage: from sage.plot.colors import rainbow
sage: C = graphs.CubeGraph(5)
sage: R = rainbow(5)
sage: edge_colors = {}
sage: for i in range(5):
....:     edge_colors[R[i]] = []
sage: for u, v, l in C.edges(sort=True):
....:     for i in range(5):
....:         if u[i] != v[i]:
....:             edge_colors[R[i]].append((u, v, l))
sage: C.graphplot(vertex_labels=False, vertex_size=0,
....:             edge_colors=edge_colors).show()
```

```
>>> from sage.all import *
>>> from sage.plot.colors import rainbow
>>> C = graphs.CubeGraph(Integer(5))
>>> R = rainbow(Integer(5))
>>> edge_colors = {}
>>> for i in range(Integer(5)):
...     edge_colors[R[i]] = []
>>> for u, v, l in C.edges(sort=True):
...     for i in range(Integer(5)):
...         if u[i] != v[i]:
...             edge_colors[R[i]].append((u, v, l))
>>> C.graphplot(vertex_labels=False, vertex_size=Integer(0),
...             edge_colors=edge_colors).show()
```



With the `partition` option, we can separate out same-color groups of vertices:

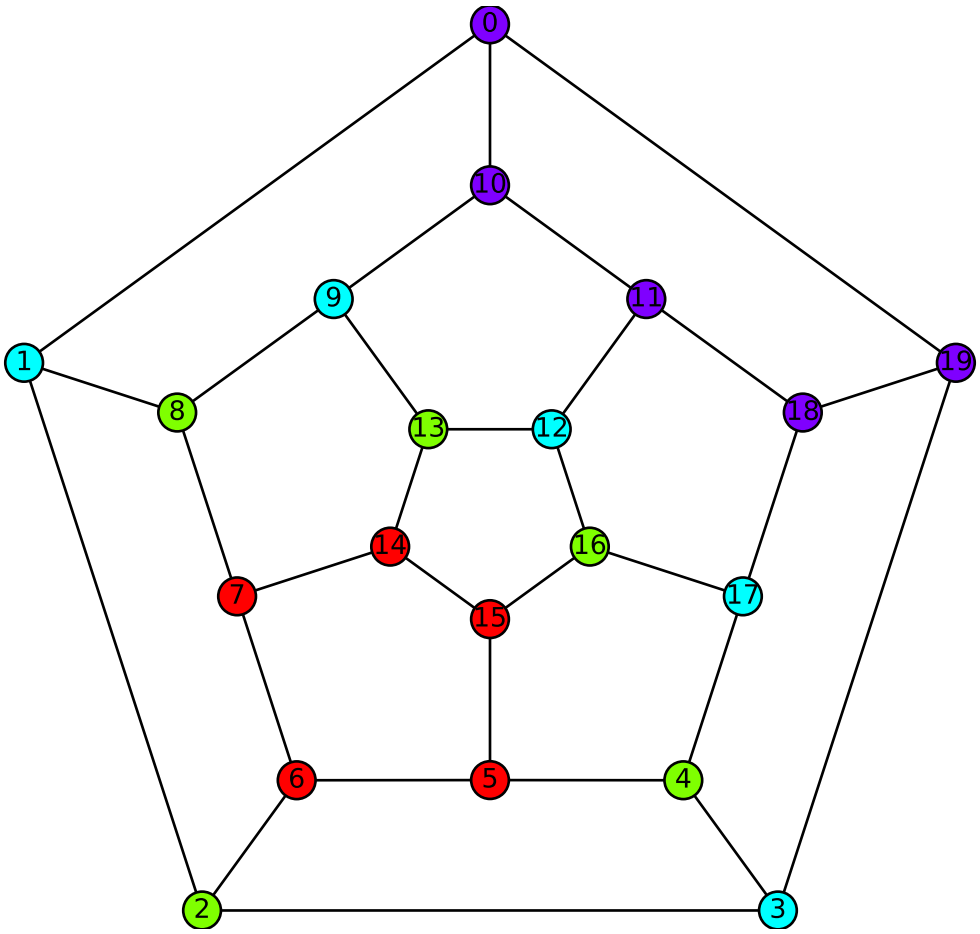
```
sage: D = graphs.DodecahedralGraph()
sage: Pi = [[6, 5, 15, 14, 7], [16, 13, 8, 2, 4],
....:       [12, 17, 9, 3, 1], [0, 19, 18, 10, 11]]
sage: D.show(partition=Pi)
```

```
>>> from sage.all import *
```

(continues on next page)

(continued from previous page)

```
>>> D = graphs.DodecahedralGraph()
>>> Pi = [[Integer(6), Integer(5), Integer(15), Integer(14), Integer(7)],␣
↪[Integer(16), Integer(13), Integer(8), Integer(2), Integer(4)],
...       [Integer(12), Integer(17), Integer(9), Integer(3), Integer(1)],␣
↪[Integer(0), Integer(19), Integer(18), Integer(10), Integer(11)]]
>>> D.show(partition=Pi)
```



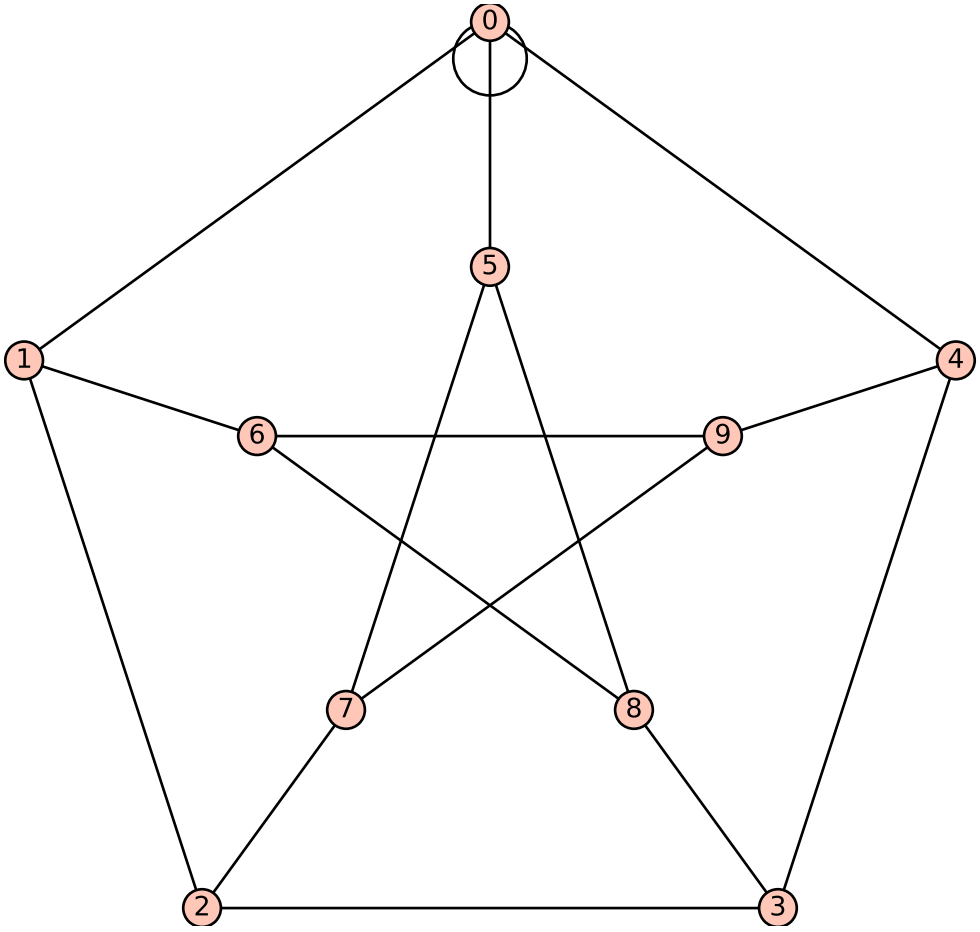
Loops are also plotted correctly:

```
sage: G = graphs.PetersenGraph()
sage: G.allow_loops(True)
sage: G.add_edge(0,0)
sage: G.show()
```

```
>>> from sage.all import *
>>> G = graphs.PetersenGraph()
>>> G.allow_loops(True)
>>> G.add_edge(Integer(0),Integer(0))
>>> G.show()
```

```
sage: D = DiGraph({0:[0,1], 1:[2], 2:[3]}, loops=True)
```

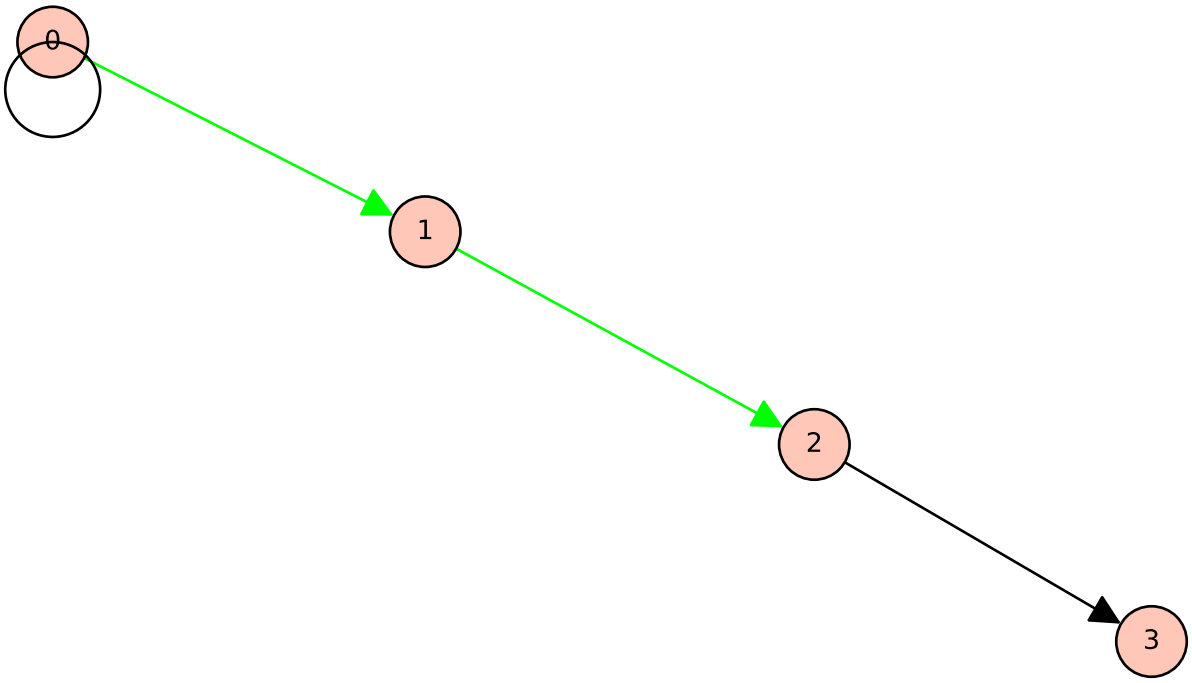
(continues on next page)


0
1
2
3
4
5
6
7
8
9

(continued from previous page)

```
sage: D.show()
sage: D.show(edge_colors={(0, 1, 0): [(0, 1, None), (1, 2, None)],
....:                     (0, 0, 0): [(2, 3, None)]})
```

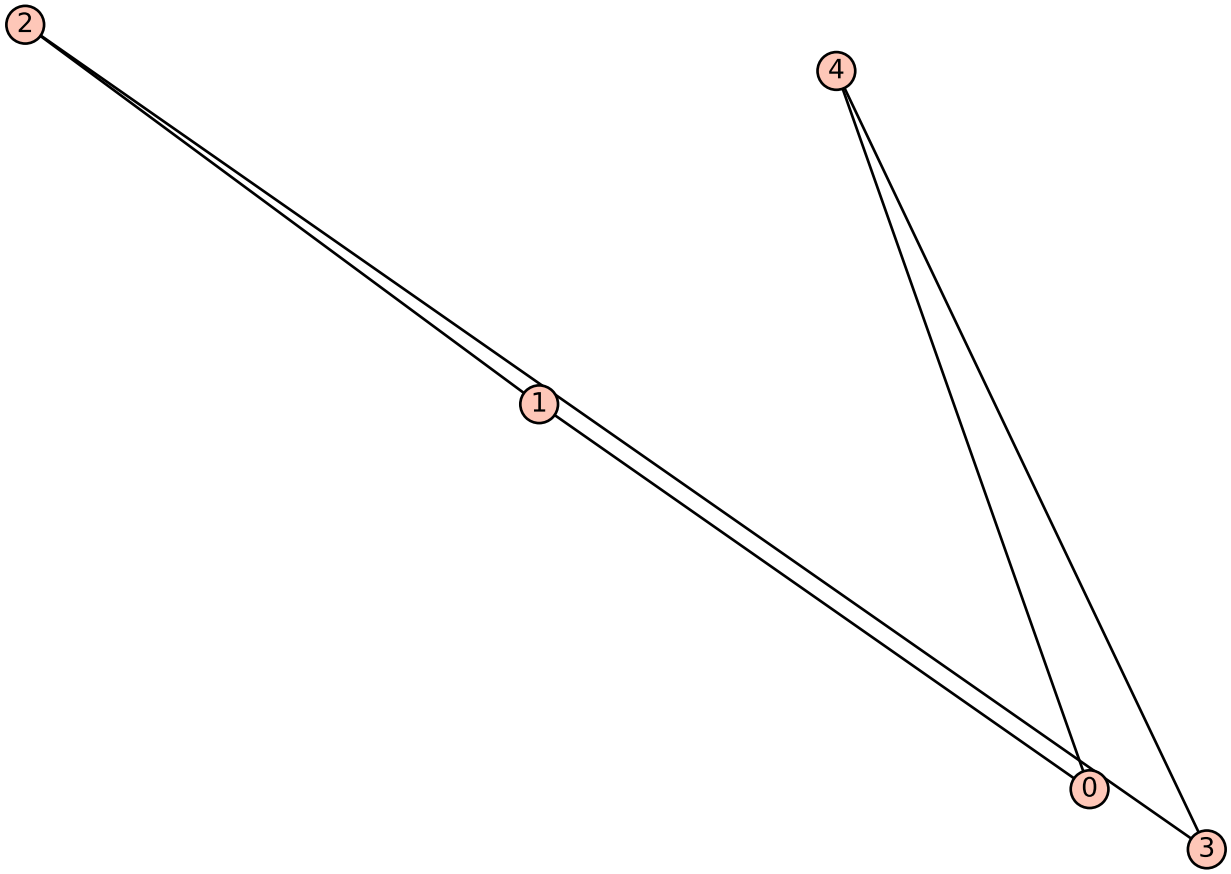
```
>>> from sage.all import *
>>> D = DiGraph({Integer(0):[Integer(0),Integer(1)], Integer(1):[Integer(2)],␣
↪Integer(2):[Integer(3)]}, loops=True)
>>> D.show()
>>> D.show(edge_colors={(Integer(0), Integer(1), Integer(0)): [(Integer(0),␣
↪Integer(1), None), (Integer(1), Integer(2), None)],
...                     (Integer(0), Integer(0), Integer(0)): [(Integer(2),␣
↪Integer(3), None)]})
```



More options:

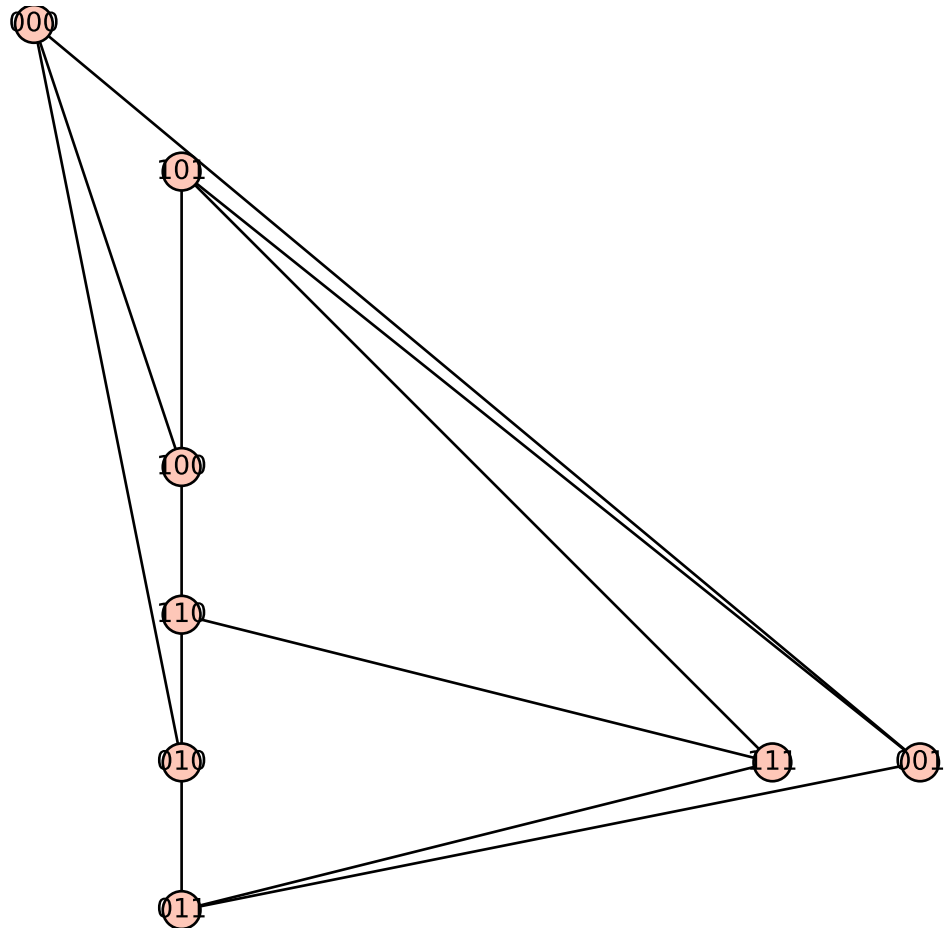
```
sage: pos = {0: [0.0, 1.5], 1: [-0.8, 0.3], 2: [-0.6, -0.8],
....:        3:[0.6, -0.8], 4:[0.8, 0.3]}
sage: g = Graph({0: [1], 1: [2], 2: [3], 3: [4], 4: [0]})
sage: g.graphplot(pos=pos, layout='spring', iterations=0).plot()
Graphics object consisting of 11 graphics primitives
```

```
>>> from sage.all import *
>>> pos = {Integer(0): [RealNumber('0.0'), RealNumber('1.5')], Integer(1): [-
↪RealNumber('0.8'), RealNumber('0.3')], Integer(2): [-RealNumber('0.6'), -
↪RealNumber('0.8')],
...         Integer(3):[RealNumber('0.6'), -RealNumber('0.8')],␣
↪Integer(4):[RealNumber('0.8'), RealNumber('0.3')]}
>>> g = Graph({Integer(0): [Integer(1)], Integer(1): [Integer(2)],␣
↪Integer(2): [Integer(3)], Integer(3): [Integer(4)], Integer(4):␣
↪[Integer(0)]})
>>> g.graphplot(pos=pos, layout='spring', iterations=Integer(0)).plot()
Graphics object consisting of 11 graphics primitives
```




```
sage: D = graphs.CubeGraph(3)
sage: D.graphplot(layout='planar').plot()
Graphics object consisting of 21 graphics primitives
```

```
>>> from sage.all import *
>>> D = graphs.CubeGraph(Integer(3))
>>> D.graphplot(layout='planar').plot()
Graphics object consisting of 21 graphics primitives
```


000
101
100
110
010
111
001
011

```
sage: G = Graph()
sage: P = G.graphplot().plot()
sage: P.axes()
False
sage: G = DiGraph()
sage: P = G.graphplot().plot()
sage: P.axes()
False
```

```
>>> from sage.all import *
>>> G = Graph()
>>> P = G.graphplot().plot()
>>> P.axes()
False
>>> G = DiGraph()
>>> P = G.graphplot().plot()
>>> P.axes()
False
```

We can plot multiple graphs:

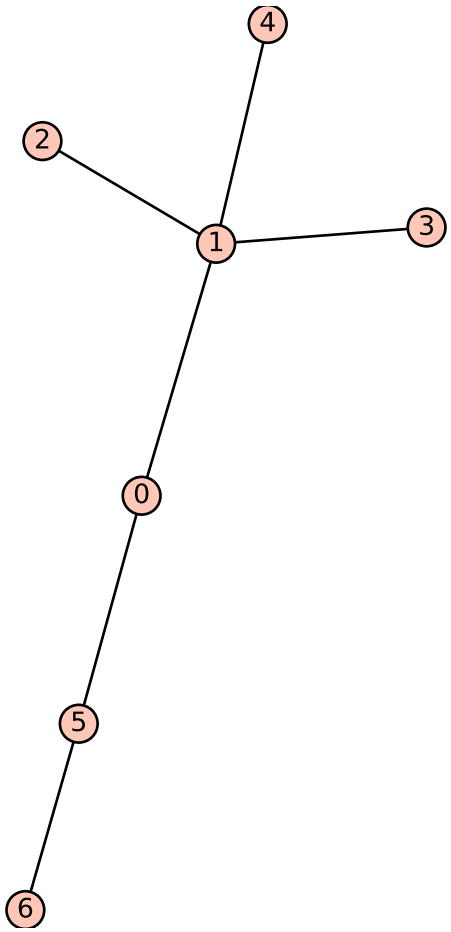
```
sage: T = list(graphs.trees(7))
sage: t = T[3]
sage: t.graphplot(heights={0: [0], 1: [4, 5, 1],
....:                      2: [2], 3: [3, 6]}
....:             ).plot()
Graphics object consisting of 14 graphics primitives
```

```
>>> from sage.all import *
>>> T = list(graphs.trees(Integer(7)))
>>> t = T[Integer(3)]
>>> t.graphplot(heights={Integer(0): [Integer(0)], Integer(1): [Integer(4),␣
↪Integer(5), Integer(1)],
...                      Integer(2): [Integer(2)], Integer(3): [Integer(3),␣
↪Integer(6)]}
...             ).plot()
Graphics object consisting of 14 graphics primitives
```

```
sage: T = list(graphs.trees(7))
sage: t = T[3]
sage: t.graphplot(heights={0: [0], 1: [4, 5, 1],
....:                      2: [2], 3: [3, 6]}
....:             ).plot()
Graphics object consisting of 14 graphics primitives
```

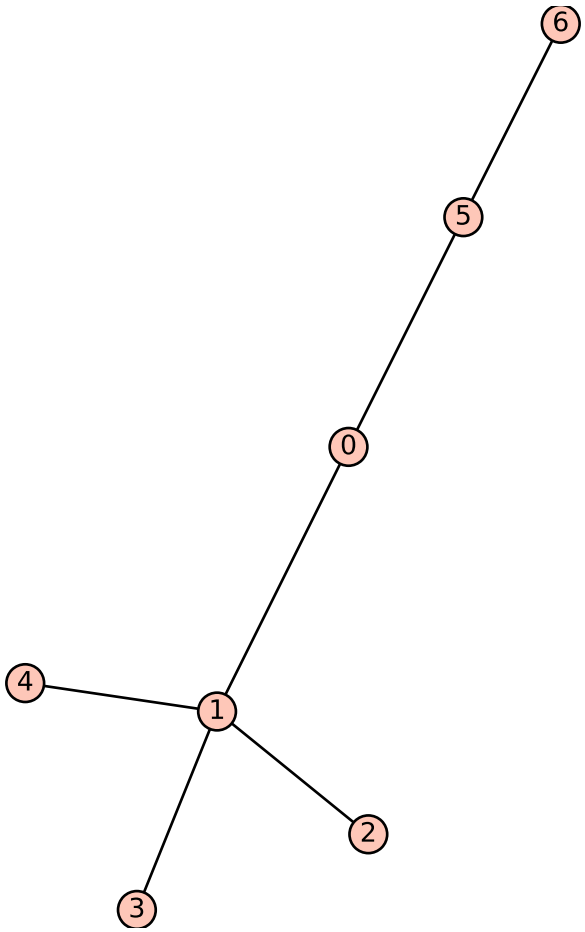
```
>>> from sage.all import *
>>> T = list(graphs.trees(Integer(7)))
>>> t = T[Integer(3)]
>>> t.graphplot(heights={Integer(0): [Integer(0)], Integer(1): [Integer(4),␣
↪Integer(5), Integer(1)],
...                      Integer(2): [Integer(2)], Integer(3): [Integer(3),␣
↪Integer(6)]}
```

(continues on next page)


4
2
3
1
0
5
6

(continued from previous page)

```
...            ).plot()
Graphics object consisting of 14 graphics primitives
```



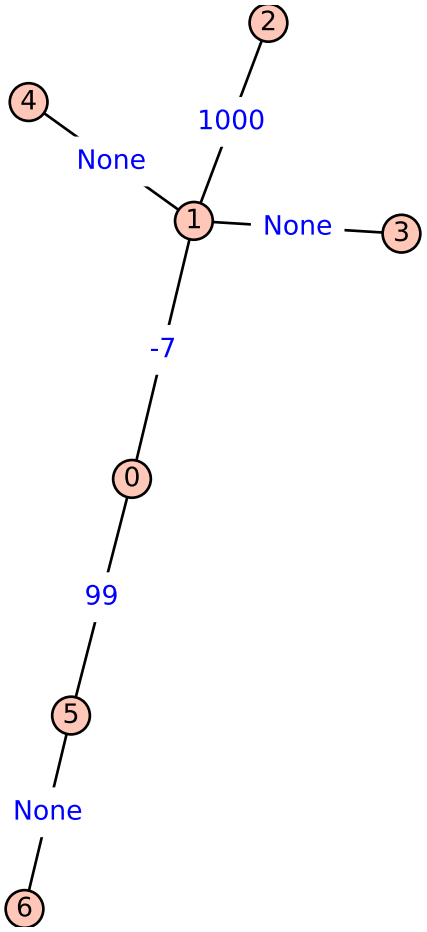

```
sage: t.set_edge_label(0, 1, -7)
sage: t.set_edge_label(0, 5, 3)
sage: t.set_edge_label(0, 5, 99)
sage: t.set_edge_label(1, 2, 1000)
sage: t.set_edge_label(3, 2, 'spam')
sage: t.set_edge_label(2, 6, 3/2)
sage: t.set_edge_label(0, 4, 66)
sage: t.graphplot(heights={0: [0], 1: [4, 5, 1],
....:                      2: [2], 3: [3, 6]},
....:             edge_labels=True
....:            ).plot()
Graphics object consisting of 20 graphics primitives
```

```
>>> from sage.all import *
>>> t.set_edge_label(Integer(0), Integer(1), -Integer(7))
>>> t.set_edge_label(Integer(0), Integer(5), Integer(3))
>>> t.set_edge_label(Integer(0), Integer(5), Integer(99))
>>> t.set_edge_label(Integer(1), Integer(2), Integer(1000))
```

(continues on next page)

(continued from previous page)

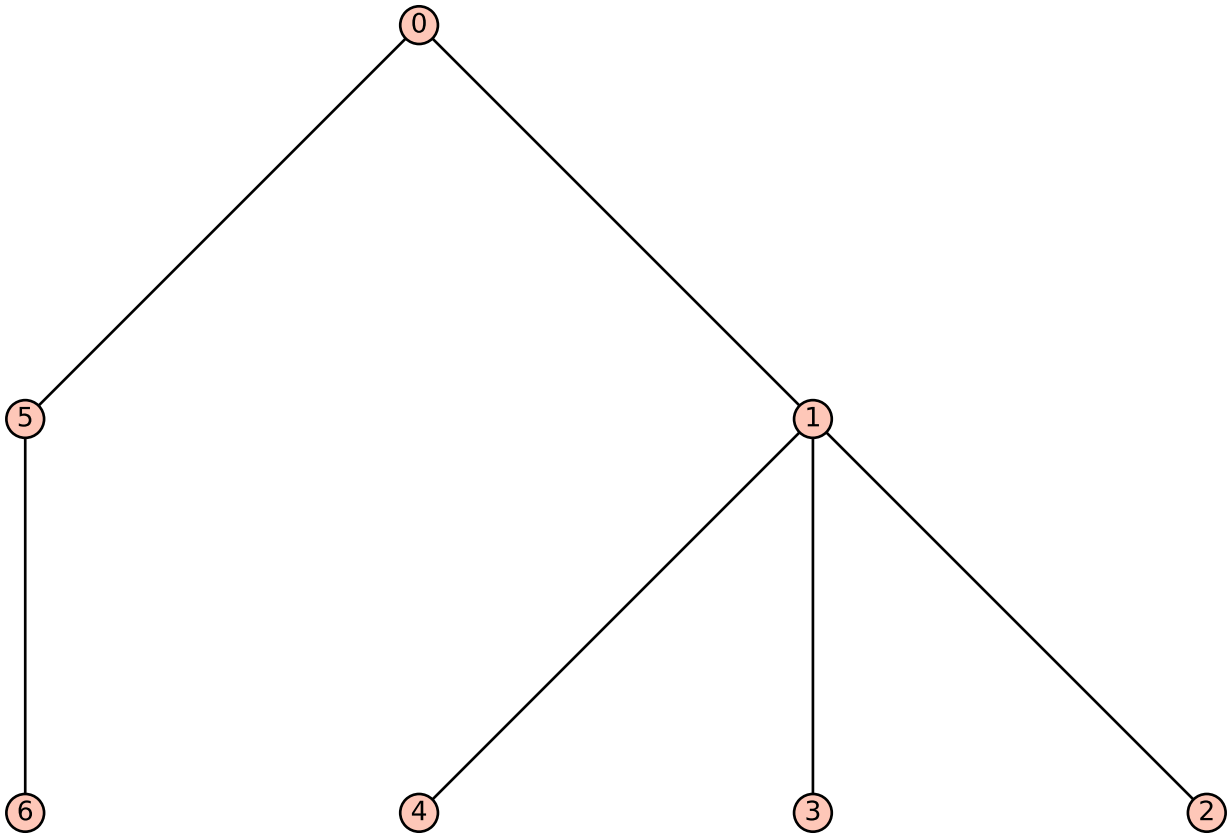
```
>>> t.set_edge_label(Integer(3), Integer(2), 'spam')
>>> t.set_edge_label(Integer(2), Integer(6), Integer(3)/Integer(2))
>>> t.set_edge_label(Integer(0), Integer(4), Integer(66))
>>> t.graphplot(heights={Integer(0): [Integer(0)], Integer(1): [Integer(4),␣
↪Integer(5), Integer(1)],
...                       Integer(2): [Integer(2)], Integer(3): [Integer(3),␣
↪Integer(6)]},
...              edge_labels=True
...             ).plot()
Graphics object consisting of 20 graphics primitives
```




```
sage: T = list(graphs.trees(7))
sage: t = T[3]
sage: t.graphplot(layout='tree').show()
```

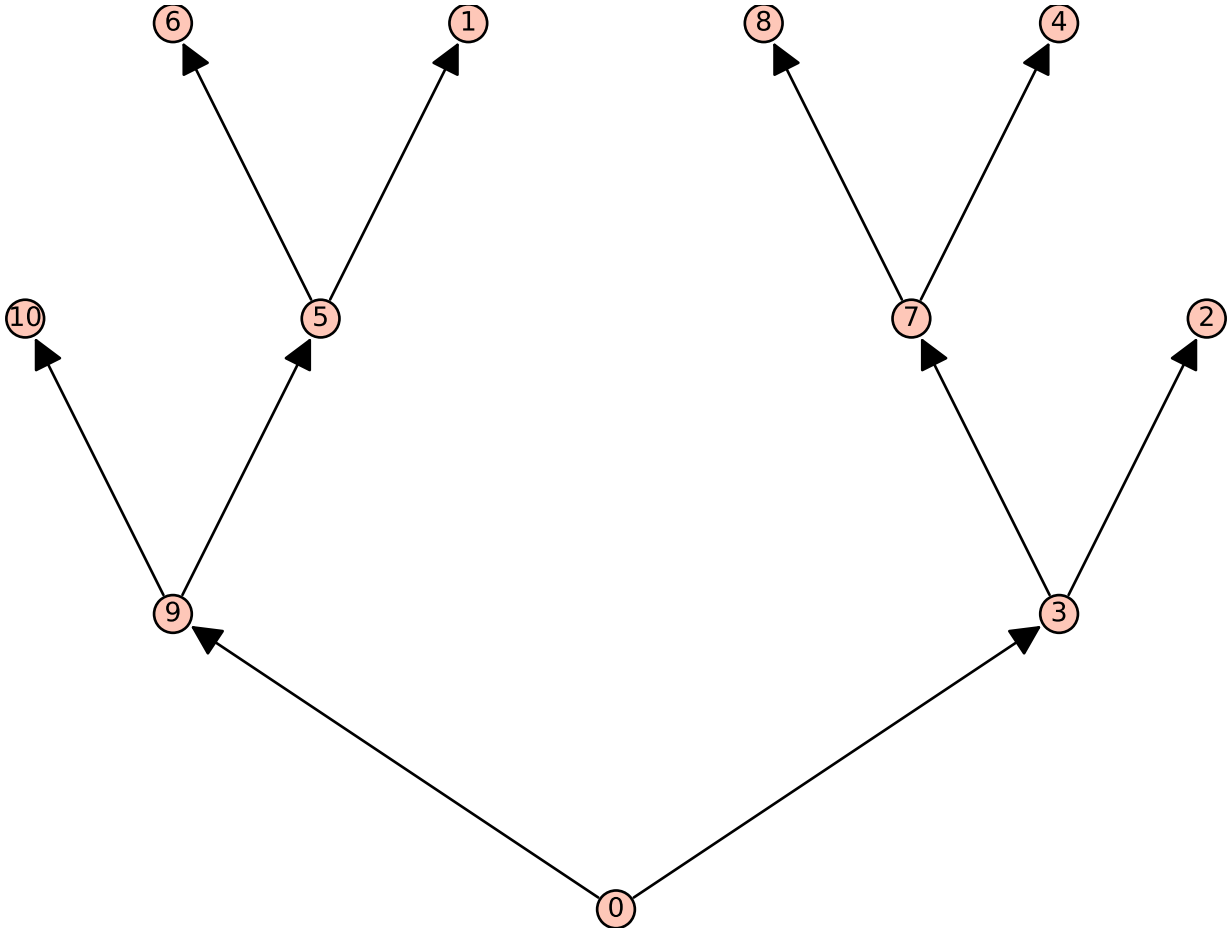
```
>>> from sage.all import *
>>> T = list(graphs.trees(Integer(7)))
>>> t = T[Integer(3)]
>>> t.graphplot(layout='tree').show()
```

The tree layout is also useful:


0
5
1
6
4
3
2

```
sage: t = DiGraph('JCC???@A??GO??CO??GO??')
sage: t.graphplot(layout='tree', tree_root=0,
....:             tree_orientation="up"
....:            ).show()
```

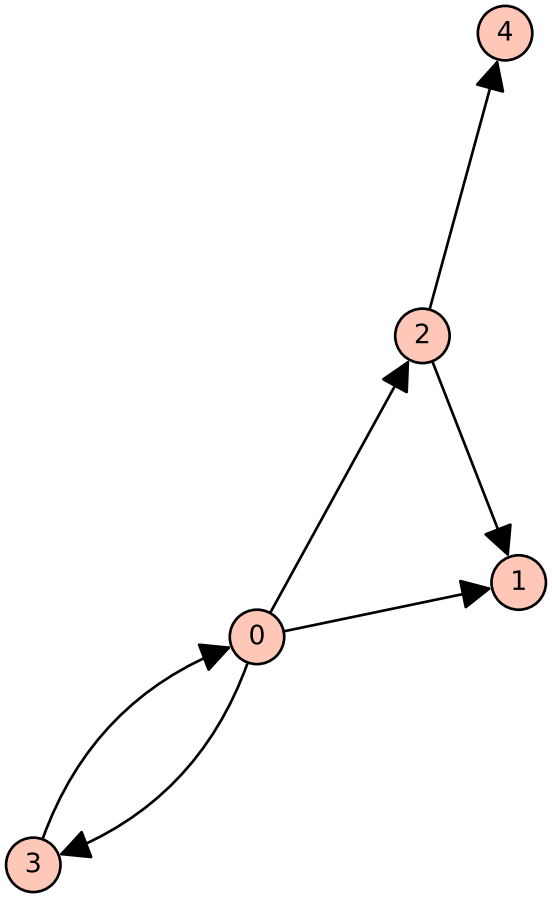
```
>>> from sage.all import *
>>> t = DiGraph('JCC???@A??GO??CO??GO??')
>>> t.graphplot(layout='tree', tree_root=Integer(0),
...              tree_orientation="up"
...             ).show()
```



More examples:

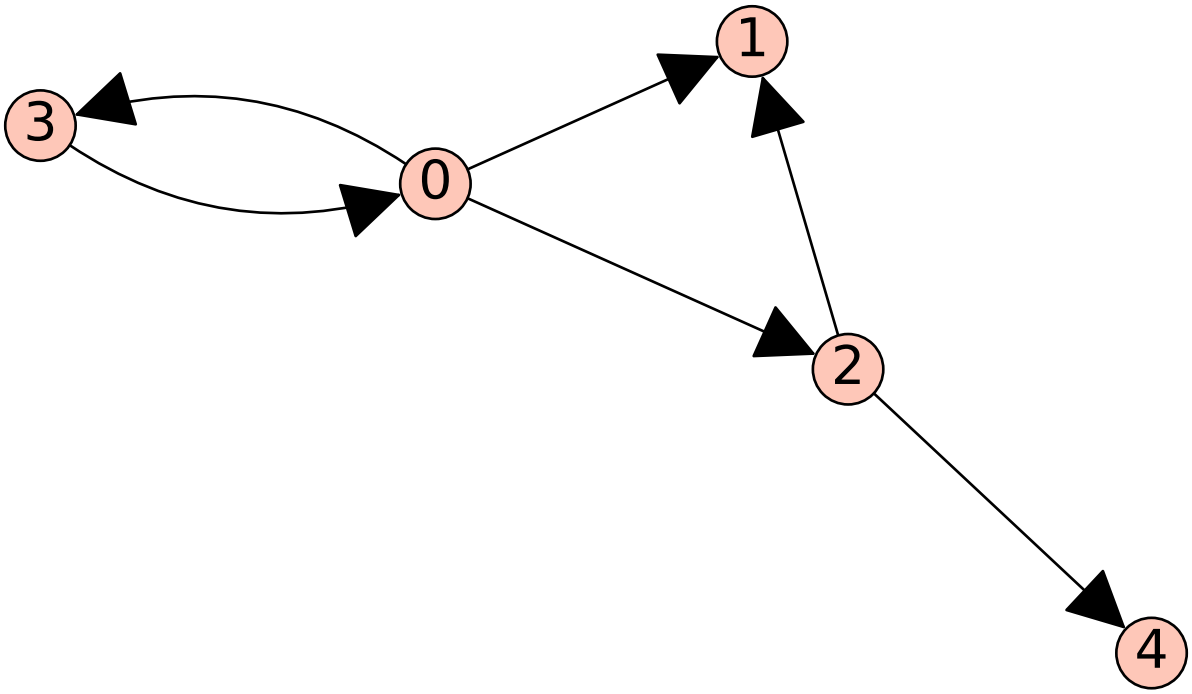
```
sage: D = DiGraph({0:[1,2,3], 2:[1,4], 3:[0]})
sage: D.graphplot().show()
```

```
>>> from sage.all import *
>>> D = DiGraph({Integer(0):[Integer(1),Integer(2),Integer(3)],␣
↪Integer(2):[Integer(1),Integer(4)], Integer(3):[Integer(0)]})
>>> D.graphplot().show()
```


4
2
1
0
3

```
sage: D = DiGraph({0:[1,2,3], 2:[1,4], 3:[0]})
sage: D.graphplot(label_fontsize=20, arrowsize=10).show()
```

```
>>> from sage.all import *
>>> D = DiGraph({Integer(0):[Integer(1),Integer(2),Integer(3)],␣
↪Integer(2):[Integer(1),Integer(4)], Integer(3):[Integer(0)]})
>>> D.graphplot(label_fontsize=Integer(20), arrowsize=Integer(10)).show()
```



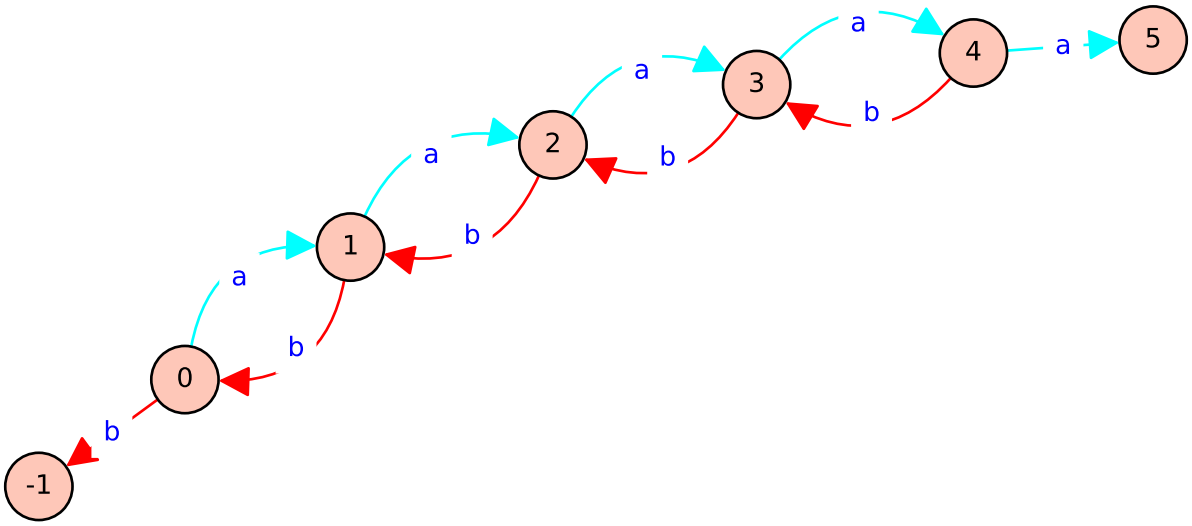

```
sage: D = DiGraph(multiedges=True, sparse=True)
sage: for i in range(5):
....:     D.add_edge((i, i + 1, 'a'))
....:     D.add_edge((i, i - 1, 'b'))
sage: D.graphplot(edge_labels=True,
....:             edge_colors=D._color_by_label()
....:            ).plot()
Graphics object consisting of 34 graphics primitives
```

```
>>> from sage.all import *
>>> D = DiGraph(multiedges=True, sparse=True)
>>> for i in range(Integer(5)):
...     D.add_edge((i, i + Integer(1), 'a'))
...     D.add_edge((i, i - Integer(1), 'b'))
```

(continues on next page)

(continued from previous page)

```
>>> D.graphplot(edge_labels=True,
...              edge_colors=D._color_by_label()
...             ).plot()
Graphics object consisting of 34 graphics primitives
```



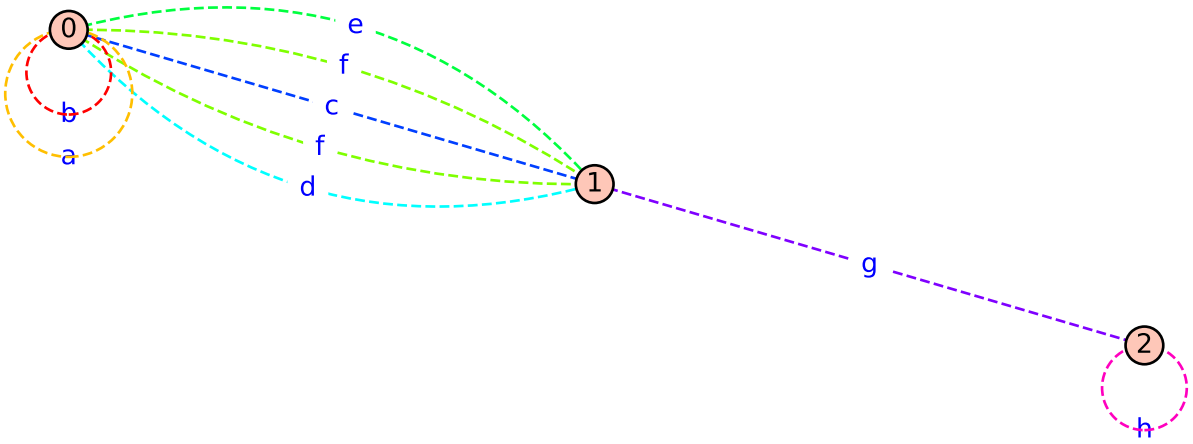

```
sage: g = Graph({}, loops=True, multiedges=True, sparse=True)
sage: g.add_edges([(0, 0, 'a'), (0, 0, 'b'), (0, 1, 'c'),
....:              (0, 1, 'd'), (0, 1, 'e'), (0, 1, 'f'),
....:              (0, 1, 'f'), (2, 1, 'g'), (2, 2, 'h')])
sage: g.graphplot(edge_labels=True,
....:             color_by_label=True,
....:             edge_style='dashed'
....:            ).plot()
Graphics object consisting of 22 graphics primitives
```

```
>>> from sage.all import *
>>> g = Graph({}, loops=True, multiedges=True, sparse=True)
>>> g.add_edges([(Integer(0), Integer(0), 'a'), (Integer(0), Integer(0), 'b'),
↪ (Integer(0), Integer(1), 'c'),
...              (Integer(0), Integer(1), 'd'), (Integer(0), Integer(1), 'e'),
↪ (Integer(0), Integer(1), 'f'),
```

(continues on next page)

(continued from previous page)

```
...               (Integer(0), Integer(1), 'f'), (Integer(2), Integer(1), 'g'),
↪ (Integer(2), Integer(2), 'h')])
>>> g.graphplot(edge_labels=True,
...              color_by_label=True,
...              edge_style='dashed'
...             ).plot()
Graphics object consisting of 22 graphics primitives
```



The `edge_style` option may be provided in the short format too:

```
sage: g.graphplot(edge_labels=True,
....:             color_by_label=True,
....:             edge_style='--'
....:            ).plot()
Graphics object consisting of 22 graphics primitives
```

```
>>> from sage.all import *
>>> g.graphplot(edge_labels=True,
...              color_by_label=True,
...              edge_style='--'
...             ).plot()
Graphics object consisting of 22 graphics primitives
```

The `edge_styles` option may be provided if you need only certain edges to have certain styles:

```
sage: g = Graph(loops=True, multiedges=True, sparse=True)
sage: g.add_edges([(0, 0, 'a'), (0, 0, 'b'), (0, 1, 'c'),
....:              (0, 1, 'd'), (0, 1, 'e'), (0, 1, 'f'),
....:              (0, 1, 'f'), (2, 1, 'g'), (2, 2, 'h')])
sage: GP = g.graphplot(vertex_size=100, edge_labels=True,
....:                  color_by_label=True, edge_style='dashed')
sage: GP.set_edges(edge_styles={'a':'dashed', 'g':'dotted'})
sage: GP.plot()
Graphics object consisting of 22 graphics primitives
```

```
>>> from sage.all import *
>>> g = Graph(loops=True, multiedges=True, sparse=True)
>>> g.add_edges([(Integer(0), Integer(0), 'a'), (Integer(0), Integer(0), 'b'),
↪ (Integer(0), Integer(1), 'c'),
...              (Integer(0), Integer(1), 'd'), (Integer(0), Integer(1), 'e'),
↪ (Integer(0), Integer(1), 'f'),
...              (Integer(0), Integer(1), 'f'), (Integer(2), Integer(1), 'g'),
↪ (Integer(2), Integer(2), 'h')])
>>> GP = g.graphplot(vertex_size=Integer(100), edge_labels=True,
...                  color_by_label=True, edge_style='dashed')
>>> GP.set_edges(edge_styles={'a':'dashed', 'g':'dotted'})
>>> GP.plot()
Graphics object consisting of 22 graphics primitives
```

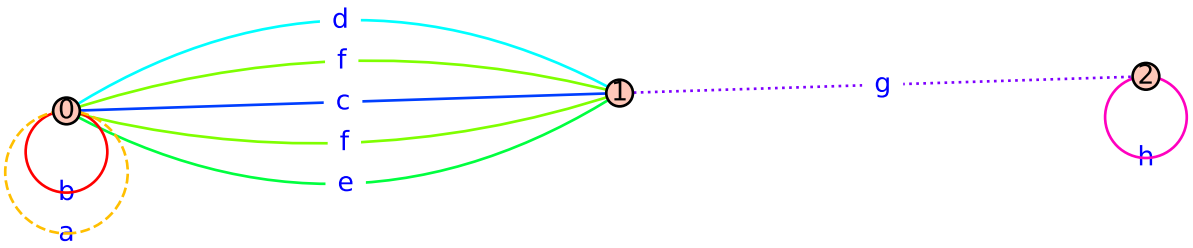
```
sage: g = Graph(loops=True, multiedges=True, sparse=True)
sage: g.add_edges([(0, 0, 'a'), (0, 0, 'b'), (0, 1, 'c'),
....:              (0, 1, 'd'), (0, 1, 'e'), (0, 1, 'f'),
....:              (0, 1, 'f'), (2, 1, 'g'), (2, 2, 'h')])
sage: GP = g.graphplot(vertex_size=100, edge_labels=True,
....:                  color_by_label=True, edge_thickness=3)
sage: GP.set_edges(edge_thicknesses={'a':1, 'g':5})
sage: GP.plot()
Graphics object consisting of 22 graphics primitives
```

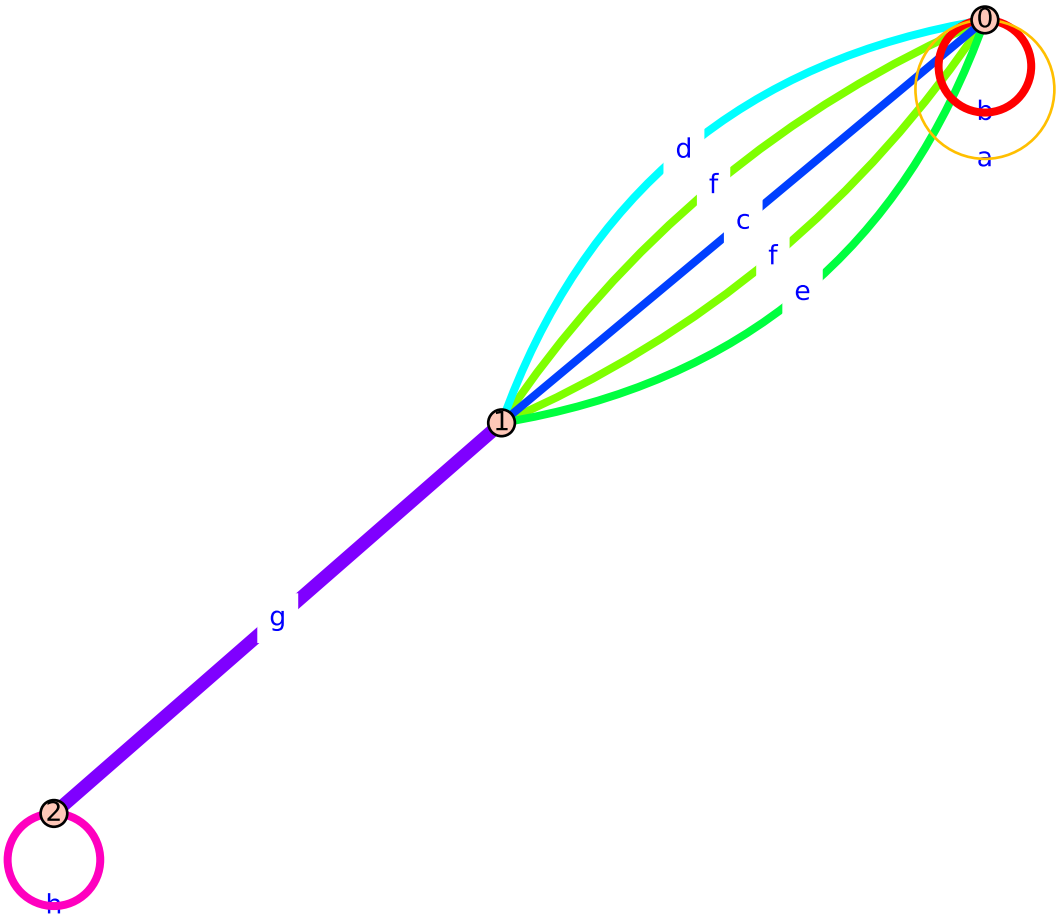
```
>>> from sage.all import *
>>> g = Graph(loops=True, multiedges=True, sparse=True)
>>> g.add_edges([(Integer(0), Integer(0), 'a'), (Integer(0), Integer(0), 'b'),
↪ (Integer(0), Integer(1), 'c'),
...              (Integer(0), Integer(1), 'd'), (Integer(0), Integer(1), 'e'),
↪ (Integer(0), Integer(1), 'f'),
...              (Integer(0), Integer(1), 'f'), (Integer(2), Integer(1), 'g'),
↪ (Integer(2), Integer(2), 'h')])
>>> GP = g.graphplot(vertex_size=Integer(100), edge_labels=True,
...                  color_by_label=True, edge_thickness=Integer(3))
>>> GP.set_edges(edge_thicknesses={'a':Integer(1), 'g':Integer(5)})
>>> GP.plot()
Graphics object consisting of 22 graphics primitives
```

set_edges(***edge_options*)

Set edge plotting parameters for the `GraphPlot` object.

This function is called by the constructor but can also be called to update the edge options of an existing


d
f
c
f
e
g
0
1
2
b
a
h


0
1
2
b
a
d
f
c
f
e
g
h

`GraphPlot` object. Note that the changes are cumulative.

EXAMPLES:

```
sage: g = Graph(loops=True, multiedges=True, sparse=True)
sage: g.add_edges([(0, 0, 'a'), (0, 0, 'b'), (0, 1, 'c'),
....:              (0, 1, 'd'), (0, 1, 'e'), (0, 1, 'f'),
....:              (0, 1, 'f'), (2, 1, 'g'), (2, 2, 'h')])
sage: GP = g.graphplot(vertex_size=100, edge_labels=True,
....:                  color_by_label=True, edge_style='dashed')
sage: GP.set_edges(edge_style='solid')
sage: GP.plot()
Graphics object consisting of 22 graphics primitives
```

```
>>> from sage.all import *
>>> g = Graph(loops=True, multiedges=True, sparse=True)
>>> g.add_edges([(Integer(0), Integer(0), 'a'), (Integer(0), Integer(0), 'b'),
↪ (Integer(0), Integer(1), 'c'),
...              (Integer(0), Integer(1), 'd'), (Integer(0), Integer(1), 'e'),
↪ (Integer(0), Integer(1), 'f'),
...              (Integer(0), Integer(1), 'f'), (Integer(2), Integer(1), 'g'),
↪ (Integer(2), Integer(2), 'h')])
>>> GP = g.graphplot(vertex_size=Integer(100), edge_labels=True,
...                  color_by_label=True, edge_style='dashed')
>>> GP.set_edges(edge_style='solid')
>>> GP.plot()
Graphics object consisting of 22 graphics primitives
```

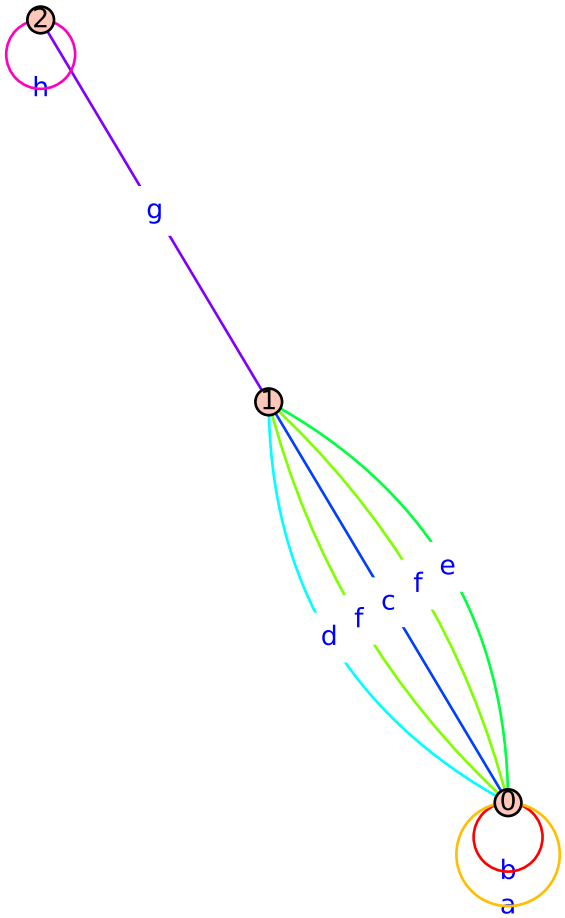
```
sage: GP.set_edges(edge_color='black')
sage: GP.plot()
Graphics object consisting of 22 graphics primitives
```

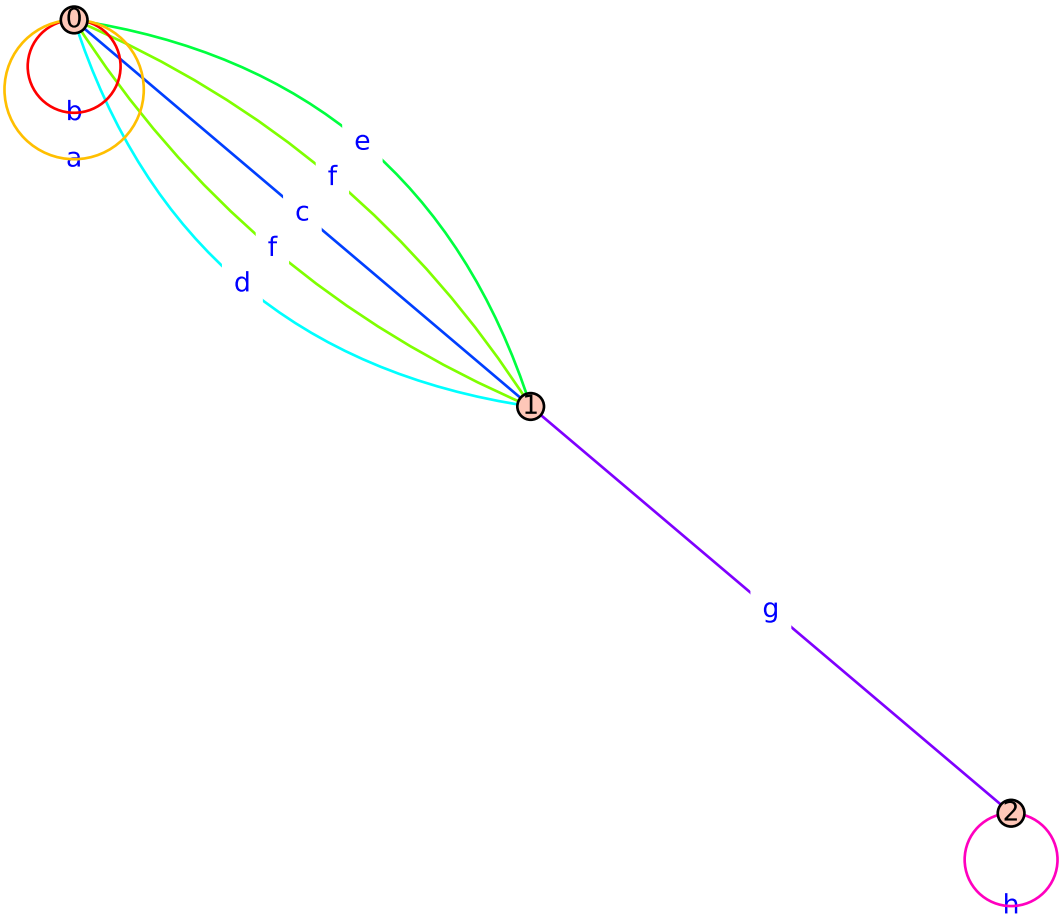
```
>>> from sage.all import *
>>> GP.set_edges(edge_color='black')
>>> GP.plot()
Graphics object consisting of 22 graphics primitives
```

```
sage: d = DiGraph(loops=True, multiedges=True, sparse=True)
sage: d.add_edges([(0, 0, 'a'), (0, 0, 'b'), (0, 1, 'c'),
....:              (0, 1, 'd'), (0, 1, 'e'), (0, 1, 'f'),
....:              (0, 1, 'f'), (2, 1, 'g'), (2, 2, 'h')])
sage: GP = d.graphplot(vertex_size=100, edge_labels=True,
....:                  color_by_label=True, edge_style='dashed')
sage: GP.set_edges(edge_style='solid')
sage: GP.plot()
Graphics object consisting of 24 graphics primitives
```

```
>>> from sage.all import *
>>> d = DiGraph(loops=True, multiedges=True, sparse=True)
>>> d.add_edges([(Integer(0), Integer(0), 'a'), (Integer(0), Integer(0), 'b'),
↪ (Integer(0), Integer(1), 'c'),
...              (Integer(0), Integer(1), 'd'), (Integer(0), Integer(1), 'e'),
↪ (Integer(0), Integer(1), 'f'),
```

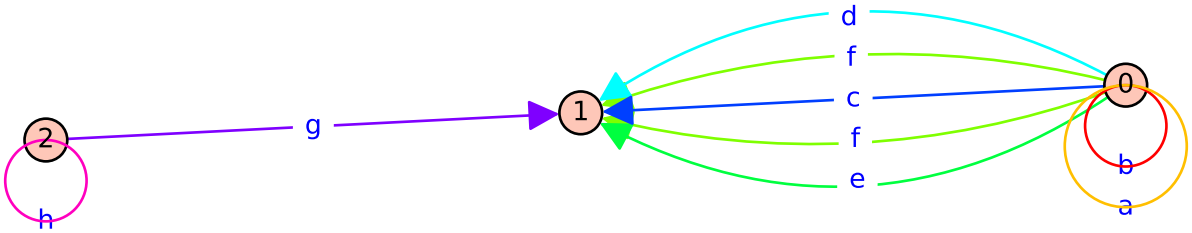
(continues on next page)


2
h
g
1
e
f
c
f
d
0
b
a


0
b
a
e
f
c
f
d
1
g
2
h

(continued from previous page)

```
...              (Integer(0), Integer(1), 'f'), (Integer(2), Integer(1), 'g'),
↪ (Integer(2), Integer(2), 'h')])
>>> GP = d.graphplot(vertex_size=Integer(100), edge_labels=True,
...                  color_by_label=True, edge_style='dashed')
>>> GP.set_edges(edge_style='solid')
>>> GP.plot()
Graphics object consisting of 24 graphics primitives
```




```
sage: GP.set_edges(edge_color='black')
sage: GP.plot()
Graphics object consisting of 24 graphics primitives
```

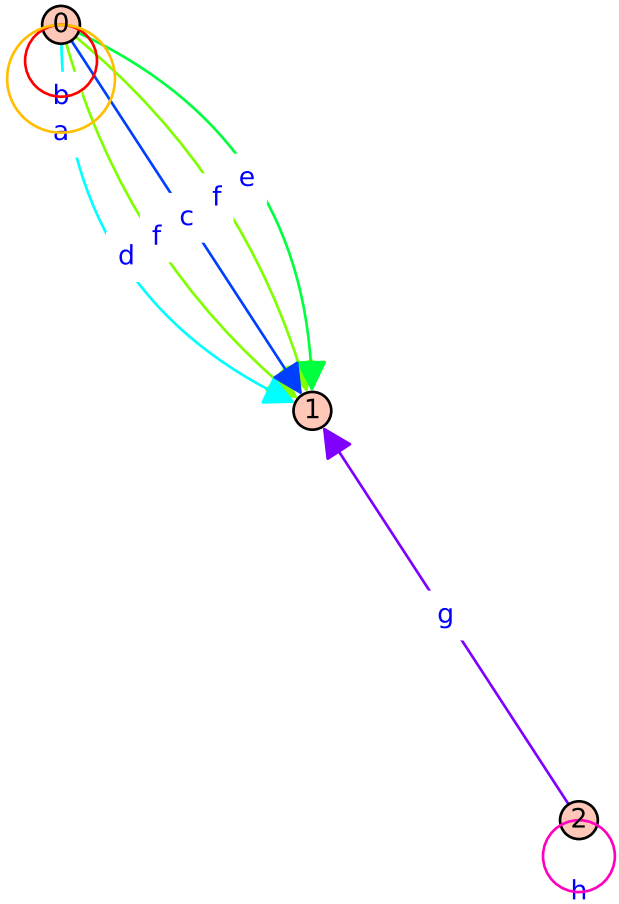
```
>>> from sage.all import *
>>> GP.set_edges(edge_color='black')
>>> GP.plot()
Graphics object consisting of 24 graphics primitives
```

set_pos()

Set the position plotting parameters for this GraphPlot.

EXAMPLES:

This function is called implicitly by the code below:


0
b
a
d
f
c
f
e
1
g
2
h

```
sage: g = Graph({0: [1, 2], 2: [3], 4: [0, 1]})
sage: g.graphplot(save_pos=True, layout='circular')  # indirect doctest
GraphPlot object for Graph on 5 vertices
```

```
>>> from sage.all import *
>>> g = Graph({Integer(0): [Integer(1), Integer(2)], Integer(2): [Integer(3)],
↪ Integer(4): [Integer(0), Integer(1)]})
>>> g.graphplot(save_pos=True, layout='circular')  # indirect doctest
GraphPlot object for Graph on 5 vertices
```

The following illustrates the format of a position dictionary, but due to numerical noise we do not check the values themselves:

```
sage: g.get_pos()
{0: (0.0, 1.0),
 1: (-0.951..., 0.309...),
 2: (-0.587..., -0.809...),
 3: (0.587..., -0.809...),
 4: (0.951..., 0.309...)}
```

```
>>> from sage.all import *
>>> g.get_pos()
{0: (0.0, 1.0),
 1: (-0.951..., 0.309...),
 2: (-0.587..., -0.809...),
 3: (0.587..., -0.809...),
 4: (0.951..., 0.309...)}
```

```
sage: T = list(graphs.trees(7))
sage: t = T[3]
sage: t.plot(heights={0: [0], 1: [4, 5, 1], 2: [2], 3: [3, 6]})
Graphics object consisting of 14 graphics primitives
```

```
>>> from sage.all import *
>>> T = list(graphs.trees(Integer(7)))
>>> t = T[Integer(3)]
>>> t.plot(heights={Integer(0): [Integer(0)], Integer(1): [Integer(4),␣
↪Integer(5), Integer(1)], Integer(2): [Integer(2)], Integer(3): [Integer(3),␣
↪Integer(6)]})
Graphics object consisting of 14 graphics primitives
```

set_vertices(***vertex_options*)

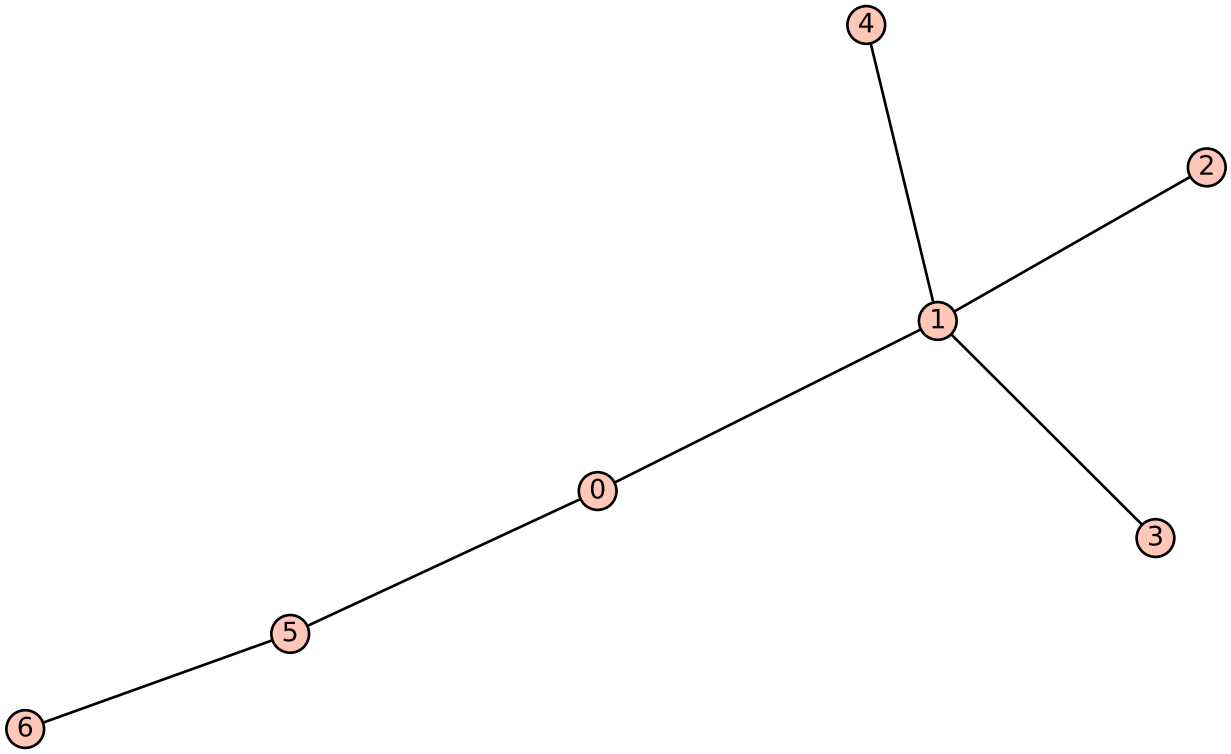
Set the vertex plotting parameters for this `GraphPlot`.

This function is called by the constructor but can also be called to make updates to the vertex options of an existing `GraphPlot` object. Note that the changes are cumulative.

EXAMPLES:

```
sage: g = Graph({}, loops=True, multiedges=True, sparse=True)
sage: g.add_edges([(0, 0, 'a'), (0, 0, 'b'), (0, 1, 'c'),
....:              (0, 1, 'd'), (0, 1, 'e'), (0, 1, 'f'),
....:              (0, 1, 'f'), (2, 1, 'g'), (2, 2, 'h')])
```

(continues on next page)


4
2
1
0
3
5
6

(continued from previous page)

```
sage: GP = g.graphplot(vertex_size=100, edge_labels=True,
....:                  color_by_label=True, edge_style='dashed')
sage: GP.set_vertices(talk=True)
sage: GP.plot()
Graphics object consisting of 22 graphics primitives
sage: GP.set_vertices(vertex_color='green', vertex_shape='^')
sage: GP.plot()
Graphics object consisting of 22 graphics primitives
```

```
>>> from sage.all import *
>>> g = Graph({}, loops=True, multiedges=True, sparse=True)
>>> g.add_edges([(Integer(0), Integer(0), 'a'), (Integer(0), Integer(0), 'b'),
↪ (Integer(0), Integer(1), 'c'),
...               (Integer(0), Integer(1), 'd'), (Integer(0), Integer(1), 'e'),
↪ (Integer(0), Integer(1), 'f'),
...               (Integer(0), Integer(1), 'f'), (Integer(2), Integer(1), 'g'),
↪ (Integer(2), Integer(2), 'h')])
>>> GP = g.graphplot(vertex_size=Integer(100), edge_labels=True,
...                  color_by_label=True, edge_style='dashed')
>>> GP.set_vertices(talk=True)
>>> GP.plot()
Graphics object consisting of 22 graphics primitives
>>> GP.set_vertices(vertex_color='green', vertex_shape='^')
>>> GP.plot()
Graphics object consisting of 22 graphics primitives
```

Vertex labels are flexible:

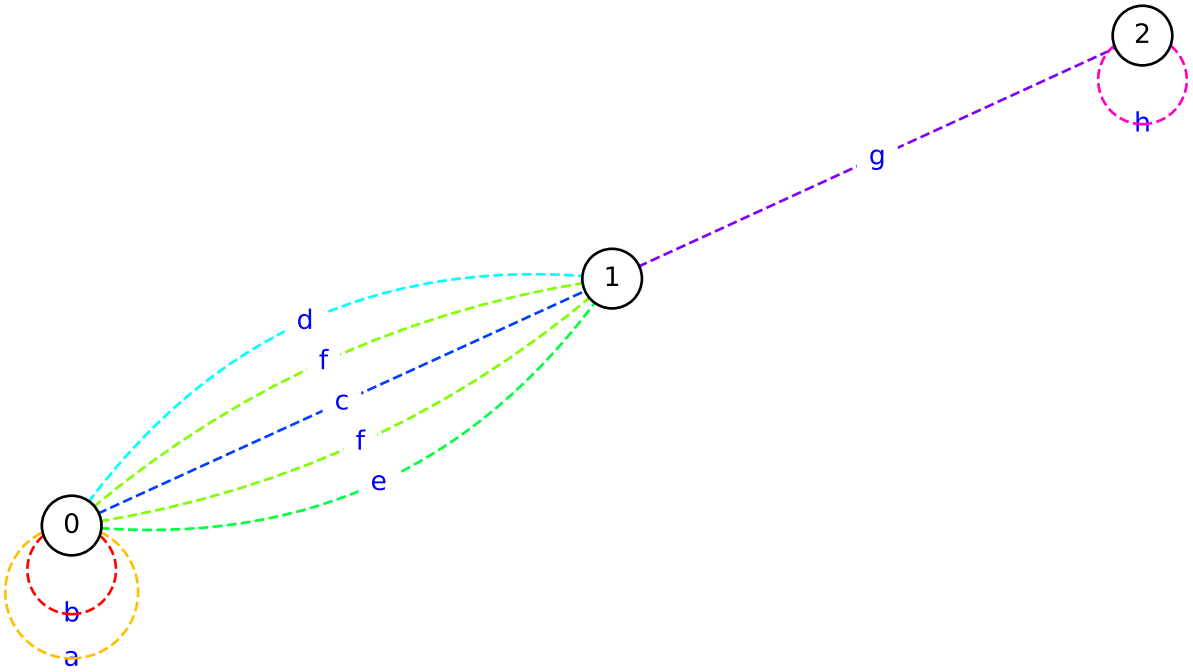
```
sage: g = graphs.PathGraph(4)
sage: g.plot(vertex_labels=False)
Graphics object consisting of 4 graphics primitives
```

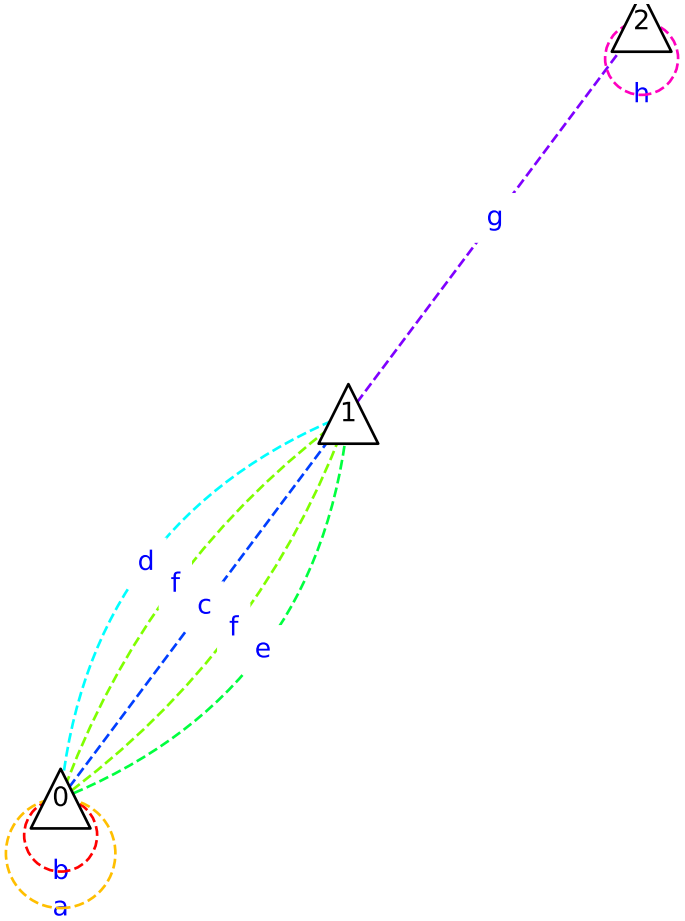
```
>>> from sage.all import *
>>> g = graphs.PathGraph(Integer(4))
>>> g.plot(vertex_labels=False)
Graphics object consisting of 4 graphics primitives
```

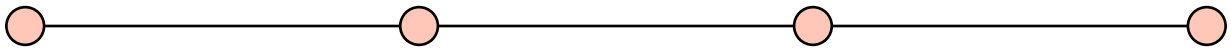
```
sage: g = graphs.PathGraph(4)
sage: g.plot(vertex_labels=True)
Graphics object consisting of 8 graphics primitives
```

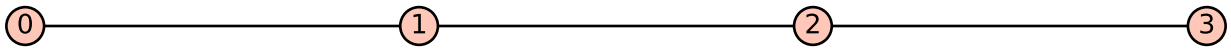
```
>>> from sage.all import *
>>> g = graphs.PathGraph(Integer(4))
>>> g.plot(vertex_labels=True)
Graphics object consisting of 8 graphics primitives
```

```
sage: g = graphs.PathGraph(4)
sage: g.plot(vertex_labels=dict(zip(g, ['+', '-', '/', '*'])))
Graphics object consisting of 8 graphics primitives
```

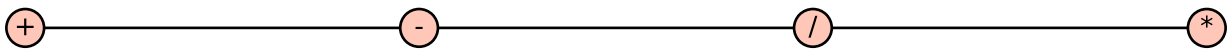

2
h
g
1
d
f
c
f
e
0
b
a


2
h
g
1
d
f
c
f
e
0
b
a




0
1
2
3

```
>>> from sage.all import *
>>> g = graphs.PathGraph(Integer(4))
>>> g.plot(vertex_labels=dict(zip(g, ['+', '-', '/', '*'])))
Graphics object consisting of 8 graphics primitives
```




```
sage: g = graphs.PathGraph(4)
sage: g.plot(vertex_labels=lambda x: str(x % 2))
Graphics object consisting of 8 graphics primitives
```

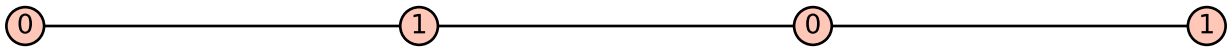
```
>>> from sage.all import *
>>> g = graphs.PathGraph(Integer(4))
>>> g.plot(vertex_labels=lambda x: str(x % Integer(2)))
Graphics object consisting of 8 graphics primitives
```

show(***kwds*)

Show the (di)graph associated with this `GraphPlot` object.

INPUT:

This method accepts all parameters of `sage.plot.graphics.Graphics.show()`.


0
1
0
1

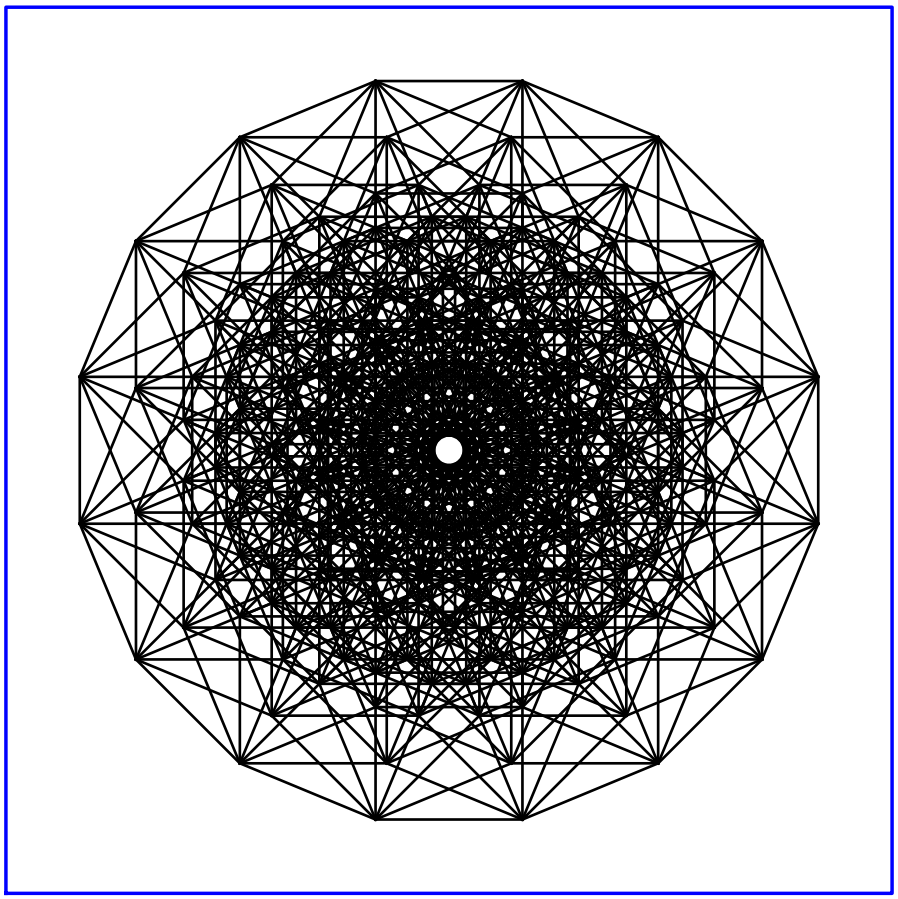
Note

- See *the module's documentation* for information on default values of this method.
- Any options not used by plot will be passed on to the `show()` method.

EXAMPLES:

```
sage: C = graphs.CubeGraph(8)
sage: P = C.graphplot(vertex_labels=False, vertex_size=0,
....:                 graph_border=True)
sage: P.show()
```

```
>>> from sage.all import *
>>> C = graphs.CubeGraph(Integer(8))
>>> P = C.graphplot(vertex_labels=False, vertex_size=Integer(0),
...                 graph_border=True)
>>> P.show()
```



5.22 Graph plotting in Javascript with d3.js

This module implements everything that can be used to draw graphs with d3.js in Sage.

On Python's side, this is mainly done by wrapping a graph's edges and vertices in a structure that can then be used in the javascript code. This javascript code is then inserted into a .html file to be opened by a browser.

In the browser, the displayed page contains at the bottom right a menu that allows to save the picture under the svg file format.

What Sage feeds javascript with is a "graph" object with the following content:

- `vertices` – each vertex is a dictionary defining :
  - `name` – the vertex's label
  - `group` – the vertex' color (integer)

The ID of a vertex is its index in the vertex list.

- `edges` – each edge is a dictionary defining :
  - `source` – the ID (int) of the edge's source
  - `target` – the ID (int) of the edge's destination
  - `color` – the edge's color (integer)
  - `value` – thickness of the edge
  - `strength` – the edge's strength in the automatic layout
  - `color` – color (hexadecimal code)
  - `curve` – distance from the barycenter of the two endpoints and the center of the edge. It defines the curve of the edge, which can be useful for multigraphs.
- `pos` – list whose i th element is a dictionary defining the position of the i th vertex

It also contains the definition of some numerical/boolean variables whose definition can be found in the documentation of *`show()`* : `directed`, `charge`, `link_distance`, `link_strength`, `gravity`, `vertex_size`, `edge_thickness`.

Warning

Since the d3js package is not standard yet, the javascript is fetched from d3js.org website by the browser. If you want to avoid that (e.g. to protect your privacy or by lack of internet connection), you can install the d3js package for offline use by running `sage -i d3js` from the command line.

Todo

- Add tooltip like in http://bl.ocks.org/bentwonk/2514276.
- Add a zoom through scrolling (http://bl.ocks.org/mbostock/3681006).

Authors:

- Nathann Cohen, Brice Onfroy – July 2013 – Initial version of the Sage code, Javascript code, using examples from d3.js.
- Thierry Monteil (June 2014): allow offline use of d3.js provided by d3js spkg.

5.22.1 Functions

sage.graphs.graph_plot_js.**gen_html_code**(*G*, *vertex_labels=True*, *edge_labels=False*, *vertex_partition=[]*, *vertex_colors=None*, *edge_partition=[]*, *force_spring_layout=False*, *charge=-120*, *link_distance=30*, *link_strength=2*, *gravity=0.04*, *vertex_size=7*, *edge_thickness=4*)

Create a .html file showing the graph using d3.js.

This function returns the name of the .html file. If you want to visualize the actual graph use `show()`.

INPUT:

- `G` – the graph
- `vertex_labels` – boolean (default: `False`); whether to display vertex labels
- `edge_labels` – boolean (default: `False`); whether to display edge labels
- `vertex_partition` – list (default: `[]`); a list of lists representing a partition of the vertex set. Vertices are then colored in the graph according to the partition.
- `vertex_colors` – dictionary (default: `None`); a dictionary representing a partition of the vertex set. Keys are colors (ignored) and values are lists of vertices. Vertices are then colored in the graph according to the partition.
- `edge_partition` – list (default: `[]`); same as `vertex_partition`, with edges instead
- `force_spring_layout` – boolean (default: `False`); whether to take previously computed position of nodes into account if there is one, or to compute a spring layout
- `vertex_size` – integer (default: 7); the size of a vertex' circle
- `edge_thickness` – integer (default: 4); thickness of an edge
- `charge` – integer (default: -120); the vertices' charge. Defines how they repulse each other. See https://github.com/mbostock/d3/wiki/Force-Layout for more information
- `link_distance` – integer (default: 30); see https://github.com/mbostock/d3/wiki/Force-Layout for more information
- `link_strength` – integer (default: 2); see https://github.com/mbostock/d3/wiki/Force-Layout for more information
- `gravity` – float (default: 0.04); see https://github.com/mbostock/d3/wiki/Force-Layout for more information

Warning

Since the d3js package is not standard yet, the javascript is fetched from d3js.org website by the browser. If you want to avoid that (e.g. to protect your privacy or by lack of internet connection), you can install the d3js package for offline use by running `sage -i d3js` from the command line.

EXAMPLES:

```
sage: graphs.RandomTree(50).show(method='js')                    #␣
↪optional - internet

sage: g = graphs.PetersenGraph()
sage: g.show(method='js', vertex_partition=g.coloring())         #␣
↪optional - internet
```

(continues on next page)

(continued from previous page)

```
sage: graphs.DodecahedralGraph().show(method='js',                          #␣
↪optional - internet
....:                                 force_spring_layout=True)

sage: graphs.DodecahedralGraph().show(method='js')                          #␣
↪optional - internet

sage: g = digraphs.DeBruijn(2, 2)
sage: g.allow_multiple_edges(True)
sage: g.add_edge("10", "10", "a")
sage: g.add_edge("10", "10", "b")
sage: g.add_edge("10", "10", "c")
sage: g.add_edge("10", "10", "d")
sage: g.add_edge("01", "11", "1")
sage: g.show(method='js', vertex_labels=True, edge_labels=True,             #␣
↪optional - internet
....:        link_distance=200, gravity=.05, charge=-500,
....:        edge_partition=[[("11", "12", "2"), ("21", "21", "a")]],
....:        edge_thickness=4)
```

```
>>> from sage.all import *
>>> graphs.RandomTree(Integer(50)).show(method='js')                        #␣
↪optional - internet

>>> g = graphs.PetersenGraph()
>>> g.show(method='js', vertex_partition=g.coloring())                  # optional -
↪ internet

>>> graphs.DodecahedralGraph().show(method='js',                          # optional -
↪ internet
...                                 force_spring_layout=True)

>>> graphs.DodecahedralGraph().show(method='js')                          # optional -
↪ internet

>>> g = digraphs.DeBruijn(Integer(2), Integer(2))
>>> g.allow_multiple_edges(True)
>>> g.add_edge("10", "10", "a")
>>> g.add_edge("10", "10", "b")
>>> g.add_edge("10", "10", "c")
>>> g.add_edge("10", "10", "d")
>>> g.add_edge("01", "11", "1")
>>> g.show(method='js', vertex_labels=True, edge_labels=True,             # optional -
↪ internet
...        link_distance=Integer(200), gravity=RealNumber('.05'), charge=-
↪Integer(500),
...        edge_partition=[[("11", "12", "2"), ("21", "21", "a")]],
...        edge_thickness=Integer(4))
```

5.23 Tree decompositions

This module implements tree-decomposition methods.

A tree-decomposition of a graph $G = (V, E)$ is a pair (X, T), where $X = \{X_1, X_2, \ldots, X_t\}$ is a family of subsets of V, usually called *bags*, and T is a tree of order t whose nodes are the subsets X_i satisfying the following properties:

- The union of all sets X_i equals V. That is, each vertex of the graph G is associated with at least one tree node.
- For every edge (v, w) in the graph, there is a subset X_i that contains both v and w. That is, each edge of the graph G appears in a tree node.
- The nodes associated with vertex $v \in V$ form a connected subtree of T. That is, if X_i and X_j both contain a vertex $v \in V$, then all nodes X_k of the tree in the (unique) path between X_i and X_j contain v as well, and we have $X_i \cap X_j \subseteq X_k$.

The *width* of a tree decomposition is the size of the largest set X_i minus one, i.e., $\max_{X_i \in X} |X_i| - 1$, and the *treewidth* $tw(G)$ of a graph G is the minimum width among all possible tree decompositions of G. Observe that, the size of the largest set is diminished by one in order to make the treewidth of a tree equal to one.

The *length* of a tree decomposition, as proposed in [DG2006], is the maximum *diameter* in G of its bags, where the diameter of a bag X_i is the largest distance in G between the vertices in X_i (i.e., $\max_{u,v \in X_i} \text{dist}_G(u, v)$). The *treelength* $tl(G)$ of a graph G is the minimum length among all possible tree decompositions of G.

While deciding whether a graph has treelength 1 can be done in linear time (equivalent to deciding if the graph is chordal), deciding if it has treelength at most k for any fixed constant $k \leq 2$ is NP-complete [Lokshtanov2009].

Treewidth and treelength are different measures of tree-likeness. In particular, trees have treewidth and treelength 1:

```
sage: T = graphs.RandomTree(20)
sage: T.treewidth()
1
sage: T.treelength()
1
```

```
>>> from sage.all import *
>>> T = graphs.RandomTree(Integer(20))
>>> T.treewidth()
1
>>> T.treelength()
1
```

The treewidth of a cycle is 2 and its treelength is $\lceil n/3 \rceil$:

```
sage: [graphs.CycleGraph(n).treewidth() for n in range(3, 11)]
[2, 2, 2, 2, 2, 2, 2, 2]
sage: [graphs.CycleGraph(n).treelength() for n in range(3, 11)]
[1, 2, 2, 2, 3, 3, 3, 4]
```

```
>>> from sage.all import *
>>> [graphs.CycleGraph(n).treewidth() for n in range(Integer(3), Integer(11))]
[2, 2, 2, 2, 2, 2, 2, 2]
>>> [graphs.CycleGraph(n).treelength() for n in range(Integer(3), Integer(11))]
[1, 2, 2, 2, 3, 3, 3, 4]
```

The treewidth of a clique is $n - 1$ and its treelength is 1:

```
sage: [graphs.CompleteGraph(n).treewidth() for n in range(3, 11)]
[2, 3, 4, 5, 6, 7, 8, 9]
sage: [graphs.CompleteGraph(n).treelength() for n in range(3, 11)]
[1, 1, 1, 1, 1, 1, 1, 1]
```

```
>>> from sage.all import *
>>> [graphs.CompleteGraph(n).treewidth() for n in range(Integer(3), Integer(11))]
[2, 3, 4, 5, 6, 7, 8, 9]
>>> [graphs.CompleteGraph(n).treelength() for n in range(Integer(3), Integer(11))]
[1, 1, 1, 1, 1, 1, 1, 1]
```

See also

- Wikipedia article Tree_decomposition
- Wikipedia article Treewidth

This module contains the following methods

treewidth()	Compute the treewidth of G (and provide a decomposition).
treelength()	Compute the treelength of G (and provide a decomposition).
make_nice_tree_decomposition()	Return a *nice* tree decomposition (TD) of the TD `tree_decomp`.
label_nice_tree_decomposition()	Return a nice tree decomposition with nodes labelled accordingly.
is_valid_tree_decomposition()	Check whether T is a valid tree-decomposition for G.
reduced_tree_decomposition()	Return a reduced tree-decomposition of T.
width_of_tree_decomposition()	Return the width of the tree decomposition T of G.
length_of_tree_decomposition()	Return the length of the tree decomposition T of G.

5.23.1 Methods

class sage.graphs.graph_decompositions.tree_decomposition.**TreelengthConnected**

Bases: `object`

Compute the treelength of a connected graph (and provide a decomposition).

This class implements an algorithm for computing the treelength of a connected graph that virtually explores the graph of all pairs `(vertex_cut, connected_component)`, where `vertex_cut` is a vertex cut of the graph of length $\leq k$, and `connected_component` is a connected component of the graph induced by `G - vertex_cut`.

We deduce that the pair `(vertex_cut, connected_component)` is feasible with treelength k if `connected_component` is empty, or if a vertex `v` from `vertex_cut` can be replaced with a vertex from `connected_component`, such that the pair `(vertex_cut + v, connected_component - v)` is feasible.

INPUT:

- `G` – a sage Graph

- `k` – integer (default: `None`); indicates the length to be considered. When k is an integer, the method checks that the graph has treelength $\leq k$. If k is `None` (default), the method computes the optimal treelength.
- `certificate` – boolean (default: `False`); whether to also compute the tree-decomposition itself

OUTPUT:

`TreelengthConnected(G)` returns the treelength of G. When k is specified, it returns `False` when no tree-decomposition of length $\leq k$ exists or `True` otherwise. When `certificate=True`, the tree-decomposition is also returned.

EXAMPLES:

A clique has treelength 1:

```
sage: from sage.graphs.graph_decompositions.tree_decomposition import␣
↪TreelengthConnected
sage: TreelengthConnected(graphs.CompleteGraph(3)).get_length()
1
sage: TC = TreelengthConnected(graphs.CompleteGraph(4), certificate=True)
sage: TC.get_length()
1
sage: TC.get_tree_decomposition()
Tree decomposition of Complete graph: Graph on 1 vertex
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_decompositions.tree_decomposition import␣
↪TreelengthConnected
>>> TreelengthConnected(graphs.CompleteGraph(Integer(3))).get_length()
1
>>> TC = TreelengthConnected(graphs.CompleteGraph(Integer(4)), certificate=True)
>>> TC.get_length()
1
>>> TC.get_tree_decomposition()
Tree decomposition of Complete graph: Graph on 1 vertex
```

A cycle has treelength $\lceil n/3 \rceil$:

```
sage: TreelengthConnected(graphs.CycleGraph(6)).get_length()
2
sage: TreelengthConnected(graphs.CycleGraph(7)).get_length()
3
sage: TreelengthConnected(graphs.CycleGraph(7), k=3).is_less_than_k()
True
sage: TreelengthConnected(graphs.CycleGraph(7), k=2).is_less_than_k()
False
```

```
>>> from sage.all import *
>>> TreelengthConnected(graphs.CycleGraph(Integer(6))).get_length()
2
>>> TreelengthConnected(graphs.CycleGraph(Integer(7))).get_length()
3
>>> TreelengthConnected(graphs.CycleGraph(Integer(7)), k=Integer(3)).is_less_than_
↪k()
True
>>> TreelengthConnected(graphs.CycleGraph(Integer(7)), k=Integer(2)).is_less_than_
```

(continues on next page)

(continued from previous page)

```
↪k()
False
```

get_length()

Return the length of the tree decomposition.

EXAMPLES:

```
sage: from sage.graphs.graph_decompositions.tree_decomposition import␣
↪TreelengthConnected
sage: G = graphs.CycleGraph(4)
sage: TreelengthConnected(G).get_length()
2
sage: TreelengthConnected(G, k=2).get_length()
2
sage: TreelengthConnected(G, k=1).get_length()
Traceback (most recent call last):
...
ValueError: no tree decomposition with length <= 1 was found
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_decompositions.tree_decomposition import␣
↪TreelengthConnected
>>> G = graphs.CycleGraph(Integer(4))
>>> TreelengthConnected(G).get_length()
2
>>> TreelengthConnected(G, k=Integer(2)).get_length()
2
>>> TreelengthConnected(G, k=Integer(1)).get_length()
Traceback (most recent call last):
...
ValueError: no tree decomposition with length <= 1 was found
```

get_tree_decomposition()

Return the tree-decomposition.

EXAMPLES:

```
sage: from sage.graphs.graph_decompositions.tree_decomposition import␣
↪TreelengthConnected
sage: G = graphs.CycleGraph(4)
sage: TreelengthConnected(G, certificate=True).get_tree_decomposition()
Tree decomposition of Cycle graph: Graph on 2 vertices
sage: G.diameter()
2
sage: TreelengthConnected(G, k=2, certificate=True).get_tree_decomposition()
Tree decomposition of Cycle graph: Graph on 1 vertex
sage: TreelengthConnected(G, k=1, certificate=True).get_tree_decomposition()
Traceback (most recent call last):
...
ValueError: no tree decomposition with length <= 1 was found
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_decompositions.tree_decomposition import␣
↪TreelengthConnected
>>> G = graphs.CycleGraph(Integer(4))
>>> TreelengthConnected(G, certificate=True).get_tree_decomposition()
Tree decomposition of Cycle graph: Graph on 2 vertices
>>> G.diameter()
2
>>> TreelengthConnected(G, k=Integer(2), certificate=True).get_tree_
↪decomposition()
Tree decomposition of Cycle graph: Graph on 1 vertex
>>> TreelengthConnected(G, k=Integer(1), certificate=True).get_tree_
↪decomposition()
Traceback (most recent call last):
...
ValueError: no tree decomposition with length <= 1 was found
```

is_less_than_k()

Return whether a tree decomposition with length at most k was found.

EXAMPLES:

```
sage: from sage.graphs.graph_decompositions.tree_decomposition import␣
↪TreelengthConnected
sage: G = graphs.CycleGraph(4)
sage: TreelengthConnected(G, k=1).is_less_than_k()
False
sage: TreelengthConnected(G, k=2).is_less_than_k()
True
sage: TreelengthConnected(G).is_less_than_k()
Traceback (most recent call last):
...
ValueError: parameter 'k' has not been specified
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_decompositions.tree_decomposition import␣
↪TreelengthConnected
>>> G = graphs.CycleGraph(Integer(4))
>>> TreelengthConnected(G, k=Integer(1)).is_less_than_k()
False
>>> TreelengthConnected(G, k=Integer(2)).is_less_than_k()
True
>>> TreelengthConnected(G).is_less_than_k()
Traceback (most recent call last):
...
ValueError: parameter 'k' has not been specified
```

sage.graphs.graph_decompositions.tree_decomposition.**is_valid_tree_decomposition**(G, T)

Check whether T is a valid tree-decomposition for G.

INPUT:

- G – a sage Graph
- T – a tree decomposition, i.e., a tree whose vertices are the bags (subsets of vertices) of the decomposition

EXAMPLES:

```
sage: from sage.graphs.graph_decompositions.tree_decomposition import is_valid_
↪tree_decomposition
sage: K = graphs.CompleteGraph(4)
sage: T = Graph()
sage: T.add_vertex(Set(K))
sage: is_valid_tree_decomposition(K, T)
True

sage: G = graphs.RandomGNP(10, .2)
sage: T = G.treewidth(certificate=True)
sage: is_valid_tree_decomposition(G, T)
True
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_decompositions.tree_decomposition import is_valid_tree_
↪decomposition
>>> K = graphs.CompleteGraph(Integer(4))
>>> T = Graph()
>>> T.add_vertex(Set(K))
>>> is_valid_tree_decomposition(K, T)
True

>>> G = graphs.RandomGNP(Integer(10), RealNumber('.2'))
>>> T = G.treewidth(certificate=True)
>>> is_valid_tree_decomposition(G, T)
True
```

The union of the bags is the set of vertices of G:

```
sage: G = graphs.PathGraph(4)
sage: T = G.treewidth(certificate=True)
sage: _ = G.add_vertex()
sage: is_valid_tree_decomposition(G, T)
False
```

```
>>> from sage.all import *
>>> G = graphs.PathGraph(Integer(4))
>>> T = G.treewidth(certificate=True)
>>> _ = G.add_vertex()
>>> is_valid_tree_decomposition(G, T)
False
```

Each edge of G is contained in a bag:

```
sage: G = graphs.PathGraph(4)
sage: T = G.treewidth(certificate=True)
sage: G.add_edge(0, 3)
sage: is_valid_tree_decomposition(G, T)
False
```

```
>>> from sage.all import *
>>> G = graphs.PathGraph(Integer(4))
>>> T = G.treewidth(certificate=True)
>>> G.add_edge(Integer(0), Integer(3))
>>> is_valid_tree_decomposition(G, T)
False
```

The bags containing a vertex v form a subtree of T:

```
sage: G = graphs.PathGraph(4)
sage: X1, X2, X3 = Set([0, 1]), Set([1, 2]), Set([2, 3])
sage: T = Graph([(X1, X3), (X3, X2)])
sage: is_valid_tree_decomposition(G, T)
False
```

```
>>> from sage.all import *
>>> G = graphs.PathGraph(Integer(4))
>>> X1, X2, X3 = Set([Integer(0), Integer(1)]), Set([Integer(1), Integer(2)]),␣
↪Set([Integer(2), Integer(3)])
>>> T = Graph([(X1, X3), (X3, X2)])
>>> is_valid_tree_decomposition(G, T)
False
```

sage.graphs.graph_decompositions.tree_decomposition.**label_nice_tree_decomposition**(*nice_TD*, *root=None*, *directed=False*)

Return a nice tree decomposition with nodes labelled accordingly.

INPUT:

- `nice_TD` – a nice tree decomposition
- `root` – the root of the nice tree decomposition (default: `min(nice_TD)`). The default value is suitable for the output of *make_nice_tree_decomposition()*.
- `directed` – boolean (default: `False`); whether to return the nice tree decomposition as a directed graph rooted at vertex `root` (with edges oriented away from the root) or as an undirected graph

OUTPUT: a nice tree decomposition with nodes labelled

EXAMPLES:

```
sage: from sage.graphs.graph_decompositions.tree_decomposition import make_nice_
↪tree_decomposition, label_nice_tree_decomposition
sage: claw = graphs.CompleteBipartiteGraph(1, 3)
sage: claw_TD = claw.treewidth(certificate=True)
sage: nice_TD = make_nice_tree_decomposition(claw, claw_TD)
sage: label_TD = label_nice_tree_decomposition(nice_TD, directed=True)
sage: label_TD.name()
'Labelled Nice tree decomposition of Tree decomposition'
sage: for node in sorted(label_TD):  # random
....:     print(node, label_TD.get_vertex(node))
(0, {}) forget
(1, {0}) forget
```

(continues on next page)

(continued from previous page)

```
(2, {0, 1}) intro
(3, {0}) forget
(4, {0, 3}) intro
(5, {0}) forget
(6, {0, 2}) intro
(7, {2}) intro
(8, {}) leaf
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_decompositions.tree_decomposition import make_nice_
↪tree_decomposition, label_nice_tree_decomposition
>>> claw = graphs.CompleteBipartiteGraph(Integer(1), Integer(3))
>>> claw_TD = claw.treewidth(certificate=True)
>>> nice_TD = make_nice_tree_decomposition(claw, claw_TD)
>>> label_TD = label_nice_tree_decomposition(nice_TD, directed=True)
>>> label_TD.name()
'Labelled Nice tree decomposition of Tree decomposition'
>>> for node in sorted(label_TD):  # random
...     print(node, label_TD.get_vertex(node))
(0, {}) forget
(1, {0}) forget
(2, {0, 1}) intro
(3, {0}) forget
(4, {0, 3}) intro
(5, {0}) forget
(6, {0, 2}) intro
(7, {2}) intro
(8, {}) leaf
```

sage.graphs.graph_decompositions.tree_decomposition.**length_of_tree_decomposition**(*G*, *T*, *check=True*)

Return the length of the tree decomposition T of G.

The *length* of a tree decomposition, as proposed in [DG2006], is the maximum *diameter* in G of its bags, where the diameter of a bag X_i is the largest distance in G between the vertices in X_i (i.e., $\max_{u,v \in X_i} \text{dist}_G(u,v)$). See the documentation of the *tree_decomposition* module for more details.

INPUT:

- `G` – a graph
- `T` – a tree-decomposition for G
- `check` – boolean (default: `True`); whether to check that the tree-decomposition T is valid for G

EXAMPLES:

Trees and cliques have treelength 1:

```
sage: from sage.graphs.graph_decompositions.tree_decomposition import length_of_
↪tree_decomposition
sage: G = graphs.CompleteGraph(5)
sage: tl, T = G.treelength(certificate=True)
sage: tl
1
```

(continues on next page)

(continued from previous page)

```
sage: length_of_tree_decomposition(G, T, check=True)
1
sage: G = graphs.RandomTree(20)
sage: tl, T = G.treelength(certificate=True)
sage: tl
1
sage: length_of_tree_decomposition(G, T, check=True)
1
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_decompositions.tree_decomposition import length_of_
↪tree_decomposition
>>> G = graphs.CompleteGraph(Integer(5))
>>> tl, T = G.treelength(certificate=True)
>>> tl
1
>>> length_of_tree_decomposition(G, T, check=True)
1
>>> G = graphs.RandomTree(Integer(20))
>>> tl, T = G.treelength(certificate=True)
>>> tl
1
>>> length_of_tree_decomposition(G, T, check=True)
1
```

The Petersen graph has treelength 2:

```
sage: G = graphs.PetersenGraph()
sage: tl, T = G.treelength(certificate=True)
sage: tl
2
sage: length_of_tree_decomposition(G, T)
2
```

```
>>> from sage.all import *
>>> G = graphs.PetersenGraph()
>>> tl, T = G.treelength(certificate=True)
>>> tl
2
>>> length_of_tree_decomposition(G, T)
2
```

When a tree-decomposition has a single bag containing all vertices of a graph, the length of this tree-decomposition is the diameter of the graph:

```
sage: G = graphs.Grid2dGraph(2, 5)
sage: G.treelength()
2
sage: G.diameter()
5
sage: T = Graph({Set(G): []})
sage: length_of_tree_decomposition(G, T)
```

(continues on next page)

(continued from previous page)

```
5
```

```
>>> from sage.all import *
>>> G = graphs.Grid2dGraph(Integer(2), Integer(5))
>>> G.treelength()
2
>>> G.diameter()
5
>>> T = Graph({Set(G): []})
>>> length_of_tree_decomposition(G, T)
5
```

sage.graphs.graph_decompositions.tree_decomposition.**make_nice_tree_decomposition**(*graph*, *tree_decomp*)

Return a *nice* tree decomposition (TD) of the TD `tree_decomp`.

See page 161 of [CFKLMPPS15] for a description of the nice tree decomposition.

A *nice* TD NT is a rooted tree with four types of nodes:

- *Leaf* nodes have no children and bag size 0;
- *Introduce* nodes have one child: If $v \in NT$ is an introduce node and $w \in NT$ its child, then $Bag(v) = Bag(w) \cup \{x\}$, where x is the introduced node;
- *Forget* nodes have one child: If $v \in NT$ is a forget node and $w \in NT$ its child, then $Bag(v) = Bag(w) \backslash \{x\}$, where x is the forgotten node;
- *Join* nodes have two children, both identical to the parent.

INPUT:

- `graph` – a Sage graph
- `tree_decomp` – a tree decomposition

OUTPUT: a nice tree decomposition whose root vertex is a tuple of the form `(0, bag)`.

Warning

This method assumes that the vertices of the input tree `tree_decomp` are hashable and have attribute `issuperset`, e.g., `frozenset` or `Set_object_enumerated_with_category`.

EXAMPLES:

```
sage: from sage.graphs.graph_decompositions.tree_decomposition import make_nice_
↪tree_decomposition
sage: petersen = graphs.PetersenGraph()
sage: petersen_TD = petersen.treewidth(certificate=True)
sage: make_nice_tree_decomposition(petersen, petersen_TD)
Nice tree decomposition of Tree decomposition: Graph on 28 vertices
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_decompositions.tree_decomposition import make_nice_
```

(continues on next page)

(continued from previous page)

```
↪tree_decomposition
>>> petersen = graphs.PetersenGraph()
>>> petersen_TD = petersen.treewidth(certificate=True)
>>> make_nice_tree_decomposition(petersen, petersen_TD)
Nice tree decomposition of Tree decomposition: Graph on 28 vertices
```

```
sage: from sage.graphs.graph_decompositions.tree_decomposition import make_nice_
↪tree_decomposition
sage: cherry = graphs.CompleteBipartiteGraph(1, 2)
sage: cherry_TD = cherry.treewidth(certificate=True)
sage: make_nice_tree_decomposition(cherry, cherry_TD)
Nice tree decomposition of Tree decomposition: Graph on 7 vertices
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_decompositions.tree_decomposition import make_nice_
↪tree_decomposition
>>> cherry = graphs.CompleteBipartiteGraph(Integer(1), Integer(2))
>>> cherry_TD = cherry.treewidth(certificate=True)
>>> make_nice_tree_decomposition(cherry, cherry_TD)
Nice tree decomposition of Tree decomposition: Graph on 7 vertices
```

```
sage: from sage.graphs.graph_decompositions.tree_decomposition import make_nice_
↪tree_decomposition
sage: bip_one_four = graphs.CompleteBipartiteGraph(1, 4)
sage: bip_one_four_TD = bip_one_four.treewidth(certificate=True)
sage: make_nice_tree_decomposition(bip_one_four, bip_one_four_TD)
Nice tree decomposition of Tree decomposition: Graph on 15 vertices
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_decompositions.tree_decomposition import make_nice_
↪tree_decomposition
>>> bip_one_four = graphs.CompleteBipartiteGraph(Integer(1), Integer(4))
>>> bip_one_four_TD = bip_one_four.treewidth(certificate=True)
>>> make_nice_tree_decomposition(bip_one_four, bip_one_four_TD)
Nice tree decomposition of Tree decomposition: Graph on 15 vertices
```

Check that Issue #36843 is fixed:

```
sage: from sage.graphs.graph_decompositions.tree_decomposition import make_nice_
↪tree_decomposition
sage: triangle = graphs.CompleteGraph(3)
sage: triangle_TD = triangle.treewidth(certificate=True)
sage: make_nice_tree_decomposition(triangle, triangle_TD)
Nice tree decomposition of Tree decomposition: Graph on 7 vertices
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_decompositions.tree_decomposition import make_nice_
↪tree_decomposition
>>> triangle = graphs.CompleteGraph(Integer(3))
>>> triangle_TD = triangle.treewidth(certificate=True)
>>> make_nice_tree_decomposition(triangle, triangle_TD)
```

(continues on next page)

(continued from previous page)

```
Nice tree decomposition of Tree decomposition: Graph on 7 vertices
```

```
sage: from sage.graphs.graph_decompositions.tree_decomposition import make_nice_
↪tree_decomposition
sage: graph = graphs.CompleteBipartiteGraph(2, 5)
sage: graph_TD = graph.treewidth(certificate=True)
sage: make_nice_tree_decomposition(graph, graph_TD)
Nice tree decomposition of Tree decomposition: Graph on 25 vertices
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_decompositions.tree_decomposition import make_nice_
↪tree_decomposition
>>> graph = graphs.CompleteBipartiteGraph(Integer(2), Integer(5))
>>> graph_TD = graph.treewidth(certificate=True)
>>> make_nice_tree_decomposition(graph, graph_TD)
Nice tree decomposition of Tree decomposition: Graph on 25 vertices
```

```
sage: from sage.graphs.graph_decompositions.tree_decomposition import make_nice_
↪tree_decomposition
sage: empty_graph = graphs.EmptyGraph()
sage: tree_decomp = empty_graph.treewidth(certificate=True)
sage: len(make_nice_tree_decomposition(empty_graph, tree_decomp))
0
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_decompositions.tree_decomposition import make_nice_
↪tree_decomposition
>>> empty_graph = graphs.EmptyGraph()
>>> tree_decomp = empty_graph.treewidth(certificate=True)
>>> len(make_nice_tree_decomposition(empty_graph, tree_decomp))
0
```

```
sage: from sage.graphs.graph_decompositions.tree_decomposition import make_nice_
↪tree_decomposition
sage: singleton = graphs.CompleteGraph(1)
sage: tree_decomp = singleton.treewidth(certificate=True)
sage: make_nice_tree_decomposition(singleton, tree_decomp)
Nice tree decomposition of Tree decomposition: Graph on 3 vertices
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_decompositions.tree_decomposition import make_nice_
↪tree_decomposition
>>> singleton = graphs.CompleteGraph(Integer(1))
>>> tree_decomp = singleton.treewidth(certificate=True)
>>> make_nice_tree_decomposition(singleton, tree_decomp)
Nice tree decomposition of Tree decomposition: Graph on 3 vertices
```

```
sage: from sage.graphs.graph_decompositions.tree_decomposition import make_nice_
↪tree_decomposition
sage: an_edge = graphs.CompleteGraph(2)
```

(continues on next page)

(continued from previous page)

```
sage: tree_decomp = an_edge.treewidth(certificate=True)
sage: make_nice_tree_decomposition(an_edge, tree_decomp)
Nice tree decomposition of Tree decomposition: Graph on 5 vertices
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_decompositions.tree_decomposition import make_nice_
↪tree_decomposition
>>> an_edge = graphs.CompleteGraph(Integer(2))
>>> tree_decomp = an_edge.treewidth(certificate=True)
>>> make_nice_tree_decomposition(an_edge, tree_decomp)
Nice tree decomposition of Tree decomposition: Graph on 5 vertices
```

sage.graphs.graph_decompositions.tree_decomposition.**reduced_tree_decomposition**(T)

Return a reduced tree-decomposition of T.

We merge all edges between two sets S and S' where S is a subset of S'. To do so, we use a simple union-find data structure to record merge operations and the good sets.

Warning

This method assumes that the vertices of the input tree T are hashable and have attribute `issuperset`, e.g., `frozenset` or `Set_object_enumerated`.

INPUT:

- `T` – a tree-decomposition

EXAMPLES:

```
sage: from sage.graphs.graph_decompositions.tree_decomposition import reduced_
↪tree_decomposition
sage: from sage.graphs.graph_decompositions.tree_decomposition import is_valid_
↪tree_decomposition
sage: G = graphs.PathGraph(3)
sage: T = Graph()
sage: T.add_path([Set([0]), Set([0, 1]), Set([1]), Set([1, 2]),  Set([2])])
sage: T.order()
5
sage: is_valid_tree_decomposition(G, T)
True
sage: T2 = reduced_tree_decomposition(T)
sage: is_valid_tree_decomposition(G, T2)
True
sage: T2.order()
2
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_decompositions.tree_decomposition import reduced_tree_
↪decomposition
>>> from sage.graphs.graph_decompositions.tree_decomposition import is_valid_tree_
↪decomposition
>>> G = graphs.PathGraph(Integer(3))
```

(continues on next page)

(continued from previous page)

```
>>> T = Graph()
>>> T.add_path([Set([Integer(0)]), Set([Integer(0), Integer(1)]),␣
↪Set([Integer(1)]), Set([Integer(1), Integer(2)]),  Set([Integer(2)])])
>>> T.order()
5
>>> is_valid_tree_decomposition(G, T)
True
>>> T2 = reduced_tree_decomposition(T)
>>> is_valid_tree_decomposition(G, T2)
True
>>> T2.order()
2
```

sage.graphs.graph_decompositions.tree_decomposition.**treelength**(*G*, *k=None*, *certificate=False*)

Compute the treelength of G (and provide a decomposition).

The *length* of a tree decomposition, as proposed in [DG2006], is the maximum *diameter* in G of its bags, where the diameter of a bag X_i is the largest distance in G between the vertices in X_i (i.e., $\max_{u,v \in X_i} \text{dist}_G(u,v)$). The *treelength* $tl(G)$ of a graph G is the minimum length among all possible tree decompositions of G. See the documentation of the *tree_decomposition* module for more details.

INPUT:

- `G` – a sage Graph
- `k` – integer (default: `None`); indicates the length to be considered. When k is an integer, the method checks that the graph has treelength $\leq k$. If k is `None` (default), the method computes the optimal treelength.
- `certificate` – boolean (default: `False`); whether to also return the tree-decomposition itself

OUTPUT:

`G.treelength()` returns the treelength of G. When k is specified, it returns `False` when no tree-decomposition of length $\leq k$ exists or `True` otherwise. When `certificate=True`, the tree-decomposition is also returned.

ALGORITHM:

This method virtually explores the graph of all pairs `(vertex_cut, connected_component)`, where `vertex_cut` is a vertex cut of the graph of length $\leq k$, and `connected_component` is a connected component of the graph induced by `G - vertex_cut`.

We deduce that the pair `(vertex_cut, connected_component)` is feasible with treelength k if `connected_component` is empty, or if a vertex `v` from `vertex_cut` can be replaced with a vertex from `connected_component`, such that the pair `(vertex_cut + v, connected_component - v)` is feasible.

In practice, this method decomposes the graph by its clique minimal separators into atoms, computes the treelength of each of atom and returns the maximum value over all the atoms. Indeed, we have that $tl(G) = \max_{X \in A} tl(G[X])$ where A is the set of atoms of the decomposition by clique separators of G. When `certificate == True`, the tree-decompositions of the atoms are connected to each others by adding edges with respect to the clique separators.

See also

- *treewidth()* computes the treewidth of a graph.
- *path_decomposition()* computes the pathwidth of a graph.
- module *vertex_separation*.

- *atoms_and_clique_separators()*

EXAMPLES:

The PetersenGraph has treelength 2:

```
sage: G = graphs.PetersenGraph()
sage: G.treelength()
2
```

```
>>> from sage.all import *
>>> G = graphs.PetersenGraph()
>>> G.treelength()
2
```

Disconnected graphs have infinite treelength:

```
sage: G = Graph(2)
sage: G.treelength()
+Infinity
sage: G.treelength(k=+Infinity)
True
sage: G.treelength(k=2)
False
sage: G.treelength(certificate=True)
Traceback (most recent call last):
...
ValueError: the tree decomposition of a disconnected graph is not defined
```

```
>>> from sage.all import *
>>> G = Graph(Integer(2))
>>> G.treelength()
+Infinity
>>> G.treelength(k=+Infinity)
True
>>> G.treelength(k=Integer(2))
False
>>> G.treelength(certificate=True)
Traceback (most recent call last):
...
ValueError: the tree decomposition of a disconnected graph is not defined
```

Chordal graphs have treelength 1:

```
sage: G = graphs.RandomChordalGraph(30)
sage: while not G.is_connected():
....:     G = graphs.RandomChordalGraph(30)
sage: G.treelength()
1
```

```
>>> from sage.all import *
>>> G = graphs.RandomChordalGraph(Integer(30))
```

(continues on next page)

(continued from previous page)

```
>>> while not G.is_connected():
...     G = graphs.RandomChordalGraph(Integer(30))
>>> G.treelength()
1
```

Cycles have treelength $\lceil n/3 \rceil$:

```
sage: [graphs.CycleGraph(n).treelength() for n in range(3, 11)]
[1, 2, 2, 2, 3, 3, 3, 4]
```

```
>>> from sage.all import *
>>> [graphs.CycleGraph(n).treelength() for n in range(Integer(3), Integer(11))]
[1, 2, 2, 2, 3, 3, 3, 4]
```

sage.graphs.graph_decompositions.tree_decomposition.**treelength_lowerbound**(G)

Return a lower bound on the treelength of G.

See [DG2006] for more details.

INPUT:

- `G` – a sage Graph

EXAMPLES:

```
sage: from sage.graphs.graph_decompositions.tree_decomposition import treelength_
↪lowerbound
sage: G = graphs.PetersenGraph()
sage: treelength_lowerbound(G)
1
sage: G.treelength()
2
sage: G = graphs.CycleGraph(5)
sage: treelength_lowerbound(G)
2
sage: G.treelength()
2
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_decompositions.tree_decomposition import treelength_
↪lowerbound
>>> G = graphs.PetersenGraph()
>>> treelength_lowerbound(G)
1
>>> G.treelength()
2
>>> G = graphs.CycleGraph(Integer(5))
>>> treelength_lowerbound(G)
2
>>> G.treelength()
2
```

sage.graphs.graph_decompositions.tree_decomposition.**treewidth**(*g*, *k=None*, *kmin=None*, *certificate=False*, *algorithm=None*, *nice=False*)

Compute the treewidth of g (and provide a decomposition).

INPUT:

- `g` – a Sage Graph
- `k` – integer (default: `None`); indicates the width to be considered. When `k` is an integer, the method checks that the graph has treewidth $\leq k$. If `k` is `None` (default), the method computes the optimal treewidth.
- `kmin` – integer (default: `None`); when specified, search for a tree-decomposition of width at least `kmin`. This parameter is useful when the graph can be decomposed into atoms. This parameter is ignored when `k` is not `None` or when `algorithm == 'tdlib'`.
- `certificate` – boolean (default: `False`); whether to return the tree-decomposition itself
- `algorithm` – whether to use `'sage'` or `'tdlib'` (requires the installation of the tdlib: Algorithms for computing tree decompositions of graphs package). The default behaviour is to use 'tdlib' if it is available, and Sage's own algorithm when it is not.
- `nice` – boolean (default: `False`); whether or not to return the nice tree decomposition, provided `certificate` is `True`

OUTPUT:

`g.treewidth()` returns treewidth of the graph `g`.

When `k` is specified, it returns `False` if there is no tree decomposition of width $\leq k$, and `True` otherwise.

When `certificate=True`, the tree decomposition is returned.

When `nice=True`, the nice tree decomposition is returned. See *make_nice_tree_decomposition()* for details.

ALGORITHM:

This function virtually explores the graph of all pairs `(vertex_cut, cc)`, where `vertex_cut` is a vertex cut of the graph of cardinality $\leq k+1$, and `connected_component` is a connected component of the graph induced by `G-vertex_cut`.

We deduce that the pair `(vertex_cut, cc)` is feasible with treewidth k if `cc` is empty, or if a vertex `v` from `vertex_cut` can be replaced with a vertex from `cc`, such that the pair `(vertex_cut+v,cc-v)` is feasible.

Note

The implementation would be much faster if `cc`, the argument of the recursive function, was a bitset. It would also be very nice to not copy the graph in order to compute connected components, for this is really a waste of time.

See also

- The utility functions in the *tree_decomposition* module, especially *label_nice_tree_decomposition()*.
- *path_decomposition()* computes the pathwidth of a graph.
- module *vertex_separation*.

EXAMPLES:

The PetersenGraph has treewidth 4:

```
sage: petersen = graphs.PetersenGraph()
sage: petersen.treewidth(algorithm='sage')
4
sage: petersen.treewidth(algorithm='sage', certificate=True)
Tree decomposition: Graph on 6 vertices
```

```
>>> from sage.all import *
>>> petersen = graphs.PetersenGraph()
>>> petersen.treewidth(algorithm='sage')
4
>>> petersen.treewidth(algorithm='sage', certificate=True)
Tree decomposition: Graph on 6 vertices
```

The PetersenGraph has treewidth 4 (with `tdlib`):

```
sage: petersen = graphs.PetersenGraph()
sage: petersen.treewidth(algorithm='tdlib')                    # optional - tdlib
4
sage: petersen.treewidth(algorithm='tdlib', certificate=True)  # optional - tdlib
Tree decomposition: Graph on 6 vertices
```

```
>>> from sage.all import *
>>> petersen = graphs.PetersenGraph()
>>> petersen.treewidth(algorithm='tdlib')                    # optional - tdlib
4
>>> petersen.treewidth(algorithm='tdlib', certificate=True)  # optional - tdlib
Tree decomposition: Graph on 6 vertices
```

Nice tree decomposition of the PetersenGraph has 28 nodes:

```
sage: petersen = graphs.PetersenGraph()
sage: petersen.treewidth(algorithm='sage', certificate=True, nice=True)
Nice tree decomposition of Tree decomposition: Graph on 28 vertices
sage: petersen.treewidth(algorithm='tdlib', certificate=True, nice=True)  #␣
↪optional - tdlib
Nice tree decomposition of Tree decomposition: Graph on 28 vertices
```

```
>>> from sage.all import *
>>> petersen = graphs.PetersenGraph()
>>> petersen.treewidth(algorithm='sage', certificate=True, nice=True)
Nice tree decomposition of Tree decomposition: Graph on 28 vertices
>>> petersen.treewidth(algorithm='tdlib', certificate=True, nice=True)  #␣
↪optional - tdlib
Nice tree decomposition of Tree decomposition: Graph on 28 vertices
```

The treewidth of a 2-dimensional grid is its smallest side:

```
sage: graphs.Grid2dGraph(2,5).treewidth()
2
sage: graphs.Grid2dGraph(3,5).treewidth()
3
```

```
>>> from sage.all import *
>>> graphs.Grid2dGraph(Integer(2),Integer(5)).treewidth()
2
>>> graphs.Grid2dGraph(Integer(3),Integer(5)).treewidth()
3
```

When parameter `kmin` is specified, the method searches for a tree-decomposition of width at least `kmin`:

```
sage: g = graphs.PetersenGraph()
sage: g.treewidth()
4
sage: g.treewidth(kmin=2, algorithm='sage')
4
sage: g.treewidth(kmin=g.order(), certificate=True, algorithm='sage')
Tree decomposition: Graph on 1 vertex
```

```
>>> from sage.all import *
>>> g = graphs.PetersenGraph()
>>> g.treewidth()
4
>>> g.treewidth(kmin=Integer(2), algorithm='sage')
4
>>> g.treewidth(kmin=g.order(), certificate=True, algorithm='sage')
Tree decomposition: Graph on 1 vertex
```

sage.graphs.graph_decompositions.tree_decomposition.**width_of_tree_decomposition**(*G*, *T*, *check=True*)

Return the width of the tree decomposition T of G.

The width of a tree-decomposition is the size of the largest bag minus 1. The empty graph and a graph of order 1 have treewidth 0.

INPUT:

- `G` – a sage Graph
- `T` – a tree-decomposition for G
- `check` – boolean (default: `True`); whether to check that the tree-decomposition T is valid for G

EXAMPLES:

```
sage: from sage.graphs.graph_decompositions.tree_decomposition import width_of_
↪tree_decomposition
sage: G = graphs.PathGraph(3)
sage: T = G.treewidth(certificate=True)
sage: width_of_tree_decomposition(G, T, check=True)
1
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_decompositions.tree_decomposition import width_of_tree_
↪decomposition
>>> G = graphs.PathGraph(Integer(3))
>>> T = G.treewidth(certificate=True)
>>> width_of_tree_decomposition(G, T, check=True)
1
```

5.24 Vertex separation

This module implements several algorithms to compute the vertex separation of a digraph and the corresponding ordering of the vertices. It also implements tests functions for evaluation the width of a linear ordering.

Given an ordering $v_1, \cdots, v_n$ of the vertices of $V(G)$, its *cost* is defined as:

$$c(v_1, ..., v_n) = \max_{1 \leq i \leq n} c'(\{v_1, ..., v_i\})$$

Where

$$c'(S) = |N_G^+(S) \backslash S|$$

The *vertex separation* of a digraph G is equal to the minimum cost of an ordering of its vertices.

Vertex separation and pathwidth

The vertex separation is defined on a digraph, but one can obtain from a graph G a digraph D with the same vertex set, and in which each edge uv of G is replaced by two edges uv and vu in D. The vertex separation of D is equal to the pathwidth of G, and the corresponding ordering of the vertices of D, also called a *layout*, encodes an optimal path-decomposition of G. This is a result of Kinnersley [Kin1992] and Bodlaender [Bod1998].

This module contains the following methods

pathwidth()	Compute the pathwidth of `self` (and provides a decomposition)
path_decomposition()	Return the pathwidth of the given graph and the ordering of the vertices resulting in a corresponding path decomposition
vertex_separation()	Return an optimal ordering of the vertices and its cost for vertex-separation
vertex_separation_exp()	Compute the vertex separation of G using an exponential time and space algorithm
vertex_separation_MILP()	Compute the vertex separation of G and the optimal ordering of its vertices using an MILP formulation
vertex_separation_BAB()	Compute the vertex separation of G and the optimal ordering of its vertices using a branch and bound algorithm
lower_bound()	Return a lower bound on the vertex separation of G
is_valid_ordering()	Test if the linear vertex ordering L is valid for (di)graph G
width_of_path_decomposition()	Return the width of the path decomposition induced by the linear ordering L of the vertices of G
linear_ordering_to_path_decomposition()	Return the path decomposition encoded in the ordering L

5.24.1 Exponential algorithm for vertex separation

In order to find an optimal ordering of the vertices for the vertex separation, this algorithm tries to save time by computing the function $c'(S)$ **at most once** once for each of the sets $S \subseteq V(G)$. These values are stored in an array of size 2^n where reading the value of $c'(S)$ or updating it can be done in constant (and small) time.

Assuming that we can compute the cost of a set S and remember it, finding an optimal ordering is an easy task. Indeed, we can think of the sequence $v_1, ..., v_n$ of vertices as a sequence of *sets* $\{v_1\}, \{v_1, v_2\}, ..., \{v_1, ..., v_n\}$, whose cost is precisely $\max c'(\{v_1\}), c'(\{v_1, v_2\}), ..., c'(\{v_1, ..., v_n\})$. Hence, when considering the digraph on the 2^n sets $S \subseteq V(G)$ where there is an arc from S to S' if $S' = S \cap \{v\}$ for some v (that is, if the sets S and S' can be consecutive in a sequence), an ordering of the vertices of G corresponds to a *path* from $\emptyset$ to $\{v_1, ..., v_n\}$. In this setting, checking whether there exists a ordering of cost less than k can be achieved by checking whether there exists a directed path $\emptyset$ to $\{v_1, ..., v_n\}$ using only sets of cost less than k. This is just a depth-first-search, for each k.

Lazy evaluation of c'

In the previous algorithm, most of the time is actually spent on the computation of $c'(S)$ for each set $S \subseteq V(G)$ – i.e. 2^n computations of neighborhoods. This can be seen as a huge waste of time when noticing that it is useless to know that the value $c'(S)$ for a set S is less than k if all the paths leading to S have a cost greater than k. For this reason, the value of $c'(S)$ is computed lazily during the depth-first search. Explanation :

When the depth-first search discovers a set of size less than k, the costs of its out-neighbors (the potential sets that could follow it in the optimal ordering) are evaluated. When an out-neighbor is found that has a cost smaller than k, the depth-first search continues with this set, which is explored with the hope that it could lead to a path toward $\{v_1, ..., v_n\}$. On the other hand, if an out-neighbour has a cost larger than k it is useless to attempt to build a cheap sequence going though this set, and the exploration stops there. This way, a large number of sets will never be evaluated and *a lot* of computational time is saved this way.

Besides, some improvement is also made by "improving" the values found by c'. Indeed, $c'(S)$ is a lower bound on the cost of a sequence containing the set S, but if all out-neighbors of S have a cost of $c'(S) + 5$ then one knows that having S in a sequence means a total cost of at least $c'(S) + 5$. For this reason, for each set S we store the value of $c'(S)$, and replace it by $\max(c'(S), \min_{\text{next}})$ (where $\min_{\text{next}}$ is the minimum of the costs of the out-neighbors of S) once the costs of these out-neighbors have been evaluated by the algorithm.

Note

Because of its current implementation, this algorithm only works on graphs on less than 32 vertices. This can be changed to 64 if necessary, but 32 vertices already require 4GB of memory. Running it on 64 bits is not expected to be doable by the computers of the next decade : $-D$

Lower bound on the vertex separation

One can obtain a lower bound on the vertex separation of a graph in exponential time but *small* memory by computing once the cost of each set S. Indeed, the cost of a sequence $v_1, ..., v_n$ corresponding to sets $\{v_1\}, \{v_1, v_2\}, ..., \{v_1, ..., v_n\}$ is

$$\max c'(\{v_1\}), c'(\{v_1, v_2\}), ..., c'(\{v_1, ..., v_n\}) \geq \max c'_1, ..., c'_n$$

where c_i is the minimum cost of a set S on i vertices. Evaluating the c_i can take time (and in particular more than the previous exact algorithm), but it does not need much memory to run.

5.24.2 MILP formulation for the vertex separation

We describe below a mixed integer linear program (MILP) for determining an optimal layout for the vertex separation of G, which is an improved version of the formulation proposed in [SP2010]. It aims at building a sequence S_t of sets such that an ordering $v_1, ..., v_n$ of the vertices correspond to $S_0 = \{v_1\}, S_2 = \{v_1, v_2\}, ..., S_{n-1} = \{v_1, ..., v_n\}$.

Variables:

- y_v^t – variable set to 1 if $v \in S_t$, and 0 otherwise. The order of v in the layout is the smallest t such that $y_v^t == 1$.
- u_v^t – variable set to 1 if $v \notin S_t$ and v has an in-neighbor in S_t. It is set to 0 otherwise.
- x_v^t – variable set to 1 if either $v \in S_t$ or if v has an in-neighbor in S_t. It is set to 0 otherwise.
- z – objective value to minimize. It is equal to the maximum over all step t of the number of vertices such that $u_v^t == 1$.

MILP formulation:

$$\begin{aligned}
\text{Minimize:} & \quad z && && (5.1)\\
\text{Such that:} & \quad x_v^t \leq x_v^{t+1} && \forall v \in V,\ 0 \leq t \leq n-2 && (5.2)\\
& \quad y_v^t \leq y_v^{t+1} && \forall v \in V,\ 0 \leq t \leq n-2 && (5.3)\\
& \quad y_v^t \leq x_w^t && \forall v \in V,\ \forall w \in N^+(v),\ 0 \leq t \leq n-1 && (5.4)\\
& \quad \sum_{v \in V} y_v^t = t+1 && 0 \leq t \leq n-1 && (5.5)\\
& \quad x_v^t - y_v^t \leq u_v^t && \forall v \in V,\ 0 \leq t \leq n-1 && (5.6)\\
& \quad \sum_{v \in V} u_v^t \leq z && 0 \leq t \leq n-1 && (5.7)\\
& \quad 0 \leq x_v^t \leq 1 && \forall v \in V,\ 0 \leq t \leq n-1 && (5.8)\\
& \quad 0 \leq u_v^t \leq 1 && \forall v \in V,\ 0 \leq t \leq n-1 && (5.9)\\
& \quad y_v^t \in \{0,1\} && \forall v \in V,\ 0 \leq t \leq n-1 && (5.10)\\
& \quad 0 \leq z \leq n && && (5.11)
\end{aligned}$$

The vertex separation of G is given by the value of z, and the order of vertex v in the optimal layout is given by the smallest t for which $y_v^t == 1$.

5.24.3 Branch and Bound algorithm for the vertex separation

We describe below the principle of a branch and bound algorithm (BAB) for determining an optimal ordering for the vertex separation of G, as proposed in [CMN2014].

Greedy steps:

Let us denote $\mathcal{L}(S)$ the set of all possible orderings of the vertices in S, and let $\mathcal{L}_P(S) \subseteq \mathcal{L}(S)$ be the orderings starting with a prefix P. Let also $c(L)$ be the cost of the ordering $L \in \mathcal{L}(V)$ as defined above.

Given a digraph $D = (V, A)$, a set $S \subset V$, and a prefix P, it has been proved in [CMN2014] that $\min_{L \in \mathcal{L}_P(V)} c(L) = \min_{L \in \mathcal{L}_{P+v}(V)} c(L)$ holds in two (non exhaustive) cases:

$$\text{or } \begin{cases} N^+(v) \subseteq S \cup N^+(S) \\ v \in N^+(S) \text{ and } N^+(v) \setminus (S \cup N^+(S)) = \{w\} \end{cases}$$

In other words, if we find a vertex v satisfying the above conditions, the best possible ordering with prefix P has the same cost as the best possible ordering with prefix $P + v$. So we can greedily extend the prefix with vertices satisfying the conditions which results in a significant reduction of the search space.

The algorithm:

Given the current prefix P and the current upper bound UB (either an input upper bound or the cost of the best solution found so far), apply the following steps:

- Extend the prefix P into a prefix P' using the greedy steps as described above.
- Sort the vertices $v \in V \setminus P'$ by increasing values of $|N^+(P + v)|$, and prune the vertices with a value larger or equal to UB. Let Δ be the resulting sorted list.
- Repeat with prefix $P' + v$ for all $v \in \Delta$ and keep the best found solution.

If a lower bound is passed to the algorithm, it will stop as soon as a solution with cost equal to that lower bound is found.

Storing prefixes:

If for a prefix P we have $c(P) < \min_{L \in \mathcal{L}_P(V)} c(L) = C$, then for any permutation P' of P we have $\min_{L \in \mathcal{L}_{P'}(V)} c(L) \geq C$.

Thus, given such a prefix P there is no need to explore any of the orderings starting with one of its permutations. To do so, we store P (as a set of vertices) to cut branches later. See [CMN2014] for more details.

Since the number of stored sets can get very large, one can control the maximum length and the maximum number of stored prefixes.

5.24.4 Authors

- Nathann Cohen (2011-10): Initial version and exact exponential algorithm
- David Coudert (2012-04): MILP formulation and tests functions
- David Coudert (2015-01): BAB formulation and tests functions

5.24.5 Methods

sage.graphs.graph_decompositions.vertex_separation.**is_valid_ordering**(G, L)

Test if the linear vertex ordering L is valid for (di)graph G.

A linear ordering L of the vertices of a (di)graph G is valid if all vertices of G are in L, and if L contains no other vertex and no duplicated vertices.

INPUT:

- `G` – a Graph or a DiGraph
- `L` – an ordered list of the vertices of `G`

OUTPUT:

Returns `True` if L is a valid vertex ordering for G, and `False` otherwise.

EXAMPLES:

Path decomposition of a cycle:

```
sage: from sage.graphs.graph_decompositions import vertex_separation
sage: G = graphs.CycleGraph(6)
sage: L = G.vertices(sort=True)
sage: vertex_separation.is_valid_ordering(G, L)
True
sage: vertex_separation.is_valid_ordering(G, [1,2])
False
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_decompositions import vertex_separation
>>> G = graphs.CycleGraph(Integer(6))
>>> L = G.vertices(sort=True)
>>> vertex_separation.is_valid_ordering(G, L)
True
>>> vertex_separation.is_valid_ordering(G, [Integer(1),Integer(2)])
False
```

sage.graphs.graph_decompositions.vertex_separation.**linear_ordering_to_path_decomposition**(G, L, *im-mutable=None*)

Return the path decomposition encoded in the ordering L.

INPUT:

- G – a Graph
- L – a linear ordering for G
- immutable – boolean (default: None); whether to create a mutable/immutable graph. immutable=None (default) means that the graph and its path decomposition will behave the same way.

OUTPUT: a path graph whose vertices are the bags of the path decomposition

EXAMPLES:

The bags of an optimal path decomposition of a path-graph have two vertices each:

```
sage: from sage.graphs.graph_decompositions.vertex_separation import vertex_
↪separation
sage: from sage.graphs.graph_decompositions.vertex_separation import linear_
↪ordering_to_path_decomposition
sage: g = graphs.PathGraph(5)
sage: pw, L = vertex_separation(g, algorithm = "BAB"); pw
1
sage: h = linear_ordering_to_path_decomposition(g, L)
sage: sorted(h, key=str)
[{0, 1}, {1, 2}, {2, 3}, {3, 4}]
sage: sorted(h.edge_iterator(labels=None), key=str)
[({0, 1}, {1, 2}), ({1, 2}, {2, 3}), ({2, 3}, {3, 4})]
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_decompositions.vertex_separation import vertex_
↪separation
>>> from sage.graphs.graph_decompositions.vertex_separation import linear_
↪ordering_to_path_decomposition
>>> g = graphs.PathGraph(Integer(5))
>>> pw, L = vertex_separation(g, algorithm = "BAB"); pw
1
>>> h = linear_ordering_to_path_decomposition(g, L)
>>> sorted(h, key=str)
[{0, 1}, {1, 2}, {2, 3}, {3, 4}]
>>> sorted(h.edge_iterator(labels=None), key=str)
[({0, 1}, {1, 2}), ({1, 2}, {2, 3}), ({2, 3}, {3, 4})]
```

Giving a non-optimal linear ordering:

```
sage: g = graphs.PathGraph(5)
sage: L = [1, 4, 0, 2, 3]
sage: from sage.graphs.graph_decompositions.vertex_separation import width_of_
↪path_decomposition
sage: width_of_path_decomposition(g, L)
3
sage: h = linear_ordering_to_path_decomposition(g, L)
sage: h.vertices(sort=True)
[{0, 2, 3, 4}, {0, 1, 2}]
```

```
>>> from sage.all import *
>>> g = graphs.PathGraph(Integer(5))
>>> L = [Integer(1), Integer(4), Integer(0), Integer(2), Integer(3)]
>>> from sage.graphs.graph_decompositions.vertex_separation import width_of_path_
```

(continues on next page)

(continued from previous page)

```
↪decomposition
>>> width_of_path_decomposition(g, L)
3
>>> h = linear_ordering_to_path_decomposition(g, L)
>>> h.vertices(sort=True)
[{0, 2, 3, 4}, {0, 1, 2}]
```

The bags of the path decomposition of a cycle have three vertices each:

```
sage: g = graphs.CycleGraph(6)
sage: pw, L = vertex_separation(g, algorithm = "BAB"); pw
2
sage: h = linear_ordering_to_path_decomposition(g, L)
sage: sorted(h, key=str)
[{0, 1, 5}, {1, 2, 5}, {2, 3, 4}, {2, 4, 5}]
sage: sorted(h.edge_iterator(labels=None), key=str)
[({0, 1, 5}, {1, 2, 5}), ({1, 2, 5}, {2, 4, 5}), ({2, 4, 5}, {2, 3, 4})]
```

```
>>> from sage.all import *
>>> g = graphs.CycleGraph(Integer(6))
>>> pw, L = vertex_separation(g, algorithm = "BAB"); pw
2
>>> h = linear_ordering_to_path_decomposition(g, L)
>>> sorted(h, key=str)
[{0, 1, 5}, {1, 2, 5}, {2, 3, 4}, {2, 4, 5}]
>>> sorted(h.edge_iterator(labels=None), key=str)
[({0, 1, 5}, {1, 2, 5}), ({1, 2, 5}, {2, 4, 5}), ({2, 4, 5}, {2, 3, 4})]
```

sage.graphs.graph_decompositions.vertex_separation.**lower_bound**(G)

Return a lower bound on the vertex separation of G.

INPUT:

- `G` – a Graph or a DiGraph

OUTPUT:

A lower bound on the vertex separation of D (see the module's documentation).

Note

This method runs in exponential time but has no memory constraint.

EXAMPLES:

On a circuit:

```
sage: from sage.graphs.graph_decompositions.vertex_separation import lower_bound
sage: g = digraphs.Circuit(6)
sage: lower_bound(g)
1
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_decompositions.vertex_separation import lower_bound
>>> g = digraphs.Circuit(Integer(6))
>>> lower_bound(g)
1
```

sage.graphs.graph_decompositions.vertex_separation.**path_decomposition**(*G*, *algorithm='BAB'*, *cut_off=None*, *upper_bound=None*, *verbose=False*, *max_prefix_length=20*, *max_prefix_number=1000000*)

Return the pathwidth of the given graph and the ordering of the vertices resulting in a corresponding path decomposition.

INPUT:

- `G` – a Graph
- `algorithm` – string (default: `'BAB'`); algorithm to use among:
  - `'BAB'` – use a branch-and-bound algorithm. This algorithm has no size restriction but could take a very long time on large graphs. It can also be used to test is the input (di)graph has vertex separation at most `upper_bound` or to return the first found solution with vertex separation less or equal to a `cut_off` value.
  - `exponential` – use an exponential time and space algorithm. This algorithm only works of graphs on less than 32 vertices
  - `MILP` – use a mixed integer linear programming formulation. This algorithm has no size restriction but could take a very long time
- `upper_bound` – integer (default: `None`); parameter used by the `'BAB'` algorithm. If specified, the algorithm searches for a solution with `width < upper_bound`. It helps cutting branches. However, if the given upper bound is too low, the algorithm may not be able to find a solution.
- `cut_off` – integer (default: `None`); parameter used by the `'BAB'` algorithm. This bound allows us to stop the search as soon as a solution with width at most `cut_off` is found, if any. If this bound cannot be reached, the best solution found is returned, unless a too low `upper_bound` is given.
- `verbose` – boolean (default: `False`); whether to display information on the computations
- `max_prefix_length` – integer (default: 20); limits the length of the stored prefixes to prevent storing too many prefixes. This parameter is used only when `algorithm=="BAB"`.
- `max_prefix_number` – integer (default: 10**6); upper bound on the number of stored prefixes used to prevent using too much memory. This parameter is used only when `algorithm=="BAB"`.

OUTPUT:

A pair `(cost, ordering)` representing the optimal ordering of the vertices and its cost.

See also

- *`Graph.treewidth()`* – computes the treewidth of a graph

EXAMPLES:

The pathwidth of a cycle is equal to 2:

```
sage: from sage.graphs.graph_decompositions.vertex_separation import path_
↪decomposition
sage: g = graphs.CycleGraph(6)
sage: pw, L = path_decomposition(g, algorithm='BAB'); pw
2
sage: pw, L = path_decomposition(g, algorithm='exponential'); pw
2
sage: pw, L = path_decomposition(g, algorithm='MILP'); pw                    #␣
↪needs sage.numerical.mip
2
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_decompositions.vertex_separation import path_
↪decomposition
>>> g = graphs.CycleGraph(Integer(6))
>>> pw, L = path_decomposition(g, algorithm='BAB'); pw
2
>>> pw, L = path_decomposition(g, algorithm='exponential'); pw
2
>>> pw, L = path_decomposition(g, algorithm='MILP'); pw                      #␣
↪needs sage.numerical.mip
2
```

sage.graphs.graph_decompositions.vertex_separation.**pathwidth**(*G*, *k=None*, *certificate=False*, *algorithm='BAB'*, *verbose=False*, *max_prefix_length=20*, *max_prefix_number=1000000*, *immutable=None*, *solver=None*)

Compute the pathwidth of `G` (and provides a decomposition).

INPUT:

- `G` – a graph
- `k` – integer (default: `None`); the width to be considered. When `k` is an integer, the method checks that the graph has pathwidth $\leq k$. If `k` is `None` (default), the method computes the optimal pathwidth.
- `certificate` – boolean (default: `False`); whether to return the path-decomposition itself
- `algorithm` – string (default: `'BAB'`); algorithm to use among:
  - `'BAB'` – use a branch-and-bound algorithm. This algorithm has no size restriction but could take a very long time on large graphs. It can also be used to test is the input graph has pathwidth $\leq k$, in which cas it will return the first found solution with width $\leq k$ is `certificate==True`.
  - `exponential` – use an exponential time and space algorithm. This algorithm only works of graphs on less than 32 vertices
  - `MILP` – use a mixed integer linear programming formulation. This algorithm has no size restriction but could take a very long time
- `verbose` – boolean (default: `False`); whether to display information on the computations
- `max_prefix_length` – integer (default: 20); limits the length of the stored prefixes to prevent storing too many prefixes. This parameter is used only when `algorithm=="BAB"`.

- `max_prefix_number` – integer (default: 10**6); upper bound on the number of stored prefixes used to prevent using too much memory. This parameter is used only when `algorithm=="BAB"`.
- `immutable` – boolean (default: `None`); whether to create a mutable/immutable graph. `immutable=None` (default) means that the graph and its path decomposition will behave the same way.
- `solver` – string (default: `None`); specifies a Mixed Integer Linear Programming (MILP) solver to be used. If set to `None`, the default one is used. For more information on MILP solvers and which default solver is used, see the method `solve` of the class `MixedIntegerLinearProgram`.

OUTPUT:

Return the pathwidth of `self`. When `k` is specified, it returns `False` when no path-decomposition of width $\leq k$ exists or `True` otherwise. When `certificate=True`, the path-decomposition is also returned.

See also

- *`Graph.treewidth()`* – computes the treewidth of a graph
- *`vertex_separation()`* – computes the vertex separation of a (di)graph

EXAMPLES:

The pathwidth of a cycle is equal to 2:

```
sage: g = graphs.CycleGraph(6)
sage: g.pathwidth()
2
sage: pw, decomp = g.pathwidth(certificate=True)
sage: sorted(decomp, key=str)
[{0, 1, 5}, {1, 2, 5}, {2, 3, 4}, {2, 4, 5}]
```

```
>>> from sage.all import *
>>> g = graphs.CycleGraph(Integer(6))
>>> g.pathwidth()
2
>>> pw, decomp = g.pathwidth(certificate=True)
>>> sorted(decomp, key=str)
[{0, 1, 5}, {1, 2, 5}, {2, 3, 4}, {2, 4, 5}]
```

The pathwidth of a Petersen graph is 5:

```
sage: g = graphs.PetersenGraph()
sage: g.pathwidth()
5
sage: g.pathwidth(k=2)
False
sage: g.pathwidth(k=6)
True
sage: g.pathwidth(k=6, certificate=True)
(True, Graph on 5 vertices)
```

```
>>> from sage.all import *
>>> g = graphs.PetersenGraph()
```

(continues on next page)

(continued from previous page)

```
>>> g.pathwidth()
5
>>> g.pathwidth(k=Integer(2))
False
>>> g.pathwidth(k=Integer(6))
True
>>> g.pathwidth(k=Integer(6), certificate=True)
(True, Graph on 5 vertices)
```

sage.graphs.graph_decompositions.vertex_separation.**vertex_separation**(*G*, *algorithm='BAB'*, *cut_off=None*, *upper_bound=None*, *verbose=False*, *max_prefix_length=20*, *max_prefix_number=1000000*, *solver=None*, *integrality_tolerance=0.001*)

Return an optimal ordering of the vertices and its cost for vertex-separation.

INPUT:

- `G` – a Graph or a DiGraph
- `algorithm` – string (default: `'BAB'`); algorithm to use among:
  - `'BAB'` – use a branch-and-bound algorithm. This algorithm has no size restriction but could take a very long time on large graphs. It can also be used to test is the input (di)graph has vertex separation at most `upper_bound` or to return the first found solution with vertex separation less or equal to a `cut_off` value.
  - `exponential` – use an exponential time and space algorithm. This algorithm only works of graphs on less than 32 vertices
  - `MILP` – use a mixed integer linear programming formulation. This algorithm has no size restriction but could take a very long time
- `upper_bound` – integer (default: `None`); parameter used by the `'BAB'` algorithm. If specified, the algorithm searches for a solution with `width < upper_bound`. It helps cutting branches. However, if the given upper bound is too low, the algorithm may not be able to find a solution.
- `cut_off` – integer (default: `None`); parameter used by the `'BAB'` algorithm. This bound allows us to stop the search as soon as a solution with width at most `cut_off` is found, if any. If this bound cannot be reached, the best solution found is returned, unless a too low `upper_bound` is given.
- `verbose` – boolean (default: `False`); whether to display information on the computations
- `max_prefix_length` – integer (default: 20); limits the length of the stored prefixes to prevent storing too many prefixes. This parameter is used only when `algorithm=="BAB"`.
- `max_prefix_number` – integer (default: 10**6); upper bound on the number of stored prefixes used to prevent using too much memory. This parameter is used only when `algorithm=="BAB"`.
- `solver` – string (default: `None`); specifies a Mixed Integer Linear Programming (MILP) solver to be used. If set to `None`, the default one is used. For more information on MILP solvers and which default solver is used, see the method `solve` of the class `MixedIntegerLinearProgram`.
- `integrality_tolerance` – float; parameter for use with MILP solvers over an inexact base ring; see `MixedIntegerLinearProgram.get_values()`.

OUTPUT:

A pair `(cost, ordering)` representing the optimal ordering of the vertices and its cost.

EXAMPLES:

Comparison of methods:

```
sage: from sage.graphs.graph_decompositions.vertex_separation import vertex_
↪separation

sage: G = digraphs.DeBruijn(2,3)
sage: vs,L = vertex_separation(G, algorithm='BAB'); vs
2
sage: vs,L = vertex_separation(G, algorithm='exponential'); vs
2
sage: vs,L = vertex_separation(G, algorithm='MILP'); vs                    #␣
↪needs sage.numerical.mip
2

sage: G = graphs.Grid2dGraph(3,3)
sage: vs,L = vertex_separation(G, algorithm='BAB'); vs
3
sage: vs,L = vertex_separation(G, algorithm='exponential'); vs
3
sage: vs,L = vertex_separation(G, algorithm='MILP'); vs                    #␣
↪needs sage.numerical.mip
3
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_decompositions.vertex_separation import vertex_
↪separation

>>> G = digraphs.DeBruijn(Integer(2),Integer(3))
>>> vs,L = vertex_separation(G, algorithm='BAB'); vs
2
>>> vs,L = vertex_separation(G, algorithm='exponential'); vs
2
>>> vs,L = vertex_separation(G, algorithm='MILP'); vs                      #␣
↪needs sage.numerical.mip
2

>>> G = graphs.Grid2dGraph(Integer(3),Integer(3))
>>> vs,L = vertex_separation(G, algorithm='BAB'); vs
3
>>> vs,L = vertex_separation(G, algorithm='exponential'); vs
3
>>> vs,L = vertex_separation(G, algorithm='MILP'); vs                      #␣
↪needs sage.numerical.mip
3
```

Digraphs with multiple strongly connected components:

```
sage: from sage.graphs.graph_decompositions.vertex_separation import vertex_
↪separation
```

(continues on next page)

(continued from previous page)

```
sage: D = digraphs.Path(8)
sage: print(vertex_separation(D))
(0, [7, 6, 5, 4, 3, 2, 1, 0])
sage: D = digraphs.RandomDirectedAcyclicGraph(10, .5)
sage: vs,L = vertex_separation(D); vs
0
sage: K4 = DiGraph( graphs.CompleteGraph(4) )
sage: D = K4+K4
sage: D.add_edge(0, 4)
sage: print(vertex_separation(D))
(3, [4, 5, 6, 7, 0, 1, 2, 3])
sage: D = K4+K4+K4
sage: D.add_edge(0, 4)
sage: D.add_edge(0, 8)
sage: print(vertex_separation(D))
(3, [10, 11, 8, 9, 4, 5, 6, 7, 0, 1, 2, 3])
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_decompositions.vertex_separation import vertex_
↪separation
>>> D = digraphs.Path(Integer(8))
>>> print(vertex_separation(D))
(0, [7, 6, 5, 4, 3, 2, 1, 0])
>>> D = digraphs.RandomDirectedAcyclicGraph(Integer(10), RealNumber('.5'))
>>> vs,L = vertex_separation(D); vs
0
>>> K4 = DiGraph( graphs.CompleteGraph(Integer(4)) )
>>> D = K4+K4
>>> D.add_edge(Integer(0), Integer(4))
>>> print(vertex_separation(D))
(3, [4, 5, 6, 7, 0, 1, 2, 3])
>>> D = K4+K4+K4
>>> D.add_edge(Integer(0), Integer(4))
>>> D.add_edge(Integer(0), Integer(8))
>>> print(vertex_separation(D))
(3, [10, 11, 8, 9, 4, 5, 6, 7, 0, 1, 2, 3])
```

Using a specific MILP solver:

```
sage: from sage.graphs.graph_decompositions.vertex_separation import vertex_
↪separation
sage: G = graphs.PetersenGraph()
sage: vs, L = vertex_separation(G, algorithm='MILP', solver='SCIP'); vs     #␣
↪optional - pyscipopt, needs sage.numerical.mip
5
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_decompositions.vertex_separation import vertex_
↪separation
>>> G = graphs.PetersenGraph()
>>> vs, L = vertex_separation(G, algorithm='MILP', solver='SCIP'); vs     #␣
↪optional - pyscipopt, needs sage.numerical.mip
```

(continues on next page)

(continued from previous page)

```
5
```

sage.graphs.graph_decompositions.vertex_separation.**vertex_separation_BAB**(*G*, *cut_off=None*, *upper_bound=None*, *max_prefix_length=20*, *max_prefix_number=1000000*, *verbose=False*)

Branch and Bound algorithm for the vertex separation.

This method implements the branch and bound algorithm for the vertex separation of directed graphs and the pathwidth of undirected graphs proposed in [CMN2014]. The implementation is valid for both Graph and DiGraph. See the documentation of the *vertex_separation* module.

INPUT:

- `G` – a Graph or a DiGraph
- `cut_off` – integer (default: `None`); bound to consider in the branch and bound algorithm. This allows us to stop the search as soon as a solution with width at most `cut_off` is found, if any. If this bound cannot be reached, the best solution found is returned, unless a too low `upper_bound` is given.
- `upper_bound` – integer (default: `None`); if specified, the algorithm searches for a solution with `width < upper_bound`. It helps cutting branches. However, if the given upper bound is too low, the algorithm may not be able to find a solution.
- `max_prefix_length` – integer (default: 20); limits the length of the stored prefixes to prevent storing too many prefixes
- `max_prefix_number` – integer (default: 10**6); upper bound on the number of stored prefixes used to prevent using too much memory
- `verbose` – boolean (default: `False`); display some information when set to `True`

OUTPUT: `width` – the computed vertex separation

- `seq` – an ordering of the vertices of width `width`

EXAMPLES:

The algorithm is valid for the vertex separation:

```
sage: from sage.graphs.graph_decompositions import vertex_separation as VS
sage: D = digraphs.RandomDirectedGNP(15, .2)
sage: vb, seqb = VS.vertex_separation_BAB(D)
sage: vd, seqd = VS.vertex_separation_exp(D)
sage: vb == vd
True
sage: vb == VS.width_of_path_decomposition(D, seqb)
True
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_decompositions import vertex_separation as VS
>>> D = digraphs.RandomDirectedGNP(Integer(15), RealNumber('.2'))
>>> vb, seqb = VS.vertex_separation_BAB(D)
```

(continues on next page)

(continued from previous page)

```
>>> vd, seqd = VS.vertex_separation_exp(D)
>>> vb == vd
True
>>> vb == VS.width_of_path_decomposition(D, seqb)
True
```

The vertex separation of a $N \times N$ grid is N:

```
sage: from sage.graphs.graph_decompositions import vertex_separation as VS
sage: G = graphs.Grid2dGraph(4,4)
sage: vs, seq = VS.vertex_separation_BAB(G); vs
4
sage: vs == VS.width_of_path_decomposition(G, seq)
True
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_decompositions import vertex_separation as VS
>>> G = graphs.Grid2dGraph(Integer(4),Integer(4))
>>> vs, seq = VS.vertex_separation_BAB(G); vs
4
>>> vs == VS.width_of_path_decomposition(G, seq)
True
```

The vertex separation of a $N \times M$ grid with $N < M$ is N:

```
sage: from sage.graphs.graph_decompositions import vertex_separation as VS
sage: G = graphs.Grid2dGraph(3,5)
sage: vs, seq = VS.vertex_separation_BAB(G); vs
3
sage: vs == VS.width_of_path_decomposition(G, seq)
True
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_decompositions import vertex_separation as VS
>>> G = graphs.Grid2dGraph(Integer(3),Integer(5))
>>> vs, seq = VS.vertex_separation_BAB(G); vs
3
>>> vs == VS.width_of_path_decomposition(G, seq)
True
```

The vertex separation of circuit of order $N \geq 2$ is 1:

```
sage: from sage.graphs.graph_decompositions import vertex_separation as VS
sage: D = digraphs.Circuit(10)
sage: vs, seq = VS.vertex_separation_BAB(D); vs
1
sage: vs == VS.width_of_path_decomposition(D, seq)
True
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_decompositions import vertex_separation as VS
>>> D = digraphs.Circuit(Integer(10))
```

(continues on next page)

(continued from previous page)

```
>>> vs, seq = VS.vertex_separation_BAB(D); vs
1
>>> vs == VS.width_of_path_decomposition(D, seq)
True
```

The vertex separation of cycle of order $N \geq 3$ is 2:

```
sage: from sage.graphs.graph_decompositions import vertex_separation as VS
sage: G = graphs.CycleGraph(10)
sage: vs, seq = VS.vertex_separation_BAB(G); vs
2
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_decompositions import vertex_separation as VS
>>> G = graphs.CycleGraph(Integer(10))
>>> vs, seq = VS.vertex_separation_BAB(G); vs
2
```

The vertex separation of `MycielskiGraph(5)` is 10:

```
sage: from sage.graphs.graph_decompositions import vertex_separation as VS
sage: G = graphs.MycielskiGraph(5)
sage: vs, seq = VS.vertex_separation_BAB(G); vs
10
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_decompositions import vertex_separation as VS
>>> G = graphs.MycielskiGraph(Integer(5))
>>> vs, seq = VS.vertex_separation_BAB(G); vs
10
```

Searching for any solution with width less or equal to `cut_off`:

```
sage: from sage.graphs.graph_decompositions import vertex_separation as VS
sage: G = graphs.MycielskiGraph(5)
sage: VS.vertex_separation_BAB(G, cut_off=11)[0] <= 11
True
sage: VS.vertex_separation_BAB(G, cut_off=10)[0] <= 10
True
sage: VS.vertex_separation_BAB(G, cut_off=9)[0] <= 9
False
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_decompositions import vertex_separation as VS
>>> G = graphs.MycielskiGraph(Integer(5))
>>> VS.vertex_separation_BAB(G, cut_off=Integer(11))[Integer(0)] <= Integer(11)
True
>>> VS.vertex_separation_BAB(G, cut_off=Integer(10))[Integer(0)] <= Integer(10)
True
>>> VS.vertex_separation_BAB(G, cut_off=Integer(9))[Integer(0)] <= Integer(9)
False
```

Testing for the existence of a solution with width strictly less than `upper_bound`:

```
sage: from sage.graphs.graph_decompositions import vertex_separation as VS
sage: G = graphs.MycielskiGraph(5)
sage: vs, seq = VS.vertex_separation_BAB(G, upper_bound=11); vs
10
sage: vs, seq = VS.vertex_separation_BAB(G, upper_bound=10); vs
-1
sage: vs, seq = VS.vertex_separation_BAB(G, cut_off=11, upper_bound=10); vs
-1
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_decompositions import vertex_separation as VS
>>> G = graphs.MycielskiGraph(Integer(5))
>>> vs, seq = VS.vertex_separation_BAB(G, upper_bound=Integer(11)); vs
10
>>> vs, seq = VS.vertex_separation_BAB(G, upper_bound=Integer(10)); vs
-1
>>> vs, seq = VS.vertex_separation_BAB(G, cut_off=Integer(11), upper_
↪bound=Integer(10)); vs
-1
```

Changing the parameters of the prefix storage:

```
sage: from sage.graphs.graph_decompositions import vertex_separation as VS
sage: G = graphs.MycielskiGraph(5)
sage: vs, seq = VS.vertex_separation_BAB(G, max_prefix_length=0); vs
10
sage: vs, seq = VS.vertex_separation_BAB(G, max_prefix_number=5); vs
10
sage: vs, seq = VS.vertex_separation_BAB(G, max_prefix_number=0); vs
10
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_decompositions import vertex_separation as VS
>>> G = graphs.MycielskiGraph(Integer(5))
>>> vs, seq = VS.vertex_separation_BAB(G, max_prefix_length=Integer(0)); vs
10
>>> vs, seq = VS.vertex_separation_BAB(G, max_prefix_number=Integer(5)); vs
10
>>> vs, seq = VS.vertex_separation_BAB(G, max_prefix_number=Integer(0)); vs
10
```

sage.graphs.graph_decompositions.vertex_separation.**vertex_separation_MILP**(*G*, *integrality=False*, *solver=None*, *verbose=0*, *integrality_tolerance=0.001*)

Compute the vertex separation of G and the optimal ordering of its vertices using an MILP formulation.

This function uses a mixed integer linear program (MILP) for determining an optimal layout for the vertex separation of G. This MILP is an improved version of the formulation proposed in [SP2010]. See the *module's documentation* for more details on this MILP formulation.

INPUT:

- `G` – a Graph or a DiGraph
- `integrality` – boolean (default: `False`); specify if variables x_v^t and u_v^t must be integral or if they can be relaxed. This has no impact on the validity of the solution, but it is sometimes faster to solve the problem using binary variables only.
- `solver` – string (default: `None`); specifies a Mixed Integer Linear Programming (MILP) solver to be used. If set to `None`, the default one is used. For more information on MILP solvers and which default solver is used, see the method `solve` of the class `MixedIntegerLinearProgram`.
- `verbose` – integer (default: 0); sets the level of verbosity. Set to 0 by default, which means quiet.
- `integrality_tolerance` – float; parameter for use with MILP solvers over an inexact base ring; see `MixedIntegerLinearProgram.get_values()`.

OUTPUT:

A pair `(cost, ordering)` representing the optimal ordering of the vertices and its cost.

EXAMPLES:

Vertex separation of a De Bruijn digraph:

```
sage: from sage.graphs.graph_decompositions import vertex_separation
sage: G = digraphs.DeBruijn(2,3)
sage: vs, L = vertex_separation.vertex_separation_MILP(G); vs                    #␣
↪needs sage.numerical.mip
2
sage: vs == vertex_separation.width_of_path_decomposition(G, L)                  #␣
↪needs sage.numerical.mip
True
sage: vse, Le = vertex_separation.vertex_separation(G); vse
2
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_decompositions import vertex_separation
>>> G = digraphs.DeBruijn(Integer(2),Integer(3))
>>> vs, L = vertex_separation.vertex_separation_MILP(G); vs                      #␣
↪needs sage.numerical.mip
2
>>> vs == vertex_separation.width_of_path_decomposition(G, L)                    #␣
↪needs sage.numerical.mip
True
>>> vse, Le = vertex_separation.vertex_separation(G); vse
2
```

The vertex separation of a circuit is 1:

```
sage: from sage.graphs.graph_decompositions import vertex_separation
sage: G = digraphs.Circuit(6)
sage: vs, L = vertex_separation.vertex_separation_MILP(G); vs                    #␣
↪needs sage.numerical.mip
1
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_decompositions import vertex_separation
>>> G = digraphs.Circuit(Integer(6))
```

(continues on next page)

(continued from previous page)

```
>>> vs, L = vertex_separation.vertex_separation_MILP(G); vs                    #␣
↪needs sage.numerical.mip
1
```

sage.graphs.graph_decompositions.vertex_separation.**vertex_separation_exp**(*G*, *verbose=False*)

Return an optimal ordering of the vertices and its cost for vertex-separation.

INPUT:

- `G` – a Graph or a DiGraph
- `verbose` – boolean (default: `False`); whether to display information on the computations

OUTPUT:

A pair `(cost, ordering)` representing the optimal ordering of the vertices and its cost.

Note

Because of its current implementation, this algorithm only works on graphs on less than 32 vertices. This can be changed to 54 if necessary, but 32 vertices already require 4GB of memory.

EXAMPLES:

The vertex separation of a circuit is equal to 1:

```
sage: from sage.graphs.graph_decompositions.vertex_separation import vertex_
↪separation_exp
sage: g = digraphs.Circuit(6)
sage: vertex_separation_exp(g)
(1, [0, 1, 2, 3, 4, 5])
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_decompositions.vertex_separation import vertex_
↪separation_exp
>>> g = digraphs.Circuit(Integer(6))
>>> vertex_separation_exp(g)
(1, [0, 1, 2, 3, 4, 5])
```

sage.graphs.graph_decompositions.vertex_separation.**width_of_path_decomposition**(*G*, *L*)

Return the width of the path decomposition induced by the linear ordering L of the vertices of G.

If G is an instance of *Graph*, this function returns the width $pw_L(G)$ of the path decomposition induced by the linear ordering L of the vertices of G. If G is a *DiGraph*, it returns instead the width $vs_L(G)$ of the directed path decomposition induced by the linear ordering L of the vertices of G, where

$$vs_L(G) = \max_{0 \leq i < |V|-1} |N^+(L[:i]) \setminus L[:i]|$$
$$pw_L(G) = \max_{0 \leq i < |V|-1} |N(L[:i]) \setminus L[:i]|$$

INPUT:

- `G` – a Graph or a DiGraph
- `L` – a linear ordering of the vertices of `G`

EXAMPLES:

Path decomposition of a cycle:

```
sage: from sage.graphs.graph_decompositions import vertex_separation
sage: G = graphs.CycleGraph(6)
sage: L = G.vertices(sort=True)
sage: vertex_separation.width_of_path_decomposition(G, L)
2
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_decompositions import vertex_separation
>>> G = graphs.CycleGraph(Integer(6))
>>> L = G.vertices(sort=True)
>>> vertex_separation.width_of_path_decomposition(G, L)
2
```

Directed path decomposition of a circuit:

```
sage: from sage.graphs.graph_decompositions import vertex_separation
sage: G = digraphs.Circuit(6)
sage: L = G.vertices(sort=True)
sage: vertex_separation.width_of_path_decomposition(G, L)
1
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_decompositions import vertex_separation
>>> G = digraphs.Circuit(Integer(6))
>>> L = G.vertices(sort=True)
>>> vertex_separation.width_of_path_decomposition(G, L)
1
```

5.25 Rank Decompositions of graphs

This module wraps C code from Philipp Klaus Krause computing an optimal rank-decomposition.

Definitions :

Given a graph G and a subset $S \subseteq V(G)$ of vertices, the *rank-width* of S in G, denoted $rw_G(S)$, is equal to the rank in $GF(2)$ of the $|S| \times (|V| - |S|)$ matrix of the adjacencies between the vertices of S and $V \backslash S$. By definition, $rw_G(S)$ is equal to $rw_G(\overline{S})$ where $\overline{S}$ is the complement of S in $V(G)$.

A *rank-decomposition* of G is a tree whose n leaves are the elements of $V(G)$, and whose internal nodes have degree 3. In a tree, any edge naturally corresponds to a bipartition of the vertex set : indeed, the removal of any edge splits the tree into two connected components, thus splitting the set of leaves (i.e. vertices of G) into two sets. Hence we can define for any edge $e \in E(G)$ a width equal to the value $rw_G(S)$ or $rw_G(\overline{S})$, where $S, \overline{S}$ is the bipartition obtained from e. The *rank-width* associated to the whole decomposition is then set to the maximum of the width of all the edges it contains.

A *rank-decomposition* is said to be optimal for G if it is the decomposition achieving the minimal *rank-width*.

RW – The original source code :

RW is a program that calculates rank-width and rank-decompositions. It is based on ideas from:

- “Computing rank-width exactly” by Sang-il Oum [Oum2009]
- “Sopra una formula numerica” by Ernesto Pascal

- "Generation of a Vector from the Lexicographical Index" by B.P. Buckles and M. Lybanon [BL1977]
- "Fast additions on masked integers" by Michael D. Adams and David S. Wise [AW2006]

OUTPUT:

The rank decomposition is returned as a tree whose vertices are subsets of $V(G)$. Its leaves, corresponding to the vertices of G are sets of 1 elements, i.e. singletons.

```
sage: # needs rankwidth
sage: g = graphs.PetersenGraph()
sage: rw, tree = g.rank_decomposition()
sage: all(len(v)==1 for v in tree if tree.degree(v) == 1)
True
```

```
>>> from sage.all import *
>>> # needs rankwidth
>>> g = graphs.PetersenGraph()
>>> rw, tree = g.rank_decomposition()
>>> all(len(v)==Integer(1) for v in tree if tree.degree(v) == Integer(1))
True
```

The internal nodes are sets of the decomposition. This way, it is easy to deduce the bipartition associated to an edge from the tree. Indeed, two adjacent vertices of the tree are comparable sets : they yield the bipartition obtained from the smaller of the two and its complement.

```
sage: # needs rankwidth
sage: g = graphs.PetersenGraph()
sage: rw, tree = g.rank_decomposition()
sage: u = Set([8, 9, 3, 7])
sage: v = Set([8, 9])
sage: tree.has_edge(u,v)
True
sage: m = min(u,v)
sage: bipartition = (m, Set(g.vertices(sort=False)) - m)
sage: bipartition
({8, 9}, {0, 1, 2, 3, 4, 5, 6, 7})
```

```
>>> from sage.all import *
>>> # needs rankwidth
>>> g = graphs.PetersenGraph()
>>> rw, tree = g.rank_decomposition()
>>> u = Set([Integer(8), Integer(9), Integer(3), Integer(7)])
>>> v = Set([Integer(8), Integer(9)])
>>> tree.has_edge(u,v)
True
>>> m = min(u,v)
>>> bipartition = (m, Set(g.vertices(sort=False)) - m)
>>> bipartition
({8, 9}, {0, 1, 2, 3, 4, 5, 6, 7})
```

Warning

- The current implementation cannot handle graphs of ≥ 32 vertices.

- A bug that has been reported upstream make the code crash immediately on instances of size 30. If you experience this kind of bug please report it to us, what we need is some information on the hardware you run to know where it comes from !

EXAMPLES:

```
sage: # needs rankwidth
sage: g = graphs.PetersenGraph()
sage: g.rank_decomposition()
(3, Graph on 19 vertices)
```

```
>>> from sage.all import *
>>> # needs rankwidth
>>> g = graphs.PetersenGraph()
>>> g.rank_decomposition()
(3, Graph on 19 vertices)
```

AUTHORS:

- Philipp Klaus Krause : Implementation of the C algorithm
- Nathann Cohen : Interface with Sage and documentation

5.25.1 Methods

sage.graphs.graph_decompositions.rankwidth.**mkgraph**(*num_vertices*)

Return the graph corresponding to the current rank-decomposition.

(This function is for internal use)

EXAMPLES:

```
sage: # needs rankwidth
sage: from sage.graphs.graph_decompositions.rankwidth import rank_decomposition
sage: g = graphs.PetersenGraph()
sage: rank_decomposition(g)
(3, Graph on 19 vertices)
```

```
>>> from sage.all import *
>>> # needs rankwidth
>>> from sage.graphs.graph_decompositions.rankwidth import rank_decomposition
>>> g = graphs.PetersenGraph()
>>> rank_decomposition(g)
(3, Graph on 19 vertices)
```

sage.graphs.graph_decompositions.rankwidth.**rank_decomposition**(*G*, *verbose=False*, *immutable=None*)

Compute an optimal rank-decomposition of the given graph.

This function is available as a method of the *Graph* class. See *rank_decomposition*.

INPUT:

- `verbose` – boolean (default: `False`); whether to display progress information while computing the decomposition

- `immutable` – boolean (default: `None`); whether to create a mutable/immutable graph. `immutable=None` (default) means that the graph and its decomposition tree will behave the same way.

OUTPUT:

A pair `(rankwidth, decomposition_tree)`, where `rankwidth` is a numerical value and `decomposition_tree` is a ternary tree describing the decomposition (cf. the module's documentation).

EXAMPLES:

```
sage: # needs rankwidth
sage: from sage.graphs.graph_decompositions.rankwidth import rank_decomposition
sage: g = graphs.PetersenGraph()
sage: rank_decomposition(g)
(3, Graph on 19 vertices)
```

```
>>> from sage.all import *
>>> # needs rankwidth
>>> from sage.graphs.graph_decompositions.rankwidth import rank_decomposition
>>> g = graphs.PetersenGraph()
>>> rank_decomposition(g)
(3, Graph on 19 vertices)
```

On more than 32 vertices:

```
sage: # needs rankwidth
sage: g = graphs.RandomGNP(40, .5)
sage: rank_decomposition(g)
Traceback (most recent call last):
...
RuntimeError: the rank decomposition cannot be computed on graphs of >= 32␣
↪vertices
```

```
>>> from sage.all import *
>>> # needs rankwidth
>>> g = graphs.RandomGNP(Integer(40), RealNumber('.5'))
>>> rank_decomposition(g)
Traceback (most recent call last):
...
RuntimeError: the rank decomposition cannot be computed on graphs of >= 32␣
↪vertices
```

The empty graph:

```
sage: # needs rankwidth
sage: g = Graph()
sage: rank_decomposition(g)
(0, Graph on 0 vertices)
```

```
>>> from sage.all import *
>>> # needs rankwidth
>>> g = Graph()
>>> rank_decomposition(g)
(0, Graph on 0 vertices)
```

Check the behavior of parameter `immutable`:

```
sage: # needs rankwidth
sage: G = Graph()
sage: rank_decomposition(G)[1].is_immutable()
False
sage: rank_decomposition(G, immutable=True)[1].is_immutable()
True
sage: G = Graph(immutable=True)
sage: rank_decomposition(G)[1].is_immutable()
True
sage: rank_decomposition(G, immutable=False)[1].is_immutable()
False
```

```
>>> from sage.all import *
>>> # needs rankwidth
>>> G = Graph()
>>> rank_decomposition(G)[Integer(1)].is_immutable()
False
>>> rank_decomposition(G, immutable=True)[Integer(1)].is_immutable()
True
>>> G = Graph(immutable=True)
>>> rank_decomposition(G)[Integer(1)].is_immutable()
True
>>> rank_decomposition(G, immutable=False)[Integer(1)].is_immutable()
False
```

5.26 Bandwidth of undirected graphs

5.26.1 Definition

The bandwidth $bw(M)$ of a matrix M is the smallest integer k such that all nonzero entries of M are at distance k from the diagonal. The bandwidth $bw(G)$ of an undirected graph G is the minimum bandwidth of the adjacency matrix of G, over all possible relabellings of its vertices.

Path spanner: alternatively, the bandwidth measures how tightly a path represents the distance of a graph G. Indeed, if the vertices of G can be ordered as $v_1, ..., v_n$ in such a way that $k \times d_G(v_i, v_j) \geq |i - j|$ then $bw(G) \leq k$.

Proof: for all $v_i \sim v_j$ (i.e. $d_G(v_i, v_j) = 1$), the constraint ensures that $k \geq |i - j|$, meaning that adjacent vertices are at distance at most k in the path ordering. That alone is sufficient to ensure that $bw(G) \leq k$.

As a byproduct, we obtain that $k \times d_G(v_i, v_j) \geq |i - j|$ in general: let $v_{s_0}, ..., v_{s_i}$ be the vertices of a shortest (v_i, v_j)-path. We have:

$$\begin{aligned} k \times d_G(v_i, v_j) &= k \times d_G(v_i, v_{s_0}) + k \times d_G(v_{s_0}, v_{s_1}) + \cdots + k \times d_G(v_{s_{i-1}}, v_{s_i}) + k \times d_G(v_{s_i}, v_j) \\ &\geq |v_i - v_{s_0}| + |v_{s_0} - v_{s_1}| + \cdots + |v_{s_{i-1}} - v_{s_i}| + |v_{s_i} - v_j| \\ &\geq |v_i - v_j| \end{aligned}$$

5.26.2 Satisfiability of a partial assignment

Let us suppose that the first i vertices $v_1, ..., v_i$ of G have already been assigned positions $p_1, ..., p_i$ in an ordering of $V(G)$ of bandwidth $\leq k$. Where can v_{i+1} appear ?

Because of the previous definition, p_{i+1} must be at distance at most $k \times d_G(v_1, v_{i+1})$ from p_1, and in general at distance at most $k \times d_G(v_j, v_{i+1})$ from p_j. Each range is an interval of $\{1, ..., n\} \backslash \{p_1, ..., p_i\}$, and because the intersection of

two intervals is again an interval we deduce that in order to satisfy all these constraints simultaneously p_j must belong to an interval defined from this partial assignment.

Applying this rule to all non-assigned vertices, we deduce that each of them must be assigned to a given interval of $\{1, ..., n\}$. Note that this can also be extended to the already assigned vertices, by saying that v_j with $j < i$ must be assigned within the interval $[p_j, p_j]$.

This problem is not always satisfiable, e.g. 5 vertices cannot all be assigned to the elements of $[10, 13]$. This is a matching problem which, because all admissible sets are intervals, can be solved quickly.

5.26.3 Solving the matching problem

Let n points $v_1, ..., v_n$ be given, along with two functions $m, M : [n] \mapsto [n]$. Is there an ordering $p_1, ..., p_n$ of them such that $m(v_i) \leq p_i \leq M(v_i)$? This is equivalent to Hall's bipartite matching theorem, and can in this specific case be solved by the following algorithm:

- Consider all vertices v sorted increasingly according to $M(v)$
- For each of them, assign to v the smallest position in $[m(v), M(v)]$ which has not been assigned yet. If there is none, the assignment problem is not satisfiable.

Note that the latest operation can be performed with very few bitset operations (provided that $n < 64$).

5.26.4 The algorithm

This section contains totally subjective choices, that may be changed in the hope to get better performances.

- Try to find a satisfiable ordering by filling positions, one after the other (and not by trying to find each vertex' position)
- Fill the positions in this order: $0, n-1, 1, n-2, 3, n-3, ...$

Note

There is some symmetry to break as the reverse of a satisfiable ordering is also a satisfiable ordering.

5.26.5 This module contains the following methods

bandwidth()	Compute the bandwidth of an undirected graph
bandwidth_heuristics()	Use Boost heuristics to approximate the bandwidth of the input graph

5.26.6 Functions

sage.graphs.graph_decompositions.bandwidth.**bandwidth**(*G*, *k=None*)

Compute the bandwidth of an undirected graph.

For a definition of the bandwidth of a graph, see the documentation of the *bandwidth* module.

INPUT:

- `G` – a graph
- `k` – integer (default: `None`); set to an integer value to test whether $bw(G) \leq k$, or to `None` (default) to compute $bw(G)$

OUTPUT:

When k is an integer value, the function returns either `False` or an ordering of cost $\leq k$.

When k is equal to `None`, the function returns a pair `(bw, ordering)`.

See also

sage.graphs.generic_graph.GenericGraph.adjacency_matrix() – return the adjacency matrix from an ordering of the vertices.

EXAMPLES:

```
sage: from sage.graphs.graph_decompositions.bandwidth import bandwidth
sage: G = graphs.PetersenGraph()
sage: bandwidth(G,3)
False
sage: bandwidth(G)
(5, [0, 4, 5, 8, 1, 9, 3, 7, 6, 2])
sage: G.adjacency_matrix(vertices=[0, 4, 5, 8, 1, 9, 3, 7, 6, 2])                #␣
↪needs sage.modules
[0 1 1 0 1 0 0 0 0 0]
[1 0 0 0 0 1 1 0 0 0]
[1 0 0 1 0 0 0 1 0 0]
[0 0 1 0 0 0 1 0 1 0]
[1 0 0 0 0 0 0 0 1 1]
[0 1 0 0 0 0 0 1 1 0]
[0 1 0 1 0 0 0 0 0 1]
[0 0 1 0 0 1 0 0 0 1]
[0 0 0 1 1 1 0 0 0 0]
[0 0 0 0 1 0 1 1 0 0]
sage: G = graphs.ChvatalGraph()
sage: bandwidth(G)
(6, [0, 5, 9, 4, 10, 1, 6, 11, 3, 8, 7, 2])
sage: G.adjacency_matrix(vertices=[0, 5, 9, 4, 10, 1, 6, 11, 3, 8, 7, 2])        #␣
↪needs sage.modules
[0 0 1 1 0 1 1 0 0 0 0 0]
[0 0 0 1 1 1 0 1 0 0 0 0]
[1 0 0 0 1 0 0 1 1 0 0 0]
[1 1 0 0 0 0 0 0 1 1 0 0]
[0 1 1 0 0 0 1 0 0 1 0 0]
[1 1 0 0 0 0 0 0 0 0 1 1]
[1 0 0 0 1 0 0 1 0 0 0 1]
[0 1 1 0 0 0 1 0 0 0 1 0]
[0 0 1 1 0 0 0 0 0 0 1 1]
[0 0 0 1 1 0 0 0 0 0 1 1]
[0 0 0 0 0 1 0 1 1 1 0 0]
[0 0 0 0 0 1 1 0 1 1 0 0]
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_decompositions.bandwidth import bandwidth
>>> G = graphs.PetersenGraph()
>>> bandwidth(G,Integer(3))
```

(continues on next page)

(continued from previous page)

```
False
>>> bandwidth(G)
(5, [0, 4, 5, 8, 1, 9, 3, 7, 6, 2])
>>> G.adjacency_matrix(vertices=[Integer(0), Integer(4), Integer(5), Integer(8),␣
↪Integer(1), Integer(9), Integer(3), Integer(7), Integer(6), Integer(2)])       ␣
↪       # needs sage.modules
[0 1 1 0 1 0 0 0 0 0]
[1 0 0 0 0 1 1 0 0 0]
[1 0 0 1 0 0 0 1 0 0]
[0 0 1 0 0 0 1 0 1 0]
[1 0 0 0 0 0 0 0 1 1]
[0 1 0 0 0 0 0 1 1 0]
[0 1 0 1 0 0 0 0 0 1]
[0 0 1 0 0 1 0 0 0 1]
[0 0 0 1 1 1 0 0 0 0]
[0 0 0 0 1 0 1 1 0 0]
>>> G = graphs.ChvatalGraph()
>>> bandwidth(G)
(6, [0, 5, 9, 4, 10, 1, 6, 11, 3, 8, 7, 2])
>>> G.adjacency_matrix(vertices=[Integer(0), Integer(5), Integer(9), Integer(4),␣
↪Integer(10), Integer(1), Integer(6), Integer(11), Integer(3), Integer(8),␣
↪Integer(7), Integer(2)])       # needs sage.modules
[0 0 1 1 0 1 1 0 0 0 0 0]
[0 0 0 1 1 1 0 1 0 0 0 0]
[1 0 0 0 1 0 0 1 1 0 0 0]
[1 1 0 0 0 0 0 0 1 1 0 0]
[0 1 1 0 0 0 1 0 0 1 0 0]
[1 1 0 0 0 0 0 0 0 0 1 1]
[1 0 0 0 1 0 0 1 0 0 0 1]
[0 1 1 0 0 0 1 0 0 0 1 0]
[0 0 1 1 0 0 0 0 0 0 1 1]
[0 0 0 1 1 0 0 0 0 0 1 1]
[0 0 0 0 0 1 0 1 1 1 0 0]
[0 0 0 0 0 1 1 0 1 1 0 0]
```

5.27 Cutwidth

This module implements several algorithms to compute the cutwidth of a graph and the corresponding ordering of the vertices. It also implements tests functions for evaluation the width of a linear ordering (or layout).

Given an ordering $v_1, \cdots, v_n$ of the vertices of $V(G)$, its *cost* is defined as:

$$c(v_1, ..., v_n) = \max_{1 \leq i \leq n-1} c'(\{v_1, ..., v_i\})$$

Where

$$c'(S) = |\{(u, w) \in E(G) \mid u \in S \text{ and } w \in V(G) \backslash S\}|$$

The *cutwidth* of a graph G is equal to the minimum cost of an ordering of its vertices.

This module contains the following methods

cutwidth()	Return the cutwidth of the graph and the corresponding vertex ordering.
cutwidth_dyn()	Compute the cutwidth of G using an exponential time and space algorithm based on dynamic programming
cutwidth_MILP()	Compute the cutwidth of G and the optimal ordering of its vertices using an MILP formulation
width_of_cut_decomposition()	Return the width of the cut decomposition induced by the linear ordering L of the vertices of G

5.27.1 Exponential algorithm for cutwidth

In order to find an optimal ordering of the vertices for the vertex separation, this algorithm tries to save time by computing the function $c'(S)$ **at most once** once for each of the sets $S \subseteq V(G)$. These values are stored in an array of size 2^n where reading the value of $c'(S)$ or updating it can be done in constant time.

Assuming that we can compute the cost of a set S and remember it, finding an optimal ordering is an easy task. Indeed, we can think of the sequence $v_1, ..., v_n$ of vertices as a sequence of *sets* $\{v_1\}, \{v_1, v_2\}, ..., \{v_1, ..., v_n\}$, whose cost is precisely $\max c'(\{v_1\}), c'(\{v_1, v_2\}), ..., c'(\{v_1, ..., v_n\})$. Hence, when considering the digraph on the 2^n sets $S \subseteq V(G)$ where there is an arc from S to S' if $S' = S \cap \{v\}$ for some v (that is, if the sets S and S' can be consecutive in a sequence), an ordering of the vertices of G corresponds to a *path* from $\emptyset$ to $\{v_1, ..., v_n\}$. In this setting, checking whether there exists a ordering of cost less than k can be achieved by checking whether there exists a directed path $\emptyset$ to $\{v_1, ..., v_n\}$ using only sets of cost less than k. This is just a depth-first-search, for each k.

Lazy evaluation of c'

In the previous algorithm, most of the time is actually spent on the computation of $c'(S)$ for each set $S \subseteq V(G)$ – i.e. 2^n computations of neighborhoods. This can be seen as a huge waste of time when noticing that it is useless to know that the value $c'(S)$ for a set S is less than k if all the paths leading to S have a cost greater than k. For this reason, the value of $c'(S)$ is computed lazily during the depth-first search. Explanation :

When the depth-first search discovers a set of size less than k, the costs of its out-neighbors (the potential sets that could follow it in the optimal ordering) are evaluated. When an out-neighbor is found that has a cost smaller than k, the depth-first search continues with this set, which is explored with the hope that it could lead to a path toward $\{v_1, ..., v_n\}$. On the other hand, if an out-neighbour has a cost larger than k it is useless to attempt to build a cheap sequence going though this set, and the exploration stops there. This way, a large number of sets will never be evaluated and *a lot* of computational time is saved this way.

Besides, some improvement is also made by "improving" the values found by c'. Indeed, $c'(S)$ is a lower bound on the cost of a sequence containing the set S, but if all out-neighbors of S have a cost of $c'(S) + 5$ then one knows that having S in a sequence means a total cost of at least $c'(S) + 5$. For this reason, for each set S we store the value of $c'(S)$, and replace it by $\max(c'(S), \min_{\text{next}})$ (where $\min_{\text{next}}$ is the minimum of the costs of the out-neighbors of S) once the costs of these out-neighbors have been evaluated by the algorithm.

This algorithm and its implementation are very similar to *sage.graphs.graph_decompositions.vertex_separation.vertex_separation_exp()*. The main difference is in the computation of $c'(S)$. See the *vertex separation module's documentation* for more details on this algorithm.

Note

Because of its current implementation, this algorithm only works on graphs on strictly less than 32 vertices. This can be changed to 64 if necessary, but 32 vertices already require 4GB of memory.

5.27.2 MILP formulation for the cutwidth

We describe a mixed integer linear program (MILP) for determining an optimal layout for the cutwidth of G.

Variables:

- x_v^k – variable set to 1 if vertex v is placed in the ordering at position i with $i \leq k$, and 0 otherwise
- $y_{u,v}^k$ – variable set to 1 if one of u or v is at a position $i \leq k$ and the other is at a position $j > k$, and so we have to count edge uv at position k. Otherwise, $y_{u,v}^k = 0$. The value of $y_{u,v}^k$ is a xor of the values of x_u^k and x_v^k.
- z – objective value to minimize. It is equal to the maximum over all position k of the number of edges with one extremity at position at most k and the other at position strictly more than k, that is $\sum_{uv \in E} y_{u,v}^k$.

MILP formulation:

Minimize:

$$z$$

Subject to:

$$\begin{aligned}
\sum_{i=0}^{k-1} x_v^i \leq k * x_v^k & \qquad \forall v \in V,\ k \in [1, n-1] \quad (1)\\
x_v^n = 1 & \qquad \forall v \in V \quad (2)\\
\sum_{v \in V} x_v^k = k+1 & \qquad \forall k \in [0, n-1] \quad (3)\\
x_u^k - x_v^k \leq y_{u,v}^k & \qquad \forall uv \in E,\ \forall k \in [0, n-1] \quad (4)\\
x_v^k - x_u^k \leq y_{u,v}^k & \qquad \forall uv \in E,\ \forall k \in [0, n-1] \quad (5)\\
\sum_{uv \in E} y_{u,v}^k \leq z & \qquad \forall k \in [0, n-1] \quad (6)\\
0 \leq z \leq |E| &
\end{aligned}$$

Constraints (1)-(3) ensure that all vertices have a distinct position. Constraints (4)-(5) force variable $y_{u,v}^k$ to 1 if the edge is in the cut. Constraint (6) count the number of edges starting at position at most k and ending at a position strictly larger than k.

This formulation corresponds to method `cutwidth_MILP()`.

5.27.3 Authors

- David Coudert (2015-06): Initial version

5.27.4 Methods

sage.graphs.graph_decompositions.cutwidth.**cutwidth**(*G*, *algorithm='exponential'*, *cut_off=0*, *solver=None*, *verbose=False*, *integrality_tolerance=0.001*)

Return the cutwidth of the graph and the corresponding vertex ordering.

INPUT:

- `G` – a Graph or a DiGraph
- `algorithm` – string (default: `'exponential'`); algorithm to use among:
  - `exponential` – use an exponential time and space algorithm based on dynamic programming. This algorithm only works on graphs with strictly less than 32 vertices.
  - `MILP` – use a mixed integer linear programming formulation. This algorithm has no size restriction but could take a very long time
- `cut_off` – integer (default: 0); used to stop the search as soon as a solution with width at most `cut_off` is found, if any. If this bound cannot be reached, the best solution found is returned.
- `solver` – string (default: `None`); specifies a Mixed Integer Linear Programming (MILP) solver to be used. If set to `None`, the default one is used. For more information on MILP solvers and which default solver is used, see the method `solve` of the class `MixedIntegerLinearProgram`.
- `verbose` – boolean (default: `False`); whether to display information on the computations
- `integrality_tolerance` – float; parameter for use with MILP solvers over an inexact base ring; see `MixedIntegerLinearProgram.get_values()`.

OUTPUT:

A pair `(cost, ordering)` representing the optimal ordering of the vertices and its cost.

EXAMPLES:

Cutwidth of a Complete Graph:

```
sage: from sage.graphs.graph_decompositions.cutwidth import cutwidth
sage: G = graphs.CompleteGraph(5)
sage: cw,L = cutwidth(G); cw
6
sage: K = graphs.CompleteGraph(6)
sage: cw,L = cutwidth(K); cw
9
sage: cw,L = cutwidth(K+K); cw
9
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_decompositions.cutwidth import cutwidth
>>> G = graphs.CompleteGraph(Integer(5))
>>> cw,L = cutwidth(G); cw
6
>>> K = graphs.CompleteGraph(Integer(6))
>>> cw,L = cutwidth(K); cw
9
>>> cw,L = cutwidth(K+K); cw
9
```

The cutwidth of a $p \times q$ Grid Graph with $p \leq q$ is $p + 1$:

```
sage: from sage.graphs.graph_decompositions.cutwidth import cutwidth
sage: G = graphs.Grid2dGraph(3,3)
sage: cw,L = cutwidth(G); cw
4
sage: G = graphs.Grid2dGraph(3,5)
sage: cw,L = cutwidth(G); cw
4
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_decompositions.cutwidth import cutwidth
>>> G = graphs.Grid2dGraph(Integer(3),Integer(3))
>>> cw,L = cutwidth(G); cw
4
>>> G = graphs.Grid2dGraph(Integer(3),Integer(5))
>>> cw,L = cutwidth(G); cw
4
```

sage.graphs.graph_decompositions.cutwidth.**cutwidth_MILP**(*G*, *lower_bound=0*, *solver=None*, *verbose=0*, *integrality_tolerance=0.001*)

MILP formulation for the cutwidth of a Graph.

This method uses a mixed integer linear program (MILP) for determining an optimal layout for the cutwidth of G. See the *module's documentation* for more details on this MILP formulation.

INPUT:

- `G` – a Graph
- `lower_bound` – integer (default: 0); the algorithm searches for a solution with cost larger or equal to `lower_bound`. If the given bound is larger than the optimal solution the returned solution might not be optimal. If the given bound is too high, the algorithm might not be able to find a feasible solution.
- `solver` – string (default: `None`); specifies a Mixed Integer Linear Programming (MILP) solver to be used. If set to `None`, the default one is used. For more information on MILP solvers and which default solver is used, see the method `solve` of the class `MixedIntegerLinearProgram`.
- `verbose` – integer (default: 0); sets the level of verbosity. Set to 0 by default, which means quiet.
- `integrality_tolerance` – float; parameter for use with MILP solvers over an inexact base ring; see `MixedIntegerLinearProgram.get_values()`.

OUTPUT:

A pair `(cost, ordering)` representing the optimal ordering of the vertices and its cost.

EXAMPLES:

Cutwidth of a Cycle graph:

```
sage: from sage.graphs.graph_decompositions import cutwidth
sage: G = graphs.CycleGraph(5)
sage: cw, L = cutwidth.cutwidth_MILP(G); cw                                    #␣
↪needs sage.numerical.mip
2
sage: cw == cutwidth.width_of_cut_decomposition(G, L)                          #␣
↪needs sage.numerical.mip
True
```

(continues on next page)

(continued from previous page)

```
sage: cwe, Le = cutwidth.cutwidth_dyn(G); cwe
2
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_decompositions import cutwidth
>>> G = graphs.CycleGraph(Integer(5))
>>> cw, L = cutwidth.cutwidth_MILP(G); cw                                    #␣
↪needs sage.numerical.mip
2
>>> cw == cutwidth.width_of_cut_decomposition(G, L)                          #␣
↪needs sage.numerical.mip
True
>>> cwe, Le = cutwidth.cutwidth_dyn(G); cwe
2
```

Cutwidth of a Complete graph:

```
sage: from sage.graphs.graph_decompositions import cutwidth
sage: G = graphs.CompleteGraph(4)
sage: cw, L = cutwidth.cutwidth_MILP(G); cw                                  #␣
↪needs sage.numerical.mip
4
sage: cw == cutwidth.width_of_cut_decomposition(G, L)                        #␣
↪needs sage.numerical.mip
True
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_decompositions import cutwidth
>>> G = graphs.CompleteGraph(Integer(4))
>>> cw, L = cutwidth.cutwidth_MILP(G); cw                                    #␣
↪needs sage.numerical.mip
4
>>> cw == cutwidth.width_of_cut_decomposition(G, L)                          #␣
↪needs sage.numerical.mip
True
```

Cutwidth of a Path graph:

```
sage: from sage.graphs.graph_decompositions import cutwidth
sage: G = graphs.PathGraph(3)
sage: cw, L = cutwidth.cutwidth_MILP(G); cw                                  #␣
↪needs sage.numerical.mip
1
sage: cw == cutwidth.width_of_cut_decomposition(G, L)                        #␣
↪needs sage.numerical.mip
True
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_decompositions import cutwidth
>>> G = graphs.PathGraph(Integer(3))
>>> cw, L = cutwidth.cutwidth_MILP(G); cw                                    #␣
↪needs sage.numerical.mip
```

(continues on next page)

(continued from previous page)

```
1
>>> cw == cutwidth.width_of_cut_decomposition(G, L)                    #␣
↪needs sage.numerical.mip
True
```

sage.graphs.graph_decompositions.cutwidth.**cutwidth_dyn**(*G*, *lower_bound=0*)

Dynamic programming algorithm for the cutwidth of a Graph.

This function uses dynamic programming algorithm for determining an optimal layout for the cutwidth of G. See the *module's documentation* for more details on this method.

INPUT:

- `G` – a Graph
- `lower_bound` – integer (default: 0); the algorithm returns immediately if it finds a solution lower or equal to `lower_bound` (in which case it may not be optimal).

OUTPUT:

A pair `(cost, ordering)` representing the optimal ordering of the vertices and its cost.

Note

Because of its current implementation, this algorithm only works on graphs on strictly less than 32 vertices. This can be changed to 63 if necessary, but 32 vertices already require 4GB of memory.

sage.graphs.graph_decompositions.cutwidth.**width_of_cut_decomposition**(*G*, *L*)

Return the width of the cut decomposition induced by the linear ordering L of the vertices of G.

If G is an instance of *Graph*, this function returns the width $cw_L(G)$ of the cut decomposition induced by the linear ordering L of the vertices of G.

$$cw_L(G) = \max_{0 \leq i < |V|-1} |\{(u, w) \in E(G) \mid u \in L[:i] \text{ and } w \in V(G) \setminus L[:i]\}|$$

INPUT:

- `G` – a Graph
- `L` – a linear ordering of the vertices of `G`

EXAMPLES:

Cut decomposition of a Cycle graph:

```
sage: from sage.graphs.graph_decompositions import cutwidth
sage: G = graphs.CycleGraph(6)
sage: L = G.vertices(sort=False)
sage: cutwidth.width_of_cut_decomposition(G, L)
2
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_decompositions import cutwidth
>>> G = graphs.CycleGraph(Integer(6))
>>> L = G.vertices(sort=False)
>>> cutwidth.width_of_cut_decomposition(G, L)
2
```

Cut decomposition of a Path graph:

```
sage: from sage.graphs.graph_decompositions import cutwidth
sage: P = graphs.PathGraph(6)
sage: cutwidth.width_of_cut_decomposition(P, [0, 1, 2, 3, 4, 5])
1
sage: cutwidth.width_of_cut_decomposition(P, [5, 0, 1, 2, 3, 4])
2
sage: cutwidth.width_of_cut_decomposition(P, [0, 2, 4, 1, 3, 5])
5
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_decompositions import cutwidth
>>> P = graphs.PathGraph(Integer(6))
>>> cutwidth.width_of_cut_decomposition(P, [Integer(0), Integer(1), Integer(2),␣
↪Integer(3), Integer(4), Integer(5)])
1
>>> cutwidth.width_of_cut_decomposition(P, [Integer(5), Integer(0), Integer(1),␣
↪Integer(2), Integer(3), Integer(4)])
2
>>> cutwidth.width_of_cut_decomposition(P, [Integer(0), Integer(2), Integer(4),␣
↪Integer(1), Integer(3), Integer(5)])
5
```

5.28 Compact structure for fast operations on less than 32 vertices

This module implements a digraph structure meant to be used in Cython in **highly enumerative** algorithms. It can store graphs on less than `sizeof(int)` vertices and perform several basic operations **quickly** (add/remove arcs, count the out-neighborhood of a set of vertices or return its cardinality).

Sets and integers :

In the following code, sets are represented as integers, where the i-th bit is set if element i belongs to the set.

class sage.graphs.graph_decompositions.fast_digraph.**FastDigraph**

Bases: `object`

print_adjacency_matrix()

Display the adjacency matrix of `self`.

EXAMPLES:

```
sage: cython_code = [
....: 'from sage.graphs.graph import Graph',
....: 'from sage.graphs.graph_decompositions.fast_digraph cimport FastDigraph
↪',
....: 'FastDigraph(Graph([(0, 1), (1, 2)])).print_adjacency_matrix()']
sage: cython(os.linesep.join(cython_code))                                  #␣
↪needs sage.misc.cython
010
101
010
```

```
>>> from sage.all import *
>>> cython_code = [
... 'from sage.graphs.graph import Graph',
... 'from sage.graphs.graph_decompositions.fast_digraph cimport FastDigraph',
... 'FastDigraph(Graph([(0, 1), (1, 2)])).print_adjacency_matrix()']
>>> cython(os.linesep.join(cython_code))                                    #␣
↪needs sage.misc.cython
010
101
010
```

sage.graphs.graph_decompositions.fast_digraph.**test_popcount**()

Correction test for popcount32.

EXAMPLES:

```
sage: from sage.graphs.graph_decompositions.fast_digraph import test_popcount
sage: test_popcount() # not tested
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_decompositions.fast_digraph import test_popcount
>>> test_popcount() # not tested
```

5.29 Products of graphs

This module gathers everything related to graph products. At the moment it contains an implementation of a recognition algorithm for graphs that can be written as a Cartesian product of smaller ones.

Author:

- Nathann Cohen (May 2012 – coded while watching the election of Francois Hollande on TV)

5.29.1 Cartesian product of graphs – the recognition problem

First, a definition:

> **Definition** The Cartesian product of two graphs G and H, denoted $G\Box H$, is a graph defined on the pairs $(g, h) \in V(G) \times V(H)$.
>
> Two elements $(g, h), (g', h') \in V(G\Box H)$ are adjacent in $G\Box H$ if and only if :
>
> - $g = g'$ and $hh' \in H$; or
> - $h = h'$ and $gg' \in G$

Two remarks follow :

1. The Cartesian product is commutative
2. Any edge uv of a graph $G_1\Box \cdots \Box G_k$ can be given a color i corresponding to the unique index i such that u_i and v_i differ.

The problem that is of interest to us in the present module is the following:

> **Recognition problem** Given a graph G, can we guess whether there exist graphs $G_1, ..., G_k$ such that $G = G_1\Box \cdots \Box G_k$?

This problem can actually be solved, and the resulting factorization is unique. What is explained below can be found in the book *Handbook of Product Graphs* [HIK2011].

Everything is actually based on simple observations. Given a graph G, finding out whether G can be written as the product of several graphs can be attempted by trying to color its edges according to some rules. Indeed, if we are to color the edges of G in such a way that each color class represents a factor of G, we must ensure several things.

Remark 1 In any cycle of G no color can appear exactly once.

Indeed, if only one edge uv of a cycle were labelled with color i, it would mean that:

1. The only difference between u and v lies in their i th coordinate
2. It is possible to go from u to v by changing only coordinates different from the i th

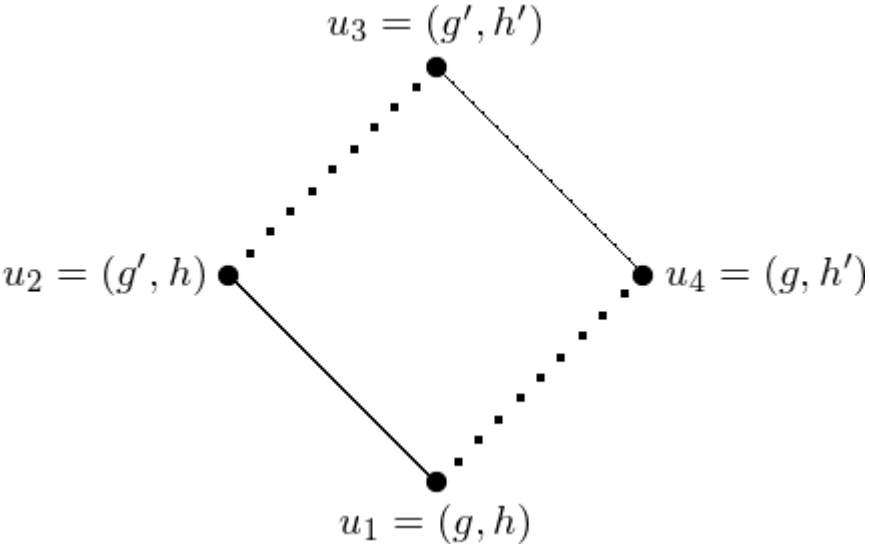
A contradiction indeed.



That means that, for instance, the edges of a triangle necessarily have the same color.

Remark 2 If two consecutive edges u_1u_2 and u_2u_3 have different colors, there necessarily exists a unique vertex u_4 different from u_2 and incident to both u_1 and u_3.

In this situation, opposed edges necessarily have the same colors because of the previous remark.



1st criterion : As a corollary, we know that:

1. If two vertices u, v have a *unique* common neighbor x, then ux and xv have the same color.
2. If two vertices u, v have more that two common neighbors $x_1, ..., x_k$ then all edges between the x_i and the vertices of u, v have the same color. This is also a consequence of the first remark.

2nd criterion : if two edges uv and $u'v'$ of the product graph $G\Box H$ are such that $d(u,u') + d(v,v') \neq d(u,v') + d(v,u')$ then the two edges uv and $u'v'$ necessarily have the same color.

This is a consequence of the fact that for any two vertices u, v of $G\Box H$ (where $u = (u_G, u_H)$ and $v = (v_G, v_H)$), we have $d(u,v) = d_G(u_G, v_G) + d_H(u_H, v_H)$. Indeed, a shortest path from u to v in $G\Box H$ contains the information of a shortest path from u_G to v_G in G, and a shortest path from u_H to v_H in H.

The algorithm

The previous remarks tell us that some edges are in some way equivalent to some others, i.e. that their colors are equal. In order to compute the coloring we are looking for, we therefore build a graph on the *edges* of a graph G, linking two edges whenever they are found to be equivalent according to the previous remarks.

All that is left to do is to compute the connected components of this new graph, as each of them representing the edges of a factor. Of course, only one connected component indicates that the graph has no factorization.

Then again, please refer to [HIK2011] for any technical question.

To Do

This implementation is made at Python level, and some parts of the algorithm could be rewritten in Cython to save time. Especially when enumerating all pairs of edges and computing their distances. This can easily be done in C with the functions from the *sage.graphs.distances_all_pairs* module.

5.29.2 Methods

sage.graphs.graph_decompositions.graph_products.**is_cartesian_product**(*g*, *certificate=False*, *relabeling=False*, *immutable=None*)

Test whether the graph is a Cartesian product.

INPUT:

- `certificate` – boolean (default: `False`); if `certificate = False` (default) the method only returns `True` or `False` answers. If `certificate = True`, the `True` answers are replaced by the list of the factors of the graph.
- `relabeling` – boolean (default: `False`); if `relabeling = True` (implies `certificate = True`), the method also returns a dictionary associating to each vertex its natural coordinates as a vertex of a product graph. If g is not a Cartesian product, `None` is returned instead.
- `immutable` – boolean (default: `None`); whether to create a mutable/immutable graph. `immutable=None` (default) means that the graph and its factors will behave the same way.

See also

- *cartesian_product()*
- *graph_products* – a module on graph products

Note

This algorithm may run faster whenever the graph's vertices are integers (see *relabel()*). Give it a try if it is too slow !

EXAMPLES:

The Petersen graph is prime:

```
sage: from sage.graphs.graph_decompositions.graph_products import is_cartesian_
↪product
sage: g = graphs.PetersenGraph()
sage: is_cartesian_product(g)
False
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_decompositions.graph_products import is_cartesian_
↪product
>>> g = graphs.PetersenGraph()
>>> is_cartesian_product(g)
False
```

A 2d grid is the product of paths:

```
sage: g = graphs.Grid2dGraph(5,5)
sage: p1, p2 = is_cartesian_product(g, certificate = True)
sage: p1.is_isomorphic(graphs.PathGraph(5))
True
sage: p2.is_isomorphic(graphs.PathGraph(5))
True
```

```
>>> from sage.all import *
>>> g = graphs.Grid2dGraph(Integer(5),Integer(5))
>>> p1, p2 = is_cartesian_product(g, certificate = True)
>>> p1.is_isomorphic(graphs.PathGraph(Integer(5)))
True
>>> p2.is_isomorphic(graphs.PathGraph(Integer(5)))
True
```

Forgetting the graph's labels, then finding them back:

```
sage: g.relabel()
sage: b,D = g.is_cartesian_product(g, relabeling=True)
sage: b
True
sage: D  # random isomorphism
{0: (20, 0), 1: (20, 1), 2: (20, 2), 3: (20, 3), 4: (20, 4),
 5: (15, 0), 6: (15, 1), 7: (15, 2), 8: (15, 3), 9: (15, 4),
 10: (10, 0), 11: (10, 1), 12: (10, 2), 13: (10, 3), 14: (10, 4),
 15: (5, 0), 16: (5, 1), 17: (5, 2), 18: (5, 3), 19: (5, 4),
 20: (0, 0), 21: (0, 1), 22: (0, 2), 23: (0, 3), 24: (0, 4)}
```

```
>>> from sage.all import *
>>> g.relabel()
>>> b,D = g.is_cartesian_product(g, relabeling=True)
>>> b
True
>>> D  # random isomorphism
{0: (20, 0), 1: (20, 1), 2: (20, 2), 3: (20, 3), 4: (20, 4),
 5: (15, 0), 6: (15, 1), 7: (15, 2), 8: (15, 3), 9: (15, 4),
 10: (10, 0), 11: (10, 1), 12: (10, 2), 13: (10, 3), 14: (10, 4),
 15: (5, 0), 16: (5, 1), 17: (5, 2), 18: (5, 3), 19: (5, 4),
 20: (0, 0), 21: (0, 1), 22: (0, 2), 23: (0, 3), 24: (0, 4)}
```

And of course, we find the factors back when we build a graph from a product:

```
sage: g = graphs.PetersenGraph().cartesian_product(graphs.CycleGraph(3))
sage: g1, g2 = is_cartesian_product(g, certificate = True)
sage: any( x.is_isomorphic(graphs.PetersenGraph()) for x in [g1,g2])
True
sage: any( x.is_isomorphic(graphs.CycleGraph(3)) for x in [g1,g2])
True
```

```
>>> from sage.all import *
>>> g = graphs.PetersenGraph().cartesian_product(graphs.CycleGraph(Integer(3)))
>>> g1, g2 = is_cartesian_product(g, certificate = True)
>>> any( x.is_isomorphic(graphs.PetersenGraph()) for x in [g1,g2])
True
>>> any( x.is_isomorphic(graphs.CycleGraph(Integer(3))) for x in [g1,g2])
True
```

sage.graphs.graph_decompositions.graph_products.**rooted_product**(*G*, *H*, *root=None*, *immutable=None*)

Return the rooted product of G and H.

The rooted product of two graphs G and H is the graph R defined as follows: take a copy of G and $|V(G)|$ copies of H, and for every vertex g_i of G, identify g_i with the root of the i-th copy of H. Mode formally, let $V(G) = \{g_1, g_2, \ldots, g_n\}$, $V(H) = \{h_1, h_2, \ldots, h_m\}$, and let h_1 be the root vertex of H. The vertex set $V(R)$ is equal to the cartesian product of the sets of vertices $V(G)$ and $V(H)$, that is $V(R) = \{(g_i, h_j) : g_i \in V(G), h_j \in V(H)\}$. The edge set $E(R)$ is the union of the edges of a copy of G, that is $\{((g_i, h_1), (g_j, h_1)) : (g_i, g_j) \in E(G)\}$, and the edges of the copies of H for every $g_i \in V(G)$, that is $\{((g_i, h_j), (g_i, h_k)) : (h_j, h_k) \in V(H)\}$.

See Wikipedia article Rooted_product_of_graphs for more details.

See also

- *cartesian_product()* – return the cartesian product of two graphs
- *graph_products* – a module on graph products

INPUT:

- `G, H` – two (di)graphs
- `immutable` – boolean (default: `None`); whether to create a mutable/immutable (di)graph. When `immutable=None` (default) the rooted product will be mutable if one of `G` or `H` is mutable and immutable

otherwise.

EXAMPLES:

The rooted product of two trees is a tree:

```
sage: T1 = graphs.RandomTree(7)
sage: T2 = graphs.RandomTree(8)
sage: T = T1.rooted_product(T2)
sage: T.is_tree()
True
```

```
>>> from sage.all import *
>>> T1 = graphs.RandomTree(Integer(7))
>>> T2 = graphs.RandomTree(Integer(8))
>>> T = T1.rooted_product(T2)
>>> T.is_tree()
True
```

The rooted product of G and H depends on the selected root in H:

```
sage: G = graphs.CycleGraph(4)
sage: H = graphs.PathGraph(3)
sage: R1 = G.rooted_product(H, root=0)
sage: R2 = G.rooted_product(H, root=1)
sage: R1.is_isomorphic(R2)
False
sage: sorted(R1.degree())
[1, 1, 1, 1, 2, 2, 2, 2, 3, 3, 3, 3]
sage: sorted(R2.degree())
[1, 1, 1, 1, 1, 1, 1, 1, 4, 4, 4, 4]
```

```
>>> from sage.all import *
>>> G = graphs.CycleGraph(Integer(4))
>>> H = graphs.PathGraph(Integer(3))
>>> R1 = G.rooted_product(H, root=Integer(0))
>>> R2 = G.rooted_product(H, root=Integer(1))
>>> R1.is_isomorphic(R2)
False
>>> sorted(R1.degree())
[1, 1, 1, 1, 2, 2, 2, 2, 3, 3, 3, 3]
>>> sorted(R2.degree())
[1, 1, 1, 1, 1, 1, 1, 1, 4, 4, 4, 4]
```

The domination number of the rooted product of any graph G and a path of order 2 is the order of G:

```
sage: G = graphs.RandomGNP(20, .3)
sage: P = graphs.PathGraph(2)
sage: R = G.rooted_product(P)
sage: len(R.dominating_set()) == G.order()                                    #␣
↪needs sage.numerical.mip
True
sage: G = digraphs.RandomDirectedGNP(20, .3)
sage: P = digraphs.Path(2)
```

(continues on next page)

(continued from previous page)

```
sage: R = G.rooted_product(P)
sage: len(R.dominating_set()) == G.order()                                  #␣
↪needs sage.numerical.mip
True
```

```
>>> from sage.all import *
>>> G = graphs.RandomGNP(Integer(20), RealNumber('.3'))
>>> P = graphs.PathGraph(Integer(2))
>>> R = G.rooted_product(P)
>>> len(R.dominating_set()) == G.order()                                    #␣
↪needs sage.numerical.mip
True
>>> G = digraphs.RandomDirectedGNP(Integer(20), RealNumber('.3'))
>>> P = digraphs.Path(Integer(2))
>>> R = G.rooted_product(P)
>>> len(R.dominating_set()) == G.order()                                    #␣
↪needs sage.numerical.mip
True
```

The rooted product of two graphs is a subgraph of the cartesian product of the same two graphs:

```
sage: G = graphs.RandomGNP(6, .4)
sage: H = graphs.RandomGNP(7, .4)
sage: R = G.rooted_product(H)
sage: C = G.cartesian_product(H)
sage: R.is_subgraph(C, induced=False)
True
```

```
>>> from sage.all import *
>>> G = graphs.RandomGNP(Integer(6), RealNumber('.4'))
>>> H = graphs.RandomGNP(Integer(7), RealNumber('.4'))
>>> R = G.rooted_product(H)
>>> C = G.cartesian_product(H)
>>> R.is_subgraph(C, induced=False)
True
```

Corner cases:

```
sage: Graph().rooted_product(Graph())
Rooted product of Graph on 0 vertices and Graph on 0 vertices: Graph on 0 vertices
sage: Graph(1).rooted_product(Graph())
Rooted product of Graph on 1 vertex and Graph on 0 vertices: Graph on 0 vertices
sage: Graph().rooted_product(Graph(1))
Rooted product of Graph on 0 vertices and Graph on 1 vertex: Graph on 0 vertices
sage: Graph(1).rooted_product(Graph(1))
Rooted product of Graph on 1 vertex and Graph on 1 vertex: Graph on 1 vertex
```

```
>>> from sage.all import *
>>> Graph().rooted_product(Graph())
Rooted product of Graph on 0 vertices and Graph on 0 vertices: Graph on 0 vertices
>>> Graph(Integer(1)).rooted_product(Graph())
Rooted product of Graph on 1 vertex and Graph on 0 vertices: Graph on 0 vertices
```

(continues on next page)

(continued from previous page)

```
>>> Graph().rooted_product(Graph(Integer(1)))
Rooted product of Graph on 0 vertices and Graph on 1 vertex: Graph on 0 vertices
>>> Graph(Integer(1)).rooted_product(Graph(Integer(1)))
Rooted product of Graph on 1 vertex and Graph on 1 vertex: Graph on 1 vertex
```

5.30 Slice decomposition

This module implements an extended lexBFS algorithm for computing the slice decomposition of undirected graphs and the class *SliceDecomposition* to represent such decompositions.

A formal definition of slice decompositions can be found in Section 3.2 of [TCHP2008] and a description of the extended lexBFS algorithm is given in Section 3.3 of [TCHP2008].

AUTHORS:

- Cyril Bouvier (2024-06-25): initial version

class sage.graphs.graph_decompositions.slice_decomposition.**SliceDecomposition**

Bases: SageObject

Represents a slice decomposition of a simple directed graph.

INPUT:

- G – a Sage graph.
- initial_vertex – (default: None); the first vertex to consider.

See also

- *slice_decomposition()* – compute a slice decomposition of the simple undirected graph
- Section 3.2 of [TCHP2008] for a formal definition.

EXAMPLES:

The constructor of the *SliceDecomposition* class is called by the *slice_decomposition()* method of undirected graphs:

```
sage: from sage.graphs.graph_decompositions.slice_decomposition import␣
↪SliceDecomposition
sage: G = graphs.PetersenGraph()
sage: SliceDecomposition(G) == G.slice_decomposition()
True
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_decompositions.slice_decomposition import␣
↪SliceDecomposition
>>> G = graphs.PetersenGraph()
>>> SliceDecomposition(G) == G.slice_decomposition()
True
```

The vertex appearing first in the slice decomposition can be specified:

```
sage: from sage.graphs.graph_decompositions.slice_decomposition import␣
↪SliceDecomposition
sage: SliceDecomposition(graphs.PetersenGraph(), initial_vertex=3)
[3[2[4[8]]] [1[7]] [0] [9] [6] [5]]
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_decompositions.slice_decomposition import␣
↪SliceDecomposition
>>> SliceDecomposition(graphs.PetersenGraph(), initial_vertex=Integer(3))
[3[2[4[8]]] [1[7]] [0] [9] [6] [5]]
```

Slice decompositions are not defined for directed graphs:

```
sage: from sage.graphs.graph_decompositions.slice_decomposition import␣
↪SliceDecomposition
sage: SliceDecomposition(DiGraph())
Traceback (most recent call last):
...
ValueError: parameter G must be an undirected graph
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_decompositions.slice_decomposition import␣
↪SliceDecomposition
>>> SliceDecomposition(DiGraph())
Traceback (most recent call last):
...
ValueError: parameter G must be an undirected graph
```

__getitem__(*v*)

Return the data about the x-slice of the vertex v.

INPUT:

- `v` – a vertex of the graph corresponding to the slice decomposition.

OUTPUT:

A dictionary with the keys:

- `"pivot"` – the vertex v given as parameter
- `"slice"` – the slice of v (see *slice()*)
- `"active_edges"` – the actives edges of v (see *active_edges()*)
- `"lexicographic_label"` – the lexicographic label of v (see *lexicographic_label()*)
- `"sequence"` – the x-slice sequence of v (see *xslice_sequence()*)

This method can also be called via *xslice_data()*.

EXAMPLES:

```
sage: G = Graph('L~mpn~Nrv{^o~_').relabel('abcdefguvwxyz',inplace=False)
sage: SD = G.slice_decomposition(initial_vertex='x')
sage: SD.xslice_data('a')
{'active_edges': [('a', 'b'),
  ('a', 'c'),
```

(continues on next page)

(continued from previous page)

```
  ('a', 'd'),
  ('a', 'e'),
  ('a', 'f'),
  ('c', 'g'),
  ('d', 'g'),
  ('f', 'g')],
 'lexicographic_label': ['x'],
 'pivot': 'a',
 'sequence': [['a'], ['b', 'c', 'd', 'e', 'f'], ['g']],
 'slice': ['a', 'b', 'c', 'd', 'e', 'f', 'g']}
sage: SD.xslice_data('u')
{'active_edges': [],
 'lexicographic_label': ['a', 'b', 'c', 'd', 'e', 'f', 'g'],
 'pivot': 'u',
 'sequence': [['u'], ['y', 'z']],
 'slice': ['u', 'y', 'z']}
```

```
>>> from sage.all import *
>>> G = Graph('L~mpn~Nrv{^o~_').relabel('abcdefguvwxyz',inplace=False)
>>> SD = G.slice_decomposition(initial_vertex='x')
>>> SD.xslice_data('a')
{'active_edges': [('a', 'b'),
  ('a', 'c'),
  ('a', 'd'),
  ('a', 'e'),
  ('a', 'f'),
  ('c', 'g'),
  ('d', 'g'),
  ('f', 'g')],
 'lexicographic_label': ['x'],
 'pivot': 'a',
 'sequence': [['a'], ['b', 'c', 'd', 'e', 'f'], ['g']],
 'slice': ['a', 'b', 'c', 'd', 'e', 'f', 'g']}
>>> SD.xslice_data('u')
{'active_edges': [],
 'lexicographic_label': ['a', 'b', 'c', 'd', 'e', 'f', 'g'],
 'pivot': 'u',
 'sequence': [['u'], ['y', 'z']],
 'slice': ['u', 'y', 'z']}
```

Some values of the returned dictionary can be obtained via other methods (*slice()*, *xslice_sequence()*, *active_edges()*, *lexicographic_label()*):

```
sage: SD.slice('a')
['a', 'b', 'c', 'd', 'e', 'f', 'g']
sage: SD.xslice_data('a')['slice']
['a', 'b', 'c', 'd', 'e', 'f', 'g']

sage: SD.xslice_sequence('a')
[['a'], ['b', 'c', 'd', 'e', 'f'], ['g']]
sage: SD.xslice_data('a')['sequence']
[['a'], ['b', 'c', 'd', 'e', 'f'], ['g']]
```

(continues on next page)

(continued from previous page)

```
sage: SD.active_edges('b') == SD.xslice_data('b')['active_edges']
True

sage: SD.lexicographic_label('u')
['a', 'b', 'c', 'd', 'e', 'f', 'g']
sage: SD.xslice_data('u')['lexicographic_label']
['a', 'b', 'c', 'd', 'e', 'f', 'g']
```

```
>>> from sage.all import *
>>> SD.slice('a')
['a', 'b', 'c', 'd', 'e', 'f', 'g']
>>> SD.xslice_data('a')['slice']
['a', 'b', 'c', 'd', 'e', 'f', 'g']

>>> SD.xslice_sequence('a')
[['a'], ['b', 'c', 'd', 'e', 'f'], ['g']]
>>> SD.xslice_data('a')['sequence']
[['a'], ['b', 'c', 'd', 'e', 'f'], ['g']]

>>> SD.active_edges('b') == SD.xslice_data('b')['active_edges']
True

>>> SD.lexicographic_label('u')
['a', 'b', 'c', 'd', 'e', 'f', 'g']
>>> SD.xslice_data('u')['lexicographic_label']
['a', 'b', 'c', 'd', 'e', 'f', 'g']
```

active_edges(v)

Return the active edges of the vertex v.

An edge (u, w) is said to be active for v if u and w belongs to two differents slices of the x-slice sequence of v. Note that it defines a partition of the edges of the underlying graph.

INPUT:

- v – a vertex of the graph corresponding to the slice decomposition.

OUTPUT:

A list of edges

EXAMPLES:

```
sage: G = Graph('L~mpn~Nrv{^o~_').relabel('abcdefguvwxyz',inplace=False)
sage: SD = G.slice_decomposition(initial_vertex='x')
sage: SD.xslice_sequence('a')
[['a'], ['b', 'c', 'd', 'e', 'f'], ['g']]
sage: ('c', 'g') in SD.active_edges('a')
True
sage: ('a', 'c') in SD.active_edges('a')
True
sage: ('c', 'd') in SD.active_edges('a')  # c and d in same slice
False
```

(continues on next page)

(continued from previous page)

```
sage: ('a', 'u') in SD.active_edges('a')  # u not in x-slice of a
False
```

```
>>> from sage.all import *
>>> G = Graph('L~mpn~Nrv{^o~_').relabel('abcdefguvwxyz',inplace=False)
>>> SD = G.slice_decomposition(initial_vertex='x')
>>> SD.xslice_sequence('a')
[['a'], ['b', 'c', 'd', 'e', 'f'], ['g']]
>>> ('c', 'g') in SD.active_edges('a')
True
>>> ('a', 'c') in SD.active_edges('a')
True
>>> ('c', 'd') in SD.active_edges('a')  # c and d in same slice
False
>>> ('a', 'u') in SD.active_edges('a')  # u not in x-slice of a
False
```

The set of active edges of every vertex is a partition of the edges:

```
sage: from itertools import chain
sage: E = list(chain(*(SD.active_edges(v) for v in G)))
sage: G.size() == len(E) == len(set(E)) \
....:   and all(G.has_edge(u, w) for v in G for u, w in SD.active_edges(v))
True
```

```
>>> from sage.all import *
>>> from itertools import chain
>>> E = list(chain(*(SD.active_edges(v) for v in G)))
>>> G.size() == len(E) == len(set(E))   and all(G.has_edge(u, w) for v in G␣
↪for u, w in SD.active_edges(v))
True
```

lexBFS_order()

Return the lexBFS order corresponding to the slice decomposition.

EXAMPLES:

```
sage: from sage.graphs.traversals import _is_valid_lex_BFS_order
sage: G = graphs.PetersenGraph(); SD = G.slice_decomposition()
sage: SD.lexBFS_order()
[0, 1, 4, 5, 2, 6, 3, 9, 7, 8]
sage: _is_valid_lex_BFS_order(G, SD.lexBFS_order())
True
```

```
>>> from sage.all import *
>>> from sage.graphs.traversals import _is_valid_lex_BFS_order
>>> G = graphs.PetersenGraph(); SD = G.slice_decomposition()
>>> SD.lexBFS_order()
[0, 1, 4, 5, 2, 6, 3, 9, 7, 8]
>>> _is_valid_lex_BFS_order(G, SD.lexBFS_order())
True
```

lexicographic_label(*v*)

Return the lexicographic label of the vertex v.

The lexicographic label of a vertex v is the list of all the neighbors of v that appear before v in the lexBFS ordering corresponding to the slice decomposition.

INPUT:

- v – a vertex of the graph corresponding to the slice decomposition.

OUTPUT:

A list of vertices.

EXAMPLES:

```
sage: G = Graph('L~mpn~Nrv{^o~_').relabel('abcdefguvwxyz',inplace=False)
sage: SD = G.slice_decomposition(initial_vertex='x')
sage: SD.lexicographic_label('f')
['x', 'a', 'c', 'd']
sage: pos = lambda v: SD.lexBFS_order().index(v)
sage: set(SD.lexicographic_label('f')) \
....:   == {v for v in G.neighbors('f') if pos(v) < pos('f')}
True
```

```
>>> from sage.all import *
>>> G = Graph('L~mpn~Nrv{^o~_').relabel('abcdefguvwxyz',inplace=False)
>>> SD = G.slice_decomposition(initial_vertex='x')
>>> SD.lexicographic_label('f')
['x', 'a', 'c', 'd']
>>> pos = lambda v: SD.lexBFS_order().index(v)
>>> set(SD.lexicographic_label('f'))   == {v for v in G.neighbors('f') if␣
↪pos(v) < pos('f')}
True
```

slice(*v*)

Return the slice of the vertex v.

The slice of v is the list of vertices u such that the neighbors of u that are before v in the lexBFS order are that same that the neighbors of v that are before v in the lexBFS order (*i.e.*, the lexicographic label of v). It can be shown that it is a factor of the lexBFS order.

INPUT:

- v – a vertex of the graph corresponding to the slice decomposition.

OUTPUT:

A list of vertices

EXAMPLES:

```
sage: G = Graph('L~mpn~Nrv{^o~_').relabel('abcdefguvwxyz',inplace=False)
sage: SD = G.slice_decomposition(initial_vertex='x')
sage: SD.slice('a')
['a', 'b', 'c', 'd', 'e', 'f', 'g']
```

```
>>> from sage.all import *
>>> G = Graph('L~mpn~Nrv{^o~_').relabel('abcdefguvwxyz',inplace=False)
>>> SD = G.slice_decomposition(initial_vertex='x')
>>> SD.slice('a')
['a', 'b', 'c', 'd', 'e', 'f', 'g']
```

The vertices of the slice have the same neighborhood “on the left”:

```
sage: pos = lambda v: SD.lexBFS_order().index(v)
sage: lla = set(SD.lexicographic_label('a'))
sage: all(lla == {u for u in G.neighbors(v) if pos(u) < pos('a')} \
....:       for v in SD.slice('a'))
True
```

```
>>> from sage.all import *
>>> pos = lambda v: SD.lexBFS_order().index(v)
>>> lla = set(SD.lexicographic_label('a'))
>>> all(lla == {u for u in G.neighbors(v) if pos(u) < pos('a')}       for v␣
↪in SD.slice('a'))
True
```

The slice is a factor of the lexBFS order:

```
sage: ''.join(SD.slice('a')) in ''.join(SD.lexBFS_order())
True
```

```
>>> from sage.all import *
>>> ''.join(SD.slice('a')) in ''.join(SD.lexBFS_order())
True
```

The slice of the initial vertex is the whole graph:

```
sage: SD.slice('x') == SD.lexBFS_order()
True
```

```
>>> from sage.all import *
>>> SD.slice('x') == SD.lexBFS_order()
True
```

underlying_graph()

Return the underlying graph corresponding to the slice decomposition.

If G was the graph given as parameter to compute the slice decomposition, the underlying graph corresponds to `G.to_simple()` where labels are ignored, *i.e.*, it is the input graph without loops, multiple edges and labels.

Note

This method is mostly defined to test the computation of lexicographic labels and actives edges.

EXAMPLES:

```
sage: G = Graph('L~mpn~Nrv{^o~_').relabel('abcdefguvwxyz',inplace=False)
sage: SD = G.slice_decomposition(initial_vertex='x')
sage: SD.underlying_graph() == G
True
```

```
>>> from sage.all import *
>>> G = Graph('L~mpn~Nrv{^o~_').relabel('abcdefguvwxyz',inplace=False)
>>> SD = G.slice_decomposition(initial_vertex='x')
>>> SD.underlying_graph() == G
True
```

The graph can have loops or multiple edges but they are ignored:

```
sage: G = graphs.CubeConnectedCycle(2)  # multiple edges
sage: SD = G.slice_decomposition()
sage: SD.underlying_graph() == G.to_simple(immutable=True)
True

sage: G = graphs.CubeConnectedCycle(1)  # loops
sage: SD = G.slice_decomposition()
sage: SD.underlying_graph() == G.to_simple(immutable=True)
True
```

```
>>> from sage.all import *
>>> G = graphs.CubeConnectedCycle(Integer(2))  # multiple edges
>>> SD = G.slice_decomposition()
>>> SD.underlying_graph() == G.to_simple(immutable=True)
True

>>> G = graphs.CubeConnectedCycle(Integer(1))  # loops
>>> SD = G.slice_decomposition()
>>> SD.underlying_graph() == G.to_simple(immutable=True)
True
```

xslice_data(v)

Return the data about the x-slice of the vertex v.

This method is a wrapper around *SliceDecomposition.__getitem__()*

xslice_sequence(v)

Return the x-slice sequence of the vertex v.

INPUT:

- v – a vertex of the graph corresponding to the slice decomposition.

OUTPUT:

A list of list corresponding to the x-slice sequence of v.

EXAMPLES:

```
sage: G = Graph('L~mpn~Nrv{^o~_').relabel('abcdefguvwxyz',inplace=False)
sage: SD = G.slice_decomposition(initial_vertex='x')
sage: SD.xslice_sequence('x')
```

(continues on next page)

(continued from previous page)

```
[['x'], ['a', 'b', 'c', 'd', 'e', 'f', 'g'], ['u', 'y', 'z'], ['v', 'w']]
sage: SD.xslice_sequence('a')
[['a'], ['b', 'c', 'd', 'e', 'f'], ['g']]
```

```
>>> from sage.all import *
>>> G = Graph('L~mpn~Nrv{^o~_').relabel('abcdefguvwxyz',inplace=False)
>>> SD = G.slice_decomposition(initial_vertex='x')
>>> SD.xslice_sequence('x')
[['x'], ['a', 'b', 'c', 'd', 'e', 'f', 'g'], ['u', 'y', 'z'], ['v', 'w']]
>>> SD.xslice_sequence('a')
[['a'], ['b', 'c', 'd', 'e', 'f'], ['g']]
```

The flatten x-slice sequence of a vertex corresponds to the slice of the same vertex:

```
sage: from itertools import chain
sage: all(list(chain(*SD.xslice_sequence(v))) == SD.slice(v) \
....:       for v in G)
True
```

```
>>> from sage.all import *
>>> from itertools import chain
>>> all(list(chain(*SD.xslice_sequence(v))) == SD.slice(v)       for v in G)
True
```

The first list of the sequence is always a singleton containing the input vertex:

```
sage: all(SD.xslice_sequence(v)[0] == [v] for v in G)
True
```

```
>>> from sage.all import *
>>> all(SD.xslice_sequence(v)[Integer(0)] == [v] for v in G)
True
```

If the length of the slice if more than 1, the second list of the sequence is either, all the remaining vertices of the slice of v, if v is isolated in the subgraph induced by the slice of v, or the neighbors of v in the subgraph induced by the slice of v:

```
sage: all(SD.xslice_sequence(v)[1] == SD.slice(v)[1:] for v in G \
....:           if G.subgraph(SD.slice(v)).degree(v) == 0 \
....:               and len(SD.slice(v)) > 1)
True
sage: for v in G:
....:     if len(SD.slice(v)) > 1:
....:         xslice_seq = SD.xslice_sequence(v)
....:         S = G.subgraph(SD.slice(v))
....:         if S.degree(v) > 0:
....:             set(xslice_seq[1]) == set(S.neighbor_iterator(v))
True
True
True
True
```

```
>>> from sage.all import *
>>> all(SD.xslice_sequence(v)[Integer(1)] == SD.slice(v)[Integer(1):] for v␣
↪in G           if G.subgraph(SD.slice(v)).degree(v) == Integer(0)          ␣
↪    and len(SD.slice(v)) > Integer(1))
True
>>> for v in G:
...     if len(SD.slice(v)) > Integer(1):
...         xslice_seq = SD.xslice_sequence(v)
...         S = G.subgraph(SD.slice(v))
...         if S.degree(v) > Integer(0):
...             set(xslice_seq[Integer(1)]) == set(S.neighbor_iterator(v))
True
True
True
True
```

sage.graphs.graph_decompositions.slice_decomposition.**slice_decomposition**(*G*, *initial_vertex=None*)

Compute a slice decomposition of the simple undirected graph

INPUT:

- `G` – a Sage graph.
- `initial_vertex` – (default: `None`); the first vertex to consider.

OUTPUT:

An object of type *`SliceDecomposition`* that represents a slice decomposition of `G`

Note

Loops and multiple edges are ignored during the computation of the slice decomposition.

ALGORITHM:

The method use the algorithm based on "partition refinement" described in [HMPV2000] and [TCHP2008]. The time complexity of this algorithm is in $O(n+m)$, and our implementation follows that complexity for `SparseGraph`. For `DenseGraph`, the complexity is $O(n^2)$.

EXAMPLES:

Slice decomposition of the Petersen Graph:

```
sage: G = graphs.PetersenGraph()
sage: SD = G.slice_decomposition(); SD
[0[1[4[5]]] [2[6]] [3] [9] [7] [8]]
```

```
>>> from sage.all import *
>>> G = graphs.PetersenGraph()
>>> SD = G.slice_decomposition(); SD
[0[1[4[5]]] [2[6]] [3] [9] [7] [8]]
```

The graph can have loops or multiple edges but they are ignored:

```
sage: H = Graph(G,loops=True,multiedges=True)
sage: H.add_edges([(4, 4), (2, 2), (1, 6)])
sage: SD2 = H.slice_decomposition()
sage: SD2 == SD
True
sage: SD2.underlying_graph() == G.to_simple(immutable=True)
True
```

```
>>> from sage.all import *
>>> H = Graph(G,loops=True,multiedges=True)
>>> H.add_edges([(Integer(4), Integer(4)), (Integer(2), Integer(2)), (Integer(1),␣
↪Integer(6))])
>>> SD2 = H.slice_decomposition()
>>> SD2 == SD
True
>>> SD2.underlying_graph() == G.to_simple(immutable=True)
True
```

The tree corresponding to the slice decomposition can be displayed using `view`:

```
sage: from sage.graphs.graph_latex import check_tkz_graph
sage: check_tkz_graph()  # random - depends on Tex installation
sage: view(G)  # not tested
sage: latex(G)  # to obtain the corresponding LaTeX code
\begin{tikzpicture}
...
\end{tikzpicture}
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_latex import check_tkz_graph
>>> check_tkz_graph()  # random - depends on Tex installation
>>> view(G)  # not tested
>>> latex(G)  # to obtain the corresponding LaTeX code
\begin{tikzpicture}
...
\end{tikzpicture}
```

Slice decompositions are only defined for undirected graphs:

```
sage: from sage.graphs.graph_decompositions.slice_decomposition import slice_
↪decomposition
sage: slice_decomposition(DiGraph())
Traceback (most recent call last):
...
ValueError: parameter G must be an undirected graph
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_decompositions.slice_decomposition import slice_
↪decomposition
>>> slice_decomposition(DiGraph())
Traceback (most recent call last):
...
ValueError: parameter G must be an undirected graph
```

5.31 Modular Decomposition

This module implements the function for computing the modular decomposition of undirected graphs.

AUTHORS:

- Lokesh Jain (2017): first implementation of the linear time algorithm of D. Corneil, M. Habib, C. Paul and M. Tedder [TCHP2008]
- David Einstein (2018): added the algorithm of M. Habib and M. Maurer [HM1979]
- Cyril Bouvier (2024): second implementation of the linear time algorithm of D. Corneil, M. Habib, C. Paul and M. Tedder [TCHP2008]

class sage.graphs.graph_decompositions.modular_decomposition.**Node**(*node_type*)

Bases: object

Node class stores information about the node type.

Node type can be PRIME, SERIES, PARALLEL, NORMAL or EMPTY.

- node_type – is of type NodeType and specifies the type of node

classmethod create_leaf(*v*)

Return Node object that is a leaf corresponding to the vertex v.

INPUT:

- vertex – vertex number

OUTPUT: a node object representing the vertex with node_type set as NodeType.NORMAL

EXAMPLES:

```
sage: from sage.graphs.graph_decompositions.modular_decomposition import Node
sage: node = Node.create_leaf(2)
sage: node
NORMAL [2]
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_decompositions.modular_decomposition import Node
>>> node = Node.create_leaf(Integer(2))
>>> node
NORMAL [2]
```

is_empty()

Check whether self is an empty node.

EXAMPLES:

```
sage: from sage.graphs.graph_decompositions.modular_decomposition import *
sage: Node(NodeType.EMPTY).is_empty()
True
sage: Node.create_leaf(1).is_empty()
False
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_decompositions.modular_decomposition import *
>>> Node(NodeType.EMPTY).is_empty()
```

(continues on next page)

(continued from previous page)

```
True
>>> Node.create_leaf(Integer(1)).is_empty()
False
```

is_leaf()

Check whether `self` is a leaf.

EXAMPLES:

```
sage: from sage.graphs.graph_decompositions.modular_decomposition import *
sage: n = Node(NodeType.PRIME)
sage: n.children.append(Node.create_leaf(1))
sage: n.children.append(Node.create_leaf(2))
sage: n.is_leaf()
False
sage: all(c.is_leaf() for c in n.children)
True
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_decompositions.modular_decomposition import *
>>> n = Node(NodeType.PRIME)
>>> n.children.append(Node.create_leaf(Integer(1)))
>>> n.children.append(Node.create_leaf(Integer(2)))
>>> n.is_leaf()
False
>>> all(c.is_leaf() for c in n.children)
True
```

is_prime()

Check whether `self` is a prime node.

EXAMPLES:

```
sage: from sage.graphs.graph_decompositions.modular_decomposition import *
sage: n = Node(NodeType.PRIME)
sage: n.children.append(Node.create_leaf(1))
sage: n.children.append(Node.create_leaf(2))
sage: n.is_prime()
True
sage: (n.children[0].is_prime(), n.children[1].is_prime())
(False, False)
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_decompositions.modular_decomposition import *
>>> n = Node(NodeType.PRIME)
>>> n.children.append(Node.create_leaf(Integer(1)))
>>> n.children.append(Node.create_leaf(Integer(2)))
>>> n.is_prime()
True
>>> (n.children[Integer(0)].is_prime(), n.children[Integer(1)].is_prime())
(False, False)
```

is_series()

Check whether `self` is a series node.

EXAMPLES:

```
sage: from sage.graphs.graph_decompositions.modular_decomposition import *
sage: n = Node(NodeType.SERIES)
sage: n.children.append(Node.create_leaf(1))
sage: n.children.append(Node.create_leaf(2))
sage: n.is_series()
True
sage: (n.children[0].is_series(), n.children[1].is_series())
(False, False)
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_decompositions.modular_decomposition import *
>>> n = Node(NodeType.SERIES)
>>> n.children.append(Node.create_leaf(Integer(1)))
>>> n.children.append(Node.create_leaf(Integer(2)))
>>> n.is_series()
True
>>> (n.children[Integer(0)].is_series(), n.children[Integer(1)].is_series())
(False, False)
```

class sage.graphs.graph_decompositions.modular_decomposition.**NodeType**(**values*)

Bases: IntEnum

NodeType is an enumeration class used to define the various types of nodes in modular decomposition tree.

The various node types defined are

- `PARALLEL` – indicates the node is a parallel module
- `SERIES` – indicates the node is a series module
- `PRIME` – indicates the node is a prime module
- `EMPTY` – indicates a empty tree
- `NORMAL` – indicates the node is normal containing a vertex

EMPTY = -1

NORMAL = 3

PARALLEL = 2

PRIME = 0

SERIES = 1

sage.graphs.graph_decompositions.modular_decomposition.**check_algos_are_equivalent**(*trials*, *graph_gen*, *verbose=False*)

Verify that both algorithms compute the same tree (up to equivalence) for random graphs.

INPUT:

- `trials` – integer; the number of tests the function will run.

- `graph_gen` – function; a function that can be called without argument and returns a random graph.
- `verbose` – boolean (default: `False`); enable printing debug information.

OUTPUT: `None`. Raises an `AssertionError` on failure.

EXAMPLES:

```
sage: from sage.graphs.graph_decompositions.modular_decomposition import *
sage: check_algos_are_equivalent(3, lambda : graphs.RandomGNP(10, 0.1))
sage: check_algos_are_equivalent(3, lambda : graphs.RandomGNP(10, 0.5))
sage: check_algos_are_equivalent(3, lambda : graphs.RandomGNP(10, 0.9))
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_decompositions.modular_decomposition import *
>>> check_algos_are_equivalent(Integer(3), lambda : graphs.RandomGNP(Integer(10),␣
↪RealNumber('0.1')))
>>> check_algos_are_equivalent(Integer(3), lambda : graphs.RandomGNP(Integer(10),␣
↪RealNumber('0.5')))
>>> check_algos_are_equivalent(Integer(3), lambda : graphs.RandomGNP(Integer(10),␣
↪RealNumber('0.9')))
```

sage.graphs.graph_decompositions.modular_decomposition.**check_gamma_modules**(*trials*, *vertices*, *prob*, *verbose=False*)

Verify that the vertices of each gamma class of a random graph are modules of that graph.

INPUT:

- `trials` – the number of trials to run
- `vertices` – the size of the graph to use
- `prob` – the probability that any given edge is in the graph See *RandomGNP()* for more details
- `verbose` – print information on each trial

EXAMPLES:

```
sage: from sage.graphs.graph_decompositions.modular_decomposition import *
sage: check_gamma_modules(3, 7, 0.5)
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_decompositions.modular_decomposition import *
>>> check_gamma_modules(Integer(3), Integer(7), RealNumber('0.5'))
```

sage.graphs.graph_decompositions.modular_decomposition.**check_maximal_modules**(*tree_root*, *graph*)

Test the maximal nature of modules in a modular decomposition tree.

Suppose the module $M = [M_1, M_2, \cdots, n]$ is the input modular decomposition tree. Algorithm forms pairs like $(M_1, M_2), (M_1, M_3), \cdots, (M_1, M_n); (M_2, M_3), (M_2, M_4), \cdots, (M_2, M_n); \cdots$ and so on and tries to form a module using the pair. If the module formed has same type as M and is of type `SERIES` or `PARALLEL` then the formed module is not considered maximal. Otherwise it is considered maximal and M is not a modular decomposition tree.

INPUT:

- `tree_root` – modular decomposition tree whose modules are tested for maximal nature

- `graph` – graph whose modular decomposition tree is tested

OUTPUT:

`True` if all modules at first level in the modular decomposition tree are maximal in nature

EXAMPLES:

```
sage: from sage.graphs.graph_decompositions.modular_decomposition import *
sage: g = graphs.HexahedralGraph()
sage: check_maximal_modules(modular_decomposition(g), g)
True
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_decompositions.modular_decomposition import *
>>> g = graphs.HexahedralGraph()
>>> check_maximal_modules(modular_decomposition(g), g)
True
```

sage.graphs.graph_decompositions.modular_decomposition.**check_modular_decomposition**(*tree_root*, *graph*)

Test the input modular decomposition tree using recursion.

INPUT:

- `tree_root` – root of the modular decomposition tree to be tested
- `graph` – graph whose modular decomposition tree needs to be tested

OUTPUT: `True` if input tree is a modular decomposition else `False`

EXAMPLES:

```
sage: from sage.graphs.graph_decompositions.modular_decomposition import *
sage: g = graphs.HexahedralGraph()
sage: check_modular_decomposition(modular_decomposition(g), g)
True
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_decompositions.modular_decomposition import *
>>> g = graphs.HexahedralGraph()
>>> check_modular_decomposition(modular_decomposition(g), g)
True
```

sage.graphs.graph_decompositions.modular_decomposition.**check_module**(*module*, *graph*)

Test whether input module is actually a module.

INPUT:

- `module` – module which needs to be tested
- `graph` – input sage graph which contains the module

OUTPUT: `True` if input module is a module by definition else `False`

EXAMPLES:

```
sage: from sage.graphs.graph_decompositions.modular_decomposition import *
sage: g = graphs.HexahedralGraph()
```

(continues on next page)

(continued from previous page)

```
sage: tree_root = modular_decomposition(g)
sage: check_module(tree_root, g)
True
sage: check_module(tree_root.children[0], g)
True
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_decompositions.modular_decomposition import *
>>> g = graphs.HexahedralGraph()
>>> tree_root = modular_decomposition(g)
>>> check_module(tree_root, g)
True
>>> check_module(tree_root.children[Integer(0)], g)
True
```

sage.graphs.graph_decompositions.modular_decomposition.**children_node_type**(*module*, *node_type*)

Check whether the node type of the children of `module` is `node_type`.

INPUT:

- `module` – module which is tested
- `node_type` – input node_type

EXAMPLES:

```
sage: from sage.graphs.graph_decompositions.modular_decomposition import *
sage: g = graphs.OctahedralGraph()
sage: tree_root = modular_decomposition(g)
sage: print_md_tree(tree_root)
SERIES
 PARALLEL
  2
  3
 PARALLEL
  1
  4
 PARALLEL
  0
  5
sage: children_node_type(tree_root, NodeType.SERIES)
False
sage: children_node_type(tree_root, NodeType.PARALLEL)
True
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_decompositions.modular_decomposition import *
>>> g = graphs.OctahedralGraph()
>>> tree_root = modular_decomposition(g)
>>> print_md_tree(tree_root)
SERIES
 PARALLEL
```

(continues on next page)

(continued from previous page)

```
  2
  3
 PARALLEL
  1
  4
 PARALLEL
  0
  5
>>> children_node_type(tree_root, NodeType.SERIES)
False
>>> children_node_type(tree_root, NodeType.PARALLEL)
True
```

sage.graphs.graph_decompositions.modular_decomposition.**corneil_habib_paul_tedder_algorithm**(*G*)

Compute the modular decomposition by the algorithm of Corneil, Habib, Paul and Tedder.

INPUT:

- `G` – the graph for which modular decomposition tree needs to be computed

OUTPUT: an object of type Node representing the modular decomposition tree of the graph G

This function computes the modular decomposition of the given graph by the algorithm of Corneil, Habib, Paul and Tedder [TCHP2008]. It is a recursive, linear-time algorithm that first computes the slice decomposition of the graph (via the extended lexBFS algorithm) and then computes the modular decomposition by calling itself recursively on the slices of the previously computed slice decomposition.

This functions is based on the last version of the paper [TCHP2008]. Previous versions of the paper and previous implementations were found to contains errors, see [AP2024].

See also

- *`slice_decomposition`* – compute a slice decomposition of the simple undirect graph

This function should not be used directly, it should be called via the `modular_decomposition` method of `Graph` with the parameter `algorithm='corneil_habib_paul_tedder'`.

This functions assumes that `graph` is a object of the class `Graph` and is a simple graph.

sage.graphs.graph_decompositions.modular_decomposition.**either_connected_or_not_connected**(*v*, *vertices_in_module*, *graph*)

Check whether `v` is connected or disconnected to all vertices in the module.

INPUT:

- `v` – vertex tested
- `vertices_in_module` – list containing vertices in the module
- `graph` – graph to which the vertices belong

EXAMPLES:

```
sage: from sage.graphs.graph_decompositions.modular_decomposition import *
sage: g = graphs.OctahedralGraph()
sage: print_md_tree(modular_decomposition(g))
SERIES
 PARALLEL
  2
  3
 PARALLEL
  1
  4
 PARALLEL
  0
  5
sage: either_connected_or_not_connected(2, [1, 4], g)
True
sage: either_connected_or_not_connected(2, [3, 4], g)
False
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_decompositions.modular_decomposition import *
>>> g = graphs.OctahedralGraph()
>>> print_md_tree(modular_decomposition(g))
SERIES
 PARALLEL
  2
  3
 PARALLEL
  1
  4
 PARALLEL
  0
  5
>>> either_connected_or_not_connected(Integer(2), [Integer(1), Integer(4)], g)
True
>>> either_connected_or_not_connected(Integer(2), [Integer(3), Integer(4)], g)
False
```

sage.graphs.graph_decompositions.modular_decomposition.**equivalent_trees**(*root1*, *root2*)

Check that two modular decomposition trees are the same.

Verify that the structure of the trees is the same. Two leaves are equivalent if they represent the same vertex, two internal nodes are equivalent if they have the same nodes type and the same number of children and there is a matching between the children such that each pair of children is a pair of equivalent subtrees.

EXAMPLES:

```
sage: from sage.graphs.graph_decompositions.modular_decomposition import *
sage: t1 = nested_tuple_to_tree((NodeType.SERIES, 1, 2,
....:              (NodeType.PARALLEL, 3, 4)))
sage: t2 = nested_tuple_to_tree((NodeType.SERIES,
....:              (NodeType.PARALLEL, 4, 3), 2, 1))
sage: equivalent_trees(t1, t2)
True
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_decompositions.modular_decomposition import *
>>> t1 = nested_tuple_to_tree((NodeType.SERIES, Integer(1), Integer(2),
...              (NodeType.PARALLEL, Integer(3), Integer(4))))
>>> t2 = nested_tuple_to_tree((NodeType.SERIES,
...              (NodeType.PARALLEL, Integer(4), Integer(3)), Integer(2),␣
↪Integer(1)))
>>> equivalent_trees(t1, t2)
True
```

sage.graphs.graph_decompositions.modular_decomposition.**form_module**(*index*, *other_index*, *tree_root*, *graph*)

Forms a module out of the modules in the module pair.

Let M_1 and M_2 be the input modules. Let V be the set of vertices in these modules. Suppose x is a neighbor of subset of the vertices in V but not all the vertices and x does not belong to V. Then the set of modules also include the module which contains x. This process is repeated until a module is formed and the formed module if subset of V is returned.

INPUT:

- `index` – first module in the module pair
- `other_index` – second module in the module pair
- `tree_root` – modular decomposition tree which contains the modules in the module pair
- `graph` – graph whose modular decomposition tree is created

OUTPUT:

`[module_formed, vertices]` where `module_formed` is `True` if module is formed else `False` and `vertices` is a list of vertices included in the formed module

EXAMPLES:

```
sage: from sage.graphs.graph_decompositions.modular_decomposition import *
sage: g = graphs.HexahedralGraph()
sage: tree_root = modular_decomposition(g)
sage: form_module(0, 2, tree_root, g)
[False, {0, 1, 2, 3, 4, 5, 6, 7}]
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_decompositions.modular_decomposition import *
>>> g = graphs.HexahedralGraph()
>>> tree_root = modular_decomposition(g)
>>> form_module(Integer(0), Integer(2), tree_root, g)
[False, {0, 1, 2, 3, 4, 5, 6, 7}]
```

sage.graphs.graph_decompositions.modular_decomposition.**gamma_classes**(*graph*)

Partition the edges of the graph into Gamma classes.

Two distinct edges are Gamma related if they share a vertex but are not part of a triangle. A Gamma class of edges is a collection of edges such that any edge in the class can be reached from any other by a chain of Gamma related edges (that are also in the class).

The two important properties of the Gamma class

- The vertex set corresponding to a Gamma class is a module

- If the graph is not fragile (neither it or its complement is disconnected) then there is exactly one class that visits all the vertices of the graph, and this class consists of just the edges that connect the maximal strong modules of that graph.

EXAMPLES:

The gamma_classes of the octahedral graph are the three 4-cycles corresponding to the slices through the center of the octahedron:

```
sage: from sage.graphs.graph_decompositions.modular_decomposition import gamma_
↪classes
sage: g = graphs.OctahedralGraph()
sage: sorted(gamma_classes(g), key=str)
[frozenset({0, 1, 4, 5}), frozenset({0, 2, 3, 5}), frozenset({1, 2, 3, 4})]
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_decompositions.modular_decomposition import gamma_
↪classes
>>> g = graphs.OctahedralGraph()
>>> sorted(gamma_classes(g), key=str)
[frozenset({0, 1, 4, 5}), frozenset({0, 2, 3, 5}), frozenset({1, 2, 3, 4})]
```

sage.graphs.graph_decompositions.modular_decomposition.**get_module_type**(*graph*)

Return the module type of the root of the modular decomposition tree of `graph`.

INPUT:

- `graph` – input sage graph

OUTPUT:

`PRIME` if graph is PRIME, `PARALLEL` if graph is PARALLEL and `SERIES` if graph is of type SERIES

EXAMPLES:

```
sage: from sage.graphs.graph_decompositions.modular_decomposition import get_
↪module_type
sage: g = graphs.HexahedralGraph()
sage: get_module_type(g)
PRIME
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_decompositions.modular_decomposition import get_module_
↪type
>>> g = graphs.HexahedralGraph()
>>> get_module_type(g)
PRIME
```

sage.graphs.graph_decompositions.modular_decomposition.**get_vertices**(*component_root*)

Compute the list of vertices in the (co)component.

INPUT:

- `component_root` – root of the (co)component whose vertices need to be returned as a list

OUTPUT:

list of vertices in the (co)component

EXAMPLES:

```
sage: from sage.graphs.graph_decompositions.modular_decomposition import *
sage: forest = Node(NodeType.PRIME)
sage: forest.children = [Node.create_leaf(2), Node.create_leaf(0),
....:                    Node.create_leaf(3), Node.create_leaf(1)]
sage: series_node = Node(NodeType.SERIES)
sage: series_node.children = [Node.create_leaf(4),
....:                         Node.create_leaf(5)]
sage: parallel_node = Node(NodeType.PARALLEL)
sage: parallel_node.children = [Node.create_leaf(6),
....:                           Node.create_leaf(7)]
sage: forest.children.insert(1, series_node)
sage: forest.children.insert(3, parallel_node)
sage: get_vertices(forest)
[2, 4, 5, 0, 6, 7, 3, 1]
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_decompositions.modular_decomposition import *
>>> forest = Node(NodeType.PRIME)
>>> forest.children = [Node.create_leaf(Integer(2)), Node.create_leaf(Integer(0)),
...                    Node.create_leaf(Integer(3)), Node.create_leaf(Integer(1))]
>>> series_node = Node(NodeType.SERIES)
>>> series_node.children = [Node.create_leaf(Integer(4)),
...                         Node.create_leaf(Integer(5))]
>>> parallel_node = Node(NodeType.PARALLEL)
>>> parallel_node.children = [Node.create_leaf(Integer(6)),
...                           Node.create_leaf(Integer(7))]
>>> forest.children.insert(Integer(1), series_node)
>>> forest.children.insert(Integer(3), parallel_node)
>>> get_vertices(forest)
[2, 4, 5, 0, 6, 7, 3, 1]
```

sage.graphs.graph_decompositions.modular_decomposition.**habib_maurer_algorithm**(*graph*, *g_classes=None*)

Compute the modular decomposition by the algorithm of Habib and Maurer.

Compute the modular decomposition of the given graph by the algorithm of Habib and Maurer [HM1979] . If the graph is disconnected or its complement is disconnected return a tree with a `PARALLEL` or `SERIES` node at the root and children being the modular decomposition of the subgraphs induced by the components. Otherwise, the root is `PRIME` and the modules are identified by having identical neighborhoods in the gamma class that spans the vertices of the subgraph (exactly one is guaranteed to exist). The gamma classes only need to be computed once, as the algorithm computes the the classes for the current root and each of the submodules. See also [BM1983] for an equivalent algorithm described in greater detail.

This function should not be used directly, it should be called via the `modular_decomposition` method of `Graph` with the parameter `algorithm='habib_maurer'`.

This functions assumes that `graph` is a object of the class `Graph`, is a simple graph and has at least 1 vertex.

INPUT:

- `graph` – the graph for which modular decomposition tree needs to be computed
- `g_classes` – dictionary (default: `None`); a dictionary whose values are the gamma classes of the graph, and whose keys are a frozenset of the vertices corresponding to the class. Used internally.

OUTPUT: the modular decomposition tree of the graph

EXAMPLES:

The Icosahedral graph is Prime:

```
sage: from sage.graphs.graph_decompositions.modular_decomposition import *
sage: print_md_tree(habib_maurer_algorithm(graphs.IcosahedralGraph()))
PRIME
 3
 4
 7
 9
 11
 1
 5
 8
 0
 2
 6
 10
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_decompositions.modular_decomposition import *
>>> print_md_tree(habib_maurer_algorithm(graphs.IcosahedralGraph()))
PRIME
 3
 4
 7
 9
 11
 1
 5
 8
 0
 2
 6
 10
```

The Octahedral graph is not Prime:

```
sage: print_md_tree(habib_maurer_algorithm(graphs.OctahedralGraph()))
SERIES
 PARALLEL
  0
  5
 PARALLEL
  1
  4
 PARALLEL
  2
  3
```

```
>>> from sage.all import *
>>> print_md_tree(habib_maurer_algorithm(graphs.OctahedralGraph()))
SERIES
```

(continues on next page)

(continued from previous page)

```
 PARALLEL
  0
  5
 PARALLEL
  1
  4
 PARALLEL
  2
  3
```

Tetrahedral Graph is Series:

```
sage: print_md_tree(habib_maurer_algorithm(graphs.TetrahedralGraph()))
SERIES
 0
 1
 2
 3
```

```
>>> from sage.all import *
>>> print_md_tree(habib_maurer_algorithm(graphs.TetrahedralGraph()))
SERIES
 0
 1
 2
 3
```

Modular Decomposition tree containing both parallel and series modules:

```
sage: d = {2:[4,3,5], 1:[4,3,5], 5:[3,2,1,4], 3:[1,2,5], 4:[1,2,5]}
sage: g = Graph(d)
sage: print_md_tree(habib_maurer_algorithm(g))
SERIES
 PARALLEL
  1
  2
 PARALLEL
  3
  4
 5
```

```
>>> from sage.all import *
>>> d = {Integer(2):[Integer(4),Integer(3),Integer(5)], Integer(1):[Integer(4),
↪Integer(3),Integer(5)], Integer(5):[Integer(3),Integer(2),Integer(1),
↪Integer(4)], Integer(3):[Integer(1),Integer(2),Integer(5)],␣
↪Integer(4):[Integer(1),Integer(2),Integer(5)]}
>>> g = Graph(d)
>>> print_md_tree(habib_maurer_algorithm(g))
SERIES
 PARALLEL
  1
  2
```

(continues on next page)

(continued from previous page)

```
 PARALLEL
  3
  4
 5
```

Graph from Marc Tedder implementation of modular decomposition:

```
sage: d = {1:[5,4,3,24,6,7,8,9,2,10,11,12,13,14,16,17], 2:[1],
....:       3:[24,9,1], 4:[5,24,9,1], 5:[4,24,9,1], 6:[7,8,9,1],
....:       7:[6,8,9,1], 8:[6,7,9,1], 9:[6,7,8,5,4,3,1], 10:[1],
....:       11:[12,1], 12:[11,1], 13:[14,16,17,1], 14:[13,17,1],
....:       16:[13,17,1], 17:[13,14,16,18,1], 18:[17], 24:[5,4,3,1]}
sage: g = Graph(d)
sage: check_modular_decomposition(habib_maurer_algorithm(g), g)
True
```

```
>>> from sage.all import *
>>> d = {Integer(1):[Integer(5),Integer(4),Integer(3),Integer(24),Integer(6),
↪Integer(7),Integer(8),Integer(9),Integer(2),Integer(10),Integer(11),Integer(12),
↪Integer(13),Integer(14),Integer(16),Integer(17)], Integer(2):[Integer(1)],
...       Integer(3):[Integer(24),Integer(9),Integer(1)], Integer(4):[Integer(5),
↪Integer(24),Integer(9),Integer(1)], Integer(5):[Integer(4),Integer(24),
↪Integer(9),Integer(1)], Integer(6):[Integer(7),Integer(8),Integer(9),
↪Integer(1)],
...       Integer(7):[Integer(6),Integer(8),Integer(9),Integer(1)],␣
↪Integer(8):[Integer(6),Integer(7),Integer(9),Integer(1)],␣
↪Integer(9):[Integer(6),Integer(7),Integer(8),Integer(5),Integer(4),Integer(3),
↪Integer(1)], Integer(10):[Integer(1)],
...       Integer(11):[Integer(12),Integer(1)], Integer(12):[Integer(11),
↪Integer(1)], Integer(13):[Integer(14),Integer(16),Integer(17),Integer(1)],␣
↪Integer(14):[Integer(13),Integer(17),Integer(1)],
...       Integer(16):[Integer(13),Integer(17),Integer(1)],␣
↪Integer(17):[Integer(13),Integer(14),Integer(16),Integer(18),Integer(1)],␣
↪Integer(18):[Integer(17)], Integer(24):[Integer(5),Integer(4),Integer(3),
↪Integer(1)]}
>>> g = Graph(d)
>>> check_modular_decomposition(habib_maurer_algorithm(g), g)
True
```

Tetrahedral Graph is Series:

```
sage: print_md_tree(habib_maurer_algorithm(graphs.TetrahedralGraph()))
SERIES
 0
 1
 2
 3
```

```
>>> from sage.all import *
>>> print_md_tree(habib_maurer_algorithm(graphs.TetrahedralGraph()))
SERIES
 0
```

(continues on next page)

(continued from previous page)

```
 1
 2
 3
```

Modular Decomposition tree containing both parallel and series modules:

```
sage: d = {2:[4,3,5], 1:[4,3,5], 5:[3,2,1,4], 3:[1,2,5], 4:[1,2,5]}
sage: g = Graph(d)
sage: print_md_tree(habib_maurer_algorithm(g))
SERIES
 PARALLEL
  1
  2
 PARALLEL
  3
  4
 5
```

```
>>> from sage.all import *
>>> d = {Integer(2):[Integer(4),Integer(3),Integer(5)], Integer(1):[Integer(4),
↪Integer(3),Integer(5)], Integer(5):[Integer(3),Integer(2),Integer(1),
↪Integer(4)], Integer(3):[Integer(1),Integer(2),Integer(5)],␣
↪Integer(4):[Integer(1),Integer(2),Integer(5)]}
>>> g = Graph(d)
>>> print_md_tree(habib_maurer_algorithm(g))
SERIES
 PARALLEL
  1
  2
 PARALLEL
  3
  4
 5
```

sage.graphs.graph_decompositions.modular_decomposition.**md_tree_to_graph**(*root*, *prime_node_generator=None*)

Create a graph having the given MD tree.

For the prime nodes, the parameter `prime_node_generator` is called with the number of vertices as the only argument. If it is `None`, the path graph is used (it is prime when the length is 4 or more).

EXAMPLES:

```
sage: from sage.graphs.graph_decompositions.modular_decomposition import *
sage: from sage.graphs.graph_generators import graphs
sage: tup1 = (NodeType.PRIME, 1, (NodeType.SERIES, 2, 3),
....:         (NodeType.PARALLEL, 4, 5), 6)
sage: tree1 = nested_tuple_to_tree(tup1)
sage: g1 = md_tree_to_graph(tree1)
sage: g2 = Graph({1: [2, 3], 2: [1, 3, 4, 5], 3: [1, 2, 4, 5],
....:             4: [2, 3, 6], 5: [2, 3, 6], 6: [4, 5]})
sage: g1.is_isomorphic(g2)
```

(continues on next page)

(continued from previous page)

```
True

sage: G = md_tree_to_graph(Node(NodeType.EMPTY))
sage: G.is_isomorphic(Graph())
True

sage: tree = Node(NodeType.SERIES)
sage: tree.children.extend(Node.create_leaf(i) for i in range(5))
sage: G = md_tree_to_graph(tree)
sage: G.is_isomorphic(graphs.CompleteGraph(5))
True

sage: tree = Node(NodeType.PRIME)
sage: tree.children.extend(Node.create_leaf(i) for i in range(5))
sage: png = lambda n: (graphs.PathGraph if n == 4 else graphs.CycleGraph)(n)
sage: G = md_tree_to_graph(tree, prime_node_generator=png)
sage: G.is_isomorphic(graphs.CycleGraph(5))
True
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_decompositions.modular_decomposition import *
>>> from sage.graphs.graph_generators import graphs
>>> tup1 = (NodeType.PRIME, Integer(1), (NodeType.SERIES, Integer(2), Integer(3)),
...         (NodeType.PARALLEL, Integer(4), Integer(5)), Integer(6))
>>> tree1 = nested_tuple_to_tree(tup1)
>>> g1 = md_tree_to_graph(tree1)
>>> g2 = Graph({Integer(1): [Integer(2), Integer(3)], Integer(2): [Integer(1),␣
↪Integer(3), Integer(4), Integer(5)], Integer(3): [Integer(1), Integer(2),␣
↪Integer(4), Integer(5)],
...             Integer(4): [Integer(2), Integer(3), Integer(6)], Integer(5):␣
↪[Integer(2), Integer(3), Integer(6)], Integer(6): [Integer(4), Integer(5)]})
>>> g1.is_isomorphic(g2)
True

>>> G = md_tree_to_graph(Node(NodeType.EMPTY))
>>> G.is_isomorphic(Graph())
True

>>> tree = Node(NodeType.SERIES)
>>> tree.children.extend(Node.create_leaf(i) for i in range(Integer(5)))
>>> G = md_tree_to_graph(tree)
>>> G.is_isomorphic(graphs.CompleteGraph(Integer(5)))
True

>>> tree = Node(NodeType.PRIME)
>>> tree.children.extend(Node.create_leaf(i) for i in range(Integer(5)))
>>> png = lambda n: (graphs.PathGraph if n == Integer(4) else graphs.
↪CycleGraph)(n)
>>> G = md_tree_to_graph(tree, prime_node_generator=png)
>>> G.is_isomorphic(graphs.CycleGraph(Integer(5)))
True
```

sage.graphs.graph_decompositions.modular_decomposition.**modular_decomposition**(*G*, *algorithm=None*)

Return the modular decomposition of the current graph.

This function should not be used directly, it should be called via the `modular_decomposition` method of `Graph`.

INPUT:

- `G` – graph whose modular decomposition tree is to be computed
- `algorithm` – string (default: `None`); the algorithm to use among:
  - `None` or `'corneil_habib_paul_tedder'` – will use the Corneil-Habib-Paul-Tedder algorithm from [TCHP2008], its complexity is linear in the number of vertices and edges.
  - `'habib_maurer'` – will use the Habib-Maurer algorithm from [HM1979], its complexity is cubic in the number of vertices.

OUTPUT: The modular decomposition tree, as an object of type `Node`.

sage.graphs.graph_decompositions.modular_decomposition.**nested_tuple_to_tree**(*nest*)

Turn a tuple representing the modular decomposition into a tree.

INPUT:

- `nest` – a nested tuple of the form returned by *tree_to_nested_tuple()*

OUTPUT: the root node of a modular decomposition tree

EXAMPLES:

```
sage: from sage.graphs.graph_decompositions.modular_decomposition import *
sage: tree = (NodeType.SERIES, 1, 2, (NodeType.PARALLEL, 3, 4))
sage: print_md_tree(nested_tuple_to_tree(tree))
SERIES
 1
 2
 PARALLEL
  3
  4
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_decompositions.modular_decomposition import *
>>> tree = (NodeType.SERIES, Integer(1), Integer(2), (NodeType.PARALLEL,␣
↪Integer(3), Integer(4)))
>>> print_md_tree(nested_tuple_to_tree(tree))
SERIES
 1
 2
 PARALLEL
  3
  4
```

sage.graphs.graph_decompositions.modular_decomposition.**permute_decomposition**(*trials*, *algorithm*, *vertices*, *prob*, *verbose=False*)

Check that a graph and its permuted relabeling have the same modular decomposition.

We generate a `trials` random graphs and then generate an isomorphic graph by relabeling the original graph. We then verify

EXAMPLES:

```
sage: from sage.graphs.graph_decompositions.modular_decomposition import *
sage: permute_decomposition(30, habib_maurer_algorithm, 10, 0.5)
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_decompositions.modular_decomposition import *
>>> permute_decomposition(Integer(30), habib_maurer_algorithm, Integer(10),␣
↪RealNumber('0.5'))
```

sage.graphs.graph_decompositions.modular_decomposition.**print_md_tree**(*root*)

Print the modular decomposition tree.

INPUT:

- `root` – root of the modular decomposition tree

EXAMPLES:

```
sage: from sage.graphs.graph_decompositions.modular_decomposition import *
sage: print_md_tree(habib_maurer_algorithm(graphs.IcosahedralGraph()))
PRIME
 3
 4
 7
 9
 11
 1
 5
 8
 0
 2
 6
 10
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_decompositions.modular_decomposition import *
>>> print_md_tree(habib_maurer_algorithm(graphs.IcosahedralGraph()))
PRIME
 3
 4
 7
 9
 11
 1
 5
 8
 0
 2
 6
 10
```

sage.graphs.graph_decompositions.modular_decomposition.**random_md_tree**(*max_depth*, *max_fan_out*, *leaf_probability*)

Create a random MD tree.

INPUT:

- `max_depth` – the maximum depth of the tree
- `max_fan_out` – the maximum number of children a node can have (must be >=4 as a prime node must have at least 4 vertices)
- `leaf_probability` – the probability that a subtree is a leaf

EXAMPLES:

```
sage: from sage.graphs.graph_decompositions.modular_decomposition import *
sage: set_random_seed(0)
sage: tree_to_nested_tuple(random_md_tree(2, 5, 0.5))
(PRIME, [0, (PRIME, [1, 2, 3, 4]), 5, 6])
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_decompositions.modular_decomposition import *
>>> set_random_seed(Integer(0))
>>> tree_to_nested_tuple(random_md_tree(Integer(2), Integer(5), RealNumber('0.5
↪')))
(PRIME, [0, (PRIME, [1, 2, 3, 4]), 5, 6])
```

sage.graphs.graph_decompositions.modular_decomposition.**recreate_decomposition**(*trials*, *algorithm*, *max_depth*, *max_fan_out*, *leaf_probability*, *verbose=False*)

Verify that we can recreate a random MD tree.

We create a random MD tree, then create a graph having that decomposition, then find a modular decomposition for that graph, and verify that the two modular decomposition trees are equivalent.

EXAMPLES:

```
sage: from sage.graphs.graph_decompositions.modular_decomposition import *
sage: recreate_decomposition(3, habib_maurer_algorithm, 4, 6, 0.5,
....:                        verbose=False)
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_decompositions.modular_decomposition import *
>>> recreate_decomposition(Integer(3), habib_maurer_algorithm, Integer(4),␣
↪Integer(6), RealNumber('0.5'),
...                        verbose=False)
```

sage.graphs.graph_decompositions.modular_decomposition.**relabel_tree**(*root*, *perm*)

Relabel the leaves of a tree according to a dictionary.

INPUT:

- `root` – the root of the tree
- `perm` – a function, dictionary, list, permutation, or `None` representing the relabeling. See *relabel()* for description of the permutation input.

EXAMPLES:

```
sage: from sage.graphs.graph_decompositions.modular_decomposition import *
sage: tuple_tree = (NodeType.SERIES, 1, 2, (NodeType.PARALLEL, 3, 4))
sage: tree = nested_tuple_to_tree(tuple_tree)
sage: print_md_tree(relabel_tree(tree, (4,3,2,1)))
SERIES
 4
 3
 PARALLEL
  2
  1
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_decompositions.modular_decomposition import *
>>> tuple_tree = (NodeType.SERIES, Integer(1), Integer(2), (NodeType.PARALLEL,␣
↪Integer(3), Integer(4)))
>>> tree = nested_tuple_to_tree(tuple_tree)
>>> print_md_tree(relabel_tree(tree, (Integer(4),Integer(3),Integer(2),
↪Integer(1))))
SERIES
 4
 3
 PARALLEL
  2
  1
```

sage.graphs.graph_decompositions.modular_decomposition.**tree_to_nested_tuple**(*root*)

Convert a modular decomposition tree to a nested tuple.

INPUT:

- `root` – the root of the modular decomposition tree

OUTPUT:

A tuple whose first element is the type of the root of the tree and whose subsequent nodes are either vertex labels in the case of leaves or tuples representing the child subtrees.

EXAMPLES:

```
sage: from sage.graphs.graph_decompositions.modular_decomposition import *
sage: g = graphs.OctahedralGraph()
sage: tree_to_nested_tuple(modular_decomposition(g))
(SERIES, [(PARALLEL, [2, 3]), (PARALLEL, [1, 4]), (PARALLEL, [0, 5])])
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_decompositions.modular_decomposition import *
>>> g = graphs.OctahedralGraph()
>>> tree_to_nested_tuple(modular_decomposition(g))
(SERIES, [(PARALLEL, [2, 3]), (PARALLEL, [1, 4]), (PARALLEL, [0, 5])])
```

5.32 Decomposition by clique minimal separators

This module implements methods related to the decomposition of a graph by clique minimal separators. See [TY1984] and [BPS2010] for more details on the algorithms.

5.32.1 Methods

sage.graphs.graph_decompositions.clique_separators.**atoms_and_clique_separators**(*G*, *tree=False*, *rooted_tree=False*, *separators=False*)

Return the atoms of the decomposition of G by clique minimal separators.

Let $G = (V, E)$ be a graph. A set $S \subset V$ is a clique separator if $G[S]$ is a clique and the graph $G \setminus S$ has at least 2 connected components. Let $C \subset V$ be the vertices of a connected component of $G \setminus S$. The graph $G[C + S]$ is an *atom* if it has no clique separator.

This method implements the algorithm proposed in [BPS2010], that improves upon the algorithm proposed in [TY1984], for computing the atoms and the clique minimal separators of a graph. This algorithm is based on the *maximum_cardinality_search_M()* graph traversal and has time complexity in $O(|V| \cdot |E|)$.

If the graph is not connected, we insert empty separators between the lists of separators of each connected components. See the examples below for more details.

INPUT:

- `G` – a Sage graph
- `tree` – boolean (default: `False`); whether to return the result as a directed tree in which internal nodes are clique separators and leaves are the atoms of the decomposition. Since a clique separator is repeated when its removal partition the graph into 3 or more connected components, vertices are labels by tuples (i, S), where S is the set of vertices of the atom or the clique separator, and $0 \leq i \leq |T|$.
- `rooted_tree` – boolean (default: `False`); whether to return the result as a `LabelledRootedTree`. When `tree` is `True`, this parameter is ignored.
- `separators` – boolean (default: `False`); whether to also return the complete list of separators considered during the execution of the algorithm. When `tree` or `rooted_tree` is `True`, this parameter is ignored.

OUTPUT:

- By default, return a tuple (A, S_c), where A is the list of atoms of the graph in the order of discovery, and S_c is the list of clique separators, with possible repetitions, in the order the separator has been considered. If furthermore `separators` is `True`, return a tuple (A, S_h, S_c), where S_c is the list of considered separators of the graph in the order they have been considered.
- When `tree` is `True`, format the result as a directed tree
- When `rooted_tree` is `True` and `tree` is `False`, format the output as a `LabelledRootedTree`

EXAMPLES:

Example of [BPS2010]:

```
sage: G = Graph({'a': ['b', 'k'], 'b': ['c'], 'c': ['d', 'j', 'k'],
....:            'd': ['e', 'f', 'j', 'k'], 'e': ['g'],
....:            'f': ['g', 'j', 'k'], 'g': ['j', 'k'], 'h': ['i', 'j'],
....:            'i': ['k'], 'j': ['k']})
sage: atoms, cliques = G.atoms_and_clique_separators()
```

(continues on next page)

(continued from previous page)

```
sage: sorted(sorted(a) for a in atoms)
[['a', 'b', 'c', 'k'],
 ['c', 'd', 'j', 'k'],
 ['d', 'e', 'f', 'g', 'j', 'k'],
 ['h', 'i', 'j', 'k']]
sage: sorted(sorted(c) for c in cliques)
[['c', 'k'], ['d', 'j', 'k'], ['j', 'k']]
sage: T = G.atoms_and_clique_separators(tree=True)
sage: T.is_tree()
True
sage: T.diameter() == len(atoms)
True
sage: all(u[1] in atoms for u in T if T.degree(u) == 1)
True
sage: all(u[1] in cliques for u in T if T.degree(u) != 1)
True
```

```
>>> from sage.all import *
>>> G = Graph({'a': ['b', 'k'], 'b': ['c'], 'c': ['d', 'j', 'k'],
...            'd': ['e', 'f', 'j', 'k'], 'e': ['g'],
...            'f': ['g', 'j', 'k'], 'g': ['j', 'k'], 'h': ['i', 'j'],
...            'i': ['k'], 'j': ['k']})
>>> atoms, cliques = G.atoms_and_clique_separators()
>>> sorted(sorted(a) for a in atoms)
[['a', 'b', 'c', 'k'],
 ['c', 'd', 'j', 'k'],
 ['d', 'e', 'f', 'g', 'j', 'k'],
 ['h', 'i', 'j', 'k']]
>>> sorted(sorted(c) for c in cliques)
[['c', 'k'], ['d', 'j', 'k'], ['j', 'k']]
>>> T = G.atoms_and_clique_separators(tree=True)
>>> T.is_tree()
True
>>> T.diameter() == len(atoms)
True
>>> all(u[Integer(1)] in atoms for u in T if T.degree(u) == Integer(1))
True
>>> all(u[Integer(1)] in cliques for u in T if T.degree(u) != Integer(1))
True
```

A graph without clique separator:

```
sage: G = graphs.CompleteGraph(5)
sage: G.atoms_and_clique_separators()
([{0, 1, 2, 3, 4}], [])
sage: ascii_art(G.atoms_and_clique_separators(rooted_tree=True))
{0, 1, 2, 3, 4}
```

```
>>> from sage.all import *
>>> G = graphs.CompleteGraph(Integer(5))
>>> G.atoms_and_clique_separators()
([{0, 1, 2, 3, 4}], [])
```

(continues on next page)

(continued from previous page)

```
>>> ascii_art(G.atoms_and_clique_separators(rooted_tree=True))
{0, 1, 2, 3, 4}
```

Graphs with several biconnected components:

```
sage: G = graphs.PathGraph(4)
sage: ascii_art(G.atoms_and_clique_separators(rooted_tree=True))
  ____{2}____
 /           /
{2, 3}    __{1}__
         /       /
       {1, 2} {0, 1}

sage: G = graphs.WindmillGraph(3, 4)
sage: G.atoms_and_clique_separators()
([{0, 1, 2}, {0, 3, 4}, {0, 5, 6}, {0, 8, 7}], [{0}, {0}, {0}])
sage: ascii_art(G.atoms_and_clique_separators(rooted_tree=True))
  _________{0}_________
 /                     /
{0, 1, 2}    ________{0}______
            /                /
          {0, 3, 4}    ____{0}___
                      /          /
                    {0, 8, 7} {0, 5, 6}
```

```
>>> from sage.all import *
>>> G = graphs.PathGraph(Integer(4))
>>> ascii_art(G.atoms_and_clique_separators(rooted_tree=True))
  ____{2}____
 /           /
{2, 3}    __{1}__
         /       /
       {1, 2} {0, 1}

>>> G = graphs.WindmillGraph(Integer(3), Integer(4))
>>> G.atoms_and_clique_separators()
([{0, 1, 2}, {0, 3, 4}, {0, 5, 6}, {0, 8, 7}], [{0}, {0}, {0}])
>>> ascii_art(G.atoms_and_clique_separators(rooted_tree=True))
  _________{0}_________
 /                     /
{0, 1, 2}    ________{0}______
            /                /
          {0, 3, 4}    ____{0}___
                      /          /
                    {0, 8, 7} {0, 5, 6}
```

When the removal of a clique separator results in $k > 2$ connected components, this separator is repeated $k - 1$ times, but the repetitions are not necessarily contiguous:

```
sage: G = Graph(2)
sage: for i in range(5):
....:     G.add_cycle([0, 1, G.add_vertex()])
```

(continues on next page)

(continued from previous page)

```
sage: ascii_art(G.atoms_and_clique_separators(rooted_tree=True))
  __________{0, 1}_____
 /                    /
{0, 1, 4}   _________{0, 1}_____
           /                    /
          {0, 1, 2}   ________{0, 1}___
                     /                /
                    {0, 1, 3}   ____{0, 1}
                               /         /
                              {0, 1, 5} {0, 1, 6}

sage: G = graphs.StarGraph(3)
sage: G.subdivide_edges(G.edges(sort=False), 2)
sage: ascii_art(G.atoms_and_clique_separators(rooted_tree=True))
  ______{5}______
 /              /
{1, 5}   ______{7}______
        /               /
       {2, 7}   ______{9}______
                /               /
               {9, 3}   ______{6}______
                       /               /
                      {6, 7}   ______{4}_____
                              /              /
                             {4, 5}   _____{0}_____
                                     /             /
                                    {0, 6}   ____{8}____
                                            /          /
                                           {8, 9}   __{0}__
                                                   /      /
                                                  {0, 8} {0, 4}
```

```
>>> from sage.all import *
>>> G = Graph(Integer(2))
>>> for i in range(Integer(5)):
...     G.add_cycle([Integer(0), Integer(1), G.add_vertex()])
>>> ascii_art(G.atoms_and_clique_separators(rooted_tree=True))
  __________{0, 1}_____
 /                    /
{0, 1, 4}   _________{0, 1}_____
           /                    /
          {0, 1, 2}   ________{0, 1}___
                     /                /
                    {0, 1, 3}   ____{0, 1}
                               /         /
                              {0, 1, 5} {0, 1, 6}

>>> G = graphs.StarGraph(Integer(3))
>>> G.subdivide_edges(G.edges(sort=False), Integer(2))
>>> ascii_art(G.atoms_and_clique_separators(rooted_tree=True))
  ______{5}______
 /              /
```

(continues on next page)

(continued from previous page)

```
{1, 5}   _______{7}_______
        /                 /
       {2, 7}   _______{9}_______
               /                 /
              {9, 3}   _______{6}_______
                      /                 /
                     {6, 7}   _______{4}_____
                             /               /
                            {4, 5}   ______{0}_____
                                    /              /
                                   {0, 6}   _____{8}_____
                                           /             /
                                          {8, 9}   __{0}__
                                                  /       /
                                                 {0, 8} {0, 4}
```

If the graph is not connected, we insert empty separators between the lists of separators of each connected components. For instance, let G be a graph with 3 connected components. The method returns the list $S_c = [S_0, \cdots, S_i, \ldots, S_j, \ldots, S_{k-1}]$ of k clique separators, where i and j are the indexes of the inserted empty separators and $0 \leq i < j < k-1$. The method also returns the list $A = [A_0, \ldots, S_k]$ of the $k+1$ atoms, with $k+1 \geq 3$. The lists of atoms and clique separators of each of the connected components are respectively $[A_0, \ldots, A_i]$ and $[S_0, \ldots, S_{i-1}]$, $[A_{i+1}, \ldots, A_j]$ and $[S_{i+1}, \ldots, S_{j-1}]$, and $[A_{j+1}, \ldots, A_k]$ and $[S_{j+1}, \ldots, S_{k-1}]$. One can check that for each connected component, we get one atom more than clique separators:

```
sage: G = graphs.PathGraph(3) * 3
sage: A, Sc = G.atoms_and_clique_separators()
sage: A
[{1, 2}, {0, 1}, {4, 5}, {3, 4}, {8, 7}, {6, 7}]
sage: Sc
[{1}, {}, {4}, {}, {7}]
sage: i , j = [i for i, s in enumerate(Sc) if not s]
sage: i, j
(1, 3)
sage: A[:i+1], Sc[:i]
([{1, 2}, {0, 1}], [{1}])
sage: A[i+1:j+1], Sc[i+1:j]
([{4, 5}, {3, 4}], [{4}])
sage: A[j+1:], Sc[j+1:]
([{8, 7}, {6, 7}], [{7}])
sage: I = [-1, i, j, len(Sc)]
sage: for i, j in zip(I[:-1], I[1:]):
....:     print(A[i+1:j+1], Sc[i+1:j])
[{1, 2}, {0, 1}] [{1}]
[{4, 5}, {3, 4}] [{4}]
[{8, 7}, {6, 7}] [{7}]
sage: ascii_art(G.atoms_and_clique_separators(rooted_tree=True))
  _______{1}_______
 /                 /
{1, 2}   _______{}_______
        /                /
       {0, 1}   ______{4}_____
               /              /
```

(continues on next page)

(continued from previous page)

```
              {4, 5}    ____{}_____
                       /           /
                      {3, 4}    __{7}__
                               /       /
                              {6, 7} {8, 7}
```

```
>>> from sage.all import *
>>> G = graphs.PathGraph(Integer(3)) * Integer(3)
>>> A, Sc = G.atoms_and_clique_separators()
>>> A
[{1, 2}, {0, 1}, {4, 5}, {3, 4}, {8, 7}, {6, 7}]
>>> Sc
[{1}, {}, {4}, {}, {7}]
>>> i , j = [i for i, s in enumerate(Sc) if not s]
>>> i, j
(1, 3)
>>> A[:i+Integer(1)], Sc[:i]
([{1, 2}, {0, 1}], [{1}])
>>> A[i+Integer(1):j+Integer(1)], Sc[i+Integer(1):j]
([{4, 5}, {3, 4}], [{4}])
>>> A[j+Integer(1):], Sc[j+Integer(1):]
([{8, 7}, {6, 7}], [{7}])
>>> I = [-Integer(1), i, j, len(Sc)]
>>> for i, j in zip(I[:-Integer(1)], I[Integer(1):]):
...     print(A[i+Integer(1):j+Integer(1)], Sc[i+Integer(1):j])
[{1, 2}, {0, 1}] [{1}]
[{4, 5}, {3, 4}] [{4}]
[{8, 7}, {6, 7}] [{7}]
>>> ascii_art(G.atoms_and_clique_separators(rooted_tree=True))
  ______{1}______
 /               /
{1, 2}    ______{}______
         /             /
       {0, 1}    _____{4}_____
                /             /
              {4, 5}    ____{}_____
                       /           /
                     {3, 4}    __{7}__
                              /       /
                             {6, 7} {8, 7}
```

Loops and multiple edges are ignored:

```
sage: G.allow_loops(True)
sage: G.add_edges([(u, u) for u in G])
sage: G.allow_multiple_edges(True)
sage: G.add_edges(G.edges(sort=False))
sage: ascii_art(G.atoms_and_clique_separators(rooted_tree=True))
  ______{1}______
 /               /
{1, 2}    ______{}______
         /             /
```

(continues on next page)

(continued from previous page)

```
       {0, 1}   _____{4}_____
               /             /
              {4, 5}   ____{}_____
                      /          /
                     {3, 4}   __{7}__
                             /       /
                            {6, 7} {8, 7}
```

```
>>> from sage.all import *
>>> G.allow_loops(True)
>>> G.add_edges([(u, u) for u in G])
>>> G.allow_multiple_edges(True)
>>> G.add_edges(G.edges(sort=False))
>>> ascii_art(G.atoms_and_clique_separators(rooted_tree=True))
  ______{1}______
 /               /
{1, 2}   ______{}______
        /              /
       {0, 1}   _____{4}_____
               /             /
              {4, 5}   ____{}_____
                      /          /
                     {3, 4}   __{7}__
                             /       /
                            {6, 7} {8, 7}
```

We can check that the returned list of separators is valid:

```
sage: G = graphs.RandomGNP(50, .1)
sage: while not G.is_connected():
....:     G = graphs.RandomGNP(50, .1)
sage: _, separators, _ = G.atoms_and_clique_separators(separators=True)
sage: for S in separators:
....:     H = G.copy()
....:     H.delete_vertices(S)
....:     if H.is_connected():
....:         raise ValueError("something goes wrong")
```

```
>>> from sage.all import *
>>> G = graphs.RandomGNP(Integer(50), RealNumber('.1'))
>>> while not G.is_connected():
...     G = graphs.RandomGNP(Integer(50), RealNumber('.1'))
>>> _, separators, _ = G.atoms_and_clique_separators(separators=True)
>>> for S in separators:
...     H = G.copy()
...     H.delete_vertices(S)
...     if H.is_connected():
...         raise ValueError("something goes wrong")
```

sage.graphs.graph_decompositions.clique_separators.**make_labelled_rooted_tree**(*atoms*, *cliques*)

Return a LabelledRootedTree of atoms and cliques.

The atoms are the leaves of the tree and the cliques are the internal vertices. The number of atoms is the number of cliques plus one.

EXAMPLES:

```
sage: G = graphs.PathGraph(5)
sage: ascii_art(G.atoms_and_clique_separators(rooted_tree=True))
  _____{3}_____
 /             /
{3, 4}    ____{2}____
         /           /
       {2, 3}    __{1}__
                /       /
              {0, 1} {1, 2}
```

```
>>> from sage.all import *
>>> G = graphs.PathGraph(Integer(5))
>>> ascii_art(G.atoms_and_clique_separators(rooted_tree=True))
  _____{3}_____
 /             /
{3, 4}    ____{2}____
         /           /
       {2, 3}    __{1}__
                /       /
              {0, 1} {1, 2}
```

sage.graphs.graph_decompositions.clique_separators.**make_tree**(*atoms*, *cliques*)

Return a tree of atoms and cliques.

The atoms are the leaves of the tree and the cliques are the internal vertices. The number of atoms is the number of cliques plus one.

As a clique may appear several times in the list `cliques`, vertices are numbered by pairs (i, S), where $0 \leq i < |atoms| + |cliques|$ and S is either an atom or a clique.

The root of the tree is the only vertex with even or null degree, i.e., 0 if `cliques` is empty and 2 otherwise. When `cliques` is not empty, other internal vertices (each of which is a clique) have degree 3, and the leaves (vertices of degree 1) are the atoms.

INPUT:

- `atoms` – list of atoms
- `cliques` – list of cliques

EXAMPLES:

```
sage: from sage.graphs.graph_decompositions.clique_separators import make_tree
sage: G = graphs.Grid2dGraph(2, 4)
sage: A, Sc = G.atoms_and_clique_separators()
sage: T = make_tree(A, Sc)
sage: all(u[1] in A for u in T if T.degree(u) == 1)
True
sage: all(u[1] in Sc for u in T if T.degree(u) != 1)
True
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_decompositions.clique_separators import make_tree
>>> G = graphs.Grid2dGraph(Integer(2), Integer(4))
>>> A, Sc = G.atoms_and_clique_separators()
>>> T = make_tree(A, Sc)
>>> all(u[Integer(1)] in A for u in T if T.degree(u) == Integer(1))
True
>>> all(u[Integer(1)] in Sc for u in T if T.degree(u) != Integer(1))
True
```

5.33 Convexity properties of graphs

This class gathers the algorithms related to convexity in a graph. It implements the following methods:

ConvexityProperties.hull()	Return the convex hull of a set of vertices
ConvexityProperties.hull_number()	Compute the hull number of a graph and a corresponding generating set
geodetic_closure()	Return the geodetic closure of a set of vertices
is_geodetic()	Check whether the input (di)graph is geodetic

Some of these methods can be used through the *ConvexityProperties* object returned by *Graph.convexity_properties()*.

5.33.1 Methods

class sage.graphs.convexity_properties.**ConvexityProperties**

Bases: object

This class gathers the algorithms related to convexity in a graph.

Definitions

A set $S \subseteq V(G)$ of vertices is said to be convex if for all $u, v \in S$ the set S contains all the vertices located on a shortest path between u and v. Alternatively, a set S is said to be convex if the distances satisfy $\forall u, v \in S, \forall w \in V \backslash S : d_G(u, w) + d_G(w, v) > d_G(u, v)$.

The convex hull $h(S)$ of a set S of vertices is defined as the smallest convex set containing S.

It is a closure operator, as trivially $S \subseteq h(S)$ and $h(h(S)) = h(S)$.

What this class contains

As operations on convex sets generally involve the computation of distances between vertices, this class' purpose is to cache that information so that computing the convex hulls of several different sets of vertices does not imply recomputing several times the distances between the vertices.

In order to compute the convex hull of a set S it is possible to write the following algorithm:

> For any pair u, v of elements in the set S, and for any vertex w outside of it, add w to S if $d_G(u, w) + d_G(w, v) = d_G(u, v)$. When no vertex can be added anymore, the set S is convex

The distances are not actually that relevant. The same algorithm can be implemented by remembering for each pair u, v of vertices the list of elements w satisfying the condition, and this is precisely what this class remembers, encoded as bitsets to make storage and union operations more efficient.

Note

- This class is useful if you compute the convex hulls of many sets in the same graph, or if you want to compute the hull number itself as it involves many calls to `hull()`
- Using this class on non-connected graphs is a waste of space and efficiency ! If your graph is disconnected, the best for you is to deal independently with each connected component, whatever you are doing.

Possible improvements

When computing a convex set, all the pairs of elements belonging to the set S are enumerated several times.

- There should be a smart way to avoid enumerating pairs of vertices which have already been tested. The cost of each of them is not very high, so keeping track of those which have been tested already may be too expensive to gain any efficiency.
- The ordering in which they are visited is currently purely lexicographic, while there is a Poset structure to exploit. In particular, when two vertices u, v are far apart and generate a set $h(\{u, v\})$ of vertices, all the pairs of vertices $u', v' \in h(\{u, v\})$ satisfy $h(\{u', v'\}) \subseteq h(\{u, v\})$, and so it is useless to test the pair u', v' when both u and v where present.
- The information cached is for any pair u, v of vertices the list of elements z with $d_G(u, w) + d_G(w, v) = d_G(u, v)$. This is not in general equal to $h(\{u, v\})$!

Nothing says these recommendations will actually lead to any actual improvements. There are just some ideas remembered while writing this code. Trying to optimize may well lead to lost in efficiency on many instances.

EXAMPLES:

```
sage: from sage.graphs.convexity_properties import ConvexityProperties
sage: g = graphs.PetersenGraph()
sage: CP = ConvexityProperties(g)
sage: CP.hull([1, 3])
[1, 2, 3]
sage: CP.hull_number()                                                      #␣
↪needs sage.numerical.mip
3
```

```
>>> from sage.all import *
>>> from sage.graphs.convexity_properties import ConvexityProperties
>>> g = graphs.PetersenGraph()
>>> CP = ConvexityProperties(g)
>>> CP.hull([Integer(1), Integer(3)])
[1, 2, 3]
>>> CP.hull_number()                                                        #␣
↪needs sage.numerical.mip
3
```

hull(*vertices*)

Return the convex hull of a set of vertices.

INPUT:

- `vertices` – list of vertices

EXAMPLES:

```
sage: from sage.graphs.convexity_properties import ConvexityProperties
sage: g = graphs.PetersenGraph()
sage: CP = ConvexityProperties(g)
sage: CP.hull([1, 3])
[1, 2, 3]
```

```
>>> from sage.all import *
>>> from sage.graphs.convexity_properties import ConvexityProperties
>>> g = graphs.PetersenGraph()
>>> CP = ConvexityProperties(g)
>>> CP.hull([Integer(1), Integer(3)])
[1, 2, 3]
```

hull_number(*value_only=True*, *verbose=False*)

Compute the hull number and a corresponding generating set.

The hull number $hn(G)$ of a graph G is the cardinality of a smallest set of vertices S such that $h(S) = V(G)$.

INPUT:

- `value_only` – boolean (default: `True`); whether to return only the hull number (default) or a minimum set whose convex hull is the whole graph
- `verbose` – boolean (default: `False`); whether to display information on the LP

COMPLEXITY:

This problem is NP-Hard [HLT1993], but seems to be of the "nice" kind. Update this comment if you fall on hard instances : $-$)

ALGORITHM:

This is solved by linear programming.

As the function $h(S)$ associating to each set S its convex hull is a closure operator, it is clear that any set S_G of vertices such that $h(S_G) = V(G)$ must satisfy $S_G \not\subseteq C$ for any *proper* convex set $C \subsetneq V(G)$. The following formulation is hence correct

$$\text{Minimize : } \sum_{v \in G} b_v$$

$$\text{Such that : }$$

$$\forall C \subsetneq V(G) \text{ a proper convex set}$$

$$\sum_{v \in V(G) \setminus C} b_v \geq 1$$

Of course, the number of convex sets – and so the number of constraints – can be huge, and hard to enumerate, so at first an incomplete formulation is solved (it is missing some constraints). If the answer returned by the LP solver is a set S generating the whole graph, then it is optimal and so is returned. Otherwise, the constraint corresponding to the set $h(S)$ can be added to the LP, which makes the answer S infeasible, and another solution computed.

This being said, simply adding the constraint corresponding to $h(S)$ is a bit slow, as these sets can be large (and the corresponding constraint a bit weak). To improve it a bit, before being added, the set $h(S)$ is "greedily enriched" to a set S' with vertices for as long as $h(S') \neq V(G)$. This way, we obtain a set S' with $h(S) \subseteq h(S') \subsetneq V(G)$, and the constraint corresponding to $h(S')$ – which is stronger than the one corresponding to $h(S)$ – is added.

This can actually be seen as a hitting set problem on the complement of convex sets.

EXAMPLES:

The Hull number of Petersen's graph:

```
sage: from sage.graphs.convexity_properties import ConvexityProperties
sage: g = graphs.PetersenGraph()
sage: CP = ConvexityProperties(g)
sage: CP.hull_number()                                                      #␣
↪needs sage.numerical.mip
3
sage: generating_set = CP.hull_number(value_only=False)                     #␣
↪needs sage.numerical.mip
sage: CP.hull(generating_set)                                               #␣
↪needs sage.numerical.mip
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

```
>>> from sage.all import *
>>> from sage.graphs.convexity_properties import ConvexityProperties
>>> g = graphs.PetersenGraph()
>>> CP = ConvexityProperties(g)
>>> CP.hull_number()                                                        #␣
↪needs sage.numerical.mip
3
>>> generating_set = CP.hull_number(value_only=False)                       #␣
↪needs sage.numerical.mip
>>> CP.hull(generating_set)                                                 #␣
↪needs sage.numerical.mip
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

sage.graphs.convexity_properties.**geodetic_closure**(G, S)

Return the geodetic closure of the set of vertices S in G.

The geodetic closure $g(S)$ of a subset of vertices S of a graph G is in [HLT1993] as the set of all vertices that lie on a shortest $u - v$ path for any pair of vertices $u, v \in S$. We assume that $g(\emptyset) = \emptyset$ and that $g(\{u\}) = \{u\}$ for any u in G.

Warning

This operation is **not** a closure function. Indeed, a closure function must satisfy the property that $f(f(X))$ should be equal to $f(X)$, which is not always the case here. The term `closure` is used here to follow the terminology of the domain. See for instance [HLT1993].

Here, we implement a simple algorithm to determine this set. Roughly, for each vertex $u \in S$, the algorithm first performs a breadth first search from u to get distances, and then identifies the vertices of G lying on a shortest path from u to any $v \in S$ using a reversal traversal from vertices in S. This algorithm has time complexity in $O(|S|(n+m))$ for `SparseGraph`, $O(|S|(n+m)+n^2)$ for `DenseGraph` and space complexity in $O(n+m)$.

INPUT:

- `G` – a Sage graph
- `S` – a subset of vertices of G

EXAMPLES:

The vertices of the Petersen graph can be obtained by a geodetic closure of four of its vertices:

```
sage: from sage.graphs.convexity_properties import geodetic_closure
sage: G = graphs.PetersenGraph()
sage: geodetic_closure(G, [0, 2, 8, 9])
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

```
>>> from sage.all import *
>>> from sage.graphs.convexity_properties import geodetic_closure
>>> G = graphs.PetersenGraph()
>>> geodetic_closure(G, [Integer(0), Integer(2), Integer(8), Integer(9)])
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

The vertices of a 2D grid can be obtained by a geodetic closure of two vertices:

```
sage: G = graphs.Grid2dGraph(4, 4)
sage: c = G.geodetic_closure([(0, 0), (3, 3)])
sage: len(c) == G.order()
True
```

```
>>> from sage.all import *
>>> G = graphs.Grid2dGraph(Integer(4), Integer(4))
>>> c = G.geodetic_closure([(Integer(0), Integer(0)), (Integer(3), Integer(3))])
>>> len(c) == G.order()
True
```

If two vertices belong to different connected components of a graph, their geodetic closure is trivial:

```
sage: G = Graph([(0, 1), (2, 3)])
sage: geodetic_closure(G, [0, 2])
[0, 2]
```

```
>>> from sage.all import *
>>> G = Graph([(Integer(0), Integer(1)), (Integer(2), Integer(3))])
>>> geodetic_closure(G, [Integer(0), Integer(2)])
[0, 2]
```

The geodetic closure does not satisfy the closure function property that $f(f(X))$ should be equal to $f(X)$:

```
sage: G = graphs.DiamondGraph()
sage: G.subdivide_edge((1, 2), 1)
sage: geodetic_closure(G, [0, 3])
[0, 1, 2, 3]
sage: geodetic_closure(G, geodetic_closure(G, [0, 3]))
[0, 1, 2, 3, 4]
```

```
>>> from sage.all import *
>>> G = graphs.DiamondGraph()
>>> G.subdivide_edge((Integer(1), Integer(2)), Integer(1))
>>> geodetic_closure(G, [Integer(0), Integer(3)])
[0, 1, 2, 3]
>>> geodetic_closure(G, geodetic_closure(G, [Integer(0), Integer(3)]))
[0, 1, 2, 3, 4]
```

sage.graphs.convexity_properties.**is_geodetic**(G)

Check whether the input (di)graph is geodetic.

A graph G is *geodetic* if there exists at most one shortest path between every pair of its vertices. This can be checked in time $O(nm)$ for `SparseGraph` and $O(nm+n^2)$ for `DenseGraph` in unweighted (di)graphs with n nodes and m edges. Examples of geodetic graphs are trees, cliques and odd cycles. See the Wikipedia article Geodetic_graph for more details. Note that we do not require geodetic graphs to be connected.

(Di)graphs with multiple edges are not considered geodetic.

INPUT:

- `G` – a graph or a digraph

EXAMPLES:

Trees, cliques and odd cycles are geodetic:

```
sage: T = graphs.RandomTree(20)
sage: T.is_geodetic()
True
sage: all(graphs.CompleteGraph(n).is_geodetic() for n in range(8))
True
sage: all(graphs.CycleGraph(n).is_geodetic() for n in range(3, 16, 2))
True
```

```
>>> from sage.all import *
>>> T = graphs.RandomTree(Integer(20))
>>> T.is_geodetic()
True
>>> all(graphs.CompleteGraph(n).is_geodetic() for n in range(Integer(8)))
True
>>> all(graphs.CycleGraph(n).is_geodetic() for n in range(Integer(3), Integer(16),
↪ Integer(2)))
True
```

Even cycles of order at least 4 are not geodetic:

```
sage: all(graphs.CycleGraph(n).is_geodetic() for n in range(4, 17, 2))
False
```

```
>>> from sage.all import *
>>> all(graphs.CycleGraph(n).is_geodetic() for n in range(Integer(4), Integer(17),
↪ Integer(2)))
False
```

The Petersen graph is geodetic:

```
sage: P = graphs.PetersenGraph()
sage: P.is_geodetic()
True
```

```
>>> from sage.all import *
>>> P = graphs.PetersenGraph()
>>> P.is_geodetic()
True
```

Grid graphs are not geodetic:

```
sage: G = graphs.Grid2dGraph(2, 3)
sage: G.is_geodetic()
False
```

```
>>> from sage.all import *
>>> G = graphs.Grid2dGraph(Integer(2), Integer(3))
>>> G.is_geodetic()
False
```

A graph is geodetic iff all its connected components are geodetic:

```
sage: all(G.is_geodetic() == all(H.is_geodetic() for H in G.connected_components_
↪subgraphs()) for G in graphs(5))
True
```

```
>>> from sage.all import *
>>> all(G.is_geodetic() == all(H.is_geodetic() for H in G.connected_components_
↪subgraphs()) for G in graphs(Integer(5)))
True
```

This method is also valid for digraphs:

```
sage: G = DiGraph(graphs.PetersenGraph())
sage: G.is_geodetic()
True
sage: G = digraphs.Path(5)
sage: G.add_path([0, 'a', 'b', 'c', 4])
sage: G.is_geodetic()
False
```

```
>>> from sage.all import *
>>> G = DiGraph(graphs.PetersenGraph())
>>> G.is_geodetic()
True
>>> G = digraphs.Path(Integer(5))
>>> G.add_path([Integer(0), 'a', 'b', 'c', Integer(4)])
>>> G.is_geodetic()
False
```

5.34 Weakly chordal graphs

This module deals with everything related to weakly chordal graphs. It currently contains the following functions:

is_long_hole_free()	Test whether g contains an induced cycle of length at least 5.
is_long_antihole_free()	Test whether g contains an induced anticycle of length at least 5.
is_weakly_chordal()	Test whether g is weakly chordal.

Author:

- Birk Eisermann (initial implementation)

- Nathann Cohen (some doc and optimization)
- David Coudert (remove recursion)

5.34.1 Methods

sage.graphs.weakly_chordal.**is_long_antihole_free**(*g*, *certificate=False*)

Test whether the given graph contains an induced subgraph that is isomorphic to the complement of a cycle of length at least 5.

INPUT:

- `certificate` – boolean (default: `False`)

  Whether to return a certificate. When `certificate = True`, then the function returns

  - `(False, Antihole)` if `g` contains an induced complement of a cycle of length at least 5 returned as `Antihole`.
  - `(True, [])` if `g` does not contain an induced complement of a cycle of length at least 5. For this case it is not known how to provide a certificate.

  When `certificate = False`, the function returns just `True` or `False` accordingly.

ALGORITHM:

This algorithm tries to find a cycle in the graph of all induced $\overline{P_4}$ of g, where two copies $\overline{P}$ and $\overline{P'}$ of $\overline{P_4}$ are adjacent if there exists a (not necessarily induced) copy of $\overline{P_5} = u_1u_2u_3u_4u_5$ such that $\overline{P} = u_1u_2u_3u_4$ and $\overline{P'} = u_2u_3u_4u_5$.

This is done through a depth-first-search. For efficiency, the auxiliary graph is constructed on-the-fly and never stored in memory.

The run time of this algorithm is $O(n+m^2)$ for `SparseGraph` and $O(n^2 \log m+m^2)$ for `DenseGraph` [NP2007] (where n is the number of vertices and m is the number of edges of the graph).

EXAMPLES:

The Petersen Graph contains an antihole:

```
sage: g = graphs.PetersenGraph()
sage: g.is_long_antihole_free()
False
```

```
>>> from sage.all import *
>>> g = graphs.PetersenGraph()
>>> g.is_long_antihole_free()
False
```

The complement of a cycle is an antihole:

```
sage: g = graphs.CycleGraph(6).complement()
sage: r,a = g.is_long_antihole_free(certificate=True)
sage: r
False
sage: a.complement().is_isomorphic(graphs.CycleGraph(6))
True
```

```
>>> from sage.all import *
>>> g = graphs.CycleGraph(Integer(6)).complement()
>>> r,a = g.is_long_antihole_free(certificate=True)
>>> r
False
>>> a.complement().is_isomorphic(graphs.CycleGraph(Integer(6)))
True
```

sage.graphs.weakly_chordal.**is_long_hole_free**(*g*, *certificate=False*)

Test whether `g` contains an induced cycle of length at least 5.

INPUT:

- `certificate` – boolean (default: `False`)

  Whether to return a certificate. When `certificate = True`, then the function returns

  - `(True, [])` if `g` does not contain such a cycle. For this case, it is not known how to provide a certificate.
  - `(False, Hole)` if `g` contains an induced cycle of length at least 5. `Hole` returns this cycle.

  If `certificate = False`, the function returns just `True` or `False` accordingly.

ALGORITHM:

This algorithm tries to find a cycle in the graph of all induced P_4 of g, where two copies P and P' of P_4 are adjacent if there exists a (not necessarily induced) copy of $P_5 = u_1u_2u_3u_4u_5$ such that $P = u_1u_2u_3u_4$ and $P' = u_2u_3u_4u_5$.

This is done through a depth-first-search. For efficiency, the auxiliary graph is constructed on-the-fly and never stored in memory.

The run time of this algorithm is $O(n + m^2)$ for `SparseGraph` and $O(n^2 + m^2)$ for `DenseGraph` [NP2007] (where n is the number of vertices and m is the number of edges of the graph).

EXAMPLES:

The Petersen Graph contains a hole:

```
sage: g = graphs.PetersenGraph()
sage: g.is_long_hole_free()
False
```

```
>>> from sage.all import *
>>> g = graphs.PetersenGraph()
>>> g.is_long_hole_free()
False
```

The following graph contains a hole, which we want to display:

```
sage: g = graphs.FlowerSnark()
sage: r,h = g.is_long_hole_free(certificate=True)
sage: r
False
sage: Graph(h).is_isomorphic(graphs.CycleGraph(h.order()))
True
```

```
>>> from sage.all import *
>>> g = graphs.FlowerSnark()
>>> r,h = g.is_long_hole_free(certificate=True)
>>> r
False
>>> Graph(h).is_isomorphic(graphs.CycleGraph(h.order()))
True
```

sage.graphs.weakly_chordal.**is_weakly_chordal**(*g*, *certificate=False*)

Test whether the given graph is weakly chordal, i.e., the graph and its complement have no induced cycle of length at least 5.

INPUT:

- `certificate` – boolean (default: `False`); whether to return a certificate. If `certificate = False`, return `True` or `False` according to the graph. If `certificate = True`, return
  - `(False, forbidden_subgraph)` when the graph contains a forbidden subgraph H, this graph is returned.
  - **`(True, [])` when the graph is weakly chordal.**
    For this case, it is not known how to provide a certificate.

ALGORITHM:

This algorithm checks whether the graph `g` or its complement contain an induced cycle of length at least 5.

Using is_long_hole_free() and is_long_antihole_free() yields a run time of $O(n + m^2)$ for `SparseGraph` and $O(n^2 + m^2)$ for `DenseGraph` (where n is the number of vertices and m is the number of edges of the graph).

EXAMPLES:

The Petersen Graph is not weakly chordal and contains a hole:

```
sage: g = graphs.PetersenGraph()
sage: r,s = g.is_weakly_chordal(certificate=True)
sage: r
False
sage: l = s.order()
sage: s.is_isomorphic(graphs.CycleGraph(l))
True
```

```
>>> from sage.all import *
>>> g = graphs.PetersenGraph()
>>> r,s = g.is_weakly_chordal(certificate=True)
>>> r
False
>>> l = s.order()
>>> s.is_isomorphic(graphs.CycleGraph(l))
True
```

5.35 Distances/shortest paths between all pairs of vertices

This module implements a few functions that deal with the computation of distances or shortest paths between all pairs of vertices.

Efficiency : Because these functions involve listing many times the (out)-neighborhoods of (di)-graphs, it is useful in terms of efficiency to build a temporary copy of the graph in a data structure that makes it easy to compute quickly. These functions also work on large volume of data, typically dense matrices of size n^2, and are expected to return corresponding dictionaries of size n^2, where the integers corresponding to the vertices have first been converted to the vertices' labels. Sadly, this last translating operation turns out to be the most time-consuming, and for this reason it is also nice to have a Cython module, and version of these functions that return C arrays, in order to avoid these operations when they are not necessary.

Memory cost : The methods implemented in the current module sometimes need large amounts of memory to return their result. Storing the distances between all pairs of vertices in a graph on 1500 vertices as a dictionary of dictionaries takes around 200MB, while storing the same information as a C array requires 4MB.

5.35.1 The module's main function

The C function `all_pairs_shortest_path_BFS` actually does all the computations, and all the others (except for `Floyd_Warshall`) are just wrapping it. This function begins with copying the graph in a data structure that makes it fast to query the out-neighbors of a vertex, then starts one Breadth First Search per vertex of the (di)graph.

What can this function compute ?

- The matrix of predecessors.

  This matrix P has size n^2, and is such that vertex $P[u, v]$ is a predecessor of v on a shortest uv-path. Hence, this matrix efficiently encodes the information of a shortest uv-path for any $u, v \in G$: indeed, to go from u to v you should first find a shortest $uP[u, v]$-path, then jump from $P[u, v]$ to v as it is one of its outneighbors. Apply recursively and find out what the whole path is !.

- The matrix of distances.

  This matrix has size n^2 and associates to any uv the distance from u to v.

- The vector of eccentricities.

  This vector of size n encodes for each vertex v the distance to vertex which is furthest from v in the graph. In particular, the diameter of the graph is the maximum of these values.

What does it take as input ?

- `gg` a (Di)Graph.
- `unsigned short * predecessors` – a pointer toward an array of size $n^2 \cdot$ sizeof(unsigned short). Set to `NULL` if you do not want to compute the predecessors.
- `unsigned short * distances` – a pointer toward an array of size $n^2 \cdot$ sizeof(unsigned short). The computation of the distances is necessary for the algorithm, so this value can **not** be set to `NULL`.
- `int * eccentricity` – a pointer toward an array of size $n \cdot$ sizeof(int). Set to `NULL` if you do not want to compute the eccentricity.

Technical details

- The vertices are encoded as $1, ..., n$ as they appear in the ordering of `G.vertices(sort=True)`, unless another ordering is specified by the user.
- Because this function works on matrices whose size is quadratic compared to the number of vertices when computing all distances or predecessors, it uses short variables to store the vertices' names instead of long ones to divide by 2 the size in memory. This means that only the diameter/eccentricities can be computed on a graph of more than 65536 nodes. For information, the current version of the algorithm on a graph with $65536 = 2^{16}$ nodes creates in memory 2 tables on 2^{32} short elements (2bytes each), for a total of 2^{33} bytes or 8 gigabytes. In order to support larger sizes, we would have to replace shorts by 32-bits int or 64-bits int, which would then require respectively 16GB or 32GB.

- In the C version of these functions, infinite distances are represented with `<unsigned short> -1 = 65535` for `unsigned short` variables, and by `INT32_MAX` otherwise. These case happens when the input is a disconnected graph, or a non-strongly-connected digraph.
- A memory error is raised when data structures allocation failed. This could happen with large graphs on computers with low memory space.

Warning

The function `all_pairs_shortest_path_BFS` has **no reason** to be called by the user, even though he would be writing his code in Cython and look for efficiency. This module contains wrappers for this function that feed it with the good parameters. As the function is inlined, using those wrappers actually saves time as it should avoid testing the parameters again and again in the main function's body.

AUTHOR:

- Nathann Cohen (2011)
- David Coudert (2014) – 2sweep, multi-sweep and iFUB for diameter computation

5.35.2 Functions

`sage.graphs.distances_all_pairs.`**`antipodal_graph`**(*G*, *immutable=None*)

Return the antipodal graph of G.

The antipodal graph of a graph G has the same vertex set of G and two vertices are adjacent if their distance in G is equal to the diameter of G.

This method first computes the eccentricity of all vertices and determines the diameter of the graph. Then, it for each vertex u with eccentricity the diameter, it computes BFS distances from u and add an edge in the antipodal graph for each vertex v at diameter distance from u (i.e., for each antipodal vertex).

The drawback of this method is that some BFS distances may be computed twice, one time to determine the eccentricities and another time is the vertex has eccentricity equal to the diameter. However, in practice, this is much more efficient. See *`eccentricity()`* with `algorithm='DHV'`.

INPUT:

- `G` – a *`Graph`*
- `immutable` – boolean (default: `None`); whether to create a mutable/immutable graph. `immutable=None` (default) means that the graph and its antipodal graph will behave the same way.

EXAMPLES:

The antipodal graph of a grid graph has only 2 edges:

```
sage: from sage.graphs.distances_all_pairs import antipodal_graph
sage: G = graphs.Grid2dGraph(5, 5)
sage: A = antipodal_graph(G)
sage: A
Antipodal graph of 2D Grid Graph for [5, 5]: Graph on 25 vertices
sage: A.order(), A.size()
(25, 2)
```

```
>>> from sage.all import *
>>> from sage.graphs.distances_all_pairs import antipodal_graph
```

(continues on next page)

(continued from previous page)

```
>>> G = graphs.Grid2dGraph(Integer(5), Integer(5))
>>> A = antipodal_graph(G)
>>> A
Antipodal graph of 2D Grid Graph for [5, 5]: Graph on 25 vertices
>>> A.order(), A.size()
(25, 2)
```

The antipodal graph of a disconnected graph is its complement:

```
sage: G = Graph(5)
sage: H = G.antipodal_graph()
sage: H
Antipodal graph of Graph on 5 vertices: Graph on 5 vertices
sage: H.is_isomorphic(G.complement())
True
```

```
>>> from sage.all import *
>>> G = Graph(Integer(5))
>>> H = G.antipodal_graph()
>>> H
Antipodal graph of Graph on 5 vertices: Graph on 5 vertices
>>> H.is_isomorphic(G.complement())
True
```

The antipodal graph can also be constructed as the *distance_graph()* for diameter distance:

```
sage: G = graphs.RandomGNP(10, .2)
sage: A = G.antipodal_graph()
sage: B = G.distance_graph(G.diameter())
sage: A.is_isomorphic(B)
True
```

```
>>> from sage.all import *
>>> G = graphs.RandomGNP(Integer(10), RealNumber('.2'))
>>> A = G.antipodal_graph()
>>> B = G.distance_graph(G.diameter())
>>> A.is_isomorphic(B)
True
```

sage.graphs.distances_all_pairs.**diameter**(*G*, *algorithm=None*, *source=None*)

Return the diameter of G.

This method returns Infinity if the (di)graph is not connected. It can also quickly return a lower bound on the diameter using the `2sweep`, `2Dsweep` and `multi-sweep` schemes.

INPUT:

- `algorithm` – string (default: `None`); specifies the algorithm to use among:
  - `'standard'` – computes the diameter of the input (di)graph as the largest eccentricity of its vertices. This is the classical algorithm with time complexity in $O(nm)$.
  - `'2sweep'` – computes a lower bound on the diameter of an unweighted undirected graph using 2 BFS, as proposed in [MLH2008]. It first selects a vertex v that is at largest distance from an initial vertex

source using BFS. Then it performs a second BFS from v. The largest distance from v is returned as a lower bound on the diameter of G. The time complexity of this algorithm is linear in the size of G.

- `'2Dsweep'` – computes lower bound on the diameter of an unweighted directed graph using directed version of `2sweep` as proposed in [Broder2000]. If the digraph is not strongly connected, the returned value is infinity.
- `'DHV'` – computes diameter of unweighted undirected graph using the algorithm proposed in [Dragan2018]
- `'multi-sweep'` – computes a lower bound on the diameter of an unweighted undirected graph using several iterations of the `2sweep` algorithms [CGHLM2013]. Roughly, it first uses `2sweep` to identify two vertices u and v that are far apart. Then it selects a vertex w that is at same distance from u and v. This vertex w will serve as the new source for another iteration of the `2sweep` algorithm that may improve the current lower bound on the diameter. This process is repeated as long as the lower bound on the diameter is improved.
- `'iFUB'` – the iFUB (iterative Fringe Upper Bound) algorithm, proposed in [CGILM2010], computes the exact value of the diameter of an unweighted undirected graph. It is based on the following observation:

  The diameter of the graph is equal to the maximum eccentricity of a vertex. Let v be any vertex, and let V be partitioned into $A \cup B$ where:

  $$d(v, a) \leq i, \forall a \in A$$
  $$d(v, b) \geq i, \forall b \in B$$

  As all vertices from A are at distance $\leq 2i$ from each other, a vertex $a \in A$ with eccentricity $ecc(a) > 2i$ is at distance $ecc(a)$ from some vertex $b \in B$.

  Consequently, if we have already computed the maximum eccentricity m of all vertices in B and if $m > 2i$, then we do not need to compute the eccentricity of the vertices in A.

  Starting from a vertex v obtained through a multi-sweep computation (which refines the 4sweep algorithm used in [CGHLM2013]), we compute the diameter by computing the eccentricity of all vertices sorted decreasingly according to their distance to v, and stop as allowed by the remark above. The worst case time complexity of the iFUB algorithm is $O(nm)$, but it can be very fast in practice.
- `'DiFUB'` – the directed version of iFUB (iterative Fringe Upper Bound) algorithm. See the code's documentation and [CGLM2012] for more details. If the digraph is not strongly connected, the returned value is infinity.

- `source` – (default: `None`) vertex from which to start the first BFS. If `source==None`, an arbitrary vertex of the graph is chosen. Raise an error if the initial vertex is not in G. This parameter is not used when `algorithm=='standard'`.

Note

As the graph is first converted to a short_digraph, all complexity have an extra $O(m + n)$ for `SparseGraph` and $O(n^2)$ for `DenseGraph`.

EXAMPLES:

```
sage: from sage.graphs.distances_all_pairs import diameter
sage: G = graphs.PetersenGraph()
sage: diameter(G, algorithm='iFUB')
2
sage: G = Graph({0: [], 1: [], 2: [1]})
```

(continues on next page)

(continued from previous page)

```
sage: diameter(G, algorithm='iFUB')
+Infinity
sage: G = digraphs.Circuit(6)
sage: diameter(G, algorithm='2Dsweep')
5
sage: G = graphs.PathGraph(7).to_directed()
sage: diameter(G, algorithm='DiFUB')
6
```

```
>>> from sage.all import *
>>> from sage.graphs.distances_all_pairs import diameter
>>> G = graphs.PetersenGraph()
>>> diameter(G, algorithm='iFUB')
2
>>> G = Graph({Integer(0): [], Integer(1): [], Integer(2): [Integer(1)]})
>>> diameter(G, algorithm='iFUB')
+Infinity
>>> G = digraphs.Circuit(Integer(6))
>>> diameter(G, algorithm='2Dsweep')
5
>>> G = graphs.PathGraph(Integer(7)).to_directed()
>>> diameter(G, algorithm='DiFUB')
6
```

Although max( ) is usually defined as -Infinity, since the diameter will never be negative, we define it to be zero:

```
sage: G = graphs.EmptyGraph()
sage: diameter(G, algorithm='iFUB')
0
```

```
>>> from sage.all import *
>>> G = graphs.EmptyGraph()
>>> diameter(G, algorithm='iFUB')
0
```

Comparison of exact algorithms for graphs:

```
sage: # needs networkx
sage: G = graphs.RandomBarabasiAlbert(100, 2)
sage: d1 = diameter(G, algorithm='standard')
sage: d2 = diameter(G, algorithm='iFUB')
sage: d3 = diameter(G, algorithm='iFUB', source=G.random_vertex())
sage: d4 = diameter(G, algorithm='DHV')
sage: if d1 != d2 or d1 != d3 or d1 != d4: print("Something goes wrong!")
```

```
>>> from sage.all import *
>>> # needs networkx
>>> G = graphs.RandomBarabasiAlbert(Integer(100), Integer(2))
>>> d1 = diameter(G, algorithm='standard')
>>> d2 = diameter(G, algorithm='iFUB')
>>> d3 = diameter(G, algorithm='iFUB', source=G.random_vertex())
```

(continues on next page)

(continued from previous page)

```
>>> d4 = diameter(G, algorithm='DHV')
>>> if d1 != d2 or d1 != d3 or d1 != d4: print("Something goes wrong!")
```

Comparison of lower bound algorithms:

```
sage: lb2 = diameter(G, algorithm='2sweep')                                    #␣
↪needs networkx
sage: lbm = diameter(G, algorithm='multi-sweep')                               #␣
↪needs networkx
sage: if not (lb2 <= lbm and lbm <= d3): print("Something goes wrong!")        #␣
↪needs networkx
```

```
>>> from sage.all import *
>>> lb2 = diameter(G, algorithm='2sweep')                                      #␣
↪needs networkx
>>> lbm = diameter(G, algorithm='multi-sweep')                                 #␣
↪needs networkx
>>> if not (lb2 <= lbm and lbm <= d3): print("Something goes wrong!")          #␣
↪needs networkx
```

Comparison of exact algorithms for digraphs:

```
sage: # needs networkx
sage: D = DiGraph(graphs.RandomBarabasiAlbert(50, 2))
sage: d1 = diameter(D, algorithm='standard')
sage: d2 = diameter(D, algorithm='DiFUB')
sage: d3 = diameter(D, algorithm='DiFUB', source=D.random_vertex())
sage: d1 == d2 and d1 == d3
True
```

```
>>> from sage.all import *
>>> # needs networkx
>>> D = DiGraph(graphs.RandomBarabasiAlbert(Integer(50), Integer(2)))
>>> d1 = diameter(D, algorithm='standard')
>>> d2 = diameter(D, algorithm='DiFUB')
>>> d3 = diameter(D, algorithm='DiFUB', source=D.random_vertex())
>>> d1 == d2 and d1 == d3
True
```

sage.graphs.distances_all_pairs.**distances_all_pairs**(G)

Return the matrix of distances in G.

This function returns a double dictionary `D` of vertices, in which the distance between vertices `u` and `v` is `D[u][v]`.

EXAMPLES:

```
sage: from sage.graphs.distances_all_pairs import distances_all_pairs
sage: g = graphs.PetersenGraph()
sage: distances_all_pairs(g)
{0: {0: 0, 1: 1, 2: 2, 3: 2, 4: 1, 5: 1, 6: 2, 7: 2, 8: 2, 9: 2},
 1: {0: 1, 1: 0, 2: 1, 3: 2, 4: 2, 5: 2, 6: 1, 7: 2, 8: 2, 9: 2},
 2: {0: 2, 1: 1, 2: 0, 3: 1, 4: 2, 5: 2, 6: 2, 7: 1, 8: 2, 9: 2},
 3: {0: 2, 1: 2, 2: 1, 3: 0, 4: 1, 5: 2, 6: 2, 7: 2, 8: 1, 9: 2},
```

(continues on next page)

(continued from previous page)

```
 4: {0: 1, 1: 2, 2: 2, 3: 1, 4: 0, 5: 2, 6: 2, 7: 2, 8: 2, 9: 1},
 5: {0: 1, 1: 2, 2: 2, 3: 2, 4: 2, 5: 0, 6: 2, 7: 1, 8: 1, 9: 2},
 6: {0: 2, 1: 1, 2: 2, 3: 2, 4: 2, 5: 2, 6: 0, 7: 2, 8: 1, 9: 1},
 7: {0: 2, 1: 2, 2: 1, 3: 2, 4: 2, 5: 1, 6: 2, 7: 0, 8: 2, 9: 1},
 8: {0: 2, 1: 2, 2: 2, 3: 1, 4: 2, 5: 1, 6: 1, 7: 2, 8: 0, 9: 2},
 9: {0: 2, 1: 2, 2: 2, 3: 2, 4: 1, 5: 2, 6: 1, 7: 1, 8: 2, 9: 0}}
```

```
>>> from sage.all import *
>>> from sage.graphs.distances_all_pairs import distances_all_pairs
>>> g = graphs.PetersenGraph()
>>> distances_all_pairs(g)
{0: {0: 0, 1: 1, 2: 2, 3: 2, 4: 1, 5: 1, 6: 2, 7: 2, 8: 2, 9: 2},
 1: {0: 1, 1: 0, 2: 1, 3: 2, 4: 2, 5: 2, 6: 1, 7: 2, 8: 2, 9: 2},
 2: {0: 2, 1: 1, 2: 0, 3: 1, 4: 2, 5: 2, 6: 2, 7: 1, 8: 2, 9: 2},
 3: {0: 2, 1: 2, 2: 1, 3: 0, 4: 1, 5: 2, 6: 2, 7: 2, 8: 1, 9: 2},
 4: {0: 1, 1: 2, 2: 2, 3: 1, 4: 0, 5: 2, 6: 2, 7: 2, 8: 2, 9: 1},
 5: {0: 1, 1: 2, 2: 2, 3: 2, 4: 2, 5: 0, 6: 2, 7: 1, 8: 1, 9: 2},
 6: {0: 2, 1: 1, 2: 2, 3: 2, 4: 2, 5: 2, 6: 0, 7: 2, 8: 1, 9: 1},
 7: {0: 2, 1: 2, 2: 1, 3: 2, 4: 2, 5: 1, 6: 2, 7: 0, 8: 2, 9: 1},
 8: {0: 2, 1: 2, 2: 2, 3: 1, 4: 2, 5: 1, 6: 1, 7: 2, 8: 0, 9: 2},
 9: {0: 2, 1: 2, 2: 2, 3: 2, 4: 1, 5: 2, 6: 1, 7: 1, 8: 2, 9: 0}}
```

sage.graphs.distances_all_pairs.**distances_and_predecessors_all_pairs**(G)

Return the matrix of distances in G and the matrix of predecessors.

Distances : the matrix M returned is of length n^2, and the distance between vertices u and v is $M[u, v]$. The integer corresponding to a vertex is its index in the list `G.vertices(sort=True)`.

Predecessors : the matrix P returned has size n^2, and is such that vertex $P[u, v]$ is a predecessor of v on a shortest uv-path. Hence, this matrix efficiently encodes the information of a shortest uv-path for any $u, v \in G$: indeed, to go from u to v you should first find a shortest $uP[u, v]$-path, then jump from $P[u, v]$ to v as it is one of its outneighbors.

EXAMPLES:

```
sage: from sage.graphs.distances_all_pairs import distances_and_predecessors_all_
↪pairs
sage: g = graphs.PetersenGraph()
sage: distances_and_predecessors_all_pairs(g)
({0: {0: 0, 1: 1, 2: 2, 3: 2, 4: 1, 5: 1, 6: 2, 7: 2, 8: 2, 9: 2},
  1: {0: 1, 1: 0, 2: 1, 3: 2, 4: 2, 5: 2, 6: 1, 7: 2, 8: 2, 9: 2},
  2: {0: 2, 1: 1, 2: 0, 3: 1, 4: 2, 5: 2, 6: 2, 7: 1, 8: 2, 9: 2},
  3: {0: 2, 1: 2, 2: 1, 3: 0, 4: 1, 5: 2, 6: 2, 7: 2, 8: 1, 9: 2},
  4: {0: 1, 1: 2, 2: 2, 3: 1, 4: 0, 5: 2, 6: 2, 7: 2, 8: 2, 9: 1},
  5: {0: 1, 1: 2, 2: 2, 3: 2, 4: 2, 5: 0, 6: 2, 7: 1, 8: 1, 9: 2},
  6: {0: 2, 1: 1, 2: 2, 3: 2, 4: 2, 5: 2, 6: 0, 7: 2, 8: 1, 9: 1},
  7: {0: 2, 1: 2, 2: 1, 3: 2, 4: 2, 5: 1, 6: 2, 7: 0, 8: 2, 9: 1},
  8: {0: 2, 1: 2, 2: 2, 3: 1, 4: 2, 5: 1, 6: 1, 7: 2, 8: 0, 9: 2},
  9: {0: 2, 1: 2, 2: 2, 3: 2, 4: 1, 5: 2, 6: 1, 7: 1, 8: 2, 9: 0}},
 {0: {0: None, 1: 0, 2: 1, 3: 4, 4: 0, 5: 0, 6: 1, 7: 5, 8: 5, 9: 4},
  1: {0: 1, 1: None, 2: 1, 3: 2, 4: 0, 5: 0, 6: 1, 7: 2, 8: 6, 9: 6},
  2: {0: 1, 1: 2, 2: None, 3: 2, 4: 3, 5: 7, 6: 1, 7: 2, 8: 3, 9: 7},
  3: {0: 4, 1: 2, 2: 3, 3: None, 4: 3, 5: 8, 6: 8, 7: 2, 8: 3, 9: 4},
```

(continues on next page)

(continued from previous page)

```
  4: {0: 4, 1: 0, 2: 3, 3: 4, 4: None, 5: 0, 6: 9, 7: 9, 8: 3, 9: 4},
  5: {0: 5, 1: 0, 2: 7, 3: 8, 4: 0, 5: None, 6: 8, 7: 5, 8: 5, 9: 7},
  6: {0: 1, 1: 6, 2: 1, 3: 8, 4: 9, 5: 8, 6: None, 7: 9, 8: 6, 9: 6},
  7: {0: 5, 1: 2, 2: 7, 3: 2, 4: 9, 5: 7, 6: 9, 7: None, 8: 5, 9: 7},
  8: {0: 5, 1: 6, 2: 3, 3: 8, 4: 3, 5: 8, 6: 8, 7: 5, 8: None, 9: 6},
  9: {0: 4, 1: 6, 2: 7, 3: 4, 4: 9, 5: 7, 6: 9, 7: 9, 8: 6, 9: None}})
```

```
>>> from sage.all import *
>>> from sage.graphs.distances_all_pairs import distances_and_predecessors_all_
↪pairs
>>> g = graphs.PetersenGraph()
>>> distances_and_predecessors_all_pairs(g)
({0: {0: 0, 1: 1, 2: 2, 3: 2, 4: 1, 5: 1, 6: 2, 7: 2, 8: 2, 9: 2},
  1: {0: 1, 1: 0, 2: 1, 3: 2, 4: 2, 5: 2, 6: 1, 7: 2, 8: 2, 9: 2},
  2: {0: 2, 1: 1, 2: 0, 3: 1, 4: 2, 5: 2, 6: 2, 7: 1, 8: 2, 9: 2},
  3: {0: 2, 1: 2, 2: 1, 3: 0, 4: 1, 5: 2, 6: 2, 7: 2, 8: 1, 9: 2},
  4: {0: 1, 1: 2, 2: 2, 3: 1, 4: 0, 5: 2, 6: 2, 7: 2, 8: 2, 9: 1},
  5: {0: 1, 1: 2, 2: 2, 3: 2, 4: 2, 5: 0, 6: 2, 7: 1, 8: 1, 9: 2},
  6: {0: 2, 1: 1, 2: 2, 3: 2, 4: 2, 5: 2, 6: 0, 7: 2, 8: 1, 9: 1},
  7: {0: 2, 1: 2, 2: 1, 3: 2, 4: 2, 5: 1, 6: 2, 7: 0, 8: 2, 9: 1},
  8: {0: 2, 1: 2, 2: 2, 3: 1, 4: 2, 5: 1, 6: 1, 7: 2, 8: 0, 9: 2},
  9: {0: 2, 1: 2, 2: 2, 3: 2, 4: 1, 5: 2, 6: 1, 7: 1, 8: 2, 9: 0}},
 {0: {0: None, 1: 0, 2: 1, 3: 4, 4: 0, 5: 0, 6: 1, 7: 5, 8: 5, 9: 4},
  1: {0: 1, 1: None, 2: 1, 3: 2, 4: 0, 5: 0, 6: 1, 7: 2, 8: 6, 9: 6},
  2: {0: 1, 1: 2, 2: None, 3: 2, 4: 3, 5: 7, 6: 1, 7: 2, 8: 3, 9: 7},
  3: {0: 4, 1: 2, 2: 3, 3: None, 4: 3, 5: 8, 6: 8, 7: 2, 8: 3, 9: 4},
  4: {0: 4, 1: 0, 2: 3, 3: 4, 4: None, 5: 0, 6: 9, 7: 9, 8: 3, 9: 4},
  5: {0: 5, 1: 0, 2: 7, 3: 8, 4: 0, 5: None, 6: 8, 7: 5, 8: 5, 9: 7},
  6: {0: 1, 1: 6, 2: 1, 3: 8, 4: 9, 5: 8, 6: None, 7: 9, 8: 6, 9: 6},
  7: {0: 5, 1: 2, 2: 7, 3: 2, 4: 9, 5: 7, 6: 9, 7: None, 8: 5, 9: 7},
  8: {0: 5, 1: 6, 2: 3, 3: 8, 4: 3, 5: 8, 6: 8, 7: 5, 8: None, 9: 6},
  9: {0: 4, 1: 6, 2: 7, 3: 4, 4: 9, 5: 7, 6: 9, 7: 9, 8: 6, 9: None}})
```

sage.graphs.distances_all_pairs.**distances_distribution**(G)

Return the distances distribution of the (di)graph in a dictionary.

This method *ignores all edge labels*, so that the distance considered is the topological distance.

OUTPUT:

A dictionary `d` such that the number of pairs of vertices at distance `k` (if any) is equal to $d[k] \cdot |V(G)| \cdot (|V(G)| - 1)$.

Note

We consider that two vertices that do not belong to the same connected component are at infinite distance, and we do not take the trivial pairs of vertices (v, v) at distance 0 into account. Empty (di)graphs and (di)graphs of order 1 have no paths and so we return the empty dictionary `{}`.

EXAMPLES:

An empty Graph:

```
sage: g = Graph()
sage: g.distances_distribution()
{}
```

```
>>> from sage.all import *
>>> g = Graph()
>>> g.distances_distribution()
{}
```

A Graph of order 1:

```
sage: g = Graph()
sage: g.add_vertex(1)
sage: g.distances_distribution()
{}
```

```
>>> from sage.all import *
>>> g = Graph()
>>> g.add_vertex(Integer(1))
>>> g.distances_distribution()
{}
```

A Graph of order 2 without edge:

```
sage: g = Graph()
sage: g.add_vertices([1,2])
sage: g.distances_distribution()
{+Infinity: 1}
```

```
>>> from sage.all import *
>>> g = Graph()
>>> g.add_vertices([Integer(1),Integer(2)])
>>> g.distances_distribution()
{+Infinity: 1}
```

The Petersen Graph:

```
sage: g = graphs.PetersenGraph()
sage: g.distances_distribution()
{1: 1/3, 2: 2/3}
```

```
>>> from sage.all import *
>>> g = graphs.PetersenGraph()
>>> g.distances_distribution()
{1: 1/3, 2: 2/3}
```

A graph with multiple disconnected components:

```
sage: g = graphs.PetersenGraph()
sage: g.add_edge('good','wine')
sage: g.distances_distribution()
{1: 8/33, 2: 5/11, +Infinity: 10/33}
```

```
>>> from sage.all import *
>>> g = graphs.PetersenGraph()
>>> g.add_edge('good','wine')
>>> g.distances_distribution()
{1: 8/33, 2: 5/11, +Infinity: 10/33}
```

The de Bruijn digraph dB(2,3):

```
sage: D = digraphs.DeBruijn(2,3)                                                #␣
↪needs sage.combinat
sage: D.distances_distribution()                                                #␣
↪needs sage.combinat
{1: 1/4, 2: 11/28, 3: 5/14}
```

```
>>> from sage.all import *
>>> D = digraphs.DeBruijn(Integer(2),Integer(3))                                ␣
↪              # needs sage.combinat
>>> D.distances_distribution()                                                 #␣
↪needs sage.combinat
{1: 1/4, 2: 11/28, 3: 5/14}
```

sage.graphs.distances_all_pairs.**eccentricity**(*G*, *algorithm='standard'*, *vertex_list=None*)

Return the vector of eccentricities in G.

The array returned is of length n, and its i-th component is the eccentricity of the i-th vertex in `G.vertices(sort=True)`.

INPUT:

- `G` – a Graph or a DiGraph
- `algorithm` – string (default: `'standard'`); name of the method used to compute the eccentricity of the vertices
  - `'standard'` – computes eccentricity by performing a BFS from each vertex
  - `'bounds'` – computes eccentricity using the fast algorithm proposed in [TK2013] for undirected graphs
  - `'DHV'` – computes all eccentricities of undirected graph using the algorithm proposed in [Dragan2018]
- `vertex_list` – list (default: `None`); a list of n vertices specifying a mapping from $(0, \ldots, n-1)$ to vertex labels in G. When set, `ecc[i]` is the eccentricity of vertex `vertex_list[i]`. When `vertex_list` is `None`, `ecc[i]` is the eccentricity of vertex `G.vertices(sort=True)[i]`.

EXAMPLES:

```
sage: from sage.graphs.distances_all_pairs import eccentricity
sage: g = graphs.PetersenGraph()
sage: eccentricity(g)
[2, 2, 2, 2, 2, 2, 2, 2, 2, 2]
sage: g.add_edge(0, g.add_vertex())
sage: V = list(g)
sage: eccentricity(g, vertex_list=V)
[2, 2, 3, 3, 2, 2, 3, 3, 3, 3]
sage: eccentricity(g, vertex_list=V[::-1])
[3, 3, 3, 3, 3, 2, 2, 3, 3, 2, 2]
```

```
>>> from sage.all import *
>>> from sage.graphs.distances_all_pairs import eccentricity
>>> g = graphs.PetersenGraph()
>>> eccentricity(g)
[2, 2, 2, 2, 2, 2, 2, 2, 2, 2]
>>> g.add_edge(Integer(0), g.add_vertex())
>>> V = list(g)
>>> eccentricity(g, vertex_list=V)
[2, 2, 3, 3, 2, 2, 3, 3, 3, 3, 3]
>>> eccentricity(g, vertex_list=V[::-Integer(1)])
[3, 3, 3, 3, 3, 2, 2, 3, 3, 2, 2]
```

sage.graphs.distances_all_pairs.**floyd_warshall**(*gg*, *paths=True*, *distances=False*)

Compute the shortest path/distances between all pairs of vertices.

For more information on the Floyd-Warshall algorithm, see the Wikipedia article Floyd-Warshall_algorithm.

INPUT:

- `gg` – the graph on which to work
- `paths` – boolean (default: `True`); whether to return the dictionary of shortest paths
- `distances` – boolean (default: `False`); whether to return the dictionary of distances

OUTPUT:

Depending on the input, this function return the dictionary of paths, the dictionary of distances, or a pair of dictionaries `(distances, paths)` where `distance[u][v]` denotes the distance of a shortest path from u to v and `paths[u][v]` denotes an inneighbor w of v such that $dist(u, v) = 1 + dist(u, w)$.

Warning

Because this function works on matrices whose size is quadratic compared to the number of vertices, it uses short variables instead of long ones to divide by 2 the size in memory. This means that the current implementation does not run on a graph of more than 65536 nodes (this can be easily changed if necessary, but would require much more memory. It may be worth writing two versions). For information, the current version of the algorithm on a graph with $65536 = 2^{16}$ nodes creates in memory 2 tables on 2^{32} short elements (2bytes each), for a total of 2^{34} bytes or 16 gigabytes. Let us also remember that if the memory size is quadratic, the algorithm runs in cubic time.

Note

When `paths = False` the algorithm saves roughly half of the memory as it does not have to maintain the matrix of predecessors. However, setting `distances=False` produces no such effect as the algorithm can not run without computing them. They will not be returned, but they will be stored while the method is running.

EXAMPLES:

Shortest paths in a small grid

```
sage: g = graphs.Grid2dGraph(2,2)
sage: from sage.graphs.distances_all_pairs import floyd_warshall
sage: print(floyd_warshall(g))
```

(continues on next page)

(continued from previous page)

```
{(0, 0): {(0, 0): None, (0, 1): (0, 0), (1, 0): (0, 0), (1, 1): (0, 1)},
 (0, 1): {(0, 1): None, (0, 0): (0, 1), (1, 0): (0, 0), (1, 1): (0, 1)},
 (1, 0): {(1, 0): None, (0, 0): (1, 0), (0, 1): (0, 0), (1, 1): (1, 0)},
 (1, 1): {(1, 1): None, (0, 0): (0, 1), (0, 1): (1, 1), (1, 0): (1, 1)}}
```

```
>>> from sage.all import *
>>> g = graphs.Grid2dGraph(Integer(2),Integer(2))
>>> from sage.graphs.distances_all_pairs import floyd_warshall
>>> print(floyd_warshall(g))
{(0, 0): {(0, 0): None, (0, 1): (0, 0), (1, 0): (0, 0), (1, 1): (0, 1)},
 (0, 1): {(0, 1): None, (0, 0): (0, 1), (1, 0): (0, 0), (1, 1): (0, 1)},
 (1, 0): {(1, 0): None, (0, 0): (1, 0), (0, 1): (0, 0), (1, 1): (1, 0)},
 (1, 1): {(1, 1): None, (0, 0): (0, 1), (0, 1): (1, 1), (1, 0): (1, 1)}}
```

Checking the distances are correct

```
sage: g = graphs.Grid2dGraph(5,5)
sage: dist,path = floyd_warshall(g, distances=True)
sage: all(dist[u][v] == g.distance(u, v) for u in g for v in g)
True
```

```
>>> from sage.all import *
>>> g = graphs.Grid2dGraph(Integer(5),Integer(5))
>>> dist,path = floyd_warshall(g, distances=True)
>>> all(dist[u][v] == g.distance(u, v) for u in g for v in g)
True
```

Checking a random path is valid

```
sage: u,v = g.random_vertex(), g.random_vertex()
sage: p = [v]
sage: while p[0] is not None:
....:   p.insert(0,path[u][p[0]])
sage: len(p) == dist[u][v] + 2
True
```

```
>>> from sage.all import *
>>> u,v = g.random_vertex(), g.random_vertex()
>>> p = [v]
>>> while p[Integer(0)] is not None:
...   p.insert(Integer(0),path[u][p[Integer(0)]])
>>> len(p) == dist[u][v] + Integer(2)
True
```

Distances for all pairs of vertices in a diamond:

```
sage: g = graphs.DiamondGraph()
sage: floyd_warshall(g, paths=False, distances=True)
{0: {0: 0, 1: 1, 2: 1, 3: 2},
 1: {0: 1, 1: 0, 2: 1, 3: 1},
 2: {0: 1, 1: 1, 2: 0, 3: 1},
 3: {0: 2, 1: 1, 2: 1, 3: 0}}
```

```
>>> from sage.all import *
>>> g = graphs.DiamondGraph()
>>> floyd_warshall(g, paths=False, distances=True)
{0: {0: 0, 1: 1, 2: 1, 3: 2},
 1: {0: 1, 1: 0, 2: 1, 3: 1},
 2: {0: 1, 1: 1, 2: 0, 3: 1},
 3: {0: 2, 1: 1, 2: 1, 3: 0}}
```

sage.graphs.distances_all_pairs.**is_distance_regular**(*G*, *parameters=False*)

Test if the graph is distance-regular.

A graph G is distance-regular if for any integers j, k the value of $|\{x : d_G(x,u) = j, x \in V(G)\} \cap \{y : d_G(y,v) = j, y \in V(G)\}|$ is constant for any two vertices $u, v \in V(G)$ at distance i from each other. In particular G is regular, of degree b_0 (see below), as one can take $u = v$.

Equivalently a graph is distance-regular if there exist integers b_i, c_i such that for any two vertices u, v at distance i we have

- $b_i = |\{x : d_G(x,u) = i+1, x \in V(G)\} \cap N_G(v)\}|, \ 0 \leq i \leq d-1$
- $c_i = |\{x : d_G(x,u) = i-1, x \in V(G)\} \cap N_G(v)\}|, \ 1 \leq i \leq d,$

where d is the diameter of the graph. For more information on distance-regular graphs, see the Wikipedia article Distance-regular_graph.

INPUT:

- `parameters` – boolean (default: `False`); if set to `True`, the function returns the pair `(b, c)` of lists of integers instead of a boolean answer (see the definition above)

See also

- *is_regular()*
- *is_strongly_regular()*

EXAMPLES:

```
sage: g = graphs.PetersenGraph()
sage: g.is_distance_regular()
True
sage: g.is_distance_regular(parameters = True)
([3, 2, None], [None, 1, 1])
```

```
>>> from sage.all import *
>>> g = graphs.PetersenGraph()
>>> g.is_distance_regular()
True
>>> g.is_distance_regular(parameters = True)
([3, 2, None], [None, 1, 1])
```

Cube graphs, which are not strongly regular, are a bit more interesting:

```
sage: graphs.CubeGraph(4).is_distance_regular()
True
```

(continues on next page)

(continued from previous page)

```
sage: graphs.OddGraph(5).is_distance_regular()
True
```

```
>>> from sage.all import *
>>> graphs.CubeGraph(Integer(4)).is_distance_regular()
True
>>> graphs.OddGraph(Integer(5)).is_distance_regular()
True
```

Disconnected graph:

```
sage: (2*graphs.CubeGraph(4)).is_distance_regular()
True
```

```
>>> from sage.all import *
>>> (Integer(2)*graphs.CubeGraph(Integer(4))).is_distance_regular()
True
```

sage.graphs.distances_all_pairs.**radius_DHV**(G)

Return the radius of unweighted graph G.

This method computes the radius of unweighted undirected graph using the algorithm given in [Dragan2018].

This method returns Infinity if graph is not connected.

EXAMPLES:

```
sage: from sage.graphs.distances_all_pairs import radius_DHV
sage: G = graphs.PetersenGraph()
sage: radius_DHV(G)
2
sage: G = graphs.RandomGNP(20,0.3)
sage: from sage.graphs.distances_all_pairs import eccentricity
sage: radius_DHV(G) == min(eccentricity(G, algorithm='bounds'))
True
```

```
>>> from sage.all import *
>>> from sage.graphs.distances_all_pairs import radius_DHV
>>> G = graphs.PetersenGraph()
>>> radius_DHV(G)
2
>>> G = graphs.RandomGNP(Integer(20),RealNumber('0.3'))
>>> from sage.graphs.distances_all_pairs import eccentricity
>>> radius_DHV(G) == min(eccentricity(G, algorithm='bounds'))
True
```

sage.graphs.distances_all_pairs.**shortest_path_all_pairs**(G)

Return the matrix of predecessors in G.

The matrix P returned has size n^2, and is such that vertex $P[u, v]$ is a predecessor of v on a shortest uv-path. Hence, this matrix efficiently encodes the information of a shortest uv-path for any $u, v \in G$: indeed, to go from u to v you should first find a shortest $uP[u, v]$-path, then jump from $P[u, v]$ to v as it is one of its outneighbors.

EXAMPLES:

```
sage: from sage.graphs.distances_all_pairs import shortest_path_all_pairs
sage: g = graphs.PetersenGraph()
sage: shortest_path_all_pairs(g)
{0: {0: None, 1: 0, 2: 1, 3: 4, 4: 0, 5: 0, 6: 1, 7: 5, 8: 5, 9: 4},
 1: {0: 1, 1: None, 2: 1, 3: 2, 4: 0, 5: 0, 6: 1, 7: 2, 8: 6, 9: 6},
 2: {0: 1, 1: 2, 2: None, 3: 2, 4: 3, 5: 7, 6: 1, 7: 2, 8: 3, 9: 7},
 3: {0: 4, 1: 2, 2: 3, 3: None, 4: 3, 5: 8, 6: 8, 7: 2, 8: 3, 9: 4},
 4: {0: 4, 1: 0, 2: 3, 3: 4, 4: None, 5: 0, 6: 9, 7: 9, 8: 3, 9: 4},
 5: {0: 5, 1: 0, 2: 7, 3: 8, 4: 0, 5: None, 6: 8, 7: 5, 8: 5, 9: 7},
 6: {0: 1, 1: 6, 2: 1, 3: 8, 4: 9, 5: 8, 6: None, 7: 9, 8: 6, 9: 6},
 7: {0: 5, 1: 2, 2: 7, 3: 2, 4: 9, 5: 7, 6: 9, 7: None, 8: 5, 9: 7},
 8: {0: 5, 1: 6, 2: 3, 3: 8, 4: 3, 5: 8, 6: 8, 7: 5, 8: None, 9: 6},
 9: {0: 4, 1: 6, 2: 7, 3: 4, 4: 9, 5: 7, 6: 9, 7: 9, 8: 6, 9: None}}
```

```
>>> from sage.all import *
>>> from sage.graphs.distances_all_pairs import shortest_path_all_pairs
>>> g = graphs.PetersenGraph()
>>> shortest_path_all_pairs(g)
{0: {0: None, 1: 0, 2: 1, 3: 4, 4: 0, 5: 0, 6: 1, 7: 5, 8: 5, 9: 4},
 1: {0: 1, 1: None, 2: 1, 3: 2, 4: 0, 5: 0, 6: 1, 7: 2, 8: 6, 9: 6},
 2: {0: 1, 1: 2, 2: None, 3: 2, 4: 3, 5: 7, 6: 1, 7: 2, 8: 3, 9: 7},
 3: {0: 4, 1: 2, 2: 3, 3: None, 4: 3, 5: 8, 6: 8, 7: 2, 8: 3, 9: 4},
 4: {0: 4, 1: 0, 2: 3, 3: 4, 4: None, 5: 0, 6: 9, 7: 9, 8: 3, 9: 4},
 5: {0: 5, 1: 0, 2: 7, 3: 8, 4: 0, 5: None, 6: 8, 7: 5, 8: 5, 9: 7},
 6: {0: 1, 1: 6, 2: 1, 3: 8, 4: 9, 5: 8, 6: None, 7: 9, 8: 6, 9: 6},
 7: {0: 5, 1: 2, 2: 7, 3: 2, 4: 9, 5: 7, 6: 9, 7: None, 8: 5, 9: 7},
 8: {0: 5, 1: 6, 2: 3, 3: 8, 4: 3, 5: 8, 6: 8, 7: 5, 8: None, 9: 6},
 9: {0: 4, 1: 6, 2: 7, 3: 4, 4: 9, 5: 7, 6: 9, 7: 9, 8: 6, 9: None}}
```

sage.graphs.distances_all_pairs.**szeged_index**(*G*, *algorithm=None*)

Return the Szeged index of the graph G.

Let $G = (V, E)$ be a connected graph, and for any $uv \in E$, let $N_u(uv) = \{w \in V : d(u, w) < d(v, w)\}$ and $n_u(uv) = |N_u(uv)|$. The Szeged index of G is then defined as [KRG1996]

$$\sum_{uv \in E(G)} n_u(uv) \times n_v(uv)$$

See the Wikipedia article Szeged_index for more details.

INPUT:

- `G` – a Sage graph
- `algorithm` – string (default: `None`); algorithm to use among:
  - `'low'` – algorithm with time complexity in $O(nm)$ and space complexity in $O(m)$. This implementation is currently valid only for simple (without loops or multiple edges) connected graphs.
  - `'high'` – algorithm with time complexity in $O(nm)$ and space complexity in $O(n^2)$. It cannot be used on graphs with more than $65536 = 2^{16}$ vertices.

  By default (`None`), the `'low'` algorithm is used for graphs and the `'high'` algorithm for digraphs.

EXAMPLES:

True for any connected graph [KRG1996]:

```
sage: from sage.graphs.distances_all_pairs import szeged_index
sage: g = graphs.PetersenGraph()
sage: g.wiener_index() <= szeged_index(g)
True
```

```
>>> from sage.all import *
>>> from sage.graphs.distances_all_pairs import szeged_index
>>> g = graphs.PetersenGraph()
>>> g.wiener_index() <= szeged_index(g)
True
```

True for all trees [KRG1996]:

```
sage: g = Graph()
sage: g.add_edges(graphs.CubeGraph(5).min_spanning_tree())
sage: g.wiener_index() == szeged_index(g)
True
```

```
>>> from sage.all import *
>>> g = Graph()
>>> g.add_edges(graphs.CubeGraph(Integer(5)).min_spanning_tree())
>>> g.wiener_index() == szeged_index(g)
True
```

Check that both algorithms return same value:

```
sage: # long time, needs networkx
sage: G = graphs.RandomBarabasiAlbert(100, 2)
sage: a = szeged_index(G, algorithm='low')
sage: b = szeged_index(G, algorithm='high')
sage: a == b
True
```

```
>>> from sage.all import *
>>> # long time, needs networkx
>>> G = graphs.RandomBarabasiAlbert(Integer(100), Integer(2))
>>> a = szeged_index(G, algorithm='low')
>>> b = szeged_index(G, algorithm='high')
>>> a == b
True
```

The Szeged index of a directed circuit of order n is $(n-1)^2$:

```
sage: [digraphs.Circuit(n).szeged_index() for n in range(1, 8)]
[0, 1, 4, 9, 16, 25, 36]
```

```
>>> from sage.all import *
>>> [digraphs.Circuit(n).szeged_index() for n in range(Integer(1), Integer(8))]
[0, 1, 4, 9, 16, 25, 36]
```

sage.graphs.distances_all_pairs.**wiener_index**(G)

Return the Wiener index of the graph.

The Wiener index of an undirected graph G is defined as $W(G) = \frac{1}{2}\sum_{u,v\in G} d(u,v)$ where $d(u,v)$ denotes the distance between vertices u and v (see [KRG1996]).

The Wiener index of a directed graph G is defined as the sum of the distances between each pairs of vertices, $W(G) = \sum_{u,v\in G} d(u,v)$.

EXAMPLES:

From [GYLL1993], cited in [KRG1996]:

```
sage: g=graphs.PathGraph(10)
sage: w=lambda x: (x*(x*x -1)/6)
sage: g.wiener_index()==w(10)
True
```

```
>>> from sage.all import *
>>> g=graphs.PathGraph(Integer(10))
>>> w=lambda x: (x*(x*x -Integer(1))/Integer(6))
>>> g.wiener_index()==w(Integer(10))
True
```

Wiener index of complete (di)graphs:

```
sage: n = 5
sage: g = graphs.CompleteGraph(n)
sage: g.wiener_index() == (n * (n - 1)) / 2
True
sage: g = digraphs.Complete(n)
sage: g.wiener_index() == n * (n - 1)
True
```

```
>>> from sage.all import *
>>> n = Integer(5)
>>> g = graphs.CompleteGraph(n)
>>> g.wiener_index() == (n * (n - Integer(1))) / Integer(2)
True
>>> g = digraphs.Complete(n)
>>> g.wiener_index() == n * (n - Integer(1))
True
```

Wiener index of a graph of order 1:

```
sage: Graph(1).wiener_index()
0
```

```
>>> from sage.all import *
>>> Graph(Integer(1)).wiener_index()
0
```

The Wiener index is not defined on the empty graph:

```
sage: Graph().wiener_index()
Traceback (most recent call last):
...
ValueError: Wiener index is not defined for the empty graph
```

```
>>> from sage.all import *
>>> Graph().wiener_index()
Traceback (most recent call last):
...
ValueError: Wiener index is not defined for the empty graph
```

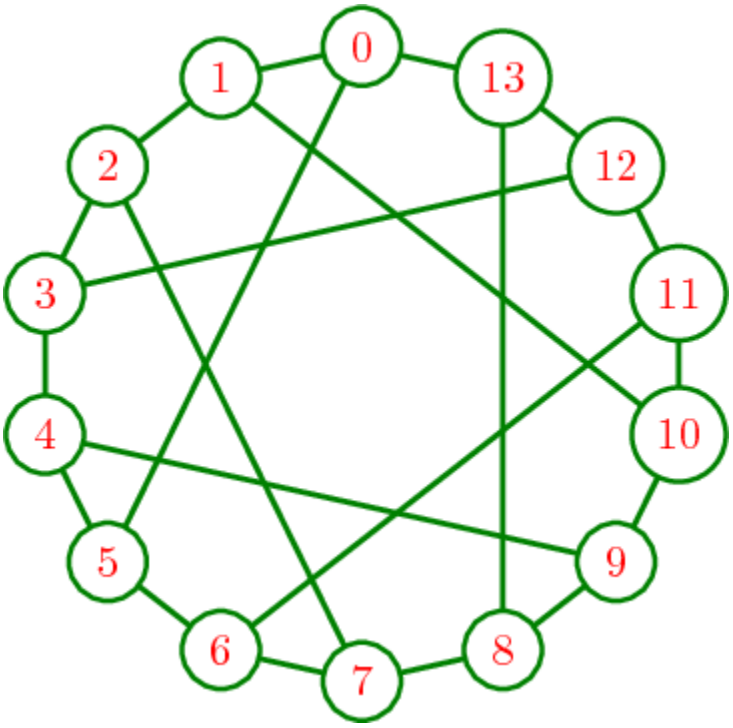
5.36 LaTeX options for graphs

This module provides a class to hold, manipulate and employ various options for rendering a graph in LaTeX, in addition to providing the code that actually generates a LaTeX representation of a (combinatorial) graph.

AUTHORS:

- Rob Beezer (2009-05-20): *GraphLatex* class
- Fidel Barerra Cruz (2009-05-20): `tkz-graph` commands to render a graph
- Nicolas M. Thiéry (2010-02): dot2tex/graphviz interface
- Rob Beezer (2010-05-29): Extended range of `tkz-graph` options

5.36.1 LaTeX Versions of Graphs



Many mathematical objects in Sage have LaTeX representations, and graphs are no exception. For a graph `g`, the command `view(g)`, issued at the Sage command line or in the notebook, will create a graphic version of `g`. Similarly, `latex(g)` will return a (long) string that is a representation of the graph in LaTeX. Other ways of employing LaTeX in Sage, such as `%latex` in a notebook cell, or the Typeset checkbox in the notebook, will handle `g` appropriately.

Support through the `tkz-graph` package is by Alain Matthes, the author of `tkz-graph`, whose work can be found at https://www.ctan.org/pkg/tkz-graph.

The range of possible options for customizing the appearance of a graph are carefully documented at *sage.graphs.graph_latex.GraphLatex.set_option()*. As a broad overview, the following options are supported:

- Pre-built Styles: the pre-built styles of the tkz-graph package provide nice drawings quickly
- Dimensions: can be specified in natural units, then uniformly scaled after design work
- Vertex Colors: the perimeter and fill color for vertices can be specified, including on a per-vertex basis
- Vertex Shapes: may be circles, shaded spheres, rectangles or diamonds, including on a per-vertex basis
- Vertex Sizes: may be specified as minimums, and will automatically sized to contain vertex labels, including on a per-vertex basis
- Vertex Labels: can use latex formatting, and may have their colors specified, including on a per-vertex basis
- Vertex Label Placement: can be interior to the vertex, or external at a configurable location
- Edge Colors: a solid color with or without a second color down the middle, on a per-edge basis
- Edge Thickness: can be set, including on a per-edge basis
- Edge Labels: can use latex formatting, and may have their colors specified, including on a per-edge basis
- Edge Label Placement: can be to the left, right, above, below, inline, and then sloped or horizontal
- Digraph Edges: are slightly curved, with arrowheads
- Loops: may be specified by their size, and with a direction equaling one of the four compass points

To use LaTeX in Sage you of course need a working TeX installation and it will work best if you have the `dvipng` and `magick` utilities. For graphs you need the `tkz-graph.sty` and `tkz-berge.sty` style files of the tkz-graph package. TeX, dvipng, and convert should be widely available through package managers or installers. You may need to install the tkz-graph style files in the appropriate locations, a task beyond the scope of this introduction. Primary locations for these programs are:

- TeX: http://ctan.org/
- dvipng: http://sourceforge.net/projects/dvipng/
- magick: http://www.imagemagick.org (the ImageMagick suite)
- tkz-graph: https://www.ctan.org/pkg/tkz-graph

Customizing the output is accomplished in several ways. Suppose `g` is a graph, then `g.set_latex_options()` can be used to efficiently set or modify various options. Setting individual options, or querying options, can be accomplished by first using a command like `opts = g.latex_options()` to obtain a *sage.graphs.graph_latex.GraphLatex* object which has several methods to set and retrieve options.

Here is a minimal session demonstrating how to use these features. The following setup should work in the notebook or at the command-line.:

```
sage: H = graphs.HeawoodGraph()
sage: H.set_latex_options(
....:     graphic_size=(5,5),
....:     vertex_size=0.2,
....:     edge_thickness=0.04,
....:     edge_color='green',
....:     vertex_color='green',
....:     vertex_label_color='red'
....: )
```

```
>>> from sage.all import *
>>> H = graphs.HeawoodGraph()
>>> H.set_latex_options(
...     graphic_size=(Integer(5),Integer(5)),
...     vertex_size=RealNumber('0.2'),
...     edge_thickness=RealNumber('0.04'),
...     edge_color='green',
...     vertex_color='green',
...     vertex_label_color='red'
... )
```

At this point, `view(H)` should call `pdflatex` to process the string created by `latex(H)` and then display the resulting graphic.

To use this image in a LaTeX document, you could of course just copy and save the resulting graphic. However, the `latex()` command will produce the underlying LaTeX code, which can be incorporated into a standalone LaTeX document.:

```
sage: from sage.graphs.graph_latex import check_tkz_graph
sage: check_tkz_graph()  # random - depends on TeX installation
sage: latex(H)
\begin{tikzpicture}
\definecolor{cv0}{rgb}{0.0,0.502,0.0}
\definecolor{cfv0}{rgb}{1.0,1.0,1.0}
\definecolor{clv0}{rgb}{1.0,0.0,0.0}
\definecolor{cv1}{rgb}{0.0,0.502,0.0}
\definecolor{cfv1}{rgb}{1.0,1.0,1.0}
\definecolor{clv1}{rgb}{1.0,0.0,0.0}
\definecolor{cv2}{rgb}{0.0,0.502,0.0}
\definecolor{cfv2}{rgb}{1.0,1.0,1.0}
\definecolor{clv2}{rgb}{1.0,0.0,0.0}
\definecolor{cv3}{rgb}{0.0,0.502,0.0}
\definecolor{cfv3}{rgb}{1.0,1.0,1.0}
\definecolor{clv3}{rgb}{1.0,0.0,0.0}
\definecolor{cv4}{rgb}{0.0,0.502,0.0}
\definecolor{cfv4}{rgb}{1.0,1.0,1.0}
\definecolor{clv4}{rgb}{1.0,0.0,0.0}
\definecolor{cv5}{rgb}{0.0,0.502,0.0}
\definecolor{cfv5}{rgb}{1.0,1.0,1.0}
\definecolor{clv5}{rgb}{1.0,0.0,0.0}
\definecolor{cv6}{rgb}{0.0,0.502,0.0}
\definecolor{cfv6}{rgb}{1.0,1.0,1.0}
\definecolor{clv6}{rgb}{1.0,0.0,0.0}
\definecolor{cv7}{rgb}{0.0,0.502,0.0}
\definecolor{cfv7}{rgb}{1.0,1.0,1.0}
\definecolor{clv7}{rgb}{1.0,0.0,0.0}
\definecolor{cv8}{rgb}{0.0,0.502,0.0}
\definecolor{cfv8}{rgb}{1.0,1.0,1.0}
\definecolor{clv8}{rgb}{1.0,0.0,0.0}
\definecolor{cv9}{rgb}{0.0,0.502,0.0}
\definecolor{cfv9}{rgb}{1.0,1.0,1.0}
\definecolor{clv9}{rgb}{1.0,0.0,0.0}
\definecolor{cv10}{rgb}{0.0,0.502,0.0}
```

(continues on next page)

(continued from previous page)

```
\definecolor{cfv10}{rgb}{1.0,1.0,1.0}
\definecolor{clv10}{rgb}{1.0,0.0,0.0}
\definecolor{cv11}{rgb}{0.0,0.502,0.0}
\definecolor{cfv11}{rgb}{1.0,1.0,1.0}
\definecolor{clv11}{rgb}{1.0,0.0,0.0}
\definecolor{cv12}{rgb}{0.0,0.502,0.0}
\definecolor{cfv12}{rgb}{1.0,1.0,1.0}
\definecolor{clv12}{rgb}{1.0,0.0,0.0}
\definecolor{cv13}{rgb}{0.0,0.502,0.0}
\definecolor{cfv13}{rgb}{1.0,1.0,1.0}
\definecolor{clv13}{rgb}{1.0,0.0,0.0}
\definecolor{cv0v1}{rgb}{0.0,0.502,0.0}
\definecolor{cv0v5}{rgb}{0.0,0.502,0.0}
\definecolor{cv0v13}{rgb}{0.0,0.502,0.0}
\definecolor{cv1v2}{rgb}{0.0,0.502,0.0}
\definecolor{cv1v10}{rgb}{0.0,0.502,0.0}
\definecolor{cv2v3}{rgb}{0.0,0.502,0.0}
\definecolor{cv2v7}{rgb}{0.0,0.502,0.0}
\definecolor{cv3v4}{rgb}{0.0,0.502,0.0}
\definecolor{cv3v12}{rgb}{0.0,0.502,0.0}
\definecolor{cv4v5}{rgb}{0.0,0.502,0.0}
\definecolor{cv4v9}{rgb}{0.0,0.502,0.0}
\definecolor{cv5v6}{rgb}{0.0,0.502,0.0}
\definecolor{cv6v7}{rgb}{0.0,0.502,0.0}
\definecolor{cv6v11}{rgb}{0.0,0.502,0.0}
\definecolor{cv7v8}{rgb}{0.0,0.502,0.0}
\definecolor{cv8v9}{rgb}{0.0,0.502,0.0}
\definecolor{cv8v13}{rgb}{0.0,0.502,0.0}
\definecolor{cv9v10}{rgb}{0.0,0.502,0.0}
\definecolor{cv10v11}{rgb}{0.0,0.502,0.0}
\definecolor{cv11v12}{rgb}{0.0,0.502,0.0}
\definecolor{cv12v13}{rgb}{0.0,0.502,0.0}
%
\begin{scope}[VertexStyle/.append style={minimum size=0.2cm,draw=cv0,fill=cfv0,
↪text=clv0,shape=circle}]
\Vertex[LabelOut=false,L=\hbox{$0$},x=2.5cm,y=5.0cm]{v0}
\end{scope}
...
\begin{scope}[VertexStyle/.append style={minimum size=0.2cm,draw=cv13,fill=cfv13,
↪text=clv13,shape=circle}]
\Vertex[LabelOut=false,L=\hbox{$13$},x=3.6126cm,y=4.7524cm]{v13}
\end{scope}
%
\Edge[lw=0.04cm,style={color=cv0v1,},](v0)(v1)
\Edge[lw=0.04cm,style={color=cv0v5,},](v0)(v5)
\Edge[lw=0.04cm,style={color=cv0v13,},](v0)(v13)
\Edge[lw=0.04cm,style={color=cv1v2,},](v1)(v2)
\Edge[lw=0.04cm,style={color=cv1v10,},](v1)(v10)
\Edge[lw=0.04cm,style={color=cv2v3,},](v2)(v3)
\Edge[lw=0.04cm,style={color=cv2v7,},](v2)(v7)
\Edge[lw=0.04cm,style={color=cv3v4,},](v3)(v4)
\Edge[lw=0.04cm,style={color=cv3v12,},](v3)(v12)
```

(continues on next page)

(continued from previous page)

```
\Edge[lw=0.04cm,style={color=cv4v5,},](v4)(v5)
\Edge[lw=0.04cm,style={color=cv4v9,},](v4)(v9)
\Edge[lw=0.04cm,style={color=cv5v6,},](v5)(v6)
\Edge[lw=0.04cm,style={color=cv6v7,},](v6)(v7)
\Edge[lw=0.04cm,style={color=cv6v11,},](v6)(v11)
\Edge[lw=0.04cm,style={color=cv7v8,},](v7)(v8)
\Edge[lw=0.04cm,style={color=cv8v9,},](v8)(v9)
\Edge[lw=0.04cm,style={color=cv8v13,},](v8)(v13)
\Edge[lw=0.04cm,style={color=cv9v10,},](v9)(v10)
\Edge[lw=0.04cm,style={color=cv10v11,},](v10)(v11)
\Edge[lw=0.04cm,style={color=cv11v12,},](v11)(v12)
\Edge[lw=0.04cm,style={color=cv12v13,},](v12)(v13)
%
\end{tikzpicture}
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_latex import check_tkz_graph
>>> check_tkz_graph()  # random - depends on TeX installation
>>> latex(H)
\begin{tikzpicture}
\definecolor{cv0}{rgb}{0.0,0.502,0.0}
\definecolor{cfv0}{rgb}{1.0,1.0,1.0}
\definecolor{clv0}{rgb}{1.0,0.0,0.0}
\definecolor{cv1}{rgb}{0.0,0.502,0.0}
\definecolor{cfv1}{rgb}{1.0,1.0,1.0}
\definecolor{clv1}{rgb}{1.0,0.0,0.0}
\definecolor{cv2}{rgb}{0.0,0.502,0.0}
\definecolor{cfv2}{rgb}{1.0,1.0,1.0}
\definecolor{clv2}{rgb}{1.0,0.0,0.0}
\definecolor{cv3}{rgb}{0.0,0.502,0.0}
\definecolor{cfv3}{rgb}{1.0,1.0,1.0}
\definecolor{clv3}{rgb}{1.0,0.0,0.0}
\definecolor{cv4}{rgb}{0.0,0.502,0.0}
\definecolor{cfv4}{rgb}{1.0,1.0,1.0}
\definecolor{clv4}{rgb}{1.0,0.0,0.0}
\definecolor{cv5}{rgb}{0.0,0.502,0.0}
\definecolor{cfv5}{rgb}{1.0,1.0,1.0}
\definecolor{clv5}{rgb}{1.0,0.0,0.0}
\definecolor{cv6}{rgb}{0.0,0.502,0.0}
\definecolor{cfv6}{rgb}{1.0,1.0,1.0}
\definecolor{clv6}{rgb}{1.0,0.0,0.0}
\definecolor{cv7}{rgb}{0.0,0.502,0.0}
\definecolor{cfv7}{rgb}{1.0,1.0,1.0}
\definecolor{clv7}{rgb}{1.0,0.0,0.0}
\definecolor{cv8}{rgb}{0.0,0.502,0.0}
\definecolor{cfv8}{rgb}{1.0,1.0,1.0}
\definecolor{clv8}{rgb}{1.0,0.0,0.0}
\definecolor{cv9}{rgb}{0.0,0.502,0.0}
\definecolor{cfv9}{rgb}{1.0,1.0,1.0}
\definecolor{clv9}{rgb}{1.0,0.0,0.0}
\definecolor{cv10}{rgb}{0.0,0.502,0.0}
\definecolor{cfv10}{rgb}{1.0,1.0,1.0}
```

(continues on next page)

(continued from previous page)

```
\definecolor{clv10}{rgb}{1.0,0.0,0.0}
\definecolor{cv11}{rgb}{0.0,0.502,0.0}
\definecolor{cfv11}{rgb}{1.0,1.0,1.0}
\definecolor{clv11}{rgb}{1.0,0.0,0.0}
\definecolor{cv12}{rgb}{0.0,0.502,0.0}
\definecolor{cfv12}{rgb}{1.0,1.0,1.0}
\definecolor{clv12}{rgb}{1.0,0.0,0.0}
\definecolor{cv13}{rgb}{0.0,0.502,0.0}
\definecolor{cfv13}{rgb}{1.0,1.0,1.0}
\definecolor{clv13}{rgb}{1.0,0.0,0.0}
\definecolor{cv0v1}{rgb}{0.0,0.502,0.0}
\definecolor{cv0v5}{rgb}{0.0,0.502,0.0}
\definecolor{cv0v13}{rgb}{0.0,0.502,0.0}
\definecolor{cv1v2}{rgb}{0.0,0.502,0.0}
\definecolor{cv1v10}{rgb}{0.0,0.502,0.0}
\definecolor{cv2v3}{rgb}{0.0,0.502,0.0}
\definecolor{cv2v7}{rgb}{0.0,0.502,0.0}
\definecolor{cv3v4}{rgb}{0.0,0.502,0.0}
\definecolor{cv3v12}{rgb}{0.0,0.502,0.0}
\definecolor{cv4v5}{rgb}{0.0,0.502,0.0}
\definecolor{cv4v9}{rgb}{0.0,0.502,0.0}
\definecolor{cv5v6}{rgb}{0.0,0.502,0.0}
\definecolor{cv6v7}{rgb}{0.0,0.502,0.0}
\definecolor{cv6v11}{rgb}{0.0,0.502,0.0}
\definecolor{cv7v8}{rgb}{0.0,0.502,0.0}
\definecolor{cv8v9}{rgb}{0.0,0.502,0.0}
\definecolor{cv8v13}{rgb}{0.0,0.502,0.0}
\definecolor{cv9v10}{rgb}{0.0,0.502,0.0}
\definecolor{cv10v11}{rgb}{0.0,0.502,0.0}
\definecolor{cv11v12}{rgb}{0.0,0.502,0.0}
\definecolor{cv12v13}{rgb}{0.0,0.502,0.0}
%
\begin{scope}[VertexStyle/.append style={minimum size=0.2cm,draw=cv0,fill=cfv0,
↪text=clv0,shape=circle}]
\Vertex[LabelOut=false,L=\hbox{$0$},x=2.5cm,y=5.0cm]{v0}
\end{scope}
...
\begin{scope}[VertexStyle/.append style={minimum size=0.2cm,draw=cv13,fill=cfv13,
↪text=clv13,shape=circle}]
\Vertex[LabelOut=false,L=\hbox{$13$},x=3.6126cm,y=4.7524cm]{v13}
\end{scope}
%
\Edge[lw=0.04cm,style={color=cv0v1,},](v0)(v1)
\Edge[lw=0.04cm,style={color=cv0v5,},](v0)(v5)
\Edge[lw=0.04cm,style={color=cv0v13,},](v0)(v13)
\Edge[lw=0.04cm,style={color=cv1v2,},](v1)(v2)
\Edge[lw=0.04cm,style={color=cv1v10,},](v1)(v10)
\Edge[lw=0.04cm,style={color=cv2v3,},](v2)(v3)
\Edge[lw=0.04cm,style={color=cv2v7,},](v2)(v7)
\Edge[lw=0.04cm,style={color=cv3v4,},](v3)(v4)
\Edge[lw=0.04cm,style={color=cv3v12,},](v3)(v12)
\Edge[lw=0.04cm,style={color=cv4v5,},](v4)(v5)
```

(continues on next page)

(continued from previous page)

```
\Edge[lw=0.04cm,style={color=cv4v9,},](v4)(v9)
\Edge[lw=0.04cm,style={color=cv5v6,},](v5)(v6)
\Edge[lw=0.04cm,style={color=cv6v7,},](v6)(v7)
\Edge[lw=0.04cm,style={color=cv6v11,},](v6)(v11)
\Edge[lw=0.04cm,style={color=cv7v8,},](v7)(v8)
\Edge[lw=0.04cm,style={color=cv8v9,},](v8)(v9)
\Edge[lw=0.04cm,style={color=cv8v13,},](v8)(v13)
\Edge[lw=0.04cm,style={color=cv9v10,},](v9)(v10)
\Edge[lw=0.04cm,style={color=cv10v11,},](v10)(v11)
\Edge[lw=0.04cm,style={color=cv11v12,},](v11)(v12)
\Edge[lw=0.04cm,style={color=cv12v13,},](v12)(v13)
%
\end{tikzpicture}
```

EXAMPLES:

This example illustrates switching between the built-in styles when using the tkz_graph format.:

```
sage: g = graphs.PetersenGraph()
sage: g.set_latex_options(tkz_style='Classic')
sage: from sage.graphs.graph_latex import check_tkz_graph
sage: check_tkz_graph()  # random - depends on TeX installation
sage: latex(g)
\begin{tikzpicture}
\GraphInit[vstyle=Classic]
...
\end{tikzpicture}
sage: opts = g.latex_options()
sage: opts
LaTeX options for Petersen graph: {'tkz_style': 'Classic'}
sage: g.set_latex_options(tkz_style = 'Art')
sage: opts.get_option('tkz_style')
'Art'
sage: opts
LaTeX options for Petersen graph: {'tkz_style': 'Art'}
sage: latex(g)
\begin{tikzpicture}
\GraphInit[vstyle=Art]
...
\end{tikzpicture}
```

```
>>> from sage.all import *
>>> g = graphs.PetersenGraph()
>>> g.set_latex_options(tkz_style='Classic')
>>> from sage.graphs.graph_latex import check_tkz_graph
>>> check_tkz_graph()  # random - depends on TeX installation
>>> latex(g)
\begin{tikzpicture}
\GraphInit[vstyle=Classic]
...
\end{tikzpicture}
>>> opts = g.latex_options()
```

(continues on next page)

(continued from previous page)

```
>>> opts
LaTeX options for Petersen graph: {'tkz_style': 'Classic'}
>>> g.set_latex_options(tkz_style = 'Art')
>>> opts.get_option('tkz_style')
'Art'
>>> opts
LaTeX options for Petersen graph: {'tkz_style': 'Art'}
>>> latex(g)
\begin{tikzpicture}
\GraphInit[vstyle=Art]
...
\end{tikzpicture}
```

This example illustrates using the optional dot2tex module:

```
sage: g = graphs.PetersenGraph()
sage: g.set_latex_options(format='dot2tex', prog='neato')
sage: from sage.graphs.graph_latex import check_tkz_graph
sage: check_tkz_graph()  # random - depends on TeX installation
sage: latex(g)  # optional - dot2tex graphviz
\begin{tikzpicture}[>=latex,line join=bevel,]
...
\end{tikzpicture}
```

```
>>> from sage.all import *
>>> g = graphs.PetersenGraph()
>>> g.set_latex_options(format='dot2tex', prog='neato')
>>> from sage.graphs.graph_latex import check_tkz_graph
>>> check_tkz_graph()  # random - depends on TeX installation
>>> latex(g)  # optional - dot2tex graphviz
\begin{tikzpicture}[>=latex,line join=bevel,]
...
\end{tikzpicture}
```

Among other things, this supports the flexible `edge_options` option (see *sage.graphs.generic_graph.GenericGraph.graphviz_string()*); here we color in red all edges touching the vertex `0`:

```
sage: g = graphs.PetersenGraph()
sage: g.set_latex_options(format='dot2tex', edge_options=lambda u_v_label: {"color":
↪"red"} if u_v_label[0] == 0 else {})
sage: latex(g)  # optional - dot2tex graphviz
\begin{tikzpicture}[>=latex,line join=bevel,]
...
\end{tikzpicture}
```

```
>>> from sage.all import *
>>> g = graphs.PetersenGraph()
>>> g.set_latex_options(format='dot2tex', edge_options=lambda u_v_label: {"color":
↪"red"} if u_v_label[Integer(0)] == Integer(0) else {})
>>> latex(g)  # optional - dot2tex graphviz
\begin{tikzpicture}[>=latex,line join=bevel,]
...
```

(continues on next page)

(continued from previous page)

```
\end{tikzpicture}
```

5.36.2 GraphLatex class and functions

class sage.graphs.graph_latex.**GraphLatex**(*graph*, ***options*)

Bases: SageObject

A class to hold, manipulate and employ options for converting a graph to LaTeX.

This class serves two purposes. First it holds the values of various options designed to work with the `tkz-graph` LaTeX package for rendering graphs. As such, a graph that uses this class will hold a reference to it. Second, this class contains the code to convert a graph into the corresponding LaTeX constructs, returning a string.

EXAMPLES:

```
sage: from sage.graphs.graph_latex import GraphLatex
sage: opts = GraphLatex(graphs.PetersenGraph())
sage: opts
LaTeX options for Petersen graph: {}
sage: g = graphs.PetersenGraph()
sage: opts = g.latex_options()
sage: g == loads(dumps(g))
True
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_latex import GraphLatex
>>> opts = GraphLatex(graphs.PetersenGraph())
>>> opts
LaTeX options for Petersen graph: {}
>>> g = graphs.PetersenGraph()
>>> opts = g.latex_options()
>>> g == loads(dumps(g))
True
```

dot2tex_picture()

Call `dot2tex` to construct a string of LaTeX commands representing a graph as a `tikzpicture`.

EXAMPLES:

```
sage: g = digraphs.ButterflyGraph(1)
sage: from sage.graphs.graph_latex import check_tkz_graph
sage: check_tkz_graph()  # random - depends on TeX installation
sage: print(g.latex_options().dot2tex_picture())  # optional - dot2tex␣
↪graphviz
\begin{tikzpicture}[>=latex,line join=bevel,]
%%
  \node (node_...) at (...bp,...bp) [draw,draw=none] {$\left(...\right)$};
  \node (node_...) at (...bp,...bp) [draw,draw=none] {$\left(...\right)$};
  \node (node_...) at (...bp,...bp) [draw,draw=none] {$\left(...\right)$};
  \node (node_...) at (...bp,...bp) [draw,draw=none] {$\left(...\right)$};
  \draw [black,->] (node_...) ..controls (...bp,...bp) and (...bp,...bp)  ..␣
↪(node_...);
  \draw [black,->] (node_...) ..controls (...bp,...bp) and (...bp,...bp)  ..␣
```

(continues on next page)

(continued from previous page)

```
↪(node_...);
  \draw [black,->] (node_...) ..controls (...bp,...bp) and (...bp,...bp)  ..␣
↪(node_...);
  \draw [black,->] (node_...) ..controls (...bp,...bp) and (...bp,...bp)  ..␣
↪(node_...);
%
\end{tikzpicture}
```

```
>>> from sage.all import *
>>> g = digraphs.ButterflyGraph(Integer(1))
>>> from sage.graphs.graph_latex import check_tkz_graph
>>> check_tkz_graph()  # random - depends on TeX installation
>>> print(g.latex_options().dot2tex_picture())  # optional - dot2tex graphviz
\begin{tikzpicture}[>=latex,line join=bevel,]
%%
  \node (node_...) at (...bp,...bp) [draw,draw=none] {$\left(...\right)$};
  \node (node_...) at (...bp,...bp) [draw,draw=none] {$\left(...\right)$};
  \node (node_...) at (...bp,...bp) [draw,draw=none] {$\left(...\right)$};
  \node (node_...) at (...bp,...bp) [draw,draw=none] {$\left(...\right)$};
  \draw [black,->] (node_...) ..controls (...bp,...bp) and (...bp,...bp)  ..␣
↪(node_...);
  \draw [black,->] (node_...) ..controls (...bp,...bp) and (...bp,...bp)  ..␣
↪(node_...);
  \draw [black,->] (node_...) ..controls (...bp,...bp) and (...bp,...bp)  ..␣
↪(node_...);
  \draw [black,->] (node_...) ..controls (...bp,...bp) and (...bp,...bp)  ..␣
↪(node_...);
%
\end{tikzpicture}
```

We make sure Issue #13624 is fixed:

```
sage: G = DiGraph()
sage: G.add_edge(3333, 88, 'my_label')
sage: G.set_latex_options(edge_labels=True)
sage: print(G.latex_options().dot2tex_picture())  # optional - dot2tex␣
↪graphviz
\begin{tikzpicture}[>=latex,line join=bevel,]
%%
\node (node_...) at (...bp,...bp) [draw,draw=none] {$...$};
  \node (node_...) at (...bp,...bp) [draw,draw=none] {$...$};
  \draw [black,->] (node_...) ..controls (...bp,...bp) and (...bp,...bp)  ..␣
↪(node_...);
  \definecolor{strokecol}{rgb}{0.0,0.0,0.0}
  \pgfsetstrokecolor{strokecol}
  \draw (...bp,...bp) node {$\text{\texttt{my{\char`\_}label}}$};
%
\end{tikzpicture}
```

```
>>> from sage.all import *
>>> G = DiGraph()
>>> G.add_edge(Integer(3333), Integer(88), 'my_label')
```

(continues on next page)

(continued from previous page)

```
>>> G.set_latex_options(edge_labels=True)
>>> print(G.latex_options().dot2tex_picture())  # optional - dot2tex graphviz
\begin{tikzpicture}[>=latex,line join=bevel,]
%%
\node (node_...) at (...bp,...bp) [draw,draw=none] {$...$};
  \node (node_...) at (...bp,...bp) [draw,draw=none] {$...$};
  \draw [black,->] (node_...) ..controls (...bp,...bp) and (...bp,...bp)  ..␣
↪(node_...);
  \definecolor{strokecol}{rgb}{0.0,0.0,0.0}
  \pgfsetstrokecolor{strokecol}
  \draw (...bp,...bp) node {$\text{\texttt{my{\char`\_}label}}$};
%
\end{tikzpicture}
```

Check that Issue #25120 is fixed:

```
sage: G = Graph([(0,1)])
sage: G.set_latex_options(edge_colors = {(0,1): 'red'})
sage: print(G.latex_options().dot2tex_picture())  # optional - dot2tex␣
↪graphviz
\begin{tikzpicture}[>=latex,line join=bevel,]
...
\draw [red,] (node_0) ... (node_1);
...
\end{tikzpicture}
```

```
>>> from sage.all import *
>>> G = Graph([(Integer(0),Integer(1))])
>>> G.set_latex_options(edge_colors = {(Integer(0),Integer(1)): 'red'})
>>> print(G.latex_options().dot2tex_picture())  # optional - dot2tex graphviz
\begin{tikzpicture}[>=latex,line join=bevel,]
...
\draw [red,] (node_0) ... (node_1);
...
\end{tikzpicture}
```

Note

There is a lot of overlap between what `tkz_picture` and `dot2tex` do. It would be best to merge them! `dot2tex` probably can work without `graphviz` if layout information is provided.

get_option(*option_name*)

Return the current value of the named option.

INPUT:

- `option_name` – the name of an option

OUTPUT:

If the name is not present in `__graphlatex_options` it is an error to ask for it. If an option has not been set then the default value is returned. Otherwise, the value of the option is returned.

EXAMPLES:

```
sage: g = graphs.PetersenGraph()
sage: opts = g.latex_options()
sage: opts.set_option('tkz_style', 'Art')
sage: opts.get_option('tkz_style')
'Art'
sage: opts.set_option('tkz_style')
sage: opts.get_option('tkz_style') == "Custom"
True
sage: opts.get_option('bad_name')
Traceback (most recent call last):
...
ValueError: bad_name is not a Latex option for a graph.
```

```
>>> from sage.all import *
>>> g = graphs.PetersenGraph()
>>> opts = g.latex_options()
>>> opts.set_option('tkz_style', 'Art')
>>> opts.get_option('tkz_style')
'Art'
>>> opts.set_option('tkz_style')
>>> opts.get_option('tkz_style') == "Custom"
True
>>> opts.get_option('bad_name')
Traceback (most recent call last):
...
ValueError: bad_name is not a Latex option for a graph.
```

latex()

Return a string in LaTeX representing a graph.

This is the command that is invoked by `sage.graphs.generic_graph.GenericGraph._latex_` for a graph, so it returns a string of LaTeX commands that can be incorporated into a LaTeX document unmodified. The exact contents of this string are influenced by the options set via the methods *sage.graphs.generic_graph.GenericGraph.set_latex_options()*, *set_option()*, and *set_options()*.

By setting the `format` option different packages can be used to create the latex version of a graph. Supported packages are `tkz-graph` and `dot2tex`.

EXAMPLES:

```
sage: from sage.graphs.graph_latex import check_tkz_graph
sage: check_tkz_graph()  # random - depends on TeX installation
sage: g = graphs.CompleteGraph(2)
sage: opts = g.latex_options()
sage: print(opts.latex())
\begin{tikzpicture}
\definecolor{cv0}{rgb}{0.0,0.0,0.0}
\definecolor{cfv0}{rgb}{1.0,1.0,1.0}
\definecolor{clv0}{rgb}{0.0,0.0,0.0}
\definecolor{cv1}{rgb}{0.0,0.0,0.0}
\definecolor{cfv1}{rgb}{1.0,1.0,1.0}
\definecolor{clv1}{rgb}{0.0,0.0,0.0}
\definecolor{cv0v1}{rgb}{0.0,0.0,0.0}
%
```

(continues on next page)

(continued from previous page)

```
\begin{scope}[VertexStyle/.append style={minimum size=1.0cm,draw=cv0,
↪fill=cfv0,text=clv0,shape=circle}]
\Vertex[LabelOut=false,L=\hbox{$0$},x=2.5cm,y=5.0cm]{v0}
\end{scope}
\begin{scope}[VertexStyle/.append style={minimum size=1.0cm,draw=cv1,
↪fill=cfv1,text=clv1,shape=circle}]
\Vertex[LabelOut=false,L=\hbox{$1$},x=2.5cm,y=0.0cm]{v1}
\end{scope}
%
\Edge[lw=0.1cm,style={color=cv0v1,},](v0)(v1)
%
\end{tikzpicture}
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_latex import check_tkz_graph
>>> check_tkz_graph()  # random - depends on TeX installation
>>> g = graphs.CompleteGraph(Integer(2))
>>> opts = g.latex_options()
>>> print(opts.latex())
\begin{tikzpicture}
\definecolor{cv0}{rgb}{0.0,0.0,0.0}
\definecolor{cfv0}{rgb}{1.0,1.0,1.0}
\definecolor{clv0}{rgb}{0.0,0.0,0.0}
\definecolor{cv1}{rgb}{0.0,0.0,0.0}
\definecolor{cfv1}{rgb}{1.0,1.0,1.0}
\definecolor{clv1}{rgb}{0.0,0.0,0.0}
\definecolor{cv0v1}{rgb}{0.0,0.0,0.0}
%
\begin{scope}[VertexStyle/.append style={minimum size=1.0cm,draw=cv0,
↪fill=cfv0,text=clv0,shape=circle}]
\Vertex[LabelOut=false,L=\hbox{$0$},x=2.5cm,y=5.0cm]{v0}
\end{scope}
\begin{scope}[VertexStyle/.append style={minimum size=1.0cm,draw=cv1,
↪fill=cfv1,text=clv1,shape=circle}]
\Vertex[LabelOut=false,L=\hbox{$1$},x=2.5cm,y=0.0cm]{v1}
\end{scope}
%
\Edge[lw=0.1cm,style={color=cv0v1,},](v0)(v1)
%
\end{tikzpicture}
```

We check that Issue #22070 is fixed:

```
sage: edges = [(i,(i+1)%3,a) for i,a in enumerate('abc')]
sage: G_with_labels = DiGraph(edges)
sage: C = [[0,1], [2]]
sage: kwds = dict(subgraph_clusters=C,color_by_label=True,prog='dot',format=
↪'dot2tex')
sage: opts = G_with_labels.latex_options()
sage: opts.set_options(edge_labels=True, **kwds)  # optional - dot2tex␣
↪graphviz
sage: latex(G_with_labels)                        # optional - dot2tex␣
```

(continues on next page)

(continued from previous page)

```
↪graphviz
\begin{tikzpicture}[>=latex,line join=bevel,]
%%
\begin{scope}
  \pgfsetstrokecolor{black}
  \definecolor{strokecol}{rgb}{...}
  \pgfsetstrokecolor{strokecol}
  \definecolor{fillcol}{rgb}{...}
  \pgfsetfillcolor{fillcol}
  \filldraw ... cycle;
\end{scope}
\begin{scope}
  \pgfsetstrokecolor{black}
  \definecolor{strokecol}{rgb}{...}
  \pgfsetstrokecolor{strokecol}
  \definecolor{fillcol}{rgb}{...}
  \pgfsetfillcolor{fillcol}
  \filldraw ... cycle;
\end{scope}
...
\end{tikzpicture}
```

```
>>> from sage.all import *
>>> edges = [(i,(i+Integer(1))%Integer(3),a) for i,a in enumerate('abc')]
>>> G_with_labels = DiGraph(edges)
>>> C = [[Integer(0),Integer(1)], [Integer(2)]]
>>> kwds = dict(subgraph_clusters=C,color_by_label=True,prog='dot',format=
↪'dot2tex')
>>> opts = G_with_labels.latex_options()
>>> opts.set_options(edge_labels=True, **kwds)  # optional - dot2tex graphviz
>>> latex(G_with_labels)                        # optional - dot2tex graphviz
\begin{tikzpicture}[>=latex,line join=bevel,]
%%
\begin{scope}
  \pgfsetstrokecolor{black}
  \definecolor{strokecol}{rgb}{...}
  \pgfsetstrokecolor{strokecol}
  \definecolor{fillcol}{rgb}{...}
  \pgfsetfillcolor{fillcol}
  \filldraw ... cycle;
\end{scope}
\begin{scope}
  \pgfsetstrokecolor{black}
  \definecolor{strokecol}{rgb}{...}
  \pgfsetstrokecolor{strokecol}
  \definecolor{fillcol}{rgb}{...}
  \pgfsetfillcolor{fillcol}
  \filldraw ... cycle;
\end{scope}
...
\end{tikzpicture}
```

set_option(*option_name*, *option_value=None*)

Set, modify, clear a LaTeX option for controlling the rendering of a graph.

The possible options are documented here, because ultimately it is this routine that sets the values. However, the *sage.graphs.generic_graph.GenericGraph.set_latex_options()* method is the easiest way to set options, and allows several to be set at once.

INPUT:

- option_name – string for a latex option contained in the list sage.graphs.graph_latex.GraphLatex.__graphlatex_options; a ValueError is raised if the option is not allowed
- option_value – a value for the option; if omitted, or set to None, the option will use the default value

The output can be either handled internally by Sage, or delegated to the external software dot2tex and graphviz. This is controlled by the option format:

- format – string (default: 'tkz_graph'); either 'dot2tex' or 'tkz_graph'

If format is 'dot2tex', then all the LaTeX generation will be delegated to dot2tex (which must be installed).

For tkz_graph, the possible option names, and associated values are given below. This first group allows you to set a style for a graph and specify some sizes related to the eventual image. (For more information consult the documentation for the tkz-graph package.)

- tkz_style – string (default: 'Custom'); the name of a pre-defined tkz-graph style such as 'Shade', 'Art', 'Normal', 'Dijkstra', 'Welsh', 'Classic', and 'Simple', or the string 'Custom'. Using one of these styles alone will often give a reasonably good drawing with minimal effort. For a custom appearance set this to 'Custom' and use the options described below to override the default values.
- units – string (default: 'cm'); a natural unit of measurement used for all dimensions. Possible values are: 'in', 'mm', 'cm', 'pt', 'em', 'ex'.
- scale – float (default: 1.0); a dimensionless number that multiplies every linear dimension. So you can design at sizes you are accustomed to, then shrink or expand to meet other needs. Though fonts do not scale.
- graphic_size – tuple (default: (5, 5)); overall dimensions (width, length) of the bounding box around the entire graphic image
- margins – 4-tuple (default: (0, 0, 0, 0)); portion of graphic given over to a plain border as a tuple of four numbers: (left, right, top, bottom). These are subtracted from the graphic_size to create the area left for the vertices of the graph itself. Note that the processing done by Sage will trim the graphic down to the minimum possible size, removing any border. So this is only useful if you use the latex string in a latex document.

If not using a pre-built style the following options are used, so the following defaults will apply. It is not possible to begin with a pre-built style and modify it (other than editing the latex string by hand after the fact).

- vertex_color – (default: 'black') a single color to use as the default for outline of vertices. For the sphere shape this color is used for the entire vertex, which is drawn with a 3D shading. Colors must be specified as a string recognized by the matplotlib library: a standard color name like 'red', or a hex string like '#2D87A7', or a single character from the choices 'rgbcmykw'. Additionally, a number between 0 and 1 will create a grayscale value. These color specifications are consistent throughout the options for a tikzpicture.
- vertex_colors – dictionary whose keys are vertices of the graph and whose values are colors. These will be used to color the outline of vertices. See the explanation above for the vertex_color option

to see possible values. These values need only be specified for a proper subset of the vertices. Specified values will supersede a default value.

- `vertex_fill_color` – (default: `'white'`) a single color to use as the default for the fill color of vertices. See the explanation above for the `vertex_color` option to see possible values. This color is ignored for the `sphere` vertex shape.
- `vertex_fill_colors` – dictionary whose keys are vertices of the graph and whose values are colors. These will be used to fill the interior of vertices. See the explanation above for the `vertex_color` option to see possible values. These values need only be specified for a proper subset of the vertices. Specified values will supersede a default value.
- `vertex_shape` – string (default: `'circle'`); specifies the shape of the vertices. Allowable values are `'circle'`, `'sphere'`, `'rectangle'`, `'diamond'`. The sphere shape has a 3D look to its coloring and is uses only one color, that specified by `vertex_color` and `vertex_colors`, which are normally used for the outline of the vertex.
- `vertex_shapes` – dictionary whose keys are vertices of the graph and whose values are shapes. See `vertex_shape` for the allowable possibilities.
- `vertex_size` – float (default: 1.0); the minimum size of a vertex as a number. Vertices will expand to contain their labels if the labels are placed inside the vertices. If you set this value to zero the vertex will be as small as possible (up to tkz-graph's "inner sep" parameter), while still containing labels. However, if labels are not of a uniform size, then the vertices will not be either.
- `vertex_sizes` – dictionary of sizes for some of the vertices
- `vertex_labels` – boolean (default: `True`); determine whether or not to display the vertex labels. If `False` subsequent options about vertex labels are ignored.
- `vertex_labels_math` – boolean (default: `True`); when `True`, if a label is a string that begins and ends with dollar signs, then the string will be rendered as a latex string. Otherwise, the label will be automatically subjected to the `latex()` method and rendered accordingly. If `False` the label is rendered as its textual representation according to the `_repr` method. Support for arbitrarily-complicated mathematics is not especially robust.
- `vertex_label_color` – (default: `'black'`) a single color to use as the default for labels of vertices. See the explanation above for the `vertex_color` option to see possible values.
- `vertex_label_colors` – dictionary whose keys are vertices of the graph and whose values are colors. These will be used for the text of the labels of vertices. See the explanation above for the `vertex_color` option to see possible values. These values need only be specified for a proper subset of the vertices. Specified values will supersede a default value.
- `vertex_label_placement` – (default: `'center'`) if `'center'` the label is centered in the interior of the vertex and the vertex will expand to contain the label. Giving instead a pair of numbers will place the label exterior to the vertex at a certain distance from the edge, and at an angle to the positive x-axis, similar in spirit to polar coordinates.
- `vertex_label_placements` – dictionary of placements indexed by the vertices. See the explanation for `vertex_label_placement` for the possible values.
- `edge_color` – (default: `'black'`) a single color to use as the default for an edge. See the explanation above for the `vertex_color` option to see possible values.
- `edge_colors` – dictionary whose keys are edges of the graph and whose values are colors. These will be used to color the edges. See the explanation above for the `vertex_color` option to see possible values. These values need only be specified for a proper subset of the vertices. Specified values will supersede a default value.

- edge_fills – boolean (default: False); whether an edge has a second color running down the middle. This can be a useful effect for highlighting edge crossings.
- edge_fill_color – (default: 'black') a single color to use as the default for the fill color of an edge. The boolean switch edge_fills must be set to True for this to have an effect. See the explanation above for the vertex_color option to see possible values.
- edge_fill_colors – dictionary whose keys are edges of the graph and whose values are colors. See the explanation above for the vertex_color option to see possible values. These values need only be specified for a proper subset of the vertices. Specified values will supersede a default value.
- edge_thickness – float (default: 0.1); specifies the width of the edges. Note that tkz-graph does not interpret this number for loops.
- edge_thicknesses – dictionary of thicknesses for some of the edges of a graph. These values need only be specified for a proper subset of the vertices. Specified values will supersede a default value.
- edge_labels – boolean (default: False); determine if edge labels are shown. If False subsequent options about edge labels are ignored.
- edge_labels_math – boolean (default: True); control how edge labels are rendered. Read the explanation for the vertex_labels_math option, which behaves identically. Support for arbitrarily-complicated mathematics is not especially robust.
- edge_label_color – (default: 'black') a single color to use as the default for labels of edges. See the explanation above for the vertex_color option to see possible values.
- edge_label_colors – dictionary whose keys are edges of the graph and whose values are colors. These will be used for the text of the labels of edges. See the explanation above for the vertex_color option to see possible values. These values need only be specified for a proper subset of the vertices. Specified values will supersede a default value. Note that labels must be used for this to have any effect, and no care is taken to ensure that label and fill colors work well together.
- edge_label_sloped – boolean (default: True); specifies how edge labels are place. False results in a horizontal label, while True means the label is rotated to follow the direction of the edge it labels.
- edge_label_slopes – dictionary of booleans, indexed by some subset of the edges. See the edge_label_sloped option for a description of sloped edge labels.
- edge_label_placement – (default: 0.50) either a number between 0.0 and 1.0, or one of: 'above', 'below', 'left', 'right'. These adjust the location of an edge label along an edge. A number specifies how far along the edge the label is located. 'left' and 'right' are conveniences. 'above' and 'below' move the label off the edge itself while leaving it near the midpoint of the edge. The default value of 0.50 places the label on the midpoint of the edge.
- edge_label_placements – dictionary of edge placements, indexed by the edges. See the edge_label_placement option for a description of the allowable values.
- loop_placement – (default: (3.0, 'NO')); determine how loops are rendered. the first element of the pair is a distance, which determines how big the loop is and the second element is a string specifying a compass point (North, South, East, West) as one of 'NO', 'SO', 'EA', 'WE'.
- loop_placements – dictionary of loop placements. See the loop_placements option for the allowable values. While loops are technically edges, this dictionary is indexed by vertices.

For the 'dot2tex' format, the possible option names and associated values are given below:

- prog – string; the program used for the layout. It must be a string corresponding to one of the software of the graphviz suite: 'dot', 'neato', 'twopi', 'circo' or 'fdp'.
- edge_labels – boolean (default: False); whether to display the labels on edges
- edge_colors – a color; can be used to set a global color to the edge of the graph

- `color_by_label` – boolean (default: `False`); colors the edges according to their labels
- `subgraph_clusters` – (default: `[]`) a list of lists of vertices, if supported by the layout engine, nodes belonging to the same cluster subgraph are drawn together, with the entire drawing of the cluster contained within a bounding rectangle.

OUTPUT: none; success happens silently

EXAMPLES:

Set, then modify, then clear the `tkz_style` option, and finally show an error for an unrecognized option name:

```
sage: g = graphs.PetersenGraph()
sage: opts = g.latex_options()
sage: opts
LaTeX options for Petersen graph: {}
sage: opts.set_option('tkz_style', 'Art')
sage: opts
LaTeX options for Petersen graph: {'tkz_style': 'Art'}
sage: opts.set_option('tkz_style', 'Simple')
sage: opts
LaTeX options for Petersen graph: {'tkz_style': 'Simple'}
sage: opts.set_option('tkz_style')
sage: opts
LaTeX options for Petersen graph: {}
sage: opts.set_option('bad_name', 'nonsense')
Traceback (most recent call last):
...
ValueError: bad_name is not a LaTeX option for a graph.
```

```
>>> from sage.all import *
>>> g = graphs.PetersenGraph()
>>> opts = g.latex_options()
>>> opts
LaTeX options for Petersen graph: {}
>>> opts.set_option('tkz_style', 'Art')
>>> opts
LaTeX options for Petersen graph: {'tkz_style': 'Art'}
>>> opts.set_option('tkz_style', 'Simple')
>>> opts
LaTeX options for Petersen graph: {'tkz_style': 'Simple'}
>>> opts.set_option('tkz_style')
>>> opts
LaTeX options for Petersen graph: {}
>>> opts.set_option('bad_name', 'nonsense')
Traceback (most recent call last):
...
ValueError: bad_name is not a LaTeX option for a graph.
```

See *sage.graphs.generic_graph.GenericGraph.layout_graphviz()* for installation instructions for `graphviz` and `dot2tex`. Furthermore, pgf >= 2.00 should be available inside LaTeX's tree for LaTeX compilation (e.g. when using `view`). In case your LaTeX distribution does not provide it, here are short instructions:

- download pgf from http://sourceforge.net/projects/pgf/

- unpack it in `/usr/share/texmf/tex/generic` (depends on your system)
- clean out remaining pgf files from older version
- run texhash

set_options(***kwds*)

Set several LaTeX options for a graph all at once.

INPUT:

- `kwds` – any number of option/value pairs to set many graph latex options at once (a variable number, in any order). Existing values are overwritten, new values are added. Existing values can be cleared by setting the value to `None`. Errors are raised in the *set_option()* method.

EXAMPLES:

```
sage: g = graphs.PetersenGraph()
sage: opts = g.latex_options()
sage: opts.set_options(tkz_style='Welsh')
sage: opts.get_option('tkz_style')
'Welsh'
```

```
>>> from sage.all import *
>>> g = graphs.PetersenGraph()
>>> opts = g.latex_options()
>>> opts.set_options(tkz_style='Welsh')
>>> opts.get_option('tkz_style')
'Welsh'
```

tkz_picture()

Return a string of LaTeX commands representing a graph as a `tikzpicture`.

This routine interprets the graph's properties and the options in `_options` to render the graph with commands from the `tkz-graph` LaTeX package.

This requires that the LaTeX optional packages `tkz-graph` and `tkz-berge` be installed. You may also need a current version of the pgf package. If the `tkz-graph` and `tkz-berge` packages are present in the system's TeX installation, the appropriate `\\usepackage{}` commands will be added to the LaTeX preamble as part of the initialization of the graph. If these two packages are not present, then this command will return a warning on its first use, but will return a string that could be used elsewhere, such as a LaTeX document.

For more information about tkz-graph you can visit https://www.ctan.org/pkg/tkz-graph.

EXAMPLES:

With a pre-built `tkz-graph` style specified, the latex representation will be relatively simple.

```
sage: from sage.graphs.graph_latex import check_tkz_graph
sage: check_tkz_graph()  # random - depends on TeX installation
sage: g = graphs.CompleteGraph(3)
sage: opts = g.latex_options()
sage: g.set_latex_options(tkz_style='Art')
sage: print(opts.tkz_picture())
\begin{tikzpicture}
\GraphInit[vstyle=Art]
%
\Vertex[L=\hbox{$0$},x=2.5cm,y=5.0cm]{v0}
```

(continues on next page)

(continued from previous page)

```
\Vertex[L=\hbox{$1$},x=0.0cm,y=0.0cm]{v1}
\Vertex[L=\hbox{$2$},x=5.0cm,y=0.0cm]{v2}
%
\Edge[](v0)(v1)
\Edge[](v0)(v2)
\Edge[](v1)(v2)
%
\end{tikzpicture}
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_latex import check_tkz_graph
>>> check_tkz_graph()  # random - depends on TeX installation
>>> g = graphs.CompleteGraph(Integer(3))
>>> opts = g.latex_options()
>>> g.set_latex_options(tkz_style='Art')
>>> print(opts.tkz_picture())
\begin{tikzpicture}
\GraphInit[vstyle=Art]
%
\Vertex[L=\hbox{$0$},x=2.5cm,y=5.0cm]{v0}
\Vertex[L=\hbox{$1$},x=0.0cm,y=0.0cm]{v1}
\Vertex[L=\hbox{$2$},x=5.0cm,y=0.0cm]{v2}
%
\Edge[](v0)(v1)
\Edge[](v0)(v2)
\Edge[](v1)(v2)
%
\end{tikzpicture}
```

Setting the style to "Custom" results in various configurable aspects set to the defaults, so the string is more involved.

```
sage: from sage.graphs.graph_latex import check_tkz_graph
sage: check_tkz_graph()  # random - depends on TeX installation
sage: g = graphs.CompleteGraph(3)
sage: opts = g.latex_options()
sage: g.set_latex_options(tkz_style='Custom')
sage: print(opts.tkz_picture())
\begin{tikzpicture}
\definecolor{cv0}{rgb}{0.0,0.0,0.0}
\definecolor{cfv0}{rgb}{1.0,1.0,1.0}
\definecolor{clv0}{rgb}{0.0,0.0,0.0}
\definecolor{cv1}{rgb}{0.0,0.0,0.0}
\definecolor{cfv1}{rgb}{1.0,1.0,1.0}
\definecolor{clv1}{rgb}{0.0,0.0,0.0}
\definecolor{cv2}{rgb}{0.0,0.0,0.0}
\definecolor{cfv2}{rgb}{1.0,1.0,1.0}
\definecolor{clv2}{rgb}{0.0,0.0,0.0}
\definecolor{cv0v1}{rgb}{0.0,0.0,0.0}
\definecolor{cv0v2}{rgb}{0.0,0.0,0.0}
\definecolor{cv1v2}{rgb}{0.0,0.0,0.0}
%
```

(continues on next page)

(continued from previous page)

```
\begin{scope}[VertexStyle/.append style={minimum size=1.0cm,draw=cv0,
↪fill=cfv0,text=clv0,shape=circle}]
\Vertex[LabelOut=false,L=\hbox{$0$},x=2.5cm,y=5.0cm]{v0}
\end{scope}
\begin{scope}[VertexStyle/.append style={minimum size=1.0cm,draw=cv1,
↪fill=cfv1,text=clv1,shape=circle}]
\Vertex[LabelOut=false,L=\hbox{$1$},x=0.0cm,y=0.0cm]{v1}
\end{scope}
\begin{scope}[VertexStyle/.append style={minimum size=1.0cm,draw=cv2,
↪fill=cfv2,text=clv2,shape=circle}]
\Vertex[LabelOut=false,L=\hbox{$2$},x=5.0cm,y=0.0cm]{v2}
\end{scope}
%
\Edge[lw=0.1cm,style={color=cv0v1,},](v0)(v1)
\Edge[lw=0.1cm,style={color=cv0v2,},](v0)(v2)
\Edge[lw=0.1cm,style={color=cv1v2,},](v1)(v2)
%
\end{tikzpicture}
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_latex import check_tkz_graph
>>> check_tkz_graph()  # random - depends on TeX installation
>>> g = graphs.CompleteGraph(Integer(3))
>>> opts = g.latex_options()
>>> g.set_latex_options(tkz_style='Custom')
>>> print(opts.tkz_picture())
\begin{tikzpicture}
\definecolor{cv0}{rgb}{0.0,0.0,0.0}
\definecolor{cfv0}{rgb}{1.0,1.0,1.0}
\definecolor{clv0}{rgb}{0.0,0.0,0.0}
\definecolor{cv1}{rgb}{0.0,0.0,0.0}
\definecolor{cfv1}{rgb}{1.0,1.0,1.0}
\definecolor{clv1}{rgb}{0.0,0.0,0.0}
\definecolor{cv2}{rgb}{0.0,0.0,0.0}
\definecolor{cfv2}{rgb}{1.0,1.0,1.0}
\definecolor{clv2}{rgb}{0.0,0.0,0.0}
\definecolor{cv0v1}{rgb}{0.0,0.0,0.0}
\definecolor{cv0v2}{rgb}{0.0,0.0,0.0}
\definecolor{cv1v2}{rgb}{0.0,0.0,0.0}
%
\begin{scope}[VertexStyle/.append style={minimum size=1.0cm,draw=cv0,
↪fill=cfv0,text=clv0,shape=circle}]
\Vertex[LabelOut=false,L=\hbox{$0$},x=2.5cm,y=5.0cm]{v0}
\end{scope}
\begin{scope}[VertexStyle/.append style={minimum size=1.0cm,draw=cv1,
↪fill=cfv1,text=clv1,shape=circle}]
\Vertex[LabelOut=false,L=\hbox{$1$},x=0.0cm,y=0.0cm]{v1}
\end{scope}
\begin{scope}[VertexStyle/.append style={minimum size=1.0cm,draw=cv2,
↪fill=cfv2,text=clv2,shape=circle}]
\Vertex[LabelOut=false,L=\hbox{$2$},x=5.0cm,y=0.0cm]{v2}
\end{scope}
```

(continues on next page)

(continued from previous page)

```
%
\Edge[lw=0.1cm,style={color=cv0v1,},](v0)(v1)
\Edge[lw=0.1cm,style={color=cv0v2,},](v0)(v2)
\Edge[lw=0.1cm,style={color=cv1v2,},](v1)(v2)
%
\end{tikzpicture}
```

Non-default vertex appearance is encoded through `VertexStyle` so it is not overwritten by tkz-graph defaults (Issue #38282):

```
sage: from sage.graphs.graph_latex import check_tkz_graph
sage: check_tkz_graph()  # random - depends on TeX installation
sage: g = graphs.PathGraph(2)
sage: g.set_latex_options(vertex_shape='rectangle', vertex_color='red')
sage: print(g.latex_options().tkz_picture())
\begin{tikzpicture}
\definecolor{cv0}{rgb}{1.0,0.0,0.0}
...
\begin{scope}[VertexStyle/.append style={minimum size=1.0cm,draw=cv0,
↪fill=cfv0,text=clv0,shape=rectangle}]
\Vertex[LabelOut=false,L=\hbox{$0$},x=...,y=...]{v0}
\end{scope}
...
\end{tikzpicture}
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_latex import check_tkz_graph
>>> check_tkz_graph()  # random - depends on TeX installation
>>> g = graphs.PathGraph(Integer(2))
>>> g.set_latex_options(vertex_shape='rectangle', vertex_color='red')
>>> print(g.latex_options().tkz_picture())
\begin{tikzpicture}
\definecolor{cv0}{rgb}{1.0,0.0,0.0}
...
\begin{scope}[VertexStyle/.append style={minimum size=1.0cm,draw=cv0,
↪fill=cfv0,text=clv0,shape=rectangle}]
\Vertex[LabelOut=false,L=\hbox{$0$},x=...,y=...]{v0}
\end{scope}
...
\end{tikzpicture}
```

See the introduction to the *graph_latex* module for more information on the use of this routine.

`sage.graphs.graph_latex.`**`check_tkz_graph`**`()`

Check if the proper LaTeX packages for the `tikzpicture` environment are installed in the user's environment, and issue a warning otherwise.

The warning is only issued on the first call to this function. So any doctest that illustrates the use of the tkz-graph packages should call this once as having random output to exhaust the warnings before testing output.

See also `sage.misc.latex.Latex.check_file()`

`sage.graphs.graph_latex.`**`have_tkz_graph`**`()`

Return `True` if the proper LaTeX packages for the `tikzpicture` environment are installed in the user's environ-

ment, namely `tikz`, `tkz-graph` and `tkz-berge`.

The result is cached.

See also `sage.misc.latex.Latex.has_file()`

`sage.graphs.graph_latex.`**`setup_latex_preamble`**`()`

Add appropriate `\usepackage{...}`, and other instructions to the latex preamble for the packages that are needed for processing graphs(`tikz`, `tkz-graph`, `tkz-berge`), if available in the `LaTeX` installation.

See also `sage.misc.latex.Latex.add_package_to_preamble_if_available()`.

EXAMPLES:

```
sage: sage.graphs.graph_latex.setup_latex_preamble()
```

```
>>> from sage.all import *
>>> sage.graphs.graph_latex.setup_latex_preamble()
```

5.37 Graph editor widget

This module adds an interface to `phitigra`, a graph editor widget for Jupyter and JupyterLab. The `phitigra` optional package should be installed on your Sage installation.

AUTHORS:

- Radoslav Kirov (2009): initial editor for use with the old sage notebook
- Jean-Florent Raymond (2022-04-12): replacement with the `phitigra` package

`sage.graphs.graph_editor.`**`graph_editor`**`(`*`graph=None`*`, `*`**display_options`*`)`

Return a graph editor widget.

The graph editor widget can be displayed with Jupyter or JupyterLab. It is provided by the `phitigra` optional package, see https://github.com/jfraymond/phitigra for details about the possible options (changing the width/height of the canvas, the default size and color of vertices, etc.).

INPUT:

- `graph` – a graph to edit (default: `None`)
- `display_options` – options for the widget

EXAMPLES:

```
sage: e = graph_editor()              # optional - phitigra
sage: e.show()                        # not tested
```

```
>>> from sage.all import *
>>> e = graph_editor()              # optional - phitigra
>>> e.show()                        # not tested
```

Opening an existing graph:

```
sage: G = graphs.RandomGNP(10, 0.5)
sage: e = graph_editor(G)             # optional - phitigra
sage: e.show()                        # not tested
```

```
>>> from sage.all import *
>>> G = graphs.RandomGNP(Integer(10), RealNumber('0.5'))
>>> e = graph_editor(G)            # optional - phitigra
>>> e.show()                       # not tested
```

Retrieving a copy of the drawn graph:

```
sage: G = graphs.RandomGNP(10, 0.5)
sage: e = graph_editor(G)             # optional - phitigra
sage: H = e.get_graph()               # optional - phitigra
sage: H == G and not H is G           # optional - phitigra
True
```

```
>>> from sage.all import *
>>> G = graphs.RandomGNP(Integer(10), RealNumber('0.5'))
>>> e = graph_editor(G)             # optional - phitigra
>>> H = e.get_graph()               # optional - phitigra
>>> H == G and not H is G           # optional - phitigra
True
```

Using different display options:

```
sage: e = graph_editor(graphs.PetersenGraph(), width=300, height=300,       #␣
↪optional - phitigra
....:                  default_radius=12, default_vertex_color='orange',
....:                  default_edge_color='#666', show_vertex_labels=False)
sage: e.show()                                                              # not␣
↪tested
```

```
>>> from sage.all import *
>>> e = graph_editor(graphs.PetersenGraph(), width=Integer(300),␣
↪height=Integer(300),       # optional - phitigra
...                  default_radius=Integer(12), default_vertex_color='orange',
...                  default_edge_color='#666', show_vertex_labels=False)
>>> e.show()                                                                # not␣
↪tested
```

Note

The editor does not support multigraphs.

5.38 Lists of graphs

AUTHORS:

- Robert L. Miller (2007-02-10): initial version
- Emily A. Kirkman (2007-02-13): added show functions (to_graphics_array and show_graphs)

`sage.graphs.graph_list.`**`from_graph6`**(*data*, *immutable=False*)

Return a list of Sage Graphs, given a list of graph6 data.

INPUT:

- `data` – can be a string, a list of strings, or a file stream
- `immutable` – boolean (default: `False`); whether to return immutable or mutable graphs

EXAMPLES:

```
sage: l = ['N@@?N@UGAGG?gGlKCMO', 'XsGGWOW?CC?C@HQKHqOjYKC_uHWGX?P?~
↪TqIKA`OA@SAOEcEA??']
sage: graphs_list.from_graph6(l)
[Graph on 15 vertices, Graph on 25 vertices]
```

```
>>> from sage.all import *
>>> l = ['N@@?N@UGAGG?gGlKCMO', 'XsGGWOW?CC?C@HQKHqOjYKC_uHWGX?P?~
↪TqIKA`OA@SAOEcEA??']
>>> graphs_list.from_graph6(l)
[Graph on 15 vertices, Graph on 25 vertices]
```

Check the behaviour of parameter `immutable`:

```
sage: any(g.is_immutable() for g in graphs_list.from_graph6(l))
False
sage: all(g.is_immutable() for g in graphs_list.from_graph6(l, immutable=True))
True
```

```
>>> from sage.all import *
>>> any(g.is_immutable() for g in graphs_list.from_graph6(l))
False
>>> all(g.is_immutable() for g in graphs_list.from_graph6(l, immutable=True))
True
```

`sage.graphs.graph_list.`**`from_sparse6`**(*data*, *immutable=False*)

Return a list of Sage Graphs, given a list of sparse6 data.

INPUT:

- `data` – can be a string, a list of strings, or a file stream
- `immutable` – boolean (default: `False`); whether to return immutable or mutable graphs

EXAMPLES:

```
sage: g1 = ':P_`cBaC_ACd`C_@BC`ABDHaEH_@BF_@CHIK_@BCEHKL_BIKM_BFGHI'
sage: g2 = ':f`??KO?B_OOSCGE_?OWONDBO?GOJBDO?_SSJdApcOIG`?og_UKEbg?_SKF'
sage: g2 += 'q@[CCBA`p?oYMFp@gw]Qaa@xEMHDb@hMCBCbQ@ECHEcAKKQKFPOwo[PIDQ'
sage: g2 += '{KIHEcQPOkVKEW_WMNKqPWwcRKOOWSKIGCqhWt??___WMJFCahWzEBa`xO'
sage: g2 += 'u[MpPPKqYNoOOOKHHDBPs|??__gWMKEcAHKgTLErqA?A@a@G{kVLErs?GD'
sage: g2 += 'BA@XCs\\NggWSOJIDbHh@?A@aF'
sage: graphs_list.from_sparse6([g1, g2])
[Looped multi-graph on 17 vertices, Looped multi-graph on 39 vertices]
```

```
>>> from sage.all import *
>>> g1 = ':P_`cBaC_ACd`C_@BC`ABDHaEH_@BF_@CHIK_@BCEHKL_BIKM_BFGHI'
>>> g2 = ':f`??KO?B_OOSCGE_?OWONDBO?GOJBDO?_SSJdApcOIG`?og_UKEbg?_SKF'
>>> g2 += 'q@[CCBA`p?oYMFp@gw]Qaa@xEMHDb@hMCBCbQ@ECHEcAKKQKFPOwo[PIDQ'
>>> g2 += '{KIHEcQPOkVKEW_WMNKqPWwcRKOOWSKIGCqhWt??___WMJFCahWzEBa`xO'
>>> g2 += 'u[MpPPKqYNoOOOKHHDBPs|??__gWMKEcAHKgTLErqA?A@a@G{kVLErs?GD'
```

(continues on next page)

(continued from previous page)

```
>>> g2 += 'BA@XCs\\NggWSOJIDbHh@?A@aF'
>>> graphs_list.from_sparse6([g1, g2])
[Looped multi-graph on 17 vertices, Looped multi-graph on 39 vertices]
```

Check the behaviour of parameter `immutable`:

```
sage: any(g.is_immutable() for g in graphs_list.from_sparse6([g1, g2]))
False
sage: all(g.is_immutable() for g in graphs_list.from_sparse6([g1, g2],␣
↪immutable=True))
True
```

```
>>> from sage.all import *
>>> any(g.is_immutable() for g in graphs_list.from_sparse6([g1, g2]))
False
>>> all(g.is_immutable() for g in graphs_list.from_sparse6([g1, g2],␣
↪immutable=True))
True
```

sage.graphs.graph_list.**from_whatever**(*data*, *immutable=False*)

Return a list of Sage Graphs, given a list of whatever kind of data.

INPUT:

- `data` – can be a string, a list/iterable of strings, or a readable file-like object
- `immutable` – boolean (default: `False`); whether to return immutable or mutable graphs

EXAMPLES:

```
sage: l = ['N@@?N@UGAGG?gGlKCMO', ':P_`cBaC_ACd`C_@BC`ABDHaEH_@BF_@CHIK_@BCEHKL_
↪BIKM_BFGHI']
sage: graphs_list.from_whatever(l)
[Graph on 15 vertices, Looped multi-graph on 17 vertices]
sage: graphs_list.from_whatever('\n'.join(l))
[Graph on 15 vertices, Looped multi-graph on 17 vertices]
```

```
>>> from sage.all import *
>>> l = ['N@@?N@UGAGG?gGlKCMO', ':P_`cBaC_ACd`C_@BC`ABDHaEH_@BF_@CHIK_@BCEHKL_
↪BIKM_BFGHI']
>>> graphs_list.from_whatever(l)
[Graph on 15 vertices, Looped multi-graph on 17 vertices]
>>> graphs_list.from_whatever('\n'.join(l))
[Graph on 15 vertices, Looped multi-graph on 17 vertices]
```

This example happens to be a mix a sparse and non-sparse graphs, so we don't explicitly put a `.g6` or `.s6` extension, which implies just one or the other:

```
sage: filename = tmp_filename()
sage: with open(filename, 'w') as fobj:
....:     _ = fobj.write('\n'.join(l))
sage: with open(filename) as fobj:
....:     graphs_list.from_whatever(fobj)
[Graph on 15 vertices, Looped multi-graph on 17 vertices]
```

```
>>> from sage.all import *
>>> filename = tmp_filename()
>>> with open(filename, 'w') as fobj:
...     _ = fobj.write('\n'.join(l))
>>> with open(filename) as fobj:
...     graphs_list.from_whatever(fobj)
[Graph on 15 vertices, Looped multi-graph on 17 vertices]
```

Check the behaviour of parameter `immutable`:

```
sage: with open(filename) as fobj:
....:     any(g.is_immutable() for g in graphs_list.from_whatever(fobj))
False
sage: with open(filename) as fobj:
....:     all(g.is_immutable() for g in graphs_list.from_whatever(fobj,␣
↪immutable=True))
True
```

```
>>> from sage.all import *
>>> with open(filename) as fobj:
...     any(g.is_immutable() for g in graphs_list.from_whatever(fobj))
False
>>> with open(filename) as fobj:
...     all(g.is_immutable() for g in graphs_list.from_whatever(fobj,␣
↪immutable=True))
True
```

`sage.graphs.graph_list.`**`show_graphs`**(*graph_list*, ***kwds*)

Show a maximum of 20 graphs from `graph_list` in a sage graphics array.

If more than 20 graphs are given in the list argument, then it will display one graphics array after another with each containing at most 20 graphs.

Note that to save the image output from the notebook, you must save each graphics array individually. (There will be a small space between graphics arrays).

INPUT:

- `graph_list` – a Python list of Sage Graphs

GRAPH PLOTTING: Defaults to circular layout for graphs. This allows for a nicer display in a small area and takes much less time to compute than the spring-layout algorithm for many graphs.

EXAMPLES: Create a list of graphs:

```
sage: glist = []
sage: glist.append(graphs.CompleteGraph(6))
sage: glist.append(graphs.CompleteBipartiteGraph(4, 5))
sage: glist.append(graphs.BarbellGraph(7, 4))
sage: glist.append(graphs.CycleGraph(15))
sage: glist.append(graphs.DiamondGraph())
sage: glist.append(graphs.GemGraph())
sage: glist.append(graphs.DartGraph())
sage: glist.append(graphs.ForkGraph())
sage: glist.append(graphs.HouseGraph())
```

(continues on next page)

(continued from previous page)

```
sage: glist.append(graphs.HouseXGraph())
sage: glist.append(graphs.KrackhardtKiteGraph())
sage: glist.append(graphs.LadderGraph(5))
sage: glist.append(graphs.LollipopGraph(5, 6))
sage: glist.append(graphs.PathGraph(15))
sage: glist.append(graphs.PetersenGraph())
sage: glist.append(graphs.StarGraph(17))
sage: glist.append(graphs.WheelGraph(9))
```

```
>>> from sage.all import *
>>> glist = []
>>> glist.append(graphs.CompleteGraph(Integer(6)))
>>> glist.append(graphs.CompleteBipartiteGraph(Integer(4), Integer(5)))
>>> glist.append(graphs.BarbellGraph(Integer(7), Integer(4)))
>>> glist.append(graphs.CycleGraph(Integer(15)))
>>> glist.append(graphs.DiamondGraph())
>>> glist.append(graphs.GemGraph())
>>> glist.append(graphs.DartGraph())
>>> glist.append(graphs.ForkGraph())
>>> glist.append(graphs.HouseGraph())
>>> glist.append(graphs.HouseXGraph())
>>> glist.append(graphs.KrackhardtKiteGraph())
>>> glist.append(graphs.LadderGraph(Integer(5)))
>>> glist.append(graphs.LollipopGraph(Integer(5), Integer(6)))
>>> glist.append(graphs.PathGraph(Integer(15)))
>>> glist.append(graphs.PetersenGraph())
>>> glist.append(graphs.StarGraph(Integer(17)))
>>> glist.append(graphs.WheelGraph(Integer(9)))
```

Check that length is <= 20:

```
sage: len(glist)
17
```

```
>>> from sage.all import *
>>> len(glist)
17
```

Show the graphs in a graphics array:

```
sage: graphs_list.show_graphs(glist)                                    #␣
↪needs sage.plot
```

```
>>> from sage.all import *
>>> graphs_list.show_graphs(glist)                                      #␣
↪needs sage.plot
```

Example where more than one graphics array is used:

```
sage: gq = GraphQuery(display_cols=['graph6'], num_vertices=5)
sage: g = gq.get_graphs_list()
sage: len(g)
```

(continues on next page)

(continued from previous page)

```
34
sage: graphs_list.show_graphs(g)                                                #␣
↪needs sage.plot
```

```
>>> from sage.all import *
>>> gq = GraphQuery(display_cols=['graph6'], num_vertices=Integer(5))
>>> g = gq.get_graphs_list()
>>> len(g)
34
>>> graphs_list.show_graphs(g)                                                  #␣
↪needs sage.plot
```

See the .plot() or .show() documentation for an individual graph for options, all of which are available from *to_graphics_array()*:

```
sage: glist = []
sage: for _ in range(10):                                                       #␣
↪needs networkx
....:     glist.append(graphs.RandomLobster(41, .3, .4))
sage: graphs_list.show_graphs(glist, layout='spring', vertex_size=20)           #␣
↪needs sage.plot
```

```
>>> from sage.all import *
>>> glist = []
>>> for _ in range(Integer(10)):                                                ␣
↪     # needs networkx
...     glist.append(graphs.RandomLobster(Integer(41), RealNumber('.3'),␣
↪RealNumber('.4')))
>>> graphs_list.show_graphs(glist, layout='spring', vertex_size=Integer(20))    ␣
↪     # needs sage.plot
```

sage.graphs.graph_list.**to_graph6**(*graphs*, *file=None*, *output_list=False*)

Convert a list of Sage graphs to a single string of graph6 graphs.

If `file` is specified, then the string will be written quietly to the file. If `output_list` is `True`, then a list of strings will be returned, one string per graph.

INPUT:

- `graphs` – a Python list of Sage Graphs
- `file` – (optional) a file stream to write to (must be in 'w' mode)
- `output_list` – boolean (default: `False`); whether to return a string (when set to `True`) or a list of strings. This parameter is ignored if file gets specified

EXAMPLES:

```
sage: l = [graphs.DodecahedralGraph(), graphs.PetersenGraph()]
sage: graphs_list.to_graph6(l)
'ShCHGD@?K?_@?@?C_GGG@??cG?G?GK_?C\nIheA@GUAo\n'
```

```
>>> from sage.all import *
>>> l = [graphs.DodecahedralGraph(), graphs.PetersenGraph()]
```

(continues on next page)

(continued from previous page)

```
>>> graphs_list.to_graph6(l)
'ShCHGD@?K?_@?@?C_GGG@??cG?G?GK_?C\nIheA@GUAo\n'
```

sage.graphs.graph_list.**to_graphics_array**(*graph_list*, ***kwds*)

Draw all graphs in a graphics array.

INPUT:

- `graph_list` – a Python list of Sage Graphs

GRAPH PLOTTING:

Defaults to circular layout for graphs. This allows for a nicer display in a small area and takes much less time to compute than the spring- layout algorithm for many graphs.

EXAMPLES:

```
sage: glist = []
sage: for i in range(999):
....:     glist.append(graphs.RandomGNP(6, .45))
sage: garray = graphs_list.to_graphics_array(glist)                    #␣
↪needs sage.plot
sage: garray.nrows(), garray.ncols()                                   #␣
↪needs sage.plot
(250, 4)
```

```
>>> from sage.all import *
>>> glist = []
>>> for i in range(Integer(999)):
...     glist.append(graphs.RandomGNP(Integer(6), RealNumber('.45')))
>>> garray = graphs_list.to_graphics_array(glist)                      #␣
↪needs sage.plot
>>> garray.nrows(), garray.ncols()                                     #␣
↪needs sage.plot
(250, 4)
```

See the .plot() or .show() documentation for an individual graph for options, all of which are available from *to_graphics_array()*:

```
sage: glist = []
sage: for _ in range(10):                                              #␣
↪needs networkx
....:     glist.append(graphs.RandomLobster(41, .3, .4))
sage: graphs_list.to_graphics_array(glist, layout='spring', vertex_size=20)     #␣
↪needs networkx sage.plot
Graphics Array of size 3 x 4
```

```
>>> from sage.all import *
>>> glist = []
>>> for _ in range(Integer(10)):                                        ␣
↪     # needs networkx
...     glist.append(graphs.RandomLobster(Integer(41), RealNumber('.3'),␣
↪RealNumber('.4')))
>>> graphs_list.to_graphics_array(glist, layout='spring', vertex_
```

(continues on next page)

(continued from previous page)

```
↪size=Integer(20))      # needs networkx sage.plot
Graphics Array of size 3 x 4
```

sage.graphs.graph_list.**to_sparse6**(*graphs*, *file=None*, *output_list=False*)

Convert a list of Sage graphs to a single string of sparse6 graphs.

If `file` is specified, then the string will be written quietly to the file. If `output_list` is `True`, then a list of strings will be returned, one string per graph.

INPUT:

- `graphs` – a Python list of Sage Graphs
- `file` – (optional) a file stream to write to (must be in 'w' mode)
- `output_list` – boolean (default: `False`); whether to return a string (when set to `True`) or a list of strings. This parameter is ignored if file gets specified

EXAMPLES:

```
sage: l = [graphs.DodecahedralGraph(), graphs.PetersenGraph()]
sage: graphs_list.to_sparse6(l)
':S_`abcaDe`Fg_HijhKfLdMkNcOjP_BQ\n:I`ES@obGkqegW~\n'
```

```
>>> from sage.all import *
>>> l = [graphs.DodecahedralGraph(), graphs.PetersenGraph()]
>>> graphs_list.to_sparse6(l)
':S_`abcaDe`Fg_HijhKfLdMkNcOjP_BQ\n:I`ES@obGkqegW~\n'
```

5.39 Functions for reading/building graphs/digraphs

This module gathers functions needed to build a graph from any other data.

Note

This is an **internal** module of Sage. All features implemented here are made available to end-users through the constructors of *Graph* and *DiGraph*.

Note that because they are called by the constructors of *Graph* and *DiGraph*, most of these functions modify a graph inplace.

from_adjacency_matrix()	Fill `G` with the data of an adjacency matrix.
from_dict_of_dicts()	Fill `G` with the data of a dictionary of dictionaries.
from_dict_of_lists()	Fill `G` with the data of a dictionary of lists.
from_dig6()	Fill `G` with the data of a dig6 string.
from_graph6()	Fill `G` with the data of a graph6 string.
from_incidence_matrix()	Fill `G` with the data of an incidence matrix.
from_networkx_graph()	Fill *G* with the data of a NetworkX (di)graph.
from_oriented_incidence_matrix()	Fill `G` with the data of an *oriented* incidence matrix.
from_seidel_adjacency_matrix()	Fill `G` with the data of a Seidel adjacency matrix.
from_sparse6()	Fill `G` with the data of a sparse6 string.

5.39.1 Functions

sage.graphs.graph_input.**from_adjacency_matrix**(*G*, *M*, *loops=False*, *multiedges=False*, *weighted=False*)

Fill `G` with the data of an adjacency matrix.

INPUT:

- `G` – a *Graph* or *DiGraph*
- `M` – an adjacency matrix
- `loops`, `multiedges`, `weighted` – booleans (default: `False`); whether to consider the graph as having loops, multiple edges, or weights

EXAMPLES:

```
sage: from sage.graphs.graph_input import from_adjacency_matrix
sage: g = Graph()
sage: from_adjacency_matrix(g, graphs.PetersenGraph().adjacency_matrix())       #␣
↪needs sage.modules
sage: g.is_isomorphic(graphs.PetersenGraph())                                   #␣
↪needs sage.modules
True
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_input import from_adjacency_matrix
>>> g = Graph()
>>> from_adjacency_matrix(g, graphs.PetersenGraph().adjacency_matrix())       #␣
↪needs sage.modules
>>> g.is_isomorphic(graphs.PetersenGraph())                                   #␣
↪needs sage.modules
True
```

sage.graphs.graph_input.**from_dict_of_dicts**(*G*, *M*, *loops=False*, *multiedges=False*, *weighted=False*, *convert_empty_dict_labels_to_None=False*)

Fill `G` with the data of a dictionary of dictionaries.

INPUT:

- `G` – a graph
- `M` – dictionary of dictionaries
- `loops`, `multiedges`, `weighted` – booleans (default: `False`); whether to consider the graph as having loops, multiple edges, or weights
- `convert_empty_dict_labels_to_None` – booleans (default: `False`); whether to adjust for empty dicts instead of `None` in NetworkX default edge labels

EXAMPLES:

```
sage: from sage.graphs.graph_input import from_dict_of_dicts
sage: g = Graph()
sage: from_dict_of_dicts(g, graphs.PetersenGraph().to_dictionary(edge_
↪labels=True))
sage: g.is_isomorphic(graphs.PetersenGraph())
True
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_input import from_dict_of_dicts
>>> g = Graph()
>>> from_dict_of_dicts(g, graphs.PetersenGraph().to_dictionary(edge_labels=True))
>>> g.is_isomorphic(graphs.PetersenGraph())
True
```

The resulting order of vertices is unspecified but deterministic:

```
sage: from sage.graphs.graph_input import from_dict_of_dicts
sage: g = Graph()
sage: from_dict_of_dicts(g, {i: {} for i in range(99, 90, -1)})
sage: g.vertices(sort=False)
[99, 98, 97, 96, 95, 94, 93, 92, 91]
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_input import from_dict_of_dicts
>>> g = Graph()
>>> from_dict_of_dicts(g, {i: {} for i in range(Integer(99), Integer(90), -
↪Integer(1))})
>>> g.vertices(sort=False)
[99, 98, 97, 96, 95, 94, 93, 92, 91]
```

sage.graphs.graph_input.**from_dict_of_lists**(*G*, *D*, *loops=False*, *multiedges=False*, *weighted=False*)

Fill `G` with the data of a dictionary of lists.

INPUT:

- `G` – a *Graph* or *DiGraph*
- `D` – dictionary of lists
- `loops`, `multiedges`, `weighted` – booleans (default: `False`); whether to consider the graph as having loops, multiple edges, or weights

EXAMPLES:

```
sage: from sage.graphs.graph_input import from_dict_of_lists
sage: g = Graph()
sage: from_dict_of_lists(g, graphs.PetersenGraph().to_dictionary())
sage: g.is_isomorphic(graphs.PetersenGraph())
True
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_input import from_dict_of_lists
>>> g = Graph()
>>> from_dict_of_lists(g, graphs.PetersenGraph().to_dictionary())
>>> g.is_isomorphic(graphs.PetersenGraph())
True
```

The resulting order of vertices is unspecified but deterministic:

```
sage: from sage.graphs.graph_input import from_dict_of_lists
sage: g = Graph()
sage: from_dict_of_lists(g, {i: [] for i in range(99, 90, -1)})
```

(continues on next page)

(continued from previous page)

```
sage: g.vertices(sort=False)
[99, 98, 97, 96, 95, 94, 93, 92, 91]
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_input import from_dict_of_lists
>>> g = Graph()
>>> from_dict_of_lists(g, {i: [] for i in range(Integer(99), Integer(90), -
↪Integer(1))})
>>> g.vertices(sort=False)
[99, 98, 97, 96, 95, 94, 93, 92, 91]
```

sage.graphs.graph_input.**from_dig6**(*G*, *dig6_string*)

Fill G with the data of a dig6 string.

INPUT:

- G – a graph
- dig6_string – a dig6 string

EXAMPLES:

```
sage: from sage.graphs.graph_input import from_dig6
sage: g = DiGraph()
sage: from_dig6(g, digraphs.Circuit(10).dig6_string())
sage: g.is_isomorphic(digraphs.Circuit(10))
True
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_input import from_dig6
>>> g = DiGraph()
>>> from_dig6(g, digraphs.Circuit(Integer(10)).dig6_string())
>>> g.is_isomorphic(digraphs.Circuit(Integer(10)))
True
```

The string may represent a directed graph with loops:

```
sage: L = DiGraph(loops=True)
sage: from_dig6(L, 'CW`C')
sage: L.edges(labels=False, sort=True)
[(0, 1), (0, 2), (1, 2), (2, 3), (3, 3)]
```

```
>>> from sage.all import *
>>> L = DiGraph(loops=True)
>>> from_dig6(L, 'CW`C')
>>> L.edges(labels=False, sort=True)
[(0, 1), (0, 2), (1, 2), (2, 3), (3, 3)]
```

sage.graphs.graph_input.**from_graph6**(*G*, *g6_string*)

Fill G with the data of a graph6 string.

INPUT:

- G – a graph

- `g6_string` – a graph6 string

EXAMPLES:

```
sage: from sage.graphs.graph_input import from_graph6
sage: g = Graph()
sage: from_graph6(g, 'IheA@GUAo')
sage: g.is_isomorphic(graphs.PetersenGraph())
True
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_input import from_graph6
>>> g = Graph()
>>> from_graph6(g, 'IheA@GUAo')
>>> g.is_isomorphic(graphs.PetersenGraph())
True
```

sage.graphs.graph_input.**from_incidence_matrix**(*G*, *M*, *loops=False*, *multiedges=False*, *weighted=False*)

Fill `G` with the data of an incidence matrix.

INPUT:

- `G` – a graph
- `M` – an incidence matrix
- `loops`, `multiedges`, `weighted` – booleans (default: `False`); whether to consider the graph as having loops, multiple edges, or weights

EXAMPLES:

```
sage: from sage.graphs.graph_input import from_incidence_matrix
sage: g = Graph()
sage: from_incidence_matrix(g, graphs.PetersenGraph().incidence_matrix())       #␣
↪needs sage.modules
sage: g.is_isomorphic(graphs.PetersenGraph())                                   #␣
↪needs sage.modules
True
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_input import from_incidence_matrix
>>> g = Graph()
>>> from_incidence_matrix(g, graphs.PetersenGraph().incidence_matrix())       #␣
↪needs sage.modules
>>> g.is_isomorphic(graphs.PetersenGraph())                                   #␣
↪needs sage.modules
True
```

sage.graphs.graph_input.**from_networkx_graph**(*G*, *gnx*, *weighted=None*, *loops=None*, *multiedges=None*, *convert_empty_dict_labels_to_None=None*)

Fill *G* with the data of a NetworkX (di)graph.

INPUT:

- `G` – a *Graph* or *DiGraph*
- `gnx` – a NetworkX `Graph`, `MultiGraph`, `DiGraph` or `MultiDiGraph`

- `weighted` – boolean (default: `None`); whether graph thinks of itself as weighted or not. See *weighted()*.
- `loops` – boolean (default: `None`); whether to allow loops
- `multiedges` – boolean (default: `None`); whether to allow multiple edges
- `convert_empty_dict_labels_to_None` – boolean (default: `None`); whether to replace the default edge labels used by NetworkX (empty dictionaries) by `None`, the default Sage edge label. When set to `False`, empty dictionaries are not converted to `None`.

EXAMPLES:

Feeding a *Graph* with a NetworkX `Graph`:

```
sage: # needs networkx
sage: from sage.graphs.graph_input import from_networkx_graph
sage: import networkx
sage: G = Graph()
sage: _ = gnx = networkx.Graph()
sage: _ = gnx.add_edge(0, 1)
sage: _ = gnx.add_edge(1, 2)
sage: from_networkx_graph(G, gnx)
sage: G.edges(sort=True, labels=False)
[(0, 1), (1, 2)]
```

```
>>> from sage.all import *
>>> # needs networkx
>>> from sage.graphs.graph_input import from_networkx_graph
>>> import networkx
>>> G = Graph()
>>> _ = gnx = networkx.Graph()
>>> _ = gnx.add_edge(Integer(0), Integer(1))
>>> _ = gnx.add_edge(Integer(1), Integer(2))
>>> from_networkx_graph(G, gnx)
>>> G.edges(sort=True, labels=False)
[(0, 1), (1, 2)]
```

Feeding a *Graph* with a NetworkX `MultiGraph`:

```
sage: # needs networkx
sage: G = Graph()
sage: gnx = networkx.MultiGraph()
sage: _ = gnx.add_edge(0, 1)
sage: _ = gnx.add_edge(0, 1)
sage: from_networkx_graph(G, gnx)
sage: G.edges(sort=True, labels=False)
[(0, 1), (0, 1)]
sage: G = Graph()
sage: from_networkx_graph(G, gnx, multiedges=False)
sage: G.edges(sort=True, labels=False)
[(0, 1)]
```

```
>>> from sage.all import *
>>> # needs networkx
>>> G = Graph()
>>> gnx = networkx.MultiGraph()
```

(continues on next page)

(continued from previous page)

```
>>> _ = gnx.add_edge(Integer(0), Integer(1))
>>> _ = gnx.add_edge(Integer(0), Integer(1))
>>> from_networkx_graph(G, gnx)
>>> G.edges(sort=True, labels=False)
[(0, 1), (0, 1)]
>>> G = Graph()
>>> from_networkx_graph(G, gnx, multiedges=False)
>>> G.edges(sort=True, labels=False)
[(0, 1)]
```

When feeding a *Graph* G with a NetworkX `DiGraph` D, G has one edge (u, v) whenever D has arc (u, v) or (v, u) or both:

```
sage: # needs networkx
sage: G = Graph()
sage: D = networkx.DiGraph()
sage: _ = D.add_edge(0, 1)
sage: from_networkx_graph(G, D)
sage: G.edges(sort=True, labels=False)
[(0, 1)]
sage: G = Graph()
sage: _ = D.add_edge(1, 0)
sage: from_networkx_graph(G, D)
sage: G.edges(sort=True, labels=False)
[(0, 1)]
```

```
>>> from sage.all import *
>>> # needs networkx
>>> G = Graph()
>>> D = networkx.DiGraph()
>>> _ = D.add_edge(Integer(0), Integer(1))
>>> from_networkx_graph(G, D)
>>> G.edges(sort=True, labels=False)
[(0, 1)]
>>> G = Graph()
>>> _ = D.add_edge(Integer(1), Integer(0))
>>> from_networkx_graph(G, D)
>>> G.edges(sort=True, labels=False)
[(0, 1)]
```

When feeding a *Graph* G with a NetworkX `MultiDiGraph` D, the number of edges between u and v in G is the maximum between the number of arcs (u, v) and the number of arcs (v, u) in D`:

```
sage: # needs networkx
sage: G = Graph()
sage: D = networkx.MultiDiGraph()
sage: _ = D.add_edge(0, 1)
sage: _ = D.add_edge(1, 0)
sage: _ = D.add_edge(1, 0)
sage: D.edges()
OutMultiEdgeDataView([(0, 1), (1, 0), (1, 0)])
sage: from_networkx_graph(G, D)
```

(continues on next page)

(continued from previous page)

```
sage: G.edges(sort=True, labels=False)
[(0, 1), (0, 1)]
```

```
>>> from sage.all import *
>>> # needs networkx
>>> G = Graph()
>>> D = networkx.MultiDiGraph()
>>> _ = D.add_edge(Integer(0), Integer(1))
>>> _ = D.add_edge(Integer(1), Integer(0))
>>> _ = D.add_edge(Integer(1), Integer(0))
>>> D.edges()
OutMultiEdgeDataView([(0, 1), (1, 0), (1, 0)])
>>> from_networkx_graph(G, D)
>>> G.edges(sort=True, labels=False)
[(0, 1), (0, 1)]
```

Feeding a *DiGraph* with a NetworkX `DiGraph`:

```
sage: # needs networkx
sage: from sage.graphs.graph_input import from_networkx_graph
sage: import networkx
sage: G = DiGraph()
sage: _ = gnx = networkx.DiGraph()
sage: _ = gnx.add_edge(0, 1)
sage: _ = gnx.add_edge(1, 2)
sage: from_networkx_graph(G, gnx)
sage: G.edges(sort=True, labels=False)
[(0, 1), (1, 2)]
```

```
>>> from sage.all import *
>>> # needs networkx
>>> from sage.graphs.graph_input import from_networkx_graph
>>> import networkx
>>> G = DiGraph()
>>> _ = gnx = networkx.DiGraph()
>>> _ = gnx.add_edge(Integer(0), Integer(1))
>>> _ = gnx.add_edge(Integer(1), Integer(2))
>>> from_networkx_graph(G, gnx)
>>> G.edges(sort=True, labels=False)
[(0, 1), (1, 2)]
```

Feeding a *DiGraph* with a NetworkX `MultiDiGraph`:

```
sage: # needs networkx
sage: G = DiGraph()
sage: gnx = networkx.MultiDiGraph()
sage: _ = gnx.add_edge(0, 1)
sage: _ = gnx.add_edge(0, 1)
sage: from_networkx_graph(G, gnx)
sage: G.edges(sort=True, labels=False)
[(0, 1), (0, 1)]
sage: G = DiGraph()
```

(continues on next page)

(continued from previous page)

```
sage: from_networkx_graph(G, gnx, multiedges=False)
sage: G.edges(sort=True, labels=False)
[(0, 1)]
```

```
>>> from sage.all import *
>>> # needs networkx
>>> G = DiGraph()
>>> gnx = networkx.MultiDiGraph()
>>> _ = gnx.add_edge(Integer(0), Integer(1))
>>> _ = gnx.add_edge(Integer(0), Integer(1))
>>> from_networkx_graph(G, gnx)
>>> G.edges(sort=True, labels=False)
[(0, 1), (0, 1)]
>>> G = DiGraph()
>>> from_networkx_graph(G, gnx, multiedges=False)
>>> G.edges(sort=True, labels=False)
[(0, 1)]
```

When feeding a *DiGraph* G with a NetworkX `Graph` H, G has both arcs (u, v) and (v, u) if G has edge (u, v):

```
sage: # needs networkx
sage: G = DiGraph()
sage: H = networkx.Graph()
sage: _ = H.add_edge(0, 1)
sage: from_networkx_graph(G, H)
sage: G.edges(labels=False, sort=True)
[(0, 1), (1, 0)]
```

```
>>> from sage.all import *
>>> # needs networkx
>>> G = DiGraph()
>>> H = networkx.Graph()
>>> _ = H.add_edge(Integer(0), Integer(1))
>>> from_networkx_graph(G, H)
>>> G.edges(labels=False, sort=True)
[(0, 1), (1, 0)]
```

When feeding a *DiGraph* G with a NetworkX `MultiGraph` H, G has k arcs (u, v) and k arcs (v, u) if H has k edges (u, v), unless parameter `multiedges` is set to `False`:

```
sage: # needs networkx
sage: G = DiGraph()
sage: H = networkx.MultiGraph()
sage: _ = H.add_edge(0, 1)
sage: _ = H.add_edge(0, 1)
sage: _ = H.add_edge(0, 1)
sage: H.edges()
MultiEdgeDataView([(0, 1), (0, 1), (0, 1)])
sage: from_networkx_graph(G, H)
sage: G.edges(labels=False, sort=True)
[(0, 1), (0, 1), (0, 1), (1, 0), (1, 0), (1, 0)]
sage: G = DiGraph()
```

(continues on next page)

(continued from previous page)

```
sage: from_networkx_graph(G, H, multiedges=False)
sage: G.edges(labels=False, sort=True)
[(0, 1), (1, 0)]
```

```
>>> from sage.all import *
>>> # needs networkx
>>> G = DiGraph()
>>> H = networkx.MultiGraph()
>>> _ = H.add_edge(Integer(0), Integer(1))
>>> _ = H.add_edge(Integer(0), Integer(1))
>>> _ = H.add_edge(Integer(0), Integer(1))
>>> H.edges()
MultiEdgeDataView([(0, 1), (0, 1), (0, 1)])
>>> from_networkx_graph(G, H)
>>> G.edges(labels=False, sort=True)
[(0, 1), (0, 1), (0, 1), (1, 0), (1, 0), (1, 0)]
>>> G = DiGraph()
>>> from_networkx_graph(G, H, multiedges=False)
>>> G.edges(labels=False, sort=True)
[(0, 1), (1, 0)]
```

sage.graphs.graph_input.**from_oriented_incidence_matrix**(*G*, *M*, *loops=False*, *multiedges=False*, *weighted=False*)

Fill `G` with the data of an *oriented* incidence matrix.

An oriented incidence matrix is the incidence matrix of a directed graph, in which each non-loop edge corresponds to a $+1$ and a -1, indicating its source and destination.

INPUT:

- `G` – a *DiGraph*
- `M` – an incidence matrix
- `loops`, `multiedges`, `weighted` – booleans (default: `False`); whether to consider the graph as having loops, multiple edges, or weights

Note

`weighted` is currently ignored.

EXAMPLES:

```
sage: from sage.graphs.graph_input import from_oriented_incidence_matrix
sage: g = DiGraph()
sage: im = digraphs.Circuit(10).incidence_matrix()                          #␣
↪needs sage.modules
sage: from_oriented_incidence_matrix(g, im)                                 #␣
↪needs sage.modules
sage: g.is_isomorphic(digraphs.Circuit(10))                                 #␣
↪needs sage.modules
True
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_input import from_oriented_incidence_matrix
>>> g = DiGraph()
>>> im = digraphs.Circuit(Integer(10)).incidence_matrix()                    ␣
↪     # needs sage.modules
>>> from_oriented_incidence_matrix(g, im)                                   #␣
↪needs sage.modules
>>> g.is_isomorphic(digraphs.Circuit(Integer(10)))                           ␣
↪     # needs sage.modules
True
```

sage.graphs.graph_input.**from_seidel_adjacency_matrix**(*G*, *M*)

Fill `G` with the data of a Seidel adjacency matrix.

INPUT:

- `G` – a graph
- `M` – a Seidel adjacency matrix

EXAMPLES:

```
sage: from sage.graphs.graph_input import from_seidel_adjacency_matrix
sage: g = Graph()
sage: sam = graphs.PetersenGraph().seidel_adjacency_matrix()              #␣
↪needs sage.modules
sage: from_seidel_adjacency_matrix(g, sam)                                #␣
↪needs sage.modules
sage: g.is_isomorphic(graphs.PetersenGraph())                             #␣
↪needs sage.modules
True
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_input import from_seidel_adjacency_matrix
>>> g = Graph()
>>> sam = graphs.PetersenGraph().seidel_adjacency_matrix()                #␣
↪needs sage.modules
>>> from_seidel_adjacency_matrix(g, sam)                                  #␣
↪needs sage.modules
>>> g.is_isomorphic(graphs.PetersenGraph())                               #␣
↪needs sage.modules
True
```

sage.graphs.graph_input.**from_sparse6**(*G*, *g6_string*)

Fill `G` with the data of a sparse6 string.

INPUT:

- `G` – a graph
- `g6_string` – a sparse6 string

EXAMPLES:

```
sage: from sage.graphs.graph_input import from_sparse6
sage: g = Graph()
```

(continues on next page)

(continued from previous page)

```
sage: from_sparse6(g, ':I`ES@obGkqegW~')
sage: g.is_isomorphic(graphs.PetersenGraph())
True
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_input import from_sparse6
>>> g = Graph()
>>> from_sparse6(g, ':I`ES@obGkqegW~')
>>> g.is_isomorphic(graphs.PetersenGraph())
True
```

5.40 Hyperbolicity

Definition :

The hyperbolicity δ of a graph G has been defined by Gromov [Gro1987] as follows (we give here the so-called 4-points condition):

Let a, b, c, d be vertices of the graph, let S_1, S_2 and S_3 be defined by

$$S_1 = dist(a, b) + dist(d, c)$$
$$S_2 = dist(a, c) + dist(b, d)$$
$$S_3 = dist(a, d) + dist(b, c)$$

and let M_1 and M_2 be the two largest values among S_1, S_2, and S_3. We define $hyp(a, b, c, d) = M_1 - M_2$, and the hyperbolicity $\delta(G)$ of the graph is the maximum of hyp over all possible 4-tuples (a, b, c, d) divided by 2. That is, the graph is said δ-hyperbolic when

$$\delta(G) = \frac{1}{2} \max_{a,b,c,d \in V(G)} hyp(a, b, c, d)$$

(note that $hyp(a, b, c, d) = 0$ whenever two elements among a, b, c, d are equal)

Some known results :

- Trees and cliques are 0-hyperbolic
- $n \times n$ grids are $n - 1$-hyperbolic
- Cycles are approximately $n/4$-hyperbolic
- Chordal graphs are ≤ 1-hyperbolic

Besides, the hyperbolicity of a graph is the maximum over all its biconnected components.

Algorithms and complexity :

The time complexity of the naive implementation (i.e. testing all 4-tuples) is $O(n^4)$, and an algorithm with time complexity $O(n^{3.69})$ has been proposed in [FIV2012]. This remains very long for large-scale graphs, and much harder to implement.

Several improvements over the naive algorithm have been proposed and are implemented in the current module.

- Another upper bound on $hyp(a, b, c, d)$ has been proved in [CCL2015]. It is used to design an algorithm with worse case time complexity in $O(n^4)$ but that behaves much better in practice.

Assume that $S_1 = dist(a,b) + dist(c,d)$ is the largest sum among S_1, S_2, S_3. We have

$$\begin{aligned} S_2 + S_3 =& dist(a,c) + dist(b,d) + dist(a,d) + dist(b,c) \\ =& [dist(a,c) + dist(b,c)] + [dist(a,d) + dist(b,d)] \\ \geq& dist(a,b) + dist(a,b) \\ \geq& 2dist(a,b) \end{aligned}$$

Now, since S_1 is the largest sum, we have

$$\begin{aligned} hyp(a,b,c,d) =& S_1 - \max\{S_2, S_3\} \\ \leq& S_1 - \frac{S_2 + S_3}{2} \\ \leq& S_1 - dist(a,b) \\ =& dist(c,d) \end{aligned}$$

We obtain similarly that $hyp(a,b,c,d) \leq dist(a,b)$. Consequently, in the implementation of the 'CCL' algorithm, we ensure that S_1 is larger than S_2 and S_3 using an ordering of the pairs by decreasing lengths. Then, we use the best value h found so far to stop exploration as soon as $dist(a,b) \leq h$.

The worst case time complexity of this algorithm is $O(n^4)$, but it performs very well in practice since it cuts the search space. This algorithm can be turned into an approximation algorithm since at any step of its execution we maintain an upper and a lower bound. We can thus stop execution as soon as a multiplicative approximation factor or an additive one is proven.

- The notion of "far-apart pairs" has been introduced in [Sot2011] to further reduce the number of 4-tuples to consider. We say that the pair (a,b) is far-apart if for every w in $V \setminus \{a,b\}$ we have

$$dist(w,a) + dist(a,b) > dist(w,b) \text{ and } dist(w,b) + dist(a,b) > dist(w,a)$$

Determining the set of far-apart pairs can be done in time $O(nm)$ using BFS. Now, it is proved in [Sot2011] that there exists two far-apart pairs (a,b) and (c,d) satisfying $\delta(G) = hyp(a,b,c,d)/2$. For instance, the $n \times m$-grid has only two far-apart pairs, and so computing its hyperbolicity is immediate once the far-apart pairs are found. The 'CCL+FA' or 'CCL+' algorithm improves the 'CCL' algorithm since it uses far-apart pairs.

- This algorithm was further improved in [BCCM2015]: instead of iterating twice over all pairs of vertices, in the "inner" loop, we cut several pairs by exploiting properties of the underlying graph.

Todo

- Add exact methods for the hyperbolicity of chordal graphs
- Add method for partitioning the graph with clique separators

This module contains the following functions

At Python level :

hyperbolicity()	Return the hyperbolicity of the graph or an approximation of this value.
hyperbolicity_distribution()	Return the hyperbolicity distribution of the graph or a sampling of it.

AUTHORS:

- David Coudert (2012): initial version, exact and approximate algorithm, distribution, sampling

- David Coudert (2014): improved exact algorithm using far-apart pairs
- Michele Borassi (2015): cleaned the code and implemented the new algorithm
- Karan Desai (2016): fixed minor typo in documentation

5.40.1 Methods

sage.graphs.hyperbolicity.**hyperbolicity**(*G*, *algorithm='BCCM'*, *approximation_factor=None*, *additive_gap=None*, *verbose=False*)

Return the hyperbolicity of the graph or an approximation of this value.

The hyperbolicity of a graph has been defined by Gromov [Gro1987] as follows: Let a, b, c, d be vertices of the graph, let $S_1 = dist(a, b) + dist(b, c)$, $S_2 = dist(a, c) + dist(b, d)$, and $S_3 = dist(a, d) + dist(b, c)$, and let M_1 and M_2 be the two largest values among S_1, S_2, and S_3. We have $hyp(a, b, c, d) = |M_1 - M_2|$, and the hyperbolicity of the graph is the maximum over all possible 4-tuples (a, b, c, d) divided by 2. The worst case time complexity is in $O(n^4)$.

See the documentation of *sage.graphs.hyperbolicity* for more information.

INPUT:

- `G` – a connected Graph
- `algorithm` – (default: `'BCCM'`) specifies the algorithm to use among:
  - `'basic'` is an exhaustive algorithm considering all possible 4-tuples and so have time complexity in $O(n^4)$.
  - `'CCL'` is an exact algorithm proposed in [CCL2015]. It considers the 4-tuples in an ordering allowing to cut the search space as soon as a new lower bound is found (see the module's documentation). This algorithm can be turned into a approximation algorithm.
  - `'CCL+FA'` or `'CCL+'` uses the notion of far-apart pairs as proposed in [Sot2011] to significantly reduce the overall computation time of the `'CCL'` algorithm.
  - `'BCCM'` is an exact algorithm proposed in [BCCM2015]. It improves `'CCL+FA'` by cutting several 4-tuples (for more information, see the module's documentation).
  - `'dom'` is an approximation with additive constant four. It computes the hyperbolicity of the vertices of a dominating set of the graph. This is sometimes slower than `'CCL'` and sometimes faster. Try it to know if it is interesting for you. The `additive_gap` and `approximation_factor` parameters cannot be used in combination with this method and so are ignored.
- `approximation_factor` – (default: `None`) when the approximation factor is set to some value (larger than 1.0), the function stop computations as soon as the ratio between the upper bound and the best found solution is less than the approximation factor. When the approximation factor is 1.0, the problem is solved optimally. This parameter is used only when the chosen algorithm is `'CCL'`, `'CCL+FA'`, or `'BCCM'`.
- `additive_gap` – (default: `None`) when set to a positive number, the function stop computations as soon as the difference between the upper bound and the best found solution is less than additive gap. When the gap is 0.0, the problem is solved optimally. This parameter is used only when the chosen algorithm is `'CCL'` or `'CCL+FA'`, or `'BCCM'`.
- `verbose` – boolean (default: `False`); set to `True` to display some information during execution: new upper and lower bounds, etc.

OUTPUT:

This function returns the tuple (delta, certificate, delta_UB), where:

- `delta` – the hyperbolicity of the graph (half-integer value)

- `certificate` – is the list of the 4 vertices for which the maximum value has been computed, and so the hyperbolicity of the graph
- `delta_UB` – is an upper bound for `delta`. When `delta == delta_UB`, the returned solution is optimal. Otherwise, the approximation factor if `delta_UB/delta`.

EXAMPLES:

Hyperbolicity of a 3×3 grid:

```
sage: from sage.graphs.hyperbolicity import hyperbolicity
sage: G = graphs.Grid2dGraph(3, 3)
sage: L,C,U = hyperbolicity(G, algorithm='BCCM'); L,sorted(C),U
(2, [(0, 0), (0, 2), (2, 0), (2, 2)], 2)
sage: L,C,U = hyperbolicity(G, algorithm='CCL'); L,sorted(C),U
(2, [(0, 0), (0, 2), (2, 0), (2, 2)], 2)
sage: L,C,U = hyperbolicity(G, algorithm='basic'); L,sorted(C),U
(2, [(0, 0), (0, 2), (2, 0), (2, 2)], 2)
```

```
>>> from sage.all import *
>>> from sage.graphs.hyperbolicity import hyperbolicity
>>> G = graphs.Grid2dGraph(Integer(3), Integer(3))
>>> L,C,U = hyperbolicity(G, algorithm='BCCM'); L,sorted(C),U
(2, [(0, 0), (0, 2), (2, 0), (2, 2)], 2)
>>> L,C,U = hyperbolicity(G, algorithm='CCL'); L,sorted(C),U
(2, [(0, 0), (0, 2), (2, 0), (2, 2)], 2)
>>> L,C,U = hyperbolicity(G, algorithm='basic'); L,sorted(C),U
(2, [(0, 0), (0, 2), (2, 0), (2, 2)], 2)
```

Hyperbolicity of a PetersenGraph:

```
sage: from sage.graphs.hyperbolicity import hyperbolicity
sage: G = graphs.PetersenGraph()
sage: L,C,U = hyperbolicity(G, algorithm='BCCM'); L,sorted(C),U
(1/2, [6, 7, 8, 9], 1/2)
sage: L,C,U = hyperbolicity(G, algorithm='CCL'); L,sorted(C),U
(1/2, [0, 1, 2, 3], 1/2)
sage: L,C,U = hyperbolicity(G, algorithm='CCL+'); L,sorted(C),U
(1/2, [0, 1, 2, 3], 1/2)
sage: L,C,U = hyperbolicity(G, algorithm='CCL+FA'); L,sorted(C),U
(1/2, [0, 1, 2, 3], 1/2)
sage: L,C,U = hyperbolicity(G, algorithm='basic'); L,sorted(C),U
(1/2, [0, 1, 2, 3], 1/2)
sage: L,C,U = hyperbolicity(G, algorithm='dom'); L,U
(0, 1)
sage: sorted(C)  # random
[0, 1, 2, 6]
```

```
>>> from sage.all import *
>>> from sage.graphs.hyperbolicity import hyperbolicity
>>> G = graphs.PetersenGraph()
>>> L,C,U = hyperbolicity(G, algorithm='BCCM'); L,sorted(C),U
(1/2, [6, 7, 8, 9], 1/2)
>>> L,C,U = hyperbolicity(G, algorithm='CCL'); L,sorted(C),U
(1/2, [0, 1, 2, 3], 1/2)
```

(continues on next page)

(continued from previous page)

```
>>> L,C,U = hyperbolicity(G, algorithm='CCL+'); L,sorted(C),U
(1/2, [0, 1, 2, 3], 1/2)
>>> L,C,U = hyperbolicity(G, algorithm='CCL+FA'); L,sorted(C),U
(1/2, [0, 1, 2, 3], 1/2)
>>> L,C,U = hyperbolicity(G, algorithm='basic'); L,sorted(C),U
(1/2, [0, 1, 2, 3], 1/2)
>>> L,C,U = hyperbolicity(G, algorithm='dom'); L,U
(0, 1)
>>> sorted(C)  # random
[0, 1, 2, 6]
```

Asking for an approximation in a grid graph:

```
sage: from sage.graphs.hyperbolicity import hyperbolicity
sage: G = graphs.Grid2dGraph(2, 10)
sage: L,C,U = hyperbolicity(G, algorithm='CCL', approximation_factor=1.5); L,U
(1, 3/2)
sage: L,C,U = hyperbolicity(G, algorithm='CCL+', approximation_factor=1.5); L,U
(1, 1)
sage: L,C,U = hyperbolicity(G, algorithm='CCL', approximation_factor=4); L,U
(1, 4)
sage: L,C,U = hyperbolicity(G, algorithm='CCL', additive_gap=2); L,U
(1, 3)
sage: L,C,U = hyperbolicity(G, algorithm='dom'); L,U
(1, 5)
```

```
>>> from sage.all import *
>>> from sage.graphs.hyperbolicity import hyperbolicity
>>> G = graphs.Grid2dGraph(Integer(2), Integer(10))
>>> L,C,U = hyperbolicity(G, algorithm='CCL', approximation_factor=RealNumber('1.5
↪')); L,U
(1, 3/2)
>>> L,C,U = hyperbolicity(G, algorithm='CCL+', approximation_factor=RealNumber('1.
↪5')); L,U
(1, 1)
>>> L,C,U = hyperbolicity(G, algorithm='CCL', approximation_factor=Integer(4)); L,
↪U
(1, 4)
>>> L,C,U = hyperbolicity(G, algorithm='CCL', additive_gap=Integer(2)); L,U
(1, 3)
>>> L,C,U = hyperbolicity(G, algorithm='dom'); L,U
(1, 5)
```

Asking for an approximation in a cycle graph:

```
sage: from sage.graphs.hyperbolicity import hyperbolicity
sage: G = graphs.CycleGraph(10)
sage: L,C,U = hyperbolicity(G, algorithm='CCL', approximation_factor=1.5); L,U
(2, 5/2)
sage: L,C,U = hyperbolicity(G, algorithm='CCL+FA', approximation_factor=1.5); L,U
(2, 5/2)
sage: L,C,U = hyperbolicity(G, algorithm='CCL+FA', additive_gap=1); L,U
```

(continues on next page)

(continued from previous page)

```
(2, 5/2)
```

```
>>> from sage.all import *
>>> from sage.graphs.hyperbolicity import hyperbolicity
>>> G = graphs.CycleGraph(Integer(10))
>>> L,C,U = hyperbolicity(G, algorithm='CCL', approximation_factor=RealNumber('1.5
↪')); L,U
(2, 5/2)
>>> L,C,U = hyperbolicity(G, algorithm='CCL+FA', approximation_factor=RealNumber(
↪'1.5')); L,U
(2, 5/2)
>>> L,C,U = hyperbolicity(G, algorithm='CCL+FA', additive_gap=Integer(1)); L,U
(2, 5/2)
```

Comparison of results:

```
sage: from sage.graphs.hyperbolicity import hyperbolicity
sage: for i in range(10):                # long time                          #␣
↪needs networkx
....:     G = graphs.RandomBarabasiAlbert(100,2)
....:     d1,_,_ = hyperbolicity(G, algorithm='basic')
....:     d2,_,_ = hyperbolicity(G, algorithm='CCL')
....:     d3,_,_ = hyperbolicity(G, algorithm='CCL+')
....:     d4,_,_ = hyperbolicity(G, algorithm='CCL+FA')
....:     d5,_,_ = hyperbolicity(G, algorithm='BCCM')
....:     l3,_,u3 = hyperbolicity(G, approximation_factor=2)
....:     if (not d1==d2==d3==d4==d5) or l3>d1 or u3<d1:
....:        print("That's not good!")

sage: from sage.graphs.hyperbolicity import hyperbolicity
sage: import random
sage: random.seed()
sage: for i in range(10):                # long time                          #␣
↪needs networkx
....:     n = random.randint(2, 20)
....:     m = random.randint(0, n*(n-1) / 2)
....:     G = graphs.RandomGNM(n, m)
....:     for cc in G.connected_components_subgraphs():
....:         d1,_,_ = hyperbolicity(cc, algorithm='basic')
....:         d2,_,_ = hyperbolicity(cc, algorithm='CCL')
....:         d3,_,_ = hyperbolicity(cc, algorithm='CCL+')
....:         d4,_,_ = hyperbolicity(cc, algorithm='CCL+FA')
....:         d5,_,_ = hyperbolicity(cc, algorithm='BCCM')
....:         l3,_,u3 = hyperbolicity(cc, approximation_factor=2)
....:         if (not d1==d2==d3==d4==d5) or l3>d1 or u3<d1:
....:             print("Error in graph ", cc.edges(sort=True))
```

```
>>> from sage.all import *
>>> from sage.graphs.hyperbolicity import hyperbolicity
>>> for i in range(Integer(10)):                # long time                  ␣
↪     # needs networkx
...     G = graphs.RandomBarabasiAlbert(Integer(100),Integer(2))
```

(continues on next page)

(continued from previous page)

```
...     d1,_,_ = hyperbolicity(G, algorithm='basic')
...     d2,_,_ = hyperbolicity(G, algorithm='CCL')
...     d3,_,_ = hyperbolicity(G, algorithm='CCL+')
...     d4,_,_ = hyperbolicity(G, algorithm='CCL+FA')
...     d5,_,_ = hyperbolicity(G, algorithm='BCCM')
...     l3,_,u3 = hyperbolicity(G, approximation_factor=Integer(2))
...     if (not d1==d2==d3==d4==d5) or l3>d1 or u3<d1:
...        print("That's not good!")

>>> from sage.graphs.hyperbolicity import hyperbolicity
>>> import random
>>> random.seed()
>>> for i in range(Integer(10)):                # long time
...     # needs networkx
...     n = random.randint(Integer(2), Integer(20))
...     m = random.randint(Integer(0), n*(n-Integer(1)) / Integer(2))
...     G = graphs.RandomGNM(n, m)
...     for cc in G.connected_components_subgraphs():
...         d1,_,_ = hyperbolicity(cc, algorithm='basic')
...         d2,_,_ = hyperbolicity(cc, algorithm='CCL')
...         d3,_,_ = hyperbolicity(cc, algorithm='CCL+')
...         d4,_,_ = hyperbolicity(cc, algorithm='CCL+FA')
...         d5,_,_ = hyperbolicity(cc, algorithm='BCCM')
...         l3,_,u3 = hyperbolicity(cc, approximation_factor=Integer(2))
...         if (not d1==d2==d3==d4==d5) or l3>d1 or u3<d1:
...             print("Error in graph ", cc.edges(sort=True))
```

The hyperbolicity of a graph is the maximum value over all its biconnected components:

```
sage: from sage.graphs.hyperbolicity import hyperbolicity
sage: G = graphs.PetersenGraph() * 2
sage: G.add_edge(0, 11)
sage: L,C,U = hyperbolicity(G); L,sorted(C),U
(1/2, [6, 7, 8, 9], 1/2)
```

```
>>> from sage.all import *
>>> from sage.graphs.hyperbolicity import hyperbolicity
>>> G = graphs.PetersenGraph() * Integer(2)
>>> G.add_edge(Integer(0), Integer(11))
>>> L,C,U = hyperbolicity(G); L,sorted(C),U
(1/2, [6, 7, 8, 9], 1/2)
```

sage.graphs.hyperbolicity.**hyperbolicity_distribution**(*G*, *algorithm='sampling'*, *sampling_size=1000000*)

Return the hyperbolicity distribution of the graph or a sampling of it.

The hyperbolicity of a graph has been defined by Gromov [Gro1987] as follows: Let a, b, c, d be vertices of the graph, let $S_1 = dist(a, b) + dist(b, c)$, $S_2 = dist(a, c) + dist(b, d)$, and $S_3 = dist(a, d) + dist(b, c)$, and let M_1 and M_2 be the two largest values among S_1, S_2, and S_3. We have $hyp(a, b, c, d) = |M_1 - M_2|$, and the hyperbolicity of the graph is the maximum over all possible 4-tuples (a, b, c, d) divided by 2.

The computation of the hyperbolicity of each 4-tuple, and so the hyperbolicity distribution, takes time in $O(n^4)$.

INPUT:

- `G` – a Graph
- `algorithm` – (default: `'sampling'`) when algorithm is 'sampling', it returns the distribution of the hyperbolicity over a sample of `sampling_size` 4-tuples. When algorithm is 'exact', it computes the distribution of the hyperbolicity over all 4-tuples. Be aware that the computation time can be HUGE.
- `sampling_size` – (default: 10^6) number of 4-tuples considered in the sampling. Used only when `algorithm == 'sampling'`

OUTPUT:

- `hdict` – dictionary such that hdict[i] is the number of 4-tuples of hyperbolicity i

EXAMPLES:

Exact hyperbolicity distribution of the Petersen Graph:

```
sage: from sage.graphs.hyperbolicity import hyperbolicity_distribution
sage: G = graphs.PetersenGraph()
sage: hyperbolicity_distribution(G,algorithm='exact')
{0: 3/7, 1/2: 4/7}
```

```
>>> from sage.all import *
>>> from sage.graphs.hyperbolicity import hyperbolicity_distribution
>>> G = graphs.PetersenGraph()
>>> hyperbolicity_distribution(G,algorithm='exact')
{0: 3/7, 1/2: 4/7}
```

Exact hyperbolicity distribution of a 3×3 grid:

```
sage: from sage.graphs.hyperbolicity import hyperbolicity_distribution
sage: G = graphs.GridGraph([3,3])
sage: hyperbolicity_distribution(G,algorithm='exact')
{0: 11/18, 1: 8/21, 2: 1/126}
```

```
>>> from sage.all import *
>>> from sage.graphs.hyperbolicity import hyperbolicity_distribution
>>> G = graphs.GridGraph([Integer(3),Integer(3)])
>>> hyperbolicity_distribution(G,algorithm='exact')
{0: 11/18, 1: 8/21, 2: 1/126}
```

5.41 Tutte polynomial

This module implements a deletion-contraction algorithm for computing the Tutte polynomial as described in the paper [HPR2010].

tutte_polynomial()	Compute the Tutte polynomial of the input graph

Authors:

- Mike Hansen (06-2013), Implemented the algorithm.
- Jernej Azarija (06-2013), Tweaked the code, added documentation

5.41.1 Definition

Given a graph G, with n vertices and m edges and $k(G)$ connected components we define the Tutte polynomial of G as

$$\sum_H (x-1)^{k(H)-c}(y-1)^{k(H)-|E(H)|-n}$$

where the sum ranges over all induced subgraphs H of G.

5.41.2 Functions

class sage.graphs.tutte_polynomial.**Ear**(*graph*, *end_points*, *interior*, *is_cycle*)

Bases: object

An ear is a sequence of vertices

Here is the definition from [HPR2010]:

An ear in a graph is a path $v_1 - v_2 - \cdots - v_n - v_{n+1}$ where $d(v_1) > 2$, $d(v_{n+1}) > 2$ and $d(v_2) = d(v_3) = \cdots = d(v_n) = 2$.

A cycle is viewed as a special ear where $v_1 = v_{n+1}$ and the restriction on the degree of this vertex is lifted.

INPUT:

static find_ear(*g*)

Finds the first ear in a graph.

EXAMPLES:

```
sage: G = graphs.PathGraph(4)
sage: G.add_edges([(0,4),(0,5),(3,6),(3,7)])
sage: from sage.graphs.tutte_polynomial import Ear
sage: E = Ear.find_ear(G)
sage: E.s
3
sage: E.unlabeled_edges
[(0, 1), (1, 2), (2, 3)]
sage: E.vertices
[0, 1, 2, 3]
```

```
>>> from sage.all import *
>>> G = graphs.PathGraph(Integer(4))
>>> G.add_edges([(Integer(0),Integer(4)),(Integer(0),Integer(5)),(Integer(3),
↪Integer(6)),(Integer(3),Integer(7))])
>>> from sage.graphs.tutte_polynomial import Ear
>>> E = Ear.find_ear(G)
>>> E.s
3
>>> E.unlabeled_edges
[(0, 1), (1, 2), (2, 3)]
>>> E.vertices
[0, 1, 2, 3]
```

removed_from(**args*, ***kwds*)

A context manager which removes the ear from the graph G.

EXAMPLES:

```
sage: G = graphs.PathGraph(4)
sage: G.add_edges([(0,4),(0,5),(3,6),(3,7)])
sage: len(G.edges(sort=True))
7
sage: from sage.graphs.tutte_polynomial import Ear
sage: E = Ear.find_ear(G)
sage: with E.removed_from(G) as Y:
....:     G.edges(sort=True)
[(0, 4, None), (0, 5, None), (3, 6, None), (3, 7, None)]
sage: len(G.edges(sort=True))
7
```

```
>>> from sage.all import *
>>> G = graphs.PathGraph(Integer(4))
>>> G.add_edges([(Integer(0),Integer(4)),(Integer(0),Integer(5)),(Integer(3),
↪Integer(6)),(Integer(3),Integer(7))])
>>> len(G.edges(sort=True))
7
>>> from sage.graphs.tutte_polynomial import Ear
>>> E = Ear.find_ear(G)
>>> with E.removed_from(G) as Y:
...     G.edges(sort=True)
[(0, 4, None), (0, 5, None), (3, 6, None), (3, 7, None)]
>>> len(G.edges(sort=True))
7
```

property s

Return the number of distinct edges in this ear.

EXAMPLES:

```
sage: G = graphs.PathGraph(4)
sage: G.add_edges([(0,4),(0,5),(3,6),(3,7)])
sage: from sage.graphs.tutte_polynomial import Ear
sage: E = Ear(G,[0,3],[1,2],False)
sage: E.s
3
```

```
>>> from sage.all import *
>>> G = graphs.PathGraph(Integer(4))
>>> G.add_edges([(Integer(0),Integer(4)),(Integer(0),Integer(5)),(Integer(3),
↪Integer(6)),(Integer(3),Integer(7))])
>>> from sage.graphs.tutte_polynomial import Ear
>>> E = Ear(G,[Integer(0),Integer(3)],[Integer(1),Integer(2)],False)
>>> E.s
3
```

unlabeled_edges()

Return the edges in this ear.

EXAMPLES:

```
sage: G = graphs.PathGraph(4)
sage: G.add_edges([(0,4),(0,5),(3,6),(3,7)])
sage: from sage.graphs.tutte_polynomial import Ear
sage: E = Ear(G,[0,3],[1,2],False)
sage: E.unlabeled_edges
[(0, 1), (1, 2), (2, 3)]
```

```
>>> from sage.all import *
>>> G = graphs.PathGraph(Integer(4))
>>> G.add_edges([(Integer(0),Integer(4)),(Integer(0),Integer(5)),(Integer(3),
↪Integer(6)),(Integer(3),Integer(7))])
>>> from sage.graphs.tutte_polynomial import Ear
>>> E = Ear(G,[Integer(0),Integer(3)],[Integer(1),Integer(2)],False)
>>> E.unlabeled_edges
[(0, 1), (1, 2), (2, 3)]
```

property vertices

Return the vertices of this ear.

EXAMPLES:

```
sage: G = graphs.PathGraph(4)
sage: G.add_edges([(0,4),(0,5),(3,6),(3,7)])
sage: from sage.graphs.tutte_polynomial import Ear
sage: E = Ear(G,[0,3],[1,2],False)
sage: E.vertices
[0, 1, 2, 3]
```

```
>>> from sage.all import *
>>> G = graphs.PathGraph(Integer(4))
>>> G.add_edges([(Integer(0),Integer(4)),(Integer(0),Integer(5)),(Integer(3),
↪Integer(6)),(Integer(3),Integer(7))])
>>> from sage.graphs.tutte_polynomial import Ear
>>> E = Ear(G,[Integer(0),Integer(3)],[Integer(1),Integer(2)],False)
>>> E.vertices
[0, 1, 2, 3]
```

class sage.graphs.tutte_polynomial.**EdgeSelection**

Bases: object

class sage.graphs.tutte_polynomial.**MaximizeDegree**

Bases: *EdgeSelection*

class sage.graphs.tutte_polynomial.**MinimizeDegree**

Bases: *EdgeSelection*

class sage.graphs.tutte_polynomial.**MinimizeSingleDegree**

Bases: *EdgeSelection*

class sage.graphs.tutte_polynomial.**VertexOrder**(*order*)

Bases: *EdgeSelection*

EXAMPLES:

```
sage: from sage.graphs.tutte_polynomial import VertexOrder
sage: A = VertexOrder([4,6,3,2,1,7])
sage: A.order
[4, 6, 3, 2, 1, 7]
sage: A.inverse_order
{1: 4, 2: 3, 3: 2, 4: 0, 6: 1, 7: 5}
```

```
>>> from sage.all import *
>>> from sage.graphs.tutte_polynomial import VertexOrder
>>> A = VertexOrder([Integer(4),Integer(6),Integer(3),Integer(2),Integer(1),
↪Integer(7)])
>>> A.order
[4, 6, 3, 2, 1, 7]
>>> A.inverse_order
{1: 4, 2: 3, 3: 2, 4: 0, 6: 1, 7: 5}
```

sage.graphs.tutte_polynomial.**contracted_edge**(**args*, ***kwds*)

Delete the first vertex in the edge, and make all the edges that went from it go to the second vertex.

EXAMPLES:

```
sage: from sage.graphs.tutte_polynomial import contracted_edge
sage: G = Graph(multiedges=True)
sage: G.add_edges([(0,1,'a'),(1,2,'b'),(0,3,'c')])
sage: G.edges(sort=True)
[(0, 1, 'a'), (0, 3, 'c'), (1, 2, 'b')]
sage: with contracted_edge(G,(0,1)) as Y:
....:     G.edges(sort=True); G.vertices(sort=True)
[(1, 2, 'b'), (1, 3, 'c')]
[1, 2, 3]
sage: G.edges(sort=True)
[(0, 1, 'a'), (0, 3, 'c'), (1, 2, 'b')]
```

```
>>> from sage.all import *
>>> from sage.graphs.tutte_polynomial import contracted_edge
>>> G = Graph(multiedges=True)
>>> G.add_edges([(Integer(0),Integer(1),'a'),(Integer(1),Integer(2),'b'),
↪(Integer(0),Integer(3),'c')])
>>> G.edges(sort=True)
[(0, 1, 'a'), (0, 3, 'c'), (1, 2, 'b')]
>>> with contracted_edge(G,(Integer(0),Integer(1))) as Y:
...     G.edges(sort=True); G.vertices(sort=True)
[(1, 2, 'b'), (1, 3, 'c')]
[1, 2, 3]
>>> G.edges(sort=True)
[(0, 1, 'a'), (0, 3, 'c'), (1, 2, 'b')]
```

sage.graphs.tutte_polynomial.**edge_multiplicities**(*G*)

Return the dictionary of multiplicities of the edges in the graph *G*.

EXAMPLES:

```
sage: from sage.graphs.tutte_polynomial import edge_multiplicities
sage: G = Graph({1: [2,2,3], 2: [2], 3: [4,4], 4: [2,2,2]})
sage: sorted(edge_multiplicities(G).items())
[((1, 2), 2), ((1, 3), 1), ((2, 2), 1), ((2, 4), 3), ((3, 4), 2)]
```

```
>>> from sage.all import *
>>> from sage.graphs.tutte_polynomial import edge_multiplicities
>>> G = Graph({Integer(1): [Integer(2),Integer(2),Integer(3)], Integer(2):␣
↪[Integer(2)], Integer(3): [Integer(4),Integer(4)], Integer(4): [Integer(2),
↪Integer(2),Integer(2)]})
>>> sorted(edge_multiplicities(G).items())
[((1, 2), 2), ((1, 3), 1), ((2, 2), 1), ((2, 4), 3), ((3, 4), 2)]
```

sage.graphs.tutte_polynomial.**removed_edge**(**args*, ***kwds*)

A context manager which removes an edge from the graph G and restores it upon exiting.

EXAMPLES:

```
sage: from sage.graphs.tutte_polynomial import removed_edge
sage: G = Graph()
sage: G.add_edge(0,1)
sage: G.edges(sort=True)
[(0, 1, None)]
sage: with removed_edge(G,(0,1)) as Y:
....:     G.edges(sort=True); G.vertices(sort=True)
[]
[0, 1]
sage: G.edges(sort=True)
[(0, 1, None)]
```

```
>>> from sage.all import *
>>> from sage.graphs.tutte_polynomial import removed_edge
>>> G = Graph()
>>> G.add_edge(Integer(0),Integer(1))
>>> G.edges(sort=True)
[(0, 1, None)]
>>> with removed_edge(G,(Integer(0),Integer(1))) as Y:
...     G.edges(sort=True); G.vertices(sort=True)
[]
[0, 1]
>>> G.edges(sort=True)
[(0, 1, None)]
```

sage.graphs.tutte_polynomial.**removed_loops**(**args*, ***kwds*)

A context manager which removes all the loops in the graph G. It yields a list of the loops, and restores the loops upon exiting.

EXAMPLES:

```
sage: from sage.graphs.tutte_polynomial import removed_loops
sage: G = Graph(multiedges=True, loops=True)
sage: G.add_edges([(0,1,'a'),(1,2,'b'),(0,0,'c')])
sage: G.edges(sort=True)
```

(continues on next page)

(continued from previous page)

```
[(0, 0, 'c'), (0, 1, 'a'), (1, 2, 'b')]
sage: with removed_loops(G) as Y:
....:     G.edges(sort=True); G.vertices(sort=True); Y
[(0, 1, 'a'), (1, 2, 'b')]
[0, 1, 2]
[(0, 0, 'c')]
sage: G.edges(sort=True)
[(0, 0, 'c'), (0, 1, 'a'), (1, 2, 'b')]
```

```
>>> from sage.all import *
>>> from sage.graphs.tutte_polynomial import removed_loops
>>> G = Graph(multiedges=True, loops=True)
>>> G.add_edges([(Integer(0),Integer(1),'a'),(Integer(1),Integer(2),'b'),
↪(Integer(0),Integer(0),'c')])
>>> G.edges(sort=True)
[(0, 0, 'c'), (0, 1, 'a'), (1, 2, 'b')]
>>> with removed_loops(G) as Y:
...     G.edges(sort=True); G.vertices(sort=True); Y
[(0, 1, 'a'), (1, 2, 'b')]
[0, 1, 2]
[(0, 0, 'c')]
>>> G.edges(sort=True)
[(0, 0, 'c'), (0, 1, 'a'), (1, 2, 'b')]
```

sage.graphs.tutte_polynomial.**removed_multiedge**(**args*, ***kwds*)

A context manager which removes an edge with multiplicity from the graph G and restores it upon exiting.

EXAMPLES:

```
sage: from sage.graphs.tutte_polynomial import removed_multiedge
sage: G = Graph(multiedges=True)
sage: G.add_edges([(0,1,'a'),(0,1,'b')])
sage: G.edges(sort=True)
[(0, 1, 'a'), (0, 1, 'b')]
sage: with removed_multiedge(G,(0,1)) as Y:
....:     G.edges(sort=True)
[]
sage: G.edges(sort=True)
[(0, 1, 'a'), (0, 1, 'b')]
```

```
>>> from sage.all import *
>>> from sage.graphs.tutte_polynomial import removed_multiedge
>>> G = Graph(multiedges=True)
>>> G.add_edges([(Integer(0),Integer(1),'a'),(Integer(0),Integer(1),'b')])
>>> G.edges(sort=True)
[(0, 1, 'a'), (0, 1, 'b')]
>>> with removed_multiedge(G,(Integer(0),Integer(1))) as Y:
...     G.edges(sort=True)
[]
>>> G.edges(sort=True)
[(0, 1, 'a'), (0, 1, 'b')]
```

sage.graphs.tutte_polynomial.**tutte_polynomial**(*G*, *edge_selector=None*, *cache=None*)

Return the Tutte polynomial of the graph G.

INPUT:

- `edge_selector` – method (optional); this argument allows the user to specify his own heuristic for selecting edges used in the deletion contraction recurrence
- `cache` – (optional) dictionary to cache the Tutte polynomials generated in the recursive process. One will be created automatically if not provided.

EXAMPLES:

The Tutte polynomial of any tree of order n is x^{n-1}:

```
sage: all(T.tutte_polynomial() == x**9 for T in graphs.trees(10))                #␣
↪needs sage.symbolic
True
```

```
>>> from sage.all import *
>>> all(T.tutte_polynomial() == x**Integer(9) for T in graphs.trees(Integer(10)))␣
↪              # needs sage.symbolic
True
```

The Tutte polynomial of the Petersen graph is:

```
sage: P = graphs.PetersenGraph()
sage: P.tutte_polynomial()
x^9 + 6*x^8 + 21*x^7 + 56*x^6 + 12*x^5*y + y^6 + 114*x^5 + 70*x^4*y
+ 30*x^3*y^2 + 15*x^2*y^3 + 10*x*y^4 + 9*y^5 + 170*x^4 + 170*x^3*y
+ 105*x^2*y^2 + 65*x*y^3 + 35*y^4 + 180*x^3 + 240*x^2*y + 171*x*y^2
+ 75*y^3 + 120*x^2 + 168*x*y + 84*y^2 + 36*x + 36*y
```

```
>>> from sage.all import *
>>> P = graphs.PetersenGraph()
>>> P.tutte_polynomial()
x^9 + 6*x^8 + 21*x^7 + 56*x^6 + 12*x^5*y + y^6 + 114*x^5 + 70*x^4*y
+ 30*x^3*y^2 + 15*x^2*y^3 + 10*x*y^4 + 9*y^5 + 170*x^4 + 170*x^3*y
+ 105*x^2*y^2 + 65*x*y^3 + 35*y^4 + 180*x^3 + 240*x^2*y + 171*x*y^2
+ 75*y^3 + 120*x^2 + 168*x*y + 84*y^2 + 36*x + 36*y
```

The Tutte polynomial of a connected graph G evaluated at (1,1) is the number of spanning trees of G:

```
sage: G = graphs.RandomGNP(10,0.6)
sage: while not G.is_connected():
....:     G = graphs.RandomGNP(10,0.6)
sage: G.tutte_polynomial()(1,1) == G.number_of_spanning_trees()                  #␣
↪needs sage.modules
True
```

```
>>> from sage.all import *
>>> G = graphs.RandomGNP(Integer(10),RealNumber('0.6'))
>>> while not G.is_connected():
...     G = graphs.RandomGNP(Integer(10),RealNumber('0.6'))
>>> G.tutte_polynomial()(Integer(1),Integer(1)) == G.number_of_spanning_trees()  ␣
```

(continues on next page)

(continued from previous page)

```
↪              # needs sage.modules
True
```

Given that $T(x, y)$ is the Tutte polynomial of a graph G with n vertices and c connected components, then $(-1)^{n-c}x^kT(1-x, 0)$ is the chromatic polynomial of G.

```
sage: G = graphs.OctahedralGraph()
sage: T = G.tutte_polynomial()
sage: R = PolynomialRing(ZZ, 'x')
sage: R((-1)^5*x*T(1-x,0)).factor()                                          #␣
↪needs sage.symbolic
(x - 2) * (x - 1) * x * (x^3 - 9*x^2 + 29*x - 32)
sage: G.chromatic_polynomial().factor()                                      #␣
↪needs sage.libs.flint
(x - 2) * (x - 1) * x * (x^3 - 9*x^2 + 29*x - 32)
```

```
>>> from sage.all import *
>>> G = graphs.OctahedralGraph()
>>> T = G.tutte_polynomial()
>>> R = PolynomialRing(ZZ, 'x')
>>> R((-Integer(1))**Integer(5)*x*T(Integer(1)-x,Integer(0))).factor()          ␣
↪                                  # needs sage.symbolic
(x - 2) * (x - 1) * x * (x^3 - 9*x^2 + 29*x - 32)
>>> G.chromatic_polynomial().factor()                                        #␣
↪needs sage.libs.flint
(x - 2) * (x - 1) * x * (x^3 - 9*x^2 + 29*x - 32)
```

sage.graphs.tutte_polynomial.**underlying_graph**(G)

Given a graph G with multi-edges, returns a graph where all the multi-edges are replaced with a single edge.

EXAMPLES:

```
sage: from sage.graphs.tutte_polynomial import underlying_graph
sage: G = Graph(multiedges=True)
sage: G.add_edges([(0,1,'a'),(0,1,'b')])
sage: G.edges(sort=True)
[(0, 1, 'a'), (0, 1, 'b')]
sage: underlying_graph(G).edges(sort=True)
[(0, 1, None)]
```

```
>>> from sage.all import *
>>> from sage.graphs.tutte_polynomial import underlying_graph
>>> G = Graph(multiedges=True)
>>> G.add_edges([(Integer(0),Integer(1),'a'),(Integer(0),Integer(1),'b')])
>>> G.edges(sort=True)
[(0, 1, 'a'), (0, 1, 'b')]
>>> underlying_graph(G).edges(sort=True)
[(0, 1, None)]
```

5.42 Partial cubes

The code in this module that recognizes partial cubes is originally from the PADS library by David Eppstein, which is available at http://www.ics.uci.edu/~eppstein/PADS/ under the MIT license. It has a quadratic runtime and has been described in [Epp2008].

For more information on partial cubes, see the Wikipedia article Partial cube.

5.42.1 Recognition algorithm

Definitions

A **partial cube** is an isometric subgraph G of a `CubeGraph()` (of possibly high dimension). Consequently, the vertices of G can be labelled with binary sequences in such a way that the distance between two vertices $u, v \in G$ is the Hamming distance between their labels.

Tokens and their **action**: in the terminology of [Epp2008], a token represents a transition of the form:

> *switch the `k`-th bit of the binary string from 0 to 1*

Each token can be matched with a 'reversed' token that performs the same switch in the opposite direction. Alternatively, a token can be seen as a set of disjoint (directed) edges of G, corresponding to the transitions. When a vertex $v \in G$ is the source of such an edge, it is said that the token *acts* on v.

Observations

Shortest paths: in a hypercube, a shortest path between two vertices uses each token at most once. Furthermore, it cannot use both a token and it reverse.

Cycles: a cycle in a partial cube is necessarily even, as hypercubes are bipartite. If an edge e of a cycle C belongs to a token T, then the edge opposite to e in C belongs to the reverse of T.

Incident edges: all $2d_G(v)$ arcs incident to a given vertex belong to as many different tokens.

Algorithm

Labeling: Iteratively, the algorithm selects a vertex $v \in G$, which is naturally associated to $2d(v)$ tokens. It then performs a breadth-first search from v, applying the previous observation on cycles to attribute a token to some of the edges it meets. None of the edges whose token remains undecided after this step can belong to one of those $2d(v)$ tokens, by virtue of the observation on shortest paths.

The labeled edges can then be simplified (contracted) if the previous step did not lead to a contradiction, and the procedure is applied again until the graph is contracted to a single vertex and all edges are labeled.

A partial cube is correctly labeled at this step, but some other graphs can also satisfy the procedure.

Checking the labeling: once all tokens are defined and the vertices are labeled with a binary string, we check that they define an isometric subgraph of the hypercube. To ensure that the distance $d(v_0, u)$ is what we expect for any vertex u, it is sufficient to find, for any vertex u, a neighbor n_u of u whose Hamming distance with v_0 is strictly less than the Hamming distance between u and v_0. Here is the algorithm used to check the labeling:

- For an initial vertex v, run a BFS starting from v, and associate to every other vertex u a token that brings u closer to v. This yields shortest paths from every vertex to v.
- Assuming that the information is computed (and correct) for v, it is easy to update it for a neighbor v' of v. Indeed, if we write T the token that turns v into v', only the vertices which were associated with the reverse of T need to select a new neighbour. All others can remain as they were previously.

  With this second observation, one can efficiently check that the distance between all pairs of vertices are what they should be. In the implementation, the sequence of the sources $(v, v', ...)$ is given by a depth-first search.

5.42.2 Functions

sage.graphs.partial_cube.**breadth_first_level_search**(*G*, *start*)

Generate a sequence of dictionaries, each mapping the vertices at distance `i` from `start` to the set of their neighbours at distance `i+1`.

Originally written by D. Eppstein for the PADS library (http://www.ics.uci.edu/~eppstein/PADS/).

INPUT:

- `G` – a graph to perform the search on
- `start` – vertex or list of vertices from which to start the traversal

EXAMPLES:

```
sage: H = digraphs.DeBruijn(3,2)                                                #␣
↪needs sage.combinat
sage: list(sage.graphs.partial_cube.breadth_first_level_search(H, '00'))        #␣
↪needs sage.combinat
[{'00': {'01', '02'}},
 {'01': {'10', '11', '12'}, '02': {'20', '21', '22'}},
 {'10': set(),
  '11': set(),
  '12': set(),
  '20': set(),
  '21': set(),
  '22': set()}]
```

```
>>> from sage.all import *
>>> H = digraphs.DeBruijn(Integer(3),Integer(2))                                ␣
↪              # needs sage.combinat
>>> list(sage.graphs.partial_cube.breadth_first_level_search(H, '00'))        #␣
↪needs sage.combinat
[{'00': {'01', '02'}},
 {'01': {'10', '11', '12'}, '02': {'20', '21', '22'}},
 {'10': set(),
  '11': set(),
  '12': set(),
  '20': set(),
  '21': set(),
  '22': set()}]
```

sage.graphs.partial_cube.**depth_first_traversal**(*G*, *start*)

Generate a sequence of triples (v,w,edgetype) for DFS of graph G.

Originally written by D. Eppstein for the PADS library (http://www.ics.uci.edu/~eppstein/PADS/).

INPUT:

- `G` – a graph to perform the search on
- `start` – vertex or list of vertices from which to start the traversal

OUTPUT:

- a generator of triples `(v,w,edgetype)`, where `edgetype` is `True` if the algorithm is progressing via the edge `vw`, or `False` if the algorithm is backtracking via the edge `wv`.

EXAMPLES:

```
sage: H = digraphs.DeBruijn(3,2)                                                #␣
↪needs sage.combinat
sage: t = list(sage.graphs.partial_cube.depth_first_traversal(H, '00'))         #␣
↪needs sage.combinat
sage: len(t)                                                                    #␣
↪needs sage.combinat
16
```

```
>>> from sage.all import *
>>> H = digraphs.DeBruijn(Integer(3),Integer(2))                                ␣
↪              # needs sage.combinat
>>> t = list(sage.graphs.partial_cube.depth_first_traversal(H, '00'))          #␣
↪needs sage.combinat
>>> len(t)                                                                      #␣
↪needs sage.combinat
16
```

sage.graphs.partial_cube.**is_partial_cube**(*G*, *certificate=False*)

Test whether the given graph is a partial cube.

A partial cube is a graph that can be isometrically embedded into a hypercube, i.e., its vertices can be labelled with (0,1)-vectors of some fixed length such that the distance between any two vertices in the graph equals the Hamming distance of their labels.

Originally written by D. Eppstein for the PADS library (http://www.ics.uci.edu/~eppstein/PADS/), see also [Epp2008]. The algorithm runs in $O(n^2)$ time, where n is the number of vertices. See the documentation of *partial_cube* for an overview of the algorithm.

INPUT:

- `certificate` – boolean (default: `False`); this function returns `True` or `False` according to the graph, when `certificate = False`. When `certificate = True` and the graph is a partial cube, the function returns `(True, mapping)`, where `mapping` is an isometric mapping of the vertices of the graph to the vertices of a hypercube ((0, 1)-strings of a fixed length). When `certificate = True` and the graph is not a partial cube, `(False, None)` is returned.

EXAMPLES:

The Petersen graph is not a partial cube:

```
sage: g = graphs.PetersenGraph()
sage: g.is_partial_cube()
False
```

```
>>> from sage.all import *
>>> g = graphs.PetersenGraph()
>>> g.is_partial_cube()
False
```

All prisms are partial cubes:

```
sage: g = graphs.CycleGraph(10).cartesian_product(graphs.CompleteGraph(2))
sage: g.is_partial_cube()
True
```

```
>>> from sage.all import *
>>> g = graphs.CycleGraph(Integer(10)).cartesian_product(graphs.
↪CompleteGraph(Integer(2)))
>>> g.is_partial_cube()
True
```

5.43 Path enumeration

This module is meant for all functions related to path enumeration in graphs.

all_paths()	Return the list of all paths between a pair of vertices.
yen_k_shortest_simple_paths()	Return an iterator over the simple paths between a pair of vertices in increasing order of weights.
nc_k_shortest_simple_paths()	Return an iterator over the simple paths between a pair of vertices in increasing order of weights.
feng_k_shortest_simple_paths()	Return an iterator over the simple paths between a pair of vertices in increasing order of weights.
pnc_k_shortest_simple_paths()	Return an iterator over the simple paths between a pair of vertices in increasing order of weights.
all_paths_iterator()	Return an iterator over the paths of `self`.
all_simple_paths()	Return a list of all the simple paths of `self` starting with one of the given vertices.
shortest_simple_paths()	Return an iterator over the simple paths between a pair of vertices.

5.43.1 Functions

sage.graphs.path_enumeration.**all_paths**(*G*, *start*, *end*, *use_multiedges=False*, *report_edges=False*, *labels=False*)

Return the list of all paths between a pair of vertices.

If `start` is the same vertex as `end`, then `[[start]]` is returned – a list containing the 1-vertex, 0-edge path "`start`".

If `G` has multiple edges, a path will be returned as many times as the product of the multiplicity of the edges along that path depending on the value of the flag `use_multiedges`.

INPUT:

- `start` – a vertex of a graph, where to start
- `end` – a vertex of a graph, where to end
- `use_multiedges` – boolean (default: `False`); this parameter is used only if the graph has multiple edges
  - If `False`, the graph is considered as simple and an edge label is arbitrarily selected for each edge as in *sage.graphs.generic_graph.GenericGraph.to_simple()* if `report_edges` is `True`
  - If `True`, a path will be reported as many times as the edges multiplicities along that path (when `report_edges = False` or `labels = False`), or with all possible combinations of edge labels (when `report_edges = True` and `labels = True`)
- `report_edges` – boolean (default: `False`); whether to report paths as list of vertices (default) or list of edges, if `False` then `labels` parameter is ignored
- `labels` – boolean (default: `False`); if `False`, each edge is simply a pair `(u, v)` of vertices. Otherwise a list of edges along with its edge labels are used to represent the path.

EXAMPLES:

```
sage: eg1 = Graph({0:[1, 2], 1:[4], 2:[3, 4], 4:[5], 5:[6]})
sage: eg1.all_paths(0, 6)
[[0, 1, 4, 5, 6], [0, 2, 4, 5, 6]]
sage: eg2 = graphs.PetersenGraph()
sage: sorted(eg2.all_paths(1, 4))
[[1, 0, 4],
 [1, 0, 5, 7, 2, 3, 4],
 [1, 0, 5, 7, 2, 3, 8, 6, 9, 4],
 [1, 0, 5, 7, 9, 4],
 [1, 0, 5, 7, 9, 6, 8, 3, 4],
 [1, 0, 5, 8, 3, 2, 7, 9, 4],
 [1, 0, 5, 8, 3, 4],
 [1, 0, 5, 8, 6, 9, 4],
 [1, 0, 5, 8, 6, 9, 7, 2, 3, 4],
 [1, 2, 3, 4],
 [1, 2, 3, 8, 5, 0, 4],
 [1, 2, 3, 8, 5, 7, 9, 4],
 [1, 2, 3, 8, 6, 9, 4],
 [1, 2, 3, 8, 6, 9, 7, 5, 0, 4],
 [1, 2, 7, 5, 0, 4],
 [1, 2, 7, 5, 8, 3, 4],
 [1, 2, 7, 5, 8, 6, 9, 4],
 [1, 2, 7, 9, 4],
 [1, 2, 7, 9, 6, 8, 3, 4],
 [1, 2, 7, 9, 6, 8, 5, 0, 4],
 [1, 6, 8, 3, 2, 7, 5, 0, 4],
 [1, 6, 8, 3, 2, 7, 9, 4],
 [1, 6, 8, 3, 4],
 [1, 6, 8, 5, 0, 4],
 [1, 6, 8, 5, 7, 2, 3, 4],
 [1, 6, 8, 5, 7, 9, 4],
 [1, 6, 9, 4],
 [1, 6, 9, 7, 2, 3, 4],
 [1, 6, 9, 7, 2, 3, 8, 5, 0, 4],
 [1, 6, 9, 7, 5, 0, 4],
 [1, 6, 9, 7, 5, 8, 3, 4]]
sage: dg = DiGraph({0:[1, 3], 1:[3], 2:[0, 3]})
sage: sorted(dg.all_paths(0, 3))
[[0, 1, 3], [0, 3]]
sage: ug = dg.to_undirected()
sage: sorted(ug.all_paths(0, 3))
[[0, 1, 3], [0, 2, 3], [0, 3]]

sage: g = Graph([(0, 1), (0, 1), (1, 2), (1, 2)], multiedges=True)
sage: g.all_paths(0, 2, use_multiedges=True)
[[0, 1, 2], [0, 1, 2], [0, 1, 2], [0, 1, 2]]

sage: dg = DiGraph({0:[1, 2, 1], 3:[0, 0]}, multiedges=True)
sage: dg.all_paths(3, 1, use_multiedges=True)
[[3, 0, 1], [3, 0, 1], [3, 0, 1], [3, 0, 1]]
```

(continues on next page)

(continued from previous page)

```
sage: g = Graph([(0, 1, 'a'), (0, 1, 'b'), (1, 2, 'c'), (1, 2, 'd')],␣
↪multiedges=True)
sage: g.all_paths(0, 2, use_multiedges=False)
[[0, 1, 2]]
sage: g.all_paths(0, 2, use_multiedges=True)
[[0, 1, 2], [0, 1, 2], [0, 1, 2], [0, 1, 2]]
sage: g.all_paths(0, 2, use_multiedges=True, report_edges=True)
[[(0, 1), (1, 2)], [(0, 1), (1, 2)], [(0, 1), (1, 2)], [(0, 1), (1, 2)]]
sage: g.all_paths(0, 2, use_multiedges=True, report_edges=True, labels=True)
[((0, 1, 'b'), (1, 2, 'd')),
 ((0, 1, 'b'), (1, 2, 'c')),
 ((0, 1, 'a'), (1, 2, 'd')),
 ((0, 1, 'a'), (1, 2, 'c'))]
sage: g.all_paths(0, 2, use_multiedges=False, report_edges=True, labels=True)
[((0, 1, 'b'), (1, 2, 'd'))]
sage: g.all_paths(0, 2, use_multiedges=False, report_edges=False, labels=True)
[[0, 1, 2]]
sage: g.all_paths(0, 2, use_multiedges=True, report_edges=False, labels=True)
[[0, 1, 2], [0, 1, 2], [0, 1, 2], [0, 1, 2]]
```

```
>>> from sage.all import *
>>> eg1 = Graph({Integer(0):[Integer(1), Integer(2)], Integer(1):[Integer(4)],␣
↪Integer(2):[Integer(3), Integer(4)], Integer(4):[Integer(5)],␣
↪Integer(5):[Integer(6)]})
>>> eg1.all_paths(Integer(0), Integer(6))
[[0, 1, 4, 5, 6], [0, 2, 4, 5, 6]]
>>> eg2 = graphs.PetersenGraph()
>>> sorted(eg2.all_paths(Integer(1), Integer(4)))
[[1, 0, 4],
 [1, 0, 5, 7, 2, 3, 4],
 [1, 0, 5, 7, 2, 3, 8, 6, 9, 4],
 [1, 0, 5, 7, 9, 4],
 [1, 0, 5, 7, 9, 6, 8, 3, 4],
 [1, 0, 5, 8, 3, 2, 7, 9, 4],
 [1, 0, 5, 8, 3, 4],
 [1, 0, 5, 8, 6, 9, 4],
 [1, 0, 5, 8, 6, 9, 7, 2, 3, 4],
 [1, 2, 3, 4],
 [1, 2, 3, 8, 5, 0, 4],
 [1, 2, 3, 8, 5, 7, 9, 4],
 [1, 2, 3, 8, 6, 9, 4],
 [1, 2, 3, 8, 6, 9, 7, 5, 0, 4],
 [1, 2, 7, 5, 0, 4],
 [1, 2, 7, 5, 8, 3, 4],
 [1, 2, 7, 5, 8, 6, 9, 4],
 [1, 2, 7, 9, 4],
 [1, 2, 7, 9, 6, 8, 3, 4],
 [1, 2, 7, 9, 6, 8, 5, 0, 4],
 [1, 6, 8, 3, 2, 7, 5, 0, 4],
 [1, 6, 8, 3, 2, 7, 9, 4],
 [1, 6, 8, 3, 4],
 [1, 6, 8, 5, 0, 4],
```

(continues on next page)

(continued from previous page)

```
 [1, 6, 8, 5, 7, 2, 3, 4],
 [1, 6, 8, 5, 7, 9, 4],
 [1, 6, 9, 4],
 [1, 6, 9, 7, 2, 3, 4],
 [1, 6, 9, 7, 2, 3, 8, 5, 0, 4],
 [1, 6, 9, 7, 5, 0, 4],
 [1, 6, 9, 7, 5, 8, 3, 4]]
>>> dg = DiGraph({Integer(0):[Integer(1), Integer(3)], Integer(1):[Integer(3)],␣
↪Integer(2):[Integer(0), Integer(3)]})
>>> sorted(dg.all_paths(Integer(0), Integer(3)))
[[0, 1, 3], [0, 3]]
>>> ug = dg.to_undirected()
>>> sorted(ug.all_paths(Integer(0), Integer(3)))
[[0, 1, 3], [0, 2, 3], [0, 3]]

>>> g = Graph([(Integer(0), Integer(1)), (Integer(0), Integer(1)), (Integer(1),␣
↪Integer(2)), (Integer(1), Integer(2))], multiedges=True)
>>> g.all_paths(Integer(0), Integer(2), use_multiedges=True)
[[0, 1, 2], [0, 1, 2], [0, 1, 2], [0, 1, 2]]

>>> dg = DiGraph({Integer(0):[Integer(1), Integer(2), Integer(1)],␣
↪Integer(3):[Integer(0), Integer(0)]}, multiedges=True)
>>> dg.all_paths(Integer(3), Integer(1), use_multiedges=True)
[[3, 0, 1], [3, 0, 1], [3, 0, 1], [3, 0, 1]]

>>> g = Graph([(Integer(0), Integer(1), 'a'), (Integer(0), Integer(1), 'b'),␣
↪(Integer(1), Integer(2), 'c'), (Integer(1), Integer(2), 'd')], multiedges=True)
>>> g.all_paths(Integer(0), Integer(2), use_multiedges=False)
[[0, 1, 2]]
>>> g.all_paths(Integer(0), Integer(2), use_multiedges=True)
[[0, 1, 2], [0, 1, 2], [0, 1, 2], [0, 1, 2]]
>>> g.all_paths(Integer(0), Integer(2), use_multiedges=True, report_edges=True)
[[(0, 1), (1, 2)], [(0, 1), (1, 2)], [(0, 1), (1, 2)], [(0, 1), (1, 2)]]
>>> g.all_paths(Integer(0), Integer(2), use_multiedges=True, report_edges=True,␣
↪labels=True)
[((0, 1, 'b'), (1, 2, 'd')),
 ((0, 1, 'b'), (1, 2, 'c')),
 ((0, 1, 'a'), (1, 2, 'd')),
 ((0, 1, 'a'), (1, 2, 'c'))]
>>> g.all_paths(Integer(0), Integer(2), use_multiedges=False, report_edges=True,␣
↪labels=True)
[((0, 1, 'b'), (1, 2, 'd'))]
>>> g.all_paths(Integer(0), Integer(2), use_multiedges=False, report_edges=False,␣
↪labels=True)
[[0, 1, 2]]
>>> g.all_paths(Integer(0), Integer(2), use_multiedges=True, report_edges=False,␣
↪labels=True)
[[0, 1, 2], [0, 1, 2], [0, 1, 2], [0, 1, 2]]
```

sage.graphs.path_enumeration.**all_paths_iterator**(*self*, *starting_vertices=None*, *ending_vertices=None*, *simple=False*, *max_length=None*, *trivial=False*, *use_multiedges=False*, *report_edges=False*, *labels=False*)

Return an iterator over the paths of `self`.

The paths are enumerated in increasing length order.

INPUT:

- `starting_vertices` – iterable (default: `None`); vertices from which the paths must start. If `None`, then all vertices of the graph can be starting points.
- `ending_vertices` – iterable (default: `None`); allowed ending vertices of the paths. If `None`, then all vertices are allowed.
- `simple` – boolean (default: `False`); if set to `True`, then only simple paths are considered. Simple paths are paths in which no two arcs share a head or share a tail, i.e. every vertex in the path is entered at most once and exited at most once.
- `max_length` – nonnegative integer (default: `None`); the maximum length of the enumerated paths. If set to `None`, then all lengths are allowed.
- `trivial` – boolean (default: `False`); if set to `True`, then the empty paths are also enumerated
- `use_multiedges` – boolean (default: `False`); this parameter is used only if the graph has multiple edges
  - If `False`, the graph is considered as simple and an edge label is arbitrarily selected for each edge as in *sage.graphs.generic_graph.GenericGraph.to_simple()* if `report_edges` is `True`
  - If `True`, a path will be reported as many times as the edges multiplicities along that path (when `report_edges = False` or `labels = False`), or with all possible combinations of edge labels (when `report_edges = True` and `labels = True`)
- `report_edges` – boolean (default: `False`); whether to report paths as list of vertices (default) or list of edges, if `False` then `labels` parameter is ignored
- `labels` – boolean (default: `False`); if `False`, each edge is simply a pair `(u, v)` of vertices. Otherwise a list of edges along with its edge labels are used to represent the path.

OUTPUT: iterator

AUTHOR:

Alexandre Blondin Masse

EXAMPLES:

```
sage: G = graphs.CompleteGraph(4)
sage: list(G.all_paths_iterator(starting_vertices=[1], ending_vertices=[3],␣
↪simple=True))
[[1, 3], [1, 0, 3], [1, 2, 3], [1, 0, 2, 3], [1, 2, 0, 3]]
sage: list(G.shortest_simple_paths(1, 3))
[[1, 3], [1, 2, 3], [1, 0, 3], [1, 0, 2, 3], [1, 2, 0, 3]]
sage: pi = G.all_paths_iterator(starting_vertices=[1], ending_vertices=[3])
sage: for _ in range(6):
....:     print(next(pi))
[1, 3]
[1, 0, 3]
[1, 2, 3]
[1, 0, 1, 3]
[1, 0, 2, 3]
[1, 2, 0, 3]
```

(continues on next page)

(continued from previous page)

```
sage: g = DiGraph({'a': ['a', 'b'], 'b': ['c'], 'c': ['d'], 'd': ['c']},␣
↪loops=True)
sage: pi = g.all_paths_iterator(starting_vertices=['a'], ending_vertices=['d'],␣
↪report_edges=True, simple=True)
sage: list(pi)
[[('a', 'b'), ('b', 'c'), ('c', 'd')]]

sage: g = DiGraph([(0, 1, 'a'), (0, 1, 'b'), (1, 2,'c'), (1, 2,'d')],␣
↪multiedges=True)
sage: pi =  g.all_paths_iterator(starting_vertices=[0], use_multiedges=True)
sage: for _ in range(6):
....:     print(next(pi))
[0, 1]
[0, 1]
[0, 1, 2]
[0, 1, 2]
[0, 1, 2]
[0, 1, 2]
sage: pi =  g.all_paths_iterator(starting_vertices=[0], use_multiedges=True,␣
↪report_edges=True, labels=True)
sage: for _ in range(6):
....:     print(next(pi))
[(0, 1, 'b')]
[(0, 1, 'a')]
[(0, 1, 'b'), (1, 2, 'd')]
[(0, 1, 'b'), (1, 2, 'c')]
[(0, 1, 'a'), (1, 2, 'd')]
[(0, 1, 'a'), (1, 2, 'c')]
sage: list(g.all_paths_iterator(starting_vertices=[0, 1], ending_vertices=[2],␣
↪use_multiedges=False, report_edges=True, labels=True, simple=True))
[[(1, 2, 'd')], [(0, 1, 'b'), (1, 2, 'd')]]
sage: list(g.all_paths_iterator(starting_vertices=[0, 1], ending_vertices=[2],␣
↪use_multiedges=False, report_edges=False, labels=True))
[[1, 2], [0, 1, 2]]
sage: list(g.all_paths_iterator(use_multiedges=True, report_edges=False,␣
↪labels=True, max_length=1))
[[1, 2], [1, 2], [0, 1], [0, 1]]
sage: list(g.all_paths_iterator(use_multiedges=True, report_edges=True,␣
↪labels=True, max_length=1))
[[(1, 2, 'd')], [(1, 2, 'c')], [(0, 1, 'b')], [(0, 1, 'a')]]

sage: g = DiGraph({'a': ['a', 'b'], 'b': ['c'], 'c': ['d'], 'd': ['c']},␣
↪loops=True)
sage: pi = g.all_paths_iterator()
sage: [len(next(pi)) - 1 for _ in range(7)]
[1, 1, 1, 1, 1, 2, 2]
```

```
>>> from sage.all import *
>>> G = graphs.CompleteGraph(Integer(4))
>>> list(G.all_paths_iterator(starting_vertices=[Integer(1)], ending_
↪vertices=[Integer(3)], simple=True))
[[1, 3], [1, 0, 3], [1, 2, 3], [1, 0, 2, 3], [1, 2, 0, 3]]
```

(continues on next page)

(continued from previous page)

```
>>> list(G.shortest_simple_paths(Integer(1), Integer(3)))
[[1, 3], [1, 2, 3], [1, 0, 3], [1, 0, 2, 3], [1, 2, 0, 3]]
>>> pi = G.all_paths_iterator(starting_vertices=[Integer(1)], ending_
↪vertices=[Integer(3)])
>>> for _ in range(Integer(6)):
...     print(next(pi))
[1, 3]
[1, 0, 3]
[1, 2, 3]
[1, 0, 1, 3]
[1, 0, 2, 3]
[1, 2, 0, 3]

>>> g = DiGraph({'a': ['a', 'b'], 'b': ['c'], 'c': ['d'], 'd': ['c']}, loops=True)
>>> pi = g.all_paths_iterator(starting_vertices=['a'], ending_vertices=['d'],␣
↪report_edges=True, simple=True)
>>> list(pi)
[[('a', 'b'), ('b', 'c'), ('c', 'd')]]

>>> g = DiGraph([(Integer(0), Integer(1), 'a'), (Integer(0), Integer(1), 'b'),␣
↪(Integer(1), Integer(2),'c'), (Integer(1), Integer(2),'d')], multiedges=True)
>>> pi =  g.all_paths_iterator(starting_vertices=[Integer(0)], use_
↪multiedges=True)
>>> for _ in range(Integer(6)):
...     print(next(pi))
[0, 1]
[0, 1]
[0, 1, 2]
[0, 1, 2]
[0, 1, 2]
[0, 1, 2]
>>> pi =  g.all_paths_iterator(starting_vertices=[Integer(0)], use_
↪multiedges=True, report_edges=True, labels=True)
>>> for _ in range(Integer(6)):
...     print(next(pi))
[(0, 1, 'b')]
[(0, 1, 'a')]
[(0, 1, 'b'), (1, 2, 'd')]
[(0, 1, 'b'), (1, 2, 'c')]
[(0, 1, 'a'), (1, 2, 'd')]
[(0, 1, 'a'), (1, 2, 'c')]
>>> list(g.all_paths_iterator(starting_vertices=[Integer(0), Integer(1)], ending_
↪vertices=[Integer(2)], use_multiedges=False, report_edges=True, labels=True,␣
↪simple=True))
[[(1, 2, 'd')], [(0, 1, 'b'), (1, 2, 'd')]]
>>> list(g.all_paths_iterator(starting_vertices=[Integer(0), Integer(1)], ending_
↪vertices=[Integer(2)], use_multiedges=False, report_edges=False, labels=True))
[[1, 2], [0, 1, 2]]
>>> list(g.all_paths_iterator(use_multiedges=True, report_edges=False,␣
↪labels=True, max_length=Integer(1)))
[[1, 2], [1, 2], [0, 1], [0, 1]]
>>> list(g.all_paths_iterator(use_multiedges=True, report_edges=True, labels=True,
```

(continues on next page)

(continued from previous page)

```
↪ max_length=Integer(1)))
[[(1, 2, 'd')], [(1, 2, 'c')], [(0, 1, 'b')], [(0, 1, 'a')]]

>>> g = DiGraph({'a': ['a', 'b'], 'b': ['c'], 'c': ['d'], 'd': ['c']}, loops=True)
>>> pi = g.all_paths_iterator()
>>> [len(next(pi)) - Integer(1) for _ in range(Integer(7))]
[1, 1, 1, 1, 1, 2, 2]
```

It is possible to precise the allowed starting and/or ending vertices:

```
sage: pi = g.all_paths_iterator(starting_vertices=['a'])
sage: [len(next(pi)) - 1 for _ in range(5)]
[1, 1, 2, 2, 2]
sage: pi = g.all_paths_iterator(starting_vertices=['a'], ending_vertices=['b'])
sage: for _ in range(5):
....:     print(next(pi))
['a', 'b']
['a', 'a', 'b']
['a', 'a', 'a', 'b']
['a', 'a', 'a', 'a', 'b']
['a', 'a', 'a', 'a', 'a', 'b']
```

```
>>> from sage.all import *
>>> pi = g.all_paths_iterator(starting_vertices=['a'])
>>> [len(next(pi)) - Integer(1) for _ in range(Integer(5))]
[1, 1, 2, 2, 2]
>>> pi = g.all_paths_iterator(starting_vertices=['a'], ending_vertices=['b'])
>>> for _ in range(Integer(5)):
...     print(next(pi))
['a', 'b']
['a', 'a', 'b']
['a', 'a', 'a', 'b']
['a', 'a', 'a', 'a', 'b']
['a', 'a', 'a', 'a', 'a', 'b']
```

One may prefer to enumerate only simple paths (see *all_simple_paths()*):

```
sage: pi = g.all_paths_iterator(simple=True)
sage: sorted(list(pi), key=lambda x:(len(x), x))
[['a', 'a'], ['a', 'b'], ['b', 'c'], ['c', 'd'], ['d', 'c'],
    ['a', 'b', 'c'], ['b', 'c', 'd'], ['c', 'd', 'c'], ['d', 'c', 'd'],
    ['a', 'b', 'c', 'd']]
sage: pi = g.all_paths_iterator(simple=True)
sage: [len(p) - 1 for p in pi]
[1, 1, 1, 1, 1, 2, 2, 2, 2, 3]
```

```
>>> from sage.all import *
>>> pi = g.all_paths_iterator(simple=True)
>>> sorted(list(pi), key=lambda x:(len(x), x))
[['a', 'a'], ['a', 'b'], ['b', 'c'], ['c', 'd'], ['d', 'c'],
    ['a', 'b', 'c'], ['b', 'c', 'd'], ['c', 'd', 'c'], ['d', 'c', 'd'],
    ['a', 'b', 'c', 'd']]
```

(continues on next page)

(continued from previous page)

```
>>> pi = g.all_paths_iterator(simple=True)
>>> [len(p) - Integer(1) for p in pi]
[1, 1, 1, 1, 1, 2, 2, 2, 2, 3]
```

Or simply bound the length of the enumerated paths:

```
sage: pi = g.all_paths_iterator(starting_vertices=['a'], ending_vertices=['b', 'c
↪'], max_length=6)
sage: sorted(list(pi), key=lambda x:(len(x), x))
[['a', 'b'], ['a', 'a', 'b'], ['a', 'b', 'c'], ['a', 'a', 'a', 'b'],
    ['a', 'a', 'b', 'c'], ['a', 'a', 'a', 'a', 'b'],
    ['a', 'a', 'a', 'b', 'c'], ['a', 'b', 'c', 'd', 'c'],
    ['a', 'a', 'a', 'a', 'a', 'b'], ['a', 'a', 'a', 'a', 'b', 'c'],
    ['a', 'a', 'b', 'c', 'd', 'c'],
    ['a', 'a', 'a', 'a', 'a', 'a', 'b'],
    ['a', 'a', 'a', 'a', 'a', 'b', 'c'],
    ['a', 'a', 'a', 'b', 'c', 'd', 'c'],
    ['a', 'b', 'c', 'd', 'c', 'd', 'c']]
sage: pi = g.all_paths_iterator(starting_vertices=['a'], ending_vertices=['b', 'c
↪'], max_length=6)
sage: [len(p) - 1 for p in pi]
[1, 2, 2, 3, 3, 4, 4, 4, 5, 5, 5, 6, 6, 6, 6]
```

```
>>> from sage.all import *
>>> pi = g.all_paths_iterator(starting_vertices=['a'], ending_vertices=['b', 'c'],
↪ max_length=Integer(6))
>>> sorted(list(pi), key=lambda x:(len(x), x))
[['a', 'b'], ['a', 'a', 'b'], ['a', 'b', 'c'], ['a', 'a', 'a', 'b'],
    ['a', 'a', 'b', 'c'], ['a', 'a', 'a', 'a', 'b'],
    ['a', 'a', 'a', 'b', 'c'], ['a', 'b', 'c', 'd', 'c'],
    ['a', 'a', 'a', 'a', 'a', 'b'], ['a', 'a', 'a', 'a', 'b', 'c'],
    ['a', 'a', 'b', 'c', 'd', 'c'],
    ['a', 'a', 'a', 'a', 'a', 'a', 'b'],
    ['a', 'a', 'a', 'a', 'a', 'b', 'c'],
    ['a', 'a', 'a', 'b', 'c', 'd', 'c'],
    ['a', 'b', 'c', 'd', 'c', 'd', 'c']]
>>> pi = g.all_paths_iterator(starting_vertices=['a'], ending_vertices=['b', 'c'],
↪ max_length=Integer(6))
>>> [len(p) - Integer(1) for p in pi]
[1, 2, 2, 3, 3, 4, 4, 4, 5, 5, 5, 6, 6, 6, 6]
```

By default, empty paths are not enumerated, but it may be parametrized:

```
sage: pi = g.all_paths_iterator(simple=True, trivial=True)
sage: sorted(list(pi), key=lambda x:(len(x), x))
[['a'], ['b'], ['c'], ['d'], ['a', 'a'], ['a', 'b'], ['b', 'c'],
    ['c', 'd'], ['d', 'c'], ['a', 'b', 'c'], ['b', 'c', 'd'],
    ['c', 'd', 'c'], ['d', 'c', 'd'], ['a', 'b', 'c', 'd']]
sage: pi = g.all_paths_iterator(simple=True, trivial=True)
sage: [len(p) - 1 for p in pi]
[0, 0, 0, 0, 1, 1, 1, 1, 1, 2, 2, 2, 2, 3]
sage: pi = g.all_paths_iterator(simple=True, trivial=False)
```

(continues on next page)

(continued from previous page)

```
sage: sorted(list(pi), key=lambda x:(len(x), x))
[['a', 'a'], ['a', 'b'], ['b', 'c'], ['c', 'd'], ['d', 'c'],
    ['a', 'b', 'c'], ['b', 'c', 'd'], ['c', 'd', 'c'], ['d', 'c', 'd'],
    ['a', 'b', 'c', 'd']]
sage: pi = g.all_paths_iterator(simple=True, trivial=False)
sage: [len(p) - 1 for p in pi]
[1, 1, 1, 1, 1, 2, 2, 2, 2, 3]
```

```
>>> from sage.all import *
>>> pi = g.all_paths_iterator(simple=True, trivial=True)
>>> sorted(list(pi), key=lambda x:(len(x), x))
[['a'], ['b'], ['c'], ['d'], ['a', 'a'], ['a', 'b'], ['b', 'c'],
    ['c', 'd'], ['d', 'c'], ['a', 'b', 'c'], ['b', 'c', 'd'],
    ['c', 'd', 'c'], ['d', 'c', 'd'], ['a', 'b', 'c', 'd']]
>>> pi = g.all_paths_iterator(simple=True, trivial=True)
>>> [len(p) - Integer(1) for p in pi]
[0, 0, 0, 0, 1, 1, 1, 1, 1, 2, 2, 2, 2, 3]
>>> pi = g.all_paths_iterator(simple=True, trivial=False)
>>> sorted(list(pi), key=lambda x:(len(x), x))
[['a', 'a'], ['a', 'b'], ['b', 'c'], ['c', 'd'], ['d', 'c'],
    ['a', 'b', 'c'], ['b', 'c', 'd'], ['c', 'd', 'c'], ['d', 'c', 'd'],
    ['a', 'b', 'c', 'd']]
>>> pi = g.all_paths_iterator(simple=True, trivial=False)
>>> [len(p) - Integer(1) for p in pi]
[1, 1, 1, 1, 1, 2, 2, 2, 2, 3]
```

sage.graphs.path_enumeration.**all_simple_paths**(*self*, *starting_vertices=None*, *ending_vertices=None*, *max_length=None*, *trivial=False*, *use_multiedges=False*, *report_edges=False*, *labels=False*)

Return a list of all the simple paths of `self` starting with one of the given vertices.

Simple paths are paths in which no two arcs share a head or share a tail, i.e. every vertex in the path is entered at most once and exited at most once.

INPUT:

- `starting_vertices` – list (default: `None`); vertices from which the paths must start. If `None`, then all vertices of the graph can be starting points.
- `ending_vertices` – iterable (default: `None`); allowed ending vertices of the paths. If `None`, then all vertices are allowed.
- `max_length` – nonnegative integer (default: `None`); the maximum length of the enumerated paths. If set to `None`, then all lengths are allowed.
- `trivial` – boolean (default: `False`); if set to `True`, then the empty paths are also enumerated
- `use_multiedges` – boolean (default: `False`); this parameter is used only if the graph has multiple edges
  - If `False`, the graph is considered as simple and an edge label is arbitrarily selected for each edge as in *sage.graphs.generic_graph.GenericGraph.to_simple()* if `report_edges` is `True`
  - If `True`, a path will be reported as many times as the edges multiplicities along that path (when `report_edges = False` or `labels = False`), or with all possible combinations of edge labels (when `report_edges = True` and `labels = True`)

- `report_edges` – boolean (default: `False`); whether to report paths as list of vertices (default) or list of edges, if `False` then `labels` parameter is ignored
- `labels` – boolean (default: `False`); if `False`, each edge is simply a pair `(u, v)` of vertices. Otherwise a list of edges along with its edge labels are used to represent the path.

OUTPUT: list

Note

Although the number of simple paths of a finite graph is always finite, computing all its paths may take a very long time.

EXAMPLES:

```
sage: G = graphs.CompleteGraph(4)
sage: G.all_simple_paths([1], [3])
[[1, 3], [1, 0, 3], [1, 2, 3], [1, 0, 2, 3], [1, 2, 0, 3]]
sage: list(G.shortest_simple_paths(1, 3))
[[1, 3], [1, 2, 3], [1, 0, 3], [1, 0, 2, 3], [1, 2, 0, 3]]
sage: G.all_simple_paths([0, 1], [2, 3])
[[1, 2],
 [1, 3],
 [0, 2],
 [0, 3],
 [0, 1, 2],
 [0, 1, 3],
 [0, 2, 3],
 [0, 3, 2],
 [1, 0, 2],
 [1, 0, 3],
 [1, 2, 3],
 [1, 3, 2],
 [1, 0, 2, 3],
 [1, 0, 3, 2],
 [1, 2, 0, 3],
 [1, 3, 0, 2],
 [0, 1, 2, 3],
 [0, 1, 3, 2],
 [0, 2, 1, 3],
 [0, 3, 1, 2]]

sage: g = DiGraph({0: [0, 1], 1: [2], 2: [3], 3: [2]}, loops=True)
sage: g.all_simple_paths()
[[3, 2],
 [2, 3],
 [1, 2],
 [0, 0],
 [0, 1],
 [0, 1, 2],
 [1, 2, 3],
 [2, 3, 2],
 [3, 2, 3],
```

(continues on next page)

(continued from previous page)

```
 [0, 1, 2, 3]]

sage: g = DiGraph([(0, 1, 'a'), (0, 1, 'b'), (1, 2,'c'), (1, 2,'d')],␣
↪multiedges=True)
sage: g.all_simple_paths(starting_vertices=[0], ending_vertices=[2], use_
↪multiedges=False)
[[0, 1, 2]]
sage: g.all_simple_paths(starting_vertices=[0], ending_vertices=[2], use_
↪multiedges=True)
[[0, 1, 2], [0, 1, 2], [0, 1, 2], [0, 1, 2]]
sage: g.all_simple_paths(starting_vertices=[0], ending_vertices=[2], use_
↪multiedges=True, report_edges=True)
[[(0, 1), (1, 2)], [(0, 1), (1, 2)], [(0, 1), (1, 2)], [(0, 1), (1, 2)]]
sage: g.all_simple_paths(starting_vertices=[0], ending_vertices=[2], use_
↪multiedges=True, report_edges=True, labels=True)
[[(0, 1, 'b'), (1, 2, 'd')],
 [(0, 1, 'b'), (1, 2, 'c')],
 [(0, 1, 'a'), (1, 2, 'd')],
 [(0, 1, 'a'), (1, 2, 'c')]]
sage: g.all_simple_paths(starting_vertices=[0, 1], ending_vertices=[2], use_
↪multiedges=False, report_edges=True, labels=True)
[[(1, 2, 'd')], [(0, 1, 'b'), (1, 2, 'd')]]
sage: g.all_simple_paths(starting_vertices=[0, 1], ending_vertices=[2], use_
↪multiedges=False, report_edges=False, labels=True)
[[1, 2], [0, 1, 2]]
sage: g.all_simple_paths(use_multiedges=True, report_edges=False, labels=True)
[[1, 2], [1, 2], [0, 1], [0, 1], [0, 1, 2], [0, 1, 2], [0, 1, 2], [0, 1, 2]]
sage: g.all_simple_paths(starting_vertices=[0, 1], ending_vertices=[2], use_
↪multiedges=False, report_edges=True, labels=True, trivial=True)
[[(1, 2, 'd')], [(0, 1, 'b'), (1, 2, 'd')]]
```

```
>>> from sage.all import *
>>> G = graphs.CompleteGraph(Integer(4))
>>> G.all_simple_paths([Integer(1)], [Integer(3)])
[[1, 3], [1, 0, 3], [1, 2, 3], [1, 0, 2, 3], [1, 2, 0, 3]]
>>> list(G.shortest_simple_paths(Integer(1), Integer(3)))
[[1, 3], [1, 2, 3], [1, 0, 3], [1, 0, 2, 3], [1, 2, 0, 3]]
>>> G.all_simple_paths([Integer(0), Integer(1)], [Integer(2), Integer(3)])
[[1, 2],
 [1, 3],
 [0, 2],
 [0, 3],
 [0, 1, 2],
 [0, 1, 3],
 [0, 2, 3],
 [0, 3, 2],
 [1, 0, 2],
 [1, 0, 3],
 [1, 2, 3],
 [1, 3, 2],
 [1, 0, 2, 3],
 [1, 0, 3, 2],
```

(continues on next page)

(continued from previous page)

```
 [1, 2, 0, 3],
 [1, 3, 0, 2],
 [0, 1, 2, 3],
 [0, 1, 3, 2],
 [0, 2, 1, 3],
 [0, 3, 1, 2]]

>>> g = DiGraph({Integer(0): [Integer(0), Integer(1)], Integer(1): [Integer(2)],␣
↪Integer(2): [Integer(3)], Integer(3): [Integer(2)]}, loops=True)
>>> g.all_simple_paths()
[[3, 2],
 [2, 3],
 [1, 2],
 [0, 0],
 [0, 1],
 [0, 1, 2],
 [1, 2, 3],
 [2, 3, 2],
 [3, 2, 3],
 [0, 1, 2, 3]]

>>> g = DiGraph([(Integer(0), Integer(1), 'a'), (Integer(0), Integer(1), 'b'),␣
↪(Integer(1), Integer(2),'c'), (Integer(1), Integer(2),'d')], multiedges=True)
>>> g.all_simple_paths(starting_vertices=[Integer(0)], ending_
↪vertices=[Integer(2)], use_multiedges=False)
[[0, 1, 2]]
>>> g.all_simple_paths(starting_vertices=[Integer(0)], ending_
↪vertices=[Integer(2)], use_multiedges=True)
[[0, 1, 2], [0, 1, 2], [0, 1, 2], [0, 1, 2]]
>>> g.all_simple_paths(starting_vertices=[Integer(0)], ending_
↪vertices=[Integer(2)], use_multiedges=True, report_edges=True)
[[(0, 1), (1, 2)], [(0, 1), (1, 2)], [(0, 1), (1, 2)], [(0, 1), (1, 2)]]
>>> g.all_simple_paths(starting_vertices=[Integer(0)], ending_
↪vertices=[Integer(2)], use_multiedges=True, report_edges=True, labels=True)
[[(0, 1, 'b'), (1, 2, 'd')],
 [(0, 1, 'b'), (1, 2, 'c')],
 [(0, 1, 'a'), (1, 2, 'd')],
 [(0, 1, 'a'), (1, 2, 'c')]]
>>> g.all_simple_paths(starting_vertices=[Integer(0), Integer(1)], ending_
↪vertices=[Integer(2)], use_multiedges=False, report_edges=True, labels=True)
[[(1, 2, 'd')], [(0, 1, 'b'), (1, 2, 'd')]]
>>> g.all_simple_paths(starting_vertices=[Integer(0), Integer(1)], ending_
↪vertices=[Integer(2)], use_multiedges=False, report_edges=False, labels=True)
[[1, 2], [0, 1, 2]]
>>> g.all_simple_paths(use_multiedges=True, report_edges=False, labels=True)
[[1, 2], [1, 2], [0, 1], [0, 1], [0, 1, 2], [0, 1, 2], [0, 1, 2], [0, 1, 2]]
>>> g.all_simple_paths(starting_vertices=[Integer(0), Integer(1)], ending_
↪vertices=[Integer(2)], use_multiedges=False, report_edges=True, labels=True,␣
↪trivial=True)
[[(1, 2, 'd')], [(0, 1, 'b'), (1, 2, 'd')]]
```

One may compute all paths having specific starting and/or ending vertices:

```
sage: g = DiGraph({'a': ['a', 'b'], 'b': ['c'], 'c': ['d'], 'd': ['c']},␣
↪loops=True)
sage: g.all_simple_paths(starting_vertices=['a'])
[['a', 'a'], ['a', 'b'], ['a', 'b', 'c'], ['a', 'b', 'c', 'd']]
sage: g.all_simple_paths(starting_vertices=['a'], ending_vertices=['c'])
[['a', 'b', 'c']]
sage: g.all_simple_paths(starting_vertices=['a'], ending_vertices=['b', 'c'])
[['a', 'b'], ['a', 'b', 'c']]
```

```
>>> from sage.all import *
>>> g = DiGraph({'a': ['a', 'b'], 'b': ['c'], 'c': ['d'], 'd': ['c']}, loops=True)
>>> g.all_simple_paths(starting_vertices=['a'])
[['a', 'a'], ['a', 'b'], ['a', 'b', 'c'], ['a', 'b', 'c', 'd']]
>>> g.all_simple_paths(starting_vertices=['a'], ending_vertices=['c'])
[['a', 'b', 'c']]
>>> g.all_simple_paths(starting_vertices=['a'], ending_vertices=['b', 'c'])
[['a', 'b'], ['a', 'b', 'c']]
```

It is also possible to bound the length of the paths:

```
sage: g = DiGraph({0: [0, 1], 1: [2], 2: [3], 3: [2]}, loops=True)
sage: g.all_simple_paths(max_length=2)
[[3, 2],
 [2, 3],
 [1, 2],
 [0, 0],
 [0, 1],
 [0, 1, 2],
 [1, 2, 3],
 [2, 3, 2],
 [3, 2, 3]]
```

```
>>> from sage.all import *
>>> g = DiGraph({Integer(0): [Integer(0), Integer(1)], Integer(1): [Integer(2)],␣
↪Integer(2): [Integer(3)], Integer(3): [Integer(2)]}, loops=True)
>>> g.all_simple_paths(max_length=Integer(2))
[[3, 2],
 [2, 3],
 [1, 2],
 [0, 0],
 [0, 1],
 [0, 1, 2],
 [1, 2, 3],
 [2, 3, 2],
 [3, 2, 3]]
```

By default, empty paths are not enumerated, but this can be parametrized:

```
sage: g = DiGraph({'a': ['a', 'b'], 'b': ['c'], 'c': ['d'], 'd': ['c']},␣
↪loops=True)
sage: g.all_simple_paths(starting_vertices=['a'], trivial=True)
[['a'], ['a', 'a'], ['a', 'b'], ['a', 'b', 'c'],
 ['a', 'b', 'c', 'd']]
```

(continues on next page)

(continued from previous page)

```
sage: g.all_simple_paths(starting_vertices=['a'], trivial=False)
[['a', 'a'], ['a', 'b'], ['a', 'b', 'c'], ['a', 'b', 'c', 'd']]
```

```
>>> from sage.all import *
>>> g = DiGraph({'a': ['a', 'b'], 'b': ['c'], 'c': ['d'], 'd': ['c']}, loops=True)
>>> g.all_simple_paths(starting_vertices=['a'], trivial=True)
[['a'], ['a', 'a'], ['a', 'b'], ['a', 'b', 'c'],
 ['a', 'b', 'c', 'd']]
>>> g.all_simple_paths(starting_vertices=['a'], trivial=False)
[['a', 'a'], ['a', 'b'], ['a', 'b', 'c'], ['a', 'b', 'c', 'd']]
```

sage.graphs.path_enumeration.**feng_k_shortest_simple_paths**(*self*, *source*, *target*, *weight_function=None*, *by_weight=False*, *check_weight=True*, *report_edges=False*, *labels=False*, *report_weight=False*)

Return an iterator over the simple paths between a pair of vertices in increasing order of weights.

For unweighted graphs, paths are returned in order of increasing number of edges.

In case of weighted graphs, negative weights are not allowed.

If `source` is the same vertex as `target`, then `[[source]]` is returned – a list containing the 1-vertex, 0-edge path `source`.

The loops and the multiedges if present in the given graph are ignored and only minimum of the edge labels is kept in case of multiedges.

INPUT:

- `source` – a vertex of the graph, where to start
- `target` – a vertex of the graph, where to end
- `weight_function` – function (default: `None`); a function that takes as input an edge `(u, v, l)` and outputs its weight. If not `None`, `by_weight` is automatically set to `True`. If `None` and `by_weight` is `True`, we use the edge label `l` as a weight.
- `by_weight` – boolean (default: `False`); if `True`, the edges in the graph are weighted, otherwise all edges have weight 1
- `check_weight` – boolean (default: `True`); whether to check that the `weight_function` outputs a number for each edge
- `report_edges` – boolean (default: `False`); whether to report paths as list of vertices (default) or list of edges, if `False` then `labels` parameter is ignored
- `labels` – boolean (default: `False`); if `False`, each edge is simply a pair `(u, v)` of vertices. Otherwise a list of edges along with its edge labels are used to represent the path.
- `report_weight` – boolean (default: `False`); if `False`, just the path between `source` and `target` is returned. Otherwise a tuple of path length and path is returned.

ALGORITHM:

The same algorithm as *nc_k_shortest_simple_paths()*, when `postponed=False`.

EXAMPLES:

```
sage: from sage.graphs.path_enumeration import feng_k_shortest_simple_paths
sage: g = DiGraph([(1, 2, 20), (1, 3, 10), (1, 4, 30), (2, 5, 20), (3, 5, 10), (4,
↪ 5, 30)])
sage: list(feng_k_shortest_simple_paths(g, 1, 5, by_weight=True, report_
↪weight=True))
[(20.0, [1, 3, 5]), (40.0, [1, 2, 5]), (60.0, [1, 4, 5])]
sage: list(feng_k_shortest_simple_paths(g, 1, 5, report_weight=True))
[(2.0, [1, 2, 5]), (2.0, [1, 4, 5]), (2.0, [1, 3, 5])]
```

```
>>> from sage.all import *
>>> from sage.graphs.path_enumeration import feng_k_shortest_simple_paths
>>> g = DiGraph([(Integer(1), Integer(2), Integer(20)), (Integer(1), Integer(3),␣
↪Integer(10)), (Integer(1), Integer(4), Integer(30)), (Integer(2), Integer(5),␣
↪Integer(20)), (Integer(3), Integer(5), Integer(10)), (Integer(4), Integer(5),␣
↪Integer(30))])
>>> list(feng_k_shortest_simple_paths(g, Integer(1), Integer(5), by_weight=True,␣
↪report_weight=True))
[(20.0, [1, 3, 5]), (40.0, [1, 2, 5]), (60.0, [1, 4, 5])]
>>> list(feng_k_shortest_simple_paths(g, Integer(1), Integer(5), report_
↪weight=True))
[(2.0, [1, 2, 5]), (2.0, [1, 4, 5]), (2.0, [1, 3, 5])]
```

sage.graphs.path_enumeration.**nc_k_shortest_simple_paths**(*self*, *source*, *target*, *weight_function=None*, *by_weight=False*, *check_weight=True*, *report_edges=False*, *labels=False*, *report_weight=False*, *postponed=True*)

Return an iterator over the simple paths between a pair of vertices in increasing order of weights.

For unweighted graphs, paths are returned in order of increasing number of edges.

In case of weighted graphs, negative weights are not allowed.

If `source` is the same vertex as `target`, then `[[source]]` is returned – a list containing the 1-vertex, 0-edge path `source`.

The loops and the multiedges if present in the given graph are ignored and only minimum of the edge labels is kept in case of multiedges.

INPUT:

- `source` – a vertex of the graph, where to start
- `target` – a vertex of the graph, where to end
- `weight_function` – function (default: `None`); a function that takes as input an edge `(u, v, l)` and outputs its weight. If not `None`, `by_weight` is automatically set to `True`. If `None` and `by_weight` is `True`, we use the edge label `l` as a weight.
- `by_weight` – boolean (default: `False`); if `True`, the edges in the graph are weighted, otherwise all edges have weight 1
- `check_weight` – boolean (default: `True`); whether to check that the `weight_function` outputs a number for each edge
- `report_edges` – boolean (default: `False`); whether to report paths as list of vertices (default) or list of edges, if `False` then `labels` parameter is ignored

- `labels` – boolean (default: `False`); if `False`, each edge is simply a pair `(u, v)` of vertices. Otherwise a list of edges along with its edge labels are used to represent the path.
- `report_weight` – boolean (default: `False`); if `False`, just the path between `source` and `target` is returned. Otherwise a tuple of path length and path is returned.
- `postponed` – boolean (default: `True`); if `True`, the postponed node classification algorithm is used, otherwise the node classification algorithm is used. See below for details.

ALGORITHM:

- `postponed=False` This algorithm can be divided into two parts. Firstly, it determines the shortest path from `source` to `target`. Then, it determines all the other k-shortest paths. This algorithm finds the deviations of previous shortest paths to determine the next shortest paths. This algorithm finds the candidate paths more efficiently using a node classification technique. At first the candidate path is separated by its deviation node as prefix and suffix. Then the algorithm classify the nodes as red, yellow and green. A node on the prefix is assigned a red color, a node that can reach t (the destination node) through a shortest path without visiting a red node is assigned a green color, and all other nodes are assigned a yellow color. When searching for the suffix of a candidate path, all green nodes are bypassed, and `Dijkstra's algorithm` is applied to find an all-yellow-node subpath. Since on average the number of yellow nodes is much smaller than n, this algorithm has a much lower average-case running time.

  Time complexity is $O(kn(m + n \log n))$ where n is the number of vertices and m is the number of edges and k is the number of shortest paths needed to find. Its average running time is much smaller as compared to $Yen's$ algorithm.

  See [Feng2014] for more details on this algorithm.

- `postponed=True` This algorithm is based on the the above algorithm in [Feng2014], but postpones the shortest path tree computation when non-simple deviations occur. See Postponed Node Classification algorithm in [ACN2023] for the algorithm description. When not all simple paths are needed, this algorithm is more efficient than the algorithm for `postponed=False`.

EXAMPLES:

```
sage: from sage.graphs.path_enumeration import nc_k_shortest_simple_paths
sage: g = DiGraph([(1, 2, 20), (1, 3, 10), (1, 4, 30), (2, 5, 20), (3, 5, 10), (4,
↪ 5, 30)])
sage: list(nc_k_shortest_simple_paths(g, 1, 5, by_weight=True, report_
↪weight=True))
[(20.0, [1, 3, 5]), (40.0, [1, 2, 5]), (60.0, [1, 4, 5])]
sage: list(nc_k_shortest_simple_paths(g, 1, 5, report_weight=True))
[(2.0, [1, 2, 5]), (2.0, [1, 4, 5]), (2.0, [1, 3, 5])]
sage: list(nc_k_shortest_simple_paths(g, 1, 5, by_weight=True, report_weight=True,
↪ postponed=True))
[(20.0, [1, 3, 5]), (40.0, [1, 2, 5]), (60.0, [1, 4, 5])]
sage: list(nc_k_shortest_simple_paths(g, 1, 5, report_weight=True,␣
↪postponed=True))
[(2.0, [1, 2, 5]), (2.0, [1, 4, 5]), (2.0, [1, 3, 5])]

sage: list(nc_k_shortest_simple_paths(g, 1, 1))
[[1]]
sage: list(nc_k_shortest_simple_paths(g, 1, 5, report_edges=True, labels=True))
[[(1, 2, 20), (2, 5, 20)], [(1, 4, 30), (4, 5, 30)], [(1, 3, 10), (3, 5, 10)]]
sage: list(nc_k_shortest_simple_paths(g, 1, 5, report_edges=True, labels=True, by_
↪weight=True))
[[(1, 3, 10), (3, 5, 10)], [(1, 2, 20), (2, 5, 20)], [(1, 4, 30), (4, 5, 30)]]
```

(continues on next page)

(continued from previous page)

```
sage: list(nc_k_shortest_simple_paths(g, 1, 5, report_edges=True, labels=True, by_
↪weight=True, report_weight=True))
[(20.0, [(1, 3, 10), (3, 5, 10)]),
 (40.0, [(1, 2, 20), (2, 5, 20)]),
 (60.0, [(1, 4, 30), (4, 5, 30)])]
```

```
>>> from sage.all import *
>>> from sage.graphs.path_enumeration import nc_k_shortest_simple_paths
>>> g = DiGraph([(Integer(1), Integer(2), Integer(20)), (Integer(1), Integer(3),␣
↪Integer(10)), (Integer(1), Integer(4), Integer(30)), (Integer(2), Integer(5),␣
↪Integer(20)), (Integer(3), Integer(5), Integer(10)), (Integer(4), Integer(5),␣
↪Integer(30))])
>>> list(nc_k_shortest_simple_paths(g, Integer(1), Integer(5), by_weight=True,␣
↪report_weight=True))
[(20.0, [1, 3, 5]), (40.0, [1, 2, 5]), (60.0, [1, 4, 5])]
>>> list(nc_k_shortest_simple_paths(g, Integer(1), Integer(5), report_
↪weight=True))
[(2.0, [1, 2, 5]), (2.0, [1, 4, 5]), (2.0, [1, 3, 5])]
>>> list(nc_k_shortest_simple_paths(g, Integer(1), Integer(5), by_weight=True,␣
↪report_weight=True, postponed=True))
[(20.0, [1, 3, 5]), (40.0, [1, 2, 5]), (60.0, [1, 4, 5])]
>>> list(nc_k_shortest_simple_paths(g, Integer(1), Integer(5), report_weight=True,
↪ postponed=True))
[(2.0, [1, 2, 5]), (2.0, [1, 4, 5]), (2.0, [1, 3, 5])]

>>> list(nc_k_shortest_simple_paths(g, Integer(1), Integer(1)))
[[1]]
>>> list(nc_k_shortest_simple_paths(g, Integer(1), Integer(5), report_edges=True,␣
↪labels=True))
[[(1, 2, 20), (2, 5, 20)], [(1, 4, 30), (4, 5, 30)], [(1, 3, 10), (3, 5, 10)]]
>>> list(nc_k_shortest_simple_paths(g, Integer(1), Integer(5), report_edges=True,␣
↪labels=True, by_weight=True))
[[(1, 3, 10), (3, 5, 10)], [(1, 2, 20), (2, 5, 20)], [(1, 4, 30), (4, 5, 30)]]
>>> list(nc_k_shortest_simple_paths(g, Integer(1), Integer(5), report_edges=True,␣
↪labels=True, by_weight=True, report_weight=True))
[(20.0, [(1, 3, 10), (3, 5, 10)]),
 (40.0, [(1, 2, 20), (2, 5, 20)]),
 (60.0, [(1, 4, 30), (4, 5, 30)])]
```

Algorithm works for undirected graphs as well:

```
sage: g = Graph([(1, 2, 20), (1, 3, 10), (1, 4, 30), (2, 5, 20), (3, 5, 10), (4,␣
↪5, 30)])
sage: list(nc_k_shortest_simple_paths(g, 5, 1, by_weight=True))
[[5, 3, 1], [5, 2, 1], [5, 4, 1]]
sage: [len(P) for P in nc_k_shortest_simple_paths(g, 5, 1)]
[3, 3, 3]
```

```
>>> from sage.all import *
>>> g = Graph([(Integer(1), Integer(2), Integer(20)), (Integer(1), Integer(3),␣
↪Integer(10)), (Integer(1), Integer(4), Integer(30)), (Integer(2), Integer(5),␣
↪Integer(20)), (Integer(3), Integer(5), Integer(10)), (Integer(4), Integer(5),␣
```

(continues on next page)

(continued from previous page)

```
↪Integer(30))])
>>> list(nc_k_shortest_simple_paths(g, Integer(5), Integer(1), by_weight=True))
[[5, 3, 1], [5, 2, 1], [5, 4, 1]]
>>> [len(P) for P in nc_k_shortest_simple_paths(g, Integer(5), Integer(1))]
[3, 3, 3]
```

sage.graphs.path_enumeration.**pnc_k_shortest_simple_paths**(*self*, *source*, *target*, *weight_function=None*, *by_weight=False*, *check_weight=True*, *report_edges=False*, *labels=False*, *report_weight=False*)

Return an iterator over the simple paths between a pair of vertices in increasing order of weights.

In case of weighted graphs, negative weights are not allowed.

If `source` is the same vertex as `target`, then `[[source]]` is returned – a list containing the 1-vertex, 0-edge path `source`.

The loops and the multiedges if present in the given graph are ignored and only minimum of the edge labels is kept in case of multiedges.

INPUT:

- `source` – a vertex of the graph, where to start
- `target` – a vertex of the graph, where to end
- `weight_function` – function (default: `None`); a function that takes as input an edge `(u, v, l)` and outputs its weight. If not `None`, `by_weight` is automatically set to `True`. If `None` and `by_weight` is `True`, we use the edge label `l` as a weight.
- `by_weight` – boolean (default: `False`); if `True`, the edges in the graph are weighted, otherwise all edges have weight 1
- `check_weight` – boolean (default: `True`); whether to check that the `weight_function` outputs a number for each edge
- `report_edges` – boolean (default: `False`); whether to report paths as list of vertices (default) or list of edges, if `False` then `labels` parameter is ignored
- `labels` – boolean (default: `False`); if `False`, each edge is simply a pair `(u, v)` of vertices. Otherwise a list of edges along with its edge labels are used to represent the path.
- `report_weight` – boolean (default: `False`); if `False`, just a path is returned. Otherwise a tuple of path length and path is returned.

ALGORITHM:

The same algorithm as *nc_k_shortest_simple_paths()*, when `postponed=True`.

EXAMPLES:

```
sage: from sage.graphs.path_enumeration import pnc_k_shortest_simple_paths
sage: g = DiGraph([(1, 2, 20), (1, 3, 10), (1, 4, 30), (2, 5, 20), (3, 5, 10), (4,
↪ 5, 30)])
sage: list(pnc_k_shortest_simple_paths(g, 1, 5, by_weight=True, report_
↪weight=True))
[(20.0, [1, 3, 5]), (40.0, [1, 2, 5]), (60.0, [1, 4, 5])]
sage: list(pnc_k_shortest_simple_paths(g, 1, 5, report_weight=True))
[(2.0, [1, 2, 5]), (2.0, [1, 4, 5]), (2.0, [1, 3, 5])]
```

```
>>> from sage.all import *
>>> from sage.graphs.path_enumeration import pnc_k_shortest_simple_paths
>>> g = DiGraph([(Integer(1), Integer(2), Integer(20)), (Integer(1), Integer(3),␣
↪Integer(10)), (Integer(1), Integer(4), Integer(30)), (Integer(2), Integer(5),␣
↪Integer(20)), (Integer(3), Integer(5), Integer(10)), (Integer(4), Integer(5),␣
↪Integer(30))])
>>> list(pnc_k_shortest_simple_paths(g, Integer(1), Integer(5), by_weight=True,␣
↪report_weight=True))
[(20.0, [1, 3, 5]), (40.0, [1, 2, 5]), (60.0, [1, 4, 5])]
>>> list(pnc_k_shortest_simple_paths(g, Integer(1), Integer(5), report_
↪weight=True))
[(2.0, [1, 2, 5]), (2.0, [1, 4, 5]), (2.0, [1, 3, 5])]
```

sage.graphs.path_enumeration.**shortest_simple_paths**(*self*, *source*, *target*, *weight_function=None*, *by_weight=False*, *check_weight=True*, *algorithm=None*, *report_edges=False*, *labels=False*, *report_weight=False*)

Return an iterator over the simple paths between a pair of vertices.

This method returns an iterator over the simple paths (i.e., without repetition) from `source` to `target`. By default (`by_weight` is `False`), the paths are reported by increasing number of edges. When `by_weight` is `True`, the paths are reported by increasing weights.

In case of weighted graphs negative weights are not allowed.

If `source` is the same vertex as `target`, then `[[source]]` is returned – a list containing the 1-vertex, 0-edge path `source`.

By default Postponed Node Classification (`algorithm=PNC`) algorithm [ACN2023] is used.

The loops and the multiedges if present in the given graph are ignored and only minimum of the edge labels is kept in case of multiedges.

INPUT:

- `source` – a vertex of the graph, where to start
- `target` – a vertex of the graph, where to end
- `weight_function` – function (default: `None`); a function that takes as input an edge `(u, v, l)` and outputs its weight. If not `None`, `by_weight` is automatically set to `True`. If `None` and `by_weight` is `True`, we use the edge label `l` as a weight.
- `by_weight` – boolean (default: `False`); if `True`, the edges in the graph are weighted, otherwise all edges have weight 1
- `check_weight` – boolean (default: `True`); whether to check that the `weight_function` outputs a number for each edge
- `algorithm` – string (default: `None`); the algorithm to use in computing `k` shortest paths of `self`. The following algorithms are supported:
  - `'Yen'` – Yen's algorithm [Yen1970] (*yen_k_shortest_simple_paths()*)
  - `'Feng'` – an improved version of Yen's algorithm [Feng2014] (*feng_k_shortest_simple_paths()*)
  - `'PNC'` – an improved version of Feng's algorithm [ACN2023] (*pnc_k_shortest_simple_paths()*)
- `report_edges` – boolean (default: `False`); whether to report paths as list of vertices (default) or list of edges. When set to `False`, the `labels` parameter is ignored.

- `labels` – boolean (default: `False`); if `False`, each edge is simply a pair `(u, v)` of vertices. Otherwise a list of edges along with its edge labels are used to represent the path.
- `report_weight` – boolean (default: `False`); if `False`, just the path between `source` and `target` is returned. Otherwise a tuple of path length and path is returned.

EXAMPLES:

```
sage: g = DiGraph([(1, 2, 20), (1, 3, 10), (1, 4, 30),
....:              (2, 5, 20), (3, 5, 10), (4, 5, 30)])
sage: list(g.shortest_simple_paths(1, 5, by_weight=True, algorithm='Yen'))
[[1, 3, 5], [1, 2, 5], [1, 4, 5]]
sage: list(g.shortest_simple_paths(1, 5, algorithm='Yen'))
[[1, 2, 5], [1, 3, 5], [1, 4, 5]]
sage: list(g.shortest_simple_paths(1, 1))
[[1]]
sage: list(g.shortest_simple_paths(1, 5, by_weight=True,
....:                              report_edges=True, report_weight=True,␣
↪labels=True))
[(20.0, [(1, 3, 10), (3, 5, 10)]),
 (40.0, [(1, 2, 20), (2, 5, 20)]),
 (60.0, [(1, 4, 30), (4, 5, 30)])]
sage: list(g.shortest_simple_paths(1, 5, by_weight=True, algorithm='Feng',
....:                              report_edges=True, report_weight=True))
[(20.0, [(1, 3), (3, 5)]), (40.0, [(1, 2), (2, 5)]), (60.0, [(1, 4), (4, 5)])]
sage: list(g.shortest_simple_paths(1, 5, report_edges=True, report_weight=True))
[(2.0, [(1, 2), (2, 5)]), (2.0, [(1, 4), (4, 5)]), (2.0, [(1, 3), (3, 5)])]
sage: list(g.shortest_simple_paths(1, 5, by_weight=True, report_edges=True))
[[(1, 3), (3, 5)], [(1, 2), (2, 5)], [(1, 4), (4, 5)]]
sage: list(g.shortest_simple_paths(1, 5, by_weight=True, algorithm='Feng',
....:                              report_edges=True, labels=True))
[[(1, 3, 10), (3, 5, 10)], [(1, 2, 20), (2, 5, 20)], [(1, 4, 30), (4, 5, 30)]]
sage: g = Graph([(1, 2, 20), (1, 3, 10), (1, 4, 30), (2, 5, 20),
....:            (3, 5, 10), (4, 5, 30), (1, 6, 100), (5, 6, 5)])
sage: list(g.shortest_simple_paths(1, 6, by_weight = True))
[[1, 3, 5, 6], [1, 2, 5, 6], [1, 4, 5, 6], [1, 6]]
sage: list(g.shortest_simple_paths(1, 6, algorithm='Yen'))
[[1, 6], [1, 2, 5, 6], [1, 3, 5, 6], [1, 4, 5, 6]]
sage: list(g.shortest_simple_paths(1, 6,
....:                              report_edges=True, report_weight=True,␣
↪labels=True))
[(1.0, [(1, 6, 100)]),
 (3.0, [(1, 4, 30), (4, 5, 30), (5, 6, 5)]),
 (3.0, [(1, 3, 10), (3, 5, 10), (5, 6, 5)]),
 (3.0, [(1, 2, 20), (2, 5, 20), (5, 6, 5)])]
sage: list(g.shortest_simple_paths(1, 6, by_weight=True,
....:                              report_edges=True, report_weight=True,␣
↪labels=True))
[(25.0, [(1, 3, 10), (3, 5, 10), (5, 6, 5)]),
 (45.0, [(1, 2, 20), (2, 5, 20), (5, 6, 5)]),
 (65.0, [(1, 4, 30), (4, 5, 30), (5, 6, 5)]),
 (100.0, [(1, 6, 100)])]
sage: list(g.shortest_simple_paths(1, 6, by_weight=True,
....:                              report_edges=True, labels=True))
```

(continues on next page)

(continued from previous page)

```
[[(1, 3, 10), (3, 5, 10), (5, 6, 5)],
 [(1, 2, 20), (2, 5, 20), (5, 6, 5)],
 [(1, 4, 30), (4, 5, 30), (5, 6, 5)],
 [(1, 6, 100)]]
sage: list(g.shortest_simple_paths(1, 6, report_edges=True, labels=True))
[[(1, 6, 100)],
 [(1, 4, 30), (4, 5, 30), (5, 6, 5)],
 [(1, 3, 10), (3, 5, 10), (5, 6, 5)],
 [(1, 2, 20), (2, 5, 20), (5, 6, 5)]]
```

```
>>> from sage.all import *
>>> g = DiGraph([(Integer(1), Integer(2), Integer(20)), (Integer(1), Integer(3),␣
↪Integer(10)), (Integer(1), Integer(4), Integer(30)),
...              (Integer(2), Integer(5), Integer(20)), (Integer(3), Integer(5),␣
↪Integer(10)), (Integer(4), Integer(5), Integer(30))])
>>> list(g.shortest_simple_paths(Integer(1), Integer(5), by_weight=True,␣
↪algorithm='Yen'))
[[1, 3, 5], [1, 2, 5], [1, 4, 5]]
>>> list(g.shortest_simple_paths(Integer(1), Integer(5), algorithm='Yen'))
[[1, 2, 5], [1, 3, 5], [1, 4, 5]]
>>> list(g.shortest_simple_paths(Integer(1), Integer(1)))
[[1]]
>>> list(g.shortest_simple_paths(Integer(1), Integer(5), by_weight=True,
...                              report_edges=True, report_weight=True,␣
↪labels=True))
[(20.0, [(1, 3, 10), (3, 5, 10)]),
 (40.0, [(1, 2, 20), (2, 5, 20)]),
 (60.0, [(1, 4, 30), (4, 5, 30)])]
>>> list(g.shortest_simple_paths(Integer(1), Integer(5), by_weight=True,␣
↪algorithm='Feng',
...                              report_edges=True, report_weight=True))
[(20.0, [(1, 3), (3, 5)]), (40.0, [(1, 2), (2, 5)]), (60.0, [(1, 4), (4, 5)])]
>>> list(g.shortest_simple_paths(Integer(1), Integer(5), report_edges=True,␣
↪report_weight=True))
[(2.0, [(1, 2), (2, 5)]), (2.0, [(1, 4), (4, 5)]), (2.0, [(1, 3), (3, 5)])]
>>> list(g.shortest_simple_paths(Integer(1), Integer(5), by_weight=True, report_
↪edges=True))
[[(1, 3), (3, 5)], [(1, 2), (2, 5)], [(1, 4), (4, 5)]]
>>> list(g.shortest_simple_paths(Integer(1), Integer(5), by_weight=True,␣
↪algorithm='Feng',
...                              report_edges=True, labels=True))
[[(1, 3, 10), (3, 5, 10)], [(1, 2, 20), (2, 5, 20)], [(1, 4, 30), (4, 5, 30)]]
>>> g = Graph([(Integer(1), Integer(2), Integer(20)), (Integer(1), Integer(3),␣
↪Integer(10)), (Integer(1), Integer(4), Integer(30)), (Integer(2), Integer(5),␣
↪Integer(20)),
...            (Integer(3), Integer(5), Integer(10)), (Integer(4), Integer(5),␣
↪Integer(30)), (Integer(1), Integer(6), Integer(100)), (Integer(5), Integer(6),␣
↪Integer(5))])
>>> list(g.shortest_simple_paths(Integer(1), Integer(6), by_weight = True))
[[1, 3, 5, 6], [1, 2, 5, 6], [1, 4, 5, 6], [1, 6]]
>>> list(g.shortest_simple_paths(Integer(1), Integer(6), algorithm='Yen'))
[[1, 6], [1, 2, 5, 6], [1, 3, 5, 6], [1, 4, 5, 6]]
```

(continues on next page)

(continued from previous page)

```
>>> list(g.shortest_simple_paths(Integer(1), Integer(6),
...                              report_edges=True, report_weight=True,␣
↪labels=True))
[(1.0, [(1, 6, 100)]),
 (3.0, [(1, 4, 30), (4, 5, 30), (5, 6, 5)]),
 (3.0, [(1, 3, 10), (3, 5, 10), (5, 6, 5)]),
 (3.0, [(1, 2, 20), (2, 5, 20), (5, 6, 5)])]
>>> list(g.shortest_simple_paths(Integer(1), Integer(6), by_weight=True,
...                              report_edges=True, report_weight=True,␣
↪labels=True))
[(25.0, [(1, 3, 10), (3, 5, 10), (5, 6, 5)]),
 (45.0, [(1, 2, 20), (2, 5, 20), (5, 6, 5)]),
 (65.0, [(1, 4, 30), (4, 5, 30), (5, 6, 5)]),
 (100.0, [(1, 6, 100)])]
>>> list(g.shortest_simple_paths(Integer(1), Integer(6), by_weight=True,
...                              report_edges=True, labels=True))
[[(1, 3, 10), (3, 5, 10), (5, 6, 5)],
 [(1, 2, 20), (2, 5, 20), (5, 6, 5)],
 [(1, 4, 30), (4, 5, 30), (5, 6, 5)],
 [(1, 6, 100)]]
>>> list(g.shortest_simple_paths(Integer(1), Integer(6), report_edges=True,␣
↪labels=True))
[[(1, 6, 100)],
 [(1, 4, 30), (4, 5, 30), (5, 6, 5)],
 [(1, 3, 10), (3, 5, 10), (5, 6, 5)],
 [(1, 2, 20), (2, 5, 20), (5, 6, 5)]]
```

sage.graphs.path_enumeration.**yen_k_shortest_simple_paths**(*self*, *source*, *target*, *weight_function=None*, *by_weight=False*, *check_weight=True*, *report_edges=False*, *labels=False*, *report_weight=False*)

Return an iterator over the simple paths between a pair of vertices in increasing order of weights.

For unweighted graphs paths are returned in order of increasing number of edges.

In case of weighted graphs negative weights are not allowed.

If `source` is the same vertex as `target`, then `[[source]]` is returned – a list containing the 1-vertex, 0-edge path `source`.

The loops and the multiedges if present in the given graph are ignored and only minimum of the edge labels is kept in case of multiedges.

INPUT:

- `source` – a vertex of the graph, where to start
- `target` – a vertex of the graph, where to end
- `weight_function` – function (default: `None`); a function that takes as input an edge `(u, v, l)` and outputs its weight. If not `None`, `by_weight` is automatically set to `True`. If `None` and `by_weight` is `True`, we use the edge label `l` as a weight.
- `by_weight` – boolean (default: `False`); if `True`, the edges in the graph are weighted, otherwise all edges have weight 1

- `check_weight` – boolean (default: `True`); whether to check that the `weight_function` outputs a number for each edge
- `report_edges` – boolean (default: `False`); whether to report paths as list of vertices (default) or list of edges, if `False` then `labels` parameter is ignored
- `labels` – boolean (default: `False`); if `False`, each edge is simply a pair `(u, v)` of vertices. Otherwise a list of edges along with its edge labels are used to represent the path.
- `report_weight` – boolean (default: `False`); if `False`, just the path between `source` and `target` is returned. Otherwise a tuple of path length and path is returned.

ALGORITHM:

This algorithm can be divided into two parts. Firstly, it determines a shortest path from `source` to `target`. Then, it determines all the other k-shortest paths. This algorithm finds the deviations of previous shortest paths to determine the next shortest paths.

Time complexity is $O(kn(m + n \log n))$ where n is the number of vertices and m is the number of edges and k is the number of shortest paths needed to find.

See [Yen1970] and the Wikipedia article Yen%27s_algorithm for more details on the algorithm.

EXAMPLES:

```
sage: from sage.graphs.path_enumeration import yen_k_shortest_simple_paths
sage: g = DiGraph([(1, 2, 20), (1, 3, 10), (1, 4, 30), (2, 5, 20), (3, 5, 10), (4,
↪ 5, 30)])
sage: list(yen_k_shortest_simple_paths(g, 1, 5, by_weight=True))
[[1, 3, 5], [1, 2, 5], [1, 4, 5]]
sage: list(yen_k_shortest_simple_paths(g, 1, 5))
[[1, 2, 5], [1, 3, 5], [1, 4, 5]]
sage: list(yen_k_shortest_simple_paths(g, 1, 1))
[[1]]
sage: list(yen_k_shortest_simple_paths(g, 1, 5, by_weight=True, report_edges=True,
↪ report_weight=True, labels=True))
[(20, [(1, 3, 10), (3, 5, 10)]),
 (40, [(1, 2, 20), (2, 5, 20)]),
 (60, [(1, 4, 30), (4, 5, 30)])]
sage: list(yen_k_shortest_simple_paths(g, 1, 5, by_weight=True, report_edges=True,
↪ report_weight=True))
[(20, [(1, 3), (3, 5)]), (40, [(1, 2), (2, 5)]), (60, [(1, 4), (4, 5)])]
sage: list(yen_k_shortest_simple_paths(g, 1, 5, report_edges=True, report_
↪weight=True))
[(2, [(1, 2), (2, 5)]), (2, [(1, 3), (3, 5)]), (2, [(1, 4), (4, 5)])]
sage: list(yen_k_shortest_simple_paths(g, 1, 5, by_weight=True, report_
↪edges=True))
[[(1, 3), (3, 5)], [(1, 2), (2, 5)], [(1, 4), (4, 5)]]
sage: list(yen_k_shortest_simple_paths(g, 1, 5, by_weight=True, report_edges=True,
↪ labels=True))
[[(1, 3, 10), (3, 5, 10)], [(1, 2, 20), (2, 5, 20)], [(1, 4, 30), (4, 5, 30)]]
sage: from sage.graphs.path_enumeration import yen_k_shortest_simple_paths
sage: g = Graph([(1, 2, 20), (1, 3, 10), (1, 4, 30), (2, 5, 20), (3, 5, 10), (4,␣
↪5, 30), (1, 6, 100), (5, 6, 5)])
sage: list(yen_k_shortest_simple_paths(g, 1, 6, by_weight = True))
[[1, 3, 5, 6], [1, 2, 5, 6], [1, 4, 5, 6], [1, 6]]
sage: list(yen_k_shortest_simple_paths(g, 1, 6))
```

(continues on next page)

(continued from previous page)

```
[[1, 6], [1, 2, 5, 6], [1, 3, 5, 6], [1, 4, 5, 6]]
sage: list(yen_k_shortest_simple_paths(g, 1, 6, report_edges=True, report_
↪weight=True, labels=True))
[(1, [(1, 6, 100)]),
 (3, [(1, 2, 20), (2, 5, 20), (5, 6, 5)]),
 (3, [(1, 3, 10), (3, 5, 10), (5, 6, 5)]),
 (3, [(1, 4, 30), (4, 5, 30), (5, 6, 5)])]
sage: list(yen_k_shortest_simple_paths(g, 1, 6, report_edges=True, report_
↪weight=True, labels=True, by_weight=True))
[(25, [(1, 3, 10), (3, 5, 10), (5, 6, 5)]),
 (45, [(1, 2, 20), (2, 5, 20), (5, 6, 5)]),
 (65, [(1, 4, 30), (4, 5, 30), (5, 6, 5)]),
 (100, [(1, 6, 100)])]
sage: list(yen_k_shortest_simple_paths(g, 1, 6, report_edges=True, labels=True,␣
↪by_weight=True))
[[(1, 3, 10), (3, 5, 10), (5, 6, 5)],
 [(1, 2, 20), (2, 5, 20), (5, 6, 5)],
 [(1, 4, 30), (4, 5, 30), (5, 6, 5)],
 [(1, 6, 100)]]
sage: list(yen_k_shortest_simple_paths(g, 1, 6, report_edges=True, labels=True))
[[(1, 6, 100)],
 [(1, 2, 20), (2, 5, 20), (5, 6, 5)],
 [(1, 3, 10), (3, 5, 10), (5, 6, 5)],
 [(1, 4, 30), (4, 5, 30), (5, 6, 5)]]
```

```
>>> from sage.all import *
>>> from sage.graphs.path_enumeration import yen_k_shortest_simple_paths
>>> g = DiGraph([(Integer(1), Integer(2), Integer(20)), (Integer(1), Integer(3),␣
↪Integer(10)), (Integer(1), Integer(4), Integer(30)), (Integer(2), Integer(5),␣
↪Integer(20)), (Integer(3), Integer(5), Integer(10)), (Integer(4), Integer(5),␣
↪Integer(30))])
>>> list(yen_k_shortest_simple_paths(g, Integer(1), Integer(5), by_weight=True))
[[1, 3, 5], [1, 2, 5], [1, 4, 5]]
>>> list(yen_k_shortest_simple_paths(g, Integer(1), Integer(5)))
[[1, 2, 5], [1, 3, 5], [1, 4, 5]]
>>> list(yen_k_shortest_simple_paths(g, Integer(1), Integer(1)))
[[1]]
>>> list(yen_k_shortest_simple_paths(g, Integer(1), Integer(5), by_weight=True,␣
↪report_edges=True, report_weight=True, labels=True))
[(20, [(1, 3, 10), (3, 5, 10)]),
 (40, [(1, 2, 20), (2, 5, 20)]),
 (60, [(1, 4, 30), (4, 5, 30)])]
>>> list(yen_k_shortest_simple_paths(g, Integer(1), Integer(5), by_weight=True,␣
↪report_edges=True, report_weight=True))
[(20, [(1, 3), (3, 5)]), (40, [(1, 2), (2, 5)]), (60, [(1, 4), (4, 5)])]
>>> list(yen_k_shortest_simple_paths(g, Integer(1), Integer(5), report_edges=True,
↪ report_weight=True))
[(2, [(1, 2), (2, 5)]), (2, [(1, 3), (3, 5)]), (2, [(1, 4), (4, 5)])]
>>> list(yen_k_shortest_simple_paths(g, Integer(1), Integer(5), by_weight=True,␣
↪report_edges=True))
[[(1, 3), (3, 5)], [(1, 2), (2, 5)], [(1, 4), (4, 5)]]
>>> list(yen_k_shortest_simple_paths(g, Integer(1), Integer(5), by_weight=True,␣
```

(continues on next page)

(continued from previous page)

```
↪report_edges=True, labels=True))
[[(1, 3, 10), (3, 5, 10)], [(1, 2, 20), (2, 5, 20)], [(1, 4, 30), (4, 5, 30)]]
>>> from sage.graphs.path_enumeration import yen_k_shortest_simple_paths
>>> g = Graph([(Integer(1), Integer(2), Integer(20)), (Integer(1), Integer(3),␣
↪Integer(10)), (Integer(1), Integer(4), Integer(30)), (Integer(2), Integer(5),␣
↪Integer(20)), (Integer(3), Integer(5), Integer(10)), (Integer(4), Integer(5),␣
↪Integer(30)), (Integer(1), Integer(6), Integer(100)), (Integer(5), Integer(6),␣
↪Integer(5))])
>>> list(yen_k_shortest_simple_paths(g, Integer(1), Integer(6), by_weight = True))
[[1, 3, 5, 6], [1, 2, 5, 6], [1, 4, 5, 6], [1, 6]]
>>> list(yen_k_shortest_simple_paths(g, Integer(1), Integer(6)))
[[1, 6], [1, 2, 5, 6], [1, 3, 5, 6], [1, 4, 5, 6]]
>>> list(yen_k_shortest_simple_paths(g, Integer(1), Integer(6), report_edges=True,
↪ report_weight=True, labels=True))
[(1, [(1, 6, 100)]),
 (3, [(1, 2, 20), (2, 5, 20), (5, 6, 5)]),
 (3, [(1, 3, 10), (3, 5, 10), (5, 6, 5)]),
 (3, [(1, 4, 30), (4, 5, 30), (5, 6, 5)])]
>>> list(yen_k_shortest_simple_paths(g, Integer(1), Integer(6), report_edges=True,
↪ report_weight=True, labels=True, by_weight=True))
[(25, [(1, 3, 10), (3, 5, 10), (5, 6, 5)]),
 (45, [(1, 2, 20), (2, 5, 20), (5, 6, 5)]),
 (65, [(1, 4, 30), (4, 5, 30), (5, 6, 5)]),
 (100, [(1, 6, 100)])]
>>> list(yen_k_shortest_simple_paths(g, Integer(1), Integer(6), report_edges=True,
↪ labels=True, by_weight=True))
[[(1, 3, 10), (3, 5, 10), (5, 6, 5)],
 [(1, 2, 20), (2, 5, 20), (5, 6, 5)],
 [(1, 4, 30), (4, 5, 30), (5, 6, 5)],
 [(1, 6, 100)]]
>>> list(yen_k_shortest_simple_paths(g, Integer(1), Integer(6), report_edges=True,
↪ labels=True))
[[(1, 6, 100)],
 [(1, 2, 20), (2, 5, 20), (5, 6, 5)],
 [(1, 3, 10), (3, 5, 10), (5, 6, 5)],
 [(1, 4, 30), (4, 5, 30), (5, 6, 5)]]
```

5.44 GenericGraph Cython functions

AUTHORS:

- Robert L. Miller (2007-02-13): initial version
- Robert W. Bradshaw (2007-03-31): fast spring layout algorithms
- Nathann Cohen : exhaustive search

class sage.graphs.generic_graph_pyx.**GenericGraph_pyx**

Bases: SageObject

class sage.graphs.generic_graph_pyx.**SubgraphSearch**

Bases: object

This class implements methods to exhaustively search for copies of a graph H in a larger graph G.

It is possible to look for induced subgraphs instead, and to iterate or count the number of their occurrences.

ALGORITHM:

The algorithm is a brute-force search. Let $V(H) = \{h_1, \ldots, h_k\}$. It first tries to find in G a possible representative of h_1, then a representative of h_2 compatible with h_1, then a representative of h_3 compatible with the first two, etc.

This way, most of the time we need to test far less than $k!\binom{|V(G)|}{k}$ subsets, and hope this brute-force technique can sometimes be useful.

Note

This algorithm does not take vertex/edge labels into account.

cardinality()

Return the number of labelled subgraphs of G isomorphic to H.

Note

This method counts the subgraphs by enumerating them all ! Hence it probably is not a good idea to count their number before enumerating them :-)

EXAMPLES:

Counting the number of labelled P_3 in P_5:

```
sage: from sage.graphs.generic_graph_pyx import SubgraphSearch
sage: g = graphs.PathGraph(5)
sage: h = graphs.PathGraph(3)
sage: S = SubgraphSearch(g, h)                                              #␣
↪needs sage.modules
sage: S.cardinality()                                                       #␣
↪needs sage.modules
6
```

```
>>> from sage.all import *
>>> from sage.graphs.generic_graph_pyx import SubgraphSearch
>>> g = graphs.PathGraph(Integer(5))
>>> h = graphs.PathGraph(Integer(3))
>>> S = SubgraphSearch(g, h)                                                #␣
↪needs sage.modules
>>> S.cardinality()                                                         #␣
↪needs sage.modules
6
```

Check that the method is working even when vertices or edges are of incomparable types (see Issue #35904):

```
sage: from sage.graphs.generic_graph_pyx import SubgraphSearch
sage: G = Graph()
sage: G.add_cycle(['A', 1, 2, 3, ('a', 1)])
sage: H = Graph()
sage: H.add_path("xyz")
```

(continues on next page)

(continued from previous page)

```
sage: S = SubgraphSearch(G, H)                                                  #␣
↪needs sage.modules
sage: S.cardinality()                                                           #␣
↪needs sage.modules
10
```

```
>>> from sage.all import *
>>> from sage.graphs.generic_graph_pyx import SubgraphSearch
>>> G = Graph()
>>> G.add_cycle(['A', Integer(1), Integer(2), Integer(3), ('a', Integer(1))])
>>> H = Graph()
>>> H.add_path("xyz")
>>> S = SubgraphSearch(G, H)                                                    #␣
↪needs sage.modules
>>> S.cardinality()                                                             #␣
↪needs sage.modules
10
```

sage.graphs.generic_graph_pyx.**binary_string_from_dig6**(*s*, *n*)

A helper function for the dig6 format.

INPUT:

- `s` – a graph6 string
- `n` – the length of the binary string encoded by `s`

EXAMPLES:

```
sage: from sage.graphs.generic_graph_pyx import binary_string_from_dig6
sage: d6 = '?????_@?CG??B??@OG?C?G???GO??W@a???CO???OACC?OA?P@G??O?????'
sage: d6 += '?G??C????c?G?CC?_?@???C_??_?C????PO?C_??AA?OOAHCA___?CC?A?'
sage: d6 += 'CAOGO??????A??G?GR?C?_o`???g???A_C?OG??O?G_IA????_QO@EG???'
sage: d6 += 'O??C?_?C@?G???@?_??AC?AO?a???O?????A?_Dw?H???__O@AAOAACd?_'
sage: d6 += 'C??G?G@??GO?_???O@?_O??W??@P???AG??B?????G??GG???A??@?aC_G'
sage: d6 += '@A??O??_?A?????O@Z?_@M????GQ@_G@?C?'
sage: binary_string_from_dig6(d6, 63)
'000000000000000000000000000000100000000001000000000100001000000000000000000011000
↪000000000000010100000010000000000001000000000010000000000...100000000000000000
↪000000000000010000000001011011000000100000000001001110000000000000000000000000
↪010000100100000011000000010000000010000000010000000'
sage: d6 = '???C?@AA?_?A?O?C??S??O?q_?P?CHD??@?C?GC???C??GG?C_??O?COG??'
sage: d6 += '??I?J??Q??O?_@@??@??????'
sage: binary_string_from_dig6(d6, 32)
'000000000000000000001000000000000010000100000100000001000000000000000010000000010
↪0000...010000000000001000100000010000000000000000000000000000010100000000010110
↪000000000001001000000000000001000000000010000000001000001000000000000000001000
↪000000000000000000000000000000'
```

```
>>> from sage.all import *
>>> from sage.graphs.generic_graph_pyx import binary_string_from_dig6
>>> d6 = '?????_@?CG??B??@OG?C?G???GO??W@a???CO???OACC?OA?P@G??O?????'
>>> d6 += '?G??C????c?G?CC?_?@???C_??_?C????PO?C_??AA?OOAHCA___?CC?A?'
```

(continues on next page)

(continued from previous page)

```
>>> d6 += 'CAOGO??????A??G?GR?C?_o`???g???A_C?OG??O?G_IA????_QO@EG???'
>>> d6 += 'O??C?_?C@?G???@?_??AC?AO?a???O?????A?_Dw?H???__O@AAOAACd?_'
>>> d6 += 'C??G?G@??GO?_???O@?_O??W??@P???AG??B?????G??GG???A??@?aC_G'
>>> d6 += '@A??O??_?A?????O@Z?_@M????GQ@_G@?C?'
>>> binary_string_from_dig6(d6, Integer(63))
'000000000000000000000000000001000000000100000000010000100000000000000000011000
↪0000000000000101000000100000000000010000000001000000000...100000000000000000
↪0000000000000100000000010110110000001000000000100111000000000000000000000000
↪0100001001000000110000000100000000100000000100000000'
>>> d6 = '???C?@AA?_?A?O?C??S??O?q_?P?CHD??@?C?GC???C??GG?C_??O?COG??'
>>> d6 += '??I?J??Q??O?_@@??@??????'
>>> binary_string_from_dig6(d6, Integer(32))
'00000000000000000000010000000000001000010000010000000100000000000000010000000010
↪0000...0100000000000001000100000010000000000000000000000000000001010000000010110
↪0000000000001001000000000000001000000000010000000001000001000000000000000001000
↪0000000000000000000000000000000000'
```

sage.graphs.generic_graph_pyx.**binary_string_from_graph6**(*s*, *n*)

Decode a binary string from its graph6 representation.

This helper function is the inverse of R from [McK2015].

INPUT:

- `s` – a graph6 string
- `n` – the length of the binary string encoded by `s`

EXAMPLES:

```
sage: from sage.graphs.generic_graph_pyx import binary_string_from_graph6
sage: g6 = '?????_@?CG??B??@OG?C?G???GO??W@a???CO???OACC?OA?P@G??O?????'
sage: g6 += '?G??C????c?G?CC?_?@???C_??_?C????PO?C_??AA?OOAHCA___?CC?A?'
sage: g6 += 'CAOGO??????A??G?GR?C?_o`???g???A_C?OG??O?G_IA????_QO@EG???'
sage: g6 += 'O??C?_?C@?G???@?_??AC?AO?a???O?????A?_Dw?H???__O@AAOAACd?_'
sage: g6 += 'C??G?G@??GO?_???O@?_O??W??@P???AG??B?????G??GG???A??@?aC_G'
sage: g6 += '@A??O??_?A?????O@Z?_@M????GQ@_G@?C?'
sage: binary_string_from_graph6(g6, 63)
'000000000000000000000000000001000000000100000000010000100000000000000000011000
↪0000000000000101000000100000000000010000000001000000000...100000000000000000
↪0000000000000100000000010110110000001000000000100111000000000000000000000000
↪0100001001000000110000000100000000100000000100000000'
sage: g6 = '???C?@AA?_?A?O?C??S??O?q_?P?CHD??@?C?GC???C??GG?C_??O?COG??'
sage: g6 += '??I?J??Q??O?_@@??@??????'
sage: binary_string_from_graph6(g6, 32)
'00000000000000000000010000000000001000010000010000000100000000000000010000000010
↪0000...0100000000000001000100000010000000000000000000000000000001010000000010110
↪0000000000001001000000000000001000000000010000000001000001000000000000000001000
↪0000000000000000000000000000000000'
```

```
>>> from sage.all import *
>>> from sage.graphs.generic_graph_pyx import binary_string_from_graph6
>>> g6 = '?????_@?CG??B??@OG?C?G???GO??W@a???CO???OACC?OA?P@G??O?????'
>>> g6 += '?G??C????c?G?CC?_?@???C_??_?C????PO?C_??AA?OOAHCA___?CC?A?'
```

(continues on next page)

(continued from previous page)

```
>>> g6 += 'CAOGO??????A??G?GR?C?_o`???g???A_C?OG??O?G_IA????_QO@EG???'
>>> g6 += 'O??C?_?C@?G???@?_??AC?AO?a???O?????A?_Dw?H???__O@AAOAACd?_'
>>> g6 += 'C??G?G@??GO?_???O@?_O??W??@P???AG??B?????G??GG???A??@?aC_G'
>>> g6 += '@A??O??_?A?????O@Z?_@M????GQ@_G@?C?'
>>> binary_string_from_graph6(g6, Integer(63))
'000000000000000000000000000000010000000001000000001000010000000000000000011000
↪0000000000000101000000100000000000010000000001000000000...1000000000000000000
↪0000000000000100000000010110110000001000000000010011100000000000000000000000000
↪0100001001000000110000000100000000100000000100000000'
>>> g6 = '???C?@AA?_?A?O?C??S??O?q_?P?CHD??@?C?GC???C??GG?C_??O?COG??'
>>> g6 += '??I?J??Q??O?_@@??@??????'
>>> binary_string_from_graph6(g6, Integer(32))
'00000000000000000000010000000000001000010000010000000100000000000000010000000010
↪0000...0100000000000001000100000010000000000000000000000000000001010000000010110
↪0000000000001001000000000000001000000000010000000001000001000000000000000001000
↪0000000000000000000000000000000'
```

sage.graphs.generic_graph_pyx.**binary_string_to_graph6**(*x*)

Transform a binary string into its graph6 representation.

This helper function is named R in [McK2015].

INPUT:

- `x` – a binary string

EXAMPLES:

```
sage: from sage.graphs.generic_graph_pyx import binary_string_to_graph6
sage: binary_string_to_graph6('1101110101101100101110000011000000010000000001')
'vUqwK@?G'
```

```
>>> from sage.all import *
>>> from sage.graphs.generic_graph_pyx import binary_string_to_graph6
>>> binary_string_to_graph6('1101110101101100101110000011000000010000000001')
'vUqwK@?G'
```

sage.graphs.generic_graph_pyx.**find_hamiltonian**(*G*, *max_iter=100000*, *reset_bound=30000*, *backtrack_bound=1000*, *find_path=False*)

Randomized backtracking for finding Hamiltonian cycles and paths.

ALGORITHM:

A path `P` is maintained during the execution of the algorithm. Initially the path will contain an edge of the graph. Every 10 iterations the path is reversed. Every `reset_bound` iterations the path will be cleared and the procedure is restarted. Every `backtrack_bound` steps we discard the last five vertices and continue with the procedure. The total number of steps in the algorithm is controlled by `max_iter`. If a Hamiltonian cycle or Hamiltonian path is found it is returned. If the number of steps reaches `max_iter` then a longest path is returned. See OUTPUT for more details.

INPUT:

- `G` – graph
- `max_iter` – maximum number of iterations
- `reset_bound` – number of iterations before restarting the procedure

- `backtrack_bound` – number of iterations to elapse before discarding the last 5 vertices of the path
- `find_path` – boolean (default: `False`); if set to `True`, will search a Hamiltonian path. If `False`, will search for a Hamiltonian cycle.

OUTPUT:

A pair `(B, P)`, where `B` is a Boolean and `P` is a list of vertices.

- If `B` is `True` and `find_path` is `False`, `P` represents a Hamiltonian cycle.
- If `B` is `True` and `find_path` is `True`, `P` represents a Hamiltonian path.
- If `B` is `False`, then `P` represents the longest path found during the execution of the algorithm.

Warning

May loop endlessly when run on a graph with vertices of degree 1.

EXAMPLES:

For demonstration purposes we fix a random seed:

```
sage: set_random_seed(0)
```

```
>>> from sage.all import *
>>> set_random_seed(Integer(0))
```

First we try the algorithm in the Dodecahedral graph, which is Hamiltonian, so we are able to find a Hamiltonian cycle and a Hamiltonian path:

```
sage: from sage.graphs.generic_graph_pyx import find_hamiltonian as fh
sage: G=graphs.DodecahedralGraph()
sage: fh(G)
(True, [9, 13, 14, 7, 8, 1, 0, 19, 18, 17, 4, 3, 2, 6, 5, 15, 16, 12, 11, 10])
sage: fh(G,find_path=True)
(True, [4, 5, 6, 7, 14, 15, 16, 17, 18, 11, 12, 13, 9, 10, 0, 19, 3, 2, 1, 8])
```

```
>>> from sage.all import *
>>> from sage.graphs.generic_graph_pyx import find_hamiltonian as fh
>>> G=graphs.DodecahedralGraph()
>>> fh(G)
(True, [9, 13, 14, 7, 8, 1, 0, 19, 18, 17, 4, 3, 2, 6, 5, 15, 16, 12, 11, 10])
>>> fh(G,find_path=True)
(True, [4, 5, 6, 7, 14, 15, 16, 17, 18, 11, 12, 13, 9, 10, 0, 19, 3, 2, 1, 8])
```

Another test, now in the Möbius-Kantor graph which is also Hamiltonian, as in our previous example, we are able to find a Hamiltonian cycle and path:

```
sage: G=graphs.MoebiusKantorGraph()
sage: fh(G)
(True, [2, 10, 13, 5, 4, 12, 15, 7, 6, 14, 9, 1, 0, 8, 11, 3])
sage: fh(G,find_path=True)
(True, [13, 8, 11, 14, 9, 12, 15, 10, 2, 3, 4, 5, 6, 7, 0, 1])
```

```
>>> from sage.all import *
>>> G=graphs.MoebiusKantorGraph()
>>> fh(G)
(True, [2, 10, 13, 5, 4, 12, 15, 7, 6, 14, 9, 1, 0, 8, 11, 3])
>>> fh(G,find_path=True)
(True, [13, 8, 11, 14, 9, 12, 15, 10, 2, 3, 4, 5, 6, 7, 0, 1])
```

Now, we try the algorithm on a non Hamiltonian graph, the Petersen graph. This graph is known to be hypohamiltonian, so a Hamiltonian path can be found:

```
sage: G=graphs.PetersenGraph()
sage: fh(G)
(False, [5, 7, 2, 1, 0, 4, 9, 6, 8, 3])
sage: fh(G,find_path=True)
(True, [6, 9, 4, 3, 8, 5, 7, 2, 1, 0])
```

```
>>> from sage.all import *
>>> G=graphs.PetersenGraph()
>>> fh(G)
(False, [5, 7, 2, 1, 0, 4, 9, 6, 8, 3])
>>> fh(G,find_path=True)
(True, [6, 9, 4, 3, 8, 5, 7, 2, 1, 0])
```

We now show the algorithm working on another known hypohamiltonian graph, the generalized Petersen graph with parameters 11 and 2:

```
sage: G=graphs.GeneralizedPetersenGraph(11,2)
sage: fh(G)
(False,
 [12, 21, 19, 17, 6, 7, 18, 20, 11, 13, 15,
  4, 5, 16, 14, 3, 2, 1, 0, 10, 9, 8])

sage: fh(G,find_path=True)
(True,
 [20, 11, 13, 15, 4, 5, 16, 18, 7, 6, 17,
  19, 8, 9, 10, 21, 12, 14, 3, 2, 1, 0])
```

```
>>> from sage.all import *
>>> G=graphs.GeneralizedPetersenGraph(Integer(11),Integer(2))
>>> fh(G)
(False,
 [12, 21, 19, 17, 6, 7, 18, 20, 11, 13, 15,
  4, 5, 16, 14, 3, 2, 1, 0, 10, 9, 8])

>>> fh(G,find_path=True)
(True,
 [20, 11, 13, 15, 4, 5, 16, 18, 7, 6, 17,
  19, 8, 9, 10, 21, 12, 14, 3, 2, 1, 0])
```

Finally, an example on a graph which does not have a Hamiltonian path:

```
sage: G = graphs.HyperStarGraph(5, 2)
sage: G.order()
```

(continues on next page)

(continued from previous page)

```
10
sage: b, P = fh(G,find_path=False)
sage: b, len(P)
(False, 9)
sage: b, P = fh(G,find_path=True)
sage: b, len(P)
(False, 9)
```

```
>>> from sage.all import *
>>> G = graphs.HyperStarGraph(Integer(5), Integer(2))
>>> G.order()
10
>>> b, P = fh(G,find_path=False)
>>> b, len(P)
(False, 9)
>>> b, P = fh(G,find_path=True)
>>> b, len(P)
(False, 9)
```

The method can also be used for directed graphs:

```
sage: G = DiGraph([(0, 1), (1, 2), (2, 3)])
sage: fh(G)
(False, [0, 1, 2, 3])
sage: G = G.reverse()
sage: fh(G)
(False, [3, 2, 1, 0])
sage: G = DiGraph()
sage: G.add_cycle([0, 1, 2, 3, 4, 5])
sage: b, P = fh(G)
sage: b, len(P)
(True, 6)
```

```
>>> from sage.all import *
>>> G = DiGraph([(Integer(0), Integer(1)), (Integer(1), Integer(2)), (Integer(2),␣
↪Integer(3))])
>>> fh(G)
(False, [0, 1, 2, 3])
>>> G = G.reverse()
>>> fh(G)
(False, [3, 2, 1, 0])
>>> G = DiGraph()
>>> G.add_cycle([Integer(0), Integer(1), Integer(2), Integer(3), Integer(4),␣
↪Integer(5)])
>>> b, P = fh(G)
>>> b, len(P)
(True, 6)
```

sage.graphs.generic_graph_pyx.**int_to_binary_string**(*n*)

A quick python int to binary string conversion.

INPUT:

- n – integer

EXAMPLES:

```
sage: sage.graphs.generic_graph_pyx.int_to_binary_string(389)
'110000101'
sage: Integer(389).binary()
'110000101'
sage: sage.graphs.generic_graph_pyx.int_to_binary_string(2007)
'11111010111'
```

```
>>> from sage.all import *
>>> sage.graphs.generic_graph_pyx.int_to_binary_string(Integer(389))
'110000101'
>>> Integer(Integer(389)).binary()
'110000101'
>>> sage.graphs.generic_graph_pyx.int_to_binary_string(Integer(2007))
'11111010111'
```

sage.graphs.generic_graph_pyx.**layout_split**(*layout_function*, *G*, ***options*)

Graph each component of G separately with layout_function, placing them adjacent to each other.

This is done because several layout methods need the input graph to be connected. For instance, on a disconnected graph, the spring layout will push components further and further from each other without bound, resulting in very tight clumps for each component.

Note

If the axis are scaled to fit the plot in a square, the horizontal distance may end up being "squished" due to the several adjacent components.

EXAMPLES:

```
sage: G = graphs.DodecahedralGraph()
sage: for i in range(10): G.add_cycle(list(range(100*i, 100*i+3)))
sage: from sage.graphs.generic_graph_pyx import layout_split, spring_layout_fast
sage: D = layout_split(spring_layout_fast, G); D  # random
{0: [0.77..., 0.06...],
 ...
 902: [3.13..., 0.22...]}
```

```
>>> from sage.all import *
>>> G = graphs.DodecahedralGraph()
>>> for i in range(Integer(10)): G.add_cycle(list(range(Integer(100)*i,␣
↪Integer(100)*i+Integer(3))))
>>> from sage.graphs.generic_graph_pyx import layout_split, spring_layout_fast
>>> D = layout_split(spring_layout_fast, G); D  # random
{0: [0.77..., 0.06...],
 ...
 902: [3.13..., 0.22...]}
```

AUTHOR:

Robert Bradshaw

sage.graphs.generic_graph_pyx.**length_and_string_from_graph6**(*s*)

Return a pair `(length, graph6_string)` from a graph6 string of unknown length.

This helper function is the inverse of N from [McK2015].

INPUT:

- `s` – a graph6 string describing a binary vector (and encoding its length)

EXAMPLES:

```
sage: from sage.graphs.generic_graph_pyx import length_and_string_from_graph6
sage: g6 = '~??~?????_@?CG??B??@OG?C?G???GO??W@a???CO???OACC?OA?P@G??O?'
sage: g6 += '?????G??C????c?G?CC?_?@???C_??_?C????PO?C_??AA?OOAHCA___?C'
sage: g6 += 'C?A?CAOGO??????A??G?GR?C?_o`???g???A_C?OG??O?G_IA????_QO@E'
sage: g6 += 'G???O??C?_?C@?G???@?_??AC?AO?a???O?????A?_Dw?H???__O@AAOAA'
sage: g6 += 'Cd?_C??G?G@??GO?_???O@?_O??W??@P???AG??B?????G??GG???A??@?'
sage: g6 += 'aC_G@A??O??_?A?????O@Z?_@M????GQ@_G@?C?'
sage: length_and_string_from_graph6(g6)
(63, '?????_@?CG??B??@OG?C?G???GO??W@a???CO???OACC?OA?P@G??O??????G??C????c?G?CC?_
↪?@???C_??_?C????PO?C_??AA?OOAHCA___?CC?A?CAOGO??????A??G?GR?C?_o`???g???A_C?OG??
↪O?G_IA????_QO@EG???O??C?_?C@?G???@?_??AC?AO?a???O?????A?_Dw?H???__O@AAOAACd?_C??
↪G?G@??GO?_???O@?_O??W??@P???AG??B?????G??GG???A??@?aC_G@A??O??_?A?????O@Z?_@M???
↪?GQ@_G@?C?')
sage: g6 = '_???C?@AA?_?A?O?C??S??O?q_?P?CHD??@?C?GC???C??GG?C_??O?COG?'
sage: g6 += '???I?J??Q??O?_@@??@??????'
sage: length_and_string_from_graph6(g6)
(32, '???C?@AA?_?A?O?C??S??O?q_?P?CHD??@?C?GC???C??GG?C_??O?COG????I?J??Q??O?_@@??
↪@??????')
```

```
>>> from sage.all import *
>>> from sage.graphs.generic_graph_pyx import length_and_string_from_graph6
>>> g6 = '~??~?????_@?CG??B??@OG?C?G???GO??W@a???CO???OACC?OA?P@G??O?'
>>> g6 += '?????G??C????c?G?CC?_?@???C_??_?C????PO?C_??AA?OOAHCA___?C'
>>> g6 += 'C?A?CAOGO??????A??G?GR?C?_o`???g???A_C?OG??O?G_IA????_QO@E'
>>> g6 += 'G???O??C?_?C@?G???@?_??AC?AO?a???O?????A?_Dw?H???__O@AAOAA'
>>> g6 += 'Cd?_C??G?G@??GO?_???O@?_O??W??@P???AG??B?????G??GG???A??@?'
>>> g6 += 'aC_G@A??O??_?A?????O@Z?_@M????GQ@_G@?C?'
>>> length_and_string_from_graph6(g6)
(63, '?????_@?CG??B??@OG?C?G???GO??W@a???CO???OACC?OA?P@G??O??????G??C????c?G?CC?_
↪?@???C_??_?C????PO?C_??AA?OOAHCA___?CC?A?CAOGO??????A??G?GR?C?_o`???g???A_C?OG??
↪O?G_IA????_QO@EG???O??C?_?C@?G???@?_??AC?AO?a???O?????A?_Dw?H???__O@AAOAACd?_C??
↪G?G@??GO?_???O@?_O??W??@P???AG??B?????G??GG???A??@?aC_G@A??O??_?A?????O@Z?_@M???
↪?GQ@_G@?C?')
>>> g6 = '_???C?@AA?_?A?O?C??S??O?q_?P?CHD??@?C?GC???C??GG?C_??O?COG?'
>>> g6 += '???I?J??Q??O?_@@??@??????'
>>> length_and_string_from_graph6(g6)
(32, '???C?@AA?_?A?O?C??S??O?q_?P?CHD??@?C?GC???C??GG?C_??O?COG????I?J??Q??O?_@@??
↪@??????')
```

sage.graphs.generic_graph_pyx.**small_integer_to_graph6**(*n*)

Encode a small integer (i.e. a number of vertices) as a graph6 string.

This helper function is named N [McK2015].

INPUT:

- n – integer

EXAMPLES:

```
sage: from sage.graphs.generic_graph_pyx import small_integer_to_graph6
sage: small_integer_to_graph6(13)
'L'
sage: small_integer_to_graph6(136)
'~?AG'
```

```
>>> from sage.all import *
>>> from sage.graphs.generic_graph_pyx import small_integer_to_graph6
>>> small_integer_to_graph6(Integer(13))
'L'
>>> small_integer_to_graph6(Integer(136))
'~?AG'
```

sage.graphs.generic_graph_pyx.**spring_layout_fast**(*G*, *iterations=50*, *dim=2*, *vpos=None*, *rescale=True*, *height=False*, *by_component=False*, ***options*)

Spring force model layout.

This function primarily acts as a wrapper around run_spring, converting to and from raw C types.

This kind of speed cannot be achieved by naive Cythonification of the function alone, especially if we require a function call (let alone an object creation) every time we want to add a pair of doubles.

INPUT:

- by_component – boolean

EXAMPLES:

```
sage: G = graphs.DodecahedralGraph()
sage: for i in range(10): G.add_cycle(list(range(100*i, 100*i+3)))
sage: from sage.graphs.generic_graph_pyx import spring_layout_fast
sage: pos = spring_layout_fast(G)
sage: pos[0]  # random
[0.00..., 0.03...]
sage: sorted(pos.keys()) == sorted(G)
True
```

```
>>> from sage.all import *
>>> G = graphs.DodecahedralGraph()
>>> for i in range(Integer(10)): G.add_cycle(list(range(Integer(100)*i,␣
↪Integer(100)*i+Integer(3))))
>>> from sage.graphs.generic_graph_pyx import spring_layout_fast
>>> pos = spring_layout_fast(G)
>>> pos[Integer(0)]  # random
[0.00..., 0.03...]
>>> sorted(pos.keys()) == sorted(G)
True
```

With split=True, each component of G is laid out separately, placing them adjacent to each other. This is done because on a disconnected graph, the spring layout will push components further and further from each other without bound, resulting in very tight clumps for each component.

If the axis are scaled to fit the plot in a square, the horizontal distance may end up being "squished" due to the several adjacent components.

```
sage: G = graphs.DodecahedralGraph()
sage: for i in range(10): G.add_cycle(list(range(100*i, 100*i+3)))
sage: from sage.graphs.generic_graph_pyx import spring_layout_fast
sage: pos = spring_layout_fast(G, by_component = True)
sage: pos[0]  # random
[2.21..., -0.00...]
sage: len(pos) == G.order()
True
```

```
>>> from sage.all import *
>>> G = graphs.DodecahedralGraph()
>>> for i in range(Integer(10)): G.add_cycle(list(range(Integer(100)*i,␣
↪Integer(100)*i+Integer(3))))
>>> from sage.graphs.generic_graph_pyx import spring_layout_fast
>>> pos = spring_layout_fast(G, by_component = True)
>>> pos[Integer(0)]  # random
[2.21..., -0.00...]
>>> len(pos) == G.order()
True
```

sage.graphs.generic_graph_pyx.**transitive_reduction_acyclic**(*G*, *immutable=None*)

Return the transitive reduction of an acyclic digraph.

INPUT:

- `G` – an acyclic digraph
- `immutable` – boolean (default: `None`); whether to create a mutable/immutable transitive closure. `immutable=None` (default) means that the (di)graph and its transitive closure will behave the same way.

EXAMPLES:

```
sage: from sage.graphs.generic_graph_pyx import transitive_reduction_acyclic
sage: G = posets.BooleanLattice(4).hasse_diagram()
sage: G == transitive_reduction_acyclic(G.transitive_closure())
True
```

```
>>> from sage.all import *
>>> from sage.graphs.generic_graph_pyx import transitive_reduction_acyclic
>>> G = posets.BooleanLattice(Integer(4)).hasse_diagram()
>>> G == transitive_reduction_acyclic(G.transitive_closure())
True
```

5.45 Orientations

This module implements several methods to compute orientations of undirected graphs subject to specific constraints (e.g., acyclic, strongly connected, etc.). It also implements some iterators over all these orientations.

This module contains the following methods

orient()	Return an oriented version of G according the input function f.
orientations()	Return an iterator over orientations of G.
acyclic_orientations()	Return an iterator over all acyclic orientations of an undirected graph G.
strong_orientation()	Return a strongly connected orientation of the graph G.
strong_orientations_iterator()	Return an iterator over all strong orientations of a graph G
random_orientation()	Return a random orientation of a graph G
minimum_outdegree_orientation()	Return an orientation of G with the smallest possible maximum outdegree.
bounded_outdegree_orientation()	Return an orientation of G such that every vertex v has out-degree less than $b(v)$.
eulerian_orientation()	Return an Eulerian orientation of the graph G.

5.45.1 Authors

- Kolja Knauer, Petru Valicov (2017-01-10) – initial version

5.45.2 Methods

sage.graphs.orientations.**acyclic_orientations**(G)

Return an iterator over all acyclic orientations of an undirected graph G.

ALGORITHM:

The algorithm is based on [Sq1998]. It presents an efficient algorithm for listing the acyclic orientations of a graph. The algorithm is shown to require O(n) time per acyclic orientation generated, making it the most efficient known algorithm for generating acyclic orientations.

The function uses a recursive approach to generate acyclic orientations of the graph. It reorders the vertices and edges of the graph, creating a new graph with updated labels. Then, it iteratively generates acyclic orientations by considering subsets of edges and checking whether they form upsets in a corresponding poset.

INPUT:

- `G` – an undirected graph

OUTPUT: an iterator over all acyclic orientations of the input graph

Note

The function assumes that the input graph is undirected and the edges are unlabelled.

EXAMPLES:

To count the number of acyclic orientations for a graph:

```
sage: g = Graph([(0, 3), (0, 4), (3, 4), (1, 3), (1, 2), (2, 3), (2, 4)])
sage: it = g.acyclic_orientations()
sage: len(list(it))
54
```

```
>>> from sage.all import *
>>> g = Graph([(Integer(0), Integer(3)), (Integer(0), Integer(4)), (Integer(3),␣
↪Integer(4)), (Integer(1), Integer(3)), (Integer(1), Integer(2)), (Integer(2),␣
```

(continues on next page)

(continued from previous page)

```
↪Integer(3)), (Integer(2), Integer(4))])
>>> it = g.acyclic_orientations()
>>> len(list(it))
54
```

Test for arbitrary vertex labels:

```
sage: g_str = Graph([('abc', 'def'), ('ghi', 'def'), ('xyz', 'abc'),
....:                ('xyz', 'uvw'), ('uvw', 'abc'), ('uvw', 'ghi')])
sage: it = g_str.acyclic_orientations()
sage: len(list(it))
42
```

```
>>> from sage.all import *
>>> g_str = Graph([('abc', 'def'), ('ghi', 'def'), ('xyz', 'abc'),
...                ('xyz', 'uvw'), ('uvw', 'abc'), ('uvw', 'ghi')])
>>> it = g_str.acyclic_orientations()
>>> len(list(it))
42
```

Check that the method returns properly relabeled acyclic digraphs:

```
sage: g = Graph([(0, 1), (1, 2), (2, 3), (3, 0), (0, 2)])
sage: orientations = set([frozenset(d.edges(labels=false)) for d in g.acyclic_
↪orientations()])
sage: len(orientations)
18
sage: all(d.is_directed_acyclic() for d in g.acyclic_orientations())
True
```

```
>>> from sage.all import *
>>> g = Graph([(Integer(0), Integer(1)), (Integer(1), Integer(2)), (Integer(2),␣
↪Integer(3)), (Integer(3), Integer(0)), (Integer(0), Integer(2))])
>>> orientations = set([frozenset(d.edges(labels=false)) for d in g.acyclic_
↪orientations()])
>>> len(orientations)
18
>>> all(d.is_directed_acyclic() for d in g.acyclic_orientations())
True
```

sage.graphs.orientations.**bounded_outdegree_orientation**(*G*, *bound*, *solver*, *verbose=None*, *integrality_tolerance=False*)

Return an orientation of G such that every vertex v has out-degree less than $b(v)$.

INPUT:

- `G` – an undirected graph
- `bound` – maximum bound on the out-degree. Can be of three different types :
  - An integer k. In this case, computes an orientation whose maximum out-degree is less than k.
  - A dictionary associating to each vertex its associated maximum out-degree.
  - A function associating to each vertex its associated maximum out-degree.

- `solver` – string (default: `None`); specifies a Mixed Integer Linear Programming (MILP) solver to be used. If set to `None`, the default one is used. For more information on MILP solvers and which default solver is used, see the method `solve` of the class `MixedIntegerLinearProgram`.
- `verbose` – integer (default: 0); sets the level of verbosity. Set to 0 by default, which means quiet.
- `integrality_tolerance` – float; parameter for use with MILP solvers over an inexact base ring; see `MixedIntegerLinearProgram.get_values()`.

OUTPUT:

A DiGraph representing the orientation if it exists. A `ValueError` exception is raised otherwise.

ALGORITHM:

The problem is solved through a maximum flow :

Given a graph G, we create a `DiGraph` D defined on $E(G) \cup V(G) \cup \{s, t\}$. We then link s to all of $V(G)$ (these edges having a capacity equal to the bound associated to each element of $V(G)$), and all the elements of $E(G)$ to t . We then link each $v \in V(G)$ to each of its incident edges in G. A maximum integer flow of value $|E(G)|$ corresponds to an admissible orientation of G. Otherwise, none exists.

EXAMPLES:

There is always an orientation of a graph G such that a vertex v has out-degree at most $\lceil \frac{d(v)}{2} \rceil$:

```
sage: g = graphs.RandomGNP(40, .4)
sage: b = lambda v: integer_ceil(g.degree(v)/2)
sage: D = g.bounded_outdegree_orientation(b)
sage: all(D.out_degree(v) <= b(v) for v in g)
True
```

```
>>> from sage.all import *
>>> g = graphs.RandomGNP(Integer(40), RealNumber('.4'))
>>> b = lambda v: integer_ceil(g.degree(v)/Integer(2))
>>> D = g.bounded_outdegree_orientation(b)
>>> all(D.out_degree(v) <= b(v) for v in g)
True
```

Chvatal's graph, being 4-regular, can be oriented in such a way that its maximum out-degree is 2:

```
sage: g = graphs.ChvatalGraph()
sage: D = g.bounded_outdegree_orientation(2)
sage: max(D.out_degree())
2
```

```
>>> from sage.all import *
>>> g = graphs.ChvatalGraph()
>>> D = g.bounded_outdegree_orientation(Integer(2))
>>> max(D.out_degree())
2
```

For any graph G, it is possible to compute an orientation such that the maximum out-degree is at most the maximum average degree of G divided by 2. Anything less, though, is impossible.

sage: g = graphs.RandomGNP(40, .4) sage: mad = g.maximum_average_degree() # needs sage.numerical.mip

Hence this is possible

```
sage: d = g.bounded_outdegree_orientation(integer_ceil(mad/2))                #␣
↪needs sage.numerical.mip
```

```
>>> from sage.all import *
>>> d = g.bounded_outdegree_orientation(integer_ceil(mad/Integer(2)))           ␣
↪ # needs sage.numerical.mip
```

While this is not:

```
sage: try:                                                                    #␣
↪needs sage.numerical.mip
....:     g.bounded_outdegree_orientation(integer_ceil(mad/2-1))
....:     print("Error")
....: except ValueError:
....:     pass
```

```
>>> from sage.all import *
>>> try:                                                                  # needs␣
↪sage.numerical.mip
...     g.bounded_outdegree_orientation(integer_ceil(mad/Integer(2)-Integer(1)))
...     print("Error")
... except ValueError:
...     pass
```

The bounds can be specified in different ways:

```
sage: g = graphs.PetersenGraph()
sage: b = lambda v: integer_ceil(g.degree(v)/2)
sage: D = g.bounded_outdegree_orientation(b)
sage: b_dict = {u: b(u) for u in g}
sage: D = g.bounded_outdegree_orientation(b_dict)
sage: unique_bound = 2
sage: D = g.bounded_outdegree_orientation(unique_bound)
```

```
>>> from sage.all import *
>>> g = graphs.PetersenGraph()
>>> b = lambda v: integer_ceil(g.degree(v)/Integer(2))
>>> D = g.bounded_outdegree_orientation(b)
>>> b_dict = {u: b(u) for u in g}
>>> D = g.bounded_outdegree_orientation(b_dict)
>>> unique_bound = Integer(2)
>>> D = g.bounded_outdegree_orientation(unique_bound)
```

sage.graphs.orientations.**eulerian_orientation**(G)

Return an Eulerian orientation of the graph G.

An Eulerian graph being a graph such that any vertex has an even degree, an Eulerian orientation of a graph is an orientation of its edges such that each vertex v verifies $d^+(v) = d^-(v) = d(v)/2$, where d^+ and d^- respectively represent the out-degree and the in-degree of a vertex.

If the graph is not Eulerian, the orientation verifies for any vertex v that $|d^+(v) - d^-(v)| \leq 1$.

INPUT:

- `G` – an undirected graph

ALGORITHM:

This algorithm is a random walk through the edges of the graph, which orients the edges according to the walk. When a vertex is reached which has no non-oriented edge (this vertex must have odd degree), the walk resumes at another vertex of odd degree, if any.

This algorithm has time complexity in $O(n+m)$ for `SparseGraph` and $O(n^2)$ for `DenseGraph`, where m is the number of edges in the graph and n is the number of vertices in the graph.

EXAMPLES:

The CubeGraph with parameter 4, which is regular of even degree, has an Eulerian orientation such that $d^+ = d^-$:

```
sage: g = graphs.CubeGraph(4)
sage: g.degree()
[4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4]
sage: o = g.eulerian_orientation()
sage: o.in_degree()
[2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2]
sage: o.out_degree()
[2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2]
```

```
>>> from sage.all import *
>>> g = graphs.CubeGraph(Integer(4))
>>> g.degree()
[4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4]
>>> o = g.eulerian_orientation()
>>> o.in_degree()
[2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2]
>>> o.out_degree()
[2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2]
```

Secondly, the Petersen Graph, which is 3 regular has an orientation such that the difference between d^+ and d^- is at most 1:

```
sage: g = graphs.PetersenGraph()
sage: o = g.eulerian_orientation()
sage: o.in_degree()
[2, 2, 2, 2, 2, 1, 1, 1, 1, 1]
sage: o.out_degree()
[1, 1, 1, 1, 1, 2, 2, 2, 2, 2]
```

```
>>> from sage.all import *
>>> g = graphs.PetersenGraph()
>>> o = g.eulerian_orientation()
>>> o.in_degree()
[2, 2, 2, 2, 2, 1, 1, 1, 1, 1]
>>> o.out_degree()
[1, 1, 1, 1, 1, 2, 2, 2, 2, 2]
```

`sage.graphs.orientations.`**`minimum_outdegree_orientation`**(*G*, *by_weight*, *weight_function=False*, *check_weight=None*, *solver=True*, *verbose=None*, *integrality_tolerance=0*)

Return an orientation of G with the smallest possible maximum outdegree.

Given a Graph G, it is polynomial to compute an orientation D of the edges of G such that the maximum out-degree in D is minimized. This problem, though, is NP-complete in the weighted case [AMOZ2006].

INPUT:

- `G` – an undirected graph
- `by_weight` – boolean (default: `False`); if `True`, uses the weight given by `weight_function`, or the edge label if `weight_function` is `None`, to compute the orientation. If `False`, gives a weight of 1 to all the edges.
- `weight_function` – function (default: `None`); a function that takes as input an edge `(u, v, l)` and outputs its weight. If not `None`, `by_weight` is automatically set to `True`. If `None` and `by_weight` is `True`, we use the edge label `l`, if `l` is not `None`, else 1 as a weight.
- `check_weight` – boolean (default: `True`); whether to check that the `weight_function` outputs a number for each edge
- `solver` – string (default: `None`); specifies a Mixed Integer Linear Programming (MILP) solver to be used. If set to `None`, the default one is used. For more information on MILP solvers and which default solver is used, see the method `solve` of the class `MixedIntegerLinearProgram`.
- `verbose` – integer (default: 0); sets the level of verbosity. Set to 0 by default, which means quiet.
- `integrality_tolerance` – float; parameter for use with MILP solvers over an inexact base ring; see `MixedIntegerLinearProgram.get_values()`.

EXAMPLES:

Given a complete bipartite graph $K_{n,m}$, the maximum out-degree of an optimal orientation is $\left\lceil \frac{nm}{n+m} \right\rceil$:

```
sage: g = graphs.CompleteBipartiteGraph(3,4)
sage: o = g.minimum_outdegree_orientation()                                    #␣
↪needs sage.numerical.mip
sage: max(o.out_degree()) == integer_ceil((4*3)/(3+4))                         #␣
↪needs sage.numerical.mip
True
```

```
>>> from sage.all import *
>>> g = graphs.CompleteBipartiteGraph(Integer(3),Integer(4))
>>> o = g.minimum_outdegree_orientation()                                      #␣
↪needs sage.numerical.mip
>>> max(o.out_degree()) == integer_ceil((Integer(4)*Integer(3))/
↪(Integer(3)+Integer(4)))                          # needs sage.numerical.mip
True
```

Show the influence of edge labels on the solution:

```
sage: g = graphs.PetersenGraph()
sage: o = g.minimum_outdegree_orientation(by_weight=False)
sage: max(o.out_degree())
2
sage: _ = [g.set_edge_label(u, v, 1) for u, v in g.edge_iterator(labels=False)]
sage: o = g.minimum_outdegree_orientation(by_weight=True)
sage: max(o.out_degree())
2
sage: g.set_edge_label(0, 1, 100)
sage: o = g.minimum_outdegree_orientation(by_weight=True)
```

(continues on next page)

(continued from previous page)

```
sage: max(o.out_degree())
3
```

```
>>> from sage.all import *
>>> g = graphs.PetersenGraph()
>>> o = g.minimum_outdegree_orientation(by_weight=False)
>>> max(o.out_degree())
2
>>> _ = [g.set_edge_label(u, v, Integer(1)) for u, v in g.edge_
↪iterator(labels=False)]
>>> o = g.minimum_outdegree_orientation(by_weight=True)
>>> max(o.out_degree())
2
>>> g.set_edge_label(Integer(0), Integer(1), Integer(100))
>>> o = g.minimum_outdegree_orientation(by_weight=True)
>>> max(o.out_degree())
3
```

A `weight_function` can read the weight from anywhere in the edge, and setting it implies `by_weight=True`:

```
sage: g = graphs.PetersenGraph()
sage: _ = [g.set_edge_label(u, v, {'w': 1})
....:      for u, v in g.edge_iterator(labels=False)]
sage: g.set_edge_label(0, 1, {'w': 100})
sage: o = g.minimum_outdegree_orientation(                                  #␣
↪needs sage.numerical.mip
....:         weight_function=lambda e: e[2]['w'])
sage: max(o.out_degree())                                                   #␣
↪needs sage.numerical.mip
3
```

```
>>> from sage.all import *
>>> g = graphs.PetersenGraph()
>>> _ = [g.set_edge_label(u, v, {'w': Integer(1)})
...      for u, v in g.edge_iterator(labels=False)]
>>> g.set_edge_label(Integer(0), Integer(1), {'w': Integer(100)})
>>> o = g.minimum_outdegree_orientation(                                    #␣
↪needs sage.numerical.mip
...         weight_function=lambda e: e[Integer(2)]['w'])
>>> max(o.out_degree())                                                     #␣
↪needs sage.numerical.mip
3
```

The `use_edge_labels` keyword is deprecated in favour of `by_weight`:

```
sage: g = graphs.PetersenGraph()
sage: o = g.minimum_outdegree_orientation(use_edge_labels=True)             #␣
↪needs sage.numerical.mip
doctest:warning...
DeprecationWarning: use the option 'by_weight' instead of 'use_edge_labels'
See https://github.com/sagemath/sage/issues/42760 for details.
sage: max(o.out_degree())                                                   #␣
```

(continues on next page)

(continued from previous page)

```
↪needs sage.numerical.mip
2
```

```
>>> from sage.all import *
>>> g = graphs.PetersenGraph()
>>> o = g.minimum_outdegree_orientation(use_edge_labels=True)                    #␣
↪needs sage.numerical.mip
doctest:warning...
DeprecationWarning: use the option 'by_weight' instead of 'use_edge_labels'
See https://github.com/sagemath/sage/issues/42760 for details.
>>> max(o.out_degree())                                                          #␣
↪needs sage.numerical.mip
2
```

sage.graphs.orientations.**orient**(*G*, *f*, *weighted=None*, *data_structure=None*, *sparse=None*, *immutable=None*, *hash_labels=None*)

Return an oriented version of G according the input function f.

INPUT:

- `G` – an undirected graph
- `f` – a function that inputs an edge and outputs an orientation of this edge
- `weighted` – boolean (default: `None`); weightedness for the oriented digraph. By default (`None`), the graph and its orientation will behave the same.
- `sparse` – boolean (default: `None`); `sparse=True` is an alias for `data_structure="sparse"`, and `sparse=False` is an alias for `data_structure="dense"`. Only used when `data_structure=None`.
- `data_structure` – string (default: `None`); one of `'sparse'`, `'static_sparse'`, or `'dense'`. See the documentation of *DiGraph*.
- `immutable` – boolean (default: `None`); whether to create a mutable/immutable digraph. Only used when `data_structure=None`.
  - `immutable=None` (default) means that the graph and its orientation will behave the same way.
  - `immutable=True` is a shortcut for `data_structure='static_sparse'`
  - `immutable=False` means that the created digraph is mutable. When used to orient an immutable graph, the data structure used is `'sparse'` unless anything else is specified.
- `hash_labels` – boolean (default: `None`); whether to include edge labels during hashing of the oriented digraph. This parameter defaults to `True` if the graph is weighted. This parameter is ignored when parameter `immutable` is not `True`. Beware that trying to hash unhashable labels will raise an error.

OUTPUT: a *DiGraph* object

Note

This method behaves similarly to method *copy()*. That is, the returned digraph uses the same data structure by default, unless the user asks to use another data structure, and the attributes of the input graph are copied.

EXAMPLES:

```
sage: G = graphs.CycleGraph(4); G
Cycle graph: Graph on 4 vertices
sage: D = G.orient(lambda e:e if e[0] < e[1] else (e[1], e[0], e[2])); D
Orientation of Cycle graph: Digraph on 4 vertices
sage: sorted(D.edges(labels=False))
[(0, 1), (0, 3), (1, 2), (2, 3)]
```

```
>>> from sage.all import *
>>> G = graphs.CycleGraph(Integer(4)); G
Cycle graph: Graph on 4 vertices
>>> D = G.orient(lambda e:e if e[Integer(0)] < e[Integer(1)] else (e[Integer(1)],␣
↪e[Integer(0)], e[Integer(2)])); D
Orientation of Cycle graph: Digraph on 4 vertices
>>> sorted(D.edges(labels=False))
[(0, 1), (0, 3), (1, 2), (2, 3)]
```

sage.graphs.orientations.**orientations**(*G*, *data_structure=None*, *sparse=None*)

Return an iterator over orientations of G.

An *orientation* of an undirected graph is a directed graph such that every edge is assigned a direction. Hence there are 2^s oriented digraphs for a simple graph with s edges.

INPUT:

- `G` – an undirected graph
- `data_structure` – one of `'sparse'`, `'static_sparse'`, or `'dense'`; see the documentation of *Graph* or *DiGraph*; default is the data structure of G
- `sparse` – boolean (default: `None`); `sparse=True` is an alias for `data_structure="sparse"`, and `sparse=False` is an alias for `data_structure="dense"`. By default (`None`), guess the most suitable data structure.

Warning

This always considers multiple edges of graphs as distinguishable, and hence, may have repeated digraphs.

See also

- *strong_orientation()*
- *strong_orientations_iterator()*
- *nauty_directg()*
- *random_orientation()*

EXAMPLES:

```
sage: G = Graph([[1,2,3], [(1, 2, 'a'), (1, 3, 'b')]], format='vertices_and_edges
↪')
sage: it = G.orientations()
sage: D = next(it)
```

(continues on next page)

(continued from previous page)

```
sage: D.edges(sort=True)
[(1, 2, 'a'), (1, 3, 'b')]
sage: D = next(it)
sage: D.edges(sort=True)
[(1, 2, 'a'), (3, 1, 'b')]
```

```
>>> from sage.all import *
>>> G = Graph([[Integer(1),Integer(2),Integer(3)], [(Integer(1), Integer(2), 'a'),
↪ (Integer(1), Integer(3), 'b')]], format='vertices_and_edges')
>>> it = G.orientations()
>>> D = next(it)
>>> D.edges(sort=True)
[(1, 2, 'a'), (1, 3, 'b')]
>>> D = next(it)
>>> D.edges(sort=True)
[(1, 2, 'a'), (3, 1, 'b')]
```

sage.graphs.orientations.**random_orientation**(G)

Return a random orientation of a graph G.

An *orientation* of an undirected graph is a directed graph such that every edge is assigned a direction. Hence there are 2^m oriented digraphs for a simple graph with m edges.

INPUT:

- `G` – a Graph

EXAMPLES:

```
sage: from sage.graphs.orientations import random_orientation
sage: G = graphs.PetersenGraph()
sage: D = random_orientation(G)
sage: D.order() == G.order(), D.size() == G.size()
(True, True)
```

```
>>> from sage.all import *
>>> from sage.graphs.orientations import random_orientation
>>> G = graphs.PetersenGraph()
>>> D = random_orientation(G)
>>> D.order() == G.order(), D.size() == G.size()
(True, True)
```

See also

- *orientations()*
- *strong_orientation()*
- *strong_orientations_iterator()*
- *nauty_directg()*

sage.graphs.orientations.**strong_orientation**(G)

Return a strongly connected orientation of the graph G.

An orientation of an undirected graph is a digraph obtained by giving an unique direction to each of its edges. An orientation is said to be strong if there is a directed path between each pair of vertices. See also the Wikipedia article Strongly_connected_component.

If the graph is 2-edge-connected, a strongly connected orientation can be found in linear time. If the given graph is not 2-connected, the orientation returned will ensure that each 2-connected component has a strongly connected orientation.

INPUT:

- `G` – an undirected graph

OUTPUT: a digraph representing an orientation of the current graph

Note

- This method assumes that the input the graph is connected.
- The time complexity is $O(n+m)$ for `SparseGraph` and $O(n^2)$ for `DenseGraph` .

See also

- *orientations()*
- *strong_orientations_iterator()*
- *nauty_directg()*
- *random_orientation()*

EXAMPLES:

For a 2-regular graph, a strong orientation gives to each vertex an out-degree equal to 1:

```
sage: g = graphs.CycleGraph(5)
sage: g.strong_orientation().out_degree()
[1, 1, 1, 1, 1]
```

```
>>> from sage.all import *
>>> g = graphs.CycleGraph(Integer(5))
>>> g.strong_orientation().out_degree()
[1, 1, 1, 1, 1]
```

The Petersen Graph is 2-edge connected. It then has a strongly connected orientation:

```
sage: g = graphs.PetersenGraph()
sage: o = g.strong_orientation()
sage: len(o.strongly_connected_components())
1
```

```
>>> from sage.all import *
>>> g = graphs.PetersenGraph()
>>> o = g.strong_orientation()
>>> len(o.strongly_connected_components())
1
```

The same goes for the CubeGraph in any dimension:

```
sage: all(len(graphs.CubeGraph(i).strong_orientation().strongly_connected_
↪components()) == 1
....:     for i in range(2,6))
True
```

```
>>> from sage.all import *
>>> all(len(graphs.CubeGraph(i).strong_orientation().strongly_connected_
↪components()) == Integer(1)
...     for i in range(Integer(2),Integer(6)))
True
```

A multigraph also has a strong orientation:

```
sage: g = Graph([(0, 1), (0, 2), (1, 2)] * 2, multiedges=True)
sage: g.strong_orientation()
Multi-digraph on 3 vertices
```

```
>>> from sage.all import *
>>> g = Graph([(Integer(0), Integer(1)), (Integer(0), Integer(2)), (Integer(1),␣
↪Integer(2))] * Integer(2), multiedges=True)
>>> g.strong_orientation()
Multi-digraph on 3 vertices
```

sage.graphs.orientations.**strong_orientations_iterator**(G)

Return an iterator over all strong orientations of a graph G.

A strong orientation of a graph is an orientation of its edges such that the obtained digraph is strongly connected (i.e. there exist a directed path between each pair of vertices). According to Robbins' theorem (see the Wikipedia article Robbins_theorem), the graphs that have strong orientations are exactly the 2-edge-connected graphs (i.e., the bridgeless graphs).

ALGORITHM:

It is an adaptation of the algorithm published in [CGMRV16]. It runs in $O(mn)$ amortized time, where m is the number of edges and n is the number of vertices. The amortized time can be improved to $O(m)$ with a more involved method. In this function, first the graph is preprocessed and a spanning tree is generated. Then every orientation of the non-tree edges of the graph can be extended to at least one new strong orientation by orienting properly the edges of the spanning tree (this property is proved in [CGMRV16]). Therefore, this function generates all partial orientations of the non-tree edges and then launches a helper function corresponding to the generation algorithm described in [CGMRV16]. In order to avoid trivial symmetries, the orientation of an arbitrary edge is fixed before the start of the enumeration process.

INPUT:

- `G` – an undirected graph

OUTPUT: an iterator which will produce all strong orientations of this graph

Note

Works only for simple graphs (no multiple edges). To avoid symmetries an orientation of an arbitrary edge is fixed.

See also

- *orientations()*
- *strong_orientation()*
- *nauty_directg()*
- *random_orientation()*

EXAMPLES:

A cycle has one possible (non-symmetric) strong orientation:

```
sage: g = graphs.CycleGraph(4)
sage: it = g.strong_orientations_iterator()
sage: len(list(it))
1
```

```
>>> from sage.all import *
>>> g = graphs.CycleGraph(Integer(4))
>>> it = g.strong_orientations_iterator()
>>> len(list(it))
1
```

A tree cannot be strongly oriented:

```
sage: g = graphs.RandomTree(10)
sage: len(list(g.strong_orientations_iterator()))
0
```

```
>>> from sage.all import *
>>> g = graphs.RandomTree(Integer(10))
>>> len(list(g.strong_orientations_iterator()))
0
```

Neither can be a disconnected graph:

```
sage: g = graphs.CompleteGraph(6)
sage: g.add_vertex(7)
sage: len(list(g.strong_orientations_iterator()))
0
```

```
>>> from sage.all import *
>>> g = graphs.CompleteGraph(Integer(6))
>>> g.add_vertex(Integer(7))
>>> len(list(g.strong_orientations_iterator()))
0
```

5.46 Connectivity related functions

This module implements the connectivity based functions for graphs and digraphs. The methods in this module are also available as part of GenericGraph, DiGraph or Graph classes as aliases, and these methods can be accessed through this module or as class methods. Here is what the module can do:

For both directed and undirected graphs:

is_connected()	Check whether the (di)graph is connected.
connected_components()	Return the list of connected components
number_of_connected_components()	Return the number of connected components.
connected_components_subgraphs()	Return a list of connected components as graph objects.
connected_component_containing_vertex()	Return a list of the vertices connected to vertex.
connected_components_sizes()	Return the sizes of the connected components as a list.
blocks_and_cut_vertices()	Return the blocks and cut vertices of the graph.
blocks_and_cuts_tree()	Return the blocks-and-cuts tree of the graph.
is_biconnected()	Check whether the graph is biconnected.
biconnected_components()	Return the list of biconnected components.
biconnected_components_subgraphs()	Return a list of biconnected components as graph objects.
number_of_biconnected_components()	Return the number of biconnected components.
is_cut_edge()	Check whether the input edge is a cut-edge or a bridge.
is_edge_cut()	Check whether the input edges form an edge cut.
is_cut_vertex()	Check whether the input vertex is a cut-vertex.
is_vertex_cut()	Check whether the input vertices form a vertex cut.
edge_connectivity()	Return the edge connectivity of the graph.
vertex_connectivity()	Return the vertex connectivity of the graph.

For DiGraph:

is_strongly_connected()	Check whether the current `DiGraph` is strongly connected.
strongly_connected_components_digraph()	Return the digraph of the strongly connected components
strongly_connected_components_subgraphs()	Return the strongly connected components as a list of subgraphs.
strongly_connected_component_containing_vertex()	Return the strongly connected component containing a given vertex.
strong_articulation_points()	Return the strong articulation points of this digraph.

For undirected graphs:

bridges()	Return an iterator over the bridges (or cut edges) of given undirected graph.
cleave()	Return the connected subgraphs separated by the input vertex cut.
is_triconnected()	Check whether the graph is triconnected.
spqr_tree()	Return a SPQR-tree representing the triconnected components of the graph.
spqr_tree_to_graph()	Return the graph represented by the SPQR-tree T.
minimal_separators()	Return an iterator over the minimal separators of G.

5.46.1 Methods

class sage.graphs.connectivity.**TriconnectivitySPQR**

Bases: object

Decompose a graph into triconnected components and build SPQR-tree.

This class implements the algorithm proposed by Hopcroft and Tarjan in [Hopcroft1973], and later corrected by Gutwenger and Mutzel in [Gut2001], for finding the triconnected components of a biconnected graph. It then organizes these components into a SPQR-tree. See the:wikipedia:$SPQR_tree$.

A SPQR-tree is a tree data structure used to represent the triconnected components of a biconnected (multi)graph and the 2-vertex cuts separating them. A node of a SPQR-tree, and the graph associated with it, can be one of the following four types:

- 'S' – the associated graph is a cycle with at least three vertices 'S' stands for series and is also called a polygon
- 'P' – the associated graph is a dipole graph, a multigraph with two vertices and three or more edges. 'P' stands for parallel and the node is called a bond.
- 'Q' – the associated graph has a single real edge. This trivial case is necessary to handle the graph that has only one edge.
- 'R' – the associated graph is a 3-vertex-connected graph that is not a cycle or dipole. 'R' stands for rigid.

The edges of the tree indicate the 2-vertex cuts of the graph.

INPUT:

- G – graph; if G is a *DiGraph*, the computation is done on the underlying *Graph* (i.e., ignoring edge orientation)
- check – boolean (default: True); indicates whether G needs to be tested for biconnectivity

See also

- *sage.graphs.connectivity.spqr_tree()*
- *is_biconnected()*
- Wikipedia article SPQR_tree

EXAMPLES:

Example from the Wikipedia article SPQR_tree:

```
sage: from sage.graphs.connectivity import TriconnectivitySPQR
sage: from sage.graphs.connectivity import spqr_tree_to_graph
sage: G = Graph([(1, 2), (1, 4), (1, 8), (1, 12), (3, 4), (2, 3),
....: (2, 13), (3, 13), (4, 5), (4, 7), (5, 6), (5, 8), (5, 7), (6, 7),
```

(continues on next page)

(continued from previous page)

```
....: (8, 11), (8, 9), (8, 12), (9, 10), (9, 11), (9, 12), (10, 12)])
sage: tric = TriconnectivitySPQR(G)
sage: T = tric.get_spqr_tree()
sage: G.is_isomorphic(spqr_tree_to_graph(T))
True
```

```
>>> from sage.all import *
>>> from sage.graphs.connectivity import TriconnectivitySPQR
>>> from sage.graphs.connectivity import spqr_tree_to_graph
>>> G = Graph([(Integer(1), Integer(2)), (Integer(1), Integer(4)), (Integer(1),␣
↪Integer(8)), (Integer(1), Integer(12)), (Integer(3), Integer(4)), (Integer(2),␣
↪Integer(3)),
... (Integer(2), Integer(13)), (Integer(3), Integer(13)), (Integer(4),␣
↪Integer(5)), (Integer(4), Integer(7)), (Integer(5), Integer(6)), (Integer(5),␣
↪Integer(8)), (Integer(5), Integer(7)), (Integer(6), Integer(7)),
... (Integer(8), Integer(11)), (Integer(8), Integer(9)), (Integer(8),␣
↪Integer(12)), (Integer(9), Integer(10)), (Integer(9), Integer(11)), (Integer(9),
↪ Integer(12)), (Integer(10), Integer(12))])
>>> tric = TriconnectivitySPQR(G)
>>> T = tric.get_spqr_tree()
>>> G.is_isomorphic(spqr_tree_to_graph(T))
True
```

An example from [Hopcroft1973]:

```
sage: G = Graph([(1, 2), (1, 4), (1, 8), (1, 12), (1, 13), (2, 3),
....: (2, 13), (3, 4), (3, 13), (4, 5), (4, 7), (5, 6), (5, 7), (5, 8),
....: (6, 7), (8, 9), (8, 11), (8, 12), (9, 10), (9, 11), (9, 12),
....: (10, 11), (10, 12)])
sage: tric = TriconnectivitySPQR(G)
sage: T = tric.get_spqr_tree()
sage: G.is_isomorphic(spqr_tree_to_graph(T))
True
sage: tric.print_triconnected_components()
Triconnected: [(8, 9, None), (9, 12, None), (9, 11, None), (8, 11, None), (10, 11,
↪ None), (9, 10, None), (10, 12, None), (8, 12, 'newVEdge0')]
Bond: [(8, 12, None), (8, 12, 'newVEdge0'), (8, 12, 'newVEdge1')]
Polygon: [(6, 7, None), (5, 6, None), (7, 5, 'newVEdge2')]
Bond: [(7, 5, 'newVEdge2'), (5, 7, 'newVEdge3'), (5, 7, None)]
Polygon: [(5, 7, 'newVEdge3'), (4, 7, None), (5, 4, 'newVEdge4')]
Bond: [(5, 4, 'newVEdge4'), (4, 5, 'newVEdge5'), (4, 5, None)]
Polygon: [(4, 5, 'newVEdge5'), (5, 8, None), (1, 4, 'newVEdge9'), (1, 8,
↪'newVEdge10')]
Triconnected: [(1, 2, None), (2, 13, None), (1, 13, None), (3, 13, None), (2, 3,␣
↪None), (1, 3, 'newVEdge7')]
Polygon: [(1, 3, 'newVEdge7'), (3, 4, None), (1, 4, 'newVEdge8')]
Bond: [(1, 4, None), (1, 4, 'newVEdge8'), (1, 4, 'newVEdge9')]
Bond: [(1, 8, None), (1, 8, 'newVEdge10'), (1, 8, 'newVEdge11')]
Polygon: [(8, 12, 'newVEdge1'), (1, 8, 'newVEdge11'), (1, 12, None)]
```

```
>>> from sage.all import *
>>> G = Graph([(Integer(1), Integer(2)), (Integer(1), Integer(4)), (Integer(1),␣
```

(continues on next page)

(continued from previous page)

```
↪Integer(8)), (Integer(1), Integer(12)), (Integer(1), Integer(13)), (Integer(2),␣
↪Integer(3)),
... (Integer(2), Integer(13)), (Integer(3), Integer(4)), (Integer(3),␣
↪Integer(13)), (Integer(4), Integer(5)), (Integer(4), Integer(7)), (Integer(5),␣
↪Integer(6)), (Integer(5), Integer(7)), (Integer(5), Integer(8)),
... (Integer(6), Integer(7)), (Integer(8), Integer(9)), (Integer(8), Integer(11)),
↪ (Integer(8), Integer(12)), (Integer(9), Integer(10)), (Integer(9),␣
↪Integer(11)), (Integer(9), Integer(12)),
... (Integer(10), Integer(11)), (Integer(10), Integer(12))])
>>> tric = TriconnectivitySPQR(G)
>>> T = tric.get_spqr_tree()
>>> G.is_isomorphic(spqr_tree_to_graph(T))
True
>>> tric.print_triconnected_components()
Triconnected: [(8, 9, None), (9, 12, None), (9, 11, None), (8, 11, None), (10, 11,
↪ None), (9, 10, None), (10, 12, None), (8, 12, 'newVEdge0')]
Bond: [(8, 12, None), (8, 12, 'newVEdge0'), (8, 12, 'newVEdge1')]
Polygon: [(6, 7, None), (5, 6, None), (7, 5, 'newVEdge2')]
Bond: [(7, 5, 'newVEdge2'), (5, 7, 'newVEdge3'), (5, 7, None)]
Polygon: [(5, 7, 'newVEdge3'), (4, 7, None), (5, 4, 'newVEdge4')]
Bond: [(5, 4, 'newVEdge4'), (4, 5, 'newVEdge5'), (4, 5, None)]
Polygon: [(4, 5, 'newVEdge5'), (5, 8, None), (1, 4, 'newVEdge9'), (1, 8,
↪'newVEdge10')]
Triconnected: [(1, 2, None), (2, 13, None), (1, 13, None), (3, 13, None), (2, 3,␣
↪None), (1, 3, 'newVEdge7')]
Polygon: [(1, 3, 'newVEdge7'), (3, 4, None), (1, 4, 'newVEdge8')]
Bond: [(1, 4, None), (1, 4, 'newVEdge8'), (1, 4, 'newVEdge9')]
Bond: [(1, 8, None), (1, 8, 'newVEdge10'), (1, 8, 'newVEdge11')]
Polygon: [(8, 12, 'newVEdge1'), (1, 8, 'newVEdge11'), (1, 12, None)]
```

An example from [Gut2001]:

```
sage: G = Graph([(1, 2), (1, 4), (2, 3), (2, 5), (3, 4), (3, 5), (4, 5),
....: (4, 6), (5, 7), (5, 8), (5, 14), (6, 8), (7, 14), (8, 9), (8, 10),
....: (8, 11), (8, 12), (9, 10), (10, 13), (10, 14), (10, 15), (10, 16),
....: (11, 12), (11, 13), (12, 13), (14, 15), (14, 16), (15, 16)])
sage: T = TriconnectivitySPQR(G).get_spqr_tree()
sage: G.is_isomorphic(spqr_tree_to_graph(T))
True
```

```
>>> from sage.all import *
>>> G = Graph([(Integer(1), Integer(2)), (Integer(1), Integer(4)), (Integer(2),␣
↪Integer(3)), (Integer(2), Integer(5)), (Integer(3), Integer(4)), (Integer(3),␣
↪Integer(5)), (Integer(4), Integer(5)),
... (Integer(4), Integer(6)), (Integer(5), Integer(7)), (Integer(5), Integer(8)),␣
↪(Integer(5), Integer(14)), (Integer(6), Integer(8)), (Integer(7), Integer(14)),␣
↪(Integer(8), Integer(9)), (Integer(8), Integer(10)),
... (Integer(8), Integer(11)), (Integer(8), Integer(12)), (Integer(9),␣
↪Integer(10)), (Integer(10), Integer(13)), (Integer(10), Integer(14)),␣
↪(Integer(10), Integer(15)), (Integer(10), Integer(16)),
... (Integer(11), Integer(12)), (Integer(11), Integer(13)), (Integer(12),␣
↪Integer(13)), (Integer(14), Integer(15)), (Integer(14), Integer(16)),␣
```

(continues on next page)

(continued from previous page)

```
↪(Integer(15), Integer(16))])
>>> T = TriconnectivitySPQR(G).get_spqr_tree()
>>> G.is_isomorphic(spqr_tree_to_graph(T))
True
```

An example with multi-edges and accessing the triconnected components:

```
sage: G = Graph([(1, 2), (1, 5), (1, 5), (2, 3), (2, 3), (3, 4), (4, 5)],␣
↪multiedges=True)
sage: tric = TriconnectivitySPQR(G)
sage: T = tric.get_spqr_tree()
sage: G.is_isomorphic(spqr_tree_to_graph(T))
True
sage: tric.print_triconnected_components()
Bond:  [(1, 5, None), (1, 5, None), (1, 5, 'newVEdge0')]
Bond:  [(2, 3, None), (2, 3, None), (2, 3, 'newVEdge1')]
Polygon:  [(4, 5, None), (1, 5, 'newVEdge0'), (3, 4, None), (2, 3, 'newVEdge1'),␣
↪(1, 2, None)]
```

```
>>> from sage.all import *
>>> G = Graph([(Integer(1), Integer(2)), (Integer(1), Integer(5)), (Integer(1),␣
↪Integer(5)), (Integer(2), Integer(3)), (Integer(2), Integer(3)), (Integer(3),␣
↪Integer(4)), (Integer(4), Integer(5))], multiedges=True)
>>> tric = TriconnectivitySPQR(G)
>>> T = tric.get_spqr_tree()
>>> G.is_isomorphic(spqr_tree_to_graph(T))
True
>>> tric.print_triconnected_components()
Bond:  [(1, 5, None), (1, 5, None), (1, 5, 'newVEdge0')]
Bond:  [(2, 3, None), (2, 3, None), (2, 3, 'newVEdge1')]
Polygon:  [(4, 5, None), (1, 5, 'newVEdge0'), (3, 4, None), (2, 3, 'newVEdge1'),␣
↪(1, 2, None)]
```

An example of a triconnected graph:

```
sage: G = Graph([('a', 'b'), ('a', 'c'), ('a', 'd'), ('b', 'c'), ('b', 'd'), ('c',
↪ 'd')])
sage: T = TriconnectivitySPQR(G).get_spqr_tree()
sage: print(T.vertices(sort=True))
[('R', Multi-graph on 4 vertices)]
sage: G.is_isomorphic(spqr_tree_to_graph(T))
True
```

```
>>> from sage.all import *
>>> G = Graph([('a', 'b'), ('a', 'c'), ('a', 'd'), ('b', 'c'), ('b', 'd'), ('c',
↪'d')])
>>> T = TriconnectivitySPQR(G).get_spqr_tree()
>>> print(T.vertices(sort=True))
[('R', Multi-graph on 4 vertices)]
>>> G.is_isomorphic(spqr_tree_to_graph(T))
True
```

An example of a directed graph with multi-edges:

```
sage: G = DiGraph([(1, 2), (2, 3), (3, 4), (4, 5), (1, 5), (5, 1)])
sage: tric = TriconnectivitySPQR(G)
sage: tric.print_triconnected_components()
Bond:  [(1, 5, None), (5, 1, None), (1, 5, 'newVEdge0')]
Polygon:  [(4, 5, None), (1, 5, 'newVEdge0'), (3, 4, None), (2, 3, None), (1, 2,␣
↪None)]
```

```
>>> from sage.all import *
>>> G = DiGraph([(Integer(1), Integer(2)), (Integer(2), Integer(3)), (Integer(3),␣
↪Integer(4)), (Integer(4), Integer(5)), (Integer(1), Integer(5)), (Integer(5),␣
↪Integer(1))])
>>> tric = TriconnectivitySPQR(G)
>>> tric.print_triconnected_components()
Bond:  [(1, 5, None), (5, 1, None), (1, 5, 'newVEdge0')]
Polygon:  [(4, 5, None), (1, 5, 'newVEdge0'), (3, 4, None), (2, 3, None), (1, 2,␣
↪None)]
```

Edge labels are preserved by the construction:

```
sage: G = Graph([(0, 1, '01'), (0, 4, '04'), (1, 2, '12'), (1, 5, '15'),
....: (2, 3, '23'), (2, 6, '26'), (3, 7, '37'), (4, 5, '45'),
....: (5, 6, '56'), (6, 7, 67)])
sage: T = TriconnectivitySPQR(G).get_spqr_tree()
sage: H = spqr_tree_to_graph(T)
sage: all(G.has_edge(e) for e in H.edge_iterator())
True
sage: all(H.has_edge(e) for e in G.edge_iterator())
True
```

```
>>> from sage.all import *
>>> G = Graph([(Integer(0), Integer(1), '01'), (Integer(0), Integer(4), '04'),␣
↪(Integer(1), Integer(2), '12'), (Integer(1), Integer(5), '15'),
... (Integer(2), Integer(3), '23'), (Integer(2), Integer(6), '26'), (Integer(3),␣
↪Integer(7), '37'), (Integer(4), Integer(5), '45'),
... (Integer(5), Integer(6), '56'), (Integer(6), Integer(7), Integer(67))])
>>> T = TriconnectivitySPQR(G).get_spqr_tree()
>>> H = spqr_tree_to_graph(T)
>>> all(G.has_edge(e) for e in H.edge_iterator())
True
>>> all(H.has_edge(e) for e in G.edge_iterator())
True
```

get_spqr_tree()

Return an SPQR-tree representing the triconnected components of the graph.

An SPQR-tree is a tree data structure used to represent the triconnected components of a biconnected (multi)graph and the 2-vertex cuts separating them. A node of a SPQR-tree, and the graph associated with it, can be one of the following four types:

- `'S'` – the associated graph is a cycle with at least three vertices. `'S'` stands for `series`.
- `'P'` – the associated graph is a dipole graph, a multigraph with two vertices and three or more edges. `'P'` stands for `parallel`.

- `'Q'` – the associated graph has a single real edge. This trivial case is necessary to handle the graph that has only one edge.
- `'R'` – the associated graph is a 3-connected graph that is not a cycle or dipole. `'R'` stands for `rigid`.

The edges of the tree indicate the 2-vertex cuts of the graph.

OUTPUT:

`SPQR-tree` a tree whose vertices are labeled with the block's type and the subgraph of three-blocks in the decomposition.

EXAMPLES:

```
sage: from sage.graphs.connectivity import TriconnectivitySPQR
sage: G = Graph(2)
sage: for i in range(3):
....:     G.add_clique([0, 1, G.add_vertex(), G.add_vertex()])
sage: tric = TriconnectivitySPQR(G)
sage: Tree = tric.get_spqr_tree()
sage: K4 = graphs.CompleteGraph(4)
sage: all(u[1].is_isomorphic(K4) for u in Tree if u[0] == 'R')
True
sage: from sage.graphs.connectivity import spqr_tree_to_graph
sage: G.is_isomorphic(spqr_tree_to_graph(Tree))
True

sage: G = Graph(2)
sage: for i in range(3):
....:     G.add_path([0, G.add_vertex(), G.add_vertex(), 1])
sage: tric = TriconnectivitySPQR(G)
sage: Tree = tric.get_spqr_tree()
sage: C4 = graphs.CycleGraph(4)
sage: all(u[1].is_isomorphic(C4) for u in Tree if u[0] == 'S')
True
sage: G.is_isomorphic(spqr_tree_to_graph(Tree))
True

sage: G.allow_multiple_edges(True)
sage: G.add_edges(G.edge_iterator())
sage: tric = TriconnectivitySPQR(G)
sage: Tree = tric.get_spqr_tree()
sage: all(u[1].is_isomorphic(C4) for u in Tree if u[0] == 'S')
True
sage: G.is_isomorphic(spqr_tree_to_graph(Tree))
True

sage: G = graphs.CycleGraph(6)
sage: tric = TriconnectivitySPQR(G)
sage: Tree = tric.get_spqr_tree()
sage: Tree.order()
1
sage: G.is_isomorphic(spqr_tree_to_graph(Tree))
True
sage: G.add_edge(0, 3)
sage: tric = TriconnectivitySPQR(G)
```

(continues on next page)

(continued from previous page)

```
sage: Tree = tric.get_spqr_tree()
sage: Tree.order()
3
sage: G.is_isomorphic(spqr_tree_to_graph(Tree))
True

sage: G = Graph([(0, 1)], multiedges=True)
sage: tric = TriconnectivitySPQR(G)
sage: Tree = tric.get_spqr_tree()
sage: Tree.vertices(sort=True)
[('Q', Multi-graph on 2 vertices)]
sage: G.add_edge(0, 1)
sage: Tree = TriconnectivitySPQR(G).get_spqr_tree()
sage: Tree.vertices(sort=True)
[('P', Multi-graph on 2 vertices)]
```

```
>>> from sage.all import *
>>> from sage.graphs.connectivity import TriconnectivitySPQR
>>> G = Graph(Integer(2))
>>> for i in range(Integer(3)):
...     G.add_clique([Integer(0), Integer(1), G.add_vertex(), G.add_vertex()])
>>> tric = TriconnectivitySPQR(G)
>>> Tree = tric.get_spqr_tree()
>>> K4 = graphs.CompleteGraph(Integer(4))
>>> all(u[Integer(1)].is_isomorphic(K4) for u in Tree if u[Integer(0)] == 'R')
True
>>> from sage.graphs.connectivity import spqr_tree_to_graph
>>> G.is_isomorphic(spqr_tree_to_graph(Tree))
True

>>> G = Graph(Integer(2))
>>> for i in range(Integer(3)):
...     G.add_path([Integer(0), G.add_vertex(), G.add_vertex(), Integer(1)])
>>> tric = TriconnectivitySPQR(G)
>>> Tree = tric.get_spqr_tree()
>>> C4 = graphs.CycleGraph(Integer(4))
>>> all(u[Integer(1)].is_isomorphic(C4) for u in Tree if u[Integer(0)] == 'S')
True
>>> G.is_isomorphic(spqr_tree_to_graph(Tree))
True

>>> G.allow_multiple_edges(True)
>>> G.add_edges(G.edge_iterator())
>>> tric = TriconnectivitySPQR(G)
>>> Tree = tric.get_spqr_tree()
>>> all(u[Integer(1)].is_isomorphic(C4) for u in Tree if u[Integer(0)] == 'S')
True
>>> G.is_isomorphic(spqr_tree_to_graph(Tree))
True

>>> G = graphs.CycleGraph(Integer(6))
>>> tric = TriconnectivitySPQR(G)
```

(continues on next page)

(continued from previous page)

```
>>> Tree = tric.get_spqr_tree()
>>> Tree.order()
1
>>> G.is_isomorphic(spqr_tree_to_graph(Tree))
True
>>> G.add_edge(Integer(0), Integer(3))
>>> tric = TriconnectivitySPQR(G)
>>> Tree = tric.get_spqr_tree()
>>> Tree.order()
3
>>> G.is_isomorphic(spqr_tree_to_graph(Tree))
True

>>> G = Graph([(Integer(0), Integer(1))], multiedges=True)
>>> tric = TriconnectivitySPQR(G)
>>> Tree = tric.get_spqr_tree()
>>> Tree.vertices(sort=True)
[('Q', Multi-graph on 2 vertices)]
>>> G.add_edge(Integer(0), Integer(1))
>>> Tree = TriconnectivitySPQR(G).get_spqr_tree()
>>> Tree.vertices(sort=True)
[('P', Multi-graph on 2 vertices)]
```

get_triconnected_components()

Return the triconnected components as a list of tuples.

Each component is represented as a tuple of the type of the component and the list of edges of the component.

EXAMPLES:

```
sage: from sage.graphs.connectivity import TriconnectivitySPQR
sage: G = Graph(2)
sage: for i in range(3):
....:     G.add_path([0, G.add_vertex(), G.add_vertex(), 1])
sage: tric = TriconnectivitySPQR(G)
sage: tric.get_triconnected_components()
[('Polygon', [(4, 5, None), (0, 4, None), (1, 5, None), (1, 0, 'newVEdge1')]),
('Polygon', [(6, 7, None), (0, 6, None), (1, 7, None), (1, 0, 'newVEdge3')]),
('Bond', [(1, 0, 'newVEdge1'), (1, 0, 'newVEdge3'), (1, 0, 'newVEdge4')]),
('Polygon', [(1, 3, None), (1, 0, 'newVEdge4'), (2, 3, None), (0, 2, None)])]
```

```
>>> from sage.all import *
>>> from sage.graphs.connectivity import TriconnectivitySPQR
>>> G = Graph(Integer(2))
>>> for i in range(Integer(3)):
...     G.add_path([Integer(0), G.add_vertex(), G.add_vertex(), Integer(1)])
>>> tric = TriconnectivitySPQR(G)
>>> tric.get_triconnected_components()
[('Polygon', [(4, 5, None), (0, 4, None), (1, 5, None), (1, 0, 'newVEdge1')]),
('Polygon', [(6, 7, None), (0, 6, None), (1, 7, None), (1, 0, 'newVEdge3')]),
('Bond', [(1, 0, 'newVEdge1'), (1, 0, 'newVEdge3'), (1, 0, 'newVEdge4')]),
('Polygon', [(1, 3, None), (1, 0, 'newVEdge4'), (2, 3, None), (0, 2, None)])]
```

print_triconnected_components()

Print the type and list of edges of each component.

EXAMPLES:

An example from [Hopcroft1973]:

```
sage: from sage.graphs.connectivity import TriconnectivitySPQR
sage: from sage.graphs.connectivity import spqr_tree_to_graph
sage: G = Graph([(1, 2), (1, 4), (1, 8), (1, 12), (1, 13), (2, 3),
....: (2, 13), (3, 4), (3, 13), (4, 5), (4, 7), (5, 6), (5, 7), (5, 8),
....: (6, 7), (8, 9), (8, 11), (8, 12), (9, 10), (9, 11), (9, 12),
....: (10, 11), (10, 12)])
sage: tric = TriconnectivitySPQR(G)
sage: T = tric.get_spqr_tree()
sage: G.is_isomorphic(spqr_tree_to_graph(T))
True
sage: tric.print_triconnected_components()
Triconnected: [(8, 9, None), (9, 12, None), (9, 11, None), (8, 11, None), (10,
↪ 11, None), (9, 10, None), (10, 12, None), (8, 12, 'newVEdge0')]
Bond: [(8, 12, None), (8, 12, 'newVEdge0'), (8, 12, 'newVEdge1')]
Polygon: [(6, 7, None), (5, 6, None), (7, 5, 'newVEdge2')]
Bond: [(7, 5, 'newVEdge2'), (5, 7, 'newVEdge3'), (5, 7, None)]
Polygon: [(5, 7, 'newVEdge3'), (4, 7, None), (5, 4, 'newVEdge4')]
Bond: [(5, 4, 'newVEdge4'), (4, 5, 'newVEdge5'), (4, 5, None)]
Polygon: [(4, 5, 'newVEdge5'), (5, 8, None), (1, 4, 'newVEdge9'), (1, 8,
↪'newVEdge10')]
Triconnected: [(1, 2, None), (2, 13, None), (1, 13, None), (3, 13, None), (2,␣
↪3, None), (1, 3, 'newVEdge7')]
Polygon: [(1, 3, 'newVEdge7'), (3, 4, None), (1, 4, 'newVEdge8')]
Bond: [(1, 4, None), (1, 4, 'newVEdge8'), (1, 4, 'newVEdge9')]
Bond: [(1, 8, None), (1, 8, 'newVEdge10'), (1, 8, 'newVEdge11')]
Polygon: [(8, 12, 'newVEdge1'), (1, 8, 'newVEdge11'), (1, 12, None)]
```

```
>>> from sage.all import *
>>> from sage.graphs.connectivity import TriconnectivitySPQR
>>> from sage.graphs.connectivity import spqr_tree_to_graph
>>> G = Graph([(Integer(1), Integer(2)), (Integer(1), Integer(4)),␣
↪(Integer(1), Integer(8)), (Integer(1), Integer(12)), (Integer(1),␣
↪Integer(13)), (Integer(2), Integer(3)),
... (Integer(2), Integer(13)), (Integer(3), Integer(4)), (Integer(3),␣
↪Integer(13)), (Integer(4), Integer(5)), (Integer(4), Integer(7)),␣
↪(Integer(5), Integer(6)), (Integer(5), Integer(7)), (Integer(5),␣
↪Integer(8)),
... (Integer(6), Integer(7)), (Integer(8), Integer(9)), (Integer(8),␣
↪Integer(11)), (Integer(8), Integer(12)), (Integer(9), Integer(10)),␣
↪(Integer(9), Integer(11)), (Integer(9), Integer(12)),
... (Integer(10), Integer(11)), (Integer(10), Integer(12))])
>>> tric = TriconnectivitySPQR(G)
>>> T = tric.get_spqr_tree()
>>> G.is_isomorphic(spqr_tree_to_graph(T))
True
>>> tric.print_triconnected_components()
Triconnected: [(8, 9, None), (9, 12, None), (9, 11, None), (8, 11, None), (10,
```

(continues on next page)

(continued from previous page)

```
↪ 11, None), (9, 10, None), (10, 12, None), (8, 12, 'newVEdge0')]
Bond: [(8, 12, None), (8, 12, 'newVEdge0'), (8, 12, 'newVEdge1')]
Polygon: [(6, 7, None), (5, 6, None), (7, 5, 'newVEdge2')]
Bond: [(7, 5, 'newVEdge2'), (5, 7, 'newVEdge3'), (5, 7, None)]
Polygon: [(5, 7, 'newVEdge3'), (4, 7, None), (5, 4, 'newVEdge4')]
Bond: [(5, 4, 'newVEdge4'), (4, 5, 'newVEdge5'), (4, 5, None)]
Polygon: [(4, 5, 'newVEdge5'), (5, 8, None), (1, 4, 'newVEdge9'), (1, 8,
↪'newVEdge10')]
Triconnected: [(1, 2, None), (2, 13, None), (1, 13, None), (3, 13, None), (2,␣
↪3, None), (1, 3, 'newVEdge7')]
Polygon: [(1, 3, 'newVEdge7'), (3, 4, None), (1, 4, 'newVEdge8')]
Bond: [(1, 4, None), (1, 4, 'newVEdge8'), (1, 4, 'newVEdge9')]
Bond: [(1, 8, None), (1, 8, 'newVEdge10'), (1, 8, 'newVEdge11')]
Polygon: [(8, 12, 'newVEdge1'), (1, 8, 'newVEdge11'), (1, 12, None)]
```

sage.graphs.connectivity.**biconnected_components**(*G*, *forbidden_vertices=None*)

Return the list of biconnected components.

A biconnected component is a maximal subgraph on two or more vertices that is biconnected, i.e., removing any vertex does not disconnect it.

INPUT:

- `G` – the input graph
- `forbidden_vertices` – list (default: `None`); set of vertices to avoid during the search. This is equivalent to getting the biconnected components of the graph after the removal of the forbidden vertices.

EXAMPLES:

```
sage: from sage.graphs.connectivity import biconnected_components
sage: G = Graph({0: [1, 2], 1: [0, 2], 2: [0, 1, 3], 3: [2]})
sage: sorted(len(b) for b in biconnected_components(G))
[2, 3]
sage: sorted(len(b) for b in biconnected_components(2 * G))
[2, 2, 3, 3]
```

```
>>> from sage.all import *
>>> from sage.graphs.connectivity import biconnected_components
>>> G = Graph({Integer(0): [Integer(1), Integer(2)], Integer(1): [Integer(0),␣
↪Integer(2)], Integer(2): [Integer(0), Integer(1), Integer(3)], Integer(3):␣
↪[Integer(2)]})
>>> sorted(len(b) for b in biconnected_components(G))
[2, 3]
>>> sorted(len(b) for b in biconnected_components(Integer(2) * G))
[2, 2, 3, 3]
```

Check the behavior of parameter `forbidden_vertices`:

```
sage: G = graphs.CycleGraph(5)
sage: len(G.biconnected_components())
1
sage: len(G.biconnected_components(forbidden_vertices=[0]))
3
```

```
>>> from sage.all import *
>>> G = graphs.CycleGraph(Integer(5))
>>> len(G.biconnected_components())
1
>>> len(G.biconnected_components(forbidden_vertices=[Integer(0)]))
3
```

sage.graphs.connectivity.**biconnected_components_subgraphs**(*G*, *forbidden_vertices=None*, *immutable=None*)

Return a list of biconnected components as graph objects.

A biconnected component is a maximal subgraph on two or more vertices that is biconnected, i.e., removing any vertex does not disconnect it.

INPUT:

- `G` – the input graph
- `forbidden_vertices` – list (default: `None`); set of vertices to avoid during the search. This is equivalent to getting the biconnected components subgraphs of the graph after the removal of the forbidden vertices.
- `immutable` – boolean (default: `None`); whether to create a mutable/immutable graphs. `immutable=None` (default) means that the graph and the subgraphs of its connected components will behave the same way.

EXAMPLES:

```
sage: from sage.graphs.connectivity import biconnected_components_subgraphs
sage: G = Graph({0: [1, 2], 1: [0, 2], 2: [0, 1, 3], 3: [2]})
sage: L = biconnected_components_subgraphs(G)
sage: L
[Subgraph of (): Graph on 2 vertices, Subgraph of (): Graph on 3 vertices]
sage: L[0].edges()
[(2, 3, None)]
sage: L[1].edges()
[(0, 1, None), (0, 2, None), (1, 2, None)]
```

```
>>> from sage.all import *
>>> from sage.graphs.connectivity import biconnected_components_subgraphs
>>> G = Graph({Integer(0): [Integer(1), Integer(2)], Integer(1): [Integer(0),␣
↪Integer(2)], Integer(2): [Integer(0), Integer(1), Integer(3)], Integer(3):␣
↪[Integer(2)]})
>>> L = biconnected_components_subgraphs(G)
>>> L
[Subgraph of (): Graph on 2 vertices, Subgraph of (): Graph on 3 vertices]
>>> L[Integer(0)].edges()
[(2, 3, None)]
>>> L[Integer(1)].edges()
[(0, 1, None), (0, 2, None), (1, 2, None)]
```

Check the behavior of parameter `forbidden_vertices`:

```
sage: G = graphs.CycleGraph(5)
sage: len(G.biconnected_components_subgraphs())
1
sage: len(G.biconnected_components_subgraphs(forbidden_vertices=[0]))
3
```

```
>>> from sage.all import *
>>> G = graphs.CycleGraph(Integer(5))
>>> len(G.biconnected_components_subgraphs())
1
>>> len(G.biconnected_components_subgraphs(forbidden_vertices=[Integer(0)]))
3
```

sage.graphs.connectivity.**blocks_and_cut_vertices**(*G*, *algorithm='Tarjan_Boost'*, *sort=False*, *key=None*, *forbidden_vertices=None*)

Return the blocks and cut vertices of the graph.

In the case of a digraph, this computation is done on the underlying graph.

A cut vertex is one whose deletion increases the number of connected components. A block is a maximal induced subgraph which itself has no cut vertices. Two distinct blocks cannot overlap in more than a single cut vertex.

INPUT:

- `algorithm` – string (default: `'Tarjan_Boost'`); the algorithm to use among:
  - `'Tarjan_Boost'` – default; Tarjan's algorithm (Boost implementation)
  - `'Tarjan_Sage'` – Tarjan's algorithm (Sage implementation)
- `sort` – boolean (default: `False`); whether to sort vertices inside the components and the list of cut vertices **currently only available for ```'Tarjan_Sage'```**
- `key` – a function (default: `None`); a function that takes a vertex as its one argument and returns a value that can be used for comparisons in the sorting algorithm (we must have `sort=True`)
- `forbidden_vertices` – list (default: `None`); set of vertices to avoid during the search

OUTPUT: `(B, C)`, where `B` is a list of blocks - each is a list of vertices and the blocks are the corresponding induced subgraphs - and `C` is a list of cut vertices.

ALGORITHM:

We implement the algorithm proposed by Tarjan in [Tarjan72]. The original version is recursive. We emulate the recursion using a stack.

See also

- *blocks_and_cuts_tree()*
- *sage.graphs.base.boost_graph.blocks_and_cut_vertices()*
- *is_biconnected()*
- *bridges()*

EXAMPLES:

We construct a trivial example of a graph with one cut vertex:

```
sage: from sage.graphs.connectivity import blocks_and_cut_vertices
sage: rings = graphs.CycleGraph(10)
sage: rings.merge_vertices([0, 5])
sage: blocks_and_cut_vertices(rings)
([[0, 1, 4, 2, 3], [0, 6, 9, 7, 8]], [0])
```

(continues on next page)

(continued from previous page)

```
sage: rings.blocks_and_cut_vertices()
([[0, 1, 4, 2, 3], [0, 6, 9, 7, 8]], [0])
sage: B, C = blocks_and_cut_vertices(rings, algorithm='Tarjan_Sage', sort=True)
sage: B, C
([[0, 1, 2, 3, 4], [0, 6, 7, 8, 9]], [0])
sage: B2, C2 = blocks_and_cut_vertices(rings, algorithm='Tarjan_Sage', sort=False)
sage: Set(map(Set, B)) == Set(map(Set, B2)) and set(C) == set(C2)
True
```

```
>>> from sage.all import *
>>> from sage.graphs.connectivity import blocks_and_cut_vertices
>>> rings = graphs.CycleGraph(Integer(10))
>>> rings.merge_vertices([Integer(0), Integer(5)])
>>> blocks_and_cut_vertices(rings)
([[0, 1, 4, 2, 3], [0, 6, 9, 7, 8]], [0])
>>> rings.blocks_and_cut_vertices()
([[0, 1, 4, 2, 3], [0, 6, 9, 7, 8]], [0])
>>> B, C = blocks_and_cut_vertices(rings, algorithm='Tarjan_Sage', sort=True)
>>> B, C
([[0, 1, 2, 3, 4], [0, 6, 7, 8, 9]], [0])
>>> B2, C2 = blocks_and_cut_vertices(rings, algorithm='Tarjan_Sage', sort=False)
>>> Set(map(Set, B)) == Set(map(Set, B2)) and set(C) == set(C2)
True
```

The Petersen graph is biconnected, hence has no cut vertices:

```
sage: blocks_and_cut_vertices(graphs.PetersenGraph())
([[0, 1, 4, 5, 2, 6, 3, 7, 8, 9]], [])
```

```
>>> from sage.all import *
>>> blocks_and_cut_vertices(graphs.PetersenGraph())
([[0, 1, 4, 5, 2, 6, 3, 7, 8, 9]], [])
```

Decomposing paths to pairs:

```
sage: g = graphs.PathGraph(4) + graphs.PathGraph(5)
sage: blocks_and_cut_vertices(g)
([[2, 3], [1, 2], [0, 1], [7, 8], [6, 7], [5, 6], [4, 5]], [1, 2, 5, 6, 7])
```

```
>>> from sage.all import *
>>> g = graphs.PathGraph(Integer(4)) + graphs.PathGraph(Integer(5))
>>> blocks_and_cut_vertices(g)
([[2, 3], [1, 2], [0, 1], [7, 8], [6, 7], [5, 6], [4, 5]], [1, 2, 5, 6, 7])
```

A disconnected graph:

```
sage: g = Graph({1: {2: 28, 3: 10}, 2: {1: 10, 3: 16}, 4: {}, 5: {6: 3, 7: 10, 8:␣
↪4}})
sage: blocks_and_cut_vertices(g)
([[1, 2, 3], [5, 6], [5, 7], [5, 8], [4]], [5])
```

```
>>> from sage.all import *
>>> g = Graph({Integer(1): {Integer(2): Integer(28), Integer(3): Integer(10)},␣
↪Integer(2): {Integer(1): Integer(10), Integer(3): Integer(16)}, Integer(4): {},␣
↪Integer(5): {Integer(6): Integer(3), Integer(7): Integer(10), Integer(8):␣
↪Integer(4)}})
>>> blocks_and_cut_vertices(g)
([[1, 2, 3], [5, 6], [5, 7], [5, 8], [4]], [5])
```

A directed graph with Boost's algorithm (Issue #25994):

```
sage: rings = graphs.CycleGraph(10)
sage: rings.merge_vertices([0, 5])
sage: rings = rings.to_directed()
sage: blocks_and_cut_vertices(rings, algorithm='Tarjan_Boost')
([[0, 1, 4, 2, 3], [0, 6, 9, 7, 8]], [0])
```

```
>>> from sage.all import *
>>> rings = graphs.CycleGraph(Integer(10))
>>> rings.merge_vertices([Integer(0), Integer(5)])
>>> rings = rings.to_directed()
>>> blocks_and_cut_vertices(rings, algorithm='Tarjan_Boost')
([[0, 1, 4, 2, 3], [0, 6, 9, 7, 8]], [0])
```

Check the behavior of parameter `forbidden_vertices`:

```
sage: G = graphs.WindmillGraph(4, 3)
sage: G.blocks_and_cut_vertices(sort=True)
([[0, 1, 2, 3], [0, 4, 5, 6], [0, 7, 8, 9]], [0])
sage: G.blocks_and_cut_vertices(sort=True, algorithm='Tarjan_Boost', forbidden_
↪vertices=[0])
([[1, 2, 3], [4, 5, 6], [7, 8, 9]], [])
sage: G.blocks_and_cut_vertices(sort=True, algorithm='Tarjan_Sage', forbidden_
↪vertices=[0])
([[1, 2, 3], [4, 5, 6], [7, 8, 9]], [])
sage: G.blocks_and_cut_vertices(sort=True, algorithm='Tarjan_Boost', forbidden_
↪vertices=[1])
([[0, 2, 3], [0, 4, 5, 6], [0, 7, 8, 9]], [0])
sage: G.blocks_and_cut_vertices(sort=True, algorithm='Tarjan_Sage', forbidden_
↪vertices=[1])
([[0, 2, 3], [0, 4, 5, 6], [0, 7, 8, 9]], [0])
sage: G = graphs.PathGraph(3)
sage: G.blocks_and_cut_vertices(sort=True, algorithm='Tarjan_Sage')
([[1, 2], [0, 1]], [1])
sage: G.blocks_and_cut_vertices(sort=True, algorithm='Tarjan_Sage', forbidden_
↪vertices=[0])
([[1, 2]], [])
sage: G.blocks_and_cut_vertices(sort=True, algorithm='Tarjan_Sage', forbidden_
↪vertices=[1])
([[0], [2]], [])
sage: G.blocks_and_cut_vertices(sort=True, algorithm='Tarjan_Boost', forbidden_
↪vertices=[1])
([[0], [2]], [])
```

```
>>> from sage.all import *
>>> G = graphs.WindmillGraph(Integer(4), Integer(3))
>>> G.blocks_and_cut_vertices(sort=True)
([[0, 1, 2, 3], [0, 4, 5, 6], [0, 7, 8, 9]], [0])
>>> G.blocks_and_cut_vertices(sort=True, algorithm='Tarjan_Boost', forbidden_
↪vertices=[Integer(0)])
([[1, 2, 3], [4, 5, 6], [7, 8, 9]], [])
>>> G.blocks_and_cut_vertices(sort=True, algorithm='Tarjan_Sage', forbidden_
↪vertices=[Integer(0)])
([[1, 2, 3], [4, 5, 6], [7, 8, 9]], [])
>>> G.blocks_and_cut_vertices(sort=True, algorithm='Tarjan_Boost', forbidden_
↪vertices=[Integer(1)])
([[0, 2, 3], [0, 4, 5, 6], [0, 7, 8, 9]], [0])
>>> G.blocks_and_cut_vertices(sort=True, algorithm='Tarjan_Sage', forbidden_
↪vertices=[Integer(1)])
([[0, 2, 3], [0, 4, 5, 6], [0, 7, 8, 9]], [0])
>>> G = graphs.PathGraph(Integer(3))
>>> G.blocks_and_cut_vertices(sort=True, algorithm='Tarjan_Sage')
([[1, 2], [0, 1]], [1])
>>> G.blocks_and_cut_vertices(sort=True, algorithm='Tarjan_Sage', forbidden_
↪vertices=[Integer(0)])
([[1, 2]], [])
>>> G.blocks_and_cut_vertices(sort=True, algorithm='Tarjan_Sage', forbidden_
↪vertices=[Integer(1)])
([[0], [2]], [])
>>> G.blocks_and_cut_vertices(sort=True, algorithm='Tarjan_Boost', forbidden_
↪vertices=[Integer(1)])
([[0], [2]], [])
```

sage.graphs.connectivity.**blocks_and_cuts_tree**(*G*, *forbidden_vertices=None*)

Return the blocks-and-cuts tree of `self`.

This new graph has two different kinds of vertices, some representing the blocks (type B) and some other the cut vertices of the graph (type C).

There is an edge between a vertex u of type B and a vertex v of type C if the cut-vertex corresponding to v is in the block corresponding to u.

The resulting graph is a tree, with the additional characteristic property that the distance between two leaves is even. When `self` is not connected, the resulting graph is a forest.

When `self` is biconnected, the tree is reduced to a single node of type B.

We referred to [HarPri] and [Gallai] for blocks and cuts tree.

INPUT:

- `G` – a Sage graph
- `forbidden_vertices` – list (default: `None`); set of vertices to avoid during the search. This is equivalent to getting the blocks and cut tree of a graph in which the forbidden vertices have been removed.

See also

- *blocks_and_cut_vertices()*

- *is_biconnected()*

EXAMPLES:

```
sage: from sage.graphs.connectivity import blocks_and_cuts_tree
sage: T = blocks_and_cuts_tree(graphs.KrackhardtKiteGraph()); T
Graph on 5 vertices
sage: T.is_isomorphic(graphs.PathGraph(5))
True
sage: from sage.graphs.connectivity import blocks_and_cuts_tree
sage: T = graphs.KrackhardtKiteGraph().blocks_and_cuts_tree(); T
Graph on 5 vertices
```

```
>>> from sage.all import *
>>> from sage.graphs.connectivity import blocks_and_cuts_tree
>>> T = blocks_and_cuts_tree(graphs.KrackhardtKiteGraph()); T
Graph on 5 vertices
>>> T.is_isomorphic(graphs.PathGraph(Integer(5)))
True
>>> from sage.graphs.connectivity import blocks_and_cuts_tree
>>> T = graphs.KrackhardtKiteGraph().blocks_and_cuts_tree(); T
Graph on 5 vertices
```

The distance between two leaves is even:

```
sage: T = blocks_and_cuts_tree(graphs.RandomTree(40))
sage: T.is_tree()
True
sage: leaves = [v for v in T if T.degree(v) == 1]
sage: all(T.distance(u,v) % 2 == 0 for u in leaves for v in leaves)
True
```

```
>>> from sage.all import *
>>> T = blocks_and_cuts_tree(graphs.RandomTree(Integer(40)))
>>> T.is_tree()
True
>>> leaves = [v for v in T if T.degree(v) == Integer(1)]
>>> all(T.distance(u,v) % Integer(2) == Integer(0) for u in leaves for v in␣
↪leaves)
True
```

The tree of a biconnected graph has a single vertex, of type B:

```
sage: T = blocks_and_cuts_tree(graphs.PetersenGraph())
sage: T.vertices(sort=True)
[('B', (0, 1, 4, 5, 2, 6, 3, 7, 8, 9))]
```

```
>>> from sage.all import *
>>> T = blocks_and_cuts_tree(graphs.PetersenGraph())
>>> T.vertices(sort=True)
[('B', (0, 1, 4, 5, 2, 6, 3, 7, 8, 9))]
```

Check the behavior of parameter `forbidden_vertices`:

```
sage: G = graphs.CycleGraph(5)
sage: G.blocks_and_cut_vertices()
([[0, 1, 4, 2, 3]], [])
sage: G.blocks_and_cuts_tree()
Graph on 1 vertex
sage: G.blocks_and_cut_vertices(forbidden_vertices=[0])
([[3, 4], [2, 3], [1, 2]], [2, 3])
sage: G.blocks_and_cuts_tree(forbidden_vertices=[0])
Graph on 5 vertices
```

```
>>> from sage.all import *
>>> G = graphs.CycleGraph(Integer(5))
>>> G.blocks_and_cut_vertices()
([[0, 1, 4, 2, 3]], [])
>>> G.blocks_and_cuts_tree()
Graph on 1 vertex
>>> G.blocks_and_cut_vertices(forbidden_vertices=[Integer(0)])
([[3, 4], [2, 3], [1, 2]], [2, 3])
>>> G.blocks_and_cuts_tree(forbidden_vertices=[Integer(0)])
Graph on 5 vertices
```

sage.graphs.connectivity.**bridges**(*G*, *labels=True*)

Return an iterator over the bridges (or cut edges).

A bridge is an edge whose deletion disconnects the undirected graph. A disconnected graph has no bridge.

INPUT:

- `labels` – boolean (default: `True`); if `False`, each bridge is a tuple (u, v) of vertices

EXAMPLES:

```
sage: from sage.graphs.connectivity import bridges
sage: from sage.graphs.connectivity import is_connected
sage: g = 2 * graphs.PetersenGraph()
sage: g.add_edge(1, 10)
sage: is_connected(g)
True
sage: list(bridges(g))
[(1, 10, None)]
sage: list(g.bridges())
[(1, 10, None)]
```

```
>>> from sage.all import *
>>> from sage.graphs.connectivity import bridges
>>> from sage.graphs.connectivity import is_connected
>>> g = Integer(2) * graphs.PetersenGraph()
>>> g.add_edge(Integer(1), Integer(10))
>>> is_connected(g)
True
>>> list(bridges(g))
[(1, 10, None)]
>>> list(g.bridges())
[(1, 10, None)]
```

Every edge of a tree is a bridge:

```
sage: g = graphs.RandomTree(100)
sage: sum(1 for _ in g.bridges()) == 99
True
```

```
>>> from sage.all import *
>>> g = graphs.RandomTree(Integer(100))
>>> sum(Integer(1) for _ in g.bridges()) == Integer(99)
True
```

sage.graphs.connectivity.**cleave**(*G*, *cut_vertices=None*, *virtual_edges=True*, *solver=None*, *verbose=0*, *integrality_tolerance=0.001*)

Return the connected subgraphs separated by the input vertex cut.

Given a connected (multi)graph G and a vertex cut X, this method computes the list of subgraphs of G induced by each connected component c of $G \setminus X$ plus X, i.e., $G[c \cup X]$.

INPUT:

- `G` – a Graph
- `cut_vertices` – iterable container of vertices (default: `None`); a set of vertices representing a vertex cut of `G`. If no vertex cut is given, the method will compute one via a call to *vertex_connectivity()*.
- `virtual_edges` – boolean (default: `True`); whether to add virtual edges to the sides of the cut or not. A virtual edge is an edge between a pair of vertices of the cut that are not connected by an edge in `G`.
- `solver` – string (default: `None`); specifies a Mixed Integer Linear Programming (MILP) solver to be used. If set to `None`, the default one is used. For more information on MILP solvers and which default solver is used, see the method `solve` of the class `MixedIntegerLinearProgram`.
- `verbose` – integer (default: 0); sets the level of verbosity. Set to 0 by default, which means quiet.
- `integrality_tolerance` – float; parameter for use with MILP solvers over an inexact base ring; see `MixedIntegerLinearProgram.get_values()`.

OUTPUT: a triple (S, C, f), where

- S is a list of the graphs that are sides of the vertex cut.
- C is the graph of the cocycles. For each pair of vertices of the cut, if there exists an edge between them, C has one copy of each edge connecting them in `G` per sides of the cut plus one extra copy. Furthermore, when `virtual_edges == True`, if a pair of vertices of the cut is not connected by an edge in `G`, then it has one virtual edge between them per sides of the cut.
- f is the complement of the subgraph of `G` induced by the vertex cut. Hence, its vertex set is the vertex cut, and its edge set is the set of virtual edges (i.e., edges between pairs of vertices of the cut that are not connected by an edge in `G`). When `virtual_edges == False`, the edge set is empty.

EXAMPLES:

If there is an edge between cut vertices:

```
sage: from sage.graphs.connectivity import cleave
sage: G = Graph(2)
sage: for _ in range(3):
....:     G.add_clique([0, 1, G.add_vertex(), G.add_vertex()])
sage: S1,C1,f1 = cleave(G, cut_vertices=[0, 1])
sage: [g.order() for g in S1]
```

(continues on next page)

(continued from previous page)

```
[4, 4, 4]
sage: C1.order(), C1.size()
(2, 4)
sage: f1.vertices(sort=True), f1.edges(sort=True)
([0, 1], [])
```

```
>>> from sage.all import *
>>> from sage.graphs.connectivity import cleave
>>> G = Graph(Integer(2))
>>> for _ in range(Integer(3)):
...     G.add_clique([Integer(0), Integer(1), G.add_vertex(), G.add_vertex()])
>>> S1,C1,f1 = cleave(G, cut_vertices=[Integer(0), Integer(1)])
>>> [g.order() for g in S1]
[4, 4, 4]
>>> C1.order(), C1.size()
(2, 4)
>>> f1.vertices(sort=True), f1.edges(sort=True)
([0, 1], [])
```

If `virtual_edges == False` and there is an edge between cut vertices:

```
sage: G.subgraph([0, 1]).complement() == Graph([[0, 1], []])
True
sage: S2,C2,f2 = cleave(G, cut_vertices=[0, 1], virtual_edges=False)
sage: (S1 == S2, C1 == C2, f1 == f2)
(True, True, True)
```

```
>>> from sage.all import *
>>> G.subgraph([Integer(0), Integer(1)]).complement() == Graph([[Integer(0),␣
↪Integer(1)], []])
True
>>> S2,C2,f2 = cleave(G, cut_vertices=[Integer(0), Integer(1)], virtual_
↪edges=False)
>>> (S1 == S2, C1 == C2, f1 == f2)
(True, True, True)
```

If cut vertices doesn't have edge between them:

```
sage: G.delete_edge(0, 1)
sage: S1,C1,f1 = cleave(G, cut_vertices=[0, 1])
sage: [g.order() for g in S1]
[4, 4, 4]
sage: C1.order(), C1.size()
(2, 3)
sage: f1.vertices(sort=True), f1.edges(sort=True)
([0, 1], [(0, 1, None)])
```

```
>>> from sage.all import *
>>> G.delete_edge(Integer(0), Integer(1))
>>> S1,C1,f1 = cleave(G, cut_vertices=[Integer(0), Integer(1)])
>>> [g.order() for g in S1]
[4, 4, 4]
```

(continues on next page)

(continued from previous page)

```
>>> C1.order(), C1.size()
(2, 3)
>>> f1.vertices(sort=True), f1.edges(sort=True)
([0, 1], [(0, 1, None)])
```

If `virtual_edges == False` and the cut vertices are not connected by an edge:

```
sage: G.subgraph([0, 1]).complement() == Graph([[0, 1], []])
False
sage: S2,C2,f2 = cleave(G, cut_vertices=[0, 1], virtual_edges=False)
sage: [g.order() for g in S2]
[4, 4, 4]
sage: C2.order(), C2.size()
(2, 0)
sage: f2.vertices(sort=True), f2.edges(sort=True)
([0, 1], [])
sage: (S1 == S2, C1 == C2, f1 == f2)
(False, False, False)
```

```
>>> from sage.all import *
>>> G.subgraph([Integer(0), Integer(1)]).complement() == Graph([[Integer(0),␣
↪Integer(1)], []])
False
>>> S2,C2,f2 = cleave(G, cut_vertices=[Integer(0), Integer(1)], virtual_
↪edges=False)
>>> [g.order() for g in S2]
[4, 4, 4]
>>> C2.order(), C2.size()
(2, 0)
>>> f2.vertices(sort=True), f2.edges(sort=True)
([0, 1], [])
>>> (S1 == S2, C1 == C2, f1 == f2)
(False, False, False)
```

If G is a biconnected multigraph:

```
sage: G = graphs.CompleteBipartiteGraph(2, 3)
sage: G.add_edge(2, 3)
sage: G.allow_multiple_edges(True)
sage: G.add_edges(G.edge_iterator())
sage: G.add_edges([(0, 1), (0, 1), (0, 1)])
sage: S,C,f = cleave(G, cut_vertices=[0, 1])
sage: for g in S:
....:     print(g.edges(sort=True, labels=0))
[(0, 1), (0, 1), (0, 1), (0, 2), (0, 2), (0, 3), (0, 3), (1, 2), (1, 2), (1, 3),␣
↪(1, 3), (2, 3), (2, 3)]
[(0, 1), (0, 1), (0, 1), (0, 4), (0, 4), (1, 4), (1, 4)]
```

```
>>> from sage.all import *
>>> G = graphs.CompleteBipartiteGraph(Integer(2), Integer(3))
>>> G.add_edge(Integer(2), Integer(3))
>>> G.allow_multiple_edges(True)
```

(continues on next page)

(continued from previous page)

```
>>> G.add_edges(G.edge_iterator())
>>> G.add_edges([(Integer(0), Integer(1)), (Integer(0), Integer(1)), (Integer(0),␣
↪Integer(1))])
>>> S,C,f = cleave(G, cut_vertices=[Integer(0), Integer(1)])
>>> for g in S:
...     print(g.edges(sort=True, labels=Integer(0)))
[(0, 1), (0, 1), (0, 1), (0, 2), (0, 2), (0, 3), (0, 3), (1, 2), (1, 2), (1, 3),␣
↪(1, 3), (2, 3), (2, 3)]
[(0, 1), (0, 1), (0, 1), (0, 4), (0, 4), (1, 4), (1, 4)]
```

sage.graphs.connectivity.**connected_component_containing_vertex**(*G*, *vertex*, *sort=False*, *key=None*, *forbidden_vertices=None*)

Return a list of the vertices connected to vertex.

INPUT:

- `G` – the input graph
- `vertex` – the vertex to search for
- `sort` – boolean (default: `False`); if `True`, vertices inside the component are sorted according to the default ordering
- `key` – a function (default: `None`); a function that takes a vertex as its one argument and returns a value that can be used for comparisons in the sorting algorithm (we must have `sort=True`)
- `forbidden_vertices` – list (default: `None`); set of vertices to avoid during the search. The start `vertex` cannot be in this set.

EXAMPLES:

```
sage: from sage.graphs.connectivity import connected_component_containing_vertex
sage: G = Graph({0: [1, 3], 1: [2], 2: [3], 4: [5, 6], 5: [6]})
sage: connected_component_containing_vertex(G, 0, sort=True)
[0, 1, 2, 3]
sage: G.connected_component_containing_vertex(0, sort=True)
[0, 1, 2, 3]
sage: G.connected_component_containing_vertex(0, sort=True, forbidden_vertices=[1,
↪ 3])
[0]
sage: D = DiGraph({0: [1, 3], 1: [2], 2: [3], 4: [5, 6], 5: [6]})
sage: connected_component_containing_vertex(D, 0, sort=True)
[0, 1, 2, 3]
sage: connected_component_containing_vertex(D, 0, sort=True, key=lambda x: -x)
[3, 2, 1, 0]
```

```
>>> from sage.all import *
>>> from sage.graphs.connectivity import connected_component_containing_vertex
>>> G = Graph({Integer(0): [Integer(1), Integer(3)], Integer(1): [Integer(2)],␣
↪Integer(2): [Integer(3)], Integer(4): [Integer(5), Integer(6)], Integer(5):␣
↪[Integer(6)]})
>>> connected_component_containing_vertex(G, Integer(0), sort=True)
[0, 1, 2, 3]
>>> G.connected_component_containing_vertex(Integer(0), sort=True)
[0, 1, 2, 3]
```

(continues on next page)

(continued from previous page)

```
>>> G.connected_component_containing_vertex(Integer(0), sort=True, forbidden_
↪vertices=[Integer(1), Integer(3)])
[0]
>>> D = DiGraph({Integer(0): [Integer(1), Integer(3)], Integer(1): [Integer(2)],␣
↪Integer(2): [Integer(3)], Integer(4): [Integer(5), Integer(6)], Integer(5):␣
↪[Integer(6)]})
>>> connected_component_containing_vertex(D, Integer(0), sort=True)
[0, 1, 2, 3]
>>> connected_component_containing_vertex(D, Integer(0), sort=True, key=lambda x:␣
↪-x)
[3, 2, 1, 0]
```

sage.graphs.connectivity.**connected_components**(*G*, *sort=False*, *key=None*, *forbidden_vertices=None*)

Return the list of connected components.

This returns a list of lists of vertices, each list representing a connected component. The list is ordered from largest to smallest component.

INPUT:

- `G` – the input graph
- `sort` – boolean (default: `False`); if `True`, vertices inside each component are sorted according to the default ordering
- `key` – a function (default: `None`); a function that takes a vertex as its one argument and returns a value that can be used for comparisons in the sorting algorithm (we must have `sort=True`)
- `forbidden_vertices` – list (default: `None`); set of vertices to avoid during the search

EXAMPLES:

```
sage: from sage.graphs.connectivity import connected_components
sage: G = Graph({0: [1, 3], 1: [2], 2: [3], 4: [5, 6], 5: [6]})
sage: connected_components(G, sort=True)
[[0, 1, 2, 3], [4, 5, 6]]
sage: G.connected_components(sort=True)
[[0, 1, 2, 3], [4, 5, 6]]
sage: D = DiGraph({0: [1, 3], 1: [2], 2: [3], 4: [5, 6], 5: [6]})
sage: connected_components(D, sort=True)
[[0, 1, 2, 3], [4, 5, 6]]
sage: connected_components(D, sort=True, key=lambda x: -x)
[[3, 2, 1, 0], [6, 5, 4]]
```

```
>>> from sage.all import *
>>> from sage.graphs.connectivity import connected_components
>>> G = Graph({Integer(0): [Integer(1), Integer(3)], Integer(1): [Integer(2)],␣
↪Integer(2): [Integer(3)], Integer(4): [Integer(5), Integer(6)], Integer(5):␣
↪[Integer(6)]})
>>> connected_components(G, sort=True)
[[0, 1, 2, 3], [4, 5, 6]]
>>> G.connected_components(sort=True)
[[0, 1, 2, 3], [4, 5, 6]]
>>> D = DiGraph({Integer(0): [Integer(1), Integer(3)], Integer(1): [Integer(2)],␣
↪Integer(2): [Integer(3)], Integer(4): [Integer(5), Integer(6)], Integer(5):␣
```

(continues on next page)

(continued from previous page)

```
↪[Integer(6)]})
>>> connected_components(D, sort=True)
[[0, 1, 2, 3], [4, 5, 6]]
>>> connected_components(D, sort=True, key=lambda x: -x)
[[3, 2, 1, 0], [6, 5, 4]]
```

Connected components in a graph with forbidden vertices:

```
sage: G = graphs.PathGraph(5)
sage: connected_components(G, sort=True, forbidden_vertices=[2])
[[0, 1], [3, 4]]
sage: connected_components(G, sort=True,
....:     forbidden_vertices=G.neighbor_iterator(2, closed=True))
[[0], [4]]
```

```
>>> from sage.all import *
>>> G = graphs.PathGraph(Integer(5))
>>> connected_components(G, sort=True, forbidden_vertices=[Integer(2)])
[[0, 1], [3, 4]]
>>> connected_components(G, sort=True,
...     forbidden_vertices=G.neighbor_iterator(Integer(2), closed=True))
[[0], [4]]
```

sage.graphs.connectivity.**connected_components_number**(*G*, *forbidden_vertices=None*)

alias of *number_of_connected_components()*.

sage.graphs.connectivity.**connected_components_sizes**(*G*, *forbidden_vertices=None*)

Return the sizes of the connected components as a list.

The list is sorted from largest to lower values.

INPUT:

- `G` – the input graph
- `forbidden_vertices` – list (default: `None`); set of vertices to avoid during the search

EXAMPLES:

```
sage: from sage.graphs.connectivity import connected_components_sizes
sage: for x in graphs(3):
....:     print(connected_components_sizes(x))
[1, 1, 1]
[2, 1]
[3]
[3]
sage: for x in graphs(3):
....:     print(x.connected_components_sizes())
[1, 1, 1]
[2, 1]
[3]
[3]
sage: G = graphs.PathGraph(5)
sage: G.connected_components_sizes()
[5]
```

(continues on next page)

(continued from previous page)

```
sage: G.connected_components_sizes(forbidden_vertices=[1])
[3, 1]
sage: G.connected_components_sizes(forbidden_vertices=[1, 3])
[1, 1, 1]
```

```
>>> from sage.all import *
>>> from sage.graphs.connectivity import connected_components_sizes
>>> for x in graphs(Integer(3)):
...     print(connected_components_sizes(x))
[1, 1, 1]
[2, 1]
[3]
[3]
>>> for x in graphs(Integer(3)):
...     print(x.connected_components_sizes())
[1, 1, 1]
[2, 1]
[3]
[3]
>>> G = graphs.PathGraph(Integer(5))
>>> G.connected_components_sizes()
[5]
>>> G.connected_components_sizes(forbidden_vertices=[Integer(1)])
[3, 1]
>>> G.connected_components_sizes(forbidden_vertices=[Integer(1), Integer(3)])
[1, 1, 1]
```

sage.graphs.connectivity.**connected_components_subgraphs**(*G*, *forbidden_vertices=None*, *immutable=None*)

Return a list of connected components as graph objects.

INPUT:

- `G` – the input graph
- `forbidden_vertices` – list (default: `None`); set of vertices to avoid during the search
- `immutable` – boolean (default: `None`); whether to create a mutable/immutable subgraphs. `immutable=None` (default) means that the graph and the subgraphs of its connected components will behave the same way.

EXAMPLES:

```
sage: from sage.graphs.connectivity import connected_components_subgraphs
sage: G = Graph({0: [1, 3], 1: [2], 2: [3], 4: [5, 6], 5: [6]})
sage: L = connected_components_subgraphs(G)
sage: graphs_list.show_graphs(L)                                                #␣
↪needs sage.plot
sage: L = connected_components_subgraphs(G, forbidden_vertices=[1, 3])
sage: graphs_list.show_graphs(L)                                                #␣
↪needs sage.plot
sage: D = DiGraph({0: [1, 3], 1: [2], 2: [3], 4: [5, 6], 5: [6]})
sage: L = connected_components_subgraphs(D)
sage: graphs_list.show_graphs(L)                                                #␣
```

(continues on next page)

(continued from previous page)

```
↪needs sage.plot
sage: L = D.connected_components_subgraphs()
sage: graphs_list.show_graphs(L)                                    #␣
↪needs sage.plot
```

```
>>> from sage.all import *
>>> from sage.graphs.connectivity import connected_components_subgraphs
>>> G = Graph({Integer(0): [Integer(1), Integer(3)], Integer(1): [Integer(2)],␣
↪Integer(2): [Integer(3)], Integer(4): [Integer(5), Integer(6)], Integer(5):␣
↪[Integer(6)]})
>>> L = connected_components_subgraphs(G)
>>> graphs_list.show_graphs(L)                                      #␣
↪needs sage.plot
>>> L = connected_components_subgraphs(G, forbidden_vertices=[Integer(1),␣
↪Integer(3)])
>>> graphs_list.show_graphs(L)                                      #␣
↪needs sage.plot
>>> D = DiGraph({Integer(0): [Integer(1), Integer(3)], Integer(1): [Integer(2)],␣
↪Integer(2): [Integer(3)], Integer(4): [Integer(5), Integer(6)], Integer(5):␣
↪[Integer(6)]})
>>> L = connected_components_subgraphs(D)
>>> graphs_list.show_graphs(L)                                      #␣
↪needs sage.plot
>>> L = D.connected_components_subgraphs()
>>> graphs_list.show_graphs(L)                                      #␣
↪needs sage.plot
```

sage.graphs.connectivity.**edge_connectivity**(*G*, *value_only=True*, *algorithm=None*, *by_weight=False*, *weight_function=None*, *check_weight=True*, *vertices=False*, *solver=None*, *verbose=0*, *integrality_tolerance=0.001*)

Return the edge connectivity of the graph.

For more information, see the Wikipedia article Connectivity_(graph_theory).

Note

When the graph is a directed graph, this method actually computes the *strong* connectivity, (i.e. a directed graph is strongly k-connected if there are k disjoint paths between any two vertices u, v). If you do not want to consider strong connectivity, the best is probably to convert your `DiGraph` object to a `Graph` object, and compute the connectivity of this other graph.

INPUT:

- `G` – the input Sage (Di)Graph
- `value_only` – boolean (default: `True`)
  - When set to `True` (default), only the value is returned.
  - When set to `False`, both the value and a minimum vertex cut are returned.
- `algorithm` – string (default: `None`); selects an algorithm:
  - `None` – default; selects the best algorithm available

  - `'boost'` – use the Boost graph library (which is much more efficient). It is not available when `edge_labels=True`, and it is rejected for directed graphs: the Boost implementation is for undirected graphs only and returns wrong values on digraphs (see Issue #18753).
  - `'Sage'` – use Sage's implementation based on integer linear programming
  - `'gabow'` – use the algorithm of Gabow [Gabow1995] implemented in *GabowEdgeConnectivity*. It is for directed graphs only, does not handle edge labels, and only computes the value of the edge connectivity, so it cannot be used with `value_only=False` or `vertices=True`. It is much faster than the integer linear program and is the default for digraphs when only the value is requested.

- `by_weight` – boolean (default: `False`); if `True`, computes a weighted minimum cut where each edge has the weight given by `weight_function`, or its label if `weight_function` is `None`. If `False`, each edge has weight 1. Implies `algorithm='sage'`.
- `weight_function` – function (default: `None`); a function that takes as input an edge `(u, v, l)` and outputs its weight. If not `None`, `by_weight` is automatically set to `True`. If `None` and `by_weight` is `True`, we use the edge label `l`, if `l` is not `None`, else 1 as a weight.
- `check_weight` – boolean (default: `True`); whether to check that the `weight_function` outputs a number for each edge
- `vertices` – boolean (default: `False`)
  - When set to `True`, also returns the two sets of vertices that are disconnected by the cut. Implies `value_only=False`.
- `solver` – string (default: `None`); specifies a Mixed Integer Linear Programming (MILP) solver to be used. If set to `None`, the default one is used. For more information on MILP solvers and which default solver is used, see the method `solve` of the class `MixedIntegerLinearProgram`.
- `verbose` – integer (default: 0); sets the level of verbosity. Set to 0 by default, which means quiet.
- `integrality_tolerance` – float; parameter for use with MILP solvers over an inexact base ring; see `MixedIntegerLinearProgram.get_values()`.

EXAMPLES:

A basic application on the PappusGraph:

```
sage: from sage.graphs.connectivity import edge_connectivity
sage: g = graphs.PappusGraph()
sage: edge_connectivity(g)
3
sage: g.edge_connectivity()
3
```

```
>>> from sage.all import *
>>> from sage.graphs.connectivity import edge_connectivity
>>> g = graphs.PappusGraph()
>>> edge_connectivity(g)
3
>>> g.edge_connectivity()
3
```

The edge connectivity of a complete graph is its minimum degree, and one of the two parts of the bipartition is reduced to only one vertex. The graph of the cut edges is isomorphic to a Star graph:

```
sage: g = graphs.CompleteGraph(5)
sage: [ value, edges, [ setA, setB ]] = edge_connectivity(g,vertices=True)
sage: value
4
sage: len(setA) == 1 or len(setB) == 1
True
sage: cut = Graph()
sage: cut.add_edges(edges)
sage: cut.is_isomorphic(graphs.StarGraph(4))
True
```

```
>>> from sage.all import *
>>> g = graphs.CompleteGraph(Integer(5))
>>> [ value, edges, [ setA, setB ]] = edge_connectivity(g,vertices=True)
>>> value
4
>>> len(setA) == Integer(1) or len(setB) == Integer(1)
True
>>> cut = Graph()
>>> cut.add_edges(edges)
>>> cut.is_isomorphic(graphs.StarGraph(Integer(4)))
True
```

Even if obviously in any graph we know that the edge connectivity is less than the minimum degree of the graph:

```
sage: g = graphs.RandomGNP(10,.3)
sage: min(g.degree()) >= edge_connectivity(g)
True
```

```
>>> from sage.all import *
>>> g = graphs.RandomGNP(Integer(10),RealNumber('.3'))
>>> min(g.degree()) >= edge_connectivity(g)
True
```

If we build a tree then assign to its edges a random value, the minimum cut will be the edge with minimum value:

```
sage: tree = graphs.RandomTree(10)
sage: for u,v in tree.edge_iterator(labels=None):
....:     tree.set_edge_label(u, v, random())
sage: minimum = min(tree.edge_labels())
sage: [_, [(_, _, l)]] = edge_connectivity(tree, value_only=False,              #␣
↪needs sage.numerical.mip
....:                                      by_weight=True)
sage: l == minimum                                                              #␣
↪needs sage.numerical.mip
True
```

```
>>> from sage.all import *
>>> tree = graphs.RandomTree(Integer(10))
>>> for u,v in tree.edge_iterator(labels=None):
...     tree.set_edge_label(u, v, random())
>>> minimum = min(tree.edge_labels())
```

(continues on next page)

(continued from previous page)

```
>>> [_, [(_, _, l)]] = edge_connectivity(tree, value_only=False,                #␣
↪needs sage.numerical.mip
...                                           by_weight=True)
>>> l == minimum                                                               #␣
↪needs sage.numerical.mip
True
```

A `weight_function` can be used to read the weight from anywhere in the edge, and setting it implies `by_weight=True`:

```
sage: G = Graph([(0, 1, {'cost': 3}), (1, 2, {'cost': 1}),
....:            (2, 0, {'cost': 2})])
sage: edge_connectivity(G, weight_function=lambda e: e[2]['cost'])              #␣
↪needs sage.numerical.mip
3.0
```

```
>>> from sage.all import *
>>> G = Graph([(Integer(0), Integer(1), {'cost': Integer(3)}), (Integer(1),␣
↪Integer(2), {'cost': Integer(1)}),
...            (Integer(2), Integer(0), {'cost': Integer(2)})])
>>> edge_connectivity(G, weight_function=lambda e: e[Integer(2)]['cost'])      ␣
↪     # needs sage.numerical.mip
3.0
```

When `value_only=True` and `algorithm="sage"`, this function is optimized for small connectivity values and does not need to build a linear program.

It is the case for graphs which are not connected

```
sage: g = 2 * graphs.PetersenGraph()
sage: edge_connectivity(g, algorithm='sage')
0.0
```

```
>>> from sage.all import *
>>> g = Integer(2) * graphs.PetersenGraph()
>>> edge_connectivity(g, algorithm='sage')
0.0
```

For directed graphs, the strong connectivity is tested through the dedicated function:

```
sage: g = digraphs.ButterflyGraph(3)
sage: edge_connectivity(g, algorithm='sage')
0.0
```

```
>>> from sage.all import *
>>> g = digraphs.ButterflyGraph(Integer(3))
>>> edge_connectivity(g, algorithm='sage')
0.0
```

We check that the result with Boost is the same as the result without Boost:

```
sage: g = graphs.RandomGNP(15, .3)
sage: (edge_connectivity(g, algorithm='boost')
```

(continues on next page)

(continued from previous page)

```
↪# needs sage.numerical.mip
....:     == edge_connectivity(g, algorithm='sage'))
True
```

```
>>> from sage.all import *
>>> g = graphs.RandomGNP(Integer(15), RealNumber('.3'))
>>> (edge_connectivity(g, algorithm='boost')                                #␣
↪needs sage.numerical.mip
...     == edge_connectivity(g, algorithm='sage'))
True
```

The Boost implementation is for undirected graphs only. It considers a digraph as undirected and therefore returns wrong values, so it is rejected (see Issue #18753):

```
sage: g = digraphs.Path(3)
sage: edge_connectivity(g)
0
sage: edge_connectivity(g, algorithm='boost')
Traceback (most recent call last):
...
ValueError: the Boost implementation of the edge connectivity is for
undirected graphs only and returns wrong values on digraphs, see
https://github.com/sagemath/sage/issues/18753
```

```
>>> from sage.all import *
>>> g = digraphs.Path(Integer(3))
>>> edge_connectivity(g)
0
>>> edge_connectivity(g, algorithm='boost')
Traceback (most recent call last):
...
ValueError: the Boost implementation of the edge connectivity is for
undirected graphs only and returns wrong values on digraphs, see
https://github.com/sagemath/sage/issues/18753
```

The algorithm of Gabow is the default for digraphs when only the value is requested. It cannot return a cut, nor handle edge labels or undirected graphs:

```
sage: D = digraphs.Complete(5)
sage: edge_connectivity(D)
4
sage: edge_connectivity(D, algorithm='gabow')
4
sage: edge_connectivity(D, algorithm='gabow', value_only=False)
Traceback (most recent call last):
...
ValueError: the Gabow algorithm only computes the value of the edge
connectivity, it cannot return a cut
sage: edge_connectivity(graphs.PetersenGraph(), algorithm='gabow')
Traceback (most recent call last):
...
ValueError: the Gabow algorithm is for directed graphs only
```

```
>>> from sage.all import *
>>> D = digraphs.Complete(Integer(5))
>>> edge_connectivity(D)
4
>>> edge_connectivity(D, algorithm='gabow')
4
>>> edge_connectivity(D, algorithm='gabow', value_only=False)
Traceback (most recent call last):
...
ValueError: the Gabow algorithm only computes the value of the edge
connectivity, it cannot return a cut
>>> edge_connectivity(graphs.PetersenGraph(), algorithm='gabow')
Traceback (most recent call last):
...
ValueError: the Gabow algorithm is for directed graphs only
```

It also supports digraphs with multiple edges:

```
sage: D = DiGraph([(0, 1), (0, 1), (1, 0)], multiedges=True)
sage: edge_connectivity(D)
1
sage: D.add_edge(1, 0)
sage: edge_connectivity(D)
2
```

```
>>> from sage.all import *
>>> D = DiGraph([(Integer(0), Integer(1)), (Integer(0), Integer(1)), (Integer(1),␣
↪Integer(0))], multiedges=True)
>>> edge_connectivity(D)
1
>>> D.add_edge(Integer(1), Integer(0))
>>> edge_connectivity(D)
2
```

The `implementation` keyword is deprecated in favour of `algorithm`:

```
sage: edge_connectivity(graphs.PetersenGraph(), implementation='sage')
doctest:warning...
DeprecationWarning: use the option 'algorithm' instead of 'implementation'
See https://github.com/sagemath/sage/issues/42652 for details.
3
```

```
>>> from sage.all import *
>>> edge_connectivity(graphs.PetersenGraph(), implementation='sage')
doctest:warning...
DeprecationWarning: use the option 'algorithm' instead of 'implementation'
See https://github.com/sagemath/sage/issues/42652 for details.
3
```

The `use_edge_labels` keyword is deprecated in favour of `by_weight`:

```
sage: G = Graph([(0, 1, 3), (1, 2, 1), (2, 0, 2)])
sage: edge_connectivity(G, use_edge_labels=True)                                #␣
```

(continues on next page)

(continued from previous page)

```
↪needs sage.numerical.mip
doctest:warning...
DeprecationWarning: use the option 'by_weight' instead of 'use_edge_labels'
See https://github.com/sagemath/sage/issues/42740 for details.
3.0
```

```
>>> from sage.all import *
>>> G = Graph([(Integer(0), Integer(1), Integer(3)), (Integer(1), Integer(2),␣
↪Integer(1)), (Integer(2), Integer(0), Integer(2))])
>>> edge_connectivity(G, use_edge_labels=True)                                #␣
↪needs sage.numerical.mip
doctest:warning...
DeprecationWarning: use the option 'by_weight' instead of 'use_edge_labels'
See https://github.com/sagemath/sage/issues/42740 for details.
3.0
```

sage.graphs.connectivity.**is_biconnected**(*G*, *forbidden_vertices=None*)

Check whether the graph is biconnected.

A biconnected graph is a connected graph on two or more vertices that is not broken into disconnected pieces by deleting any single vertex.

INPUT:

- `G` – a Sage graph
- `forbidden_vertices` – list (default: `None`); set of vertices to avoid during the search. This is equivalent to checking whether the graph in which the forbidden vertices have been removed is biconnected.

See also

- *is_connected()*
- *blocks_and_cut_vertices()*
- *blocks_and_cuts_tree()*
- Wikipedia article Biconnected_graph

EXAMPLES:

```
sage: G = graphs.PetersenGraph()
sage: G.is_biconnected()
True
sage: G.add_path([0,'a','b'])
sage: G.is_biconnected()
False
sage: G.add_edge('b', 1)
sage: G.is_biconnected()
True
```

```
>>> from sage.all import *
>>> G = graphs.PetersenGraph()
```

(continues on next page)

(continued from previous page)

```
>>> G.is_biconnected()
True
>>> G.add_path([Integer(0),'a','b'])
>>> G.is_biconnected()
False
>>> G.add_edge('b', Integer(1))
>>> G.is_biconnected()
True
```

Check the behavior of parameter `forbidden_vertices`:

```
sage: G = graphs.CycleGraph(5)
sage: G.is_biconnected()
True
sage: G.is_biconnected(forbidden_vertices=[0])
False
```

```
>>> from sage.all import *
>>> G = graphs.CycleGraph(Integer(5))
>>> G.is_biconnected()
True
>>> G.is_biconnected(forbidden_vertices=[Integer(0)])
False
```

`sage.graphs.connectivity.`**`is_connected`**(*G*, *forbidden_vertices=None*)

Check whether the (di)graph is connected.

Note that in a graph, path connected is equivalent to connected.

INPUT:

- `G` – the input graph
- `forbidden_vertices` – list (default: `None`); set of vertices to avoid during the search

See also

- *is_biconnected()*

EXAMPLES:

```
sage: from sage.graphs.connectivity import is_connected
sage: G = Graph({0: [1, 2], 1: [2], 3: [4, 5], 4: [5]})
sage: is_connected(G)
False
sage: G.is_connected()
False
sage: G.add_edge(0,3)
sage: is_connected(G)
True
sage: is_connected(G, forbidden_vertices=[3])
False
sage: is_connected(G, forbidden_vertices=[1])
```

(continues on next page)

(continued from previous page)

```
True
sage: D = DiGraph({0: [1, 2], 1: [2], 3: [4, 5], 4: [5]})
sage: is_connected(D)
False
sage: D.add_edge(0, 3)
sage: is_connected(D)
True
sage: D = DiGraph({1: [0], 2: [0]})
sage: is_connected(D)
True
```

```
>>> from sage.all import *
>>> from sage.graphs.connectivity import is_connected
>>> G = Graph({Integer(0): [Integer(1), Integer(2)], Integer(1): [Integer(2)],␣
↪Integer(3): [Integer(4), Integer(5)], Integer(4): [Integer(5)]})
>>> is_connected(G)
False
>>> G.is_connected()
False
>>> G.add_edge(Integer(0),Integer(3))
>>> is_connected(G)
True
>>> is_connected(G, forbidden_vertices=[Integer(3)])
False
>>> is_connected(G, forbidden_vertices=[Integer(1)])
True
>>> D = DiGraph({Integer(0): [Integer(1), Integer(2)], Integer(1): [Integer(2)],␣
↪Integer(3): [Integer(4), Integer(5)], Integer(4): [Integer(5)]})
>>> is_connected(D)
False
>>> D.add_edge(Integer(0), Integer(3))
>>> is_connected(D)
True
>>> D = DiGraph({Integer(1): [Integer(0)], Integer(2): [Integer(0)]})
>>> is_connected(D)
True
```

sage.graphs.connectivity.**is_cut_edge**(*G*, *u*, *v=None*, *label=None*)

Check whether the edge `(u, v)` is a cut-edge or a bridge of graph `G`.

A cut edge (or bridge) is an edge that when removed increases the number of connected components. This function works with simple graphs as well as graphs with loops and multiedges. In a digraph, a cut edge is an edge that when removed increases the number of (weakly) connected components.

INPUT: The following forms are accepted

- is_cut_edge(G, 1, 2)
- is_cut_edge(G, (1, 2))
- is_cut_edge(G, 1, 2, 'label')
- is_cut_edge(G, (1, 2, 'label'))

OUTPUT:

- Returns `True` if (u,v) is a cut edge, False otherwise

EXAMPLES:

```
sage: from sage.graphs.connectivity import is_cut_edge
sage: G = graphs.CompleteGraph(4)
sage: is_cut_edge(G,0,2)
False
sage: G.is_cut_edge(0,2)
False

sage: G = graphs.CompleteGraph(4)
sage: G.add_edge((0,5,'silly'))
sage: is_cut_edge(G,(0,5,'silly'))
True

sage: G = Graph([[0,1],[0,2],[3,4],[4,5],[3,5]])
sage: is_cut_edge(G,(0,1))
True

sage: G = Graph([[0,1],[0,2],[1,1]], loops = True)
sage: is_cut_edge(G,(1,1))
False

sage: G = digraphs.Circuit(5)
sage: is_cut_edge(G,(0,1))
False

sage: G = graphs.CompleteGraph(6)
sage: is_cut_edge(G,(0,7))
Traceback (most recent call last):
...
ValueError: edge not in graph
```

```
>>> from sage.all import *
>>> from sage.graphs.connectivity import is_cut_edge
>>> G = graphs.CompleteGraph(Integer(4))
>>> is_cut_edge(G,Integer(0),Integer(2))
False
>>> G.is_cut_edge(Integer(0),Integer(2))
False

>>> G = graphs.CompleteGraph(Integer(4))
>>> G.add_edge((Integer(0),Integer(5),'silly'))
>>> is_cut_edge(G,(Integer(0),Integer(5),'silly'))
True

>>> G = Graph([[Integer(0),Integer(1)],[Integer(0),Integer(2)],[Integer(3),
↪Integer(4)],[Integer(4),Integer(5)],[Integer(3),Integer(5)]])
>>> is_cut_edge(G,(Integer(0),Integer(1)))
True

>>> G = Graph([[Integer(0),Integer(1)],[Integer(0),Integer(2)],[Integer(1),
↪Integer(1)]], loops = True)
```

(continues on next page)

(continued from previous page)

```
>>> is_cut_edge(G,(Integer(1),Integer(1)))
False

>>> G = digraphs.Circuit(Integer(5))
>>> is_cut_edge(G,(Integer(0),Integer(1)))
False

>>> G = graphs.CompleteGraph(Integer(6))
>>> is_cut_edge(G,(Integer(0),Integer(7)))
Traceback (most recent call last):
...
ValueError: edge not in graph
```

sage.graphs.connectivity.**is_cut_vertex**(*G*, *u*, *weak=False*)

Check whether the input vertex is a cut-vertex.

A vertex is a cut-vertex if its removal from the (di)graph increases the number of (strongly) connected components. Isolated vertices or leaves are not cut-vertices. This function works with simple graphs as well as graphs with loops and multiple edges.

INPUT:

- `G` – a Sage (Di)Graph
- `u` – a vertex
- `weak` – boolean (default: `False`); whether the connectivity of directed graphs is to be taken in the weak sense, that is ignoring edges orientations

OUTPUT:

Return `True` if `u` is a cut-vertex, and `False` otherwise.

EXAMPLES:

Giving a LollipopGraph(4,2), that is a complete graph with 4 vertices with a pending edge:

```
sage: from sage.graphs.connectivity import is_cut_vertex
sage: G = graphs.LollipopGraph(4, 2)
sage: is_cut_vertex(G, 0)
False
sage: is_cut_vertex(G, 3)
True
sage: G.is_cut_vertex(3)
True
```

```
>>> from sage.all import *
>>> from sage.graphs.connectivity import is_cut_vertex
>>> G = graphs.LollipopGraph(Integer(4), Integer(2))
>>> is_cut_vertex(G, Integer(0))
False
>>> is_cut_vertex(G, Integer(3))
True
>>> G.is_cut_vertex(Integer(3))
True
```

Comparing the weak and strong connectivity of a digraph:

```
sage: D = digraphs.Circuit(6)
sage: D.is_strongly_connected()
True
sage: is_cut_vertex(D, 2)
True
sage: is_cut_vertex(D, 2, weak=True)
False
```

```
>>> from sage.all import *
>>> D = digraphs.Circuit(Integer(6))
>>> D.is_strongly_connected()
True
>>> is_cut_vertex(D, Integer(2))
True
>>> is_cut_vertex(D, Integer(2), weak=True)
False
```

Giving a vertex that is not in the graph:

```
sage: G = graphs.CompleteGraph(4)
sage: is_cut_vertex(G, 7)
Traceback (most recent call last):
...
ValueError: vertex (7) is not a vertex of the graph
```

```
>>> from sage.all import *
>>> G = graphs.CompleteGraph(Integer(4))
>>> is_cut_vertex(G, Integer(7))
Traceback (most recent call last):
...
ValueError: vertex (7) is not a vertex of the graph
```

sage.graphs.connectivity.**is_edge_cut**(*G*, *edges*)

Check whether `edges` form an edge cut.

A set of edges is an edge cut of a graph if its removal increases the number of connected components. In a digraph, we consider the number of (weakly) connected components.

This method is not working for (di)graphs with multiple edges. Furthermore, edge labels are ignored.

INPUT:

- `G` – a (di)graph
- `edges` – a set of edges

EXAMPLES:

A cycle graph of order 4:

```
sage: from sage.graphs.connectivity import is_edge_cut
sage: G = graphs.CycleGraph(4)
sage: is_edge_cut(G, [(1, 2)])
False
sage: is_edge_cut(G, [(1, 2), (2, 3)])
True
```

(continues on next page)

(continued from previous page)

```
sage: is_edge_cut(G, [(1, 2), (3, 0)])
True
```

```
>>> from sage.all import *
>>> from sage.graphs.connectivity import is_edge_cut
>>> G = graphs.CycleGraph(Integer(4))
>>> is_edge_cut(G, [(Integer(1), Integer(2))])
False
>>> is_edge_cut(G, [(Integer(1), Integer(2)), (Integer(2), Integer(3))])
True
>>> is_edge_cut(G, [(Integer(1), Integer(2)), (Integer(3), Integer(0))])
True
```

A pending edge is a cut-edge:

```
sage: G.add_edge((0, 5, 'silly'))
sage: is_edge_cut(G, [(0, 5, 'silly')])
True
```

```
>>> from sage.all import *
>>> G.add_edge((Integer(0), Integer(5), 'silly'))
>>> is_edge_cut(G, [(Integer(0), Integer(5), 'silly')])
True
```

Edge labels are ignored, even if specified:

```
sage: G.add_edge((2, 5, 'xyz'))
sage: is_edge_cut(G, [(0, 5), (2, 5)])
True
sage: is_edge_cut(G, [(0, 5), (2, 5, 'xyz')])
True
sage: is_edge_cut(G, [(0, 5, 'silly'), (2, 5)])
True
sage: is_edge_cut(G, [(0, 5, 'aa'), (2, 5, 'bb')])
True
```

```
>>> from sage.all import *
>>> G.add_edge((Integer(2), Integer(5), 'xyz'))
>>> is_edge_cut(G, [(Integer(0), Integer(5)), (Integer(2), Integer(5))])
True
>>> is_edge_cut(G, [(Integer(0), Integer(5)), (Integer(2), Integer(5), 'xyz')])
True
>>> is_edge_cut(G, [(Integer(0), Integer(5), 'silly'), (Integer(2), Integer(5))])
True
>>> is_edge_cut(G, [(Integer(0), Integer(5), 'aa'), (Integer(2), Integer(5), 'bb
↪')])
True
```

The graph can have loops:

```
sage: G.allow_loops(True)
sage: G.add_edge(0, 0)
```

(continues on next page)

(continued from previous page)

```
sage: is_edge_cut(G, [(0, 5), (2, 5)])
True
sage: is_edge_cut(G, [(0, 0), (0, 5), (2, 5)])
True
```

```
>>> from sage.all import *
>>> G.allow_loops(True)
>>> G.add_edge(Integer(0), Integer(0))
>>> is_edge_cut(G, [(Integer(0), Integer(5)), (Integer(2), Integer(5))])
True
>>> is_edge_cut(G, [(Integer(0), Integer(0)), (Integer(0), Integer(5)),␣
↪(Integer(2), Integer(5))])
True
```

Multiple edges are not allowed:

```
sage: G.allow_multiple_edges(True)
sage: is_edge_cut(G, [(0, 5), (2, 5)])
Traceback (most recent call last):
...
ValueError: This method is not known to work on graphs with
 multiedges. Perhaps this method can be updated to handle them, but in
 the meantime if you want to use it please disallow multiedges using
 allow_multiple_edges().
```

```
>>> from sage.all import *
>>> G.allow_multiple_edges(True)
>>> is_edge_cut(G, [(Integer(0), Integer(5)), (Integer(2), Integer(5))])
Traceback (most recent call last):
...
ValueError: This method is not known to work on graphs with
 multiedges. Perhaps this method can be updated to handle them, but in
 the meantime if you want to use it please disallow multiedges using
 allow_multiple_edges().
```

An error is raised if an element of `edges` is not an edge of G:

```
sage: G = graphs.CycleGraph(4)
sage: is_edge_cut(G, [(0, 2)])
Traceback (most recent call last):
...
ValueError: edge (0, 2) is not an edge of the graph
```

```
>>> from sage.all import *
>>> G = graphs.CycleGraph(Integer(4))
>>> is_edge_cut(G, [(Integer(0), Integer(2))])
Traceback (most recent call last):
...
ValueError: edge (0, 2) is not an edge of the graph
```

For digraphs, this method considers the number of (weakly) connected components:

```
sage: G = digraphs.Circuit(4)
sage: is_edge_cut(G, [(0, 1)])
False
sage: G = digraphs.Circuit(4)
sage: is_edge_cut(G, [(0, 1), (1, 2)])
True
```

```
>>> from sage.all import *
>>> G = digraphs.Circuit(Integer(4))
>>> is_edge_cut(G, [(Integer(0), Integer(1))])
False
>>> G = digraphs.Circuit(Integer(4))
>>> is_edge_cut(G, [(Integer(0), Integer(1)), (Integer(1), Integer(2))])
True
```

For disconnected (di)graphs, the method checks if the number of (weakly) connected components increases:

```
sage: G = graphs.CycleGraph(4) * 2
sage: is_edge_cut(G, [(1, 2), (2, 3)])
True
sage: G = digraphs.Circuit(4) * 2
sage: is_edge_cut(G, [(0, 1), (1, 2)])
True
```

```
>>> from sage.all import *
>>> G = graphs.CycleGraph(Integer(4)) * Integer(2)
>>> is_edge_cut(G, [(Integer(1), Integer(2)), (Integer(2), Integer(3))])
True
>>> G = digraphs.Circuit(Integer(4)) * Integer(2)
>>> is_edge_cut(G, [(Integer(0), Integer(1)), (Integer(1), Integer(2))])
True
```

sage.graphs.connectivity.**is_strongly_connected**(G)

Check whether the current `DiGraph` is strongly connected.

EXAMPLES:

The circuit is obviously strongly connected:

```
sage: from sage.graphs.connectivity import is_strongly_connected
sage: g = digraphs.Circuit(5)
sage: is_strongly_connected(g)
True
sage: g.is_strongly_connected()
True
```

```
>>> from sage.all import *
>>> from sage.graphs.connectivity import is_strongly_connected
>>> g = digraphs.Circuit(Integer(5))
>>> is_strongly_connected(g)
True
>>> g.is_strongly_connected()
True
```

But a transitive triangle is not:

```
sage: g = DiGraph({0: [1, 2], 1: [2]})
sage: is_strongly_connected(g)
False
```

```
>>> from sage.all import *
>>> g = DiGraph({Integer(0): [Integer(1), Integer(2)], Integer(1): [Integer(2)]})
>>> is_strongly_connected(g)
False
```

sage.graphs.connectivity.**is_triconnected**(G)

Check whether the graph is triconnected.

A triconnected graph is a connected graph on 3 or more vertices that is not broken into disconnected pieces by deleting any pair of vertices.

EXAMPLES:

The Petersen graph is triconnected:

```
sage: G = graphs.PetersenGraph()
sage: G.is_triconnected()
True
```

```
>>> from sage.all import *
>>> G = graphs.PetersenGraph()
>>> G.is_triconnected()
True
```

But a 2D grid is not:

```
sage: G = graphs.Grid2dGraph(3, 3)
sage: G.is_triconnected()
False
```

```
>>> from sage.all import *
>>> G = graphs.Grid2dGraph(Integer(3), Integer(3))
>>> G.is_triconnected()
False
```

By convention, a cycle of order 3 is triconnected:

```
sage: G = graphs.CycleGraph(3)
sage: G.is_triconnected()
True
```

```
>>> from sage.all import *
>>> G = graphs.CycleGraph(Integer(3))
>>> G.is_triconnected()
True
```

But cycles of order 4 and more are not:

```
sage: [graphs.CycleGraph(i).is_triconnected() for i in range(4, 8)]
[False, False, False, False]
```

```
>>> from sage.all import *
>>> [graphs.CycleGraph(i).is_triconnected() for i in range(Integer(4),␣
↪Integer(8))]
[False, False, False, False]
```

Comparing different methods on random graphs that are not always triconnected:

```
sage: G = graphs.RandomBarabasiAlbert(50, 3)                                    #␣
↪needs networkx
sage: G.is_triconnected() == G.vertex_connectivity(k=3)                         #␣
↪needs networkx
True
```

```
>>> from sage.all import *
>>> G = graphs.RandomBarabasiAlbert(Integer(50), Integer(3))                    ␣
↪              # needs networkx
>>> G.is_triconnected() == G.vertex_connectivity(k=Integer(3))                  ␣
↪     # needs networkx
True
```

See also

- *is_connected()*
- *is_biconnected()*
- *spqr_tree()*
- Wikipedia article SPQR_tree

sage.graphs.connectivity.**is_vertex_cut**(*G*, *cut*, *weak=False*)

Check whether the input vertices form a vertex cut.

A set of vertices is a vertex cut if its removal from the (di)graph increases the number of (strongly) connected components. This function works with simple graphs as well as graphs with loops and multiple edges.

INPUT:

- `G` – a Sage (Di)Graph
- `cut` – a set of vertices
- `weak` – boolean (default: `False`); whether the connectivity of directed graphs is to be taken in the weak sense, that is ignoring edges orientations

EXAMPLES:

Giving a cycle graph of order 4:

```
sage: from sage.graphs.connectivity import is_vertex_cut
sage: G = graphs.CycleGraph(4)
sage: is_vertex_cut(G, [0, 1])
```

(continues on next page)

(continued from previous page)

```
False
sage: is_vertex_cut(G, [0, 2])
True
```

```
>>> from sage.all import *
>>> from sage.graphs.connectivity import is_vertex_cut
>>> G = graphs.CycleGraph(Integer(4))
>>> is_vertex_cut(G, [Integer(0), Integer(1)])
False
>>> is_vertex_cut(G, [Integer(0), Integer(2)])
True
```

Giving a disconnected graph:

```
sage: from sage.graphs.connectivity import is_vertex_cut
sage: G = graphs.CycleGraph(4) * 2
sage: G.connected_components(sort=True)
[[0, 1, 2, 3], [4, 5, 6, 7]]
sage: is_vertex_cut(G, [0, 2])
True
sage: is_vertex_cut(G, [4, 6])
True
sage: is_vertex_cut(G, [0, 6])
False
sage: is_vertex_cut(G, [0, 4, 6])
True
```

```
>>> from sage.all import *
>>> from sage.graphs.connectivity import is_vertex_cut
>>> G = graphs.CycleGraph(Integer(4)) * Integer(2)
>>> G.connected_components(sort=True)
[[0, 1, 2, 3], [4, 5, 6, 7]]
>>> is_vertex_cut(G, [Integer(0), Integer(2)])
True
>>> is_vertex_cut(G, [Integer(4), Integer(6)])
True
>>> is_vertex_cut(G, [Integer(0), Integer(6)])
False
>>> is_vertex_cut(G, [Integer(0), Integer(4), Integer(6)])
True
```

Comparing the weak and strong connectivity of a digraph:

```
sage: D = digraphs.Circuit(6)
sage: D.is_strongly_connected()
True
sage: is_vertex_cut(D, [2])
True
sage: is_vertex_cut(D, [2], weak=True)
False
```

```
>>> from sage.all import *
>>> D = digraphs.Circuit(Integer(6))
>>> D.is_strongly_connected()
True
>>> is_vertex_cut(D, [Integer(2)])
True
>>> is_vertex_cut(D, [Integer(2)], weak=True)
False
```

Giving a vertex that is not in the graph:

```
sage: G = graphs.CompleteGraph(4)
sage: is_vertex_cut(G, [7])
Traceback (most recent call last):
...
ValueError: vertex (7) is not a vertex of the graph
```

```
>>> from sage.all import *
>>> G = graphs.CompleteGraph(Integer(4))
>>> is_vertex_cut(G, [Integer(7)])
Traceback (most recent call last):
...
ValueError: vertex (7) is not a vertex of the graph
```

sage.graphs.connectivity.**minimal_separators**(*G*, *forbidden_vertices=None*)

Return an iterator over the minimal separators of `G`.

A separator in a graph is a set of vertices whose removal increases the number of connected components. In other words, a separator is a vertex cut. This method implements the algorithm proposed in [BBC2000]. It computes the set S of minimal separators of a graph in $O(n^3)$ time per separator, and so overall in $O(n^3|S|)$ time.

Warning

Note that all separators are recorded during the execution of the algorithm and so the memory consumption of this method might be huge.

INPUT:

- `G` – an undirected graph
- `forbidden_vertices` – list (default: `None`); set of vertices to avoid during the search

EXAMPLES:

```
sage: P = graphs.PathGraph(5)
sage: sorted(sorted(sep) for sep in P.minimal_separators())
[[1], [2], [3]]
sage: C = graphs.CycleGraph(6)
sage: sorted(sorted(sep) for sep in C.minimal_separators())
[[0, 2], [0, 3], [0, 4], [1, 3], [1, 4], [1, 5], [2, 4], [2, 5], [3, 5]]
sage: sorted(sorted(sep) for sep in C.minimal_separators(forbidden_vertices=[0]))
[[2], [3], [4]]
sage: sorted(sorted(sep) for sep in (P + C).minimal_separators())
```

(continues on next page)

(continued from previous page)

```
[[1], [2], [3], [5, 7], [5, 8], [5, 9], [6, 8],
 [6, 9], [6, 10], [7, 9], [7, 10], [8, 10]]
sage: sorted(sorted(sep) for sep in (P + C).minimal_separators(forbidden_
↪vertices=[10]))
[[1], [2], [3], [6], [7], [8]]

sage: G = graphs.RandomGNP(10, .3)
sage: all(G.is_vertex_cut(sep) for sep in G.minimal_separators())
True
```

```
>>> from sage.all import *
>>> P = graphs.PathGraph(Integer(5))
>>> sorted(sorted(sep) for sep in P.minimal_separators())
[[1], [2], [3]]
>>> C = graphs.CycleGraph(Integer(6))
>>> sorted(sorted(sep) for sep in C.minimal_separators())
[[0, 2], [0, 3], [0, 4], [1, 3], [1, 4], [1, 5], [2, 4], [2, 5], [3, 5]]
>>> sorted(sorted(sep) for sep in C.minimal_separators(forbidden_
↪vertices=[Integer(0)]))
[[2], [3], [4]]
>>> sorted(sorted(sep) for sep in (P + C).minimal_separators())
[[1], [2], [3], [5, 7], [5, 8], [5, 9], [6, 8],
 [6, 9], [6, 10], [7, 9], [7, 10], [8, 10]]
>>> sorted(sorted(sep) for sep in (P + C).minimal_separators(forbidden_
↪vertices=[Integer(10)]))
[[1], [2], [3], [6], [7], [8]]

>>> G = graphs.RandomGNP(Integer(10), RealNumber('.3'))
>>> all(G.is_vertex_cut(sep) for sep in G.minimal_separators())
True
```

Example of [BPS2010]:

```
sage: G = Graph({'a': ['b', 'k'], 'b': ['c'], 'c': ['d', 'j', 'k'],
....:            'd': ['e', 'f', 'j', 'k'], 'e': ['g'],
....:            'f': ['g', 'j', 'k'], 'g': ['j', 'k'], 'h': ['i', 'j'],
....:            'i': ['k'], 'j': ['k']})
sage: sorted(sorted(sep) for sep in G.minimal_separators())
[['a', 'c'], ['b', 'k'], ['c', 'k'], ['d', 'j', 'k'],
 ['h', 'k'], ['i', 'j'], ['j', 'k']]
```

```
>>> from sage.all import *
>>> G = Graph({'a': ['b', 'k'], 'b': ['c'], 'c': ['d', 'j', 'k'],
...            'd': ['e', 'f', 'j', 'k'], 'e': ['g'],
...            'f': ['g', 'j', 'k'], 'g': ['j', 'k'], 'h': ['i', 'j'],
...            'i': ['k'], 'j': ['k']})
>>> sorted(sorted(sep) for sep in G.minimal_separators())
[['a', 'c'], ['b', 'k'], ['c', 'k'], ['d', 'j', 'k'],
 ['h', 'k'], ['i', 'j'], ['j', 'k']]
```

sage.graphs.connectivity.**number_of_biconnected_components**(*G*, *forbidden_vertices=None*)

Return the number of biconnected components.

A biconnected component is a maximal subgraph on two or more vertices that is biconnected, i.e., removing any vertex does not disconnect it.

INPUT:

- `G` – the input graph
- `forbidden_vertices` – list (default: `None`); set of vertices to avoid during the search. This is equivalent to getting the number of biconnected components of the graph after the removal of the forbidden vertices.

See also

- *blocks_and_cut_vertices()*
- *is_biconnected()*

EXAMPLES:

The disjoint union of cycles has as many biconnected components as the number of cycles:

```
sage: G = graphs.CycleGraph(5)
sage: G.number_of_biconnected_components()
1
sage: (3 * G).number_of_biconnected_components()
3
```

```
>>> from sage.all import *
>>> G = graphs.CycleGraph(Integer(5))
>>> G.number_of_biconnected_components()
1
>>> (Integer(3) * G).number_of_biconnected_components()
3
```

A block graph is a connected graph in which every biconnected component (block) is a clique. Hence its number of biconnected components is its number of blocks:

```
sage: number_of_blocks = randint(4, 10)
sage: G = graphs.RandomBlockGraph(number_of_blocks, 5)
sage: G.number_of_biconnected_components() == number_of_blocks
True
```

```
>>> from sage.all import *
>>> number_of_blocks = randint(Integer(4), Integer(10))
>>> G = graphs.RandomBlockGraph(number_of_blocks, Integer(5))
>>> G.number_of_biconnected_components() == number_of_blocks
True
```

By definition, an edge is a biconnected component. Hence, the number of biconnected components of a tree is its number of edges:

```
sage: T = graphs.RandomTree(randint(0, 10))
sage: T.number_of_biconnected_components() == T.size()
True
```

```
>>> from sage.all import *
>>> T = graphs.RandomTree(randint(Integer(0), Integer(10)))
>>> T.number_of_biconnected_components() == T.size()
True
```

An isolated vertex is a block but not a biconnected component:

```
sage: G = Graph(3)
sage: len(G.blocks_and_cut_vertices()[0])
3
sage: G.number_of_biconnected_components()
0
```

```
>>> from sage.all import *
>>> G = Graph(Integer(3))
>>> len(G.blocks_and_cut_vertices()[Integer(0)])
3
>>> G.number_of_biconnected_components()
0
```

Check the behavior of parameter `forbidden_vertices`:

```
sage: G = graphs.CycleGraph(5)
sage: G.number_of_biconnected_components()
1
sage: G.number_of_biconnected_components(forbidden_vertices=[0])
3
```

```
>>> from sage.all import *
>>> G = graphs.CycleGraph(Integer(5))
>>> G.number_of_biconnected_components()
1
>>> G.number_of_biconnected_components(forbidden_vertices=[Integer(0)])
3
```

`sage.graphs.connectivity.`**`number_of_connected_components`**(*G*, *forbidden_vertices=None*)

Return the number of connected components.

INPUT:

- `G` – the input graph
- `forbidden_vertices` – list (default: `None`); set of vertices to avoid during the search

EXAMPLES:

```
sage: from sage.graphs.connectivity import number_of_connected_components
sage: G = Graph({0: [1, 3], 1: [2], 2: [3], 4: [5, 6], 5: [6]})
sage: number_of_connected_components(G)
2
sage: G.number_of_connected_components()
2
sage: D = DiGraph({0: [1, 3], 1: [2], 2: [3], 4: [5, 6], 5: [6]})
sage: number_of_connected_components(D)
```

(continues on next page)

(continued from previous page)

```
2
sage: number_of_connected_components(D, forbidden_vertices=[1, 3])
3
```

```
>>> from sage.all import *
>>> from sage.graphs.connectivity import number_of_connected_components
>>> G = Graph({Integer(0): [Integer(1), Integer(3)], Integer(1): [Integer(2)],␣
↪Integer(2): [Integer(3)], Integer(4): [Integer(5), Integer(6)], Integer(5):␣
↪[Integer(6)]})
>>> number_of_connected_components(G)
2
>>> G.number_of_connected_components()
2
>>> D = DiGraph({Integer(0): [Integer(1), Integer(3)], Integer(1): [Integer(2)],␣
↪Integer(2): [Integer(3)], Integer(4): [Integer(5), Integer(6)], Integer(5):␣
↪[Integer(6)]})
>>> number_of_connected_components(D)
2
>>> number_of_connected_components(D, forbidden_vertices=[Integer(1), Integer(3)])
3
```

sage.graphs.connectivity.**spqr_tree**(*G*, *algorithm='Hopcroft_Tarjan'*, *solver=None*, *verbose=0*, *integrality_tolerance=0.001*)

Return an SPQR-tree representing the triconnected components of the graph.

An SPQR-tree is a tree data structure used to represent the triconnected components of a biconnected (multi)graph and the 2-vertex cuts separating them. A node of a SPQR-tree, and the graph associated with it, can be one of the following four types:

- `'S'` – the associated graph is a cycle with at least three vertices `'S'` stands for `series`
- `'P'` – the associated graph is a dipole graph, a multigraph with two vertices and three or more edges. `'P'` stands for `parallel`
- `'Q'` – the associated graph has a single real edge. This trivial case is necessary to handle the graph that has only one edge
- `'R'` – the associated graph is a 3-connected graph that is not a cycle or dipole. `'R'` stands for `rigid`

This method decomposes a biconnected graph into cycles, cocycles, and 3-connected blocks summed over cocycles, and arranges them as a SPQR-tree. More precisely, it splits the graph at each of its 2-vertex cuts, giving a unique decomposition into 3-connected blocks, cycles and cocycles. The cocycles are dipole graphs with one edge per real edge between the included vertices and one additional (virtual) edge per connected component resulting from deletion of the vertices in the cut. See the Wikipedia article SPQR_tree.

INPUT:

- `G` – the input graph
- `algorithm` – string (default: `'Hopcroft_Tarjan'`); the algorithm to use among:
  - `'Hopcroft_Tarjan'` – default; use the algorithm proposed by Hopcroft and Tarjan in [Hopcroft1973] and later corrected by Gutwenger and Mutzel in [Gut2001]. See *TriconnectivitySPQR*.
  - `'cleave'` – using method *cleave()*
- `solver` – string (default: `None`); specifies a Mixed Integer Linear Programming (MILP) solver to be used. If set to `None`, the default one is used. For more information on MILP solvers and which default solver is

used, see the method `solve` of the class `MixedIntegerLinearProgram`.

- `verbose` – integer (default: 0); sets the level of verbosity. Set to 0 by default, which means quiet.
- `integrality_tolerance` – float; parameter for use with MILP solvers over an inexact base ring; see `MixedIntegerLinearProgram.get_values()`.

OUTPUT: `SPQR-tree` a tree whose vertices are labeled with the block's type and the subgraph of three-blocks in the decomposition.

EXAMPLES:

```
sage: from sage.graphs.connectivity import spqr_tree
sage: G = Graph(2)
sage: for i in range(3):
....:     G.add_clique([0, 1, G.add_vertex(), G.add_vertex()])
sage: Tree = spqr_tree(G)
sage: Tree.order()
4
sage: K4 = graphs.CompleteGraph(4)
sage: all(u[1].is_isomorphic(K4) for u in Tree if u[0] == 'R')
True
sage: from sage.graphs.connectivity import spqr_tree_to_graph
sage: G.is_isomorphic(spqr_tree_to_graph(Tree))
True

sage: G = Graph(2)
sage: for i in range(3):
....:     G.add_path([0, G.add_vertex(), G.add_vertex(), 1])
sage: Tree = spqr_tree(G)
sage: Tree.order()
4
sage: C4 = graphs.CycleGraph(4)
sage: all(u[1].is_isomorphic(C4) for u in Tree if u[0] == 'S')
True
sage: G.is_isomorphic(spqr_tree_to_graph(Tree))
True

sage: G.allow_multiple_edges(True)
sage: G.add_edges(G.edge_iterator())
sage: Tree = spqr_tree(G)
sage: Tree.order()
13
sage: all(u[1].is_isomorphic(C4) for u in Tree if u[0] == 'S')
True
sage: G.is_isomorphic(spqr_tree_to_graph(Tree))
True

sage: G = graphs.CycleGraph(6)
sage: Tree = spqr_tree(G)
sage: Tree.order()
1
sage: G.is_isomorphic(spqr_tree_to_graph(Tree))
True
sage: G.add_edge(0, 3)
```

(continues on next page)

(continued from previous page)

```
sage: Tree = spqr_tree(G)
sage: Tree.order()
3
sage: G.is_isomorphic(spqr_tree_to_graph(Tree))
True

sage: G = Graph('LlCG{O@?GBoMw?')
sage: T = spqr_tree(G, algorithm='Hopcroft_Tarjan')
sage: G.is_isomorphic(spqr_tree_to_graph(T))
True
sage: T2 = spqr_tree(G, algorithm='cleave')                                 #␣
↪needs sage.numerical.mip
sage: G.is_isomorphic(spqr_tree_to_graph(T2))                               #␣
↪needs sage.numerical.mip
True

sage: G = Graph([(0, 1)], multiedges=True)
sage: T = spqr_tree(G, algorithm='cleave')                                  #␣
↪needs sage.numerical.mip
sage: T.vertices(sort=True)                                                 #␣
↪needs sage.numerical.mip
[('Q', Multi-graph on 2 vertices)]
sage: G.is_isomorphic(spqr_tree_to_graph(T))                                #␣
↪needs sage.numerical.mip
True
sage: T = spqr_tree(G, algorithm='Hopcroft_Tarjan')
sage: T.vertices(sort=True)
[('Q', Multi-graph on 2 vertices)]
sage: G.add_edge(0, 1)
sage: spqr_tree(G, algorithm='cleave').vertices(sort=True)                  #␣
↪needs sage.numerical.mip
[('P', Multi-graph on 2 vertices)]

sage: from collections import Counter
sage: G = graphs.PetersenGraph()
sage: T = G.spqr_tree(algorithm='Hopcroft_Tarjan')
sage: Counter(u[0] for u in T)
Counter({'R': 1})
sage: T = G.spqr_tree(algorithm='cleave')                                   #␣
↪needs sage.numerical.mip
sage: Counter(u[0] for u in T)                                              #␣
↪needs sage.numerical.mip
Counter({'R': 1})
sage: for u,v in list(G.edges(labels=False, sort=False)):
....:     G.add_path([u, G.add_vertex(), G.add_vertex(), v])
sage: T = G.spqr_tree(algorithm='Hopcroft_Tarjan')
sage: sorted(Counter(u[0] for u in T).items())
[('P', 15), ('R', 1), ('S', 15)]
sage: T = G.spqr_tree(algorithm='cleave')                                   #␣
↪needs sage.numerical.mip
sage: G.is_isomorphic(spqr_tree_to_graph(T))                                #␣
↪needs sage.numerical.mip
```

(continues on next page)

(continued from previous page)

```
True
sage: for u,v in list(G.edges(labels=False, sort=False)):
....:     G.add_path([u, G.add_vertex(), G.add_vertex(), v])
sage: T = G.spqr_tree(algorithm='Hopcroft_Tarjan')
sage: sorted(Counter(u[0] for u in T).items())
[('P', 60), ('R', 1), ('S', 75)]
sage: T = G.spqr_tree(algorithm='cleave')       # long time                    #␣
↪needs sage.numerical.mip
sage: G.is_isomorphic(spqr_tree_to_graph(T))    # long time                    #␣
↪needs sage.numerical.mip
True
```

```
>>> from sage.all import *
>>> from sage.graphs.connectivity import spqr_tree
>>> G = Graph(Integer(2))
>>> for i in range(Integer(3)):
...     G.add_clique([Integer(0), Integer(1), G.add_vertex(), G.add_vertex()])
>>> Tree = spqr_tree(G)
>>> Tree.order()
4
>>> K4 = graphs.CompleteGraph(Integer(4))
>>> all(u[Integer(1)].is_isomorphic(K4) for u in Tree if u[Integer(0)] == 'R')
True
>>> from sage.graphs.connectivity import spqr_tree_to_graph
>>> G.is_isomorphic(spqr_tree_to_graph(Tree))
True

>>> G = Graph(Integer(2))
>>> for i in range(Integer(3)):
...     G.add_path([Integer(0), G.add_vertex(), G.add_vertex(), Integer(1)])
>>> Tree = spqr_tree(G)
>>> Tree.order()
4
>>> C4 = graphs.CycleGraph(Integer(4))
>>> all(u[Integer(1)].is_isomorphic(C4) for u in Tree if u[Integer(0)] == 'S')
True
>>> G.is_isomorphic(spqr_tree_to_graph(Tree))
True

>>> G.allow_multiple_edges(True)
>>> G.add_edges(G.edge_iterator())
>>> Tree = spqr_tree(G)
>>> Tree.order()
13
>>> all(u[Integer(1)].is_isomorphic(C4) for u in Tree if u[Integer(0)] == 'S')
True
>>> G.is_isomorphic(spqr_tree_to_graph(Tree))
True

>>> G = graphs.CycleGraph(Integer(6))
>>> Tree = spqr_tree(G)
>>> Tree.order()
```

(continues on next page)

(continued from previous page)

```
1
>>> G.is_isomorphic(spqr_tree_to_graph(Tree))
True
>>> G.add_edge(Integer(0), Integer(3))
>>> Tree = spqr_tree(G)
>>> Tree.order()
3
>>> G.is_isomorphic(spqr_tree_to_graph(Tree))
True

>>> G = Graph('LlCG{O@?GBoMw?')
>>> T = spqr_tree(G, algorithm='Hopcroft_Tarjan')
>>> G.is_isomorphic(spqr_tree_to_graph(T))
True
>>> T2 = spqr_tree(G, algorithm='cleave')                                #␣
↪needs sage.numerical.mip
>>> G.is_isomorphic(spqr_tree_to_graph(T2))                              #␣
↪needs sage.numerical.mip
True

>>> G = Graph([(Integer(0), Integer(1))], multiedges=True)
>>> T = spqr_tree(G, algorithm='cleave')                                 #␣
↪needs sage.numerical.mip
>>> T.vertices(sort=True)                                                #␣
↪needs sage.numerical.mip
[('Q', Multi-graph on 2 vertices)]
>>> G.is_isomorphic(spqr_tree_to_graph(T))                               #␣
↪needs sage.numerical.mip
True
>>> T = spqr_tree(G, algorithm='Hopcroft_Tarjan')
>>> T.vertices(sort=True)
[('Q', Multi-graph on 2 vertices)]
>>> G.add_edge(Integer(0), Integer(1))
>>> spqr_tree(G, algorithm='cleave').vertices(sort=True)                 #␣
↪needs sage.numerical.mip
[('P', Multi-graph on 2 vertices)]

>>> from collections import Counter
>>> G = graphs.PetersenGraph()
>>> T = G.spqr_tree(algorithm='Hopcroft_Tarjan')
>>> Counter(u[Integer(0)] for u in T)
Counter({'R': 1})
>>> T = G.spqr_tree(algorithm='cleave')                                  #␣
↪needs sage.numerical.mip
>>> Counter(u[Integer(0)] for u in T)                                     ␣
↪     # needs sage.numerical.mip
Counter({'R': 1})
>>> for u,v in list(G.edges(labels=False, sort=False)):
...     G.add_path([u, G.add_vertex(), G.add_vertex(), v])
>>> T = G.spqr_tree(algorithm='Hopcroft_Tarjan')
>>> sorted(Counter(u[Integer(0)] for u in T).items())
[('P', 15), ('R', 1), ('S', 15)]
```

(continues on next page)

(continued from previous page)

```
>>> T = G.spqr_tree(algorithm='cleave')                                    #␣
↪needs sage.numerical.mip
>>> G.is_isomorphic(spqr_tree_to_graph(T))                                 #␣
↪needs sage.numerical.mip
True
>>> for u,v in list(G.edges(labels=False, sort=False)):
...     G.add_path([u, G.add_vertex(), G.add_vertex(), v])
>>> T = G.spqr_tree(algorithm='Hopcroft_Tarjan')
>>> sorted(Counter(u[Integer(0)] for u in T).items())
[('P', 60), ('R', 1), ('S', 75)]
>>> T = G.spqr_tree(algorithm='cleave')       # long time                  #␣
↪needs sage.numerical.mip
>>> G.is_isomorphic(spqr_tree_to_graph(T))    # long time                  #␣
↪needs sage.numerical.mip
True
```

sage.graphs.connectivity.**spqr_tree_to_graph**(*T*)

Return the graph represented by the SPQR-tree T.

The main purpose of this method is to test *spqr_tree()*.

INPUT:

- `T` – a SPQR tree as returned by *spqr_tree()*

OUTPUT: a (multi) graph

EXAMPLES:

Wikipedia article SPQR_tree reference paper example:

```
sage: from sage.graphs.connectivity import spqr_tree
sage: from sage.graphs.connectivity import spqr_tree_to_graph
sage: G = Graph([(1, 2), (1, 4), (1, 8), (1, 12), (3, 4), (2, 3),
....: (2, 13), (3, 13), (4, 5), (4, 7), (5, 6), (5, 8), (5, 7), (6, 7),
....: (8, 11), (8, 9), (8, 12), (9, 10), (9, 11), (9, 12), (10, 12)])
sage: T = spqr_tree(G)
sage: H = spqr_tree_to_graph(T)
sage: H.is_isomorphic(G)
True
```

```
>>> from sage.all import *
>>> from sage.graphs.connectivity import spqr_tree
>>> from sage.graphs.connectivity import spqr_tree_to_graph
>>> G = Graph([(Integer(1), Integer(2)), (Integer(1), Integer(4)), (Integer(1),␣
↪Integer(8)), (Integer(1), Integer(12)), (Integer(3), Integer(4)), (Integer(2),␣
↪Integer(3)),
... (Integer(2), Integer(13)), (Integer(3), Integer(13)), (Integer(4),␣
↪Integer(5)), (Integer(4), Integer(7)), (Integer(5), Integer(6)), (Integer(5),␣
↪Integer(8)), (Integer(5), Integer(7)), (Integer(6), Integer(7)),
... (Integer(8), Integer(11)), (Integer(8), Integer(9)), (Integer(8),␣
↪Integer(12)), (Integer(9), Integer(10)), (Integer(9), Integer(11)), (Integer(9),
↪ Integer(12)), (Integer(10), Integer(12))])
>>> T = spqr_tree(G)
>>> H = spqr_tree_to_graph(T)
```

(continues on next page)

(continued from previous page)

```
>>> H.is_isomorphic(G)
True
```

A small multigraph

```
sage: G = Graph([(0, 2), (0, 2), (1, 3), (2, 3)], multiedges=True)
sage: for i in range(3):
....:     G.add_clique([0, 1, G.add_vertex(), G.add_vertex()])
sage: for i in range(3):
....:     G.add_clique([2, 3, G.add_vertex(), G.add_vertex()])
sage: T = spqr_tree(G)
sage: H = spqr_tree_to_graph(T)
sage: H.is_isomorphic(G)
True
```

```
>>> from sage.all import *
>>> G = Graph([(Integer(0), Integer(2)), (Integer(0), Integer(2)), (Integer(1),␣
↪Integer(3)), (Integer(2), Integer(3))], multiedges=True)
>>> for i in range(Integer(3)):
...     G.add_clique([Integer(0), Integer(1), G.add_vertex(), G.add_vertex()])
>>> for i in range(Integer(3)):
...     G.add_clique([Integer(2), Integer(3), G.add_vertex(), G.add_vertex()])
>>> T = spqr_tree(G)
>>> H = spqr_tree_to_graph(T)
>>> H.is_isomorphic(G)
True
```

sage.graphs.connectivity.**strong_articulation_points**(*G*)

Return the strong articulation points of this digraph.

A vertex is a strong articulation point if its deletion increases the number of strongly connected components. This method implements the algorithm described in [ILS2012]. The time complexity is dominated by the time complexity of the immediate dominators finding algorithm.

OUTPUT: the list of strong articulation points

EXAMPLES:

Two cliques sharing a vertex:

```
sage: from sage.graphs.connectivity import strong_articulation_points
sage: D = digraphs.Complete(4)
sage: D.add_clique([3, 4, 5, 6])
sage: strong_articulation_points(D)
[3]
sage: D.strong_articulation_points()
[3]
```

```
>>> from sage.all import *
>>> from sage.graphs.connectivity import strong_articulation_points
>>> D = digraphs.Complete(Integer(4))
>>> D.add_clique([Integer(3), Integer(4), Integer(5), Integer(6)])
>>> strong_articulation_points(D)
[3]
```

(continues on next page)

(continued from previous page)

```
>>> D.strong_articulation_points()
[3]
```

Two cliques connected by some arcs:

```
sage: D = digraphs.Complete(4) * 2
sage: D.add_edges([(0, 4), (7, 3)])
sage: sorted(strong_articulation_points(D))
[0, 3, 4, 7]
sage: D.add_edge(1, 5)
sage: sorted(strong_articulation_points(D))
[3, 7]
sage: D.add_edge(6, 2)
sage: strong_articulation_points(D)
[]
```

```
>>> from sage.all import *
>>> D = digraphs.Complete(Integer(4)) * Integer(2)
>>> D.add_edges([(Integer(0), Integer(4)), (Integer(7), Integer(3))])
>>> sorted(strong_articulation_points(D))
[0, 3, 4, 7]
>>> D.add_edge(Integer(1), Integer(5))
>>> sorted(strong_articulation_points(D))
[3, 7]
>>> D.add_edge(Integer(6), Integer(2))
>>> strong_articulation_points(D)
[]
```

See also

- *strongly_connected_components()*
- *dominator_tree()*

sage.graphs.connectivity.**strongly_connected_component_containing_vertex**(G, v)

Return the strongly connected component containing a given vertex.

INPUT:

- `G` – the input DiGraph
- `v` – a vertex

EXAMPLES:

In the symmetric digraph of a graph, the strongly connected components are the connected components:

```
sage: from sage.graphs.connectivity import strongly_connected_component_
↪containing_vertex
sage: g = graphs.PetersenGraph()
sage: d = DiGraph(g)
sage: strongly_connected_component_containing_vertex(d, 0)
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

(continues on next page)

(continued from previous page)

```
sage: d.strongly_connected_component_containing_vertex(0)
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

```
>>> from sage.all import *
>>> from sage.graphs.connectivity import strongly_connected_component_containing_
↪vertex
>>> g = graphs.PetersenGraph()
>>> d = DiGraph(g)
>>> strongly_connected_component_containing_vertex(d, Integer(0))
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
>>> d.strongly_connected_component_containing_vertex(Integer(0))
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

```
sage: g = DiGraph([(0, 1), (1, 0), (1, 2), (2, 3), (3, 2)])
sage: strongly_connected_component_containing_vertex(g, 0)
[0, 1]
```

```
>>> from sage.all import *
>>> g = DiGraph([(Integer(0), Integer(1)), (Integer(1), Integer(0)), (Integer(1),␣
↪Integer(2)), (Integer(2), Integer(3)), (Integer(3), Integer(2))])
>>> strongly_connected_component_containing_vertex(g, Integer(0))
[0, 1]
```

sage.graphs.connectivity.**strongly_connected_components_digraph**(*G*, *keep_labels=False*)

Return the digraph of the strongly connected components.

The digraph of the strongly connected components of a graph G has a vertex per strongly connected component included in G. There is an edge from a component C_1 to a component C_2 if there is an edge in G from a vertex $u_1 \in C_1$ to a vertex $u_2 \in C_2$.

INPUT:

- `G` – the input DiGraph
- `keep_labels` – boolean (default: `False`); when `keep_labels=True`, the resulting digraph has an edge from a component C_i to a component C_j for each edge in G from a vertex $u_i \in C_i$ to a vertex $u_j \in C_j$. Hence the resulting digraph may have loops and multiple edges. However, edges in the result with same source, target, and label are not duplicated (see examples below). When `keep_labels=False`, the return digraph is simple, so without loops nor multiple edges, and edges are unlabelled.

EXAMPLES:

Such a digraph is always acyclic:

```
sage: from sage.graphs.connectivity import strongly_connected_components_digraph
sage: g = digraphs.RandomDirectedGNP(15, .1)
sage: scc_digraph = strongly_connected_components_digraph(g)
sage: scc_digraph.is_directed_acyclic()
True
sage: scc_digraph = g.strongly_connected_components_digraph()
sage: scc_digraph.is_directed_acyclic()
True
```

```
>>> from sage.all import *
>>> from sage.graphs.connectivity import strongly_connected_components_digraph
>>> g = digraphs.RandomDirectedGNP(Integer(15), RealNumber('.1'))
>>> scc_digraph = strongly_connected_components_digraph(g)
>>> scc_digraph.is_directed_acyclic()
True
>>> scc_digraph = g.strongly_connected_components_digraph()
>>> scc_digraph.is_directed_acyclic()
True
```

The vertices of the digraph of strongly connected components are exactly the strongly connected components:

```
sage: g = digraphs.ButterflyGraph(2)
sage: scc_digraph = strongly_connected_components_digraph(g)
sage: g.is_directed_acyclic()
True
sage: V_scc = list(scc_digraph)
sage: all(Set(scc) in V_scc for scc in g.strongly_connected_components())
True
```

```
>>> from sage.all import *
>>> g = digraphs.ButterflyGraph(Integer(2))
>>> scc_digraph = strongly_connected_components_digraph(g)
>>> g.is_directed_acyclic()
True
>>> V_scc = list(scc_digraph)
>>> all(Set(scc) in V_scc for scc in g.strongly_connected_components())
True
```

The following digraph has three strongly connected components, and the digraph of those is a *TransitiveTournament()*:

```
sage: g = DiGraph({0: {1: "01", 2: "02", 3: "03"}, 1: {2: "12"}, 2:{1: "21", 3:
↪"23"}})
sage: scc_digraph = strongly_connected_components_digraph(g)
sage: scc_digraph.is_isomorphic(digraphs.TransitiveTournament(3))
True
```

```
>>> from sage.all import *
>>> g = DiGraph({Integer(0): {Integer(1): "01", Integer(2): "02", Integer(3): "03
↪"}, Integer(1): {Integer(2): "12"}, Integer(2):{Integer(1): "21", Integer(3):
↪"23"}})
>>> scc_digraph = strongly_connected_components_digraph(g)
>>> scc_digraph.is_isomorphic(digraphs.TransitiveTournament(Integer(3)))
True
```

By default, the labels are discarded, and the result has no loops nor multiple edges. If `keep_labels` is `True`, then the labels are kept, and the result is a multi digraph, possibly with multiple edges and loops. However, edges in the result with same source, target, and label are not duplicated (see the edges from 0 to the strongly connected component $\{1, 2\}$ below):

```
sage: g = DiGraph({0: {1: "0-12", 2: "0-12", 3: "0-3"}, 1: {2: "1-2", 3: "1-3"},␣
↪2: {1: "2-1", 3: "2-3"}})
```

(continues on next page)

(continued from previous page)

```
sage: g.order(), g.size()
(4, 7)
sage: scc_digraph = strongly_connected_components_digraph(g, keep_labels=True)
sage: (scc_digraph.order(), scc_digraph.size())
(3, 6)
sage: set(g.edge_labels()) == set(scc_digraph.edge_labels())
True
```

```
>>> from sage.all import *
>>> g = DiGraph({Integer(0): {Integer(1): "0-12", Integer(2): "0-12", Integer(3):
↪"0-3"}, Integer(1): {Integer(2): "1-2", Integer(3): "1-3"}, Integer(2):
↪{Integer(1): "2-1", Integer(3): "2-3"}})
>>> g.order(), g.size()
(4, 7)
>>> scc_digraph = strongly_connected_components_digraph(g, keep_labels=True)
>>> (scc_digraph.order(), scc_digraph.size())
(3, 6)
>>> set(g.edge_labels()) == set(scc_digraph.edge_labels())
True
```

sage.graphs.connectivity.**strongly_connected_components_subgraphs**(*G*, *immutable=None*)

Return the strongly connected components as a list of subgraphs.

INPUT:

- `G` – a directed graph
- `immutable` – boolean (default: `None`); whether to create a mutable/immutable graphs. `immutable=None` (default) means that the graph and the subgraphs of its connected components will behave the same way.

EXAMPLES:

In the symmetric digraph of a graph, the strongly connected components are the connected components:

```
sage: from sage.graphs.connectivity import strongly_connected_components_subgraphs
sage: g = graphs.PetersenGraph()
sage: d = DiGraph(g)
sage: strongly_connected_components_subgraphs(d)
[Subgraph of (Petersen graph): Digraph on 10 vertices]
sage: d.strongly_connected_components_subgraphs()
[Subgraph of (Petersen graph): Digraph on 10 vertices]
```

```
>>> from sage.all import *
>>> from sage.graphs.connectivity import strongly_connected_components_subgraphs
>>> g = graphs.PetersenGraph()
>>> d = DiGraph(g)
>>> strongly_connected_components_subgraphs(d)
[Subgraph of (Petersen graph): Digraph on 10 vertices]
>>> d.strongly_connected_components_subgraphs()
[Subgraph of (Petersen graph): Digraph on 10 vertices]
```

```
sage: g = DiGraph([(0, 1), (1, 0), (1, 2), (2, 3), (3, 2)])
sage: strongly_connected_components_subgraphs(g)
[Subgraph of (): Digraph on 2 vertices, Subgraph of (): Digraph on 2 vertices]
```

```
>>> from sage.all import *
>>> g = DiGraph([(Integer(0), Integer(1)), (Integer(1), Integer(0)), (Integer(1),␣
↪Integer(2)), (Integer(2), Integer(3)), (Integer(3), Integer(2))])
>>> strongly_connected_components_subgraphs(g)
[Subgraph of (): Digraph on 2 vertices, Subgraph of (): Digraph on 2 vertices]
```

sage.graphs.connectivity.**vertex_connectivity**(*G*, *value_only=True*, *sets=False*, *k=None*, *solver=None*, *verbose=0*, *integrality_tolerance=0.001*)

Return the vertex connectivity of the graph.

For more information, see the Wikipedia article Connectivity_(graph_theory) and the Wikipedia article K-vertex-connected_graph.

Note

- When the graph is directed, this method actually computes the *strong* connectivity, (i.e. a directed graph is strongly k-connected if there are k vertex disjoint paths between any two vertices u, v). If you do not want to consider strong connectivity, the best is probably to convert your `DiGraph` object to a `Graph` object, and compute the connectivity of this other graph.
- By convention, a complete graph on n vertices is $n-1$ connected. In this case, no certificate can be given as there is no pair of vertices split by a cut of order $k-1$. For this reason, the certificates returned in this situation are empty.

INPUT:

- `G` – the input Sage (Di)Graph
- `value_only` – boolean (default: `True`)
  - When set to `True` (default), only the value is returned.
  - When set to `False`, both the value and a minimum vertex cut are returned.
- **`sets` – boolean (default: `False`); whether to also return the two**
  sets of vertices that are disconnected by the cut (implies `value_only=False`)
- `k` – integer (default: `None`); when specified, check if the vertex connectivity of the (di)graph is larger or equal to k. The method thus outputs a boolean only.
- `solver` – string (default: `None`); specifies a Mixed Integer Linear Programming (MILP) solver to be used. If set to `None`, the default one is used. For more information on MILP solvers and which default solver is used, see the method `solve` of the class `MixedIntegerLinearProgram`.
- `verbose` – integer (default: 0); sets the level of verbosity. Set to 0 by default, which means quiet.
- `integrality_tolerance` – float; parameter for use with MILP solvers over an inexact base ring; see `MixedIntegerLinearProgram.get_values()`.

EXAMPLES:

A basic application on a `PappusGraph`:

```
sage: from sage.graphs.connectivity import vertex_connectivity
sage: g = graphs.PappusGraph()
sage: vertex_connectivity(g)
↪# needs sage.numerical.mip
```

(continues on next page)

(continued from previous page)

```
3
sage: g.vertex_connectivity()
↪# needs sage.numerical.mip
3
```

```
>>> from sage.all import *
>>> from sage.graphs.connectivity import vertex_connectivity
>>> g = graphs.PappusGraph()
>>> vertex_connectivity(g)                                                    #␣
↪needs sage.numerical.mip
3
>>> g.vertex_connectivity()                                                   #␣
↪needs sage.numerical.mip
3
```

In a grid, the vertex connectivity is equal to the minimum degree, in which case one of the two sets is of cardinality 1:

```
sage: g = graphs.GridGraph([ 3,3 ])
sage: [value, cut, [ setA, setB ]] = vertex_connectivity(g, sets=True)
↪# needs sage.numerical.mip
sage: len(setA) == 1 or len(setB) == 1
↪# needs sage.numerical.mip
True
```

```
>>> from sage.all import *
>>> g = graphs.GridGraph([ Integer(3),Integer(3) ])
>>> [value, cut, [ setA, setB ]] = vertex_connectivity(g, sets=True)          #␣
↪needs sage.numerical.mip
>>> len(setA) == Integer(1) or len(setB) == Integer(1)                          ␣
↪               # needs sage.numerical.mip
True
```

A vertex cut in a tree is any internal vertex:

```
sage: tree = graphs.RandomTree(15)
sage: val, [cut_vertex] = vertex_connectivity(tree, value_only=False)
↪# needs sage.numerical.mip
sage: tree.degree(cut_vertex) > 1
↪# needs sage.numerical.mip
True
```

```
>>> from sage.all import *
>>> tree = graphs.RandomTree(Integer(15))
>>> val, [cut_vertex] = vertex_connectivity(tree, value_only=False)           #␣
↪needs sage.numerical.mip
>>> tree.degree(cut_vertex) > Integer(1)                                        ␣
↪      # needs sage.numerical.mip
True
```

When `value_only = True`, this function is optimized for small connectivity values and does not need to build a linear program.

It is the case for connected graphs which are not connected:

```
sage: g = 2 * graphs.PetersenGraph()
sage: vertex_connectivity(g)
↪# needs sage.numerical.mip
0
```

```
>>> from sage.all import *
>>> g = Integer(2) * graphs.PetersenGraph()
>>> vertex_connectivity(g)                                                    #␣
↪needs sage.numerical.mip
0
```

Or if they are just 1-connected:

```
sage: g = graphs.PathGraph(10)
sage: vertex_connectivity(g)
↪# needs sage.numerical.mip
1
```

```
>>> from sage.all import *
>>> g = graphs.PathGraph(Integer(10))
>>> vertex_connectivity(g)                                                    #␣
↪needs sage.numerical.mip
1
```

For directed graphs, the strong connectivity is tested through the dedicated function:

```
sage: g = digraphs.ButterflyGraph(3)
sage: vertex_connectivity(g)
↪# needs sage.numerical.mip
0
```

```
>>> from sage.all import *
>>> g = digraphs.ButterflyGraph(Integer(3))
>>> vertex_connectivity(g)                                                    #␣
↪needs sage.numerical.mip
0
```

A complete graph on 10 vertices is 9-connected:

```
sage: g = graphs.CompleteGraph(10)
sage: vertex_connectivity(g)
↪# needs sage.numerical.mip
9
```

```
>>> from sage.all import *
>>> g = graphs.CompleteGraph(Integer(10))
>>> vertex_connectivity(g)                                                    #␣
↪needs sage.numerical.mip
9
```

A complete digraph on 10 vertices is 9-connected:

```
sage: g = DiGraph(graphs.CompleteGraph(10))
sage: vertex_connectivity(g)
↪# needs sage.numerical.mip
9
```

```
>>> from sage.all import *
>>> g = DiGraph(graphs.CompleteGraph(Integer(10)))
>>> vertex_connectivity(g)                                                    #␣
↪needs sage.numerical.mip
9
```

When parameter `k` is set, we only check for the existence of a vertex cut of order at least `k`:

```
sage: g = graphs.PappusGraph()
sage: vertex_connectivity(g, k=3)
↪# needs sage.numerical.mip
True
sage: vertex_connectivity(g, k=4)
↪# needs sage.numerical.mip
False
```

```
>>> from sage.all import *
>>> g = graphs.PappusGraph()
>>> vertex_connectivity(g, k=Integer(3))                                      ␣
↪      # needs sage.numerical.mip
True
>>> vertex_connectivity(g, k=Integer(4))                                      ␣
↪      # needs sage.numerical.mip
False
```

5.47 Edge connectivity

This module implements methods for computing the edge-connectivity of graphs and digraphs. It also implements methods to extract k edge-disjoint spanning trees from a $2k$ edge-connected graph or a k edge-connected digraph, and to pack k edge-disjoint spanning arborescences in a directed graph following the matroid approach of [Gabow1995].

Todo

- Implement the tree-packing strengthening proposed in [BHKP2008]
- Fix the DFS-based speed-up initialization of [GKLP2021] for digraphs with multiple edges, for which it is currently disabled
- Extend to weighted digraphs

class sage.graphs.edge_connectivity.**GabowEdgeConnectivity**

Bases: object

Gabow's algorithm for finding the edge connectivity of digraphs.

This class implements the algorithm proposed in [Gabow1995] for finding the edge connectivity of a directed graph and k edge disjoint spanning trees if the digraph is k edge connected.

Digraphs with multiple edges are supported: every parallel arc counts separately, both for the edge connectivity and for the arborescence packing. Loops are tolerated and ignored.

INPUT:

- D – a *DiGraph*

EXAMPLES:

A random d-regular digraph is d-edge-connected:

```
sage: from sage.graphs.edge_connectivity import GabowEdgeConnectivity
sage: D = DiGraph(graphs.RandomRegular(6, 50))                                  #␣
↪needs networkx
sage: while not D.is_strongly_connected():                                      #␣
↪needs networkx
....:     D = DiGraph(graphs.RandomRegular(6, 50))
sage: GabowEdgeConnectivity(D).edge_connectivity()                              #␣
↪needs networkx
6
```

```
>>> from sage.all import *
>>> from sage.graphs.edge_connectivity import GabowEdgeConnectivity
>>> D = DiGraph(graphs.RandomRegular(Integer(6), Integer(50)))                  ␣
↪              # needs networkx
>>> while not D.is_strongly_connected():                                        #␣
↪needs networkx
...     D = DiGraph(graphs.RandomRegular(Integer(6), Integer(50)))
>>> GabowEdgeConnectivity(D).edge_connectivity()                                #␣
↪needs networkx
6
```

A complete digraph with n vertices is $n-1$-edge-connected:

```
sage: from sage.graphs.edge_connectivity import GabowEdgeConnectivity
sage: D = DiGraph(digraphs.Complete(10))
sage: GabowEdgeConnectivity(D, use_rec=True).edge_connectivity()
9
```

```
>>> from sage.all import *
>>> from sage.graphs.edge_connectivity import GabowEdgeConnectivity
>>> D = DiGraph(digraphs.Complete(Integer(10)))
>>> GabowEdgeConnectivity(D, use_rec=True).edge_connectivity()
9
```

Check that we get the same result when with and without the DFS-based speed-up initialization proposed in [GKLP2021]:

```
sage: # needs networkx
sage: G = graphs.RandomBarabasiAlbert(100, 2)
sage: D = DiGraph(G)
sage: ec1 = GabowEdgeConnectivity(D,
....:                             dfs_preprocessing=False).edge_connectivity()
sage: ec2 = GabowEdgeConnectivity(D,
....:                             dfs_preprocessing=True).edge_connectivity()
```

(continues on next page)

(continued from previous page)

```
sage: ec3 = GabowEdgeConnectivity(D, dfs_preprocessing=True,
....:                             use_rec=True).edge_connectivity()
sage: ec1 == ec2 and ec2 == ec3
True
```

```
>>> from sage.all import *
>>> # needs networkx
>>> G = graphs.RandomBarabasiAlbert(Integer(100), Integer(2))
>>> D = DiGraph(G)
>>> ec1 = GabowEdgeConnectivity(D,
...                             dfs_preprocessing=False).edge_connectivity()
>>> ec2 = GabowEdgeConnectivity(D,
...                             dfs_preprocessing=True).edge_connectivity()
>>> ec3 = GabowEdgeConnectivity(D, dfs_preprocessing=True,
...                             use_rec=True).edge_connectivity()
>>> ec1 == ec2 and ec2 == ec3
True
```

edge_connectivity()

Return the edge connectivity of the digraph.

Digraphs with multiple edges are supported: every parallel arc counts separately in the cuts.

EXAMPLES:

```
sage: from sage.graphs.edge_connectivity import GabowEdgeConnectivity
sage: D = digraphs.Complete(5)
sage: GabowEdgeConnectivity(D).edge_connectivity()
4
```

```
>>> from sage.all import *
>>> from sage.graphs.edge_connectivity import GabowEdgeConnectivity
>>> D = digraphs.Complete(Integer(5))
>>> GabowEdgeConnectivity(D).edge_connectivity()
4
```

On a digraph with multiple edges:

```
sage: D = DiGraph([(0, 1), (0, 1), (1, 0)], multiedges=True)
sage: GabowEdgeConnectivity(D).edge_connectivity()
1
```

```
>>> from sage.all import *
>>> D = DiGraph([(Integer(0), Integer(1)), (Integer(0), Integer(1)),␣
↪(Integer(1), Integer(0))], multiedges=True)
>>> GabowEdgeConnectivity(D).edge_connectivity()
1
```

edge_disjoint_spanning_trees(*k=None*, *root=None*, *labels=False*)

Return `k` edge-disjoint spanning arborescences rooted at `root`.

The returned arborescences are out-arborescences (rooted spanning trees with all edges directed away from the root). Following [Gabow1995] and Edmonds' branching theorem, the maximum number of edge-disjoint

spanning out-arborescences rooted at a vertex `r` equals the minimum `r`-cut of the digraph. When the digraph is strongly connected, this is at least the edge connectivity `ec` of the digraph.

INPUT:

- `k` – integer or `None` (default: `None`); the number of edge-disjoint spanning arborescences to return. If `None`, the method returns `self.ec` arborescences (the value computed by *edge_connectivity()*).
- `root` – vertex of the input digraph or `None` (default: `None`); the common root of the returned arborescences. If `None`, defaults to the first vertex of the digraph (the same convention used by the MILP backend in *edge_disjoint_spanning_trees()*).
- `labels` – boolean (default: `False`); whether the edges of the returned arborescences carry the labels of the corresponding input edges. With labels, parallel edges of a digraph with multiple edges remain distinguishable in the output: every returned edge corresponds to a distinct input edge.

OUTPUT:

A list of *DiGraph*, each on the same vertex set as the input. Every returned arborescence is rooted at `root`: the root has indegree 0, every other vertex has indegree 1, and every vertex is reachable from `root`. The returned arborescences are pairwise edge-disjoint.

ALGORITHM:

This uses the matroid-intersection approach of [Gabow1995], in two phases. Phase 1 builds a `k`-intersection whose union is a `k`-in-regular subgraph rooted at `root` (every non-root vertex has in-degree `k`). Phase 2 decomposes that union into `k` spanning out-arborescences one at a time (`find_tree`): it grows the set of vertices reachable from `root`, and whenever the next edge it needs is already used by another tree, an augmenting-path *swap* (`search_step`) frees a usable edge while keeping the other trees valid.

The extraction scheme follows the practical implementation studied in [GKMN2022]. Phase 1 runs in time $O(km\log(n^2/m))$ [Gabow1995]. Phase 2 performs k extraction rounds, each rebuilding an intersection on the remaining edge pool before carving out one arborescence, and its swaps dominate the running time in practice: the observed total grows roughly like k^2m (about n^4 on complete digraphs, where $k = n - 1$).

EXAMPLES:

On a complete digraph, the algorithm returns `n-1` arborescences:

```
sage: from sage.graphs.edge_connectivity import GabowEdgeConnectivity
sage: D = digraphs.Complete(5)
sage: trees = GabowEdgeConnectivity(D).edge_disjoint_spanning_trees(k=4)
sage: len(trees)
4
sage: all(T.num_edges() == 4 for T in trees)
True
```

```
>>> from sage.all import *
>>> from sage.graphs.edge_connectivity import GabowEdgeConnectivity
>>> D = digraphs.Complete(Integer(5))
>>> trees = GabowEdgeConnectivity(D).edge_disjoint_spanning_
↪trees(k=Integer(4))
>>> len(trees)
4
>>> all(T.num_edges() == Integer(4) for T in trees)
True
```

By default (`k=None`) the method returns `ec` arborescences, where `ec` is the edge connectivity:

```
sage: from sage.graphs.edge_connectivity import GabowEdgeConnectivity
sage: D = digraphs.Complete(5)
sage: G = GabowEdgeConnectivity(D)
sage: G.edge_connectivity()
4
sage: len(G.edge_disjoint_spanning_trees())
4
```

```
>>> from sage.all import *
>>> from sage.graphs.edge_connectivity import GabowEdgeConnectivity
>>> D = digraphs.Complete(Integer(5))
>>> G = GabowEdgeConnectivity(D)
>>> G.edge_connectivity()
4
>>> len(G.edge_disjoint_spanning_trees())
4
```

The arborescences can be rooted at any vertex; the root then has indegree 0, every other vertex has indegree 1, and every vertex is reachable from the root:

```
sage: D = digraphs.Complete(5)
sage: trees = GabowEdgeConnectivity(D).edge_disjoint_spanning_trees(k=4,␣
↪root=2)
sage: all(T.in_degree(2) == 0 for T in trees)
True
sage: all(T.in_degree(v) == 1 for T in trees for v in D if v != 2)
True
sage: all(set(T.depth_first_search(2)) == set(D) for T in trees)
True
```

```
>>> from sage.all import *
>>> D = digraphs.Complete(Integer(5))
>>> trees = GabowEdgeConnectivity(D).edge_disjoint_spanning_
↪trees(k=Integer(4), root=Integer(2))
>>> all(T.in_degree(Integer(2)) == Integer(0) for T in trees)
True
>>> all(T.in_degree(v) == Integer(1) for T in trees for v in D if v !=␣
↪Integer(2))
True
>>> all(set(T.depth_first_search(Integer(2))) == set(D) for T in trees)
True
```

The returned arborescences are pairwise edge-disjoint:

```
sage: D = digraphs.Complete(5)
sage: trees = GabowEdgeConnectivity(D).edge_disjoint_spanning_trees(k=4)
sage: all_edges = sum((T.edges(labels=False, sort=False) for T in trees), [])
sage: len(all_edges) == len(set(all_edges))
True
```

```
>>> from sage.all import *
>>> D = digraphs.Complete(Integer(5))
```

(continues on next page)

(continued from previous page)

```
>>> trees = GabowEdgeConnectivity(D).edge_disjoint_spanning_
↪trees(k=Integer(4))
>>> all_edges = sum((T.edges(labels=False, sort=False) for T in trees), [])
>>> len(all_edges) == len(set(all_edges))
True
```

On a directed cycle the edge connectivity is 1, so the packing has a single arborescence: the directed path from the root:

```
sage: from sage.graphs.edge_connectivity import GabowEdgeConnectivity
sage: D = digraphs.Circuit(4)
sage: trees = GabowEdgeConnectivity(D).edge_disjoint_spanning_trees(root=0)
sage: len(trees)
1
sage: sorted(trees[0].edges(labels=False, sort=False))
[(0, 1), (1, 2), (2, 3)]
```

```
>>> from sage.all import *
>>> from sage.graphs.edge_connectivity import GabowEdgeConnectivity
>>> D = digraphs.Circuit(Integer(4))
>>> trees = GabowEdgeConnectivity(D).edge_disjoint_spanning_
↪trees(root=Integer(0))
>>> len(trees)
1
>>> sorted(trees[Integer(0)].edges(labels=False, sort=False))
[(0, 1), (1, 2), (2, 3)]
```

On a digraph with multiple edges, `labels=True` shows which copy of a parallel edge each arborescence uses; every returned edge corresponds to a distinct input edge:

```
sage: D = DiGraph([(0, 1, 'a'), (0, 1, 'b'), (1, 0, 'c'), (1, 0, 'd')],
....:             multiedges=True)
sage: trees = GabowEdgeConnectivity(D).edge_disjoint_spanning_trees(k=2,␣
↪root=0, labels=True)
sage: sorted(e[2] for T in trees for e in T.edge_iterator())
['a', 'b']
```

```
>>> from sage.all import *
>>> D = DiGraph([(Integer(0), Integer(1), 'a'), (Integer(0), Integer(1), 'b'),
↪ (Integer(1), Integer(0), 'c'), (Integer(1), Integer(0), 'd')],
...             multiedges=True)
>>> trees = GabowEdgeConnectivity(D).edge_disjoint_spanning_
↪trees(k=Integer(2), root=Integer(0), labels=True)
>>> sorted(e[Integer(2)] for T in trees for e in T.edge_iterator())
['a', 'b']
```

Trivial case:

```
sage: from sage.graphs.edge_connectivity import GabowEdgeConnectivity
sage: D = digraphs.Complete(5)
sage: GabowEdgeConnectivity(D).edge_disjoint_spanning_trees(k=0)
[]
```

```
>>> from sage.all import *
>>> from sage.graphs.edge_connectivity import GabowEdgeConnectivity
>>> D = digraphs.Complete(Integer(5))
>>> GabowEdgeConnectivity(D).edge_disjoint_spanning_trees(k=Integer(0))
[]
```

Asking for more arborescences than the digraph can support raises `EmptySetError`:

```
sage: from sage.graphs.edge_connectivity import GabowEdgeConnectivity
sage: from sage.categories.sets_cat import EmptySetError
sage: D = digraphs.Complete(5)
sage: try:
....:     _ = GabowEdgeConnectivity(D).edge_disjoint_spanning_trees(k=5)
....: except EmptySetError:
....:     print("EmptySetError")
EmptySetError
```

```
>>> from sage.all import *
>>> from sage.graphs.edge_connectivity import GabowEdgeConnectivity
>>> from sage.categories.sets_cat import EmptySetError
>>> D = digraphs.Complete(Integer(5))
>>> try:
...     _ = GabowEdgeConnectivity(D).edge_disjoint_spanning_
↪trees(k=Integer(5))
... except EmptySetError:
...     print("EmptySetError")
EmptySetError
```

An unknown `root` raises a `ValueError`:

```
sage: from sage.graphs.edge_connectivity import GabowEdgeConnectivity
sage: D = digraphs.Complete(5)
sage: GabowEdgeConnectivity(D).edge_disjoint_spanning_trees(k=2, root=99)
Traceback (most recent call last):
...
ValueError: vertex 99 is not a vertex of the graph
```

```
>>> from sage.all import *
>>> from sage.graphs.edge_connectivity import GabowEdgeConnectivity
>>> D = digraphs.Complete(Integer(5))
>>> GabowEdgeConnectivity(D).edge_disjoint_spanning_trees(k=Integer(2),␣
↪root=Integer(99))
Traceback (most recent call last):
...
ValueError: vertex 99 is not a vertex of the graph
```

5.48 Domination

This module implements methods related to the notion of domination in graphs, and more precisely:

dominating_set()	Return a minimum distance-k dominating set of the graph.
dominating_sets()	Return an iterator over the minimum distance-k dominating sets of the graph.
minimal_dominating_sets()	Return an iterator over the minimal dominating sets of a graph.
is_dominating()	Check whether a set of vertices dominates a graph.
is_redundant()	Check whether a set of vertices has redundant vertices (with respect to domination).
private_neighbors()	Return the private neighbors of a vertex with respect to other vertices.
greedy_dominating_set()	Return a greedy distance-k dominating set of the graph.
maximum_leaf_number()	Return the maximum leaf number of the graph.

EXAMPLES:

We compute the size of a minimum dominating set of the Petersen graph:

```
sage: g = graphs.PetersenGraph()
sage: g.dominating_set(value_only=True)                                    #␣
↪needs sage.numerical.mip
3
```

```
>>> from sage.all import *
>>> g = graphs.PetersenGraph()
>>> g.dominating_set(value_only=True)                                      #␣
↪needs sage.numerical.mip
3
```

We enumerate the minimal dominating sets of the 5-star graph:

```
sage: g = graphs.StarGraph(5)
sage: list(g.minimal_dominating_sets())
[{0}, {1, 2, 3, 4, 5}]
```

```
>>> from sage.all import *
>>> g = graphs.StarGraph(Integer(5))
>>> list(g.minimal_dominating_sets())
[{0}, {1, 2, 3, 4, 5}]
```

Now only those that dominate the middle vertex:

```
sage: list(g.minimal_dominating_sets([0]))
[{0}, {1}, {2}, {3}, {4}, {5}]
```

```
>>> from sage.all import *
>>> list(g.minimal_dominating_sets([Integer(0)]))
[{0}, {1}, {2}, {3}, {4}, {5}]
```

Now the minimal dominating sets of the 5-path graph:

```
sage: g = graphs.PathGraph(5)
sage: list(g.minimal_dominating_sets())
[{0, 2, 4}, {1, 4}, {0, 3}, {1, 3}]
```

```
>>> from sage.all import *
>>> g = graphs.PathGraph(Integer(5))
>>> list(g.minimal_dominating_sets())
[{0, 2, 4}, {1, 4}, {0, 3}, {1, 3}]
```

We count the minimal dominating sets of the Petersen graph:

```
sage: sum(1 for _ in graphs.PetersenGraph().minimal_dominating_sets())
27
```

```
>>> from sage.all import *
>>> sum(Integer(1) for _ in graphs.PetersenGraph().minimal_dominating_sets())
27
```

5.48.1 Methods

sage.graphs.domination.**dominating_set**(*g*, *k*, *independent=1*, *total=False*, *connected=False*, *value_only=False*, *solver=False*, *verbose=None*, *integrality_tolerance=0*)

Return a minimum distance-k dominating set of the graph.

A minimum dominating set S of a graph G is a set of its vertices of minimal cardinality such that any vertex of G is in S or has one of its neighbors in S. See the Wikipedia article Dominating_set.

A minimum distance-k dominating set is a set S of vertices of G of minimal cardinality such that any vertex of G is in S or at distance at most k from a vertex in S. A distance-0 dominating set is the set of vertices itself, and when k is the radius of the graph, any vertex dominates all the other vertices.

As an optimization problem, it can be expressed as follows, where $N^k(u)$ denotes the set of vertices at distance at most k from u (the set of neighbors when $k = 1$):

$$\text{Minimize : } \sum_{v \in G} b_v$$

$$\text{Such that : } \forall v \in G, b_v + \sum_{u \in N^k(v)} b_u \geq 1$$

$$\forall x \in G, b_x \text{ is a binary variable}$$

INPUT:

- `k` – nonnegative integer (default: 1); the domination distance
- `independent` – boolean (default: `False`); when `True`, computes a minimum independent dominating set, that is a minimum dominating set that is also an independent set (see also *`independent_set()`*)
- `total` – boolean (default: `False`); when `True`, computes a total dominating set (see the See the Wikipedia article Dominating_set)
- `connected` – boolean (default: `False`); when `True`, computes a connected dominating set (see Wikipedia article Connected_dominating_set)
- `value_only` – boolean (default: `False`); whether to only return the cardinality of the computed dominating set, or to return its list of vertices (default)
- `solver` – string (default: `None`); specifies a Mixed Integer Linear Programming (MILP) solver to be used. If set to `None`, the default one is used. For more information on MILP solvers and which default solver is used, see the method `solve` of the class `MixedIntegerLinearProgram`.

- `verbose` – integer (default: 0); sets the level of verbosity. Set to 0 by default, which means quiet.
- `integrality_tolerance` – float; parameter for use with MILP solvers over an inexact base ring; see `MixedIntegerLinearProgram.get_values()`.

EXAMPLES:

A basic illustration on a `PappusGraph`:

```
sage: g = graphs.PappusGraph()
sage: g.dominating_set(value_only=True)                                     #␣
↪needs sage.numerical.mip
5
```

```
>>> from sage.all import *
>>> g = graphs.PappusGraph()
>>> g.dominating_set(value_only=True)                                       #␣
↪needs sage.numerical.mip
5
```

If we build a graph from two disjoint stars, then link their centers we will find a difference between the cardinality of an independent set and a stable independent set:

```
sage: g = 2 * graphs.StarGraph(5)
sage: g.add_edge(0, 6)
sage: len(g.dominating_set())                                               #␣
↪needs sage.numerical.mip
2
sage: len(g.dominating_set(independent=True))                               #␣
↪needs sage.numerical.mip
6
```

```
>>> from sage.all import *
>>> g = Integer(2) * graphs.StarGraph(Integer(5))
>>> g.add_edge(Integer(0), Integer(6))
>>> len(g.dominating_set())                                                 #␣
↪needs sage.numerical.mip
2
>>> len(g.dominating_set(independent=True))                                 #␣
↪needs sage.numerical.mip
6
```

The total dominating set of the Petersen graph has cardinality 4:

```
sage: G = graphs.PetersenGraph()
sage: G.dominating_set(total=True, value_only=True)                         #␣
↪needs sage.numerical.mip
4
```

```
>>> from sage.all import *
>>> G = graphs.PetersenGraph()
>>> G.dominating_set(total=True, value_only=True)                           #␣
↪needs sage.numerical.mip
4
```

The dominating set is calculated for both the directed and undirected graphs (modification introduced in Issue #17905):

```
sage: g = digraphs.Path(3)
sage: g.dominating_set(value_only=True)                                    #␣
↪needs sage.numerical.mip
2
sage: g = graphs.PathGraph(3)
sage: g.dominating_set(value_only=True)                                    #␣
↪needs sage.numerical.mip
1
```

```
>>> from sage.all import *
>>> g = digraphs.Path(Integer(3))
>>> g.dominating_set(value_only=True)                                      #␣
↪needs sage.numerical.mip
2
>>> g = graphs.PathGraph(Integer(3))
>>> g.dominating_set(value_only=True)                                      #␣
↪needs sage.numerical.mip
1
```

Cardinality of distance-k dominating sets:

```
sage: G = graphs.PetersenGraph()
sage: [G.dominating_set(k=k, value_only=True) for k in range(G.radius() + 1)]   #␣
↪needs sage.numerical.mip
[10, 3, 1]
sage: G = graphs.PathGraph(5)
sage: [G.dominating_set(k=k, value_only=True) for k in range(G.radius() + 1)]   #␣
↪needs sage.numerical.mip
[5, 2, 1]
```

```
>>> from sage.all import *
>>> G = graphs.PetersenGraph()
>>> [G.dominating_set(k=k, value_only=True) for k in range(G.radius() +␣
↪Integer(1))]   # needs sage.numerical.mip
[10, 3, 1]
>>> G = graphs.PathGraph(Integer(5))
>>> [G.dominating_set(k=k, value_only=True) for k in range(G.radius() +␣
↪Integer(1))]   # needs sage.numerical.mip
[5, 2, 1]
```

sage.graphs.domination.**dominating_sets**(*g*, *k*, *independent=1*, *total=False*, *connected=False*, *solver=False*, *verbose=None*, *integrality_tolerance=0*)

Return an iterator over the minimum distance-k dominating sets of the graph.

A minimum dominating set S of a graph G is a set of its vertices of minimal cardinality such that any vertex of G is in S or has one of its neighbors in S. See the Wikipedia article Dominating_set.

A minimum distance-k dominating set is a set S of vertices of G of minimal cardinality such that any vertex of G is in S or at distance at most k from a vertex in S. A distance-0 dominating set is the set of vertices itself, and when k is the radius of the graph, any vertex dominates all the other vertices.

As an optimization problem, it can be expressed as follows, where $N^k(u)$ denotes the set of vertices at distance at

most k from u (the set of neighbors when $k = 1$):

$$\text{Minimize : } \sum_{v \in G} b_v$$

$$\text{Such that : } \forall v \in G, b_v + \sum_{u \in N^k(v)} b_u \geq 1$$

$$\forall x \in G, b_x \text{ is a binary variable}$$

We use constraints generation to iterate over the minimum distance-k dominating sets. That is, after reporting a solution, we add a constraint to discard it and solve the problem again until no more solution can be found.

INPUT:

- `k` – nonnegative integer (default: 1); the domination distance
- `independent` – boolean (default: `False`); when `True`, computes minimum independent dominating sets, that is minimum dominating sets that are also independent sets (see also *`independent_set()`*)
- `total` – boolean (default: `False`); when `True`, computes total dominating sets (see the See the Wikipedia article Dominating_set)
- `connected` – boolean (default: `False`); when `True`, computes connected dominating sets (see Wikipedia article Connected_dominating_set)
- `solver` – string (default: `None`); specifies a Mixed Integer Linear Programming (MILP) solver to be used. If set to `None`, the default one is used. For more information on MILP solvers and which default solver is used, see the method `solve` of the class `MixedIntegerLinearProgram`.
- `verbose` – integer (default: 0); sets the level of verbosity. Set to 0 by default, which means quiet.
- `integrality_tolerance` – float; parameter for use with MILP solvers over an inexact base ring; see `MixedIntegerLinearProgram.get_values()`.

EXAMPLES:

Number of distance-k dominating sets of a Path graph of order 10:

```
sage: g = graphs.PathGraph(10)
sage: [sum(1 for _ in g.dominating_sets(k=k)) for k in range(11)]                #␣
↪needs sage.numerical.mip
[1, 13, 1, 13, 25, 2, 4, 6, 8, 10, 10]
```

```
>>> from sage.all import *
>>> g = graphs.PathGraph(Integer(10))
>>> [sum(Integer(1) for _ in g.dominating_sets(k=k)) for k in range(Integer(11))]␣
↪              # needs sage.numerical.mip
[1, 13, 1, 13, 25, 2, 4, 6, 8, 10, 10]
```

If we build a graph from two disjoint stars, then link their centers we will find a difference between the cardinality of an independent set and a stable independent set:

```
sage: g = 2 * graphs.StarGraph(5)
sage: g.add_edge(0, 6)
sage: [sum(1 for _ in g.dominating_sets(k=k)) for k in range(11)]                #␣
↪needs sage.numerical.mip
[1, 1, 2, 12, 12, 12, 12, 12, 12, 12, 12]
```

```
>>> from sage.all import *
>>> g = Integer(2) * graphs.StarGraph(Integer(5))
>>> g.add_edge(Integer(0), Integer(6))
>>> [sum(Integer(1) for _ in g.dominating_sets(k=k)) for k in range(Integer(11))]␣
↪              # needs sage.numerical.mip
[1, 1, 2, 12, 12, 12, 12, 12, 12, 12, 12]
```

The total dominating set of the Petersen graph has cardinality 4:

```
sage: G = graphs.PetersenGraph()
sage: G.dominating_set(total=True, value_only=True)                             #␣
↪needs sage.numerical.mip
4
sage: sorted(sorted(d) for d in G.dominating_sets(k=1))                         #␣
↪needs sage.numerical.mip
[[0, 2, 6],
 [0, 3, 9],
 [0, 7, 8],
 [1, 3, 7],
 [1, 4, 5],
 [1, 8, 9],
 [2, 4, 8],
 [2, 5, 9],
 [3, 5, 6],
 [4, 6, 7]]
```

```
>>> from sage.all import *
>>> G = graphs.PetersenGraph()
>>> G.dominating_set(total=True, value_only=True)                               #␣
↪needs sage.numerical.mip
4
>>> sorted(sorted(d) for d in G.dominating_sets(k=Integer(1)))                  ␣
↪     # needs sage.numerical.mip
[[0, 2, 6],
 [0, 3, 9],
 [0, 7, 8],
 [1, 3, 7],
 [1, 4, 5],
 [1, 8, 9],
 [2, 4, 8],
 [2, 5, 9],
 [3, 5, 6],
 [4, 6, 7]]
```

Independent distance-k dominating sets of a Path graph:

```
sage: G = graphs.PathGraph(6)
sage: sorted(sorted(d) for d in G.dominating_sets(k=1, independent=True))
[[1, 4]]
sage: sorted(sorted(d) for d in G.dominating_sets(k=2, independent=True))
[[0, 3], [0, 4], [0, 5], [1, 3], [1, 4], [1, 5], [2, 4], [2, 5]]
sage: sorted(G.dominating_sets(k=3, independent=True))
[[2], [3]]
```

```
>>> from sage.all import *
>>> G = graphs.PathGraph(Integer(6))
>>> sorted(sorted(d) for d in G.dominating_sets(k=Integer(1), independent=True))
[[1, 4]]
>>> sorted(sorted(d) for d in G.dominating_sets(k=Integer(2), independent=True))
[[0, 3], [0, 4], [0, 5], [1, 3], [1, 4], [1, 5], [2, 4], [2, 5]]
>>> sorted(G.dominating_sets(k=Integer(3), independent=True))
[[2], [3]]
```

The dominating set is calculated for both the directed and undirected graphs (modification introduced in Issue #17905):

```
sage: g = digraphs.Path(3)
sage: g.dominating_set(value_only=True)
2
sage: sorted(sorted(d) for d in g.dominating_sets())
[[0, 1], [0, 2]]
sage: list(g.dominating_sets(k=2))
[[0]]
sage: g = graphs.PathGraph(3)
sage: g.dominating_set(value_only=True)
1
sage: next(g.dominating_sets())
[1]
```

```
>>> from sage.all import *
>>> g = digraphs.Path(Integer(3))
>>> g.dominating_set(value_only=True)
2
>>> sorted(sorted(d) for d in g.dominating_sets())
[[0, 1], [0, 2]]
>>> list(g.dominating_sets(k=Integer(2)))
[[0]]
>>> g = graphs.PathGraph(Integer(3))
>>> g.dominating_set(value_only=True)
1
>>> next(g.dominating_sets())
[1]
```

Minimum connected dominating sets of the Petersen graph:

```
sage: G = graphs.PetersenGraph()
sage: G.dominating_set(total=True, value_only=True)                    #␣
↪needs sage.numerical.mip
4
sage: sorted(sorted(d) for d in G.dominating_sets(k=1, connected=True))          #␣
↪needs sage.numerical.mip
[[0, 1, 2, 6],
 [0, 1, 4, 5],
 [0, 3, 4, 9],
 [0, 5, 7, 8],
 [1, 2, 3, 7],
 [1, 6, 8, 9],
```

(continues on next page)

(continued from previous page)

```
 [2, 3, 4, 8],
 [2, 5, 7, 9],
 [3, 5, 6, 8],
 [4, 6, 7, 9]]
```

```
>>> from sage.all import *
>>> G = graphs.PetersenGraph()
>>> G.dominating_set(total=True, value_only=True)                              #␣
↪needs sage.numerical.mip
4
>>> sorted(sorted(d) for d in G.dominating_sets(k=Integer(1), connected=True))  ␣
↪     # needs sage.numerical.mip
[[0, 1, 2, 6],
 [0, 1, 4, 5],
 [0, 3, 4, 9],
 [0, 5, 7, 8],
 [1, 2, 3, 7],
 [1, 6, 8, 9],
 [2, 3, 4, 8],
 [2, 5, 7, 9],
 [3, 5, 6, 8],
 [4, 6, 7, 9]]
```

Subgraph induced by the dominating set is connected:

```
sage: G = graphs.PetersenGraph()
sage: all(G.subgraph(vertices=dom).is_connected()
....:     for dom in G.dominating_set(k=1, connected=True))
True
```

```
>>> from sage.all import *
>>> G = graphs.PetersenGraph()
>>> all(G.subgraph(vertices=dom).is_connected()
...     for dom in G.dominating_set(k=Integer(1), connected=True))
True
```

Minimum distance-k connected dominating sets of the Tietze graph:

```
sage: G = graphs.TietzeGraph()
sage: sorted(sorted(d) for d in G.dominating_sets(k=2, connected=True))
[[0, 1], [0, 8], [0, 9], [2, 3], [3, 4], [3, 10], [5, 6], [6, 7], [6, 11]]
sage: sorted(G.dominating_sets(k=3, connected=True))
[[0], [1], [2], [3], [4], [5], [6], [7], [8], [9], [10], [11]]
```

```
>>> from sage.all import *
>>> G = graphs.TietzeGraph()
>>> sorted(sorted(d) for d in G.dominating_sets(k=Integer(2), connected=True))
[[0, 1], [0, 8], [0, 9], [2, 3], [3, 4], [3, 10], [5, 6], [6, 7], [6, 11]]
>>> sorted(G.dominating_sets(k=Integer(3), connected=True))
[[0], [1], [2], [3], [4], [5], [6], [7], [8], [9], [10], [11]]
```

sage.graphs.domination.**greedy_dominating_set**(*G*, *k=1*, *vertices=None*, *ordering=None*, *return_sets=False*, *closest=False*)

Return a greedy distance-k dominating set of the graph.

A distance-k dominating set S of a graph G is a set of its vertices of minimal cardinality such that any vertex of G is in S or is at distance at most k from a vertex in S. See the Wikipedia article Dominating_set.

When G is directed, vertex u can be a dominator of vertex v if there is a directed path of length at most k from u to v.

This method implements a greedy heuristic to find a minimal dominatic set.

INPUT:

- `G` – a Graph
- `k` – integer (default: 1); the domination distance to consider
- `vertices` – iterable container of vertices (default: `None`); when specified, return a dominating set of the specified vertices only
- `ordering` – string (default: `None`); specify the order in which to consider the vertices
  - `None` – if `vertices` is `None`, then consider the vertices in the order given by `list(G)`. Otherwise, consider the vertices in the order of iteration of `vertices`.
  - `'degree_min'` – consider the vertices by increasing degree
  - `'degree_max'` – consider the vertices by decreasing degree
- `return_sets` – boolean (default: `False`); whether to return the vertices of the dominating set only (default), or a dictionary mapping each vertex of the dominating set to the set of vertices it dominates.
- `closest` – boolean (default: `False`); whether to attach a vertex to its closest dominator or not. This parameter is use only when `return_sets` is `True`.

EXAMPLES:

Dominating sets of a path:

```
sage: from sage.graphs.domination import greedy_dominating_set
sage: G = graphs.PathGraph(5)
sage: sorted(greedy_dominating_set(G, ordering=None))
[0, 2, 4]
sage: sorted(greedy_dominating_set(G, ordering='degree_min'))
[0, 2, 4]
sage: sorted(greedy_dominating_set(G, ordering='degree_max'))
[1, 3]
sage: sorted(greedy_dominating_set(G, k=2, ordering=None))
[0, 3]
sage: sorted(greedy_dominating_set(G, k=2, ordering='degree_min'))
[0, 4]
sage: sorted(greedy_dominating_set(G, k=2, ordering='degree_max'))
[1, 4]
sage: greedy_dominating_set(G, k=3, ordering='degree_min', return_sets=True,␣
↪closest=False)
{0: {0, 1, 2, 3}, 4: {4}}
sage: greedy_dominating_set(G, k=3, ordering='degree_min', return_sets=True,␣
↪closest=True)
{0: {0, 2, 3}, 4: {1, 4}}
```

```
>>> from sage.all import *
>>> from sage.graphs.domination import greedy_dominating_set
>>> G = graphs.PathGraph(Integer(5))
>>> sorted(greedy_dominating_set(G, ordering=None))
[0, 2, 4]
>>> sorted(greedy_dominating_set(G, ordering='degree_min'))
[0, 2, 4]
>>> sorted(greedy_dominating_set(G, ordering='degree_max'))
[1, 3]
>>> sorted(greedy_dominating_set(G, k=Integer(2), ordering=None))
[0, 3]
>>> sorted(greedy_dominating_set(G, k=Integer(2), ordering='degree_min'))
[0, 4]
>>> sorted(greedy_dominating_set(G, k=Integer(2), ordering='degree_max'))
[1, 4]
>>> greedy_dominating_set(G, k=Integer(3), ordering='degree_min', return_
↪sets=True, closest=False)
{0: {0, 1, 2, 3}, 4: {4}}
>>> greedy_dominating_set(G, k=Integer(3), ordering='degree_min', return_
↪sets=True, closest=True)
{0: {0, 2, 3}, 4: {1, 4}}
```

Asking for a dominating set of a subset of vertices:

```
sage: from sage.graphs.domination import greedy_dominating_set
sage: from sage.graphs.domination import is_dominating
sage: G = graphs.PetersenGraph()
sage: vertices = {0, 1, 2, 3, 4, 5}
sage: dom = greedy_dominating_set(G, vertices=vertices, return_sets=True)
sage: sorted(dom)
[0, 2]
sage: is_dominating(G, dom, focus=vertices)
True
sage: is_dominating(G, dom)
False
sage: dominated = [u for v in dom for u in dom[v]]
sage: sorted(dominated) == sorted(vertices)
True
```

```
>>> from sage.all import *
>>> from sage.graphs.domination import greedy_dominating_set
>>> from sage.graphs.domination import is_dominating
>>> G = graphs.PetersenGraph()
>>> vertices = {Integer(0), Integer(1), Integer(2), Integer(3), Integer(4),␣
↪Integer(5)}
>>> dom = greedy_dominating_set(G, vertices=vertices, return_sets=True)
>>> sorted(dom)
[0, 2]
>>> is_dominating(G, dom, focus=vertices)
True
>>> is_dominating(G, dom)
False
>>> dominated = [u for v in dom for u in dom[v]]
```

(continues on next page)

(continued from previous page)

```
>>> sorted(dominated) == sorted(vertices)
True
```

Influence of the ordering of the vertices on the result:

```
sage: from sage.graphs.domination import greedy_dominating_set
sage: G = graphs.StarGraph(4)
sage: greedy_dominating_set(G, vertices=[0, 1, 2, 3, 4])
[0]
sage: sorted(greedy_dominating_set(G, vertices=[1, 2, 3, 4, 0]))
[1, 2, 3, 4]
```

```
>>> from sage.all import *
>>> from sage.graphs.domination import greedy_dominating_set
>>> G = graphs.StarGraph(Integer(4))
>>> greedy_dominating_set(G, vertices=[Integer(0), Integer(1), Integer(2),␣
↪Integer(3), Integer(4)])
[0]
>>> sorted(greedy_dominating_set(G, vertices=[Integer(1), Integer(2), Integer(3),␣
↪Integer(4), Integer(0)]))
[1, 2, 3, 4]
```

Dominating set of a directed graph:

```
sage: from sage.graphs.domination import greedy_dominating_set
sage: D = digraphs.Path(3)
sage: sorted(greedy_dominating_set(D, vertices=[0, 1, 2]))
[0, 2]
```

```
>>> from sage.all import *
>>> from sage.graphs.domination import greedy_dominating_set
>>> D = digraphs.Path(Integer(3))
>>> sorted(greedy_dominating_set(D, vertices=[Integer(0), Integer(1),␣
↪Integer(2)]))
[0, 2]
```

sage.graphs.domination.**is_dominating**(*G*, *dom*, *focus=None*)

Check whether `dom` is a dominating set of `G`.

We say that a set D of vertices of a graph G dominates a set S if every vertex of S either belongs to D or is adjacent to a vertex of D. Also, D is a dominating set of G if it dominates $V(G)$.

INPUT:

- `dom` – iterable of vertices of `G`; the vertices of the supposed dominating set
- `focus` – iterable of vertices of `G` (default: `None`); if specified, this method checks instead if `dom` dominates the vertices in `focus`

EXAMPLES:

```
sage: g = graphs.CycleGraph(5)
sage: g.is_dominating([0,1], [4, 2])
True
```

(continues on next page)

(continued from previous page)

```
sage: g.is_dominating([0,1])
False
```

```
>>> from sage.all import *
>>> g = graphs.CycleGraph(Integer(5))
>>> g.is_dominating([Integer(0),Integer(1)], [Integer(4), Integer(2)])
True

>>> g.is_dominating([Integer(0),Integer(1)])
False
```

sage.graphs.domination.**is_redundant**(*G*, *dom*, *focus=None*)

Check whether `dom` has redundant vertices.

For a graph G and sets D and S of vertices, we say that a vertex $v \in D$ is *redundant* in S if v has no private neighbor with respect to D in S. In other words, there is no vertex in S that is dominated by v but not by $D \setminus \{v\}$.

INPUT:

- `dom` – iterable of vertices of `G`; where we look for redundant vertices
- `focus` – iterable of vertices of `G` (default: `None`); if specified, this method checks instead whether `dom` has a redundant vertex in `focus`

Warning

The assumption is made that `focus` (if provided) does not contain repeated vertices.

EXAMPLES:

```
sage: G = graphs.CubeGraph(3)
sage: G.is_redundant(['000', '101'], ['011'])
True
sage: G.is_redundant(['000', '101'])
False
```

```
>>> from sage.all import *
>>> G = graphs.CubeGraph(Integer(3))
>>> G.is_redundant(['000', '101'], ['011'])
True
>>> G.is_redundant(['000', '101'])
False
```

sage.graphs.domination.**maximum_leaf_number**(*G*, *solver=None*, *verbose=0*, *integrality_tolerance=0.001*)

Return the maximum leaf number of the graph.

The maximum leaf number is the maximum possible number of leaves of a spanning tree of G. This is also the cardinality of the complement of a minimum connected dominating set. See the Wikipedia article Connected_dominating_set.

The MLN of a graph with less than 2 vertices is 0, while the MLN of a connected graph with 2 or 3 vertices is 1 or 2 respectively.

INPUT:

- G – a Graph
- solver – string (default: None); specifies a Mixed Integer Linear Programming (MILP) solver to be used. If set to None, the default one is used. For more information on MILP solvers and which default solver is used, see the method solve of the class MixedIntegerLinearProgram.
- verbose – integer (default: 0); sets the level of verbosity. Set to 0 by default, which means quiet.
- integrality_tolerance – float; parameter for use with MILP solvers over an inexact base ring; see MixedIntegerLinearProgram.get_values().

EXAMPLES:

Empty graph:

```
sage: G = Graph()
sage: G.maximum_leaf_number()
0
```

```
>>> from sage.all import *
>>> G = Graph()
>>> G.maximum_leaf_number()
0
```

Petersen graph:

```
sage: G = graphs.PetersenGraph()
sage: G.maximum_leaf_number()
6
```

```
>>> from sage.all import *
>>> G = graphs.PetersenGraph()
>>> G.maximum_leaf_number()
6
```

sage.graphs.domination.**minimal_dominating_sets**(*G*, *to_dominate=None*, *work_on_copy=True*, *k=1*)

Return an iterator over the minimal dominating sets of a graph.

INPUT:

- G – a graph
- to_dominate – vertex iterable or None (default: None); the set of vertices to be dominated
- work_on_copy – boolean (default: True); whether or not to work on a copy of the input graph; if set to False, the input graph will be modified (relabeled)
- k – nonnegative integer (default: 1); the domination distance

OUTPUT:

An iterator over the inclusion-minimal sets of vertices of G. If to_dominate is provided, return an iterator over the inclusion-minimal sets of vertices that dominate the vertices of to_dominate.

ALGORITHM: The algorithm described in [BDHPR2019].

AUTHOR: Jean-Florent Raymond (2019-03-04) – initial version.

EXAMPLES:

```
sage: G = graphs.ButterflyGraph()
sage: ll = list(G.minimal_dominating_sets())
sage: pp = [{0, 1}, {1, 3}, {0, 2}, {2, 3}, {4}]
sage: len(ll) == len(pp) and all(x in pp for x in ll) and all(x in ll for x in pp)
True

sage: ll = list(G.minimal_dominating_sets([0,3]))
sage: pp = [{0}, {3}, {4}]
sage: len(ll) == len(pp) and all(x in pp for x in ll) and all(x in ll for x in pp)
True

sage: ll = list(G.minimal_dominating_sets([4]))
sage: pp = [{4}, {0}, {1}, {2}, {3}]
sage: len(ll) == len(pp) and all(x in pp for x in ll) and all(x in ll for x in pp)
True
```

```
>>> from sage.all import *
>>> G = graphs.ButterflyGraph()
>>> ll = list(G.minimal_dominating_sets())
>>> pp = [{Integer(0), Integer(1)}, {Integer(1), Integer(3)}, {Integer(0),␣
↪Integer(2)}, {Integer(2), Integer(3)}, {Integer(4)}]
>>> len(ll) == len(pp) and all(x in pp for x in ll) and all(x in ll for x in pp)
True

>>> ll = list(G.minimal_dominating_sets([Integer(0),Integer(3)]))
>>> pp = [{Integer(0)}, {Integer(3)}, {Integer(4)}]
>>> len(ll) == len(pp) and all(x in pp for x in ll) and all(x in ll for x in pp)
True

>>> ll = list(G.minimal_dominating_sets([Integer(4)]))
>>> pp = [{Integer(4)}, {Integer(0)}, {Integer(1)}, {Integer(2)}, {Integer(3)}]
>>> len(ll) == len(pp) and all(x in pp for x in ll) and all(x in ll for x in pp)
True
```

```
sage: ll = list(graphs.PetersenGraph().minimal_dominating_sets())
sage: pp = [{0, 2, 6},
....:       {0, 9, 3},
....:       {0, 8, 7},
....:       {1, 3, 7},
....:       {1, 4, 5},
....:       {8, 1, 9},
....:       {8, 2, 4},
....:       {9, 2, 5},
....:       {3, 5, 6},
....:       {4, 6, 7},
....:       {0, 8, 2, 9},
....:       {0, 3, 6, 7},
....:       {1, 3, 5, 9},
....:       {8, 1, 4, 7},
....:       {2, 4, 5, 6},
....:       {0, 1, 2, 3, 4},
....:       {0, 1, 2, 5, 7},
```

(continues on next page)

(continued from previous page)

```
....:        {0, 1, 4, 6, 9},
....:        {0, 1, 5, 6, 8},
....:        {0, 8, 3, 4, 5},
....:        {0, 9, 4, 5, 7},
....:        {8, 1, 2, 3, 6},
....:        {1, 2, 9, 6, 7},
....:        {9, 2, 3, 4, 7},
....:        {8, 2, 3, 5, 7},
....:        {8, 9, 3, 4, 6},
....:        {8, 9, 5, 6, 7}]
sage: len(ll) == len(pp) and all(x in pp for x in ll) and all(x in ll for x in pp)
True
```

```
>>> from sage.all import *
>>> ll = list(graphs.PetersenGraph().minimal_dominating_sets())
>>> pp = [{Integer(0), Integer(2), Integer(6)},
...       {Integer(0), Integer(9), Integer(3)},
...       {Integer(0), Integer(8), Integer(7)},
...       {Integer(1), Integer(3), Integer(7)},
...       {Integer(1), Integer(4), Integer(5)},
...       {Integer(8), Integer(1), Integer(9)},
...       {Integer(8), Integer(2), Integer(4)},
...       {Integer(9), Integer(2), Integer(5)},
...       {Integer(3), Integer(5), Integer(6)},
...       {Integer(4), Integer(6), Integer(7)},
...       {Integer(0), Integer(8), Integer(2), Integer(9)},
...       {Integer(0), Integer(3), Integer(6), Integer(7)},
...       {Integer(1), Integer(3), Integer(5), Integer(9)},
...       {Integer(8), Integer(1), Integer(4), Integer(7)},
...       {Integer(2), Integer(4), Integer(5), Integer(6)},
...       {Integer(0), Integer(1), Integer(2), Integer(3), Integer(4)},
...       {Integer(0), Integer(1), Integer(2), Integer(5), Integer(7)},
...       {Integer(0), Integer(1), Integer(4), Integer(6), Integer(9)},
...       {Integer(0), Integer(1), Integer(5), Integer(6), Integer(8)},
...       {Integer(0), Integer(8), Integer(3), Integer(4), Integer(5)},
...       {Integer(0), Integer(9), Integer(4), Integer(5), Integer(7)},
...       {Integer(8), Integer(1), Integer(2), Integer(3), Integer(6)},
...       {Integer(1), Integer(2), Integer(9), Integer(6), Integer(7)},
...       {Integer(9), Integer(2), Integer(3), Integer(4), Integer(7)},
...       {Integer(8), Integer(2), Integer(3), Integer(5), Integer(7)},
...       {Integer(8), Integer(9), Integer(3), Integer(4), Integer(6)},
...       {Integer(8), Integer(9), Integer(5), Integer(6), Integer(7)}]
>>> len(ll) == len(pp) and all(x in pp for x in ll) and all(x in ll for x in pp)
True
```

Listing minimal distance-k dominating sets:

```
sage: G = graphs.Grid2dGraph(2, 3)
sage: list(G.minimal_dominating_sets(k=0))
[{(0, 0), (0, 1), (0, 2), (1, 0), (1, 1), (1, 2)}]
sage: list(G.minimal_dominating_sets(k=1))
[{(0, 0), (0, 2), (1, 1)},
```

(continues on next page)

(continued from previous page)

```
 {(0, 1), (1, 1)},
 {(0, 0), (0, 1), (0, 2)},
 {(0, 2), (1, 0)},
 {(0, 0), (1, 2)},
 {(0, 1), (1, 0), (1, 2)},
 {(1, 0), (1, 1), (1, 2)}]
sage: list(G.minimal_dominating_sets(k=2))
[{(0, 0), (1, 2)},
 {(0, 2), (1, 2)},
 {(1, 0), (1, 2)},
 {(0, 1)},
 {(0, 0), (0, 2)},
 {(0, 2), (1, 0)},
 {(0, 0), (1, 0)},
 {(1, 1)}]
sage: list(G.minimal_dominating_sets(k=3))
[{(0, 0)}, {(0, 1)}, {(0, 2)}, {(1, 0)}, {(1, 1)}, {(1, 2)}]
```

```
>>> from sage.all import *
>>> G = graphs.Grid2dGraph(Integer(2), Integer(3))
>>> list(G.minimal_dominating_sets(k=Integer(0)))
[{(0, 0), (0, 1), (0, 2), (1, 0), (1, 1), (1, 2)}]
>>> list(G.minimal_dominating_sets(k=Integer(1)))
[{(0, 0), (0, 2), (1, 1)},
 {(0, 1), (1, 1)},
 {(0, 0), (0, 1), (0, 2)},
 {(0, 2), (1, 0)},
 {(0, 0), (1, 2)},
 {(0, 1), (1, 0), (1, 2)},
 {(1, 0), (1, 1), (1, 2)}]
>>> list(G.minimal_dominating_sets(k=Integer(2)))
[{(0, 0), (1, 2)},
 {(0, 2), (1, 2)},
 {(1, 0), (1, 2)},
 {(0, 1)},
 {(0, 0), (0, 2)},
 {(0, 2), (1, 0)},
 {(0, 0), (1, 0)},
 {(1, 1)}]
>>> list(G.minimal_dominating_sets(k=Integer(3)))
[{(0, 0)}, {(0, 1)}, {(0, 2)}, {(1, 0)}, {(1, 1)}, {(1, 2)}]
```

When parameter `work_on_copy` is `False`, the input graph is modified (relabeled):

```
sage: G = Graph([('A', 'B')])
sage: _ = list(G.minimal_dominating_sets(work_on_copy=True))
sage: set(G) == {'A', 'B'}
True
sage: _ = list(G.minimal_dominating_sets(work_on_copy=False))
sage: set(G) == {'A', 'B'}
False
sage: set(G) == {0, 1}
```

(continues on next page)

(continued from previous page)

```
True
```

```
>>> from sage.all import *
>>> G = Graph([('A', 'B')])
>>> _ = list(G.minimal_dominating_sets(work_on_copy=True))
>>> set(G) == {'A', 'B'}
True
>>> _ = list(G.minimal_dominating_sets(work_on_copy=False))
>>> set(G) == {'A', 'B'}
False
>>> set(G) == {Integer(0), Integer(1)}
True
```

sage.graphs.domination.**private_neighbors**(*G*, *vertex*, *dom*)

Return the private neighbors of a vertex with respect to other vertices.

A private neighbor of a vertex v with respect to a vertex subset D is a closed neighbor of v that is not dominated by a vertex of $D \setminus \{v\}$.

INPUT:

- `vertex` – a vertex of `G`
- `dom` – iterable of vertices of `G`; the vertices possibly stealing private neighbors from `vertex`

OUTPUT:

Return the closed neighbors of `vertex` that are not closed neighbors of any other vertex of `dom`.

EXAMPLES:

```
sage: g = graphs.PathGraph(5)
sage: list(g.private_neighbors(1, [1, 3, 4]))
[1, 0]

sage: list(g.private_neighbors(1, [3, 4]))
[1, 0]

sage: list(g.private_neighbors(1, [3, 4, 0]))
[]
```

```
>>> from sage.all import *
>>> g = graphs.PathGraph(Integer(5))
>>> list(g.private_neighbors(Integer(1), [Integer(1), Integer(3), Integer(4)]))
[1, 0]

>>> list(g.private_neighbors(Integer(1), [Integer(3), Integer(4)]))
[1, 0]

>>> list(g.private_neighbors(Integer(1), [Integer(3), Integer(4), Integer(0)]))
[]
```

5.49 Cycle enumeration

This module is meant for all functions related to cycle enumeration in graphs.

all_cycles_iterator()	Return an iterator over the cycles of `self`.
all_simple_cycles()	Return an iterator over the simple cycles of `self`.

5.49.1 Functions

sage.graphs.cycle_enumeration.**all_cycles_iterator**(*self*, *starting_vertices=None*, *simple=False*, *rooted=False*, *max_length=None*, *trivial=False*, *weight_function=None*, *by_weight=False*, *check_weight=True*, *report_weight=False*, *algorithm='A'*)

Return an iterator over all the cycles of `self` starting with one of the given vertices. Each edge must have a positive weight.

The cycles are enumerated in increasing length order. Here, a cycle means a closed walk $v_0e_0v_1e_1\dots v_{k-1}e_{k-1}v_0$ where v_i are vertices and e_i are edges. v_i and e_i $(0 \leq i < k)$ may not be distinct. If `simple=True`, then cycles with distinct v_i and e_i $(0 \leq i < k)$ are enumerated.

INPUT:

- `starting_vertices` – iterable (default: `None`); vertices from which the cycles must start. If `None`, then all vertices of the graph can be starting points. This argument is necessary if `rooted` is set to `True`.
- `simple` – boolean (default: `False`); if set to `True`, then only simple cycles are considered. A cycle is simple if the only vertex occurring twice in it is the starting and ending one and no edge occurs twice.
- `rooted` – boolean (default: `False`); if set to False, then cycles differing only by their starting vertex are considered the same (e.g. `['a', 'b', 'c', 'a']` and `['b', 'c', 'a', 'b']`). Otherwise, all cycles are enumerated.
- `max_length` – nonnegative integer (default: `None`); the maximum length of the enumerated paths. If set to `None`, then all lengths are allowed.
- `trivial` – boolean (default: `False`); if set to `True`, then the empty paths are also enumerated
- `weight_function` – function (default: `None`); a function that takes as input an edge `(u, v, l)` and outputs its weight. If not `None`, `by_weight` is automatically set to `True`. If `None` and `by_weight` is `True`, we use the edge label `l` as a weight.
- `by_weight` – boolean (default: `False`); if `True`, the edges in the graph are weighted, otherwise all edges have weight 1
- `check_weight` – boolean (default: `True`); whether to check that the `weight_function` outputs a number for each edge
- `report_weight` – boolean (default: `False`); if `False`, just a cycle is returned. Otherwise a tuple of cycle length and cycle is returned.
- `algorithm` – string (default: `'A'`); the algorithm used to enumerate the cycles.
  - The algorithm `'A'` holds cycle iterators starting with each vertex, and output them in increasing length order.
  - The algorithm `'B'` holds cycle iterators starting with each edge, and output them in increasing length order. It depends on the k-shortest simple paths algorithm. Thus, it is not available if `simple=False` or `rooted=True` or `starting_vertices` is not `None`.

OUTPUT: iterator

See also

- *all_simple_cycles()*
- *shortest_simple_paths()*

AUTHOR:

Alexandre Blondin Masse

EXAMPLES:

```
sage: g = DiGraph({'a': ['a', 'b'], 'b': ['c'], 'c': ['d'], 'd': ['c']},␣
↪loops=True)
sage: it = g.all_cycles_iterator()
sage: for _ in range(7): print(next(it))
['a', 'a']
['a', 'a', 'a']
['c', 'd', 'c']
['a', 'a', 'a', 'a']
['a', 'a', 'a', 'a', 'a']
['c', 'd', 'c', 'd', 'c']
['a', 'a', 'a', 'a', 'a', 'a']
```

```
>>> from sage.all import *
>>> g = DiGraph({'a': ['a', 'b'], 'b': ['c'], 'c': ['d'], 'd': ['c']}, loops=True)
>>> it = g.all_cycles_iterator()
>>> for _ in range(Integer(7)): print(next(it))
['a', 'a']
['a', 'a', 'a']
['c', 'd', 'c']
['a', 'a', 'a', 'a']
['a', 'a', 'a', 'a', 'a']
['c', 'd', 'c', 'd', 'c']
['a', 'a', 'a', 'a', 'a', 'a']
```

There are no cycles in the empty graph and in acyclic graphs:

```
sage: g = DiGraph()
sage: it = g.all_cycles_iterator()
sage: list(it)
[]
sage: g = DiGraph({0:[1]})
sage: it = g.all_cycles_iterator()
sage: list(it)
[]
```

```
>>> from sage.all import *
>>> g = DiGraph()
>>> it = g.all_cycles_iterator()
>>> list(it)
```

(continues on next page)

(continued from previous page)

```
[]
>>> g = DiGraph({Integer(0):[Integer(1)]})
>>> it = g.all_cycles_iterator()
>>> list(it)
[]
```

It is possible to restrict the starting vertices of the cycles:

```
sage: g = DiGraph({'a': ['a', 'b'], 'b': ['c'], 'c': ['d'], 'd': ['c']},␣
↪loops=True)
sage: it = g.all_cycles_iterator(starting_vertices=['b', 'c'])
sage: for _ in range(3): print(next(it))
['c', 'd', 'c']
['c', 'd', 'c', 'd', 'c']
['c', 'd', 'c', 'd', 'c', 'd', 'c']
```

```
>>> from sage.all import *
>>> g = DiGraph({'a': ['a', 'b'], 'b': ['c'], 'c': ['d'], 'd': ['c']}, loops=True)
>>> it = g.all_cycles_iterator(starting_vertices=['b', 'c'])
>>> for _ in range(Integer(3)): print(next(it))
['c', 'd', 'c']
['c', 'd', 'c', 'd', 'c']
['c', 'd', 'c', 'd', 'c', 'd', 'c']
```

Also, one can bound the length of the cycles:

```
sage: it = g.all_cycles_iterator(max_length=3)
sage: list(it)
[['a', 'a'], ['a', 'a', 'a'], ['c', 'd', 'c'],
 ['a', 'a', 'a', 'a']]
```

```
>>> from sage.all import *
>>> it = g.all_cycles_iterator(max_length=Integer(3))
>>> list(it)
[['a', 'a'], ['a', 'a', 'a'], ['c', 'd', 'c'],
 ['a', 'a', 'a', 'a']]
```

By default, cycles differing only by their starting point are not all enumerated, but this may be parametrized:

```
sage: it = g.all_cycles_iterator(max_length=3, rooted=False)
sage: list(it)
[['a', 'a'], ['a', 'a', 'a'], ['c', 'd', 'c'],
 ['a', 'a', 'a', 'a']]
sage: it = g.all_cycles_iterator(max_length=3, rooted=True)
sage: list(it)
[['a', 'a'], ['a', 'a', 'a'], ['c', 'd', 'c'], ['d', 'c', 'd'],
 ['a', 'a', 'a', 'a']]
```

```
>>> from sage.all import *
>>> it = g.all_cycles_iterator(max_length=Integer(3), rooted=False)
>>> list(it)
[['a', 'a'], ['a', 'a', 'a'], ['c', 'd', 'c'],
```

(continues on next page)

(continued from previous page)

```
 ['a', 'a', 'a', 'a']]
>>> it = g.all_cycles_iterator(max_length=Integer(3), rooted=True)
>>> list(it)
[['a', 'a'], ['a', 'a', 'a'], ['c', 'd', 'c'], ['d', 'c', 'd'],
 ['a', 'a', 'a', 'a']]
```

One may prefer to enumerate simple cycles, i.e. cycles such that the only vertex occurring twice in it is the starting and ending one (see also *all_simple_cycles()*):

```
sage: it = g.all_cycles_iterator(simple=True)
sage: list(it)
[['a', 'a'], ['c', 'd', 'c']]
sage: g = digraphs.Circuit(4)
sage: list(g.all_cycles_iterator(simple=True))
[[0, 1, 2, 3, 0]]
```

```
>>> from sage.all import *
>>> it = g.all_cycles_iterator(simple=True)
>>> list(it)
[['a', 'a'], ['c', 'd', 'c']]
>>> g = digraphs.Circuit(Integer(4))
>>> list(g.all_cycles_iterator(simple=True))
[[0, 1, 2, 3, 0]]
```

A cycle is enumerated in increasing length order for a weighted graph:

```
sage: g = DiGraph()
sage: g.add_edges([('a', 'b', 2), ('a', 'e', 2), ('b', 'a', 1), ('b', 'c', 1),
....:              ('c', 'd', 2), ('d', 'b', 1), ('e', 'a', 2)])
sage: it = g.all_cycles_iterator(simple=False, max_length=None,
....:                            by_weight=True, report_weight=True)
sage: for i in range(9): print(next(it))
(3, ['a', 'b', 'a'])
(4, ['a', 'e', 'a'])
(4, ['b', 'c', 'd', 'b'])
(6, ['a', 'b', 'a', 'b', 'a'])
(7, ['a', 'b', 'a', 'e', 'a'])
(7, ['a', 'b', 'c', 'd', 'b', 'a'])
(7, ['a', 'e', 'a', 'b', 'a'])
(8, ['a', 'e', 'a', 'e', 'a'])
(8, ['b', 'c', 'd', 'b', 'c', 'd', 'b'])
```

```
>>> from sage.all import *
>>> g = DiGraph()
>>> g.add_edges([('a', 'b', Integer(2)), ('a', 'e', Integer(2)), ('b', 'a',␣
↪Integer(1)), ('b', 'c', Integer(1)),
...              ('c', 'd', Integer(2)), ('d', 'b', Integer(1)), ('e', 'a',␣
↪Integer(2))])
>>> it = g.all_cycles_iterator(simple=False, max_length=None,
...                            by_weight=True, report_weight=True)
>>> for i in range(Integer(9)): print(next(it))
(3, ['a', 'b', 'a'])
```

(continues on next page)

(continued from previous page)

```
(4, ['a', 'e', 'a'])
(4, ['b', 'c', 'd', 'b'])
(6, ['a', 'b', 'a', 'b', 'a'])
(7, ['a', 'b', 'a', 'e', 'a'])
(7, ['a', 'b', 'c', 'd', 'b', 'a'])
(7, ['a', 'e', 'a', 'b', 'a'])
(8, ['a', 'e', 'a', 'e', 'a'])
(8, ['b', 'c', 'd', 'b', 'c', 'd', 'b'])
```

Each edge must have a positive weight:

```
sage: g = DiGraph()
sage: g.add_edges([('a', 'b', -2), ('b', 'a', 1)])
sage: next(g.all_cycles_iterator(simple=False, max_length=None,
....:                            by_weight=True, report_weight=True))
Traceback (most recent call last):
...
ValueError: negative weight is not allowed
```

```
>>> from sage.all import *
>>> g = DiGraph()
>>> g.add_edges([('a', 'b', -Integer(2)), ('b', 'a', Integer(1))])
>>> next(g.all_cycles_iterator(simple=False, max_length=None,
...                            by_weight=True, report_weight=True))
Traceback (most recent call last):
...
ValueError: negative weight is not allowed
```

The algorithm `'B'` works for undirected graphs as well:

```
sage: g = Graph({0: [1, 2], 1: [0, 2], 2: [0, 1, 3, 5], 3: [2, 4], 4: [3, 5], 5:␣
↪[4, 2]})
sage: for cycle in g.all_cycles_iterator(algorithm='B', simple=True, by_
↪weight=True):
....:     print(cycle)
[0, 1, 2, 0]
[2, 3, 4, 5, 2]
```

```
>>> from sage.all import *
>>> g = Graph({Integer(0): [Integer(1), Integer(2)], Integer(1): [Integer(0),␣
↪Integer(2)], Integer(2): [Integer(0), Integer(1), Integer(3), Integer(5)],␣
↪Integer(3): [Integer(2), Integer(4)], Integer(4): [Integer(3), Integer(5)],␣
↪Integer(5): [Integer(4), Integer(2)]})
>>> for cycle in g.all_cycles_iterator(algorithm='B', simple=True, by_
↪weight=True):
...     print(cycle)
[0, 1, 2, 0]
[2, 3, 4, 5, 2]
```

The algorithm `'B'` is unavailable when `simple=False` or `rooted=True` or `starting_vertices is not None`:

```
sage: g = DiGraph()
sage: g.add_edges([('a', 'b', 1), ('b', 'a', 1)])
sage: next(g.all_cycles_iterator(algorithm='B', simple=False))
....:
Traceback (most recent call last):
...
ValueError: The algorithm 'B' is unavailable when simple=False.
sage: next(g.all_cycles_iterator(algorithm='B', simple=True, rooted=True))
Traceback (most recent call last):
...
ValueError: The algorithm 'B' is unavailable when rooted=True.
sage: next(g.all_cycles_iterator(algorithm='B', simple=True, starting_vertices=['a
↪'']))
Traceback (most recent call last):
...
ValueError: The algorithm 'B' is unavailable when starting_vertices is not None.
```

```
>>> from sage.all import *
>>> g = DiGraph()
>>> g.add_edges([('a', 'b', Integer(1)), ('b', 'a', Integer(1))])
>>> next(g.all_cycles_iterator(algorithm='B', simple=False))
....:
Traceback (most recent call last):
...
ValueError: The algorithm 'B' is unavailable when simple=False.
>>> next(g.all_cycles_iterator(algorithm='B', simple=True, rooted=True))
Traceback (most recent call last):
...
ValueError: The algorithm 'B' is unavailable when rooted=True.
>>> next(g.all_cycles_iterator(algorithm='B', simple=True, starting_vertices=['a
↪'']))
Traceback (most recent call last):
...
ValueError: The algorithm 'B' is unavailable when starting_vertices is not None.
```

The algorithm `'A'` works for undirected graphs as well. Specifically, each cycle is enumerated exactly once, meaning a cycle and its reverse are not listed separately:

```
sage: g = Graph({0: [1, 2], 1: [0, 2], 2: [0, 1]})
sage: for cycle in g.all_cycles_iterator(algorithm='A', simple=True):
....:     print(cycle)
[0, 1, 2, 0]
```

```
>>> from sage.all import *
>>> g = Graph({Integer(0): [Integer(1), Integer(2)], Integer(1): [Integer(0),␣
↪Integer(2)], Integer(2): [Integer(0), Integer(1)]})
>>> for cycle in g.all_cycles_iterator(algorithm='A', simple=True):
...     print(cycle)
[0, 1, 2, 0]
```

sage.graphs.cycle_enumeration.**all_simple_cycles**(*self*, *starting_vertices=None*, *rooted=False*, *max_length=None*, *trivial=False*, *weight_function=None*, *by_weight=False*, *check_weight=True*, *report_weight=False*, *algorithm='B'*)

Return a list of all simple cycles of `self`. The cycles are enumerated in increasing length order. Each edge must have a positive weight.

INPUT:

- `starting_vertices` – iterable (default: `None`); vertices from which the cycles must start. If `None`, then all vertices of the graph can be starting points. This argument is necessary if `rooted` is set to `True`.
- `rooted` – boolean (default: `False`); if set to False, then cycles differing only by their starting vertex are considered the same (e.g. `['a', 'b', 'c', 'a']` and `['b', 'c', 'a', 'b']`). Otherwise, all cycles are enumerated.
- `max_length` – nonnegative integer (default: `None`); the maximum length of the enumerated paths. If set to `None`, then all lengths are allowed.
- `trivial` – boolean (default: `False`); if set to `True`, then the empty paths are also enumerated
- `weight_function` – function (default: `None`); a function that takes as input an edge `(u, v, l)` and outputs its weight. If not `None`, `by_weight` is automatically set to `True`. If `None` and `by_weight` is `True`, we use the edge label `l` as a weight.
- `by_weight` – boolean (default: `False`); if `True`, the edges in the graph are weighted, otherwise all edges have weight 1
- `check_weight` – boolean (default: `True`); whether to check that the `weight_function` outputs a number for each edge
- `report_weight` – boolean (default: `False`); if `False`, just a cycle is returned. Otherwise a tuple of cycle length and cycle is returned.
- `algorithm` – string (default: `'B'`); the algorithm used to enumerate the cycles.
  - The algorithm `'A'` holds cycle iterators starting with each vertex, and output them in increasing length order.
  - The algorithm `'B'` holds cycle iterators starting with each edge, and output them in increasing length order.

OUTPUT: list

Note

Although the number of simple cycles of a finite graph is always finite, computing all its cycles may take a very long time.

EXAMPLES:

```
sage: g = DiGraph({'a': ['a', 'b'], 'b': ['c'], 'c': ['d'], 'd': ['c']},␣
↪loops=True)
sage: g.all_simple_cycles()
[['a', 'a'], ['c', 'd', 'c']]
```

```
>>> from sage.all import *
>>> g = DiGraph({'a': ['a', 'b'], 'b': ['c'], 'c': ['d'], 'd': ['c']}, loops=True)
>>> g.all_simple_cycles()
[['a', 'a'], ['c', 'd', 'c']]
```

The directed version of the Petersen graph:

```
sage: g = graphs.PetersenGraph().to_directed()
sage: g.all_simple_cycles(max_length=4)
[[0, 1, 0], [0, 4, 0], [0, 5, 0], [1, 2, 1], [1, 6, 1], [2, 3, 2],
 [2, 7, 2], [3, 4, 3], [3, 8, 3], [4, 9, 4], [5, 7, 5], [5, 8, 5],
 [6, 8, 6], [6, 9, 6], [7, 9, 7]]
sage: g.all_simple_cycles(max_length=6)
[[0, 1, 0], [0, 4, 0], [0, 5, 0], [1, 2, 1], [1, 6, 1], [2, 3, 2],
 [2, 7, 2], [3, 4, 3], [3, 8, 3], [4, 9, 4], [5, 7, 5], [5, 8, 5],
 [6, 8, 6], [6, 9, 6], [7, 9, 7], [0, 1, 6, 8, 5, 0],
 [0, 1, 6, 9, 4, 0], [0, 1, 2, 7, 5, 0], [0, 1, 2, 3, 4, 0],
 [0, 4, 9, 7, 5, 0], [0, 4, 9, 6, 1, 0], [0, 4, 3, 8, 5, 0],
 [0, 4, 3, 2, 1, 0], [0, 5, 8, 6, 1, 0], [0, 5, 8, 3, 4, 0],
 [0, 5, 7, 2, 1, 0], [0, 5, 7, 9, 4, 0], [1, 2, 7, 9, 6, 1],
 [1, 2, 3, 8, 6, 1], [1, 6, 9, 7, 2, 1], [1, 6, 8, 3, 2, 1],
 [2, 3, 8, 5, 7, 2], [2, 3, 4, 9, 7, 2], [2, 7, 9, 4, 3, 2],
 [2, 7, 5, 8, 3, 2], [3, 4, 9, 6, 8, 3], [3, 8, 6, 9, 4, 3],
 [5, 7, 9, 6, 8, 5], [5, 8, 6, 9, 7, 5], [0, 1, 2, 3, 8, 5, 0],
 [0, 1, 2, 7, 9, 4, 0], [0, 1, 6, 9, 7, 5, 0],
 [0, 1, 6, 8, 3, 4, 0], [0, 4, 3, 2, 7, 5, 0],
 [0, 4, 3, 8, 6, 1, 0], [0, 4, 9, 6, 8, 5, 0],
 [0, 4, 9, 7, 2, 1, 0], [0, 5, 7, 9, 6, 1, 0],
 [0, 5, 7, 2, 3, 4, 0], [0, 5, 8, 3, 2, 1, 0],
 [0, 5, 8, 6, 9, 4, 0], [1, 2, 3, 4, 9, 6, 1],
 [1, 2, 7, 5, 8, 6, 1], [1, 6, 8, 5, 7, 2, 1],
 [1, 6, 9, 4, 3, 2, 1], [2, 3, 8, 6, 9, 7, 2],
 [2, 7, 9, 6, 8, 3, 2], [3, 4, 9, 7, 5, 8, 3],
 [3, 8, 5, 7, 9, 4, 3]]
```

```
>>> from sage.all import *
>>> g = graphs.PetersenGraph().to_directed()
>>> g.all_simple_cycles(max_length=Integer(4))
[[0, 1, 0], [0, 4, 0], [0, 5, 0], [1, 2, 1], [1, 6, 1], [2, 3, 2],
 [2, 7, 2], [3, 4, 3], [3, 8, 3], [4, 9, 4], [5, 7, 5], [5, 8, 5],
 [6, 8, 6], [6, 9, 6], [7, 9, 7]]
>>> g.all_simple_cycles(max_length=Integer(6))
[[0, 1, 0], [0, 4, 0], [0, 5, 0], [1, 2, 1], [1, 6, 1], [2, 3, 2],
 [2, 7, 2], [3, 4, 3], [3, 8, 3], [4, 9, 4], [5, 7, 5], [5, 8, 5],
 [6, 8, 6], [6, 9, 6], [7, 9, 7], [0, 1, 6, 8, 5, 0],
 [0, 1, 6, 9, 4, 0], [0, 1, 2, 7, 5, 0], [0, 1, 2, 3, 4, 0],
 [0, 4, 9, 7, 5, 0], [0, 4, 9, 6, 1, 0], [0, 4, 3, 8, 5, 0],
 [0, 4, 3, 2, 1, 0], [0, 5, 8, 6, 1, 0], [0, 5, 8, 3, 4, 0],
 [0, 5, 7, 2, 1, 0], [0, 5, 7, 9, 4, 0], [1, 2, 7, 9, 6, 1],
 [1, 2, 3, 8, 6, 1], [1, 6, 9, 7, 2, 1], [1, 6, 8, 3, 2, 1],
 [2, 3, 8, 5, 7, 2], [2, 3, 4, 9, 7, 2], [2, 7, 9, 4, 3, 2],
 [2, 7, 5, 8, 3, 2], [3, 4, 9, 6, 8, 3], [3, 8, 6, 9, 4, 3],
 [5, 7, 9, 6, 8, 5], [5, 8, 6, 9, 7, 5], [0, 1, 2, 3, 8, 5, 0],
```

(continues on next page)

(continued from previous page)

```
 [0, 1, 2, 7, 9, 4, 0], [0, 1, 6, 9, 7, 5, 0],
 [0, 1, 6, 8, 3, 4, 0], [0, 4, 3, 2, 7, 5, 0],
 [0, 4, 3, 8, 6, 1, 0], [0, 4, 9, 6, 8, 5, 0],
 [0, 4, 9, 7, 2, 1, 0], [0, 5, 7, 9, 6, 1, 0],
 [0, 5, 7, 2, 3, 4, 0], [0, 5, 8, 3, 2, 1, 0],
 [0, 5, 8, 6, 9, 4, 0], [1, 2, 3, 4, 9, 6, 1],
 [1, 2, 7, 5, 8, 6, 1], [1, 6, 8, 5, 7, 2, 1],
 [1, 6, 9, 4, 3, 2, 1], [2, 3, 8, 6, 9, 7, 2],
 [2, 7, 9, 6, 8, 3, 2], [3, 4, 9, 7, 5, 8, 3],
 [3, 8, 5, 7, 9, 4, 3]]
```

The complete graph (without loops) on 4 vertices:

```
sage: g = graphs.CompleteGraph(4).to_directed()
sage: g.all_simple_cycles()
[[0, 1, 0], [0, 2, 0], [0, 3, 0], [1, 2, 1], [1, 3, 1], [2, 3, 2],
 [0, 1, 3, 0], [0, 1, 2, 0], [0, 2, 3, 0], [0, 2, 1, 0],
 [0, 3, 2, 0], [0, 3, 1, 0], [1, 2, 3, 1], [1, 3, 2, 1],
 [0, 1, 2, 3, 0], [0, 1, 3, 2, 0], [0, 2, 1, 3, 0],
 [0, 2, 3, 1, 0], [0, 3, 1, 2, 0], [0, 3, 2, 1, 0]]
```

```
>>> from sage.all import *
>>> g = graphs.CompleteGraph(Integer(4)).to_directed()
>>> g.all_simple_cycles()
[[0, 1, 0], [0, 2, 0], [0, 3, 0], [1, 2, 1], [1, 3, 1], [2, 3, 2],
 [0, 1, 3, 0], [0, 1, 2, 0], [0, 2, 3, 0], [0, 2, 1, 0],
 [0, 3, 2, 0], [0, 3, 1, 0], [1, 2, 3, 1], [1, 3, 2, 1],
 [0, 1, 2, 3, 0], [0, 1, 3, 2, 0], [0, 2, 1, 3, 0],
 [0, 2, 3, 1, 0], [0, 3, 1, 2, 0], [0, 3, 2, 1, 0]]
```

If the graph contains a large number of cycles, one can bound the length of the cycles, or simply restrict the possible starting vertices of the cycles:

```
sage: g = graphs.CompleteGraph(20).to_directed()
sage: g.all_simple_cycles(max_length=2)
[[0, 1, 0], [0, 2, 0], [0, 3, 0], [0, 4, 0], [0, 5, 0], [0, 6, 0], [0, 7, 0],
 [0, 8, 0], [0, 9, 0], [0, 10, 0], [0, 11, 0], [0, 12, 0], [0, 13, 0], [0, 14, 0],
 [0, 15, 0], [0, 16, 0], [0, 17, 0], [0, 18, 0], [0, 19, 0], [1, 2, 1], [1, 3, 1],
 [1, 4, 1], [1, 5, 1], [1, 6, 1], [1, 7, 1], [1, 8, 1], [1, 9, 1], [2, 3, 2],
 [2, 4, 2], [2, 5, 2], [2, 6, 2], [2, 7, 2], [2, 8, 2], [2, 9, 2], [3, 4, 3],
 [3, 5, 3], [3, 6, 3], [3, 7, 3], [3, 8, 3], [3, 9, 3], [4, 5, 4], [4, 6, 4],
 [4, 7, 4], [4, 8, 4], [4, 9, 4], [5, 6, 5], [5, 7, 5], [5, 8, 5], [5, 9, 5],
 [6, 7, 6], [6, 8, 6], [6, 9, 6], [7, 8, 7], [7, 9, 7], [8, 9, 8], [10, 1, 10],
 [10, 2, 10], [10, 3, 10], [10, 4, 10], [10, 5, 10], [10, 6, 10], [10, 7, 10],
 [10, 8, 10], [10, 9, 10], [11, 1, 11], [11, 2, 11], [11, 3, 11], [11, 4, 11],
 [11, 5, 11], [11, 6, 11], [11, 7, 11], [11, 8, 11], [11, 9, 11], [11, 10, 11],
 [12, 1, 12], [12, 2, 12], [12, 3, 12], [12, 4, 12], [12, 5, 12], [12, 6, 12],
 [12, 7, 12], [12, 8, 12], [12, 9, 12], [12, 10, 12], [12, 11, 12], [13, 1, 13],
 [13, 2, 13], [13, 3, 13], [13, 4, 13], [13, 5, 13], [13, 6, 13], [13, 7, 13],
 [13, 8, 13], [13, 9, 13], [13, 10, 13], [13, 11, 13], [13, 12, 13], [14, 1, 14],
 [14, 2, 14], [14, 3, 14], [14, 4, 14], [14, 5, 14], [14, 6, 14], [14, 7, 14],
 [14, 8, 14], [14, 9, 14], [14, 10, 14], [14, 11, 14], [14, 12, 14], [14, 13, 14],
```

(continues on next page)

(continued from previous page)

```
 [15, 1, 15], [15, 2, 15], [15, 3, 15], [15, 4, 15], [15, 5, 15], [15, 6, 15],
 [15, 7, 15], [15, 8, 15], [15, 9, 15], [15, 10, 15], [15, 11, 15], [15, 12, 15],
 [15, 13, 15], [15, 14, 15], [16, 1, 16], [16, 2, 16], [16, 3, 16], [16, 4, 16],
 [16, 5, 16], [16, 6, 16], [16, 7, 16], [16, 8, 16], [16, 9, 16], [16, 10, 16],
 [16, 11, 16], [16, 12, 16], [16, 13, 16], [16, 14, 16], [16, 15, 16], [17, 1,␣
↪17],
 [17, 2, 17], [17, 3, 17], [17, 4, 17], [17, 5, 17], [17, 6, 17], [17, 7, 17],
 [17, 8, 17], [17, 9, 17], [17, 10, 17], [17, 11, 17], [17, 12, 17], [17, 13, 17],
 [17, 14, 17], [17, 15, 17], [17, 16, 17], [18, 1, 18], [18, 2, 18], [18, 3, 18],
 [18, 4, 18], [18, 5, 18], [18, 6, 18], [18, 7, 18], [18, 8, 18], [18, 9, 18],
 [18, 10, 18], [18, 11, 18], [18, 12, 18], [18, 13, 18], [18, 14, 18], [18, 15,␣
↪18],
 [18, 16, 18], [18, 17, 18], [19, 1, 19], [19, 2, 19], [19, 3, 19], [19, 4, 19],
 [19, 5, 19], [19, 6, 19], [19, 7, 19], [19, 8, 19], [19, 9, 19], [19, 10, 19],
 [19, 11, 19], [19, 12, 19], [19, 13, 19], [19, 14, 19], [19, 15, 19], [19, 16,␣
↪19],
 [19, 17, 19], [19, 18, 19]]

sage: g = graphs.CompleteGraph(20).to_directed()
sage: g.all_simple_cycles(max_length=2, starting_vertices=[0], algorithm='A')
[[0, 1, 0], [0, 2, 0], [0, 3, 0], [0, 4, 0], [0, 5, 0],
 [0, 6, 0], [0, 7, 0], [0, 8, 0], [0, 9, 0], [0, 10, 0],
 [0, 11, 0], [0, 12, 0], [0, 13, 0], [0, 14, 0], [0, 15, 0],
 [0, 16, 0], [0, 17, 0], [0, 18, 0], [0, 19, 0]]
```

```
>>> from sage.all import *
>>> g = graphs.CompleteGraph(Integer(20)).to_directed()
>>> g.all_simple_cycles(max_length=Integer(2))
[[0, 1, 0], [0, 2, 0], [0, 3, 0], [0, 4, 0], [0, 5, 0], [0, 6, 0], [0, 7, 0],
 [0, 8, 0], [0, 9, 0], [0, 10, 0], [0, 11, 0], [0, 12, 0], [0, 13, 0], [0, 14, 0],
 [0, 15, 0], [0, 16, 0], [0, 17, 0], [0, 18, 0], [0, 19, 0], [1, 2, 1], [1, 3, 1],
 [1, 4, 1], [1, 5, 1], [1, 6, 1], [1, 7, 1], [1, 8, 1], [1, 9, 1], [2, 3, 2],
 [2, 4, 2], [2, 5, 2], [2, 6, 2], [2, 7, 2], [2, 8, 2], [2, 9, 2], [3, 4, 3],
 [3, 5, 3], [3, 6, 3], [3, 7, 3], [3, 8, 3], [3, 9, 3], [4, 5, 4], [4, 6, 4],
 [4, 7, 4], [4, 8, 4], [4, 9, 4], [5, 6, 5], [5, 7, 5], [5, 8, 5], [5, 9, 5],
 [6, 7, 6], [6, 8, 6], [6, 9, 6], [7, 8, 7], [7, 9, 7], [8, 9, 8], [10, 1, 10],
 [10, 2, 10], [10, 3, 10], [10, 4, 10], [10, 5, 10], [10, 6, 10], [10, 7, 10],
 [10, 8, 10], [10, 9, 10], [11, 1, 11], [11, 2, 11], [11, 3, 11], [11, 4, 11],
 [11, 5, 11], [11, 6, 11], [11, 7, 11], [11, 8, 11], [11, 9, 11], [11, 10, 11],
 [12, 1, 12], [12, 2, 12], [12, 3, 12], [12, 4, 12], [12, 5, 12], [12, 6, 12],
 [12, 7, 12], [12, 8, 12], [12, 9, 12], [12, 10, 12], [12, 11, 12], [13, 1, 13],
 [13, 2, 13], [13, 3, 13], [13, 4, 13], [13, 5, 13], [13, 6, 13], [13, 7, 13],
 [13, 8, 13], [13, 9, 13], [13, 10, 13], [13, 11, 13], [13, 12, 13], [14, 1, 14],
 [14, 2, 14], [14, 3, 14], [14, 4, 14], [14, 5, 14], [14, 6, 14], [14, 7, 14],
 [14, 8, 14], [14, 9, 14], [14, 10, 14], [14, 11, 14], [14, 12, 14], [14, 13, 14],
 [15, 1, 15], [15, 2, 15], [15, 3, 15], [15, 4, 15], [15, 5, 15], [15, 6, 15],
 [15, 7, 15], [15, 8, 15], [15, 9, 15], [15, 10, 15], [15, 11, 15], [15, 12, 15],
 [15, 13, 15], [15, 14, 15], [16, 1, 16], [16, 2, 16], [16, 3, 16], [16, 4, 16],
 [16, 5, 16], [16, 6, 16], [16, 7, 16], [16, 8, 16], [16, 9, 16], [16, 10, 16],
 [16, 11, 16], [16, 12, 16], [16, 13, 16], [16, 14, 16], [16, 15, 16], [17, 1,␣
↪17],
 [17, 2, 17], [17, 3, 17], [17, 4, 17], [17, 5, 17], [17, 6, 17], [17, 7, 17],
```

(continues on next page)

(continued from previous page)

```
 [17, 8, 17], [17, 9, 17], [17, 10, 17], [17, 11, 17], [17, 12, 17], [17, 13, 17],
 [17, 14, 17], [17, 15, 17], [17, 16, 17], [18, 1, 18], [18, 2, 18], [18, 3, 18],
 [18, 4, 18], [18, 5, 18], [18, 6, 18], [18, 7, 18], [18, 8, 18], [18, 9, 18],
 [18, 10, 18], [18, 11, 18], [18, 12, 18], [18, 13, 18], [18, 14, 18], [18, 15,␣
↪18],
 [18, 16, 18], [18, 17, 18], [19, 1, 19], [19, 2, 19], [19, 3, 19], [19, 4, 19],
 [19, 5, 19], [19, 6, 19], [19, 7, 19], [19, 8, 19], [19, 9, 19], [19, 10, 19],
 [19, 11, 19], [19, 12, 19], [19, 13, 19], [19, 14, 19], [19, 15, 19], [19, 16,␣
↪19],
 [19, 17, 19], [19, 18, 19]]

>>> g = graphs.CompleteGraph(Integer(20)).to_directed()
>>> g.all_simple_cycles(max_length=Integer(2), starting_vertices=[Integer(0)],␣
↪algorithm='A')
[[0, 1, 0], [0, 2, 0], [0, 3, 0], [0, 4, 0], [0, 5, 0],
 [0, 6, 0], [0, 7, 0], [0, 8, 0], [0, 9, 0], [0, 10, 0],
 [0, 11, 0], [0, 12, 0], [0, 13, 0], [0, 14, 0], [0, 15, 0],
 [0, 16, 0], [0, 17, 0], [0, 18, 0], [0, 19, 0]]
```

One may prefer to distinguish equivalent cycles having distinct starting vertices (compare the following examples):

```
sage: g = graphs.CompleteGraph(4).to_directed()
sage: g.all_simple_cycles(max_length=2, rooted=False, algorithm='A')
[[0, 1, 0], [0, 2, 0], [0, 3, 0], [1, 2, 1], [1, 3, 1], [2, 3, 2]]
sage: g.all_simple_cycles(max_length=2, rooted=True, algorithm='A')
[[0, 1, 0], [0, 2, 0], [0, 3, 0], [1, 0, 1], [1, 2, 1], [1, 3, 1],
 [2, 0, 2], [2, 1, 2], [2, 3, 2], [3, 0, 3], [3, 1, 3], [3, 2, 3]]
```

```
>>> from sage.all import *
>>> g = graphs.CompleteGraph(Integer(4)).to_directed()
>>> g.all_simple_cycles(max_length=Integer(2), rooted=False, algorithm='A')
[[0, 1, 0], [0, 2, 0], [0, 3, 0], [1, 2, 1], [1, 3, 1], [2, 3, 2]]
>>> g.all_simple_cycles(max_length=Integer(2), rooted=True, algorithm='A')
[[0, 1, 0], [0, 2, 0], [0, 3, 0], [1, 0, 1], [1, 2, 1], [1, 3, 1],
 [2, 0, 2], [2, 1, 2], [2, 3, 2], [3, 0, 3], [3, 1, 3], [3, 2, 3]]
```

A cycle is enumerated in increasing length order for a weighted graph:

```
sage: cycles = g.all_simple_cycles(weight_function=lambda e:e[0]+e[1],
....:                              by_weight=True, report_weight=True)
sage: cycles
[(2.0, [0, 1, 0]), (4.0, [0, 2, 0]), (6.0, [0, 1, 2, 0]), (6.0, [0, 2, 1, 0]),
 (6.0, [0, 3, 0]), (6.0, [1, 2, 1]), (8.0, [0, 1, 3, 0]), (8.0, [0, 3, 1, 0]),
 (8.0, [1, 3, 1]), (10.0, [0, 2, 3, 0]), (10.0, [0, 3, 2, 0]), (10.0, [2, 3, 2]),
 (12.0, [0, 1, 3, 2, 0]), (12.0, [0, 1, 2, 3, 0]), (12.0, [0, 2, 3, 1, 0]),
 (12.0, [0, 2, 1, 3, 0]), (12.0, [0, 3, 2, 1, 0]), (12.0, [0, 3, 1, 2, 0]),
 (12.0, [1, 2, 3, 1]), (12.0, [1, 3, 2, 1])]
```

```
>>> from sage.all import *
>>> cycles = g.all_simple_cycles(weight_function=lambda␣
↪e:e[Integer(0)]+e[Integer(1)],
...                              by_weight=True, report_weight=True)
```

(continues on next page)

(continued from previous page)

```
>>> cycles
[(2.0, [0, 1, 0]), (4.0, [0, 2, 0]), (6.0, [0, 1, 2, 0]), (6.0, [0, 2, 1, 0]),
 (6.0, [0, 3, 0]), (6.0, [1, 2, 1]), (8.0, [0, 1, 3, 0]), (8.0, [0, 3, 1, 0]),
 (8.0, [1, 3, 1]), (10.0, [0, 2, 3, 0]), (10.0, [0, 3, 2, 0]), (10.0, [2, 3, 2]),
 (12.0, [0, 1, 3, 2, 0]), (12.0, [0, 1, 2, 3, 0]), (12.0, [0, 2, 3, 1, 0]),
 (12.0, [0, 2, 1, 3, 0]), (12.0, [0, 3, 2, 1, 0]), (12.0, [0, 3, 1, 2, 0]),
 (12.0, [1, 2, 3, 1]), (12.0, [1, 3, 2, 1])]
```

The algorithm `'B'` can be used:

```
sage: cycles_B = g.all_simple_cycles(weight_function=lambda e:e[0]+e[1], by_
↪weight=True,
....:                                report_weight=True, algorithm='B')
sage: cycles_B
[(2.0, [0, 1, 0]), (4.0, [0, 2, 0]), (6.0, [0, 1, 2, 0]), (6.0, [0, 2, 1, 0]),
 (6.0, [0, 3, 0]), (6.0, [1, 2, 1]), (8.0, [0, 1, 3, 0]), (8.0, [0, 3, 1, 0]),
 (8.0, [1, 3, 1]), (10.0, [0, 2, 3, 0]), (10.0, [0, 3, 2, 0]), (10.0, [2, 3, 2]),
 (12.0, [0, 1, 3, 2, 0]), (12.0, [0, 1, 2, 3, 0]), (12.0, [0, 2, 3, 1, 0]),
 (12.0, [0, 2, 1, 3, 0]), (12.0, [0, 3, 2, 1, 0]), (12.0, [0, 3, 1, 2, 0]),
 (12.0, [1, 2, 3, 1]), (12.0, [1, 3, 2, 1])]
sage: cycles.sort() == cycles_B.sort()
True
```

```
>>> from sage.all import *
>>> cycles_B = g.all_simple_cycles(weight_function=lambda␣
↪e:e[Integer(0)]+e[Integer(1)], by_weight=True,
...                                report_weight=True, algorithm='B')
>>> cycles_B
[(2.0, [0, 1, 0]), (4.0, [0, 2, 0]), (6.0, [0, 1, 2, 0]), (6.0, [0, 2, 1, 0]),
 (6.0, [0, 3, 0]), (6.0, [1, 2, 1]), (8.0, [0, 1, 3, 0]), (8.0, [0, 3, 1, 0]),
 (8.0, [1, 3, 1]), (10.0, [0, 2, 3, 0]), (10.0, [0, 3, 2, 0]), (10.0, [2, 3, 2]),
 (12.0, [0, 1, 3, 2, 0]), (12.0, [0, 1, 2, 3, 0]), (12.0, [0, 2, 3, 1, 0]),
 (12.0, [0, 2, 1, 3, 0]), (12.0, [0, 3, 2, 1, 0]), (12.0, [0, 3, 1, 2, 0]),
 (12.0, [1, 2, 3, 1]), (12.0, [1, 3, 2, 1])]
>>> cycles.sort() == cycles_B.sort()
True
```

The algorithm `'A'` is available for undirected graphs. Specifically, each cycle is enumerated exactly once, meaning a cycle and its reverse are not listed separately:

```
sage: g = Graph({0: [1, 2], 1: [0, 2], 2: [0, 1]})
sage: g.all_simple_cycles(algorithm='A')
[[0, 1, 2, 0]]
```

```
>>> from sage.all import *
>>> g = Graph({Integer(0): [Integer(1), Integer(2)], Integer(1): [Integer(0),␣
↪Integer(2)], Integer(2): [Integer(0), Integer(1)]})
>>> g.all_simple_cycles(algorithm='A')
[[0, 1, 2, 0]]
```

CHAPTER

SIX

INDICES AND TABLES

- Index
- Module Index
- Search Page

BIBLIOGRAPHY

[Lein] Tom Leinster, *The magnitude of metric spaces*. Doc. Math. 18 (2013), 857-905.

PYTHON MODULE INDEX

c

sage.combinat.designs.incidence_structures, 1824

g

sage.graphs.asteroidal_triples, 1884
sage.graphs.base.boost_graph, 1801
sage.graphs.base.c_graph, 1680
sage.graphs.base.dense_graph, 1755
sage.graphs.base.graph_backends, 1795
sage.graphs.base.overview, 1679
sage.graphs.base.sparse_graph, 1743
sage.graphs.base.static_dense_graph, 1766
sage.graphs.base.static_sparse_backend, 1784
sage.graphs.base.static_sparse_graph, 1776
sage.graphs.bipartite_graph, 830
sage.graphs.centrality, 1880
sage.graphs.chrompoly, 1888
sage.graphs.cliquer, 1876
sage.graphs.cographs, 1892
sage.graphs.comparability, 1897
sage.graphs.connectivity, 2299
sage.graphs.convexity_properties, 2162
sage.graphs.cycle_enumeration, 2384
sage.graphs.digraph, 764
sage.graphs.digraph_generators, 1282
sage.graphs.distances_all_pairs, 2171
sage.graphs.domination, 2366
sage.graphs.edge_connectivity, 2360
sage.graphs.generators.basic, 1387
sage.graphs.generators.chessboard, 1414
sage.graphs.generators.classical_geometries, 1421
sage.graphs.generators.degree_sequence, 1438
sage.graphs.generators.distance_regular, 1442
sage.graphs.generators.families, 1466
sage.graphs.generators.intersection, 1526
sage.graphs.generators.luw_graphs, 1373
sage.graphs.generators.platonic_solids, 1536
sage.graphs.generators.random, 1542
sage.graphs.generators.smallgraphs, 1566
sage.graphs.generators.trees, 1648
sage.graphs.generators.world_map, 1661
sage.graphs.generic_graph, 1
sage.graphs.generic_graph_pyx, 2274
sage.graphs.genus, 1980
sage.graphs.graph, 498
sage.graphs.graph_coloring, 1857
sage.graphs.graph_database, 1307
sage.graphs.graph_decompositions.bandwidth, 2105
sage.graphs.graph_decompositions.clique_separators, 2154
sage.graphs.graph_decompositions.cutwidth, 2108
sage.graphs.graph_decompositions.fast_digraph, 2115
sage.graphs.graph_decompositions.graph_products, 2116
sage.graphs.graph_decompositions.modular_decomposition, 2134
sage.graphs.graph_decompositions.rankwidth, 2101
sage.graphs.graph_decompositions.slice_decomposition, 2123
sage.graphs.graph_decompositions.tree_decomposition, 2064
sage.graphs.graph_decompositions.vertex_separation, 2083
sage.graphs.graph_editor, 2211
sage.graphs.graph_generators, 963
sage.graphs.graph_generators_pyx, 1307
sage.graphs.graph_input, 2219
sage.graphs.graph_latex, 2189
sage.graphs.graph_list, 2212
sage.graphs.graph_plot, 2012
sage.graphs.graph_plot_js, 2061
sage.graphs.hyperbolicity, 2230
sage.graphs.hypergraph_generators, 1821
sage.graphs.independent_sets, 1886
sage.graphs.isgci, 1665
sage.graphs.isoperimetric_inequalities, 1907
sage.graphs.line_graph, 1911

sage.graphs.lovasz_theta, 1985
sage.graphs.matching, 1945
sage.graphs.matching_covered_graph, 868
sage.graphs.matchpoly, 1966
sage.graphs.morphisms, 1972
sage.graphs.orientations, 2285
sage.graphs.partial_cube, 2246
sage.graphs.path_enumeration, 2249
sage.graphs.planarity, 1991
sage.graphs.pq_trees, 1935
sage.graphs.schnyder, 1986
sage.graphs.spanning_tree, 1918
sage.graphs.strongly_regular_db, 1334
sage.graphs.traversals, 1992
sage.graphs.tutte_polynomial, 2237
sage.graphs.views, 952
sage.graphs.weakly_chordal, 2168

INDEX

Non-alphabetical

__eq__() (*sage.graphs.generic_graph.GenericGraph method*), 8
__getitem__() (*sage.graphs.graph_decompositions.slice_decomposition.SliceDecomposition method*), 2124
_circle_embedding() (*sage.graphs.generic_graph.GenericGraph method*), 10
_get_weight_function() (*sage.graphs.generic_graph.GenericGraph method*), 12
_line_embedding() (*sage.graphs.generic_graph.GenericGraph method*), 13

A

active_edges() (*sage.graphs.graph_decompositions.slice_decomposition.SliceDecomposition method*), 2126
acyclic_edge_coloring() (*in module sage.graphs.graph_coloring*), 1858
acyclic_orientations() (*in module sage.graphs.orientations*), 2286
acyclic_orientations() (*sage.graphs.graph.Graph method*), 529
add_arc() (*sage.graphs.base.c_graph.CGraph method*), 1680
add_arc_label() (*sage.graphs.base.sparse_graph.SparseGraph method*), 1748
add_child() (*sage.graphs.cographs.CoTree method*), 1893
add_clique() (*sage.graphs.generic_graph.GenericGraph method*), 14
add_cycle() (*sage.graphs.generic_graph.GenericGraph method*), 16
add_edge() (*sage.graphs.base.c_graph.CGraphBackend method*), 1707
add_edge() (*sage.graphs.base.graph_backends.GenericGraphBackend method*), 1795
add_edge() (*sage.graphs.base.static_sparse_backend.StaticSparseBackend method*), 1789
add_edge() (*sage.graphs.bipartite_graph.BipartiteGraph method*), 839
add_edge() (*sage.graphs.generic_graph.GenericGraph method*), 16
add_edge() (*sage.graphs.matching_covered_graph.MatchingCoveredGraph method*), 879
add_edges() (*sage.graphs.base.c_graph.CGraphBackend method*), 1707
add_edges() (*sage.graphs.base.dense_graph.DenseGraphBackend method*), 1762
add_edges() (*sage.graphs.base.graph_backends.GenericGraphBackend method*), 1796
add_edges() (*sage.graphs.base.static_sparse_backend.StaticSparseBackend method*), 1789
add_edges() (*sage.graphs.bipartite_graph.BipartiteGraph method*), 839
add_edges() (*sage.graphs.generic_graph.GenericGraph method*), 18
add_edges() (*sage.graphs.matching_covered_graph.MatchingCoveredGraph method*), 886
add_path() (*sage.graphs.generic_graph.GenericGraph method*), 19
add_vertex() (*sage.graphs.base.c_graph.CGraph method*), 1682
add_vertex() (*sage.graphs.base.c_graph.CGraphBackend method*), 1708
add_vertex() (*sage.graphs.base.graph_backends.GenericGraphBackend method*), 1796
add_vertex() (*sage.graphs.base.static_sparse_backend.StaticSparseBackend method*), 1789
add_vertex() (*sage.graphs.base.static_sparse_backend.StaticSparseCGraph method*), 1794
add_vertex() (*sage.graphs.bipartite_graph.BipartiteGraph method*), 841
add_vertex() (*sage.graphs.generic_graph.GenericGraph method*), 20
add_vertex() (*sage.graphs.matching_covered_graph.MatchingCoveredGraph method*), 892
add_vertices() (*sage.graphs.base.c_graph.CGraph*

method), 1684
add_vertices() (*sage.graphs.base.c_graph.CGraphBackend method*), 1709
add_vertices() (*sage.graphs.base.graph_backends.GenericGraphBackend method*), 1796
add_vertices() (*sage.graphs.base.static_sparse_backend.StaticSparseBackend method*), 1789
add_vertices() (*sage.graphs.bipartite_graph.BipartiteGraph method*), 842
add_vertices() (*sage.graphs.generic_graph.GenericGraph method*), 21
add_vertices() (*sage.graphs.matching_covered_graph.MatchingCoveredGraph method*), 893
adjacency_matrix() (*sage.graphs.generators.luw_graphs.LUWGraphDescriptor method*), 1376
adjacency_matrix() (*sage.graphs.generic_graph.GenericGraph method*), 22
AffineOrthogonalPolarGraph() (*in module sage.graphs.generators.classical_geometries*), 1421
AffineOrthogonalPolarGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 981
AfricaMap() (*in module sage.graphs.generators.world_map*), 1661
AfricaMap() (*sage.graphs.graph_generators.GraphGenerators static method*), 983
AhrensSzekeresGeneralizedQuadrangleGraph() (*in module sage.graphs.generators.classical_geometries*), 1422
AhrensSzekeresGeneralizedQuadrangleGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 984
Akq() (*in module sage.graphs.generators.luw_graphs*), 1373
Akq() (*sage.graphs.graph_generators.GraphGenerators static method*), 985
all_arcs() (*sage.graphs.base.c_graph.CGraph method*), 1686
all_cliques() (*in module sage.graphs.cliquer*), 1877
all_cliques() (*sage.graphs.graph.Graph method*), 530
all_cycles_iterator() (*in module sage.graphs.cycle_enumeration*), 2384
all_cycles_iterator() (*sage.graphs.generic_graph.GenericGraph method*), 27
all_graph_colorings() (*in module sage.graphs.graph_coloring*), 1860
all_max_clique() (*in module sage.graphs.cliquer*), 1878
all_paths() (*in module sage.graphs.path_enumeration*), 2249
all_paths() (*sage.graphs.generic_graph.GenericGraph method*), 33
all_paths_iterator() (*in module sage.graphs.path_enumeration*), 2252
all_paths_iterator() (*sage.graphs.generic_graph.GenericGraph method*), 36
all_simple_cycles() (*in module sage.graphs.cycle_enumeration*), 2389
all_simple_cycles() (*sage.graphs.generic_graph.GenericGraph method*), 42
all_simple_paths() (*in module sage.graphs.path_enumeration*), 2258
all_simple_paths() (*sage.graphs.generic_graph.GenericGraph method*), 48
allow_loops() (*sage.graphs.bipartite_graph.BipartiteGraph method*), 843
allow_loops() (*sage.graphs.generic_graph.GenericGraph method*), 53
allow_loops() (*sage.graphs.matching_covered_graph.MatchingCoveredGraph method*), 895
allow_multiple_edges() (*sage.graphs.generic_graph.GenericGraph method*), 55
allows_loops() (*sage.graphs.base.static_sparse_backend.StaticSparseBackend method*), 1789
allows_loops() (*sage.graphs.generic_graph.GenericGraph method*), 57
allows_loops() (*sage.graphs.matching_covered_graph.MatchingCoveredGraph method*), 896
allows_multiple_edges() (*sage.graphs.generic_graph.GenericGraph method*), 59
AlternatingFormsGraph() (*in module sage.graphs.generators.distance_regular*), 1443
AlternatingFormsGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 985
am() (*sage.graphs.generic_graph.GenericGraph method*), 60
antipodal_graph() (*in module sage.graphs.distances_all_pairs*), 2173
antipodal_graph() (*sage.graphs.graph.Graph method*), 532
antisymmetric() (*sage.graphs.generic_graph.GenericGraph method*), 60
apex_vertices() (*sage.graphs.graph.Graph method*), 533
apparently_feasible_parameters() (*in module sage.graphs.strongly_regular_db*), 1345
append_child() (*sage.graphs.schnyder.TreeNode method*), 1987

arboricity() (*sage.graphs.graph.Graph method*), 536
arc_label() (*sage.graphs.base.c_graph.CGraph method*), 1686
atoms_and_clique_separators() (*in module sage.graphs.graph_decompositions.clique_separators*), 2154
atoms_and_clique_separators() (*sage.graphs.graph.Graph method*), 536
auslander_reiten_quiver() (*sage.graphs.digraph.DiGraph method*), 773
automorphism_group() (*sage.combinat.designs.incidence_structures.IncidenceStructure method*), 1828
automorphism_group() (*sage.graphs.generic_graph.GenericGraph method*), 62
average_degree() (*sage.graphs.generic_graph.GenericGraph method*), 67
average_distance() (*sage.graphs.generic_graph.GenericGraph method*), 67
AztecDiamondGraph() (*in module sage.graphs.generators.families*), 1466
AztecDiamondGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 986

B

b_coloring() (*in module sage.graphs.graph_coloring*), 1862
Balaban10Cage() (*in module sage.graphs.generators.smallgraphs*), 1566
Balaban10Cage() (*sage.graphs.graph_generators.GraphGenerators static method*), 986
Balaban11Cage() (*in module sage.graphs.generators.smallgraphs*), 1567
Balaban11Cage() (*sage.graphs.graph_generators.GraphGenerators static method*), 987
BalancedTree() (*in module sage.graphs.generators.trees*), 1651
BalancedTree() (*sage.graphs.graph_generators.GraphGenerators static method*), 989
ball() (*sage.graphs.generators.luw_graphs.LUWGraphDescriptor method*), 1377
bandwidth() (*in module sage.graphs.graph_decompositions.bandwidth*), 2106
bandwidth() (*sage.graphs.graph.Graph method*), 543
bandwidth_heuristics() (*in module sage.graphs.base.boost_graph*), 1801
BarbellGraph() (*in module sage.graphs.generators.families*), 1466
BarbellGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 991
base_ring() (*sage.graphs.generators.luw_graphs.LUWGraphDescriptor method*), 1378
basic (*sage.graphs.graph_generators.GraphGenerators attribute*), 1237
BFS_with_condition() (*in module sage.graphs.traversals*), 1993
biconnected_components() (*in module sage.graphs.connectivity*), 2309
biconnected_components() (*sage.graphs.generic_graph.GenericGraph method*), 68
biconnected_components_subgraphs() (*in module sage.graphs.connectivity*), 2310
biconnected_components_subgraphs() (*sage.graphs.generic_graph.GenericGraph method*), 69
BidiakisCube() (*in module sage.graphs.generators.smallgraphs*), 1569
BidiakisCube() (*sage.graphs.graph_generators.GraphGenerators static method*), 992
bidirectional_dijkstra() (*sage.graphs.base.c_graph.CGraphBackend method*), 1710
bidirectional_dijkstra_special() (*sage.graphs.base.c_graph.CGraphBackend method*), 1711
BiggsSmithGraph() (*in module sage.graphs.generators.smallgraphs*), 1571
BiggsSmithGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 994
BilinearFormsGraph() (*in module sage.graphs.generators.distance_regular*), 1444
BilinearFormsGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 995
binary_string_from_dig6() (*in module sage.graphs.generic_graph_pyx*), 2276
binary_string_from_graph6() (*in module sage.graphs.generic_graph_pyx*), 2277
binary_string_to_graph6() (*in module sage.graphs.generic_graph_pyx*), 2278
BinomialRandomUniform() (*sage.graphs.hypergraph_generators.HypergraphGenerators method*), 1821
bipartite_color() (*sage.graphs.graph.Graph method*), 545
bipartite_double() (*sage.graphs.graph.Graph method*), 545
bipartite_sets() (*sage.graphs.graph.Graph method*), 547
BipartiteGraph (*class in sage.graphs.bipartite_graph*), 830
bipartition() (*sage.graphs.bipartite_graph.BipartiteGraph method*), 843
BishopGraph() (*in module sage.graphs.generators.chessboard*), 1414
BishopGraph() (*sage.graphs.graph_generators.Graph-*

Generators static method), 996
BiwheelGraph() (*in module sage.graphs.generators.families*), 1468
BiwheelGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 997
BlanusaFirstSnarkGraph() (*in module sage.graphs.generators.smallgraphs*), 1572
BlanusaFirstSnarkGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 998
BlanusaSecondSnarkGraph() (*in module sage.graphs.generators.smallgraphs*), 1572
BlanusaSecondSnarkGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 999
block_sizes() (*sage.combinat.designs.incidence_structures.IncidenceStructure method*), 1830
blocks() (*sage.combinat.designs.incidence_structures.IncidenceStructure method*), 1830
blocks_and_cut_vertices() (*in module sage.graphs.base.boost_graph*), 1802
blocks_and_cut_vertices() (*in module sage.graphs.connectivity*), 2311
blocks_and_cut_vertices() (*sage.graphs.generic_graph.GenericGraph method*), 70
blocks_and_cuts_tree() (*in module sage.graphs.connectivity*), 2314
blocks_and_cuts_tree() (*sage.graphs.generic_graph.GenericGraph method*), 74
blossoming_contour() (*in module sage.graphs.generators.random*), 1563
boruvka() (*in module sage.graphs.spanning_tree*), 1918
bounded_outdegree_orientation() (*in module sage.graphs.orientations*), 2287
bounded_outdegree_orientation() (*sage.graphs.graph.Graph method*), 547
breadth_first_level_search() (*in module sage.graphs.partial_cube*), 2247
breadth_first_search() (*sage.graphs.base.c_graph.CGraphBackend method*), 1712
breadth_first_search() (*sage.graphs.generic_graph.GenericGraph method*), 75
bridges() (*in module sage.graphs.connectivity*), 2316
bridges() (*sage.graphs.graph.Graph method*), 549
BrinkmannGraph() (*in module sage.graphs.generators.smallgraphs*), 1573
BrinkmannGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1000
BrouwerHaemersGraph() (*in module sage.graphs.generators.smallgraphs*), 1575
BrouwerHaemersGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1002
BubbleSortGraph() (*in module sage.graphs.generators.families*), 1469
BubbleSortGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1002
BuckyBall() (*in module sage.graphs.generators.smallgraphs*), 1576
BuckyBall() (*sage.graphs.graph_generators.GraphGenerators static method*), 1004
BullGraph() (*in module sage.graphs.generators.basic*), 1387
BullGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1004
ButterflyGraph() (*in module sage.graphs.generators.basic*), 1389
ButterflyGraph() (*sage.graphs.digraph_generators.DiGraphGenerators method*), 1286
ButterflyGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1006

C

c_graph() (*sage.graphs.base.c_graph.CGraphBackend method*), 1714
CaiFurerImmermanGraph() (*in module sage.graphs.generators.families*), 1470
CaiFurerImmermanGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1008
CameronGraph() (*in module sage.graphs.generators.smallgraphs*), 1577
CameronGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1009
canaug_traverse_edge() (*in module sage.graphs.graph_generators*), 1278
canaug_traverse_vert() (*in module sage.graphs.graph_generators*), 1279
canonical_label() (*sage.combinat.designs.incidence_structures.IncidenceStructure method*), 1830
canonical_label() (*sage.graphs.bipartite_graph.BipartiteGraph method*), 843
canonical_label() (*sage.graphs.generic_graph.GenericGraph method*), 79
canonical_partition() (*sage.graphs.matching_covered_graph.MatchingCoveredGraph method*), 897
cardinality() (*sage.graphs.generic_graph_pyx.SubgraphSearch method*), 2275
cardinality() (*sage.graphs.independent_sets.IndependentSets method*), 1888
cardinality() (*sage.graphs.pq_trees.P method*), 1937
cardinality() (*sage.graphs.pq_trees.Q method*), 1942
cartesian_product() (*sage.graphs.generic_graph.GenericGraph method*), 83

categorical_product() (*sage.graphs.generic_graph.GenericGraph method*), 83
Caterpillar() (*in module sage.graphs.generators.trees*), 1653
Caterpillar() (*sage.graphs.graph_generators.GraphGenerators static method*), 1010
Cell120() (*in module sage.graphs.generators.smallgraphs*), 1577
Cell120() (*sage.graphs.graph_generators.GraphGenerators static method*), 1011
Cell600() (*in module sage.graphs.generators.smallgraphs*), 1578
Cell600() (*sage.graphs.graph_generators.GraphGenerators static method*), 1012
center() (*sage.graphs.digraph.DiGraph method*), 774
center() (*sage.graphs.graph.Graph method*), 550
centrality_betweenness() (*in module sage.graphs.centrality*), 1881
centrality_betweenness() (*sage.graphs.generic_graph.GenericGraph method*), 83
centrality_closeness() (*sage.graphs.generic_graph.GenericGraph method*), 85
centrality_closeness_random_k() (*in module sage.graphs.centrality*), 1882
centrality_closeness_top_k() (*in module sage.graphs.centrality*), 1883
centrality_degree() (*sage.graphs.graph.Graph method*), 551
CGraph (*class in sage.graphs.base.c_graph*), 1680
CGraphBackend (*class in sage.graphs.base.c_graph*), 1706
chang_graphs() (*in module sage.graphs.generators.families*), 1522
chang_graphs() (*sage.graphs.graph_generators.GraphGenerators static method*), 1237
change_label() (*in module sage.graphs.cographs*), 1894
characteristic_polynomial() (*sage.graphs.generic_graph.GenericGraph method*), 88
charpoly() (*sage.graphs.generic_graph.GenericGraph method*), 89
check_algos_are_equivalent() (*in module sage.graphs.graph_decompositions.modular_decomposition*), 2136
check_aut() (*in module sage.graphs.graph_generators*), 1281
check_aut_edge() (*in module sage.graphs.graph_generators*), 1281
check_gamma_modules() (*in module sage.graphs.graph_decompositions.modular_decomposition*), 2137
check_maximal_modules() (*in module sage.graphs.graph_decompositions.modular_decomposition*), 2137
check_modular_decomposition() (*in module sage.graphs.graph_decompositions.modular_decomposition*), 2138
check_module() (*in module sage.graphs.graph_decompositions.modular_decomposition*), 2138
check_tkz_graph() (*in module sage.graphs.graph_latex*), 2210
check_vertex() (*sage.graphs.base.c_graph.CGraph method*), 1687
cheeger_constant() (*in module sage.graphs.isoperimetric_inequalities*), 1907
cheeger_constant() (*sage.graphs.graph.Graph method*), 552
chessboard (*sage.graphs.graph_generators.GraphGenerators attribute*), 1239
ChessboardGraphGenerator() (*in module sage.graphs.generators.chessboard*), 1415
ChessboardGraphGenerator() (*sage.graphs.graph_generators.GraphGenerators static method*), 1012
children_node_type() (*in module sage.graphs.graph_decompositions.modular_decomposition*), 2139
chromatic_index() (*sage.graphs.graph.Graph method*), 554
chromatic_number() (*in module sage.graphs.graph_coloring*), 1863
chromatic_number() (*sage.graphs.graph.Graph method*), 555
chromatic_polynomial() (*in module sage.graphs.chrompoly*), 1889
chromatic_polynomial() (*sage.graphs.graph.Graph method*), 557
chromatic_polynomial_with_cache() (*in module sage.graphs.chrompoly*), 1891
chromatic_quasisymmetric_function() (*sage.graphs.graph.Graph method*), 560
chromatic_symmetric_function() (*sage.graphs.graph.Graph method*), 562
ChvatalGraph() (*in module sage.graphs.generators.smallgraphs*), 1578
ChvatalGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1014
Circuit() (*sage.graphs.digraph_generators.DiGraphGenerators method*), 1287
Circulant() (*sage.graphs.digraph_generators.DiGraphGenerators method*), 1288
CirculantGraph() (*in module sage.graphs.generators.families*), 1472
CirculantGraph() (*sage.graphs.graph_genera-*

tors.GraphGenerators static method), 1015
CircularLadderGraph() (*in module sage.graphs.generators.basic*), 1390
CircularLadderGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1017
classes() (*sage.graphs.isgci.GraphClasses method*), 1674
classical_geometries (*sage.graphs.graph_generators.GraphGenerators attribute*), 1239
ClawGraph() (*in module sage.graphs.generators.basic*), 1392
ClawGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1019
clear() (*sage.graphs.generic_graph.GenericGraph method*), 89
cleave() (*in module sage.graphs.connectivity*), 2317
cleave() (*sage.graphs.graph.Graph method*), 564
ClebschGraph() (*in module sage.graphs.generators.smallgraphs*), 1579
ClebschGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1019
clique_complex() (*sage.graphs.graph.Graph method*), 567
clique_maximum() (*sage.graphs.graph.Graph method*), 568
clique_number() (*in module sage.graphs.cliquer*), 1879
clique_number() (*sage.graphs.graph.Graph method*), 569
clique_polynomial() (*sage.graphs.graph.Graph method*), 571
cliques_containing_vertex() (*sage.graphs.graph.Graph method*), 571
cliques_get_clique_bipartite() (*sage.graphs.graph.Graph method*), 573
cliques_get_max_clique_graph() (*sage.graphs.graph.Graph method*), 574
cliques_maximal() (*sage.graphs.graph.Graph method*), 575
cliques_maximum() (*sage.graphs.graph.Graph method*), 576
cliques_number_of() (*sage.graphs.graph.Graph method*), 576
cliques_vertex_clique_number() (*sage.graphs.graph.Graph method*), 578
cluster_transitivity() (*sage.graphs.generic_graph.GenericGraph method*), 90
cluster_triangles() (*sage.graphs.generic_graph.GenericGraph method*), 90
clustering_average() (*sage.graphs.generic_graph.GenericGraph method*), 91
clustering_coeff() (*in module sage.graphs.base.boost_graph*), 1804
clustering_coeff() (*sage.graphs.generic_graph.GenericGraph method*), 92
coarsest_equitable_refinement() (*sage.graphs.generic_graph.GenericGraph method*), 93
cocliques_HoffmannSingleton() (*in module sage.graphs.generators.distance_regular*), 1454
cocliques_HoffmannSingleton() (*sage.graphs.graph_generators.GraphGenerators static method*), 1239
cographs() (*in module sage.graphs.cographs*), 1895
cographs() (*sage.graphs.graph_generators.GraphGenerators static method*), 1239
cographs_module (*sage.graphs.graph_generators.GraphGenerators attribute*), 1240
coloring() (*sage.combinat.designs.incidence_structures.IncidenceStructure method*), 1831
coloring() (*sage.graphs.graph.Graph method*), 579
common_neighbors_matrix() (*sage.graphs.graph.Graph method*), 580
complement() (*sage.combinat.designs.incidence_structures.IncidenceStructure method*), 1832
complement() (*sage.graphs.base.dense_graph.DenseGraph method*), 1758
complement() (*sage.graphs.bipartite_graph.BipartiteGraph method*), 846
complement() (*sage.graphs.generic_graph.GenericGraph method*), 95
complement_bipartite() (*sage.graphs.bipartite_graph.BipartiteGraph method*), 847
Complete() (*sage.graphs.digraph_generators.DiGraphGenerators method*), 1289
complete_poly() (*in module sage.graphs.matchpoly*), 1967
CompleteBipartiteGraph() (*in module sage.graphs.generators.basic*), 1392
CompleteBipartiteGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1020
CompleteGraph() (*in module sage.graphs.generators.basic*), 1395
CompleteGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1023
CompleteMultipartiteGraph() (*in module sage.graphs.generators.basic*), 1398
CompleteMultipartiteGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1025
CompleteUniform() (*sage.graphs.hypergraph_generators.HypergraphGenerators method*), 1822
compute_depth_of_self_and_children() (*sage.graphs.schnyder.TreeNode method*), 1988
compute_number_of_descendants()

(*sage.graphs.schnyder.TreeNode method*), 1988
connected_component_containing_vertex() (*in module sage.graphs.connectivity*), 2320
connected_component_containing_vertex() (*sage.graphs.generic_graph.GenericGraph method*), 96
connected_components() (*in module sage.graphs.connectivity*), 2321
connected_components() (*sage.graphs.generic_graph.GenericGraph method*), 97
connected_components_number() (*in module sage.graphs.connectivity*), 2322
connected_components_number() (*sage.graphs.generic_graph.GenericGraph method*), 98
connected_components_sizes() (*in module sage.graphs.connectivity*), 2322
connected_components_sizes() (*sage.graphs.generic_graph.GenericGraph method*), 98
connected_components_subgraphs() (*in module sage.graphs.connectivity*), 2323
connected_components_subgraphs() (*sage.graphs.generic_graph.GenericGraph method*), 99
connected_full_subgraphs() (*in module sage.graphs.base.static_dense_graph*), 1766
connected_subgraph_iterator() (*in module sage.graphs.base.static_dense_graph*), 1769
connected_subgraph_iterator() (*sage.graphs.generic_graph.GenericGraph method*), 100
connecting_nodes() (*in module sage.graphs.generators.random*), 1564
contract_edge() (*sage.graphs.generic_graph.GenericGraph method*), 103
contract_edges() (*sage.graphs.generic_graph.GenericGraph method*), 104
contracted_edge() (*in module sage.graphs.tutte_polynomial*), 2241
convexity_properties() (*sage.graphs.graph.Graph method*), 584
ConvexityProperties (*class in sage.graphs.convexity_properties*), 2162
ConwaySmith_for_3S7() (*in module sage.graphs.generators.distance_regular*), 1444
ConwaySmith_for_3S7() (*sage.graphs.graph_generators.GraphGenerators static method*), 1026
copy() (*sage.combinat.designs.incidence_structures.IncidenceStructure method*), 1833
copy() (*sage.graphs.generic_graph.GenericGraph method*), 106
copy_tree() (*sage.graphs.cographs.CoTree method*), 1893
cores() (*sage.graphs.graph.Graph method*), 585
corneil_habib_paul_tedder_algorithm() (*in module sage.graphs.graph_decompositions.modular_decomposition*), 2140
CorrelationGraph() (*in module sage.graphs.generators.basic*), 1398
CorrelationGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1027
cospectral_graphs() (*sage.graphs.graph_generators.GraphGenerators method*), 1240
CossidentePenttilaGraph() (*in module sage.graphs.generators.classical_geometries*), 1423
CossidentePenttilaGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1027
CoTree (*class in sage.graphs.cographs*), 1893
CoxeterGraph() (*in module sage.graphs.generators.smallgraphs*), 1580
CoxeterGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1028
create_leaf() (*sage.graphs.graph_decompositions.modular_decomposition.Node class method*), 2134
crossing_number() (*sage.graphs.generic_graph.GenericGraph method*), 108
CubeConnectedCycle() (*in module sage.graphs.generators.families*), 1475
CubeConnectedCycle() (*sage.graphs.graph_generators.GraphGenerators static method*), 1029
CubeGraph() (*in module sage.graphs.generators.families*), 1476
CubeGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1030
CubeplexGraph() (*in module sage.graphs.generators.smallgraphs*), 1580
CubeplexGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1032
current_allocation() (*sage.graphs.base.c_graph.CGraph method*), 1689
cutwidth() (*in module sage.graphs.graph_decompositions.cutwidth*), 2110
cutwidth() (*sage.graphs.graph.Graph method*), 588
cutwidth_dyn() (*in module sage.graphs.graph_decompositions.cutwidth*), 2114
cutwidth_MILP() (*in module sage.graphs.graph_decompositions.cutwidth*), 2112
cycle_basis() (*sage.graphs.generic_graph.GenericGraph method*), 109
CycleGraph() (*in module sage.graphs.generators.basic*), 1399

CycleGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1033

D

DartGraph() (*in module sage.graphs.generators.basic*), 1401
DartGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1035
data_to_degseq() (*in module sage.graphs.graph_database*), 1331
DeBruijn() (*sage.graphs.digraph_generators.DiGraphGenerators method*), 1289
define_Akq() (*in module sage.graphs.generators.luw_graphs*), 1384
define_Dkq() (*in module sage.graphs.generators.luw_graphs*), 1385
define_luw_graph() (*in module sage.graphs.generators.luw_graphs*), 1386
define_WengerGraph() (*in module sage.graphs.generators.luw_graphs*), 1385
degree() (*sage.combinat.designs.incidence_structures.IncidenceStructure method*), 1834
degree() (*sage.graphs.base.c_graph.CGraphBackend method*), 1714
degree() (*sage.graphs.base.graph_backends.GenericGraphBackend method*), 1796
degree() (*sage.graphs.base.static_sparse_backend.StaticSparseBackend method*), 1790
degree() (*sage.graphs.generic_graph.GenericGraph method*), 113
degree_constrained_subgraph() (*sage.graphs.graph.Graph method*), 589
degree_histogram() (*sage.graphs.generic_graph.GenericGraph method*), 115
degree_iterator() (*sage.graphs.generic_graph.GenericGraph method*), 115
degree_polynomial() (*sage.graphs.digraph.DiGraph method*), 775
degree_sequence (*sage.graphs.graph_generators.GraphGenerators attribute*), 1243
degree_sequence() (*sage.graphs.generic_graph.GenericGraph method*), 117
degree_to_cell() (*sage.graphs.generic_graph.GenericGraph method*), 118
degrees() (*sage.combinat.designs.incidence_structures.IncidenceStructure method*), 1834
DegreeSequence() (*in module sage.graphs.generators.degree_sequence*), 1438
DegreeSequence() (*sage.graphs.graph_generators.GraphGenerators static method*), 1036
DegreeSequenceBipartite() (*in module sage.graphs.generators.degree_sequence*), 1439
DegreeSequenceBipartite() (*sage.graphs.graph_generators.GraphGenerators static method*), 1037
DegreeSequenceConfigurationModel() (*in module sage.graphs.generators.degree_sequence*), 1440
DegreeSequenceConfigurationModel() (*sage.graphs.graph_generators.GraphGenerators static method*), 1038
DegreeSequenceExpected() (*in module sage.graphs.generators.degree_sequence*), 1441
DegreeSequenceExpected() (*sage.graphs.graph_generators.GraphGenerators static method*), 1039
DegreeSequenceTree() (*in module sage.graphs.generators.degree_sequence*), 1442
DegreeSequenceTree() (*sage.graphs.graph_generators.GraphGenerators static method*), 1040
degseq_to_data() (*in module sage.graphs.graph_database*), 1331
DejterGraph() (*in module sage.graphs.generators.smallgraphs*), 1582
DejterGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1040
del_all_arcs() (*sage.graphs.base.c_graph.CGraph method*), 1691
del_arc_label() (*sage.graphs.base.c_graph.CGraph method*), 1692
del_edge() (*sage.graphs.base.c_graph.CGraphBackend method*), 1715
del_edge() (*sage.graphs.base.graph_backends.GenericGraphBackend method*), 1796
del_edge() (*sage.graphs.base.static_sparse_backend.StaticSparseBackend method*), 1790
del_edges() (*sage.graphs.base.c_graph.CGraphBackend method*), 1716
del_edges() (*sage.graphs.base.static_sparse_backend.StaticSparseBackend method*), 1790
del_vertex() (*sage.graphs.base.c_graph.CGraph method*), 1693
del_vertex() (*sage.graphs.base.c_graph.CGraphBackend method*), 1717
del_vertex() (*sage.graphs.base.graph_backends.GenericGraphBackend method*), 1797
del_vertex() (*sage.graphs.base.static_sparse_backend.StaticSparseBackend method*), 1790
del_vertex() (*sage.graphs.base.static_sparse_backend.StaticSparseCGraph method*), 1794
del_vertices() (*sage.graphs.base.c_graph.CGraphBackend method*), 1718
del_vertices() (*sage.graphs.base.graph_backends.GenericGraphBackend method*), 1797
del_vertices() (*sage.graphs.base.static_sparse_backend.StaticSparseBackend method*), 1790
delete_edge() (*sage.graphs.generic_graph.GenericGraph method*), 119
delete_edges() (*sage.graphs.generic_graph.GenericGraph method*), 121

delete_multiedge() (*sage.graphs.generic_graph.GenericGraph method*), 122
delete_vertex() (*sage.graphs.bipartite_graph.BipartiteGraph method*), 847
delete_vertex() (*sage.graphs.generic_graph.GenericGraph method*), 123
delete_vertex() (*sage.graphs.matching_covered_graph.MatchingCoveredGraph method*), 899
delete_vertices() (*sage.graphs.bipartite_graph.BipartiteGraph method*), 849
delete_vertices() (*sage.graphs.generic_graph.GenericGraph method*), 124
delete_vertices() (*sage.graphs.matching_covered_graph.MatchingCoveredGraph method*), 900
DenseGraph (*class in sage.graphs.base.dense_graph*), 1758
DenseGraphBackend (*class in sage.graphs.base.dense_graph*), 1761
density() (*sage.graphs.generic_graph.GenericGraph method*), 125
depth_first_search() (*sage.graphs.base.c_graph.CGraphBackend method*), 1718
depth_first_search() (*sage.graphs.generic_graph.GenericGraph method*), 126
depth_first_traversal() (*in module sage.graphs.partial_cube*), 2247
DesarguesGraph() (*in module sage.graphs.generators.smallgraphs*), 1582
DesarguesGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1041
description() (*sage.graphs.isgci.GraphClass method*), 1672
diameter() (*in module sage.graphs.base.boost_graph*), 1805
diameter() (*in module sage.graphs.distances_all_pairs*), 2174
diameter() (*sage.graphs.digraph.DiGraph method*), 775
diameter() (*sage.graphs.graph.Graph method*), 590
diameter_DHV() (*in module sage.graphs.base.boost_graph*), 1806
DiamondGraph() (*in module sage.graphs.generators.basic*), 1402
DiamondGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1041
dig6_string() (*sage.graphs.digraph.DiGraph method*), 777
DiGraph (*class in sage.graphs.digraph*), 766
DiGraphGenerators (*class in sage.graphs.digraph_generators*), 1283
dimension() (*sage.graphs.generators.luw_graphs.LUWGraphDescriptor method*), 1378
DipoleGraph() (*in module sage.graphs.generators.families*), 1478
DipoleGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1042
disjoint_routed_paths() (*sage.graphs.generic_graph.GenericGraph method*), 128
disjoint_union() (*sage.graphs.generic_graph.GenericGraph method*), 129
disjunctive_product() (*sage.graphs.generic_graph.GenericGraph method*), 130
distance() (*sage.graphs.generic_graph.GenericGraph method*), 131
distance_3_doubly_truncated_Golay_code_graph() (*in module sage.graphs.generators.distance_regular*), 1455
distance_3_doubly_truncated_Golay_code_graph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1243
distance_all_pairs() (*sage.graphs.generic_graph.GenericGraph method*), 132
distance_graph() (*sage.graphs.graph.Graph method*), 591
distance_matrix() (*sage.graphs.generic_graph.GenericGraph method*), 134
distance_regular (*sage.graphs.graph_generators.GraphGenerators attribute*), 1244
distance_regular_graph() (*in module sage.graphs.generators.distance_regular*), 1455
distance_regular_graph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1244
distances_all_pairs() (*in module sage.graphs.distances_all_pairs*), 2177
distances_and_predecessors_all_pairs() (*in module sage.graphs.distances_all_pairs*), 2178
distances_distribution() (*in module sage.graphs.distances_all_pairs*), 2179
distances_distribution() (*sage.graphs.generic_graph.GenericGraph method*), 136
Dkq() (*in module sage.graphs.generators.luw_graphs*), 1374
Dkq() (*sage.graphs.graph_generators.GraphGenerators static method*), 1042
DodecahedralGraph() (*in module sage.graphs.generators.platonic_solids*), 1536
DodecahedralGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1043
dominating_set() (*in module sage.graphs.domination*), 2368
dominating_set() (*sage.graphs.generic_graph.Generic-*

Graph method), 138
dominating_sets() (*in module sage.graphs.domination*), 2370
dominating_sets() (*sage.graphs.generic_graph.GenericGraph method*), 140
dominator_tree() (*in module sage.graphs.base.boost_graph*), 1806
dominator_tree() (*sage.graphs.generic_graph.GenericGraph method*), 145
DorogovtsevGoltsevMendesGraph() (*in module sage.graphs.generators.families*), 1478
DorogovtsevGoltsevMendesGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1044
dot2tex_picture() (*sage.graphs.graph_latex.GraphLatex method*), 2197
DoubleGeneralizedPetersenGraph() (*in module sage.graphs.generators.families*), 1479
DoubleGeneralizedPetersenGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1044
DoubleGrassmannGraph() (*in module sage.graphs.generators.distance_regular*), 1445
DoubleGrassmannGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1045
DoubleOddGraph() (*in module sage.graphs.generators.distance_regular*), 1446
DoubleOddGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1046
DoubleStarSnark() (*in module sage.graphs.generators.smallgraphs*), 1583
DoubleStarSnark() (*sage.graphs.graph_generators.GraphGenerators static method*), 1047
DoublyTruncatedWittGraph() (*in module sage.graphs.generators.distance_regular*), 1446
DoublyTruncatedWittGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1047
dual() (*sage.combinat.designs.incidence_structures.IncidenceStructure method*), 1835
DurerGraph() (*in module sage.graphs.generators.smallgraphs*), 1584
DurerGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1048
DyckGraph() (*in module sage.graphs.generators.smallgraphs*), 1585
DyckGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1049

E

Ear (*class in sage.graphs.tutte_polynomial*), 2238
ear_decomposition() (*sage.graphs.graph.Graph method*), 594
eccentricity() (*in module sage.graphs.distances_all_pairs*), 2181
eccentricity() (*sage.graphs.digraph.DiGraph method*), 777
eccentricity() (*sage.graphs.graph.Graph method*), 596
eccentricity_DHV() (*in module sage.graphs.base.boost_graph*), 1808
edge_boundary() (*sage.graphs.generic_graph.GenericGraph method*), 146
edge_coloring() (*in module sage.graphs.graph_coloring*), 1864
edge_coloring() (*sage.combinat.designs.incidence_structures.IncidenceStructure method*), 1835
edge_connectivity() (*in module sage.graphs.base.boost_graph*), 1809
edge_connectivity() (*in module sage.graphs.connectivity*), 2324
edge_connectivity() (*sage.graphs.edge_connectivity.GabowEdgeConnectivity method*), 2362
edge_connectivity() (*sage.graphs.generic_graph.GenericGraph method*), 148
edge_cut() (*sage.graphs.generic_graph.GenericGraph method*), 154
edge_disjoint_paths() (*sage.graphs.generic_graph.GenericGraph method*), 156
edge_disjoint_spanning_trees() (*in module sage.graphs.spanning_tree*), 1920
edge_disjoint_spanning_trees() (*sage.graphs.edge_connectivity.GabowEdgeConnectivity method*), 2362
edge_disjoint_spanning_trees() (*sage.graphs.generic_graph.GenericGraph method*), 157
edge_isoperimetric_number() (*in module sage.graphs.isoperimetric_inequalities*), 1908
edge_isoperimetric_number() (*sage.graphs.graph.Graph method*), 599
edge_iterator() (*sage.graphs.generic_graph.GenericGraph method*), 162
edge_label() (*sage.graphs.generic_graph.GenericGraph method*), 163
edge_labels() (*sage.graphs.generic_graph.GenericGraph method*), 164
edge_multiplicities() (*in module sage.graphs.tutte_polynomial*), 2241
edge_polytope() (*sage.graphs.generic_graph.GenericGraph method*), 164
edges() (*sage.graphs.generic_graph.GenericGraph method*), 167
edges_incident() (*sage.graphs.generic_graph.Generic-*

Graph method), 171
EdgeSelection (*class in sage.graphs.tutte_polynomial*), 2240
EdgesView (*class in sage.graphs.views*), 952
effective_resistance() (*sage.graphs.graph.Graph method*), 600
effective_resistance_matrix() (*sage.graphs.graph.Graph method*), 602
EgawaGraph() (*in module sage.graphs.generators.families*), 1479
EgawaGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1051
eigenmatrix() (*in module sage.graphs.strongly_regular_db*), 1346
eigenspaces() (*sage.graphs.generic_graph.GenericGraph method*), 172
eigenvectors() (*sage.graphs.generic_graph.GenericGraph method*), 176
either_connected_or_not_connected() (*in module sage.graphs.graph_decompositions.modular_decomposition*), 2140
EllinghamHorton54Graph() (*in module sage.graphs.generators.smallgraphs*), 1587
EllinghamHorton54Graph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1052
EllinghamHorton78Graph() (*in module sage.graphs.generators.smallgraphs*), 1588
EllinghamHorton78Graph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1053
EMPTY (*sage.graphs.graph_decompositions.modular_decomposition.NodeType attribute*), 2136
EmptyGraph() (*in module sage.graphs.generators.basic*), 1402
EmptyGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1054
equations() (*sage.graphs.generators.luw_graphs.LUWGraphDescriptor method*), 1379
equivalent_trees() (*in module sage.graphs.graph_decompositions.modular_decomposition*), 2141
ErreraGraph() (*in module sage.graphs.generators.smallgraphs*), 1589
ErreraGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1055
eulerian_circuit() (*sage.graphs.generic_graph.GenericGraph method*), 179
eulerian_orientation() (*in module sage.graphs.orientations*), 2289
eulerian_orientation() (*sage.graphs.graph.Graph method*), 604
EuropeMap() (*in module sage.graphs.generators.world_map*), 1662
EuropeMap() (*sage.graphs.graph_generators.GraphGenerators static method*), 1056
export_to_file() (*sage.graphs.generic_graph.GenericGraph method*), 180

F

F26AGraph() (*in module sage.graphs.generators.smallgraphs*), 1590
F26AGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1057
faces() (*sage.graphs.generic_graph.GenericGraph method*), 182
families (*sage.graphs.graph_generators.GraphGenerators attribute*), 1245
FastDigraph (*class in sage.graphs.graph_decompositions.fast_digraph*), 2115
feedback_edge_set() (*sage.graphs.digraph.DiGraph method*), 780
feedback_vertex_set() (*sage.graphs.generic_graph.GenericGraph method*), 183
feng_k_shortest_simple_paths() (*in module sage.graphs.path_enumeration*), 2263
FibonacciTree() (*in module sage.graphs.generators.trees*), 1654
FibonacciTree() (*sage.graphs.graph_generators.GraphGenerators static method*), 1058
filter_kruskal() (*in module sage.graphs.spanning_tree*), 1922
filter_kruskal_iterator() (*in module sage.graphs.spanning_tree*), 1923
find_ear() (*sage.graphs.tutte_polynomial.Ear static method*), 2238
find_hamiltonian() (*in module sage.graphs.generic_graph_pyx*), 2278
find_pivot() (*in module sage.graphs.cographs*), 1896
first_coloring() (*in module sage.graphs.graph_coloring*), 1866
flatten() (*sage.graphs.pq_trees.PQ method*), 1939
flow() (*sage.graphs.generic_graph.GenericGraph method*), 186
flow_polytope() (*sage.graphs.digraph.DiGraph method*), 782
FlowerSnark() (*in module sage.graphs.generators.smallgraphs*), 1591
FlowerSnark() (*sage.graphs.graph_generators.GraphGenerators static method*), 1059
floyd_warshall() (*in module sage.graphs.distances_all_pairs*), 2182
floyd_warshall_shortest_paths() (*in module sage.graphs.base.boost_graph*), 1809
folded_graph() (*sage.graphs.graph.Graph method*), 605
FoldedCubeGraph() (*in module sage.graphs.generators.families*), 1480
FoldedCubeGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1060

FolkmanGraph() (*in module sage.graphs.generators.smallgraphs*), 1592
FolkmanGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1060
forbidden_subgraphs() (*sage.graphs.isgci.GraphClass method*), 1673
ForkGraph() (*in module sage.graphs.generators.basic*), 1403
ForkGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1061
form_module() (*in module sage.graphs.graph_decompositions.modular_decomposition*), 2142
format_coloring() (*in module sage.graphs.graph_coloring*), 1867
FosterGraph() (*in module sage.graphs.generators.smallgraphs*), 1593
FosterGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1062
FosterGraph3S6() (*in module sage.graphs.generators.distance_regular*), 1447
FosterGraph3S6() (*sage.graphs.graph_generators.GraphGenerators static method*), 1062
fractional_chromatic_index() (*in module sage.graphs.graph_coloring*), 1868
fractional_chromatic_index() (*sage.graphs.graph.Graph method*), 606
fractional_chromatic_number() (*in module sage.graphs.graph_coloring*), 1869
fractional_chromatic_number() (*sage.graphs.graph.Graph method*), 607
fractional_clique_number() (*sage.graphs.graph.Graph method*), 609
FranklinGraph() (*in module sage.graphs.generators.smallgraphs*), 1594
FranklinGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1063
FriendshipGraph() (*in module sage.graphs.generators.families*), 1481
FriendshipGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1064
from_adjacency_matrix() (*in module sage.graphs.graph_input*), 2220
from_dict_of_dicts() (*in module sage.graphs.graph_input*), 2220
from_dict_of_lists() (*in module sage.graphs.graph_input*), 2221
from_dig6() (*in module sage.graphs.graph_input*), 2222
from_graph6() (*in module sage.graphs.graph_input*), 2222
from_graph6() (*in module sage.graphs.graph_list*), 2212
from_incidence_matrix() (*in module sage.graphs.graph_input*), 2223
from_networkx_graph() (*in module sage.graphs.graph_input*), 2223
from_oriented_incidence_matrix() (*in module sage.graphs.graph_input*), 2228
from_seidel_adjacency_matrix() (*in module sage.graphs.graph_input*), 2229
from_sparse6() (*in module sage.graphs.graph_input*), 2229
from_sparse6() (*in module sage.graphs.graph_list*), 2213
from_whatever() (*in module sage.graphs.graph_list*), 2214
FruchtGraph() (*in module sage.graphs.generators.smallgraphs*), 1595
FruchtGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1067
fullerenes() (*sage.graphs.graph_generators.GraphGenerators method*), 1245
FurerGadget() (*in module sage.graphs.generators.families*), 1483
FurerGadget() (*sage.graphs.graph_generators.GraphGenerators static method*), 1067
fusenes() (*sage.graphs.graph_generators.GraphGenerators method*), 1247
FuzzyBallGraph() (*in module sage.graphs.generators.families*), 1485
FuzzyBallGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1069

G

GabowEdgeConnectivity (*class in sage.graphs.edge_connectivity*), 2360
gamma_classes() (*in module sage.graphs.graph_decompositions.modular_decomposition*), 2142
GemGraph() (*in module sage.graphs.generators.basic*), 1404
GemGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1070
gen_html_code() (*in module sage.graphs.graph_plot_js*), 2062
gen_trees (*sage.graphs.graph_generators.GraphGenerators attribute*), 1248
GeneralisedDodecagonGraph() (*in module sage.graphs.generators.distance_regular*), 1447
GeneralisedDodecagonGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1071
GeneralisedHexagonGraph() (*in module sage.graphs.generators.distance_regular*), 1448
GeneralisedHexagonGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1071
GeneralisedOctagonGraph() (*in module sage.graphs.generators.distance_regular*), 1449
GeneralisedOctagonGraph() (*sage.graphs.graph_generators.GraphGener-*

ators static method), 1072
GeneralizedDeBruijn() (*sage.graphs.digraph_generators.DiGraphGenerators method*), 1291
GeneralizedPetersenGraph() (*in module sage.graphs.generators.families*), 1486
GeneralizedPetersenGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1073
GeneralizedSierpinskiGraph() (*in module sage.graphs.generators.families*), 1487
GeneralizedSierpinskiGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1074
GenericGraph (*class in sage.graphs.generic_graph*), 8
GenericGraph_pyx (*class in sage.graphs.generic_graph_pyx*), 2274
GenericGraphBackend (*class in sage.graphs.base.graph_backends*), 1795
GenericGraphQuery (*class in sage.graphs.graph_database*), 1308
genus() (*sage.graphs.generic_graph.GenericGraph method*), 188
genus() (*sage.graphs.genus.simple_connected_genus_backtracker method*), 1982
geodetic_closure() (*in module sage.graphs.convexity_properties*), 2165
geodetic_closure() (*sage.graphs.graph.Graph method*), 609
get_class() (*sage.graphs.isgci.GraphClasses method*), 1674
get_edge_label() (*sage.graphs.base.dense_graph.DenseGraphBackend method*), 1763
get_edge_label() (*sage.graphs.base.graph_backends.GenericGraphBackend method*), 1797
get_edge_label() (*sage.graphs.base.sparse_graph.SparseGraphBackend method*), 1753
get_edge_label() (*sage.graphs.base.static_sparse_backend.StaticSparseBackend method*), 1790
get_embedding() (*sage.graphs.generic_graph.GenericGraph method*), 192
get_embedding() (*sage.graphs.genus.simple_connected_genus_backtracker method*), 1983
get_graphs_list() (*sage.graphs.graph_database.GraphQuery method*), 1324
get_length() (*sage.graphs.graph_decompositions.tree_decomposition.TreelengthConnected method*), 2067
get_matching() (*sage.graphs.matching_covered_graph.MatchingCoveredGraph method*), 903
get_module_type() (*in module sage.graphs.graph_decompositions.modular_decomposition*), 2143
get_option() (*sage.graphs.graph_latex.GraphLatex method*), 2199
get_pos() (*sage.graphs.generic_graph.GenericGraph method*), 193
get_spqr_tree() (*sage.graphs.connectivity.TriconnectivitySPQR method*), 2304
get_tree_decomposition() (*sage.graphs.graph_decompositions.tree_decomposition.TreelengthConnected method*), 2067
get_triconnected_components() (*sage.graphs.connectivity.TriconnectivitySPQR method*), 2307
get_vertex() (*sage.graphs.generic_graph.GenericGraph method*), 195
get_vertices() (*in module sage.graphs.graph_decompositions.modular_decomposition*), 2143
get_vertices() (*sage.graphs.generic_graph.GenericGraph method*), 196
girth() (*sage.graphs.generic_graph.GenericGraph method*), 196
GoethalsSeidelGraph() (*in module sage.graphs.generators.families*), 1490
GoethalsSeidelGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1077
GoldnerHararyGraph() (*in module sage.graphs.generators.smallgraphs*), 1596
GoldnerHararyGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1078
GolombGraph() (*in module sage.graphs.generators.smallgraphs*), 1597
GolombGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1079
gomory_hu_tree() (*sage.graphs.graph.Graph method*), 611
GossetGraph() (*in module sage.graphs.generators.smallgraphs*), 1598
GossetGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1080
Graph (*class in sage.graphs.graph*), 515
graph() (*sage.graphs.generators.luw_graphs.LUWGraphDescriptor method*), 1379
graph6_string() (*sage.graphs.graph.Graph method*), 614
graph6_to_plot() (*in module sage.graphs.graph_database*), 1332
graph_3O73() (*in module sage.graphs.generators.distance_regular*), 1456
graph_3O73() (*sage.graphs.graph_generators.GraphGenerators static method*), 1248
graph_db_info() (*in module sage.graphs.graph_database*), 1332
graph_editor() (*in module sage.graphs.graph_editor*), 2211
graph_from_GQ_spread() (*in module sage.graphs.generators.distance_regular*), 1457
graph_isom_equivalent_non_edge_labeled_graph() (*in module*

sage.graphs.generic_graph), 494
graph_with_classical_parameters() (*in module sage.graphs.generators.distance_regular*), 1457
GraphClass (*class in sage.graphs.isgci*), 1672
GraphClasses (*class in sage.graphs.isgci*), 1674
GraphDatabase (*class in sage.graphs.graph_database*), 1309
GraphGenerators (*class in sage.graphs.graph_generators*), 975
graphics_array_defaults (*sage.graphs.generic_graph.GenericGraph attribute*), 197
GraphLatex (*class in sage.graphs.graph_latex*), 2197
GraphPlot (*class in sage.graphs.graph_plot*), 2018
graphplot() (*sage.graphs.generic_graph.GenericGraph method*), 197
GraphQuery (*class in sage.graphs.graph_database*), 1322
graphs (*in module sage.graphs.graph_generators*), 963
graphviz_string() (*sage.graphs.generic_graph.GenericGraph method*), 198
graphviz_to_file_named() (*sage.graphs.generic_graph.GenericGraph method*), 210
GrassmannGraph() (*in module sage.graphs.generators.distance_regular*), 1450
GrassmannGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1080
GrayGraph() (*in module sage.graphs.generators.smallgraphs*), 1598
GrayGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1081
greedy_dominating_set() (*in module sage.graphs.domination*), 2374
greedy_dominating_set() (*sage.graphs.generic_graph.GenericGraph method*), 211
greedy_is_comparability() (*in module sage.graphs.comparability*), 1900
greedy_is_comparability_with_certificate() (*in module sage.graphs.comparability*), 1901
Grid2dGraph() (*in module sage.graphs.generators.basic*), 1404
Grid2dGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1082
GridGraph() (*in module sage.graphs.generators.basic*), 1405
GridGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1082
GritsenkoGraph() (*in module sage.graphs.generators.smallgraphs*), 1599
GritsenkoGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1083
GrotzschGraph() (*in module sage.graphs.generators.smallgraphs*), 1599
GrotzschGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1083
ground_set() (*sage.combinat.designs.incidence_structures.IncidenceStructure method*), 1836
growing_subtrees() (*in module sage.graphs.generators.random*), 1565
grundy_coloring() (*in module sage.graphs.graph_coloring*), 1870

H

habib_maurer_algorithm() (*in module sage.graphs.graph_decompositions.modular_decomposition*), 2144
HaemersGraph() (*in module sage.graphs.generators.classical_geometries*), 1424
HaemersGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1085
HalfCube() (*in module sage.graphs.generators.distance_regular*), 1450
HalfCube() (*sage.graphs.graph_generators.GraphGenerators static method*), 1086
HallJankoGraph() (*in module sage.graphs.generators.smallgraphs*), 1601
HallJankoGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1086
hamiltonian_cycle() (*sage.graphs.generic_graph.GenericGraph method*), 214
hamiltonian_path() (*sage.graphs.generic_graph.GenericGraph method*), 216
HammingGraph() (*in module sage.graphs.generators.families*), 1491
HammingGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1088
HanoiTowerGraph() (*in module sage.graphs.generators.families*), 1492
HanoiTowerGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1089
HararyGraph() (*in module sage.graphs.generators.families*), 1495
HararyGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1092
HarborthGraph() (*in module sage.graphs.generators.smallgraphs*), 1602
HarborthGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1092
HarriesGraph() (*in module sage.graphs.generators.smallgraphs*), 1602
HarriesGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1093
HarriesWongGraph() (*in module sage.graphs.generators.smallgraphs*), 1603
HarriesWongGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1094

has_arc() (*sage.graphs.base.c_graph.CGraph method*), 1696
has_arc() (*sage.graphs.base.static_sparse_backend.StaticSparseCGraph method*), 1794
has_arc_label() (*sage.graphs.base.c_graph.CGraph method*), 1697
has_edge() (*sage.graphs.base.dense_graph.DenseGraphBackend method*), 1764
has_edge() (*sage.graphs.base.graph_backends.GenericGraphBackend method*), 1797
has_edge() (*sage.graphs.base.sparse_graph.SparseGraphBackend method*), 1753
has_edge() (*sage.graphs.base.static_sparse_backend.StaticSparseBackend method*), 1791
has_edge() (*sage.graphs.generic_graph.GenericGraph method*), 218
has_homomorphism_to() (*in module sage.graphs.morphisms*), 1972
has_homomorphism_to() (*sage.graphs.generic_graph.GenericGraph method*), 218
has_loops() (*sage.graphs.generic_graph.GenericGraph method*), 221
has_loops() (*sage.graphs.matching_covered_graph.MatchingCoveredGraph method*), 904
has_multiple_edges() (*sage.graphs.generic_graph.GenericGraph method*), 222
has_perfect_matching() (*in module sage.graphs.matching*), 1949
has_perfect_matching() (*sage.graphs.graph.Graph method*), 614
has_perfect_matching() (*sage.graphs.matching_covered_graph.MatchingCoveredGraph method*), 906
has_vertex() (*sage.graphs.base.c_graph.CGraph method*), 1698
has_vertex() (*sage.graphs.base.c_graph.CGraphBackend method*), 1720
has_vertex() (*sage.graphs.base.graph_backends.GenericGraphBackend method*), 1797
has_vertex() (*sage.graphs.base.static_sparse_backend.StaticSparseBackend method*), 1791
has_vertex() (*sage.graphs.base.static_sparse_backend.StaticSparseCGraph method*), 1794
has_vertex() (*sage.graphs.generic_graph.GenericGraph method*), 225
have_tkz_graph() (*in module sage.graphs.graph_latex*), 2210
HeawoodGraph() (*in module sage.graphs.generators.smallgraphs*), 1605
HeawoodGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1095
HermitianFormsGraph() (*in module sage.graphs.generators.distance_regular*), 1451
HermitianFormsGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1096
HerschelGraph() (*in module sage.graphs.generators.smallgraphs*), 1605
HerschelGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1096
HexahedralGraph() (*in module sage.graphs.generators.platonic_solids*), 1537
HexahedralGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1098
HigmanSimsGraph() (*in module sage.graphs.generators.smallgraphs*), 1607
HigmanSimsGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1099
HoffmanGraph() (*in module sage.graphs.generators.smallgraphs*), 1608
HoffmanGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1100
HoffmanSingletonGraph() (*in module sage.graphs.generators.smallgraphs*), 1609
HoffmanSingletonGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1101
HoltGraph() (*in module sage.graphs.generators.smallgraphs*), 1610
HoltGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1102
HortonGraph() (*in module sage.graphs.generators.smallgraphs*), 1611
HortonGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1103
HouseGraph() (*in module sage.graphs.generators.basic*), 1405
HouseGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1104
HouseXGraph() (*in module sage.graphs.generators.basic*), 1406
HouseXGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1104
hull() (*sage.graphs.convexity_properties.ConvexityProperties method*), 2163
hull_number() (*sage.graphs.convexity_properties.ConvexityProperties method*), 2164
hyperbolicity() (*in module sage.graphs.hyperbolicity*), 2232
hyperbolicity() (*sage.graphs.graph.Graph method*), 616
hyperbolicity_distribution() (*in module sage.graphs.hyperbolicity*), 2236
HypergraphGenerators (*class in sage.graphs.hypergraph_generators*), 1821
HyperStarGraph() (*in module sage.graphs.generators.families*), 1495

HyperStarGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1105

I

IcosahedralGraph() (*in module sage.graphs.generators.platonic_solids*), 1538
IcosahedralGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1106
IGraph() (*in module sage.graphs.generators.families*), 1496
IGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1106
igraph_graph() (*sage.graphs.generic_graph.GenericGraph method*), 225
ihara_zeta_function_inverse() (*sage.graphs.graph.Graph method*), 620
ImaseItoh() (*sage.graphs.digraph_generators.DiGraphGenerators method*), 1291
in_branchings() (*sage.graphs.digraph.DiGraph method*), 787
in_degree() (*sage.graphs.base.c_graph.CGraphBackend method*), 1721
in_degree() (*sage.graphs.base.dense_graph.DenseGraph method*), 1759
in_degree() (*sage.graphs.base.graph_backends.GenericGraphBackend method*), 1797
in_degree() (*sage.graphs.base.sparse_graph.SparseGraph method*), 1749
in_degree() (*sage.graphs.base.static_sparse_backend.StaticSparseBackend method*), 1791
in_degree() (*sage.graphs.base.static_sparse_backend.StaticSparseCGraph method*), 1794
in_degree() (*sage.graphs.digraph.DiGraph method*), 789
in_degree_iterator() (*sage.graphs.digraph.DiGraph method*), 790
in_degree_sequence() (*sage.graphs.digraph.DiGraph method*), 790
in_neighbors() (*sage.graphs.base.c_graph.CGraph method*), 1699
in_neighbors() (*sage.graphs.base.static_sparse_backend.StaticSparseCGraph method*), 1794
incidence_graph() (*sage.combinat.designs.incidence_structures.IncidenceStructure method*), 1836
incidence_matrix() (*sage.combinat.designs.incidence_structures.IncidenceStructure method*), 1837
incidence_matrix() (*sage.graphs.generic_graph.GenericGraph method*), 228
IncidenceStructure (*class in sage.combinat.designs.incidence_structures*), 1826
inclusion_digraph() (*sage.graphs.isgci.GraphClasses method*), 1675
inclusions() (*sage.graphs.isgci.GraphClasses method*), 1675
incoming_edge_iterator() (*sage.graphs.digraph.DiGraph method*), 791
incoming_edges() (*sage.graphs.digraph.DiGraph method*), 791
independent_set() (*sage.graphs.graph.Graph method*), 621
independent_set_of_representatives() (*sage.graphs.graph.Graph method*), 623
IndependentSets (*class in sage.graphs.independent_sets*), 1886
induced_substructure() (*sage.combinat.designs.incidence_structures.IncidenceStructure method*), 1838
int_to_binary_string() (*in module sage.graphs.generic_graph_pyx*), 2281
interactive_query() (*sage.graphs.graph_database.GraphDatabase method*), 1316
intersection (*sage.graphs.graph_generators.GraphGenerators attribute*), 1249
intersection_graph() (*sage.combinat.designs.incidence_structures.IncidenceStructure method*), 1839
IntersectionGraph() (*in module sage.graphs.generators.intersection*), 1526
IntersectionGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1107
IntervalGraph() (*in module sage.graphs.generators.intersection*), 1526
IntervalGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1108
IoninKharaghani765Graph() (*in module sage.graphs.generators.smallgraphs*), 1612
IoninKharaghani765Graph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1110
is_affine_polar() (*in module sage.graphs.strongly_regular_db*), 1353
is_antipodal() (*sage.graphs.graph.Graph method*), 625
is_aperiodic() (*sage.graphs.digraph.DiGraph method*), 792
is_apex() (*sage.graphs.graph.Graph method*), 625
is_arc_transitive() (*sage.graphs.graph.Graph method*), 626
is_asteroidal_triple_free() (*in module sage.graphs.asteroidal_triples*), 1884
is_asteroidal_triple_free() (*sage.graphs.graph.Graph method*), 627
is_berge_cyclic() (*sage.combinat.designs.incidence_structures.IncidenceStructure method*), 1839

is_biconnected() (*in module sage.graphs.connectivity*), 2330
is_biconnected() (*sage.graphs.generic_graph.GenericGraph method*), 235
is_biconnected() (*sage.graphs.matching_covered_graph.MatchingCoveredGraph method*), 908
is_bicritical() (*in module sage.graphs.matching*), 1950
is_bicritical() (*sage.graphs.graph.Graph method*), 628
is_bipartite() (*sage.graphs.bipartite_graph.BipartiteGraph method*), 849
is_bipartite() (*sage.graphs.generic_graph.GenericGraph method*), 236
is_block_graph() (*sage.graphs.graph.Graph method*), 633
is_brace() (*sage.graphs.matching_covered_graph.MatchingCoveredGraph method*), 909
is_brick() (*sage.graphs.matching_covered_graph.MatchingCoveredGraph method*), 919
is_cactus() (*sage.graphs.graph.Graph method*), 634
is_cartesian_product() (*in module sage.graphs.graph_decompositions.graph_products*), 2118
is_cartesian_product() (*sage.graphs.graph.Graph method*), 635
is_cayley() (*sage.graphs.generic_graph.GenericGraph method*), 237
is_chordal() (*sage.graphs.generic_graph.GenericGraph method*), 239
is_chordal_bipartite() (*sage.graphs.graph.Graph method*), 637
is_circulant() (*sage.graphs.generic_graph.GenericGraph method*), 242
is_circular_planar() (*sage.graphs.generic_graph.GenericGraph method*), 243
is_circumscribable() (*sage.graphs.graph.Graph method*), 639
is_classical_parameters_graph() (*in module sage.graphs.generators.distance_regular*), 1459
is_clique() (*sage.graphs.generic_graph.GenericGraph method*), 246
is_cograph() (*sage.graphs.graph.Graph method*), 641
is_comparability() (*in module sage.graphs.comparability*), 1902
is_comparability() (*sage.graphs.graph.Graph method*), 641
is_comparability_MILP() (*in module sage.graphs.comparability*), 1903
is_complete_multipartite() (*in module sage.graphs.strongly_regular_db*), 1354
is_connected() (*in module sage.graphs.connectivity*), 2331
is_connected() (*sage.combinat.designs.incidence_structures.IncidenceStructure method*), 1840
is_connected() (*sage.graphs.base.c_graph.CGraphBackend method*), 1721
is_connected() (*sage.graphs.generic_graph.GenericGraph method*), 247
is_cossidente_penttila() (*in module sage.graphs.strongly_regular_db*), 1355
is_cut_edge() (*in module sage.graphs.connectivity*), 2332
is_cut_edge() (*sage.graphs.generic_graph.GenericGraph method*), 248
is_cut_vertex() (*in module sage.graphs.connectivity*), 2334
is_cut_vertex() (*sage.graphs.generic_graph.GenericGraph method*), 250
is_cycle() (*sage.graphs.generic_graph.GenericGraph method*), 251
is_directed() (*sage.graphs.base.sparse_graph.SparseGraph method*), 1749
is_directed() (*sage.graphs.digraph.DiGraph method*), 792
is_directed() (*sage.graphs.graph.Graph method*), 642
is_directed_acyclic() (*sage.graphs.base.c_graph.CGraphBackend method*), 1722
is_directed_acyclic() (*sage.graphs.digraph.DiGraph method*), 793
is_distance_regular() (*in module sage.graphs.distances_all_pairs*), 2184
is_distance_regular() (*sage.graphs.graph.Graph method*), 642
is_dominating() (*in module sage.graphs.domination*), 2377
is_dominating() (*sage.graphs.graph.Graph method*), 643
is_drawn_free_of_edge_crossings() (*sage.graphs.generic_graph.GenericGraph method*), 253
is_edge_cut() (*in module sage.graphs.connectivity*), 2335
is_edge_cut() (*sage.graphs.generic_graph.GenericGraph method*), 253
is_edge_transitive() (*sage.graphs.graph.Graph method*), 644
is_empty() (*sage.graphs.graph_decompositions.modular_decomposition.Node method*), 2134
is_equitable() (*sage.graphs.generic_graph.GenericGraph method*), 256
is_eulerian() (*sage.graphs.generic_graph.Generic-*

Graph method), 258
is_even_hole_free() (*sage.graphs.graph.Graph method*), 645
is_factor_critical() (*in module sage.graphs.matching*), 1955
is_factor_critical() (*sage.graphs.graph.Graph method*), 646
is_forest() (*sage.graphs.graph.Graph method*), 649
is_from_GQ_spread() (*in module sage.graphs.generators.distance_regular*), 1460
is_gallai_tree() (*sage.graphs.generic_graph.GenericGraph method*), 259
is_generalized_quadrangle() (*sage.combinat.designs.incidence_structures.IncidenceStructure method*), 1840
is_geodetic() (*in module sage.graphs.convexity_properties*), 2166
is_geodetic() (*sage.graphs.generic_graph.GenericGraph method*), 260
is_goethals_seidel() (*in module sage.graphs.strongly_regular_db*), 1355
is_GQqmqp() (*in module sage.graphs.strongly_regular_db*), 1348
is_haemers() (*in module sage.graphs.strongly_regular_db*), 1357
is_half_transitive() (*sage.graphs.graph.Graph method*), 649
is_hamiltonian() (*sage.graphs.generic_graph.GenericGraph method*), 262
is_homeomorphic() (*in module sage.graphs.morphisms*), 1975
is_homeomorphic() (*sage.graphs.generic_graph.GenericGraph method*), 263
is_immutable() (*sage.graphs.generic_graph.GenericGraph method*), 265
is_independent_set() (*sage.graphs.generic_graph.GenericGraph method*), 266
is_inscribable() (*sage.graphs.graph.Graph method*), 650
is_interval() (*sage.graphs.generic_graph.GenericGraph method*), 266
is_isomorphic() (*sage.combinat.designs.incidence_structures.IncidenceStructure method*), 1841
is_isomorphic() (*sage.graphs.generic_graph.GenericGraph method*), 267
is_johnson() (*in module sage.graphs.strongly_regular_db*), 1357
is_leaf() (*sage.graphs.graph_decompositions.modular_decomposition.Node method*), 2135
is_less_than_k() (*sage.graphs.graph_decompositions.tree_decomposition.TreelengthConnected method*), 2068
is_line_graph() (*in module sage.graphs.line_graph*), 1912
is_line_graph() (*sage.graphs.graph.Graph method*), 652
is_long_antihole_free() (*in module sage.graphs.weakly_chordal*), 2169
is_long_antihole_free() (*sage.graphs.graph.Graph method*), 654
is_long_hole_free() (*in module sage.graphs.weakly_chordal*), 2170
is_long_hole_free() (*sage.graphs.graph.Graph method*), 655
is_matching_covered() (*in module sage.graphs.matching*), 1957
is_matching_covered() (*sage.graphs.graph.Graph method*), 656
is_mathon_PC_srg() (*in module sage.graphs.strongly_regular_db*), 1358
is_module() (*sage.graphs.graph.Graph method*), 661
is_muzychuk_S6() (*in module sage.graphs.strongly_regular_db*), 1359
is_near_polygon() (*in module sage.graphs.generators.distance_regular*), 1460
is_NO_F2() (*in module sage.graphs.strongly_regular_db*), 1349
is_NO_F3() (*in module sage.graphs.strongly_regular_db*), 1350
is_NOodd() (*in module sage.graphs.strongly_regular_db*), 1351
is_NOperp_F5() (*in module sage.graphs.strongly_regular_db*), 1351
is_nowhere0_twoweight() (*in module sage.graphs.strongly_regular_db*), 1359
is_NU() (*in module sage.graphs.strongly_regular_db*), 1352
is_odd_hole_free() (*sage.graphs.graph.Graph method*), 663
is_orthogonal_array_block_graph() (*in module sage.graphs.strongly_regular_db*), 1360
is_orthogonal_polar() (*in module sage.graphs.strongly_regular_db*), 1361
is_overfull() (*sage.graphs.graph.Graph method*), 664
is_paley() (*in module sage.graphs.strongly_regular_db*), 1362
is_partial_cube() (*in module sage.graphs.partial_cube*), 2248
is_partial_cube() (*sage.graphs.graph.Graph method*), 667
is_path() (*sage.graphs.graph.Graph method*), 668
is_perfect() (*sage.graphs.graph.Graph method*), 668
is_permutation() (*in module sage.graphs.comparability*), 1904
is_permutation() (*sage.graphs.graph.Graph method*), 670

is_planar() (*in module sage.graphs.planarity*), 1991
is_planar() (*sage.graphs.generic_graph.GenericGraph method*), 271
is_polhill() (*in module sage.graphs.strongly_regular_db*), 1362
is_polyhedral() (*sage.graphs.graph.Graph method*), 671
is_prime() (*sage.graphs.graph_decompositions.modular_decomposition.Node method*), 2135
is_prime() (*sage.graphs.graph.Graph method*), 672
is_projective_planar() (*sage.graphs.graph.Graph method*), 673
is_pseudo_partition_graph() (*in module sage.graphs.generators.distance_regular*), 1461
is_redundant() (*in module sage.graphs.domination*), 2378
is_redundant() (*sage.graphs.graph.Graph method*), 674
is_regular() (*sage.combinat.designs.incidence_structures.IncidenceStructure method*), 1841
is_regular() (*sage.graphs.generic_graph.GenericGraph method*), 276
is_resolvable() (*sage.combinat.designs.incidence_structures.IncidenceStructure method*), 1842
is_RSHCD() (*in module sage.graphs.strongly_regular_db*), 1353
is_self_complementary() (*sage.graphs.generic_graph.GenericGraph method*), 277
is_semi_symmetric() (*sage.graphs.graph.Graph method*), 674
is_series() (*sage.graphs.graph_decompositions.modular_decomposition.Node method*), 2135
is_simple() (*sage.combinat.designs.incidence_structures.IncidenceStructure method*), 1844
is_split() (*sage.graphs.graph.Graph method*), 675
is_spread() (*sage.combinat.designs.incidence_structures.IncidenceStructure method*), 1845
is_steiner() (*in module sage.graphs.strongly_regular_db*), 1363
is_strongly_connected() (*in module sage.graphs.connectivity*), 2338
is_strongly_connected() (*sage.graphs.base.c_graph.CGraphBackend method*), 1724
is_strongly_connected() (*sage.graphs.digraph.DiGraph method*), 795
is_strongly_regular() (*in module sage.graphs.base.static_dense_graph*), 1772
is_strongly_regular() (*sage.graphs.graph.Graph method*), 677
is_subgraph() (*sage.graphs.base.c_graph.CGraphBackend method*), 1725
is_subgraph() (*sage.graphs.generic_graph.GenericGraph method*), 278
is_switch_OA_srg() (*in module sage.graphs.strongly_regular_db*), 1364
is_switch_skewhad() (*in module sage.graphs.strongly_regular_db*), 1364
is_t_design() (*sage.combinat.designs.incidence_structures.IncidenceStructure method*), 1846
is_taylor_twograph_srg() (*in module sage.graphs.strongly_regular_db*), 1365
is_tournament() (*sage.graphs.digraph.DiGraph method*), 795
is_transitive() (*in module sage.graphs.comparability*), 1905
is_transitive() (*sage.graphs.digraph.DiGraph method*), 796
is_transitively_reduced() (*sage.graphs.generic_graph.GenericGraph method*), 280
is_tree() (*sage.graphs.graph.Graph method*), 679
is_triangle_free() (*in module sage.graphs.base.static_dense_graph*), 1775
is_triangle_free() (*sage.graphs.graph.Graph method*), 681
is_triconnected() (*in module sage.graphs.connectivity*), 2339
is_triconnected() (*sage.graphs.graph.Graph method*), 682
is_twograph_descendant_of_srg() (*in module sage.graphs.strongly_regular_db*), 1365
is_uniform() (*sage.combinat.designs.incidence_structures.IncidenceStructure method*), 1849
is_unitary_dual_polar() (*in module sage.graphs.strongly_regular_db*), 1366
is_unitary_polar() (*in module sage.graphs.strongly_regular_db*), 1367
is_valid_lex_M_order() (*in module sage.graphs.traversals*), 1993
is_valid_ordering() (*in module sage.graphs.graph_decompositions.vertex_separation*), 2086
is_valid_tree_decomposition() (*in module sage.graphs.graph_decompositions.tree_decomposition*), 2068
is_vertex_cut() (*in module sage.graphs.connectivity*), 2340
is_vertex_cut() (*sage.graphs.generic_graph.GenericGraph method*), 280
is_vertex_transitive() (*sage.graphs.generic_graph.GenericGraph method*), 282
is_weakly_chordal() (*in module sage.graphs.weakly_chordal*), 2171
is_weakly_chordal() (*sage.graphs.graph.Graph*

method), 684
isomorphic_substructures_iterator() (*sage.combinat.designs.incidence_structures.IncidenceStructure method*), 1850
iterator_edges() (*sage.graphs.base.c_graph.CGraphBackend method*), 1726
iterator_edges() (*sage.graphs.base.graph_backends.GenericGraphBackend method*), 1797
iterator_edges() (*sage.graphs.base.static_sparse_backend.StaticSparseBackend method*), 1791
iterator_in_edges() (*sage.graphs.base.c_graph.CGraphBackend method*), 1727
iterator_in_edges() (*sage.graphs.base.graph_backends.GenericGraphBackend method*), 1798
iterator_in_edges() (*sage.graphs.base.static_sparse_backend.StaticSparseBackend method*), 1791
iterator_in_nbrs() (*sage.graphs.base.c_graph.CGraphBackend method*), 1727
iterator_in_nbrs() (*sage.graphs.base.graph_backends.GenericGraphBackend method*), 1798
iterator_in_nbrs() (*sage.graphs.base.static_sparse_backend.StaticSparseBackend method*), 1791
iterator_nbrs() (*sage.graphs.base.c_graph.CGraphBackend method*), 1728
iterator_nbrs() (*sage.graphs.base.graph_backends.GenericGraphBackend method*), 1798
iterator_nbrs() (*sage.graphs.base.static_sparse_backend.StaticSparseBackend method*), 1792
iterator_out_edges() (*sage.graphs.base.c_graph.CGraphBackend method*), 1729
iterator_out_edges() (*sage.graphs.base.graph_backends.GenericGraphBackend method*), 1798
iterator_out_edges() (*sage.graphs.base.static_sparse_backend.StaticSparseBackend method*), 1792
iterator_out_nbrs() (*sage.graphs.base.c_graph.CGraphBackend method*), 1729
iterator_out_nbrs() (*sage.graphs.base.graph_backends.GenericGraphBackend method*), 1799
iterator_out_nbrs() (*sage.graphs.base.static_sparse_backend.StaticSparseBackend method*), 1792
iterator_unsorted_edges() (*sage.graphs.base.c_graph.CGraphBackend method*), 1730
iterator_unsorted_edges() (*sage.graphs.base.static_sparse_backend.StaticSparseBackend method*), 1793
iterator_verts() (*sage.graphs.base.c_graph.CGraphBackend method*), 1730
iterator_verts() (*sage.graphs.base.graph_backends.GenericGraphBackend method*), 1799
iterator_verts() (*sage.graphs.base.static_sparse_backend.StaticSparseBackend method*), 1793
IvanovIvanovFaradjevGraph() (*in module sage.graphs.generators.distance_regular*), 1451
IvanovIvanovFaradjevGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1111

J

J2Graph() (*in module sage.graphs.generators.distance_regular*), 1452
J2Graph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1112
JankoKharaghaniGraph() (*in module sage.graphs.generators.smallgraphs*), 1613
JankoKharaghaniGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1112
JankoKharaghaniTonchevGraph() (*in module sage.graphs.generators.smallgraphs*), 1614
JankoKharaghaniTonchevGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1113
johnson_closeness_centrality() (*in module sage.graphs.base.boost_graph*), 1811
johnson_shortest_paths() (*in module sage.graphs.base.boost_graph*), 1812
JohnsonGraph() (*in module sage.graphs.generators.families*), 1497
JohnsonGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1113
join() (*sage.graphs.graph.Graph method*), 684

K

katz_centrality() (*sage.graphs.generic_graph.GenericGraph method*), 283
katz_matrix() (*sage.graphs.generic_graph.GenericGraph method*), 284
Kautz() (*sage.graphs.digraph_generators.DiGraphGenerators method*), 1292
KingGraph() (*in module sage.graphs.generators.chessboard*), 1416
KingGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1114
kirchhoff_matrix() (*sage.graphs.generic_graph.GenericGraph method*), 286
kirchhoff_symanzik_polynomial() (*sage.graphs.graph.Graph method*), 686
KittellGraph() (*in module sage.graphs.generators.smallgraphs*), 1614
KittellGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1115

Klein3RegularGraph() (*in module sage.graphs.generators.smallgraphs*), 1615
Klein3RegularGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1116
Klein7RegularGraph() (*in module sage.graphs.generators.smallgraphs*), 1616
Klein7RegularGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1117
KneserGraph() (*in module sage.graphs.generators.families*), 1498
KneserGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1117
KnightGraph() (*in module sage.graphs.generators.chessboard*), 1417
KnightGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1118
KrackhardtKiteGraph() (*in module sage.graphs.generators.smallgraphs*), 1616
KrackhardtKiteGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1119
kronecker_product() (*sage.graphs.generic_graph.GenericGraph method*), 291
kruskal() (*in module sage.graphs.spanning_tree*), 1925
kruskal_iterator() (*in module sage.graphs.spanning_tree*), 1929
kruskal_iterator_from_edges() (*in module sage.graphs.spanning_tree*), 1930

L

label_nice_tree_decomposition() (*in module sage.graphs.graph_decompositions.tree_decomposition*), 2070
LadderGraph() (*in module sage.graphs.generators.basic*), 1406
LadderGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1122
laplacian_matrix() (*sage.graphs.generic_graph.GenericGraph method*), 291
LargeWittGraph() (*in module sage.graphs.generators.distance_regular*), 1452
LargeWittGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1123
latex() (*sage.graphs.graph_latex.GraphLatex method*), 2200
latex_options() (*sage.graphs.generic_graph.GenericGraph method*), 291
latin_squares_graph_parameters() (*in module sage.graphs.strongly_regular_db*), 1368
layout() (*sage.graphs.generic_graph.GenericGraph method*), 292
layout_acyclic() (*sage.graphs.digraph.DiGraph method*), 797
layout_acyclic_dummy() (*sage.graphs.digraph.DiGraph method*), 798
layout_circular() (*sage.graphs.generic_graph.GenericGraph method*), 295
layout_default() (*sage.graphs.generic_graph.GenericGraph method*), 296
layout_extend_randomly() (*sage.graphs.generic_graph.GenericGraph method*), 296
layout_forest() (*sage.graphs.generic_graph.GenericGraph method*), 297
layout_graphviz() (*sage.graphs.generic_graph.GenericGraph method*), 298
layout_planar() (*sage.graphs.generic_graph.GenericGraph method*), 300
layout_ranked() (*sage.graphs.generic_graph.GenericGraph method*), 303
layout_split() (*in module sage.graphs.generic_graph_pyx*), 2282
layout_spring() (*sage.graphs.generic_graph.GenericGraph method*), 303
layout_tree() (*sage.graphs.generic_graph.GenericGraph method*), 304
layout_tree() (*sage.graphs.graph_plot.GraphPlot method*), 2019
layout_tutte() (*sage.graphs.generic_graph.GenericGraph method*), 307
LCFGraph() (*in module sage.graphs.generators.families*), 1499
LCFGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1120
least_effective_resistance() (*sage.graphs.graph.Graph method*), 688
length_and_string_from_graph6() (*in module sage.graphs.generic_graph_pyx*), 2282
length_of_tree_decomposition() (*in module sage.graphs.graph_decompositions.tree_decomposition*), 2071
LeonardGraph() (*in module sage.graphs.generators.distance_regular*), 1453
LeonardGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1124
level_sets() (*sage.graphs.digraph.DiGraph method*), 800
lex_BFS() (*in module sage.graphs.traversals*), 1994
lex_BFS() (*sage.graphs.generic_graph.GenericGraph method*), 308
lex_DFS() (*in module sage.graphs.traversals*), 1996
lex_DFS() (*sage.graphs.generic_graph.GenericGraph method*), 311
lex_DOWN() (*in module sage.graphs.traversals*), 1998
lex_DOWN() (*sage.graphs.generic_graph.GenericGraph method*), 312
lex_M() (*in module sage.graphs.traversals*), 2000

lex_M() (*sage.graphs.graph.Graph method*), 689
lex_M_fast() (*in module sage.graphs.traversals*), 2002
lex_M_slow() (*in module sage.graphs.traversals*), 2004
lex_UP() (*in module sage.graphs.traversals*), 2005
lex_UP() (*sage.graphs.generic_graph.GenericGraph method*), 314
lexBFS_order() (*sage.graphs.graph_decompositions.slice_decomposition.SliceDecomposition method*), 2127
lexicographic_label() (*sage.graphs.graph_decompositions.slice_decomposition.SliceDecomposition method*), 2127
lexicographic_product() (*sage.graphs.generic_graph.GenericGraph method*), 316
lift() (*sage.graphs.generators.luw_graphs.LUWGraphDescriptor method*), 1379
line_count() (*sage.graphs.generators.luw_graphs.LUWGraphDescriptor method*), 1380
line_first_coordinates() (*sage.graphs.generators.luw_graphs.LUWGraphDescriptor method*), 1380
line_graph() (*in module sage.graphs.line_graph*), 1914
line_graph() (*sage.graphs.generic_graph.GenericGraph method*), 317
line_graph_forbidden_subgraphs() (*in module sage.graphs.generators.families*), 1524
line_graph_forbidden_subgraphs() (*sage.graphs.graph_generators.GraphGenerators static method*), 1249
linear_arboricity() (*in module sage.graphs.graph_coloring*), 1872
linear_ordering_to_path_decomposition() (*in module sage.graphs.graph_decompositions.vertex_separation*), 2086
LivingstoneGraph() (*in module sage.graphs.generators.smallgraphs*), 1617
LivingstoneGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1124
LjubljanaGraph() (*in module sage.graphs.generators.smallgraphs*), 1618
LjubljanaGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1125
load_afile() (*sage.graphs.bipartite_graph.BipartiteGraph method*), 850
locally_GQ42_distance_transitive_graph() (*in module sage.graphs.generators.distance_regular*), 1462
locally_GQ42_distance_transitive_graph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1249
LocalMcLaughlinGraph() (*in module sage.graphs.generators.smallgraphs*), 1619
LocalMcLaughlinGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1126
LollipopGraph() (*in module sage.graphs.generators.families*), 1500
LollipopGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1126
longest_cycle() (*sage.graphs.generic_graph.GenericGraph method*), 321
longest_dag_path() (*sage.graphs.digraph.DiGraph method*), 801
longest_path() (*sage.graphs.generic_graph.GenericGraph method*), 323
loop_edges() (*sage.graphs.generic_graph.GenericGraph method*), 326
loop_edges() (*sage.graphs.matching_covered_graph.MatchingCoveredGraph method*), 928
loop_vertices() (*sage.graphs.generic_graph.GenericGraph method*), 328
loop_vertices() (*sage.graphs.matching_covered_graph.MatchingCoveredGraph method*), 930
loops() (*sage.graphs.base.c_graph.CGraphBackend method*), 1731
loops() (*sage.graphs.base.graph_backends.GenericGraphBackend method*), 1799
loops() (*sage.graphs.generic_graph.GenericGraph method*), 328
loops() (*sage.graphs.matching_covered_graph.MatchingCoveredGraph method*), 932
lovasz_theta() (*in module sage.graphs.lovasz_theta*), 1985
lovasz_theta() (*sage.graphs.graph.Graph method*), 692
lower_bound() (*in module sage.graphs.graph_decompositions.vertex_separation*), 2088
luw_graphs (*sage.graphs.graph_generators.GraphGenerators attribute*), 1250
LUWGraph() (*in module sage.graphs.generators.luw_graphs*), 1375
LUWGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1121
LUWGraphDescriptor (*class in sage.graphs.generators.luw_graphs*), 1376

M

M22Graph() (*in module sage.graphs.generators.smallgraphs*), 1619
M22Graph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1127
M_alternating_even_mark() (*in module sage.graphs.matching*), 1945
magnitude_function() (*sage.graphs.graph.Graph method*), 692

make_labelled_rooted_tree() (*in module sage.graphs.graph_decompositions.clique_separators*), 2160
make_nice_tree_decomposition() (*in module sage.graphs.graph_decompositions.tree_decomposition*), 2073
make_tree() (*in module sage.graphs.graph_decompositions.clique_separators*), 2161
MarkstroemGraph() (*in module sage.graphs.generators.smallgraphs*), 1620
MarkstroemGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1127
matching() (*in module sage.graphs.matching*), 1962
matching() (*sage.graphs.bipartite_graph.BipartiteGraph method*), 851
matching() (*sage.graphs.graph.Graph method*), 694
matching_polynomial() (*in module sage.graphs.matchpoly*), 1967
matching_polynomial() (*sage.graphs.bipartite_graph.BipartiteGraph method*), 853
matching_polynomial() (*sage.graphs.graph.Graph method*), 695
MatchingCoveredGraph (*class in sage.graphs.matching_covered_graph*), 871
MathonPseudocyclicMergingGraph() (*in module sage.graphs.generators.families*), 1501
MathonPseudocyclicMergingGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1128
MathonPseudocyclicStronglyRegularGraph() (*in module sage.graphs.generators.families*), 1501
MathonPseudocyclicStronglyRegularGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1128
MathonStronglyRegularGraph() (*in module sage.graphs.generators.smallgraphs*), 1620
MathonStronglyRegularGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1130
max_clique() (*in module sage.graphs.cliquer*), 1880
max_cut() (*sage.graphs.generic_graph.GenericGraph method*), 329
maximal_barrier() (*sage.graphs.matching_covered_graph.MatchingCoveredGraph method*), 932
MaximizeDegree (*class in sage.graphs.tutte_polynomial*), 2240
maximum_average_degree() (*sage.graphs.graph.Graph method*), 701
maximum_cardinality_search() (*in module sage.graphs.traversals*), 2007
maximum_cardinality_search() (*sage.graphs.graph.Graph method*), 703
maximum_cardinality_search_M() (*in module sage.graphs.traversals*), 2009
maximum_cardinality_search_M() (*sage.graphs.graph.Graph method*), 704
maximum_leaf_number() (*in module sage.graphs.domination*), 2378
maximum_leaf_number() (*sage.graphs.generic_graph.GenericGraph method*), 330
McGeeGraph() (*in module sage.graphs.generators.smallgraphs*), 1621
McGeeGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1131
McLaughlinGraph() (*in module sage.graphs.generators.smallgraphs*), 1622
McLaughlinGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1131
md_tree_to_graph() (*in module sage.graphs.graph_decompositions.modular_decomposition*), 2148
MeredithGraph() (*in module sage.graphs.generators.smallgraphs*), 1622
MeredithGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1132
merge_vertices() (*sage.graphs.generic_graph.GenericGraph method*), 330
min_cycle_basis() (*in module sage.graphs.base.boost_graph*), 1814
min_spanning_tree() (*in module sage.graphs.base.boost_graph*), 1815
min_spanning_tree() (*sage.graphs.generic_graph.GenericGraph method*), 332
minimal_dominating_sets() (*in module sage.graphs.domination*), 2379
minimal_dominating_sets() (*sage.graphs.graph.Graph method*), 707
minimal_schnyder_wood() (*in module sage.graphs.schnyder*), 1989
minimal_separators() (*in module sage.graphs.connectivity*), 2342
minimal_separators() (*sage.graphs.graph.Graph method*), 711
MinimizeDegree (*class in sage.graphs.tutte_polynomial*), 2240
MinimizeSingleDegree (*class in sage.graphs.tutte_polynomial*), 2240
minimum_cycle_basis() (*sage.graphs.generic_graph.GenericGraph method*), 336
minimum_outdegree_orientation() (*in module sage.graphs.orientations*), 2290
minimum_outdegree_orientation() (*sage.graphs.graph.Graph method*), 712
minor() (*sage.graphs.graph.Graph method*), 715
mkgraph() (*in module sage.graphs.graph_decomposi-*

tions.rankwidth), 2103
modular_decomposition() (*in module sage.graphs.graph_decompositions.modular_decomposition*), 2149
modular_decomposition() (*sage.graphs.graph.Graph method*), 717
module
sage.combinat.designs.incidence_structures, 1824
sage.graphs.asteroidal_triples, 1884
sage.graphs.base.boost_graph, 1801
sage.graphs.base.c_graph, 1680
sage.graphs.base.dense_graph, 1755
sage.graphs.base.graph_backends, 1795
sage.graphs.base.overview, 1679
sage.graphs.base.sparse_graph, 1743
sage.graphs.base.static_dense_graph, 1766
sage.graphs.base.static_sparse_backend, 1784
sage.graphs.base.static_sparse_graph, 1776
sage.graphs.bipartite_graph, 830
sage.graphs.centrality, 1880
sage.graphs.chrompoly, 1888
sage.graphs.cliquer, 1876
sage.graphs.cographs, 1892
sage.graphs.comparability, 1897
sage.graphs.connectivity, 2299
sage.graphs.convexity_properties, 2162
sage.graphs.cycle_enumeration, 2384
sage.graphs.digraph, 764
sage.graphs.digraph_generators, 1282
sage.graphs.distances_all_pairs, 2171
sage.graphs.domination, 2366
sage.graphs.edge_connectivity, 2360
sage.graphs.generators.basic, 1387
sage.graphs.generators.chessboard, 1414
sage.graphs.generators.classical_geometries, 1421
sage.graphs.generators.degree_sequence, 1438
sage.graphs.generators.distance_regular, 1442
sage.graphs.generators.families, 1466
sage.graphs.generators.intersection, 1526
sage.graphs.generators.luw_graphs, 1373
sage.graphs.generators.platonic_solids, 1536
sage.graphs.generators.random, 1542
sage.graphs.generators.smallgraphs, 1566
sage.graphs.generators.trees, 1648
sage.graphs.generators.world_map, 1661
sage.graphs.generic_graph, 1
sage.graphs.generic_graph_pyx, 2274
sage.graphs.genus, 1980
sage.graphs.graph, 498
sage.graphs.graph_coloring, 1857
sage.graphs.graph_database, 1307
sage.graphs.graph_decompositions.bandwidth, 2105
sage.graphs.graph_decompositions.clique_separators, 2154
sage.graphs.graph_decompositions.cutwidth, 2108
sage.graphs.graph_decompositions.fast_digraph, 2115
sage.graphs.graph_decompositions.graph_products, 2116
sage.graphs.graph_decompositions.modular_decomposition, 2134
sage.graphs.graph_decompositions.rankwidth, 2101
sage.graphs.graph_decompositions.slice_decomposition, 2123
sage.graphs.graph_decompositions.tree_decomposition, 2064
sage.graphs.graph_decompositions.vertex_separation, 2083
sage.graphs.graph_editor, 2211
sage.graphs.graph_generators, 963
sage.graphs.graph_generators_pyx, 1307
sage.graphs.graph_input, 2219
sage.graphs.graph_latex, 2189
sage.graphs.graph_list, 2212
sage.graphs.graph_plot, 2012
sage.graphs.graph_plot_js, 2061
sage.graphs.hyperbolicity, 2230
sage.graphs.hypergraph_generators, 1821
sage.graphs.independent_sets, 1886
sage.graphs.isgci, 1665
sage.graphs.isoperimetric_inequalities, 1907
sage.graphs.line_graph, 1911
sage.graphs.lovasz_theta, 1985
sage.graphs.matching, 1945
sage.graphs.matching_covered_graph, 868
sage.graphs.matchpoly, 1966
sage.graphs.morphisms, 1972
sage.graphs.orientations, 2285
sage.graphs.partial_cube, 2246
sage.graphs.path_enumeration, 2249
sage.graphs.planarity, 1991
sage.graphs.pq_trees, 1935
sage.graphs.schnyder, 1986
sage.graphs.spanning_tree, 1918
sage.graphs.strongly_regular_db, 1334

sage.graphs.traversals, 1992
sage.graphs.tutte_polynomial, 2237
sage.graphs.views, 952
sage.graphs.weakly_chordal, 2168
MoebiusKantorGraph() (*in module sage.graphs.generators.smallgraphs*), 1623
MoebiusKantorGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1133
MoebiusLadderGraph() (*in module sage.graphs.generators.basic*), 1407
MoebiusLadderGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1134
MoserSpindle() (*in module sage.graphs.generators.smallgraphs*), 1624
MoserSpindle() (*sage.graphs.graph_generators.GraphGenerators static method*), 1135
most_common_neighbors() (*sage.graphs.graph.Graph method*), 720
multicommodity_flow() (*sage.graphs.generic_graph.GenericGraph method*), 338
multiple_edges() (*sage.graphs.base.dense_graph.DenseGraphBackend method*), 1765
multiple_edges() (*sage.graphs.base.graph_backends.GenericGraphBackend method*), 1799
multiple_edges() (*sage.graphs.base.sparse_graph.SparseGraphBackend method*), 1754
multiple_edges() (*sage.graphs.base.static_sparse_backend.StaticSparseBackend method*), 1793
multiple_edges() (*sage.graphs.generic_graph.GenericGraph method*), 339
multiway_cut() (*sage.graphs.generic_graph.GenericGraph method*), 342
MurtyGraph() (*in module sage.graphs.generators.smallgraphs*), 1625
MurtyGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1137
MuzychukS6Graph() (*in module sage.graphs.generators.families*), 1503
MuzychukS6Graph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1138
MycielskiGraph() (*in module sage.graphs.generators.families*), 1504
MycielskiGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1139
MycielskiStep() (*in module sage.graphs.generators.families*), 1505
MycielskiStep() (*sage.graphs.graph_generators.GraphGenerators static method*), 1140

N

n_blocks() (*sage.combinat.designs.incidence_structures.IncidenceStructure method*), 1852
n_edges() (*sage.graphs.base.c_graph.CGraphBackend method*), 1731
n_edges() (*sage.graphs.base.graph_backends.GenericGraphBackend method*), 1799
n_edges() (*sage.graphs.base.static_sparse_backend.StaticSparseBackend method*), 1793
n_edges() (*sage.graphs.generic_graph.GenericGraph method*), 344
n_faces() (*sage.graphs.generic_graph.GenericGraph method*), 344
n_points() (*sage.combinat.designs.incidence_structures.IncidenceStructure method*), 1852
n_vertices() (*sage.graphs.base.c_graph.CGraphBackend method*), 1732
n_vertices() (*sage.graphs.base.graph_backends.GenericGraphBackend method*), 1799
n_vertices() (*sage.graphs.base.static_sparse_backend.StaticSparseBackend method*), 1793
n_vertices() (*sage.graphs.generic_graph.GenericGraph method*), 345
name() (*sage.graphs.base.graph_backends.GenericGraphBackend method*), 1799
name() (*sage.graphs.generic_graph.GenericGraph method*), 345
NauruGraph() (*in module sage.graphs.generators.smallgraphs*), 1627
NauruGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1142
nauty() (*sage.graphs.hypergraph_generators.HypergraphGenerators method*), 1823
nauty_directg() (*sage.graphs.digraph_generators.DiGraphGenerators method*), 1303
nauty_genbg() (*sage.graphs.graph_generators.GraphGenerators method*), 1250
nauty_geng() (*sage.graphs.graph_generators.GraphGenerators method*), 1253
nauty_genktreeg() (*sage.graphs.graph_generators.GraphGenerators method*), 1255
nauty_gentreeg() (*in module sage.graphs.generators.trees*), 1659
nauty_gentreeg() (*sage.graphs.graph_generators.GraphGenerators static method*), 1257
nauty_posetg() (*sage.graphs.digraph_generators.DiGraphGenerators method*), 1305
nc_k_shortest_simple_paths() (*in module sage.graphs.path_enumeration*), 2264
near_polygon_graph() (*in module sage.graphs.generators.distance_regular*), 1462
neighbor_in_iterator() (*sage.graphs.digraph.DiGraph method*), 803
neighbor_iterator() (*sage.graphs.generic_graph.GenericGraph method*), 346
neighbor_out_iterator() (*sage.graphs.digraph.Di-*

Graph method), 804
neighbors() (*sage.graphs.generators.luw_graphs.LUWGraphDescriptor method*), 1381
neighbors() (*sage.graphs.generic_graph.GenericGraph method*), 347
neighbors_in() (*sage.graphs.digraph.DiGraph method*), 804
neighbors_out() (*sage.graphs.digraph.DiGraph method*), 805
nested_tuple_to_tree() (*in module sage.graphs.graph_decompositions.modular_decomposition*), 2150
networkx_graph() (*sage.graphs.generic_graph.GenericGraph method*), 348
next_tree() (*in module sage.graphs.cographs*), 1896
NKStarGraph() (*in module sage.graphs.generators.families*), 1505
NKStarGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1141
Node (*class in sage.graphs.graph_decompositions.modular_decomposition*), 2134
NodeType (*class in sage.graphs.graph_decompositions.modular_decomposition*), 2136
NonisotropicOrthogonalPolarGraph() (*in module sage.graphs.generators.classical_geometries*), 1425
NonisotropicOrthogonalPolarGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1142
NonisotropicUnitaryPolarGraph() (*in module sage.graphs.generators.classical_geometries*), 1428
NonisotropicUnitaryPolarGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1145
NORMAL (*sage.graphs.graph_decompositions.modular_decomposition.NodeType attribute*), 2136
Nowhere0WordsTwoWeightCodeGraph() (*in module sage.graphs.generators.classical_geometries*), 1428
Nowhere0WordsTwoWeightCodeGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1145
nowhere_zero_flow() (*sage.graphs.generic_graph.GenericGraph method*), 349
NStarGraph() (*in module sage.graphs.generators.families*), 1506
NStarGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1141
num_blocks() (*sage.combinat.designs.incidence_structures.IncidenceStructure method*), 1853
num_edges() (*sage.graphs.base.c_graph.CGraphBackend method*), 1732
num_edges() (*sage.graphs.base.graph_backends.GenericGraphBackend method*), 1799
num_edges() (*sage.graphs.base.static_sparse_backend.StaticSparseBackend method*), 1793
num_edges() (*sage.graphs.generic_graph.GenericGraph method*), 351
num_faces() (*sage.graphs.generic_graph.GenericGraph method*), 351
num_points() (*sage.combinat.designs.incidence_structures.IncidenceStructure method*), 1853
num_verts() (*sage.graphs.base.c_graph.CGraphBackend method*), 1732
num_verts() (*sage.graphs.base.graph_backends.GenericGraphBackend method*), 1799
num_verts() (*sage.graphs.base.static_sparse_backend.StaticSparseBackend method*), 1793
num_verts() (*sage.graphs.generic_graph.GenericGraph method*), 351
number_of() (*sage.graphs.graph_database.GraphQuery method*), 1325
number_of_biconnected_components() (*in module sage.graphs.connectivity*), 2343
number_of_biconnected_components() (*sage.graphs.generic_graph.GenericGraph method*), 351
number_of_children() (*sage.graphs.pq_trees.PQ method*), 1940
number_of_connected_components() (*in module sage.graphs.connectivity*), 2345
number_of_connected_components() (*sage.graphs.generic_graph.GenericGraph method*), 353
number_of_loops() (*sage.graphs.generic_graph.GenericGraph method*), 353
number_of_loops() (*sage.graphs.matching_covered_graph.MatchingCoveredGraph method*), 936
number_of_n_colorings() (*in module sage.graphs.graph_coloring*), 1873
number_of_spanning_trees() (*sage.graphs.generic_graph.GenericGraph method*), 354
numbers_of_colorings() (*in module sage.graphs.graph_coloring*), 1874

O

OctahedralGraph() (*in module sage.graphs.generators.platonic_solids*), 1539
OctahedralGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1147
odd_girth() (*sage.graphs.generic_graph.GenericGraph method*), 356
OddGraph() (*in module sage.graphs.generators.families*), 1507

OddGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1148
order() (*sage.graphs.generators.luw_graphs.LUWGraphDescriptor method*), 1381
order() (*sage.graphs.generic_graph.GenericGraph method*), 358
ordering() (*sage.graphs.pq_trees.PQ method*), 1940
orderings() (*sage.graphs.pq_trees.P method*), 1937
orderings() (*sage.graphs.pq_trees.Q method*), 1942
orient() (*in module sage.graphs.orientations*), 2293
orient() (*sage.graphs.graph.Graph method*), 721
orientations() (*in module sage.graphs.orientations*), 2294
orientations() (*sage.graphs.graph.Graph method*), 722
OrthogonalArrayBlockGraph() (*in module sage.graphs.generators.intersection*), 1528
OrthogonalArrayBlockGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1148
OrthogonalDualPolarGraph() (*in module sage.graphs.generators.classical_geometries*), 1430
OrthogonalDualPolarGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1152
OrthogonalPolarGraph() (*in module sage.graphs.generators.classical_geometries*), 1430
OrthogonalPolarGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1153
out_branchings() (*sage.graphs.digraph.DiGraph method*), 805
out_degree() (*sage.graphs.base.c_graph.CGraphBackend method*), 1732
out_degree() (*sage.graphs.base.dense_graph.DenseGraph method*), 1759
out_degree() (*sage.graphs.base.graph_backends.GenericGraphBackend method*), 1799
out_degree() (*sage.graphs.base.sparse_graph.SparseGraph method*), 1750
out_degree() (*sage.graphs.base.static_sparse_backend.StaticSparseBackend method*), 1793
out_degree() (*sage.graphs.base.static_sparse_backend.StaticSparseCGraph method*), 1794
out_degree() (*sage.graphs.digraph.DiGraph method*), 807
out_degree_iterator() (*sage.graphs.digraph.DiGraph method*), 808
out_degree_sequence() (*sage.graphs.digraph.DiGraph method*), 808
out_neighbors() (*sage.graphs.base.c_graph.CGraph method*), 1700
out_neighbors() (*sage.graphs.base.static_sparse_backend.StaticSparseCGraph method*), 1794
outgoing_edge_iterator() (*sage.graphs.digraph.DiGraph method*), 809
outgoing_edges() (*sage.graphs.digraph.DiGraph method*), 809

P

P (*class in sage.graphs.pq_trees*), 1937
p2_forbidden_minors() (*in module sage.graphs.generators.families*), 1524
p2_forbidden_minors() (*sage.graphs.graph_generators.GraphGenerators static method*), 1259
packing() (*sage.combinat.designs.incidence_structures.IncidenceStructure method*), 1853
pagerank() (*sage.graphs.generic_graph.GenericGraph method*), 358
Paley() (*sage.graphs.digraph_generators.DiGraphGenerators method*), 1294
PaleyGraph() (*in module sage.graphs.generators.families*), 1507
PaleyGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1154
PappusGraph() (*in module sage.graphs.generators.smallgraphs*), 1627
PappusGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1154
PARALLEL (*sage.graphs.graph_decompositions.modular_decomposition.NodeType attribute*), 2136
PasechnikGraph() (*in module sage.graphs.generators.families*), 1508
PasechnikGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1155
Path() (*sage.graphs.digraph_generators.DiGraphGenerators method*), 1295
path_decomposition() (*in module sage.graphs.graph_decompositions.vertex_separation*), 2089
path_semigroup() (*sage.graphs.digraph.DiGraph method*), 810
PathGraph() (*in module sage.graphs.generators.basic*), 1409
PathGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1156
pathwidth() (*in module sage.graphs.graph_decompositions.vertex_separation*), 2090
pathwidth() (*sage.graphs.graph.Graph method*), 723
perfect_matchings() (*in module sage.graphs.matching*), 1964
perfect_matchings() (*sage.graphs.bipartite_graph.BipartiteGraph method*), 854
perfect_matchings() (*sage.graphs.graph.Graph method*), 725
period() (*sage.graphs.digraph.DiGraph method*), 810
periphery() (*sage.graphs.digraph.DiGraph method*), 811

periphery() (*sage.graphs.graph.Graph method*), 727
PerkelGraph() (*in module sage.graphs.generators.smallgraphs*), 1628
PerkelGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1157
PermutationGraph() (*in module sage.graphs.generators.intersection*), 1531
PermutationGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1157
permute_decomposition() (*in module sage.graphs.graph_decompositions.modular_decomposition*), 2150
petersen_family() (*in module sage.graphs.generators.families*), 1525
petersen_family() (*sage.graphs.graph_generators.GraphGenerators static method*), 1259
PetersenGraph() (*in module sage.graphs.generators.smallgraphs*), 1628
PetersenGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1160
planar_dual() (*sage.graphs.generic_graph.GenericGraph method*), 362
planar_graphs() (*sage.graphs.graph_generators.GraphGenerators method*), 1260
plantri_gen() (*sage.graphs.graph_generators.GraphGenerators method*), 1263
platonic_solids (*sage.graphs.graph_generators.GraphGenerators attribute*), 1267
plot() (*sage.graphs.bipartite_graph.BipartiteGraph method*), 857
plot() (*sage.graphs.generic_graph.GenericGraph method*), 363
plot() (*sage.graphs.graph_plot.GraphPlot method*), 2020
plot3d() (*sage.graphs.generic_graph.GenericGraph method*), 372
pnc_k_shortest_simple_paths() (*in module sage.graphs.path_enumeration*), 2267
point_count() (*sage.graphs.generators.luw_graphs.LUWGraphDescriptor method*), 1382
point_first_coordinates() (*sage.graphs.generators.luw_graphs.LUWGraphDescriptor method*), 1382
PoussinGraph() (*in module sage.graphs.generators.smallgraphs*), 1629
PoussinGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1161
power() (*sage.graphs.generic_graph.GenericGraph method*), 376
PQ (*class in sage.graphs.pq_trees*), 1939
PRIME (*sage.graphs.graph_decompositions.modular_decomposition.NodeType attribute*), 2136
print_adjacency_matrix() (*sage.graphs.graph_decompositions.fast_digraph.FastDigraph method*), 2115
print_md_tree() (*in module sage.graphs.graph_decompositions.modular_decomposition*), 2151
print_triconnected_components() (*sage.graphs.connectivity.TriconnectivitySPQR method*), 2307
private_neighbors() (*in module sage.graphs.domination*), 2383
private_neighbors() (*sage.graphs.graph.Graph method*), 728
project_down() (*sage.graphs.generators.luw_graphs.LUWGraphDescriptor method*), 1382
project_left() (*sage.graphs.bipartite_graph.BipartiteGraph method*), 857
project_right() (*sage.graphs.bipartite_graph.BipartiteGraph method*), 857
pruned_tree() (*in module sage.graphs.generators.random*), 1566
pseudo_partition_graph() (*in module sage.graphs.generators.distance_regular*), 1463

Q

Q (*class in sage.graphs.pq_trees*), 1942
quadrangulations() (*sage.graphs.graph_generators.GraphGenerators method*), 1267
QueenGraph() (*in module sage.graphs.generators.chessboard*), 1418
QueenGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1162
query() (*sage.graphs.graph_database.GraphDatabase method*), 1317
query_iterator() (*sage.graphs.graph_database.GraphQuery method*), 1326

R

radius() (*sage.graphs.digraph.DiGraph method*), 812
radius() (*sage.graphs.graph.Graph method*), 729
radius_DHV() (*in module sage.graphs.base.boost_graph*), 1816
radius_DHV() (*in module sage.graphs.distances_all_pairs*), 2185
random (*sage.graphs.graph_generators.GraphGenerators attribute*), 1269
random() (*sage.graphs.graph_coloring.Test method*), 1858
random_all_graph_colorings() (*sage.graphs.graph_coloring.Test method*), 1858
random_edge() (*sage.graphs.generic_graph.GenericGraph method*), 377
random_edge_iterator() (*sage.graphs.generic_graph.GenericGraph method*), 377

random_md_tree() (*in module sage.graphs.graph_decompositions.modular_decomposition*), 2151
random_orientation() (*in module sage.graphs.orientations*), 2295
random_orientation() (*sage.graphs.graph.Graph method*), 730
random_spanning_tree() (*in module sage.graphs.spanning_tree*), 1931
random_spanning_tree() (*sage.graphs.graph.Graph method*), 730
random_subgraph() (*sage.graphs.generic_graph.GenericGraph method*), 378
random_vertex() (*sage.graphs.generic_graph.GenericGraph method*), 378
random_vertex_iterator() (*sage.graphs.generic_graph.GenericGraph method*), 379
RandomBarabasiAlbert() (*in module sage.graphs.generators.random*), 1542
RandomBarabasiAlbert() (*sage.graphs.graph_generators.GraphGenerators static method*), 1163
RandomBicubicPlanar() (*in module sage.graphs.generators.random*), 1543
RandomBicubicPlanar() (*sage.graphs.graph_generators.GraphGenerators static method*), 1165
RandomBipartite() (*in module sage.graphs.generators.random*), 1544
RandomBipartite() (*sage.graphs.graph_generators.GraphGenerators static method*), 1166
RandomBlockGraph() (*in module sage.graphs.generators.random*), 1545
RandomBlockGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1167
RandomBoundedToleranceGraph() (*in module sage.graphs.generators.random*), 1548
RandomBoundedToleranceGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1169
RandomChordalGraph() (*in module sage.graphs.generators.random*), 1548
RandomChordalGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1170
RandomDirectedAcyclicGraph() (*sage.graphs.digraph_generators.DiGraphGenerators method*), 1296
RandomDirectedGN() (*sage.graphs.digraph_generators.DiGraphGenerators method*), 1296
RandomDirectedGNC() (*sage.graphs.digraph_generators.DiGraphGenerators method*), 1297
RandomDirectedGNM() (*sage.graphs.digraph_generators.DiGraphGenerators method*), 1298
RandomDirectedGNP() (*sage.graphs.digraph_generators.DiGraphGenerators method*), 1299
RandomDirectedGNR() (*sage.graphs.digraph_generators.DiGraphGenerators method*), 1299
RandomGNM() (*in module sage.graphs.generators.random*), 1550
RandomGNM() (*sage.graphs.graph_generators.GraphGenerators static method*), 1171
RandomGNP() (*in module sage.graphs.generators.random*), 1551
RandomGNP() (*in module sage.graphs.graph_generators_pyx*), 1307
RandomGNP() (*sage.graphs.graph_generators.GraphGenerators static method*), 1173
RandomHolmeKim() (*in module sage.graphs.generators.random*), 1553
RandomHolmeKim() (*sage.graphs.graph_generators.GraphGenerators static method*), 1174
RandomIntervalGraph() (*in module sage.graphs.generators.random*), 1554
RandomIntervalGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1175
RandomKTree() (*in module sage.graphs.generators.random*), 1554
RandomKTree() (*sage.graphs.graph_generators.GraphGenerators static method*), 1176
RandomLobster() (*in module sage.graphs.generators.trees*), 1655
RandomLobster() (*sage.graphs.graph_generators.GraphGenerators static method*), 1176
RandomNewmanWattsStrogatz() (*in module sage.graphs.generators.random*), 1555
RandomNewmanWattsStrogatz() (*sage.graphs.graph_generators.GraphGenerators static method*), 1178
RandomPartialKTree() (*in module sage.graphs.generators.random*), 1556
RandomPartialKTree() (*sage.graphs.graph_generators.GraphGenerators static method*), 1179
RandomProperIntervalGraph() (*in module sage.graphs.generators.random*), 1557
RandomProperIntervalGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1179
RandomRegular() (*in module sage.graphs.generators.random*), 1557
RandomRegular() (*sage.graphs.graph_generators.GraphGenerators static method*), 1180
RandomRegularBipartite() (*in module sage.graphs.generators.random*), 1558
RandomRegularBipartite() (*sage.graphs.graph_generators.GraphGenerators static method*), 1181
RandomSemiComplete() (*sage.graphs.digraph_generators.DiGraphGenerators method*), 1300
RandomShell() (*in module sage.graphs.generators.random*), 1559
RandomShell() (*sage.graphs.graph_generators.Graph-*

Generators static method), 1182
RandomToleranceGraph() (*in module sage.graphs.generators.random*), 1560
RandomToleranceGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1183
RandomTournament() (*sage.graphs.digraph_generators.DiGraphGenerators method*), 1301
RandomTree() (*in module sage.graphs.generators.trees*), 1656
RandomTree() (*sage.graphs.graph_generators.GraphGenerators static method*), 1183
RandomTreePowerlaw() (*in module sage.graphs.generators.trees*), 1656
RandomTreePowerlaw() (*sage.graphs.graph_generators.GraphGenerators static method*), 1184
RandomTriangulation() (*in module sage.graphs.generators.random*), 1561
RandomTriangulation() (*sage.graphs.graph_generators.GraphGenerators static method*), 1185
RandomUnitDiskGraph() (*in module sage.graphs.generators.random*), 1562
RandomUnitDiskGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1186
rank() (*sage.combinat.designs.incidence_structures.IncidenceStructure method*), 1853
rank_decomposition() (*in module sage.graphs.graph_decompositions.rankwidth*), 2103
rank_decomposition() (*sage.graphs.graph.Graph method*), 733
RankWidth (*sage.graphs.graph.Graph attribute*), 529
realloc() (*sage.graphs.base.c_graph.CGraph method*), 1702
realloc() (*sage.graphs.base.dense_graph.DenseGraph method*), 1760
realloc() (*sage.graphs.base.sparse_graph.SparseGraph method*), 1750
rebuild_node() (*in module sage.graphs.cographs*), 1897
recreate_decomposition() (*in module sage.graphs.graph_decompositions.modular_decomposition*), 2152
reduced_adjacency_matrix() (*sage.graphs.bipartite_graph.BipartiteGraph method*), 858
reduced_homeomorphic_graph() (*in module sage.graphs.morphisms*), 1977
reduced_homeomorphic_graph() (*sage.graphs.generic_graph.GenericGraph method*), 379
reduced_tree_decomposition() (*in module sage.graphs.graph_decompositions.tree_decomposition*), 2076
relabel() (*sage.combinat.designs.incidence_structures.IncidenceStructure method*), 1854
relabel() (*sage.graphs.base.c_graph.CGraphBackend method*), 1733
relabel() (*sage.graphs.base.graph_backends.GenericGraphBackend method*), 1800
relabel() (*sage.graphs.base.static_sparse_backend.StaticSparseBackend method*), 1793
relabel() (*sage.graphs.bipartite_graph.BipartiteGraph method*), 860
relabel() (*sage.graphs.generic_graph.GenericGraph method*), 382
relabel_tree() (*in module sage.graphs.graph_decompositions.modular_decomposition*), 2152
remove_loops() (*sage.graphs.generic_graph.GenericGraph method*), 388
remove_loops() (*sage.graphs.matching_covered_graph.MatchingCoveredGraph method*), 938
remove_multiple_edges() (*sage.graphs.generic_graph.GenericGraph method*), 389
removed_edge() (*in module sage.graphs.tutte_polynomial*), 2242
removed_from() (*sage.graphs.tutte_polynomial.Ear method*), 2238
removed_loops() (*in module sage.graphs.tutte_polynomial*), 2242
removed_multiedge() (*in module sage.graphs.tutte_polynomial*), 2243
reorder_sets() (*in module sage.graphs.pq_trees*), 1944
reset_info() (*sage.graphs.cographs.CoTree method*), 1894
reverse() (*sage.graphs.digraph.DiGraph method*), 813
reverse() (*sage.graphs.pq_trees.PQ method*), 1940
reverse_edge() (*sage.graphs.digraph.DiGraph method*), 814
reverse_edges() (*sage.graphs.digraph.DiGraph method*), 818
RingedTree() (*in module sage.graphs.generators.families*), 1509
RingedTree() (*sage.graphs.graph_generators.GraphGenerators static method*), 1187
RobertsonGraph() (*in module sage.graphs.generators.smallgraphs*), 1629
RobertsonGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1188
RookGraph() (*in module sage.graphs.generators.chessboard*), 1420
RookGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1189
root_graph() (*in module sage.graphs.line_graph*), 1917
rooted_product() (*in module sage.graphs.graph_decompositions.graph_products*), 2120
rooted_product() (*sage.graphs.generic_graph.GenericGraph method*), 390

RoseWindowGraph() (*in module sage.graphs.generators.families*), 1509
RoseWindowGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1190
round_robin() (*in module sage.graphs.graph_coloring*), 1874

S

s (*sage.graphs.tutte_polynomial.Ear property*), 2239
sage.combinat.designs.incidence_structures
    module, 1824
sage.graphs.asteroidal_triples
    module, 1884
sage.graphs.base.boost_graph
    module, 1801
sage.graphs.base.c_graph
    module, 1680
sage.graphs.base.dense_graph
    module, 1755
sage.graphs.base.graph_backends
    module, 1795
sage.graphs.base.overview
    module, 1679
sage.graphs.base.sparse_graph
    module, 1743
sage.graphs.base.static_dense_graph
    module, 1766
sage.graphs.base.static_sparse_backend
    module, 1784
sage.graphs.base.static_sparse_graph
    module, 1776
sage.graphs.bipartite_graph
    module, 830
sage.graphs.centrality
    module, 1880
sage.graphs.chrompoly
    module, 1888
sage.graphs.cliquer
    module, 1876
sage.graphs.cographs
    module, 1892
sage.graphs.comparability
    module, 1897
sage.graphs.connectivity
    module, 2299
sage.graphs.convexity_properties
    module, 2162
sage.graphs.cycle_enumeration
    module, 2384
sage.graphs.digraph
    module, 764
sage.graphs.digraph_generators
    module, 1282
sage.graphs.distances_all_pairs
    module, 2171
sage.graphs.domination
    module, 2366
sage.graphs.edge_connectivity
    module, 2360
sage.graphs.generators.basic
    module, 1387
sage.graphs.generators.chessboard
    module, 1414
sage.graphs.generators.classical_geometries
    module, 1421
sage.graphs.generators.degree_sequence
    module, 1438
sage.graphs.generators.distance_regular
    module, 1442
sage.graphs.generators.families
    module, 1466
sage.graphs.generators.intersection
    module, 1526
sage.graphs.generators.luw_graphs
    module, 1373
sage.graphs.generators.platonic_solids
    module, 1536
sage.graphs.generators.random
    module, 1542
sage.graphs.generators.smallgraphs
    module, 1566
sage.graphs.generators.trees
    module, 1648
sage.graphs.generators.world_map
    module, 1661
sage.graphs.generic_graph
    module, 1
sage.graphs.generic_graph_pyx
    module, 2274
sage.graphs.genus
    module, 1980
sage.graphs.graph
    module, 498
sage.graphs.graph_coloring
    module, 1857
sage.graphs.graph_database
    module, 1307
sage.graphs.graph_decompositions.bandwidth
    module, 2105
sage.graphs.graph_decompositions.clique_separators
    module, 2154
sage.graphs.graph_decompositions.cutwidth
    module, 2108
sage.graphs.graph_decompositions.fast_digraph
    module, 2115

sage.graphs.graph_decomposi-
tions.graph_products
module, 2116
sage.graphs.graph_decompositions.modu-
lar_decomposition
module, 2134
sage.graphs.graph_decompositions.rankwidth
module, 2101
sage.graphs.graph_decompositions.slice_de-
composition
module, 2123
sage.graphs.graph_decompositions.tree_de-
composition
module, 2064
sage.graphs.graph_decompositions.ver-
tex_separation
module, 2083
sage.graphs.graph_editor
module, 2211
sage.graphs.graph_generators
module, 963
sage.graphs.graph_generators_pyx
module, 1307
sage.graphs.graph_input
module, 2219
sage.graphs.graph_latex
module, 2189
sage.graphs.graph_list
module, 2212
sage.graphs.graph_plot
module, 2012
sage.graphs.graph_plot_js
module, 2061
sage.graphs.hyperbolicity
module, 2230
sage.graphs.hypergraph_generators
module, 1821
sage.graphs.independent_sets
module, 1886
sage.graphs.isgci
module, 1665
sage.graphs.isoperimetric_inequalities
module, 1907
sage.graphs.line_graph
module, 1911
sage.graphs.lovasz_theta
module, 1985
sage.graphs.matching
module, 1945
sage.graphs.matching_covered_graph
module, 868
sage.graphs.matchpoly
module, 1966
sage.graphs.morphisms
module, 1972
sage.graphs.orientations
module, 2285
sage.graphs.partial_cube
module, 2246
sage.graphs.path_enumeration
module, 2249
sage.graphs.planarity
module, 1991
sage.graphs.pq_trees
module, 1935
sage.graphs.schnyder
module, 1986
sage.graphs.spanning_tree
module, 1918
sage.graphs.strongly_regular_db
module, 1334
sage.graphs.traversals
module, 1992
sage.graphs.tutte_polynomial
module, 2237
sage.graphs.views
module, 952
sage.graphs.weakly_chordal
module, 2168
save_afile() (*sage.graphs.bipartite_graph.BipartiteGraph method*), 865
SchlaefliGraph() (*in module sage.graphs.generators.smallgraphs*), 1630
SchlaefliGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1191
Search_iterator (*class in sage.graphs.base.c_graph*), 1742
seidel_adjacency_matrix() (*sage.graphs.graph.Graph method*), 735
seidel_switching() (*sage.graphs.graph.Graph method*), 736
SERIES (*sage.graphs.graph_decompositions.modular_decomposition.NodeType attribute*), 2136
set_contiguous() (*sage.graphs.pq_trees.P method*), 1938
set_contiguous() (*sage.graphs.pq_trees.Q method*), 1943
set_edge_label() (*sage.graphs.base.dense_graph.DenseGraphBackend method*), 1765
set_edge_label() (*sage.graphs.base.graph_backends.GenericGraphBackend method*), 1800
set_edge_label() (*sage.graphs.base.sparse_graph.SparseGraphBackend method*), 1754
set_edge_label() (*sage.graphs.base.static_sparse_backend.StaticSparseBackend method*), 1794
set_edge_label() (*sage.graphs.generic_graph.GenericGraph method*), 393

set_edges() (*sage.graphs.graph_plot.GraphPlot method*), 2043
set_embedding() (*sage.graphs.generic_graph.GenericGraph method*), 396
set_latex_options() (*sage.graphs.generic_graph.GenericGraph method*), 397
set_option() (*sage.graphs.graph_latex.GraphLatex method*), 2202
set_options() (*sage.graphs.graph_latex.GraphLatex method*), 2207
set_pos() (*sage.graphs.generic_graph.GenericGraph method*), 398
set_pos() (*sage.graphs.graph_plot.GraphPlot method*), 2049
set_vertex() (*sage.graphs.generic_graph.GenericGraph method*), 399
set_vertices() (*sage.graphs.generic_graph.GenericGraph method*), 399
set_vertices() (*sage.graphs.graph_plot.GraphPlot method*), 2051
setup_latex_preamble() (*in module sage.graphs.graph_latex*), 2211
shortened_00_11_binary_Golay_code_graph() (*in module sage.graphs.generators.distance_regular*), 1465
shortened_00_11_binary_Golay_code_graph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1270
shortened_000_111_extended_binary_Golay_code_graph() (*in module sage.graphs.generators.distance_regular*), 1464
shortened_000_111_extended_binary_Golay_code_graph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1269
shortest_path() (*sage.graphs.base.c_graph.CGraphBackend method*), 1734
shortest_path() (*sage.graphs.generic_graph.GenericGraph method*), 400
shortest_path_all_pairs() (*in module sage.graphs.distances_all_pairs*), 2185
shortest_path_all_pairs() (*sage.graphs.generic_graph.GenericGraph method*), 402
shortest_path_all_vertices() (*sage.graphs.base.c_graph.CGraphBackend method*), 1734
shortest_path_length() (*sage.graphs.generic_graph.GenericGraph method*), 410
shortest_path_lengths() (*sage.graphs.generic_graph.GenericGraph method*), 413
shortest_path_special() (*sage.graphs.base.c_graph.CGraphBackend method*), 1735
shortest_path_to_set() (*sage.graphs.base.c_graph.CGraphBackend method*), 1736
shortest_paths() (*in module sage.graphs.base.boost_graph*), 1816
shortest_paths() (*sage.graphs.generic_graph.GenericGraph method*), 416
shortest_paths_from_vertices() (*in module sage.graphs.base.boost_graph*), 1817
shortest_simple_paths() (*in module sage.graphs.path_enumeration*), 2268
shortest_simple_paths() (*sage.graphs.generic_graph.GenericGraph method*), 419
show() (*sage.graphs.generic_graph.GenericGraph method*), 423
show() (*sage.graphs.graph_database.GraphQuery method*), 1327
show() (*sage.graphs.graph_plot.GraphPlot method*), 2058
show3d() (*sage.graphs.generic_graph.GenericGraph method*), 423
show_all() (*sage.graphs.isgci.GraphClasses method*), 1675
show_graphs() (*in module sage.graphs.graph_list*), 2215
ShrikhandeGraph() (*in module sage.graphs.generators.smallgraphs*), 1631
ShrikhandeGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1192
SierpinskiGasketGraph() (*in module sage.graphs.generators.families*), 1510
SierpinskiGasketGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1194
simple_connected_genus_backtracker (*class in sage.graphs.genus*), 1980
simple_connected_graph_genus() (*in module sage.graphs.genus*), 1984
simplify() (*sage.graphs.pq_trees.PQ method*), 1941
SimsGewirtzGraph() (*in module sage.graphs.generators.smallgraphs*), 1633
SimsGewirtzGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1196
sinks() (*sage.graphs.digraph.DiGraph method*), 821
size() (*sage.graphs.generators.luw_graphs.LUWGraphDescriptor method*), 1383
size() (*sage.graphs.generic_graph.GenericGraph method*), 426
slice() (*sage.graphs.graph_decompositions.slice_decomposition.SliceDecomposition method*), 2128
slice_decomposition() (*in module sage.graphs.graph_decompositions.slice_decomposition*), 2132
slice_decomposition() (*sage.graphs.graph.Graph*

method), 736
SliceDecomposition (*class in sage.graphs.graph_decompositions.slice_decomposition*), 2123
small_integer_to_graph6() (*in module sage.graphs.generic_graph_pyx*), 2283
smallgraphs (*sage.graphs.graph_generators.GraphGenerators attribute*), 1270
smallgraphs() (*sage.graphs.isgci.GraphClasses method*), 1676
sources() (*sage.graphs.digraph.DiGraph method*), 821
SousselierGraph() (*in module sage.graphs.generators.smallgraphs*), 1634
SousselierGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1197
spanning_trees() (*in module sage.graphs.spanning_tree*), 1934
spanning_trees() (*sage.graphs.graph.Graph method*), 738
spanning_trees_count() (*sage.graphs.generic_graph.GenericGraph method*), 426
sparse6_string() (*sage.graphs.graph.Graph method*), 739
SparseGraph (*class in sage.graphs.base.sparse_graph*), 1747
SparseGraphBackend (*class in sage.graphs.base.sparse_graph*), 1752
spectral_radius() (*in module sage.graphs.base.static_sparse_graph*), 1779
spectral_radius() (*sage.graphs.generic_graph.GenericGraph method*), 426
spectrum() (*sage.graphs.generic_graph.GenericGraph method*), 430
spqr_tree() (*in module sage.graphs.connectivity*), 2346
spqr_tree() (*sage.graphs.graph.Graph method*), 740
spqr_tree_to_graph() (*in module sage.graphs.connectivity*), 2351
spring_layout_fast() (*in module sage.graphs.generic_graph_pyx*), 2284
SquaredSkewHadamardMatrixGraph() (*in module sage.graphs.generators.families*), 1512
SquaredSkewHadamardMatrixGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1198
SRG_100_44_18_20() (*in module sage.graphs.strongly_regular_db*), 1334
SRG_100_45_20_20() (*in module sage.graphs.strongly_regular_db*), 1334
SRG_105_32_4_12() (*in module sage.graphs.strongly_regular_db*), 1335
SRG_120_63_30_36() (*in module sage.graphs.strongly_regular_db*), 1335
SRG_120_77_52_44() (*in module sage.graphs.strongly_regular_db*), 1336
SRG_126_25_8_4() (*in module sage.graphs.strongly_regular_db*), 1336
SRG_126_50_13_24() (*in module sage.graphs.strongly_regular_db*), 1336
SRG_144_39_6_12() (*in module sage.graphs.strongly_regular_db*), 1337
SRG_175_72_20_36() (*in module sage.graphs.strongly_regular_db*), 1338
SRG_176_49_12_14() (*in module sage.graphs.strongly_regular_db*), 1338
SRG_176_90_38_54() (*in module sage.graphs.strongly_regular_db*), 1339
SRG_176_105_68_54() (*in module sage.graphs.strongly_regular_db*), 1338
SRG_196_91_42_42() (*in module sage.graphs.strongly_regular_db*), 1339
SRG_210_99_48_45() (*in module sage.graphs.strongly_regular_db*), 1339
SRG_220_84_38_28() (*in module sage.graphs.strongly_regular_db*), 1340
SRG_243_110_37_60() (*in module sage.graphs.strongly_regular_db*), 1340
SRG_253_140_87_65() (*in module sage.graphs.strongly_regular_db*), 1341
SRG_276_140_58_84() (*in module sage.graphs.strongly_regular_db*), 1341
SRG_280_117_44_52() (*in module sage.graphs.strongly_regular_db*), 1342
SRG_280_135_70_60() (*in module sage.graphs.strongly_regular_db*), 1342
SRG_416_100_36_20() (*in module sage.graphs.strongly_regular_db*), 1342
SRG_560_208_72_80() (*in module sage.graphs.strongly_regular_db*), 1343
SRG_630_85_20_10() (*in module sage.graphs.strongly_regular_db*), 1343
SRG_1288_792_476_504() (*in module sage.graphs.strongly_regular_db*), 1337
SRG_from_RSHCD() (*in module sage.graphs.strongly_regular_db*), 1344
StaircaseGraph() (*in module sage.graphs.generators.families*), 1512
StaircaseGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1198
StarGraph() (*in module sage.graphs.generators.basic*), 1410
StarGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1200
StaticSparseBackend (*class in sage.graphs.base.static_sparse_backend*), 1785
StaticSparseCGraph (*class in sage.graphs.base.static_sparse_backend*), 1794
steiner_tree() (*sage.graphs.generic_graph.GenericGraph method*), 431

strong_articulation_points() (*in module sage.graphs.connectivity*), 2352
strong_articulation_points() (*sage.graphs.digraph.DiGraph method*), 822
strong_orientation() (*in module sage.graphs.orientations*), 2295
strong_orientation() (*sage.graphs.graph.Graph method*), 745
strong_orientations_iterator() (*in module sage.graphs.orientations*), 2297
strong_orientations_iterator() (*sage.graphs.graph.Graph method*), 746
strong_product() (*sage.graphs.generic_graph.GenericGraph method*), 433
strongly_connected_component_containing_vertex() (*in module sage.graphs.connectivity*), 2353
strongly_connected_component_containing_vertex() (*sage.graphs.base.c_graph.CGraphBackend method*), 1737
strongly_connected_component_containing_vertex() (*sage.graphs.digraph.DiGraph method*), 823
strongly_connected_components() (*sage.graphs.digraph.DiGraph method*), 824
strongly_connected_components_digraph() (*in module sage.graphs.base.static_sparse_graph*), 1782
strongly_connected_components_digraph() (*in module sage.graphs.connectivity*), 2354
strongly_connected_components_digraph() (*sage.graphs.digraph.DiGraph method*), 824
strongly_connected_components_subgraphs() (*in module sage.graphs.connectivity*), 2356
strongly_connected_components_subgraphs() (*sage.graphs.digraph.DiGraph method*), 826
strongly_regular_db (*sage.graphs.graph_generators.GraphGenerators attribute*), 1270
strongly_regular_from_two_intersection_set() (*in module sage.graphs.strongly_regular_db*), 1368
strongly_regular_from_two_weight_code() (*in module sage.graphs.strongly_regular_db*), 1369
strongly_regular_graph() (*in module sage.graphs.strongly_regular_db*), 1370
strongly_regular_graph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1270
strongly_regular_graph_lazy() (*in module sage.graphs.strongly_regular_db*), 1372
StronglyRegular() (*sage.graphs.digraph_generators.DiGraphGenerators method*), 1302
subdivide_edge() (*sage.graphs.generic_graph.GenericGraph method*), 434
subdivide_edge() (*sage.graphs.matching_covered_graph.MatchingCoveredGraph method*), 941
subdivide_edges() (*sage.graphs.generic_graph.GenericGraph method*), 436
subdivide_edges() (*sage.graphs.matching_covered_graph.MatchingCoveredGraph method*), 946
subgraph() (*sage.graphs.generic_graph.GenericGraph method*), 437
subgraph_decompositions() (*sage.graphs.generic_graph.GenericGraph method*), 441
subgraph_given_vertices() (*sage.graphs.base.c_graph.CGraphBackend method*), 1738
subgraph_search() (*sage.graphs.generic_graph.GenericGraph method*), 444
subgraph_search_count() (*sage.graphs.generic_graph.GenericGraph method*), 448
subgraph_search_iterator() (*sage.graphs.generic_graph.GenericGraph method*), 450
subgraphs_to_query() (*in module sage.graphs.graph_database*), 1333
SubgraphSearch (*class in sage.graphs.generic_graph_pyx*), 2274
SuzukiGraph() (*in module sage.graphs.generators.smallgraphs*), 1635
SuzukiGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1202
SwitchedSquaredSkewHadamardMatrixGraph() (*in module sage.graphs.generators.families*), 1514
SwitchedSquaredSkewHadamardMatrixGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1203
SylvesterGraph() (*in module sage.graphs.generators.smallgraphs*), 1636
SylvesterGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1203
symmetric_edge_polytope() (*sage.graphs.generic_graph.GenericGraph method*), 453
SymplecticDualPolarGraph() (*in module sage.graphs.generators.classical_geometries*), 1431
SymplecticDualPolarGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1204
SymplecticPolarGraph() (*in module sage.graphs.generators.classical_geometries*), 1432
SymplecticPolarGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1205

szeged_index() (*in module sage.graphs.distances_all_pairs*), 2186
szeged_index() (*sage.graphs.generic_graph.GenericGraph method*), 458
SzekeresSnarkGraph() (*in module sage.graphs.generators.smallgraphs*), 1636
SzekeresSnarkGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1206

T

T2starGeneralizedQuadrangleGraph() (*in module sage.graphs.generators.classical_geometries*), 1433
T2starGeneralizedQuadrangleGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1206
TabacjnGraph() (*in module sage.graphs.generators.families*), 1515
TabacjnGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1208
tachyon_vertex_plot() (*in module sage.graphs.generic_graph*), 498
TadpoleGraph() (*in module sage.graphs.generators.families*), 1516
TadpoleGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1208
tarjan_strongly_connected_components() (*in module sage.graphs.base.static_sparse_graph*), 1782
TaylorTwographDescendantSRG() (*in module sage.graphs.generators.classical_geometries*), 1434
TaylorTwographDescendantSRG() (*sage.graphs.graph_generators.GraphGenerators static method*), 1209
TaylorTwographSRG() (*in module sage.graphs.generators.classical_geometries*), 1435
TaylorTwographSRG() (*sage.graphs.graph_generators.GraphGenerators static method*), 1210
tensor_product() (*sage.graphs.generic_graph.GenericGraph method*), 459
Test (*class in sage.graphs.graph_coloring*), 1858
test_popcount() (*in module sage.graphs.graph_decompositions.fast_digraph*), 2116
TetrahedralGraph() (*in module sage.graphs.generators.platonic_solids*), 1540
TetrahedralGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1211
ThomsenGraph() (*in module sage.graphs.generators.smallgraphs*), 1637
ThomsenGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1212
TietzeGraph() (*in module sage.graphs.generators.smallgraphs*), 1637
TietzeGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1212
tikz() (*sage.graphs.generic_graph.GenericGraph method*), 461
tkz_picture() (*sage.graphs.graph_latex.GraphLatex method*), 2207
to_dictionary() (*sage.graphs.generic_graph.GenericGraph method*), 465
to_directed() (*sage.graphs.digraph.DiGraph method*), 826
to_directed() (*sage.graphs.graph.Graph method*), 748
to_graph6() (*in module sage.graphs.graph_list*), 2217
to_graphics_array() (*in module sage.graphs.graph_list*), 2218
to_simple() (*sage.graphs.generic_graph.GenericGraph method*), 467
to_sparse6() (*in module sage.graphs.graph_list*), 2219
to_undirected() (*sage.graphs.bipartite_graph.BipartiteGraph method*), 866
to_undirected() (*sage.graphs.digraph.DiGraph method*), 827
to_undirected() (*sage.graphs.graph.Graph method*), 748
ToleranceGraph() (*in module sage.graphs.generators.intersection*), 1534
ToleranceGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1213
topological_minor() (*sage.graphs.graph.Graph method*), 748
topological_sort() (*sage.graphs.digraph.DiGraph method*), 827
topological_sort_generator() (*sage.graphs.digraph.DiGraph method*), 829
Toroidal6RegularGrid2dGraph() (*in module sage.graphs.generators.basic*), 1412
Toroidal6RegularGrid2dGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1215
ToroidalGrid2dGraph() (*in module sage.graphs.generators.basic*), 1413
ToroidalGrid2dGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1216
tournaments_nauty() (*sage.graphs.digraph_generators.DiGraphGenerators method*), 1306
trace() (*sage.combinat.designs.incidence_structures.IncidenceStructure method*), 1855
transitive_closure() (*sage.graphs.generic_graph.GenericGraph method*), 469
transitive_reduction() (*sage.graphs.generic_graph.GenericGraph method*), 471
transitive_reduction_acyclic() (*in module sage.graphs.generic_graph_pyx*), 2285

TransitiveTournament() (*sage.graphs.digraph_generators.DiGraphGenerators method*), 1302
traveling_salesman_problem() (*sage.graphs.generic_graph.GenericGraph method*), 472
tree_to_graph() (*in module sage.graphs.cographs*), 1897
tree_to_nested_tuple() (*in module sage.graphs.graph_decompositions.modular_decomposition*), 2153
TreeIterator (*class in sage.graphs.generators.trees*), 1657
treelength() (*in module sage.graphs.graph_decompositions.tree_decomposition*), 2077
treelength() (*sage.graphs.graph.Graph method*), 750
treelength_lowerbound() (*in module sage.graphs.graph_decompositions.tree_decomposition*), 2079
TreelengthConnected (*class in sage.graphs.graph_decompositions.tree_decomposition*), 2065
TreeNode (*class in sage.graphs.schnyder*), 1986
trees() (*in module sage.graphs.generators.trees*), 1660
trees() (*sage.graphs.graph_generators.GraphGenerators static method*), 1273
treewidth() (*in module sage.graphs.graph_decompositions.tree_decomposition*), 2079
treewidth() (*sage.graphs.graph.Graph method*), 752
triangles_count() (*in module sage.graphs.base.static_dense_graph*), 1775
triangles_count() (*in module sage.graphs.base.static_sparse_graph*), 1784
triangles_count() (*sage.graphs.generic_graph.GenericGraph method*), 475
triangulations() (*sage.graphs.graph_generators.GraphGenerators method*), 1274
TriconnectivitySPQR (*class in sage.graphs.connectivity*), 2300
TricornGraph() (*in module sage.graphs.generators.smallgraphs*), 1638
TricornGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1216
TruncatedBiwheelGraph() (*in module sage.graphs.generators.families*), 1516
TruncatedBiwheelGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1218
TruncatedIcosidodecahedralGraph() (*in module sage.graphs.generators.smallgraphs*), 1640
TruncatedIcosidodecahedralGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1219
TruncatedTetrahedralGraph() (*in module sage.graphs.generators.smallgraphs*), 1640
TruncatedTetrahedralGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1220
TruncatedWittGraph() (*in module sage.graphs.generators.distance_regular*), 1453
TruncatedWittGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1220
TuranGraph() (*in module sage.graphs.generators.families*), 1518
TuranGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1221
Tutte12Cage() (*in module sage.graphs.generators.smallgraphs*), 1641
Tutte12Cage() (*sage.graphs.graph_generators.GraphGenerators static method*), 1222
tutte_polynomial() (*in module sage.graphs.tutte_polynomial*), 2243
tutte_polynomial() (*sage.graphs.graph.Graph method*), 755
tutte_symmetric_function() (*sage.graphs.graph.Graph method*), 756
TutteCoxeterGraph() (*in module sage.graphs.generators.smallgraphs*), 1641
TutteCoxeterGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1223
TutteGraph() (*in module sage.graphs.generators.smallgraphs*), 1642
TutteGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1223
TwinplexGraph() (*in module sage.graphs.generators.smallgraphs*), 1643
TwinplexGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1224
two_factor_petersen() (*sage.graphs.graph.Graph method*), 758
twograph() (*sage.graphs.graph.Graph method*), 759

U

U42Graph216() (*in module sage.graphs.generators.smallgraphs*), 1645
U42Graph216() (*sage.graphs.graph_generators.GraphGenerators static method*), 1226
U42Graph540() (*in module sage.graphs.generators.smallgraphs*), 1645
U42Graph540() (*sage.graphs.graph_generators.GraphGenerators static method*), 1226
underlying_graph() (*in module sage.graphs.tutte_polynomial*), 2245
underlying_graph() (*sage.graphs.graph_decompositions.slice_decomposition.SliceDecomposition method*), 2129
UniformRandomUniform() (*sage.graphs.hypergraph_generators.HypergraphGenerators method*), 1822
union() (*sage.graphs.generic_graph.GenericGraph method*), 477

UnitaryDualPolarGraph() (*in module sage.graphs.generators.classical_geometries*), 1436
UnitaryDualPolarGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1227
UnitaryPolarGraph() (*in module sage.graphs.generators.classical_geometries*), 1437
UnitaryPolarGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1229
unlabeled_edges() (*sage.graphs.tutte_polynomial.Ear method*), 2239
unpickle_graph_backend() (*in module sage.graphs.base.graph_backends*), 1800
update_db() (*sage.graphs.isgci.GraphClasses method*), 1677
update_matching() (*sage.graphs.matching_covered_graph.MatchingCoveredGraph method*), 951
USAMap() (*in module sage.graphs.generators.world_map*), 1663
USAMap() (*sage.graphs.graph_generators.GraphGenerators static method*), 1227
UstimenkoGraph() (*in module sage.graphs.generators.distance_regular*), 1454
UstimenkoGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1229

V

vanLintSchrijverGraph() (*in module sage.graphs.generators.distance_regular*), 1465
vanLintSchrijverGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1278
vertex_boundary() (*sage.graphs.generic_graph.GenericGraph method*), 477
vertex_coloring() (*in module sage.graphs.graph_coloring*), 1875
vertex_connectivity() (*in module sage.graphs.connectivity*), 2357
vertex_connectivity() (*sage.graphs.generic_graph.GenericGraph method*), 478
vertex_cover() (*sage.graphs.bipartite_graph.BipartiteGraph method*), 866
vertex_cover() (*sage.graphs.graph.Graph method*), 761
vertex_cut() (*sage.graphs.generic_graph.GenericGraph method*), 482
vertex_disjoint_paths() (*sage.graphs.generic_graph.GenericGraph method*), 483
vertex_isoperimetric_number() (*in module sage.graphs.isoperimetric_inequalities*), 1909
vertex_isoperimetric_number() (*sage.graphs.graph.Graph method*), 762
vertex_iterator() (*sage.graphs.generic_graph.GenericGraph method*), 484
vertex_separation() (*in module sage.graphs.graph_decompositions.vertex_separation*), 2092
vertex_separation_BAB() (*in module sage.graphs.graph_decompositions.vertex_separation*), 2095
vertex_separation_exp() (*in module sage.graphs.graph_decompositions.vertex_separation*), 2100
vertex_separation_MILP() (*in module sage.graphs.graph_decompositions.vertex_separation*), 2098
VertexOrder (*class in sage.graphs.tutte_polynomial*), 2240
vertices (*sage.graphs.tutte_polynomial.Ear property*), 2240
vertices() (*sage.graphs.generic_graph.GenericGraph method*), 485
verts() (*sage.graphs.base.c_graph.CGraph method*), 1705
verts() (*sage.graphs.base.static_sparse_backend.StaticSparseCGraph method*), 1794

W

WagnerGraph() (*in module sage.graphs.generators.smallgraphs*), 1645
WagnerGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1230
WatkinsSnarkGraph() (*in module sage.graphs.generators.smallgraphs*), 1646
WatkinsSnarkGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1230
weighted() (*sage.graphs.generic_graph.GenericGraph method*), 487
weighted_adjacency_matrix() (*sage.graphs.generic_graph.GenericGraph method*), 488
WellsGraph() (*in module sage.graphs.generators.smallgraphs*), 1647
WellsGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1231
WengerGraph() (*in module sage.graphs.generators.luw_graphs*), 1383
WengerGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1232
WheelGraph() (*in module sage.graphs.generators.families*), 1519
WheelGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1232
width_of_cut_decomposition() (*in module sage.graphs.graph_decompositions.cutwidth*), 2114

width_of_path_decomposition() (*in module sage.graphs.graph_decompositions.vertex_separation*), 2100
width_of_tree_decomposition() (*in module sage.graphs.graph_decompositions.tree_decomposition*), 2082
wiener_index() (*in module sage.graphs.base.boost_graph*), 1819
wiener_index() (*in module sage.graphs.distances_all_pairs*), 2187
wiener_index() (*sage.graphs.generic_graph.GenericGraph method*), 491
WienerArayaGraph() (*in module sage.graphs.generators.smallgraphs*), 1647
WienerArayaGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1234
WindmillGraph() (*in module sage.graphs.generators.families*), 1521
WindmillGraph() (*sage.graphs.graph_generators.GraphGenerators static method*), 1235
world_map (*sage.graphs.graph_generators.GraphGenerators attribute*), 1278
WorldMap() (*in module sage.graphs.generators.world_map*), 1664
WorldMap() (*sage.graphs.graph_generators.GraphGenerators static method*), 1237
write_to_eps() (*sage.graphs.graph.Graph method*), 763

X

xslice_data() (*sage.graphs.graph_decompositions.slice_decomposition.SliceDecomposition method*), 2130
xslice_sequence() (*sage.graphs.graph_decompositions.slice_decomposition.SliceDecomposition method*), 2130

Y

yen_k_shortest_simple_paths() (*in module sage.graphs.path_enumeration*), 2271